Low-Level Design Handbook

OOP · UML · SOLID · Clean Code · Refactoring · Design Patterns · Concurrency · 45+ case studies in Java

22 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — The LLD Interview & How to Approach It**
   - 1.1 What this round is, and what this handbook covers
   - 1.2 How the interview runs
   - 1.3 A complete answer in five minutes: "Design a music playlist"
   - 1.4 The step-by-step answer framework
   - 1.5 From one line to a list of requirements
   - 1.6 Finding the classes
   - 1.7 Connecting the classes (preview)
   - 1.8 Keeping the design honest with SOLID
   - 1.9 The design patterns that come up in this round
   - 1.10 Code, edge cases and concurrency: how to write interview code
   - 1.11 Worked mini-example, end to end: a small parking lot
   - 1.12 Communication tips
   - 1.13 How to use this handbook, and a study plan
   - 1.14 Common mistakes
   - In the interview
   - Self-check
   - TL;DR
2. **Chapter 2 — OOP Fundamentals**
   - 2.1 What object-oriented programming is
   - 2.2 Classes, objects, constructors, `this`, `static` and `final`
   - 2.3 Encapsulation
   - 2.4 Abstraction
   - 2.5 Inheritance
   - 2.6 Polymorphism (and the full bank example)
   - 2.7 Compile-time vs runtime polymorphism: overloading vs overriding
   - 2.8 Interfaces (default, static, private methods, functional interfaces)
   - 2.9 Abstract classes vs interfaces
   - 2.10 Enums with fields and behaviour
   - 2.11 Composition over inheritance
   - 2.12 Object identity vs equality: the `equals` / `hashCode` contract
   - 2.13 Immutability and records
   - 2.14 Access modifiers and packages
   - 2.15 Generics basics for LLD
   - 2.16 Coupling and cohesion (the yardsticks)
   - 2.17 Object-oriented analysis and design (OOA / OOD / OOP)
   - Chapter summary: in the interview
   - Self-check (whole chapter)
   - TL;DR
3. **Chapter 3 — Class Relationships & UML**
4. **Part A — Class Relationships**
   - 3.1 The big picture: what a "relationship" really claims
   - 3.2 Association (uni-directional, bi-directional, multiplicity)
   - 3.3 Aggregation
   - 3.4 Composition
   - 3.5 Dependency
   - 3.6 Generalization (inheritance)
   - 3.7 Realization (interface implementation)
   - 3.8 Decision guide: choosing the relationship between two classes
5. **Part B — UML**
   - 3.9 What UML is (structural vs behavioural diagrams)
   - 3.10 Use-case diagrams
   - 3.11 Class diagrams
   - 3.12 Sequence diagrams
   - 3.13 Activity diagrams
   - 3.14 State machine diagrams
6. **Part C — UML in the interview**
   - 3.15 How much UML to draw in an interview
   - 3.16 Chapter self-check
   - 3.17 TL;DR
7. **Chapter 4 — SOLID Principles**
   - 4.1 What SOLID is and why you should learn it
   - 4.2 Single Responsibility Principle (SRP)
   - 4.3 Cohesion
   - 4.4 Coupling
   - 4.5 Separation of Concerns (SoC)
   - 4.6 SRP vs cohesion vs coupling vs SoC — one table
   - 4.7 Open/Closed Principle (OCP)
   - 4.8 Liskov Substitution Principle (LSP)
   - 4.9 Interface Segregation Principle (ISP)
   - 4.10 Dependency Inversion Principle (DIP)
   - 4.11 Refactoring one class through all five principles
   - 4.12 SOLID and design patterns
   - 4.13 SOLID in the design interview
   - 4.14 Common questions about SOLID (with short answers)
   - 4.15 Over-engineering: the warnings in one place
   - Chapter self-check
   - TL;DR
8. **Chapter 5 — More Design Principles**
   - 1. DRY — Don't Repeat Yourself
   - 2. KISS — Keep It Simple, Stupid
   - 3. YAGNI — You Aren't Gonna Need It
   - 4. Law of Demeter (Principle of Least Knowledge)
   - 5. Tell, Don't Ask
   - 6. Composition over Inheritance
   - 7. Program to an Interface, Not an Implementation
   - 8. Encapsulate What Varies
   - 9. Strive for Loosely Coupled Designs
   - 10. The Hollywood Principle (Inversion of Control)
   - 11. Separation of Concerns (SoC)
   - 12. Principle of Least Astonishment (POLA)

- 13. Fail Fast
- 14. Immutability
- 15. Design by Contract (DbC)
- 16. GRASP — General Responsibility Assignment Software Patterns
- 16.1 Information Expert
- 16.2 Creator
- 16.3 Controller
- 16.4 Low Coupling
- 16.5 High Cohesion
- 16.6 Polymorphism
- 16.7 Pure Fabrication
- 16.8 Indirection
- 16.9 Protected Variations
- 17. Domain-Driven Design (DDD) Essentials
- Chapter summary — how the principles fit together

9. **Chapter 6 — Clean Code**
   - What is clean code, and why bother?
   - 1. Meaningful Names
   - 2. Functions
   - 3. Comments
   - 4. Formatting
   - 5. Objects and Data Structures
   - 6. Error Handling
   - 7. Boundaries
   - 8. Unit Tests
   - 9. Classes
   - 10. Systems
   - 11. Emergence — Kent Beck's Four Rules of Simple Design
   - 12. Concurrency
   - 13. Successive Refinement and the Case Studies (Chapters 14–16: how clean code is actually written)
   - 14. Smells and Heuristics (the checklist)
   - 15. Code Review Checklist (best practices)
   - 16. Error-Handling Strategy (best practices for real services)
   - 17. Testing Strategies (unit vs integration, test doubles, mocks)
   - Chapter summary
10. **Chapter 7 — Refactoring**
    - 7.1 Refactoring fundamentals
    - 7.2 Code smells
    - 7.3 The refactoring catalogue
    - 7.4 Smell → refactorings lookup table
    - 7.5 Worked example — from messy to clean in six compiling steps
    - 7.6 Refactoring toward patterns
    - 7.7 Putting it together
    - 7.8 Full source code and output
11. **Chapter 8 — Design Patterns: Introduction & Creational Patterns**
12. **Part A — Design patterns: the big picture**
    - 8.1 What is a design pattern?
    - 8.2 A short history: from buildings to the "Gang of Four"
    - 8.3 The three categories and all 23 GoF patterns
    - 8.4 The OO principles behind every pattern
    - 8.5 How to pick a pattern
    - 8.6 Anti-patterns: when patterns go wrong
13. **Part B — Creational patterns**
    - 8.7 Creational patterns at a glance
    - 8.8 Singleton
    - 8.9 Simple Factory (an idiom, not a GoF pattern)
    - 8.10 Factory Method
    - 8.11 Abstract Factory
    - 8.12 Builder
    - 8.13 Prototype
    - 8.14 Bonus — Object Pool
    - 8.15 Creational patterns compared
    - 8.16 Chapter self-check
    - 8.17 Chapter TL;DR
14. **Chapter 9 — Structural Design Patterns**
    - 9.0 What structural patterns are
    - 9.1 Adapter Pattern
    - 9.2 Bridge Pattern
    - 9.3 Composite Pattern
    - 9.4 Decorator Pattern
    - 9.5 Facade Pattern
    - 9.6 Flyweight Pattern
    - 9.7 Proxy Pattern
    - 9.8 Putting it all together — comparison
15. **Chapter 10 — Behavioral Design Patterns (Part 1)**
    - 10.0 Behavioral patterns: the big picture
    - 10.1 Chain of Responsibility
    - 10.2 Command
    - 10.3 Iterator
    - 10.4 Mediator
    - 10.5 Memento
    - 10.6 Interpreter
    - 10.7 Chapter recap: the six patterns side by side
16. **Chapter 11 — Behavioral Design Patterns (Part 2) & Beyond GoF**
    - 11.1 Observer
    - 11.2 State
    - 11.3 Strategy
    - 11.4 Template Method
    - 11.5 Visitor
    - 11.6 GoF behavioral patterns: quick-reference summary
    - 11.7 Beyond GoF: patterns every LLD interview expects
    - 11.8 Choose the right behavioral pattern
17. **Chapter 12 — Concurrency & Multithreading for LLD (Java)**
18. **Part A — Foundations**
    - 12.1 Concurrency vs parallelism
    - 12.2 Processes vs threads

- 12.3 Thread lifecycle & states
- 12.4 Creating threads: `Thread`, `Runnable`, `Callable`, `ExecutorService`, virtual threads

19. **Part B — What goes wrong**
    - 12.5 Race conditions & critical sections
    - 12.6 Java Memory Model basics: visibility, happens-before, `volatile`
20. **Part C — Locks**
    - 12.7 Mutex (mutual exclusion) and `synchronized`
    - 12.8 `ReentrantLock` and reentrancy
    - 12.9 Try-lock & timed locking
    - 12.10 Read-write locks: `ReadWriteLock` and `StampedLock`
    - 12.11 Coarse-grained vs fine-grained locking
21. **Part D — Coordination**
    - 12.12 Semaphores
    - 12.13 Condition variables: `wait/notify` and `Condition`
    - 12.14 `CountDownLatch` and `CyclicBarrier`
22. **Part E — Lock-free**
    - 12.15 Compare-and-swap (CAS) & atomics
    - 12.16 Concurrent collections
23. **Part F — Liveness failures**
    - 12.17 Deadlock
    - 12.18 Livelock
    - 12.19 Starvation (and fairness)
24. **Part G — Concurrency patterns**
    - 12.20 Signaling pattern
    - 12.21 Thread pool pattern
    - 12.22 Producer-consumer pattern
    - 12.23 Reader-writer pattern
    - 12.24 Immutability & thread confinement
25. **Part H — Concurrency in LLD interviews**
    - 12.25 How concurrency shows up in LLD interviews (parking, seat booking, inventory)
    - 12.26 Choosing the right tool (cheat sheet)
    - 12.27 Chapter self-check
    - 12.28 Chapter TL;DR
26. **Chapter 13 — Classic Concurrency Problems**
    - 13.1 Print FooBar Alternately (LeetCode 1115)
    - 13.2 Print Zero Even Odd (LeetCode 1116)
    - 13.3 Fizz Buzz Multithreaded (LeetCode 1195)
    - 13.4 Building H2O (LeetCode 1117)
    - 13.5 Dining Philosophers (LeetCode 1226)
    - 13.6 Thread-safe cache with TTL
    - 13.7 Design a (simplified) Concurrent HashMap — lock striping
    - 13.8 Thread-safe bounded Blocking Queue (two ways)
    - 13.9 Concurrent Bloom Filter
    - 13.10 Multi-threaded Merge Sort (Fork/Join)
    - 13.11 Rate limiter — token bucket (thread-safe)
    - 13.12 Chapter summary
    - 13.13 Chapter self-check
    - 13.14 Chapter TL;DR
27. **Chapter 14 — Case Studies: Everyday Objects**
    - 14.1 Design a Parking Lot
    - 14.2 Design a Vending Machine
    - 14.3 Design a Coffee Vending Machine
    - 14.4 Design an ATM
    - 14.5 Design an Elevator System
    - 14.6 Design a Traffic Signal Control System
    - 14.7 Design a Shipping Locker System
    - Chapter summary
28. **Chapter 15 — Case Studies: Booking & Inventory (Part 1)**
    - Why booking problems are interview favourites
    - 15.1 Design a Library Management System
    - 15.2 Design a Movie Ticket Booking System (BookMyShow)
    - 15.3 Design a Concert Ticket Booking System
    - 15.4 Design a Hotel Management System
    - Chapter summary
29. **Chapter 16 — Case Studies: Booking & Inventory (Part 2)**
    - Design a Car Rental System
    - Design a Restaurant Management System
    - Design an Airline Management System
    - Design a Course Registration System
    - Chapter summary
30. **Chapter 17 — Case Studies: Games & Rules**
    - Design Blackjack and a Deck of Cards
    - Design Chess
    - Design Tic-Tac-Toe
    - Design Snake and Ladder
    - Design 2048
    - Chapter summary
31. **Chapter 18 — Case Studies: Platforms & Feeds (Part 1)**
    - 18.0 What platform problems test
    - 18.1 Design Stack Overflow
    - 18.2 Design Cricinfo
    - 18.3 Design LinkedIn
    - 18.4 Design Facebook
    - 18.5 Chapter summary
32. **Chapter 19 — Case Studies: Commerce & Marketplaces (Part 2 of Platforms)**

- Design Amazon — Online Shopping System
- Design an Online Stock Brokerage System
- Design an Online Auction System
- Design a Music Streaming Service (Spotify)
- Design an Online Food Delivery Service (Swiggy) + FoodKart
- Design a Grocery Store (POS & Checkout)
- Chapter summary

33. **Chapter 20 — Case Studies: Pattern-Driven Components**
    - 20.1 Design Splitwise
    - 20.2 Design a Notification Service
    - 20.3 Design an In-Memory Cache (and an LRU Cache from scratch)
    - 20.4 Design a Ride Sharing Service (Uber)
    - 20.5 Design a Logging Framework
    - 20.6 Design a Pub-Sub System
    - 20.7 Design a Rate Limiter
    - 20.8 Chapter summary
    - 20.9 Chapter self-check
    - 20.10 Chapter TL;DR
34. **Chapter 21 — Case Studies: Machine-Coding Round Problems**
    - 21.0 How the machine-coding round works
    - 21.1 Design a Task Management System
    - 21.2 Design a Digital Wallet Service
    - 21.3 Design an Event Calendar (Flipkart)
    - 21.4 Design a LeetCode-like Coding Platform — "Coding Blox" (Flipkart)
    - 21.5 Design a Property Listing Site — "PropertyHunt" (ClearTrip)
    - 21.6 Design a Loan Ledger — "Ledger Co" (Navi)
    - 21.7 Design an Inventory Management System
    - 21.8 Design a Unix File Search (`find`)
    - Chapter summary
35. **Chapter 22 — Revision: Cheat Sheets & Glossary**
    - 22.1 How to revise
    - 22.2 The LLD interview answer framework (one page)
    - 22.3 OOP & relationships cheat sheet
    - 22.4 SOLID one-pager
    - 22.5 Other design principles and Clean Code rules
    - 22.6 Code smells → refactorings (all 24)
    - 22.7 Design patterns: all 23 GoF + extras
    - 22.8 Concurrency cheat sheet
    - 22.9 Case-study map
    - 22.10 Machine-coding and edge-case checklists
    - 22.11 Glossary
    - Self-check
    - TL;DR

Chapter 1 — The LLD Interview & How to Approach It

What you will learn in this chapter

- What the low-level design (LLD) round is, what it is called at different companies, and how it differs from the system design round.
- The difference between the **object-oriented design (OOD) round** and the **machine-coding round**.
- A complete answer, start to finish, before any theory (a music playlist in Java).
- A repeatable, step-by-step framework that turns one sentence into working code, with a time budget.
- How to turn one line into requirements, how to find classes, how to check a design with SOLID (five rules of thumb that keep classes easy to change), and which six patterns come up most.
- What interviewers actually score, how to talk while you design, and the mistakes that cost the most.
- A study plan and a map of this handbook (Chapters 1–22).
- A second worked mini-example (a small parking lot) run through every step of the framework.

1.1 What this round is, and what this handbook covers

What it is

The **low-level design** round gives you **a single application** to design while an interviewer watches, usually within 45–60 minutes (a machine-coding round stretches this to 90 minutes or 2 hours). The servers underneath are not your concern. What you design is **the set of classes inside the program**: a parking lot, a vending machine, a chess game, Splitwise (an app that splits shared expenses between friends), an elevator.

A few words we will use all the time:

Term	Meaning
Class	A named type in code. It holds some data (fields) and the methods (functions that belong to the class) that work on that data.
Object	One concrete instance of a class, with real values in its fields.
Responsibility	The one job a class does, which you should be able to state in a single sentence.
Relationship	How two classes are connected: one uses, holds, owns, or extends the other.
UML	Unified Modeling Language: the standard set of symbols for drawing classes and their relationships.
Design pattern	A named, reusable answer to a design problem that keeps coming up (for example Strategy or Observer).

Everyday analogy: system design is the **city plan** (roads, power stations, water pipes). Low-level design is the **floor plan of one house**: which rooms exist, what each room is for, and which doors connect them.

Plenty of engineers write classes daily and still struggle here, because day-to-day work seldom makes you **explain aloud** why each class exists and **justify** the way they connect. This round is exactly that.

What you should be able to do after this handbook

- Turn one sentence ("Design a parking lot") into a bounded list of requirements, with an explicit out-of-scope list.
- Pick out the classes a problem calls for, with **exactly one job** apiece.
- Decide how any pair of classes is related, and draw that link without mistakes.
- Code the handful of Java methods behind the core flows until they actually execute.
- Say which design pattern you used **and which one you rejected**, and why.

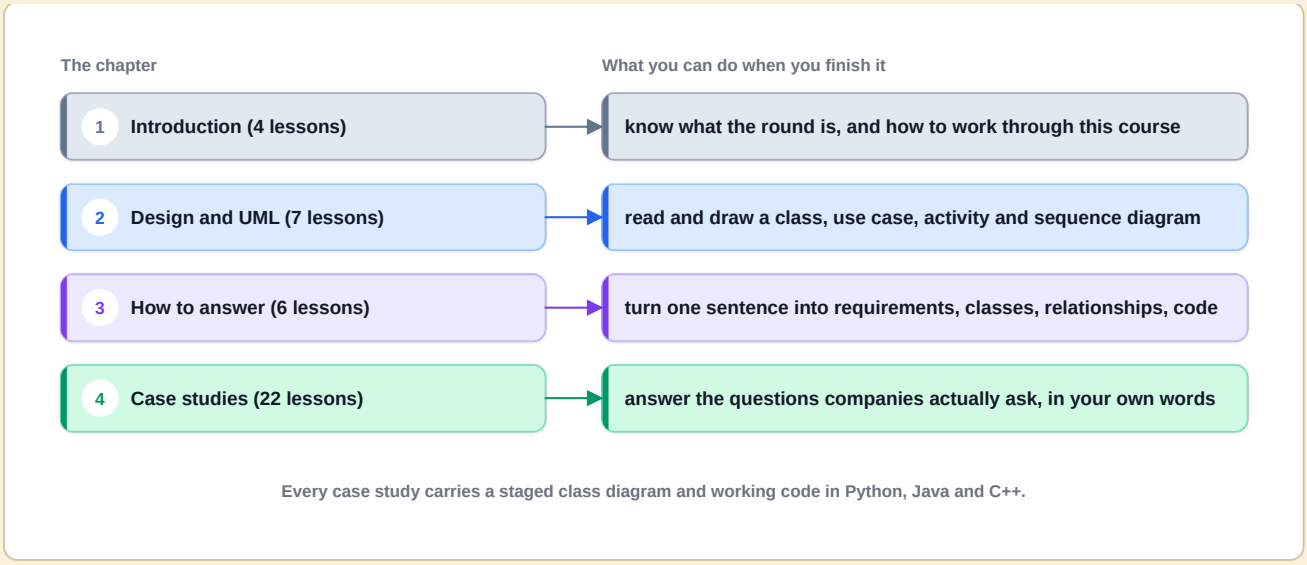


Figure — The four parts of the course material this handbook absorbs: introduction, design and UML notation, the answering method, and the case studies (course diagram)

The figure above shows the four building blocks of preparation: first understand the round (this chapter), then the notation (UML), then the **method** for answering, then many worked case studies. This handbook follows the same arc but goes wider: it adds full chapters on OOP, SOLID, the classic design principles, clean code, refactoring (improving the structure of code without changing what it does), all 23 GoF patterns (the classic catalogue from the 1994 "Gang of Four" book), and concurrency (several pieces of work running at the same time and sharing data), so that you do not need a second resource.

What every case study in this handbook contains

Once you have read one case study you always know what comes next, because each one follows the same template:

- The question, and the requirements chosen to design against (with an out-of-scope list).
- The actors and their use cases (often as a use case diagram, a picture of who uses the system and for what).
- The class diagram, built up in stages so you follow the reasoning instead of staring at a finished picture.
- Diagrams for the most important flows (activity diagrams show the steps of a process; sequence diagrams show the messages objects send each other, in time order).
- Working Java code with a driver `Main` (a small class whose `main` method runs the example) and its real output.
- The design patterns the answer uses, named, **only where it really uses one**.
- An "Extensions & follow-up questions" section: the simplifications made, and what to say when the interviewer notices them.

Who this is for

- You have an interview coming up and this round is part of it.
- You write object-oriented code at work but have never had to defend a class diagram out loud.
- You prepared for system design and discovered this is a different round with different rules.

It also pays off outside interviews: the same step-by-step method is how you split a new feature into classes at work when nobody has designed it yet.

What you need already

- **Some object-oriented language**, well enough that a class with a constructor (the special method that sets up a new object) plus a few methods reads easily to you. Every example here is in **Java 21** (a recent long-term-support release of Java).
- You do **not** need UML (Chapter 3 teaches it from zero).
- You do **not** need system design knowledge. That round asks how a product survives millions of users (servers, databases, caches, queues; see System Design Fundamentals if those words are new). This round stays **inside one process** (one running program: classes, fields, methods, relationships). Practising one round gives you little for the other.
- You do not need data structures and algorithms beyond the basics (lists, maps, sets, queues, heaps). That is a separate round. (Quick refresher: a **list** keeps items in order, a **map** looks a value up by its key, a **set**

holds no duplicates, a **queue** hands items out first-in, first-out, and a **heap** always gives you the smallest or largest item quickly.)

What this handbook will not do

It will not teach you a programming language from scratch (it assumes you can already read one). It will not give you scripts to memorise. A memorised answer is useless the moment the question is one you have not seen. **The reusable part is the method.**

TL;DR

The LLD round = design the classes inside one application, out loud, then code the main flows. Prerequisite: one OO language. The method beats memorised answers.

1.2 How the interview runs

What it is

The interviewer says one sentence, for example "Design a parking lot", and then stops talking. Nothing in that sentence tells you how many classes they want, whether to code, or how long you have. **That silence is the hard part.** This section removes it.

The round has several names, all meaning roughly the same thing:

Name	Where you hear it	What is different
Object-oriented design (OOD) interview	US big tech, many product companies	Whiteboard/doc: requirements, classes, diagram, code for main flows.
Low-level design (LLD)	Indian product companies, general usage	Same as OOD; the umbrella term.
Machine-coding round	Commonly reported at Indian product companies such as Flipkart, Swiggy and PhonePe	Same idea, but **working, runnable code is mandatory** at the end, usually in a 90-minute to 2-hour window.

LLD vs HLD (system design). Same product, different zoom level:

	Low-level design (this handbook)	High-level / system design
Scope	Inside one application or service	Many services and machines
Vocabulary	Classes, interfaces, fields, methods, relationships, patterns	Servers, load balancers, databases, caches, queues, sharding
Output	Class diagram + code	Architecture diagram + capacity numbers
Typical question	"Design the classes for a parking lot"	"Design a parking-lot booking service for 10 million users"

(For the right-hand column: a **load balancer** spreads incoming requests across several servers, and **sharding** splits one big database into pieces stored on different machines. You will not need either in this round.)

OOD round vs machine-coding round

	OOD / LLD discussion round	Machine-coding round
Duration	45–60 minutes	Usually about 90 minutes of coding (often 2 hours including evaluation)
Where	Whiteboard, shared doc, or IDE (code editor with build and run tools)	Your own IDE, on your laptop or a remote machine
What you produce	Requirements, use cases, class diagram, code for 2–3 main flows	A **complete, running program** implementing the minimum required features
Storage	Not discussed much	**In-memory** (maps and lists); no real database unless asked
Interaction	Continuous conversation with the interviewer	Problem statement up front, mostly silent coding, then a **demo and code walkthrough**
What is judged	Bounding the problem, responsibilities, relationships, trade-offs, patterns	Working code, **data models**, **service interfaces** a client could call, **extensibility** (can a new feature be added without rewriting), readability, edge cases, sometimes tests and thread safety
Common failure	Talking without converging, or coding too early	Not finishing, or finishing with one giant class that cannot be extended

The machine-coding round, as commonly described by candidates, works like this: you get a problem statement with a **minimum set of features**; within the time window you must choose the data structures, design **service interfaces** that a client (for example a command-line driver, a small program you run from a terminal) would use, and present **working code** together with the reasoning behind your design and the patterns you used. It is especially common for SDE-1 and SDE-2 hiring (Software Development Engineer job levels: entry level and mid level).

A few words from the table above: **data models** are the classes that hold the problem's data (a `Ticket`, a `Spot`); a **service interface** is the set of public methods a caller uses (`park`, `exit`); **edge cases** are unusual inputs at the boundaries (empty, full, negative, duplicate); **thread safety** means the code stays correct when several **threads** (independent lines of execution inside one program) use it at the same moment.

How it works: the five core steps

Every question in this handbook is answered with the same five core steps (Section 1.4 expands them into a finer checklist):

1. **Clarify the requirements** — turn one sentence into a short written list of things the system must do.
2. **Find the actors and use cases** — who uses the system (actors) and what each of them does (use cases).
3. **Find the classes** — turn requirements into classes; give each one responsibility.
4. **Connect the classes** — decide how each pair relates; record it in a class diagram.
5. **Write the code** — the classes plus the 2–3 methods that implement the main flows.

Getting the order right matters more than going fast. Each step feeds the next. Skip step 1 and step 3 has nothing to work from: you end up inventing classes no requirement asked for.

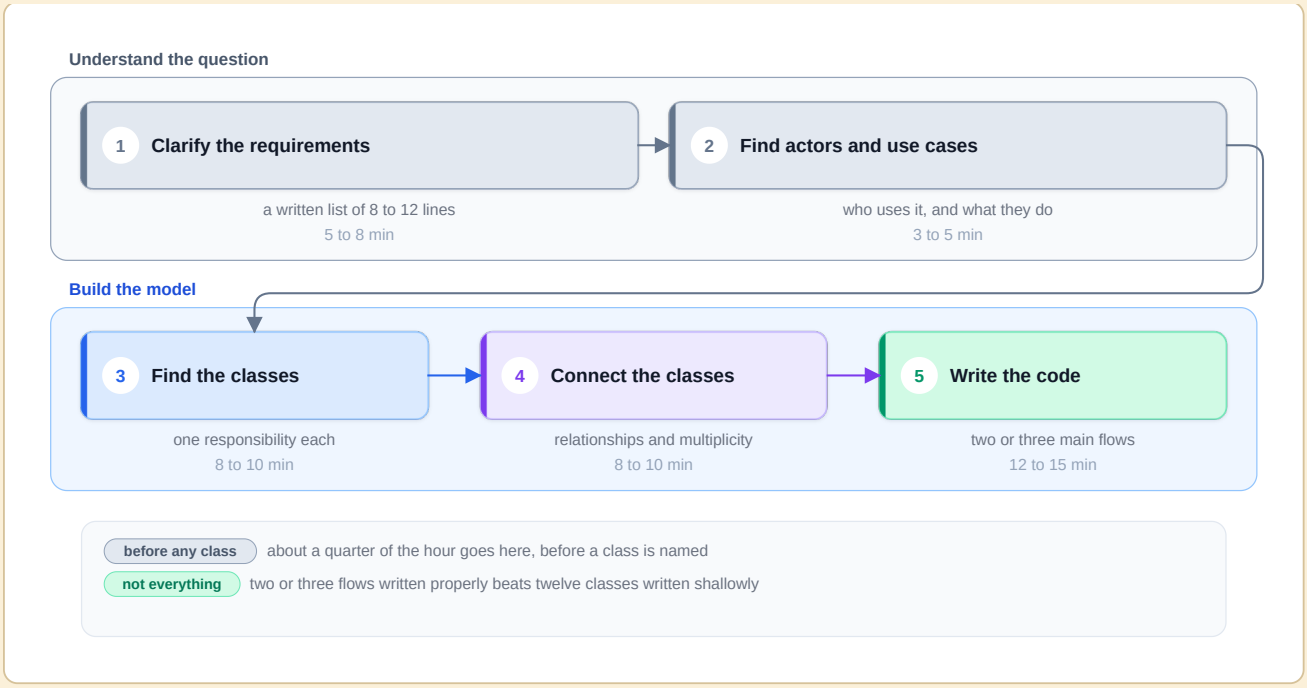


Figure — The five steps split into "understand the question" and "build the model", with a time budget for each (course diagram)

In the figure above, notice two things. First, up to **a quarter of the hour is spent before a single class is named** (requirements plus actors). Second, the code step is the largest single block, yet it covers only two or three flows.

Nobody completes the whole product in the time, and you are not expected to. The design is judged on the part you chose to build properly, not on how much of the product you touched.

Time-boxing

45-minute OOD round:

Step	Time	What exists at the end
Clarify requirements	5–8 min	A written list of 8–12 requirements + out-of-scope list
Actors and use cases	3–5 min	2–4 actors, 6–10 use cases
Find the classes	8–10 min	Class list, one responsibility each
Connect the classes	8–10 min	Class diagram with relationships and multiplicity (how many of each)
Write the code	12–15 min	2–3 methods that implement the main flows

90-minute machine-coding round (a budget that works well in practice):

Phase	Time	Output
Read the statement, list requirements and assumptions	5–10 min	Comment block at the top of `Main.java`
Model: entities, enums, service interfaces	10–15 min	Empty classes/interfaces that compile
Implement the core flow end to end	30–40 min	One happy path running from a driver `main`
Remaining features + validation + edge cases	15–20 min	All "must have" features working
Polish: names, exceptions, one extension point, demo script	5–10 min	Clean output you can demo
Buffer	5 min	Never zero

In this table, code that **compiles** is code the Java compiler accepts, so it can run; **enums** are types with a fixed list of named values (more in Section 1.6); the **happy path** is the main flow with valid input and nothing going wrong.

Rule of thumb: **get something running by the halfway mark**. A running partial solution beats a beautiful non-compiling one.

What the interviewer is actually judging

The interviewer is not counting classes. They watch for:

1. **Did you bound the problem?** A design for a clear, bounded set of requirements beats a broad design for an unbounded one.
2. **Does each class have one job you can state in one sentence?** If one sentence cannot describe a class, it is normally two classes glued together.
3. **Are the relationships right?** Holding, owning, and extending are different choices; a wrong one is visible immediately.
4. **Does the code match the diagram?** If the code has a method that the diagram has nowhere to put, either the code or the diagram is wrong.
5. **Can you explain a trade-off?** Each design decision has a road taken and a road not taken; naming both scores higher than naming only one.

Machine-coding rounds add: **does it run**, is it **modular** (can a new vehicle type / payment method / rule be added by adding a class), is it **readable** (names, small methods), does it **handle bad input** gracefully, and (for senior roles) is shared state **thread-safe**.

Notice what is **not** on the list: perfect UML notation, or using every diagram type. **What is graded is the model; the notation is just the way you present it.**

A worked example of size

Take the parking lot. About 12 requirements produce about 12 classes with about 15 relationships between them. Three classes implement the main flows: one hands out a ticket, one holds a vehicle, one prices the stay. You write those three properly. You name the other nine and describe each in a sentence. That is a complete answer in 45 minutes.

The most expensive mistake is writing all twelve classes at the same level of detail. It feels thorough, but the clock runs out before any single flow works end to end.

> **TL;DR**
>
> OOD, LLD and machine coding are one family of rounds; machine coding requires running code in ~90 minutes. Follow five steps in order, spend a quarter of the time before naming a class, and build two or three flows properly rather than twelve classes shallowly.

1.3 A complete answer in five minutes: "Design a music playlist"

Let's see a complete answer first and save the theory for later. It is small on purpose: a screenful of classes, a single diagram, a page of Java. Later case studies are roughly ten times the size, yet they follow **the identical outline**. Don't worry if some diagram notation is unfamiliar; Chapter 3 covers it.

Step 1: requirements

The interviewer says only: "Design a music playlist." Three clarifying questions produce three requirements:

- Songs sit in a playlist, and **their sequence is significant**.
- Someone can ask for **the following song**, and the player keeps track of its current position.
- A user can turn **shuffle** on, and the play order changes.

Out of scope, agreed out loud: user accounts, downloads, storage, and the audio itself. An explicit "not building" list earns as much credit as the "building" list.

Step 2: actors and use cases

Anyone or anything outside the application that interacts with it is an **actor**. This example has just one, the **listener**, who does three things: adds a song, plays the next one, switches shuffle on. For so small a question this takes ten seconds; for a parking lot it costs three minutes and repays ten, since **the actors reveal which classes have to exist**.

Step 3: classes, one sentence each

Class	Responsibility (one sentence, no "and" where possible)
`Song`	Holds a title, an artist and a length; knows about itself and nothing else.
`Playlist`	Holds songs in order; stores them, does not play them.
`PlayOrder` (interface)	Picks the position to play after the current one. It is an interface since several ordering rules exist.
`InOrder`	Walks through positions in insertion order.
`Shuffled`	Plays positions in a shuffled order decided once, up front.
`Player`	Holds a playlist and an order, and remembers the current position.

An **interface** names a set of methods without saying how they work. The key insight: **a rule that can vary becomes its own type.** Without shuffle, `Player` could just count upwards and `PlayOrder` would not exist. With shuffle, "what plays next" has two versions, so it becomes `PlayOrder`, and the player no longer needs to know which version it holds.

A useful test: if a responsibility sentence needs the word **"and"** to join two unrelated jobs, the class is probably two classes.

Step 4: connect them

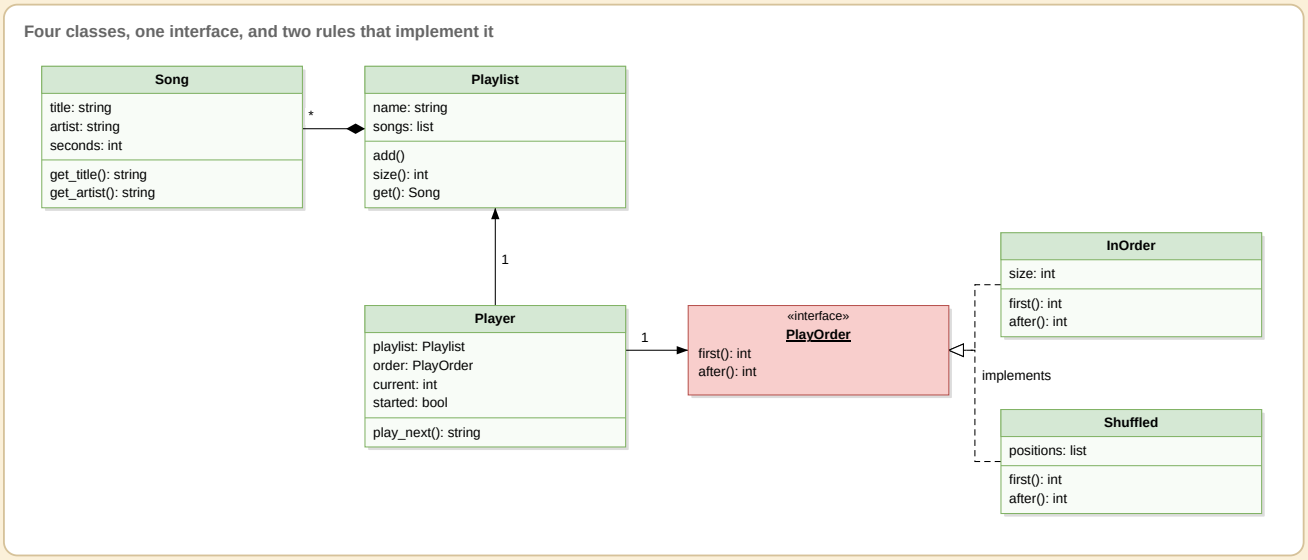
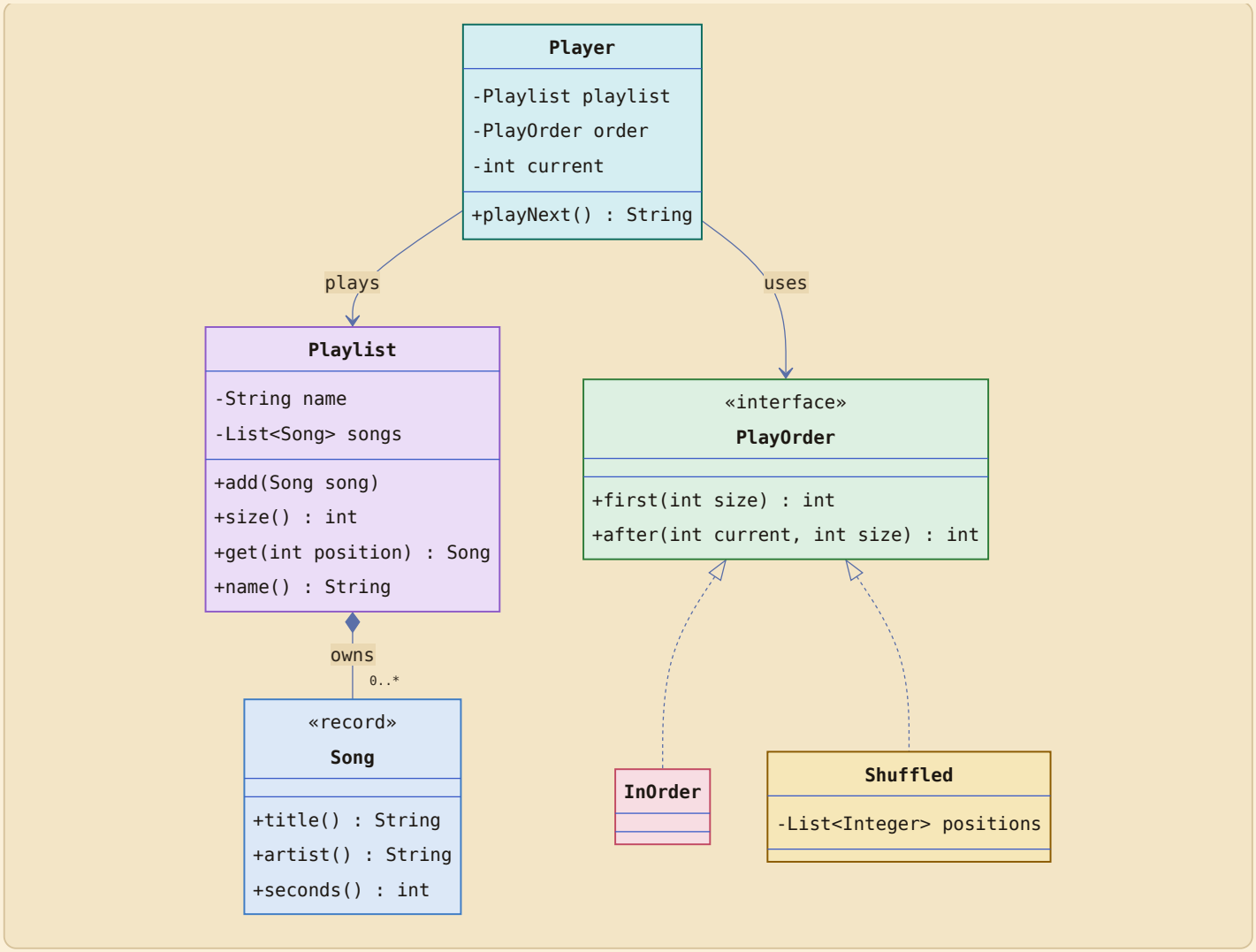


Figure — The playlist design: Playlist owns many Songs, Player uses a PlayOrder, and InOrder and Shuffled implement PlayOrder (course diagram)

Three facts from the figure above are exactly what an interviewer checks:

- The **filled diamond** between `Playlist` and `Song` means the playlist **owns** its songs (composition); the `*` means it owns **many**.
- A **plain arrow** running from `Player` to `PlayOrder` says the player **relies on** an ordering without owning one. Its head touches the **interface** only, not `InOrder` or `Shuffled`.
- The **dashed arrows with hollow triangles** mean `InOrder` and `Shuffled` **implement** `PlayOrder`. A third rule later is one extra box; nothing already drawn changes.

Here is the same design for the Java code below:



Step 5: the code

> **Key term — reading the Java below:** A `package` line groups related classes, much like a folder. `implements PlayOrder` promises that the class provides every method of that interface, and `@Override` marks a method that fulfils such a promise (the compiler checks it). `final` on a class means no other class may extend it; on a field it means the field is set once and never reassigned. `private` hides a field or method from code outside the class.

```java
// File: code/lld/ch01/playlist/InOrder.java
package lld.ch01.playlist;

/** Plays positions in the order the songs were added. */
public final class InOrder implements PlayOrder {
    @Override public int first(int size) { return size == 0 ? NONE : 0; }
    @Override public int after(int current, int size) { return current + 1 < size ? current + 1 : NONE; }
}
```

```java
// File: code/lld/ch01/playlist/PlayOrder.java
package lld.ch01.playlist;

/** The rule that decides which position plays next. A rule that varies becomes its own type. */
public interface PlayOrder {
    int NONE = -1;                          // "there is no next position"

    int first(int size);                    // first position to play, or NONE if empty
    int after(int current, int size);       // position after 'current', or NONE at the end
}
```

```java
// File: code/lld/ch01/playlist/Player.java
package lld.ch01.playlist;

/** Holds a playlist and an order, and remembers where it is. It never asks WHICH order it holds. */
public final class Player {
    private final Playlist playlist;
    private final PlayOrder order;          // depends on the interface, not on InOrder/Shuffled
    private int current = PlayOrder.NONE;
    private boolean started = false;

    public Player(Playlist playlist, PlayOrder order) {
        this.playlist = playlist;
        this.order = order;
    }

    public String playNext() {
        if (!started) {
            current = order.first(playlist.size());
            started = true;
        } else if (current != PlayOrder.NONE) {          // once at the end, stay at the end
            current = order.after(current, playlist.size());
        }
        if (current == PlayOrder.NONE) return "end of playlist";
        Song s = playlist.get(current);
        return "playing " + s.title() + " by " + s.artist();
    }
}
```

```java
// File: code/lld/ch01/playlist/Playlist.java
package lld.ch01.playlist;

import java.util.ArrayList;
import java.util.List;

/** Holds songs in order. It stores them; it does not play them. */
public final class Playlist {
    private final String name;
    private final List<Song> songs = new ArrayList<>();   // composition: the playlist owns its list

    public Playlist(String name) { this.name = name; }

    public void add(Song song) { songs.add(song); }
    public int size()          { return songs.size(); }
    public Song get(int position) { return songs.get(position); }
    public String name()       { return name; }
}
```

```java
// File: code/lld/ch01/playlist/Shuffled.java
package lld.ch01.playlist;

import java.util.List;

/**
 * Plays positions in an order decided once, up front. The order is injected
 * (not generated inside) so the program prints the same output every run: deterministic = testable.
 */
public final class Shuffled implements PlayOrder {
    private final List<Integer> positions;

    public Shuffled(List<Integer> positions) { this.positions = List.copyOf(positions); }

    @Override public int first(int size) { return positions.isEmpty() ? NONE : positions.get(0); }

    @Override public int after(int current, int size) {
        int i = positions.indexOf(current);
        return (i >= 0 && i + 1 < positions.size()) ? positions.get(i + 1) : NONE;
    }
}
```

```java
// File: code/lld/ch01/playlist/Song.java
package lld.ch01.playlist;

/** A song knows about itself and nothing else. A record = immutable data carrier. */
public record Song(String title, String artist, int seconds) {
    public Song {
        if (title == null || title.isBlank()) throw new IllegalArgumentException("title required");
        if (seconds <= 0) throw new IllegalArgumentException("length must be positive");
    }
}
```

```java
// File: code/lld/ch01/playlist/Main.java
package lld.ch01.playlist;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        Playlist mix = new Playlist("Morning mix");
        mix.add(new Song("Sunrise", "Ada Lane", 201));
        mix.add(new Song("Blue Line", "The Meridians", 187));
        mix.add(new Song("Paper Boats", "Nori Kim", 243));

        run("in order", new Player(mix, new InOrder()), mix.size());
        run("shuffled", new Player(mix, new Shuffled(List.of(2, 0, 1))), mix.size());
    }

    private static void run(String label, Player player, int songs) {
        System.out.println(label + ":");
        for (int i = 0; i <= songs; i++) {          // one extra call shows the end
            System.out.println("  " + player.playNext());
        }
    }
}
```

Run it: `cd chapters/01-lld-interview-approach/code && javac --release 21 -d /tmp/lld_01 $(find . -name '*.java') && java -cp /tmp/lld_01 lld.ch01.playlist.Main`

(`javac` is the Java compiler, which turns `.java` source files into runnable `.class` files; `java` then runs them, and `-cp` (classpath) tells it where those files are.)

Output:

```
in order:
  playing Sunrise by Ada Lane
  playing Blue Line by The Meridians
  playing Paper Boats by Nori Kim
  end of playlist
shuffled:
  playing Paper Boats by Nori Kim
  playing Sunrise by Ada Lane
  playing Blue Line by The Meridians
  end of playlist
```

Explaining each part:

- `Song` is a Java **record**: an immutable (cannot change after creation) data carrier whose fields, constructor, accessors (getter methods such as `title()`), `equals`, `hashCode` and `toString` (the standard methods for comparing objects, using them as map keys, and printing them) are generated. The compact constructor (a constructor with no parameter list, used just to check the values) validates input.
- `Playlist` keeps its list `private` (encapsulation: hiding a class's data so it can only be reached through the class's own methods). Callers can add and read, but cannot replace the list.
- `PlayOrder` is the variation point (the place where different behaviours can be plugged in). `NONE = -1` is an interface constant meaning "nothing more to play".
- `Shuffled` is **given** its order instead of generating one with `Random`. This makes the output identical on every run, which makes the design **testable**. (In production you would inject a `Random` with a seed, or a shuffled list produced elsewhere. To **inject** something is to hand it in from outside, usually through the constructor; a **seed** is a fixed starting number that makes `Random` produce the same sequence every time.)
- `Player` depends on the interface `PlayOrder`, not on a concrete class (a real class you can create objects from, as opposed to an interface). One `Player` gives both behaviours, and **nothing in it tests which mode is active**. (Its only `if`s track *where* it is: the first call asks `first`, later calls ask `after`, and once it has reached the end it stays there instead of asking the order again.)

What to notice

- **One requirement turned into a type.** Shuffle occupied a single line of the requirements, yet it produced an interface plus two implementations. That is the usual flow: **classes come out of requirements**, and it never runs backwards.
- **Each class has exactly one job.** A `Song` has no idea any playlist contains it. A `Playlist` is unaware that playback happens. A `Player` never learns how the next position gets picked.
- **The design is open to changes you did not build.** "Repeat one song" and "play in reverse" are just new classes implementing `PlayOrder`; nothing already written is reopened. (This is the Strategy pattern and the Open/Closed Principle, Chapters 4 and 11.)
- **Things were left out on purpose:** pause, volume, queue, time. In an interview you **announce** these gaps yourself instead of waiting to be caught. Every case study in this handbook ends with an "Extensions & follow-up questions" section for the same reason.

TL;DR

A complete answer = a short bounded requirement list, a few one-sentence classes, one diagram, and code for the main flow. A rule that varies becomes an interface, and the class that uses it never asks which version it holds.

1.4 The step-by-step answer framework

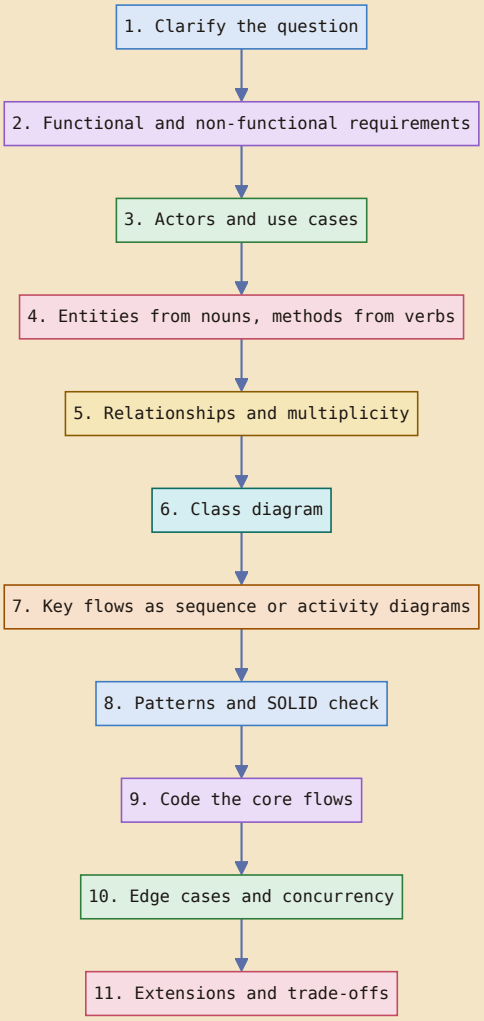
What it is

A checklist you run on **every** LLD question, so the question stops being "where do I start?" and becomes a series of small questions with known answers. It expands the five core steps into eleven finer ones.

The problem it solves

Without a framework, candidates either freeze, or jump straight into code and discover missing requirements at minute 35. The framework makes your progress **visible** to the interviewer, which is itself part of the score.

How it works



#	Step	Questions to ask yourself	Output	Core step
1	**Clarify**	Who uses it? What exactly is "a parking lot" here? Single location or many?	Shared understanding	Requirements
2	**Requirements**	What must it do (functional)? How well (non-functional: thread-safe, extensible, in-memory, latency, i.e. response time)? What is out of scope?	8–12 numbered lines + out-of-scope list	Requirements
3	**Use cases**	For each actor, what can they do?	Actor → use case list (or use case diagram)	Actors & use cases
4	**Entities**	Which nouns hold state or behaviour? Which are just values (fields/enums)?	Class list with one-sentence responsibilities	Classes
5	**Relationships**	Uses, has, owns, or is-a? How many (1, 0..1, *)?	Relationship list	Connect
6	**Class diagram**	Fields, key methods, interfaces, enums	UML class diagram	Connect
7	**Key flows**	What happens step by step for the 2–3 main use cases? What states does the core object go through?	Sequence / activity / state diagram (states and the moves between them)	Connect
8	**Patterns & SOLID**	Is there a rule that varies (Strategy)? A lifecycle (fixed states an object moves through: State)? Many listeners (Observer)? Any class with two reasons to change?	Named patterns + rejected alternatives	Check
9	**Code**	Interfaces, enums, models, services, a driver `main`	Running code for main flows	Code
10	**Edge cases & concurrency**	Invalid input? Full capacity? Two users grabbing the same seat?	Validation, exceptions, locks	Code
11	**Extensions**	What would change if requirement X is added?	Short answers, maybe one extra class	Wrap-up

(Row 10 in plain words: an **exception** is Java's way of signalling an error that stops the normal flow, and a **lock** lets only one thread at a time run a protected piece of code.)

Trade-offs

Do this	Instead of this	Why
Spend 8–13 minutes on steps 1–3	Coding at minute 2	Unbounded scope wastes far more time later
Code 2–3 flows fully	Stub 12 classes (empty placeholders)	Interviewers judge depth on the chosen part
Name one or two patterns where the signal appears	Apply six patterns	Over-engineering (building more flexibility than the problem needs) is scored as poor judgement
Pass time into methods (`nowMinute`) and hold money as `BigDecimal` (Java's exact decimal type; or whole cents in a `long`)	Call `System.currentTimeMillis()` inside the logic, and use `double` (binary floating-point number) for money	Deterministic (same input, same output) and testable; `double` cannot represent values like 0.10 exactly, so sums drift

When not to use it (rigidly)

If the interviewer steers ("skip requirements, I care about the class model"), follow them. The framework is a default order, not a contract. In a machine-coding round, steps 5–7 are often done in your head or as a short comment; the code **is** the diagram.

TL;DR

Clarify → requirements → use cases → entities → relationships → class diagram → key flows → patterns/SOLID → code → edge cases/concurrency → extensions. Say each step out loud as you enter it.

1.5 From one line to a list of requirements

What it is

Each **requirement** states a single capability, phrased so you can test whether the design delivers it. "Customers can pay at the exit" qualifies. "The parking lot should be good" does not.

- **Functional requirement** — a behaviour: "a customer takes a ticket at an entry panel".
- **Non-functional requirement** — a quality: "two entry panels must never assign the same spot" (thread safety), "adding a new vehicle type must not change existing classes" (extensibility), "data kept in memory" (storage).

Analogy: an architect never starts drawing before asking how many people will live in the house and whether they need a garage.

The problem it solves

This is not an interview trick: real products start the same way, with a short request someone has to turn into a specification. "Design a parking lot" is four words. The expected design has about twelve classes. Everything in between is yours to decide. Without a written list:

- **The design has no edges.** You keep adding classes (season tickets, number-plate recognition) and write no code.
- **You cannot defend a choice.** Asked "why is there no `Reservation` class?", the only honest answer is "I did not think of it". With a list, the answer is "reservations were out of scope; we agreed that at minute four".
- **You cannot tell when you are finished.** A bounded list turns an open request into a task with an end.

How it works: the four questions

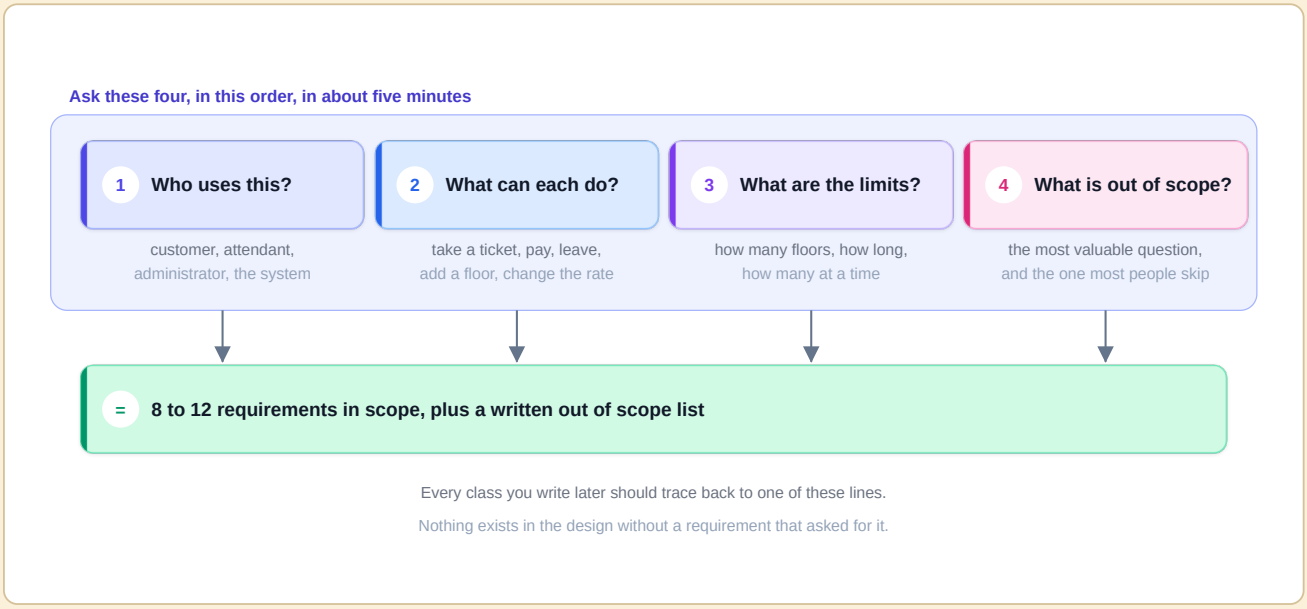


Figure — Four questions asked in order produce 8–12 in-scope requirements plus a written out-of-scope list (course diagram)

As the figure above shows, ask these in order; they take about five minutes and cover most products:

1. **Who uses this?** These are the **actors**. Parking lot: customer, attendant, administrator, and the system itself. Each brings requirements the others do not.
2. **What can each of them do?** One actor at a time, list actions. Each action is a **use case**. A customer takes a ticket, parks, pays, leaves. An admin adds floors and changes prices. Most of the list comes from here.
3. **What are the limits?** The numbers and rules: floor count, kinds of spot, maximum booking length, how many books one member can borrow. **Limits turn into constants and validation checks.** They are also the simplest requirements to gather.
4. **What is out of scope?** Hardly anyone asks this, yet it pays off most. It is the only one of the four questions that makes the problem **smaller**.

Scope questions worth asking in almost any problem:

- "Do payments belong to my design, or can I treat an external payment service as given?"
- "Is there one site, or do several sites share an account?"
- "Are users and permissions part of the model, or may I take a logged-in user for granted?"

Each answer adds or removes two or three classes, so ask early. If the interviewer says "your choice", treat it as permission, not a dodge: **choose, say it out loud, write it down**. An assumption you state clearly counts just like an answer you were given.

What good looks like: the parking lot in ten lines

In scope

1. The lot has multiple floors; each floor has many parking spots.
2. Spot types: compact, large, handicapped, motorbike, electric.
3. Vehicle types: car, van, truck, motorbike, electric car.
4. A customer takes a ticket at an entrance panel on arrival.
5. A customer pays at an exit panel, at a floor portal, or to an attendant.
6. Payment is by cash or card.
7. Charges are per hour, with the hourly rate depending on how long the stay lasts.
8. Each floor shows how many spots of each type are free.
9. The system refuses entry when no suitable spot is free.
10. Admins may add floors, spots and panels, and may update the rate.

Out of scope: season passes, licence-plate recognition, booking before arrival, refunds, reports across several sites.

Every line above becomes a class, a field, or a method. **If no line asked for it, it does not go in the design.**

Make each requirement checkable

- **Put a number on it.** Rather than "a member can borrow a few books", write "a member may have **no more than five** books out at once". That number turns into a constant, and the constant turns into a check inside one method.
- **Name the actor.** Not "books can be reserved" but "**a member** can reserve a copy that is currently on loan" (you reserve a copy that is out so you get it next). Now you know which class the method belongs to.

A line written like this is close to a test case: you can point at one method and say "requirement 5 lives here".

Use cases

A **use case** is one goal an actor achieves with the system ("customer pays for parking"). A **use case diagram** (Chapter 3) draws actors as stick figures and use cases as ovals. In a 45-minute round a bulleted list is enough:

Actor	Use cases
Customer	take ticket, park, pay (cash/card), exit
Attendant	collect cash payment, help customer
Admin	add floor, add spot, add panel, change rate
System	assign spot, update display boards, refuse entry when full

Common mistakes

- **Saying yes to every idea.** Reservations come up and you accept. Multiple sites, accepted too. Twenty minutes on, you have 22 requirements and nothing designed. Declining is allowed: "Let me park reservations for now and get ticketing and payment working end to end first; I'll come back to it if time allows." Interviewers see that as a **strong** move, because it proves you can protect a fixed time budget from an open-ended ask.
- Forgetting non-functional requirements (concurrency, extensibility).
- Writing vague lines with no number and no actor.

In the interview

> **30-second answer** (to "how do you start an LLD question?"): "I ask four questions: who uses it, what each actor does, what the limits are, and what is out of scope. I write 8 to 12 numbered requirements and an out-of-scope list, confirm them with you, and then every class I create must trace back to one of those lines."

Follow-ups:

- *"Why not just start designing?"* — Without a boundary the design grows forever and I cannot defend omissions. Five minutes here saves fifteen later.
- *"What if I say 'your choice'?"* — I pick the simpler option, say it out loud, and write it as an assumption, so the decision is visible and reversible.

Self-check

- **Q:** Which of the four questions makes the problem smaller? **A:** "What is out of scope?"
- **Q:** Rewrite "users can book some seats" as a checkable requirement. **A:** "A customer can hold at most 10 seats for one show; held seats are released after 10 minutes if unpaid."
- **Q:** Where do limits end up in code? **A:** As constants plus a validation check in the method that owns the data.

TL;DR

Four questions (who, what, limits, out of scope) → 8–12 checkable lines with numbers and actors, plus an out-of-scope list. Saying "no" to extra scope is a strength.

1.6 Finding the classes

What it is

A reliable ten-minute method to go from requirements to a class list: **collect candidates, cut the ones that are not classes, give each survivor one job.**

How it works

1. **Underline the nouns** in the requirements. For the parking lot they fall into rough groups. Structure: parking lot, floor, spot, compact spot, large spot, entrance panel, exit panel, display board, portal. Vehicles: vehicle, car, van, truck, motorbike. People: customer, attendant, administrator. Money and time: ticket, payment, cash, card, rate, hour. That is deliberately too many: treat them as **candidates**. Since every one was taken from the requirements, none of them is made up.
2. **Underline the verbs.** Park, take a ticket, pay, leave, refuse entry, show free spots, add a floor, change the rate. **Each verb turns into a method** and has to land on some class. A verb with no home means a class is missing.
3. **Cut candidates with five tests:**

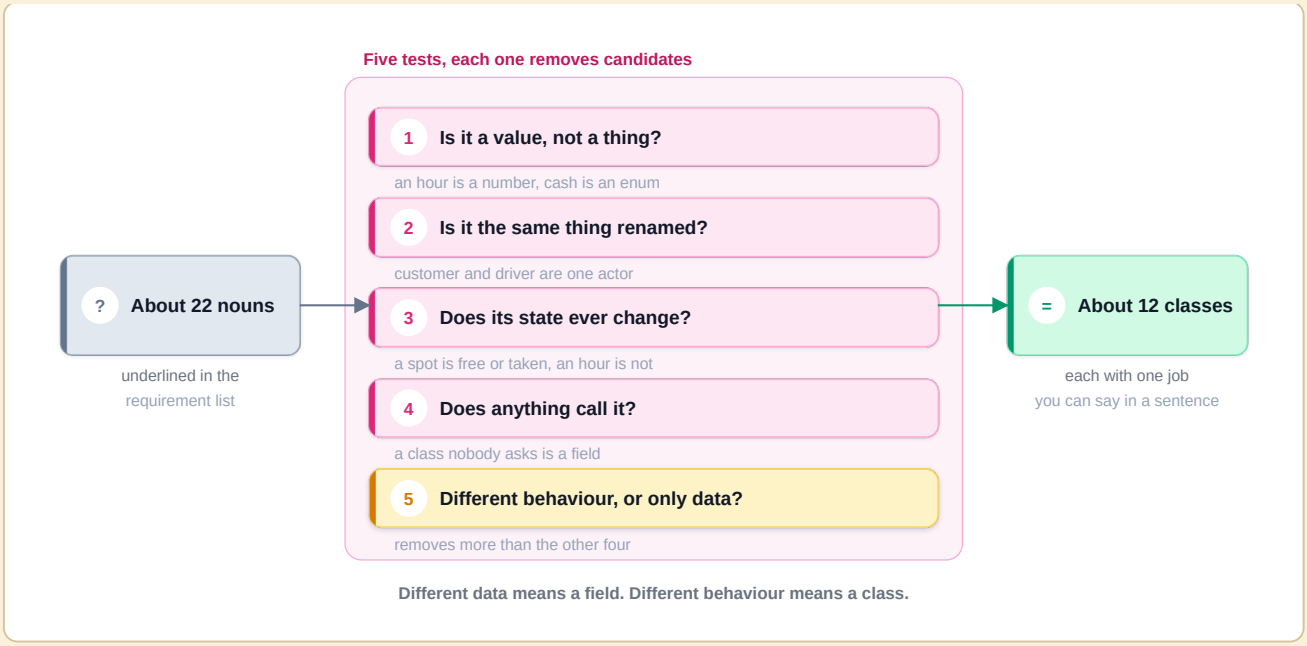


Figure — About 22 nouns pass through five tests and about 12 classes survive, each with one job (course diagram)

#	Test	Example	Result
1	Is it a **value** rather than a thing?	An hour is just a number, a licence plate just a string, and cash or card are options of a payment type	Field or **enum** constant (an enum is a fixed set of named values)
2	Is it just **another name** for a candidate already listed?	Customer and driver describe one actor; portal and panel could be a single class installed in several places	Merge, pick one name
3	Does it have **state that changes**?	A spot is free or occupied; an hour never changes	Changing state → class
4	Does anything **call** it?	Nobody ever asks an `Hour` object a question	Never called → it is a value
5	Does it differ in **behaviour** or only in **data**?	Compact vs large spot differ only in which vehicle fits; cash vs card payment behave differently	**Different data → a field. Different behaviour → a class.**

Applied to the parking-lot nouns, tests one to four cut the list to about fifteen; test five brings it to roughly twelve, removing more than the other four combined. Compact, large and handicapped spots all hold a vehicle, report free or occupied, and carry a number; they differ only in which vehicle kind is allowed. A difference like that is **data**, so a single `ParkingSpot` class gets a `type` field. Paying cash means checking enough money was given; paying by card means calling a provider and waiting for its reply. That difference is **behaviour**, so it earns two classes. Watch for this smell (a surface sign that something is probably wrong) in a diagram: five subclasses (classes that extend another class, their parent) whose only content is handing a different constant up to the parent. They are a single field pretending to be classes.

4. **Give each class one responsibility**, in one sentence with no "and":

Class	Responsibility
`ParkingLot`	Issues tickets and can tell when no space is left
`ParkingFloor`	Owns a level's spots and locates an empty one
`ParkingSpot`	Tracks if it is occupied and by which vehicle
`Vehicle`	Carries its plate number and the spot types it can use
`ParkingTicket`	Notes the vehicle, the spot it took, and the entry time
`ParkingRate`	Converts a duration in hours into a price
`Payment`	Collects the money and says if that worked
`ParkingDisplayBoard`	Shows how many spots of each type are free

`ParkingLot`'s sentence admits two jobs (issuing tickets **and** knowing fullness/assigning spots). In a longer design you would split it into a spot-assignment part and a ticketing part. In 45 minutes, **naming the tension out loud is enough**.

5. **Put each method where the data is.** "Find a free spot" touches the spot collection, which the floor owns → `ParkingFloor`. "Price the stay" touches the rate card → `ParkingRate`. "Take a ticket" touches the ticket store → `ParkingLot`. If a method needs data from three classes, it is probably in the wrong place; move it to the class that owns most of what it touches. (This is the GRASP "Information Expert" idea and the cure for the *Feature Envy* smell, Chapters 5 and 7. GRASP is a set of guidelines for deciding which class gets which job; *Feature Envy* is a method that uses another class's data more than its own.)

Common mistakes

- One subclass per value (`CompactSpot`, `LargeSpot` ... with no behaviour).
- Missing the **record-keeping** class (ticket, loan, booking) and the **rule** class (rate, strategy). These two are the classes beginners most often forget.
- "Manager" god classes that do everything (a *god class* is one huge class that owns most of the program's logic).

Self-check

- **Q:** Car, van and truck: subclasses or enum? **A:** If they differ only in size/which spots fit, an enum `VehicleType` (or a field). If they have genuinely different behaviour, subclasses.
- **Q:** A verb has no class to live on. What does that tell you? **A:** A class is missing.
- **Q:** A class has fields but no verbs. What is it? **A:** Probably a value (record/field), not a full class.

TL;DR

Nouns → candidate classes, verbs → methods. Cut values, duplicates, stateless and never-called nouns. **Different data = field; different behaviour = class.** One sentence per class, method goes where its data is. Twelve classes with one clear job each beat twenty that overlap.

1.7 Connecting the classes (preview)

Once you have the classes, decide for each pair: does one **use** the other briefly (dependency), **hold** a reference (association), **contain** parts that can live alone (aggregation), **own** parts that die with it (composition), or **is** it a kind of the other (inheritance, where a class extends another and reuses its code / realization, where a class implements an interface)? Then add **multiplicity** (1, 0..1, 1..*, *). Chapter 3 teaches every relationship and UML symbol in detail. For now, remember the most common beginner error: using **inheritance** where **composition** is right. The test is the sentence "A **is a** B". A car is not an engine, so `Car` has an `Engine`.

1.8 Keeping the design honest with SOLID

What it is

SOLID is five rules (one per letter) that catch the common ways a class diagram goes wrong. Chapter 4 teaches them in depth. In the interview, use them as **five quick checks** run over the diagram you just drew, in about five minutes.

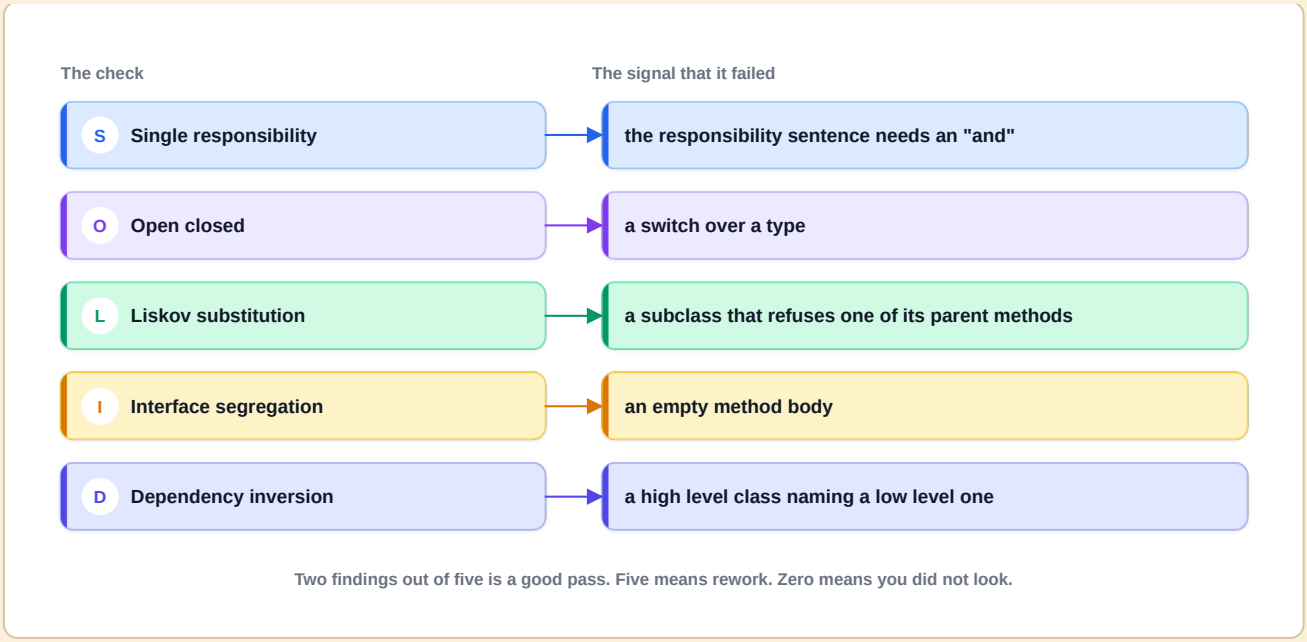


Figure — The five SOLID checks and the signal that each one has failed (course diagram)

How it works: run the five checks in this order

Check	Rule in one line	Signal that it failed (from the figure above)	Quick check
S — Single responsibility	A class should have one reason to change	The responsibility sentence needs an "and"	Name the change that would force an edit; two unrelated changes → split
O — Open/closed	Add behaviour without editing existing code	A `switch` / `if` chain over a type	Pick one likely new feature; if it edits three classes, the design is closed to the wrong thing
L — Liskov substitution	A subclass must work anywhere its parent works	A subclass that throws, does nothing, or refuses a parent method	Read each inheritance arrow: can the child be handed to code expecting the parent?
I — Interface segregation	No class should implement methods it does not use	Empty method bodies	Split the interface until every implementer uses everything
D — Dependency inversion	Depend on an interface, not a concrete class	A high-level class constructs or names a low-level one	Could you test the class without the real database/provider?

(In the **D** row, **high-level** code holds the business rules, while **low-level** code does one technical job such as talking to a database or a payment provider.)

Examples from the parking lot:

- **S:** a `ParkingLot` that both assigns spots and prints receipts changes when pricing changes **and** when the layout changes → two classes.
- **O:** pricing written as a switch over rate types means a weekend rate edits the class that prices everything. With pricing behind a small interface, the weekend rate is a new class.
- **L:** re-read every inheritance arrow at the end, because arrows are often added late when time is short. A `ReadOnlySpot extends ParkingSpot` whose `occupy` throws would fail the check: code that expects any spot to accept a vehicle breaks when handed that child.
- **I:** a library `Search` interface with four methods is fine if the catalog truly implements all four; it is not fine if half the implementers throw on two of them.
- **D:** a `ParkingLot` that does `new CreditCardTransaction()` cannot be tested without a payment provider and cannot take cash without an edit. Make it depend on `Payment`, and let something else decide which kind arrives.

The two that earn the most credit

- **Single responsibility** improves your class list: twelve well-named classes beat six classes doing three things each.

- **Open/closed** is what interviewers probe, usually as a late new requirement: "What if trucks are charged differently?" Answering "add a class" beats "add a branch to this switch".

When not to use it

- **Do not add an interface for everything.** An interface with one implementer, added "because SOLID", is indirection (an extra layer between the caller and the real code) with no benefit. Add it when a second implementation exists or is clearly coming (or when you need a test double, a fake stand-in object used in tests).
- **Do not split beyond what a reader can follow.** Thirty two-method classes are as bad as one god class. The one-sentence test is the boundary.

Each rule targets a **specific problem**. Apply it where your design shows that problem; applying it where the problem is absent only adds cost.

A five-minute pass over the parking lot

Check	Finding
S	`ParkingLot` issues tickets and finds spots: two reasons to change. Split if time allows; otherwise say it.
O	Pricing sits behind `ParkingRate`, so a new rate is a new class. Passes.
L (re-reading the inheritance arrows)	Five spot subclasses add no behaviour. Strictly this is not a Liskov violation (each would substitute fine); it is the "different data → field" test from 1.6 caught a second time while reading the arrows. Collapse them into a `type` field.
I	No wide interfaces. Passes.
D	The lot depends on `Payment`, not on the card class. Passes.

Two real findings, each explainable in a sentence: that is a good pass. **Five findings** usually means the design needs rework; **zero** usually means you did not really look.

TL;DR

Run S-O-L-I-D as five two-minute checks on your own diagram before the interviewer does. S and O earn the most credit. Do not add single-implementer interfaces or over-split.

1.9 The design patterns that come up in this round

What it is

A **design pattern** is a named solution to a design problem that keeps recurring. There are dozens (the Gang of Four book catalogues 23; Chapters 8–11 cover all of them). **Six** appear in this round often enough to know by name. The name turns a paragraph into two words and tells the interviewer you have seen the problem before. **A misapplied pattern hurts you more than an unknown one**, so learn each pattern together with the **signal** that says it applies.

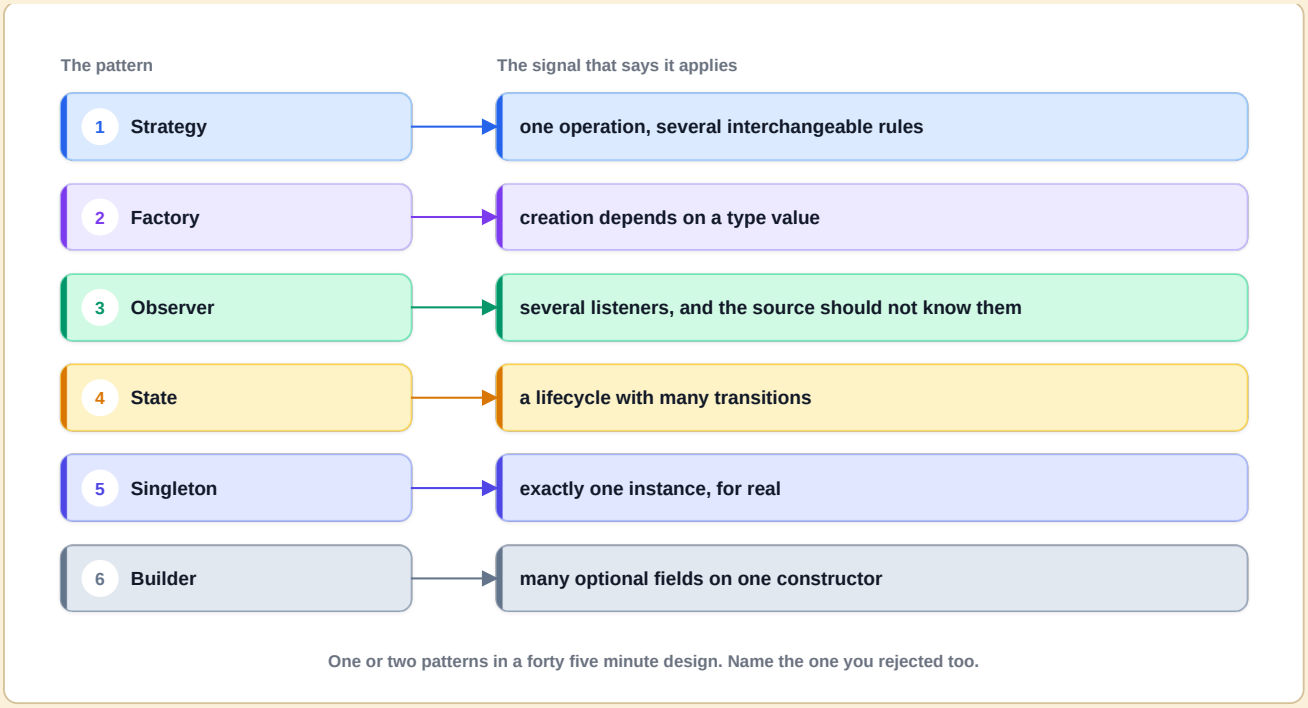


Figure — The six patterns most used in this round and the signal for each (course diagram)

Pattern	Signal (from the figure above)	Typical place	Parking lot / library example
Strategy	One operation, several interchangeable rules chosen at run time (while the program runs)	Pricing, matching, sorting, ranking, routing	`ParkingRate` interface with `HourlyRate`, `WeekendRate`, `MonthlyPass`. Answers "what if the rules change?", the most common late question
Factory	Creation depends on a type value; caller should not know the concrete class	Vehicles, spots, payments, notifications	A factory turns a type/enum into a `Vehicle`, so the entrance panel does not know every vehicle class. Keep it small: a factory **method** is usually enough
Observer	One thing changes, several others must know, and the source should not know them	Display boards, notifications, live updates, event logs	When a spot fills, all boards/panels registered on it update; adding a board changes nothing in the spot. With only **one** listener, a direct call is enough (e.g. notifying the member who reserved a returned book)
State	Behaviour depends on the current state, with a fixed set of transitions	Tickets, orders, bookings, loans, vending machines	Ticket: active → paid → closed. Book copy: available, reserved, loaned, lost. Each state class knows what is allowed, so illegal transitions become impossible. For 3–4 states, an **enum + one guarded method** is simpler: say so
Singleton	Exactly one instance must exist	The top-level object (`ParkingLot`, `Library`)	Private constructor + one accessor. Say out loud: it must be **thread-safe** (lock or eager creation, i.e. built when the class loads), and it is **global state** (data any code anywhere can reach and change), which hurts testing; many teams inject one shared instance instead
Builder	Many optional fields; the constructor is getting long	A book copy, a booking, a search query	`BookItem` with barcode, price, format, dates, rack: eight constructor args are easy to swap by mistake (two strings side by side compile fine). A builder sets fields **by name**. It is a readability fix, not a structural one

What to say about each one

- **Strategy:** without it, the pricing method grows a `switch` and every new rate edits it. With it, the lot holds one rate object and calls it; a new rate is a new class.
- **Factory:** it goes hand in hand with open/closed, since registering a new type happens in **a single spot**: the factory. Giving every product type its own factory class is heavier than a 45-minute design calls for.
- **Observer:** the simple version has the floor recompute and push to its single board. With Observer the **spot is the subject** and boards, entrance panels and floor portals register as observers; the spot never learns who is listening.

- **State:** without it, every method starts by checking the current status and the legal transitions are scattered across the class. With it, each state class knows what is allowed from there, so illegal moves cannot be expressed. Choose it only when the transition table is large, and say why.
- **Singleton:** expect the same two follow-ups (thread safety and testability) nearly every time; being able to state that trade-off counts for more than knowing the pattern.
- **Builder:** mention it when your own constructor reaches about six arguments, then move on.

Choosing patterns without overusing them

Pattern	Reach for it if	Skip it if
Strategy	A single operation has several swappable rules	Only one rule exists and a second is not in sight
Factory	Which object to build hinges on a type value	Just one concrete class exists
Observer	Several listeners; source should not know them	One listener the source already holds
State	The lifecycle has lots of transitions	A single guarded method covers three states
Singleton	There truly must be one instance	The real goal is just easy global access
Builder	Many optional fields	Three required fields

One or two patterns is the right number in a 45-minute design. Six patterns in one answer usually means the design is driven by patterns rather than by the requirements.

Nothing lands better than a **comparison**: "The rate gets Strategy because a second rate is likely. The ticket does not get State, since one guard is enough for three statuses." It proves you understand what the pattern gives and what it costs.

Real-world examples

- **Strategy:** `java.util.Comparator` passed to `List.sort`; Spring Security's pluggable `PasswordEncoder` implementations.
- **Factory** (static factory methods that hide the concrete class returned): `java.util.concurrent.Executors.newFixedThreadPool(...)`, `List.of(...)`, `Calendar.getInstance()`.
- **Observer:** Swing/AWT listeners (`ActionListener`), `java.beans.PropertyChangeListener`, Spring `ApplicationEvent` listeners.
- **State:** order lifecycle in e-commerce (placed → paid → shipped → delivered); TCP connection states.
- **Singleton:** `java.lang.Runtime.getRuntime()`; Spring beans are singleton-scoped **per container** by default (not the GoF class-enforced singleton).
- **Builder:** `StringBuilder`, `java.net.http.HttpRequest.newBuilder()`, `Stream.builder()`.

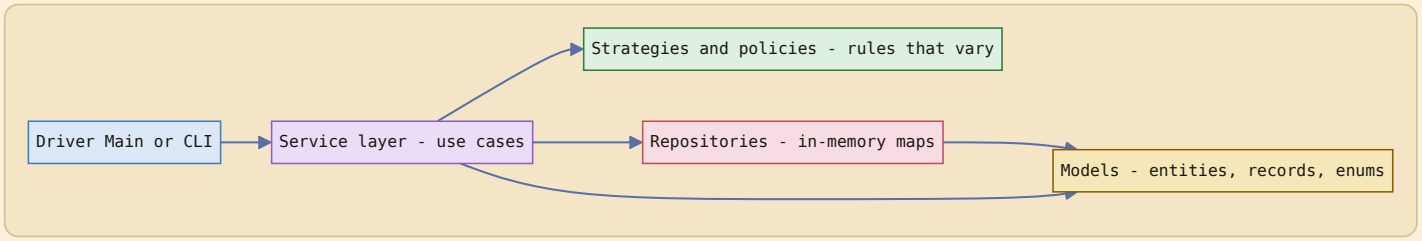
(Names in this list: **Spring** is a popular Java framework whose *container* creates and wires your application objects, called *beans*; **Swing/AWT** are Java's older desktop user-interface toolkits; **TCP** is the network protocol that keeps a reliable connection between two machines, see System Design Fundamentals; a **thread pool** is a fixed set of reusable worker threads.)

TL;DR

Know six by name: Strategy, Factory, Observer, State, Singleton, Builder. Use one or two where their signal appears, and name the one you rejected.

1.10 Code, edge cases and concurrency: how to write interview code

How it works: a machine-coding skeleton that scales



1. **Models** — entities (`ParkingSpot`; objects with an identity whose state changes over time), value objects (`record Vehicle`; plain data compared by its values), enums (`VehicleType`). Encapsulate state; validate in constructors.
2. **Repositories** — classes whose only job is storing and finding objects: an in-memory `Map<Id, Entity>` behind a small interface, so a database could replace it later.
3. **Strategies / policies** — every rule that might vary (pricing, spot selection, splitting a bill).
4. **Services** — the use cases (`park`, `exit`, `bookSeats`). This is the **service interface** a client calls.
5. **Driver** — a `Main` that runs a scripted scenario (or reads commands) and prints clear output for the demo.

Habits that interviewers notice:

- **Pass time in** (`long nowMinute`, `Instant now`, where `Instant` is Java's type for a point in time) rather than reading the clock inside the method: same input → same output → testable.
- **Hold money as** `BigDecimal` (built from strings such as `new BigDecimal("3.50")`) or as whole cents in a `long`; never `double`. This handbook uses `BigDecimal` in domain code.
- **Fail loudly on bad input**: `IllegalArgumentException` for bad arguments, `IllegalStateException` for wrong state, custom exceptions (`SpotNotAvailableException`) where they clarify.
- **Return** `Optional` (a box that either holds a value or is empty, used instead of returning `null`) when "not found" is a normal outcome.
- **Keep fields** `private final` unless they must change.
- **Name things after the domain** (`ParkingTicket`, not `Data2`).

Edge cases and concurrency checklist

Category	Questions
Input	null (no object at all), empty, negative, duplicate IDs, unknown IDs
Capacity	Full lot, empty playlist, zero seats left, max loans reached
State	Paying twice, exiting without entering, cancelling a delivered order
Time	Exit before entry, bookings expiring, part hours (round up?)
Concurrency	Two threads booking the same seat or taking the same spot: use `synchronized`, a `ReentrantLock`, or atomic structures such as `ConcurrentHashMap.computeIfAbsent`; lock the **smallest** shared thing that keeps the invariant

> **Key term — threads, locks and atomicity:** A **thread** is one independent line of execution; a server handling two panels at once runs two threads over the same objects. `synchronized` and `ReentrantLock` are Java's ways to let only one thread at a time into a block of code. An **atomic** operation happens all at once from other threads' point of view, with no half-finished moment they can see; `ConcurrentHashMap` is a thread-safe map that offers such operations. An **invariant** is a rule about the data that must always hold, such as "one spot holds at most one vehicle".

Chapters 12 and 13 cover concurrency in depth.

TL;DR

Models → repositories → strategies → services → driver. Inject time, use `BigDecimal` (or cents) for money, validate, return `Optional`, guard shared state.

1.11 Worked mini-example, end to end: a small parking lot

This runs every step of the framework on a deliberately small version of the classic question, so you can see the whole shape once. (A full parking lot with multiple panels, payments and display boards appears in the case-study chapters.)

Problem & clarifying questions

"Design a parking lot." Questions asked, with the answers we assume:

Question	Answer
Single lot or many?	Single lot, one or more floors
Which vehicles?	Motorbike, car, truck
Which spot types?	Motorbike, compact, large; a smaller vehicle may use a bigger spot
Payment?	Out of scope: we compute the fee; a payment service collects it
Pricing?	Hourly, tiered, part hours round up; trucks pay double; must be swappable (weekend flat rate)
Concurrency?	Several entry/exit panels may act at the same time

Requirements

Functional

1. The lot has floors; each floor has spots of type MOTORBIKE, COMPACT or LARGE.
2. A motorbike fits MOTORBIKE or COMPACT; a car fits COMPACT or LARGE; a truck fits LARGE only. The smallest fitting spot is tried first.
3. On entry, the system assigns a free fitting spot and issues a ticket (id, vehicle, spot, entry time).
4. If no fitting spot is free, entry is refused.
5. On exit, the ticket is closed, the spot freed and the fee returned.
6. Fee: $4.00 first hour, $3.50 for hours 2–3, $2.50 per hour after; part hours round up; minimum one hour; trucks pay double.
7. The pricing rule can be swapped at run time (e.g. a flat $10 weekend rate).
8. A floor can report how many spots of a type are free.

Non-functional: in-memory; thread-safe entry/exit; new pricing rules or vehicle types must not require editing existing services; deterministic (time passed in).

Out of scope: payment collection, reservations, display boards, number-plate recognition, multiple lots.

Use cases

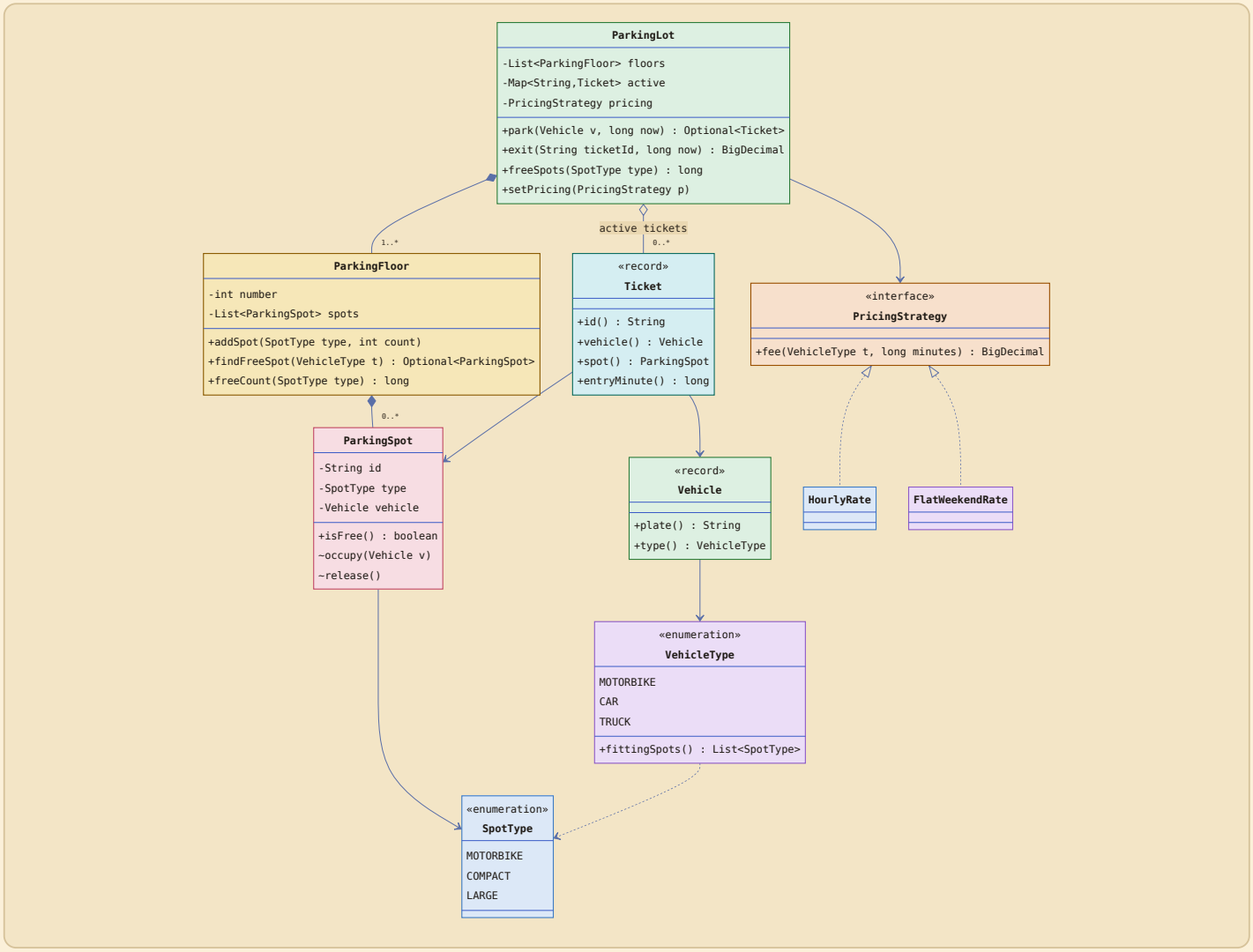
Actor	Use cases
Driver (customer)	enter and get ticket, exit and see fee
Admin	add floor/spots, change pricing rule
System	find free spot, refuse entry when full, compute fee

Core entities

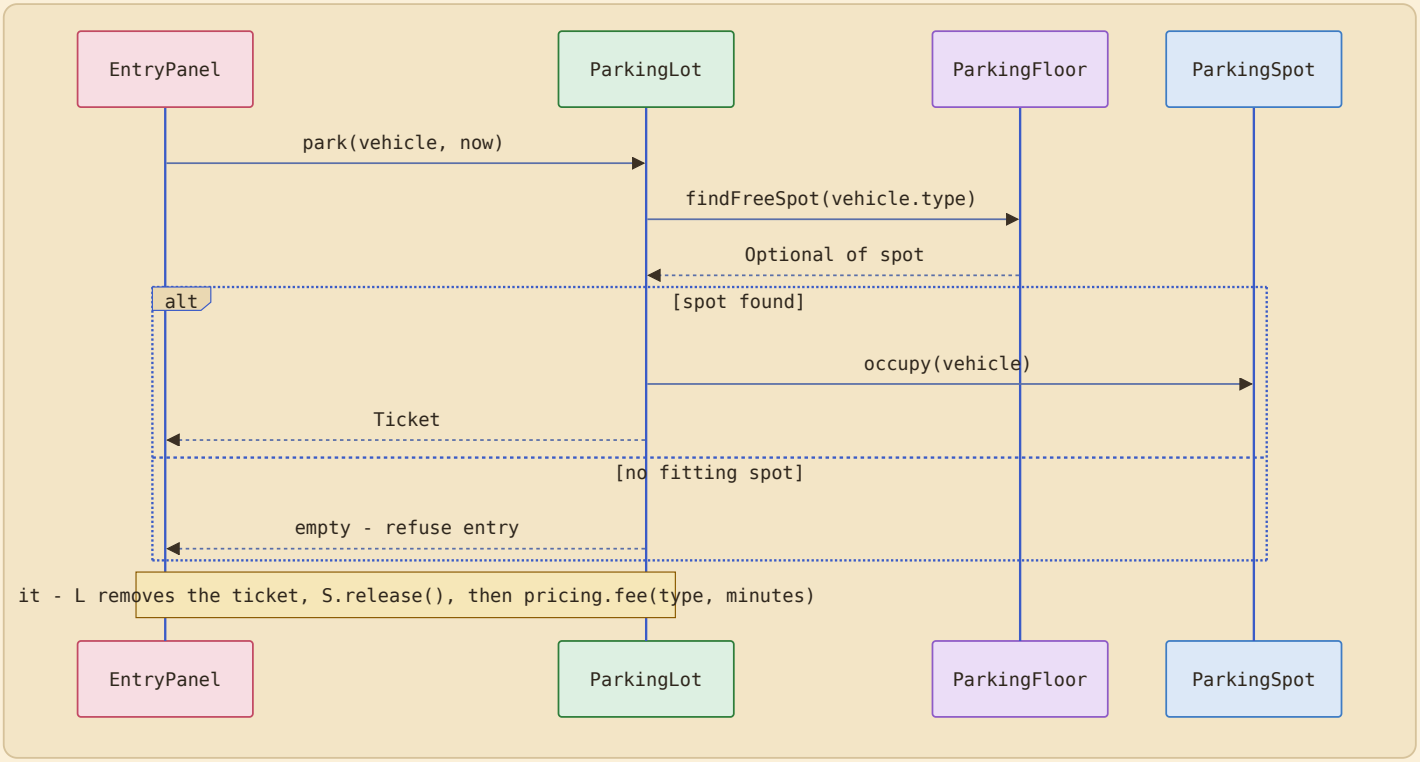
Class	Kind	Responsibility
`SpotType`	enum	The spot sizes
`VehicleType`	enum	Vehicle kinds, each knowing which spot types fit (smallest first)
`Vehicle`	record	Plate + type (pure data)
`ParkingSpot`	class	Tracks if it is occupied and by which vehicle
`ParkingFloor`	class	Holds spots on one floor and finds a free fitting one
`Ticket`	record	Records which vehicle went into which spot, and when (the "connection object")
`PricingStrategy`	interface	Turns a stay into a fee (`BigDecimal`)
`HourlyRate`, `FlatWeekendRate`	classes	Two pricing rules
`ParkingLot`	class	Issues and closes tickets; the service the panels call

Notice the **"different data → field"** rule: spot sizes are an enum, not three subclasses, because they do not behave differently.

Class diagram



Key flows



Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for pricing (`PricingStrategy`)	A second rule (weekend) is explicitly required; new rules = new classes (open/closed)	A `switch` over a rate enum inside `ParkingLot`
Spot and vehicle types as **enums**	They differ in data only	One subclass per spot type
`Ticket` as a **record**	A ticket is a fact that never changes after issue. (A record is only *shallowly* immutable: its fields cannot be reassigned, but the `ParkingSpot` it points to still changes state.)	Mutable ticket with setters (methods that change fields)
Money as `BigDecimal`	Exact decimal arithmetic; `3.50 + 3.50` is exactly `7.00`	`double` (binary fractions, rounding drift)
`synchronized` on `park` / `exit` / `freeSpots`	Two panels must never get the same spot; lot-wide lock is simple and correct for this size	Per-floor locks (better throughput, i.e. more requests handled per second; more complex; mention as an extension)
No Singleton for `ParkingLot`	Injecting one instance keeps it testable	Global `getInstance()`
No State pattern for tickets	Only active/closed; a map of active tickets is enough	State classes per ticket status

Java implementation

```java
// File: code/lld/ch01/parkinglot/FlatWeekendRate.java
package lld.ch01.parkinglot;

import java.math.BigDecimal;

/** A NEW rule added later: flat $10.00 per visit. Added as a class; no existing code was edited (open-closed). */
public final class FlatWeekendRate implements PricingStrategy {
    private static final BigDecimal FLAT = new BigDecimal("10.00");

    @Override public BigDecimal fee(VehicleType type, long minutesParked) { return FLAT; }
}
```

```java
// File: code/lld/ch01/parkinglot/HourlyRate.java
package lld.ch01.parkinglot;

import java.math.BigDecimal;

/** $4.00 for the first hour, $3.50 for hours 2 and 3, $2.50 per hour after that. Part hours round up. Trucks pay double. */
public final class HourlyRate implements PricingStrategy {
    private static final BigDecimal FIRST_HOUR = new BigDecimal("4.00");
    private static final BigDecimal HOURS_2_TO_3 = new BigDecimal("3.50");
    private static final BigDecimal LATER_HOURS = new BigDecimal("2.50");

    @Override
    public BigDecimal fee(VehicleType type, long minutesParked) {
        long hours = Math.max(1, (minutesParked + 59) / 60);    // ceiling, minimum one hour
        BigDecimal total = BigDecimal.ZERO;
        for (long h = 1; h <= hours; h++) {
            total = total.add(h == 1 ? FIRST_HOUR : (h <= 3 ? HOURS_2_TO_3 : LATER_HOURS));
        }
        return type == VehicleType.TRUCK ? total.multiply(BigDecimal.TWO) : total;
    }
}
```

```java
// File: code/lld/ch01/parkinglot/ParkingFloor.java
package lld.ch01.parkinglot;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/** Holds the spots on one floor and finds a free one. Not thread-safe on its own: ParkingLot calls it under its lock. */
public final class ParkingFloor {
    private final int number;
    private final List<ParkingSpot> spots = new ArrayList<>();

    public ParkingFloor(int number) { this.number = number; }

    public void addSpot(SpotType type, int count) {
        for (int i = 1; i <= count; i++) {
            spots.add(new ParkingSpot("F" + number + "-" + type.name().charAt(0) + i, type));
        }
    }

    /** First free spot, trying the vehicle's preferred (smallest) spot type first. */
    public Optional<ParkingSpot> findFreeSpot(VehicleType vehicleType) {
        for (SpotType wanted : vehicleType.fittingSpots()) {
            for (ParkingSpot s : spots) {
                if (s.type() == wanted && s.isFree()) return Optional.of(s);
            }
        }
        return Optional.empty();
    }

    public long freeCount(SpotType type) {
        return spots.stream().filter(s -> s.type() == type && s.isFree()).count();
    }
}
```

```java
// File: code/lld/ch01/parkinglot/ParkingLot.java
package lld.ch01.parkinglot;

import java.math.BigDecimal;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/**
 * Issues tickets and closes them. Time is PASSED IN (not read from the clock) so runs are
 * repeatable. Every method that reads or changes spots, tickets or pricing is synchronized on
 * the lot, so two entrance panels cannot hand out the same spot.
 */
public final class ParkingLot {
    private final List<ParkingFloor> floors;
    private final Map<String, Ticket> active = new HashMap<>();
    private PricingStrategy pricing;
    private int nextTicket = 1;

    public ParkingLot(List<ParkingFloor> floors, PricingStrategy pricing) {
        this.floors = List.copyOf(floors);
        this.pricing = pricing;
    }

    public synchronized Optional<Ticket> park(Vehicle v, long nowMinute) {
        for (ParkingFloor floor : floors) {
            Optional<ParkingSpot> spot = floor.findFreeSpot(v.type());
            if (spot.isPresent()) {
                spot.get().occupy(v);
                Ticket t = new Ticket("T" + nextTicket++, v, spot.get(), nowMinute);
                active.put(t.id(), t);
                return Optional.of(t);
            }
        }
        return Optional.empty();              // lot full for this vehicle type -> refuse entry
    }

    /** Closes the ticket, frees the spot and returns the fee. */
    public synchronized BigDecimal exit(String ticketId, long nowMinute) {
        Ticket t = active.get(ticketId);
        if (t == null) throw new IllegalArgumentException("unknown or already closed ticket " + ticketId);
        if (nowMinute < t.entryMinute()) throw new IllegalArgumentException("exit before entry");
        active.remove(ticketId);               // validate first, then change state
        t.spot().release();
        return pricing.fee(t.vehicle().type(), nowMinute - t.entryMinute());
    }

    /** Free spots of one type across all floors, read under the same lock as park/exit. */
    public synchronized long freeSpots(SpotType type) {
        return floors.stream().mapToLong(f -> f.freeCount(type)).sum();
    }

    public synchronized void setPricing(PricingStrategy pricing) { this.pricing = pricing; }
}
```

```java
// File: code/lld/ch01/parkinglot/ParkingSpot.java
package lld.ch01.parkinglot;

/** Knows whether it is free and which vehicle is in it. */
public final class ParkingSpot {
    private final String id;
    private final SpotType type;
    private Vehicle vehicle;                // null = free

    public ParkingSpot(String id, SpotType type) { this.id = id; this.type = type; }

    public boolean isFree()      { return vehicle == null; }
    public SpotType type()       { return type; }
    public String id()           { return id; }

    void occupy(Vehicle v) {                // package-private: only the lot may change a spot
        if (!isFree()) throw new IllegalStateException(id + " is taken");
        vehicle = v;
    }
    void release() { vehicle = null; }
}
```

```java
// File: code/lld/ch01/parkinglot/PricingStrategy.java
package lld.ch01.parkinglot;

import java.math.BigDecimal;

/** Strategy: turns a stay into an amount of money. Money is BigDecimal, never double (no binary rounding errors).
*/
public interface PricingStrategy {
    BigDecimal fee(VehicleType type, long minutesParked);
}
```

```java
// File: code/lld/ch01/parkinglot/SpotType.java
package lld.ch01.parkinglot;

/** Spot kinds differ only in data (which vehicles fit), so they are enum values, not subclasses. */
public enum SpotType { MOTORBIKE, COMPACT, LARGE }
```

```java
// File: code/lld/ch01/parkinglot/Ticket.java
package lld.ch01.parkinglot;

/** Records which vehicle went into which spot, and when. The "connection object" between Vehicle and Spot. */
public record Ticket(String id, Vehicle vehicle, ParkingSpot spot, long entryMinute) { }
```

```java
// File: code/lld/ch01/parkinglot/Vehicle.java
package lld.ch01.parkinglot;

/** A vehicle is pure data: its plate and its type. */
public record Vehicle(String plate, VehicleType type) { }
```

```java
// File: code/lld/ch01/parkinglot/VehicleType.java
package lld.ch01.parkinglot;

import java.util.List;

/** Each vehicle type knows which spot types fit it, smallest first. */
public enum VehicleType {
    MOTORBIKE(List.of(SpotType.MOTORBIKE, SpotType.COMPACT)),
    CAR(List.of(SpotType.COMPACT, SpotType.LARGE)),
    TRUCK(List.of(SpotType.LARGE));

    private final List<SpotType> fits;

    VehicleType(List<SpotType> fits) { this.fits = fits; }

    public List<SpotType> fittingSpots() { return fits; }
}
```

```java
// File: code/lld/ch01/parkinglot/Main.java
package lld.ch01.parkinglot;

import java.math.BigDecimal;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        ParkingFloor ground = new ParkingFloor(0);
        ground.addSpot(SpotType.MOTORBIKE, 1);
        ground.addSpot(SpotType.COMPACT, 1);
        ground.addSpot(SpotType.LARGE, 1);
        ParkingLot lot = new ParkingLot(List.of(ground), new HourlyRate());

        Vehicle car   = new Vehicle("KA-01-1234", VehicleType.CAR);
        Vehicle truck = new Vehicle("KA-02-9999", VehicleType.TRUCK);
        Vehicle car2  = new Vehicle("KA-03-5555", VehicleType.CAR);

        Ticket t1 = lot.park(car, 0).orElseThrow();
        System.out.println(t1.id() + ": " + car.plate() + " -> spot " + t1.spot().id());
        Ticket t2 = lot.park(truck, 10).orElseThrow();
        System.out.println(t2.id() + ": " + truck.plate() + " -> spot " + t2.spot().id());
        System.out.println(car2.plate() + " -> " + lot.park(car2, 20).map(Ticket::id).orElse("refused, no suitable
spot"));

        System.out.println("free compact spots: " + lot.freeSpots(SpotType.COMPACT));
        System.out.println(t1.id() + " exits after 150 min, pays " + money(lot.exit(t1.id(), 150)));
        System.out.println(t2.id() + " exits after 50 min, pays " + money(lot.exit(t2.id(), 60)));

        lot.setPricing(new FlatWeekendRate());          // swap the strategy at run time
        Ticket t3 = lot.park(car2, 200).orElseThrow();
        System.out.println(t3.id() + ": " + car2.plate() + " -> spot " + t3.spot().id()
                + ", weekend exit pays " + money(lot.exit(t3.id(), 500)));

        try {
            lot.exit(t1.id(), 600);
        } catch (IllegalArgumentException e) {
            System.out.println("error: " + e.getMessage());
        }
    }

    private static String money(BigDecimal amount) { return "$" + amount.toPlainString(); }
}
```

Run: `java -cp /tmp/lld_01 lld.ch01.parkinglot.Main`

Output:

```
T1: KA-01-1234 -> spot F0-C1
T2: KA-02-9999 -> spot F0-L1
KA-03-5555 -> refused, no suitable spot
free compact spots: 0
T1 exits after 150 min, pays $11.00
T2 exits after 50 min, pays $8.00
T3: KA-03-5555 -> spot F0-C1, weekend exit pays $10.00
error: unknown or already closed ticket T1
```

Reading the output: the car took the compact spot, the truck the only large spot, so the third car was refused (requirement 4). 150 minutes = 3 hours = $4.00 + $3.50 + $3.50 = $11.00. The truck's 50 minutes round up to one hour, doubled = $8.00. After swapping the strategy at run time, the same service charges the flat $10. Closing a ticket twice is rejected.

Concurrency & edge cases

- `park` and `exit` are `synchronized` on the lot, so find-then-occupy is **atomic**: two panels cannot both see the same spot as free and both occupy it (a classic *check-then-act* race: a **race condition** is a bug whose result depends on which thread happens to run first).
- `freeSpots` is read under the **same** lock, so a display never sees a half-finished `park`. `ParkingFloor` itself is not thread-safe; it is only ever called through the lot.
- `exit` validates the ticket and the time **before** changing anything, so a rejected exit (for example exit before entry) leaves the ticket open.
- `ParkingSpot.occupy` also throws if the spot is taken: a second line of defence.
- Exit before entry, unknown ticket, and double exit all throw `IllegalArgumentException`.
- `occupy` / `release` are **package-private**, so code outside the package cannot change spot state directly.

Extensions & follow-up questions

Follow-up	Answer
"Add electric-vehicle spots with charging."	Add `ELECTRIC` to `SpotType`, add an `ELECTRIC_CAR` vehicle type whose fitting list starts with `ELECTRIC`. If charging has **behaviour** (meter, charging fee), then a `ChargingSpot` class or a charging component composed into the spot.
"Choose the nearest spot to the entrance instead of the first free one."	Extract a `SpotSelectionStrategy` interface (Strategy again), injected into `ParkingLot`.
"Update display boards when spots change."	Observer: spots/floors publish "spot freed/occupied" events; boards subscribe.
"Improve throughput with many panels."	Lock per floor (or per spot type) instead of the whole lot; or keep free spots in a `ConcurrentLinkedQueue` (a thread-safe queue) per type and `poll()` atomically.
"Add payment by cash or card."	`Payment` interface with `CashPayment`, `CardPayment`; `exit` returns the fee and a `PaymentService` collects it (dependency inversion).

In the interview

Spend about 5 minutes on the questions and requirement list above, 3 on actors, 8 on entities and the "enum vs subclass" decision, 8 on the diagram, and 15 on `park`, `exit` and `HourlyRate`. Close by naming Strategy (used), Singleton and State (rejected, with reasons), and the per-floor locking extension.

Self-check

- **Q:** Why is `SpotType` an enum and not a class hierarchy? **A:** Spot types differ only in data (which vehicles fit), not in behaviour.
- **Q:** What race does `synchronized` on `park` prevent? **A:** Two threads finding the same free spot and both occupying it (check-then-act).
- **Q:** How did adding a weekend rate respect open/closed? **A:** It was a new `PricingStrategy` class; no existing class was edited.

TL;DR

Small, complete, and honest: 8 requirements, 10 types, one Strategy, one lock, and a clear list of what was left out.

1.12 Communication tips

Do	Why
Think out loud, narrating the step you are in ("Now I'll list actors")	The interviewer scores your process, not only the result
Write things down (requirements, class list) where the interviewer can see them	Shared artefacts prevent misunderstandings
Confirm before moving on ("Does this scope look right?")	Cheap course corrections
State assumptions explicitly	An assumption said out loud counts like an answer from the interviewer
Offer trade-offs in pairs: "I chose X over Y because..."	Shows judgement
Welcome hints; treat the interviewer as a teammate	Collaboration is part of the evaluation
Say what you are leaving out and how you would add it	Turns a gap into a strength
Keep a clock in view; announce when you move to code	Prevents the "no code at minute 40" failure
Use domain names, not `Manager`, `Helper`, `Data`	Clear names are clear thinking
When stuck, go back to the requirements	Every class must trace back to a line

What **not** to do: go silent for minutes, argue with feedback, name `HashMap`s and threads in minute two (that is implementation, skipping analysis and design), or recite pattern definitions without a reason to use them.

1.13 How to use this handbook, and a study plan

Attempt first, then compare

There are two ways to work through case studies. The first is to **read** them. That is quick and pleasant but builds very little skill, because any finished design looks obvious after someone else has drawn it. The second is to **attempt each one yourself, then compare**. That is slower and less comfortable, and it is the approach that actually improves how you perform under pressure.

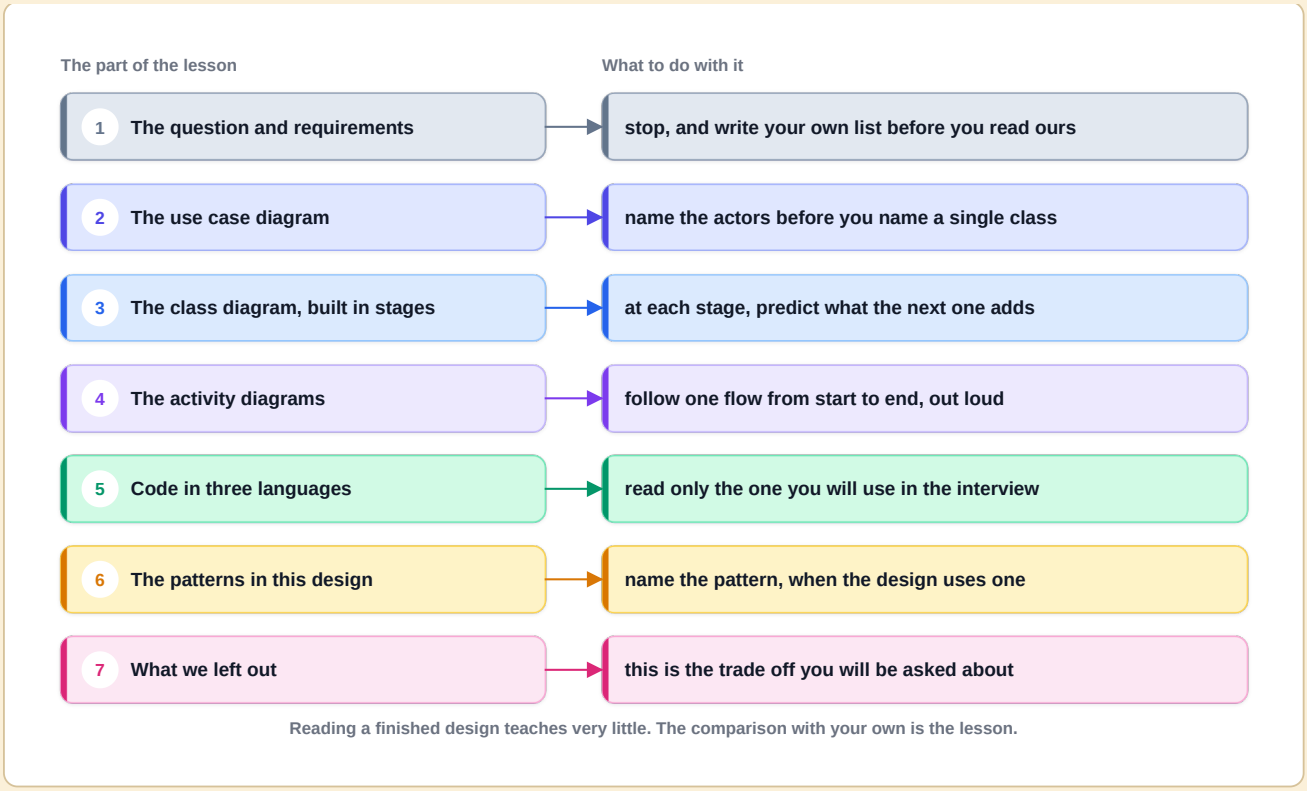


Figure — The parts of a case-study lesson and what to do at each one (course diagram)

As the figure above shows, every case study has the same parts, so you can decide in advance where to stop and think. Three stopping points matter most:

1. **Stop after the question.** Write your own requirement list (8–12 lines plus out of scope), then compare. If yours is shorter, the cause is usually not missing knowledge but **not asking enough questions**.
2. **Stop after the requirements.** Write your class list, one responsibility each, then read the class diagram. Most people are close but **two classes short**, and they are usually the same kind: **the thing that keeps a record** (ticket, loan, booking) and **the thing that decides a rule** (rate, strategy).
3. **Stop at each stage of the class diagram.** Predict what the next stage adds. A wrong guess is free, and each one shows you how an experienced designer thinks.

More habits:

- **Read the class diagram in stages.** Many case-study diagrams are built up step by step; you do not need to read the UML chapter first. Seeing a symbol inside a real design and then looking it up in Chapter 3 works well.
- **If a design decision looks wrong to you, write down why.** Arguing with a worked answer is good practice; several of these designs exist in their current form because someone disagreed with an earlier version.
- **Read one language's code** (here, Java). Reading multiple translations teaches nothing about design.
- **Do not skip the extensions / "what we left out" section.** It lists the simplifications and their trade-offs, and interviewers ask about exactly these. "I merged these two responsibilities to save time; here is how I would split them" is worth far more than hoping nobody asks.
- **Do not invent pattern names.** Where plain classes are enough, say so. Labelling a pattern the design does not use loses credibility.
- **After six or seven case studies, switch to timed runs**: a question you have already done, 45 minutes, blank page. You will not finish; nobody does. Measure two things: did you spend the first ten minutes on requirements and actors, and did you reach running code for at least one flow? These predict the real result better than the number of classes you drew.
- **Compile and run every example** (`bash tools/compile_all.sh` runs all chapters; `bash` is the Unix command shell and the `.sh` file is a script of commands).

Map of this handbook

Ch	Title	Use it for
1	The LLD interview & how to approach it	The method, time budget, study plan (this chapter)
2	OOP fundamentals	Classes, objects, the four pillars, interfaces, enums, equality, immutability, generics, OOA/OOD (object-oriented analysis / design)
3	Relationships & UML	Association, aggregation, composition, dependency; class, use case, activity, sequence, state diagrams
4	SOLID principles	The five principles in depth
5	Design principles	DRY (don't repeat yourself), KISS (keep it simple), YAGNI (you aren't gonna need it), composition over inheritance, Law of Demeter, GRASP and friends
6	Clean code	Naming, functions, comments, error handling, tests
7	Refactoring	Code smells and the named refactorings that fix them
8	Creational patterns	Singleton, Factory Method, Abstract Factory, Builder, Prototype
9	Structural patterns	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
10–11	Behavioural patterns	Strategy, Observer, State, Command, Template Method, Iterator, Chain of Responsibility, Mediator, Memento, Visitor, Interpreter; MVC (Model-View-Controller)
12	Concurrency concepts	Threads, locks, `java.util.concurrent`, thread-safety patterns
13	Concurrency problems	Producer–consumer, readers–writers, rate limiters and other classic problems
14	Case studies: everyday objects	Parking lot, vending machine, ATM, elevator, traffic signal, coffee machine
15–16	Case studies: booking and inventory	Library, movie tickets, concerts, hotels, car rental, restaurant, airline, course registration
17	Case studies: games	Chess, tic-tac-toe, snake and ladder, 2048, blackjack
18–19	Case studies: platforms	Social and Q&A sites, cricket scores, Amazon-style shopping, food delivery, auctions, stock brokerage and exchange, music streaming
20	Case studies: components	Caches (including LRU, which evicts the least recently used item), logger, rate limiter (caps how many requests are allowed per time window), pub-sub (publishers send messages to all subscribers of a topic), notifications, Splitwise, ride sharing and similar
21	Case studies: machine coding	90-minute style problems with runnable solutions: task manager, digital wallet, event calendar, coding contest platform, property listing, loan ledger, inventory
22	Cheat sheets & glossary	Last-day revision

Study plans

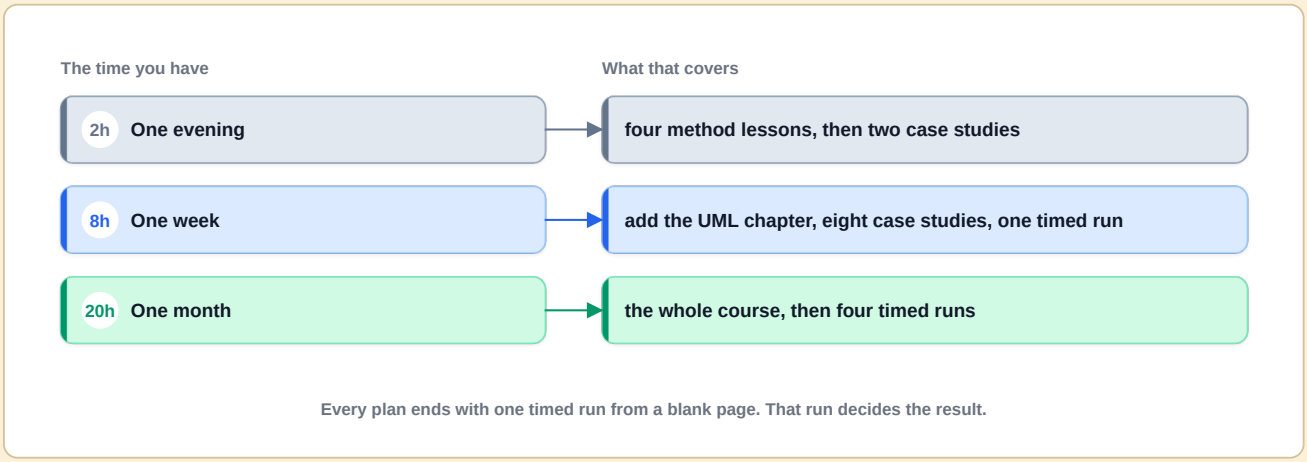


Figure — Three study plans for three amounts of time; every plan ends with one timed run from a blank page (course diagram)

Choose by the time you really have (your **calendar**), not the time you wish you had. The **method is worth more than any single case study**, and the first six case studies are worth more than the rest, because later ones repeat skills the earlier ones taught.

If the interview is tomorrow (about 2 hours)

Order	Read	Time
1	Section 1.2 (for the time budget)	10 min
2	Section 1.5 (requirements)	15 min
3	Section 1.6 (finding the classes)	15 min
4	Chapter 3: choosing the relationship between two classes	15 min
5	Chapter 3: class diagram notation	10 min
6	Parking lot case study, whole	30 min
7	Vending machine case study, whole	25 min

Two case studies show the structure twice. The parking lot is the most-asked question; the vending machine shows what to do when a design has a **lifecycle** (a fixed set of states an object moves through), the second most common kind of question. Do not start a third; write the five steps on a card and rehearse them.

If you have one week (about 8 hours, an hour a day)

Day	Do
1	Chapter 1 fully, plus Chapter 2 sections on the four pillars
2	Chapter 3: use case, class and activity diagrams (sequence diagrams later)
3	Parking lot, then library management (write your requirements first)
4	ATM, then vending machine
5	Movie ticket booking, then hotel management
6	Splitwise, then Amazon-style online shopping
7	**No new material.** Redo the parking lot from a blank page in 45 minutes

Day 7 matters most: days 1–6 fill your head with material, and day 7 checks that you can actually produce a design against the clock.

If you have one month (about 20 hours)

Week	Do
1	Chapters 1–4, then the first three case studies
2	Chapters 5, 8–11 (patterns) alongside case studies 4–10, two per session, always drawing your classes first
3	Chapters 12–13 (concurrency) and case studies 11–18; one timed run at the end of the week
4	Remaining case studies, Chapter 21 machine-coding problems, then three timed runs on questions you have read; Chapter 22 for revision

By week three the case studies feel repetitive. That means the method has been learned; keep going but read faster, checking your class list against the first diagram stages.

Beginner plan (six weeks, 5–6 hours a week) — if OOP itself is new to you:

Week	Do
1	Chapters 1–2; write and run every Chapter 2 example yourself
2	Chapters 3–5 (UML, SOLID, principles)
3	Chapters 6–7 (clean code, refactoring) and Chapter 8
4	Chapters 9–11 (remaining patterns); two small case studies
5	Chapters 12–13; four case studies including one with concurrency (ticket booking)
6	Chapter 21 machine-coding problems in 90-minute timed runs; Chapter 22

If you can only do six case studies, do these; they cover the skills the others repeat:

Case study	What it teaches
Parking lot	The canonical question, and how to bound one
Library management	Borrow, return and limits, a common family of questions
Vending machine	A lifecycle, one class per state
Movie ticket booking	Two people wanting the same seat: the concurrency question
Splitwise	Money, and a rule with several versions
Amazon online shopping	A question too big to finish, and choosing the part you build

Add a seventh, **an in-memory cache (LRU)**, if your target companies ask for a data structure rather than a product.

The night before: do not learn a new question (a half-understood design adds anxiety and will not be remembered). Instead: re-read the time budget (knowing you are *allowed* eight minutes on requirements stops you rushing), re-read the extensions section of one case study you have done, and write the five steps from memory: **requirements, actors and use cases, classes, relationships, code.**

1.14 Common mistakes

Mistake	Why it hurts	Fix
Coding in minute two	Missing requirements appear at minute 35	Spend 8–13 minutes on requirements and actors
No out-of-scope list	Scope creeps; omissions look like oversights	Write it down and agree it
Agreeing to every suggested feature	Nothing gets finished	"I'll leave X out and add it if time allows"
One subclass per value	Class explosion with no behaviour	Different data → field/enum
God class (`ParkingLotManager` does everything)	Fails single responsibility; untestable	One sentence per class, split on "and"
Missing the record class (ticket/loan/booking)	Data ends up on the wrong class (`dueDate` on `Book`)	Connection that carries data = an object
Inheritance for code reuse ("Car extends Engine")	Wrong "is-a"; fragile	Composition
Pattern stuffing	Looks like memorisation, not judgement	One or two patterns, with the signal
`switch` on type everywhere	Every new type edits many places	Polymorphism (one call, different behaviour per class) / Strategy
All twelve classes at equal depth	No flow works end to end	Two or three flows fully
Silent designing	Interviewer cannot credit your reasoning	Narrate each step
Ignoring concurrency when two users share a resource	Double booking	Lock or atomic operation on the shared state
Non-running code in machine coding	Often an automatic reject	Get a thin end-to-end path running by halfway
Using `double` for money, reading the clock inside logic	Rounding bugs, flaky demo	Cents/`BigDecimal`, inject time

---

In the interview

30-second answer (to "How do you approach a low-level design problem?"): "First I clarify and write 8 to 12 requirements with an explicit out-of-scope list. Then I list the actors and their use cases. From the nouns I pick the classes, cutting anything that is only a value, and give each class one responsibility. I connect them with the right relationships and draw a class diagram, sketch the key flow, and run a quick SOLID check and pick at most one or two patterns where the signal is clear. Then I code the two or three core flows, handle edge cases and concurrency, and finish by discussing extensions and trade-offs."

Follow-ups:

- *"How is this different from system design?"* — System design is about many machines and scale (databases, caches, queues). LLD stays inside one application: classes, interfaces, relationships and code.
- *"You won't finish; what matters?"* — Depth on the chosen part: a bounded scope, clean responsibilities, correct relationships, code that matches the diagram, and one trade-off I can defend.
- *"What changes in a machine-coding round?"* — The code must run. I keep storage in memory, define service interfaces and a driver, get the happy path working by halfway, then add features, validation and one extension point.
- *"How do you decide whether to use a pattern?"* — I look for the signal: several interchangeable rules → Strategy, lifecycle with many transitions → State, many listeners → Observer. If there is no signal, plain

classes are the better answer.

Self-check

- **Q:** Name the five core steps in order. **A:** Requirements, actors & use cases, classes, relationships, code.
- **Q:** How much of a 45-minute round goes before naming a class? **A:** Roughly a quarter (8–13 minutes).
- **Q:** What are the five things an interviewer judges? **A:** Bounded problem, one-sentence responsibilities, correct relationships, code matching the diagram, an explained trade-off.
- **Q:** What does a machine-coding round add? **A:** Working code within about 90 minutes, data models, service interfaces, extensibility, and a demo.
- **Q:** Which two classes do beginners most often miss? **A:** The record-keeping class (ticket/loan/booking) and the rule class (rate/strategy).
- **Q:** "Different data" vs "different behaviour"? **A:** Different data → field or enum. Different behaviour → separate class.
- **Q:** How many patterns in a 45-minute design? **A:** One or two, placed where the signal appears; name a rejected one too.
- **Q:** What should the last day of any study plan be? **A:** A timed run from a blank page.

TL;DR

- LLD/OOD = design the classes inside one application and code the main flows; machine coding = the same with running code in ~90 minutes.
- Clarify → requirements → use cases → entities → relationships → diagram → flows → patterns/SOLID → code → edge cases/concurrency → extensions.
- Spend a quarter of the time before naming a class; write an out-of-scope list.
- Different data → field; different behaviour → class; one sentence per class.
- Run SOLID as five quick checks; use one or two patterns where the signal appears.
- Code two or three flows properly; inject time, use `BigDecimal` (or cents) for money, guard shared state.
- Practise by attempting first, then comparing; finish every plan with a timed blank-page run.

Chapter 2 — OOP Fundamentals

What you will learn in this chapter

- What object-oriented programming (OOP) is and why the LLD round is really a test of it.
- Classes and objects, constructors, `this`, `static` vs instance members, and `final`.
- The four pillars — **encapsulation, abstraction, inheritance, polymorphism** — on one running bank example.
- Inheritance details (`super`, overriding rules) and polymorphism details (overloading vs overriding, dynamic dispatch).
- Interfaces (default, static, private methods, functional interfaces), abstract classes, and when to pick which.
- Enums with fields and behaviour.
- Composition over inheritance, identity vs equality (`equals`/`hashCode`), immutability and records.
- Access modifiers and packages, and the generics you actually need in LLD.
- Object-oriented analysis vs design vs programming (OOA / OOD / OOP), with a four-step process.

Every concept has a Java 21 example that compiles and runs; the real output is shown under each program.

How to run all examples: `cd chapters/02-oop-fundamentals/code && javac --release 21 -d /tmp/lld_02 $(find . -name '*.java') && java -cp /tmp/lld_02 lld.ch02.<package>.Main`

(`javac` is the Java compiler: it turns `.java` source files into `.class` files. `java` runs them on the **JVM**, the Java Virtual Machine, the program that executes compiled Java code; `-cp` says where the compiled classes live.)

Concept checklist (the commonly listed "must-know OOP concepts" and where each is covered):

Concept	Section	Concept	Section
Classes & objects	2.2	Interfaces (default/static/functional)	2.8
Constructors, constructor chaining	2.2	Abstract class vs interface	2.9
`this` keyword	2.2	Enums	2.10
`static` vs instance, `final`	2.2	Composition over inheritance	2.11
Encapsulation	2.3	Identity vs equality, `equals`/`hashCode`	2.12
Abstraction	2.4	Immutability & records	2.13
Inheritance, `super`, overriding rules	2.5	Access modifiers & packages	2.14
Polymorphism, overloading, dynamic dispatch	2.6–2.7	Generics basics	2.15
Coupling & cohesion	2.16	OOA / OOD / OOP	2.17

2.1 What object-oriented programming is

What it is

Object-oriented programming (OOP) is a style of programming that **bundles data with the code allowed to modify it**, packaging both into a single unit called an **object**.

Procedural programming separates the two: data sits in one place, and free-standing functions anywhere in the program can read and modify it. OOP instead **fences the data in**: the only code that can reach it is the object's own methods.

Analogy: a bank vault vs a pile of cash on a table. With the pile (procedural), anyone can take or add money and nobody enforces rules. With the vault (OOP), you must go through the teller (the methods), who checks every request.

The problem it solves

When any code can change any data, a rule such as "a balance can never go negative" has to be copied into every caller, and one forgotten copy is a bug. OOP puts each rule in **one place**, next to the data it protects, and lets a design grow by **adding new classes** instead of editing old ones.

Why this matters for the interview

Nobody in the LLD round will ask you to recite textbook definitions. What they want to see is you **breaking a problem into objects, assigning each one its responsibilities and data, and justifying that breakdown**. Every idea in this chapter is a tool for that.

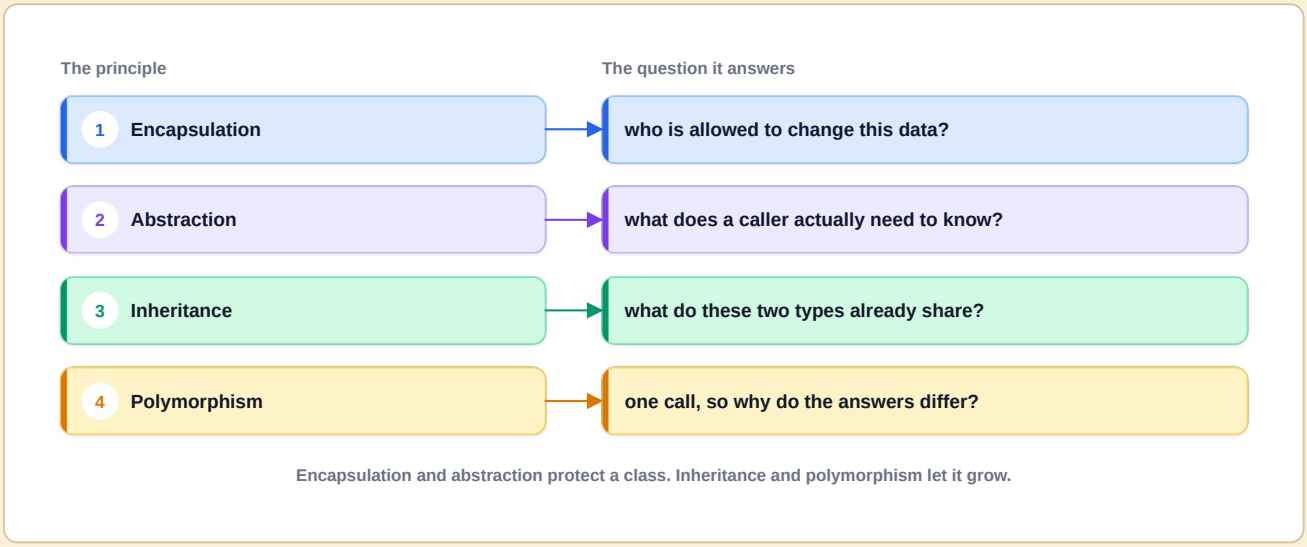


Figure — The four OOP principles and the question each one answers (course diagram)

As the figure above summarises, each pillar answers one design question:

Pillar	Question it answers
Encapsulation	Who is allowed to change this data?
Abstraction	What does a caller actually need to know?
Inheritance	What do these two types already share?
Polymorphism	One call, so why do the answers differ?

The first two pillars guard a single class; the last two let the design expand with new classes.

2.2 Classes, objects, constructors, `this`, `static` and `final`

What it is

- A **class** is the **blueprint**: it lists the fields a thing has and the operations it supports. You write it once.
- An **object** (instance) is **one concrete thing built from that blueprint**, holding actual values in its fields. You create as many as you need with `new`.
- A **field** is a variable that belongs to a class or object. A **method** is a function that belongs to a class or object.
- A **constructor** is special code that runs when an object is created. It has the class's name and **no return type**. Its job: put the new object into a **valid** state.
- `this` is a reference to **the current object** inside its own instance methods and constructors.
- `static` means "belongs to the **class**, shared by all objects". Without `static`, a member is an **instance** member: each object has its own copy.
- `final` means "cannot change": a `final` variable is assigned once, a `final` method cannot be overridden (replaced by a child class with its own version), a `final` class cannot be extended (used as the parent of another class; Section 2.5).

Examples: the `Account` class defines what any account looks like, while account `S-1` (owner Ada, balance 50,000 cents) is a single object of it. In an online shop, `Customer` is the class, and the particular shopper behind order 4192 from last Tuesday is an object.

How it works

1. `new Account("S-1", "Ada", 50_000)` allocates memory for the fields, sets them to defaults (0, `null` meaning "no object", `false`), then runs the constructor.
2. **Initialisation order:** static initialisers (`static` field assignments and `static { ... }` blocks of set-up code) run **once**, when the class is initialised (just before its first real use, such as the first `new` or the first static method call). For each `new`: the parent constructor runs first (`super(...)`, implicit if you do not write it), then instance field initialisers and instance initialiser blocks (plain `{ ... }` blocks inside the class body) in source order, then the rest of the constructor body.
3. If you write **no constructor**, the compiler adds a public no-argument **default constructor**. If you write any constructor, it does not.
4. **Overloaded constructors** (same name, different parameters) should **chain** to one canonical constructor (the single "main" constructor that all the others call) with `this(...)`, so validation lives in one place. In Java 21, `this(...)` or `super(...)` must be the **first statement**. (Java 25 relaxes this with "flexible constructor bodies", JEP 513 (a JDK Enhancement Proposal, the numbered documents that describe new Java features): statements that do not use the object under construction may come before the call. This book targets Java 21, so keep the call first.)
5. Uses of `this`: disambiguate a field from a parameter (`this.id = id`), pass the current object to another method, chain constructors (`this(...)`), and return the object for **fluent chaining** (`return this`).
6. `static` methods have **no** `this`: they cannot touch instance fields directly. Use them for utilities (small helper functions such as `Math.max`) and factories (methods that create and return objects).
7. Variables of class type hold **references** (like an address), not the object itself. `Account alias = a;` copies the reference: both names now point to the **same** object.

Java example

```java
// File: code/lld/ch02/classes/Account.java
package lld.ch02.classes;

/** A CLASS is the blueprint. Every Account OBJECT gets its own copy of the instance fields. */
public class Account {
    // static = belongs to the CLASS, shared by all objects
    private static int openedCount = 0;
    public static final long MIN_OPENING_CENTS = 0;

    // instance fields = belong to EACH object
    private final String id;
    private final String owner;
    private long balanceCents;

    /** Canonical constructor: does all validation in one place. */
    public Account(String id, String owner, long openingCents) {
        if (openingCents < MIN_OPENING_CENTS) throw new IllegalArgumentException("negative opening balance");
        this.id = id;                // 'this.id' is the field, 'id' is the parameter
        this.owner = owner;
        this.balanceCents = openingCents;
        openedCount++;               // shared counter
    }

    /** Overloaded constructor: chains to the canonical one with this(...). Must be the first statement. */
    public Account(String id, String owner) {
        this(id, owner, 0);
    }

    public Account deposit(long cents) {   // returns 'this' so calls can be chained
        if (cents <= 0) throw new IllegalArgumentException("deposit must be positive");
        this.balanceCents += cents;
        return this;
    }

    public long balanceCents() { return balanceCents; }
    public String id()         { return id; }
    public String owner()      { return owner; }

    public static int openedCount() { return openedCount; }  // static method: no 'this' available here

    @Override public String toString() { return id + "(" + owner + ", " + balanceCents + "c)"; }
}
```

```java
// File: code/lld/ch02/classes/Main.java
package lld.ch02.classes;

public class Main {
    public static void main(String[] args) {
        Account a = new Account("S-1", "Ada", 50_000);          // 3-arg constructor
        Account b = new Account("C-1", "Kai");                  // 2-arg constructor chains to 3-arg
        b.deposit(1_000).deposit(500);                          // chaining via 'return this'

        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("accounts opened (static, shared): " + Account.openedCount());

        Account alias = a;                                      // copies the REFERENCE, not the object
        alias.deposit(1);
        System.out.println("after alias.deposit(1), a = " + a + ", same object? " + (alias == a));

        try {
            new Account("X", "Bad", -5);
        } catch (IllegalArgumentException e) {
            System.out.println("constructor refused: " + e.getMessage());
        }
        System.out.println("accounts opened: " + Account.openedCount() + " (failed construction did not count)");
    }
}
```

Output:

```
a = S-1(Ada, 50000c)
b = C-1(Kai, 1500c)
accounts opened (static, shared): 2
after alias.deposit(1), a = S-1(Ada, 50001c), same object? true
constructor refused: negative opening balance
accounts opened: 2 (failed construction did not count)
```

Explaining each part: `openedCount` is `static`, so there is one counter for the class; `balanceCents` is an instance field, so each account has its own. The two-argument constructor calls `this(id, owner, 0)`, so validation is written once. `deposit` returns `this`, enabling `b.deposit(1_000).deposit(500)`. `alias == a` is `true` because both variables hold the same reference. The failing constructor threw **before** incrementing the counter, so no half-built object was counted: **validate first, then change state**.

Trade-offs

Choice	Benefit	Cost
Static members	Shared, no object needed	Global state (reachable from anywhere): hard to test and dangerous with threads (parts of a program running at the same time)
Many constructor overloads	Convenient	Confusing beyond 3–4; use a **Builder** (Chapter 8; a helper that collects settings step by step, then creates the object) or static factory methods
`final` fields	Safer, thread-friendlier	Must be set in every constructor

Common mistakes

- Putting per-object data in a `static` field (every object then shares it).
- Doing heavy work or calling overridable methods inside constructors (a subclass override may run before the subclass's fields are set).
- Forgetting `this.` so the parameter is assigned to itself (`id = id;`).
- Thinking `=` copies an object; it copies a reference.

Real-world examples

`Math.max`, `Integer.parseInt`, `List.of` are static methods; `StringBuilder.append` returns `this` for chaining; `String` is a `final` class.

In the interview

> **30-second answer:** "A class is the blueprint, an object is an instance with its own state. Constructors put an object into a valid state; I chain overloads with `this(...)` so validation lives once. Static members belong to the class and are shared, so I keep them for constants, utilities and factories and avoid mutable (changeable) static state."

- *"What happens if you define a constructor with arguments only?"* — The default no-arg constructor is no longer generated; `new X()` stops compiling.
- *"Can a static method use `this`?"* — No; there is no current object in a static context.
- *"Why avoid mutable static fields?"* — They are hidden global state shared by all threads and tests.

Self-check

- **Q:** How many copies of a `static` field exist for 1,000 objects? **A:** One.
- **Q:** What runs first: parent constructor or child field initialisers? **A:** Parent constructor.
- **Q:** What does `alias == a` compare? **A:** References (whether both point to the same object).

TL;DR

Class = blueprint, object = instance. Constructors create valid objects; chain them with `this(...)`. `static` = one per class; `final` = cannot change. Variables hold references.

2.3 Encapsulation

What it is

Encapsulation means an object hides its own data (**private**) and allows the outside world to modify it **only by calling methods the object itself defines and guards**. Analogy: a medicine bottle with a child-proof cap: the contents are reachable, but only through a mechanism that enforces the rules.

The problem it solves

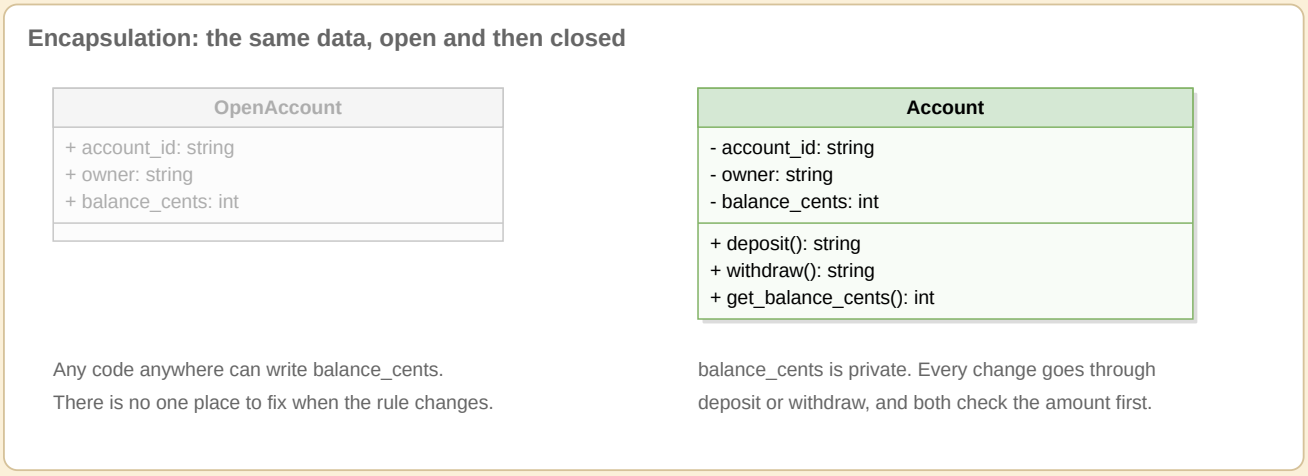


Figure — The same account data, first open (public fields anyone can write) and then closed (private fields changed only through deposit and withdraw) (course diagram)

The left side of the figure shows unprotected, public fields: **code in any file** may write a negative number into `balance`, and if the rule ever changes you have **nowhere central** to update it. (That open version exists only as a contrast; our code does not contain it.) The right side makes the balance private, and both `deposit` and `withdraw` validate the amount first. Overdrawing is therefore impossible: **every route to the balance goes through a method that can say no**. (The default `monthlyUpdate` on `Account` does nothing yet; it becomes useful in Section 2.6.)

How it works

1. Make fields `private` (and `final` where possible).
2. Expose **behaviour** (`deposit`, `withdraw`), not raw setters (`setBalance`; methods that simply overwrite a field).
3. Validate inside those methods: the rule lives **once**.
4. Return copies or unmodifiable views (read-only wrappers that throw if someone tries to change them) of internal collections (never the live list).
5. For every field, settle **which code may modify it**. When the honest answer is "nobody but this class", state it aloud in the interview: it turns a list of fields into a visible design decision.

Java example

The `Account` class of the bank example in Section 2.6 (package `lld.ch02.bank`) is the encapsulation example: `balanceCents` is `private`; `deposit` rejects non-positive amounts; `withdraw` rejects amounts larger than the balance; both are `final` so a subclass cannot weaken the rule.

Trade-offs

Benefit	Cost
Invariants (rules that must always hold) enforced in one place	A little more code than public fields
Internals can change without breaking callers	Over-encapsulating trivial data carriers is noise (use records)

When not to use it (strictly)

Plain data carriers with no rules (DTOs, Data Transfer Objects that only carry data between parts of a system; events, records of something that happened) do not need hand-written getters and validation; a **record** gives you private final fields and accessors for free.

Common mistakes

- **Getter/setter for every field** (a getter is a method that just returns a field; "anemic" objects): the data is technically private but anyone can still set anything. That is not encapsulation.
- Returning a mutable internal list (`return items;`), so callers modify your state behind your back.

Real-world examples

`java.lang.String` hides its internal byte array; `ArrayList` hides its backing array (the plain array that actually stores the elements) and `size`; `LocalDate` exposes only validated factory methods (`LocalDate.of(2026, 2, 30)` throws).

In the interview

> **30-second answer:** "Encapsulation hides state behind methods, and those methods are the single home of every rule about that state. For each field I ask who may change it; if only this class, it is private and changed through intention-revealing methods (named after the business action they perform) like `withdraw`, not a setter."

- *"Are getters and setters encapsulation?"* — Only partly. A setter with no rule is a public field in disguise. Prefer behaviour methods that enforce rules.

Self-check

- **Q:** Where should the "no negative balance" rule live? **A:** Inside `Account.withdraw`, once.

TL;DR

Private data + methods that enforce the rules = encapsulation. Expose behaviour, not setters.

2.4 Abstraction

What it is

Abstraction means giving a caller **just the operations it requires** while concealing **how** they are carried out. Put simply: encapsulation conceals **state**, abstraction conceals **design choices**, and the short list of operations stays valid even if everything behind it is rewritten. Analogy: a car's steering wheel and pedals. You drive without knowing about fuel injection, and the engine can be replaced by an electric motor without changing how you drive.

The problem it solves

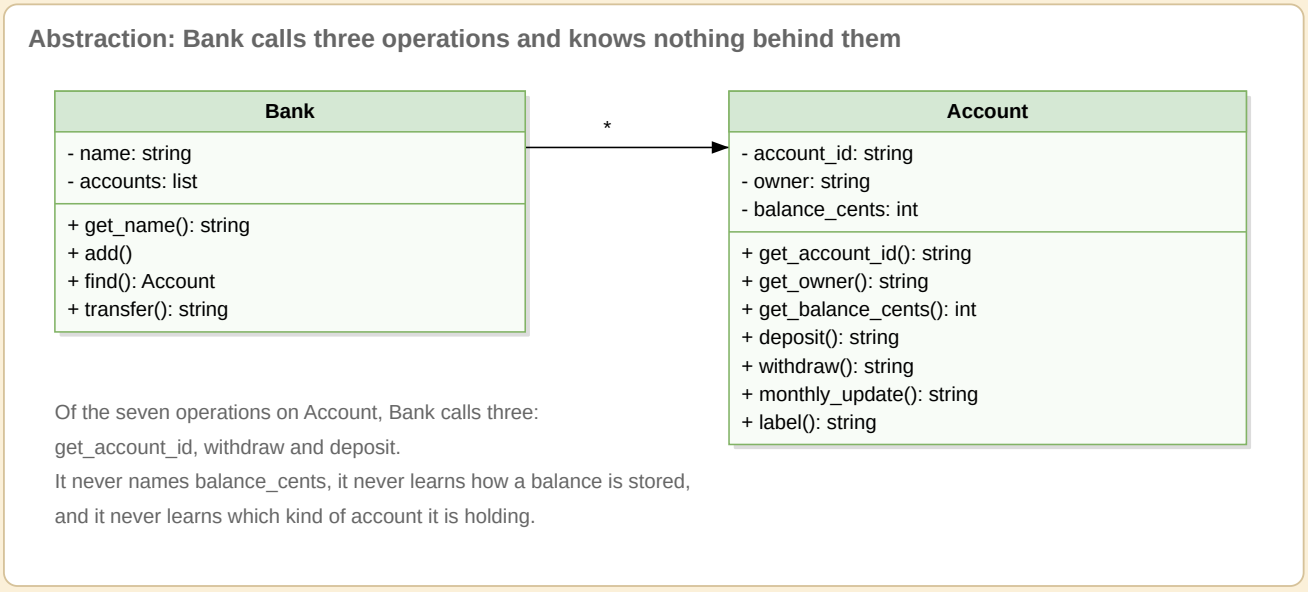


Figure — Of Account's seven public operations, Bank relies on three and sees nothing of what lies behind them (course diagram)

In the course diagram `Account` has seven public operations (it includes a `label()` helper; our Java `Account` has six: `id`, `owner`, `balanceCents`, `deposit`, `withdraw`, `monthlyUpdate`). Either way, `Bank.transfer` relies on only three of them: read the id, withdraw, deposit. `Bank` has no idea that balances are stored as whole cents, what checks an account performs, or that several account subclasses exist. That ignorance is the point: the balance could be moved from a `long` field into a database table and `transfer` would stay exactly as it is.

How it works

1. Decide the **smallest set of operations** a caller needs.
2. Name them after **what** they do (`transfer`), not how.
3. Put them on an interface (a named list of methods with no code, which classes promise to provide; Section 2.8) or the public surface of a class; keep everything else private.
4. Callers depend on that surface only.

Java example

`Bank.transfer` in Section 2.6 uses only `id()`, `withdraw()` and `deposit()`.

Trade-offs

Benefit	Cost
Callers are simpler and survive internal rewrites	A wrong abstraction is expensive to change later
Enables polymorphism (many implementations behind one surface)	Too many layers make code hard to follow

Common mistakes

- **Leaky abstraction:** the interface exposes implementation details (`getHashMapOfSeats()`).
- Abstracting too early, before two concrete cases exist.

Real-world examples

`java.util.List` (callers do not care whether it is an `ArrayList` or `LinkedList`); JDBC's `Connection`/`Statement` (JDBC, Java Database Connectivity, is Java's standard way to talk to SQL databases; same code for different databases); `java.io.InputStream`.

In the interview

> **30-second answer:** "Abstraction is exposing only what the caller needs. My test is the sentence 'the caller does not need to know that': whenever I can honestly say it, something moves behind the interface, the design shrinks, and I can change the implementation freely."

- *"Encapsulation vs abstraction?"* — Encapsulation hides and protects **data** with access control (keywords such as `private` that decide who may see a member, Section 2.14); abstraction hides **complexity and decisions** behind a small set of operations. They work together.

Self-check

- **Q:** Which three `Account` operations does `Bank.transfer` need? **A:** `id()`, `withdraw()`, `deposit()`.

TL;DR

Show the minimum surface; hide the how. "The caller does not need to know that."

2.5 Inheritance

What it is

Inheritance lets a new class (the **child**, subclass) start with everything an existing class (the **parent**, superclass) has, then **add** fields/methods or **replace** (override) some methods. In Java: `class SavingsAccount extends Account`. Analogy: a savings account **is a** bank account with one extra rule.

The problem it solves

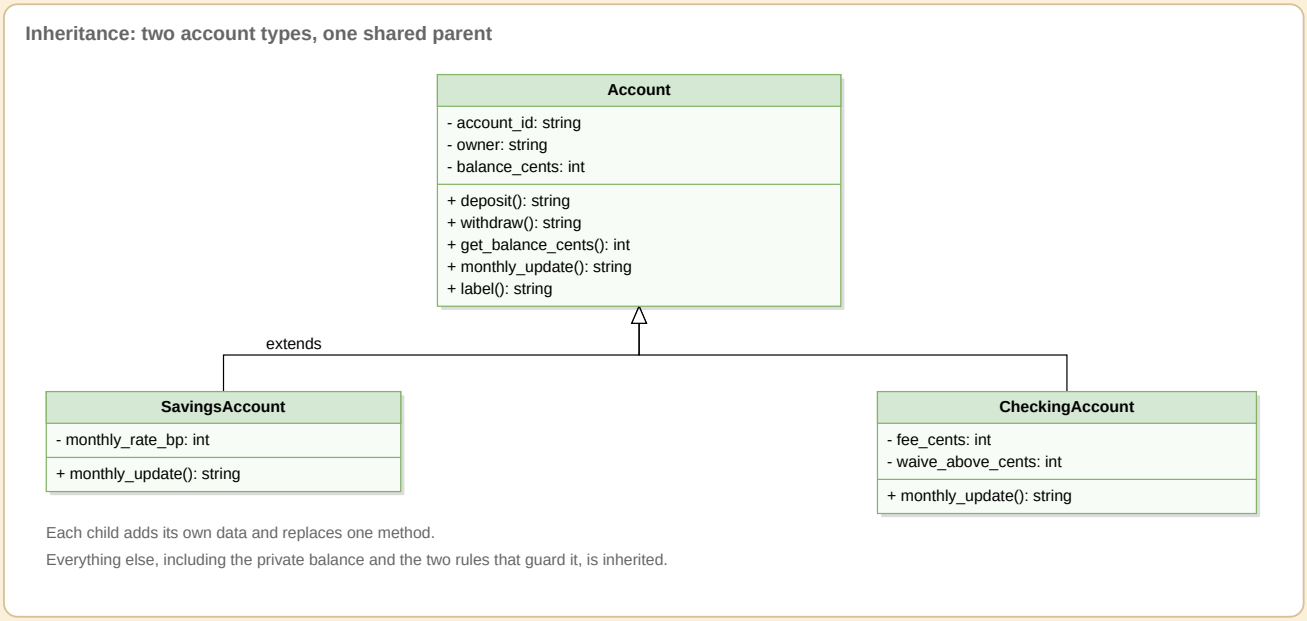


Figure — SavingsAccount and CheckingAccount extend Account; each adds its own data and replaces one method (course diagram)

Savings accounts earn interest; checking accounts take a monthly fee unless their balance clears a threshold. Apart from that, the two share everything: the balance, the deposit check and the withdrawal check, so all of that comes from the parent. In the figure, savings contributes one new field and checking two, and **each overrides a single method**; neither duplicates the balance or its rules. If the is-a sentence were false, the difference would belong in a **field**, not in a parent class.

How it works

1. Declare `class Child extends Parent`. Java allows **one** parent class (single inheritance of classes), but many interfaces.
2. The child inherits all **accessible** (non-private) members. Private fields still exist in the object but are reached only through the parent's methods. **Constructors are not inherited.**
3. The child constructor must call a parent constructor, `super(...)`, as its first statement (implicitly `super()` if omitted, which fails to compile if the parent has no no-arg constructor).
4. `super.method()` calls the parent's version from inside an override (reuse, then extend).
5. **Overriding rules:**

Rule	Detail
Same signature	Same method name and parameter types
Return type	Same, or a **subtype** (a type that extends or implements it) for reference types (*covariant return*); primitives (`int`, `long`, `boolean`...) must match exactly
Access	Same or **wider** (protected, i.e. visible to subclasses, → public is fine; public → protected is not)
Checked exceptions	May throw fewer or narrower checked exceptions, never new or broader ones; unchecked are free
Cannot override	`final` methods, `static` methods (they are **hidden**, not overridden), `private` methods (not inherited)
Annotation	Always write `@Override` (an annotation is an `@` tag that gives the compiler extra information): the compiler then catches typos and wrong signatures

(About the exceptions row: a **checked exception**, such as `IOException`, is one the compiler forces callers to catch or declare; an **unchecked exception**, such as `IllegalArgumentException`, extends `RuntimeException` and needs no declaration.)

6. Every class ultimately extends `java.lang.Object`, which is why every object has `toString`, `equals`, `hashCode` and `getClass`.
7. **The is-a test:** the sentence "a SavingsAccount **is an** Account" must be true in every situation. A car is not an engine, so `Car` should **hold** an `Engine` (Section 2.11).

Java example

See `SavingsAccount` and `CheckingAccount` in Section 2.6 (each calls `super(...)` and overrides `monthlyUpdate()`), and `Circle` in Section 2.7 (covariant return, `super.who()`, static-method hiding, field hiding).

Trade-offs

Benefit	Cost
Reuse of shared state and behaviour	**Tight coupling** (a change in one class forces changes in another): child depends on parent internals (fragile base class)
Enables polymorphism	Hierarchy is fixed at compile time (when the code is compiled, before it runs); single parent only
Natural for true "is-a" families	Deep hierarchies (more than 2–3 levels) are hard to reason about

When not to use it

- Only to **reuse code** when the is-a sentence is false (use composition).
- When the child would need to **disable** a parent method (throw `UnsupportedOperationException` or do nothing): that breaks the **Liskov Substitution Principle** (Chapter 4): the child can no longer stand in for the parent.
- For variations that differ **only in data** (use a field or enum).

Common mistakes

- `Square extends Rectangle` with independent `setWidth`/`setHeight`: code expecting a rectangle breaks.
- Forgetting `@Override` and silently **overloading** instead (e.g. `equals(Point p)` instead of `equals(Object o)`).
- Calling overridable methods from a parent constructor.

Real-world examples

Good: `FileInputStream extends InputStream`; Spring's (Spring is a popular Java application framework) `AbstractController`-style base classes. Known mistakes in the JDK (Java Development Kit: the compiler plus the standard library) itself: `java.util.Stack extends Vector` (a stack, which should only add and remove at the top, exposes random-access `insertElementAt`, which reaches any position), and `java.util.Properties extends Hashtable` (lets you put non-string entries).

In the interview

> **30-second answer:** "Inheritance lets a subclass reuse all of its parent's members, and it is only right when the sentence 'the child is a kind of the parent' holds in every situation. I use it for genuine type families, override with `@Override`, and if a child would have to disable a parent method I switch to composition, because that is a Liskov violation."

- *"Can you override a static method?"* — No. A static method with the same signature in the child **hides** the parent's; which one runs depends on the compile-time type (see Section 2.7 output).
- *"Can an override return a different type?"* — Only a subtype of the original (covariant return), e.g. `Circle copy()` overriding `Shape copy()`.

Self-check

- **Q:** Is `private` inherited? **A:** The field exists in the child object but the child cannot access it directly; private methods are not overridden.
- **Q:** Can an override reduce visibility from `public` to `protected`? **A:** No.

TL;DR

`extends` for true is-a families only. `super(...)` first, `@Override` always, never narrow access or disable parent behaviour.

2.6 Polymorphism (and the full bank example)

What it is

Polymorphism (Greek for "many forms") means **the same method call can do different things depending on the object it lands on**. The calling code works with a variable of the **parent type** and invokes a method by name; the concrete object behind that variable determines the behaviour. Analogy: a "start" button that starts a car, a washing machine or a laptop, each in its own way.

The problem it solves

Without it, code asks "what type are you?" everywhere: `if (a instanceof SavingsAccount) ... else if (a instanceof CheckingAccount) ...`. Every new type means finding and editing every such branch. With polymorphism, the branch disappears.

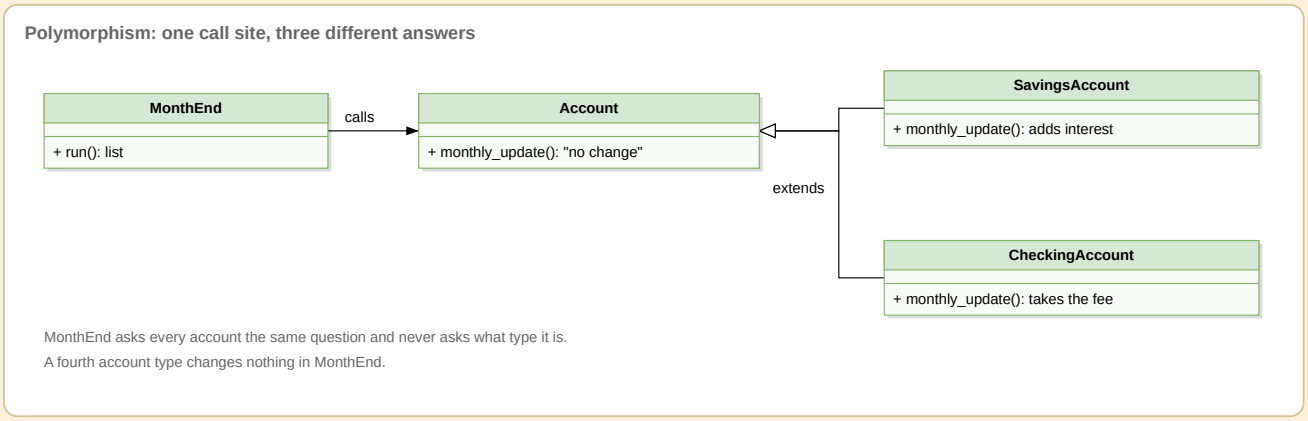


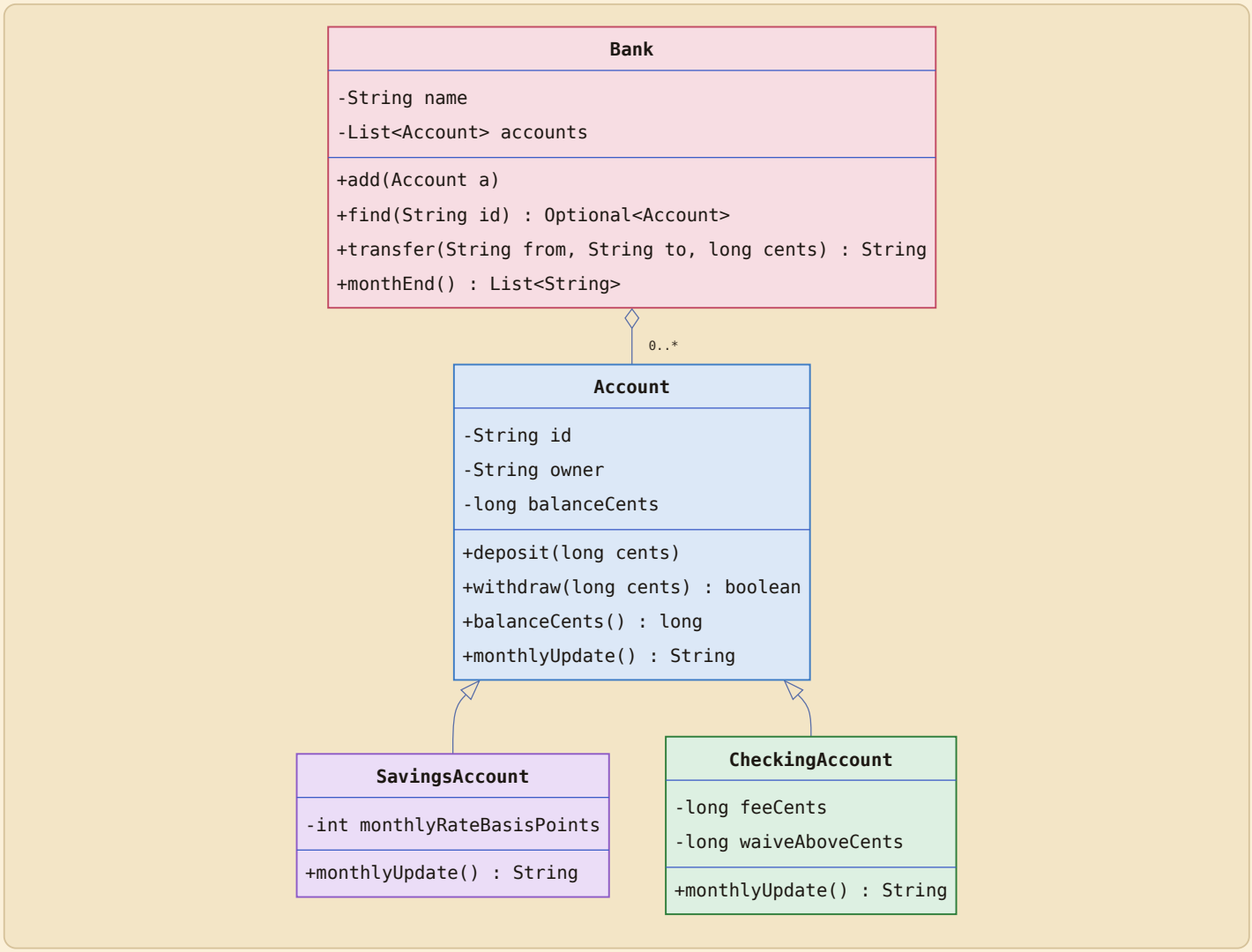
Figure — MonthEnd calls monthlyUpdate on every account and never asks which type it holds (course diagram)

In the figure, the month-end loop sends the identical request to each account and **never checks which kind it is**; it contains no `if` about account types. Introduce a fourth account type next month and the loop needs no edit.

How it works

1. Declare the method on the parent type (or interface).
2. Each child overrides it.
3. Callers hold the **parent type** (`Account a`) and call the method.
4. At run time (while the program is running) the JVM looks at the **actual class of the object** and runs that class's version: **dynamic dispatch** (also called late binding).

Java example: the bank (encapsulation + abstraction + inheritance + polymorphism)



```java
// File: code/lld/ch02/bank/Account.java
package lld.ch02.bank;

/**
 * ENCAPSULATION: the balance is private; the only way to change it is through deposit/withdraw,
 * and both enforce the rules. The parent also defines the hook monthlyUpdate() for POLYMORPHISM.
 */
public class Account {
    private final String id;
    private final String owner;
    private long balanceCents;                    // hidden: nobody outside can set it to -5

    public Account(String id, String owner, long openingCents) {
        if (openingCents < 0) throw new IllegalArgumentException("negative opening balance");
        this.id = id;
        this.owner = owner;
        this.balanceCents = openingCents;
    }

    public final void deposit(long cents) {         // final: children may not weaken the rule
        if (cents <= 0) throw new IllegalArgumentException("deposit must be positive");
        balanceCents += cents;
    }

    public final boolean withdraw(long cents) {
        if (cents <= 0 || cents > balanceCents) return false;   // the one place the rule lives
        balanceCents -= cents;
        return true;
    }

    public final long balanceCents() { return balanceCents; }
    public final String id()         { return id; }
    public final String owner()      { return owner; }

    /** Default month-end behaviour. Subclasses OVERRIDE it. */
    public String monthlyUpdate() { return "no change"; }
}
```

```java
// File: code/lld/ch02/bank/Bank.java
package lld.ch02.bank;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/**
 * ABSTRACTION: Bank uses only id(), withdraw() and deposit(). It does not know how a balance is
 * stored, what rules an account applies, or which subclass it holds.
 */
public class Bank {
    private final String name;
    private final List<Account> accounts = new ArrayList<>();

    public Bank(String name) { this.name = name; }

    public void add(Account a) { accounts.add(a); }

    public Optional<Account> find(String id) {
        return accounts.stream().filter(a -> a.id().equals(id)).findFirst();
    }

    public String transfer(String fromId, String toId, long cents) {
        Account from = find(fromId).orElseThrow();
        Account to = find(toId).orElseThrow();
        if (!from.withdraw(cents)) return fromId + " rejected, not enough money";
        to.deposit(cents);
        return "moved " + Money.format(cents) + " from " + fromId + " to " + toId;
    }

    /** POLYMORPHISM: one call, a different result per object, and no 'if' on the account type. */
    public List<String> monthEnd() {
        List<String> lines = new ArrayList<>();
        for (Account a : accounts) {
            String what = a.monthlyUpdate();                // dynamic dispatch picks the override
            lines.add(a.id() + " (" + a.owner() + "): " + what + ", balance " + Money.format(a.balanceCents()));
        }
        return lines;
    }

    public List<Account> accounts() { return List.copyOf(accounts); }
    public String name() { return name; }
}
```

```java
// File: code/lld/ch02/bank/CheckingAccount.java
package lld.ch02.bank;

/** A checking account charges a monthly fee unless the balance is above a threshold. */
public class CheckingAccount extends Account {
    private final long feeCents;
    private final long waiveAboveCents;

    public CheckingAccount(String id, String owner, long openingCents, long feeCents, long waiveAboveCents) {
        super(id, owner, openingCents);
        this.feeCents = feeCents;
        this.waiveAboveCents = waiveAboveCents;
    }

    @Override
    public String monthlyUpdate() {
        if (balanceCents() >= waiveAboveCents) return "fee waived";
        return withdraw(feeCents) ? "fee " + Money.format(feeCents) : "fee could not be taken";
    }
}
```

```java
// File: code/lld/ch02/bank/Money.java
package lld.ch02.bank;

/** Utility: formats whole cents as dollars. Private constructor = cannot be instantiated. */
public final class Money {
    private Money() { }
    public static String format(long cents) { return String.format("$%d.%02d", cents / 100, cents % 100); }
}
```

```java
// File: code/lld/ch02/bank/SavingsAccount.java
package lld.ch02.bank;

/** INHERITANCE: a savings account IS an account; it adds a rate and replaces one method. */
public class SavingsAccount extends Account {
    private final int monthlyRateBasisPoints;      // 25 bp = 0.25 %

    public SavingsAccount(String id, String owner, long openingCents, int monthlyRateBasisPoints) {
        super(id, owner, openingCents);            // parent constructor runs first
        this.monthlyRateBasisPoints = monthlyRateBasisPoints;
    }

    @Override
    public String monthlyUpdate() {
        long interest = balanceCents() * monthlyRateBasisPoints / 10_000;
        if (interest > 0) deposit(interest);
        return "interest " + Money.format(interest);
    }
}
```

```java
// File: code/lld/ch02/bank/Main.java
package lld.ch02.bank;

public class Main {
    public static void main(String[] args) {
        Bank bank = new Bank("Gurus Bank");
        bank.add(new SavingsAccount("S-1", "Ada", 50_000, 25));
        bank.add(new CheckingAccount("C-1", "Kai", 8_000, 250, 50_000));
        bank.add(new Account("P-1", "Ravi", 3_000));

        System.out.println(bank.name() + ", opening balances:");
        for (Account a : bank.accounts()) {
            System.out.println("  " + a.id() + " (" + a.owner() + "): " + Money.format(a.balanceCents()));
        }
        System.out.println(bank.transfer("S-1", "C-1", 10_000));
        System.out.println(bank.transfer("S-1", "C-1", 90_000));
        System.out.println("month end:");
        bank.monthEnd().forEach(line -> System.out.println("  " + line));
    }
}
```

Output:

```
Gurus Bank, opening balances:
  S-1 (Ada): $500.00
  C-1 (Kai): $80.00
  P-1 (Ravi): $30.00
moved $100.00 from S-1 to C-1
S-1 rejected, not enough money
month end:
  S-1 (Ada): interest $1.00, balance $401.00
  C-1 (Kai): fee $2.50, balance $177.50
  P-1 (Ravi): no change, balance $30.00
```

Explaining each part:

- **Encapsulation:** `balanceCents` is private; `deposit`/`withdraw` are `final` and enforce the rules. The second transfer ($900 from an account holding $400) is refused inside `withdraw`.
- **Abstraction:** `Bank.transfer` only uses `id()`, `withdraw()`, `deposit()`.
- **Inheritance:** each child calls `super(...)` and adds one or two fields.
- **Polymorphism:** `monthEnd()` calls `a.monthlyUpdate()`: three accounts, one method name, three results (interest 0.25% of $400 = $1.00; checking balance $180 is below the $500 waiver so $2.50 is charged; the plain account uses the parent's "no change"). **No `if` asks which is which.**
- **Why `long` cents and not `BigDecimal` here:** money must never be a `double` (binary floating point cannot store 0.10 exactly, so sums drift). A `long` count of whole **cents** is exact too, and keeps this example about the four pillars rather than about arithmetic. Its limit is that it cannot hold a fraction of a cent: `balanceCents() * monthlyRateBasisPoints / 10_000` uses integer division, which **truncates** (rounds toward zero). With 40,000 cents at 25 bp the result is exactly 100 cents, but for 40,001 cents the 0.0025-cent remainder would be dropped. When a domain needs fractional amounts or an explicit rounding rule (interest, tax, currency conversion), use `BigDecimal` (Java's exact decimal-number type) with a stated `RoundingMode`, as the `Money` value object (a small object defined only by its values, Section 2.12) in Section 2.13 does. Interest is in **basis points** (1 bp = 0.01%).

How the four fit together

Encapsulation and abstraction guard a class: they settle **what is hidden** and what outsiders may see. Inheritance and polymorphism let the design expand: they make **a new requirement show up as an added class** rather than as surgery on an existing one. Designs that apply all four tend to look alike: hidden state, a small public API (application programming interface: the set of methods other code is allowed to call), a parent for the common parts, and subclasses that each override one method.

Trade-offs

Benefit	Cost
Removes type switches; open to new types	Behaviour is spread over several classes; you must look at the runtime type to know what runs
Enables Strategy, State, Command and most patterns (named, reusable designs; Chapters 8–11)	A virtual call (one resolved at run time) is marginally slower than a direct call (almost never matters)

Common mistakes

- Long `if/else` or `switch` over a type: almost always where polymorphism belongs. **Pointing this out to the interviewer earns more credit than actually refactoring (restructuring without changing behaviour) it.**
- Using `instanceof` chains to decide behaviour (fine for occasional data inspection, e.g. pattern matching in Java 21, which tests a type and binds a variable in one step, but not as the main dispatch mechanism in an OO model).

Real-world examples

`List.add` behaves differently for `ArrayList` and `LinkedList`; `Runnable.run()` executed by any `Executor` (a Java object that runs tasks, often on background threads); Spring injecting (handing in from outside) any implementation of an interface; `Comparator.compare` in `Collections.sort`.

In the interview

> **30-second answer:** "Polymorphism lets one call behave differently depending on the object receiving it. I code against the parent type or an interface, each subtype overrides the method, and the JVM picks the right one at run time. It removes type switches, so a new type is a new class, not an edit."

- *"Where would you use it in a parking lot?"* — Pricing (`PricingStrategy.feeCents`), payments (`Payment.pay`), spot selection.

Self-check

- **Q:** Which version of `monthlyUpdate` runs for `Account a = new SavingsAccount(...)`? **A:** `SavingsAccount`'s, chosen at run time.

TL;DR

Hold the parent type, call the method, let the object decide. Type switches are the signal that polymorphism is missing.

2.7 Compile-time vs runtime polymorphism: overloading vs overriding

What it is

	Overloading (compile-time / static polymorphism)	**Overriding (runtime / dynamic polymorphism)**
What	Same method name, **different parameter lists**, usually in the same class	Same signature re-implemented in a **subclass**
Decided by	The **compiler**, using the **declared (static) types** of the arguments	The **JVM at run time**, using the **actual class** of the receiver object
Return type	May differ (but cannot be the only difference)	Same or covariant
Needs inheritance?	No	Yes (class or interface)
Example	`add(int,int)`, `add(double,double)`	`Circle.area()` overriding `Shape.area()`

How it works: overload resolution

The compiler picks an overload in three phases and stops at the first phase that finds a match:

1. Exact match or **widening** (automatic conversion to a bigger or more general type) of primitives/references (`int` → `long` → `double`, `Circle` → `Shape`), no boxing, no varargs.
2. Allow **boxing/unboxing** (`int` ↔ `Integer`: wrapping a primitive value in its object form, and back).
3. Allow **varargs** (`int...`: a parameter that accepts any number of arguments). If several candidates remain in a phase, the **most specific** wins; if none is most specific, it is a compile error (ambiguous call).

Not polymorphic: fields (resolved by the static type; a child field with the same name **hides** the parent's), `static` methods (hidden, chosen by the static type), and `private` methods.

Java example

```java
// File: code/lld/ch02/polymorphism/Calculator.java
package lld.ch02.polymorphism;

/** OVERLOADING: same name, different parameter lists. Chosen by the COMPILER from static types. */
public class Calculator {
    public String add(int a, int b)       { return "add(int,int) = " + (a + b); }
    public String add(long a, long b)     { return "add(long,long) = " + (a + b); }
    public String add(double a, double b) { return "add(double,double) = " + (a + b); }
    public String add(int... xs)          { int s = 0; for (int x : xs) s += x; return "add(int...) = " + s; }

    public String greet(Shape s)  { return "greet(Shape)"; }
    public String greet(Circle c) { return "greet(Circle)"; }
}
```

```java
// File: code/lld/ch02/polymorphism/Circle.java
package lld.ch02.polymorphism;

public class Circle extends Shape {
    public String kind = "circle";                              // HIDES Shape.kind, does not override it
    private final double r;

    public Circle(double r) {
        super();                                                // implicit anyway; parent constructed first
        this.r = r;
    }

    @Override public double area() { return Math.PI * r * r; }  // OVERRIDE: same signature

    @Override public Circle copy() { return new Circle(r); }    // covariant return type is allowed

    public static String describe() { return "static Circle.describe"; }

    @Override public String who() { return super.who() + " (radius " + r + ")"; }  // super.method() reuses parent
code
}
```

```java
// File: code/lld/ch02/polymorphism/Shape.java
package lld.ch02.polymorphism;

public class Shape {
    public String kind = "shape";                               // fields are NOT polymorphic

    public double area() { return 0; }

    public Shape copy() { return new Shape(); }

    public static String describe() { return "static Shape.describe"; }   // static: hidden, not overridden

    public String who() { return "I am a " + kind + " with area " + String.format("%.2f", area()); }
}
```

```java
// File: code/lld/ch02/polymorphism/Main.java
package lld.ch02.polymorphism;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(2, 3));           // exact match int,int
        System.out.println(calc.add(2L, 3));          // widening to long,long
        System.out.println(calc.add(2.5, 3));         // widening to double,double
        System.out.println(calc.add(1, 2, 3));        // varargs, chosen last

        Shape s = new Circle(1.0);                    // static type Shape, runtime type Circle
        System.out.println("s.area() -> " + String.format("%.2f", s.area()) + "  (runtime override: dynamic
dispatch)");
        System.out.println("calc.greet(s) -> " + calc.greet(s) + "  (overload picked at compile time from static
type)");
        System.out.println("s.kind -> " + s.kind + "  (fields use the static type)");
        System.out.println("((Circle) s).kind -> " + ((Circle) s).kind);
        System.out.println("Shape.describe() -> " + Shape.describe() + ", Circle.describe() -> " +
Circle.describe());
        System.out.println("s.copy() class -> " + s.copy().getClass().getSimpleName());
        System.out.println(s.who());

        List<Shape> shapes = List.of(new Shape(), new Circle(2), new Circle(0.5));
        double total = 0;
        for (Shape sh : shapes) total += sh.area();     // one call site, many behaviours
        System.out.println("total area = " + String.format("%.2f", total));

        if (s instanceof Circle c) {                    // pattern matching instanceof (Java 16+)
            System.out.println("pattern-matched a circle: " + c.who());
        }
    }
}
```

Output:

```
add(int,int) = 5
add(long,long) = 5
add(double,double) = 5.5
add(int...) = 6
s.area() -> 3.14  (runtime override: dynamic dispatch)
calc.greet(s) -> greet(Shape)  (overload picked at compile time from static type)
s.kind -> shape  (fields use the static type)
((Circle) s).kind -> circle
Shape.describe() -> static Shape.describe, Circle.describe() -> static Circle.describe
s.copy() class -> Circle
I am a shape with area 3.14 (radius 1.0)
total area = 13.35
pattern-matched a circle: I am a shape with area 3.14 (radius 1.0)
```

Explaining each part:

- `add(2L, 3)`: no `(long,int)` overload, so `3` widens to `long` (phase 1).
- `s.area()` runs `Circle.area()` although `s` is declared `Shape`: **dynamic dispatch**.
- `calc.greet(s)` prints `greet(Shape)`, not `greet(Circle)`: overloads are chosen **at compile time from the declared type** of `s`. This surprises many people and is a favourite interview question.
- `s.kind` is `"shape"`: fields are not polymorphic. Inside `Shape.who()`, `kind` also means `Shape.kind`, which is why the output says "I am a shape". Lesson: **never hide fields**; use private fields plus overridable methods.
- `Circle copy()` overrides `Shape copy()` with a **covariant return type**.
- `super.who()` reuses the parent's text and appends the radius.
- `s instanceof Circle c` is **pattern matching for instanceof** (Java 16+): test and cast (treat the value as the more specific type) in one step.

Common mistakes

- Expecting overloads to be picked by runtime type (they are not; that would require double dispatch, choosing a method by the runtime types of two objects; see the Visitor pattern in Chapter 11).
- Writing `public boolean equals(Point p)`: an **overload**, not an override of `equals(Object)`, so collections (Java's built-in containers such as `List`, `Set` and `Map`) never call it. `@Override` would have caught it.

In the interview

> **30-second answer:** "Overloading is compile-time: same name, different parameters, chosen by declared types. Overriding is runtime: a subclass re-implements the same signature and the JVM dispatches on the actual object. Fields and static methods are not polymorphic."

- *"Why does `greet(s)` call `greet(Shape)`?"* — Overload resolution uses the static type of `s`, which is `Shape`.

Self-check

- **Q:** Can two methods differ only in return type? **A:** No; that is a compile error.
- **Q:** Which phase allows varargs? **A:** The third (last).

TL;DR

Overloading = compiler picks by declared types. Overriding = JVM picks by actual object. Fields and statics never dispatch dynamically.

2.8 Interfaces (default, static, private methods, functional interfaces)

What it is

An **interface** is a **contract**: a named list of methods a class promises to provide, without (necessarily) saying how. `class EmailNotifier implements Notifier`. Analogy: a wall socket standard. Any appliance with the right plug works with any socket, whoever made them.

Key facts:

- Abstract methods are implicitly `public abstract`; fields are implicitly `public static final` (constants).
- A class can **implement many interfaces** (but extend only one class).
- **Default methods** (Java 8+): a method with a body that implementers inherit and may override. Added so interfaces can evolve without breaking every implementer (for example `Collection.stream()`, `Iterable.forEach`).
- **Static methods** (Java 8+): belong to the interface; called as `Notifier.console()`; **not inherited** by implementing classes.
- **Private methods** (Java 9+): helpers shared by default methods, invisible to implementers.
- A **functional interface** has **exactly one abstract method** (methods inherited from `Object`, like `equals`, do not count; default and static methods do not count). It can be implemented by a **lambda** (a short unnamed function written inline, such as `(to, msg) -> ...`) or **method reference** (a pointer to an existing method, such as `String::isBlank`). `@FunctionalInterface` asks the compiler to enforce the rule.

The problem it solves

Callers depend on **what** can be done, not on a concrete class. New implementations can be added without touching callers (open/closed: open to extension, closed to modification), and tests can pass fakes, simple stand-in objects (dependency inversion: depend on interfaces, not concrete classes; both are covered in Chapter 4). Interfaces also give Java a safe form of multiple inheritance (one class inheriting from several parents; here of **type**, not state).

How it works

1. Define the interface with the smallest set of methods a caller needs.
2. Implement it in one or more classes (or with a lambda if it is functional).
3. Callers declare variables and parameters of the interface type.

4. If two interfaces supply the same default method, the class **must** override it and may pick one with `A.super.method()`. If a superclass declares the method, **the class wins** over interface defaults.

Java example

```java
// File: code/lld/ch02/interfaces/DiscountRule.java
package lld.ch02.interfaces;

/** A FUNCTIONAL interface has exactly one abstract method, so a lambda can implement it. */
@FunctionalInterface
public interface DiscountRule {
    long apply(long priceCents);

    default DiscountRule then(DiscountRule next) { return p -> next.apply(apply(p)); }  // defaults are allowed
}
```

```java
// File: code/lld/ch02/interfaces/EmailNotifier.java
package lld.ch02.interfaces;

public class EmailNotifier implements Notifier {
    @Override public String send(String to, String message) { return "[email] to " + to + ": " + message; }
}
```

```java
// File: code/lld/ch02/interfaces/Notifier.java
package lld.ch02.interfaces;

/** An interface is a contract: WHAT can be done, not HOW. */
public interface Notifier {
    String send(String to, String message);        // abstract method: every implementer must provide it

    /** default method: implementers inherit it and may override it (added in Java 8). */
    default String sendAll(java.util.List<String> recipients, String message) {
        StringBuilder sb = new StringBuilder();
        for (String r : recipients) sb.append(send(r, clean(message))).append('\n');
        return sb.toString().stripTrailing();
    }

    /** static method: belongs to the interface itself; called as Notifier.console(). */
    static Notifier console() { return (to, msg) -> "[console] " + to + ": " + msg; }

    /** private method (Java 9+): a helper shared by default methods, hidden from implementers. */
    private String clean(String message) { return message.strip(); }
}
```

```java
// File: code/lld/ch02/interfaces/Report.java
package lld.ch02.interfaces;

/**
 * An ABSTRACT class can hold state and constructors, and mix finished and unfinished methods.
 * Here it fixes the skeleton of an algorithm (Template Method) and lets subclasses fill the steps.
 */
public abstract class Report {
    private final String title;                    // state: interfaces cannot have instance fields

    protected Report(String title) { this.title = title; }

    public final String render() {                 // final: subclasses cannot change the skeleton
        return header() + "\n" + body() + "\n-- end of " + title + " --";
    }

    protected String header() { return "== " + title + " =="; }   // concrete, overridable
    protected abstract String body();                              // must be supplied
}
```

```java
// File: code/lld/ch02/interfaces/SalesReport.java
package lld.ch02.interfaces;

public class SalesReport extends Report {
    private final long totalCents;
    public SalesReport(long totalCents) { super("Sales"); this.totalCents = totalCents; }
    @Override protected String body() { return "total: " + totalCents + " cents"; }
}
```

```java
// File: code/lld/ch02/interfaces/Main.java
package lld.ch02.interfaces;

import java.util.List;
import java.util.function.Predicate;

public class Main {
    public static void main(String[] args) {
        Notifier email = new EmailNotifier();                // class implementing the interface
        Notifier sms = (to, msg) -> "[sms] " + to + ": " + msg; // Notifier is functional too (one abstract method)
        for (Notifier n : List.of(email, sms, Notifier.console())) {
            System.out.println(n.send("ada", "your order shipped"));
        }
        System.out.println(email.sendAll(List.of("kai", "ravi"), "  sale starts today  "));

        DiscountRule tenPercent = p -> p * 90 / 100;
        DiscountRule minusFiveDollars = p -> Math.max(0, p - 500);
        System.out.println("10% then -$5 on 10000c = " + tenPercent.then(minusFiveDollars).apply(10_000) + "c");

        Predicate<String> blank = String::isBlank;             // JDK functional interface + method reference
        System.out.println("'  ' blank? " + blank.test("  "));

        Report r = new SalesReport(123_45);
        System.out.println(r.render());
    }
}
```

Output:

```
[email] to ada: your order shipped
[sms] ada: your order shipped
[console] ada: your order shipped
[email] to kai: sale starts today
[email] to ravi: sale starts today
10% then -$5 on 10000c = 8500c
'  ' blank? true
== Sales ==
total: 12345 cents
-- end of Sales --
```

Explaining each part: `Notifier` has one abstract method, so it is also usable with lambdas (`sms`). `sendAll` is a default method that uses the private helper `clean`. `Notifier.console()` is a static factory on the interface. `DiscountRule` is an explicit `@FunctionalInterface` with a default combinator (a method that joins two functions into one) `then` (like `Function.andThen`). `Predicate<String>` is one of the JDK's built-in functional interfaces (`java.util.function`: `Function`, `Supplier`, `Consumer`, `Predicate`, `BiFunction`, `UnaryOperator`...). `Report`/`SalesReport` show an **abstract class** (Section 2.9).

Trade-offs

Benefit	Cost
Loose coupling (classes know little about each other), easy testing, multiple implementations	One more type to read; pointless if only one implementation will ever exist
Multiple inheritance of type	Cannot hold instance state
Default methods allow evolution	Overusing defaults turns interfaces into mini base classes

When not to use it

An interface with a single implementation and no realistic second one (and no test-double need, i.e. no fake version for tests) is indirection (an extra layer between caller and code) without benefit. Add it when the second implementation appears.

Common mistakes

- **Fat interfaces** that force implementers to leave methods empty (Interface Segregation, Chapter 4).
- "Constant interfaces" used only to hold constants (use a final class or enum).

Real-world examples

`Comparable`, `Comparator`, `Runnable`, `Callable`, `Iterable`, `AutoCloseable` (enables try-with-resources, a `try (...)` block that closes files or connections automatically), `List`/`Map`; Spring's `Repository` interfaces; SLF4J's (a common Java logging library) `Logger`.

In the interview

> **30-second answer:** "An interface is a contract of methods. I make services depend on interfaces such as `PaymentGateway` or `PricingStrategy`, so I can add implementations without editing callers and test with fakes. Since Java 8 interfaces can have default and static methods, and an interface with one abstract method is functional, so a lambda can implement it."

- *"Why were default methods added?"* — To add methods to widely-implemented interfaces (like `Collection.stream`) without breaking existing implementations.
- *"Two interfaces with the same default method?"* — Compile error until the class overrides it; it can delegate with `X.super.m()`.

Self-check

- **Q:** Is `Comparator` functional although it has many methods? **A:** Yes; only `compare` is abstract (`equals` comes from `Object`, the rest are default/static).

TL;DR

Interfaces are contracts; depend on them. Defaults evolve them, statics provide factories/helpers, one abstract method = lambda-friendly.

2.9 Abstract classes vs interfaces

What it is

An **abstract class** is a class marked `abstract` that **cannot be instantiated**. It may contain fields, constructors, finished methods and **abstract methods** (declared without a body) that subclasses must implement. Analogy: a form with some fields pre-printed and some left blank for each department to fill in.

How it works

In the `interfaces` package above, `Report` holds state (`title`), has a constructor, a `final` skeleton method `render()`, a concrete overridable `header()`, and an abstract `body()`. `SalesReport` supplies only `body()`. This "fixed skeleton, variable steps" shape is the **Template Method** pattern (Chapter 11).

	Abstract class	Interface
Instantiable?	No	No
Instance fields (state)	Yes	No (only `public static final` constants)
Constructors	Yes	No
Method bodies	Any	`default`, `static`, `private` only
Access modifiers on members	Any	Abstract/default/static are public; helpers can be private
How many can a class extend/implement?	**One**	**Many**
Relationship expressed	"is a" (shared identity and state)	"can do" (capability / role)
Evolution	Add concrete methods freely	Add `default` methods
Typical use	Shared state + partial implementation for a close family	Contracts, roles, strategies, test seams (points where a test can swap in a fake)

Rule of thumb

Start with an **interface** for the contract. Add an **abstract class** underneath when several implementations share state or a fixed algorithm (JDK example: `List` interface + `AbstractList` skeleton + `ArrayList`). Callers still depend on the interface.

Common mistakes

- Using an abstract class just to share one helper method (use composition or a static helper).
- Making everything abstract "for flexibility".

In the interview

> **30-second answer:** "Interfaces define capabilities and allow multiple implementation; abstract classes share state and partial implementation but allow only one parent. I default to interfaces and add an abstract base class when implementations share real state or a template algorithm."

Self-check

- **Q:** Can an abstract class have no abstract methods? **A:** Yes; it just cannot be instantiated.
- **Q:** Can an interface have a constructor? **A:** No.

TL;DR

Interface = contract, many allowed, no state. Abstract class = partial implementation with state, one allowed. Interface first; abstract base class when sharing real code.

2.10 Enums with fields and behaviour

What it is

An **enum** (enumerated type) is a type with a **fixed set of named instances**, such as `Coin.PENNY` or `OrderStatus.SHIPPED`. In Java an enum is a full class: it can have **fields, constructors, methods, implement interfaces**, and even give each constant its own method body. Analogy: the fixed denominations of coins; you cannot invent a 7-cent coin.

The problem it solves

String or int constants (`"PAID"`, `2`) allow typos and invalid values and carry no behaviour. Enums are **type-safe** (only valid values compile), readable, and can hold the data and rules that belong to each value, which removes `switch` statements scattered around the code. In LLD, enums are the natural home for "**different data, not different behaviour**" (spot types, vehicle types, coin values, statuses).

How it works

1. Every enum implicitly extends `java.lang.Enum` and cannot extend anything else; it can implement interfaces.
2. Constructors are always private; constants are created once when the enum class is initialised.
3. Built-ins: `values()`, `valueOf(String)` (throws `IllegalArgumentException` for unknown names), `name()`, `ordinal()` (position, 0-based; do **not** persist it, i.e. save it to a database or file, since reordering changes it), `compareTo` (by ordinal).
4. Compare with `==` (each constant is a single instance).
5. Constant-specific bodies (`PAID { ... }`) let each constant override an abstract method: a tiny state machine (a fixed set of states plus the allowed moves between them) or strategy.
6. `EnumMap` / `EnumSet` are compact, fast collections keyed by enums.
7. `switch` expressions over an enum that cover every constant are **exhaustive**: no `default` needed, and adding a constant makes the compiler flag missing cases.
8. A single-element enum is the simplest thread-safe, serialization-safe (serialization turns an object into bytes to store or send) **Singleton** (a class allowed exactly one instance; Chapter 8).

Java example

```java
// File: code/lld/ch02/enums/Coin.java
package lld.ch02.enums;

/** An enum with a FIELD, a constructor and a method. Enum constructors are always private. */
public enum Coin {
    PENNY(1), NICKEL(5), DIME(10), QUARTER(25);

    private final int cents;

    Coin(int cents) { this.cents = cents; }

    public int cents() { return cents; }
}
```

```java
// File: code/lld/ch02/enums/OrderStatus.java
package lld.ch02.enums;

import java.util.Set;

/** An enum with BEHAVIOUR: each constant says which statuses it may move to (a tiny state machine). */
public enum OrderStatus {
    PLACED    { @Override Set<OrderStatus> next() { return Set.of(PAID, CANCELLED); } },
    PAID      { @Override Set<OrderStatus> next() { return Set.of(SHIPPED, CANCELLED); } },
    SHIPPED   { @Override Set<OrderStatus> next() { return Set.of(DELIVERED); } },
    DELIVERED { @Override Set<OrderStatus> next() { return Set.of(); } },
    CANCELLED { @Override Set<OrderStatus> next() { return Set.of(); } };

    abstract Set<OrderStatus> next();          // constant-specific method bodies

    public boolean canMoveTo(OrderStatus target) { return next().contains(target); }
    public boolean isFinal() { return next().isEmpty(); }
}
```

```java
// File: code/lld/ch02/enums/Main.java
package lld.ch02.enums;

import java.util.EnumMap;
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        List<Coin> inserted = List.of(Coin.QUARTER, Coin.QUARTER, Coin.DIME, Coin.PENNY);
        int total = inserted.stream().mapToInt(Coin::cents).sum();
        System.out.println("inserted " + inserted + " = " + total + " cents");

        Map<Coin, Integer> counts = new EnumMap<>(Coin.class);   // compact, ordered by declaration
        for (Coin c : inserted) counts.merge(c, 1, Integer::sum);
        System.out.println("counts: " + counts);

        System.out.println("valueOf(\"DIME\") = " + Coin.valueOf("DIME") + ", ordinal " + Coin.DIME.ordinal());

        OrderStatus s = OrderStatus.PLACED;
        for (OrderStatus target : List.of(OrderStatus.SHIPPED, OrderStatus.PAID, OrderStatus.SHIPPED,
OrderStatus.DELIVERED)) {
            if (s.canMoveTo(target)) { s = target; System.out.println("moved to " + s); }
            else System.out.println("refused " + s + " -> " + target);
        }
        System.out.println(s + " is final? " + s.isFinal());

        String label = switch (s) {                          // exhaustive switch: no default needed
            case PLACED, PAID -> "in progress";
            case SHIPPED -> "on the way";
            case DELIVERED -> "done";
            case CANCELLED -> "stopped";
        };
        System.out.println("label: " + label);
    }
}
```

Output:

```
inserted [QUARTER, QUARTER, DIME, PENNY] = 61 cents
counts: {PENNY=1, DIME=1, QUARTER=2}
valueOf("DIME") = DIME, ordinal 2
refused PLACED -> SHIPPED
moved to PAID
moved to SHIPPED
moved to DELIVERED
DELIVERED is final? true
label: done
```

Explaining each part: `Coin` stores its value in a field, so summing is `Coin::cents`, with no switch. `EnumMap` prints in declaration order. `OrderStatus` gives each constant its own `next()` set, so illegal transitions (`PLACED → SHIPPED`) are refused in one method. This is the "enum plus a guarded method" alternative to the full State pattern that Chapter 1 recommends for small lifecycles.

Trade-offs

Benefit	Cost
Type-safe, self-documenting, can hold data/behaviour	Closed set: adding a value means recompiling
Great for small state machines and lookup tables	Large per-constant behaviour becomes hard to read; move to State/Strategy classes

When not to use it

When values are **open-ended or user-defined at run time** (product categories created by an admin) or when each value needs substantial, independently evolving behaviour (use classes).

Common mistakes

- Persisting `ordinal()` in a database (store `name()` or an explicit code field).
- Giant `switch` statements over an enum in many places instead of a method on the enum.

Real-world examples

`java.time.DayOfWeek` and `Month` (with methods like `plus`), `java.util.concurrent.TimeUnit` (each constant converts units: `TimeUnit.SECONDS.toMillis(5)`), `java.nio.file.StandardOpenOption`, `Thread.State`.

In the interview

> **30-second answer:** "I use enums for fixed sets like vehicle types, spot types and statuses. Java enums can carry fields and methods, so data like 'which spots fit this vehicle' or 'which statuses can follow this one' lives on the enum rather than in switches around the code."

- *"Enum or class hierarchy for vehicle types?"* — Enum if they differ only in data; classes if they behave differently.

Self-check

- **Q:** What does `Coin.valueOf("dime")` do? **A:** Throws `IllegalArgumentException` (names are case-sensitive).

TL;DR

Enums are type-safe fixed sets that can carry data and behaviour. Use them for "different data, same behaviour"; compare with `==`; never persist ordinals.

2.11 Composition over inheritance

What it is

Composition means building a class by **holding references to other objects** and delegating work to them ("has-a"), rather than inheriting from them ("is-a"). The principle **"favour composition over inheritance"** comes from the Gang of Four book and is a core idea in *Head First Design Patterns*. Analogy: a phone **has** a camera module; it is not a kind of camera. The module can be upgraded without redesigning the phone.

The problem it solves

Inheritance couples a child to the **internal implementation** of its parent (the **fragile base class problem**): a change in the parent, or a hidden detail of how it works, can silently break the child. Inheritance is also fixed at compile time and single-parent. Composition keeps classes independent, lets you **swap parts at run time**, and combine behaviours freely.

How it works

1. Identify the behaviour you want to reuse or vary.
2. Put it behind an interface (`Engine`, `Set`).
3. Hold it in a private field, received through the constructor (so it can be replaced or faked).
4. **Forward** (delegate) calls to it, adding your own logic around them.

Java example

```java
// File: code/lld/ch02/composition/Car.java
package lld.ch02.composition;

/** A car IS NOT an engine; it HAS an engine. The part can be swapped at run time. */
public class Car {
    private final String model;
    private Engine engine;

    public Car(String model, Engine engine) { this.model = model; this.engine = engine; }

    public void replaceEngine(Engine e) { this.engine = e; }

    public String drive() { return model + ": " + engine.start(); }
}
```

```java
// File: code/lld/ch02/composition/CountingSet.java
package lld.ch02.composition;

import java.util.Collection;
import java.util.Set;

/**
 * FIXED with composition: HAS-A Set and forwards to it. Works with ANY Set implementation
 * and does not care how that set implements addAll internally.
 */
public class CountingSet<E> {
    private final Set<E> inner;
    private int attempts = 0;

    public CountingSet(Set<E> inner) { this.inner = inner; }

    public boolean add(E e) { attempts++; return inner.add(e); }

    public boolean addAll(Collection<? extends E> c) {
        attempts += c.size();
        boolean changed = false;
        for (E e : c) changed |= inner.add(e);     // talk to the inner set, not to our own add()
        return changed;
    }

    public int size()     { return inner.size(); }
    public int attempts() { return attempts; }
}
```

```java
// File: code/lld/ch02/composition/Engine.java
package lld.ch02.composition;

public interface Engine {
    String start();
}
```

```java
// File: code/lld/ch02/composition/InheritingCountingSet.java
package lld.ch02.composition;

import java.io.Serial;
import java.util.Collection;
import java.util.HashSet;

/**
 * BROKEN on purpose: counts attempted additions by EXTENDING HashSet.
 * HashSet.addAll is inherited from AbstractCollection, which calls add() once per element,
 * so every element of addAll is counted twice. The subclass silently depends on how the parent
 * uses its own methods internally ("self-use"): the "fragile base class" problem.
 */
public class InheritingCountingSet<E> extends HashSet<E> {
    @Serial private static final long serialVersionUID = 1L;   // HashSet is Serializable, so subclasses should declare one

    private int attempts = 0;

    @Override public boolean add(E e) { attempts++; return super.add(e); }

    @Override public boolean addAll(Collection<? extends E> c) { attempts += c.size(); return super.addAll(c); }

    public int attempts() { return attempts; }
}
```

```java
// File: code/lld/ch02/composition/Main.java
package lld.ch02.composition;

import java.util.List;
import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        InheritingCountingSet<String> bad = new InheritingCountingSet<>();
        bad.addAll(List.of("a", "b", "c"));
        System.out.println("inheritance: added 3, attempts counted = " + bad.attempts());

        CountingSet<String> good = new CountingSet<>(new TreeSet<>());
        good.addAll(List.of("a", "b", "c"));
        good.add("a");
        System.out.println("composition: 4 attempts counted = " + good.attempts() + ", size = " + good.size());

        Car car = new Car("City", () -> "petrol engine vroom");
        System.out.println(car.drive());
        car.replaceEngine(() -> "electric motor hum");
        System.out.println(car.drive());
    }
}
```

Output:

```
inheritance: added 3, attempts counted = 6
composition: 4 attempts counted = 4, size = 3
City: petrol engine vroom
City: electric motor hum
```

Explaining each part: `InheritingCountingSet` counts 6 for 3 elements because `HashSet` inherits `addAll` from `AbstractCollection`, which calls `add` once per element, and `add` is overridden to count again. The subclass depended on **how the parent uses its own methods internally** ("self-use"). `AbstractCollection` describes this in an implementation note, but `HashSet` does not promise to keep it, so a future version could change it and silently fix or break the count. `CountingSet` **has a** `Set`, forwards to it, and works with any `Set` implementation (here a `TreeSet`). `Car` has an `Engine`, and the engine is swapped at run time, which inheritance cannot do.

Trade-offs

	Inheritance	Composition
Relationship	is-a	has-a
Coupling	Tight (to parent internals)	Loose (to an interface)
Change at run time	No	Yes
Boilerplate (repetitive filler code)	Low	Forwarding methods
Polymorphism	Built in	Via the interface the parts implement

When not to use it

When there is a **genuine is-a relationship** designed for extension (e.g. extending an abstract template class meant to be subclassed), inheritance is fine and simpler.

Common mistakes

- Extending a concrete library class (`HashMap`, `ArrayList`) to add a feature.
- Composing but then exposing the inner object directly (`getInner()`), losing control again.

Real-world examples

`java.io.BufferedReader` wraps any `Reader` (Decorator, a wrapper that adds behaviour around an object = composition); `Collections.unmodifiableList` wraps a list; Spring beans (objects the framework creates) are composed via dependency injection (the framework hands each object the collaborators it needs); strategy objects such as `Comparator` passed into `TreeMap`.

In the interview

> **30-second answer:** "I prefer composition: a class holds an interface-typed collaborator and delegates to it. It avoids the fragile base class problem, lets me swap behaviour at run time, and most patterns such as Strategy, Decorator and State are built on it. I use inheritance only for true is-a relationships."

- *"Show a case where inheritance breaks."* — The counting set above: overriding `add` and `addAll` double counts because the parent's `addAll` calls `add`.

Self-check

- **Q:** Why does `CountingSet.addAll` call `inner.add`, not `this.add`? **A:** To avoid double counting and to depend only on the inner set's public contract.

TL;DR

Has-a + delegate beats is-a for reuse. Use inheritance only for real, designed-for-extension type families.

2.12 Object identity vs equality: the `equals` / `hashCode` contract

What it is

- **Identity:** are these two references pointing at the **same object**? Tested with `==`.
- **Equality:** do these two objects **represent the same value**? Tested with `equals`. By default `Object.equals` is identity; classes override it to mean value equality (`String`, `Integer`, `LocalDate`, records).

Analogy: two printed copies of the same book are **equal** (same content) but not **identical** (two physical objects).

The problem it solves

Hash-based collections (`HashMap`, `HashSet`) first use `hashCode()` to pick a bucket, then `equals()` to compare inside the bucket. If you get either wrong, objects "disappear" from sets and maps.

> **Key term — hash table:** A `HashMap` or `HashSet` keeps its entries in an array of slots called **buckets**. To store or find an object it computes the object's **hash code** (an `int` derived from its fields), uses that number to jump straight to one bucket, and only then compares the few objects in that bucket with `equals`. This is why lookups are fast, and why two equal objects must produce the same hash code: otherwise the search starts in the wrong bucket.

How it works: the contracts

`equals` must be:

1. **Reflexive:** `x.equals(x)` is true.
2. **Symmetric:** `x.equals(y)` ⇔ `y.equals(x)`.
3. **Transitive:** if `x.equals(y)` and `y.equals(z)` then `x.equals(z)`.
4. **Consistent:** repeated calls give the same answer if nothing used in the comparison changed.
5. **Non-null:** `x.equals(null)` is false.

`hashCode` must:

1. Return the same value on repeated calls in one run, if the fields used by `equals` do not change.
2. Return **equal hash codes for equal objects**.
3. Unequal objects **may** share a hash code (collisions are allowed, just slower).

So: whenever you override `equals`, override `hashCode` using the same fields. Records and IDE generators (the "generate equals/hashCode" command in a code editor) do this for you.

Java example

```java
// File: code/lld/ch02/equality/BrokenPoint.java
package lld.ch02.equality;

/**
 * WRONG on purpose: overrides equals but NOT hashCode, breaking the contract "equal objects -> equal hash codes".
 * javac -Xlint warns about exactly this bug ([overrides]); the warning is silenced ONLY because this class
 * exists to demonstrate the bug. In real code, treat that warning as an error and add hashCode().
 */
@SuppressWarnings("overrides")
public final class BrokenPoint {
    private final int x, y;

    public BrokenPoint(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object o) {
        return o instanceof BrokenPoint p && x == p.x && y == p.y;
    }
    // hashCode() inherited from Object: usually different for two distinct objects
}
```

```java
// File: code/lld/ch02/equality/Point.java
package lld.ch02.equality;

import java.util.Objects;

/** Correct value-equality: equals and hashCode are overridden TOGETHER, on the same fields. */
public final class Point {
    private final int x, y;

    public Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                          // reflexive fast path
        if (!(o instanceof Point p)) return false;           // also handles null
        return x == p.x && y == p.y;
    }

    @Override public int hashCode() { return Objects.hash(x, y); }

    @Override public String toString() { return "(" + x + "," + y + ")"; }
}
```

```java
// File: code/lld/ch02/equality/Main.java
package lld.ch02.equality;

import java.util.HashSet;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        Point a = new Point(1, 2), b = new Point(1, 2);
        System.out.println("a == b (identity): " + (a == b));
        System.out.println("a.equals(b) (equality): " + a.equals(b));
        System.out.println("hash codes equal: " + (a.hashCode() == b.hashCode()));
        Set<Point> good = new HashSet<>();
        good.add(a); good.add(b);
        System.out.println("HashSet<Point> size after adding a and b: " + good.size() + ", contains (1,2)? " +
good.contains(new Point(1, 2)));

        BrokenPoint x = new BrokenPoint(1, 2), y = new BrokenPoint(1, 2);
        Set<BrokenPoint> bad = new HashSet<>();
        bad.add(x);
        System.out.println("BrokenPoint equals: " + x.equals(y) + ", but set.contains(equal one): " +
bad.contains(y));

        String s1 = "hi", s2 = new String("hi");
        System.out.println("\"hi\" == new String(\"hi\"): " + (s1 == s2) + ", equals: " + s1.equals(s2));

        Integer i1 = 1000, i2 = 1000;
        System.out.println("Integer 1000 equals: " + i1.equals(i2) + "  (never compare boxed numbers with ==)");
    }
}
```

Output:

```
a == b (identity): false
a.equals(b) (equality): true
hash codes equal: true
HashSet<Point> size after adding a and b: 1, contains (1,2)? true
BrokenPoint equals: true, but set.contains(equal one): false
"hi" == new String("hi"): false, equals: true
Integer 1000 equals: true  (never compare boxed numbers with ==)
```

Explaining each part: `Point` overrides both methods on the same fields, so the set treats `a` and `b` as one element. `BrokenPoint` overrides only `equals`; its inherited identity-based `hashCode` (almost always different for two distinct objects) sends an equal object to a different bucket, so `contains` fails. `javac -Xlint` (the compiler run with extra warnings switched on) even warns about this (`[overrides] ... overrides equals, but neither it nor any superclass overrides hashCode`); `BrokenPoint` carries `@SuppressWarnings("overrides")` only because it exists to show the bug. In real code, treat that warning as an error. Strings and boxed numbers must be compared with `equals`: `Integer` caches (keeps ready-made, reused objects for) only small values (at least −128 to 127), so `==` on boxed numbers happens to work for small values and fails for large ones.

Trade-offs / design choice

Kind of class	Equality
Value object (Money, Point, Coordinate)	Value equality on all fields; ideally immutable
Entity (User, Order, ParkingSpot)	Identity or equality by a stable **ID** only

Common mistakes

- Overriding `equals` without `hashCode` (above).
- `equals(Point p)` instead of `equals(Object o)` (an overload, never called by collections).
- Using **mutable fields** in `hashCode` and then mutating an object that is already a key in a `HashMap` (it becomes unfindable).
- Using `getClass() !=` vs `instanceof` inconsistently across a hierarchy, breaking symmetry.

Real-world examples

`String`, `Integer`, `BigDecimal` (note: `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` is false because scale (the number of decimal places) is part of equality; use `compareTo` for numeric comparison), `LocalDate`, `List.equals` (element-wise).

In the interview

> **30-second answer:** "`==` compares identity, `equals` compares logical value. If I override `equals` I must override `hashCode` on the same fields, because hash collections use the hash to find the bucket and `equals` inside it. For value objects I use records; for entities I compare by ID."

- *"What happens if only `equals` is overridden?"* — Equal objects can land in different buckets; `HashSet.contains` and `HashMap.get` fail.

Self-check

- **Q:** Can two unequal objects have the same hash code? **A:** Yes; that is a collision and is allowed.
- **Q:** Can two equal objects have different hash codes? **A:** No; that breaks the contract.

TL;DR

`==` = same object; `equals` = same value. Override `equals` and `hashCode` together, on the same (preferably immutable) fields.

2.13 Immutability and records

What it is

An **immutable** object cannot change after construction. "Changes" produce **new** objects. A **record** (Java 16+) is a concise way to declare an immutable **data carrier**: `record Order(String id, List<String> items, Money total) {}`. Analogy: a printed receipt. To correct it, you print a new receipt; you do not edit the old one.

The problem it solves

Mutable shared objects cause bugs when one part of the program changes something another part relies on, and they need locking when shared between threads. Immutable objects (all fields `final`, `this` not leaked during

construction) are **thread-safe without locks**, safe as `HashMap` keys and `HashSet` elements, easy to cache and share, and easy to reason about.

How it works

Hand-written immutable class (checklist):

1. Make the class `final` (or give it private constructors and static factories) so no subclass can add mutability.
2. Make all fields `private final`.
3. Provide no setters; methods that "modify" return a new object (`plus`, `withItem`).
4. **Defensive copies:** copy mutable inputs in the constructor and never hand out internal mutable objects (return copies or unmodifiable views).
5. Do not let `this` escape during construction.

Records generate: `private final` fields for each component, a canonical constructor, accessors named after the components (`id()`, not `getId()`), and `equals`, `hashCode`, `toString` based on all components. Records are implicitly `final`, extend `java.lang.Record`, cannot declare extra **instance** fields (static fields and extra methods are fine), and can implement interfaces. A **compact constructor** (`public Order { ... }`) validates or normalises the parameters before they are assigned.

Records are only shallowly immutable: if a component is a mutable `List`, the list can still change unless you copy it (`List.copyOf`).

Java example

```java
// File: code/lld/ch02/immutability/Money.java
package lld.ch02.immutability;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Objects;

/**
 * A hand-written immutable VALUE OBJECT: final class, private final fields, no setters,
 * "changes" return NEW objects. The amount is a BigDecimal (exact decimal arithmetic, never double),
 * normalised to 2 decimal places so that 5 and 5.00 are the same Money.
 */
public final class Money {
    private final BigDecimal amount;
    private final String currency;

    private Money(BigDecimal amount, String currency) {
        this.currency = Objects.requireNonNull(currency);
        // UNNECESSARY = refuse (ArithmeticException) anything finer than a cent instead of silently rounding it
        this.amount = amount.setScale(2, RoundingMode.UNNECESSARY);
    }

    public static Money of(String amount, String currency) { return new Money(new BigDecimal(amount), currency); }

    public Money plus(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentException("currency mismatch");
        return new Money(amount.add(other.amount), currency);   // new object; 'this' is untouched
    }

    public BigDecimal amount() { return amount; }

    @Override public boolean equals(Object o) {        // safe: the scale is always 2, so equals == numeric
equality
        return o instanceof Money m && amount.equals(m.amount) && currency.equals(m.currency);
    }
    @Override public int hashCode() { return Objects.hash(amount, currency); }
    @Override public String toString() { return amount.toPlainString() + " " + currency; }
}
```

```java
// File: code/lld/ch02/immutability/Order.java
package lld.ch02.immutability;

import java.util.List;

/**
 * A RECORD: the compiler generates the private final fields, canonical constructor, accessors,
 * equals, hashCode and toString. The compact constructor validates and makes a DEFENSIVE COPY,
 * because a record is only shallowly immutable.
 */
public record Order(String id, List<String> items, Money total) {
    public Order {
        if (id == null || id.isBlank()) throw new IllegalArgumentException("id required");
        items = List.copyOf(items);                // unmodifiable copy: caller's list cannot leak in
    }

    public Order withItem(String item, Money price) {  // "wither": returns a modified copy
        var more = new java.util.ArrayList<>(items);
        more.add(item);
        return new Order(id, more, total.plus(price));
    }
}
```

```java
// File: code/lld/ch02/immutability/Main.java
package lld.ch02.immutability;

import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Money five = Money.of("5.00", "USD");
        Money seven = five.plus(Money.of("2.00", "USD"));
        System.out.println("five = " + five + ", seven = " + seven + "  (five unchanged)");
        System.out.println("Money.of(\"5\") equals Money.of(\"5.00\")? " + Money.of("5", "USD").equals(five) + "
(scale normalised)");

        List<String> callerList = new ArrayList<>(List.of("pen"));
        Order o1 = new Order("O-1", callerList, five);
        callerList.add("SNEAKY");                          // does not reach the record
        System.out.println("o1 = " + o1);

        Order o2 = o1.withItem("ink", Money.of("1.50", "USD"));
        System.out.println("o2 = " + o2);
        System.out.println("o1 still = " + o1.items() + ", equals copy? " + o1.equals(new Order("O-1",
List.of("pen"), Money.of("5.00", "USD"))));

        try {
            o1.items().add("hack");
        } catch (UnsupportedOperationException e) {
            System.out.println("items() is unmodifiable: " + e.getClass().getSimpleName());
        }
    }
}
```

Output:

```
five = 5.00 USD, seven = 7.00 USD  (five unchanged)
Money.of("5") equals Money.of("5.00")? true  (scale normalised)
o1 = Order[id=O-1, items=[pen], total=5.00 USD]
o2 = Order[id=O-1, items=[pen, ink], total=6.50 USD]
o1 still = [pen], equals copy? true
items() is unmodifiable: UnsupportedOperationException
```

Explaining each part: `Money.plus` returns a new object; `five` stays 5.00. The amount is a `BigDecimal` (exact decimal arithmetic; `double` cannot represent 0.10 exactly, so money in `double` drifts). The constructor fixes the **scale** (number of decimal places) at 2, because `BigDecimal.equals` compares scale too (`5` and `5.00` would otherwise be unequal, see Section 2.12); `RoundingMode.UNNECESSARY` makes it throw instead of silently rounding away a fraction of a cent. The constructor is private and `Money.of(...)` is a **static factory**, one of the options in the checklist above. The record's compact constructor copies the caller's list with `List.copyOf`, so adding `"SNEAKY"` to the caller's list later has no effect, and `items()` is unmodifiable. `withItem` is a "wither" returning a changed copy. Two records with equal components are `equals`.

Trade-offs

Benefit	Cost
Thread-safe without locks; safe hash keys; no defensive reasoning needed	New object per change (usually cheap; can matter in hot loops, i.e. code that runs millions of times)
Records remove boilerplate	Records cannot hide their components or extend classes

When not to use it

Entities whose identity persists while state changes (a `ParkingSpot` becoming occupied, an `Account` balance) are naturally **mutable**, with encapsulated changes. Make them immutable only if you adopt an event-sourced (state rebuilt from a stored list of past events) or copy-on-write (every change makes a new copy) style deliberately.

Common mistakes

- Forgetting defensive copies (a "final" list field whose contents anyone can change).
- Assuming `final` makes the referenced object immutable (it only fixes the reference).
- Using records for entities that need controlled mutation.

Real-world examples

`String`, boxed primitives (`Integer`, `Long`), `BigDecimal`, `BigInteger`, all of `java.time` (`LocalDate`, `Instant`, `Duration`), `List.of`/`Map.of` collections; Kafka (a widely used message-streaming system) messages and event-sourcing events are treated as immutable facts.

In the interview

> **30-second answer:** "For value objects like Money, Coordinate or Ticket I use immutable records: final fields, no setters, validation in the compact constructor, and defensive copies of collections. They are thread-safe and safe as map keys. Entities whose state changes stay mutable but encapsulated."

- *"Is a record fully immutable?"* — Shallowly. Mutable components must be copied.

Self-check

- **Q:** What does `final List<String> items` guarantee? **A:** The reference never changes; the list contents still can.

TL;DR

Immutable = final class, private final fields, no setters, defensive copies. Records give you this (shallowly) in one line. Use for value objects; keep entities mutable but encapsulated.

2.14 Access modifiers and packages

What it is

A **package** is a named group of related classes (a namespace and a folder), e.g. `lld.ch02.access.core`. **Access modifiers** control who can see a class or member.

Modifier	Same class	Same package	Subclass in another package	Everyone
`private`	✔	✘	✘	✘
(none) package-private	✔	✔	✘	✘
`protected`	✔	✔	✔ (through inheritance)	✘
`public`	✔	✔	✔	✔

Top-level classes can only be `public` or package-private. Analogy: rooms in an office: your desk drawer (private), the team room (package), the company floor accessible to affiliated teams (protected), the lobby (public).

The problem it solves

Access modifiers make encapsulation **enforceable by the compiler**. Packages group classes by feature and let you hide implementation classes, exposing only a small public API.

How it works

1. Start everything as `private`; widen only when needed.
2. Group by **feature** (`parking`, `payment`) rather than by layer only.
3. Keep helper classes package-private; expose a small `public` facade (one simple front class that hides the others).
4. `protected` is for members designed for subclasses; note that it **also** grants access to the whole package.
5. Since Java 9, **modules** (`module-info.java`) add a further level: only exported packages are visible to other modules.

Java example

```java
// File: code/lld/ch02/access/client/PremiumWallet.java
package lld.ch02.access.client;

import lld.ch02.access.core.Wallet;

/** A subclass in ANOTHER package: may use protected members, not package-private or private ones. */
public class PremiumWallet extends Wallet {
    public PremiumWallet(String owner) { super(owner); }

    public void grantBonus(long cents) {
        bonusCents += cents;                    // OK: protected, accessed through inheritance
        // auditCount++;                        // would NOT compile: package-private in another package
        // balanceCents += cents;               // would NOT compile: private
    }

    @Override public String describe() {       // may WIDEN protected -> public when overriding
        return "[premium] " + super.describe();
    }
}
```

```java
// File: code/lld/ch02/access/client/Main.java
package lld.ch02.access.client;

import lld.ch02.access.core.Audits;
import lld.ch02.access.core.Wallet;

public class Main {
    public static void main(String[] args) {
        Wallet basic = new Wallet("Ada");
        basic.add(1_000);
        PremiumWallet premium = new PremiumWallet("Kai");
        premium.add(2_000);
        premium.add(500);
        premium.grantBonus(300);

        System.out.println("public field owner: " + basic.owner + ", public method balance: " +
basic.balanceCents());
        System.out.println(premium.describe());
        System.out.println("audit lines: " + Audits.all());
        System.out.println("audits for Kai: " + Audits.countFor(premium));
        // new lld.ch02.access.core.AuditLog();  // would NOT compile: class is package-private
    }
}
```

```java
// File: code/lld/ch02/access/core/AuditLog.java
package lld.ch02.access.core;

import java.util.ArrayList;
import java.util.List;

/** Package-private CLASS: an implementation detail of the core package. */
final class AuditLog {
    private static final List<String> LINES = new ArrayList<>();

    private AuditLog() { }

    static void record(Wallet w, String what) {
        w.auditCount++;                          // allowed: same package
        LINES.add(w.owner + ": " + what);
    }

    static List<String> lines() { return List.copyOf(LINES); }
}
```

```java
// File: code/lld/ch02/access/core/Audits.java
package lld.ch02.access.core;

import java.util.List;

/** The PUBLIC face of the package: exposes only what outsiders are allowed to read. */
public final class Audits {
    private Audits() { }
    public static List<String> all() { return AuditLog.lines(); }
    public static long countFor(Wallet w) { return w.auditCount; }
}
```

```java
// File: code/lld/ch02/access/core/Wallet.java
package lld.ch02.access.core;

/** Shows all four access levels on one class. */
public class Wallet {
    public final String owner;                // public: visible everywhere
    protected long bonusCents;                // protected: same package + subclasses anywhere
    long auditCount;                          // package-private (no keyword): same package only
    private long balanceCents;                // private: this class only

    public Wallet(String owner) { this.owner = owner; }

    public void add(long cents) {
        if (cents <= 0) throw new IllegalArgumentException("positive only");
        balanceCents += cents;
        AuditLog.record(this, "add " + cents);   // package-private helper class, invisible outside core
    }

    public long balanceCents() { return balanceCents; }

    protected String describe() { return owner + " has " + balanceCents + "c (+" + bonusCents + "c bonus)"; }
}
```

(`Main` is in `lld.ch02.access.client`; run `java -cp /tmp/lld_02 lld.ch02.access.client.Main`.)

Output:

```
public field owner: Ada, public method balance: 1000
[premium] Kai has 2500c (+300c bonus)
audit lines: [Ada: add 1000, Kai: add 2000, Kai: add 500]
audits for Kai: 2
```

Explaining each part: `AuditLog` is a package-private class and `auditCount` a package-private field, so only code in `core` touches them; outsiders read audit data only through the public `Audits` facade. `PremiumWallet` in another package can update `protected bonusCents` but the commented lines (package-private and private access) would not compile. The override of `describe()` widens `protected` to `public`, which overriding allows.

Common mistakes

- Making everything `public` "to be safe".
- Using `protected` fields as a shortcut instead of protected methods (subclasses then break invariants).
- Putting every class of a machine-coding solution in one package.

In the interview

30-second answer: "Fields are private by default, collaborators within a feature package can be package-private, protected is only for members designed for subclassing, and public is the deliberate API. I organise machine-coding solutions into packages such as model, repository, service and strategy."

Self-check

- **Q:** Can a class in another package, which is not a subclass, read a `protected` field? **A:** No.

TL;DR

private < package-private < protected < public. Default to private; expose a small public API per package.

2.15 Generics basics for LLD

What it is

Generics let a class, interface or method work with **any type while keeping compile-time type safety**. `List<String>` holds only strings; `Repository<String, User>` stores only users. The letters in angle brackets (`T`,

`ID`, `E`, `K`, `V`) are **type parameters**. Analogy: a labelled storage box: "SHOES ONLY". The label is checked when you put things in, so you never need to check when you take them out.

The problem it solves

Without generics you store `Object` (the root type of every class) and cast on every read, discovering mistakes at run time (`ClassCastException`). With generics, mistakes are **compile errors**, and one class (`Repository<ID, T>`) serves every entity, which is extremely handy in machine-coding rounds.

How it works

1. **Generic class/interface:** `class Repository<ID, T extends Identifiable<ID>>`.
2. **Bounded type parameter:** `T extends X` means T must be X or a subtype, so you can call X's methods on T.
3. **Generic method:** `static <T extends Comparable<? super T>> T max(List<? extends T> items)`.
4. **Wildcards:** `? extends T` (read-only producer), `? super T` (write-capable consumer), `?` (unknown). Mnemonic **PECS**: *Producer Extends, Consumer Super* (from *Effective Java*, Joshua Bloch's well-known Java best-practice book).
5. **Diamond operator:** `new Repository<>()` infers the type arguments.
6. **Type erasure:** type arguments exist only at compile time; at run time `List<String>` and `List<Integer>` are the same class. Consequences: no `new T()`, no `instanceof List<String>`, no primitive type arguments (use `Integer`, not `int`).
7. **Avoid raw types** (`List` without `<...>`); they switch the checks off.

Java example

```java
// File: code/lld/ch02/generics/Identifiable.java
package lld.ch02.generics;

/** Generic interface: ID is a TYPE PARAMETER filled in by each implementer. */
public interface Identifiable<ID> {
    ID id();
}
```

```java
// File: code/lld/ch02/generics/Product.java
package lld.ch02.generics;

public record Product(Integer id, String title, long priceCents)
        implements Identifiable<Integer>, Comparable<Product> {
    @Override public int compareTo(Product o) { return Long.compare(priceCents, o.priceCents); }
}
```

```java
// File: code/lld/ch02/generics/Repository.java
package lld.ch02.generics;

import java.util.*;

/**
 * A generic in-memory repository, a very common helper in machine-coding rounds.
 * T is BOUNDED: it must be Identifiable by the same ID type.
 */
public class Repository<ID, T extends Identifiable<ID>> {
    private final Map<ID, T> store = new LinkedHashMap<>();

    public void save(T item)            { store.put(item.id(), item); }
    public Optional<T> findById(ID id)  { return Optional.ofNullable(store.get(id)); }
    public List<T> findAll()            { return List.copyOf(store.values()); }
}
```

```java
// File: code/lld/ch02/generics/User.java
package lld.ch02.generics;

public record User(String id, String name) implements Identifiable<String> { }
```

```java
// File: code/lld/ch02/generics/Main.java
package lld.ch02.generics;

import java.util.ArrayList;
import java.util.List;

public class Main {
    /** Generic METHOD with a bounded type parameter. */
    static <T extends Comparable<? super T>> T max(List<? extends T> items) {
        T best = items.get(0);
        for (T t : items) if (t.compareTo(best) > 0) best = t;
        return best;
    }

    /** PECS: Producer Extends (we read from src), Consumer Super (we write into dst). */
    static <T> void copy(List<? extends T> src, List<? super T> dst) {
        dst.addAll(src);
    }

    public static void main(String[] args) {
        Repository<String, User> users = new Repository<>();
        users.save(new User("u1", "Ada"));
        users.save(new User("u2", "Kai"));
        System.out.println("user u2: " + users.findById("u2").map(User::name).orElse("none"));
        System.out.println("user u9: " + users.findById("u9").map(User::name).orElse("none"));

        Repository<Integer, Product> products = new Repository<>();
        products.save(new Product(1, "pen", 150));
        products.save(new Product(2, "lamp", 2_500));
        products.save(new Product(3, "mug", 800));
        System.out.println("most expensive: " + max(products.findAll()).title());

        List<Integer> ints = List.of(3, 1, 2);
        List<Number> numbers = new ArrayList<>();
        copy(ints, numbers);                            // Integer list into a Number list
        System.out.println("copied into List<Number>: " + numbers + ", max int = " + max(ints));

        // users.save(new Product(4, "x", 1));          // would NOT compile: type safety at compile time
        List<String> a = new ArrayList<>();
        List<Integer> b = new ArrayList<>();
        System.out.println("same runtime class (type erasure)? " + (a.getClass() == b.getClass()));
    }
}
```

Output:

```
user u2: Kai
user u9: none
most expensive: lamp
copied into List<Number>: [3, 1, 2], max int = 3
same runtime class (type erasure)? true
```

Explaining each part: one `Repository` class stores `User`s keyed by `String` and `Product`s keyed by `Integer`; `findById` returns `Optional` (a box that either holds a value or is empty, used instead of `null`) for a normal "not found". `max` works for any comparable type (`Product` compares by price). `copy` accepts a `List<Integer>` as a producer and a `List<Number>` as a consumer (PECS). The final line demonstrates erasure.

Common mistakes

- Raw types and unchecked casts.
- Thinking `List<Integer>` is a subtype of `List<Number>` (it is not; use `List<? extends Number>`).
- Over-generic designs with five type parameters nobody can read.

Real-world examples

The entire Collections framework (`List<E>`, `Map<K,V>`), `Optional<T>`, `CompletableFuture<T>` (a result that will arrive later from background work), `Comparator<T>`, Spring Data's `CrudRepository<T, ID>` (the same idea as our `Repository`).

In the interview

> **30-second answer:** "I use generics for reusable infrastructure like an in-memory `Repository<ID, T>` or an `EventBus<E>`, so type errors are caught at compile time. I know bounded types, wildcards with PECS, and that erasure removes type arguments at run time."

Self-check

- **Q:** Why can't you write `new T()`? **A:** Type erasure: the runtime does not know T; pass a `Supplier<T>` instead.

TL;DR

Generics = type-safe reuse. Bounds let you call methods on T; PECS for wildcards; erasure means no runtime type info.

2.16 Coupling and cohesion (the yardsticks)

- **Cohesion** — how closely the responsibilities **inside** one class belong together. **High cohesion is good**: every method works on the same data toward one purpose (the "one sentence, no 'and'" test from Chapter 1).
- **Coupling** — how much one class **depends on** others' details. **Low coupling is good**: classes talk through small interfaces, so a change in one does not ripple through many.

Every idea in this chapter pushes toward **high cohesion, low coupling**: encapsulation and access modifiers reduce coupling; single-responsibility classes raise cohesion; interfaces and composition decouple callers from implementations. Chapters 4–5 turn these yardsticks into named principles (SOLID, Law of Demeter, GRASP).

2.17 Object-oriented analysis and design (OOA / OOD / OOP)

What it is

Object-oriented analysis and design (OOAD) is a long-established, standard method for converting a written description of a system into objects plus the links between them. Most of the vocabulary used in an LLD interview originates here. The name covers two separate activities, analysis and design, followed by implementation; each asks a different question and produces a different result:

Activity	Question it answers	What exists at the end	Talks about
Analysis (OOA)	What does the system consist of, and what is it required to do?	The things in the problem, named in **business (domain) terms**	books, members, loans
Design (OOD)	Which objects, with which responsibilities, will provide that behaviour?	Classes, what each is responsible for, and how they relate	`Book`, `Member`, `Loan` and who holds whom
Implementation (OOP)	How do we express it in a programming language?	Running code	`ArrayList`, `HashMap`, `synchronized`

If you are talking about a `HashMap` two minutes in, you have leapt straight to implementation and **left out the two activities the interviewer wanted to watch**.

The problem it solves

Blending the activities is the classic time sink: if you pick operations before the objects are settled, operations end up on the wrong class, and moving them later is expensive.

How it works: the four steps

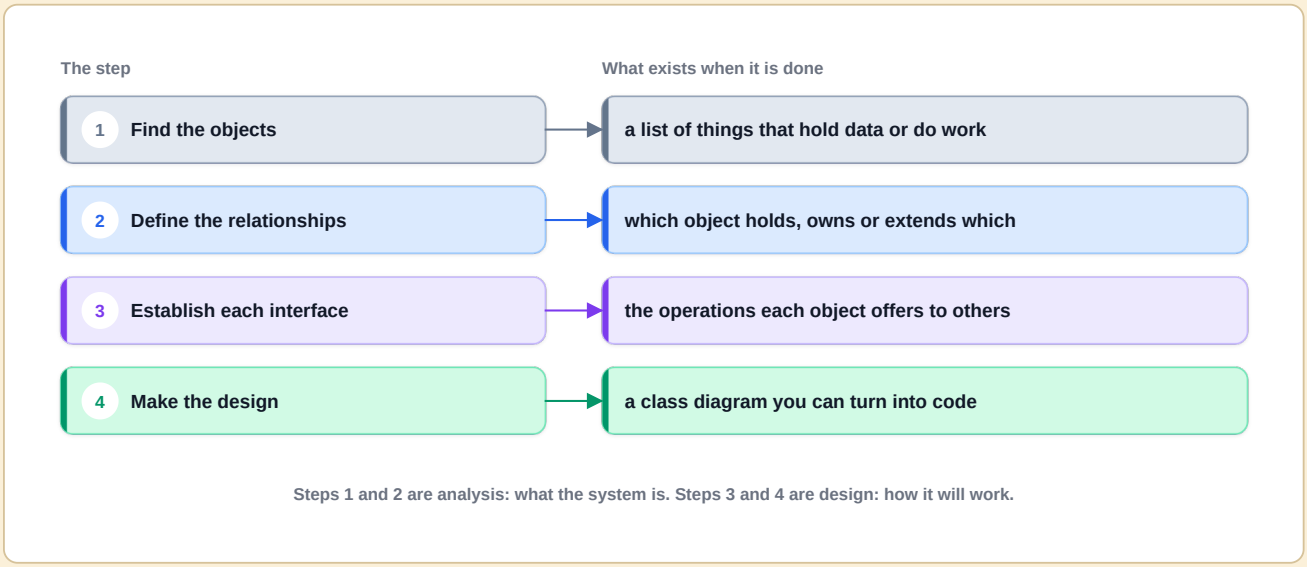


Figure — The four OOAD steps and what exists after each; steps 1–2 are analysis and steps 3–4 are design (course diagram)

1. **Find the objects.** Go through the description and note everything that **stores information or performs work**; most candidates are nouns. Anything that carries only a single value (a colour, a date) becomes a **field** of some object, not an object itself.
2. **Define the relationships.** For every pair of objects mentioned together, answer three questions: which one holds a reference to the other, does it **own** it, and **how many** of each are involved? (E.g. a library holds many books; a loan points at one book and one member.)
3. **Establish each interface.** Here *interface* means the set of operations an object makes available **to other objects**. Write that set down per object. Whatever is left off becomes **private**, so this step is where encapsulation really gets decided.
4. **Make the design.** Put classes, fields, operations and relationships into a single drawing (a UML class diagram) and verify it against the description you began with. That drawing is the design; the code is derived from it.

In the figure, steps 1–2 form the analysis (what the system is) and steps 3–4 form the design (how it will work). **The order is fixed because each step consumes the output of the one before.**

Worked example: "library members borrow books; each book on loan must be returned by a due date"

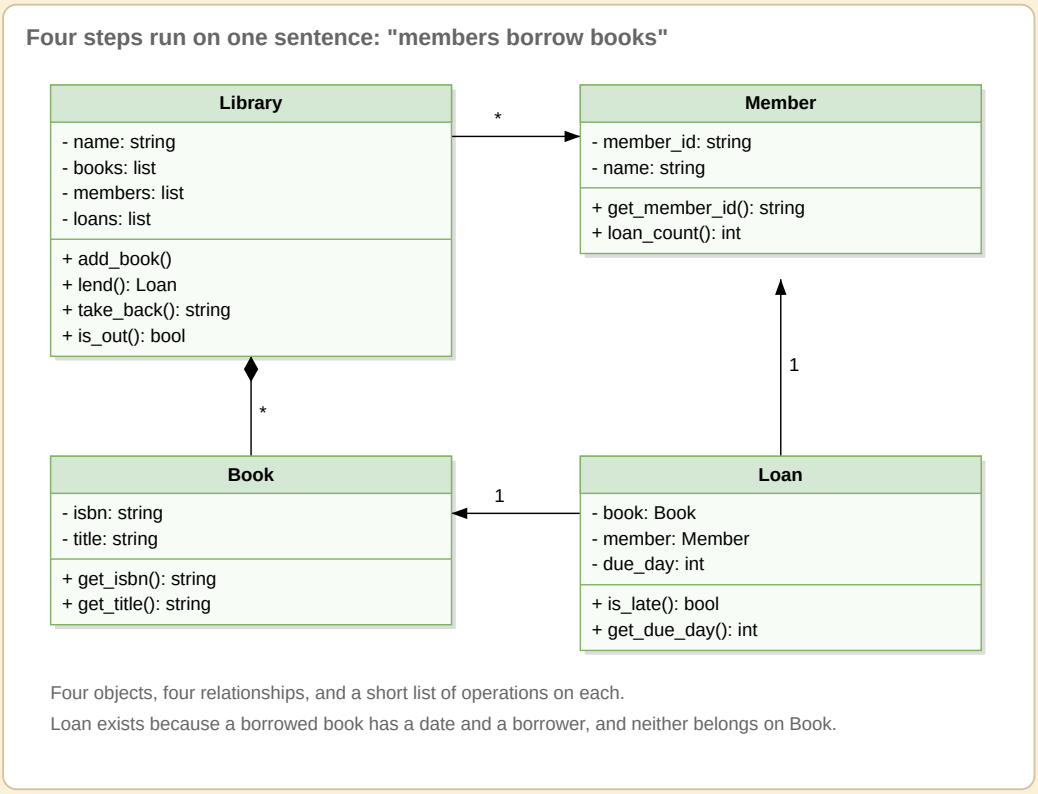


Figure — The four steps applied to one sentence: Library, Member, Book and the connection object Loan (course diagram)

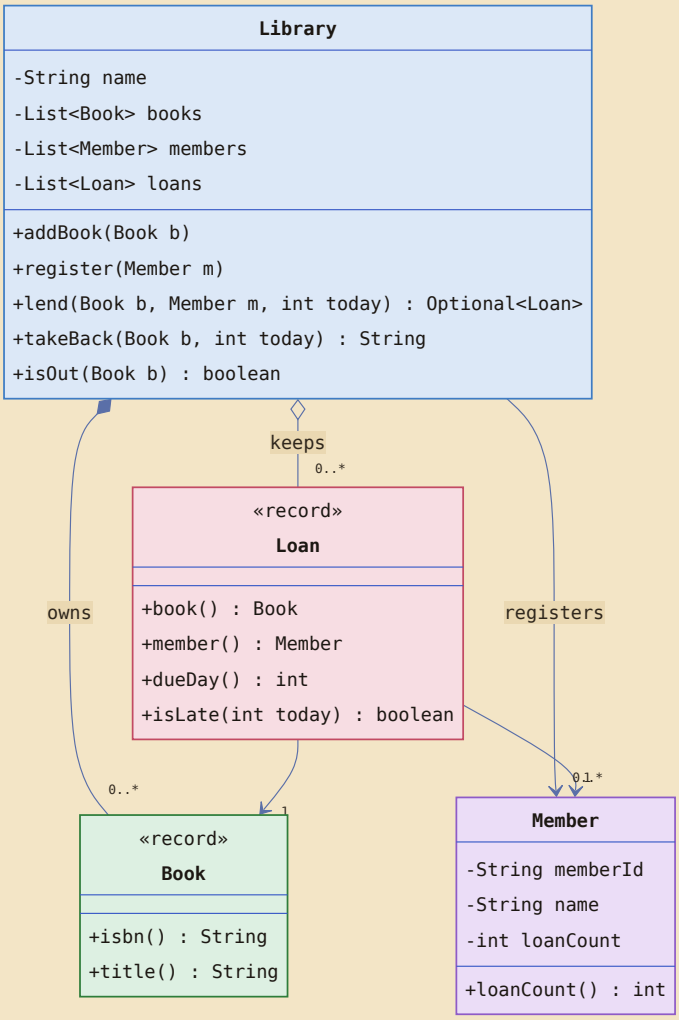
Step 1, objects. Candidate nouns: member, library, book, date, borrowed book. A *date* is one value, so it becomes a field. That leaves four objects: `Library`, `Member`, `Book`, and the borrowed book, which reads better as **`Loan`**. `Loan` is the one candidates overlook most often, since the sentence never uses it as a noun.

> **General rule:** if two objects are linked and **the link has data of its own**, model the link as an object. A lent book comes with someone who borrowed it and a date it must be back by; neither can live on `Book`, because a book sitting on the shelf has neither. Other examples: `Reservation` in a hotel, `Ticket` in a parking lot, `Enrollment` between student and course.

Step 2, relationships. The library **owns** its many book records in the strong sense (delete the library and its book records disappear with it): that is **composition**, drawn as the filled diamond. The library also **registers** many members without owning them (people go on existing without the library): a plain **association** arrow. Each loan **points to** a single book and a single member, owning neither.

Step 3, interfaces. Library: lend a book, accept a return, report whether a copy is currently out. Book: expose its title and ISBN (the International Standard Book Number printed on every book). Member: report its number of loans. Loan: report whether it is overdue. All other members, every field included, stay private. Pay attention to **which class answers "is this book out?"**: `Book` cannot, since it has no knowledge of loans; `Library` keeps the loans, so only it can answer.

Step 4, design. The figure fully answers the original sentence in four steps without a line of code. Our Java version (which adds a `register` operation so members can be checked):




```java
// File: code/lld/ch02/ooad/Book.java
package lld.ch02.ooad;

/** A book on the shelf: knows its ISBN and title, and nothing about who borrowed it. */
public record Book(String isbn, String title) { }
```

```java
// File: code/lld/ch02/ooad/Library.java
package lld.ch02.ooad;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/** Owns its books (composition), refers to members (association), and keeps the loans. */
public final class Library {
    public static final int LOAN_DAYS = 14;
    public static final int MAX_LOANS_PER_MEMBER = 5;

    private final String name;
    private final List<Book> books = new ArrayList<>();
    private final List<Member> members = new ArrayList<>();
    private final List<Loan> loans = new ArrayList<>();

    public Library(String name) { this.name = name; }

    public void addBook(Book b)     { books.add(b); }
    public void register(Member m)  { members.add(m); }

    /** Only the library can answer this: a Book does not know about loans. */
    public boolean isOut(Book b) { return loans.stream().anyMatch(l -> l.book().equals(b)); }

    public Optional<Loan> lend(Book b, Member m, int today) {
        if (!books.contains(b) || !members.contains(m)) return Optional.empty();
        if (isOut(b) || m.loanCount() >= MAX_LOANS_PER_MEMBER) return Optional.empty();
        Loan loan = new Loan(b, m, today + LOAN_DAYS);
        loans.add(loan);
        m.loanStarted();
        return Optional.of(loan);
    }

    public String takeBack(Book b, int today) {
        for (Loan l : loans) {
            if (l.book().equals(b)) {
                loans.remove(l);
                l.member().loanEnded();
                return b.title() + " returned " + (l.isLate(today) ? "LATE" : "on time");
            }
        }
        return b.title() + " was not on loan";
    }

    public String name() { return name; }
}
```

```java
// File: code/lld/ch02/ooad/Loan.java
package lld.ch02.ooad;

/**
 * The CONNECTION OBJECT: "a borrowed book" carries its own data (borrower, due day),
 * which belongs neither on Book nor on Member.
 */
public record Loan(Book book, Member member, int dueDay) {
    public boolean isLate(int today) { return today > dueDay; }
}
```

```java
// File: code/lld/ch02/ooad/Member.java
package lld.ch02.ooad;

/** A member exists independently of the library (the library refers to it, it does not own it). */
public final class Member {
    private final String memberId;
    private final String name;
    private int loanCount;

    public Member(String memberId, String name) { this.memberId = memberId; this.name = name; }

    public String memberId() { return memberId; }
    public String name()     { return name; }
    public int loanCount()   { return loanCount; }

    void loanStarted() { loanCount++; }        // package-private: only Library changes it
    void loanEnded()   { loanCount--; }
}
```

```java
// File: code/lld/ch02/ooad/Main.java
package lld.ch02.ooad;

public class Main {
    public static void main(String[] args) {
        Library lib = new Library("City Library");
        Book ddd = new Book("978-0321125217", "Domain-Driven Design");
        Book gof = new Book("978-0201633610", "Design Patterns");
        lib.addBook(ddd);
        lib.addBook(gof);
        Member ada = new Member("M1", "Ada");
        Member kai = new Member("M2", "Kai");
        lib.register(ada);
        lib.register(kai);

        System.out.println("Ada borrows DDD on day 1: " + lib.lend(ddd, ada, 1).map(l -> "due day " +
l.dueDay()).orElse("refused"));
        System.out.println("Kai borrows DDD on day 2: " + lib.lend(ddd, kai, 2).map(l -> "due day " +
l.dueDay()).orElse("refused (already out)"));
        System.out.println("Kai borrows GoF on day 2: " + lib.lend(gof, kai, 2).map(l -> "due day " +
l.dueDay()).orElse("refused"));
        System.out.println("is DDD out? " + lib.isOut(ddd) + ", Ada's loans: " + ada.loanCount());
        System.out.println(lib.takeBack(ddd, 10));
        System.out.println(lib.takeBack(gof, 20));
        System.out.println(lib.takeBack(gof, 21));
        System.out.println("is DDD out? " + lib.isOut(ddd));
    }
}
```

Output:

```
Ada borrows DDD on day 1: due day 15
Kai borrows DDD on day 2: refused (already out)
Kai borrows GoF on day 2: due day 16
is DDD out? true, Ada's loans: 1
Domain-Driven Design returned on time
Design Patterns returned LATE
Design Patterns was not on loan
is DDD out? false
```

Explaining each part: limits from the requirements became constants (`LOAN_DAYS = 14`, `MAX_LOANS_PER_MEMBER = 5`). `Loan` is an immutable record holding the connection data. `Member.loanStarted/loanEnded` are package-private so only `Library` changes loan counts. `isOut` lives on `Library`, the only object with the information.

Why UML

A design only helps if another engineer can read it. **UML (Unified Modeling Language)** is the standard drawing notation for that; it displaced several rival notations decades ago because it can express exactly the OOAD ideas above: a **filled diamond** marks ownership (composition), a **plain arrow** marks one object referring to another, and **+ / −** mark public / private. No symbol is decorative; each records a design decision. That is also why the concepts are taught before the notation: a shape means nothing until you know the decision behind it. Chapter 3 teaches UML in full.

Three ways this goes wrong

1. **Naming classes after tables** (`UserRow`, `OrderTable`, `BookData`). That is a database schema (the layout of tables and columns) wearing OO clothes. An object pairs data with the behaviour that modifies it, so a class holding only fields and getters should prompt the question: what is it for, what does it **do**?
2. **Missing the connection object** (`Loan`, `Reservation`, `Ticket`). If `borrower` and `dueDate` are drifting onto `Book`, that is the signal that a connection class is missing.
3. **Listing every operation you can imagine.** Step 3 is about what an object provides **to other objects**, not every conceivable action. Long method lists on every class show that no decisions were made.

In the interview

30-second answer: "I separate analysis from design. First I find the objects from the nouns, dropping single values into fields, and define relationships with ownership and multiplicity. Then I decide each object's public operations, which also decides what is private, and draw the class diagram. When the link between two objects has data of its own, like a loan's due date, that link becomes a class."

- *"Where does `isOut(book)` belong?"* — On `Library`, because it owns the loans; a `Book` does not know about loans.
- *"Why is `date` not a class?"* — It holds a single value with no behaviour of its own in this problem; it is a field.

Self-check

- **Q:** Which OOAD steps are analysis? **A:** Find the objects, define the relationships.
- **Q:** What is the connection object in a parking lot? **A:** The parking ticket (vehicle + spot + entry time).
- **Q:** When is encapsulation actually decided? **A:** In step 3, when you list each object's public operations; everything unlisted is private.

TL;DR

Analysis = what the system is (objects, relationships). Design = how objects deliver it (interfaces, class diagram). Implementation = code. Connections with data become objects. Don't name a `HashMap` in minute two.

Chapter summary: in the interview

30-second answer (to "Explain the pillars of OOP"): "Encapsulation hides data and gathers the rules about it into the class that owns it. Abstraction exposes only what callers need. Inheritance lets a child reuse a parent when 'child is a parent' is truly correct. Polymorphism lets one call behave differently per object, which removes type switches. In design I prefer interfaces and composition over deep inheritance, use enums for fixed sets, records for immutable values, and I keep equals and hashCode consistent."

Common follow-ups:

- *"Abstract class or interface?"* — Interface for the contract (many allowed, no state); abstract class when implementations share state or a template algorithm.
- *"Overloading vs overriding?"* — Compile-time by declared parameter types vs run-time by actual object.
- *"Why composition over inheritance?"* — Avoids fragile base classes, allows run-time swapping, keeps coupling low.
- *"What breaks if `hashCode` is not overridden with `equals`?"* — Hash collections fail to find equal objects.

- *"How do you make a class immutable?"* — Final class, private final fields, no setters, defensive copies; or a record with a compact constructor that copies collections.

Self-check (whole chapter)

- **Q:** Class vs object? **A:** Blueprint vs one instance with real values.
- **Q:** What does `this(...)` do? **A:** Calls another constructor of the same class (constructor chaining).
- **Q:** Encapsulation vs abstraction? **A:** Hiding data behind rule-enforcing methods vs hiding complexity behind a minimal set of operations.
- **Q:** What is dynamic dispatch? **A:** The JVM choosing an overridden method's implementation from the object's actual class at run time.
- **Q:** Are static methods overridden? **A:** No, hidden.
- **Q:** Can an interface have a method with a body? **A:** Yes: default, static and private methods.
- **Q:** Where should "which spot types fit a truck" live? **A:** As data on the `VehicleType` enum.
- **Q:** State the hashCode rule in one line. **A:** Equal objects must have equal hash codes.
- **Q:** Are records deeply immutable? **A:** No, shallowly; copy mutable components.
- **Q:** Who can see a package-private member? **A:** Only classes in the same package.
- **Q:** What does PECS stand for? **A:** Producer `extends`, consumer `super`.
- **Q:** Name the OOAD steps. **A:** Find objects, define relationships, establish interfaces, make the design.

TL;DR

- OOP = data bundled with the only code allowed to modify it.
- Class = blueprint; object = instance. Constructors create valid objects; `this` = current object; `static` = one per class.
- Four pillars: encapsulation (who may change data), abstraction (what callers need), inheritance (true is-a only), polymorphism (one call, many behaviours; kills type switches).
- Overloading is compile-time; overriding is run-time; fields and statics are not polymorphic.
- Interfaces = contracts (default/static/private methods, functional interfaces for lambdas); abstract classes = shared state and templates.
- Enums hold data and behaviour for fixed sets; composition beats inheritance for reuse.
- `==` identity vs `equals` value; override `hashCode` with `equals`.
- Immutable value objects (records + defensive copies); mutable but encapsulated entities.
- Default to `private`; group by feature packages; generics for type-safe reusable infrastructure.
- OOA (what) → OOD (how, with objects) → OOP (code). Connections that carry data become objects.

Chapter 3 — Class Relationships & UML

> **What you will learn.** How two classes can be connected (association, aggregation, composition, dependency, generalization, realization), how each connection looks in Java, how to pick the right one in ten seconds, and how to *draw* a design with UML (Unified Modeling Language, the standard picture notation for software designs; Section 3.9): use-case, class, sequence, activity and state machine diagrams. The chapter ends with how much UML you should actually draw in a 45-minute interview.

How this chapter is organised

Part	Topics	Java package
A. Relationships	3.1 Big picture · 3.2 Association (uni/bi, multiplicity) · 3.3 Aggregation · 3.4 Composition · 3.5 Dependency · 3.6 Generalization · 3.7 Realization · 3.8 Decision guide	`lld.ch03.association`, `aggregation`, `composition`, `dependency`, `generalization`, `realization`
B. UML	3.9 What UML is · 3.10 Use-case · 3.11 Class · 3.12 Sequence · 3.13 Activity · 3.14 State machine	`lld.ch03.library` (use case → class → sequence → activity), `lld.ch03.statemachine`
C. Interview	3.15 How much UML to draw · 3.16 Chapter self-check · 3.17 TL;DR	—

Every package has a `Main` that you can run (`javac` is the Java compiler, which turns `.java` files into runnable `.class` files; `java` runs them, and `-cp` says where the compiled classes are):

```
cd chapters/03-relationships-and-uml/code
javac --release 21 -d /tmp/lld_03 $(find . -name '*.java')
java -cp /tmp/lld_03 lld.ch03.association.Main     # or aggregation, composition, dependency,
                                                   # generalization, realization, statemachine, library
```

Part A — Class Relationships

3.1 The big picture: what a "relationship" really claims

What it is

A **relationship** is a line between two classes on a class diagram. It says that objects of one class are connected to objects of the other. That is all a beginner usually hears. But the line actually makes two precise claims:

1. **A claim about lifetime.** If object A is destroyed, is object B destroyed too? ("Lifetime" = the time between an object being created and it no longer being used.)
2. **A claim about substitution.** Can an object of class B be used anywhere an object of class A is expected? ("Substitution" = handing a B to code that asked for an A and having everything still work.)

Everyday analogy: your **body** and your **heart** share a lifetime (composition). A **car** and its **driver** do not — the driver walks away after parking (association). A **sports car** *is a* car and can go anywhere a car can (generalization).

There are six relationships you must know. Three of them are "has-a", one is "uses-a", and two are "is-a":

Family	Relationship	One-line meaning	Coupling (rough order)
uses-a	**Dependency**	A uses B briefly (parameter, local variable, return type)	weakest
has-a	**Association**	A holds a reference to B and can call it; nobody owns anybody	↓
has-a	**Aggregation**	A is a whole holding parts B; the parts can outlive A	↓
has-a	**Composition**	A is a whole *owning* parts B; the parts die with A	↓
is-a	**Realization**	class A implements interface B	↓
is-a	**Generalization**	class A extends class B (inheritance)	strongest coupling

"Coupling" means how much one class must change when the other changes. Inheritance couples a subclass to every detail of its parent, which is why it is at the bottom of the list. The order is a rule of thumb for discussion, not something the UML standard defines.

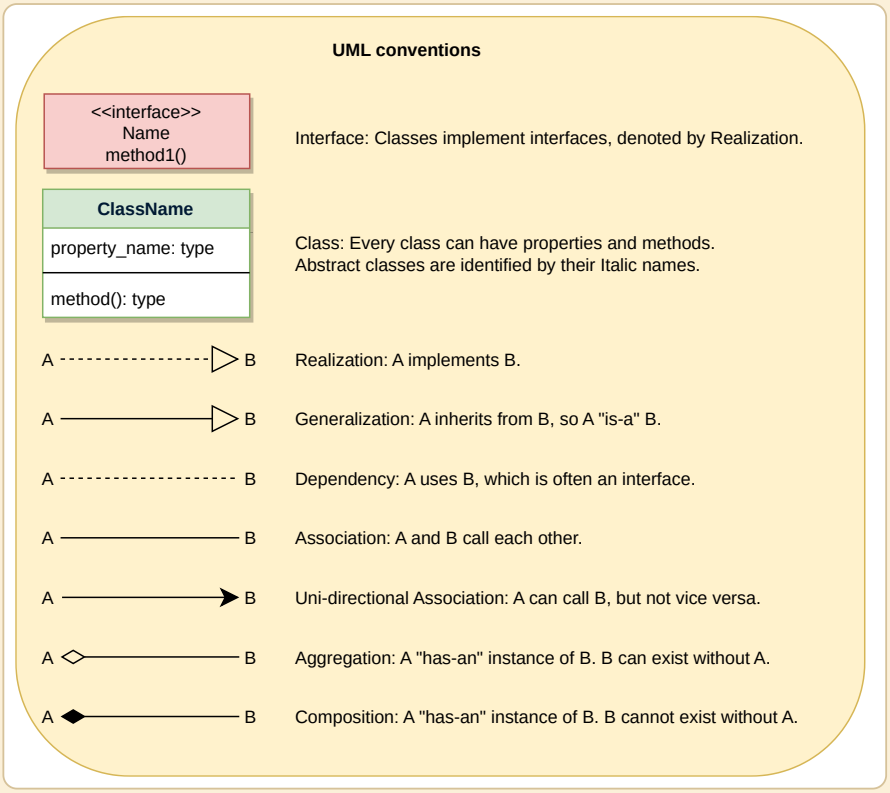


Figure — The UML conventions legend used throughout this book: class and interface boxes, and the line style for realization, generalization, dependency, association, uni-directional association, aggregation and composition (course diagram)

In the figure above, look at the *ends* of each line. The shape at the end (triangle, diamond, arrowhead) and whether the line is solid or dashed is the entire vocabulary. The rest of Part A goes through each line one at a time.

Two small differences between this legend and strict UML, so you are not confused later: the legend draws the **uni-directional association** with a *filled* arrowhead and the **dependency** as a dashed line with *no* head. Standard UML (and this book's Mermaid diagrams; Mermaid is a text-based diagram language that renders into pictures) uses an **open** arrowhead (`→`) for both — solid line for association, dashed line for dependency. Either drawing is understood in an interview if you say what you mean.

The problem it solves

Without a precise vocabulary, "Order has a Customer" and "ParkingFloor has ParkingSpots" sound the same. In code they are very different: one only points at an object that lives elsewhere, the other creates, owns and deletes its parts. If you mix them up you get bugs such as deleting data you did not own, or leaking objects nobody is responsible for. In an interview the mistake is visible on the whiteboard before a single line of code is written.

TL;DR

A relationship line is a claim about **who owns whom (lifetime)** and **who can stand in for whom (substitution)**. Learn the six lines and the one question that separates each.

3.2 Association (uni-directional, bi-directional, multiplicity)

What it is

An **association** means one class keeps a *reference* (a field) to another class and can call its methods over a long period — longer than one method call. Neither side owns the other. Destroy one and the other is unaffected.

Analogy: a **ticket** refers to a **parking spot**. Throw the ticket away and the spot is still in the building.

Three sub-ideas come with it:

- **Direction (navigability).** "Navigable" means "can be reached from". In a **uni-directional** association only one side has a field pointing at the other. In a **bi-directional** association both sides have fields pointing at each other.
- **Multiplicity.** How many objects take part at each end: `1`, `0..1` (optional), `*` or `0..*` (any number), `1..*` (at least one), `2` (exactly two), `2..4` (a range).
- **Role name / association name.** An optional label on the line such as `assigned to` or `borrower`.

The problem it solves

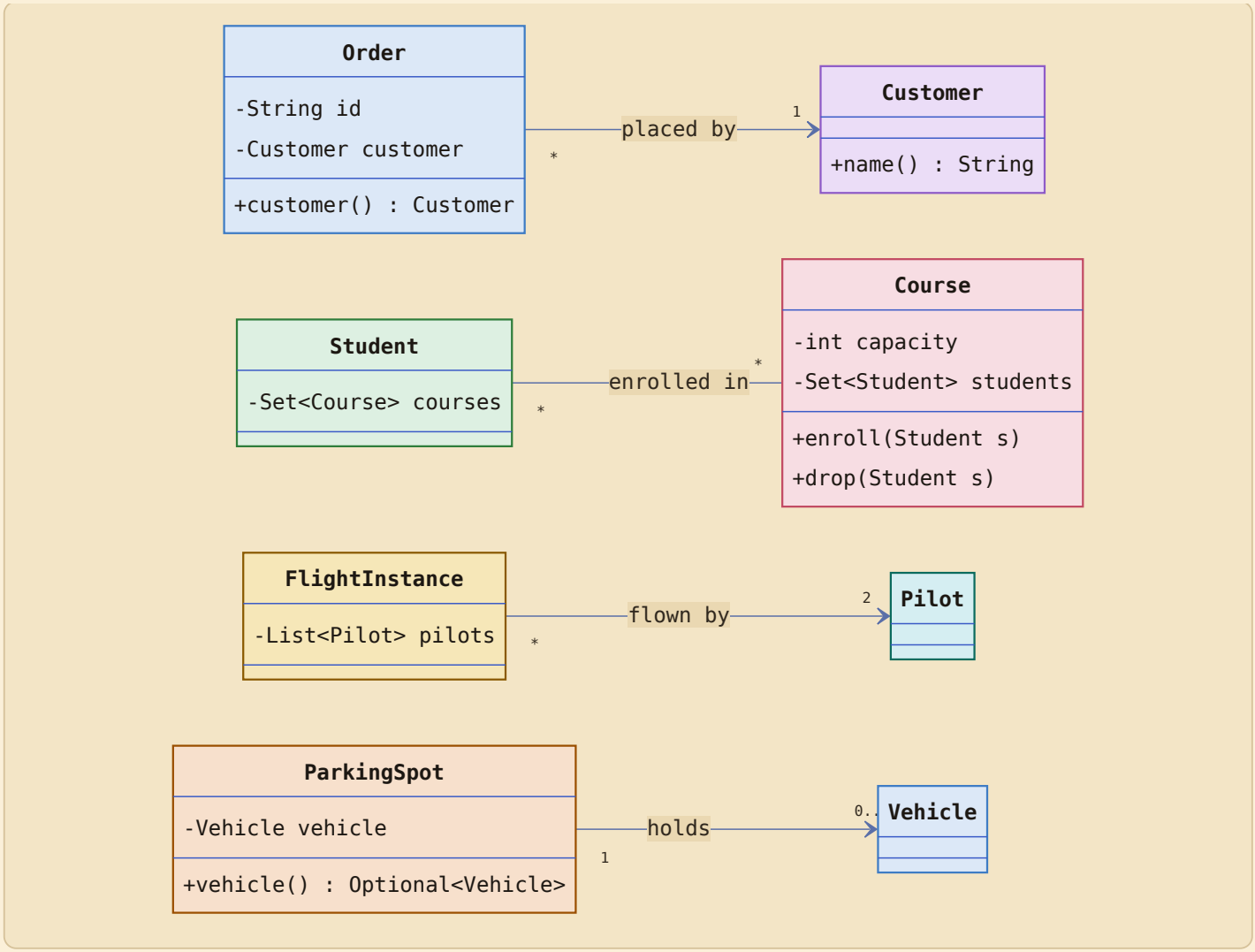
Objects need to collaborate over time. An `Order` must always know which `Customer` placed it. Association is the plain, honest way to record "A knows B" without claiming ownership. It is the most common relationship in a real design, and that is correct.

How it works

1. Decide who needs to call whom. If only `Order` ever asks for its customer, make it **uni-directional**: put a `customer` field in `Order` only.
2. Only make it **bi-directional** if *both* directions are needed by real use cases (e.g. "list a student's courses" *and* "list a course's students").
3. For a bi-directional link, choose **one side to own the update** and make it update both fields in one method, so the two ends can never disagree.
4. Write the **multiplicity** at each end of the line, and then **enforce it in code**:
    - `1` → the field is required: `Objects.requireNonNull` (throws an error if the value is `null`, i.e. no object) in the constructor.
    - `0..1` → the field may be empty: expose it as `Optional<T>` (a box that either holds one value or is empty, so callers must check).
    - `*` → a collection; `1..*` → a collection that must never be empty.
    - exact numbers like `2` → check the size in the constructor.

UML notation

- Plain solid line = association (UML reads a plain line as "navigability unspecified"; in interviews it is read as "both know each other").
- Solid line with an **open arrowhead** = uni-directional; the class at the tail knows the class at the head.
- An **×** at an end (rarely drawn) = explicitly not navigable in that direction.
- Numbers at each end = multiplicity. The number sits next to the class it counts. `Order "*" --> "1" Customer` reads "many orders, each with exactly one customer".



Java example

```java
// File: code/lld/ch03/association/Customer.java
package lld.ch03.association;

/** Knows nothing about orders: the association Order -> Customer is one-way. */
public record Customer(String name) { }
```

```java
// File: code/lld/ch03/association/Order.java
package lld.ch03.association;

import java.util.Objects;

/**
 * Uni-directional association, multiplicity "many orders to exactly 1 customer".
 * Order holds a reference to Customer; Customer has no field pointing back.
 */
public final class Order {
    private final String id;
    private final Customer customer;   // "1" end: never null

    public Order(String id, Customer customer) {
        this.id = id;
        this.customer = Objects.requireNonNull(customer, "an order needs exactly one customer");
    }

    public String id() { return id; }
    public Customer customer() { return customer; }
}
```

```java
// File: code/lld/ch03/association/Student.java
package lld.ch03.association;

import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.Set;

/** One side of a bi-directional many-to-many association (Student * -- * Course). */
public final class Student {
    private final String name;
    private final Set<Course> courses = new LinkedHashSet<>();

    public Student(String name) { this.name = name; }

    public String name() { return name; }
    public Set<Course> courses() { return Collections.unmodifiableSet(courses); }

    /** Package-private: only Course.enroll/drop may touch this side, so both ends stay in sync. */
    void linkCourse(Course c)   { courses.add(c); }
    void unlinkCourse(Course c) { courses.remove(c); }

    @Override public String toString() { return name; }
}
```

```java
// File: code/lld/ch03/association/Course.java
package lld.ch03.association;

import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.Set;

/**
 * Other side of the bi-directional association. Course is the "owner" of the link:
 * enroll/drop update BOTH ends in one place, which is the main rule for two-way links.
 * Multiplicity on this end: a course holds 0..capacity students.
 */
public final class Course {
    private final String code;
    private final int capacity;
    private final Set<Student> students = new LinkedHashSet<>();

    public Course(String code, int capacity) {
        this.code = code;
        this.capacity = capacity;
    }

    public void enroll(Student s) {
        if (students.contains(s)) return;
        if (students.size() == capacity) {
            throw new IllegalStateException(code + " is full (max " + capacity + ")");
        }
        students.add(s);
        s.linkCourse(this);
    }

    public void drop(Student s) {
        if (students.remove(s)) s.unlinkCourse(this);
    }

    public Set<Student> students() { return Collections.unmodifiableSet(students); }
    public String code() { return code; }

    @Override public String toString() { return code; }
}
```

```java
// File: code/lld/ch03/association/Pilot.java
package lld.ch03.association;

public record Pilot(String name) { }
```

```java
// File: code/lld/ch03/association/FlightInstance.java
package lld.ch03.association;

import java.util.List;

/** Multiplicity "exactly 2": the constructor enforces the number written on the diagram. */
public final class FlightInstance {
    private final String flightNo;
    private final List<Pilot> pilots;

    public FlightInstance(String flightNo, List<Pilot> pilots) {
        if (pilots.size() != 2) {
            throw new IllegalArgumentException("a flight needs exactly 2 pilots, got " + pilots.size());
        }
        this.flightNo = flightNo;
        this.pilots = List.copyOf(pilots);
    }

    public List<Pilot> pilots() { return pilots; }
    public String flightNo() { return flightNo; }
}
```

```java
// File: code/lld/ch03/association/Vehicle.java
package lld.ch03.association;

public record Vehicle(String plate) { }
```

```java
// File: code/lld/ch03/association/ParkingSpot.java
package lld.ch03.association;

import java.util.Optional;

/** Multiplicity "0..1": the field may be empty, so the getter returns Optional and callers must check. */
public final class ParkingSpot {
    private final int number;
    private Vehicle vehicle;   // null means "0" of 0..1

    public ParkingSpot(int number) { this.number = number; }

    public void park(Vehicle v) {
        if (vehicle != null) throw new IllegalStateException("spot " + number + " is taken");
        vehicle = v;
    }

    public void leave() { vehicle = null; }

    public Optional<Vehicle> vehicle() { return Optional.ofNullable(vehicle); }
    public int number() { return number; }
}
```

```java
// File: code/lld/ch03/association/Main.java
package lld.ch03.association;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        System.out.println("== Uni-directional: Order -> Customer ==");
        Customer asha = new Customer("Asha");
        Order o1 = new Order("O-1", asha);
        Order o2 = new Order("O-2", asha);
        System.out.println(o1.id() + " belongs to " + o1.customer().name());
        System.out.println(o2.id() + " belongs to " + o2.customer().name());
        System.out.println("Customer has no way to list its orders (one-way link).");

        System.out.println("\n== Bi-directional many-to-many: Student <-> Course ==");
        Student ravi = new Student("Ravi");
        Student meera = new Student("Meera");
        Course java = new Course("JAVA101", 2);
        Course uml = new Course("UML201", 30);
        java.enroll(ravi);
        java.enroll(meera);
        uml.enroll(ravi);
        System.out.println("JAVA101 students: " + java.students());
        System.out.println("Ravi's courses:   " + ravi.courses());
        try {
            java.enroll(new Student("Kiran"));
        } catch (IllegalStateException e) {
            System.out.println("Multiplicity 0..2 enforced: " + e.getMessage());
        }
        java.drop(ravi);
        System.out.println("After drop, JAVA101: " + java.students() + ", Ravi: " + ravi.courses());

        System.out.println("\n== Multiplicity 2 and 0..1 ==");
        FlightInstance f = new FlightInstance("AI-202", List.of(new Pilot("Capt. Rao"), new Pilot("F/O Das")));
        System.out.println(f.flightNo() + " pilots: " + f.pilots());
        try {
            new FlightInstance("AI-303", List.of(new Pilot("Solo")));
        } catch (IllegalArgumentException e) {
            System.out.println("Rejected: " + e.getMessage());
        }
        ParkingSpot spot = new ParkingSpot(7);
        System.out.println("Spot 7 vehicle: " + spot.vehicle().map(Vehicle::plate).orElse("<empty>"));
        spot.park(new Vehicle("KA-01-1234"));
        System.out.println("Spot 7 vehicle: " + spot.vehicle().map(Vehicle::plate).orElse("<empty>"));
    }
}
```

What each part shows:

- `Order` has a `final Customer customer` field and `Customer` has nothing pointing back → **uni-directional**, multiplicity `1` enforced with `requireNonNull`.
- `Student` and `Course` each hold a `Set` of the other → **bi-directional many-to-many**. Only `Course.enroll/drop` may change the link; `Student.linkCourse` is package-private (no access keyword, so only classes in the same Java package can call it). This single "owning side" rule is what keeps both ends consistent.
- `Course(capacity)` turns multiplicity `0..capacity` into a runtime check.
- `FlightInstance` rejects anything but exactly **2** pilots; `ParkingSpot` exposes **0..1** as `Optional<Vehicle>`.
- Lifetimes: every object here is created by `Main` and merely *referenced* by the others. Dropping a course does not delete the student.

Output:

```
== Uni-directional: Order -> Customer ==
O-1 belongs to Asha
O-2 belongs to Asha
Customer has no way to list its orders (one-way link).

== Bi-directional many-to-many: Student <-> Course ==
JAVA101 students: [Ravi, Meera]
Ravi's courses:   [JAVA101, UML201]
Multiplicity 0..2 enforced: JAVA101 is full (max 2)
After drop, JAVA101: [Meera], Ravi: [UML201]

== Multiplicity 2 and 0..1 ==
AI-202 pilots: [Pilot[name=Capt. Rao], Pilot[name=F/O Das]]
Rejected: a flight needs exactly 2 pilots, got 1
Spot 7 vehicle: <empty>
Spot 7 vehicle: KA-01-1234
```

Trade-offs

Choice	Pros	Cons
Uni-directional	Simple; one field; no sync bugs; lower coupling	Reverse question ("which orders does this customer have?") needs a query or repository (a class that stores and looks up objects)
Bi-directional	Fast navigation both ways	Two fields to keep in sync; tighter coupling; risk of infinite loops in `toString` / `equals` /JSON serialisation (turning objects into JSON, a common text data format)
Strict multiplicity checks	Invalid states cannot be created	A little more constructor code

When not to use it

- Do not store a long-lived field if you only need the object inside one method — that is a **dependency** (3.5), which couples less.
- Do not default to bi-directional. Most associations should be uni-directional.

Common mistakes

- Making every line bi-directional "just in case".
- Updating only one side of a bi-directional link (`student.courses.add(c)` without `course.students.add(s)`).
- Forgetting multiplicity on the line, then forgetting the null-check / non-empty check in the code.
- Including both sides in `toString()` or `equals()` → stack overflow from infinite recursion (each object's method calls the other's, forever, until Java runs out of room to track the calls).

Real-world examples

- **JPA / Hibernate** (Java's standard API for saving objects into SQL database tables, and its most popular implementation): `@ManyToOne` on `Order.customer` is a uni-directional association. A bi-directional `@OneToMany(mappedBy = "customer")` needs an *owning side* (the side with the foreign key, the column that stores the other row's ID) and helper methods that set both ends — exactly the `Course.enroll` rule above.
- **Spring** (a popular Java application framework): a service with a constructor-injected repository (one handed in through the constructor) holds an association to it; the Spring container (the part of Spring that creates and wires objects), not the service, creates and destroys the repository.
- **E-commerce:** an order row stores `customer_id` (Order → Customer); a customer does not store a list of order ids.

In the interview

30-second answer: "Association means one class holds a reference to another and can call it, with no ownership. I make it uni-directional unless both directions are needed, and I write multiplicity on every

line because it changes the code: `1` is a required field, `0..1` is an Optional, `1..*` is a non-empty collection."

Follow-ups:

- *Q: How do you keep a bi-directional association consistent?* — A: Pick one owning side and put a single method there (`enroll`) that updates both ends; make the other side's mutators (methods that change state) package-private.
- *Q: What changes if a parking spot can hold two motorbikes?* — A: The multiplicity changes from `0..1` to `0..2`, so the field becomes a small collection with a size check instead of an `Optional`.
- *Q: Why not make everything bi-directional?* — A: It doubles the places that must change together and increases coupling; the reverse direction can usually be answered by a repository query.

Self-check

- *Q: `Order "*" --> "1" Customer` — which class has the field?* → A: `Order`; the arrow points at the class being referenced.
- *Q: What Java type fits multiplicity `0..1`?* → A: A nullable field exposed as `Optional<T>`.
- *Q: Is a many-to-many association always bi-directional?* → A: No. "Many-to-many" is about counts; direction is a separate choice.

TL;DR

Association = "A knows B", no ownership. Prefer one-way. Put multiplicity on every line and enforce it in the constructor/types.

3.3 Aggregation

What it is

Aggregation is a special association that says "A is a *whole* made of parts B, but the parts have their own life". The parts exist before the whole, can move to another whole, and survive when the whole is destroyed. It is sometimes called *shared aggregation* because a part may even belong to several wholes.

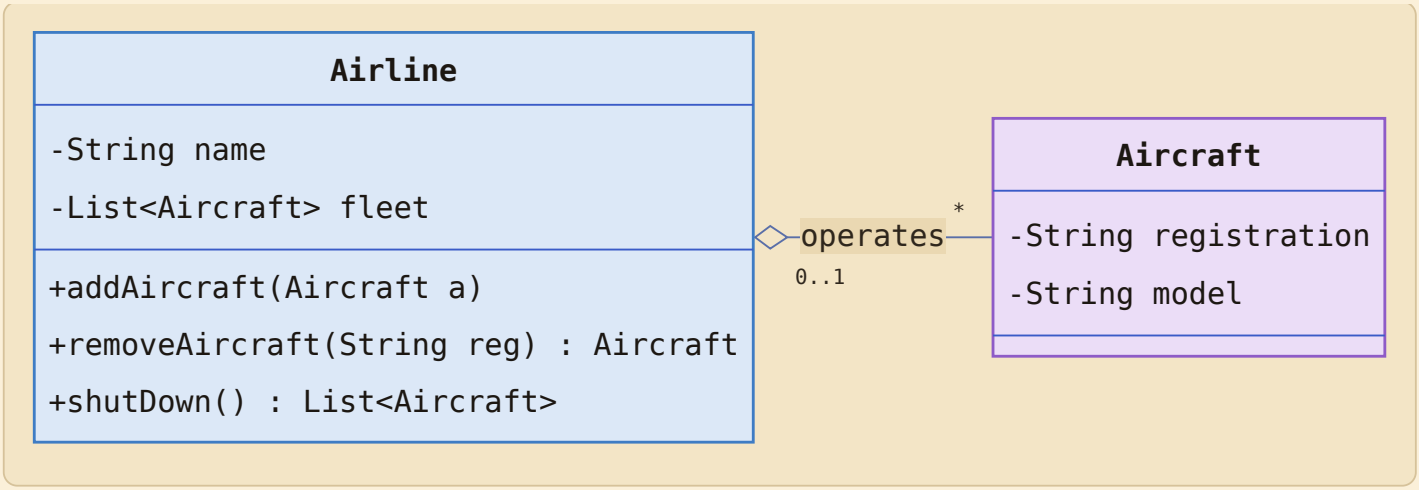
Analogy: an **airline** and its **aircraft**. If the airline shuts down, the planes are sold to another airline — they are not scrapped. A **team** and its **players** is another: disband the team and the players still exist.

The problem it solves

It lets you say "these objects form a group" (a whole–part meaning, stronger than a plain association) while making clear that the group does **not** control their lifetime. That tells readers: "do not delete the parts when you delete the whole".

How it works

1. The part is **created outside** the whole (by the caller, a factory, or a repository).
2. The part is **passed in** (`addAircraft(a)`), and the whole stores a reference.
3. Removing the part from the whole **returns it alive**; destroying the whole **releases** its parts.
4. In UML, draw a **hollow diamond at the whole's end** of the line.



Java example

```java
// File: code/lld/ch03/aggregation/Aircraft.java
package lld.ch03.aggregation;

/** A part that has its own life: it is created outside any airline and can move between airlines. */
public final class Aircraft {
    private final String registration;
    private final String model;

    public Aircraft(String registration, String model) {
        this.registration = registration;
        this.model = model;
    }

    public String registration() { return registration; }

    @Override public String toString() { return registration + " (" + model + ")"; }
}
```

```java
// File: code/lld/ch03/aggregation/Airline.java
package lld.ch03.aggregation;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Aggregation (hollow diamond at Airline): Airline 0..1 o-- * Aircraft
 * (an aircraft belongs to at most one airline at a time and may belong to none).
 * The airline HOLDS aircraft that were created elsewhere and handed in.
 * It never creates them and never destroys them.
 */
public final class Airline {
    private final String name;
    private final List<Aircraft> fleet = new ArrayList<>();

    public Airline(String name) { this.name = name; }

    public void addAircraft(Aircraft a) { fleet.add(a); }        // part is passed in

    public Aircraft removeAircraft(String registration) {         // part is handed back, still alive
        for (var it = fleet.iterator(); it.hasNext(); ) {
            Aircraft a = it.next();
            if (a.registration().equals(registration)) { it.remove(); return a; }
        }
        throw new IllegalArgumentException("no aircraft " + registration + " in " + name);
    }

    /** Shutting down the airline releases the parts instead of destroying them. */
    public List<Aircraft> shutDown() {
        List<Aircraft> released = List.copyOf(fleet);
        fleet.clear();
        return released;
    }

    public List<Aircraft> fleet() { return Collections.unmodifiableList(fleet); }
    public String name() { return name; }
}
```

```java
// File: code/lld/ch03/aggregation/Main.java
package lld.ch03.aggregation;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        // The parts exist BEFORE any whole.
        Aircraft a1 = new Aircraft("VT-AAA", "A320");
        Aircraft a2 = new Aircraft("VT-BBB", "B737");
        Aircraft a3 = new Aircraft("VT-CCC", "ATR72");

        Airline skyways = new Airline("Skyways");
        skyways.addAircraft(a1);
        skyways.addAircraft(a2);
        Airline blueJet = new Airline("BlueJet");
        blueJet.addAircraft(a3);

        System.out.println(skyways.name() + " fleet: " + skyways.fleet());
        System.out.println(blueJet.name() + " fleet: " + blueJet.fleet());

        // Transfer one part between wholes.
        blueJet.addAircraft(skyways.removeAircraft("VT-BBB"));
        System.out.println("After transfer, BlueJet fleet: " + blueJet.fleet());

        // Destroy a whole: its parts survive and can join another whole.
        List<Aircraft> released = skyways.shutDown();
        skyways = null;   // the Airline object is now unreachable
        System.out.println("Skyways shut down. Released aircraft still exist: " + released);
        released.forEach(blueJet::addAircraft);
        System.out.println("BlueJet fleet now: " + blueJet.fleet());
    }
}
```

What each part shows: `Aircraft` objects are built in `Main` *before* any airline exists. `Airline.addAircraft` receives them; `removeAircraft` and `shutDown` hand them back. After `skyways = null` the airline is garbage (nothing points to it any more, so Java's garbage collector, which frees the memory of unreachable objects, will reclaim it), but its aircraft are still referenced and join `BlueJet`.

Output:

```
Skyways fleet: [VT-AAA (A320), VT-BBB (B737)]
BlueJet fleet: [VT-CCC (ATR72)]
After transfer, BlueJet fleet: [VT-CCC (ATR72), VT-BBB (B737)]
Skyways shut down. Released aircraft still exist: [VT-AAA (A320)]
BlueJet fleet now: [VT-CCC (ATR72), VT-BBB (B737), VT-AAA (A320)]
```

Trade-offs

	Aggregation	Plain association
Extra meaning	"whole–part" grouping	just "knows"
Lifetime claim	parts outlive whole	none
Code difference	usually **none** — both are a field holding a reference	—

The honest trade-off: aggregation adds meaning for the reader but almost nothing in code. The UML specification itself leaves the precise meaning of shared aggregation to the modeller, and several well-known UML authors (Martin Fowler's *UML Distilled* among them) advise using it sparingly. In interviews, it is still expected when there is a clear whole–part grouping.

When not to use it

- If there is no "whole made of parts" idea (a ticket and a spot), use plain association.
- If the parts are created by and die with the whole, it is composition.

Common mistakes

- Putting the diamond at the **part's** end. The diamond always sits at the **whole**.
- Choosing aggregation vs composition at random. Use the test: *imagine the whole is gone — is the part still meaningful by itself?* Yes → aggregation.
- Deleting parts in the whole's "close/delete" method even though the diagram says aggregation.

Real-world examples

- A `java.util.List<Customer>`: clearing the list does not destroy the customers; they are still referenced elsewhere.
- A Slack/Teams **channel** and its **members**: archiving a channel does not delete user accounts.
- A **playlist** and **songs** (Spotify): deleting your playlist does not delete the songs from the catalogue.
- A **department** and its **professors** in a university model.

In the interview

> **30-second answer:** "Aggregation is a whole–part association where the parts can outlive the whole — hollow diamond at the whole. An airline aggregates aircraft: shut the airline down and the aircraft still exist. In Java the parts are created outside and passed in, and the whole never destroys them."

Follow-ups:

- *Q: Can an airline exist without aircraft?* — A: That is a multiplicity question, not an aggregation question. If the business says an airline must have at least one aircraft, write `1..*` on the aircraft end; it is still aggregation because the aircraft survive the airline.
- *Q: How is aggregation different in code from association?* — A: Usually it is not; the difference is the modelling intent (whole–part). That is why you say the lifetime test out loud.

Self-check

- *Q: Where does the hollow diamond go?* → A: At the whole (`Airline`).
- *Q: A library closes; do its members disappear?* → A: No — a person exists either way, so Library ◇— Member is aggregation.

TL;DR

Aggregation = "has-a, parts live on". Parts are passed in, never destroyed by the whole. Hollow diamond at the whole.

3.4 Composition

What it is

Composition is the strongest has-a relationship: the whole **owns** its parts. A part belongs to **exactly one** whole at a time, is usually **created by** the whole, and is **destroyed with** it. (UML also allows a part to be explicitly removed from the whole *before* the whole is deleted.)

Analogy: a **house** and its **rooms**. Demolish the house and the rooms are gone — a "room of a house that does not exist" means nothing. Likewise a **parking floor** and its **spots**, a **parking lot** and its **floors**, a **flight** and its **weekly schedule**.

The problem it solves

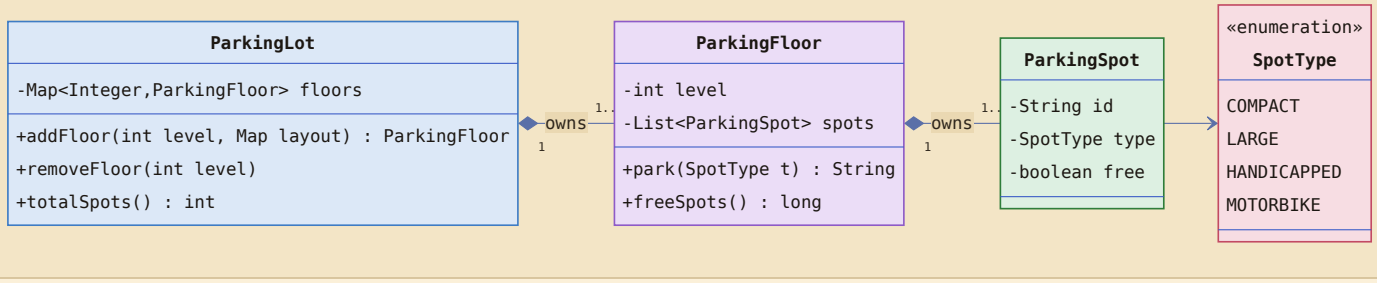
Some objects make no sense on their own, and letting outsiders create or share them causes bugs (two floors pointing at the same spot, orphan spots after a floor is removed). Composition makes the owner responsible for the parts' full life cycle, so there is one place that creates, changes and removes them.

How it works

1. The whole **creates** the parts itself (in its constructor or an `addX(...)` method that takes *data*, not a ready-made part).

2. The part's constructor is **not public** (package-private or a private nested class, i.e. a class declared inside the whole's class) so nobody else can make one.
3. The whole **never hands out** its internal collection — only read-only views or copies.
4. **Removing** the whole (or removing a part from it) drops the only reference, so the part becomes garbage.
5. In UML, draw a **filled (black) diamond at the whole's end**.

A note on Java: Java has no destructors (methods that run automatically when an object is destroyed, as in C++). "The part dies with the whole" means *logically*: nobody else holds a reference, so when the whole is removed the part becomes unreachable and the garbage collector reclaims it. If you need clean-up (closing files, sockets, i.e. open network connections), the whole's `close()` must close its parts too — for example, closing a `BufferedReader` also closes the `FileReader` it wraps.



Java example

```java
// File: code/lld/ch03/composition/SpotType.java
package lld.ch03.composition;

/** A spot "kind" is a value, not a subclass (see the generalization chapter section). */
public enum SpotType { COMPACT, LARGE, HANDICAPPED, MOTORBIKE }
```

```java
// File: code/lld/ch03/composition/ParkingSpot.java
package lld.ch03.composition;

/**
 * A part of a composition. The constructor is package-private, so only ParkingFloor
 * (same package) can create a spot. Outside code can read a spot but never make one.
 */
public final class ParkingSpot {
    private final String id;
    private final SpotType type;
    private boolean free = true;

    ParkingSpot(String id, SpotType type) {
        this.id = id;
        this.type = type;
    }

    public String id() { return id; }
    public SpotType type() { return type; }
    public boolean isFree() { return free; }
    void occupy() { free = false; }

    @Override public String toString() { return id + ":" + type; }
}
```

```java
// File: code/lld/ch03/composition/ParkingFloor.java
package lld.ch03.composition;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Map;

/**
 * Composition (filled diamond at ParkingFloor): ParkingFloor 1 *-- 1..* ParkingSpot.
 * The floor CREATES its spots in its own constructor, never shares them with another floor,
 * and when the floor is removed the spots go with it.
 */
public final class ParkingFloor {
    private final int level;
    private final List<ParkingSpot> spots = new ArrayList<>();

    ParkingFloor(int level, Map<SpotType, Integer> layout) {
        this.level = level;
        int n = 1;
        for (var e : layout.entrySet()) {
            for (int i = 0; i < e.getValue(); i++) {
                spots.add(new ParkingSpot("L" + level + "-" + n++, e.getKey()));
            }
        }
        if (spots.isEmpty()) throw new IllegalArgumentException("a floor needs 1..* spots");
    }

    public int level() { return level; }
    public List<ParkingSpot> spots() { return Collections.unmodifiableList(spots); }

    public long freeSpots() { return spots.stream().filter(ParkingSpot::isFree).count(); }

    /** Parks in the first free spot of the type; the spot itself never leaves the floor. */
    public String park(SpotType type) {
        for (ParkingSpot s : spots) {
            if (s.isFree() && s.type() == type) { s.occupy(); return s.id(); }
        }
        return null;
    }
}
```

```java
// File: code/lld/ch03/composition/ParkingLot.java
package lld.ch03.composition;

import java.util.Collections;
import java.util.Map;
import java.util.TreeMap;

/** Composition again: ParkingLot 1 *-- 1..* ParkingFloor. The lot builds and removes its floors. */
public final class ParkingLot {
    private final String name;
    private final Map<Integer, ParkingFloor> floors = new TreeMap<>();

    public ParkingLot(String name) { this.name = name; }

    /** The caller describes the floor; the lot constructs it. No ParkingFloor object is passed in. */
    public ParkingFloor addFloor(int level, Map<SpotType, Integer> layout) {
        ParkingFloor f = new ParkingFloor(level, layout);
        floors.put(level, f);
        return f;
    }

    /** Removing the whole removes its parts: no one else holds the spots, so they become garbage. */
    public void removeFloor(int level) { floors.remove(level); }

    public int totalSpots() {
        return floors.values().stream().mapToInt(f -> f.spots().size()).sum();
    }

    public Map<Integer, ParkingFloor> floors() { return Collections.unmodifiableMap(floors); }
    public String name() { return name; }
}
```

```java
// File: code/lld/ch03/composition/Main.java
package lld.ch03.composition;

import java.util.LinkedHashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        ParkingLot lot = new ParkingLot("City Centre");
        Map<SpotType, Integer> layout = new LinkedHashMap<>();
        layout.put(SpotType.COMPACT, 3);
        layout.put(SpotType.LARGE, 1);

        ParkingFloor ground = lot.addFloor(0, layout);
        lot.addFloor(1, Map.of(SpotType.MOTORBIKE, 4));

        System.out.println("Floor 0 spots: " + ground.spots());
        System.out.println("Total spots in lot: " + lot.totalSpots());
        System.out.println("Parked compact car at " + ground.park(SpotType.COMPACT));
        System.out.println("Free spots on floor 0: " + ground.freeSpots());

        try {
            ground.spots().clear();
        } catch (UnsupportedOperationException e) {
            System.out.println("Outsiders cannot change the parts: spots() is read-only");
        }

        lot.removeFloor(1);
        System.out.println("After removing floor 1, total spots: " + lot.totalSpots()
                + " (its 4 spots are gone with it)");
        System.out.println("Floors left: " + lot.floors().keySet());
    }
}
```

What each part shows: `ParkingLot.addFloor` receives a *layout description* and builds the `ParkingFloor` itself. `ParkingFloor`'s constructor builds its `ParkingSpot`s; `ParkingSpot`'s constructor is package-private, so code outside the package cannot create a spot. `spots()` returns an unmodifiable view, so outsiders cannot add or remove parts. `removeFloor(1)` drops the floor and, with it, its four spots.

Output:

```
Floor 0 spots: [L0-1:COMPACT, L0-2:COMPACT, L0-3:COMPACT, L0-4:LARGE]
Total spots in lot: 8
Parked compact car at L0-1
Free spots on floor 0: 3
Outsiders cannot change the parts: spots() is read-only
After removing floor 1, total spots: 4 (its 4 spots are gone with it)
Floors left: [0]
```

Trade-offs

	Composition	Aggregation
Who creates the part	the whole	someone else
Can the part be shared?	no (one owner at a time)	yes
Part outlives whole?	no	yes
Encapsulation	strong — whole controls all changes	weaker
Flexibility	lower — parts cannot be reused elsewhere	higher

When not to use it

- When the part has an independent identity the business cares about (a *person*, an *aircraft*, a *song*) — use aggregation/association.
- When the part must be shared between wholes.

Common mistakes

- Accepting a ready-made part from outside (`addSpot(ParkingSpot s)`) and calling it composition — the caller may still hold and share it.
- Returning the internal mutable list (`return spots;`) — outsiders can now add/remove parts behind the owner's back.
- Mixing up composition (object design) with "favour composition over inheritance" (3.6): in that slogan "composition" means *holding* an object instead of extending its class, and it covers association/aggregation too.

Real-world examples

- `java.util.HashMap` creates and manages its internal `Node` entries (a package-private nested class); they are never shared with another map and go away with it.
- **JPA:** `@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)` models composition — delete the `Order` and its `OrderLine` rows are deleted too.
- **Documents:** deleting a Google Doc deletes its comments and revision history.
- **Operating systems:** killing a process (a running program) frees its threads (its parallel lines of execution) and memory pages (the blocks of memory the OS gave it).

In the interview

30-second answer: "Composition means the whole owns the parts: it creates them, they belong to only it, and they die with it — filled diamond at the whole. A parking floor composes its spots: remove the floor and the spots are gone. In Java I build the parts inside the owner, hide their constructor, and only expose read-only views."

Follow-ups:

- *Q: Java has garbage collection — how can a part 'die'?* — A: It dies logically: the whole holds the only reference, so removing the whole makes the part unreachable. If the part holds resources, the whole's `close()` closes it.
- *Q: Is `Flight` → `WeeklySchedule` composition?* — A: Yes; a schedule of a flight that no longer exists is meaningless.

Self-check

- *Q: The test that separates aggregation from composition?* → A: Imagine the whole is gone — is the part still meaningful by itself? No → composition.
- *Q: Why is `ParkingSpot`'s constructor package-private?* → A: So only the owning floor can create spots.

TL;DR

Composition = "owns, parts die with it". Whole creates parts, never shares them, exposes read-only views. Filled diamond at the whole.

3.5 Dependency

What it is

A **dependency** means class A *uses* class B for a short time — inside one method — without keeping a field that points at B. B appears as a **method parameter**, a **local variable**, a **return type**, or a **static method call**. When the method returns, the relationship is over.

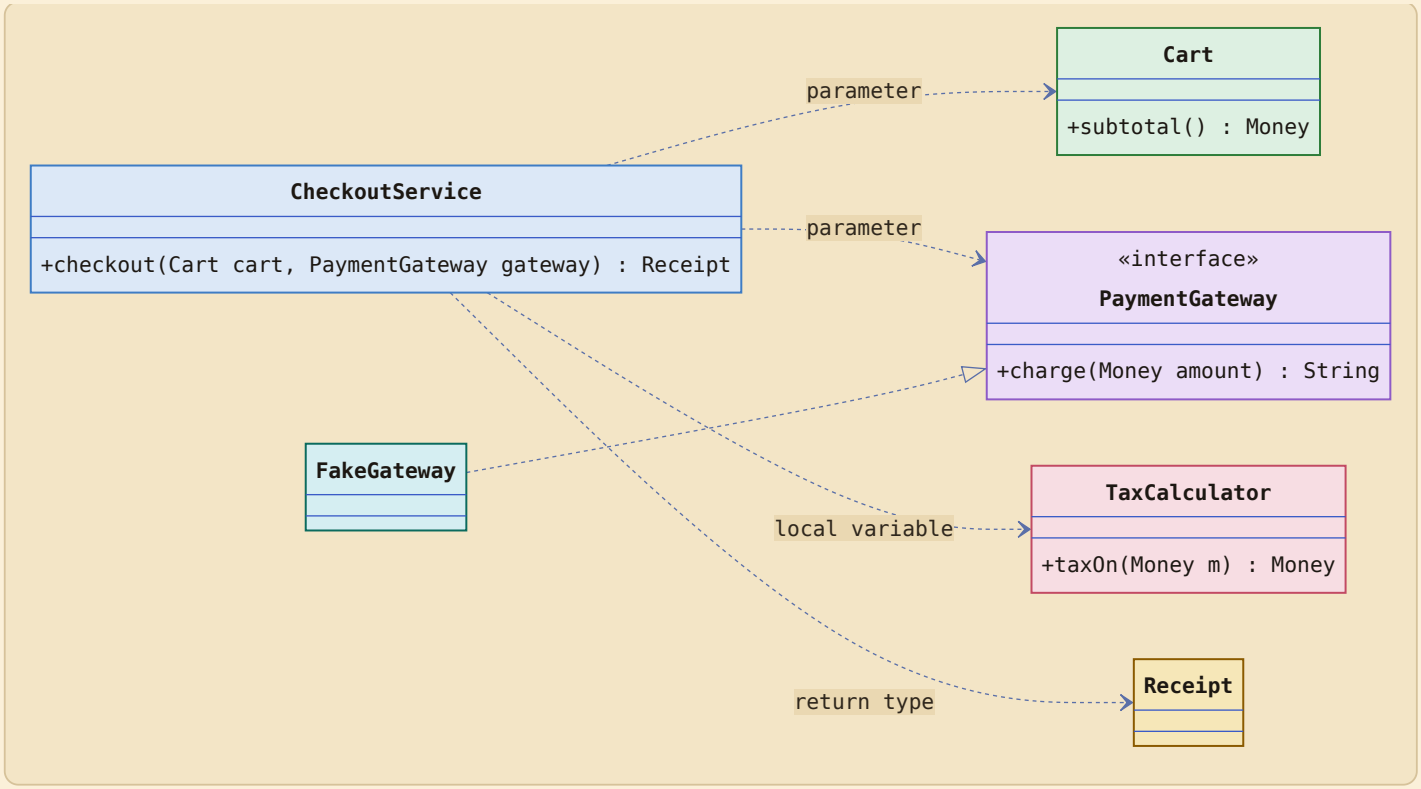
Analogy: a **chef** *uses* a **delivery van** to receive groceries this morning. The chef does not own the van and does not keep it in the kitchen; tomorrow it may be a different van.

The problem it solves

It records the weakest possible link: "if B's interface changes, A may have to change", nothing more. Recognising a dependency (instead of an association) keeps classes stateless (holding no data between calls) and loosely coupled — the caller can pass in a different collaborator on every call.

How it works

1. A method receives B as a parameter (`checkout(Cart cart, PaymentGateway gateway)`), or
2. creates B as a local variable (`new TaxCalculator()`), or
3. returns a B (`Receipt`), or calls a static method on B.
4. No field of type B exists in A.
5. In UML, draw a **dashed line with an open arrowhead** from the user (A) to the used class (B). You may add a keyword such as `«use»` or `«create»`.



Java example

```java
// File: code/lld/ch03/dependency/Money.java
package lld.ch03.dependency;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** A small value type. Anything that returns or accepts Money depends on it. */
public record Money(BigDecimal amount) {
    public static Money of(String s) { return new Money(new BigDecimal(s).setScale(2, RoundingMode.HALF_UP)); }
    public Money plus(Money o) { return new Money(amount.add(o.amount)); }
    public Money times(BigDecimal f) { return new Money(amount.multiply(f).setScale(2, RoundingMode.HALF_UP)); }
    @Override public String toString() { return "Rs " + amount.toPlainString(); }
}
```

```java
// File: code/lld/ch03/dependency/Cart.java
package lld.ch03.dependency;

import java.util.List;

public record Cart(List<Money> itemPrices) {
    public Money subtotal() {
        return itemPrices.stream().reduce(Money.of("0"), Money::plus);
    }
}
```

```java
// File: code/lld/ch03/dependency/PaymentGateway.java
package lld.ch03.dependency;

public interface PaymentGateway {
    String charge(Money amount);   // returns a transaction id
}
```

```java
// File: code/lld/ch03/dependency/FakeGateway.java
package lld.ch03.dependency;

public final class FakeGateway implements PaymentGateway {
    private int next = 1;
    @Override public String charge(Money amount) {
        System.out.println("  [gateway] charging " + amount);
        return "TXN-" + next++;
    }
}
```

```java
// File: code/lld/ch03/dependency/TaxCalculator.java
package lld.ch03.dependency;

import java.math.BigDecimal;

public final class TaxCalculator {
    private static final BigDecimal GST = new BigDecimal("0.18");
    public Money taxOn(Money m) { return m.times(GST); }
}
```

```java
// File: code/lld/ch03/dependency/CheckoutService.java
package lld.ch03.dependency;

/**
 * Dependency (dashed arrow, "uses"): CheckoutService has NO fields pointing at
 * Cart, PaymentGateway, TaxCalculator or Receipt. It only touches them for the
 * duration of one method call:
 *   - Cart and PaymentGateway arrive as PARAMETERS,
 *   - TaxCalculator is a LOCAL variable,
 *   - Receipt is the RETURN type.
 * When checkout() returns, the relationship is over.
 */
public final class CheckoutService {

    public Receipt checkout(Cart cart, PaymentGateway gateway) {
        TaxCalculator tax = new TaxCalculator();            // local-variable dependency
        Money subtotal = cart.subtotal();                   // parameter dependency
        Money total = subtotal.plus(tax.taxOn(subtotal));
        String txn = gateway.charge(total);                 // parameter dependency (interface)
        return new Receipt(txn, subtotal, total);           // return-type dependency
    }

    public record Receipt(String txnId, Money subtotal, Money total) { }
}
```

```java
// File: code/lld/ch03/dependency/Main.java
package lld.ch03.dependency;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        CheckoutService service = new CheckoutService();   // holds no collaborators at all
        Cart cart = new Cart(List.of(Money.of("250"), Money.of("99.50")));

        // The caller decides which gateway to use on every call.
        CheckoutService.Receipt r = service.checkout(cart, new FakeGateway());
        System.out.println("Receipt: " + r);

        PaymentGateway loggingGateway = amount -> {
            System.out.println("  [lambda gateway] charging " + amount);
            return "LMB-42";
        };
        System.out.println("Receipt: " + service.checkout(cart, loggingGateway));
    }
}
```

What each part shows: `CheckoutService` has **no fields at all**. It depends on `Cart` and `PaymentGateway` through parameters, on `TaxCalculator` through a local variable, and on `Receipt` through its return type. `Main` passes a different gateway (even a lambda, a short unnamed function written inline) on each call — something you cannot do as easily with a stored field. `Money` uses `BigDecimal` (Java's exact decimal-number type), not `double` (a binary floating-point number that cannot hold values like 0.10 exactly), because currency needs exact decimal arithmetic.

Output:

```
  [gateway] charging Rs 412.41
Receipt: Receipt[txnId=TXN-1, subtotal=Rs 349.50, total=Rs 412.41]
  [lambda gateway] charging Rs 412.41
Receipt: Receipt[txnId=LMB-42, subtotal=Rs 349.50, total=Rs 412.41]
```

Trade-offs

	Dependency (parameter)	Association (field)
Coupling	lowest	higher
Object state	can be stateless	holds state
Swapping collaborator	per call	per object (or via setter)
Call-site noise	caller passes B every time	caller passes B once

When not to use it

If A needs B across many methods and many calls (e.g. a service always uses the same repository), a stored field (association via constructor injection, where the collaborator is passed into the constructor once) is cleaner than passing B to every method.

Common mistakes

- Drawing a solid association line for something that is only a parameter — it overstates coupling.
- Drawing *every* dependency: a real class depends on `String`, `List`, `LocalDate`… Draw only the ones that matter to the design.
- Hiding a dependency with `new ConcreteThing()` deep inside a method when the caller should decide (this blocks testing). Prefer taking an interface as a parameter.

Real-world examples

- `Collections.sort(list, comparator)` depends on the `Comparator` only during the call.

- A Spring MVC controller method (the code that answers one web request) `handle(HttpServletRequest req)` depends on the request only for that call.
- `LocalDate.now(Clock clock)` depends on a `Clock` parameter — the JDK's way to make time testable.
- In build tools (Maven/Gradle, programs that download libraries and compile a Java project) a "dependency" is the same idea at package level: your code uses a library's classes.

In the interview

> **30-second answer:** "Dependency is 'uses briefly': B is a parameter, local variable or return type of A's method, and A keeps no field for it. Dashed arrow from A to B. It is the loosest coupling, so when a collaborator is only needed in one method I pass it in rather than store it."

Follow-ups:

- *Q: `FlightReservation` takes a `Payment` in `pay(Payment p)` — association or dependency?* — A: Dependency, unless the reservation stores the payment in a field; then it becomes an association.
- *Q: How is this related to Dependency Injection?* — A: DI is about *who supplies* the collaborator (from outside, not `new` inside). The supplied object may be stored (association) or passed per call (dependency). The Dependency Inversion Principle adds: depend on interfaces, not concrete classes (Chapter 4).

Self-check

- *Q: Which UML line: dashed + open arrow, or dashed + hollow triangle?* → A: Dashed + open arrow is dependency; dashed + hollow triangle is realization.
- *Q: A method returns a `Receipt`. Does that create a relationship?* → A: Yes, a dependency on `Receipt`.

TL;DR

Dependency = "uses for one call". Parameter, local variable, return type or static call; no field. Dashed arrow.

3.6 Generalization (inheritance)

What it is

Generalization is UML's word for **inheritance**: class B `extends` class A, so B *is a kind of* A. A is the **parent** (superclass, general); B is the **child** (subclass, specialised). B inherits A's fields and methods and may override them.

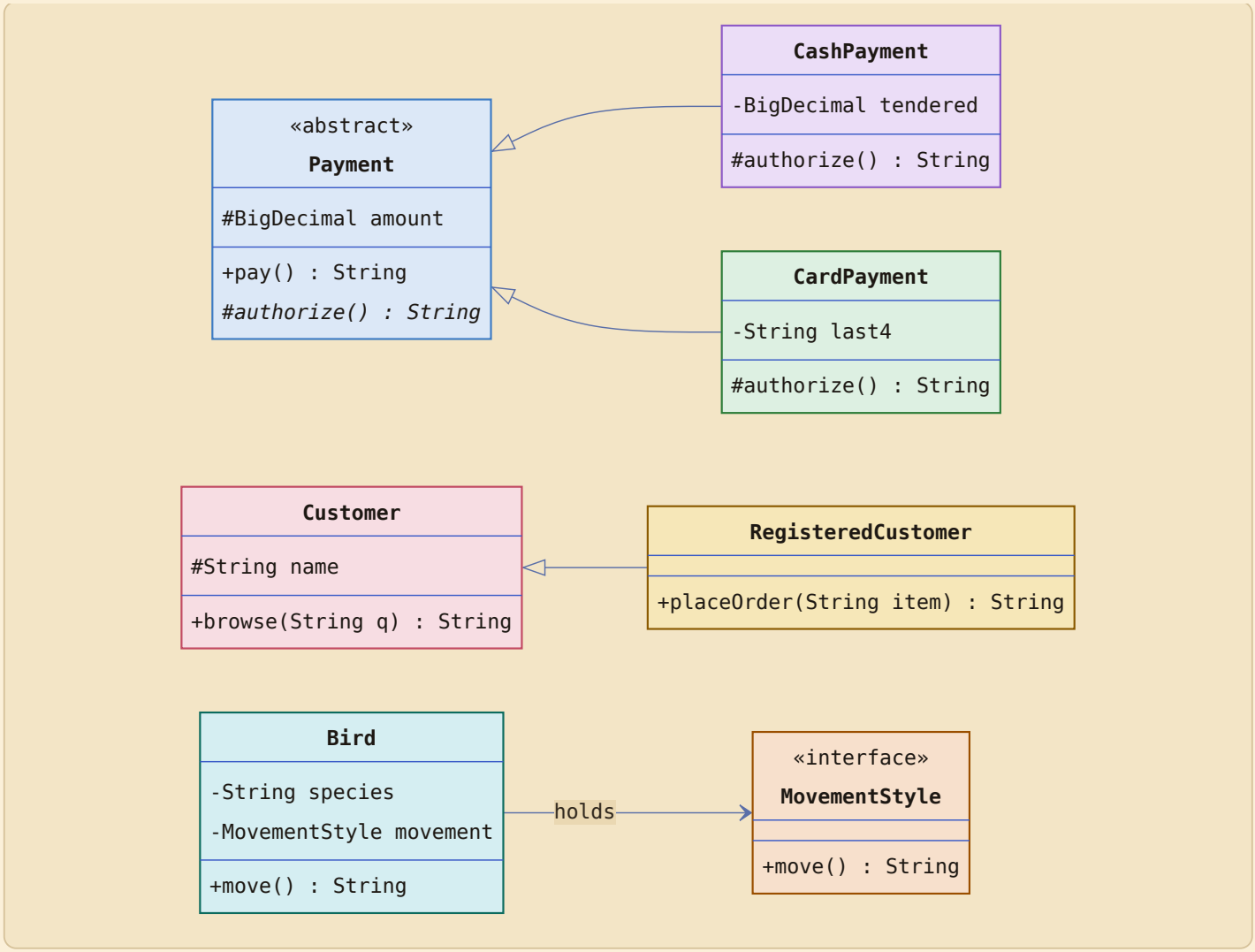
Analogy: a **credit-card payment** *is a* **payment**. Anywhere the shop's code can handle "a payment", it can handle a credit-card payment.

The problem it solves

When several classes truly share a contract **and** some behaviour, inheritance puts the shared parts in one place and lets calling code work with the general type (`Payment`) without knowing the specific one (**polymorphism** — one call, different behaviour depending on the actual object).

How it works

1. Before extending, run the **substitution test**: *can B be used anywhere A is expected, with no surprises?* If there is any method of A that B would have to refuse or break, the answer is no. (This is the Liskov Substitution Principle — Chapter 4.)
2. If yes, put the shared state and workflow in A (often `abstract`: a class that cannot be instantiated and may leave some methods for subclasses to fill in), and let subclasses override only the varying steps.
3. If no, **hold instead of extend**: give A a field of an interface type and plug in different behaviour objects.
4. In UML, draw a **solid line with a hollow triangle pointing at the parent**. An abstract class is shown with its name in *italics* (or `{abstract}` / `«abstract»`); abstract operations are italic too.



Java example

```java
// File: code/lld/ch03/generalization/Payment.java
package lld.ch03.generalization;

import java.math.BigDecimal;

/**
 * Generalization parent (abstract, drawn in italics or with {abstract}).
 * Every subclass genuinely "is a" Payment: each one really can take money, in its own way.
 * Money is BigDecimal, never double: double cannot represent amounts like 0.10 exactly.
 */
public abstract class Payment {
    protected final BigDecimal amount;   // '#' protected: visible to subclasses

    protected Payment(BigDecimal amount) {
        if (amount.signum() <= 0) throw new IllegalArgumentException("amount must be positive");
        this.amount = amount;
    }

    /** Shared, final workflow; the varying step is abstract. */
    public final String pay() {
        String ref = authorize();
        return getClass().getSimpleName() + " of " + amount + " -> " + ref;
    }

    protected abstract String authorize();   // abstract operation (italics in UML)
}
```

```java
// File: code/lld/ch03/generalization/CashPayment.java
package lld.ch03.generalization;

import java.math.BigDecimal;

public final class CashPayment extends Payment {
    private final BigDecimal tendered;

    public CashPayment(BigDecimal amount, BigDecimal tendered) {
        super(amount);
        this.tendered = tendered;
    }

    @Override protected String authorize() {
        if (tendered.compareTo(amount) < 0) return "DECLINED (short by " + amount.subtract(tendered) + ")";
        return "OK, change " + tendered.subtract(amount);
    }
}
```

```java
// File: code/lld/ch03/generalization/CardPayment.java
package lld.ch03.generalization;

import java.math.BigDecimal;

public final class CardPayment extends Payment {
    private final String last4;

    public CardPayment(BigDecimal amount, String last4) {
        super(amount);
        this.last4 = last4;
    }

    @Override protected String authorize() { return "OK, card ****" + last4 + " approved"; }
}
```

```java
// File: code/lld/ch03/generalization/Customer.java
package lld.ch03.generalization;

/** Base class holds only what EVERY customer can do, so no subclass must refuse anything. */
public class Customer {
    protected final String name;
    public Customer(String name) { this.name = name; }
    public String browse(String query) { return name + " browses '" + query + "'"; }
}
```

```java
// File: code/lld/ch03/generalization/RegisteredCustomer.java
package lld.ch03.generalization;

/** Ordering lives here, not on Customer, because a guest may not order. */
public class RegisteredCustomer extends Customer {
    public RegisteredCustomer(String name) { super(name); }
    public String placeOrder(String item) { return name + " orders " + item; }
}
```

```java
// File: code/lld/ch03/generalization/MovementStyle.java
package lld.ch03.generalization;

/** "Prefer holding over extending": behaviour that varies is held, not inherited. */
public interface MovementStyle {
    String move();

    MovementStyle FLY = () -> "flies";
    MovementStyle SWIM = () -> "swims";
    MovementStyle WALK = () -> "walks";
}
```

```java
// File: code/lld/ch03/generalization/Bird.java
package lld.ch03.generalization;

public final class Bird {
    private final String species;
    private MovementStyle movement;   // held, so it can be swapped at run time

    public Bird(String species, MovementStyle movement) {
        this.species = species;
        this.movement = movement;
    }

    public void setMovement(MovementStyle m) { this.movement = m; }
    public String move() { return species + " " + movement.move(); }
}
```

```java
// File: code/lld/ch03/generalization/Main.java
package lld.ch03.generalization;

import java.math.BigDecimal;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        System.out.println("== Good hierarchy: substitution works ==");
        List<Payment> payments = List.of(
                new CashPayment(new BigDecimal("500.00"), new BigDecimal("600.00")),
                new CardPayment(new BigDecimal("1200.00"), "4242"),
                new CashPayment(new BigDecimal("300.00"), new BigDecimal("200.00")));
        for (Payment p : payments) System.out.println(p.pay());   // caller only knows Payment

        System.out.println("\n== Guest vs registered customer ==");
        Customer guest = new Customer("Guest-17");
        RegisteredCustomer reg = new RegisteredCustomer("Asha");
        List<Customer> visitors = List.of(guest, reg);   // a RegisteredCustomer IS a Customer
        visitors.forEach(c -> System.out.println(c.browse("headphones")));
        System.out.println(reg.placeOrder("headphones"));
        System.out.println("guest.placeOrder(...) does not compile: guests never inherit ordering");

        System.out.println("\n== Holding instead of extending ==");
        Bird eagle = new Bird("Eagle", MovementStyle.FLY);
        Bird penguin = new Bird("Penguin", MovementStyle.SWIM);
        System.out.println(eagle.move());
        System.out.println(penguin.move());
        penguin.setMovement(MovementStyle.WALK);   // impossible with a subclass
        System.out.println(penguin.move() + " (behaviour swapped at run time)");
    }
}
```

What each part shows:

- `Payment` is abstract, its `pay()` is `final` (the shared workflow), and `authorize()` is abstract (the step that varies). Both subclasses honour every promise of `Payment`, so `Main` loops over `List<Payment>` without caring

which is which. Amounts are `BigDecimal` and compared with `compareTo`, because `double` cannot store values such as `0.10` exactly and money must not drift.

- **Wrong-inheritance fix #1 — the subclass that must refuse something.** If `Guest extends Customer` and `Customer` can place orders, a guest could order, which the requirement forbids. The fix: `Customer` only has what *every* customer can do (browse); ordering moves down to `RegisteredCustomer`.
- **Wrong-inheritance fix #2 — the subclass that only carries a constant.** `CompactSpot`, `LargeSpot`, `HandicappedSpot` whose constructors only pass a different value up are three classes with no new behaviour. That is a field: see the `SpotType` enum in the composition example. A fourth type becomes one enum constant, not a new class.
- **Prefer holding over extending.** `FlyingBird` / `Penguin` subclasses break when a penguin arrives (penguins don't fly). `Bird` *holds* a `MovementStyle` instead, and it can even be swapped at run time — impossible with `extends`.

Output:

```
== Good hierarchy: substitution works ==
CashPayment of 500.00 -> OK, change 100.00
CardPayment of 1200.00 -> OK, card ****4242 approved
CashPayment of 300.00 -> DECLINED (short by 100.00)

== Guest vs registered customer ==
Guest-17 browses 'headphones'
Asha browses 'headphones'
Asha orders headphones
guest.placeOrder(...) does not compile: guests never inherit ordering

== Holding instead of extending ==
Eagle flies
Penguin swims
Penguin walks (behaviour swapped at run time)
```

Trade-offs

	Inheritance (extends)	Holding an object (composition/association)
Reuse	automatic	explicit delegation
Change at run time	no — fixed at compile time	yes — swap the held object
Coupling	tight: subclass sees parent internals ("fragile base class")	loose: only the interface
Number of parents in Java	one class	many interfaces / many fields
Best for	true is-a with different behaviour	varying behaviour, features that combine

When not to use it

- To share a few fields or a helper method ("code reuse" alone is not a reason).
- When a subclass would have to throw `UnsupportedOperationException` (Java's standard "this operation is not supported" error) for an inherited method.
- When the variation is just data (use a field or enum).
- When behaviours need to be mixed (fly + swim + walk) — the class count explodes.

Common mistakes

- Deep hierarchies (more than 2–3 levels) that nobody can follow.
- `Stack extends Vector` style mistakes: the child inherits methods that break its own rules (a JDK `Stack` lets you insert in the middle).
- Arrow direction: the triangle points **at the parent**, not the child.
- Using inheritance between two classes when one can do what the other does only in *some* situations. Generalization needs "always"; for "sometimes", hold a shared interface or a role object instead.

Real-world examples

- `java.util.ArrayList extends AbstractList` (reuses the skeleton implementation).
- `java.io.FileInputStream extends InputStream`; any code taking an `InputStream` accepts it.
- Every Java exception: `IOException extends Exception extends Throwable`, which is why `catch (Exception e)` catches an `IOException`.
- **Counter-example in the JDK:** `java.util.Stack extends Vector` and `java.util.Properties extends Hashtable` are widely cited as inheritance that should have been composition.

In the interview

> **30-second answer:** "Generalization is inheritance — solid line with a hollow triangle at the parent. I use it only when the child passes the substitution test: it can go anywhere the parent goes and never refuses a parent method. A subclass that only passes a constant up becomes an enum field; varying behaviour I hold as an interface instead of extending."

Follow-ups:

- *Q: Why not `CompactSpot extends ParkingSpot`?* — A: It adds no behaviour; a `SpotType` field does the same job and new types are one enum constant.
- *Q: When is inheritance right?* — A: When subclasses behave differently but all keep the parent's promises, like `Payment` → cash/card: both really take money, differently.
- *Q: Can a Java class have two parents?* — A: It can extend only one class but implement many interfaces.

Self-check

- *Q: Which way does the hollow triangle point?* → A: At the parent (general class).
- *Q: Guest cannot order, but `Guest extends Customer` and `Customer.placeOrder()` exists. What principle is broken and how do you fix it?* → A: Substitution (LSP). Move `placeOrder` to `RegisteredCustomer`.

TL;DR

Generalization = "is-a", `extends`. Only after the substitution test. Constants → fields; varying behaviour → hold an interface. Triangle points at the parent.

3.7 Realization (interface implementation)

What it is

Realization means a class *implements* an **interface**. An interface is a named list of operations (a contract) with no instance state. The class promises to provide those operations. In Java: `class EmailNotifier implements Notifier`.

Analogy: a **wall socket standard**. Any appliance whose plug "realizes" the standard works in any socket; the socket does not care who made the appliance.

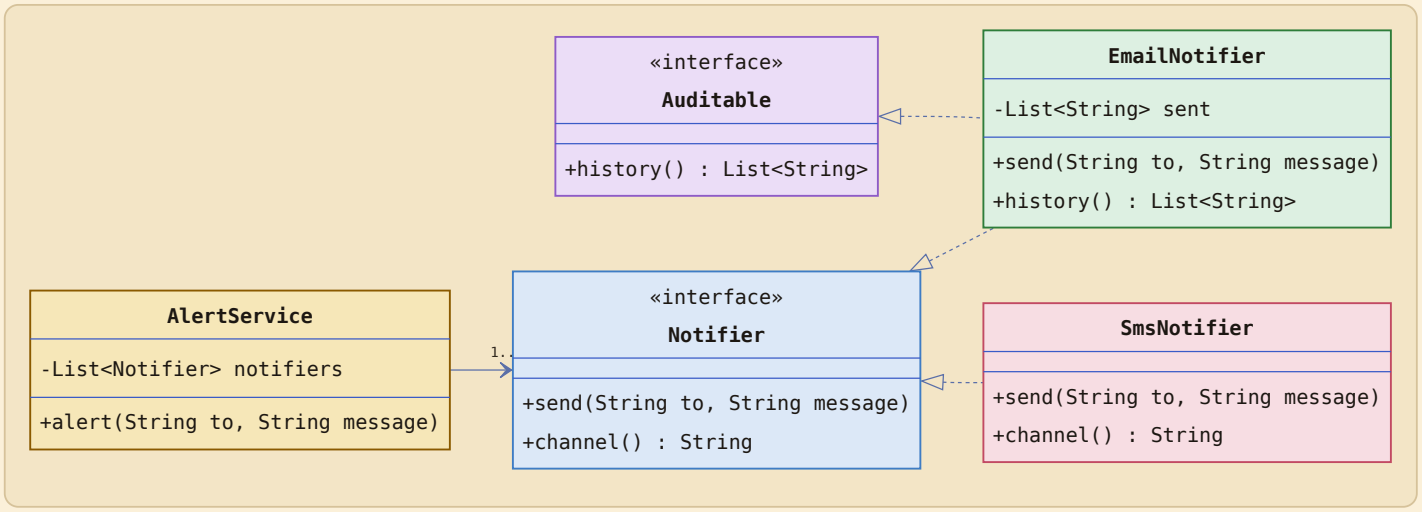
The problem it solves

Callers can depend on the **contract**, not the concrete class. You can add new implementations (SMS, push, WhatsApp) without changing the caller, test with fakes, and give one class several roles (a class may implement many interfaces but extend only one class).

How it works

1. Define an interface with the operations callers need (`send(to, message)`).
2. Each concrete class `implements` it and provides the method bodies.
3. Callers hold / receive the **interface type** (`List<Notifier>`).
4. Since Java 8 an interface may have `default` methods (with a body) and `static` methods; implementers may override defaults.
5. In UML, draw a **dashed line with a hollow triangle pointing at the interface**, and mark the interface with `«interface»`. (A compact alternative is the "lollipop": a small circle on a stick labelled with the interface name.)

Realization looks like generalization on purpose: it is the same "is-a" idea, but nothing but the contract is inherited.



Java example

```java
// File: code/lld/ch03/realization/Notifier.java
package lld.ch03.realization;

/** An interface: a contract of operations with no state. Drawn as <<interface>>. */
public interface Notifier {
    void send(String to, String message);

    /** Java 8+ default method: implementers inherit this unless they override it. */
    default String channel() {
        String n = getClass().getSimpleName();
        return n.contains("$$Lambda") ? "lambda" : n;   // lambdas get generated class names
    }
}
```

```java
// File: code/lld/ch03/realization/Auditable.java
package lld.ch03.realization;

import java.util.List;

public interface Auditable {
    List<String> history();
}
```

```java
// File: code/lld/ch03/realization/EmailNotifier.java
package lld.ch03.realization;

import java.util.ArrayList;
import java.util.List;

/** Realizes TWO interfaces. A class may implement many interfaces but extend only one class. */
public final class EmailNotifier implements Notifier, Auditable {
    private final List<String> sent = new ArrayList<>();

    @Override public void send(String to, String message) {
        String line = "EMAIL to " + to + ": " + message;
        sent.add(line);
        System.out.println("  " + line);
    }

    @Override public List<String> history() { return List.copyOf(sent); }
}
```

```java
// File: code/lld/ch03/realization/SmsNotifier.java
package lld.ch03.realization;

public final class SmsNotifier implements Notifier {
    @Override public void send(String to, String message) {
        String text = message.length() > 20 ? message.substring(0, 20) + "..." : message;
        System.out.println("  SMS to " + to + ": " + text);
    }

    @Override public String channel() { return "SMS"; }   // overrides the default
}
```

```java
// File: code/lld/ch03/realization/AlertService.java
package lld.ch03.realization;

import java.util.List;

/**
 * AlertService is associated with the INTERFACE, not with any concrete class.
 * Adding a PushNotifier later needs no change here.
 */
public final class AlertService {
    private final List<Notifier> notifiers;

    public AlertService(List<Notifier> notifiers) { this.notifiers = List.copyOf(notifiers); }

    public void alert(String to, String message) {
        for (Notifier n : notifiers) {
            System.out.println("via " + n.channel());
            n.send(to, message);
        }
    }
}
```

```java
// File: code/lld/ch03/realization/Main.java
package lld.ch03.realization;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        EmailNotifier email = new EmailNotifier();
        Notifier sms = new SmsNotifier();
        Notifier console = (to, msg) -> System.out.println("  CONSOLE " + to + ": " + msg); // lambda realizes it
too

        AlertService alerts = new AlertService(List.of(email, sms, console));
        alerts.alert("ops@shop.example", "Disk usage above 90 percent on db-1");

        Auditable audit = email;   // same object, seen through its other interface
        System.out.println("Email audit trail: " + audit.history());
    }
}
```

What each part shows: `EmailNotifier` realizes **two** interfaces; `SmsNotifier` realizes one and overrides the `default` method; a **lambda** also realizes `Notifier`, because it has exactly one abstract method (a *functional interface*). `AlertService` holds `List<Notifier>` — an association to the interface — so a new channel needs no change in it. The same `email` object is seen as a `Notifier` by `AlertService` and as an `Auditable` by `Main`.

Output:

```
via EmailNotifier
  EMAIL to ops@shop.example: Disk usage above 90 percent on db-1
via SMS
  SMS to ops@shop.example: Disk usage above 90 ...
via lambda
  CONSOLE ops@shop.example: Disk usage above 90 percent on db-1
Email audit trail: [EMAIL to ops@shop.example: Disk usage above 90 percent on db-1]
```

Trade-offs

	Interface + realization	Abstract class + generalization
Multiple per class	yes	no (single inheritance)
Instance fields	no	yes
Shared code	`default` methods only	any methods, constructors
Best for	roles/capabilities, plug-ins, test seams	a family sharing state and a workflow

When not to use it

- An interface with exactly one implementation that will never vary, and no testing need, is ceremony. Add it when a second implementation or a test double (a fake stand-in used in tests) appears (YAGNI: "you aren't gonna need it", so don't build things before they are needed).
- "Marker" interfaces with no methods are rarely needed now; annotations (`@` tags such as `@Override` that attach information to code) usually serve better.

Common mistakes

- Solid line instead of dashed (that would mean class inheritance).
- Fat interfaces that force implementers to stub (write empty placeholder versions of) methods they do not need (breaks the Interface Segregation Principle, Chapter 4).
- Callers declaring variables with the concrete type (`EmailNotifier n = ...`) and losing the benefit.

Real-world examples

- `java.util.List` realized by `ArrayList` and `LinkedList`; `Comparable`, `Runnable`, `AutoCloseable`.
- **JDBC** (Java Database Connectivity, Java's standard way to talk to SQL databases): `java.sql.Connection` is an interface; each database vendor's driver realizes it, and your code never names the vendor class.
- **Spring:** you inject `PaymentGateway`; the bean (an object Spring creates) may be a Stripe, Razorpay (online payment providers) or mock (fake, for testing) implementation.
- **SLF4J** (logging facade, i.e. one simple front for several logging libraries): your code calls the `Logger` interface; Logback or Log4j (two logging libraries) realizes it.

In the interview

30-second answer: "Realization is a class implementing an interface — dashed line, hollow triangle at the interface. I put an interface wherever behaviour can vary or I need a test double (payment gateway, notifier, pricing strategy), and the rest of the design depends on the interface only."

Follow-ups:

- *Q: Interface or abstract class?* — A: Interface for a capability many unrelated classes can have, or when a class needs several; abstract class when subclasses share state and a fixed workflow. They combine well: interface on top, abstract skeleton class below (like `List` → `AbstractList` → `ArrayList`).
- *Q: Is a lambda a realization?* — A: Yes; a lambda is an instance of a functional interface.

Self-check

- *Q: Draw the difference between generalization and realization.* → A: Same hollow triangle; generalization has a solid line, realization a dashed line.
- *Q: How many interfaces can a Java class implement?* → A: Any number.

TL;DR

Realization = "implements". Dashed line + hollow triangle at the `«interface»`. Depend on the interface, plug in implementations.

3.8 Decision guide: choosing the relationship between two classes

Ask these questions **in order** and stop at the first "yes":

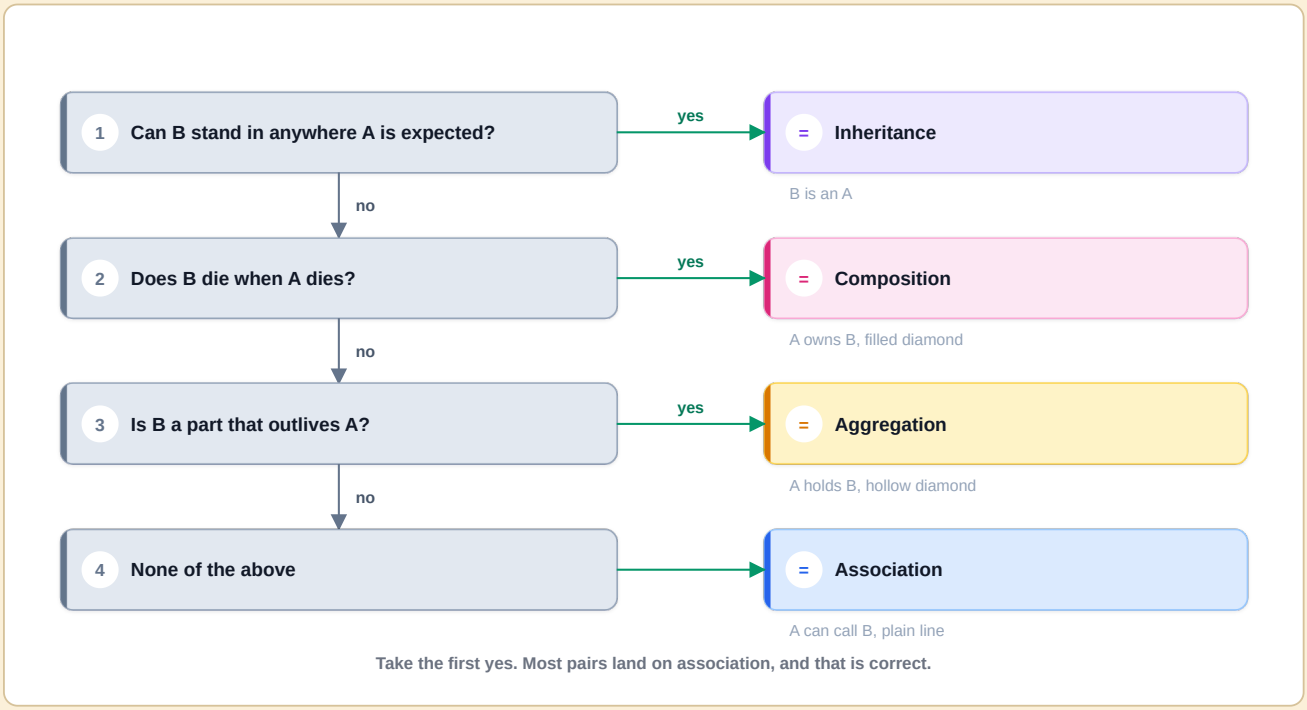
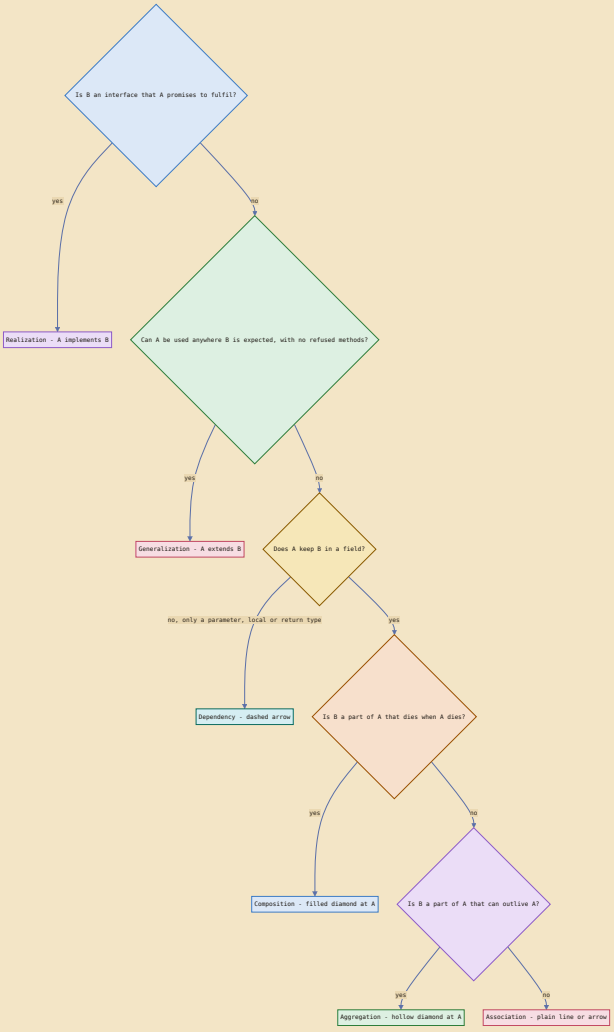


Figure — The four-question decision: inheritance, then composition, then aggregation, else association (course diagram)

As the figure above shows, inheritance is asked first only because it is the easiest to *rule out*: it needs a strict yes to the substitution test, and most pairs fail it. Most pairs end up as plain association, which is healthy. If nearly every box in your diagram extends another, the design needs another look.

The full guide including the two "light" relationships:



Summary table (keep this one)

Relationship	Phrase	UML line	Java shape	Lifetime	Example
Dependency	A *uses* B	dashed, open arrow → B	B as parameter / local / return / static call	none	`CheckoutService` uses `PaymentGateway`
Association	A *knows* B	solid line (arrow for one-way)	field of type B, B created elsewhere	independent	`Order` → `Customer`
Aggregation	A *has* B (parts live on)	solid, hollow ◇ at A	field/collection, parts passed in	B outlives A	`Airline` ◇— `Aircraft`
Composition	A *owns* B	solid, filled ◆ at A	A creates B, hides it, B not shared	B dies with A	`ParkingFloor` ◆— `ParkingSpot`
Realization	A *implements* B	dashed, hollow △ at B	`implements`	—	`EmailNotifier` ⇢ `Notifier`
Generalization	A *is a* B	solid, hollow △ at B	`extends`	—	`CardPayment` → `Payment`

Mermaid cheat-sheet for these lines (used in all later chapters):

Mermaid	Meaning
`A ..> B`	A depends on B
`A --> B`	A associated with B, navigable A to B
`A -- B`	association (both ways / unspecified)
`A o-- B`	aggregation, A is the whole
`A *-- B`	composition, A is the whole
`B <|.. A`	A realizes interface B
`B <|-- A`	A extends B
`A "1" *-- "1..*" B`	with multiplicities

Common mistakes (all relationships)

- Diamond at the wrong end (it goes at the **whole**).
- Triangle at the wrong end (it goes at the **parent / interface**).
- Lines without multiplicity. A line with no type or count means the design question behind it is still open. Candidates who write the numbers on every line from the very first box report that the interviewer simply stops asking about them — it is cheap to add and removes a whole class of follow-up questions.
- Inheritance used to share data.

In the interview

Say the test **out loud** as you draw each line, e.g. *"When a floor is removed its spots go too, so this is composition"* or *"Cash and card can both be asked to take money, so inheriting from Payment is honest."* If you spot a subclass that only carries a constant, say so and collapse it into a field yourself. Expect two follow-ups: "why not inheritance here?" (answer with the substitution test) and "what if a spot can hold two motorbikes?" (change the multiplicity, then say what it does to the code).

Self-check

- *Q: `Library` and `Member` — which relationship?* → A: Aggregation (or plain association): members exist whether or not the library does.
- *Q: `Flight` and `WeeklySchedule`?* → A: Composition.
- *Q: `ParkingTicket` and `ParkingSpot`?* → A: Association: the ticket refers to the spot but is not a whole made of spots.
- *Q: `Report.render(Formatter f)`?* → A: Dependency.

TL;DR

Realization/generalization for "is-a" (only after the substitution test), composition when parts die with the whole, aggregation when they live on, association when one just knows the other, dependency when it is only used inside a method.

Part B — UML

3.9 What UML is (structural vs behavioural diagrams)

What it is

UML (Unified Modeling Language) is a standard set of symbols for drawing software designs. Because everybody uses the same symbols, a diagram drawn by one engineer can be read by another without a legend. It is maintained by the OMG (Object Management Group); the current version is UML 2.5.1.

UML is **not** a programming language. It does not run and you do not need a tool to use it — a whiteboard is enough. It is an agreement about what, for example, "a line with a filled diamond on one end" means.

A little history explains its size: in the 1990s three competing object-oriented methods (Grady Booch's, James Rumbaugh's OMT and Ivar Jacobson's OOSE) each had their own notation. They were merged into UML so that the industry had one language. Because it had to satisfy three groups of users, UML is large — and every team uses only the subset it needs.

The problem it solves

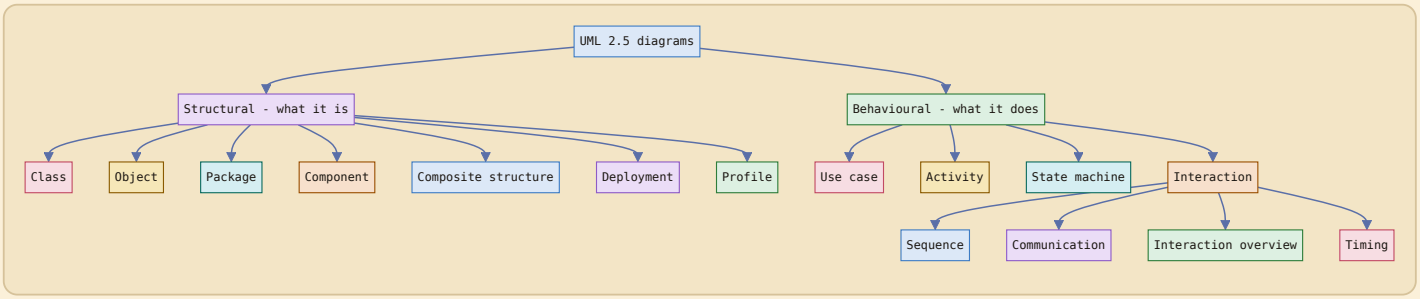
1. **It makes a design visible.** Twelve classes in sentences are hard to hold in your head; twelve boxes with lines are readable in seconds.
2. **It makes decisions explicit.** In a sentence you can stay vague about whether A owns B. On a diagram you must pick a diamond — filled or hollow.
3. **It makes disagreement cheap.** The interviewer can put a finger on one line and ask "why composition here?". That kind of back-and-forth is exactly what a good interview looks like.

How it works — the 14 diagram types

UML 2.5 defines **14 diagram types** in two families:

- **Structural diagrams** show what the system *is made of* — the static parts that exist at any moment.
- **Behavioural diagrams** show what the system *does* over time. A subgroup of behavioural diagrams, **interaction diagrams**, focus on messages between objects.

Family	Diagram	Shows	Interview use
Structural	**Class**	classes, attributes, operations, relationships	**always**
Structural	Object	a snapshot of real objects and their links	rarely (to explain an example)
Structural	Package	how classes group into packages/modules	rarely
Structural	Component	deployable parts and their interfaces	system design rounds
Structural	Composite structure	internal parts of one class	rarely
Structural	Deployment	software on hardware nodes (machines)	system design rounds
Structural	Profile	extensions to UML itself	never
Behavioural	**Use case**	actors and what they can achieve	early, often spoken
Behavioural	**Activity**	steps of one flow with branches	one per key flow
Behavioural	**State machine**	states of one object and allowed transitions	when an object has a lifecycle
Behavioural → interaction	**Sequence**	who calls whom, in time order	on request
Behavioural → interaction	Communication	same messages, drawn on a network of objects	rarely
Behavioural → interaction	Interaction overview	an activity diagram whose nodes are interactions	rarely
Behavioural → interaction	Timing	state changes against a time axis	embedded (software inside devices) / hardware



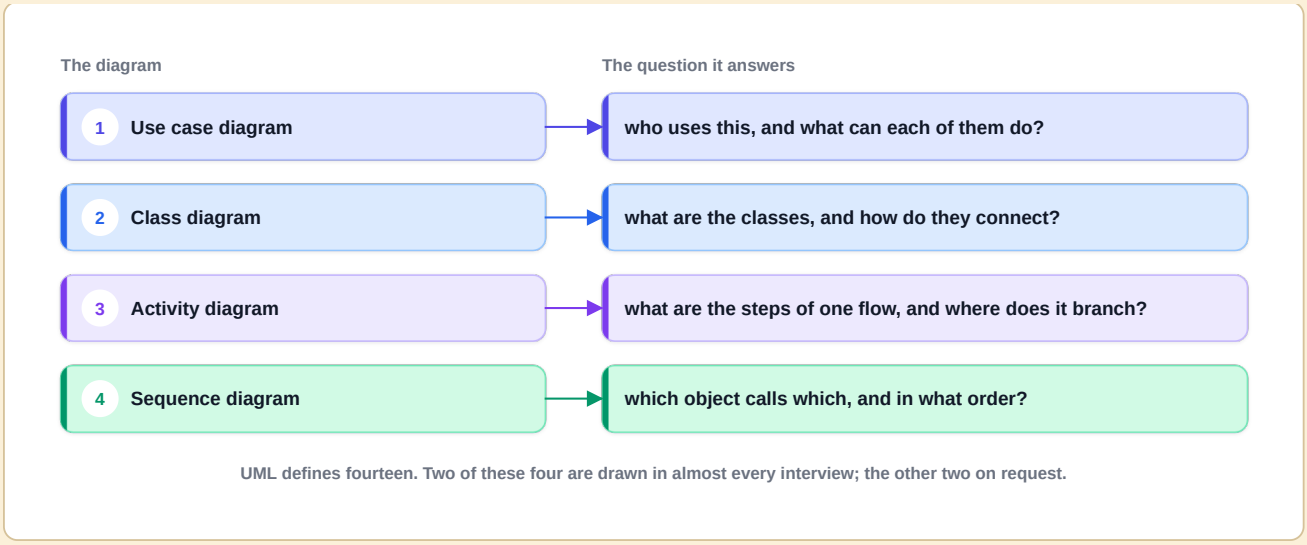


Figure — The four diagrams an OOD interview uses and the question each answers (course diagram)

As the figure above shows, four diagrams carry an object-oriented design interview, and this book adds a fifth that often helps:

1. **Use-case diagram** — who uses the system and what can each of them do? Drawn first, before any class exists.
2. **Class diagram** — what are the classes and how do they connect? *The* answer; the only one that maps directly to code.
3. **Activity diagram** — what are the steps of one flow and where does it branch?
4. **Sequence diagram** — which object calls which, in what order?
5. **State machine diagram** — what states can one object be in, and what moves it between them? (Parking spot, order, elevator, vending machine…)

Trade-offs

Benefit	Cost
Shared vocabulary, no legend needed	Large spec; easy to over-draw
Forces explicit decisions	Takes whiteboard time from code/discussion
Tool support (PlantUML, Mermaid, draw.io, StarUML: diagram-drawing tools)	Diagrams go stale if not kept with code

When not to use it

As documentation that nobody maintains, or in full formal detail on a whiteboard. Many teams today use lightweight "sketch" UML, the C4 model for architecture (a simple style that draws a system at four zoom levels), or text-based tools (Mermaid/PlantUML) kept next to the code. For an interview, UML-lite is exactly right.

Common mistakes

- Thinking the interviewer grades notation. They grade whether your **diagram and your words agree**. If you forget the realization arrow, say "this one implements that one" and draw something. If you draw a hollow diamond where you meant a filled one and then say which you meant, that is a discussion, not a lost point.
- Drawing a line you cannot justify. It hurts more than a missing line.

Real-world examples

- Mermaid diagrams render natively in GitHub and GitLab Markdown (a plain-text formatting language), which is why many design docs (RFCs, written design proposals; ADRs, Architecture Decision Records) embed sequence and class diagrams as text.
- PlantUML is widely used in enterprise wikis (Confluence, a company wiki tool, via plug-ins) for sequence diagrams of service calls.

In the interview

> **30-second answer:** "UML is a standard notation for designs. It has 14 diagrams: structural ones (class, object, component…) describe what the system is made of; behavioural ones (use case, activity, state machine, and the interaction diagrams such as sequence) describe what it does. In an OOD round I use a quick use-case list, one class diagram, and an activity or sequence diagram for the key flow."

Follow-ups:

- *Q: Is UML outdated?* — A: The full formal UML is rarely used, but its core notations (class, sequence, state) remain the shared language of design discussions and appear in Mermaid/PlantUML in modern repos (code repositories). You only need the subset.
- *Q: Which diagram maps to code?* — A: The class diagram.

Self-check

- *Q: Structural or behavioural: sequence diagram?* → A: Behavioural (an interaction diagram).
- *Q: How many UML diagram types exist?* → A: 14 (7 structural, 7 behavioural).

TL;DR

UML = standard symbols, not a language that runs. 14 diagrams: structural (what it is) vs behavioural (what it does). Interviews use class (always), use case, activity, sequence, and state machine when an object has a lifecycle.

3.10 Use-case diagrams

What it is

A **use-case diagram** shows **who** uses the system and **what** each of them can achieve. It has no classes, no methods, no data and no ordering.

- An **actor** is a *role* that interacts with the system, drawn as a stick figure outside the system. Roles, not people: one person can be both a Member and a Librarian; one actor can stand for thousands of users. An actor need not be human — a payment gateway, an email service or a timer ("System") is an actor too.
- A **use case** is one goal an actor can achieve that produces a result the actor cares about, drawn as an oval with a verb phrase: "Borrow a book". "Click the search button" is not a use case; it is a step inside one.
- The **system boundary** is a rectangle around all use cases, labelled with the system's name. Inside = what you build. Outside = everything else.

Analogy: the menu board at a restaurant. It lists what customers can order (use cases) and who can order what (a kids' menu for children), without saying anything about the kitchen.

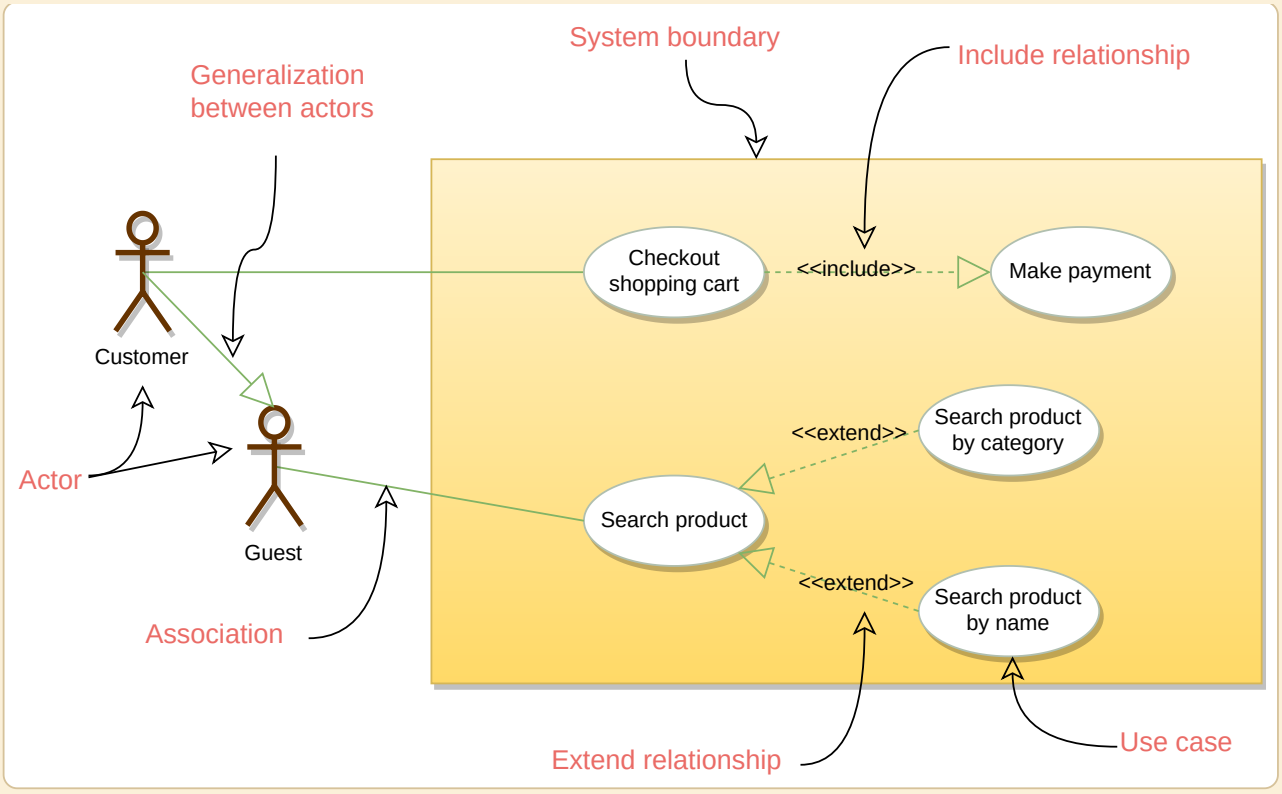


Figure — The parts of a use-case diagram for online shopping: actors (Customer, Guest), use cases, system boundary, generalization between actors, and «include» / «extend» relationships (course diagram)

The problem it solves

It turns a vague one-line question ("design a library") into an **agreed scope**. Drawing the boundary lets you say "these nine things are in; refunds, seasonal pricing and reporting are out" — and the interviewer, who judges whether your answer is complete, has agreed to that list. It also hands you your first classes: **actors usually become classes and use cases usually become methods**.

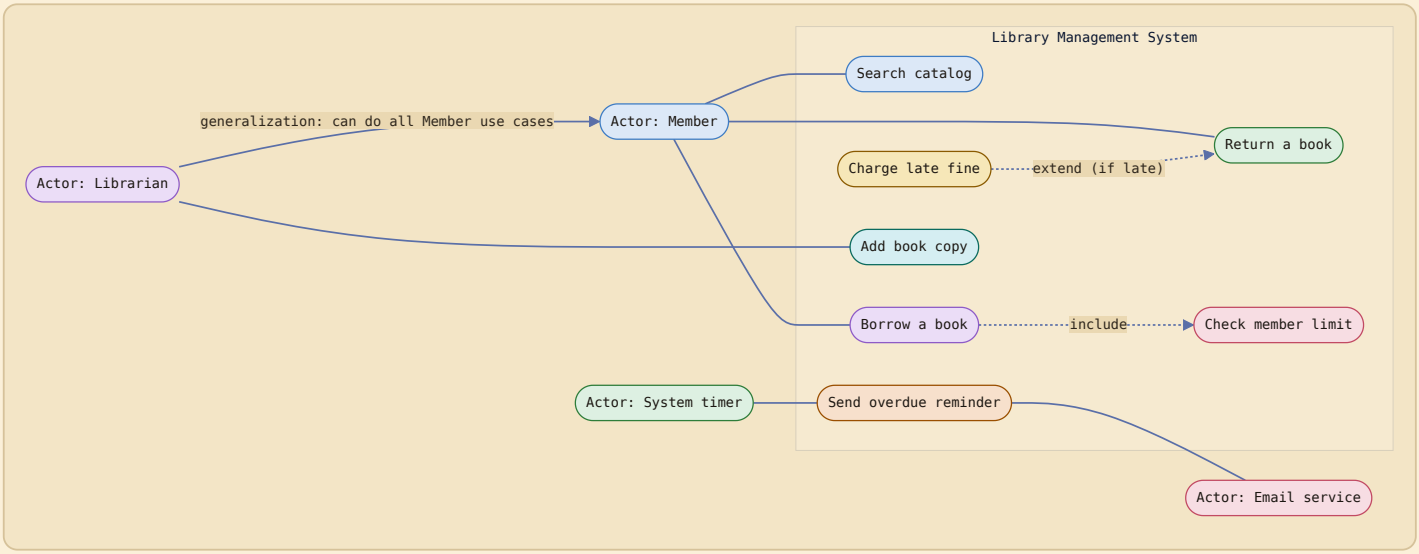
How it works

1. **Find the actors** with four questions:
   - *Who starts something?* (customer, member, player)
   - *Who administers it?* (adds books, sets rates, blocks accounts — the most-forgotten actor, and it brings a whole column of use cases)
   - *Who gets told?* (anyone receiving notifications)
   - *What acts with nobody present?* (timers, background jobs, external services — draw as "System")
2. Put **primary actors** (who start use cases) on the left and **secondary/supporting actors** (whom the system calls, e.g. payment gateway) on the right.
3. Draw the **system boundary** and put each **use case** (verb phrase) inside.
4. Connect actors to their use cases with **plain lines (associations)**.
5. Add relationships between use cases only where they help:
   - **«include»** — *always*. The base use case always performs the included one. Dashed arrow **from the base to the included** case. E.g. *Borrow a book* «include» *Check member's limit*. In code: one method always calls another.
   - **«extend»** — *sometimes*. An optional behaviour that is inserted into the base under a condition, at a named **extension point**. Dashed arrow **from the extending case back to the base**. E.g. *Charge fine* «extend» *Return a book* (only if late). Test: can the base use case complete without it? Yes → extend; no → include.
   - **Generalization** — solid line with a hollow triangle: between actors (a Librarian can do everything a Member can) or between use cases (*Pay by card* is a kind of *Make payment*).
6. Spell the keywords singular: «include», «extend». Older material uses «includes», «extends» or «uses»; «uses» was replaced by «include» long ago.

Reading the course figure above: the Customer→Guest line has its hollow triangle at **Guest**, so it reads "a Customer is a kind of Guest — a Customer can do everything a Guest can, plus check out". The figure also puts

hollow triangles on its «include»/«extend» arrows; strict UML draws both as a dashed line with an **open** arrowhead (the triangle is reserved for generalization). What matters is the direction and the label: «include» points *Checkout shopping cart → Make payment*; both «extend» arrows point from the search variants back to *Search product*.

Mermaid has no native use-case diagram, so this book draws them as flowcharts (stadium shapes = use cases, a subgraph = the boundary):



From use cases to code (see `lld.ch03.library`)

Use-case element	Becomes in Java
Actor *Member*	class `Member`
Actor *Librarian* (generalizes Member)	`class Librarian extends Member`
Borrow a book	`Member.checkout(BookItem, LocalDate)`
«include» *Check member limit*	private `checkLimit()` called on **every** checkout
Return a book	`Member.returnBook(BookItem, LocalDate)`
«extend» *Charge late fine*	`chargeFine(...)` called **only if** the book is late
Add book copy	`Librarian.addCopy(...)`

This is a starting point, not a rule: a use-case method sometimes belongs on a service class instead of the actor class.

Trade-offs

Pro	Con
Fast scope agreement; finds forgotten actors	No ordering, no data — it cannot show *how*
Maps to classes and methods	Easy to bloat with UI (user interface) steps

When not to use it

When the interviewer has already given precise requirements — then a spoken list of actors and their use cases is enough.

Common mistakes

- **Use cases that are UI steps.** Typing a password, pressing submit and seeing a confirmation are all part of a single use case, "Log in". A diagram with dozens of ovals is usually a list of screens, not goals.
- **One actor per person.** A librarian who also borrows is *two roles*. Draw two actors, linked by generalization if one can do everything the other can.
- Reversing the «extend» arrow (it points from the optional case to the base).

- Using «include» for something that only happens sometimes.
- Leaving off the system boundary — the most useful part.

Real-world examples

- **An e-commerce checkout (Amazon-style), modelled:** *Place order* always «include»s *Make payment*; *Apply gift card* «extend»s it only when the customer has one.
- **BookMyShow:** Customer, Theatre admin, Payment gateway and a timer that releases held seats are the actors.

In the interview

> **30-second answer:** "Before classes, I list actors and what each can do, and draw a boundary to agree scope. Actors are roles — including admins, external services and timers. «include» is always, «extend» is sometimes. Actors become my first classes and use cases become methods."

Follow-ups:

- *Q: Is 'make payment' an include or an extend of 'checkout'?* — A: If every checkout requires payment, it is «include». Something like "apply coupon", which checkout can complete without, is «extend».
- *Q: How long should this take?* — A: Three to five minutes, early.

Self-check

- *Q: Which direction does the «include» arrow point?* → A: From the base use case to the included one.
- *Q: Is a payment gateway an actor?* → A: Yes — an external system that interacts with ours (a secondary actor).
- *Q: "Search product by name" and "by category" relate to "Search product" how?* → A: As in the course diagram: optional variants, drawn as «extend» (some modellers draw them as use-case generalization — both are defensible if explained).

TL;DR

Actors (roles, outside) + use cases (goals, inside) + boundary (scope). «include» = always, arrow base→included; «extend» = sometimes, arrow extension→base. Actors → classes, use cases → methods.

3.11 Class diagrams

What it is

A **class diagram** shows the **static structure** of a design: which classes exist, what each holds (attributes), what each can do (operations), and how they connect (relationships with multiplicity). It is the answer to an OOD question and the only UML diagram that maps directly to code.

The problem it solves

It captures the whole design on one page so it can be reviewed, challenged and turned into code. Every other diagram exists either to get you to it (use case) or to check/explain it (activity, sequence, state).

How it works — the class box

A class is a rectangle with three compartments: **name** / **attributes** / **operations**.

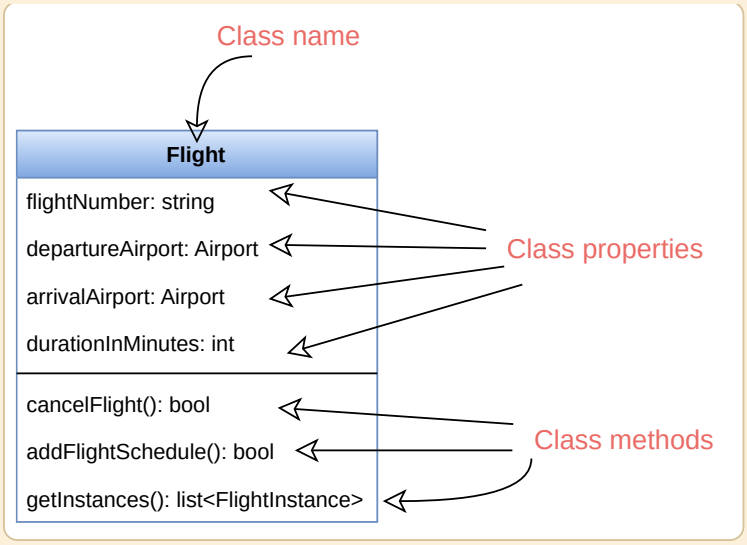


Figure — A class box: name on top, properties (attributes) in the middle, methods (operations) at the bottom, using a Flight class (course diagram)

As in the figure above, attributes are written `name: Type` and operations `name(params): ReturnType`. The full UML syntax is:

```
visibility name : Type [multiplicity] = default      e.g.  - spots : ParkingSpot [1..*]
visibility name(param : Type, ...) : ReturnType      e.g.  + park(v : Vehicle) : Ticket
```

Visibility markers (mapped to Java):

UML	Meaning	Java
`+`	public — anyone	`public`
`-`	private — only this class	`private`
`#`	protected — this class and subclasses	`protected` (Java also allows the same package)
`~`	package — same package only	no modifier (package-private)

Other notation:

Notation	Meaning	Java
Italic name, or `{abstract}` / `«abstract»`	abstract class (one you never create objects of directly) / abstract operation (no body)	`abstract`
`«interface»` above the name	interface	`interface`
`«enumeration»` + literals	enum	`enum`
underlined member	static (class-level)	`static`
`{readOnly}`	cannot change after creation	`final` field
`List<T>` / `T[*]`	collection attribute	`List<T>`

You do **not** have to fill all three compartments. Early in an interview a box with only a name is a perfectly good class. Attributes and operations arrive as the design settles.

In Mermaid, visibility uses the same symbols; `*` after a method means abstract and `$` means static; `<<interface>>`, `<<abstract>>`, `<<enumeration>>` are annotations; generics are written `List~T~`.

How it works — the relationships (all arrow types)

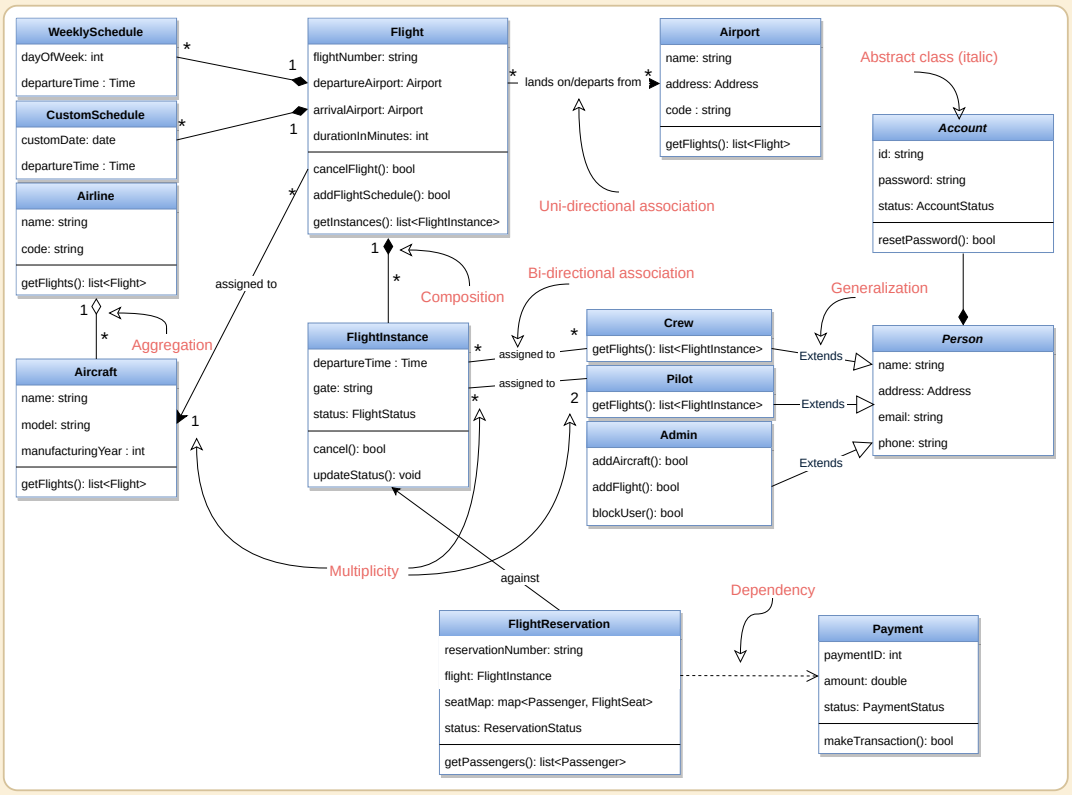


Figure — An airline-system class diagram using every relationship: associations (e.g. Flight to Aircraft, uni-directional), multiplicities (FlightInstance has 2 Pilots), aggregation (Airline–Aircraft), composition (Flight–WeeklySchedule), generalization (Person to Crew/Pilot/Admin) and dependency (course diagram)

The figure above uses the complete vocabulary: the six relationships of Part A (association appearing in both its two-way and one-way form) plus multiplicity. Those are the only symbols a class diagram needs:

Line	Drawn as	Read as	Example from the figure
Association (bi-directional)	solid line	"A and B know each other"	FlightInstance — Pilot, FlightInstance — Crew ("assigned to")
Uni-directional association	solid line, arrowhead at B	"A can call B, not vice versa"	Flight → Airport ("lands on/departs from"), Flight → Aircraft ("assigned to")
Multiplicity	numbers at the line ends: `1`, `*`, `0..1`, `2..4` (a range)	"how many"	FlightInstance `*` — `2` Pilot: each flight instance has exactly 2 pilots, each pilot flies many instances
Aggregation	hollow diamond at the whole	"has-a, part lives on"	Airline `1` ◇— `*` Aircraft
Composition	filled diamond at the whole	"owns, part dies with it"	Flight ◆— WeeklySchedule, Flight ◆— CustomSchedule, Flight ◆— FlightInstance
Generalization	solid line, hollow triangle at parent	"is-a"	Pilot, Crew, Admin → Person
Realization	dashed line, hollow triangle at interface	"implements"	(any `«interface»`)
Dependency	dashed line, open arrowhead	"uses"	FlightReservation ⇢ Payment

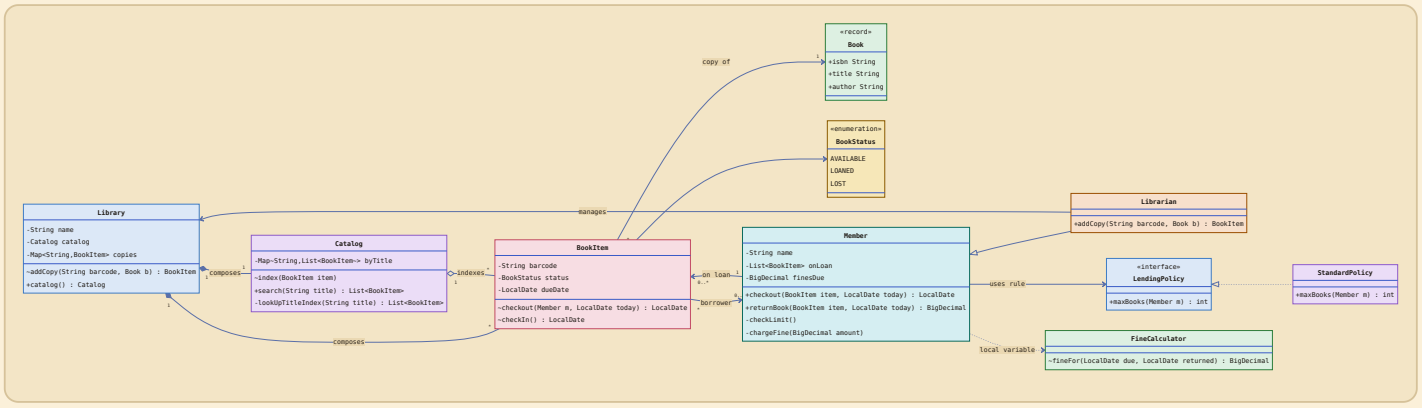
(The figure draws its one-way arrowheads filled; strict UML uses an open arrowhead — see the legend note in 3.1.)

A note on reading the figure: some of its classes carry getters for the "other" direction (e.g. `Aircraft.getFlights()`) even though the line is drawn one-way. Strictly, such a getter makes the association navigable both ways; treat the arrow as "the primary direction", and in your own diagrams keep arrows and fields consistent.

Two extra notations worth recognising:

- **Association class** — a class hanging off an association by a dashed line, holding data about the *link itself* (e.g. `Enrollment{grade}` on Student–Course).
- **Qualified association** — a small box on one end with a key (e.g. `Library [isbn] → Book`), meaning "look up by this key" — in Java a `Map<Isbn, Book>`.

Worked example — the library design (code in `lld.ch03.library`)



Every line type appears: composition (Library ◆ Catalog, Library ◆ BookItem), aggregation (Catalog ◇ BookItem — the index points at copies it does not own), associations with multiplicity, generalization (Librarian → Member), realization (StandardPolicy ⇢ LendingPolicy) and dependency (Member ⇢ FineCalculator). Visibility `~` marks package-private methods such as `BookItem.checkout`, which only `Member` (same package) may call. The Java code is listed in full in 3.12.

How much detail to draw

- **Every class gets a box and a name.** 10–15 boxes is a full answer.
- **Attributes only where they matter** — the two or three fields that carry the design.
- **Operations only where they are interesting** — the methods that appear in your flows.
- **Every line gets a relationship type and a multiplicity.** This is the part not to skip.
- If short of time, **drop attributes before relationships**. Boxes with only names but precise, labelled lines already describe a design; richly detailed boxes joined by vague lines do not.

Trade-offs

Detailed diagram	Sketch diagram
Near code-ready; fewer surprises later	Faster; easier to change during discussion
Takes 10+ minutes	Leaves details for the coding phase

When not to use it

It does not show *order* or *time* — use sequence/activity/state diagrams for behaviour.

Common mistakes

- Unlabelled lines (no type, no multiplicity).
- Diamonds/triangles at the wrong end.
- Listing every getter/setter and `String` field.
- Associations drawn in the diagram that do not exist as fields in the code (or vice versa).

Real-world examples

- IntelliJ IDEA (Ultimate), a popular Java IDE (code editor with build and run tools), can generate class diagrams from code; many teams use them to explore unfamiliar modules.
- JPA entity diagrams (ER-style, i.e. entity-relationship pictures of database tables) are essentially class diagrams with multiplicities (`@OneToMany` = `1..*`).

In the interview

> **30-second answer:** "I start with named boxes from the nouns and actors, then draw every line with a type and multiplicity, saying the lifetime/substitution test aloud. I add only the attributes and methods that drive the key flows. `+ - # ~` for visibility, italics or «abstract» for abstract, «interface» for interfaces."

Follow-ups:

- *Q: What does `#` mean and what is the Java nuance?* — A: Protected. In Java, protected is also visible to classes in the same package, which UML's `#` does not imply.
- *Q: How do you show a static method?* — A: Underline it (Mermaid: `$` suffix).

Self-check

- *Q: `~` means?* → A: Package visibility (Java's default, no modifier).
- *Q: What is the one test for aggregation vs composition?* → A: Imagine the whole is gone — is the part still meaningful by itself?

TL;DR

Box = name / attributes / operations with `+ - # ~`. Seven lines: association (± arrow), aggregation ◇, composition ◆, generalization (solid △), realization (dashed △), dependency (dashed arrow), plus multiplicity on every line.

3.12 Sequence diagrams

What it is

A **sequence diagram** shows **which object calls which, and in what order**, for one scenario. Objects run across the top; **time runs down the page**.

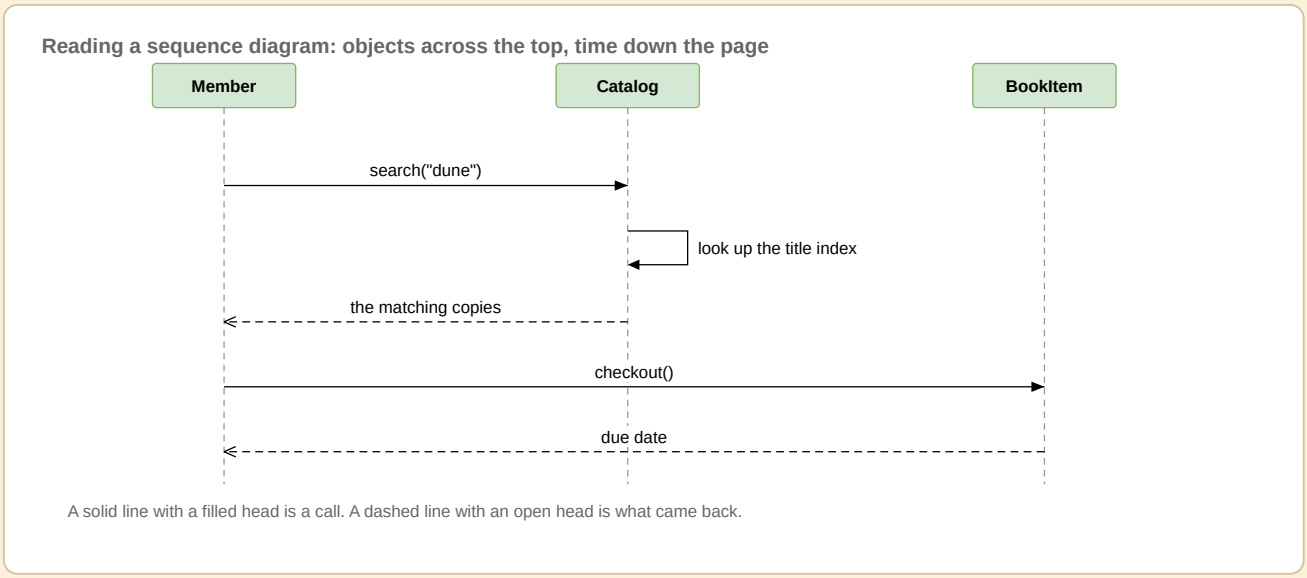


Figure — A member searches the catalog for "dune" and borrows a copy: lifelines, a self-call on Catalog, calls with filled arrowheads and dashed returns (course diagram)

The problem it solves

A class diagram says objects *can* talk; a sequence diagram shows what they *actually say* and in which order. It is the right diagram when the order is easy to get wrong (reserve seat → take payment → confirm — any other order is a bug, and the diagram is the argument for this one), when several objects take part in one flow (two objects need no diagram; five usually do), or when you are explaining a class diagram you have already drawn.

How it works — the notation

Element	Drawn as	Meaning
Participant / lifeline	box at top + dashed vertical line	one *object* (strictly `catalog : Catalog`; most people just write the class name)
Actor	stick figure at top	a user or external system starting the flow
Synchronous message	solid line, **filled** arrowhead	a call; the caller waits for it to finish
Asynchronous message	solid line, **open** arrowhead	fire-and-forget; the caller continues (events, queues: lists of messages waiting to be processed)
Return (reply)	dashed line, open arrowhead	the value coming back — draw only when it matters
Self-message	arrow looping back to the same lifeline	the object calls its own method
Activation bar (execution specification)	thin rectangle on a lifeline	the object is busy executing
Create message	dashed arrow to the new object's box, often labelled «create»	the object is created at that moment
Destruction	**X** at the end of a lifeline	the object is destroyed
Combined fragment	labelled box around messages	control logic: see below

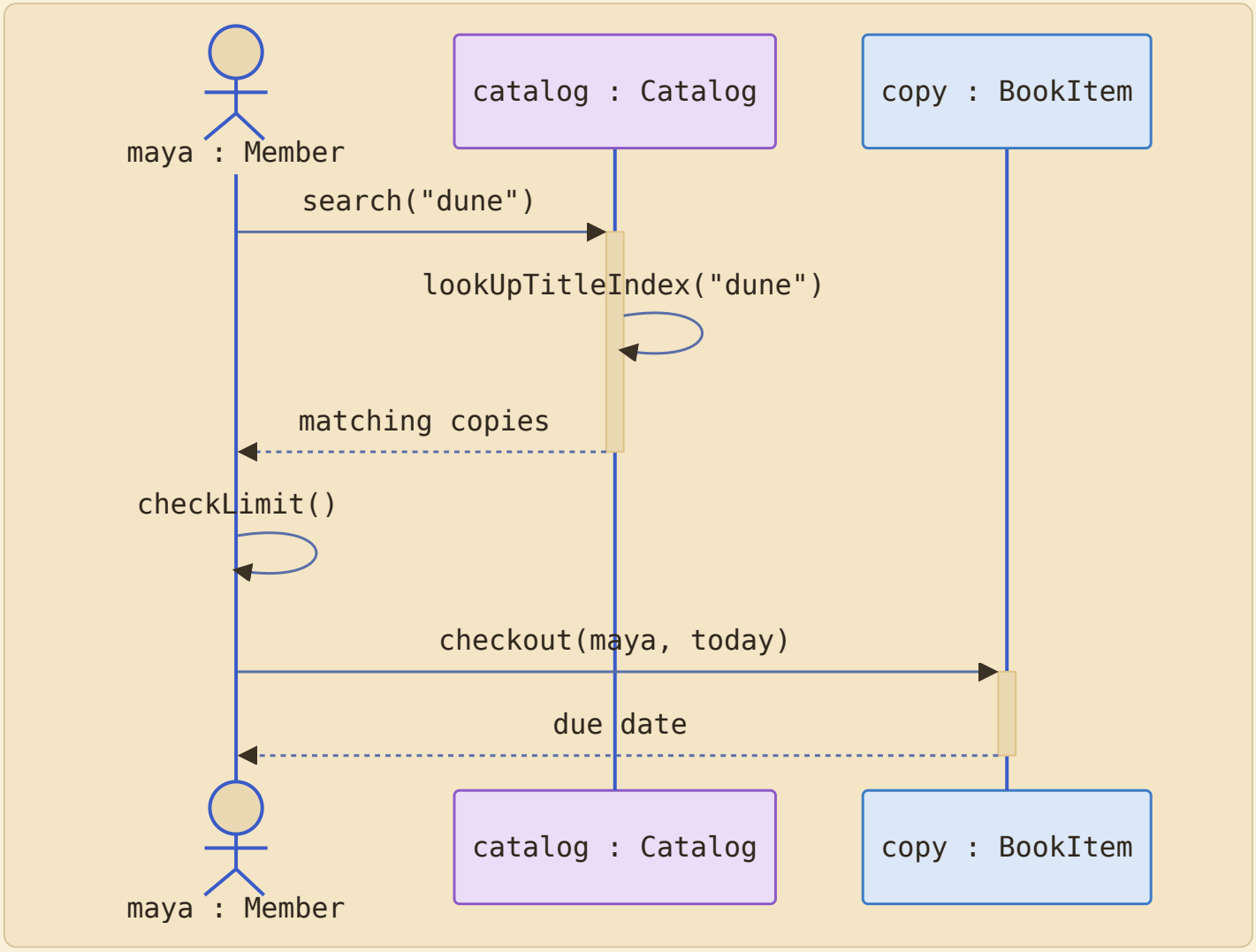
Combined fragments (the box's top-left label is the *operator*; conditions called *guards* are in `[ ]`):

Operator	Meaning	Java equivalent
`alt`	alternatives, one runs (sections split by dashed lines, each with a guard, last can be `[else]`)	`if / else if / else`
`opt`	optional, runs only if the guard is true	`if` without `else`
`loop`	repeats while the guard holds (`loop [for each item]`)	`for` / `while`
`par`	sections run in parallel	threads (independent lines of execution) / `CompletableFuture` (a result that arrives later from background work)
`break`	if guard true, run this and leave the enclosing fragment	early `return` / `break`
`critical`	atomic region (runs as one indivisible step), no interleaving	`synchronized` (Java keyword: one thread at a time) / lock
`ref`	reference to another diagram	calling a helper flow

(UML also has `neg`, `strict`, `seq`, `assert`, `ignore` and `consider`; you will not need them in an interview.)

On a whiteboard, activation bars and fragments are both optional and usually skipped: lifelines, calls, the returns that matter and self-calls are enough. They are shown in full here so you can read them when you meet them.

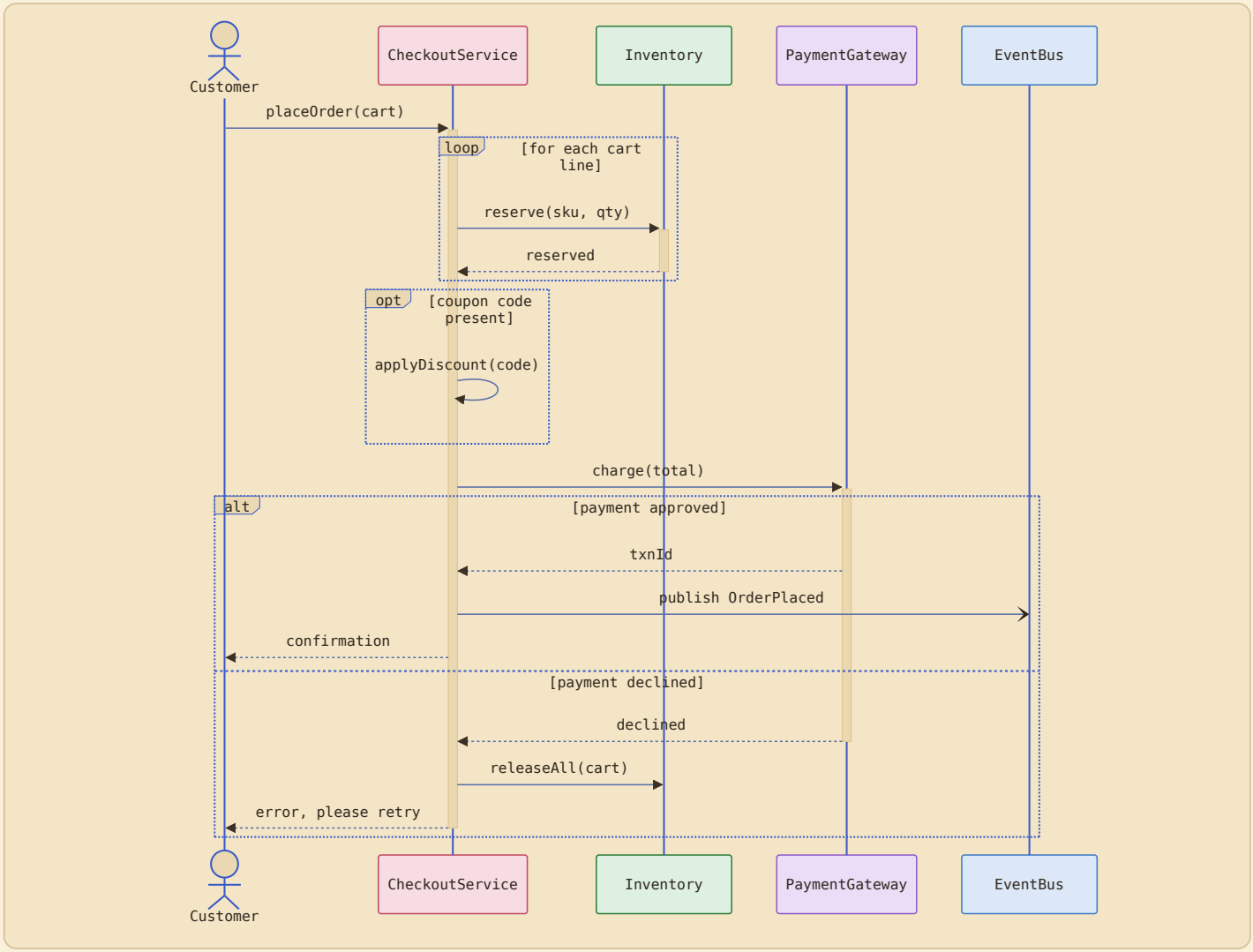
Example 1 — the library checkout, matching the Java code



Four things are worth noticing (the same as in the course figure above):

1. **The first message starts the flow** — a flow is always kicked off from outside the objects shown; here the member calls `search`.
2. **The catalog calls itself** — looking up the title index needs no other object, so the arrow loops back.
3. **Only two returns are drawn** — the copies and the due date; other calls return nothing worth naming.
4. **The member talks to the copy directly** — borrowing goes straight to `BookItem`, not back through `Catalog`. No other diagram shows this choice, and an interviewer can question it simply by tapping that arrow.

Example 2 — all fragments and async messages (checkout with payment)



Here `->>` is a synchronous call, `-->>` a reply, and `-)` an **asynchronous** message: publishing an event does not wait for subscribers. (An `EventBus` is an object that delivers each published event to everyone subscribed to it; `sku` is a stock-keeping unit, a product's ID. Queues and asynchronous messaging are explained in System Design Fundamentals.) The order is the point of the diagram: reserve stock *before* charging, and release it if the charge fails.

Java code for Example 1 (`lld.ch03.library`)

```java
// File: code/lld/ch03/library/Book.java
package lld.ch03.library;

/** The title (one per ISBN). Many physical copies (BookItem) point at one Book. */
public record Book(String isbn, String title, String author) { }
```

```java
// File: code/lld/ch03/library/BookStatus.java
package lld.ch03.library;

public enum BookStatus { AVAILABLE, LOANED, LOST }
```

```java
// File: code/lld/ch03/library/BookItem.java
package lld.ch03.library;

import java.time.LocalDate;
import java.util.Optional;

/** One physical copy. Association BookItem * --> 1 Book, and 0..1 borrower. */
public final class BookItem {
    static final int LOAN_DAYS = 14;

    private final String barcode;
    private final Book book;
    private BookStatus status = BookStatus.AVAILABLE;
    private Member borrower;      // 0..1
    private LocalDate dueDate;    // set only while LOANED

    public BookItem(String barcode, Book book) {
        this.barcode = barcode;
        this.book = book;
    }

    /** Called by Member. Returns the due date: the one return the sequence diagram draws. */
    LocalDate checkout(Member m, LocalDate today) {
        if (status != BookStatus.AVAILABLE) {
            throw new IllegalStateException(barcode + " is " + status);
        }
        status = BookStatus.LOANED;
        borrower = m;
        dueDate = today.plusDays(LOAN_DAYS);
        return dueDate;
    }

    LocalDate checkIn() {
        if (status != BookStatus.LOANED) throw new IllegalStateException(barcode + " is not on loan");
        LocalDate due = dueDate;
        status = BookStatus.AVAILABLE;
        borrower = null;
        dueDate = null;
        return due;
    }

    public String barcode() { return barcode; }
    public Book book() { return book; }
    public BookStatus status() { return status; }
    public Optional<Member> borrower() { return Optional.ofNullable(borrower); }

    @Override public String toString() { return barcode + "[" + book.title() + ", " + status + "]"; }
}
```

```java
// File: code/lld/ch03/library/Catalog.java
package lld.ch03.library;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

/** An index over copies. It aggregates BookItems: it points at them but does not create or own them. */
public final class Catalog {
    private final Map<String, List<BookItem>> byTitle = new HashMap<>();

    void index(BookItem item) {
        byTitle.computeIfAbsent(key(item.book().title()), k -> new ArrayList<>()).add(item);
    }

    public List<BookItem> search(String title) {
        List<BookItem> hits = lookUpTitleIndex(title);   // self-call in the sequence diagram
        return List.copyOf(hits);
    }

    private List<BookItem> lookUpTitleIndex(String title) {
        return byTitle.getOrDefault(key(title), List.of());
    }

    private static String key(String title) { return title.toLowerCase(Locale.ROOT); }
}
```

```java
// File: code/lld/ch03/library/LendingPolicy.java
package lld.ch03.library;

/** Realized by concrete policies. Member depends on this interface, not on a class. */
public interface LendingPolicy {
    int maxBooks(Member m);
}
```

```java
// File: code/lld/ch03/library/StandardPolicy.java
package lld.ch03.library;

public final class StandardPolicy implements LendingPolicy {
    @Override public int maxBooks(Member m) { return m instanceof Librarian ? 10 : 2; }
}
```

```java
// File: code/lld/ch03/library/FineCalculator.java
package lld.ch03.library;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

/** Used as a local helper inside Member.returnBook: a dependency, not a field. */
final class FineCalculator {
    private static final BigDecimal PER_DAY = new BigDecimal("5.00");

    BigDecimal fineFor(LocalDate due, LocalDate returned) {
        long late = ChronoUnit.DAYS.between(due, returned);
        return late > 0 ? PER_DAY.multiply(BigDecimal.valueOf(late)) : BigDecimal.ZERO.setScale(2);
    }
}
```

```java
// File: code/lld/ch03/library/Member.java
package lld.ch03.library;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Actor "Member" becomes a class; its use cases become methods:
 *   Borrow a book  -> checkout(...)  which always <<include>>s the limit check
 *   Return a book  -> returnBook(...) which is sometimes <<extend>>ed by "charge a fine"
 */
public class Member {
    private final String name;
    private final LendingPolicy policy;             // association to an interface
    private final List<BookItem> onLoan = new ArrayList<>();
    private BigDecimal finesDue = BigDecimal.ZERO.setScale(2);   // money: BigDecimal, not double

    public Member(String name, LendingPolicy policy) {
        this.name = name;
        this.policy = policy;
    }

    public LocalDate checkout(BookItem item, LocalDate today) {
        checkLimit();                               // <<include>>: happens every time
        LocalDate due = item.checkout(this, today);
        onLoan.add(item);
        return due;
    }

    public BigDecimal returnBook(BookItem item, LocalDate today) {
        if (!onLoan.remove(item)) throw new IllegalStateException(name + " does not hold " + item.barcode());
        LocalDate due = item.checkIn();
        BigDecimal fine = new FineCalculator().fineFor(due, today);
        if (fine.signum() > 0) chargeFine(fine);       // <<extend>>: only when late
        return fine;
    }

    private void checkLimit() {
        int max = policy.maxBooks(this);
        if (onLoan.size() >= max) {
            throw new IllegalStateException(name + " already has " + max + " books (limit)");
        }
    }

    private void chargeFine(BigDecimal amount) { finesDue = finesDue.add(amount); }

    public String name() { return name; }
    public BigDecimal finesDue() { return finesDue; }
    public List<BookItem> onLoan() { return Collections.unmodifiableList(onLoan); }
}
```

```java
// File: code/lld/ch03/library/Librarian.java
package lld.ch03.library;

/** Generalization between actors: a librarian can do everything a member can, plus more. */
public final class Librarian extends Member {
    private final Library library;

    public Librarian(String name, LendingPolicy policy, Library library) {
        super(name, policy);
        this.library = library;
    }

    public BookItem addCopy(String barcode, Book book) { return library.addCopy(barcode, book); }
}
```

```java
// File: code/lld/ch03/library/Library.java
package lld.ch03.library;

import java.util.LinkedHashMap;
import java.util.Map;

/**
 * The system boundary made concrete. Library composes its Catalog (created here,
 * never shared) and composes its copies (only a Library creates a BookItem it registers).
 */
public final class Library {
    private final String name;
    private final Catalog catalog = new Catalog();                  // composition
    private final Map<String, BookItem> copies = new LinkedHashMap<>();

    public Library(String name) { this.name = name; }

    BookItem addCopy(String barcode, Book book) {
        BookItem item = new BookItem(barcode, book);
        copies.put(barcode, item);
        catalog.index(item);
        return item;
    }

    public Catalog catalog() { return catalog; }
    public String name() { return name; }
}
```

```java
// File: code/lld/ch03/library/Main.java
package lld.ch03.library;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        LendingPolicy policy = new StandardPolicy();
        Library lib = new Library("Town Library");
        Librarian lee = new Librarian("Lee", policy, lib);

        Book dune = new Book("978-0441013593", "Dune", "Frank Herbert");
        Book hobbit = new Book("978-0547928227", "The Hobbit", "J.R.R. Tolkien");
        lee.addCopy("B-1", dune);
        lee.addCopy("B-2", dune);
        lee.addCopy("B-3", hobbit);
        lee.addCopy("B-4", hobbit);

        Member maya = new Member("Maya", policy);
        LocalDate day1 = LocalDate.of(2026, 3, 1);

        // Sequence: member -> catalog.search -> (self) lookUpTitleIndex -> copies; member -> copy.checkout -> due
        List<BookItem> hits = lib.catalog().search("dune");
        System.out.println("search(\"dune\") -> " + hits);
        LocalDate due = maya.checkout(hits.get(0), day1);
        System.out.println("Maya borrowed B-1, due " + due);
        maya.checkout(lib.catalog().search("the hobbit").get(0), day1);
        System.out.println("Maya on loan: " + maya.onLoan());

        try {
            maya.checkout(hits.get(1), day1);   // <<include>> limit check fires
        } catch (IllegalStateException e) {
            System.out.println("Blocked: " + e.getMessage());
        }

        try {
            new Member("Omar", policy).checkout(hits.get(0), day1);   // copy already LOANED
        } catch (IllegalStateException e) {
            System.out.println("Blocked: " + e.getMessage());
        }

        BigDecimal onTime = maya.returnBook(hits.get(0), day1.plusDays(10));
        System.out.println("Returned B-1 on time, fine = " + onTime);
        BookItem hobbitCopy = maya.onLoan().get(0);
        BigDecimal late = maya.returnBook(hobbitCopy, day1.plusDays(17));   // <<extend>> fine applies
        System.out.println("Returned " + hobbitCopy.barcode() + " 3 days late, fine = " + late
                + ", Maya owes " + maya.finesDue());
        System.out.println("Librarian limit: " + policy.maxBooks(lee) + ", member limit: " +
policy.maxBooks(maya));
    }
}
```

Output:

```
search("dune") -> [B-1[Dune, AVAILABLE], B-2[Dune, AVAILABLE]]
Maya borrowed B-1, due 2026-03-15
Maya on loan: [B-1[Dune, LOANED], B-3[The Hobbit, LOANED]]
Blocked: Maya already has 2 books (limit)
Blocked: B-1 is LOANED
Returned B-1 on time, fine = 0.00
Returned B-3 3 days late, fine = 15.00, Maya owes 15.00
Librarian limit: 10, member limit: 2
```

Check the diagram against the code: every arrow is a real method — `Catalog.search`, `Catalog.lookUpTitleIndex`, `Member.checkLimit`, `BookItem.checkout`. **If a message has no method, your class diagram is missing one** — the same check the activity diagram does, from another direction.

Trade-offs

Good at	Poor at
Order of calls, who talks to whom	Branch-heavy flows (nested `alt` becomes unreadable)
Showing sync vs async	Showing the whole design
Spotting missing methods	Being quick to draw — each message is a line

When not to use it

- For a flow with three or more decisions: use an **activity diagram**. A second `alt`/`opt` box on the same diagram is the warning sign.
- Unprompted in most OOD interviews — it costs time you need for the class diagram.

Common mistakes

- Drawing a return for every call — the diagram doubles in height and says nothing more.
- Messages that are not methods on the receiving class.
- Mixing up arrowheads: filled = sync call, open solid = async, dashed = reply.
- Drawing classes instead of the specific objects when two objects of the same class take part (e.g. `from : Account`, `to : Account`).

Real-world examples

- **OAuth 2.0 / OpenID Connect** (standard protocols for "log in with another account") login flows (browser → app → authorization server → token endpoint) are almost always documented as sequence diagrams.
- **Payment integrations** (Stripe, Razorpay) document "create order → redirect → webhook" (a webhook is an HTTP call the provider makes to your server when something happens) as sequence diagrams, with the webhook as an asynchronous message.
- **Microservice sagas** (microservices are small, separately deployed services; a saga is a multi-step transaction in which every step has an "undo"; e.g. a travel booking that reserves a flight, a hotel and a car) are usually explained with sequence diagrams that show the compensating "undo" calls when one step fails.

In the interview

> **30-second answer:** "A sequence diagram shows one scenario: objects across the top, time downwards. Solid filled arrows are sync calls, open arrows async, dashed arrows returns. `alt` / `opt` / `loop` fragments show if/else, if, and loops. I keep it to three or four objects and six to eight messages, and check that every arrow is a method on my class diagram."

Follow-ups:

- *Q: Sync vs async message?* — A: Sync: caller blocks until the callee returns (a normal method call). Async: caller continues immediately (a queue message, event, `CompletableFuture`).
- *Q: When would you use a sequence diagram over an activity diagram?* — A: When the interesting part is *who calls whom* in what order; activity when the interesting part is branching.

Self-check

- *Q: What does a thin rectangle on a lifeline mean?* → A: An activation bar — the object is executing.
- *Q: Which fragment models "retry up to 3 times"?* → A: `loop [attempt <= 3]`.
- *Q: How do you show that `Order` creates a `Payment` mid-flow?* → A: A create message (dashed arrow, «create») pointing to the new lifeline's box, which starts lower on the page.

TL;DR

Objects across, time down. Filled = sync call, open = async, dashed = return. `alt` = if/else, `opt` = if, `loop` = loop, `par` = parallel. Keep it small and check each arrow is a real method.

3.13 Activity diagrams

What it is

An **activity diagram** traces **a single flow end to end**, marking each point where it **branches** (and any steps that run in parallel). Where a class diagram captures the design at rest, an activity diagram follows one route through it. Among UML diagrams it is the closest thing to a flowchart, so nearly everyone can follow it — which also makes a careless one easy to produce.

The problem it solves

It makes the business logic of a flow — every `if`, `unless`, `otherwise` in the requirements — visible and checkable. Its best interview use is as a **test of your class diagram**: walk the flow action by action and ask *which class does this?* If an action has no home, you have found a missing class — on the whiteboard, not in the code.

How it works — the symbols

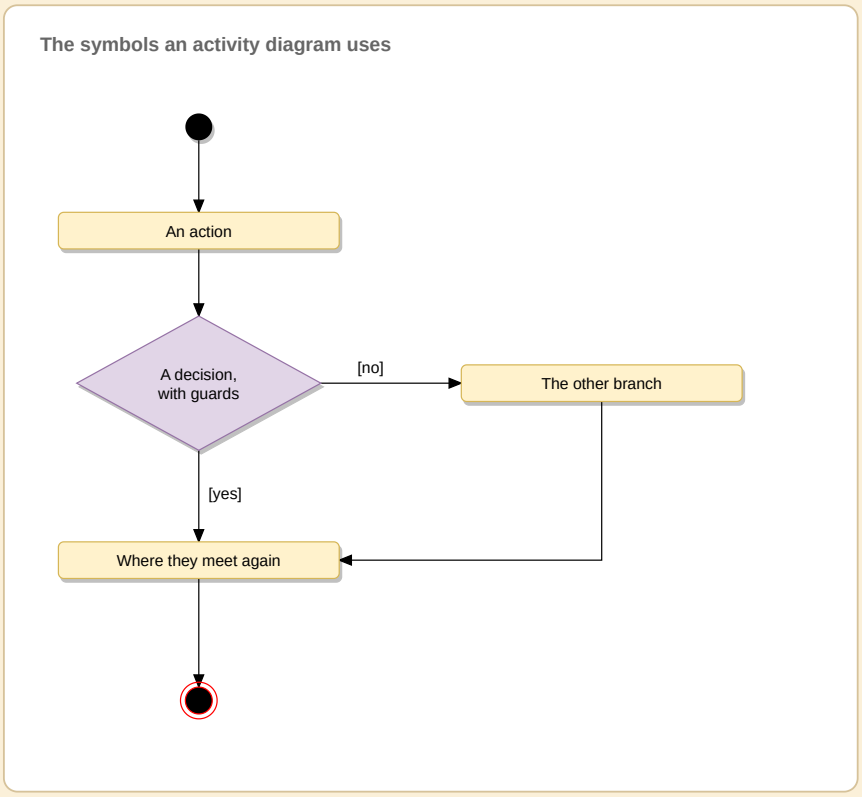


Figure — The core activity-diagram symbols: start, action, a decision with [yes]/[no] guards, the other branch, where the branches meet again, and the end (course diagram)

Symbol	Drawn as	Meaning
Initial node (start)	filled black circle	where the flow begins (draw exactly one; UML technically allows several, but one keeps the diagram readable)
Action	rounded rectangle with a verb phrase	one step: "System calculates the parking fee"
Control flow	arrow	time runs along the arrows
Decision	diamond, each outgoing arrow has a **guard** `[condition]`	a branch; guards must cover every case (use `[else]`)
Merge	diamond (or arrows meeting one node)	branches come back together
Fork	thick bar, one arrow in, many out	the following actions run **in parallel**
Join	thick bar, many in, one out	wait until **all** parallel actions finish
Activity final (end)	filled circle inside a ring (bullseye)	the whole activity stops; there may be several ("payment failed", "gate opened")
Flow final	circle with an X	only *this* branch stops; others continue
Swimlane (partition)	vertical or horizontal lanes labelled with an actor/class	*who* performs each action
Object node	rectangle	data passed between actions (e.g. `[Ticket]`)
Send / receive signal	convex / concave pentagon	sending or waiting for an event

Steps to draw one:

1. Pick **one** flow, ideally the one with an interesting branch. A requirement containing "if", "unless" or "otherwise" describes a branch — draw that.
2. Start node → actions in the language of the problem ("The system calculates the fee", **not** "call `calculateFee()`").
3. At each decision, draw every outcome with a guard.
4. Use fork/join only when steps really happen at the same time. Most interview flows do not need them.
5. Add swimlanes if it helps to show which actor/class does each step.
6. End node(s).

Example 1 — online shopping (course)

Every activity diagram has the shape of the small symbols figure above, and it reads as a single sentence: *begin, perform an action, ask a question, follow one of two paths, merge, end.* Here is a bigger example:

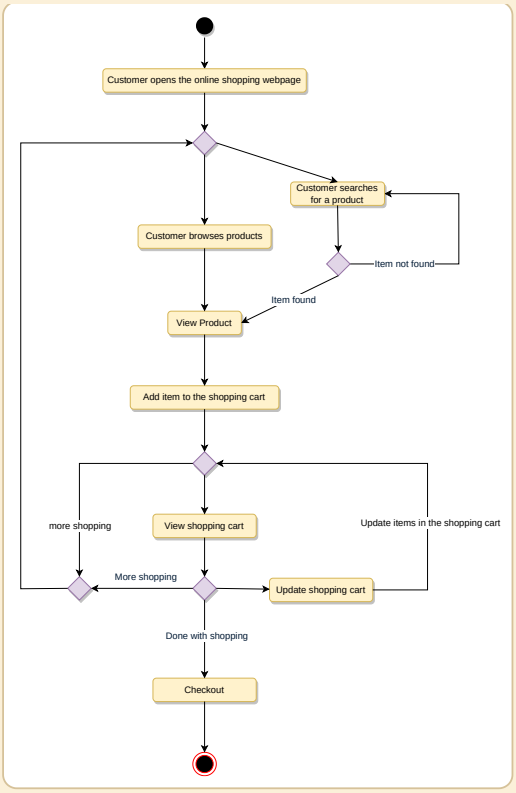
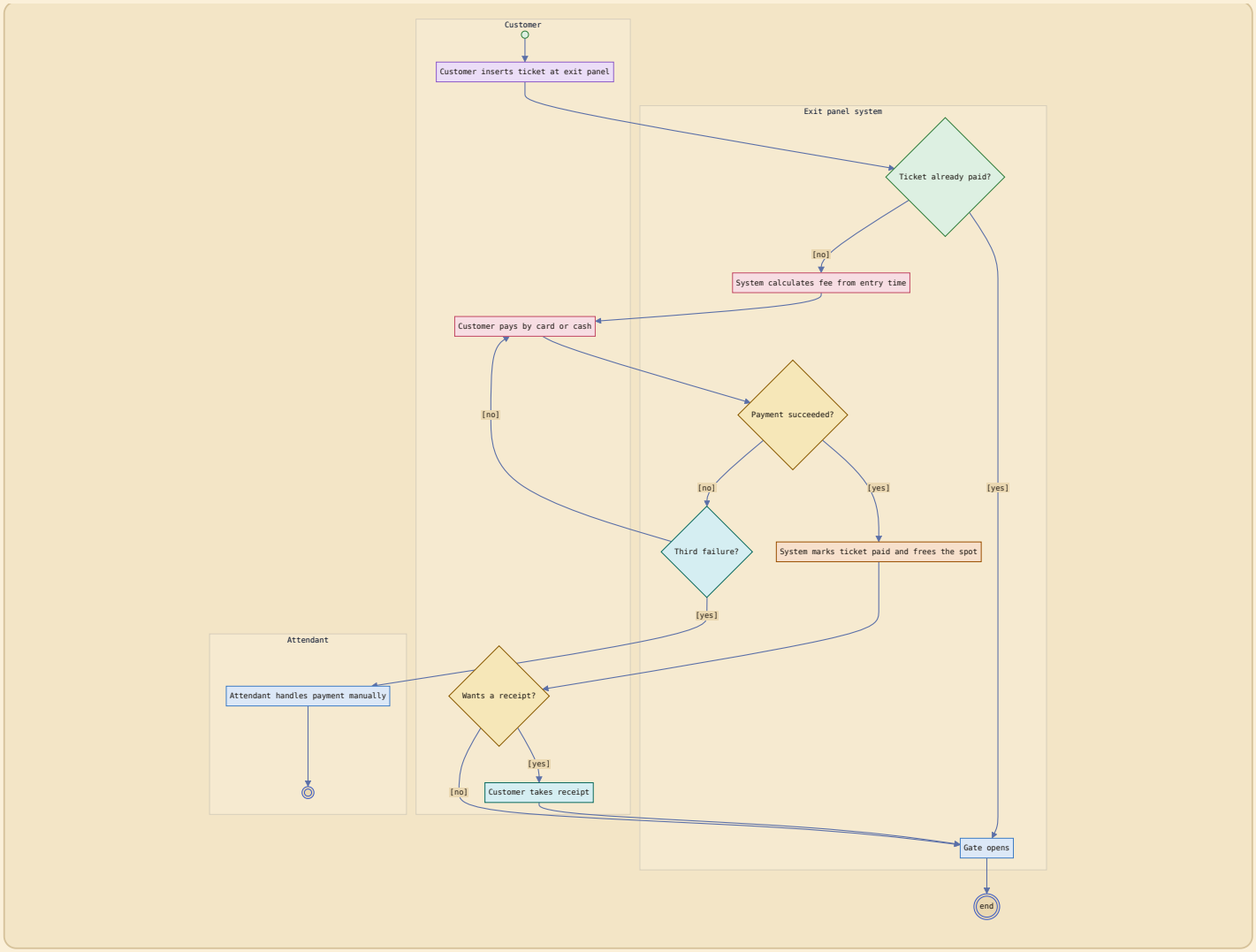


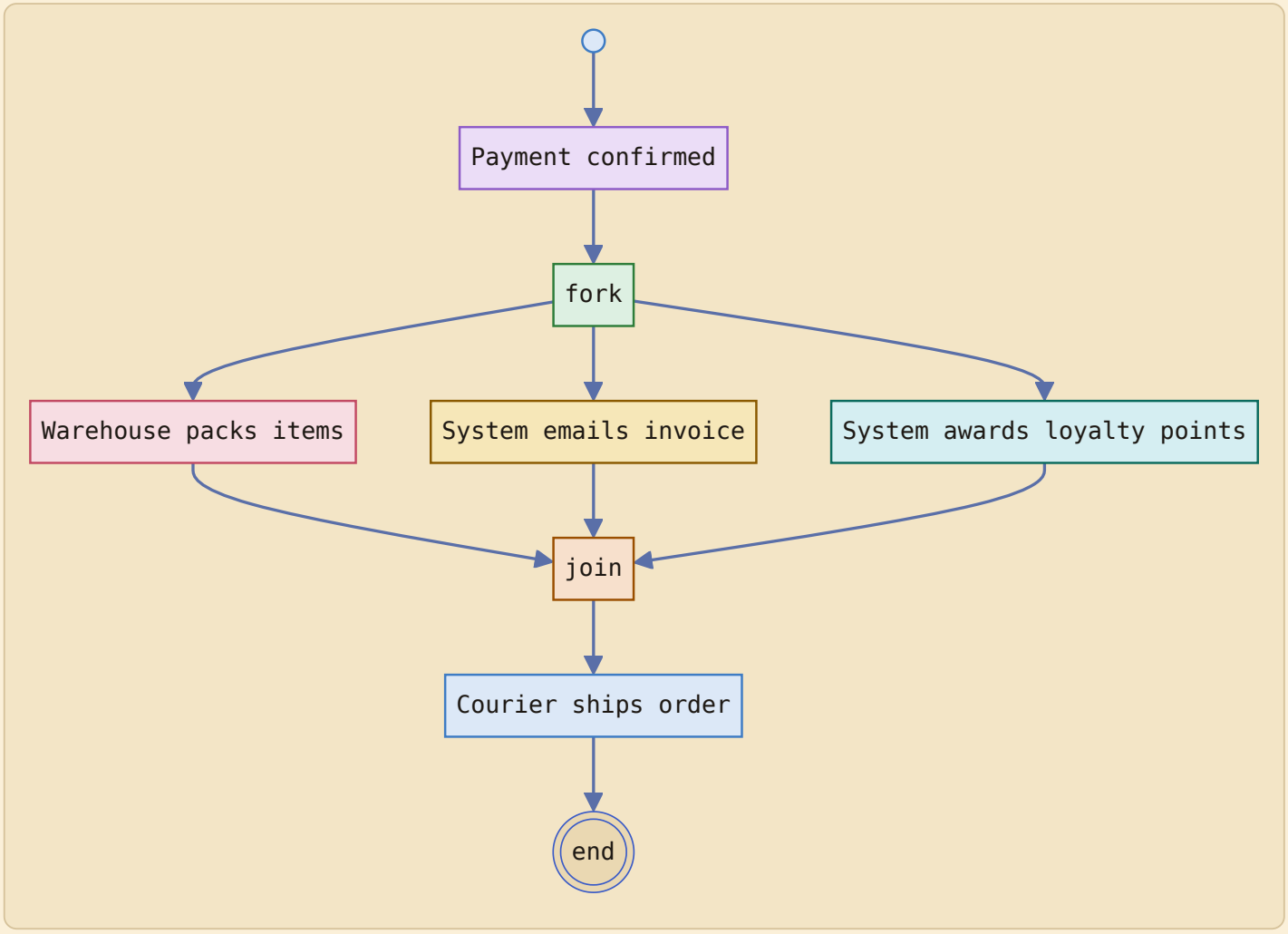
Figure — A customer shopping online: search, item found or not found, add to cart, view/update the cart, more shopping or checkout (course diagram)

Example 2 — pay at the parking exit (three decisions, with swimlanes)

Three decisions appear in the Parking Lot case study (Chapter 14): *is the ticket paid already?*, *did payment go through?* and *does the customer ask for a receipt?* Every one traces back to a requirement. Below it is drawn with swimlanes (subgraphs), a retry loop, and a hand-off to an attendant once three attempts fail:


Customer
Customer inserts ticket at exit panel
Exit panel system
Ticket already paid?
[no]
System calculates fee from entry time
Customer pays by card or cash
[no]
Payment succeeded?
[no]
[yes]
[yes]
Third failure?
System marks ticket paid and frees the spot
[yes]
Attendant
Attendant handles payment manually
Wants a receipt?
[yes]
[no]
Customer takes receipt
Gate opens
end

Example 3 — fork and join (order fulfilment)



After the **fork** bar the three actions happen in parallel; the **join** waits for all three before shipping.

Activity vs sequence diagram

	Activity diagram	Sequence diagram
Shows	a process as a series of steps	objects sending messages to each other
Branches	natural: decisions with guards	awkward: nested `alt` fragments
Names	actions, in business language	objects and method calls
Use when	the flow has interesting branches	the order of calls is the interesting part
Answers	"what happens?"	"who calls whom?"

Rule of thumb: a flow with **three branches** calls for an activity diagram; **four objects** whose call order is easy to confuse call for a sequence diagram.

Trade-offs

Pro	Con
Anyone (including non-engineers) can read it	Does not show which object does what unless you add swimlanes
Great at branches and parallelism	Big flows sprawl quickly

When not to use it

For a straight-line flow ("customer pays and leaves") — there is nothing to decide, so the picture adds nothing. For object-to-object call order, use a sequence diagram.

Common mistakes

- **A diamond with only one exit.** If only one outcome is possible, nothing is being decided — draw the missing branch or remove the diamond.
- **Actions written as code** ("call `calculateFee()`"). Keep it in the language of the problem so it can be checked against the requirements.
- Guards that do not cover every case (`[amount > 100]` and `[amount < 100]` — what about exactly 100?).
- Using fork/join for steps that are merely sequential.

Real-world examples

- **BPMN** (Business Process Model and Notation), used by workflow engines (software that runs business processes step by step) such as Camunda, is a close cousin of UML activity diagrams, with lanes and gateways.
- **CI/CD pipelines** (Continuous Integration / Continuous Delivery: automatically building, testing and deploying code on every change; e.g. GitHub Actions, Jenkins) are activity diagrams in practice: parallel jobs (fork), a job that `needs:` several others (join), conditional steps (guards).
- Food-delivery apps (Swiggy, Zomato, DoorDash) typically let the restaurant prepare the food while a delivery partner is being assigned — two parallel branches that join at "order picked up".

In the interview

> **30-second answer:** "An activity diagram walks one flow: start, actions, decisions with guards covering every case, merges, optional fork/join for parallel steps, swimlanes for who does what, and one or more ends. I draw the flow with the interesting branch, then check that every action has a class to live in."

Follow-ups:

- *Q: Activity final vs flow final?* — A: Activity final ends the whole activity; flow final ends only that one path while other parallel paths continue.
- *Q: Which flow do you draw?* — A: The one whose requirement contains "if/unless/otherwise" — e.g. payment failure and retry.

Self-check

- *Q: What must every arrow leaving a decision have?* → A: A guard in square brackets; together they must cover all cases.
- *Q: How do you show two steps that happen at the same time?* → A: A fork bar before them and a join bar after them.

TL;DR

One flow, start → actions → decisions with guards → merge → end. Fork/join for parallel, swimlanes for who. Draw the branching flow and use it to test your class diagram.

3.14 State machine diagrams

What it is

A **state machine diagram** (also called a *statechart* or *state diagram*) shows the **states one object can be in** during its life and the **transitions** that move it from one state to another in response to **events**.

- **State** — a condition during which the object behaves in a certain way or waits (rounded rectangle): `CREATED`, `PAID`, `SHIPPED`.
- **Initial pseudostate** — filled circle, arrow to the first state.
- **Final state** — bullseye; the object's life (in this machine) is over.
- **Transition** — arrow labelled `event [guard] / action`:
  - **event (trigger)** — what happens: `pay(amount)`.
  - **guard** — boolean condition that must be true for the transition to fire: `[amount == total]`.
  - **action (effect)** — what the object does while transitioning: `/ refund()`.

- **Internal activities** inside a state: `entry / ...` (on entering), `exit / ...` (on leaving), `do / ...` (while in the state).
- **Self-transition** — an arrow from a state back to itself (exit and re-entry actions run).
- **Choice pseudostate** — a small diamond that picks the next state based on guards.
- **Composite state** — a state containing sub-states (e.g. `Active` containing `Idle` and `Moving`); **history** `(H)` remembers the last sub-state.
- **Orthogonal regions** — a state split by dashed lines into parts that are active at the same time.

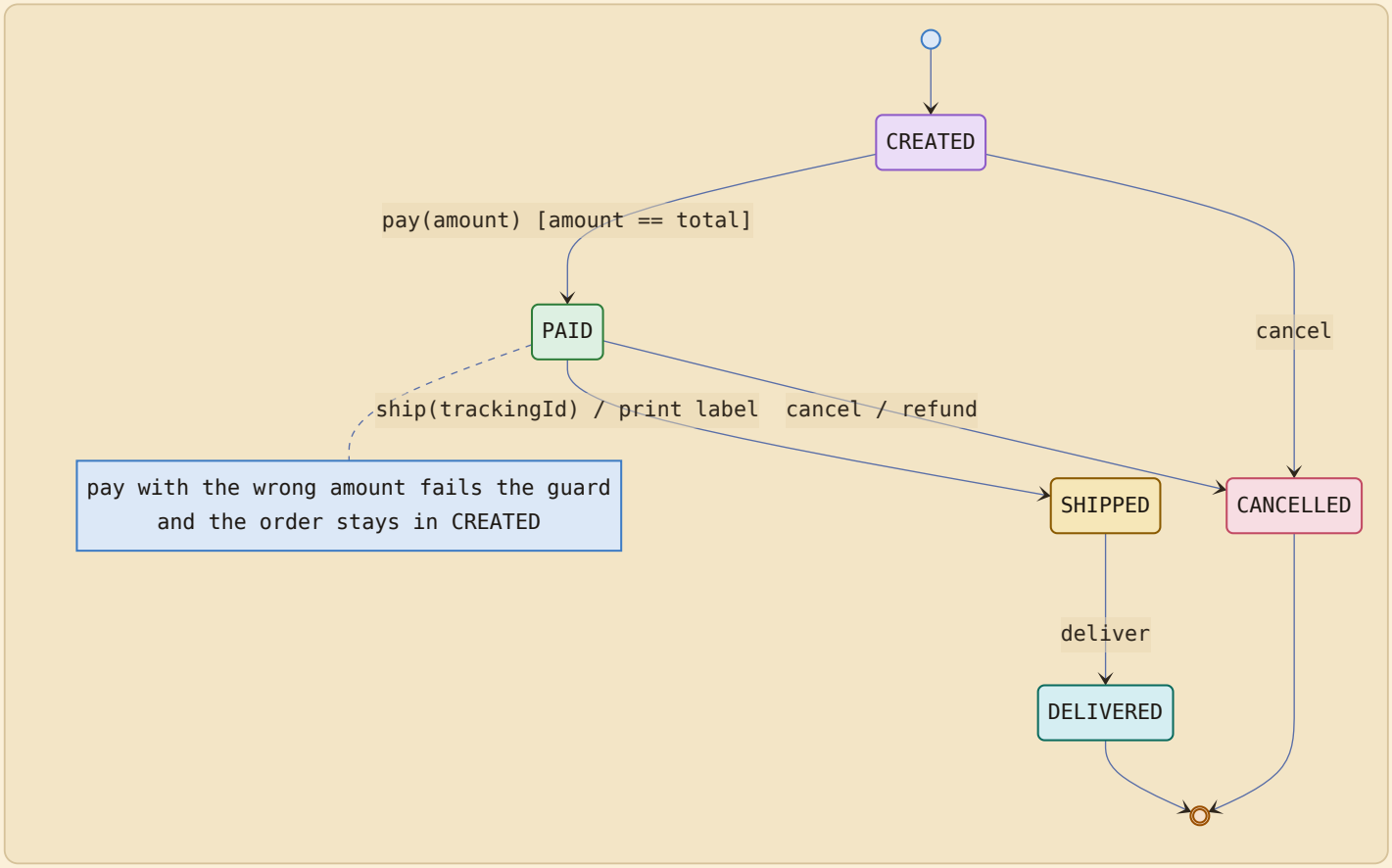
Analogy: a **traffic light**. It is always in exactly one state (red, green, yellow), a timer event moves it on, and the allowed order is fixed — it never goes straight from green to red.

The problem it solves

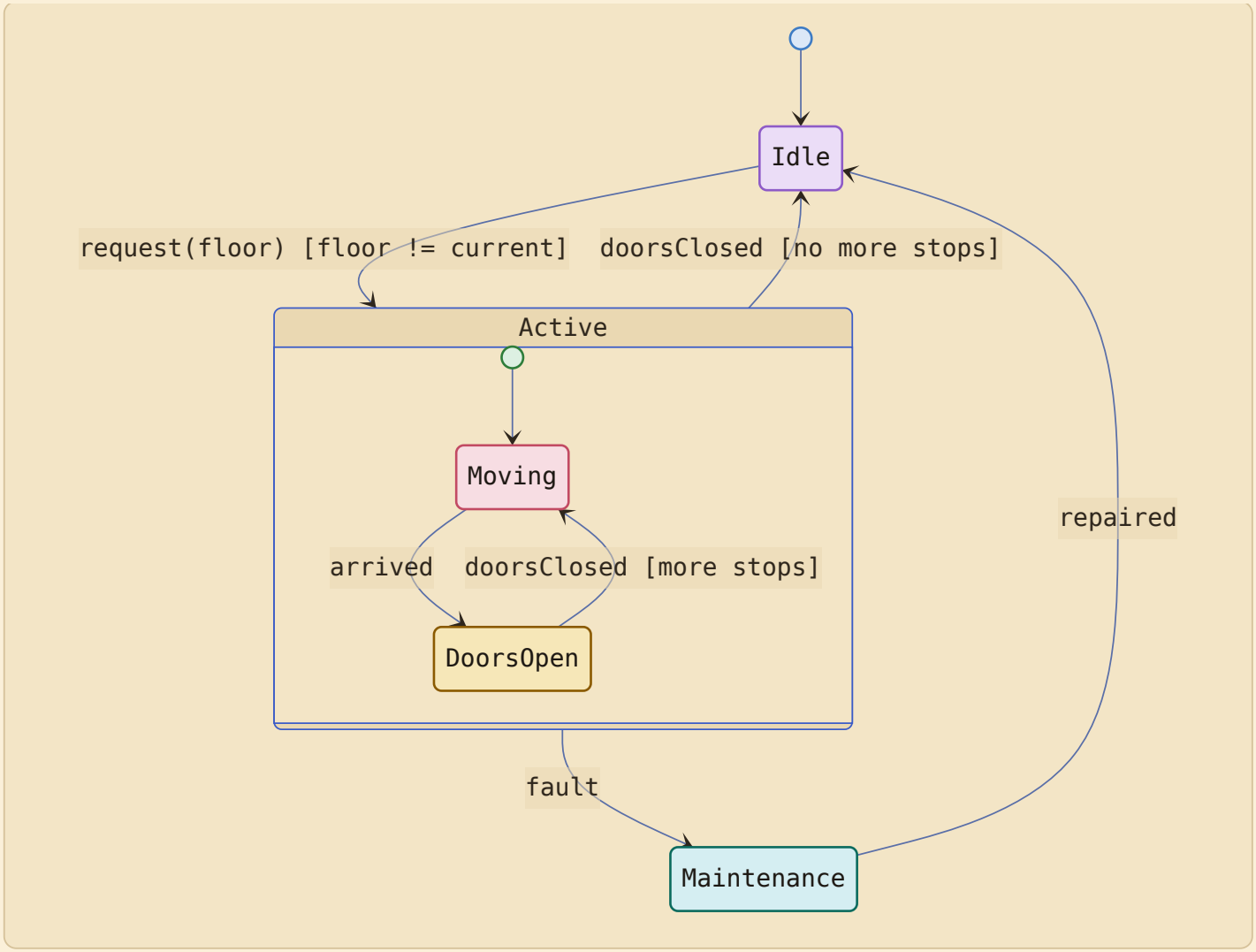
Objects with a lifecycle (orders, tickets, elevators, vending machines, parking spots, TCP connections, TCP being the network protocol for reliable connections, see System Design Fundamentals) are the source of many bugs: shipping a cancelled order, paying twice, refunding an undelivered item. A state diagram lists **every legal transition** so that everything else can be rejected. It also maps directly to code (an `enum` of states plus a transition method, or the **State pattern**, Chapter 11).

How it works

1. Pick **one** class with a lifecycle.
2. List its states (usually an `enum`).
3. For each state, list the events it accepts, the guard (if any), the target state and the action.
4. Everything not listed is **illegal** — the code must reject it.
5. Mark the initial and final states.



A composite-state example (an elevator car) for recognition:



Java example (`lld.ch03.statemachine`)

```java
// File: code/lld/ch03/statemachine/OrderState.java
package lld.ch03.statemachine;

public enum OrderState { CREATED, PAID, SHIPPED, DELIVERED, CANCELLED;
    public boolean isFinal() { return this == DELIVERED || this == CANCELLED; }
}
```

```java
// File: code/lld/ch03/statemachine/OrderEvent.java
package lld.ch03.statemachine;

import java.math.BigDecimal;

/** Events that can trigger transitions. Sealed, so the switch in Order is checked for completeness. */
public sealed interface OrderEvent {
    record Pay(BigDecimal amount) implements OrderEvent { }
    record Ship(String trackingId) implements OrderEvent { }
    record Deliver() implements OrderEvent { }
    record Cancel(String reason) implements OrderEvent { }
}
```

```java
// File: code/lld/ch03/statemachine/Order.java
package lld.ch03.statemachine;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

/**
 * Code for the state machine diagram. Each transition is written as
 *   event [guard] / action
 * An event that has no transition from the current state is rejected.
 */
public final class Order {
    private final String id;
    private final BigDecimal total;   // money: BigDecimal, not double
    private OrderState state = OrderState.CREATED;   // initial pseudostate -> CREATED
    private final List<String> log = new ArrayList<>();

    public Order(String id, BigDecimal total) {
        this.id = id;
        this.total = total;
        log.add("[*] -> CREATED");
    }

    public OrderState state() { return state; }
    public List<String> log() { return List.copyOf(log); }

    public void handle(OrderEvent event) {
        OrderState from = state;
        OrderState to = switch (event) {
            case OrderEvent.Pay p when state == OrderState.CREATED && p.amount().compareTo(total) == 0 ->
OrderState.PAID;
            case OrderEvent.Pay p when state == OrderState.CREATED ->
                    reject(event, "guard failed: paid " + p.amount() + " but total is " + total);
            case OrderEvent.Ship s when state == OrderState.PAID -> {
                log.add("  action: print label " + s.trackingId());
                yield OrderState.SHIPPED;
            }
            case OrderEvent.Deliver d when state == OrderState.SHIPPED -> OrderState.DELIVERED;
            case OrderEvent.Cancel c when state == OrderState.CREATED -> OrderState.CANCELLED;
            case OrderEvent.Cancel c when state == OrderState.PAID -> {
                log.add("  action: refund " + total);
                yield OrderState.CANCELLED;
            }
            default -> reject(event, "no transition from " + state);
        };
        state = to;
        log.add(from + " --" + event.getClass().getSimpleName() + "--> " + to);
    }

    private OrderState reject(OrderEvent e, String why) {
        throw new IllegalStateException(id + ": " + e.getClass().getSimpleName() + " rejected, " + why);
    }
}
```

```java
// File: code/lld/ch03/statemachine/Main.java
package lld.ch03.statemachine;

import java.math.BigDecimal;

public class Main {
    public static void main(String[] args) {
        Order happy = new Order("ORD-1", new BigDecimal("499.00"));
        happy.handle(new OrderEvent.Pay(new BigDecimal("499.00")));
        happy.handle(new OrderEvent.Ship("TRK-77"));
        happy.handle(new OrderEvent.Deliver());
        System.out.println("ORD-1 path:");
        happy.log().forEach(l -> System.out.println("  " + l));
        System.out.println("  final? " + happy.state().isFinal());

        Order refund = new Order("ORD-2", new BigDecimal("120.00"));
        tryEvent(refund, new OrderEvent.Pay(new BigDecimal("100.00")));   // guard fails
        tryEvent(refund, new OrderEvent.Pay(new BigDecimal("120.00")));
        tryEvent(refund, new OrderEvent.Cancel("changed mind"));
        tryEvent(refund, new OrderEvent.Ship("TRK-99"));                        // no transition from CANCELLED
        System.out.println("ORD-2 path:");
        refund.log().forEach(l -> System.out.println("  " + l));
    }

    private static void tryEvent(Order o, OrderEvent e) {
        try {
            o.handle(e);
        } catch (IllegalStateException ex) {
            System.out.println("Rejected: " + ex.getMessage());
        }
    }
}
```

What each part shows: `OrderState` is the list of states. Amounts are `BigDecimal` (never `double` for money), so the guard `[amount == total]` is written `p.amount().compareTo(total) == 0` — `BigDecimal.equals` would treat `120.0` and `120.00` as different because it also compares scale. `OrderEvent` is a **sealed** interface (only the types listed inside it may implement it) of event records, so the `switch` in `Order.handle` uses Java 21 **pattern matching with `when` guards** — each `case` line is literally one arrow of the diagram: `event [guard] -> target` with its action. The `default` branch rejects every transition that is not on the diagram.

Output:

```
ORD-1 path:
  [*] -> CREATED
  CREATED --Pay--> PAID
    action: print label TRK-77
  PAID --Ship--> SHIPPED
  SHIPPED --Deliver--> DELIVERED
  final? true
Rejected: ORD-2: Pay rejected, guard failed: paid 100.00 but total is 120.00
Rejected: ORD-2: Ship rejected, no transition from CANCELLED
ORD-2 path:
  [*] -> CREATED
  CREATED --Pay--> PAID
    action: refund 120.00
  PAID --Cancel--> CANCELLED
```

Trade-offs

enum + switch (this example)	State pattern (class per state, Ch. 11)
All transitions visible in one place	Each state's behaviour in its own class
Best for few states, simple actions	Best when states have lots of different behaviour
Adding a state edits the switch	Adding a state adds a class (Open/Closed)

When not to use it

For objects with only a flag or two and no real lifecycle (`active/inactive`). For flows that span many objects, use an activity diagram instead — a state machine describes **one** object.

Common mistakes

- Confusing states with actions: "Paying" (an action) vs "PAID" (a state).
- Missing transitions (e.g. forgetting that a paid order can be cancelled with a refund) or allowing illegal ones by default.
- Guards that overlap, so two transitions could fire for the same event.
- Drawing a state machine for the *system* rather than for one object.

Real-world examples

- `java.lang.Thread.State`: `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, `TERMINATED`.
- **TCP** connection states (`LISTEN`, `SYN_SENT`, `ESTABLISHED`, `FIN_WAIT_1`, …) — the classic protocol state machine.
- **Amazon / Flipkart order tracking** (ordered → packed → shipped → out for delivery → delivered; cancel allowed only before shipping).
- **Uber trip** (requested → driver assigned → driver arrived → in progress → completed / cancelled).
- **Spring Statemachine** and AWS Step Functions (Amazon's cloud workflow service) are libraries/services built around state machines.

In the interview

> **30-second answer:** "When an object has a lifecycle — order, ticket, elevator, vending machine — I draw its states with transitions labelled `event [guard] / action`. Anything not on the diagram is illegal and the code rejects it. In Java it is an enum of states with one transition method, or the State pattern if each state has a lot of behaviour."

Follow-ups:

- *Q: What is a guard?* — A: A boolean condition in `[ ]` that must be true for the transition to fire, e.g. `[amount == total]`.
- *Q: State machine vs activity diagram?* — A: A state machine is about one object's states over its lifetime; an activity diagram is about the steps of a process, possibly across many objects.
- *Q: How do you make transitions thread-safe?* — A: Make the transition method `synchronized` (or use a lock / compare-and-set on an `AtomicReference<State>`; compare-and-set changes a value only if it still equals what you last read, in one atomic step), so two events cannot both read the old state and both move on.

Self-check

- *Q: Read `PAID --> CANCELLED : cancel / refund`.* → A: In PAID, the event `cancel` moves to CANCELLED and performs the action `refund`.
- *Q: What happens to an event that has no transition from the current state?* → A: It is ignored or rejected — in our code, rejected with an exception.

TL;DR

One object, its states, and `event [guard] / action` arrows. Initial dot, final bullseye. Everything not drawn is illegal. Code: enum + guarded switch, or the State pattern.

Part C — UML in the interview

3.15 How much UML to draw in an interview

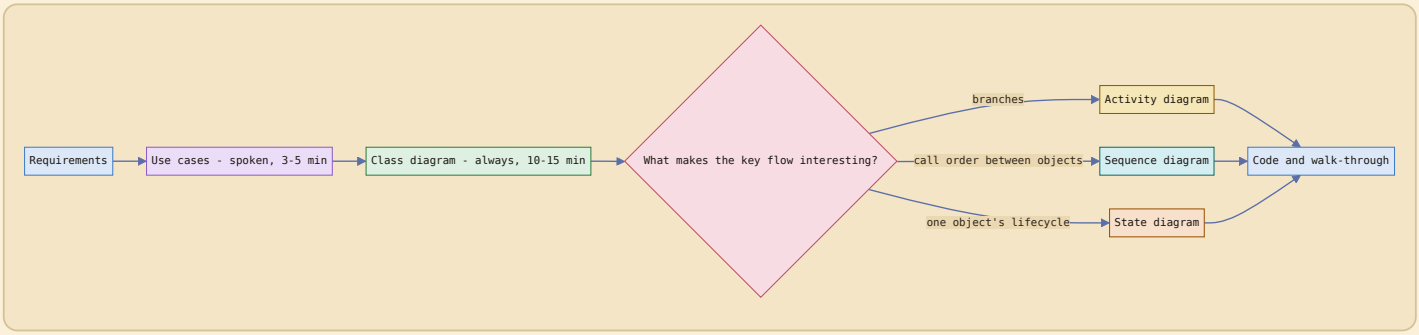
The interviewer is not marking your arrowheads. What is checked is whether **your diagram and your words agree**. Leaving a line out costs you less than drawing one you cannot justify.

Suggested budget for a 45-minute OOD round:

Minute	Diagram	Depth
0–5	**Use case**	Usually *spoken* or a quick list: actors + what each does + what is out of scope (the boundary). Draw it only if asked.
5–20	**Class diagram**	*Always drawn.* 10–15 boxes, every line typed with multiplicity; key attributes and methods only.
20–25	**Activity** *or* **sequence** *or* **state**	One diagram for the one flow that carries the design: activity if it branches, sequence if call order matters, state if one object has a lifecycle.
25–45	Code + discussion	Code the core classes; walk the flow against your diagrams.

Rules of thumb

1. **If you only have one picture, make it the class diagram.** It is the answer.
2. **Drop attributes before relationships** when short of time.
3. **Never leave a line unlabelled** — type + multiplicity.
4. **Say the test aloud** for every diamond and triangle.
5. **Use the other diagrams as tests:** every action in the activity diagram and every arrow in the sequence diagram must be a method on a class in the class diagram.
6. **Keep sequence diagrams small:** 3–4 objects, 6–8 messages, no branches.
7. **Machine-coding rounds** (90–120 min, working code expected): a 2-minute class sketch is enough — spend the time on code.
8. If you forget a symbol, **say what you mean** ("this implements that") and move on.



In the interview

> **30-second answer:** "I list use cases verbally to fix scope, spend most of my diagram time on one class diagram with typed, multiplicity-labelled lines, then draw one behaviour diagram for the key flow and use it to check that every step has a class and method."

3.16 Chapter self-check

1. *Q: Name the six relationships from weakest to strongest coupling.* → A: Dependency, association, aggregation, composition, realization, generalization.
2. *Q: Where does the diamond go, and what does filled vs hollow mean?* → A: At the whole; filled = composition (parts die with it), hollow = aggregation (parts live on).
3. *Q: Code smell (a surface sign of a likely design problem):* `CompactSpot`, `LargeSpot` *subclasses that only pass a constant to* `super`*. Fix?* → A: One `ParkingSpot` with a `SpotType` enum field.
4. *Q: Java type for multiplicity* `0..1` *and* `1..*`*?* → A: `Optional<T>` (nullable field); a non-empty collection checked in the constructor.
5. *Q: «include» vs «extend»?* → A: Include = always, arrow from base to included; extend = sometimes (conditional), arrow from extension to base.
6. *Q:* `+ - # ~`*?* → A: public, private, protected, package.
7. *Q: Arrowheads in sequence diagrams?* → A: Solid + filled = sync call; solid + open = async; dashed + open = return.

8. *Q: `alt` vs `opt`?* → A: `alt` = if/else with several guarded sections; `opt` = a single guarded section (if without else).
9. *Q: Fork/join?* → A: Fork splits one flow into parallel flows; join waits for all of them.
10. *Q: Transition label syntax in a state machine?* → A: `event [guard] / action`.
11. *Q: 14 UML diagrams split how?* → A: 7 structural (class, object, package, component, composite structure, deployment, profile) and 7 behavioural (use case, activity, state machine, sequence, communication, interaction overview, timing).
12. *Q: If you can draw only one diagram?* → A: The class diagram.

3.17 TL;DR

- **Relationships are claims about lifetime and substitution.** Dependency (uses, dashed arrow) · Association (knows, line/arrow) · Aggregation (has, parts live on, ◇ at whole) · Composition (owns, parts die, ◆ at whole) · Realization (implements, dashed △) · Generalization (extends, solid △).
- **Choose in order:** substitution test → dies with whole? → part that outlives? → else association; parameter-only → dependency. Inheritance last.
- **Multiplicity changes code:** `1` required, `0..1` Optional, `1..*` non-empty collection, exact numbers checked in constructors.
- **UML:** 14 diagrams, structural vs behavioural. Interviews: use case (scope), class (the answer), activity (branches), sequence (call order), state machine (lifecycle).
- **Draw little, label everything, say the test out loud,** and use behaviour diagrams to test the class diagram.

Chapter 4 — SOLID Principles

> **What you will learn.** The five SOLID principles, one at a time and then all together. For each one you get a plain definition, an everyday analogy, a broken Java example, the refactored Java example, the rules behind it, the cost of applying it, and how to talk about it in a design interview. Along the way you also learn the ideas SOLID is built on: **cohesion**, **coupling** and **separation of concerns**.
>
> **How to use this chapter.** Read 4.1 first. Then read the principles in order, because each one sets up the next. Section 4.11 runs one class through all five principles, and it is the best single thing to rehearse before an interview. Sections 4.13 and 4.14 are about saying all this out loud.

Run the code. All code for this chapter is under `chapters/04-solid-principles/code/`, one package per topic. From that folder (`javac` is the Java compiler; `java` runs the compiled classes, and `-cp` tells it where they are):

```
javac --release 21 -d /tmp/lld_04 $(find . -name '*.java')
java -cp /tmp/lld_04 lld.ch04.srp.Main        # also: cohesion, coupling, soc, ocp, lsp, isp, dip, checkout
```

Package	What it shows
`lld.ch04.srp`	SRP: an `Employee` class split by actor
`lld.ch04.cohesion`	Low cohesion vs high cohesion
`lld.ch04.coupling`	Tight coupling vs loose coupling, in three steps
`lld.ch04.soc`	Separation of concerns: rules vs storage
`lld.ch04.ocp`	OCP: invoice formats behind an interface
`lld.ch04.lsp`	LSP: cars, the four contract rules, the history rule, birds and composition
`lld.ch04.isp`	ISP: fat printer interface, over-splitting, role interfaces
`lld.ch04.dip`	DIP: notifications, dependency injection, service locator, plug-ins
`lld.ch04.checkout`	One checkout class refactored through all five principles

A note on money. Every amount of money in this chapter's code (pay, invoice totals, prices, balances, order totals) is a `java.math.BigDecimal` (Java's exact decimal-number type) created from a string (`new BigDecimal("19.99")`), and every result that could gain extra decimal places (a multiplication by a rate) is rounded explicitly with `setScale(2, RoundingMode.HALF_EVEN)` (two decimal places, "banker's rounding": a value exactly halfway, such as 2.345, goes to the neighbour whose last digit is even, 2.34, so rounding errors do not all lean one way). Adding or subtracting two-decimal amounts stays at two decimals on its own. Never use `double` for money: binary floating point cannot store values such as 0.1 exactly, so `0.1 + 0.2` prints `0.30000000000000004`, and those tiny errors add up in totals. `BigDecimal` also has one trap: `equals` compares the scale (the number of decimal places) too (`2.0` is not equal to `2.00`), so compare amounts with `compareTo` or `signum()`, as the code here does.

4.1 What SOLID is and why you should learn it

What it is

SOLID is a set of five design guidelines for object-oriented code. Each letter names one principle:

Letter	Principle	The one-line rule
S	Single Responsibility Principle (SRP)	A class should have one reason to change.
O	Open/Closed Principle (OCP)	Add new behaviour by adding code, not by editing working code.
L	Liskov Substitution Principle (LSP)	A subtype must be usable anywhere its parent type is expected.
I	Interface Segregation Principle (ISP)	No client should be forced to depend on methods it does not use.
D	Dependency Inversion Principle (DIP)	Depend on abstractions, not on concrete details.

A few terms first, because the rest of the chapter uses them:

- A **class** is a blueprint for objects. A **module** is a bigger unit: a package, a library, a service.
- A **client** (or **caller**) is any code that uses another piece of code.
- An **interface** is a list of methods a class promises to provide, with no bodies. In Java it is the `interface` keyword. In general design talk it also means "the set of methods a caller can see".
- An **abstraction** is an interface or abstract class (a class marked `abstract` that cannot be created directly and may leave some methods for subclasses to write): something that says *what* is done without saying *how*.
- A **concrete class** is a class you can actually create with `new`.
- A **subtype** is a class that extends another class or implements an interface; the type it extends or implements is its **parent** (or base) type.

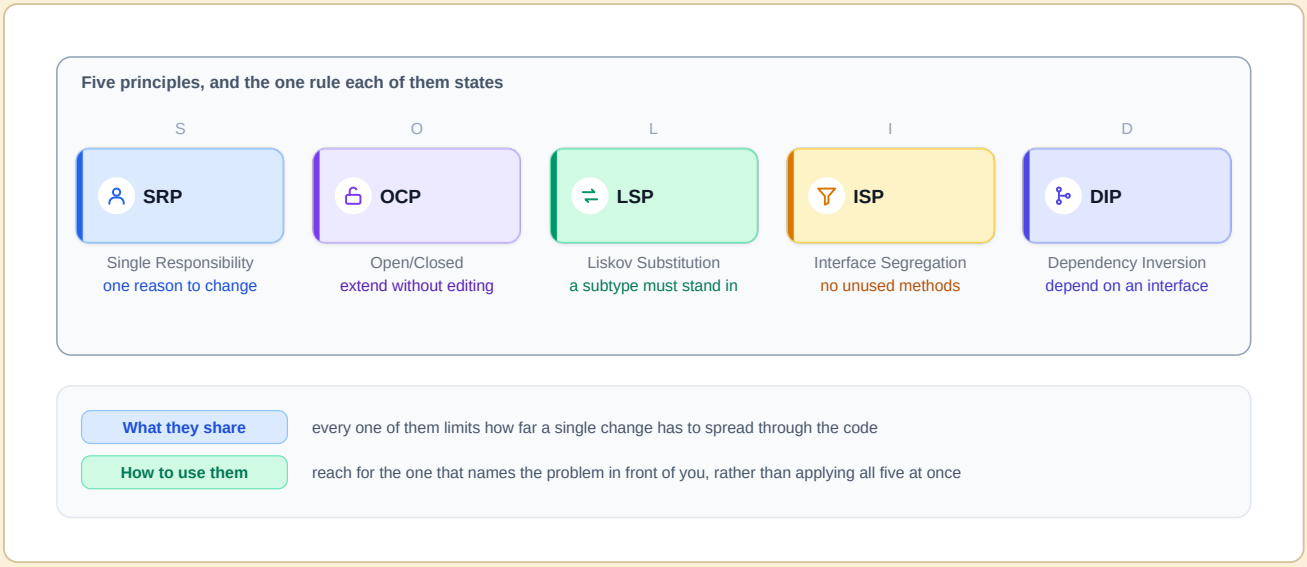


Figure — the five principles and the single rule each one states (course diagram)

In the figure above, look at the bottom line: every principle **limits how far one change spreads through the code**. That is the common thread. SRP limits it by giving each class one owner. OCP limits it by letting you add instead of edit. LSP makes sure a new subclass does not break old callers. ISP stops a change in one method from reaching callers that never use it. DIP stops a change in a low-level detail (a technical part such as a database driver) from reaching business logic (the rules the application exists for, such as pricing).

A little history. Robert C. Martin ("Uncle Bob") collected these principles in his writing around 2000, most fully in the paper *Design Principles and Design Patterns* and the book *Agile Software Development: Principles, Patterns, and Practices* (2002). Michael Feathers noticed the initials spelled SOLID. Two of the ideas are older: the Open/Closed Principle comes from Bertrand Meyer (1988), and the Substitution Principle from Barbara Liskov (1987).

The problem it solves

Code that works today still has to change tomorrow. Without guidelines, classes grow by small additions until one edit breaks three unrelated features. SOLID gives you:

- **Cleaner code.** Small, focused classes are easier to read and to hand to a teammate.
- **Flexibility.** New features plug in without breaking old ones.
- **Fewer bugs.** Less code is touched per change, so there is less to break.
- **Reuse.** A class with one job can be used in places its author never imagined.
- **Easier change.** When requirements shift, the change lands in one place.

- **A shared vocabulary.** This is the benefit people forget. When a teammate says "this class breaks ISP", everybody knows what the problem is, and a disagreement about design can be decided by an argument rather than by personal preference.

How to use SOLID (the honest version)

1. Treat each principle as a **name for a problem**, not a rule to obey everywhere.
2. When code is painful to change, find the principle that names the pain.
3. Apply the fix **when a real change asks for it**, not in advance.
4. Always say what the fix costs (more classes, more interfaces, more wiring, i.e. code that creates objects and connects them).

Applied without judgement, SOLID produces a maze of interfaces that each have one implementation. This chapter shows the cost of every recommendation along with its benefit.

> **TL;DR**
>
> SOLID = five guidelines that keep one change from spreading. Use them to **diagnose** code, apply them when a real change needs them, and always state the cost.

4.2 Single Responsibility Principle (SRP)

What it is

Robert C. Martin's first wording was roughly: **a class should have only one reason to change.** Later (in *Clean Architecture*, 2017) he sharpened it to: **a module should be responsible to one, and only one, actor.** An **actor** is a person or group who can ask for a change to the code.

The everyday version you will often hear is "every component should have one job". That version is a useful start, but it is hard to measure, as we will see.

> **Analogy: army knife vs kitchen knife.** A Swiss army knife has scissors, a bottle opener and a screwdriver. It does many jobs and is not the best tool for any of them. A kitchen knife has one job, cutting, and it is excellent at it.

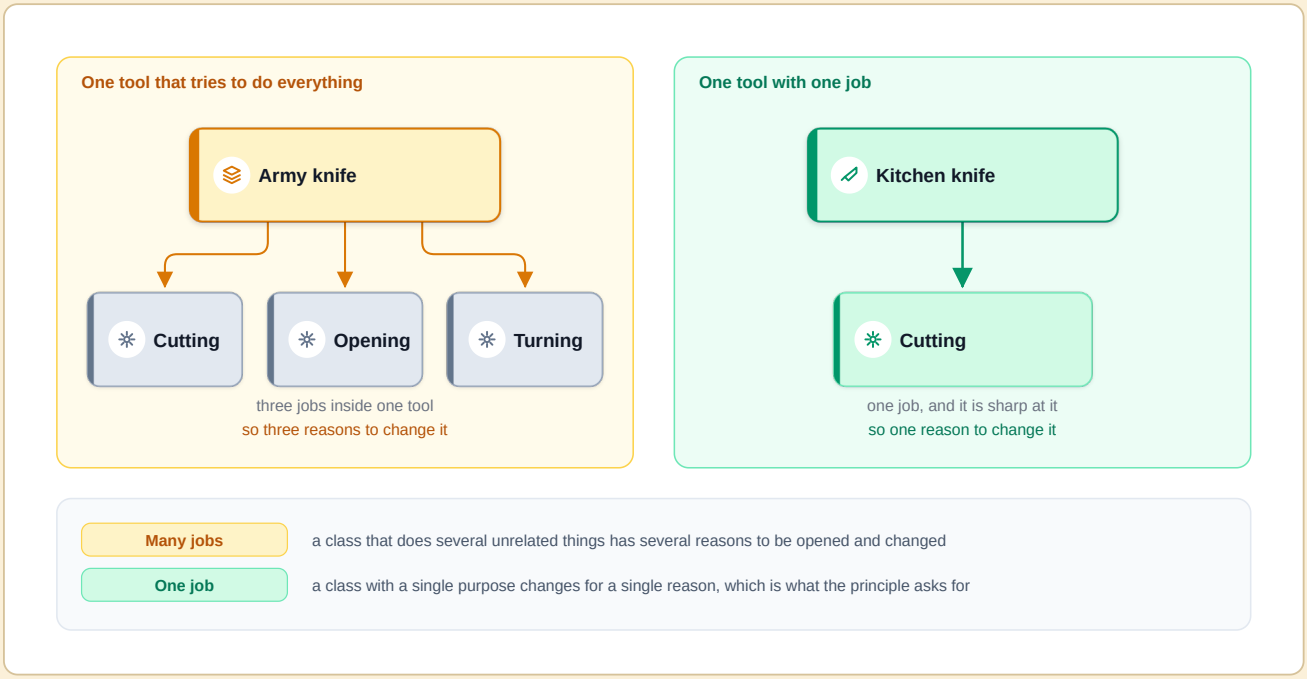


Figure — an army knife with three jobs (three reasons to change) next to a knife with one job (course diagram)

In the figure above, the army knife is the class that breaks SRP: three jobs, three reasons to open it and change it. The kitchen knife follows SRP.

The "one job" idea is often applied at every size of code:

- a **method** should perform one action,
- a **class** should serve one purpose,
- a **module** should cover one area of functionality.

Be precise about the names, though. Martin's SRP is a rule about **classes and modules** and their reasons to change. "A function should do one thing" is a separate (good) rule about functions from *Clean Code*. Both push in the same direction, but only the first is SRP.

The problem it solves

When one class serves several groups, every edit made for one group ships inside code that the other groups rely on, so a payroll tweak can break an operations report. Unrelated logic is also mixed together, which makes it slow to find the right place for a change. With one responsibility per class, you know exactly which class to open.

What counts as one responsibility?

"Do one thing" has no fixed size. Is "register a user" one thing? Is checking the email address part of it, or a job of its own? Reasonable people can debate that without end, because the phrase offers no yardstick.

The **actor** view gives you something to measure. A reason to change always comes from someone. So ask: **who can send me a change request for this code?** If two different groups can, the class has two responsibilities, however small it looks.

Here is an `Employee` class. Every method is "about employees", so the "one thing" test passes. Now ask who can demand a change to each method:

Method	Who asks for the change	Example request
`calculatePay`	Finance	"Overtime starts after 38 hours, not 40."
`reportHours`	Operations	"Show the week-ending date in the report."
`save`	Database team	"Move from SQL (table database) to a document store (stores JSON-like documents)."

So one class has three owners and three independent reasons to change. A new overtime rule from finance means editing `Employee`, which also ships the report code and the storage code again, although neither of those teams wanted anything. If the edit accidentally breaks the report layout, operations pays for a decision it had no part in.

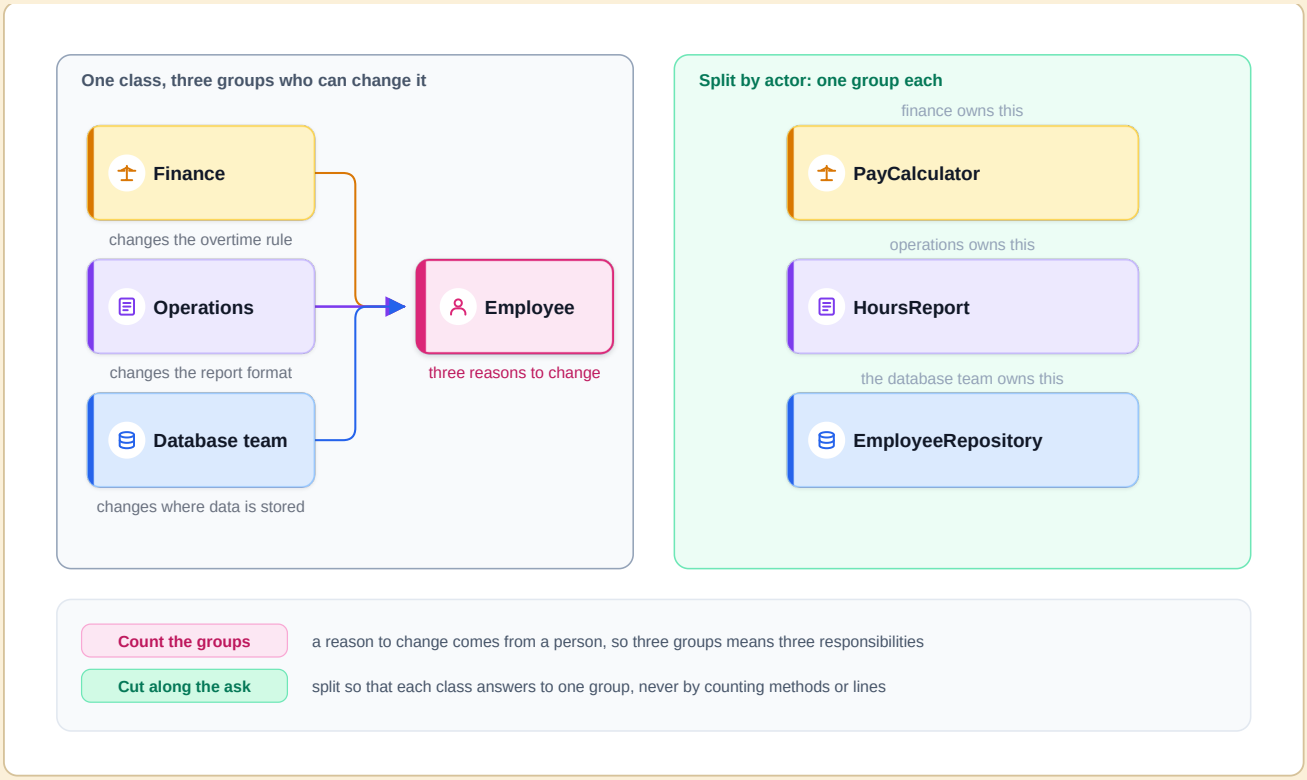


Figure — finance, operations and the database team all reach into one Employee class; splitting by actor gives each group its own class (course diagram)

Martin tells a sharper version of this story in *Clean Architecture*. Suppose `calculatePay` and `reportHours` both call a shared private helper, `regularHours()`, because the two calculations happen to match today. Finance changes the helper for its own reasons. The operations report now shows wrong numbers, and nobody on the operations side was told. The code looked like good reuse, but the two uses belonged to different actors. That is **accidental duplication**: two pieces of code that look the same but change for different reasons should not be merged.

How it works

1. List the methods of the class.
2. For each one, name **who** would ask for it to change.
3. Group the methods by actor.
4. Move each group into its own class, named after its job.
5. Keep plain data in a simple data class (a Java `record` is ideal: a compact class that only holds a fixed set of fields).
6. Check the split with the three checks below.

Java example: splitting `Employee` by actor

The violation. Three actors share one class:

```java
// File: code/lld/ch04/srp/before/Employee.java
package lld.ch04.srp.before;

import java.math.BigDecimal;
import java.math.RoundingMode;

/**
 * VIOLATION: three different groups (actors) can each demand a change here.
 * Finance owns the pay rule, operations owns the report, the database team owns storage.
 */
public class Employee {
    private final String name;
    private final int hoursWorked;
    private final BigDecimal hourlyRate; // money: BigDecimal, never double

    public Employee(String name, int hoursWorked, BigDecimal hourlyRate) {
        this.name = name;
        this.hoursWorked = hoursWorked;
        this.hourlyRate = hourlyRate;
    }

    // Actor 1: Finance -- "overtime should start after 38 hours, not 40"
    public BigDecimal calculatePay() {
        int overtime = Math.max(0, hoursWorked - 40);
        int normal = hoursWorked - overtime;
        BigDecimal normalPay = hourlyRate.multiply(BigDecimal.valueOf(normal));
        BigDecimal overtimePay = hourlyRate.multiply(new BigDecimal("1.5"))
                .multiply(BigDecimal.valueOf(overtime));
        return normalPay.add(overtimePay).setScale(2, RoundingMode.HALF_EVEN);
    }

    // Actor 2: Operations -- "show the week-ending date in the report"
    public String reportHours() {
        return name + " worked " + hoursWorked + " hours";
    }

    // Actor 3: Database team -- "move from SQL to a document store"
    public void save() {
        System.out.println("INSERT INTO employees VALUES ('" + name + "')");
    }
}
```

The refactor. `Employee` keeps only the data. Each actor gets its own class:

```java
// File: code/lld/ch04/srp/after/Employee.java
package lld.ch04.srp.after;

import java.math.BigDecimal;

/** Just the data. No group owns a business rule in here. */
public record Employee(String name, int hoursWorked, BigDecimal hourlyRate) { }
```

```java
// File: code/lld/ch04/srp/after/PayCalculator.java
package lld.ch04.srp.after;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Only Finance asks for changes to this class. */
public class PayCalculator {
    private static final int OVERTIME_THRESHOLD = 40;
    private static final BigDecimal OVERTIME_MULTIPLIER = new BigDecimal("1.5");

    public BigDecimal calculatePay(Employee e) {
        int overtime = Math.max(0, e.hoursWorked() - OVERTIME_THRESHOLD);
        int normal = e.hoursWorked() - overtime;
        BigDecimal normalPay = e.hourlyRate().multiply(BigDecimal.valueOf(normal));
        BigDecimal overtimePay = e.hourlyRate().multiply(OVERTIME_MULTIPLIER)
                .multiply(BigDecimal.valueOf(overtime));
        return normalPay.add(overtimePay).setScale(2, RoundingMode.HALF_EVEN); // cents
    }
}
```

```java
// File: code/lld/ch04/srp/after/HoursReport.java
package lld.ch04.srp.after;

/** Only Operations asks for changes to this class. */
public class HoursReport {
    public String format(Employee e) {
        return e.name() + " worked " + e.hoursWorked() + " hours";
    }
}
```

```java
// File: code/lld/ch04/srp/after/EmployeeRepository.java
package lld.ch04.srp.after;

/** Only the database team asks for changes to this class. */
public class EmployeeRepository {
    public void save(Employee e) {
        System.out.println("INSERT INTO employees VALUES ('" + e.name() + "')");
    }
}
```

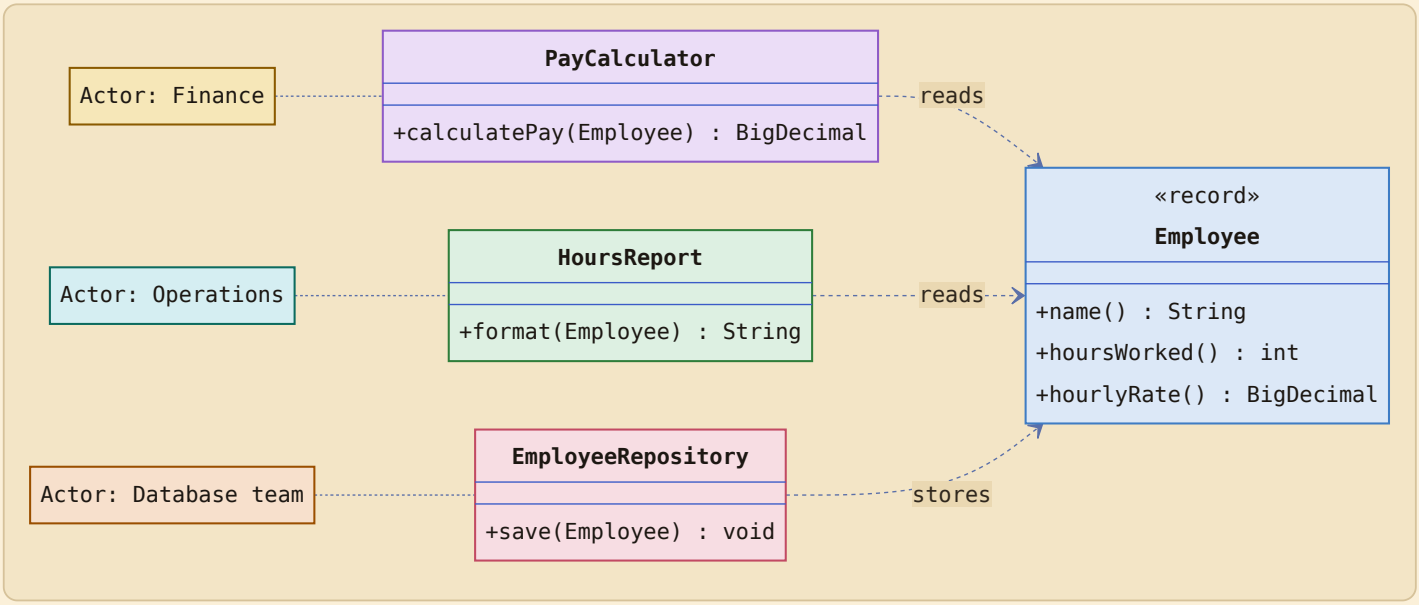
```java
// File: code/lld/ch04/srp/Main.java
package lld.ch04.srp;

import java.math.BigDecimal;
import lld.ch04.srp.after.EmployeeRepository;
import lld.ch04.srp.after.HoursReport;
import lld.ch04.srp.after.PayCalculator;

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Before: one class, three actors ---");
        var before = new lld.ch04.srp.before.Employee("Ada", 45, new BigDecimal("20.00"));
        System.out.println("Pay: " + before.calculatePay());
        System.out.println(before.reportHours());
        before.save();

        System.out.println("--- After: split by actor ---");
        var ada = new lld.ch04.srp.after.Employee("Ada", 45, new BigDecimal("20.00"));
        System.out.println("Pay: " + new PayCalculator().calculatePay(ada));
        System.out.println(new HoursReport().format(ada));
        new EmployeeRepository().save(ada);
    }
}
```

Output:

```
--- Before: one class, three actors ---
Pay: 950.00
Ada worked 45 hours
INSERT INTO employees VALUES ('Ada')
--- After: split by actor ---
Pay: 950.00
Ada worked 45 hours
INSERT INTO employees VALUES ('Ada')
```

What each part does:

- `Employee` is a `record`: an immutable (cannot change after creation) data carrier. No business rule lives there, so no actor needs to edit it for a rule change.
- `PayCalculator` belongs to finance. The overtime threshold and multiplier are named constants, so the next finance request is a one-line change in one class.
- `HoursReport` belongs to operations. `EmployeeRepository` belongs to the database team.
- The output is **identical** before and after. Behaviour did not change. What changed is who must be consulted before an edit. Finance can now change the overtime rule without anyone touching the report or the database code.



Where splitting goes wrong

A badly applied SRP leaves the code in a **worse** state than no SRP at all. The typical mistake: a large class is chopped up by verb, one class per method or per step, without asking who owns what. The cuts fall in arbitrary places, and the result is a handful of tiny classes that must always be edited as a group, so each change now spreads over half a dozen files. Three checks catch this:

1. **Do the pieces change together?** After a few months, read the version-control log (the history kept by a tool such as Git, where each **commit** is one saved change). Two classes that show up in nearly every commit side by side were really one responsibility that got cut in two.
2. **Does each new class have a name that stands on its own?** `EmployeePayCalculator` describes a job. `EmployeeHelper` and `EmployeeUtils` describe nothing. If the name is meaningless without the original class next to it, the class usually is too.
3. **Could the new class be used from somewhere else?** A pay calculator could serve a payroll export, a quote for a contractor, or a unit test (a small automated test of one class on its own). If you cannot imagine any second caller, question why you split it off.

Splitting does not automatically reduce dependencies. After an extraction the two classes still call each other. If the extracted class was designed around the precise needs of the class it came from, every change to one will drag the other along. So the split is only step one; step two is deciding what each class is allowed to know about the other. Those two questions have names: **cohesion** (4.3) and **coupling** (4.4). SRP usually helps both, but promises neither.

Why SRP is important

- **Simpler maintenance.** A class with one job is easy to understand, debug and change.
- **Reuse.** A focused class fits in more places.
- **Scales with the codebase.** New features add classes instead of bloating old ones.
- **High cohesion.** Everything in the class serves one purpose.
- **Opens the door to low coupling, but does not finish the job.** Separating responsibilities gives you a seam (a boundary where one side can be swapped without editing the other). Coupling drops only when the dependency across that seam points at an abstraction (see 4.4).

Trade-offs

You gain	You pay
A change requested by one group touches only that group's code	More classes and files to navigate
Smaller classes are easier to test in isolation	The caller now wires several objects together
Clear ownership (who reviews what)	Risk of over-splitting into classes that always change together

When not to use it

- When you cannot name two different actors. If you cannot name the groups, you do not yet know where the seam is.
- In short-lived code such as a one-off script.
- When the pieces would always change together anyway.

Common mistakes

- **Splitting by counting methods or verbs** instead of by actor.
- **Helper/Utils classes** that exist only because something had to be moved somewhere.
- **Merging "duplicate" code that belongs to different actors** (the `regularHours()` trap).
- **Thinking SRP means "a class has one method".** A class can have ten methods and one responsibility.
- **Confusing SRP with "a function should do one thing".** That is a separate (good) rule about functions from *Clean Code*. SRP is about reasons to change.

Code smells that point to SRP (from *Refactoring*)

A **code smell** is a surface sign that the design is probably wrong somewhere; *Refactoring* is Martin Fowler's book that names the common smells and the step-by-step fixes (called refactorings) for each.

- **Divergent Change** — one class is edited for many different kinds of reasons. This is SRP broken.
- **Large Class** — too many fields and methods, often a sign of several jobs.
- **Shotgun Surgery** — one change forces small edits in many classes. This is often the result of **over-splitting**, the opposite mistake.
- The usual fix is the **Extract Class** refactoring (and **Move Function**/**Move Field**).

Real-world examples

- **Spring layers** (Spring is a popular Java application framework): `@Controller` handles HTTP (the web's request/response protocol, see System Design Fundamentals), `@Service` holds business rules, `@Repository` handles persistence (saving data permanently, usually in a database). Each layer answers to a different kind of change.
- **JDK**: `java.time.format.DateTimeFormatter` formats and parses dates; `LocalDate` holds the date. Formatting rules can change without touching the date value type.
- **Logging frameworks** (Logback, Log4j 2: libraries that write a program's log messages) split the *logger* (what to log), the *appender* (where it goes) and the *layout* (how it looks). A new output format does not touch the destination code.

In the interview

> **30-second answer:** "SRP says a class should have one reason to change. The sharp version is: one actor, one group of people who can request changes. I find the actors by asking who would ask for each method to change. If finance, operations and the DBAs (database administrators) can all ask for edits to `Employee`, I split it into `PayCalculator`, `HoursReport` and `EmployeeRepository`, and keep `Employee` as data. The cost is more classes and some wiring, so I only split when I can name the different groups."

Follow-ups:

- *"Would you always split?"* — No. If the two halves always change together, I have turned one file into two for nothing. I split when two different people can ask for two different changes.
- *"How do you know you split too far?"* — The pieces show up in the same commits, their names only make sense next to each other (`XHelper`), and nothing else could use them.
- *"Is SRP about methods?"* — It applies to methods, classes and modules, but the test is always reasons to change, not the number of methods.

Self-check

1. **Q:** What is an "actor" in SRP? **A:** A person or group who can request a change to the code, for example finance or the database team.
2. **Q:** A `ReportService` formats a report and emails it. Is that an SRP problem? **A:** Yes, if different people own the wording and the delivery. Formatting and delivery change for different reasons.
3. **Q:** Name the three checks for a good split. **A:** Do the pieces change together? Does each name stand alone? Could each piece be used elsewhere?
4. **Q:** Does splitting a class always lower coupling? **A:** No. If the new class names a concrete collaborator, the dependency just moved. Point it at an abstraction to loosen it.

TL;DR

One class, one actor. Find the actors, split along them, keep data in a record, check the split, and do not split what always changes together.

4.3 Cohesion

What it is

Cohesion is how closely the contents of one class (or module, or method) belong together. **High cohesion**: every method works toward the same purpose. **Low cohesion**: unrelated jobs are bundled into one place.

> **Analogy: a garbage dump vs sorted bins.** A dump holds plastic, paper and food waste in one pile. Nothing in it has a clear purpose, and finding anything is hard. Separate bins, one per material, each have one job.

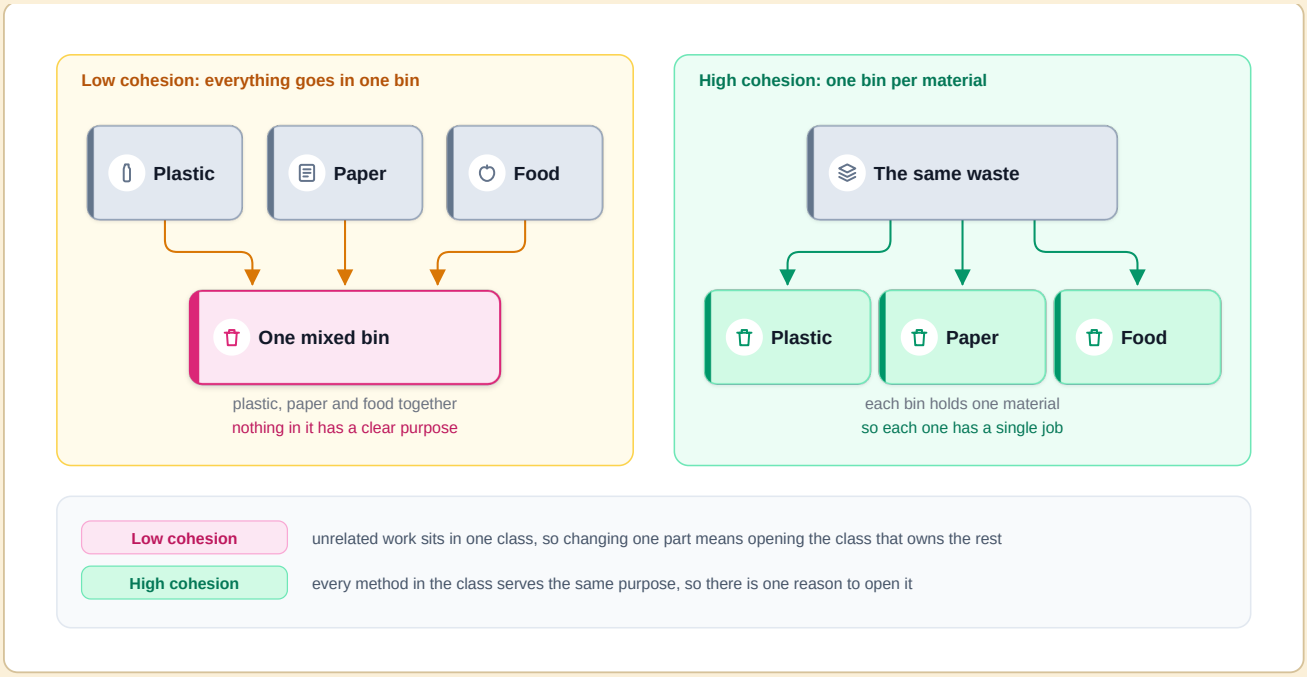


Figure — one mixed bin (low cohesion) next to one bin per material (high cohesion) (course diagram)

In the figure above, the mixed bin is a class whose methods have nothing to do with each other. The sorted bins are classes where every method serves the same purpose, so each one has a single reason to change.

The problem it solves

In a low-cohesion class, someone who wants to change one job has to open, read and redeploy (release to production again) a class that also owns other jobs. Low cohesion also works in the other direction: if the code for **one** job is **scattered** over several classes (say `addUser` in one class and `deleteUser` in another), a change to that job has to hunt for all the pieces. Both are cohesion problems: the first mixes unrelated things, the second separates related things.

The classic types of cohesion

Structured-design authors (Larry Constantine, Ed Yourdon, Glenford Myers, 1970s) ranked cohesion from worst to best. The names are still useful for describing what you see:

Type (worst → best)	What groups the code	Java-flavoured example
Coincidental	Nothing, it just ended up together	`Utils` with `formatDate`, `sendEmail`, `parseCsv`
Logical	Same category, chosen by a flag	`handle(type)` with a big `switch` over unrelated actions
Temporal	Things that run at the same time	`startup()` that opens the DB, warms caches (fills fast temporary copies of data) and prints a banner
Procedural	Steps that follow an order	A method that validates, then logs, then retries
Communicational	Work on the same data	`ReportBuilder` methods that all read the same `Order` list
Sequential	Output of one step is input to the next	`parse → validate → transform` pipeline in one class
Functional (best)	Everything contributes to one well-defined task	`PayCalculator`

You do not need to memorise the list. Aim for **functional** cohesion and recognise **coincidental** cohesion (the "Utils" class) as a warning sign.

How it works

1. Look at each method in the class. Which fields does it use?
2. If the methods fall into groups that use **different** fields for **different** purposes, the class has low cohesion.

3. Move each group into its own class.
4. If the code for one job is scattered over several classes, gather it into one.

Java example

Before: user bookkeeping and audit-file work share a class. `addUser` / `deleteUser` use `users`; `writeToFile` / `readFromFile` use `auditLines`. Two disjoint groups:

```java
// File: code/lld/ch04/cohesion/before/UserManager.java
package lld.ch04.cohesion.before;

import java.util.ArrayList;
import java.util.List;

/** LOW cohesion: user bookkeeping and audit-file work share one class but not one purpose. */
public class UserManager {
    private final List<String> users = new ArrayList<>();
    private final List<String> auditLines = new ArrayList<>();

    // --- user work ---
    public void addUser(String user) {
        users.add(user);
        System.out.println("user added: " + user);
    }

    public void deleteUser(String user) {
        users.remove(user);
        System.out.println("user deleted: " + user);
    }

    // --- file work, unrelated to the two methods above ---
    public void writeToFile(String line) {
        auditLines.add(line);
        System.out.println("wrote a line to the audit file: " + line);
    }

    public List<String> readFromFile() {
        System.out.println("read " + auditLines.size() + " line(s) back from the audit file");
        return List.copyOf(auditLines);
    }
}
```

After: two classes, each with one purpose:

```java
// File: code/lld/ch04/cohesion/after/UserManager.java
package lld.ch04.cohesion.after;

import java.util.ArrayList;
import java.util.List;

/** HIGH cohesion: every method works on the same field for the same purpose. */
public class UserManager {
    private final List<String> users = new ArrayList<>();

    public void addUser(String user) {
        users.add(user);
        System.out.println("user added: " + user);
    }

    public void deleteUser(String user) {
        users.remove(user);
        System.out.println("user deleted: " + user);
    }
}
```

```java
// File: code/lld/ch04/cohesion/after/AuditFile.java
package lld.ch04.cohesion.after;

import java.util.ArrayList;
import java.util.List;

/** HIGH cohesion: only audit-log work lives here. */
public class AuditFile {
    private final List<String> lines = new ArrayList<>();

    public void writeLine(String line) {
        lines.add(line);
        System.out.println("wrote a line to the audit file: " + line);
    }

    public List<String> readAll() {
        System.out.println("read " + lines.size() + " line(s) back from the audit file");
        return List.copyOf(lines);
    }
}
```

```java
// File: code/lld/ch04/cohesion/Main.java
package lld.ch04.cohesion;

import lld.ch04.cohesion.after.AuditFile;
import lld.ch04.cohesion.after.UserManager;

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Before: one class, two unrelated jobs ---");
        var manager = new lld.ch04.cohesion.before.UserManager();
        manager.addUser("ada");
        manager.writeToFile("created ada");
        manager.deleteUser("ada");
        manager.readFromFile();

        System.out.println("--- After: two classes, one job each ---");
        var users = new UserManager();
        var audit = new AuditFile();
        users.addUser("ada");
        audit.writeLine("created ada");
        users.deleteUser("ada");
        audit.readAll();
    }
}
```

Output:

```
--- Before: one class, two unrelated jobs ---
user added: ada
wrote a line to the audit file: created ada
user deleted: ada
read 1 line(s) back from the audit file
--- After: two classes, one job each ---
user added: ada
wrote a line to the audit file: created ada
user deleted: ada
read 1 line(s) back from the audit file
```

Both halves of the output are identical. The difference is structural: each class now has a single purpose, so a new audit-log format is an edit to `AuditFile` alone and never touches user management. The **cost** is that the caller must create and hold two objects instead of one, which becomes worth it as soon as the two jobs change for different reasons.

Cohesion and SRP

- Low cohesion usually means several unrelated jobs in one class, which breaks SRP.
- High cohesion means the class is built around one job, which is what SRP asks for.
- So aiming for high cohesion tends to give you SRP for free.

Trade-offs

High cohesion gains	Costs
Easy to name, read and test each class	More classes to wire
A change lands in one class	Callers may hold several objects

When not to use it

Do not split a class that already has functional cohesion just to make it smaller. Five tiny classes can be harder to follow than one clear one.

Common mistakes

- Calling every mixed class a "coupling problem". A single class mixing jobs is a **cohesion** problem; coupling needs two classes.
- Creating a `Utils` dumping ground (coincidental cohesion).

Real-world examples

- `java.lang.Math` is a set of static math functions: functional cohesion around "numeric calculations".
- `java.util.Collections` is looser: sorting, searching, wrapping and synchronising collections all live together. It is still "operations on collections", but it shows how a utility class drifts toward logical cohesion.

In the interview

> **30-second answer:** "Cohesion is how much the things inside one class belong together. I check which fields each method uses. If the methods split into two groups that touch different fields for different reasons, the class has low cohesion and I split it. High cohesion is basically SRP seen from inside the class."

Follow-up: *"Is a big class always low-cohesion?"* — No. If every method serves one task, it is cohesive even if it is large. Size is a hint, not the measure.

Self-check

1. **Q:** What is the worst kind of cohesion? **A:** Coincidental: code grouped for no reason, like a `Utils` class.
2. **Q:** Is "the code for one job is spread over three classes" a cohesion problem? **A:** Yes. Related code should live together.

TL;DR

Cohesion = do the parts of this class belong together? Aim for one purpose per class; split disjoint groups; gather scattered pieces.

4.4 Coupling

What it is

Coupling is how much one class depends on another. It always describes a pair: a lone class cannot be "tightly coupled", because there is nothing else in the picture for it to depend on.

- **Low (loose) coupling**: you can change, replace or test one class without changing the other.
- **High (tight) coupling**: a change to one forces a change to the other.

> **Analogy: TV and remote.** A remote that controls only one specific TV, and a TV that accepts only that remote, are tightly coupled: replace one and you must replace both. A universal remote and a TV that accepts any remote are loosely coupled: each follows a shared standard (the infrared signal), and each can be swapped on its own.

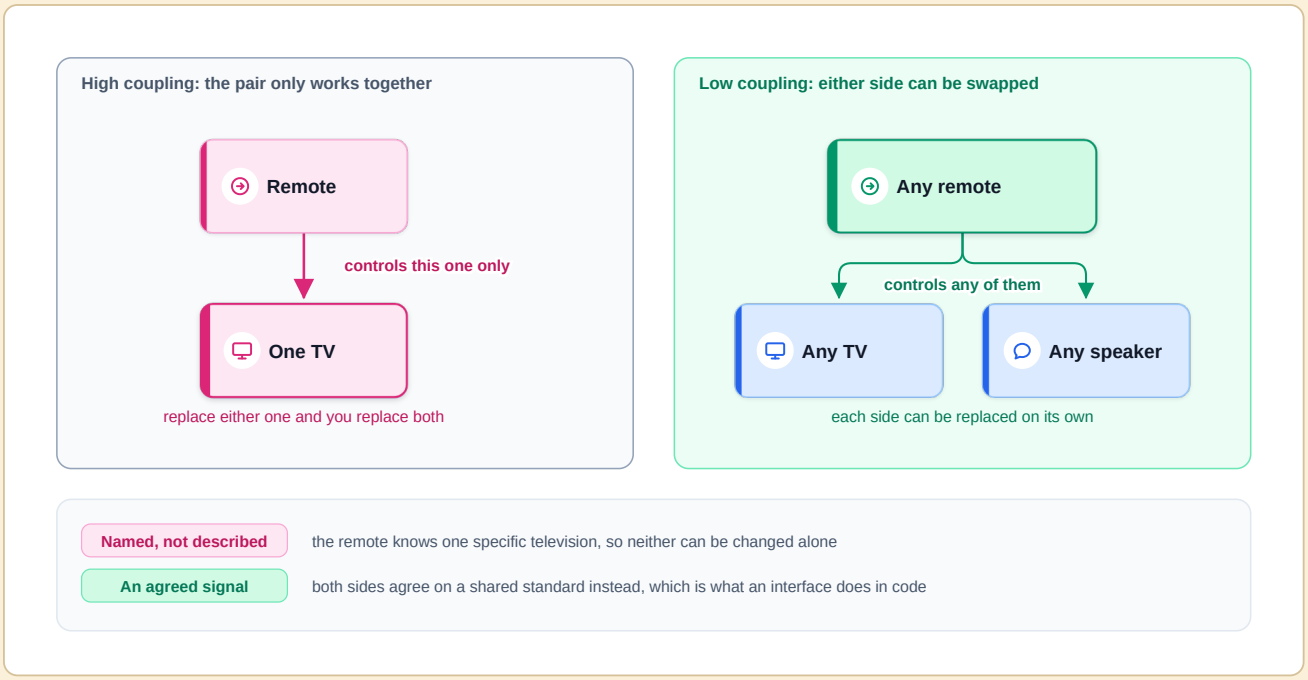


Figure — a remote tied to one TV (high coupling) vs any remote working with any device through an agreed signal (low coupling) (course diagram)

In the figure above, the "agreed signal" on the right is exactly what an **interface** does in code: both sides agree on a contract instead of knowing each other.

The problem it solves

With tight coupling:

- you cannot swap an implementation (MySQL → Postgres, two popular SQL databases) without editing the caller,
- you cannot unit-test the caller without the real dependency (a running database),
- a change in how the dependency is **constructed** ripples into every class that builds it.

The classic types of coupling

From structured design again, from worst to best:

Type (worst → best)	What A knows about B	Java-flavoured example
Content	A reaches into B's internals	Reflection (Java's ability to inspect and change classes while running) that sets B's private field
Common	A and B share global mutable data	Both read/write a `public static Map`
External	Both depend on an external format or device	Both parse the same hand-rolled file format
Control	A passes a flag telling B *how* to work	`print(doc, boolean asPdf)`
Stamp	A passes a whole structure when B needs one field	`sendInvoice(Customer c)` that only reads `c.email()`
Data	A passes only the values B needs	`sendInvoice(String email, Money total)`
Message (lowest)	A sends a message through an interface	`notifier.send(message)` on an interface

In object-oriented design, the practical goal is: **depend on interfaces, pass only what is needed, never share mutable globals.**

How it works

1. Find where class A **names** class B (a field type, a `new B()`, a static call `B.x()`).
2. Ask: does A need *this particular* B, or just *something that does the job*?
3. If the latter, describe the job as an interface.
4. Let A accept any object that implements the interface, usually through its constructor.
5. Let the caller (or the program's entry point, the `main` method where it starts) choose which implementation to pass.

Java example: three steps with `Student`

Start: one class, two jobs. `Student` holds data *and* talks to MySQL. With a single class in play this is a **cohesion** problem; coupling needs a second class:

```java
// File: code/lld/ch04/coupling/start/Student.java
package lld.ch04.coupling.start;

/** Starting point: ONE class, two jobs. A cohesion problem -- nothing to be coupled to yet. */
public class Student {
    private final String studentId;

    public Student(String studentId) {
        this.studentId = studentId;
    }

    public String studentId() {
        return studentId;
    }

    // Student holds the data AND knows how to store it
    public void save() {
        System.out.println("Connecting to MySQL...");
        System.out.println("Saving student with ID: " + studentId);
    }
}
```

Step 1: extract the database work. Cohesion is fixed. But now there are two classes, and the coupling is **tight**: `Student` names `MySqlDatabase` directly.

```java
// File: code/lld/ch04/coupling/extracted/MySqlDatabase.java
package lld.ch04.coupling.extracted;

public class MySqlDatabase {
    public void saveStudent(String studentId) {
        System.out.println("Connecting to MySQL...");
        System.out.println("Saving student with ID: " + studentId);
    }
}
```

```java
// File: code/lld/ch04/coupling/extracted/Student.java
package lld.ch04.coupling.extracted;

/** Step 1: cohesion fixed, but Student now names one concrete class -> TIGHT coupling. */
public class Student {
    private final String studentId;
    private final MySqlDatabase database = new MySqlDatabase(); // hard-wired

    public Student(String studentId) {
        this.studentId = studentId;
    }

    public void save() {
        database.saveStudent(studentId);
    }
}
```

Three costs follow: storing students in any other place requires editing `Student`; every test of `Student` needs a live MySQL server; and if `MySqlDatabase` ever needs constructor arguments, `Student` must change too. The extraction relocated the database code, but `Student` still **depends** on it just as much as before.

Step 2: depend on an interface. Describe what `Student` needs as `StudentRepository`, and let the caller choose the implementation:

```java
// File: code/lld/ch04/coupling/decoupled/StudentRepository.java
package lld.ch04.coupling.decoupled;

/** What Student needs, described as a contract rather than a working class. */
public interface StudentRepository {
    void save(String studentId);
}
```

```java
// File: code/lld/ch04/coupling/decoupled/MySqlStudentRepository.java
package lld.ch04.coupling.decoupled;

public class MySqlStudentRepository implements StudentRepository {
    @Override
    public void save(String studentId) {
        System.out.println("Saving student " + studentId + " to MySQL");
    }
}
```

```java
// File: code/lld/ch04/coupling/decoupled/InMemoryStudentRepository.java
package lld.ch04.coupling.decoupled;

import java.util.ArrayList;
import java.util.List;

/** A stand-in that a unit test can use -- no database server needed. */
public class InMemoryStudentRepository implements StudentRepository {
    private final List<String> saved = new ArrayList<>();

    @Override
    public void save(String studentId) {
        saved.add(studentId);
        System.out.println("Keeping student " + studentId + " in memory for a test");
    }

    public List<String> saved() {
        return List.copyOf(saved);
    }
}
```

```java
// File: code/lld/ch04/coupling/decoupled/Student.java
package lld.ch04.coupling.decoupled;

import java.util.Objects;

/** Step 2: Student names no database at all -> LOOSE coupling. */
public class Student {
    private final String studentId;
    private final StudentRepository repository;

    public Student(String studentId, StudentRepository repository) {
        this.studentId = studentId;
        this.repository = Objects.requireNonNull(repository);
    }

    public void save() {
        repository.save(studentId);
    }
}
```

```java
// File: code/lld/ch04/coupling/Main.java
package lld.ch04.coupling;

import lld.ch04.coupling.decoupled.InMemoryStudentRepository;
import lld.ch04.coupling.decoupled.MySqlStudentRepository;
import lld.ch04.coupling.decoupled.Student;

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Start: one class holds data and SQL ---");
        new lld.ch04.coupling.start.Student("S-1024").save();

        System.out.println("--- Step 1: extracted, but tightly coupled to MySqlDatabase ---");
        new lld.ch04.coupling.extracted.Student("S-1024").save();

        System.out.println("--- Step 2: depends on the StudentRepository interface ---");
        new Student("S-1024", new MySqlStudentRepository()).save();
        var fake = new InMemoryStudentRepository();
        new Student("S-2048", fake).save();
        System.out.println("test double recorded: " + fake.saved());
    }
}
```

Output:

```
--- Start: one class holds data and SQL ---
Connecting to MySQL...
Saving student with ID: S-1024
--- Step 1: extracted, but tightly coupled to MySqlDatabase ---
Connecting to MySQL...
Saving student with ID: S-1024
--- Step 2: depends on the StudentRepository interface ---
Saving student S-1024 to MySQL
Keeping student S-2048 in memory for a test
test double recorded: [S-2048]
```

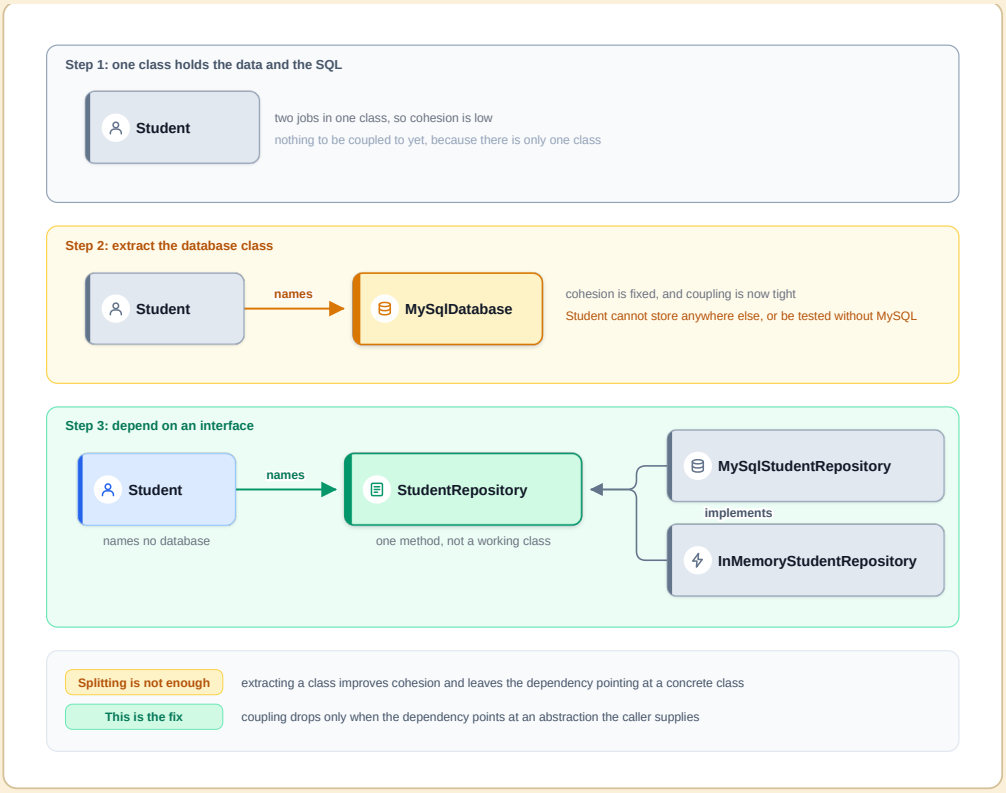


Figure — the three steps: one class with two jobs, an extracted but tightly coupled class, and a class that depends on an interface (course diagram)

In the figure above, follow the arrows. In the middle panel `Student` *names* `MySqlDatabase`. In the last panel it names only `StudentRepository`, a one-method contract, and the concrete repositories point *toward* that contract. Moving to Postgres is now a new repository class handed in by the caller, with no edit to `Student`, and a unit test simply hands in the in-memory version.

This final move is the rule **depend on abstractions, not on concrete classes**, the D in SOLID (4.10). DIP is, in effect, the standard cure for tight coupling.

Coupling and SRP

- **Separating the jobs** (SRP) repaired cohesion, yet by itself it left `Student` bound tightly to `MySqlDatabase`.
- **Aiming the dependency at an interface** is what repaired coupling, and SRP alone never requires that step.
- Hence SRP *usually* reduces coupling (a single-job class needs fewer collaborators) without *guaranteeing* it. After each split, inspect what the two resulting classes depend on; a class carved out to serve its parent's exact needs can remain welded to it.

Trade-offs

Loose coupling gains	Costs
Swap implementations without editing callers	An extra interface and a constructor parameter
Unit tests with in-memory fakes (simple stand-ins that keep data in a list instead of a database)	Someone must do the wiring (the entry point or a framework)
Changes stop rippling	"Go to definition" lands on an interface, one more hop to the real code

When not to use it

- Do not put an interface in front of a **stable** class that will never vary and never needs faking (`String`, `LocalDate`, a pure calculation). Coupling to stable things is harmless.
- An interface with one implementation and no test double (a fake stand-in object used in tests) is usually just an extra file.

Common mistakes

- Thinking "I extracted a class, so coupling is lower". Only depending on an abstraction lowers it.

- Hiding `new ConcreteThing()` inside a constructor. The dependency is still hard-wired.
- Passing whole objects when only one field is needed (stamp coupling).

Real-world examples

- **JDBC** (Java Database Connectivity, Java's standard way to talk to SQL databases): your code talks to `java.sql.Connection`, `Statement` and `ResultSet`, which are interfaces. The MySQL or Postgres driver (the library that speaks one database's protocol) supplies the implementation.
- **SLF4J** (a common Java logging facade, i.e. one front for several logging libraries): code logs through the `org.slf4j.Logger` interface; Logback or Log4j is plugged in at deployment (when the program is installed and started).
- **USB, HTTP, SQL** themselves are "agreed signals", the real-world version of an interface.

In the interview

> **30-second answer:** "Coupling is how much one class depends on another. I check where a class names a concrete collaborator. If it only needs something that does the job, I describe the job as an interface and pass the implementation in. That lets me swap MySQL for Postgres and test with an in-memory fake. I do not add interfaces in front of stable things like `String`."

Follow-up: *"Can SRP increase coupling?"* — It can. Extract a class built around the old class's exact needs and the two are tightly bound. Check the dependencies after every split.

Self-check

1. **Q:** Can a single class be "tightly coupled"? **A:** Not to anything inside itself. Coupling is between two things. A single class with two jobs has a cohesion problem.
2. **Q:** `Student` does `new MySqlDatabase()` in its constructor. Tight or loose? **A:** Tight. It names a concrete class and creates it.
3. **Q:** What is "control coupling"? **A:** Passing a flag that tells the other code how to behave, e.g. `print(doc, asPdf)`.

TL;DR

Coupling = how much A must know about B. Loosen it by depending on an interface and passing the implementation in. Extracting a class alone is not enough.

4.5 Separation of Concerns (SoC)

What it is

Separation of Concerns means splitting a program into parts where each part handles one **concern**. A concern is one area of the work: storing data is one concern, deciding whether an email address is acceptable is another, drawing the screen is a third. The term comes from Edsger Dijkstra (1974).

> **Analogy:** in a restaurant, the kitchen cooks, the waiters serve, and the cashier takes payment. A new menu does not change how payment works.

The problem it solves

When concerns are mixed, one change forces you to read and edit code about other concerns. When they are separate, each change lands in one module.

Benefits:

- **Modularity** — each part covers one area, so the system is easier to follow.
- **Maintainability** — a change in one part rarely affects the others.
- **Reusability** — a well-defined module can be reused elsewhere.
- **Testability** — test one concern with the others replaced by simple stand-ins.

How it works

1. List the concerns in the feature (for registration: storage, business rules).
2. Give each concern its own module.
3. Let modules talk only through small, clear methods.
4. Test the result: for each likely requirement change, **count the modules you would have to edit**. The ideal answer is one.

Java example: user registration

Two concerns. **Storage** keeps users and never judges whether a user is valid. **Business rules** decide what is acceptable and ask storage to save it, without knowing how storage works.

```java
// File: code/lld/ch04/soc/UserStore.java
package lld.ch04.soc;

import java.util.ArrayList;
import java.util.List;

/** Storage concern only. It has no opinion about what a valid user is. */
public class UserStore {
    public record User(String username, String email) { }

    private final List<User> users = new ArrayList<>();

    public void addUser(String username, String email) {
        users.add(new User(username, email));
        System.out.println("Stored user: " + username);
    }

    public int count() {
        return users.size();
    }
}
```

```java
// File: code/lld/ch04/soc/UserService.java
package lld.ch04.soc;

/** Business-rules concern only. Storage is somebody else's job. */
public class UserService {
    private final UserStore store;

    public UserService(UserStore store) {
        this.store = store;
    }

    boolean isValidUsername(String username) {
        return username.length() >= 3;
    }

    boolean isValidEmail(String email) {
        return email.contains("@") && email.contains(".");
    }

    public boolean registerUser(String username, String email) {
        if (!isValidUsername(username)) {
            System.out.println("Rejected: " + username + " is too short");
            return false;
        }
        if (!isValidEmail(email)) {
            System.out.println("Rejected: " + email + " is not an email address");
            return false;
        }
        store.addUser(username, email);
        return true;
    }
}
```

```java
// File: code/lld/ch04/soc/Main.java
package lld.ch04.soc;

public class Main {
    public static void main(String[] args) {
        UserStore store = new UserStore();
        UserService service = new UserService(store);
        service.registerUser("ada", "ada@example.com");
        service.registerUser("jo", "jo@example.com");
        service.registerUser("grace", "grace-at-example");
        System.out.println("Users in store: " + store.count());
    }
}
```

Output:

```
Stored user: ada
Rejected: jo is too short
Rejected: grace-at-example is not an email address
Users in store: 1
```

Of the three attempts, only `ada` made it into storage: `jo` failed the length rule and `grace-at-example` failed the email rule. Every accept/reject decision was made by `UserService`, which never touched the list itself; `UserStore` simply kept whatever it was handed, without judging it.

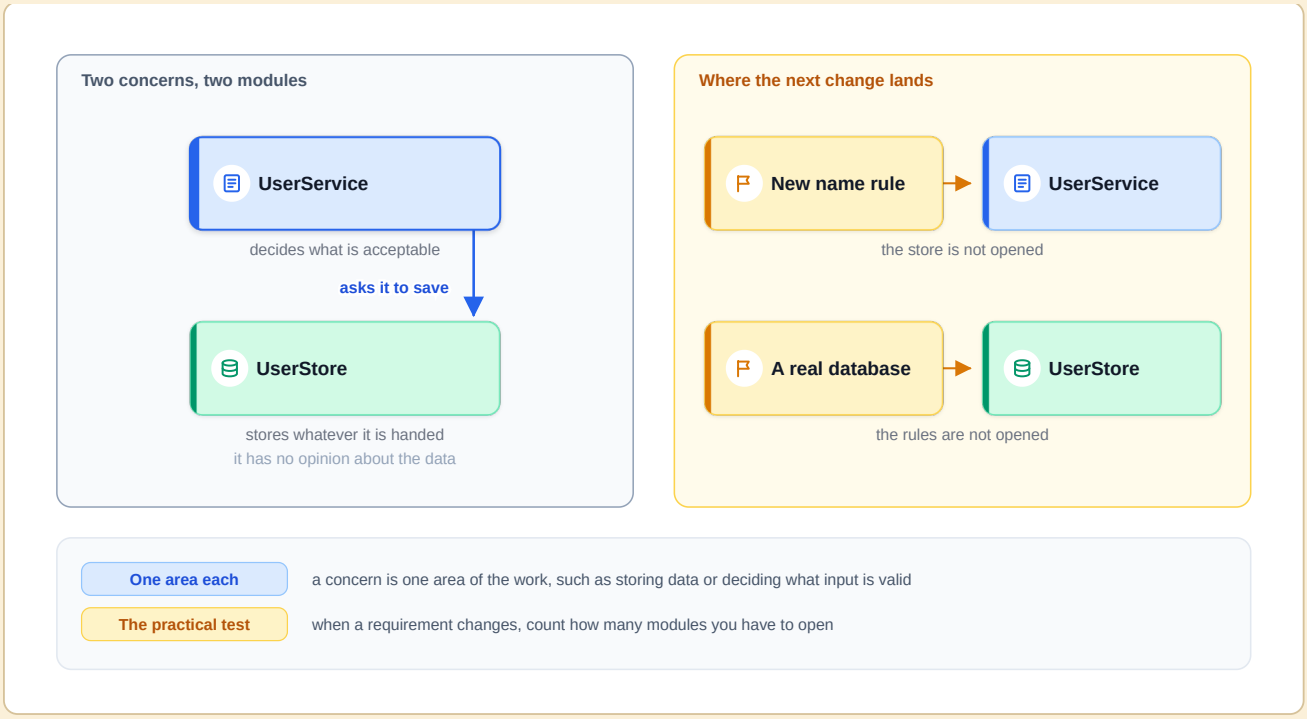


Figure — UserService decides what is acceptable and asks UserStore to save it; each change lands in one module (course diagram)

As the figure above shows, each likely change lands in exactly one place:

Change	Module you open
Move from an in-memory list to a real database	`UserStore` only
Require usernames of at least eight characters	`UserService` only
Test the rules without a database	Pass `UserService` a stand-in store

(In a bigger system you would also put `UserStore` behind an interface, which is the coupling fix from 4.4.)

SoC and SRP

The two are the same instinct applied at different scales. **SRP** is normally discussed for a single class; **SoC** for modules, layers and whole subsystems. Giving every class in a module a single responsibility is one route to well-separated concerns.

Trade-offs

Gains	Costs
Changes stay local	More modules and boundaries to design
Each concern tested alone	Cross-cutting concerns (logging, security, transactions, i.e. groups of database changes that succeed or fail together) still touch many modules; they need special tools such as aspects (code woven automatically around many methods) or decorators (wrappers that add behaviour)

When not to use it

A 50-line script does not need layers. Separate concerns when they change for different reasons or at different speeds.

Common mistakes

- Validation that quietly happens inside the storage class ("the DAO also checks the email"; a DAO, Data Access Object, is a class that reads and writes one kind of data in the database).
- Layers that exist on paper but call each other in every direction.

Real-world examples

- **MVC** (Model–View–Controller): model = data and rules, view = display, controller = input handling. Spring MVC and most web frameworks (libraries that handle the plumbing of web applications) follow it.
- **HTML / CSS / JavaScript**: structure, style and behaviour are three concerns in three languages.
- **Layered architecture**: controller → service → repository.
- **Spring** `@Transactional`: transaction handling (a cross-cutting concern) is separated from business code by a proxy (a stand-in object that wraps the real one and adds behaviour before and after each call).

In the interview

> **30-second answer:** "Separation of concerns means each module handles one area of the work, like storage vs business rules. My test is: when a requirement changes, how many modules do I open? Ideally one. It is SRP at module size."

Self-check

1. **Q:** What is a "concern"? **A:** One area of the work, such as storage, validation or presentation.
2. **Q:** How do SoC and SRP differ? **A:** Mostly in size: SRP is about a class, SoC about modules and layers.

TL;DR

One module per area of work; measure it by how many modules a change forces you to open.

4.6 SRP vs cohesion vs coupling vs SoC — one table

Idea	Question it asks	Scope	How it relates to SRP
SRP	How many actors can ask this class to change?	Class / module	—
Cohesion	Do the parts of this class belong together?	Inside one unit	Applying SRP usually raises cohesion, because one job means related parts.
Coupling	How much does A depend on B?	Between two units	SRP tends to lower coupling but does not guarantee it; you still need to depend on abstractions.
SoC	Is each area of work in its own module?	Modules, layers	SRP applied to every class is one way to achieve SoC.

The rule of thumb every textbook repeats, **"high cohesion, low coupling"**, is the goal. SRP is the tool that gets you the first half; depending on abstractions (DIP) gets you the second.

4.7 Open/Closed Principle (OCP)

What it is

Software entities (classes, modules, functions) should be open for extension but closed for modification. In plain words: you should be able to add new behaviour **by adding new code**, without editing code that already works.

- **Closed for modification**: once a class is written and tested, adding the next variation should not require opening it. Every edit to working code risks breaking it.
- **Open for extension**: new behaviour arrives as new parts, usually through **interfaces**, **polymorphism** (one call, many implementations) or **inheritance**.

Where it comes from. Bertrand Meyer stated the principle in 1988 (*Object-Oriented Software Construction*). In **Meyer's OCP** a class is *closed* once it is compiled, published and used by clients (it is changed only to fix bugs), and it stays *open* because a new class can **inherit** from it and add or redefine features. That is extension by implementation inheritance. Robert C. Martin's 1996 article restated it as the **polymorphic OCP**: the stable code depends on an **abstract interface**, and new behaviour arrives as new implementations of that interface, which

can be swapped for one another. The polymorphic version is the one people usually mean today, and it is the one used in this chapter.

> **Analogy: a game console and its accessories.** You buy a console with a standard controller. Later you add a racing wheel, a motion controller and a VR headset. Nobody opens the console and rewires it. The console is **closed** (its internals stay sealed) and **open** (it offers a standard port that new accessories plug into).

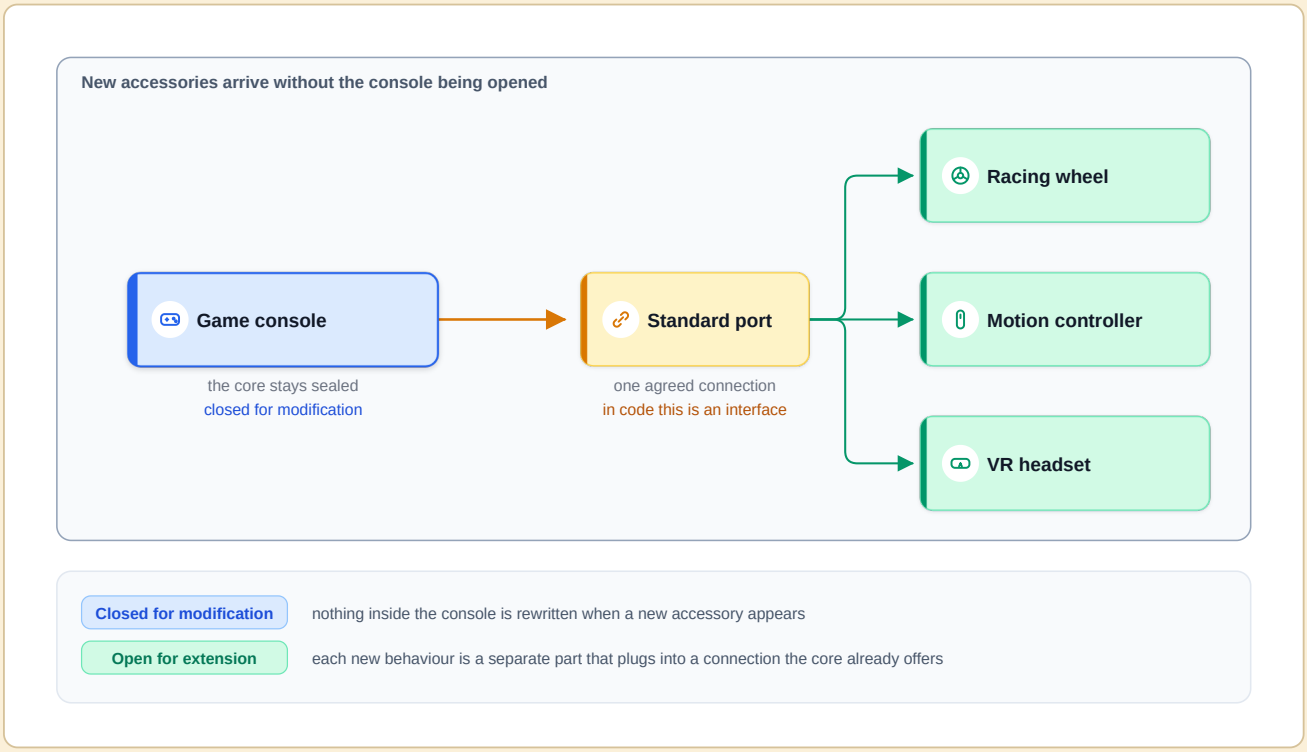


Figure — new accessories plug into a standard port; the console itself is never opened (course diagram)

In the figure above, the **standard port** is the interface. The accessories are new classes that implement it.

The problem it solves

The textbook symptom is an `if/else` chain or `switch` on a type code that keeps growing. Each new case is an edit to a tested, working method, and each edit can break the cases that were fine. Often the same `switch` also appears in several places, and one copy gets missed.

Quick test: *can I add the next case without opening this file?* If not, the class is not closed against that kind of change.

How it works

1. Wait until you have seen the variation happen (at least a second case).
2. Identify **the part that varies** (here: the invoice format).
3. Describe it as an interface.
4. Move each variant into its own class that implements the interface.
5. Make the stable class **delegate** (hand the work over) to the interface instead of branching.
6. Add the next variant as a new class. Nothing existing is opened.
7. Choose the variant in one place, at the edge of the program (the entry point, a factory, i.e. a method or class whose job is creating objects, a map, or configuration).

Java example: invoice formats

Before. While `Invoice` had a single format, there was nothing to fix: OCP only matters for code that actually receives new variations. The trouble starts when sales wants **international invoices** and the quickest route is a type parameter and a branch:

```java
// File: code/lld/ch04/ocp/before/Invoice.java
package lld.ch04.ocp.before;

import java.math.BigDecimal;

/** VIOLATION: every new invoice type means opening this working, tested method. */
public class Invoice {
    private final BigDecimal amount;

    public Invoice(BigDecimal amount) {
        this.amount = amount;
    }

    public void generate(String invoiceType) {
        if (invoiceType.equals("basic")) {
            System.out.println("Generating basic invoice for amount: " + amount);
        } else if (invoiceType.equals("international")) {
            System.out.println("Generating international invoice for amount: " + amount);
        } else {
            throw new IllegalArgumentException("Unknown invoice type: " + invoiceType);
        }
    }

    public void applyDiscount() {
        System.out.println("Applying discount on invoice: " + amount);
    }
}
```

Note exactly what is wrong. Doing two jobs would be an SRP complaint; the OCP complaint is that **each new invoice type forces an edit to code that already works**.

After. Step 1: turn the part that varies into an interface. Step 2: give each format its own class. These classes *produce* invoices, so they are named *generators*; a name like `BasicInvoice` would falsely imply a subtype of invoice. Step 3: `Invoice` hands the work to its generator. Step 4: a third format costs one new class plus one line at the place where formats are chosen.

```java
// File: code/lld/ch04/ocp/after/InvoiceGenerator.java
package lld.ch04.ocp.after;

import java.math.BigDecimal;

/** Step 1: the part that varies (the invoice format) gets its own abstraction. */
@FunctionalInterface
public interface InvoiceGenerator {
    void generate(BigDecimal amount);
}
```

```java
// File: code/lld/ch04/ocp/after/BasicInvoiceGenerator.java
package lld.ch04.ocp.after;

import java.math.BigDecimal;

public class BasicInvoiceGenerator implements InvoiceGenerator {
    @Override
    public void generate(BigDecimal amount) {
        System.out.println("Generating basic invoice for amount: " + amount);
    }
}
```

```java
// File: code/lld/ch04/ocp/after/InternationalInvoiceGenerator.java
package lld.ch04.ocp.after;

import java.math.BigDecimal;

public class InternationalInvoiceGenerator implements InvoiceGenerator {
    @Override
    public void generate(BigDecimal amount) {
        System.out.println("Generating international invoice for amount: " + amount);
        System.out.println("  including customs declaration and currency conversion");
    }
}
```

```java
// File: code/lld/ch04/ocp/after/DetailedInvoiceGenerator.java
package lld.ch04.ocp.after;

import java.math.BigDecimal;

/** Step 4: the ONLY new code for a third format. No existing class was opened. */
public class DetailedInvoiceGenerator implements InvoiceGenerator {
    @Override
    public void generate(BigDecimal amount) {
        System.out.println("Generating detailed invoice for amount: " + amount);
        System.out.println("  line items, tax breakdown and payment terms");
    }
}
```

```java
// File: code/lld/ch04/ocp/after/Invoice.java
package lld.ch04.ocp.after;

import java.math.BigDecimal;
import java.util.Objects;

/** Step 3: Invoice delegates. It no longer knows how many formats exist. */
public class Invoice {
    private final BigDecimal amount;
    private final InvoiceGenerator generator;

    public Invoice(BigDecimal amount, InvoiceGenerator generator) {
        this.amount = Objects.requireNonNull(amount);
        this.generator = Objects.requireNonNull(generator);
    }

    public void generate() {
        generator.generate(amount); // no branching
    }

    // Still closed? No -- a new discount rule would still edit this class.
    // That is fine until discounts actually start to vary.
    public void applyDiscount() {
        System.out.println("Applying discount on invoice: " + amount);
    }
}
```

```java
// File: code/lld/ch04/ocp/Main.java
package lld.ch04.ocp;

import java.math.BigDecimal;
import java.util.List;
import lld.ch04.ocp.after.BasicInvoiceGenerator;
import lld.ch04.ocp.after.DetailedInvoiceGenerator;
import lld.ch04.ocp.after.InternationalInvoiceGenerator;
import lld.ch04.ocp.after.Invoice;
import lld.ch04.ocp.after.InvoiceGenerator;

public class Main {
    public static void main(String[] args) {
        BigDecimal amount = new BigDecimal("120.00"); // money: BigDecimal with 2 decimals

        System.out.println("--- Before: branch per type ---");
        var old = new lld.ch04.ocp.before.Invoice(amount);
        old.generate("basic");
        old.generate("international");

        System.out.println("--- After: one class per format ---");
        List<InvoiceGenerator> generators = List.of(
                new BasicInvoiceGenerator(),
                new InternationalInvoiceGenerator(),
                new DetailedInvoiceGenerator(),
                // A lambda is also an extension: no new file, no edited file.
                a -> System.out.println("Generating pro-forma invoice for amount: " + a));
        for (InvoiceGenerator g : generators) {
            new Invoice(amount, g).generate();
        }
    }
}
```

Output:

```
--- Before: branch per type ---
Generating basic invoice for amount: 120.00
Generating international invoice for amount: 120.00
--- After: one class per format ---
Generating basic invoice for amount: 120.00
Generating international invoice for amount: 120.00
  including customs declaration and currency conversion
Generating detailed invoice for amount: 120.00
  line items, tax breakdown and payment terms
Generating pro-forma invoice for amount: 120.00
```

What to notice:

- `Invoice` has **no branch**. It does not know how many formats exist.
- `DetailedInvoiceGenerator` is the only new code for format three. `Invoice`, `BasicInvoiceGenerator` and `InternationalInvoiceGenerator` were not opened.
- `InvoiceGenerator` is a `@FunctionalInterface` (an interface with exactly one abstract method), so in Java a **lambda** (a short unnamed function written inline, such as `a -> ...`) is also an extension (the pro-forma line in `Main`). That is OCP with zero new files.

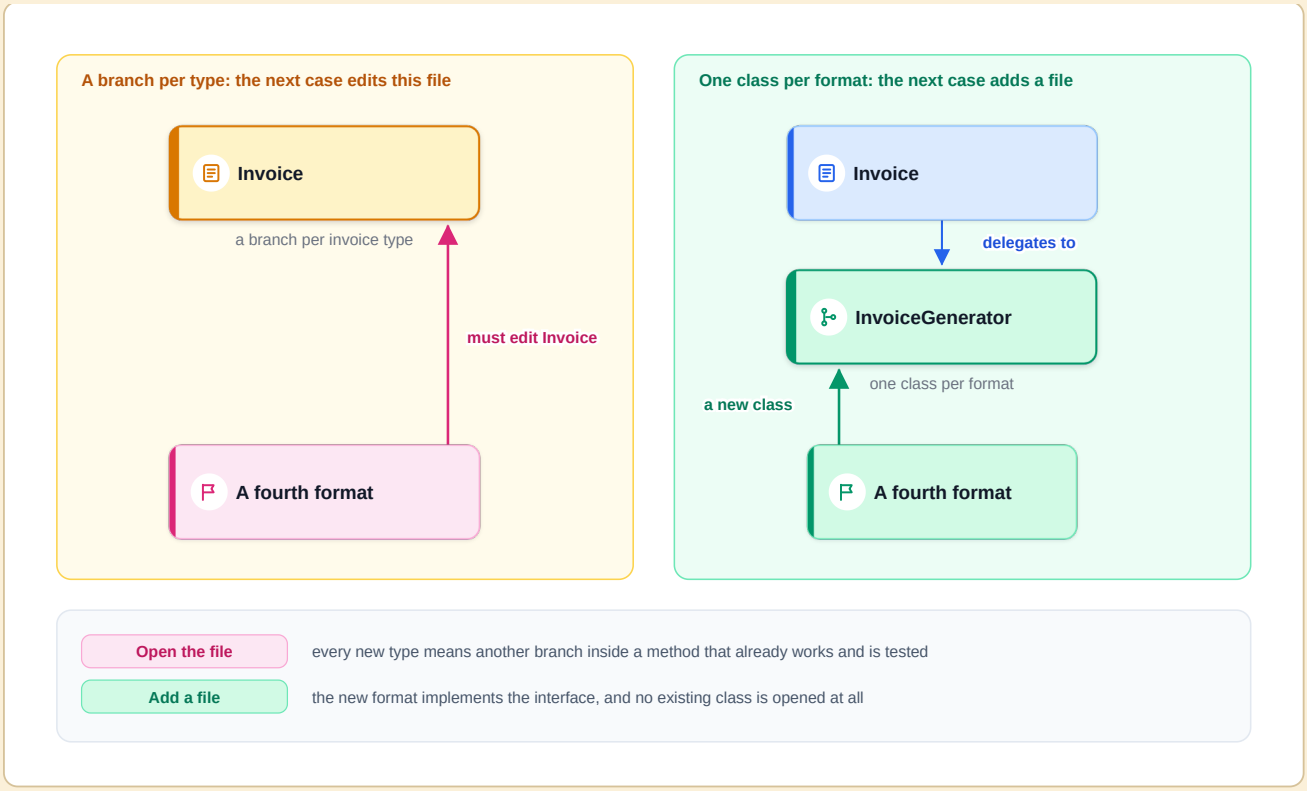
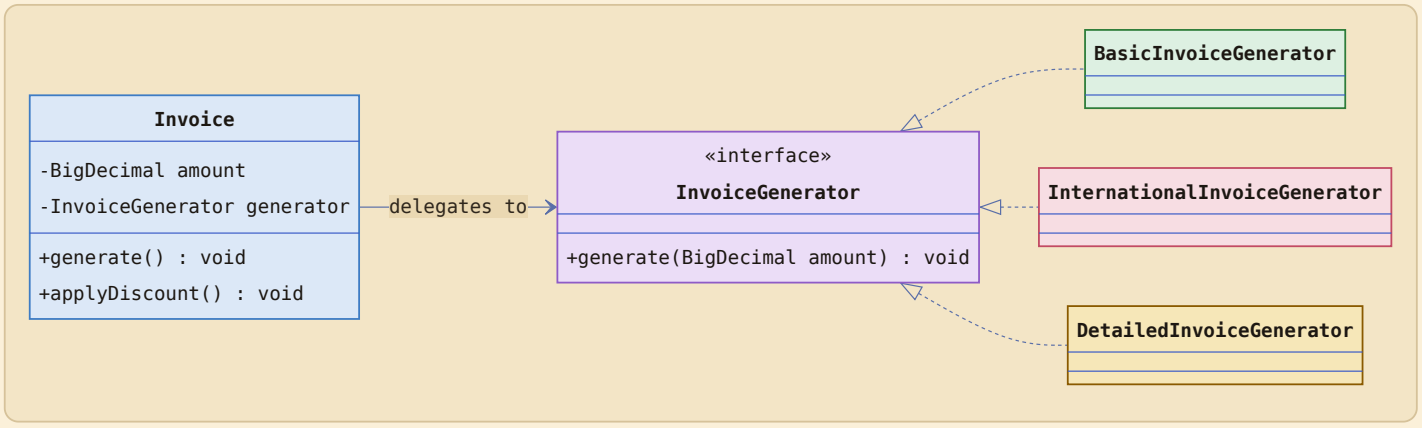


Figure — a branch per type forces an edit to Invoice; one class per format lets a new format be added as a file (course diagram)



This shape (a context object holding an interface-typed field, with interchangeable implementations) is the **Strategy pattern** (a design pattern, i.e. a named reusable design; Chapters 8–11 cover them all). Strategy is OCP applied to an algorithm that varies.

Why not just subclass `Invoice`?

Most readers ask this. Writing `class InternationalInvoice extends Invoice` and overriding `generate()` is also OCP-compliant (it is Meyer's original style): a class is added, nothing is edited. In a small program that is perfectly acceptable. Delegation wins as the system grows, for two reasons:

- **A second dimension of change.** If invoices differ by format **and** by currency, subclassing needs a class for every pairing (3 formats × 4 currencies = 12 classes), while delegation combines two independent objects (3 + 4 = 7 classes).
- **Deciding while the program runs.** An object's class is fixed at `new`. A generator can be picked once the customer's country is known, and even replaced later on the same invoice.

Inheritance is therefore a legitimate option; delegation simply copes with more situations. (*Head First Design Patterns*, a popular introductory book, calls this "favour composition over inheritance".)

What is still not closed

Only invoice *generation* was opened up. `applyDiscount` still lives inside `Invoice`, so a new kind of discount still means an edit there, and that is fine. **"Closed" is never absolute: a class is closed against a specific**

kind of change, normally one you have already watched happen. Discount rules have not varied yet, so there is no reason to abstract them yet. Martin's term for this is **strategic closure**: since no real program can be closed against every possible change, the designer picks which kinds of change to close against, guided by experience.

Approaches to OCP

Approach	How it extends	Pattern / Java feature	Good when
Interface + delegation	New class implements the interface	Strategy, `Comparator`	The algorithm varies, choose at run time
Inheritance + overriding	Subclass overrides a hook method (an overridable step the parent calls)	Template Method, abstract classes	Fixed skeleton, a few steps vary
Wrapping	New class wraps the old and adds behaviour	Decorator (`BufferedInputStream`)	Add features around existing behaviour
Listening	New subscriber registers	Observer, event listeners	New reactions to existing events
Plug-ins / registry	New implementation registers itself in a map or via `ServiceLoader` (Java's built-in way to discover implementations at start-up)	Plug-in architecture	Third parties add features
Functions	Pass a lambda	`java.util.function`, `@FunctionalInterface`	Small, one-method variations
Data / configuration	New entry in a table or config file	Rules engines, feature flags (on/off switches kept in configuration)	Variations that are just different numbers

The `switch` does not always disappear. It moves to **one** place at the edge of the program, often a factory or a `Map<String, InvoiceGenerator>`, instead of being repeated inside business logic.

Importance of OCP

- **Extensibility** — features are added without touching the tested core.
- **Faster testing** — you test the new class, and the core's tests stay valid.
- **Fewer regressions** (old features that break after a change) — working code is not edited, so it is not broken.
- **Adaptability** — the software absorbs new requirements without rewrites.

Caution: do not over-engineer

- **Start simple.** Write the `if` first. Introduce the interface when the second or third case actually arrives (the "rule of three").
- **Balance.** Too many abstraction layers make the code harder to read, which defeats the purpose.
- **Think about real needs.** Extension points for futures that never arrive are wasted effort. *Refactoring* calls this smell **Speculative Generality**.

How OCP relates to SRP

The SOLID principles reinforce one another. OCP pulls the parts that change away from the core that stays put, and each changing part ends up in a class with a single job, which is SRP. In practice SRP usually has to happen first, because the extension point stays hidden until the class is down to one job.

Trade-offs

You gain	You pay
New variants without editing old code	An interface plus one class per variant
Each variant tested on its own	Behaviour is spread over files; reading the whole flow takes more hops
Variants selectable at run time	Someone must choose the variant (factory, map, DI, i.e. dependency injection, Section 4.10)

When not to use it

- Nothing has varied yet.
- The "variants" are just different numbers (use data, not classes).
- The set of cases is fixed and small and will not grow (e.g. the two values of a boolean). Java 21's **sealed interfaces with pattern-matching** `switch` (a sealed interface lists the only types allowed to implement it, so a `switch` can check it has covered them all) are a good fit here: the compiler checks every case is handled, which is the opposite trade (easy to add operations, hard to add types).

Common mistakes

- Thinking OCP means "never edit a file". Bug fixes, refactoring and changes to the extension point itself all involve editing.
- Building plug-in systems for one implementation.
- Replacing a `switch` in business logic with a `switch` in five factories.

Refactorings and smells

- Smell: **Repeated Switches** (called *Switch Statements* in the first edition of *Refactoring*).
- Fix: **Replace Conditional with Polymorphism**; also **Replace Type Code with Subclasses**.

Real-world examples

- **`Comparator`**: `list.sort(comparator)` never changes; you pass a new comparator for a new ordering.
- **Java I/O decorators**: `new BufferedInputStream(new FileInputStream(f))` adds buffering without editing `FileInputStream`.
- **Servlet filters / Spring `HandlerInterceptor`** (code that runs before and after every web request in a Java web server): new cross-cutting behaviour is a new filter registered in a chain.
- **JDBC drivers** via `ServiceLoader`: a new database is supported by adding a driver JAR (a zipped bundle of compiled Java classes), not by editing `DriverManager`.
- **IDE plug-ins** (IntelliJ, VS Code extensions) and **browser extensions**: the core is closed, the plug-in API is the port.

In the interview

> **30-second answer:** "OCP says you should be able to add the next variation by adding code, not editing working code. When I see a `switch` over a type that keeps growing, I put the varying behaviour behind an interface, one class per variant, and the stable class delegates to it. That is Strategy. I only do it once the variation has actually appeared, and I say that a class is closed against a specific kind of change, not against everything."

Follow-ups:

- *"Does OCP mean you never edit a class?"* — No. It means the next variation of a known kind is an addition. Bug fixes and refactoring still edit.
- *"Inheritance or delegation?"* — Both satisfy OCP. Delegation handles several axes of variation and run-time choice better.
- *"Would you add an interface for pricing up front?"* — Only if pricing has already varied or is clearly about to. Otherwise I leave the `if` and refactor when the second case shows up.
- *"The interviewer adds: 'trucks are charged a different rate'."* — A design that answers "add a class" beats one that answers "add a branch in this switch".

Self-check

1. **Q:** What does "closed for modification" mean? **A:** Adding the next variation does not require editing tested code.
2. **Q:** After the refactor, is `Invoice` closed against new discount rules? **A:** No, only against new formats. That is fine until discounts vary.
3. **Q:** Which pattern is "OCP for an algorithm"? **A:** Strategy.
4. **Q:** Give a quick OCP test. **A:** "To add the next case, must I open this file?"

TL;DR

Put the part that varies behind an interface, add variants as new classes, choose them at the edge. Do it when variation is real, and remember closure is always against a specific kind of change.

4.8 Liskov Substitution Principle (LSP)

What it is

Objects of a subtype must be usable wherever objects of the parent type are expected, without making the program incorrect. Barbara Liskov introduced the idea in a 1987 keynote; with Jeannette Wing she later (1994) defined it precisely as *behavioural subtyping*. Robert C. Martin's short form is: **subtypes must be substitutable for their base types.**

"Substitutable" means more than "it compiles". Code written against the parent type must still get **correct results** when handed a child.

> **Analogy:** a hire-car company promises "a car that takes petrol". If they hand you an electric car and you drive to a petrol station, the promise was broken, even though an electric car is certainly a car.

The problem it solves

Polymorphism only works if callers can trust the base type. If some subclass secretly behaves differently, every caller has to check which subclass it got (`instanceof`), and the benefit of polymorphism, and of OCP, is gone.

Example 1: `Car` and `ElectricCar`

`Car` has `start()`, `stop()` and `refuel()`. `ElectricCar extends Car`. An electric car starts and stops fine, but `refuel()` makes no sense, so it throws. It also adds its own `recharge()`.

```java
// File: code/lld/ch04/lsp/car/before/Car.java
package lld.ch04.lsp.car.before;

public class Car {
    public void start() {
        System.out.println("Car is starting");
    }

    public void stop() {
        System.out.println("Car is stopping");
    }

    /** Names a MECHANISM (liquid fuel) instead of a goal. */
    public void refuel() {
        System.out.println("Car is refueling");
    }
}
```

```java
// File: code/lld/ch04/lsp/car/before/ElectricCar.java
package lld.ch04.lsp.car.before;

/** VIOLATION: inherits a promise (refuel) it cannot keep. */
public class ElectricCar extends Car {
    @Override
    public void refuel() {
        throw new UnsupportedOperationException("Electric cars don't need refueling");
    }

    public void recharge() {
        System.out.println("Electric car is recharging");
    }
}
```

Code that *knows* it holds an `ElectricCar` can call `recharge()` and never notice a problem. But the new method does not make `refuel()` go away; the broken method is still part of the object's public face. LSP is tested by code that does **not** know the subclass, such as `service(Car car)` in `Main` below: it calls `start`, `refuel` and `stop` on whatever `Car` it receives, and it crashes when that car is electric.

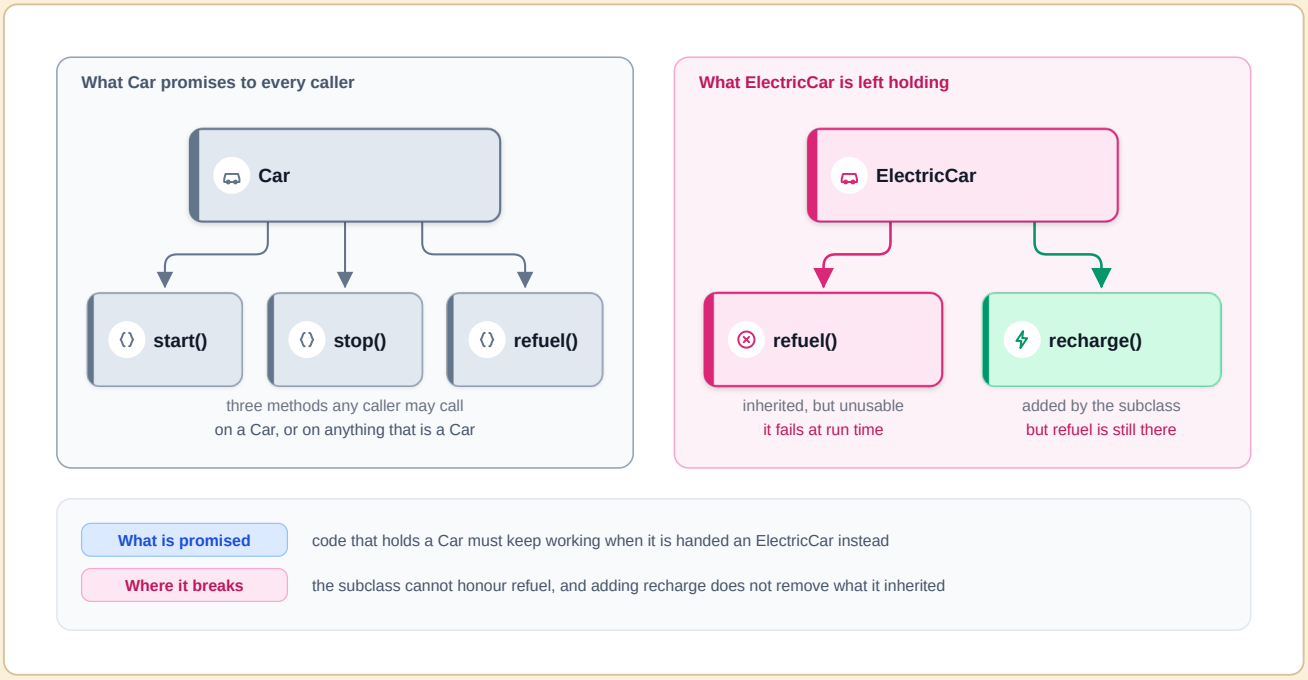


Figure — what Car promises every caller, and the unusable refuel() that ElectricCar is left holding (course diagram)

As the figure above shows, `ElectricCar` cannot fulfil the contract of `Car`, so it is not a valid substitute.

Name the real mistake. "`ElectricCar` should not extend `Car`" is only half the diagnosis. The root cause is the method itself: `refuel()` describes one **mechanism** (pouring in liquid fuel) that only some vehicles use. A method on a base class should describe the **goal** that all subclasses have in common. Every vehicle must take on energy before driving, so the common goal is `replenishEnergy()`.

The fix: raise a general parent. `Vehicle` declares the goal. `Car` and `ElectricCar` both extend `Vehicle`, and **neither extends the other**:

```java
// File: code/lld/ch04/lsp/car/after/Vehicle.java
package lld.ch04.lsp.car.after;

/** The base type promises the GOAL every vehicle shares, not one mechanism. */
public abstract class Vehicle {
    public void start() {
        System.out.println(getClass().getSimpleName() + " is starting");
    }

    public void stop() {
        System.out.println(getClass().getSimpleName() + " is stopping");
    }

    public abstract void replenishEnergy();
}
```

```java
// File: code/lld/ch04/lsp/car/after/Car.java
package lld.ch04.lsp.car.after;

public class Car extends Vehicle {
    @Override
    public void replenishEnergy() {
        System.out.println("Car is pumping gasoline");
    }
}
```

```java
// File: code/lld/ch04/lsp/car/after/ElectricCar.java
package lld.ch04.lsp.car.after;

/** A sibling of Car, not a child of it. Both can stand in for Vehicle. */
public class ElectricCar extends Vehicle {
    @Override
    public void replenishEnergy() {
        System.out.println("Electric car is charging its battery");
    }
}
```

Three points worth stating carefully:

- **The easy workaround is still wrong.** You could keep `refuel()` and have `ElectricCar.refuel()` secretly charge the battery. The exception disappears, but the method name keeps telling callers something untrue about half the fleet. Renaming fixes the promise itself rather than disguising it.
- **Why not just add `recharge()` to `ElectricCar`?** Code typed as `Vehicle` has no way to call it. A caller may depend only on what the base type declares, never on extras one subclass happens to offer.
- **Substitution goes from child to parent, never between siblings.** `Car` and `ElectricCar` can each replace a `Vehicle`. An `ElectricCar` was never meant to replace a `Car`; the two are siblings.

When to break a hierarchy, watch for:

- a subclass that **cannot implement** a parent method (it throws or does nothing),
- a base method that describes a **mechanism** rather than a goal,
- callers that **test the type** (`if (v instanceof ElectricCar)`), a sign that the base type promises too little or promises the wrong thing,
- subclasses with almost nothing in common, where the shared parent invents a relationship that is not really there.

The fix is usually one of two moves: **raise a general parent** (as above) or **take the varying behaviour out of the hierarchy** with composition (next).

Example 2: birds, and composition instead of inheritance

Here `fly()` is declared on `Bird`, so penguins inherit it and have to reject it. The fault is not the penguin; it is that `Bird` made a promise only part of its family can keep. (In *Refactoring*, a subclass that does not want what it inherits is the **Refused Bequest** smell.)

Two ways to connect classes:

- **Inheritance is an is-a relationship.** `Sparrow extends Bird`: a sparrow is a bird and receives **every** method of `Bird`, whether it suits it or not.
- **Composition is a has-a relationship.** `Bird` holds a field of type `MovementStyle`: a bird *has* a way of moving and calls it. It inherits nothing from it.

The fix: turn the varying behaviour (how a bird moves) into its own small type, and give each bird one through its constructor. There are **no bird subclasses** at all. A sparrow and a penguin are both `Bird`, built with a different `MovementStyle`.

```java
// File: code/lld/ch04/lsp/birds/inheritance/Bird.java
package lld.ch04.lsp.birds.inheritance;

/** Promises fly() to EVERY bird -- a promise only some birds can keep. */
public class Bird {
    public void fly() {
        System.out.println(getClass().getSimpleName() + " is flying");
    }
}
```

```java
// File: code/lld/ch04/lsp/birds/inheritance/Sparrow.java
package lld.ch04.lsp.birds.inheritance;

public class Sparrow extends Bird { }
```

```java
// File: code/lld/ch04/lsp/birds/inheritance/Penguin.java
package lld.ch04.lsp.birds.inheritance;

/** VIOLATION ("refused bequest"): inherits fly() and has to refuse it. */
public class Penguin extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("Penguins can't fly");
    }
}
```

```java
// File: code/lld/ch04/lsp/birds/composition/MovementStyle.java
package lld.ch04.lsp.birds.composition;

/** The behaviour that varies, pulled out into its own small type. */
@FunctionalInterface
public interface MovementStyle {
    void move(String birdName);
}
```

```java
// File: code/lld/ch04/lsp/birds/composition/Flying.java
package lld.ch04.lsp.birds.composition;

public class Flying implements MovementStyle {
    @Override
    public void move(String birdName) {
        System.out.println(birdName + " is flying through the air");
    }
}
```

```java
// File: code/lld/ch04/lsp/birds/composition/Swimming.java
package lld.ch04.lsp.birds.composition;

public class Swimming implements MovementStyle {
    @Override
    public void move(String birdName) {
        System.out.println(birdName + " is swimming through the water");
    }
}
```

```java
// File: code/lld/ch04/lsp/birds/composition/Running.java
package lld.ch04.lsp.birds.composition;

public class Running implements MovementStyle {
    @Override
    public void move(String birdName) {
        System.out.println(birdName + " is running along the ground");
    }
}
```

```java
// File: code/lld/ch04/lsp/birds/composition/Bird.java
package lld.ch04.lsp.birds.composition;

import java.util.Objects;

/** A bird HAS-A movement style. No subclasses, so nothing is inherited and refused. */
public final class Bird {
    private final String name;
    private MovementStyle movement;

    public Bird(String name, MovementStyle movement) {
        this.name = name;
        this.movement = Objects.requireNonNull(movement);
    }

    public void eat() {
        System.out.println(name + " is eating");
    }

    /** Tell, don't ask: callers say "travel", the bird decides how. */
    public void travel() {
        movement.move(name);
    }

    /** Composition can change behaviour at run time (e.g. an injured bird). */
    public void changeMovement(MovementStyle movement) {
        this.movement = Objects.requireNonNull(movement);
    }
}
```

Why this satisfies LSP: `Bird` now promises only what every bird can do (`eat()` and `travel()`), so there is nothing any bird must refuse. Flight has become one optional object that some birds are given, not a promise to all of them. Supporting the ostrich took one new movement style instead of an `Ostrich` subclass stuck with a meaningless `fly()`. And since the style is just a field, it can be replaced while the program runs (`changeMovement`).

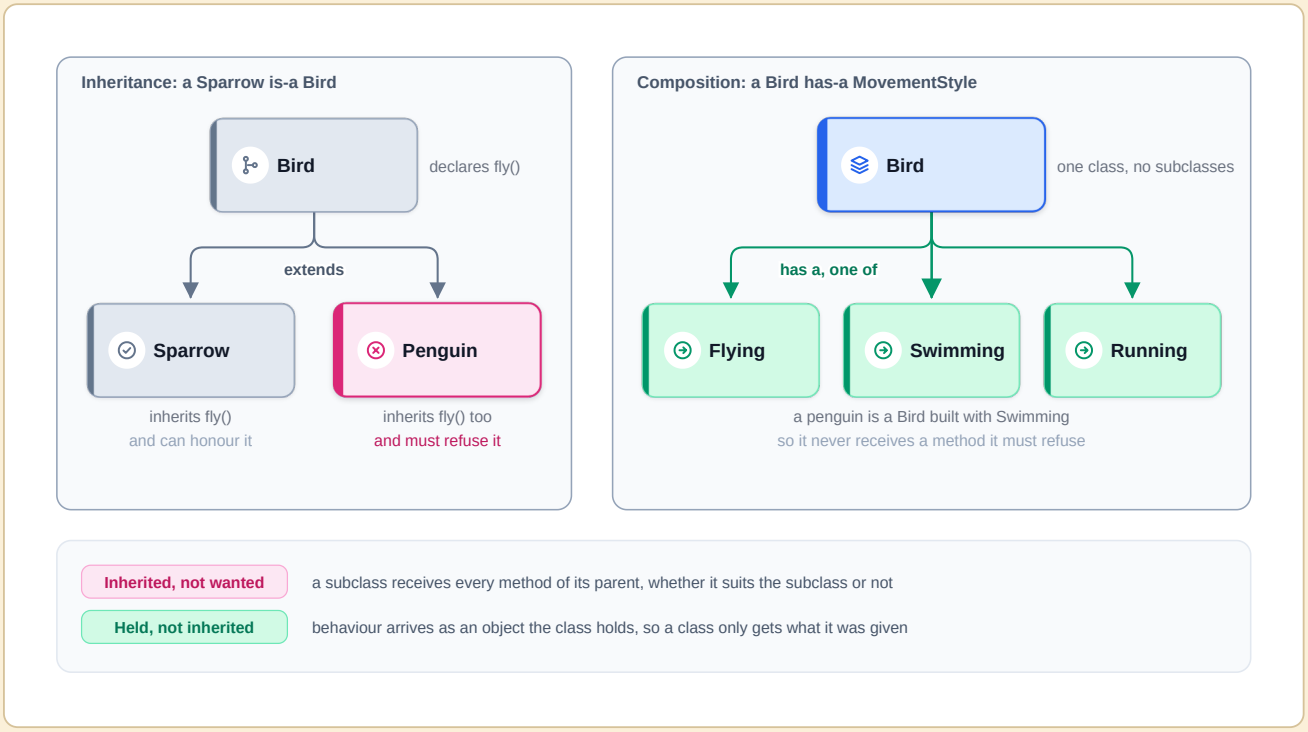
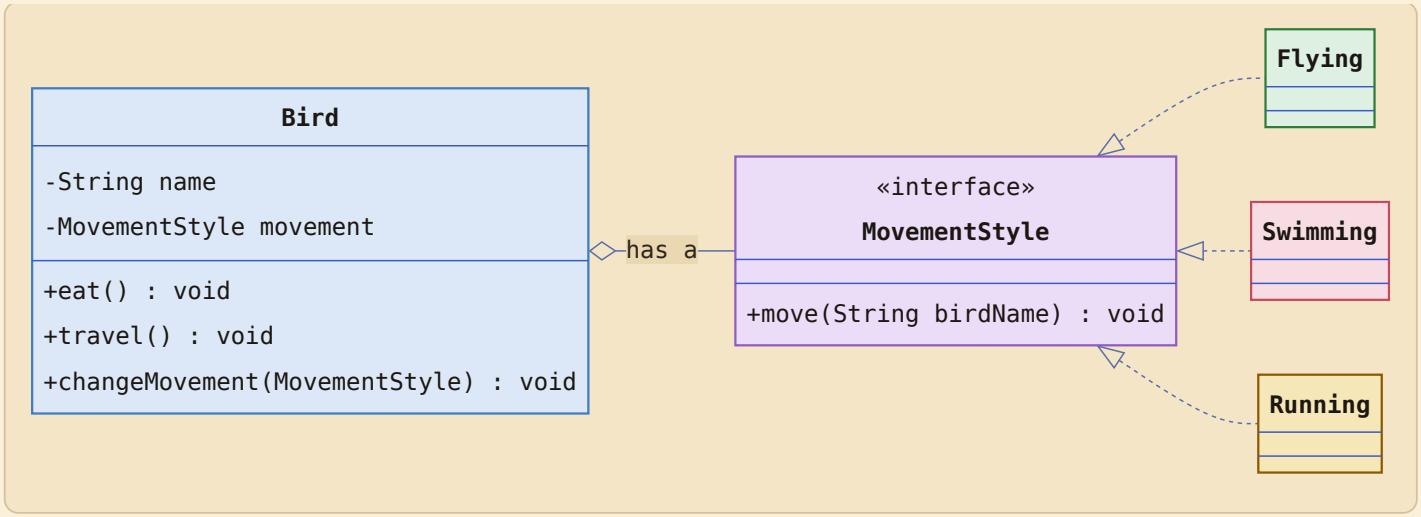


Figure — inheritance (Penguin inherits fly() and refuses it) vs composition (a Bird has a MovementStyle) (course diagram)

In the figure above, the left side is the inheritance tree where the penguin receives a method it does not want. The right side is composition: one `Bird` class holding one of `Flying`, `Swimming`, `Running`.



Is-a and has-a side by side:

	Inheritance	Composition
Kind of link	Penguin **is a** Bird	Bird **has a** MovementStyle
In code	`class Penguin extends Bird`	`Bird` holds a `MovementStyle` field
What it gets	all of the parent's methods	just the object handed to it
To change behaviour	add another subclass	supply another object
Change while running?	no, fixed at creation	yes, replace the field
LSP risk	high: inherited methods may not suit	low: nothing is inherited

When to use which. Choose **inheritance** when the child genuinely is a kind of the parent *and* can keep every inherited promise. Choose **composition** when what differs between objects is a behaviour, or the moment you find a subclass rejecting something it inherited. That rejection is the tell: an overridden method that throws, silently does nothing, or carries a "never call this on X" comment should become a separate object.

Tell, Don't Ask

The composition design above follows a related guideline, popularised by Andy Hunt and Dave Thomas (*The Pragmatic Programmer* authors): **tell an object what to do; do not ask it about its state and then decide for it.**

```
// Asking: the caller inspects the object and makes the decision.
if (bird instanceof Penguin) { swim(bird); } else { fly(bird); }

// Telling: the caller states the goal, the object decides how.
bird.travel();
```

Asking code has to know every subtype, so it breaks OCP and hides LSP problems behind `instanceof` checks. Telling code works with any substitute. In the composition design, `bird.travel()` is the "tell". A caller that needs `instanceof` on a base type is a strong hint that the base type promises the wrong thing.

The rules a subclass must follow

So far every violation crashed with an exception, which makes it easy to spot. The harmful ones are those that **quietly produce a wrong result and keep running**.

Example: `Square extends Rectangle`. Geometry says every square is a rectangle, so the subclass looks natural. To remain square, `Square` keeps both sides equal: setting the width also changes the height. No exception is thrown. Yet a method written for `Rectangle` that sets width 5 and height 4 expects an area of 20 and receives 16. In production that kind of error becomes an incorrect bill or inventory figure that nobody notices for weeks.

The rule that `Rectangle` implied without ever stating is **width and height can be changed independently**; the two separate setters invited every caller to rely on it. Mathematically a square is a rectangle, but it is **not a valid subtype of this `Rectangle` class**, because the class makes a promise a square cannot honour. **Base inheritance on the promises written in the code, not on how things are classified in the real world.**

A subclass is free to do the job *differently*. What it may not do is **require more** from its callers, **give them less**, or **break an assumption they depend on**. The rules below spell that out. Their vocabulary comes from Bertrand Meyer's **Design by Contract**: a method has **preconditions** (what the caller must provide), **postconditions** (what the method guarantees afterwards) and **invariants** (what is always true of the object).

Rule 1 — Preconditions may not be strengthened. A precondition is what must be true before the call. An overriding method may accept a *wider* range of inputs than the parent, never a narrower one. In the example, `Discount` accepts any amount of zero or more, while `SmallOrderDiscount` refuses anything from 100 upward. A caller that passes 250, which the parent allows, now fails, and would have to find out which discount it was given, the very check substitution was meant to make unnecessary.

Rule 2 — Postconditions may not be weakened. A postcondition is what the method guarantees when it returns. An override may guarantee *more*, never less. `Account.withdraw` guarantees the balance stays at zero or above. `OverdraftAccount` lets it go negative: a reasonable banking feature, but a broken guarantee. Every caller that relied on "balance ≥ 0" is now silently wrong. Nothing crashes; a payment job might, for example, keep paying suppliers from an account that is already overdrawn. An overdraft account can be a perfectly good class, just not a subtype of an account that promises a non-negative balance.

Rule 3 — Invariants must be preserved. An invariant is a condition that holds throughout the object's life. `Square` broke "width and height are independent". (The same bug can equally be described as a weakened postcondition: `Rectangle.setWidth` implicitly guarantees "the height is unchanged", and `Square.setWidth` drops that guarantee. Martin describes it that way; either description leads to the same conclusion.) Invariants are the hardest rule to follow because they are rarely written down: a subclass inherits every promise of its parent, including the ones nobody documented.

Rule 4 — No new exceptions callers cannot handle. An override may throw the exceptions the parent already declares (or subtypes of them), nothing new: a new exception type arrives at callers that have no `catch` for it. `ElectricCar.refuel()` broke this rule. Note that Java's compiler enforces this only for **checked** exceptions; `UnsupportedOperationException` is unchecked, so the violating code compiled without complaint. This is the most visible rule to break and the least harmful, because the failure is immediate and loud.

Rule 5 — The history constraint. Liskov and Wing added a rule about how state changes over time: **a subtype must not allow state changes that the parent type does not allow.** In the example, `Price` promises that its value never changes after creation. `AdjustablePrice` adds an `adjust()` method. Each call looks innocent, but a caller that captured a quote of 100 is later charged 120 on "the same" price. This is why immutable Java classes such as `String` and `Integer` are declared `final`: a mutable subclass could break the immutability callers rely on.

Here are all of them in runnable code:

```java
// File: code/lld/ch04/lsp/rules/Discount.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Contract -- precondition: any amount >= 0. Postcondition: result = 90% of amount. */
public class Discount {
    private static final BigDecimal RATE = new BigDecimal("0.90");

    public BigDecimal apply(BigDecimal amount) {
        if (amount.signum() < 0) {
            throw new IllegalArgumentException("amount must be >= 0");
        }
        return amount.multiply(RATE).setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/SmallOrderDiscount.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;

/** RULE 1 BROKEN: demands MORE than the parent (amount must also be under 100). */
public class SmallOrderDiscount extends Discount {
    private static final BigDecimal LIMIT = new BigDecimal("100");

    @Override
    public BigDecimal apply(BigDecimal amount) {
        if (amount.compareTo(LIMIT) >= 0) {
            throw new IllegalArgumentException("only for orders under 100");
        }
        return super.apply(amount);
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/Account.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;

/** Contract -- postcondition of withdraw: the balance is never negative. */
public class Account {
    protected BigDecimal balance;

    public Account(BigDecimal balance) {
        this.balance = balance;
    }

    /** Withdraws only if funds allow; returns the balance afterwards (always >= 0). */
    public BigDecimal withdraw(BigDecimal amount) {
        if (amount.compareTo(balance) <= 0) {
            balance = balance.subtract(amount);
        }
        return balance;
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/OverdraftAccount.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;

/** RULE 2 BROKEN: promises LESS -- the balance may now go negative. Nothing throws. */
public class OverdraftAccount extends Account {
    public OverdraftAccount(BigDecimal balance) {
        super(balance);
    }

    @Override
    public BigDecimal withdraw(BigDecimal amount) {
        balance = balance.subtract(amount);
        return balance;
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/Rectangle.java
package lld.ch04.lsp.rules;

/** Unwritten invariant: width and height can be set independently. */
public class Rectangle {
    protected int width;
    protected int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public int area() {
        return width * height;
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/Square.java
package lld.ch04.lsp.rules;

/** RULE 3 BROKEN: to stay square it couples the two setters -- a silent wrong answer. */
public class Square extends Rectangle {
    public Square(int side) {
        super(side, side);
    }

    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width;
    }

    @Override
    public void setHeight(int height) {
        this.width = height;
        this.height = height;
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/Price.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;

/** Contract (history): once created, a Price never changes value. */
public class Price {
    private final BigDecimal value;

    public Price(BigDecimal value) {
        this.value = value;
    }

    public BigDecimal value() {
        return value;
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/AdjustablePrice.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;

/**
 * HISTORY CONSTRAINT BROKEN: adds a mutator, so an object the caller believed was
 * frozen can now change between two reads. No single method call looks wrong.
 */
public class AdjustablePrice extends Price {
    private BigDecimal current;

    public AdjustablePrice(BigDecimal value) {
        super(value);
        this.current = value;
    }

    @Override
    public BigDecimal value() {
        return current;
    }

    public void adjust(BigDecimal newValue) {
        this.current = newValue;
    }
}
```

Why it matters so much. Code written against a base type has no way to protect itself: it cannot see which subclass it received, and it should not need to. Only the subclass author can keep the contract.

The fix for Rectangle/Square. Do not share a mutable promise that one of them cannot keep. Make both **immutable** and give them a common interface that promises only what both can do (`area()`). "Resizing" returns a new object, so no setter can break an invariant. Java 21 `record`s and a `sealed` interface make this short:

```java
// File: code/lld/ch04/lsp/shapes/Shape.java
package lld.ch04.lsp.shapes;

/** The fix for Rectangle/Square: promise only what every shape can keep. */
public sealed interface Shape permits Rect, Sq {
    int area();
}
```

```java
// File: code/lld/ch04/lsp/shapes/Rect.java
package lld.ch04.lsp.shapes;

/** Immutable: "resizing" returns a new object, so no setter can break an invariant. */
public record Rect(int width, int height) implements Shape {
    public Rect withWidth(int w) {
        return new Rect(w, height);
    }

    @Override
    public int area() {
        return width * height;
    }
}
```

```java
// File: code/lld/ch04/lsp/shapes/Sq.java
package lld.ch04.lsp.shapes;

public record Sq(int side) implements Shape {
    @Override
    public int area() {
        return side * side;
    }
}
```

And the driver that runs every example, with each client method written only against the base type:

```java
// File: code/lld/ch04/lsp/Main.java
package lld.ch04.lsp;

import java.math.BigDecimal;
import java.util.List;
import lld.ch04.lsp.birds.composition.Bird;
import lld.ch04.lsp.birds.composition.Flying;
import lld.ch04.lsp.birds.composition.Running;
import lld.ch04.lsp.birds.composition.Swimming;
import lld.ch04.lsp.car.after.Vehicle;
import lld.ch04.lsp.rules.Account;
import lld.ch04.lsp.rules.AdjustablePrice;
import lld.ch04.lsp.rules.Discount;
import lld.ch04.lsp.rules.OverdraftAccount;
import lld.ch04.lsp.rules.Price;
import lld.ch04.lsp.rules.Rectangle;
import lld.ch04.lsp.rules.SmallOrderDiscount;
import lld.ch04.lsp.rules.Square;
import lld.ch04.lsp.shapes.Rect;
import lld.ch04.lsp.shapes.Shape;
import lld.ch04.lsp.shapes.Sq;

public class Main {

    // ---- Client code: every method below knows ONLY the base type ----

    static void service(lld.ch04.lsp.car.before.Car car) {
        car.start();
        car.refuel();
        car.stop();
    }

    static void takeATrip(Vehicle v) {
        v.start();
        v.replenishEnergy();
        v.stop();
    }

    static void checkout(Discount d, BigDecimal amount) {
        try {
            System.out.println("  Charged: " + d.apply(amount));
        } catch (IllegalArgumentException e) {
            System.out.println("  Broke on a valid amount: " + e.getMessage());
        }
    }

    static void paySupplier(Account account, BigDecimal amount, String label) {
        BigDecimal balance = account.withdraw(amount);
        System.out.println("  " + label + " balance after payment: " + balance
                + (balance.signum() < 0 ? "   <-- the caller never expected this" : ""));
    }

    /** A contract test written against Rectangle -- run it for every subclass. */
    static void resizeAndCheck(Rectangle r, String label) {
        r.setWidth(5);
        r.setHeight(4);
        System.out.println("  " + label + " expected area 20, got " + r.area());
    }

    static void quoteThenCharge(Price p, Runnable somethingElseHappens) {
        BigDecimal quoted = p.value();
        somethingElseHappens.run();
        System.out.println("  quoted " + quoted + ", charged " + p.value());
    }

    public static void main(String[] args) {
        System.out.println("=== 1. Car / ElectricCar (exception rule) ===");
        service(new lld.ch04.lsp.car.before.Car());
```

```
        try {
            service(new lld.ch04.lsp.car.before.ElectricCar());
        } catch (UnsupportedOperationException e) {
            System.out.println("  substitution broke: " + e.getMessage());
        }

        System.out.println("=== 2. Fixed hierarchy: Vehicle -> Car | ElectricCar ===");
        List<Vehicle> fleet = List.of(new lld.ch04.lsp.car.after.Car(),
                new lld.ch04.lsp.car.after.ElectricCar());
        fleet.forEach(Main::takeATrip);

        System.out.println("=== 3. Precondition strengthened ===");
        checkout(new Discount(), new BigDecimal("250.00"));
        checkout(new SmallOrderDiscount(), new BigDecimal("250.00"));

        System.out.println("=== 4. Postcondition weakened ===");
        paySupplier(new Account(new BigDecimal("50.00")), new BigDecimal("80.00"), "Account:         ");
        paySupplier(new OverdraftAccount(new BigDecimal("50.00")), new BigDecimal("80.00"),
                "OverdraftAccount:");

        System.out.println("=== 5. Invariant broken (Rectangle / Square) ===");
        resizeAndCheck(new Rectangle(1, 1), "Rectangle:");
        resizeAndCheck(new Square(1), "Square:   ");

        System.out.println("=== 6. History constraint broken ===");
        quoteThenCharge(new Price(new BigDecimal("100.00")), () -> { });
        AdjustablePrice sneaky = new AdjustablePrice(new BigDecimal("100.00"));
        quoteThenCharge(sneaky, () -> sneaky.adjust(new BigDecimal("120.00")));

        System.out.println("=== 7. Fix for shapes: no shared mutable promise ===");
        List<Shape> shapes = List.of(new Rect(5, 4), new Sq(4), new Rect(2, 3).withWidth(5));
        for (Shape s : shapes) {
            System.out.println("  " + s + " area=" + s.area());
        }

        System.out.println("=== 8. Birds: inheritance vs composition ===");
        List<lld.ch04.lsp.birds.inheritance.Bird> old = List.of(
                new lld.ch04.lsp.birds.inheritance.Sparrow(),
                new lld.ch04.lsp.birds.inheritance.Penguin());
        for (var b : old) {
            try {
                b.fly();
            } catch (UnsupportedOperationException e) {
                System.out.println("  Substitution broke: " + e.getMessage());
            }
        }
        List<Bird> birds = List.of(
                new Bird("Sparrow", new Flying()),
                new Bird("Penguin", new Swimming()),
                new Bird("Ostrich", new Running()));
        birds.forEach(Bird::travel);
        Bird sparrow = birds.get(0);
        sparrow.changeMovement(new Running()); // behaviour swapped at run time
        sparrow.travel();
    }
}
```

Output:

```
=== 1. Car / ElectricCar (exception rule) ===
Car is starting
Car is refueling
Car is stopping
Car is starting
  substitution broke: Electric cars don't need refueling
=== 2. Fixed hierarchy: Vehicle -> Car | ElectricCar ===
Car is starting
Car is pumping gasoline
Car is stopping
ElectricCar is starting
Electric car is charging its battery
ElectricCar is stopping
=== 3. Precondition strengthened ===
  Charged: 225.00
  Broke on a valid amount: only for orders under 100
=== 4. Postcondition weakened ===
  Account:          balance after payment: 50.00
  OverdraftAccount: balance after payment: -30.00   <-- the caller never expected this
=== 5. Invariant broken (Rectangle / Square) ===
  Rectangle: expected area 20, got 20
  Square:    expected area 20, got 16
=== 6. History constraint broken ===
  quoted 100.00, charged 100.00
  quoted 100.00, charged 120.00
=== 7. Fix for shapes: no shared mutable promise ===
  Rect[width=5, height=4] area=20
  Sq[side=4] area=16
  Rect[width=5, height=3] area=15
=== 8. Birds: inheritance vs composition ===
Sparrow is flying
  Substitution broke: Penguins can't fly
Sparrow is flying through the air
Penguin is swimming through the water
Ostrich is running along the ground
Sparrow is running along the ground
```

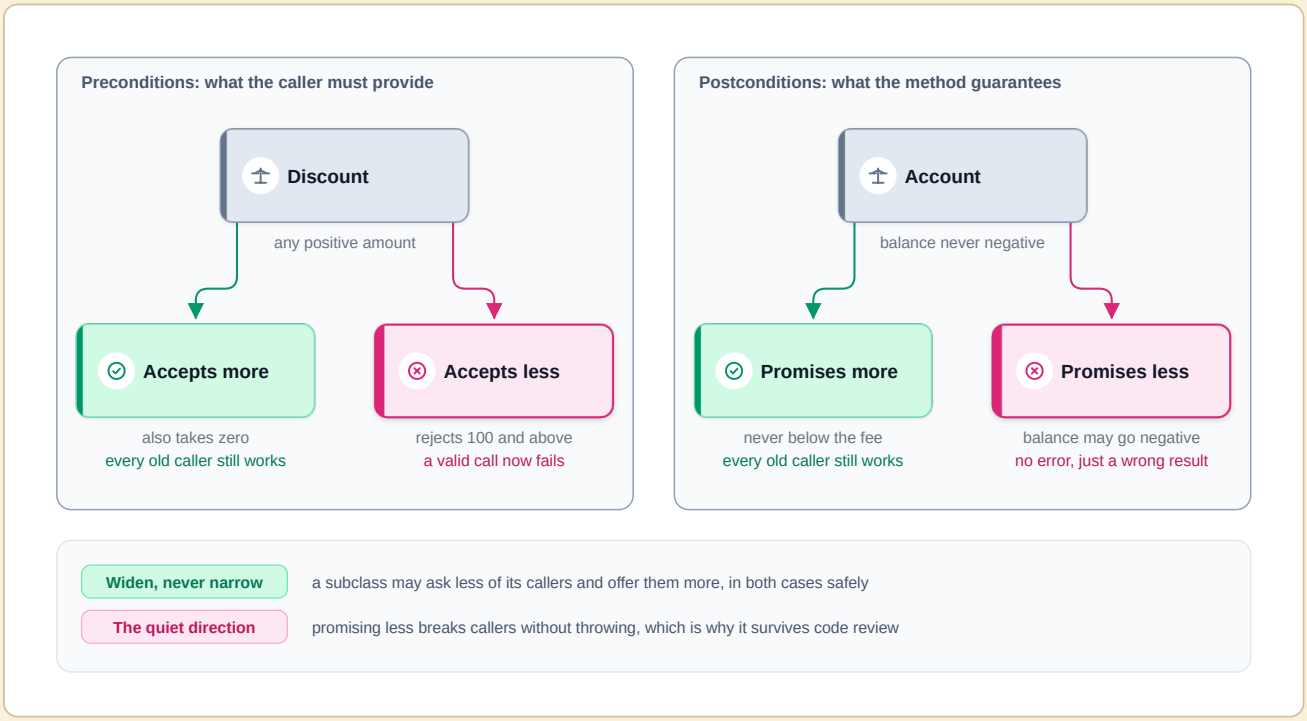


Figure — preconditions and postconditions: accepting more or promising more is safe; accepting less or promising less breaks callers (course diagram)

The figure above sums up the direction: **widen, never narrow.** A subclass may ask less of its callers and offer them more. (In the figure the parent `Discount` accepts positive amounts and the safe subclass also accepts zero;

in this chapter's code the parent already accepts zero, but the direction of the rule is the same. Likewise the figure's "promises more" account keeps the balance above a fee, a stronger guarantee than "never negative".)

The contract in one table:

Rule	The subclass may not…	How it usually shows up
Preconditions	demand more from the caller	valid input is rejected
Postconditions	guarantee less to the caller	a returned value breaks a caller's assumption
Invariants	break a rule that always held	a silently wrong result (the square)
Exceptions	throw something new	a crash in code with no matching `catch`
History	allow state changes the parent forbids	a "fixed" value changes between two reads

Signature rules (the part the compiler checks). Behavioural subtyping also has type-level rules: parameter types may be *wider* (contravariant), return types may be *narrower* (covariant), and exceptions may only be *narrower*. Java enforces a version of this for you:

- an overriding method may return a **subtype** of the parent's return type (covariant returns),
- it may **not** declare new or broader **checked** exceptions,
- it may **not** reduce visibility (a `public` method stays `public`),
- parameter types must match exactly (a different parameter type is an **overload**, not an override; use `@Override` so the compiler catches this).

Java checks none of the **behavioural** rules. Unchecked exceptions, weakened postconditions and broken invariants all compile happily. That part is on you.

How to check a subclass

1. **List the parent's promises before writing the subclass.** Promises, not method signatures: "the balance never drops below zero", "width and height change independently". Most violations slip in because the promise was never stated, so its breach went unnoticed.
2. **Write one test against the parent type and run it for every subclass.** `resizeAndCheck(Rectangle)` is such a test: it never mentions `Square`, yet `Square` fails it. Whenever a base-type test fails for some subclass, that subclass violates LSP, regardless of how reasonable its code looks. (In JUnit, the standard Java testing library, this is an *abstract contract test* that each subclass's test class extends.)

Why LSP is important

- **Prevents fragile code** — callers are not surprised by a subclass.
- **Keeps polymorphism usable** — new subclasses plug in without `instanceof` checks, which is what makes OCP work.
- **Correctness** — results stay right, not just "no crash".
- **Reuse** — base types can be extended safely.

Trade-offs

Following LSP gains	Costs
Callers trust the base type; no type checks	You must think about and document contracts
New subclasses are safe to add	Some "natural" hierarchies (Square/Rectangle) must be abandoned
Composition gives run-time flexibility	More small objects; behaviour is spread out

When not to use it

LSP is not optional where you use polymorphism. What you can skip is inheritance itself: if a subclass cannot keep the parent's promises, **do not make it a subclass**.

Common mistakes

- Modelling the real world ("a square is a rectangle", "a penguin is a bird") instead of the code's promises.
- "Fixing" a violation by making the method do nothing silently. That just changes a loud failure into a quiet wrong result.

- Thinking LSP is only about exceptions. The silent rules (postconditions, invariants, history) are more dangerous.
- Believing that siblings must substitute for each other. Substitution is only child → parent.

Refactorings and smells

- Smell: **Refused Bequest** (a subclass does not want what it inherits).
- Fixes: **Replace Subclass with Delegate**, **Replace Superclass with Delegate** (composition), **Extract Superclass** (raise a general parent), **Push Down Method** (move a method only some subclasses support out of the parent).

Real-world examples

- **`java.sql.Date extends java.util.Date`**: its time methods such as `getHours()` throw `IllegalArgumentException`, because an SQL date has no time part. Code written against `java.util.Date` can break.
- **`java.sql.Timestamp`**: its Javadoc (the official documentation generated from code comments) warns that its `equals` is not symmetric with `java.util.Date.equals`, and says the inheritance is implementation inheritance, not type inheritance. A textbook LSP warning in the JDK itself.
- **`java.util.Stack extends Vector`**: a stack inherits `insertElementAt` and `get(int)`, which let callers break stack order. The Javadoc now recommends `Deque` (a double-ended queue that adds and removes only at its ends) instead.
- **`List.of(...)` and `Collections.unmodifiableList(...)`** throw `UnsupportedOperationException` from `add`. The `List` interface marks `add` as an *optional operation*, so this is written into the contract, but it is a known design compromise: a caller holding a `List` cannot know whether `add` works.

In the interview

30-second answer: "LSP says a subclass must work anywhere its parent is expected, giving correct results, not just no crash. Concretely: it cannot demand more input, promise less output, break an invariant, throw new exceptions, or allow state changes the parent forbids. Square extending a mutable Rectangle is the classic silent break: set width 5 and height 4 and you get 16. I test it by writing one test against the base type and running it for every subclass. When a subclass must refuse a method, I either raise a better parent or move the behaviour into a composed object."

Follow-ups:

- *"Give an LSP violation you've seen."* — The Rectangle/Square case, or `java.sql.Date` throwing from `getHours()`. Then state the rule: demand no more, promise no less, keep the invariants.
- *"Composition over inheritance, always?"* — No. Inheritance is fine when the child truly honours every parent method. The signal to switch is a subclass refusing an inherited method.
- *"Is `ElectricCar` a substitute for `Car` after your fix?"* — No, and it should not be. Both substitute for `Vehicle`; siblings do not substitute for each other.

Self-check

1. **Q:** A subclass accepts **fewer** inputs than its parent. Which rule? **A:** Preconditions strengthened (rule 1).
2. **Q:** Why is the Square/Rectangle bug more dangerous than `ElectricCar.refuel()` throwing? **A:** It gives a silent wrong answer; the exception fails loudly and is found at once.
3. **Q:** What is the history constraint? **A:** A subtype must not allow state changes the parent does not allow, e.g. a mutable subclass of an immutable class.
4. **Q:** What does `@Override` protect you from? **A:** Accidentally overloading (different parameter type) instead of overriding.
5. **Q:** What is "Tell, Don't Ask"? **A:** Tell an object to do the job (`bird.travel()`), instead of inspecting its type or state and deciding for it.

TL;DR

A subclass must keep every promise of its parent: accept at least as much, deliver at least as much, keep invariants and history, throw nothing new. Inheritance follows promises, not real-world categories. When a subclass must refuse something, raise a better parent or use composition.

4.9 Interface Segregation Principle (ISP)

What it is

Clients should not be forced to depend on methods they do not use. Instead of one large, general interface, offer smaller interfaces shaped around what each kind of caller needs.

Where it comes from. Robert C. Martin describes meeting the problem while consulting for Xerox. A printer system had one big `Job` class used by every task (printing, stapling and so on), so a change for one task forced every other task's code to be rebuilt. Splitting the class's services into separate interfaces, one per kind of client, fixed it.

> **Analogy:** a restaurant that hands every guest the full 40-page menu, including the wine list, the kids' menu and the catering price list, even to someone who only wants a coffee. A coffee card is enough for them.

A **fat interface** is an interface with many methods, usually because it grew one feature at a time. A **role interface** is a small interface describing the role an object plays for one kind of caller.

The problem it solves

- Implementers are forced to write methods they cannot honour, so they throw or do nothing (which also breaks LSP).
- Callers depend on methods they never call, so a change to those methods still reaches them (recompiles, redeploys, retests).
- Test doubles become long, because a fake must implement every method.

Example: a bloated `Printer` interface

One interface for everything a printer might do: print, scan, fax, staple.

```java
// File: code/lld/ch04/isp/fat/Printer.java
package lld.ch04.isp.fat;

/** A FAT interface: one list of everything any printer might ever do. */
public interface Printer {
    void printDocument(String doc);

    void scanDocument(String doc);

    void faxDocument(String doc);

    void stapleDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/fat/BasicPrinter.java
package lld.ch04.isp.fat;

/** VIOLATION: forced to "implement" three methods it can only refuse. */
public class BasicPrinter implements Printer {
    @Override
    public void printDocument(String doc) {
        System.out.println("BasicPrinter printing " + doc);
    }

    @Override
    public void scanDocument(String doc) {
        throw new UnsupportedOperationException("BasicPrinter cannot scan");
    }

    @Override
    public void faxDocument(String doc) {
        throw new UnsupportedOperationException("BasicPrinter cannot fax");
    }

    @Override
    public void stapleDocument(String doc) {
        throw new UnsupportedOperationException("BasicPrinter cannot staple");
    }
}
```

```java
// File: code/lld/ch04/isp/fat/AdvancedPrinter.java
package lld.ch04.isp.fat;

/** The wide interface is honest for this device -- it really does all four jobs. */
public class AdvancedPrinter implements Printer {
    @Override
    public void printDocument(String doc) {
        System.out.println("AdvancedPrinter printing " + doc);
    }

    @Override
    public void scanDocument(String doc) {
        System.out.println("AdvancedPrinter scanning " + doc);
    }

    @Override
    public void faxDocument(String doc) {
        System.out.println("AdvancedPrinter faxing " + doc);
    }

    @Override
    public void stapleDocument(String doc) {
        System.out.println("AdvancedPrinter stapling " + doc);
    }
}
```

`BasicPrinter` only prints, but it must "implement" scanning, faxing and stapling, and the only honest thing it can do is throw. `AdvancedPrinter` really does all four, so for it the wide interface is fine.

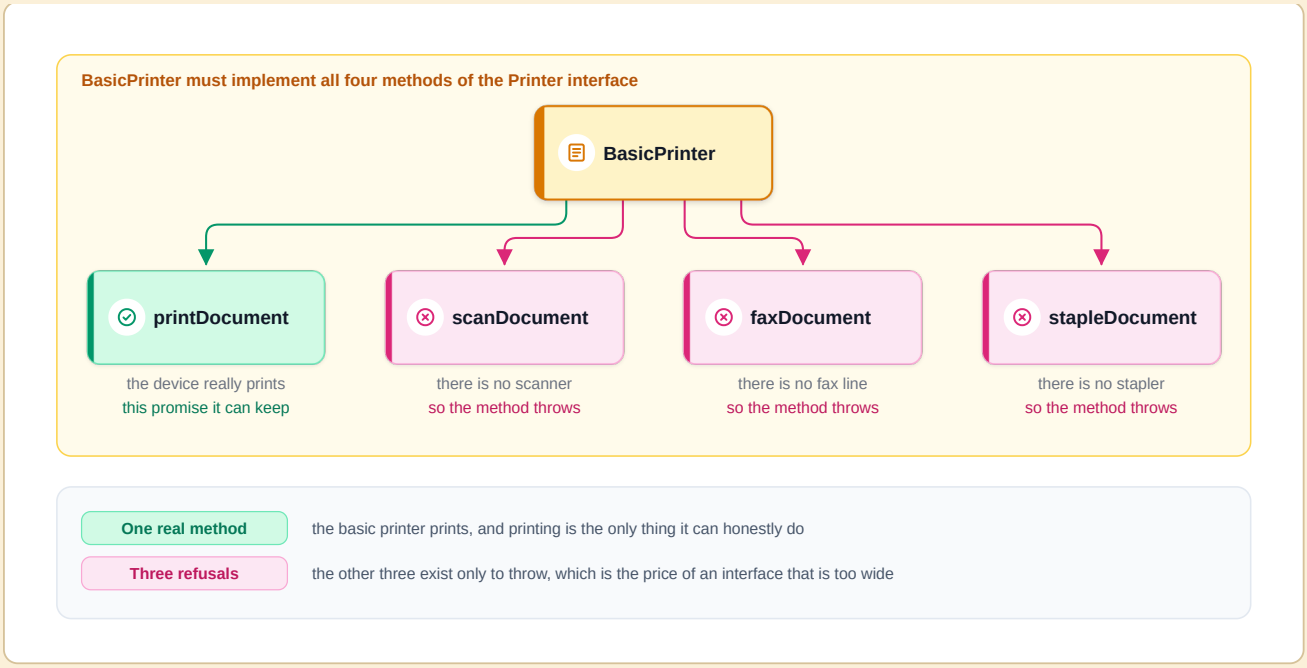


Figure — BasicPrinter keeps one real method and three that exist only to throw (course diagram)

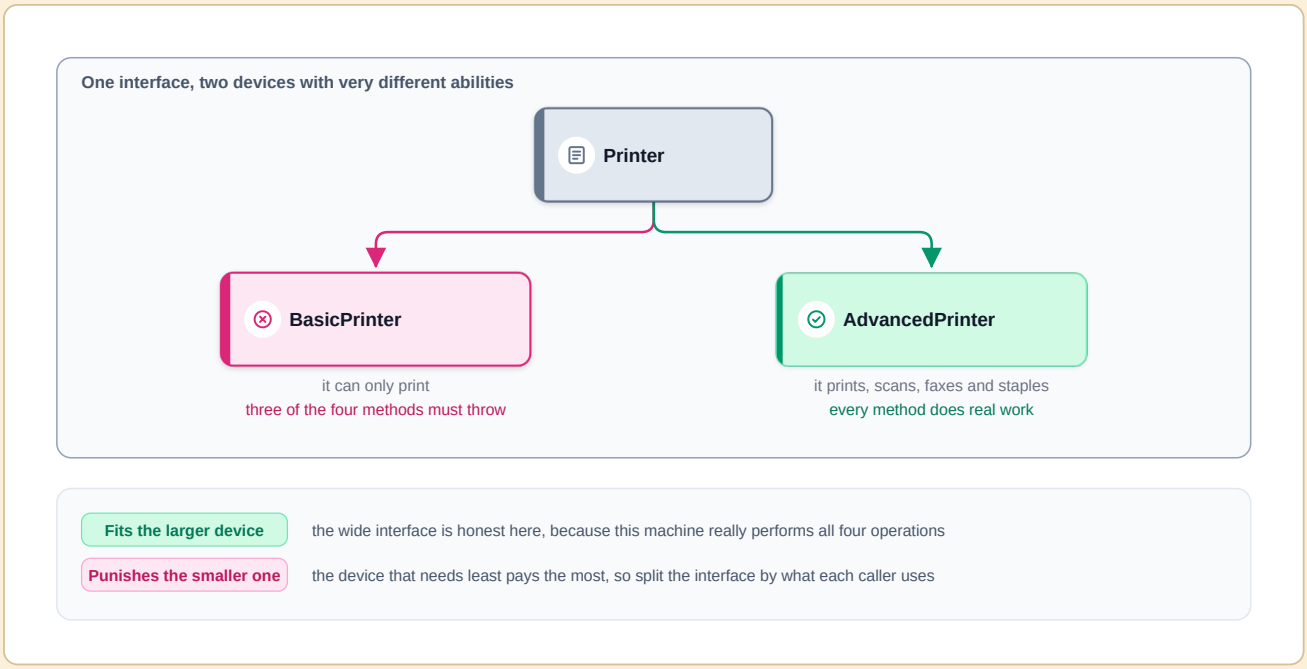


Figure — one interface, two devices: it fits the larger device and punishes the smaller one (course diagram)

The two figures above capture ISP in one picture: the simplest device carries the heaviest burden. Split the interface according to what each caller actually uses.

Restructuring: first attempt, one interface per capability

Split the big interface into `Printer`, `Scanner`, `Fax` and `Stapler`. `BasicPrinter` implements only `Printer`; `AdvancedPrinter` implements all four. No class implements anything it cannot do.

```java
// File: code/lld/ch04/isp/split/Printer.java
package lld.ch04.isp.split;

public interface Printer {
    void printDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/split/Scanner.java
package lld.ch04.isp.split;

public interface Scanner {
    void scanDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/split/Fax.java
package lld.ch04.isp.split;

public interface Fax {
    void faxDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/split/Stapler.java
package lld.ch04.isp.split;

public interface Stapler {
    void stapleDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/split/BasicPrinter.java
package lld.ch04.isp.split;

/** Implements only what it can honour. */
public class BasicPrinter implements Printer {
    @Override
    public void printDocument(String doc) {
        System.out.println("BasicPrinter printing " + doc);
    }
}
```

```java
// File: code/lld/ch04/isp/split/AdvancedPrinter.java
package lld.ch04.isp.split;

/** Opts in to every capability it really has. */
public class AdvancedPrinter implements Printer, Scanner, Fax, Stapler {
    @Override
    public void printDocument(String doc) {
        System.out.println("AdvancedPrinter printing " + doc);
    }

    @Override
    public void scanDocument(String doc) {
        System.out.println("AdvancedPrinter scanning " + doc);
    }

    @Override
    public void faxDocument(String doc) {
        System.out.println("AdvancedPrinter faxing " + doc);
    }

    @Override
    public void stapleDocument(String doc) {
        System.out.println("AdvancedPrinter stapling " + doc);
    }
}
```

This satisfies ISP and makes the mechanics easy to see. But **one interface per method is not the goal.** If every scanning device can also fax, and every caller uses both, putting them in separate interfaces adds names while removing no dependency at all.

How small should an interface be?

Group by caller, not by method: an interface should carry the methods that one kind of caller needs. The size is right when both answers below are "no":

1. **Is any implementer forced to supply a method it cannot honour?** If yes, the interface is too wide.
2. **Does any caller depend on methods it never calls?** If yes, it is too wide for that caller.

It might have one method or six. The count was never the measure.

What over-splitting costs. Now add a caller to the one-method-per-interface design. `OfficeWorkflow` scans, prints and staples, so it needs three constructor parameters, and whoever creates it hands over **one device three times:**

```java
// File: code/lld/ch04/isp/split/OfficeWorkflow.java
package lld.ch04.isp.split;

/** OVER-SPLIT: needs all three, so it takes three dependencies -- and gains nothing. */
public class OfficeWorkflow {
    private final Printer printer;
    private final Scanner scanner;
    private final Stapler stapler;

    public OfficeWorkflow(Printer printer, Scanner scanner, Stapler stapler) {
        this.printer = printer;
        this.scanner = scanner;
        this.stapler = stapler;
    }

    public void run(String doc) {
        scanner.scanDocument(doc);
        printer.printDocument(doc);
        stapler.stapleDocument(doc);
    }
}
```

The code runs and technically follows ISP, yet nothing was gained: `OfficeWorkflow` calls all three methods, so the split took away no dependency. It only added two extra type names and two extra parameters.

Grouping by role. `OfficeWorkflow` wants one device that can scan, print and staple, so that is a single role, `DocumentProcessor`. `PrintQueue` only ever prints, which is a different role, `Printer`. The reason two interfaces are justified is that a `BasicPrinter` can play the second role but not the first. In Java, `DocumentProcessor extends Printer`, so any document processor can also be placed in a print queue.

```java
// File: code/lld/ch04/isp/roles/Printer.java
package lld.ch04.isp.roles;

/** Role interface for callers that only print (e.g. PrintQueue). */
public interface Printer {
    void printDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/roles/DocumentProcessor.java
package lld.ch04.isp.roles;

/** Role interface for callers that scan, print AND staple (e.g. OfficeWorkflow). */
public interface DocumentProcessor extends Printer {
    void scanDocument(String doc);

    void stapleDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/roles/BasicPrinter.java
package lld.ch04.isp.roles;

public class BasicPrinter implements Printer {
    @Override
    public void printDocument(String doc) {
        System.out.println("Basic printer printing " + doc);
    }
}
```

```java
// File: code/lld/ch04/isp/roles/OfficeDevice.java
package lld.ch04.isp.roles;

public class OfficeDevice implements DocumentProcessor {
    @Override
    public void printDocument(String doc) {
        System.out.println("Printing " + doc);
    }

    @Override
    public void scanDocument(String doc) {
        System.out.println("Scanning " + doc);
    }

    @Override
    public void stapleDocument(String doc) {
        System.out.println("Stapling " + doc);
    }
}
```

```java
// File: code/lld/ch04/isp/roles/PrintQueue.java
package lld.ch04.isp.roles;

/** Depends on printing only, so a basic printer is enough. */
public class PrintQueue {
    private final Printer printer;

    public PrintQueue(Printer printer) {
        this.printer = printer;
    }

    public void submit(String doc) {
        printer.printDocument(doc);
    }
}
```

```java
// File: code/lld/ch04/isp/roles/OfficeWorkflow.java
package lld.ch04.isp.roles;

/** One dependency, because one role covers everything it needs. */
public class OfficeWorkflow {
    private final DocumentProcessor processor;

    public OfficeWorkflow(DocumentProcessor processor) {
        this.processor = processor;
    }

    public void run(String doc) {
        processor.scanDocument(doc);
        processor.printDocument(doc);
        processor.stapleDocument(doc);
    }
}
```

```java
// File: code/lld/ch04/isp/Main.java
package lld.ch04.isp;

import java.util.List;
import lld.ch04.isp.roles.BasicPrinter;
import lld.ch04.isp.roles.OfficeDevice;
import lld.ch04.isp.roles.OfficeWorkflow;
import lld.ch04.isp.roles.PrintQueue;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Fat interface ===");
        List<lld.ch04.isp.fat.Printer> printers = List.of(
                new lld.ch04.isp.fat.AdvancedPrinter(), new lld.ch04.isp.fat.BasicPrinter());
        for (var p : printers) {
            p.printDocument("report.pdf");
            try {
                p.scanDocument("report.pdf");
            } catch (UnsupportedOperationException e) {
                System.out.println("  forced to implement: " + e.getMessage());
            }
        }

        System.out.println("=== 2. One interface per method ===");
        new lld.ch04.isp.split.BasicPrinter().printDocument("memo.pdf");
        var adv = new lld.ch04.isp.split.AdvancedPrinter();
        adv.faxDocument("memo.pdf");
        // The same object passed three times -- a sign the split went too far.
        new lld.ch04.isp.split.OfficeWorkflow(adv, adv, adv).run("contract.pdf");

        System.out.println("=== 3. Role interfaces (grouped by caller) ===");
        new PrintQueue(new BasicPrinter()).submit("receipt.pdf");
        new PrintQueue(new OfficeDevice()).submit("receipt.pdf"); // bigger device fits the small role
        new OfficeWorkflow(new OfficeDevice()).run("contract.pdf");
    }
}
```

Output:

```
=== 1. Fat interface ===
AdvancedPrinter printing report.pdf
AdvancedPrinter scanning report.pdf
BasicPrinter printing report.pdf
  forced to implement: BasicPrinter cannot scan
=== 2. One interface per method ===
BasicPrinter printing memo.pdf
AdvancedPrinter faxing memo.pdf
AdvancedPrinter scanning contract.pdf
AdvancedPrinter printing contract.pdf
AdvancedPrinter stapling contract.pdf
=== 3. Role interfaces (grouped by caller) ===
Basic printer printing receipt.pdf
Printing receipt.pdf
Scanning contract.pdf
Printing contract.pdf
Stapling contract.pdf
```

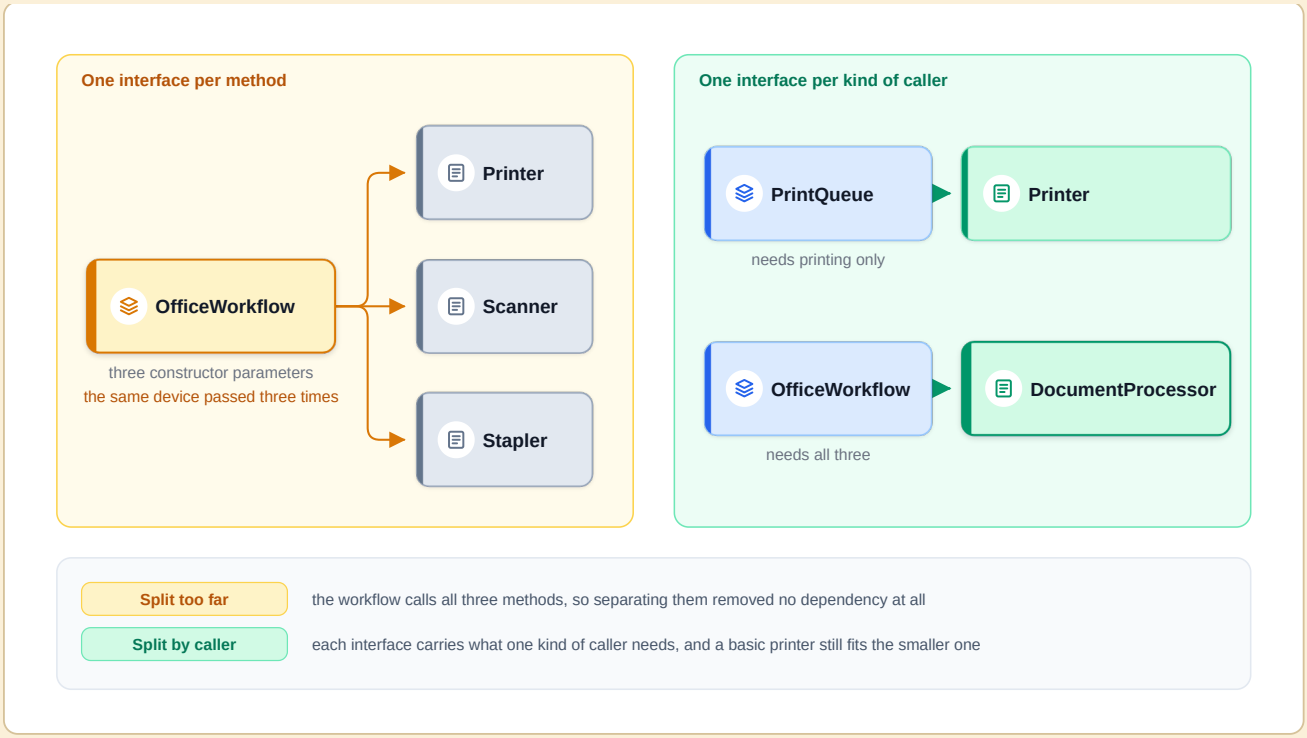
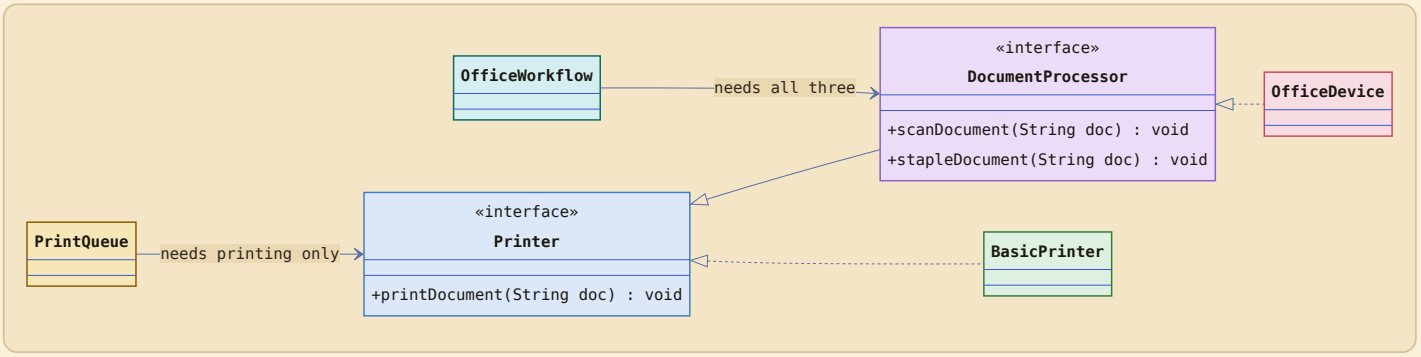


Figure — split too far (three interfaces, the same device passed three times) vs split by caller (one role per kind of caller) (course diagram)



	Four interfaces, one method each	Two role interfaces
Implementer forced to supply what it cannot do?	No	No
Caller depending on unused methods?	No	No
Dependencies for `OfficeWorkflow`	3	1
Names to learn	4	2

Both designs satisfy ISP. The role-based one is leaner because its interfaces were shaped around callers instead of around methods.

Rule of thumb: only split an interface when you can **point to the caller** that needs the smaller piece; without such a caller the split is speculation. The converse also holds: two interfaces that are always implemented by the same classes and always consumed by the same callers should probably be merged.

Languages without an `interface` keyword. Python and JavaScript have no `interface` keyword, but the interface is still there, just invisible: it is **the set of methods a caller actually calls**. Handing a 20-method object to a function that calls one of them creates exactly the same needless dependency, only no compiler points it out. Testing hides it further: in Java a wide interface forces a long stub (a fake class with every method filled in), while in Python you can pass a tiny object with one method and the test passes, which is convenient but masks the problem. So check deliberately: look at what a function calls on its argument and ask whether the argument is far bigger than that. Python's `abc` module and `typing.Protocol` (Python's two ways to declare an interface) let you write the role down when it is worth it; for a role used in only one place, a comment listing the methods the function needs is often enough.

Five signals of an ISP violation

1. Low cohesion in the interface. The methods belong to unrelated groups.

```
interface MediaDevice { void playAudio(); void playVideo(); void recordVideo(); }
// A Speaker can only playAudio(); playVideo() and recordVideo() must throw.
```

2. A fat interface. A long method list that grew one feature at a time. Method names that name a specific mechanism are a tell: some implementer will not use them.

```
interface PaymentProcessor { void processCreditCard(BigDecimal a); void processPayPal(BigDecimal a); void processBankTransfer(BigDecimal a); }
// A shop that only takes PayPal must still supply the other two.
// Better: interface PaymentMethod { void pay(BigDecimal amount); } with one class per method.
```

3. Empty or throwing methods. The clearest signal: the interface asks for something the class cannot provide.

```
interface Vehicle { void drive(); void fly(); void sail(); }
// A Car drives; fly() and sail() throw UnsupportedOperationException.
```

4. One interface serving several kinds of client. It describes a whole category ("everything an employee can do") instead of one caller's needs, so every role implements the union of all roles.

```
interface Employee { void calculateSalary(); void handleCustomerQueries(); void manageTeam(); }
// A support agent handles queries; the other two methods must throw.
```

5. Tests that need long stand-ins. This one appears while writing tests rather than reading code. A fake for a wide interface must implement every method, even if the test exercises just one; dozens of empty method bodies for a single call mean the interface is too wide **for that caller**. What this signal really measures is the distance between what a caller uses and what it is forced to depend on.

Signals 1 and 2 show up when you read the interface, 3 and 4 when you read its implementers, and 5 when you write a test.

Signal	When it shows up	Typical cause
Low cohesion	Reading the interface	Groups of unrelated methods should be split into their own interfaces
Fat interface	Reading the interface	It grew feature by feature and was never split
Empty / throwing methods	Reading an implementer	The implementer was forced to accept what it cannot do
One interface, many roles	Comparing implementers	It describes a category, not a caller's needs
Long test stand-ins	Writing a test	Callers are tied to much more than they actually call

How it works (step by step)

1. For each caller of an interface, list the methods it actually calls.
2. Group callers whose lists are the same; each group is a **role**.
3. Create one interface per role (interfaces may extend each other, like `DocumentProcessor extends Printer`).
4. Let each class implement every role it can honestly fill.
5. Change each caller's parameter type to the narrowest role it needs.

Importance of ISP

- **No bloated classes** forced to implement unneeded methods.
- **Maintainability** — a change to one role does not ripple to callers of another.
- **Flexibility** — small roles are easy to implement in new ways.
- **Simpler tests** — fakes implement one or two methods.
- **Clearer design** — each interface documents one relationship.

How ISP relates to the other principles

- **SRP**: ISP is SRP for interfaces. Each interface has one focused purpose.
- **OCP**: new capabilities arrive as new small interfaces instead of edits to a big one.
- **LSP**: implementers of small interfaces can honour every method, so substitution is safe. A fat interface **forces** LSP violations (the throwing `BasicPrinter`).
- **DIP**: high-level code depends on the smallest abstraction it needs.

Trade-offs

Small role interfaces gain	Costs
No forced, throwing methods	More interface names
Callers insulated from unrelated changes	Over-splitting adds wiring with no benefit
Tiny test doubles; lambdas for one-method roles	Harder to see "everything this device can do" in one place

When not to use it

- When every implementer supports every method and every caller uses them all (the interface is already right-sized).
- When you cannot name the caller that wants the smaller piece.

Common mistakes

- **One interface per method everywhere.** The most common mistake among people who just learned ISP. A fat interface at least makes its cost visible; hundreds of tiny ones cost you slowly, in names and wiring, and are harder to undo.
- Splitting by the implementer's features instead of the caller's needs.
- Keeping the fat interface and adding an "adapter" base class with empty methods (like AWT's `MouseAdapter`; AWT is Java's original desktop user-interface toolkit): convenient, but a symptom that the interface is too wide.

Real-world examples

- **Java's tiny role interfaces**: `Runnable`, `Comparable`, `AutoCloseable`, `Iterable`, `CharSequence`, and the whole `java.util.function` package. `try-with-resources` needs only `AutoCloseable`, not "a file".
- **AWT** `MouseListener` has five methods; most listeners need one, so the JDK added `MouseAdapter` with empty bodies. Later APIs (JavaFX event handlers) use one-method handlers.
- **Collections**: `List.of(...)` and unmodifiable views throw on `add`, because `Collection` bundles reading and writing. Kotlin (another language that runs on the JVM), by contrast, separates `List` (read) and `MutableList` (write).
- **Spring Data** (a Spring library that generates repository code for you): `CrudRepository`, `PagingAndSortingRepository`, and the bare `Repository` marker let you expose only the operations a repository needs.

In the interview

> **30-second answer:** "ISP says no caller should depend on methods it does not use. The symptom is a class implementing an interface method by throwing, or a test fake full of empty methods. I split interfaces by caller: one role interface per kind of caller, not one per method. For printers, a print queue needs `Printer`, an office workflow needs `DocumentProcessor`. I split only when I can name the caller that wants the smaller piece."

Follow-ups:

- *"Difference between SRP and ISP?"* — SRP protects the **class** (one reason to change); ISP protects the **caller** (no unused dependencies). SRP splits classes, ISP splits interfaces; you can break either without the other.
- *"How small is too small?"* — When a caller has to take several interfaces that are always the same object. Always implemented together and always used together means one interface.

Self-check

1. **Q:** What two questions decide if an interface is the right size? **A:** Is any implementer forced to supply something it cannot do? Does any caller depend on methods it never calls?
2. **Q:** Name three signals of an ISP violation. **A:** Throwing/empty methods, a fat interface with mechanism-named methods, long test stand-ins (also: low cohesion, one interface for several roles).
3. **Q:** Why did the four-interface version not help `OfficeWorkflow`? **A:** It uses all three methods, so no dependency was removed; only names and parameters were added.

TL;DR

One interface per kind of caller. Throwing methods and long fakes mean it is too wide; the same object passed three times means it is too thin.

4.10 Dependency Inversion Principle (DIP)

What it is

Robert C. Martin's two statements, paraphrased:

1. **High-level modules should not depend on low-level modules. Both should depend on abstractions.**
2. **Abstractions should not depend on details. Details should depend on abstractions.**

Terms:

- A **high-level module** holds the business policy: *what* the application does (e.g. "notify the user when an order ships").
- A **low-level module** handles a technical detail: *how* it is done (SMTP email, SMTP being the protocol for sending mail; SMS gateway; MySQL).
- An **abstraction** is the interface in between.

Why "inversion"? In traditional layered code, high-level code calls low-level code **and** its source code depends on it (`import com.mail.SmtpClient`). DIP keeps the direction of the call (policy still calls the detail at run time) but **reverses the direction of the source-code dependency**: the detail now imports and implements an interface that belongs to the policy.

> **Analogy: a wall socket.** Your lamp does not wire itself directly into the power station. It plugs into a standard socket. The building's wiring (the detail) was built to the socket standard too. Either side can be replaced without touching the other.

The problem it solves

- Business logic is tied to one vendor or technology; switching (email → SMS) means editing the business code.
- Business logic cannot be tested without the real infrastructure (the servers and services the program runs on: a mail server, a database).
- A change in a low-level library ripples up into the policy.

Example: notifications

Before. `NotificationService` (high level) creates and names `EmailService` (low level):

```java
// File: code/lld/ch04/dip/before/EmailService.java
package lld.ch04.dip.before;

/** Low-level detail. */
public class EmailService {
    public void sendEmail(String message) {
        System.out.println("Sending email: " + message);
    }
}
```

```java
// File: code/lld/ch04/dip/before/NotificationService.java
package lld.ch04.dip.before;

/** VIOLATION: high-level policy creates and names a low-level detail. */
public class NotificationService {
    private final EmailService emailService = new EmailService();

    public void notifyUser(String message) {
        emailService.sendEmail(message);
    }
}
```

It is tightly coupled, and switching to SMS or push means editing `NotificationService`.

After, in four steps.

1. **Introduce an abstraction**, `NotificationSender`.
2. **Low-level modules implement it** (`EmailSender`, `SmsSender`): details depend on the abstraction.
3. **The high-level module depends only on the abstraction**, received through its constructor.
4. **Test it** with different senders, including a recording fake.

Look at the packages. The interface lives in `app`, **next to the policy that uses it**, not in `infra` next to the implementations. That is the part of DIP most people miss: **the high-level side owns the abstraction.** It is shaped by what the policy needs, not by what email happens to offer. `app` imports nothing from `infra`; `infra` imports `app`.

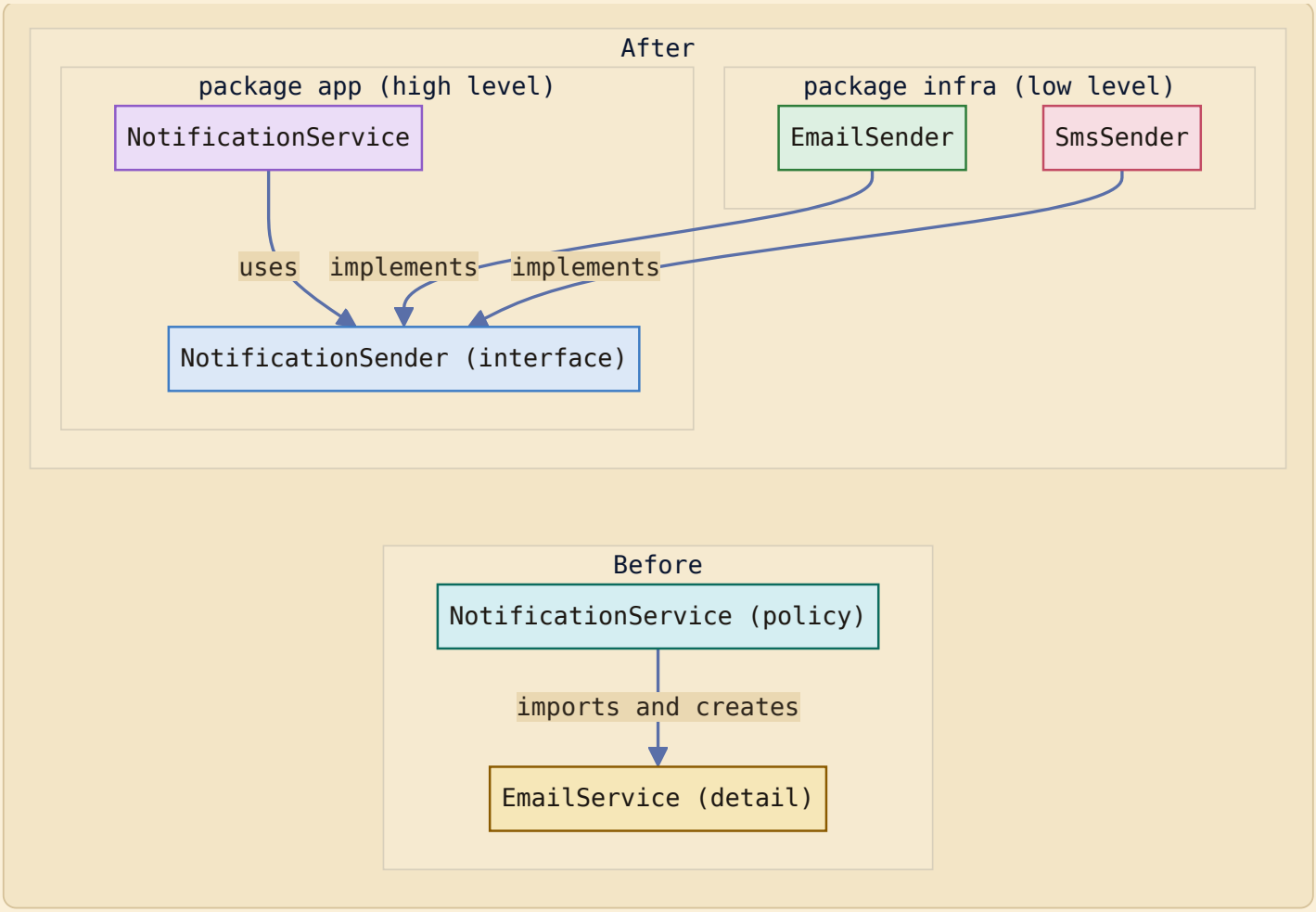
```java
// File: code/lld/ch04/dip/app/NotificationSender.java
package lld.ch04.dip.app;

/**
 * The abstraction. Note the PACKAGE: it lives with the high-level policy (app),
 * not with the implementations (infra). The policy owns the contract.
 */
public interface NotificationSender {
    void send(String recipient, String message);
}
```

```java
// File: code/lld/ch04/dip/app/NotificationService.java
package lld.ch04.dip.app;

import java.util.Objects;

/** High-level policy: depends only on an abstraction it owns. Imports nothing from infra. */
public class NotificationService {
    private final NotificationSender sender;

    public NotificationService(NotificationSender sender) { // constructor injection
        this.sender = Objects.requireNonNull(sender);
    }

    public void notifyUser(String userId, String message) {
        if (message.isBlank()) {
            throw new IllegalArgumentException("empty notification");
        }
        sender.send(userId, message);
    }
}
```

```java
// File: code/lld/ch04/dip/infra/EmailSender.java
package lld.ch04.dip.infra;

import lld.ch04.dip.app.NotificationSender;

/** Detail depends on the abstraction -- the arrow now points UP into app. */
public class EmailSender implements NotificationSender {
    @Override
    public void send(String recipient, String message) {
        System.out.println("[email -> " + recipient + "] " + message);
    }
}
```

```java
// File: code/lld/ch04/dip/infra/SmsSender.java
package lld.ch04.dip.infra;

import lld.ch04.dip.app.NotificationSender;

public class SmsSender implements NotificationSender {
    @Override
    public void send(String recipient, String message) {
        System.out.println("[sms -> " + recipient + "] " + message);
    }
}
```

```java
// File: code/lld/ch04/dip/infra/RecordingSender.java
package lld.ch04.dip.infra;

import java.util.ArrayList;
import java.util.List;
import lld.ch04.dip.app.NotificationSender;

/** A test double: proves NotificationService can be tested with no mail server. */
public class RecordingSender implements NotificationSender {
    private final List<String> sent = new ArrayList<>();

    @Override
    public void send(String recipient, String message) {
        sent.add(recipient + ":" + message);
    }

    public List<String> sent() {
        return List.copyOf(sent);
    }
}
```



In the "After" box, every arrow between packages points **from the detail into the policy**. That is the inversion.

DIP in practice: four real-world techniques

1. Dependency Injection (DI). Give a class its dependencies from outside instead of letting it create them. Three flavours:

Kind	How	Use when
Constructor injection	Pass dependencies to the constructor	Default choice: required dependencies, immutable fields, obvious in the signature
Setter injection	Call `setX(...)` after construction	Optional dependencies with a sensible default
Method (parameter) injection	Pass the dependency to the one method that needs it	It varies per call (e.g. if the format changed on every call, `invoice.generate(generator)` instead of the constructor injection used in 4.7)

A typical web-application case: a `DataService` needs database access, so it receives a `DataRepository` interface in its constructor instead of building a MySQL repository itself; moving to another database then means passing in a different implementation, with no edit to `DataService`.

The class that does all the `new`-ing is called the **composition root**. In this chapter it is each `Main`: the one place that knows the concrete classes.

2. Service Locator. A central registry hands out services by name or type. The high-level class depends on a `Locator` abstraction and asks it for what it needs at run time.

```java
// File: code/lld/ch04/dip/locator/PaymentService.java
package lld.ch04.dip.locator;

import java.math.BigDecimal;

public interface PaymentService {
    void processPayment(BigDecimal amount);
}
```

```java
// File: code/lld/ch04/dip/locator/PayPalService.java
package lld.ch04.dip.locator;

import java.math.BigDecimal;

public class PayPalService implements PaymentService {
    @Override
    public void processPayment(BigDecimal amount) {
        System.out.println("Processing payment via PayPal: $" + amount);
    }
}
```

```java
// File: code/lld/ch04/dip/locator/Locator.java
package lld.ch04.dip.locator;

/** The high-level code depends on this abstraction, not on a concrete registry. */
public interface Locator {
    PaymentService paymentService(String name);
}
```

```java
// File: code/lld/ch04/dip/locator/ServiceLocator.java
package lld.ch04.dip.locator;

import java.util.Map;
import java.util.NoSuchElementException;
import java.util.concurrent.ConcurrentHashMap;

/** A central registry that hands out services by name. */
public class ServiceLocator implements Locator {
    private final Map<String, PaymentService> services = new ConcurrentHashMap<>();

    public void register(String name, PaymentService service) {
        services.put(name, service);
    }

    @Override
    public PaymentService paymentService(String name) {
        PaymentService s = services.get(name);
        if (s == null) {
            throw new NoSuchElementException("No payment service named " + name);
        }
        return s;
    }
}
```

```java
// File: code/lld/ch04/dip/locator/CheckoutFlow.java
package lld.ch04.dip.locator;

import java.math.BigDecimal;

/** Asks the locator at run time. Its real dependency (PaymentService) is hidden inside. */
public class CheckoutFlow {
    private final Locator locator;

    public CheckoutFlow(Locator locator) {
        this.locator = locator;
    }

    public void pay(String serviceName, BigDecimal amount) {
        locator.paymentService(serviceName).processPayment(amount);
    }
}
```

This does follow DIP: `CheckoutFlow` names no concrete payment class, and it receives the `Locator` *interface* through its constructor, so even the registry itself can be replaced (for example by a test locator) without touching `CheckoutFlow`. But be aware of the criticism (Martin Fowler compares the two approaches; Mark Seemann calls Service Locator an anti-pattern, a common solution that causes more problems than it solves): the class's **real** dependencies are hidden inside method bodies, so you cannot see from the constructor what it needs, and a missing registration fails only at run time. Prefer constructor injection; use a locator mainly at framework boundaries.

3. Inversion of Control (IoC) containers. Frameworks such as **Spring** (Java) and the built-in container in **.NET** (Microsoft's application platform) create objects and inject dependencies for you. In Spring, a class with a single constructor gets its dependencies injected automatically (the `@Autowired` annotation, an `@` tag that marks where Spring should inject, is optional for a single constructor since Spring 4.3; a **bean** is an object Spring creates and manages, and the **classpath** is the list of places Java searches for compiled classes and libraries):

```java
// Spring version -- NOT part of this chapter's compiled code (needs Spring on the classpath).
@Service
public class OrderService {
    private final PaymentService payments;          // an interface

    public OrderService(PaymentService payments) {  // Spring finds the one PaymentService bean
        this.payments = payments;
    }

    public void placeOrder(BigDecimal amount) { payments.processPayment(amount); }
}
```

Without Spring, the same wiring is one line in `main`: `new OrderService(new StripePayments())`. **The framework is a convenience for wiring, not the principle.** `OrderService` was already inverted the moment it accepted an abstraction instead of creating a concrete class.

4. Plug-in architecture. The core defines an abstraction; plug-ins implement it and are installed without changing the core. A media player defines `AudioDecoder`; MP3 and FLAC support arrive as plug-ins.

```java
// File: code/lld/ch04/dip/plugin/AudioDecoder.java
package lld.ch04.dip.plugin;

/** The plug-in contract, owned by the core media player. */
public interface AudioDecoder {
    String extension();

    void decode(String fileName);
}
```

```java
// File: code/lld/ch04/dip/plugin/Mp3Decoder.java
package lld.ch04.dip.plugin;

public class Mp3Decoder implements AudioDecoder {
    @Override
    public String extension() {
        return "mp3";
    }

    @Override
    public void decode(String fileName) {
        System.out.println("Decoding MP3 file: " + fileName);
    }
}
```

```java
// File: code/lld/ch04/dip/plugin/FlacDecoder.java
package lld.ch04.dip.plugin;

public class FlacDecoder implements AudioDecoder {
    @Override
    public String extension() {
        return "flac";
    }

    @Override
    public void decode(String fileName) {
        System.out.println("Decoding FLAC file: " + fileName);
    }
}
```

```java
// File: code/lld/ch04/dip/plugin/MediaPlayer.java
package lld.ch04.dip.plugin;

import java.util.HashMap;
import java.util.Map;

/** The core. New formats arrive as plug-ins; this class is never edited for them. */
public class MediaPlayer {
    private final Map<String, AudioDecoder> decoders = new HashMap<>();

    public void install(AudioDecoder decoder) {
        decoders.put(decoder.extension(), decoder);
    }

    public void play(String fileName) {
        String ext = fileName.substring(fileName.lastIndexOf('.') + 1);
        AudioDecoder decoder = decoders.get(ext);
        if (decoder == null) {
            System.out.println("No plug-in installed for ." + ext);
            return;
        }
        decoder.decode(fileName);
        System.out.println("Playing audio...");
    }
}
```

```java
// File: code/lld/ch04/dip/Main.java
package lld.ch04.dip;

import java.math.BigDecimal;
import lld.ch04.dip.app.NotificationService;
import lld.ch04.dip.infra.EmailSender;
import lld.ch04.dip.infra.RecordingSender;
import lld.ch04.dip.infra.SmsSender;
import lld.ch04.dip.locator.CheckoutFlow;
import lld.ch04.dip.locator.PayPalService;
import lld.ch04.dip.locator.ServiceLocator;
import lld.ch04.dip.plugin.FlacDecoder;
import lld.ch04.dip.plugin.MediaPlayer;
import lld.ch04.dip.plugin.Mp3Decoder;

/** This class is the COMPOSITION ROOT: the one place that knows the concrete classes. */
public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Before: NotificationService builds EmailService itself ===");
        new lld.ch04.dip.before.NotificationService().notifyUser("Your order shipped");

        System.out.println("=== 2. After: inject any NotificationSender ===");
        new NotificationService(new EmailSender()).notifyUser("u-42", "Your order shipped");
        new NotificationService(new SmsSender()).notifyUser("u-42", "Your order shipped");
        var recorder = new RecordingSender();
        new NotificationService(recorder).notifyUser("u-7", "Test message");
        System.out.println("test double captured: " + recorder.sent());

        System.out.println("=== 3. Service locator ===");
        var locator = new ServiceLocator();
        locator.register("PayPal", new PayPalService());
        new CheckoutFlow(locator).pay("PayPal", new BigDecimal("100.00"));

        System.out.println("=== 4. Plug-in architecture ===");
        var player = new MediaPlayer();
        player.install(new Mp3Decoder());
        player.install(new FlacDecoder());
        player.play("song.mp3");
        player.play("concert.flac");
        player.play("voice.ogg");
    }
}
```

Output:

```
=== 1. Before: NotificationService builds EmailService itself ===
Sending email: Your order shipped
=== 2. After: inject any NotificationSender ===
[email -> u-42] Your order shipped
[sms -> u-42] Your order shipped
test double captured: [u-7:Test message]
=== 3. Service locator ===
Processing payment via PayPal: $100.00
=== 4. Plug-in architecture ===
Decoding MP3 file: song.mp3
Playing audio...
Decoding FLAC file: concert.flac
Playing audio...
No plug-in installed for .ogg
```

DI vs plug-in architecture. At first sight they are the same thing (an implementation of an interface is handed to a core), but they work at different scales. **DI** is a small-scale mechanism: how one class obtains its collaborators. **Plug-in architecture** is a system-level design for extensibility: whole modules, often built by other teams, are added independently and frequently discovered while the program runs (Java's `ServiceLoader`, OSGi, a Java module system built for plug-ins, IDE plug-in APIs).

DIP vs DI vs IoC — do not mix them up

Term	What it is	Kind	Example
Dependency Inversion (DIP)	Depend on abstractions; the policy owns the abstraction	A **principle** (what good structure looks like)	`NotificationService` depends on `NotificationSender`
Dependency Injection (DI)	Pass dependencies in from outside instead of creating them	A **technique** (one way to achieve DIP)	`new NotificationService(new SmsSender())`
Inversion of Control (IoC)	The framework calls your code, not the other way round ("don't call us, we'll call you" — the Hollywood Principle)	A broad **idea**; DI is one form of it	Spring creating beans; a servlet container (a Java web server such as Tomcat) calling `doGet`; JUnit calling your `@Test` methods
IoC container	A framework that performs DI for you	A **tool**	Spring `ApplicationContext`, Google Guice (a lightweight DI library)

Two combinations prove they are different:

- **Injection without inversion**: `new OrderService(new MySqlDatabase())` where the parameter type is the concrete `MySqlDatabase`. It is injected, but the policy still depends on a detail.
- **Inversion without a framework**: everything in this chapter. Constructor + interface + a `main` that wires it. No container needed.

Stable vs volatile dependencies

DIP targets **volatile** dependencies: things that change often or that you want to fake in tests (databases, networks, clocks, third-party APIs, i.e. services run by other companies that your code calls, business rules that vary). Depending directly on **stable** concrete classes is fine: `String`, `List`, `LocalDate`, `Math`. Nobody wraps `String` in an interface. (Head First's version of DIP gives guidelines like "no variable should hold a reference to a concrete class", and then admits they are ideals you will break; the stable/volatile distinction is how you decide when.)

Importance of DIP

- **Flexibility** — swap implementations without touching business logic.
- **Maintainability** — detail changes stay in the detail.
- **Testability** — pass fakes and mocks (fakes that also record how they were called, so a test can check it); no real database or mail server in unit tests.

How DIP relates to the other principles

- **SRP**: injected collaborators keep each class focused on its own job; creating and configuring dependencies becomes the composition root's job.
- **OCP**: high-level code depends on abstractions, so new implementations extend it without edits.
- **LSP**: all implementations of the abstraction must be proper substitutes, or injection becomes dangerous.
- **ISP**: the abstraction the high-level module owns should be the **smallest** role it needs.

How it works (step by step)

1. Find a high-level class that names or creates a volatile low-level class.
2. Write down what the high-level class **needs** from it, in its own words (`send(recipient, message)`).
3. Make that an interface **in the high-level package**.
4. Make the low-level class implement it.
5. Inject the implementation (constructor by default).
6. Move all `new` calls for concrete classes to the composition root (`main`, a factory, or a DI container).

Trade-offs

You gain	You pay
Policy independent of technology	An interface per volatile dependency
Easy unit tests with fakes	Wiring code (or a framework and its learning curve)
Swappable implementations	Indirection: "go to implementation" takes an extra hop

When not to use it

- For stable classes (`String`, collections, value objects).
- For an interface that would have exactly one implementation forever and no test double.
- In tiny scripts.

Common mistakes

- Putting the interface in the **low-level** package (e.g. `mysql.MySqlRepository` and `mysql.Repository`). The policy then still depends on the detail's package.
- Designing the interface around one vendor's API (`sendSmtpMessage(...)`) instead of the policy's need (`send(...)`). Changing vendors then changes the interface too.
- Breaking the **second** clause ("abstractions should not depend on details"): an interface whose methods return `java.sql.ResultSet` or throw `javax.mail.MessagingException` is technically an abstraction, but it drags the detail's types into the policy. Use the policy's own types (`Optional<Order>`, a box that may or may not hold an order; a domain exception, an error type named in business terms) instead.
- Believing "we use Spring, so we follow DIP". Spring can inject concrete classes just as happily.
- Calling `new ConcreteThing()` inside a class "just this once".

Real-world examples

- **JDBC**: your code depends on `java.sql.Connection` / `DataSource` interfaces; drivers from MySQL, PostgreSQL, Oracle implement them. `DriverManager` finds drivers through `ServiceLoader`.
- **SLF4J**: the application depends on the logging facade; Logback or Log4j 2 is the plugged-in detail.
- **Spring** and **Jakarta EE CDI** (the enterprise-Java standard's built-in injection): IoC containers doing constructor injection.
- **JPA** (Java's standard API for saving objects to SQL databases): code uses `EntityManager`; Hibernate or EclipseLink implements it.
- **Hexagonal architecture (ports and adapters)**: the core defines *ports* (interfaces); *adapters* for HTTP, databases and queues (message queues, see System Design Fundamentals) implement them. This is DIP applied to a whole service.

In the interview

30-second answer: "DIP says business logic should not depend on technical details; both depend on an abstraction, and the business side owns that abstraction. So `NotificationService` depends on a `NotificationSender` interface in its own package, and email and SMS senders implement it. I inject the implementation through the constructor and wire everything in `main`. That makes the service testable with a fake and lets me swap providers. DI is the technique; DIP is the principle; Spring is just a tool that does the wiring."

Follow-ups:

- *"Difference between dependency inversion and dependency injection?"* — Inversion is the principle (depend on abstractions); injection is one technique (pass dependencies in). You can inject a concrete class, which is injection without inversion.
- *"Who should own the interface?"* — The high-level module that uses it.
- *"Is Service Locator OK?"* — It satisfies DIP but hides dependencies and fails late. I prefer constructor injection.

Self-check

1. **Q:** What is "inverted" in DIP? **A:** The source-code dependency: the detail depends on (implements) an abstraction owned by the policy, instead of the policy importing the detail.
2. **Q:** Is `new OrderService(new MySqlDatabase())` with a `MySqlDatabase` parameter DIP? **A:** No. It is injection of a concrete class; the policy still depends on the detail.
3. **Q:** What is a composition root? **A:** The one place (usually `main` or the container config) where concrete classes are created and wired.
4. **Q:** Should you put `String` behind an interface? **A:** No. It is stable; DIP is for volatile dependencies.

TL;DR

Business code depends on interfaces it owns; details implement them; `main` (or a container) wires them. DI is the how, DIP is the why, IoC containers are the tool.

4.11 Refactoring one class through all five principles

In real code the principles seldom show up one at a time; they tend to appear **in a fixed sequence**, each exposing the need for the next. This section refactors a single order-checkout class five times. The class checks the order, prices it, saves it and emails the customer, the sort of class that has been growing unnoticed for a couple of years.

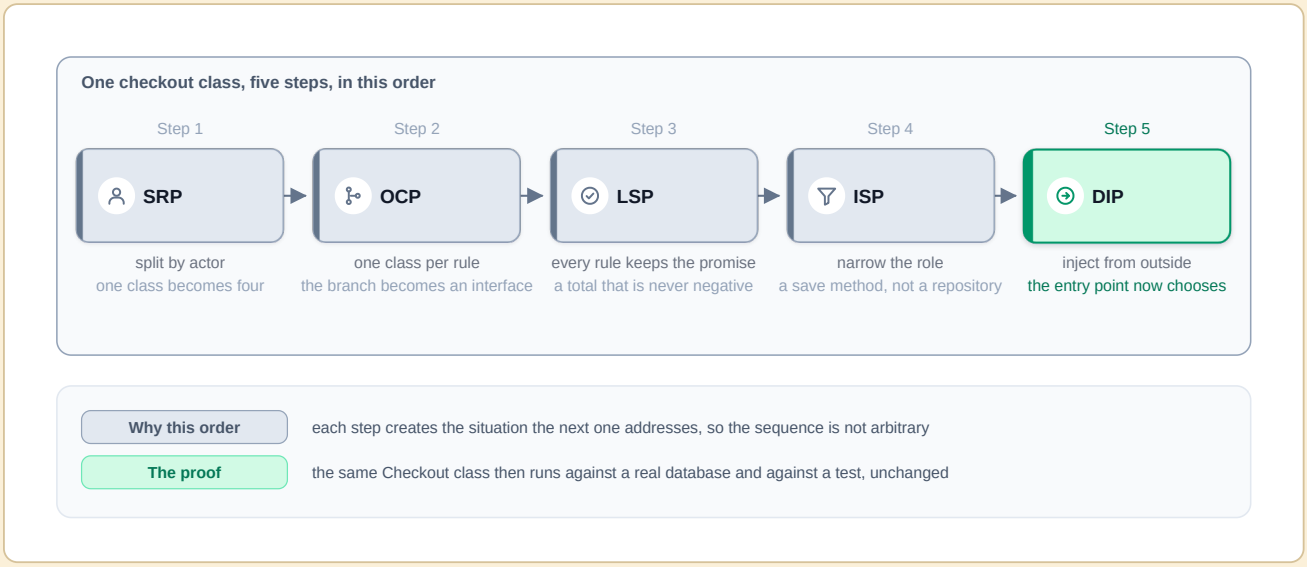


Figure — one checkout class, five steps: SRP, OCP, LSP, ISP, DIP, each creating the situation the next addresses (course diagram)

Step 0: the starting class

The code runs correctly, and yet each of the five principles finds something to object to.

```java
// File: code/lld/ch04/checkout/step0/CustomerOrder.java
package lld.ch04.checkout.step0;

import java.math.BigDecimal;

/** The original order shape: the customer type travels with the order as a string. */
public record CustomerOrder(String orderId, String customerType, BigDecimal amount, String email) { }
```

```java
// File: code/lld/ch04/checkout/step0/OrderService.java
package lld.ch04.checkout.step0;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** STEP 0 -- the class that grew quietly for two years. It works. */
public class OrderService {
    public void process(CustomerOrder order) {
        // 1. Validation  (business-rules owner)
        if (order.amount().signum() <= 0) {
            System.out.println("Rejected: amount must be positive");
            return;
        }
        // 2. Pricing, one branch per customer type  (finance)
        BigDecimal total = order.amount();
        if (order.customerType().equals("member")) {
            total = total.multiply(new BigDecimal("0.90"));
        } else if (order.customerType().equals("staff")) {
            total = total.multiply(new BigDecimal("0.70"));
        }
        total = total.setScale(2, RoundingMode.HALF_EVEN); // money: always 2 decimals
        // 3. Storage  (database team)
        System.out.println("INSERT INTO orders VALUES ('" + order.orderId() + "', " + total + ")");
        // 4. Telling the customer  (marketing)
        System.out.println("Emailing " + order.email() + ": your order total is " + total);
    }
}
```

Step 1: Single Responsibility — split by actor

List everyone who might request a change to `process`:

Section of `process`	Requested by
Validation	The business-rules owner
Pricing	Finance
The `INSERT` statement	The database team
The email wording	Marketing

One method answers to four different groups, so rewording the email forces a redeploy of the pricing logic. Split it by actor into `OrderValidator`, `PriceCalculator`, `OrderRepository` and `Notifier`:

```java
// File: code/lld/ch04/checkout/step1/OrderValidator.java
package lld.ch04.checkout.step1;

import lld.ch04.checkout.step0.CustomerOrder;

/** Actor: business-rules owner. */
public class OrderValidator {
    public boolean isValid(CustomerOrder order) {
        return order.amount().signum() > 0;
    }
}
```

```java
// File: code/lld/ch04/checkout/step1/PriceCalculator.java
package lld.ch04.checkout.step1;

import java.math.BigDecimal;
import java.math.RoundingMode;
import lld.ch04.checkout.step0.CustomerOrder;

/** Actor: finance. SRP is satisfied, but the branch is still here -> OCP problem next. */
public class PriceCalculator {
    public BigDecimal total(CustomerOrder order) {
        BigDecimal total = switch (order.customerType()) {
            case "member" -> order.amount().multiply(new BigDecimal("0.90"));
            case "staff" -> order.amount().multiply(new BigDecimal("0.70"));
            default -> order.amount();
        };
        return total.setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/step1/OrderRepository.java
package lld.ch04.checkout.step1;

import java.math.BigDecimal;
import lld.ch04.checkout.step0.CustomerOrder;

/** Actor: database team. */
public class OrderRepository {
    public void save(CustomerOrder order, BigDecimal total) {
        System.out.println("INSERT INTO orders VALUES ('" + order.orderId() + "', " + total + ")");
    }
}
```

```java
// File: code/lld/ch04/checkout/step1/Notifier.java
package lld.ch04.checkout.step1;

import java.math.BigDecimal;
import lld.ch04.checkout.step0.CustomerOrder;

/** Actor: marketing (owns the wording). */
public class Notifier {
    public void send(CustomerOrder order, BigDecimal total) {
        System.out.println("Emailing " + order.email() + ": your order total is " + total);
    }
}
```

```java
// File: code/lld/ch04/checkout/step1/Checkout.java
package lld.ch04.checkout.step1;

import java.math.BigDecimal;
import lld.ch04.checkout.step0.CustomerOrder;

/** STEP 1 (SRP): coordinates four single-actor classes. Still news them up itself. */
public class Checkout {
    private final OrderValidator validator = new OrderValidator();
    private final PriceCalculator calculator = new PriceCalculator();
    private final OrderRepository repository = new OrderRepository();
    private final Notifier notifier = new Notifier();

    public void process(CustomerOrder order) {
        if (!validator.isValid(order)) {
            System.out.println("Rejected: amount must be positive");
            return;
        }
        BigDecimal total = calculator.total(order);
        repository.save(order, total);
        notifier.send(order, total);
    }
}
```

Step 2: Open/Closed — one class per pricing rule

Inside `PriceCalculator` the branch on customer type survived the split, so a new "student" rate would still mean editing working code. Give each rate its own class behind a `PricingRule` interface. From here on the order stops carrying a type string; whoever calls the checkout picks the rule:

```java
// File: code/lld/ch04/checkout/Order.java
package lld.ch04.checkout;

import java.math.BigDecimal;

/** From step 2 on, the order no longer carries a customer-type string: the rule is chosen outside. */
public record Order(String orderId, BigDecimal amount, String email) { }
```

```java
// File: code/lld/ch04/checkout/pricing/PricingRule.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;

/**
 * STEP 2 (OCP): one class per rate; a new rate is a new class.
 * CONTRACT every implementation must keep (STEP 3, LSP):
 *   precondition : amount >= 0
 *   postcondition: result >= 0, rounded to 2 decimals   (a total is never negative)
 */
@FunctionalInterface
public interface PricingRule {
    BigDecimal apply(BigDecimal amount);
}
```

```java
// File: code/lld/ch04/checkout/pricing/RegularPrice.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class RegularPrice implements PricingRule {
    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/pricing/MemberDiscount.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class MemberDiscount implements PricingRule {
    private static final BigDecimal RATE = new BigDecimal("0.90");

    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.multiply(RATE).setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/pricing/StaffDiscount.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class StaffDiscount implements PricingRule {
    private static final BigDecimal RATE = new BigDecimal("0.70");

    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.multiply(RATE).setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/step2/PriceCalculator.java
package lld.ch04.checkout.step2;

import java.math.BigDecimal;
import lld.ch04.checkout.Order;
import lld.ch04.checkout.pricing.PricingRule;

/** STEP 2: no branch left. The calculator never learns how many rates exist. */
public class PriceCalculator {
    public BigDecimal total(Order order, PricingRule rule) {
        return rule.apply(order.amount());
    }
}
```

A student rate is now one new class and no edits.

Step 3: Liskov Substitution — every rule keeps the promise

Once one interface has several implementations, you have to ask whether each of them can really stand in for the others. Finance now wants a **fixed** discount of 20 rather than a percentage. A naive version turns a 10 order into a total of **−10**, with no exception at all: the negative total is saved, and later the customer may be credited

money they never paid. This is the postcondition/invariant rule from 4.8: every pricing rule must guarantee **a total that is never negative**, and a rule that does not is not a valid substitute. The fix is to state the promise explicitly (it is in the `PricingRule` Javadoc), make every rule honour it, and verify that with one contract test applied to every implementation:

```java
// File: code/lld/ch04/checkout/pricing/CarelessFixedDiscount.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** LSP VIOLATION kept for the demo: 20 off a 10 order gives -10. Nothing throws. */
public class CarelessFixedDiscount implements PricingRule {
    private final BigDecimal off;

    public CarelessFixedDiscount(BigDecimal off) {
        this.off = off;
    }

    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.subtract(off).setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/pricing/FixedDiscount.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** STEP 3 fix: keeps the promise that a total is never negative. */
public class FixedDiscount implements PricingRule {
    private final BigDecimal off;

    public FixedDiscount(BigDecimal off) {
        if (off.signum() < 0) {
            throw new IllegalArgumentException("discount must be >= 0");
        }
        this.off = off;
    }

    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.subtract(off).max(BigDecimal.ZERO).setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/step3/PricingRuleContract.java
package lld.ch04.checkout.step3;

import java.math.BigDecimal;
import java.util.List;
import lld.ch04.checkout.pricing.PricingRule;

/** STEP 3: ONE test written against the interface, run for EVERY implementation. */
public final class PricingRuleContract {
    private static final List<BigDecimal> SAMPLE_AMOUNTS = List.of(
            new BigDecimal("0.00"), new BigDecimal("10.00"),
            new BigDecimal("100.00"), new BigDecimal("250.00"));

    private PricingRuleContract() { }

    public static boolean holds(PricingRule rule) {
        for (BigDecimal amount : SAMPLE_AMOUNTS) {
            BigDecimal total = rule.apply(amount);
            if (total.signum() < 0) {
                System.out.println("  " + rule.getClass().getSimpleName()
                        + " broke the contract: apply(" + amount + ") = " + total);
                return false;
            }
        }
        System.out.println("  " + rule.getClass().getSimpleName() + " keeps the contract");
        return true;
    }
}
```

Step 4: Interface Segregation — the checkout needs a save method, not a repository

Repositories tend to accumulate methods (`save`, `findById`, `delete`, `listAll`) as other parts of the system ask for them. The checkout, however, only ever **saves**. If it is typed against the full repository, each new query added for reporting becomes one more method it depends on without calling. So describe the **role** the checkout needs instead of the class it is handed: an order saver, exactly one method wide. The real repository is free to play that role and others:

```java
// File: code/lld/ch04/checkout/fin/OrderSaver.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import lld.ch04.checkout.Order;

/** STEP 4 (ISP): the role the checkout needs -- one method wide, owned by the checkout. */
@FunctionalInterface
public interface OrderSaver {
    void save(Order order, BigDecimal total);
}
```

```java
// File: code/lld/ch04/checkout/fin/OrderFinder.java
package lld.ch04.checkout.fin;

import java.util.List;
import java.util.Optional;
import lld.ch04.checkout.Order;

/** A different role, for reporting callers. The checkout never sees it. */
public interface OrderFinder {
    Optional<Order> findById(String orderId);

    List<Order> listAll();
}
```

```java
// File: code/lld/ch04/checkout/fin/OrderNotifier.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import lld.ch04.checkout.Order;

@FunctionalInterface
public interface OrderNotifier {
    void send(Order order, BigDecimal total);
}
```

```java
// File: code/lld/ch04/checkout/fin/SqlOrderRepository.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import lld.ch04.checkout.Order;

/** The full repository grows (save, find, list...) but plays two separate roles. */
public class SqlOrderRepository implements OrderSaver, OrderFinder {
    private final List<Order> table = new ArrayList<>(); // stands in for a real DB table

    @Override
    public void save(Order order, BigDecimal total) {
        table.add(order);
        System.out.println("  INSERT INTO orders VALUES ('" + order.orderId() + "', " + total + ")");
    }

    @Override
    public Optional<Order> findById(String orderId) {
        return table.stream().filter(o -> o.orderId().equals(orderId)).findFirst();
    }

    @Override
    public List<Order> listAll() {
        return List.copyOf(table);
    }
}
```

Step 5: Dependency Inversion — the entry point decides

A last problem remains: the step-1 `Checkout` constructs its collaborators itself, so it refers to concrete classes (a SQL repository, an email notifier) and cannot run in a test without a database and a mail server. Instead, have the abstractions passed in. The checkout then mentions no concrete class, and the program's entry point chooses the implementations:

```java
// File: code/lld/ch04/checkout/fin/InMemoryOrders.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import lld.ch04.checkout.Order;

/** Test double: one method to implement, because the role is one method wide. */
public class InMemoryOrders implements OrderSaver {
    private final List<String> saved = new ArrayList<>();

    @Override
    public void save(Order order, BigDecimal total) {
        saved.add(order.orderId() + "=" + total);
        System.out.println("  kept " + order.orderId() + " in memory");
    }

    public List<String> saved() {
        return List.copyOf(saved);
    }
}
```

```java
// File: code/lld/ch04/checkout/fin/EmailNotifier.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import lld.ch04.checkout.Order;

public class EmailNotifier implements OrderNotifier {
    @Override
    public void send(Order order, BigDecimal total) {
        System.out.println("  emailing " + order.email() + ": total " + total);
    }
}
```

```java
// File: code/lld/ch04/checkout/fin/SilentNotifier.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import lld.ch04.checkout.Order;

public class SilentNotifier implements OrderNotifier {
    @Override
    public void send(Order order, BigDecimal total) {
        System.out.println("  (test run, no email sent)");
    }
}
```

```java
// File: code/lld/ch04/checkout/fin/Checkout.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import java.util.Objects;
import lld.ch04.checkout.Order;
import lld.ch04.checkout.pricing.PricingRule;

/**
 * STEP 5 (DIP): every volatile collaborator arrives from outside.
 * No concrete database, mail or pricing class is named here.
 */
public class Checkout {
    private final PricingRule rule;
    private final OrderSaver orders;
    private final OrderNotifier notifier;

    public Checkout(PricingRule rule, OrderSaver orders, OrderNotifier notifier) {
        this.rule = Objects.requireNonNull(rule);
        this.orders = Objects.requireNonNull(orders);
        this.notifier = Objects.requireNonNull(notifier);
    }

    public boolean process(Order order) {
        // Validation has not varied yet, so it stays a plain check (see "What we did not do").
        if (order.amount().signum() <= 0) {
            System.out.println("  rejected: amount must be positive");
            return false;
        }
        BigDecimal total = rule.apply(order.amount());
        orders.save(order, total);
        notifier.send(order, total);
        return true;
    }
}
```

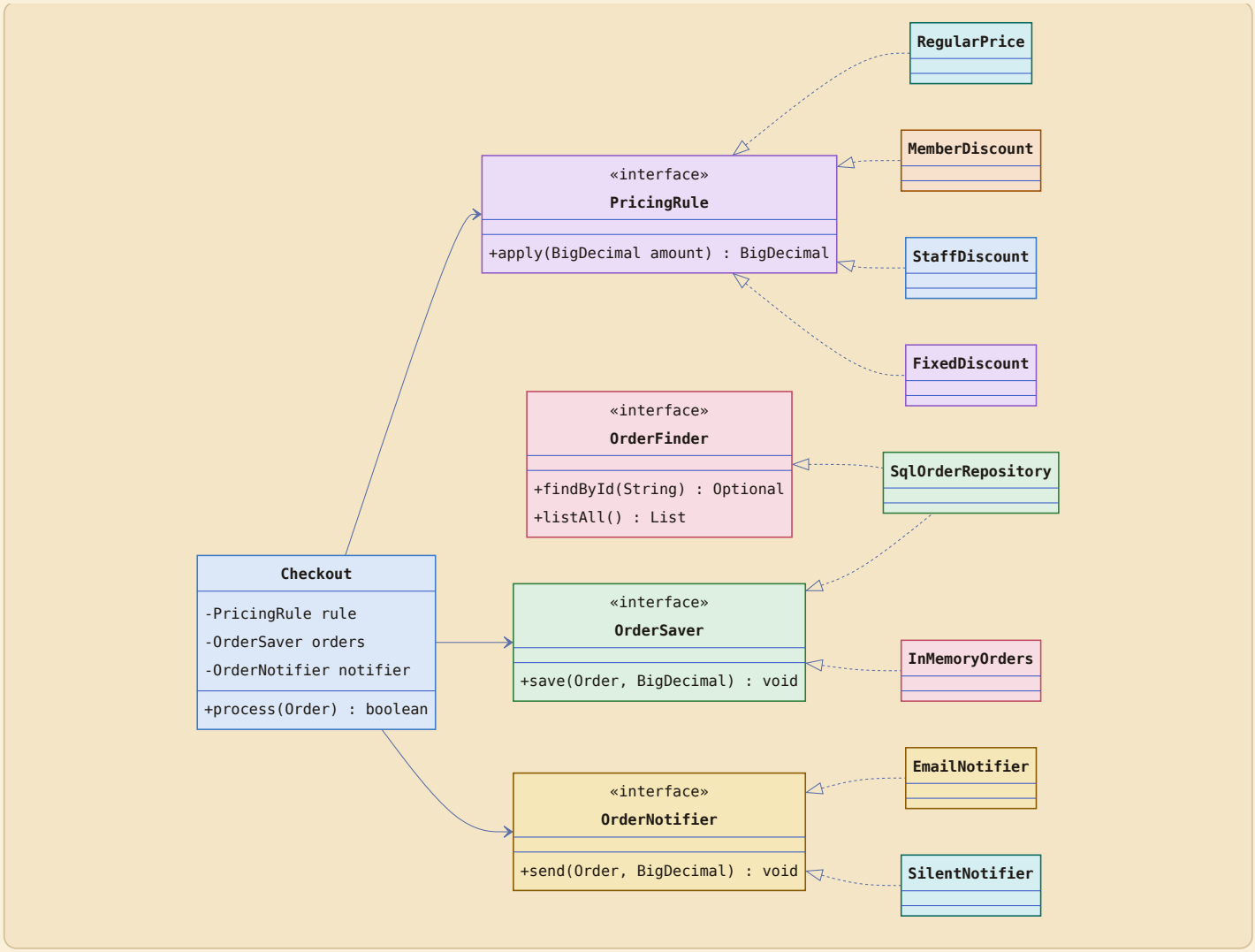
```java
// File: code/lld/ch04/checkout/Main.java
package lld.ch04.checkout;

import java.math.BigDecimal;
import java.util.List;
import lld.ch04.checkout.fin.Checkout;
import lld.ch04.checkout.fin.EmailNotifier;
import lld.ch04.checkout.fin.InMemoryOrders;
import lld.ch04.checkout.fin.SilentNotifier;
import lld.ch04.checkout.fin.SqlOrderRepository;
import lld.ch04.checkout.pricing.CarelessFixedDiscount;
import lld.ch04.checkout.pricing.FixedDiscount;
import lld.ch04.checkout.pricing.MemberDiscount;
import lld.ch04.checkout.pricing.PricingRule;
import lld.ch04.checkout.pricing.RegularPrice;
import lld.ch04.checkout.pricing.StaffDiscount;
import lld.ch04.checkout.step0.CustomerOrder;
import lld.ch04.checkout.step0.OrderService;
import lld.ch04.checkout.step2.PriceCalculator;
import lld.ch04.checkout.step3.PricingRuleContract;

public class Main {
    private static final BigDecimal TWENTY = new BigDecimal("20.00");

    public static void main(String[] args) {
        System.out.println("=== Step 0: the original class ===");
        var legacy = new OrderService();
        legacy.process(new CustomerOrder("A-1", "regular", new BigDecimal("100.00"), "ada@example.com"));
        legacy.process(new CustomerOrder("A-2", "staff", new BigDecimal("100.00"), "grace@example.com"));

        System.out.println("=== Step 1: SRP -- split by actor (same output) ===");
        new lld.ch04.checkout.step1.Checkout()
                .process(new CustomerOrder("A-2", "staff", new BigDecimal("100.00"), "grace@example.com"));

        System.out.println("=== Step 2: OCP -- one class per pricing rule ===");
        var calc = new PriceCalculator();
        var order = new Order("A-3", new BigDecimal("100.00"), "ada@example.com");
        for (PricingRule r : List.of(new RegularPrice(), new MemberDiscount(), new StaffDiscount())) {
            System.out.println("  " + r.getClass().getSimpleName() + " -> " + calc.total(order, r));
        }

        System.out.println("=== Step 3: LSP -- every rule must keep 'total >= 0' ===");
        var small = new Order("A-4", new BigDecimal("10.00"), "ada@example.com");
        System.out.println("  careless FixedDiscount(20) on 10 -> "
                + calc.total(small, new CarelessFixedDiscount(TWENTY)));
        System.out.println("  correct  FixedDiscount(20) on 10 -> "
                + calc.total(small, new FixedDiscount(TWENTY)));
        List<PricingRule> all = List.of(new RegularPrice(), new MemberDiscount(),
                new StaffDiscount(), new FixedDiscount(TWENTY), new CarelessFixedDiscount(TWENTY));
        long passing = all.stream().filter(PricingRuleContract::holds).count();
        System.out.println("  " + passing + "/" + all.size() + " rules are safe substitutes");

        System.out.println("=== Steps 4+5: ISP + DIP -- same Checkout, two wirings ===");
        var a9 = new Order("A-9", new BigDecimal("200.00"), "ada@example.com");
        System.out.println("Production wiring:");
        var db = new SqlOrderRepository();
        new Checkout(new MemberDiscount(), db, new EmailNotifier()).process(a9);
        System.out.println("  reporting side can still find it: " + db.findById("A-9").isPresent());

        System.out.println("Test wiring, same Checkout class:");
        var fake = new InMemoryOrders();
        new Checkout(new MemberDiscount(), fake, new SilentNotifier()).process(a9);
        System.out.println("  test double recorded " + fake.saved());

        System.out.println("Lambda wiring (roles are functional interfaces):");
        new Checkout(amount -> amount, (o, t) -> System.out.println("  saved " + o.orderId()),
                (o, t) -> System.out.println("  notified " + o.email())).process(a9);
        new Checkout(new RegularPrice(), fake, new SilentNotifier())
```

```
                .process(new Order("A-10", BigDecimal.ZERO, "x@example.com"));
    }
}
```

Output:

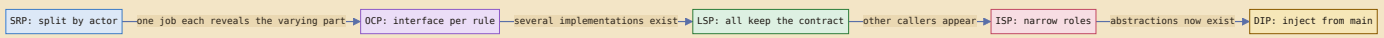
```
=== Step 0: the original class ===
INSERT INTO orders VALUES ('A-1', 100.00)
Emailing ada@example.com: your order total is 100.00
INSERT INTO orders VALUES ('A-2', 70.00)
Emailing grace@example.com: your order total is 70.00
=== Step 1: SRP -- split by actor (same output) ===
INSERT INTO orders VALUES ('A-2', 70.00)
Emailing grace@example.com: your order total is 70.00
=== Step 2: OCP -- one class per pricing rule ===
  RegularPrice -> 100.00
  MemberDiscount -> 90.00
  StaffDiscount -> 70.00
=== Step 3: LSP -- every rule must keep 'total >= 0' ===
  careless FixedDiscount(20) on 10 -> -10.00
  correct  FixedDiscount(20) on 10 -> 0.00
  RegularPrice keeps the contract
  MemberDiscount keeps the contract
  StaffDiscount keeps the contract
  FixedDiscount keeps the contract
  CarelessFixedDiscount broke the contract: apply(0.00) = -20.00
  4/5 rules are safe substitutes
=== Steps 4+5: ISP + DIP -- same Checkout, two wirings ===
Production wiring:
  INSERT INTO orders VALUES ('A-9', 180.00)
  emailing ada@example.com: total 180.00
  reporting side can still find it: true
Test wiring, same Checkout class:
  kept A-9 in memory
  (test run, no email sent)
  test double recorded [A-9=180.00]
Lambda wiring (roles are functional interfaces):
  saved A-9
  notified ada@example.com
  rejected: amount must be positive
```

One unmodified `Checkout` class ran against production collaborators, against test doubles and even against lambdas, **without being edited or subclassed**. That is what the five steps bought.



Reading the five steps back

Step	The question it asked	What changed
SRP	Who might request changes here?	The single class split into four, one per actor
OCP	What breaks when a new case shows up?	Each pricing branch turned into its own class
LSP	Does each rule work when called through the interface?	All rules guarantee the total never drops below zero
ISP	Is the caller tied to more than it calls?	Checkout depends on one save method rather than a whole repository
DIP	Who decides which concrete class is used?	The entry point decides, not the checkout



The sequence has a reason. SRP goes first because the point where behaviour varies stays hidden until each class has a single job. OCP then introduces an interface. LSP becomes relevant as soon as that interface has a second implementation. ISP becomes relevant when new callers (reporting) show up. DIP is last because inversion needs abstractions to point at. Every step sets up the problem the next step solves, which is why the five are normally learned as a group.

What we did not do

- **Not every principle was applied to every class.** `Order` remains a simple record without an interface, because no requirement has made it vary.
- **Not every class was closed against every change.** Adding a validation rule still means editing `Checkout`. That is fine while validation stays stable; the day a third validation rule is requested is the day validation

deserves the treatment pricing received.

- **Not everything was injected.** `Order` and the `BigDecimal` arithmetic are stable, so they are used directly.

Pre-emptively applying all five principles to every class turns a small, clear program into a confusing one. **Each step above was triggered by an actual change request, never by a checklist.**

Self-check

1. **Q:** Why does SRP come before OCP here? **A:** Until pricing lived in its own class, the varying part (the branch on customer type) was buried in a method that did four things.
2. **Q:** What contract did step 3 add? **A:** Every `PricingRule` returns a total ≥ 0 for any amount ≥ 0.
3. **Q:** Why is `OrderSaver` one method wide? **A:** The checkout only saves; depending on the full repository would tie it to reporting methods it never calls.
4. **Q:** What proves the design worked? **A:** One `Checkout` class ran with three different wirings (production, test doubles, lambdas) without being edited.

TL;DR

SRP → OCP → LSP → ISP → DIP, each triggered by a real change. Proof: one class, several wirings, zero edits.

4.12 SOLID and design patterns

Most classic design patterns (Chapters 8–11) exist because of these principles. Learning the principles first makes the patterns look inevitable rather than arbitrary: **the principles say what good structure looks like; the patterns are named, reusable ways to produce it.**

Pattern	Principle(s) it serves	How
Strategy	OCP, DIP	A varying algorithm behind an interface (`InvoiceGenerator`, `PricingRule`)
Template Method	OCP	A fixed skeleton in a base class; subclasses fill in hook steps
Decorator	OCP, SRP	Adds behaviour by wrapping, without modifying the original class
Adapter	DIP, ISP	Lets a caller keep depending on the interface it already owns while a foreign class is plugged in
Factory Method / Abstract Factory	DIP, OCP	Keeps callers away from concrete constructors; the `switch` lives in one place
Observer	OCP, DIP	New reactions are new subscribers; the subject depends only on a listener interface
Composite	LSP, OCP	Leaves and groups share one interface, so callers treat them the same
Proxy	SRP, OCP	Adds access control, caching or laziness (creating things only when first needed) in a separate class
Command	SRP, OCP	Each request is its own object; new commands are new classes
Visitor	OCP (for operations)	Easy to add operations, hard to add element types: OCP along a different axis
Null Object	LSP	A do-nothing implementation that still keeps the contract, instead of `null` checks

Head First Design Patterns states several of the same ideas as its "OO design principles": **encapsulate what varies**, **favour composition over inheritance**, **program to an interface, not an implementation**, **classes should be open for extension but closed for modification** (its Decorator chapter), **depend upon abstractions, do not depend upon concrete classes** (its Factory chapter's name for DIP), and **a class should have only one reason to change** (its Iterator/Composite chapter). *Clean Code* covers SRP and "organising for change" in its chapter on classes, and "isolating from change" is DIP by another name.

4.13 SOLID in the design interview

Keeping the design honest: five checks over your class diagram

A complete class diagram can still be a poor design. In an OOD interview, SOLID is most useful as **five quick checks over the diagram you just drew**, taking about five minutes. Run them in this order:

1. **Single responsibility.** Say each class's responsibility in one sentence. If the sentence needs an "and", the class has two jobs. A `ParkingLot` that assigns spots **and** prints receipts changes when pricing changes and again when the layout changes: two classes. *Quick check:* name the change that would force you to edit the class; if you can name two unrelated changes, split it.
2. **Open/closed.** Look for a `switch` or `if` chain over a type. In a parking lot, pricing as a switch over rate types means a weekend rate edits the class that prices everything. Put pricing behind a small interface and the weekend rate is a new class. *Quick check:* pick one likely feature; if adding it edits three existing classes, the design is closed to the wrong thing.
3. **Liskov substitution.** Read each inheritance arrow: can the child be handed to code expecting the parent? If any parent method would throw, do nothing or refuse, the arrow is wrong. Check this again at the end: inheritance arrows get added late, when time is short.
4. **Interface segregation.** Look for a big interface where implementers leave methods empty. Split until every implementer uses everything it promised. A library `Search` interface with four methods is fine if the catalog genuinely implements all four.
5. **Dependency inversion.** Find high-level classes naming low-level ones. A `ParkingLot` that constructs a `CreditCardTransaction` cannot be tested without a payment provider and cannot take cash without an edit. Make it depend on `Payment` and let something else decide which kind arrives.

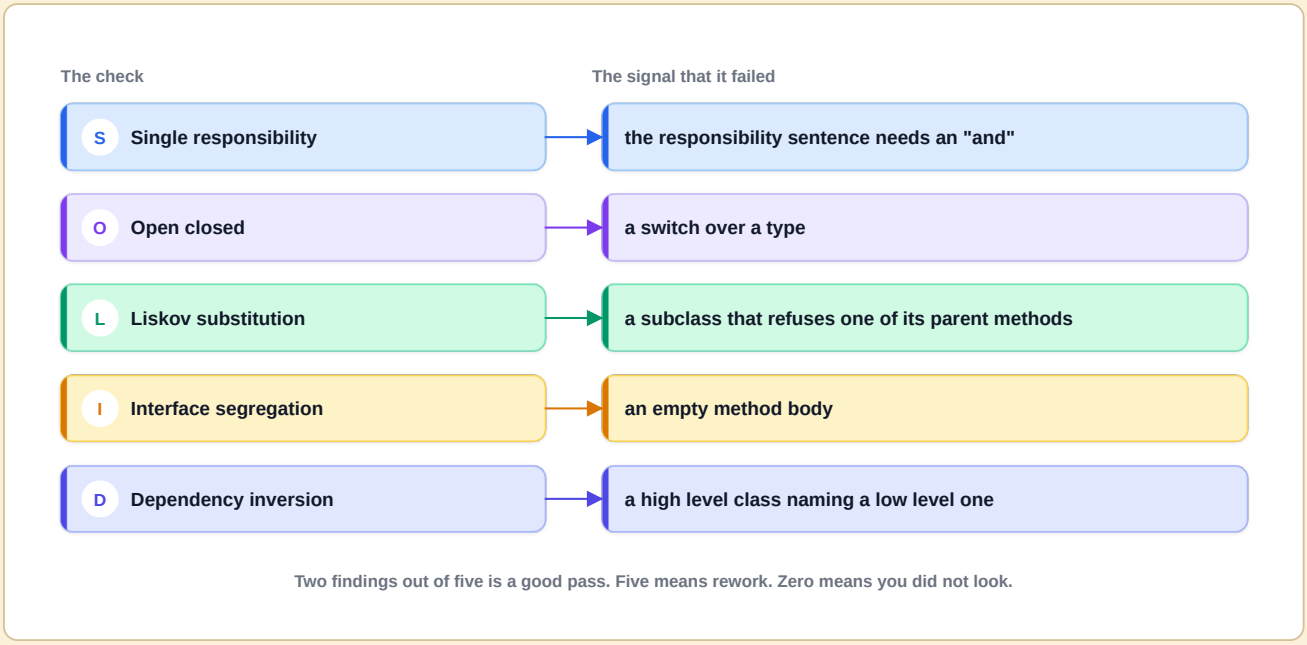


Figure — each check and the signal that it failed: an "and" in the sentence, a switch over a type, a refused parent method, an empty method body, a high-level class naming a low-level one (course diagram)

In the figure above, the right-hand column is what to scan for. The note at the bottom is the calibration: **two findings out of five is a good pass; five means rework; zero means you did not look.**

The two that earn the most credit. SRP improves your class list: twelve well-named classes beat six that each do three things. **OCP** is the one interviewers probe, usually as a late requirement ("what if trucks are charged a different rate?"). "Add a class" is a stronger answer than "add a branch in this switch".

A five-minute pass over a parking lot design:

Check	Finding
Single responsibility	`ParkingLot` issues tickets and finds spots: two reasons to change. Split if there is time; name it out loud if not.
Open/closed	Pricing sits behind `ParkingRate`, so a new rate is a new class. Passes.
Liskov substitution	Five spot subclasses add no behaviour. Collapse them into one class with a `type` field.
Interface segregation	No wide interfaces. Passes.
Dependency inversion	The lot depends on `Payment`, not on the card class. Passes.

Two real findings, each explainable in a sentence. That is what a good pass looks like.

Where SOLID is not the answer. It is a set of checks, not a scoring system. Do **not** add an interface for everything: an interface with one implementer, added because a rule "seemed to ask for it", is indirection with no benefit; add it when a second implementation exists or is clearly coming. Do **not** split classes further than a reader can follow: thirty classes with two methods each cannot be held in anyone's head. The one-sentence test is the boundary: if the responsibility fits in one sentence, the class is small enough.

Naming a design problem out loud

Interviewers seldom ask for a definition of OCP. They ask for a parking lot, a rate limiter (a component that caps how many requests are allowed per time window) or a file store, and then judge whether you can **explain the reasons behind the shape of your design**, which separates someone who memorised five acronyms from someone who uses them. Three things are weighed, most important first:

1. **Can you spot and name a problem in the code in front of you?** Shown a class with an ever-growing `if/else` on a type field, can you state the flaw in one crisp sentence?
2. **Can you defend a choice?** "SOLID says so" earns nothing. "Pricing rules change every quarter, and I don't want every change to reopen the checkout" earns a lot.
3. **Do you know when to hold back?** This is what marks a senior candidate. Adding abstractions is easy; choosing which ones to skip is the rarer skill.

A sentence pattern that works — say three things, in this order:

1. **What you see** — a specific fact about the code.
2. **What it will cost** — the future change that this makes painful.
3. **What you would do, and what that costs you** — the remedy and its price. This third part is the one most people leave out, and it is exactly what makes an answer sound like it comes from experience rather than from memory.

Example: a `ReportGenerator` that formats a report, writes it to disk and emails it.

- Weak: "It breaks SRP since it does three things; I'd make three classes." Correct, but it sounds like a definition recalled from last night's reading.
- Strong: "Three different owners can request changes here: marketing controls the wording, the platform team controls where files land, and the email owners control delivery. That means a copy edit redeploys the file-writing code. I'd pull formatting apart from delivery, likely into a formatter and a sink (the destination the output is sent to). It costs me two more classes and a little wiring, which pays off because the wording changes frequently." It names the actors, the pain, the fix and its price, and ends in a decision.

Say the observation before the principle. "`Reporting` implements nine methods but calls only two of them, which is an interface segregation problem" shows that you noticed something. Opening with "This violates ISP because…" sounds like you went looking for a place to use the acronym. Do still name the principle at the end, because it hands the interviewer a label for the write-up.

"Would you always do that?" This follow-up checks whether you understand the price of your own suggestion; it is rarely a trap, and the right answer is almost never "yes". Keep honest limits ready:

- *On splitting classes:* "No. If the two parts would always be edited together, I've just created an extra file. I split when two different people would ask for two different changes."
- *On extension points:* "No. I keep the `if/else` until a second or third case actually appears. An extension point built in advance is usually wasted effort and makes the code harder to follow in the meantime."

- *On interfaces:* "Only where behaviour varies or a test needs a substitute. An interface with a single implementation and no fake is just one more file."

Treating a principle as unconditional is the quickest way to show you have never actually applied it.

Three things that cost marks:

- **Reciting definitions** without tying them to the code on the whiteboard.
- **Applying everything at once.** Five interfaces for a forty-line problem looks inexperienced, not thorough; real systems earn their abstractions.
- **Naming the wrong principle confidently.** Mixing up cohesion and coupling reveals that the vocabulary is second-hand. When unsure, describe the issue in plain words and leave the label out: an accurate unnamed description beats a confidently wrong name.

A one-minute drill for any class you own: (1) Who can ask me to change this class, and which parts belong to whom? (2) What is the next feature likely to be, and would I have to edit this class for it? (3) Which dependencies could I not replace in a test, and does that matter? Practising this on your own code prepares you better than rereading definitions, and the interview will ask you to do it on code you have never seen.

> `In the interview:` describe before you label, and end every recommendation with its cost, for example: "I'd move delivery into its own class; that costs a file and some wiring, and in return a wording change no longer touches file handling."

4.14 Common questions about SOLID (with short answers)

Give the two-line version and stop; expand only if asked. Long unprompted answers about definitions eat the time you need for the design itself.

1. What is the difference between SRP and ISP? The difference is *whom* each one guards. **SRP guards the class** that does the work: does it have more than one reason to change? **ISP guards the caller** at a boundary: must it depend on methods it never calls? A `Printer` interface with `print`, `scan` and `fax` is an ISP issue for a basic printer, which has to supply two operations it cannot perform. A `ReportService` that both formats and emails is an SRP issue with no interface in sight. *Short answer:* SRP splits classes, ISP splits interfaces, and either can be violated without the other.

2. When would you not apply these principles? This question separates candidates, so have a concrete answer. Skip them when nothing has varied yet (an extension point for a case that never arrives adds indirection and returns nothing); in small or throwaway code (a one-off script needs no interfaces); when the split would be arbitrary (you cannot name the two groups who would each ask for changes); and when it would hurt readability (five four-line files can be harder to follow than one twenty-line file). *Framing:* each principle buys **flexibility later** with **simplicity now**; if "later" never comes, the price was paid for nothing.

3. Is SOLID still relevant, given the criticism? Yes, and being aware of the criticism counts in your favour. The objections are reasonable: the principles came from large object-oriented systems and are frequently taught as absolute rules to people building small ones, which yields layers of interfaces with a single implementation each. Some engineers argue that plainer advice (keep functions small, make dependencies explicit) captures most of the benefit. What outlasts the criticism is the **vocabulary and the diagnosis**: a compact way to explain why a piece of code will resist change, in any language or paradigm. *Short answer:* valuable as diagnostic tools, misleading as rules; treat them as names for problems, not a checklist.

4. Dependency inversion vs dependency injection? Candidates mix these up constantly, so a crisp answer stands out. Inversion is the **principle** (depend on abstractions); injection is one **technique** for reaching it (supply dependencies from outside rather than constructing them inside). Injecting a concrete class is injection without inversion, and a hand-written constructor with no framework is inversion without a container. Spring only automates the wiring; it is not the principle.

5. Give an example of an LSP violation. Prefer one from your own work; failing that, Square/Rectangle, but explain the underlying rule rather than just the story: `Rectangle` lets callers set width and height separately, `Square` cannot permit that and remain a square, so code written for `Rectangle` computes a wrong area without any error. The general rule: a subclass must not require more, guarantee less, or break an assumption callers depend on; an exception is merely the most obvious way to break it.

6. Does OCP mean I can never edit a file? No, and saying so plainly is a good answer. It means one more variation *of a kind you already support* can be added without touching existing code; the file is not frozen. Bug fixes, refactorings and changes to the extension point itself all involve edits. Closure is also relative to a kind of change: the invoice design in 4.7 was closed to new formats yet fully open to new discount rules, and that was the right trade at the time.

7. Composition over inheritance, always? No; it is a guideline, not a rule. Inherit when the child really is a kind of the parent and keeps every inherited promise. Compose when a behaviour differs between objects or a subclass would have to reject an inherited method. That rejection is the thing to watch for: an override that throws, does nothing, or is documented as "don't call this on X" belongs in a separate object.

8. How do these principles relate to design patterns? Many patterns are these principles in concrete form, which is why learning the principles first makes the patterns feel natural. **Strategy** applies OCP to an algorithm that varies; **Adapter** lets a caller go on using the interface it already knows; **Decorator** adds behaviour without editing the wrapped class; **Factory** hides concrete constructors from callers, which serves DIP. The principles describe good structure; the patterns are named, reusable recipes for producing it (see 4.12).

9. Which principle is violated most often? There is no single right answer, but two defensible ones. **SRP**, because classes grow by small accumulations: no one sets out to write a two-thousand-line service; it builds up one sensible-looking addition at a time. **ISP in reverse** (over-splitting): engineers fresh from learning it create an interface per method, meeting the wording of the rule while adding names and removing no dependency. Citing the second shows you have watched the principles being misused, not merely ignored.

> **Interview tip:** if the interviewer invites you to go deeper, the two questions worth expanding are "when would you not apply this?" and "dependency inversion vs dependency injection", since most candidates stay vague on both and a precise answer stands out. The strongest answers on SOLID always **admit a limit**: anyone can praise a principle; saying where it stops paying off is what sounds like real maintenance experience.

4.15 Over-engineering: the warnings in one place

Warning sign	Principle misapplied	What to do instead
Classes named `XHelper`, `XUtils`, `XManager2` after a split	SRP	Split by actor; each new class needs a name that stands alone
Six small classes that always change in the same commit	SRP (over-split) — *Shotgun Surgery*	Merge them back
An interface + factory + registry for a single implementation	OCP — *Speculative Generality*	Write the `if`; refactor at the second or third case (rule of three)
A subclass that "works" by silently doing nothing	LSP	Raise a better parent or use composition
One interface per method; the same object passed three times	ISP (over-split)	Group by caller into role interfaces
An interface in front of `String`-like stable classes	DIP	Invert only volatile dependencies
A DI container for a 200-line program	DIP	Wire by hand in `main`

YAGNI ("You Aren't Gonna Need It", from Extreme Programming, an agile method built on small frequent releases and heavy testing) is the counterweight to all of SOLID: do not build for requirements you do not have. SOLID tells you **what** a good seam looks like; YAGNI tells you **when** to cut it: when a real change asks for it.

Chapter self-check

1. **Q:** State each SOLID principle in one line. **A:** SRP: one reason (actor) to change. OCP: add behaviour by adding code, not editing. LSP: subtypes must be substitutable for their base types. ISP: no caller depends on

methods it does not use. DIP: policy and details both depend on abstractions owned by the policy.

2. **Q:** A class has a `switch (vehicleType)` that computes parking fees. Which principle, and what fix? **A:** OCP. Put fees behind a `ParkingRate` interface with one class per rate (Strategy), chosen at the edge.
3. **Q:** `ReadOnlyList extends ArrayList` and throws on `add`. Which principle? **A:** LSP (a new exception / refused bequest). Prefer composition or a read-only interface.
4. **Q:** A `Robot` class implements a `Worker` interface with `work()` and `eat()`. What is wrong? **A:** ISP: the robot must implement `eat()`. Split into `Workable` and `Eater` roles based on callers.
5. **Q:** `OrderService` has `private final MySqlOrderDao dao = new MySqlOrderDao();`. What two problems? **A:** Tight coupling / DIP violation (names and creates a volatile detail) and untestable without MySQL. Inject an `OrderSaver` interface owned by the service.
6. **Q:** Why is Square/Rectangle a worse bug than `Penguin.fly()` throwing? **A:** It is silent: the program continues with a wrong result.
7. **Q:** Coupling vs cohesion in one sentence each. **A:** Cohesion is how well the insides of one class belong together; coupling is how much one class depends on another.
8. **Q:** In what order do the principles usually appear during a refactor, and why? **A:** SRP → OCP → LSP → ISP → DIP; each creates the situation the next one addresses.
9. **Q:** What is the difference between DIP, DI and IoC? **A:** DIP is the principle (depend on abstractions); DI is a technique (pass dependencies in); IoC is the broader idea that a framework calls your code, and IoC containers perform DI.
10. **Q:** When should you not add an interface? **A:** When there is one implementation, no test double, and nothing has varied; or when the dependency is stable.

TL;DR

- **SOLID** = five names for the ways code becomes painful to change. Use them to diagnose, not as a checklist.
- **SRP**: one actor per class. Split by *who asks for changes*, not by method count. High cohesion inside, and check coupling after every split.
- **OCP**: put the part that *has* varied behind an interface; add variants as classes (Strategy). A class is closed against a specific kind of change.
- **LSP**: a subclass must accept at least as much, promise at least as much, keep invariants and history, and throw nothing new. Inheritance follows promises, not real-world categories. Prefer composition when a subclass would refuse a method.
- **ISP**: one role interface per kind of caller. Throwing methods mean too wide; the same object passed three times means too thin.
- **DIP**: policy owns the abstraction; details implement it; `main` wires them. DI is the technique, IoC containers are the tool.
- **Order**: SRP → OCP → LSP → ISP → DIP, each triggered by a real change.
- **In the interview**: observation → cost → fix → price of the fix. Always know when *not* to apply a principle.

Chapter 5 — More Design Principles

SOLID (Chapter 4) is only five of the rules good designers use. This chapter covers the rest of the everyday toolkit: short rules of thumb (DRY, KISS, YAGNI), rules about how objects talk to each other (Law of Demeter, Tell-Don't-Ask, loose coupling, the Hollywood Principle), the object-oriented principles that *Head First Design Patterns* and the Gang of Four book (*Design Patterns*, 1994, by Gamma, Helm, Johnson and Vlissides — "GoF" for short) build every pattern on (encapsulate what varies, favour composition, program to an interface), rules about correctness (fail fast, immutability, Design by Contract) **GRASP** — nine guidelines for deciding *which class should do what* — and, finally, the essentials of **Domain-Driven Design**, which decides *which classes exist and where their boundaries are* by starting from the business.

Key terms used throughout (defined once here):

Term	Meaning
Coupling	How much one class depends on (knows about) another. High coupling = a change in one forces changes in the other.
Cohesion	How closely the things inside one class belong together. High cohesion = the class does one focused job.
Abstraction	A simplified view (usually an interface or abstract class) that hides details that callers do not need.
Encapsulation	Keeping an object's data private and letting other code change it only through the object's methods.
Invariant	A fact about an object that must always be true (e.g. "balance is never negative").
Side effect	Anything a method changes besides returning a value: fields, files, the database, the screen.
Responsibility	Something a class *knows* (its data) or *does* (its behaviour).

Chapter map

#	Principle	One-line idea	Comes from
1	DRY	Each piece of knowledge has one authoritative home	Hunt & Thomas, *The Pragmatic Programmer* (1999)
2	KISS	Choose the simplest design that works	US Navy (1960); popularised by Kelly Johnson, Lockheed Skunk Works
3	YAGNI	Don't build features before they are needed	Extreme Programming, late 1990s (usually credited to Ron Jeffries)
4	Law of Demeter	Only talk to your immediate friends	Ian Holland / Karl Lieberherr, Demeter project, Northeastern University (1987)
5	Tell, Don't Ask	Tell objects what to do; don't pull their data out and decide for them	Andy Hunt & Dave Thomas (Pragmatic Programmers), building on Alec Sharp's *Smalltalk by Example* (1997); explained by Martin Fowler
6	Composition over inheritance	Prefer "has-a" to "is-a" for reuse	GoF book (1994), *Effective Java* (Joshua Bloch)
7	Program to an interface	Depend on supertypes, not concrete classes	GoF book
8	Encapsulate what varies	Isolate the part that changes	GoF book, *Head First Design Patterns*
9	Loose coupling	Interacting objects should know little about each other	*Head First Design Patterns*
10	Hollywood Principle / IoC	"Don't call us, we'll call you"	*Head First*; frameworks, IoC containers
11	Separation of Concerns	Each part of the program handles one concern	Edsger Dijkstra (1974)
12	Least Astonishment	Behave the way readers expect	User-interface and programming-language design folklore (no single author)
13	Fail fast	Detect problems early and loudly	Popularised by Jim Shore's "Fail Fast" article, *IEEE Software* (2004)
14	Immutability	Objects that never change after creation	*Effective Java* ("minimize mutability")
15	Design by Contract	Preconditions, postconditions, invariants	Bertrand Meyer, Eiffel language (term coined 1986)
16	GRASP (9 patterns)	Guidelines for assigning responsibilities	Craig Larman, *Applying UML and Patterns* (1st ed. 1997; today's nine from the 2nd ed. 2001)
17	Domain-Driven Design essentials	Model the business: bounded contexts, entities vs value objects, aggregates, domain events, repositories	Eric Evans, *Domain-Driven Design* (2003); Vaughn Vernon, *Implementing Domain-Driven Design* (2013)

How to read each section. Every principle follows the same path: what it is → the problem → how to apply it → a Java *before/after* program you can run → trade-offs → when not to use it → mistakes → real-world examples → interview answer → self-check → TL;DR. All code lives in `code/lld/ch05/<topic>/Main.java` and was compiled with `javac --release 21` (`javac` is the Java compiler, which turns source code into runnable bytecode; `--release 21` targets Java 21) and run; the output shown is the real output.

---

1. DRY — Don't Repeat Yourself

What it is

DRY says: every piece of *knowledge* in a system — a business rule, a formula, a validation, a constant — should have **a single, unambiguous, authoritative representation** — one place where it is defined. Everything else refers to that place. The principle comes from Andy Hunt and Dave Thomas's *The Pragmatic Programmer* (1999).

Analogy: a company keeps one master price list. Shops look prices up from it. If each shop kept its own hand-written copy, a price change would reach some shops and not others.

Important: DRY is about **knowledge**, not about lines of text that happen to look the same. Two pieces of code that look identical but represent *different* rules (which may change for different reasons) are *not* a DRY violation.

The problem it solves

When a rule is copied into several places, the copies eventually **drift**: someone updates one copy and forgets the others. The system then gives two different answers to the same question — a bug that is hard to spot because each copy looks reasonable on its own. Duplication also multiplies the effort of every change.

How it works

1. Notice a rule that appears in more than one place (a regex, a tax rate, a status check, a SQL fragment). A *regex* (regular expression) is a compact pattern that text must match, such as "something@something"; *SQL* is the standard language for querying relational databases.
2. Ask: *"If this rule changes, must all copies change together?"* If yes, it is one piece of knowledge.
3. Give it one home: a constant, a method, a value object, an enum, a config entry or a shared module.
4. Replace every copy with a reference to that home.
5. Apply the **Rule of Three** for code-level duplication (a heuristic credited to Don Roberts and popularised by Martin Fowler's *Refactoring* — refactoring means improving code's structure without changing what it does, see Chapter 7): the first time, just write it; the second time, notice the duplication but tolerate it; the third time, extract.

Java example — before/after

The *before* has two services that each carry their own copy of "what a valid email is" and "what the tax is". The copies have already drifted. The *after* puts the email rule inside an `EmailAddress` value object (a small immutable type — one whose value can never change after it is created — whose constructor validates its value) and the tax rule inside `TaxPolicy`.

```java
// File: lld/ch05/dry/Main.java
package lld.ch05.dry;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.regex.Pattern;

/** DRY: one piece of knowledge (what a valid email is, what the tax rate is) lives in ONE place. */
public class Main {

    // ---------------- BEFORE: the same rule is written twice and the copies have drifted ----------------
    static class Before {
        static class SignupService {
            boolean signup(String email) {
                // copy #1 of the rule (someone relaxed it later)
                return email != null && email.contains("@");
            }
            BigDecimal priceWithTax(BigDecimal price) {
                return price.multiply(new BigDecimal("1.18")).setScale(2, RoundingMode.HALF_UP); // 18% hard-coded
            }
        }
        static class ProfileService {
            boolean changeEmail(String email) {
                // copy #2 of the rule (the stricter original)
                return email != null && email.matches("[^@\\s]+@[^@\\s]+\\.[a-z]{2,}");
            }
            BigDecimal priceWithTax(BigDecimal price) {
                return price.multiply(new BigDecimal("1.18")).setScale(2, RoundingMode.HALF_UP); // 18% again
            }
        }
    }

    // ---------------- AFTER: each rule has a single, authoritative home ----------------
    static class After {
        /** The ONLY definition of "valid email" in the system. */
        record EmailAddress(String value) {
            private static final Pattern FORMAT = Pattern.compile("[^@\\s]+@[^@\\s]+\\.[a-z]{2,}");
            EmailAddress {
                if (value == null || !FORMAT.matcher(value).matches())
                    throw new IllegalArgumentException("invalid email: " + value);
            }
        }

        /** The ONLY definition of the tax rule. */
        static final class TaxPolicy {
            private static final BigDecimal RATE = new BigDecimal("0.18");
            private TaxPolicy() {}
            static BigDecimal withTax(BigDecimal price) {
                return price.add(price.multiply(RATE)).setScale(2, RoundingMode.HALF_UP);
            }
        }

        static class SignupService {
            String signup(String rawEmail) { return "signed up " + new EmailAddress(rawEmail).value(); }
        }
        static class ProfileService {
            String changeEmail(String rawEmail) { return "email changed to " + new EmailAddress(rawEmail).value();
}
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        var signup = new Before.SignupService();
        var profile = new Before.ProfileService();
        String sloppy = "bob@localhost";
        System.out.println("signup accepts '" + sloppy + "'? " + signup.signup(sloppy));
        System.out.println("profile accepts '" + sloppy + "'? " + profile.changeEmail(sloppy) + "   <-- same rule,
```

```
two answers");

        System.out.println("== AFTER ==");
        var s = new After.SignupService();
        var p = new After.ProfileService();
        System.out.println(s.signup("bob@example.com"));
        System.out.println(p.changeEmail("bob@example.com"));
        for (String candidate : new String[]{sloppy}) {
            try { s.signup(candidate); } catch (IllegalArgumentException e) { System.out.println("signup rejected:
" + e.getMessage()); }
            try { p.changeEmail(candidate); } catch (IllegalArgumentException e) { System.out.println("profile
rejected: " + e.getMessage()); }
        }
        System.out.println("price 100.00 with tax = " + After.TaxPolicy.withTax(new BigDecimal("100.00")));
    }
}
```

Output:

```
== BEFORE ==
signup accepts 'bob@localhost'? true
profile accepts 'bob@localhost'? false   <-- same rule, two answers
== AFTER ==
signed up bob@example.com
email changed to bob@example.com
signup rejected: invalid email: bob@localhost
profile rejected: invalid email: bob@localhost
price 100.00 with tax = 118.00
```

What each part does:

- `Before.SignupService` and `Before.ProfileService` each re-implement the email rule; the output shows them disagreeing about `bob@localhost`.
- `After.EmailAddress` is a Java **record** (a compact, immutable data class). Its *compact constructor* (a record constructor written without a parameter list, used only to check or normalise the incoming values) runs the check, so an invalid email simply cannot exist as an `EmailAddress`.
- `After.TaxPolicy` holds the rate in one private constant; changing GST from 18% means editing one line.

Trade-offs

Benefit	Cost
One change point per rule; no drift	A shared abstraction couples its users together
Less code to read and test	A premature or wrong abstraction is worse than duplication
Rules get a name (self-documenting)	Extra indirection to find the real code

When not to use it

- When two snippets look alike **by coincidence** and will evolve separately (e.g. "max username length 30" and "max tag length 30"). Merging them creates false coupling.
- Across service boundaries in **microservices** (an architecture where one application is split into many small services, each deployed on its own and talking over the network): sharing a domain library between independently deployed services can couple their release cycles. Some duplication is the lesser evil.
- In tests, a little repetition often makes each test easier to read on its own (this is sometimes called DAMP — "descriptive and meaningful phrases").

Common mistakes

- Treating DRY as "never have two similar lines" and building one monster function with flags for every caller.
- Extracting after the first repetition, before you understand what really varies.
- Forgetting non-code duplication: the same rule in code *and* in a database constraint *and* in docs.

Real-world examples

- **Spring's `@Value` / `application.yml`** (Spring is the most widely used Java framework for server applications; `@Value` is an *annotation* — a `@`-prefixed tag on code that tools and frameworks read — and `application.yml` is its configuration file): configuration values live in one file and are injected wherever needed.
- **Database normalisation** is DRY for data: a customer's address is stored once and referenced by key (a unique ID column that other tables point to).
- **Bean Validation (`jakarta.validation`) annotations** such as `@Email` on a field define the rule once and reuse it for REST input (data arriving through a web API built on HTTP's standard verbs and URLs), forms and persistence (saving data so it outlives the program, usually in a database).
- **OpenAPI / Protobuf schemas** (machine-readable descriptions of an API's messages — OpenAPI for HTTP APIs that exchange JSON, a plain-text key–value data format; Protobuf for Google's compact binary format) generate client and server code from one contract, so the message format is not hand-copied.

In the interview

> **30-second answer:** "DRY means each piece of knowledge has one authoritative representation. It is about rules, not about identical-looking text. I extract a rule when copies must change together — for example, validation into a value object — and I use the Rule of Three so I don't abstract too early."

Follow-ups:

- *Q: Is duplicated code always a DRY violation?* A: No. If the two copies represent different rules that change for different reasons, merging them adds wrong coupling.
- *Q: DRY vs. SRP?* A: They cooperate (SRP is the Single Responsibility Principle from Chapter 4: a class should have only one reason to change). DRY gives each rule one home; SRP makes sure that home has only one reason to change.
- *Q: How do you spot duplication of knowledge?* A: Ask "if this business rule changes, how many files do I edit?" More than one is a smell.

Self-check

1. Q: What exactly must not be repeated? → A: Knowledge (rules, formulas, decisions), not merely similar-looking lines.
2. Q: What is the Rule of Three? → A: Tolerate a second copy; extract a shared abstraction when a third appears.
3. Q: What bug does duplication cause? → A: Drift — copies change independently and start disagreeing.

TL;DR

One rule, one home. Extract when copies must change together; leave coincidental look-alikes alone.

---

2. KISS — Keep It Simple, Stupid

What it is

KISS says most systems work best when they are kept simple. The phrase was used by the US Navy around 1960 and is associated with Kelly Johnson, lead engineer of Lockheed's Skunk Works, who wanted jet aircraft a mechanic could repair in the field with basic tools. Choose the plainest design that fully solves the real problem. *Simple* means easy to understand and change — not necessarily the fewest lines.

Analogy: to hang one picture you use a nail, not a motorised track system with remote control.

The problem it solves

Engineers love clever solutions: home-made frameworks, deep inheritance trees, generic engines configured by lambdas (short unnamed functions written inline, e.g. `x -> x * 2`). Every extra moving part must be read, tested, debugged and explained to new team members. Accidental complexity (complexity we add, as opposed to complexity the problem truly has) is the biggest source of bugs and slow delivery.

How it works

1. State the actual requirement in one sentence.

2. Write the most direct solution using standard language features and libraries.
3. Ask: "Could a new team member understand this in two minutes?"
4. Add structure (patterns, abstractions) only when a concrete need — a second variant, a test seam, a real performance issue — shows up.
5. Delete cleverness that no longer earns its keep.

Java example — before/after

Three fixed customer tiers get three fixed discounts. The *before* builds a priority-sorted rule engine of predicates and functions. The *after* is a `switch` expression, which the compiler also checks for completeness: a `switch` *expression* over an enum (a type with a fixed list of named constants) with no `default` must cover every constant, so adding a new tier makes it fail to compile until you handle the new case. (Old-style `switch` *statements* are not checked this way.) Money is `BigDecimal` (Java's exact decimal-number class), never `double` (binary floating point cannot represent most decimal amounts exactly).

```java
// File: lld/ch05/kiss/Main.java
package lld.ch05.kiss;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.function.Function;
import java.util.function.Predicate;

/** KISS: the simplest design that fully solves today's problem wins. Money is BigDecimal, never double. */
public class Main {

    enum Tier { REGULAR, SILVER, GOLD }

    static final BigDecimal SILVER_FACTOR = new BigDecimal("0.90");
    static final BigDecimal GOLD_FACTOR = new BigDecimal("0.80");

    static BigDecimal times(BigDecimal amount, BigDecimal factor) {
        return amount.multiply(factor).setScale(2, RoundingMode.HALF_UP);
    }

    // ---------------- BEFORE: a home-made "rule engine" for three fixed discounts ----------------
    static class Before {
        record Rule(int priority, Predicate<Tier> when, Function<BigDecimal, BigDecimal> apply) {}

        static class DiscountRuleEngine {
            private final List<Rule> rules = new ArrayList<>();
            DiscountRuleEngine register(int priority, Predicate<Tier> when, Function<BigDecimal, BigDecimal> apply)
{
                rules.add(new Rule(priority, when, apply));
                rules.sort(Comparator.comparingInt(Rule::priority).reversed());
                return this;
            }
            BigDecimal evaluate(Tier tier, BigDecimal amount) {
                return rules.stream().filter(r -> r.when().test(tier)).findFirst()
                        .map(r -> r.apply().apply(amount)).orElse(amount);
            }
        }

        static DiscountRuleEngine build() {
            return new DiscountRuleEngine()
                    .register(10, t -> t == Tier.GOLD, a -> times(a, GOLD_FACTOR))
                    .register(5, t -> t == Tier.SILVER, a -> times(a, SILVER_FACTOR))
                    .register(0, t -> true, a -> a);
        }
    }

    // ---------------- AFTER: a plain switch expression; the compiler checks every tier is handled ---------------
-
    static class After {
        static BigDecimal discounted(Tier tier, BigDecimal amount) {
            return switch (tier) {
                case GOLD -> times(amount, GOLD_FACTOR);
                case SILVER -> times(amount, SILVER_FACTOR);
                case REGULAR -> amount;
            };
        }
    }

    public static void main(String[] args) {
        BigDecimal price = new BigDecimal("100.00");
        var engine = Before.build();
        System.out.println("== BEFORE (rule engine) ==");
        for (Tier t : Tier.values()) System.out.printf("%-7s -> %s%n", t, engine.evaluate(t, price));
        System.out.println("== AFTER (switch) ==");
        for (Tier t : Tier.values()) System.out.printf("%-7s -> %s%n", t, After.discounted(t, price));
```

```
    }
}
```

Output:

```
== BEFORE (rule engine) ==
REGULAR -> 100.00
SILVER  -> 90.00
GOLD    -> 80.00
== AFTER (switch) ==
REGULAR -> 100.00
SILVER  -> 90.00
GOLD    -> 80.00
```

Both produce identical results, but the *after* has no ordering bugs to worry about (in the engine, forgetting a priority would silently change behaviour), no mutable list, and reads like the business rule.

Trade-offs

Simple design	"Flexible" complex design
Fast to read, change, test	Handles hypothetical cases
May need refactoring when real variation arrives	Costs you every day, even if variation never arrives
Uses standard, well-known constructs	Custom mechanisms need docs and training

When not to use it

KISS is never "wrong", but it is not an excuse to ignore the real complexity of the problem. A payment system *needs* idempotency (repeating the same request — say, after a timeout — has the same effect as doing it once, so nobody is charged twice; see System Design Fundamentals), retries and auditing; leaving them out is not simplicity, it is incompleteness. Also, when a rule-engine really is the requirement (business users edit rules at runtime — while the program is running), build or buy one.

Common mistakes

- Confusing *simple* with *easy* or *short*: a one-line stream with five nested lambdas may be shorter but harder to read than a loop.
- Using KISS to justify skipping tests, error handling or validation.
- Copy-pasting instead of simplifying (that violates DRY).

Real-world examples

- **Unix philosophy** — small tools that do one thing and compose through pipes (`|`, which feeds one program's output into the next program's input).
- **Go language design** deliberately left out many features to keep the language simple.
- `java.util.Optional` was intentionally kept small (no `Serializable`, not meant as a field type) to steer people toward its one purpose: return values that might be absent.

In the interview

30-second answer: "KISS means picking the simplest design that fully meets the requirements. In an interview I start with a straightforward model and only introduce patterns when a requirement demands variation or extension — and I say why."

Follow-ups:

- *Q: Isn't a design pattern always better?* A: No. A pattern adds classes and indirection; it's worth it only if it solves a problem you actually have.
- *Q: How do KISS and YAGNI differ?* A: KISS is about *how* you build (the simplest form); YAGNI is about *what* you build (only needed features).

Self-check

1. Q: What is accidental complexity? → A: Complexity introduced by our solution rather than required by the problem.
2. Q: Why did the `switch` version win? → A: Same behaviour, fewer moving parts, compiler-checked exhaustiveness.

TL;DR

Solve the real problem in the plainest way; add machinery only when a real need appears.

---

3. YAGNI — You Aren't Gonna Need It

What it is

YAGNI says: do not build a capability until you actually need it. It comes from Extreme Programming (XP), the agile method (a way of building software in short cycles with frequent feedback) created by Kent Beck in the late 1990s; the slogan is usually credited to XP co-founder Ron Jeffries. The opposite habit — building hooks, options and layers "just in case" — is called **speculative generality**, a **code smell** (a visible sign in the code that usually points to a deeper design problem) in Martin Fowler's *Refactoring*.

Analogy: you don't buy a 7-seater car because you *might* have five children one day.

The problem it solves

Features built early cost four times: the time to build them, the time to maintain and test them, the complexity they add to everything around them, and the cost of being wrong (the real future requirement usually looks different from the guess, so the speculative code must be rewritten anyway).

How it works

1. Implement the stories/requirements you have **now** (a *user story* is a short feature request written from the user's point of view: "As a shopper I can …").
2. Keep the code clean (tested, well-factored) so that it is cheap to change later — YAGNI depends on refactoring being easy.
3. When a new requirement actually arrives, extend the design then.
4. Delete unused options, parameters and classes you discover.

Java example — before/after

The requirement: *export orders as CSV* (comma-separated values — a plain-text table with one row per line). The *before* builds a universal exporter with five formats, encodings and compression — four formats and compression throw `UnsupportedOperationException`. The *after* is one method.

```java
// File: lld/ch05/yagni/Main.java
package lld.ch05.yagni;

import java.math.BigDecimal;
import java.util.List;
import java.util.stream.Collectors;

/** YAGNI: build what the current requirement needs ("export orders as CSV"), not what might be asked one day. */
public class Main {

    record Order(String id, String customer, BigDecimal total) {}   // money as BigDecimal, never double

    // ---------------- BEFORE: speculative generality ----------------
    static class Before {
        enum Format { CSV, PDF, XML, EXCEL, YAML }

        /** Five formats, pluggable encodings, compression... only CSV was requested. */
        interface ReportExporter {
            String export(List<Order> orders, Format format, String encoding, boolean compress);
        }

        static class UniversalExporter implements ReportExporter {
            @Override
            public String export(List<Order> orders, Format format, String encoding, boolean compress) {
                if (compress) throw new UnsupportedOperationException("compression not implemented yet");
                return switch (format) {
                    case CSV -> orders.stream().map(o -> o.id() + "," + o.customer() + "," +
o.total().toPlainString())
                            .collect(Collectors.joining("\n", "id,customer,total\n", ""));
                    case PDF, XML, EXCEL, YAML -> throw new UnsupportedOperationException(format + " not
implemented yet");
                };
            }
        }
    }

    // ---------------- AFTER: exactly what is needed today ----------------
    static class After {
        static String toCsv(List<Order> orders) {
            return orders.stream()
                    .map(o -> String.join(",", o.id(), o.customer(), o.total().toPlainString()))
                    .collect(Collectors.joining("\n", "id,customer,total\n", ""));
        }
    }

    public static void main(String[] args) {
        List<Order> orders = List.of(new Order("o1", "asha", new BigDecimal("250.00")),
                                     new Order("o2", "ravi", new BigDecimal("99.50")));
        System.out.println("== BEFORE ==");
        var exporter = new Before.UniversalExporter();
        System.out.println(exporter.export(orders, Before.Format.CSV, "UTF-8", false));
        try {
            exporter.export(orders, Before.Format.PDF, "UTF-8", false);
        } catch (UnsupportedOperationException e) {
            System.out.println("dead option: " + e.getMessage());
        }
        System.out.println("== AFTER ==");
        System.out.println(After.toCsv(orders));
    }
}
```

Output:

```
== BEFORE ==
id,customer,total
o1,asha,250.00
o2,ravi,99.50
dead option: PDF not implemented yet
== AFTER ==
id,customer,total
o1,asha,250.00
o2,ravi,99.50
```

Every caller of the *before* API (Application Programming Interface — the methods and types that other code calls) must pass three meaningless arguments, and every reader wonders which formats really work.

Trade-offs

YAGNI	Build ahead
Less code, faster delivery	Might save time if the guess is right
Design fits real needs	Guesses are usually wrong in detail
Needs good tests + refactoring discipline	Leaves dead options that confuse readers

When not to use it

- **Hard-to-change decisions** — public API formats, database schemas (the structure of tables and columns) with huge data, security, data retention laws. Think ahead there because changing later is very expensive.
- Architectural qualities that are known requirements (e.g. "must scale to 10k requests/s" — a *throughput* target, i.e. how much work per second the system must handle) are not speculation; they are requirements.

Common mistakes

- Using YAGNI as an excuse for messy code ("we'll never need clean code").
- Ignoring a requirement the interviewer *did* mention because "YAGNI".
- Adding `interface` + `Impl` pairs for every class "in case we swap it" — only add the interface when there is a second implementation or a test seam need (a *test seam* is a place where a test can swap in a fake collaborator).

Real-world examples

- **Minimum Viable Product (MVP)** thinking in startups is YAGNI at the product level.
- **Feature flags** (on/off switches in configuration that enable a code path without redeploying; e.g. LaunchDarkly, Unleash) let teams ship only what is needed now and turn on more later, rather than building every variant up front.
- XP teams (and many agile teams) plan per iteration so that only near-term work is built.

In the interview

30-second answer: "YAGNI means I build what the requirements need today and keep the design easy to extend. In an LLD (low-level design — designing the classes, interfaces and their interactions) interview I'll mention an extension point — say a `PaymentStrategy` interface — only if the interviewer hinted at multiple variants; otherwise I note it verbally as a possible extension."

Follow-ups:

- *Q: Doesn't YAGNI conflict with OCP?* A: Not really. OCP (the Open/Closed Principle from Chapter 4: open for extension, closed for modification) is applied where variation is known or likely; YAGNI stops you from inventing variation that isn't there.
- *Q: What makes YAGNI safe?* A: A good test suite (the full set of automated tests for the code) and clean code, so adding the feature later is cheap.

Self-check

1. Q: Name the smell YAGNI prevents. → A: Speculative generality.

2. Q: When should you design ahead anyway? → A: For expensive-to-reverse decisions (public APIs, schemas, security).

TL;DR

Build for today's requirements with clean, changeable code; tomorrow's features get built tomorrow.

---

4. Law of Demeter (Principle of Least Knowledge)

What it is

The **Law of Demeter (LoD)**, also called the **Principle of Least Knowledge**, says a method should only talk to its **immediate friends**, not to strangers it reaches through other objects. Formally, a method `m` of object `O` should only call methods on:

1. `O` itself (`this`);
2. objects passed to `m` as parameters;
3. objects `m` creates;
4. `O`'s own fields (its direct components).

(The original formulation by Ian Holland and Karl Lieberherr at Northeastern University, 1987, also allows global objects accessible to `O`; modern summaries usually list the four above.)

The famous slogan: **"use only one dot"** — e.g. `a.getB().getC().doIt()` is a **train wreck** (a chain of calls that looks like connected railway cars).

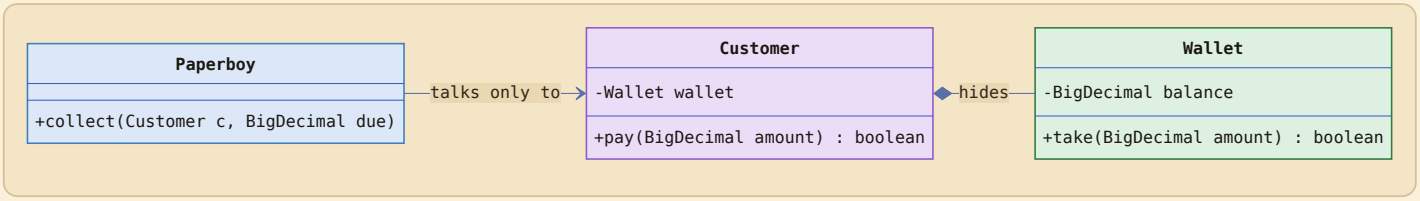
Analogy: when a paperboy collects money, he asks the customer to pay. He doesn't reach into the customer's pocket, pull out the wallet and take the cash himself.

The problem it solves

A train wreck couples the caller to the **whole chain's structure**. In `customer.getWallet().getBalance()`, the paperboy now depends on `Customer` *and* `Wallet`. If the customer starts paying by UPI (India's instant bank-to-bank mobile payment system) instead of a wallet, the paperboy breaks. Knowledge of internal structure leaks everywhere.

How it works

1. Spot a chain that navigates through other objects' internals.
2. Ask: *what does the caller really want?* ("get paid", not "get the wallet").
3. Add a method on the first object that does that job (`customer.pay(amount)`), delegating internally.
4. Keep the internal object private.



Java example — before/after

```java
// File: lld/ch05/demeter/Main.java
package lld.ch05.demeter;

import java.math.BigDecimal;
import java.util.List;

/** Law of Demeter ("only talk to your immediate friends"): the paperboy should not reach into the customer's
wallet. */
public class Main {

    // ---------------- BEFORE: a train wreck -- paperboy -> customer -> wallet -> balance ----------------
    static class Before {
        static class Wallet {
            private BigDecimal balance;
            Wallet(BigDecimal balance) { this.balance = balance; }
            BigDecimal getBalance() { return balance; }
            void setBalance(BigDecimal b) { balance = b; }
        }
        static class Customer {
            private final String name;
            private final Wallet wallet;
            Customer(String name, Wallet wallet) { this.name = name; this.wallet = wallet; }
            String getName() { return name; }
            Wallet getWallet() { return wallet; }
        }
        static class Paperboy {
            BigDecimal collected = BigDecimal.ZERO;
            void collect(Customer c, BigDecimal due) {
                // knows Customer has a Wallet, knows Wallet has a balance, changes it directly
                if (c.getWallet().getBalance().compareTo(due) >= 0) {
                    c.getWallet().setBalance(c.getWallet().getBalance().subtract(due));
                    collected = collected.add(due);
                    System.out.println("collected " + due + " from " + c.getName());
                } else {
                    System.out.println(c.getName() + " cannot pay");
                }
            }
        }
    }

    // ---------------- AFTER: the paperboy asks the customer (a direct friend) to pay ----------------
    static class After {
        static class Wallet {
            private BigDecimal balance;
            Wallet(BigDecimal balance) { this.balance = balance; }
            boolean take(BigDecimal amount) {
                if (amount.compareTo(balance) > 0) return false;
                balance = balance.subtract(amount);
                return true;
            }
        }
        static class Customer {
            private final String name;
            private final Wallet wallet;   // private detail -- could become a card or UPI tomorrow
            Customer(String name, Wallet wallet) { this.name = name; this.wallet = wallet; }
            String name() { return name; }
            /** Customer decides HOW to pay. */
            boolean pay(BigDecimal amount) { return wallet.take(amount); }
        }
        static class Paperboy {
            BigDecimal collected = BigDecimal.ZERO;
            void collect(Customer c, BigDecimal due) {
                if (c.pay(due)) {                       // talk only to a parameter: allowed
                    collected = collected.add(due);
                    System.out.println("collected " + due + " from " + c.name());
                } else {
```

```
                    System.out.println(c.name() + " cannot pay");
                }
            }
        }
    }

    public static void main(String[] args) {
        BigDecimal due = new BigDecimal("4.00");
        System.out.println("== BEFORE ==");
        var boy1 = new Before.Paperboy();
        boy1.collect(new Before.Customer("Meera", new Before.Wallet(new BigDecimal("10.00"))), due);
        boy1.collect(new Before.Customer("Kabir", new Before.Wallet(new BigDecimal("2.00"))), due);

        System.out.println("== AFTER ==");
        var boy2 = new After.Paperboy();
        boy2.collect(new After.Customer("Meera", new After.Wallet(new BigDecimal("10.00"))), due);
        boy2.collect(new After.Customer("Kabir", new After.Wallet(new BigDecimal("2.00"))), due);
        System.out.println("total collected = " + boy2.collected);

        // Chained calls on a fluent API / stream are NOT Demeter violations: each call returns the same kind of
thing.
        long longNames = List.of("Meera", "Kabir", "Jo").stream().filter(n -> n.length() > 2).count();
        System.out.println("stream chain is fine: " + longNames + " long names");
    }
}
```

Output:

```
== BEFORE ==
collected 4.00 from Meera
Kabir cannot pay
== AFTER ==
collected 4.00 from Meera
Kabir cannot pay
total collected = 4.00
stream chain is fine: 2 long names
```

What changed: in *before*, `Paperboy` knows three classes and even sets the balance directly. In *after*, `Paperboy` knows only `Customer.pay`, and `Customer` is free to change how it pays.

Trade-offs

Following LoD	Ignoring LoD
Changes to internals stay local	Structural changes ripple to callers
Easier mocking (mock = fake object used in tests) — one collaborator	Tests must build whole object graphs
May add small **wrapper/delegate methods**	No extra methods
Too many wrappers can bloat the middle class	—

When not to use it

- **Fluent interfaces and builders** (APIs whose methods return an object so calls can be chained; a builder assembles an object step by step — `StringBuilder.append(..).append(..)`, `stream().filter().map()`) return the same object or a sibling of the same abstraction — they are *not* violations.
- **Data structures / DTOs** (Data Transfer Objects — plain data with no behaviour, see Chapter 6) are meant to expose their data; `order.shippingAddress().city()` on a record is fine.
- Don't mechanically add dozens of pass-through methods — that is the **Middle Man** smell. Rethink responsibilities instead.

Common mistakes

- Counting dots instead of thinking about *knowledge*: splitting a chain into temp variables (`var w = c.getWallet(); w.getBalance()`) still violates the law.

- Adding getters (methods like `getWallet()` that just return a field) for everything, which invites train wrecks.

Real-world examples

- *Head First Design Patterns* presents it as "talk only to your immediate friends" in the Facade chapter — a **Facade** (one simple class placed in front of a group of complex classes) reduces how many classes a client talks to.
- **Spring Data** repositories (a *repository* is an object that loads and saves domain objects while hiding the database): services call `orderRepository.findByCustomerId(id)` rather than navigating `entityManager.getSession().getConnection()...`.
- `java.nio.file.Files.readString(path)` hides channels, buffers and decoders behind a single call.

In the interview

> **30-second answer:** "The Law of Demeter says a method should only call methods on itself, its parameters, objects it creates and its own fields. Long chains like `a.getB().getC()` couple the caller to internal structure. I fix them by adding a method that expresses intent — `customer.pay()` instead of reaching for the wallet."

Follow-ups:

- *Q: Is `list.stream().map(...).collect(...)` a violation?* A: No — each call returns a stream/collection abstraction meant for chaining (a Java *stream* is a pipeline that processes a sequence of elements step by step); you're not navigating an object's internals.
- *Q: What's the cost of following it strictly?* A: Wrapper methods can pile up (Middle Man). Balance by moving behaviour to where the data is (Tell-Don't-Ask).

Self-check

1. Q: List the four "friends" of a method. → A: `this`, parameters, objects it creates, its own fields.
2. Q: What is a train wreck? → A: A chain of getters navigating through several objects.
3. Q: Do records/DTOs have to obey it? → A: Not really; they are data structures meant to expose data.

TL;DR

Only talk to your friends. Replace `a.getB().getC().do()` with a method on `a` that says what you want.

---

5. Tell, Don't Ask

What it is

Tell, Don't Ask (TDA) says: instead of *asking* an object for its data and then making a decision for it, *tell* the object what you want and let it decide using its own data. Behaviour should live with the data it needs. The idea was popularised by Andy Hunt and Dave Thomas (the Pragmatic Programmers), who built on Alec Sharp's observation in *Smalltalk by Example* (1997) that procedural code (code organised as functions acting on separate data) fetches data and then decides, while object-oriented code tells objects to act.

Analogy: you tell a taxi driver "take me to the airport". You don't ask for the steering wheel angle and the speed every second and decide the turns yourself.

The problem it solves

When callers ask-then-act (`if (acc.getBalance() >= x) acc.setBalance(acc.getBalance() - x)`), the business rule is spread over every caller. Soon one caller forgets part of the rule (e.g. "frozen accounts can't withdraw") and the object's **invariant** breaks. The object becomes a passive data bag — the **Anemic Domain Model** anti-pattern.

How it works

1. Look for code that reads an object's fields, decides something, then writes back.
2. Move that decision into a method on the object (`withdraw(amount)`).
3. Make the object reject illegal requests (exceptions or result types).

4. Remove setters (methods like `setBalance(x)` that just overwrite a field) that are no longer needed.

Java example — before/after

```
// File: lld/ch05/telldontask/Main.java
package lld.ch05.telldontask;

/** Tell, Don't Ask: tell an object what you want; don't pull its data out, decide, and push data back in. */
public class Main {

    // ---------------- BEFORE: callers ask for state and make the decision themselves ----------------
    static class Before {
        static class Account {
            private long balancePaise;
            private boolean frozen;
            Account(long balance) { this.balancePaise = balance; }
            long getBalance() { return balancePaise; }
            void setBalance(long b) { balancePaise = b; }
            boolean isFrozen() { return frozen; }
            void setFrozen(boolean f) { frozen = f; }
        }
        // ATM remembered the "frozen" rule...
        static void atmWithdraw(Account a, long amt) {
            if (!a.isFrozen() && a.getBalance() >= amt) a.setBalance(a.getBalance() - amt);
        }
        // ...the mobile app forgot it. The business rule is scattered and now inconsistent.
        static void appWithdraw(Account a, long amt) {
            if (a.getBalance() >= amt) a.setBalance(a.getBalance() - amt);
        }
    }

    // ---------------- AFTER: the rule lives with the data it protects ----------------
    static class After {
        static final class Account {
            private long balancePaise;
            private boolean frozen;
            Account(long balance) { this.balancePaise = balance; }
            void freeze() { frozen = true; }
            void withdraw(long amt) {
                if (amt <= 0) throw new IllegalArgumentException("amount must be positive");
                if (frozen) throw new IllegalStateException("account is frozen");
                if (amt > balancePaise) throw new IllegalStateException("insufficient funds");
                balancePaise -= amt;
            }
            long balance() { return balancePaise; }   // queries are fine for display -- just don't decide outside
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        var a = new Before.Account(1_000);
        a.setFrozen(true);
        Before.atmWithdraw(a, 300);
        System.out.println("after ATM on frozen account: " + a.getBalance());
        Before.appWithdraw(a, 300);
        System.out.println("after APP on frozen account: " + a.getBalance() + "   <-- rule bypassed");

        System.out.println("== AFTER ==");
        var b = new After.Account(1_000);
        b.withdraw(300);
        System.out.println("withdrew 300, balance " + b.balance());
        b.freeze();
        try { b.withdraw(300); } catch (IllegalStateException e) { System.out.println("rejected: " +
e.getMessage()); }
        System.out.println("balance still " + b.balance());
    }
}
```

Output:

```
== BEFORE ==
after ATM on frozen account: 1000
after APP on frozen account: 700   <-- rule bypassed
== AFTER ==
withdrew 300, balance 700
rejected: account is frozen
balance still 700
```

In *before*, the ATM and the app each reimplement the rule and the app forgets `frozen`. In *after*, `withdraw` is the only way to take money, so the rule cannot be bypassed.

Trade-offs

Tell	Ask
Invariants protected in one place	Rules duplicated across callers
Richer, more expressive objects	Simple data objects
Risk: object grows too many responsibilities	Fine for pure data (DTOs, reports)

When not to use it

- For **queries whose purpose is display or reporting** — asking for a balance to show it on screen is fine. The rule is about *deciding* outside.
- When the decision genuinely belongs to another object that combines data from several sources (then that other object is the Information Expert — see GRASP).

Common mistakes

- Adding a getter "just for this one check" — that is where the rule starts leaking.
- Pushing *every* behaviour into the entity, including emails and database calls (breaks SRP; use services for cross-cutting work).

Real-world examples

- `java.util.List.removeIf(predicate)` — you tell the list what to remove instead of iterating and removing yourself.
- `Map.computeIfAbsent`, `Map.merge` — tell the map the update you want; it handles the check-then-act.
- `AtomicInteger.incrementAndGet()` — the object performs the read-modify-write atomically (as one indivisible step that no other thread can interrupt); asking then setting would be a **race condition** (a bug where the result depends on the unlucky timing of two *threads* — independent paths of execution running at the same time — touching the same data).

In the interview

30-second answer: "Tell, Don't Ask means sending commands to the object that owns the data, e.g. `account.withdraw(100)`, rather than reading its balance and setting a new one. It keeps invariants in one place and avoids anemic models."

Follow-ups:

- *Q: How is it related to the Law of Demeter?* A: Both push behaviour to where the data is; LoD focuses on *who* you talk to, TDA on *what* you ask for.
- *Q: Concurrency angle?* A: Ask-then-act across two calls is a check-then-act race; a single `withdraw` method can be made atomic.

Self-check

1. Q: What anti-pattern does TDA fight? → A: The Anemic Domain Model (data bags with logic outside).
2. Q: Are getters forbidden? → A: No; they're fine for display. Avoid deciding outside the object.

TL;DR

Don't pull data out to make the object's decisions — tell the object what you want.

---

6. Composition over Inheritance

What it is

Favour object composition over class inheritance (from the GoF book). *Inheritance* ("is-a") reuses code by making a subclass of another class. *Composition* ("has-a") reuses code by holding a reference to another object and **delegating** (forwarding) calls to it.

Analogy: a car *has* an engine. You can swap a petrol engine for an electric one. If "car" *inherited from* "petrol engine", you could never swap it.

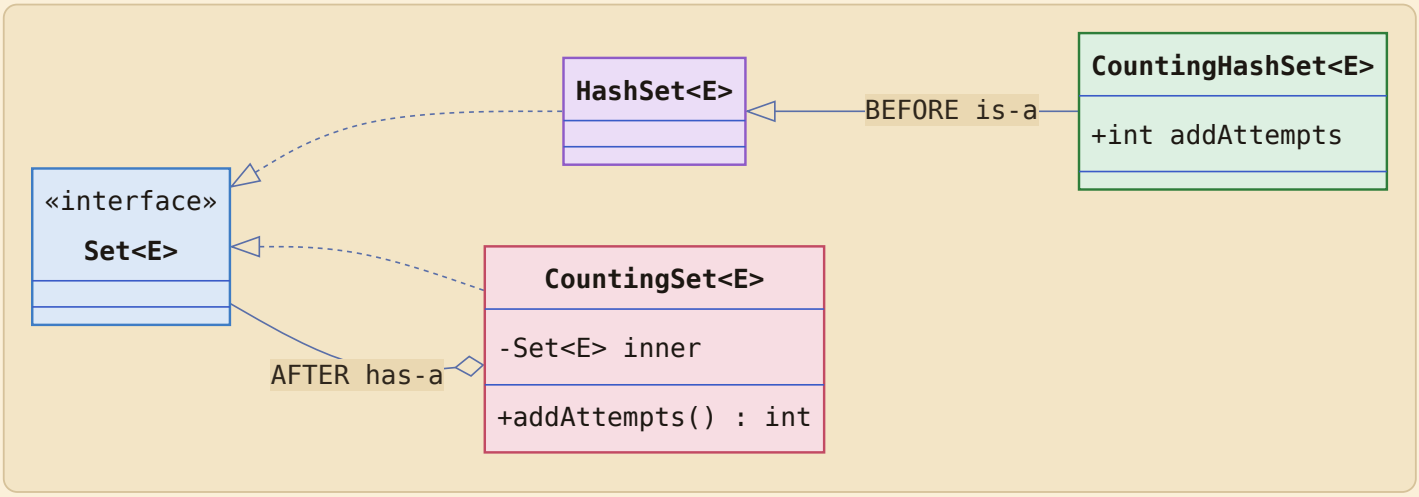
The problem it solves

Inheritance is the tightest coupling in object-oriented programming:

- **Fragile base class problem** — a subclass can break when the parent's *internal* implementation changes, even if the parent's public API is unchanged. The subclass depends on details like "does `addAll` call `add`?".
- **Fixed at compile time** (decided when the code is compiled, before it ever runs) — you cannot change the parent at runtime.
- **Class explosion** — combining features by subclassing (`LoggingCachingBufferedStream`) multiplies classes.
- It exposes all parent methods, even those that make no sense for the child.

How it works

1. Instead of `class B extends A`, give `B` a private field of type `A` (preferably an interface type).
2. Implement `B`'s methods by adding your behaviour and **forwarding** to the wrapped object.
3. Receive the wrapped object through the constructor, so any implementation can be plugged in.
4. Keep inheritance for genuine "is-a" relationships where the subclass truly is substitutable (Liskov Substitution, Chapter 4) and the parent was **designed for extension** (documented hooks, often abstract).



Java example — before/after

We want to count how many elements were ever added to a set. This is the classic trap described in *Effective Java*.

```java
// File: lld/ch05/composition/Main.java
package lld.ch05.composition;

import java.util.Collection;
import java.util.HashSet;
import java.util.Iterator;
import java.util.List;
import java.util.Set;
import java.util.AbstractSet;

/** Composition over inheritance: extending a concrete class couples you to its private implementation details. */
public class Main {

    // ---------------- BEFORE: inherit from HashSet to count insert attempts ----------------
    static class Before {
        @SuppressWarnings("serial")
        static class CountingHashSet<E> extends HashSet<E> {
            int addAttempts;
            @Override public boolean add(E e) { addAttempts++; return super.add(e); }
            @Override public boolean addAll(Collection<? extends E> c) {
                addAttempts += c.size();
                return super.addAll(c);   // HashSet's inherited addAll calls add() for each element -> counted
twice!
            }
        }
    }

    // ---------------- AFTER: wrap ANY Set and forward calls (a decorator) ----------------
    static class After {
        static final class CountingSet<E> extends AbstractSet<E> {
            private final Set<E> inner;   // "has-a", not "is-a"
            private int addAttempts;
            CountingSet(Set<E> inner) { this.inner = inner; }
            @Override public boolean add(E e) { addAttempts++; return inner.add(e); }
            @Override public boolean addAll(Collection<? extends E> c) { addAttempts += c.size(); return
inner.addAll(c); }
            @Override public boolean contains(Object o) { return inner.contains(o); }   // keep the inner set's
O(1)/O(log n) lookup
            @Override public boolean remove(Object o) { return inner.remove(o); }
            @Override public Iterator<E> iterator() { return inner.iterator(); }
            @Override public int size() { return inner.size(); }
            int addAttempts() { return addAttempts; }
        }
    }

    public static void main(String[] args) {
        List<String> words = List.of("red", "green", "blue");

        var before = new Before.CountingHashSet<String>();
        before.addAll(words);
        System.out.println("BEFORE: added 3 items, counter says " + before.addAttempts);

        var after = new After.CountingSet<String>(new HashSet<>());
        after.addAll(words);
        System.out.println("AFTER : added 3 items, counter says " + after.addAttempts());

        // Bonus: composition works with any Set implementation, chosen at runtime.
        var sorted = new After.CountingSet<String>(new java.util.TreeSet<>());
        sorted.addAll(words);
        System.out.println("AFTER over a TreeSet: " + sorted + " attempts=" + sorted.addAttempts());
    }
}
```

Output:

```
BEFORE: added 3 items, counter says 6
AFTER : added 3 items, counter says 3
AFTER over a TreeSet: [blue, green, red] attempts=3
```

Why the *before* prints 6: `HashSet` does not override `addAll`; it inherits `AbstractCollection.addAll`, which loops and calls `add` for every element. Our `addAll` adds 3, then the parent's loop calls *our overridden* `add` three more times. We depended on an implementation detail ("self-use") that `HashSet` does not promise to keep — the fragile base class problem. The *after* wraps any `Set` and forwards; the inner set's internal calls never reach our counter. It also forwards `contains` and `remove`: `AbstractSet` would otherwise implement them by scanning the iterator (an object that walks through a collection one element at a time) — O(n), i.e. time grows in step with the number of elements *n* (*Big-O notation*) — instead of using the inner set's fast lookup. It also works with a `TreeSet` chosen at runtime.

Trade-offs

Composition	Inheritance
Loose coupling to the public API only	Coupled to parent internals
Swap behaviour at runtime	Fixed at compile time
Combine behaviours freely (decorators)	Class explosion for combinations
More boilerplate (forwarding methods)	Free reuse of all parent methods
No automatic polymorphism (one variable working with many types) — need an interface	Subclass is automatically a subtype

When not to use it

- True "is-a" hierarchies with a base class **designed for extension**: e.g. extending `AbstractList` (implement `get` and `size`) or a Template Method base (see Hollywood Principle below).
- Sealed type hierarchies (a `sealed` type lists exactly which subtypes may exist) modelling a closed set of variants (`sealed interface Shape permits Circle, Square`).
- When you control both classes and they are in the same package and evolve together.

Common mistakes

- Inheriting just to reuse a few helper methods ("`OrderService extends BaseDao`").
- Composition without an interface — then you can't substitute implementations.
- Exposing the wrapped object with a getter, defeating encapsulation.

Real-world examples

- **JDK mistakes admitted by its own authors:** `java.util.Stack extends Vector` (so a stack lets you insert in the middle) and `java.util.Properties extends Hashtable` (so you can put non-String values). *Effective Java* cites both.
- **`java.io` streams** — `new BufferedInputStream(new FileInputStream(f))` composes behaviour (Decorator pattern, Chapter 9).
- **`Collections.unmodifiableList(list)` / `synchronizedList(list)`** wrap a list instead of subclassing each list type.
- **Game engines (Unity)** use an Entity–Component model: a game object *has* components (physics, renderer) instead of deep inheritance trees.

In the interview

30-second answer: "Inheritance couples a subclass to its parent's implementation, is fixed at compile time and explodes combinatorially. Composition holds a reference to an interface and delegates, so behaviour can be swapped at runtime. I use inheritance only for true is-a relationships with classes designed for extension."

Follow-ups:

- *Q: Which patterns are built on composition?* A: Strategy, Decorator, Composite, State, Bridge, Proxy, Adapter (object form) — most GoF patterns.

- *Q: Give a concrete inheritance failure.* A: The counting `HashSet` above, or `Stack extends Vector` exposing `insertElementAt`.
- *Q: When is inheritance OK?* A: When substitutable (LSP holds) and the parent documents its self-use or provides abstract hooks.

Self-check

1. Q: What is the fragile base class problem? → A: Subclasses breaking because the parent's internal implementation changed.
2. Q: What does "forwarding" mean? → A: The wrapper's method calls the same method on the wrapped object.
3. Q: Why did `CountingHashSet` count 6? → A: The inherited `addAll` calls the overridden `add`.

TL;DR

Reuse by holding and delegating ("has-a"); reserve "is-a" for real substitutable subtypes designed for extension.

---

7. Program to an Interface, Not an Implementation

What it is

Declare variables, fields, parameters and return types using the **most general supertype** (a parent type higher up the hierarchy) that offers what you need (an interface or abstract class), not a concrete class. "Interface" here means *supertype*, not only the Java `interface` keyword.

Analogy: a phone charger is designed for the USB-C *standard*, not for one specific phone model. Any USB-C phone works.

The problem it solves

Code written against a concrete class (`ArrayList`, `MySqlUserStore`) can only ever work with that class. You cannot test it without the real thing (e.g. a MySQL server), cannot swap implementations and cannot reuse it with other types. Changing the implementation means editing every caller.

How it works

1. Identify the **role** a collaborator plays ("stores users", "a collection of names").
2. Express the role as an interface with only the needed methods (`UserRepository.findById`).
3. Type fields/parameters by that interface (`List<String>`, `Collection<String>`, `UserRepository`).
4. Create concrete objects only at the edges — in `main`, a factory (an object or method whose job is creating other objects), or a DI container (a framework component, such as Spring's, that builds objects and wires their dependencies automatically) — and pass them in (**Dependency Injection**: giving an object its collaborators from outside instead of letting it `new` them).
5. For parameters, accept the widest type you can (`Collection` beats `List` beats `ArrayList`); for return types, return an interface but a specific enough one to be useful.

Java example — before/after

```
// File: lld/ch05/interfaces/Main.java
package lld.ch05.interfaces;

import java.util.ArrayList;
import java.util.Collection;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.TreeSet;

/** Program to an interface (a supertype), not an implementation. */
public class Main {

    record User(String id, String name) {}

    // ---------------- BEFORE: every signature names a concrete class ----------------
    static class Before {
        /** Pretend this talks to a real MySQL server -- impossible to use in a unit test. */
        static class MySqlUserStore {
            User load(String id) { throw new IllegalStateException("no MySQL server on this laptop"); }
        }
        static class GreetingService {
            private final MySqlUserStore store = new MySqlUserStore();   // hard-wired concrete dependency
            String greet(String id) { return "Hello, " + store.load(id).name(); }
        }
        static int countLongNames(ArrayList<String> names) {         // only accepts ArrayList
            int n = 0;
            for (String s : names) if (s.length() > 4) n++;
            return n;
        }
    }

    // ---------------- AFTER: depend on the supertype; pick the implementation at the edge ----------------
    static class After {
        interface UserRepository { Optional<User> findById(String id); }

        static final class InMemoryUserRepository implements UserRepository {
            private final Map<String, User> data = new HashMap<>();
            InMemoryUserRepository save(User u) { data.put(u.id(), u); return this; }
            @Override public Optional<User> findById(String id) { return Optional.ofNullable(data.get(id)); }
        }

        static final class GreetingService {
            private final UserRepository users;                  // any implementation
            GreetingService(UserRepository users) { this.users = users; }
            String greet(String id) {
                return users.findById(id).map(u -> "Hello, " + u.name()).orElse("Hello, stranger");
            }
        }

        static int countLongNames(Collection<String> names) {   // works for List, Set, Queue...
            return (int) names.stream().filter(s -> s.length() > 4).count();
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        try {
            System.out.println(new Before.GreetingService().greet("u1"));
        } catch (IllegalStateException e) {
            System.out.println("cannot even run it: " + e.getMessage());
        }
        System.out.println("long names in ArrayList: " + Before.countLongNames(new ArrayList<>(List.of("Anand",
"Li", "Priya"))));
        // Before.countLongNames(new TreeSet<>(...)) would not compile.
```

```
        System.out.println("== AFTER ==");
        After.UserRepository repo = new After.InMemoryUserRepository().save(new User("u1", "Asha"));
        var service = new After.GreetingService(repo);
        System.out.println(service.greet("u1"));
        System.out.println(service.greet("u2"));
        System.out.println("long names in List:    " + After.countLongNames(List.of("Anand", "Li", "Priya")));
        System.out.println("long names in TreeSet: " + After.countLongNames(new TreeSet<>(List.of("Anand", "Li",
"Priya"))));
    }
}
```

Output:

```
== BEFORE ==
cannot even run it: no MySQL server on this laptop
long names in ArrayList: 2
== AFTER ==
Hello, Asha
Hello, stranger
long names in List:    2
long names in TreeSet: 2
```

The *before* service creates its own `MySqlUserStore`, so it cannot even run on a laptop, and `countLongNames` accepts only `ArrayList`. The *after* depends on `UserRepository` and `Collection`; an in-memory implementation makes it runnable and testable, and any collection works.

Trade-offs

Interface-typed	Concrete-typed
Swappable, testable (fakes/mocks)	Simple, one less type
Callers can't use implementation-only methods	Access to everything (e.g. `ArrayList.trimToSize`)
Extra interface to maintain	Changes ripple to callers

When not to use it

- **Value types / data classes** (`String`, `LocalDate`, records) — no need for an interface.
- A class with exactly one implementation and no testing seam need — creating `FooService` + `FooServiceImpl` for everything is ceremony (see YAGNI). Add the interface when the second implementation or test double (any stand-in object used in tests: fake, stub or mock) appears.
- When you truly need implementation-specific behaviour (e.g. `LinkedHashMap` access order for an LRU cache (a *cache* keeps recently used data close at hand for fast reuse; LRU = evict the Least Recently Used entry when full) — then declare it as such).

Common mistakes

- Interfaces that mirror a class one-to-one with 30 methods (violates Interface Segregation, Chapter 4).
- Declaring `List` but then casting to `ArrayList` inside.
- Still calling `new ConcreteThing()` inside the class that uses the interface — the dependency is typed abstractly but still hard-wired.

Real-world examples

- **Java Collections Framework**: code uses `List`, `Map`, `Set`; `Collections.sort(List)` works on any list.
- **JDBC** (Java Database Connectivity — the standard Java API for talking to SQL databases): your code uses `java.sql.Connection`, `PreparedStatement`, `ResultSet` interfaces; the driver (MySQL, PostgreSQL) supplies the implementation.
- **SLF4J** (a logging *facade*: one logging API in front of many logging libraries): code logs through the `org.slf4j.Logger` interface; Logback or Log4j2 is plugged in at deployment.
- **Spring** injects beans (objects created and managed by the Spring container) by interface type (`@Autowired PaymentGateway gateway`).

In the interview

> **30-second answer:** "I type dependencies by roles — interfaces — and inject concrete implementations from the outside. It lets me swap implementations, e.g. an in-memory repository for tests, and keeps callers stable when implementations change. It's the practical form of the Dependency Inversion Principle."

Follow-ups:

- *Q: Relationship to DIP?* A: DIP says high-level modules depend on abstractions; programming to an interface is how you do it at each variable/field.
- *Q: Should every class have an interface?* A: No — only roles with multiple implementations or a need for test doubles.

Self-check

1. Q: What does "interface" mean in this principle? → A: Any supertype (interface or abstract class).
2. Q: Where should `new ConcreteClass()` appear? → A: At the edges: `main`, factories, DI configuration.

TL;DR

Depend on roles (`List`, `UserRepository`), not on concrete classes; choose the concrete class once, at the edge.

---

8. Encapsulate What Varies

What it is

Find the parts of your application that **change** (or differ between cases) and **separate them** from the parts that stay the same. Put the varying part behind its own abstraction so it can change without touching stable code. *Head First Design Patterns* calls this the first design principle; the GoF book phrases it as "encapsulate the concept that varies".

Analogy: a camera body stays the same while you swap lenses. The lens mount is the stable interface; lenses are the varying part.

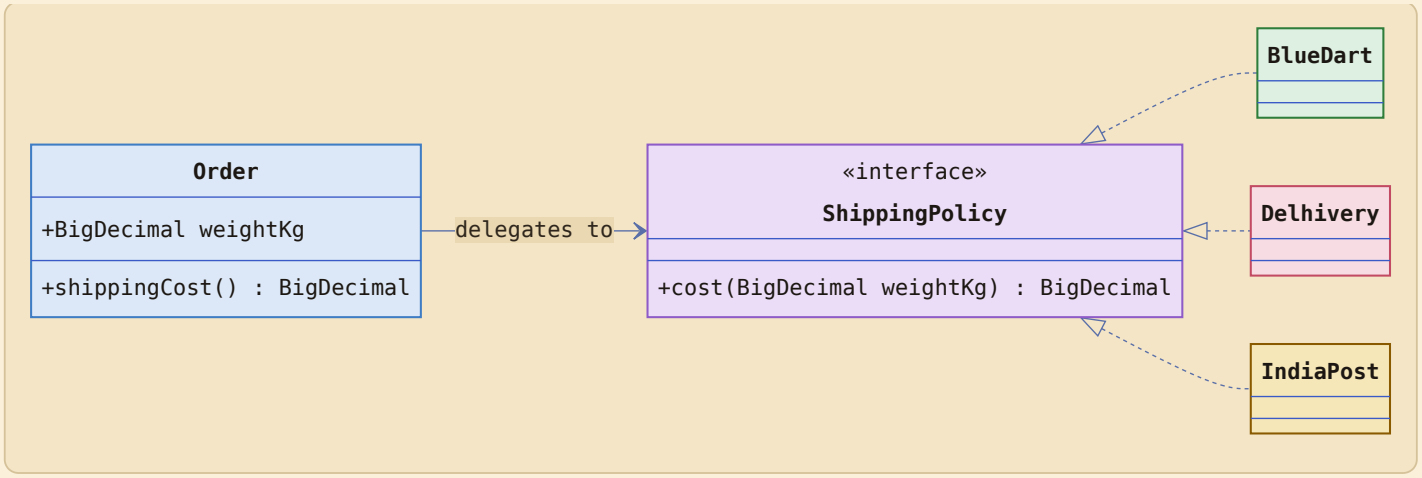
The problem it solves

When a volatile rule (shipping prices, tax, discounts, file formats) is written inline in a stable class, every change to the rule edits that class, re-tests it and risks breaking unrelated behaviour. Long `if/else` or `switch` chains on a "type" string grow forever (violating the Open/Closed Principle).

How it works

1. Ask: "Which parts of this class changed in the last few releases, or differ per customer/region/type?"
2. Extract that part behind an interface (or a functional interface — an interface with exactly one abstract method, which a lambda can implement).
3. Give the stable class a field of that interface type and delegate to it.
4. Add new variants as new implementations; the stable class does not change.

Almost every design pattern (a named, reusable solution to a recurring design problem; Chapters 8–11) is an application of this principle: Strategy (varying algorithm), State (varying state-dependent behaviour), Factory (varying creation), Observer (varying set of listeners), Template Method (varying steps).


Order
+BigDecimal weightKg
+shippingCost() : BigDecimal
delegates to
«interface»
ShippingPolicy
+cost(BigDecimal weightKg) : BigDecimal
BlueDart
Delhivery
IndiaPost

Java example — before/after

```
// File: lld/ch05/encapsulatevaries/Main.java
package lld.ch05.encapsulatevaries;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Map;

/** Encapsulate what varies: find the part that keeps changing (shipping rules) and put it behind its own
abstraction. */
public class Main {

    /** Rupee amount = base + perKg * weight, exact decimal arithmetic (money is never a double). */
    static BigDecimal linear(String base, String perKg, BigDecimal weightKg) {
        return new BigDecimal(base).add(new BigDecimal(perKg).multiply(weightKg)).setScale(2,
RoundingMode.HALF_UP);
    }

    // ---------------- BEFORE: the changing part is tangled into a stable class ----------------
    static class Before {
        static class Order {
            final BigDecimal weightKg;
            final String carrier;
            Order(BigDecimal weightKg, String carrier) { this.weightKg = weightKg; this.carrier = carrier; }

            BigDecimal shippingCost() {
                // every new carrier or price change edits Order -- a class that should be stable
                if (carrier.equals("BLUEDART")) return linear("50", "20", weightKg);
                else if (carrier.equals("DELHIVERY")) return linear("40", "25", weightKg);
                else if (carrier.equals("INDIA_POST")) return linear("30", "10", weightKg);
                throw new IllegalArgumentException("unknown carrier " + carrier);
            }
        }
    }

    // ---------------- AFTER: the varying part is a separate, swappable object ----------------
    static class After {
        @FunctionalInterface
        interface ShippingPolicy { BigDecimal cost(BigDecimal weightKg); }

        /** Stable catalogue of today's policies. Adding one = adding one entry; Order never changes. */
        static final Map<String, ShippingPolicy> POLICIES = Map.of(
                "BLUEDART", w -> linear("50", "20", w),
                "DELHIVERY", w -> linear("40", "25", w),
                "INDIA_POST", w -> linear("30", "10", w));

        record Order(BigDecimal weightKg, ShippingPolicy shipping) {
            BigDecimal shippingCost() { return shipping.cost(weightKg); }
        }
    }

    public static void main(String[] args) {
        BigDecimal twoKg = new BigDecimal("2");
        System.out.println("== BEFORE ==");
        for (String c : new String[]{"BLUEDART", "DELHIVERY", "INDIA_POST"})
            System.out.printf("%-10s %s%n", c, new Before.Order(twoKg, c).shippingCost());

        System.out.println("== AFTER ==");
        for (String c : new String[]{"BLUEDART", "DELHIVERY", "INDIA_POST"})
            System.out.printf("%-10s %s%n", c, new After.Order(twoKg, After.POLICIES.get(c)).shippingCost());
        // A brand new flat-rate promo, with zero edits to Order:
        System.out.printf("%-10s %s%n", "PROMO", new After.Order(twoKg, w -> new
BigDecimal("0.00")).shippingCost());
    }
}
```

Output:

```
== BEFORE ==
BLUEDART   90.00
DELHIVERY  90.00
INDIA_POST 50.00
== AFTER ==
BLUEDART   90.00
DELHIVERY  90.00
INDIA_POST 50.00
PROMO      0.00
```

In *after*, `Order` never changes when a carrier is added or re-priced; a brand-new policy (the promo) was added with zero edits to `Order`. The diagram draws named classes for clarity; the code uses lambdas implementing the same interface.

Trade-offs

Encapsulated	Inline
New variants without touching stable code	Everything in one place, easy to see
Each variant testable alone	Simple for 2 variants that never change
More types / indirection	Grows into long conditionals

When not to use it

When the "varying" thing has never varied and there is no sign it will (YAGNI). Two fixed branches are fine inline; extract when the third arrives or change becomes frequent.

Common mistakes

- Guessing the wrong axis of variation (extracting something that never changes while the real volatile part stays inline).
- Leaving the `switch` on type in the client code after introducing the strategy (just moving the conditional).

Real-world examples

- `java.util.Comparator` — sorting algorithm stable, comparison varies.
- `java.util.concurrent.ThreadPoolExecutor` takes a `RejectedExecutionHandler` (what to do when full varies) and a `ThreadFactory` (how threads are made varies).
- **Payment gateways** in e-commerce apps (Razorpay, Stripe, PayPal) behind one `PaymentGateway` interface.

In the interview

> **30-second answer:** "I identify what is likely to change — say pricing or payment method — and hide it behind an interface so the core classes stay closed to modification. It's the principle behind Strategy, State and most other patterns."

Follow-ups:

- *Q: How do you know what varies?* A: From requirements ("multiple payment methods"), from history (what changed often), and from the interviewer's hints.
- *Q: Relation to OCP?* A: Encapsulating the variation is how you make a class open for extension and closed for modification.

Self-check

1. Q: Which pattern encapsulates a varying algorithm? → A: Strategy.
2. Q: What code smell signals a missing encapsulation? → A: A growing `if/else` / `switch` on a type code.

TL;DR

Separate what changes from what stays the same; put the changing part behind an interface.

---

9. Strive for Loosely Coupled Designs

What it is

Loose coupling means objects that interact know as little as possible about each other — ideally only a small interface. *Head First Design Patterns* states it as: strive for loosely coupled designs between objects that interact.

Analogy: a radio station broadcasts; it doesn't know who is listening or what brand of radio they own. Listeners tune in or out freely.

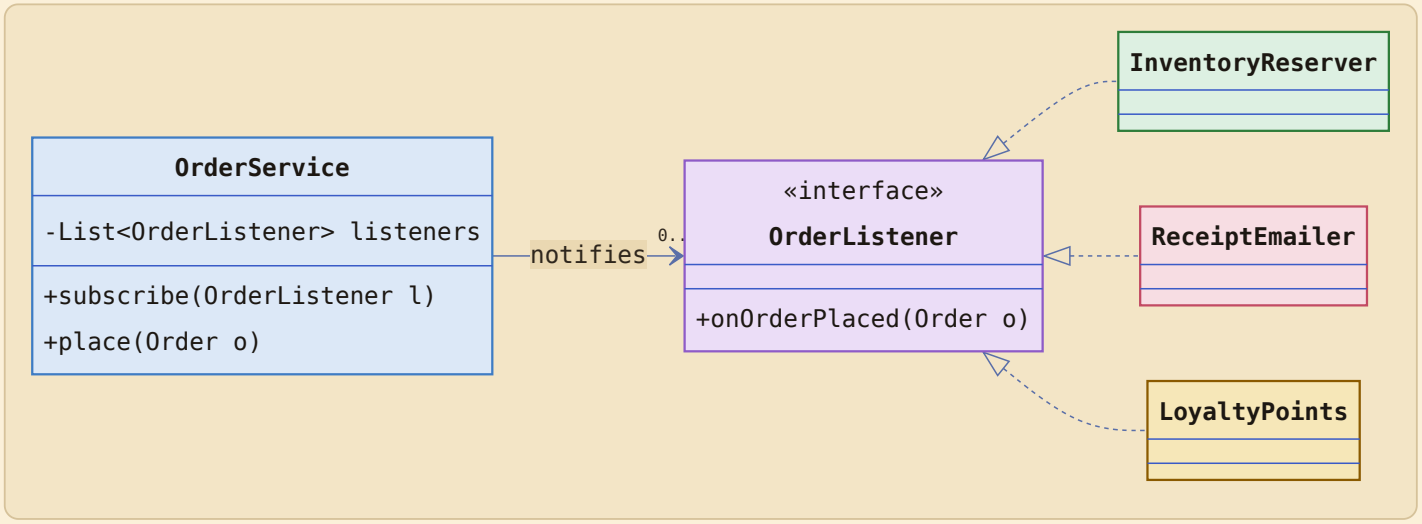
The problem it solves

In a **tightly coupled** design, a class creates and calls many concrete collaborators directly. Adding, removing or changing any collaborator edits that class. You can't reuse it without dragging all its dependencies along, and you can't test it in isolation.

How it works

1. Depend on **interfaces**, not concrete classes.
2. **Inject** dependencies instead of creating them inside.
3. Communicate through **events or messages** when the sender shouldn't know the receivers (Observer / publish-subscribe — senders publish events and any number of subscribers receive them without knowing each other).
4. Keep interfaces **narrow** (a single method is ideal for listeners).
5. Pass data as immutable values (records) rather than shared mutable objects.

Kinds of coupling from tightest to loosest (a common classification): *content* (reaching into another's internals) → *common* (shared global data) → *external* (sharing an externally imposed format, protocol or device interface) → *control* (passing flags that steer the other's logic) → *stamp* (passing a big structure when only part is needed) → *data* (passing only needed values) → *message* (communicating only through messages/events).



Java example — before/after

```
// File: lld/ch05/loosecoupling/Main.java
package lld.ch05.loosecoupling;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

/** Strive for loosely coupled designs between objects that interact. */
public class Main {

    record Order(String id, String email, int items) {}

    // ---------------- BEFORE: OrderService knows every class that cares about an order ----------------
    static class Before {
        static class EmailSender { void sendReceipt(Order o) { System.out.println("  email receipt to " +
o.email()); } }
        static class Inventory { void reserve(Order o) { System.out.println("  reserve " + o.items() + " items"); }
}
        static class Analytics { void track(Order o) { System.out.println("  track order " + o.id()); } }

        static class OrderService {
            private final EmailSender email = new EmailSender();   // concrete, created inside
            private final Inventory inventory = new Inventory();
            private final Analytics analytics = new Analytics();
            void place(Order o) {
                System.out.println("placed " + o.id());
                inventory.reserve(o);
                email.sendReceipt(o);
                analytics.track(o);          // adding "loyalty points" means editing this class
            }
        }
    }

    // ---------------- AFTER: OrderService only knows a tiny listener interface ----------------
    static class After {
        @FunctionalInterface
        interface OrderListener { void onOrderPlaced(Order o); }

        static final class OrderService {
            private final List<OrderListener> listeners = new CopyOnWriteArrayList<>();
            void subscribe(OrderListener l) { listeners.add(l); }
            void place(Order o) {
                System.out.println("placed " + o.id());
                listeners.forEach(l -> l.onOrderPlaced(o));   // doesn't know who listens or what they do
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        new Before.OrderService().place(new Order("o-1", "asha@example.com", 2));

        System.out.println("== AFTER ==");
        var service = new After.OrderService();
        service.subscribe(o -> System.out.println("  reserve " + o.items() + " items"));
        service.subscribe(o -> System.out.println("  email receipt to " + o.email()));
        service.subscribe(o -> System.out.println("  track order " + o.id()));
        service.subscribe(o -> System.out.println("  +" + o.items() * 10 + " loyalty points"));  // new, no edits
        service.place(new Order("o-2", "ravi@example.com", 3));
    }
}
```

Output:

```
== BEFORE ==
placed o-1
  reserve 2 items
  email receipt to asha@example.com
  track order o-1
== AFTER ==
placed o-2
  reserve 3 items
  email receipt to ravi@example.com
  track order o-2
  +30 loyalty points
```

The *after* `OrderService` depends on one tiny interface. Adding "loyalty points" needed zero edits to it. A `CopyOnWriteArrayList` (a thread-safe list that copies its internal array on every write) is used so listeners can subscribe safely even while notifications run on other threads.

Trade-offs

Loose	Tight
Change and test parts independently	Easy to follow control flow
Add features by adding listeners	Direct calls are faster to trace in a debugger
Flow is less visible ("who handles this event?")	Every change edits the central class
Error handling/ordering between listeners must be designed	Order is explicit

When not to use it

- Inside a small, cohesive module where classes always change together — decoupling them adds ceremony.
- When the order and atomicity (all steps happen, or none do) of steps is critical (e.g. debit then credit in a transfer) — explicit orchestration is clearer and safer than fire-and-forget events.

Common mistakes

- "Decoupling" with a global event bus (one shared channel that every component publishes to and listens on) for everything, making the flow impossible to follow.
- Hidden coupling via shared mutable static state.
- Listeners that throw and stop the other listeners from running (decide a policy: catch and log per listener).

Real-world examples

- **Swing / JavaFX / Android** event listeners (`ActionListener`, `OnClickListener`).
- **Spring** `ApplicationEventPublisher` and `@EventListener`.
- **Apache Kafka** (a distributed *message broker*: producers append messages to named *topics*, and *consumer groups* read them at their own pace; see System Design Fundamentals) — producers write to topics without knowing which consumer groups read them; an order service at an e-commerce company publishes `OrderPlaced`, and inventory, notifications and analytics each consume independently.

In the interview

> **30-second answer:** "Loose coupling means collaborators know only small interfaces about each other. I achieve it with interfaces, dependency injection and events. In the order example, the order service just publishes 'order placed' and new features subscribe without touching it."

Follow-ups:

- *Q: Coupling vs cohesion?* A: Coupling is between modules (want low); cohesion is within a module (want high).
- *Q: Downside of event-driven decoupling?* A: Harder to trace flows, ordering and failure handling; needs good logging and clear contracts.

Self-check

1. Q: Which pattern is the classic loose-coupling tool for one-to-many notifications? → A: Observer.

2. Q: Name two techniques for loose coupling. → A: Depend on interfaces; inject dependencies (also: events, narrow interfaces).

TL;DR

Let interacting objects share only small interfaces; inject them and use events so parts change independently.

---

10. The Hollywood Principle (Inversion of Control)

What it is

"Don't call us, we'll call you." High-level components (a framework or base class) control the flow and **call** low-level components at the right moments. Low-level components **hook into** the system but never call the high-level components directly. This reversal of who-calls-whom is called **Inversion of Control (IoC)**.

Analogy: actors audition and leave their number; the studio calls them when there's a role. Actors don't phone the studio every hour.

Library vs framework: you call a library; a framework calls you. That is IoC.

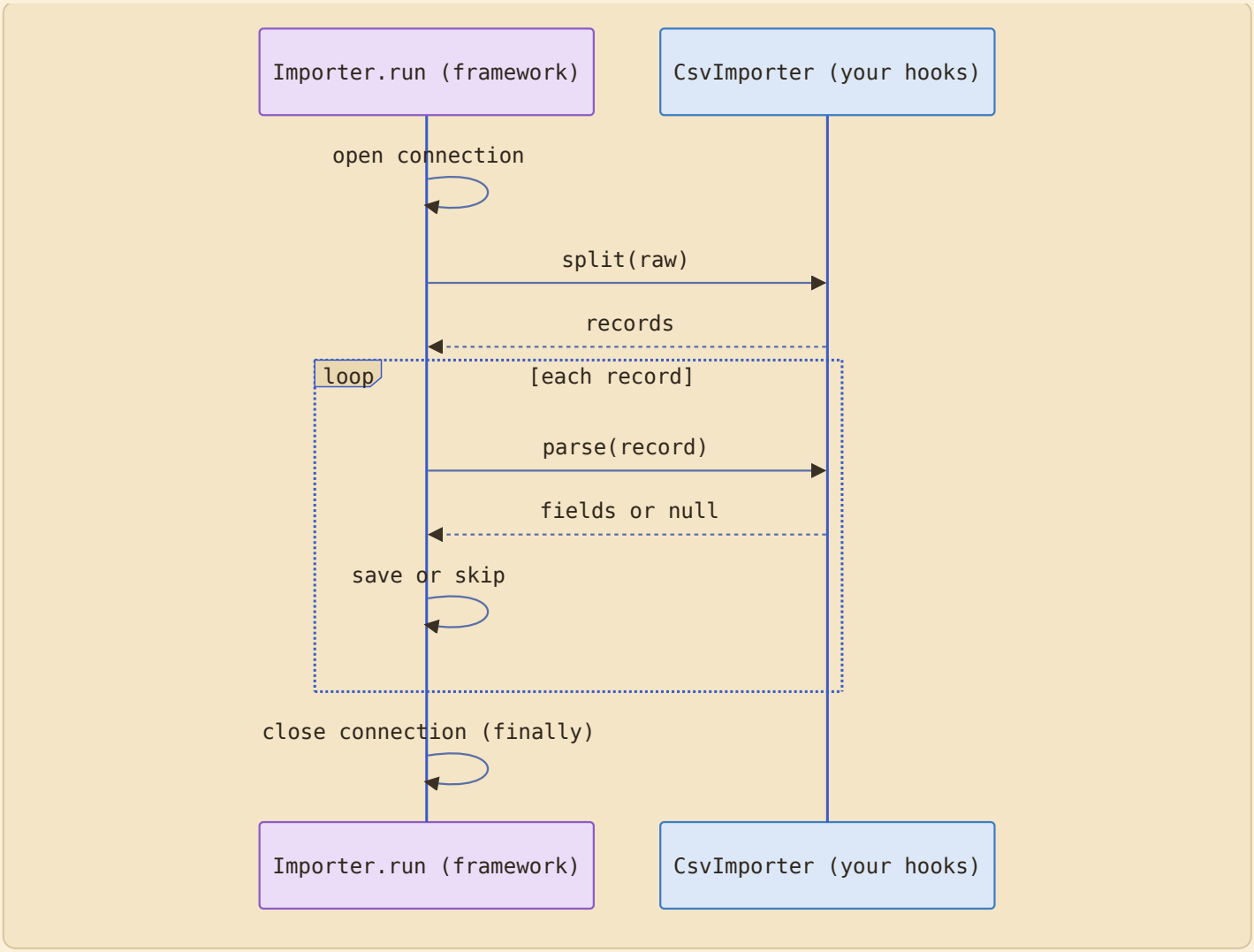
The problem it solves

Without it, every component drives the whole process itself: it re-implements the algorithm's skeleton, decides the order of steps and calls shared services directly. You get duplicated control flow, forgotten steps (like closing a database connection — the open session a program holds with a database) and **dependency rot** — high-level and low-level components depending on each other in tangled ways.

How it works

There are several ways to invert control:

1. **Template Method** — a base class defines a `final` algorithm and calls abstract "hook" methods that subclasses fill in.
2. **Callbacks / listeners** — you register a function; the system calls it on an event (Observer).
3. **Dependency Injection** — a container creates your objects and passes them their dependencies; your class never looks them up.
4. **Lifecycle management** — a container starts/stops your components (`@PostConstruct`, servlet `init`/`destroy`; a *servlet* is a Java class that handles web requests inside a server such as Tomcat).


Importer.run (framework)
CsvImporter (your hooks)
open connection
split(raw)
records
loop
[each record]
parse(record)
fields or null
save or skip
close connection (finally)
Importer.run (framework)
CsvImporter (your hooks)

Java example — before/after

```java
// File: lld/ch05/hollywood/Main.java
package lld.ch05.hollywood;

import java.util.ArrayList;
import java.util.List;

/** Hollywood Principle: "Don't call us, we'll call you." The high-level framework owns the flow and calls your
hooks. */
public class Main {

    // ---------------- BEFORE: each importer re-implements the whole flow and drives it itself ----------------
    static class Before {
        static class CsvImporter {
            void run(String raw) {
                System.out.println("  open connection");              // step order copied into every importer
                for (String row : raw.split(";")) {
                    String[] f = row.split(",");
                    if (f.length != 2) { System.out.println("  skip bad row '" + row + "'"); continue; }
                    System.out.println("  save " + f[0] + "=" + f[1]);
                }
                // forgot to close the connection -- nothing forced it
            }
        }
    }

    // ---------------- AFTER: Template Method -- the framework calls you ----------------
    static class After {
        abstract static class Importer {
            /** The fixed algorithm. final = subclasses cannot reorder or skip steps. */
            public final void run(String raw) {
                System.out.println("  open connection");
                try {
                    for (String record : split(raw)) {
                        String[] fields = parse(record);
                        if (fields == null) { System.out.println("  skip bad record '" + record + "'"); continue; }
                        System.out.println("  save " + fields[0] + "=" + fields[1]);
                    }
                } finally {
                    System.out.println("  close connection");       // guaranteed by the framework
                }
            }
            protected abstract List<String> split(String raw);       // hooks: WE call YOU
            protected abstract String[] parse(String record);
        }

        static final class CsvImporter extends Importer {
            @Override protected List<String> split(String raw) { return List.of(raw.split(";")); }
            @Override protected String[] parse(String r) { String[] f = r.split(","); return f.length == 2 ? f :
null; }
        }
        static final class KeyValueImporter extends Importer {
            @Override protected List<String> split(String raw) { return List.of(raw.split("\n")); }
            @Override protected String[] parse(String r) { String[] f = r.split("="); return f.length == 2 ? f :
null; }
        }

        /** A tiny IoC "container": components register, the container decides when to start/stop them. */
        interface Lifecycle { void start(); void stop(); }
        static final class Container {
            private final List<Lifecycle> components = new ArrayList<>();
            void register(Lifecycle c) { components.add(c); }
            void boot() { components.forEach(Lifecycle::start); }
            void shutdown() { components.reversed().forEach(Lifecycle::stop); }  // reverse order, like Spring
        }
    }
```

```
    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        new Before.CsvImporter().run("a,1;b,2;oops");
        System.out.println("== AFTER ==");
        new After.CsvImporter().run("a,1;b,2;oops");
        new After.KeyValueImporter().run("x=9\ny=8");

        var container = new After.Container();
        for (String name : List.of("database", "cache", "web")) {
            container.register(new After.Lifecycle() {
                public void start() { System.out.println("  start " + name); }
                public void stop() { System.out.println("  stop " + name); }
            });
        }
        container.boot();
        container.shutdown();
    }
}
```

Output:

```
== BEFORE ==
  open connection
  save a=1
  save b=2
  skip bad row 'oops'
== AFTER ==
  open connection
  save a=1
  save b=2
  skip bad record 'oops'
  close connection
  open connection
  save x=9
  save y=8
  close connection
  start database
  start cache
  start web
  stop web
  stop cache
  stop database
```

The *before* importer drives the flow and forgot to close the connection. In *after*, `Importer.run` is `final`: subclasses only supply `split` and `parse`, and the framework guarantees the close. The tiny `Container` shows IoC for lifecycles: components never start themselves; the container starts them in order and stops them in reverse order (the same idea Spring uses when destroying singletons — objects of which only one instance exists in the application).

Trade-offs

IoC	Direct control
Framework guarantees flow, ordering, cleanup	You see the whole flow in your code
Less duplication across components	Duplicated skeletons
"Magic": harder to debug who calls what	Explicit, simple for tiny programs
Components tied to the framework's hooks	Independent

When not to use it

For small scripts or one-off logic where there is no repeated skeleton — a framework or template adds indirection with nothing to share.

Common mistakes

- Letting low-level hooks call back up into the framework (e.g. a subclass calling `run()` recursively).
- Forgetting `final` on the template method, so subclasses override the skeleton.
- Confusing IoC (the general idea) with DI (one specific form of it).

Real-world examples

- **JUnit** (the standard Java unit-testing framework) discovers and calls your `@Test` methods; you never call them.
- **Servlets**: the container calls `doGet`/`doPost` (handlers for HTTP GET and POST requests); **Spring MVC** calls your `@GetMapping` methods.
- **Spring IoC container** creates beans, injects dependencies and calls `@PostConstruct`/`@PreDestroy`.
- **Android** calls `Activity.onCreate`, `onResume`, `onDestroy`.
- **`java.util.AbstractList`** — you implement `get` and `size`; the base class implements iteration, `indexOf`, `equals` by calling your methods.
- **Spring `JdbcTemplate`** runs the connection/statement/cleanup skeleton and calls your `RowMapper`.

In the interview

> **30-second answer:** "The Hollywood Principle — 'don't call us, we'll call you' — means the high-level component controls the flow and calls low-level hooks. It's what separates a framework from a library. Template Method, callbacks and dependency injection are all ways to apply it, and it prevents low-level parts from depending on high-level ones."

Follow-ups:

- *Q: IoC vs DI?* A: IoC is the general reversal of control; DI is IoC applied to obtaining dependencies (they're given to you rather than looked up).
- *Q: Hollywood vs Dependency Inversion?* A: DIP is about depending on abstractions; Hollywood is about who calls whom. They often appear together.

Self-check

1. Q: In Template Method, who calls whom? → A: The base class's template method calls the subclass hooks.
2. Q: Why make the template method `final`? → A: So subclasses cannot reorder or skip steps.
3. Q: Library or framework: which one calls your code? → A: Framework.

TL;DR

Let the framework own the flow and call your hooks; low-level parts plug in and never call up.

---

11. Separation of Concerns (SoC)

What it is

A **concern** is one aspect of what a program does: reading input, business rules, storage, presentation, security, logging. **Separation of Concerns** means each part of the program deals with one concern, so you can think about, change and test it without thinking about the others. Edsger Dijkstra coined the phrase in his 1974 essay "On the role of scientific thought" (EWD 447).

Analogy: in a restaurant the waiter takes orders, the chef cooks, the cashier bills. If one person did all three at once, every change (a new menu, a new payment method) would disrupt everything.

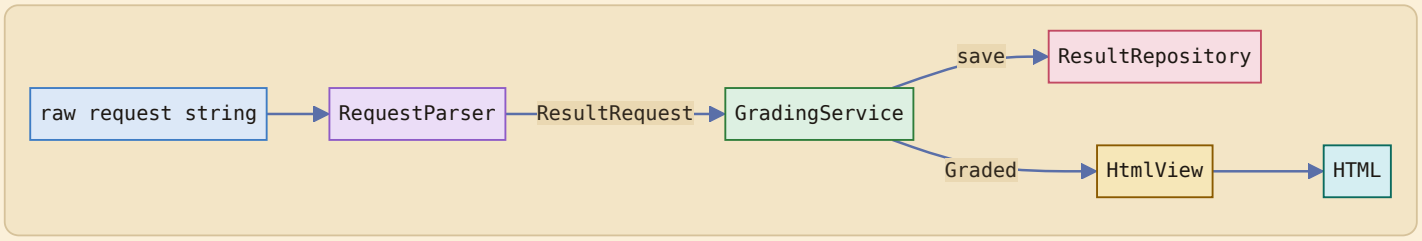
SoC is the **big-picture** version of the Single Responsibility Principle: SRP is about one class; SoC applies at every level — functions, classes, layers (MVC — Model–View–Controller, which keeps data/rules, display and input handling apart; controller/service/repository — a common backend layering where the controller receives web requests, the service holds business rules and the repository talks to the database), services and even languages (HTML for structure, CSS for style, JS for behaviour).

The problem it solves

Mixed concerns create code where you can't change the HTML without risking the grading rule, can't test the rule without parsing strings, and can't switch storage without editing presentation. Everything is tangled — *spaghetti code*.

How it works

1. List the concerns in the feature (input parsing, validation/rules, persistence, output format).
2. Give each its own unit (function → class → layer), with clear inputs and outputs (typed objects, not raw strings).
3. Let data flow in one direction: parse → apply rules → store → present.
4. Make lower layers not know about higher ones (the service doesn't know about HTML).
5. Handle **cross-cutting concerns** (logging, security, transactions — a *transaction* is a group of database changes that succeed or fail as one) with dedicated mechanisms — filters, decorators, proxies (stand-in objects that intercept calls to the real one) or aspects (AOP, aspect-oriented programming: write code such as logging once and have it applied around many methods automatically) — rather than sprinkling them everywhere (see Chapter 6, *Systems*).



Java example — before/after

```java
// File: lld/ch05/soc/Main.java
package lld.ch05.soc;

import java.util.HashMap;
import java.util.Map;

/** Separation of Concerns: parsing, business rules, storage and presentation each live in their own place. */
public class Main {

    // ---------------- BEFORE: one method does input parsing, rules, persistence AND HTML ----------------
    static class Before {
        static final Map<String, Integer> DB = new HashMap<>();
        static String handle(String request) {                    // "name=Asha&marks=87"
            String name = null; int marks = -1;
            for (String kv : request.split("&")) {
                String[] p = kv.split("=");
                if (p[0].equals("name")) name = p[1];
                if (p[0].equals("marks")) marks = Integer.parseInt(p[1]);
            }
            if (marks < 0 || marks > 100) return "<p style='color:red'>bad marks</p>";
            String grade = marks >= 90 ? "A" : marks >= 75 ? "B" : marks >= 50 ? "C" : "F";
            DB.put(name, marks);
            return "<p>" + name + " got " + grade + "</p>";
        }
    }

    // ---------------- AFTER: each concern is a separate class with one job ----------------
    static class After {
        record ResultRequest(String name, int marks) {}
        record Graded(String name, int marks, String grade) {}

        /** Concern 1: turn raw input into a typed request. */
        static final class RequestParser {
            ResultRequest parse(String raw) {
                Map<String, String> m = new HashMap<>();
                for (String kv : raw.split("&")) { String[] p = kv.split("=", 2); m.put(p[0], p[1]); }
                return new ResultRequest(m.get("name"), Integer.parseInt(m.get("marks")));
            }
        }
        /** Concern 2: business rules only. No HTML, no storage. */
        static final class GradingService {
            private final ResultRepository repo;
            GradingService(ResultRepository repo) { this.repo = repo; }
            Graded record(ResultRequest r) {
                if (r.marks() < 0 || r.marks() > 100) throw new IllegalArgumentException("marks must be 0..100");
                String grade = r.marks() >= 90 ? "A" : r.marks() >= 75 ? "B" : r.marks() >= 50 ? "C" : "F";
                repo.save(r.name(), r.marks());
                return new Graded(r.name(), r.marks(), grade);
            }
        }
        /** Concern 3: storage. */
        interface ResultRepository { void save(String name, int marks); int count(); }
        static final class InMemoryResultRepository implements ResultRepository {
            private final Map<String, Integer> rows = new HashMap<>();
            public void save(String name, int marks) { rows.put(name, marks); }
            public int count() { return rows.size(); }
        }
        /** Concern 4: presentation. Could be swapped for JSON without touching rules. */
        static final class HtmlView {
            String render(Graded g) { return "<p>" + g.name() + " got " + g.grade() + "</p>"; }
            String error(String msg) { return "<p style='color:red'>" + msg + "</p>"; }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
```

```
        System.out.println(Before.handle("name=Asha&marks=87"));
        System.out.println(Before.handle("name=Ravi&marks=140"));

        System.out.println("== AFTER ==");
        var repo = new After.InMemoryResultRepository();
        var parser = new After.RequestParser();
        var service = new After.GradingService(repo);
        var view = new After.HtmlView();
        for (String raw : new String[]{"name=Asha&marks=87", "name=Ravi&marks=140"}) {
            try {
                System.out.println(view.render(service.record(parser.parse(raw))));
            } catch (IllegalArgumentException e) {
                System.out.println(view.error(e.getMessage()));
            }
        }
        System.out.println("rows stored: " + repo.count());
    }
}
```

Output:

```
== BEFORE ==
<p>Asha got B</p>
<p style='color:red'>bad marks</p>
== AFTER ==
<p>Asha got B</p>
<p style='color:red'>marks must be 0..100</p>
rows stored: 1
```

Each *after* class can now be tested alone: `GradingService` with an in-memory repository and no strings to parse; `HtmlView` could be replaced with a JSON view without touching the rules.

Trade-offs

Separated	Mixed
Change one concern safely	Quick to write for a one-off
Parallel team work (UI vs backend)	Everything visible in one place
More types and wiring	Tangled, hard to test

When not to use it

Tiny throwaway scripts. Also avoid over-layering (five layers that just pass data through — "lasagna code").

Common mistakes

- Business rules leaking into controllers or SQL.
- Separating by *technical type* only (all DTOs in one package) while one feature's logic is scattered — prefer separating by concern and cohesive feature packages.

Real-world examples

- **MVC** (Model–View–Controller) in Spring MVC, Rails, ASP.NET.
- **HTML / CSS / JavaScript** separation on the web.
- **TCP/IP network layers** (the stack of protocols the internet runs on, from physical links up through IP addressing, TCP's reliable delivery and HTTP): each layer handles one concern and talks only to its neighbours.
- **Spring AOP / servlet filters** separate cross-cutting concerns (security, transactions, logging) from business code.

In the interview

30-second answer: "Separation of Concerns means each module handles one aspect — parsing, rules, storage, presentation — so each can change and be tested independently. In LLD I show it as layers: controller,

> service, repository, with domain objects in the middle."

Follow-ups:

- *Q: SoC vs SRP?* A: SRP is SoC applied to a single class — one reason to change.
- *Q: What is a cross-cutting concern?* A: A concern that touches many modules (logging, auth, transactions); handled by AOP, proxies, decorators or filters.

Self-check

1. Q: Name four concerns in a typical web request. → A: Parsing input, business rules, persistence, presentation.
2. Q: Who coined the term? → A: Edsger Dijkstra (1974).

TL;DR

Give each concern its own home and let them communicate through clear, typed interfaces.

---

12. Principle of Least Astonishment (POLA)

What it is

Also called the **Principle of Least Surprise**: a component should behave the way most of its users *expect* from its name, signature and context. If a behaviour would surprise a reasonable reader, change the behaviour or the name.

Analogy: a light switch that also locks the front door would be "correct" according to its designer, but it surprises everybody.

The problem it solves

Surprising code causes bugs even when it "works as designed", because callers use it based on expectations. Typical surprises: methods named like queries that change state, methods that mutate and return `this` so aliases (other variables pointing to the same object) change, inconsistent naming (`size()` vs `length()` vs `count()` in one API), 0-based vs 1-based confusion, and silently swallowed errors.

How it works

1. Make names tell the truth: `find` / `get` / `is` must not modify state; if it creates, say so (`getOrCreate`).
2. Follow language and platform conventions (the `equals`/`hashCode` contract — equal objects must return the same *hash code*, the number hash-based collections like `HashMap` use to locate them — Java naming, `close()` via `AutoCloseable`).
3. Be consistent within your API — same word for same concept.
4. Prefer immutable results and no hidden side effects.
5. Signal "maybe absent" in the type (`Optional`, a box that either holds a value or is empty), not with a surprise `null` or a magic value.
6. Document any unavoidable surprise prominently.

Java example — before/after

```
// File: lld/ch05/leastastonishment/Main.java
package lld.ch05.leastastonishment;

import java.math.BigDecimal;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/** Principle of Least Astonishment: code should behave the way its name and its readers expect. */
public class Main {

    // ---------------- BEFORE: surprising behaviour hidden behind innocent names ----------------
    static class Before {
        static class Money {
            long paise;
            Money(long paise) { this.paise = paise; }
            Money add(Money other) { this.paise += other.paise; return this; }   // looks pure, actually mutates
            @Override public String toString() { return "Rs " + BigDecimal.valueOf(paise, 2); }   // exact, no
double
        }
        static class UserDirectory {
            final Map<String, String> users = new HashMap<>();
            String findUser(String id) {                        // "find" silently CREATES a user
                return users.computeIfAbsent(id, k -> "new-user-" + k);
            }
        }
    }

    // ---------------- AFTER: names tell the truth; no hidden side effects ----------------
    static class After {
        record Money(long paise) {
            Money plus(Money other) { return new Money(paise + other.paise); }   // returns a NEW value
            @Override public String toString() { return "Rs " + BigDecimal.valueOf(paise, 2); }   // exact, no
double
        }
        static class UserDirectory {
            private final Map<String, String> users = new HashMap<>();
            Optional<String> findUser(String id) { return Optional.ofNullable(users.get(id)); }
            String getOrCreateUser(String id) { return users.computeIfAbsent(id, k -> "new-user-" + k); }
            int size() { return users.size(); }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        var price = new Before.Money(10_000);
        var withTip = price.add(new Before.Money(1_000));
        System.out.println("price=" + price + " withTip=" + withTip + "   <-- price changed too!");
        var dir = new Before.UserDirectory();
        dir.findUser("ghost");
        System.out.println("users after a *find*: " + dir.users.size());

        System.out.println("== AFTER ==");
        var p = new After.Money(10_000);
        var t = p.plus(new After.Money(1_000));
        System.out.println("price=" + p + " withTip=" + t);
        var d = new After.UserDirectory();
        System.out.println("find ghost -> " + d.findUser("ghost") + ", users=" + d.size());
        System.out.println("getOrCreate ghost -> " + d.getOrCreateUser("ghost") + ", users=" + d.size());

        System.out.println("== Real JDK surprises (good to know) ==");
        System.out.println("new BigDecimal(\"2.0\").equals(new BigDecimal(\"2.00\")) = "
                + new BigDecimal("2.0").equals(new BigDecimal("2.00")));
        System.out.println("new BigDecimal(\"2.0\").compareTo(new BigDecimal(\"2.00\")) = "
```

```
                + new BigDecimal("2.0").compareTo(new BigDecimal("2.00")));
        List<String> fixed = Arrays.asList("a", "b");
        try { fixed.add("c"); } catch (UnsupportedOperationException e) {
            System.out.println("Arrays.asList(...).add -> UnsupportedOperationException (fixed-size list)");
        }
    }
}
```

Output:

```
== BEFORE ==
price=Rs 110.00 withTip=Rs 110.00   <-- price changed too!
users after a *find*: 1
== AFTER ==
price=Rs 100.00 withTip=Rs 110.00
find ghost -> Optional.empty, users=0
getOrCreate ghost -> new-user-ghost, users=1
== Real JDK surprises (good to know) ==
new BigDecimal("2.0").equals(new BigDecimal("2.00")) = false
new BigDecimal("2.0").compareTo(new BigDecimal("2.00")) = 0
Arrays.asList(...).add -> UnsupportedOperationException (fixed-size list)
```

The last block shows *real* JDK behaviour that surprises many developers: `BigDecimal.equals` compares scale as well as value (`2.0` ≠ `2.00`), while `compareTo` says they are equal; and `Arrays.asList` returns a fixed-size list backed by the array. These are documented, but they are classic interview gotchas — use `compareTo` for numeric equality and `new ArrayList<>(Arrays.asList(...))` or `List.of(...)` depending on whether you need mutability.

Trade-offs

Following POLA sometimes means a longer name (`getOrCreateUser`) or an extra method. That's cheap compared to the bugs surprises cause. The only real tension: what's "expected" depends on the audience — follow the conventions of *your* platform and team.

When not to use it

There's no case where surprising users is good. But don't twist a design to match a *wrong* expectation; instead, fix the name so expectations become correct.

Common mistakes

- Getters with side effects (lazy creation, logging to DB, incrementing counters).
- Overloading `equals` (writing `equals(Money m)`) instead of overriding `equals(Object o)` — the collection classes silently ignore it.
- Methods whose behaviour changes based on a hidden global flag.

Real-world examples

- **`java.time`** was designed to fix the surprises of `java.util.Date`/`Calendar` (mutable dates, months numbered 0–11).
- `java.net.URL.equals` may perform DNS resolution (a network lookup that turns a host name into an IP address) (documented) — a notorious surprise; `java.net.URI` is usually preferred as a map key.
- **Unix** `rm` has no undo; tools like `trash` exist to reduce astonishment for new users.
- UI design: "Save" always in the same place; "Cancel" never saves.

In the interview

> **30-second answer:** "Least astonishment means an API should do what its name and conventions suggest. Queries don't mutate, creation is named explicitly, absence is typed with Optional, and value objects are immutable. Surprises like `BigDecimal.equals` considering scale are exactly what we try not to create."

Follow-ups:

- *Q: Example of violating it in Java?* A: A `get` method that lazily writes to the database, or `Money.add` mutating `this`.

- *Q: How does CQS help?* A: Command-Query Separation (Chapter 6) makes it obvious which methods change state.

Self-check

1. Q: What should `findUser` do if the user doesn't exist? → A: Return an empty `Optional` (or throw a documented exception), never silently create one.
2. Q: Why is `a.add(b)` returning a mutated `this` surprising? → A: Other references to `a` change unexpectedly (aliasing).

TL;DR

Do what the name promises and nothing more; follow conventions; make absence and side effects visible.

---

13. Fail Fast

What it is

A **fail-fast** system detects an invalid state or input **immediately**, at the point where it arises, and stops with a clear error — rather than carrying on and failing mysteriously later (**fail slow / fail silently**).

Analogy: a smoke alarm that beeps at the first smoke, not after the house is on fire.

The problem it solves

If a `null` endpoint (the URL of a service to call) or a negative retry count is accepted in a constructor, the crash happens hours later, deep in unrelated code, with a stack trace (the list of method calls that were active when the error happened) that points to the symptom, not the cause. Bad data may also be saved and corrupt other data before anyone notices. The longer the gap between cause and failure, the more expensive the bug.

How it works

1. **Validate at boundaries**: constructors, public method entry, API/controller input, configuration load at startup.
2. Use **guard clauses** (checks at the top of a method that throw or return immediately on bad input) — `Objects.requireNonNull`, `if (bad) throw new IllegalArgumentException(...)` at the top of the method.
3. Throw specific exceptions with a helpful message (what was wrong, what value, what was expected).
4. Make invalid objects **unconstructable** (validation in constructor/factory + immutability).
5. Fail at **startup** for misconfiguration rather than on the first request.
6. Don't swallow exceptions (`catch (Exception e) {}`).

Fail fast vs fail safe: *fail-safe* means a failure leaves the system in a safe state (e.g. a lift's brakes engage on power loss). Interview literature (not the JDK documentation, which says "snapshot" or "weakly consistent") also calls **iterators** "fail-safe" when they never throw `ConcurrentModificationException` — `CopyOnWriteArrayList` iterates over a snapshot, `ConcurrentHashMap` iterators are weakly consistent — in contrast to *fail-fast* iterators (e.g. `ArrayList`) that throw `ConcurrentModificationException` when the collection is structurally modified during iteration. Note the JDK documents fail-fast iterators as **best-effort** — you must not rely on them for correctness.

Java example — before/after

```java
// File: lld/ch05/failfast/Main.java
package lld.ch05.failfast;

import java.time.Duration;
import java.util.ArrayList;
import java.util.ConcurrentModificationException;
import java.util.List;
import java.util.Objects;

/** Fail fast: detect a bad state at the earliest point, loudly, close to its cause. */
public class Main {

    // ---------------- BEFORE: garbage is accepted and blows up far away, much later ----------------
    static class Before {
        static class RetryConfig {
            final String endpoint; final int maxAttempts; final Duration backoff;
            RetryConfig(String endpoint, int maxAttempts, Duration backoff) {
                this.endpoint = endpoint; this.maxAttempts = maxAttempts; this.backoff = backoff;  // no checks
            }
        }
        static String callWithRetry(RetryConfig c) {
            // Hours later, deep inside business code:
            return "calling " + c.endpoint.toUpperCase() + " up to " + c.maxAttempts + " times, backoff "
                    + c.backoff.toMillis() + "ms";
        }
    }

    // ---------------- AFTER: validate at the boundary -- the object can never exist in a bad state ----------
--
    static class After {
        record RetryConfig(String endpoint, int maxAttempts, Duration backoff) {
            RetryConfig {
                Objects.requireNonNull(endpoint, "endpoint");
                Objects.requireNonNull(backoff, "backoff");
                if (maxAttempts < 1) throw new IllegalArgumentException("maxAttempts must be >= 1, was " +
maxAttempts);
                if (backoff.isNegative()) throw new IllegalArgumentException("backoff must not be negative");
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        var bad = new Before.RetryConfig(null, 0, null);   // constructed "successfully"
        System.out.println("config created, nothing complained...");
        try { Before.callWithRetry(bad); }
        catch (NullPointerException e) { System.out.println("much later: NullPointerException far from the real
mistake"); }

        System.out.println("== AFTER ==");
        try { new After.RetryConfig(null, 3, Duration.ofMillis(100)); }
        catch (NullPointerException e) { System.out.println("rejected at construction: " + e.getMessage()); }
        try { new After.RetryConfig("https://pay", 0, Duration.ofMillis(100)); }
        catch (IllegalArgumentException e) { System.out.println("rejected at construction: " + e.getMessage()); }
        System.out.println("ok: " + new After.RetryConfig("https://pay", 3, Duration.ofMillis(100)));

        System.out.println("== JDK fail-fast iterator ==");
        List<String> list = new ArrayList<>(List.of("a", "b", "c"));
        try {
            for (String s : list) if (s.equals("a")) list.remove(s);   // structural change during iteration
        } catch (ConcurrentModificationException e) {
            System.out.println("ArrayList iterator threw ConcurrentModificationException");
        }
        list.removeIf(s -> s.equals("b"));                              // the safe way
        System.out.println("after removeIf: " + list);
```

```
    }
}
```

Output:

```
== BEFORE ==
config created, nothing complained...
much later: NullPointerException far from the real mistake
== AFTER ==
rejected at construction: endpoint
rejected at construction: maxAttempts must be >= 1, was 0
ok: RetryConfig[endpoint=https://pay, maxAttempts=3, backoff=PT0.1S]
== JDK fail-fast iterator ==
ArrayList iterator threw ConcurrentModificationException
after removeIf: [c]
```

`RetryConfig` in *after* cannot exist with bad values; the error message names exactly what's wrong. The last block shows the JDK's own fail-fast iterator and the safe alternative `removeIf`. (Note: `list.remove("a")` itself succeeded before the iterator noticed the change on its next step, which is why only `"b"` was left for `removeIf` to delete.)

Trade-offs

Fail fast	Fail slow / silently
Bugs found early, near their cause	System "keeps running"
Clear diagnostics	Data corruption, confusing symptoms
Some upfront validation code	Less code, more debugging
May stop a whole process for a minor issue	—

When not to use it

Fail fast *inside* a component, but think about **resilience at system boundaries**: a UI shouldn't crash because one widget failed to load, and a batch job (a program that processes a large set of records in one run, often on a schedule) may skip and log a bad record rather than abort millions of good ones. The rule is: detect early, then decide deliberately how to handle (reject, retry, degrade) — never ignore.

Common mistakes

- Catching and ignoring exceptions, or returning `null` / `-1` on error.
- Validating deep inside business logic instead of at entry.
- Using `assert` for argument validation in public methods — Java assertions are **disabled by default** (enabled with `-ea`), so they can't be the only guard.

Real-world examples

- `Objects.requireNonNull`, `List.of(null)` throws `NullPointerException` immediately.
- **Spring Boot** (Spring's ready-made setup for quickly building runnable applications) fails at startup if a required bean or property is missing, and `@Validated` configuration properties fail on bad values.
- **Circuit breakers** (a wrapper around remote calls that "trips" after repeated failures and rejects further calls for a while; libraries: Resilience4j; earlier Netflix Hystrix, now in maintenance mode) fail fast: when a downstream service (one your service depends on and calls) is unhealthy, calls are rejected immediately instead of waiting for timeouts.
- **Erlang/OTP "let it crash"**: processes fail fast and supervisors restart them.
- **CI pipelines** (Continuous Integration: an automated build-and-test run on every code change) stop on the first failing stage.

In the interview

30-second answer: "Fail fast means validating at the boundary and throwing a clear error immediately, so the failure is close to its cause. In LLD I put guard clauses in constructors and public methods so invalid

objects can't exist. At the system level, circuit breakers fail fast to avoid piling up on slow dependencies."

Follow-ups:

- *Q: Fail-fast vs fail-safe iterators?* A: Fail-fast (`ArrayList`, `HashMap`) throw `ConcurrentModificationException` on structural modification during iteration (best-effort); fail-safe/weakly consistent ones (`CopyOnWriteArrayList`, `ConcurrentHashMap`) iterate over a snapshot or tolerate concurrent changes and never throw it.
- *Q: Isn't crashing bad for users?* A: Fail fast inside; handle gracefully at the edge (show an error, retry, degrade).

Self-check

1. Q: Where should validation go? → A: At boundaries: constructors, public method entry, startup config, API input.
2. Q: Why not rely on `assert` for public argument checks? → A: Assertions are disabled by default at runtime.

TL;DR

Check early, throw clearly, never swallow; make invalid states impossible to construct.

---

14. Immutability

What it is

An **immutable object** cannot change its observable state after it is constructed. To "change" it, you create a new object. `String`, `Integer`, `LocalDate`, `BigDecimal` and records with immutable components are examples.

Analogy: a printed receipt. If the amount is wrong, you print a new receipt; you don't edit the old one in everyone's hands.

The problem it solves

Mutable shared objects cause:

- **Aliasing bugs** — two references to the same object; one changes it, the other is surprised.
- **Broken invariants** — someone calls a setter and leaves the object half-valid.
- **Thread-safety problems** — concurrent reads and writes need locks (a mechanism that lets only one thread at a time run a protected piece of code).
- **Hash-based collection bugs** — changing a field used by `hashCode` after putting the object in a `HashSet` makes it "disappear".

Immutable objects are **inherently thread-safe** (can be shared freely without locks), can be cached and reused, are safe as map keys, and are easy to reason about.

How it works

The five rules from *Effective Java*'s item "Minimize mutability" (Joshua Bloch):

1. Don't provide methods that modify the object's state (no setters, no mutators).
2. Ensure the class can't be extended (`final` class, or private constructors + static factories — `static` methods such as `of(...)` that create instances). Records are implicitly final.
3. Make all fields `final`.
4. Make all fields `private`.
5. Ensure exclusive access to any mutable components: **defensively copy** mutable inputs in the constructor (`List.copyOf`) and never hand out references to mutable internals (return copies or unmodifiable views).

A common extra convenience: "wither" methods that return a new instance with one thing changed (`withStart`, `plusCity`).

Unmodifiable vs immutable: `Collections.unmodifiableList(list)` is a read-only *view* — if someone still holds the original list and changes it, the view shows the change. `List.copyOf` / `List.of` create an independent unmodifiable list. Records are **shallowly** immutable: a `List` component could still be mutable unless you copy it.

Java example — before/after

```
// File: lld/ch05/immutability/Main.java
package lld.ch05.immutability;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/** Immutability: once created, an object's observable state never changes. */
public class Main {

    // ---------------- BEFORE: a mutable class that leaks its internals ----------------
    static class Before {
        static class Itinerary {
            private final List<String> cities;              // 'final' reference, but the LIST is mutable
            private LocalDate start;
            Itinerary(List<String> cities, LocalDate start) { this.cities = cities; this.start = start; }
            List<String> getCities() { return cities; }      // hands out the internal list
            void setStart(LocalDate d) { start = d; }
            @Override public String toString() { return start + " " + cities; }
        }
    }

    // ---------------- AFTER: final class, final fields, defensive copies, "wither" methods ----------------
    static class After {
        /** A record is shallowly immutable; List.copyOf makes the list part immutable too. */
        record Itinerary(List<String> cities, LocalDate start) {
            Itinerary {
                cities = List.copyOf(cities);   // copy in: caller's later edits cannot reach us
            }
            Itinerary withStart(LocalDate newStart) { return new Itinerary(cities, newStart); }
            Itinerary plusCity(String c) {
                List<String> next = new ArrayList<>(cities);
                next.add(c);
                return new Itinerary(next, start);
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        List<String> input = new ArrayList<>(List.of("Delhi", "Agra"));
        var trip = new Before.Itinerary(input, LocalDate.of(2026, 10, 1));
        input.add("Jaipur");                          // caller edits its own list...
        trip.getCities().clear();                     // ...another caller wipes ours
        System.out.println("shared trip is now: " + trip + "   <-- changed by outsiders");

        System.out.println("== AFTER ==");
        List<String> input2 = new ArrayList<>(List.of("Delhi", "Agra"));
        var t1 = new After.Itinerary(input2, LocalDate.of(2026, 10, 1));
        input2.add("Jaipur");
        try { t1.cities().clear(); } catch (UnsupportedOperationException e) { System.out.println("cannot clear:
unmodifiable"); }
        var t2 = t1.plusCity("Jaipur").withStart(LocalDate.of(2026, 11, 5));
        System.out.println("t1 = " + t1);
        System.out.println("t2 = " + t2);

        System.out.println("== JDK immutables ==");
        String s = "hello";
        String upper = s.toUpperCase();
        LocalDate d = LocalDate.of(2026, 1, 31);
        System.out.println(s + " / " + upper + " ; " + d + " plusMonths(1) = " + d.plusMonths(1) + " (d unchanged:
" + d + ")");
    }
}
```

Output:

```
== BEFORE ==
shared trip is now: 2026-10-01 []   <-- changed by outsiders
== AFTER ==
cannot clear: unmodifiable
t1 = Itinerary[cities=[Delhi, Agra], start=2026-10-01]
t2 = Itinerary[cities=[Delhi, Agra, Jaipur], start=2026-11-05]
== JDK immutables ==
hello / HELLO ; 2026-01-31 plusMonths(1) = 2026-02-28 (d unchanged: 2026-01-31)
```

In *before*, `final List` only stops the *reference* from changing; outsiders both added to and cleared the list. In *after*, the compact constructor copies the list, the accessor returns an unmodifiable list, and "changes" produce new `Itinerary` values while `t1` stays intact. The JDK line shows `LocalDate.plusMonths` returning a new date (and clamping Jan 31 + 1 month to Feb 28).

Trade-offs

Immutable	Mutable
Thread-safe, no locks	Needs synchronisation when shared
Safe map keys, cacheable, easy reasoning	In-place updates, no allocation
Allocation for every "change" (usually cheap on modern JVMs — Java Virtual Machines, which run Java code)	Aliasing and invariant bugs
Large objects: copying cost (use builders or persistent data structures, which share unchanged parts between versions)	—

When not to use it

- Hot loops building big results — use a mutable builder locally (`StringBuilder`, then `toString()`), and expose the immutable result.
- Entities whose identity persists while state changes (a `BankAccount` balance) — keep them mutable but encapsulated; make their *values* (Money, Address) immutable.
- ORM entities (ORM = Object-Relational Mapping, a library that maps classes to database tables; JPA is Java's standard ORM API) usually need no-arg constructors and setters.

Common mistakes

- Thinking `final` fields make an object immutable (not if the field refers to a mutable object).
- Returning the internal list from a getter.
- Using `Collections.unmodifiableList` over a list the caller still holds.
- Letting `this` escape during construction (e.g. registering a listener in the constructor) — other threads may see a half-built object.

Real-world examples

- `String`, wrapper types, `BigDecimal`, `java.time.*`, `List.of`, `Map.of`, records.
- **Git commits** (saved versions in Git, the version-control tool) are immutable snapshots identified by hash (a fingerprint-like ID computed from the content).
- **Kafka log records** are immutable once appended.
- **Redux / React state** is treated as immutable; updates produce new state objects so change detection is a cheap reference check.

In the interview

30-second answer: "Immutable objects never change after creation: final class, private final fields, no setters, defensive copies in and out. They're thread-safe without locks and safe as map keys. In designs I make value objects like Money, Address and configuration immutable, and keep mutable state encapsulated inside entities."

Follow-ups:

- *Q: Are records immutable?* A: Shallowly — fields are final, but a mutable component (like an `ArrayList`) must be defensively copied.
- *Q:* `unmodifiableList` *vs* `List.copyOf`*?* A: The first is a live read-only view; the second is an independent unmodifiable copy.
- *Q: Why must immutable classes be final?* A: A subclass could add mutable state or override methods to break the guarantees.

Self-check

1. Q: Does `private final List<String> items` make a class immutable? → A: No; the list itself can still be modified.
2. Q: What is a "wither" method? → A: A method returning a new instance with one field changed.
3. Q: Why are immutable objects thread-safe? → A: No thread can modify them, so there are no data races on their state (given safe construction).

TL;DR

No setters, final everything, copy mutable inputs and outputs; "changing" means creating a new value.

---

15. Design by Contract (DbC)

What it is

Design by Contract, from Bertrand Meyer and the Eiffel language, treats a method like a business contract between a **client** (caller) and a **supplier** (the method):

- **Precondition** — what must be true *before* the call. The caller's obligation. (e.g. `pop` requires a non-empty stack.)
- **Postcondition** — what the method guarantees *after* the call if the precondition held. The supplier's obligation. (e.g. after `push(x)`, size grew by one and the top is `x`.)
- **Class invariant** — what is always true about every object between public calls. (e.g. `0 ≤ size ≤ capacity`.)

Analogy: a courier contract: you must pack the parcel and pay (precondition); they promise delivery within 2 days (postcondition); the parcel is never opened in transit (invariant).

The problem it solves

Without explicit contracts, each side guesses: the caller doesn't know what inputs are allowed; the method either checks everything defensively (duplicated checks everywhere) or checks nothing and silently corrupts state. Errors show up as ambiguous return values (`-1`) or lost data.

How it works

1. For each public method, write down the precondition, postcondition and the class invariant (in Javadoc — the structured `/** ... */` comments that generate Java API docs — at least).
2. Check preconditions at entry and throw a clear exception when violated (`IllegalArgumentException`, `IllegalStateException`).
3. Check postconditions and invariants during development/testing (assertions or internal checks).
4. Give callers **query methods** to check preconditions themselves (`isEmpty()`, `isFull()`), so they can avoid violations.
5. With inheritance, obey the **contract rules of subtyping** (which is what the Liskov Substitution Principle formalises): a subclass may **weaken** (accept more) preconditions and **strengthen** (promise more) postconditions, never the reverse, and must preserve the invariant.

Java example — before/after

```java
// File: lld/ch05/contract/Main.java
package lld.ch05.contract;

import java.util.Arrays;

/** Design by Contract: preconditions (caller's duty), postconditions (method's promise), invariants (always true).
*/
public class Main {

    // ---------------- BEFORE: no stated contract; misuse silently corrupts state ----------------
    static class Before {
        static class BoundedStack {
            private final int[] items;
            private int size;
            BoundedStack(int capacity) { items = new int[Math.max(capacity, 0)]; }
            void push(int x) { if (size < items.length) items[size++] = x; }        // silently drops when full
            int pop() { return size > 0 ? items[--size] : -1; }                    // -1: is that data or an error?
        }
    }

    // ---------------- AFTER: every public method states and checks its contract ----------------
    static class After {
        static final class BoundedStack {
            private final int[] items;
            private int size;

            BoundedStack(int capacity) {
                require(capacity > 0, "capacity > 0");
                items = new int[capacity];
                checkInvariant();
            }
            /** pre: !isFull(). post: size() == old size + 1 and peek() == x. */
            void push(int x) {
                require(!isFull(), "push requires !isFull()");
                int oldSize = size;
                items[size++] = x;
                ensure(size == oldSize + 1 && peek() == x, "push postcondition");
                checkInvariant();
            }
            /** pre: !isEmpty(). post: size() == old size - 1. */
            int pop() {
                require(!isEmpty(), "pop requires !isEmpty()");
                int oldSize = size;
                int top = items[--size];
                ensure(size == oldSize - 1, "pop postcondition");
                checkInvariant();
                return top;
            }
            int peek() { require(!isEmpty(), "peek requires !isEmpty()"); return items[size - 1]; }
            boolean isEmpty() { return size == 0; }
            boolean isFull() { return size == items.length; }
            int size() { return size; }
            @Override public String toString() { return Arrays.toString(Arrays.copyOf(items, size)); }

            // invariant: 0 <= size <= capacity
            private void checkInvariant() { if (size < 0 || size > items.length) throw new
AssertionError("invariant broken"); }
            private static void require(boolean ok, String what) { if (!ok) throw new
IllegalStateException("precondition failed: " + what); }
            private static void ensure(boolean ok, String what) { if (!ok) throw new AssertionError("postcondition
failed: " + what); }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
```

```
        var s = new Before.BoundedStack(2);
        s.push(1); s.push(2); s.push(3);                          // 3 vanishes
        System.out.println("pop=" + s.pop() + " pop=" + s.pop() + " pop=" + s.pop() + "   <-- 3 lost, -1
ambiguous");

        System.out.println("== AFTER ==");
        var t = new After.BoundedStack(2);
        t.push(1); t.push(2);
        System.out.println("stack " + t + " full=" + t.isFull());
        try { t.push(3); } catch (IllegalStateException e) { System.out.println(e.getMessage()); }
        System.out.println("pop=" + t.pop() + " pop=" + t.pop());
        try { t.pop(); } catch (IllegalStateException e) { System.out.println(e.getMessage()); }
    }
}
```

Output:

```
== BEFORE ==
pop=2 pop=1 pop=-1   <-- 3 lost, -1 ambiguous
== AFTER ==
stack [1, 2] full=true
precondition failed: push requires !isFull()
pop=2 pop=1
precondition failed: pop requires !isEmpty()
```

The *before* stack silently drops pushes when full and returns `-1` when empty (is `-1` a value or an error?). The *after* stack documents and enforces its contract: violations are reported immediately and precisely, and callers can check `isFull()`/`isEmpty()` first.

Trade-offs

Explicit contracts	Implicit
Clear responsibilities, precise errors	Less code
Documentation that can be executed	Ambiguity, defensive checks everywhere
Runtime cost of checks (can be disabled for postconditions/invariants in production)	Silent corruption

When not to use it

Full postcondition/invariant checking in performance-critical inner loops — keep precondition checks on public APIs, and move expensive postcondition checks into tests or `assert` statements.

Common mistakes

- Using `assert` for public preconditions (disabled by default).
- Subclasses strengthening preconditions (e.g. a `SavingsAccount.withdraw` that rejects amounts the base `Account.withdraw` contract allowed) — an LSP violation. (The classic `Square extends Rectangle` problem is the mirror case: `Square.setWidth` breaks the *postcondition* that the height stays unchanged.)
- Returning special values instead of stating a precondition.

Real-world examples

- **Java's own API docs** are contracts: `List.get(int)` states it throws `IndexOutOfBoundsException` if the index is out of range; `Comparable.compareTo` and `Object.equals`/`hashCode` specify contracts every implementation must honour (e.g. equal objects must have equal hash codes).
- **Guava** `Preconditions.checkArgument/checkState` and Spring's `Assert` utilities are precondition helpers.
- **Kotlin** `require()` / `check()` mirror preconditions and state checks.
- **Eiffel** has `require`, `ensure` and `invariant` built into the language; **Ada 2012** has `Pre`/`Post` aspects.

In the interview

> **30-second answer:** "Design by Contract defines preconditions the caller must satisfy, postconditions the method guarantees, and invariants always true for the object. I enforce preconditions with guard clauses, expose query methods like `isEmpty`, and remember that subclasses can only weaken preconditions and strengthen postconditions — that's LSP."

Follow-ups:

- *Q: DbC vs defensive programming?* A: Defensive programming checks everything everywhere; DbC assigns each check to one party — preconditions to the caller (verified at entry), postconditions to the supplier.
- *Q: Which exceptions fit?* A: `IllegalArgumentException` for bad arguments, `IllegalStateException` when the object's state forbids the call, `NullPointerException` for nulls (via `requireNonNull`).

Self-check

1. Q: Whose duty is a precondition? → A: The caller's.
2. Q: What can a subclass do to preconditions? → A: Weaken (accept more), never strengthen.
3. Q: State the invariant of a bounded stack. → A: `0 ≤ size ≤ capacity`.

TL;DR

State and enforce who promises what: preconditions (caller), postconditions (method), invariants (always).

---

16. GRASP — General Responsibility Assignment Software Patterns

What it is

GRASP is a set of nine patterns (Larman calls them patterns; many people say principles or guidelines) by Craig Larman, from *Applying UML and Patterns* (UML, the Unified Modeling Language, is the standard diagram notation for classes and their interactions), for the most frequent question in object design: **"Which class should be responsible for this?"** A *responsibility* is either *knowing* something (data, related objects, derived values) or *doing* something (computing, creating, coordinating). GRASP is a thinking aid — a vocabulary for justifying design decisions — rather than a set of code templates.

History note: GRASP first appeared in the book's 1st edition (1997), where the ninth item was "Don't Talk to Strangers" (the Law of Demeter). From the 2nd edition (2001) it was replaced by the more general **Protected Variations**, giving the nine listed below.

#	GRASP guideline	Question it answers	Short answer
1	Information Expert	Who does this job?	The class that has the information needed
2	Creator	Who creates an object A?	A class that contains, records, closely uses or has the data to initialise A
3	Controller	Who first handles a system event from the UI?	A non-UI object representing the use case (one user goal, e.g. checkout) or the whole system
4	Low Coupling	How do we reduce the impact of change?	Assign responsibilities to keep dependencies few
5	High Cohesion	How do we keep classes focused and understandable?	Assign responsibilities so each class does related work
6	Polymorphism	How do we handle behaviour that varies by type?	Give each type its own implementation of a shared operation
7	Pure Fabrication	What if no domain class fits without hurting cohesion/coupling?	Invent a helper class (e.g. a repository)
8	Indirection	How do we avoid direct coupling between two things?	Put an intermediate object between them
9	Protected Variations	How do we shield elements from changes in others?	Wrap points of instability behind a stable interface

Analogy: staffing a hospital. The person who has the patient's chart (Expert) writes the summary; the maternity ward registers newborns (Creator); the front desk routes every visitor to the right department (Controller); nobody needs to know everyone (Low Coupling); each department has a focused speciality (High Cohesion).

The problem it solves

Beginners often end up with one "god" class (one class that knows and does far too much — often the UI screen or a `Manager`) that knows everything, and many anemic data classes. Or they spread logic randomly so that no one knows where anything lives. GRASP gives repeatable reasons for placing each responsibility, which also makes your interview explanations concrete ("I put `total()` on `Sale` because it's the Information Expert").

How it works

1. List the system events/use cases (e.g. `scan item`, `pay`).
2. For each event, choose a **Controller** to receive it.
3. For each thing that must be *known* or *computed*, find the **Information Expert**.
4. For each object that must be *created*, find the **Creator**.
5. Check **Low Coupling** and **High Cohesion** as evaluative forces; if a choice hurts them, reconsider.
6. Where behaviour varies by type, use **Polymorphism**; where an external or unstable thing touches you, use **Protected Variations** via **Indirection** (an interface or adapter — a class that converts one interface into the one your code expects).
7. When no domain object is a good home (persistence, I/O — input/output such as files and network calls), create a **Pure Fabrication**.

Java example — before/after (all nine in one checkout)

The *before* is a UI `CheckoutScreen` that creates line items, reaches into them to total the sale, hard-codes tax rules and builds SQL. The *after* assigns each responsibility using GRASP; each class's Javadoc names the guideline it illustrates.

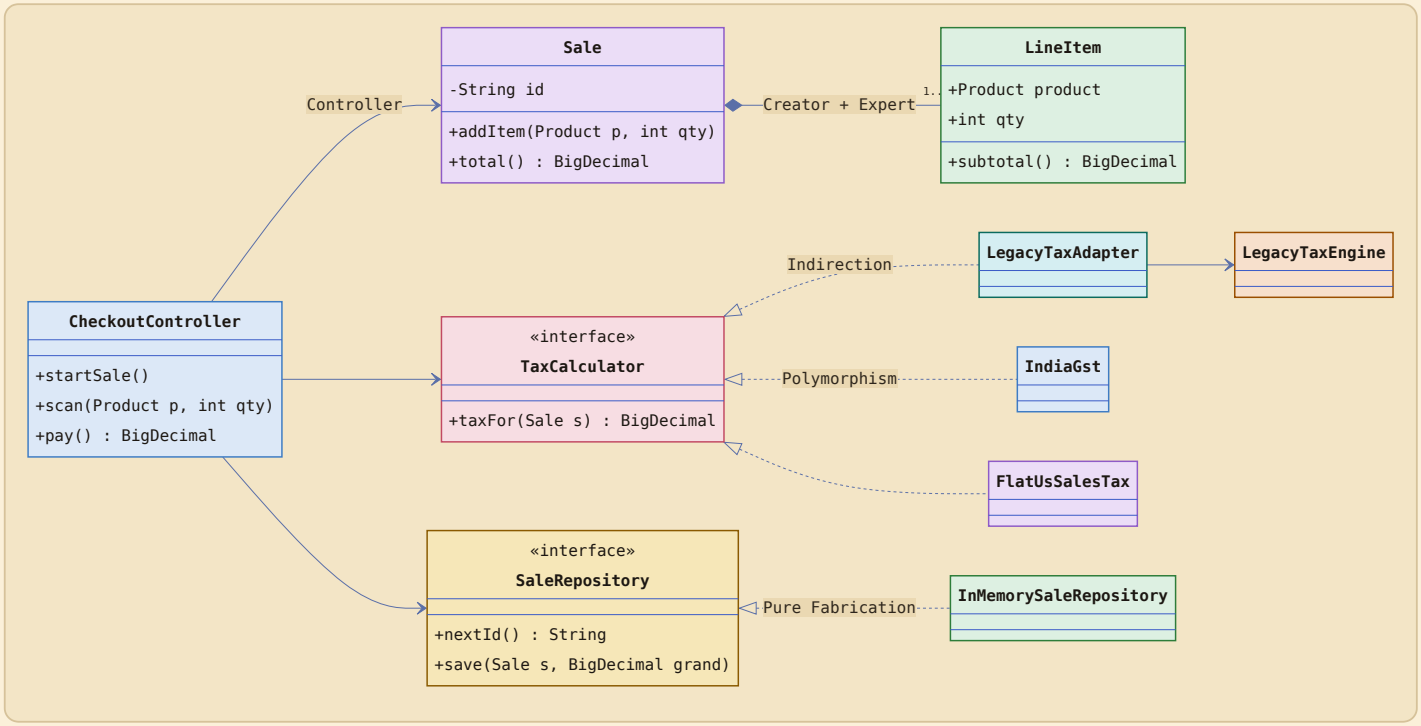
```java
// File: lld/ch05/grasp/Main.java
package lld.ch05.grasp;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * GRASP on a tiny checkout: who should do what?
 * Information Expert, Creator, Controller, Low Coupling, High Cohesion,
 * Polymorphism, Pure Fabrication, Indirection, Protected Variations.
 */
public class Main {

    record Product(String sku, String name, BigDecimal price) {}

    // ---------------- BEFORE: a "god" screen class does everything ----------------
    static class Before {
        static class LineItem { Product product; int qty; }
        static class Sale { List<LineItem> items = new ArrayList<>(); }

        /** UI class that computes totals, calls the tax vendor directly and builds SQL. */
        static class CheckoutScreen {
            final Sale sale = new Sale();
            final List<String> sqlLog = new ArrayList<>();
            void onScan(Product p, int qty) {
                LineItem li = new LineItem();               // UI creates domain parts
                li.product = p; li.qty = qty;
                sale.items.add(li);
            }
            void onPayClicked(String region) {
                BigDecimal total = BigDecimal.ZERO;         // UI reaches into Sale -> LineItem -> Product
                for (LineItem li : sale.items) total =
total.add(li.product.price().multiply(BigDecimal.valueOf(li.qty)));
                BigDecimal tax = region.equals("IN") ? total.multiply(new BigDecimal("0.18"))   // vendor rule
inline
                        : total.multiply(new BigDecimal("0.07"));
                sqlLog.add("INSERT INTO sales VALUES(" + total.add(tax) + ")");
                System.out.println("screen shows total " + total.add(tax).setScale(2, RoundingMode.HALF_UP));
            }
        }
    }

    // ---------------- AFTER: responsibilities assigned with GRASP ----------------
    static class After {

        /** INFORMATION EXPERT: a line item knows its product and quantity, so it computes its own subtotal. */
        record LineItem(Product product, int qty) {
            BigDecimal subtotal() { return product.price().multiply(BigDecimal.valueOf(qty)); }
        }

        /** EXPERT for the total + CREATOR of LineItems (a Sale contains/aggregates them). HIGH COHESION: only sale
logic. */
        static final class Sale {
            private final String id;
            private final List<LineItem> items = new ArrayList<>();
            Sale(String id) { this.id = id; }
            void addItem(Product p, int qty) { items.add(new LineItem(p, qty)); }        // Creator
            BigDecimal total() { return items.stream().map(LineItem::subtotal).reduce(BigDecimal.ZERO,
BigDecimal::add); }
            String id() { return id; }
            List<LineItem> items() { return List.copyOf(items); }
        }

        /** PROTECTED VARIATIONS: tax rules vary by region/vendor -> hide them behind a stable interface. */
```

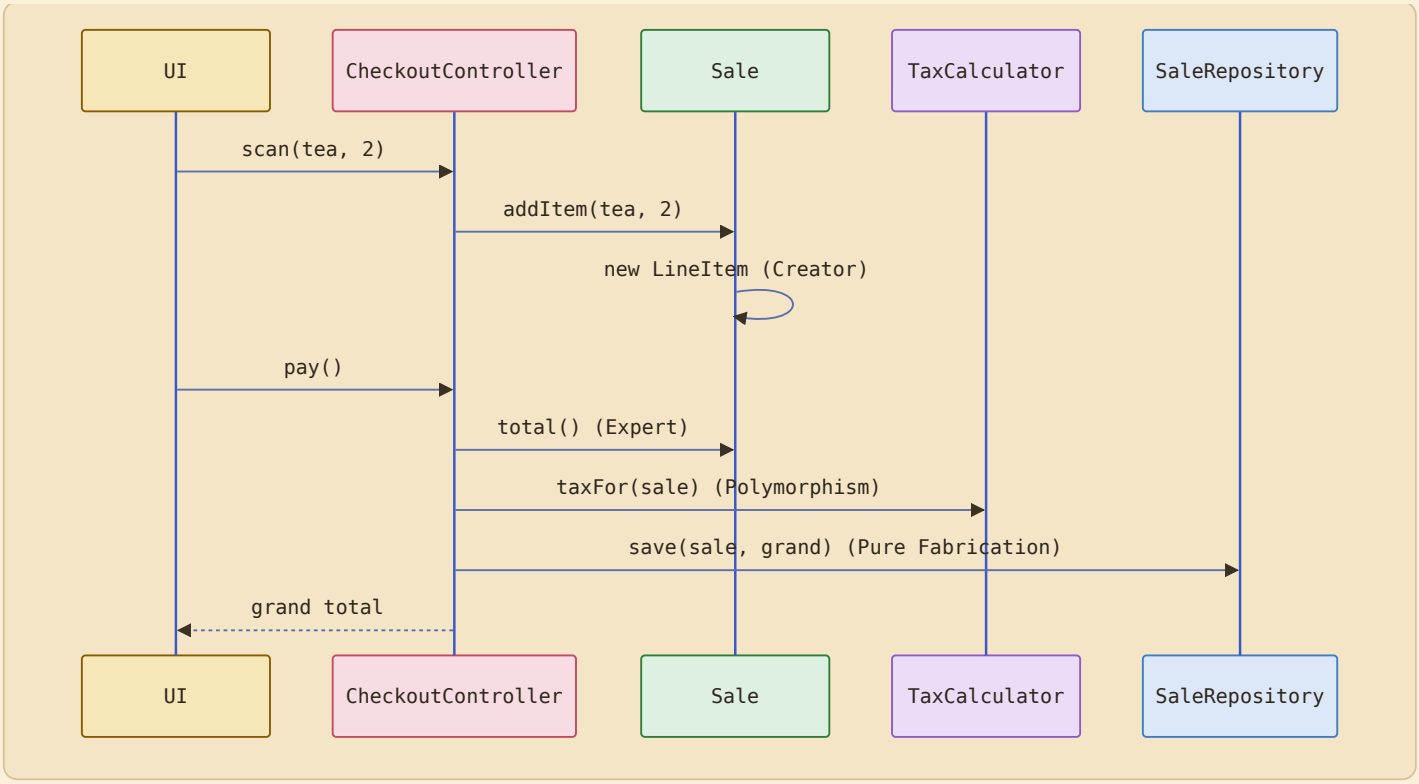
```
        interface TaxCalculator { BigDecimal taxFor(Sale sale); }

        /** POLYMORPHISM: each variant is its own type; no if/else on region in the domain. */
        static final class IndiaGst implements TaxCalculator {
            public BigDecimal taxFor(Sale s) { return s.total().multiply(new BigDecimal("0.18")); }
        }
        static final class FlatUsSalesTax implements TaxCalculator {
            public BigDecimal taxFor(Sale s) { return s.total().multiply(new BigDecimal("0.07")); }
        }

        /** A third-party library with its own, awkward API (we don't control it): it even returns tax as a double.
*/
        static final class LegacyTaxEngine {
            double computeTaxCents(long amountCents, String countryCode) { return amountCents *
("GB".equals(countryCode) ? 0.20 : 0.0); }
        }
        /** INDIRECTION: an adapter sits between our domain and the vendor, so neither knows the other. */
        static final class LegacyTaxAdapter implements TaxCalculator {
            private final LegacyTaxEngine engine; private final String country;
            LegacyTaxAdapter(LegacyTaxEngine engine, String country) { this.engine = engine; this.country =
country; }
            public BigDecimal taxFor(Sale s) {   // converts to/from the vendor's types at the boundary only
                long cents = s.total().movePointRight(2).longValueExact();
                return BigDecimal.valueOf(Math.round(engine.computeTaxCents(cents, country))).movePointLeft(2);
// whole cents
            }
        }

        /** PURE FABRICATION: not a domain concept -- invented to keep persistence out of Sale (cohesion +
coupling). */
        interface SaleRepository { String nextId(); void save(Sale sale, BigDecimal grandTotal); int count(); }
        static final class InMemorySaleRepository implements SaleRepository {
            private final Map<String, BigDecimal> rows = new HashMap<>();
            private int seq;
            public String nextId() { return "S" + (++seq); }
            public void save(Sale s, BigDecimal g) { rows.put(s.id(), g); }
            public int count() { return rows.size(); }
        }

        /** CONTROLLER: the first non-UI object that receives system events (scan, pay) and delegates. LOW
COUPLING: depends on interfaces. */
        static final class CheckoutController {
            private final TaxCalculator tax; private final SaleRepository repo;
            private Sale current;
            CheckoutController(TaxCalculator tax, SaleRepository repo) { this.tax = tax; this.repo = repo; }
            void startSale() { current = new Sale(repo.nextId()); }
            void scan(Product p, int qty) { current.addItem(p, qty); }
            BigDecimal pay() {
                BigDecimal grand = current.total().add(tax.taxFor(current)).setScale(2, RoundingMode.HALF_UP);
                repo.save(current, grand);
                return grand;
            }
        }
    }

    public static void main(String[] args) {
        Product tea = new Product("T1", "Tea", new BigDecimal("120.00"));
        Product cup = new Product("C1", "Cup", new BigDecimal("80.00"));

        System.out.println("== BEFORE ==");
        var screen = new Before.CheckoutScreen();
        screen.onScan(tea, 2); screen.onScan(cup, 1);
        screen.onPayClicked("IN");
        System.out.println("sql: " + screen.sqlLog);

        System.out.println("== AFTER ==");
        var repo = new After.InMemorySaleRepository();
        for (After.TaxCalculator calc : List.of(new After.IndiaGst(), new After.FlatUsSalesTax(),
                new After.LegacyTaxAdapter(new After.LegacyTaxEngine(), "GB"))) {
```

```
            var controller = new After.CheckoutController(calc, repo);
            controller.startSale();
            controller.scan(tea, 2);
            controller.scan(cup, 1);
            System.out.println(calc.getClass().getSimpleName() + " -> grand total " + controller.pay());
        }
        System.out.println("sales saved: " + repo.count());
    }
}
```

Output:

```
== BEFORE ==
screen shows total 377.60
sql: [INSERT INTO sales VALUES(377.6000)]
== AFTER ==
IndiaGst -> grand total 377.60
FlatUsSalesTax -> grand total 342.40
LegacyTaxAdapter -> grand total 384.00
sales saved: 3
```





The rest of this section explains each guideline in the book's standard format.

16.1 Information Expert

What it is

Give a responsibility to the class that **has the information** needed to fulfil it. `LineItem` knows price and quantity → it computes `subtotal()`. `Sale` knows its line items → it computes `total()`.

The problem it solves

Prevents "manager" classes that pull data out of other objects to compute things (the *Feature Envy* smell), which duplicates logic and couples the manager to everyone's internals.

How it works

1. State the responsibility ("compute sale total").
2. Ask what information is needed (line item subtotals).
3. Find who holds it (Sale holds line items; each LineItem holds price × qty).
4. Split the work along those lines — partial experts cooperate.

Trade-offs

Keeps behaviour with data (Tell-Don't-Ask). But blindly applying it can hurt cohesion: `Sale` has the data to save itself, yet putting SQL into `Sale` mixes concerns — that's why Pure Fabrication exists.

When not to use it

When following it would put technical concerns (persistence, UI, network) into domain objects.

Common mistakes

Computing everything in a service with getters; or the opposite, making an entity save/print/email itself.

Real-world examples

`String.length()`, `List.size()`, `LocalDate.isLeapYear()` — the object with the data answers. In a Swiggy/Zomato-style cart, the `Cart` computes its own item total.

In the interview

"`total()` lives on `Order` because it owns the line items — Information Expert. Each `LineItem` computes its own subtotal." *Follow-up: when does Expert conflict with other principles?* When it would add persistence or UI to a domain class; then use Pure Fabrication.

Self-check

Q: Who should compute a line item's subtotal? → A: `LineItem` — it has price and quantity.

TL;DR

The one with the data does the work.

16.2 Creator

What it is

Class **B** should create instances of class **A** if one or more is true: B *contains* or aggregates A; B *records* A; B *closely uses* A; B *has the initialising data* for A. Here `Sale` creates `LineItem`s.

The problem it solves

Who calls `new`? Random placement spreads coupling to A's constructor everywhere. Choosing a class that already relates to A adds no new coupling.

How it works

1. Find the class that aggregates/contains A (strongest reason).
2. Otherwise one that records or closely uses it, or has its initial data.
3. If creation is complex or varies (families, pooling, choosing a subclass), use a **Factory** or **Builder** instead (Chapter 8).

Trade-offs

Simple and low-coupling for plain objects; not suitable when creation logic is complex or conditional.

When not to use it

Complex creation → Factory/Builder; framework-managed objects → DI container.

Common mistakes

UI or controller code doing `new LineItem(...)` and pushing it into the sale's list (the *before* code).

Real-world examples

`HashMap` creating its own internal node objects for the entries it contains; a `Document` creating its `Paragraph`s; `Thread` objects created by an `ExecutorService` through its `ThreadFactory` (a factory, because creation varies).

In the interview

"`Sale.addItem(product, qty)` creates the `LineItem` because the sale contains them — GRASP Creator. If creation got complex I'd move it to a factory." *Follow-up: Creator vs Factory?* Creator picks an existing class that already relates to A (no new coupling); a Factory is a Pure Fabrication you add when creation is complex or must choose among subclasses.

Self-check

Q: Strongest reason for B to create A? → A: B contains/aggregates A.

TL;DR

The container (or the one with the initial data) creates.

16.3 Controller

What it is

The first object **beyond the UI layer** that receives and coordinates a **system operation** (an event from an actor — a person or external system using ours — such as "scan item" or "pay"). Two kinds: a **facade controller** representing the whole system/device (e.g. `Register`, `ParkingLotSystem`) or a **use-case controller** for one scenario (e.g. `CheckoutController`, `BookTicketHandler`).

The problem it solves

Keeps business logic out of UI classes, so the same logic works from a web page, a mobile app or a test, and the UI can change freely.

How it works

1. List system events from use cases.
2. Assign each to a controller that represents the system or the use case.
3. The controller **delegates** — it coordinates, it doesn't do the domain work itself.

Trade-offs

Clean UI/domain separation; but a controller that does real work becomes **bloated** (low cohesion, many responsibilities) — split into use-case controllers and push work to experts.

When not to use it

Very small programs where the UI and logic are trivial.

Common mistakes

Fat controllers containing pricing, validation and SQL. Controllers that are UI classes (e.g. a Swing `JFrame` computing totals).

Real-world examples

Spring `@RestController` → `@Service` (the service often plays GRASP Controller for a use case); `DispatcherServlet` is a front controller (a single entry point that receives every web request and routes it); in an Uber-like app a `TripService.requestRide()` handles the rider's "request ride" event.

In the interview

"`ParkingLotController.park(vehicle)` receives the system event and delegates to `ParkingFloor` and `SpotAllocator` — GRASP Controller, kept thin." *Follow-up: facade or use-case controller?* A facade controller (one per system) is fine while there are few system events; when it grows bloated, split into use-case controllers (one per scenario).

Self-check

Q: Should a controller calculate tax? → A: No — it delegates to the expert or a tax strategy.

TL;DR

A thin non-UI coordinator receives each system event and delegates.

16.4 Low Coupling

What it is

An **evaluative** guideline: among design choices, prefer the one that keeps **dependencies between classes low**. Here the controller depends only on interfaces (`TaxCalculator`, `SaleRepository`), not on vendors or databases.

The problem it solves

High coupling means changes ripple, classes can't be reused alone, and understanding one class requires understanding many.

How it works

Compare alternatives by counting and weighing dependencies: who knows about whom? Prefer interfaces, fewer collaborators, one-directional dependencies, no cycles. Use DI.

Trade-offs

Coupling to *stable* things (JDK, `String`, `List`) is harmless. Striving for zero coupling produces bloated indirection.

When not to use it

Don't decouple from stable, standard libraries — focus on unstable or external dependencies.

Common mistakes

Circular dependencies between packages; UI knowing the database; everything depending on a god class.

Real-world examples

Microservices communicating over well-defined APIs/events; JDBC drivers decoupled from application code by interfaces.

In the interview

"My service depends on `PaymentGateway`, not on Stripe's SDK (Software Development Kit — the vendor's client library), so switching provider touches one adapter." *Follow-up: can coupling ever be zero?* No — objects must collaborate. Aim for few dependencies, pointing at stable abstractions, with no cycles.

Self-check

Q: Is depending on `java.util.List` bad coupling? → A: No — it is stable.

TL;DR

Choose the design with fewer, more stable dependencies.

16.5 High Cohesion

What it is

The other **evaluative** guideline: keep each class's responsibilities **focused and related**, so it's easy to understand and maintain. `Sale` does sale maths; persistence is in the repository; tax is in the calculator.

The problem it solves

Low-cohesion classes (e.g. `Utils`, `Manager`, a screen doing everything) are hard to understand, reuse and change; they change for many reasons (violating SRP).

How it works

When adding a responsibility, ask "is this related to what the class already does?" If not, place it elsewhere or create a new class. Larman describes four levels, from worst to best: *very low* (one class responsible for many things in very different areas) → *low* (one class alone responsible for a big, complex job in one area) → *moderate* (a few light responsibilities in different areas that relate to the class concept but not to each other) → *high* (a moderate number of related responsibilities in one area, collaborating with other objects for the rest — the goal).

Trade-offs

Many small focused classes vs a few big ones — more files, but each is simple. Coupling and cohesion usually improve together.

When not to use it

Rarely; sometimes grouping for performance (e.g. a remote API returning coarse-grained data) lowers cohesion on purpose.

Common mistakes

"Helper" and "Utils" dumping grounds; entities that also send emails.

Real-world examples

`java.time.LocalDate` (dates only), `java.util.regex.Pattern` (regex only). Spring Data repositories focus purely on persistence.

In the interview

"I split `BookingService` into seat locking, pricing and payment so each class has one focused job — high cohesion." *Follow-up: how do cohesion and coupling interact?* They usually improve together: moving unrelated work out of a class raises its cohesion and removes the dependencies that work dragged in.

Self-check

Q: Name a common low-cohesion smell. → A: A `Utils`/`Manager` class with unrelated methods.

TL;DR

Each class does one related set of things.

16.6 Polymorphism

What it is

When behaviour **varies by type**, give each type its own implementation of a shared operation (an interface method), instead of branching on the type with `if/else` or `switch`. `IndiaGst`, `FlatUsSalesTax` and `LegacyTaxAdapter` each implement `taxFor`.

The problem it solves

Type-checking conditionals repeated across the code; adding a type means finding and editing every branch (violates OCP).

How it works

1. Identify the operation that varies (`taxFor`).
2. Declare it on an interface/abstract class.
3. Implement it per variant.
4. Callers invoke it without knowing the concrete type.

Trade-offs

Easy to add new types; harder to add new operations across all types (you must touch each class — the Visitor pattern, Chapter 11, flips this trade-off).

When not to use it

For closed sets of types where you often add *operations*, a `switch` over a `sealed` hierarchy (Java 21 pattern matching — a `switch` that tests an object's type and unpacks it in one step) can be clearer.

Common mistakes

Creating polymorphic classes for variation that doesn't exist (YAGNI); `instanceof` chains after introducing the interface.

Real-world examples

`Comparator` implementations, `InputStream` subclasses, `Runnable`/`Callable`, payment methods in any checkout (card, UPI, wallet).

In the interview

"Instead of `if (vehicle instanceof Car)`, each `Vehicle` type answers `requiredSpotSize()` — GRASP Polymorphism." *Follow-up: what if I need a new operation across all types?* Adding it touches every class; if that happens often, consider a `sealed` hierarchy with pattern-matching `switch`, or the Visitor pattern.

Self-check

Q: Which kind of change does polymorphism make easy? → A: Adding a new type/variant.

TL;DR

Let each type implement the varying behaviour; don't switch on types.

16.7 Pure Fabrication

What it is

A class that **doesn't represent a real-world domain concept**, invented purely to keep high cohesion and low coupling when Information Expert would give a bad result. `SaleRepository` is a fabrication: the shop has no "repository", but putting SQL into `Sale` would ruin cohesion.

The problem it solves

Some responsibilities (persistence, logging, ID generation, messaging) don't fit any domain class without dragging technical concerns into it.

How it works

1. Notice the Expert choice would make a domain class do technical work.
2. Create a service class whose name describes the job (`SaleRepository`, `NotificationSender`, `IdGenerator`).
3. Keep it focused and reusable.

Trade-offs

Great for cohesion/reuse; overusing it produces an anemic domain where all behaviour sits in "services".

When not to use it

When a real domain object is a natural home — prefer the Expert first.

Common mistakes

Moving domain logic (pricing, rules) into fabricated "Manager" services.

Real-world examples

DAOs (Data Access Objects — classes that hide database queries)/repositories, `ObjectMapper` (Jackson, the common Java JSON library), `Clock` providers, Spring `@Service` classes for infrastructure, most GoF helper objects (Strategy, Adapter, Factory objects are fabrications).

In the interview

"`TicketRepository` isn't a domain thing — it's a pure fabrication that keeps persistence out of `Ticket`." *Follow-up: risk of too many fabrications?* An anemic domain: all rules drift into "service" classes. Keep domain rules on domain objects (Expert) and fabricate only for technical or cross-cutting work.

Self-check

Q: Is `Sale` a pure fabrication? → A: No — it's a domain concept. `SaleRepository` is.

TL;DR

Invent a helper class when no domain class is a clean home.

16.8 Indirection

What it is

Assign a responsibility to an **intermediate object** that mediates between two components so they aren't directly coupled. `LegacyTaxAdapter` sits between our `CheckoutController` and the vendor's `LegacyTaxEngine`.

The problem it solves

Direct dependencies on external or volatile components (vendor SDKs, hardware, other subsystems).

How it works

Introduce an interface plus an adapter, a facade, a mediator (a hub that objects talk through instead of to each other) or a proxy; the two sides talk through it.

Trade-offs

Decoupling vs an extra hop (small performance cost and one more class to read). A famous saying, usually attributed to David Wheeler, is that any problem in computer science can be solved by another level of indirection; the usual retort is "except the problem of too many levels of indirection".

When not to use it

Between stable, cohesive classes that always change together.

Common mistakes

Layers that add nothing (pure pass-through), making navigation painful.

Real-world examples

Adapter/Facade/Mediator/Proxy patterns; JDBC's `DriverManager`; API gateways in microservices (a single front door that routes client calls to many services); message brokers between producers and consumers.

In the interview

"I don't call the SMS vendor directly — an `SmsSender` interface with a `TwilioSmsSender` adapter provides indirection." *Follow-up: Indirection vs Protected Variations?* Indirection is the mechanism (an intermediate object); Protected Variations is the goal (shield the rest of the system from a point of change). Indirection is one common way to achieve it.

Self-check

Q: Which GoF patterns are forms of indirection? → A: Adapter, Facade, Mediator, Proxy (and Bridge).

TL;DR

Put a middleman between things that shouldn't know each other.

16.9 Protected Variations

What it is

Identify **points of predicted variation or instability** and wrap them with a **stable interface**, so variation doesn't ripple to other elements. `TaxCalculator` protects the checkout from tax-rule and vendor changes. Larman notes it is essentially David Parnas's *information hiding* (keeping design decisions that may change hidden inside one module) and Meyer's Open/Closed Principle stated as a responsibility guideline (and close to "encapsulate what varies"); the Law of Demeter — GRASP's "Don't Talk to Strangers" in the 1st edition — is one mechanism for it.

The problem it solves

Changes in one place (a vendor, a regulation, a format) forcing edits throughout the system.

How it works

1. Find variation points (external systems, rules that change, platforms).
2. Define a stable interface around each.
3. Put variants behind it (Polymorphism, Indirection, data-driven config).

Trade-offs

Protection has a cost; over-protecting things that never vary ("speculative generality") adds complexity. Protect *predicted* variation.

When not to use it

Stable, unlikely-to-change code (YAGNI).

Common mistakes

Leaky interfaces that expose vendor-specific types (e.g. returning a `StripeCharge` from `PaymentGateway`).

Real-world examples

JDBC, JPA, SLF4J, Java's `ServiceLoader` for pluggable providers; `java.nio.file.FileSystem` abstraction for different file systems.

In the interview

"Payment providers are a variation point, so `PaymentGateway` is my protected variation — the booking flow never sees provider details." *Follow-up: which variation points should you protect?* Predicted ones — external vendors, regulations, formats, platforms — not every class (that is speculative generality).

Self-check

Q: Which SOLID principle is closest to Protected Variations? → A: Open/Closed (also Dependency Inversion).

TL;DR

Wrap what's unstable behind a stable interface.

GRASP in the interview (whole set)

> **30-second answer:** "GRASP is Larman's set of nine guidelines for assigning responsibilities. I use Information Expert and Creator to place most behaviour, a thin Controller for system events, Polymorphism and Protected Variations for things that vary, Indirection and Pure Fabrication to keep technical concerns out of the domain, and I judge every choice by Low Coupling and High Cohesion."

Follow-ups:

- *Q: How does GRASP relate to GoF patterns?* A: GRASP is more fundamental: patterns like Strategy (Polymorphism + Protected Variations), Adapter (Indirection) and Repository (Pure Fabrication) are specific applications.
- *Q: Which two GRASP items are evaluative rather than assignment rules?* A: Low Coupling and High Cohesion — you use them to compare alternatives.
- *Q: Expert says `Sale` should save itself — why not?* A: It hurts cohesion and couples the domain to storage; use a Pure Fabrication (repository).

Self-check (GRASP)

1. Q: Who handles the "pay" button event after the UI? → A: A Controller (use-case or facade).
2. Q: Which guideline justifies an adapter around a vendor library? → A: Indirection (serving Protected Variations).
3. Q: Name the guideline for inventing `IdGenerator`. → A: Pure Fabrication.

TL;DR

Put work where the data is, let containers create, keep controllers thin, vary by polymorphism behind stable interfaces, invent helpers when needed — and judge it all by coupling and cohesion.

---

17. Domain-Driven Design (DDD) Essentials

What it is

Domain-Driven Design is an approach, named by Eric Evans in *Domain-Driven Design: Tackling Complexity in the Heart of Software* (2003), for building software whose code mirrors the business it serves. The **domain** is the subject area the software is about — lending, shipping, grocery retail. DDD has two halves:

- **Strategic design** — how to split a large business into parts, each with its own model: **ubiquitous language**, **bounded contexts**, **context maps** and the **anti-corruption layer**.
- **Tactical design** — the building blocks *inside* one part: **entities**, **value objects**, **aggregates**, **domain events**, **repositories**, plus domain services and factories.

The principles earlier in this chapter tell you how to shape classes; DDD tells you **which classes to have and where their boundaries are**, starting from the business rather than the database.

Analogy: a hospital. The word "patient" means one thing at the reception desk (name, insurance, appointment), another in the operating theatre (blood type, allergies, vitals) and another in billing (procedures, amounts owed). Each department keeps its own file with its own vocabulary; they exchange only what they need. Forcing one giant "Patient" form on all three would please nobody. Each department is a bounded context.

Key terms (defined once):

Term	Meaning
Ubiquitous language	The shared vocabulary that domain experts and developers both use — in conversation, in documents *and in the code*. If the business says "place an order", the method is `place()`, not `setStatus(2)`.
Bounded context	A boundary inside which one model and one language apply consistently. The same word may mean something different in another context (a Sales "order" vs a Shipping "consignment"). Often one team and one service.
Context map	A picture of the bounded contexts and how they relate (who depends on whom, who translates).
Anti-corruption layer (ACL)	A translation layer at a context's border that converts another context's (or a legacy — old, still-running — system's) model into yours, so theirs cannot leak in.
Entity	An object defined by its **identity**, which stays the same while its attributes change (an `Order` with id 1001 is the same order after items are added). Equality = same id.
Value object	An object defined only by its **attributes**, with no identity; immutable; interchangeable when equal (`Money(₹350)`, an `Address`, a `DateRange`). Equality = same values.
Aggregate	A cluster of entities and value objects treated as **one unit for changes**, with one entity as its **aggregate root** — the only object outsiders may hold or call. The root enforces the cluster's invariants.
Domain event	A record of something that happened that the business cares about, named in the past tense (`OrderPlaced`, `PaymentFailed`). Other parts react to it.
Repository	A collection-like interface for loading and saving **aggregates** (one repository per aggregate root), hiding the database (Chapter 11, §11.7.3).
Domain service	An operation that belongs to the domain but not naturally to one entity (e.g. "transfer money between two accounts"). Stateless.

The problem it solves

Big business systems fail less from bad algorithms than from **muddled models**:

- **One model for everything.** A single `Customer` or `Product` class grows fields for every department until nobody can change it safely (a "big ball of mud").

- **Translation errors.** Developers call it `status=3`, the business calls it "on hold"; requirements get lost between the two vocabularies.
- **Anemic domain model** (Martin Fowler's term): classes are data bags with getters and setters, and every rule lives in services. Rules get duplicated, and anyone can put an object into an invalid state (Tell, Don't Ask — section 5 — violated at scale).
- **No consistency boundary.** Without aggregates, code updates any object anywhere, locking and transactions sprawl, and invariants ("an order's total never exceeds the credit limit") are checked in some paths but not others.
- **Tight coupling between teams.** If Shipping reads Sales' tables directly, Sales cannot change its schema without breaking Shipping.

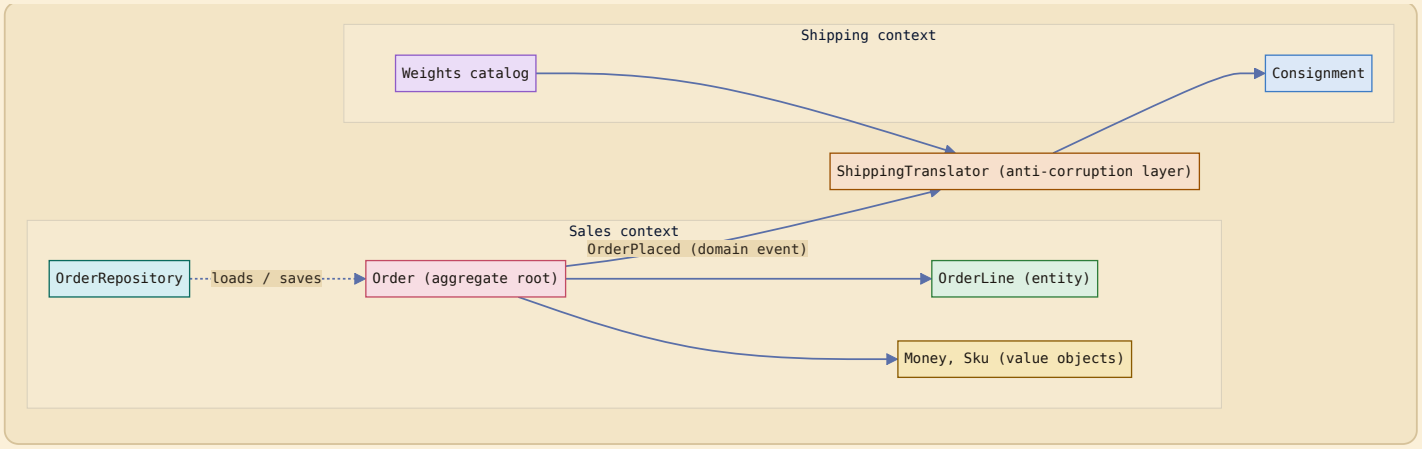
How it works

Strategic design — split by meaning, not by table

1. **Talk to domain experts and write down the language.** Nouns become candidate entities and value objects, verbs become methods, "when X happens" becomes a domain event.
2. **Draw the bounded contexts.** Where the same word starts meaning different things, you have a boundary. In an e-commerce business: *Catalog* (product descriptions, images), *Sales* (orders, prices, promotions), *Payments*, *Shipping* (parcels, weights, carriers), *Support*.
3. **Decide how contexts talk (the context map).** The common relationships: one side publishes events or an API and the other consumes it; a **shared kernel** (a small shared library both own — use sparingly); **conformist** (you simply accept the other model); and the **anti-corruption layer** (you translate the other model into yours — the safest choice against a legacy or third-party system).
4. **Map contexts to teams and services.** A bounded context is the most natural boundary for a microservice; that is why DDD is the usual starting point for splitting a monolith (an application built and deployed as one single unit) (see the Strangler Fig notes for migrating piece by piece).

Tactical design — the building blocks inside one context

- **Prefer value objects.** Wrap primitives (raw built-in types such as `double` or `String`) that carry rules: `Money` (amount + currency, no negatives), `Email`, `Quantity`, `DateRange`. They are immutable (section 14), validate themselves (Fail fast, section 13) and make method signatures honest (`transfer(Money)` instead of `transfer(double)`).
- **Entities have identity and a lifecycle.** Give them an id value object (`OrderId`), and methods named in the ubiquitous language that protect their invariants.
- **Aggregate design rules** (from Vaughn Vernon's *Implementing Domain-Driven Design*, 2013):
  1. Protect **true invariants** inside the aggregate boundary — the things that must be consistent *immediately*.
  2. Keep aggregates **small** (often the root plus a few value objects or child entities).
  3. **Reference other aggregates by id**, not by object (`CustomerId`, not `Customer`).
  4. **Change one aggregate per transaction**; keep other aggregates in step through domain events (**eventual consistency** — the other aggregates catch up shortly afterwards instead of instantly; see System Design Fundamentals).
- **Domain events** are raised inside the aggregate when a state change happens, and published **after** the change is saved. In a database-backed system the reliable way is the **transactional outbox**: write the event to an outbox table in the same transaction as the aggregate, and let a relay publish it (see the Saga notes and Event-Driven Architecture notes).
- **Repositories** exist only for aggregate roots (`OrderRepository`, never `OrderLineRepository`), because outsiders may only reach an `OrderLine` through its `Order`.
- **Application services** (a thin layer above the domain) load an aggregate from its repository, call one domain method, save it — they orchestrate, the aggregate decides (GRASP Controller, section 16.3).
- **Reads can use a different model.** When screens need data joined across aggregates, a separate read model fed by domain events is often simpler — that is **CQRS** (Command Query Responsibility Segregation — separate models for writing and reading; see the CQRS notes); storing the events themselves as the source of truth is **Event Sourcing** (Chapter 11, §11.7.6).


Shipping context
Weights catalog
Consignment
ShippingTranslator (anti-corruption layer)
Sales context
OrderPlaced (domain event)
OrderRepository
loads / saves
Order (aggregate root)
OrderLine (entity)
Money, Sku (value objects)

Java example — before/after

```java
// File: lld/ch05/ddd/Main.java
package lld.ch05.ddd;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.function.Consumer;

/** Domain-Driven Design, tactical building blocks: value objects, entities, an aggregate, domain events, a
repository, and two bounded contexts. */
public class Main {

    // ---------------- BEFORE: an anemic model — data bags with setters, rules scattered in services ------------
---
    static class Before {
        static class OrderLine { public String sku; public int qty; public double price; }
        static class Order {
            public List<OrderLine> lines = new ArrayList<>();
            public String status = "NEW";
            public double total;
        }
        static class OrderService {                       // the only place that "knows" the rules... in theory
            void addLine(Order o, String sku, int qty, double price) {
                OrderLine l = new OrderLine(); l.sku = sku; l.qty = qty; l.price = price;
                o.lines.add(l);
                o.total += qty * price;
            }
        }
    }

    // ---------------- AFTER: the rules live inside the model, in the business's own words ----------------
    static class After {
        /** Value object: no identity, immutable, equal when all fields are equal, validates itself. */
        record Money(long paise) {
            Money { if (paise < 0) throw new IllegalArgumentException("Money cannot be negative"); }
            static Money rupees(long r) { return new Money(r * 100); }
            Money plus(Money o) { return new Money(paise + o.paise); }
            Money times(int n) { return new Money(paise * n); }
            @Override public String toString() { return "Rs %d.%02d".formatted(paise / 100, paise % 100); }
        }
        record OrderId(String value) { }
        record CustomerId(String value) { }               // another aggregate is referenced by ID, not by object
        record Sku(String code) { }

        /** Domain events: facts, named in the past tense, that other parts of the system react to. */
        sealed interface DomainEvent permits OrderPlaced, OrderCancelled { }
        record OrderPlaced(OrderId orderId, CustomerId customer, Map<Sku, Integer> quantities, Money total)
implements DomainEvent { }
        record OrderCancelled(OrderId orderId, String reason) implements DomainEvent { }

        enum OrderStatus { DRAFT, PLACED, CANCELLED }

        /** Entity inside the aggregate: identity (lineNo) that is unique only within its Order. */
        static final class OrderLine {
            final int lineNo; final Sku sku; final Money unitPrice; private int quantity;
            OrderLine(int lineNo, Sku sku, Money unitPrice, int quantity) {
                this.lineNo = lineNo; this.sku = sku; this.unitPrice = unitPrice; this.quantity = quantity;
            }
            Money subtotal() { return unitPrice.times(quantity); }
        }

        /** Aggregate root: the only door into the cluster. Every invariant is checked here, in one place. */
        static final class Order {
```

```
            static final int MAX_UNITS_PER_SKU = 10;
            private final OrderId id;
            private final CustomerId customer;
            private final List<OrderLine> lines = new ArrayList<>();
            private final List<DomainEvent> events = new ArrayList<>();
            private OrderStatus status = OrderStatus.DRAFT;

            Order(OrderId id, CustomerId customer) { this.id = id; this.customer = customer; }

            OrderId id() { return id; }
            OrderStatus status() { return status; }

            void addItem(Sku sku, Money unitPrice, int quantity) {
                requireDraft("add items");
                if (quantity <= 0) throw new IllegalArgumentException("Quantity must be positive");
                int already = lines.stream().filter(l -> l.sku.equals(sku)).mapToInt(l -> l.quantity).sum();
                if (already + quantity > MAX_UNITS_PER_SKU)
                    throw new IllegalStateException("At most " + MAX_UNITS_PER_SKU + " units of " + sku.code() + "
per order");
                lines.add(new OrderLine(lines.size() + 1, sku, unitPrice, quantity));
            }

            Money total() { return lines.stream().map(OrderLine::subtotal).reduce(new Money(0), Money::plus); }

            /** Ubiquitous language: "place" an order, not setStatus("PLACED"). */
            void place() {
                requireDraft("place");
                if (lines.isEmpty()) throw new IllegalStateException("Cannot place an empty order");
                status = OrderStatus.PLACED;
                Map<Sku, Integer> q = new HashMap<>();
                lines.forEach(l -> q.merge(l.sku, l.quantity, Integer::sum));
                events.add(new OrderPlaced(id, customer, Map.copyOf(q), total()));
            }

            void cancel(String reason) {
                if (status == OrderStatus.CANCELLED) return;              // idempotent
                status = OrderStatus.CANCELLED;
                events.add(new OrderCancelled(id, reason));
            }

            List<DomainEvent> pullEvents() { List<DomainEvent> e = List.copyOf(events); events.clear(); return e; }

            private void requireDraft(String action) {
                if (status != OrderStatus.DRAFT) throw new IllegalStateException("Cannot " + action + ": order is "
+ status);
            }

            @Override public boolean equals(Object o) { return o instanceof Order other && other.id.equals(id); }
// entity: by identity
            @Override public int hashCode() { return Objects.hash(id); }
        }

        /** Repository: a collection-like interface per AGGREGATE ROOT (no OrderLineRepository). */
        interface OrderRepository {
            Optional<Order> byId(OrderId id);
            void save(Order order);
        }

        /** Saves the aggregate, then publishes its events (in production: outbox table in the same transaction).
*/
        static final class InMemoryOrderRepository implements OrderRepository {
            private final Map<OrderId, Order> store = new HashMap<>();
            private final List<Consumer<DomainEvent>> subscribers = new ArrayList<>();
            void subscribe(Consumer<DomainEvent> s) { subscribers.add(s); }
            @Override public Optional<Order> byId(OrderId id) { return Optional.ofNullable(store.get(id)); }
            @Override public void save(Order order) {
                store.put(order.id(), order);
                for (DomainEvent e : order.pullEvents()) subscribers.forEach(s -> s.accept(e));
            }
```

```
        }

        // ---- A second bounded context: Shipping has its OWN model and words ("consignment", weight), not Sales'
Order ----
        record Consignment(String reference, int parcels, int weightGrams) { }

        /** Anti-corruption layer: translates a Sales event into Shipping's language, so Sales can change freely.
*/
        static final class ShippingTranslator implements Consumer<DomainEvent> {
            private final Map<String, Integer> gramsPerItem;          // Shipping's own catalog: weight, not price
            final List<Consignment> created = new ArrayList<>();
            ShippingTranslator(Map<String, Integer> gramsPerItem) { this.gramsPerItem = gramsPerItem; }
            @Override public void accept(DomainEvent e) {
                if (e instanceof OrderPlaced p) {
                    int grams = p.quantities().entrySet().stream()
                            .mapToInt(en -> gramsPerItem.getOrDefault(en.getKey().code(), 500) *
en.getValue()).sum();
                    Consignment c = new Consignment("SO-" + p.orderId().value(), (grams + 9_999) / 10_000, grams);
                    created.add(c);
                    System.out.println("   shipping context: created " + c);
                } else if (e instanceof OrderCancelled c) {
                    System.out.println("   shipping context: stop consignment SO-" + c.orderId().value() + " (" +
c.reason() + ")");
                }
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE: anemic model ==");
        var svc = new Before.OrderService();
        var o = new Before.Order();
        svc.addLine(o, "TEA-250", 1, 0.1);
        svc.addLine(o, "MUG-01", 1, 0.2);
        o.status = "SHIPPED";
        o.lines.add(new Before.OrderLine());                  // bypasses the service: nothing stops it
        o.lines.get(2).qty = -5;
        System.out.println("total=" + o.total + " status=" + o.status + " lines=" + o.lines.size() + " (a line was
added after shipping, with qty -5)");

        System.out.println("== AFTER: aggregate + value objects + events ==");
        var repo = new After.InMemoryOrderRepository();
        var shipping = new After.ShippingTranslator(Map.of("TEA-250", 260, "MUG-01", 400));
        repo.subscribe(shipping);
        repo.subscribe(e -> System.out.println("   event published: " + e.getClass().getSimpleName()));

        var order = new After.Order(new After.OrderId("1001"), new After.CustomerId("C-42"));
        order.addItem(new After.Sku("TEA-250"), After.Money.rupees(180), 2);
        order.addItem(new After.Sku("MUG-01"), new After.Money(34950), 1);
        try { order.addItem(new After.Sku("TEA-250"), After.Money.rupees(180), 9); }
        catch (IllegalStateException e) { System.out.println("rejected: " + e.getMessage()); }
        System.out.println("total " + order.total() + " (money in paise: 0.1 + 0.2 problems cannot happen)");
        order.place();
        repo.save(order);
        try { order.addItem(new After.Sku("MUG-01"), new After.Money(34950), 1); }
        catch (IllegalStateException e) { System.out.println("rejected: " + e.getMessage()); }

        System.out.println("== identity vs value equality ==");
        System.out.println("Money(100) equals Money(100)? " + new After.Money(100).equals(new After.Money(100)) + "
(value object)");
        var sameId = new After.Order(new After.OrderId("1001"), new After.CustomerId("someone-else"));
        System.out.println("two Order objects with id 1001 equal? " + order.equals(sameId) + "   (entity: identity
decides)");
        System.out.println("loaded from repository: " + repo.byId(new
After.OrderId("1001")).map(After.Order::status).orElseThrow());
        order.cancel("customer request");
        repo.save(order);
    }
}
```

Output:

```
== BEFORE: anemic model ==
total=0.30000000000000004 status=SHIPPED lines=3 (a line was added after shipping, with qty -5)
== AFTER: aggregate + value objects + events ==
rejected: At most 10 units of TEA-250 per order
total Rs 709.50 (money in paise: 0.1 + 0.2 problems cannot happen)
   shipping context: created Consignment[reference=SO-1001, parcels=1, weightGrams=920]
   event published: OrderPlaced
rejected: Cannot add items: order is PLACED
== identity vs value equality ==
Money(100) equals Money(100)? true   (value object)
two Order objects with id 1001 equal? true   (entity: identity decides)
loaded from repository: PLACED
   shipping context: stop consignment SO-1001 (customer request)
   event published: OrderCancelled
```

Before, the order is a bag of public fields: `double` money prints `0.30000000000000004`, the status is a free-form string, and code outside the service added a line with quantity −5 *after* the order "shipped" — nothing could stop it. *After*, `Money` is a value object (paise — 1/100 of a rupee — in a `long`, never negative, equal by value), `Order` is the aggregate root that owns its `OrderLine` entities and refuses changes once placed or beyond 10 units per SKU (Stock Keeping Unit — a product's unique code), `place()` speaks the business language and records an `OrderPlaced` event, and the repository publishes events only after saving. The Shipping context never sees Sales' `Order`: its translator turns the event into a `Consignment` with Shipping's own concepts (weight, parcels), using Shipping's own weight catalog. Two `Order` objects with the same id are equal (entity), two `Money(100)` values are equal (value object).

Trade-offs

DDD brings	DDD costs
Code that reads like the business; rules in one place	Time with domain experts; learning the vocabulary
Clear boundaries for teams and services	More types (value objects, events, translators)
Aggregates give a clear transaction and locking unit	Eventual consistency between aggregates must be designed and explained
Models can change independently per context	Translation code at every context border
Rich model is easy to unit-test (test one class in isolation) without a database	ORMs sometimes fight rich models (private setters, value object mapping)

When not to use it

- **CRUD-shaped apps** (Create, Read, Update, Delete — apps that mostly store and show records: forms over tables, an admin panel) with few rules — a simple layered design is enough; tactical DDD would be ceremony (KISS, YAGNI).
- **Small teams with one small model** — strategic splitting pays off when there are several teams or sub-domains.
- **Technical problems** (a compiler, a video encoder) where the complexity is algorithmic, not in business rules.
- Still worth borrowing even then: **value objects** and **ubiquitous naming** cost little and help everywhere.

Common mistakes

- **Anemic aggregates** — entities with public setters and "services" holding the rules. That is the *before* code with DDD names on it.
- **Huge aggregates** ("Customer" containing all orders, addresses and payments) — every change locks everything; keep aggregates small and reference others by id.
- **One repository per table** instead of per aggregate root.
- **Publishing events before the transaction commits** — consumers react to changes that may be rolled back (undone instead of committed, i.e. made permanent); use an outbox.
- **Sharing one domain model across services** "to avoid duplication" — it couples the teams; duplicate the small bits each context needs (DRY applies *within* a context, not across).

- **Treating DDD as folders** (`entities/`, `valueobjects/`) rather than as modelling with domain experts.
- Using **primitives for domain concepts** (`String email`, `double price`) — "primitive obsession" (Chapter 7).

Real-world examples

- **Microservice boundaries** at many companies are drawn along bounded contexts (orders, payments, shipping, catalog) — the default advice in Sam Newman's *Building Microservices* and in Microsoft's .NET microservices architecture guide, which uses DDD aggregates and domain events in its reference application *eShopOnContainers*.
- **Payments**: a `Payment` aggregate with an `Amount` value object and events such as `PaymentAuthorized`, `PaymentCaptured`, `PaymentRefunded`, consumed by ledger and notification contexts.
- **Java libraries**: `java.time` types (`LocalDate`, `Duration`) and `BigDecimal` are value objects in all but name; frameworks such as Axon (used in the CQRS notes' example) are built around aggregates and events.
- In this book: the grocery POS (point-of-sale checkout system, Chapter 19) keeps `Money` and tenders as value types and a `Sale` that guards its own states; the shipping locker (Chapter 14, §14.7) separates `Compartment` (reusable door) from `Booking` (one parcel's lifecycle).

In the interview

> **30-second answer:** "DDD models the software on the business. Strategically, I split the domain into bounded contexts — each with its own model and ubiquitous language, like Sales and Shipping — and connect them through events or APIs, translating at the border with an anti-corruption layer. Tactically, inside a context I use entities for things with identity, immutable value objects for things like Money, and small aggregates whose root enforces the invariants; one aggregate changes per transaction, others follow through domain events, and each aggregate root has a repository."

Follow-ups:

- *Q: Entity or value object — how do you decide?* A: Ask whether you track it over time by identity. An `Order` is (id 1001 stays 1001); an `Address` or `Money` is not — you replace it with a new value.
- *Q: How big should an aggregate be?* A: As small as the invariants allow; reference other aggregates by id and use events for rules that can be eventually consistent.
- *Q: How do bounded contexts relate to microservices?* A: A context is a good candidate boundary for a service (often one service per context), but a context can also be a module inside a monolith.
- *Q: How do you keep two aggregates consistent?* A: Change one per transaction, publish a domain event via an outbox, let a handler (or a saga — a chain of local transactions with compensating "undo" steps — for multi-step processes) update the other.

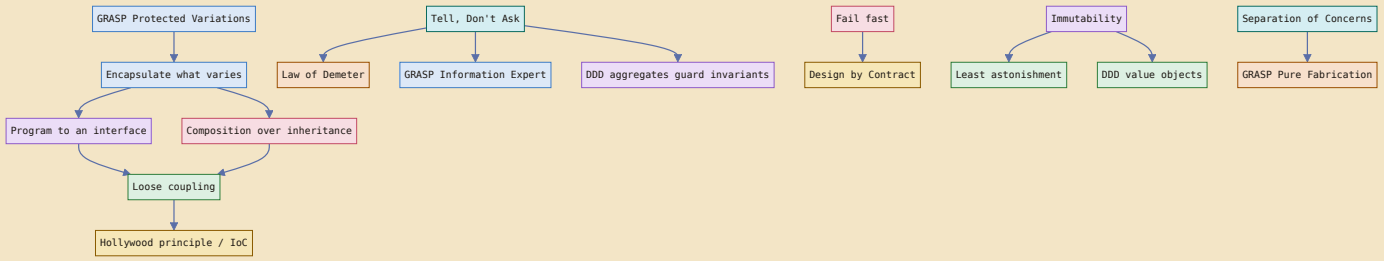
Self-check

1. Q: Two `Money(₹100)` objects — equal? Two customers named "Asha Rao" — equal? → A: Money yes (value object); customers only if they have the same id (entities).
2. Q: Why must outsiders not hold a reference to an `OrderLine`? → A: They could change it without the `Order` root checking its invariants.
3. Q: Why reference `CustomerId` instead of `Customer` inside `Order`? → A: Keeps aggregates small and independent; avoids loading or locking the customer when changing an order.
4. Q: What is an anti-corruption layer for? → A: Translating another context's or a legacy system's model into ours so its concepts don't leak in.
5. Q: When do you publish a domain event? → A: After the aggregate's change is committed — reliably via a transactional outbox.

TL;DR

Speak the business's language in code; split the domain into bounded contexts with their own models and translate at the borders; inside one context use entities (identity), immutable value objects (values), small aggregates whose root guards the invariants, domain events for everything that can happen later, and one repository per aggregate root.

Chapter summary — how the principles fit together



Principle	Main goal	Typical smell it fixes	Key Java tool
DRY	One home per rule	Duplicated rules drifting	Value objects, constants, methods
KISS	Simplicity	Over-engineering	Plain code, `switch` expressions
YAGNI	Build only what's needed	Speculative generality	Delete unused code
Law of Demeter	Hide structure	Train wrecks	Intent-revealing delegate methods
Tell, Don't Ask	Behaviour with data	Anemic model, feature envy	Command methods on the owner
Composition over inheritance	Flexible reuse	Fragile base class	Forwarding/decorators
Program to an interface	Substitutability	Hard-wired concrete deps	Interfaces + DI
Encapsulate what varies	Isolate change	Growing type-switches	Strategy, lambdas
Loose coupling	Independent change	God orchestrators	Observer, events, DI
Hollywood / IoC	Framework owns flow	Duplicated skeletons	Template Method, callbacks, DI
Separation of Concerns	One concern per unit	Spaghetti code	Layers, MVC
Least Astonishment	Predictability	Hidden side effects	Honest names, `Optional`
Fail fast	Early, clear errors	Distant NPEs (NullPointerExceptions)	Guard clauses, `requireNonNull`
Immutability	Safe sharing	Aliasing, races	Records, `List.copyOf`
Design by Contract	Clear obligations	Magic error values	Pre/postconditions, invariants
GRASP	Place responsibilities	God classes	Expert, Creator, Controller, ...
DDD	Model the business; clear boundaries	Anemic model, big ball of mud, primitive obsession	Records as value objects, aggregate roots, domain events, repositories

Tensions to remember (and mention in interviews):

- DRY vs loose coupling — sharing code couples its users; sometimes duplicate across boundaries.
- YAGNI vs Encapsulate-what-varies/OCP — only abstract variation that is real or clearly predicted.
- Law of Demeter vs cohesion — too many delegate methods create a Middle Man; move behaviour instead.
- Information Expert vs Separation of Concerns — domain objects compute, fabrications persist.
- DRY vs bounded contexts — share nothing but events and ids across contexts; a little duplication beats a shared model that couples teams.

Chapter self-check

1. Q: Which principle is violated by `order.getCustomer().getAddress().getCity()` inside a service that then decides shipping? → A: Law of Demeter (and Tell-Don't-Ask); move the decision to where the data is or ask `order.shippingZone()`.

2. Q: A `find` method that inserts a row violates? → A: Least Astonishment (and Command-Query Separation).
3. Q: A `HashSet` subclass double-counts in `addAll` — which principle? → A: Composition over inheritance (fragile base class).
4. Q: Which GRASP guideline says "put `total()` on `Order`"? → A: Information Expert.
5. Q: What makes a record *truly* immutable when it holds a list? → A: Defensively copy it with `List.copyOf` in the compact constructor.
6. Q: In DDD, why is `Money` a value object but `Order` an entity? → A: Money is defined by its amount and currency and is replaced, not changed; an order keeps its identity while its lines and status change.

TL;DR (chapter)

Keep rules in one place, keep things simple, build only what's needed, let objects own their behaviour and talk only to friends, depend on small interfaces and compose, let frameworks call you, separate concerns, don't surprise people, fail early, prefer immutable values, state your contracts, use GRASP to decide who does what — and let DDD's bounded contexts, aggregates and value objects decide which classes exist.

Chapter 6 — Clean Code

Design principles (Chapters 4–5) decide *which classes exist and how they relate*. **Clean code** is about what happens *inside* those classes: names, functions, comments, error handling, tests. In machine-coding rounds (interview rounds where you build a small, working program in about 60–120 minutes), interviewers read your code line by line — clean code is often what separates "hire" from "no hire" when two candidates reach similar designs.

This chapter covers every major topic of Robert C. Martin's *Clean Code* (2008), in the book's order (Chapters 1–17; the two long case studies, Chapters 15–16, are summarised in §13), explained in our own words with runnable Java 21 before/after programs. It finishes with three practical best-practice topics: a **code review checklist**, an **error-handling strategy** for real services and **testing strategies** (unit vs integration tests, test doubles, mocks).

Key terms (defined once here):

Term	Meaning
Clean code	Code that is easy to read, understand and change by someone other than its author. It does one thing well, has tests, and has no surprises.
Code smell	A surface symptom that *usually* signals a deeper design problem (e.g. a 200-line method). Not a bug by itself.
Refactoring	Changing the internal structure of code without changing its observable behaviour (Chapter 7).
Level of abstraction	How "high-level" a statement is. `processPayroll()` is high; `sb.append(',')` is low.
Side effect	Any change a function makes besides returning a value (fields, files, globals, output).
Test double	Any object that stands in for a real dependency during a test (dummy, stub, spy, mock, fake).
TDD	Test-Driven Development: write a failing test, make it pass, refactor — in very short cycles.

What is clean code, and why bother?

What it is

Well-known programmers describe clean code in similar ways: it reads like well-written prose, it does one thing, it is elegant and efficient, it has tests, it contains no duplication, it expresses its intent, and it looks like it was written by someone who **cares**. A simple working definition: *code whose intent is obvious to the next reader, and which is easy to change safely.*

Analogy: a well-organised kitchen. Anyone can walk in, find the knives, cook, and clean up. A messy kitchen still "works" for its owner — until someone else has to cook in it.

The problem it solves

Messy code slows teams down more and more over time. Every change takes longer because nobody understands the code, and each change adds more mess. Eventually productivity approaches zero and the team asks for a "grand redesign", which often reproduces the same mess. We read code far more than we write it (the ratio is often quoted as more than 10:1), so making code easy to read makes writing faster too. "I'll clean it up later" rarely happens — *later equals never* (a saying known as LeBlanc's law).

How it works

1. Treat readability as a first-class requirement, not decoration.
2. Follow the **Boy Scout Rule**: *always leave the code a little cleaner than you found it* — rename one variable, split one long function, delete one dead comment each time you touch a file.
3. Write tests so you can clean without fear.
4. Clean in small steps (successive refinement): first make it work, then make it right. Never ship "it works, but it's a mess".

Trade-offs

Clean code takes slightly longer *the first time* you write it; it saves time on every later read and change. For truly throwaway code (a one-off script) the balance is different.

When not to use it

There is no case where unreadable code is a goal. But avoid **gold-plating** — endlessly polishing code that is already clear and not changing.

Common mistakes

- Thinking "working" means "done".
- Blaming deadlines: going fast by making a mess is an illusion; the mess slows you down immediately.
- Big-bang rewrites instead of continuous small improvements.

Real-world examples

- **Google's code review culture** requires reviewer approval for readability, and it has a "readability" certification process per language.
- **Linters and formatters** (tools that automatically flag suspicious code or rewrite its layout) running in CI — Continuous Integration, a server that builds and tests every change automatically — (Checkstyle, SpotBugs, SonarQube, google-java-format) enforce parts of clean code automatically.

In the interview

> **30-second answer:** "Clean code is code whose intent is obvious and which is cheap to change. In a machine-coding round I use intention-revealing names, small single-purpose functions, clear exceptions instead of error codes, immutable value objects (small objects such as `Money` that never change after creation) and a few tests — and I leave time to refactor once it works."

Self-check

1. Q: What is the Boy Scout Rule? → A: Leave the code a little cleaner than you found it.
2. Q: Why does readability matter more than writing speed? → A: Code is read far more often than it is written.

TL;DR

Clean code = obvious intent + safe to change. Clean a little every time you touch code.

---

1. Meaningful Names

What it is

Names are everywhere — variables, functions, parameters, classes, packages. A good name answers three questions: **why does it exist, what does it do, how is it used**. If a name needs a comment to explain it, the name is not good enough.

Analogy: labels on moving boxes. "Kitchen — glasses — fragile" helps; "Box 7" doesn't.

The problem it solves

Bad names force readers to keep a **mental map** ("`d` is elapsed days, `i1` is status, 4 means done"). Every reader pays that cost every time, and misunderstanding causes bugs.

How it works — the rules

Rule	Bad	Good
Reveal intention	`int d;`	`int elapsedTimeInDays;`
Avoid disinformation (names that suggest something false)	`accountList` for a `Set`; `l` / `O` that look like 1/0	`accounts`
Make meaningful distinctions — no number series or noise words	`a1, a2`; `ProductInfo` vs `ProductData`; `nameString`	`source, destination`; one `Product`
Use pronounceable names	`crtdTs`	`createdAt`
Use searchable names — avoid single letters and bare numbers except tiny local scopes	`7`, `e`	`MAX_LOGIN_ATTEMPTS`, `employee`
Avoid encodings — no Hungarian notation (type codes in names), no `m_` prefixes, no `I` prefix on interfaces	`strName`, `m_desc`, `IShapeFactory`	`name`, `description`, `ShapeFactory` (+ `ShapeFactoryImpl` or better, a specific name)
Avoid mental mapping	`r` for "lower-cased URL without host"	`normalizedPath`
Class names = nouns	`Manager`, `Processor`, `Data`, `Info` (vague)	`Customer`, `Invoice`, `AddressParser`
Method names = verbs; accessors `get` / `is` / `has`	`data()`	`postPayment()`, `isPosted()`
Prefer named static factories (static creation methods like `of...`) to overloaded constructors (several constructors with different parameter lists)	`new Money(23)` (rupees? paise, 1/100 of a rupee?)	`Money.ofRupees(23)`
Don't be cute	`yeetAll()`, `nuke()`	`deleteAll()`, `terminate()`
One word per concept	`fetch` / `retrieve` / `get` in sibling classes	pick one, e.g. `get`
Don't pun (same word, different meaning)	`add` for both "sum" and "append to collection"	`append`, `insert`
Use solution-domain names for technical concepts	—	`OrderRepository`, `RetryPolicy`, `EventQueue` (programmers know these)
Use problem-domain names for business concepts	—	`Premium`, `Policy`, `Claim`
Add meaningful context	lone `state`	`shippingState` or better, an `Address` class with `state`
Don't add gratuitous context	`AcmeShopCustomerAddress` in an app called "Acme Shop"	`Address`

Length rule of thumb: the **longer the scope, the longer the name**. `i` is fine for a 3-line loop; a field used across a class deserves a full name.

Java example — before/after

```
// File: lld/ch06/names/Main.java
package lld.ch06.names;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;

/** Meaningful names: the same logic, before and after renaming. */
public class Main {

    // ---------------- BEFORE ----------------
    static class Before {
        static class DtaRcrd102 {                       // unpronounceable, encoded, meaningless
            String strNm; int i1; int i2; boolean flg;
        }
        static class Mgr {                               // vague noun
            List<DtaRcrd102> lst = new ArrayList<>();   // "lst" of what?
            // what is 4? what is 18000? why flg?
            List<DtaRcrd102> getThem() {
                List<DtaRcrd102> l1 = new ArrayList<>();
                for (DtaRcrd102 x : lst) if (x.i1 == 4 && x.i2 > 18000 && !x.flg) l1.add(x);
                return l1;
            }
            int d;                                       // elapsed time in days? seconds? since what?
        }
    }

    // ---------------- AFTER ----------------
    static class After {
        enum TaskStatus { OPEN, IN_PROGRESS, BLOCKED, DONE }

        /** Class name = noun. Field names = what they hold, with units where it matters. */
        record Task(String title, TaskStatus status, Duration timeSpent, boolean archived) {
            boolean isCompleted() { return status == TaskStatus.DONE; }          // boolean method reads as a
question
            boolean tookLongerThan(Duration limit) { return timeSpent.compareTo(limit) > 0; }
        }

        static final class TaskBoard {
            private static final Duration LONG_RUNNING = Duration.ofHours(5);  // named constant, not a magic
number
            private final List<Task> tasks = new ArrayList<>();
            void add(Task task) { tasks.add(task); }                            // method name = verb
            List<Task> findCompletedLongRunningTasks() {                        // says exactly what it returns
                List<Task> result = new ArrayList<>();
                for (Task task : tasks)
                    if (task.isCompleted() && task.tookLongerThan(LONG_RUNNING) && !task.archived())
result.add(task);
                return result;
            }
        }

        /** Static factory methods with descriptive names beat overloaded constructors. */
        record Temperature(double kelvin) {
            static Temperature ofCelsius(double c) { return new Temperature(c + 273.15); }
            static Temperature ofFahrenheit(double f) { return new Temperature((f - 32) * 5 / 9 + 273.15); }
        }
    }

    public static void main(String[] args) {
        var mgr = new Before.Mgr();
        var r = new Before.DtaRcrd102(); r.strNm = "Deploy"; r.i1 = 4; r.i2 = 20000; mgr.lst.add(r);
        System.out.println("BEFORE getThem(): " + mgr.getThem().size() + " record(s) -- but what are they?");

        var board = new After.TaskBoard();
        board.add(new After.Task("Deploy", After.TaskStatus.DONE, Duration.ofHours(6), false));
```

```
        board.add(new After.Task("Write docs", After.TaskStatus.DONE, Duration.ofHours(1), false));
        System.out.println("AFTER findCompletedLongRunningTasks(): " + board.findCompletedLongRunningTasks());
        System.out.printf("Temperature.ofCelsius(25) = %.2f K, ofFahrenheit(212) = %.2f K%n",
                After.Temperature.ofCelsius(25).kelvin(), After.Temperature.ofFahrenheit(212).kelvin());
    }
}
```

Output:

```
BEFORE getThem(): 1 record(s) -- but what are they?
AFTER findCompletedLongRunningTasks(): [Task[title=Deploy, status=DONE, timeSpent=PT6H, archived=false]]
Temperature.ofCelsius(25) = 298.15 K, ofFahrenheit(212) = 373.15 K
```

The logic is the same shape — filter a list by three conditions — but *after* you can read the rule aloud: "completed tasks that took longer than five hours and aren't archived". Magic numbers (unexplained literal values dropped into code) became a named constant, a status code became an enum (a type with a fixed set of named values), and `Temperature.ofCelsius(25)` is unambiguous where `new Temperature(25)` would not be.

Trade-offs

Longer names cost a few keystrokes (IDEs — Integrated Development Environments such as IntelliJ IDEA — autocomplete them). Very long names can hint the function does too much — split it rather than abbreviate.

When not to use it

Conventional short names are fine where universally understood: `i`/`j` loop indexes in tiny loops, `e` for a caught exception, `x`/`y` for coordinates, `T`/`K`/`V` for generics (type parameters, as in `List<T>`, that let one class work with many types).

Common mistakes

- Renaming is cheap — not doing it because "everyone knows what `tmp2` means".
- Abbreviations only the author understands (`calcAmtPrc`).
- Inconsistent vocabulary across a codebase (`customer`, `client`, `user` for the same concept).

Real-world examples

- The JDK's naming: `isEmpty()`, `getOrDefault()`, `computeIfAbsent()`, `List.of()`, `Path.of()`, `Duration.ofMinutes()` — verbs for methods, named static factories.
- **Google Java Style Guide** and Oracle's code conventions standardise casing (`UpperCamelCase` classes, `lowerCamelCase` methods, `UPPER_SNAKE_CASE` constants).

In the interview

> **30-second answer:** "I name things so the code reads like the requirement: nouns for classes, verbs for methods, booleans as questions, named constants instead of magic numbers, enums instead of status codes, and one consistent word per concept."

Follow-ups:

- *Q: Should interfaces start with `I`?* A: Not in Java convention; name the role (`PaymentGateway`) and give implementations specific names (`RazorpayGateway`).
- *Q: When is a short name okay?* A: When the scope is tiny and the meaning conventional (`i` in a short loop).

Self-check

1. Q: What's wrong with `ProductInfo` and `ProductData` in one codebase? → A: Noise words; no meaningful distinction.
2. Q: Why prefer `Temperature.ofCelsius(x)` over a constructor? → A: The name says what the argument means.
3. Q: What's a pun in naming? → A: Using one word for two different meanings.

TL;DR

Names should reveal intent, be pronounceable and searchable, use one word per concept, and let the code read like the requirement.

2. Functions

What it is

Functions (methods) are the first line of organisation in any program. Clean functions are **small**, **do one thing**, stay at **one level of abstraction**, take **few arguments**, have **no hidden side effects**, and either **do something or answer something** — never both.

Analogy: a recipe with numbered steps ("make the sauce", "boil pasta", "combine") where each step has its own sub-recipe. You'd never write one 400-line paragraph.

The problem it solves

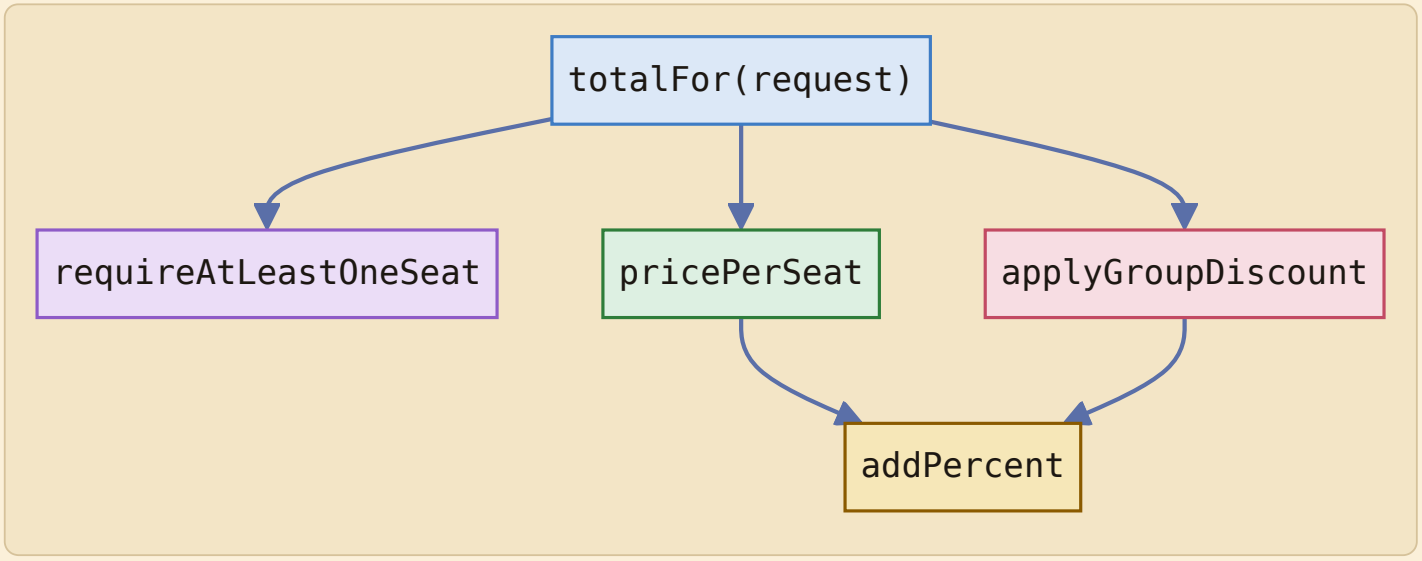
Long functions mix levels of detail, hide bugs, can't be reused, can't be named well, and can't be tested in pieces. Flag arguments and output arguments confuse callers; hidden side effects create surprising bugs; error codes clutter every caller with `if` checks.

How it works — the rules

1. **Small, then smaller.** The book says functions should hardly ever reach 20 lines, and praises functions of just two to four lines. Blocks inside `if`/`else`/`while` should ideally be one line — usually a call to a well-named function. Indentation should rarely exceed one or two levels.
2. **Do one thing.** A function does one thing if you can't extract another function from it with a name that isn't just a restatement of its body. If a function has "sections" (declarations, initialisation, processing), it's doing several things.
3. **One level of abstraction per function.** Don't mix `getHtml()` (high) with `.append("\n")` (low) in the same body.
4. **The Stepdown Rule.** Code should read top-down like a narrative: each function is followed by the functions it calls, one level of abstraction lower. "To do X, we do A, then B. To do A, we …".
5. **Switch statements.** A `switch` on a type code tends to be repeated in many places and to grow. Tolerate one only if it appears **once**, is buried in a low-level class (e.g. inside a factory — a class or method whose job is creating objects — that creates polymorphic objects, i.e. objects of different classes used through one shared interface), and is never repeated. Otherwise replace it with polymorphism (or enum data/behaviour).
6. **Use descriptive names.** A long descriptive name beats a short enigmatic one and a long comment. Small functions are easier to name.
7. **Arguments: fewer is better.** Zero (niladic) is ideal, then one (monadic), then two (dyadic); three (triadic) should be avoided where possible; more than three (polyadic) requires very special justification — and then shouldn't be used anyway. Arguments are hard to read and each one multiplies test cases.
   - Common one-argument forms: asking a question (`isValid(email)`), transforming (`Path resolve(name)`), or an event (`onLoginFailed(attempts)`) — make the kind obvious.
   - **Flag arguments** (`render(true)`) are ugly: they announce the function does two things. Split into two functions or pass an enum.
   - **Argument objects**: when several arguments travel together, wrap them (`book(BookingRequest request)` instead of `book(show, seatType, count, day)`).
   - **Argument lists** (varargs — a parameter like `Object... args` that accepts any number of values) count as one argument (`String.format(fmt, args...)`).
   - **Verbs and keywords**: encode argument roles in the name (`copyFromTo(source, target)`, `renameFileTo(newName)`).
8. **Have no side effects.** A function named `checkPassword` must not also initialise the session (the server-side record of a logged-in user). Hidden side effects create **temporal coupling** (it only works if called at the right time). If a side effect is necessary, put it in the name — or better, separate it.
9. **Avoid output arguments.** `addSignature(letter)` — does it add a signature *to* the letter, or the letter *to* something? Prefer `letter.addSignature()` (change the state of the owning object).
10. **Command-Query Separation (CQS).** A function should either *change state* (command, returns `void`) or *return information* (query, no state change), not both. `if (enable("darkMode"))` is confusing: is it asking whether it's enabled, or enabling it?
11. **Prefer exceptions to returning error codes.** Error codes force immediate `if` checks at every call site (each place in the code that calls the function) and deeply nested code; exceptions separate the happy path (the

normal, no-error flow) from error handling.

12. **Extract try/catch blocks.** Put the body of a `try` into its own function, so the function that handles errors does *only* error handling ("error handling is one thing": if `try` appears, it should be the first word in the function and nothing should follow the `catch`/`finally` blocks).
13. **Avoid an "error code magnet".** A central `enum Error` that everything imports forces recompiles and discourages new codes; exception subclasses can be added without touching existing code.
14. **Don't Repeat Yourself** — duplication is the root of much evil in software (see Chapter 5).
15. **Structured programming** (Dijkstra's one-entry, one-exit rule: one `return`, no `break`/`continue` in loops, never `goto`) matters mostly for large functions; small functions can use an occasional early `return`, `break` or `continue` when that makes them clearer (but still never `goto`).
16. **How to get there:** nobody writes clean functions the first time. Write a working rough draft with tests, then refine — split, rename, remove duplication — keeping the tests green (green = all tests passing).



Java example — before/after

```java
// File: lld/ch06/functions/Main.java
package lld.ch06.functions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/** Clean functions: small, one thing, one level of abstraction, few args, no flags, no hidden side effects, CQS, exceptions. */
@SuppressWarnings("serial") // the nested exception classes are never serialized
public class Main {

    // ---------------- BEFORE ----------------
    static class Before {
        static final Map<String, Object> SESSION = new HashMap<>();

        /**
         * Returns total in rupees, or -1 for bad input.
         * Does validation, pricing, surcharges, discounts, logging and session updates -- all in one place.
         */
        static int book(String seatType, int count, boolean weekend, List<String> log) {   // flag arg + output arg
            if (count <= 0) return -1;                                                      // error code
            int price;
            switch (seatType) {                                                             // switch on a type code
                case "SILVER": price = 150; break;
                case "GOLD": price = 250; break;
                case "RECLINER": price = 450; break;
                default: return -1;
            }
            if (weekend) price = price + price * 20 / 100;
            int total = price * count;
            if (count >= 4) total = total - total * 10 / 100;
            log.add("booked " + count + " " + seatType);                                    // mutates the caller's list
            SESSION.put("lastTotal", total);                                                // hidden side effect
            return total;
        }
    }

    // ---------------- AFTER ----------------
    static class After {
        /** The type-code switch is replaced by data on the enum; parsing happens once, at the boundary. */
        enum SeatType {
            SILVER(150), GOLD(250), RECLINER(450);
            final int basePrice;
            SeatType(int basePrice) { this.basePrice = basePrice; }
            static SeatType parse(String raw) {
                try { return valueOf(raw); }
                catch (IllegalArgumentException e) { throw new InvalidBookingException("unknown seat type '" + raw + "'"); }
            }
        }
        enum DayType {                          // replaces the boolean flag: callers write DayType.WEEKEND, not 'true'
            WEEKDAY(0), WEEKEND(20);
            final int surchargePercent;
            DayType(int p) { surchargePercent = p; }
        }
        /** Argument object: three related arguments become one concept. */
        record BookingRequest(SeatType seat, int seats, DayType day) {}

        static final class InvalidBookingException extends RuntimeException {
            InvalidBookingException(String message) { super(message); }
        }
```

```
        /** Stepdown rule: read top to bottom, each function one level more detailed than the one above. */
        static final class TicketPricer {
            private static final int GROUP_SIZE = 4;
            private static final int GROUP_DISCOUNT_PERCENT = 10;

            int totalFor(BookingRequest request) {                      // level 1: the story
                requireAtLeastOneSeat(request);
                int total = pricePerSeat(request) * request.seats();
                return applyGroupDiscount(total, request.seats());
            }

            private void requireAtLeastOneSeat(BookingRequest r) {      // level 2: the details
                if (r.seats() <= 0) throw new InvalidBookingException("seats must be positive, was " + r.seats());
            }
            private int pricePerSeat(BookingRequest r) {
                return addPercent(r.seat().basePrice, r.day().surchargePercent);
            }
            private int applyGroupDiscount(int total, int seats) {
                return seats >= GROUP_SIZE ? addPercent(total, -GROUP_DISCOUNT_PERCENT) : total;
            }
            private static int addPercent(int amount, int percent) {    // level 3: arithmetic
                return amount + amount * percent / 100;
            }
        }

        /** Command-Query Separation: book() changes state and returns nothing; history() answers and changes
    nothing. */
        static final class BookingService {
            private final TicketPricer pricer = new TicketPricer();
            private final List<String> history = new ArrayList<>();

            void book(BookingRequest request) {
                int total = pricer.totalFor(request);
                history.add(request.seats() + " x " + request.seat() + " on " + request.day() + " = Rs " + total);
            }
            List<String> history() { return List.copyOf(history); }
        }

        /** Error handling is "one thing": this function only translates failures for the UI. */
        static String bookFromForm(BookingService service, String seat, int seats, DayType day) {
            try {
                service.book(new BookingRequest(SeatType.parse(seat), seats, day));
                return "OK";
            } catch (InvalidBookingException e) {
                return "Sorry: " + e.getMessage();
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        List<String> log = new ArrayList<>();
        int result = Before.book("GOLD", 4, true, log);           // what does 'true' mean here?
        System.out.println("total=" + result + " log=" + log + " session=" + Before.SESSION);
        int bad = Before.book("PLATINUM", 2, false, log);
        System.out.println("bad seat returns " + bad + " -- caller must remember to check for -1");

        System.out.println("== AFTER ==");
        var service = new After.BookingService();
        System.out.println(After.bookFromForm(service, "GOLD", 4, After.DayType.WEEKEND));
        System.out.println(After.bookFromForm(service, "SILVER", 2, After.DayType.WEEKDAY));
        System.out.println(After.bookFromForm(service, "PLATINUM", 2, After.DayType.WEEKDAY));
        System.out.println(After.bookFromForm(service, "GOLD", 0, After.DayType.WEEKDAY));
        service.history().forEach(h -> System.out.println("  history: " + h));
    }
}
```

Output:

```
== BEFORE ==
total=1080 log=[booked 4 GOLD] session={lastTotal=1080}
bad seat returns -1 -- caller must remember to check for -1
== AFTER ==
OK
OK
Sorry: unknown seat type 'PLATINUM'
Sorry: seats must be positive, was 0
  history: 4 x GOLD on WEEKEND = Rs 1080
  history: 2 x SILVER on WEEKDAY = Rs 300
```

What changed and why:

- The **boolean flag** `weekend` became `DayType`, so call sites read `DayType.WEEKEND`, not a mysterious `true`.
- The **type-code switch** disappeared: each `SeatType` carries its base price; string parsing happens once at the boundary (`SeatType.parse`).
- The **error codes** (`-1`) became `InvalidBookingException` with a message saying exactly what was wrong.
- The **output argument** (`log`) and the **hidden side effect** (`SESSION`) became explicit state owned by `BookingService`.
- **CQS**: `book` is a command (returns `void`); `history` is a query (returns a copy, changes nothing). (Many real APIs pragmatically let a command return the created ID — that's a widely accepted exception.)
- **Stepdown**: `totalFor` reads as the story; the helpers below it are one level more detailed.
- **Error handling is one thing**: `bookFromForm` only translates failures into a UI message.

Trade-offs

Many small functions	Few large functions
Each is nameable, testable, reusable	Everything visible in one place
Need good names to navigate	Hard to understand, test, change
Too-tiny splitting can scatter logic	Mixed abstraction levels

When not to use it

Don't split a genuinely simple 8-line function into 8 one-liners just to hit a number; the goal is clarity, not a line count. Performance-critical inner loops rarely need manual inlining (copying a small method's body into its caller) in Java — the JIT (the Just-In-Time compiler inside the JVM, the Java Virtual Machine that runs Java programs, which turns frequently run bytecode into machine code while the program runs) inlines small methods — so size is almost never a real performance excuse.

Common mistakes

- Boolean parameters in public APIs (the methods and classes other code is meant to call).
- Getters that change state (breaking CQS and least astonishment).
- Catching exceptions deep inside and returning `null`.
- Functions whose name says one thing and body does three.

Real-world examples

- `java.util.Collections.sort(list)` vs `list.sort(comparator)` — command methods return `void`.
- `Iterator.next()` (an `Iterator` walks through a collection one element at a time) is a well-known pragmatic CQS exception (it both advances and returns); `Queue.poll()` likewise. Know them, but don't design new APIs that way without reason.
- `Files.readString`, `Files.writeString` — tiny, single-purpose APIs.

In the interview

30-second answer: "I keep functions small and single-purpose, at one level of abstraction, top-down by the stepdown rule. I avoid flag arguments with enums or separate methods, keep argument counts low with parameter objects, separate commands from queries, and throw descriptive exceptions instead of returning error codes."

Follow-ups:

- *Q: How small is small?* A: Small enough to have one clear name and one level of abstraction — often under ~20 lines, frequently 3–10.
- *Q: What's wrong with `boolean` parameters?* A: The call site is unreadable and the function does two things.
- *Q: What's CQS?* A: Commands change state and return nothing; queries return data and change nothing.

Self-check

1. Q: How do you know a function does "one thing"? → A: You can't extract another meaningful function from it that isn't a mere restatement.
2. Q: When is a `switch` acceptable? → A: Once, buried in a factory/low level, never repeated.
3. Q: What replaces 4+ related arguments? → A: An argument (parameter) object.

TL;DR

Small, one thing, one level, top-down, ≤2–3 args, no flags, no hidden side effects, commands or queries, exceptions not codes.

---

3. Comments

What it is

Comments are, at best, a necessary evil: we write them to make up for our failure to express ourselves in code. Code changes; comments often don't, so they drift and **lie**. The only truly accurate description of behaviour is the code itself. So: **explain yourself in code first**, and keep only comments that code *cannot* express.

Analogy: a road sign that says "Bridge closed" long after the bridge was reopened is worse than no sign.

The problem it solves

Comments used as a crutch for unclear code, redundant comments that repeat code, and outdated comments that mislead.

How it works

Replace a comment with a well-named function or variable whenever possible:

```
// free shipping if the order is big enough and domestic
if (order.total() >= 499 && !order.isInternational())
```

becomes `if (order.qualifiesForFreeShipping())` — the comment is no longer needed.

Good comments (keep these):

Kind	Example
Legal	Copyright/licence header
Informative	What a regex (text-matching pattern) matches, the format of a returned string
Explanation of intent	*Why* a decision was made ("we copy because callers may pass an unmodifiable list")
Clarification	Translating an obscure argument or return value from a library you can't change
Warning of consequences	"Not thread-safe" (unsafe to use from several threads at once), "takes 10 minutes to run", "billed per call"
TODO	Work that should be done but can't be now — with an owner; review them regularly
Amplification	Stressing the importance of something that looks trivial ("the trim is critical: …")
Javadoc in public APIs	Contracts callers depend on: params, return, exceptions

Bad comments (remove or replace):

Kind	What it looks like
Mumbling	A vague comment only the author understands
Redundant	`i++; // increment i`
Misleading	Subtly inaccurate description
Mandated	Javadoc on every function "because the rule says so" (`@param a the a`)
Journal	Change logs at the top of the file — version control (a tool such as Git that records every change) does this
Noise	`/** Default constructor. */`
Scary noise	Copy-pasted Javadoc noise with errors
Comment instead of a function/variable	Explaining an expression that could be named
Position markers	`// ********** Actions **********` banners
Closing-brace comments	`} // end while` — make the function shorter instead
Attributions and bylines	`/* Added by Rick */` — version control knows
Commented-out code	Others are afraid to delete it; it rots. Delete it — git remembers
HTML comments	Markup inside comments, unreadable in the editor
Nonlocal information	Commenting a system-wide default next to an unrelated function
Too much information	Historical discussions, RFC (internet standards document) excerpts
Inobvious connection	A comment that itself needs explaining
Function headers	Short, well-named functions don't need them
Javadoc in non-public code	Formal docs for internal code is usually noise

Java example — before/after

```java
// File: lld/ch06/comments/Main.java
/*
 * Copyright (c) 2026 Example Corp. Licensed under the Apache License, Version 2.0.   <-- GOOD: legal comment
 */
package lld.ch06.comments;

import java.util.List;
import java.util.regex.Pattern;

/** Comments: explain yourself in code first; keep only comments that code cannot express. */
public class Main {

    record Applicant(String name, int age, int creditScore, long monthlyIncomePaise, boolean hasDefaulted) {}

    // ---------------- BEFORE: comments used to excuse unclear code ----------------
    static class Before {
        // Changes:
        //   2019-03-01 RK: created
        //   2021-07-12 AB: fixed bug                                    <-- BAD: journal comment (git does this)
        /**
         * The eligible method.
         * @param a the a                                                <-- BAD: noise / mandated Javadoc
         * @return the boolean
         */
        static boolean eligible(Applicant a) {
            // check if applicant is eligible for a premium loan         <-- BAD: comment instead of a good name
            if (a.age() >= 21 && a.age() <= 60 && a.creditScore() > 750 && !a.hasDefaulted()
                    && a.monthlyIncomePaise() >= 5_000_000) {
                return true;   // return true                           <-- BAD: redundant
            }
            // if (a.age() > 65) return false;                          <-- BAD: commented-out code
            return false;
        } // end eligible                                               <-- BAD: closing-brace comment
    }

    // ---------------- AFTER: the code explains itself; comments add what code can't ----------------
    static class After {
        private static final int MIN_AGE = 21, MAX_AGE = 60;
        private static final int GOOD_CREDIT_SCORE = 750;
        private static final long MIN_MONTHLY_INCOME_PAISE = 5_000_000;   // Rs 50,000

        static boolean isEligibleForPremiumLoan(Applicant a) {
            return isWorkingAge(a) && hasGoodCredit(a) && earnsEnough(a);
        }
        private static boolean isWorkingAge(Applicant a) { return a.age() >= MIN_AGE && a.age() <= MAX_AGE; }
        private static boolean hasGoodCredit(Applicant a) { return a.creditScore() > GOOD_CREDIT_SCORE &&
!a.hasDefaulted(); }
        private static boolean earnsEnough(Applicant a) { return a.monthlyIncomePaise() >=
MIN_MONTHLY_INCOME_PAISE; }

        // GOOD: informative comment -- clarifies what a regex matches (format: 10-character PAN, e.g. ABCDE1234F)
        private static final Pattern PAN_FORMAT = Pattern.compile("[A-Z]{5}[0-9]{4}[A-Z]");

        static boolean looksLikePan(String s) { return PAN_FORMAT.matcher(s).matches(); }

        /**
         * GOOD: Javadoc on a public API -- states the contract callers rely on.
         * @param scores credit scores in the range 300..900
         * @return the median score; for an even count, the lower of the two middle values
         * @throws IllegalArgumentException if {@code scores} is empty
         */
        static int medianScore(List<Integer> scores) {
            if (scores.isEmpty()) throw new IllegalArgumentException("no scores");
            // GOOD: explanation of intent -- we copy because callers may pass an unmodifiable list, and sorting in
place would fail
            List<Integer> sorted = scores.stream().sorted().toList();
```

```
            return sorted.get((sorted.size() - 1) / 2);
        }

        // GOOD: warning of consequences -- this call hits the credit bureau and is billed per request.
        // TODO(loans-team): cache bureau responses for 24h once the bureau contract allows it.   <-- GOOD: TODO
        static int fetchBureauScore(String pan) { return 780; }
    }

    public static void main(String[] args) {
        var asha = new Applicant("Asha", 30, 790, 6_000_000, false);
        var ravi = new Applicant("Ravi", 30, 790, 6_000_000, true);
        System.out.println("BEFORE eligible(asha)=" + Before.eligible(asha) + ", eligible(ravi)=" +
Before.eligible(ravi));
        System.out.println("AFTER  isEligibleForPremiumLoan(asha)=" + After.isEligibleForPremiumLoan(asha)
                + ", (ravi)=" + After.isEligibleForPremiumLoan(ravi));
        System.out.println("looksLikePan(ABCDE1234F)=" + After.looksLikePan("ABCDE1234F"));
        System.out.println("medianScore([700, 820, 650, 900])=" + After.medianScore(List.of(700, 820, 650, 900)));
        System.out.println("bureau score=" + After.fetchBureauScore("ABCDE1234F"));
    }
}
```

Output:

```
BEFORE eligible(asha)=true, eligible(ravi)=false
AFTER  isEligibleForPremiumLoan(asha)=true, (ravi)=false
looksLikePan(ABCDE1234F)=true
medianScore([700, 820, 650, 900])=700
bureau score=780
```

In *before*, every comment is a smell: journal, mandated noise, a comment used instead of a name, redundant, commented-out code and a closing-brace marker. In *after*, the long condition became three named predicates and named constants; the remaining comments are the good kinds — legal header, informative (regex format), intent, public-API Javadoc, warning and a TODO with an owner.

Trade-offs

Comments are cheap to write and expensive to maintain. A few precise "why" comments are valuable; many "what" comments are a liability.

When not to use it

"No comments" is not the goal. Public libraries need Javadoc; complex algorithms (a published paper's method, a tricky concurrency protocol) deserve a comment explaining the approach and why.

Common mistakes

- Updating code but not the comment next to it.
- Keeping commented-out code "just in case".
- Using comments as a to-do list that no one ever reviews.

Real-world examples

- The **JDK's Javadoc** is a model of contract documentation (e.g. `HashMap` explains iteration order is not guaranteed; `ConcurrentHashMap` explains weakly consistent iterators).
- **SonarQube** (a server that scans code for problems) flags commented-out code; many teams' linters reject `TODO` without a ticket reference (an ID in the team's issue tracker).

In the interview

30-second answer: "I prefer expressive code over comments: named functions and constants replace most 'what' comments. I keep comments for why, warnings, legal headers, TODOs with owners and public API contracts, and I delete commented-out code because version control remembers it."

Self-check

1. Q: Name three good kinds of comment. → A: Legal, explanation of intent, warning of consequences (also informative, TODO, amplification, public Javadoc).
2. Q: What should you do with commented-out code? → A: Delete it.
3. Q: Why do comments lie? → A: Code changes and comments aren't updated.

TL;DR

Say it in code; comment only the *why*, warnings, legal text and public contracts; delete dead comments and commented-out code.

---

4. Formatting

What it is

Formatting is about **communication**: the layout of code tells the reader what belongs together and what matters most. A team should agree on one set of formatting rules and apply them automatically everywhere.

Analogy: a newspaper. The headline tells you the story, the first paragraph gives the summary, details come further down. You can stop reading at any depth and still understand the gist.

The problem it solves

Inconsistent, cramped or random layout makes readers work to understand structure before they can understand logic. It also produces noisy diffs (the line-by-line view of what changed) in code review (a teammate reading your change before it is accepted; §15).

How it works

Vertical formatting (top-to-bottom):

1. **File size** — small files are easier to understand. Many well-known Java projects keep most files around 200 lines, with very few over 500.
2. **Newspaper metaphor** — the class name and the top of the file give the high-level picture; details increase as you move down.
3. **Vertical openness** — separate distinct concepts (package, imports, each method) with blank lines.
4. **Vertical density** — lines that are tightly related appear together, without blank lines or useless comments between them.
5. **Vertical distance** — keep related concepts close:
   - declare local variables close to their first use;
   - instance variables in one well-known place (Java convention: top of the class);
   - **dependent functions**: the caller sits *above* the callee, close by;
   - **conceptual affinity**: functions that do similar things (overloads, related operations) sit near each other.
6. **Vertical ordering** — high-level functions first, details later (the stepdown rule again).

Horizontal formatting (left-to-right):

1. **Line width** — keep lines short enough to read without scrolling. Martin sets his own limit at 120 characters; Google's Java style uses 100.
2. **Horizontal openness and density** — spaces around assignment operators and between arguments; no space between a function name and its `(`; use spacing to show precedence where it helps.
3. **No horizontal alignment** of declarations in columns — it draws the eye to the wrong thing (types vs names) and breaks with every rename.
4. **Indentation** — shows scope hierarchy; never collapse a short `if` or loop body onto one line if it hides structure.
5. **Dummy scopes** — if a `while` or `for` has an empty body, make it visible (put the semicolon on its own line or use `{}`), because a trailing `;` is easy to miss.

Team rules — a codebase should look like one person wrote it. Encode the rules in an automatic formatter (IDE settings, `google-java-format`, Spotless, Checkstyle) and let CI enforce them.

Java example — before/after

```java
// File: lld/ch06/formatting/Main.java
package lld.ch06.formatting;

import java.util.ArrayList;
import java.util.List;

/** Formatting: same code, formatted two ways. The compiler doesn't care; readers do. */
public class Main {

    // ---------------- BEFORE: cramped, random order, aligned columns, no vertical openness ----------------
    static class Before {
        static class Playlist{private int cursor;
            private String helper(int i){return songs.get(i).toUpperCase();}
            public String next(){if(songs.isEmpty())return null;String s=helper(cursor);cursor=
(cursor+1)%songs.size();return s;}
            private final List<String>   songs   =   new ArrayList<>();
            private final String         name;
            public void add(String s){songs.add(s);}Playlist(String n){name=n;}
            public String name(){return name;}}
    }

    // ---------------- AFTER: newspaper order -- headline first, details below ----------------
    static class After {
        /** A looping playlist. (Headline: the class name and this one line tell you what it is.) */
        static final class Playlist {
            // 1. Fields at the top, together (vertical density for closely related lines).
            private final String name;
            private final List<String> songs = new ArrayList<>();
            private int cursor;

            // 2. Constructor next.
            Playlist(String name) {
                this.name = name;
            }

            // 3. Public API -- each concept separated by a blank line (vertical openness).
            public String name() {
                return name;
            }

            public void add(String song) {
                songs.add(song);
            }

            public String next() {
                if (songs.isEmpty()) {
                    return null;
                }
                String song = displayTitle(cursor);    // caller ABOVE callee (dependent functions close together)
                cursor = (cursor + 1) % songs.size();
                return song;
            }

            // 4. Private helpers just below the functions that use them.
            private String displayTitle(int index) {
                return songs.get(index).toUpperCase();
            }
        }
    }

    public static void main(String[] args) {
        var before = new Before.Playlist("Road trip");
        var after = new After.Playlist("Road trip");
        for (String s : List.of("Kesariya", "Yellow", "Tum Hi Ho")) { before.add(s); after.add(s); }
        StringBuilder b = new StringBuilder(), a = new StringBuilder();
        for (int i = 0; i < 4; i++) { b.append(before.next()).append(' '); a.append(after.next()).append(' '); }
```

```
        System.out.println("BEFORE " + before.name() + ": " + b.toString().trim());
        System.out.println("AFTER  " + after.name() + ": " + a.toString().trim());
        System.out.println("Same behaviour -- formatting only changes how fast a human understands it.");
    }
}
```

Output:

```
BEFORE Road trip: KESARIYA YELLOW TUM HI HO KESARIYA
AFTER  Road trip: KESARIYA YELLOW TUM HI HO KESARIYA
Same behaviour -- formatting only changes how fast a human understands it.
```

Both classes behave identically. The *after* uses the newspaper order (fields → constructor → public API → private helpers, with each helper right under its caller), blank lines between concepts, no column alignment and consistent indentation.

Trade-offs

Almost none — formatting is cheap when automated. The only cost is agreeing on the rules once.

When not to use it

Generated code (e.g. from Protobuf, Google's schema-based message format, whose compiler writes classes for you) doesn't need hand formatting; exclude it from formatters.

Common mistakes

- Mixing formatting changes with logic changes in the same commit (one saved, described change in version control) — it makes the diffs unreadable.
- Personal styles inside a team codebase.
- Aligning declarations in columns.

Real-world examples

- **google-java-format**, **Spotless** (Gradle/Maven — Java build tools), **Prettier** (JS), **gofmt** (Go formats all Go code identically — style debates disappear), **Black** (Python).
- **EditorConfig** files share indentation settings across editors.

In the interview

30-second answer: "Formatting communicates structure. I keep related lines together, separate concepts with blank lines, put callers above callees, fields at the top, and let an automatic formatter enforce one team style."

Self-check

1. Q: What is the newspaper metaphor? → A: High-level summary at the top, increasing detail below.
2. Q: Where should a private helper go? → A: Just below the function that calls it.

TL;DR

Format for the reader: headline first, related things close, concepts separated, one automated team style.

---

5. Objects and Data Structures

What it is

There are two fundamentally different kinds of types:

- An **object** *hides* its data behind abstractions and *exposes behaviour* (functions that operate on that data). Example: an `Order` with `isMetroDelivery()`.
- A **data structure** *exposes* its data and has *no meaningful behaviour*. Example: a record `Address(city, pincode)`, a DTO (Data Transfer Object — a plain data carrier).

Data abstraction: hiding implementation is not just putting getters and setters on private fields. It means exposing abstract interfaces that let users manipulate the *essence* of the data without knowing its representation (e.g. `fuelPercentRemaining()` rather than `getFuelTankCapacityInLitres()` and `getLitresOfFuel()`).

Analogy: a vending machine (object) — you press buttons and get results; you can't touch its coin box. A spreadsheet (data structure) — all values are visible; formulas live elsewhere.

The problem it solves

Confusion between the two creates **hybrids** — half object, half data structure — with public fields *and* significant behaviour. Hybrids get the worst of both worlds: hard to add new functions *and* hard to add new types. Knowing the difference also tells you which design makes your likely changes easy.

How it works — the data/object anti-symmetry

	Procedural code (data structures + functions)	**OO code (objects)**
Add a new **function** (e.g. `perimeter`)	**Easy**: add one function; data types untouched	**Hard**: add a method to every class
Add a new **type** (e.g. `Triangle`)	**Hard**: edit every function	**Easy**: add one class; existing code untouched

So: *everything is an object* is a myth. When you expect **new types**, choose objects and polymorphism. When you expect **new operations** over a stable set of types, data structures with functions (or the Visitor pattern, Chapter 11, which adds operations to a set of classes without editing them) may be better. Java 21's `sealed` interfaces (which list exactly which classes may implement them) with records and pattern-matching `switch` (a `switch` that branches on an object's type) make the procedural style safe: the compiler tells you if a `switch` misses a type.

The Law of Demeter (Chapter 5, §4) applies to objects: a method should not navigate an object's internals (`order.getCustomer().getAddress().city()`), because that exposes their structure — a **train wreck**. It does *not* strictly apply to data structures, whose whole job is to expose data. The fix for objects is to **hide structure**: ask the object to do the job (`order.isMetroDelivery()`).

Data Transfer Objects (DTOs): the purest data structure — public (or record) fields, no behaviour. Used to move data across boundaries: database rows, JSON (a plain-text key–value data format) request/response bodies, messages. *Bean* form (private fields + getters/setters) gives no real encapsulation benefit but is required by some frameworks.

Active Record: a special DTO that also has navigation methods like `save()` and `find()` and maps to one database row (Rails' ActiveRecord, some Java ORMs — Object-Relational Mappers, libraries that map classes to database tables). Common mistake: putting business rules into active records, making them hybrids. Keep business rules in separate objects that *use* the active record.

Java example — before/after

```java
// File: lld/ch06/objects/Main.java
package lld.ch06.objects;

import java.util.List;

/**
 * Objects vs data structures.
 * Objects hide data and expose behaviour; data structures expose data and have no behaviour.
 * Data/object anti-symmetry: each makes a different kind of change easy.
 */
public class Main {

    // ---------------- STYLE 1: data structures + procedures (procedural) ----------------
    static class Procedural {
        sealed interface Shape permits Square, Circle {}
        record Square(double side) implements Shape {}
        record Circle(double radius) implements Shape {}

        /** All behaviour lives here. Adding perimeter() = one new method, shapes untouched. Adding Triangle = edit
every method. */
        static final class Geometry {
            static double area(Shape s) {
                return switch (s) {
                    case Square sq -> sq.side() * sq.side();
                    case Circle c -> Math.PI * c.radius() * c.radius();
                };
            }
            static double perimeter(Shape s) {                // NEW FUNCTION: easy -- no shape class changes
                return switch (s) {
                    case Square sq -> 4 * sq.side();
                    case Circle c -> 2 * Math.PI * c.radius();
                };
            }
        }
    }

    // ---------------- STYLE 2: objects (polymorphic) ----------------
    static class ObjectOriented {
        interface Shape { double area(); }
        record Square(double side) implements Shape { public double area() { return side * side; } }
        record Circle(double radius) implements Shape { public double area() { return Math.PI * radius * radius; }
}
        record Triangle(double base, double height) implements Shape {   // NEW TYPE: easy -- nothing else changes
            public double area() { return 0.5 * base * height; }
        }
        // ...but adding perimeter() would mean editing Shape and EVERY implementation.
    }

    // ---------------- Hiding structure (Law of Demeter) ----------------
    static class Before {
        record Address(String city, String pincode) {}
        static class Customer { Address address = new Address("Pune", "411001"); Address getAddress() { return
address; } }
        static class Order { Customer customer = new Customer(); Customer getCustomer() { return customer; } }
        static boolean isMetroDelivery(Order o) {
            return List.of("Mumbai", "Delhi", "Pune").contains(o.getCustomer().getAddress().city());  // train
wreck
        }
    }
    static class After {
        record Address(String city, String pincode) {                    // a data structure: exposing data is its
job
            boolean isMetro() { return List.of("Mumbai", "Delhi", "Pune").contains(city); }
        }
        static final class Order {                                        // an object: hides its structure
            private final Address shipTo;
```

```
            Order(Address shipTo) { this.shipTo = shipTo; }
            boolean isMetroDelivery() { return shipTo.isMetro(); }        // tell the order what you need
        }
    }

    // ---------------- DTO: pure data for crossing a boundary ----------------
    /** A Data Transfer Object: public data, no behaviour; e.g. JSON returned by a REST endpoint. */
    record OrderDto(String orderId, String city, long totalPaise) {}

    public static void main(String[] args) {
        var shapes = List.<Procedural.Shape>of(new Procedural.Square(2), new Procedural.Circle(1));
        for (var s : shapes)
            System.out.printf("procedural %-22s area=%.2f perimeter=%.2f%n", s, Procedural.Geometry.area(s),
Procedural.Geometry.perimeter(s));

        var objects = List.<ObjectOriented.Shape>of(new ObjectOriented.Square(2), new ObjectOriented.Circle(1),
                new ObjectOriented.Triangle(3, 4));
        for (var s : objects) System.out.printf("object     %-22s area=%.2f%n", s, s.area());

        System.out.println("BEFORE train wreck says metro? " + Before.isMetroDelivery(new Before.Order()));
        System.out.println("AFTER  order.isMetroDelivery()? " + new After.Order(new After.Address("Pune",
"411001")).isMetroDelivery());
        System.out.println("DTO: " + new OrderDto("o-9", "Pune", 45_000));
    }
}
```

Output:

```
procedural Square[side=2.0]       area=4.00 perimeter=8.00
procedural Circle[radius=1.0]     area=3.14 perimeter=6.28
object     Square[side=2.0]       area=4.00
object     Circle[radius=1.0]     area=3.14
object     Triangle[base=3.0, height=4.0] area=6.00
BEFORE train wreck says metro? true
AFTER  order.isMetroDelivery()? true
DTO: OrderDto[orderId=o-9, city=Pune, totalPaise=45000]
```

The procedural `Geometry` gained `perimeter()` without touching the shapes; the OO version gained `Triangle` without touching anything. The train wreck `o.getCustomer().getAddress().city()` became `order.isMetroDelivery()`, while the `Address` record (a data structure) freely exposes `city`. `OrderDto` is a pure data carrier.

Trade-offs

Summarised in the anti-symmetry table above. Choose based on which change you expect more.

When not to use it

Don't force behaviour into DTOs that cross process boundaries (travel between separately running programs, e.g. over the network) — keep them dumb and version them carefully.

Common mistakes

- Adding getters and setters for every private field by reflex (that's not encapsulation).
- Hybrids: public mutable fields plus business methods.
- Business logic inside JPA (Java's standard ORM API) entities that also serve as API response bodies.

Real-world examples

- Records are Java's official data carriers; `java.awt.Point` has public fields (a data structure).
- `java.time.LocalDate` is an object: you call `plusDays`, `isLeapYear`; you never see its internal fields.
- DTOs in Spring (the main Java server framework) REST controllers (classes that handle HTTP API requests) (`@RequestBody CreateOrderRequest`); Rails ActiveRecord / Spring Data JPA entities.

In the interview

> **30-second answer:** "Objects hide data and expose behaviour; data structures expose data with no behaviour. OO makes adding types easy and functions hard; procedural does the opposite. I use records for DTOs and value data, rich objects for domain logic, and avoid hybrids and train wrecks."

Follow-ups:

- *Q: Is `a.getB().getC()` always wrong?* A: For objects, yes (Demeter). For pure data structures like records it's acceptable.
- *Q: Which pattern lets OO code add operations easily?* A: Visitor (Chapter 11), at the cost of making new types harder.

Self-check

1. Q: Which style makes adding a new shape type easy? → A: OO (polymorphic objects).
2. Q: What is a hybrid? → A: A type with both exposed data and significant behaviour — to be avoided.
3. Q: What's an Active Record? → A: A DTO with persistence/navigation methods mapped to a DB row.

TL;DR

Objects hide data and expose behaviour; data structures expose data. Pick based on whether you'll add types or operations. No hybrids, no train wrecks through objects.

---

6. Error Handling

What it is

Error handling is important, but if it **obscures the logic** it's wrong. Clean error handling keeps the happy path readable and makes failures explicit, informative and hard to ignore.

Analogy: a fire exit plan — clearly marked, separate from normal corridors, and never blocking daily traffic.

The problem it solves

Error codes clutter every caller with `if` checks and are easy to forget; `null` returns cause `NullPointerException`s far away; checked exceptions (ones the Java compiler forces every caller to catch or declare) from low-level libraries leak into every signature; vague messages make debugging slow.

How it works — the rules

1. **Use exceptions rather than return codes.** The caller's logic stays clean; errors can't be silently ignored.
2. **Write your `try-catch-finally` statement first** when writing code that can throw. The `try` block is like a transaction (a unit of work that either fully happens or is fully undone): whatever happens inside, the `catch` must leave the program consistent. In Java, prefer **try-with-resources** for anything `AutoCloseable` (any class with a `close()` method, such as a file or connection) — the resource is closed even if an exception is thrown.
3. **Use unchecked exceptions.** Checked exceptions violate the Open/Closed Principle: if a low-level method starts throwing a new checked exception, every method between it and the handler must change its signature. Modern languages (C#, Kotlin, Python, Scala) have no checked exceptions. Use checked exceptions only when a caller can and must realistically recover (a rare, deliberate choice).
4. **Provide context with exceptions.** Message should say what failed, with which values, and why (`account 'B' has 100 but 900 was requested`). Keep the original exception as the **cause** when wrapping.
5. **Define exception classes in terms of the caller's needs.** How will the caller *catch* it? Often one exception type (per area) is enough. **Wrap third-party APIs** so their exceptions are translated into yours; this minimises dependency on the vendor and makes swapping libraries easy.
6. **Define the normal flow — the Special Case pattern** (Martin Fowler): instead of throwing or returning `null` for an expected "absence" case, return an object that handles that case (`NoMembership` with 0 points, `GuestCustomer`, an empty list). Callers need no special handling.
7. **Don't return `null`.** Return an empty collection, `Optional` (a container that holds a value or is empty), a special-case object, or throw. Every `null` return is a future `NullPointerException`.

8. **Don't pass `null`.** Unless an API expects it, reject `null` arguments immediately (`Objects.requireNonNull`) — fail fast (Chapter 5).

Java example — before/after

```java
// File: lld/ch06/errors/Main.java
package lld.ch06.errors;

import java.io.IOException;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;

/** Error handling: exceptions not codes, unchecked with context, wrap third-party errors, special case, no null.
*/
@SuppressWarnings("serial") // the nested exception classes are never serialized
public class Main {

    // ---------------- BEFORE ----------------
    static class Before {
        static final int OK = 0, E_NO_ACCOUNT = 1, E_NO_MONEY = 2;
        static final Map<String, Long> BALANCES = new HashMap<>(Map.of("A", 500L, "B", 100L));
        static final Map<String, Integer> POINTS = new HashMap<>(Map.of("A", 40));

        static int transfer(String from, String to, long amount) {        // caller must check an int
            Long f = BALANCES.get(from), t = BALANCES.get(to);
            if (f == null || t == null) return E_NO_ACCOUNT;               // which one? no context
            if (f < amount) return E_NO_MONEY;                             // how much was missing?
            BALANCES.put(from, f - amount); BALANCES.put(to, t + amount);
            return OK;
        }
        static List<String> history(String id) { return null; }          // "no history" = null
        static Integer loyaltyPoints(String id) { return POINTS.get(id); } // null for non-members
    }

    // ---------------- AFTER ----------------
    static class After {
        /** Unchecked, domain-specific, with context. Defined by what the CALLER needs to handle. */
        static class BankingException extends RuntimeException {
            BankingException(String msg) { super(msg); }
            BankingException(String msg, Throwable cause) { super(msg, cause); }
        }
        static final class AccountNotFoundException extends BankingException {
            AccountNotFoundException(String id) { super("account '" + id + "' does not exist"); }
        }
        static final class InsufficientFundsException extends BankingException {
            InsufficientFundsException(String id, long requested, long available) {
                super("account '" + id + "' has " + available + " but " + requested + " was requested");
            }
        }

        /** SPECIAL CASE pattern: an object that represents "no loyalty membership" -- callers need no null check.
*/
        interface Loyalty { int points(); String tier(); }
        record Member(int points) implements Loyalty { public String tier() { return points >= 100 ? "GOLD" :
"SILVER"; } }
        enum NoMembership implements Loyalty { INSTANCE; public int points() { return 0; } public String tier() {
return "NONE"; } }

        /** Third-party library that throws its own checked exceptions (we can't change it). */
        static final class VendorSmsClient {
            void push(String phone, String text) throws IOException {
                if (phone.startsWith("000")) throw new IOException("socket timeout to sms-gw-3");
            }
        }
        /** Wrapper: one exception type our code understands; original kept as the cause. */
        static final class SmsNotifier {
            private final VendorSmsClient client = new VendorSmsClient();
            void send(String phone, String text) {
```

```
                try { client.push(phone, text); }
                catch (IOException e) { throw new BankingException("could not send SMS to " + phone, e); }
            }
        }

        static final class Bank {
            private final Map<String, Long> balances = new HashMap<>(Map.of("A", 500L, "B", 100L));
            private final Map<String, Integer> points = new HashMap<>(Map.of("A", 40));

            void transfer(String from, String to, long amount) {
                Objects.requireNonNull(from, "from"); Objects.requireNonNull(to, "to");     // don't accept null
                long available = balanceOf(from);
                balanceOf(to);                                                              // validates 'to'
exists
                if (available < amount) throw new InsufficientFundsException(from, amount, available);
                balances.merge(from, -amount, Long::sum);
                balances.merge(to, amount, Long::sum);
            }
            long balanceOf(String id) {
                Long b = balances.get(id);
                if (b == null) throw new AccountNotFoundException(id);
                return b;
            }
            List<String> history(String id) { return List.of(); }                          // empty, never null
            Optional<Long> findBalance(String id) { return Optional.ofNullable(balances.get(id)); }
            Loyalty loyaltyOf(String id) {
                Integer p = points.get(id);
                return p == null ? NoMembership.INSTANCE : new Member(p);
            }
        }

        /** "Write the try first": a resource that is always released, via try-with-resources. */
        static final class AuditFile implements AutoCloseable {
            AuditFile() { System.out.println("  audit file opened"); }
            void write(String line) { System.out.println("  audit: " + line); }
            @Override public void close() { System.out.println("  audit file closed"); }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        int code = Before.transfer("A", "Z", 50);
        System.out.println("transfer returned " + code + " (which account? caller has to decode)");
        try {
            System.out.println("history size " + Before.history("A").size());
        } catch (NullPointerException e) { System.out.println("history(): NullPointerException -- someone forgot
the null check"); }
        try {
            int pts = Before.loyaltyPoints("B");            // auto-unboxing a null Integer
            System.out.println(pts);
        } catch (NullPointerException e) { System.out.println("loyaltyPoints(B): NullPointerException on
unboxing"); }

        System.out.println("== AFTER ==");
        var bank = new After.Bank();
        for (String[] t : new String[][]{{"A", "Z", "50"}, {"B", "A", "900"}, {"A", "B", "200"}}) {
            try {
                bank.transfer(t[0], t[1], Long.parseLong(t[2]));
                System.out.println("transfer " + t[0] + "->" + t[1] + " of " + t[2] + " ok; A=" +
bank.balanceOf("A") + " B=" + bank.balanceOf("B"));
            } catch (After.BankingException e) {
                System.out.println("transfer failed: " + e.getMessage());
            }
        }
        System.out.println("history(A) size = " + bank.history("A").size());
        System.out.println("findBalance(Z) = " + bank.findBalance("Z"));
        for (String id : List.of("A", "B")) {
            var l = bank.loyaltyOf(id);                     // no null check needed
            System.out.println("loyalty " + id + ": " + l.points() + " pts, tier " + l.tier());
```

```
        }
        try {
            new After.SmsNotifier().send("000123", "hi");
        } catch (After.BankingException e) {
            System.out.println(e.getMessage() + " (cause: " + e.getCause().getMessage() + ")");
        }
        try (var audit = new After.AuditFile()) {
            audit.write("transfer A->B 200");
            throw new IllegalStateException("simulated crash");
        } catch (IllegalStateException e) {
            System.out.println("  caught: " + e.getMessage() + " -- file was still closed first");
        }
    }
}
```

Output:

```
== BEFORE ==
transfer returned 1 (which account? caller has to decode)
history(): NullPointerException -- someone forgot the null check
loyaltyPoints(B): NullPointerException on unboxing
== AFTER ==
transfer failed: account 'Z' does not exist
transfer failed: account 'B' has 100 but 900 was requested
transfer A->B of 200 ok; A=300 B=300
history(A) size = 0
findBalance(Z) = Optional.empty
loyalty A: 40 pts, tier SILVER
loyalty B: 0 pts, tier NONE
could not send SMS to 000123 (cause: socket timeout to sms-gw-3)
  audit file opened
  audit: transfer A->B 200
  audit file closed
  caught: simulated crash -- file was still closed first
```

What each part shows:

- *Before*: `transfer` returns `1` without saying which account; `history` returns `null`; `loyaltyPoints` returns `null` that crashes on auto-unboxing (Java's automatic conversion from `Integer` to `int`, which throws on `null`).
- *After*: an unchecked hierarchy under `BankingException`, messages with context; `history` returns an empty list; `findBalance` returns `Optional`; `loyaltyOf` returns a **special case** (`NoMembership.INSTANCE`) so callers never check for `null`; the vendor's checked `IOException` is **wrapped** with the original as its cause; `try-with-resources` closes the audit file even when an exception is thrown.

Trade-offs

Choice	Pro	Con
Unchecked exceptions	Clean signatures, OCP-friendly	Compiler doesn't remind callers; document them
Checked exceptions	Compiler-enforced handling	Signature ripple; often caught and swallowed
`Optional` return	Absence visible in the type	Not for fields/parameters; small allocation
Special Case object	No checks at all in callers	Can hide a real problem if absence *should* be noticed
Result/Either type (holds either a success value or an error)	Expected failures as values, composable	More verbose in Java (see Error-handling strategy)

When not to use it

Exceptions are for exceptional or error situations, not normal control flow — e.g. don't throw to exit a loop. For *expected, frequent* outcomes (validation of user input rows), returning a result value can be cleaner (see §16).

Common mistakes

- `catch (Exception e) {}` — swallowing.
- Logging and rethrowing at every layer (the same error logged 5 times).
- Losing the cause when wrapping (`new MyException(e.getMessage())` instead of `new MyException(msg, e)`).
- Returning `null` from collections-returning methods.

Real-world examples

- **Spring's** `DataAccessException` hierarchy translates vendor-specific `SQLException`s into unchecked, meaningful exceptions (e.g. `DuplicateKeyException`) — exactly "wrap third-party APIs, define exceptions by caller's needs".
- `Collections.emptyList()`, `Optional.empty()` — JDK support for "don't return null".
- `java.util.Objects.requireNonNull` — "don't pass null".
- `UncheckedIOException` (Java 8) wraps `IOException` for use in lambdas/streams (inline functions and Java's element-processing pipelines, whose standard interfaces cannot throw checked exceptions).

In the interview

> **30-second answer:** "I throw unchecked, domain-specific exceptions with context instead of returning codes or null. I wrap third-party exceptions at the boundary, return empty collections or Optional for absence, use a special-case object when absence is normal, validate nulls at entry, and use try-with-resources for cleanup."

Follow-ups:

- *Q: Checked vs unchecked?* A: Checked forces handling but ripples through signatures and breaks OCP; I default to unchecked and reserve checked for recoverable conditions callers must handle.
- *Q: What is the Special Case pattern?* A: A subclass/object representing a special situation (e.g. no membership) that behaves sensibly, so callers need no null checks.

Self-check

1. Q: Why are error codes bad? → A: They clutter callers, are easy to ignore, and lack context.
2. Q: What should a collection-returning method return when there's nothing? → A: An empty collection.
3. Q: Why keep the cause when wrapping? → A: So the original stack trace and message aren't lost.

TL;DR

Exceptions not codes; unchecked with context; wrap vendors; special cases, empty collections or Optional instead of null; never pass null; always clean up.

---

7. Boundaries

What it is

A **boundary** is where your code meets code you don't control: third-party libraries, frameworks, other teams' services, even very general JDK types like `Map`. Clean boundaries keep that foreign code at the edge, behind interfaces shaped for *your* needs.

Analogy: a power adapter when travelling. Your laptop doesn't care about each country's socket; the adapter handles it.

The problem it solves

Providers of libraries want broad applicability; users want interfaces focused on their needs. That tension leaks: a raw `Map<String, Session>` passed around lets any code `clear()` it, put wrong values in, and forces

everyone to know the key format. When the library's API changes, every usage site breaks.

How it works

1. **Wrap boundary types.** Don't pass `Map`, vendor clients or framework types around your application. Hide them inside a class that exposes only what your app needs (`SessionRegistry.open/activeSession/close`). Keep the boundary type **`private`** or at least out of public APIs.
2. **Write learning tests.** When adopting a library, write small tests that call its API the way you intend to use it. They document your understanding, cost nothing extra (you had to learn the API anyway), and — re-run on each upgrade — tell you immediately if the library's behaviour changed.
3. **Use code that does not exist yet.** When another team's API isn't ready, define the interface *you wish you had* (`FraudCheck`), code and test against it with a fake, and later write an **`Adapter`** (Chapter 9) that bridges it to the real API. This also gives you a clean seam (a place where a test can plug in a substitute) for testing.
4. **Keep boundary knowledge in very few places** so a vendor change touches one adapter.

Java example — before/after

```java
// File: lld/ch06/boundaries/Main.java
package lld.ch06.boundaries;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

import lld.ch06.testing.MiniTest;

/** Boundaries: wrap third-party / general-purpose APIs, learn them with tests, and define interfaces for code that doesn't exist yet. */
public class Main {

    record Session(String userId, Instant expiresAt) {}

    // ---------------- BEFORE: a raw Map passed all over the codebase ----------------
    static class Before {
        // Every caller can clear(), put(null, ...), put wrong types... and every caller must know the key format.
        static void loginHandler(Map<String, Session> sessions, String token, String user) {
            sessions.put(token, new Session(user, Instant.parse("2026-09-28T12:00:00Z")));
        }
        static void buggyAdminTool(Map<String, Session> sessions) { sessions.clear(); }   // oops: everyone logged out
    }

    // ---------------- AFTER: the Map is hidden behind an interface shaped for OUR needs ----------------
    static class After {
        static final class SessionRegistry {
            private final Map<String, Session> byToken = new ConcurrentHashMap<>();   // boundary type stays private

            void open(String token, Session session) { byToken.put(token, session); }
            Optional<Session> activeSession(String token, Instant now) {
                return Optional.ofNullable(byToken.get(token)).filter(s -> now.isBefore(s.expiresAt()));
            }
            void close(String token) { byToken.remove(token); }
            int size() { return byToken.size(); }
        }

        /** The fraud team's API isn't written yet: we define the interface WE want and code against it. */
        interface FraudCheck { boolean isSuspicious(String userId, long amountPaise); }

        /** Fake used until the real client exists; later an Adapter will wrap the real API behind the same interface. */
        static final class FakeFraudCheck implements FraudCheck {
            public boolean isSuspicious(String userId, long amountPaise) { return amountPaise > 10_000_000; }
        }
    }

    /** LEARNING TESTS: executable notes about how third-party (here: JDK) APIs really behave. Re-run them on every upgrade. */
    public static class JdkLearningTest {
        @lld.ch06.testing.Test void hashmap_allows_one_null_key() {
            Map<String, Integer> m = new HashMap<>();
            m.put(null, 1);
            MiniTest.assertEquals(1, m.get(null));
        }
        @lld.ch06.testing.Test void concurrenthashmap_rejects_null_keys() {
            MiniTest.assertThrows(NullPointerException.class, () -> new ConcurrentHashMap<String, Integer>().put(null, 1));
        }
        @lld.ch06.testing.Test void map_of_rejects_duplicate_keys() {
            MiniTest.assertThrows(IllegalArgumentException.class, () -> Map.of("k", 1, "k", 2));
        }
```

```
        @lld.ch06.testing.Test void string_split_drops_trailing_empty_strings() {
            MiniTest.assertEquals(2, "a,b,,".split(",").length);
            MiniTest.assertEquals(4, "a,b,,".split(",", -1).length);
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        Map<String, Session> raw = new HashMap<>();
        Before.loginHandler(raw, "t1", "asha");
        Before.buggyAdminTool(raw);
        System.out.println("sessions after admin tool: " + raw.size() + "  <-- nothing stopped the clear()");

        System.out.println("== AFTER ==");
        var registry = new After.SessionRegistry();
        registry.open("t1", new Session("asha", Instant.parse("2026-09-28T12:00:00Z")));
        System.out.println("active at 11:00? " + registry.activeSession("t1", Instant.parse("2026-09-
28T11:00:00Z")).isPresent());
        System.out.println("active at 13:00? " + registry.activeSession("t1", Instant.parse("2026-09-
28T13:00:00Z")).isPresent());
        After.FraudCheck fraud = new After.FakeFraudCheck();
        System.out.println("fraud(asha, Rs 2,00,000)? " + fraud.isSuspicious("asha", 20_000_000));

        System.out.println("== Learning tests ==");
        var summary = MiniTest.run(JdkLearningTest.class);
        System.out.println(summary.passed() + " passed, " + summary.failed() + " failed");
    }
}
```

Output:

```
== BEFORE ==
sessions after admin tool: 0  <-- nothing stopped the clear()
== AFTER ==
active at 11:00? true
active at 13:00? false
fraud(asha, Rs 2,00,000)? true
== Learning tests ==
JdkLearningTest
  PASS  concurrenthashmap rejects null keys
  PASS  hashmap allows one null key
  PASS  map of rejects duplicate keys
  PASS  string split drops trailing empty strings
4 passed, 0 failed
```

- The raw `Map` let an admin tool wipe every session. `SessionRegistry` exposes only open/lookup/close and adds the "not expired" rule in one place.
- `FraudCheck` is the interface we want; `FakeFraudCheck` unblocks us until the real client exists.
- `JdkLearningTest` captures surprising but real API facts (`HashMap` allows one `null` key, `ConcurrentHashMap` rejects it, `Map.of` rejects duplicate keys, `String.split` drops trailing empty strings unless you pass a negative limit).

Trade-offs

Wrapping costs a class and some delegation. It pays off whenever the boundary type is used in more than one place or might change.

When not to use it

Don't wrap stable, universal types used locally (a `Map` used inside one method is fine). The rule is about *passing* boundary types around your system.

Common mistakes

- Returning vendor types from your interfaces (`PaymentGateway.charge()` returning a `StripeCharge`) — leaks the vendor.

- Skipping learning tests and discovering library behaviour in production (the live system that real users use).

Real-world examples

- **Anti-Corruption Layer** in Domain-Driven Design: a translation layer between your model and a legacy (old, still-in-use)/external system's model.
- **SLF4J** (Simple Logging Facade for Java) wraps the actual logging library; **Spring's** `RestClient` / `WebClient` abstractions let you swap the HTTP engine.
- Contract tests (e.g. Pact) between microservices (small, independently deployed services) are a team-level version of learning tests.

In the interview

> **30-second answer:** "I keep third-party code at the edges: wrap it behind an interface shaped for my domain, don't pass raw library types around, write learning tests when adopting a library, and if a dependency doesn't exist yet I define the interface I need and use a fake until an adapter can bridge to the real API."

Self-check

1. Q: What is a learning test? → A: A test that exercises a third-party API to learn and pin down its behaviour.
2. Q: How do you handle an API that isn't built yet? → A: Define your own interface, use a fake, adapt later.

TL;DR

Keep foreign code at the edge behind your own interfaces; learn it with tests; adapt, don't leak.

---

8. Unit Tests

What it is

Unit tests are small, automated tests that check one unit of behaviour (usually a class or a few collaborating objects) in isolation. *Clean Code* argues test code is **as important as production code**: it must be clean, because it's what lets you change production code without fear.

Analogy: the safety net under a trapeze artist. Without it, every new move is dangerous; with it, you can try, fall and adjust.

The problem it solves

Without tests, every change risks breaking something, so people stop cleaning code and the design rots. With *dirty* tests, the tests themselves become a burden: they break with every change, take forever to update, and are eventually abandoned — and then production code rots too.

How it works

The three laws of TDD (the book says they lock you into a cycle perhaps thirty seconds long, with the tests just a few seconds ahead of the production code):

1. You may not write production code until you have written a failing unit test.
2. You may not write more of a unit test than is sufficient to fail (and not compiling counts as failing).
3. You may not write more production code than is sufficient to pass the currently failing test.

A related, coarser description (from Kent Beck's TDD, not from these three laws) is **Red → Green → Refactor**: write a failing test (red), make it pass in the simplest way (green), then clean up both code and tests (refactor).

Keep tests clean — they enable the "-ilities": tests make code flexible, maintainable and reusable, because you can change anything as long as the tests pass.

Clean tests are readable tests:

- Use the **Build–Operate–Check** pattern (also called **Arrange–Act–Assert** or **Given–When–Then**): build the test data, operate on it, check the results.

- Build a small **domain-specific testing language**: helper functions like `cart("TEA", 3)` and `checkoutWith(coupons, audit)` hide setup noise.
- **Dual standard:** test code must be clean and readable, but it need not be as efficient as production code (e.g. building strings for assertions is fine).

One assert per test? The book suggests minimising the number of asserts, and — more importantly — **one concept per test**. Several asserts are fine if they all check the same concept.

F.I.R.S.T. — clean tests are:

Letter	Meaning	How our code does it
Fast	Run in milliseconds so you run them constantly	In-memory doubles, no network/DB
Independent	No test depends on another or on order	`MiniTest` creates a fresh instance per test
Repeatable	Same result in any environment, any time	Fixed `Clock` instead of `now()`
Self-validating	Pass/fail automatically, no reading logs	Assertions, not `println`
Timely	Written just before (or with) the production code	TDD cycle

Java example — a JUnit-free runner, clean tests and a dirty one

Instead of JUnit (the standard Java testing framework), our tiny test framework (`@Test` + `MiniTest`) uses reflection (Java's ability to inspect classes and call methods by name at runtime) to find annotated methods (methods tagged with `@Test`), runs each on a new instance and reports PASS/FAIL. The production class `Checkout` has four collaborators behind interfaces, so tests can replace them with doubles.

```
// File: lld/ch06/testing/BeforeStyleTest.java
package lld.ch06.testing;

import java.time.Clock;
import java.util.List;
import java.util.Optional;

/**
 * BEFORE: how NOT to write tests. It "passes", but:
 *  - tests several concepts at once, so a failure doesn't say what broke
 *  - uses the real system clock (not Repeatable)
 *  - prints instead of asserting (not Self-validating: a human must read the output)
 *  - cryptic name and magic numbers
 */
public class BeforeStyleTest {
    @Test void test1() {
        var c = new Checkout(sku -> 12_000L, code -> Optional.empty(), r -> {}, e -> {},
Clock.systemDefaultZone());
        long a = c.quote(List.of(new Checkout.Line("X", 3)), null);
        long b = c.quote(List.of(new Checkout.Line("X", 3)), "NOPE");
        var r = c.checkout(List.of(new Checkout.Line("X", 1)), null, "z");
        System.out.println("        a=" + a + " b=" + b + " r=" + r + "  (is this right? someone must check by
eye)");
    }
}
```

```java
// File: lld/ch06/testing/Checkout.java
package lld.ch06.testing;

import java.time.Clock;
import java.time.LocalDate;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

/** The production code under test: a small checkout with four collaborators behind interfaces. */
public final class Checkout {

    public record Line(String sku, int qty) {}
    public record Coupon(String code, int percentOff, LocalDate expiresOn) {}
    public record Receipt(String email, long totalPaise) {}

    public interface PriceCatalog { long pricePaise(String sku); }
    public interface CouponRepository { Optional<Coupon> find(String code); }
    public interface ReceiptSender { void send(Receipt receipt); }
    public interface AuditLog { void record(String event); }

    static final long HIGH_VALUE_PAISE = 1_000_000;   // Rs 10,000

    private final PriceCatalog catalog;
    private final CouponRepository coupons;
    private final ReceiptSender sender;
    private final AuditLog audit;
    private final Clock clock;

    public Checkout(PriceCatalog catalog, CouponRepository coupons, ReceiptSender sender, AuditLog audit, Clock
clock) {
        this.catalog = catalog; this.coupons = coupons; this.sender = sender; this.audit = audit; this.clock =
clock;
    }

    /**
     * Query: computes the total, changes nothing.
     * {@code couponCode} may be {@code null}, meaning "no coupon" -- a documented part of this API's contract
     * (the "don't pass null" rule allows null only where an API explicitly expects it).
     */
    public long quote(List<Line> lines, String couponCode) {
        long subtotal = lines.stream().mapToLong(l -> catalog.pricePaise(l.sku()) * l.qty()).sum();
        return subtotal - discount(subtotal, couponCode);
    }

    /** Command: charges, sends the receipt, audits big orders. {@code couponCode} may be {@code null} (no coupon),
as in {@link #quote}. */
    public Receipt checkout(List<Line> lines, String couponCode, String email) {
        if (lines.isEmpty()) throw new IllegalArgumentException("cart is empty");
        Receipt receipt = new Receipt(email, quote(lines, couponCode));
        sender.send(receipt);
        if (receipt.totalPaise() >= HIGH_VALUE_PAISE) audit.record("high-value order for " + email);
        return receipt;
    }

    private long discount(long subtotal, String couponCode) {
        if (couponCode == null) return 0;
        return coupons.find(couponCode)
                .filter(c -> !LocalDate.now(clock).isAfter(c.expiresOn()))
                .map(c -> subtotal * c.percentOff() / 100)
                .orElse(0L);
    }

    // ---------- simple real implementations, used by the integration test ----------
    public static final class InMemoryCatalog implements PriceCatalog {
        private final Map<String, Long> prices = new ConcurrentHashMap<>();
```

```
        public InMemoryCatalog put(String sku, long paise) { prices.put(sku, paise); return this; }
        public long pricePaise(String sku) {
            Long p = prices.get(sku);
            if (p == null) throw new IllegalArgumentException("unknown sku " + sku);
            return p;
        }
    }
    public static final class InMemoryCoupons implements CouponRepository {
        private final Map<String, Coupon> byCode = new ConcurrentHashMap<>();
        public InMemoryCoupons save(Coupon c) { byCode.put(c.code(), c); return this; }
        public Optional<Coupon> find(String code) { return Optional.ofNullable(byCode.get(code)); }
    }
    public static final class OutboxSender implements ReceiptSender {
        private final List<Receipt> outbox = new CopyOnWriteArrayList<>();
        public void send(Receipt r) { outbox.add(r); }
        public List<Receipt> outbox() { return List.copyOf(outbox); }
    }
}
```

```
// File: lld/ch06/testing/CheckoutIntegrationTest.java
package lld.ch06.testing;

import static lld.ch06.testing.MiniTest.assertEquals;

import java.time.Clock;
import java.time.Instant;
import java.time.LocalDate;
import java.time.ZoneOffset;
import java.util.List;

/** INTEGRATION test: real collaborating implementations wired together, no doubles for our own code. */
public class CheckoutIntegrationTest {

    @Test void full_checkout_flow_with_real_components() {
        var catalog = new Checkout.InMemoryCatalog().put("TEA", 12_000).put("CUP", 8_000);
        var coupons = new Checkout.InMemoryCoupons().save(new Checkout.Coupon("WELCOME", 25, LocalDate.of(2030, 1,
1)));
        var outbox = new Checkout.OutboxSender();
        var checkout = new Checkout(catalog, coupons, outbox, event -> {}, Clock.fixed(Instant.parse("2026-09-
28T10:00:00Z"), ZoneOffset.UTC));

        checkout.checkout(List.of(new Checkout.Line("TEA", 2), new Checkout.Line("CUP", 1)), "WELCOME",
"meera@example.com");

        assertEquals(List.of(new Checkout.Receipt("meera@example.com", 24_000L)), outbox.outbox());
    }
}
```

```java
// File: lld/ch06/testing/CheckoutTest.java
package lld.ch06.testing;

import static lld.ch06.testing.MiniTest.assertEquals;
import static lld.ch06.testing.MiniTest.assertThrows;
import static lld.ch06.testing.MiniTest.assertTrue;

import java.time.Clock;
import java.time.Instant;
import java.time.LocalDate;
import java.time.ZoneOffset;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/** Clean UNIT tests: one concept each, Given/When/Then, every collaborator replaced by a test double. */
public class CheckoutTest {

    // ===== The five kinds of test double =====

    /** DUMMY: passed only to satisfy a parameter list; must never be used. */
    static final class DummyAuditLog implements Checkout.AuditLog {
        public void record(String event) { throw new AssertionError("dummy must not be called"); }
    }
    /** STUB: returns canned answers to the calls made during the test. */
    static final class StubCatalog implements Checkout.PriceCatalog {
        private final Map<String, Long> prices;
        StubCatalog(Map<String, Long> prices) { this.prices = prices; }
        public long pricePaise(String sku) { return prices.get(sku); }
    }
    /** FAKE: a working but simplified implementation (in-memory instead of a database). */
    static final class FakeCoupons implements Checkout.CouponRepository {
        private final List<Checkout.Coupon> rows = new ArrayList<>();
        FakeCoupons add(Checkout.Coupon c) { rows.add(c); return this; }
        public Optional<Checkout.Coupon> find(String code) { return rows.stream().filter(c ->
c.code().equals(code)).findFirst(); }
    }
    /** SPY: a stub that also RECORDS how it was called, so the test can inspect it afterwards. */
    static final class SpySender implements Checkout.ReceiptSender {
        final List<Checkout.Receipt> sent = new ArrayList<>();
        public void send(Checkout.Receipt r) { sent.add(r); }
    }
    /** MOCK: pre-programmed with EXPECTATIONS and verifies them itself. */
    static final class MockAuditLog implements Checkout.AuditLog {
        private final List<String> expected = new ArrayList<>();
        private final List<String> actual = new ArrayList<>();
        MockAuditLog expect(String event) { expected.add(event); return this; }
        public void record(String event) { actual.add(event); }
        void verify() { assertEquals(expected, actual); }
    }

    // ===== Fixture helpers (a tiny domain-specific testing language) =====
    private static final LocalDate TODAY = LocalDate.of(2026, 9, 28);
    private static final Clock FIXED_CLOCK = Clock.fixed(Instant.parse("2026-09-28T10:00:00Z"), ZoneOffset.UTC); //
Repeatable

    private final StubCatalog catalog = new StubCatalog(Map.of("TEA", 12_000L, "TV", 5_000_000L));
    private final SpySender sender = new SpySender();

    private Checkout checkoutWith(Checkout.CouponRepository coupons, Checkout.AuditLog audit) {
        return new Checkout(catalog, coupons, sender, audit, FIXED_CLOCK);
    }
    private static List<Checkout.Line> cart(Object... skuQtyPairs) {
        List<Checkout.Line> lines = new ArrayList<>();
        for (int i = 0; i < skuQtyPairs.length; i += 2) lines.add(new Checkout.Line((String) skuQtyPairs[i],
(Integer) skuQtyPairs[i + 1]));
        return lines;
```

```
    }

    // ===== Tests =====
    @Test void quote_sums_price_times_quantity() {
        Checkout checkout = checkoutWith(new FakeCoupons(), new DummyAuditLog());   // Given
        long total = checkout.quote(cart("TEA", 3), null);                          // When
        assertEquals(36_000L, total);                                               // Then
    }

    @Test void valid_coupon_takes_percentage_off() {
        var coupons = new FakeCoupons().add(new Checkout.Coupon("DIWALI10", 10, TODAY));
        long total = checkoutWith(coupons, new DummyAuditLog()).quote(cart("TEA", 10), "DIWALI10");
        assertEquals(108_000L, total);
    }

    @Test void expired_coupon_is_ignored() {
        var coupons = new FakeCoupons().add(new Checkout.Coupon("OLD", 50, TODAY.minusDays(1)));
        long total = checkoutWith(coupons, new DummyAuditLog()).quote(cart("TEA", 1), "OLD");
        assertEquals(12_000L, total);
    }

    @Test void checkout_sends_exactly_one_receipt() {
        checkoutWith(new FakeCoupons(), new DummyAuditLog()).checkout(cart("TEA", 1), null, "asha@example.com");
        assertEquals(List.of(new Checkout.Receipt("asha@example.com", 12_000L)), sender.sent);   // inspect the spy
    }

    @Test void high_value_order_is_audited() {
        var audit = new MockAuditLog().expect("high-value order for ravi@example.com");
        checkoutWith(new FakeCoupons(), audit).checkout(cart("TV", 1), null, "ravi@example.com");
        audit.verify();                                                             // the mock checks itself
    }

    @Test void empty_cart_is_rejected() {
        var checkout = checkoutWith(new FakeCoupons(), new DummyAuditLog());
        var e = assertThrows(IllegalArgumentException.class, () -> checkout.checkout(List.of(), null,
"x@example.com"));
        assertTrue(e.getMessage().contains("empty"), "message should explain the problem");
        assertTrue(sender.sent.isEmpty(), "nothing must be sent");
    }
}
```

```
// File: lld/ch06/testing/Main.java
package lld.ch06.testing;

/** Runs all test classes with the MiniTest runner and prints a summary. */
public class Main {
    public static void main(String[] args) {
        MiniTest.Summary total = new MiniTest.Summary(0, 0);
        for (Class<?> c : new Class<?>[]{BeforeStyleTest.class, CheckoutTest.class, CheckoutIntegrationTest.class})
{
            total = total.plus(MiniTest.run(c));
        }
        System.out.println("TOTAL: " + total.passed() + " passed, " + total.failed() + " failed");
        if (total.failed() > 0) System.exit(1);
    }
}
```

```java
// File: lld/ch06/testing/MiniTest.java
package lld.ch06.testing;

import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.Arrays;
import java.util.Comparator;
import java.util.Objects;

/**
 * A ~60-line, JUnit-free test runner.
 * - finds methods annotated with @Test (reflection)
 * - creates a FRESH instance of the test class for every test (tests stay Independent)
 * - reports PASS / FAIL / ERROR by itself (Self-validating)
 */
public final class MiniTest {
    private MiniTest() {}

    public record Summary(int passed, int failed) {
        public Summary plus(Summary o) { return new Summary(passed + o.passed, failed + o.failed); }
    }

    public static Summary run(Class<?> testClass) {
        System.out.println(testClass.getSimpleName());
        int passed = 0, failed = 0;
        Method[] tests = Arrays.stream(testClass.getDeclaredMethods())
                .filter(m -> m.isAnnotationPresent(Test.class))
                .sorted(Comparator.comparing(Method::getName))      // stable output; tests must not depend on
order
                .toArray(Method[]::new);
        for (Method m : tests) {
            String name = m.getName().replace('_', ' ');
            try {
                Object instance = testClass.getDeclaredConstructor().newInstance();
                m.setAccessible(true);
                m.invoke(instance);
                passed++;
                System.out.println("  PASS  " + name);
            } catch (InvocationTargetException e) {
                failed++;
                Throwable cause = e.getCause();
                String kind = cause instanceof AssertionError ? "FAIL " : "ERROR";
                System.out.println("  " + kind + " " + name + " -> " + cause);
            } catch (ReflectiveOperationException e) {
                failed++;
                System.out.println("  ERROR " + name + " -> cannot run: " + e);
            }
        }
        return new Summary(passed, failed);
    }

    // ---------- assertions ----------
    public static void assertEquals(Object expected, Object actual) {
        if (!Objects.equals(expected, actual))
            throw new AssertionError("expected <" + expected + "> but was <" + actual + ">");
    }
    public static void assertTrue(boolean condition, String message) {
        if (!condition) throw new AssertionError(message);
    }
    public static <T extends Throwable> T assertThrows(Class<T> type, Runnable action) {
        try {
            action.run();
        } catch (Throwable t) {
            if (type.isInstance(t)) return type.cast(t);
            throw new AssertionError("expected " + type.getSimpleName() + " but got " + t);
        }
        throw new AssertionError("expected " + type.getSimpleName() + " but nothing was thrown");
```

```
    }
}
```

```java
// File: lld/ch06/testing/Test.java
package lld.ch06.testing;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

/** Marks a method as a test for {@link MiniTest} -- our tiny stand-in for JUnit's @Test. */
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Test {}
```

Output:

```
BeforeStyleTest
        a=36000 b=36000 r=Receipt[email=z, totalPaise=12000]  (is this right? someone must check by eye)
  PASS  test1
CheckoutTest
  PASS  checkout sends exactly one receipt
  PASS  empty cart is rejected
  PASS  expired coupon is ignored
  PASS  high value order is audited
  PASS  quote sums price times quantity
  PASS  valid coupon takes percentage off
CheckoutIntegrationTest
  PASS  full checkout flow with real components
TOTAL: 8 passed, 0 failed
```

Reading the output: `BeforeStyleTest.test1` "passes" but proves nothing — it checks three concepts, uses the real clock, prints values for a human to judge and has a meaningless name. The `CheckoutTest` methods each check **one concept**, are named as behaviour specifications (`expired_coupon_is_ignored`), use Given/When/Then, a fixed clock and self-validating assertions. `CheckoutIntegrationTest` wires the real in-memory components together (see §17 for unit vs integration testing and the five kinds of test double used here).

Trade-offs

Well-tested code	Untested code
Fearless refactoring	Fear of change → rot
Tests are executable documentation	Knowledge in people's heads
Test code must be maintained too	Faster to write the first time

When not to use it

TDD is less useful for exploratory spikes (quick experiments to learn something — throw them away afterwards) and for pure UI layout. Don't unit-test trivial getters or framework code you don't own (use learning tests for that instead).

Common mistakes

- Testing implementation details (private methods, exact call order) so tests break on every refactor.
- Shared mutable fixtures (the data and objects set up before tests) that make tests order-dependent.
- Using `Thread.sleep` or the real clock/network in unit tests.
- Test names like `test1`, `testCheckout`.

Real-world examples

- **JUnit 5** (`@Test`, `@BeforeEach`, `assertThrows`, `@DisplayName`), **AssertJ** for fluent assertions, **Mockito** for doubles.

- JUnit 5's default lifecycle creates a **new test-class instance per test method** — the same independence choice `MiniTest` makes.
- **`java.time.Clock`** exists partly so time-dependent code can be tested with `Clock.fixed(...)`.

In the interview

> **30-second answer:** "I write small, fast, independent tests — one concept each, named as behaviour, arranged Given/When/Then. I inject collaborators behind interfaces so I can use stubs, fakes or mocks, and inject a `Clock` for time. In a machine-coding round I at least write a `main` that exercises key flows, and a few unit tests for the core logic if time allows."

Follow-ups:

- *Q: What are the three laws of TDD?* A: No production code without a failing test; write only enough test to fail; write only enough code to pass.
- *Q: What does F.I.R.S.T. stand for?* A: Fast, Independent, Repeatable, Self-validating, Timely.
- *Q: One assert per test?* A: The real rule is one concept per test; several asserts on one concept are fine.

Self-check

1. Q: What's wrong with printing results in a test? → A: Not self-validating — a human has to check.
2. Q: How do you make a time-based test repeatable? → A: Inject a `Clock` and use `Clock.fixed` in tests.
3. Q: What is Build–Operate–Check? → A: Arrange data, act on it, assert results (Given/When/Then).

TL;DR

Tests are first-class code: TDD cycle, one concept per test, readable Given/When/Then, and F.I.R.S.T.

---

9. Classes

What it is

Rules for clean classes: follow the standard **organisation**, keep classes **small** (measured in responsibilities, not lines), give each **one reason to change** (SRP), keep them **cohesive**, and **organise them for change** so new features mean new classes rather than edits to old ones.

Analogy: a well-run office has departments with clear jobs. When a new service is added, you open a new desk; you don't make accounting also handle legal.

The problem it solves

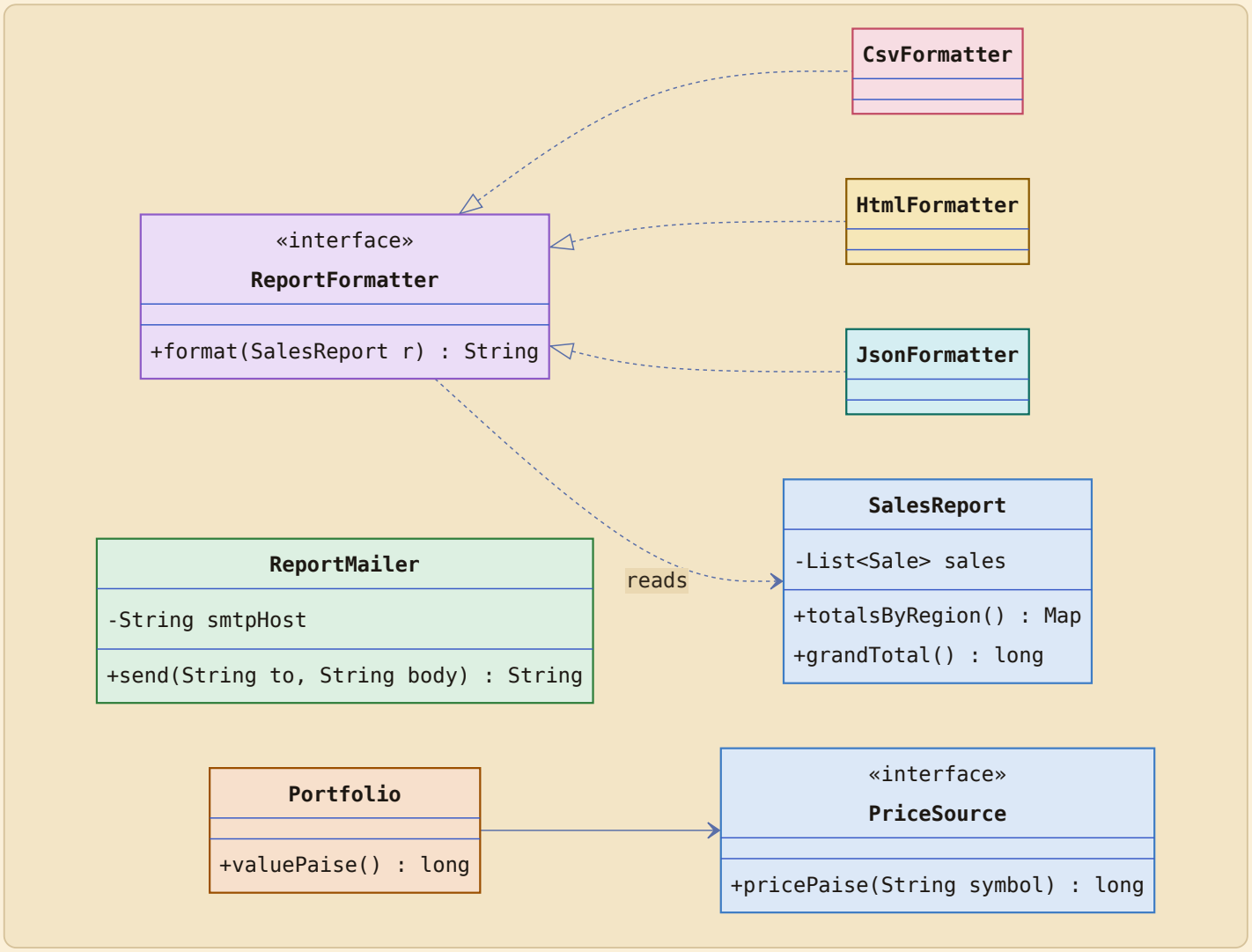
Big "god" classes with many responsibilities are hard to understand, change for many reasons, and every change risks breaking unrelated features. A long list of `if/else` branches for formats or types means reopening the same class repeatedly.

How it works

1. **Class organisation** (Java convention): public static constants → private static variables → private instance variables → public functions → private helpers right after the public function that uses them (stepdown rule). Public instance variables are rarely justified.
2. **Encapsulation**: keep fields and helpers private. Loosening to package-private/protected (visible only within the same package, or also to subclasses) *for tests* is acceptable as a last resort, but look for another way first.
3. **Classes should be small** — measured by *responsibilities*. The class name should describe them; if you can't name it concisely, or the name needs words like *Manager*, *Processor*, *Super*, or you describe it with "and"/"or"/"but", it does too much. You should be able to describe the class in about 25 words.
4. **Single Responsibility Principle**: a class should have **one reason to change** (Chapter 4). Many small classes are better than a few large ones — the total amount of code is the same; small classes are just better organised, like tools in labelled drawers instead of one big box.
5. **Cohesion**: a class is maximally cohesive if every method uses every instance variable. Aim for high cohesion. When a subset of methods uses a subset of fields, that subset wants to be its own class — **maintaining**

cohesion results in many small classes.

6. **Organise for change**: when adding a feature means *opening* a class to edit it (e.g. adding a branch to `render()`), consider splitting so that new features are new classes (Open/Closed Principle). Each report format became its own `ReportFormatter`.
7. **Isolate from change**: depend on abstractions (interfaces) rather than volatile concrete details. `Portfolio` depends on `PriceSource`, not a live stock-exchange client, so it can be tested with fixed prices (Dependency Inversion Principle).



Java example — before/after

```java
// File: lld/ch06/classes/Main.java
package lld.ch06.classes;

import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

/** Classes: small (few responsibilities), cohesive, organised for change, isolated from change. */
public class Main {

    record Sale(String region, long amountPaise) {}

    // ---------------- BEFORE: one class, many reasons to change ----------------
    static class Before {
        static class ReportManager {
            private final List<Sale> sales;
            private String smtpHost = "smtp.example.com";          // email concern
            private String outputFormat = "CSV";                   // formatting concern
            ReportManager(List<Sale> sales) { this.sales = sales; }
            void setOutputFormat(String f) { outputFormat = f; }
            Map<String, Long> totalsByRegion() {                   // business concern
                return sales.stream().collect(Collectors.groupingBy(Sale::region, java.util.TreeMap::new,
                        Collectors.summingLong(Sale::amountPaise)));
            }
            String render() {                                      // grows a new branch for every format
                if (outputFormat.equals("CSV")) {
                    StringBuilder sb = new StringBuilder("region,total\n");
                    totalsByRegion().forEach((r, t) -> sb.append(r).append(',').append(t).append('\n'));
                    return sb.toString().trim();
                } else if (outputFormat.equals("HTML")) {
                    StringBuilder sb = new StringBuilder("<table>");
                    totalsByRegion().forEach((r, t) -> sb.append("<tr><td>").append(r).append("</td>
<td>").append(t).append("</td></tr>"));
                    return sb.append("</table>").toString();
                }
                throw new IllegalStateException("unknown format");
            }
            String email(String to) { return "SMTP " + smtpHost + " -> " + to + ": " + render(); }
        }
    }

    // ---------------- AFTER ----------------
    static class After {
        /** Cohesive: every method uses the same field (sales). One reason to change: the report's business rules.
*/
        static final class SalesReport {
            private final List<Sale> sales;
            SalesReport(List<Sale> sales) { this.sales = List.copyOf(sales); }
            Map<String, Long> totalsByRegion() {
                return sales.stream().collect(Collectors.groupingBy(Sale::region, java.util.TreeMap::new,
                        Collectors.summingLong(Sale::amountPaise)));
            }
            long grandTotal() { return sales.stream().mapToLong(Sale::amountPaise).sum(); }
        }

        /** Organised for change: a new format is a NEW class; existing formatters are never reopened (OCP). */
        interface ReportFormatter { String format(SalesReport report); }

        static final class CsvFormatter implements ReportFormatter {
            public String format(SalesReport r) {
                StringBuilder sb = new StringBuilder("region,total\n");
                r.totalsByRegion().forEach((region, total) ->
sb.append(region).append(',').append(total).append('\n'));
                return sb.toString().trim();
            }
        }
```

```
        static final class HtmlFormatter implements ReportFormatter {
            public String format(SalesReport r) {
                StringBuilder sb = new StringBuilder("<table>");
                r.totalsByRegion().forEach((region, total) ->
                        sb.append("<tr><td>").append(region).append("</td><td>").append(total).append("</td>
</tr>"));
                return sb.append("</table>").toString();
            }
        }
        static final class JsonFormatter implements ReportFormatter {   // added later, nothing else edited
            public String format(SalesReport r) {
                return r.totalsByRegion().entrySet().stream()
                        .map(e -> "\"" + e.getKey() + "\":" + e.getValue())
                        .collect(Collectors.joining(",", "{", "}"));
            }
        }

        /** Separate reason to change: delivery. */
        static final class ReportMailer {
            private final String smtpHost;
            ReportMailer(String smtpHost) { this.smtpHost = smtpHost; }
            String send(String to, String body) { return "SMTP " + smtpHost + " -> " + to + ": " + body; }
        }

        /** Isolating from change: Portfolio depends on an abstraction, not on a live stock-exchange client. */
        interface PriceSource { long pricePaise(String symbol); }
        static final class Portfolio {
            private final PriceSource prices;
            private final Map<String, Integer> holdings;
            Portfolio(PriceSource prices, Map<String, Integer> holdings) { this.prices = prices; this.holdings =
Map.copyOf(holdings); }
            long valuePaise() {
                return holdings.entrySet().stream().mapToLong(h -> prices.pricePaise(h.getKey()) *
h.getValue()).sum();
            }
        }
    }

    public static void main(String[] args) {
        List<Sale> sales = List.of(new Sale("North", 5_000), new Sale("South", 7_000), new Sale("North", 1_000));

        System.out.println("== BEFORE ==");
        var manager = new Before.ReportManager(sales);
        System.out.println(manager.render().replace('\n', ' '));
        manager.setOutputFormat("HTML");
        System.out.println(manager.email("boss@example.com"));

        System.out.println("== AFTER ==");
        var report = new After.SalesReport(sales);
        for (After.ReportFormatter f : List.of(new After.CsvFormatter(), new After.HtmlFormatter(), new
After.JsonFormatter()))
            System.out.println(f.getClass().getSimpleName() + ": " + f.format(report).replace('\n', ' '));
        System.out.println(new After.ReportMailer("smtp.example.com").send("boss@example.com", new
After.JsonFormatter().format(report)));

        After.PriceSource fixedPrices = symbol -> Map.of("INFY", 150_000L, "TCS", 400_000L).get(symbol);  // test-
friendly stub
        var portfolio = new After.Portfolio(fixedPrices, Map.of("INFY", 10, "TCS", 2));
        System.out.println("portfolio value (paise) = " + portfolio.valuePaise());
    }
}
```

Output:

```
== BEFORE ==
region,total North,6000 South,7000
SMTP smtp.example.com -> boss@example.com: <table><tr><td>North</td><td>6000</td></tr><tr><td>South</td>
<td>7000</td></tr></table>
== AFTER ==
CsvFormatter: region,total North,6000 South,7000
HtmlFormatter: <table><tr><td>North</td><td>6000</td></tr><tr><td>South</td><td>7000</td></tr></table>
JsonFormatter: {"North":6000,"South":7000}
SMTP smtp.example.com -> boss@example.com: {"North":6000,"South":7000}
portfolio value (paise) = 2300000
```

`ReportManager` had three reasons to change (business totals, output format, email delivery) and an `if/else` that grows with every format. After the split, `JsonFormatter` was added without editing any existing class, and `Portfolio` is testable because it depends on the `PriceSource` abstraction.

Trade-offs

Many small classes	Few big classes
Each is easy to understand and test	Fewer files to navigate
Changes are localised	Every change touches the same class
Need good names and packages to navigate	Merge conflicts (clashing edits to the same lines), fear of change

When not to use it

Don't split a small cohesive class into many micro-classes that always change together — that's fragmentation, not SRP.

Common mistakes

- Classes named `*Manager`, `*Helper`, `*Utils` collecting unrelated methods.
- Splitting by *technical layer* only, leaving a god service.
- Making everything `public` "for tests".

Real-world examples

- JDK's `java.time` splits responsibilities across `LocalDate`, `LocalTime`, `ZonedDateTime`, `Duration`, `Period`, `DateTimeFormatter` — versus the old `java.util.Date` / `Calendar`, which mixed many concerns.
- Spring's `JdbcTemplate` delegates row mapping to `RowMapper` and parameter setting to `PreparedStatementSetter`.

In the interview

30-second answer: "I keep classes small by responsibility — one reason to change — and highly cohesive, so methods use most of the fields. When a feature would mean adding another branch to an existing class, I split so the new feature is a new class, and I depend on interfaces to isolate volatile details."

Self-check

1. Q: How do you measure a class's size? → A: By its responsibilities, not lines.
2. Q: What is maximal cohesion? → A: Every method uses every instance variable.
3. Q: What does "organise for change" mean? → A: Structure classes so new features are added as new classes, not edits.

TL;DR

Small (one responsibility), cohesive, open for extension, dependent on abstractions.

---

10. Systems

What it is

At the system level, clean code means **separating the startup process (constructing and wiring objects) from the runtime logic that uses them**, using **Dependency Injection**, handling **cross-cutting concerns** in one place, and letting the architecture **grow incrementally**.

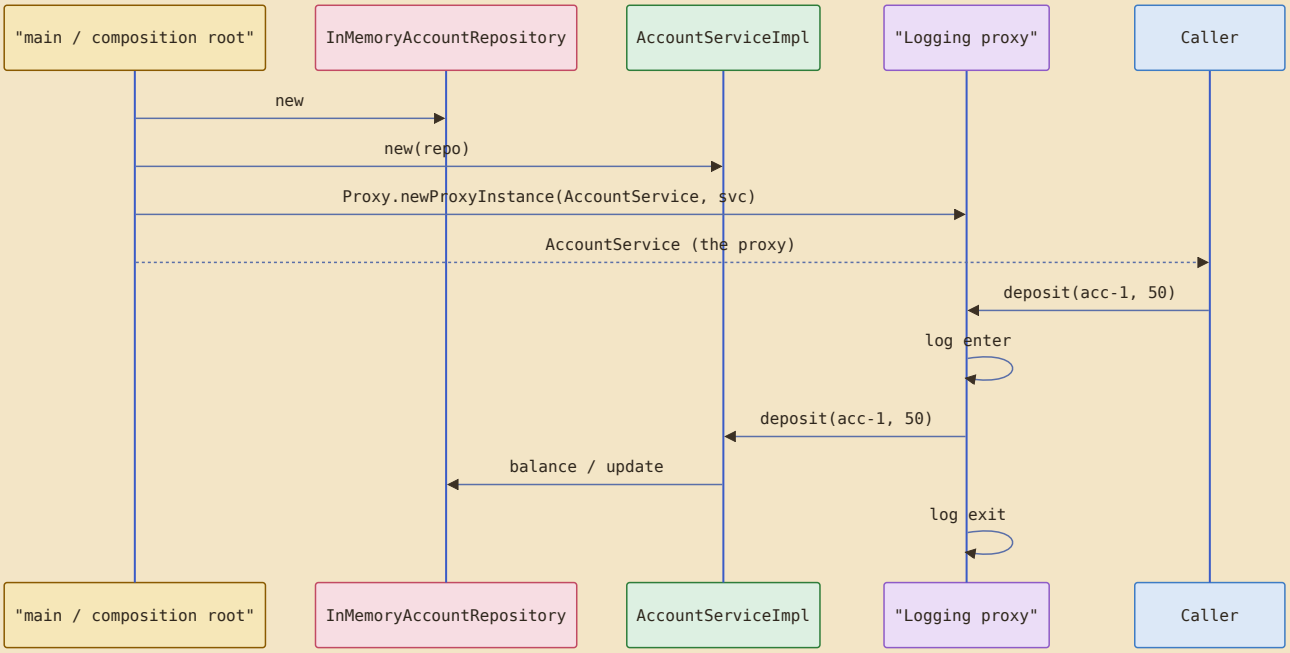
Analogy: building a city. Nobody designs every street up front, and no single person manages everything. Cities work because of abstraction and modularity — water, power, traffic are separate systems with clear interfaces, and they grow over time.

The problem it solves

When objects create their own dependencies (`if (service == null) service = new MyServiceImpl(...)` — lazy initialisation, i.e. creating something only the first time it is needed, inside business code), the business code is hard-wired to concrete classes, can't be tested in isolation, mixes construction with use, and violates SRP. When logging, security and transactions are copied into every method, every business method becomes noisy and inconsistent.

How it works

1. **Separate construction from use.**
   - **Separation of `main`**: build all objects in `main` (or a module called at startup — the **composition root**) and pass them to the application. The application knows nothing about how its objects were built; dependencies point from `main` toward the application, never back.
   - **Factories**: when the *application* must decide *when* to create something (e.g. a `LineItem` for each scan), give it an **Abstract Factory** interface (an interface whose methods create objects, so the caller never names the concrete classes); the concrete factory still lives on the `main` side.
   - **Dependency Injection (DI)**: an object receives its dependencies (constructor, setter or method parameter) instead of creating or looking them up. DI is the application of Inversion of Control (outside code, often a framework, takes charge of creating objects and driving the flow instead of your class) to dependency management. Constructor injection is preferred: dependencies are explicit, final, and the object is complete after construction.
2. **Scale up incrementally.** Build today's system well-modularised; well-separated concerns let the architecture grow (e.g. from a monolith — one application deployed as a single unit — module to a separate service) without a rewrite.
3. **Cross-cutting concerns** (persistence, transactions, security, logging, caching — keeping recent results in fast storage for reuse; see System Design Fundamentals) cut across many objects. Handle them modularly:
   - **Java dynamic proxies** (`java.lang.reflect.Proxy`) wrap an interface and intercept every call — shown in our code.
   - **Pure-Java AOP frameworks** (AOP = aspect-oriented programming: write a concern such as logging once, as an *aspect*, and apply it to many methods; e.g. Spring AOP) generate proxies for you from annotations (`@`-tags on code)/config (`@Transactional`, `@Cacheable`, `@PreAuthorize`).
   - **AspectJ** — a full aspect-oriented language extension that weaves advice (the aspect's code) into bytecode (the compiled form of Java classes that the JVM runs).
   - Also: **decorators** and **servlet filters / interceptors** (code that runs before/after every web request) for simpler cases.
4. **Test-drive the architecture**: with business logic in plain objects (POJOs — Plain Old Java Objects, with no framework dependencies) and concerns decoupled, you can start simple and evolve the architecture as needs become clear.
5. **Optimise decision making**: postpone decisions until the last responsible moment, when you have the most information; modularity makes postponement possible.
6. **Use standards wisely** — only when they add demonstrable value (heavyweight standards can bury simple logic).
7. **Systems need domain-specific languages**: a good DSL (domain-specific language — a mini-language tailored to one problem) (or fluent API — methods designed to chain into readable sentences) lets code read like the domain (e.g. a test-fixture builder or a routing DSL).


"main / composition root"
InMemoryAccountRepository
AccountServiceImpl
"Logging proxy"
Caller
new
new(repo)
Proxy.newProxyInstance(AccountService, svc)
AccountService (the proxy)
deposit(acc-1, 50)
log enter
deposit(acc-1, 50)
balance / update
log exit
"main / composition root"
InMemoryAccountRepository
AccountServiceImpl
"Logging proxy"
Caller

Java example — before/after

```
// File: lld/ch06/systems/Main.java
package lld.ch06.systems;

import java.lang.reflect.InvocationHandler;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Proxy;
import java.util.HashMap;
import java.util.Map;

/** Systems: separate construction from use, inject dependencies, handle cross-cutting concerns in one place. */
public class Main {

    interface AccountRepository { long balance(String id); void update(String id, long balance); }
    interface AccountService { long balance(String id); void deposit(String id, long amount); }

    static final class InMemoryAccountRepository implements AccountRepository {
        private final Map<String, Long> rows = new HashMap<>(Map.of("acc-1", 100L));
        public long balance(String id) { return rows.getOrDefault(id, 0L); }
        public void update(String id, long b) { rows.put(id, b); }
    }

    // ---------------- BEFORE: the service builds its own dependency (lazy init) and mixes in logging/timing -----------
    static class Before {
        static final class AccountServiceImpl implements AccountService {
            private AccountRepository repo;
            private AccountRepository repo() {
                if (repo == null) repo = new InMemoryAccountRepository();   // construction mixed with use; hard-coded impl
                return repo;
            }
            public long balance(String id) {
                System.out.println("  LOG enter balance " + id);          // cross-cutting code copy #1
                long b = repo().balance(id);
                System.out.println("  LOG exit balance = " + b);
                return b;
            }
            public void deposit(String id, long amount) {
                System.out.println("  LOG enter deposit " + id);          // cross-cutting code copy #2
                if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
                repo().update(id, repo().balance(id) + amount);
                System.out.println("  LOG exit deposit");
            }
        }
    }

    // ---------------- AFTER ----------------
    static class After {
        /** Pure business logic. Gets its dependency from outside (constructor injection). No logging code. */
        static final class AccountServiceImpl implements AccountService {
            private final AccountRepository repo;
            AccountServiceImpl(AccountRepository repo) { this.repo = repo; }
            public long balance(String id) { return repo.balance(id); }
            public void deposit(String id, long amount) {
                if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
                repo.update(id, repo.balance(id) + amount);
            }
        }

        /** Cross-cutting concern written ONCE and applied to any interface via a JDK dynamic proxy (how Spring AOP works for interfaces). */
        @SuppressWarnings("unchecked")
        static <T> T withLogging(Class<T> type, T target) {
            InvocationHandler handler = (proxy, method, args) -> {
                String argText = args == null ? "" : String.join(", ",
java.util.Arrays.stream(args).map(String::valueOf).toList());
```

```
                System.out.println("  LOG -> " + method.getName() + "(" + argText + ")");
                try {
                    Object result = method.invoke(target, args);
                    System.out.println("  LOG <- " + method.getName() + (result == null ? "" : " = " + result));
                    return result;
                } catch (InvocationTargetException e) {
                    System.out.println("  LOG !! " + method.getName() + " threw " + e.getCause().getMessage());
                    throw e.getCause();                         // rethrow the real exception, not the reflection
wrapper
                }
            };
            return (T) Proxy.newProxyInstance(type.getClassLoader(), new Class<?>[]{type}, handler);
        }

        /** Composition root: the ONLY place that knows concrete classes and wires them together (the "main" side).
*/
        static AccountService buildApplication() {
            AccountRepository repo = new InMemoryAccountRepository();
            AccountService core = new AccountServiceImpl(repo);
            return withLogging(AccountService.class, core);
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        var before = new Before.AccountServiceImpl();
        before.deposit("acc-1", 50);
        System.out.println("balance = " + before.balance("acc-1"));

        System.out.println("== AFTER ==");
        AccountService service = After.buildApplication();      // construction happens here, once
        service.deposit("acc-1", 50);                            // use happens everywhere else
        service.balance("acc-1");
        try { service.deposit("acc-1", -5); } catch (IllegalArgumentException e) { System.out.println("caller sees:
" + e.getMessage()); }

        // In a unit test, construct the core with a different repository -- no logging, no real storage.
        var testService = new After.AccountServiceImpl(new InMemoryAccountRepository());
        testService.deposit("acc-1", 1);
        System.out.println("test double wiring works: " + testService.balance("acc-1"));
    }
}
```

Output:

```
== BEFORE ==
  LOG enter deposit acc-1
  LOG exit deposit
  LOG enter balance acc-1
  LOG exit balance = 150
balance = 150
== AFTER ==
  LOG -> deposit(acc-1, 50)
  LOG <- deposit
  LOG -> balance(acc-1)
  LOG <- balance = 150
  LOG -> deposit(acc-1, -5)
  LOG !! deposit threw amount must be positive
caller sees: amount must be positive
test double wiring works: 101
```

In *before*, the service builds its own repository and repeats logging in every method. In *after*, `buildApplication()` (the composition root) is the only place that knows concrete classes; `AccountServiceImpl` receives its repository via its constructor; logging is written **once** in an `InvocationHandler` and applied through a JDK dynamic proxy. The last lines show that tests can wire the core differently. The proxy rethrows the *original* exception (`e.getCause()`), not the reflection wrapper.

Trade-offs

DI + composition root + proxies	Self-constructing objects
Testable, swappable, one place for wiring	Simple for tiny programs
Cross-cutting code in one place	Duplicated, inconsistent concerns
"Magic" with frameworks; proxies only intercept calls *through* the proxy	Explicit and visible

A real Spring gotcha that follows from the proxy mechanism: calling a `@Transactional` method *from another method of the same bean* (an object managed by Spring) (self-invocation) bypasses the proxy, so no transaction starts.

When not to use it

Small scripts and tiny interview programs don't need a DI framework — manual constructor injection in `main` is enough (and is what interviewers expect).

Common mistakes

- Service Locator (a global registry that code asks for its dependencies) calls (`Registry.get(Foo.class)`) scattered in business code — hides dependencies.
- Field injection with `@Autowired` on private fields — hides dependencies and blocks immutability; prefer constructor injection.
- Business logic depending on framework annotations everywhere.

Real-world examples

- **Spring IoC container / Spring Boot**, **Google Guice**, **Dagger** (compile-time DI, common on Android), **Jakarta CDI**.
- **Spring AOP** implements `@Transactional`, `@Cacheable`, `@Async` via JDK dynamic proxies (for interfaces) or CGLIB subclass proxies (generated subclasses that wrap the real class).
- Servlet filters and Spring `HandlerInterceptor`s for authentication and request logging.

In the interview

> **30-second answer:** "I separate construction from use: a composition root in `main` builds and wires everything, and classes receive dependencies through constructors. Cross-cutting concerns like logging or transactions go in one place — a decorator, proxy or aspect — not in every method. That keeps the domain in plain, testable objects."

Follow-ups:

- *Q: DI vs Service Locator?* A: DI pushes dependencies in (explicit in the constructor); a locator pulls them (hidden, harder to test).
- *Q: How does Spring implement `@Transactional`?* A: With a proxy around the bean that begins/commits/rolls back around the method call.

Self-check

1. Q: What is the composition root? → A: The one place (near `main`) where objects are constructed and wired.
2. Q: Name three ways to handle cross-cutting concerns in Java. → A: Dynamic proxies, AOP frameworks/AspectJ, decorators (also filters/interceptors).

TL;DR

Build objects in one place and inject them; put cross-cutting concerns in proxies/aspects; grow the architecture incrementally.

---

11. Emergence — Kent Beck's Four Rules of Simple Design

What it is

Good design can **emerge** from following four simple rules. In priority order, a design is simple if it:

1. **Runs all the tests** (passes them).
2. **Contains no duplication.**
3. **Expresses the intent of the programmer.**
4. **Minimises the number of classes and methods.**

(*Clean Code*'s Chapter 12, written by Jeff Langr, lists them in this order. **Kent Beck's own usual wording and order** — as summarised in Martin Fowler's "BeckDesignRules" article — is: **1. Passes the tests, 2. Reveals intention, 3. No duplication, 4. Fewest elements.** Fowler notes that the middle two are the ones people swap, and that it rarely matters because they reinforce each other. Rules 1 and 4 are always first and last. In an interview, quote Beck's order and mention that *Clean Code* swaps 2 and 3.)

Analogy: sculpting — first make sure the statue stands (tests), then remove what's repeated, then refine the expression, then trim anything unnecessary.

The problem it solves

Design decisions can feel overwhelming. These four rules give a simple, ordered checklist that consistently pushes code toward SRP, DIP and good structure.

How it works

1. **Runs all the tests** — a system you can't verify shouldn't be deployed. Making code testable pushes you toward small, single-purpose, loosely coupled classes (you need DI and interfaces to test in isolation). So rule 1 alone drives much good design.
2. **Refactor, guided by rules 2–4.** With tests in place, clean up without fear after every few lines:
   - **No duplication** — duplicate lines, and duplicate *implementation* (e.g. `isEmpty()` could be implemented as `size() == 0`). Extract methods, and use patterns like **Template Method** to remove higher-level duplication (the shared algorithm with varying steps).
   - **Expressive** — good names, small functions and classes, standard pattern names in class names (`...Command`, `...Visitor`), well-written tests as documentation. Most importantly: *care*.
   - **Minimal classes and methods** — don't create an interface for every class or split fields from behaviour dogmatically. This rule has the lowest priority: SRP and no-duplication win when they conflict.

Java example — before/after

```java
// File: lld/ch06/emergence/Main.java
package lld.ch06.emergence;

import lld.ch06.testing.MiniTest;
import lld.ch06.testing.Test;

/**
 * Kent Beck's four rules of simple design, in the priority order Clean Code gives:
 *  1. runs all the tests   2. no duplication   3. expresses intent   4. fewest classes and methods
 */
public class Main {

    record Employee(String name, int monthsWorked, boolean onProbation) {}

    // ---------------- BEFORE: duplicated algorithm, magic numbers, needless layers ----------------
    static class Before {
        interface LeaveCalcStrategyProvider { LeaveCalc get(); }            // extra element with no purpose
        static class LeaveCalc {
            int calcIN(Employee e) {                                         // unclear name
                if (e.onProbation()) return 0;
                int d = e.monthsWorked() * 18 / 12;                          // 18 per year? magic
                if (d > 45) d = 45;                                          // 45 cap? magic
                return d;
            }
            int calcUS(Employee e) {                                         // the same algorithm, copied
                if (e.onProbation()) return 0;
                int d = e.monthsWorked() * 15 / 12;
                if (d > 30) d = 30;
                return d;
            }
        }
    }

    // ---------------- AFTER: one algorithm (Template Method), named rules, only needed elements ----------------
    static class After {
        abstract static class LeavePolicy {
            /** The shared algorithm, written once (rule 2: no duplication). */
            final int earnedLeaveDays(Employee e) {
                if (e.onProbation()) return 0;
                return Math.min(daysPerYear() * e.monthsWorked() / 12, maxCarryForwardDays());
            }
            protected abstract int daysPerYear();              // what varies, with intention-revealing names (rule 3)
            protected abstract int maxCarryForwardDays();
        }
        static final class IndiaLeavePolicy extends LeavePolicy {
            protected int daysPerYear() { return 18; }
            protected int maxCarryForwardDays() { return 45; }
        }
        static final class UsLeavePolicy extends LeavePolicy {
            protected int daysPerYear() { return 15; }
            protected int maxCarryForwardDays() { return 30; }
        }
    }

    /** Rule 1 first: the refactoring is only safe because these tests pass before AND after. */
    public static class LeavePolicyTest {
        private final Employee veteran = new Employee("Asha", 48, false);
        private final Employee newHire = new Employee("Ravi", 8, false);
        private final Employee probationer = new Employee("Kabir", 3, true);

        @Test void india_policy_matches_old_behaviour() {
            var old = new Before.LeaveCalc();
            var policy = new After.IndiaLeavePolicy();
            for (Employee e : new Employee[]{veteran, newHire, probationer})
                MiniTest.assertEquals(old.calcIN(e), policy.earnedLeaveDays(e));
```

```
        }
        @Test void us_policy_matches_old_behaviour() {
            var old = new Before.LeaveCalc();
            var policy = new After.UsLeavePolicy();
            for (Employee e : new Employee[]{veteran, newHire, probationer})
                MiniTest.assertEquals(old.calcUS(e), policy.earnedLeaveDays(e));
        }
        @Test void carry_forward_is_capped() {
            MiniTest.assertEquals(45, new After.IndiaLeavePolicy().earnedLeaveDays(veteran));
        }
        @Test void probationers_earn_nothing() {
            MiniTest.assertEquals(0, new After.UsLeavePolicy().earnedLeaveDays(probationer));
        }
    }

    public static void main(String[] args) {
        var summary = MiniTest.run(LeavePolicyTest.class);
        System.out.println(summary.passed() + " passed, " + summary.failed() + " failed");
        var in = new After.IndiaLeavePolicy();
        var us = new After.UsLeavePolicy();
        for (Employee e : new Employee[]{new Employee("Asha", 48, false), new Employee("Ravi", 8, false)})
            System.out.println(e.name() + ": India " + in.earnedLeaveDays(e) + " days, US " + us.earnedLeaveDays(e)
+ " days");
    }
}
```

Output:

```
LeavePolicyTest
  PASS  carry forward is capped
  PASS  india policy matches old behaviour
  PASS  probationers earn nothing
  PASS  us policy matches old behaviour
4 passed, 0 failed
Asha: India 45 days, US 30 days
Ravi: India 12 days, US 10 days
```

The tests (rule 1) prove the refactoring kept behaviour identical. Duplicate algorithms became one Template Method (rule 2). Magic numbers became `daysPerYear()` / `maxCarryForwardDays()` (rule 3). The purposeless `LeaveCalcStrategyProvider` interface was dropped (rule 4).

Trade-offs

Rules 2–4 can pull against each other (removing duplication can add a class; expressiveness can add a method). Resolve by priority, and by looking for a better design that satisfies both.

When not to use it

Always applicable; it's a lightweight mindset, not a process.

Common mistakes

- Refactoring without tests (breaking rule 1 to satisfy the others).
- Dogmatically minimising classes at the expense of SRP.

Real-world examples

Extreme Programming teams apply these rules continuously. The Template Method refactoring in our example mirrors how `java.util.AbstractList` removes duplication across list implementations.

In the interview

30-second answer: "Kent Beck's rules of simple design, in priority order: passes the tests, reveals intention, no duplication, fewest elements. (*Clean Code* lists no-duplication before intention; the middle two reinforce each other.) I get the core working with a quick test or demo, then refactor out duplication and improve names, and I don't add classes that don't earn their place."

Self-check

1. Q: Which rule comes first and why? → A: Runs all the tests — without them you can't refactor safely, and testability itself improves design.
2. Q: Which rule has the lowest priority? → A: Minimal classes and methods.

TL;DR

Tests pass → no duplication → clear intent → nothing extra.

---

12. Concurrency

What it is

Concurrency — having several tasks in progress at once, usually on separate *threads* (independent paths of execution inside one program) — is a **decoupling strategy**: it separates *what* gets done from *when* it gets done. It can improve throughput (work completed per second; see System Design Fundamentals) and structure, but concurrent code is hard to get right. Clean concurrency means keeping concurrency code small, separate and built on well-tested library tools.

Analogy: a restaurant kitchen. Several cooks work at once. It works because each cook has their own station and ingredients; trouble starts when two cooks grab the same pan.

The problem it solves

Myths lead to bugs: *concurrency always improves performance* (only when there's waiting or multiple cores — CPU units that can each run a thread at the same moment — to exploit), *design doesn't change* (it changes a lot), *understanding concurrency doesn't matter with a container like a web server* (you still share state between requests). In reality, concurrency adds overhead, bugs are often not repeatable, and it requires a different design strategy.

How it works — defence principles

1. **Single Responsibility**: keep concurrency-related code separate from other code. Our `countWords` is pure single-threaded logic; the executor code (an executor runs tasks on a pool of reusable threads) is separate.
2. **Limit the scope of shared data**: encapsulate it and minimise the places that touch it; guard critical sections (code that touches shared data and must be run by one thread at a time, usually enforced with a *lock*).
3. **Use copies of data**: share immutable objects or give each thread its own copy and merge results at the end.
4. **Threads should be as independent as possible**: each thread processes its own data with no sharing (like each request having its own local variables).
5. **Know your library**: use thread-safe collections (`ConcurrentHashMap`), the executor framework (`ExecutorService`), non-blocking atomics (`AtomicInteger`, `LongAdder` — classes that update a value in one indivisible step without locks), and synchronisers (`CountDownLatch`, `Semaphore`, `CyclicBarrier`) instead of hand-made `wait/notify` code.
6. **Know your execution models** (Chapter 13 covers them in depth): **producer–consumer** (bounded queue between producers and consumers), **readers–writers** (many readers, exclusive writers; balance throughput vs starvation), **dining philosophers** (competing for multiple resources → deadlock, livelock, starvation). *Deadlock*: threads wait for each other forever; *starvation*: a thread never gets its turn.
7. **Beware dependencies between synchronised methods** (`synchronized` methods, which only one thread at a time can run on a given object) on the same shared object: if you must call several, use client-based locking, server-based locking (one method that does it all under its lock) or an adapted server.
8. **Keep synchronised sections small** — locks create contention (threads queuing for the same lock); lock as little code as possible, but correctness first.
9. **Writing correct shutdown code is hard** — plan graceful shutdown (letting running tasks finish before the program stops) from the start (`shutdown()` + `awaitTermination()`, then `shutdownNow()`), beware deadlocks where threads wait for signals that never come.
10. **Testing threaded code:**
    - Treat spurious, one-off failures as potential threading issues — don't dismiss them as "cosmic rays".
    - Get non-threaded code working first.

- Make threaded code **pluggable** (run with 1, 2, many threads) and **tunable** (thread counts configurable).
- Run with **more threads than processors** to force more task switching.
- Run on different platforms.
- **Instrument** code to try to force failures (hand-coded or automated "jiggling" — inserting yields/sleeps to change interleavings — the order in which different threads' steps happen to mix; tools like jcstress exist for this).

Key vocabulary: **bound resources** (fixed-size resources like connection pools — a reusable set of open database connections), **mutual exclusion** (only one thread at a time may use a resource), **starvation**, **deadlock**, **livelock** (threads keep reacting to each other and make no progress). These are covered in Chapters 12–13. (*Clean Code*'s **Appendix A, "Concurrency II"**, goes deeper into the same ideas: client- vs server-based locking, deadlock's four conditions and how to break them, and testing multithreaded code.)

Java example — before/after

```
// File: lld/ch06/concurrency/Main.java
package lld.ch06.concurrency;

import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;

/** Clean concurrency: separate concurrency from logic, limit shared data, prefer copies and independence, use the
library. */
public class Main {

    static final List<String> PAGES = List.of(
            "the cat sat on the mat", "the dog sat", "a cat and a dog", "the end");

    /** Pure, single-threaded logic -- testable without any threads (SRP: concurrency kept separate). */
    static Map<String, Integer> countWords(String page) {
        Map<String, Integer> counts = new HashMap<>();
        for (String w : page.split(" ")) counts.merge(w, 1, Integer::sum);
        return counts;
    }

    // ---------------- BEFORE: threads share an unsynchronised counter ----------------
    static class Before {
        static int theCount = 0;                                        // shared mutable state, no synchronisation
        static int run(int repeats) throws InterruptedException {
            Thread[] threads = new Thread[4];
            for (int t = 0; t < threads.length; t++) {
                threads[t] = new Thread(() -> {
                    for (int r = 0; r < repeats; r++)
                        for (String page : PAGES)
                            for (String w : page.split(" "))
                                if (w.equals("the")) theCount++;      // read-modify-write race: updates get lost
                });
                threads[t].start();
            }
            for (Thread t : threads) t.join();
            return theCount;
        }
        // (Sharing a plain HashMap this way is even worse: concurrent resizes can corrupt it.)
    }

    // ---------------- AFTER ----------------
    static class After {
        /** Each task works on its OWN data and returns a result; results are combined once, at the end. */
        static Map<String, Integer> run(int repeats, int threads) throws Exception {
            ExecutorService pool = Executors.newFixedThreadPool(threads);   // know your library
            try {
                List<Callable<Map<String, Integer>>> tasks = new java.util.ArrayList<>();
                for (int t = 0; t < threads; t++) {
                    tasks.add(() -> {
                        Map<String, Integer> local = new HashMap<>();       // thread-confined: no sharing, no
locks
                        for (int r = 0; r < repeats; r++)
                            for (String page : PAGES) countWords(page).forEach((w, c) -> local.merge(w, c,
Integer::sum));
                        return local;
                    });
                }
                Map<String, Integer> total = new HashMap<>();               // combined by ONE thread: no sharing
                for (Future<Map<String, Integer>> f : pool.invokeAll(tasks))
                    f.get().forEach((w, c) -> total.merge(w, c, Integer::sum));   // Future.get() safely hands over
```

```
each result
                return Map.copyOf(total);                                   // publish an immutable copy
            } finally {
                pool.shutdown();                                            // clean shutdown is part of
the design
                if (!pool.awaitTermination(5, TimeUnit.SECONDS)) pool.shutdownNow();
            }
        }
    }

    public static void main(String[] args) throws Exception {
        int repeats = 20_000;
        int expectedThe = 4 * repeats * 4;   // "the" appears 4 times per pass over PAGES, 4 threads
        System.out.println("pure logic: " + new java.util.TreeMap<>(countWords("the cat sat on the mat")));

        System.out.println("== BEFORE (racy; the number varies from run to run) ==");
        int racy = Before.run(repeats);
        System.out.println("count(the) = " + racy + " expected " + expectedThe + " -> lost " + (expectedThe - racy)
+ " updates");

        System.out.println("== AFTER (deterministic) ==");
        for (int threads : new int[]{1, 4, 16}) {                   // more threads than cores is a good stress
test
            Map<String, Integer> safe = After.run(repeats, threads);
            System.out.println(threads + " thread(s): count(the) = " + safe.get("the") + " expected " + (4 *
repeats * threads));
        }
    }
}
```

Output:

```
pure logic: {cat=1, mat=1, on=1, sat=1, the=2}
== BEFORE (racy; the number varies from run to run) ==
count(the) = 182526 expected 320000 -> lost 137474 updates
== AFTER (deterministic) ==
1 thread(s): count(the) = 80000 expected 80000
4 thread(s): count(the) = 320000 expected 320000
16 thread(s): count(the) = 1280000 expected 1280000
```

The *before* shares one `int` among four threads; `theCount++` is a read-modify-write, so concurrent increments overwrite each other and updates are **lost** (the exact number varies from run to run — and on rare runs it may even be correct, which is exactly why such bugs escape testing). The *after* follows the principles: pure logic separate from threading; each task owns its data; results are handed back through `Future.get()` (a `Future` is a handle to a result that a task will produce later) (which guarantees the result is safely visible to the caller) and merged by one thread; the final map is an immutable copy; the pool is shut down properly; and it is tested with 1, 4 and 16 threads (more threads than cores).

Trade-offs

Confinement + merge (after)	Shared state + locks
No locks, no races, scales well	Simple mental model for small cases
Extra memory for per-thread copies	Contention, deadlock risk
Needs a merge step	Every access must be correctly synchronised

When not to use it

Don't add concurrency without a measured need; a single thread is simpler and often fast enough. Prefer higher-level tools (parallel streams for pure CPU work, virtual threads in Java 21 — lightweight threads managed by the JVM, cheap enough to create one per task — for many blocking I/O tasks, i.e. input/output where the thread just waits, such as a network call) over manual thread management.

Common mistakes

- Sharing a `HashMap`/`ArrayList` between threads without synchronisation.
- Check-then-act races (`if (!map.containsKey(k)) map.put(k, v)`) — use `putIfAbsent`/`computeIfAbsent`.
- Forgetting to shut down executors (the JVM may not exit).
- Holding locks while calling unknown code (callbacks — functions passed in to be called later) → deadlock risk.

Real-world examples

- **`java.util.concurrent`** (Doug Lea): `ExecutorService`, `ConcurrentHashMap`, `CompletableFuture`, `ForkJoinPool` (work-stealing — idle threads take queued tasks from busy ones; used by parallel streams).
- **Map-reduce / Hadoop / Spark** (frameworks that split big-data jobs across many machines): independent workers on partitions + a merge step — the same confinement idea at cluster scale.
- **Node.js and Redis** (a JavaScript server runtime and an in-memory data store) use single-threaded event loops (one thread that handles ready events one after another) for their main processing, avoiding shared-memory races by design.

In the interview

> **30-second answer:** "I keep concurrency separate from business logic, minimise shared mutable state — preferring thread confinement (each piece of data is used by only one thread), immutable data and merging results — and use `java.util.concurrent` rather than raw threads. I plan shutdown and test with more threads than cores."

Follow-ups:

- *Q: Why is `count++` not thread-safe?* A: It's read, add, write — two threads can read the same value and one update is lost. Use `AtomicInteger`/`LongAdder` or a lock.
- *Q: Name three concurrency execution models.* A: Producer–consumer, readers–writers, dining philosophers.

Self-check

1. Q: What does concurrency decouple? → A: What gets done from when it gets done.
2. Q: How do you avoid sharing data between threads? → A: Give each task its own data (confinement) and merge results, or share only immutable data.
3. Q: How should you treat a one-off test failure in concurrent code? → A: As a likely real threading bug.

TL;DR

Separate concurrency code, share as little as possible (copies, immutability, confinement), use the library, keep locks small, plan shutdown, test hard.

---

13. Successive Refinement and the Case Studies (Chapters 14–16: how clean code is actually written)

Chapters 14–16 of *Clean Code* are long case studies rather than new rules: the author takes real code and cleans it step by step. They matter because they show the *process*.

Chapter 14 — Successive Refinement refines a command-line argument parser (`Args`) — code that reads the options typed after a program's name, like `-l -p 8080`. The lessons, in brief:

1. **Write it dirty first, then clean it.** Nobody writes clean code in one pass; a working first draft is normal. What's not acceptable is *stopping* there.
2. **Stop when it starts to rot.** When adding each new feature (a new argument type) makes the code noticeably messier, stop adding features and restructure.
3. **Refactor in tiny steps with tests** — after every small change, run the tests; never leave the system broken for long. Large "big-bang" restructurings tend to fail.
4. **Use design to absorb variation**: in the case study, each argument type became its own class behind a common interface (a polymorphic "marshaler"), so new types didn't touch the parser — the same Open/Closed move as our report formatters.

5. **Clean the error handling too** — collect error codes and messages into one exception class instead of scattering them.

Chapter 15 — JUnit Internals critiques a small class from JUnit (`ComparisonCompactor`, by Kent Beck and Erich Gamma), which was already good and well tested, and still improves it. Lessons (codes such as N6 and G28 refer to the smells-and-heuristics catalogue in §14):

- Remove encoding prefixes on fields (`fExpected` → `expected`) — N6.
- Encapsulate conditionals and prefer positive ones (`if (shouldNotCompact())` → `if (canBeCompacted())`) — G28, G29.
- Rename a function whose name hides a side effect or a second job (a `compact` method that also formats → `formatCompactedComparison`) — N7, G30.
- Make hidden temporal coupling explicit (finding the common suffix depends on first finding the common prefix, so make the order visible) — G31.
- Tidy `+1`/`-1` boundary arithmetic into well-named variables — G33.
- The point: even good code can be left a little cleaner — the Boy Scout Rule in action.

Chapter 16 — Refactoring SerialDate cleans up a date class from the open-source (published for anyone to use and modify) JCommon library. The process is the lesson:

1. **First, make it work** — write more tests: coverage (the share of code lines the tests actually execute) rose from about 50% to about 92%, and the new tests exposed real bugs, which were fixed.
2. **Then make it right** — rename (`SerialDate` → `DayDate`), replace integer constants with enums (a `Month` enum; J3), move methods to the classes that own the data (G14/G17), introduce an **Abstract Factory** so the base class no longer knows its concrete subclass (G7), and delete dead code, duplication and noise comments (G9, G5, C3).
3. Every step keeps the tests green.

TL;DR: make it work, then make it right, in small tested steps — and don't let a mess grow.

---

14. Smells and Heuristics (the checklist)

What it is

Clean Code ends with a catalogue of **smells and heuristics** — short rules, each with a code (C1, G5, N1, …), grouped by area. It extends the smells from Fowler's *Refactoring* (Chapter 7). Use it as a review checklist: each item is a *hint* that something could be cleaner, not an absolute law.

The problem it solves

Knowing principles is not enough; you need to *recognise* problems quickly while reading code. A named catalogue gives you and your reviewers a shared vocabulary ("this is G28 — encapsulate that conditional").

How it works — the catalogue, grouped (names listed; meaning in our words)

Comments (C)

Code	Heuristic	Meaning
C1	Inappropriate information	Change history, authors, ticket metadata belong in version control / issue tracker
C2	Obsolete comment	Old, irrelevant or wrong comments — update or delete
C3	Redundant comment	Says what the code already says clearly
C4	Poorly written comment	If worth writing, write it well: brief, correct grammar, precise
C5	Commented-out code	Delete it; version control remembers

Environment (E)

Code	Heuristic	Meaning
E1	Build requires more than one step	Check out, run one command, get a build (compiled, packaged program)
E2	Tests require more than one step	One command (or one click) runs all tests

Functions (F)

Code	Heuristic	Meaning
F1	Too many arguments	Zero best, then one, two, three; more needs strong justification
F2	Output arguments	Surprising; change the owning object's state instead
F3	Flag arguments	A boolean argument means the function does two things
F4	Dead function	Never called? Delete it

General (G) — the largest group, clustered by theme:

Correctness and safety

Code	Heuristic	Meaning
G2	Obvious behaviour is unimplemented	Honour the principle of least surprise; implement what users would expect from the name
G3	Incorrect behaviour at the boundaries	Test edge and corner cases (inputs at the limits: empty, zero, maximum); don't rely on intuition
G4	Overridden safeties	Don't turn off compiler warnings, skip failing tests, or ignore errors
G21	Understand the algorithm	Don't fiddle with ifs until tests pass; know *why* it works
G26	Be precise	e.g. don't use floating point (`double`, inexact for decimals) for currency; handle concurrency, nulls and ties deliberately
G31	Hidden temporal couplings	If calls must happen in order, make the order explicit (each step returns what the next needs)
G33	Encapsulate boundary conditions	Put `+1`/`-1` edge logic in one named place (`int lastIndex = size - 1;`)

Duplication and dead weight

Code	Heuristic	Meaning
G5	Duplication	"One of the most important rules" in the book: every duplication is a missed opportunity for abstraction (methods, polymorphism, Template Method, Strategy)
G9	Dead code	Unreachable/unused code — delete
G12	Clutter	Unused variables, empty default constructors, meaningless comments

Structure and placement

Code	Heuristic	Meaning
G1	Multiple languages in one source file	Minimise mixing (e.g. Java + SQL + HTML + JS in one file)
G6	Code at wrong level of abstraction	Keep high-level concepts in base classes/interfaces and details in implementations
G7	Base classes depending on their derivatives	A base class shouldn't know its subclasses
G8	Too much information	Small, focused interfaces; hide data, utilities, constants
G10	Vertical separation	Define variables and private functions close to where they're used
G13	Artificial coupling	Don't put things together that don't depend on each other (e.g. a general enum inside a specific class)
G14	Feature envy	A method more interested in another class's data than its own should move there
G17	Misplaced responsibility	Put code where a reader would expect to find it
G18	Inappropriate static	Make a method static only if it will never need polymorphic behaviour
G22	Make logical dependencies physical	If a module depends on another's assumption, ask it explicitly instead of assuming
G35	Keep configurable data at high levels	Defaults/constants that are configuration belong near the top (e.g. in `main`/config), passed down
G36	Avoid transitive navigation	Law of Demeter — don't write `a.getB().getC()`

Readability and intent

Code	Heuristic	Meaning
G11	Inconsistency	Do similar things the same way throughout
G15	Selector arguments	Arguments (booleans, enums, ints) that select behaviour → split into functions
G16	Obscured intent	Dense, clever expressions and magic hex numbers hide meaning
G19	Use explanatory variables	Break calculations into named intermediate values
G20	Function names should say what they do	`invoice.adjust(5)` — five what? up or down? Name it `applyDiscountPercent(5)`
G23	Prefer polymorphism to if/else or switch/case	"One switch" rule: at most one switch per type selection, used to create polymorphic objects
G24	Follow standard conventions	Team/industry coding standard, applied consistently
G25	Replace magic numbers with named constants	Also applies to magic strings and tokens
G27	Structure over convention	Enforce design decisions with structure (e.g. abstract methods) rather than naming conventions
G28	Encapsulate conditionals	`if (isRefundable(order))` beats `if (order.isDelivered() && order.daysSinceDelivery() <= 7)`
G29	Avoid negative conditionals	`if (user.isActive())` beats `if (!user.isNotActive())`
G30	Functions should do one thing	Split multi-step functions
G32	Don't be arbitrary	Structure code for a reason and let the reason show
G34	Functions should descend only one level of abstraction	Statements in a function sit one level below the function's name

Java (J)

Code	Heuristic	Meaning
J1	Avoid long import lists by using wildcards (`import java.util.*;`)	The book's advice; note many modern style guides (e.g. Google Java Style) forbid wildcard imports — follow your team's standard
J2	Don't inherit constants	Don't implement an interface just to get its constants; use static imports
J3	Constants versus enums	Prefer `enum` (type-safe, can carry data and behaviour) over `public static final int` codes

Names (N)

Code	Heuristic	Meaning
N1	Choose descriptive names	Names make up most of readability
N2	Choose names at the appropriate level of abstraction	Name for the concept, not the implementation (`notify(recipient)`, not `sendSms(phone)` if it may later be email or push)
N3	Use standard nomenclature where possible	Pattern names, domain terms (`...Decorator`, `...Factory`)
N4	Unambiguous names	Name should make the function's work clear
N5	Use long names for long scopes	Short names only in tiny scopes
N6	Avoid encodings	No type/scope prefixes
N7	Names should describe side effects	`getOrCreateUser`, not `getUser`, if it creates

Tests (T)

Code	Heuristic	Meaning
T1	Insufficient tests	Test everything that could break
T2	Use a coverage tool	Find untested lines and branches (e.g. JaCoCo)
T3	Don't skip trivial tests	They're cheap and document behaviour
T4	An ignored test is a question about an ambiguity	`@Disabled` tests mark unclear requirements — resolve them
T5	Test boundary conditions	Off-by-one errors (one too many or too few) live at the edges
T6	Exhaustively test near bugs	Bugs cluster; where you find one, look for more
T7	Patterns of failure are revealing	Which tests fail together often points to the cause
T8	Test coverage patterns can be revealing	Code not executed by passing tests hints at why failures happen
T9	Tests should be fast	Slow tests don't get run

Java example — several heuristics on one function

```java
// File: lld/ch06/smells/Main.java
package lld.ch06.smells;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;

/**
 * A handful of Clean Code heuristics applied to one function:
 * G25 magic numbers, G19 explanatory variables, G16 obscured intent, G28 encapsulate conditionals,
 * G29 avoid negative conditionals, F3 flag arguments, G26 be precise (no floating point for money),
 * N1/G20 descriptive names that say what the function does.
 */
public class Main {

    enum Membership { NONE, PRIME }
    record Parcel(double weightKg, int distanceKm, Membership membership, boolean fragile) {}

    // ---------------- BEFORE ----------------
    static class Before {
        static double calc(Parcel p, boolean express) {                                  // double for money (G26)
            double c;
            if (!(p.membership() != Membership.PRIME)) c = 0; else c = 40;           // double negative
            if (p.weightKg() > 20 || p.distanceKm() > 1500 && !p.fragile() == false) c += 250;   // what does this
mean?
            c += p.weightKg() * 12.5;
            if (express) c = c * 1.5;                                                    // flag argument
            if (express && p.membership() == Membership.PRIME) c = c * 0.9;
            return Math.round(c * 100) / 100.0;
        }
    }

    // ---------------- AFTER ----------------
    static class After {
        enum Speed {                                            // replaces the flag; each speed knows its
multiplier
            STANDARD("1.0"), EXPRESS("1.5");
            final BigDecimal multiplier;
            Speed(String m) { multiplier = new BigDecimal(m); }
        }

        private static final BigDecimal BASE_FEE = new BigDecimal("40");          // rupees, exact decimal (G26)
        private static final BigDecimal RATE_PER_KG = new BigDecimal("12.5");
        private static final double HEAVY_KG = 20;
        private static final int LONG_HAUL_KM = 1500;
        private static final BigDecimal SPECIAL_HANDLING_FEE = new BigDecimal("250");
        private static final BigDecimal PRIME_EXPRESS_DISCOUNT = new BigDecimal("0.9");

        static BigDecimal shippingCost(Parcel p, Speed speed) {
            BigDecimal baseFee = isPrime(p) ? BigDecimal.ZERO : BASE_FEE;
            BigDecimal handlingFee = needsSpecialHandling(p) ? SPECIAL_HANDLING_FEE : BigDecimal.ZERO;
            BigDecimal weightCharge = BigDecimal.valueOf(p.weightKg()).multiply(RATE_PER_KG);   // explanatory
variables
            BigDecimal cost = baseFee.add(handlingFee).add(weightCharge).multiply(speed.multiplier);
            if (isPrime(p) && speed == Speed.EXPRESS) cost = cost.multiply(PRIME_EXPRESS_DISCOUNT);
            return cost.setScale(2, RoundingMode.HALF_UP);                           // round once, explicitly, to
paise
        }

        private static boolean isPrime(Parcel p) { return p.membership() == Membership.PRIME; }   // positive
conditional
        private static boolean needsSpecialHandling(Parcel p) {                                  // encapsulated
conditional
            boolean heavy = p.weightKg() > HEAVY_KG;
            boolean fragileLongHaul = p.distanceKm() > LONG_HAUL_KM && p.fragile();
            return heavy || fragileLongHaul;
```

```
        }
    }

    public static void main(String[] args) {
        List<Parcel> parcels = List.of(
                new Parcel(2, 300, Membership.NONE, false),
                new Parcel(25, 100, Membership.PRIME, false),
                new Parcel(3, 2000, Membership.NONE, true),
                new Parcel(3, 2000, Membership.PRIME, false));
        for (Parcel p : parcels) {
            for (boolean express : new boolean[]{false, true}) {
                double before = Before.calc(p, express);
                BigDecimal after = After.shippingCost(p, express ? After.Speed.EXPRESS : After.Speed.STANDARD);
                boolean same = BigDecimal.valueOf(before).compareTo(after) == 0;
                System.out.printf("%-55s express=%-5s before=%7.2f after=%7s %s%n",
                        p, express, before, after, same ? "same" : "DIFFERENT");
            }
        }
    }
}
```

Output:

```
Parcel[weightKg=2.0, distanceKm=300, membership=NONE, fragile=false] express=false before=  65.00 after=  65.00
same
Parcel[weightKg=2.0, distanceKm=300, membership=NONE, fragile=false] express=true  before=  97.50 after=  97.50
same
Parcel[weightKg=25.0, distanceKm=100, membership=PRIME, fragile=false] express=false before= 562.50 after= 562.50
same
Parcel[weightKg=25.0, distanceKm=100, membership=PRIME, fragile=false] express=true  before= 759.38 after= 759.38
same
Parcel[weightKg=3.0, distanceKm=2000, membership=NONE, fragile=true] express=false before= 327.50 after= 327.50
same
Parcel[weightKg=3.0, distanceKm=2000, membership=NONE, fragile=true] express=true  before= 491.25 after= 491.25
same
Parcel[weightKg=3.0, distanceKm=2000, membership=PRIME, fragile=false] express=false before=  37.50 after=  37.50
same
Parcel[weightKg=3.0, distanceKm=2000, membership=PRIME, fragile=false] express=true  before=  50.63 after=  50.63
same
```

The "same" column proves behaviour didn't change. Fixed smells: double negative (G29), unreadable mixed `&&` / `||` / `== false` condition (G16, G28 → `needsSpecialHandling`), magic numbers (G25), flag argument (F3 → `Speed` enum), unnamed intermediate values (G19), a cryptic name `calc` (N1/G20 → `shippingCost`), and floating-point money (G26 → `BigDecimal` with one explicit `HALF_UP` rounding to paise). The *before* deliberately keeps `double` so you can see the smell; never do that in real money code.

Trade-offs

A checklist can be applied dogmatically. Each item is a heuristic: when two conflict, favour readability and correctness.

When not to use it

Don't block a critical fix on style nits; log them as follow-ups.

Common mistakes

- Treating smell counts as a metric to game.
- Ignoring the **Java** notes that are dated (J1): follow your team's style guide.

Real-world examples

Static analysers (tools that inspect code without running it) encode many of these: **SonarQube** (cognitive complexity — a score for how hard code is to follow, duplicated blocks, commented-out code), **PMD** (unused code, god class), **Checkstyle** (naming, magic numbers), **SpotBugs** (bug patterns), **Error Prone** (Google).

In the interview

> **30-second answer:** "When reviewing I look for duplication, long functions, flag and output arguments, magic numbers, negative or complex conditionals, feature envy, train wrecks, misplaced responsibilities, dead code, and weak tests. Most fixes are small refactorings: extract method, introduce constant, replace conditional with polymorphism."

Self-check

1. Q: Which heuristic does the book call one of its most important rules? → A: G5, Duplication.
2. Q: What does G28 ask you to do? → A: Encapsulate conditionals in a well-named function.
3. Q: What does T4 say about a disabled test? → A: It signals an unresolved ambiguity in requirements.

TL;DR

Use the C/E/F/G/J/N/T catalogue as a shared review vocabulary; fix with small, tested refactorings.

---

15. Code Review Checklist (best practices)

What it is

A **code review** is a peer reading a change before it's merged (combined into the team's main branch), to catch defects, share knowledge and keep the codebase consistent. A checklist makes reviews thorough and fair — and in interviews, it's what you mentally run over your own code before saying "done".

Analogy: a pilot's pre-flight checklist — experienced pilots still use it because it catches what memory misses.

The problem it solves

Unreviewed changes introduce bugs, security holes and inconsistent styles; unfocused reviews waste time on nits while missing real issues.

How it works — the checklist (from most to least important)

1. **Correctness**
   - Does it do what the requirement/ticket says? Are edge cases handled (empty, null, zero, max, duplicates, time zones, rounding)?
   - Are errors handled — no swallowed exceptions, meaningful messages, resources closed?
   - Concurrency: shared mutable state guarded? check-then-act races? thread-safe collections?
2. **Design**
   - Does it fit the existing architecture? Right class/layer for each responsibility (SRP, GRASP)?
   - Dependencies on abstractions (DIP)? Easy to extend without modification (OCP)?
   - Any over-engineering (YAGNI) or duplication (DRY)?
3. **Readability**
   - Intention-revealing names; small functions; no magic numbers; no flag arguments.
   - Comments explain *why*, not *what*; no commented-out code.
4. **Tests**
   - New behaviour covered by tests? Bug fixes include a regression test (one that makes sure the fixed bug can't return)?
   - Tests are F.I.R.S.T., one concept each, readable names?
5. **Security**
   - Input validation; no SQL/command injection (an attack where user input gets run as part of a database query or shell command; prevented by parameterised queries, which send input separately as plain data); output encoding (escaping data before putting it into HTML so it can't run as script).
   - No secrets or credentials in code or logs; sensitive data (PII — personally identifiable information such as names, phone numbers, emails) masked in logs.
   - Authorisation checks (is *this* user allowed to do *this* action?) on every sensitive operation.

6. **Performance and resources**
    - Obvious inefficiencies: N+1 queries (one query to load a list, then one more query per item), repeated work in loops, unbounded collections/caches, missing pagination (returning results page by page instead of all at once).
    - Timeouts (a maximum wait, after which the call gives up) on remote calls; connection/stream closing.
7. **Operability**
    - Useful logs at the right level (and not too many); metrics (counters and timings collected for dashboards and alerts) for important events; feature flags (switches that turn a feature on or off without redeploying) for risky changes.
    - Backwards compatibility (old clients keep working after the change) of APIs, messages and database migrations (scripted changes to the database's table structure).
8. **Style and consistency** — ideally automated by formatter/linter so humans focus on the above.

How to review well (and be reviewed well):

- Keep changes **small** (a few hundred lines at most); large diffs get rubber-stamped (approved without real reading).
- Author: write a clear description (what, why, how tested), self-review first.
- Reviewer: be specific and kind; explain *why*; separate must-fix from optional suggestions ("nit:"); ask questions rather than accuse.
- Automate the mechanical (formatting, static analysis, tests in CI) so humans review design and logic.

Java example

No separate program: every before/after in this chapter is a review finding. For example, reviewing `Before.book` in §2 you would flag: flag argument (F3), error codes, output argument (F2), hidden side effect (SESSION), switch on type code (G23), magic numbers (G25) — exactly the fixes shown in its *after* version.

Trade-offs

Reviews slow merging slightly but catch defects early, spread knowledge and keep style consistent. Too many reviewers or nit-heavy reviews slow teams without adding quality.

When not to use it

Emergency hotfixes (urgent fixes deployed straight to production) may be reviewed after merge (with a follow-up), but should still be reviewed.

Common mistakes

- Reviewing only style while missing logic and security bugs.
- Huge pull requests (a pull request is a proposed change submitted for review and merging).
- Personal preferences presented as rules.

Real-world examples

- **GitHub pull requests / GitLab merge requests / Gerrit** (used by Android and Chromium) implement review workflows.
- Google's engineering practices publish guidance for reviewers and authors (focus on design, functionality, complexity, tests, naming, comments, style).
- **CODEOWNERS** files route reviews to responsible teams.

In the interview

> **30-second answer:** "I review for correctness first — requirements, edge cases, errors, concurrency — then design, readability, tests, security and performance, and leave formatting to tools. I keep feedback specific, explain why, and mark nits as optional."

Self-check

1. Q: What should a reviewer check first? → A: Correctness (including edge cases and error handling).
2. Q: Why keep pull requests small? → A: Large diffs get shallow reviews.

TL;DR

Correctness → design → readability → tests → security → performance → operability; automate style; keep PRs small and feedback kind.

---

16. Error-Handling Strategy (best practices for real services)

What it is

Section 6 covered error-handling *rules* inside code. An **error-handling strategy** is the *system-wide policy*: which exception types exist, where errors are translated, which are retried, how they're logged, and what callers (users/other services) see.

Analogy: a hospital triage system — every problem is classified (critical, urgent, minor), routed to the right place, and recorded once in the patient chart.

The problem it solves

Without a strategy, each developer invents their own: some return `null`, some throw checked exceptions, some log and rethrow at every layer (same error logged five times), some expose stack traces to users, and some retry everything — including permanent failures, which overloads already-struggling services.

How it works

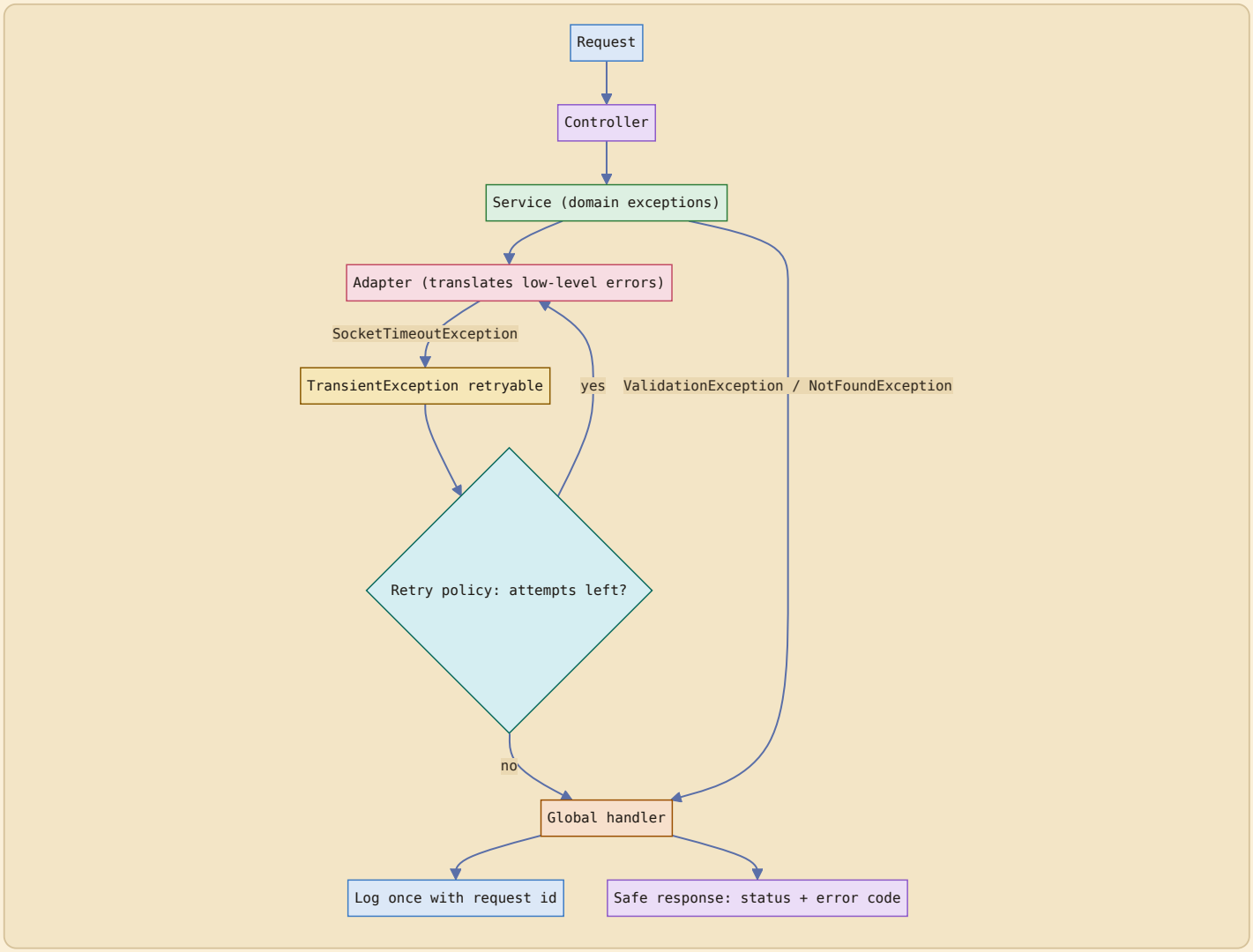
> **Key term — HTTP status codes:** every reply from a web API carries a three-digit status code. **2xx** means success (200 OK); **4xx** means the *client* sent something wrong (400 bad request, 404 not found, 409 conflict with the current state); **5xx** means the *server* failed (500 internal error, 503 service unavailable — often temporary). Retrying usually only makes sense for some 5xx errors and timeouts. See System Design Fundamentals.

1. **Classify errors:**

Class	Examples	Handling
Validation / client errors	bad input, missing field	Reject immediately, 4xx, clear message, no retry
Business rule violations	insufficient funds, seat already booked	Domain exception or result value, 4xx (e.g. 409 conflict)
Not found	unknown id	404 / empty `Optional`
Transient infrastructure	timeouts, 503, connection reset, deadlock victim	Retry with backoff — waiting longer between attempts (only if the operation is **idempotent** — safe to repeat)
Permanent infrastructure	auth failure to DB, misconfiguration	Fail fast, alert
Programming bugs	NPE, index out of bounds	Don't catch locally; generic 500 at the edge, log with details, fix the bug

2. **One exception hierarchy** with an error code and a *retryable* flag, so handlers don't need `instanceof` chains on vendor exceptions.
3. **Translate at boundaries**: adapters convert low-level exceptions (`SocketTimeoutException`, `SQLException`) into domain/infrastructure exceptions, keeping the cause.
4. **Retry only transient errors**, with a maximum number of attempts, **exponential backoff** (each wait doubles: 10 ms, 20 ms, 40 ms…) and **jitter** (randomness to avoid many clients retrying in sync). Combine with **timeouts** and **circuit breakers** (a guard that stops calling a failing dependency for a while after repeated errors) for dependencies.
5. **Use result values for expected, per-item failures** (e.g. validating 10,000 imported rows — collect all errors instead of stopping at the first).

6. **Handle globally at the edge**: one handler maps exceptions to responses (status code, stable error code, safe message) and **logs once** with a correlation/request id (a unique ID attached to one request and stamped on all its log lines, so they can be found together). Never leak stack traces or internals to clients.
7. **Keep the system consistent**: use transactions or compensating actions (follow-up steps that undo an earlier one, e.g. a refund) so partial failures don't leave corrupt state; make handlers idempotent.



Java example

```java
// File: lld/ch06/errorstrategy/Main.java
package lld.ch06.errorstrategy;

import java.net.SocketTimeoutException;
import java.util.ArrayList;
import java.util.List;
import java.util.function.Function;
import java.util.function.Supplier;

/**
 * An end-to-end error-handling STRATEGY for a service:
 *  1. one exception hierarchy with error codes (+ "retryable" flag)
 *  2. translate low-level errors at the infrastructure boundary
 *  3. retry only transient errors, with backoff and a limit
 *  4. use a Result type for EXPECTED failures (e.g. validating many rows)
 *  5. one global handler maps exceptions to responses and logs ONCE, hiding internals
 */
@SuppressWarnings("serial") // the nested exception classes are never serialized
public class Main {

    // ---------- 1. exception hierarchy ----------
    enum ErrorCode {
        VALIDATION_FAILED(400), NOT_FOUND(404), CONFLICT(409), DEPENDENCY_UNAVAILABLE(503);
        final int httpStatus;
        ErrorCode(int s) { httpStatus = s; }
    }
    static class AppException extends RuntimeException {
        final ErrorCode code;
        final boolean retryable;
        AppException(ErrorCode code, String message, boolean retryable, Throwable cause) {
            super(message, cause); this.code = code; this.retryable = retryable;
        }
    }
    static final class ValidationException extends AppException {
        ValidationException(String m) { super(ErrorCode.VALIDATION_FAILED, m, false, null); }
    }
    static final class NotFoundException extends AppException {
        NotFoundException(String m) { super(ErrorCode.NOT_FOUND, m, false, null); }
    }
    static final class TransientException extends AppException {
        TransientException(String m, Throwable cause) { super(ErrorCode.DEPENDENCY_UNAVAILABLE, m, true, cause); }
    }

    // ---------- 2. translate at the infrastructure boundary ----------
    static final class InventoryClient {
        private int callsUntilHealthy;
        InventoryClient(int failFirst) { this.callsUntilHealthy = failFirst; }
        int stockOf(String sku) {
            try {
                return rawHttpCall(sku);
            } catch (SocketTimeoutException e) {
                throw new TransientException("inventory service timed out", e);   // low-level -> domain meaning
            }
        }
        private int rawHttpCall(String sku) throws SocketTimeoutException {
            if (callsUntilHealthy-- > 0) throw new SocketTimeoutException("read timed out after 2000ms");
            if (sku.startsWith("X")) throw new NotFoundException("sku " + sku + " not found");
            return 7;
        }
    }

    // ---------- 3. retry transient errors only ----------
    static <T> T withRetry(Supplier<T> action, int maxAttempts, long initialBackoffMs) {
        long backoff = initialBackoffMs;
        for (int attempt = 1; ; attempt++) {
            try {
```

```
                return action.get();
            } catch (AppException e) {
                if (!e.retryable || attempt == maxAttempts) throw e;
                System.out.println("    attempt " + attempt + " failed (" + e.getMessage() + "), retrying in " +
backoff + "ms");
                sleep(backoff);
                backoff *= 2;                                   // exponential backoff (add jitter in production)
            }
        }
    }
    private static void sleep(long ms) {
        try { Thread.sleep(ms); } catch (InterruptedException e) { Thread.currentThread().interrupt(); throw new
IllegalStateException(e); }
    }

    // ---------- 4. Result type for expected, per-item failures ----------
    sealed interface Result<T> {
        record Ok<T>(T value) implements Result<T> {}
        record Err<T>(String error) implements Result<T> {}
    }
    record Row(int line, String sku, int qty) {}
    static Result<Row> parseRow(int line, String csv) {
        String[] f = csv.split(",");
        if (f.length != 2) return new Result.Err<>("line " + line + ": expected 2 fields");
        try {
            int qty = Integer.parseInt(f[1].trim());
            if (qty <= 0) return new Result.Err<>("line " + line + ": qty must be positive");
            return new Result.Ok<>(new Row(line, f[0].trim(), qty));
        } catch (NumberFormatException e) {
            return new Result.Err<>("line " + line + ": qty '" + f[1].trim() + "' is not a number");
        }
    }

    // ---------- 5. global handler at the edge ----------
    record Response(int status, String body) {}
    static <T> Response handle(String requestId, Supplier<T> endpoint, Function<T, String> render) {
        try {
            return new Response(200, render.apply(endpoint.get()));
        } catch (AppException e) {
            System.out.println("    [log once] " + requestId + " " + e.code + ": " + e.getMessage());
            return new Response(e.code.httpStatus, "{\"error\":\"" + e.code + "\",\"message\":\"" + e.getMessage()
+ "\"}");
        } catch (RuntimeException e) {
            System.out.println("    [log once] " + requestId + " UNEXPECTED " + e);   // full details only in logs
            return new Response(500, "{\"error\":\"INTERNAL\",\"requestId\":\"" + requestId + "\"}");
        }
    }

    public static void main(String[] args) {
        System.out.println("Retry a transient failure, then succeed:");
        var flaky = new InventoryClient(2);
        System.out.println("  " + handle("req-1", () -> withRetry(() -> flaky.stockOf("TEA"), 3, 10), n -> "
{\"stock\":" + n + "}"));

        System.out.println("Do NOT retry a permanent failure:");
        var healthy = new InventoryClient(0);
        System.out.println("  " + handle("req-2", () -> withRetry(() -> healthy.stockOf("X-404"), 3, 10), n -> "
{\"stock\":" + n + "}"));

        System.out.println("Give up after max attempts:");
        var down = new InventoryClient(99);
        System.out.println("  " + handle("req-3", () -> withRetry(() -> down.stockOf("TEA"), 3, 10), n -> "
{\"stock\":" + n + "}"));

        System.out.println("Unexpected bug -> generic 500, no internals leaked:");
        System.out.println("  " + handle("req-4", () -> List.<String>of().get(0), s -> s));

        System.out.println("Bulk import: collect ALL row errors instead of stopping at the first:");
        List<String> csv = List.of("TEA,2", "CUP", "PEN,abc", "INK,-1", "MUG,1");
```

```
        List<Row> ok = new ArrayList<>();
        List<String> errors = new ArrayList<>();
        for (int i = 0; i < csv.size(); i++) {
            switch (parseRow(i + 1, csv.get(i))) {
                case Result.Ok<Row> r -> ok.add(r.value());
                case Result.Err<Row> e -> errors.add(e.error());
            }
        }
        System.out.println("  imported " + ok);
        errors.forEach(e -> System.out.println("  rejected " + e));
        if (!errors.isEmpty())
            System.out.println("  " + handle("req-5", () -> { throw new ValidationException(errors.size() + "
invalid rows"); }, x -> "" + x));
    }
}
```

Output:

```
Retry a transient failure, then succeed:
    attempt 1 failed (inventory service timed out), retrying in 10ms
    attempt 2 failed (inventory service timed out), retrying in 20ms
  Response[status=200, body={"stock":7}]
Do NOT retry a permanent failure:
    [log once] req-2 NOT_FOUND: sku X-404 not found
  Response[status=404, body={"error":"NOT_FOUND","message":"sku X-404 not found"}]
Give up after max attempts:
    attempt 1 failed (inventory service timed out), retrying in 10ms
    attempt 2 failed (inventory service timed out), retrying in 20ms
    [log once] req-3 DEPENDENCY_UNAVAILABLE: inventory service timed out
  Response[status=503, body={"error":"DEPENDENCY_UNAVAILABLE","message":"inventory service timed out"}]
Unexpected bug -> generic 500, no internals leaked:
    [log once] req-4 UNEXPECTED java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0
  Response[status=500, body={"error":"INTERNAL","requestId":"req-4"}]
Bulk import: collect ALL row errors instead of stopping at the first:
  imported [Row[line=1, sku=TEA, qty=2], Row[line=5, sku=MUG, qty=1]]
  rejected line 2: expected 2 fields
  rejected line 3: qty 'abc' is not a number
  rejected line 4: qty must be positive
    [log once] req-5 VALIDATION_FAILED: 3 invalid rows
  Response[status=400, body={"error":"VALIDATION_FAILED","message":"3 invalid rows"}]
```

- **req-1**: a transient timeout is retried with exponential backoff (10 ms, 20 ms) and then succeeds.
- **req-2**: `NotFoundException` is *not* retryable, so it fails immediately with 404.
- **req-3**: the dependency stays down; after 3 attempts we give up with 503.
- **req-4**: a programming bug becomes a generic 500 with only a request id; details go to the log.
- **Bulk import**: a sealed `Result` type (`Ok`/`Err`) with pattern-matching `switch` collects all row errors; the caller then decides to reject with 400.

Trade-offs

Choice	Benefit	Cost
Retries	Survive blips	Duplicate side effects unless idempotent; can amplify load
Global handler	Consistent responses, log once	Must avoid swallowing context
Result types	Explicit expected failures	Verbose in Java; mixing with exceptions needs a clear rule

When not to use it

Don't retry non-idempotent operations (e.g. "charge card") unless you use an **idempotency key** (a unique ID the client sends with a request so the server can recognise and ignore repeats), and never retry validation or authorisation failures.

Common mistakes

- Log-and-rethrow at every layer.

- Catching `Exception` in business code.
- Retrying immediately in a tight loop without backoff.
- Returning 200 with an error body.

Real-world examples

- **Spring** `@ControllerAdvice` + `@ExceptionHandler` — a global exception-to-response mapper; **RFC 9457 "Problem Details"** (successor of RFC 7807) standardises JSON error bodies, supported by Spring's `ProblemDetail`.
- **Resilience4j** (successor to Netflix Hystrix, which is in maintenance mode): retry, circuit breaker, rate limiter (caps calls per second), bulkhead (isolates resources so one failing part can't exhaust them all).
- **AWS SDKs** (Amazon Web Services' client libraries) retry throttling ("slow down" responses)/transient errors with exponential backoff and jitter by default.
- **Stripe's API** supports idempotency keys so clients can safely retry payment requests.

In the interview

> **30-second answer:** "I classify errors into validation, business, not-found, transient and bugs; use one exception hierarchy with codes and a retryable flag; translate low-level errors in adapters; retry only transient, idempotent operations with backoff and jitter; and map everything to safe responses in one global handler that logs once with a request id."

Follow-ups:

- *Q: Why jitter?* A: Without it, many clients retry at the same moments and overload the recovering service (thundering herd).
- *Q: How do you make payment retries safe?* A: Idempotency keys — the server stores the result per key and returns it for repeats.

Self-check

1. Q: Which errors should be retried? → A: Transient ones, and only for idempotent operations.
2. Q: Where should errors be logged? → A: Once, at the boundary/global handler (with a correlation id).

TL;DR

Classify, translate at boundaries, retry only transient + idempotent with backoff, result values for expected failures, one global handler that logs once and hides internals.

---

17. Testing Strategies (unit vs integration, test doubles, mocks)

What it is

A **testing strategy** decides *what kinds* of tests to write, *how many* of each, and *how to isolate* code under test. It builds on §8 (clean unit tests).

Analogy: car manufacturing — each part is tested alone (unit), assembled systems are tested together (integration), and finally the whole car is test-driven (end-to-end).

The problem it solves

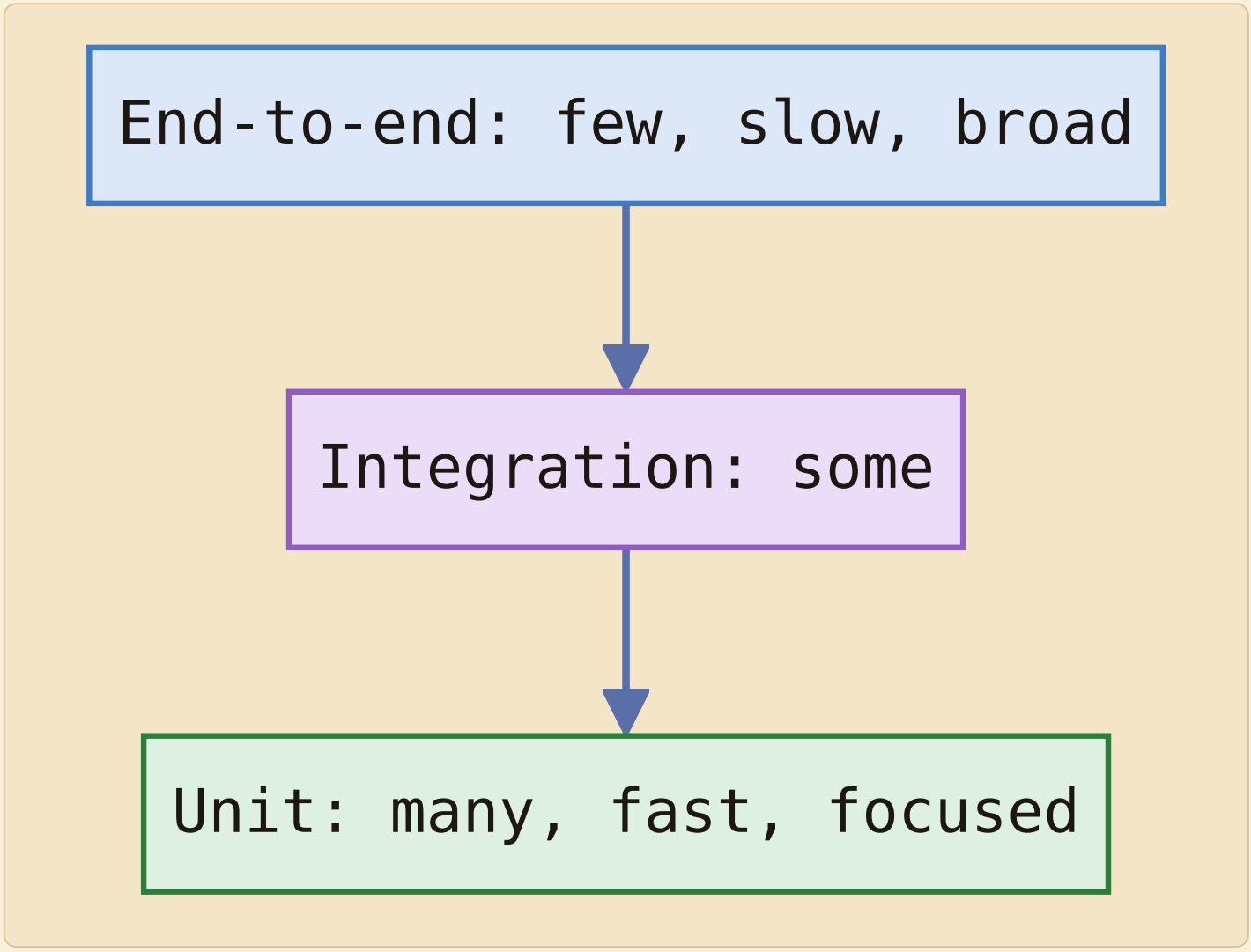
Too few tests → bugs reach production. The wrong mix (mostly slow end-to-end UI tests) → slow, flaky (sometimes passing, sometimes failing with no code change) pipelines (the automated build-and-test runs in CI) that people stop trusting. Over-mocking → tests that pass while the real system is broken, and break on every refactor.

How it works

Kinds of tests:

Kind	Scope	Speed	Catches
Unit	One class/function, collaborators replaced by doubles	Milliseconds	Logic errors
Integration	Several real components together (DB, message broker — a server that passes messages between services, your modules)	Seconds	Wiring, queries, serialisation (turning objects into bytes/text and back), config
Contract	Agreement between a consumer and a provider service	Fast	API mismatches between services
End-to-end (E2E)	The whole system through its UI/API	Slow	User-journey breakage
Others	Performance/load, security, exploratory, property-based, mutation	Varies	Non-functional issues, weak tests

The Test Pyramid (popularised by Mike Cohn): many fast unit tests at the base, fewer integration tests in the middle, very few E2E tests at the top. Avoid the "ice-cream cone" (mostly manual/E2E tests).



Test doubles (Gerard Meszaros' vocabulary, popularised by Martin Fowler):

Double	What it does	In our code
Dummy	Passed to fill a parameter; never used	`DummyAuditLog` (throws if called)
Stub	Returns canned answers to calls	`StubCatalog`
Spy	A stub that records calls for later assertions	`SpySender.sent`
Mock	Pre-programmed with expectations; verifies the interaction itself	`MockAuditLog.expect(...).verify()`
Fake	A working, simplified implementation	`FakeCoupons`, in-memory repositories

State vs behaviour verification: stubs/fakes support *state verification* (check the result or the resulting state); mocks support *behaviour verification* (check that the right calls happened). Prefer state verification; use mocks for outgoing side effects you can't observe otherwise (emails sent, audit events).

What to test: public behaviour, business rules, edge cases (empty, null, boundaries, duplicates), error paths, and every bug fix (regression tests). Don't test private methods directly, trivial accessors, or the framework itself.

Mocking rules of thumb: don't mock types you don't own (wrap them first and mock your wrapper — see Boundaries); don't mock value objects; one or two mocks per test at most; if a test needs many mocks, the class has too many dependencies.

Coverage: a coverage tool (e.g. JaCoCo) shows untested code; high coverage is *necessary but not sufficient* — mutation testing (e.g. PIT) checks whether tests actually detect changes.

Java example

The code is in §8 (`lld.ch06.testing`): `CheckoutTest` uses all five double types for fast unit tests; `CheckoutIntegrationTest` wires real in-memory implementations together; `BeforeStyleTest` shows what to avoid. The output there shows all eight tests passing.

Trade-offs

More unit tests	More integration/E2E tests
Fast, precise failure location	Higher confidence the pieces fit
May miss wiring/config bugs	Slow, flakier, harder to debug
Over-mocking couples tests to implementation	Need environments (DB, containers — isolated runtime packages such as Docker)

When not to use it

Don't insist on unit tests for thin glue code that only wires frameworks — an integration test covers it better. Don't write E2E tests for every edge case — push those down to unit tests.

Common mistakes

- Mocking everything, including value objects and the class under test.
- Flaky tests (time, randomness, order dependence, shared DB state) left unfixed — they destroy trust.
- Testing implementation details (exact internal calls) instead of outcomes.
- No tests for error paths.

Real-world examples

- **JUnit 5**, **Mockito** (mocks/spies/stubs), **AssertJ**, **Testcontainers** (real PostgreSQL/Kafka in Docker for integration tests), **WireMock** (fake HTTP services), **Pact** (consumer-driven contract tests), **PIT** (mutation testing), **JaCoCo** (coverage).
- Spring Boot test slices (`@WebMvcTest`, `@DataJpaTest`) for focused integration tests.

In the interview

> **30-second answer:** "I follow the test pyramid: many fast unit tests with test doubles, fewer integration tests with real components like the database via Testcontainers, and a handful of end-to-end tests. I prefer state verification with stubs and fakes, use mocks only for outgoing side effects, and never mock types I don't own."

Follow-ups:

- *Q: Mock vs stub?* A: A stub supplies canned answers so the test can check state; a mock is set up with expectations and verifies the calls themselves.
- *Q: Spy vs mock?* A: A spy records calls for the test to assert afterwards; a mock checks expectations itself.
- *Q: What's a fake?* A: A lightweight working implementation, e.g. an in-memory repository.

Self-check

1. Q: Name the five test doubles. → A: Dummy, stub, spy, mock, fake.
2. Q: Why a pyramid shape? → A: Lower-level tests are faster, cheaper and more precise; top-level tests are slow and brittle, so keep few.
3. Q: Is 100% coverage proof of good tests? → A: No — it shows code ran, not that assertions would catch bugs (mutation testing checks that).

TL;DR

Many unit tests, some integration, few E2E; pick the right double (dummy/stub/spy/mock/fake); verify state over interactions; don't mock what you don't own.

Chapter summary

Topic	Remember this
Names	Reveal intent, pronounceable, searchable, one word per concept
Functions	Small, one thing, one level, stepdown, ≤2–3 args, no flags, CQS, exceptions
Comments	Explain in code; comment only why/warnings/legal/public API
Formatting	Newspaper order, related things close, automated team style
Objects vs data	Objects hide data; data structures expose it; anti-symmetry decides
Error handling	Unchecked exceptions with context, wrap vendors, no null in or out
Boundaries	Wrap third-party code; learning tests; define interfaces you need
Unit tests	TDD laws, one concept per test, F.I.R.S.T.
Classes	Small by responsibility, cohesive, organised for change
Systems	Composition root + DI; cross-cutting via proxies/aspects
Emergence	Beck: tests pass → reveals intent → no duplication → fewest elements (*Clean Code* swaps the middle two)
Concurrency	Separate it, share little, use the library, test hard
Smells	C/E/F/G/J/N/T checklist
Code review	Correctness → design → readability → tests → security → performance
Error strategy	Classify, translate, retry transient, handle globally, log once
Testing strategy	Pyramid; right test double; state over behaviour verification

Chapter self-check

1. Q: A method `boolean saveAndValidate(Order o, boolean sendEmail)` breaks which rules? → A: Does two things, flag argument (F3), likely violates CQS (returns a boolean while changing state), name reveals multiple responsibilities.
2. Q: What should `findOrders(customerId)` return when there are none? → A: An empty list, never `null`.
3. Q: How do you test code that calls `LocalDate.now()`? → A: Inject a `Clock`; use `Clock.fixed` in tests.
4. Q: What does a dynamic proxy let you do cleanly? → A: Apply a cross-cutting concern (logging, transactions) to every method of an interface in one place.
5. Q: Which of Beck's rules has the highest priority? → A: Passes all the tests.

TL;DR (chapter)

Write code for the next reader: honest names, tiny single-purpose functions, few comments that explain why, consistent layout, clear exceptions instead of codes or nulls, foreign code behind your own interfaces, clean fast tests, small cohesive classes wired from one place, simple design that emerges through refactoring, careful concurrency — and review everything against a shared checklist.

Chapter 7 — Refactoring

> **What you will learn.** What refactoring is (and is not), when to do it, how tests make it safe, the full catalogue of **code smells** (the signals that tell you *where* to refactor), the full catalogue of **refactorings** from Martin Fowler's *Refactoring* (2nd edition, 2018), a smell → refactoring lookup table, a complete worked example in Java that goes from messy to clean in six compiling steps, and how refactoring leads you toward design patterns (named, reusable solutions to recurring design problems; Chapters 8–11).
>
> **Why it matters for LLD interviews.** Machine-coding rounds (interview rounds where you build a small working program in about 60–120 minutes; a core part of LLD — low-level design — interviews) often end with "now add feature X" or "what would you clean up?". Interviewers want to hear you name a smell ("this switch is repeated in three places — a Repeated Switches smell") and name the fix ("Replace Conditional with Polymorphism, which gives us a Strategy" — the Strategy pattern puts each interchangeable algorithm in its own class). This chapter gives you that vocabulary.

Chapter map

Part	Content
7.1	Refactoring fundamentals — definition, two hats, why, when, when not, tests, performance, rewrite vs refactor, IDE (Integrated Development Environment, e.g. IntelliJ IDEA) automation, legacy code
7.2	Code smells — the 24 smells of the 2nd edition, grouped the refactoring.guru way
7.3	The refactoring catalogue — all 61 named refactorings of the 2nd edition, by book chapter, plus the extra ones on refactoring.com
7.4	Smell → refactorings lookup table
7.5	Worked example — a messy invoice class refactored in six compiling steps
7.6	Refactoring toward patterns — Strategy, State, Special Case, Template Method, Factory
7.7	Trade-offs, mistakes, real-world examples, interview answers, self-check, TL;DR
7.8	Full source code and program output

7.1 Refactoring fundamentals

What it is

Refactoring (noun) is a change made to the internal structure of software to make it easier to understand and cheaper to modify, *without changing its observable behaviour*.

Refactoring (verb) is restructuring software by applying a *series* of such small, behaviour-preserving changes.

Two words in that definition do all the work:

- **Behaviour-preserving.** After the change, the program does the same thing from the outside. Same outputs for the same inputs, same side effects (changes made besides returning a value, such as writing a file), same errors. The only things that may change are *internal* — names, where code lives, how it is split up. (Performance may change a little; see 7.1.9.)
- **Small steps.** Each step is so small that it is almost impossible to get wrong, and the code compiles and the tests pass after *every* step. You chain many tiny steps to get a big change. If you ever sit with broken code for an hour, you were not refactoring — you were rewriting.

Everyday analogy. Think of reorganising a kitchen while the restaurant stays open. You move one shelf at a time. After each move, the cooks can still make every dish. You never tear the whole kitchen down on a Friday night.

> **Term — observable behaviour.** What a user (or calling code) can see: return values, printed output, data written, exceptions thrown. The *call stack* (the chain of method calls currently in progress), the *class structure*, and the *names of private methods* are not observable behaviour.

What refactoring is NOT.

Activity	Is it refactoring?	Why
Renaming a method with the IDE	Yes	Behaviour unchanged, tiny step
Fixing a bug	No	Behaviour changes (on purpose)
Adding a feature	No	Behaviour changes
Performance tuning	No	Goal is speed, not clarity (though behaviour is preserved)
"I spent two days restructuring and nothing worked until the end"	No	That is restructuring/rewriting, not refactoring
Reformatting code	Borderline	Behaviour-preserving, but it does not improve design

> **Term — restructuring.** The general word for reorganising code. Refactoring is a *specific kind* of restructuring: small, behaviour-preserving steps.

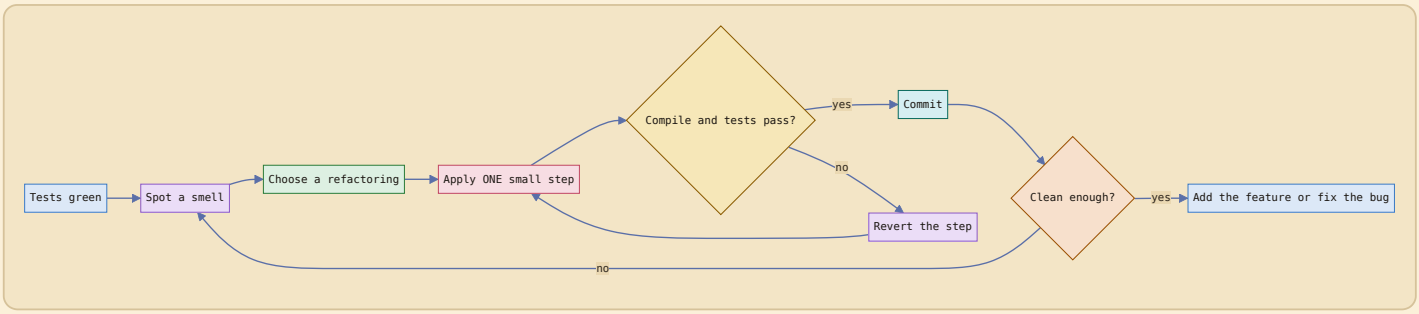
The problem it solves

Code decays. Every quick fix, every "just add another `if`", every copy-paste makes the next change a little harder. This is called **design decay** or **technical debt** — the extra cost you pay on every future change because the code is harder to work with than it needs to be.

Refactoring pays down that debt. It keeps the design good *continuously*, instead of hoping to get the design right up front (which never fully works, because requirements change).

How it works

1. **Make sure you have tests** that pin down current behaviour (7.1.7). Run them — they must be green (all passing).
2. **Spot a smell** — a surface sign that the design could be better (7.2).
3. **Pick a named refactoring** that cures the smell (7.3, 7.4).
4. **Apply one small step** of it (ideally with the IDE's automated refactoring).
5. **Compile and run the tests.** Green? Commit (locally) — i.e. save a snapshot of the code in version control such as Git. Red? Undo the step (don't debug) and take a smaller step.
6. **Repeat** until the code is clean enough for the job at hand.



7.1.1 The two hats

Kent Beck's metaphor: when you program you wear **one of two hats** at any moment.

Hat	You are allowed to	You are NOT allowed to
Adding function	Add new behaviour, add new tests	Restructure existing code
Refactoring	Restructure code	Add behaviour or tests for new behaviour (you may add tests for *existing* behaviour you forgot to cover)

You swap hats often — maybe every few minutes — but you always know which one you are wearing. Mixing them ("I'll just fix this bug while I extract this method") is how refactoring becomes risky: when a test fails you cannot tell which change caused it.

7.1.2 Why refactor

- **It improves the design.** Without it, the internal design (architecture) of a program decays as people change code to meet short-term goals. Regular refactoring keeps the code in shape. A big part of this is **removing duplication** — making sure each fact lives in exactly one place.
- **It makes software easier to understand.** Code is read far more often than it is written. Refactoring lets you move your understanding *out of your head and into the code*, so the next reader (including you in six months) does not need to rediscover it.
- **It helps you find bugs.** When you clarify the structure, assumptions become visible and bugs have fewer places to hide.
- **It helps you program faster.** This is the surprising one. Fowler calls it the **Design Stamina Hypothesis**: a team that keeps its internal quality high will, after a short while, add features *faster* than a team that does not, because each feature builds on clean code instead of fighting a mess. Speed is the economic argument for refactoring — not "clean code for its own sake".

7.1.3 When to refactor

Trigger	What it means	Example
Rule of Three (Don Roberts)	First time, just do it. Second time you do something similar, wince but duplicate. Third time, refactor.	Third copy of the same validation code → extract a validator
Preparatory refactoring	Refactor *just before* adding a feature, to make the feature easy to add. "Make the change easy, then make the easy change."	Before adding HTML output, split calculation from text formatting (see 7.5, step 3)
Comprehension refactoring	Refactor to understand code you are reading. Rename a variable once you understand it; extract a block once you know what it does. The understanding moves into the code.	Rename `d` to `daysOverdue` after working out what it holds
Litter-pickup refactoring	You notice mess while working on something else. Fix it if it is quick; if not, note it and come back. The **Boy Scout Rule** — leave the campground cleaner than you found it.	Remove an unused parameter while passing by
Planned vs opportunistic	Most refactoring should be *opportunistic* — woven into feature work and bug fixes, not scheduled separately. Planned refactoring episodes are sometimes needed when a team has neglected the code, but they should be rare.	Good: every feature PR (pull request — a change submitted for review) contains a few small refactoring commits. Warning sign: a "refactoring sprint" (a fixed 1–4-week work period) every quarter
Long-term refactoring	Big changes (replacing a library, splitting a module) done gradually over weeks, keeping the system working the whole time — e.g. with **Branch by Abstraction** (introduce an abstraction layer, switch callers to it, swap the implementation behind it, delete the old one).	Moving from one ORM (library mapping objects to database tables) to another behind a repository interface
During code review	Reviewing is a natural moment to suggest (or pair on) refactorings. Fowler prefers doing it *with the author*, trying the refactoring ideas as you go, rather than only leaving comments.	"This method has four flag parameters — can we Remove Flag Argument?"

> **Term — Rule of Three.** A heuristic: do not abstract on the first or second occurrence of a pattern (you do not yet know what varies); abstract on the third, when the shape is clear.

Should you tell your manager? Fowler's position: for a manager who does not understand technical work, do not ask permission for refactoring any more than you ask permission to write tests. It is part of how professionals deliver features quickly. (For large, planned efforts, of course, discuss them.)

7.1.4 When NOT to refactor

- **Code you never need to touch or understand.** Messy code hidden behind a clean API (the set of methods other code calls), that works and never changes, can stay messy. Refactor only what gets in your way.
- **When rewriting is cheaper** than refactoring (see 7.1.10). This is a judgement call and usually needs experience.
- **Without a safety net** — if you cannot test it and cannot use safe automated refactorings, first add tests (7.1.11 shows how for legacy code).
- **On a deadline that truly cannot move**, a big refactoring may wait — but note that the Design Stamina Hypothesis says skipping it usually makes you *slower*, even in the short term. Deadlines are rarely a good excuse for small refactorings.
- **Published interfaces** that you do not control the callers of (a public library API, a REST endpoint — a URL of a web API, see System Design Fundamentals — used by other teams). You can still refactor them, but you must keep the old version working for a while and deprecate it (mark it as outdated and due for removal) (see *Change Function Declaration*, migration mechanics).

7.1.5 Problems and costs of refactoring

Problem	What goes wrong	Mitigation
Slowing down new features	Refactoring "takes time away" from features	Refactor *because* it speeds features up; focus on code you are about to change
Code ownership	You cannot rename a method whose callers are in another team's repo (repository — a project's version-controlled code store)	Keep the old name as a delegating, `@Deprecated` method; migrate callers later; prefer weak (team-wide) code ownership
Long-lived feature branches (lines of work kept apart from the main code for a long time)	Refactoring renames and moves code, which makes merges painful	**Continuous Integration** (merge into the shared main line often, with automatic builds and tests) / trunk-based development (everyone works on one main branch): integrate at least daily, so refactorings conflict less
Missing tests	Refactoring without tests is risky	Add tests first; use only IDE-automated, provably-safe refactorings until tests exist
Legacy code	Big, untested, tangled	Feathers' techniques (7.1.11)
Databases	Schema (table structure) changes are hard to reverse	**Evolutionary database design**: small migration scripts (scripts that change the database structure), versioned with the code; use **parallel change** (add new column, copy, switch readers, remove old)

> **Term — parallel change (expand / contract).** Make a breaking change in three phases: *expand* (add the new thing alongside the old), *migrate* (move all users to the new thing), *contract* (remove the old thing). At no point is anything broken.

7.1.6 Refactoring, architecture and YAGNI

In the past people tried to get the architecture "right" before writing code. Refactoring changes that: because you can reshape code safely later, you can build **only what today's requirements need** and evolve the design when new needs appear. This is **YAGNI** ("You Aren't Gonna Need It") — do not add flexibility for imagined future requirements. YAGNI does not mean "never think about architecture"; it means you trust refactoring to add the flexibility when the need becomes real (and you think harder up front only about things that are *truly* hard to change later, like a public API or a database schema).

7.1.7 Tests as the safety net

Refactoring is safe only if you can detect a behaviour change immediately. That is what **self-testing code** gives you: a suite of automated tests that checks itself (pass/fail, no human reading output) and runs in seconds.

- Run tests after **every** small step. With fast tests, a failure points at the few lines you just changed.
- If a test fails, **revert** — do not start debugging a refactoring. Then redo it in smaller steps.

- Tests should focus on **behaviour** (public API, outputs), not on private structure. Tests that check private methods break on every refactoring and become a cost instead of a safety net.
- **Characterisation tests** (a term from Michael Feathers): for code without tests, write tests that record what the code *currently does* — even if that looks wrong — so you notice if a refactoring changes it. The worked example in 7.5 uses exactly this: a checker that compares every step's output with the original, byte for byte.
- Consider **mutation testing** (e.g. PIT for Java) to check your tests actually catch changes — it deliberately introduces small bugs and reports any your tests fail to detect.

> **Term — safety net.** Anything that tells you quickly and reliably that you broke something: unit tests (one class tested in isolation), integration tests (several real parts tested together), the compiler's type checks, and the IDE's automated refactorings (which are proven safe by the tool).

7.1.8 IDE automation

Modern IDEs perform many refactorings *automatically and safely*, using the compiler's knowledge of the code (the **syntax tree** and **symbol resolution** — the parsed structure of the code, and knowing exactly which declaration each name refers to), not text search. A text search-and-replace of `name` would also hit comments, strings and unrelated fields; the IDE's Rename only changes that one symbol and all its real references.

History in one line: Bill Opdyke's 1992 PhD thesis formalised refactorings; John Brant and Don Roberts built the Smalltalk (an early object-oriented language) *Refactoring Browser*, the first automated tool; Java IDEs (IntelliJ IDEA, Eclipse) popularised it.

Refactoring	IntelliJ IDEA (Windows/Linux · macOS)	Eclipse (Windows/Linux)
Rename	Shift+F6 · Shift+F6	Alt+Shift+R
Extract Method (Extract Function)	Ctrl+Alt+M · Cmd+Opt+M	Alt+Shift+M
Extract Variable	Ctrl+Alt+V · Cmd+Opt+V	Alt+Shift+L
Extract Field / Constant / Parameter	Ctrl+Alt+F / C / P · Cmd+Opt+F / C / P	(Refactor menu)
Inline	Ctrl+Alt+N · Cmd+Opt+N	Alt+Shift+I
Change Signature	Ctrl+F6 · Cmd+F6	Alt+Shift+C
Move	F6 · F6	Alt+Shift+V
"Refactor This" menu	Ctrl+Alt+Shift+T · Ctrl+T	Alt+Shift+T

(Shortcuts are the default keymaps; check your IDE if you use a custom keymap.)

Beyond the IDE there are **large-scale automated refactoring tools**, such as OpenRewrite (recipes that, for example, migrate a whole codebase between framework versions) and Google's Refaster templates in Error Prone (Google's bug-finding plugin for the Java compiler). These let a company apply the same refactoring across thousands of files.

> **Caution.** Automated refactorings are safe for statically known calls. They can miss **reflection** (`Class.forName("…")` — code that looks up classes or methods by name at runtime), names used in **configuration files**, **serialised field names** (serialising = turning objects into text or bytes, e.g. JSON, a plain-text data format; JSON/Jackson, JPA — Java's standard ORM API — column names inferred from field names), and code in other repositories. After an IDE rename of a field that is serialised, the JSON your API returns changes — that *is* an observable behaviour change.

7.1.9 Performance and refactoring

A common worry: "refactoring adds method calls and small objects, so the program gets slower." Fowler's answer has three parts:

1. **Most refactorings have negligible performance effect.** The JIT (Just-In-Time) compiler — the part of the JVM, Java's runtime, that compiles frequently run code to machine code while the program runs — inlines small methods (copies their body into the caller); extra method calls are nearly free in modern Java.

2. **Programs spend most of their time in a small fraction of the code.** Optimising everywhere wastes effort on code that is rarely run. (The book retells a story from Ron Jeffries about Chrysler's slow payroll system: nobody's guesses about the cause were right; profiling (measuring where a program actually spends its time) found the time going to one unexpected spot, and a small fix there made it fast.)
3. **Well-factored code is easier to tune.** When you do need speed, you profile, find the hot spot (the small piece of code where most time goes), and — because the code is well-structured — the hot spot is small and isolated, so it is easy to optimise.

So the recommended order is: **make it work → make it clean (refactor) → make it fast (profile, then optimise only the hot spots)**. Sometimes a refactoring makes code slightly slower; that is fine if it makes the later, targeted optimisation easier.

Example from this chapter. In 7.5 step 2 we replace a summing loop with a stream (Java's pipeline for processing a sequence of elements). `DoubleStream.sum()` is allowed to use *compensated summation* (a technique that tracks rounding errors to make floating-point sums more accurate; OpenJDK, the reference Java implementation, does), so it can give a result that differs in the last bit from a plain loop — a tiny *behaviour change*. To stay a true refactoring we used `reduce(0.0, Double::sum)`, which adds left to right exactly like the loop. Refactoring demands this kind of care.

7.1.10 Refactoring vs rewriting

	Refactoring	Rewriting
Unit of change	Many tiny, safe steps	One big replacement
System works during the change?	Always	Usually not until the end
Risk	Low	High — known bugs fixed years ago get reintroduced; the old system keeps evolving while you rewrite
Delivers value	Continuously	Only at the end
Good when	Code mostly works, needs reshaping	Code is so broken it does not work at all, or the technology must change completely

Even when you must replace a system, you rarely do it in one go. The **Strangler Fig** approach (named by Fowler after a vine that slowly grows around a tree) builds the new system *around* the old one, routing one feature at a time to the new code until the old code can be switched off. It is the rewrite equivalent of "small steps".

7.1.11 Legacy code

Michael Feathers (*Working Effectively with Legacy Code*, 2004) defines **legacy code as code without tests**. Such code is hard to refactor because you have no safety net, and hard to test because it was not designed for it. The way out is a loop:

1. **Identify change points** — where you need to change code.
2. **Find test points** — where you can observe behaviour.
3. **Break dependencies** — using a **seam**: a place where you can change behaviour without editing the code there, e.g. by passing in an interface instead of calling `new Database()` inside a method (constructor injection), or overriding a method in a test subclass.
4. **Write characterisation tests** that pin down current behaviour.
5. **Make the change and refactor** under that safety net.

Other useful techniques from that book:

- **Sprout Method / Sprout Class** — write the new feature as a *new*, fully tested method or class, and call it from the old code with a one-line change.
- **Wrap Method / Wrap Class** — rename the old method and create a new method with the old name that calls the old one plus the new behaviour (a Decorator-like move — a Decorator wraps an object to add behaviour).
- Use **safe automated refactorings only** (Rename, Extract Method with the IDE) until tests exist.
- Do not try to clean everything. Refactor the areas you touch; over time the most-changed code becomes the cleanest.

7.2 Code smells

What it is

A **code smell** (a term coined by Kent Beck) is a *surface sign* in the code that usually points to a deeper design problem. A smell is not a bug — the code works. It is a hint that the code will be hard to understand or change. Like a smell in a fridge, it tells you *where to look*, not exactly what is wrong.

Smells are **heuristics, not rules**. A long method is not always bad; a switch statement is not always bad. The smell tells you to look closer and use judgement.

The problem it solves

Knowing *how* to refactor is not enough; you must also know *when* and *where*. Fowler and Beck deliberately give no precise metrics ("no method longer than 10 lines"). Instead they give a catalogue of named smells, each with the refactorings that usually cure it. Names matter: in a code review, "Feature Envy" communicates in two words what would otherwise take a paragraph.

How it works

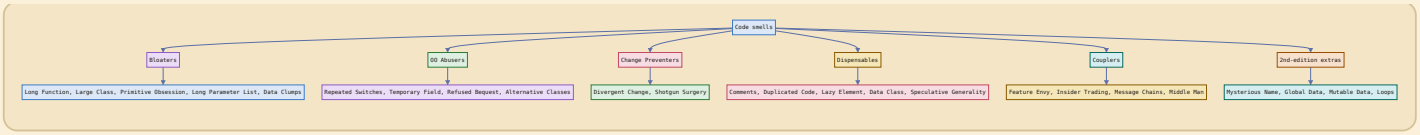
1. Read the code and notice discomfort: "I had to scroll", "I changed five files for one feature", "I can't tell what `x2` is".
2. Match the discomfort to a named smell (the table below).
3. Look up the refactorings that cure it (each smell below, and 7.4).
4. Apply them in small steps under tests.

Smell groups (refactoring.guru classification)

refactoring.guru (a popular website on refactoring and design patterns) groups smells into five families. It uses mostly the 1st-edition names, so the table maps them to the 2nd-edition names used in this chapter.

Group	Idea	Smells (2nd-ed name; refactoring.guru name in brackets if different)
Bloaters	Code, methods and classes that have grown so big they are hard to work with. They usually grow slowly.	Long Function *(Long Method)*, Large Class, Primitive Obsession, Long Parameter List, Data Clumps
Object-Orientation Abusers	Incomplete or wrong use of OO (object-oriented) principles.	Repeated Switches *(Switch Statements)*, Temporary Field, Refused Bequest, Alternative Classes with Different Interfaces
Change Preventers	One change forces many other changes.	Divergent Change, Shotgun Surgery *(also: Parallel Inheritance Hierarchies — a 1st-ed smell)*
Dispensables	Something pointless whose removal makes the code cleaner.	Comments, Duplicated Code *(Duplicate Code)*, Lazy Element *(Lazy Class)*, Data Class, Speculative Generality *(also: Dead Code — in the 2nd ed. this is the refactoring Remove Dead Code)*
Couplers	Too much coupling between classes, or too much delegation (handing work off to another object).	Feature Envy, Insider Trading *(Inappropriate Intimacy)*, Message Chains, Middle Man *(also: Incomplete Library Class — a 1st-ed smell)*
New in the 2nd edition (not in refactoring.guru's five groups)		Mysterious Name (a comprehension smell), Global Data and Mutable Data (both about uncontrolled shared state — closest to Couplers/Change Preventers), Loops (closest to Bloaters)

> **Term — coupling.** How much one piece of code depends on another. High coupling means a change in one place forces changes in others. **Cohesion** is the opposite idea inside one unit: how strongly the things inside a class or module belong together. Good design: low coupling, high cohesion.



Below are all 24 smells from the 2nd edition, in the book's order. Each has: **what it is**, **why it hurts**, a small **Java example** (smelly → cured), and **refactorings that fix it** (the ones Fowler names for that smell; anything extra is marked). The names match the catalogue in 7.3.

> **Note on the snippets.** Several short snippets use `double` for prices or balances to stay tiny. Real money should use `BigDecimal` (Java's exact decimal type) or a `Money` value object (a small immutable object compared by its value) — see 7.2.11 and `code/smells/Money.java`.

7.2.1 Mysterious Name

What it is. A function, variable, field, class or module whose name does not tell you what it does or holds: `data`, `tmp`, `proc()`, `x2`, `Manager`, `handle()`.

Why it hurts. Names are the main tool for making code understandable. A bad name forces every reader to decode the implementation. And if you *cannot* think of a good name, that is often a sign of a deeper design problem — the thing does too much or has no clear purpose.

```
// Smelly
double calc(List<Double> l, int d) { ... }

// Cured
double averageDailyRevenue(List<Double> dailyRevenues, int daysInPeriod) { ... }
```

Refactorings. Change Function Declaration (rename), Rename Variable, Rename Field.

7.2.2 Duplicated Code

What it is. The same (or nearly the same) code structure in more than one place.

Why it hurts. Every change must be made in every copy, and you must *find* every copy and check whether the small differences matter. Sooner or later one copy is missed and a bug appears.

```
// Smelly: two methods repeat the same tax rule
double invoiceTotal(double net)  { return net + net * 0.18; }
double quoteTotal(double net)    { return net + net * 0.18; }

// Cured: one place owns the rule
private static final double GST_RATE = 0.18;
double withTax(double net)       { return net + net * GST_RATE; }
double invoiceTotal(double net)  { return withTax(net); }
double quoteTotal(double net)    { return withTax(net); }
```

Refactorings. Fowler lists: Extract Function; Slide Statements (to bring similar code together so it can be extracted); Pull Up Method (if the copies are in sibling subclasses). Also useful (not in Fowler's list for this smell): Replace Inline Code with Function Call (if a library already does it); Substitute Algorithm (if two different algorithms do the same thing).

7.2.3 Long Function

What it is. A function that does too much and is too long to understand at a glance. Fowler's heuristic: whenever you feel you need a **comment** to explain a block, extract that block into a function named after the *intent* of the block.

Why it hurts. Long functions are hard to understand, hard to reuse partially, and hard to test. Short, well-named functions are self-documenting: you read the name and skip the body.

```java
// Smelly
void printOwing(Invoice inv) {
    System.out.println("*** Customer Owes ***");
    double outstanding = 0;
    for (Order o : inv.orders()) outstanding += o.amount();      // calculate outstanding
    inv.setDueDate(LocalDate.now().plusDays(30));               // record due date
    System.out.println("name: " + inv.customer());               // print details
    System.out.println("amount: " + outstanding);
}

// Cured: each block became a named function
void printOwing(Invoice inv) {
    printBanner();
    double outstanding = outstandingOf(inv);
    recordDueDate(inv);
    printDetails(inv, outstanding);
}
```

Refactorings. Extract Function (the main one, ~99% of the time), Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop.

7.2.4 Long Parameter List

What it is. A function that takes many parameters (often more than three or four), especially several of the same type, or boolean flags.

Why it hurts. Callers mix up the order (`move(10, 20, 5, 3)` — which is which?), every new parameter changes every caller, and long lists often hide a missing concept.

```java
// Smelly
Booking book(String hotelId, LocalDate from, LocalDate to, int adults, int children, boolean breakfast) { ... }

// Cured: parameter objects for the clumps, and an explicit enum instead of an unreadable boolean flag
record Stay(LocalDate from, LocalDate to) { }
record Guests(int adults, int children) { }
Booking book(Hotel hotel, Stay stay, Guests guests, MealPlan mealPlan) { ... }
```

Refactorings. Replace Parameter with Query, Preserve Whole Object, Introduce Parameter Object, Remove Flag Argument, Combine Functions into Class (several functions share the same parameters → make them fields of a class).

7.2.5 Global Data

What it is. Data that can be read and modified from anywhere: global variables, public static mutable fields (`static` fields belong to the class and are shared by the whole program), and mutable singletons (classes with exactly one shared instance).

Why it hurts. Any code anywhere can change it, and there is no way to find *who* changed it. Bugs appear far away from their cause. Fowler calls it one of the most pungent smells; the problems grow with the size of the program. (Class variables and singletons count too.)

```java
// Smelly
public class Config { public static double discountRate = 0.1; }      // anyone can write it

// Cured: Encapsulate Variable - one controlled access point, easy to find every writer.
// (The range check is a NEW rule added afterwards, wearing the "adding function" hat - it is a behaviour change,
not part of the refactoring.)
public final class Config {
    private static volatile double discountRate = 0.1;
    public static double discountRate() { return discountRate; }
    public static void setDiscountRate(double rate) {
        if (rate < 0 || rate > 0.5) throw new IllegalArgumentException("rate out of range");
        discountRate = rate;
    }
}
// Even better: pass the value in (dependency injection) instead of reading a static at all.
```

Refactorings. Encapsulate Variable (first step, always), then narrow its scope — ideally move it into a class or module so only a few functions can see it.

7.2.6 Mutable Data

What it is. Data that is changed after creation, especially when it is shared, or when the same variable is updated for different purposes.

Why it hurts. A change in one place can break code in another place that assumed the value stayed the same. These bugs are hard to find, especially with threads (several paths of execution running at the same time). Functional programming is built on the idea that data should never change; you do not need to go that far, but you should limit where mutation can happen.

```java
// Smelly: callers can change the list inside the object, and total is a stored copy that can go stale
class Cart {
    public List<Item> items = new ArrayList<>();
    public double total;                          // must be updated by hand after every change
}

// Cured
final class Cart {
    private final List<Item> items = new ArrayList<>();
    void add(Item item) { items.add(item); }
    List<Item> items() { return List.copyOf(items); }                    // Encapsulate Collection
    double total() { return items.stream().mapToDouble(Item::price).sum(); } // Replace Derived Variable with Query
}
```

Refactorings. Encapsulate Variable, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class, Combine Functions into Transform, Change Reference to Value (use immutable value objects such as Java records — compact, immutable data classes).

7.2.7 Divergent Change

What it is. **One class is changed for many different reasons.** "I have to edit this class when we add a database, *and* when we add a financial instrument, *and* when we change the report format."

Why it hurts. The class mixes several responsibilities (it breaks the **Single Responsibility Principle**, Chapter 4). Each kind of change risks breaking the others, and different people edit the same file for unrelated reasons.

```java
// Smelly: storage, pricing and formatting in one class
class ProductService {
    void saveToMySql(Product p) { ... }         // changes when the database changes
    double priceWithTax(Product p) { ... }      // changes when tax rules change
    String toCsv(Product p) { ... }             // changes when the export format changes
}

// Cured: one reason to change per class
class ProductRepository { void save(Product p) { ... } }
class PricingService    { double priceWithTax(Product p) { ... } }
class ProductCsvWriter  { String toCsv(Product p) { ... } }
```

Refactorings. Split Phase (when the changes fall into sequential phases, e.g. parse then compute), Move Function, Extract Function, Extract Class.

7.2.8 Shotgun Surgery

What it is. The opposite of Divergent Change: **one change forces many small edits in many classes.**

Why it hurts. You have to find all the places, and it is easy to miss one. The logic for one concept is scattered.

```java
// Smelly: the "free-shipping threshold" rule appears in 4 classes
class CartView      { boolean showBanner(double t)  { return t >= 500; } }
class CheckoutCtrl  { double shipping(double t)     { return t >= 500 ? 0 : 40; } }
class EmailBuilder  { String line(double t)         { return t >= 500 ? "Free shipping!" : ""; } }
class ReportJob     { long freeOrders(List<Double> ts) { return ts.stream().filter(t -> t >= 500).count(); } }

// Cured: the concept lives in one class, and the four callers use it
final class ShippingPolicy {
    static final double FREE_SHIPPING_THRESHOLD = 500;
    static boolean isFree(double total)   { return total >= FREE_SHIPPING_THRESHOLD; }
    static double charge(double total)    { return isFree(total) ? 0 : 40; }
}
```

Refactorings. Move Function and Move Field (gather everything into one module), Combine Functions into Class, Combine Functions into Transform, Split Phase; sometimes Inline Function / Inline Class first to pull the scattered bits together, then extract a better-shaped unit.

	Divergent Change	Shotgun Surgery
Shape	One class, many reasons to change	One reason to change, many classes
Fix	Split the class	Gather the scattered code

7.2.9 Feature Envy

What it is. A method that is more interested in *another* class's data than in its own — it calls many getters (methods that just return a field) on another object to compute something.

Why it hurts. The behaviour lives away from the data it uses, so a change to that data needs changes elsewhere. The basic OO rule: **put behaviour next to the data it uses**. (Exception: patterns like Strategy and Visitor — an object that carries one operation across a group of classes — deliberately separate them, to allow swapping behaviour.)

```java
// Smelly: Order computes a line total using only Line's data
class Order {
    double lineTotal(Line line) {
        return line.getUnitPrice() * line.getQuantity() * (1 - line.getDiscount());
    }
}

// Cured: Move Function - Line computes its own total
record Line(double unitPrice, int quantity, double discount) {
    double total() { return unitPrice * quantity * (1 - discount); }
}
```

Refactorings. Move Function; Extract Function first if only *part* of the method is envious, then move that part.

7.2.10 Data Clumps

What it is. The same group of data items always appear together — as fields in several classes, or as parameters in many signatures: `(street, city, zip)`, `(startDate, endDate)`, `(x, y)`, `(amount, currency)`.

Why it hurts. The group is really a concept that has not been named. Without a class, behaviour about it (e.g. "does this range contain a date?") gets duplicated everywhere. **Test:** delete one item of the clump — if the others no longer make sense alone, they belong in an object.

```java
// Smelly
boolean isAvailable(LocalDate start, LocalDate end) { ... }
double price(LocalDate start, LocalDate end, double nightly) { ... }

// Cured (see code/smells/DateRange.java)
record DateRange(LocalDate start, LocalDate end) {
    boolean contains(LocalDate day) { return !day.isBefore(start) && !day.isAfter(end); }
}
boolean isAvailable(DateRange stay) { ... }
```

Refactorings. Extract Class (for clumps of fields), Introduce Parameter Object and Preserve Whole Object (for clumps of parameters). Then look for Feature Envy — behaviour will want to move onto the new class.

7.2.11 Primitive Obsession

What it is. Using primitive types (`int`, `double`, `String`, `boolean`) for domain concepts that deserve their own type: money as `double`, phone numbers as `String`, a status as `"ACTIVE"`, a temperature as a bare `int` without a unit.

Why it hurts. No validation (`"banana"` is a valid `String` phone number), no behaviour (formatting and comparing logic is duplicated), easy mix-ups (`transfer(double amount, double fee)` — swap them and nothing complains). Money as `double` also suffers from floating-point error (binary `double` numbers cannot store most decimal fractions exactly): `0.1 + 0.2` prints `0.30000000000000004` (see the output in 7.8).

```java
// Smelly
void charge(double amount, String currency) { ... }

// Cured (see code/smells/Money.java): validation and arithmetic live in one type
record Money(BigDecimal amount, Currency currency) {
    Money plus(Money other) { /* checks same currency, then adds */ }
}
void charge(Money amount) { ... }
```

Refactorings. Replace Primitive with Object, Replace Type Code with Subclasses, Replace Conditional with Polymorphism (for type codes that drive behaviour), Extract Class, Introduce Parameter Object.

7.2.12 Repeated Switches

What it is. The *same* `switch` (or `if/else if` chain) on the same type code (a field, such as a string or int, that says which kind of thing an object is) appears in several places. (In the 1st edition this smell was "Switch Statements" — *any* switch. In the 2nd edition Fowler narrowed it: a single switch is fine; the problem is the *repetition*.)

Why it hurts. Adding a new case means finding and updating every switch — Shotgun Surgery. Miss one and you have a bug.

```
// Smelly: three switches on the same type
double speed(Bird b)    { switch (b.type) { case "EUROPEAN": ...; case "AFRICAN": ...; case "PARROT": ...; } }
String plumage(Bird b)  { switch (b.type) { case "EUROPEAN": ...; case "AFRICAN": ...; case "PARROT": ...; } }
int    lifespan(Bird b) { switch (b.type) { ... } }

// Cured: Replace Conditional with Polymorphism
sealed interface Bird permits EuropeanSwallow, AfricanSwallow, NorwegianBlueParrot {
    double speed();
    String plumage();
}
```

Refactorings. Replace Conditional with Polymorphism. (Java note: with a `sealed` type (one that lists all its permitted subtypes) or an `enum` (a fixed set of named constants), a *switch expression* is checked for exhaustiveness — covering every case — by the compiler, which removes much of the risk. A few such switches may be acceptable; polymorphism is still better when the behaviour per case is large.)

7.2.13 Loops

What it is. A classic `for`/`while` loop that filters, maps or aggregates a collection, where a pipeline would state the intent more clearly. (New in the 2nd edition — languages now have good first-class functions (functions that can be stored in variables and passed around); in Java, the Stream API since Java 8.)

Why it hurts. A loop makes the reader simulate the steps to see what it does. A pipeline names each step: *filter*, *map*, *collect*.

```
// Smelly
List<String> names = new ArrayList<>();
for (Employee e : employees) {
    if (e.department().equals("IT")) {
        names.add(e.name());
    }
}

// Cured: Replace Loop with Pipeline
List<String> names = employees.stream()
        .filter(e -> e.department().equals("IT"))
        .map(Employee::name)
        .toList();                      // Java 16+: returns an unmodifiable list
```

Refactorings. Replace Loop with Pipeline. (Note: `Stream.toList()` returns an *unmodifiable* list, while the loop built a mutable `ArrayList`. If a caller later adds to the list, that is a behaviour change — use `collect(Collectors.toCollection(ArrayList::new))` in that case.)

7.2.14 Lazy Element

What it is. A class, function or other structure that does not pull its weight — for example a class with one trivial method, a function whose body is as clear as its name, or a subclass that adds nothing. (1st edition: "Lazy Class".)

Why it hurts. Every element costs something: a name to learn, a file to open, an indirection to follow. If it adds nothing, it is pure cost. Lazy elements are often left behind after other refactorings, or created for growth that never happened.

```
// Smelly: a wrapper that adds nothing
class NameFormatter { String format(String s) { return s.trim(); } }

// Cured: Inline Class / Inline Function
String cleanName = rawName.trim();
```

Refactorings. Inline Function, Inline Class, Collapse Hierarchy.

7.2.15 Speculative Generality

What it is. Hooks, abstract classes, parameters and special cases added "because we might need them someday" — but nobody uses them. Typical signs: an abstract class with only one subclass, unused parameters, methods only called by tests, a plug-in system with one plug-in.

Why it hurts. It makes the code harder to understand and maintain for no benefit. It violates **YAGNI**. If the need does come, refactoring can add the flexibility then — usually in a better shape, because you then know the real requirement.

```
// Smelly
interface PaymentProcessorFactoryProvider { PaymentProcessorFactory get(); }   // one implementation, ever
void pay(Money m, Map<String, Object> futureOptions) { ... }                  // futureOptions never used

// Cured
void pay(Money m) { processor.process(m); }
```

Refactorings. Collapse Hierarchy (useless abstract class), Inline Function and Inline Class (useless delegation), Change Function Declaration (remove unused parameters), Remove Dead Code (for code used only by its own tests — delete the tests too).

7.2.16 Temporary Field

What it is. A field that is set and used only in some situations (e.g. only during one complex calculation) and is empty/null the rest of the time.

Why it hurts. Readers expect an object to need all its fields. A field that is usually `null` is confusing — "why is this null? is that a bug?"

```
// Smelly: these fields are only meaningful while computeRoute() runs
class Planner {
    private Graph graph;
    private Map<Node, Integer> tmpDistances;   // null except inside computeRoute
    private Node tmpTarget;                    // null except inside computeRoute
}

// Cured: Extract Class - the temporary state gets its own short-lived object
class Planner {
    private Graph graph;
    Route computeRoute(Node from, Node to) { return new RouteSearch(graph, from, to).run(); }
}
final class RouteSearch {                      // lives only for one search
    private final Map<Node, Integer> distances = new HashMap<>();
    ...
}
```

Refactorings. Extract Class (put the fields and the code that uses them in their own class), Move Function, Introduce Special Case (for the conditional code that checks whether the field is set).

7.2.17 Message Chains

What it is. A client asks one object for another object, then asks that for another, and so on: `order.getCustomer().getAddress().getCity().getName()`. Also a long chain of temporary variables doing the same thing.

Why it hurts. The client is coupled to the *whole structure* of the navigation. Any change in an intermediate relationship breaks the client. It violates the **Law of Demeter** ("talk only to your immediate friends", Chapter 5).

```
// Smelly
String city = order.getCustomer().getAddress().getCity();

// Cured: Hide Delegate (see code/smells/Order.java)
String city = order.shippingCity();
```

Caution: do not blindly hide every delegate — that creates Middle Men. Often better: look at what the final object is *used for*, and Extract Function + Move Function that usage down the chain.

Refactorings. Hide Delegate, Extract Function, Move Function.

7.2.18 Middle Man

What it is. A class whose methods mostly just forward calls to another class, adding nothing. (Some delegation is good — that is encapsulation. The smell is when *most* of the class is pass-through.)

Why it hurts. An extra layer to read and maintain, with no value. Every new feature of the delegate requires a new forwarding method.

```
// Smelly
class Person {
    private Department dept;
    Person getManager()           { return dept.getManager(); }
    String getDepartmentCode()    { return dept.getCode(); }
    Budget getDepartmentBudget()  { return dept.getBudget(); }
    // ... ten more forwarding methods
}

// Cured: Remove Middle Man - let clients talk to the Department directly
Department department() { return dept; }
```

Refactorings. Remove Middle Man, Inline Function (for a few forwarders), Replace Superclass with Delegate or Replace Subclass with Delegate (if the middle man is tied up in inheritance).

Message Chains and Middle Man pull in opposite directions — Hide Delegate cures one and can cause the other. Use judgement to find the balance.

7.2.19 Insider Trading

What it is. Two modules (classes, packages) that exchange too much private information — they poke at each other's internals. Subclasses that know too much about their parents' internals are a common case. (1st edition: "Inappropriate Intimacy".)

Why it hurts. The modules are tightly coupled: you cannot change one without changing the other. Some trading is necessary; the goal is to keep it small and visible.

```
// Smelly: Invoice reaches into Customer's internal map
double discount = customer.internalLoyaltyTable.get(customer.tierCode) * 0.01;

// Cured: Customer owns its own rule, Invoice asks
double discount = customer.loyaltyDiscount();
```

Refactorings. Move Function and Move Field (reduce the need to talk), Hide Delegate (make a third module the go-between), Replace Subclass with Delegate or Replace Superclass with Delegate (when inheritance creates the

intimacy).

7.2.20 Large Class

What it is. A class trying to do too much — too many fields, too many methods, too much code. Often called a **God Class**.

Why it hurts. It becomes a magnet for duplication and a bottleneck for changes (everyone edits it). It usually violates the Single Responsibility Principle.

```
// Smelly
class Employee {
    String name; String email;                    // identity
    String street, city, zip;                     // address clump
    double baseSalary, bonus, taxRate;            // payroll
    List<Review> reviews; int rating;             // performance
    ...
}

// Cured: Extract Class for each cohesive subset (look for fields with common prefixes/suffixes,
// or fields used together by the same methods)
class Employee {
    String name; String email;
    Address address;
    Compensation compensation;
    PerformanceRecord performance;
}
```

Refactorings. Extract Class, Extract Superclass, Replace Type Code with Subclasses (Extract Subclass). Tip: look at how *clients* use the class — distinct groups of clients using distinct subsets of methods suggest how to split it.

7.2.21 Alternative Classes with Different Interfaces

What it is. Two classes that do the same kind of job but have different method names or signatures, so you cannot substitute one for the other: `EmailSender.sendMail(to, body)` and `SmsGateway.dispatch(msg, phone)`.

Why it hurts. You cannot treat them polymorphically (through one shared interface, without caring which class you have), so callers write `if email ... else if sms ...`. It also often hides duplicated code.

```
// Smelly
class EmailSender { void sendMail(String to, String body) { ... } }
class SmsGateway  { void dispatch(String body, String phone) { ... } }

// Cured: rename and align signatures, then extract a common interface
interface Notifier { void send(String recipient, String message); }
class EmailNotifier implements Notifier { ... }
class SmsNotifier   implements Notifier { ... }
```

Refactorings. Change Function Declaration (make signatures match), Move Function (until the protocols match), Extract Superclass (if the implementations then share code). If you cannot edit one of the classes (third-party), wrap it in an **Adapter** (Chapter 9) — a class that converts one interface into the one you expect.

7.2.22 Data Class

What it is. A class with only fields and getters/setters (setters are methods that just overwrite a field), and no behaviour. Other classes manipulate its data in detail.

Why it hurts. The behaviour that belongs with the data lives elsewhere (usually as Feature Envy in its clients), often duplicated across several clients. Public setters also allow anyone to break the object's invariants (rules that must always hold, e.g. "balance is never negative").

Important exceptions. Immutable data carriers are fine and often good: a Java **record** that is the *result* of a computation (e.g. the output of Split Phase), a DTO (Data Transfer Object) crossing a network boundary, or a value object. The smell is about *mutable* data classes whose logic is scattered around them.

```
// Smelly
class Account { private double balance; public double getBalance() {...} public void setBalance(double b) {...} }
// ...and in a service:
if (acc.getBalance() >= amt) acc.setBalance(acc.getBalance() - amt); else throw ...;

// Cured: Remove Setting Method + Move Function - the rule lives with the data
class Account {
    private double balance;
    void withdraw(double amount) {
        if (amount > balance) throw new IllegalStateException("insufficient funds");
        balance -= amount;
    }
}
```

Refactorings. Encapsulate Record, Remove Setting Method (for fields that should not change), Move Function and Extract Function (bring behaviour in from the clients), Split Phase (when the data class is the result of a phase — then it is fine as an immutable record). Also useful (not in Fowler's list for this smell): Encapsulate Collection for public collection fields.

7.2.23 Refused Bequest

What it is. A subclass inherits methods and data from its parent but does not want or need them — or worse, refuses the parent's *interface* (overrides methods to throw `UnsupportedOperationException`).

Why it hurts. The inheritance relationship is wrong. Refusing *implementation* is a mild smell (Fowler says it is often not worth fixing). Refusing the *interface* is serious: it breaks the **Liskov Substitution Principle** (Chapter 4: a subclass must work anywhere its parent is expected) — code that uses the parent type will fail with this subclass.

```
// Smelly: Stack extends a list type, and so inherits add(index, e), remove(index) ... which break stack rules
class Stack<T> extends ArrayList<T> {
    void push(T t) { add(t); }
    T pop() { return remove(size() - 1); }
}
// (java.util.Stack extends Vector - the JDK's own famous example of this smell.)

// Cured: Replace Superclass with Delegate - use a list, don't be one
final class Stack<T> {
    private final Deque<T> items = new ArrayDeque<>();
    void push(T t) { items.push(t); }
    T pop()        { return items.pop(); }
}
// Behaviour check: pop() on an empty stack now throws NoSuchElementException instead of
// IndexOutOfBoundsException, and ArrayDeque rejects null elements. If callers rely on either,
// keep the old behaviour (e.g. an ArrayList delegate) - a refactoring must not change it.
```

Refactorings. Push Down Method and Push Down Field (move unused parent members into a sibling that does use them, leaving the parent with only what is shared), Replace Subclass with Delegate, Replace Superclass with Delegate.

7.2.24 Comments

What it is. Comments are not bad in themselves — Fowler calls them a *sweet* smell. The smell is comments used as **deodorant**: a long comment explaining *what* a confusing block does, when the block should have been made clear instead.

Why it hurts. Comments go stale — the code changes and the comment does not. A comment that explains *what* code does is a sign the code does not explain itself.

Good comments (keep them): *why* something is done a particular way, a warning, a reference to an algorithm or bug ticket, a note about something you are unsure of, public API documentation (Javadoc).

```
// Smelly
// check if the customer is eligible for the senior discount
if (customer.getAge() >= 60 && customer.getMembershipYears() > 2 && !customer.isBlocked()) { ... }

// Cured: Extract Function - the name replaces the comment
if (isEligibleForSeniorDiscount(customer)) { ... }

// Smelly: comment states an assumption
// rate must never be negative here
// Cured: Introduce Assertion
assert rate >= 0 : "rate must not be negative";
```

Refactorings. Extract Function (comment explains a block), Change Function Declaration (comment explains what a function does — rename it), Introduce Assertion (comment states a required condition).

1st-edition smell names (renamed or dropped in the 2nd edition)

refactoring.guru still uses the 1st-edition (1999) names. You may hear them in interviews:

1st-edition smell	Meaning	Where it went / fix
Long Method	Method too long to understand	Renamed **Long Function**
Switch Statements	Any complex switch	Narrowed to **Repeated Switches**
Lazy Class	Class that does too little	Generalised to **Lazy Element**
Inappropriate Intimacy	Classes too involved in each other's privates	Renamed **Insider Trading**
Parallel Inheritance Hierarchies	Every time you add a subclass in one hierarchy you must add one in another	A special case of Shotgun Surgery; fix with Move Function / Move Field so one hierarchy disappears
Incomplete Library Class	A library lacks a method you need and you cannot change it	Add a wrapper or utility (Introduce Foreign Method / Introduce Local Extension in the 1st ed.), or an Adapter
Dead Code	Unused variables, parameters, methods, classes	The 2nd ed. treats it as the refactoring **Remove Dead Code**

7.3 The refactoring catalogue

What it is

A **named refactoring** is a small, well-defined transformation with a name, a *motivation* (when and why to use it), *mechanics* (the safe steps), and examples. Fowler's 2nd edition catalogues 61 of them in seven chapters (Chapters 6–12 of the book). The same catalogue, with a few extra entries, is online at refactoring.com/catalog. Many 1st-edition names (1999) were changed to be language-neutral ("function" instead of "method") — the old names are shown as *aliases*.

Many refactorings come in **inverse pairs** (Extract Function ↔ Inline Function). There is no "right direction" — you move whichever way makes the code clearer *for the change you are about to make*.

How to read each entry

- **Motivation** — when and why.
- **Before → After** — a short Java sketch. `// ...` marks irrelevant code. Sketches are illustrative fragments; the fully compiling examples are in `code/` (7.5, 7.6, 7.8).
- **Inverse** — the opposite refactoring, if one exists.

Catalogue at a glance

Book chapter	Refactorings
6 — A First Set	Extract Function, Inline Function, Extract Variable, Inline Variable, Change Function Declaration, Encapsulate Variable, Rename Variable, Introduce Parameter Object, Combine Functions into Class, Combine Functions into Transform, Split Phase
7 — Encapsulation	Encapsulate Record, Encapsulate Collection, Replace Primitive with Object, Replace Temp with Query, Extract Class, Inline Class, Hide Delegate, Remove Middle Man, Substitute Algorithm
8 — Moving Features	Move Function, Move Field, Move Statements into Function, Move Statements to Callers, Replace Inline Code with Function Call, Slide Statements, Split Loop, Replace Loop with Pipeline, Remove Dead Code
9 — Organizing Data	Split Variable, Rename Field, Replace Derived Variable with Query, Change Reference to Value, Change Value to Reference
10 — Simplifying Conditional Logic	Decompose Conditional, Consolidate Conditional Expression, Replace Nested Conditional with Guard Clauses, Replace Conditional with Polymorphism, Introduce Special Case, Introduce Assertion
11 — Refactoring APIs	Separate Query from Modifier, Parameterize Function, Remove Flag Argument, Preserve Whole Object, Replace Parameter with Query, Replace Query with Parameter, Remove Setting Method, Replace Constructor with Factory Function, Replace Function with Command, Replace Command with Function
12 — Dealing with Inheritance	Pull Up Method, Pull Up Field, Pull Up Constructor Body, Push Down Method, Push Down Field, Replace Type Code with Subclasses, Remove Subclass, Extract Superclass, Collapse Hierarchy, Replace Subclass with Delegate, Replace Superclass with Delegate
refactoring.com extras (web edition)	Replace Control Flag with Break, Replace Error Code with Exception, Replace Exception with Precheck, Replace Magic Literal, Return Modified Value

11 + 9 + 9 + 5 + 6 + 10 + 11 = **61** book refactorings.

7.3.1 A First Set of Refactorings (book chapter 6)

These are the refactorings you will use most — every day.

Extract Function *(alias: Extract Method)* · inverse: Inline Function

Motivation. Separate *intention* from *implementation*. If you have to spend effort looking at a fragment of code to work out *what* it does, extract it into a function named after that "what". Even a one-line function is worth it if the name says more than the code. Short functions are cheap in Java (the JIT inlines them).

```
// Before
void printOwing(Invoice invoice) {
    double outstanding = invoice.outstanding();
    System.out.println("name: " + invoice.customer());
    System.out.println("amount: " + outstanding);
}
// After
void printOwing(Invoice invoice) {
    printDetails(invoice, invoice.outstanding());
}
private void printDetails(Invoice invoice, double outstanding) {
    System.out.println("name: " + invoice.customer());
    System.out.println("amount: " + outstanding);
}
```

Inline Function *(alias: Inline Method)* · inverse: Extract Function

Motivation. When a function's body is as clear as its name, or when you have a tangle of badly factored small functions, put them back together — then extract again in better places. Also useful to remove needless indirection (a function that only delegates).

```
// Before
int rating(Driver d) { return moreThanFiveLateDeliveries(d) ? 2 : 1; }
boolean moreThanFiveLateDeliveries(Driver d) { return d.lateDeliveries() > 5; }
// After
int rating(Driver d) { return d.lateDeliveries() > 5 ? 2 : 1; }
```

Extract Variable ***(alias: Introduce Explaining Variable)*** **· inverse: Inline Variable**

Motivation. Complex expressions are hard to read. Name the parts with local variables. If the name makes sense beyond this function, prefer Extract Function (or Replace Temp with Query) instead so others can reuse it.

```
// Before
return order.quantity() * order.itemPrice()
     - Math.max(0, order.quantity() - 500) * order.itemPrice() * 0.05
     + Math.min(order.quantity() * order.itemPrice() * 0.1, 100);
// After
double basePrice = order.quantity() * order.itemPrice();
double quantityDiscount = Math.max(0, order.quantity() - 500) * order.itemPrice() * 0.05;
double shipping = Math.min(basePrice * 0.1, 100);
return basePrice - quantityDiscount + shipping;
```

Inline Variable ***(alias: Inline Temp)*** **· inverse: Extract Variable**

Motivation. A variable whose name says no more than the expression, or which gets in the way of another refactoring (e.g. Extract Function), should be removed.

```
// Before
double basePrice = order.basePrice();
return basePrice > 1000;
// After
return order.basePrice() > 1000;
```

Change Function Declaration ***(aliases: Rename Function, Rename Method, Add Parameter, Remove Parameter, Change Signature)***

Motivation. Function names and parameters are the joints of the system; bad ones make everything harder. Rename a function the moment you understand it better; add or remove parameters to change what the function depends on. Two ways: the **simple mechanics** (change the declaration and all callers at once — fine when you can see all callers, e.g. with the IDE) and the **migration mechanics** (create the new function, make the old one delegate to it and mark it `@Deprecated`, migrate callers gradually, then delete the old one — needed for published APIs).

```
// Before
double circum(double radius) { return 2 * Math.PI * radius; }
// After (migration mechanics)
double circumference(double radius) { return 2 * Math.PI * radius; }
@Deprecated(forRemoval = true)
double circum(double radius) { return circumference(radius); }   // delete when no callers remain
```

Encapsulate Variable ***(aliases: Encapsulate Field, Self-Encapsulate Field)***

Motivation. Data is harder to refactor than functions, because every reference must change at once. Wrapping widely accessed data in accessor functions (getter/setter methods) turns "reorganise data" into "reorganise functions" and gives you one place to add validation, logging or change notifications. Always the first step for Global Data.

```
// Before
public static Owner defaultOwner = new Owner("Martin", "Fowler");
// After
private static Owner defaultOwner = new Owner("Martin", "Fowler");
public static Owner defaultOwner() { return defaultOwner; }
public static void setDefaultOwner(Owner owner) { defaultOwner = owner; }   // add checks later, as a separate
change
```

Rename Variable

Motivation. Good names are the heart of clear code. The wider the scope of a variable, the more its name matters. A short name is fine in a three-line lambda; a field needs a descriptive name.

```
// Before
double a = height * width;
// After
double area = height * width;
```

Introduce Parameter Object

Motivation. A group of parameters that always travel together (a Data Clump) becomes one object. Signatures shrink, the relationship between the values is made explicit, and behaviour about them now has a home — often the start of a useful domain class.

```
// Before
List<Reading> readingsOutsideRange(Station s, int min, int max) { ... }
// After
record NumberRange(int min, int max) {
    boolean contains(int v) { return v >= min && v <= max; }
}
List<Reading> readingsOutsideRange(Station s, NumberRange range) { ... }
```

Combine Functions into Class

Motivation. When several functions work on the same data (often passed as the same argument to all of them), create a class that holds the data as fields and turns the functions into methods. The shared data and the common environment become explicit, and clients no longer pass the data around.

```
// Before
double base(Reading r) { ... }
double taxableCharge(Reading r) { ... }
double calculateBaseCharge(Reading r) { ... }
// After
final class ReadingCalculator {
    private final Reading reading;
    ReadingCalculator(Reading reading) { this.reading = reading; }
    double base() { ... }
    double taxableCharge() { ... }
}
```

Combine Functions into Transform

Motivation. An alternative to Combine Functions into Class when the source data is read-only: a *transform* function takes the raw data and returns an enriched copy with all derived values computed. All derivation logic lives in one place. (Use a class instead if the source data can change, since a transform's copy would go stale.)

```
// Before (derived values computed in several places)
double base = baseRate(r.month(), r.year()) * r.quantity();
double taxable = Math.max(0, base - taxThreshold(r.year()));
// After
record EnrichedReading(Reading raw, double baseCharge, double taxableCharge) { }
static EnrichedReading enrichReading(Reading r) {
    double base = baseRate(r.month(), r.year()) * r.quantity();
    return new EnrichedReading(r, base, Math.max(0, base - taxThreshold(r.year())));
}
```

Split Phase

Motivation. When code deals with two different things in sequence (e.g. parse input, then compute; or compute, then format), split it into two phases connected by an intermediate data structure. Each phase can then be understood and changed separately — a classic cure for Divergent Change. Compilers are the textbook example

(tokenise — split text into words and symbols → parse — build a tree of its structure → generate code). See the worked example, step 3.

```
// Before
String priceOrder(String orderLine) {
    String[] parts = orderLine.split("\\s+");
    Product p = catalog.get(parts[0].split("-")[1]);
    int qty = Integer.parseInt(parts[1]);
    return "Total: " + qty * p.price();
}
// After
record OrderRequest(String productId, int quantity) { }           // the intermediate data structure
OrderRequest parse(String line) {                                  // phase 1
    String[] parts = line.split("\\s+");
    return new OrderRequest(parts[0].split("-")[1], Integer.parseInt(parts[1]));
}
String price(OrderRequest req) {                                   // phase 2
    return "Total: " + req.quantity() * catalog.get(req.productId()).price();
}
String priceOrder(String orderLine) { return price(parse(orderLine)); }
```

7.3.2 Encapsulation (book chapter 7)

The key to modular design is deciding what each module *hides*. These refactorings hide data structures, temporary variables, and relationships between classes.

Encapsulate Record ***(alias: Replace Record with Data Class)***

Motivation. A raw record-like structure (a `Map<String,Object>`, an array, a struct with public fields) forces clients to know its layout and lets anyone change it. Replace it with a class (or a Java `record`) whose accessors hide the storage and can compute derived values.

```
// Before
Map<String, Object> organization = Map.of("name", "Acme", "country", "GB");
String name = (String) organization.get("name");
// After
record Organization(String name, String country) { }
Organization organization = new Organization("Acme", "GB");
String name = organization.name();
```

Encapsulate Collection

Motivation. If a getter returns the internal collection, clients can modify it without the owner knowing. Offer `add`/`remove` methods on the owner, and have the getter return a read-only view or a copy.

```
// Before
class Person {
    private List<Course> courses = new ArrayList<>();
    List<Course> getCourses() { return courses; }                  // caller can add/clear!
    void setCourses(List<Course> c) { courses = c; }
}
// After
class Person {
    private final List<Course> courses = new ArrayList<>();
    List<Course> courses()        { return Collections.unmodifiableList(courses); }
    void addCourse(Course c)      { courses.add(c); }
    void removeCourse(Course c)   { courses.remove(c); }
}
```

Replace Primitive with Object ***(aliases: Replace Data Value with Object, Replace Type Code with Class)***

Motivation. A simple value (a phone number string, a priority `"high"`) soon needs behaviour: validation, formatting, comparison. Wrap it in its own type as soon as you see that. A cure for Primitive Obsession.

```
// Before
List<Order> urgent = orders.stream().filter(o -> o.priority().equals("high") ||
o.priority().equals("rush")).toList();
// After
enum Priority { LOW, NORMAL, HIGH, RUSH;
    boolean higherThan(Priority other) { return compareTo(other) > 0; } }
List<Order> urgent = orders.stream().filter(o -> o.priority().higherThan(Priority.NORMAL)).toList();
```

Replace Temp with Query

Motivation. A temporary variable holding a computed value can only be used inside its function. Turn the computation into a method, and every other method of the class can use it too; it also makes Extract Function easier (fewer locals to pass). Works best when the temp is assigned once and its computation has no side effects.

```
// Before
double price() {
    double basePrice = quantity * itemPrice;
    return basePrice > 1000 ? basePrice * 0.95 : basePrice * 0.98;
}
// After
double price()     { return basePrice() > 1000 ? basePrice() * 0.95 : basePrice() * 0.98; }
double basePrice() { return quantity * itemPrice; }
```

Extract Class · inverse: Inline Class

Motivation. Classes grow. When a subset of fields and methods clearly go together (and change together), move them to a new class. A cure for Large Class and Divergent Change.

```
// Before
class Person {
    String name; String officeAreaCode; String officeNumber;
    String telephoneNumber() { return "(" + officeAreaCode + ") " + officeNumber; }
}
// After
class Person {
    String name; TelephoneNumber officeTelephone;
    String telephoneNumber() { return officeTelephone.toString(); }
}
record TelephoneNumber(String areaCode, String number) {
    @Override public String toString() { return "(" + areaCode + ") " + number; }
}
```

Inline Class · inverse: Extract Class

Motivation. A class that no longer does enough (often after its responsibilities moved away) should be folded into its main user. Also a useful first step when re-splitting two classes differently: merge them, then extract again.

```
// Before
class Shipment { TrackingInformation trackingInfo; }
class TrackingInformation { String shippingCompany; String trackingNumber; }
// After
class Shipment { String shippingCompany; String trackingNumber; }
```

Hide Delegate · inverse: Remove Middle Man

Motivation. If a client calls `a.getB().doSomething()`, it depends on `B`. Adding a delegating method `a.doSomething()` hides `B`; if `B` changes, only `A` changes. A cure for Message Chains and the Law of Demeter.

```
// Before
Person manager = person.getDepartment().getManager();
// After
class Person { Person manager() { return department.getManager(); } }
Person manager = person.manager();
```

Remove Middle Man · inverse: Hide Delegate

Motivation. When a class has so many delegating methods that it is just a pass-through, let clients call the delegate directly. The right amount of hiding changes as the system changes.

```
// Before
Person manager = person.manager();              // manager() only forwards
// After
Person manager = person.department().getManager();
```

Substitute Algorithm

Motivation. If you find a clearer way to do something, replace the whole body with the clearer algorithm (after decomposing the old one as far as possible, and with tests proving they agree). Also used to replace hand-written code with a library call.

```
// Before
String foundPerson(List<String> people) {
    for (String p : people) {
        if (p.equals("Don")) return "Don";
        if (p.equals("John")) return "John";
        if (p.equals("Kent")) return "Kent";
    }
    return "";
}
// After
String foundPerson(List<String> people) {
    Set<String> candidates = Set.of("Don", "John", "Kent");
    return people.stream().filter(candidates::contains).findFirst().orElse("");
}
```

7.3.3 Moving Features (book chapter 8)

These refactorings move code between contexts — between classes, between functions, and around inside a function.

Move Function *(alias: Move Method)*

Motivation. Put a function in the context (class/module) whose data and functions it references most. A cure for Feature Envy and Shotgun Surgery. Good modules keep related things together.

```
// Before
class Account {
    double overdraftCharge() {
        if (type.isPremium()) { double base = 10; return daysOverdrawn > 7 ? base + (daysOverdrawn - 7) * 0.85 :
base; }
        return daysOverdrawn * 1.75;
    }
}
// After - the rule depends mostly on AccountType, so it moves there
class AccountType {
    double overdraftCharge(int daysOverdrawn) { /* same logic, using isPremium() */ }
}
class Account { double overdraftCharge() { return type.overdraftCharge(daysOverdrawn); } }
```

Move Field

Motivation. Data structures are the backbone of a program. If you always pass a field of A alongside B, or a change in B forces a change to A's field, the field probably belongs in B. Encapsulate it first (Encapsulate Variable) to make the move easy.

```
// Before
class Customer { Plan plan; double discountRate; }
class Plan { }
// After - the discount rate is a property of the plan, not of each customer
class Customer { Plan plan; double discountRate() { return plan.discountRate(); } }
class Plan { double discountRate; double discountRate() { return discountRate; } }
```

Move Statements into Function · inverse: Move Statements to Callers

Motivation. If the same statements run every time a function is called, move them into the function, removing duplication at every call site.

```
// Before
result.add("<p>title: " + photo.title() + "</p>");
result.addAll(photoData(photo));
List<String> photoData(Photo p) { return List.of("<p>location: " + p.location() + "</p>"); }
// After
result.addAll(photoData(photo));
List<String> photoData(Photo p) {
    return List.of("<p>title: " + p.title() + "</p>", "<p>location: " + p.location() + "</p>");
}
```

Move Statements to Callers · inverse: Move Statements into Function

Motivation. When a function that was shared starts needing *different* behaviour for different callers, move the varying statements out to the callers.

```
// Before
void emitPhotoData(Writer out, Photo p) { out.write(p.title()); out.write(p.location()); }
// After - the location line moves out to every caller; behaviour is still identical
void emitPhotoData(Writer out, Photo p) { out.write(p.title()); }
// caller A:
emitPhotoData(out, p); out.write(p.location());
// caller B:
emitPhotoData(out, p); out.write(p.location());
// Now caller B can change its own line later (e.g. hide the location) - as a separate, deliberate behaviour
change.
```

Replace Inline Code with Function Call

Motivation. If code does what an existing function (yours or a library's) already does, call the function. You gain a name and one place to fix bugs.

```
// Before
boolean appliesToMass = false;
for (String s : states) { if (s.equals("MA")) appliesToMass = true; }
// After
boolean appliesToMass = states.contains("MA");
```

Slide Statements *(alias: Consolidate Duplicate Conditional Fragments)*

Motivation. Code is easier to understand when related statements are next to each other — e.g. declare a variable right before its first use. Sliding is often a preparation for Extract Function. The 1st-edition alias covers one case: a statement repeated in every branch of a conditional can slide out of the conditional. Be careful: you cannot slide past code that the statement interferes with (shared variables, side effects).

```
// Before
String pricingPlan = retrievePricingPlan();
Order order = retrieveOrder();
double baseCharge = pricingPlan.base();
// After - declare each value next to its use
Order order = retrieveOrder();
String pricingPlan = retrievePricingPlan();
double baseCharge = pricingPlan.base();
```

Split Loop

Motivation. A loop that does two things forces you to understand both whenever you change either. Split it into two loops, each doing one thing; then each can be extracted or turned into a pipeline. Worried about performance? Iterating twice is rarely a bottleneck — refactor first, optimise later if the profiler says so.

```
// Before
int youngest = Integer.MAX_VALUE; double totalSalary = 0;
for (Person p : people) {
    if (p.age() < youngest) youngest = p.age();
    totalSalary += p.salary();
}
// After
double totalSalary = 0;
for (Person p : people) totalSalary += p.salary();
int youngest = Integer.MAX_VALUE;
for (Person p : people) if (p.age() < youngest) youngest = p.age();
```

Replace Loop with Pipeline

Motivation. Collection pipelines (`filter`, `map`, `reduce`) describe the processing as a series of named operations, which is easier to read than a loop. See the Loops smell.

```
// Before
List<String> result = new ArrayList<>();
for (String line : lines) {
    if (line.isBlank() || line.startsWith("#")) continue;
    result.add(line.split(",")[1].trim());
}
// After
List<String> result = lines.stream()
        .filter(l -> !l.isBlank() && !l.startsWith("#"))
        .map(l -> l.split(",")[1].trim())
        .toList();
```

Remove Dead Code

Motivation. Unused code still costs readers time ("is this called somewhere?"). Delete it — version control remembers it if you ever need it back. Do not comment it out.

```
// Before
if (false) { legacyDiscount(); }
private void legacyDiscount() { /* unused since 2019 */ }
// After
// (gone)
```

7.3.4 Organizing Data (book chapter 9)

Data structures play an important role; these refactorings improve how data is named and shared.

Split Variable ***(aliases: Remove Assignments to Parameters, Split Temp)***

Motivation. A variable (other than a loop counter or an accumulator) assigned more than once usually has more than one responsibility. Give each responsibility its own variable (ideally `final`). The 1st-ed "Remove Assignments to Parameters" is a special case: do not reuse a parameter as a local.

```java
// Before
double temp = 2 * (height + width);
System.out.println(temp);
temp = height * width;
System.out.println(temp);
// After
final double perimeter = 2 * (height + width);
System.out.println(perimeter);
final double area = height * width;
System.out.println(area);
```

Rename Field

Motivation. Field names explain the data structure; they deserve as much care as function names. Encapsulate first if the field is widely used, then rename. (For records, the accessor name changes too.)

```java
// Before
record Organization(String nm, String country) { }
// After
record Organization(String name, String country) { }
```

Replace Derived Variable with Query

Motivation. Mutable data that is *computed from* other data can drift out of sync. Compute it on demand instead. Exception: if the source data is immutable, a stored derived value is safe (e.g. a record computed once).

```java
// Before
class ProductionPlan {
    private double production;
    private final List<Adjustment> adjustments = new ArrayList<>();
    void apply(Adjustment a) { adjustments.add(a); production += a.amount(); }  // two updates to keep in sync
    double production() { return production; }
}
// After
class ProductionPlan {
    private final List<Adjustment> adjustments = new ArrayList<>();
    void apply(Adjustment a) { adjustments.add(a); }
    double production() { return adjustments.stream().mapToDouble(Adjustment::amount).sum(); }
}
```

Change Reference to Value · inverse: Change Value to Reference

Motivation. If an inner object does not need identity (being tracked as one specific object) (two telephone numbers with the same digits are "the same"), make it an immutable **value object**: replace it wholesale instead of mutating it, and compare by value (`equals` / `hashCode`). Values are safe to share and to use across threads. Java `record`s are ideal.

```java
// Before
class TelephoneNumber { String areaCode; String number; void setAreaCode(String a) { areaCode = a; } }
person.telephone().setAreaCode("020");
// After
record TelephoneNumber(String areaCode, String number) { }                 // immutable, equals by value
person.setTelephone(new TelephoneNumber("020", person.telephone().number()));
```

Change Value to Reference · inverse: Change Reference to Value

Motivation. If many objects hold *copies* of the same logical entity (every order holds its own `Customer` copy) and that entity needs to be updated, a change to one copy is not seen by the others. Share **one** reference instead, usually via a **repository** (an object that stores domain objects and looks them up, usually by ID) that hands out the single instance per ID.

```
// Before
Order(String customerId) { this.customer = new Customer(customerId); }      // each order its own copy
// After
Order(String customerId, CustomerRepository repo) { this.customer = repo.findOrCreate(customerId); }
```

7.3.5 Simplifying Conditional Logic (book chapter 10)

Conditionals are where much of a program's complexity hides.

Decompose Conditional

Motivation. A complex `if`/`else` mixes *why* (the condition) and *what* (each branch). Extract the condition and each branch into well-named functions. A special case of Extract Function.

```
// Before
if (!date.isBefore(plan.summerStart()) && !date.isAfter(plan.summerEnd()))
    charge = quantity * plan.summerRate();
else
    charge = quantity * plan.regularRate() + plan.regularServiceCharge();
// After
charge = isSummer(date) ? summerCharge() : regularCharge();
```

Consolidate Conditional Expression

Motivation. Several checks that all lead to the same result are really one condition. Combine them with `&&`/`||` and extract the result into a named function. (Only when the checks have no side effects and truly form one decision.)

```
// Before
if (employee.seniority() < 2) return 0;
if (employee.monthsDisabled() > 12) return 0;
if (employee.isPartTime()) return 0;
// After
if (isNotEligibleForDisability(employee)) return 0;
boolean isNotEligibleForDisability(Employee e) {
    return e.seniority() < 2 || e.monthsDisabled() > 12 || e.isPartTime();
}
```

Replace Nested Conditional with Guard Clauses

Motivation. When one branch is the normal case and the others are unusual, check the unusual cases first and return immediately (**guard clauses**). The normal path then reads straight down, without deep nesting. ("One exit point" is not a rule worth keeping if it hurts clarity.)

```
// Before
double payAmount(Employee e) {
    double result;
    if (e.isSeparated()) result = 0;
    else { if (e.isRetired()) result = 0; else { result = normalPay(e); } }
    return result;
}
// After
double payAmount(Employee e) {
    if (e.isSeparated()) return 0;
    if (e.isRetired()) return 0;
    return normalPay(e);
}
```

Replace Conditional with Polymorphism

Motivation. When a switch on a type appears in several places (Repeated Switches), or when there is a base behaviour with variants, create a class per case and let dynamic dispatch (the runtime choosing which class's method to run from the object's actual type) pick the behaviour. Adding a new case means adding a class, not

editing every switch. This is how you arrive at **Strategy** or **State** (7.6) — State gives each state of an object its own class, which supplies the behaviour for that state. The worked example's step 5 does exactly this.

```
// Before
double speed(Bird b) {
    return switch (b.type()) {
        case "EUROPEAN" -> 35;
        case "AFRICAN"  -> 40 - 2 * b.numberOfCoconuts();
        case "PARROT"   -> b.isNailed() ? 0 : 10 + b.voltage() / 10.0;
        default -> throw new IllegalArgumentException(b.type());
    };
}
// After
sealed interface Bird permits EuropeanSwallow, AfricanSwallow, NorwegianBlueParrot { double speed(); }
record EuropeanSwallow() implements Bird { public double speed() { return 35; } }
record AfricanSwallow(int numberOfCoconuts) implements Bird { public double speed() { return 40 - 2 *
numberOfCoconuts; } }
record NorwegianBlueParrot(boolean isNailed, int voltage) implements Bird {
    public double speed() { return isNailed ? 0 : 10 + voltage / 10.0; }
}
```

Introduce Special Case ***(alias: Introduce Null Object)***

Motivation. Many clients check for the same special value (usually `null`, or an "unknown customer") and do the same thing. Create a **special-case object** that answers those questions itself. The **Null Object** pattern is the most common form. See 7.6 for a runnable example.

```
// Before
String name = (customer == null) ? "occupant" : customer.name();
Plan plan  = (customer == null) ? Plan.BASIC : customer.plan();
// After
final class UnknownCustomer implements Customer {
    public String name() { return "occupant"; }
    public Plan plan()   { return Plan.BASIC; }
}
String name = customer.name();          // repository returns UnknownCustomer instead of null
```

(Java's `Optional<T>` is a related tool: it makes "may be absent" visible in the type, but callers still decide what to do. A special case object *centralises* that decision.)

Introduce Assertion

Motivation. Some code only works if a condition holds (e.g. a discount rate is never negative). Make the assumption explicit with an assertion. An assertion documents the assumption and catches programmer errors early; the program must be correct *without* it. In Java, `assert` is disabled by default (enable with `java -ea`), so do not use it to validate external input — use exceptions for that.

```
// Before
double applyDiscount(double amount) {
    // discountRate should never be negative here
    return amount - discountRate * amount;
}
// After
double applyDiscount(double amount) {
    assert discountRate >= 0 : "discountRate must not be negative";
    return amount - discountRate * amount;
}
```

7.3.6 Refactoring APIs (book chapter 11)

Modules and their functions are the building blocks; APIs are the joints. These make APIs easier to understand and use.

Separate Query from Modifier

Motivation. A function that returns a value should have no observable side effects — then you can call it anywhere, as often as you like, safely. This is **Command–Query Separation** (Bertrand Meyer). If a function both returns something and changes state, split it into a query and a modifier.

```
// Before
String alertForMiscreant(List<String> people) {
    for (String p : people) {
        if (p.equals("Don") || p.equals("John")) { setOffAlarms(); return p; }
    }
    return "";
}
// After
String findMiscreant(List<String> people) {                        // query - no side effects
    return people.stream().filter(p -> p.equals("Don") || p.equals("John")).findFirst().orElse("");
}
void alertForMiscreant(List<String> people) {                       // modifier
    if (!findMiscreant(people).isEmpty()) setOffAlarms();
}
```

Parameterize Function ***(alias: Parameterize Method)***

Motivation. Several functions with very similar logic that differ only in some literal values can become one function with a parameter.

```
// Before
void tenPercentRaise(Person p)  { p.setSalary(p.salary() * 1.10); }
void fivePercentRaise(Person p) { p.setSalary(p.salary() * 1.05); }
// After
void raise(Person p, double factor) { p.setSalary(p.salary() * (1 + factor)); }
```

Remove Flag Argument ***(alias: Replace Parameter with Explicit Methods)***

Motivation. A **flag argument** (a boolean or enum literal that the caller passes to choose which code path runs) makes calls unreadable: what does `book(customer, true)` mean? Provide an explicit function per path.

```
// Before
void setDimension(String name, double value) { if (name.equals("height")) height = value; else if
(name.equals("width")) width = value; }
LocalDate deliveryDate(Order o, boolean isRush) { ... }
deliveryDate(order, true);
// After
void setHeight(double value) { height = value; }
void setWidth(double value)  { width = value; }
LocalDate rushDeliveryDate(Order o)    { ... }
LocalDate regularDeliveryDate(Order o) { ... }
```

Preserve Whole Object

Motivation. If you pull several values out of one object only to pass them to a function, pass the whole object. The function can then take whatever it needs, and signatures stay stable when needs change. (Don't do it if you want to keep the function independent of that object's type.)

```
// Before
int low = room.daysTempRange().low();
int high = room.daysTempRange().high();
if (!plan.withinRange(low, high)) alerts.add("room temperature out of range");
// After
if (!plan.withinRange(room.daysTempRange())) alerts.add("room temperature out of range");
```

Replace Parameter with Query ***(alias: Replace Parameter with Method)*** **· inverse: Replace Query with Parameter**

Motivation. If the function can easily get a value itself (from its own object, or from another parameter), the caller should not have to pass it. Removing the parameter simplifies every call. (Don't do it if it would add an

unwanted dependency.)

```
// Before
double finalPrice() { return discountedPrice(basePrice(), discountLevel()); }
double discountedPrice(double base, int discountLevel) { return discountLevel == 2 ? base * 0.9 : base * 0.95; }
// After
double finalPrice() { return discountedPrice(basePrice()); }
double discountedPrice(double base) { return discountLevel() == 2 ? base * 0.9 : base * 0.95; }
```

Replace Query with Parameter · inverse: Replace Parameter with Query

Motivation. If a function reaches out to a global, a singleton or a mutable field, it is harder to test and not **referentially transparent** (same inputs do not always give the same output). Pass the value in as a parameter instead — the caller takes on the dependency.

```
// Before
double targetTemperature(HeatingPlan plan) {
    double current = Thermostat.INSTANCE.currentTemperature();        // hidden global dependency
    return Math.clamp(current, plan.min(), plan.max());
}
// After
double targetTemperature(HeatingPlan plan, double currentTemperature) {
    return Math.clamp(currentTemperature, plan.min(), plan.max());      // Math.clamp: Java 21+
}
```

Remove Setting Method

Motivation. If a field should not change after construction, remove its setter and set it in the constructor (make it `final`). The class then clearly communicates immutability.

```
// Before
class Person { private String id; void setId(String id) { this.id = id; } }
// After
class Person { private final String id; Person(String id) { this.id = id; } String id() { return id; } }
```

Replace Constructor with Factory Function *(alias: Replace Constructor with Factory Method)*

Motivation. Constructors have limitations: they must return a new instance of exactly that class, their name is fixed, and `new` is needed. A static factory method (a `static` method that creates and returns an instance) can have an intention-revealing name, return a subclass or cached instance, and hide construction details. (Joshua Bloch's *Effective Java*, Item 1, makes the same argument. JDK examples: `List.of`, `Integer.valueOf`, `LocalDate.of`.)

```
// Before
Employee leadEngineer = new Employee(document.leadEngineer(), "E");
// After
static Employee createEngineer(String name) { return new Employee(name, "E"); }
Employee leadEngineer = Employee.createEngineer(document.leadEngineer());
```

Replace Function with Command *(alias: Replace Method with Method Object)* · inverse: Replace Command with Function

Motivation. Wrap a function in its own object (a **command**, with usually one `execute()` method). The object can hold state across steps, support undo, be queued, and — most commonly in refactoring — lets you break a big function with many local variables into small methods that share those variables as fields.

```
// Before
int score(Candidate c, MedicalExam exam, ScoringGuide guide) {
    int result = 0; int healthLevel = 0; boolean highMedicalRisk = false;
    // ... 60 lines using result, healthLevel, highMedicalRisk
    return result;
}
// After
final class Scorer {
    private final Candidate candidate; private final MedicalExam exam; private final ScoringGuide guide;
    private int result; private int healthLevel; private boolean highMedicalRisk;
    Scorer(Candidate c, MedicalExam e, ScoringGuide g) { candidate = c; exam = e; guide = g; }
    int execute() { scoreSmoking(); scoreCertification(); return result - Math.max(healthLevel - 5, 0); }
    private void scoreSmoking() { /* uses fields, no parameter passing */ }
    private void scoreCertification() { /* ... */ }
}
int score(Candidate c, MedicalExam exam, ScoringGuide guide) { return new Scorer(c, exam, guide).execute(); }
```

Replace Command with Function · inverse: Replace Function with Command

Motivation. If a command object is simple and adds nothing a function couldn't do, turn it back into a function. Commands are powerful but cost extra complexity.

```
// Before
final class ChargeCalculator {
    private final Customer customer; private final int usage;
    ChargeCalculator(Customer c, int usage) { this.customer = c; this.usage = usage; }
    double charge() { return customer.rate() * usage; }
}
double monthCharge = new ChargeCalculator(customer, usage).charge();
// After
static double charge(Customer customer, int usage) { return customer.rate() * usage; }
double monthCharge = charge(customer, usage);
```

7.3.7 Dealing with Inheritance (book chapter 12)

Inheritance is powerful and easy to misuse. These refactorings move features up and down a hierarchy and replace inheritance with delegation (composition — one object holding another and forwarding work to it) where needed.

Pull Up Method · inverse: Push Down Method

Motivation. Duplicate methods in sibling subclasses belong in the superclass. If they are similar but not identical, first make them identical (Parameterize Function), or use a Template Method (7.6) — a base-class method that fixes an algorithm's steps and lets subclasses fill some in.

```
// Before
class Salesman extends Employee { String name() { return name; } }
class Engineer extends Employee { String name() { return name; } }
// After
class Employee { String name() { return name; } }
```

Pull Up Field · inverse: Push Down Field

Motivation. Subclasses developed independently often have the same field (maybe with different names). Rename to match, then move the field to the superclass.

```
// Before
class Salesman extends Employee { protected String name; }
class Engineer extends Employee { protected String name; }
// After
class Employee { protected String name; }
```

Pull Up Constructor Body

Motivation. Constructors in subclasses that repeat the same initialisation should call a superclass constructor that does it. In Java 21 `super(...)` must be the first statement of a constructor (Java 25 relaxed this with flexible constructor bodies, JEP 513 — a JDK Enhancement Proposal), so the common code goes into the superclass constructor.

```
// Before
class Manager extends Employee {
    Manager(String name, String id, int grade) { this.name = name; this.id = id; this.grade = grade; }
}
// After
class Employee { Employee(String name, String id) { this.name = name; this.id = id; } }
class Manager extends Employee {
    Manager(String name, String id, int grade) { super(name, id); this.grade = grade; }
}
```

Push Down Method · inverse: Pull Up Method

Motivation. If a superclass method is only relevant to one (or a few) subclasses, move it down to them. Callers must then know they have that subclass — only do it when they do.

```
// Before
class Employee { int quota() { ... } }                    // only salesmen have a quota
// After
class Salesman extends Employee { int quota() { ... } }
```

Push Down Field · inverse: Pull Up Field

Motivation. A field used by only one subclass should live in that subclass.

```
// Before
class Employee { protected int quota; }
// After
class Salesman extends Employee { private int quota; }
```

Replace Type Code with Subclasses *(aliases: Extract Subclass, Replace Type Code with State/Strategy)* · inverse: Remove Subclass

Motivation. A type code field (`"engineer"`, `"manager"`) that drives behaviour is a hint that you need subclasses; then conditionals can become polymorphism. Two ways: **direct inheritance** (subclass the class itself) or **indirect** (the class holds a separate type object, e.g. `EmployeeType`, which is subclassed — needed if the type can change at runtime or the class is already subclassed for another reason; this is the old "Replace Type Code with State/Strategy").

```
// Before
class Employee { String type; Employee(String name, String type) { ... } }
// After (direct)
abstract sealed class Employee permits Engineer, Manager, Salesman { ... }
static Employee create(String name, String type) {
    return switch (type) {
        case "engineer" -> new Engineer(name);
        case "manager"  -> new Manager(name);
        case "salesman" -> new Salesman(name);
        default -> throw new IllegalArgumentException("Employee cannot be of type " + type);
    };
}
```

Remove Subclass *(alias: Replace Subclass with Fields)* · inverse: Replace Type Code with Subclasses

Motivation. Subclasses that no longer differ in behaviour (e.g. they differ only in a constant) are not worth their cost. Replace them with a field in the superclass.

```
// Before
abstract class Person { abstract String genderCode(); }
class Male extends Person   { String genderCode() { return "M"; } }
class Female extends Person { String genderCode() { return "F"; } }
// After
class Person {
    private final String genderCode;
    Person(String name, String genderCode) { ...; this.genderCode = genderCode; }
    String genderCode() { return genderCode; }
}
```

Extract Superclass

Motivation. Two classes doing similar things have duplicated fields and methods. Create a common superclass and pull the shared parts up. (Alternative: Extract Class and use delegation. Fowler notes you can start with inheritance and switch to delegation later with Replace Superclass with Delegate if it becomes a problem.)

```
// Before
class Department { String name; double totalAnnualCost() { ... } int headCount() { ... } }
class Employee   { String name; double annualCost() { ... } String id() { ... } }
// After
abstract class Party { protected String name; abstract double annualCost(); }
class Department extends Party { ... }
class Employee extends Party { ... }
```

Collapse Hierarchy

Motivation. When a subclass and its superclass are no longer different enough to be worth keeping separate, merge them. A cure for Lazy Element and Speculative Generality.

```
// Before
class Employee { ... }
class Salesman extends Employee { }          // adds nothing any more
// After
class Employee { ... }
```

Replace Subclass with Delegate

Motivation. Inheritance has two limits: you can vary only **one** axis (a person cannot be subclassed by both age-group and income-level), and subclasses are tightly coupled to their parent. Delegation (composition) handles both: the varying behaviour moves to a separate delegate object that can be changed at runtime. This is "favour object composition over class inheritance" (GoF — the "Gang of Four" authors of the 1994 *Design Patterns* book), and the core of the **State** and **Strategy** patterns.

```
// Before
class Booking { double basePrice() { return show.price(); } }
class PremiumBooking extends Booking { @Override double basePrice() { return super.basePrice() +
extras.premiumFee(); } }
// After
class Booking {
    private PremiumDelegate premiumDelegate;           // null unless premium - can be set at runtime
    void bePremium(Extras extras) { premiumDelegate = new PremiumDelegate(this, extras); }
    double basePrice() {
        double result = show.price();
        return premiumDelegate == null ? result : premiumDelegate.extendBasePrice(result);
    }
}
```

Replace Superclass with Delegate ***(alias: Replace Inheritance with Delegation)***

Motivation. If a subclass uses only part of its superclass, or the superclass's methods do not make sense for it (Refused Bequest, broken Liskov substitution), stop inheriting and hold an instance instead, forwarding only what you need. Classic example: a `Stack` that extends a list type (the JDK's `java.util.Stack extends Vector` is often cited).

```
// Before
class Scroll extends CatalogItem { ... }     // but a scroll is a physical copy, not a catalogue entry
// After
class Scroll {
    private final CatalogItem catalogItem;   // delegate
    String title() { return catalogItem.title(); }
    boolean hasTag(String tag) { return catalogItem.hasTag(tag); }
    LocalDate lastCleaned;                   // scroll's own data
}
```

7.3.8 Extra refactorings on refactoring.com

The online catalogue (web edition) lists a few more refactorings that are not among the 61 book chapters' entries. They are worth knowing:

Replace Control Flag with Break ***(alias: Remove Control Flag)***

Motivation. A boolean "found" flag that only exists to stop a loop is clumsy. Use `break`, `continue` or `return`.

```
// Before
boolean found = false;
for (String p : people) {
    if (!found && p.equals("Don")) { sendAlert(); found = true; }
}
// After
for (String p : people) {
    if (p.equals("Don")) { sendAlert(); break; }
}
```

Replace Error Code with Exception

Motivation. Returning special codes (`-1`, `null`) for errors makes every caller check them — and callers forget. Throw an exception so errors cannot be silently ignored and the normal path stays clean.

```
// Before
int withdraw(double amount) { if (amount > balance) return -1; balance -= amount; return 0; }
// After
void withdraw(double amount) {
    if (amount > balance) throw new InsufficientFundsException(amount, balance);
    balance -= amount;
}
```

Replace Exception with Precheck ***(alias: Replace Exception with Test)***

Motivation. Exceptions are for *exceptional* behaviour. If the caller can easily check the condition first, check it instead of catching an exception.

```
// Before
double valueAt(int i) { try { return values[i]; } catch (ArrayIndexOutOfBoundsException e) { return 0; } }
// After
double valueAt(int i) { return (i >= 0 && i < values.length) ? values[i] : 0; }
```

Replace Magic Literal ***(alias: Replace Magic Number with Symbolic Constant)***

Motivation. A literal like `9.81` or `"STD"` with special meaning should get a name, so readers know what it means and there is one place to change it. (Don't bother when the meaning is obvious, e.g. `0` as an initial sum.) The worked example's step 4 does this.

```
// Before
double potentialEnergy(double mass, double height) { return mass * 9.81 * height; }
// After
static final double STANDARD_GRAVITY = 9.81;
double potentialEnergy(double mass, double height) { return mass * STANDARD_GRAVITY * height; }
```

Return Modified Value

Motivation. When a function modifies a variable passed in or shared, make it *return* the new value instead and let the caller assign it. The data flow becomes visible.

```
// Before
double totalAscent = 0;
calculateAscent();                      // secretly updates the field totalAscent
// After
double totalAscent = calculateAscent();
double calculateAscent() {
    double result = 0;
    for (int i = 1; i < points.size(); i++) {
        double diff = points.get(i).elevation() - points.get(i - 1).elevation();
        result += Math.max(diff, 0);
    }
    return result;
}
```

1st-edition names you may still hear

1st-edition name	2nd-edition name
Extract Method / Inline Method	Extract Function / Inline Function
Introduce Explaining Variable / Inline Temp	Extract Variable / Inline Variable
Rename Method, Add Parameter, Remove Parameter	Change Function Declaration
Self-Encapsulate Field / Encapsulate Field	Encapsulate Variable
Replace Record with Data Class	Encapsulate Record
Replace Data Value with Object, Replace Type Code with Class	Replace Primitive with Object
Move Method	Move Function
Consolidate Duplicate Conditional Fragments	Slide Statements
Split Temp, Remove Assignments to Parameters	Split Variable
Introduce Null Object	Introduce Special Case
Parameterize Method	Parameterize Function
Replace Parameter with Explicit Methods	Remove Flag Argument
Replace Parameter with Method	Replace Parameter with Query
Replace Constructor with Factory Method	Replace Constructor with Factory Function
Replace Method with Method Object	Replace Function with Command
Extract Subclass, Replace Type Code with State/Strategy	Replace Type Code with Subclasses
Replace Subclass with Fields	Remove Subclass
Replace Inheritance with Delegation	Replace Superclass with Delegate
Remove Control Flag	Replace Control Flag with Break (web)
Replace Magic Number with Symbolic Constant	Replace Magic Literal (web)
Replace Exception with Test	Replace Exception with Precheck (web)

7.4 Smell → refactorings lookup table

Use this when you have named a smell and need to choose a cure. **Bold** = the usual first move.

Smell	Group	Refactorings that cure it
Mysterious Name	2nd-ed extra	**Change Function Declaration**, Rename Variable, Rename Field
Duplicated Code	Dispensables	**Extract Function**, Slide Statements, Pull Up Method *(also useful, not in Fowler's list: Replace Inline Code with Function Call, Substitute Algorithm)*
Long Function	Bloaters	**Extract Function**, Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop
Long Parameter List	Bloaters	Replace Parameter with Query, **Preserve Whole Object**, **Introduce Parameter Object**, Remove Flag Argument, Combine Functions into Class
Global Data	2nd-ed extra	**Encapsulate Variable** (then shrink its scope)
Mutable Data	2nd-ed extra	**Encapsulate Variable**, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class, Combine Functions into Transform, Change Reference to Value
Divergent Change	Change Preventers	**Split Phase**, Move Function, Extract Function, **Extract Class**
Shotgun Surgery	Change Preventers	**Move Function**, **Move Field**, Combine Functions into Class, Combine Functions into Transform, Split Phase, Inline Function, Inline Class
Feature Envy	Couplers	**Move Function**, Extract Function
Data Clumps	Bloaters	**Extract Class**, **Introduce Parameter Object**, Preserve Whole Object
Primitive Obsession	Bloaters	**Replace Primitive with Object**, Replace Type Code with Subclasses, Replace Conditional with Polymorphism, Extract Class, Introduce Parameter Object
Repeated Switches	OO Abusers	**Replace Conditional with Polymorphism**
Loops	2nd-ed extra	**Replace Loop with Pipeline**
Lazy Element	Dispensables	**Inline Function**, **Inline Class**, Collapse Hierarchy
Speculative Generality	Dispensables	Collapse Hierarchy, Inline Function, Inline Class, Change Function Declaration, Remove Dead Code
Temporary Field	OO Abusers	**Extract Class**, Move Function, Introduce Special Case
Message Chains	Couplers	**Hide Delegate**, Extract Function, Move Function
Middle Man	Couplers	**Remove Middle Man**, Inline Function, Replace Superclass with Delegate, Replace Subclass with Delegate
Insider Trading	Couplers	**Move Function**, **Move Field**, Hide Delegate, Replace Subclass with Delegate, Replace Superclass with Delegate
Large Class	Bloaters	**Extract Class**, Extract Superclass, Replace Type Code with Subclasses
Alternative Classes with Different Interfaces	OO Abusers	**Change Function Declaration**, Move Function, Extract Superclass
Data Class	Dispensables	**Encapsulate Record**, Remove Setting Method, Move Function, Extract Function, Split Phase
Refused Bequest	OO Abusers	Push Down Method, Push Down Field, **Replace Subclass with Delegate**, **Replace Superclass with Delegate**
Comments	Dispensables	**Extract Function**, Change Function Declaration, Introduce Assertion

And the reverse view — which **design principle** each family of smells violates (links to Chapters 4–6):

Smell family	Principle it violates
Divergent Change, Large Class	Single Responsibility Principle
Repeated Switches, Shotgun Surgery	Open/Closed Principle
Refused Bequest	Liskov Substitution Principle
Long Parameter List with flags, fat interfaces	Interface Segregation / clean function design
Global Data, Mutable Data	Encapsulation / information hiding (control who can change what)
Hard-wired `new` of concrete dependencies	Dependency Inversion
Duplicated Code	DRY (Don't Repeat Yourself)
Speculative Generality	YAGNI
Message Chains, Insider Trading	Law of Demeter / encapsulation
Feature Envy, Data Class	"Tell, Don't Ask" — put behaviour with data

---

7.5 Worked example — from messy to clean in six compiling steps

The starting point

A courier company prints invoices. Each shipment has a **service type** (`STD` standard, `EXP` express, `FRG` fragile), a weight and a distance.

Tariffs:

Type	Charge	Loyalty points
STD	5 + 1.2 per kg, plus 0.05 per km beyond 100 km	1 per full 100 km
EXP	12 + 2.0 per kg, plus 0.10 per km beyond 50 km	1 per full 100 km, +2 bonus
FRG	8 + 1.5 per kg + 3 handling; ×1.2 if heavier than 10 kg	1 per full 100 km

The new feature request: "We also need an HTML invoice for the web portal, and next quarter we are adding an OVERNIGHT service."

The current code (step 0, abridged — full file in 7.8):

```
static String print(Inv inv) {
    double tot = 0;
    int pts = 0;
    int unused = 0;                                  // dead code
    StringBuilder r = new StringBuilder("Invoice for " + inv.cust + "\n");
    for (Shp s : inv.list) {
        double amt = 0;
        switch (s.t) {
            case "STD": amt = 5 + s.kg * 1.2;       if (s.km > 100) amt += (s.km - 100) * 0.05; break;
            case "EXP": amt = 12 + s.kg * 2.0;      if (s.km > 50)  amt += (s.km - 50) * 0.10;  break;
            case "FRG": amt = 8 + s.kg * 1.5 + 3;   if (s.kg > 10)  amt = amt * 1.2;            break;
            default: throw new IllegalArgumentException("unknown type: " + s.t);
        }
        pts += s.km / 100;
        if ("EXP".equals(s.t)) pts += 2;
        r.append("  ").append(s.t).append(" ").append(s.kg).append("kg ")
         .append(s.km).append("km: ").append(String.format(Locale.ROOT, "%.2f", amt)).append("\n");
        tot += amt;
    }
    r.append("Amount owed: ").append(String.format(Locale.ROOT, "%.2f", tot)).append("\n");
    r.append("Points earned: ").append(pts).append("\n");
    return r.toString();
}
```

Smells spotted.

Smell	Where
Mysterious Name	`Shp`, `Inv`, `t`, `kg`, `km`, `amt`, `tot`, `pts`, `r`
Long Function	`print` calculates charges, calculates points, and formats — three jobs
Divergent Change	HTML output would force copying the whole function; a new tariff means editing the formatting function
Repeated Switches (waiting to happen)	The type code `"EXP"` is checked in two places already
Primitive Obsession	The service type is a `String`; any typo compiles
Magic literals	`5`, `1.2`, `100`, `0.05`, `12`, ...
Data Class / Mutable Data	Public mutable fields in `Shp` and `Inv`
Dead code	`int unused`

Adding HTML now would mean duplicating everything. So we do **preparatory refactoring** first: make the change easy, then make the easy change.

The safety net

Before touching anything we write a **characterisation check** — `lld.ch07.worked.Main`. It runs step 0 to record the "golden" output, then runs every later step and compares the output **byte for byte**. The fixture (the fixed test input data) includes boundary cases (exactly 100 km for STD, exactly 10 kg for FRG) because boundaries are where refactorings most often break behaviour. In a real project this would be a JUnit test (JUnit is the standard Java testing framework); here it is a plain `main` so it runs with the JDK (Java Development Kit — the compiler and runtime) alone.

Each step lives in its own package (`step0` … `step5`) so you can compile and diff them (compare them line by line). After *every* step the checker prints `SAME output`.

Step 1 — Extract Function, Rename Variable/Field, Remove Dead Code

- **Rename Field**: `t → type`, `kg → weightKg`, `km → distanceKm`; classes `Shp → Shipment`, `Inv → Invoice`.
- **Rename Variable**: `tot → totalAmount`, `pts → loyaltyPoints`, `r → result`.
- **Extract Function** ×3: the switch becomes `amountFor(shipment)`, the points logic becomes `pointsFor(shipment)`, the formatting becomes `money(amount)`.
- **Remove Dead Code**: `unused` is gone.

```
for (Shipment shipment : invoice.shipments) {
    double thisAmount = amountFor(shipment);
    loyaltyPoints += pointsFor(shipment);
    result.append(...).append(money(thisAmount)).append("\n");
    totalAmount += thisAmount;
}
```

The switch itself was *moved*, not changed. Moving before changing is a key habit: two small safe steps instead of one risky one.

Step 2 — Inline Variable, Split Loop, Replace Temp with Query, Replace Loop with Pipeline, Slide Statements

The loop still does three jobs (print a line, add to total, add points). We:

1. **Inline Variable** `thisAmount` — call `amountFor(shipment)` directly.
2. **Split Loop** — one loop for printing, one for the total, one for points.
3. **Replace Temp with Query** — the total and the points become `totalAmount(invoice)` and `totalPoints(invoice)`.
4. **Replace Loop with Pipeline** — those two queries become streams.
5. **Slide Statements** — the declarations disappear; each value is computed where it is used.

```
static double totalAmount(Invoice invoice) {
    return invoice.shipments.stream().map(Main::amountFor).reduce(0.0, Double::sum);
}
static int totalPoints(Invoice invoice) {
    return invoice.shipments.stream().mapToInt(Main::pointsFor).sum();
}
```

A subtle trap. The obvious pipeline is `mapToDouble(...).sum()`. But `DoubleStream.sum()` may use compensated summation, so it can differ from a plain left-to-right loop in the last bit of a `double`. That is a (tiny) behaviour change. `reduce(0.0, Double::sum)` on a sequential stream adds in encounter order — exactly like the loop. (For `int`, addition is exact, so `IntStream.sum()` is safe.) Performance note: we now call `amountFor` twice per shipment — the kind of cost Fowler says to ignore until a profiler shows it matters.

Step 3 — Split Phase, Encapsulate Record, Encapsulate Collection, Combine Functions into Transform

Now the key preparatory move. Calculation and formatting are two **phases**. We split them with an intermediate data structure:

- `StatementData` (a record) — everything the renderer needs: customer, one `Line` per shipment with its computed amount, total, points.
- `StatementCalculator.calculate(Invoice)` — phase 1, a **transform** (Combine Functions into Transform): raw invoice in, enriched data out.
- `TextRenderer.render(StatementData)` — phase 2. It knows nothing about tariffs.
- `Shipment` and `Invoice` become **records** (Encapsulate Record — no more public mutable fields), and `Invoice` copies its list with `List.copyOf` (Encapsulate Collection).

```
return new TextRenderer().render(new StatementCalculator().calculate(invoice));
```

Step 4 — Replace Primitive with Object, Replace Magic Literal

- **Replace Primitive with Object**: the `String` type code becomes `enum ServiceType { STANDARD("STD"), EXPRESS("EXP"), FRAGILE("FRG") }` with `fromCode(String)`. An unknown code now fails *at the boundary* (when the shipment is created) instead of deep inside the calculation. It is the same exception type and message, and in this program shipments are always built and printed together, so the observable behaviour is unchanged; if some caller created invoices without ever printing them, moving the failure earlier would be a behaviour change to check first. The `"STD"` strings survive only where data enters the system and where it is printed.
- **Replace Magic Literal**: every tariff number gets a name (`STD_FREE_KM`, `FRG_HEAVY_FACTOR`, ...).
- The switch becomes a **switch expression** over the enum; with no `default` branch, the compiler checks it is exhaustive.

```
return switch (s.type()) {
    case STANDARD -> STD_BASE + s.weightKg() * STD_PER_KG
            + (s.distanceKm() > STD_FREE_KM ? (s.distanceKm() - STD_FREE_KM) * STD_PER_EXTRA_KM : 0);
    case EXPRESS  -> ...
    case FRAGILE  -> { double base = ...; yield s.weightKg() > FRG_HEAVY_KG ? base * FRG_HEAVY_FACTOR : base; }
};
```

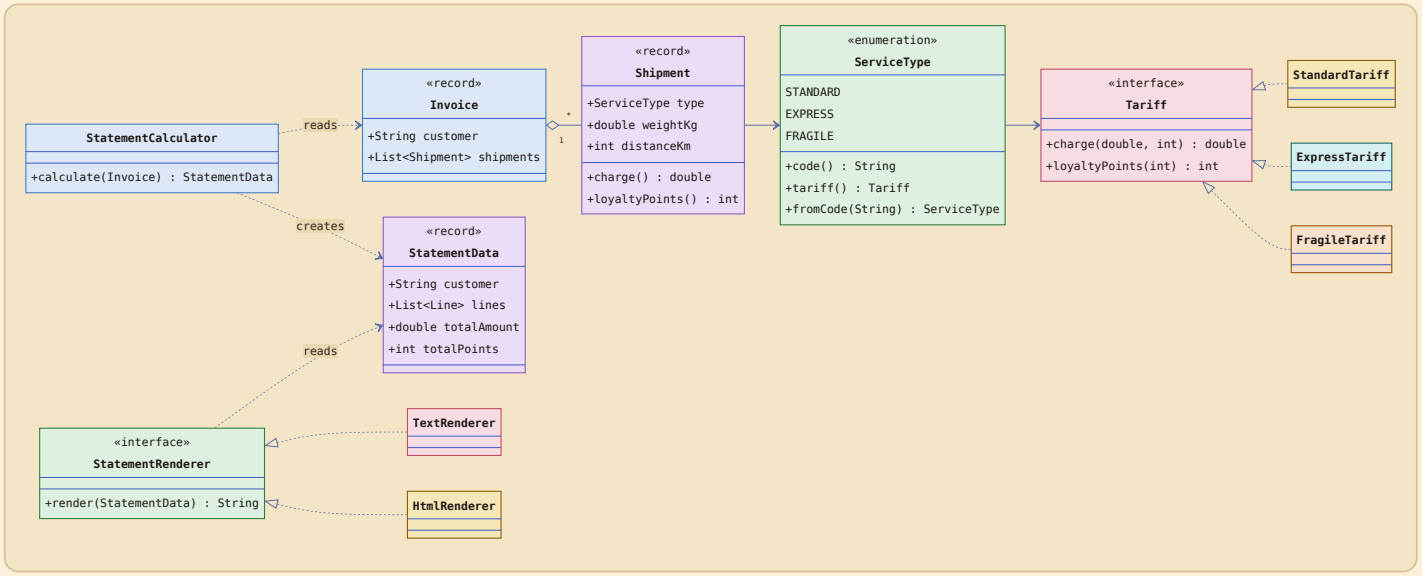
(Is `x + 0` the same as the old "`if` not taken"? For `double`, adding `+0.0` leaves any value unchanged except `-0.0`, which cannot occur here. The checker confirms it.)

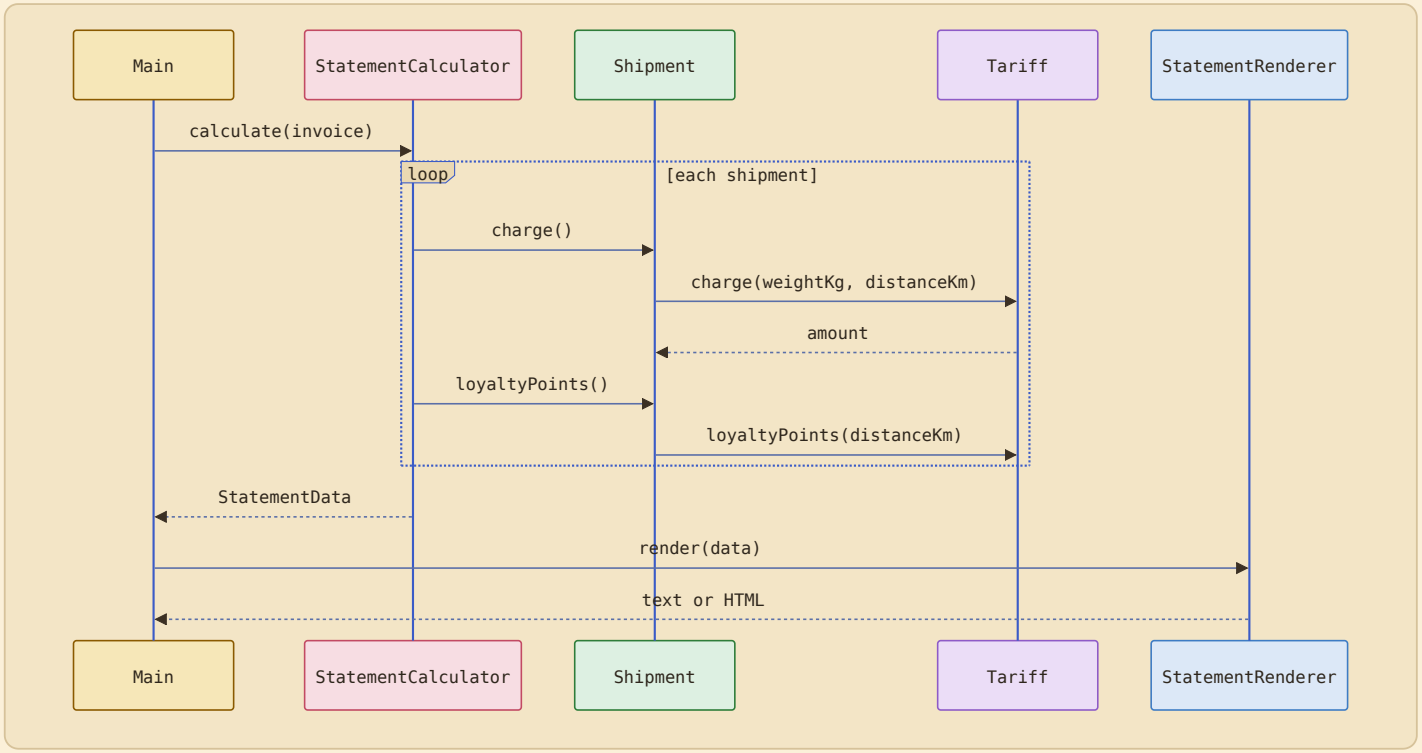
Step 5 — Replace Conditional with Polymorphism, Move Function, then the new feature

- **Replace Conditional with Polymorphism**: each switch branch becomes a class implementing `sealed interface Tariff { double charge(double weightKg, int distanceKm); default int loyaltyPoints(int distanceKm) }`. `ExpressTariff` overrides `loyaltyPoints` to add its bonus — the second `"EXP"` check disappears too. Each enum constant holds its `Tariff` (a **Strategy**).
- **Move Function**: `Shipment.charge()` and `Shipment.loyaltyPoints()` — the calculator no longer reaches into shipment data (Feature Envy cured).
- `StatementCalculator` has **no conditionals left**.

Then — wearing the *adding function* hat now — the new feature is easy:

- Extract a `StatementRenderer` interface; `TextRenderer` implements it.
- Add `HtmlRenderer` — about 15 lines, reusing phase 1 unchanged.
- Adding OVERNIGHT next quarter = one new `Tariff` record, one new enum constant, and one more name in `Tariff`'s `permits` list (a sealed interface must list every implementation). The calculator and the renderers do not change (Open/Closed Principle for the calculation and rendering code).





What changed, step by step

Step	Refactorings	Smells removed
1	Extract Function ×3, Rename Variable, Rename Field, Remove Dead Code	Mysterious Name, part of Long Function, dead code
2	Inline Variable, Split Loop, Replace Temp with Query, Replace Loop with Pipeline, Slide Statements	Long Function (loop doing 3 jobs), Loops
3	Split Phase, Combine Functions into Transform, Encapsulate Record, Encapsulate Collection	Divergent Change, Data Class, Mutable Data
4	Replace Primitive with Object, Replace Magic Literal	Primitive Obsession, magic numbers
5	Replace Conditional with Polymorphism, Move Function, (then Extract interface for renderers)	Repeated Switches, Feature Envy

Checker output (`java lld.ch07.worked.Main`):

```
Golden output from step 0:
Invoice for Asha Traders
  STD 4.0kg 250km: 17.30
  EXP 2.5kg 80km: 20.00
  FRG 12.0kg 40km: 34.80
  STD 1.0kg 90km: 6.20
  STD 0.5kg 100km: 5.60
  FRG 10.0kg 150km: 26.00
Amount owed: 109.90
Points earned: 6
---
step0 (messy original)         SAME output
step1 extract + rename         SAME output
step2 split loop + pipeline    SAME output
step3 split phase              SAME output
step4 primitive -> object      SAME output
step5 polymorphism (final)     SAME output
All steps preserve behaviour.
```

What we deliberately did NOT do. Money is still a `double`. Switching to `BigDecimal` (or the `Money` type from `code/smells`) would change rounding in some cases — that is a *behaviour change*, so it belongs under the "adding function" hat, with its own tests and a product decision about rounding rules. Refactoring and behaviour changes are kept apart.

7.6 Refactoring toward patterns

What it is

Design patterns (Chapters 8–11) are not only designed up front. Very often you **arrive at a pattern by refactoring** when the code asks for it. Joshua Kerievsky's book *Refactoring to Patterns* (2004) catalogues these moves; many are compositions of Fowler's small refactorings. The idea: do not force patterns in early (Speculative Generality); refactor *toward* a pattern when a smell appears, and sometimes *away* from one when it is no longer pulling its weight.

How it works

> **Key term — patterns in this table not met earlier in this chapter** (all are covered in Chapters 8–11). **Factory**: an object or method whose only job is creating other objects; a **Builder** assembles one complex object step by step. **Observer**: subscribers register with an object and are notified automatically when it changes. **Bridge**: separates an abstraction from its implementation so the two can vary independently. **Composite**: a tree structure in which single items and groups of items are treated the same way.

Smell you see	Refactor toward	Via (Fowler's steps)
A switch/if-chain choosing an algorithm (e.g. discount by customer tier)	**Strategy**	Extract Function per branch → Replace Conditional with Polymorphism (Kerievsky: *Replace Conditional Logic with Strategy*)
Methods that check a status field and change it (`if status == PAID then ...`)	**State**	Replace Type Code with Subclasses (indirect, via a state object) → Replace Conditional with Polymorphism (Kerievsky: *Replace State-Altering Conditionals with State*)
`if (x == null)` repeated at many call sites	**Null Object / Special Case**	Introduce Special Case (Kerievsky: *Introduce Null Object*)
Sibling subclasses with the same algorithm skeleton but different steps	**Template Method**	Extract Function in each → Pull Up Method (Kerievsky: *Form Template Method*)
Many constructors with confusing boolean/int parameters	**Factory Method / Creation Methods**	Replace Constructor with Factory Function (Kerievsky: *Replace Constructors with Creation Methods*)
Many `new ConcreteX()` calls scattered; clients should not know which class	**Factory**	Replace Constructor with Factory Function + Move Function to a factory class (Kerievsky: *Encapsulate Classes with Factory*)
Complex object (in Kerievsky's refactoring, a Composite tree) built step by step with many calls in the right order	**Builder**	Kerievsky: *Encapsulate Composite with Builder*
A function too big to decompose because of many shared locals; or you need undo/queueing	**Command**	Replace Function with Command
Embellishment code (logging, caching) mixed into a core class	**Decorator**	Kerievsky: *Move Embellishment to Decorator*
Many objects polling one object for changes; hard-coded notification	**Observer**	Kerievsky: *Replace Hard-Coded Notifications with Observer*
Two classes with different interfaces doing the same job	**Adapter**	Change Function Declaration / Extract Superclass; wrap third-party code (Kerievsky: *Extract Adapter*, *Unify Interfaces with Adapter*)
A switch on a type code to create objects	**Factory + polymorphism**	Replace Type Code with Subclasses + Replace Constructor with Factory Function
Subclass explosion for combinations of features	**Strategy / Decorator / Bridge**	Replace Subclass with Delegate
A pattern that no longer earns its keep (one strategy, one decorator)	*Remove the pattern*	Inline Class, Collapse Hierarchy (Kerievsky: *Inline Singleton* and similar "away from" moves)

The runnable package `lld.ch07.towardpatterns` shows five of these with the **before** code and the **after** code side by side, and checks that both produce the same results. (The prices and loan amounts there are `double` only to keep the before/after comparison short; in production code use `BigDecimal` or a `Money` type.)

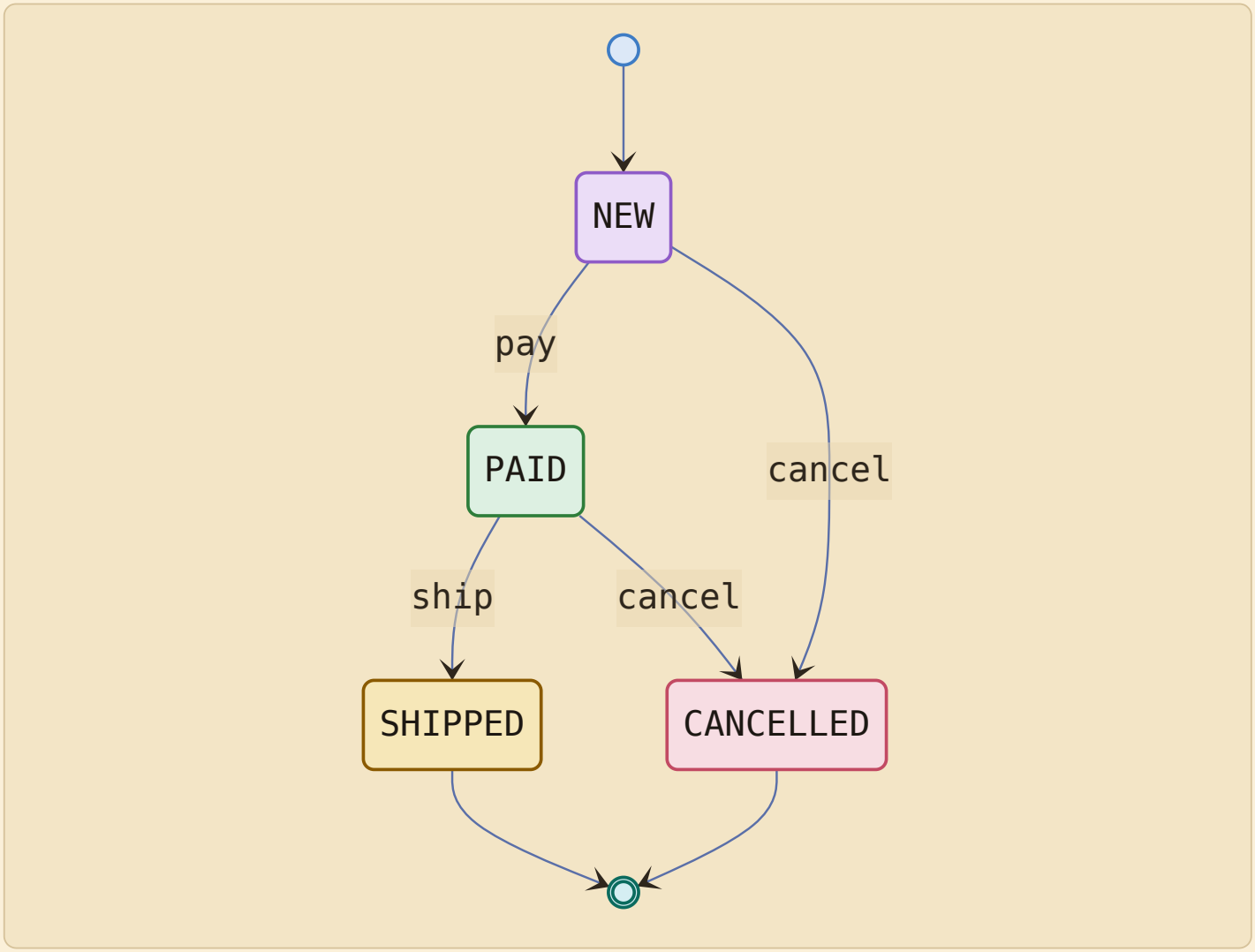
Replace Conditional Logic with Strategy

Before: a `switch (tier)` that every new customer tier must edit. **After:** a `DiscountPolicy` functional interface (an interface with a single abstract method, so a lambda — a short inline function — can implement it), one policy per tier (lambdas here; classes if a policy grows), stored in an `EnumMap` (a map whose keys are enum constants). New tiers are *registered*, not coded into a switch.

```
@FunctionalInterface
interface DiscountPolicy { double apply(double amount); }

double price(Tier tier, double amount) {
    return policies.getOrDefault(tier, fallback).apply(amount);
}
```

Replace State-Altering Conditionals with State

Before: every method checks `status` strings and changes them — the rules of the lifecycle are scattered across methods. **After:** each state is an enum constant implementing `OrderState`; it overrides only the moves it allows. Default methods (interface methods with a built-in body) reject everything else. `Order` just delegates: `state = state.pay(this)`.




```
enum States implements OrderState {
    NEW {
        @Override public OrderState pay(Order o)    { o.log("paid"); return PAID; }
        @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
    },
    PAID {
        @Override public OrderState ship(Order o)   { o.log("shipped"); return SHIPPED; }
        @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
    },
    SHIPPED, CANCELLED
}
```

Strategy vs State: they have the same class structure. The difference is intent — a **strategy** is usually chosen by the client and stays fixed; a **state** changes *itself* as the object moves through its lifecycle.

Introduce Special Case (Null Object)

Before: `(c == null) ? "occupant" : c.name()` in every client. **After:** `find(id)` returns an `UnknownCustomer` that answers `"occupant"` and `"basic"` itself. Clients have no null checks.

Form Template Method

Before: `legacyCsv` and `legacyMarkdown` repeat the same skeleton (header, one row per item, footer). **After:** an abstract `Exporter.export()` (declared `final`) holds the skeleton once and calls abstract `header()`, `row()`, `footer()` steps; `CsvExporter` and `MarkdownExporter` fill them in.

Replace Constructors with Creation Methods

Before: `new LegacyLoan(5000, false, 12)` — what does `false` mean? **After:** a private constructor plus `Loan.termLoan(5000, 12)` and `Loan.revolver(2000)`.

Output (`java lld.ch07.towardpatterns.Main`):

```
Strategy:        GOLD 250 -> 200.0, NEW 250 -> 240.0 | same as switch: true
State:           cannot ship when NEW; paid; cannot pay when PAID; shipped; cannot cancel when SHIPPED; | same as
if-chains: true
Special Case:    Hello Ravi on premium / Hello occupant on basic | same: true
Template Method: CSV and Markdown exporters | same: true
Factory:         term 5000.0/12, revolver 2000.0/0 | same: true
All pattern refactorings preserve behaviour.
```

Small smell cures (package `lld.ch07.smells`)

A separate runnable package shows the cures for several smells from 7.2 as real code: `Money` (Primitive Obsession), `DateRange` (Data Clumps), and `Order` with `shippingCity()` (Message Chains → Hide Delegate), `Line.total()` (Feature Envy → Move Function) and a read-only `lines()` view (Mutable Data → Encapsulate Collection).

Output (`java lld.ch07.smells.Main`):

```
double 0.1+0.2       = 0.30000000000000004
Money  0.10+0.20     = USD 0.30
mixing currencies    -> currency mismatch: USD vs EUR
booking contains 3rd = true, overlaps other = true
ship to Bengaluru, total INR 301.00
outsider tried lines().clear() -> UnsupportedOperationException (collection is encapsulated)
```

7.7 Putting it together

Trade-offs

Benefit	Cost / risk
Code easier to read and change; features get faster over time (Design Stamina)	Takes time now; the pay-off is later and hard to measure
Bugs become visible	Without tests, you can introduce bugs
Design can evolve with requirements (less up-front guessing)	Renames and moves cause merge conflicts on long-lived branches
Small steps are safe and can stop at any point	Small steps feel slow to beginners (they get fast with practice and IDE support)
More small classes and methods, each easy to understand	More indirection — too much creates Middle Men and Lazy Elements
Removes duplication	Premature abstraction (before the Rule of Three) can create the *wrong* abstraction, which is costlier than duplication

When not to use it

(Summary of 7.1.4.) Do not refactor code you do not need to touch; do not refactor without a safety net (add tests first); do not refactor instead of a rewrite when the code does not work at all; do not break published interfaces without a migration path; do not mix refactoring with behaviour changes in the same step.

Common mistakes

Mistake	Why it is a problem	Do this instead
Calling any code change "refactoring"	Behaviour changes hide inside "refactoring" commits; reviewers relax	Keep refactoring commits separate from feature/bug-fix commits
Big-bang refactoring (days with broken code)	High risk, merge pain, no way to stop halfway	Tiny steps, compile and test after each, commit often
Refactoring without tests	Silent behaviour changes	Write characterisation tests first; use IDE-automated refactorings
Debugging a failed refactoring step	Wastes time	Revert and redo in smaller steps
Tests coupled to private implementation	Every refactoring breaks tests	Test behaviour through public APIs
Text search-and-replace for renames	Changes strings, comments, unrelated symbols	IDE Rename (symbol-aware) — and check reflection/config/serialisation
Abstracting on the first duplication	Wrong abstraction; many flags later	Rule of Three
Changing rounding/ordering "while you're there" (e.g. `DoubleStream.sum()` vs loop, `toList()` vs mutable `ArrayList`)	Subtle behaviour changes	Know the exact semantics of the replacement API
Adding patterns everywhere "for flexibility"	Speculative Generality	Refactor toward a pattern when a smell demands it
Refactoring sprints as the main approach	Business sees refactoring as a separate, optional cost	Opportunistic refactoring inside every feature

Real-world examples

- **IntelliJ IDEA, Eclipse, VS Code (Java extension)** — automated Rename, Extract Method, Inline, Change Signature, Move, Pull Up / Push Down: these are direct implementations of catalogue entries.
- **OpenRewrite** — open-source engine for automated, large-scale source refactoring with "recipes"; used, for example, to migrate codebases between Spring Boot (a popular Java application framework) versions or from JUnit 4 to JUnit 5.
- **Google** — Error Prone with Refaster templates, used for large automated changes across its monorepo (one giant repository holding most of the company's code).
- **JDK history.** `java.util.Stack extends Vector` and `java.util.Properties extends Hashtable` are classic, often-cited cases of inheritance where delegation would have been better (Refused Bequest). For new code, the JDK docs recommend `ArrayDeque` (via the `Deque` interface) over `Stack`. `Date` → `java.time` (Java 8) shows Primitive Obsession/Mutable Data being replaced by immutable value types (`LocalDate`, `Instant`, `Duration`) — a new API alongside the old one rather than an in-place refactor, because the old API is published.
- **Deprecation as migration mechanics.** The JDK and Spring mark old methods `@Deprecated` (Java 9 added `forRemoval`) and keep them delegating for several releases — exactly *Change Function Declaration* with migration mechanics.
- **Java language features that support refactoring**: records (Encapsulate Record, value objects), `sealed` interfaces + switch pattern matching (safe type hierarchies), streams (Replace Loop with Pipeline), `Optional` (making absence explicit), `List.copyOf` / `Collections.unmodifiableList` (Encapsulate Collection).
- **Strangler Fig in industry.** Many companies that split a monolith (an application built and deployed as one unit) into services (Fowler's own write-ups describe this pattern) route one feature at a time from the old system to the new, instead of a big-bang rewrite.

In the interview

30-second answer — "What is refactoring?" "Refactoring is changing the internal structure of code to make it easier to understand and modify, without changing its observable behaviour. You do it in very small steps — each one compiles and passes the tests — so the system is never broken. You use code smells like Long Function, Duplicated Code or Repeated Switches to find where to refactor, and named refactorings like

> Extract Function, Move Function or Replace Conditional with Polymorphism to fix them. Tests are the safety net. The goal is economic: clean code lets you add features faster."

Follow-ups (with answers)

- **Q: How do you refactor code that has no tests?** A: Treat it as legacy code. First write characterisation tests that record current behaviour. If the code is hard to test, break dependencies at a seam — for example, pass an interface through the constructor instead of calling `new` inside. Until tests exist, use only IDE-automated, symbol-aware refactorings. Sprout Method/Class lets me add new, tested code without first cleaning everything.
- **Q: Refactor or rewrite?** A: Default to refactoring — small safe steps and the system keeps working. Consider a rewrite only when the code really does not work or the platform must change, and even then use a Strangler Fig approach to replace it piece by piece.
- **Q: Doesn't refactoring hurt performance?** A: Rarely in a way that matters: small methods are inlined by the JIT, and most time is spent in a small part of the code. I refactor for clarity, then profile and optimise the real hot spots, which is easier in well-factored code.
- **Q: What would you refactor in this code?** (In a machine-coding round.) A: Name smells and cures: "`processOrder` is a Long Function — I'd Extract Function for validation, pricing and persistence. The switch on `vehicleType` appears in three places — Repeated Switches — I'd Replace Conditional with Polymorphism, giving a Strategy per vehicle type. `double price` is Primitive Obsession — a `Money` value object. And `getParkingLot().getFloor(1).getSpot(3)` is a Message Chain — Hide Delegate."
- **Q: When do you remove duplication?** A: Rule of Three. After three occurrences the shape of what varies is clear, so the abstraction is likely right. Earlier abstraction risks the wrong abstraction.
- **Q: Difference between Divergent Change and Shotgun Surgery?** A: Divergent Change: one class changes for many different reasons → split it. Shotgun Surgery: one reason to change touches many classes → gather the code into one place.
- **Q: How do you get from a switch statement to the Strategy or State pattern?** A: Extract each branch into a function, then Replace Conditional with Polymorphism: create an interface with one implementation per case and let the object hold the right implementation. If the choice is fixed by the client, it's Strategy; if the object moves between cases itself, it's State.
- **Q: Is a data class always bad?** A: No. Immutable records used as values, DTOs or the output of a Split Phase are fine. The smell is a *mutable* class with getters/setters whose behaviour lives in other classes.
- **Q: How do you rename a public API method used by other teams?** A: Migration mechanics of Change Function Declaration: add the new method, make the old one delegate to it and mark it `@Deprecated`, migrate callers over time, then remove the old one. For REST APIs, versioning or expand/contract.

Self-check

1. **Q:** Is fixing a bug while tidying the method one refactoring? **A:** No. The bug fix changes behaviour. Do the refactoring and the fix as separate steps (two hats), ideally separate commits.
2. **Q:** A method prints a banner, sums orders, sets a due date and prints details. Which smell and which refactoring? **A:** Long Function; Extract Function for each block (and maybe Split Loop / Replace Temp with Query).
3. **Q:** `book(hotel, from, to, 2, 1, true)` — which smells? **A:** Long Parameter List, Data Clumps (`from`, `to` and the two guest counts), a flag argument (`true`). Cures: Introduce Parameter Object (`Stay`, `Guests`), Remove Flag Argument (or replace the boolean with an explicit enum such as `MealPlan`).
4. **Q:** What is the inverse of Hide Delegate, and when do you use it? **A:** Remove Middle Man — when a class has become mostly forwarding methods.
5. **Q:** Why did the worked example use `reduce(0.0, Double::sum)` instead of `DoubleStream.sum()`? **A:** `DoubleStream.sum()` may use compensated summation and can differ from a plain loop in the last bit; `reduce` on a sequential stream adds left to right like the loop, so behaviour is preserved.
6. **Q:** What is a characterisation test? **A:** A test that records what existing code currently does (right or wrong), so a refactoring that changes behaviour is detected.
7. **Q:** Which smell does `class Stack extends ArrayList` have, and what is the cure? **A:** Refused Bequest (it inherits list operations that break stack rules, violating Liskov). Replace Superclass with Delegate — hold a `Deque` instead.

8. **Q:** Which refactoring turns a status-checking `if` chain into the State pattern? **A:** Replace Type Code with Subclasses (via a state object) followed by Replace Conditional with Polymorphism (Kerievsky: Replace State-Altering Conditionals with State).
9. **Q:** Name the five refactoring.guru smell groups. **A:** Bloaters, Object-Orientation Abusers, Change Preventers, Dispensables, Couplers.
10. **Q:** What is preparatory refactoring? **A:** Refactoring just before a feature so that the feature becomes easy to add — "make the change easy, then make the easy change". Step 3 (Split Phase) of the worked example did this for HTML output.
11. **Q:** When is a comment not a smell? **A:** When it explains *why* (a decision, a trade-off, a reference, a warning), not *what* the code does.
12. **Q:** What does Separate Query from Modifier enforce? **A:** Command–Query Separation: functions that return values have no observable side effects.
13. **Q:** A Java IDE rename of a field broke the JSON response. Was it a refactoring? **A:** It was meant to be, but the JSON field name (observable behaviour) changed, so behaviour was not preserved. Check serialisation, reflection and config when renaming.

TL;DR

- **Refactoring = small, behaviour-preserving changes** to internal structure, done continuously, with tests after every step. Two hats: never refactor and change behaviour at the same time.
- **When:** Rule of Three, preparatory ("make the change easy, then make the easy change"), comprehension, litter-pickup, during code review. Mostly opportunistic, rarely planned. **Not** for code you never touch, and not without a safety net.
- **Smells** tell you *where*: 24 in the 2nd edition — Mysterious Name, Duplicated Code, Long Function, Long Parameter List, Global Data, Mutable Data, Divergent Change, Shotgun Surgery, Feature Envy, Data Clumps, Primitive Obsession, Repeated Switches, Loops, Lazy Element, Speculative Generality, Temporary Field, Message Chains, Middle Man, Insider Trading, Large Class, Alternative Classes with Different Interfaces, Data Class, Refused Bequest, Comments. Groups: Bloaters, OO Abusers, Change Preventers, Dispensables, Couplers.
- **Refactorings** tell you *how*: 61 named ones in seven families; the everyday core is Extract/Inline Function, Rename, Move Function, Extract Class, Split Phase, Replace Conditional with Polymorphism.
- **Performance:** refactor first, then profile and optimise hot spots. **Rewrite** only rarely, and then with a Strangler Fig. **Legacy code** = code without tests → characterisation tests, seams, sprout/wrap.
- **Patterns** are often reached by refactoring: switch → Strategy/State, null checks → Null Object, duplicated skeleton → Template Method, confusing constructors → Factory methods.

7.8 Full source code and output

How to compile and run (Java 21, standard library only):

```
cd chapters/07-refactoring/code
javac --release 21 -d /tmp/lld_07 $(find . -name '*.java')
java -cp /tmp/lld_07 lld.ch07.worked.Main           # characterisation check over all six steps
java -cp /tmp/lld_07 lld.ch07.worked.step5.Main     # final design + new HTML feature
java -cp /tmp/lld_07 lld.ch07.towardpatterns.Main   # refactoring toward patterns
java -cp /tmp/lld_07 lld.ch07.smells.Main           # small smell cures
```

Packages: `lld.ch07.worked` (checker), `lld.ch07.worked.step0` … `step5` (each has its own `Main`; steps 0–4 print the text invoice only), `lld.ch07.towardpatterns`, `lld.ch07.smells`. All 9 `Main` classes compile and run.

Worked example — characterisation check

```java
// File: code/worked/Main.java
package lld.ch07.worked;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.function.Supplier;

/**
 * Characterisation check - the safety net for the whole worked example.
 * Every step must print exactly what the messy STEP 0 printed. If one byte differs,
 * that step was not a refactoring (it changed behaviour) and we exit with an error.
 */
public class Main {

    public static void main(String[] args) {
        Map<String, Supplier<String>> steps = new LinkedHashMap<>();
        steps.put("step0 (messy original)", lld.ch07.worked.step0.Main::run);
        steps.put("step1 extract + rename", lld.ch07.worked.step1.Main::run);
        steps.put("step2 split loop + pipeline", lld.ch07.worked.step2.Main::run);
        steps.put("step3 split phase", lld.ch07.worked.step3.Main::run);
        steps.put("step4 primitive -> object", lld.ch07.worked.step4.Main::run);
        steps.put("step5 polymorphism (final)", lld.ch07.worked.step5.Main::run);

        String golden = lld.ch07.worked.step0.Main.run();
        System.out.print("Golden output from step 0:\n" + golden);
        System.out.println("---");
        boolean allSame = true;
        for (Map.Entry<String, Supplier<String>> e : steps.entrySet()) {
            boolean same = golden.equals(e.getValue().get());
            allSame &= same;
            System.out.printf("%-30s %s%n", e.getKey(), same ? "SAME output" : "DIFFERENT output!");
        }
        if (!allSame) {
            System.err.println("A step changed behaviour - that was not a refactoring.");
            System.exit(1);
        }
        System.out.println("All steps preserve behaviour.");
    }
}
```

Worked example — step 0 (messy original)

```java
// File: code/worked/step0/Main.java
package lld.ch07.worked.step0;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

/** STEP 0 - the messy starting point. It works, but it smells. */
public class Main {

    // Mutable public data class (smells: Data Class, Mysterious Name)
    static class Shp {
        public String t;   // "STD", "EXP" or "FRG"
        public double kg;
        public int km;
        Shp(String t, double kg, int km) { this.t = t; this.kg = kg; this.km = km; }
    }

    static class Inv {
        public String cust;
        public List<Shp> list = new ArrayList<>();
    }

    // One Long Function that calculates AND formats (smells: Long Function,
    // Repeated Switches waiting to happen, magic numbers, temps everywhere)
    static String print(Inv inv) {
        double tot = 0;
        int pts = 0;
        int unused = 0;                                  // dead code
        StringBuilder r = new StringBuilder("Invoice for " + inv.cust + "\n");
        for (Shp s : inv.list) {
            double amt = 0;
            switch (s.t) {
                case "STD":
                    amt = 5 + s.kg * 1.2;
                    if (s.km > 100) amt += (s.km - 100) * 0.05;
                    break;
                case "EXP":
                    amt = 12 + s.kg * 2.0;
                    if (s.km > 50) amt += (s.km - 50) * 0.10;
                    break;
                case "FRG":
                    amt = 8 + s.kg * 1.5 + 3;
                    if (s.kg > 10) amt = amt * 1.2;
                    break;
                default:
                    throw new IllegalArgumentException("unknown type: " + s.t);
            }
            // loyalty points
            pts += s.km / 100;
            if ("EXP".equals(s.t)) pts += 2;
            r.append("  ").append(s.t).append(" ").append(s.kg).append("kg ")
             .append(s.km).append("km: ")
             .append(String.format(Locale.ROOT, "%.2f", amt)).append("\n");
            tot += amt;
        }
        r.append("Amount owed: ").append(String.format(Locale.ROOT, "%.2f", tot)).append("\n");
        r.append("Points earned: ").append(pts).append("\n");
        return r.toString();
    }

    /** Used by the characterisation check in lld.ch07.worked.Main. */
    public static String run() {
        Inv inv = new Inv();
        inv.cust = "Asha Traders";
        inv.list.add(new Shp("STD", 4.0, 250));
```

```
        inv.list.add(new Shp("EXP", 2.5, 80));
        inv.list.add(new Shp("FRG", 12.0, 40));
        inv.list.add(new Shp("STD", 1.0, 90));
        inv.list.add(new Shp("STD", 0.5, 100));   // boundary: exactly 100 km
        inv.list.add(new Shp("FRG", 10.0, 150));  // boundary: exactly 10 kg
        return print(inv);
    }

    public static void main(String[] args) {
        System.out.print(run());
    }
}
```

Worked example — step 1

```java
// File: code/worked/step1/Main.java
package lld.ch07.worked.step1;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

/**
 * STEP 1 - Extract Function + Rename Variable + Remove Dead Code.
 * The loop body is split into amountFor() and pointsFor(); names now say what they mean.
 */
public class Main {

    static class Shipment {
        public String type;          // Rename Field: t -> type
        public double weightKg;      // kg -> weightKg
        public int distanceKm;       // km -> distanceKm
        Shipment(String type, double weightKg, int distanceKm) {
            this.type = type; this.weightKg = weightKg; this.distanceKm = distanceKm;
        }
    }

    static class Invoice {
        public String customer;
        public List<Shipment> shipments = new ArrayList<>();
    }

    static String statement(Invoice invoice) {
        double totalAmount = 0;
        int loyaltyPoints = 0;
        StringBuilder result = new StringBuilder("Invoice for " + invoice.customer + "\n");
        for (Shipment shipment : invoice.shipments) {
            double thisAmount = amountFor(shipment);
            loyaltyPoints += pointsFor(shipment);
            result.append("  ").append(shipment.type).append(" ").append(shipment.weightKg).append("kg ")
                  .append(shipment.distanceKm).append("km: ").append(money(thisAmount)).append("\n");
            totalAmount += thisAmount;
        }
        result.append("Amount owed: ").append(money(totalAmount)).append("\n");
        result.append("Points earned: ").append(loyaltyPoints).append("\n");
        return result.toString();
    }

    /** Extracted from the loop body. The switch is unchanged - only moved. */
    static double amountFor(Shipment shipment) {
        double result;
        switch (shipment.type) {
            case "STD":
                result = 5 + shipment.weightKg * 1.2;
                if (shipment.distanceKm > 100) result += (shipment.distanceKm - 100) * 0.05;
                break;
            case "EXP":
                result = 12 + shipment.weightKg * 2.0;
                if (shipment.distanceKm > 50) result += (shipment.distanceKm - 50) * 0.10;
                break;
            case "FRG":
                result = 8 + shipment.weightKg * 1.5 + 3;
                if (shipment.weightKg > 10) result = result * 1.2;
                break;
            default:
                throw new IllegalArgumentException("unknown type: " + shipment.type);
        }
        return result;
    }

    static int pointsFor(Shipment shipment) {
```

```
        int result = shipment.distanceKm / 100;
        if ("EXP".equals(shipment.type)) result += 2;
        return result;
    }

    static String money(double amount) {
        return String.format(Locale.ROOT, "%.2f", amount);
    }

    public static String run() {
        Invoice invoice = new Invoice();
        invoice.customer = "Asha Traders";
        invoice.shipments.add(new Shipment("STD", 4.0, 250));
        invoice.shipments.add(new Shipment("EXP", 2.5, 80));
        invoice.shipments.add(new Shipment("FRG", 12.0, 40));
        invoice.shipments.add(new Shipment("STD", 1.0, 90));
        invoice.shipments.add(new Shipment("STD", 0.5, 100));
        invoice.shipments.add(new Shipment("FRG", 10.0, 150));
        return statement(invoice);
    }

    public static void main(String[] args) {
        System.out.print(run());
    }
}
```

Worked example — step 2

```java
// File: code/worked/step2/Main.java
package lld.ch07.worked.step2;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

/**
 * STEP 2 - Inline Variable (thisAmount), Split Loop, Replace Temp with Query,
 * Replace Loop with Pipeline, Slide Statements.
 * The printing loop now only prints; totals come from their own small queries.
 */
public class Main {

    static class Shipment {
        public String type;
        public double weightKg;
        public int distanceKm;
        Shipment(String type, double weightKg, int distanceKm) {
            this.type = type; this.weightKg = weightKg; this.distanceKm = distanceKm;
        }
    }

    static class Invoice {
        public String customer;
        public List<Shipment> shipments = new ArrayList<>();
    }

    static String statement(Invoice invoice) {
        StringBuilder result = new StringBuilder("Invoice for " + invoice.customer + "\n");
        for (Shipment shipment : invoice.shipments) {
            result.append("  ").append(shipment.type).append(" ").append(shipment.weightKg).append("kg ")
                  .append(shipment.distanceKm).append("km: ").append(money(amountFor(shipment))).append("\n");
        }
        result.append("Amount owed: ").append(money(totalAmount(invoice))).append("\n");
        result.append("Points earned: ").append(totalPoints(invoice)).append("\n");
        return result.toString();
    }

    // reduce(0.0, Double::sum) is a plain left-to-right sum, exactly like the old loop.
    // DoubleStream.sum() may use compensated summation and could differ in the last bit,
    // which would NOT be a pure refactoring.
    static double totalAmount(Invoice invoice) {
        return invoice.shipments.stream().map(Main::amountFor).reduce(0.0, Double::sum);
    }

    static int totalPoints(Invoice invoice) {
        return invoice.shipments.stream().mapToInt(Main::pointsFor).sum();
    }

    static double amountFor(Shipment shipment) {
        double result;
        switch (shipment.type) {
            case "STD":
                result = 5 + shipment.weightKg * 1.2;
                if (shipment.distanceKm > 100) result += (shipment.distanceKm - 100) * 0.05;
                break;
            case "EXP":
                result = 12 + shipment.weightKg * 2.0;
                if (shipment.distanceKm > 50) result += (shipment.distanceKm - 50) * 0.10;
                break;
            case "FRG":
                result = 8 + shipment.weightKg * 1.5 + 3;
                if (shipment.weightKg > 10) result = result * 1.2;
                break;
            default:
```

```
                throw new IllegalArgumentException("unknown type: " + shipment.type);
        }
        return result;
    }

    static int pointsFor(Shipment shipment) {
        int result = shipment.distanceKm / 100;
        if ("EXP".equals(shipment.type)) result += 2;
        return result;
    }

    static String money(double amount) {
        return String.format(Locale.ROOT, "%.2f", amount);
    }

    public static String run() {
        Invoice invoice = new Invoice();
        invoice.customer = "Asha Traders";
        invoice.shipments.add(new Shipment("STD", 4.0, 250));
        invoice.shipments.add(new Shipment("EXP", 2.5, 80));
        invoice.shipments.add(new Shipment("FRG", 12.0, 40));
        invoice.shipments.add(new Shipment("STD", 1.0, 90));
        invoice.shipments.add(new Shipment("STD", 0.5, 100));
        invoice.shipments.add(new Shipment("FRG", 10.0, 150));
        return statement(invoice);
    }

    public static void main(String[] args) {
        System.out.print(run());
    }
}
```

Worked example — step 3

```
// File: code/worked/step3/Shipment.java
package lld.ch07.worked.step3;

/** Encapsulate Record: the public mutable class became an immutable record. */
public record Shipment(String type, double weightKg, int distanceKm) { }
```

```
// File: code/worked/step3/Invoice.java
package lld.ch07.worked.step3;

import java.util.List;

/** Encapsulate Collection: callers get an unmodifiable copy, never the live list. */
public record Invoice(String customer, List<Shipment> shipments) {
    public Invoice {
        shipments = List.copyOf(shipments);
    }
}
```

```
// File: code/worked/step3/StatementData.java
package lld.ch07.worked.step3;

import java.util.List;

/** The intermediate data structure produced by phase 1 and consumed by phase 2 (Split Phase). */
public record StatementData(String customer, List<Line> lines, double totalAmount, int totalPoints) {
    public record Line(String type, double weightKg, int distanceKm, double amount) { }
}
```

```java
// File: code/worked/step3/StatementCalculator.java
package lld.ch07.worked.step3;

import java.util.List;

/** Phase 1: pure calculation. Combine Functions into Transform - all derived values in one place. */
public final class StatementCalculator {

    public StatementData calculate(Invoice invoice) {
        List<StatementData.Line> lines = invoice.shipments().stream()
                .map(s -> new StatementData.Line(s.type(), s.weightKg(), s.distanceKm(), amountFor(s)))
                .toList();
        double total = lines.stream().map(StatementData.Line::amount).reduce(0.0, Double::sum);
        int points = invoice.shipments().stream().mapToInt(this::pointsFor).sum();
        return new StatementData(invoice.customer(), lines, total, points);
    }

    double amountFor(Shipment shipment) {
        double result;
        switch (shipment.type()) {
            case "STD":
                result = 5 + shipment.weightKg() * 1.2;
                if (shipment.distanceKm() > 100) result += (shipment.distanceKm() - 100) * 0.05;
                break;
            case "EXP":
                result = 12 + shipment.weightKg() * 2.0;
                if (shipment.distanceKm() > 50) result += (shipment.distanceKm() - 50) * 0.10;
                break;
            case "FRG":
                result = 8 + shipment.weightKg() * 1.5 + 3;
                if (shipment.weightKg() > 10) result = result * 1.2;
                break;
            default:
                throw new IllegalArgumentException("unknown type: " + shipment.type());
        }
        return result;
    }

    int pointsFor(Shipment shipment) {
        int result = shipment.distanceKm() / 100;
        if ("EXP".equals(shipment.type())) result += 2;
        return result;
    }
}
```

```java
// File: code/worked/step3/TextRenderer.java
package lld.ch07.worked.step3;

import java.util.Locale;

/** Phase 2: pure formatting. Knows nothing about tariffs. */
public final class TextRenderer {

    public String render(StatementData data) {
        StringBuilder result = new StringBuilder("Invoice for " + data.customer() + "\n");
        for (StatementData.Line line : data.lines()) {
            result.append("  ").append(line.type()).append(" ").append(line.weightKg()).append("kg ")
                  .append(line.distanceKm()).append("km: ").append(money(line.amount())).append("\n");
        }
        result.append("Amount owed: ").append(money(data.totalAmount())).append("\n");
        result.append("Points earned: ").append(data.totalPoints()).append("\n");
        return result.toString();
    }

    static String money(double amount) {
        return String.format(Locale.ROOT, "%.2f", amount);
    }
}
```

```java
// File: code/worked/step3/Main.java
package lld.ch07.worked.step3;

import java.util.List;

/** STEP 3 - Split Phase: calculate (StatementCalculator) then render (TextRenderer). */
public class Main {

    public static String run() {
        Invoice invoice = new Invoice("Asha Traders", List.of(
                new Shipment("STD", 4.0, 250),
                new Shipment("EXP", 2.5, 80),
                new Shipment("FRG", 12.0, 40),
                new Shipment("STD", 1.0, 90),
                new Shipment("STD", 0.5, 100),
                new Shipment("FRG", 10.0, 150)));
        return new TextRenderer().render(new StatementCalculator().calculate(invoice));
    }

    public static void main(String[] args) {
        System.out.print(run());
    }
}
```

Worked example — step 4

```java
// File: code/worked/step4/ServiceType.java
package lld.ch07.worked.step4;

/** Replace Primitive with Object (type code String -> enum). Invalid codes fail at the boundary. */
public enum ServiceType {
    STANDARD("STD"), EXPRESS("EXP"), FRAGILE("FRG");

    private final String code;

    ServiceType(String code) { this.code = code; }

    public String code() { return code; }

    public static ServiceType fromCode(String code) {
        for (ServiceType t : values()) {
            if (t.code.equals(code)) return t;
        }
        throw new IllegalArgumentException("unknown type: " + code);
    }
}
```

```java
// File: code/worked/step4/Shipment.java
package lld.ch07.worked.step4;

import java.util.Objects;

public record Shipment(ServiceType type, double weightKg, int distanceKm) {
    public Shipment {
        Objects.requireNonNull(type, "type");
    }
}
```

```java
// File: code/worked/step4/Invoice.java
package lld.ch07.worked.step4;

import java.util.List;

public record Invoice(String customer, List<Shipment> shipments) {
    public Invoice {
        shipments = List.copyOf(shipments);
    }
}
```

```java
// File: code/worked/step4/StatementData.java
package lld.ch07.worked.step4;

import java.util.List;

public record StatementData(String customer, List<Line> lines, double totalAmount, int totalPoints) {
    public record Line(ServiceType type, double weightKg, int distanceKm, double amount) { }
}
```

```java
// File: code/worked/step4/StatementCalculator.java
package lld.ch07.worked.step4;

import java.util.List;

/** Replace Magic Literal: every tariff number now has a name. The switch is exhaustive over the enum. */
public final class StatementCalculator {

    static final double STD_BASE = 5, STD_PER_KG = 1.2, STD_FREE_KM = 100, STD_PER_EXTRA_KM = 0.05;
    static final double EXP_BASE = 12, EXP_PER_KG = 2.0, EXP_FREE_KM = 50, EXP_PER_EXTRA_KM = 0.10;
    static final double FRG_BASE = 8, FRG_PER_KG = 1.5, FRG_HANDLING = 3, FRG_HEAVY_KG = 10, FRG_HEAVY_FACTOR =
1.2;
    static final int KM_PER_POINT = 100, EXPRESS_BONUS_POINTS = 2;

    public StatementData calculate(Invoice invoice) {
        List<StatementData.Line> lines = invoice.shipments().stream()
                .map(s -> new StatementData.Line(s.type(), s.weightKg(), s.distanceKm(), amountFor(s)))
                .toList();
        double total = lines.stream().map(StatementData.Line::amount).reduce(0.0, Double::sum);
        int points = invoice.shipments().stream().mapToInt(this::pointsFor).sum();
        return new StatementData(invoice.customer(), lines, total, points);
    }

    double amountFor(Shipment s) {
        return switch (s.type()) {
            case STANDARD -> STD_BASE + s.weightKg() * STD_PER_KG
                    + (s.distanceKm() > STD_FREE_KM ? (s.distanceKm() - STD_FREE_KM) * STD_PER_EXTRA_KM : 0);
            case EXPRESS -> EXP_BASE + s.weightKg() * EXP_PER_KG
                    + (s.distanceKm() > EXP_FREE_KM ? (s.distanceKm() - EXP_FREE_KM) * EXP_PER_EXTRA_KM : 0);
            case FRAGILE -> {
                double base = FRG_BASE + s.weightKg() * FRG_PER_KG + FRG_HANDLING;
                yield s.weightKg() > FRG_HEAVY_KG ? base * FRG_HEAVY_FACTOR : base;
            }
        };
    }

    int pointsFor(Shipment s) {
        int result = s.distanceKm() / KM_PER_POINT;
        if (s.type() == ServiceType.EXPRESS) result += EXPRESS_BONUS_POINTS;
        return result;
    }
}
```

```java
// File: code/worked/step4/TextRenderer.java
package lld.ch07.worked.step4;

import java.util.Locale;

public final class TextRenderer {

    public String render(StatementData data) {
        StringBuilder result = new StringBuilder("Invoice for " + data.customer() + "\n");
        for (StatementData.Line line : data.lines()) {
            result.append("  ").append(line.type().code()).append(" ").append(line.weightKg()).append("kg ")
                  .append(line.distanceKm()).append("km: ").append(money(line.amount())).append("\n");
        }
        result.append("Amount owed: ").append(money(data.totalAmount())).append("\n");
        result.append("Points earned: ").append(data.totalPoints()).append("\n");
        return result.toString();
    }

    static String money(double amount) {
        return String.format(Locale.ROOT, "%.2f", amount);
    }
}
```

```java
// File: code/worked/step4/Main.java
package lld.ch07.worked.step4;

import java.util.List;

/** STEP 4 - Replace Primitive with Object (ServiceType enum) + Replace Magic Literal. */
public class Main {

    public static String run() {
        Invoice invoice = new Invoice("Asha Traders", List.of(
                new Shipment(ServiceType.fromCode("STD"), 4.0, 250),
                new Shipment(ServiceType.fromCode("EXP"), 2.5, 80),
                new Shipment(ServiceType.fromCode("FRG"), 12.0, 40),
                new Shipment(ServiceType.fromCode("STD"), 1.0, 90),
                new Shipment(ServiceType.fromCode("STD"), 0.5, 100),
                new Shipment(ServiceType.fromCode("FRG"), 10.0, 150)));
        return new TextRenderer().render(new StatementCalculator().calculate(invoice));
    }

    public static void main(String[] args) {
        System.out.print(run());
    }
}
```

Worked example — step 5 (final)

```java
// File: code/worked/step5/Tariff.java
package lld.ch07.worked.step5;

/**
 * Replace Conditional with Polymorphism: each branch of the old switch became a class.
 * A sealed interface lists every permitted implementation, so the compiler knows the full set.
 */
public sealed interface Tariff permits StandardTariff, ExpressTariff, FragileTariff {

    double charge(double weightKg, int distanceKm);

    /** Default behaviour shared by most tariffs; ExpressTariff overrides it. */
    default int loyaltyPoints(int distanceKm) {
        return distanceKm / 100;
    }
}
```

```java
// File: code/worked/step5/StandardTariff.java
package lld.ch07.worked.step5;

public record StandardTariff() implements Tariff {
    private static final double BASE = 5, PER_KG = 1.2, FREE_KM = 100, PER_EXTRA_KM = 0.05;

    @Override
    public double charge(double weightKg, int distanceKm) {
        return BASE + weightKg * PER_KG + (distanceKm > FREE_KM ? (distanceKm - FREE_KM) * PER_EXTRA_KM : 0);
    }
}
```

```java
// File: code/worked/step5/ExpressTariff.java
package lld.ch07.worked.step5;

public record ExpressTariff() implements Tariff {
    private static final double BASE = 12, PER_KG = 2.0, FREE_KM = 50, PER_EXTRA_KM = 0.10;
    private static final int BONUS_POINTS = 2;

    @Override
    public double charge(double weightKg, int distanceKm) {
        return BASE + weightKg * PER_KG + (distanceKm > FREE_KM ? (distanceKm - FREE_KM) * PER_EXTRA_KM : 0);
    }

    @Override
    public int loyaltyPoints(int distanceKm) {
        return Tariff.super.loyaltyPoints(distanceKm) + BONUS_POINTS;
    }
}
```

```java
// File: code/worked/step5/FragileTariff.java
package lld.ch07.worked.step5;

public record FragileTariff() implements Tariff {
    private static final double BASE = 8, PER_KG = 1.5, HANDLING = 3, HEAVY_KG = 10, HEAVY_FACTOR = 1.2;

    @Override
    public double charge(double weightKg, int distanceKm) {
        double base = BASE + weightKg * PER_KG + HANDLING;
        return weightKg > HEAVY_KG ? base * HEAVY_FACTOR : base;
    }
}
```

```java
// File: code/worked/step5/ServiceType.java
package lld.ch07.worked.step5;

/** The enum now just names a service and points at its Tariff (Strategy). */
public enum ServiceType {
    STANDARD("STD", new StandardTariff()),
    EXPRESS("EXP", new ExpressTariff()),
    FRAGILE("FRG", new FragileTariff());

    private final String code;
    private final Tariff tariff;

    ServiceType(String code, Tariff tariff) { this.code = code; this.tariff = tariff; }

    public String code() { return code; }
    public Tariff tariff() { return tariff; }

    public static ServiceType fromCode(String code) {
        for (ServiceType t : values()) {
            if (t.code.equals(code)) return t;
        }
        throw new IllegalArgumentException("unknown type: " + code);
    }
}
```

```java
// File: code/worked/step5/Shipment.java
package lld.ch07.worked.step5;

import java.util.Objects;

/** Move Function: the shipment now answers its own charge and points (cures Feature Envy). */
public record Shipment(ServiceType type, double weightKg, int distanceKm) {
    public Shipment {
        Objects.requireNonNull(type, "type");
    }

    public double charge() { return type.tariff().charge(weightKg, distanceKm); }

    public int loyaltyPoints() { return type.tariff().loyaltyPoints(distanceKm); }
}
```

```java
// File: code/worked/step5/Invoice.java
package lld.ch07.worked.step5;

import java.util.List;

public record Invoice(String customer, List<Shipment> shipments) {
    public Invoice {
        shipments = List.copyOf(shipments);
    }
}
```

```java
// File: code/worked/step5/StatementData.java
package lld.ch07.worked.step5;

import java.util.List;

public record StatementData(String customer, List<Line> lines, double totalAmount, int totalPoints) {
    public record Line(ServiceType type, double weightKg, int distanceKm, double amount) { }
}
```

```java
// File: code/worked/step5/StatementCalculator.java
package lld.ch07.worked.step5;

import java.util.List;

/** Phase 1. No switch left: adding a tariff never touches this class (Open/Closed). */
public final class StatementCalculator {

    public StatementData calculate(Invoice invoice) {
        List<StatementData.Line> lines = invoice.shipments().stream()
                .map(s -> new StatementData.Line(s.type(), s.weightKg(), s.distanceKm(), s.charge()))
                .toList();
        double total = lines.stream().map(StatementData.Line::amount).reduce(0.0, Double::sum);
        int points = invoice.shipments().stream().mapToInt(Shipment::loyaltyPoints).sum();
        return new StatementData(invoice.customer(), lines, total, points);
    }
}
```

```java
// File: code/worked/step5/StatementRenderer.java
package lld.ch07.worked.step5;

import java.util.Locale;

/** Phase 2 abstraction. Extracted once a second renderer (HTML) was needed. */
public interface StatementRenderer {

    String render(StatementData data);

    static String money(double amount) {
        return String.format(Locale.ROOT, "%.2f", amount);
    }
}
```

```java
// File: code/worked/step5/TextRenderer.java
package lld.ch07.worked.step5;

import static lld.ch07.worked.step5.StatementRenderer.money;

public final class TextRenderer implements StatementRenderer {

    @Override
    public String render(StatementData data) {
        StringBuilder result = new StringBuilder("Invoice for " + data.customer() + "\n");
        for (StatementData.Line line : data.lines()) {
            result.append("  ").append(line.type().code()).append(" ").append(line.weightKg()).append("kg ")
                  .append(line.distanceKm()).append("km: ").append(money(line.amount())).append("\n");
        }
        result.append("Amount owed: ").append(money(data.totalAmount())).append("\n");
        result.append("Points earned: ").append(data.totalPoints()).append("\n");
        return result.toString();
    }
}
```

```java
// File: code/worked/step5/HtmlRenderer.java
package lld.ch07.worked.step5;

import static lld.ch07.worked.step5.StatementRenderer.money;

/** The NEW feature. Thanks to Split Phase it reuses the calculator untouched. */
public final class HtmlRenderer implements StatementRenderer {

    @Override
    public String render(StatementData data) {
        StringBuilder result = new StringBuilder("<h1>Invoice for " + data.customer() + "</h1>\n<table>\n");
        for (StatementData.Line line : data.lines()) {
            result.append("  <tr><td>").append(line.type().code()).append("</td><td>")
                  .append(line.weightKg()).append("kg</td><td>").append(line.distanceKm())
                  .append("km</td><td>").append(money(line.amount())).append("</td></tr>\n");
        }
        result.append("</table>\n<p>Amount owed: <b>").append(money(data.totalAmount())).append("</b></p>\n");
        result.append("<p>Points earned: <b>").append(data.totalPoints()).append("</b></p>\n");
        return result.toString();
    }
}
```

```java
// File: code/worked/step5/Main.java
package lld.ch07.worked.step5;

import java.util.List;

/** STEP 5 (final) - Replace Conditional with Polymorphism + Move Function, then add the HTML feature. */
public class Main {

    static Invoice sampleInvoice() {
        return new Invoice("Asha Traders", List.of(
                new Shipment(ServiceType.fromCode("STD"), 4.0, 250),
                new Shipment(ServiceType.fromCode("EXP"), 2.5, 80),
                new Shipment(ServiceType.fromCode("FRG"), 12.0, 40),
                new Shipment(ServiceType.fromCode("STD"), 1.0, 90),
                new Shipment(ServiceType.fromCode("STD"), 0.5, 100),
                new Shipment(ServiceType.fromCode("FRG"), 10.0, 150)));
    }

    public static String run() {
        return new TextRenderer().render(new StatementCalculator().calculate(sampleInvoice()));
    }

    public static void main(String[] args) {
        System.out.print(run());
        System.out.println("--- new feature: HTML statement ---");
        System.out.print(new HtmlRenderer().render(new StatementCalculator().calculate(sampleInvoice())));
    }
}
```

Output (`java -cp /tmp/lld_07 lld.ch07.worked.step5.Main`):

```
Invoice for Asha Traders
  STD 4.0kg 250km: 17.30
  EXP 2.5kg 80km: 20.00
  FRG 12.0kg 40km: 34.80
  STD 1.0kg 90km: 6.20
  STD 0.5kg 100km: 5.60
  FRG 10.0kg 150km: 26.00
Amount owed: 109.90
Points earned: 6
--- new feature: HTML statement ---
<h1>Invoice for Asha Traders</h1>
<table>
  <tr><td>STD</td><td>4.0kg</td><td>250km</td><td>17.30</td></tr>
  <tr><td>EXP</td><td>2.5kg</td><td>80km</td><td>20.00</td></tr>
  <tr><td>FRG</td><td>12.0kg</td><td>40km</td><td>34.80</td></tr>
  <tr><td>STD</td><td>1.0kg</td><td>90km</td><td>6.20</td></tr>
  <tr><td>STD</td><td>0.5kg</td><td>100km</td><td>5.60</td></tr>
  <tr><td>FRG</td><td>10.0kg</td><td>150km</td><td>26.00</td></tr>
</table>
<p>Amount owed: <b>109.90</b></p>
<p>Points earned: <b>6</b></p>
```

Output of `lld.ch07.worked.Main` is shown in 7.5. Steps 0–4 each print the same text invoice as the first nine lines above.

Refactoring toward patterns

```java
// File: code/towardpatterns/StrategyRefactoring.java
package lld.ch07.towardpatterns;

import java.util.EnumMap;
import java.util.Map;

/** Replace Conditional Logic with Strategy. */
public final class StrategyRefactoring {

    // ---------- BEFORE: a switch that every new tier must edit ----------
    static double legacyPrice(String tier, double amount) {
        switch (tier) {
            case "GOLD":   return amount * 0.80;
            case "SILVER": return amount * 0.90;
            case "NEW":    return amount > 100 ? amount - 10 : amount;
            default:       return amount;
        }
    }

    // ---------- AFTER: one small class (here a lambda) per rule ----------
    @FunctionalInterface
    interface DiscountPolicy {
        double apply(double amount);
    }

    enum Tier { GOLD, SILVER, NEW, REGULAR }

    static final class PriceCalculator {
        private final Map<Tier, DiscountPolicy> policies = new EnumMap<>(Tier.class);
        private final DiscountPolicy fallback = amount -> amount;

        PriceCalculator() {
            policies.put(Tier.GOLD, amount -> amount * 0.80);
            policies.put(Tier.SILVER, amount -> amount * 0.90);
            policies.put(Tier.NEW, amount -> amount > 100 ? amount - 10 : amount);
        }

        /** New tiers are registered, not coded into a switch (Open/Closed). */
        void register(Tier tier, DiscountPolicy policy) { policies.put(tier, policy); }

        double price(Tier tier, double amount) {
            return policies.getOrDefault(tier, fallback).apply(amount);
        }
    }

    static boolean demo() {
        PriceCalculator calc = new PriceCalculator();
        boolean same = true;
        for (Tier t : Tier.values()) {
            for (double amount : new double[] {50, 100, 250}) {
                double before = legacyPrice(t.name(), amount);
                double after = calc.price(t, amount);
                same &= before == after;
            }
        }
        System.out.println("Strategy:        GOLD 250 -> " + calc.price(Tier.GOLD, 250)
                + ", NEW 250 -> " + calc.price(Tier.NEW, 250) + " | same as switch: " + same);
        return same;
    }
}
```

```java
// File: code/towardpatterns/StateRefactoring.java
package lld.ch07.towardpatterns;

/** Replace State-Altering Conditionals with State. */
public final class StateRefactoring {

    // ---------- BEFORE: every method re-checks a status string ----------
    static final class LegacyOrder {
        String status = "NEW";

        String pay() {
            if (status.equals("NEW")) { status = "PAID"; return "paid"; }
            return "cannot pay when " + status;
        }
        String ship() {
            if (status.equals("PAID")) { status = "SHIPPED"; return "shipped"; }
            return "cannot ship when " + status;
        }
        String cancel() {
            if (status.equals("NEW") || status.equals("PAID")) { status = "CANCELLED"; return "cancelled"; }
            return "cannot cancel when " + status;
        }
    }

    // ---------- AFTER: each state is an object that knows its own legal moves ----------
    interface OrderState {
        String name();
        default OrderState pay(Order o)    { return o.reject("pay", this); }
        default OrderState ship(Order o)   { return o.reject("ship", this); }
        default OrderState cancel(Order o) { return o.reject("cancel", this); }
    }

    enum States implements OrderState {
        NEW {
            @Override public OrderState pay(Order o)    { o.log("paid"); return PAID; }
            @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
        },
        PAID {
            @Override public OrderState ship(Order o)   { o.log("shipped"); return SHIPPED; }
            @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
        },
        SHIPPED, CANCELLED
    }

    static final class Order {
        private OrderState state = States.NEW;
        private String lastMessage = "";

        String pay()    { state = state.pay(this);    return lastMessage; }
        String ship()   { state = state.ship(this);   return lastMessage; }
        String cancel() { state = state.cancel(this); return lastMessage; }

        void log(String msg) { lastMessage = msg; }

        OrderState reject(String action, OrderState current) {
            lastMessage = "cannot " + action + " when " + current.name();
            return current;                       // stay in the same state
        }
    }

    static boolean demo() {
        LegacyOrder before = new LegacyOrder();
        Order after = new Order();
        String[] script = {"ship", "pay", "pay", "ship", "cancel"};
        boolean same = true;
        StringBuilder trace = new StringBuilder();
        for (String step : script) {
            String b = switch (step) { case "pay" -> before.pay(); case "ship" -> before.ship(); default ->
```

```
before.cancel(); };
            String a = switch (step) { case "pay" -> after.pay();  case "ship" -> after.ship();  default ->
after.cancel(); };
            same &= a.equals(b);
            trace.append(a).append("; ");
        }
        System.out.println("State:           " + trace + "| same as if-chains: " + same);
        return same;
    }
}
```

```
// File: code/towardpatterns/SpecialCaseRefactoring.java
package lld.ch07.towardpatterns;

import java.util.Map;

/** Introduce Special Case (Null Object): stop repeating "if (customer == null)" at every call site. */
public final class SpecialCaseRefactoring {

    // ---------- BEFORE ----------
    record LegacyCustomer(String name, String plan) { }

    static final Map<String, LegacyCustomer> LEGACY_DB = Map.of("c1", new LegacyCustomer("Ravi", "premium"));

    static String legacyGreeting(String id) {
        LegacyCustomer c = LEGACY_DB.get(id);
        String name = (c == null) ? "occupant" : c.name();       // null check #1
        String plan = (c == null) ? "basic" : c.plan();          // null check #2 ... and so on in every client
        return "Hello " + name + " on " + plan;
    }

    // ---------- AFTER ----------
    sealed interface Customer permits KnownCustomer, UnknownCustomer {
        String name();
        String plan();
    }
    record KnownCustomer(String name, String plan) implements Customer { }
    /** The special case: one place holds the default behaviour. */
    record UnknownCustomer() implements Customer {
        @Override public String name() { return "occupant"; }
        @Override public String plan() { return "basic"; }
    }

    static final Map<String, Customer> DB = Map.of("c1", new KnownCustomer("Ravi", "premium"));

    static Customer find(String id) {
        return DB.getOrDefault(id, new UnknownCustomer());
    }

    static String greeting(String id) {
        Customer c = find(id);                                   // no null checks at all
        return "Hello " + c.name() + " on " + c.plan();
    }

    static boolean demo() {
        boolean same = legacyGreeting("c1").equals(greeting("c1")) && legacyGreeting("zz").equals(greeting("zz"));
        System.out.println("Special Case:    " + greeting("c1") + " / " + greeting("zz") + " | same: " + same);
        return same;
    }
}
```

```java
// File: code/towardpatterns/TemplateMethodRefactoring.java
package lld.ch07.towardpatterns;

import java.util.List;

/** Form Template Method: two siblings with the same algorithm skeleton but different details. */
public final class TemplateMethodRefactoring {

    // ---------- BEFORE: duplicated skeleton ----------
    static String legacyCsv(List<String> rows) {
        StringBuilder sb = new StringBuilder();
        sb.append("name\n");                                  // header
        for (String r : rows) sb.append(r).append("\n");      // body
        sb.append("# ").append(rows.size()).append(" rows");  // footer
        return sb.toString();
    }
    static String legacyMarkdown(List<String> rows) {
        StringBuilder sb = new StringBuilder();
        sb.append("| name |\n|---|\n");
        for (String r : rows) sb.append("| ").append(r).append(" |\n");
        sb.append("_").append(rows.size()).append(" rows_");
        return sb.toString();
    }

    // ---------- AFTER: the skeleton lives once, subclasses fill in the steps ----------
    abstract static class Exporter {
        /** The template method - final so subclasses cannot change the order of steps. */
        final String export(List<String> rows) {
            StringBuilder sb = new StringBuilder(header());
            for (String r : rows) sb.append(row(r));
            return sb.append(footer(rows.size())).toString();
        }
        protected abstract String header();
        protected abstract String row(String value);
        protected abstract String footer(int count);
    }
    static final class CsvExporter extends Exporter {
        protected String header() { return "name\n"; }
        protected String row(String v) { return v + "\n"; }
        protected String footer(int n) { return "# " + n + " rows"; }
    }
    static final class MarkdownExporter extends Exporter {
        protected String header() { return "| name |\n|---|\n"; }
        protected String row(String v) { return "| " + v + " |\n"; }
        protected String footer(int n) { return "_" + n + " rows_"; }
    }

    static boolean demo() {
        List<String> rows = List.of("Asha", "Ravi");
        boolean same = legacyCsv(rows).equals(new CsvExporter().export(rows))
                && legacyMarkdown(rows).equals(new MarkdownExporter().export(rows));
        System.out.println("Template Method: CSV and Markdown exporters | same: " + same);
        return same;
    }
}
```

```java
// File: code/towardpatterns/FactoryRefactoring.java
package lld.ch07.towardpatterns;

/** Replace Constructor with Factory Function (Kerievsky: Replace Constructors with Creation Methods). */
public final class FactoryRefactoring {

    // ---------- BEFORE: which boolean means what? ----------
    static final class LegacyLoan {
        final double amount; final boolean revolving; final int months;
        LegacyLoan(double amount, boolean revolving, int months) {
            this.amount = amount; this.revolving = revolving; this.months = months;
        }
        @Override public String toString() { return (revolving ? "revolver" : "term") + " " + amount + "/" +
months; }
    }

    // ---------- AFTER: intention-revealing creation methods, private constructor ----------
    static final class Loan {
        private final double amount; private final boolean revolving; private final int months;
        private Loan(double amount, boolean revolving, int months) {
            this.amount = amount; this.revolving = revolving; this.months = months;
        }
        static Loan termLoan(double amount, int months) { return new Loan(amount, false, months); }
        static Loan revolver(double limit)              { return new Loan(limit, true, 0); }
        @Override public String toString() { return (revolving ? "revolver" : "term") + " " + amount + "/" +
months; }
    }

    static boolean demo() {
        boolean same = new LegacyLoan(5000, false, 12).toString().equals(Loan.termLoan(5000, 12).toString())
                && new LegacyLoan(2000, true, 0).toString().equals(Loan.revolver(2000).toString());
        System.out.println("Factory:         " + Loan.termLoan(5000, 12) + ", " + Loan.revolver(2000) + " | same: "
+ same);
        return same;
    }
}
```

```java
// File: code/towardpatterns/Main.java
package lld.ch07.towardpatterns;

/** Refactoring toward patterns: each demo runs the BEFORE and AFTER code and checks they agree. */
public class Main {
    public static void main(String[] args) {
        boolean ok = StrategyRefactoring.demo()
                & StateRefactoring.demo()
                & SpecialCaseRefactoring.demo()
                & TemplateMethodRefactoring.demo()
                & FactoryRefactoring.demo();
        System.out.println(ok ? "All pattern refactorings preserve behaviour." : "BEHAVIOUR CHANGED!");
        if (!ok) System.exit(1);
    }
}
```

Output:

```
Strategy:        GOLD 250 -> 200.0, NEW 250 -> 240.0 | same as switch: true
State:           cannot ship when NEW; paid; cannot pay when PAID; shipped; cannot cancel when SHIPPED; | same as
if-chains: true
Special Case:    Hello Ravi on premium / Hello occupant on basic | same: true
Template Method: CSV and Markdown exporters | same: true
Factory:         term 5000.0/12, revolver 2000.0/0 | same: true
All pattern refactorings preserve behaviour.
```

Smell cures

```java
// File: code/smells/Money.java
package lld.ch07.smells;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Currency;
import java.util.Objects;

/** Cure for Primitive Obsession: a value object instead of a bare double. */
public record Money(BigDecimal amount, Currency currency) {

    public Money {
        Objects.requireNonNull(amount, "amount");
        Objects.requireNonNull(currency, "currency");
        amount = amount.setScale(currency.getDefaultFractionDigits(), RoundingMode.HALF_EVEN);
    }

    public static Money of(String amount, String currencyCode) {
        return new Money(new BigDecimal(amount), Currency.getInstance(currencyCode));
    }

    public Money plus(Money other) {
        requireSameCurrency(other);
        return new Money(amount.add(other.amount), currency);
    }

    public Money times(int quantity) {
        return new Money(amount.multiply(BigDecimal.valueOf(quantity)), currency);
    }

    private void requireSameCurrency(Money other) {
        if (!currency.equals(other.currency)) {
            throw new IllegalArgumentException("currency mismatch: " + currency + " vs " + other.currency);
        }
    }

    @Override public String toString() { return currency.getCurrencyCode() + " " + amount.toPlainString(); }
}
```

```java
// File: code/smells/DateRange.java
package lld.ch07.smells;

import java.time.LocalDate;

/** Cure for Data Clumps: start and end always travelled together, so they become one object. */
public record DateRange(LocalDate start, LocalDate end) {

    public DateRange {
        if (end.isBefore(start)) throw new IllegalArgumentException("end before start");
    }

    public boolean contains(LocalDate day) {
        return !day.isBefore(start) && !day.isAfter(end);
    }

    public boolean overlaps(DateRange other) {
        return !other.end.isBefore(start) && !other.start.isAfter(end);
    }
}
```

```java
// File: code/smells/Order.java
package lld.ch07.smells;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Cures for Message Chains (Hide Delegate), Feature Envy (Move Function)
 * and Mutable Data (Encapsulate Collection).
 */
public final class Order {

    public record Address(String street, String city) { }
    public record Customer(String name, Address address) { }
    public record Line(String sku, Money unitPrice, int quantity) {
        /** Moved here from Order: it only used Line's data (Feature Envy cured). */
        public Money total() { return unitPrice.times(quantity); }
    }

    private final Customer customer;
    private final List<Line> lines = new ArrayList<>();

    public Order(Customer customer) { this.customer = customer; }

    /** Hide Delegate: callers ask the order, not order.customer().address().city(). */
    public String shippingCity() { return customer.address().city(); }

    /** Encapsulate Collection: modification goes through the owner... */
    public void add(Line line) { lines.add(line); }

    /** ...and readers get a read-only view. */
    public List<Line> lines() { return Collections.unmodifiableList(lines); }

    public Money total() {
        return lines.stream().map(Line::total).reduce(Money::plus).orElse(Money.of("0", "INR"));
    }
}
```

```java
// File: code/smells/Main.java
package lld.ch07.smells;

import java.time.LocalDate;

/** Small, runnable cures for common smells. */
public class Main {
    public static void main(String[] args) {
        // Primitive Obsession -> Money: 0.1 + 0.2 is exact, currencies cannot be mixed by accident
        System.out.println("double 0.1+0.2       = " + (0.1 + 0.2));
        System.out.println("Money  0.10+0.20     = " + Money.of("0.10", "USD").plus(Money.of("0.20", "USD")));
        try {
            Money.of("1", "USD").plus(Money.of("1", "EUR"));
        } catch (IllegalArgumentException e) {
            System.out.println("mixing currencies    -> " + e.getMessage());
        }

        // Data Clumps -> DateRange
        DateRange booking = new DateRange(LocalDate.of(2026, 10, 1), LocalDate.of(2026, 10, 5));
        DateRange other = new DateRange(LocalDate.of(2026, 10, 5), LocalDate.of(2026, 10, 8));
        System.out.println("booking contains 3rd = " + booking.contains(LocalDate.of(2026, 10, 3))
                + ", overlaps other = " + booking.overlaps(other));

        // Message Chains, Feature Envy, Mutable Data
        Order order = new Order(new Order.Customer("Asha", new Order.Address("MG Road", "Bengaluru")));
        order.add(new Order.Line("PEN", Money.of("25.50", "INR"), 4));
        order.add(new Order.Line("BOOK", Money.of("199", "INR"), 1));
        System.out.println("ship to " + order.shippingCity() + ", total " + order.total());
        try {
            order.lines().clear();
        } catch (UnsupportedOperationException e) {
            System.out.println("outsider tried lines().clear() -> UnsupportedOperationException (collection is
encapsulated)");
        }
    }
}
```

Output:

```
double 0.1+0.2       = 0.30000000000000004
Money  0.10+0.20     = USD 0.30
mixing currencies    -> currency mismatch: USD vs EUR
booking contains 3rd = true, overlaps other = true
ship to Bengaluru, total INR 301.00
outsider tried lines().clear() -> UnsupportedOperationException (collection is encapsulated)
```

Sources combined in this chapter

Martin Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. (2018) and the online catalogue at refactoring.com; refactoring.guru's smell groups; Michael Feathers, *Working Effectively with Legacy Code* (2004); Joshua Kerievsky, *Refactoring to Patterns* (2004). All explanations and code are original.

Chapter 8 — Design Patterns: Introduction & Creational Patterns

What you will learn. What a design pattern is and where the idea came from, the full Gang of Four catalogue of 23 patterns, the object-oriented principles that sit underneath all of them, how to choose a pattern (and how to *not* choose one), the anti-patterns that come from overusing them, and then — in depth — the five creational patterns (Singleton, Factory Method, Abstract Factory, Builder, Prototype), the Simple Factory idiom, and the Object Pool bonus pattern.

How to read it. Part A is the map. Part B walks through each creational pattern with the same template: *what it is → the problem → how it works → Java code (with real output) → trade-offs → when not to use it → common mistakes → real-world examples → interview answers → self-check → TL;DR*. The chapter ends with one big comparison table.

Code. Everything lives in `chapters/08-patterns-creational/code/lld/ch08/<pattern>/`. Each package (a named folder of related Java classes that also acts as a namespace) has a `Main` (the class whose `main` method the program starts from). Compile and run all of them with:

```
cd chapters/08-patterns-creational/code
javac --release 21 -d /tmp/lld_08 $(find . -name '*.java')
java -cp /tmp/lld_08 lld.ch08.singleton.Main        # or .simplefactory / .factorymethod / .abstractfactory / .builder / .prototype / .objectpool
```

Part A — Design patterns: the big picture

8.1 What is a design pattern?

What it is

A **design pattern** is a named, reusable description of a good solution to a problem that keeps showing up in software design. It is not a library and not a piece of code you paste in. It is closer to a *recipe* you adapt to your kitchen.

Everyday analogy. Think about building houses. Every builder has to decide where to put the front door so people can move through the house easily. Over hundreds of houses, builders learned which placements work. If someone wrote that experience down as a guideline — "put the door here, for these reasons, and this is what you give up" — that guideline would be a pattern. A design pattern is the same thing for code: the combined experience of many developers, written down so you do not have to rediscover it.

Two words you will see everywhere:

- **Intent** — the one-sentence purpose of a pattern ("ensure a class has only one instance").
- **Participants** — the roles (classes/interfaces — an **interface** is a type that lists method signatures without bodies, which classes then implement) that take part in the pattern, such as *Creator*, *Product*, *Client*.

The problem it solves

Without patterns, every team solves the same recurring problems (how to create objects without hard-coding classes, how to add behaviour without editing old code, how to notify many listeners) from scratch, each in a slightly different, slightly buggy way. Patterns fix that in seven ways:

Benefit	What it means in practice
Solve common design problems	You start from a proven solution that has been refined over years.
Avoid reinventing the wheel	Less time designing, fewer surprises.
Readability and maintainability	Code that follows a known shape is easier for the next person to understand.
Shared vocabulary	Saying "this is a Factory" or "use an Observer" replaces a paragraph of explanation. This is the biggest win in interviews.
Best practices built in	Patterns embody good principles (loose coupling — parts know little about each other; open/closed — extend without editing), so code tends to be more testable.
Adaptability and scalability	Many patterns exist precisely so code can absorb change later.
Faster development	Tested paradigms speed up design decisions.

How it works

Every classic pattern description has four essential parts (this is how the Gang of Four book frames them):

1. **Name** — a word or two that becomes part of the team's vocabulary.
2. **Problem** — when to apply it: the situation and the forces in tension.
3. **Solution** — the participants, their responsibilities and how they collaborate. It is abstract: a *template*, not concrete code.
4. **Consequences** — the costs and benefits. A pattern without its trade-offs is only half a pattern.

You then **adapt** the template to your language and your problem. Two correct implementations of the same pattern can look quite different.

When not to use it

Patterns must be used with judgement. Not every problem needs one. Forcing a pattern where it is not needed adds classes, indirection (extra layers you must step through to reach the code that does the real work) and confusion — which is worse than having no pattern at all. We come back to this in 8.6 Anti-patterns.

TL;DR

A design pattern = **name + problem + solution template + consequences**. It is shared experience, not copy-paste code. Its biggest value is vocabulary and proven trade-offs; its biggest risk is overuse.

8.2 A short history: from buildings to the "Gang of Four"

Year	Event	Why it matters
1977	Architect **Christopher Alexander** publishes *A Pattern Language*, a catalogue of solutions for towns and buildings.	The idea of a "pattern" as a named, reusable solution comes from architecture, not software.
1987	**Kent Beck** and **Ward Cunningham** experiment with applying pattern languages to programming (user-interface design in Smalltalk, an early object-oriented language).	First bridge from architecture to software.
1994	**Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides** publish *Design Patterns: Elements of Reusable Object-Oriented Software*. The four authors are nicknamed the **Gang of Four (GoF)**.	Catalogues **23 patterns** in three categories, with C++ and Smalltalk examples. This is the book people mean when they say "GoF patterns".
2004	*Head First Design Patterns* (Freeman & Robson, with Sierra and Bates) teaches the same patterns in Java with pictures and stories (the pizza store for factories, the chocolate boiler for Singleton, the Starbuzz coffee shop for Decorator).	Made the GoF catalogue approachable, and put the **OO principles** behind patterns front and centre.

The GoF book states two principles that run through the whole catalogue:

1. **Program to an interface, not an implementation.** Variables and parameters should be typed by abstractions (interfaces or abstract classes that say *what* something does, not *how*), so callers do not care which concrete class (an ordinary, fully implemented class you can create with `new`) they get.
2. **Favour object composition over class inheritance.** Build behaviour by *holding* other objects ("has-a"), not only by extending classes ("is-a"). Composition is "black-box" reuse; inheritance exposes a parent's internals to its children.

Creational patterns are what make principle 1 practical: if code must not name concrete classes, *someone* still has to create those objects — and creational patterns decide who and how.

8.3 The three categories and all 23 GoF patterns

What it is

The GoF grouped patterns by **purpose** — what problem family they address:

Category	Question it answers	In one line
Creational	*How do objects get made?*	Control object creation so the system does not depend on concrete classes or on the messy details of construction.
Structural	*How are objects put together?*	Compose classes and objects into larger structures while keeping them flexible — simple ways to realise relationships between entities.
Behavioral	*How do objects talk and share work?*	Assign responsibilities and design communication between objects, so you think about how objects are connected instead of about raw control flow.

The GoF also used a second axis, **scope**:

- **Class patterns** use inheritance, fixed at compile time (when the compiler translates the source code, before the program runs) (Factory Method, class Adapter, Template Method, Interpreter).
- **Object patterns** use composition, changeable at run time (while the program is running) (almost everything else).

The full catalogue

#	Pattern	Category	Intent in one line	Chapter
1	**Abstract Factory**	Creational	Create *families* of related objects without naming their concrete classes.	8
2	**Builder**	Creational	Build a complex object step by step, separating construction from the final representation.	8
3	**Factory Method**	Creational	Let subclasses decide which concrete class to instantiate through an overridable creation method.	8
4	**Prototype**	Creational	Create new objects by copying a pre-configured instance.	8
5	**Singleton**	Creational	Ensure a class has exactly one instance and give a global access point to it.	8
6	**Adapter** (also called Wrapper)	Structural	Convert one interface into another that clients expect.	9
7	**Bridge**	Structural	Split an abstraction from its implementation so both can vary independently.	9
8	**Composite**	Structural	Treat individual objects and trees of objects uniformly.	9
9	**Decorator**	Structural	Attach extra responsibilities to an object dynamically by wrapping it.	9
10	**Facade**	Structural	Give a simple, unified entry point to a complex subsystem.	9
11	**Flyweight**	Structural	Share common (intrinsic) state to support huge numbers of fine-grained objects cheaply.	9
12	**Proxy**	Structural	Provide a stand-in that controls access to another object (lazy, remote, protective…).	9
13	**Chain of Responsibility**	Behavioral	Pass a request along a chain of handlers until one handles it.	10/11
14	**Command**	Behavioral	Turn a request into an object so it can be queued, logged or undone.	10/11
15	**Interpreter**	Behavioral	Represent a small language's grammar as classes and evaluate sentences in it.	10/11
16	**Iterator**	Behavioral	Traverse a collection without exposing how it is stored.	10/11
17	**Mediator**	Behavioral	Centralise complex communication between objects in one mediator object.	10/11
18	**Memento**	Behavioral	Capture and restore an object's state without breaking encapsulation (keeping internals private).	10/11
19	**Observer**	Behavioral	Notify many dependents automatically when one object changes.	10/11
20	**State**	Behavioral	Let an object change behaviour when its internal state changes, as if it changed class.	10/11
21	**Strategy**	Behavioral	Make a family of algorithms interchangeable behind one interface.	10/11
22	**Template Method**	Behavioral	Fix an algorithm's skeleton in a base class and let subclasses fill in steps.	10/11
23	**Visitor**	Behavioral	Add new operations to a set of classes without changing those classes.	10/11

Count check: 5 creational + 7 structural + 11 behavioral = **23**. (Some course lists show ten behavioral patterns and leave out Interpreter, which is rarely used in practice — it is still one of the 23.)

Not in the GoF 23 but you will meet them: *Simple Factory* (an idiom), *Object Pool* (covered at the end of this chapter), *Dependency Injection* (handing an object the things it needs from outside instead of letting it create them), *Null Object*, *Repository*, *MVC* (Model–View–Controller, which splits an app into data, display and input handling; Head First calls it a "compound pattern" built from Observer, Strategy and Composite).

Self-check

- **Q:** Which creational pattern is a *class* pattern (uses inheritance) rather than an object pattern? **A:** Factory Method — subclasses override the creation method.
- **Q:** How many behavioral GoF patterns are there? **A:** Eleven.

8.4 The OO principles behind every pattern

Patterns are not magic; they are principles applied to recurring situations. *Head First Design Patterns* collects these principles as it goes; together with SOLID (Chapter 4 — five object-oriented design rules: Single responsibility, Open–closed, Liskov substitution, Interface segregation, Dependency inversion) they explain *why* each pattern is shaped the way it is.

Principle (plain words)	What it means	Patterns that apply it (examples)
Encapsulate what varies	Find the part of the code that changes and put it behind its own abstraction, separate from what stays the same.	Factory Method (the created type varies), Strategy, State, Builder
Favour composition over inheritance	Get behaviour by holding objects, not only by subclassing.	Abstract Factory (client *holds* a factory), Strategy, Decorator
Program to interfaces, not implementations	Depend on the supertype. Callers should not care which concrete class they get.	Every creational pattern returns an interface type
Strive for loosely coupled designs between objects that interact	Objects should know as little about each other as possible.	Observer, Mediator, factories
Open–Closed Principle — open for extension, closed for modification	Add new behaviour by adding code, not by editing tested code.	Factory Method (new product = new subclass), Decorator
Dependency Inversion — depend on abstractions, not concrete classes	High-level code and low-level code both depend on interfaces.	The core of all factory patterns: the client never calls `new ConcreteThing()`
Least Knowledge ("only talk to your friends")	A method should call only its own object, its parameters, objects it creates and its fields.	Facade
Hollywood Principle ("don't call us, we'll call you")	High-level components decide when low-level ones run.	Template Method, Factory Method (the base class calls the subclass's hook), frameworks/IoC (inversion of control)
Single Responsibility — one reason to change	Each class does one job.	Builder (construction separated from the product), Iterator

Why creational patterns exist, in one sentence: Dependency Inversion says "do not depend on concrete classes", but `new PdfDocument()` *is* a dependency on a concrete class — so creational patterns move that `new` somewhere controlled (a factory, a subclass, a builder, a prototype, a container).

8.5 How to pick a pattern

The problem it solves

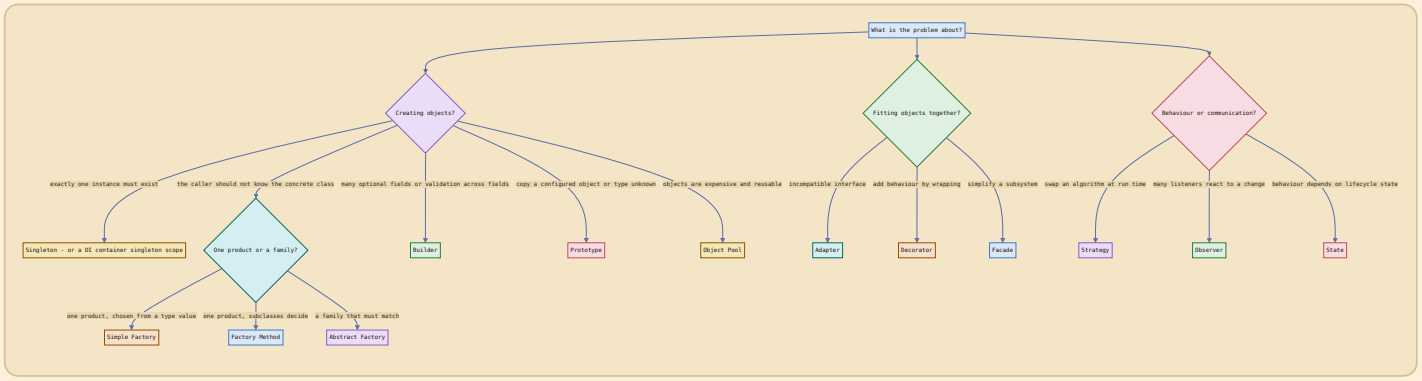
Beginners often learn 23 patterns and then look for places to use them. That is backwards. You should start from a **signal** in the requirements and only then reach for the pattern that answers it.

How it works

1. **Name the problem first**, in plain words: "the pricing rule will change", "there must be exactly one of these", "the constructor has ten arguments".
2. **Decide the category**: is it about *creating*, *structuring*, or *communicating/behaving*?

3. **Match the signal** to a pattern using the tables below.
4. **Check the cost**: how many classes does it add? Is there a simpler option (an enum — a type with a fixed list of named constants; a lambda — a short unnamed function; a constructor; a DI container — a framework that creates objects and hands them their dependencies)?
5. **Say what you rejected and why.** In an interview this is worth as much as the pattern you picked.

The flowchart below is a decision guide for the creational family plus the most common neighbours.



The six patterns that come up in almost every LLD interview

Six patterns turn up again and again in an object-oriented design round. Knowing their names pays off twice: a single word replaces a long explanation, and it signals to the interviewer that you have met this kind of problem before. The flip side matters more — dropping a pattern into a place that does not need it costs you more than never having heard of it. So learn each one together with its **signal**: the clue in the problem statement that says it fits.

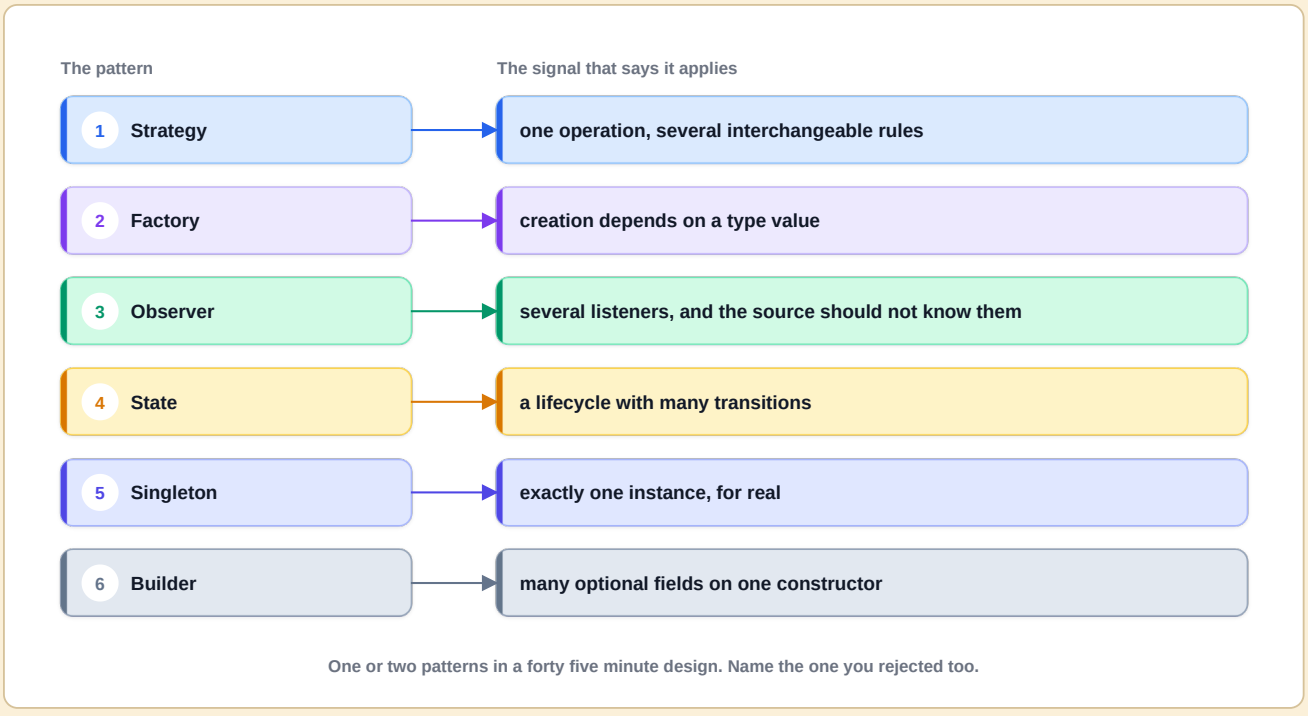


Figure — the six interview patterns and the signal that tells you each one applies (course diagram)

As the figure above shows, each pattern is paired with a trigger, not a place to force it in:

Pattern	The signal	Where it typically applies	Use it when	Do not use it when
Strategy	A single operation comes in several swappable versions, and which one runs is decided at run time	Pricing, matching, sorting, ranking, routing	There is one operation with several interchangeable rules	Only one rule exists and nothing hints at a second
Factory	Something has to be created, and the caller should not care which concrete class it gets	Turning a type value into a vehicle, spot, payment or notification	The class to create depends on a type value	Only one concrete class exists
Observer	One object changes and several others must react, without the changing object knowing who they are	Display boards, notifications, live updates, event logs	There are several listeners the source should stay ignorant of	There is a single listener the source already has a reference to
State	What an object does depends on its current status, and the allowed moves between statuses are fixed	Tickets, orders, bookings, loans — anything with a lifecycle	The lifecycle has many states and transitions	Three or four states can be handled by one guarded method
Singleton	The program genuinely needs one and only one instance	The top-level object, e.g. `ParkingLot`, `Library`	There really must be exactly one	All you want is an easy global handle
Builder	Many fields are optional and the constructor keeps getting longer	A book copy, a booking, a search query	Lots of optional fields	A handful of required fields

How each signal shows up in a parking-lot or library design:

- **Strategy.** At first the price is simply "per hour". Later the business wants a weekend tariff and a monthly pass too. With no pattern, the fee method grows a `switch` that someone has to edit whenever a tariff appears. Strategy instead defines an **interface** (a type that lists method signatures without bodies) named `ParkingRate`, whose one method turns hours into an amount; `HourlyRate`, `WeekendRate` and `MonthlyPass` implement it, and the lot holds a reference to the one currently in force. Adding a tariff means writing one class and editing nothing. Use this when the interviewer asks near the end, "what if the pricing rules change?" — the follow-up you will hear more than any other.
- **Factory.** At some point a string or enum value has to turn into a `Vehicle` object. If that conversion lives inside the entrance panel, the panel must know every vehicle type and changes whenever a type is added. Hand the job to a factory instead: it receives the type and gives back the right object, so the panel simply asks for "a vehicle". The factory becomes the single registration point for new types — the Open–Closed Principle in practice. Keep it lean: in a 45-minute round a creation method on a class you already have is normally enough; one factory class per vehicle type is more structure than the time allows.
- **Observer.** When a spot is taken, every display board on that floor must refresh. The naive version has the floor recompute its counts and push them to its single board. With Observer the spot is the *subject*, and boards, entrance panels and floor portals *subscribe* to it; on each status change the spot notifies all subscribers without knowing what they are. Adding another board needs no change to the spot. Contrast the library: when a reserved book is returned, exactly one member must be told — a direct method call is enough, because a single listener does not justify Observer.
- **State.** A parking ticket goes active → paid → closed; a book copy can be available, reserved, on loan or lost. Without the pattern every method opens with "if status is…", and the rules about which move is legal end up scattered through the class. State gives each status its own small class that knows the legal next steps, so an illegal move cannot even be written, rather than merely being absent. Be candid about the cost: with four states and three transitions, an `enum` plus one method with a guard reads more easily. Reach for State when the transition table is big, and explain why.
- **Singleton.** A car park is one physical place, so the design has one `ParkingLot`; a private constructor plus one accessor enforce that. Interviewers nearly always probe two points, so raise them yourself: fetching the instance must be correct when several threads (independent paths of execution running at the same time inside one program) ask at once (use a lock — a guard that lets only one thread at a time run a piece of code — or create it eagerly), and a singleton is global state (data that any part of the program can read and change) that makes tests harder — which is why many teams create one instance and pass it around instead. Stating that trade-off earns more credit than naming the pattern.

- **Builder.** A `BookItem` needs a barcode, price, format, purchase date, publication date and rack — an eight-argument constructor is easy to call wrongly, because two neighbouring `String` parameters can be exchanged and the compiler says nothing. A builder names every field and produces the finished object at the end. Bring it up once a constructor in your design passes about six arguments, then move on: it improves readability, it does not change the structure.

Rules of thumb for the interview:

- **One or two patterns** in a 45-minute design is the right number, placed exactly where their signal appears. If you count six, the design has probably drifted away from the requirements.
- **Compare out loud.** For example: "Strategy for the rates, because a second tariff is clearly coming; no State for the ticket, because a single guard handles three statuses." Explaining the pattern you *rejected* shows as much judgement as the one you chose — and that comparison is the real reason to learn the names.

8.6 Anti-patterns: when patterns go wrong

What it is

An **anti-pattern** is a common "solution" that looks reasonable but reliably causes more problems than it solves. Knowing them is how you avoid misusing real patterns.

Anti-pattern	What it looks like	Why it hurts	The fix
God Object (God class, the Blob)	One class knows and does almost everything: `SystemManager` with 80 methods and 40 fields.	Violates Single Responsibility. Every change touches it, merge conflicts (clashing edits by different people to the same lines) everywhere, impossible to test in isolation.	Split by responsibility (Extract Class — Chapter 7), push behaviour to the objects that own the data.
Golden Hammer	"When all you have is a hammer, everything looks like a nail." The team's favourite tool or pattern is applied to every problem.	The solution is forced onto problems it does not fit, e.g. every object is a Singleton, every `if` becomes a Strategy.	Start from the problem's signal (8.5), not from the tool.
Pattern overuse ("patternitis")	Factories that create one class, interfaces with one implementation "just in case", five layers of indirection for a CRUD (create, read, update, delete) screen.	More classes, more hops to read, no benefit. Fowler calls the underlying smell **Speculative Generality** (building for imagined future needs).	Apply YAGNI ("You Aren't Gonna Need It", Chapter 5): add the pattern when the *second* variant actually arrives. Refactor towards a pattern, do not start from one.
Singletonitis	Singletons used as convenient global variables.	Hidden dependencies, order-dependent tests, hard to replace.	Dependency injection; pass the object in (8.8).
Cargo-cult patterns	Copying a pattern's structure without understanding its intent (e.g. a "Builder" that only copies fields with no validation, a "Factory Method" that is really a switch).	Ceremony with none of the guarantees.	Know the *intent*: what property does the pattern give you? If you cannot name it, drop it.

TL;DR

Patterns are a vocabulary, not a checklist. The God Object comes from ignoring design; the Golden Hammer and pattern overuse come from over-applying it.

Part B — Creational patterns

8.7 Creational patterns at a glance

What it is

Creational patterns control *how objects are created*, so that the rest of the system is not tied to the concrete classes or the construction details. They hide *which* class is instantiated, *how* it is assembled, and sometimes *whether* a new object is made at all.

Using them lets a system:

- be **independent** of how its objects are created, composed and represented;
- **hide the complexity** of creation (long setup, validation, choosing a subtype);
- keep **flexibility** about *what* gets created, *who* creates it, *how*, and *when*.

Pattern	Intent in one line
Singleton	Exactly one instance of a class, with one global way to reach it.
Factory Method	A base class defines a creation method; subclasses decide which concrete class it returns.
Abstract Factory	One factory object creates a whole *family* of related objects that must match.
Builder	Assemble one complex object step by step; the same construction process can produce different representations.
Prototype	Create new objects by cloning a prototypical instance.
Simple Factory (idiom)	One method with a switch/registry decides which class to create.
Object Pool (bonus)	Reuse a bounded set of expensive objects instead of creating and destroying them.

8.8 Singleton

What it is

The **Singleton** pattern restricts a class to **exactly one instance** (per process — one running program — or, strictly, per class loader — the part of the JVM, the Java Virtual Machine that runs compiled Java code, which loads classes into memory) and gives every caller **one well-known way to reach it**.

Everyday analogy. A country has one official government. Whoever needs "the government" — a citizen, a company, a foreign embassy — reaches the same one through a known channel. Nobody is allowed to set up a second one.

Terms used below:

- **Instance** — one object created from a class.
- **Eager initialization** — create the object as soon as the class is loaded.
- **Lazy initialization** — create it only the first time someone asks for it.
- **Thread-safe** — gives correct results even when many threads call it at the same moment.

GoF intent and participants. Intent: guarantee that a class never has more than one instance, and give every client a well-known way to reach it. There is a single participant, **Singleton**, which (a) defines a class-level operation (`getInstance()`) that lets clients reach the unique instance and (b) is usually responsible for creating that instance itself. Clients simply call that operation.

The problem it solves

A few things in a program really are "one per process": the settings loaded from a file when the app starts, the pool of database connections (a set of open connections kept ready for reuse), the registry that collects metrics (counters and timings the app records about itself). Let every module create its own and you end up with many copies of the settings, the same file parsed many times, and metrics split across registries whose totals no one can combine.

Picture a `ConfigStore` filled from a settings file: the cloud region to talk to, the retry count, the timeout (how long to wait before giving up). The billing service reads it, so does the nightly reporting job, so does

the mailer. If each of them builds a private copy, the file is parsed three times, and a value one service changes never reaches the other two.

A plain global variable looks like the easy fix, but it has two problems. Tests all share it, so a test's result can depend on which test happened to run before it. And *some* code has to assign it before anyone reads it — get the start-up order wrong and a caller finds `null` (Java's "no object here" value). Singleton moves both jobs into the class: **the class decides how many instances may exist and creates the one it allows**, on demand. Callers never assign anything, so there is no ordering mistake to make.

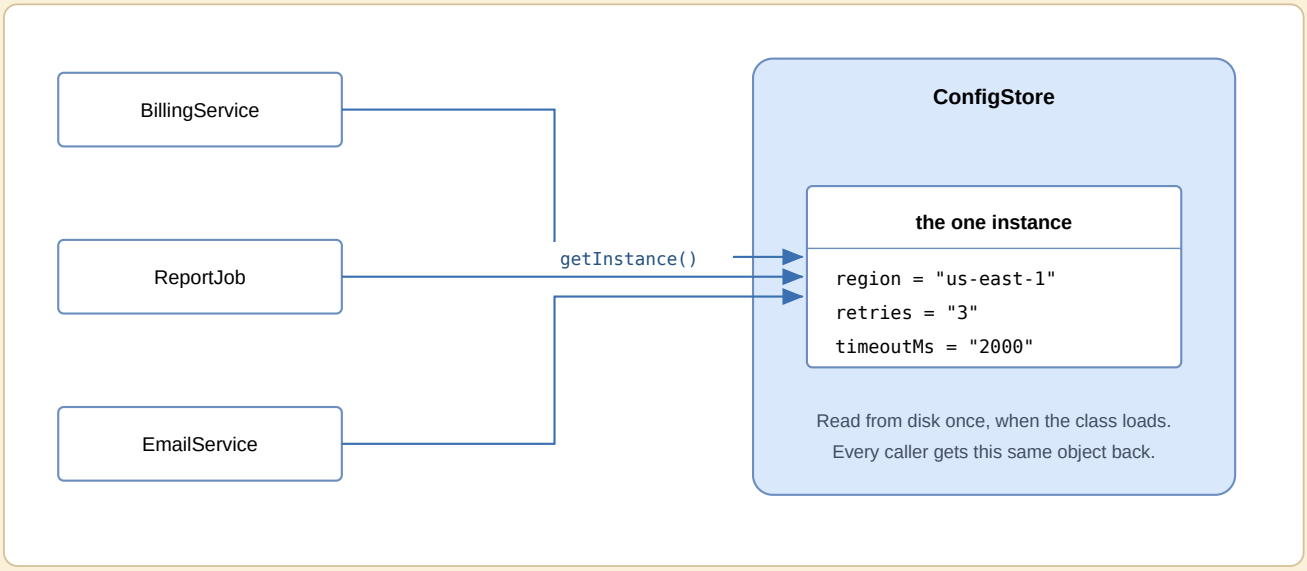


Figure — BillingService, ReportJob and EmailService all call getInstance() and receive the same ConfigStore, which was read from disk once (course diagram)

In the figure, none of the three services constructs a `ConfigStore`; each one asks the class for it and gets back the very same object, whose settings were loaded a single time.

How it works

1. Make the **constructor** `private`, so code outside the class cannot write `new ConfigStore()`. That access modifier (the `public` / `protected` / `private` keyword that says who may use a member) is the whole enforcement mechanism; make the constructor public and any caller can create a second one.
2. Keep the single instance in a `private static` **field** — static because it is owned by the class itself, not by whichever object asks for it.
3. Expose a `public static getInstance()` method. It has to be static: a caller has no instance yet (getting one is the whole point), so it can only call something on the class.
4. Inside `getInstance()`, return the stored instance, creating it first if needed (and doing that creation safely when threads are involved).
5. Add ordinary instance methods (`get`, `set`, …) for the actual work.

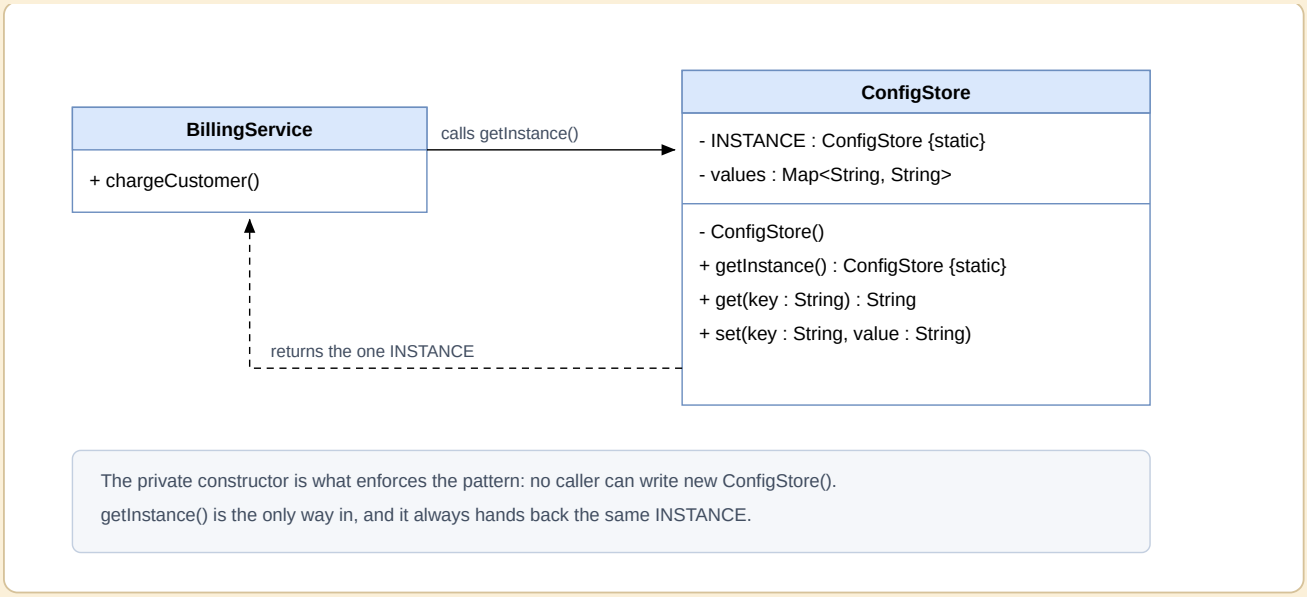
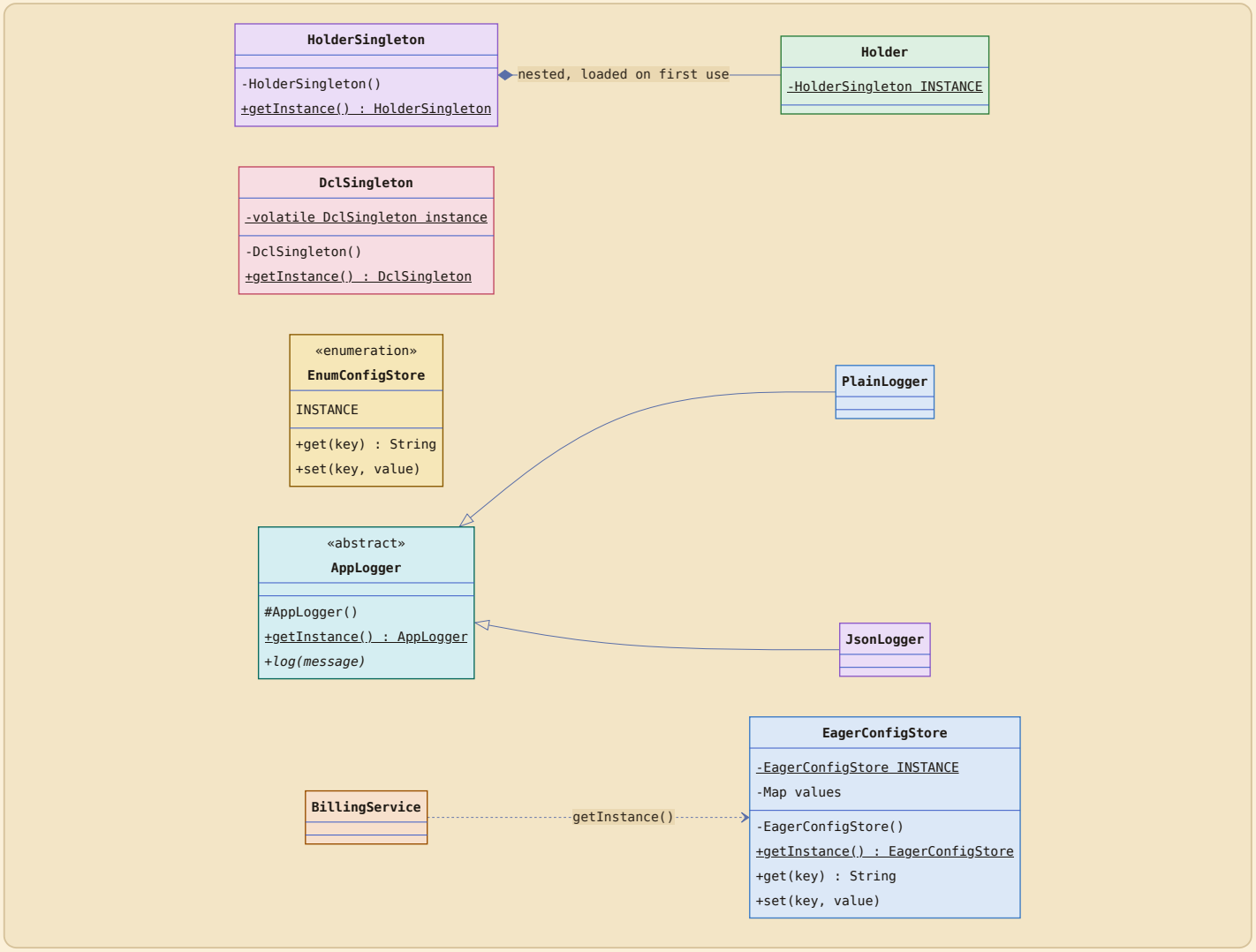


Figure — ConfigStore with a private static INSTANCE, a private constructor and a static getInstance(); BillingService only calls getInstance() (course diagram)

Our Java design (all the variants in the code package):



The six ways to write a Singleton in Java

Two terms in the table: **serialization** turns an object into bytes so it can be saved or sent (and **deserialization** rebuilds an object from those bytes); **reflection** is Java's ability to inspect and call a class's members — even private ones — at run time. "Contention" means many threads competing for the same lock.

#	Variant	Lazy?	Thread-safe?	Cost on each call	Survives serialization / reflection?	Verdict
1	**Eager** `static final` field	No (built when the class is initialized)	Yes — the JVM initializes a class exactly once	None	No / No (needs defences)	Default choice when construction is cheap
2	**Naive lazy** `if (instance == null)`	Yes	**No**	None	No / No	Never in multi-threaded code
3	`synchronized` **getInstance()**	Yes	Yes	A lock on *every* call	No / No	Correct but slow under contention
4	**Double-checked locking** + `volatile`	Yes	Yes (Java 5+)	One volatile read	No / No	When the instance depends on run-time input
5	**Bill Pugh / holder idiom**	Yes	Yes — class-loading guarantee	None	No / No	Best lazy option in plain Java
6	`enum` **with one constant**	No (created when the enum class initializes)	Yes	None	**Yes / Yes** (JVM guarantees)	Shortest and most robust; recommended by *Effective Java* (Joshua Bloch's best-practice book)

Let us walk through each one.

1 — Eager. The `static final` field is assigned while the class is being *initialized*. In Java that happens on the class's first active use — typically the first `getInstance()` call, but also the first access to any other static method or non-constant static field of the class — not at JVM start-up. The Java Language Specification (§12.4.2) guarantees that initialization runs exactly once, under an internal lock, even if many threads trigger it at the same moment, and that every thread afterwards sees the fully built object — so you get thread safety without writing any. The only cost: the object is built as soon as the class is touched for *any* reason, even if that code path never needs the instance. If the constructor opens a file or a socket (an endpoint of a network connection), that may matter.

2 — Naive lazy (broken). "Create on first use" is written as *check, then act*:

```
// Do NOT use this in multi-threaded code.
public static ConfigStore getInstance() {
    if (instance == null) {            // two threads can both see null here...
        instance = new ConfigStore();  // ...and both construct
    }
    return instance;
}
```

Nothing stops thread A and thread B from both reading `null` before either has stored its object. Each then constructs its own store, and whatever one caller writes into its copy never shows up in the other's. Run it in a single-threaded test and it behaves perfectly, which is exactly why the bug slips through review. Our demo starts eight threads at the same instant: the naive version built **more than one** object (eight in the run shown below).

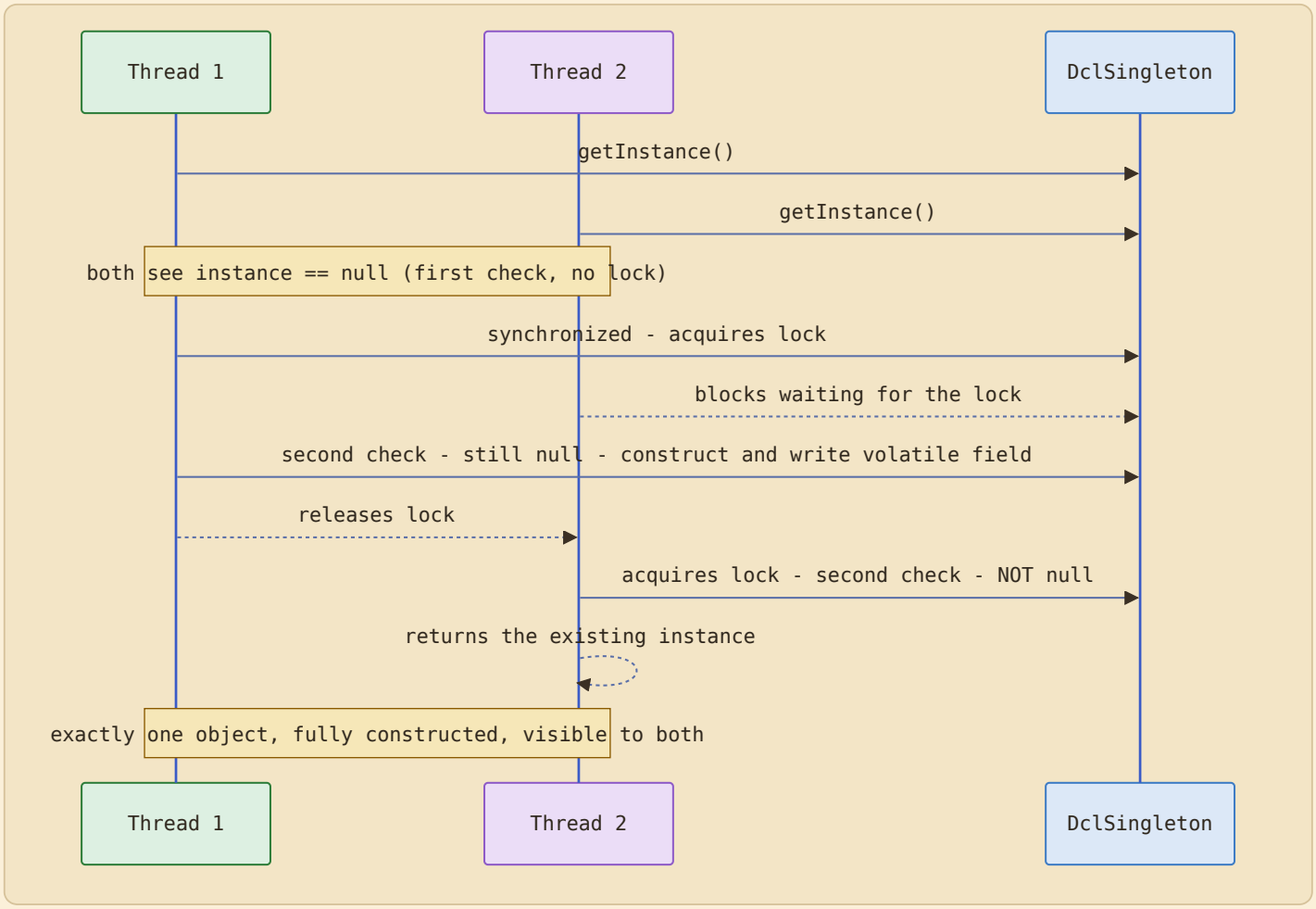
3 — `synchronized` **method.** Putting `synchronized` on `getInstance()` fixes the race (a **race condition**: a bug whose outcome depends on the unlucky timing of threads) — only one thread can be inside at a time. But the lock is taken on *every* call, forever, even though it is only needed for the first one.

4 — Double-checked locking (DCL). Check *without* the lock first (fast path), and only if the instance is missing, take the lock and **check again** (another thread may have created it while you waited):

- The second check is what prevents two constructions.
- The field **must be** `volatile` (a Java keyword that makes each write to the field immediately visible to every thread and forbids reordering around it). Creating an object is really three steps — allocate memory, run the constructor, store the reference. Without `volatile`, the Java Memory Model (the rules for when one thread's writes become visible to other threads) allows another thread to see the stored reference *before* the

constructor's writes are visible, i.e. a half-built object. A `volatile` write/read creates a **happens-before** relationship: everything the constructor did is visible to any thread that reads the reference. (Before Java 5's memory model, DCL was broken even with `volatile`.)

- Copying the field into a local variable (`local`) means the common path reads the volatile field only once — a small optimization shown in *Effective Java*.



5 — Bill Pugh / initialization-on-demand holder. Put the instance in a private *nested* class (a class declared inside another class). The JVM does not initialize `Holder` until `getInstance()` first touches `Holder.INSTANCE`, so creation is lazy; and class initialization is guaranteed to run once and be safely published (made visible to other threads only once fully built), so it is thread-safe with **no locks and no `volatile`**. Named after Bill Pugh, who led the work on the Java memory model. This is the usual choice for a lazy singleton.

6 — Enum. `enum EnumConfigStore { INSTANCE; ... }`. The JVM guarantees each enum constant exists once; `Constructor.newInstance` refuses to create enum objects; and enum serialization is special-cased to return the existing constant. It is the shortest correct version. Limits: an enum cannot extend another class (it can implement interfaces), and it is created when the enum class initializes rather than truly on demand.

Which one to pick. Default to **eager**: fewest lines and no race is possible; its only drawback matters when construction is costly. Switch to the **holder** idiom once you have *measured* a start-up cost worth avoiding. Choose **double-checked locking** when building the instance needs a value that only exists at run time (a holder's static initializer cannot take a parameter) — and always declare the field `volatile`. Choose **enum** when you also want serialization and reflection safety without writing any defensive code.

Breaking a Singleton — and the fixes

A `private` constructor is a *language* rule, not a JVM guarantee. There are three classic ways around it:

Attack	How it creates a second instance	Fix
Reflection	`getDeclaredConstructor().setAccessible(true).newInstance()` calls the private constructor.	Throw from the constructor if the instance already exists (reliable for eager/holder, racy for lazy). Or use an **enum** — the JVM refuses.
Serialization	`ObjectInputStream.readObject()` builds a new object without calling your constructor.	Add `private Object readResolve() { return INSTANCE; }` — the deserialized copy is replaced by the real one. Mark instance fields `transient` (skipped by serialization) if they should not travel. Or use an **enum**.
Cloning	If the class (or a parent) implements `Cloneable` and exposes `clone()`, `clone()` copies the object.	Override `clone()` to throw `CloneNotSupportedException` (or return `INSTANCE`). If no ancestor is `Cloneable`, `Object.clone()` is `protected` and this attack does not apply.
Multiple class loaders	Each class loader loads its own copy of the class, so each has its own "single" instance (common in app servers and plugin systems).	Load the class from a shared parent loader, or let a container own the instance.
Multiple processes / servers	A singleton is one per JVM. On four servers you have four.	Use an external coordinator (database, Redis — an in-memory data store, ZooKeeper — a coordination service) for anything that must be unique system-wide, e.g. ID generation or locks.

Our demo attacks an undefended singleton and a hardened one, then the enum (see output).

Subclassing a Singleton

The GoF book notes a singleton *can* be subclassed if its constructor is `protected` rather than `private`, and `getInstance()` decides **once, at run time**, which subclass becomes "the" instance. `AppLogger` below picks `PlainLogger` or `JsonLogger` from a system property (a key=value setting passed to the JVM at launch, e.g. `-Dlog.format=json`). With a `private` constructor (the usual case) the class cannot be subclassed at all — so the flexibility is only there if you design for it.

The dependency-injection alternative

In modern code the best singleton is often **not a Singleton class at all**. Write a normal class with a public constructor, create **one** instance at start-up (the *composition root*), and **pass it in** through constructors. "One instance" becomes a wiring decision, not a property baked into the class.

- Dependencies become visible in constructors (no hidden `getInstance()` calls).
- Tests pass a fake (a simple stand-in object written for tests) (`new BillingService(testConfig)`) — no reset hooks needed.
- Spring, Guice, Micronaut, Dagger, NestJS and friends (dependency-injection frameworks) do exactly this: in Spring every bean (an object that Spring creates and manages) is **singleton-scoped by default**, meaning *one shared instance per bean definition per container* (`ApplicationContext`) — not one per JVM, and without any private constructor. Two contexts in one JVM hold two instances, and nothing stops other code calling `new` on the class. Spring's opposite, `prototype` **scope**, creates a fresh instance on every lookup.

Java implementation

```java
// File: code/lld/ch08/singleton/EagerConfigStore.java
package lld.ch08.singleton;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/** 1. Eager: the instance is created when the class is initialised. Thread-safe for free. */
public final class EagerConfigStore {

    private static final EagerConfigStore INSTANCE = new EagerConfigStore();

    private final Map<String, String> values = new ConcurrentHashMap<>();

    private EagerConfigStore() {                        // private: nobody else can call new
        System.out.println("  [EagerConfigStore] reading settings from disk (runs once)");
        values.put("region", "us-east-1");
        values.put("retries", "3");
    }

    public static EagerConfigStore getInstance() { return INSTANCE; }

    public String get(String key)             { return values.get(key); }
    public void set(String key, String value) { values.put(key, value); }
}
```

```java
// File: code/lld/ch08/singleton/LazyVariants.java
package lld.ch08.singleton;

import java.util.concurrent.atomic.AtomicInteger;

/** 2. Naive lazy: NOT thread-safe. Two threads can both see null and both construct. */
final class NaiveLazySingleton {
    static final AtomicInteger CONSTRUCTED = new AtomicInteger();
    private static NaiveLazySingleton instance;

    private NaiveLazySingleton() {
        CONSTRUCTED.incrementAndGet();
        try { Thread.sleep(20); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }

    static NaiveLazySingleton getInstance() {
        if (instance == null) {                 // check ...
            instance = new NaiveLazySingleton(); // ... then act: a race window
        }
        return instance;
    }
}

/** 3. Synchronized lazy: correct, but every call pays for the lock. */
final class SynchronizedLazySingleton {
    static final AtomicInteger CONSTRUCTED = new AtomicInteger();
    private static SynchronizedLazySingleton instance;

    private SynchronizedLazySingleton() {
        CONSTRUCTED.incrementAndGet();
        try { Thread.sleep(20); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }

    static synchronized SynchronizedLazySingleton getInstance() {
        if (instance == null) instance = new SynchronizedLazySingleton();
        return instance;
    }
}

/** 4. Double-checked locking: lock only on the first calls. volatile is mandatory. */
final class DclSingleton {
    static final AtomicInteger CONSTRUCTED = new AtomicInteger();
    private static volatile DclSingleton instance;

    private DclSingleton() {
        CONSTRUCTED.incrementAndGet();
        try { Thread.sleep(20); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }

    static DclSingleton getInstance() {
        DclSingleton local = instance;          // one volatile read on the fast path
        if (local == null) {                    // 1st check, no lock
            synchronized (DclSingleton.class) {
                local = instance;
                if (local == null) {            // 2nd check, under the lock
                    local = new DclSingleton();
                    instance = local;           // volatile write publishes a fully built object
                }
            }
        }
        return local;
    }
}

/** 5. Bill Pugh / initialization-on-demand holder: lazy, lock-free, thread-safe via class loading. */
final class HolderSingleton {
    static final AtomicInteger CONSTRUCTED = new AtomicInteger();
```

```
    private HolderSingleton() {
        CONSTRUCTED.incrementAndGet();
        System.out.println("  [HolderSingleton] constructed on first getInstance()");
    }

    private static final class Holder {          // not initialised until first touched
        private static final HolderSingleton INSTANCE = new HolderSingleton();
    }

    static HolderSingleton getInstance() { return Holder.INSTANCE; }
}
```

```
// File: code/lld/ch08/singleton/EnumConfigStore.java
package lld.ch08.singleton;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/** 6. Enum singleton: shortest form, safe against reflection and serialization by the JVM itself. */
public enum EnumConfigStore {
    INSTANCE;

    private final Map<String, String> values = new ConcurrentHashMap<>(Map.of("region", "us-east-1"));

    public String get(String key)             { return values.get(key); }
    public void set(String key, String value) { values.put(key, value); }
}
```

```
// File: code/lld/ch08/singleton/BreakableSingleton.java
package lld.ch08.singleton;

import java.io.Serializable;

/** A parent that (unwisely) exposes a public clone(). Children inherit it. */
class CloneableBase implements Cloneable {
    @Override
    public Object clone() throws CloneNotSupportedException { return super.clone(); }
}

/** A classic eager singleton with NO defences. Reflection, serialization and cloning all break it. */
public final class BreakableSingleton extends CloneableBase implements Serializable {
    private static final long serialVersionUID = 1L;
    public static final BreakableSingleton INSTANCE = new BreakableSingleton();

    private BreakableSingleton() { }
}
```

```java
// File: code/lld/ch08/singleton/HardenedSingleton.java
package lld.ch08.singleton;

import java.io.Serial;
import java.io.Serializable;

/** The same singleton with the three classic defences. */
public final class HardenedSingleton extends CloneableBase implements Serializable {
    @Serial private static final long serialVersionUID = 1L;
    public static final HardenedSingleton INSTANCE = new HardenedSingleton();

    private HardenedSingleton() {
        // Defence 1 - reflection: refuse a second construction.
        // (Works reliably with eager init, because INSTANCE is set during class init.)
        if (INSTANCE != null) {
            throw new IllegalStateException("Singleton already exists - use INSTANCE");
        }
    }

    // Defence 2 - serialization: replace whatever was deserialized with the real instance.
    @Serial
    private Object readResolve() { return INSTANCE; }

    // Defence 3 - cloning: override the inherited public clone() and refuse.
    @Override
    public Object clone() throws CloneNotSupportedException {
        throw new CloneNotSupportedException("Singleton cannot be cloned");
    }
}
```

```java
// File: code/lld/ch08/singleton/AppLogger.java
package lld.ch08.singleton;

/**
 * GoF note: a Singleton may be subclassed if the constructor is protected.
 * getInstance() decides ONCE, at run time, which subclass becomes "the" instance.
 */
public abstract class AppLogger {

    protected AppLogger() { }                           // protected, not private

    private static final class Holder {
        private static final AppLogger INSTANCE =
                "json".equals(System.getProperty("log.format")) ? new JsonLogger() : new PlainLogger();
    }

    public static AppLogger getInstance() { return Holder.INSTANCE; }

    public abstract void log(String message);
}

final class PlainLogger extends AppLogger {
    @Override public void log(String message) { System.out.println("  LOG " + message); }
}

final class JsonLogger extends AppLogger {
    @Override public void log(String message) { System.out.println("  {\"log\":\"" + message + "\"}"); }
}
```

```java
// File: code/lld/ch08/singleton/DiAlternative.java
package lld.ch08.singleton;

import java.util.Map;

/** The dependency-injection alternative: "one instance" is decided by the wiring code, not the class. */
interface Config {
    String get(String key);
}

record MapConfig(Map<String, String> values) implements Config {
    public String get(String key) { return values.get(key); }
}

final class BillingService {
    private final Config config;                        // dependency is visible in the constructor
    BillingService(Config config) { this.config = config; }
    String charge(int cents) { return "charged " + cents + " cents in " + config.get("region"); }
}

final class ReportJob {
    private final Config config;
    ReportJob(Config config) { this.config = config; }
    String run() { return "report for " + config.get("region"); }
}
```

```java
// File: code/lld/ch08/singleton/Main.java
package lld.ch08.singleton;

import java.io.*;
import java.lang.reflect.Constructor;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.Supplier;

public class Main {

    public static void main(String[] args) throws Exception {
        System.out.println("== 1. Eager singleton: every caller shares one store ==");
        EagerConfigStore billing = EagerConfigStore.getInstance();
        EagerConfigStore reports = EagerConfigStore.getInstance();
        System.out.println("  billing reads region = " + billing.get("region"));
        reports.set("region", "eu-west-1");
        System.out.println("  after reports.set, billing reads = " + billing.get("region"));
        System.out.println("  same object? " + (billing == reports));

        System.out.println("\n== 2-4. Lazy variants hit by 8 threads at the same instant ==");
        race("naive lazy (unsafe)", NaiveLazySingleton::getInstance, NaiveLazySingleton.CONSTRUCTED::get);
        race("synchronized lazy", SynchronizedLazySingleton::getInstance,
SynchronizedLazySingleton.CONSTRUCTED::get);
        race("double-checked locking", DclSingleton::getInstance, DclSingleton.CONSTRUCTED::get);

        System.out.println("\n== 5. Bill Pugh holder: nothing built until first use ==");
        System.out.println("  constructed so far: " + HolderSingleton.CONSTRUCTED.get());
        HolderSingleton h1 = HolderSingleton.getInstance();
        HolderSingleton h2 = HolderSingleton.getInstance();
        System.out.println("  constructed now: " + HolderSingleton.CONSTRUCTED.get() + ", same? " + (h1 == h2));

        System.out.println("\n== 6. Enum singleton ==");
        EnumConfigStore.INSTANCE.set("retries", "5");
        System.out.println("  retries = " + EnumConfigStore.INSTANCE.get("retries"));

        System.out.println("\n== 7. Breaking a naive singleton ==");
        attack("BreakableSingleton", BreakableSingleton.INSTANCE, BreakableSingleton.class);
        System.out.println("\n== 8. The hardened version ==");
        attack("HardenedSingleton", HardenedSingleton.INSTANCE, HardenedSingleton.class);
        System.out.println("\n== 9. Enum resists reflection and serialization ==");
        attackEnum();

        System.out.println("\n== 10. Subclassable singleton (protected constructor) ==");
        AppLogger.getInstance().log("chosen subclass = " + AppLogger.getInstance().getClass().getSimpleName());

        System.out.println("\n== 11. DI alternative: one instance created by the composition root ==");
        Config shared = new MapConfig(Map.of("region", "ap-south-1"));
        BillingService billingService = new BillingService(shared);
        ReportJob reportJob = new ReportJob(shared);
        System.out.println("  " + billingService.charge(499));
        System.out.println("  " + reportJob.run());
        Config testConfig = new MapConfig(Map.of("region", "test-region"));
        System.out.println("  in a test: " + new BillingService(testConfig).charge(1));
    }

    /** Starts 8 threads that all call getInstance() at the same moment. */
    private static void race(String label, Supplier<Object> getInstance, Supplier<Integer> constructed)
            throws Exception {
        int threads = 8;
        CountDownLatch start = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(threads)) {
            List<Future<Object>> results = new ArrayList<>();
            for (int i = 0; i < threads; i++) {
                results.add(pool.submit(() -> { start.await(); return getInstance.get(); }));
            }
            start.countDown();
            Set<Object> distinct = Collections.newSetFromMap(new IdentityHashMap<>());
```

```
            for (Future<Object> f : results) distinct.add(f.get());
            System.out.printf("  %-24s constructor ran %d time(s), callers saw %d distinct object(s)%n",
                    label, constructed.get(), distinct.size());
        }
    }

    private static void attack(String name, Object instance, Class<?> type) {
        // Reflection
        try {
            Constructor<?> c = type.getDeclaredConstructor();
            c.setAccessible(true);
            Object other = c.newInstance();
            System.out.println("  reflection    -> new object, same? " + (other == instance));
        } catch (ReflectiveOperationException | RuntimeException e) {
            Throwable cause = e.getCause() != null ? e.getCause() : e;
            System.out.println("  reflection    -> blocked: " + cause.getMessage());
        }
        // Serialization
        try {
            Object copy = roundTrip(instance);
            System.out.println("  serialization -> same? " + (copy == instance));
        } catch (Exception e) {
            System.out.println("  serialization -> failed: " + e);
        }
        // Cloning
        try {
            Object clone = ((CloneableBase) instance).clone();
            System.out.println("  clone         -> same? " + (clone == instance));
        } catch (CloneNotSupportedException e) {
            System.out.println("  clone         -> blocked: " + e.getMessage());
        }
    }

    private static void attackEnum() throws Exception {
        try {
            Constructor<?> c = EnumConfigStore.class.getDeclaredConstructors()[0];
            c.setAccessible(true);
            c.newInstance("SECOND", 1);
        } catch (IllegalArgumentException e) {
            System.out.println("  reflection    -> blocked: " + e.getMessage());
        }
        System.out.println("  serialization -> same? " + (roundTrip(EnumConfigStore.INSTANCE) ==
EnumConfigStore.INSTANCE));
    }

    private static Object roundTrip(Object o) throws IOException, ClassNotFoundException {
        ByteArrayOutputStream bytes = new ByteArrayOutputStream();
        try (ObjectOutputStream out = new ObjectOutputStream(bytes)) { out.writeObject(o); }
        try (ObjectInputStream in = new ObjectInputStream(new ByteArrayInputStream(bytes.toByteArray()))) {
            return in.readObject();
        }
    }
}
```

Explaining the parts:

- `EagerConfigStore` — the classic shape: private constructor, `static final INSTANCE`, `getInstance()`. Section 1 of `Main` shows why anyone wants this: `reports` updates the region and `billing` immediately reads the new value, even though neither holds a reference to the other — they share the one store.
- `LazyVariants.java` — each constructor sleeps 20 ms to widen the race window, and counts how many times it ran. The `race(...)` helper releases eight threads at once with a `CountDownLatch`.
- `HolderSingleton` — prints "constructed so far: 0" before first use, proving laziness.
- `BreakableSingleton` vs `HardenedSingleton` — identical except for the three defences (constructor guard, `readResolve`, throwing `clone`).
- `AppLogger` — a subclassable singleton using a holder; `DiAlternative.java` — the DI version with no singleton code at all.

Output:

```
== 1. Eager singleton: every caller shares one store ==
  [EagerConfigStore] reading settings from disk (runs once)
  billing reads region = us-east-1
  after reports.set, billing reads = eu-west-1
  same object? true

== 2-4. Lazy variants hit by 8 threads at the same instant ==
  naive lazy (unsafe)      constructor ran 8 time(s), callers saw 8 distinct object(s)
  synchronized lazy        constructor ran 1 time(s), callers saw 1 distinct object(s)
  double-checked locking   constructor ran 1 time(s), callers saw 1 distinct object(s)

== 5. Bill Pugh holder: nothing built until first use ==
  constructed so far: 0
  [HolderSingleton] constructed on first getInstance()
  constructed now: 1, same? true

== 6. Enum singleton ==
  retries = 5

== 7. Breaking a naive singleton ==
  reflection    -> new object, same? false
  serialization -> same? false
  clone         -> same? false

== 8. The hardened version ==
  reflection    -> blocked: Singleton already exists - use INSTANCE
  serialization -> same? true
  clone         -> blocked: Singleton cannot be cloned

== 9. Enum resists reflection and serialization ==
  reflection    -> blocked: Cannot reflectively create enum objects
  serialization -> same? true

== 10. Subclassable singleton (protected constructor) ==
  LOG chosen subclass = PlainLogger

== 11. DI alternative: one instance created by the composition root ==
  charged 499 cents in ap-south-1
  report for ap-south-1
  in a test: charged 1 cents in test-region
```

(The naive-lazy line is a race, so its count varies from run to run — anything above 1 is the bug.)

Trade-offs

Pros	Cons
Controlled access — the class guarantees a single instance and there is exactly one documented door to it.	**Global state** — in practice it tends to become shared mutable (changeable) state that anything can change, with all the classic problems that brings.
You choose when it is built — eagerly (simplest) or lazily on first request (cheaper start-up).	**Awkward to test** — replacing it with a fake needs a reset/injection hook, and that hook then ships in production code (the code running the live system real users depend on) where someone may misuse it.
Right fit for a shared resource — a pool, a cache or a device handle, where a second copy would be a bug.	**Invisible dependencies** — a class that calls `getInstance()` internally does not show that dependency in its constructor, so readers and tests cannot see it.
Setup cost paid once — the expensive initialization runs once per process instead of once per caller.	**Uniqueness ends at the process** (strictly, the class loader) — it cannot coordinate anything across machines, which surprises people who try.
Subclassing is possible if designed in — with a `protected` constructor, `getInstance()` can pick a subclass at run time (see `AppLogger`). With a `private` constructor, as in most examples, it cannot.	**Hard to undo** — by the time you want an ordinary class back, `getInstance()` calls are scattered everywhere. It also mixes two jobs (SRP — Single Responsibility Principle): managing its own lifecycle *and* doing its real work.

Often confused with:

Looks like	The actual difference
Static utility class (`Math`)	A class with only static members is not an object: it cannot implement an interface or be handed to a method as an argument. A Singleton *is* an object, so it can hide behind an interface and a test can substitute another implementation. That difference is the main argument for a Singleton over a static class.
Prototype	The intents point in opposite directions: Singleton caps the count at one, Prototype exists to churn out many copies.
DI container singleton scope	Achieves the same "one shared instance" with a better mechanism: the container manages the lifecycle, classes still declare what they need in their constructors, and tests swap in replacements freely.
Multiton / registry (`Logger.getLogger(name)`)	One instance *per key*, not one per class.

In other languages: in Python, a module's top-level code runs only once, so an object created at module level is already shared by every importer (the hand-written `__new__` version has the same check-then-act race as Java's naive lazy form and needs a `threading.Lock` with a double check). In JavaScript/TypeScript, exporting an object that is already constructed gives the same effect; because a single event loop (the one loop that runs JavaScript tasks one after another) never interleaves two synchronous blocks of code, no locking is needed — but each Web Worker (a background thread in the browser) has its own separate memory, so each gets its own copy. In Go, a package-level variable initialized through `sync.Once`. In C++, a function-local `static` (the "Meyers Singleton") is created on first call and the language standard guarantees that happens once even under concurrent calls. In Java, the single-element `enum` is the shortest thread-safe form and, unlike the field-based versions, survives serialization without extra code.

When not to use it

No creational pattern is misapplied more often than this one.

- **The real motive is convenience.** Wanting to reach an object from anywhere is not a design requirement — a constructor or method parameter delivers it just as well, and this motive explains most of the singletons that should never have existed.
- **It holds mutable state that tests read or write.** All tests then share that state, and you get the maddening case of a test that passes in isolation but fails when the whole suite (all the tests together) runs.
- **A framework already manages lifecycles.** In Spring, Guice, NestJS and similar containers, declare the class with singleton scope and write no singleton code at all.

- **More than one process is involved.** Each JVM has its own instance, so four servers means four "singletons". Relying on one for a distributed lock (a lock shared by several machines) or for unique IDs is a **correctness bug**, not a matter of style.

Common mistakes

- Using the **naive lazy** version in a web server or any multi-threaded code.
- Writing double-checked locking **without** `volatile`.
- Forgetting that `implements Serializable` silently breaks the guarantee unless you add `readResolve()`.
- Putting **mutable state** in a singleton without thread-safe structures (our `ConfigStore` uses `ConcurrentHashMap` because many threads may `set` concurrently).
- Adding a `public static void reset()` "for tests" — it will be called in production one day.
- Assuming "one instance" means one in the whole system (it is one per JVM / class loader).

Real-world examples

- **JDK** (Java Development Kit — Java's compiler, tools and standard library)**:** `Runtime.getRuntime()` is the textbook case: `Runtime` has a private constructor and a single static instance for the whole JVM. `java.awt.Desktop.getDesktop()` also hands back one shared instance. `Collections.emptyList()` returns one shared immutable (unchangeable after creation) constant — singleton-*like* caching of an immutable value rather than the full pattern. `Logger.getLogger(name)` (java.util.logging) returns one shared logger per name — a registry (multiton), one instance per key.
- **Not singletons, despite the name `getInstance()`:** `Calendar.getInstance()`, `NumberFormat.getInstance()` and `Cipher.getInstance(...)` return a **new object on every call** — they are static factory methods (see 8.9). A method called `getInstance` proves nothing; what matters is whether only one instance can ever exist.
- **Spring:** beans are singleton-scoped by default — one per bean definition per `ApplicationContext`, managed by the container rather than by a private constructor. This is the DI alternative in production; it is "one per container", not "one per JVM".
- **Everyday infrastructure:** connection pools (HikariCP's pool is usually one per data source), metrics registries (Micrometer's global registry), feature-flag clients (switches that turn features on or off without redeploying) — each stands for something that truly exists once (one pool of sockets, one set of counters). Ask that question before choosing the pattern. The browser's `window` object is a singleton per page.
- **Head First's example:** a chocolate factory's boiler controller — there is one physical boiler, so two controller objects could both try to fill it and overflow it. Their fix walks through the same lazy → synchronized → double-checked / eager progression you saw above.

In the interview

> **30-second answer:** "Singleton guarantees one instance per JVM (strictly, per class loader) with a single access point: private constructor, static accessor. I'd use eager initialization — the class loader makes it thread-safe for free — or the holder idiom if construction is expensive. If it must survive serialization or reflection I'd use an enum. But it's global state with hidden dependencies, so in a Spring app I'd just make it a singleton-scoped bean and inject it."

Follow-ups:

- *Q: Why is the naive lazy version broken?* — A: It is check-then-act without synchronization; two threads can both see `null` and both construct.
- *Q: Why does DCL need `volatile`?* — A: Without it the reference can be published before the constructor's writes are visible, so another thread might use a half-initialized object. `volatile` gives a happens-before edge between the write and later reads.
- *Q: Why is the holder idiom thread-safe without locks?* — A: The JVM initializes a class once and publishes its static fields safely; the nested class is only initialized on first access.
- *Q: How can a singleton be broken?* — A: Reflection, serialization, cloning, multiple class loaders. Fixes: constructor guard, `readResolve`, throwing `clone`, or just use an enum.
- *Q: Singleton vs static class?* — A: A singleton is an object: it can implement interfaces, be passed around, be lazily created and be mocked (replaced in a test by a fake that records how it was called). A static class can do none of that.

- *Q: Is Singleton an anti-pattern?* — A: It is often *used* as one (glorified global). It is fine for genuinely single resources; prefer DI to enforce "one instance" from outside.
- *Q: How do you make a unique ID generator across 10 servers?* — A: Not with a Singleton — it is one per JVM. Use a database sequence (a database counter that hands out increasing numbers), a Snowflake-style ID (timestamp + machine id + counter), or UUIDs (128-bit randomly generated IDs that practically never repeat).

Self-check

- **Q:** Which single keyword enforces the pattern? **A:** `private` on the constructor.
- **Q:** Which Java variant is lazy, lock-free and thread-safe? **A:** The Bill Pugh holder idiom.
- **Q:** Which variant resists both reflection and serialization with no extra code? **A:** The single-element enum.
- **Q:** What does `readResolve` do? **A:** It replaces the object produced by deserialization with the one you return — the real instance.
- **Q:** In Spring, how many instances of a default bean exist? **A:** One per application context (container), not one per JVM.

TL;DR

Private constructor + static accessor = one instance per JVM. Prefer **eager**, then **holder**, then **enum** (bullet-proof); **DCL needs** `volatile`; never naive lazy. Know the four ways to break it. Mention the costs (global state, hidden deps, tests) and that DI singleton scope is usually the better tool.

8.9 Simple Factory (an idiom, not a GoF pattern)

What it is

A **Simple Factory** is one class (usually one static method) that takes a type value — a string or enum — and returns the matching object, typed as a common interface. It is extremely common and often the right answer, but it is **not** one of the 23 GoF patterns. *Head First* calls it a programming idiom rather than a real pattern.

Everyday analogy. A vending machine: you press "B3" and a snack comes out. You do not know or care how the machine picks and releases the item — you only know you asked for B3.

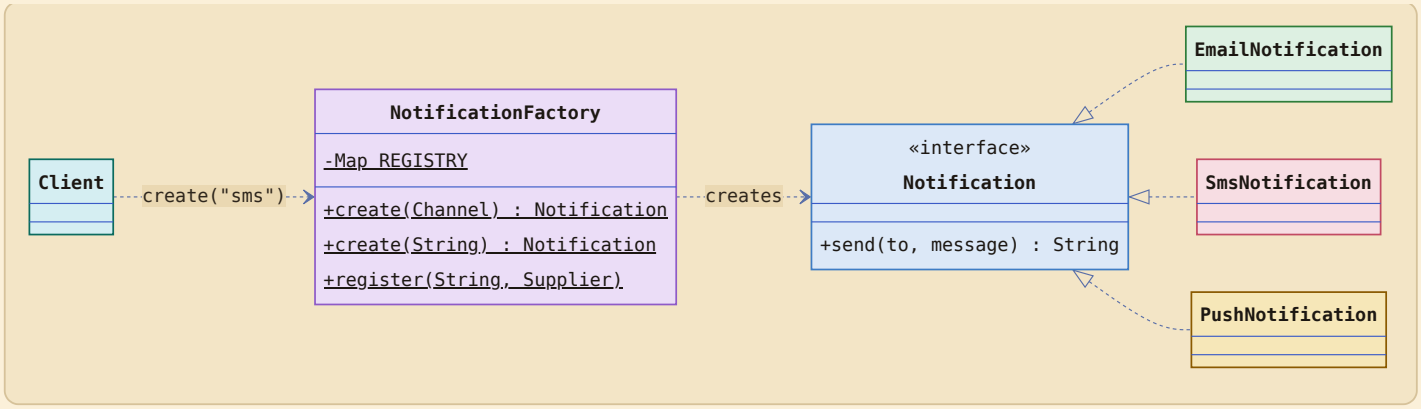
The problem it solves

Without it, the `if/else` or `switch` that turns `"sms"` into `new SmsNotification()` is copied into every place that needs a notification. A new channel means hunting down every copy. A simple factory **centralizes** that decision in one place, and callers depend only on the `Notification` interface.

Important: it **centralizes** the decision; it does not **remove** it. Adding a channel still means editing the factory, and the factory still knows every type. The registry version below softens this — new types are *registered* at start-up rather than coded into a switch.

How it works

1. Define a product **interface** (`Notification`).
2. Write the concrete classes (`EmailNotification`, `SmsNotification`, `PushNotification`).
3. Write one `create(type)` method that maps the type to a concrete class — with an exhaustive `switch` over an enum, or with a `Map<String, Supplier<Notification>>` registry (a lookup table from type names to `Supplier`s; a `Supplier` is Java's built-in "give me a new value" function type).
4. Callers call `create(...)` and hold the result as the interface. They never write `new ConcreteClass()`.
5. Reject unknown types loudly (`IllegalArgumentException`).



Java implementation

```java
// File: code/lld/ch08/simplefactory/Notification.java
package lld.ch08.simplefactory;

/** The product interface every caller programs against. */
public interface Notification {
    String send(String to, String message);
}

record EmailNotification() implements Notification {
    public String send(String to, String message) { return "EMAIL to " + to + ": " + message; }
}

record SmsNotification() implements Notification {
    public String send(String to, String message) { return "SMS to " + to + ": " + message; }
}

record PushNotification() implements Notification {
    public String send(String to, String message) { return "PUSH to " + to + ": " + message; }
}
```

```java
// File: code/lld/ch08/simplefactory/NotificationFactory.java
package lld.ch08.simplefactory;

import java.util.Map;
import java.util.Locale;
import java.util.concurrent.ConcurrentHashMap;
import java.util.function.Supplier;

/** Simple Factory (an idiom, not a GoF pattern): one static method owns the "which class?" decision. */
public final class NotificationFactory {

    public enum Channel { EMAIL, SMS, PUSH }

    private NotificationFactory() { }

    /** Version 1: a switch. Adding a channel means editing this method (and the enum). */
    public static Notification create(Channel channel) {
        return switch (channel) {                       // exhaustive switch: the compiler checks every enum value
            case EMAIL -> new EmailNotification();
            case SMS   -> new SmsNotification();
            case PUSH  -> new PushNotification();
        };
    }

    /** Version 2: a registry of suppliers. New types are registered, not coded into a switch. */
    private static final Map<String, Supplier<? extends Notification>> REGISTRY = new ConcurrentHashMap<>(Map.of(
            "email", EmailNotification::new,
            "sms", SmsNotification::new,
            "push", PushNotification::new));

    public static void register(String type, Supplier<? extends Notification> supplier) {
        REGISTRY.put(type.toLowerCase(Locale.ROOT), supplier);
    }

    public static Notification create(String type) {
        Supplier<? extends Notification> s = REGISTRY.get(type.toLowerCase(Locale.ROOT));
        if (s == null) throw new IllegalArgumentException("Unknown notification type: " + type);
        return s.get();
    }
}
```

```java
// File: code/lld/ch08/simplefactory/Main.java
package lld.ch08.simplefactory;

public class Main {
    public static void main(String[] args) {
        System.out.println("== Simple Factory with an enum switch ==");
        for (NotificationFactory.Channel c : NotificationFactory.Channel.values()) {
            Notification n = NotificationFactory.create(c);      // caller never writes "new XxxNotification"
            System.out.println("  " + n.send("ada", "Your order shipped"));
        }

        System.out.println("\n== Simple Factory with a registry (string keys from config) ==");
        System.out.println("  " + NotificationFactory.create("SMS").send("+91-98xxxx", "OTP 4821"));
        try {
            NotificationFactory.create("whatsapp");
        } catch (IllegalArgumentException e) {
            System.out.println("  rejected: " + e.getMessage());
        }
        // A new channel is registered at start-up - the factory's code is not edited.
        NotificationFactory.register("whatsapp", () -> (to, msg) -> "WHATSAPP to " + to + ": " + msg);
        System.out.println("  " + NotificationFactory.create("whatsapp").send("grace", "Table for 2 confirmed"));
    }
}
```

- The enum `switch` has **no** `default`: the Java compiler checks that every `Channel` value is covered, so adding `WHATSAPP` to the enum without handling it is a compile error. That is a nice safety net.
- The registry version maps lower-cased names to **constructor references** (`EmailNotification::new` — shorthand for "call this constructor"). `register(...)` lets start-up code add a channel without touching the factory. Because `Notification` has one abstract method, a lambda can implement it (an interface with exactly one abstract method is called a *functional interface*).
- `Locale.ROOT` avoids surprising lower-casing in some locales (language-and-region settings) (the Turkish dotless "i" problem).

Output:

```
== Simple Factory with an enum switch ==
  EMAIL to ada: Your order shipped
  SMS to ada: Your order shipped
  PUSH to ada: Your order shipped

== Simple Factory with a registry (string keys from config) ==
  SMS to +91-98xxxx: OTP 4821
  rejected: Unknown notification type: whatsapp
  WHATSAPP to grace: Table for 2 confirmed
```

Trade-offs

Simple Factory	Factory Method (next section)
One method, usually a switch	The decision is spread across subclasses, one each
Adding a type = edit the factory	Adding a type = add a class
No inheritance needed	Needs a creator subclass per product
Not a GoF pattern	A GoF pattern

Which to use: when the list of types is short, rarely changes and is selected by a configuration value, a Simple Factory is the shorter, clearer choice and is perfectly respectable. Move to Factory Method when new types will be contributed by *other* developers — perhaps in separate modules — who should be able to add them without modifying your class.

When not to use it

- There is only **one** concrete class — just call the constructor.
- The objects differ only in **data**, not behaviour — use one class with a field.

Common mistakes

- Calling a switch-based factory a "Factory Method" in an interview.
- A `default` branch that silently returns `null` or a "default" object instead of failing.
- Letting the factory grow into a God class that also configures, caches and logs.

Real-world examples

These are **static factory methods** (Joshua Bloch, *Effective Java* item 1): named static methods that return an instance, possibly of a hidden subclass. They are Simple-Factory style, **not** the GoF Factory Method — there is no Creator class whose subclasses override a creation hook — and **not** Singletons: most return a new object on each call.

- `Calendar.getInstance()` — returns a **new** `GregorianCalendar` on every call in most locales (a different calendar for a few, such as Japanese imperial or Buddhist). The name suggests Singleton; it is not one.
- `NumberFormat.getInstance()` / `getCurrencyInstance(locale)` — also a fresh (locale-specific, usually `DecimalFormat`) object per call.
- `EnumSet.noneOf(type)` — returns a `RegularEnumSet` (≤ 64 constants, one `long` bit-vector) or a `JumboEnumSet`; you never see either class.
- `List.of(...)`, `Map.of(...)`, `Optional.of(...)`, `Integer.valueOf(...)`, `Path.of(...)`.
- `Executors.newFixedThreadPool(n)`, `Executors.newVirtualThreadPerTaskExecutor()` — return configured `ExecutorService` implementations (Java's thread-pool interface for running tasks on reusable threads).

- **Spring:** `@Bean` methods (`@Bean` is an annotation — a `@` tag on code that a framework reads) in a `@Configuration` class are factory methods the container calls to create beans.

In the interview

30-second answer: "A simple factory is one method that maps a type value to a concrete class and returns it as an interface. It centralizes the switch so callers don't depend on concrete classes. It's an idiom, not a GoF pattern — adding a type still edits the factory. If I want new types without editing it, I use a registry of suppliers or Factory Method."

- *Q: Is it Open–Closed?* — A: Not strictly: the factory is edited for each new type. A registry makes it open for extension via registration.
- *Q: Static factory method vs constructor?* — A: Static factories have names, can cache/return existing instances (`Integer.valueOf`), and can return subtypes. Constructors cannot.

Self-check

- **Q:** What does a Simple Factory centralize but not remove? **A:** The switch (knowledge of every concrete type).
- **Q:** Why does the enum `switch` have no `default`? **A:** So the compiler flags any unhandled enum constant.

TL;DR

One `create(type)` method, returns an interface. Great default for small, stable sets of types. Not GoF. Registry of `Supplier`s = extensible variant.

8.10 Factory Method

What it is

Factory Method defines a method for creating an object in a base class, but lets **subclasses decide which concrete class** that method returns. The base class then uses the created object without ever naming its concrete type.

Everyday analogy. A franchise restaurant chain writes one operations manual: take the order, *prepare the dish*, serve, bill. Every branch follows the manual exactly, but "prepare the dish" is left to each branch — the New York branch makes a thin-crust pizza, the Chicago branch a deep-dish one. (This is exactly *Head First*'s pizza-store example: `PizzaStore.orderPizza()` is fixed, and each regional store overrides `createPizza()`.)

Terms:

- **Product** — the interface of the object being created (`Document`).
- **Concrete product** — an implementation (`PdfDocument`).
- **Creator** — the base class that declares the factory method and contains the shared logic (`ReportGenerator`).
- **Concrete creator** — a subclass that overrides the factory method (`PdfReportGenerator`).

GoF intent and participants. Intent: declare an operation that creates an object, but leave the choice of concrete class to subclasses — the base class postpones instantiation to them. The four participants are **Product** (the interface of the objects created), **ConcreteProduct** (implements Product), **Creator** (declares the factory method returning a Product, may supply a default implementation, and — crucially — *calls* it from its own operations) and **ConcreteCreator** (overrides the factory method to return a ConcreteProduct).

> **Factory Method is not a "static factory method".** The word "factory" is overloaded. A *static factory method* (`List.of`, `Integer.valueOf`, `Calendar.getInstance`) and a *Simple Factory* (`NotificationFactory.create(type)`, 8.9) are **one static method that decides** what to return. They cannot be overridden (static methods are not polymorphic — a call cannot be redirected to a subclass's version at run time), and there is no Creator hierarchy. The GoF **Factory Method** is an **instance method, overridden in subclasses**, that the base class itself calls. If there is no inheritance and no overriding, it is not this pattern.

The problem it solves

Plenty of classes must create an object whose exact type they should not decide: a report generator needs a document, a parser needs tree nodes, a notification service needs messages. Once `ReportGenerator` contains `new PdfDocument()`, it is hard-wired to PDF, and supporting HTML means reopening a class that is finished and tested.

Our `ReportGenerator` does four things: gather the rows, create a document from them, render it, and announce the file name. **Only the "create a document" step cares about the output format**; the other three are identical for PDF, HTML or CSV. The first instinct is a `switch` on a format string. That holds up until a second method needs the same `switch`, then a third — and from then on each new format means hunting down every copy.

Factory Method instead leaves a deliberate **gap** in the base class: the complete procedure lives there, but at the moment a document is needed it calls an abstract method that the base class does not implement. Every subclass supplies just that one method.

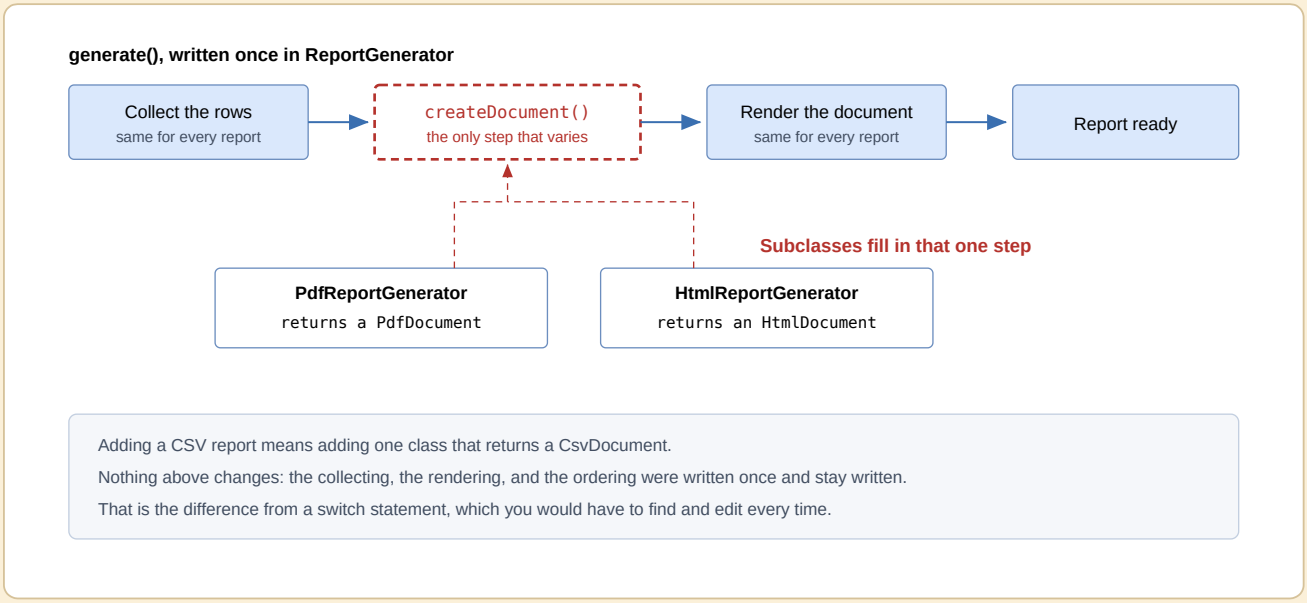


Figure — generate() is written once in ReportGenerator; only createDocument() varies, filled in by PdfReportGenerator or HtmlReportGenerator (course diagram)

In the figure, gathering, rendering and announcing are inherited unchanged; supporting CSV only requires one more subclass whose override returns a `CsvDocument`. Compare that with a `switch`, which has to be located and edited for every new format.

How it works

1. Define the **product interface** (`Document`: `render(rows)`, `fileName()`).
2. Write **concrete products** (`PdfDocument`, `HtmlDocument`, later `CsvDocument`).
3. Write the **creator** as an abstract class with an **abstract, `protected` factory method** (`createDocument()`). `protected` signals that it is an extension point meant for subclasses rather than something outside callers invoke — they call `generate()`.
4. Write the **shared logic** in the creator (`generate()`), calling `createDocument()` where it needs a product. It depends only on `Document`.
5. Write one **concrete creator** per product, overriding only the factory method. That override is the *only* place a concrete type is named.
6. The **client** picks a creator once and then works entirely through `ReportGenerator` and `Document`.

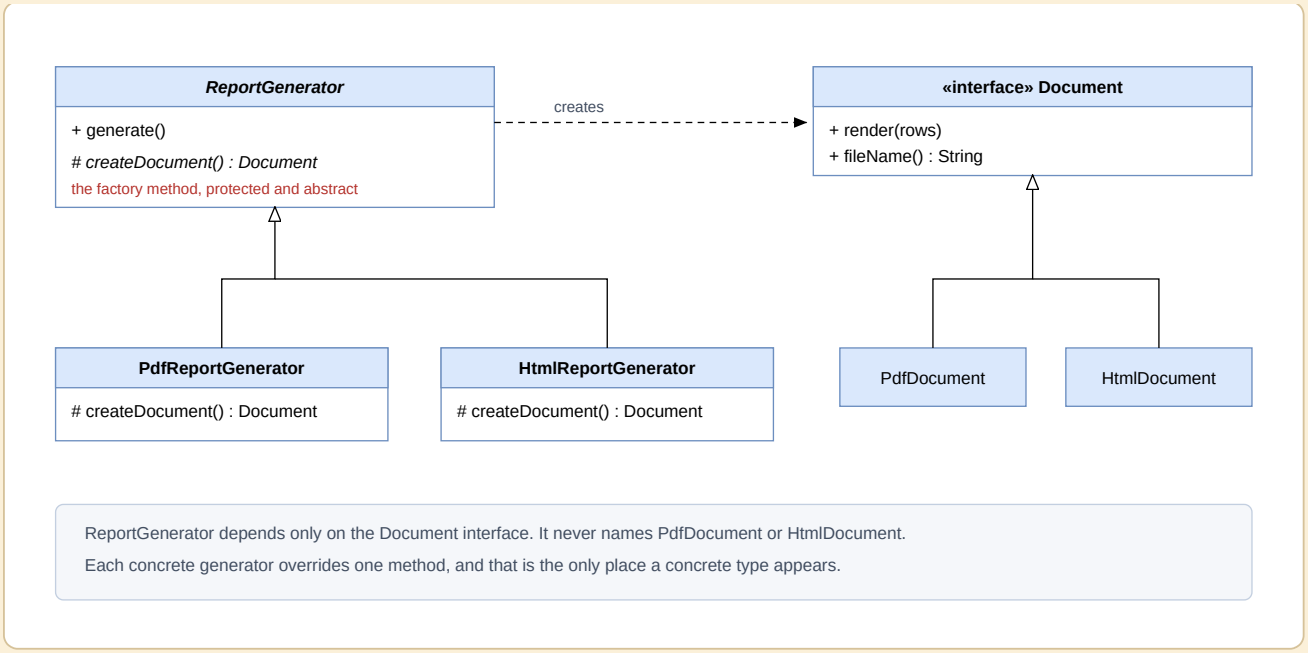
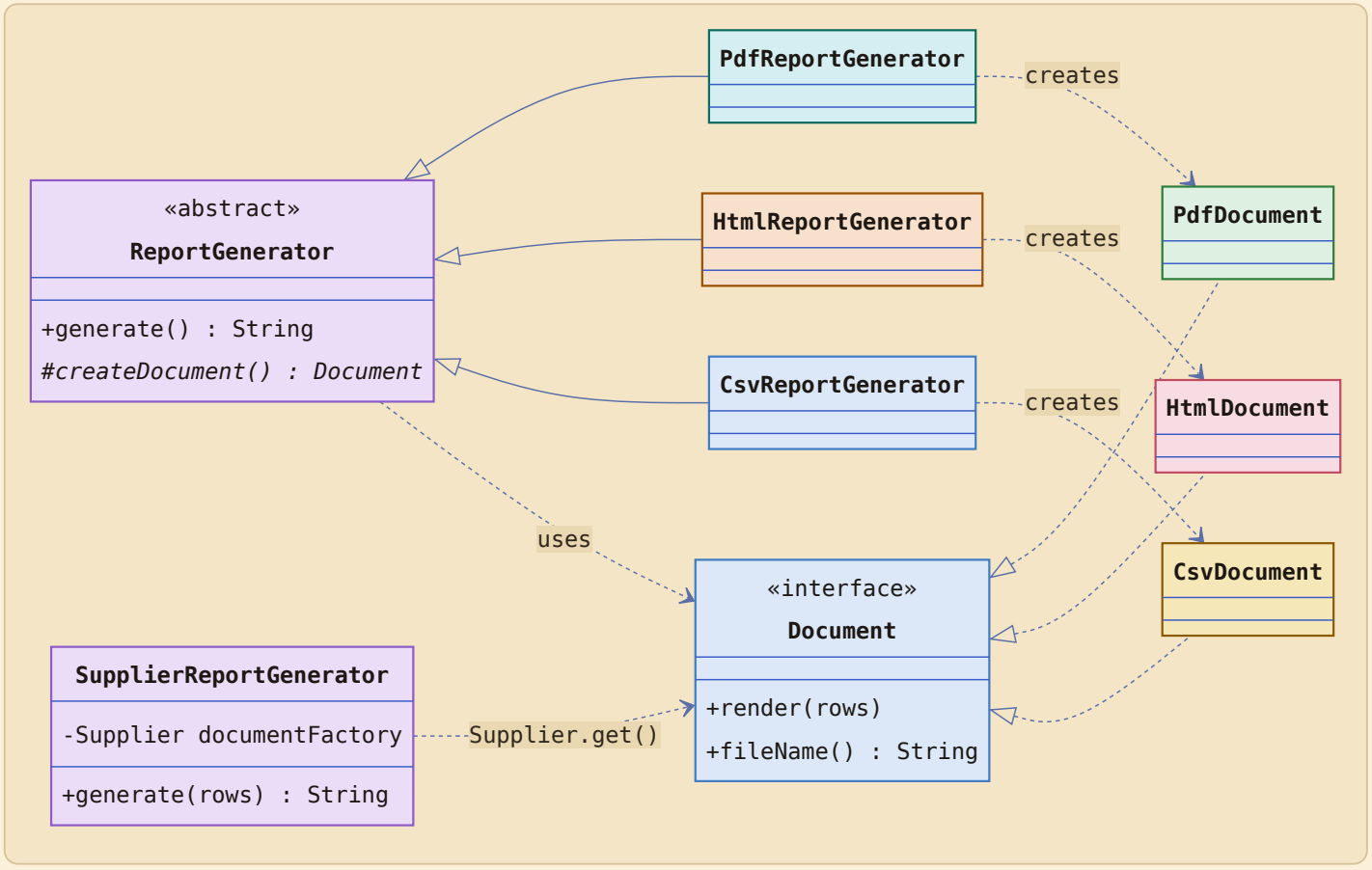


Figure — ReportGenerator declares the protected factory method createDocument(); each concrete generator overrides it to return its own Document (course diagram)



Java implementation

```java
// File: code/lld/ch08/factorymethod/Document.java
package lld.ch08.factorymethod;

import java.util.List;

/** Product: the only document type ReportGenerator knows about. */
public interface Document {
    void render(List<String> rows);
    String fileName();
}

final class PdfDocument implements Document {
    public void render(List<String> rows) { System.out.println("  Rendering as PDF: " + rows.size() + " rows"); }
    public String fileName() { return "sales.pdf"; }
}

final class HtmlDocument implements Document {
    public void render(List<String> rows) { System.out.println("  Rendering as HTML table: " + rows.size() + "
rows"); }
    public String fileName() { return "sales.html"; }
}

/** Added later - no existing class had to change. */
final class CsvDocument implements Document {
    public void render(List<String> rows) { rows.forEach(r -> System.out.println("  csv> " + r)); }
    public String fileName() { return "sales.csv"; }
}
```

```java
// File: code/lld/ch08/factorymethod/ReportGenerator.java
package lld.ch08.factorymethod;

import java.util.List;

/** Creator: writes the whole procedure once and leaves one hole - the factory method. */
public abstract class ReportGenerator {

    /** The factory method. Protected: a hook for subclasses, not public API. */
    protected abstract Document createDocument();

    /** The template that USES the factory method. It never names a concrete document. */
    public final String generate() {
        System.out.println("  Collecting rows for the monthly sales report...");
        List<String> rows = List.of("north,120", "south,95", "east,140");
        Document document = createDocument();
        document.render(rows);
        System.out.println("  Report ready: " + document.fileName());
        return document.fileName();
    }
}

final class PdfReportGenerator extends ReportGenerator {
    @Override protected Document createDocument() { return new PdfDocument(); }
}

final class HtmlReportGenerator extends ReportGenerator {
    @Override protected Document createDocument() { return new HtmlDocument(); }
}

final class CsvReportGenerator extends ReportGenerator {
    @Override protected Document createDocument() { return new CsvDocument(); }
}
```

```java
// File: code/lld/ch08/factorymethod/SupplierReportGenerator.java
package lld.ch08.factorymethod;

import java.util.List;
import java.util.function.Supplier;

/** The modern, composition-based variant: pass the "factory method" in as a Supplier. */
public final class SupplierReportGenerator {
    private final Supplier<? extends Document> documentFactory;

    public SupplierReportGenerator(Supplier<? extends Document> documentFactory) {
        this.documentFactory = documentFactory;
    }

    public String generate(List<String> rows) {
        Document d = documentFactory.get();
        d.render(rows);
        return d.fileName();
    }
}
```

```java
// File: code/lld/ch08/factorymethod/Main.java
package lld.ch08.factorymethod;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        // The client picks a creator once, then only talks to ReportGenerator and Document.
        for (ReportGenerator g : List.of(new PdfReportGenerator(), new HtmlReportGenerator(), new
CsvReportGenerator())) {
            System.out.println("== " + g.getClass().getSimpleName() + " ==");
            g.generate();
        }

        System.out.println("\n== Supplier-based variant (composition instead of subclassing) ==");
        SupplierReportGenerator gen = new SupplierReportGenerator(HtmlDocument::new);
        System.out.println("  produced " + gen.generate(List.of("west,77")));

        // A test can plug in a fake product with a lambda - no subclass needed.
        SupplierReportGenerator testGen = new SupplierReportGenerator(() -> new Document() {
            public void render(List<String> rows) { System.out.println("  (fake) captured " + rows); }
            public String fileName() { return "fake.txt"; }
        });
        System.out.println("  produced " + testGen.generate(List.of("row-1")));
    }
}
```

- `generate()` is `final`: subclasses may change *what* is created, not the procedure. Notice it never mentions PDF, HTML or CSV.
- `CsvDocument` + `CsvReportGenerator` were "added later" — **no existing class was edited** (Open–Closed Principle).
- `SupplierReportGenerator` is the modern alternative: pass a `Supplier<Document>` (a function that creates a product) instead of subclassing. It composes better and a test can supply a lambda.

Output:

```
== PdfReportGenerator ==
  Collecting rows for the monthly sales report...
  Rendering as PDF: 3 rows
  Report ready: sales.pdf
== HtmlReportGenerator ==
  Collecting rows for the monthly sales report...
  Rendering as HTML table: 3 rows
  Report ready: sales.html
== CsvReportGenerator ==
  Collecting rows for the monthly sales report...
  csv> north,120
  csv> south,95
  csv> east,140
  Report ready: sales.csv

== Supplier-based variant (composition instead of subclassing) ==
  Rendering as HTML table: 1 rows
  produced sales.html
  (fake) captured [row-1]
  produced fake.txt
```

All three generators print an identical "Collecting rows…" line, since that code exists only once, in `ReportGenerator`; the rendering line is the only thing that changes.

Trade-offs

Pros	Cons
Extensible without edits — supporting another product means writing another class; no existing class is reopened.	**More classes** — every product type needs its own creator subclass, which is a lot of files for a small feature.
Common procedure in one place — everything around the creation step is written once in the base class.	**Indirection** — the concrete type is not visible where it is used; you have to find out which subclass you are holding.
Built-in test seam (a spot where a test can plug in its own object) — a test can subclass the creator and hand back a fake product.	**Relies on inheritance** — Java allows one superclass, and inheritance combines less flexibly than passing in a function.
Nobody holds the full list of types — no class has to know every format.	**Tempting too early** — with only one product type it is all cost and no benefit.

Often confused with:

Looks like	The actual difference
Simple Factory	One static `create` method with a `switch`. Not a GoF pattern, and frequently the better choice for small stable sets. It gathers the `switch` into one place instead of eliminating it.
Abstract Factory	Factory Method produces **one** product and varies it by **subclassing** the creator. Abstract Factory produces a **family** of products and is varied by **passing in** a different factory object (composition). Abstract Factories are often built out of factory methods.
Template Method	Same mechanism (a base-class method calls an overridable hook), different goal: Template Method lets subclasses vary a *step of an algorithm*; Factory Method lets them vary *which object gets created*. Factory Method can be seen as a special case of Template Method — `generate()` here *is* a template method.
Builder	Factory Method chooses a type in a single call; Builder assembles one complex object across many calls.
"Static factory method" (`List.of`)	A named static method used instead of a constructor, *Effective Java* style. It cannot be overridden, so it is not this pattern — only the word "factory" is shared.

In other languages (and modern Java): wherever functions are values, you can hand the creator a *creation function* instead of subclassing it. Python: `ReportGenerator(create_document=PdfDocument)` (a class is callable). JavaScript/TypeScript: pass `() => new PdfDocument()` into the constructor, and a test passes its own arrow

function (JavaScript's short `() => ...` function syntax). Java: a `Supplier<Document>` field, as in `SupplierReportGenerator`. This is usually the better default because it needs no new class; keep the subclassing form for framework base classes that other teams extend — which is where the pattern was born.

When not to use it

- **There is one product type and no second one on the horizon.** Just call the constructor, and introduce the pattern on the day a second type actually appears. Adding it earlier is the most frequent over-design in the whole creational group.
- **The base class has no shared work of its own.** If `generate()` did nothing except call `createDocument()`, the abstract class would be an interface wearing a costume. The pattern pays only when real logic is written once in the creator.
- **Types differ by data, not by behaviour.** If a configuration string picks the type and the types do not behave differently, a `Map` from string to constructor reference (`Map<String, Supplier<Document>>`) is shorter than a subclass per type.
- **You need several objects that must match.** That is Abstract Factory's job.

Common mistakes

- Making the factory method **public** — it becomes part of the API (Application Programming Interface — the methods others are meant to call) and callers bypass the shared procedure.
- Calling the base class's factory method from its **constructor** — the subclass's fields are not initialized yet when the override runs.
- Calling a switch-in-one-method a Factory Method. (A learner asked exactly this about a code sample: if one method decides with `if/switch`, it is a *simple factory*; if subclasses override a creation hook used by shared base-class logic, it is a *factory method*.)

Real-world examples

- **JDK:** `Iterable.iterator()` / `Collection.iterator()` — the interface declares the creation method (Creator), and `ArrayList`, `LinkedList`, `HashSet`… (ConcreteCreators) each return their own `Iterator` implementation (ConcreteProduct); the enhanced `for` loop and every method written against `Iterator` (Product) never learn which one. `AbstractList` implements `iterator()` on top of its abstract `get(int)` — the template/factory mix in action.
- **JDK XML (as a building block):** `DocumentBuilderFactory.newDocumentBuilder()` and `TransformerFactory.newTransformer()` are abstract methods that each parser implementation's factory subclass overrides — factory methods living inside what, as a whole, is an Abstract Factory (8.11). Note that `DocumentBuilderFactory.newInstance()` itself is a *static* factory method, not a Factory Method.
- **Spring:** `FactoryBean<T>.getObject()` — a bean whose job is to create another object (e.g. proxies, `SqlSessionFactoryBean` in MyBatis-Spring). Spring's `AbstractApplicationContext` also exposes overridable creation hooks for subclasses.
- **Frameworks:** Android's `Fragment.onCreateView()` — you override a `create…` hook the framework calls. JUnit/TestNG (Java testing frameworks) fixture (shared test setup) base classes with overridable `create…` methods.

In the interview

30-second answer: "Factory Method puts a creation hook in a base class. The base class writes the whole procedure once against a product interface; each subclass overrides the protected factory method to return its concrete product. New types are new classes, so it's Open–Closed. It only pays off when there's genuine shared logic in the base class — otherwise a constructor or a simple factory is simpler. In modern Java I'd often pass a `Supplier` instead of subclassing."

- *Q: Difference from Simple Factory?* — A: Simple Factory = one (static) method with a switch that knows every type. Factory Method = subclasses override an instance hook that the base class calls; nothing knows the full list. Raise this distinction yourself before you are asked — calling a `switch` a "Factory Method" is a classic slip interviewers listen for.
- *Q: Difference from Abstract Factory?* — A: One product via inheritance vs a family of matched products via a factory object passed in.
- *Q: Why protected?* — A: It is an extension point for subclasses, not something callers should invoke directly.

- *Q: How do you test `generate()`?* — A: Subclass the creator in the test (or pass a `Supplier`) and return a fake `Document` that records calls.

Self-check

- **Q:** Where is the only place a concrete product class is named? **A:** Inside each concrete creator's override of the factory method.
- **Q:** What must exist in the base class for this pattern to be worthwhile? **A:** Shared logic that uses the product (here, `generate()`).
- **Q:** Which principle does adding `CsvReportGenerator` demonstrate? **A:** Open–Closed.

TL;DR

Base class = procedure + abstract `protected createX()`; subclasses decide the concrete type. One product, varied by inheritance. Worth it only with shared logic and multiple (growing) product types; `Supplier` is the lightweight modern form.

8.11 Abstract Factory

What it is

Abstract Factory provides an interface for creating **families of related objects** that must be used together, without the client naming any concrete class. Choosing a family = choosing a factory; everything created afterwards automatically matches.

Everyday analogy. Furniture showrooms sell matching sets. If you pick the "Victorian" catalogue, the chair, sofa and table you order all come in the Victorian style. You cannot accidentally get a modern chair with a Victorian sofa, because you ordered from one catalogue. (The course summary uses the GUI (graphical user interface) version: one factory per OS (operating system) theme — Windows or macOS — producing buttons, checkboxes and scrollbars that belong together. *Head First* uses a pizza *ingredient* factory per region: New York dough, sauce and cheese vs Chicago dough, sauce and cheese.)

Terms:

- **Family** — a set of products designed to work together (Postgres connection + Postgres query builder; Postgres is short for PostgreSQL, a popular open-source database, and a query builder is an object that writes the SQL text for you).
- **Abstract product** — an interface per product kind (`Connection`, `QueryBuilder`).
- **Abstract factory** — an interface with one `create…` method per product kind (`StorageFactory`).
- **Concrete factory** — implements *every* create method for one family (`PostgresFactory`).

GoF intent and participants. Intent: offer one interface through which a client creates a whole set of related or interdependent objects, without naming their concrete classes. Participants: **AbstractFactory** (declares one creation operation per product kind — `StorageFactory`), **ConcreteFactory** (implements those operations for one family — `PostgresFactory`), **AbstractProduct** (the interface for one kind of product — `Connection`, `QueryBuilder`), **ConcreteProduct** (a product of one family, created by the matching concrete factory — `PostgresQueryBuilder`) and **Client** (uses *only* the AbstractFactory and AbstractProduct types — `ReportService`). GoF also notes that an application usually needs just one ConcreteFactory per family, which is why it is often made a Singleton (though passing it in is better).

The problem it solves

Certain objects are only correct alongside partners from the same variant. A Postgres connection needs a Postgres-flavoured query builder: Postgres wraps identifiers (table and column names) in double quotes, MySQL in backticks. A macOS-styled button should sit next to a macOS-styled checkbox. A test run wants a fake payment client, a fake mailer *and* a fake clock — never a mixture of fake and real.

When each of those objects is constructed independently, nothing prevents a bad combination. Pairing a `PostgresConnection` with a `MySqlQueryBuilder` compiles without a murmur; the mistake only surfaces later as an SQL (the standard language for querying relational databases) syntax error from the server, with a stack trace (the list of method calls that were active when the error happened) that points nowhere near the two lines that made the wrong choice.

This is also what separates it from Factory Method: Factory Method varies **one** product, Abstract Factory varies a **set** of products that must stay consistent.

Abstract Factory hands responsibility for the **entire set to one object**. Each concrete factory can only return members of its own family, so writing a mismatched pair is no longer possible through the factory. Remember the pattern by what it guarantees: *it does not help you choose the right combination — it removes the ability to build a wrong one.*

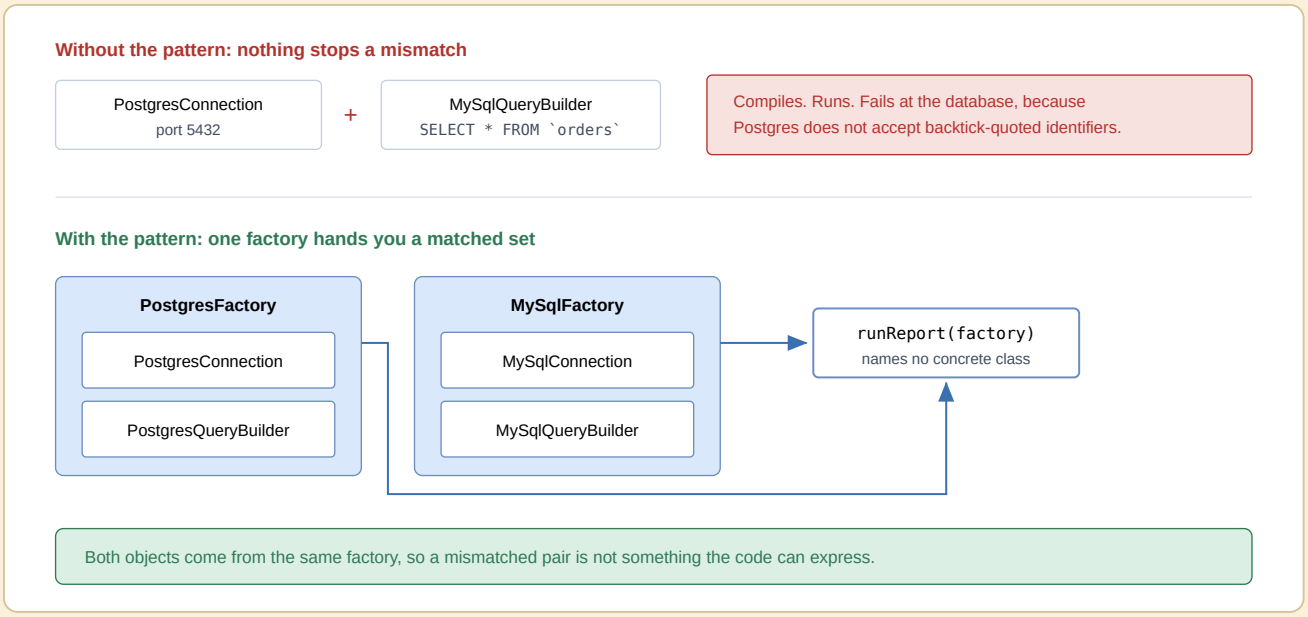


Figure — without the pattern a PostgresConnection can be paired with a MySqlQueryBuilder and fail at the database; with it, each factory hands out a matched set (course diagram)

The upper half of the figure shows the failure — the code compiles and starts, and then Postgres rejects the table name wrapped in backticks. The lower half shows the cure: `runReport(factory)` obtains both objects from one factory.

How it works

1. Identify the **product kinds** in the family and give each an interface (`Connection`, `QueryBuilder`).
2. Write the **abstract factory** interface with one creation method per product kind.
3. For each variant, write the **concrete products** and a **concrete factory** that implements *every* method and returns only that family's products (Java's covariant return types (an override may declare a more specific return type than the method it overrides) let `PostgresFactory.createConnection()` declare `PostgresConnection`, documenting the family).
4. The **client** receives a factory (composition, usually via its constructor) and only uses the abstract types.
5. Choose the concrete factory **once**, at the **composition root** — the spot close to `main` where the application wires its concrete parts together — and pass it down.

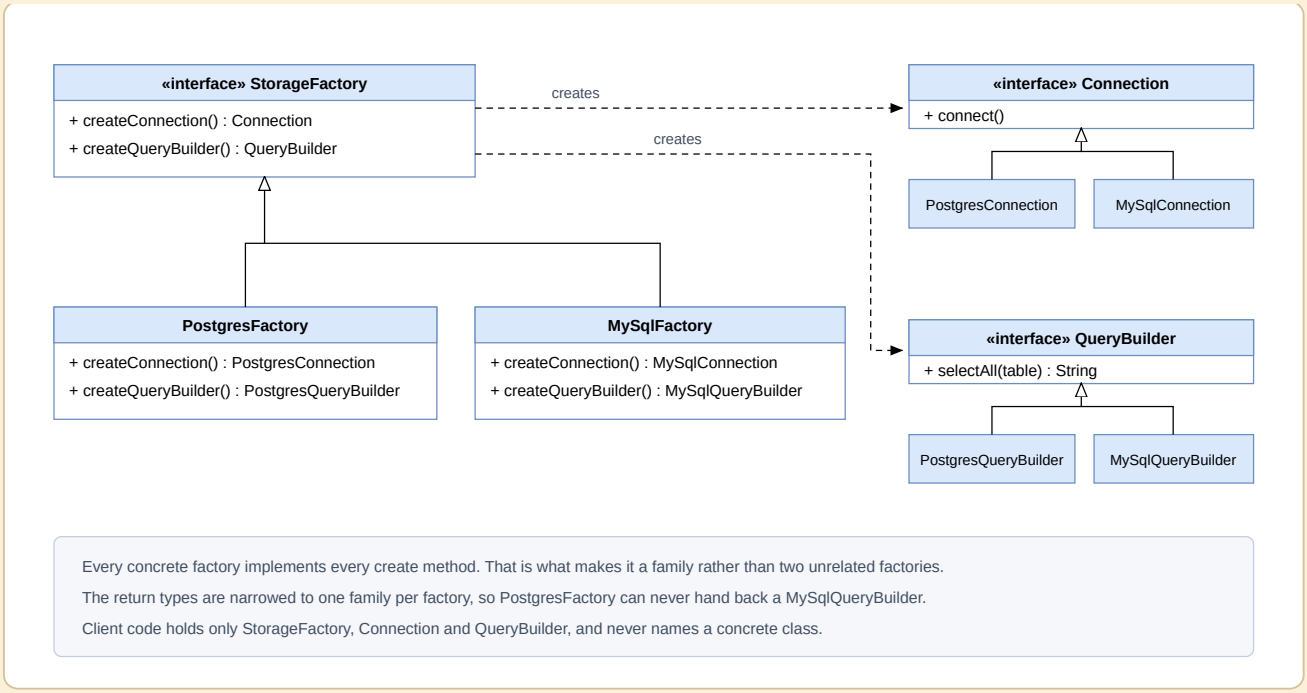
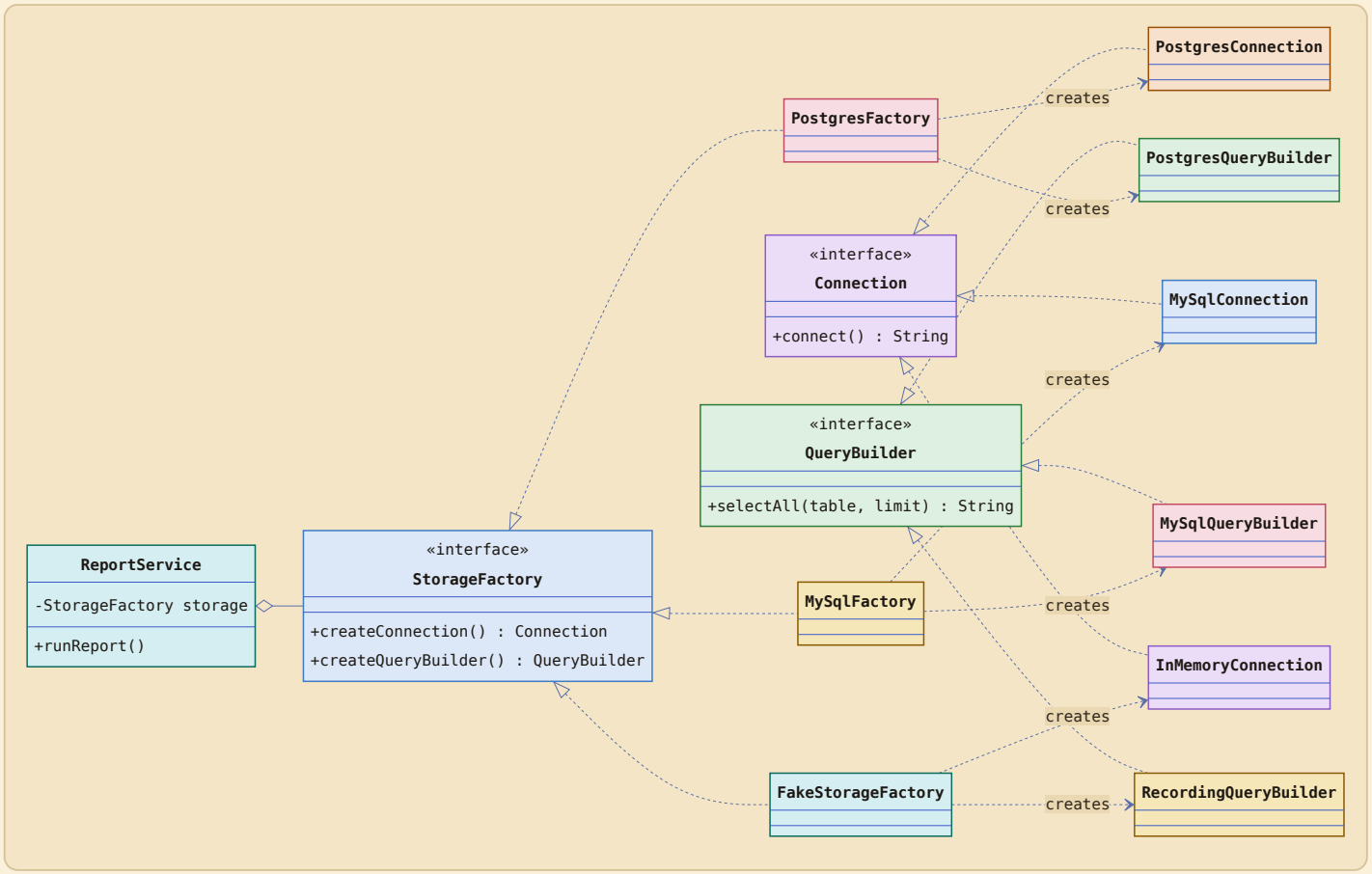
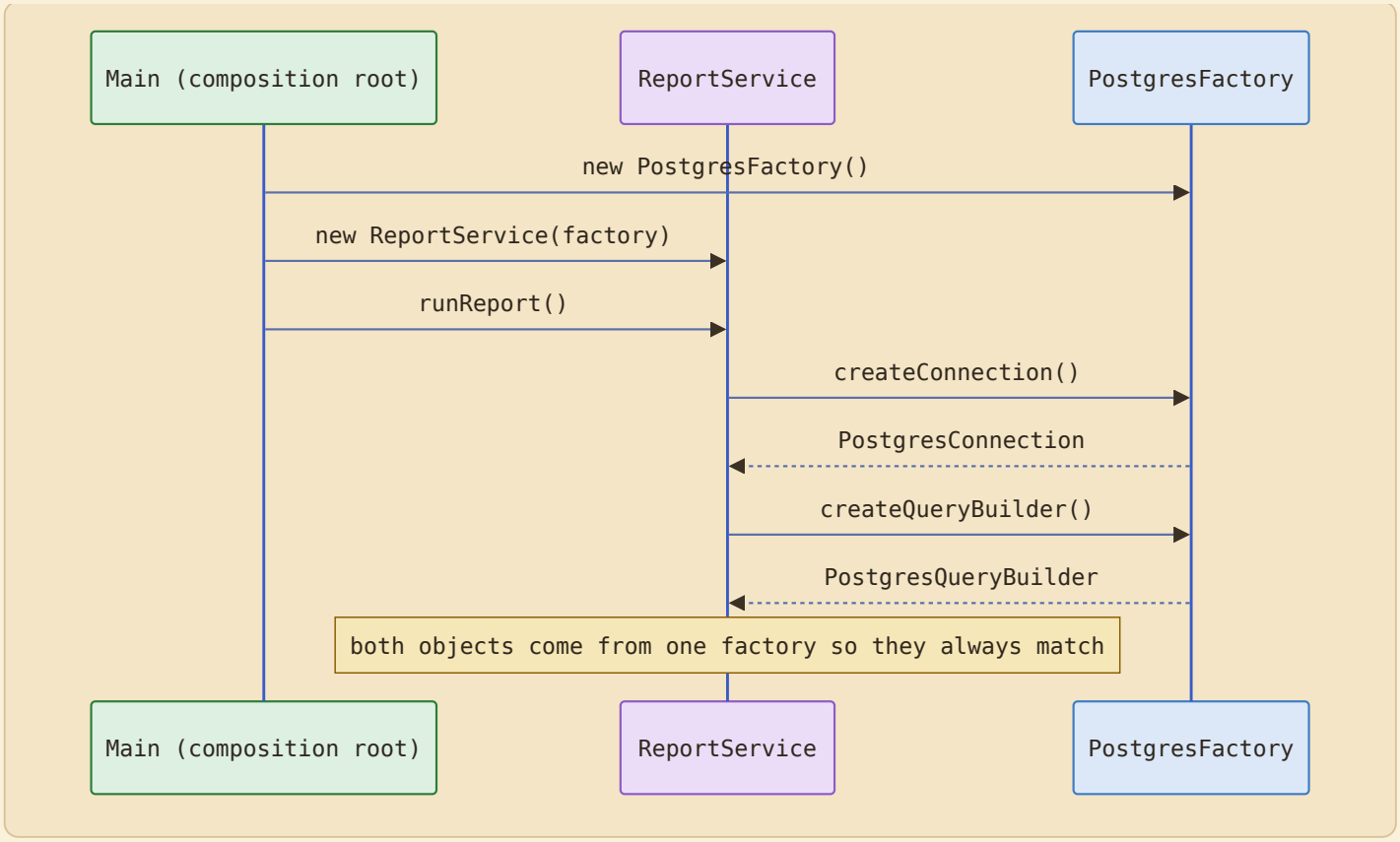


Figure — StorageFactory with createConnection() and createQueryBuilder(); PostgresFactory and MySqlFactory each create their own Connection and QueryBuilder (course diagram)

Note in the figure above that every concrete factory implements *every* create method — that is what makes it a family rather than two unrelated factories. (A learner pointed out that an older café example showed factories returning only one product each; a correct Abstract Factory must return **all** products of its family.)




Main (composition root)
ReportService
PostgresFactory
new PostgresFactory()
new ReportService(factory)
runReport()
createConnection()
PostgresConnection
createQueryBuilder()
PostgresQueryBuilder
both objects come from one factory so they always match
Main (composition root)
ReportService
PostgresFactory

Java implementation

```
// File: code/lld/ch08/abstractfactory/Products.java
package lld.ch08.abstractfactory;

/** Abstract product 1. */
interface Connection {
    String connect();
}

/** Abstract product 2. Must agree with the Connection about the SQL dialect. */
interface QueryBuilder {
    String selectAll(String table, int limit);
}

// ---- Postgres family ----
final class PostgresConnection implements Connection {
    public String connect() { return "connected to postgres://localhost:5432/app"; }
}
final class PostgresQueryBuilder implements QueryBuilder {
    public String selectAll(String table, int limit) {
        return "SELECT * FROM \"" + table + "\" LIMIT " + limit;      // Postgres quotes with double quotes
    }
}

// ---- MySQL family ----
final class MySqlConnection implements Connection {
    public String connect() { return "connected to mysql://localhost:3306/app"; }
}
final class MySqlQueryBuilder implements QueryBuilder {
    public String selectAll(String table, int limit) {
        return "SELECT * FROM `" + table + "` LIMIT " + limit;        // MySQL quotes with backticks
    }
}

// ---- Fake family for tests ----
final class InMemoryConnection implements Connection {
    public String connect() { return "connected to in-memory store (no network)"; }
}
final class RecordingQueryBuilder implements QueryBuilder {
    public String selectAll(String table, int limit) { return "[recorded] selectAll(" + table + ", " + limit + ")";
}
}
```

```java
// File: code/lld/ch08/abstractfactory/StorageFactory.java
package lld.ch08.abstractfactory;

/** Abstract factory: one creation method per product in the family. */
public interface StorageFactory {
    Connection createConnection();
    QueryBuilder createQueryBuilder();
}

/** Concrete factories implement EVERY method and return only their own family (covariant return types). */
final class PostgresFactory implements StorageFactory {
    @Override public PostgresConnection createConnection()     { return new PostgresConnection(); }
    @Override public PostgresQueryBuilder createQueryBuilder() { return new PostgresQueryBuilder(); }
}

final class MySqlFactory implements StorageFactory {
    @Override public MySqlConnection createConnection()     { return new MySqlConnection(); }
    @Override public MySqlQueryBuilder createQueryBuilder() { return new MySqlQueryBuilder(); }
}

final class FakeStorageFactory implements StorageFactory {
    @Override public Connection createConnection()     { return new InMemoryConnection(); }
    @Override public QueryBuilder createQueryBuilder() { return new RecordingQueryBuilder(); }
}
```

```java
// File: code/lld/ch08/abstractfactory/ReportService.java
package lld.ch08.abstractfactory;

/** Client: holds only the abstract factory and abstract products. It cannot mix families. */
public final class ReportService {
    private final StorageFactory storage;

    public ReportService(StorageFactory storage) { this.storage = storage; }

    public void runReport() {
        Connection connection = storage.createConnection();
        QueryBuilder queries = storage.createQueryBuilder();
        System.out.println("  " + connection.connect());
        System.out.println("  " + queries.selectAll("orders", 10));
    }
}
```

```java
// File: code/lld/ch08/abstractfactory/Main.java
package lld.ch08.abstractfactory;

public class Main {

    /** Composition root: the ONE place that chooses the family. */
    static StorageFactory chooseFactory(String db) {
        return switch (db) {
            case "postgres" -> new PostgresFactory();
            case "mysql"    -> new MySqlFactory();
            case "fake"     -> new FakeStorageFactory();
            default -> throw new IllegalArgumentException("unknown db " + db);
        };
    }

    public static void main(String[] args) {
        String fromEnv = System.getenv().getOrDefault("DB", "postgres");
        for (String db : new String[] { fromEnv, "mysql", "fake" }) {
            System.out.println("== Using " + db + " ==");
            new ReportService(chooseFactory(db)).runReport();   // ReportService never changes
        }
    }
}
```

- `ReportService` contains **no branch** on the database — the SQL differs purely because the factory differs.
- `chooseFactory(...)` is the **composition root** and holds the program's only `switch` on the database. Had we put that `switch` inside `runReport`, it would merely have been relocated, and every other class that later needs a connection would sprout a duplicate of it.
- `FakeStorageFactory` shows the most valuable use: swap the **whole environment** for tests with one line.

Output (with no `DB` environment variable set — an environment variable is a named setting the operating system passes to a program):

```
== Using postgres ==
  connected to postgres://localhost:5432/app
  SELECT * FROM "orders" LIMIT 10
== Using mysql ==
  connected to mysql://localhost:3306/app
  SELECT * FROM `orders` LIMIT 10
== Using fake ==
  connected to in-memory store (no network)
  [recorded] selectAll(orders, 10)
```

Trade-offs

Pros	Cons
Wrong combinations cannot be written — the family is fixed once, when the factory is chosen.	**Many types** — one interface per product kind plus one class per product per family adds up fast.
One switch for a whole environment — real ↔ fake, Postgres ↔ MySQL, light ↔ dark theme.	**New product kinds are expensive** — the factory interface grows, and every concrete factory must be edited.
Client never names a concrete class — it depends only on abstractions (Dependency Inversion).	**Another layer to trace** — to know what was actually created you must find out which factory was picked.
Test seam — a single fake factory stands in for everything outside the process.	**Pointless with one family** — with a single variant it is pure ceremony.

The asymmetry to remember: adding a new **family** (e.g. `SqliteFactory`) is easy and Open–Closed; adding a new **product kind** (e.g. `createMigrationRunner()`) is hard, because every factory must change.

Abstract Factory vs Factory Method

	Factory Method	Abstract Factory
Creates	**One** product	A **family** of related products
Mechanism	**Inheritance** — subclass overrides a method	**Composition** — client holds a factory object
Who chooses the concrete type	The subclass of the creator	The concrete factory passed in
Shape	One abstract method in a class that has other logic	An interface whose *only* job is creation (several methods)
Guarantees	New types without editing the creator	Products from one factory are always compatible
Adding a new variant	New creator subclass + new product	New factory + one new product per kind
Adding a new product kind	n/a	Edit the interface and **every** factory
Relationship	—	Each `create…` method in a concrete factory is often itself a factory method
Typical example	`Collection.iterator()`	`DocumentBuilderFactory`, UI toolkits, DB dialects

Often confused with: *Builder* (Abstract Factory returns several finished products immediately; Builder assembles one product step by step and hands it over at the end); a *DI container* (same outcome through a more general mechanism — Spring profiles or Guice modules bind a whole set of implementations as configuration; if you

already have a container, prefer it); a *Facade* over several factories (a Facade simplifies an existing interface, whereas Abstract Factory guarantees consistency — similar shapes, unrelated purposes).

In other languages: in Python or JavaScript, a plain dictionary/object of creation functions — `{"connection": PostgresConnection, "queries": PostgresQueryBuilder}` — already acts as a factory, no classes needed. In Java, a `record` (Java's short syntax for an immutable data class) with several `Supplier` fields (`record StorageKit(Supplier<Connection> connection, Supplier<QueryBuilder> queries)`) is often shorter than an interface plus one class per family. With a DI container, write the pattern by hand only when you are building the framework, not when using one.

When not to use it

- **The objects do not belong together.** A "factory" exposing `createLogger()` and `createInvoice()` is a grab-bag of unrelated dependencies, not a matched set — inject them separately.
- **There is only one variant** and no second is expected: an interface plus a single implementation is cost without benefit.
- **The set of product kinds keeps growing** (beyond roughly five or six) — each new kind edits every factory; a container or registry scales better.
- **Your DI container already does it** (Spring `@Profile`, Guice modules, NestJS providers) — that *is* this pattern, expressed as configuration instead of code.

Common mistakes

- A concrete factory that implements only some methods, or returns products from another family.
- Choosing the factory deep in the call stack (the chain of method calls that led to the current one) instead of once at the composition root.
- Making the factory a Singleton by reflex — pass it in instead.

Real-world examples

- **JDK:** `javax.xml.parsers.DocumentBuilderFactory`, `javax.xml.transform.TransformerFactory`, `XPathFactory` — the abstract factory class is what client code holds; the static `newInstance()` locates one implementation (e.g. the JDK's built-in Xerces/Xalan-based ones, or Saxon if configured), and that concrete factory hands out objects belonging to the same implementation (`TransformerFactory` creates both `Transformer` and `Templates`). Swapping the implementation swaps the whole set. JDBC's (Java Database Connectivity, Java's standard database API) `java.sql.Connection` acts as a factory for a driver-specific (a driver is the library that talks to one particular database) family: `createStatement()`, `prepareStatement()`, `prepareCall()`, `createBlob()` all return objects of the same driver.
- **UI toolkits:** the original GoF example — a factory per look-and-feel producing matching widgets. Swing's pluggable look-and-feel follows the same idea.
- **ORMs:** Hibernate's `Dialect` bundles database-specific SQL generation, type mapping and identifier quoting that must agree with each other. (An ORM — object-relational mapper — is a library that maps classes to database tables so you rarely write SQL by hand.)
- **Spring:** `@Profile("test")` vs `@Profile("prod")` configurations swap a whole set of beans (fake mailer, fake payments, in-memory DB) together.
- **Cloud SDKs:** switching between AWS and GCP clients in a multi-cloud abstraction (SDK = software development kit, a provider's client library; AWS and GCP are Amazon's and Google's cloud platforms) (storage client + queue client + secrets client per provider).

In the interview

> **30-second answer:** "Abstract Factory is an interface with one create method per product in a family; each concrete factory returns a matched set. The client holds only the factory and the abstract products, so it can't mix, say, a Postgres connection with a MySQL query builder. I pick the factory once at the composition root. The cost is that adding a new product kind means editing every factory."

- *Q: Factory Method vs Abstract Factory?* — A: One product via inheritance vs a family via composition; Abstract Factory often uses factory methods internally.
- *Q: How would you add SQLite?* — A: `SqliteConnection`, `SqliteQueryBuilder`, `SqliteFactory`, and one line in the composition root. Nothing else changes.

- *Q: How would you add a `MigrationRunner` product?* — A: Add `createMigrationRunner()` to the interface and implement it in every factory — this is the pattern's weak spot.
- *Q: How is this used in tests?* — A: A fake factory returns in-memory/recording products, so one line swaps the entire outside world.

Self-check

- **Q:** What property does Abstract Factory guarantee? **A:** Products from one factory are always from the same family — mismatches cannot be expressed.
- **Q:** What is the composition root? **A:** The one place near start-up where concrete choices (like which factory) are made and passed down.
- **Q:** Which change is easy and which is hard? **A:** New family = easy; new product kind = edit every factory.

TL;DR

One factory object → a matched family of products. Chosen once, passed down. Easy to add families, hard to add product kinds. Prefer a DI container's profiles if you already have one.

8.12 Builder

What it is

Builder separates the *construction* of a complex object from the object itself. A separate builder object collects the values **one named step at a time**, and a final `build()` call produces the finished — usually **immutable** — product. The GoF phrasing adds that the same construction process can create different representations.

Everyday analogy. Ordering a custom meal at a fast-food counter (the course's example) or a sandwich at Subway: bread → cheese → vegetables → sauce, each step named, many optional, and only at the end do you get the finished sandwich. Nobody hands the cashier a row of ten unlabelled values.

Terms:

- **Product** — the object being built (`HttpRequest`).
- **Builder** — collects the values and has `build()`.
- **Fluent interface** — each method returns `this` so calls can be chained: `.method("POST").timeoutSeconds(5)`.
- **Director** — optional class that knows a *recipe* (the order of steps) and runs it on a builder.
- **Immutable** — cannot change after creation (all fields `final`, no setters).

Two different "Builders" share the name — know both:

	GoF Builder (1994)	**Bloch's Builder (*Effective Java*, item 2)**
Problem it targets	One *construction process* should be able to produce *different representations* (the GoF example: one RTF reader converting a document to plain ASCII, TeX, or an editable text widget)	A constructor with many optional parameters (telescoping constructors) and the wish for an immutable, validated object
Participants	**Builder** (abstract interface with one method per part, e.g. `buildParagraph()`), **ConcreteBuilder** (implements the parts for one representation, keeps the result, exposes `getResult()`), **Director** (runs the construction steps through the Builder interface), **Product** (the complex object being built — its type can differ per ConcreteBuilder)	One product class plus one `static` nested `Builder` class: fluent setters that return `this`, and `build()` that validates and calls the product's private constructor. No Director, no abstract Builder interface
What varies	*Which* ConcreteBuilder is plugged in — the steps stay the same, the output type/format differs	Only the *values* the caller supplies
Examples	The GoF RTF converter; a report exporter whose one traversal drives a PDF builder, an HTML builder or a CSV builder	`HttpRequest.newBuilder()`, `Locale.Builder`, Lombok `@Builder`, this chapter's `HttpRequest.Builder`

Most code you will see (and write) is Bloch's fluent builder; interviews often expect you to *name* the GoF participants too. The two ideas overlap in the core insight — separate the step-by-step assembly from the finished object.

The problem it solves

A class keeps collecting fields, and most of them are optional. Its constructor grows until a call site looks like this:

```
new HttpRequest(url, "GET", headers, null, 30, 3, true, false, null);
```

A reader cannot tell what this does. Which of the `null`s is the body? What does the first `true` switch on? If someone exchanges the two booleans, the compiler accepts it, and the mistake only shows up in production — for example as requests that follow redirects although they were meant not to.

The two usual "fixes" are both bad:

Approach	What goes wrong
Telescoping constructors — one overload per useful combination: `Pizza(size)`, `Pizza(size, crust)`, `Pizza(size, crust, cheese)`, …	The number of overloads explodes with every field; call sites are still positional and unreadable; two parameters of the same type are easy to swap.
JavaBeans style — empty constructor + setters (methods that each change one field)	The object exists in a **half-built** state; nothing stops use before it is ready; it **cannot be immutable**; no single place to validate rules that span several fields.

Builder fixes both: every value is **named** at the call site, optional fields have **defaults**, the product can be **immutable**, and `build()` is **one gate** that can refuse to create an invalid object. Our API adopts the rule "a GET must not carry a body" (HTTP — the request/response protocol of the web, see System Design Fundamentals — gives a GET, its "read this resource" request, no defined meaning for a body — RFC 9110 §9.3.1, the official HTTP standard — and many servers and proxies (middlemen that forward requests) reject or drop it). That rule involves two fields at once (`method` and `body`), so no single setter can enforce it; `build()` is the natural place for it.

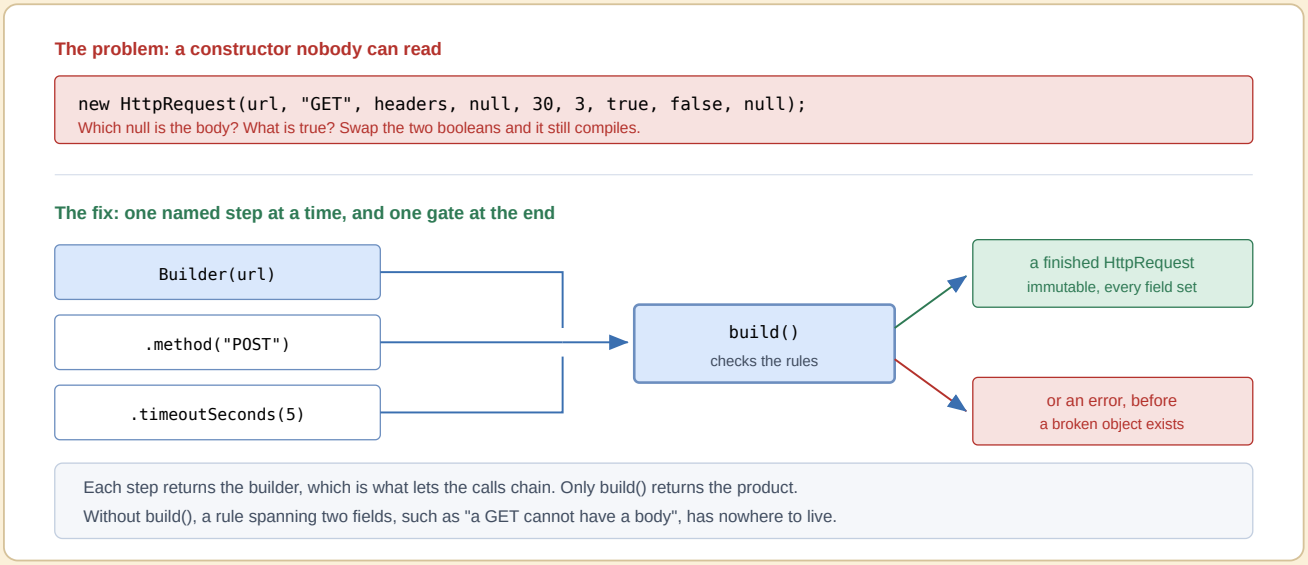


Figure — the unreadable positional constructor versus named builder steps that end in one build() gate which validates and returns an immutable HttpRequest or an error (course diagram)

In the figure, every step hands back the builder itself (which is what lets the calls chain), and only `build()` hands back a product — or throws, so an invalid request never comes into existence.

How it works

1. Make the product's fields `final` and its constructor `private`, taking the builder. Nobody can build a half-filled product directly.
2. Create a `static` `nested` `Builder` class with the same fields, mutable, each optional one with a **default**.

3. Take **required** values in the builder's constructor (or factory method `builder(url)`) so they cannot be forgotten.
4. Give every setter-like method a **name that says what it sets** and have it `return this`.
5. Write `build()`: validate cross-field rules, **copy mutable collections**, and call the private product constructor. Every other builder method returns the builder; `build()` alone returns the product.
6. Optionally add a **Director** for reusable recipes, a `toBuilder()` for copy-and-modify, or a **staged builder** so the compiler enforces required steps.

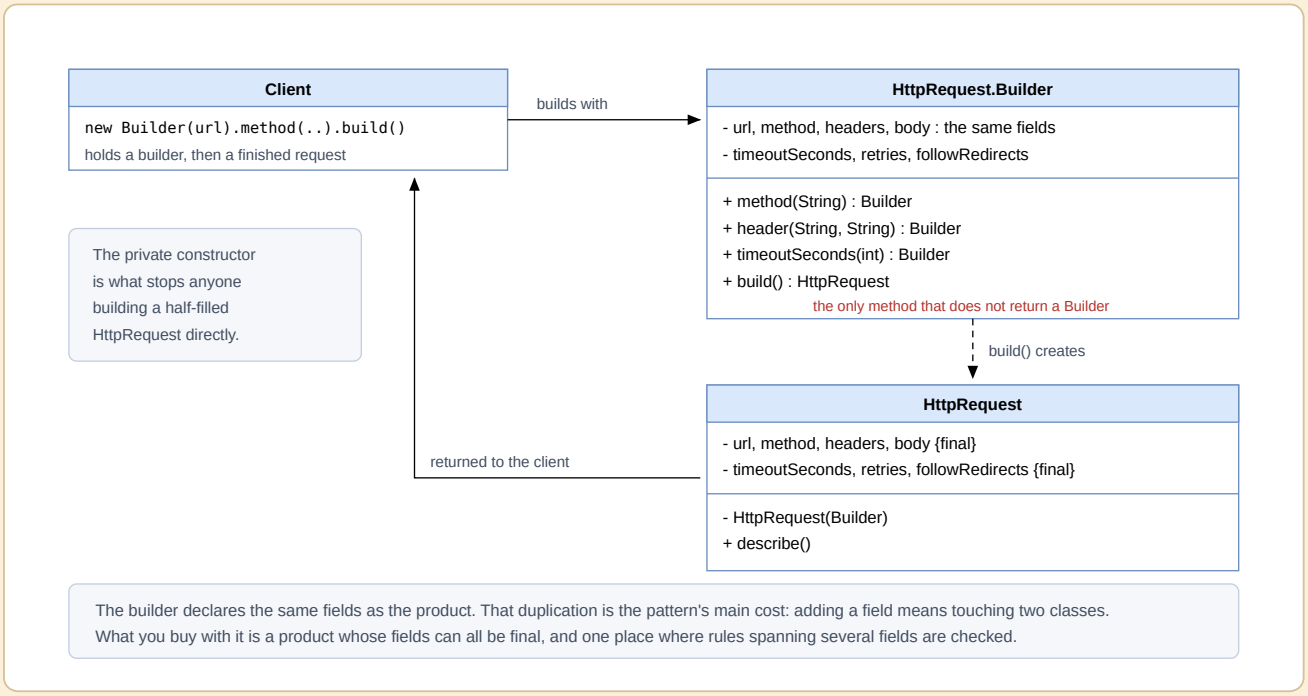
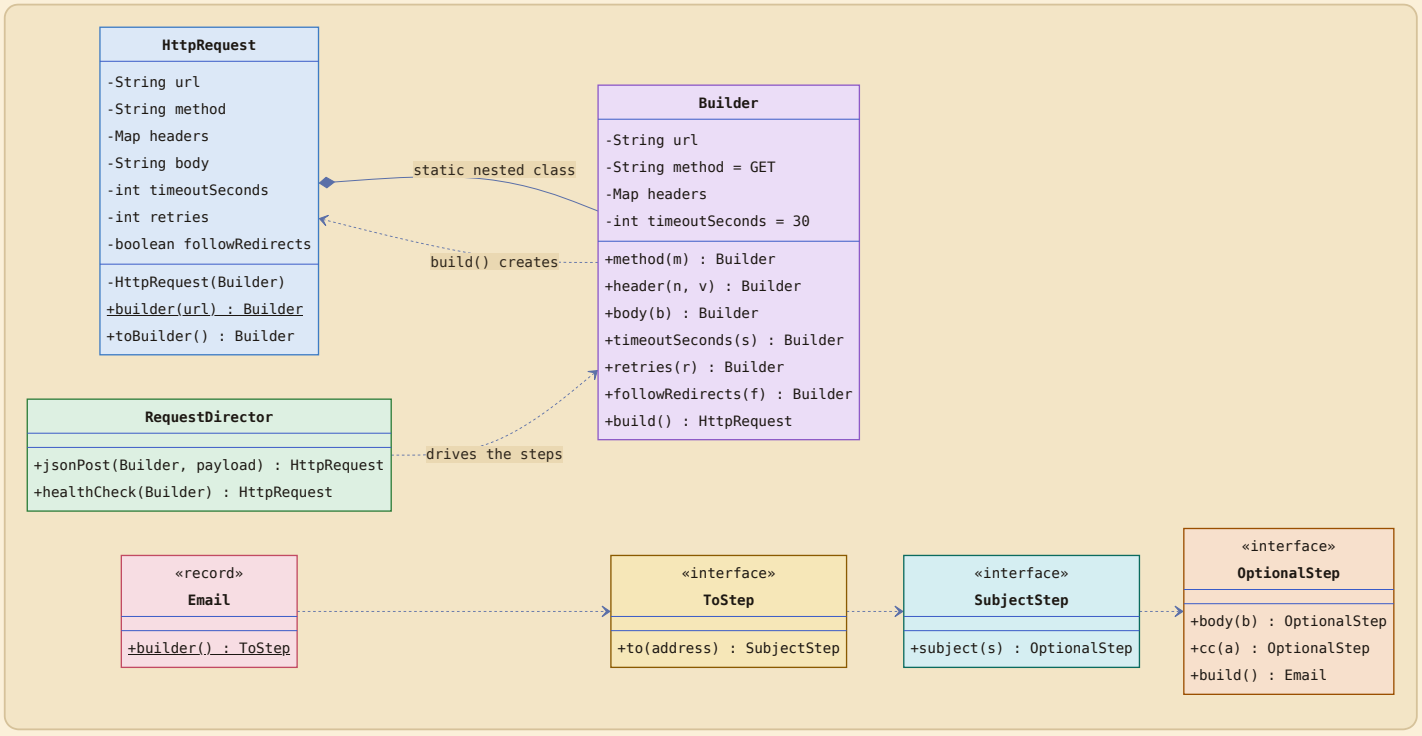


Figure — HttpRequest with final fields and a private constructor; HttpRequest.Builder with the same mutable fields, setters returning Builder, and build() returning HttpRequest (course diagram)

The diagram also exposes the price: the builder repeats every field of the product, so each new field must be added in two places. In exchange, the product has no setters, all of its fields are `final`, and every instance passes through one validation point.



The Director — and why you rarely see one

The GoF version has a fourth participant, the **Director**: a class that knows the **order of steps** and runs them on whichever builder it is given. It exists to make a *sequence* reusable — if many places build "a JSON POST with our usual headers and timeout", that recipe deserves a name, and the Director is its home. (In the GoF book's example, one director — an RTF reader — drives different builders that output plain text or TeX: same process, different representations.)

Day to day you will mostly see the **fluent** builder, because what changes between call sites is normally the *values*, not the order of steps. If only values change, a Director adds nothing a caller could not write as one chained expression. **A quick test:** will this Director ever be given a *different* kind of builder? If so, it separates the recipe from the thing being built and is worth keeping. If not, remove it and let callers chain the calls themselves. (Our `RequestDirector` is deliberately simplified: it only accepts `HttpRequest.Builder`, so it shows the "named recipe" half of the idea. A full GoF Director would be typed against an abstract Builder interface with several implementations.)

The staged (step) builder

When some steps are required or must come in order, encode it in the **types**: each step returns an interface exposing only the legal next calls. `Email.builder()` returns a `ToStep` (only `to(...)`), which returns a `SubjectStep` (only `subject(...)`), which returns an `OptionalStep` that finally has `build()`. Now `Email.builder().build()` **does not compile**. This is the answer to "how would you stop a caller forgetting a required field?" — heavier than most code needs, but good to know.

Lombok-style builders and records

- **Lombok** `@Builder` (Lombok is a library that writes Java boilerplate for you at compile time) generates all of this from an annotation: a static `builder()` method, a nested builder class with one method per field, and `build()`. `@Builder(toBuilder = true)` adds `toBuilder()` for copy-and-modify; `@Singular` generates `header(name, value)`-style methods that add to collections. Lombok's builder does **not** validate by default — you add validation in the constructor it calls.
- **Java records** give you the immutable product for free. A record plus a static nested builder (like `Email`) is now a common shape. Put validation in the record's compact constructor (a constructor with no parameter list that runs on every creation) so it runs however the record is created.
- **Immutability checklist:** `final` class or private constructor, `final` fields, no setters, defensive copies (private copies, so outside code cannot change your internals) of mutable inputs (`Map.copyOf`, `List.copyOf`), and getters that do not leak mutable internals.

Java implementation

```java
// File: code/lld/ch08/builder/TelescopingPizza.java
package lld.ch08.builder;

/** The PROBLEM: telescoping constructors. Each new optional field adds another overload. */
public final class TelescopingPizza {
    private final String size;          // required
    private final String crust;         // optional
    private final boolean cheese;       // optional
    private final boolean olives;       // optional
    private final boolean jalapenos;    // optional

    public TelescopingPizza(String size) { this(size, "regular"); }
    public TelescopingPizza(String size, String crust) { this(size, crust, true); }
    public TelescopingPizza(String size, String crust, boolean cheese) { this(size, crust, cheese, false); }
    public TelescopingPizza(String size, String crust, boolean cheese, boolean olives) {
        this(size, crust, cheese, olives, false);
    }
    public TelescopingPizza(String size, String crust, boolean cheese, boolean olives, boolean jalapenos) {
        this.size = size; this.crust = crust; this.cheese = cheese; this.olives = olives; this.jalapenos =
jalapenos;
    }

    @Override public String toString() {
        return "Pizza[size=" + size + ", crust=" + crust + ", cheese=" + cheese
               + ", olives=" + olives + ", jalapenos=" + jalapenos + "]";
    }
}
```

```java
// File: code/lld/ch08/builder/HttpRequest.java
package lld.ch08.builder;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Objects;

/** Product: immutable. Only the nested Builder can create one. */
public final class HttpRequest {
    private final String url;
    private final String method;
    private final Map<String, String> headers;
    private final String body;
    private final int timeoutSeconds;
    private final int retries;
    private final boolean followRedirects;

    private HttpRequest(Builder b) {
        this.url = b.url;
        this.method = b.method;
        this.headers = Map.copyOf(b.headers);      // defensive, unmodifiable copy
        this.body = b.body;
        this.timeoutSeconds = b.timeoutSeconds;
        this.retries = b.retries;
        this.followRedirects = b.followRedirects;
    }

    public static Builder builder(String url) { return new Builder(url); }

    /** Lombok-style toBuilder(): start a new builder pre-filled from this object. */
    public Builder toBuilder() {
        Builder b = new Builder(url).method(method).timeoutSeconds(timeoutSeconds)
                .retries(retries).followRedirects(followRedirects);
        headers.forEach(b::header);
        if (body != null) b.body(body);
        return b;
    }

    public String url() { return url; }
    public String method() { return method; }
    public Map<String, String> headers() { return headers; }

    @Override public String toString() {
        return method + " " + url + " headers=" + new java.util.TreeMap<>(headers)
                + (body == null ? "" : " body=" + body)
                + " timeout=" + timeoutSeconds + "s retries=" + retries + " followRedirects=" + followRedirects;
    }

    /** The builder: same fields, mutable, with defaults. Every setter returns this. */
    public static final class Builder {
        private final String url;                                  // required -> constructor
        private String method = "GET";                             // optional -> defaults
        private final Map<String, String> headers = new LinkedHashMap<>();
        private String body;
        private int timeoutSeconds = 30;
        private int retries = 3;
        private boolean followRedirects = true;

        private Builder(String url) { this.url = Objects.requireNonNull(url, "url"); }

        public Builder method(String m)             { this.method = m; return this; }
        public Builder header(String n, String v)   { headers.put(n, v); return this; }
        public Builder body(String b)               { this.body = b; return this; }
        public Builder timeoutSeconds(int s)        { this.timeoutSeconds = s; return this; }
        public Builder retries(int r)               { this.retries = r; return this; }
        public Builder followRedirects(boolean f)   { this.followRedirects = f; return this; }

        /** The single gate: cross-field validation happens here, before any product exists. */
```

```
        public HttpRequest build() {
            if (url.isBlank()) throw new IllegalStateException("url is required");
            if (method.equals("GET") && body != null) throw new IllegalStateException("a GET request cannot have a
body");
            if (retries < 0) throw new IllegalStateException("retries cannot be negative");
            if (timeoutSeconds <= 0) throw new IllegalStateException("timeout must be positive");
            return new HttpRequest(this);
        }
    }
}
```

```
// File: code/lld/ch08/builder/RequestDirector.java
package lld.ch08.builder;

/** Director: a named, reusable RECIPE (order of steps). It does not know the values the caller varies. */
public final class RequestDirector {

    public HttpRequest jsonPost(HttpRequest.Builder builder, String payload) {
        return builder.method("POST")
                .header("Content-Type", "application/json")
                .header("Accept", "application/json")
                .body(payload)
                .timeoutSeconds(5)
                .build();
    }

    public HttpRequest healthCheck(HttpRequest.Builder builder) {
        return builder.method("HEAD").timeoutSeconds(1).retries(0).followRedirects(false).build();
    }
}
```

```
// File: code/lld/ch08/builder/Email.java
package lld.ch08.builder;

import java.util.ArrayList;
import java.util.List;

/**
 * Staged (step) builder: each step returns an interface exposing only the legal next calls,
 * so the compiler forces "to" and "subject" before build() is even visible.
 */
public record Email(String to, String subject, String body, List<String> cc) {

    public Email { cc = List.copyOf(cc); }                    // records + builder = immutable product

    public interface ToStep      { SubjectStep to(String address); }
    public interface SubjectStep { OptionalStep subject(String subject); }
    public interface OptionalStep {
        OptionalStep body(String body);
        OptionalStep cc(String address);
        Email build();
    }

    public static ToStep builder() { return new Steps(); }

    private static final class Steps implements ToStep, SubjectStep, OptionalStep {
        private String to, subject, body = "";
        private final List<String> cc = new ArrayList<>();
        public SubjectStep to(String a)       { this.to = a; return this; }
        public OptionalStep subject(String s) { this.subject = s; return this; }
        public OptionalStep body(String b)    { this.body = b; return this; }
        public OptionalStep cc(String a)      { cc.add(a); return this; }
        public Email build()                  { return new Email(to, subject, body, cc); }
    }
}
```

```java
// File: code/lld/ch08/builder/Main.java
package lld.ch08.builder;

public class Main {
    public static void main(String[] args) {
        System.out.println("== The problem: telescoping constructors ==");
        // Which boolean is which? Swap two of them and it still compiles.
        System.out.println("  " + new TelescopingPizza("large", "thin", true, false, true));

        System.out.println("\n== Fluent builder: every value is named ==");
        HttpRequest get = HttpRequest.builder("https://api.example.com/orders")
                .header("Accept", "application/json")
                .build();
        System.out.println("  " + get);

        HttpRequest post = HttpRequest.builder("https://api.example.com/orders")
                .method("POST")
                .header("Content-Type", "application/json")
                .body("{\"item\":\"widget\"}")
                .timeoutSeconds(5)
                .retries(0)
                .followRedirects(false)
                .build();
        System.out.println("  " + post);

        System.out.println("\n== build() refuses broken objects ==");
        try {
            HttpRequest.builder("https://api.example.com/orders").body("{}").build();
        } catch (IllegalStateException e) {
            System.out.println("  rejected: " + e.getMessage());
        }

        System.out.println("\n== Reusing a builder does not leak state into earlier products ==");
        HttpRequest.Builder reused = HttpRequest.builder("https://api.example.com/users").header("X-Trace", "1");
        HttpRequest first = reused.build();
        HttpRequest second = reused.header("X-Debug", "true").build();
        System.out.println("  first headers  = " + first.headers());
        System.out.println("  second headers = " + new java.util.TreeMap<>(second.headers()));

        System.out.println("\n== Product is immutable ==");
        try {
            post.headers().put("Hack", "yes");
        } catch (UnsupportedOperationException e) {
            System.out.println("  headers().put -> UnsupportedOperationException");
        }

        System.out.println("\n== toBuilder(): copy-and-change, like Lombok @Builder(toBuilder = true) ==");
        HttpRequest slowPost = post.toBuilder().timeoutSeconds(60).build();
        System.out.println("  " + slowPost);

        System.out.println("\n== Director: named recipes over any builder ==");
        RequestDirector director = new RequestDirector();
        System.out.println("  " + director.jsonPost(HttpRequest.builder("https://api.example.com/payments"), "
{\"amount\":499}"));
        System.out.println("  " + director.healthCheck(HttpRequest.builder("https://api.example.com/health")));

        System.out.println("\n== Staged builder: the compiler enforces required steps ==");
        Email email = Email.builder()
                .to("ada@example.com")          // must come first
                .subject("Welcome")             // then this
                .body("Glad to have you")       // optional, any order
                .cc("team@example.com")
                .build();
        System.out.println("  " + email);
        // Email.builder().build();   // does not compile: ToStep has no build()
    }
}
```

- `TelescopingPizza` is the anti-example: five constructors for five fields, and the call `("large", "thin", true, false, true)` hides which boolean means what.
- In `HttpRequest`, the constructor does `Map.copyOf(b.headers)`. Without that copy, building twice from one builder would hand both products **the same map**, and a header added later would appear in the earlier product. The "reuse" demo proves `first` did not pick up `X-Debug`. `Map.copyOf` is also unmodifiable, which is why `headers().put(...)` throws.
- `build()` enforces four rules, including the cross-field one (GET + body).
- `toBuilder()` gives Lombok-style "same as this, but…" copies.

Output:

```
== The problem: telescoping constructors ==
  Pizza[size=large, crust=thin, cheese=true, olives=false, jalapenos=true]

== Fluent builder: every value is named ==
  GET https://api.example.com/orders headers={Accept=application/json} timeout=30s retries=3 followRedirects=true
  POST https://api.example.com/orders headers={Content-Type=application/json} body={"item":"widget"} timeout=5s
retries=0 followRedirects=false

== build() refuses broken objects ==
  rejected: a GET request cannot have a body

== Reusing a builder does not leak state into earlier products ==
  first headers  = {X-Trace=1}
  second headers = {X-Debug=true, X-Trace=1}

== Product is immutable ==
  headers().put -> UnsupportedOperationException

== toBuilder(): copy-and-change, like Lombok @Builder(toBuilder = true) ==
  POST https://api.example.com/orders headers={Content-Type=application/json} body={"item":"widget"} timeout=60s
retries=0 followRedirects=false

== Director: named recipes over any builder ==
  POST https://api.example.com/payments headers={Accept=application/json, Content-Type=application/json} body=
{"amount":499} timeout=5s retries=3 followRedirects=true
  HEAD https://api.example.com/health headers={} timeout=1s retries=0 followRedirects=false

== Staged builder: the compiler enforces required steps ==
  Email[to=ada@example.com, subject=Welcome, body=Glad to have you, cc=[team@example.com]]
```

Trade-offs

Pros	Cons
Self-describing call sites — each value is labelled where it is passed, so parameter order no longer matters.	**Duplicated fields** — the builder repeats every field of the product; a new field means two edits.
Immutable products — all fields can be `final` because they are fixed before the object exists; safe to share between threads.	**Forgettable last step** — omit `build()` and you are holding a builder, not the object you wanted.
A home for validation — rules that involve several fields are checked in `build()`.	**Easily overused** — for a value object (a small object defined only by its values, like `Money`) with three fields it only adds noise.
Free optional fields — defaults sit in the builder, callers mention only what they change.	**Reuse pitfall** — building twice from one builder can leave two products sharing one mutable collection unless it is copied.
Different representations (GoF) — a Director can drive different ConcreteBuilders through the same steps.	One extra short-lived object per construction — almost never measurable.

Often confused with:

Looks like	The actual difference
Factory Method	Chooses *which type* to create, in a single call. Builder assembles *one known type* over a series of calls.
Abstract Factory	Hands back several finished products straight away. Builder hands back one product after a sequence of steps.
Fluent interface	Chaining is merely a calling style. A chained API that modifies and returns the *same* object (a jQuery chain, a `StringBuilder`-style mutator used as the result itself) has no separate finished product, so on its own it is not the Builder pattern.
Prototype	Prototype begins with a ready-made object and duplicates it; Builder begins with nothing and fills values in. Both are ways around a huge constructor — one starts from the end result, the other from scratch.

In other languages: Python and Kotlin (and C#) have **named arguments with defaults** — `HttpRequest(url, method="POST", timeout=5)` — which already make call sites readable; Python's `@dataclass(frozen=True)` and Kotlin's `data class` (with `.copy(...)`) supply the immutable product. Use a builder there only for cross-field validation or staged construction. JavaScript/TypeScript usually pass an options object: `new HttpRequest({ url, method: "POST" })`.

When not to use it

- **A few fields, all required.** A plain constructor is shorter and just as clear — this is the misuse you will see most.
- **The language has named/default arguments** — see above.
- **The builder merely copies fields**, validates nothing, and the product is mutable anyway — that is an extra class with no payoff. Either add the validation or remove the builder.
- **The object is mutable and lives only inside one method** — ordinary setters are fine there.

Common mistakes

- Setters that **don't return the builder** (breaks chaining). Every step returns `Builder`; only `build()` returns the product.
- **No validation** in `build()`.
- **Sharing mutable collections** between the builder and the products.
- Making the product constructor public (then nobody has to use the builder or its validation).
- Using a Director when the order never varies.

Real-world examples

- **JDK:** `StringBuilder` (append repeatedly, `toString()` once); `java.net.http.HttpClient.newBuilder()` and `HttpRequest.newBuilder()` (Java 11+); `Locale.Builder`; `Calendar.Builder` (Java 8+); `Stream.builder()`; `Thread.ofVirtual().name("w").start(task)` — the `Thread.Builder` API in Java 21; `ProcessBuilder`.
- **Spring:** `UriComponentsBuilder`, `ResponseEntity.ok().header(...).body(...)`, `WebClient.builder()`, `MockMvcRequestBuilders.get("/x").param(...)`, Spring Security's `HttpSecurity` DSL (domain-specific language — a mini-language built for one job).
- **Libraries:** OkHttp `Request.Builder`; Protocol Buffers generated messages (immutable messages + generated builders); Guava `ImmutableList.builder()`; Lombok `@Builder`; test-data builders like `aUser().withEmail("x@y.com").build()`.

In the interview

30-second answer: "When a constructor has many optional parameters, I use a Builder: required values in the builder's constructor, one named method per optional field that returns the builder, and `build()` that validates cross-field rules and creates an immutable product through a private constructor. I'd mention the classic Director but usually skip it because only values vary. For tiny objects or languages with named arguments, I wouldn't bother."

- *Q: Why not setters?* — A: Half-built objects, no immutability, nowhere for cross-field validation.
- *Q: How do you enforce required fields?* — A: Take them in the builder's constructor, validate in `build()`, or use a staged builder so the compiler enforces them.

- *Q: Is `StringBuilder` a GoF Builder?* — A: Not in the full GoF structure (no Director, one representation), but it is the everyday example of the idea: accumulate parts step by step, produce the final `String` once.
- *Q: Is the built object thread-safe?* — A: The product is, if immutable. The builder itself is not — do not share a builder across threads.

Self-check

- **Q:** What is the only builder method that does not return the builder? **A:** `build()`.
- **Q:** What bug does copying the headers map in `build()`/the constructor prevent? **A:** Two products built from one builder sharing (and mutating) the same map.
- **Q:** When is a Director worth it? **A:** When the same recipe (order of steps) is reused, possibly with different builders.

TL;DR

Named steps + defaults + one validating `build()` → immutable product. Solves telescoping constructors and half-built JavaBeans. Director optional; staged builder for compile-time required steps; Lombok/records remove the boilerplate.

8.13 Prototype

What it is

Prototype creates new objects by **copying an existing, already-configured object** (the prototype) instead of building from scratch. The object itself offers a `copy()` method, so the caller does not need to know its concrete class.

Everyday analogy. A photocopier, or cell division: you do not rebuild a document or a cell from raw materials — you duplicate one that already exists and then change what differs. Another: a company's contract template. Legal drafted it once; each new contract is a copy with the name and date changed.

Terms:

- **Prototype** — the original object that gets copied.
- **Clone / copy** — the new object produced.
- **Shallow copy** — copies the fields; if a field is a reference, the copy points to the *same* object.
- **Deep copy** — copies the fields *and* the objects they point to, so nothing mutable is shared.
- **Copy constructor** — a constructor that takes another instance of the same class and copies it.
- **Prototype registry** — a map of named, pre-configured prototypes you ask for copies of.

GoF intent and participants. Intent: use a ready-configured instance as the model for the kind of object you want, and make new objects by copying that model. Participants: **Prototype** (declares the operation that copies itself — our `Shape` with `copy()`), **ConcretePrototype** (implements that copy operation — `Circle`, `Rectangle`, `Group`, or `NotificationTemplate`, which plays both roles) and **Client** (asks a prototype to copy itself instead of calling a constructor — `Main`, or `ShapeRegistry`, which GoF calls a *prototype manager*).

The problem it solves

There are two versions of the problem:

1. **Construction is expensive.** Initialization might load a file, hit a database, parse a template or fill in dozens of default values. Suppose a `NotificationTemplate` is parsed from disk at start-up; every welcome email differs from it only in the subject line and one added channel. Re-parsing the file for each recipient repeats the slowest work for nothing.
2. **You do not know the concrete class** — the more important case in practice. The code holds a reference typed as `Shape` or `Template` and needs *one more of the same*. Writing `new` is impossible, because you do not know which class name to write. Asking the object to copy itself solves it: the object always knows its own class. No factory or `switch` is needed to work the type out, and the copy starts from values that are already correct, so it skips the expensive setup.

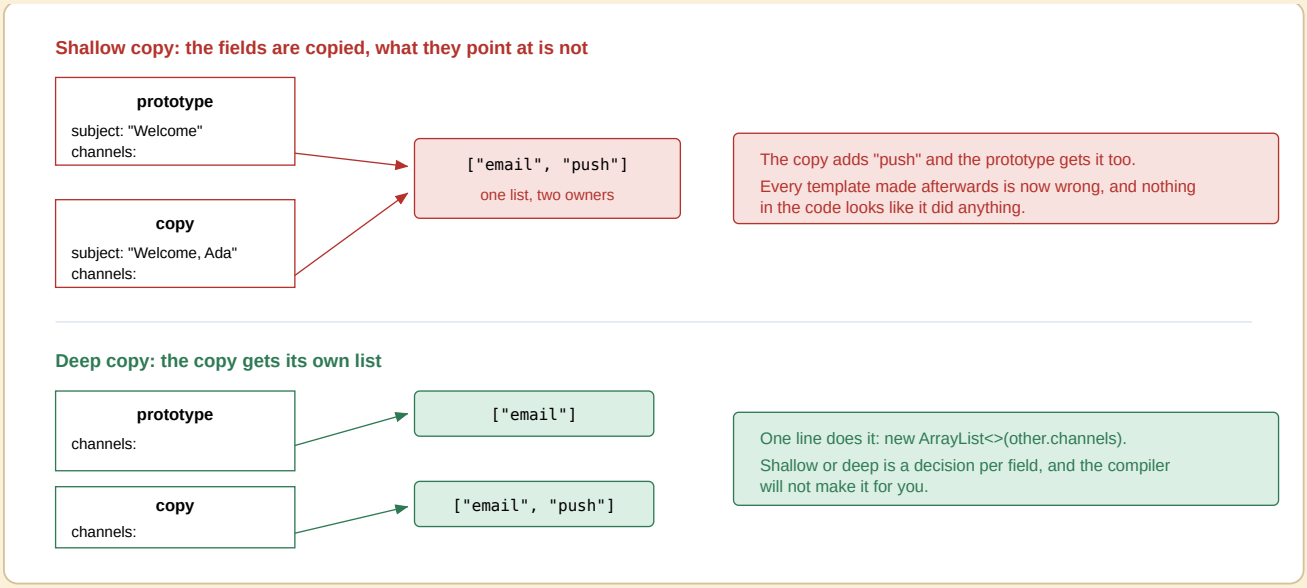


Figure — a shallow copy shares the channels list with the prototype, so adding "push" to the copy corrupts the prototype; a deep copy gets its own list (course diagram)

The figure highlights the one decision in this pattern that really needs thought: **how far down the object graph (every object reachable by following references from this one) the copy reaches.** The structure itself is simple: two entrances to the class — an ordinary constructor that does the expensive work, and a copy path that does none of it.

How it works

1. Give the class two ways in: the **normal constructor** (expensive setup) and a **copy path** that does none of it.
2. Implement the copy path, preferably as a **private copy constructor** `NotificationTemplate(NotificationTemplate other)`.
3. For each field, decide: immutable (share it — `String`, records of immutables) or mutable (copy it — `new ArrayList<>(other.channels)`), and recurse for nested mutable objects.
4. Expose a public `copy()` returning a new, independent instance.
5. If clients hold several unrelated types, add a **prototype interface** (`Shape { Shape copy(); }`) so they can copy polymorphically.
6. Optionally keep a **registry** of named prototypes that always hands out copies, never the original.

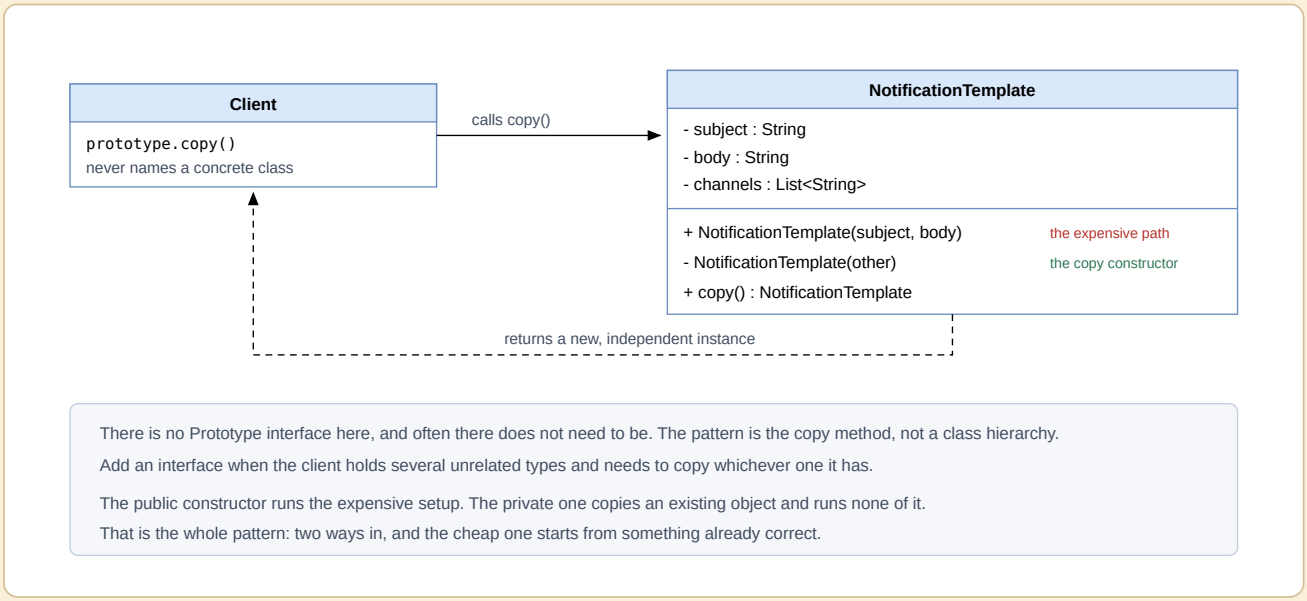
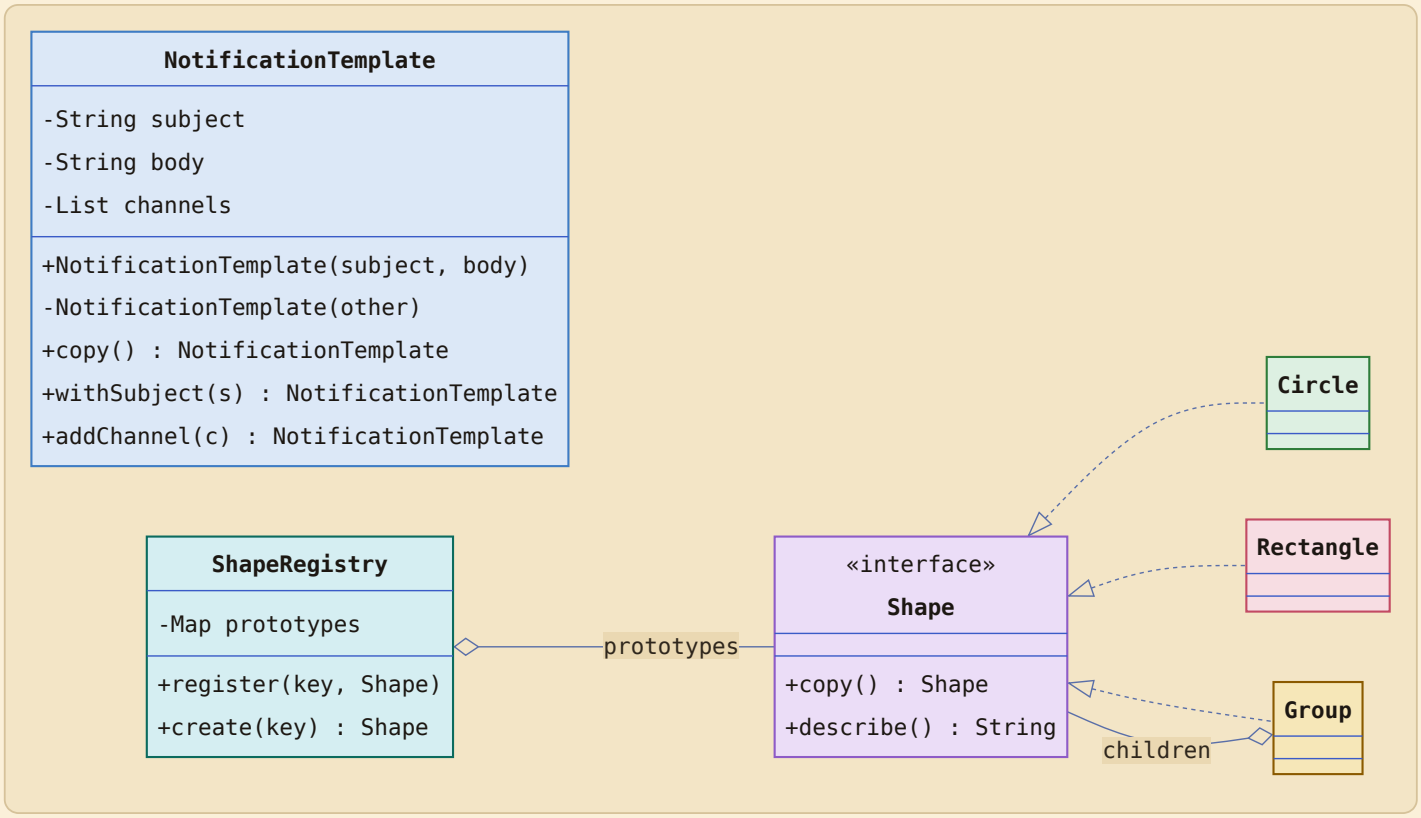


Figure — NotificationTemplate is both prototype and product: a public expensive constructor, a private copy constructor, and copy() that the Client calls (course diagram)

Notice in the class diagram that there is **no separate creator class and no Prototype interface** — unlike the other creational patterns, the product creates its own copies. What makes it Prototype is the copy method, not a

class hierarchy. Introduce a Prototype interface (like `Shape` below) only when the client holds several unrelated types and must copy whichever it happens to have.



Shallow vs deep copy

	Shallow copy	Deep copy
Primitives (`int`, `double`, `boolean`…), `String`, other immutables	Copied / safely shared	Copied / safely shared
Mutable fields (lists, maps, objects)	**Shared** — both objects point to the same one	**Duplicated** — each has its own
Cost	Cheap	Proportional to the size of the object graph
Risk	Changing the copy silently changes the original	Cycles, copying things you meant to share (caches), copying resources
Java default	`Object.clone()`	You write it (per field)

Suppose the copy constructor had written `this.channels = other.channels`. Both templates would then point at a single list, so adding "push" to the copy silently adds it to the prototype too — and **every later copy inherits the stray channel**, although no line of code visibly touched the prototype. That shared-mutable-collection bug is the one you will meet most often with this pattern. The cure is one line per mutable field — `new ArrayList<>(other.channels)` — i.e. a copy one level deeper. Had the list contained mutable objects instead of `String`s, those would need copying as well, and so on down the graph. **The depth of the copy is a design choice you make field by field**; neither Java nor the compiler will make it for you.

Three related traps:

- **A new field added without updating the copy** — the copy quietly drops it; no compiler error, and usually no test catches it, so the drift can go unnoticed for months.
- **Copying something that owns a resource** (connection, file handle) — two objects now think they own it and both try to close it.
- **Cycles** — a naive recursive deep copy of objects that point to each other never terminates. Real deep-copy utilities track visited objects.

Why Java's `Cloneable` is a trap

Java's built-in version of Prototype is `Object.clone()` + the `Cloneable` interface, and it is widely considered broken (*Effective Java*, item 13):

- `Cloneable` is a **marker interface with no methods** — it does not declare `clone()`. It only changes the behaviour of `Object.clone()` (without it, `clone()` throws `CloneNotSupportedException`).
- `Object.clone()` is **`protected`**, so you must override it as `public` to make it usable.
- It makes a **shallow** field-by-field copy.
- It **creates the object without running any constructor**, so constructor validation and invariants (rules that must always hold for the object) are skipped.
- To deep-copy you must reassign fields after `super.clone()` — which is **impossible for `final` fields**.
- It throws a **checked** exception (one the compiler forces you to catch or declare) you have to catch even when it cannot happen.

Prefer a copy constructor or a static copy factory (`static Foo copyOf(Foo other)`). It is ordinary code, runs your validation, works with `final` fields, and its behaviour is visible where you wrote it. The exception is arrays: `array.clone()` works well for primitive arrays.

Java implementation

```java
// File: code/lld/ch08/prototype/NotificationTemplate.java
package lld.ch08.prototype;

import java.util.ArrayList;
import java.util.List;

/** Prototype via a copy constructor: one expensive path, one cheap copy path. */
public final class NotificationTemplate {
    static int parses = 0;

    private String subject;
    private final String body;
    private final List<String> channels;

    /** The expensive path (imagine parsing a template file). */
    public NotificationTemplate(String subject, String body) {
        parses++;
        System.out.println("  Parsing template from disk... (expensive)");
        this.subject = subject;
        this.body = body;
        this.channels = new ArrayList<>(List.of("email"));
    }

    /** The copy path: no parsing. Deep-copies the mutable list. */
    private NotificationTemplate(NotificationTemplate other) {
        this.subject = other.subject;              // String is immutable -> sharing is safe
        this.body = other.body;
        this.channels = new ArrayList<>(other.channels);  // NEW list -> the copy owns its own
    }

    public NotificationTemplate copy() { return new NotificationTemplate(this); }

    public NotificationTemplate withSubject(String s) { this.subject = s; return this; }
    public NotificationTemplate addChannel(String c) { channels.add(c); return this; }

    @Override public String toString() { return "subject=\"" + subject + "\" channels=" + channels; }
}
```

```java
// File: code/lld/ch08/prototype/Shapes.java
package lld.ch08.prototype;

import java.util.ArrayList;
import java.util.List;

/** Prototype interface: lets code copy an object whose concrete class it does not know. */
interface Shape {
    Shape copy();
    String describe();
}

final class Circle implements Shape {
    private final int radius;
    private final String color;
    Circle(int radius, String color) { this.radius = radius; this.color = color; }
    private Circle(Circle other) { this(other.radius, other.color); }            // copy constructor
    @Override public Circle copy() { return new Circle(this); }                  // covariant return
    @Override public String describe() { return color + " circle r=" + radius; }
}

final class Rectangle implements Shape {
    private final int w, h;
    private final String color;
    Rectangle(int w, int h, String color) { this.w = w; this.h = h; this.color = color; }
    @Override public Rectangle copy() { return new Rectangle(w, h, color); }
    @Override public String describe() { return color + " rectangle " + w + "x" + h; }
}

/** A composite prototype: deep copy must copy the children too. */
final class Group implements Shape {
    private final List<Shape> children = new ArrayList<>();
    Group add(Shape s) { children.add(s); return this; }
    @Override public Group copy() {
        Group g = new Group();
        for (Shape s : children) g.add(s.copy());       // recursive deep copy
        return g;
    }
    @Override public String describe() { return "group" + children.stream().map(Shape::describe).toList(); }
    int size() { return children.size(); }
}
```

```java
// File: code/lld/ch08/prototype/ShapeRegistry.java
package lld.ch08.prototype;

import java.util.HashMap;
import java.util.Map;

/** Prototype registry (a.k.a. prototype manager): named, pre-configured objects handed out as copies. */
final class ShapeRegistry {
    private final Map<String, Shape> prototypes = new HashMap<>();

    void register(String key, Shape prototype) { prototypes.put(key, prototype); }

    Shape create(String key) {
        Shape p = prototypes.get(key);
        if (p == null) throw new IllegalArgumentException("no prototype named " + key);
        return p.copy();                                  // never hand out the original
    }
}
```

```java
// File: code/lld/ch08/prototype/CloneablePitfall.java
package lld.ch08.prototype;

import java.util.ArrayList;
import java.util.List;

/** Why Cloneable is risky: Object.clone() is shallow, so mutable fields are shared. */
class ShallowPlaylist implements Cloneable {
    String name;
    List<String> songs = new ArrayList<>();

    @Override
    public ShallowPlaylist clone() {
        try {
            return (ShallowPlaylist) super.clone();       // field-by-field copy: 'songs' reference is shared
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);                  // cannot happen: we implement Cloneable
        }
    }
}

/** The fixed clone(): deep-copy each mutable field by hand. (It cannot do this for a final field.) */
class DeepPlaylist implements Cloneable {
    String name;
    List<String> songs = new ArrayList<>();

    @Override
    public DeepPlaylist clone() {
        try {
            DeepPlaylist copy = (DeepPlaylist) super.clone();
            copy.songs = new ArrayList<>(songs);          // re-assigning is why 'songs' cannot be final
            return copy;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }
}
```

```java
// File: code/lld/ch08/prototype/Main.java
package lld.ch08.prototype;

public class Main {
    public static void main(String[] args) {
        System.out.println("== Copy instead of re-parsing ==");
        NotificationTemplate prototype = new NotificationTemplate("Welcome", "Hello there");
        NotificationTemplate ada = prototype.copy().withSubject("Welcome, Ada").addChannel("push");
        NotificationTemplate grace = prototype.copy().withSubject("Welcome, Grace").addChannel("sms");
        System.out.println("  ada       : " + ada);
        System.out.println("  grace     : " + grace);
        System.out.println("  prototype : " + prototype + "   <- unchanged (deep copy of the list)");
        System.out.println("  templates parsed: " + NotificationTemplate.parses);

        System.out.println("\n== Cloneable pitfall: shallow copy shares the list ==");
        ShallowPlaylist p1 = new ShallowPlaylist();
        p1.name = "Road trip"; p1.songs.add("Song A");
        ShallowPlaylist p2 = p1.clone();
        p2.songs.add("Song B");
        System.out.println("  original songs after editing the CLONE: " + p1.songs + "   <- bug");

        DeepPlaylist d1 = new DeepPlaylist();
        d1.name = "Road trip"; d1.songs.add("Song A");
        DeepPlaylist d2 = d1.clone();
        d2.songs.add("Song B");
        System.out.println("  deep clone -> original " + d1.songs + ", copy " + d2.songs);

        System.out.println("\n== Polymorphic copy: the caller only knows 'Shape' ==");
        Shape unknown = args.length > 0 ? new Rectangle(2, 3, "green") : new Circle(5, "red");
        Shape twin = unknown.copy();
        System.out.println("  copied a " + twin.getClass().getSimpleName() + ": " + twin.describe()
                + ", same object? " + (twin == unknown));

        System.out.println("\n== Prototype registry ==");
        ShapeRegistry registry = new ShapeRegistry();
        registry.register("button-bg", new Rectangle(120, 40, "blue"));
        registry.register("avatar", new Circle(24, "grey"));
        registry.register("badge", new Group().add(new Circle(8, "gold")).add(new Rectangle(30, 10, "gold")));
        Shape b1 = registry.create("badge");
        Shape b2 = registry.create("badge");
        System.out.println("  " + registry.create("button-bg").describe());
        System.out.println("  " + registry.create("avatar").describe());
        System.out.println("  " + b1.describe() + ", two badges distinct? " + (b1 != b2));
    }
}
```

- "Parsing template from disk" prints **once** although three templates exist — the copies skipped the expensive setup.
- The prototype still has only `[email]` — each copy owns its list.
- `ShallowPlaylist` shows the `Cloneable` bug: editing the clone changed the original. `DeepPlaylist` fixes it, but only because `songs` is not `final`.
- `Circle.copy()` returns `Circle` (a covariant return type) — handy when you know the type, while `Shape` callers still get a `Shape`.
- `Group.copy()` recursively copies its children — a deep copy of a composite.
- `ShapeRegistry.create()` always returns `p.copy()`, never the stored prototype, so callers cannot corrupt it.

Output:

```
== Copy instead of re-parsing ==
  Parsing template from disk... (expensive)
  ada       : subject="Welcome, Ada" channels=[email, push]
  grace     : subject="Welcome, Grace" channels=[email, sms]
  prototype : subject="Welcome" channels=[email]   <- unchanged (deep copy of the list)
  templates parsed: 1

== Cloneable pitfall: shallow copy shares the list ==
  original songs after editing the CLONE: [Song A, Song B]   <- bug
  deep clone -> original [Song A], copy [Song A, Song B]

== Polymorphic copy: the caller only knows 'Shape' ==
  copied a Circle: red circle r=5, same object? false

== Prototype registry ==
  blue rectangle 120x40
  grey circle r=24
  group[gold circle r=8, gold rectangle 30x10], two badges distinct? true
```

Trade-offs

Pros	Cons
Costly setup is paid once — the prototype does the parsing/loading; each copy is cheap.	**Every class carries copy code** — and it falls out of date as soon as someone adds a field and forgets the copy.
Create without naming a class — the only option when all you hold is an interface.	**Shallow vs deep must be decided per field** — a wrong call makes two objects secretly share state.
New kinds at run time — configured prototypes can be registered and copied without compiling a class for each (fewer subclasses).	**Cyclic graphs** need special care, or a naive deep copy recurses forever.
Saved starting points — users can keep a template, a filter or a layout and begin new work from it.	**Hazardous for objects with identity** — primary keys (unique row IDs in a database), handles and connections must not be duplicated.
	Uneven language support — some languages offer no built-in cloning, and Java's built-in one (`Cloneable`) is flawed, so you write the copy logic yourself.

Often confused with:

Looks like	The actual difference
Factory Method	Creates a fresh object from nothing, with the type chosen by a subclass. Prototype duplicates an existing instance, so the type comes along automatically.
Builder	Builder puts an object together from parts the caller supplies; Prototype takes a finished object and adjusts only what differs.
Copy constructor	The usual *implementation*. It becomes the pattern when callers use a polymorphic `copy()` without knowing the concrete type.
Object Pool	A pool lends out **existing** objects and takes them back, so identity is reused. Prototype produces **new** objects, so identity is never shared.
Spring `prototype` scope	Despite the name, Spring creates a **new instance** via the constructor on every `getBean()` — it does not clone anything.
JavaScript prototypes	JS prototypal inheritance (the prototype chain) is a language feature for sharing behaviour, not this pattern.

In other languages: Python has `copy.copy()` / `copy.deepcopy()` built in, and `dataclasses.replace(obj, subject="x")` returns a modified copy. JavaScript's spread `{...original, subject: "x"}` is a *shallow* copy (same trap as above), while `structuredClone()` deep-copies plain data. Kotlin/Scala `data class` gives `.copy(subject = "x")`. In C++ the copy constructor is part of the language, and the "rule of zero" means the compiler-generated one copies every member correctly as long as the class owns no raw resources. In Java, a `record` is a natural copy point (all

components are available to a copy constructor or a `withX(...)` method), and a hand-written copy constructor is still preferred to `Cloneable`.

When not to use it

- **Creating from scratch is cheap** — a constructor is clearer and has no copy semantics to get wrong.
- **The object has identity** — duplicating something that holds a database primary key, a socket or an open file yields an object that *looks* fine but is not. Two rows sharing one ID is a far worse bug than a slow constructor.
- **The object graph is large** — a deep copy may cost more than rebuilding and may duplicate things you meant to share, such as a cache.
- **The object is immutable** — no one can change it, so just share the same instance.

Common mistakes

- Shallow-copying a mutable collection (the classic bug).
- Implementing `Cloneable` and trusting `super.clone()`.
- Forgetting to update the copy constructor when adding a field.
- Handing out the registry's original prototype instead of a copy.

Real-world examples

- **JDK:** `Object.clone()` and arrays' `clone()`; `ArrayList.clone()` and `HashMap.clone()` (documented as **shallow** — elements are not copied); copy constructors everywhere: `new ArrayList<>(collection)`, `new HashMap<>(map)`, `new TreeMap<>(sortedMap)`, `new EnumMap<>(enumMap)`; `List.copyOf(...)` for immutable snapshots.
- **Game engines:** Unity's `Instantiate(prefab)` clones a configured "prefab" object to spawn enemies, bullets and so on — load the model, stats and animations once, then copy it for each of the hundreds of enemies on a level.
- **Products:** "Duplicate" in Google Docs/Slides/Figma, "Save as template" / "Use template" in Notion or Jira, saved search filters and report layouts.
- **Tests:** a valid base fixture copied and nudged per test.

In the interview

> **30-second answer:** "Prototype creates objects by copying a configured instance through a `copy()` method on the object. It saves expensive setup and — the deeper reason — it lets me create an object when I only hold an interface. The main decision is shallow vs deep copy per field; sharing a mutable list is the classic bug. In Java I'd write a copy constructor rather than use `Cloneable`, and I'd refuse to copy anything with identity like a DB id or a connection."

- *Q: Why avoid `Cloneable`?* — A: No `clone()` in the interface, protected `Object.clone()`, shallow by default, skips constructors, cannot deep-copy `final` fields, checked exception.
- *Q: How do you deep-copy an object graph with cycles?* — A: Keep an identity map from original → copy; before copying an object, check the map and reuse the copy if it exists.
- *Q: Prototype vs Spring prototype scope?* — A: Spring's scope just means "new instance per request for the bean" — it constructs, it does not clone.
- *Q: Serialization-based deep copy?* — A: Possible (serialize then deserialize) but slow, needs everything `Serializable`, and bypasses constructors — a last resort.

Self-check

- **Q:** What is the difference between shallow and deep copy? **A:** Shallow copies references (mutable parts are shared); deep also copies what they point to.
- **Q:** Why can't `DeepPlaylist` make `songs` final? **A:** `clone()` must reassign it after `super.clone()`, which is impossible for a final field.
- **Q:** What should a prototype registry return? **A:** A copy of the stored prototype, never the original.

TL;DR

Copy a configured object instead of building one — saves setup and works when you only know the interface. Decide shallow vs deep per field. Copy constructors, not `Cloneable`. Never copy things with identity.

8.14 Bonus — Object Pool

What it is

An **Object Pool** keeps a **bounded set of reusable objects**. Instead of creating an object, using it once and throwing it away, a client **borrows** one from the pool (`acquire`), uses it, and **returns** it (`release`) so the next client can reuse it. It is not one of the GoF 23, but it is a well-known creational pattern.

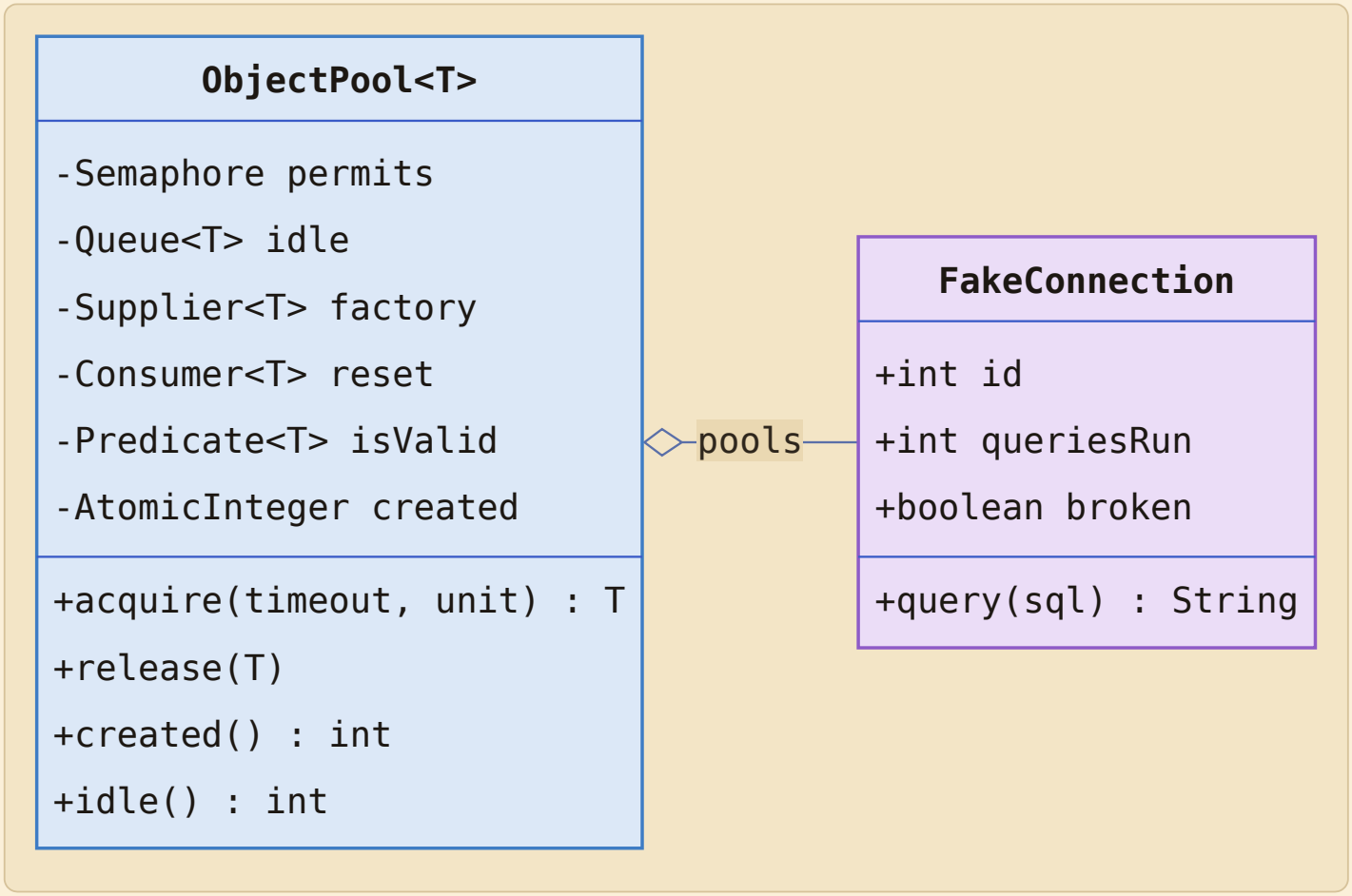
Everyday analogy. A library, or a bike-sharing dock. There are a fixed number of bikes. You take one, ride, and dock it again. If all are out, you wait (or give up after a while). Nobody builds a new bike for every ride.

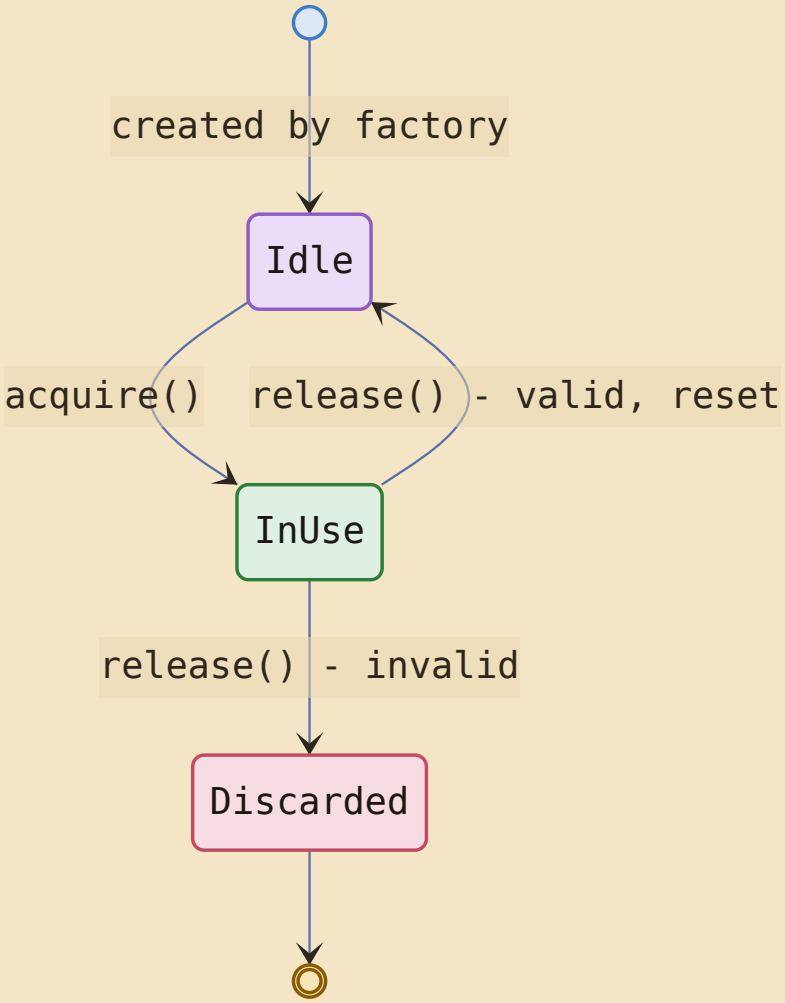
The problem it solves

Some objects are **expensive to create** (a database connection needs a TCP handshake — the back-and-forth that opens a network connection — then TLS, the encryption layer behind `https`, then authentication, proving who you are; see System Design Fundamentals) and/or **limited** (the database accepts only so many connections). Creating one per request is slow and can overwhelm the server. A pool pays the creation cost a few times and **caps** the total.

How it works

1. Keep an **idle queue** (a first-in, first-out waiting line) of ready objects, a count of how many are alive, and a cap on how many may be lent out at once (here a `Semaphore` — a counter of permits that a thread must take before going on and hand back afterwards — with one permit per object).
2. **acquire()**: wait (with a timeout) for a free slot; then take an idle object if there is one, otherwise create a new one. If no slot frees up before the timeout, fail.
3. The client uses the object and **always releases it in a `finally` block** (code that always runs, even if an exception was thrown) — a leaked (never returned) object is gone from the pool forever.
4. **release()**: **validate** the object (is the connection still alive?); if broken, discard it so a fresh one can be made later; otherwise **reset** its state (clear per-borrower data) and put it back. Either way, free the slot.
5. Make all of this **thread-safe** — pools exist *because* many threads compete.




created by factory
Idle
acquire()
release() - valid, reset
InUse
release() - invalid
Discarded

Java implementation

```java
// File: code/lld/ch08/objectpool/ObjectPool.java
package lld.ch08.objectpool;

import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Consumer;
import java.util.function.Predicate;
import java.util.function.Supplier;

/**
 * A bounded, thread-safe object pool.
 * A Semaphore holds one permit per object that may be lent out, so at most maxSize objects
 * are ever borrowed at once. Objects are created lazily, handed out with acquire(), and
 * returned with release().
 */
public final class ObjectPool<T> {
    private final Semaphore permits;                              // one permit = one object may be borrowed
    private final ConcurrentLinkedQueue<T> idle = new ConcurrentLinkedQueue<>();
    private final Supplier<T> factory;        // how to make a new object
    private final Consumer<T> reset;          // how to clean an object before reuse
    private final Predicate<T> isValid;       // is a returned object still usable?
    private final AtomicInteger created = new AtomicInteger();    // objects currently alive (idle + borrowed)

    public ObjectPool(int maxSize, Supplier<T> factory, Consumer<T> reset, Predicate<T> isValid) {
        if (maxSize <= 0) throw new IllegalArgumentException("maxSize must be positive");
        this.permits = new Semaphore(maxSize, true);              // fair: waiters are served in order
        this.factory = factory;
        this.reset = reset;
        this.isValid = isValid;
    }

    /** Borrow an object, waiting up to the timeout if all are in use. */
    public T acquire(long timeout, TimeUnit unit) throws InterruptedException {
        if (!permits.tryAcquire(timeout, unit)) {
            throw new IllegalStateException("pool exhausted: timed out after " + timeout + " " + unit);
        }
        // Holding a permit guarantees we are within the cap: reuse an idle object or create one.
        T obj = idle.poll();
        if (obj != null) return obj;
        try {
            obj = factory.get();
            created.incrementAndGet();
            return obj;
        } catch (RuntimeException e) {
            permits.release();                                    // creation failed: give the permit back
            throw e;
        }
    }

    /** Give an object back. Broken objects are discarded so a fresh one can be created later. */
    public void release(T obj) {
        try {
            if (isValid.test(obj)) {
                reset.accept(obj);
                idle.offer(obj);
            } else {
                created.decrementAndGet();                        // discard it; its permit still comes back
            }
        } finally {
            permits.release();                                    // wakes one waiting borrower, if any
        }
    }

    public int created() { return created.get(); }
```

```
    public int idle() { return idle.size(); }
}
```

```java
// File: code/lld/ch08/objectpool/Main.java
package lld.ch08.objectpool;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {

    /** Stand-in for an expensive resource such as a DB connection. */
    static final class FakeConnection {
        private static final AtomicInteger IDS = new AtomicInteger();
        final int id = IDS.incrementAndGet();
        int queriesRun = 0;
        boolean broken = false;
        String query(String sql) { queriesRun++; return "conn#" + id + " ran: " + sql; }
    }

    public static void main(String[] args) throws Exception {
        ObjectPool<FakeConnection> pool = new ObjectPool<>(
                3,
                () -> { System.out.println("  (opening a new connection - expensive)"); return new
FakeConnection(); },
                c -> c.queriesRun = 0,              // reset per-borrower state
                c -> !c.broken);                    // validate on return

        System.out.println("== 10 tasks on 6 threads share at most 3 connections ==");
        List<Integer> usedIds = new CopyOnWriteArrayList<>();
        try (ExecutorService workers = Executors.newFixedThreadPool(6)) {
            List<Future<?>> futures = new ArrayList<>();
            for (int i = 0; i < 10; i++) {
                final int task = i;
                futures.add(workers.submit(() -> {
                    FakeConnection c = pool.acquire(2, TimeUnit.SECONDS);
                    try {                                       // ALWAYS release in finally
                        c.query("SELECT " + task);
                        usedIds.add(c.id);
                        Thread.sleep(20);
                    } finally {
                        pool.release(c);
                    }
                    return null;
                }));
            }
            for (Future<?> f : futures) f.get();
        }
        System.out.println("  tasks finished: " + usedIds.size()
                + ", connections ever created: " + pool.created()
                + ", distinct connections used: " + usedIds.stream().distinct().count());

        System.out.println("\n== A broken connection is discarded, not reused ==");
        FakeConnection c = pool.acquire(1, TimeUnit.SECONDS);
        c.broken = true;
        pool.release(c);
        System.out.println("  created after discarding: " + pool.created() + ", idle: " + pool.idle());

        System.out.println("\n== Exhaustion: borrowers wait, then time out ==");
        List<FakeConnection> held = new ArrayList<>();
        for (int i = 0; i < 3; i++) held.add(pool.acquire(1, TimeUnit.SECONDS));
        try {
            pool.acquire(100, TimeUnit.MILLISECONDS);
        } catch (IllegalStateException e) {
            System.out.println("  " + e.getMessage());
        }
        held.forEach(pool::release);
        System.out.println("  after returning all: idle = " + pool.idle());
```

```
    }
}
```

- A **`Semaphore`** with `maxSize` permits enforces the cap: a borrower must obtain a permit first (`tryAcquire(timeout)` waits with a deadline), so no more than `maxSize` objects can ever be out at once — however many threads race. Holding a permit then means "reuse an idle object if there is one, otherwise it is safe to create one".
- `release()` returns the permit in a `finally` block, **even when the object is discarded** — so a borrower waiting for a slot wakes up and can create a replacement. (A design that only waits on the idle queue would leave that waiter stuck until its timeout although capacity is free.) If the factory throws, `acquire()` gives its permit back too, so a failed creation never shrinks the pool.
- `ConcurrentLinkedQueue` holds the idle objects; handing an object from one thread to another through a concurrent queue (and the semaphore) also makes the previous borrower's writes visible to the next one.
- Validation + reset on `release()` keep broken or dirty objects out.
- `Main` runs 10 tasks on 6 threads with a pool of 3: only 3 connections are ever opened.

Output:

```
== 10 tasks on 6 threads share at most 3 connections ==
  (opening a new connection - expensive)
  (opening a new connection - expensive)
  (opening a new connection - expensive)
  tasks finished: 10, connections ever created: 3, distinct connections used: 3

== A broken connection is discarded, not reused ==
  created after discarding: 2, idle: 2

== Exhaustion: borrowers wait, then time out ==
  (opening a new connection - expensive)
  pool exhausted: timed out after 100 MILLISECONDS
  after returning all: idle = 3
```

Trade-offs

Pros	Cons
Avoids repeated expensive creation	Complexity: sizing, validation, timeouts, leak detection
Caps resource usage (protects the DB)	Leaked (never released) objects shrink the pool until it deadlocks (everyone waits forever)
Predictable latency (time per request) after warm-up	Stale state if reset is incomplete — one user's data leaks to the next
	For cheap objects it is **slower** than `new` — modern JVM allocation and GC (garbage collection — the JVM's automatic memory clean-up) are very fast

When not to use it

- For **cheap** objects (small value objects, `StringBuilder`s) — the JVM allocates them faster than a pool can hand them out.
- For **virtual threads** (Java 21; lightweight threads managed by the JVM, cheap enough to create by the million) — they are cheap by design; create one per task instead of pooling them.
- When a **well-tested library** exists — use HikariCP, Apache Commons Pool 2 or your framework's pool rather than writing your own in production.

Common mistakes

- Not releasing in `finally` (or not using try-with-resources (Java's `try (...)` form that closes things automatically) wrappers).
- Not resetting borrower state.
- Unbounded waits — always use a timeout.
- Returning the same object twice (double release). In our teaching pool that would put one object in the idle queue twice *and* add an extra permit, silently raising the cap; production pools track which objects are

currently borrowed and reject a second release.

Real-world examples

- **Database connection pools:** HikariCP (Spring Boot's default), Apache DBCP, c3p0.
- **Thread pools:** `ThreadPoolExecutor` / `Executors.newFixedThreadPool` reuse worker threads.
- **Apache Commons Pool 2** `GenericObjectPool` (used by Jedis for Redis connections).
- **Netty** `PooledByteBufAllocator` pools buffers to reduce GC pressure.
- Not pools, though often confused: the `String` intern pool and the `Integer` cache (-128..127) share immutable instances — that is **Flyweight** (Chapter 9), nothing is borrowed or returned.

In the interview

> **30-second answer:** "An object pool reuses a bounded set of expensive objects like DB connections. acquire takes an idle one, creates one if under the cap, or waits with a timeout; release validates, resets and returns it. It must be thread-safe, and clients must release in finally. For cheap objects it's counter-productive, and in production I'd use HikariCP."

- *Q: How do you size a pool?* — A: From measured concurrency and downstream limits (the capacity of the services you call, e.g. the database); HikariCP's guidance is that small pools (roughly core-count scale) often beat big ones, because the database can only do so much in parallel.
- *Q: How do you detect leaks?* — A: Record who borrowed each object and when; log if held longer than a threshold (HikariCP's `leakDetectionThreshold` does this).

Self-check

- **Q:** Object Pool vs Flyweight? **A:** Pool lends mutable objects exclusively and takes them back; Flyweight shares immutable objects among everyone at once.
- **Q:** Why validate on release? **A:** So broken objects (dead connections) are discarded instead of handed to the next borrower.

TL;DR

Borrow → use → return (in `finally`). Bounded, thread-safe, validated, reset. Worth it only for expensive or limited resources; use a mature library in production.

8.15 Creational patterns compared

At a glance

Pattern	Distinctive feature	Use when	Course example	JDK / Spring example	Pros	Cons
Singleton	Only one instance of the class exists	One shared resource: configuration, pool, registry	One `ConfigStore` (or one DB connection pool)	`Runtime.getRuntime()`, Spring singleton-scoped beans	Global access; single initialization	Hurts unit testing (testing one class on its own); hidden dependencies; global state
Simple Factory *(idiom)*	One method maps a type value to a class	Small, stable set of types chosen by config	Notification by channel	`Calendar.getInstance()`, `EnumSet.noneOf`, `@Bean` methods	Short, centralizes the switch	Edit it for every new type
Factory Method	Subclasses decide the created type via an overridable method	Shared procedure, varying product; others will add types	PDF / HTML report generator (documents of different types — PDF, Word, HTML…)	`Collection.iterator()`, Spring `FactoryBean`	Open–Closed; decouples creator from products	More classes; needs subclasses
Abstract Factory	Creates *families* of related objects without concrete classes	Products must be compatible; swap a whole set at once	Postgres vs MySQL storage; Windows vs macOS widgets	`DocumentBuilderFactory`, JDBC `Connection`, Spring profiles	Guarantees compatibility; Open–Closed for families	Complex; adding a product kind edits every factory
Builder	Separates construction of a complex object from its representation	Many optional fields, validation, immutability, step-by-step control	`HttpRequest`; building a custom meal	`StringBuilder`, `HttpClient.newBuilder()`, `UriComponentsBuilder`	Step-by-step, readable, immutable, validated	Verbose for simple objects; field duplication; Director optional
Prototype	Creates objects by copying an existing one	Setup is expensive, or only the interface is known	`NotificationTemplate`; cloning a customized car configuration	Copy constructors, `ArrayList.clone()`, Unity `Instantiate`	Skips setup; fewer subclasses; runtime-configured kinds	Deep copy is hard; `Cloneable` is broken; identity issues
Object Pool *(bonus)*	Reuses a bounded set of objects	Expensive or limited resources	DB connections	HikariCP, `ThreadPoolExecutor`	Fast reuse; caps load	Leaks, stale state, pointless for cheap objects

Pick by the question you are asking

You are asking…	Pattern
"How do I make sure there is only one?"	Singleton (or DI singleton scope)
"How do I avoid `new ConcreteClass()` scattered around?"	Simple Factory
"How do I let subclasses/plug-ins decide what to create?"	Factory Method
"How do I make sure these objects always match?"	Abstract Factory
"How do I make this 10-argument constructor readable and safe?"	Builder
"How do I make another one of *this*, cheaply or without knowing its class?"	Prototype
"How do I stop creating and destroying this expensive thing?"	Object Pool

How they combine

- An **Abstract Factory** is often implemented with **Factory Methods**, may use **Prototypes** (a factory configured with prototype objects it copies), and is frequently a **Singleton** (one factory per family) — though passing it in is better.
- A **Builder** can use a **Factory** to pick which parts to build, and a **Director** can be a Singleton-like stateless (keeps no data between calls) service.
- Designs often *start* with Factory Method (simple, subclass-based) and *evolve* towards Abstract Factory, Prototype or Builder as more flexibility is needed.

8.16 Chapter self-check

1. **Q:** Name the three GoF categories and how many patterns each has. **A:** Creational 5, Structural 7, Behavioral 11 = 23.
2. **Q:** Which two principles does the GoF book put at the centre? **A:** Program to an interface, not an implementation; favour composition over inheritance.
3. **Q:** What is the Golden Hammer? **A:** Applying a favourite tool/pattern to every problem regardless of fit.
4. **Q:** Why must the DCL field be `volatile`? **A:** To prevent another thread from seeing a reference to a not-yet-fully-constructed object (it creates a happens-before edge).
5. **Q:** Simple Factory vs Factory Method in one line? **A:** A switch in one method vs a creation hook overridden by subclasses.
6. **Q:** Factory Method vs Abstract Factory in one line? **A:** One product by inheritance vs a family of matching products by composition.
7. **Q:** Builder's single most important method, and why? **A:** `build()` — the one gate that validates cross-field rules and creates the immutable product.
8. **Q:** What is the classic Prototype bug? **A:** A shallow copy sharing a mutable collection with the original.
9. **Q:** Why is `enum` the most robust Java singleton? **A:** The JVM guarantees one instance per constant and blocks reflective creation and serialization duplicates.
10. **Q:** In an interview, how many patterns should a 45-minute design use? **A:** Usually one or two — and name the one you rejected.

8.17 Chapter TL;DR

- **Patterns** are named, proven solution templates with trade-offs — vocabulary first, code second. GoF (1994) catalogued 23 in three categories.
- **Pick from the signal, not the catalogue.** One or two patterns per interview design; always state the cost and the alternative.
- **Singleton** — one per JVM; eager / holder / enum; DCL needs `volatile`; prefer DI.
- **Simple Factory** — one `create(type)`; idiom, not GoF.
- **Factory Method** — base-class procedure + subclass-chosen product.
- **Abstract Factory** — one factory, one matching family; chosen at the composition root.

- **Builder** — named steps, one validating `build()`, immutable product.
- **Prototype** — copy a configured object; decide shallow vs deep; avoid `Cloneable`.
- **Object Pool** — borrow/return expensive objects; use a real library in production.

Chapter 9 — Structural Design Patterns

> **Where this fits.** Chapter 8 was about *creating* objects. This chapter is about *putting objects together*: how to connect, wrap, nest and share them so that larger structures stay flexible. Chapters 10–11 then cover how objects *talk to each other* (behavioral patterns).

What you will learn

#	Pattern	One-line intent	Everyday analogy
9.1	Adapter	Translate a class's interface into the one your code already uses	A travel plug adapter
9.2	Bridge	Separate two independently varying dimensions into two hierarchies joined by a reference	A TV remote (control) and the TV (device) are sold separately
9.3	Composite	Treat one object and a tree of objects the same way	A folder can hold files and other folders
9.4	Decorator	Add behaviour to one object at run time by wrapping it	Toppings on a coffee
9.5	Facade	One simple entry point in front of a complicated subsystem	A hotel concierge
9.6	Flyweight	Share the repeating part of many objects to save memory	One font glyph "e" reused for every "e" on a page
9.7	Proxy	A stand-in with the same interface that controls access	A receptionist who decides whether you may see the manager
9.8	Comparison	Adapter vs Decorator vs Proxy vs Facade vs Bridge	—

How to run every example in this chapter

```
cd chapters/09-patterns-structural/code
javac --release 21 -d /tmp/lld_09 $(find . -name '*.java')
java -cp /tmp/lld_09 lld.ch09.adapter.Main     # or bridge, composite, decorator, facade, flyweight, proxy
```

All seven `Main` classes compile with Java 21 (standard library only — the classes that ship with Java itself) and run. The output printed under each example is the real output of that program.

> **Code-style note.** Every top-level type has its own file, as in a real codebase, and the types are package-private (visible only to classes in the same package — a package being a named folder of related classes) because each example lives in one package. Where a section shows several short files in a row, read them top to bottom: interface first, then implementations, then `Main`. The whole folder compiles with `javac --release 21 -Xlint:all` with **zero warnings**.

9.0 What structural patterns are

What it is

A **design pattern** is a named, reusable solution to a design problem that keeps coming back. The "Gang of Four" (GoF) book — *Design Patterns: Elements of Reusable Object-Oriented Software* (Gamma, Helm, Johnson, Vlissides, 1994) — sorted its 23 patterns into three families:

Family	Question it answers	Examples
Creational (Ch. 8)	How do I create objects?	Singleton, Factory Method, Builder
Structural (this chapter)	**How do I combine classes and objects into bigger structures?**	**Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy**
Behavioral (Ch. 10–11)	How do objects share work and talk?	Strategy, Observer, Command

Structural patterns help you combine small objects and classes into larger, more complex structures while keeping those structures flexible and efficient. Used well, they:

- **simplify complex structures** (a Facade hides many steps; a Composite hides nesting),
- **promote reuse** (an Adapter lets you reuse a class whose interface does not fit),
- **improve flexibility by decoupling** classes from each other — i.e. reducing how much each one depends on the others' details (Bridge, Decorator, Proxy),
- **improve maintainability**, because the relationships between entities are clear and each change stays local.

There are many structural patterns, but the seven the GoF named — and that interviewers ask about — are **Adapter, Bridge, Composite, Decorator, Facade, Flyweight and Proxy**.

Three words you need first

- **Composition** ("has-a"): an object holds a reference to another object and uses it. Almost every structural pattern is built on composition.
- **Delegation**: forwarding a call to the object you hold. `wrapper.pay()` calls `inner.charge()`.
- **Interface** (in the design sense): the set of methods a caller can use on an object. In Java this is often, but not always, an `interface` type.

> **Key insight.** Four of these patterns (Adapter, Decorator, Proxy, Facade) have the *same physical shape*: one object holds another and forwards calls to it. What tells them apart is **intent** — what changes as the call passes through. Keep this in mind; section 9.8 puts them side by side.

9.1 Adapter Pattern

What it is

The **Adapter** pattern lets your code use a class whose methods have the "wrong" shape — one you are not able to change — by placing a small translator class between the two. GoF's intent: *convert the interface of a class into another interface clients expect; Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.*

Analogy: your laptop charger has a US plug; the hotel wall in India has a different socket. You do not rewire the wall or cut the plug — you put a small travel adapter in between.

GoF also calls it **Wrapper**. Vocabulary:

Term	Meaning
Target	The interface your code wants to use (`PaymentGateway`).
Adaptee	The existing class with the "wrong" interface (`StripeClient`). You usually do not own it.
Adapter	The class that implements the Target and translates calls to the Adaptee (`StripeGateway`).
Client	Code that uses the Target (`checkout`).

The problem it solves

The shape your code calls and the shape the other component offers do not match. Typical cases:

- your checkout wants to call `pay(Money)`, but the payment SDK (software development kit — the vendor's ready-made client library) only offers `charge(amountInCents, currencyCode)`;

- an old service answers in XML, yet every consumer after it works with JSON (XML and JSON are two common text formats for structured data);
- the analytics library asks for `track(String, Map)`, while your code naturally produces an `Event` object.

The key constraint: **the other side is not yours to change.** It may be a third-party library (code written and owned by an outside company or project), a service owned by another team, or code with dozens of existing callers. Bending *your* code to its shape means touching every call site (each place in the code that calls the method) — and repeating the whole exercise the day that vendor is swapped out.

A concrete version: the checkout has to support Stripe now and perhaps PayPal in a few months (both are online payment providers). Stripe expects the amount as a whole number of **cents** plus a **lowercase** currency code; PayPal expects a **decimal string** such as "42.50" plus an **uppercase** code. Your own code would rather reason about a `Money` value than either of those.

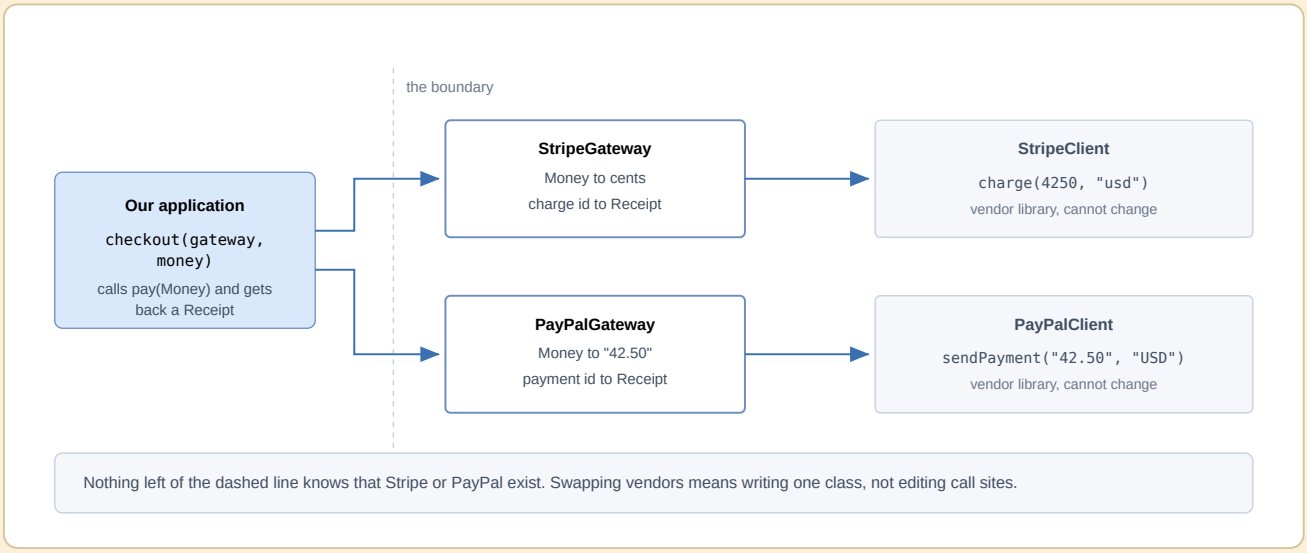


Figure — Two adapters (StripeGateway, PayPalGateway) translate our `pay(Money)` into each vendor's own call; nothing on the application side knows which vendor exists (course diagram)

In the figure above, everything to the left of the dashed boundary talks only in `Money` and `Receipt`. Swapping vendors means writing one new class, not touching call sites.

How it works

1. **Define the Target interface in your own vocabulary** (`PaymentGateway.pay(Money) -> Receipt`). It mentions no vendor.
2. **Write one adapter class per adaptee.** It `implements` the Target and **holds** the vendor object.
3. **Translate inputs**: your types into the vendor's types (`Money` → cents + lowercase code).
4. **Call the adaptee.**
5. **Translate outputs and errors back**: vendor result → `Receipt`; vendor exception → *your* exception.
6. **Client code depends only on the Target**, so it never learns which vendor answered.

> **The rule that makes it work:** the adapter's public methods mention **only your types** — parameters, return values *and* exceptions. The moment `pay` hands back a `StripeCharge`, or lets a `StripeCardException` propagate (travel up, uncaught, to the callers), callers depend on Stripe again and the adapter has decoupled nothing.

Structure

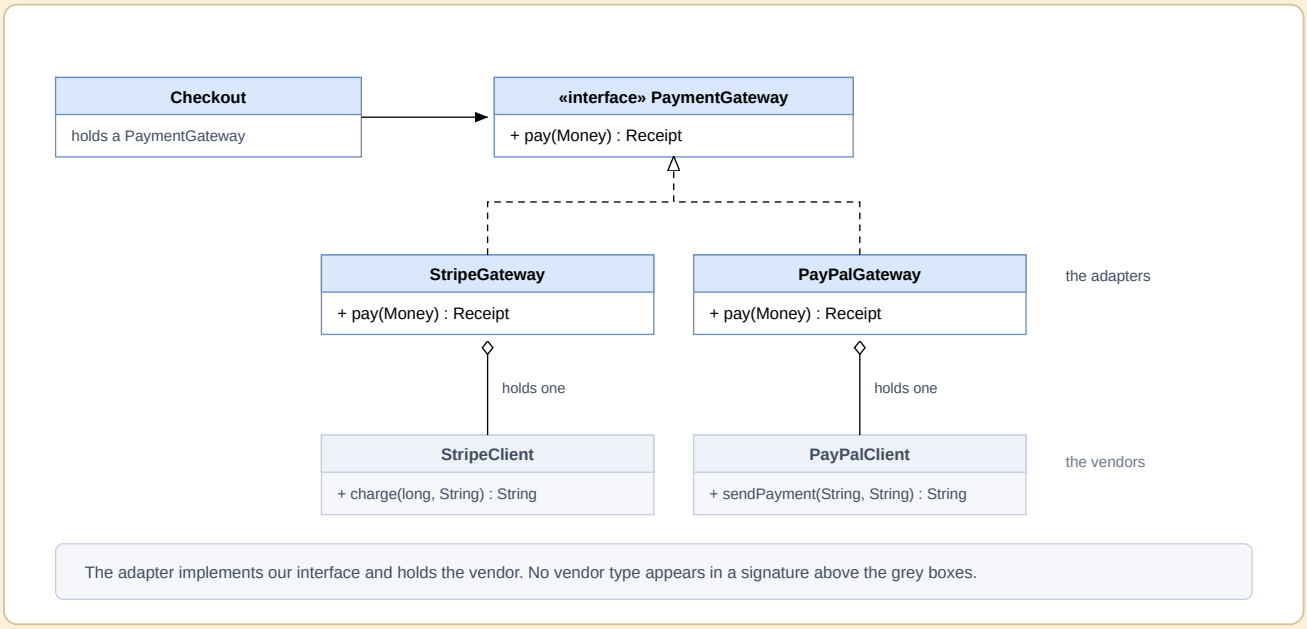
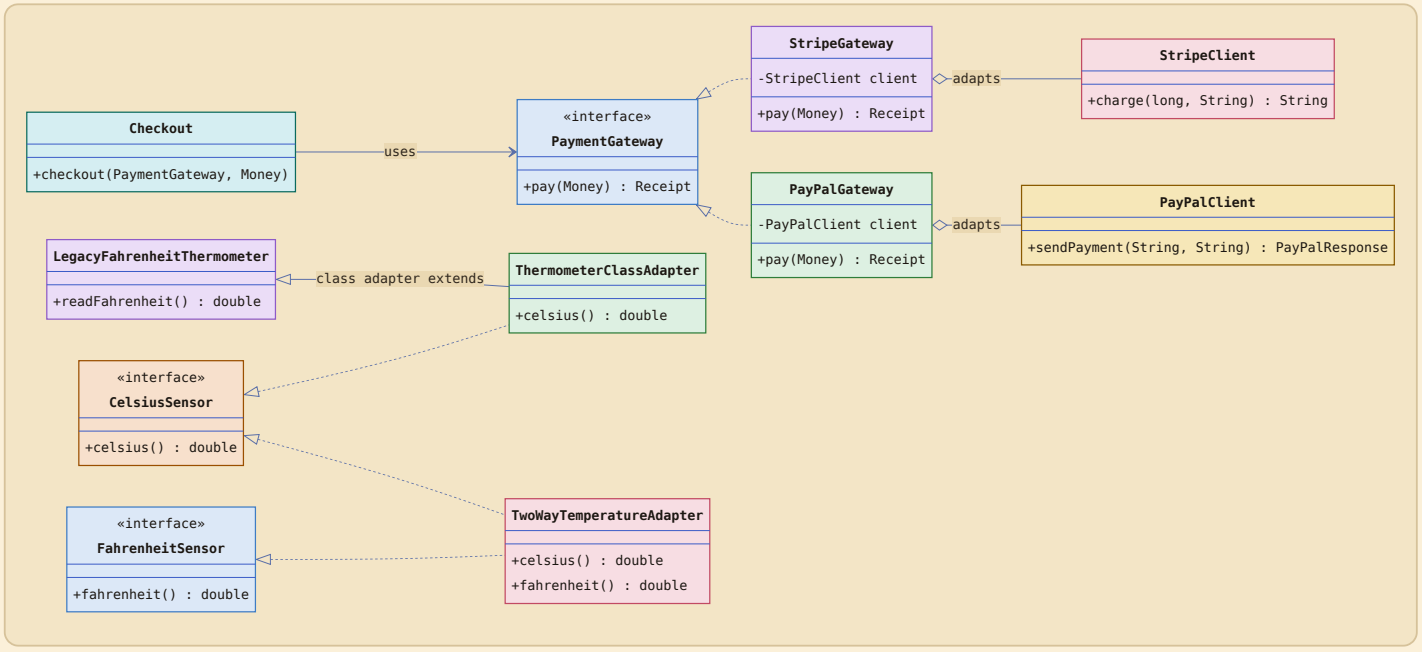


Figure — Adapter class diagram: `Checkout` holds a `PaymentGateway`; `StripeGateway` and `PayPalGateway` implement it and each wrap a vendor client, shown in grey because we do not own them (course diagram)

Reading the class diagram above: `PaymentGateway` is the Target, owned by your application; `StripeGateway` and `PayPalGateway` are the Adapters; `StripeClient` and `PayPalClient` are the Adaptees (third-party code); `Checkout` stands for any Client. Every mismatch — method names, parameter types, casing rules — is soaked up inside the two adapter classes, and nothing above them sees it.

Our Java design (including the class and two-way adapters):



Java example

```java
// File: code/lld/ch09/adapter/Money.java
package lld.ch09.adapter;

import java.util.Locale;

/** Our own vocabulary. No vendor type ever appears here. */
record Money(long cents, String currency) {
    Money {
        if (cents < 0) throw new IllegalArgumentException("negative amount");
        currency = currency.toUpperCase(Locale.ROOT);
    }

    /** 4250 cents -> "42.50" */
    String asDecimal() {
        return String.format(Locale.ROOT, "%d.%02d", cents / 100, cents % 100);
    }
}
```

```java
// File: code/lld/ch09/adapter/Receipt.java
package lld.ch09.adapter;

record Receipt(String id, String status) { }
```

```java
// File: code/lld/ch09/adapter/PaymentFailedException.java
package lld.ch09.adapter;

import java.io.Serial;

/** Our exception type: callers never see a vendor exception. */
class PaymentFailedException extends RuntimeException {
    @Serial private static final long serialVersionUID = 1L;

    PaymentFailedException(String message, Throwable cause) { super(message, cause); }
}
```

```java
// File: code/lld/ch09/adapter/PaymentGateway.java
package lld.ch09.adapter;

/** TARGET interface: the shape our application wants to talk to. */
interface PaymentGateway {
    Receipt pay(Money amount);
}
```

```java
// File: code/lld/ch09/adapter/StripeClient.java
package lld.ch09.adapter;

/*
 * ADAPTEES: pretend these come from third-party jars we cannot edit.
 * Note the different method names, argument types and conventions.
 */

/** Stripe-like SDK: integer cents + lowercase currency, throws its own exception. */
final class StripeClient {
    private int next = 1001;

    String charge(long amountInCents, String currencyCode) {
        System.out.printf("  [stripe] charge(%d, \"%s\")%n", amountInCents, currencyCode);
        if (amountInCents > 100_000) throw new StripeCardException("card_declined");
        return "ch_" + next++;
    }
}
```

```java
// File: code/lld/ch09/adapter/StripeCardException.java
package lld.ch09.adapter;

import java.io.Serial;

final class StripeCardException extends RuntimeException {
    @Serial private static final long serialVersionUID = 1L;

    StripeCardException(String code) { super(code); }
}
```

```java
// File: code/lld/ch09/adapter/PayPalClient.java
package lld.ch09.adapter;

/** PayPal-like SDK: decimal string + uppercase currency, returns a response object. */
final class PayPalClient {
    record PayPalResponse(String paymentId, String state) { }

    PayPalResponse sendPayment(String total, String currency) {
        System.out.printf("  [paypal] sendPayment(\"%s\", \"%s\")%n", total, currency);
        return new PayPalResponse("PAY-" + total.replace(".", ""), "approved");
    }
}
```

```java
// File: code/lld/ch09/adapter/StripeGateway.java
package lld.ch09.adapter;

import java.util.Locale;

/** OBJECT ADAPTER: implements our interface, HOLDS the vendor object (composition). */
final class StripeGateway implements PaymentGateway {
    private final StripeClient client;

    StripeGateway(StripeClient client) { this.client = client; }

    @Override
    public Receipt pay(Money amount) {
        try {
            String chargeId = client.charge(amount.cents(), amount.currency().toLowerCase(Locale.ROOT));
            return new Receipt(chargeId, "SUCCEEDED");
        } catch (StripeCardException e) {               // translate the vendor's error too
            throw new PaymentFailedException("Stripe declined: " + e.getMessage(), e);
        }
    }
}
```

```java
// File: code/lld/ch09/adapter/PayPalGateway.java
package lld.ch09.adapter;

final class PayPalGateway implements PaymentGateway {
    private final PayPalClient client;

    PayPalGateway(PayPalClient client) { this.client = client; }

    @Override
    public Receipt pay(Money amount) {
        PayPalClient.PayPalResponse r = client.sendPayment(amount.asDecimal(), amount.currency());
        String status = "approved".equals(r.state()) ? "SUCCEEDED" : "FAILED";
        return new Receipt(r.paymentId(), status);
    }
}
```

```java
// File: code/lld/ch09/adapter/CelsiusSensor.java
package lld.ch09.adapter;

/** Target used by our new weather dashboard. */
interface CelsiusSensor {
    double celsius();
}
```

```java
// File: code/lld/ch09/adapter/FahrenheitSensor.java
package lld.ch09.adapter;

/** Interface used by an old US reporting module. */
interface FahrenheitSensor {
    double fahrenheit();
}
```

```java
// File: code/lld/ch09/adapter/LegacyFahrenheitThermometer.java
package lld.ch09.adapter;

/** A legacy class we cannot change. It is NOT final, so a class adapter can extend it. */
class LegacyFahrenheitThermometer {
    private final double reading;

    LegacyFahrenheitThermometer(double reading) { this.reading = reading; }

    public double readFahrenheit() { return reading; }
}
```

```java
// File: code/lld/ch09/adapter/ThermometerClassAdapter.java
package lld.ch09.adapter;

/**
 * CLASS ADAPTER: inherits from the adaptee and implements the target.
 * Works in Java only because the target is an interface (no multiple class inheritance).
 */
final class ThermometerClassAdapter extends LegacyFahrenheitThermometer implements CelsiusSensor {
    ThermometerClassAdapter(double fahrenheit) { super(fahrenheit); }

    @Override
    public double celsius() { return (readFahrenheit() - 32) * 5.0 / 9.0; }
    // Downside: readFahrenheit() is still public on this object -> the adaptee's API leaks.
}
```

```java
// File: code/lld/ch09/adapter/TwoWayTemperatureAdapter.java
package lld.ch09.adapter;

/**
 * TWO-WAY ADAPTER: implements BOTH interfaces, so the same object can be handed to
 * new code (wants Celsius) and old code (wants Fahrenheit).
 */
final class TwoWayTemperatureAdapter implements CelsiusSensor, FahrenheitSensor {
    private final double celsius;

    private TwoWayTemperatureAdapter(double celsius) { this.celsius = celsius; }

    static TwoWayTemperatureAdapter from(CelsiusSensor s) {
        return new TwoWayTemperatureAdapter(s.celsius());
    }

    static TwoWayTemperatureAdapter from(FahrenheitSensor s) {
        return new TwoWayTemperatureAdapter((s.fahrenheit() - 32) * 5.0 / 9.0);
    }

    @Override public double celsius()    { return celsius; }
    @Override public double fahrenheit() { return celsius * 9.0 / 5.0 + 32; }
}
```

```java
// File: code/lld/ch09/adapter/Main.java
package lld.ch09.adapter;

import java.io.BufferedReader;
import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.List;

public class Main {

    /** Client code: talks only to OUR interface and never names a vendor. */
    static void checkout(PaymentGateway gateway, Money amount) {
        try {
            Receipt r = gateway.pay(amount);
            System.out.println("  receipt " + r.id() + " " + r.status());
        } catch (PaymentFailedException e) {
            System.out.println("  payment failed -> " + e.getMessage());
        }
    }

    public static void main(String[] args) throws IOException {
        System.out.println("== Object adapters (payment) ==");
        Money amount = new Money(4250, "usd");
        checkout(new StripeGateway(new StripeClient()), amount);
        checkout(new PayPalGateway(new PayPalClient()), amount);
        checkout(new StripeGateway(new StripeClient()), new Money(250_000, "USD"));

        System.out.println("== Class adapter ==");
        CelsiusSensor s = new ThermometerClassAdapter(98.6);
        System.out.printf("  98.6F -> %.1fC%n", s.celsius());

        System.out.println("== Two-way adapter ==");
        FahrenheitSensor oldSensor = () -> 212.0;
        TwoWayTemperatureAdapter both = TwoWayTemperatureAdapter.from(oldSensor);
        CelsiusSensor forNewCode = both;
        FahrenheitSensor forOldCode = both;
        System.out.printf("  new code sees %.1fC, old code sees %.1fF%n",
                forNewCode.celsius(), forOldCode.fahrenheit());

        System.out.println("== JDK adapters ==");
        String[] array = {"a", "b", "c"};
        List<String> view = Arrays.asList(array);   // adapts array -> List, no copy
        array[0] = "z";
        System.out.println("  Arrays.asList sees array change: " + view);
        try {
            view.add("d");
        } catch (UnsupportedOperationException e) {
            System.out.println("  cannot resize an array-backed list: UnsupportedOperationException");
        }
        var bytes = new ByteArrayInputStream("héllo".getBytes(StandardCharsets.UTF_8));
        // InputStreamReader adapts a byte stream (InputStream) to a char stream (Reader)
        try (var reader = new BufferedReader(new InputStreamReader(bytes, StandardCharsets.UTF_8))) {
            System.out.println("  InputStreamReader decoded: " + reader.readLine());
        }
    }
}
```

Output:

```
== Object adapters (payment) ==
  [stripe] charge(4250, "usd")
  receipt ch_1001 SUCCEEDED
  [paypal] sendPayment("42.50", "USD")
  receipt PAY-4250 SUCCEEDED
  [stripe] charge(250000, "usd")
  payment failed -> Stripe declined: card_declined
== Class adapter ==
  98.6F -> 37.0C
== Two-way adapter ==
  new code sees 100.0C, old code sees 212.0F
== JDK adapters ==
  Arrays.asList sees array change: [z, b, c]
  cannot resize an array-backed list: UnsupportedOperationException
  InputStreamReader decoded: héllo
```

What each part does

- `Money`, `Receipt`, `PaymentGateway`, `PaymentFailedException` — **our** vocabulary. The compact constructor (a constructor with no parameter list that runs on every creation) of the `Money` record (Java's short syntax for a data class whose fields cannot change) normalises the currency to upper case once, so every adapter can rely on it.
- `StripeClient`, `PayPalClient` — the **adaptees**. Different method names, argument types, casing and error styles. We treat them as untouchable.
- `StripeGateway` — an **object adapter**. It converts `Money` into `(cents, "usd")`, converts the returned charge id into a `Receipt`, and converts `StripeCardException` into `PaymentFailedException`. The third call in the output shows that translation: the client sees *our* error type.
- `PayPalGateway` — the same idea with a different translation (`Money.asDecimal()` → "42.50"; `"approved"` → `"SUCCEEDED"`).
- `checkout(...)` — the **client**. One function, two completely different vendors, and no `if (stripe)` branch anywhere.
- The JDK section shows two adapters you already use: `Arrays.asList` and `InputStreamReader`.

Object adapter vs class adapter

GoF describes two ways to build an adapter:

	Object adapter (composition)	**Class adapter (inheritance)**
How	Adapter *holds* an adaptee instance and implements the Target	Adapter *extends* the adaptee class and implements the Target
In our code	`StripeGateway`, `PayPalGateway`	`ThermometerClassAdapter extends LegacyFahrenheitThermometer implements CelsiusSensor`
Works when adaptee is `final`?	Yes	**No** — you cannot extend a final class (our `StripeClient` is `final`)
Can adapt a whole family (adaptee + subclasses)?	Yes — pass any subclass instance in	No — bound to one concrete class at compile time (fixed when the code is compiled)
Can override adaptee behaviour?	Harder — you would have to subclass the adaptee and adapt that subclass	Yes (it is a subclass)
Leaks the adaptee's API (its public methods)?	No — only the Target methods are visible	**Yes** — every public adaptee method (e.g. `readFahrenheit()`) is also on the adapter
Java constraint	none	Java has no multiple class inheritance (extending two classes at once), so the **Target must be an interface**

Rule of thumb: prefer the **object adapter**. It follows "favour composition over inheritance", works with final classes, can wrap several objects, and does not inherit methods you did not want. Use a class adapter only when you must override some adaptee behaviour and the adaptee is designed for extension.

Two-way adapter

A **two-way adapter** implements *both* interfaces, so the same object can be given to clients of either side. GoF mentions it for the case where two different clients need to view one object in two different ways.

In the code, `TwoWayTemperatureAdapter implements CelsiusSensor, FahrenheitSensor`. It can be built from either kind of sensor. New dashboard code sees `100.0C`; old US reporting code sees `212.0F` — the same object. This is handy during a **migration** (gradually moving code from an old system to a new one): old and new modules can share one object while you gradually move callers across.

Adapter vs Facade vs Decorator vs Proxy (quick view)

Pattern	What happens to the interface	What happens to the behaviour
Adapter	**Translated** — the client sees your interface, the wrapped object keeps its own	Same work, just re-expressed
Decorator	Kept identical on both sides	**Extended** — extra work runs around the call
Proxy	Kept identical	Same work, but **gated** — it may be postponed, served from a cache (a store of earlier results kept for quick reuse) or rejected
Facade	**Replaced** — a single simpler interface stands in for many	Same work, fewer calls for the client

In one line each: translating shapes → **Adapter**; adding work, same shape → **Decorator**; guarding access, same shape and same work → **Proxy**; one simple door in front of several objects → **Facade**.

A useful test: *adapters exist because someone else picked the interface.* Imagine the vendor rewrote its API tomorrow to match yours exactly — if your class would then have no reason to exist, it was an Adapter. The other three exist because you decided to add something. Bridge can be drawn with the same boxes, but its timing is reversed: a Bridge is planned *before* the classes exist so two hierarchies never fuse; an Adapter is bolted on *after* to reconcile two things that already exist.

Trade-offs

Pros	Cons
The foreign format is contained — only one class knows the vendor's conventions.	**An extra class and an extra call** — stack traces (the list of calls printed when an error happens) gain a frame (one entry in that list) whose only job is conversion.
Changing vendor is local — supporting PayPal beside Stripe means adding a class, not editing callers (Open/Closed Principle — open for extension, closed for modification).	**Conversions can drop information**, and callers cannot see that anything was lost.
Easy to test — faking your own one-method interface is far simpler than faking a vendor SDK.	**Two data models must stay aligned** — each vendor API change forces a mapping change.
Your code keeps its own language (`Money`, `Receipt` rather than cents and charge ids); conversion has a single home (Single Responsibility — one reason to change per class).	**Logic tends to creep in** — retries (automatically re-trying a failed call) and pricing rules drift into the adapter unless reviews stop them.

When not to use it

- **Both sides are yours.** Change one interface so they match. An adapter between two pieces of code you own is overhead you pay forever for a mismatch you could simply remove.
- **The two models really disagree.** When mapping one to the other drops data or forces guesses, the "adapter" grows into a mini-application with bugs of its own; redraw the boundary instead.
- **You want extra behaviour, not a translation.** That job belongs to a Decorator.
- **The conversion happens in exactly one place.** A few inline lines read better than a new class.

Common mistakes

- **Leaking vendor types** (`StripeCharge` return type, `StripeCardException` escaping). Then nothing is decoupled.
- **Putting business logic in the adapter** (discount rules, retries). Adapters translate; they do not decide.
- **Choosing a class adapter by default** — it exposes the adaptee's whole API and breaks on `final` classes.

- **One giant adapter for many vendors** with `if (vendor == STRIPE)` inside — that is the branch the pattern was meant to remove. One adapter per adaptee.
- **Silently dropping information** (e.g. vendor's "pending" state mapped to "SUCCEEDED"). Map every case deliberately.

Real-world examples

- **`java.util.Arrays.asList(T...)`** adapts an array to the `List` interface *without copying* — writes go through to the array (as the output shows) and the list cannot grow, so `add` throws `UnsupportedOperationException`.
- **`java.io.InputStreamReader` / `OutputStreamWriter`** adapt byte streams (raw bytes read one after another) to character streams (text) using a charset (the rule that maps bytes to characters, e.g. UTF-8).
- **`java.util.Collections.enumeration(Collection)`** adapts a modern `Collection` to the legacy (old but still supported) `Enumeration` interface. (*Head First Design Patterns* builds the reverse, an Enumeration-to-Iterator adapter.)
- **SLF4J providers/bindings** — SLF4J (Simple Logging Facade for Java) is one logging API, with an adapter module for each backend (the library that actually writes the log lines) (Logback, Log4j, `java.util.logging`).
- **JDBC (and ODBC) drivers** (JDBC is Java's standard database API; a driver is the library for one particular database) adapt each database's own wire protocol (the exact bytes it sends over the network) to the standard `java.sql` interfaces, which is why the same application code runs on several databases.
- **Spring MVC `HandlerAdapter`** lets the `DispatcherServlet` (the central object in Spring MVC, Spring's web framework, that receives every web request) invoke very different kinds of handlers (annotated controllers, `HttpRequestHandler`, etc.) through one interface.
- **Almost every "client" wrapper around a vendor SDK** in a real codebase — if it exists so the rest of the code does not import the SDK, it is an adapter.

In modern languages

- **First-class functions** (functions that can be stored in variables and passed around like values): if the Target has a single method, a **lambda** (a short unnamed function) is a complete adapter — `PaymentGateway g = m -> new Receipt(stripe.charge(m.cents(), "usd"), "SUCCEEDED");` (our `FahrenheitSensor oldSensor = () -> 212.0;` is the same trick).
- **Python**: duck typing ("if it has the right methods, it fits" — checked when the code runs, never declared) means any object with the expected method names fits, so a small wrapper class — or even a `types.SimpleNamespace` — is enough.
- **TypeScript**: structural typing accepts anything of the right shape, so adapters are frequently plain object literals (objects written inline as `{ ... }`).
- **Go**: interfaces are satisfied implicitly (a type that has the methods automatically counts — no `implements` needed), so the adapter is often just a small named type with one method.

In the interview

> **30-second answer:** "An Adapter converts one interface into another that the client expects. I use it at boundaries I do not control — a vendor SDK, a legacy service. The adapter implements *my* interface, holds the vendor object, and translates inputs, outputs and exceptions, so no vendor type leaks into my code. Swapping vendors becomes one new class. I prefer the object adapter over the class adapter because it uses composition and works with final classes."

Follow-ups

- *Q: Adapter vs Decorator?* — A: Adapter changes the interface and keeps the behaviour. Decorator keeps the interface and changes (adds to) the behaviour.
- *Q: Adapter vs Facade?* — A: An Adapter converts **one** interface to **one** other, usually because you only own one side. A Facade simplifies a **whole subsystem** of several objects into one entry point, usually over code you own.
- *Q: Why not just change the other side?* — A: Say this up front, unprompted: it is a vendor library, another team's API, or code with many callers. That constraint is what justifies the pattern — without it you are pattern-matching out of habit.
- *Q: What ruins an adapter?* — A: A vendor type or vendor exception in the adapter's public signature.
- *Q: How does it help testing?* — A: The checkout depends on a small interface I own, so in tests I pass a fake `PaymentGateway`; I never have to fake the vendor SDK.

- *Q: What is a two-way adapter?* — A: One class implementing both interfaces so it can serve clients of either side, useful during migrations.

Self-check

1. **Q:** Name the four roles in the Adapter pattern. **A:** Target, Adaptee, Adapter, Client.
2. **Q:** Why is `StripeClient` impossible to class-adapt in our code? **A:** It is `final`; a class adapter must extend the adaptee.
3. **Q:** What does `Arrays.asList(arr).add(x)` do, and why? **A:** Throws `UnsupportedOperationException`, because the list is a fixed-size view (adapter) over the array.
4. **Q:** You own both classes and their interfaces differ slightly. Adapter? **A:** No — change the interface; an adapter would be permanent overhead.
5. **Q:** Where should the vendor's exception be handled? **A:** Inside the adapter, translated into your own exception type.

TL;DR

- **Adapter = translator** between the interface you have and the interface you need.
- Implements **your** interface, **holds** the vendor object, translates **in, out and errors**.
- Prefer **object adapter** (composition) over **class adapter** (inheritance). Two-way adapters implement both sides.
- Use at boundaries you **don't control**; skip it when you own both sides.

9.2 Bridge Pattern

What it is

The **Bridge** pattern splits one class that varies in *two independent ways* into **two separate hierarchies** connected by a reference (the "bridge"), so each side can grow on its own.

Analogy: a universal TV remote. Remotes come in kinds (basic, advanced with mute); TVs come in brands (Sony, Samsung). You do not buy a "BasicSonyRemote" — any remote works any TV through a common protocol (an agreed set of signals).

GoF's intent: *decouple an abstraction from its implementation so that the two can vary independently.* GoF also lists **Handle/Body** as another name for it. Its names for the parts — the vocabulary interviewers expect:

Term	Meaning	In our code
Abstraction	The high-level side the client uses; holds a reference to an Implementor	`Notification`
Refined Abstraction	Subclasses of the abstraction	`AlertNotification`, `ReminderNotification`, `DigestNotification`
Implementor	The interface of the low-level side that does the physical work	`Channel`
Concrete Implementor	Real implementations of that low-level work	`EmailChannel`, `SmsChannel`, `PushChannel`

> **Naming trap.** In Bridge, "implementation" is *not* the everyday Java meaning ("a class that implements an interface"). It names the **low-level half of the split** — the side that does the platform-specific, physical work (sending bytes, drawing pixels). The GoF terms are awkward, but they are the ones you will be asked about.

The problem it solves — the class explosion

You have three kinds of notification (alert, reminder, digest) and three delivery channels (email, SMS, push). If both live in **one** inheritance hierarchy you need a class for every combination:

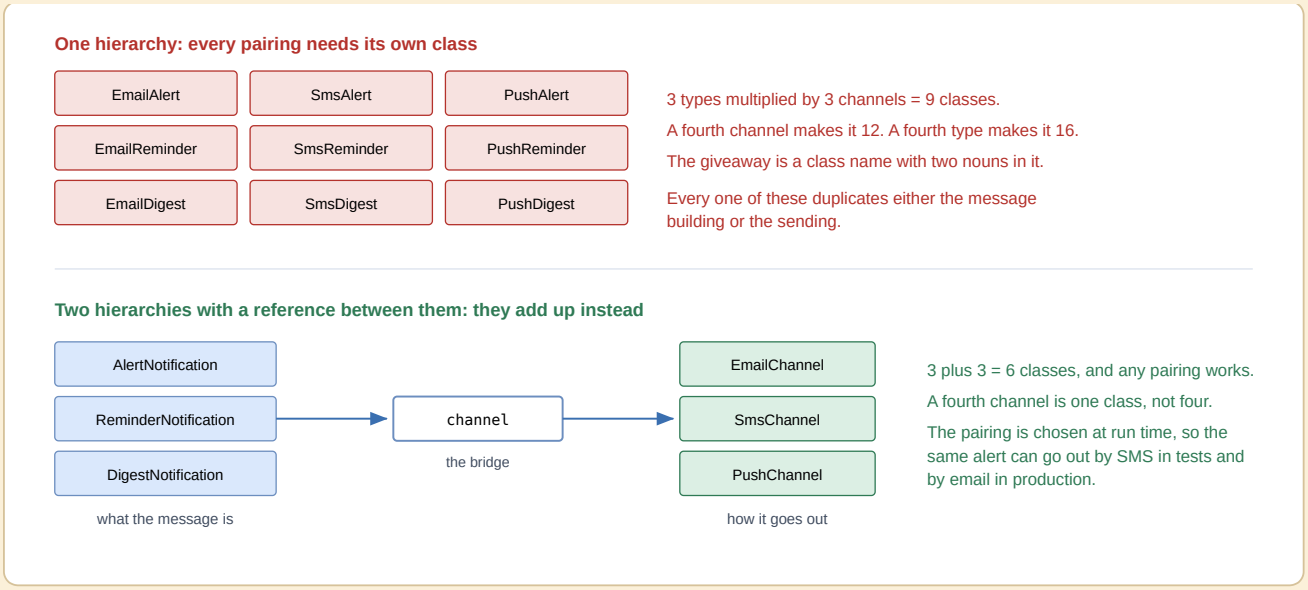


Figure — Top: one hierarchy needs 3 × 3 = 9 classes such as EmailAlert and SmsDigest. Bottom: two hierarchies joined by one `channel` reference need 3 + 3 = 6 (course diagram)

The figure shows the arithmetic. In a single hierarchy the concrete classes **multiply**: 3 × 3 = 9; one more channel gives 12; one more type on top gives 16. With a bridge they **add**: 3 + 3 = 6, and 4 + 4 = 8. (Counts are concrete classes; the `Notification` base and the `Channel` interface are the same in both designs.)

The **smell** (code smell: a surface symptom that hints at a deeper design problem) that points to a missing Bridge is **a class name made of two nouns** — `EmailAlert`, `SmsDigest`, `PushReminder`. Each such class repeats either the "build the message" code or the "send it" code, so changing how SMS delivery works means editing three classes that otherwise have nothing in common.

Splitting the two dimensions has one more pay-off: the pairing becomes a **run-time choice**. The same alert object can be built with an SMS channel in a test and an email channel in production (the live system real users depend on). The code is small; the genuinely hard part is deciding **which dimension is the abstraction and which is the implementation** (see "Getting the split wrong" below).

How it works

1. **Find the two dimensions** that vary independently (what the message *is* vs how it *goes out*).
2. **Make the lower-level dimension an interface** — the Implementor (`Channel.deliver(title, body)`). Keep it small and general.
3. **Make the higher-level dimension an abstract class** — the Abstraction — that **holds a reference** to the Implementor. That field is the bridge.
4. **Refined abstractions** build their own content and **delegate** the physical work across the bridge. They never check which implementor they have.
5. **Pair them at run time**: `new AlertNotification(new SmsChannel(...), ...)`. The same alert can go by SMS in a test and by email in production.

Structure

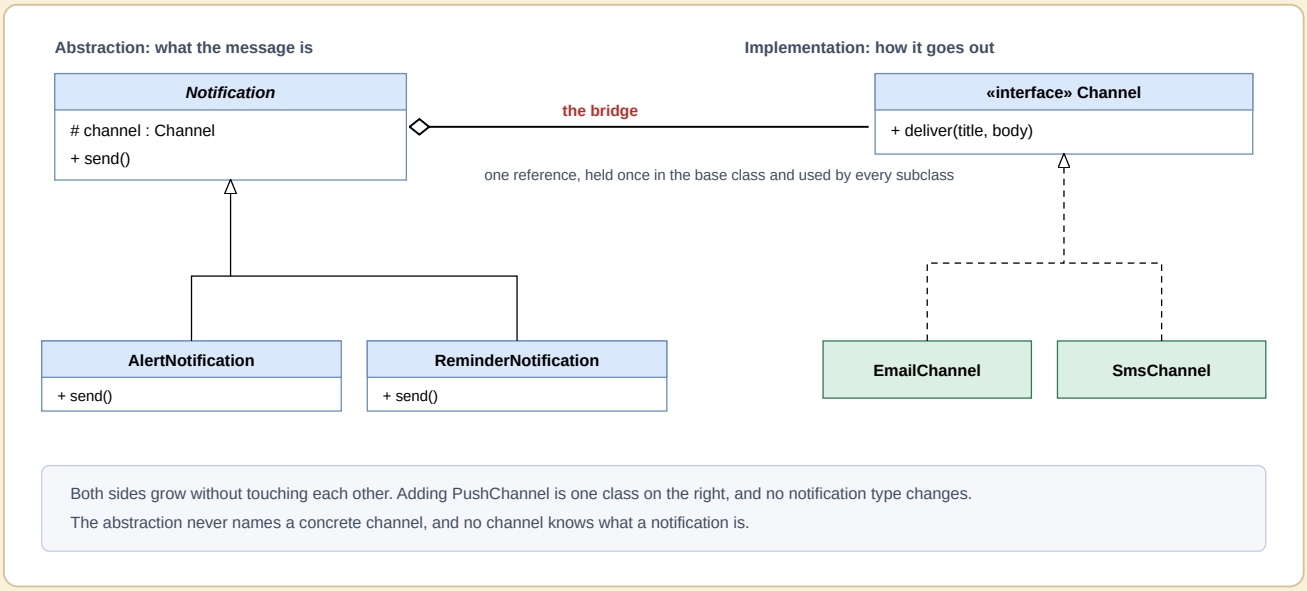
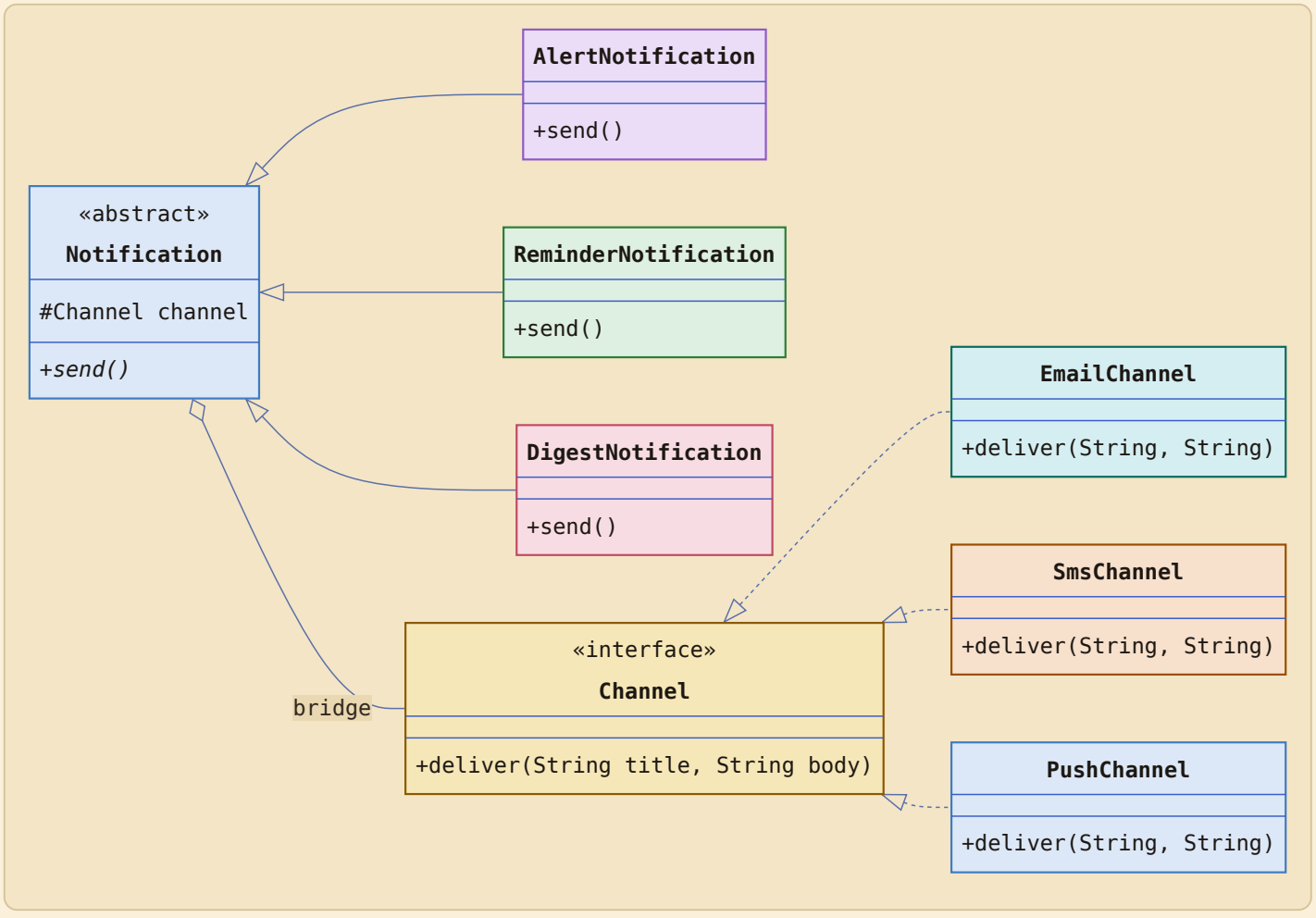


Figure — Bridge class diagram: `Notification` (abstraction) holds a `Channel` (implementor); alert/reminder notifications refine the abstraction; email/SMS channels implement the channel (course diagram)

In the class diagram above, the bridge is that one `channel` field: declared once in the base class, used by every subclass. The channel classes know nothing about notifications — neither what an alert is nor that alerts exist.



Java example

```java
// File: code/lld/ch09/bridge/Channel.java
package lld.ch09.bridge;

/**
 * IMPLEMENTOR: the low-level "how it goes out" side.
 * Kept small and general: two plain strings, nothing channel-specific.
 */
interface Channel {
    void deliver(String title, String body);
}
```

```java
// File: code/lld/ch09/bridge/EmailChannel.java
package lld.ch09.bridge;

/** CONCRETE IMPLEMENTORS: none of them knows what an alert or a digest is. */
final class EmailChannel implements Channel {
    private final String address;

    EmailChannel(String address) { this.address = address; }

    @Override
    public void deliver(String title, String body) {
        System.out.println("  [email to " + address + "] Subject: " + title + " | " + body);
    }
}
```

```java
// File: code/lld/ch09/bridge/SmsChannel.java
package lld.ch09.bridge;

final class SmsChannel implements Channel {
    private static final int LIMIT = 40;           // tiny limit so the demo shows truncation
    private final String phone;

    SmsChannel(String phone) { this.phone = phone; }

    @Override
    public void deliver(String title, String body) {
        String text = title + ": " + body;
        if (text.length() > LIMIT) text = text.substring(0, LIMIT - 3) + "...";
        System.out.println("  [sms to " + phone + "] " + text);
    }
}
```

```java
// File: code/lld/ch09/bridge/PushChannel.java
package lld.ch09.bridge;

final class PushChannel implements Channel {
    private final String deviceId;

    PushChannel(String deviceId) { this.deviceId = deviceId; }

    @Override
    public void deliver(String title, String body) {
        System.out.println("  [push to " + deviceId + "] (" + title + ") " + body);
    }
}
```

```java
// File: code/lld/ch09/bridge/Notification.java
package lld.ch09.bridge;

/**
 * ABSTRACTION: the high-level "what the message is" side.
 * The protected 'channel' field IS the bridge.
 */
abstract class Notification {
    protected final Channel channel;

    protected Notification(Channel channel) { this.channel = channel; }

    /** Builds the message, then hands it across the bridge. Never branches on channel type. */
    abstract void send();
}
```

```java
// File: code/lld/ch09/bridge/AlertNotification.java
package lld.ch09.bridge;

/** REFINED ABSTRACTIONS */
final class AlertNotification extends Notification {
    private final String system;
    private final String problem;

    AlertNotification(Channel channel, String system, String problem) {
        super(channel);
        this.system = system;
        this.problem = problem;
    }

    @Override
    void send() { channel.deliver("ALERT " + system, problem + " - act now"); }
}
```

```java
// File: code/lld/ch09/bridge/ReminderNotification.java
package lld.ch09.bridge;

final class ReminderNotification extends Notification {
    private final String what;
    private final String when;

    ReminderNotification(Channel channel, String what, String when) {
        super(channel);
        this.what = what;
        this.when = when;
    }

    @Override
    void send() { channel.deliver("Reminder", what + " at " + when); }
}
```

```java
// File: code/lld/ch09/bridge/DigestNotification.java
package lld.ch09.bridge;

import java.util.List;

final class DigestNotification extends Notification {
    private final List<String> items;

    DigestNotification(Channel channel, List<String> items) {
        super(channel);
        this.items = List.copyOf(items);
    }

    @Override
    void send() { channel.deliver("Daily digest (" + items.size() + ")", String.join("; ", items)); }
}
```

```java
// File: code/lld/ch09/bridge/Main.java
package lld.ch09.bridge;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        Channel email = new EmailChannel("ops@shop.com");
        Channel sms = new SmsChannel("+1-555-0100");
        Channel push = new PushChannel("pixel-7");

        System.out.println("== Any type x any channel, paired at run time ==");
        List<Notification> outbox = List.of(
                new AlertNotification(email, "payments", "error rate 12%"),
                new AlertNotification(sms, "payments", "error rate 12%"),
                new ReminderNotification(push, "Stand-up", "09:30"),
                new DigestNotification(email, List.of("3 orders", "1 refund", "0 incidents")),
                new DigestNotification(sms, List.of("3 orders", "1 refund", "0 incidents")));
        outbox.forEach(Notification::send);

        System.out.println("== Class count ==");
        int types = 3, channels = 3;
        System.out.println("  one hierarchy (type x channel): " + types * channels + " classes");
        System.out.println("  bridge (type + channel):        " + (types + channels) + " classes");
        System.out.println("  with 4 types and 4 channels:    " + 4 * 4 + " vs " + (4 + 4));
    }
}
```

Output:

```
== Any type x any channel, paired at run time ==
  [email to ops@shop.com] Subject: ALERT payments | error rate 12% - act now
  [sms to +1-555-0100] ALERT payments: error rate 12% - act now
  [push to pixel-7] (Reminder) Stand-up at 09:30
  [email to ops@shop.com] Subject: Daily digest (3) | 3 orders; 1 refund; 0 incidents
  [sms to +1-555-0100] Daily digest (3): 3 orders; 1 refund;...
== Class count ==
  one hierarchy (type x channel): 9 classes
  bridge (type + channel):        6 classes
  with 4 types and 4 channels:    16 vs 8
```

What each part does

- `Channel` — the Implementor. Just `deliver(title, body)` with two plain strings, so every channel can support it.
- `EmailChannel`, `SmsChannel`, `PushChannel` — each knows *only* how to deliver. `SmsChannel` truncates long text; that rule lives inside SMS, not in any notification.
- `Notification` — the Abstraction, holding `protected final Channel channel` (the bridge).

- `AlertNotification` etc. — each builds a title and body and calls `channel.deliver(...)`. No `instanceof` (Java's check of an object's concrete type), no `switch` on channel type.
- Adding `SlackChannel` = one new class; adding `PromoNotification` = one new class. Nothing else changes.

Bridge vs Strategy

Written down, the two are **indistinguishable** — an object keeps a reference to an interface and delegates through it. Interviewers and code reviewers both ask about this, so have an answer ready: the difference is the **reason the reference is there**, not how it looks.

	Bridge	Strategy
What varies	A whole **dimension** of the design	**One algorithm** or policy
When chosen	Usually once, at construction, then it stays	Often swapped repeatedly, even per call
Why it exists	To stop two hierarchies multiplying	To make one behaviour replaceable
When decided	Up front, as a structural decision	Later, when a second algorithm appears
Example	`Notification` → `Channel`	`list.sort(comparator)`

Practical test: picture adding one more implementation. If it is *a new kind of thing the system supports* (another delivery channel), you have a Bridge. If it is *one more way to perform a single step* (another compression algorithm), you have a Strategy. When you truly can't decide, relax — the code is identical and either name will be understood. What does matter is not building either one while an axis has only a single member.

Getting the split wrong

Bridge is quick to write but costly to re-cut once both sides have several classes. Two ways it goes wrong:

- **Low-level details leaking up.** An `if (channel instanceof SmsChannel)` inside `AlertNotification` means the bridge is no longer doing its job — that SMS-specific rule should move into `SmsChannel`.
- **An implementor interface shaped around one implementation.** Add `setEmailReplyTo(...)` to `Channel` and every non-email channel must now implement a method it can only throw from. Keep the low-level interface minimal and generic; that is why `deliver` accepts two plain strings instead of a rich message type.

Trade-offs

Pros	Cons
Class counts add rather than multiply — 3 types and 3 channels need 6 classes, not 9.	**Two hierarchies** to understand instead of one.
Each side extends independently (Open/Closed) — a new channel touches no notification class.	**More indirection** — following one call means reading two files.
Pairing picked at run time — real channel in production, fake in tests, per-environment config.	**Hard to undo** — once both sides have implementations, moving the split line is a large refactor (restructuring code without changing what it does).
Platform code kept apart — only one class knows about SMTP (the protocol for sending email) (Single Responsibility).	**Pointless with one member** — a bridge with a single implementor is just an interface plus ceremony.

When not to use it

- **An axis has only one member.** With a single channel the bridge abstracts nothing; introduce it when the second channel actually shows up, not in advance.
- **The axes are not truly independent.** If some combinations are forbidden (e.g. digests must never be sent by SMS), Bridge happily lets you build them, and you end up adding run-time checks that separate subclasses would have made impossible.
- **You can't state in one sentence which side is the abstraction.** Then the design isn't ready — picking the wrong split is more expensive than living with a few extra classes.
- **The second axis is really a behaviour, not a kind of thing.** Use Strategy.

Common mistakes

- Building the bridge "just in case" with a single implementor.
- Letting the abstraction branch on the concrete implementor type.
- Making the implementor interface mirror one specific implementation.
- Confusing Bridge with Adapter: Adapter is **retrofitted** to make existing things fit; Bridge is **designed in** so the two sides never get fused into one hierarchy.

Real-world examples

- **JDBC**: your code uses `java.sql.Connection` / `Statement`; each database driver supplies the implementation. Application code and drivers evolve separately, which is why the same query code can run on PostgreSQL or MySQL. (JDBC is often cited as both Bridge — two independent hierarchies — and Adapter — each driver adapts a native protocol. Both views are fair.)
- **SLF4J and similar logging APIs**: the logging API you call is one side and the backend that actually writes log lines is the other. (SLF4J calls itself a *facade*; the API/backend split underneath is Bridge-shaped.)
- **Window systems and device/graphics drivers**: GoF's own motivating example is a `Window` abstraction whose `WindowImp` implementation is written once per platform (X Window System, IBM Presentation Manager). OS drawing APIs over per-card graphics drivers follow the same idea.
- **`java.awt` peers** (the textbook Java example): each AWT component held a platform-specific peer object that did the actual native drawing.
- **Notification / payment services in most apps**: a `NotificationService` interface with pluggable providers (SES, Twilio, FCM — Amazon's email service, an SMS provider, Google's push-notification service) is this pattern, whether or not anyone called it Bridge.
- **Dependency-injection containers** (frameworks that create objects and hand each one the dependencies it needs) that bind an interface to a different implementation per environment give you Bridge-style pairing that is configured rather than coded.

In modern languages

- **First-class functions**: if the implementor has one method, it can simply be a function — `Notification(channel = send_email)` is the same design without a `Channel` class.
- **Python / JavaScript**: duck typing means the implementor side needs no declared interface — any object with a `deliver` method will do.
- **Go**: a small interface declared by the *consumer*, with implementations in other packages, is ordinary idiomatic (natural, conventional-style) Go — Bridge without the name.
- **DI containers** (Spring, Guice, .NET): the pairing is wired by configuration per environment.

Other patterns it is confused with

Looks like	Actual difference
Strategy	Same code shape; Bridge separates a permanent structural dimension, Strategy makes one algorithm swappable.
Adapter	The class diagrams can look alike, but the timing is reversed: an Adapter is retrofitted to reconcile two things that already exist; a Bridge is planned before the classes are written.
Abstract Factory	Abstract Factory decides *which family of objects gets created*; Bridge describes *how an object hands off work once it exists*. They work well together — a factory commonly supplies the implementor a bridge receives.
Decorator	A Decorator shares the interface of the object inside it and adds behaviour; in a Bridge the two sides have unrelated interfaces and neither one wraps the other.

In the interview

> **30-second answer:** "Bridge separates an abstraction from its implementation so both can vary independently. If notifications vary by type and by channel, one hierarchy needs type × channel classes — 3 × 3 = 9, and 16 with a fourth of each. I split it: a `Notification` hierarchy holding a reference to a `Channel` interface. Now it is type + channel classes — 6, then 8 — and the pairing is chosen at run time. I wouldn't build it while there is only one channel."

Follow-ups

- *Q: How is it different from Strategy?* — A: The code is the same; the intent differs. Bridge splits a permanent dimension decided up front; Strategy makes one algorithm replaceable, often per call.
- *Q: What is the smell that says you need it?* — A: Class names with two nouns (`SmsDigest`) and duplicated code across them.
- *Q: When would you not use it?* — A: With only one channel today — I would add the bridge when the second one lands. Also when some combinations are invalid.
- *Q: Who decides which implementor a notification gets?* — A: Whoever builds it — often a factory or the DI container, configured per environment.

Self-check

1. **Q:** With 5 notification types and 4 channels, how many classes without and with Bridge? **A:** 20 without, 9 with.
2. **Q:** Which field is "the bridge"? **A:** The `Channel channel` reference held by the `Notification` base class.
3. **Q:** An `AlertNotification` checks `channel instanceof SmsChannel`. What is wrong? **A:** Implementation detail has leaked upward; move that behaviour into `SmsChannel`.
4. **Q:** Is Bridge added before or after the classes exist? **A:** Before — it is an up-front structural decision (Adapter is the after-the-fact one).

TL;DR

- **Two dimensions varying → two hierarchies + one reference.** Multiplication becomes addition.
- Abstraction (what) holds Implementor (how). Keep the implementor small and general.
- Same code as Strategy; different intent. Don't build it for one implementor.

9.3 Composite Pattern

What it is

The **Composite** pattern gives an individual item and a whole nested group of items **one common interface**, so code that uses them never needs to check whether it has a **single object** or an **entire tree**.

Analogy: a company org chart. You can ask any manager "what is your team's total salary?" — a manager answers by asking each report and adding up; an individual contributor just answers with their own salary. The question is the same for both.

Term	Meaning	In our code
Component	The shared interface for leaves and containers	`FileSystemNode`, `MenuComponent`
Leaf	An object with no children; does the work itself	`FileNode`, `MenuItem`
Composite	A container that holds children (typed as Component) and delegates to them	`FolderNode`, `Menu`
Client	Code that works with Components only	`report(...)`

GoF's intent in plain words: compose objects into tree structures that represent **part-whole hierarchies**, and let clients treat individual objects and compositions **uniformly**.

The problem it solves

Plenty of data is nested: folders contain files *and* folders; an invoice contains line items *and* bundles that have their own items; a UI (user interface) panel contains widgets (buttons, text boxes and other screen controls) *and* sub-panels. Ask for a total size, a total price or a rendering of the whole thing, and — without the pattern — each operation (`size`, `print`, `search`, `copy`) opens with the same type test:

```
if (node instanceof Folder f) { /* loop over children, recurse */ }
else if (node instanceof File f) { /* read the size */ }
```

That test gets pasted into every operation. Introduce a third kind of node — say a symbolic link (a file-system entry that points to another file or folder) — and you have to hunt down each copy; overlook one and the new node type quietly gets the wrong answer.

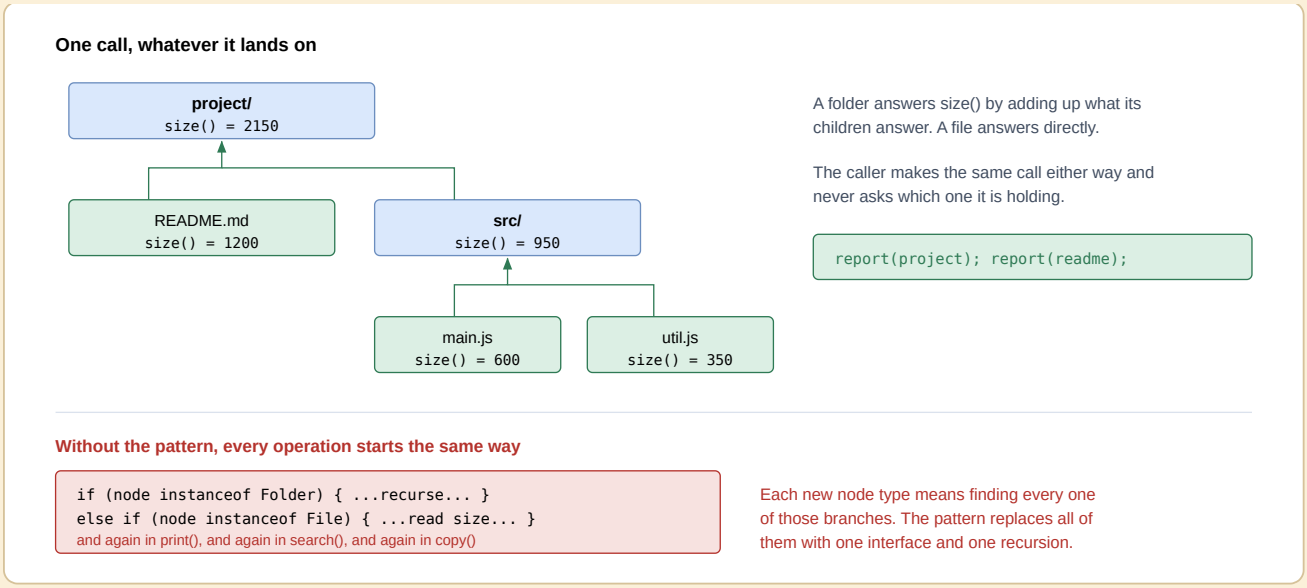


Figure — `size()` on the project folder (2150) is the sum of README.md (1200) and src/ (950), which itself sums main.js and util.js. The caller makes the same call on a tree or a single file (course diagram)

The figure shows the fix: a file answers `size()` with its own byte count, and a folder answers by asking each child and summing. Because a child may be a file or another folder and both respond to the same call, a single piece of recursive code (code that calls itself on the smaller parts) handles any depth. The caller writes `report(project)` or `report(readme)` and never asks which one it holds.

How it works

1. **Define a Component interface** with operations that make sense for *both* leaves and containers (`name()`, `size()`, `print()`).
2. **Leaf** implements them directly (a file returns its own byte count).
3. **Composite** stores a `List<Component>` — the **interface type**, not the leaf type — and implements each operation by **asking each child and combining the results**.
4. Because a child can itself be a composite, the recursion is written **once** and works to any depth.
5. **Clients** hold a `Component` and never type-check.

Structure

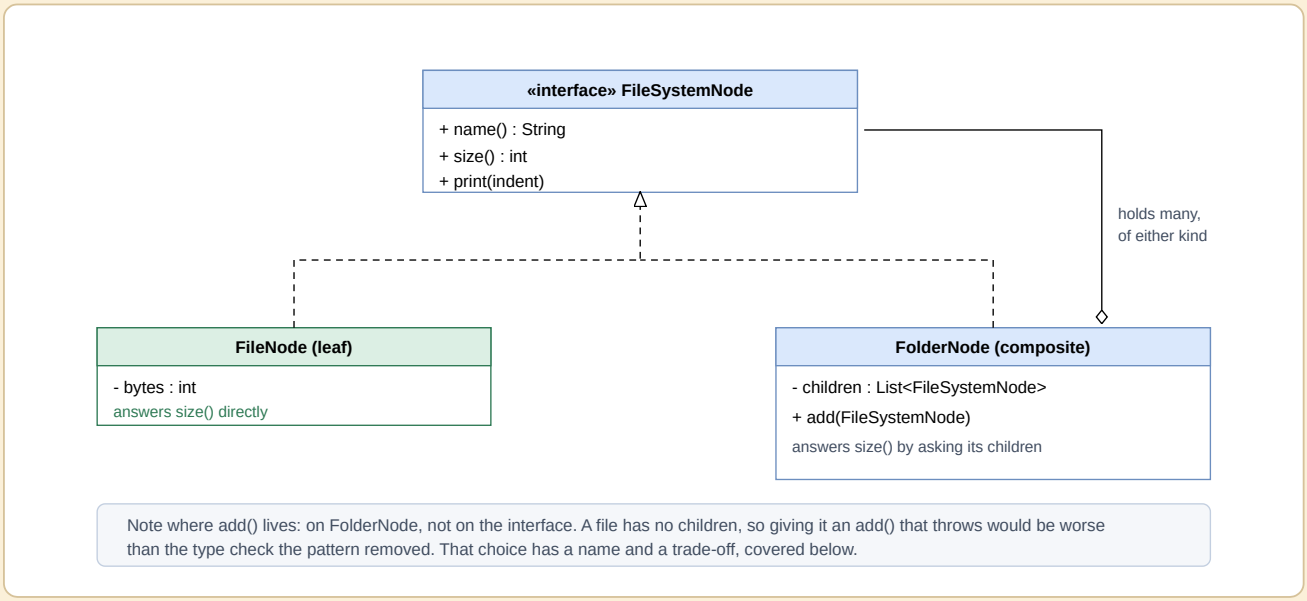
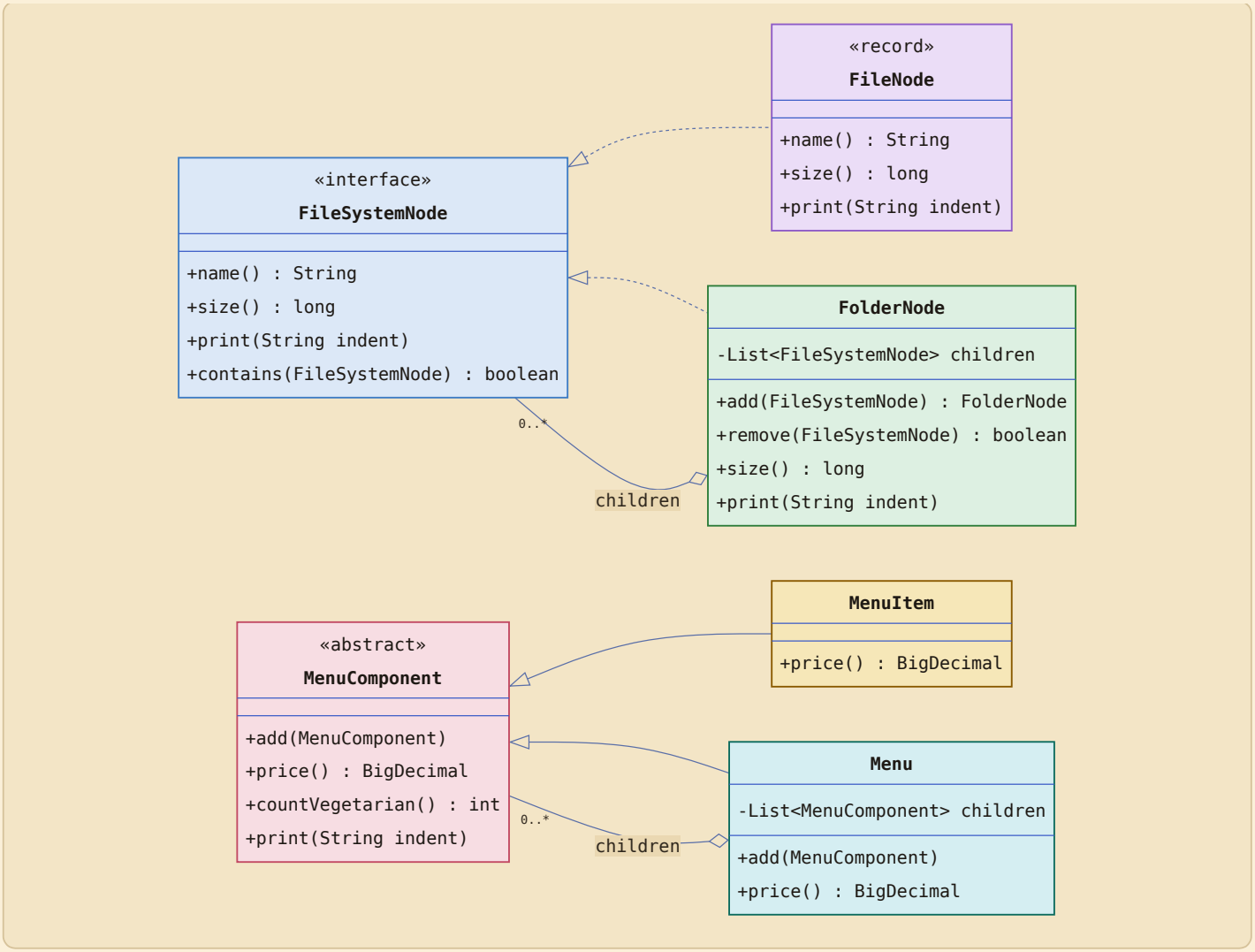


Figure — Composite class diagram: `FileNode` (leaf) and `FolderNode` (composite) implement `FileSystemNode`; the folder holds a list of `FileSystemNode` and has `add()` (course diagram)

Two details in the class diagram above matter: the folder's child list is typed as the **interface** — that is what lets folders hold folders — and `add()` is declared on `FolderNode` rather than on the shared interface. The second is a deliberate trade-off (the "safe" design), covered right after the code.



Java example

```java
// File: code/lld/ch09/composite/FileSystemNode.java
package lld.ch09.composite;

/**
 * COMPONENT (safe version): only operations that make sense for BOTH leaf and composite.
 * add/remove are NOT here - they live on FolderNode only.
 */
interface FileSystemNode {
    String name();
    long size();
    void print(String indent);

    /** True if 'target' is this node or somewhere below it. Used for the cycle check. */
    default boolean contains(FileSystemNode target) { return this == target; }
}
```

```java
// File: code/lld/ch09/composite/FileNode.java
package lld.ch09.composite;

/** LEAF: no children, answers directly. */
record FileNode(String name, long size) implements FileSystemNode {
    @Override
    public void print(String indent) {
        System.out.println(indent + "- " + name + " (" + size + " B)");
    }
}
```

```java
// File: code/lld/ch09/composite/FolderNode.java
package lld.ch09.composite;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** COMPOSITE: holds children typed as the INTERFACE, so folders can hold folders. */
final class FolderNode implements FileSystemNode {
    private final String name;
    private final List<FileSystemNode> children = new ArrayList<>();

    FolderNode(String name) { this.name = name; }

    /** Returns this so trees can be built fluently. Rejects cycles at insert time. */
    FolderNode add(FileSystemNode child) {
        if (child.contains(this)) {
            throw new IllegalArgumentException("cycle: " + child.name() + " already contains " + name);
        }
        children.add(child);
        return this;
    }

    boolean remove(FileSystemNode child) { return children.remove(child); }

    List<FileSystemNode> children() { return Collections.unmodifiableList(children); }

    @Override public String name() { return name; }

    /** The recursion lives in ONE place. */
    @Override
    public long size() {
        long total = 0;
        for (FileSystemNode child : children) total += child.size();
        return total;
    }

    @Override
    public void print(String indent) {
        System.out.println(indent + "+ " + name + "/ (" + size() + " B)");
        for (FileSystemNode child : children) child.print(indent + "   ");
    }

    @Override
    public boolean contains(FileSystemNode target) {
        if (this == target) return true;
        for (FileSystemNode child : children) {
            if (child.contains(target)) return true;
        }
        return false;
    }
}
```

```java
// File: code/lld/ch09/composite/MenuComponent.java
package lld.ch09.composite;

import java.math.BigDecimal;

/**
 * COMPONENT (transparent version): child management is on the shared type,
 * so callers never cast - but a leaf must reject it at run time.
 */
abstract class MenuComponent {
    void add(MenuComponent c) { throw new UnsupportedOperationException(name() + " is a leaf"); }

    abstract String name();
    abstract BigDecimal price();            // total price of everything under this node (money: BigDecimal, never
double)
    abstract int countVegetarian();
    abstract void print(String indent);
}
```

```java
// File: code/lld/ch09/composite/MenuItem.java
package lld.ch09.composite;

import java.math.BigDecimal;

final class MenuItem extends MenuComponent {
    private final String name;
    private final BigDecimal price;
    private final boolean vegetarian;

    MenuItem(String name, String price, boolean vegetarian) {
        this.name = name;
        this.price = new BigDecimal(price);     // from a String, so "3.49" is exactly 3.49
        this.vegetarian = vegetarian;
    }

    @Override String name() { return name; }
    @Override BigDecimal price() { return price; }
    @Override int countVegetarian() { return vegetarian ? 1 : 0; }
    @Override void print(String indent) {
        System.out.println(indent + name + (vegetarian ? " (v)" : "") + " $" + price);
    }
}
```

```java
// File: code/lld/ch09/composite/Menu.java
package lld.ch09.composite;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

final class Menu extends MenuComponent {
    private final String name;
    private final List<MenuComponent> children = new ArrayList<>();

    Menu(String name) { this.name = name; }

    @Override void add(MenuComponent c) { children.add(c); }
    @Override String name() { return name; }
    @Override BigDecimal price() {
        return children.stream().map(MenuComponent::price).reduce(BigDecimal.ZERO, BigDecimal::add);
    }
    @Override int countVegetarian() { return children.stream().mapToInt(MenuComponent::countVegetarian).sum(); }
    @Override void print(String indent) {
        System.out.println(indent + "[" + name + "]");
        children.forEach(c -> c.print(indent + "  "));
    }
}
```

```java
// File: code/lld/ch09/composite/Main.java
package lld.ch09.composite;

public class Main {

    /** Client: one parameter type, no instanceof anywhere. */
    static void report(FileSystemNode node) {
        System.out.println("  " + node.name() + " -> " + node.size() + " bytes");
    }

    public static void main(String[] args) {
        System.out.println("== Safe composite: file system ==");
        FileNode readme = new FileNode("README.md", 1200);
        FolderNode src = new FolderNode("src")
                .add(new FileNode("main.js", 600))
                .add(new FileNode("util.js", 350));
        FolderNode project = new FolderNode("project").add(readme).add(src);

        project.print("  ");
        report(project);   // a whole tree
        report(readme);    // a single file - same call

        try {
            src.add(project);   // project already contains src -> would loop forever
        } catch (IllegalArgumentException e) {
            System.out.println("  rejected: " + e.getMessage());
        }

        System.out.println("== Transparent composite: restaurant menu ==");
        MenuComponent all = new Menu("All menus");
        MenuComponent breakfast = new Menu("Breakfast");
        MenuComponent desserts = new Menu("Desserts");
        breakfast.add(new MenuItem("Pancakes", "3.49", true));
        breakfast.add(new MenuItem("Bacon & eggs", "4.99", false));
        desserts.add(new MenuItem("Apple pie", "2.59", true));
        breakfast.add(desserts);          // a menu inside a menu
        all.add(breakfast);
        all.add(new MenuItem("Coffee", "1.50", true));

        all.print("  ");
        System.out.println("  total $" + all.price() + ", vegetarian items: " + all.countVegetarian());
        try {
            new MenuItem("Tea", "1.00", true).add(new MenuItem("Milk", "0.20", true));
        } catch (UnsupportedOperationException e) {
            System.out.println("  transparent cost: " + e.getMessage());
        }
    }
}
```

Output:

```
== Safe composite: file system ==
  + project/ (2150 B)
     - README.md (1200 B)
     + src/ (950 B)
        - main.js (600 B)
        - util.js (350 B)
  project -> 2150 bytes
  README.md -> 1200 bytes
  rejected: cycle: project already contains src
== Transparent composite: restaurant menu ==
  [All menus]
    [Breakfast]
      Pancakes (v) $3.49
      Bacon & eggs $4.99
      [Desserts]
        Apple pie (v) $2.59
    Coffee (v) $1.50
  total $12.57, vegetarian items: 3
  transparent cost: Tea is a leaf
```

What each part does

- `FileSystemNode` — the Component. Only operations valid for both files and folders.
- `FileNode` (a `record`) — the Leaf: immutable, answers `size()` directly.
- `FolderNode` — the Composite. `size()` is the four-line recursion and the **only** place in the program that knows a tree can nest. `add()` returns `this` so trees build fluently.
- `contains(...)` + the check in `add()` — **cycle protection** at insert time. Trying to put `project` inside its own child `src` is rejected (see output).
- `MenuComponent` / `Menu` / `MenuItem` — the *transparent* variant (the restaurant-menu example popularised by *Head First Design Patterns*): `add` is on the shared abstract class and a leaf throws.
- `report(...)` — the client, called with a whole tree and with a single file.

Transparent vs safe composite

Where do `add` and `remove` belong? GoF named both answers:

	Transparent	Safe
Where `add/remove` live	On the shared Component	Only on the Composite
Uniformity	Full — every node looks the same; no casts (forcing a value to a more specific type)	Partial — builders must hold a `FolderNode`
Type safety	Weaker — `leaf.add(...)` compiles, then **throws at run time**	Strong — the compiler rejects `file.add(...)`
Our code	`MenuComponent` (`Tea is a leaf` in the output)	`FileSystemNode` / `FolderNode`
Choose when	Callers add children to arbitrary nodes without knowing their type (e.g. a UI editor where anything might be a container)	The tree is built in one place and read in many (most cases)

The transparent version swaps a compile-time guarantee for a possible run-time exception. The safe version keeps the read-side interface small, but any code that builds or edits the tree must hold a `FolderNode` — and code that walks a tree and then wants to insert needs a cast or type check after all. Neither is wrong; in review, check that the choice was **made on purpose**. A throwing `add` that nobody intended is the real defect.

Head First Design Patterns makes an interesting point here: the transparent composite knowingly bends the Single Responsibility Principle (the component both manages children and does work) to gain transparency. That is a conscious design trade-off, not an accident.

Two traps worth knowing

- **Cycles.** If a folder is ever placed inside one of its own descendants, `size()` recurses forever (in Java: `StackOverflowError`, thrown when nested calls use up the call stack — the memory that tracks calls in

progress). **Validate when inserting**, not during traversal (walking through every node) — once you are traversing, you are already stuck. Our `add()` does exactly this.

- **Repeated work.** Our `print()` calls `size()` on every level, so the same subtotals are recomputed again and again — about quadratic (work grows with the square of the depth) for a deep chain. For big real trees, **store each subtree's total** and invalidate upward when a child changes (which needs a parent pointer).

Trade-offs

Pros	Cons
Callers never type-check — the same call works on a leaf or a whole tree.	**The shared interface tends to bloat** with operations that only half-fit one of the two kinds.
The recursion lives in one place, reused by each new operation.	**Weaker type safety** (transparent variant) — the compiler allows adding a child to a leaf.
Adding a node type doesn't touch callers (Open/Closed) — e.g. a `SymlinkNode`.	**Deep trees mean deep call stacks**, and a cycle shows up as endless recursion instead of a clear error.
Nesting comes for free — a composite that stores the interface can store other composites.	**Totals get recomputed** on every call unless you cache them.

When not to use it

- **Nothing nests.** A flat list gains nothing from a shared leaf/container interface.
- **Leaves and containers share little.** A forced common interface fills up with methods that throw, which is worse than the type test it replaced.
- **Deep tree on a hot path.** (A hot path is code that runs so often that its speed matters.) Each level costs a stack frame and likely CPU cache misses (fetching data from slow main memory instead of the processor's fast cache); precomputed totals, or a loop that manages its own work-list instead of recursing, can be faster.
- **The tree is modified while being traversed.** Concurrent changes (made at the same time by another thread) during recursion are dangerous, and Composite offers no protection.

Common mistakes

- Typing the child list as the **leaf** type (`List<FileNode>`) — then folders cannot hold folders.
- Returning the internal mutable child list — callers can bypass `add()` and its cycle check. We return `Collections.unmodifiableList(children)`.
- Adding `add()` to the interface by accident (transparent without meaning to).
- No cycle check.
- Using Composite when a single recursive function with one type check would do — the pattern pays off when **operations multiply**.

Real-world examples

- **The HTML DOM** (Document Object Model — the browser's in-memory tree of page elements) — an element holds child nodes, each possibly an element with children; `textContent` walks the whole subtree.
- **UI toolkits** — in Swing/AWT, `java.awt.Container` *is a* `Component` and holds `Component`s, which is why panels nest without limit. React (a JavaScript UI library) component trees and Android's `ViewGroup` (a `View` that holds `View`s) follow the same idea.
- **File systems** — the example used here.
- **Abstract syntax trees** (the tree a compiler builds from source code) in compilers — an expression node holds sub-expressions; evaluating any node evaluates its subtree.
- `java.io.File` represents both files and directories with one class — a fair illustration of the transparent cost: `listFiles()` returns `null` when called on a regular file.
- **JUnit 3 test suites** (JUnit is the standard Java testing framework; a suite is a named group of tests) — `TestSuite` implements `Test` and holds `Test`s (cases or other suites); running a suite runs everything beneath it. (JUnit 5 keeps the idea as a tree of test descriptors.)
- **Spring's** `CompositeCacheManager` delegates to a list of `CacheManager`s while itself being a `CacheManager`.
- **Invoices / bundles** — an e-commerce "bundle" priced as the sum of its items, which may themselves be bundles.

In modern languages

- **Python / JavaScript**: duck typing — leaf and container just need the same method names; a composite is often a class with a `children` list.
- **Sealed types + pattern matching** (Java 21, Kotlin, Scala, Rust) — a sealed type lists exactly which subtypes may exist, and pattern matching lets a `switch` branch on which subtype it has: a `sealed interface Node permits File, Folder` plus an exhaustive `switch` is a real alternative. The compiler then reports any node type a `switch` forgets — a guarantee Composite does not give. Prefer it when the set of node types is closed.
- **Functional languages**: a recursive algebraic data type (a type defined as "one of these shapes", where a shape may contain the type itself) is Composite with language support; a `fold` (a function that combines all parts of a structure into one result) over it is the recursion written once.
- **Java streams** (a pipeline API for processing collections): `flatMap` (turns each element into several and flattens the result) makes walking a composite read nicely once the structure exists.

Other patterns it is confused with

Looks like	Actual difference
Decorator	Both hold objects of their own interface. A Decorator wraps exactly **one** and adds behaviour; a Composite holds **many** and combines their results.
Iterator	Composite gives the tree a uniform interface; Iterator gives a way to walk it without exposing its structure. Often used together.
Visitor	Composite makes the structure uniform; Visitor adds new operations over it without editing node classes — the usual next step when operations keep multiplying.
A plain recursive function	If one function with one type check does the job, you don't need the pattern. Composite pays off when **many** operations would each repeat that branch.

In the interview

30-second answer: "Composite lets a client treat a single object and a tree of objects the same way. Leaf and container share one interface; the container stores children *typed as that interface* — which is what allows arbitrary nesting — and implements each operation by delegating to its children and combining the results. Its value is what it deletes: the `instanceof` branch in every operation. I choose between the transparent and safe variants depending on whether callers need `add` on arbitrary nodes, I reject cycles on insert, and I cache subtree totals if the tree is large."

Follow-ups

- *Q: Transparent or safe?* — A: Safe if the tree is built in one place and read in many; transparent if callers add children to arbitrary nodes and casts would spread everywhere.
- *Q: What about cycles?* — A: Check on insert: reject `add(child)` if the child already contains the parent.
- *Q: Performance on a big tree?* — A: Cache subtree totals, invalidate upward on change; or use an iterative walk to avoid deep recursion.
- *Q: Composite vs Decorator?* — A: Both hold objects of their own interface. Decorator holds **one** and adds behaviour; Composite holds **many** and combines their results.
- *Q: How do you add a new operation without touching every node?* — A: Use Visitor over the composite structure.

Self-check

1. **Q:** Why must the child list be `List<FileSystemNode>`, not `List<FileNode>`? **A:** So a folder can contain other folders — arbitrary nesting.
2. **Q:** In the transparent variant, what happens when you call `add` on a leaf? **A:** It compiles but throws at run time (`UnsupportedOperationException`).
3. **Q:** Where should a cycle be detected? **A:** At insertion time.
4. **Q:** Name a JDK class that is both a component and a container of components. **A:** `java.awt.Container`.

TL;DR

- **Part-whole trees, one interface.** Leaf answers directly; composite asks children and combines.

- Child list holds the **interface**. Recursion lives in one place.
- **Safe** (add only on composite, compile-time safe) vs **transparent** (add everywhere, run-time throw). Guard against **cycles**; cache big totals.

9.4 Decorator Pattern

What it is

The **Decorator** pattern extends what **a single object** does, **while the program runs**, by enclosing it in a wrapper that implements **the same interface**. The original class is never edited, and you don't need a separate subclass for each mix of extras.

Analogy: ordering coffee. Start with an espresso, add milk, add mocha, add whipped cream. Each topping wraps the drink and adds to its price and description; the result is still "a drink".

Term	Meaning	In our code
Component	The shared interface	`UserRepository`, `Beverage`, `InputStream`
Concrete Component	The real object that does the core work	`DatabaseUserRepository`, `Espresso`
Decorator (base)	Implements the Component, holds a Component, forwards everything	`RepositoryDecorator`, `java.io.FilterInputStream`
Concrete Decorator	Adds one behaviour around the forwarded call	`LoggingRepository`, `CachingRepository`, `Mocha`, `UpperCaseInputStream`

GoF's intent: *attach additional responsibilities to an object dynamically — a flexible alternative to subclassing for extending functionality.* GoF also calls it **Wrapper** (the same nickname as Adapter).

The key property: **a decorator IS-A Component and HAS-A Component.** Since it has the very type it wraps, a decorator can sit around the real object, around another decorator, or around a test double (any stand-in object used in tests — fake, stub or mock), and code that only sees the interface cannot tell which.

The problem it solves

First a repository (an object that loads and saves data, hiding the database behind methods) needs logging, then caching, later retries and metrics (counters and timings the app records about itself). Each caller wants its own mix of these, and sometimes in its own order.

- **Subclassing can't keep up.** A `LoggingCachingRetryingUserRepository` serves one mix only. Four optional concerns give 2^4 = **16** possible mixes, and the class count doubles with every new concern — a class explosion.
- **One class with on/off flags** holds up until a caller wants "everything except the cache". One boolean turns into four, and you end with `new Repo(true, false, true, true)`, which nobody can read.

Decorator gives **each concern its own class** with the component's interface. Four concerns cost **four classes**, and each caller assembles exactly the stack it needs — by wrapping at run time rather than inheriting at compile time. That is the Open/Closed Principle at work: new behaviour arrives as new classes, and existing classes stay untouched.

How it works

1. Start from a **Component interface** (`UserRepository`).
2. Write the **concrete component** that does the real work (`DatabaseUserRepository`).
3. Write an **abstract base decorator** that implements the interface, holds an `inner` of the **interface type**, and **forwards every method** by default.
4. Each **concrete decorator** extends the base and overrides only the methods it changes, doing something **before and/or after** calling `inner`.
5. **Compose by wrapping**: `new Logging(new Caching(new Database()))`. The outermost object is what the caller holds.

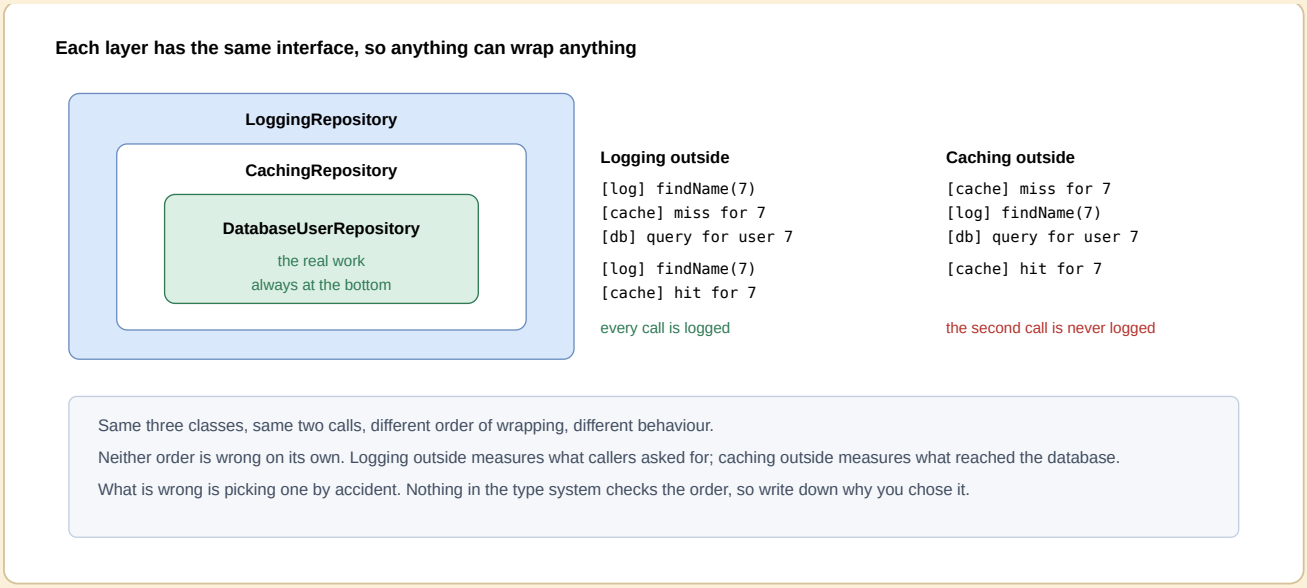


Figure — The same three classes stacked in two orders. Logging outside caching logs every call; caching outside logging means the second call (a cache hit) is never logged (course diagram)

Structure

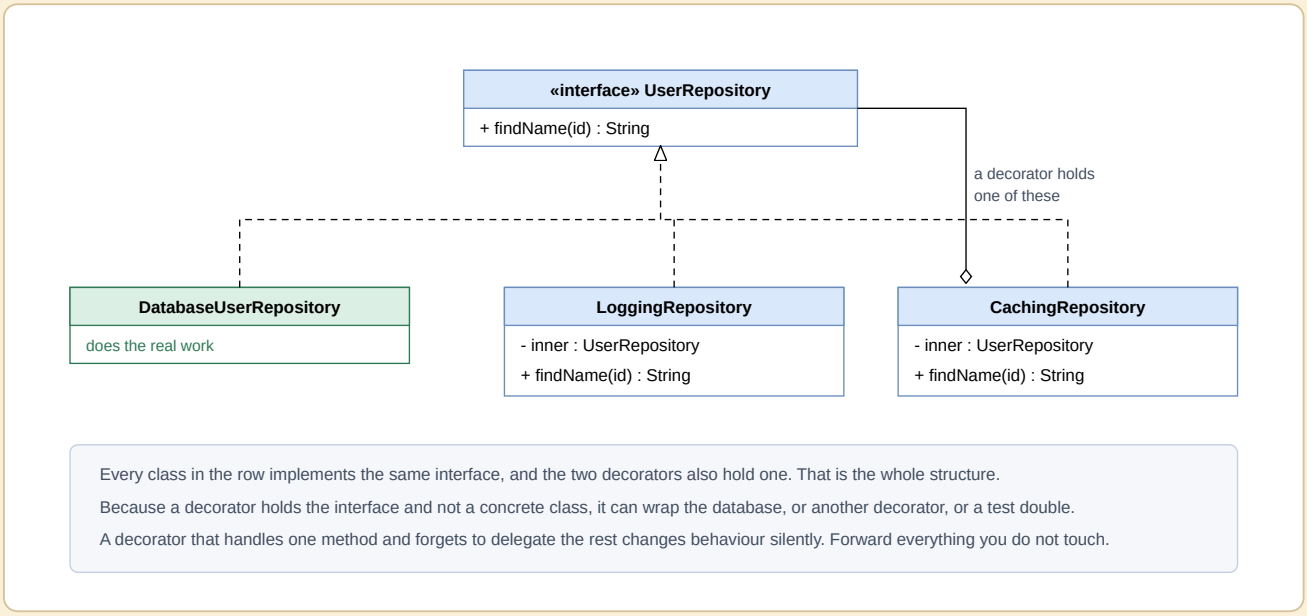
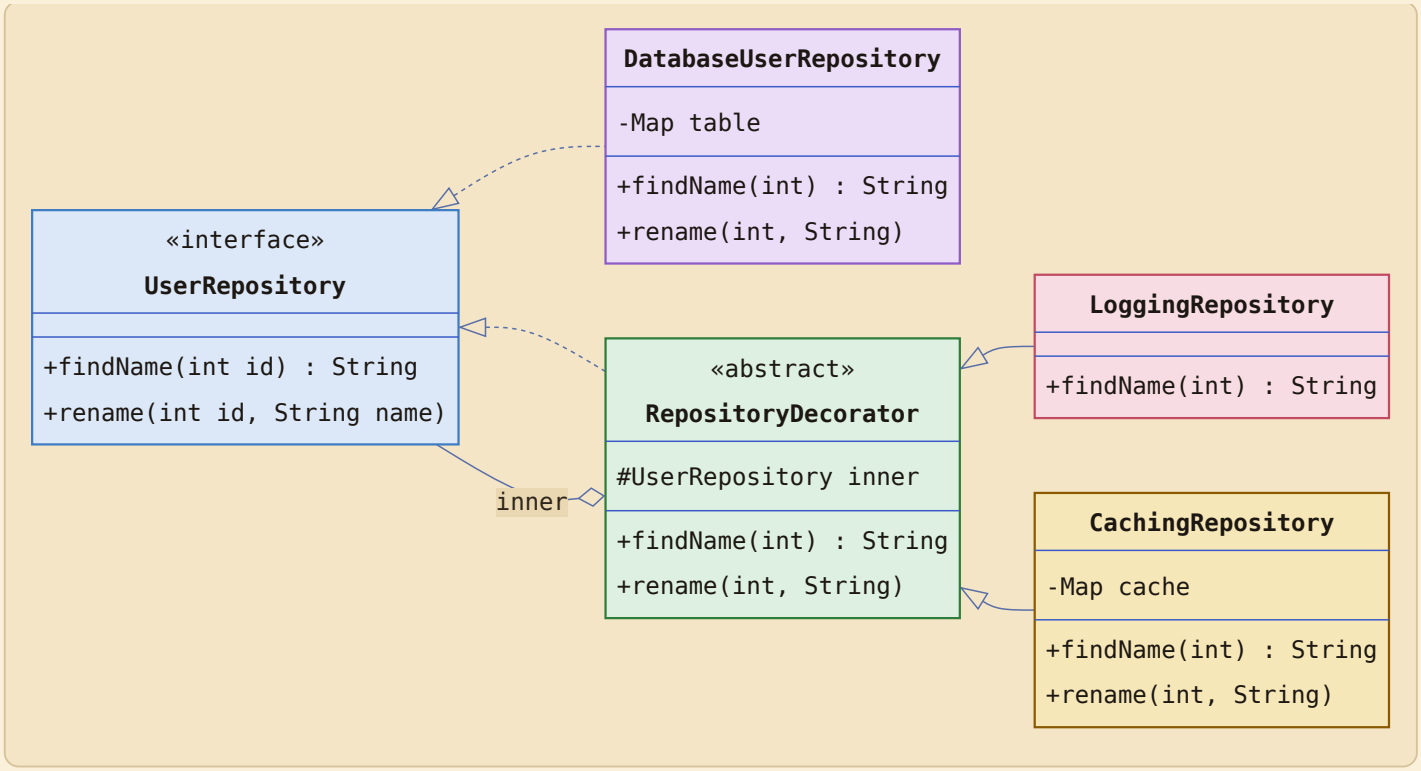


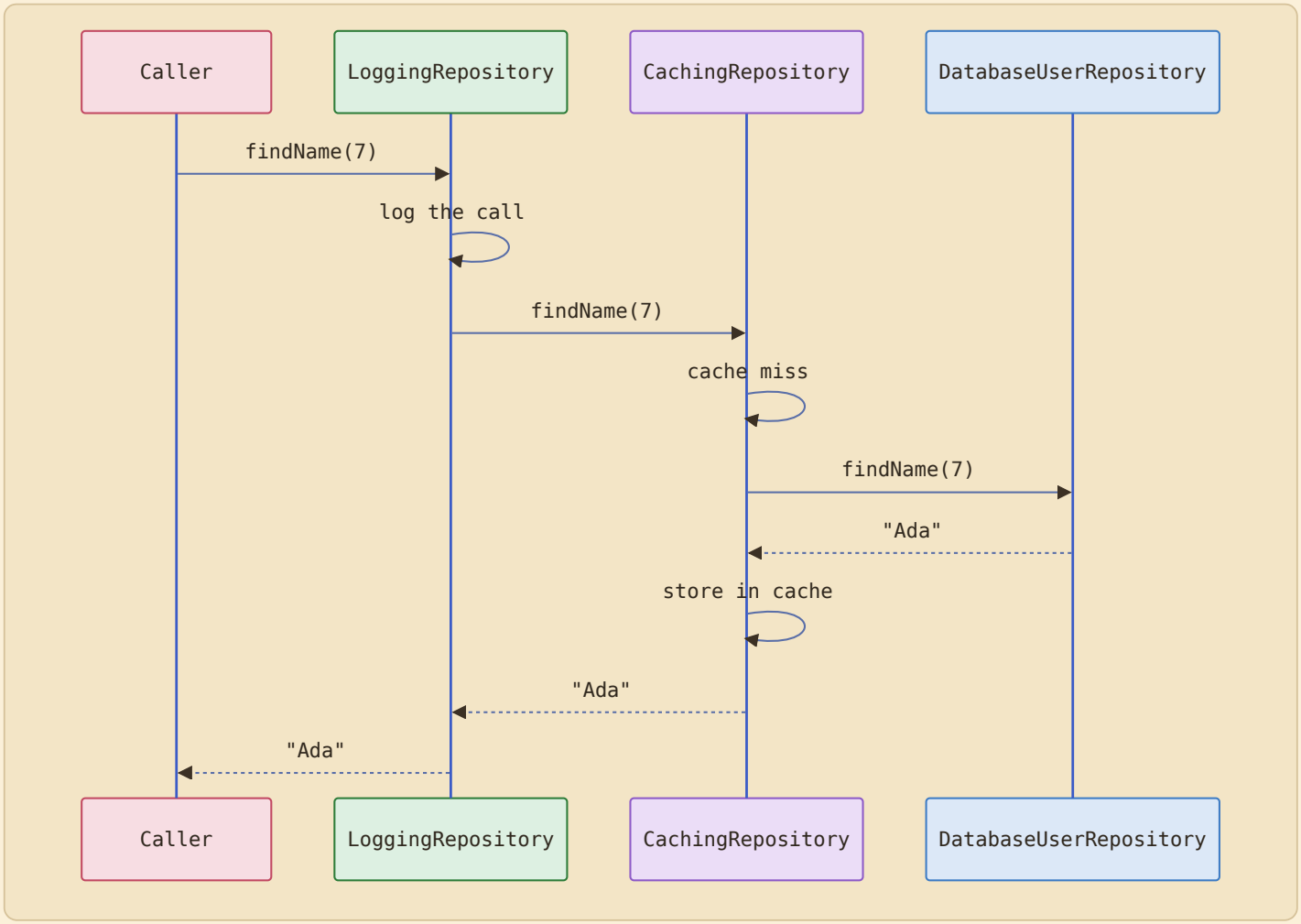
Figure — Decorator class diagram: `DatabaseUserRepository`, `LoggingRepository` and `CachingRepository` all implement `UserRepository`; each decorator holds an `inner` UserRepository (course diagram)

In the diagram above, every class implements `UserRepository` — the one property everything else depends on. `DatabaseUserRepository` always sits at the bottom and wraps nothing. Because each decorator's `inner` field is typed as the interface, decorators can wrap each other in any combination and any order.

The implementation detail that bites in practice is **delegation**: a decorator must forward every method it is not deliberately changing. One that overrides `findName` but silently swallows `rename` changes behaviour in a way that is very hard to spot. In Java the usual guard is an abstract base decorator that forwards everything by default (our `RepositoryDecorator`).



A decorated call, logging outside caching:



Java example

```java
// File: code/lld/ch09/decorator/UserRepository.java
package lld.ch09.decorator;

/** COMPONENT interface: every box in the stack implements it. */
interface UserRepository {
    String findName(int id);
    void rename(int id, String name);
}
```

```java
// File: code/lld/ch09/decorator/DatabaseUserRepository.java
package lld.ch09.decorator;

import java.util.HashMap;
import java.util.Map;

/** CONCRETE COMPONENT: the real work, always at the bottom of the stack. */
final class DatabaseUserRepository implements UserRepository {
    private final Map<Integer, String> table = new HashMap<>(Map.of(7, "Ada", 8, "Linus"));

    @Override
    public String findName(int id) {
        System.out.println("    [db]    query for user " + id);
        return table.get(id);
    }

    @Override
    public void rename(int id, String name) {
        System.out.println("    [db]    update user " + id);
        table.put(id, name);
    }
}
```

```java
// File: code/lld/ch09/decorator/RepositoryDecorator.java
package lld.ch09.decorator;

/**
 * BASE DECORATOR: holds the INTERFACE (not a concrete class) and forwards everything.
 * Subclasses override only what they change, so no method is ever forgotten.
 */
abstract class RepositoryDecorator implements UserRepository {
    protected final UserRepository inner;

    protected RepositoryDecorator(UserRepository inner) { this.inner = inner; }

    @Override public String findName(int id)          { return inner.findName(id); }
    @Override public void rename(int id, String name) { inner.rename(id, name); }
}
```

```java
// File: code/lld/ch09/decorator/LoggingRepository.java
package lld.ch09.decorator;

/** CONCRETE DECORATOR 1 */
final class LoggingRepository extends RepositoryDecorator {
    LoggingRepository(UserRepository inner) { super(inner); }

    @Override
    public String findName(int id) {
        System.out.println("    [log]   findName(" + id + ")");
        return super.findName(id);
    }
}
```

```java
// File: code/lld/ch09/decorator/CachingRepository.java
package lld.ch09.decorator;

import java.util.HashMap;
import java.util.Map;

/** CONCRETE DECORATOR 2 - note it must also invalidate on writes. */
final class CachingRepository extends RepositoryDecorator {
    private final Map<Integer, String> cache = new HashMap<>();

    CachingRepository(UserRepository inner) { super(inner); }

    @Override
    public String findName(int id) {
        if (cache.containsKey(id)) {
            System.out.println("    [cache] hit for " + id);
            return cache.get(id);
        }
        System.out.println("    [cache] miss for " + id);
        String name = super.findName(id);
        cache.put(id, name);
        return name;
    }

    @Override
    public void rename(int id, String name) {
        cache.remove(id);
        super.rename(id, name);
    }
}
```

```java
// File: code/lld/ch09/decorator/Beverage.java
package lld.ch09.decorator;

/** A second, classic decorator example: a coffee order priced by stacking add-ons.
 *  Prices are whole cents (int), so no floating-point money anywhere. */
interface Beverage {
    String description();
    int costCents();
}
```

```java
// File: code/lld/ch09/decorator/Espresso.java
package lld.ch09.decorator;

record Espresso() implements Beverage {
    public String description() { return "Espresso"; }
    public int costCents() { return 199; }
}
```

```java
// File: code/lld/ch09/decorator/HouseBlend.java
package lld.ch09.decorator;

record HouseBlend() implements Beverage {
    public String description() { return "House Blend"; }
    public int costCents() { return 89; }
}
```

```java
// File: code/lld/ch09/decorator/Milk.java
package lld.ch09.decorator;

/** Add-ons are decorators: they ARE a Beverage and HAVE a Beverage. */
record Milk(Beverage inner) implements Beverage {
    public String description() { return inner.description() + ", Milk"; }
    public int costCents() { return inner.costCents() + 10; }
}
```

```java
// File: code/lld/ch09/decorator/Mocha.java
package lld.ch09.decorator;

record Mocha(Beverage inner) implements Beverage {
    public String description() { return inner.description() + ", Mocha"; }
    public int costCents() { return inner.costCents() + 20; }
}
```

```java
// File: code/lld/ch09/decorator/Whip.java
package lld.ch09.decorator;

record Whip(Beverage inner) implements Beverage {
    public String description() { return inner.description() + ", Whip"; }
    public int costCents() { return inner.costCents() + 10; }
}
```

```java
// File: code/lld/ch09/decorator/UpperCaseInputStream.java
package lld.ch09.decorator;

import java.io.FilterInputStream;
import java.io.IOException;
import java.io.InputStream;

/**
 * Our own java.io decorator. FilterInputStream is the JDK's "base decorator":
 * it holds an InputStream and forwards every call to it.
 * Works byte-by-byte, so it is only correct for ASCII text.
 */
final class UpperCaseInputStream extends FilterInputStream {
    UpperCaseInputStream(InputStream in) { super(in); }

    @Override
    public int read() throws IOException {
        int b = super.read();
        return b == -1 ? -1 : Character.toUpperCase((char) b);
    }

    @Override
    public int read(byte[] buf, int off, int len) throws IOException {
        int n = super.read(buf, off, len);
        for (int i = off; i < off + Math.max(n, 0); i++) {
            buf[i] = (byte) Character.toUpperCase((char) buf[i]);
        }
        return n;
    }
}
```

```java
// File: code/lld/ch09/decorator/Main.java
package lld.ch09.decorator;

import java.io.BufferedInputStream;
import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.math.BigDecimal;
import java.nio.charset.StandardCharsets;

public class Main {

    static void lookUpTwice(String label, UserRepository repo) {
        System.out.println("  " + label);
        repo.findName(7);
        repo.findName(7);
    }

    public static void main(String[] args) throws IOException {
        System.out.println("== Same layers, different order ==");
        lookUpTwice("Logging outside caching (logs every caller request):",
                new LoggingRepository(new CachingRepository(new DatabaseUserRepository())));
        lookUpTwice("Caching outside logging (logs only what reaches the db):",
                new CachingRepository(new LoggingRepository(new DatabaseUserRepository())));

        System.out.println("== Write goes through every layer and invalidates the cache ==");
        UserRepository repo = new LoggingRepository(new CachingRepository(new DatabaseUserRepository()));
        repo.findName(8);
        repo.rename(8, "Linus T.");
        System.out.println("  now: " + repo.findName(8));

        System.out.println("== Coffee: stack add-ons at run time ==");
        Beverage order = new Whip(new Mocha(new Mocha(new Espresso())));
        System.out.println("  " + order.description() + " = $" + BigDecimal.valueOf(order.costCents(), 2));
        Beverage latte = new Milk(new HouseBlend());
        System.out.println("  " + latte.description() + " = $" + BigDecimal.valueOf(latte.costCents(), 2));

        System.out.println("== java.io is built from decorators ==");
        InputStream in = new UpperCaseInputStream(
                new BufferedInputStream(
                        new ByteArrayInputStream("decorators wrap streams".getBytes(StandardCharsets.US_ASCII))));
        try (in) {
            System.out.println("  " + new String(in.readAllBytes(), StandardCharsets.US_ASCII));
        }
    }
}
```

Output:

```
== Same layers, different order ==
  Logging outside caching (logs every caller request):
    [log]   findName(7)
    [cache] miss for 7
    [db]    query for user 7
    [log]   findName(7)
    [cache] hit for 7
  Caching outside logging (logs only what reaches the db):
    [cache] miss for 7
    [log]   findName(7)
    [db]    query for user 7
    [cache] hit for 7
== Write goes through every layer and invalidates the cache ==
    [log]   findName(8)
    [cache] miss for 8
    [db]    query for user 8
    [db]    update user 8
    [log]   findName(8)
    [cache] miss for 8
    [db]    query for user 8
  now: Linus T.
== Coffee: stack add-ons at run time ==
  Espresso, Mocha, Mocha, Whip = $2.49
  House Blend, Milk = $0.99
== java.io is built from decorators ==
  DECORATORS WRAP STREAMS
```

What each part does

- `RepositoryDecorator` — the **abstract base decorator**. It forwards *every* method, so a concrete decorator that only cares about `findName` cannot accidentally drop `rename`. This is the standard Java defence against forgotten delegation.
- `LoggingRepository` — prints, then delegates.
- `CachingRepository` — answers from its map on a hit (the value is already cached; a *miss* means it is not), otherwise delegates and stores. (More on caches in System Design Fundamentals.) It also overrides `rename` to **invalidate** (throw away the cached copy that is now out of date) — the third block of output shows the next read going to the database again.
- `Beverage` records — the classic coffee example (from *Head First Design Patterns*, rewritten): `new Whip(new Mocha(new Mocha(new Espresso())))` costs 1.99 + 0.20 + 0.20 + 0.10 = **$2.49**. Double mocha is just wrapping twice — impossible with one subclass per combination.
- `UpperCaseInputStream extends FilterInputStream` — our own `java.io` decorator. `FilterInputStream` is the JDK's base decorator: it holds an `InputStream` and forwards to it.

Layer order changes behaviour

Compare the two stacks in the output — they behave differently:

- **Logger on the outside, cache inside**: every request is logged, even those the cache serves → the log records *what callers requested*.
- **Cache on the outside, logger inside**: only cache misses are logged → the log records *what hit the database*, and the repeat request is absent.

Both are valid; they answer different questions. The mistake is picking one **without meaning to** and then reading the log as if it answered the other question. The same choice comes up wherever layers are stacked:

Pair	Outside	Inside
Retry vs cache	Repeats the entire lookup	Repeats only the database call, and a retry that succeeds populates the cache. Rule of thumb: put retry **next to the thing that fails**.
Metrics vs cache	Times what the caller experiences	Times the database alone — the number an on-call engineer (whoever is currently responsible for answering alerts) is paged on. Many teams record both.
Authorization (checking the caller is allowed) vs cache	**Correct** — every request is checked	**Security bug** — a cached answer is returned without the check ever running

The type checker accepts every ordering. **Record the reason for the order in a comment where the stack is built**, or the next person adding a layer will guess.

Two related traps

- **Stateful decorators that assume one wrap.** (Stateful means it keeps data between calls.) Apply a counting decorator twice to the same object and every call is counted twice.
- **Lost identity.** The wrapper is a different object from what it wraps, so `==` (same object?), `equals` (equal value?) and `instanceof` give answers callers don't expect. Code that digs through layers to "reach the real object" is working against the pattern.

Decorator vs inheritance

	Subclassing	Decorator
When behaviour is chosen	Compile time	Run time
How many at once	One subclass	Any number of layers, any order
Classes for N optional features	Up to 2^N	N
Works on `final` classes / classes you don't own	No	Yes (only needs the interface)
Can reach private state	Protected members, yes	No — only the public interface

Java I/O streams: the decorator you already use

`java.io` is built almost entirely from decorators:

```
new BufferedReader(new InputStreamReader(new FileInputStream("data.txt"), UTF_8))
```

- `FileInputStream` — concrete component (reads bytes from a file).
- `InputStreamReader` — strictly an **adapter** (bytes → characters), not a decorator.
- `BufferedReader` — a **decorator** on `Reader` (adds buffering — reading big chunks into memory to serve many small reads — and `readLine()`).

On the byte side, `FilterInputStream` is the abstract decorator, and `BufferedInputStream`, `DataInputStream`, `PushbackInputStream`, `java.util.zip.GZIPInputStream` (via `InflaterInputStream`) and `java.security.DigestInputStream` are concrete decorators. *Head First Design Patterns* notes the downside: the design produces many small classes, which can overwhelm newcomers until they see the pattern.

Trade-offs

Pros	Cons
Any mix without a class per mix — 4 concerns need 4 classes, not 16.	**Deeper stack traces** — the real work is buried under every layer.
Concerns tested in isolation — logging without a database, caching without a logger (Single Responsibility).	**Identity surprises** — `equals` / `instanceof` on the wrapper don't match the wrapped object.
Assembled at run time — different per caller, environment or feature flag (an on/off switch for a feature).	**Ordering matters for correctness**, and no compiler checks it.
The wrapped class is never edited (Open/Closed) — works even on classes you don't own.	**Over-layering** — eight wrappers deep, nobody knows which one changed a value.

When not to use it

- **Exactly one concern, forever.** Just put it in the class; a decorator for one behaviour is a whole class wrapped around a single `if`.
- **The concern needs private state.** A decorator can only use the public interface, so it is the wrong tool.
- **The framework already offers it** — middleware and interceptors (code a framework runs around every request or call), Spring AOP aspects (aspect-oriented programming: extra code applied automatically around chosen methods) are decorators with tooling. Don't hand-write ten wrappers beside them.

- **The stack would get deep.** Every extra layer is another frame in each stack trace, and with many layers "which one changed this value?" becomes a long debugging session.

Common mistakes

- Holding the **concrete class** instead of the interface (`private DatabaseUserRepository inner`) — then decorators cannot stack.
- **Forgetting to forward** some methods (use an abstract base decorator).
- A caching decorator that **doesn't invalidate on writes** (ours does in `rename`).
- Putting authorization **inside** a cache.
- Relying on `instanceof` of the outer object.

Real-world examples

- **`java.io` streams** — see above.
- **`java.util.Collections.unmodifiableList/synchronizedList/checkedList`** — each returns a wrapper with the same interface and one changed behaviour (read-only, synchronized, type-checked). Some authors call `unmodifiableList` a protection **proxy**; both labels are defensible.
- **`javax.servlet.http.HttpServletRequestWrapper`** (Jakarta: `jakarta.servlet...`) — a base decorator you subclass to change selected request methods in a filter (code that runs before and after each web request in a Java servlet — i.e. web — application).
- **HTTP middleware** (HTTP is the web's request/response protocol) — the web frameworks Express, Django, ASP.NET and Go's `http.Handler` wrap a handler in layers (logging, auth, compression, rate limiting — capping how many requests a client may make per time window). Middleware is this pattern with framework support.
- **Resilience4j (Java) and Polly (.NET)** — compose retry, circuit breaker (stops calling a failing service for a while), timeout and bulkhead (separate, limited resources so one slow dependency cannot use them all) as stacked wrappers, e.g. `Decorators.ofSupplier(...).withCircuitBreaker(...).withRetry(...).withBulkhead(...).decorate()`; both document the ordering carefully because it changes behaviour.
- **Python's `@functools.lru_cache`** (and `@retry` from libraries such as tenacity) — function decorators (LRU = least recently used: a full cache drops the entry that has gone unused longest): the same idea applied to functions, and the reason Python uses the name.

In modern languages

- **Java**: abstract forwarding base class (as here), or a dynamic proxy (a class generated at run time; see 9.7) that generates the forwarding.
- **Python**: function decorators (`@lru_cache` and friends) are this pattern with syntax; for objects, `__getattr__` forwarding (and `functools.wraps` for functions) avoids hand-writing every delegating method.
- **JavaScript / TypeScript**: a higher-order function (one that takes or returns functions) that returns a wrapped function; a `Proxy` object can forward every method automatically.
- **Go**: a struct (Go's plain record-like type) that embeds the interface value gets delegation for free — why Go middleware reads so cleanly.
- **C#**: `DelegatingHandler` in `HttpClient` is a framework-provided decorator chain.

In the interview

> **30-second answer:** "Decorator adds responsibilities to an object at run time by wrapping it in another object with the same interface. Without it, four optional concerns mean sixteen subclasses; with it, each concern — logging, caching, retries — is one class and I compose the stack I need. The decorator holds the interface, not the concrete class, which is what lets layers stack. And order matters: retry goes closest to what fails, and authorization must sit outside a cache or a cache hit skips the check."

Follow-ups

- *Q: Decorator vs Proxy?* — A: Same structure; different intent. Decorator **adds behaviour** and normally forwards the call; the client hands it the object to wrap and often stacks several. Proxy **controls access** — it may defer, cache or refuse — often creates or locates the real object itself, and is usually handed to a client that doesn't know it isn't the real thing. The boundary is genuinely fuzzy (a caching wrapper is fairly called either), and saying so is a better answer than pretending it is sharp.

- *Q: Decorator vs Adapter?* — A: Decorator keeps the interface and changes behaviour; Adapter changes the interface and keeps behaviour.
- *Q: Decorator vs Chain of Responsibility?* — A: Both are chains of objects. In Decorator every layer normally does its extra work and passes the call on to the core (a caching layer answering a hit is the usual exception). In Chain of Responsibility, typically one handler handles the request and the rest never see it.
- *Q: Decorator vs Composite?* — A: Decorator wraps exactly one child and adds behaviour; Composite holds many children and aggregates.
- *Q: Give a JDK example.* — A: `new BufferedInputStream(new FileInputStream(f))`; `FilterInputStream` is the base decorator.

Self-check

1. **Q:** How many classes for 5 optional features with decorators vs one subclass per combination? **A:** 5 vs up to 32.
2. **Q:** Why does `RepositoryDecorator` forward every method? **A:** So concrete decorators cannot silently drop methods they don't override.
3. **Q:** Why is "authorization inside cache" a bug? **A:** A cache hit returns data without ever running the authorization check.
4. **Q:** Is `InputStreamReader` a decorator? **A:** No — it changes the interface (bytes to chars), so it is an adapter. `BufferedReader` is the decorator.

TL;DR

> - **Wrap to add behaviour, keep the interface.** IS-A and HAS-A the same type.
> - N features = N classes, stacked at run time, in any order — and **order matters**.
> - Java I/O, `Collections.synchronizedList`, middleware, Resilience4j are all decorators. A decorator normally forwards every call; one that short-circuits (a cache hit) is drifting towards Proxy.

9.5 Facade Pattern

What it is

The **Facade** pattern gives a complicated subsystem **one simple front door**, so that most callers never deal with the machinery behind it. GoF's intent: *provide a unified interface to a set of interfaces in a subsystem; the facade defines a higher-level interface that makes the subsystem easier to use.*

Analogy: a hotel concierge. You say "book me dinner and a taxi"; the concierge calls the restaurant, the taxi company and the front desk in the right order. You can still phone the restaurant yourself if you want something unusual.

Term	Meaning	In our code
Facade	Knows which subsystem classes to call, in what order, and how to clean up	`OrderFacade`
Subsystem classes	Do the real work; unaware of the facade	`InventoryService`, `PaymentService`, `ShippingService`, `EmailService`
Client	Uses the facade instead of the subsystem	`Main` / a checkout page

The problem it solves

One useful action needs several objects, called in the right order. To place an order you must **reserve stock → charge the card → create a shipment → send a confirmation**. Every place in the code that places an order repeats those four calls — and each copy must also know what to do if the card is declined **after** the stock has been reserved.

That recovery step is where bugs live. Copying the success path is easy; copying the undo correctly is not. A caller that forgets to release the reservation leaves units locked for an order that was never placed.

Nothing is wrong with the subsystem itself — it is **flexible**, and flexibility means many knobs. The trouble is that nearly every caller wants the same standard sequence and writes it out yet again.

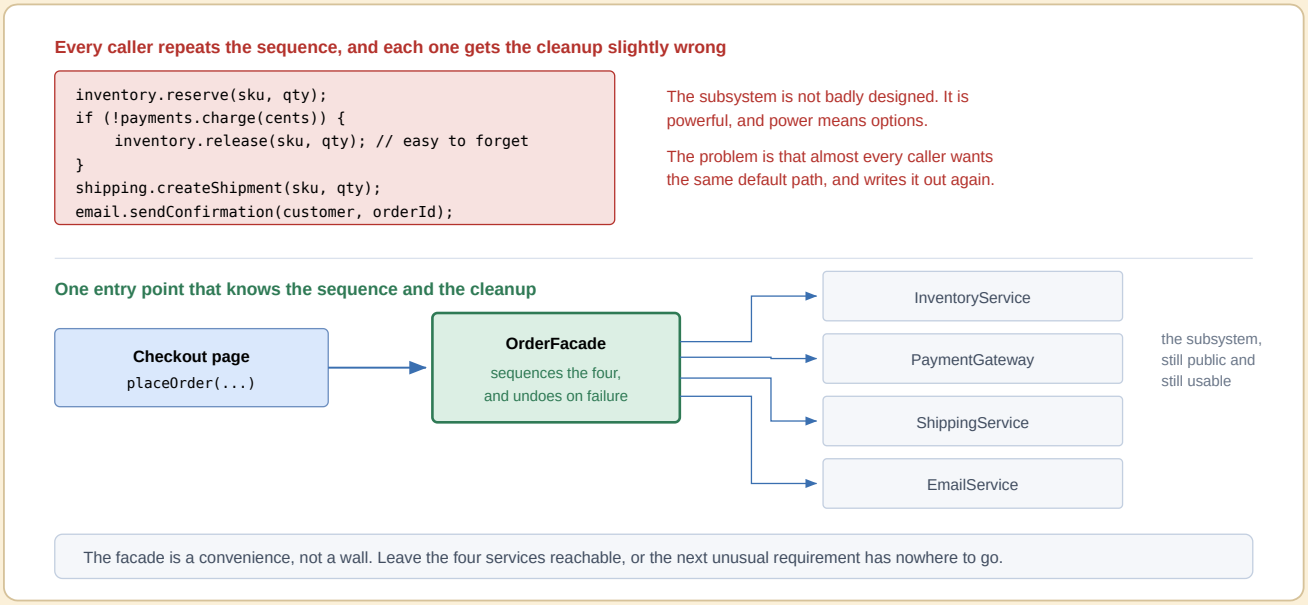


Figure — Top: each caller repeats the reserve/charge/release/ship/email sequence and may forget the release. Bottom: the checkout page calls `OrderFacade.placeOrder(...)`, which sequences the four services and undoes on failure; the services stay public (course diagram)

How it works

1. **Identify the common path** that many callers repeat.
2. **Create a facade class** that holds the subsystem objects.
3. **Expose one intention-revealing method** (`placeOrder`) that calls them in the correct order.
4. **Own the failure handling** — if payment fails, the facade releases the reservation (a *compensating action*).
5. **Keep the subsystem public**, so unusual callers can still reach past the facade.
6. **Add no business rules** — the facade *sequences*, it does not *decide*.

Structure

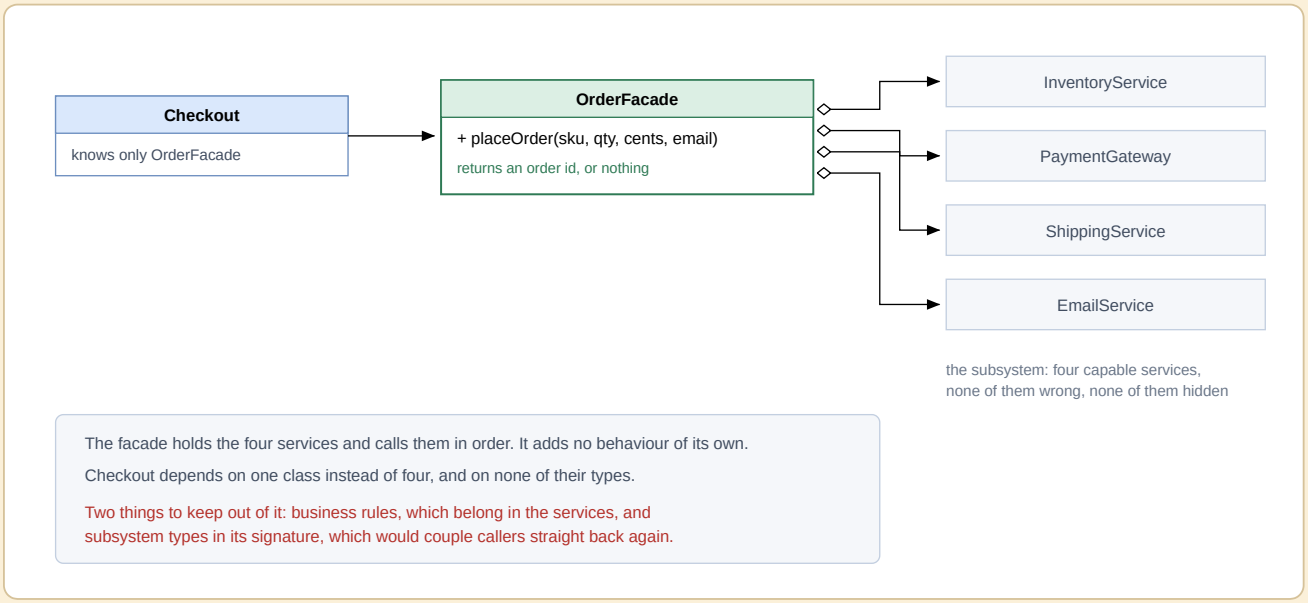
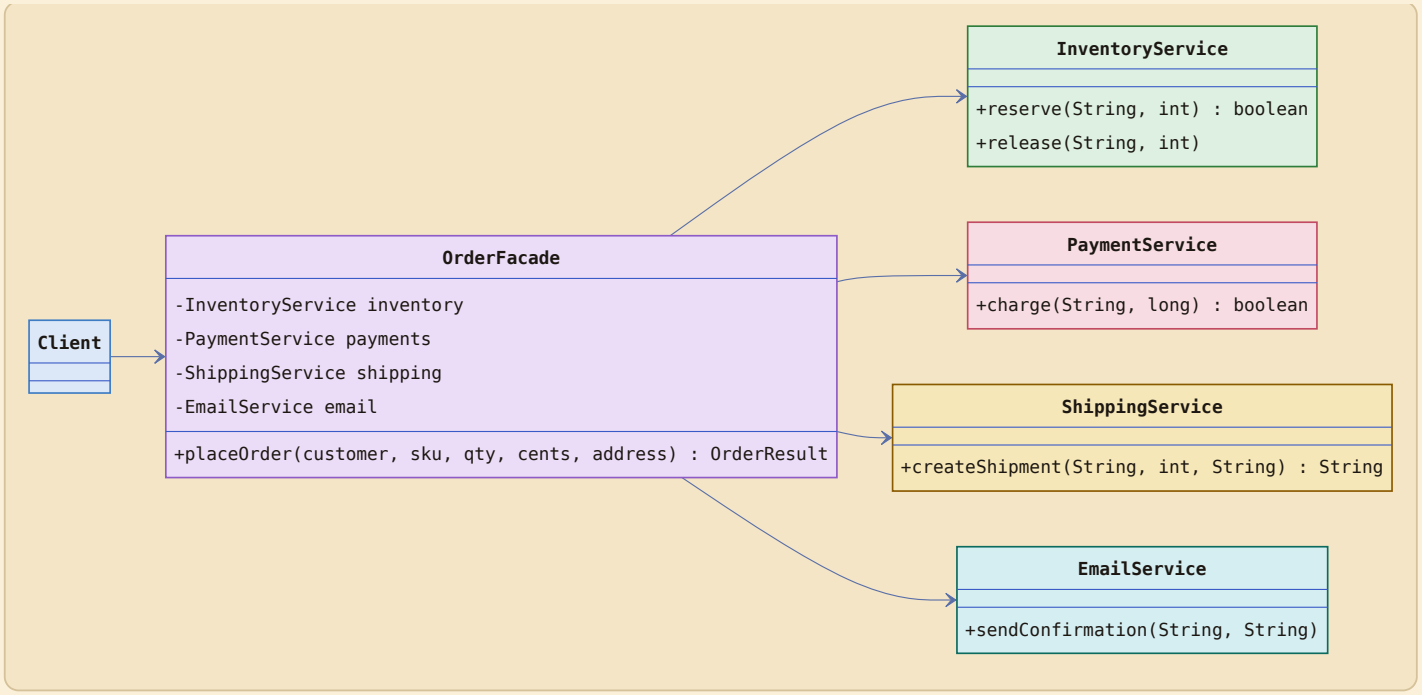
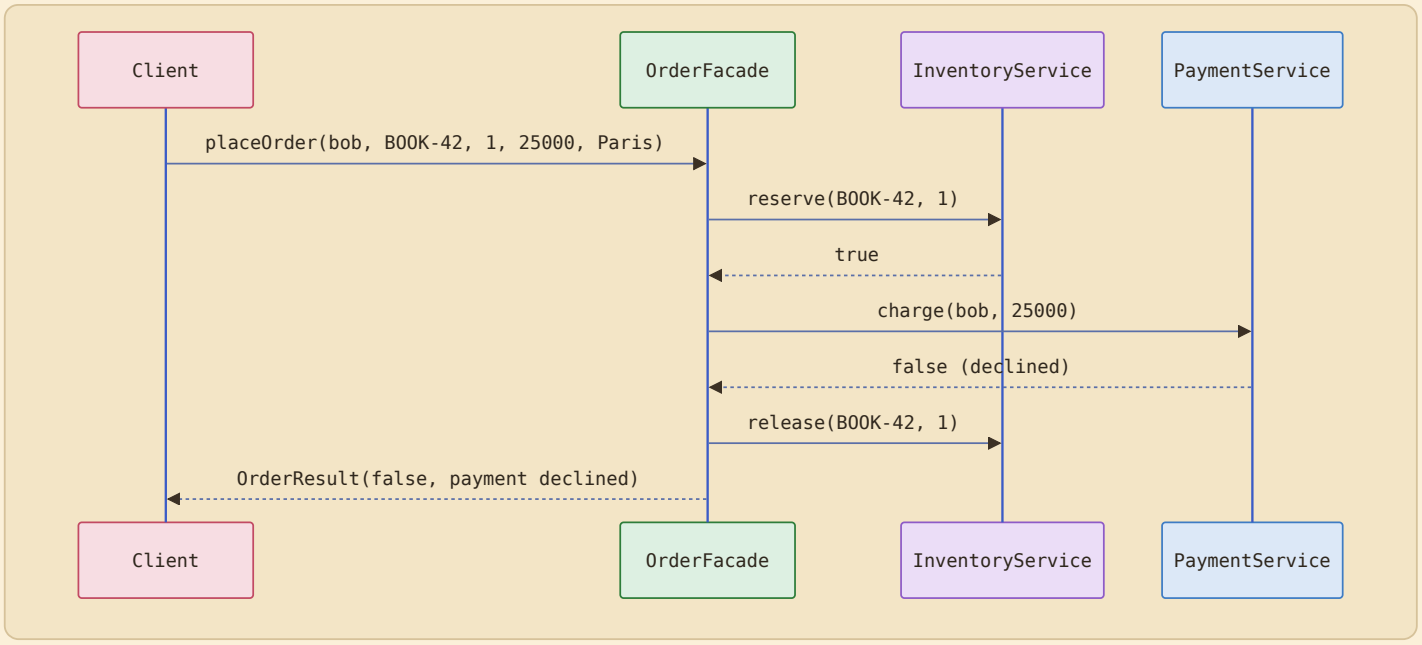


Figure — Facade class diagram: `Checkout` depends only on `OrderFacade`, which holds the four services (shown in grey) and exposes `placeOrder`. The figure labels the payment service `PaymentGateway` and shows a shorter `placeOrder` signature; our code calls it `PaymentService` and also passes customer and address (course diagram)

In the class diagram above, dependencies run in **one direction** only: none of the services knows the facade exists, so they remain usable on their own. Note also that there is no interface: a facade is normally a concrete convenience class, not an abstraction. Introduce an interface only if you plan to substitute the entire subsystem — which is a different job.


InventoryService
+reserve(String, int) : boolean
+release(String, int)
PaymentService
+charge(String, long) : boolean
Client
OrderFacade
-InventoryService inventory
-PaymentService payments
-ShippingService shipping
-EmailService email
+placeOrder(customer, sku, qty, cents, address) : OrderResult
ShippingService
+createShipment(String, int, String) : String
EmailService
+sendConfirmation(String, String)


Client
OrderFacade
InventoryService
PaymentService
placeOrder(bob, BOOK-42, 1, 25000, Paris)
reserve(BOOK-42, 1)
true
charge(bob, 25000)
false (declined)
release(BOOK-42, 1)
OrderResult(false, payment declined)
Client
OrderFacade
InventoryService
PaymentService

Java example

```java
// File: code/lld/ch09/facade/InventoryService.java
package lld.ch09.facade;

import java.util.HashMap;
import java.util.Map;

/*
 * SUBSYSTEM: four independent, still-public services.
 * None of them knows the facade exists.
 */

final class InventoryService {
    private final Map<String, Integer> stock = new HashMap<>(Map.of("BOOK-42", 5));

    boolean reserve(String sku, int qty) {
        int have = stock.getOrDefault(sku, 0);
        if (have < qty) return false;
        stock.put(sku, have - qty);
        System.out.println("  [inventory] reserved " + qty + " x " + sku + " (left " + (have - qty) + ")");
        return true;
    }

    void release(String sku, int qty) {
        stock.merge(sku, qty, Integer::sum);
        System.out.println("  [inventory] released " + qty + " x " + sku + " (left " + stock.get(sku) + ")");
    }
}
```

```java
// File: code/lld/ch09/facade/PaymentService.java
package lld.ch09.facade;

final class PaymentService {
    /** Declines anything over $100 so the demo can show the failure path. */
    boolean charge(String customer, long cents) {
        boolean ok = cents <= 10_000;
        System.out.println("  [payment]   charge " + customer + " " + cents + "c -> " + (ok ? "OK" : "DECLINED"));
        return ok;
    }
}
```

```java
// File: code/lld/ch09/facade/ShippingService.java
package lld.ch09.facade;

final class ShippingService {
    private int next = 1;

    String createShipment(String sku, int qty, String address) {
        String id = "SHP-" + next++;
        System.out.println("  [shipping]  " + id + ": " + qty + " x " + sku + " to " + address);
        return id;
    }
}
```

```java
// File: code/lld/ch09/facade/EmailService.java
package lld.ch09.facade;

final class EmailService {
    void sendConfirmation(String customer, String orderId) {
        System.out.println("  [email]     confirmation for " + orderId + " sent to " + customer);
    }
}
```

```java
// File: code/lld/ch09/facade/OrderResult.java
package lld.ch09.facade;

/** What callers get back: our own small result type. */
record OrderResult(boolean success, String orderId, String message) { }
```

```java
// File: code/lld/ch09/facade/OrderFacade.java
package lld.ch09.facade;

/**
 * FACADE: one method that knows the correct SEQUENCE and the CLEANUP.
 * It sequences; it does not decide business rules (no pricing, no fraud logic here).
 */
final class OrderFacade {
    private final InventoryService inventory;
    private final PaymentService payments;
    private final ShippingService shipping;
    private final EmailService email;
    private int orderSeq = 100;

    OrderFacade(InventoryService inventory, PaymentService payments,
                ShippingService shipping, EmailService email) {
        this.inventory = inventory;
        this.payments = payments;
        this.shipping = shipping;
        this.email = email;
    }

    OrderResult placeOrder(String customer, String sku, int qty, long cents, String address) {
        String orderId = "ORD-" + orderSeq++;
        if (!inventory.reserve(sku, qty)) {
            return new OrderResult(false, orderId, "out of stock");
        }
        if (!payments.charge(customer, cents)) {
            inventory.release(sku, qty);            // compensating step callers used to forget
            return new OrderResult(false, orderId, "payment declined");
        }
        String shipmentId = shipping.createShipment(sku, qty, address);
        email.sendConfirmation(customer, orderId);
        return new OrderResult(true, orderId, "shipped as " + shipmentId);
    }
}
```

```java
// File: code/lld/ch09/facade/Main.java
package lld.ch09.facade;

public class Main {
    public static void main(String[] args) {
        InventoryService inventory = new InventoryService();
        OrderFacade orders = new OrderFacade(inventory, new PaymentService(),
                new ShippingService(), new EmailService());

        System.out.println("== Happy path: one call instead of four ==");
        System.out.println("  -> " + orders.placeOrder("ada@x.com", "BOOK-42", 2, 5_000, "London"));

        System.out.println("== Declined card: the facade undoes the reservation ==");
        System.out.println("  -> " + orders.placeOrder("bob@x.com", "BOOK-42", 1, 25_000, "Paris"));

        System.out.println("== Out of stock ==");
        System.out.println("  -> " + orders.placeOrder("cy@x.com", "BOOK-42", 99, 1_000, "Rome"));

        System.out.println("== The subsystem is still reachable for unusual needs ==");
        inventory.reserve("BOOK-42", 1);   // e.g. a warehouse tool holding stock manually
    }
}
```

Output:

```
== Happy path: one call instead of four ==
  [inventory] reserved 2 x BOOK-42 (left 3)
  [payment]   charge ada@x.com 5000c -> OK
  [shipping]  SHP-1: 2 x BOOK-42 to London
  [email]     confirmation for ORD-100 sent to ada@x.com
  -> OrderResult[success=true, orderId=ORD-100, message=shipped as SHP-1]
== Declined card: the facade undoes the reservation ==
  [inventory] reserved 1 x BOOK-42 (left 2)
  [payment]   charge bob@x.com 25000c -> DECLINED
  [inventory] released 1 x BOOK-42 (left 3)
  -> OrderResult[success=false, orderId=ORD-101, message=payment declined]
== Out of stock ==
  -> OrderResult[success=false, orderId=ORD-102, message=out of stock]
== The subsystem is still reachable for unusual needs ==
  [inventory] reserved 1 x BOOK-42 (left 2)
```

What each part does

- The four services are ordinary classes with their own responsibilities; none mention `OrderFacade`.
- `OrderFacade.placeOrder` is the whole pattern: the correct **sequence** plus the **compensating step** (`inventory.release`) on a declined payment. Look at the second block of output — reserved, declined, released — without the caller writing any of it.
- `OrderResult` — a small result type so callers don't need subsystem types.
- The last line of `main` calls `inventory.reserve` directly: the facade is **a convenience, not a wall**.

Facade vs Adapter vs Mediator

	Facade	**Adapter**	**Proxy**	**Mediator (Ch. 10–11)**
Intent	Simplify a subsystem	Convert one interface to another	Control access to one object	Coordinate peers that would otherwise talk directly
Objects wrapped	**Several**	**One**	**One**	Several colleagues
Interface offered	New, smaller one	The client's (Target)	**Same** as the real object	Mediator's own
Who chose the interface	**You** — a simpler one over things you usually control	**Someone else** — you must fit it	Already fixed by the real object	You
Do the wrapped objects know about it?	**No** — one-way	No	No	**Yes** — colleagues talk *through* the mediator
Direction of communication	Client → facade → subsystem	Client → adapter → adaptee	Client → proxy → real subject	Colleague ↔ mediator ↔ colleague (both ways)
Example	`OrderFacade.placeOrder`	`StripeGateway.pay`	`LazyReport`	An air-traffic control tower; a chat room

The quickest way to spot a facade is the **"objects wrapped"** row: Adapter, Decorator and Proxy each stand in front of exactly one object; a Facade stands in front of **several**.

Another test: suppose the subsystem changed its API tomorrow to exactly what callers want. An **Adapter** would then be pointless; a **Facade** would still be useful, because its value was the *simplification*, not a translation.

Facade vs "service layer". (A service layer is the set of classes that expose the app's use cases — such as placing an order — to its UI or API.) Most of the time these are the same thing under two names. A service class that only *sequences* calls is a facade; once it holds business rules it has become something else (a domain/application service with logic of its own).

The failure mode to watch for

Facades **pull in features**. Day one it is four calls in order; then a discount rule lands in it ("orders are placed here, after all"), then a fraud check, and a year later it is the **god object** (one class that knows and does almost everything) everyone is afraid to edit. Keep one rule: **a facade orchestrates; it does not decide.** If it is making a business decision instead of calling the thing that makes it, that logic belongs elsewhere — and moving it out later is far harder than keeping it out now.

This connects to the **Principle of Least Knowledge** (Law of Demeter) that *Head First Design Patterns* pairs with Facade: "talk only to your immediate friends". Clients talk to the facade, not to a chain of subsystem objects, which reduces coupling.

Trade-offs

Pros	Cons
One call replaces a multi-step sequence, so the common path is no longer duplicated.	**One more class to maintain** as the subsystem evolves.
The order and the undo logic live in one place — exactly the part callers got wrong.	**Feature magnet** — grows into a god object unless someone pushes back.
One seam for swapping the subsystem later — only the facade changes.	**Opinionated** — some caller will need an option the facade left out.
Smaller public surface for the module; lower coupling.	**Can mask problems** — a neat front over a subsystem that still fails; everything funnels through it, so it can become a single point of failure (one part whose failure stops everything).

When not to use it

- **The subsystem is already simple** — wrapping two calls just adds a file to maintain.
- **Callers need the full range** — if every caller bypasses the facade for some option, its shape is wrong: widen it on purpose or remove it.
- **You are covering a mess instead of fixing it** — you get a neat front and the same failures behind it; sometimes the honest move is to repair the subsystem.
- **You want it to be the only entrance** — hiding the subsystem forces every unusual need into the facade, which is how god objects are born.

Common mistakes

- Putting business rules (pricing, fraud) in the facade.
- Making the subsystem inaccessible.
- Returning subsystem types from the facade so callers are still coupled to them.
- Forgetting the failure path — a facade that only does the happy path (the path where nothing goes wrong) misses its biggest value.
- Confusing it with Mediator (see table).

Real-world examples

- **Application service layers** — almost every `OrderService` or `RegistrationService` is a facade over repositories, gateways and notifiers (as long as it only sequences).
- **`java.net.URL`** — `url.openStream()` hides protocol handlers, connection setup and content handlers behind one call. (Since Java 20 the `URL` constructors are deprecated (officially discouraged and due for removal); create one with `URI.create(s).toURL()`. The facade idea is unchanged.)
- **SLF4J** **`LoggerFactory.getLogger(...)`** — hides configuration, appenders (log destinations such as files or the console) and formatters behind one call. (SLF4J literally calls itself a "Simple Logging Facade".)
- **`javax.faces.context.FacesContext`**, **jQuery** (historically the clearest example: one `$` function in front of DOM, events, animation and XHR (the browser's API for sending HTTP requests from JavaScript), each still usable directly), and Python's **`requests`**, which covers the common "just fetch this URL" cases on top of `urllib3`.
- **E-commerce checkout**: when you press "Place order" on Amazon or Flipkart, one backend (server-side) call orchestrates (coordinates) inventory, payment, shipping and notifications — a facade (often called an

orchestrator or backend-for-frontend — a server layer built for one particular UI).

In modern languages

- **Python / JavaScript**: a module exporting one function and keeping the rest private is a facade with no class at all.
- **Go**: a package's exported functions are the facade; lowercase identifiers stay internal.
- **Java**: a package with one public class (package-private subsystem), or a JPMS (Java Platform Module System) module that exports only the facade's package.
- **Any language**: a single well-named function is often the whole pattern — if `placeOrder` needs no state of its own, it does not have to be a class.

In the interview

> **30-second answer:** "A Facade gives one simple entry point to a complex subsystem. For order placement, `OrderFacade.placeOrder` reserves stock, charges, ships and emails in the right order, and if payment fails it releases the reservation — so no caller has to remember that cleanup. It wraps several objects, unlike Adapter, Decorator and Proxy which wrap one. I keep the subsystem public, because a facade is a convenience, not a wall, and I keep business rules out of it — the main risk is that it grows into a god object."

Follow-ups

- *Q: How is it different from Adapter?* — A: Adapter converts one interface because someone else chose it; Facade simplifies many objects behind a new interface you chose.
- *Q: And from Mediator?* — A: Subsystem classes don't know the facade exists; colleagues in a Mediator know it and communicate through it, in both directions.
- *Q: Risk?* — A: God object. A facade should sequence, not decide.
- *Q: Should the subsystem be hidden?* — A: Usually no; leave it reachable for unusual needs.
- *Q: Can a system have several facades?* — A: Yes — e.g. one per use case, or layered facades over sub-subsystems.

Self-check

1. **Q:** What is the fastest structural way to tell a Facade from Adapter/Decorator/Proxy? **A:** It wraps several objects; the others wrap one.
2. **Q:** In our code, what does the facade do when payment is declined? **A:** Releases the stock reservation and returns a failed `OrderResult`.
3. **Q:** A teammate adds a 10% discount rule inside `placeOrder`. Good idea? **A:** No — business rules belong in a service the facade calls; the facade only sequences.
4. **Q:** Which OO principle is closely related to Facade? **A:** Principle of Least Knowledge (Law of Demeter).

TL;DR

> - **One door into a complex subsystem.** Names the common path and owns the cleanup.
> - Wraps **several** objects; subsystem stays **public**; facade **sequences, never decides**.
> - Different from Adapter (translation of one) and Mediator (two-way coordination of peers).

9.6 Flyweight Pattern

What it is

The **Flyweight** pattern **shares** the portion of each object's state that is the same across many objects, so that huge numbers of them fit in memory. GoF's intent: *use sharing to support large numbers of fine-grained objects efficiently.*

Analogy: a printed book. Every letter "e" on every page uses the same metal type design (shared), placed at a different position (unique). You don't carve a new "e" for every occurrence.

Two terms carry the whole pattern:

Term	Meaning	In the map example
Intrinsic state	Part of the thing's own identity and **identical for every holder**. Lives inside the shared flyweight.	icon bitmap, colour, size
Extrinsic state	**Varies with the context** the object is used in; different for each holder. Stays outside and is **handed in** as method arguments.	latitude, longitude

The words are GoF's and interviewers use them, even though on their own they explain little — remember "intrinsic = inside the shared object", "extrinsic = supplied from outside".

Role	In our code
Flyweight (shared, immutable, intrinsic only)	`MarkerStyle`
Flyweight factory (owns the pool, returns existing or creates once)	`MarkerStyleFactory`
Context (one per logical object: extrinsic state + reference to a flyweight)	`Marker`
Client (asks the factory, supplies extrinsic state)	`Main`

GoF's own participant list is *Flyweight* (the interface that accepts extrinsic state), *ConcreteFlyweight* (the shared implementation — our `MarkerStyle` plays both roles since we need no interface), *UnsharedConcreteFlyweight* (objects of the same interface that are **not** shared, e.g. a row object holding its own children), *FlyweightFactory* and *Client*. "Context" is the name later sources (such as refactoring.guru) give to the object that stores the extrinsic state.

The problem it solves

Picture a map app showing **100,000 markers**. Every marker stores its latitude and longitude — plus an icon bitmap, a colour and a size. Positions are all different, but the icon is the same for thousands of markers, and the icon is by far the largest field. Keeping 100,000 copies of a 2 MB bitmap is not "a bit slow": it needs ~195 GB, so **the program simply runs out of memory.**

The same situation shows up in a game with a million trees that all use one mesh and texture (a 3D shape and the image painted onto it), in a text editor that would otherwise store font data for every character, and anywhere many objects **differ slightly but share a large common part**.

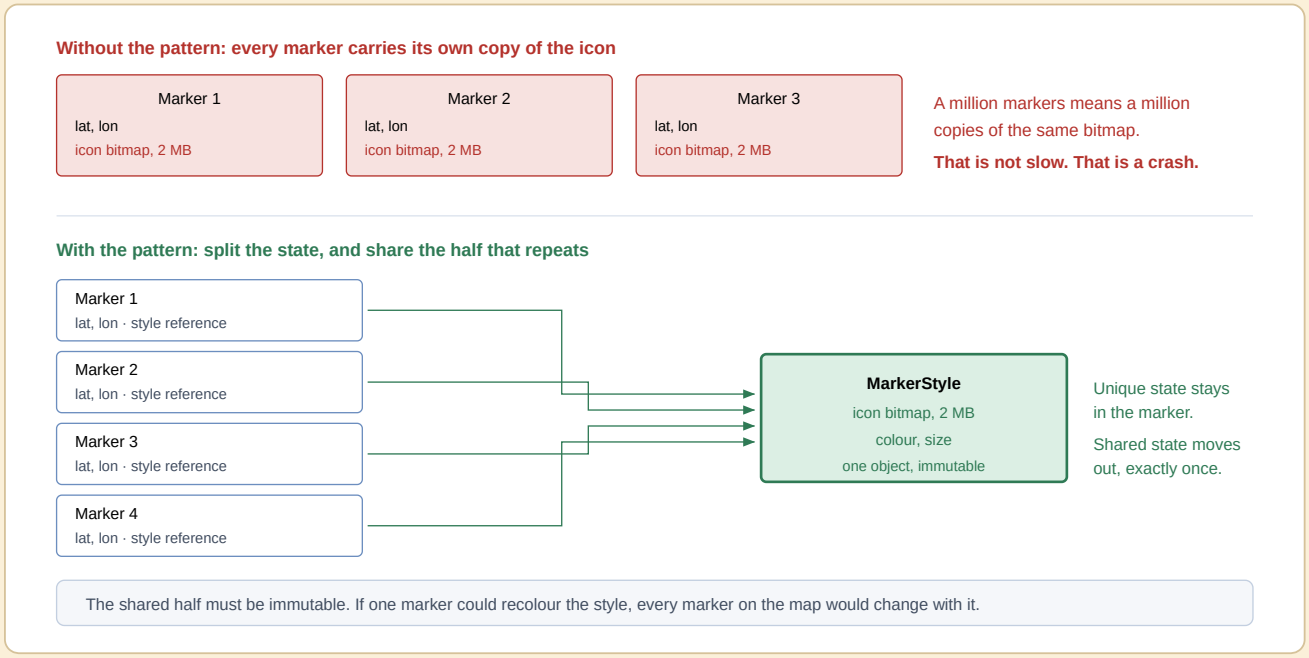


Figure — Top: every marker carries its own 2 MB icon. Bottom: each marker keeps only lat/lon and a style reference; one immutable `MarkerStyle` holds the icon, colour and size (course diagram)

How it works

1. **Split the state**: intrinsic (shared, invariant) vs extrinsic (per holder).
2. **Move intrinsic state into an immutable flyweight class** — all fields `final`, no setters.
3. **Pass extrinsic state as method arguments** (`draw(lat, lon)`) instead of storing it in the flyweight.

4. **Create a factory that owns the pool**, keyed by the intrinsic values. `get(...)` returns the existing object or creates it the first time.
5. **Clients must go through the factory**, never the constructor (enforce with package-private/private constructors).
6. **Context objects** keep only extrinsic state plus a **reference** (not a copy) to the flyweight.

> **Two rules — break either and the pattern silently stops working.**
>
> 1. **Flyweights never change.** Many holders point at the same object, so a modification made for one is seen by every other.
> 2. **Only the factory creates them.** If callers can use the constructor directly, identical copies appear and the memory saving is gone.

Structure

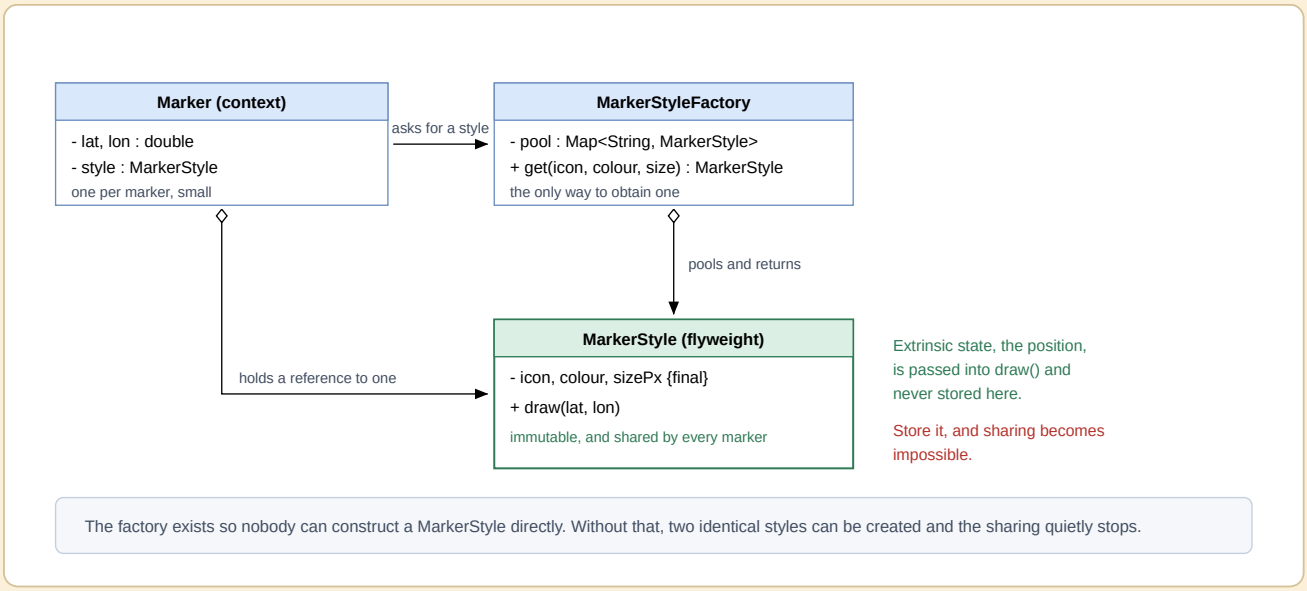
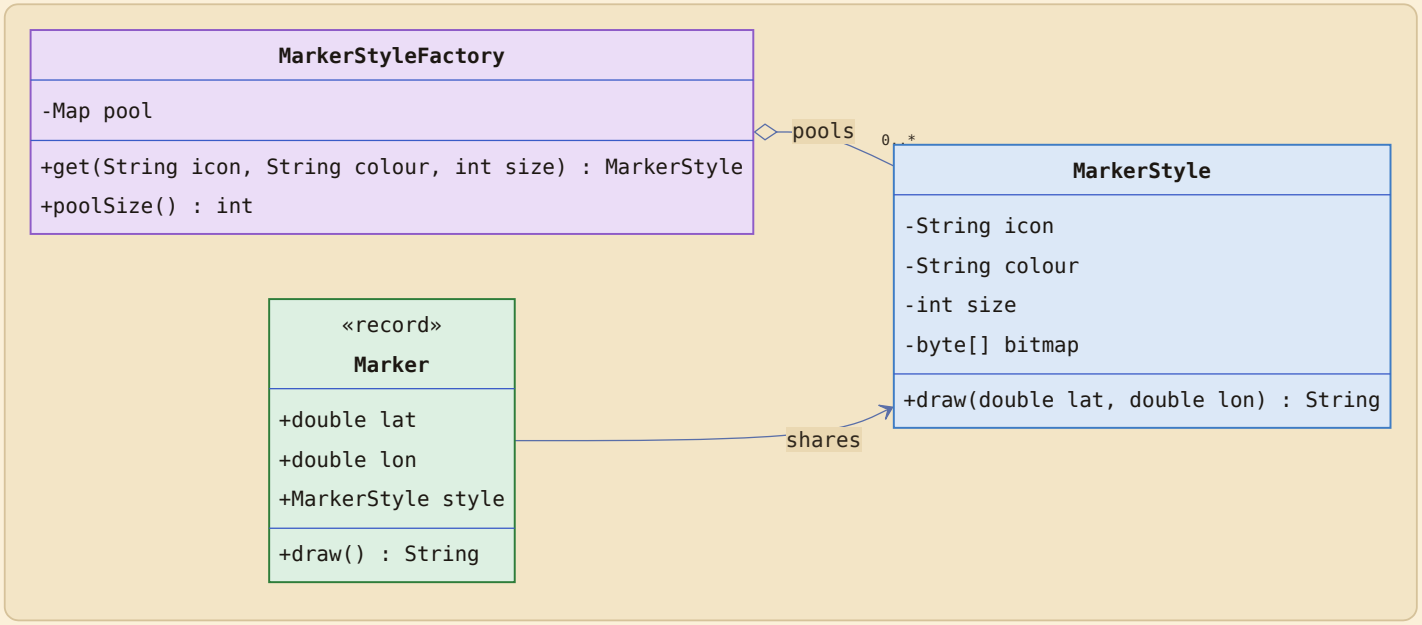


Figure — Flyweight class diagram: `MarkerStyle` (final fields, `draw(lat, lon)`), `MarkerStyleFactory` with a pool and `get()`, and `Marker` holding position plus a style reference (course diagram)

In the diagram above, `draw` receives the position as arguments rather than keeping it in a field, `Marker` holds a *reference* to a style rather than its own copy, and clients obtain styles from the factory, never from the constructor. That last rule has to be enforced — by a package-private (or private) constructor, or at least by convention and code review.



Java example

```java
// File: code/lld/ch09/flyweight/MarkerStyle.java
package lld.ch09.flyweight;

/**
 * FLYWEIGHT: holds only INTRINSIC state (same for every marker using this style).
 * Immutable: all fields final, no setters, and the bitmap array never leaves the object.
 * The constructor is package-private, so code outside this package cannot bypass the
 * factory. (Inside the package it is a convention: only MarkerStyleFactory calls it.)
 */
final class MarkerStyle {
    static final int ICON_BYTES = 2 * 1024 * 1024;   // pretend every icon bitmap is 2 MB

    private final String icon;
    private final String colour;
    private final int size;
    private final byte[] bitmap;   // the heavy part we want exactly one copy of

    MarkerStyle(String icon, String colour, int size) {
        System.out.println("  loading icon '" + icon + "' (" + colour + ", " + size + "px)");
        this.icon = icon;
        this.colour = colour;
        this.size = size;
        this.bitmap = new byte[ICON_BYTES];
    }

    /** EXTRINSIC state (the position) is passed in, never stored. */
    String draw(double lat, double lon) {
        return String.format("%s %s marker at (%.4f, %.4f)", colour, icon, lat, lon);
    }

    int bitmapBytes() { return bitmap.length; }
}
```

```java
// File: code/lld/ch09/flyweight/MarkerStyleFactory.java
package lld.ch09.flyweight;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/** FLYWEIGHT FACTORY: owns the pool. Thread-safe via ConcurrentHashMap.computeIfAbsent. */
final class MarkerStyleFactory {
    /** The pool key is the combination of intrinsic values. */
    private record Key(String icon, String colour, int size) { }

    private final Map<Key, MarkerStyle> pool = new ConcurrentHashMap<>();

    MarkerStyle get(String icon, String colour, int size) {
        return pool.computeIfAbsent(new Key(icon, colour, size),
                k -> new MarkerStyle(k.icon(), k.colour(), k.size()));
    }

    int poolSize() { return pool.size(); }
}
```

```java
// File: code/lld/ch09/flyweight/Marker.java
package lld.ch09.flyweight;

/** CONTEXT: one per marker. Holds extrinsic state + a REFERENCE to a shared style. */
record Marker(double lat, double lon, MarkerStyle style) {
    String draw() { return style.draw(lat, lon); }
}
```

```java
// File: code/lld/ch09/flyweight/Main.java
package lld.ch09.flyweight;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class Main {
    public static void main(String[] args) {
        MarkerStyleFactory styles = new MarkerStyleFactory();

        System.out.println("== Five markers, three styles ==");
        List<Marker> few = List.of(
                new Marker(51.5074, -0.1278, styles.get("cafe", "brown", 24)),
                new Marker(48.8566, 2.3522, styles.get("cafe", "brown", 24)),
                new Marker(40.7128, -74.0060, styles.get("fuel", "green", 24)),
                new Marker(35.6762, 139.6503, styles.get("hospital", "red", 32)),
                new Marker(52.5200, 13.4050, styles.get("fuel", "green", 24)));
        few.forEach(m -> System.out.println("  " + m.draw()));
        System.out.println("  same shared object? " + (few.get(0).style() == few.get(1).style()));

        System.out.println("== Scale to 100,000 markers ==");
        String[] icons = {"cafe", "fuel", "hospital"};
        String[] colours = {"brown", "green", "red"};
        int[] sizes = {24, 24, 32};
        Random rnd = new Random(42);
        List<Marker> many = new ArrayList<>();
        for (int i = 0; i < 100_000; i++) {
            int t = rnd.nextInt(3);
            many.add(new Marker(rnd.nextDouble() * 180 - 90, rnd.nextDouble() * 360 - 180,
                    styles.get(icons[t], colours[t], sizes[t])));
        }
        long withoutMb = (long) many.size() * MarkerStyle.ICON_BYTES / (1024 * 1024);
        long withMb = (long) styles.poolSize() * MarkerStyle.ICON_BYTES / (1024 * 1024);
        System.out.println("  markers: " + many.size() + ", styles in pool: " + styles.poolSize());
        System.out.println("  icon memory without flyweight: ~" + withoutMb + " MB");
        System.out.println("  icon memory with flyweight:    ~" + withMb + " MB");

        System.out.println("== Flyweights already in the JDK ==");
        Integer a = 127, b = 127;          // autoboxing calls Integer.valueOf
        Integer c = 128, d = 128;
        System.out.println("  Integer 127 == 127 -> " + (a == b) + "  (cached -128..127)");
        System.out.println("  Integer 128 == 128 -> " + (c == d) + " (outside default cache)");
        System.out.println("  equals() is always right -> " + c.equals(d));
        System.out.println("  Boolean.valueOf(true) == Boolean.TRUE -> " + (Boolean.valueOf(true) ==
Boolean.TRUE));

        String lit = "hello";
        String built = new String("hello");
        System.out.println("  \"hello\" == \"hel\" + \"lo\" -> " + (lit == "hel" + "lo") + " (compile-time
constant, pooled)");
        System.out.println("  \"hello\" == new String(\"hello\") -> " + (lit == built));
        System.out.println("  \"hello\" == built.intern() -> " + (lit == built.intern()));
    }
}
```

Output:

```
== Five markers, three styles ==
  loading icon 'cafe' (brown, 24px)
  loading icon 'fuel' (green, 24px)
  loading icon 'hospital' (red, 32px)
  brown cafe marker at (51.5074, -0.1278)
  brown cafe marker at (48.8566, 2.3522)
  green fuel marker at (40.7128, -74.0060)
  red hospital marker at (35.6762, 139.6503)
  green fuel marker at (52.5200, 13.4050)
  same shared object? true
== Scale to 100,000 markers ==
  markers: 100000, styles in pool: 3
  icon memory without flyweight: ~200000 MB
  icon memory with flyweight:    ~6 MB
== Flyweights already in the JDK ==
  Integer 127 == 127 -> true  (cached -128..127)
  Integer 128 == 128 -> false (outside default cache)
  equals() is always right -> true
  Boolean.valueOf(true) == Boolean.TRUE -> true
  "hello" == "hel" + "lo" -> true (compile-time constant, pooled)
  "hello" == new String("hello") -> false
  "hello" == built.intern() -> true
```

What each part does

- `MarkerStyle` — the flyweight. Final fields, no setters, package-private constructor, and `draw(lat, lon)` receives extrinsic state.
- `MarkerStyleFactory` — the pool. `ConcurrentHashMap.computeIfAbsent` makes "get or create once" **thread-safe and atomic** (correct when many threads — independent paths of execution running at once — use it, and done as one indivisible step), so two threads can't create two copies of the same style.
- `Key` record — the intrinsic values as a value object (an object defined only by its values); records give correct `equals`/`hashCode` (the methods a hash map uses to find a key) for free.
- `Marker` record — the context: two doubles plus one reference.
- Output: "loading icon" is printed **3 times for 5 markers**, and still only 3 styles for 100,000 markers — about 6 MB of icon data instead of ~200,000 MB (≈195 GB).
- JDK section: `Integer.valueOf` caching and the `String` pool (below).

Flyweights already in Java

- **`Integer.valueOf(int)`** (used by autoboxing — Java's automatic wrapping of an `int` into an `Integer` object) returns cached objects for **−128 to 127** — the Java Language Specification requires that range to be cached. That is why `Integer a = 127, b = 127; a == b` is `true` but the same with 128 is `false` on a default JVM (Java Virtual Machine, the program that runs Java code; the upper bound can be raised with `-XX:AutoBoxCacheMax`, so never rely on it). **Always compare boxed numbers with `equals`.** `Short`, `Byte`, `Long` (same range) and `Character` (0–127) have similar caches; `Boolean.valueOf` returns the two shared constants `Boolean.TRUE` / `Boolean.FALSE`.
- **The String pool**: string literals and compile-time constant expressions (`"hel" + "lo"`) are interned (stored once in a shared pool and reused), so equal literals share one object. `new String("hello")` always creates a new object; `intern()` returns the pooled one.
- **Enums**: exactly one instance per constant, shared everywhere.

Flyweight is not a cache

The two are mixed up all the time — even in code reviews — and the difference decides what you may do with the pool:

	Flyweight	Cache
Purpose	Make the data *fit in memory* at all	Make repeated work *quicker*
Contents	One copy of immutable state that many holders point to	Stored results of costly computations or lookups
May entries be removed?	Not while any holder still points to one	Yes, at any moment
Effect of removing one	Holders lose their shared object, or duplicates get created	None, apart from a slower next request

So an LRU cache in front of a database is not a flyweight, and a pool of shared marker styles is not a cache.

Two mistakes that undo the pattern

- **A flyweight that can change.** One caller updates a colour and every marker on the map changes with it — the classic failure, and hard to debug because the effect shows up far from the cause. Fix: `final` fields, no setters, no leaking of internal arrays.
- **An ever-growing pool keyed by open-ended input.** If users decide the keys, the pool never stops growing — a memory leak (memory that is never released) disguised as an optimisation. Flyweight is safe when the set of intrinsic values is **small and bounded** (3 styles, a few hundred glyphs). An open key space needs weak references (references the garbage collector is allowed to clear) and eviction (removing entries), and at that point you have built a cache — so call it a cache.

Measure first

Flyweight trades readability for memory, so using it without profiling (measuring where a program really spends time or memory) is textbook premature optimisation (optimising before you know it is needed). A thousand objects never need it; a million might. Answer two questions **with numbers** first: *how many of these objects will exist at once*, and *how much of each one is identical across all of them?* If you can't put numbers on both, it is too early; if most of each object is unique, there is nothing worth sharing.

Trade-offs

Pros	Cons
Memory use can fall by orders of magnitude — sometimes the difference between running and not running.	**State is split across two places**, which is harder to follow and change than one class.
Costly construction happens once per distinct value, not once per object.	**The pool keeps strong references (ordinary references, which stop the garbage collector from freeing an object)**, so it can leak memory itself.
Friendlier to CPU caches — the shared data stays hot (recently used, so fast to reach) instead of being scattered.	**Thread safety is on you** — the pool must be thread-safe and the flyweight immutable.
"Load once" logic gets one obvious home instead of being spread across callers.	**Tempting to apply too early**, and callers must pass extrinsic state on every call (some CPU cost).

When not to use it

- You **haven't measured**.
- The shared part **cannot be immutable**.
- The **counts are small**.
- The **unique state is most of the object**.
- The **key space is unbounded** (that's a cache).

Common mistakes

- Storing extrinsic state inside the flyweight (then it can't be shared).
- Public constructor so callers bypass the factory.
- Non-thread-safe pool (`HashMap` + check-then-put from many threads → duplicates or worse).
- Comparing boxed `Integer`s or `String`s with `==` because "it worked in the test" with small values or literals.

Other patterns it is confused with

Looks like	Actual difference
Cache	See table above.
Singleton	Singleton = exactly one instance of a class. Flyweight = one instance **per distinct intrinsic value** (3, or 300). A flyweight pool that happens to hold one entry only *looks* like a Singleton.
Object pool	An object pool **lends out mutable** objects and takes them back (e.g. DB connections). Flyweights are **immutable**, shared, never returned.
Prototype	Prototype makes many objects by copying; Flyweight avoids making them at all — opposite answers to the same "many similar objects" observation.

Real-world examples

- **`Integer.valueOf`**, **`Boolean.valueOf`**, **`String.intern()`**, **enums** — see above. (Python also interns small integers and many short strings automatically.)
- **Text rendering / word processors** — one glyph object per character in a font, shared by every occurrence on the page (the classic GoF example).
- **Game engines and map renderers** — meshes, textures and sprite atlases are loaded once and referenced by every entity (e.g. a forest of identical tree models, rendered with GPU *instancing* — the graphics card drawing one model many times in a single pass).
- **Map apps** — thousands of pins typically share a handful of icon styles (the example in this section).
- **`java.time.ZoneOffset`** caches offsets that are whole multiples of 15 minutes, so `ZoneOffset.ofHours(5)` returns a shared instance.

In modern languages

- **Java**: `Integer.valueOf`, `String.intern` and enums are built-in flyweights; a `record` is an immutable flyweight in one line, and a `Map` plus `computeIfAbsent` is the whole factory (as in our code).
- **Python**: small ints and many short strings are interned automatically; `functools.lru_cache` on a factory function gives a pool with a size bound built in.
- **JavaScript**: `Object.freeze` on the shared object plus a `Map` for the pool — freezing makes immutability enforced rather than a convention.
- **C++**: `std::shared_ptr<const Style>` expresses both the sharing and the immutability in the type.
- **Any language with value types**: an immutable value object plus a memoising (remembers results it already produced) factory is this pattern, usually unnamed.

In the interview

> **30-second answer:** "Flyweight saves memory when there are huge numbers of similar objects. I split state into intrinsic — shared and invariant, like an icon — and extrinsic — per object, like a position — which is passed in to methods. The intrinsic part lives in an immutable flyweight handed out by a factory that pools one instance per distinct value. `Integer.valueOf` and the String pool are JDK examples. I'd only do it after measuring."

Follow-ups

- *Q: Why must the flyweight be immutable?* — A: It is shared; mutating it changes every holder.
- *Q: How do you make the factory thread-safe?* — A: `ConcurrentHashMap.computeIfAbsent`, which runs the creation function at most once per key.
- *Q: Flyweight vs cache?* — A: A cache may evict anything and exists for speed; a flyweight exists because otherwise the program doesn't fit, and entries can't be dropped while referenced.
- *Q: Why is `Integer 128 == 128` false?* — A: Autoboxing uses `Integer.valueOf`, which only guarantees caching for −128..127; outside that range you get distinct objects on a default JVM. Use `equals`.
- *Q: Flyweight vs Singleton?* — A: One instance per distinct value vs one instance total.

Self-check

1. **Q:** In a chess game with a million saved positions, what is intrinsic and extrinsic for a piece? **A:** Intrinsic: type and colour (and image). Extrinsic: the square it is on.

2. **Q:** What goes wrong if `MarkerStyle` had `setColour`? **A:** Changing one marker's colour would change every marker sharing that style.
3. **Q:** Is `"a" + "b" == "ab"` true in Java? **A:** Yes — it is a compile-time constant expression, so it is interned. With a non-final variable in the expression, it would be a new object.
4. **Q:** When does a flyweight pool become a leak? **A:** When its key space is unbounded (e.g. keyed by user input) and it never evicts.

TL;DR

- **Share what repeats, pass in what varies.** Intrinsic inside, extrinsic outside.
- **Immutable** flyweight + **factory-owned** pool (thread-safe).
- JDK: `Integer.valueOf` (−128..127), String pool, enums. **Measure first**; not a cache.

9.7 Proxy Pattern

What it is

The **Proxy** pattern places a **stand-in with the identical interface** in front of a real object in order to control **who** may use it, **when** it gets created, or **how much** each use costs. GoF's intent: *provide a surrogate or placeholder for another object to control access to it.* (GoF also lists the name **Surrogate**.)

Analogy: a credit card is a proxy for your bank account. Shops accept it wherever cash is accepted (same "interface"), but it adds control — limits, fraud checks — and the money only moves when needed.

Term	Meaning	In our code
Subject	The interface shared by the proxy and the real object	`Report`, `RateService`, `Device`
Real Subject	The object that does the real work	`RealReport`, `LiveRateService`, `RealDevice`
Proxy	Holds (or creates) the real subject, same interface, decides what happens before/instead of/around the call	`LazyReport`, `AuditedReport`, `CachingRateProxy`, `RemoteRateProxy`, the `SharedDevice` handle

The problem it solves

There are times when callers should **not touch the real object directly — or not yet**:

- **Building it is costly and usually unnecessary.** Loading a 100 MB report during start-up is wasted work on the many runs that never display it — most screens only show its title.
- **Only some callers are permitted** to use it, and the permission logic has no business living inside the object.
- **It runs on a different machine**, and callers should not have to write networking code.
- **Each call must be logged or counted**, but the class belongs to someone else.

Since the proxy offers **the very same interface**, callers can't tell it apart from the real thing, so you can insert one **without editing any caller**. That is also its danger: a slow or remote call now **looks just like a cheap local one**.

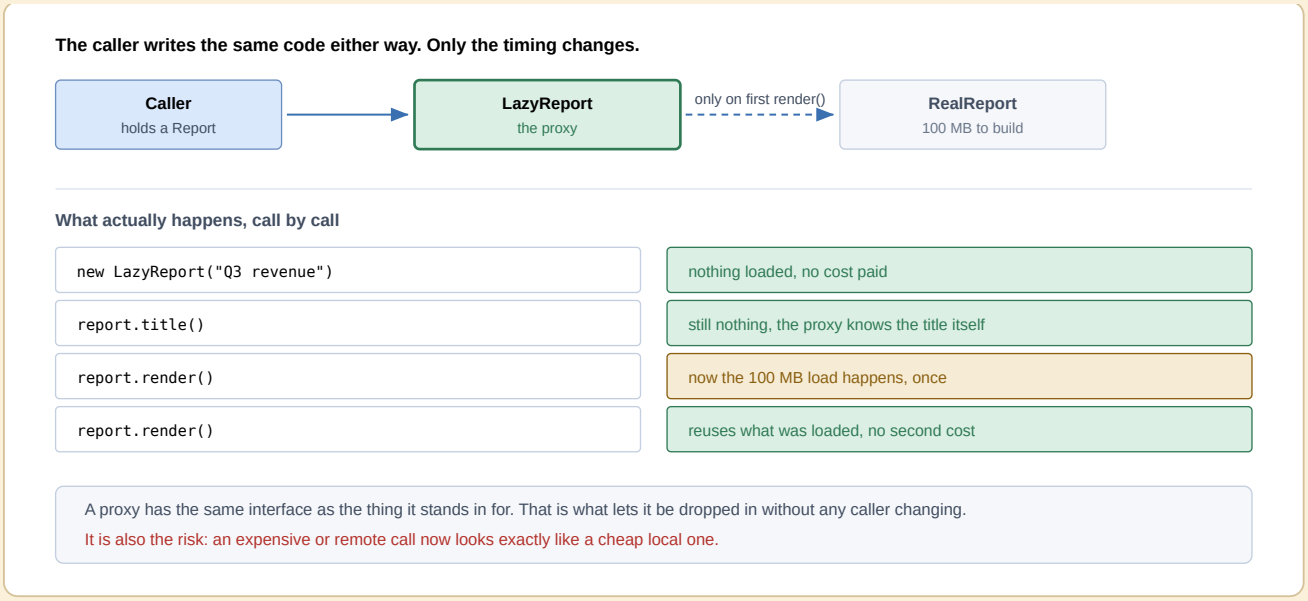


Figure — The caller holds a `Report`. Creating the `LazyReport` and calling `title()` cost nothing; the 100 MB load happens once on the first `render()`, and later renders reuse it (course diagram)

How it works

1. **Define (or reuse) the Subject interface** that callers already use.
2. **Write a proxy class implementing that interface** and holding a reference to the real subject — or the information needed to create it later.
3. In each method, **do the proxy's job first** — create lazily, check permission, look in a cache, marshal (convert into a message that can be sent) over a network, count references — then **delegate** (or refuse).
4. **Give callers the proxy** in place of the real object (often via a factory or DI container so they never see the difference).

Kinds of proxy

Kind	What it controls	Our code / real example
Virtual proxy	*When* the expensive object is created (lazy initialisation)	`LazyReport`; Hibernate (a Java ORM) lazy-loaded entities
Protection proxy	*Who* may call (access control)	`AuditedReport`; `Collections.unmodifiableList`
Remote proxy	*Where* the object lives — hides the network	`RemoteRateProxy` (stub) + `RateServer` (skeleton); RMI/gRPC (remote-call frameworks) stubs
Caching proxy	*Whether* the real call is needed — reuses results	`CachingRateProxy` (bounded LRU + TTL — time to live, how long an entry stays valid)
Logging / monitoring proxy	Records each call before/after forwarding	`LoggingHandler` (dynamic proxy)
Smart reference	Bookkeeping on access — reference counting, locking, loading on first use	`SharedDevice` handles close the device when the last one closes

GoF names four kinds — remote, virtual, protection and smart reference; they differ only in what the proxy does before delegating. Caching and logging proxies are widely used variants described by later sources such as refactoring.guru (GoF would file logging/counting under smart reference).

Structure

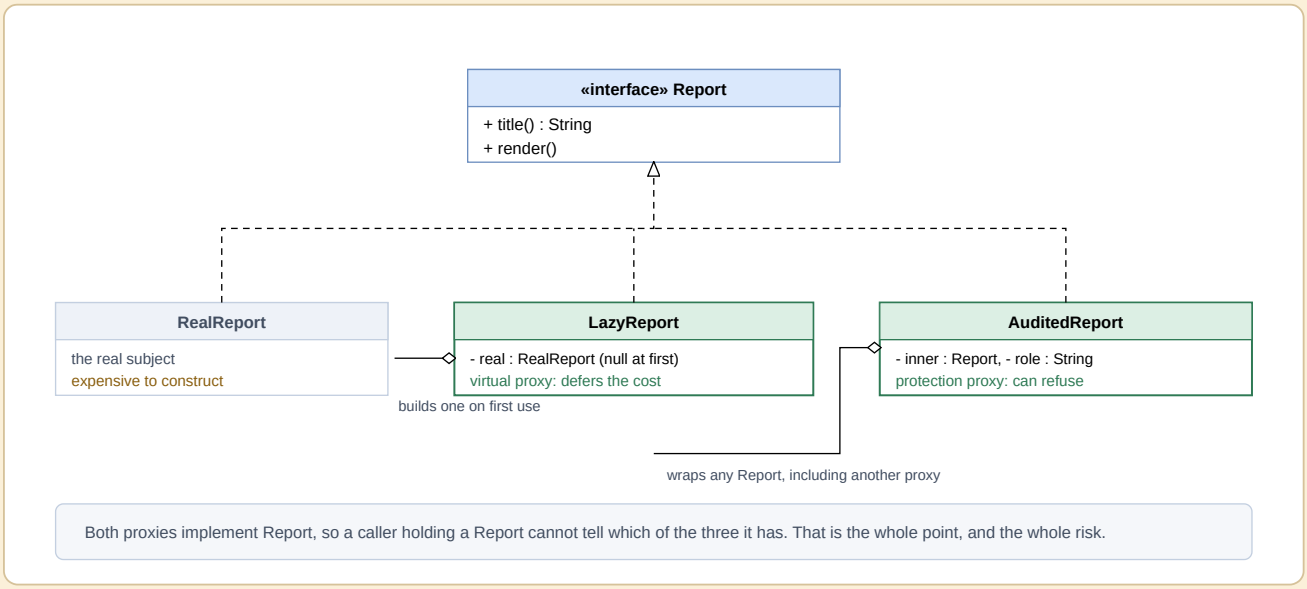
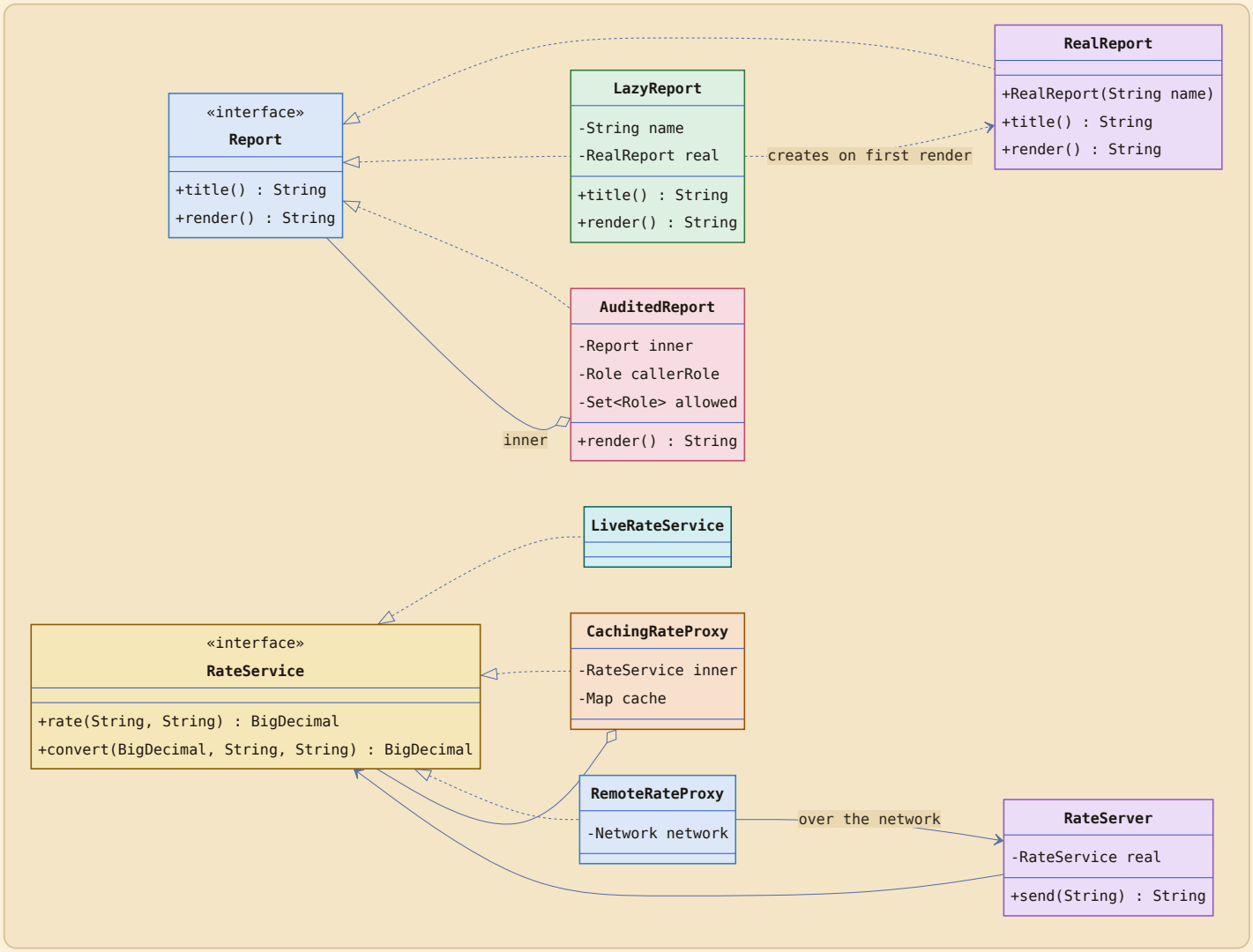
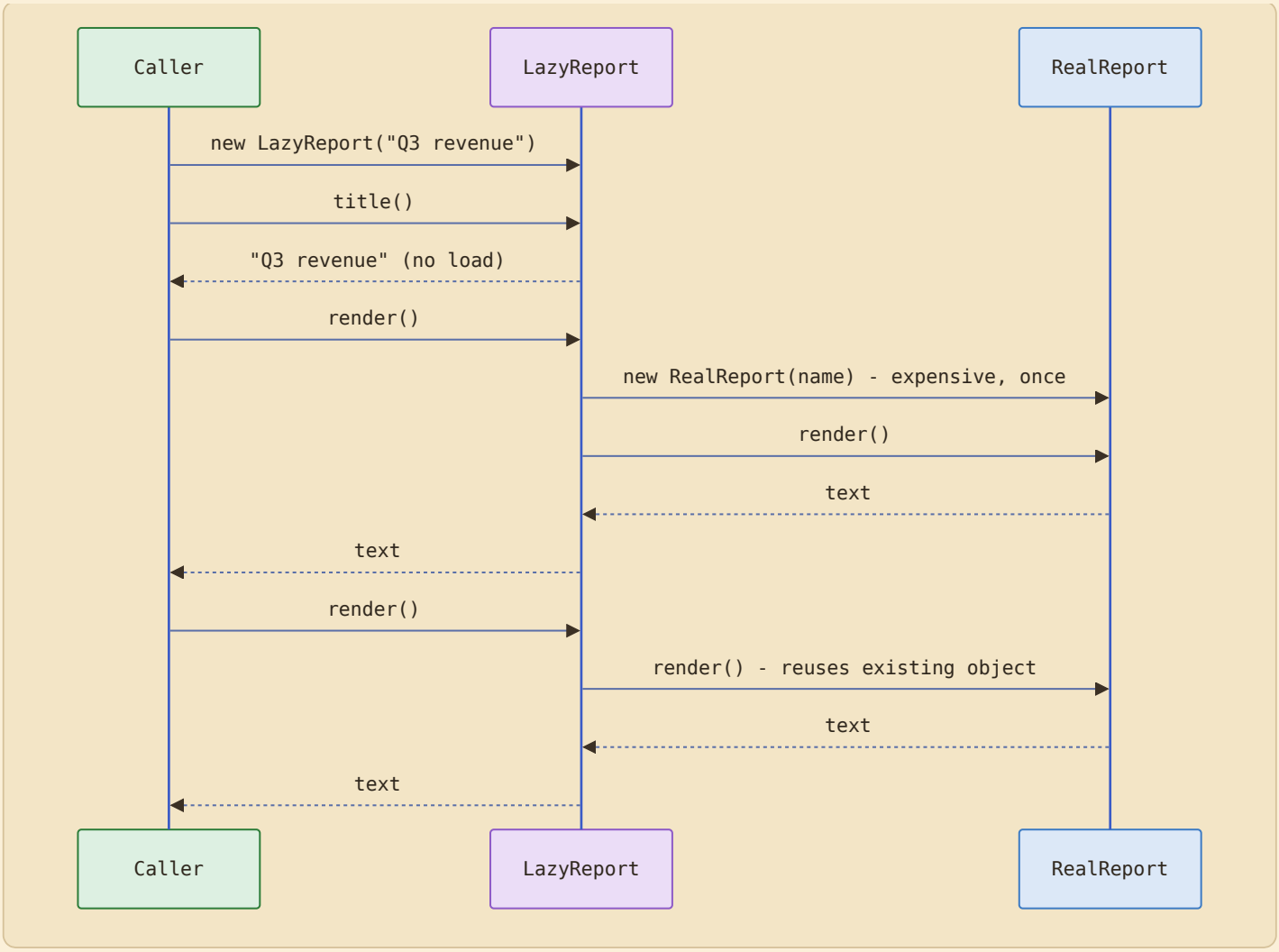


Figure — Proxy class diagram: `RealReport`, `LazyReport` (virtual proxy) and `AuditedReport` (protection proxy) all implement `Report`; `LazyReport` creates the `RealReport` itself, `AuditedReport` wraps any `Report` (course diagram)

In the class diagram above, `RealReport` does the expensive work in its constructor; `LazyReport` keeps only the name, answers `title()` itself and constructs the `RealReport` the first time `render()` is called; `AuditedReport` checks a role before letting a call through. Because `AuditedReport` holds the **`interface`**, it can guard the real report, the lazy proxy or another protection proxy — our demo wraps the lazy one. Strip `LazyReport` down and only one line is special — the null check that triggers construction; the rest simply forwards calls.





Java example

```java
// File: code/lld/ch09/proxy/Report.java
package lld.ch09.proxy;

/** SUBJECT interface: callers only ever hold this type. */
interface Report {
    String title();
    String render();
}
```

```java
// File: code/lld/ch09/proxy/RealReport.java
package lld.ch09.proxy;

/** REAL SUBJECT: expensive constructor. */
final class RealReport implements Report {
    private final String name;
    private final int rows;

    RealReport(String name) {
        System.out.println("    [real] loading 100 MB of data for '" + name + "'...");
        this.name = name;
        this.rows = 1_000_000;
    }

    @Override public String title()  { return name; }
    @Override public String render() { return name + " (" + rows + " rows)"; }
}
```

```java
// File: code/lld/ch09/proxy/LazyReport.java
package lld.ch09.proxy;

/**
 * VIRTUAL PROXY: answers title() itself, builds the RealReport on first render().
 * Thread-safe lazy init with double-checked locking (field must be volatile).
 */
final class LazyReport implements Report {
    private final String name;
    private volatile RealReport real;      // created by the proxy itself, not handed in

    LazyReport(String name) { this.name = name; }

    @Override public String title() { return name; }   // no load needed

    @Override
    public String render() { return realReport().render(); }

    private RealReport realReport() {
        RealReport r = real;
        if (r == null) {
            synchronized (this) {
                r = real;
                if (r == null) real = r = new RealReport(name);
            }
        }
        return r;
    }
}
```

```java
// File: code/lld/ch09/proxy/Role.java
package lld.ch09.proxy;

enum Role { VIEWER, FINANCE, ADMIN }
```

```java
// File: code/lld/ch09/proxy/AuditedReport.java
package lld.ch09.proxy;

import java.util.Set;

/** PROTECTION PROXY: checks permission, may REFUSE the call. */
final class AuditedReport implements Report {
    private final Report inner;
    private final Role callerRole;
    private final Set<Role> allowed;

    AuditedReport(Report inner, Role callerRole, Set<Role> allowed) {
        this.inner = inner;
        this.callerRole = callerRole;
        this.allowed = Set.copyOf(allowed);
    }

    @Override public String title() { return inner.title(); }  // titles are public

    @Override
    public String render() {
        if (!allowed.contains(callerRole)) {
            throw new SecurityException(callerRole + " may not render " + inner.title());
        }
        return inner.render();
    }
}
```

```java
// File: code/lld/ch09/proxy/RateService.java
package lld.ch09.proxy;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Subject for the caching, remote and dynamic-proxy demos. Money and rates are BigDecimal, never double. */
interface RateService {
    BigDecimal rate(String from, String to);
    BigDecimal convert(BigDecimal amount, String from, String to);

    /** Shared rounding rule: amounts are returned to 2 decimal places. */
    static BigDecimal toMoney(BigDecimal value) { return value.setScale(2, RoundingMode.HALF_EVEN); }
}
```

```java
// File: code/lld/ch09/proxy/LiveRateService.java
package lld.ch09.proxy;

import java.math.BigDecimal;
import java.math.MathContext;
import java.util.Map;

/** Real subject: pretend each call is a slow lookup. */
class LiveRateService implements RateService {
    private static final Map<String, BigDecimal> PER_USD = Map.of(
            "USD", new BigDecimal("1"), "EUR", new BigDecimal("0.92"), "INR", new BigDecimal("83"));
    int lookups = 0;

    @Override
    public BigDecimal rate(String from, String to) {
        lookups++;
        return PER_USD.get(to).divide(PER_USD.get(from), MathContext.DECIMAL64);
    }

    @Override
    public BigDecimal convert(BigDecimal amount, String from, String to) {
        return RateService.toMoney(amount.multiply(rate(from, to)));   // self-call: goes to THIS object, not
through any proxy
    }
}
```

```java
// File: code/lld/ch09/proxy/CachingRateProxy.java
package lld.ch09.proxy;

import java.math.BigDecimal;
import java.util.LinkedHashMap;
import java.util.Map;

/** CACHING PROXY: bounded (LRU) and time-limited, because an unbounded cache is a leak. */
final class CachingRateProxy implements RateService {
    private record CachedRate(BigDecimal value, long expiresAt) { }

    private final RateService inner;
    private final long ttlMillis;
    private final Map<String, CachedRate> cache;

    CachingRateProxy(RateService inner, int maxEntries, long ttlMillis) {
        this.inner = inner;
        this.ttlMillis = ttlMillis;
        this.cache = new LinkedHashMap<>(16, 0.75f, true) {   // access order = LRU
            @Override
            protected boolean removeEldestEntry(Map.Entry<String, CachedRate> eldest) {
                return size() > maxEntries;
            }
        };
    }

    @Override
    public synchronized BigDecimal rate(String from, String to) {
        String key = from + "->" + to;
        CachedRate e = cache.get(key);
        long now = System.currentTimeMillis();
        if (e != null && e.expiresAt() > now) return e.value();
        BigDecimal v = inner.rate(from, to);
        cache.put(key, new CachedRate(v, now + ttlMillis));
        return v;
    }

    @Override
    public BigDecimal convert(BigDecimal amount, String from, String to) {
        return RateService.toMoney(amount.multiply(rate(from, to)));
    }
}
```

```java
// File: code/lld/ch09/proxy/Network.java
package lld.ch09.proxy;

/*
 * REMOTE PROXY: looks like a local RateService, but marshals each call into a
 * message and sends it across a "network". Here the network is simulated in-process.
 */
interface Network {
    String send(String request);
}
```

```java
// File: code/lld/ch09/proxy/RateServer.java
package lld.ch09.proxy;

import java.math.BigDecimal;

/** Server side "skeleton": unmarshals the request and calls the real object. */
final class RateServer implements Network {
    private final RateService real;

    RateServer(RateService real) { this.real = real; }

    @Override
    public String send(String request) {                  // format: "rate|FROM|TO"
        String[] p = request.split("\\|");
        return real.rate(p[1], p[2]).toPlainString();       // BigDecimal -> exact text on the wire
    }
}
```

```java
// File: code/lld/ch09/proxy/RemoteRateProxy.java
package lld.ch09.proxy;

import java.math.BigDecimal;

/** Client side "stub" = the remote proxy. */
final class RemoteRateProxy implements RateService {
    private final Network network;

    RemoteRateProxy(Network network) { this.network = network; }

    @Override
    public BigDecimal rate(String from, String to) {
        String request = "rate|" + from + "|" + to;
        System.out.println("    [stub] sending '" + request + "' over the wire");
        return new BigDecimal(network.send(request));
    }

    @Override
    public BigDecimal convert(BigDecimal amount, String from, String to) {
        return RateService.toMoney(amount.multiply(rate(from, to)));
    }
}
```

```java
// File: code/lld/ch09/proxy/Device.java
package lld.ch09.proxy;

/** A heavyweight resource, e.g. a connection to a device. */
interface Device extends AutoCloseable {
    String query(String command);
    @Override void close();
}
```

```java
// File: code/lld/ch09/proxy/RealDevice.java
package lld.ch09.proxy;

final class RealDevice implements Device {
    RealDevice() { System.out.println("    [device] opened (expensive)"); }
    @Override public String query(String command) { return "ok:" + command; }
    @Override public void close() { System.out.println("    [device] closed"); }
}
```

```java
// File: code/lld/ch09/proxy/SharedDevice.java
package lld.ch09.proxy;

/**
 * SMART REFERENCE PROXY: reference counting. Each client gets its own handle (a proxy).
 * The real device opens on the first handle and closes when the LAST handle closes.
 * Thread-safe: every read or write of 'refs', 'real' and a handle's 'closed' flag
 * happens while holding the SharedDevice's lock, so the count can never go wrong.
 */
final class SharedDevice {
    private int refs;              // guarded by 'this'
    private RealDevice real;       // guarded by 'this'

    synchronized Device acquire() {
        if (refs++ == 0) real = new RealDevice();
        System.out.println("    [ref] handle acquired, count=" + refs);
        return new Handle();
    }

    private final class Handle implements Device {
        private boolean closed;    // guarded by SharedDevice.this

        @Override
        public String query(String command) {
            synchronized (SharedDevice.this) {
                if (closed) throw new IllegalStateException("handle already closed");
                return real.query(command);
            }
        }

        @Override
        public void close() {
            synchronized (SharedDevice.this) {
                if (closed) return;          // closing a handle twice must not double-decrement
                closed = true;
                refs--;
                System.out.println("    [ref] handle released, count=" + refs);
                if (refs == 0) {
                    real.close();
                    real = null;
                }
            }
        }
    }
}
```

```java
// File: code/lld/ch09/proxy/LoggingHandler.java
package lld.ch09.proxy;

import java.lang.reflect.InvocationHandler;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;
import java.util.Arrays;

/**
 * DYNAMIC PROXY: one InvocationHandler that logs calls on ANY interface.
 * java.lang.reflect.Proxy generates the implementing class at run time.
 */
final class LoggingHandler implements InvocationHandler {
    private final Object target;

    private LoggingHandler(Object target) { this.target = target; }

    @SuppressWarnings("unchecked")
    static <T> T wrap(T target, Class<T> iface) {
        return (T) Proxy.newProxyInstance(
                iface.getClassLoader(), new Class<?>[] {iface}, new LoggingHandler(target));
    }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        String call = method.getName() + (args == null ? "()" : Arrays.toString(args));
        System.out.println("    [proxy] -> " + call);
        try {
            Object result = method.invoke(target, args);   // forward to the real object
            System.out.println("    [proxy] <- " + call + " returned " + result);
            return result;
        } catch (InvocationTargetException e) {
            throw e.getCause();          // rethrow the real exception, not the reflection wrapper
        }
    }
}
```

```java
// File: code/lld/ch09/proxy/Main.java
package lld.ch09.proxy;

import java.lang.reflect.Proxy;
import java.math.BigDecimal;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        System.out.println("== Virtual proxy: pay only when needed ==");
        Report report = new LazyReport("Q3 revenue");
        System.out.println("  created proxy; title = " + report.title());
        System.out.println("  render #1 -> " + report.render());
        System.out.println("  render #2 -> " + report.render());

        System.out.println("== Protection proxy wrapping the virtual proxy ==");
        Set<Role> allowed = Set.of(Role.FINANCE, Role.ADMIN);
        Report forFinance = new AuditedReport(new LazyReport("Payroll"), Role.FINANCE, allowed);
        Report forViewer = new AuditedReport(new LazyReport("Payroll"), Role.VIEWER, allowed);
        System.out.println("  finance -> " + forFinance.render());
        try {
            forViewer.render();
        } catch (SecurityException e) {
            System.out.println("  viewer  -> refused: " + e.getMessage() + " (nothing was loaded)");
        }

        System.out.println("== Caching proxy (LRU, max 2 entries, 60 s TTL) ==");
        LiveRateService live = new LiveRateService();
        RateService cached = new CachingRateProxy(live, 2, 60_000);
        cached.rate("USD", "EUR");
        cached.rate("USD", "EUR");
        cached.rate("USD", "INR");
        cached.rate("EUR", "INR");   // evicts USD->EUR (least recently used)
        cached.rate("USD", "EUR");
        System.out.println("  5 calls, real lookups = " + live.lookups);

        System.out.println("== Remote proxy (stub) ==");
        RateService remote = new RemoteRateProxy(new RateServer(new LiveRateService()));
        System.out.println("  100 USD = " + remote.convert(new BigDecimal("100"), "USD", "INR") + " INR");

        System.out.println("== Smart reference (reference counting) ==");
        SharedDevice printer = new SharedDevice();
        try (Device a = printer.acquire(); Device b = printer.acquire()) {
            System.out.println("  a -> " + a.query("status") + ", b -> " + b.query("print"));
        }

        System.out.println("== Dynamic proxy: one handler, any interface ==");
        Report loggedReport = LoggingHandler.wrap(new LazyReport("Q4 plan"), Report.class);
        loggedReport.title();
        loggedReport.render();
        RateService loggedRates = LoggingHandler.wrap(new LiveRateService(), RateService.class);
        loggedRates.convert(new BigDecimal("100"), "USD", "EUR");
        System.out.println("  ^ convert() called rate() on 'this', so the inner rate() call was NOT logged");
        System.out.println("  generated at run time: isProxyClass = "
                + Proxy.isProxyClass(loggedRates.getClass()));
    }
}
```

Output:

```
== Virtual proxy: pay only when needed ==
  created proxy; title = Q3 revenue
    [real] loading 100 MB of data for 'Q3 revenue'...
  render #1 -> Q3 revenue (1000000 rows)
  render #2 -> Q3 revenue (1000000 rows)
== Protection proxy wrapping the virtual proxy ==
    [real] loading 100 MB of data for 'Payroll'...
  finance -> Payroll (1000000 rows)
  viewer  -> refused: VIEWER may not render Payroll (nothing was loaded)
== Caching proxy (LRU, max 2 entries, 60 s TTL) ==
  5 calls, real lookups = 4
== Remote proxy (stub) ==
    [stub] sending 'rate|USD|INR' over the wire
  100 USD = 8300.00 INR
== Smart reference (reference counting) ==
    [device] opened (expensive)
    [ref] handle acquired, count=1
    [ref] handle acquired, count=2
  a -> ok:status, b -> ok:print
    [ref] handle released, count=1
    [ref] handle released, count=0
    [device] closed
== Dynamic proxy: one handler, any interface ==
    [proxy] -> title()
    [proxy] <- title() returned Q4 plan
    [proxy] -> render()
    [real] loading 100 MB of data for 'Q4 plan'...
    [proxy] <- render() returned Q4 plan (1000000 rows)
    [proxy] -> convert[100, USD, EUR]
    [proxy] <- convert[100, USD, EUR] returned 92.00
  ^ convert() called rate() on 'this', so the inner rate() call was NOT logged
  generated at run time: isProxyClass = true
```

What each part does

- `LazyReport` — **virtual proxy**. `title()` is answered locally; `render()` creates the `RealReport` on first use. The "loading" line appears **once**, only at the first render.
- **Thread-safe lazy init.** Without care, two threads calling `render()` at the same time both build the expensive object (harmless for a report, but a real bug if construction opens a connection or charges a card). We use **double-checked locking** (check without a lock, then lock and check again — see Chapter 8); the field **must be** `volatile` (a Java keyword that makes a write immediately visible to every thread) so a second thread never sees a half-constructed object.
- `AuditedReport` — **protection proxy**, wrapping a virtual proxy. The viewer is refused **before** anything is loaded — notice no second "loading 'Payroll'" line for the viewer.
- `RateService` works in `BigDecimal` (Java's exact decimal number type) — rates and money are never `double` — and `convert` rounds to 2 decimal places (`HALF_EVEN`, "banker's rounding": an exact half goes to the even digit).
- `CachingRateProxy` — **caching proxy**, **bounded** (LRU via `LinkedHashMap` in access order with `removeEldestEntry`) and **time-limited** (TTL). 5 calls → 4 real lookups: the second USD→EUR is a hit, the last one is a miss because it was evicted.
- `RemoteRateProxy` + `RateServer` — **remote proxy** (client "stub") and server "skeleton". The caller just calls `convert`; the stub turns it into a message and sends it over a (simulated) network.
- `SharedDevice` — **smart reference**. Each `acquire()` returns a lightweight handle proxy; the real device opens with the first handle and closes when the last handle closes. All bookkeeping (`refs`, `real`, each handle's `closed` flag) happens under the `SharedDevice` lock, and `close()` is idempotent (calling it twice has the same effect as calling it once), so concurrent or double closes can't corrupt the count.
- `LoggingHandler` — a **dynamic proxy** (next section).

Static vs dynamic proxies

	Static proxy	Dynamic proxy
How it's made	You hand-write a class per interface (`LazyReport`, `CachingRateProxy`)	Generated at **run time** from an interface list + a handler
Code per interface	One proxy class each; every method written out	One `InvocationHandler` for **any** interface
Type safety	Full; compiler checks each method	Calls arrive as `Method` + `Object[]` — reflective, less checked
Good for	Logic specific to one type (lazy load, access rules for a domain object)	**Cross-cutting concerns** (needs shared by many classes) on many types: logging, timing, transactions, security
JDK API	—	`java.lang.reflect.Proxy.newProxyInstance(loader, interfaces, handler)`
Limitation	Boilerplate	JDK proxies work **only for interfaces**; class-based proxies need bytecode (compiled JVM instructions) libraries (CGLIB, ByteBuddy)

How the JDK dynamic proxy works:

1. `Proxy.newProxyInstance(classLoader, new Class<?>[]{Report.class}, handler)` generates a class (named like `$Proxy0`) that implements `Report`.
2. **Every** method call on that object is routed to `handler.invoke(proxy, method, args)`.
3. The handler does its extra work and calls `method.invoke(target, args)` to forward.
4. If the target throws, reflection wraps it in `InvocationTargetException` — **unwrap it** (`e.getCause()`) so callers see the real exception.

The self-invocation trap (and Spring AOP)

Look at the last lines of the proxy output: we called `convert` through the logging proxy, and `convert` internally called `rate(...)` on `this` — the real object — so that inner call **bypassed the proxy** and was never logged.

This is exactly how **Spring AOP** behaves:

- Spring implements annotations (`@` tags on code that it reads) such as `@Transactional` (run this method inside a database transaction — all its changes succeed or fail together), `@Cacheable`, `@Async`, `@PreAuthorize` and custom `@Aspect` advice (the extra code AOP runs around a method) by wrapping your bean (an object the Spring container creates and manages) in a **proxy**. Callers get the proxy injected, not your object.
- Spring uses a **JDK dynamic proxy** when the target implements at least one interface, and a **CGLIB** subclass proxy otherwise (or when `proxyTargetClass = true`; Spring Boot 2.0+ defaults to CGLIB class-based proxies).
- **Self-invocation is not intercepted**: if `placeOrder()` calls `this.saveAudit()` which is `@Transactional(propagation = REQUIRES_NEW)`, the annotation is silently ignored because the call never leaves the object.
- With CGLIB proxies, `final` **classes cannot be proxied and** `final`/`private` **methods cannot be advised**.
- Fixes, in order of preference: move the method into another bean; inject the bean's own proxy (e.g. a lazily injected self-reference); or (discouraged) `AopContext.currentProxy()`, which also requires `exposeProxy = true`. AspectJ weaving (inserting the extra code directly into the compiled classes, at compile or load time) avoids the problem because it doesn't use proxies.

Proxy vs Decorator

This is the hardest pair in the catalogue to tell apart: both implement an interface, hold another object of that interface and forward calls to it. GoF itself says the implementations can be similar and that the difference is **purpose**: a decorator *adds responsibilities*, a proxy *controls access*.

	Proxy	Decorator
Purpose	**Controls access** to the object	**Adds behaviour** around the object
Does the call get through?	Not necessarily — may defer, cache or refuse	Normally yes — the added work runs around a forwarded call
Who creates / finds the real object?	**Often the proxy itself** (`LazyReport` builds `RealReport`; a remote proxy only knows an address)	**The caller**, who hands it in
Does the client know it has a wrapper?	Usually not — a factory/DI container hands it the proxy as if it were the real object	Usually yes — the client (or its configuration) builds the stack
How many layers?	Usually one	Often several, stacked recursively on purpose

The **creation test** is the most practical, because you can check it in the constructor: `LazyReport(String name)` builds its own subject; `LoggingRepository(UserRepository inner)` is given an existing one. A protection proxy such as `AuditedReport` *is* handed its subject, and is structurally identical to a decorator — there, only intent separates them. A caching wrapper genuinely sits on the line — both names are defensible — so the more useful question in review is whether its cache is bounded and invalidated.

Where proxies go wrong

- **Unsynchronised lazy creation** — two threads call `render()` together and both construct the expensive object: sometimes harmless, sometimes a duplicate connection or a double charge.
- **A caching proxy with no size limit or expiry** — a caching proxy *is* a cache, and every cache needs a bound and an invalidation story; without them it leaks memory and serves stale data. (Ours has both.)
- **Concealing a real cost** — a remote proxy that looks local tempts callers to use it inside loops. Network calls disguised as field access cause some of the nastiest distributed-system (many machines cooperating over a network) performance bugs; the cure is usually to make the cost visible again (batch APIs that fetch many items in one call, explicit `fetch...` names), not to tune the proxy.
- **ORM N+1 queries** (an ORM, or object-relational mapper, turns database rows into objects; N+1 means one query for the list plus one more per item) — iterating 100 orders and touching `order.getCustomer().getName()` fires one query per lazy proxy.

Trade-offs

Pros	Cons
Pay later — costly construction happens only if something needs it.	**Extra indirection**; stack traces show the proxy, not the real class.
No caller changes — the interface is identical (Open/Closed).	**Latency and failures are disguised** (latency = how long a call takes) as an ordinary method call.
Somewhere to put cross-cutting rules for a class you cannot edit.	**Lazy creation brings a thread-safety question** that is easy to get wrong.
Permission checks stay outside the object, which keeps to its own job.	**A cache with no eviction** leaks memory and returns stale data.

When not to use it

- **The real object is cheap** — making something small lazy only adds a null check to every call.
- **Callers need to see the cost** — sometimes the honest interface is a method literally named `fetchOverNetwork()`.
- **The framework already generates one** — ORMs, DI containers and service meshes (a layer of proxies placed next to every service to manage the traffic between them) create proxies; stacking a hand-written one on top of those just doubles the hidden behaviour.
- **You mean to add behaviour, not guard access** — that is a Decorator.

Common mistakes

- Non-`volatile` field in double-checked locking.
- Forgetting to unwrap `InvocationTargetException` in a dynamic proxy.
- Expecting Spring annotations to work on self-calls, `private` methods or `final` methods.

- Unbounded caching proxies.
- Using a remote proxy in a tight loop.

Real-world examples

- **ORM lazy loading** (Hibernate/JPA — JPA is Java's standard ORM API — also Entity Framework and Django) — `entityManager.getReference(...)` and lazy associations return proxy objects; the SQL (database query) runs the first time you touch a non-id field. The most widely used virtual proxy — and the source of the N+1 problem and, in Hibernate, `LazyInitializationException` when the session (Hibernate's open unit of work with the database) is already closed.
- **Spring AOP** — `@Transactional`, `@Cacheable`, `@Async`, method security: all proxies. Spring's `@Configuration` classes are also CGLIB-enhanced subclasses so that calling one `@Bean` method from another returns the container's singleton (unless `proxyBeanMethods = false`).
- **Remote stubs** — Java RMI stubs, gRPC and Thrift generated clients, SOAP clients: a network call behind an ordinary method.
- **`java.lang.reflect.Proxy`** — JDK support for dynamic proxies (used by many frameworks, e.g. Spring Data repositories and MyBatis mappers are interfaces whose implementations are generated proxies).
- **`Collections.unmodifiableList`** — a protection proxy that refuses mutation (also seen as a decorator).
- **Service meshes / sidecars** (Envoy in Istio) — intercept traffic for retries, mTLS (mutual TLS: both sides prove their identity and the traffic is encrypted) and telemetry (metrics and traces sent for monitoring): the proxy idea at infrastructure level. **Reverse proxies / CDNs** (Nginx, Cloudflare) cache and protect origin servers (your own servers holding the original content). A reverse proxy is a server that sits in front of yours and receives requests on their behalf; a CDN (content delivery network) caches content on servers close to users — see System Design Fundamentals.
- **C++ `shared_ptr`** — a smart-reference proxy doing reference counting.

In modern languages

- **Java**: `java.lang.reflect.Proxy`, plus bytecode generation (CGLIB, ByteBuddy) for classes.
- **JavaScript/TypeScript**: built-in `Proxy` intercepts property access and calls.
- **Python**: `__getattr__` forwards anything not defined locally, so a proxy only writes the methods it intercepts; `functools.cached_property` is a virtual proxy for one field.
- **C#**: `DispatchProxy`, and `Lazy<T>` — a virtual proxy for one value.
- **C++**: `std::shared_ptr` and other smart pointers are smart-reference proxies that count references behind `operator->`.

Other patterns it is confused with

Looks like	Actual difference
Decorator	Same structure; see the table above. Decorator adds behaviour to an object it is handed; Proxy controls access and often creates or locates the real object itself.
Adapter	An Adapter has a **different** interface on purpose — it exists to translate. A Proxy has the **same** interface — it exists to substitute.
Facade	A Facade offers a new, simpler interface over **several** objects. A Proxy offers the same interface over **one**.
A cache	A caching proxy is a cache with a proxy's shape. Calling it a proxy does not remove the need to design its size bound and eviction.

In the interview

30-second answer: "A Proxy is a stand-in with the same interface as the real object that controls access to it. I'd name the kind: a virtual proxy defers expensive creation, a protection proxy checks permissions, a remote proxy hides the network, a caching proxy reuses results, and a smart reference does bookkeeping like reference counting. Because the interface is identical, callers don't change — which is also the danger, since an expensive call looks cheap. Lazy creation needs thread safety, and ORM lazy-loading proxies are the classic source of N+1 queries. In Java I'd use `java.lang.reflect.Proxy` for cross-cutting concerns; that's how Spring AOP implements `@Transactional`."

Follow-ups

- *Q: Proxy vs Decorator?* — A: Proxy controls access and often creates the real object; Decorator adds behaviour to an object handed to it and normally forwards every call. Check the constructor.
- *Q: Make the virtual proxy thread-safe.* — A: Double-checked locking with a `volatile` field (as in `LazyReport`), or simply make the getter `synchronized` if contention (threads waiting on each other for the lock) is low. (The static holder-class idiom only works for a *static* lazy field, so it does not fit a per-instance proxy.)
- *Q: Why doesn't `@Transactional` work when I call the method from the same class?* — A: Spring's proxy only intercepts calls that come through the proxy; `this.method()` goes straight to the target.
- *Q: JDK proxy vs CGLIB?* — A: JDK proxies implement interfaces only; CGLIB creates a subclass, so it works without interfaces but can't handle `final` classes/methods.
- *Q: How does lazy loading cause N+1?* — A: Loading N parents and then touching each parent's lazy child proxy triggers N extra queries; fix with fetch joins / entity graphs / batch fetching (ways to load the children together with their parents).

Self-check

1. **Q:** Which proxy kind is `LazyReport`? `AuditedReport`? **A:** Virtual; protection.
2. **Q:** Why is the field in `LazyReport` `volatile`? **A:** So double-checked locking is safe — other threads see a fully constructed object.
3. **Q:** In the dynamic proxy demo, why wasn't the inner `rate()` call logged? **A:** `convert()` called `rate()` on `this` (the real object), bypassing the proxy — the self-invocation trap.
4. **Q:** Can `java.lang.reflect.Proxy` proxy a class with no interface? **A:** No; it needs interfaces. Use CGLIB/ByteBuddy for classes.
5. **Q:** What two properties must a caching proxy have? **A:** A size bound (eviction) and an invalidation/expiry policy.

TL;DR

- **Same interface, controlled access.** Virtual, protection, remote, caching, logging, smart reference.
- **Static** = hand-written per interface; **dynamic** = `java.lang.reflect.Proxy` + `InvocationHandler` for any interface.
- Spring AOP = proxies → beware self-invocation, `final`, `private`. Watch thread safety and hidden costs.

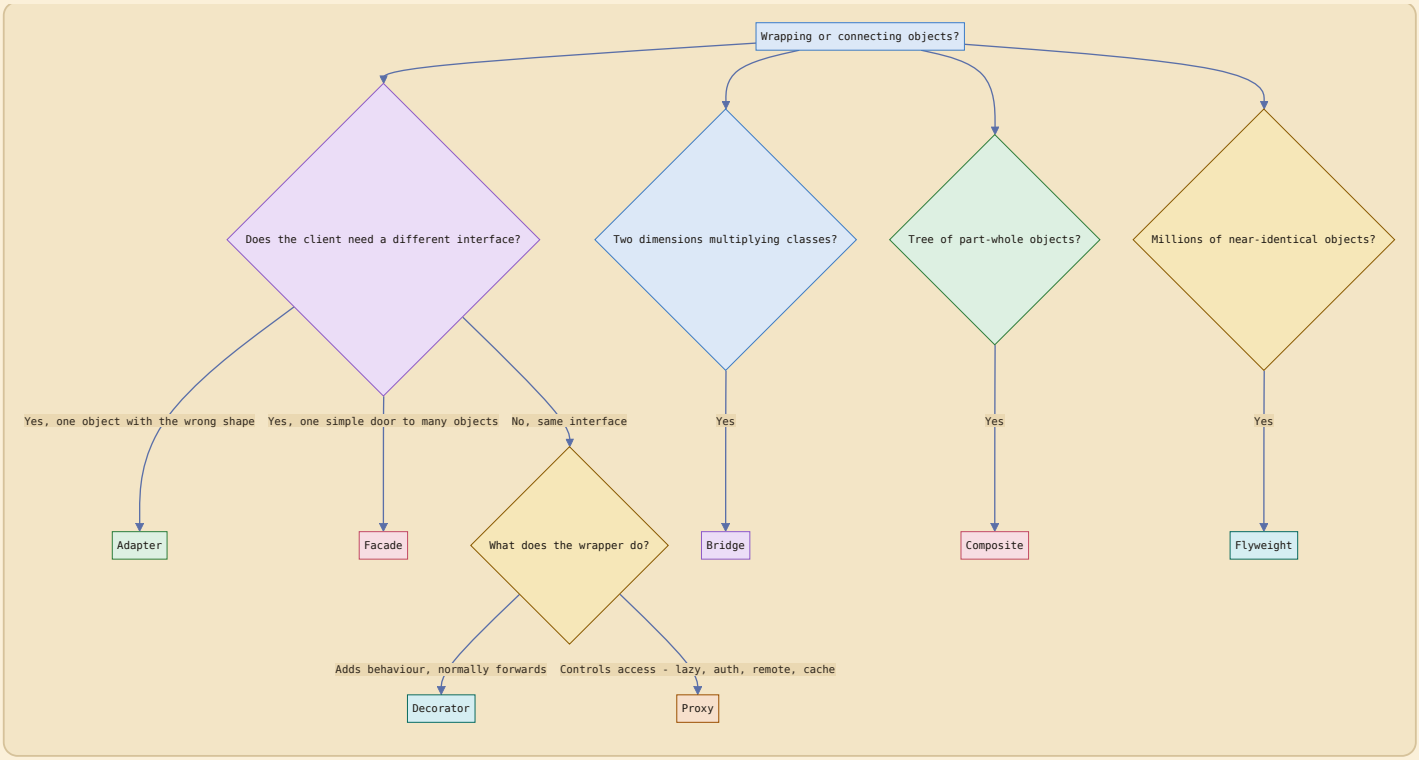
9.8 Putting it all together — comparison

Summary of all seven structural patterns

Pattern	Distinctive feature	Use it when	Example	Pros	Cons
Adapter	Bridges incompatible interfaces	You must use an existing class whose interface doesn't fit your code	Travel plug adapter; `StripeGateway`	Interoperability; reuse of existing code	Extra class and complexity
Bridge	Separates abstraction from implementation	Two dimensions vary independently; avoid permanent binding	UI shapes × platform renderers; notifications × channels	Both sides vary independently; scales by addition	More concepts up front; indirection
Composite	Part-whole tree with one interface	Treat single objects and groups uniformly	Drawing editor with shapes and groups; file system	Simpler client code; easy to add node types	Interface can become over-general
Decorator	Adds responsibilities dynamically	Extend an object's behaviour at run time, in combinations	Adding scrolling/border to a text view; Java I/O	More flexible than subclassing; no feature-bloated classes	Many small objects; hard to debug; order matters
Facade	Simple interface to a complex subsystem	Most callers need one common path through many classes	Home-theatre "watch movie" button; `OrderFacade`	Simplifies use; fewer dependencies	Can become a god object or single point of failure
Flyweight	Shares common state among many objects	Huge numbers of similar objects with little variation	Characters in a word processor; map markers	Big memory savings	Complexity; managing shared immutable state
Proxy	Placeholder that controls access	Lazy loading, access control, remote, caching, logging, smart reference	Internet proxy server; Hibernate lazy entity	Controls access; defers costly work	Indirection; hidden latency

Adapter vs Decorator vs Proxy vs Facade vs Bridge

	Adapter	Decorator	Proxy	Facade	Bridge
Intent	Convert an interface to the one the client expects	Add behaviour without changing the interface	Control access to an object	Simplify a complex subsystem	Let abstraction and implementation vary independently
Interface seen by client	**Different** from the wrapped object's (it's yours)	**Same** as wrapped	**Same** as real subject	**New, simpler** interface	Abstraction's own interface
Behaviour	Unchanged (translated)	**Extended**	Unchanged but **controlled** (may defer/cache/refuse)	Unchanged (fewer calls)	Delegated to implementor
Objects wrapped	One	One (stackable)	One	**Several**	One implementor (by reference)
Who creates the wrapped object	Usually passed in	Passed in by client	**Often the proxy itself**	Passed in / owned	Passed in at construction
Structure	Implements Target, holds Adaptee	IS-A and HAS-A Component	IS-A Subject, HAS-A Real Subject	HAS-A many subsystem objects	Abstraction HAS-A Implementor; two hierarchies
When introduced	**After the fact**, to make existing things fit	Any time a concern is added	Any time access needs controlling	When callers repeat a multi-step sequence	**Up front**, before the class explosion
Typical trigger	"The vendor API doesn't match ours"	"Sometimes we need logging, sometimes caching, sometimes both"	"It's expensive / remote / restricted"	"Every caller copies these 5 steps"	"Class names with two nouns"
JDK / framework example	`Arrays.asList`, `InputStreamReader`	`BufferedInputStream`, `Collections.synchronizedList`	`java.lang.reflect.Proxy`, Spring `@Transactional`, Hibernate lazy load	SLF4J `LoggerFactory`, service layers	JDBC `Connection` + drivers, AWT peers



Memory hooks

- **Adapter** — *"make it fit"* (changes the interface).
- **Decorator** — *"add to it"* (same interface, more behaviour, stackable).
- **Proxy** — *"guard it"* (same interface, controlled access).
- **Facade** — *"simplify it"* (one door, many rooms).
- **Bridge** — *"split it"* (two hierarchies, add instead of multiply).
- **Composite** — *"nest it"* (one interface for leaf and tree).
- **Flyweight** — *"share it"* (intrinsic shared, extrinsic passed in).

Chapter self-check

1. **Q:** Which of these patterns wrap exactly one object? **A:** Adapter, Decorator, Proxy (Bridge also holds one implementor, but by design rather than as a wrapper). Facade wraps several; Composite holds many children.
2. **Q:** Which two have identical code shape but differ in intent and timing? **A:** Adapter and Bridge (after-the-fact vs up-front); also Bridge and Strategy; also Decorator and Proxy.
3. **Q:** A wrapper returns cached results with the same interface. Decorator or Proxy? **A:** Either is defensible; the important question is whether the cache is bounded and invalidated.
4. **Q:** You have `WindowsButton`, `MacButton`, `WindowsCheckbox`, `MacCheckbox`… and a Linux port is coming. Which pattern? **A:** Bridge (widget abstraction × platform implementor) — often combined with Abstract Factory to pick the implementor.
5. **Q:** Which pattern explains why `Integer.valueOf(100) == Integer.valueOf(100)` is true? **A:** Flyweight (Integer cache).

Chapter TL;DR

- Structural patterns **compose objects into bigger structures** while keeping them flexible.
- **Adapter / Decorator / Proxy / Facade** share a shape; tell them apart by **what changes as the call passes through** and **how many objects are wrapped**.
- **Bridge** prevents class explosion across two dimensions; **Composite** makes trees uniform; **Flyweight** shares immutable state to save memory.
- In interviews: name the pattern, state the **constraint** that justifies it, name its **trap** (vendor leaks, wrong split, cycles, decorator order, god facade, mutable flyweight, self-invocation), and give a **real JDK/Spring example**.

Chapter 10 — Behavioral Design Patterns (Part 1)

> **What you will learn.** What behavioral patterns are, and six of them in depth: **Chain of Responsibility, Command, Iterator, Mediator, Memento** and **Interpreter**. For each one you get a plain definition, the problem it fixes, the class diagram, Java 21 code that compiles and runs, real products and JDK (Java Development Kit — Java's compiler, tools and standard library) classes that use it, and interview questions with answers. Chapter 11 covers the other five GoF behavioral patterns (Observer, State, Strategy, Template Method, Visitor) and some important patterns that are not in the GoF book.

How to run the code in this chapter

```
cd chapters/10-patterns-behavioral-1/code
javac --release 21 -d /tmp/lld_10 $(find . -name '*.java')
java -cp /tmp/lld_10 lld.ch10.chain.Main        # also: command, iterator, mediator, memento, interpreter
```

10.0 Behavioral patterns: the big picture

What it is

Creational patterns (Chapter 8) are about **how objects are made**. Structural patterns (Chapter 9) are about **how objects are put together**. **Behavioral patterns** are about **how objects share work and talk to each other**: who is responsible for what, and who calls whom.

Everyday analogy: a restaurant kitchen. The structural question is "which stations exist?" The behavioral question is "when an order comes in, who hears about it first, who passes it on, who decides the order of the dishes, and who can cancel it?"

Behavioral patterns help you to:

- let objects work together in a flexible, efficient way;
- cut tight coupling (classes depending heavily on each other's internal details) between classes, so each class is easier to change and test;
- turn tangled communication between many objects into something simple to follow;
- change behaviour **while the program runs** (at run time), not only when you compile it.

The eleven GoF behavioral patterns

The "Gang of Four" (GoF) book, *Design Patterns* (Gamma, Helm, Johnson, Vlissides, 1994), lists eleven behavioral patterns. This chapter covers the first six; Chapter 11 covers the rest.

Pattern	One-line intent	Chapter
Chain of Responsibility	Hand a request down a row of handler objects; each may deal with it, stop it, or forward it.	10
Command	Wrap a request in an object, which can then be stored, queued, logged or reversed.	10
Interpreter	Represent the grammar of a small language as classes, and evaluate sentences in it.	10
Iterator	Walk through a collection one element at a time without seeing how it is stored.	10
Mediator	Route communication between many objects through one coordinator.	10
Memento	Let an object save a snapshot of its own state and later return to it, while keeping that state private.	10
Observer	When one object changes, notify all the objects that depend on it.	11
State	Let an object change its behaviour when its internal state changes.	11
Strategy	Choose one of several interchangeable algorithms at run time.	11
Template Method	Fix the skeleton of an algorithm in a base class; let subclasses fill in some steps.	11
Visitor	Add a new operation to a set of classes without changing those classes.	11

Class scope vs object scope. GoF splits behavioral patterns into two kinds. *Class* behavioral patterns use **inheritance** to share behaviour: only **Template Method** and **Interpreter**. All the others are *object* behavioral patterns: they use **composition** (objects holding references to other objects) and can therefore be rewired at run time.

One idea runs through all of them: take a piece of behaviour that is hidden inside a big method (a chain of `if`s, a hard-coded call, a loop) and turn it into an **object** you can name, pass around, swap, store and test on its own.

10.1 Chain of Responsibility

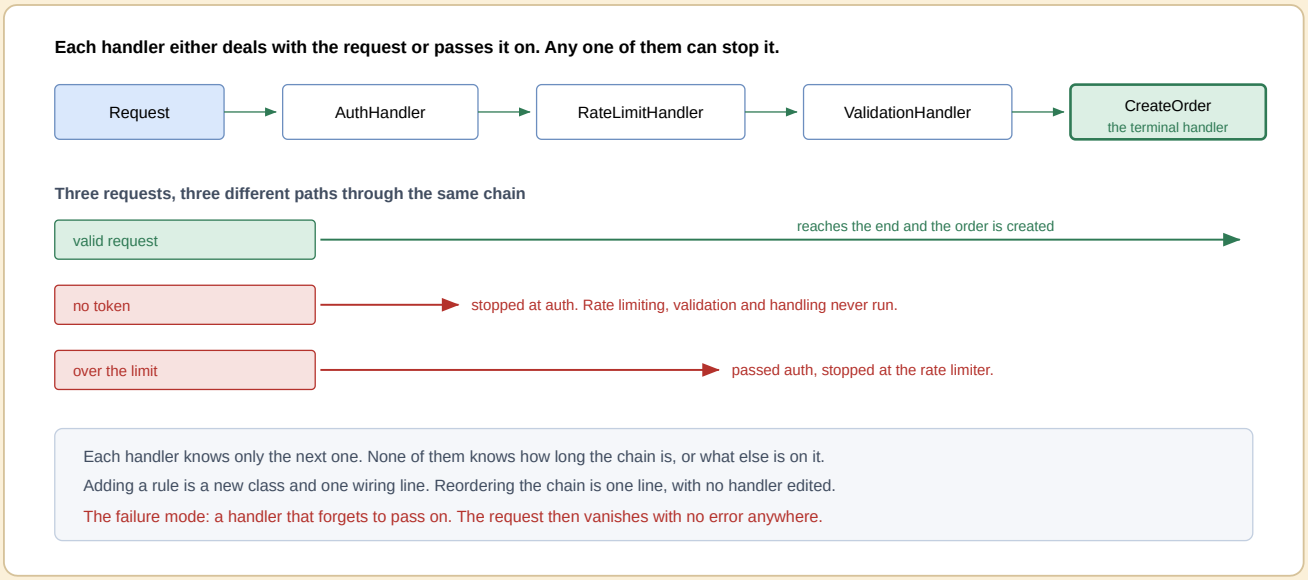


Figure — Three requests go into the same chain of handlers and each stops at a different place (course diagram)

What it is

Chain of Responsibility (CoR) sends a request through a row of **handler** objects. Every handler makes one of three choices: deal with the request, reject it (so nobody later sees it), or forward it to its successor (the next handler in the row). The sender talks only to the first handler and never learns which one finally took care of the request.

GoF intent, in plain words: *avoid tying the sender of a request to one particular receiver by giving several objects a chance to handle it; link those objects and forward the request until one handles it.* GoF

participants: **Handler** (declares the handling method, optionally holds the successor link), **ConcreteHandler** (handles what it is responsible for, otherwise forwards), and **Client** (starts the request at a handler in the chain).

Everyday analogy: a customer-support phone line. Level-1 support tries first. If they cannot help, they pass you to level 2, then to an engineer. You only ever dialled one number.

In the figure above, one chain (auth → rate limit → validation → create order) receives three requests. A valid request reaches the end. A request with no token stops at auth. A request over the limit passes auth but stops at the rate limiter. Each handler knows only the next one.

The problem it solves

Before an order is created, a request must survive several gates: who is calling (authentication), are they calling too often (rate limit), and is the body (the data sent with the request) well-formed (validation). Written directly, one method does all of them:

- The method grows every time a rule is added.
- The **order of the checks is hidden inside the method body**, yet it matters: authenticating before rate limiting is a different system from rate limiting before authenticating.
- Some routes (the URL paths a server handles, such as `/orders`) need only two of the checks. Soon there are boolean flags, then a second copy of the method that someone forgets to update.

CoR makes **each check its own small object**, and links them into a line. No handler knows how long the line is or what else is on it, so **the line itself is configuration, not logic**: a new rule costs one new class plus one wiring line; changing the order touches only the wiring; each route can be given its own line, assembled when the application starts.

How it works

1. Define a **Handler** type with a method such as `handle(request)` and a reference to the **next** handler.
2. Write one **concrete handler** per check. Each one does its own check, then either **stops** (returns a result) or **passes on** (`next.handle(request)`).
3. Put a **terminal handler** at the end that does the real work (or reports "nobody handled this").
4. **Wire** the chain once, at start-up: `auth.setNext(rateLimit).setNext(validate).setNext(createOrder)`.
5. The client calls only the **first** handler. It never knows how long the chain is.

Structure (UML)

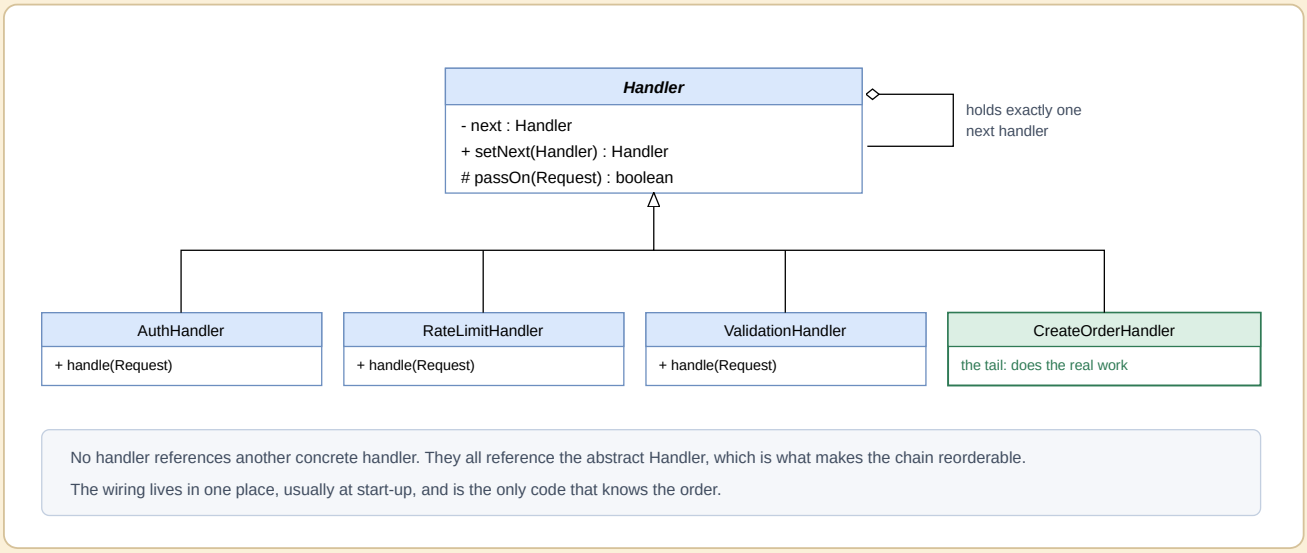
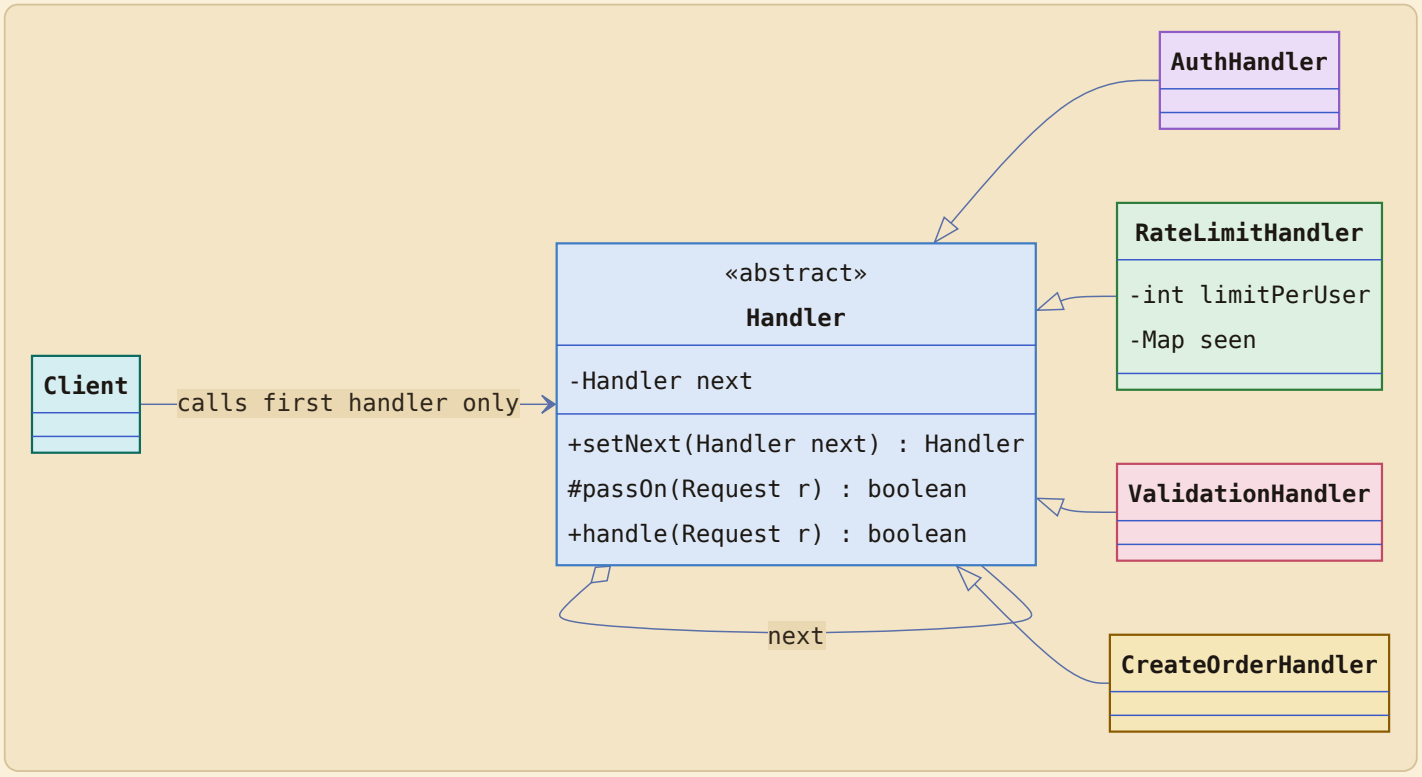


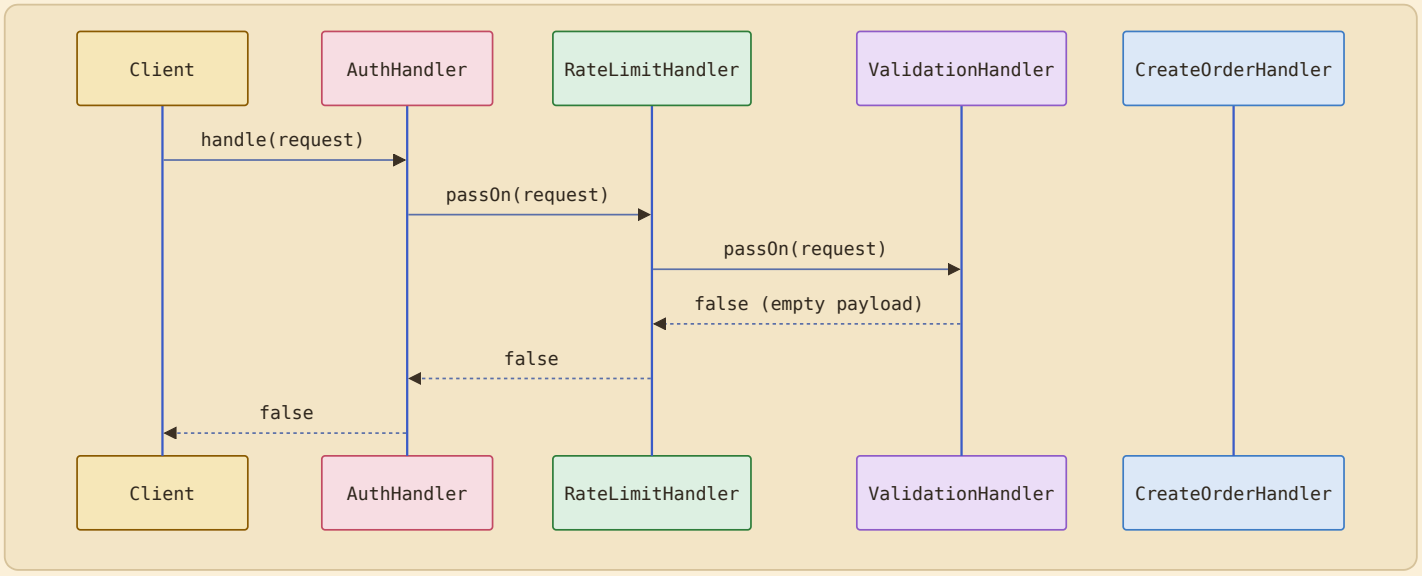
Figure — Class diagram: an abstract Handler holds `next`; concrete handlers extend it; CreateOrderHandler is the tail (course diagram)

In the class diagram above (a UML — Unified Modeling Language — diagram, the standard notation for classes and their links), notice what is **missing**: no handler refers to another *concrete* handler. Every link is typed as the abstract `Handler`, and that is why the stages can be shuffled freely. `setNext` hands back its argument, which lets the wiring be written as one fluent expression (calls chained with dots, each returning the object for the next call).

> **Our code vs the course pseudocode.** In the course version, `passOn` returns `true` ("success") when there is no next handler. Our `passOn` **throws** instead, because silently reporting success when the chain simply ran out is exactly the kind of invisible failure this pattern is prone to. Either choice is legal; just make it deliberate.

Our Java design:





Java example

The package `lld.ch10.chain` shows **four flavours** of the pattern:

1. **Classic linked handlers** (`Handler`, `AuthHandler`, `RateLimitHandler`, `ValidationHandler`, `CreateOrderHandler`). Each handler returns an *explicit* outcome: `false` = rejected, `passOn(...)` = continue. `passOn` **throws** if there is no next handler, so a chain without a terminal handler fails loudly instead of silently "succeeding".
2. **Approval chain** (`Approver`, `Expense`). A pure CoR: the *first* approver whose limit is high enough approves, and nobody after it sees the request. Note that there is only **one** class — the chain is just data (role + limit). Amounts and limits are `BigDecimal` (Java's exact decimal number type) compared with `compareTo` — never `double` for money.

3. **Servlet-style filters** (`Filter`, `FilterChain`, `Endpoint`, `HttpRequest`). This is how `jakarta.servlet.Filter` (part of the servlet API, Java's standard way to handle web requests) works: the "next" step is passed **in as an argument** (`chain.doFilter(...)`). A filter can act before *and after* the rest of the chain (see the timing filter), or stop the chain by not calling it (the auth filter).
4. **Logging levels** (`LogHandler`, `ConsoleLog`, `FileLog`, `PagerLog`, `Level`). A common *variant*: each handler acts if the message is severe enough and then **always** passes it on. Several handlers may act. Strictly, when nothing can stop the flow it is closer to a *pipeline* (a fixed series of steps that every item passes through), but textbooks often show logging this way, and it is worth knowing the difference.

```java
// File: code/lld/ch10/chain/Handler.java
package lld.ch10.chain;

/** Classic Chain of Responsibility base: each handler knows only "the next one". */
public abstract class Handler {
    private Handler next;

    /** Links the next handler and returns it, so wiring reads as one chained expression. */
    public Handler setNext(Handler next) {
        this.next = next;
        return next;
    }

    /** Forwards the request. Falling off the end means the chain has no terminal handler: a wiring bug. */
    protected boolean passOn(Request request) {
        if (next == null) {
            throw new IllegalStateException(
                getClass().getSimpleName() + " tried to pass on, but the chain has no terminal handler");
        }
        return next.handle(request);
    }

    /** @return true if the request was fully handled, false if some handler rejected it. */
    public abstract boolean handle(Request request);
}
```

```java
// File: code/lld/ch10/chain/Request.java
package lld.ch10.chain;

/** The request that travels down the API pipeline. */
public record Request(String user, String token, String payload) {}
```

```java
// File: code/lld/ch10/chain/AuthHandler.java
package lld.ch10.chain;

/** Stage 1: reject requests with no token. */
final class AuthHandler extends Handler {
    @Override
    public boolean handle(Request r) {
        if (r.token() == null || r.token().isBlank()) {
            System.out.println("  [auth] rejected: missing token");
            return false;                    // explicit outcome: the chain stops here
        }
        System.out.println("  [auth] ok for " + r.user());
        return passOn(r);                    // explicit outcome: continue
    }
}
```

```java
// File: code/lld/ch10/chain/RateLimitHandler.java
package lld.ch10.chain;

import java.util.HashMap;
import java.util.Map;

/** Stage 2: at most N requests per user (keyed by user because auth ran first). */
final class RateLimitHandler extends Handler {
    private final int limitPerUser;
    private final Map<String, Integer> seen = new HashMap<>();

    RateLimitHandler(int limitPerUser) { this.limitPerUser = limitPerUser; }

    @Override
    public boolean handle(Request r) {
        int count = seen.merge(r.user(), 1, Integer::sum);
        if (count > limitPerUser) {
            System.out.println("  [ratelimit] rejected: " + r.user() + " made " + count + " requests, limit " +
limitPerUser);
            return false;
        }
        System.out.println("  [ratelimit] ok (" + count + "/" + limitPerUser + ")");
        return passOn(r);
    }
}
```

```java
// File: code/lld/ch10/chain/ValidationHandler.java
package lld.ch10.chain;

/** Stage 3: the payload must not be empty. */
final class ValidationHandler extends Handler {
    @Override
    public boolean handle(Request r) {
        if (r.payload() == null || r.payload().isBlank()) {
            System.out.println("  [validate] rejected: empty payload");
            return false;
        }
        System.out.println("  [validate] ok");
        return passOn(r);
    }
}
```

```java
// File: code/lld/ch10/chain/CreateOrderHandler.java
package lld.ch10.chain;

/** Terminal handler: does the real work and never passes on. */
final class CreateOrderHandler extends Handler {
    @Override
    public boolean handle(Request r) {
        System.out.println("  [order] created order for " + r.user() + ": " + r.payload());
        return true;
    }
}
```

```java
// File: code/lld/ch10/chain/Expense.java
package lld.ch10.chain;

import java.math.BigDecimal;

/** An expense claim that needs someone with enough authority to approve it. Money is BigDecimal, never double. */
record Expense(String id, BigDecimal amountUsd) {}
```

```java
// File: code/lld/ch10/chain/Approver.java
package lld.ch10.chain;

import java.math.BigDecimal;

/**
 * Approval chain. Pure Chain of Responsibility: the FIRST handler able to deal
 * with the request handles it, and nobody after it sees the request.
 * Note there is only one class: the chain is data (name + limit), not code.
 */
final class Approver {
    private final String role;
    private final BigDecimal limitUsd;
    private Approver next;

    Approver(String role, BigDecimal limitUsd) {
        this.role = role;
        this.limitUsd = limitUsd;
    }

    Approver then(Approver next) {
        this.next = next;
        return next;
    }

    String approve(Expense e) {
        if (e.amountUsd().compareTo(limitUsd) <= 0) {   // compareTo, not equals: 300 and 300.00 are the same
amount
            return role + " approved " + e.id() + " ($" + e.amountUsd() + ")";
        }
        if (next == null) {   // explicit terminal outcome instead of a silent "nothing happened"
            return "REJECTED " + e.id() + " ($" + e.amountUsd() + "): above every approver's limit";
        }
        System.out.println("    " + role + " (limit $" + limitUsd + ") escalates " + e.id());
        return next.approve(e);
    }
}
```

```java
// File: code/lld/ch10/chain/HttpRequest.java
package lld.ch10.chain;

import java.util.Map;

/** Minimal HTTP request for the servlet-style demo. */
record HttpRequest(String path, Map<String, String> headers) {}
```

```java
// File: code/lld/ch10/chain/Filter.java
package lld.ch10.chain;

/** Same shape as jakarta.servlet.Filter: "next" is an explicit argument, so forgetting it is visible. */
@FunctionalInterface
interface Filter {
    void doFilter(HttpRequest request, StringBuilder response, FilterChain chain);
}
```

```java
// File: code/lld/ch10/chain/Endpoint.java
package lld.ch10.chain;

/** The endpoint at the end of the chain (like a Servlet). */
@FunctionalInterface
interface Endpoint {
    void service(HttpRequest request, StringBuilder response);
}
```

```java
// File: code/lld/ch10/chain/FilterChain.java
package lld.ch10.chain;

import java.util.List;

/** Walks the filter list. One FilterChain object per request, because it holds a position. */
final class FilterChain {
    private final List<Filter> filters;
    private final Endpoint endpoint;
    private int position = 0;

    FilterChain(List<Filter> filters, Endpoint endpoint) {
        this.filters = filters;
        this.endpoint = endpoint;
    }

    void doFilter(HttpRequest request, StringBuilder response) {
        if (position < filters.size()) {
            filters.get(position++).doFilter(request, response, this);
        } else {
            endpoint.service(request, response);   // the terminal handler
        }
    }
}
```

```java
// File: code/lld/ch10/chain/Level.java
package lld.ch10.chain;

/** Severity levels, ordered from least to most severe. */
enum Level { DEBUG, INFO, ERROR }
```

```java
// File: code/lld/ch10/chain/LogHandler.java
package lld.ch10.chain;

/**
 * Logging-level chain. A VARIANT of the pattern: each handler acts if the record
 * is severe enough, and then ALWAYS passes it on, so several handlers may act.
 * (If nobody can stop the flow, it is really a pipeline; the classic form stops.)
 */
abstract class LogHandler {
    private final Level threshold;
    private LogHandler next;

    LogHandler(Level threshold) { this.threshold = threshold; }

    LogHandler setNext(LogHandler next) {
        this.next = next;
        return next;
    }

    final void log(Level level, String message) {
        if (level.compareTo(threshold) >= 0) {
            write(level, message);
        }
        if (next != null) {
            next.log(level, message);
        }
    }

    protected abstract void write(Level level, String message);
}
```

```java
// File: code/lld/ch10/chain/ConsoleLog.java
package lld.ch10.chain;

final class ConsoleLog extends LogHandler {
    ConsoleLog() { super(Level.DEBUG); }
    @Override protected void write(Level l, String m) { System.out.println("    console <- " + l + " " + m); }
}
```

```java
// File: code/lld/ch10/chain/FileLog.java
package lld.ch10.chain;

final class FileLog extends LogHandler {
    FileLog() { super(Level.INFO); }
    @Override protected void write(Level l, String m) { System.out.println("    app.log <- " + l + " " + m); }
}
```

```java
// File: code/lld/ch10/chain/PagerLog.java
package lld.ch10.chain;

final class PagerLog extends LogHandler {
    PagerLog() { super(Level.ERROR); }
    @Override protected void write(Level l, String m) { System.out.println("    PAGER   <- " + l + " " + m); }
}
```

```java
// File: code/lld/ch10/chain/Main.java
package lld.ch10.chain;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. API request pipeline (classic linked handlers) ===");
        // Order is a design decision: auth BEFORE rate limiting so the limiter can key on the user,
        // and so anonymous traffic cannot burn a real user's quota.
        Handler pipeline = new AuthHandler();
        pipeline.setNext(new RateLimitHandler(2))
                .setNext(new ValidationHandler())
                .setNext(new CreateOrderHandler());

        List<Request> requests = List.of(
            new Request("alice", "t-123", "2 x widget"),
            new Request("bob", "", "1 x gadget"),
            new Request("alice", "t-123", " "),
            new Request("alice", "t-123", "1 x gadget"));
        for (Request r : requests) {
            System.out.println("request from " + r.user() + ":");
            System.out.println("  => handled = " + pipeline.handle(r));
        }

        System.out.println("\n=== 2. Expense approval chain (first capable handler wins) ===");
        Approver chain = new Approver("TeamLead", new BigDecimal("1000"));
        chain.then(new Approver("Manager", new BigDecimal("10000")))
             .then(new Approver("Director", new BigDecimal("50000")));
        for (Expense e : List.of(new Expense("EXP-1", new BigDecimal("300")),
                                 new Expense("EXP-2", new BigDecimal("7500")),
                                 new Expense("EXP-3", new BigDecimal("120000")))) {
            System.out.println("  " + chain.approve(e));
        }

        System.out.println("\n=== 3. Servlet-style filters (next is an explicit argument) ===");
        Filter timing = (req, res, next) -> {
            System.out.println("  [timing] start " + req.path());
            next.doFilter(req, res);                        // filters can act before AND after
            System.out.println("  [timing] end   " + req.path() + " -> " + res);
        };
        Filter auth = (req, res, next) -> {
            if (!req.headers().containsKey("Authorization")) {
                res.append("401 Unauthorized");             // does not call next: chain stops
                return;
            }
            next.doFilter(req, res);
        };
        Endpoint ordersEndpoint = (req, res) -> res.append("200 OK orders list");

        for (HttpRequest req : List.of(
                new HttpRequest("/orders", Map.of("Authorization", "Bearer abc")),
                new HttpRequest("/orders", Map.of()))) {
            new FilterChain(List.of(timing, auth), ordersEndpoint).doFilter(req, new StringBuilder());
        }

        System.out.println("\n=== 4. Logging levels (variant: every handler may act) ===");
        LogHandler logger = new ConsoleLog();
        logger.setNext(new FileLog()).setNext(new PagerLog());
        System.out.println("  log DEBUG:");
        logger.log(Level.DEBUG, "cache miss for key=42");
        System.out.println("  log ERROR:");
        logger.log(Level.ERROR, "payment service down");
    }
}
```

Output:

```
=== 1. API request pipeline (classic linked handlers) ===
request from alice:
  [auth] ok for alice
  [ratelimit] ok (1/2)
  [validate] ok
  [order] created order for alice: 2 x widget
  => handled = true
request from bob:
  [auth] rejected: missing token
  => handled = false
request from alice:
  [auth] ok for alice
  [ratelimit] ok (2/2)
  [validate] rejected: empty payload
  => handled = false
request from alice:
  [auth] ok for alice
  [ratelimit] rejected: alice made 3 requests, limit 2
  => handled = false

=== 2. Expense approval chain (first capable handler wins) ===
  TeamLead approved EXP-1 ($300)
    TeamLead (limit $1000) escalates EXP-2
  Manager approved EXP-2 ($7500)
    TeamLead (limit $1000) escalates EXP-3
    Manager (limit $10000) escalates EXP-3
  REJECTED EXP-3 ($120000): above every approver's limit

=== 3. Servlet-style filters (next is an explicit argument) ===
  [timing] start /orders
  [timing] end   /orders -> 200 OK orders list
  [timing] start /orders
  [timing] end   /orders -> 401 Unauthorized

=== 4. Logging levels (variant: every handler may act) ===
  log DEBUG:
    console <- DEBUG cache miss for key=42
  log ERROR:
    console <- ERROR payment service down
    app.log <- ERROR payment service down
    PAGER   <- ERROR payment service down
```

Walk through the output:

- Request 1 travels the full chain. Request 2 stops at auth — the rate limiter, validator and order code never run. Request 3 passes auth and the limiter (2 of 2) and stops at validation. Request 4 is the third request from alice, so the limiter stops it.
- `EXP-2` is escalated once and approved by the Manager. `EXP-3` is above every limit, so the chain gives an explicit "REJECTED" outcome rather than disappearing.
- The timing filter prints both before and after the rest of the chain. For the unauthenticated request, the auth filter wrote `401` (the HTTP status code for "not authenticated") and did not call `next`, so the endpoint never ran.
- A `DEBUG` message only reaches the console; an `ERROR` goes to all three.

Trade-offs

Pros	Cons
A new stage costs one class and one wiring line; no working class is edited (Open/Closed — open for extension, closed for modification).	To see the route a request takes you must read the wiring plus several files.
Every stage can be unit-tested (tested in isolation) on its own against a stub (a minimal stand-in) successor.	When a request is refused, you may have to step through several handlers to learn which one refused it.
Each route or environment can get its own chain, assembled at start-up.	No compiler or framework verifies that the chain is complete or correctly ordered.
The sender does not know which object will end up handling its request.	A handler that neither handles nor forwards makes the request vanish without any error.
	Each extra stage adds a call; rarely matters, but can on very hot paths (code that runs extremely often) with long chains.

The order of the stages is a design decision that no tool verifies. Auth before rate limiting means the limiter can count per *user*, and requests without valid credentials are turned away before they can use up a real user's quota (allowed number of requests). Rate limiting before auth means the limiter can only key on something like the IP address (the network address of the calling machine) — but it shields the (possibly expensive) auth step from floods (huge bursts of requests). Both are reasonable; they behave differently. **Put a comment with the reason beside the wiring** (see `Main`): the order is written down nowhere else.

When not to use it

- **Every request always goes through every stage in a fixed order.** Four calls in one method are clearer and have no wiring to get wrong.
- **Handlers need each other's results.** Then the stages are not independent: what you really have is a *pipeline* in which each step reshapes a value for the next. Call the steps one after another and pass an accumulating context object between them instead.
- **No terminal handler exists.** If the end of the chain can be reached with nothing done, failures become silent. Add an explicit terminal handler first.
- **The chain is long and on a very hot path**, where every extra call is measurable.

Common mistakes

1. **The swallowing bug.** A handler does its check and then just `return`s instead of calling `passOn`. No exception, no log; orders "sometimes don't appear". Fixes: give every path an explicit outcome (handled / rejected with reason / passed on); prefer the style where `next` is an explicit parameter (servlet filters, Express, Go `http.Handler`) so forgetting it is visible; unit-test each handler against a stub successor that remembers whether it got the request.
2. **No terminal handler.** Reaching the end should be an error or a clear "not handled" result — never a shrug.
3. **Wrong order, undocumented.** See the trade-offs above.
4. **Handlers that know about each other** (`new RateLimitHandler()` inside `AuthHandler`). The chain is then fixed in code again.
5. **Sharing a stateful chain object between requests.** The servlet-style `FilterChain` keeps a position, so it must be **one per request** (our `Main` creates a new one for each).

Real-world examples

- **Servlet filters**: `jakarta.servlet.Filter.doFilter(request, response, chain)` — calling `chain.doFilter(...)` *is* "pass on". **Spring Security** (Spring's authentication and authorisation library) is a chain of such filters (`SecurityFilterChain`).
- **Spring MVC** `HandlerInterceptor`: `preHandle` returns `false` to stop the request.
- **HTTP middleware** (code a web framework runs around every request; HTTP is the web's request/response protocol) in Express (`next()`), Django, Ruby's Rack, ASP.NET Core, Go (`func(http.Handler) http.Handler`, the usual way to structure a Go web service). **OkHttp** interceptors call `chain.proceed(request)`. **Netty**'s `ChannelPipeline` passes events from handler to handler.

- **In languages with first-class functions** (functions that can be stored and passed around like values) the chain is usually a list of `(request, next) => ...` functions rather than classes; JavaScript frameworks often build it by folding (combining a list into one value step by step) the list with `Array.reduce`, and in Python several decorators (`@` wrappers around a function) stacked on one view function (the function that handles a web request) form a chain written as syntax. Passing `next` explicitly removes the "forgot to pass on" bug by design.
- **`java.util.logging`**: a `LogRecord` is published to the logger's handlers and then, by default, to its parent logger's handlers; `setUseParentHandlers(false)` stops that. This is why setting a level or handler on a parent logger affects its children.
- **Exception handling**: each stack frame (one method call in progress) either catches the exception or lets it travel up — the language's own chain.
- **DOM event bubbling** (the DOM — Document Object Model — is the browser's tree of page elements): a click event climbs from the clicked element through its parent elements; any listener can call `stopPropagation()` to end the climb.
- **Approval workflows** (expenses, leave requests, loan approvals) where authority is based on amount or role, and **customer-support escalation** tiers.

In the interview

> **30-second answer:** "Chain of Responsibility splits a series of checks into separate handler objects, each linked only to its successor. The request moves down the line until some handler completes it or refuses it. Because only the wiring knows the order, adding, removing or reordering a rule is cheap. Servlet filters and HTTP middleware are the everyday examples. The classic bug is a handler that forgets to pass on and swallows the request, so I give every path an explicit outcome and put a terminal handler at the end."

Follow-ups:

- *How is it different from Decorator?* Both link objects that share one interface. With a **Decorator**, all wrappers run and the call is always delivered to the inner object; the point is to *add* behaviour. With **CoR**, a handler may **end** the request, and handlers further down never see it.
- *How is it different from Observer?* Observer tells **every** subscriber, and no subscriber can block the rest. CoR ends at the **first** handler that accepts the job.
- *How is it different from a pipeline?* A pipeline pushes a value through **all** of its steps and produces a transformed result. A chain *routes* a request to whichever handler takes it. Rule of thumb: if no stage can stop the flow, you have a pipeline.
- *Combine it with Command?* Yes — the request moving through the handlers is frequently itself a Command object (e.g. a job checked by validation and authorisation handlers before a worker runs it).
- *What if no handler can handle the request?* Put an explicit terminal handler (e.g. `NoHandlerFound`) that returns an error or logs, so failures are never silent.
- *How do you test it?* Test each handler with a fake next handler that records whether it was called and with what; then one integration test (a test of several parts working together) for the wiring order.
- *Where should rate limiting go, before or after auth?* Explain the trade-off above and say you would document the reason next to the wiring.

Self-check

1. **Q:** What does each handler hold a reference to? **A:** Only the next handler (through the abstract `Handler` type).
2. **Q:** Name the most common CoR bug. **A:** A handler that neither handles nor passes on — the request silently disappears.
3. **Q:** Why do modern frameworks pass `next` as a parameter? **A:** Because a missing `next(...)` call is then easy to spot in the handler's own body, and because the handler can do work both before and after the rest of the chain runs.
4. **Q:** Is a logging chain where every handler always passes on a "pure" CoR? **A:** Not strictly; since no handler can stop the flow it behaves like a pipeline. It is a common variant.
5. **Q:** When should you *not* use CoR? **A:** When every stage always runs in a fixed order, when stages depend on each other's results, or when there is no terminal handler.

TL;DR

A request walks a line of handlers; each handles, rejects, or passes on. The chain is data, so it is easy to change — but **order matters**, **always have a terminal handler**, and **never swallow a request**.

10.2 Command

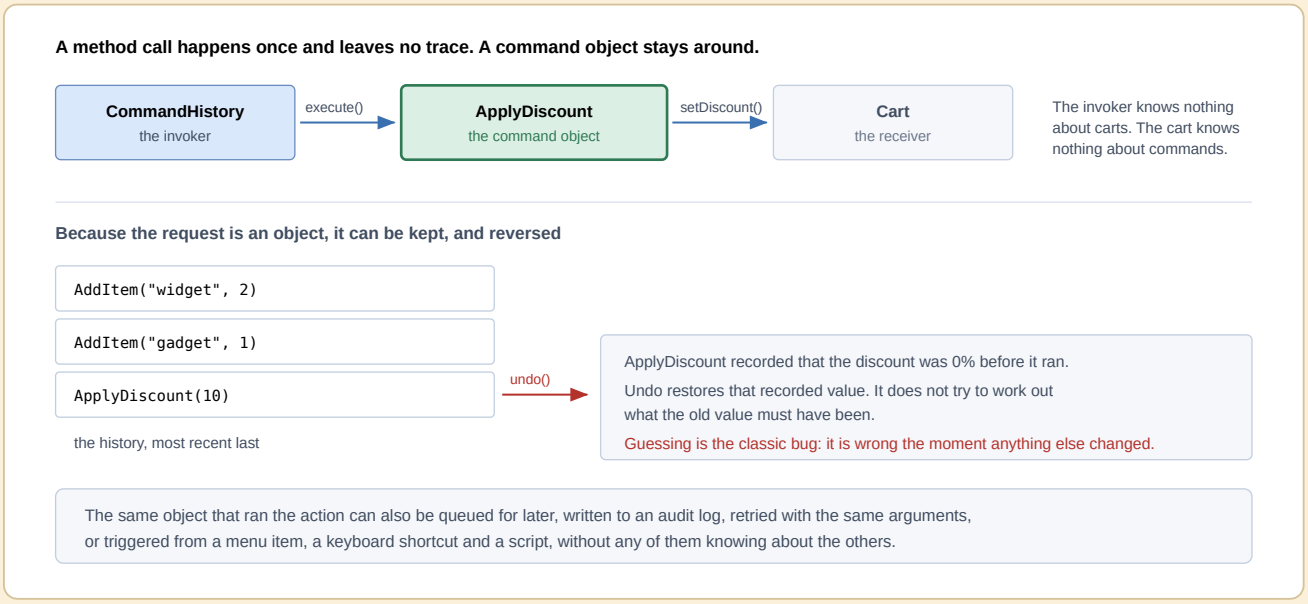


Figure — The invoker (CommandHistory) runs command objects against the receiver (Cart) and keeps them so they can be undone (course diagram). The figure assumes the discount was 0% before; our code deliberately starts at 5% to prove undo restores the recorded value.

What it is

Command packages a request ("add 2 widgets to the cart") as an **object** containing everything required to perform it later. Once the request is a value you can hold, you can **keep it in a list, put it on a queue (a first-in, first-out waiting line), write it to an audit log (a permanent record of who did what), run it again, hand it to a worker thread (a background thread — an independent path of execution — that takes jobs and runs them)** — and, when it also knows its own inverse, **reverse it**.

GoF intent, in plain words: *wrap a request in an object, so clients can be given different requests, requests can be queued or logged, and operations can be undone.*

Everyday analogy: a restaurant order slip. The waiter (invoker) does not cook; they write the order on a slip (command) and put it on the queue. The cook (receiver) acts on it later. The slip can be queued, prioritised, or cancelled.

The problem it solves

A plain method call such as `cart.setDiscount(10)` runs immediately and is forgotten the moment it returns. That is not enough once you need any of these:

- **Undo** — "reverse my last two changes".
- **A queue** — run the work later, on a worker thread, not in the request thread (the thread serving the user's request).
- **A log of intent** — an auditor asks which operations were *requested*, not just what state the data ended up in.
- **Retry** — run a failed action again with the same arguments.
- **One action, many triggers** — a menu item, a hot-key (keyboard shortcut) and an automation script should all fire the same "apply discount".

Command gives the request a body that outlives the call.

How it works

Interviewers usually want **three roles** named — **invoker**, **command**, **receiver** — plus the **client** that creates commands. GoF lists five participants in total: **Command** (the interface), **ConcreteCommand** (binds a receiver to an action), **Client**, **Invoker** and **Receiver**:

Role	Job	In our code
Command	Interface with `execute()` (and optionally `undo()`).	`Command`, `UndoableCommand`
Concrete command	Stores its receiver and arguments; can perform (and reverse) exactly one operation.	`AddItemCommand`, `ApplyDiscountCommand`, `MacroCommand`
Receiver	The object whose state is really modified. Has no idea commands exist.	`Cart`
Invoker	Triggers commands and keeps a record of them. Has no idea carts exist.	`CommandHistory`, `ExecutorService`
Client	Builds a concrete command and gives it its receiver.	`Main`

Steps:

1. The client creates a concrete command, passing the **receiver** and the **arguments**.
2. It hands the command to the **invoker**.
3. The invoker calls `execute()` and, if the command is undoable, pushes it on an **undo stack** (a stack is a last-in, first-out pile: you add and remove only at the top).
4. **Undo** = pop the last command and call `undo()`; push it on a **redo stack**.
5. **Redo** = pop from redo, `execute()` again, push back on undo. **Running a fresh command empties the redo stack**: the branch of history you might have re-applied has been replaced.

Structure (UML)

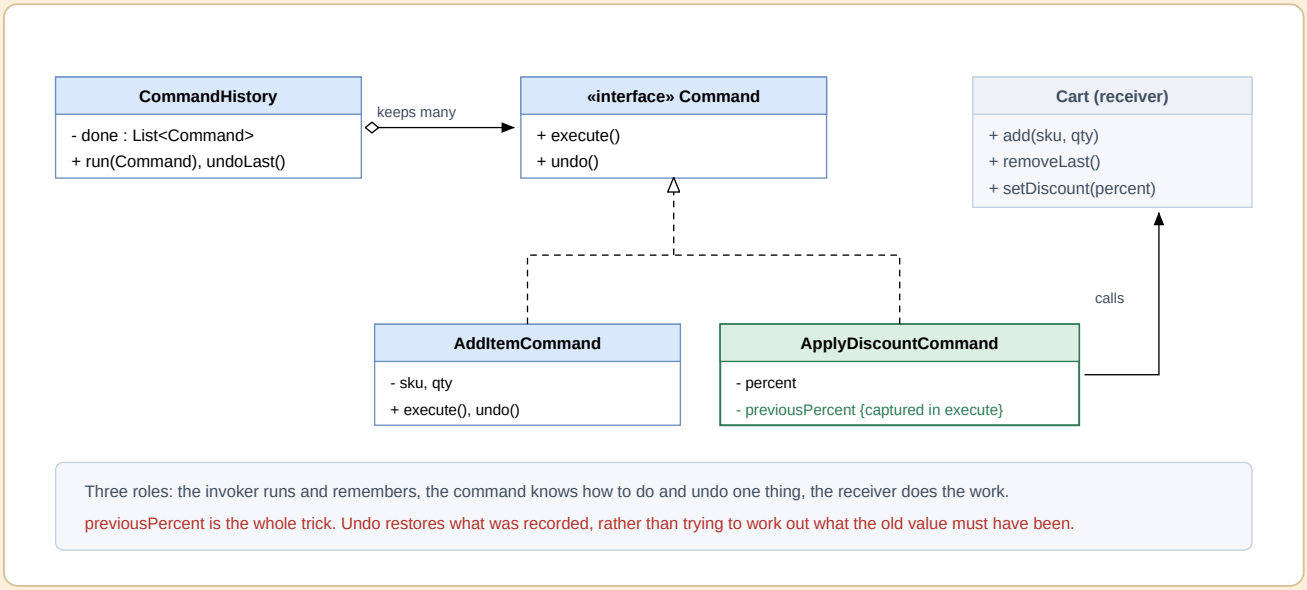
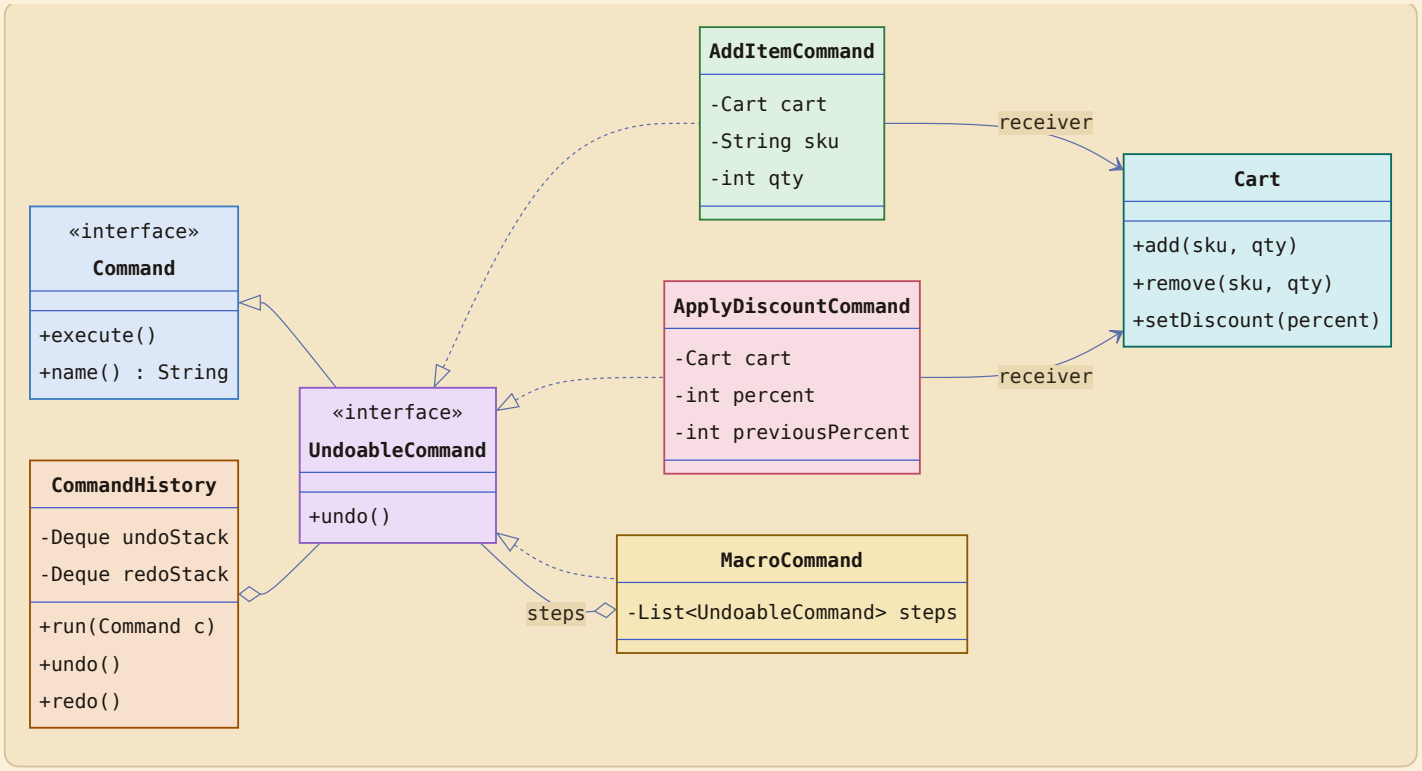
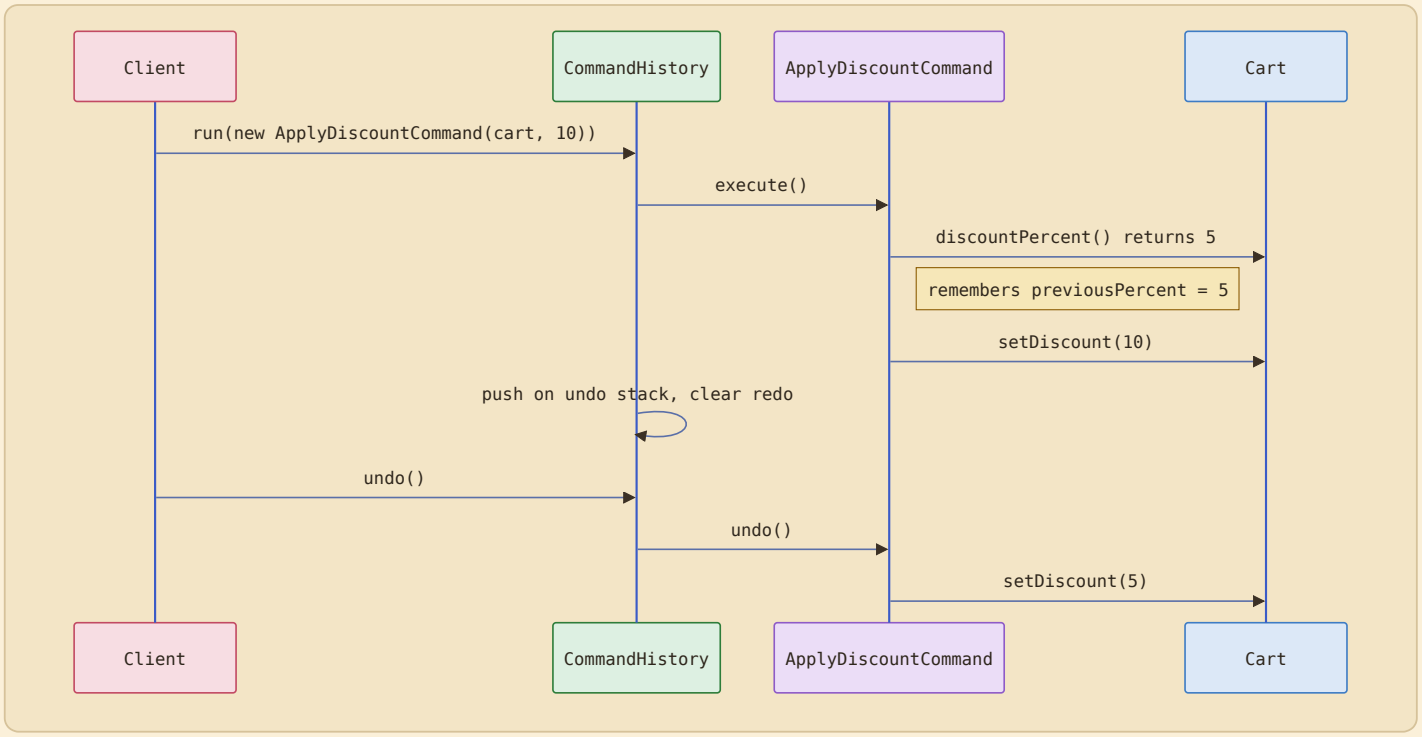


Figure — Class diagram: Command interface with execute/undo, two concrete commands, Cart as receiver, CommandHistory as invoker (course diagram; its `undoLast`/`removeLast` are named `undo`/`remove` in our code)

In the diagram above, each command keeps its own reference to the `Cart`. It has to: when it finally runs, the code that created it may have finished long ago, so the target object must travel inside the command along with the arguments. Also note `ApplyDiscountCommand.previousPercent` — the single field that makes its undo correct. And note that `CommandHistory` never mentions `Cart`; hand it any other commands and it still works, which is why one history can serve a whole application.

Our Java design:





Java example

Key design points in `lld.ch10.command`:

- `Command` has **one abstract method**, so any lambda (a short unnamed function) or `Runnable`-like block (`Runnable` is Java's built-in "code to run" interface) can be a command. Only commands that can *honestly* be reversed implement `UndoableCommand`. `CommandHistory` keeps irreversible commands out of the undo stack.
- `ApplyDiscountCommand` **captures the old value in** `execute()` and restores exactly that in `undo()`. The cart deliberately starts at 5%, to show that "undo = set to 0" would be wrong.
- `MacroCommand` is a command made of commands (a **Composite** of commands): it executes steps in order and undoes them in **reverse** order.
- Section 4 of `Main` shows the JDK's own version: `Runnable` is the command interface and an `ExecutorService` (Java's thread-pool service for running tasks) is the invoker that runs commands on a worker thread.

```java
// File: code/lld/ch10/command/Command.java
package lld.ch10.command;

/** A request turned into an object. One abstract method, so a lambda is a Command. */
@FunctionalInterface
public interface Command {
    void execute();

    default String name() { return getClass().getSimpleName(); }
}
```

```java
// File: code/lld/ch10/command/UndoableCommand.java
package lld.ch10.command;

/** Only commands that can honestly be reversed implement this. */
public interface UndoableCommand extends Command {
    void undo();
}
```

```java
// File: code/lld/ch10/command/Cart.java
package lld.ch10.command;

import java.util.ArrayList;
import java.util.List;

/** The RECEIVER: knows how to change itself and nothing about commands. */
public final class Cart {
    public record Line(String sku, int qty) {}

    private final List<Line> lines = new ArrayList<>();
    private int discountPercent = 0;

    public void add(String sku, int qty) { lines.add(new Line(sku, qty)); }

    /** Removes the most recent line matching sku and qty. */
    public void remove(String sku, int qty) {
        for (int i = lines.size() - 1; i >= 0; i--) {
            if (lines.get(i).equals(new Line(sku, qty))) {
                lines.remove(i);
                return;
            }
        }
    }

    public int discountPercent() { return discountPercent; }
    public void setDiscount(int percent) { this.discountPercent = percent; }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder("Cart[");
        for (int i = 0; i < lines.size(); i++) {
            if (i > 0) sb.append(", ");
            sb.append(lines.get(i).sku()).append(" x").append(lines.get(i).qty());
        }
        return sb.append("] discount=").append(discountPercent).append('%').toString();
    }
}
```

```java
// File: code/lld/ch10/command/AddItemCommand.java
package lld.ch10.command;

/** Adds a line; undo removes exactly that line. */
final class AddItemCommand implements UndoableCommand {
    private final Cart cart;      // the command carries its receiver...
    private final String sku;     // ...and all the arguments it needs to run later
    private final int qty;

    AddItemCommand(Cart cart, String sku, int qty) {
        this.cart = cart;
        this.sku = sku;
        this.qty = qty;
    }

    @Override public void execute() { cart.add(sku, qty); }
    @Override public void undo()    { cart.remove(sku, qty); }
    @Override public String name()  { return "AddItem(" + sku + ", " + qty + ")"; }
}
```

```java
// File: code/lld/ch10/command/ApplyDiscountCommand.java
package lld.ch10.command;

/** Sets a discount; captures the previous value during execute so undo RESTORES, never guesses. */
final class ApplyDiscountCommand implements UndoableCommand {
    private final Cart cart;
    private final int percent;
    private int previousPercent;          // filled in by execute()

    ApplyDiscountCommand(Cart cart, int percent) {
        this.cart = cart;
        this.percent = percent;
    }

    @Override
    public void execute() {
        previousPercent = cart.discountPercent();   // capture BEFORE changing
        cart.setDiscount(percent);
    }

    @Override public void undo()   { cart.setDiscount(previousPercent); }
    @Override public String name() { return "ApplyDiscount(" + percent + "%)"; }
}
```

```java
// File: code/lld/ch10/command/MacroCommand.java
package lld.ch10.command;

import java.util.List;

/** A command made of commands. Runs in order; undoes in REVERSE order. */
final class MacroCommand implements UndoableCommand {
    private final String label;
    private final List<UndoableCommand> steps;

    MacroCommand(String label, List<UndoableCommand> steps) {
        this.label = label;
        this.steps = List.copyOf(steps);
    }

    @Override public void execute() { steps.forEach(Command::execute); }

    @Override
    public void undo() {
        for (int i = steps.size() - 1; i >= 0; i--) steps.get(i).undo();
    }

    @Override public String name() { return "Macro:" + label; }
}
```

```java
// File: code/lld/ch10/command/CommandHistory.java
package lld.ch10.command;

import java.util.ArrayDeque;
import java.util.Deque;

/** The INVOKER: runs commands and remembers them. Knows nothing about carts. */
public final class CommandHistory {
    private final Deque<UndoableCommand> undoStack = new ArrayDeque<>();
    private final Deque<UndoableCommand> redoStack = new ArrayDeque<>();

    public void run(Command command) {
        command.execute();
        if (command instanceof UndoableCommand u) {
            undoStack.push(u);
            redoStack.clear();     // a new action destroys the "future" you could have redone
            System.out.println("  ran  " + command.name());
        } else {
            System.out.println("  ran  " + command.name() + " (irreversible: kept out of history)");
        }
    }

    public void undo() {
        if (undoStack.isEmpty()) { System.out.println("  nothing to undo"); return; }
        UndoableCommand c = undoStack.pop();
        c.undo();
        redoStack.push(c);
        System.out.println("  undo " + c.name());
    }

    public void redo() {
        if (redoStack.isEmpty()) { System.out.println("  nothing to redo"); return; }
        UndoableCommand c = redoStack.pop();
        c.execute();
        undoStack.push(c);
        System.out.println("  redo " + c.name());
    }
}
```

```java
// File: code/lld/ch10/command/Main.java
package lld.ch10.command;

import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Cart cart = new Cart();
        CommandHistory history = new CommandHistory();

        System.out.println("=== 1. Undo / redo ===");
        cart.setDiscount(5);   // the cart does NOT start at 0%: a guessing undo would be wrong
        history.run(new AddItemCommand(cart, "widget", 2));
        history.run(new AddItemCommand(cart, "gadget", 1));
        history.run(new ApplyDiscountCommand(cart, 10));
        System.out.println("  " + cart);
        history.undo();
        history.undo();
        System.out.println("  " + cart + "   <- discount restored to the recorded 5%, not 0%");
        history.redo();
        System.out.println("  " + cart);
        history.run(new AddItemCommand(cart, "cable", 3));   // clears the redo stack
        history.redo();

        System.out.println("\n=== 2. Macro command ===");
        UndoableCommand starterKit = new MacroCommand("starter-kit", List.of(
            new AddItemCommand(cart, "keyboard", 1),
            new AddItemCommand(cart, "mouse", 1),
            new ApplyDiscountCommand(cart, 15)));
        history.run(starterKit);
        System.out.println("  " + cart);
        history.undo();                                      // one undo reverses all three steps
        System.out.println("  " + cart);

        System.out.println("\n=== 3. Irreversible command (no undo offered) ===");
        Command sendReceipt = new Command() {
            @Override public void execute() { System.out.println("  [email] receipt sent for " + cart); }
            @Override public String name()  { return "SendReceipt"; }
        };
        history.run(sendReceipt);

        System.out.println("\n=== 4. Commands on a queue: Runnable + ExecutorService (the invoker) ===");
        List<Runnable> jobs = List.of(
            () -> System.out.println("  [worker] resize image 1"),
            () -> System.out.println("  [worker] resize image 2"),
            () -> System.out.println("  [worker] send newsletter"));
        System.out.println("  queued " + jobs.size() + " jobs");
        ExecutorService worker = Executors.newSingleThreadExecutor();   // one thread => FIFO order
        jobs.forEach(worker::submit);
        worker.shutdown();
        worker.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("  all jobs done");
    }
}
```

Output:

```
=== 1. Undo / redo ===
  ran  AddItem(widget, 2)
  ran  AddItem(gadget, 1)
  ran  ApplyDiscount(10%)
  Cart[widget x2, gadget x1] discount=10%
  undo ApplyDiscount(10%)
  undo AddItem(gadget, 1)
  Cart[widget x2] discount=5%   <- discount restored to the recorded 5%, not 0%
  redo AddItem(gadget, 1)
  Cart[widget x2, gadget x1] discount=5%
  ran  AddItem(cable, 3)
  nothing to redo

=== 2. Macro command ===
  ran  Macro:starter-kit
  Cart[widget x2, gadget x1, cable x3, keyboard x1, mouse x1] discount=15%
  undo Macro:starter-kit
  Cart[widget x2, gadget x1, cable x3] discount=5%

=== 3. Irreversible command (no undo offered) ===
  [email] receipt sent for Cart[widget x2, gadget x1, cable x3] discount=5%
  ran  SendReceipt (irreversible: kept out of history)

=== 4. Commands on a queue: Runnable + ExecutorService (the invoker) ===
  queued 3 jobs
  [worker] resize image 1
  [worker] resize image 2
  [worker] send newsletter
  all jobs done
```

Undo must restore, not recompute

It is tempting to write a discount undo as "reset to 0" or "calculate what the old value probably was". Both are **guesses**. If the cart began at 5%, or some other command touched the discount meanwhile, the user ends up with a value that never existed. The rule: `execute` **must record whatever** `undo` **will need** — normally just the fields it is about to overwrite, not a copy of the whole object.

Undo history is strictly last-in, first-out. Commands are reversed newest first: the sequence `AddItem, AddItem, ApplyDiscount` is undone as `ApplyDiscount`, then the second `AddItem`, then the first. Reversing them in any other order can produce states that never existed, hence a stack, never a list you can index into at random.

When undo is a lie. Some effects escape your system: an email that was sent, a card that was charged, a call made to a partner's API (the interface a service exposes for other programs to call). An `undo()` on those promises something impossible, which is worse than having no undo at all. Two honest options: keep them out of the history (our `SendReceipt`), or use **compensation** — a *new* action that produces a similar end state, like a **refund** for a charge. A refund is not an undo: both events stay in the record.

Trade-offs

Pros	Cons
Since the request lives on after the call, undo/redo, queues, audit logs, retries and scheduling all become possible.	Every action needs its own class, so the number of classes tracks the number of actions.
The invoker has no knowledge of what it runs, so a single history works for all commands.	Each command has to record enough to run later and to reverse itself; subtle mistakes are easy.
One action object can back a menu entry, a hot-key, a script and an API endpoint.	Extra hop when reading code: to learn what a button does you first find the command it builds.
Commands can be grouped into macro commands.	A single faulty `undo` corrupts the whole history.

When not to use it

- **The call is simple and immediate.** A `SaveCommand` class that nobody ever undoes, queues or logs is extra code with no benefit.
- **Undo is not really possible.** Use compensation and call it that.

- **You only need deferral.** A lambda (`Runnable`) already is a one-method command. Promote it to a class once you need `undo`, a readable name in logs, or serialization (turning it into bytes or text so it can be stored or sent).
- **Commands must outlive the process.** They then have to be serialized, which drags in format versioning; a durable queue (one that survives restarts — see System Design Fundamentals) or an event log (Kafka — a durable, distributed message log — or a jobs table) handles that better than hand-rolled command classes.

Common mistakes

1. **Recomputing in `undo`** instead of restoring captured state.
2. **Forgetting to clear the redo stack** when a new command runs.
3. **Putting business logic in the invoker** (`if (cmd instanceof AddItem) ...` — `instanceof` checks an object's concrete type). The invoker should only run and remember.
4. **Commands that capture a mutable object by reference** (mutable = changeable after creation) and then read it later, when it has changed. Capture values.
5. **Unbounded history** — cap the depth, as editors do.

Real-world examples

- **Undo in editors**: word processors, IDEs (integrated development environments — code editors such as IntelliJ), Photoshop. The JDK has `javax.swing.undo.UndoManager` with `UndoableEdit` objects that have `undo()`/`redo()`.
- **`Runnable` / `Callable`** are the JDK's command interfaces; `ExecutorService`, `ScheduledExecutorService` and `Thread` are invokers.
- **GUI actions** (GUI = graphical user interface): Swing's (Java's desktop UI toolkit) `javax.swing.Action` lets one action back a menu item, a toolbar button and a shortcut; WPF's `ICommand` in .NET plays the same role.
- **Event sourcing** (Chapter 11; storing every change as an event and rebuilding state by replaying them): recording the *requested changes* instead of only the resulting state is the command idea applied to a whole system.
- **Job queues**: a Sidekiq/Celery (background-job libraries for Ruby and Python) job, a Spring `@Async` task, a message on a Kafka/RabbitMQ (message brokers) queue that asks a worker to "send invoice #42" — a serialized command.
- **Database migrations** (versioned scripts that change a database's structure) with `up`/`down` (e.g. Liquibase changesets with rollback) mirror `execute`/`undo`.
- **Redux** (a JavaScript state-management library) actions are command objects: the store dispatches them (invoker role) and reducers apply them to state (receiver role); Redux DevTools' time travel replays them.
- **In other languages**: in Python, a callable object (one defining `__call__`) with an extra `undo` method works well, and `functools.partial` can bind the arguments; in C#, UI frameworks ship an `ICommand` interface, and a `record` gives you an immutable (unchangeable after creation) command in a single line.
- **Transactional outbox / task tables** (the job is written to a database table in the same transaction — the same all-or-nothing database update — as the data change, so it cannot be lost) in food-delivery or e-commerce back ends store "commands" (send SMS, notify restaurant) to be run reliably by workers.

In the interview

> **30-second answer:** "Command wraps a request, together with its receiver and arguments, in an object — so the request can be kept, queued, logged, retried or reversed. Three roles matter: the invoker triggers and records commands without knowing what they touch, each command can perform and reverse a single operation, and the receiver holds the state that actually changes. For undo, `execute` must capture the old value and `undo` restores it; history is a stack, and a new command clears redo. Irreversible actions like charging a card get compensation, not undo. In Java, `Runnable` plus an `ExecutorService` is the everyday form."

Follow-ups:

- *Command vs Strategy?* Each puts behaviour inside an object. A **Strategy** answers *how* a job is done and is picked and replaced by the caller. A **Command** captures *which* job to do, arguments included, and is held on to so it can run later or be reversed.
- *Command vs a plain lambda?* A one-method closure (a function that remembers variables from where it was created) already is a command. Write a class only when you also need `undo`, a readable log name, or to serialize it.

- *Command vs Memento for undo?* Command stores the **action** plus a small piece of state; undo runs the inverse. Memento stores the **whole state**; undo puts a snapshot back. Command is cheaper per step; Memento is always correct. Real systems combine them (snapshot occasionally, replay commands in between).
- *How do you implement redo?* A second stack; undo moves a command to it; redo moves it back after re-executing; a new command clears it.
- *How do you make a command queue thread-safe (correct when many threads use it at once)?* Use a `BlockingQueue` (a queue whose `take()` waits until an item arrives) or an `ExecutorService`; make commands immutable apart from captured undo state.
- *How would you persist commands?* Serialize them (JSON — a common text format for structured data — with a type field and a version), store in a durable queue or table, make them **idempotent** (running one twice has the same effect as running it once — see System Design Fundamentals) so a retry is safe.

Self-check

1. **Q:** What must a command carry? **A:** Its receiver and all arguments needed to run later (and any state needed for undo).
2. **Q:** Why does `ApplyDiscountCommand` store `previousPercent`? **A:** So undo restores the real previous value instead of guessing.
3. **Q:** What happens to the redo stack when a new command runs? **A:** It is cleared.
4. **Q:** How do you "undo" a card charge? **A:** You don't; you issue a compensating action (a refund) and keep both in the record.
5. **Q:** Which JDK interface is a command? **A:** `Runnable` (and `Callable`).

TL;DR

Make the request an object: **invoker** runs it, **command** knows do/undo, **receiver** changes. Undo **restores captured state**, history is a **stack**, and irreversible actions get **compensation**. A lambda is a command when you only need deferral.

10.3 Iterator

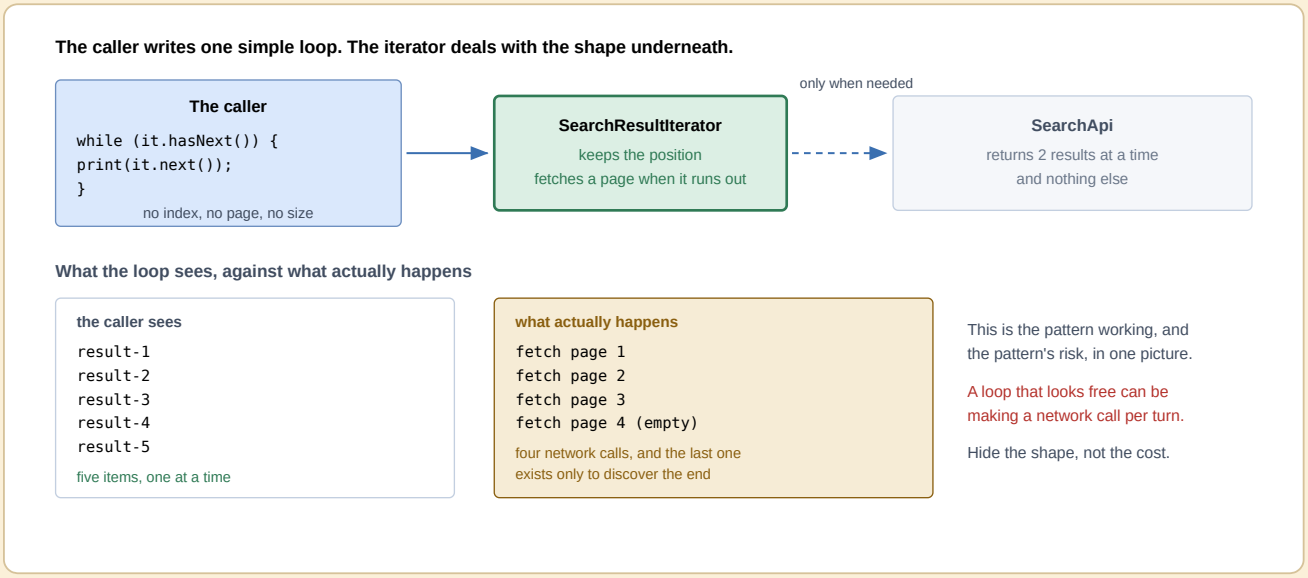


Figure — The caller writes one simple loop; the iterator fetches pages from the API only when needed (course diagram)

What it is

Iterator lets you visit the elements of a collection **one at a time, in order, without knowing how the collection stores them** — array, linked list (elements chained together by references), tree, or a remote API that returns two results per page.

GoF intent, in plain words: *give sequential access to the elements of an aggregate (GoF's word for a collection) without exposing its internal representation.* GoF participants: **Iterator** (interface for accessing and

traversing), **ConcreteIterator** (implements it and tracks the current position), **Aggregate** (interface for creating an iterator) and **ConcreteAggregate** (returns the right ConcreteIterator).

Everyday analogy: a TV remote's "next channel" button. You do not need to know how the channels are stored in the TV; you just press *next* until there are no more.

The problem it solves

- Code that loops with `get(i)` quietly assumes the data sits in an array. Move the data into a linked list, a tree or a paginated (returns results one page at a time) remote service, and each of those loops must be rewritten.
- Many sources **cannot be indexed at all**: a tree has several valid visiting orders (depth-first — follow one branch all the way down before the next; breadth-first — level by level); a graph (nodes joined by edges, possibly with loops) must remember which nodes were already seen; a search API hands out pages. When each caller writes its own traversal, each caller also re-implements paging — and some implement it wrongly.

Iterator moves the walking logic **into a separate object** that answers two questions: *are any elements left?* (`hasNext`) and *hand me the following element* (`next`).

Key detail: the current position is stored inside the iterator object, never in the collection. So two traversals of one collection can be in progress together without disturbing each other, and every call to `iterator()` must build a **new** iterator instead of rewinding a shared cursor (a marker of the current position).

How it works

1. The **aggregate** (the collection) has a method `iterator()` that creates a new iterator. In Java it implements `Iterable<T>`.
2. The **iterator** implements `Iterator<T>`: `hasNext()` and `next()` (and optionally `remove()`).
3. The iterator keeps its own **cursor** (index, stack of tree nodes, current page...).
4. The client loops: `while (it.hasNext()) use(it.next());` — or just `for (T x : collection)`, which the Java compiler turns into exactly that.

Structure (UML)

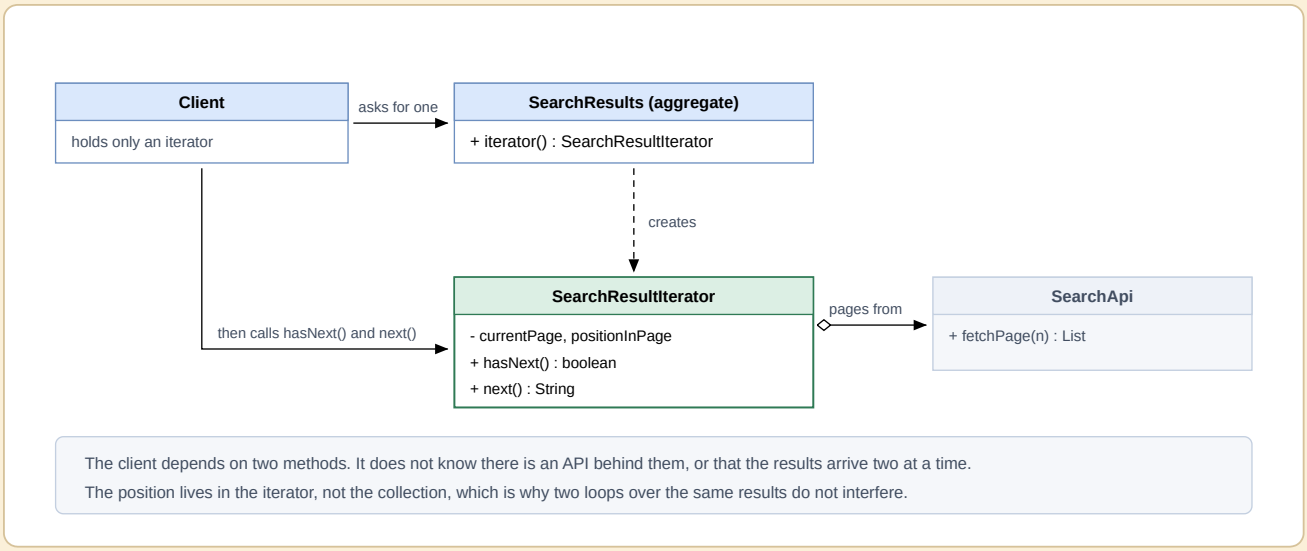
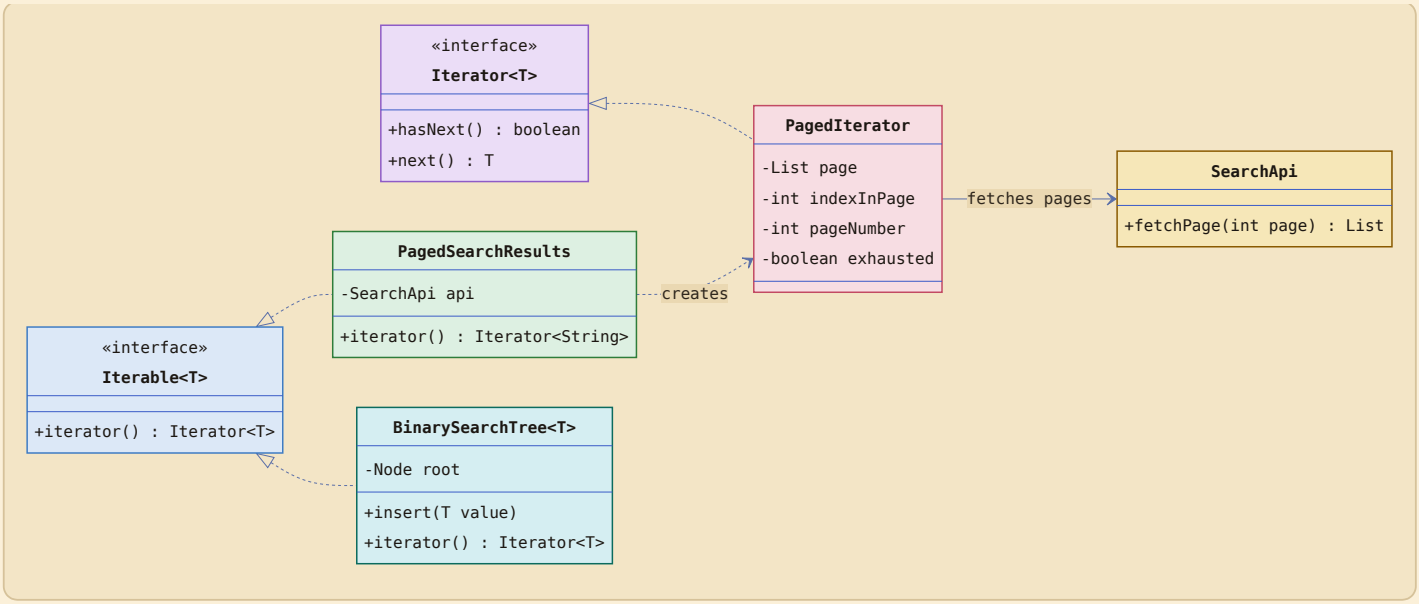


Figure — Class diagram: SearchResults (aggregate) hands out a SearchResultIterator, which calls SearchApi page by page; the client only calls hasNext/next (course diagram)

In the diagram above, `hasNext` does real work: once the buffered page runs out, it asks the API for another page. A *lazy* iterator (one that produces elements only on demand) commonly behaves like this — so never assume `hasNext` is cheap or has no side effects (effects beyond returning a value, such as a network call).

Our Java design:



Java example

`lld.ch10.iterator` shows:

1. **A paged remote source behind a plain for-each.** `PagedSearchResults implements Iterable<String>`, so `for (String r : results)` just works. The inner `PagedIterator` fetches a page only when the current one is used up, and follows the `Iterator` contract (the documented rules every implementation must obey) (`next()` throws `NoSuchElementException` when empty).
2. **A tree iterator.** `BinarySearchTree` hides an in-order walk (left subtree, then the node, then the right subtree — which yields sorted order) behind `hasNext/next` using an explicit stack; two iterators over the same tree run independently.
3. **External vs internal iteration.** *External*: the **client** drives (`hasNext/next`) and can stop early with `break`. *Internal*: the **collection** drives and calls your code for each element (`forEach(lambda)`, streams — Java's pipeline API for processing collections).
4. **Fail-fast vs fail-safe.** Structurally changing an `ArrayList` (adding or removing) while a for-each loop is walking it *usually* makes the next `next()` throw `ConcurrentModificationException`. It is only **best-effort**: the demo also removes the second-to-last element, after which `hasNext()` simply returns `false` — no exception, and the last element is silently never visited. The safe way is `Iterator.remove()` (or `removeIf`). `CopyOnWriteArrayList` iterates over a **snapshot**, so it never throws — but the loop does not see the new elements. ("Fail-safe" is an informal name; the Javadoc (Java's official API documentation) talks about *snapshot* and *weakly consistent* iterators.)
5. `Spliterator` (Java 8): an iterator that can **split** itself (`trySplit`) so parallel streams can process parts on different threads. It reports **characteristics** such as `SIZED` and `ORDERED`.

```java
// File: code/lld/ch10/iterator/SearchApi.java
package lld.ch10.iterator;

import java.util.List;

/** A remote source that returns two results per page and knows nothing about iteration. */
final class SearchApi {
    private static final List<String> ALL = List.of("result-1", "result-2", "result-3", "result-4", "result-5");
    private static final int PAGE_SIZE = 2;
    private int calls = 0;

    List<String> fetchPage(int page) {          // pages are 1-based
        calls++;
        System.out.println("    [api] fetching page " + page);
        int from = (page - 1) * PAGE_SIZE;
        if (from >= ALL.size()) return List.of();
        return ALL.subList(from, Math.min(from + PAGE_SIZE, ALL.size()));
    }

    int calls() { return calls; }
}
```

```java
// File: code/lld/ch10/iterator/PagedSearchResults.java
package lld.ch10.iterator;

import java.util.Iterator;
import java.util.List;
import java.util.NoSuchElementException;

/**
 * The AGGREGATE. Implements Iterable, so the enhanced for-loop works on it.
 * Its only job is to hand out a FRESH iterator each time.
 */
final class PagedSearchResults implements Iterable<String> {
    private final SearchApi api;

    PagedSearchResults(SearchApi api) { this.api = api; }

    @Override
    public Iterator<String> iterator() { return new PagedIterator(); }

    /** The ITERATOR. The position lives here, not in the collection. */
    private final class PagedIterator implements Iterator<String> {
        private List<String> page = List.of();
        private int indexInPage = 0;
        private int pageNumber = 0;
        private boolean exhausted = false;

        @Override
        public boolean hasNext() {                // NOT a passive question: may hit the network
            if (indexInPage < page.size()) return true;
            if (exhausted) return false;
            page = api.fetchPage(++pageNumber);
            indexInPage = 0;
            if (page.isEmpty()) {
                exhausted = true;
                return false;
            }
            return true;
        }

        @Override
        public String next() {
            if (!hasNext()) throw new NoSuchElementException();   // the Iterator contract
            return page.get(indexInPage++);
        }
    }
}
```

```java
// File: code/lld/ch10/iterator/BinarySearchTree.java
package lld.ch10.iterator;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Iterator;
import java.util.NoSuchElementException;

/** A tree has no index. The iterator hides an in-order walk behind hasNext/next. */
final class BinarySearchTree<T extends Comparable<T>> implements Iterable<T> {
    private static final class Node<T> {
        final T value;
        Node<T> left, right;
        Node(T value) { this.value = value; }
    }

    private Node<T> root;

    void insert(T value) {
        Node<T> fresh = new Node<>(value);
        if (root == null) { root = fresh; return; }
        Node<T> cur = root;
        while (true) {
            if (value.compareTo(cur.value) < 0) {
                if (cur.left == null) { cur.left = fresh; return; }
                cur = cur.left;
            } else {
                if (cur.right == null) { cur.right = fresh; return; }
                cur = cur.right;
            }
        }
    }

    @Override
    public Iterator<T> iterator() {
        return new Iterator<>() {
            // The stack holds the path still to visit: this IS the iterator's position.
            private final Deque<Node<T>> stack = new ArrayDeque<>();
            { pushLeft(root); }

            private void pushLeft(Node<T> n) {
                for (; n != null; n = n.left) stack.push(n);
            }

            @Override public boolean hasNext() { return !stack.isEmpty(); }

            @Override
            public T next() {
                if (stack.isEmpty()) throw new NoSuchElementException();
                Node<T> n = stack.pop();
                pushLeft(n.right);
                return n.value;
            }
        };
    }
}
```

```java
// File: code/lld/ch10/iterator/Main.java
package lld.ch10.iterator;

import java.util.ArrayList;
import java.util.ConcurrentModificationException;
import java.util.Iterator;
import java.util.List;
import java.util.Spliterator;
import java.util.concurrent.CopyOnWriteArrayList;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Paged remote results behind a plain for-each ===");
        SearchApi api = new SearchApi();
        for (String r : new PagedSearchResults(api)) {
            System.out.println("  loop sees " + r);
        }
        System.out.println("  5 items, " + api.calls() + " network calls (the last one only discovers the end)");

        System.out.println("\n=== 2. Two independent iterators over one tree ===");
        BinarySearchTree<Integer> tree = new BinarySearchTree<>();
        for (int v : new int[] {50, 30, 70, 20, 40, 60, 80}) tree.insert(v);
        Iterator<Integer> a = tree.iterator();
        Iterator<Integer> b = tree.iterator();
        System.out.println("  a: " + a.next() + ", " + a.next() + ", " + a.next());
        System.out.println("  b: " + b.next() + "   <- b starts from the beginning: position lives in the
iterator");
        StringBuilder all = new StringBuilder();
        for (int v : tree) all.append(v).append(' ');
        System.out.println("  in-order: " + all.toString().trim());

        System.out.println("\n=== 3. External vs internal iteration ===");
        List<String> names = List.of("ann", "bob", "cy");
        Iterator<String> it = names.iterator();               // external: the CLIENT drives
        while (it.hasNext()) {
            String n = it.next();
            if (n.equals("bob")) { System.out.println("  external: found bob, stopping early"); break; }
        }
        names.forEach(n -> System.out.println("  internal: " + n)); // internal: the COLLECTION drives

        System.out.println("\n=== 4. Fail-fast vs fail-safe ===");
        List<String> tags = new ArrayList<>(List.of("java", "tmp", "lld", "tmp"));
        try {
            for (String t : tags) {
                if (t.equals("tmp")) tags.remove(t);          // structural change behind the iterator's back
            }
        } catch (ConcurrentModificationException e) {
            System.out.println("  fail-fast: ConcurrentModificationException");
        }
        List<String> quiet = new ArrayList<>(List.of("java", "tmp", "lld"));
        List<String> visited = new ArrayList<>();
        for (String t : quiet) {
            visited.add(t);
            if (t.equals("tmp")) quiet.remove(t);             // removes the SECOND-TO-LAST element
        }
        System.out.println("  best-effort only: no exception, visited " + visited + ", 'lld' silently skipped");
        tags = new ArrayList<>(List.of("java", "tmp", "lld", "tmp"));
        for (Iterator<String> i = tags.iterator(); i.hasNext(); ) {
            if (i.next().equals("tmp")) i.remove();           // the safe way: remove through the iterator
        }
        System.out.println("  after Iterator.remove: " + tags);
        List<String> cow = new CopyOnWriteArrayList<>(List.of("x", "y"));
        for (String s : cow) cow.add(s + "!");                 // iterates a snapshot: no exception
        System.out.println("  fail-safe CopyOnWriteArrayList: " + cow);

        System.out.println("\n=== 5. Spliterator: an iterator that can split for parallel work ===");
        List<Integer> nums = List.of(1, 2, 3, 4, 5, 6, 7, 8);
        Spliterator<Integer> right = nums.spliterator();
```

```
        Spliterator<Integer> left = right.trySplit();           // hands off (roughly) the first half
        System.out.println("  SIZED=" + right.hasCharacteristics(Spliterator.SIZED)
                + " ORDERED=" + right.hasCharacteristics(Spliterator.ORDERED));
        StringBuilder l = new StringBuilder(), r = new StringBuilder();
        left.forEachRemaining(x -> l.append(x).append(' '));
        right.forEachRemaining(x -> r.append(x).append(' '));
        System.out.println("  left half:  " + l.toString().trim());
        System.out.println("  right half: " + r.toString().trim());
        System.out.println("  parallel stream sum = " + nums.parallelStream().mapToInt(Integer::intValue).sum());
    }
}
```

Output:

```
=== 1. Paged remote results behind a plain for-each ===
    [api] fetching page 1
  loop sees result-1
  loop sees result-2
    [api] fetching page 2
  loop sees result-3
  loop sees result-4
    [api] fetching page 3
  loop sees result-5
    [api] fetching page 4
  5 items, 4 network calls (the last one only discovers the end)

=== 2. Two independent iterators over one tree ===
  a: 20, 30, 40
  b: 20   <- b starts from the beginning: position lives in the iterator
  in-order: 20 30 40 50 60 70 80

=== 3. External vs internal iteration ===
  external: found bob, stopping early
  internal: ann
  internal: bob
  internal: cy

=== 4. Fail-fast vs fail-safe ===
  fail-fast: ConcurrentModificationException
  best-effort only: no exception, visited [java, tmp], 'lld' silently skipped
  after Iterator.remove: [java, lld]
  fail-safe CopyOnWriteArrayList: [x, y, x!, y!]

=== 5. Spliterator: an iterator that can split for parallel work ===
  SIZED=true ORDERED=true
  left half:  1 2 3 4
  right half: 5 6 7 8
  parallel stream sum = 36
```

Hide the shape, not the cost

Look at section 1 of the output: five results, **four** network calls. The last call comes back empty; its only purpose is to find out that the data has ended, because this API reports neither a total nor a "next page" token. That is genuine behaviour, not a bug in the demo. The takeaway: **a tidy loop conceals how much each step costs.** The **N+1 query problem** in ORMs (object-relational mappers: libraries that turn database rows into objects) has the same shape (one query for the list, then one more query *per element* as you iterate). Ways to keep the cost visible:

- **Name the type honestly** (`PagedResults`, `LazySearchResults`, not `List`).
- **Offer a batch API** for callers who want everything, with bigger pages.
- **Return the total or a next-page cursor** when the source knows it, so the extra call is not needed.
- **Log or count the fetches**, so a growing request count gets noticed.

Java's iterator family (quick reference)

Type	What it adds
`Iterable<T>`	`iterator()`; lets a type be used in the enhanced for-loop. Also `forEach` (internal iteration) and `spliterator()`.
`Iterator<T>`	`hasNext()`, `next()`, optional `remove()`, `forEachRemaining()`.
`ListIterator<T>`	Bidirectional: `hasPrevious()`, `previous()`, indexes, `set()`, `add()`.
`Spliterator<T>`	`tryAdvance`, `forEachRemaining`, `trySplit` for parallelism, `characteristics()` like `ORDERED`, `SIZED`, `DISTINCT`, `IMMUTABLE`.
`Enumeration<E>`	The old (Java 1.0) version: `hasMoreElements()`, `nextElement()`. Prefer `Iterator`.

Fail-fast, weakly consistent, snapshot. Iterators of `ArrayList`, `HashMap` and friends are **fail-fast**: they detect a structural change made behind their back and throw `ConcurrentModificationException`. The Javadoc says this is done on a **best-effort** basis — never write code that relies on the exception (section 4 of the demo shows a removal that goes unnoticed). Despite its name, the exception is most often caused by **one thread** changing a collection inside its own loop, not by concurrency (several threads running at the same time). `ConcurrentHashMap` iterators are **weakly consistent** (never throw, may or may not show concurrent changes). `CopyOnWriteArrayList` iterators work on a **snapshot** (and their `remove()` throws `UnsupportedOperationException`).

Trade-offs

Pros	Cons
Client code no longer cares whether data lives in an array, in a tree or behind an API.	One more object is created per traversal (only noticeable in extremely hot loops).
One structure can offer several named, reusable visiting orders.	Changing the collection mid-loop is a bug unless you use the iterator's own `remove` or a concurrent collection.
Elements can be produced lazily, so even huge or endless sequences work.	If a lazy iterator owns a file or DB cursor and the loop breaks early, the resource leaks unless it is closed.
Paging, buffering and prefetching (loading the next data before it is asked for) disappear behind an ordinary loop.	Per-step cost becomes invisible — the root of N+1 problems.

When not to use it

- **The data is already an in-memory list.** Hand out the list (or its own iterator); writing a custom iterator around an array adds code and no value.
- **Callers jump around** ("I need element 4,000"). Iterators only move forward step by step; offer indexed access instead.
- **All elements are in memory and always consumed in full.** An ordinary loop over the list is simpler and faster.
- **A step has side effects the caller must decide on.** Silently loading another page is acceptable; silently charging a customer is not.

Common mistakes

1. **Modifying the collection while iterating** (other than through `Iterator.remove`). Collect changes and apply them after the loop, or use `removeIf`.
2. **Leaking a resource** from a lazy iterator on early `break`. Use try-with-resources (Java's `try (...)` form that closes resources automatically) (`Files.lines`, `DirectoryStream` and JDBC (Java's standard database API) `ResultSet` are all closeable).
3. **Reusing a one-shot iterator** or stream a second time: it is already exhausted. A Java `Stream` at least throws `IllegalStateException`; a Python generator (a function that produces values one at a time with `yield`) just yields nothing the second time, so the bug shows up as missing data. That is why `iterator()` must return a *new* object.
4. **`next()` without `hasNext()`**, or a `next()` that returns `null` at the end instead of throwing `NoSuchElementException`.
5. **Storing the cursor in the collection** — then two loops interfere.

6. **Assuming every language fails loudly.** Java's fail-fast iterators usually throw `ConcurrentModificationException` when you modify a list mid-loop; Python may simply skip elements and finish with a wrong result. Record the intended changes while looping and make them once the loop has finished.

Real-world examples

- **Every for-each loop** in Java, Python, C#, JavaScript (`for...of`).
- **JDBC** `ResultSet` moves through rows that need not all be loaded at once; `Scanner` implements `Iterator<String>`; `Files.newDirectoryStream` returns an `Iterable` over directory entries.
- **Paginated SDKs** (software development kits — a provider's ready-made client libraries): the AWS (Amazon Web Services) SDK for Java v2 "paginators" and Stripe's auto-pagination give you an iterable that fetches pages behind the scenes — exactly section 1.
- **Java** `Stream`**s** are built on `Spliterator`s; that is how parallel streams split work. C#'s LINQ (built-in query syntax for collections) is likewise a lazy layer over iteration, which is why both can work on endless sequences.
- **Generators**: Python's `yield`, JavaScript's `function*` and C#'s `yield return` let you write an iterator as a *function*; the language remembers the position for you. A generator is the same pattern with less code.
- **Kafka consumer** `poll()` **loops**, and **infinite feeds** (Instagram/Twitter timelines) that load the next page as you scroll.

In the interview

> **30-second answer:** "Iterator pulls traversal out into a separate object with `hasNext` and `next`, so a caller can walk data without caring whether the data is an array, a tree or a paginated API. Each iterator keeps its own position, so two loops over the same data are independent. Rather than a home-made interface I'd implement the language's protocol — `Iterable<T>` in Java — so the native for-each works. My example would be the lazy paged case, not an array, and I'd flag the risk: when every step can hit the network, the loop hides its cost — that is how N+1 problems start."

Follow-ups:

- *Internal vs external iterator?* External: the client controls stepping (`hasNext/next`) and can stop, pause or interleave two iterators. Internal: the collection controls the loop and calls your function (`forEach`, streams); simpler, easier to parallelise, but harder to stop early or to walk two collections in lock-step (one element from each at a time).
- *What is fail-fast?* The iterator checks a modification counter and throws `ConcurrentModificationException` if the collection was structurally changed outside the iterator. Best-effort only.
- *How do you remove while iterating?* `Iterator.remove()` or `Collection.removeIf(predicate)` (a predicate is a function that returns true or false).
- *What is a Spliterator?* A splittable iterator used by streams; `trySplit` hands off part of the elements so another thread can process them; characteristics tell the stream framework what it can optimise.
- *Iterator vs Visitor?* Iterator decides **how you move** through a structure; Visitor decides **what you do** at each element. They are often used together.
- *Iterator vs Composite?* Composite makes leaves and branches of a tree look alike; an iterator supplies a way to visit them. The two are often used together.
- *Iterator vs generator?* A generator (Python `yield`, JS `function*`) is the same pattern with the class replaced by a function that `yield`s.
- *Iterator vs stream?* Streams sit on top of iteration and add operations such as `map` and `filter`. When you need to transform data, not merely visit it, reach for streams.
- *How is Iterator expressed in other languages?* Python: `__iter__`/`__next__` or a generator. JavaScript/TypeScript: `[Symbol.iterator]` or `function*`, which makes `for...of` and spread work. C#: `IEnumerable<T>` with `yield return`. C++: `begin()`/`end()` for range-based for and the standard algorithms. Go: traditionally a `Next() bool` plus a value accessor (`sql.Rows`, `bufio.Scanner`), and since Go 1.23 range-over-function iterators.

Self-check

1. **Q:** Where does the cursor live? **A:** In the iterator object.
2. **Q:** What does the enhanced for-loop need from a class? **A:** That it implements `Iterable<T>`.

3. **Q:** Why did five results take four API calls? **A:** Without a total or cursor, the iterator can only discover the end by asking for one more page.
4. **Q:** Does `CopyOnWriteArrayList`'s iterator throw on concurrent modification? **A:** No; it iterates over a snapshot taken when the iterator was created.
5. **Q:** What should `next()` do when there are no more elements? **A:** Throw `NoSuchElementException`.

TL;DR

A separate object walks the collection with `hasNext/next`; the **position lives in the iterator**. Implement `Iterable` in Java. Know **fail-fast** vs snapshot iterators, **internal vs external** iteration, and don't let a lazy loop **hide its cost**.

10.4 Mediator

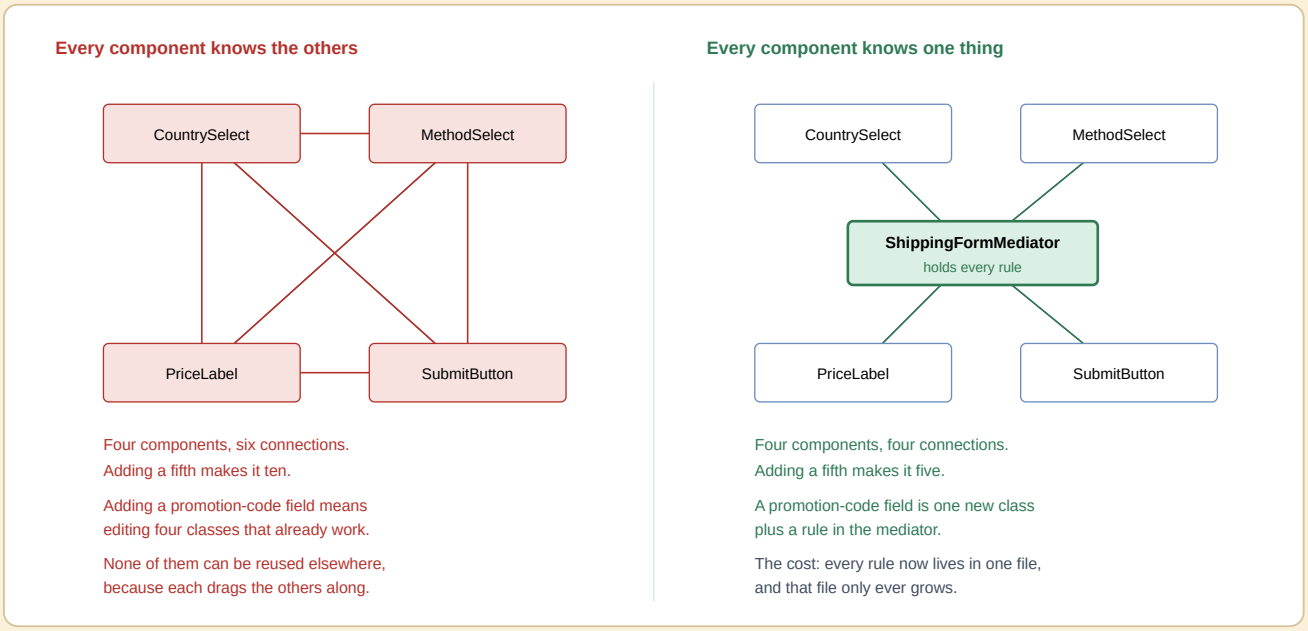


Figure — Left: four form components all know each other (six links). Right: each knows only the mediator (four links) (course diagram)

What it is

Mediator makes a group of objects talk through **a single coordinator** instead of holding references to one another. Each object only reports "this just happened" to the mediator, and the mediator works out how the rest should respond.

GoF intent, in plain words: *define an object that encapsulates how a set of objects interact, so they stop referring to each other explicitly and their interaction can be varied independently.* GoF participants: **Mediator** (interface for talking to colleagues), **ConcreteMediator** (knows the colleagues and implements the coordination), and **Colleague** classes (each knows its mediator and talks to it instead of to other colleagues). Some literature simply calls the mediator a *controller*.

Everyday analogy: **air traffic control**. Planes do not radio each other to agree who lands first. Each plane talks only to the tower, and the tower holds the rules about spacing and order.

The problem it solves

Picture a checkout form with four widgets: a **country** picker, a **delivery-method** picker, a **price** label and a **submit** button. Picking a country refreshes the list of delivery methods; the new method changes the price; the price situation decides whether submit is clickable. For some destinations no delivery is offered at all.

If each widget talks to the others directly:

- every widget holds references to three others — four widgets need **six** connections, five need **ten** (n(n−1)/2, growing roughly with the square);
- a new "promo code" field forces changes to four classes that were already tested and working;

- no widget can be reused on another screen: if `CountrySelect` calls `methodSelect.reload()`, it cannot be used anywhere without bringing `MethodSelect` too.

With a mediator, each widget has **one** reference (to the mediator): four widgets, four links; a fifth makes five. `CountrySelect` is now unaware that any price label exists, so another screen can reuse it with its own mediator and its own rules. Each widget also has no idea how many other widgets exist.

The honest trade: the dependencies are not removed — **they are relocated** into the mediator, a class that tends to keep getting bigger.

How it works

1. Define a **Mediator** interface, e.g. `notify(sender, event)`.
2. Each **component** (colleague) holds a reference to the mediator — and nothing else.
3. When something happens, a component calls `mediator.notify(this, event)`.
4. The **concrete mediator** holds all components and all the rules: "when the country changes, reload methods, then show the price, then enable or disable submit".
5. The mediator calls back **down** into specific components. Traffic is **two-way** through one object.

Structure (UML)

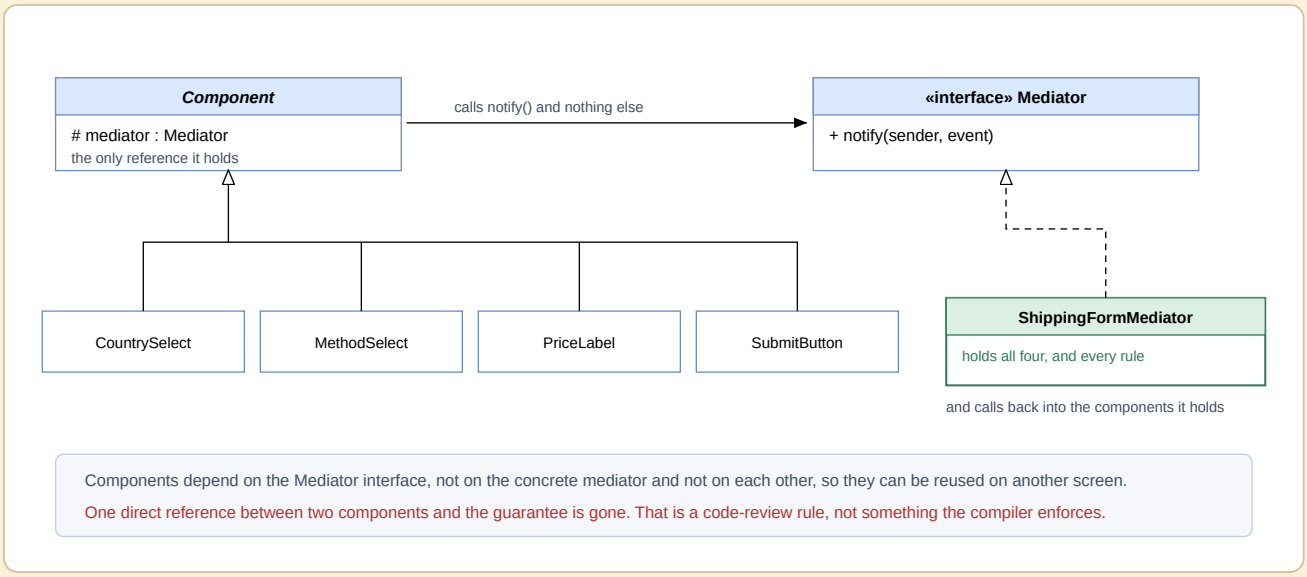
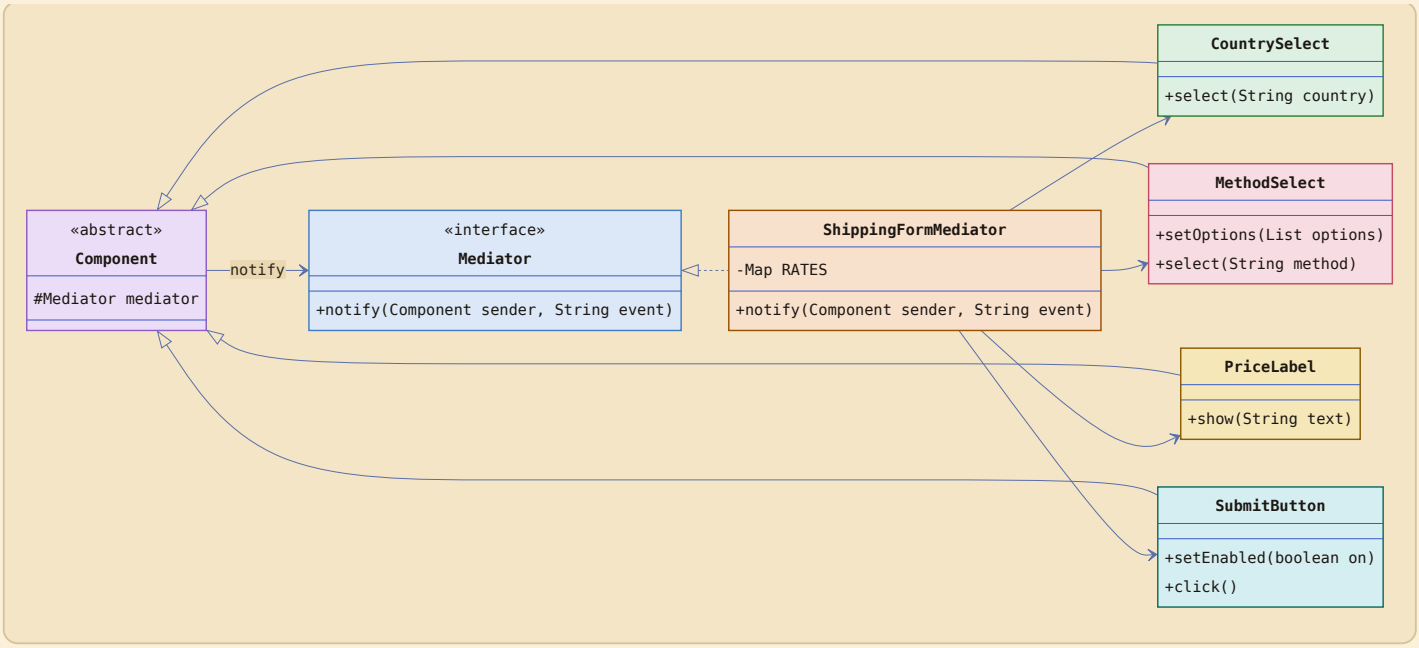
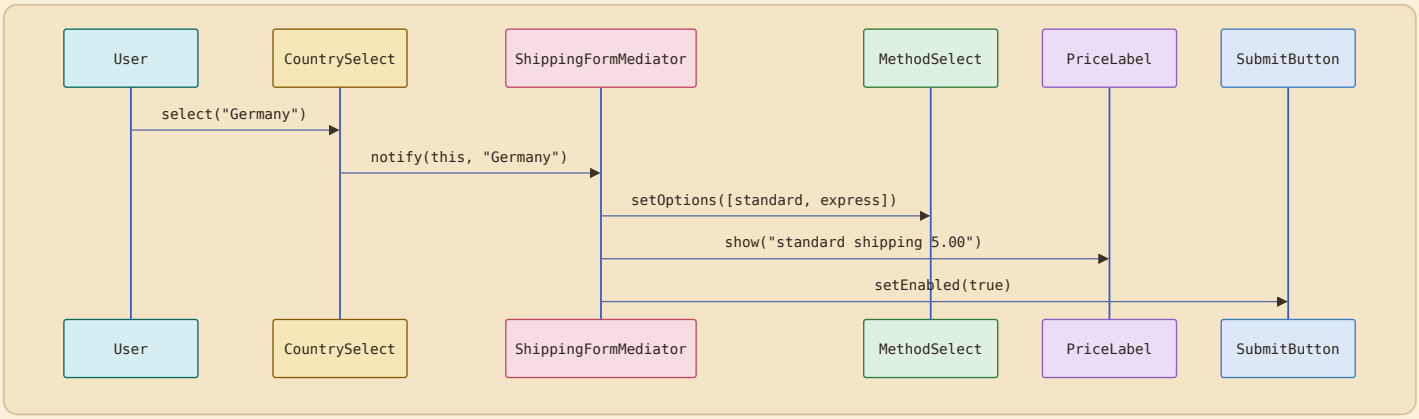


Figure — Class diagram: components extend Component (which holds a Mediator); ShippingFormMediator implements Mediator and holds all four components (course diagram)

In the diagram above, follow the arrows: widgets report **upward** by calling `notify`, and the mediator issues instructions **downward** to the widgets it owns. Traffic in both directions passing through one hub is what separates a mediator from an observer.

Our Java design:





Java example

`lld.ch10.mediator` has three mediators:

1. **Shipping form** (`ShippingFormMediator` + four components). No code except `notify` knows that choosing a country influences the price. Components identify themselves by reference (`sender == country`), not by strings. Shipping rates are `BigDecimal` — money is never stored as `double`.
2. **Chat room** (`ChatRoom`, `ChatUser`). Users never hold references to each other. The room routes broadcasts and private messages and applies **moderation** (filtering or blocking unwanted messages) rules — rules live in the mediator, not in the users.
3. **Air traffic control** (`ControlTower`, `Flight`). Flights only request landing and report "vacated"; the tower owns the runway and the holding queue.

```java
// File: code/lld/ch10/mediator/Mediator.java
package lld.ch10.mediator;

/** Components report "something happened" here. They depend on this interface only. */
public interface Mediator {
    void notify(Component sender, String event);
}
```

```java
// File: code/lld/ch10/mediator/Component.java
package lld.ch10.mediator;

/** Base for every form widget: its ONLY reference is the mediator. */
public abstract class Component {
    protected Mediator mediator;

    void setMediator(Mediator mediator) { this.mediator = mediator; }
}
```

```java
// File: code/lld/ch10/mediator/CountrySelect.java
package lld.ch10.mediator;

final class CountrySelect extends Component {
    private String selected;

    void select(String country) {
        selected = country;
        System.out.println("  [country] user picked " + country);
        mediator.notify(this, country);          // report up; never call a sibling
    }

    String selected() { return selected; }
}
```

```java
// File: code/lld/ch10/mediator/MethodSelect.java
package lld.ch10.mediator;

import java.util.List;

final class MethodSelect extends Component {
    private List<String> options = List.of();
    private String selected;

    void setOptions(List<String> options) {
        this.options = options;
        this.selected = options.isEmpty() ? null : options.get(0);
        System.out.println("  [methods] reloaded: " + options + (selected == null ? "" : ", default " + selected));
    }

    void select(String method) {
        if (!options.contains(method)) throw new IllegalArgumentException(method + " not offered");
        selected = method;
        System.out.println("  [methods] user picked " + method);
        mediator.notify(this, method);
    }

    String selected() { return selected; }
}
```

```java
// File: code/lld/ch10/mediator/PriceLabel.java
package lld.ch10.mediator;

final class PriceLabel extends Component {
    void show(String text) { System.out.println("  [price] " + text); }
}
```

```java
// File: code/lld/ch10/mediator/SubmitButton.java
package lld.ch10.mediator;

final class SubmitButton extends Component {
    private boolean enabled;

    void setEnabled(boolean enabled) {
        this.enabled = enabled;
        System.out.println("  [submit] " + (enabled ? "enabled" : "disabled"));
    }

    void click() {
        if (enabled) mediator.notify(this, "click");
        else System.out.println("  [submit] click ignored: button is disabled");
    }
}
```

```java
// File: code/lld/ch10/mediator/ShippingFormMediator.java
package lld.ch10.mediator;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.Map;

/** The ONLY place that knows how a country change affects methods, price and submit. */
final class ShippingFormMediator implements Mediator {
    private static final Map<String, Map<String, BigDecimal>> RATES = Map.of(
        "Germany", ordered("standard", "5.00", "express", "15.00"),
        "USA", ordered("standard", "8.00", "express", "20.00"),
        "Antarctica", Map.of());

    private final CountrySelect country;
    private final MethodSelect methods;
    private final PriceLabel price;
    private final SubmitButton submit;

    ShippingFormMediator(CountrySelect c, MethodSelect m, PriceLabel p, SubmitButton s) {
        this.country = c; this.methods = m; this.price = p; this.submit = s;
        for (Component comp : new Component[] {c, m, p, s}) comp.setMediator(this);
        submit.setEnabled(false);
    }

    @Override
    public void notify(Component sender, String event) {
        if (sender == country) {
            Map<String, BigDecimal> available = RATES.getOrDefault(event, Map.of());
            methods.setOptions(new ArrayList<>(available.keySet()));
            if (available.isEmpty()) {
                price.show("shipping not available");
                submit.setEnabled(false);
            } else {
                showPrice();                          // ORDER matters: reload methods first, then price
                submit.setEnabled(true);
            }
        } else if (sender == methods) {
            showPrice();
        } else if (sender == submit) {
            System.out.println("  [mediator] order placed: " + country.selected() + " / " + methods.selected());
        }
    }

    private void showPrice() {
        BigDecimal p = RATES.get(country.selected()).get(methods.selected());   // money: BigDecimal, never double
        price.show(methods.selected() + " shipping " + p.toPlainString());
    }

    private static Map<String, BigDecimal> ordered(String k1, String v1, String k2, String v2) {
        Map<String, BigDecimal> m = new LinkedHashMap<>();   // keeps "standard" before "express"
        m.put(k1, new BigDecimal(v1));
        m.put(k2, new BigDecimal(v2));
        return m;
    }
}
```

```java
// File: code/lld/ch10/mediator/ChatRoom.java
package lld.ch10.mediator;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Set;

/** Chat mediator: users never hold references to each other, only to the room. */
final class ChatRoom {
    private final Map<String, ChatUser> members = new LinkedHashMap<>();
    private final Set<String> bannedWords = Set.of("spam");

    void join(ChatUser user) {
        members.put(user.name(), user);
        user.joined(this);
        broadcast("room", user.name() + " joined");
    }

    /** Room rules (moderation, routing) live here, not in the users. */
    void send(ChatUser from, String to, String text) {
        if (bannedWords.stream().anyMatch(text.toLowerCase()::contains)) {
            from.receive("room", "message blocked by moderation");
            return;
        }
        if (to == null) {
            members.values().stream().filter(u -> u != from).forEach(u -> u.receive(from.name(), text));
        } else if (members.containsKey(to)) {
            members.get(to).receive(from.name() + " (private)", text);
        } else {
            from.receive("room", to + " is not in the room");
        }
    }

    private void broadcast(String from, String text) {
        members.values().forEach(u -> u.receive(from, text));
    }
}
```

```java
// File: code/lld/ch10/mediator/ChatUser.java
package lld.ch10.mediator;

final class ChatUser {
    private final String name;
    private ChatRoom room;

    ChatUser(String name) { this.name = name; }

    String name() { return name; }
    void joined(ChatRoom room) { this.room = room; }
    void say(String text) { room.send(this, null, text); }
    void whisper(String to, String text) { room.send(this, to, text); }
    void receive(String from, String text) { System.out.println("  " + name + " sees <" + from + "> " + text); }
}
```

```java
// File: code/lld/ch10/mediator/ControlTower.java
package lld.ch10.mediator;

import java.util.ArrayDeque;
import java.util.Queue;

/** Air traffic control: aircraft talk to the tower, never to each other. */
final class ControlTower {
    private Flight onRunway;                          // null = runway free
    private final Queue<Flight> holding = new ArrayDeque<>();

    void requestLanding(Flight f) {
        if (onRunway == null) {
            clear(f);
        } else {
            holding.add(f);
            System.out.println("  [tower] " + f.callsign() + ": hold, runway busy with " + onRunway.callsign());
        }
    }

    void runwayVacated(Flight f) {
        System.out.println("  [tower] " + f.callsign() + " vacated the runway");
        onRunway = null;
        Flight next = holding.poll();
        if (next != null) clear(next);
    }

    private void clear(Flight f) {
        onRunway = f;
        System.out.println("  [tower] " + f.callsign() + ": cleared to land");
    }
}
```

```java
// File: code/lld/ch10/mediator/Flight.java
package lld.ch10.mediator;

record Flight(String callsign, ControlTower tower) {
    void requestLanding() { tower.requestLanding(this); }
    void landed() { tower.runwayVacated(this); }
}
```

```java
// File: code/lld/ch10/mediator/Main.java
package lld.ch10.mediator;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Shipping form: widgets coordinated by a mediator ===");
        CountrySelect country = new CountrySelect();
        MethodSelect methods = new MethodSelect();
        PriceLabel price = new PriceLabel();
        SubmitButton submit = new SubmitButton();
        new ShippingFormMediator(country, methods, price, submit);

        country.select("Germany");        // reloads methods, shows price, enables submit
        methods.select("express");        // updates price only
        submit.click();
        country.select("Antarctica");     // no methods: price hidden, submit disabled
        submit.click();

        System.out.println("\n=== 2. Chat room ===");
        ChatRoom room = new ChatRoom();
        ChatUser asha = new ChatUser("asha"), ben = new ChatUser("ben"), chen = new ChatUser("chen");
        room.join(asha);
        room.join(ben);
        room.join(chen);
        asha.say("standup in 5");
        ben.whisper("chen", "coffee first?");
        chen.say("buy cheap SPAM now");

        System.out.println("\n=== 3. Air traffic control ===");
        ControlTower tower = new ControlTower();
        Flight ai101 = new Flight("AI101", tower), ba22 = new Flight("BA22", tower), lh7 = new Flight("LH7",
tower);
        ai101.requestLanding();
        ba22.requestLanding();
        lh7.requestLanding();
        ai101.landed();
        ba22.landed();
    }
}
```

Output:

```
=== 1. Shipping form: widgets coordinated by a mediator ===
  [submit] disabled
  [country] user picked Germany
  [methods] reloaded: [standard, express], default standard
  [price] standard shipping 5.00
  [submit] enabled
  [methods] user picked express
  [price] express shipping 15.00
  [mediator] order placed: Germany / express
  [country] user picked Antarctica
  [methods] reloaded: []
  [price] shipping not available
  [submit] disabled
  [submit] click ignored: button is disabled

=== 2. Chat room ===
  asha sees <room> asha joined
  asha sees <room> ben joined
  ben sees <room> ben joined
  asha sees <room> chen joined
  ben sees <room> chen joined
  chen sees <room> chen joined
  ben sees <asha> standup in 5
  chen sees <asha> standup in 5
  chen sees <ben (private)> coffee first?
  chen sees <room> message blocked by moderation

=== 3. Air traffic control ===
  [tower] AI101: cleared to land
  [tower] BA22: hold, runway busy with AI101
  [tower] LH7: hold, runway busy with AI101
  [tower] AI101 vacated the runway
  [tower] BA22: cleared to land
  [tower] BA22 vacated the runway
  [tower] LH7: cleared to land
```

One user action ("picked Germany") updated three components — and **no component called another**.

Mediator or Observer?

Both decouple objects. The difference is **what happens to the news**.

	Mediator	**Observer**
What travels	An event **and** the decision on what should follow	Only the event
Where reaction logic sits	In one place: the mediator	Spread across the observers, each deciding alone
Do the parties know each other?	No — not even how many peers exist	No; the subject only knows it has a list of subscribers
Flow	Both ways: colleagues report in, the mediator calls out to them	One way: the subject publishes, observers respond
Order of reactions	Written explicitly in the mediator — a reason to choose it	Not part of the contract; observers should not depend on each other's order

Practical test: does something need to be coordinated? "The country changed; anyone interested, update yourself" is Observer, and it is the lighter choice. "The country changed: refresh the methods, *after that* price the new default method, *after that* decide about submit" is sequenced coordination — Mediator territory. Short version: **with Observer the listeners decide; with Mediator the hub decides and gives orders.** (A mediator is often *implemented* using observer-style notifications from the components.)

Trade-offs

Pros	Cons
Every widget depends on exactly one thing, so links grow linearly instead of quadratically.	Every new rule lands in the mediator, so it keeps growing.
All interaction rules sit in one class you can read and unit-test.	Anyone changing the screen changes that same file, so merge conflicts (clashing edits to the same lines) pile up there.
Widgets are reusable because they no longer pull their neighbours with them.	To learn why a widget changed you read the mediator, not the widget.
A new widget costs one class plus one rule.	It can quietly become a **god object** (a class that knows and does too much).

Keeping the mediator from becoming a god object:

- **Split by area** — one mediator per screen section or workflow, not one per application. Two 80-line mediators are much easier to live with than one 160-line one.
- **Keep it to coordination.** Tax and pricing calculations belong in their own classes; the mediator decides *what to call*, not *how to compute* (a Facade has to respect the same boundary).
- **Reject shortcuts in code review.** A single widget-to-widget reference breaks the whole guarantee, and the compiler will not catch it.
- **Dispatch by table, not by `if`-chain.** In languages with first-class functions the mediator can keep a map from event (or sender) to handler function — e.g. a Python `dict` of `{event: method}` — so each rule is easy to find.

When not to use it

- **Only two objects interact.** Let them call each other; a coordinator for a single relationship adds a layer that does nothing.
- **The interactions are few and stable.** The pattern swaps many small, scattered links for one bigger central class; that only pays off once the tangle is large.
- **Nothing needs coordinating** — components react independently. Use Observer.
- **The mediator is already a 400-line god object.** More rules will not help; break it into smaller mediators first.

Common mistakes

1. **Components calling each other "just this once".**
2. **Computation inside the mediator** (tax, pricing), turning it into a god object.
3. **Confusing it with Facade.** A Facade is placed *in front of* a subsystem, and the subsystem's classes are unaware of it; calls flow one way. A Mediator is placed *among* peers that all know it and report to it; calls flow both ways.
4. **String-typed events everywhere** (`"country"`, `"method"`) that silently break on a typo. Use the sender reference, an enum, or typed events.

Real-world examples

- **UI controllers / view-models** coordinating several widgets (MVC — Model–View–Controller — controllers, MVVM — Model–View–ViewModel — view-models, Android `ViewModel` wiring). GoF's own example is a dialog box coordinating its widgets.
- **Chat servers** (Slack, WhatsApp): clients send messages to the server, which routes them to rooms and people and applies moderation.
- **Air traffic control** — the classic analogy.
- **Workflow / saga orchestrators** (a saga is a multi-step process spread across services, each step with its own undo; e.g. Temporal, Camunda, AWS Step Functions): a checkout orchestrator invokes payment, stock reservation and shipping in sequence and holds the recovery plan for each failure. Ride-hailing dispatch (matching riders and drivers through a central service rather than drivers talking to riders directly) has the same shape.
- **Redux store** at application scale (components dispatch actions instead of calling each other; reducers hold the rules); **MediatR** in .NET popularised the name on the server side, using it to dispatch each request to its handler.

- **Message brokers / event buses** (servers that receive messages and deliver them to the right consumers) at system scale: a service hands messages to the broker rather than calling other services; the broker's routing configuration is the mediator logic — and it can become a god object too.
- `java.util.Timer` and `Executor`/`ExecutorService` are often listed (including by the course) as JDK mediators: they stand between submitted tasks and the threads that run them, so neither side knows the other. Treat this as a *loose* fit — they do not coordinate peer colleagues the way the GoF dialog example does, so in an interview lead with UI controllers or orchestrators.

In the interview

> **30-second answer:** "Mediator swaps peer-to-peer references for a single hub. Widgets just report events to it; the hub owns the rules and instructs particular widgets. Four widgets go from six links to four, and a fifth gives five instead of ten. The dependencies aren't gone — they're concentrated in one class that I can read and test. The risk is a god object, so I split mediators per screen area or workflow and push computation into its own classes."

Follow-ups:

- *Mediator vs Observer?* With Observer each listener decides for itself; with Mediator the hub decides and instructs. When the sequence of reactions matters, pick Mediator.
- *Mediator vs Facade?* Facade simplifies a subsystem for outside callers (one-way, subsystem unaware). Mediator coordinates peers that know it (two-way).
- *How does it relate to Command?* A mediator often creates and dispatches commands.
- *How do you test a mediator?* Give it fake components and assert which calls it made for each event.
- *What about in microservices (a system split into many small, separately deployed services)?* Orchestration (a central saga orchestrator) is mediator-style; choreography (services react to each other's events) is observer-style.

Self-check

1. **Q:** How many links do 5 fully connected components need, and how many with a mediator? **A:** 10 vs 5.
2. **Q:** Does a mediator remove coupling? **A:** No, it moves it into one class.
3. **Q:** What is the main failure mode? **A:** The mediator grows into a god object.
4. **Q:** Which way does traffic flow in a Mediator vs an Observer? **A:** Mediator two-way; Observer one-way.
5. **Q:** What should *not* go into a mediator? **A:** Business calculations — only coordination.

TL;DR

Peers talk through **one coordinator** that holds the interaction rules. Links become linear and components reusable, but the coupling **moves** into the mediator — keep it small and split it before it becomes a god object.

10.5 Memento

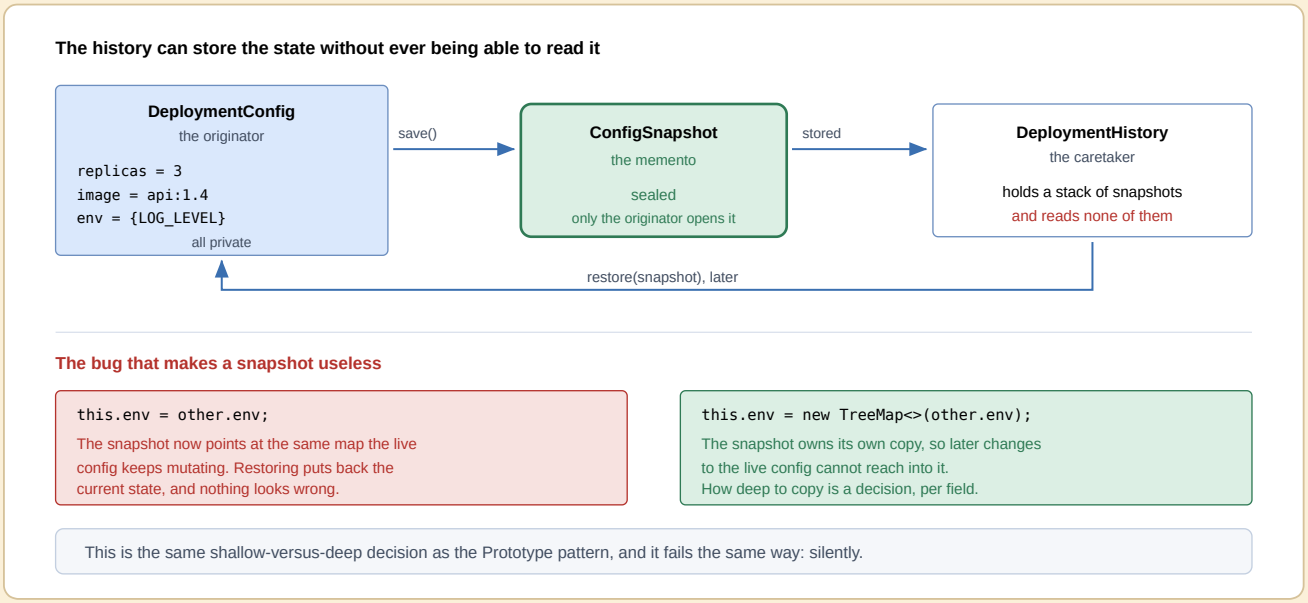


Figure — The originator (DeploymentConfig) saves a sealed snapshot; the caretaker (DeploymentHistory) stores it without being able to read it; below, the shared-map bug that makes a snapshot useless (course diagram)

What it is

Memento lets an object record its internal state in a **snapshot** and later **return** to that state — while nobody else gets to **see** what the snapshot contains.

GoF intent, in plain words: *without breaking encapsulation, capture and externalize an object's internal state so the object can be put back into that state later.* GoF participants: **Memento** (stores the state; offers a narrow interface to the caretaker and a wide one to the originator), **Originator** (creates a memento of its current state and uses one to restore itself) and **Caretaker** (keeps mementos safe but never operates on or examines their contents).

Everyday analogy: a **save game**. The game writes a save file; the menu lists the saves by name and date and lets you load one. The menu never needs to understand what is inside the file.

The problem it solves

You need a **rollback** (a return to the previous working version): record the deployment configuration just before a risky change, and if the rollout (the release of a change to the running servers) misbehaves, bring the recorded version back.

The obvious way — something *outside* reads every field and stores them — breaks two things:

1. **Encapsulation.** The history class must now understand replicas (copies of the service running side by side), image names (which packaged build to run) and environment variables (named settings passed to a program). Every new config field forces a change in the history class, and details that were private are now exposed.
2. **Atomicity.** Atomicity means all-or-nothing; a restore done one field at a time can stop half-way. If three fields are written and an exception hits before the fourth, the config holds a combination that was never real.

With Memento the object **takes its own snapshot** and hands it out as a sealed token that **only it can open**. Restoring becomes **one call into the originator**, which is in full control of the restore and can make it all-or-nothing (our `restore` validates the token first, then assigns every field; nothing in between can fail half-way).

How it works

Three roles, with standard names interviewers use:

Role	Job	In our code
Originator	Owns the state; produces snapshots (`save`) and returns to them (`restore`). No other class can look inside a snapshot.	`DeploymentConfig`
Memento	Carries the state; opaque to everyone else.	`DeploymentConfig.Snapshot` (narrow) / private `State` record (wide)
Caretaker	Decides **when** to save and restore; never looks **inside**.	`DeploymentHistory`

Steps:

1. Caretaker asks the originator: `snapshot = config.save(label)`.
2. Caretaker stores the snapshot (usually on a stack).
3. Later, caretaker calls `config.restore(snapshot)`.
4. The originator copies the state back in one step.

How opacity is enforced in Java. GoF describes a **narrow interface** (what the caretaker sees) and a **wide interface** (what the originator sees). Our design does this with types: the public interface `Snapshot` exposes only `label()`; the real data is in a **private nested record** `State` that only `DeploymentConfig` can see. `restore` checks `instanceof State`, so even a forged `Snapshot` (see section 4 of the output) is refused. (Strictly, reflection — Java's ability to inspect and call even private members at run time — or a debugger can still peek; Java access control protects against honest code, not a hostile JVM, the Java Virtual Machine that runs the program.) Other options by language: package-private accessors in Java; the originator declared a `friend` in C++; a private nested class in C#; Python and JavaScript offer no true enforcement, so it is a naming convention (leading underscore) plus code review — although a JavaScript closure *can* hide the contents in a way a class field cannot.

Structure (UML)

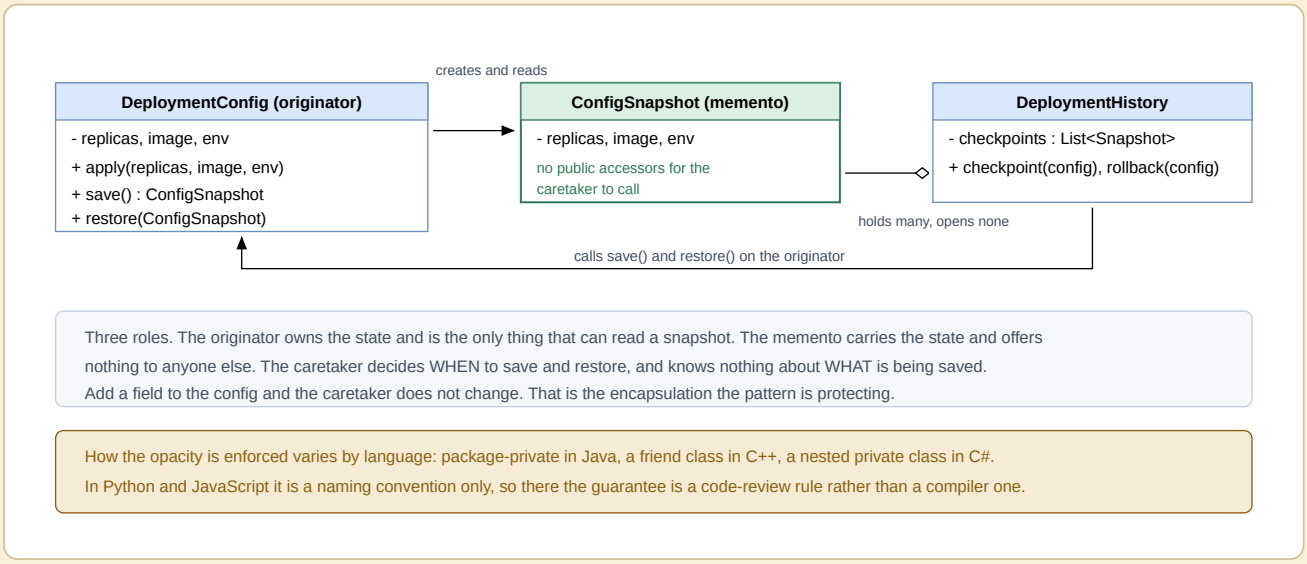
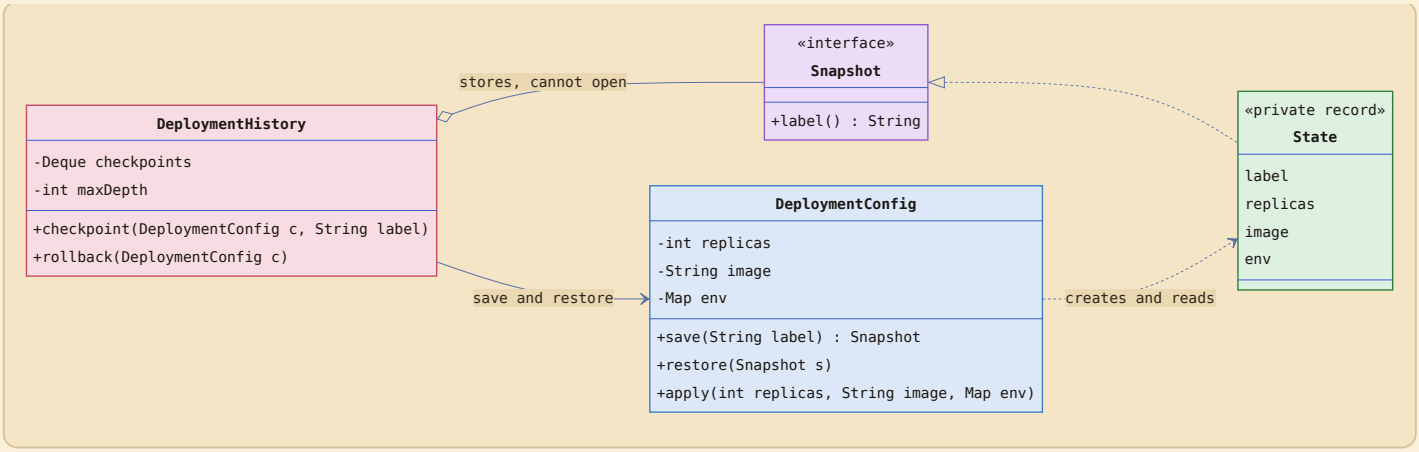
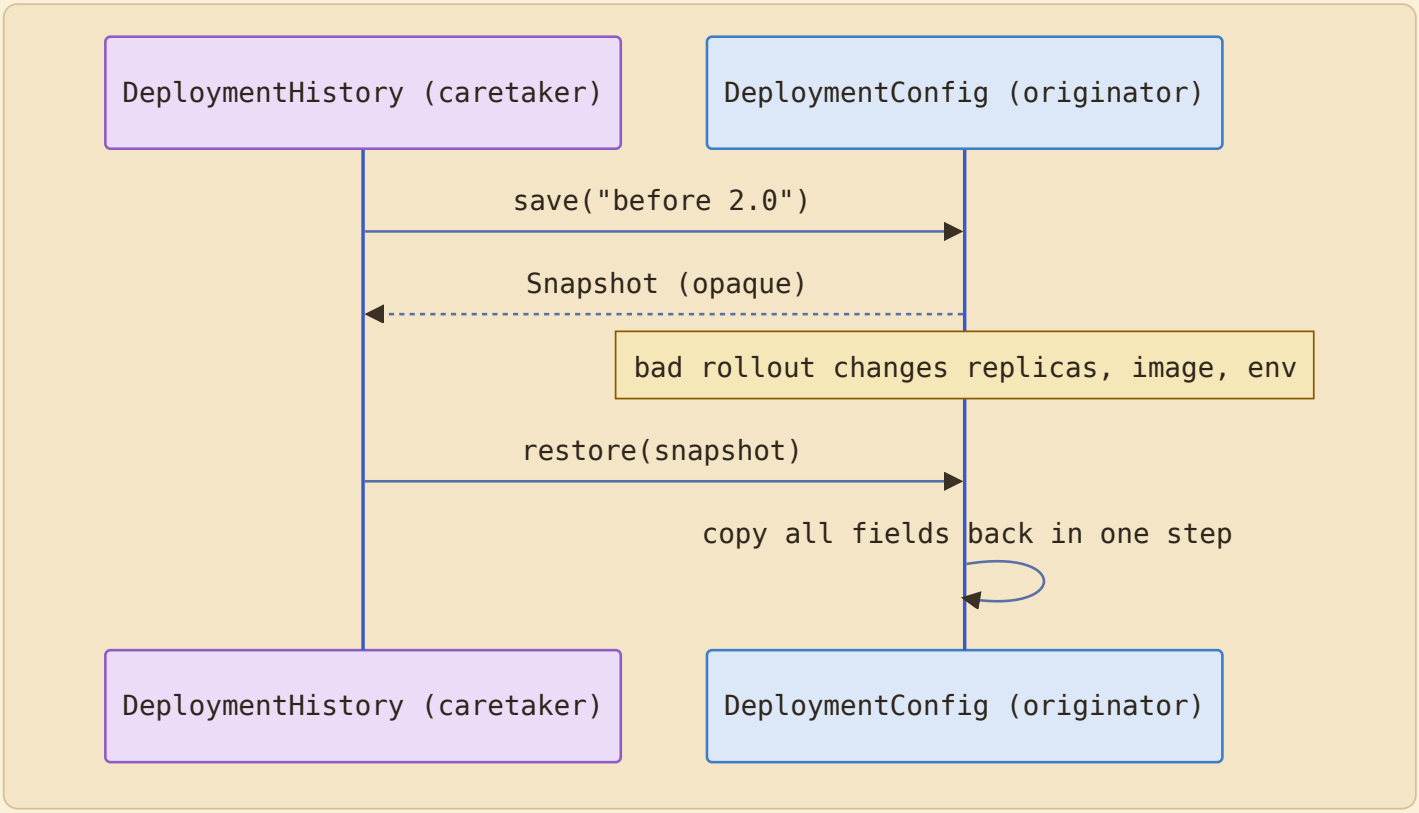


Figure — Class diagram: DeploymentConfig creates and restores ConfigSnapshot; DeploymentHistory holds a list of snapshots (course diagram)

In the diagram above, `DeploymentHistory` never refers to a single configuration field. That is the quick check for a real Memento: if the caretaker names the fields, you have only relocated them.

Our Java design:





Java example

Points to notice in `lld.ch10.memento`:

- The `State` record's (a record is Java's short syntax for an immutable data class) compact constructor (a constructor with no parameter list that runs on every creation) does `env = Map.copyOf(env)`, so the snapshot **owns its own copy**. In section 1 the live map is changed *after* saving (`FEATURE_X`), and the rollback still restores the original.
- `DeploymentHistory` is **bounded** (`maxDepth`) and drops the oldest checkpoint when full.
- Section 3 shows the **shallow-copy bug** in isolation.

```java
// File: code/lld/ch10/memento/DeploymentConfig.java
package lld.ch10.memento;

import java.util.Map;
import java.util.TreeMap;

/** The ORIGINATOR: owns the state, and is the only class that can read a snapshot. */
public final class DeploymentConfig {

    /**
     * The NARROW interface the caretaker sees: a label, nothing else.
     * (GoF: caretaker gets a narrow interface, originator a wide one.)
     */
    public interface Snapshot {
        String label();
    }

    /** The WIDE interface: private, so only DeploymentConfig can open it. Immutable record. */
    private record State(String label, int replicas, String image, Map<String, String> env)
            implements Snapshot {
        State {
            env = Map.copyOf(env);   // DEEP enough: the snapshot owns its own copy, not the live map
        }
    }

    private int replicas;
    private String image;
    private final Map<String, String> env = new TreeMap<>();

    public DeploymentConfig(int replicas, String image, Map<String, String> env) {
        apply(replicas, image, env);
    }

    public void apply(int replicas, String image, Map<String, String> env) {
        this.replicas = replicas;
        this.image = image;
        this.env.clear();
        this.env.putAll(env);
    }

    public void setEnv(String key, String value) { env.put(key, value); }

    public Snapshot save(String label) {
        return new State(label, replicas, image, env);
    }

    /** One call restores everything: all-or-nothing. */
    public void restore(Snapshot snapshot) {
        if (!(snapshot instanceof State s)) {
            throw new IllegalArgumentException("not a snapshot produced by DeploymentConfig");
        }
        apply(s.replicas(), s.image(), s.env());
    }

    @Override
    public String toString() {
        return "replicas=" + replicas + " image=" + image + " env=" + env;
    }
}
```

```java
// File: code/lld/ch10/memento/DeploymentHistory.java
package lld.ch10.memento;

import java.util.ArrayDeque;
import java.util.Deque;

/** The CARETAKER: decides WHEN to save/restore, never WHAT is inside. Bounded to avoid leaks. */
public final class DeploymentHistory {
    private final Deque<DeploymentConfig.Snapshot> checkpoints = new ArrayDeque<>();
    private final int maxDepth;

    public DeploymentHistory(int maxDepth) { this.maxDepth = maxDepth; }

    public void checkpoint(DeploymentConfig config, String label) {
        if (checkpoints.size() == maxDepth) {
            DeploymentConfig.Snapshot dropped = checkpoints.removeLast();   // oldest
            System.out.println("  [history] full, dropped oldest checkpoint '" + dropped.label() + "'");
        }
        checkpoints.push(config.save(label));
        System.out.println("  [history] checkpoint '" + label + "' taken");
    }

    public void rollback(DeploymentConfig config) {
        if (checkpoints.isEmpty()) { System.out.println("  [history] nothing to roll back to"); return; }
        DeploymentConfig.Snapshot s = checkpoints.pop();
        config.restore(s);
        System.out.println("  [history] rolled back to '" + s.label() + "'");
    }
}
```

```java
// File: code/lld/ch10/memento/Main.java
package lld.ch10.memento;

import java.util.HashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Checkpoint, bad rollout, rollback ===");
        DeploymentConfig config = new DeploymentConfig(3, "api:1.4", Map.of("LOG_LEVEL", "info"));
        DeploymentHistory history = new DeploymentHistory(2);
        System.out.println("  live: " + config);

        history.checkpoint(config, "before 2.0");
        config.apply(10, "api:2.0", Map.of("LOG_LEVEL", "debug"));   // the risky rollout
        config.setEnv("FEATURE_X", "on");                             // mutates the live map after saving
        System.out.println("  live: " + config);
        history.rollback(config);
        System.out.println("  live: " + config + "   <- snapshot was a real copy");

        System.out.println("\n=== 2. Bounded history ===");
        history.checkpoint(config, "v1");
        history.checkpoint(config, "v2");
        history.checkpoint(config, "v3");      // drops v1
        history.rollback(config);
        history.rollback(config);
        history.rollback(config);

        System.out.println("\n=== 3. The shallow-copy bug, shown in isolation ===");
        Map<String, String> liveEnv = new HashMap<>(Map.of("LOG_LEVEL", "info"));
        Map<String, String> badSnapshot = liveEnv;                  // shares the live map
        Map<String, String> goodSnapshot = new HashMap<>(liveEnv);  // owns a copy
        liveEnv.put("LOG_LEVEL", "debug");
        System.out.println("  bad snapshot  = " + badSnapshot + "  <- changed with the live object");
        System.out.println("  good snapshot = " + goodSnapshot);

        System.out.println("\n=== 4. The caretaker cannot open a snapshot ===");
        DeploymentConfig.Snapshot s = config.save("peek");
        System.out.println("  caretaker can read only: label=" + s.label());
        try {
            config.restore(() -> "forged");                         // a Snapshot not made by the originator
        } catch (IllegalArgumentException e) {
            System.out.println("  forged snapshot refused: " + e.getMessage());
        }
    }
}
```

Output:

```
=== 1. Checkpoint, bad rollout, rollback ===
  live: replicas=3 image=api:1.4 env={LOG_LEVEL=info}
  [history] checkpoint 'before 2.0' taken
  live: replicas=10 image=api:2.0 env={FEATURE_X=on, LOG_LEVEL=debug}
  [history] rolled back to 'before 2.0'
  live: replicas=3 image=api:1.4 env={LOG_LEVEL=info}   <- snapshot was a real copy

=== 2. Bounded history ===
  [history] checkpoint 'v1' taken
  [history] checkpoint 'v2' taken
  [history] full, dropped oldest checkpoint 'v1'
  [history] checkpoint 'v3' taken
  [history] rolled back to 'v3'
  [history] rolled back to 'v2'
  [history] nothing to roll back to

=== 3. The shallow-copy bug, shown in isolation ===
  bad snapshot  = {LOG_LEVEL=debug}  <- changed with the live object
  good snapshot = {LOG_LEVEL=info}

=== 4. The caretaker cannot open a snapshot ===
  caretaker can read only: label=peek
  forged snapshot refused: not a snapshot produced by DeploymentConfig
```

The bug that makes a snapshot useless

If the memento keeps a **pointer to the very map (or list) the originator goes on mutating**, you have not taken a snapshot: the "saved" copy drifts along with the live object, and a restore just re-applies today's state. Nothing throws and nothing looks wrong — until somebody notices the rollback had no effect. Always copy mutable parts (`Map.copyOf`, `new TreeMap<>(...)`, deep copies — copies that also duplicate the objects inside — of nested objects). It is the same shallow-vs-deep copy decision as the Prototype pattern (Chapter 8).

Two more things to bound: history depth — keep a cap and evict the oldest entry, because a short undo list is useful while an endless one is just a memory leak (memory that keeps growing and is never released) — and **state that cannot be captured** (live sockets — open network connections — open files, data owned by another system). A snapshot that claims to cover those will fail, usually long after the mistake was made.

Memento or Command undo?

	Memento	Command undo
Saved per step	A full copy of the state at that moment	The operation plus the little bit of state its reversal needs
How undo happens	Re-install an earlier copy	Execute the opposite operation
Memory cost per step	Proportional to the state size	Proportional to one operation
Correctness risk	Low — the copy was a genuine state	Every reverse operation must be written correctly
Prefer it when	Reversal is tricky or the state is small	Reversal is simple or the state is big

A drawing app that snapshots the whole canvas on every brush stroke runs out of memory — use Command ("stroke added, here are its points"). A wizard (a multi-step form) where step 3 changes six interacting fields is the opposite — six correct inverses are hard, one snapshot is easy. **Real systems combine them:** take a snapshot occasionally, log commands in between; to go back, restore the nearest snapshot and replay commands forward. Databases and event-sourced systems (Chapter 11) do exactly this.

Trade-offs

Pros	Cons
Rollback is possible while the internals stay private.	Memory use rises with every stored snapshot, quickly for large state.
New fields in the originator need no change in the caretaker.	Mutable parts must be deep-copied, or snapshots share live data.
The originator controls the restore, so it can be all-or-nothing, and it returns a state that really existed.	Some languages cannot truly hide the snapshot, only by convention.
Checkpoints, wizards, save games, transactions: one mechanism.	Some state (sockets, external systems) cannot be captured at all.

When not to use it

- **Large state that changes often** — use Command undo or periodic snapshots + replay.
- **State you cannot capture** (sockets, file handles, remote systems).
- **The object is immutable.** An old instance already *is* a perfect snapshot; holding a reference to it is enough (which is why functional code rarely names this pattern).
- **Each operation has an easy exact inverse** — per-step Command undo costs less memory.

Common mistakes

1. Storing references to mutable internals (the shallow-copy bug).
2. Letting the caretaker read or edit the snapshot's fields (encapsulation lost).
3. Restoring field by field from outside (not atomic).
4. Unbounded history.
5. Snapshotting things that cannot really be restored (connections, locks).

Real-world examples

- **Database transactions** (groups of changes that succeed or fail together): `BEGIN` ... `ROLLBACK`. The DB keeps enough information (undo logs) to put things back; your code never sees it.
- **Version control** (tools that record the history of files): a Git commit is a snapshot of the tree; `git checkout` restores it, and the history (the caretaker) does not need to understand file contents to do its job.
- **Kubernetes Deployments** (Kubernetes runs and manages containers across many machines) keep old ReplicaSets (the group of identical running copies for each version) as revision history (bounded by `revisionHistoryLimit`, default 10), and `kubectl rollout undo` restores a previous pod template — a bounded caretaker, just like ours.
- **VM / container snapshots** (VM = virtual machine, a whole computer simulated in software; a container is a lighter, isolated, packaged process) and infrastructure state files that make `rollback` possible.
- **Game saves and checkpoints**; **form wizards** with a "back" button; **editor undo** for small documents.
- In Java, `Serializable` or a JSON mapper is often *how* a memento is stored when it must outlive the process; a `record` gives an immutable memento in one line.
- **Other languages**: in Python, `copy.deepcopy(self.__dict__)` is a quick memento and `dataclasses.replace` covers immutable objects; in JavaScript, `structuredClone(state)` deep-copies plain data.

In the interview

30-second answer: "With Memento an object packs its state into a token that only the object itself can unpack. The originator produces and consumes snapshots, the memento holds the data, and the caretaker chooses when to save or roll back without ever inspecting the contents. Restoring is a single call the originator controls, so it can be all-or-nothing, and a new field in the object never touches the history code. I'd make sure the snapshot deep-copies mutable parts, bound the history, and compare it with Command undo: Memento stores state, Command stores actions; real systems combine periodic snapshots with a command log."

Follow-ups:

- *How do you enforce that the caretaker can't read the memento in Java?* A narrow public interface (e.g. only a label) implemented by a private nested record; or package-private accessors.
- *Memento vs Prototype?* Each copies state. A prototype clone becomes a live object you keep working with. A memento does nothing on its own; its sole purpose is to be given back to the originator.
- *Memento vs a backup?* A backup targets disaster recovery (getting systems back after a major failure) and lives outside the process (disk, another region — a separate data-centre location). A memento is usually an in-memory checkpoint for undo. Same idea, very different lifetime and cost.
- *How would you implement undo for a large document?* Command undo for each edit, plus periodic snapshots so undo far back doesn't replay thousands of commands.
- *How does it relate to serialization?* Serialization is one common *storage format* for a memento; the pattern itself is about *access* — only the originator may look inside.

Self-check

1. **Q:** Name the three roles. **A:** Originator, memento, caretaker.
2. **Q:** Who may read the memento's contents? **A:** Only the originator.
3. **Q:** Why copy the `env` map in the snapshot? **A:** Otherwise the snapshot shares the live map and changes with it.
4. **Q:** When is Command undo better than Memento? **A:** When state is large and each action has an easy inverse.
5. **Q:** What does Kubernetes' `revisionHistoryLimit` correspond to? **A:** A bounded caretaker history.

TL;DR

The object **snapshots itself**; the caretaker **stores but cannot read**; restore is **one call the originator controls**. **Deep-copy** mutable parts, **bound** the history, and choose Memento (store state) vs Command (store actions) by which is smaller.

10.6 Interpreter

(Not covered by a lesson in the course beyond its one-line entry in the behavioral-patterns table; this section is from the GoF book and standard practice.)

What it is

Interpreter represents the **grammar** of a small language as a set of classes — **one class per grammar rule** — and evaluates a sentence of that language by building a **tree** of those objects and asking the root to `interpret` itself.

Everyday analogy: a calculator. `2 + 3 * 4` is a sentence in a tiny language. The calculator understands that `*` binds tighter than `+`, builds the structure `2 + (3 * 4)` in its head, and works it out from the inside.

Key terms:

- **Grammar** — the rules saying which sentences are valid, e.g. "an expression is terms joined by `+` or `-`".
- **DSL (domain-specific language)** — a small language for one job: search filters, pricing rules, feature-flag conditions, cron schedules (a compact text format for repeating times, such as "every weekday at 9").
- **AST (abstract syntax tree)** — the tree form of a sentence, with operators as inner nodes and numbers/variables as leaves.
- **Terminal expression** — a leaf rule (a number, a variable).
- **Nonterminal expression** — a rule made of other expressions (addition, multiplication).
- **Context** — outside information the evaluation needs, e.g. the current values of variables.

The problem it solves

Some problems keep coming back as *sentences*: "price * qty − discount", "age > 18 AND country = IN", "every weekday at 9:00". Hard-coding each rule in Java means a redeploy (building and releasing the application again) for every new rule. Letting business users write rules in a tiny language, and interpreting them, turns rules into **data**. Interpreter is the simplest object-oriented way to evaluate such a language when the grammar is **small**.

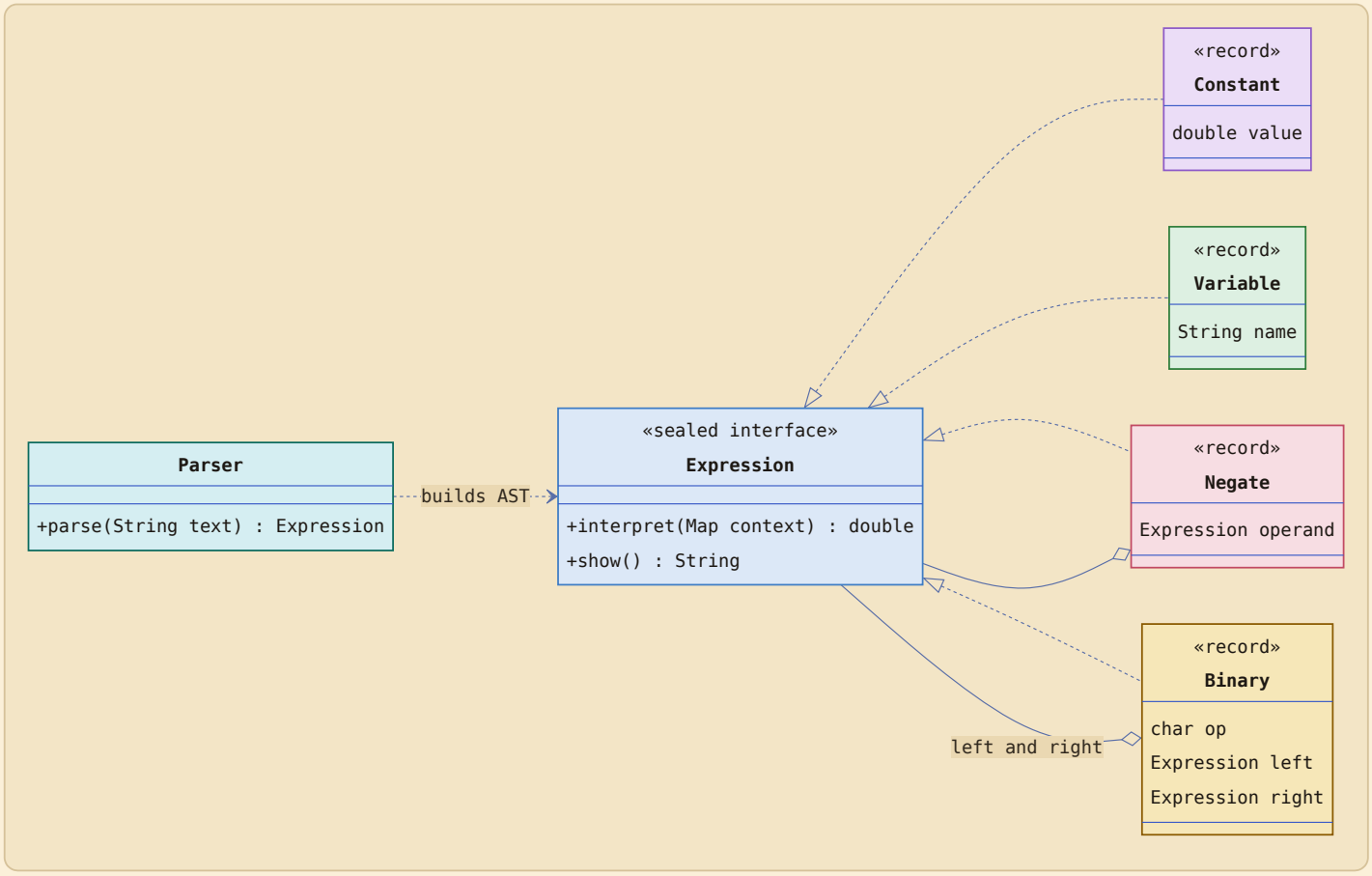
How it works

1. **Write the grammar** (here in EBNF, a standard notation):

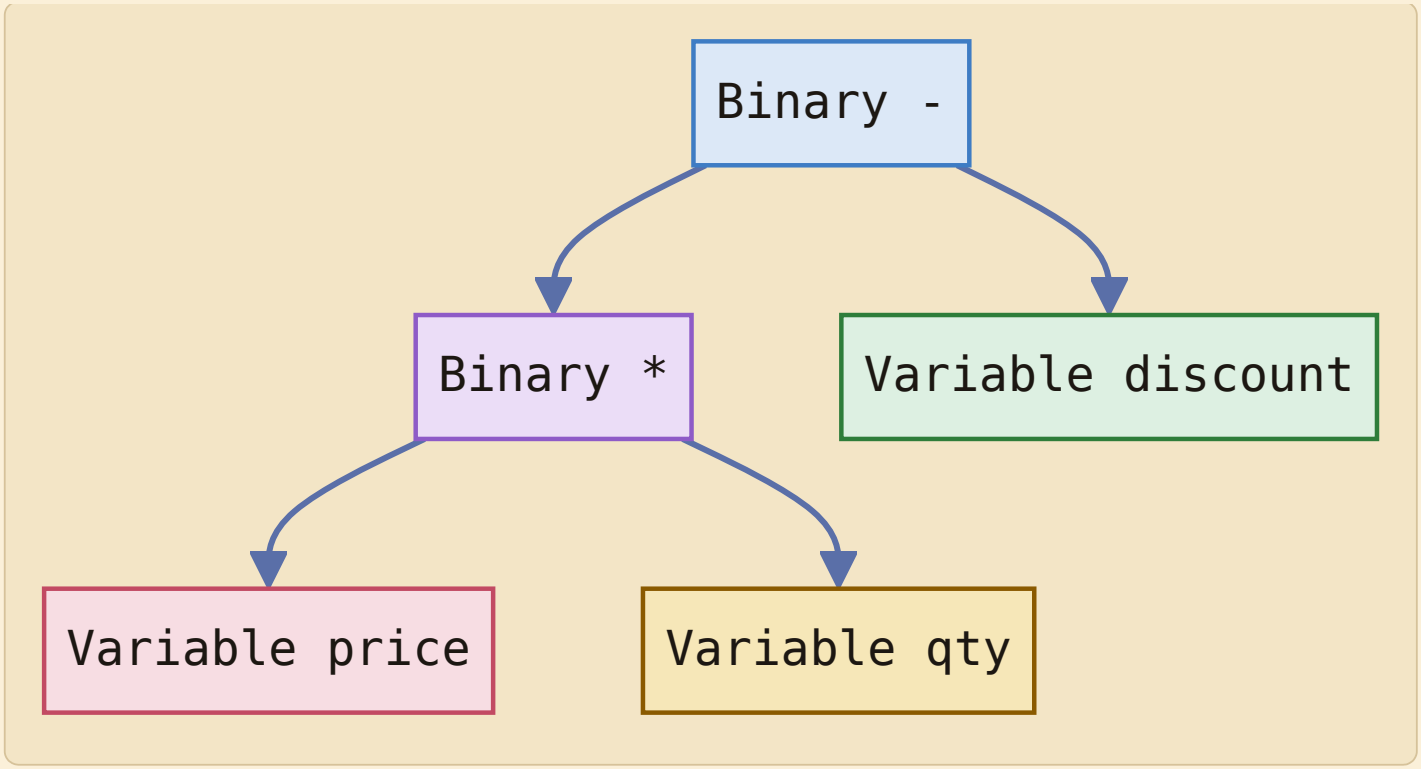
```
expr   := term   (('+' | '-') term)*
term   := factor (('*' | '/') factor)*
factor := NUMBER | NAME | '(' expr ')' | '-' factor
```

2. **One class per rule**: `Constant` and `Variable` (terminals), `Negate` and `Binary` (nonterminals), all implementing `Expression.interpret(context)`.
3. **Build the AST.** GoF deliberately leaves **parsing** (reading text and building a structure such as the AST from it) out of the pattern; any parser will do. We use a small *recursive-descent* parser: one method per grammar rule, which calls the others.
4. **Interpret**: the root calls `interpret` on its children and combines their results (a *recursive* evaluation).
5. **Reuse the tree**: parse once, interpret many times with different contexts.

Structure (UML)



The AST for `price * qty - discount`:



Notice the structure is a **Composite** (Chapter 9): leaves and inner nodes share one interface. Interpreter is essentially "Composite + an `interpret` operation".

Java example

`lld.ch10.interpreter`: `Expression` is a **sealed interface** (only the four listed records may implement it), each record is one grammar rule, and `Parser` is a recursive-descent parser whose methods `expr`, `term` and `factor` mirror the grammar lines. Errors (unknown variable, division by zero, bad syntax) are reported with clear messages.

> **Why `double` here?** This is a general-purpose arithmetic calculator used to teach the pattern, and variable names like `price` are only sample data. A real pricing or promotion DSL must evaluate money with `BigDecimal` (and an explicit `RoundingMode` for division), exactly as the domain code elsewhere in this chapter does — `double` cannot represent values like 0.10 exactly.

```java
// File: code/lld/ch10/interpreter/Expression.java
package lld.ch10.interpreter;

import java.util.Map;

/**
 * AbstractExpression. One class per grammar rule; each knows how to interpret itself.
 * Grammar (EBNF):
 *   expr   := term   (('+' | '-') term)*
 *   term   := factor (('*' | '/') factor)*
 *   factor := NUMBER | NAME | '(' expr ')' | '-' factor
 */
public sealed interface Expression
        permits Expression.Constant, Expression.Variable, Expression.Negate, Expression.Binary {

    /** Evaluate against a context (variable name -> value). */
    double interpret(Map<String, Double> context);

    /** Print the tree back as fully-parenthesised text. */
    String show();

    /** TerminalExpression: a literal number. */
    record Constant(double value) implements Expression {
        public double interpret(Map<String, Double> ctx) { return value; }
        public String show() { return value == Math.rint(value) ? String.valueOf((long) value) :
String.valueOf(value); }
    }

    /** TerminalExpression: a variable looked up in the context. */
    record Variable(String name) implements Expression {
        public double interpret(Map<String, Double> ctx) {
            Double v = ctx.get(name);
            if (v == null) throw new IllegalArgumentException("unknown variable '" + name + "'");
            return v;
        }
        public String show() { return name; }
    }

    /** NonterminalExpression: unary minus. */
    record Negate(Expression operand) implements Expression {
        public double interpret(Map<String, Double> ctx) { return -operand.interpret(ctx); }
        public String show() { return "(-" + operand.show() + ")"; }
    }

    /** NonterminalExpression: + - * / . Interprets its children, then combines. */
    record Binary(char op, Expression left, Expression right) implements Expression {
        public double interpret(Map<String, Double> ctx) {
            double l = left.interpret(ctx), r = right.interpret(ctx);
            return switch (op) {
                case '+' -> l + r;
                case '-' -> l - r;
                case '*' -> l * r;
                case '/' -> {
                    if (r == 0) throw new ArithmeticException("division by zero in " + show());
                    yield l / r;
                }
                default -> throw new IllegalStateException("unknown operator " + op);
            };
        }
        public String show() { return "(" + left.show() + " " + op + " " + right.show() + ")"; }
    }
}
```

```java
// File: code/lld/ch10/interpreter/Parser.java
package lld.ch10.interpreter;

/**
 * Recursive-descent parser: turns text into an Abstract Syntax Tree (AST).
 * Parsing is NOT part of the GoF Interpreter pattern; the pattern starts once the tree exists.
 * One method per grammar rule, mirroring the grammar in Expression.
 */
public final class Parser {
    private final String src;
    private int pos = 0;

    private Parser(String src) { this.src = src; }

    public static Expression parse(String text) {
        Parser p = new Parser(text);
        Expression e = p.expr();
        p.skipSpaces();
        if (p.pos != p.src.length()) throw p.error("unexpected '" + p.src.charAt(p.pos) + "'");
        return e;
    }

    private Expression expr() {
        Expression left = term();
        while (peek('+') || peek('-')) {
            char op = src.charAt(pos++);
            left = new Expression.Binary(op, left, term());   // left-associative
        }
        return left;
    }

    private Expression term() {
        Expression left = factor();
        while (peek('*') || peek('/')) {
            char op = src.charAt(pos++);
            left = new Expression.Binary(op, left, factor());
        }
        return left;
    }

    private Expression factor() {
        skipSpaces();
        if (pos >= src.length()) throw error("unexpected end of input");
        char c = src.charAt(pos);
        if (c == '-') { pos++; return new Expression.Negate(factor()); }
        if (c == '(') {
            pos++;
            Expression inner = expr();
            if (!peek(')')) throw error("missing ')'");
            pos++;
            return inner;
        }
        int start = pos;
        if (Character.isDigit(c) || c == '.') {
            while (pos < src.length() && (Character.isDigit(src.charAt(pos)) || src.charAt(pos) == '.')) pos++;
            return new Expression.Constant(Double.parseDouble(src.substring(start, pos)));
        }
        if (Character.isLetter(c)) {
            while (pos < src.length() && Character.isLetterOrDigit(src.charAt(pos))) pos++;
            return new Expression.Variable(src.substring(start, pos));
        }
        throw error("unexpected '" + c + "'");
    }

    private boolean peek(char expected) {
        skipSpaces();
        return pos < src.length() && src.charAt(pos) == expected;
    }
```

```
    private void skipSpaces() {
        while (pos < src.length() && Character.isWhitespace(src.charAt(pos))) pos++;
    }

    private IllegalArgumentException error(String msg) {
        return new IllegalArgumentException(msg + " at position " + pos + " in \"" + src + "\"");
    }
}
```

```
// File: code/lld/ch10/interpreter/Main.java
package lld.ch10.interpreter;

import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        Map<String, Double> context = Map.of("price", 250.0, "qty", 3.0, "discount", 50.0, "a", 7.0, "b", 3.0);
        System.out.println("context: " + new java.util.TreeMap<>(context));

        List<String> inputs = List.of(
            "price * qty - discount",
            "2 + 3 * 4",
            "(2 + 3) * 4",
            "(a + b) * (a - b)",
            "-a + 10 / 4",
            "price / (qty - 3)",
            "price * tax",
            "2 + * 3");

        for (String text : inputs) {
            try {
                Expression ast = Parser.parse(text);          // build the tree once...
                double value = ast.interpret(context);        // ...interpret it (could be many times)
                System.out.printf("  %-24s AST %-32s = %s%n", text, ast.show(), value);
            } catch (IllegalArgumentException | ArithmeticException e) {
                System.out.printf("  %-24s ERROR: %s%n", text, e.getMessage());
            }
        }

        System.out.println("\nSame tree, different contexts (parse once, evaluate many):");
        Expression total = Parser.parse("price * qty - discount");
        for (double q : new double[] {1, 2, 4}) {
            System.out.println("  qty=" + (int) q + " -> " + total.interpret(Map.of("price", 250.0, "qty", q,
"discount", 50.0)));
        }
    }
}
```

Output:

```
context: {a=7.0, b=3.0, discount=50.0, price=250.0, qty=3.0}
  price * qty - discount   AST ((price * qty) - discount)       = 700.0
  2 + 3 * 4                AST (2 + (3 * 4))                    = 14.0
  (2 + 3) * 4              AST ((2 + 3) * 4)                    = 20.0
  (a + b) * (a - b)        AST ((a + b) * (a - b))              = 40.0
  -a + 10 / 4              AST ((-a) + (10 / 4))                = -4.5
  price / (qty - 3)        ERROR: division by zero in (price / (qty - 3))
  price * tax              ERROR: unknown variable 'tax'
  2 + * 3                  ERROR: unexpected '*' at position 4 in "2 + * 3"

Same tree, different contexts (parse once, evaluate many):
  qty=1 -> 200.0
  qty=2 -> 450.0
  qty=4 -> 950.0
```

Notice operator precedence (which operator binds first) (`2 + 3 * 4` = 14, but `(2 + 3) * 4` = 20) comes from the **grammar**: `term` handles `*` and `/` below `expr`, so they bind tighter.

Trade-offs

Pros	Cons
Easy to change and extend the grammar: a new rule is a new class.	One class per rule: large grammars become a jungle of classes.
Each rule's evaluation is small and testable.	Tree-walking is slow compared with compiling to bytecode (compiled JVM instructions).
Rules become data (stored in a DB, edited by users).	Adding a new *operation* (pretty-print, type-check, optimise) means touching every class — that is where **Visitor** (Chapter 11) comes in.
Parse once, evaluate many times.	You still need a parser, which the pattern does not give you.

When not to use it

- **The grammar is large or complex** (a real programming language, SQL). Use a parser generator (a tool that writes a parser from a grammar file) such as **ANTLR** and a compiler/visitor design.
- **Performance matters** and expressions run millions of times — compile to bytecode or use an existing engine.
- **An existing language fits.** Regular expressions (patterns for matching text), SpEL (Spring's expression language), JSONPath (queries into JSON data), CEL (Google's Common Expression Language), or a scripting engine (an embedded interpreter for a full language such as JavaScript) may already do the job.
- **Security matters.** Never let users run a *general-purpose* script engine on your server; a small, restricted DSL is safer precisely because it can do only what its grammar allows.

Common mistakes

1. **Mixing parsing and evaluation** in one pass, so each evaluation re-parses the text.
2. **Ignoring precedence and associativity** (associativity decides how operators of equal precedence group: `8 - 3 - 2` must be `(8 - 3) - 2`; our `expr` loop builds left-associative trees).
3. **Poor error messages** — always report the position.
4. **Adding many operations to every node** instead of switching to Visitor or pattern matching (a `switch` that branches on an object's type and unpacks its fields).
5. **Growing a "tiny" DSL** until it is an accidental programming language.

Real-world examples

- `java.util.regex.Pattern`: compiling a regex builds an internal tree of node objects, and a `Matcher` walks it — a classic interpreter.
- **Spring Expression Language (SpEL)**: `SpelExpressionParser` parses text like `#user.age > 18` into an AST of `SpelNode`s, and `getValue()` evaluates it against a context. Jakarta Expression Language (EL) is similar.
- **SQL engines** evaluate `WHERE` clauses as expression trees; **spreadsheet formulas** (`=SUM(A1:A3)*2`) are a DSL.
- **Rule and pricing engines**: e-commerce promotions ("10% off if cart > 999 and category = electronics"), feature-flag targeting rules, and Google's **CEL** (Common Expression Language) used in Kubernetes validation rules and IAM (identity and access management — who may do what) conditions.
- **Cron expressions** (`0 9 * * MON-FRI`), e.g. Quartz's (a Java job-scheduling library) `CronExpression`.

In the interview

> **30-second answer:** "Interpreter models a small language's grammar as classes, one per rule — terminals like numbers and variables, nonterminals like addition. A sentence is parsed into an AST made of those objects and evaluated by calling `interpret` on the root with a context. It's really Composite plus an evaluate operation. It's great for small DSLs like pricing rules or filters; for big grammars I'd use a parser generator like ANTLR, and when I need many operations over the tree I'd switch to Visitor or sealed types with pattern matching."

Follow-ups:

- *Is parsing part of the pattern?* No. GoF assumes the tree already exists; parsing is a separate concern (hand-written recursive descent, or ANTLR).
- *Interpreter vs Visitor?* Interpreter puts evaluation **inside** the node classes. Visitor takes operations **out** into separate classes. A real interpreter usually starts as the first and becomes the second when a second pass (type-check, optimise, pretty-print) arrives.
- *Interpreter vs Composite?* Composite is the structure; Interpreter uses that structure to evaluate a language.
- *How would you make it faster?* Cache parsed trees; constant-fold (pre-compute parts with no variables); or compile to bytecode/lambdas once.
- *How do you design a rule engine for promotions?* A small grammar (comparisons, AND/OR/NOT, fields from the cart), rules stored as text, parsed once into trees and cached, evaluated per cart.

Self-check

1. **Q:** What is the difference between terminal and nonterminal expressions? **A:** Terminals are leaves (numbers, variables); nonterminals combine other expressions (operators).
2. **Q:** What is the "context"? **A:** External information needed for evaluation, such as variable values.
3. **Q:** Why does `2 + 3 * 4` give 14? **A:** The grammar puts `*` in `term`, below `expr`, so it binds tighter.
4. **Q:** Name a JDK class that is an interpreter. **A:** `java.util.regex.Pattern`.
5. **Q:** When should you not use it? **A:** Large grammars, performance-critical evaluation, or when an existing language fits.

TL;DR

One class per grammar rule; parse a sentence into an **AST**; evaluate by calling `interpret` recursively with a **context**. Perfect for **tiny DSLs**; for big grammars use a parser generator and Visitor.

10.7 Chapter recap: the six patterns side by side

Pattern	The object you create	The question it answers	Classic JDK / real example	Often confused with
Chain of Responsibility	A handler per check	*Who* should handle this request?	Servlet filters, logging handlers	Decorator (all layers run)
Command	An object per request	How do I store, queue or undo *this request*?	`Runnable`, `UndoManager`	Strategy (how vs what)
Iterator	A cursor object	How do I walk this without knowing its shape?	`Iterator`, `Spliterator`, `ResultSet`	Visitor (what to do at each node)
Mediator	One coordinator	How do many objects cooperate without knowing each other?	UI controllers, chat servers, saga orchestrators	Observer (broadcast only), Facade
Memento	A sealed snapshot	How do I go back to an earlier state safely?	DB rollback, Git, `kubectl rollout undo`	Command undo, Prototype
Interpreter	A node per grammar rule	How do I evaluate sentences of a small language?	`Pattern`, SpEL	Composite, Visitor

Interview combo facts to remember:

- The request flowing through a **Chain of Responsibility** is frequently a **Command** object.
- A **Mediator** often creates and dispatches **Commands**.
- **Memento + Command** together give scalable undo (snapshots + replay).
- **Iterator + Composite** walk trees; **Interpreter = Composite + evaluate**; when operations multiply, move to **Visitor** (Chapter 11).

Self-check (whole chapter)

1. **Q:** Which two GoF behavioral patterns are class-scoped (inheritance-based)? **A:** Template Method and Interpreter.
2. **Q:** A request must pass auth, then a rate limit, then validation, and any step can reject it. Which pattern? **A:** Chain of Responsibility.

3. **Q:** You need undo/redo for a text editor. Which patterns? **A:** Command (per edit), optionally Memento for periodic snapshots.
4. **Q:** Five form widgets depend on each other's values. Which pattern? **A:** Mediator.
5. **Q:** Your team needs business users to write discount rules. Which pattern? **A:** Interpreter (a small DSL), possibly with Specification (Chapter 11).

> **TL;DR**

Behavioral patterns turn **hidden control flow** into **named objects**: handlers (CoR), requests (Command), cursors (Iterator), coordinators (Mediator), snapshots (Memento) and grammar nodes (Interpreter). Each one buys flexibility with indirection — use it when the flexibility is a real requirement, not "just in case".

Chapter 11 — Behavioral Design Patterns (Part 2) & Beyond GoF

> **What you will learn.** The remaining five **GoF** ("Gang of Four" — the four authors of the 1994 book *Design Patterns*, which catalogued 23 classic patterns) behavioral patterns — **Observer, State, Strategy, Template Method, Visitor** — including the three that share one class diagram (Strategy, State, Bridge) and how to tell them apart. Then six widely used patterns that are *not* in the GoF book but come up constantly in interviews and real code: **Null Object, Dependency Injection, Repository, Specification, MVC** (the Head First "compound pattern") and **Event Sourcing (lite)**. The chapter ends with a "which behavioral pattern do I need?" decision table.

How to run the code in this chapter

```
cd chapters/11-patterns-behavioral-2/code
javac --release 21 -Xlint:all -d /tmp/lld_11 $(find . -name '*.java')   # compiles with 0 warnings
java -cp /tmp/lld_11 lld.ch11.observer.Main
# also: state, strategy, templatemethod, visitor, nullobject, di, repository, specification, mvc, eventsourcing
```

Chapter 10 introduced behavioral patterns: patterns about **how objects share work and talk to each other**. Keep one idea in mind for this chapter: most of these patterns take a conditional (`if` / `switch`) or a hard-coded call and turn it into an **object** that can be swapped, subscribed, or visited.

11.1 Observer

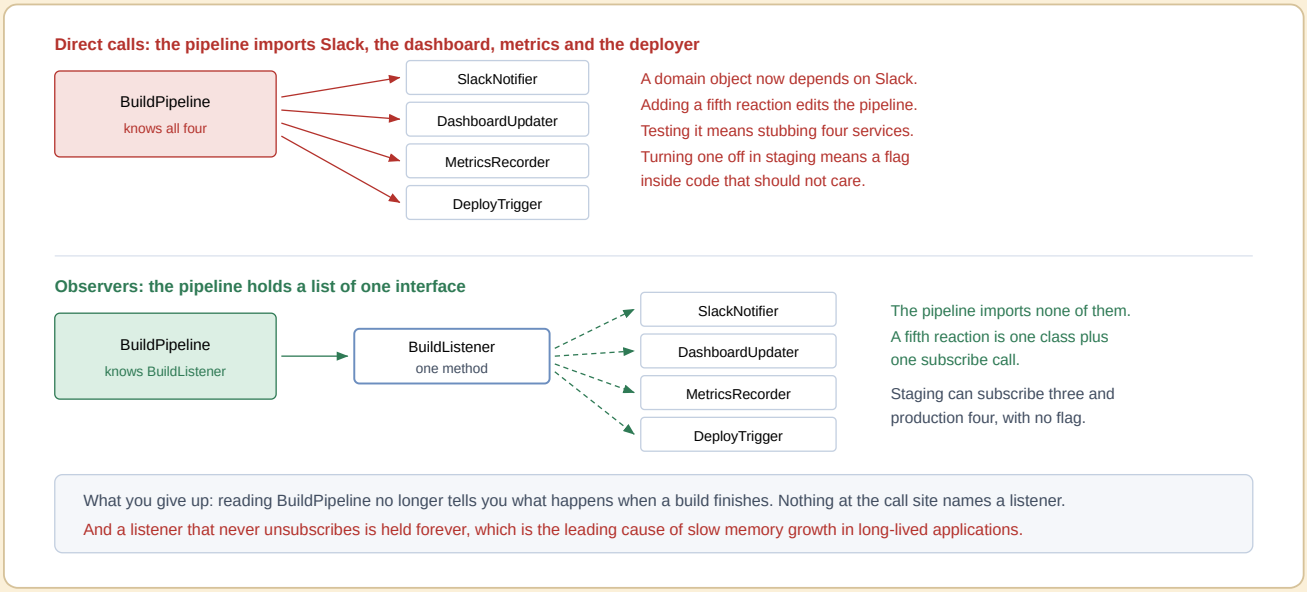


Figure — Top: the build pipeline calls Slack, dashboard, metrics and deployer directly. Bottom: it holds one list of BuildListener and imports none of them (course diagram)

What it is

Observer lets one object (the **subject**, also called *publisher* or *observable*) notify many dependent objects (the **observers**, *subscribers* or *listeners*) when something happens — **without knowing who they are or what they do**.

Everyday analogy: a YouTube channel. You subscribe; when a new video is published, every subscriber is notified. The channel does not know or care what each subscriber does with the notification, and you can unsubscribe at any time.

GoF intent (paraphrased): set up a one-to-many dependency, so that when one object's state changes, every object that depends on it is told and can update itself automatically.

GoF participants (and who plays them in our code):

Participant	Role	In `lld.ch11.observer`
Subject	Knows its observers (any number) and offers attach/detach operations	`BuildPipeline` (`addListener` + the returned unsubscribe handle)
Observer	The update interface that subjects call	`BuildListener`
ConcreteSubject	Holds the state observers care about; notifies them when it changes	`BuildPipeline` (GoF split Subject into an abstract class and a concrete one; in modern Java one class usually plays both)
ConcreteObserver	Implements the update interface; may keep its own state in step with the subject	`SlackNotifier`, `DashboardUpdater`, `MetricsRecorder`, `DeployTrigger`

The problem it solves

Suppose four reactions must follow every finished build: a Slack message, a dashboard refresh, a metrics record and a deployment. The naive version calls all four itself:

```
void finish(int buildNumber, String result) {
    slack.post(buildNumber, result);
    dashboard.update(result);
    metrics.record(buildNumber);
    deployer.start();
}
```

This costs more than it looks: the build pipeline now **depends on a chat service**; a fifth reaction means editing the pipeline; a **unit test** (an automated test that checks one class on its own) must fake four services; and disabling deployment in **staging** (the pre-production copy of the system used for final checks) needs an environment flag inside a class that ought to know nothing about environments.

Observer replaces the four references with **one list of one interface**. Anything that cares subscribes. **The source only announces; every listener chooses its own reaction.** No one orchestrates them, and listeners are unaware of one another.

How it works

1. Define an **observer interface** with one method, e.g. `onBuildFinished(buildNumber, result)`.
2. The **subject** keeps a list of observers and offers subscribe and unsubscribe operations (classically `addListener` / `removeListener`; our `addListener` instead returns a handle that unsubscribes).
3. When its state changes, the subject **first finishes changing its state**, then loops over the list and calls each observer.
4. Observers can be added and removed at run time (e.g. staging subscribes three listeners, production four).

Push vs pull — two ways to deliver the data:

	Push model	Pull model
What the observer receives	All the data about the change (e.g. a `Reading` record)	Just "I changed" plus a reference to the subject
How the observer gets data	It is in the call	It calls getters on the subject for what it needs
Pros	Simple; observers don't depend on the subject's getters	Each observer takes only what it needs; the interface doesn't change when new data is added
Cons	Sends data some observers don't need; adding data changes the interface	Observers depend on the subject; they must work out *what* changed

`java.util.Observer.update(Observable o, Object arg)` supported both: `o` for pull, `arg` for push. *Head First Design Patterns* shows the same trade-off with its weather-station example.

Structure (UML)

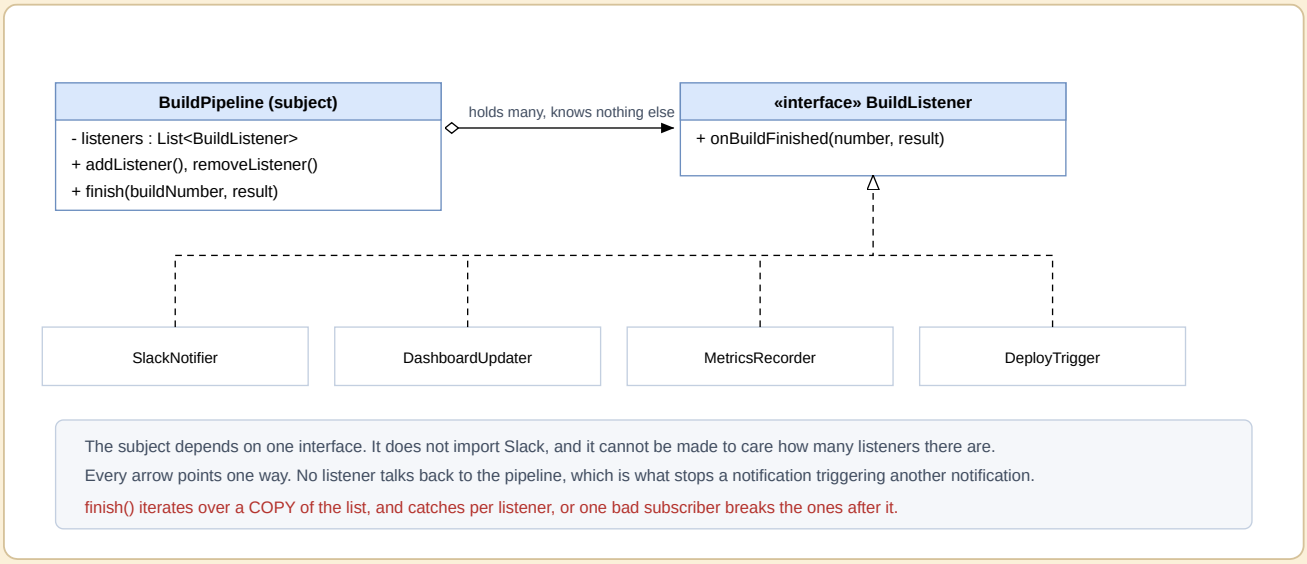
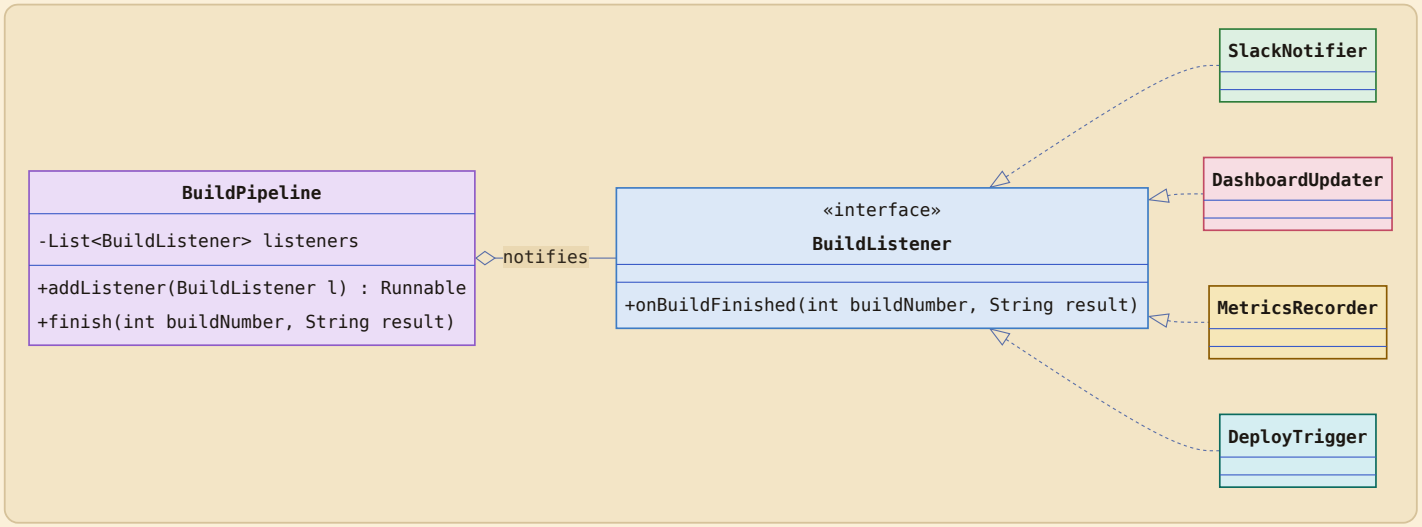
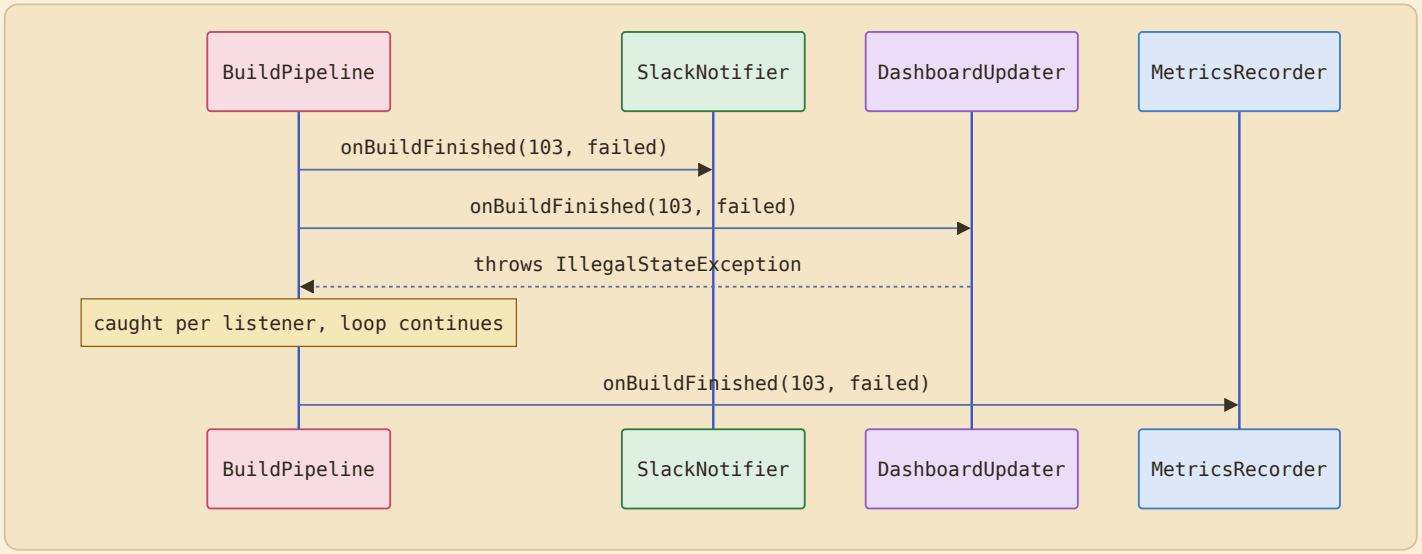


Figure — Class diagram: BuildPipeline holds a list of BuildListener; SlackNotifier, DashboardUpdater, MetricsRecorder and DeployTrigger implement it (course diagram)

In the diagram above (a **UML** class diagram — UML, the Unified Modeling Language, is the standard box-and-arrow notation for classes and how they relate) all dependencies run in a single direction. Listeners never call the pipeline back, so one notification cannot set off another.

Our Java design:





Java example

`lld.ch11.observer` shows four things:

1. **Build pipeline.** Listeners live in a `CopyOnWriteArrayList` (a JDK list that copies its internal array on every write, so readers never see it change under them), whose iterator works on a **snapshot** — so a listener that unsubscribes during notification cannot break the loop, and the list is **thread-safe** (safe to use from several *threads* — independent paths of execution running at the same time — at once). `finish` **catches exceptions per listener**, so a broken dashboard does not stop metrics. `addListener` returns a `Runnable` (Java's built-in interface for "a piece of code to run later", with one method `run()`) that unsubscribes: whoever subscribes owns the matching unsubscribe.
2. **Push vs pull** (`WeatherStation`): one display receives the whole `Reading`; the other receives the station and pulls only the temperature.
3. **A tiny event bus** (`EventBus`): subscribers register for an event *type* (a class); publishers just publish objects. The returned `Subscription` is the unsubscribe handle. `Main` closes it in a `finally` block, so the refunds handler is removed even if publishing throws. (`Subscription` is also `AutoCloseable` — an interface whose `close()` method is called automatically when a *try-with-resources* block ends — so `try (var s = bus.subscribe(...)) { ... }` works too. But if the body never uses `s`, `javac -Xlint` warns that the resource is never referenced. Java 22+ lets you write `try (var _ = ...)` to say "unused on purpose". On Java 21, `try`/`finally` is the warning-free form.) The bus's handler list is a `CopyOnWriteArrayList`, so subscribing and publishing from different threads is safe. Delivery is still **synchronous on the publisher's thread** (the publisher waits while each handler runs, one after another) and nothing is stored, so it is not a **queue** (a waiting line that stores items and hands them out later, in order).
4. `java.beans.PropertyChangeSupport` (the price is whole cents in an `int` — exact, never `double`), which the `Observable` **Javadoc** (the official API documentation generated from comments in Java source code) itself points to (the `java.beans` package) as the richer event model. Note it does **not** fire when the old and new values are both non-null and equal. (Since Java 9, `java.beans` lives in the `java.desktop` module; on the class path (the list of folders and JAR files where Java looks for classes) this needs nothing extra, but a modular app (one that declares its dependencies in a `module-info.java` file) must `requires java.desktop`.)

```java
// File: code/lld/ch11/observer/BuildListener.java
package lld.ch11.observer;

/** The OBSERVER interface: one method. A lambda can implement it. */
@FunctionalInterface
public interface BuildListener {
    void onBuildFinished(int buildNumber, String result);
}
```

```java
// File: code/lld/ch11/observer/BuildPipeline.java
package lld.ch11.observer;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

/** The SUBJECT: holds a list of one interface and imports none of the concrete reactions. */
public final class BuildPipeline {
    // CopyOnWriteArrayList iterates over a snapshot: a listener that unsubscribes
    // itself mid-notification cannot break the loop, and it is safe across threads.
    private final List<BuildListener> listeners = new CopyOnWriteArrayList<>();

    /** Returns a handle: whoever subscribes owns the matching unsubscribe (prevents leaks). */
    public Runnable addListener(BuildListener listener) {
        listeners.add(listener);
        return () -> listeners.remove(listener);
    }

    public void finish(int buildNumber, String result) {
        System.out.println("Build " + buildNumber + " finished: " + result);
        for (BuildListener l : listeners) {
            try {
                l.onBuildFinished(buildNumber, result);
            } catch (RuntimeException e) {          // one bad listener must not starve the rest
                System.out.println("  [pipeline] a listener failed: " + e.getMessage());
            }
        }
    }
}
```

```java
// File: code/lld/ch11/observer/SlackNotifier.java
package lld.ch11.observer;

final class SlackNotifier implements BuildListener {
    @Override
    public void onBuildFinished(int n, String result) {
        System.out.println("  [slack] #builds: build " + n + " " + result);
    }
}
```

```java
// File: code/lld/ch11/observer/DashboardUpdater.java
package lld.ch11.observer;

final class DashboardUpdater implements BuildListener {
    boolean offline = false;

    @Override
    public void onBuildFinished(int n, String result) {
        if (offline) throw new IllegalStateException("dashboard is offline");
        System.out.println("  [dashboard] status tile -> " + result);
    }
}
```

```java
// File: code/lld/ch11/observer/MetricsRecorder.java
package lld.ch11.observer;

final class MetricsRecorder implements BuildListener {
    private int recorded = 0;

    @Override
    public void onBuildFinished(int n, String result) {
        recorded++;
        System.out.println("  [metrics] recorded build " + n + " (total " + recorded + ")");
    }
}
```

```java
// File: code/lld/ch11/observer/DeployTrigger.java
package lld.ch11.observer;

final class DeployTrigger implements BuildListener {
    @Override
    public void onBuildFinished(int n, String result) {
        if (result.equals("success")) System.out.println("  [deploy] deploying build " + n);
    }
}
```

```java
// File: code/lld/ch11/observer/WeatherStation.java
package lld.ch11.observer;

import java.util.ArrayList;
import java.util.List;
import java.util.function.Consumer;

/** A subject that supports both notification styles, to compare PUSH and PULL. */
public final class WeatherStation {
    /** PUSH: the subject sends the data with the notification. */
    public record Reading(double tempC, double humidity, double pressureHpa) {}

    private final List<Consumer<Reading>> pushObservers = new ArrayList<>();
    private final List<Consumer<WeatherStation>> pullObservers = new ArrayList<>();
    private Reading current = new Reading(0, 0, 0);

    public void subscribePush(Consumer<Reading> o) { pushObservers.add(o); }
    public void subscribePull(Consumer<WeatherStation> o) { pullObservers.add(o); }

    // PULL: observers get only "I changed" plus a reference, then ask for what they need.
    public double temperature() { return current.tempC(); }
    public double humidity() { return current.humidity(); }

    public void measure(Reading r) {
        current = r;                                          // change state FIRST...
        pushObservers.forEach(o -> o.accept(r));              // ...then notify
        pullObservers.forEach(o -> o.accept(this));
    }
}
```

```java
// File: code/lld/ch11/observer/EventBus.java
package lld.ch11.observer;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.function.Consumer;

/**
 * A tiny in-process event bus: publishers and subscribers share only event TYPES,
 * never references to each other. Synchronous, not durable (unlike Kafka).
 */
public final class EventBus {
    private record Handler<E>(Class<E> type, Consumer<? super E> action) {
        void deliver(Object event) {
            if (type.isInstance(event)) action.accept(type.cast(event));
        }
    }

    /** AutoCloseable so try-with-resources can unsubscribe for you. */
    public interface Subscription extends AutoCloseable {
        @Override void close();
    }

    private final List<Handler<?>> handlers = new CopyOnWriteArrayList<>();

    public <E> Subscription subscribe(Class<E> type, Consumer<? super E> action) {
        Handler<E> h = new Handler<>(type, action);
        handlers.add(h);
        return () -> handlers.remove(h);
    }

    public void publish(Object event) {
        handlers.forEach(h -> h.deliver(event));
    }
}
```

```java
// File: code/lld/ch11/observer/PricedItem.java
package lld.ch11.observer;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;

/** The JDK's recommended replacement for java.util.Observable: java.beans.PropertyChangeSupport. */
public final class PricedItem {
    private final PropertyChangeSupport support = new PropertyChangeSupport(this);
    private final String name;
    private int priceCents;

    public PricedItem(String name, int priceCents) {
        this.name = name;
        this.priceCents = priceCents;
    }

    public void addPropertyChangeListener(PropertyChangeListener l) { support.addPropertyChangeListener(l); }
    public void removePropertyChangeListener(PropertyChangeListener l) { support.removePropertyChangeListener(l); }

    public void setPriceCents(int newPrice) {
        int old = this.priceCents;
        this.priceCents = newPrice;
        support.firePropertyChange("priceCents", old, newPrice);   // no event if old == new
    }

    public String name() { return name; }
}
```

```java
// File: code/lld/ch11/observer/Main.java
package lld.ch11.observer;

import java.math.BigDecimal;

public class Main {
    record OrderPlaced(String orderId, BigDecimal amount) {}
    record OrderCancelled(String orderId) {}

    public static void main(String[] args) {
        System.out.println("=== 1. Build pipeline: subscribe, unsubscribe, failing listener ===");
        BuildPipeline pipeline = new BuildPipeline();
        DashboardUpdater dashboard = new DashboardUpdater();
        pipeline.addListener(new SlackNotifier());
        pipeline.addListener(dashboard);
        pipeline.addListener(new MetricsRecorder());
        Runnable unsubscribeDeploy = pipeline.addListener(new DeployTrigger());

        pipeline.finish(101, "success");        // every listener runs
        unsubscribeDeploy.run();                // e.g. staging has no auto-deploy
        pipeline.finish(102, "success");        // three react
        dashboard.offline = true;
        pipeline.finish(103, "failed");         // dashboard throws, the others still run

        System.out.println("\n=== 2. Push vs pull ===");
        WeatherStation station = new WeatherStation();
        station.subscribePush(r -> System.out.println("  [push display] got everything: " + r));
        station.subscribePull(s -> System.out.println("  [pull display] asked only for temp: " + s.temperature() +
" C"));
        station.measure(new WeatherStation.Reading(31.5, 0.62, 1008));

        System.out.println("\n=== 3. Event bus (typed topics, scoped unsubscribe) ===");
        EventBus bus = new EventBus();
        bus.subscribe(OrderPlaced.class, e -> System.out.println("  [email] confirmation for " + e.orderId()));
        bus.subscribe(Object.class, e -> System.out.println("  [audit] " + e));
        EventBus.Subscription refunds = bus.subscribe(OrderCancelled.class,
                e -> System.out.println("  [refunds] start refund for " + e.orderId()));
        try {
            bus.publish(new OrderPlaced("A-1", new BigDecimal("499.00")));
            bus.publish(new OrderCancelled("A-1"));
        } finally {
            refunds.close();                                    // refunds handler removed here, even on error
        }
        bus.publish(new OrderCancelled("A-2"));                 // only audit sees this one

        System.out.println("\n=== 4. JDK: PropertyChangeSupport (replacement for deprecated Observable) ===");
        PricedItem phone = new PricedItem("phone", 49_900);
        phone.addPropertyChangeListener(evt -> System.out.println(
            "  [price watcher] " + phone.name() + " " + evt.getPropertyName() + ": "
            + evt.getOldValue() + " -> " + evt.getNewValue()));
        phone.setPriceCents(44_900);
        phone.setPriceCents(44_900);    // same value: no event fired
        phone.setPriceCents(39_900);
    }
}
```

Output:

```
=== 1. Build pipeline: subscribe, unsubscribe, failing listener ===
Build 101 finished: success
  [slack] #builds: build 101 success
  [dashboard] status tile -> success
  [metrics] recorded build 101 (total 1)
  [deploy] deploying build 101
Build 102 finished: success
  [slack] #builds: build 102 success
  [dashboard] status tile -> success
  [metrics] recorded build 102 (total 2)
Build 103 finished: failed
  [slack] #builds: build 103 failed
  [pipeline] a listener failed: dashboard is offline
  [metrics] recorded build 103 (total 3)

=== 2. Push vs pull ===
  [push display] got everything: Reading[tempC=31.5, humidity=0.62, pressureHpa=1008.0]
  [pull display] asked only for temp: 31.5 C

=== 3. Event bus (typed topics, scoped unsubscribe) ===
  [email] confirmation for A-1
  [audit] OrderPlaced[orderId=A-1, amount=499.00]
  [audit] OrderCancelled[orderId=A-1]
  [refunds] start refund for A-1
  [audit] OrderCancelled[orderId=A-2]

=== 4. JDK: PropertyChangeSupport (replacement for deprecated Observable) ===
  [price watcher] phone priceCents: 49900 -> 44900
  [price watcher] phone priceCents: 44900 -> 39900
```

Why `java.util.Observer` / `Observable` were deprecated

Since **Java 9**, `java.util.Observer` and `java.util.Observable` are **deprecated** (officially marked "don't use in new code; may be removed one day") but not removed. The official reasons (Javadoc): the event model is **quite limited**, the **order of notifications is unspecified**, and **state changes are not one-for-one with notifications**. Other well-known problems:

- `Observable` is a **class**, not an interface — your subject must *extend* it, using up Java's **single inheritance** (a Java class can extend only one parent class).
- `setChanged()` is **protected**, so you cannot use `Observable` by composition (holding one as a field) without subclassing.
- The `update(Observable, Object)` signature is untyped (`Object arg`), so observers must cast.
- Its thread safety is only partial: its methods are `synchronized` (only one thread at a time may run them, because each call first takes the object's **lock** — a token only one thread can hold), but `notifyObservers` calls observers outside the lock on a snapshot, so (as a comment in the **JDK** — Java Development Kit: the compiler, runtime and standard library — source admits) a newly added observer may miss a notification and a just-removed one may still get it.

What to use instead (the Javadoc names the first, third and fourth): `java.beans.PropertyChangeSupport`/`PropertyChangeListener` for bean properties; your own typed listener interface (as in `BuildPipeline`); `java.util.concurrent` data structures (queues) for reliable, ordered hand-off between threads; the `java.util.concurrent.Flow` API (Java 9) for **reactive streams** (asynchronous streams of data items flowing from a producer to consumers) with **back-pressure** (the consumer tells the producer to slow down when it cannot keep up).

Trade-offs

Pros	Cons
The source no longer depends on its reactions (a build pipeline no longer imports Slack).	Harder to trace: the call site gives no hint of which code will execute.
Reactions are added/removed at run time; environments differ without flags.	**Memory leaks** (memory that is never freed) if an observer is never removed (the subject holds them forever).
Each reaction is testable alone.	Stack traces start inside a loop over a list; debugging is harder.
A new reaction = one class + one subscribe call (Open/Closed: open to extension, closed to modification).	**No ordering or delivery guarantee**; a crash loses the reactions not yet called.
	With synchronous observers, the subject takes as long as all its listeners combined.

Observer vs Mediator, Chain of Responsibility, callbacks

Looks like	The actual difference
Mediator	Observer carries only "this changed"; each observer decides for itself, one way, and no order between reactions can be expressed. A mediator receives the change **and decides what happens next**, calling specific components back (two-way), and explicit ordering is its reason to exist.
Publish-subscribe with a broker (a server that stores and forwards messages)	Same idea, but the broker contributes persistence, redelivery, ordering guarantees and a network hop — none of which in-process observers offer.
Chain of Responsibility	Observer informs **all** listeners, and no listener can prevent the rest from running. A chain ends as soon as one handler accepts the request.
A callback	One callback is one observer. It becomes the Observer pattern when the number of listeners varies and they subscribe and unsubscribe at run time.

Observer outside Java. With **first-class functions** (functions that can be stored in variables and passed around like any other value) a listener is just a **lambda** (a short anonymous function, written `x -> ...` in Java) — keep the handle, or you cannot unsubscribe. JavaScript has `EventTarget` / `EventEmitter`, and `AbortController` can unsubscribe by signal. Python often uses a plain list of callables, and `weakref.WeakSet` avoids the leak. C# has `event`, `IObservable<T>`, and `IDisposable` subscriptions. In every language, **weak references** (references that do not keep their target alive, so the garbage collector may still free it) fix the leak, at the cost of a listener disappearing when nothing else holds it.

When not to use it

- **One listener, forever.** Call it directly.
- **You need ordering or guaranteed delivery.** Use a queue or message broker (Kafka, RabbitMQ, SQS — popular broker products): same idea plus persistence, retries and a network — and very different failure modes.
- **Reactions are slow.** Either notify **asynchronously** (hand the work to another thread and return at once — e.g. via an *executor*, a pool of worker threads, or a queue) or don't use in-process observers.
- **The whole flow must be auditable/traceable.** Explicit calls are kinder to readers.

Common mistakes (the four classic ways observers go wrong, plus one)

1. **No unsubscribe → memory leak.** The subject keeps a **strong reference** (an ordinary reference, which keeps the object alive in memory) to the listener and everything it points to. In long-running apps this is a top cause of slow memory growth. Every `add` needs an owner for the matching `remove`. (Weak references are one fix, at the cost of listeners vanishing when nothing else holds them.)
2. **A throwing listener kills the loop.** Without a per-listener try/catch, listener 3 fails and 4 and 5 never run.
3. **Changing the list during notification** → `ConcurrentModificationException` (the error Java throws when a collection is changed while it is being looped over) (or silently skipped listeners in some languages). Iterate over a copy, or use `CopyOnWriteArrayList`.

4. **Listeners calling back into the subject** → cycles or endless recursion. Finish updating state before notifying, and never let a notification synchronously cause another one.
5. **Notifying before the state is updated**, so pull-style observers read stale data.
6. **Relying on notification order.** Listeners are called in the order they happened to subscribe, which is a side effect of start-up code rather than a decision anyone made. If "update the dashboard, then decide whether to deploy" matters, that is coordination, and it belongs in a Mediator.

Real-world examples

- **Every UI event handler**: Swing `ActionListener`, Android `OnClickListener`, DOM `addEventListener`.
- **JDK**: `PropertyChangeListener`, `java.util.concurrent.Flow` (`Publisher`/`Subscriber`), servlet listeners such as `HttpSessionListener`.
- **Spring** (the most widely used Java application framework): `ApplicationEventPublisher` + `@EventListener` (in-process events, optionally `@Async` or `@TransactionalEventListener`).
- **Guava** `EventBus`, **RxJava / Project Reactor / RxJS**, Kotlin `Flow`, and signals in front-end frameworks — observers with composable operators (map, filter, debounce — wait until events pause, then pass on only the latest) layered on top.
- **Model–view separation**: a view registers with its model, which then updates freely with no idea that any screen is attached (see MVC in §11.7.5).
- **Distributed publish–subscribe**: Kafka topics, Google Pub/Sub, AWS SNS. When an order is placed on an e-commerce site, the order service publishes one event and email, inventory, analytics and recommendation services each consume it independently — Observer at system scale, with durability added.
- **Database triggers and change data capture** (CDC: streaming every row change out of a database's internal log; Debezium is a popular tool) — observers at the storage layer.

In the interview

> **30-second answer:** "Observer swaps a handful of hard-coded calls for a list of objects that share one listener interface. The subject just says "this happened" and every observer picks its own response, so each reaction can be plugged in, unplugged and unit-tested on its own, and the subject no longer depends on any of them. In production I iterate over a copy of the listener list, catch exceptions per listener, and make sure every subscribe has a matching unsubscribe to avoid leaks. When I need durability, retries or ordering I use a message broker instead."

Follow-ups:

- *Push or pull?* Push sends the data (simple, but the interface grows with the data); pull sends a reference and observers ask for what they need (flexible, but couples observers to the subject's getters).
- *Why was `java.util.Observable` deprecated?* Limited event model, unspecified notification order, notifications not one-for-one with changes; plus it's a class (uses up inheritance) with protected `setChanged`. Use `PropertyChangeSupport`, custom listeners, or `Flow`.
- *Observer vs Pub/Sub?* In Observer the subject holds its observers directly. In publish–subscribe a **broker/event bus** sits in between, so publishers and subscribers don't know each other at all; brokers add durability, retries and ordering.
- *Observer vs Mediator?* With Observer, the news is broadcast and every listener makes its own call; with Mediator, one central object makes the decision and tells particular components what to do. When the sequence of reactions matters, pick a mediator.
- *How do you make it thread-safe?* `CopyOnWriteArrayList` for listeners (reads far outnumber writes), no locks held while calling listeners, and dispatch to an executor if listeners are slow.
- *How do you avoid leaks?* Return a subscription handle, tie it to a lifecycle (try-with-resources, `onDestroy`, `@PreDestroy`), or use weak references.

Self-check

1. **Q:** What does the subject know about its observers? **A:** Only that they implement the observer interface.
2. **Q:** Why iterate over a copy of the listener list? **A:** So a listener that (un)subscribes during notification can't break the loop.
3. **Q:** What is the classic Observer memory leak? **A:** Listeners that are never removed stay strongly referenced by the subject.

4. **Q:** What should replace `java.util.Observable`? **A:** `PropertyChangeSupport`, a custom typed listener interface, or the `Flow` API.
5. **Q:** When is a message broker better than in-process observers? **A:** When you need durability, retries, ordering, or slow/remote consumers.

TL;DR

One subject, a list of listeners, one-way notification. **Iterate over a copy, catch per listener, always unsubscribe.** Push vs pull is a trade-off; `Observable` is deprecated; use a broker when you need guarantees.

11.2 State

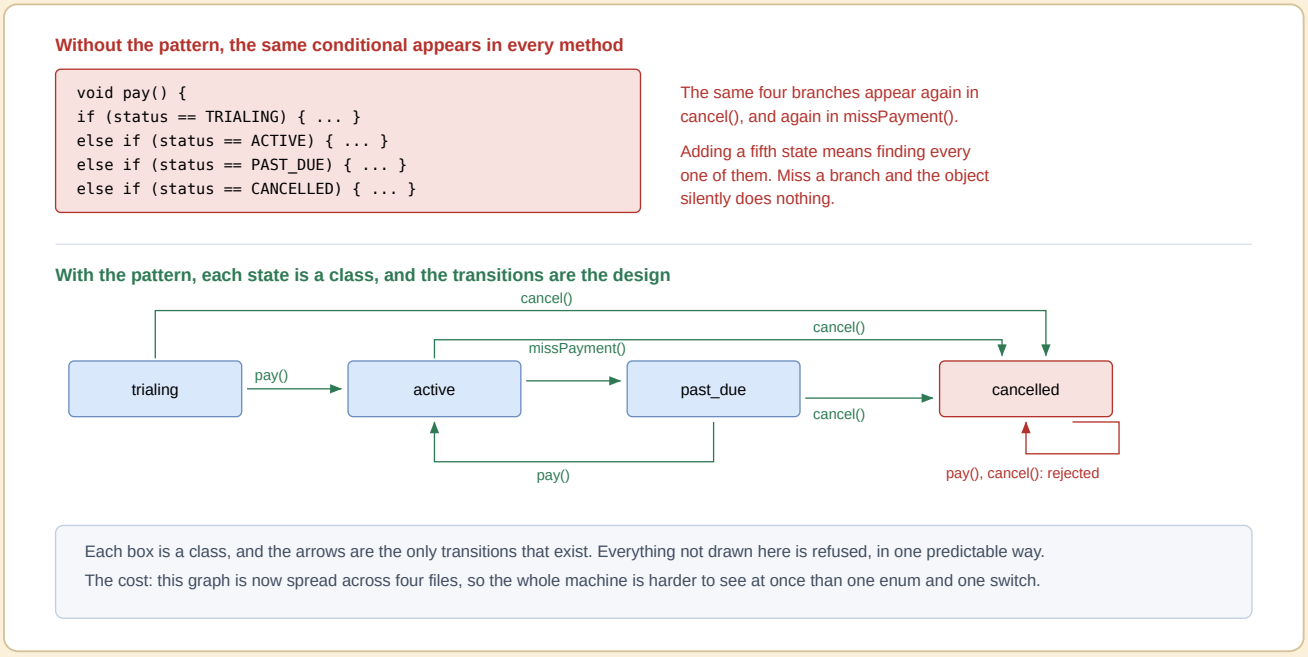


Figure — Top: the same four-branch status check repeated in every method. Bottom: each state is a class and the transitions form a graph (course diagram)

What it is

State (GoF intent, paraphrased): when an object's internal condition changes, **its behaviour changes with it**, so from the outside it looks as if the object had switched to a different class. Rather than every method testing a status field, the object hands each action to a **state object**, and replaces that object whenever the condition changes.

GoF participants (and who plays them in our code):

Participant	Role	In `lld.ch11.state`
Context	The object clients use; holds the current state object and forwards requests to it	`Subscription`, `VendingMachine`
State	Interface declaring one method per request the context forwards	`SubscriptionState`, `VendingState`
ConcreteState	One class per state, implementing the behaviour for that state	`Trialing`, `Active`, `PastDue`, `Cancelled`; `Idle`, `HasMoney`, `SoldOut`

GoF leaves open *who* performs the transitions: the context (with a fixed table) or the states themselves. Letting the states choose their successor, as here, is more flexible but makes the states know about each other.

Everyday analogy: a phone's ringer switch. Pressing "call received" does something different in *ring*, *vibrate* and *silent* mode — the phone is the same, its mode decides the behaviour.

The problem it solves

Take a subscription whose life runs **trialing → active → past due → cancelled**. Written naively, each public method opens with an identical four-way branch:

```
public void pay() {
    if (status == TRIALING)       { /* charge the card, switch to ACTIVE */ }
    else if (status == ACTIVE)    { /* renew */ }
    else if (status == PAST_DUE)  { /* settle arrears, back to ACTIVE */ }
    else if (status == CANCELLED) { /* reject */ }
}
```

`cancel()` and `missPayment()` repeat those four branches. A fifth state forces you to hunt down every copy. Everything that defines one state is spread over all the methods. Worst of all, a **forgotten branch** quietly does nothing, and the bug surfaces weeks later as a vague "subscriptions randomly stop working".

With the State pattern, **each state is a class**. The conditional vanishes (each context method shrinks to a single line), and everything about one state is gathered in one file — exactly what a reviewer or auditor wants to read.

How it works

1. Define a **State interface** with one method per action (`pay`, `missPayment`, `cancel`). Each state has to respond to each action, even when the response is "not allowed".
2. Write one **concrete state** class per state.
3. The **context** (`Subscription`) holds the current state and delegates each action: `state.pay(this)`.
4. A state rules on whether the action is **legal** and which state comes **after** it, and advances the machine by calling `context.setState(new Active())`.
5. Terminal states refuse everything, loudly, in one place.

The objects choose their own successors — the machine advances itself. That is the key difference from Strategy.

Structure (UML)

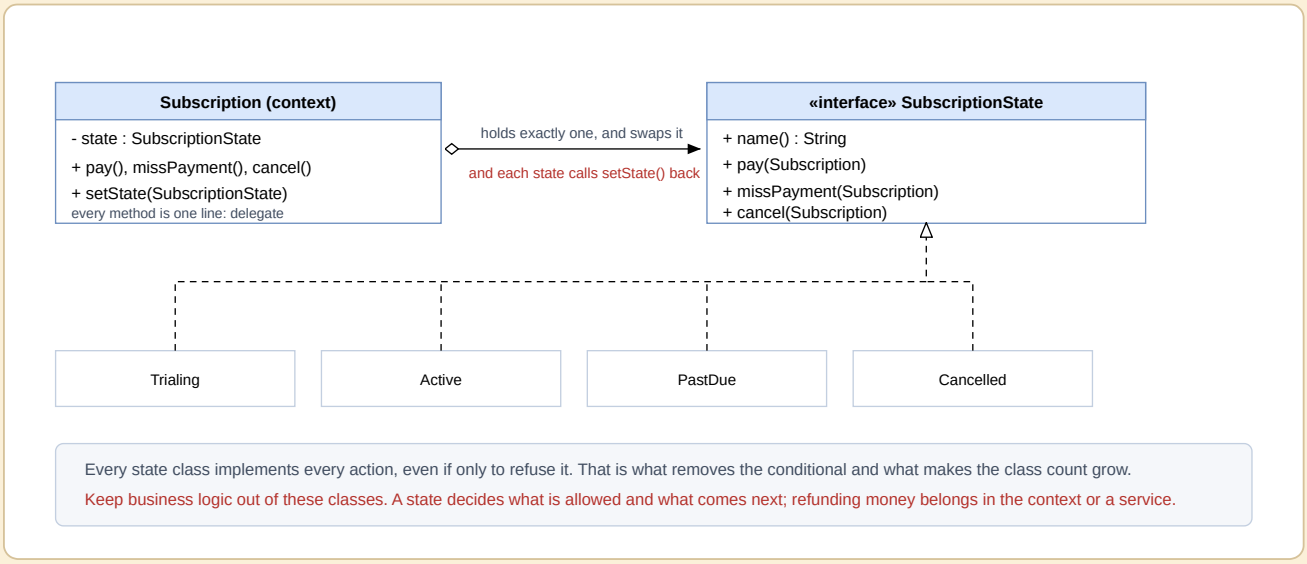
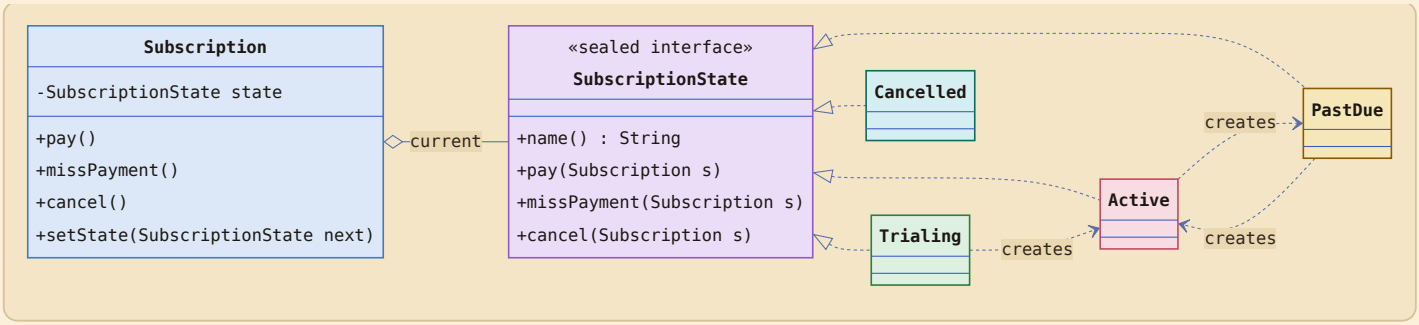


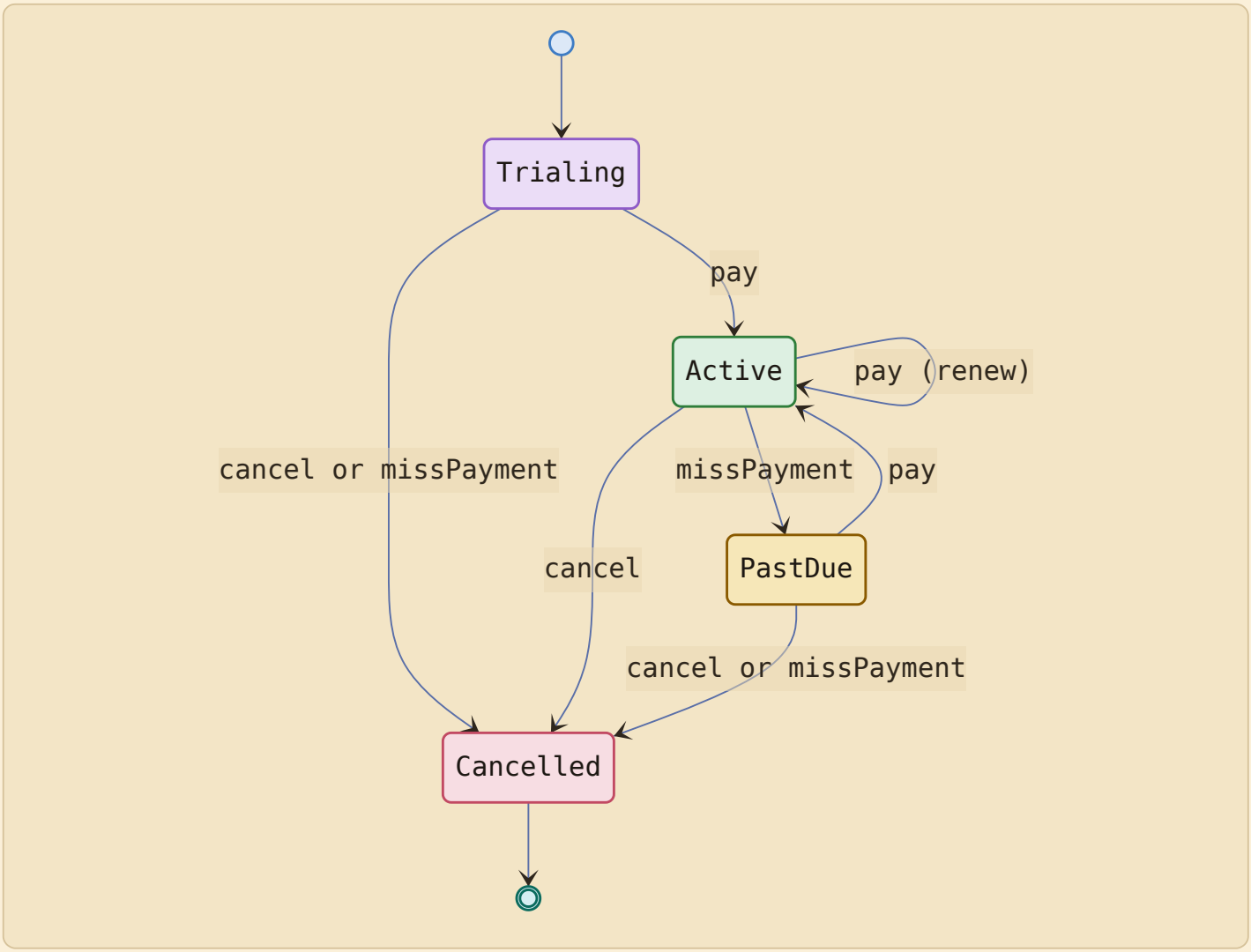
Figure — Class diagram: Subscription (context) holds a SubscriptionState; Trialing, Active, PastDue and Cancelled implement it and call setState on the context (course diagram)

In the diagram above the states reach **back** into the context to replace themselves, and the state classes know each other (Trialing creates an Active).

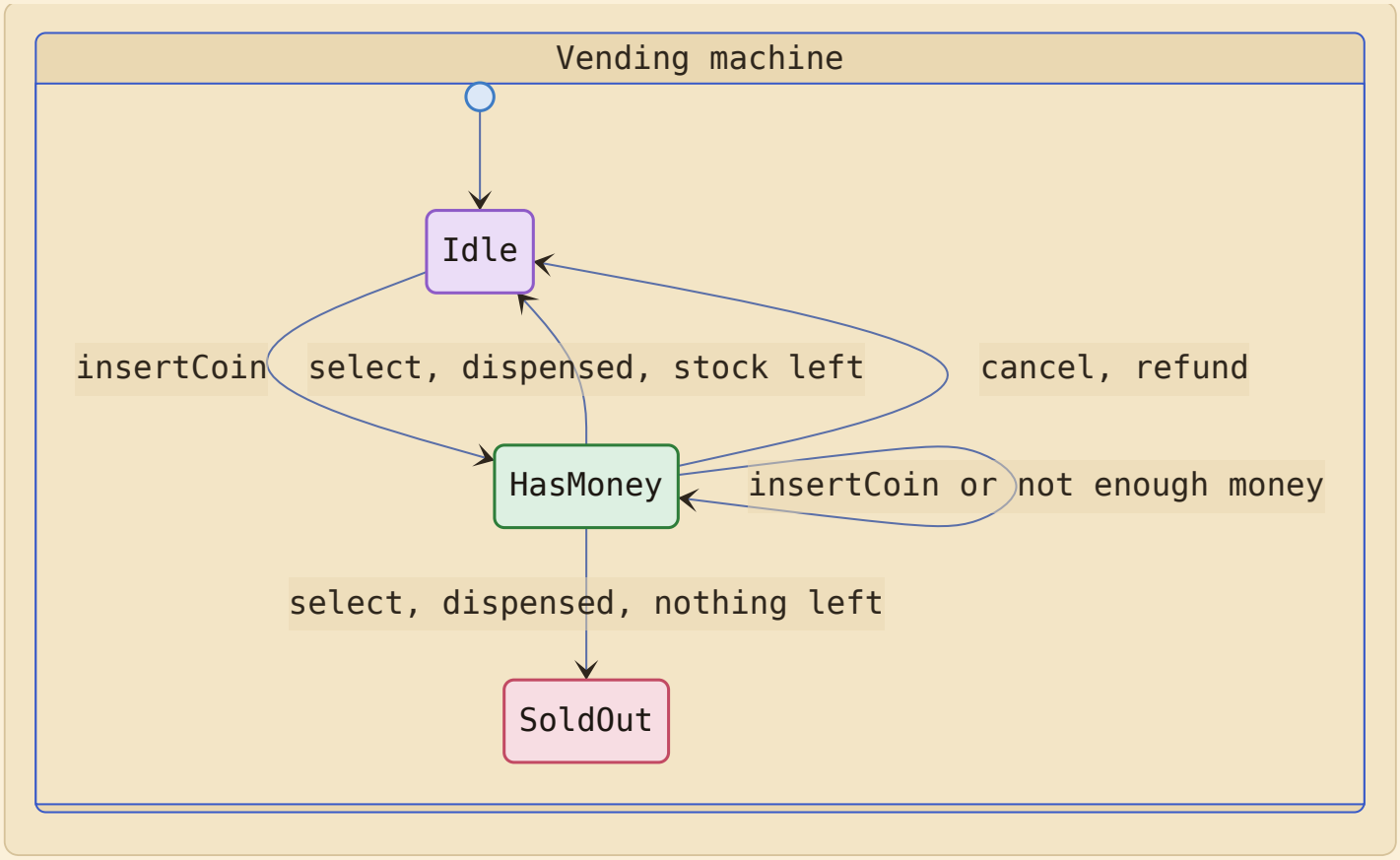
Our Java design (class-per-state version):

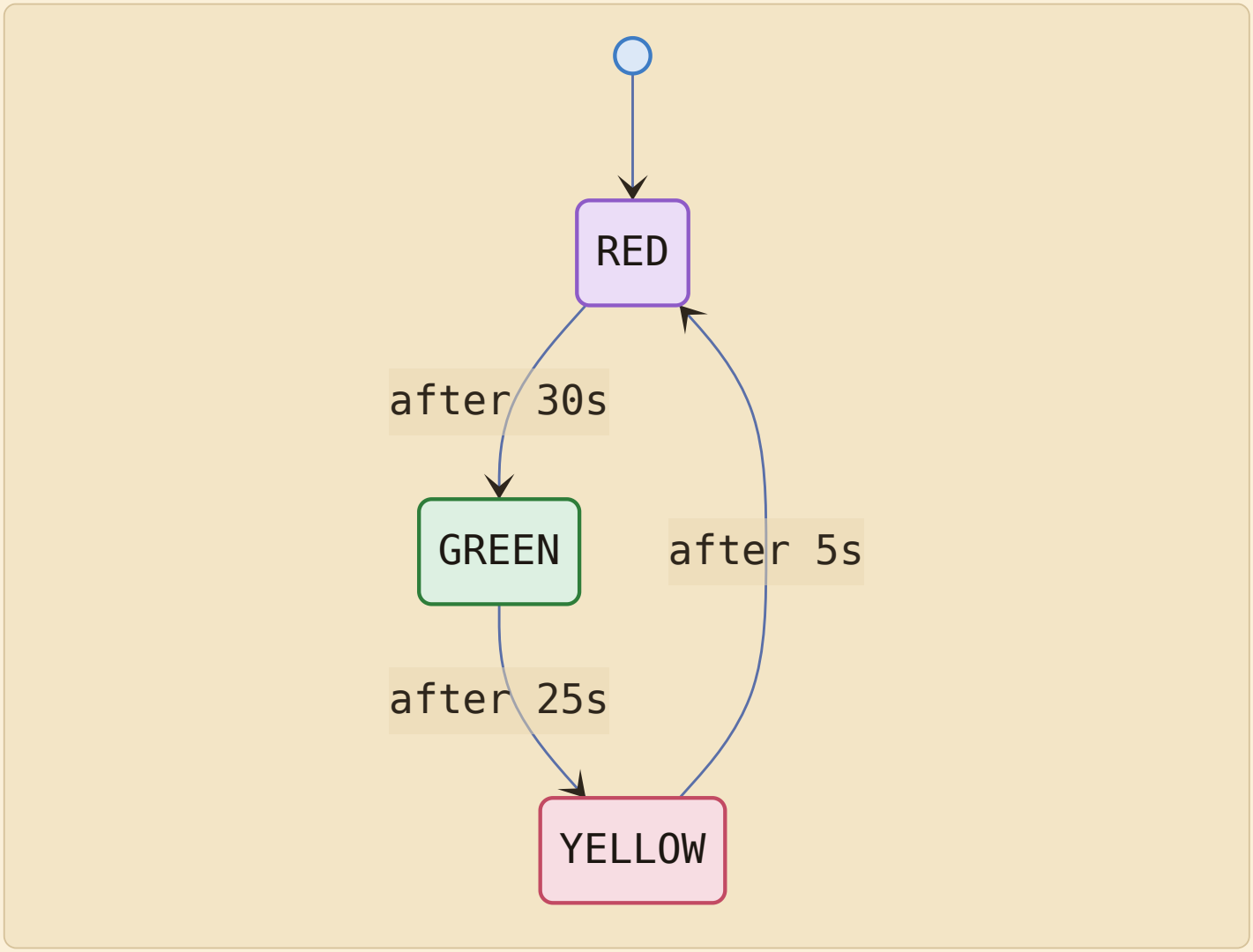


The state machine itself:



Vending machine and traffic light:


Vending machine
Idle
insertCoin
select, dispensed, stock left
cancel, refund
HasMoney
insertCoin or not enough money
select, dispensed, nothing left
SoldOut


RED
after 30s
GREEN
after 5s
after 25s
YELLOW

Java example

`lld.ch11.state` shows three styles:

1. **Class per state** (`Subscription` + four `record` states). `SubscriptionState` is a **sealed** interface (one that lists exactly which types may implement it): the compiler knows the complete set of states. The states have no fields, so **records** (Java's compact data classes whose fields are fixed at creation) with no components are a neat, **immutable** (cannot change after creation) choice.
2. **Enum-based state machine** (`TrafficLight`). Each enum constant overrides `next()`; shared behaviour is a **default method** (an interface method with a body, inherited by implementers unless they override it). The **whole machine is readable in one file** — the idiomatic Java answer for simple machines.
3. **Vending machine** (`VendingMachine` + `Idle`, `HasMoney`, `SoldOut`). The state interface uses **default methods that refuse**, so each state overrides only the actions it allows (`SoldOut` overrides nothing). Business actions (dispensing, returning change) live in the **context**; states only rule on whether an action is legal and which state follows. Money is kept as whole **cents in an `int`**, which is exact (unlike `double`); a real machine would use a `Money`/`BigDecimal` type (`BigDecimal` is Java's exact decimal-number class), as the Strategy and Visitor examples do.

```java
// File: code/lld/ch11/state/SubscriptionState.java
package lld.ch11.state;

/**
 * STATE interface. Each state implements all actions, including ones it only turns down.
 * Sealed: the compiler knows the full set of states.
 */
public sealed interface SubscriptionState permits Trialing, Active, PastDue, Cancelled {
    String name();
    void pay(Subscription s);
    void missPayment(Subscription s);
    void cancel(Subscription s);
}
```

```java
// File: code/lld/ch11/state/Trialing.java
package lld.ch11.state;

public record Trialing() implements SubscriptionState {
    public String name() { return "trialing"; }
    public void pay(Subscription s) {
        System.out.println("  [trialing] first payment taken");
        s.setState(new Active());                    // the state chooses its successor
    }
    public void missPayment(Subscription s) {
        System.out.println("  [trialing] trial ended unpaid");
        s.setState(new Cancelled());
    }
    public void cancel(Subscription s) { s.setState(new Cancelled()); }
}
```

```java
// File: code/lld/ch11/state/Active.java
package lld.ch11.state;

public record Active() implements SubscriptionState {
    public String name() { return "active"; }
    public void pay(Subscription s) { System.out.println("  [active] renewed for another period"); }
    public void missPayment(Subscription s) {
        System.out.println("  [active] payment failed, grace period started");
        s.setState(new PastDue());
    }
    public void cancel(Subscription s) { s.setState(new Cancelled()); }
}
```

```java
// File: code/lld/ch11/state/PastDue.java
package lld.ch11.state;

public record PastDue() implements SubscriptionState {
    public String name() { return "past_due"; }
    public void pay(Subscription s) {
        System.out.println("  [past_due] arrears paid");
        s.setState(new Active());
    }
    public void missPayment(Subscription s) {
        System.out.println("  [past_due] grace period over");
        s.setState(new Cancelled());
    }
    public void cancel(Subscription s) { s.setState(new Cancelled()); }
}
```

```java
// File: code/lld/ch11/state/Cancelled.java
package lld.ch11.state;

/** Terminal state: refuses everything, loudly, in one place. */
public record Cancelled() implements SubscriptionState {
    public String name() { return "cancelled"; }
    public void pay(Subscription s) { System.out.println("  [cancelled] refused: no payments after cancellation");
}
    public void missPayment(Subscription s) { System.out.println("  [cancelled] ignored: nothing is billed"); }
    public void cancel(Subscription s) { System.out.println("  [cancelled] rejected: already cancelled"); }
}
```

```java
// File: code/lld/ch11/state/Subscription.java
package lld.ch11.state;

/** The CONTEXT: every public action is one line of delegation. No status switch anywhere. */
public final class Subscription {
    private SubscriptionState state = new Trialing();

    void setState(SubscriptionState next) {
        System.out.println("    " + state.name() + " -> " + next.name());
        state = next;
    }

    public String status() { return state.name(); }

    public void pay()         { state.pay(this); }
    public void missPayment() { state.missPayment(this); }
    public void cancel()      { state.cancel(this); }
}
```

```java
// File: code/lld/ch11/state/TrafficLight.java
package lld.ch11.state;

/**
 * State pattern the idiomatic-Java way: an enum whose constants override behaviour.
 * The whole machine is readable in one file.
 */
public enum TrafficLight {
    RED(30) {
        TrafficLight next() { return GREEN; }
        boolean pedestriansMayCross() { return true; }
    },
    GREEN(25) {
        TrafficLight next() { return YELLOW; }
    },
    YELLOW(5) {
        TrafficLight next() { return RED; }
    };

    private final int seconds;

    TrafficLight(int seconds) { this.seconds = seconds; }

    int seconds() { return seconds; }
    abstract TrafficLight next();
    boolean pedestriansMayCross() { return false; }    // default for most states
}
```

```java
// File: code/lld/ch11/state/VendingState.java
package lld.ch11.state;

/** Vending machine states. Default methods = "refuse", so each state overrides only what it allows. */
interface VendingState {
    String name();

    default void insertCoin(VendingMachine m, int cents) {
        System.out.println("  [" + name() + "] coin rejected, returned " + cents);
    }

    default void select(VendingMachine m, String code) {
        System.out.println("  [" + name() + "] selection ignored");
    }

    default void cancel(VendingMachine m) {
        System.out.println("  [" + name() + "] nothing to cancel");
    }
}
```

```java
// File: code/lld/ch11/state/Idle.java
package lld.ch11.state;

final class Idle implements VendingState {
    public String name() { return "idle"; }

    @Override
    public void insertCoin(VendingMachine m, int cents) {
        m.addBalance(cents);
        m.setState(new HasMoney());
    }
}
```

```java
// File: code/lld/ch11/state/HasMoney.java
package lld.ch11.state;

final class HasMoney implements VendingState {
    public String name() { return "has_money"; }

    @Override
    public void insertCoin(VendingMachine m, int cents) { m.addBalance(cents); }

    @Override
    public void select(VendingMachine m, String code) {
        if (!m.inStock(code)) { System.out.println("  [has_money] " + code + " is empty, choose another"); return;
}
        int price = m.priceOf(code);
        if (m.balance() < price) {
            System.out.println("  [has_money] " + code + " costs " + price + ", balance " + m.balance());
            return;
        }
        m.dispense(code);                                  // business action lives in the context
        m.returnChange();
        m.setState(m.anythingInStock() ? new Idle() : new SoldOut());
    }

    @Override
    public void cancel(VendingMachine m) {
        m.returnChange();
        m.setState(new Idle());
    }
}
```

```java
// File: code/lld/ch11/state/SoldOut.java
package lld.ch11.state;

final class SoldOut implements VendingState {
    public String name() { return "sold_out"; }        // every action uses the refusing defaults
}
```

```java
// File: code/lld/ch11/state/VendingMachine.java
package lld.ch11.state;

import java.util.LinkedHashMap;
import java.util.Map;

/** CONTEXT: owns inventory and money; states rule on legality and pick the next state. */
final class VendingMachine {
    private record Slot(String product, int priceCents, int count) {}

    private final Map<String, Slot> slots = new LinkedHashMap<>();
    private VendingState state = new Idle();
    private int balance = 0;

    void load(String code, String product, int price, int count) { slots.put(code, new Slot(product, price,
count)); }

    // actions: delegate to the current state
    void insertCoin(int cents) { System.out.println("insert " + cents); state.insertCoin(this, cents); }
    void select(String code)   { System.out.println("select " + code); state.select(this, code); }
    void cancel()              { System.out.println("cancel"); state.cancel(this); }

    // helpers the states call
    void setState(VendingState next) {
        System.out.println("    " + state.name() + " -> " + next.name());
        state = next;
    }
    void addBalance(int cents) { balance += cents; System.out.println("  balance " + balance); }
    int balance() { return balance; }
    boolean inStock(String code) { return slots.containsKey(code) && slots.get(code).count() > 0; }
    boolean anythingInStock() { return slots.values().stream().anyMatch(s -> s.count() > 0); }
    int priceOf(String code) { return slots.get(code).priceCents(); }

    void dispense(String code) {
        Slot s = slots.get(code);
        slots.put(code, new Slot(s.product(), s.priceCents(), s.count() - 1));
        balance -= s.priceCents();
        System.out.println("  >> dispensed " + s.product());
    }

    void returnChange() {
        if (balance > 0) System.out.println("  >> change returned " + balance);
        balance = 0;
    }
}
```

```java
// File: code/lld/ch11/state/Main.java
package lld.ch11.state;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Subscription lifecycle (class per state) ===");
        Subscription sub = new Subscription();
        sub.pay();           // trialing -> active
        sub.missPayment();   // active -> past_due
        sub.pay();           // past_due -> active
        sub.cancel();        // active -> cancelled
        sub.pay();           // rejected, stays cancelled
        System.out.println("  final status: " + sub.status());

        System.out.println("\n=== 2. Traffic light (enum-based state machine) ===");
        TrafficLight light = TrafficLight.RED;
        for (int i = 0; i < 4; i++) {
            System.out.printf("  %-6s for %2ds, pedestrians may cross: %s%n",
                light, light.seconds(), light.pedestriansMayCross());
            light = light.next();
        }

        System.out.println("\n=== 3. Vending machine ===");
        VendingMachine vm = new VendingMachine();
        vm.load("A1", "cola", 150, 1);
        vm.load("B2", "chips", 100, 0);
        vm.select("A1");        // idle: ignored
        vm.insertCoin(100);     // idle -> has_money
        vm.select("A1");        // not enough
        vm.select("B2");        // empty slot
        vm.insertCoin(100);
        vm.select("A1");        // dispense, change 50, -> sold_out (nothing left)
        vm.insertCoin(25);      // sold_out: rejected
    }
}
```

Output:

```
=== 1. Subscription lifecycle (class per state) ===
  [trialing] first payment taken
    trialing -> active
  [active] payment failed, grace period started
    active -> past_due
  [past_due] arrears paid
    past_due -> active
    active -> cancelled
  [cancelled] refused: no payments after cancellation
  final status: cancelled

=== 2. Traffic light (enum-based state machine) ===
  RED    for 30s, pedestrians may cross: true
  GREEN  for 25s, pedestrians may cross: false
  YELLOW for  5s, pedestrians may cross: false
  RED    for 30s, pedestrians may cross: true

=== 3. Vending machine ===
select A1
  [idle] selection ignored
insert 100
  balance 100
    idle -> has_money
select A1
  [has_money] A1 costs 150, balance 100
select B2
  [has_money] B2 is empty, choose another
insert 100
  balance 200
select A1
  >> dispensed cola
  >> change returned 50
    has_money -> sold_out
insert 25
  [sold_out] coin rejected, returned 25
```

The final subscription line shows the payoff: an **illegal action** is refused the same way every time, by code in a single file.

State vs Strategy (and Bridge) — same diagram, different reasons

These patterns draw **the same class diagram** (an object keeps a reference to an interface and forwards work to it). It is one of the most common interview questions, and the diagram cannot answer it:

	State	Strategy	Bridge
Who chooses the object	The **states**, which replace themselves via the context	The **caller**, from outside	Chosen once at construction, usually by configuration
How often it changes	Repeatedly — that is the point	Set once, sometimes per call	Effectively never after wiring
Do the alternatives reference one another?	**Yes** (`Trialing` builds an `Active`)	**No** — never	No
What varies	The object's current *condition*, and therefore its allowed actions	*How* one job gets done	A whole dimension of the design
Why it exists	To model a lifecycle	To make one algorithm replaceable	To stop two class hierarchies multiplying

The reliable test: do the alternatives name each other? If yes, it is a State machine. If no, it is Strategy (or Bridge — then ask how often the choice changes).

Two more things State gets confused with:

Looks like	The actual difference
An enum plus one `switch`	Frequently the better answer, and easy to defend in an interview. Class-per-state starts to win only once the same status switch is repeated in three or more methods.
A state-machine library (Spring Statemachine, XState)	The same idea expressed as data (states, events, guards, actions) rather than classes. Once a machine grows past roughly a dozen states, the declarative form is easier to read and to draw.

Trade-offs

Pros	Cons
Everything about one state sits in a single file.	One class per state; small classes multiply.
The repeated conditional disappears; no branch can be missed.	The transitions live in several files, so no single place shows the whole machine.
Illegal actions are rejected consistently.	Each state class must implement all actions, even if only to say no.
A new state = a new class; the compiler lists what it must implement.	Persisting needs a name↔class mapping — keep it in exactly one place.

When not to use it

- **Only two states and a handful of methods** — a boolean flag with an `if` reads better.
- **Behaviour doesn't change, only data does** — store the differing value in a field; a class per state adds nothing.
- **A big, formal machine** (more than about a dozen states with guards) — use a **transition table** (`Map<(state, event), handler>`) or a state-machine library (e.g. **Spring Statemachine**), where the diagram can be the source of truth.
- **Rule of thumb from the course:** the pattern pays off once **three or more methods repeat the same status switch**. Below that, an enum with a switch is a perfectly defensible answer.
- **A signal you have outgrown it:** you keep sketching the machine on paper just to follow your own code. At that point make the sketch (a transition table or state chart) the real source of truth, instead of classes that only approximate it.

Common mistakes

1. **Successors hard-coded everywhere.** Fine at 4 states, painful at 12. Past that, let the context or a transition table own the transitions.
2. **Business logic inside state classes.** A state's job is to rule on *legality* and pick the *successor*; charging cards and sending emails belong in the context or a service it calls. Otherwise the state classes become a dumping ground.
3. **Forgetting to make illegal actions explicit** (silently doing nothing).
4. **Storing state objects in the DB** — store a name/enum and map it back to a state in exactly one place.
5. **Sharing stateful state objects** between contexts. Stateless states (like ours) can be shared safely; stateful ones cannot.

State in other languages

- **Java:** an `enum` whose constants override behaviour (like `TrafficLight`) keeps the whole graph in one file. A `sealed` interface plus exhaustive pattern-matching `switch` gives compiler-checked coverage of every state.
- **Kotlin / Scala:** `sealed` hierarchies with exhaustive `when` / `match`.
- **Rust:** enums with data, matched exhaustively. With the **typestate** pattern each state is a separate *type*, so calling an action that is illegal in that state is a compile error.
- **Python:** a dict mapping `(state, action)` to a handler is a compact transition table that shows every transition in one place.
- **TypeScript:** a **discriminated union** (a set of object types that each carry a fixed tag field saying which kind they are) plus a `switch` over its tag field, which the compiler can check for missing cases.

Real-world examples

- **Order lifecycles** in food-delivery and e-commerce apps: placed → accepted → preparing → out for delivery → delivered (or cancelled); which actions are legal depends on the status (you can cancel while "placed" but usually not once "out for delivery").
- **Payment and subscription lifecycles** (Stripe subscription statuses such as `trialing`, `active`, `past_due`, `canceled`).
- **TCP connections** (TCP is the protocol under most internet traffic, giving ordered, reliable delivery; see System Design Fundamentals) — GoF's own example: `LISTEN`, `ESTABLISHED`, `CLOSED`, each responding differently to the same call.
- **Workflow/approval engines** (draft → in review → approved/rejected), **document editors' modes**, **game characters** (idle, running, jumping).
- **Lexers/parsers** (programs that split text into tokens, then build a structure from them), **vending machines, ATMs, elevators, traffic lights** — favourite **machine-coding** questions (an interview round where you write working code for a small system in an hour or two) (see the case-study chapters).
- **Libraries**: Spring Statemachine; in the JDK, `Thread.State` is an *enum of states* (the data), not the pattern itself.

In the interview

> **30-second answer:** "State swaps a status field, and the switch statements repeated around it, for one class per state. The context forwards each action to whichever state object it currently holds, and that object decides whether the action is legal and which state follows. Each state's rules sit together in one file, and an illegal action is refused the same way every time. It draws exactly like Strategy, but states name one another and move the machine forward on their own, while strategies are picked from outside and never reference each other. In Java, for small machines, I'd use an enum with per-constant behaviour or a sealed interface."

Follow-ups:

- *State vs Strategy?* Use the naming test and the "who chooses" test (table above).
- *Where should transitions live?* In the states for small machines; in the context or a transition table when the machine grows, so the whole graph is visible in one place.
- *How do you persist it?* Store the state's name (enum) in the DB (database); rebuild the state object through one factory/mapping.
- *How do you handle concurrency?* Two requests (e.g. "cancel" and "deliver") can **race** (run at the same moment and interfere, so the result depends on timing): guard transitions with a lock or, in a DB, an **optimistic** (no lock held; the write simply fails if someone changed the row first) `UPDATE ... WHERE status = 'PLACED'` and check the affected row count.
- *Design a vending machine.* States Idle, HasMoney, SoldOut (maybe Dispensing, Maintenance); context owns inventory and balance; states allow or refuse actions (as in our code).

Self-check

1. **Q:** Who changes the current state in the State pattern? **A:** The state objects (via the context's `setState`).
2. **Q:** What is the "naming test"? **A:** If the alternatives reference each other, it's State; if never, it's Strategy/Bridge.
3. **Q:** When is an `enum` better than class-per-state? **A:** When the machine is small; the whole machine is visible in one file.
4. **Q:** What belongs in the context, not the state? **A:** Business actions and data (dispensing, charging, inventory).
5. **Q:** What does a sealed state interface give you? **A:** The compiler knows every state, so `switch`es over states can be checked for exhaustiveness.

TL;DR

A class for each state; the context forwards every action; each state picks what follows it. Same diagram as Strategy, but states **know each other**. Use enums for small machines, tables/libraries for big ones.

11.3 Strategy

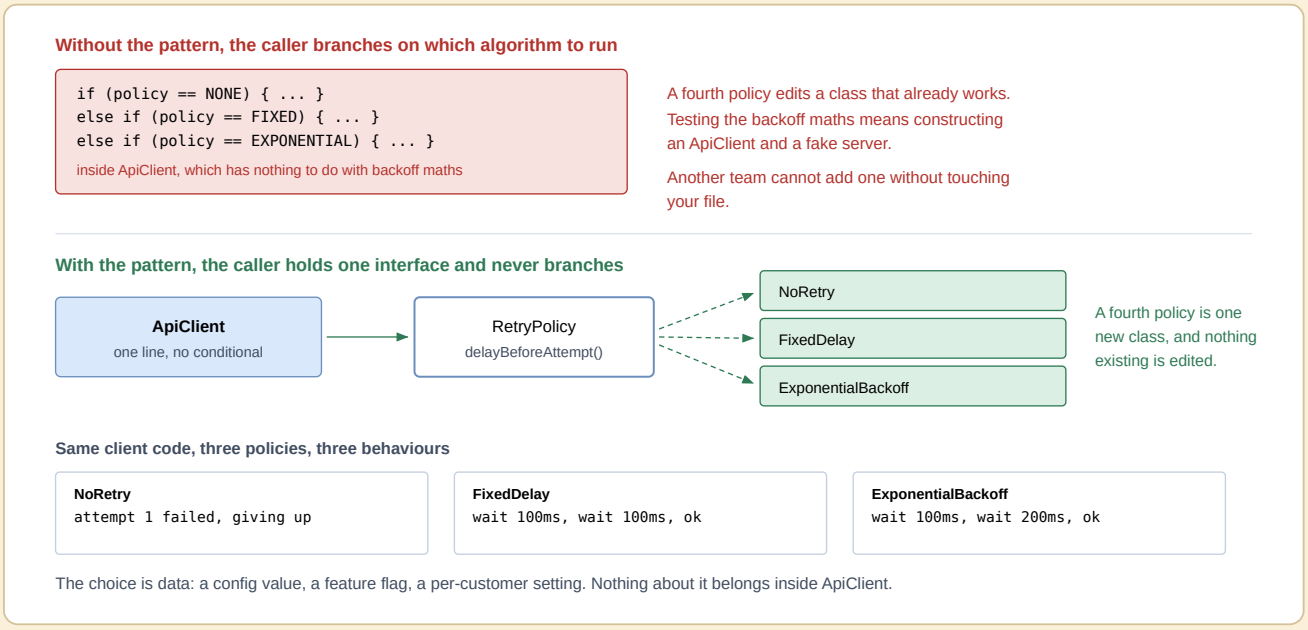


Figure — Top: ApiClient branches on which retry policy to use. Bottom: ApiClient holds one RetryPolicy interface and never branches (course diagram)

What it is

Strategy (GoF intent, paraphrased): group related algorithms into a **family**, wrap each one in its own class behind a **common interface**, and make them interchangeable, so the algorithm can change without touching the code that calls it; the choice can be made at run time. The object using it (the **context**) does not branch on which algorithm it has.

GoF participants (and who plays them in our code):

Participant	Role	In `lld.ch11.strategy`
Strategy	Interface shared by all the algorithms; the context calls only this	`RetryPolicy` (also `Comparator` in part 2)
ConcreteStrategy	One implementation per algorithm	`FixedDelay`, `ExponentialBackoff`, the `RetryPolicy.none()` lambda (the course diagram names it `NoRetry`)
Context	Is configured with a strategy object and delegates to it; may pass it the data it needs	`ApiClient`

Everyday analogy: getting to the airport. Car, train or taxi — same goal, different algorithm. You pick based on time, money and weather; the trip itself ("get to the airport") doesn't change.

The problem it solves

Retrying a failed request can mean: no retry, a fixed delay, or **exponential backoff** (doubling the wait after each failure — e.g. 100 ms, 200 ms, 400 ms). Written directly, the branch sits inside `ApiClient`:

```
if (policy == NONE)             { /* give up */ }
else if (policy == FIXED)       { /* wait 100ms, try again */ }
else if (policy == EXPONENTIAL) { /* double the wait after each failure */ }
```

`ApiClient` now contains backoff arithmetic that has nothing to do with calling an API. A fourth policy means editing a working class; you cannot test the backoff maths without building a client and faking a server; and a different team has to edit your class just to contribute a policy.

Strategy gives every algorithm a class of its own. `ApiClient` asks its `RetryPolicy` "how long before the next attempt?" and **never checks what kind of policy it holds**. The choice is made **outside** the client — from

configuration, a **feature toggle** (a setting that switches behaviour on or off without redeploying), or a customer-specific setting.

How it works

1. Define a **strategy interface** — small, ideally one question (`delayBeforeAttempt(attempt)`).
2. Write one **concrete strategy** per algorithm; each holds only its own parameters.
3. The **context** receives a strategy (constructor, setter or method parameter) and calls it.
4. A **factory, configuration or DI container** (a library that creates objects and hands them their dependencies — see §11.7.2) decides which strategy to use — not every call site.

Structure (UML)

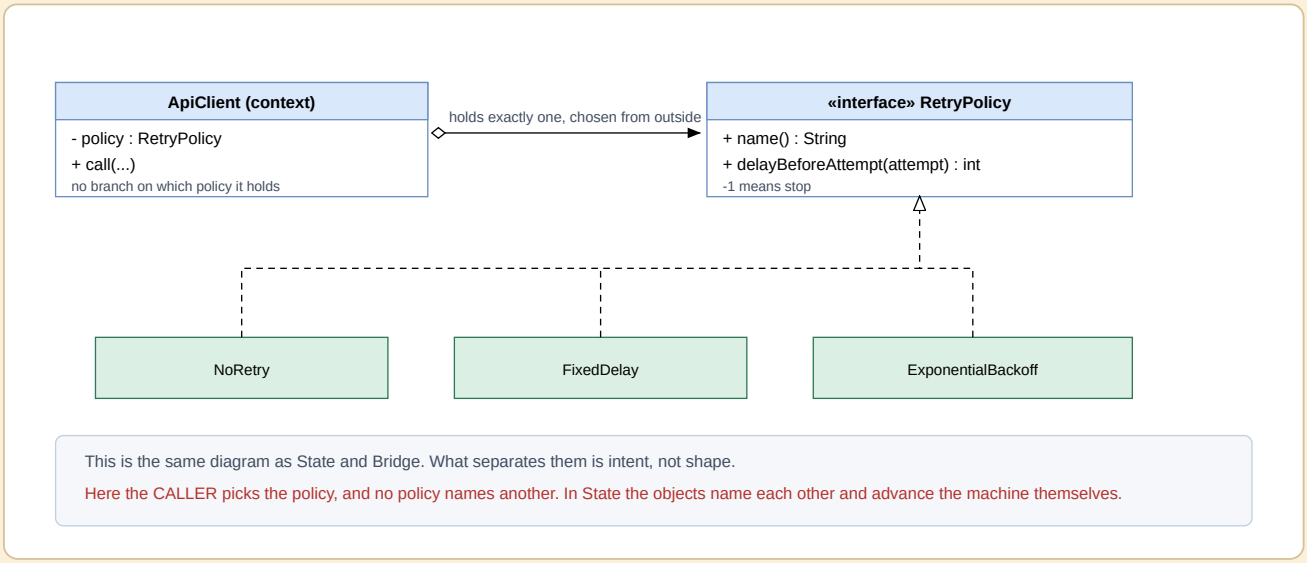
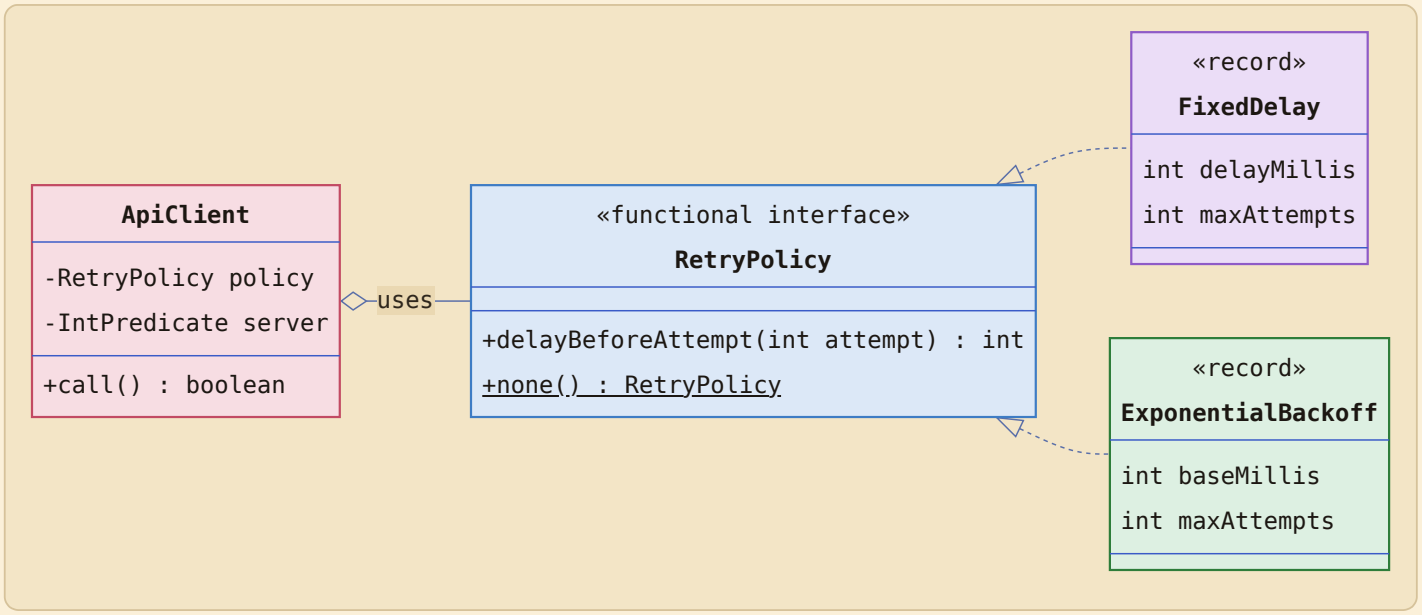


Figure — Class diagram: ApiClient holds a RetryPolicy; NoRetry, FixedDelay and ExponentialBackoff implement it (course diagram)

In the diagram above, **no strategy references another** — `ExponentialBackoff` has never heard of `FixedDelay`. That absence is how you tell Strategy from State.

Our Java design:



Java example

`lld.ch11.strategy`:

1. **Retry policies.** `RetryPolicy` has one abstract method, so `RetryPolicy.none()` is just a **lambda**. Strategies with configuration (`FixedDelay`, `ExponentialBackoff`) are **records**. `ApiClient.call()` is identical for all runs; only the injected policy differs. (Delays are printed, not slept, to keep the demo fast.)

2. `Comparator` — the JDK's everyday strategy. `List.sort(comparator)` keeps the sorting algorithm fixed and lets you plug in the comparison. `Comparator.comparing(...).reversed().thenComparing(...)` composes strategies.
3. **Strategy registry.** A `Map<String, LongUnaryOperator>` of lambdas picked by a configuration key — short and readable when each strategy is one line.

```java
// File: code/lld/ch11/strategy/RetryPolicy.java
package lld.ch11.strategy;

/**
 * STRATEGY interface: one small question. Returns the delay in ms before the given
 * attempt (attempt >= 2), or STOP. One abstract method, so a lambda can be a policy.
 */
@FunctionalInterface
public interface RetryPolicy {
    int STOP = -1;

    int delayBeforeAttempt(int attempt);

    static RetryPolicy none() { return attempt -> STOP; }
}
```

```java
// File: code/lld/ch11/strategy/FixedDelay.java
package lld.ch11.strategy;

/** Strategies with configuration earn a real type (a record here). */
public record FixedDelay(int delayMillis, int maxAttempts) implements RetryPolicy {
    public int delayBeforeAttempt(int attempt) {
        return attempt > maxAttempts ? STOP : delayMillis;
    }
}
```

```java
// File: code/lld/ch11/strategy/ExponentialBackoff.java
package lld.ch11.strategy;

/** 100, 200, 400, ... ms. Knows nothing about FixedDelay: strategies never name each other. */
public record ExponentialBackoff(int baseMillis, int maxAttempts) implements RetryPolicy {
    public int delayBeforeAttempt(int attempt) {
        return attempt > maxAttempts ? STOP : baseMillis << (attempt - 2);
    }
}
```

```java
// File: code/lld/ch11/strategy/ApiClient.java
package lld.ch11.strategy;

import java.util.function.IntPredicate;

/** CONTEXT: holds a policy and never asks WHICH policy it has. */
public final class ApiClient {
    private final RetryPolicy policy;
    private final IntPredicate server;     // fake server: attempt number -> success?

    public ApiClient(RetryPolicy policy, IntPredicate server) {
        this.policy = policy;
        this.server = server;
    }

    public boolean call() {
        for (int attempt = 1; ; attempt++) {
            if (server.test(attempt)) {
                System.out.println("  attempt " + attempt + ": ok");
                return true;
            }
            int delay = policy.delayBeforeAttempt(attempt + 1);
            if (delay == RetryPolicy.STOP) {
                System.out.println("  attempt " + attempt + ": failed, giving up");
                return false;
            }
            System.out.println("  attempt " + attempt + ": failed, waiting " + delay + "ms");  // simulated, no
sleep
        }
    }
}
```

```java
// File: code/lld/ch11/strategy/Main.java
package lld.ch11.strategy;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.function.IntPredicate;
import java.util.function.UnaryOperator;

public class Main {
    record Product(String name, long priceCents, double rating) {}

    public static void main(String[] args) {
        System.out.println("=== 1. Retry policies: same ApiClient code, three strategies ===");
        IntPredicate succeedsOnThirdTry = attempt -> attempt >= 3;
        List<Map.Entry<String, RetryPolicy>> policies = List.of(
            Map.entry("none (a lambda)", RetryPolicy.none()),
            Map.entry("FixedDelay(100, 4)", new FixedDelay(100, 4)),
            Map.entry("ExponentialBackoff(100, 4)", new ExponentialBackoff(100, 4)));
        for (Map.Entry<String, RetryPolicy> e : policies) {
            System.out.println(e.getKey() + ":");
            new ApiClient(e.getValue(), succeedsOnThirdTry).call();
        }
        System.out.println("ExponentialBackoff(100, 4) against a server that is always down:");
        new ApiClient(new ExponentialBackoff(100, 4), attempt -> false).call();

        System.out.println("\n=== 2. Comparator: the JDK's everyday strategy ===");
        List<Product> products = new ArrayList<>(List.of(
            new Product("mouse", 1_999, 4.5), new Product("keyboard", 4_999, 4.7),
            new Product("cable", 499, 4.5), new Product("monitor", 19_999, 4.2)));
        Comparator<Product> byPrice = Comparator.comparingLong(Product::priceCents);
        Comparator<Product> byRatingThenName =
            Comparator.comparingDouble(Product::rating).reversed().thenComparing(Product::name);
        products.sort(byPrice);
        System.out.println("  by price:          " + products.stream().map(Product::name).toList());
        products.sort(byRatingThenName);
        System.out.println("  by rating, name:   " + products.stream().map(Product::name).toList());

        System.out.println("\n=== 3. Strategy registry: lambdas chosen by configuration ===");
        Map<String, UnaryOperator<BigDecimal>> pricing = Map.of(
            "regular", p -> p,
            "member", p -> p.multiply(new BigDecimal("0.90")).setScale(2, RoundingMode.HALF_EVEN),  // 10% off
            "flash-sale", p -> p.subtract(new BigDecimal("20.00")).max(BigDecimal.ZERO));          // flat 20.00
off
        BigDecimal keyboardPrice = new BigDecimal("49.99");
        for (String tier : List.of("regular", "member", "flash-sale")) {   // e.g. read from config
            UnaryOperator<BigDecimal> strategy = pricing.get(tier);
            System.out.printf("  %-10s keyboard costs %s%n", tier, strategy.apply(keyboardPrice));
        }
    }
}
```

Extending it is cheap: a policy that adds random **jitter** to the delay, or one that honours a server's `Retry-After` header (an **HTTP** response header in which the server says how long to wait; HTTP is the request/response protocol of the web), is one more class (or lambda) plus one line where the policy is chosen. `ApiClient` does not change.

Output:

```
=== 1. Retry policies: same ApiClient code, three strategies ===
none (a lambda):
  attempt 1: failed, giving up
FixedDelay(100, 4):
  attempt 1: failed, waiting 100ms
  attempt 2: failed, waiting 100ms
  attempt 3: ok
ExponentialBackoff(100, 4):
  attempt 1: failed, waiting 100ms
  attempt 2: failed, waiting 200ms
  attempt 3: ok
ExponentialBackoff(100, 4) against a server that is always down:
  attempt 1: failed, waiting 100ms
  attempt 2: failed, waiting 200ms
  attempt 3: failed, waiting 400ms
  attempt 4: failed, giving up

=== 2. Comparator: the JDK's everyday strategy ===
  by price:          [cable, mouse, keyboard, monitor]
  by rating, name:   [keyboard, cable, mouse, monitor]

=== 3. Strategy registry: lambdas chosen by configuration ===
  regular    keyboard costs 49.99
  member     keyboard costs 44.99
  flash-sale keyboard costs 29.99
```

Lambdas as strategies

Of all the patterns, Strategy is the one a plain function replaces most often, and recognising that is part of understanding it. Since Java 8, any interface with a **single abstract method** (a *functional interface*) can be implemented by a lambda or method reference. So many strategies need **no class at all**: `Comparator<T>`, `Function`, `Predicate`, `Supplier`, or your own `RetryPolicy`. Keep a named class (or record) when the strategy has **configuration or state** (`FixedDelay(100, 4)`), needs a **name for logging**, or must be **discovered/registered** (e.g. Spring beans — objects the Spring container creates and manages — or `ServiceLoader`, the JDK's built-in plug-in discovery mechanism).

Other languages: Python passes a function (or `functools.partial` to bind its parameters), and a dict from name to function is a one-line registry. JavaScript/TypeScript pass a function argument, or an object whose properties are functions. Kotlin/Scala take a function-typed constructor parameter, often with a default. Go uses a one-method interface (satisfied implicitly) or a `func` field.

The registry in part 3 of the demo uses `BigDecimal` for prices. Money in domain code should never be `double`, because binary floating point cannot represent most cent values exactly.

Trade-offs

Pros	Cons
New variants without editing working code — even by other teams (Open/Closed).	One class per variant (unless lambdas), one more file to open.
Each algorithm is testable alone.	Someone must choose — ideally a factory or container, not every caller.
Behaviour is configurable at run time (flag, setting, header).	The interface must fit every variant, including future ones.
The context keeps to its real responsibility (no backoff maths in an API client).	Tempting to overuse: three strategies of one line each means three classes for three lines.

When not to use it

- **Just two branches, with no third in sight** — an `if` reads better. Introduce the pattern when a third variant actually shows up.
- **One-line branches** — a `Map` from key to lambda is shorter than a class for each.

- **Variants need different inputs** (one needs the customer, another the warehouse) — the shared interface is forced; rethink the split.
- **The choice is fixed for the life of the program** — decide it when wiring the program (or use Bridge) instead.

Common mistakes

1. **Making every call site choose** — the branch moved instead of disappearing. Let a factory/config/DI container choose.
2. **A bloated interface** with parameters half the implementations ignore — a sign the variants do different jobs and should not share one abstraction.
3. **Strategies that know about each other** — then you have State.
4. **Stateful strategies shared between threads** — keep strategies immutable (records help).

Real-world examples

- `Comparator` with `Collections.sort` / `List.sort` / `TreeMap` — the textbook example.
- `ThreadPoolExecutor`**'s** (the JDK's configurable thread pool) `RejectedExecutionHandler`: `AbortPolicy`, `CallerRunsPolicy`, `DiscardPolicy`, `DiscardOldestPolicy` — pick what happens when the pool is full.
- **Spring Security** `PasswordEncoder` (`BCryptPasswordEncoder`, `Argon2PasswordEncoder`, ...) chosen by configuration, so upgrading the **hash** (a one-way scrambling of the password, so the stored value can't be turned back into it) does not touch login code.
- **Resilience4j** (a Java fault-tolerance library) retry `IntervalFunction` (fixed, exponential, randomized backoff) and **Polly** in .NET — exactly this lesson's retry policies.
- **AWT/Swing** `LayoutManager` (`FlowLayout`, `BorderLayout`, `GridLayout`) — a container delegates layout to a strategy.
- **Payment method at checkout** (card, UPI [India's instant bank-to-bank payment system], wallet, cash on delivery), **pricing/discount rules by customer tier**, **route planning modes** (drive, walk, transit) in map apps, **compression/serialization formats** (serialization = turning objects into bytes or text for storage or sending) chosen by a content-type header.

In the interview

> **30-second answer:** "Strategy gives every variant of one job its own class behind a common interface, and something outside — usually a factory or configuration — picks the variant at run time. The context stops branching: my API client asks its retry policy how long to wait and never asks which policy it is. Each algorithm is testable alone and new ones don't touch working code. In Java a lambda is often enough, like a `Comparator`; I keep a class when the strategy has configuration. It shares its diagram with State, but strategies never reference each other."

Follow-ups:

- *Strategy vs State?* Strategies are picked from outside and don't know each other; states pick their successors.
- *Strategy vs Template Method?* Template Method swaps **one step** of an algorithm whose order is fixed, through **inheritance** (fixed once the object is created, by the subclass you instantiate). Strategy swaps **the complete algorithm** through **composition** (holding another object in a field and delegating to it) (chosen at run time, can vary several axes at once). Prefer Strategy when both fit.
- *Strategy vs Bridge?* Same diagram again. Bridge splits off a dimension of the design that is fixed when the program is assembled; Strategy is a decision that is meant to change, even from one call to the next. If a reviewer cannot tell which you meant, little is lost — the code is identical. What matters more is not using any of the three for two branches that will never become three.
- *Strategy vs Command?* Strategy is *how* to do something; Command is *what* to do, with its arguments, kept to run later or undo.
- *How is the strategy selected?* Config + factory (`Map<String, Strategy>`), DI container (Spring can inject a `Map<String, PaymentStrategy>` of all beans), or per-request (header, customer tier).
- *Relation to Open/Closed and DIP?* The context depends on an abstraction (DIP) and new strategies extend behaviour without modifying it (OCP).

Self-check

1. **Q:** Who chooses the strategy? **A:** Someone outside the context — ideally a factory/config/DI container.

2. **Q:** Why can `RetryPolicy.none()` be a lambda? **A:** `RetryPolicy` is a functional interface (one abstract method).
3. **Q:** Name two JDK strategies. **A:** `Comparator`; `RejectedExecutionHandler` (also `LayoutManager`).
4. **Q:** When is a map of lambdas better than classes? **A:** When each strategy is one line and needs no configuration.
5. **Q:** How do you tell Strategy from State in code? **A:** Strategies never reference each other.

TL;DR

Interchangeable algorithms behind one interface, chosen from outside. The context stops branching. Use lambdas for one-liners, records/classes when configured. Don't confuse with State (states know each other) or Template Method (inheritance, one step).

11.4 Template Method

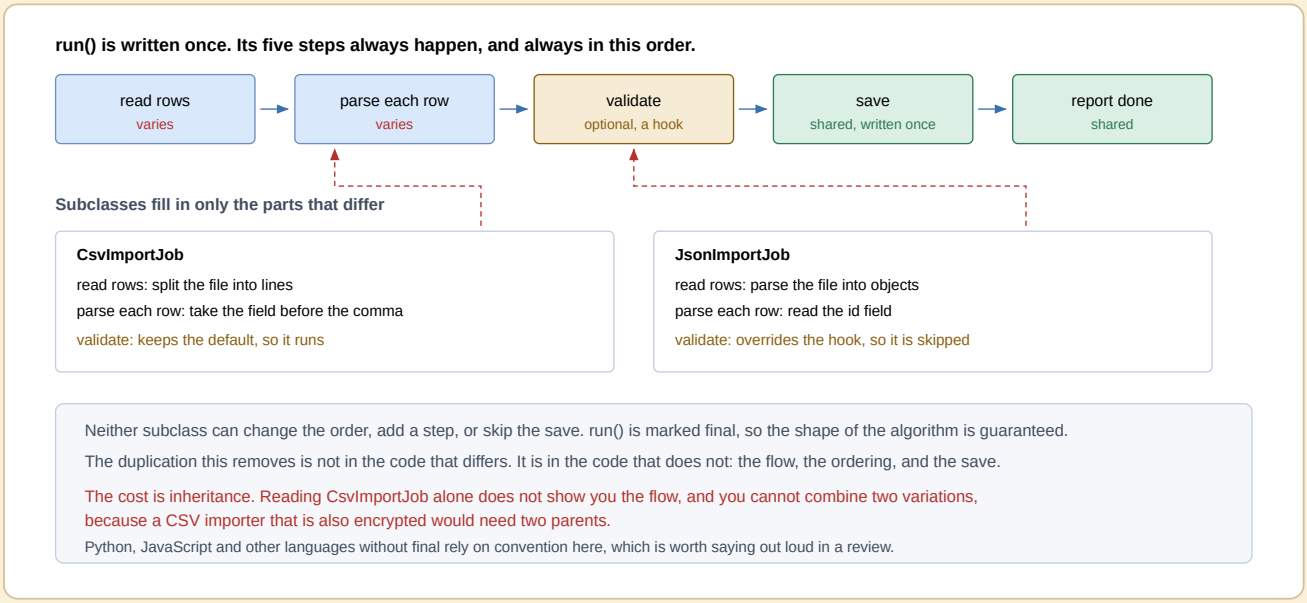


Figure — run() always performs five steps in the same order; subclasses fill in only reading and parsing, and one optional hook (course diagram)

What it is

Template Method (GoF intent, paraphrased): put the **outline of an algorithm** into one base-class operation (the *template method*) and leave certain steps for **subclasses** to fill in; subclasses may redefine those steps but cannot alter the overall sequence.

GoF participants (and who plays them in our code):

Participant	Role	In `lld.ch11.templatemethod`
AbstractClass	Declares the primitive steps (abstract or with a default) and implements the template method that calls them in order	`ImportJob`, `CaffeineBeverage`
ConcreteClass	Implements the primitive steps that differ for its variant	`CsvImportJob`, `JsonImportJob`, `Tea`, `Coffee`

Everyday analogy: a recipe card with blanks. "Boil water → brew ___ → pour into cup → add ___ (if wanted)." Tea and coffee fill the blanks differently; the order never changes.

The problem it solves

A data import job always: reads rows, parses each one, validates, saves, reports "done". Only reading and parsing differ between CSV and JSON. Written as separate classes, the **shared flow** (order, validation, saving, reporting) is copied into each. When saving must also write an audit record, three files change — and the missed one is the bug.

Template Method writes the flow once, in a method subclasses **cannot override** (`final`). The varying steps become methods the subclasses must supply.

How it works

1. Write the **template method** in an abstract class and mark it `final` so the order is guaranteed.
2. Inside it, call the steps in order. Steps come in three kinds:
   - **Required (abstract) steps** — every subclass must implement them; the compiler enforces it.
   - **Hooks** — steps with a **default** (often empty or `true`) that a subclass *may* override, e.g. `shouldValidate()`.
   - **Shared steps** — `private`, written once, not changeable.
3. Each subclass implements only what differs.

The Hollywood Principle — "Don't call us, we'll call you." (*Head First Design Patterns*.) The base class (high-level component) decides **when** the subclass's methods run; the subclass never calls the base class's flow itself. That **inversion of control** is the backbone of most **frameworks** (libraries that own the program's flow and call your code at set points): **the framework runs the sequence, and your class supplies only your own pieces.**

Structure (UML)

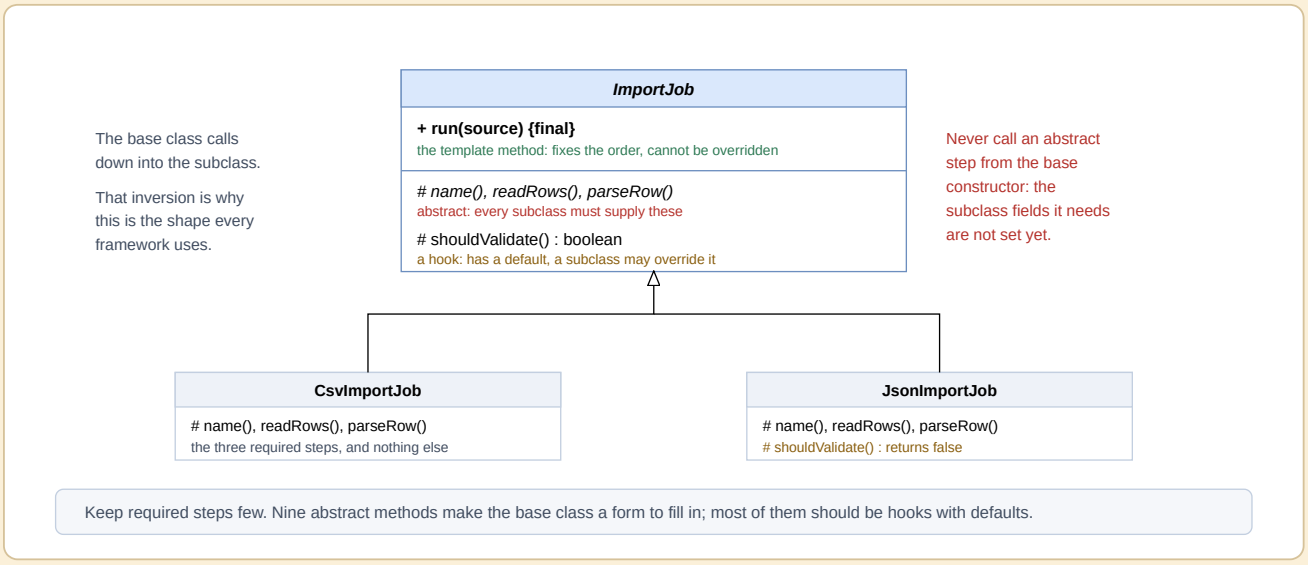
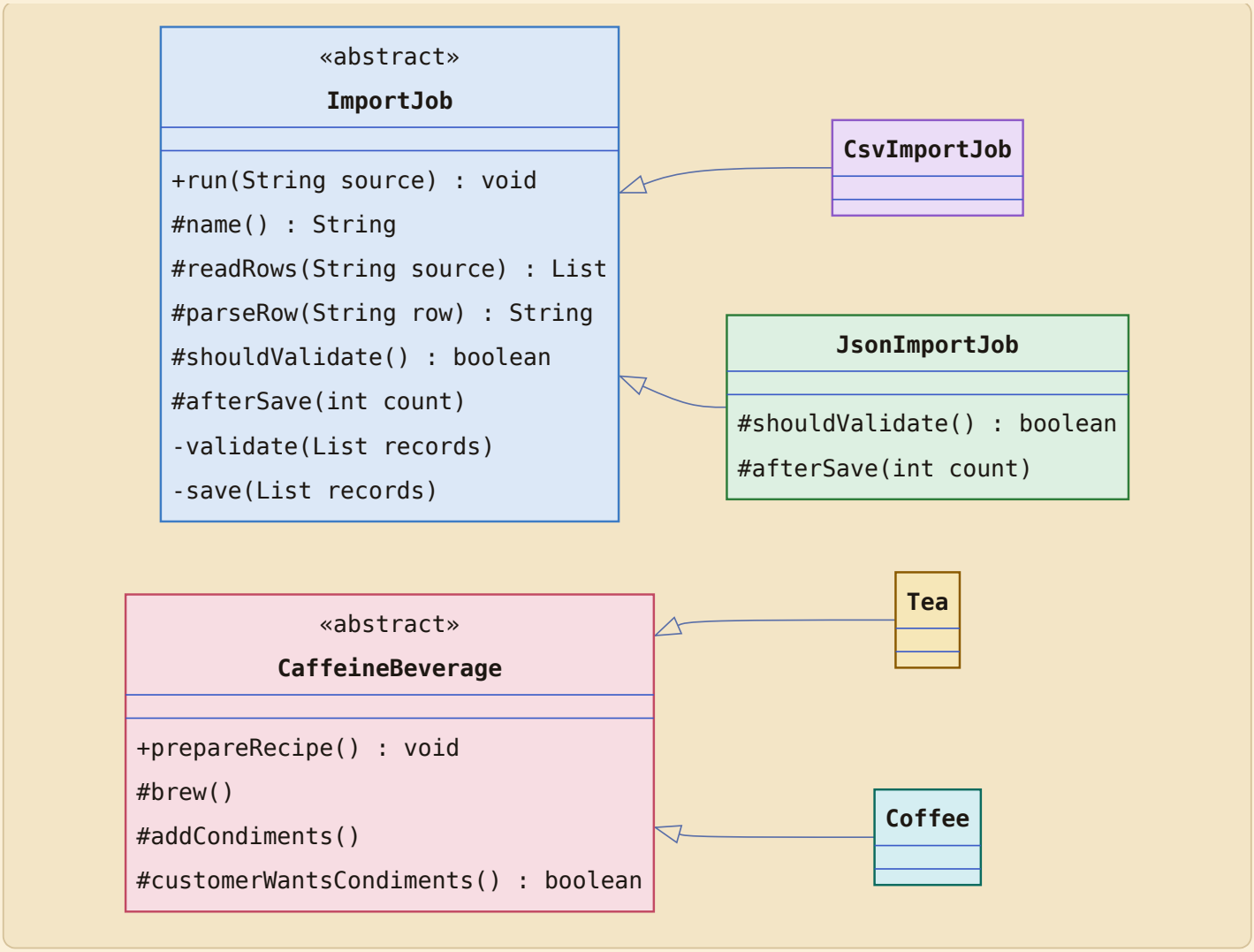


Figure — Class diagram: ImportJob with final run(), abstract steps name/readRows/parseRow, hook shouldValidate and private save; CsvImportJob and JsonImportJob override only what differs (course diagram)

In the diagram above, the subclasses are tiny — but `CsvImportJob` on its own does not reveal the sequence it will follow at run time; only the parent shows that. That is the trade-off.

Our Java design:



`run()` and `prepareRecipe()` are `final` in the code (Mermaid has no marker for `final`).

Java example

`lld.ch11.templatemethod`:

1. **Import jobs.** `ImportJob.run` is `final`. `name`, `readRows`, `parseRow` are **abstract** (required). `shouldValidate` and `afterSave` are **hooks** (defaults `true` and "do nothing"). `validate` and `save` are **private** shared steps. `JsonImportJob` overrides both hooks.
2. **Beverages** (the *Head First* example). `customerWantsCondiments()` is a hook: `Coffee(black = true)` turns off the optional step.

```java
// File: code/lld/ch11/templatemethod/ImportJob.java
package lld.ch11.templatemethod;

import java.util.ArrayList;
import java.util.List;

/** ABSTRACT CLASS holding the TEMPLATE METHOD. */
public abstract class ImportJob {

    /** Template method. Marked final: subclasses cannot change the order, drop a step or insert one. */
    public final void run(String source) {
        System.out.println("Running the " + name() + " import:");
        List<String> rows = readRows(source);                // required step
        List<String> records = new ArrayList<>();
        for (String row : rows) records.add(parseRow(row));   // required step
        System.out.println("  parsed " + records.size() + " records: " + records);
        if (shouldValidate()) {                              // hook
            validate(records);
        } else {
            System.out.println("  skipping validation, source is validated upstream");
        }
        save(records);                                       // shared, fixed
        afterSave(records.size());                           // hook with an empty default
        System.out.println("  done");
    }

    // Required steps: the compiler forces every subclass to supply them.
    protected abstract String name();
    protected abstract List<String> readRows(String source);
    protected abstract String parseRow(String row);

    // Hooks: sensible defaults, overriding is optional.
    protected boolean shouldValidate() { return true; }
    protected void afterSave(int count) { }

    // Shared steps: written once, private, cannot be changed by subclasses.
    private void validate(List<String> records) {
        long blank = records.stream().filter(String::isBlank).count();
        System.out.println("  validated " + records.size() + " records, " + blank + " blank");
    }

    private void save(List<String> records) {
        System.out.println("  saved " + records.size() + " records (+ audit entry)");
    }
}
```

```java
// File: code/lld/ch11/templatemethod/CsvImportJob.java
package lld.ch11.templatemethod;

import java.util.Arrays;
import java.util.List;

/** Supplies only what differs for CSV. */
public final class CsvImportJob extends ImportJob {
    @Override protected String name() { return "CSV"; }

    @Override
    protected List<String> readRows(String source) {
        return Arrays.stream(source.split("\n")).skip(1).toList();   // skip the header line
    }

    @Override
    protected String parseRow(String row) { return row.split(",")[0].trim(); }
}
```

```java
// File: code/lld/ch11/templatemethod/JsonImportJob.java
package lld.ch11.templatemethod;

import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/** Supplies what differs for JSON, and overrides two hooks. (Toy parsing: no JSON library.) */
public final class JsonImportJob extends ImportJob {
    private static final Pattern OBJECT = Pattern.compile("\\{[^}]*}");
    private static final Pattern ID = Pattern.compile("\"id\"\\s*:\\s*\"([^\"]*)\"");

    @Override protected String name() { return "JSON"; }

    @Override
    protected List<String> readRows(String source) {
        return OBJECT.matcher(source).results().map(m -> m.group()).toList();
    }

    @Override
    protected String parseRow(String row) {
        Matcher m = ID.matcher(row);
        return m.find() ? m.group(1) : "";
    }

    @Override protected boolean shouldValidate() { return false; }

    @Override
    protected void afterSave(int count) { System.out.println("  [hook] notified upstream: " + count + " ids
consumed"); }
}
```

```java
// File: code/lld/ch11/templatemethod/CaffeineBeverage.java
package lld.ch11.templatemethod;

/** The classic beverage example: same recipe skeleton, two steps and one hook vary. */
public abstract class CaffeineBeverage {
    public final void prepareRecipe() {          // template method
        boilWater();
        brew();
        pourInCup();
        if (customerWantsCondiments()) addCondiments();
    }

    protected abstract void brew();
    protected abstract void addCondiments();

    /** Hook: "Don't call us, we'll call you" - the base class decides WHEN this runs. */
    protected boolean customerWantsCondiments() { return true; }

    private void boilWater() { System.out.println("  boiling water"); }
    private void pourInCup() { System.out.println("  pouring into cup"); }
}
```

```java
// File: code/lld/ch11/templatemethod/Tea.java
package lld.ch11.templatemethod;

public final class Tea extends CaffeineBeverage {
    @Override protected void brew() { System.out.println("  steeping the tea"); }
    @Override protected void addCondiments() { System.out.println("  adding lemon"); }
}
```

```java
// File: code/lld/ch11/templatemethod/Coffee.java
package lld.ch11.templatemethod;

public final class Coffee extends CaffeineBeverage {
    private final boolean black;

    public Coffee(boolean black) { this.black = black; }

    @Override protected void brew() { System.out.println("  dripping coffee through filter"); }
    @Override protected void addCondiments() { System.out.println("  adding sugar and milk"); }
    @Override protected boolean customerWantsCondiments() { return !black; }
}
```

```java
// File: code/lld/ch11/templatemethod/Main.java
package lld.ch11.templatemethod;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Import jobs ===");
        new CsvImportJob().run("id,name\nu1, Asha\nu2, Ben\nu3, Chen");
        new JsonImportJob().run("[{\"id\":\"p9\",\"qty\":1},{\"id\":\"p7\",\"qty\":4}]");

        System.out.println("\n=== 2. Beverages (hook controls an optional step) ===");
        System.out.println("tea:");
        new Tea().prepareRecipe();
        System.out.println("black coffee:");
        new Coffee(true).prepareRecipe();
    }
}
```

Output:

```
=== 1. Import jobs ===
Running the CSV import:
  parsed 3 records: [u1, u2, u3]
  validated 3 records, 0 blank
  saved 3 records (+ audit entry)
  done
Running the JSON import:
  parsed 2 records: [p9, p7]
  skipping validation, source is validated upstream
  saved 2 records (+ audit entry)
  [hook] notified upstream: 2 ids consumed
  done

=== 2. Beverages (hook controls an optional step) ===
tea:
  boiling water
  steeping the tea
  pouring into cup
  adding lemon
black coffee:
  boiling water
  dripping coffee through filter
  pouring into cup
```

Template Method vs Strategy

	Template Method	Strategy
What varies	**One step**; the surrounding algorithm is fixed	**The entire** algorithm
Mechanism	**Inheritance** — subclass supplies the step	**Composition** — caller passes an object in
When chosen	When the object is created (by which subclass you instantiate); fixed after that	Run time, can change per call
How many axes can vary	One (a class has one parent)	Several (hold several strategies)
What one file shows	Just the varying pieces	The complete algorithm

Strategy is usually the more flexible choice. Template Method pays off when **most of the flow is common**, only small pieces differ, and you want the compiler to **enforce the sequence**. They combine well: the base class fixes the sequence, but each step is a **strategy object** (or lambda) handed in rather than inherited — so the order is still enforced, yet you are not limited to one parent. Spring's `JdbcTemplate` (a helper around **JDBC**, Java's standard API for talking to SQL databases) works like this: it owns the fixed flow (get connection, create statement, execute, handle exceptions, close resources) and takes your varying part as a **callback** (a function or object you hand over so the other code can call you back — here `RowMapper`, `PreparedStatementSetter`) rather than a subclass.

Trade-offs

Pros	Cons
The shared flow is written once (DRY: Don't Repeat Yourself).	No single file tells the whole story: the parent holds the order, the child holds the pieces.
The order is guaranteed (`final`).	One parent only: two independent variations can't combine.
New variants are small.	Parent and children are bound tightly; altering one step's signature forces every subclass to change.
Abstract steps make the compiler report anything a subclass has not supplied.	Base classes tend to swell until they demand many abstract methods and nobody wants to extend them.

When not to use it

- **The flow is two lines** — nothing to protect.
- **Variants must combine** (an encrypted CSV import would need both an "encrypted" and a "CSV" parent) — use composition.
- **Behaviour must change after creation** — use Strategy.
- **Subclasses override every step** — the flow was never really shared.

Common mistakes

1. **Too many required steps.** A subclass should only have to write two or three methods; give the rest a default and make them hooks.
2. **Calling an overridable step from the base constructor.** The base constructor runs **before** the subclass's fields are initialised, so in Java the step sees `null`/`0`/`false`. (In C++ it is different but still wrong: during the base constructor a virtual call is dispatched to the base version, never the subclass override.) Call steps from the template method, never the constructor.
3. **Leaving out `final` on the template method** — then a subclass can reorder or skip steps; nothing is enforced any more and the pattern becomes a mere agreement. (C++: make it non-virtual. Python and JavaScript cannot enforce it at all, so it must be documented and checked in review.)
4. **Making steps `public`** — callers could run step 3 without steps 1 and 2. Keep them `protected` (or move reusable steps into a separate object).
5. **Violating Liskov** (the Liskov Substitution Principle: a subclass must work anywhere its parent is expected) in overrides (a hook that throws, or a step that changes the contract).

Template Method in other languages

- **Java:** `final` template method, `protected abstract` required steps, `protected` hooks with a body. An interface `default` method can also act as a template method, but an interface has no instance state and a default method cannot be `final`, so any implementer may override it.
- **Python:** `abc.ABC` with `@abstractmethod`. There is no `final` enforced at run time, so a comment (or `typing.final` for type checkers) is the only guard; a leading underscore marks internal steps.
- **TypeScript:** `abstract` classes give the compile-time check; plain JavaScript uses base methods that throw.
- **C++:** a non-virtual public template method calling `protected virtual` steps — the **non-virtual interface (NVI)** idiom.
- **Go and Rust** have no inheritance: a function taking an interface/trait with the varying steps — i.e. the Strategy shape.

Real-world examples

- **`java.io.InputStream`**: `read(byte[], int, int)` is implemented in terms of the abstract `read()`; subclasses supply the single-byte read (and override the bulk method for speed).
- **`java.util.AbstractList`**: implement `get(int)` and `size()`, and you inherit `iterator()`, `indexOf`, `contains`, `equals`, `hashCode`...
- **`HttpServlet.service()`** (Jakarta/Java Servlet) dispatches to `doGet`/`doPost`/`doPut`... by HTTP method; you override the ones you support, and the ones you do not override reject the request (405 Method Not Allowed on HTTP/1.1).
- **`AbstractQueuedSynchronizer`** (AQS — the JDK's internal building block for locks; behind `ReentrantLock`, `Semaphore`, `CountDownLatch`): `acquire()` is the template; subclasses implement `tryAcquire`/`tryRelease`.
- **Test frameworks**: JUnit (`@BeforeEach`/`@AfterEach`) and pytest (fixtures, `setup_method`/`teardown_method`) own the set-up → test → tear-down flow; your hooks never call each other. **Mobile lifecycles**: Android `Activity` (`onCreate`, `onStart`, `onResume`...) and iOS `UIViewController` (`viewDidLoad`, `viewWillAppear`...). **Maven/Gradle** (Java build tools) build lifecycles (fixed phases, you plug in goals/tasks).
- **Sorting with a comparison**: `Arrays.sort` owns the algorithm and needs just one step from you, the comparison. Old designs had the elements supply it (`Comparable.compareTo`); modern APIs pass it in as a `Comparator` — the Strategy form of the same idea.
- **Spring `JdbcTemplate`, `RestTemplate`, `TransactionTemplate`** — "template + callback" variants of the same idea.

In the interview

> **30-second answer:** "Template Method puts an algorithm's outline in a single final method of a base class, and subclasses supply only the steps that differ. Steps a subclass must write are abstract; steps it may change are hooks with a default body; steps nobody should change are private. It's the Hollywood principle — the base class calls the subclass, not the other way round — which is how frameworks like servlets, JUnit and Android lifecycles work. It varies one step via inheritance; if I need run-time choice or several axes I prefer Strategy, or pass the steps in as callbacks like JdbcTemplate does."

Follow-ups:

- *What is a hook?* A step with a default implementation that subclasses may override (e.g. `shouldValidate()`), giving optional control points.
- *What is the Hollywood Principle?* "Don't call us, we'll call you": high-level components call low-level ones at the right time; low-level components plug in but don't call up. It prevents "dependency rot" (dependencies tangled in every direction, so nothing can change safely).
- *Why `final`?* It is the guarantee: without it a subclass could reorder or drop steps, and the pattern would be only a gentleman's agreement.
- *Template Method vs Factory Method?* A factory method is often one step of a template method — the step whose job is to create an object (GoF notes that template methods frequently call factory methods). The course frames it more strongly — a factory method is a special template-method step whose subclass-supplied job is creation, while most template steps create nothing. Take that as "a factory method usually lives inside a template method", not as a strict rule: a factory method can also be called directly by clients.
- *Template Method vs Decorator?* A decorator wraps an existing object at run time, and several can be layered; a template-method subclass is decided when the object is built and cannot be layered.

- *Template Method vs Builder?* Both impose an order, but a Builder director invokes the steps on a **different** object (the builder it holds), while a template method invokes them on **its own** instance via inheritance.
- *Constructor trap?* Never call overridable methods from a constructor; the subclass isn't initialised yet.

Self-check

1. **Q:** Which method is `final`? **A:** The template method (`run`, `prepareRecipe`).
2. **Q:** Difference between an abstract step and a hook? **A:** Abstract steps must be implemented; hooks have defaults and are optional.
3. **Q:** Name two JDK template methods. **A:** `InputStream.read(byte[],int,int)`, `AbstractList`'s methods built on `get`/`size`, AQS `acquire`.
4. **Q:** Why is `JdbcTemplate` more flexible than a classic template method? **A:** It takes the varying step as a callback object (composition), not a subclass.
5. **Q:** What goes wrong if the base constructor calls an abstract step? **A:** The subclass fields aren't initialised yet.

TL;DR

Fixed order in a `final` base method; subclasses fill steps; hooks are optional steps. Hollywood principle. Inheritance-based and compile-time — prefer Strategy/callbacks when you need flexibility.

11.5 Visitor

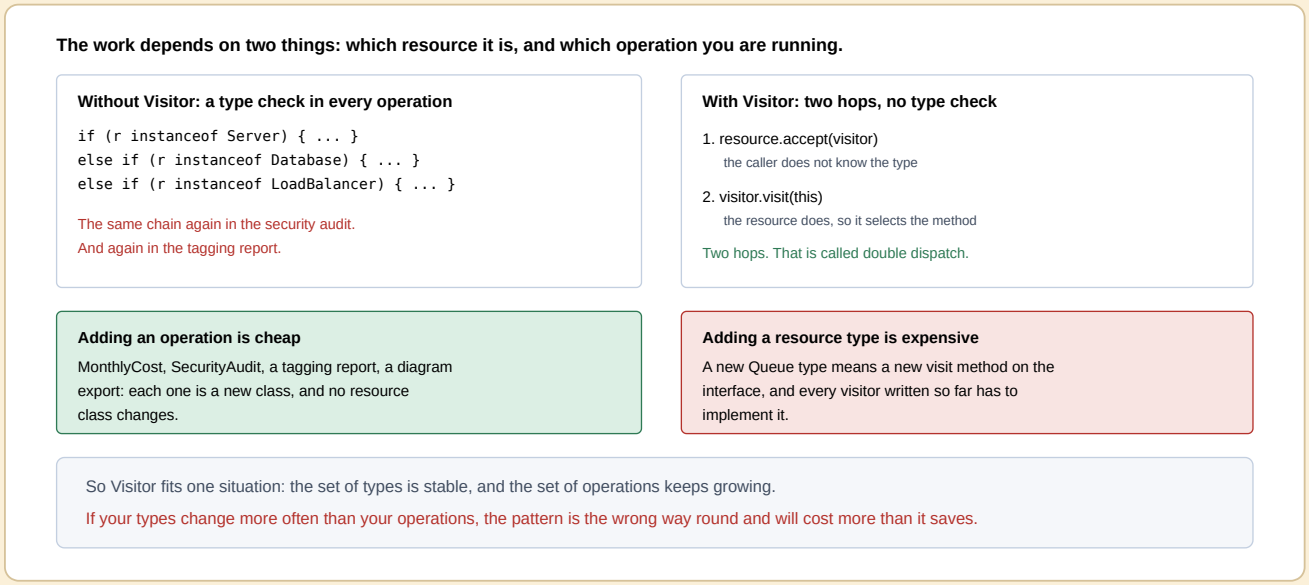


Figure — Without Visitor, every operation repeats an instanceof chain; with Visitor, two calls (accept, then visit) select the right code with no type check (course diagram)

What it is

Visitor lets you **introduce new operations** over a group of classes **while leaving those classes untouched**. Every operation lives in its own **visitor** class, which has a separate method for each kind of element; each element keeps one tiny method, `accept(visitor)`, whose body is `visitor.visit(this)`.

GoF intent (paraphrased): represent an operation to be performed on the elements of an object structure as a separate object, so you can define a new operation without changing the classes of the elements it works on.

GoF participants (and who plays them in our code):

Participant	Role	In `lld.ch11.visitor`
Visitor	Declares one `visit` operation per ConcreteElement class	`PlanVisitor`
ConcreteVisitor	Implements every `visit`; one class per operation, often accumulating a result	`MonthlyCost`, `SecurityAudit`
Element	Declares `accept(visitor)`	`Resource`
ConcreteElement	Implements `accept` as `visitor.visit(this)`	`Server`, `Database`, `LoadBalancer`, `ResourceGroup`
ObjectStructure	Holds the elements and lets a visitor reach each of them (a list or a Composite)	the `ResourceGroup` tree built in `Main`

Everyday analogy: a tax inspector visiting different businesses. The businesses don't each contain tax rules; the inspector brings them and applies the right rule for "restaurant", "shop" or "factory". Next year, a fire-safety inspector visits the same businesses with different rules — the businesses don't change.

The problem it solves

Picture an infrastructure plan made of just three resource kinds — server, database, load balancer. That **list of kinds is stable**. The **operations never stop arriving**: a monthly bill, a security review, a tag report, a diagram export, and so on.

- Put each operation **inside** the resource classes → `Server` holds pricing, security, tagging and drawing code; unrelated teams edit the same file.
- Put it **outside** with type checks → an `instanceof` chain in every operation. Add a resource kind, and you must find every chain; the compiler won't tell you which one you missed.

Visitor gives each operation its own class, and uses **double dispatch** to pick the right method without any type check.

How it works — double dispatch

Java chooses **which override** (a subclass's replacement of an inherited method) of a method runs based on the **run-time type of the receiver** — the object before the dot (single dispatch) — but chooses **which overload** (one of several same-named methods with different parameter types: `visit(Server)` vs `visit(Database)`) at **compile time**, from the **static type** (the type the compiler sees in the declaration) of the argument. If you hold a `Resource r`, `visitor.visit(r)` cannot compile to the right overload — the compiler only knows "Resource".

Visitor makes **two calls**:

1. `resource.accept(visitor)` — dispatch #1 on the **resource's** run-time type: `Server.accept` runs.
2. Inside `Server.accept`, `visitor.visit(this)` — here `this` has static type `Server`, so the compiler picks `visit(Server)`; dispatch #2 then happens on the **visitor's** run-time type (`MonthlyCost` or `SecurityAudit`).

The method that finally runs depends on **two** types — element and visitor. That is **double dispatch**.

Structure (UML)

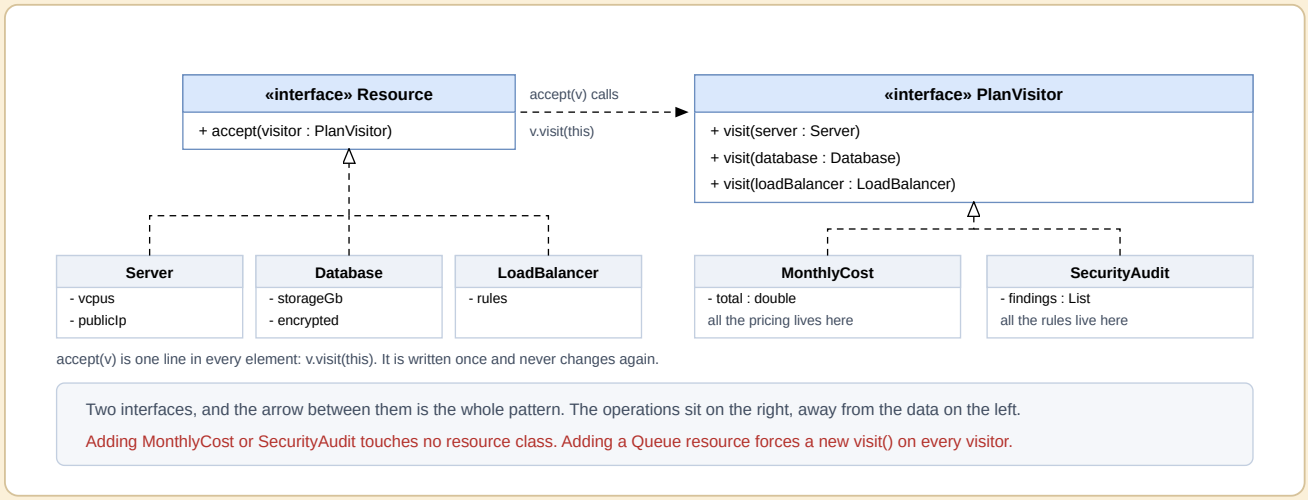
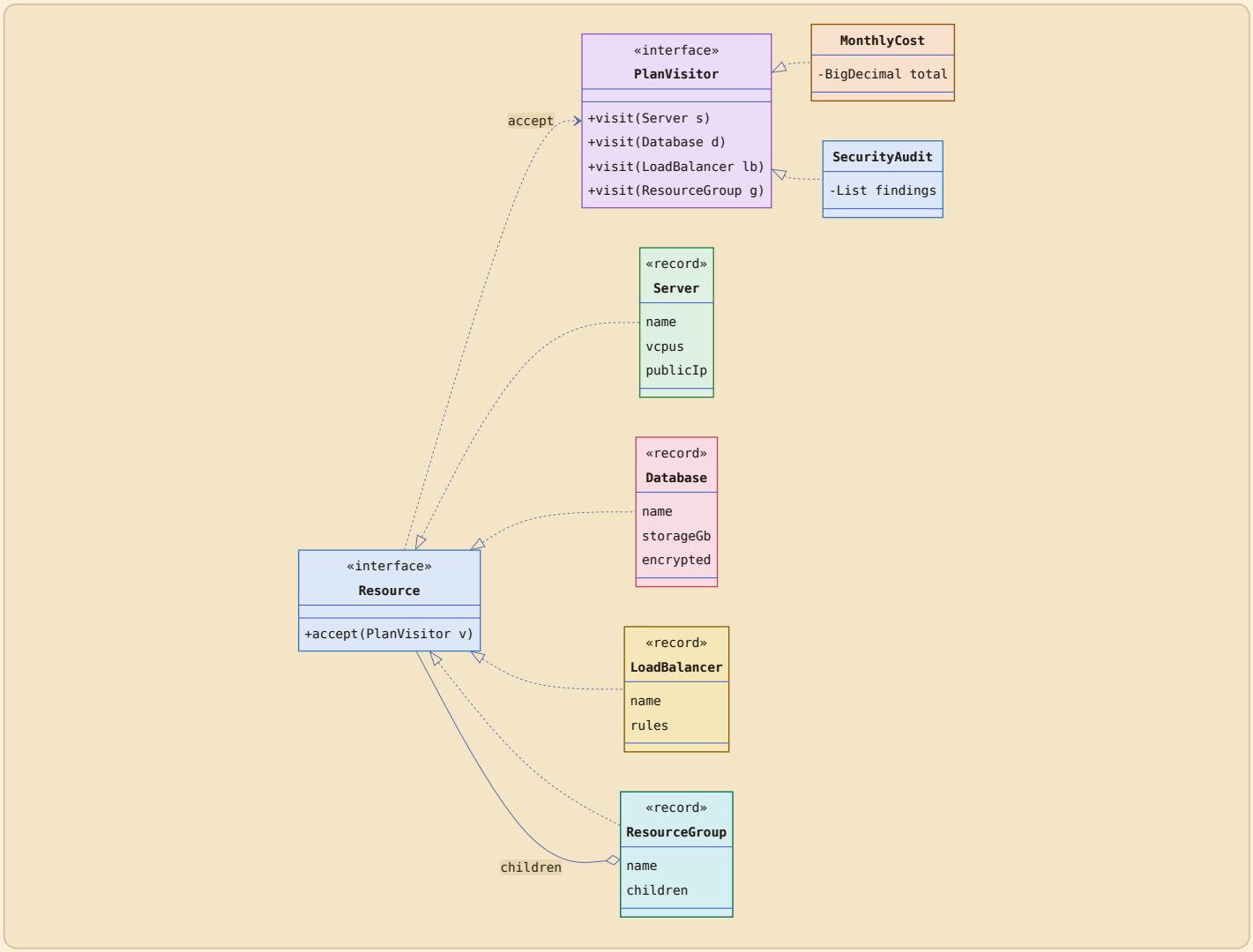
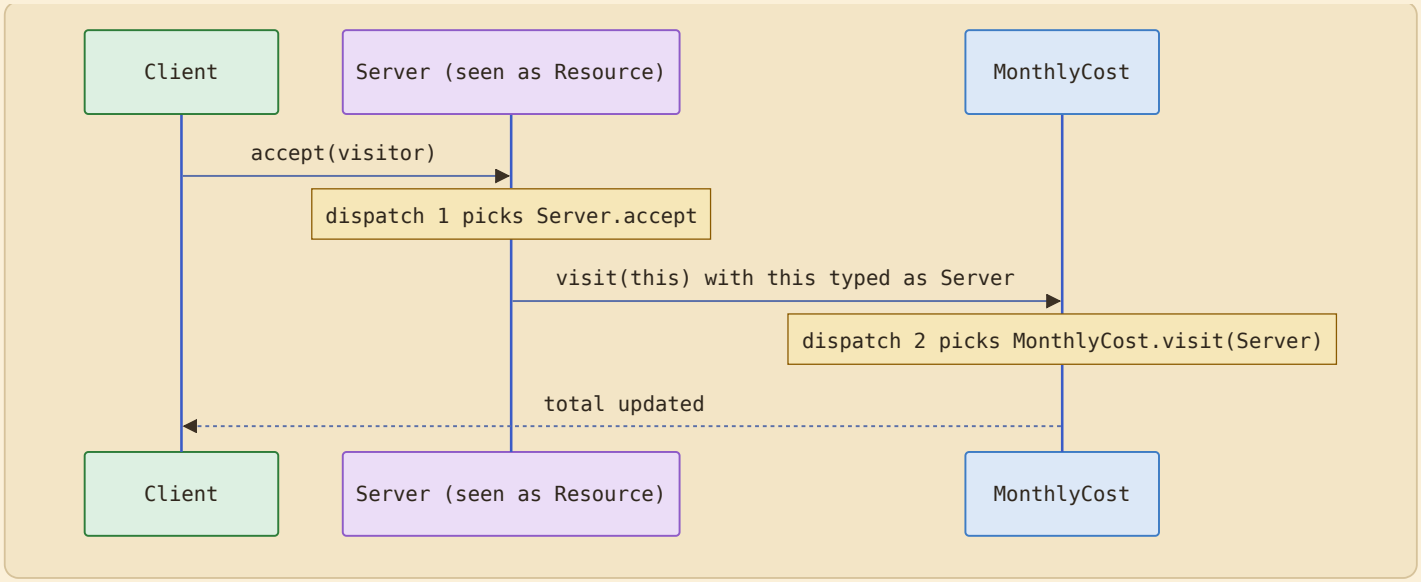


Figure — Class diagram: Resource with accept; Server, Database, LoadBalancer implement it; PlanVisitor has one visit per resource type; MonthlyCost and SecurityAudit implement it (course diagram; it shows `total : double`, our Java code uses `BigDecimal` for money)

In the diagram above the two sets of arrows point in opposite directions: each element depends on the visitor interface (its `accept` has a visitor parameter), while each visitor depends on every element type (it has a `visit` for each). That second arrow is why **adding an element type is expensive**.

Our Java design (with a Composite group element):





Java example

`lld.ch11.visitor` has **both** styles over the same domain:

1. **Classic Visitor.** Elements are **records** (so visitors read data through accessors, not public fields). `ResourceGroup` is a **Composite** (the pattern in which a group and a single item share one interface, so a tree can be handled uniformly): its `accept` visits itself and then its children. (A tree is optional: Visitor only requires a stable set of element types, and it works just as well over a flat list. Composite is simply the most common route into Visitor.) Prices are `BigDecimal`, never `double`. `MonthlyCost` and `SecurityAudit` each keep all their rules in one file; neither touches a resource class. A new visitor instance is used per run because they accumulate results.
2. **Java 21 alternative — sealed interface + pattern matching** (`ModernPlan`). `Infra` is `sealed ... permits Vm, Sql, Lb`. `monthlyCost` is a `switch` with **no** **`default`** — the compiler checks it is **exhaustive** (covers every possible case), so adding a new permitted record breaks every switch that forgot it (the same safety net Visitor gave you). `audit` uses **record patterns** (deconstruction, `case Sql(String n, int gb, boolean enc)`) and **guards** (`when !enc`). A guarded case never counts toward exhaustiveness (the compiler cannot know the guard is sometimes true), which is why `audit` also needs the three unguarded `case Vm vm` / `Sql db` / `Lb lb` lines.

```java
// File: code/lld/ch11/visitor/Resource.java
package lld.ch11.visitor;

/** ELEMENT interface: the only thing every resource must provide is accept. */
public interface Resource {
    void accept(PlanVisitor visitor);
}
```

```java
// File: code/lld/ch11/visitor/PlanVisitor.java
package lld.ch11.visitor;

/** VISITOR interface: one visit per element type. This list is what makes new types expensive. */
public interface PlanVisitor {
    void visit(Server server);
    void visit(Database database);
    void visit(LoadBalancer loadBalancer);
    void visit(ResourceGroup group);
}
```

```java
// File: code/lld/ch11/visitor/Server.java
package lld.ch11.visitor;

/** Records expose data through accessors, so visitors need no public fields. */
public record Server(String name, int vcpus, boolean publicIp) implements Resource {
    public void accept(PlanVisitor v) { v.visit(this); }   // 2nd dispatch: overload picked by static type Server
}
```

```java
// File: code/lld/ch11/visitor/Database.java
package lld.ch11.visitor;

public record Database(String name, int storageGb, boolean encrypted) implements Resource {
    public void accept(PlanVisitor v) { v.visit(this); }
}
```

```java
// File: code/lld/ch11/visitor/LoadBalancer.java
package lld.ch11.visitor;

public record LoadBalancer(String name, int rules) implements Resource {
    public void accept(PlanVisitor v) { v.visit(this); }
}
```

```java
// File: code/lld/ch11/visitor/ResourceGroup.java
package lld.ch11.visitor;

import java.util.List;

/** COMPOSITE element: visits itself, then passes the visitor to every child. */
public record ResourceGroup(String name, List<Resource> children) implements Resource {
    public void accept(PlanVisitor v) {
        v.visit(this);
        for (Resource child : children) child.accept(v);
    }
}
```

```java
// File: code/lld/ch11/visitor/MonthlyCost.java
package lld.ch11.visitor;

import java.math.BigDecimal;

/**
 * Operation 1: ALL pricing rules in one file. New instance per run (it accumulates).
 * Money is BigDecimal: exact decimal arithmetic, no binary rounding surprises.
 */
public final class MonthlyCost implements PlanVisitor {
    static final BigDecimal PER_VCPU = new BigDecimal("24.00");
    static final BigDecimal DB_BASE = new BigDecimal("40.00");
    static final BigDecimal PER_GB = new BigDecimal("0.25");
    static final BigDecimal LB_BASE = new BigDecimal("18.00");
    static final BigDecimal PER_RULE = new BigDecimal("1.50");

    private BigDecimal total = BigDecimal.ZERO;

    public BigDecimal total() { return total; }

    public void visit(Server s)        { total = total.add(PER_VCPU.multiply(BigDecimal.valueOf(s.vcpus()))); }
    public void visit(Database d)      { total =
total.add(DB_BASE.add(PER_GB.multiply(BigDecimal.valueOf(d.storageGb())))); }
    public void visit(LoadBalancer lb) { total =
total.add(LB_BASE.add(PER_RULE.multiply(BigDecimal.valueOf(lb.rules())))); }
    public void visit(ResourceGroup g) { /* groups are free */ }
}
```

```java
// File: code/lld/ch11/visitor/SecurityAudit.java
package lld.ch11.visitor;

import java.util.ArrayList;
import java.util.List;

/** Operation 2: added later without touching a single resource class. */
public final class SecurityAudit implements PlanVisitor {
    private final List<String> findings = new ArrayList<>();

    public List<String> findings() { return findings; }

    public void visit(Server s) {
        if (s.publicIp()) findings.add(s.name() + ": server has a public IP");
    }
    public void visit(Database d) {
        if (!d.encrypted()) findings.add(d.name() + ": storage not encrypted");
    }
    public void visit(LoadBalancer lb) {
        if (lb.rules() == 0) findings.add(lb.name() + ": load balancer has no rules");
    }
    public void visit(ResourceGroup g) {
        if (g.children().isEmpty()) findings.add(g.name() + ": empty group");
    }
}
```

```java
// File: code/lld/ch11/visitor/ModernPlan.java
package lld.ch11.visitor;

import java.math.BigDecimal;
import java.util.Optional;

/**
 * The Java 21 alternative: a SEALED hierarchy + pattern-matching switch.
 * No accept(), no visitor interface. The compiler still checks exhaustiveness:
 * add a new record to 'permits' and every switch without it stops compiling.
 */
public final class ModernPlan {
    private ModernPlan() {}

    public sealed interface Infra permits Vm, Sql, Lb {}
    public record Vm(String name, int vcpus, boolean publicIp) implements Infra {}
    public record Sql(String name, int storageGb, boolean encrypted) implements Infra {}
    public record Lb(String name, int rules) implements Infra {}

    /** Operation 1 as a plain function. No 'default' branch, on purpose. Same prices as MonthlyCost. */
    public static BigDecimal monthlyCost(Infra r) {
        return switch (r) {
            case Vm vm -> MonthlyCost.PER_VCPU.multiply(BigDecimal.valueOf(vm.vcpus()));
            case Sql db ->
MonthlyCost.DB_BASE.add(MonthlyCost.PER_GB.multiply(BigDecimal.valueOf(db.storageGb())));
            case Lb lb -> MonthlyCost.LB_BASE.add(MonthlyCost.PER_RULE.multiply(BigDecimal.valueOf(lb.rules())));
        };
    }

    /** Operation 2 with record deconstruction patterns and guards. */
    public static Optional<String> audit(Infra r) {
        return switch (r) {
            case Vm(String n, int cpus, boolean pub) when pub -> Optional.of(n + ": server has a public IP");
            case Sql(String n, int gb, boolean enc) when !enc -> Optional.of(n + ": storage not encrypted");
            case Lb(String n, int rules) when rules == 0 -> Optional.of(n + ": load balancer has no rules");
            case Vm vm -> Optional.empty();     // no finding
            case Sql db -> Optional.empty();
            case Lb lb -> Optional.empty();
        };
    }
}
```

```java
// File: code/lld/ch11/visitor/Main.java
package lld.ch11.visitor;

import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Classic Visitor (double dispatch) over a composite plan ===");
        Resource plan = new ResourceGroup("prod", List.of(
            new Server("web-1", 4, true),
            new Server("worker-1", 2, false),
            new Database("orders-db", 200, false),
            new LoadBalancer("edge-lb", 4),
            new ResourceGroup("staging", List.of())));

        MonthlyCost cost = new MonthlyCost();
        plan.accept(cost);
        System.out.println("  monthly cost: " + cost.total());

        SecurityAudit audit = new SecurityAudit();
        plan.accept(audit);
        audit.findings().forEach(f -> System.out.println("  finding: " + f));

        System.out.println("\n=== 2. Java 21: sealed interface + pattern matching switch ===");
        List<ModernPlan.Infra> infra = List.of(
            new ModernPlan.Vm("web-1", 4, true), new ModernPlan.Vm("worker-1", 2, false),
            new ModernPlan.Sql("orders-db", 200, false), new ModernPlan.Lb("edge-lb", 4));
        BigDecimal total = infra.stream().map(ModernPlan::monthlyCost).reduce(BigDecimal.ZERO, BigDecimal::add);
        System.out.println("  monthly cost: " + total);
        infra.stream().map(ModernPlan::audit).flatMap(Optional::stream)
             .forEach(f -> System.out.println("  finding: " + f));
    }
}
```

Output:

```
=== 1. Classic Visitor (double dispatch) over a composite plan ===
  monthly cost: 258.00
  finding: web-1: server has a public IP
  finding: orders-db: storage not encrypted
  finding: staging: empty group

=== 2. Java 21: sealed interface + pattern matching switch ===
  monthly cost: 258.00
  finding: web-1: server has a public IP
  finding: orders-db: storage not encrypted
```

Both styles give the same answer (cost 96.00 + 48.00 + 90.00 + 24.00 = 258.00; the empty `staging` group is free). The modern one needs no `accept`, no visitor interface, and each operation is just a function.

The one question that decides it: the expression problem

	Add a new operation	**Add a new type**
Methods on the classes (plain OO)	Edit every class	One new class
Visitor (or sealed + switch)	One new visitor/function	Edit the visitor interface **and every visitor** (compiler finds them)

So ask: **which changes more often here — the types or the operations?** Settled types + growing operations → Visitor (or sealed types with pattern matching). Growing types → put the operations on the classes. No ordinary OO design makes both cheap; this trade-off is called the **expression problem**.

Open/Closed trade-off: Visitor makes the element hierarchy **closed for modification but open for new operations** — at the price of being **closed to new element types** (each one forces edits to all visitors).

Sealed interfaces + pattern matching (Java 17/21) as the modern alternative: sealed classes arrived in Java 17 (JEP 409 — a JEP, JDK Enhancement Proposal, is the numbered design document for a Java feature); pattern matching for `switch` and record patterns were finalised in Java 21 (JEPs 441 and 440). Together they give the exhaustiveness check that was Visitor's main advantage, with far less code. The exact rules:

- A `switch` **expression** must always be exhaustive. A `switch` **statement** must be exhaustive too as soon as it uses patterns or `case null` (old-style statements over constants need not be).
- Over a `sealed` type, covering every permitted subtype with an **unguarded** pattern makes the switch exhaustive with no `default`. Guarded cases (`when ...`) do not count.
- Over a type that is **not** sealed, the compiler cannot know all subtypes, so you need a `default` (or a total pattern such as `case Infra i`) — and then the check is gone.
- If a new subtype appears at run time without recompiling the switch (separate compilation), the compiler-generated fallback throws `MatchException`.
- **Dominance:** cases are tried top to bottom, and a case that can never match because an earlier one already covers it is a compile error. That is why the guarded `case Vm(...) when pub` must come **before** the unguarded `case Vm vm`, never after.
- **`null`:** a pattern `switch` without a `case null` throws `NullPointerException` when the selector is `null`; `default` alone does not match `null`.

Write a classic Visitor when the element hierarchy **is not and cannot be sealed** — typically because the element classes **belong to another team or library** and you cannot edit them — but already exposes `accept` (typical of libraries such as ASM — a library for reading and rewriting Java **bytecode**, the compiled instructions the JVM (Java Virtual Machine) runs — or javac's tree API, which ship their own visitor interfaces), when you need a **walk with per-node callbacks** over a big tree, or when each operation should be a separately testable, injectable class.

Visitor outside Java. Kotlin: a sealed class with an exhaustive `when`. Python: `match` with class patterns, or `functools.singledispatch` (dispatch on the argument type with no `accept`). C++: `std::variant` + `std::visit` — the pattern as a library feature. Rust and Swift: enums with associated values plus `match`/`switch`, where a missing case is a build error. Languages without overloading (Python, JavaScript) name the methods `visitServer`, `visitDatabase`, ... instead of overloading `visit`.

Trade-offs

Pros	Cons
Adding an operation = writing one class; elements stay as they are.	Adding an element type forces a change in every existing visitor.
Related logic stays together (all pricing in one file).	Double dispatch is indirect; `accept` tells you nothing about what runs.
The compiler reports any element type a visitor has not handled.	Elements often must expose their data (weaker encapsulation).
Elements stay small: data + one-line `accept`.	Ceremony for small sets of types.

When not to use it

- **New types arrive often** — every one edits every visitor.
- **A tiny model** (say three element types and a single operation) — a type check or a plain method is simpler.
- **The operation concerns one type only** — it's a method.
- **Your language checks switch exhaustiveness** (Java 21 sealed types, Kotlin `when`, Rust `match`) — use that.

Common mistakes

1. Reaching for Visitor with two element types.
2. Making element fields **public** for visitors (use accessors/records).
3. **Reusing a stateful visitor** (like `MonthlyCost`) across runs without resetting — create a new one per run.
4. Adding a `default` branch to a sealed-type switch — it **turns off** the exhaustiveness check you wanted.

5. Forgetting that the composite's `accept` must forward to children (or deciding where traversal lives: element, visitor, or an Iterator).
6. **Writing a visitor where a sealed-type `switch` would do.** If your language checks exhaustiveness (Java 21, Kotlin, Rust, Swift), that gives the same safety with a fraction of the code.

Real-world examples

- **Compilers and linters** (tools that flag suspicious code): an **AST** (abstract syntax tree — the tree a compiler builds from source code) has a fixed set of node kinds and endless passes (type check, optimise, format, lint rules). The JDK's `com.sun.source.tree.TreeVisitor` (javac tree API) and `javax.lang.model.element.ElementVisitor` (annotation processing) are visitors; ANTLR (a parser generator) generates visitor classes for your grammar.
- **Bytecode libraries**: ASM's `ClassVisitor` / `MethodVisitor`.
- **`java.nio.file.FileVisitor`** with `Files.walkFileTree` — named after the pattern and often cited, but strictly it is a **tree-walk callback** interface (`preVisitDirectory`, `visitFile`, `visitFileFailed`, `postVisitDirectory`): there is no `accept` and no double dispatch over element classes. The *spirit* (operation kept outside the file-system objects) is Visitor's; the mechanism is closer to a Template Method walk calling your hooks.
- **Document models**: a **DOM** (Document Object Model — the tree of elements that makes up a web page or XML document) has a fixed set of node kinds and ever more operations — serialise to HTML/PDF, pull out the plain text, collect all links; **infrastructure tools** computing cost, policy and diagrams over resource plans (this lesson's example); **query planners** (the database component that decides how to run a SQL query: cost estimation, rewriting, EXPLAIN).

In the interview

> **30-second answer:** "Visitor pulls an operation out of the element classes and into a separate class that has a method for each element type. Each element has a one-line `accept` that calls `visitor.visit(this)`, which gives double dispatch: the method chosen depends on both the element type and the visitor type, so no `instanceof` chains. It makes adding operations cheap and adding types expensive — the expression problem — so it fits stable type sets like ASTs. In Java 21 I'd usually use a sealed interface with an exhaustive pattern-matching switch instead, which gives the same compiler check with much less code."

Follow-ups:

- *Explain double dispatch.* Java picks overloads at compile time from static types; `accept` runs on the element's run-time type, and inside it `this` has the concrete static type, so the right `visit` overload is chosen; then the visitor's run-time type picks the implementation.
- *Why not just overload `visit` and call `visitor.visit(resource)`?* With a `Resource` static type, the compiler can't choose `visit(Server)`; it needs `visit(Resource)`, which doesn't exist.
- *Visitor vs Iterator?* Iterator decides the route through the structure; Visitor decides the work done at each stop. Real code often combines them.
- *Visitor vs Strategy?* A strategy is a single algorithm that ignores element types; a visitor is a set of methods, one for each element type, and exists precisely to dispatch on type.
- *Visitor vs Composite?* Composite lets a client treat a whole tree and a single node alike; Visitor adds new operations over a stable set of types, whether or not they form a tree. The two frequently appear together.
- *Visitor vs Interpreter?* Interpreter keeps evaluation **in** each node class; Visitor moves it **out**. Many interpreters begin in the first style and switch to the second once another pass (type checking, pretty-printing) is needed.
- *How do you add a new element type?* Add it to the interface's `visit` list (or the `permits` clause) and fix every visitor/switch the compiler flags.

Self-check

1. **Q:** What is the body of every `accept` method? **A:** `visitor.visit(this)`.
2. **Q:** What change is cheap with Visitor, and what is expensive? **A:** New operations are cheap; new element types are expensive.
3. **Q:** What Java 21 features replace most visitors? **A:** Sealed interfaces + pattern-matching `switch` (with record patterns), checked for exhaustiveness.
4. **Q:** Why no `default` in `monthlyCost`'s switch? **A:** A `default` would hide missing cases; without it the compiler forces every permitted type to be handled.

5. **Q:** Where does the traversal live for a composite? **A:** Here, in the composite's `accept` (it visits itself, then its children).

> **TL;DR**

Operations become visitor classes; elements just `accept`; double dispatch picks the method. Cheap new operations, expensive new types (expression problem). In Java 21, prefer **sealed interfaces + exhaustive pattern matching**.

11.6 GoF behavioral patterns: quick-reference summary

A single table with the key features, when to apply each pattern, an example, and pros and cons (compiled from the course summary plus this book's material).

Pattern	Distinctive feature	Apply when	Example	Pros	Cons
Chain of Responsibility	Passes a request along a chain of handlers	Several objects may handle a request and the handler isn't known in advance	Help systems, HTTP middleware (code run before/after each web request)	Decouples sender and receiver	A request may go unhandled; long chains can be inefficient and slow to debug
Command	Wraps a request as an object	Actions must be queued, logged, retried or undone	GUI actions, job queues	Decouples invoker from receiver; undo/redo	Many small command classes
Interpreter	One class per grammar rule	Evaluating sentences of a small language	Regex, rule DSLs (small domain-specific languages)	Easy to extend the grammar	Unmanageable for large grammars
Iterator	Sequential access without exposing representation	You need a standard way to traverse any collection	Browsing a playlist	Simple client code; hides the structure	Hidden costs; state handling in lazy iterators
Mediator	Centralises communication between peers	Many objects interact in tangled ways	Chat application, air traffic control	Fewer dependencies; reusable components	The mediator can become a god object (one class that knows and does too much)
Memento	Captures and externalises state without breaking encapsulation	Undo, checkpoints, rollback	Editor undo, save games	Preserves encapsulation; atomic restore	Memory-heavy for big/frequent snapshots
Observer	Notifies dependents of state changes	A change in one object must reach others	Live stock/data feeds	Subject and observers are loosely coupled	Leaks, unexpected performance costs, no ordering
State	Behaviour changes with internal state	An object's behaviour depends on a state that changes at run time	Game modes, order lifecycles	Localises state-specific behaviour	More classes; graph spread across files
Strategy	Algorithm chosen at run time	Several ways to do one task, chosen dynamically	Compression algorithms, retry policies	Swap algorithms; isolate each	More classes; someone must choose
Template Method	Algorithm skeleton in a base class, steps in subclasses	The structure is fixed but some steps vary	Frameworks' lifecycles	Code reuse; enforced order	Inheritance rigidity; harder to follow
Visitor	Separates operations from the objects they work on	Many operations over a stable set of types	Processing document elements, compiler passes	Easy to add operations; related code together	Adding element types is hard; bloated visitor interface

11.7 Beyond GoF: patterns every LLD interview expects

The GoF book (1994) is not the whole story. The patterns below come from later sources — *Patterns of Enterprise Application Architecture* (Fowler, 2002), *Domain-Driven Design* (Evans, 2003 — DDD, an approach that shapes code around the business domain and its vocabulary), *Head First Design Patterns*, and decades of framework practice. You will use them in almost every case study in this book.

11.7.1 Null Object

What it is

A **Null Object** is a real object that implements an interface with **neutral, do-nothing behaviour**, used instead of `null` to mean "no object here". Callers can call it safely without `if (x != null)` checks.

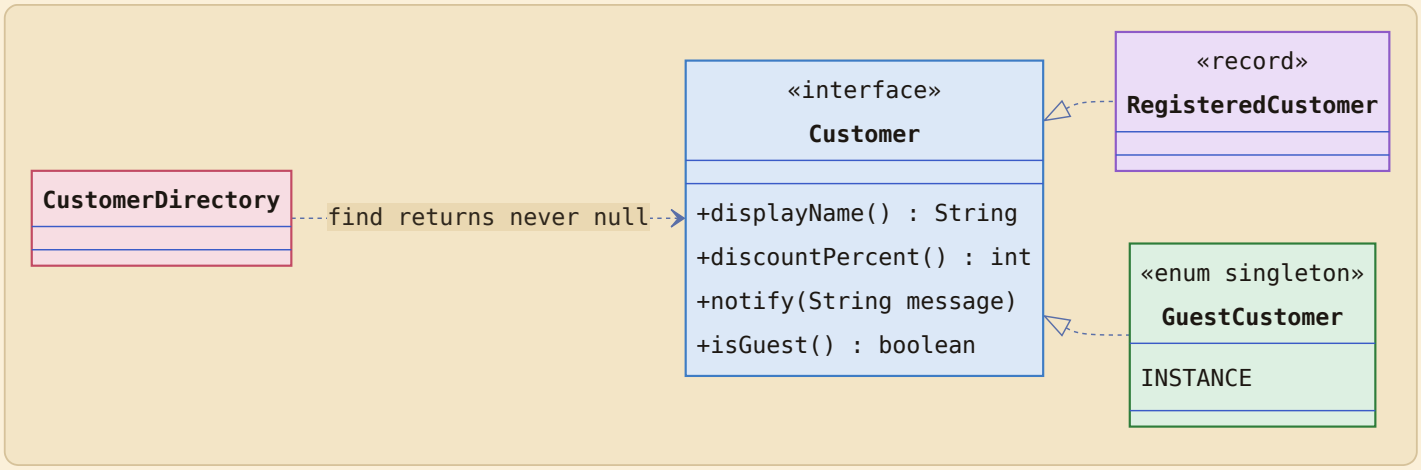
Analogy: a "guest" badge at an office. Guests can walk through the lobby like anyone else; the badge simply opens no special doors.

The problem it solves

Returning `null` for "not found" forces **every caller** to remember a null check. One forgotten check → `NullPointerException` (NPE — the error Java throws when you call a method on `null`) in production. The checks also clutter the business logic.

How it works

1. Define an interface (`Customer`).
2. Implement the real class (`RegisteredCustomer`) and a **null class** (`GuestCustomer`) whose methods return neutral values (0% discount) or do nothing (`notify` is empty).
3. Where you would return `null`, return the null object instead. Stateless null objects can be a **singleton** (a class with exactly one shared instance — here an `enum` constant).
4. If some caller really must know, offer an `isGuest()`/`isNull()` query — sparingly.



Java example

```java
// File: code/lld/ch11/nullobject/Customer.java
package lld.ch11.nullobject;

/** The abstraction both the real object and the Null Object implement. */
public interface Customer {
    String displayName();
    int discountPercent();
    void notify(String message);
    default boolean isGuest() { return false; }
}
```

```java
// File: code/lld/ch11/nullobject/RegisteredCustomer.java
package lld.ch11.nullobject;

public record RegisteredCustomer(String name, int discountPercent, String email) implements Customer {
    public String displayName() { return name; }
    public void notify(String message) { System.out.println("    email to " + email + ": " + message); }
}
```

```java
// File: code/lld/ch11/nullobject/GuestCustomer.java
package lld.ch11.nullobject;

/**
 * NULL OBJECT: a real, do-nothing implementation with neutral values.
 * Stateless, so one shared instance is enough (an enum singleton).
 */
public enum GuestCustomer implements Customer {
    INSTANCE;

    public String displayName() { return "guest"; }
    public int discountPercent() { return 0; }            // neutral value
    public void notify(String message) { }                // does nothing, on purpose
    @Override public boolean isGuest() { return true; }   // for the rare caller that must know
}
```

```java
// File: code/lld/ch11/nullobject/CustomerDirectory.java
package lld.ch11.nullobject;

import java.util.Map;
import java.util.Optional;

public final class CustomerDirectory {
    private final Map<String, Customer> byId = Map.of(
        "c1", new RegisteredCustomer("Asha", 10, "asha@example.com"));

    /** Style 1: never return null; return the Null Object instead. */
    public Customer find(String id) { return byId.getOrDefault(id, GuestCustomer.INSTANCE); }

    /** Style 2: make absence explicit in the type; the caller chooses the fallback. */
    public Optional<Customer> lookup(String id) { return Optional.ofNullable(byId.get(id)); }
}
```

```java
// File: code/lld/ch11/nullobject/Main.java
package lld.ch11.nullobject;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class Main {
    private static final BigDecimal HUNDRED = BigDecimal.valueOf(100);

    static void checkout(Customer c, BigDecimal price) {
        // No "if (customer != null)" anywhere: the Null Object answers every call safely.
        BigDecimal payable = price.multiply(BigDecimal.valueOf(100 - c.discountPercent()))
                                  .divide(HUNDRED, 2, RoundingMode.HALF_EVEN);
        System.out.printf("  %-6s pays %s%n", c.displayName(), payable);
        c.notify("thanks for your order");
    }

    public static void main(String[] args) {
        CustomerDirectory dir = new CustomerDirectory();
        System.out.println("=== Null Object ===");
        checkout(dir.find("c1"), new BigDecimal("100.00"));
        checkout(dir.find("unknown-id"), new BigDecimal("100.00"));

        System.out.println("=== Optional (absence visible in the signature) ===");
        Customer c = dir.lookup("unknown-id").orElse(GuestCustomer.INSTANCE);
        System.out.println("  lookup(unknown-id) present? " + dir.lookup("unknown-id").isPresent() + " -> using " +
c.displayName());
    }
}
```

Output:

```
=== Null Object ===
  Asha   pays 90.00
    email to asha@example.com: thanks for your order
  guest  pays 100.00
=== Optional (absence visible in the signature) ===
  lookup(unknown-id) present? false -> using guest
```

`checkout` has no null checks, and the guest's `notify` silently does nothing. (Prices are `BigDecimal` with an explicit rounding mode, never `double`.)

Trade-offs, when not to use it, mistakes

Pros	Cons
Removes repetitive null checks; fewer NPEs.	Can **hide real errors** — a missing object that *should* exist fails silently.
Polymorphism instead of conditionals.	One more class per interface.
Easy to test; a natural default.	Callers may not realise they got a "nothing" object.

- **Don't use it** when absence is an **error** (throw an exception) or when the caller must decide what absence means (return `Optional<T>` — a JDK container that either holds a value or is empty).
- **Null Object vs `Optional`:** Null Object hides absence behind neutral behaviour; `Optional` makes absence **visible in the type** and lets each caller choose (`orElse`, `orElseThrow`). Use `Optional` for return values of lookups; use Null Object for **collaborators** that are legitimately optional (a logger, a listener, a discount policy).
- **Common mistakes:** a null object with **side effects** or non-neutral values; a mutable null-object singleton; overusing `isNull()` checks (which brings the conditionals back).

Real-world examples

`Collections.emptyList()` / `emptyMap()` (empty, not null); `InputStream.nullInputStream()`, `OutputStream.nullOutputStream()`, `Reader.nullReader()`, `Writer.nullWriter()` (Java 11); SLF4J's (a common Java logging API) `NOPLogger`; a "no-op" (does nothing) `MeterRegistry`/tracer in **observability** libraries (metrics, logs and traces that show what a running system is doing).

In the interview

> **30-second answer:** "Null Object replaces `null` with an implementation that does nothing or returns neutral values, so callers never need null checks — like returning an empty list instead of null. I use it for optional collaborators like loggers or discount policies, and `Optional` for lookups where the caller should decide. I avoid it when absence is actually an error, because it would hide the bug."

Follow-up — is it a GoF pattern? No; it was described by Bobby Woolf (in *Pattern Languages of Program Design 3*) and appears in Fowler's *Refactoring* as "Introduce Special Case" (earlier "Introduce Null Object").

Self-check. Q: What does `Collections.emptyList()` have to do with this? A: It's a null object for lists — safe to iterate, never null. Q: When should you throw instead? A: When the object must exist and its absence is a bug.

> **TL;DR:** Return a harmless "nothing" object instead of `null`; use `Optional` when the caller must choose; throw when absence is a bug.

11.7.2 Dependency Injection (DI)

What it is

Dependency Injection means a class **receives** the objects it depends on (its *dependencies*) from outside, instead of **creating** them itself with `new` or looking them up. It is the most common way to implement **Inversion of Control (IoC)** — the framework or a "composition root" builds and wires objects, not the objects themselves. The name was settled by Martin Fowler in his 2004 article on IoC containers.

Analogy: a restaurant kitchen doesn't grow its own vegetables; suppliers deliver them. Change the supplier, and the chef's recipes don't change.

The problem it solves

```
class OrderService {
    private final EmailSender sender = new EmailSender();   // hard-wired
}
```

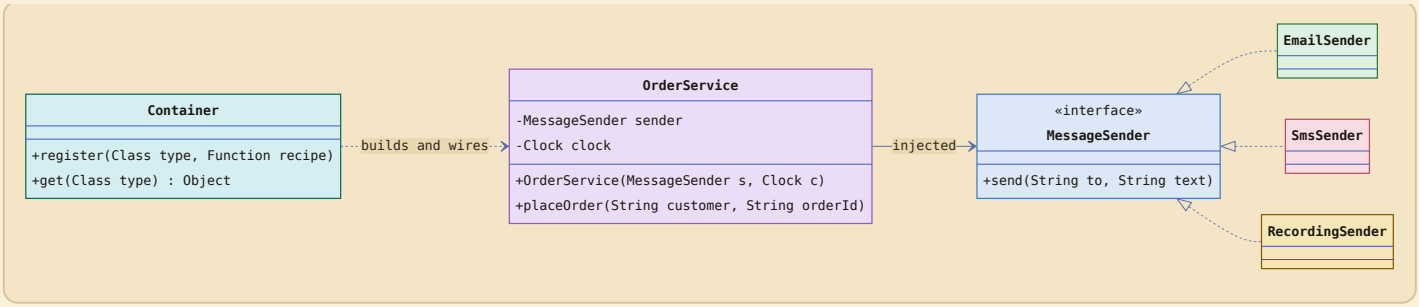
This class can never send SMS, can't be unit-tested without really sending email, and hides its dependencies. DI fixes all three: depend on an **interface** (`MessageSender`), receive it through the **constructor**, and let someone else decide which implementation.

How it works

1. The class declares dependencies as **constructor parameters** typed by **interfaces** (Dependency Inversion Principle, Chapter 4).
2. A **composition root** (usually `main`, or a DI container at start-up) creates the concrete objects and passes them in.
3. Tests pass **fakes** (simple stand-in implementations used only in tests) instead.

Three injection styles:

Style	How	Use for
Constructor injection	Dependencies are constructor parameters, stored in `final` fields	Required dependencies — the default choice
Setter injection	`setX(...)` after construction	Optional dependencies or ones that can change
Field injection	Framework writes directly into fields (`@Autowired` annotation on a field)	Avoid: hides dependencies and makes testing harder



Java example

`OrderService` receives both a `MessageSender` and a `java.time.Clock` — injecting the clock makes time **testable** (`Clock.fixed`). `Container` is a toy IoC container: you register a *recipe* per type; `get` builds objects on demand (recipes can ask the container for their own dependencies) and caches them as singletons.

```java
// File: code/lld/ch11/di/MessageSender.java
package lld.ch11.di;

/** The abstraction the service depends on (Dependency Inversion). */
public interface MessageSender {
    void send(String to, String text);
}
```

```java
// File: code/lld/ch11/di/EmailSender.java
package lld.ch11.di;

public final class EmailSender implements MessageSender {
    public void send(String to, String text) { System.out.println("  [smtp] to " + to + ": " + text); }
}
```

```java
// File: code/lld/ch11/di/SmsSender.java
package lld.ch11.di;

public final class SmsSender implements MessageSender {
    public void send(String to, String text) { System.out.println("  [sms] to " + to + ": " + text); }
}
```

```java
// File: code/lld/ch11/di/RecordingSender.java
package lld.ch11.di;

import java.util.ArrayList;
import java.util.List;

/** Test double: records instead of sending. Only possible because the dependency is injected. */
public final class RecordingSender implements MessageSender {
    public final List<String> sent = new ArrayList<>();
    public void send(String to, String text) { sent.add(to + "|" + text); }
}
```

```java
// File: code/lld/ch11/di/OrderService.java
package lld.ch11.di;

import java.time.Clock;
import java.time.LocalTime;

/**
 * CONSTRUCTOR INJECTION: the service declares what it needs and never calls 'new'
 * on its collaborators. Even the clock is injected, so time is testable.
 */
public final class OrderService {
    private final MessageSender sender;
    private final Clock clock;

    public OrderService(MessageSender sender, Clock clock) {
        this.sender = sender;
        this.clock = clock;
    }

    public void placeOrder(String customer, String orderId) {
        LocalTime now = LocalTime.now(clock);
        sender.send(customer, "order " + orderId + " placed at " + now);
    }
}
```

```java
// File: code/lld/ch11/di/Container.java
package lld.ch11.di;

import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

/**
 * A toy IoC container: register how to build each type once; ask for the type later.
 * Real containers (Spring, Guice, Dagger) add scanning, scopes, proxies and lifecycles.
 */
public final class Container {
    private final Map<Class<?>, Function<Container, ?>> recipes = new HashMap<>();
    private final Map<Class<?>, Object> singletons = new HashMap<>();

    public <T> void register(Class<T> type, Function<Container, ? extends T> recipe) {
        recipes.put(type, recipe);
    }

    public <T> T get(Class<T> type) {
        Object existing = singletons.get(type);
        if (existing != null) return type.cast(existing);
        Function<Container, ?> recipe = recipes.get(type);
        if (recipe == null) throw new IllegalStateException("no binding for " + type.getSimpleName());
        T created = type.cast(recipe.apply(this));   // the recipe may ask the container for its own deps
        singletons.put(type, created);
        return created;
    }
}
```

```java
// File: code/lld/ch11/di/Main.java
package lld.ch11.di;

import java.time.Clock;
import java.time.Instant;
import java.time.ZoneOffset;

public class Main {
    public static void main(String[] args) {
        Clock fixed = Clock.fixed(Instant.parse("2026-01-15T09:30:00Z"), ZoneOffset.UTC);

        System.out.println("=== 1. Manual DI: the composition root wires everything ===");
        new OrderService(new EmailSender(), fixed).placeOrder("asha@example.com", "A-1");
        new OrderService(new SmsSender(), fixed).placeOrder("+91-90000-00000", "A-2");

        System.out.println("\n=== 2. In a unit test: inject a fake ===");
        RecordingSender fake = new RecordingSender();
        new OrderService(fake, fixed).placeOrder("ben", "T-9");
        System.out.println("  fake recorded: " + fake.sent);

        System.out.println("\n=== 3. A toy IoC container ===");
        Container c = new Container();
        c.register(Clock.class, k -> fixed);
        c.register(MessageSender.class, k -> new EmailSender());
        c.register(OrderService.class, k -> new OrderService(k.get(MessageSender.class), k.get(Clock.class)));
        c.get(OrderService.class).placeOrder("chen@example.com", "A-3");
        System.out.println("  same instance each time: " + (c.get(OrderService.class) ==
c.get(OrderService.class)));
    }
}
```

Output:

```
=== 1. Manual DI: the composition root wires everything ===
  [smtp] to asha@example.com: order A-1 placed at 09:30
  [sms] to +91-90000-00000: order A-2 placed at 09:30

=== 2. In a unit test: inject a fake ===
  fake recorded: [ben|order T-9 placed at 09:30]

=== 3. A toy IoC container ===
  [smtp] to chen@example.com: order A-3 placed at 09:30
  same instance each time: true
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Swap implementations without editing the class (OCP, DIP).	More indirection: "where does this come from?"
Easy unit tests with fakes.	Containers add "magic" (reflection — code inspecting classes at run time; proxies — generated wrapper objects) and start-up cost.
Dependencies are explicit in the constructor.	Misconfiguration may only show at start-up (or run time).

- **Don't** inject value objects or simple data (`new Money(...)` is fine), or add interfaces with only one implementation "just for DI" when no test or variation needs them.
- **Common mistakes:** the **Service Locator** anti-pattern (classes call `container.get(...)` themselves — dependencies hidden again); field injection; **constructor with 8+ dependencies** (a sign the class does too much — split it); **circular dependencies** (A needs B while B needs A).

Real-world examples

Spring (`@Component` + constructor injection, `ApplicationContext` as the container), **Google Guice**, **Dagger/Hilt** (compile-time DI on Android), **Jakarta CDI**, **Micronaut/Quarkus** (compile-time DI for fast start-up). Spring can even inject a `Map<String, PaymentStrategy>` of all strategy beans — DI + Strategy together.

In the interview

> **30-second answer:** "Dependency injection means a class receives its collaborators through its constructor, typed by interfaces, instead of creating them. It's how we apply Dependency Inversion and Inversion of Control: a composition root or a container like Spring wires the graph. The benefits are swappable implementations and easy testing with fakes — I even inject the Clock. I prefer constructor injection, avoid field injection and service locators, and treat a huge constructor as a **design smell** (a surface sign that something deeper is wrong)."

Follow-ups: **DI vs IoC?** IoC is the general idea (the framework calls you / controls object creation); DI is one way to achieve it. **DI vs Service Locator?** With a locator the class asks for dependencies (hidden); with DI they're handed in (visible). **DI vs Factory?** A factory creates objects on request; DI supplies ready-made dependencies — containers use factories internally.

Self-check. Q: Why inject `Clock`? A: To make time-dependent code deterministic in tests. Q: Which injection style is preferred? A: Constructor injection.

> **TL;DR:** Don't `new` your collaborators — **receive them** (constructor, interfaces). Wire at the composition root or with a container.

11.7.3 Repository

What it is

A **Repository** is a **collection-like interface for loading and saving domain objects**, written in the language of the domain (`findByEmail`, `findActive`), that **hides how and where data is stored** (SQL — relational databases queried with the SQL language; NoSQL — non-relational stores such as key-value or document databases; files; memory). Described by Fowler (*PoEAA*) and central to Evans's *Domain-Driven Design*.

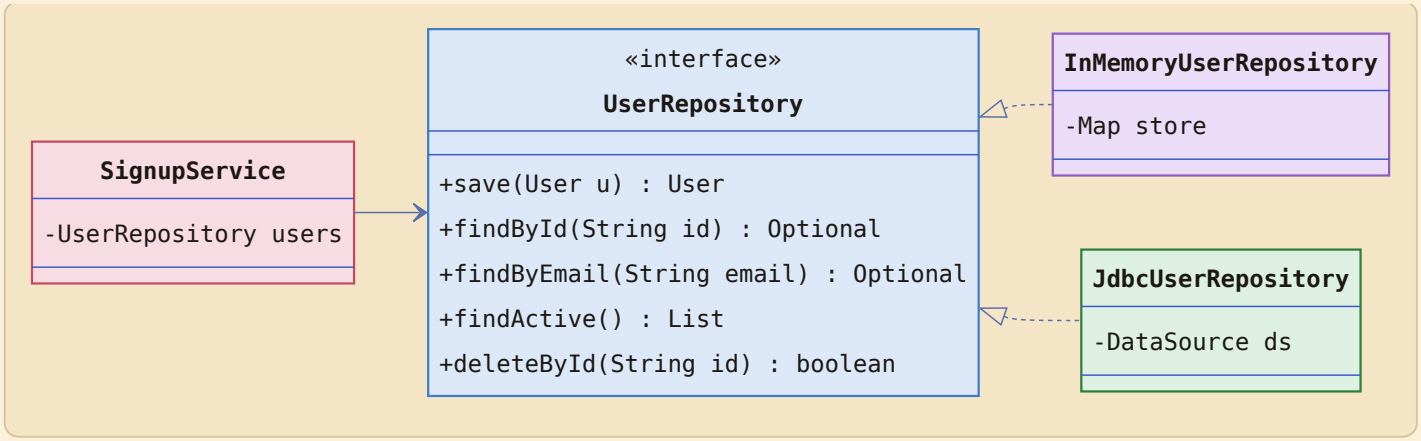
Analogy: a library's front desk. You ask for "books by this author"; you don't need to know which shelf, basement or branch they come from.

The problem it solves

When SQL, connection handling and mapping code are scattered through business classes, the business rules become hard to read, impossible to unit-test without a database, and tied to one storage technology.

How it works

1. Define the interface in the **domain layer** (the code that holds business rules) (`UserRepository`: `save`, `findById`, `findByEmail`, ...). Return `Optional` for "maybe not found".
2. Implement it in the **infrastructure layer** (code that talks to databases, networks and files): `InMemoryUserRepository` for tests, a JDBC/JPA one (JPA, the Java Persistence API, maps objects to tables) for production.
3. Services depend on the **interface** (injected — see DI).



(`JdbcUserRepository` is shown to illustrate a production implementation; the demo code includes the in-memory one.)

Java example

```java
// File: code/lld/ch11/repository/User.java
package lld.ch11.repository;

/** A domain object. It knows nothing about storage. */
public record User(String id, String email, boolean active) {}
```

```java
// File: code/lld/ch11/repository/UserRepository.java
package lld.ch11.repository;

import java.util.List;
import java.util.Optional;

/**
 * REPOSITORY: a collection-like interface for aggregates, in DOMAIN language.
 * The domain depends on this; the storage details live in implementations.
 */
public interface UserRepository {
    User save(User user);
    Optional<User> findById(String id);
    Optional<User> findByEmail(String email);
    List<User> findActive();
    boolean deleteById(String id);
}
```

```java
// File: code/lld/ch11/repository/InMemoryUserRepository.java
package lld.ch11.repository;

import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

/** One implementation: in memory (tests, prototypes). A JDBC/JPA one would implement the same interface. */
public final class InMemoryUserRepository implements UserRepository {
    private final Map<String, User> store = new ConcurrentHashMap<>();

    public User save(User user) { store.put(user.id(), user); return user; }
    public Optional<User> findById(String id) { return Optional.ofNullable(store.get(id)); }

    public Optional<User> findByEmail(String email) {
        return store.values().stream().filter(u -> u.email().equalsIgnoreCase(email)).findFirst();
    }

    public List<User> findActive() {
        return store.values().stream().filter(User::active)
                    .sorted(Comparator.comparing(User::id)).toList();
    }

    public boolean deleteById(String id) { return store.remove(id) != null; }
}
```

```java
// File: code/lld/ch11/repository/SignupService.java
package lld.ch11.repository;

/** Business logic talks to the repository interface only: no SQL, no maps, no files. */
public final class SignupService {
    private final UserRepository users;

    public SignupService(UserRepository users) { this.users = users; }

    public String signUp(String id, String email) {
        if (users.findByEmail(email).isPresent()) return "rejected: " + email + " already registered";
        users.save(new User(id, email, true));
        return "created " + id;
    }

    public void deactivate(String id) {
        users.findById(id).ifPresent(u -> users.save(new User(u.id(), u.email(), false)));
    }
}
```

```java
// File: code/lld/ch11/repository/Main.java
package lld.ch11.repository;

public class Main {
    public static void main(String[] args) {
        UserRepository repo = new InMemoryUserRepository();   // swap for a JDBC version in production
        SignupService service = new SignupService(repo);
        System.out.println("  " + service.signUp("u1", "asha@example.com"));
        System.out.println("  " + service.signUp("u2", "ben@example.com"));
        System.out.println("  " + service.signUp("u3", "ASHA@example.com"));
        service.deactivate("u2");
        System.out.println("  active users: " + repo.findActive());
        System.out.println("  findById(u9): " + repo.findById("u9"));
        System.out.println("  deleteById(u2): " + repo.deleteById("u2"));
    }
}
```

Output:

```
created u1
created u2
rejected: ASHA@example.com already registered
active users: [User[id=u1, email=asha@example.com, active=true]]
findById(u9): Optional.empty
deleteById(u2): true
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Business logic is storage-agnostic and testable in memory.	Another layer; trivial CRUD (create, read, update, delete) apps may not need it.
One place for queries per aggregate.	Can leak storage concerns (lazy loading — fetching data only when first touched; transactions — changes that succeed or fail together) anyway.
Swap storage (SQL → NoSQL) behind the interface.	"Generic repository for everything" becomes a thin, useless wrapper.

- **Don't** create a repository per table; create one per **aggregate root** (DDD: the entry point object of a cluster, e.g. `Order` with its `OrderLine`s).
- **Repository vs DAO:** a DAO (Data Access Object) is table/record-oriented and closer to the database; a repository is domain-oriented and behaves like a collection of aggregates. In small apps they often look the same.
- **Common mistakes:** returning `null` instead of `Optional`; exposing SQL types (`ResultSet`) in the interface; putting business rules in the repository; dozens of `findByXAndYOrZ` methods — use **Specification** (next).
- **Thread-safety note:** `InMemoryUserRepository` uses a `ConcurrentHashMap` (the JDK's thread-safe hash map), so each single call is safe, but `SignupService.signUp` is **check-then-act** (a race where the answer to the check can be stale by the time you act on it) (`findByEmail`, then `save`): two concurrent sign-ups with the same email can both pass the check. A real system enforces uniqueness with a database **unique constraint** (a rule that makes the database reject a duplicate value in a column) (or an atomic `putIfAbsent` keyed by email in memory).

Real-world examples

Spring Data `CrudRepository` / `JpaRepository` (derived query methods like `findByEmail` are generated from method names), Micronaut Data, Hibernate/JPA behind them; Android Room DAOs wrapped in repositories (Android architecture guidance).

In the interview

30-second answer: "A repository is a collection-like interface for aggregates in domain language — save, findById, findByEmail — that hides storage. Services depend on the interface, so business logic is testable with an in-memory implementation and storage can change. In machine-coding rounds I always put an in-memory repository behind an interface, so the design is ready for a database later."

Self-check. Q: What does a repository return when nothing is found? A: `Optional.empty()` (or an empty list). Q: One repository per table? A: No, per aggregate root.

TL;DR: A **domain-language collection interface** over storage; services use the interface; start in-memory, swap later.

11.7.4 Specification

What it is

A **Specification** is a **business rule packaged as an object** with one question — `isSatisfiedBy(candidate)` — that can be **combined** with `and`, `or` and `not`. Introduced by Eric Evans and Martin Fowler.

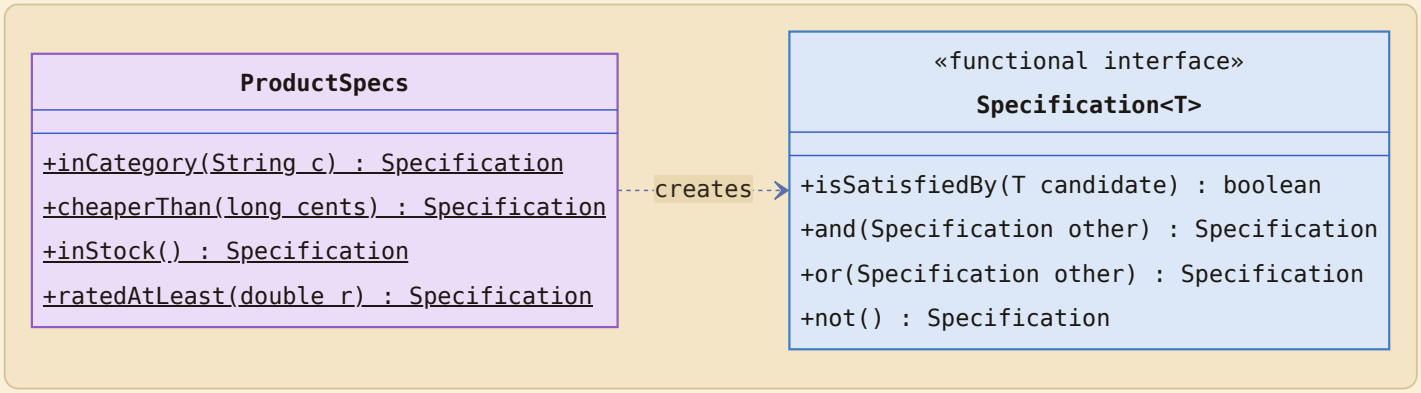
Analogy: the filter panel on a shopping site. "Electronics" ✓, "under ₹5,000" ✓, "in stock" ✓ — each is a separate rule, and the site combines the ones you tick.

The problem it solves

Rules like "bargain electronics = category electronics AND price < 50.00 AND in stock" end up duplicated in queries, validations and UI filters, or as an explosion of repository methods (`findByCategoryAndPriceLessThanAndStockGreaterThan...`). Specification gives each rule a **name**, makes it **reusable** in several places (selection, validation, building to order), and **composable**.

How it works

1. `Specification<T>` with `isSatisfiedBy(T)`; default methods `and`, `or`, `not` return new composite specifications (this is the **Composite** pattern + **Interpreter**-like evaluation).
2. Write small named factories (`inStock()`, `cheaperThan(...)`).
3. Combine them where needed; pass them to a repository or a stream filter.



Java example

```java
// File: code/lld/ch11/specification/Specification.java
package lld.ch11.specification;

/**
 * SPECIFICATION: a business rule as a small, named, combinable object.
 * isSatisfiedBy answers "does this candidate meet the rule?"
 */
@FunctionalInterface
public interface Specification<T> {
    boolean isSatisfiedBy(T candidate);

    default Specification<T> and(Specification<T> other) { return c -> isSatisfiedBy(c) && other.isSatisfiedBy(c);
}
    default Specification<T> or(Specification<T> other)  { return c -> isSatisfiedBy(c) || other.isSatisfiedBy(c);
}
    default Specification<T> not()                       { return c -> !isSatisfiedBy(c); }
}
```

```java
// File: code/lld/ch11/specification/Product.java
package lld.ch11.specification;

public record Product(String name, String category, long priceCents, int stock, double rating) {}
```

```java
// File: code/lld/ch11/specification/ProductSpecs.java
package lld.ch11.specification;

/** Named rules, in the language of the business. Each is tiny and testable alone. */
public final class ProductSpecs {
    private ProductSpecs() {}

    public static Specification<Product> inCategory(String c) { return p -> p.category().equals(c); }
    public static Specification<Product> cheaperThan(long cents) { return p -> p.priceCents() < cents; }
    public static Specification<Product> inStock() { return p -> p.stock() > 0; }
    public static Specification<Product> ratedAtLeast(double r) { return p -> p.rating() >= r; }
}
```

```java
// File: code/lld/ch11/specification/Main.java
package lld.ch11.specification;

import static lld.ch11.specification.ProductSpecs.*;

import java.util.List;

public class Main {
    static List<String> select(List<Product> catalog, Specification<Product> spec) {
        return catalog.stream().filter(spec::isSatisfiedBy).map(Product::name).toList();
    }

    public static void main(String[] args) {
        List<Product> catalog = List.of(
            new Product("usb-cable", "electronics", 499, 120, 4.1),
            new Product("headphones", "electronics", 7_999, 0, 4.6),
            new Product("keyboard", "electronics", 4_999, 8, 4.7),
            new Product("novel", "books", 1_299, 30, 4.8),
            new Product("cookbook", "books", 2_499, 0, 3.9));

        // The same rule objects are reused for querying AND for validation.
        Specification<Product> bargainElectronics =
inCategory("electronics").and(cheaperThan(5_000)).and(inStock());
        Specification<Product> recommendable = ratedAtLeast(4.5).and(inStock());
        Specification<Product> needsRestock = inStock().not();

        System.out.println("  bargain electronics: " + select(catalog, bargainElectronics));
        System.out.println("  recommendable:       " + select(catalog, recommendable));
        System.out.println("  needs restock:       " + select(catalog, needsRestock));
        System.out.println("  books or top-rated:  " + select(catalog, inCategory("books").or(ratedAtLeast(4.7))));
        System.out.println("  is 'novel' recommendable? " + recommendable.isSatisfiedBy(catalog.get(3)));
    }
}
```

Output:

```
  bargain electronics: [usb-cable, keyboard]
  recommendable:       [keyboard, novel]
  needs restock:       [headphones, cookbook]
  books or top-rated:  [keyboard, novel, cookbook]
  is 'novel' recommendable? true
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Rules are named, reusable and testable one by one.	In-memory specs don't translate to SQL automatically — you need a query-building version (e.g. JPA Criteria) for large data.
Combining rules replaces a method explosion.	Overkill for one or two fixed filters.
The same rule serves querying and validation.	Deeply nested combinations can be hard to debug.

- Java's own `Predicate<T>` (the JDK interface for a yes/no test on a value, with `and`, `or`, `negate`) is already a specification; a named `Specification` type adds domain meaning and a place for SQL translation.
- **Common mistake:** filtering millions of rows in memory — push specifications down to the database.

Real-world examples

Spring Data JPA `Specification<T>` (`where(...).and(...)`, `JpaSpecificationExecutor.findAll(spec)`), e-commerce search filters, eligibility rules (loan, coupon, delivery-slot eligibility), access-control policies.

In the interview

> **30-second answer:** "A specification is a named business rule object with `isSatisfiedBy`, combinable with and/or/not. It removes duplicated conditions and the explosion of repository finder methods, and the same rule can filter a query and validate an object. In Java, `Predicate` is the building block; Spring Data JPA has a `Specification` that translates to SQL."

Self-check. Q: What three combinators does a specification usually have? A: `and`, `or`, `not`. Q: Which pattern does combining specifications resemble? A: Composite (and Interpreter for evaluation).

> **TL;DR: Rules as small combinable objects**; reuse them for querying and validation.

11.7.5 MVC — Model–View–Controller (the Head First "compound pattern")

What it is

MVC splits an interactive application into three roles:

- **Model** — the data and business rules. Knows nothing about screens.
- **View** — displays the model and receives user input.
- **Controller** — interprets user input and tells the model what to change.

It was created by Trygve Reenskaug for Smalltalk at Xerox PARC in the late 1970s. *Head First Design Patterns* calls MVC a **compound pattern** — several patterns working together:

Pattern inside MVC	Where
Observer	The model notifies views (and sometimes controllers) when its state changes; views re-render.
Strategy	The view delegates "what should this input do?" to a controller object; swap the controller to change behaviour (e.g. read-only mode).
Composite	The view is a tree of nested components (panels containing labels, lists, buttons) rendered uniformly.

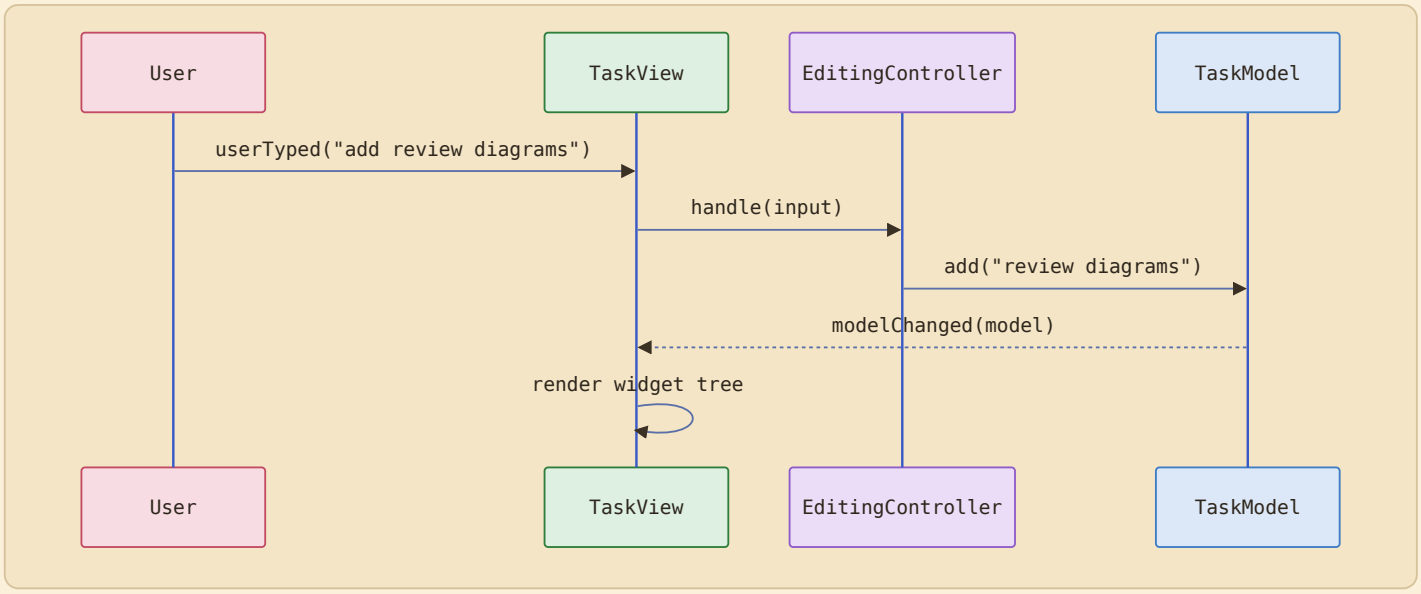
Analogy: a restaurant. The kitchen and pantry are the **model**; the dining-room menu and plates are the **view**; the waiter who takes your order and passes it to the kitchen is the **controller**.

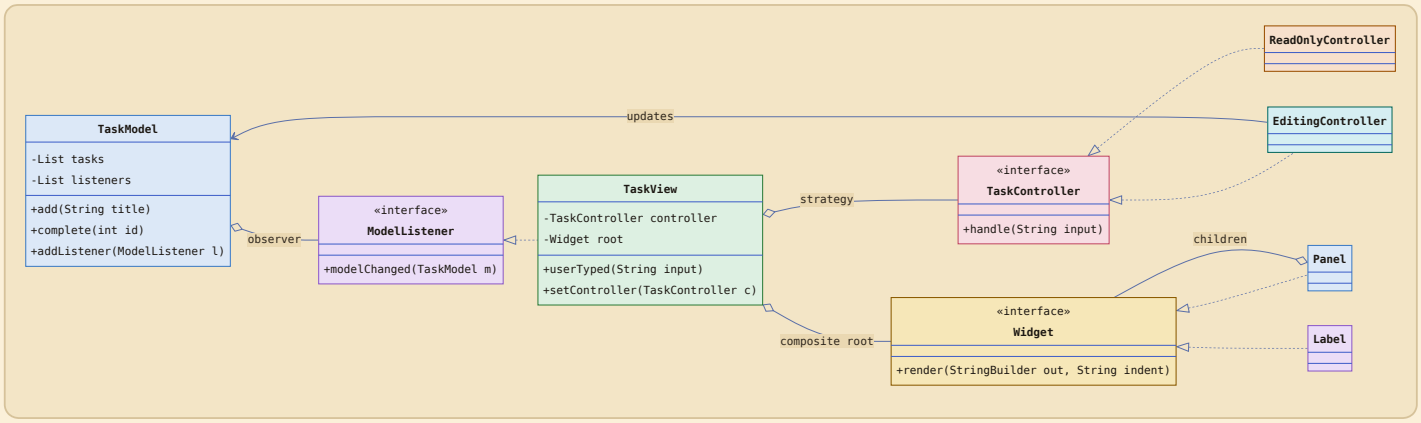
The problem it solves

When UI code, input handling and business rules are mixed, you cannot change the screen without risking the rules, cannot show the same data in two ways, and cannot test the rules without a UI.

How it works

1. The user acts on the **view** (`userTyped("add ...")`).
2. The view hands the input to its **controller** (strategy).
3. The controller calls the **model** (`model.add(title)`); it does **not** redraw anything.
4. The model changes and **notifies its listeners** (observer).
5. The view re-renders its **composite** widget tree from the model.





Java example

A console to-do app. `TaskModel` knows only `ModelListener`s. `TaskView` implements `ModelListener`, renders a **composite** of `Widget.Panel` / `Widget.Label`, and forwards input to a swappable `TaskController`. Switching to `ReadOnlyController` changes behaviour without touching the view or model.

```java
// File: code/lld/ch11/mvc/Task.java
package lld.ch11.mvc;

public record Task(int id, String title, boolean done) {}
```

```java
// File: code/lld/ch11/mvc/TaskModel.java
package lld.ch11.mvc;

import java.util.ArrayList;
import java.util.List;

/** MODEL: state + business rules. Knows NOTHING about views or controllers, only listeners. */
public final class TaskModel {
    private final List<Task> tasks = new ArrayList<>();
    private final List<ModelListener> listeners = new ArrayList<>();
    private int nextId = 1;

    public void addListener(ModelListener l) { listeners.add(l); }

    public List<Task> tasks() { return List.copyOf(tasks); }

    public void add(String title) {
        if (title.isBlank()) throw new IllegalArgumentException("title must not be blank");
        tasks.add(new Task(nextId++, title, false));
        fireChanged();
    }

    public void complete(int id) {
        for (int i = 0; i < tasks.size(); i++) {
            Task t = tasks.get(i);
            if (t.id() == id) {
                tasks.set(i, new Task(t.id(), t.title(), true));
                fireChanged();
                return;
            }
        }
        throw new IllegalArgumentException("no task " + id);
    }

    private void fireChanged() { listeners.forEach(l -> l.modelChanged(this)); }
}
```

```java
// File: code/lld/ch11/mvc/ModelListener.java
package lld.ch11.mvc;

/** OBSERVER interface between model and views. */
public interface ModelListener {
    void modelChanged(TaskModel model);
}
```

```java
// File: code/lld/ch11/mvc/Widget.java
package lld.ch11.mvc;

import java.util.List;
import java.util.function.Supplier;

/** COMPOSITE: the view is a tree of widgets rendered the same way. */
public interface Widget {
    void render(StringBuilder out, String indent);

    record Label(Supplier<String> text) implements Widget {
        public void render(StringBuilder out, String indent) { out.append(indent).append(text.get()).append('\n');
}
    }

    record Panel(String title, List<Widget> children) implements Widget {
        public void render(StringBuilder out, String indent) {
            out.append(indent).append("+ ").append(title).append('\n');
            children.forEach(c -> c.render(out, indent + "  "));
        }
    }
}
```

```java
// File: code/lld/ch11/mvc/TaskController.java
package lld.ch11.mvc;

/** CONTROLLER as a STRATEGY: the view delegates "what does this input mean?" to it. */
public interface TaskController {
    void handle(String input);
}
```

```java
// File: code/lld/ch11/mvc/EditingController.java
package lld.ch11.mvc;

/** Interprets user input and updates the MODEL. It does not redraw anything itself. */
public final class EditingController implements TaskController {
    private final TaskModel model;
    private final TaskView view;

    public EditingController(TaskModel model, TaskView view) {
        this.model = model;
        this.view = view;
    }

    public void handle(String input) {
        try {
            if (input.startsWith("add ")) model.add(input.substring(4));
            else if (input.startsWith("done ")) model.complete(Integer.parseInt(input.substring(5).trim()));
            else view.showMessage("unknown command: " + input);
        } catch (IllegalArgumentException e) {        // NumberFormatException is a subclass
            view.showMessage("error: " + e.getMessage());
        }
    }
}
```

```java
// File: code/lld/ch11/mvc/ReadOnlyController.java
package lld.ch11.mvc;

/** Alternative strategy: same view, different behaviour (e.g. a viewer role). */
public final class ReadOnlyController implements TaskController {
    private final TaskView view;

    public ReadOnlyController(TaskView view) { this.view = view; }

    public void handle(String input) { view.showMessage("read-only mode, ignored: " + input); }
}
```

```java
// File: code/lld/ch11/mvc/TaskView.java
package lld.ch11.mvc;

import java.util.List;
import java.util.stream.Collectors;

/** VIEW: observes the model, renders it (composite), forwards user input to its controller (strategy). */
public final class TaskView implements ModelListener {
    private TaskController controller;
    private final Widget root;
    private TaskModel lastSeen;

    public TaskView() {
        root = new Widget.Panel("My Tasks", List.of(
            new Widget.Label(() -> lastSeen == null ? "(empty)" : lastSeen.tasks().stream()
                .map(t -> (t.done() ? "[x] " : "[ ] ") + t.id() + ". " + t.title())
                .collect(Collectors.joining("\n    "))),
            new Widget.Panel("Summary", List.of(
                new Widget.Label(() -> lastSeen == null ? "0 open" :
                    lastSeen.tasks().stream().filter(t -> !t.done()).count() + " open")))));
    }

    public void setController(TaskController controller) { this.controller = controller; }

    /** A user action arrives at the view... and is handed straight to the controller. */
    public void userTyped(String input) {
        System.out.println("> " + input);
        controller.handle(input);
    }

    public void showMessage(String msg) { System.out.println("  (" + msg + ")"); }

    @Override
    public void modelChanged(TaskModel model) {     // Observer callback: re-render from the model
        lastSeen = model;
        StringBuilder out = new StringBuilder();
        root.render(out, "  ");
        System.out.print(out);
    }
}
```

```java
// File: code/lld/ch11/mvc/Main.java
package lld.ch11.mvc;

public class Main {
    public static void main(String[] args) {
        TaskModel model = new TaskModel();
        TaskView view = new TaskView();
        model.addListener(view);                                    // Observer: model -> view
        view.setController(new EditingController(model, view));  // Strategy: view -> controller

        view.userTyped("add write chapter 11");
        view.userTyped("add review diagrams");
        view.userTyped("done 1");
        view.userTyped("done 42");

        System.out.println("-- switching the view to a read-only controller --");
        view.setController(new ReadOnlyController(view));
        view.userTyped("add sneaky task");
    }
}
```

Output:

```
> add write chapter 11
  + My Tasks
    [ ] 1. write chapter 11
    + Summary
      1 open
> add review diagrams
  + My Tasks
    [ ] 1. write chapter 11
    [ ] 2. review diagrams
    + Summary
      2 open
> done 1
  + My Tasks
    [x] 1. write chapter 11
    [ ] 2. review diagrams
    + Summary
      1 open
> done 42
  (error: no task 42)
-- switching the view to a read-only controller --
> add sneaky task
  (read-only mode, ignored: add sneaky task)
```

Trade-offs, variants and mistakes

Pros	Cons
Separation of concerns; business rules testable without UI.	More classes and indirection for tiny apps.
Several views on one model (list, chart, widget).	Update flows via observers can be hard to trace.
Swap controllers/views independently.	Controllers tend to get "fat" if business logic leaks into them.

- **Web MVC is different.** In **Spring MVC** (a "Model 2" design), a **front controller** (`DispatcherServlet` — a *servlet* is a Java class that handles HTTP requests inside a web server) routes each HTTP request to a `@Controller` method, which fills a model and returns a view name; the view renders once per request. The view does **not** observe the model — there is no long-lived observer link over HTTP.
- **Relatives: MVP** (Model–View–Presenter: a passive view, the presenter updates it) and **MVVM** (Model–View–ViewModel: the view binds to observable properties of a view-model — Android Jetpack `ViewModel` + `LiveData`/`StateFlow`, WPF). MVVM's view-model also plays a **mediator** role.

- **Common mistakes:** business logic in controllers ("fat controller"); views changing the model directly; models that import UI classes.

Real-world examples

Spring MVC, Ruby on Rails, Django (which calls it MTV), ASP.NET MVC, Swing (each component has a model, e.g. `JTable` + `TableModel` with `TableModelListener`), iOS UIKit (`UIViewController`), Android (MVVM).

In the interview

> **30-second answer:** "MVC separates the model — data and rules — from the view that shows it and the controller that interprets input. Head First calls it a compound pattern: the model is an Observer subject for the views, the controller is a Strategy the view delegates to, and the view is a Composite of widgets. On the web, like Spring MVC, it becomes request-driven with a front controller, and the view doesn't observe the model."

Self-check. Q: Which three patterns make up MVC? A: Observer, Strategy, Composite. Q: Does the controller redraw the view? A: No — it updates the model, and the model's notification makes the view redraw.

> **TL;DR:** **Model** (state + rules, observable) → **View** (composite, observes) → **Controller** (strategy that interprets input).

11.7.6 Event Sourcing (lite)

What it is

Event Sourcing stores **every change as an immutable event** ("ItemAdded", "ItemRemoved", "CheckedOut") in an **append-only log** (a log you can only add to at the end, never edit), and derives the current state by **replaying** those events — instead of storing only the latest state and overwriting it. It is included here because it **borrows ideas from Command, Memento and Observer at system level** (events look like recorded commands, snapshots are mementos, projections observe the log), and it shows up in design interviews about carts, wallets, orders and audit trails.

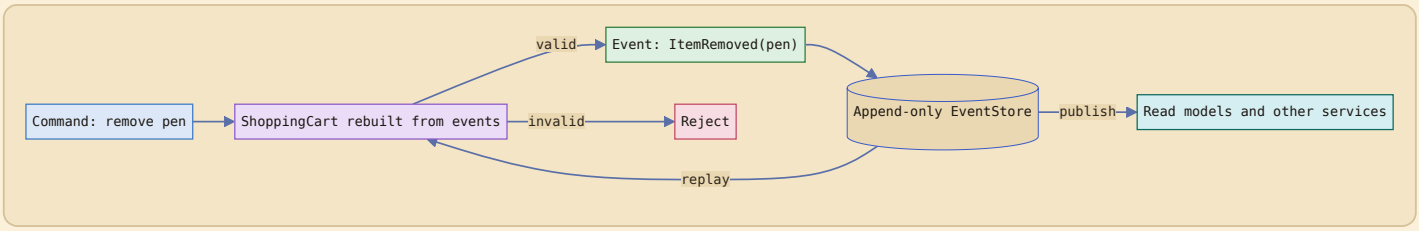
Analogy: a bank passbook. The balance is not the source of truth; the list of transactions is. The balance can always be recomputed from it.

The problem it solves

With "update in place", history is lost: you can't answer "what did the cart look like before checkout?", audit who changed what, or rebuild a **read model** (a copy of the data shaped for fast queries) after a bug. With events, the full history is kept, and you can **time-travel**, **audit**, and **build new views** later by replaying.

How it works

1. **Command side:** load the aggregate by replaying its events; **validate** the command against current state; if valid, **emit an event** (a fact, past tense).
2. **Append** the event to the store — never update or delete events.
3. **Apply**: a **pure function** (its output depends only on its inputs, and it changes nothing else) updates state from an event (no validation, no side effects — events are facts).
4. **Read side:** replay (all or the first *n* events) to get state at any point; publish events to **observers** that build read models (this split is **CQRS** — Command Query Responsibility Segregation).
5. For long streams, save periodic **snapshots** (Memento!) and replay only newer events.



Java example

A **sealed** `CartEvent` hierarchy; `ShoppingCart.apply` is an exhaustive pattern-matching `switch` over it (a `switch` *statement* with patterns must be exhaustive in Java 21, so a new event type that `apply` forgets is a compile error); `EventStore` is an append-only in-memory log whose methods are `synchronized`, so appends and reads from several threads are safe. Prices are `BigDecimal`.

```java
// File: code/lld/ch11/eventsourcing/CartEvent.java
package lld.ch11.eventsourcing;

import java.math.BigDecimal;

/** Facts that HAPPENED (past tense). Immutable; never updated or deleted. */
public sealed interface CartEvent permits CartEvent.ItemAdded, CartEvent.ItemRemoved, CartEvent.CheckedOut {
    record ItemAdded(String sku, int qty, BigDecimal unitPrice) implements CartEvent {}
    record ItemRemoved(String sku) implements CartEvent {}
    record CheckedOut() implements CartEvent {}
}
```

```java
// File: code/lld/ch11/eventsourcing/EventStore.java
package lld.ch11.eventsourcing;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/** Append-only log per stream (cart id). The log IS the source of truth. */
public final class EventStore {
    private final Map<String, List<CartEvent>> streams = new ConcurrentHashMap<>();

    public synchronized void append(String streamId, CartEvent e) {
        streams.computeIfAbsent(streamId, k -> new ArrayList<>()).add(e);
    }

    public synchronized List<CartEvent> read(String streamId) {
        return List.copyOf(streams.getOrDefault(streamId, List.of()));
    }
}
```

```java
// File: code/lld/ch11/eventsourcing/ShoppingCart.java
package lld.ch11.eventsourcing;

import java.math.BigDecimal;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** Aggregate whose current state is DERIVED by replaying events. */
public final class ShoppingCart {
    private final Map<String, BigDecimal> lineTotals = new LinkedHashMap<>();
    private boolean checkedOut;

    /** Rebuild state from history (optionally only the first n events: "time travel"). */
    public static ShoppingCart replay(List<CartEvent> history, int n) {
        ShoppingCart cart = new ShoppingCart();
        history.stream().limit(n).forEach(cart::apply);
        return cart;
    }

    /** Pure state transition: no validation, no side effects. Events are facts. */
    private void apply(CartEvent e) {
        switch (e) {
            case CartEvent.ItemAdded a -> lineTotals.merge(a.sku(),
a.unitPrice().multiply(BigDecimal.valueOf(a.qty())), BigDecimal::add);
            case CartEvent.ItemRemoved r -> lineTotals.remove(r.sku());
            case CartEvent.CheckedOut c -> checkedOut = true;
        }
    }

    /** Command side: validate against current state, THEN emit an event. */
    public CartEvent decideRemove(String sku) {
        if (checkedOut) throw new IllegalStateException("cart already checked out");
        if (!lineTotals.containsKey(sku)) throw new IllegalArgumentException(sku + " not in cart");
        return new CartEvent.ItemRemoved(sku);
    }

    @Override
    public String toString() {
        BigDecimal total = lineTotals.values().stream().reduce(new BigDecimal("0.00"), BigDecimal::add);
        return lineTotals.keySet() + " total=" + total + (checkedOut ? " CHECKED-OUT" : "");
    }
}
```

```java
// File: code/lld/ch11/eventsourcing/Main.java
package lld.ch11.eventsourcing;

import java.math.BigDecimal;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        EventStore store = new EventStore();
        String cartId = "cart-7";
        store.append(cartId, new CartEvent.ItemAdded("book", 2, new BigDecimal("12.99")));
        store.append(cartId, new CartEvent.ItemAdded("pen", 5, new BigDecimal("0.99")));

        ShoppingCart current = ShoppingCart.replay(store.read(cartId), Integer.MAX_VALUE);
        store.append(cartId, current.decideRemove("pen"));      // validated, then recorded
        store.append(cartId, new CartEvent.CheckedOut());

        List<CartEvent> history = store.read(cartId);
        System.out.println("event log:");
        history.forEach(e -> System.out.println("  " + e));

        System.out.println("state after each event (replay = time travel):");
        for (int n = 0; n <= history.size(); n++) {
            System.out.println("  after " + n + ": " + ShoppingCart.replay(history, n));
        }

        try {
            ShoppingCart.replay(history, history.size()).decideRemove("book");
        } catch (IllegalStateException e) {
            System.out.println("command rejected: " + e.getMessage());
        }
    }
}
```

Output:

```
event log:
  ItemAdded[sku=book, qty=2, unitPrice=12.99]
  ItemAdded[sku=pen, qty=5, unitPrice=0.99]
  ItemRemoved[sku=pen]
  CheckedOut[]
state after each event (replay = time travel):
  after 0: [] total=0.00
  after 1: [book] total=25.98
  after 2: [book, pen] total=30.93
  after 3: [book] total=25.98
  after 4: [book] total=25.98 CHECKED-OUT
command rejected: cart already checked out
```

Trade-offs, when not to use it, mistakes

Pros	Cons
Complete audit trail and time travel.	More complex than CRUD; a learning curve for the team.
Rebuild or add read models at any time.	**Event schema evolution** (versioning old events) is hard.
Natural fit with event-driven architectures.	Reads need projections (read models built from events); eventual consistency (the read side catches up after a short delay) between write and read sides.

- **Don't use it** for simple CRUD without audit or history needs.
- **Common mistakes:** putting validation in `apply` (replay of old facts may then fail); events named as commands (`RemoveItem` instead of `ItemRemoved`); mutable events; storing derived data inside events; no snapshots for very long streams; **appending without an expected-version check** — two commands decided on the same old state

can both append. Real event stores take an `expectedVersion` on append and reject the second writer (optimistic concurrency). The toy `EventStore` here omits it.

Real-world examples

Bank ledgers and payment systems (balances derived from ledger entries); Git (the version-control system — its history of commits); **Axon Framework** and **EventStoreDB**; Kafka used as an event log with long or unlimited **retention** (how long stored messages are kept) (not *compacted* topics — compaction keeps only the latest record per key, which throws away the history event sourcing needs); shopping carts and order histories where "what happened when" matters.

In the interview

30-second answer: "Event sourcing stores every change as an immutable, past-tense event in an append-only log and derives state by replaying. Commands are validated against the replayed state and then produce events; `apply` just folds events into state. You get audit and time travel, and can build new read models by replaying; the costs are complexity, schema evolution and eventual consistency. Snapshots keep replay fast — that's Memento plus a command log."

Self-check. Q: Where does validation happen? A: On the command side, before emitting the event — never in `apply`. Q: How do you keep replay fast? A: Periodic snapshots.

TL;DR: Store facts, not state; rebuild by replay; validate commands, never events; snapshot long streams.

11.8 Choose the right behavioral pattern

Start from the **symptom** in the code or requirement, not from the pattern name.

If you see / need...	Use	Not to be confused with
A request must pass several independent checks; any may reject it; different routes need different checks	**Chain of Responsibility**	Decorator (every layer always runs)
Undo/redo, queue, schedule, retry, or log *requested actions*	**Command**	Strategy (how, not what)
Evaluate user-written rules/expressions in a small language	**Interpreter** (+ Specification)	Visitor (for many operations over the AST)
Walk a collection/tree/paged API without exposing structure	**Iterator** (`Iterable`)	Visitor (what to do at each node)
Many objects depend on each other's changes and the *order* of reactions matters	**Mediator**	Observer (no coordination)
Roll back to an earlier state without exposing internals	**Memento**	Command undo (store actions instead)
One change must notify an open-ended set of independent listeners	**Observer** (or a message broker for durability)	Mediator
Three or more methods repeat one `switch` over a status; which actions are legal depends on that status	**State**	Strategy (chosen from outside)
Several interchangeable algorithms for one job, chosen by config/user/run time	**Strategy** (often a lambda)	State, Template Method
Several classes share a long fixed flow and differ in a few steps	**Template Method** (or template + callbacks)	Strategy
Many new operations over a stable set of types	**Visitor**, or **sealed types + pattern matching** in Java 21	Iterator, Strategy
`null` checks everywhere for an optional collaborator	**Null Object** (or `Optional` for lookups)	—
Classes create their own collaborators; hard to test	**Dependency Injection**	Service Locator (anti-pattern)
Business logic mixed with storage code	**Repository**	DAO (table-oriented)
The same business rule duplicated in queries and validation; finder-method explosion	**Specification**	—
UI, input handling and business rules tangled together	**MVC / MVVM**	—
Need full history, audit, time travel of changes	**Event Sourcing** (+ snapshots)	Plain audit log

Three questions that settle the famous look-alikes:

1. **Do the alternatives reference each other?** Yes → **State**. No → **Strategy** (or Bridge if chosen once at wiring).
2. **Can one participant stop the flow?** Yes → **Chain of Responsibility**. Every participant always runs → **Decorator** / pipeline. Everyone is told, nobody stops anyone → **Observer**.
3. **Does a central object decide what happens next?** Yes → **Mediator**. Each listener decides for itself → **Observer**.

In the interview (chapter-level)

"Which patterns have you used in production?" Pick concrete ones and name the *trade-off*: "Strategy for payment methods, chosen by a map injected by Spring; Observer via Spring events for sending emails after an order, moved to Kafka when we needed retries; State for the order lifecycle, with the DB update guarded by `WHERE status = ?` to avoid races."

"When would you avoid patterns?" When only two branches exist and a third is not expected; when a lambda or an enum does the job; when the indirection costs more readability than it buys flexibility. Patterns are a vocabulary for trade-offs, not a checklist.

Self-check (whole chapter)

1. **Q:** An order moves placed → accepted → preparing → delivered and "cancel" is only legal in early stages. Pattern? **A:** State (enum or class per state).

2. **Q:** Checkout must support card, UPI, wallet, and new methods next quarter. Pattern? **A:** Strategy, selected by a factory/DI map.
3. **Q:** After an order is placed, send email, update analytics and notify the restaurant; more reactions will come. Pattern? **A:** Observer in-process, or pub/sub via a broker for reliability.
4. **Q:** A compiler needs type-checking, optimisation and pretty-printing passes over a fixed AST. Pattern? **A:** Visitor, or sealed interfaces + pattern matching in Java 21.
5. **Q:** Three report generators share "load → filter → format → export" but differ in format. Pattern? **A:** Template Method (or a fixed pipeline taking a formatting strategy).
6. **Q:** What three GoF patterns make up MVC? **A:** Observer, Strategy, Composite.

TL;DR

- **Observer** broadcasts; **Mediator** coordinates; **CoR** lets someone stop the flow.
- **State** vs **Strategy**: same diagram — states know each other and advance themselves; strategies are chosen from outside.
- **Template Method** = inheritance, one step varies; **Strategy** = composition, whole algorithm varies.
- **Visitor** = cheap new operations, expensive new types; Java 21 **sealed + switch** usually replaces it.
- Beyond GoF: **Null Object**, **DI**, **Repository**, **Specification**, **MVC**, **Event Sourcing** — the everyday glue of real systems and machine-coding answers.

Chapter 12 — Concurrency & Multithreading for LLD (Java)

> **Why this chapter exists.** In a low-level design (LLD) interview you will often hear: *"Now two users click **Book** on the same seat at the same moment. What happens?"* If your design has no answer, it is incomplete. This chapter teaches you, from zero, the ideas and Java tools you need to answer that question well. Chapter 13 then applies them to the classic concurrency interview problems, and chapters 14–21 use them inside full case studies.

How to read this chapter. Every concept follows the same path: *What it is → The problem it solves → How it works → Java example → Trade-offs → When not to use it → Common mistakes → Real-world examples → In the interview → Self-check → TL;DR.* All code is Java 21, standard library only, and lives in `code/lld/ch12/<topic>/Main.java`. Every `Main` compiles and runs; the real output is shown under each listing.

Run any example from `chapters/12-concurrency-concepts/code`:

```
javac --release 21 -d /tmp/lld_12 $(find . -name '*.java')
java -cp /tmp/lld_12 lld.ch12.threads.Main
```

Map of the chapter

Part	Sections	Code package
A. Foundations	12.1 Concurrency vs parallelism · 12.2 Processes vs threads · 12.3 Thread lifecycle · 12.4 Creating threads	`threads`
B. What goes wrong	12.5 Race conditions & critical sections · 12.6 Java Memory Model (visibility, happens-before, `volatile`)	`race`
C. Locks	12.7 Mutex / `synchronized` · 12.8 `ReentrantLock` · 12.9 Try-lock & timed locking · 12.10 Read-write locks & `StampedLock` · 12.11 Coarse vs fine-grained locking	`locks`
D. Coordination	12.12 Semaphores · 12.13 Condition variables · 12.14 `CountDownLatch` & `CyclicBarrier`	`coordination`
E. Lock-free	12.15 Compare-and-swap & atomics · 12.16 Concurrent collections	`atomics`, `collections`
F. Liveness failures	12.17 Deadlock · 12.18 Livelock · 12.19 Starvation	`deadlock`
G. Patterns	12.20 Signaling · 12.21 Thread pool · 12.22 Producer-consumer · 12.23 Reader-writer · 12.24 Immutability & thread confinement	`patterns`
H. Interviews	12.25 Concurrency in LLD interviews (parking spots, seat booking, inventory)	`lldcases`
	12.26 Choosing the right tool · 12.27 Chapter self-check · 12.28 Chapter TL;DR	

Words you will see everywhere (defined once here):

Term	Meaning in one line
Thread	One independent sequence of instructions running inside a program.
Shared state	Data (fields, objects) that more than one thread can read or write.
Thread-safe	A class that behaves correctly no matter how threads interleave, with no extra locking by callers.
Atomic	Happens as one indivisible step: other threads see either "before" or "after", never "halfway".
Lock / mutex	A token only one thread can hold at a time. Code that needs the token waits for it.
Critical section	A piece of code that touches shared state and must not run in two threads at once.
Contention	Many threads wanting the same lock at the same time, so they queue up.
Blocking	A thread stops and waits (for a lock, I/O, a signal) instead of using the CPU.
Liveness	"Something good eventually happens" — the program keeps making progress (no deadlock, livelock, starvation).
Safety	"Nothing bad ever happens" — no corrupted data, no double booking.

Part A — Foundations

12.1 Concurrency vs parallelism

What it is

- **Concurrency** is about *dealing with* many things at once: the program is structured as several tasks whose lifetimes overlap. They may take turns on one **CPU core** (one independent processing unit inside the processor; a modern CPU has several).
- **Parallelism** is about *doing* many things at once: tasks literally execute at the same instant on different CPU cores.

> **Analogy.** One chef cooking three dishes, switching between them while each simmers, is *concurrent*. Three chefs each cooking one dish at the same time is *parallel*. A restaurant with three chefs who also switch between dishes is both.

The problem it solves

Programs wait a lot — for disks, networks, databases, users. Concurrency lets the program do useful work while one task waits (better *responsiveness* and *throughput* — how much work gets finished per second; see System Design Fundamentals). Parallelism lets CPU-heavy work (sorting, image processing) finish sooner by using all cores (better *speed-up*).

How it works

1. The operating system (OS) *scheduler* (the part of the OS that decides which thread runs next) gives each runnable thread a short time slice on a core.
2. When the slice ends, or the thread blocks (for **I/O** — input/output such as disk or network reads — a lock, `sleep`), the scheduler performs a **context switch**: it saves that thread's **registers** (the CPU's few, fastest storage slots) and loads another's.
3. On one core, switching fast creates the *illusion* of simultaneity → concurrency.
4. On many cores, several threads really run at the same instant → parallelism.

	Concurrency	Parallelism
Core idea	Structure: many tasks in progress	Execution: many tasks running at the same instant
Needs multiple cores?	No	Yes
Main benefit	Responsiveness, overlapping waiting (I/O-bound work: mostly waiting on disk/network)	Speed on CPU-bound work (mostly computing)
Java tools	Threads, executors, virtual threads, `CompletableFuture`	Fork/Join, parallel streams, `Arrays.parallelSort`
Typical example	A web server handling 10,000 connections	Sorting 100 million numbers

Java example

Section 12.4's `Main` shows both: 10,000 tasks that each *sleep* 50 ms finish in well under a second on virtual threads (concurrency hides waiting). Chapter 13's multi-threaded merge sort uses **Fork/Join** (the JDK framework that splits a task into subtasks and runs them on all cores) to split CPU work over cores (parallelism).

Trade-offs

Concurrency adds complexity (races, deadlocks) and some overhead (context switches). Parallelism has a limit: **Amdahl's law** says that if a fraction *s* of the work is inherently sequential, the speed-up with *N* cores can never exceed 1 / (s + (1 − s)/N). With 10% sequential work, even infinite cores give at most 10× speed-up.

When not to use it

Small, fast, CPU-light tasks: splitting them over threads costs more than it saves. A single-threaded design is also a valid choice when it removes all locking (Redis — an in-memory key-value database — executes commands on one main thread for this reason; Node.js, a server-side JavaScript runtime, runs JavaScript on one **event-loop** thread: a single loop that handles ready events one at a time and never blocks).

Common mistakes

- Saying "concurrent" when you mean "parallel" (and vice versa) in an interview.
- Expecting more threads to always mean more speed. Past the core count, CPU-bound work gets *slower* from switching.

Real-world examples

- **Concurrency:** Tomcat and Jetty (Java web servers) serve many HTTP requests with a **pool** of threads (a fixed set of reusable worker threads); Netty (a Java networking library) and Node.js serve thousands of connections with a few event-loop threads.
- **Parallelism:** `Arrays.parallelSort`, parallel streams, and Spark/Hadoop (distributed data-processing frameworks) jobs splitting data across cores and machines.

In the interview

> **30-second answer:** "Concurrency is about structure — several tasks in progress, possibly interleaved on one core. Parallelism is about execution — tasks running at the same instant on several cores. Concurrency helps I/O-bound work; parallelism helps CPU-bound work."

Follow-ups:

- *Can you have concurrency without parallelism?* Yes — a single-core machine running several threads by time-slicing.
- *Can you have parallelism without concurrency?* Yes, in a narrow sense: SIMD (one instruction on many data items) runs in parallel inside a single task.
- *How many threads for a CPU-bound pool?* About the number of cores (`Runtime.getRuntime().availableProcessors()`). For I/O-bound work, more — roughly cores × (1 + wait time / compute time).

Self-check

- **Q:** A chat server with 50,000 idle connections — concurrency or parallelism problem? **A:** Concurrency: most connections are waiting, not computing.
- **Q:** What limits the speed-up of parallel code? **A:** The sequential part (Amdahl's law), plus coordination overhead.

TL;DR

Concurrency = dealing with many things (structure). Parallelism = doing many things at once (execution). I/O-bound → concurrency; CPU-bound → parallelism.

12.2 Processes vs threads

What it is

- A **process** is a running program with its *own* memory space (**heap** — the memory area where objects live — code, open files). Two processes cannot see each other's memory.
- A **thread** is a unit of execution *inside* a process. All threads of a process share the process's heap memory, but each thread has its own **stack** (local variables, call frames) and **program counter** (the address of the next instruction to run).

> **Analogy.** A process is a house; threads are people living in it. People in the same house share the kitchen (heap) — fast to share, but they can bump into each other. People in different houses (processes) must phone each other (inter-process communication) — slower, but no bumping.

The problem it solves

Threads let one program do several things at once while sharing data cheaply. Processes give strong isolation: one crashing does not corrupt the others.

How it works

1. The OS creates a process with a private **virtual address space** (the process's own view of memory, which the OS maps onto real RAM).
2. The process starts with one thread (`main` in Java).
3. The program can create more threads; they share heap objects, and each gets its own stack.
4. Communication: threads just read/write shared objects (with synchronization); processes use **pipes** (one-way byte channels between processes), **sockets** (network-style connection endpoints), shared-memory segments or message queues.

	Process	Thread
Memory	Private address space	Shares the process's heap; own stack
Creation cost	High (new address space)	Lower (platform thread ≈ an OS thread with ~1 MB stack reserved by default on 64-bit JVMs — Java Virtual Machines, the programs that run Java bytecode); virtual threads are much cheaper still
Communication	IPC: sockets, pipes, files	Shared objects — fast, but needs synchronization
Failure isolation	Strong: one crash does not kill others	Weak: an unhandled error can bring down the whole process (e.g. `OutOfMemoryError`)
Context switch	Heavier (address-space switch)	Lighter
Java	`ProcessBuilder`, `Runtime.exec`	`Thread`, executors

Java example

```java
// A new PROCESS (separate memory):
Process p = new ProcessBuilder("ls", "-l").inheritIO().start();
p.waitFor();

// A new THREAD (shares this JVM's heap):
Thread t = new Thread(() -> System.out.println("same heap, different stack"));
t.start();
t.join();
```

Trade-offs

Threads: fast sharing, but every shared mutable field is a potential bug. Processes: safe isolation, but communication is slower and needs **serialization** (turning objects into bytes so they can be sent).

When not to use it

Do not reach for threads when isolation matters more than speed (running untrusted plugins, running a crash-prone **native library** — machine code, e.g. written in C, called from Java); use separate processes or **containers** (lightweight isolated environments such as Docker).

Common mistakes

- Thinking local variables are shared between threads. They are not: each thread has its own stack. Only objects reachable from shared references are shared.
- Creating one platform thread per request without limit — each has a large stack and an OS thread behind it. Use a pool, or virtual threads.

Real-world examples

- **Google Chrome** runs tabs in separate *processes* so one crashing page does not take down the browser.
- **PostgreSQL** (a popular SQL database) uses one process per connection; **MySQL** uses one thread per connection.
- A **JVM** is one process; your application's threads, **garbage-collector** threads (the GC frees memory that nothing uses any more) and **JIT-compiler** threads (Just-In-Time compiler: turns frequently run bytecode into fast machine code while the program runs) all live inside it.

In the interview

30-second answer: "A process is an isolated program with its own memory; a thread is a path of execution inside a process that shares its heap but has its own stack. Threads are cheaper and share data easily but need synchronization; processes are isolated and safer but communicate through IPC."

Follow-ups:

- *What does a thread own privately?* Its stack (locals, call frames), program counter, and `ThreadLocal` values (variables that hold a separate copy per thread).
- *Why are threads cheaper to switch?* They share the address space, so the CPU's memory mappings stay the same.

Self-check

- **Q:** Two threads each declare `int x = 0;` inside a method and increment it. Race? **A:** No — locals live on each thread's own stack.
- **Q:** Two threads increment the same object's field. Race? **A:** Yes, unless synchronized.

TL;DR

Process = isolated house with private memory. Thread = a worker inside it sharing the heap, owning only its stack. Share cheaply, synchronize carefully.

12.3 Thread lifecycle & states

What it is

A Java thread moves through well-defined states. `Thread.getState()` returns one of the six values of the enum `java.lang.Thread.State`:

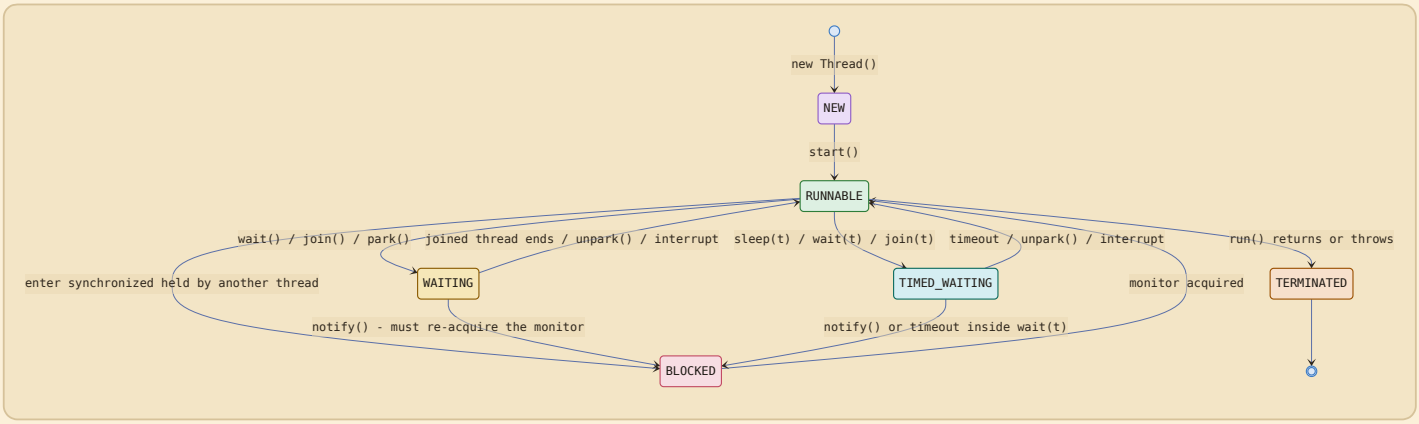
State	Meaning
`NEW`	Created with `new Thread(...)` but `start()` not called yet.
`RUNNABLE`	Started and either running on a CPU or ready to run (waiting only for the scheduler). Java does not distinguish "running" from "ready". A thread blocked in most I/O calls also shows as `RUNNABLE`.
`BLOCKED`	Waiting to *enter* a `synchronized` block/method whose monitor (the object's built-in lock, §12.7) another thread holds.
`WAITING`	Waiting with no time limit for another thread to act: `Object.wait()`, `Thread.join()`, `LockSupport.park()` (the low-level JDK call that suspends a thread; `ReentrantLock`, `Condition`, `CountDownLatch` use it).
`TIMED_WAITING`	Same, but with a timeout: `Thread.sleep(ms)`, `wait(ms)`, `join(ms)`, `parkNanos`, `tryLock(timeout)`.
`TERMINATED`	`run()` has returned or thrown. A thread cannot be restarted.

Analogy. A customer at a bank: `NEW` = has a ticket but hasn't entered; `RUNNABLE` = at the counter or next in line; `BLOCKED` = the only pen is being used, waiting to grab it; `WAITING` = told "wait until we call you"; `TIMED_WAITING` = "come back in 10 minutes"; `TERMINATED` = left the bank.

The problem it solves

Knowing the states lets you read **thread dumps** (a snapshot of every thread's state and call stack, taken with `jstack` or `jcmd <pid> Thread.print`, where pid is the process ID) and diagnose hangs: many threads `BLOCKED` on the same monitor → lock contention or **deadlock** (threads waiting on each other forever, §12.17); many `WAITING` on a queue → idle pool.

How it works



1. `new Thread(task)` → `NEW`.
2. `start()` → `RUNNABLE` (calling `start()` twice throws `IllegalThreadStateException`).
3. Trying to enter a busy `synchronized` block → `BLOCKED` until the monitor is free.
4. Calling `wait()`, `join()` or parking → `WAITING`; the timed versions → `TIMED_WAITING`.
5. A thread that wakes from `wait()` must first re-acquire the monitor, so it may pass through `BLOCKED` before returning to `RUNNABLE`.
6. When `run()` ends → `TERMINATED`.

Interruption. `t.interrupt()` sets a flag on `t`. If `t` is in `sleep`, `wait`, `join` (or other interruptible blocking calls), it wakes up with `InterruptedException` and the flag is cleared. Interruption is *cooperative*: the thread decides how to react. `Thread.stop()` is unsafe and no longer works on modern JDKs — never rely on it.

Java example

See `threadStates()` in the next section's `Main.java`: it drives one thread into each state and prints `getState()` (output lines under "Thread.State tour").

Trade-offs

Not applicable (it's a model, not a choice) — but note the `RUNNABLE` state hides whether the thread is *actually* on a CPU or stuck in native I/O.

When not to use it

Never use `getState()` for program logic (e.g. "if thread is WAITING then…"); states change between the check and your action. Use it only for monitoring and debugging.

Common mistakes

- Calling `run()` instead of `start()`: the code runs on the *current* thread; no new thread is created.
- Swallowing `InterruptedException` silently. Either rethrow it or restore the flag with `Thread.currentThread().interrupt()`.
- Thinking `BLOCKED` and `WAITING` are the same. `BLOCKED` = waiting for a *monitor*; `WAITING` = waiting for a *signal*.

Real-world examples

Production incident triage: a `jstack` dump showing 200 Tomcat worker threads `BLOCKED` on one `synchronized` logger or connection-pool method is a classic sign of a coarse lock bottleneck.

In the interview

> **30-second answer:** "Java threads have six states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED. BLOCKED means waiting for a monitor lock; WAITING/TIMED_WAITING mean waiting for another thread's action, optionally with a timeout."

Follow-ups:

- *Is there a RUNNING state?* Not in Java's enum — running and ready are both `RUNNABLE`.
- *What state is a thread in during `ReentrantLock.lock()` when the lock is taken?* `WAITING` (it parks via `LockSupport`), not `BLOCKED` — `BLOCKED` is only for intrinsic monitors.
- *Can you restart a terminated thread?* No; create a new one (or submit the task to a pool).

Self-check

- **Q:** State of a thread inside `Thread.sleep(1000)`? **A:** `TIMED_WAITING`.
- **Q:** State of a thread inside `latch.await()`? **A:** `WAITING`.

TL;DR

NEW → RUNNABLE ↔ (BLOCKED | WAITING | TIMED_WAITING) → TERMINATED. BLOCKED = waiting for a monitor; WAITING = waiting for a signal. Use `start()`, never `run()`, and respect interrupts.

12.4 Creating threads: `Thread`, `Runnable`, `Callable`, `ExecutorService`, virtual threads

What it is

Five ways to run code concurrently in Java:

Way	What you write	Returns a value?	Use when
Extend `Thread`	`class T extends Thread { run() }`	No	Almost never — mixes the task with the worker
`Runnable`	`() -> { ... }` passed to `new Thread(r)` or a pool	No	A fire-and-forget task
`Callable<V>`	`() -> value` (may throw checked exceptions — ones the compiler forces you to catch or declare)	Yes, via `Future<V>`	You need a result or an exception back
`ExecutorService`	`pool.submit(task)`	`Future<V>`	Default choice: reuse threads, limit concurrency
Virtual threads (Java 21)	`Thread.ofVirtual().start(r)` / `Executors.newVirtualThreadPerTaskExecutor()`	via `Future`	Very many *blocking* (I/O-bound) tasks

> **Analogy.** A `Runnable` is a job description. A `Thread` is a worker. An `ExecutorService` is a staffing agency that keeps a team of workers and hands them job descriptions. A `Future` is the receipt you use to collect the finished job. Virtual threads are like freelancers who cost almost nothing to hire, so you hire one per job.

The problem it solves

Creating an OS thread per task is expensive and unbounded. Executors separate **what** to run (task) from **how** it runs (pooling, queueing, scheduling). Virtual threads remove the cost problem for blocking code, so you can go back to a simple "one thread per request" style.

How it works

1. Write the task as a `Runnable` (no result) or `Callable<V>` (result/exception).
2. Submit it to an `ExecutorService` (e.g. `Executors.newFixedThreadPool(n)`); you get a `Future<V>`.
3. `future.get()` blocks until the result is ready (or rethrows the task's exception wrapped in `ExecutionException`).
4. When done, `shutdown()` the pool (stop accepting tasks) and `awaitTermination(...)`; or, since Java 19, use **try-with-resources** (a `try (...)` block that closes its resource automatically at the end) — `ExecutorService.close()` waits for tasks to finish.
5. **Virtual threads** (final in Java 21, JEP 444 — a JEP, JDK Enhancement Proposal, is the numbered design document for a Java feature) are lightweight threads managed by the JVM. Many virtual threads run on a small pool of OS "carrier" threads (a `ForkJoinPool` sized to the number of cores by default). When a virtual thread blocks on I/O, `sleep`, or a `java.util.concurrent` lock, the JVM *unmounts* it and the carrier runs another virtual thread. The exception is **pinning**: a virtual thread that cannot unmount keeps its carrier busy while it blocks. On Java 21–23 this happens when it blocks inside a `synchronized` block/method (including `Object.wait()`) or inside a native method; JDK 24 (JEP 491) removed the `synchronized` case, leaving mainly native/foreign-function calls (and blocking inside a class initializer) as pinning causes.

Java example

```java
// File: code/lld/ch12/threads/Main.java
package lld.ch12.threads;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;
import java.util.stream.IntStream;

/** Five ways to run code on another thread, plus a tour of every Thread.State. */
public class Main {

    // Way 1: subclass Thread (works, but couples "the task" with "the worker").
    static class GreeterThread extends Thread {
        GreeterThread() { super("greeter-thread"); }
        @Override public void run() {
            System.out.println("1) extends Thread   -> hello from " + getName());
        }
    }

    public static void main(String[] args) throws Exception {
        creatingThreads();
        virtualThreads();
        threadStates();
    }

    static void creatingThreads() throws Exception {
        System.out.println("=== Creating threads ===");
        Thread t1 = new GreeterThread();
        t1.start();          // start() creates a new call stack; run() would run on *this* thread
        t1.join();           // wait for it to finish so output order is predictable

        // Way 2: Runnable = a task with no result. Preferred over subclassing Thread.
        Runnable task = () -> System.out.println("2) Runnable         -> hello from " +
Thread.currentThread().getName());
        Thread t2 = new Thread(task, "runnable-thread");
        t2.start();
        t2.join();

        // Way 3 + 4: Callable (returns a value / throws) submitted to an ExecutorService (a thread pool).
        ExecutorService pool = Executors.newFixedThreadPool(2);
        try {
            Callable<Integer> sumTask = () -> IntStream.rangeClosed(1, 100).sum();
            Future<Integer> future = pool.submit(sumTask);   // returns immediately
            System.out.println("3) Callable+Future  -> sum 1..100 = " + future.get()); // get() blocks until done

            List<Callable<String>> jobs = List.of(() -> "A", () -> "B", () -> "C");
            List<String> results = new ArrayList<>();
            for (Future<String> f : pool.invokeAll(jobs)) results.add(f.get());   // invokeAll keeps input order
            System.out.println("4) ExecutorService  -> invokeAll results " + results);
        } finally {
            pool.shutdown();                                  // stop accepting work
            pool.awaitTermination(5, TimeUnit.SECONDS);       // wait for running tasks
        }
    }

    static void virtualThreads() throws Exception {
        System.out.println("=== Virtual threads (Java 21) ===");
        Thread vt = Thread.ofVirtual().name("virtual-1").start(
                () -> System.out.println("5) virtual thread   -> isVirtual=" +
Thread.currentThread().isVirtual()));
        vt.join();
```

```
        // 10,000 blocking tasks: cheap with virtual threads (each sleep parks the virtual thread,
        // freeing its carrier OS thread for other work).
        long start = System.nanoTime();
        int completed;
        try (ExecutorService vexec = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<Integer>> fs = new ArrayList<>();
            for (int i = 0; i < 10_000; i++) {
                fs.add(vexec.submit(() -> { Thread.sleep(50); return 1; }));
            }
            int sum = 0;
            for (Future<Integer> f : fs) sum += f.get();
            completed = sum;
        } // close() waits for all tasks
        long ms = (System.nanoTime() - start) / 1_000_000;
        System.out.println("10,000 tasks that each sleep 50 ms completed: " + completed
                + " (well under 10,000 x 50 ms = 500 s: " + (ms < 60_000) + ")");
    }

    static void threadStates() throws Exception {
        System.out.println("=== Thread.State tour ===");
        Object monitor = new Object();

        Thread sleeper = new Thread(() -> {
            try { Thread.sleep(10_000); } catch (InterruptedException e) { /* woken early: fine */ }
        }, "sleeper");
        System.out.println("after new Thread()            : " + sleeper.getState());      // NEW
        sleeper.start();
        awaitState(sleeper, Thread.State.TIMED_WAITING);
        System.out.println("inside Thread.sleep(10s)      : " + sleeper.getState());      // TIMED_WAITING
        sleeper.interrupt();
        sleeper.join();
        System.out.println("after run() returns           : " + sleeper.getState());      // TERMINATED

        CountDownLatch gate = new CountDownLatch(1);
        Thread waiter = new Thread(() -> {
            try { gate.await(); } catch (InterruptedException ignored) { }
        }, "waiter");
        waiter.start();
        awaitState(waiter, Thread.State.WAITING);
        System.out.println("inside latch.await()          : " + waiter.getState());       // WAITING
        gate.countDown();
        waiter.join();

        Thread blocked;
        synchronized (monitor) {             // main holds the monitor...
            blocked = new Thread(() -> { synchronized (monitor) { /* got it */ } }, "blocked");
            blocked.start();
            awaitState(blocked, Thread.State.BLOCKED);
            System.out.println("waiting to enter synchronized : " + blocked.getState()); // BLOCKED
        }                                    // ...released here, so 'blocked' can proceed
        blocked.join();

        java.util.concurrent.atomic.AtomicBoolean stop = new java.util.concurrent.atomic.AtomicBoolean();
        Thread spinner = new Thread(() -> { while (!stop.get()) Thread.onSpinWait(); }, "spinner");
        spinner.start();
        System.out.println("busy running on a CPU         : " + spinner.getState());      // RUNNABLE
        stop.set(true);
        spinner.join();
    }

    /** Poll until the thread reaches the wanted state (max ~5 s). */
    static void awaitState(Thread t, Thread.State wanted) throws InterruptedException {
        long deadline = System.nanoTime() + TimeUnit.SECONDS.toNanos(5);
        while (t.getState() != wanted && System.nanoTime() < deadline) Thread.sleep(1);
    }
}
```

Explaining each part:

- `GreeterThread` subclasses `Thread` — shown only for completeness. Prefer passing a `Runnable`.
- `t1.start()` creates the new call stack; `t1.join()` waits for it to finish, which also guarantees that everything `t1` wrote is visible to `main` afterwards (a *happens-before* edge, section 12.6).
- `pool.submit(sumTask)` returns a `Future` immediately; `future.get()` blocks.
- `invokeAll` runs a list of `Callable`s and returns futures **in the same order** as the input list.
- `shutdown()` + `awaitTermination()` is the classic clean shutdown. `newVirtualThreadPerTaskExecutor()` in try-with-resources uses `close()`, which waits for all tasks.
- 10,000 sleeping tasks finish quickly: each `sleep` parks its virtual thread instead of holding an OS thread.
- `threadStates()` drives threads into each state (NEW, TIMED_WAITING, TERMINATED, WAITING, BLOCKED, RUNNABLE). `awaitState` polls, because a state change is not instant after `start()`.

Output:

```
=== Creating threads ===
1) extends Thread   -> hello from greeter-thread
2) Runnable         -> hello from runnable-thread
3) Callable+Future  -> sum 1..100 = 5050
4) ExecutorService  -> invokeAll results [A, B, C]
=== Virtual threads (Java 21) ===
5) virtual thread   -> isVirtual=true
10,000 tasks that each sleep 50 ms completed: 10000 (well under 10,000 x 50 ms = 500 s: true)
=== Thread.State tour ===
after new Thread()            : NEW
inside Thread.sleep(10s)      : TIMED_WAITING
after run() returns           : TERMINATED
inside latch.await()          : WAITING
waiting to enter synchronized : BLOCKED
busy running on a CPU         : RUNNABLE
```

Trade-offs

	Platform thread pool	**Virtual threads**
Cost per thread	High (OS thread + large stack)	Very low (heap-allocated stack that grows as needed)
How many	Hundreds to a few thousand	Millions
Best for	CPU-bound work (size ≈ cores)	Blocking I/O-bound work
Limits concurrency?	Yes, by pool size	No — add a `Semaphore` if a downstream resource needs a limit
Pooling	Yes	No — never pool virtual threads; create one per task

When not to use it

- Do not use virtual threads for CPU-heavy computation — they give no extra cores; use a fixed pool or Fork/Join.
- On Java 21–23, a virtual thread that blocks *inside a* `synchronized` *block* stays **pinned** to its carrier thread, reducing scalability; prefer `ReentrantLock` around blocking calls on those versions. (JDK 24's JEP 491 removed most of this pinning.)
- Do not use `Executors.newCachedThreadPool()` for unbounded incoming traffic — it can create unlimited platform threads.

Common mistakes

- Forgetting `shutdown()`: non-daemon pool threads keep the JVM alive after `main` ends (a **daemon** thread is a background thread that does not keep the JVM running; ordinary threads do).
- Ignoring the `Future` of a submitted `Runnable`: exceptions thrown inside are captured in the `Future` and silently lost if you never call `get()`.
- Using `Executors.newFixedThreadPool` for untrusted load: its queue is **unbounded** (`LinkedBlockingQueue`), so a burst can exhaust memory. Build a `ThreadPoolExecutor` with a bounded queue (section 12.21).

Real-world examples

- `Executors` factory methods are the textbook **Factory** pattern in the JDK.
- **Spring** (the most widely used Java application framework): its `@Async` runs methods on a `TaskExecutor` (commonly a `ThreadPoolTaskExecutor`).
- Spring Boot 3.2+ can run request handling on virtual threads with `spring.threads.virtual.enabled=true`; Tomcat and Jetty can be configured to use virtual threads too.

In the interview

> **30-second answer:** "I never manage raw threads in business code. I describe work as `Runnable` / `Callable` and submit it to an `ExecutorService` sized for the workload: about cores for CPU-bound work, bounded queue with a rejection policy for incoming traffic. For many blocking I/O tasks on Java 21, I use virtual threads — one per task — and protect scarce resources with a semaphore." (A **semaphore** is a counter of permits that caps how many threads may proceed at once — §12.12.)

Follow-ups:

- *`Runnable` vs `Callable`?* `Callable` returns a value and may throw checked exceptions; `Runnable` does neither.
- *`submit` vs `execute`?* `execute(Runnable)` returns nothing and uncaught exceptions go to the thread's handler; `submit` returns a `Future` that captures the exception.
- *What does `CompletableFuture` add?* Non-blocking composition: `thenApply`, `thenCombine`, `allOf`, error handling with `exceptionally`, without calling `get()` at each step.

Self-check

- **Q:** What happens if you call `thread.run()`? **A:** The code runs synchronously on the current thread.
- **Q:** Why not pool virtual threads? **A:** They are cheap to create; pooling adds nothing and hides the one-thread-per-task model. Limit concurrency with a semaphore instead.

TL;DR

Tasks = `Runnable` / `Callable`. Run them with an `ExecutorService`, get results via `Future`. Java 21 virtual threads make one-thread-per-blocking-task cheap. Always shut pools down.

Part B — What goes wrong

12.5 Race conditions & critical sections

What it is

- A **race condition** is a bug where the result depends on the unlucky timing (interleaving) of threads.
- A **critical section** is the code that touches shared mutable state and must therefore run in only one thread at a time.
- A **data race** (the precise Java Memory Model term) is when two threads access the same variable, at least one writes, and there is no happens-before ordering between them (section 12.6).

> **Analogy.** Two people share a bank account with ₹100. Both check the balance at the same moment, both see ₹100, both withdraw ₹80. The bank gave out ₹160 from ₹100. The "check the balance, then withdraw" steps were the critical section.

The problem it solves

Understanding races tells you *which* code needs protection and *why* it goes wrong. The two classic shapes:

Shape	Example	Why it breaks
Read-modify-write	`count++`	It is really `tmp = count; tmp = tmp + 1; count = tmp`. Two threads read the same old value; one update is lost.
Check-then-act	`if (!map.containsKey(k)) map.put(k, v);` or `if (seat.isFree()) seat.book(me);`	The condition can become false between the check and the act.

How it works

1. Find all **shared mutable state** (fields reachable by more than one thread and changed after construction).
2. Find every **invariant** (a rule about the data that must always hold true) involving it (e.g. "a seat has at most one owner", "balance ≥ 0").
3. Mark the code that reads and then updates that state as a **critical section**.
4. Make each critical section **atomic** — with a lock (`synchronized`, `ReentrantLock`), with a single atomic operation (`AtomicInteger.incrementAndGet`, `ConcurrentHashMap.putIfAbsent`), or by removing the sharing (immutability — objects that never change; confinement — data only one thread touches; see §12.24).

Java example

```java
// File: code/lld/ch12/race/Main.java
package lld.ch12.race;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

/** Race conditions, three fixes, and the visibility problem solved with volatile. */
public class Main {

    interface Counter { void increment(); int get(); }

    /** BROKEN: count++ is read-modify-write (3 steps), so concurrent increments get lost. */
    static class UnsafeCounter implements Counter {
        private int count;
        public void increment() { count++; }
        public int get() { return count; }
    }

    /** Fix 1: a mutex (the object's intrinsic lock) makes the critical section atomic. */
    static class SynchronizedCounter implements Counter {
        private int count;
        public synchronized void increment() { count++; }
        public synchronized int get() { return count; }   // reads need the lock too (visibility)
    }

    /** Fix 2: a lock-free atomic built on compare-and-swap. */
    static class AtomicCounter implements Counter {
        private final AtomicInteger count = new AtomicInteger();
        public void increment() { count.incrementAndGet(); }
        public int get() { return count.get(); }
    }

    static final int THREADS = 4, PER_THREAD = 250_000;

    static int hammer(Counter c) throws InterruptedException {
        List<Thread> ts = new ArrayList<>();
        for (int i = 0; i < THREADS; i++) {
            Thread t = new Thread(() -> { for (int j = 0; j < PER_THREAD; j++) c.increment(); });
            ts.add(t);
            t.start();
        }
        for (Thread t : ts) t.join();   // join() also creates a happens-before edge: we see their writes
        return c.get();
    }

    // ---------- Visibility ----------
    static class Worker implements Runnable {
        private volatile boolean running = true;   // without volatile the loop may never see 'false'
        private long iterations;
        public void run() { while (running) iterations++; }
        void stop() { running = false; }
    }

    public static void main(String[] args) throws Exception {
        int expected = THREADS * PER_THREAD;
        System.out.println("expected                : " + expected);
        int unsafe = hammer(new UnsafeCounter());
        System.out.println("unsafe counter          : " + (unsafe == expected ? "correct this run (luck!)" : "LOST
" + (expected - unsafe) + " updates"));
        System.out.println("synchronized counter    : " + hammer(new SynchronizedCounter()));
        System.out.println("atomic counter          : " + hammer(new AtomicCounter()));

        // Check-then-act race, shown safely: two threads both "see" an empty seat.
        // Here we fix it with a single atomic operation instead of check + act.
        AtomicInteger seatOwner = new AtomicInteger(0);          // 0 = free
```

```
        Thread a = new Thread(() -> seatOwner.compareAndSet(0, 1));
        Thread b = new Thread(() -> seatOwner.compareAndSet(0, 2));
        a.start(); b.start(); a.join(); b.join();
        System.out.println("seat has exactly one owner: " + (seatOwner.get() == 1 || seatOwner.get() == 2));

        Worker w = new Worker();
        Thread t = new Thread(w);
        t.start();
        Thread.sleep(50);
        w.stop();                       // volatile write ...
        t.join(2_000);                  // ... is guaranteed to become visible to the worker
        System.out.println("volatile flag stopped worker: " + !t.isAlive());
    }
}
```

Explaining each part:

- `UnsafeCounter.increment()` does `count++` with no protection → updates get lost.
- `SynchronizedCounter` wraps the critical section in the object's intrinsic lock. Note that `get()` is synchronized too — that is for *visibility* (next section).
- `AtomicCounter` uses `incrementAndGet()`, a single hardware-backed atomic step (compare-and-swap, section 12.15).
- The seat example fixes a check-then-act race with one atomic call: `compareAndSet(0, id)` only succeeds if the seat is still free.
- `Worker` uses a `volatile` flag so the stop request is guaranteed to be seen (section 12.6).

Output (the number of lost updates changes every run — that unpredictability *is* the race):

```
expected                : 1000000
unsafe counter          : LOST 680044 updates
synchronized counter    : 1000000
atomic counter          : 1000000
seat has exactly one owner: true
volatile flag stopped worker: true
```

Trade-offs

Fix	Pros	Cons
Lock (`synchronized` / `ReentrantLock`)	Protects any multi-step, multi-variable invariant	Threads wait; risk of deadlock if nested
Atomic variable / atomic collection method	Fast, no blocking, no deadlock	Only covers one variable or one map entry
Immutability / confinement	No synchronization at all	Must redesign so state isn't shared or changed

When not to use it

Don't lock data that is never shared (method locals, objects confined to one thread) or never changed after safe construction (immutable objects).

Common mistakes

- Protecting writes but not reads ("reads are harmless") — a reader can see stale or half-updated state.
- Using two *separate* thread-safe calls and assuming the combination is atomic: `if (!map.containsKey(k))` `map.put(k,v)` on a `ConcurrentHashMap` is still a race. Use `putIfAbsent` / `computeIfAbsent`.
- Locking on different objects in different methods — each lock protects nothing against the other.
- Believing a race is fixed because tests pass. Races show up rarely and under load.

Real-world examples

- Double-booking in ticketing systems, overselling limited-stock items in **flash sales** (short, heavily promoted sales of limited stock), duplicate payment processing when a user double-clicks "Pay" — all check-then-act races, solved with atomic operations, locks, or database constraints (unique indexes that reject duplicate

values, `SELECT ... FOR UPDATE` — a SQL read that locks the rows it returns — and optimistic version columns: a version number checked on every update, so a write based on stale data fails).

In the interview

> **30-second answer:** "A race condition is when correctness depends on thread timing. The two common shapes are read-modify-write, like `count++`, and check-then-act, like 'if seat is free then book it'. I find the shared mutable state and its invariants, and make each critical section atomic — with a lock, an atomic operation like `putIfAbsent` or `compareAndSet`, or by removing sharing."

Follow-ups:

- *Is `volatile int count; count++` safe?* No. `volatile` gives visibility, not atomicity; the three steps can still interleave.
- *Is `ConcurrentHashMap` enough to make my class thread-safe?* Only if every invariant lives in single map operations. Multi-key invariants still need a lock or careful design.

Self-check

- **Q:** Why does `count++` lose updates? **A:** It is read, add, write — two threads can read the same value before either writes.
- **Q:** Name the race in `if (cache.get(k) == null) cache.put(k, load(k));`. **A:** Check-then-act; two threads may both load and put. Use `computeIfAbsent`.

TL;DR

Race = result depends on timing. Watch for read-modify-write and check-then-act. Make every critical section atomic, including reads.

12.6 Java Memory Model basics: visibility, happens-before, `volatile`

What it is

The **Java Memory Model (JMM)** (Java Language Specification, chapter 17) defines *when a write by one thread is guaranteed to be seen by a read in another thread*. Without the right synchronization, a thread may see a stale value — or never see the update at all.

- **Visibility**: will thread B *see* what thread A wrote?
- **Reordering**: compilers, the JIT, and CPUs may reorder instructions for speed as long as a *single* thread can't tell the difference. Other threads can.
- **Happens-before (HB)**: a guarantee that if action X *happens-before* action Y, then X's effects are visible to Y and X appears ordered before Y.
- `volatile`: a field modifier that makes reads/writes of that field visible to all threads and orders them (no caching in registers, no harmful reordering around it).

> **Analogy.** Each thread works from its own notepad (**CPU caches** — small, fast memories next to each core — and registers) and only sometimes copies to the shared whiteboard (main memory). Happens-before rules are the moments when the notepad is guaranteed to be synced with the whiteboard.

The problem it solves

The `Worker` loop `while (running) iterations++;` with a *non-volatile* `running` may never stop: the JIT is allowed to read `running` once and turn the loop into `while (true)`. Visibility bugs like this are invisible in code review unless you know the JMM.

How it works

The key **happens-before rules** (you can quote these in an interview):

Rule	Meaning
Program order	Within one thread, each action HB the next one.
Monitor lock	An **unlock** of a monitor HB every later **lock** of the *same* monitor.
Volatile	A **write** to a volatile field HB every later **read** of that field.
Thread start	`t.start()` HB every action in thread `t`.
Thread join	Every action in `t` HB another thread's successful return from `t.join()`.
Transitivity	If A HB B and B HB C, then A HB C.
Final fields	Strictly a separate guarantee (JLS §17.5), not a happens-before edge: once a constructor finishes without leaking `this`, any thread that obtains a reference to the object sees the correct values of its `final` fields (and of objects reachable through them, as of the end of the constructor) — even if the reference was shared without synchronization.
`java.util.concurrent`	E.g. putting into a `BlockingQueue` HB taking it out; `countDown()` HB a return from `await()`; `Future` completion HB `get()` returning.

Steps to apply them:

1. For each shared variable, ask: "Is every write-then-read pair connected by a happens-before edge?"
2. If they are protected by the same lock → yes (monitor rule).
3. If the variable is a single independent flag or reference → `volatile` is enough.
4. If it is a compound update (`x++`, check-then-act) → `volatile` is **not** enough; use a lock or atomic.

Safe publication. An object is *safely published* when the reference to it is made visible through an HB edge: stored in a `volatile` field (or an `AtomicReference`), stored in a `final` field of a properly constructed object, created by a static initializer, put into a concurrent collection, or written under a lock that the reader also takes. Otherwise a reader might see the reference but not the fully constructed object.

Double-checked locking (lazy Singleton — a class with exactly one shared instance, created only on first use) is only correct if the instance field is `volatile`:

```
class Config {
    private static volatile Config instance;          // volatile is essential
    static Config get() {
        Config c = instance;
        if (c == null) {
            synchronized (Config.class) {
                c = instance;
                if (c == null) instance = c = new Config();
            }
        }
        return c;
    }
}
```

(Simpler and just as lazy: the *holder idiom* — `private static class Holder { static final Config I = new Config(); }` — class initialization is thread-safe by the JLS.)

Java example

`Worker` in the race example above: `private volatile boolean running`. The write `running = false` in `main` HB the worker's next read of `running`, so the loop exits. The output line `volatile flag stopped worker: true` confirms it.

Trade-offs

Tool	Visibility	Atomic compound ops	Blocking	Cost
`volatile`	Yes	No	No	Low
`synchronized` / `Lock`	Yes (for code under the same lock)	Yes	Yes	Medium
`Atomic*`	Yes	Yes, for one variable	No	Low–medium
`final` (immutable)	Yes, after construction	N/A — never changes	No	Free

When not to use it

Don't use `volatile` for anything that combines several steps or several variables — use a lock or an atomic class.

Common mistakes

- "It works on my laptop" — x86 CPUs (Intel/AMD) reorder less than ARM ones (e.g. Apple Silicon, AWS Graviton). Code without proper HB edges may fail only on other hardware or after **JIT warm-up** (once the JIT has compiled and optimised the hot code).
- Leaking `this` from a constructor (registering a listener, starting a thread in the constructor) — other threads may see a half-built object, and final-field guarantees are lost.
- Thinking `synchronized` on the writer alone is enough. The reader must use the *same* lock.

Real-world examples

- `volatile boolean running` shutdown flags in server loops and schedulers.
- `ConcurrentHashMap`, `AtomicInteger`, `CopyOnWriteArrayList` internally use volatile reads/writes and CAS (compare-and-swap, §12.15) to give HB guarantees without locks for readers.
- The JDK's `String` is immutable with `final` fields, so strings can be shared across threads freely.

In the interview

> **30-second answer:** "The Java Memory Model says a thread is only guaranteed to see another thread's write if there's a happens-before edge — unlock-then-lock of the same monitor, a volatile write-then-read, thread start/join, or a concurrent-collection handoff. `volatile` gives visibility and ordering for a single variable but not atomicity; for compound actions I use locks or atomics."

Follow-ups:

- *Why must the double-checked-locking field be volatile?* Without it, the write of the reference can be reordered before the constructor's writes, so another thread might see a non-null but half-initialized object.
- *Does `volatile` make `long`/`double` writes atomic?* Yes — non-volatile 64-bit `long`/`double` writes may be split into two 32-bit halves by the JLS (rare on modern 64-bit JVMs, but allowed); volatile ones cannot.

Self-check

- **Q:** Thread A sets a non-volatile `ready = true` after preparing data; thread B spins on `ready`. Two possible bugs? **A:** B may never see `ready` become true, or may see `ready` true but stale data (reordering).
- **Q:** Which JDK classes give HB for you? **A:** Locks, `Atomic*`, concurrent collections, `BlockingQueue`, `CountDownLatch`, `Future`, `ExecutorService.submit`.

TL;DR

Writes are only guaranteed visible across threads through happens-before edges. `volatile` = visibility + ordering for one variable, not atomicity. Locks = visibility + atomicity. Publish objects safely.

Part C — Locks

12.7 Mutex (mutual exclusion) and `synchronized`

What it is

A **mutex** (mutual-exclusion lock) is a lock that at most one thread can hold. In Java every object has a built-in mutex called its **intrinsic lock** or **monitor**. The `synchronized` keyword acquires it:

```
public synchronized void deposit(int amt) { balance += amt; }   // locks 'this'
public static synchronized void log() { ... }                    // locks the Class object
void transfer() { synchronized (lock) { ... } }                  // locks 'lock'
```

Analogy. A single-occupancy restroom with one key on a hook. Take the key to enter; others wait; hang it back when you leave.

The problem it solves

Makes a critical section **atomic** (no interleaving) and **visible** (the next thread to take the lock sees everything done before the unlock).

How it works

1. Thread reaches `synchronized (obj)`. If nobody holds `obj`'s monitor, it takes it and continues.
2. If another thread holds it, this thread goes to `BLOCKED` until the monitor is free.
3. On leaving the block — normally *or by exception* — the monitor is released automatically.
4. Intrinsic locks are **reentrant**: the holder can enter other `synchronized` code on the same object again without blocking itself.

Java example

`SynchronizedCounter` in section 12.5, and `Mailbox` / `FooBarMonitor` style classes later. A tip: lock on a **private final** object, not on `this`, when you don't want outside code to be able to take your lock:

```
private final Object lock = new Object();
void add(Item i) { synchronized (lock) { items.add(i); } }
```

Trade-offs

Pros	Cons
Simplest syntax; auto-release even on exceptions	No timeout, no `tryLock`, not interruptible while waiting
JVM-optimised (uncontended locking is cheap)	One wait-set (the threads parked in `wait()`) per object (only one "condition")
Reentrant	No fairness option (fair = longest-waiting thread goes first); on Java 21–23 pins virtual threads when blocking inside

When not to use it

When you need a timeout, interruptible waiting, fairness, several condition queues, or read/write separation — use `ReentrantLock` / `ReentrantReadWriteLock`.

Common mistakes

- Synchronizing on a *changing* reference (`synchronized (list)` after `list = new ArrayList<>()`) — different threads lock different objects.
- Synchronizing on **boxed values** (objects such as `Integer` that wrap a primitive) or string literals (`synchronized ("LOCK")`, `synchronized (Integer.valueOf(1))`) — these may be shared JVM-wide, causing mysterious contention or deadlocks.
- Doing slow I/O while holding the lock.

Real-world examples

`Vector`, `Hashtable`, `StringBuffer` and `Collections.synchronizedList(...)` wrap every method in `synchronized` — thread-safe per call but slow under contention, which is why `ArrayList`/`HashMap`/`StringBuilder` plus concurrent collections replaced them.

In the interview

> **30-second answer:** "A mutex lets one thread at a time into a critical section. In Java, `synchronized` uses the object's monitor: it's reentrant, released automatically, and gives visibility. I lock on a private final object and keep the section short."

Follow-ups:

- *What does a static synchronized method lock?* The `Class` object, e.g. `Counter.class` — a different lock from instance methods.
- *Is `synchronized` slow?* Uncontended, it's cheap. Under heavy contention, any lock is slow — reduce sharing or use finer locks.

Self-check

- **Q:** Two synchronized instance methods on *different* objects — do they block each other? **A:** No, different monitors.
- **Q:** Does an exception inside `synchronized` release the lock? **A:** Yes.

TL;DR

Mutex = one holder at a time. `synchronized` is Java's built-in, reentrant, auto-releasing mutex. Lock a private final object; keep the section short.

12.8 `ReentrantLock` and reentrancy

What it is

`java.util.concurrent.locks.ReentrantLock` is an explicit mutex with the same basic meaning as `synchronized` plus extra features. **Reentrant** means the thread holding the lock can acquire it again (a hold count goes up) and must release it the same number of times.

> **Analogy.** A room key that remembers who has it: if you already hold it, walking into an inner door of the same room doesn't make you wait for yourself.

The problem it solves

- Without reentrancy, a locked method calling another locked method of the same object would **deadlock with itself**.
- Compared with `synchronized`, `ReentrantLock` adds: `tryLock()` / `tryLock(timeout)`, `lockInterruptibly()`, optional **fairness** (`new ReentrantLock(true)`), multiple `Condition`s per lock, and queries like `isLocked()`, `getHoldCount()`.

How it works

1. `lock.lock()` — acquire (hold count 0 → 1), or wait if another thread holds it.
2. Same thread calls `lock.lock()` again → hold count 2; no waiting.
3. Each `unlock()` decrements; at 0 the lock is released and a waiting thread may proceed.
4. **Always** release in `finally`:

```
lock.lock();
try {
    // critical section
} finally {
    lock.unlock();
}
```

Java example

```java
// File: code/lld/ch12/locks/Main.java
package lld.ch12.locks;

import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReadWriteLock;
import java.util.concurrent.locks.ReentrantLock;
import java.util.concurrent.locks.ReentrantReadWriteLock;
import java.util.concurrent.locks.StampedLock;

/** synchronized vs ReentrantLock, reentrancy, tryLock, ReadWriteLock, StampedLock. */
public class Main {

    // ---------- Reentrancy ----------
    static class Account {
        private final ReentrantLock lock = new ReentrantLock();
        private int balance = 100;

        void withdraw(int amount) {
            lock.lock();
            try {
                if (canWithdraw(amount)) balance -= amount;   // calls another locked method
            } finally {
                lock.unlock();                                // ALWAYS unlock in finally
            }
        }
        boolean canWithdraw(int amount) {
            lock.lock();                                      // same thread re-acquires: no self-deadlock
            try {
                System.out.println("hold count inside nested call = " + lock.getHoldCount());
                return balance >= amount;
            } finally {
                lock.unlock();
            }
        }
        int balance() { lock.lock(); try { return balance; } finally { lock.unlock(); } }
    }

    // ---------- Read-write lock: many readers OR one writer ----------
    static class ConfigStore {
        private final Map<String, String> map = new HashMap<>();
        private final ReadWriteLock rw = new ReentrantReadWriteLock();
        private final Lock r = rw.readLock(), w = rw.writeLock();

        String get(String k) { r.lock(); try { return map.get(k); } finally { r.unlock(); } }
        void put(String k, String v) { w.lock(); try { map.put(k, v); } finally { w.unlock(); } }

        /** Used by the demo: prove several readers are inside the read lock at once. */
        <T> T readWhile(java.util.concurrent.Callable<T> action) throws Exception {
            r.lock(); try { return action.call(); } finally { r.unlock(); }
        }
    }

    // ---------- StampedLock optimistic read ----------
    static class Point {
        private final StampedLock sl = new StampedLock();
        private double x, y;

        void move(double dx, double dy) {
            long stamp = sl.writeLock();
```

```
            try { x += dx; y += dy; } finally { sl.unlockWrite(stamp); }
        }
        double distanceFromOrigin() {
            long stamp = sl.tryOptimisticRead();   // no locking at all, just a version number
            double cx = x, cy = y;                 // copy fields to locals
            if (!sl.validate(stamp)) {             // a writer slipped in: fall back to a real read lock
                stamp = sl.readLock();
                try { cx = x; cy = y; } finally { sl.unlockRead(stamp); }
            }
            return Math.hypot(cx, cy);
        }
    }

    public static void main(String[] args) throws Exception {
        System.out.println("=== Reentrancy ===");
        Account acc = new Account();
        acc.withdraw(30);
        System.out.println("balance = " + acc.balance());

        System.out.println("=== tryLock with timeout ===");
        ReentrantLock printer = new ReentrantLock();
        CountDownLatch mainHolds = new CountDownLatch(1);
        printer.lock();                            // main grabs the printer and keeps it
        try {
            Thread impatient = new Thread(() -> {
                try {
                    mainHolds.await();
                    if (printer.tryLock(100, TimeUnit.MILLISECONDS)) {
                        try { System.out.println("worker: got the printer"); } finally { printer.unlock(); }
                    } else {
                        System.out.println("worker: printer busy for 100 ms, doing something else instead");
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            impatient.start();
            mainHolds.countDown();
            impatient.join();
        } finally {
            printer.unlock();
        }
        System.out.println("tryLock() now (free)  = " + printer.tryLock());
        printer.unlock();

        System.out.println("=== ReadWriteLock: readers share ===");
        ConfigStore store = new ConfigStore();
        store.put("mode", "fast");
        int readers = 3;
        CyclicBarrier allReadersInside = new CyclicBarrier(readers);
        Thread[] rs = new Thread[readers];
        boolean[] ok = new boolean[readers];
        for (int i = 0; i < readers; i++) {
            final int id = i;
            rs[i] = new Thread(() -> {
                try {
                    ok[id] = store.readWhile(() -> {
                        // If the read lock were exclusive, the barrier would time out.
                        allReadersInside.await(2, TimeUnit.SECONDS);
                        return "fast".equals(store.get("mode"));   // read lock is reentrant too
                    });
                } catch (TimeoutException | BrokenBarrierException e) {
                    ok[id] = false;
                } catch (Exception e) { throw new RuntimeException(e); }
            });
            rs[i].start();
        }
        for (Thread t : rs) t.join();
        System.out.println(readers + " readers held the read lock at the same time: " + (ok[0] && ok[1] && ok[2]));

        System.out.println("=== StampedLock optimistic read ===");
```

```
        Point p = new Point();
        p.move(3, 4);
        System.out.println("distance = " + p.distanceFromOrigin());
    }
}
```

Explaining each part:

- `Account.withdraw` locks, then calls `canWithdraw`, which locks again: `getHoldCount()` prints 2 — reentrancy in action.
- "tryLock with timeout": `main` holds `printer`; the worker waits at most 100 ms, then does something else (section 12.9).
- `ConfigStore` uses a `ReentrantReadWriteLock`. Three readers all wait at a `CyclicBarrier` *while holding the read lock*; if the read lock were exclusive, the barrier would time out. It prints `true` (section 12.10).
- `Point` uses `StampedLock.tryOptimisticRead()`: read without locking, then `validate(stamp)`; fall back to a real read lock if a writer interfered (section 12.10).

Output:

```
=== Reentrancy ===
hold count inside nested call = 2
balance = 70
=== tryLock with timeout ===
worker: printer busy for 100 ms, doing something else instead
tryLock() now (free)  = true
=== ReadWriteLock: readers share ===
3 readers held the read lock at the same time: true
=== StampedLock optimistic read ===
distance = 5.0
```

Trade-offs

Feature	`synchronized`	`ReentrantLock`
Release	Automatic	Manual (`finally`) — easy to forget
Try / timeout	No	`tryLock()`, `tryLock(t, unit)`
Interruptible wait	No	`lockInterruptibly()`
Fairness	No	Optional (`true` = longest waiter first; lower throughput)
Conditions	One wait set	Many `Condition` objects
Reentrant	Yes	Yes

When not to use it

If you need none of the extra features, `synchronized` is shorter and can't be left locked by mistake.

Common mistakes

- `lock()` *inside* the `try` — if `lock()` threw, `finally` would `unlock()` a lock you don't hold (`IllegalMonitorStateException`). Call `lock()` just before `try`.
- Forgetting `unlock()` on an early `return` or exception path — the lock stays held forever.
- Turning on fairness "to be safe" — fair locks are noticeably slower; use them only when starvation is a real problem.

Real-world examples

`ArrayBlockingQueue`, `LinkedBlockingQueue` and `ThreadPoolExecutor` use `ReentrantLock` internally (`CopyOnWriteArrayList` did up to Java 8; since Java 9 it uses `synchronized` on an internal lock object). `ReentrantLock`, `ReentrantReadWriteLock`, `Semaphore` and `CountDownLatch` are all built on `AbstractQueuedSynchronizer` (AQS — the JDK's shared framework for building locks and synchronizers); `CyclicBarrier` is built on a `ReentrantLock` + `Condition`.

In the interview

> **30-second answer:** "`ReentrantLock` is an explicit mutex with the same semantics as `synchronized` plus try-lock with timeout, interruptible acquisition, optional fairness, and multiple conditions. Reentrant means the owner can re-acquire it; the hold count must drop to zero to release. I always unlock in `finally`."

Follow-ups:

- *Why would a non-reentrant lock deadlock?* A method holding it that calls another method needing the same lock would wait for itself forever.
- *Is `StampedLock` reentrant?* No — e.g. a thread that holds its write lock and calls `writeLock()` or `readLock()` again blocks forever on itself. It also has no owner and no `Condition`s.

Self-check

- **Q:** A thread calls `lock()` 3 times and `unlock()` 2 times. Is the lock free? **A:** No; hold count is 1.
- **Q:** What does `new ReentrantLock(true)` change? **A:** Waiting threads get the lock in roughly FIFO (first in, first out) order (fair), at a throughput cost.

TL;DR

`ReentrantLock` = `synchronized` + tryLock/timeout/interruptible/fairness/conditions. Owner can re-enter. `lock(); try { … } finally { unlock(); }` always.

12.9 Try-lock & timed locking

What it is

- `tryLock()` — take the lock **only if it is free right now**; return `true`/`false` immediately.
- `tryLock(timeout, unit)` — wait up to the timeout; return `false` if still unavailable. Interruptible.

> **Analogy.** Knocking on a meeting room: if it's busy you don't stand there forever — you wait five minutes, then book another room.

The problem it solves

Plain `lock()` waits forever. That causes hung requests and makes deadlocks permanent. Try-lock lets a thread **give up and do something else** — fail fast, retry later, return "busy" to a user — and it breaks the *hold and wait* condition of deadlock (section 12.17).

How it works

1. Call `tryLock(timeout)`.
2. If it returns `true`: `try { work } finally { unlock(); }`.
3. If it returns `false`: take the fallback path (return an error, retry with **back-off** — waiting a short while before trying again — or release other locks you hold).
4. When acquiring several locks: if any `tryLock` fails, **release everything you already hold**, back off for a random short time, and retry.

Java example

In `locks/Main.java` above: the worker calls `printer.tryLock(100, TimeUnit.MILLISECONDS)` while `main` holds it, gets `false`, and prints the fallback message. In `deadlock/Main.java` (section 12.17), `transferTryLock` uses try-lock plus random back-off to avoid deadlock.

Trade-offs

Pros	Cons
No infinite waits; can report "busy"	Caller must handle failure paths
Breaks deadlock (hold-and-wait)	Retry loops without random back-off can **livelock**
Timeouts give predictable **latency** (time one operation takes)	Choosing the timeout is a judgement call

When not to use it

When failing is not acceptable and waiting is fine (e.g. a background **batch job** — one that processes data in bulk, with no user waiting — updating a counter) — plain `lock()` is simpler.

Common mistakes

- Calling `unlock()` when `tryLock` returned `false` → `IllegalMonitorStateException`.
- Retrying immediately in a tight loop — burns CPU and can livelock with another thread doing the same.
- Expecting untimed `tryLock()` to respect fairness — it *barges* (takes a free lock even if others are queued); `tryLock(0, unit)` respects fairness.

Real-world examples

- **Distributed locks** (locks shared by many servers, kept in an external store: Redis `SET key value NX PX 30000` — set only if absent, expire after 30,000 ms — Redisson's (a Java Redis client) `tryLock(wait, lease, unit)`, ZooKeeper (a coordination service for clusters) recipes) follow the same idea: try for a bounded time, give up, or let the lock expire.
- Seat-hold features on ticketing sites: if the seat is locked by someone else's checkout, you're told immediately rather than made to wait.

In the interview

> **30-second answer:** "`tryLock` with a timeout lets a thread give up instead of waiting forever. I use it when latency matters or when I need several locks: if I can't get all of them, I release what I hold, back off randomly and retry — which prevents deadlock."

Follow-ups:

- *Why random back-off?* Two threads retrying on the same schedule can keep colliding forever (livelock); randomness breaks the symmetry.

Self-check

- **Q:** What does `tryLock(0, SECONDS)` do? **A:** Tries once without waiting but, unlike `tryLock()`, respects a fair lock's queue.

TL;DR

Try-lock = acquire only if available (optionally within a timeout). Handle `false`, release everything on failure, back off randomly.

12.10 Read-write locks: `ReadWriteLock` and `StampedLock`

What it is

A **read-write lock** is a pair of locks: a shared **read lock** (many readers at once) and an exclusive **write lock** (one writer, no readers). Java provides `ReentrantReadWriteLock` and, since Java 8, `StampedLock` (adds cheap *optimistic reads*).

> **Analogy.** A museum exhibit: any number of visitors can look at it together, but when a curator needs to rearrange it, the room is cleared and only the curator is inside.

The problem it solves

With a plain mutex, readers block each other even though reading doesn't change anything. For **read-heavy** data (configuration, **caches** — fast copies of data kept for quick reads — price catalogues) a read-write lock lets reads proceed in parallel.

How it works

1. Reader: `readLock().lock()` — succeeds if no writer holds the write lock. Many readers can hold it at once.
2. Writer: `writeLock().lock()` — waits until there are **no readers and no writer**, then holds exclusively.
3. `ReentrantReadWriteLock` rules: a writer may **downgrade** (acquire the read lock while holding the write lock, then release the write lock). A reader may **not upgrade** (asking for the write lock while holding the read lock deadlocks — it waits for all readers, including itself).
4. `StampedLock` optimistic read: `long s = tryOptimisticRead()` (no locking) → read fields into locals → `validate(s)`. If a writer acquired the lock in between, `validate` returns `false` and you retry with a real `readLock()`.

Java example

`ConfigStore` and `Point` in `locks/Main.java` above. The output line `3 readers held the read lock at the same time: true` proves readers share the lock.

Trade-offs

	synchronized / ReentrantLock	ReentrantReadWriteLock	StampedLock
Parallel readers	No	Yes	Yes (and optimistic reads take no lock at all)
Reentrant	Yes	Yes	**No**
Conditions	Yes	Write lock only	No
Overhead	Lowest	Higher bookkeeping	Low for optimistic reads; more complex code
Best when	Short sections or write-heavy	Reads ≫ writes and reads are not trivially short	Very read-heavy, tiny critical sections (e.g. coordinates)

When not to use it

- Write-heavy or very short critical sections — the extra bookkeeping of a RW lock can make it *slower* than a plain mutex.
- When a copy-on-write snapshot is simpler: keep an immutable map in a `volatile` field and replace it on write (readers never lock; see 12.23).

Common mistakes

- Trying to upgrade read → write in `ReentrantReadWriteLock` (self-deadlock).
- Modifying state while holding only the *read* lock.
- Using data read under an optimistic stamp before calling `validate`, or reading fields directly instead of copying them to locals first.
- Writer starvation in non-fair mode under a constant stream of readers (construct it with `new ReentrantReadWriteLock(true)` if it matters; `StampedLock` gives no fairness guarantee either).

Real-world examples

- In-memory caches and configuration holders; routing tables that are read on every request and updated rarely.
- `ConcurrentHashMap` goes further: reads take no lock at all.

In the interview

> **30-second answer:** "A read-write lock lets many readers in together but gives writers exclusive access. It helps when reads vastly outnumber writes. `ReentrantReadWriteLock` allows downgrade but not upgrade; `StampedLock` adds optimistic reads that take no lock and are validated afterwards, but it's not reentrant."

Follow-ups:

- *How would you design a thread-safe config service?* Either a RW lock around a `HashMap`, or an immutable `Map` in a `volatile` field replaced atomically on reload (readers never block).

Self-check

- **Q:** Two readers and one writer arrive; which can run together? **A:** The two readers; the writer runs alone.
- **Q:** Why copy fields to locals in an optimistic read? **A:** So that after `validate` succeeds you use a consistent snapshot, not fields that might change after the check.

TL;DR

Many readers OR one writer. Use for read-heavy shared data. No upgrade in `ReentrantReadWriteLock`; `StampedLock` = optimistic, non-reentrant.

12.11 Coarse-grained vs fine-grained locking

What it is

- **Coarse-grained locking**: one lock protects a large amount of state (e.g. the whole parking lot).
- **Fine-grained locking**: many locks, each protecting a small piece (e.g. one lock per floor, per spot, or per **hash bucket** — one slot of the internal array a hash map spreads its entries over).

> **Analogy.** One key for an entire office building (coarse) vs a key per room (fine). One key is simple but everyone queues at the front door; room keys let people work in parallel but you must manage many keys.

The problem it solves

Choosing the right granularity balances **simplicity and safety** against **throughput under contention**.

How it works

1. Start coarse: one lock around the whole object. Measure.
2. If contention is the bottleneck, split state into **independent** parts whose invariants don't cross — each gets its own lock (**lock splitting**).
3. For a big collection, use **lock striping**: N locks; item *i* uses lock `hash(i) % N` (Java 7's `ConcurrentHashMap` had 16 segments by default; chapter 13 builds a striped map).
4. When an operation needs several fine locks (e.g. booking seats A1 and A2), always acquire them in a **global order** to avoid deadlock.
5. Keep critical sections short: do slow work (I/O, computation) *outside* the lock.

	Coarse-grained	Fine-grained
Simplicity	High	Lower — more locks, ordering rules
Concurrency	Low: one thread at a time	High: threads on different parts run together
Deadlock risk	Low (one lock)	Higher (many locks) — needs lock ordering
Overhead	One lock	Memory and bookkeeping for many locks
Cross-part invariants	Easy	Hard — may need to take several locks

Java example

- Coarse: `SynchronizedCounter` (12.5).
- Fine: `Show` in `lldcases/Main.java` (12.25) has **one lock per seat**, taken in sorted order; `StripedHashMap` in chapter 13 has one lock per stripe.

Trade-offs

See table above. Rule of thumb: *correct first, then fine-grained only where **profiling** (measuring where a program really spends its time) shows contention*.

When not to use it

Don't split locks when an invariant spans the parts (e.g. "total seats booked ≤ capacity" across all rows) unless you also handle the cross-part check.

Common mistakes

- Premature fine-graining → complexity and deadlocks without real benefit.
- Locking fine-grained parts in inconsistent order.
- "Fine-grained" locks that still funnel through a shared counter everyone updates (the counter becomes the **bottleneck** — the one slow point that limits the whole system — use `LongAdder`, a counter that spreads updates over several cells, §12.15).

Real-world examples

- `ConcurrentHashMap` (Java 8+) locks only the head node of one bin for writes — extremely fine-grained.
- Databases use row-level locks (fine) vs table-level locks (coarse).
- Python's GIL (Global Interpreter Lock) is a famous coarse lock: simple, but one thread runs Python **bytecode** (the compiled instructions the interpreter executes) at a time.

In the interview

> **30-second answer:** "I start with one lock for correctness, then refine where contention matters: per-floor or per-spot locks for a parking lot, per-show or per-seat locks for booking, striping for maps. Finer locks raise throughput but need a consistent lock order to avoid deadlock."

Follow-ups:

- *Per-show or per-seat locking for BookMyShow (an Indian movie-ticketing site)?* Per-show is simple and usually enough (contention is per show). Per-seat allows more parallelism but multi-seat bookings must lock seats in sorted order.

Self-check

- **Q:** What is lock striping? **A:** Using a fixed array of N locks and mapping each item to one by hash, to get fine-grained locking with bounded lock count.

TL;DR

Coarse = simple, low throughput. Fine = parallel, complex, deadlock-prone. Start coarse, refine with measurements, and order your locks.

Part D — Coordination

12.12 Semaphores

What it is

A **semaphore** holds a number of **permits**. `acquire()` takes one (waiting if none are left); `release()` gives one back. `java.util.concurrent.Semaphore` implements it. A semaphore with 1 permit behaves like a mutex, *except* it has no owner — any thread may release it.

> **Analogy.** A car park with 3 spaces and a display showing free spaces. Cars enter only if the number is above zero; leaving increases it.

The problem it solves

- **Limiting concurrency**: at most N threads use a resource (DB connections, **API** calls — calls to another service's programming interface — GPU slots).
- **Signaling** between threads: a semaphore with 0 permits acts as "wait until someone signals" (used heavily in chapter 13).

How it works

1. `new Semaphore(n)` — n permits. The default is **non-fair**: a thread calling `acquire()` may *barge* ahead of threads already waiting. `new Semaphore(n, true)` is **fair**: permits go to waiting threads in first-in-first-out order (at some throughput cost). Even on a fair semaphore, the untimed `tryAcquire()` barges; `tryAcquire(0, TimeUnit.SECONDS)` respects fairness.
2. `acquire()` — if permits > 0, decrement and continue; else wait.
3. `release()` — increment and wake a waiter.
4. Also: `tryAcquire()`, `tryAcquire(timeout, unit)`, `acquire(k)` for several permits.
5. Always `release()` in `finally` after a successful `acquire()`.

Java example

```java
// File: code/lld/ch12/coordination/Main.java
package lld.ch12.coordination;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

/** Semaphore, wait/notify, Condition, CountDownLatch, CyclicBarrier. */
public class Main {

    // ---------- Condition variable with the built-in monitor (wait / notifyAll) ----------
    /** A one-slot mailbox: put waits while full, take waits while empty. */
    static class Mailbox<T> {
        private T item;                                 // guarded by 'this'
        public synchronized void put(T x) throws InterruptedException {
            while (item != null) wait();                // ALWAYS wait in a loop (spurious wakeups)
            item = x;
            notifyAll();                                // wake the taker(s)
        }
        public synchronized T take() throws InterruptedException {
            while (item == null) wait();
            T x = item;
            item = null;
            notifyAll();                                // wake the putter(s)
            return x;
        }
    }

    // ---------- Condition variable with ReentrantLock + Condition ----------
    /** Two threads take turns: PING, PONG, PING, PONG ... */
    static class TurnTable {
        private final ReentrantLock lock = new ReentrantLock();
        private final Condition turnChanged = lock.newCondition();
        private boolean pingsTurn = true;

        void play(boolean iAmPing, String word, int rounds, List<String> out) throws InterruptedException {
            for (int i = 0; i < rounds; i++) {
                lock.lock();
                try {
                    while (pingsTurn != iAmPing) turnChanged.await();   // releases lock while waiting
                    out.add(word);
                    pingsTurn = !pingsTurn;
                    turnChanged.signalAll();
                } finally {
                    lock.unlock();
                }
            }
        }
    }

    public static void main(String[] args) throws Exception {
        semaphoreDemo();
        waitNotifyDemo();
        conditionDemo();
        latchDemo();
        barrierDemo();
    }
```

```
    static void semaphoreDemo() throws InterruptedException {
        System.out.println("=== Semaphore: at most 3 concurrent DB connections ===");
        Semaphore connections = new Semaphore(3);
        AtomicInteger inUse = new AtomicInteger(), maxSeen = new AtomicInteger();
        ExecutorService pool = Executors.newFixedThreadPool(10);
        for (int i = 0; i < 10; i++) {
            pool.submit(() -> {
                try {
                    connections.acquire();                      // blocks when 0 permits left
                    try {
                        int now = inUse.incrementAndGet();
                        maxSeen.accumulateAndGet(now, Math::max);
                        Thread.sleep(30);                       // "run a query"
                        inUse.decrementAndGet();
                    } finally {
                        connections.release();                  // give the permit back
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
        }
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("10 queries done, never more than 3 at once: " + (maxSeen.get() <= 3));
    }

    static void waitNotifyDemo() throws InterruptedException {
        System.out.println("=== wait/notifyAll: one-slot mailbox ===");
        Mailbox<String> box = new Mailbox<>();
        List<String> received = new ArrayList<>();
        Thread consumer = new Thread(() -> {
            try { for (int i = 0; i < 3; i++) received.add(box.take()); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        consumer.start();
        for (String m : List.of("m1", "m2", "m3")) box.put(m);
        consumer.join();
        System.out.println("received in order: " + received);
    }

    static void conditionDemo() throws InterruptedException {
        System.out.println("=== Lock + Condition: strict turn-taking ===");
        TurnTable table = new TurnTable();
        List<String> out = java.util.Collections.synchronizedList(new ArrayList<>());
        Thread ping = new Thread(() -> { try { table.play(true, "PING", 3, out); } catch (InterruptedException
ignored) { } });
        Thread pong = new Thread(() -> { try { table.play(false, "PONG", 3, out); } catch (InterruptedException
ignored) { } });
        pong.start(); ping.start();              // start order does not matter
        ping.join(); pong.join();
        System.out.println(String.join(" ", out));
    }

    static void latchDemo() throws InterruptedException {
        System.out.println("=== CountDownLatch: wait for 3 services to start ===");
        CountDownLatch ready = new CountDownLatch(3);
        for (String svc : List.of("db", "cache", "search")) {
            new Thread(() -> { /* ...warm up... */ ready.countDown(); }, svc).start();
        }
        boolean allUp = ready.await(2, TimeUnit.SECONDS);   // one-shot: cannot be reset
        System.out.println("all services ready: " + allUp + " (count now " + ready.getCount() + ")");
    }

    static void barrierDemo() throws Exception {
        System.out.println("=== CyclicBarrier: 3 workers, 2 phases ===");
        AtomicInteger phase = new AtomicInteger();
        // The barrier action runs once per trip, by the last thread to arrive.
        CyclicBarrier barrier = new CyclicBarrier(3,
                () -> System.out.println("phase " + phase.incrementAndGet() + " complete: all 3 workers arrived"));
        List<Thread> ws = new ArrayList<>();
```

```
        for (int i = 0; i < 3; i++) {
            Thread t = new Thread(() -> {
                try {
                    for (int p = 0; p < 2; p++) {
                        Thread.sleep((long) (Math.random() * 20)); // uneven work
                        barrier.await();                           // reusable: works for every phase
                    }
                } catch (Exception e) { throw new RuntimeException(e); }
            });
            ws.add(t);
            t.start();
        }
        for (Thread t : ws) t.join();
    }
}
```

Explaining each part:

- `semaphoreDemo`: 10 tasks, 3 permits; `maxSeen` records the peak number of concurrent users and never exceeds 3.
- `Mailbox` is a condition variable built on `wait/notifyAll` (section 12.13).
- `TurnTable` is a condition variable built on `ReentrantLock` + `Condition` (section 12.13).
- `latchDemo` / `barrierDemo`: section 12.14.

Output:

```
=== Semaphore: at most 3 concurrent DB connections ===
10 queries done, never more than 3 at once: true
=== wait/notifyAll: one-slot mailbox ===
received in order: [m1, m2, m3]
=== Lock + Condition: strict turn-taking ===
PING PONG PING PONG PING PONG
=== CountDownLatch: wait for 3 services to start ===
all services ready: true (count now 0)
=== CyclicBarrier: 3 workers, 2 phases ===
phase 1 complete: all 3 workers arrived
phase 2 complete: all 3 workers arrived
```

Trade-offs

Pros	Cons
Simple way to cap concurrency	No owner: a bug can release more than acquired (permits grow)
Works as a signal (0 permits)	Doesn't protect data by itself — only counts

When not to use it

Don't use a semaphore as a mutex around shared data when a lock would do — locks have ownership checks and reentrancy.

Common mistakes

- Releasing in a path where `acquire` never succeeded (e.g. `acquire` interrupted) → permit leak upward.
- Forgetting to release on exceptions → permits slowly disappear, system stalls.

Real-world examples

- **Bulkhead pattern** (isolating resources per dependency, like watertight compartments in a ship, so one failure can't sink everything): Resilience4j's `SemaphoreBulkhead` (and Hystrix's — Netflix's older fault-tolerance library — semaphore isolation) cap concurrent calls to a dependency.
- **Connection pools** (sets of open database connections reused across requests) bound the number of connections handed out (HikariCP, a popular JDBC pool, enforces its maximum pool size).
- With virtual threads, a `Semaphore` is the recommended way to limit calls to a slow **downstream** service (one that your code calls).

In the interview

> **30-second answer:** "A semaphore is a permit counter: acquire takes a permit or waits, release returns one. I use it to cap concurrent access to a limited resource — a bulkhead — or with zero initial permits as a signal from one thread to another. Unlike a lock it has no owner."

Follow-ups:

- *Binary semaphore vs mutex?* Both allow one holder, but a mutex has an owner (only the holder may unlock, and it's usually reentrant); a binary semaphore can be released by any thread and isn't reentrant.

Self-check

- **Q:** `new Semaphore(0)`: what does `acquire()` do? **A:** Waits until some thread calls `release()` — a signal.

TL;DR

Semaphore = N permits. Cap concurrency or signal. No ownership, so release carefully in `finally`.

12.13 Condition variables: `wait/notify` and `Condition`

What it is

A **condition variable** lets a thread *sleep until some condition about shared state becomes true*, releasing the lock while it sleeps, and lets another thread *wake it* after changing that state.

- Built-in: `Object.wait()`, `notify()`, `notifyAll()` — used inside `synchronized` on the same object.
- Explicit: `Condition c = lock.newCondition(); c.await(); c.signal(); c.signalAll();` — used while holding that `ReentrantLock`.

> **Analogy.** A restaurant pager: you (the thread) hand back your table (release the lock) and wait. When the table is ready, staff buzz the pager (notify). You then come back and check that the table really is free (re-check the condition), because someone might have taken it.

The problem it solves

Waiting for a state change without **busy-waiting** (spinning in a loop wasting CPU) and without missing the moment it happens.

How it works

The canonical pattern — memorize it:

```
synchronized (lock) {                 // or: lock.lock(); try {
    while (!conditionHolds()) {       // ALWAYS a while loop, never if
        lock.wait();                  //   cond.await();  releases the lock while waiting
    }
    doAction();                       // condition is true AND we hold the lock
    lock.notifyAll();                 //   cond.signal()/signalAll() if we changed state others wait for
}                                     // } finally { lock.unlock(); }
```

1. Acquire the lock.
2. While the condition is false, call `wait()` / `await()` — this **atomically releases the lock** and sleeps.
3. When woken, the thread **re-acquires the lock** before returning, then re-checks the condition (loop).
4. After changing state, call `notifyAll()` / `signal()` so waiters re-check.

Why the `while` loop? (a) **Spurious wakeups** — the JLS and the `Condition` docs allow a thread to wake without being notified. (b) Another thread may have grabbed the state between the notify and your re-acquisition. (c) With `notifyAll`, several threads wake but only one can succeed.

`notify` vs `notifyAll`: `notify()` wakes one arbitrary waiter. If different kinds of threads (producers and consumers) wait on the same monitor, `notify()` can wake the wrong kind and everyone sleeps forever. Use `notifyAll()` with a single monitor, or use separate `Condition`s (`notFull`, `notEmpty`) and targeted `signal()`.

Java example

`Mailbox` (monitor version) and `TurnTable` (Lock + Condition) in `coordination/Main.java` above. Output: `received in order: [m1, m2, m3]` and `PING PONG PING PONG PING PONG`.

Trade-offs

	wait/notify	Lock + Condition
Conditions per lock	One wait set	Many (e.g. `notFull`, `notEmpty`)
Targeted wake-up	No — use `notifyAll`	Yes — `signal()` on the right condition
Timed wait	`wait(ms)`	`await(t, unit)`, `awaitNanos` (returns time left)
Verbosity	Less	More (manual unlock)

When not to use it

Most of the time you don't need raw condition variables: `BlockingQueue`, `Semaphore`, `CountDownLatch`, `CompletableFuture` already solve the common waiting problems. Write your own only for custom coordination (interview problems, custom synchronizers).

Common mistakes

- `if` instead of `while` around `wait()`.
- Calling `wait()`/`notify()` without holding that object's monitor → `IllegalMonitorStateException`.
- Changing state *without* notifying → waiters sleep forever (a "lost wakeup").
- Calling `wait()` on one object while synchronized on another.

Real-world examples

`ArrayBlockingQueue` uses one `ReentrantLock` with two `Condition`s, `notEmpty` and `notFull` — exactly the pattern built by hand in chapter 13.

In the interview

> **30-second answer:** "A condition variable lets a thread release a lock and sleep until another thread changes the state and signals it. I always wait in a while loop that re-checks the condition, because of spurious wakeups and competing threads. With one monitor I use notifyAll; with ReentrantLock I use separate Conditions and targeted signal."

Follow-ups:

- *Why must `wait()` be called while holding the lock?* So the check-then-sleep is atomic; otherwise a notify could happen between your check and your wait and be lost.

Self-check

- **Q:** What does `wait()` do with the monitor? **A:** Releases it while sleeping and re-acquires it before returning.

TL;DR

Condition variable = "release lock, sleep until signalled, re-acquire, re-check". Always `while`, prefer `notifyAll` or separate `Condition`s.

12.14 `CountDownLatch` and `CyclicBarrier`

What it is

- **`CountDownLatch(n)`**: a one-time gate. Threads call `await()` and wait until `countDown()` has been called *n* times. It cannot be reset.
- **`CyclicBarrier(n, action)`**: a reusable meeting point. Each of *n* threads calls `await()`; when the *n*th arrives, the optional barrier action runs once, all are released, and the barrier resets for the next round.

> **Analogy.** Latch = a race starter who waits for all 3 judges to say "ready" and then fires the pistol once. Barrier = hikers who agree to wait for each other at every checkpoint before continuing.

The problem it solves

Coordinating *phases*: "start only after all services are up" (latch), "all workers finish step 1 before anyone starts step 2" (barrier), and "release all test threads at the same instant" (latch with count 1 — used throughout this book to maximise contention in tests).

How it works

	CountDownLatch	CyclicBarrier
Who waits	Any thread(s) calling `await()`	The *n* participating threads themselves
Who counts	Anyone calling `countDown()` (need not wait)	Arrival at `await()`
Reusable	No	Yes (auto-reset after each trip)
Extra	Count reaching 0 is permanent: later `await()` calls return at once. One thread may call `countDown()` several times	Barrier action runs once per trip; if one party is interrupted/times out, the barrier **breaks** and others get `BrokenBarrierException` (`reset()` repairs it)
Happens-before	Actions before `countDown()` HB actions after the matching `await()` returns	Actions before `await()` HB the barrier action, which HB actions after the other parties return from `await()`

`Phaser` (Java 7) generalises both: parties can register/deregister dynamically across many phases.

Java example

`latchDemo` and `barrierDemo` in `coordination/Main.java` above. Output shows `all services ready: true (count now 0)` and `phase 1 complete…`, `phase 2 complete…`.

Trade-offs

Latch: dead simple, one-shot. Barrier: reusable but fragile — one failing party breaks it for all.

When not to use it

For "wait for these N tasks and collect results", `ExecutorService.invokeAll` or `CompletableFuture.allOf(...)` is usually clearer.

Common mistakes

- Waiting on a latch forever because a worker threw before `countDown()` — put `countDown()` in `finally`, and use `await(timeout)`.
- Using a barrier with fewer threads than parties → everyone waits forever.

Real-world examples

- Test harnesses start N threads and release them together with a `CountDownLatch(1)` to maximise races (JCStress — OpenJDK's harness for testing concurrency correctness — uses stronger tools, but the idea is the same).
- Application start-up: wait for caches, DB pools, and message consumers to be ready before accepting traffic.

In the interview

> **30-second answer:** "`CountDownLatch` is a one-shot gate that opens after N countDowns; `CyclicBarrier` is a reusable rendezvous where N threads wait for each other each phase, optionally running an action when all arrive."

Follow-ups:

- *Can a latch be reset?* No — create a new one or use `CyclicBarrier`/`Phaser`.

Self-check

- **Q:** Which one would you use so 3 worker threads sync at the end of every simulation step? **A:** `CyclicBarrier(3)`.

TL;DR

Latch = one-shot "wait for N events". Barrier = reusable "wait for N threads". Put `countDown` in `finally`; use timeouts.

Part E — Lock-free

12.15 Compare-and-swap (CAS) & atomics

What it is

Compare-and-swap is a single CPU instruction (e.g. `LOCK CMPXCHG` on x86; LL/SC — load-linked/store-conditional — or CAS instructions on ARM): "if the value at this address is still `expected`, replace it with `new` and report success; otherwise change nothing and report failure." Java exposes it through `java.util.concurrent.atomic` classes: `AtomicInteger`, `AtomicLong`, `AtomicBoolean`, `AtomicReference`, `AtomicIntegerArray`, `LongAdder`, and through `VarHandle`.

> **Analogy.** Editing a shared wiki page: you save only if the page version is still the one you started from. If someone saved first, your save is rejected; you reload and try again.

The problem it solves

Atomic updates **without locks** (**lock-free**: no thread ever blocks another, and some thread always makes progress): no blocking, no deadlock, and usually faster under low-to-moderate contention. It is how lock-free data structures and the JDK's own concurrency utilities are built.

How it works

The **CAS retry loop**:

1. Read the current value.
2. Compute the new value from it.
3. `compareAndSet(current, new)`. If it succeeds, done.
4. If it fails, another thread changed the value — go back to step 1.

Helpers do the loop for you: `incrementAndGet`, `addAndGet`, `updateAndGet(fn)`, `accumulateAndGet(x, fn)` (the function may run more than once, so it must be side-effect-free).

`LongAdder` spreads updates over several internal cells when contended and sums them on `sum()` — much faster than `AtomicLong` for hot counters (metrics), but `sum()` is not an atomic snapshot.

ABA problem. A value goes A → B → A. A thread that read A earlier sees A again and its CAS succeeds, although the state changed in between. This matters for pointer-based structures (e.g. reusing nodes in a lock-free stack). Fix: attach a version — `AtomicStampedReference` (value + int stamp) or `AtomicMarkableReference`. (In Java, garbage collection prevents the classic node-reuse ABA for freshly allocated nodes, which is why the Treiber stack below is safe.)

Java example

```java
// File: code/lld/ch12/atomics/Main.java
package lld.ch12.atomics;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;
import java.util.concurrent.atomic.AtomicStampedReference;
import java.util.concurrent.atomic.LongAdder;

/** Compare-and-swap (CAS), a lock-free stack, LongAdder, and the ABA problem. */
public class Main {

    /** Increment written by hand as a CAS retry loop (this is what incrementAndGet does inside). */
    static int casIncrement(AtomicInteger a) {
        while (true) {
            int current = a.get();                    // 1. read
            int next = current + 1;                   // 2. compute
            if (a.compareAndSet(current, next)) {     // 3. write only if nobody changed it meanwhile
                return next;
            }                                         // 4. someone won the race: retry
        }
    }

    /** Treiber stack: a classic lock-free data structure built on one AtomicReference. */
    static class LockFreeStack<T> {
        private record Node<T>(T value, Node<T> next) { }
        private final AtomicReference<Node<T>> head = new AtomicReference<>();

        void push(T v) {
            Node<T> oldHead, newHead;
            do {
                oldHead = head.get();
                newHead = new Node<>(v, oldHead);
            } while (!head.compareAndSet(oldHead, newHead));
        }
        T pop() {
            Node<T> oldHead;
            do {
                oldHead = head.get();
                if (oldHead == null) return null;
            } while (!head.compareAndSet(oldHead, oldHead.next()));
            return oldHead.value();
        }
    }

    static void runThreads(int n, Runnable r) throws InterruptedException {
        List<Thread> ts = new ArrayList<>();
        for (int i = 0; i < n; i++) { Thread t = new Thread(r); ts.add(t); t.start(); }
        for (Thread t : ts) t.join();
    }

    public static void main(String[] args) throws Exception {
        AtomicInteger counter = new AtomicInteger();
        runThreads(4, () -> { for (int i = 0; i < 100_000; i++) casIncrement(counter); });
        System.out.println("hand-written CAS loop counter = " + counter.get());

        AtomicInteger highWater = new AtomicInteger();
        runThreads(4, () -> {
            for (int i = 0; i < 1_000; i++) highWater.accumulateAndGet(i, Math::max);  // atomic "max"
        });
        System.out.println("atomic max                    = " + highWater.get());

        LockFreeStack<Integer> stack = new LockFreeStack<>();
        runThreads(4, () -> { for (int i = 0; i < 10_000; i++) stack.push(i); });
        int popped = 0;
```

```
        while (stack.pop() != null) popped++;
        System.out.println("lock-free stack popped        = " + popped);

        LongAdder hits = new LongAdder();              // striped counter: fast under heavy contention
        runThreads(8, () -> { for (int i = 0; i < 100_000; i++) hits.increment(); });
        System.out.println("LongAdder sum                 = " + hits.sum());

        // ABA: value goes A -> B -> A; a plain CAS cannot tell anything happened.
        AtomicReference<String> ref = new AtomicReference<>("A");
        String seen = ref.get();
        ref.set("B"); ref.set("A");                     // another thread's changes
        System.out.println("plain CAS after A->B->A succeeds   : " + ref.compareAndSet(seen, "C"));

        AtomicStampedReference<String> sref = new AtomicStampedReference<>("A", 0);
        int[] stampHolder = new int[1];
        String seen2 = sref.get(stampHolder);
        int seenStamp = stampHolder[0];
        sref.set("B", seenStamp + 1); sref.set("A", seenStamp + 2);
        System.out.println("stamped CAS after A->B->A succeeds : " + sref.compareAndSet(seen2, "C", seenStamp,
seenStamp + 1));
    }
}
```

Explaining each part:

- `casIncrement` is the textbook retry loop — this is what `incrementAndGet` does conceptually.
- `accumulateAndGet(i, Math::max)` atomically tracks a maximum.
- `LockFreeStack` (the *Treiber stack*): push/pop swing the `head` reference with CAS; a failed CAS means another thread pushed/popped first — retry.
- `LongAdder` counts 800,000 increments from 8 threads with little contention.
- The ABA lines show a plain CAS succeeding after A→B→A, while the stamped CAS fails because the stamp moved from 0 to 2.

Output:

```
hand-written CAS loop counter = 400000
atomic max                    = 999
lock-free stack popped        = 40000
LongAdder sum                 = 800000
plain CAS after A->B->A succeeds   : true
stamped CAS after A->B->A succeeds : false
```

Trade-offs

	Locks	CAS / atomics
Blocking	Yes	No (threads retry)
Deadlock	Possible	Impossible (no locks)
Scope	Any number of variables	One variable/reference per CAS
Under heavy contention	Threads sleep (saves CPU)	Many failed retries (wastes CPU) — use `LongAdder` or striping
Difficulty	Moderate	Hard for anything beyond counters/flags/one reference

When not to use it

Invariants spanning several variables (move money between two accounts) — a single CAS can't cover them; use a lock, or put all fields into one immutable object and CAS the reference (as the lock-free token bucket in chapter 13 does).

Common mistakes

- Separate `get()` then `set()` on an atomic → still a race. Use `compareAndSet`/`updateAndGet`.
- Side effects inside `updateAndGet` lambdas (they may run several times).
- Using `AtomicLong` for a very hot metrics counter where `LongAdder` would scale better.

Real-world examples

- `ConcurrentHashMap` inserts into empty bins with CAS; `ConcurrentLinkedQueue` is a lock-free (Michael–Scott) queue; `AbstractQueuedSynchronizer` (behind `ReentrantLock`) uses CAS on its state.
- Optimistic locking in databases and ORMs (Object-Relational Mappers — libraries that map objects to database tables; JPA `@Version`, `UPDATE ... WHERE version = ?`) is CAS at the database level.
- LMAX Disruptor (a high-performance inter-thread messaging library) uses CAS-based sequences for its lock-free **ring buffer** (a fixed-size array used as a circular queue).

In the interview

> **30-second answer:** "CAS atomically updates a value only if it still equals what I read; otherwise I retry. Java's atomic classes wrap it, so counters, flags and single references can be updated without locks and without deadlock. It covers one variable at a time, can suffer ABA — fixed with a stamped reference — and under heavy contention I prefer `LongAdder`."

Follow-ups:

- *Is lock-free always faster?* No. Under heavy contention retries waste CPU; a lock that parks threads can be better.
- *How would you do a lock-free update of two fields?* Put them in an immutable object and CAS an `AtomicReference` to it.

Self-check

- **Q:** What does a failed `compareAndSet` mean? **A:** Another thread changed the value after you read it; re-read and retry.
- **Q:** What fixes ABA? **A:** A version stamp: `AtomicStampedReference`.

TL;DR

CAS = "update only if unchanged", retried in a loop. Atomics give lock-free, deadlock-free updates to one variable. Watch ABA; use `LongAdder` for hot counters.

12.16 Concurrent collections

What it is

Thread-safe collections in `java.util.concurrent` designed for high concurrency (unlike the old fully-`synchronized` `Vector` / `Hashtable`):

Need	Use	Key behaviour
Map	`ConcurrentHashMap`	Lock-free reads; writes lock only one bin; atomic `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`; **no null keys/values**; *weakly consistent* iterators (see below)
Sorted map/set	`ConcurrentSkipListMap` / `Set`	Lock-free, sorted, O(log n) (cost grows very slowly with size)
List read ≫ write	`CopyOnWriteArrayList` / `Set`	Each write copies the whole array; iteration uses a snapshot (never throws CME, `ConcurrentModificationException`)
Queue (non-blocking)	`ConcurrentLinkedQueue` / `Deque`	Lock-free; `poll()` returns null when empty
Queue (blocking)	`ArrayBlockingQueue` (bounded), `LinkedBlockingQueue` (optionally bounded), `PriorityBlockingQueue`, `DelayQueue`, `SynchronousQueue` (no capacity: hand-off), `LinkedTransferQueue`	`put` waits when full, `take` waits when empty; `offer` / `poll` with timeouts

Two `ConcurrentHashMap` guarantees worth quoting exactly:

- **`computeIfAbsent` / `compute` / `merge` are atomic.** The Javadoc says the whole call is performed atomically, and for `computeIfAbsent` the mapping function is invoked **exactly once if the key is absent, otherwise not at all** — so two racing threads can never both create a value for the same key. Some updates by other threads (at least those hitting the same bin) block while the function runs, so keep it short and never touch the same map inside it. (The default `ConcurrentMap.computeIfAbsent` used by some other maps, e.g. `ConcurrentSkipListMap`, may call the function more than once and discard results — only the final put is atomic.)
- **Iterators are weakly consistent.** They never throw `ConcurrentModificationException`, may proceed while other threads modify the map, return each element at most once, and reflect the map's state at some point at or after the iterator was created — they *may or may not* show later updates. `size()` / `isEmpty()` are likewise only estimates while updates are in flight.

> **Analogy.** A supermarket with many tills (ConcurrentHashMap's bins) instead of one till for the whole store (`Hashtable`).

The problem it solves

Wrapping a `HashMap` in one lock **serialises** every access (forces accesses to happen one at a time). Concurrent collections let many threads read and write different parts at the same time and offer **atomic compound operations** so you don't need external locks for common patterns.

How it works

1. Pick the collection by access pattern (table above).
2. Replace check-then-act sequences with a single atomic method:
   - `if (!m.containsKey(k)) m.put(k, v)` → `m.putIfAbsent(k, v)`
   - `m.put(k, m.get(k) + 1)` → `m.merge(k, 1, Integer::sum)`
   - lazy creation → `m.computeIfAbsent(k, key -> new ...)`
3. For hand-offs between threads, use a `BlockingQueue` (producer-consumer, 12.22).
4. Remember *compound operations across several calls are still not atomic*.

Java example

```java
// File: code/lld/ch12/collections/Main.java
package lld.ch12.collections;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.TimeUnit;

/** ConcurrentHashMap atomic compound ops, CopyOnWriteArrayList snapshots, BlockingQueue. */
public class Main {

    public static void main(String[] args) throws Exception {
        // ---------- ConcurrentHashMap ----------
        ConcurrentHashMap<String, Integer> wordCount = new ConcurrentHashMap<>();
        String[] words = {"apple", "banana", "apple", "cherry", "banana", "apple"};
        List<Thread> ts = new ArrayList<>();
        for (int t = 0; t < 4; t++) {
            Thread th = new Thread(() -> {
                for (int rep = 0; rep < 1_000; rep++)
                    for (String w : words) wordCount.merge(w, 1, Integer::sum);   // atomic per key
            });
            ts.add(th); th.start();
        }
        for (Thread th : ts) th.join();
        System.out.println("word counts (4 threads x 1000 reps): " + new TreeMap<>(wordCount));

        // computeIfAbsent: create-once per key, even under races (e.g. one lock/queue per user).
        ConcurrentHashMap<String, List<String>> ordersByUser = new ConcurrentHashMap<>();
        ordersByUser.computeIfAbsent("alice", k -> new CopyOnWriteArrayList<>()).add("order-1");
        ordersByUser.computeIfAbsent("alice", k -> new CopyOnWriteArrayList<>()).add("order-2");
        System.out.println("alice's orders: " + ordersByUser.get("alice"));

        // WRONG pattern (check-then-act across two calls) vs RIGHT (one atomic call):
        //   if (!map.containsKey(k)) map.put(k, v);   // two threads can both put
        //   map.putIfAbsent(k, v);                    // atomic
        Map<String, String> seats = new ConcurrentHashMap<>();
        System.out.println("first putIfAbsent returns : " + seats.putIfAbsent("A1", "bob"));
        System.out.println("second putIfAbsent returns: " + seats.putIfAbsent("A1", "eve") + "  (seat already
bob's)");

        // ---------- CopyOnWriteArrayList ----------
        CopyOnWriteArrayList<String> listeners = new CopyOnWriteArrayList<>(List.of("L1", "L2"));
        int seen = 0;
        for (String l : listeners) {          // iterates over a snapshot
            listeners.add(l + "-new");        // no ConcurrentModificationException
            seen++;
        }
        System.out.println("iterated " + seen + " items; list now " + listeners);

        // ---------- BlockingQueue ----------
        BlockingQueue<Integer> q = new ArrayBlockingQueue<>(2);   // bounded = back-pressure
        System.out.println("offer 1: " + q.offer(1) + ", offer 2: " + q.offer(2) + ", offer 3: " + q.offer(3) + "
(full)");
        System.out.println("take -> " + q.take() + ", poll -> " + q.poll() + ", poll(50ms) on empty -> "
                + q.poll(50, TimeUnit.MILLISECONDS));
    }
}
```

Explaining each part:

- `merge(w, 1, Integer::sum)` is an atomic per-key increment; totals are exact.

- `computeIfAbsent` creates each per-user list once, even under races.
- `putIfAbsent` returns `null` for the winner and the existing value for losers — perfect for "claim this seat".
- Iterating a `CopyOnWriteArrayList` while adding to it sees the old snapshot (2 items) and throws no exception.
- `ArrayBlockingQueue(2)`: `offer` returns `false` when full; `poll(50ms)` returns `null` when empty.

Output:

```
word counts (4 threads x 1000 reps): {apple=12000, banana=8000, cherry=4000}
alice's orders: [order-1, order-2]
first putIfAbsent returns : null
second putIfAbsent returns: bob  (seat already bob's)
iterated 2 items; list now [L1, L2, L1-new, L2-new]
offer 1: true, offer 2: true, offer 3: false (full)
take -> 1, poll -> 2, poll(50ms) on empty -> null
```

Trade-offs

Collection	Great for	Bad for
`ConcurrentHashMap`	General shared maps, caches, counters	Multi-key atomic operations; `size()` is only an estimate under concurrent updates
`CopyOnWriteArrayList`	Listener lists, rarely-changed config	Frequent writes (O(n) copy each)
`Collections.synchronizedMap`	Legacy code, low contention	High contention; iteration still needs manual `synchronized(map)`
`ArrayBlockingQueue`	Bounded hand-off with back-pressure	Very high throughput (single lock for put/take)
`LinkedBlockingQueue`	Higher throughput (separate put/take locks)	Default constructor is **unbounded** (capacity `Integer.MAX_VALUE`)

When not to use it

If an object is confined to one thread, a plain `HashMap`/`ArrayList` is faster. If you need a transaction across several keys, a single lock (or a database) is clearer.

Common mistakes

- Doing slow work or calling back into the same map inside `computeIfAbsent`/`compute` — the function runs while the bin is locked; recursive updates can throw `IllegalStateException` or deadlock.
- Putting `null` in a `ConcurrentHashMap` → `NullPointerException`.
- Unbounded queues that silently grow until `OutOfMemoryError`.

Real-world examples

- Caffeine's (a Java caching library) cache is built on `ConcurrentHashMap`; Guava's (Google's core Java library) `LocalCache` uses its own segmented design derived from Java 7's `ConcurrentHashMap`.
- Guava's `EventBus` keeps each event type's subscribers in a `CopyOnWriteArraySet`; listener lists in general are the textbook copy-on-write use case.
- Log4j2's async **appenders** (the components that write log events to a file or other destination) and many logging frameworks hand events from app threads to a writer thread through queues.

In the interview

30-second answer: "I use `ConcurrentHashMap` for shared maps and its atomic methods — `putIfAbsent`, `computeIfAbsent`, `merge` — instead of check-then-act. `CopyOnWriteArrayList` for read-mostly lists like listeners. `BlockingQueue`, preferably bounded, to hand work between threads."

Follow-ups:

- *How does Java 8+ `ConcurrentHashMap` achieve concurrency?* Reads are lock-free (volatile reads). Writes CAS into empty bins, otherwise `synchronized` on the bin's first node; long bins become **red-black trees** (self-balancing

sorted binary trees); resizing is done cooperatively by several threads. (Java 7 used 16 lock-striped segments by default.)

- *Why no nulls?* `get(k) == null` must unambiguously mean "absent"; with concurrency you can't follow up with `containsKey` atomically.

Self-check

- **Q:** Atomic way to count words in a shared map? **A:** `map.merge(word, 1, Integer::sum)`.
- **Q:** Why doesn't iterating a `CopyOnWriteArrayList` throw CME? **A:** It iterates an immutable snapshot of the array.

TL;DR

`ConcurrentHashMap` + atomic methods for maps, COW lists for read-mostly lists, bounded `BlockingQueue` for hand-offs. Single calls are atomic; sequences of calls are not.

Part F — Liveness failures

12.17 Deadlock

What it is

A **deadlock** is when two or more threads each wait forever for a resource another one holds. Nobody can proceed.

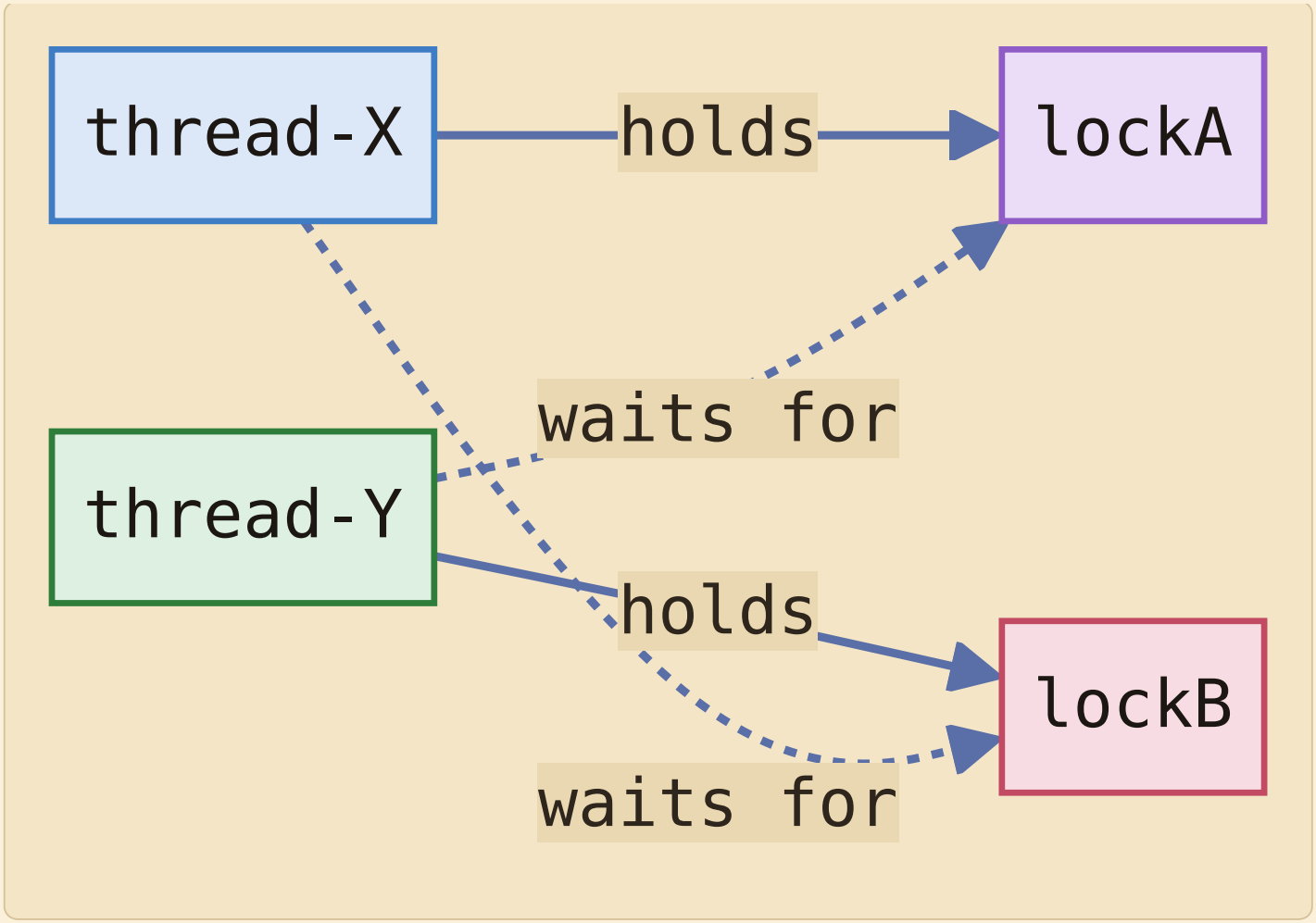
> **Analogy.** Two cars meet on a one-lane bridge from opposite ends. Each waits for the other to back up. Neither does.

The problem it solves

(Understanding deadlock lets you *design it out*.) A deadlock requires all four **Coffman conditions** (Coffman et al., 1971) at once:

#	Condition	Meaning	How to break it
1	**Mutual exclusion**	A resource can be held by only one thread	Use lock-free/immutable data, or shareable resources (read locks)
2	**Hold and wait**	A thread holds one resource while waiting for another	Acquire all resources at once, or use `tryLock` and release everything on failure
3	**No preemption**	Resources can't be taken away; only the holder releases	Timeouts (`tryLock(timeout)`), lock **leases** (locks that expire automatically after a set time)
4	**Circular wait**	A cycle: T1 waits for T2's lock, T2 waits for T1's	**Global lock ordering** (always lock lower id first)

Break **any one** condition and deadlock is impossible.



How it works

1. **Prevention by lock ordering (most common):** define a total order over locks (account id, seat id, `System.identityHashCode` with a tie-breaker lock) and always acquire in that order.
2. **Prevention by try-lock:** `tryLock(timeout)` each lock; if one fails, release all held locks, wait a random short time, retry.
3. **Avoid nested locks** where possible: call alien code (listeners, callbacks) *outside* your lock.
4. **Detection:** thread dumps (`jstack` / `jcmd`) print "Found one Java-level deadlock"; programmatically, `ThreadMXBean.findDeadlockedThreads()` (`ThreadMXBean` is the JDK's management interface for inspecting threads).
5. **Avoidance** (Banker's algorithm — grant a request only if the system can still finish every thread) exists in OS theory but is rarely used in application code.

Java example

```
// File: code/lld/ch12/deadlock/Main.java
package lld.ch12.deadlock;

import java.lang.management.ManagementFactory;
import java.lang.management.ThreadInfo;
import java.lang.management.ThreadMXBean;
import java.util.Arrays;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.ThreadLocalRandom;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;

/** Create a real deadlock, detect it, then fix it two ways: lock ordering and tryLock + back-off. */
public class Main {

    static class Account {
        final int id;
        final ReentrantLock lock = new ReentrantLock();
        int balance;
        Account(int id, int balance) { this.id = id; this.balance = balance; }
    }

    // ---------- Fix 1: global lock ordering (always lock the smaller id first) ----------
    static void transferOrdered(Account from, Account to, int amount) {
        Account first = from.id < to.id ? from : to;
        Account second = from.id < to.id ? to : from;
        first.lock.lock();
        try {
            second.lock.lock();
            try {
                if (from.balance >= amount) { from.balance -= amount; to.balance += amount; }
            } finally { second.lock.unlock(); }
        } finally { first.lock.unlock(); }
    }

    // ---------- Fix 2: tryLock with timeout + random back-off (breaks "hold and wait") ----------
    static void transferTryLock(Account from, Account to, int amount) throws InterruptedException {
        while (true) {
            if (from.lock.tryLock(1, TimeUnit.MILLISECONDS)) {
                try {
                    if (to.lock.tryLock(1, TimeUnit.MILLISECONDS)) {
                        try {
                            if (from.balance >= amount) { from.balance -= amount; to.balance += amount; }
                            return;
                        } finally { to.lock.unlock(); }
                    }
                } finally { from.lock.unlock(); }         // give up what we hold
            }
            // Random back-off: without it, both threads could retry in lock-step forever (LIVELOCK).
            Thread.sleep(0, ThreadLocalRandom.current().nextInt(1, 100_000));
        }
    }

    public static void main(String[] args) throws Exception {
        demonstrateDeadlock();

        Account a = new Account(1, 1_000), b = new Account(2, 1_000);
        Thread t1 = new Thread(() -> { for (int i = 0; i < 50_000; i++) transferOrdered(a, b, 1); });
        Thread t2 = new Thread(() -> { for (int i = 0; i < 50_000; i++) transferOrdered(b, a, 1); });
        t1.start(); t2.start(); t1.join(); t2.join();
        System.out.println("lock ordering: finished, total money = " + (a.balance + b.balance));

        Account c = new Account(3, 1_000), d = new Account(4, 1_000);
        Thread t3 = new Thread(() -> { try { for (int i = 0; i < 5_000; i++) transferTryLock(c, d, 1); } catch
(InterruptedException ignored) { } });
        Thread t4 = new Thread(() -> { try { for (int i = 0; i < 5_000; i++) transferTryLock(d, c, 1); } catch
```

```
(InterruptedException ignored) { } });
        t3.start(); t4.start(); t3.join(); t4.join();
        System.out.println("tryLock + back-off: finished, total money = " + (c.balance + d.balance));
    }

    /** Two threads lock in opposite order. A barrier forces the bad interleaving every time. */
    static void demonstrateDeadlock() throws Exception {
        Object lockA = new Object(), lockB = new Object();
        CyclicBarrier bothHoldOne = new CyclicBarrier(2);
        Thread x = new Thread(() -> {
            synchronized (lockA) { await(bothHoldOne); synchronized (lockB) { System.out.println("never printed");
} }
        }, "thread-X");
        Thread y = new Thread(() -> {
            synchronized (lockB) { await(bothHoldOne); synchronized (lockA) { System.out.println("never printed");
} }
        }, "thread-Y");
        x.setDaemon(true); y.setDaemon(true);   // daemon threads do not keep the JVM alive
        x.start(); y.start();

        ThreadMXBean mx = ManagementFactory.getThreadMXBean();
        long[] ids = null;
        for (int i = 0; i < 200 && ids == null; i++) { Thread.sleep(10); ids = mx.findDeadlockedThreads(); }
        String[] names = ids == null ? new String[0]
                :
Arrays.stream(mx.getThreadInfo(ids)).map(ThreadInfo::getThreadName).sorted().toArray(String[]::new);
        System.out.println("deadlock detected between: " + Arrays.toString(names)
                + "  (states " + x.getState() + ", " + y.getState() + ")");
    }

    static void await(CyclicBarrier b) {
        try { b.await(); } catch (Exception e) { throw new RuntimeException(e); }
    }
}
```

Explaining each part:

- `demonstrateDeadlock` forces the deadly interleaving: each thread takes one lock, then both meet at a barrier, then each asks for the other's lock. `ThreadMXBean` finds the cycle; both threads show `BLOCKED`. They are daemon threads, so the JVM can still exit.
- `transferOrdered` always locks the account with the smaller id first → circular wait is impossible. 100,000 opposite-direction transfers finish and money is conserved.
- `transferTryLock` uses `tryLock(1 ms)` and releases on failure, with a random back-off (to avoid livelock, next section).

Output:

```
deadlock detected between: [thread-X, thread-Y]  (states BLOCKED, BLOCKED)
lock ordering: finished, total money = 2000
tryLock + back-off: finished, total money = 2000
```

Trade-offs

Strategy	Pros	Cons
Lock ordering	Simple, zero runtime cost	Needs a stable global order; hard across modules
Try-lock + back-off	Works without an order; bounded waits	Retry logic; possible livelock without randomness
Single coarse lock	No deadlock among your locks	Lower throughput
Lock-free / immutable	No locks at all	Harder to design

When not to use it

N/A — you always want to prevent deadlock. But don't add complicated detection/recovery in application code when simple lock ordering would prevent it.

Common mistakes

- Calling a listener or another object's synchronized method while holding your own lock (hidden nested locking).
- Ordering by `hashCode()` without handling equal hashes.
- Also *resource* deadlocks: a pool of 10 threads where each task submits a sub-task to the **same** pool and waits for it — when all 10 wait, nothing runs (thread-starvation deadlock).

Real-world examples

- Databases (MySQL InnoDB, PostgreSQL) detect deadlocks between **transactions** (groups of database operations that succeed or fail as one unit) and abort one as a victim — the "detect and recover" strategy.
- Bank transfer systems lock accounts in id order — the lock-ordering strategy.

In the interview

> **30-second answer:** "Deadlock needs four conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait. I break circular wait with a global lock order — e.g. lock seats or accounts in ascending id — or break hold-and-wait with tryLock with timeout and release-and-retry. I avoid calling external code while holding a lock, and I'd detect it in production with thread dumps."

Follow-ups:

- *Two accounts with the same id order?* Same account → just one lock (transfer to self is a no-op or rejected).
- *How would you find a deadlock in production?* `jstack <pid>` or `jcmd <pid> Thread.print` shows it; monitoring can call `ThreadMXBean.findDeadlockedThreads()`.

Self-check

- **Q:** Which Coffman condition does lock ordering break? **A:** Circular wait.
- **Q:** Which does `tryLock` + release break? **A:** Hold and wait (and effectively no-preemption, by giving up).

TL;DR

Deadlock = cycle of waiting. Needs all 4 Coffman conditions. Break one: lock ordering (circular wait) or tryLock + release (hold and wait). Don't call alien code under a lock.

12.18 Livelock

What it is

A **livelock** is when threads are not blocked — they keep running and reacting to each other — but make **no progress**.

> **Analogy.** Two people in a corridor both step aside to let the other pass, in the same direction, again and again.

The problem it solves

(Recognising it.) Livelocks often appear *as a side effect of deadlock avoidance*: two threads that both `tryLock`, fail, release, and retry at the same rhythm can collide forever.

How it works (and how to fix it)

1. Thread 1 takes lock A, tries B, fails, releases A.
2. Thread 2 takes lock B, tries A, fails, releases B.
3. Both retry immediately → same collision, forever.
4. **Fixes:** random back-off (different waits break the symmetry), exponential back-off, a consistent lock order (then no retries are needed), or a priority/tie-breaker (e.g. the older transaction wins).

Java example

`transferTryLock` in `deadlock/Main.java` sleeps a **random** number of nanoseconds (`ThreadLocalRandom`) after a failed attempt. Remove that line and the two threads could, in principle, keep failing in lock-step.

Trade-offs

Random back-off makes progress very likely but adds latency; exponential back-off adapts to contention.

When not to use it

Don't design retry loops that react only to each other with no randomness or tie-breaker.

Common mistakes

- Fixed retry delays (`sleep(10)` for everyone).
- Retrying forever without a limit or alert.

Real-world examples

- **Ethernet (CSMA/CD)** (the classic wired-network rule: listen before sending, detect collisions) uses *binary exponential back-off* (doubling the maximum random wait after each collision) after collisions for exactly this reason.
- Retry policies in AWS SDKs (Amazon Web Services' client libraries) and Resilience4j (a Java fault-tolerance library) use exponential back-off **with jitter** (a small random amount added to each wait) to stop synchronized **retry storms** (crowds of clients all retrying at the same moment).

In the interview

> **30-second answer:** "Livelock is when threads keep reacting to each other without progress — like two people side-stepping in a corridor. It often comes from symmetric retry logic; I fix it with randomized or exponential back-off, or remove retries by using a lock order."

Self-check

- **Q:** Deadlock vs livelock — thread states? **A:** Deadlocked threads are blocked/waiting; livelocked threads are running (`RUNNABLE`) but achieving nothing.

TL;DR

Livelock = busy but no progress. Break symmetry with jittered back-off or a tie-breaker.

12.19 Starvation (and fairness)

What it is

Starvation is when a thread waits indefinitely because others keep getting the resource first, even though the system as a whole makes progress.

> **Analogy.** At a busy bar, a quiet customer never gets served because louder customers keep catching the bartender's eye.

The problem it solves

(Recognising it.) Causes: unfair (barging) locks under heavy contention, a writer waiting behind a constant stream of readers in a read-write lock, low thread priority, or a long-running task hogging a single-thread executor.

How it works (fixes)

1. Use **fair** locks/semaphores where it matters: `new ReentrantLock(true)`, `new Semaphore(n, true)`, `new ReentrantReadWriteLock(true)`. Fair = longest waiter goes next.
2. Keep critical sections short so the lock is free more often.

3. Separate pools for slow and fast work (bulkheads) so slow tasks can't starve fast ones.
4. Don't rely on `Thread.setPriority` — its effect is OS-dependent.

Java example

`Waiter` in chapter 13's Dining Philosophers uses `new Semaphore(n - 1, true)` so no philosopher waits forever for a seat.

Trade-offs

Fairness prevents starvation but reduces throughput (more context switches, no barging).

When not to use it

Don't make every lock fair "just in case"; use fairness when you've seen or can argue starvation risk.

Common mistakes

Confusing starvation with deadlock: in starvation *someone* progresses; in deadlock *nobody* does.

Real-world examples

OS schedulers use *aging* (raising the priority of long-waiting processes); Linux's schedulers (CFS, and its successor EEVDF since **kernel** — the core of the operating system — 6.6) aim to give every runnable task a fair share of CPU time.

In the interview

> **30-second answer:** "Starvation is when a thread never gets a resource because others always win — e.g. writers behind endless readers. Fair locks, short critical sections and separate pools fix it, at some throughput cost."

Self-check

- **Q:** One fix for writer starvation in `ReentrantReadWriteLock`? **A:** Construct it in fair mode (`new ReentrantReadWriteLock(true)`).

TL;DR

Starvation = some thread never gets a turn. Fairness, short sections, separate pools.

Part G — Concurrency patterns

The next five patterns share one runnable file.

```java
// File: code/lld/ch12/patterns/Main.java
package lld.ch12.patterns;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.RejectedExecutionException;
import java.util.concurrent.Semaphore;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

/** Signaling, thread pool, producer-consumer, immutability, thread confinement. */
public class Main {

    // ---------- 1. Signaling ----------
    static void signaling() throws InterruptedException {
        System.out.println("=== Signaling: B must run after A ===");
        Semaphore aDone = new Semaphore(0);                 // 0 permits = "not signalled yet"
        List<String> log = java.util.Collections.synchronizedList(new ArrayList<>());
        Thread b = new Thread(() -> {
            try { aDone.acquire(); log.add("B: send email (after payment)"); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        Thread a = new Thread(() -> { log.add("A: charge payment"); aDone.release(); });
        b.start(); a.start();                               // B starts first but must wait
        a.join(); b.join();
        log.forEach(System.out::println);
    }

    // ---------- 2. Thread pool (hand-made, to see the moving parts) ----------
    static class SimpleThreadPool {
        private static final Runnable POISON = () -> { };
        private final BlockingQueue<Runnable> tasks = new LinkedBlockingQueue<>();
        private final List<Thread> workers = new ArrayList<>();
        private boolean shutdown;                           // guarded by 'this'

        SimpleThreadPool(int size) {
            for (int i = 0; i < size; i++) {
                Thread w = new Thread(() -> {
                    try {
                        while (true) {
                            Runnable r = tasks.take();          // idle workers block here
                            if (r == POISON) return;
                            try { r.run(); } catch (RuntimeException e) { System.err.println("task failed: " + e);
}
                        }
                    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
                }, "pool-worker-" + i);
                workers.add(w);
                w.start();
            }
        }
        // submit and the pill-adding part of shutdown share one lock, so "check the flag, then
        // enqueue" is atomic: no task can slip into the queue AFTER the poison pills and be lost.
        synchronized void submit(Runnable r) {
            if (shutdown) throw new RejectedExecutionException("pool is shut down");
            tasks.add(r);                                   // unbounded queue: add() never blocks
        }
        void shutdownAndWait() throws InterruptedException {
            synchronized (this) {
                if (shutdown) return;
                shutdown = true;
                for (int i = 0; i < workers.size(); i++) tasks.add(POISON);   // one pill per worker
```

```
            }
            for (Thread w : workers) w.join();          // wait OUTSIDE the lock
        }
    }

    static void threadPool() throws InterruptedException {
        System.out.println("=== Thread pool ===");
        SimpleThreadPool pool = new SimpleThreadPool(3);
        AtomicInteger done = new AtomicInteger();
        for (int i = 0; i < 20; i++) pool.submit(done::incrementAndGet);
        pool.shutdownAndWait();
        System.out.println("hand-made pool ran " + done.get() + " tasks on 3 threads");

        // Production version: bounded queue + explicit rejection policy.
        ThreadPoolExecutor exec = new ThreadPoolExecutor(
                2, 4,                                   // core and max threads
                30, TimeUnit.SECONDS,                   // idle time before extra threads die
                new ArrayBlockingQueue<>(100),          // bounded: never an OOM from a huge backlog
                new ThreadPoolExecutor.CallerRunsPolicy()); // when full, the submitter runs it (back-pressure)
        CompletableFuture<Integer> price = CompletableFuture.supplyAsync(() -> 100, exec);
        CompletableFuture<Integer> tax = CompletableFuture.supplyAsync(() -> 18, exec);
        System.out.println("ThreadPoolExecutor + CompletableFuture: total = " + price.thenCombine(tax,
Integer::sum).join());
        exec.shutdown();
        exec.awaitTermination(5, TimeUnit.SECONDS);
    }

    // ---------- 3. Producer-consumer ----------
    static void producerConsumer() throws InterruptedException {
        System.out.println("=== Producer-consumer ===");
        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(10);   // producers block when full
        final int POISON = -1, CONSUMERS = 2;
        AtomicLong total = new AtomicLong();
        List<Thread> consumers = new ArrayList<>();
        for (int c = 0; c < CONSUMERS; c++) {
            Thread t = new Thread(() -> {
                try {
                    for (int x = queue.take(); x != POISON; x = queue.take()) total.addAndGet(x);
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            consumers.add(t); t.start();
        }
        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 1_000; i++) queue.put(i);
                for (int c = 0; c < CONSUMERS; c++) queue.put(POISON);   // tell each consumer to stop
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        producer.start();
        producer.join();
        for (Thread t : consumers) t.join();
        System.out.println("consumed sum of 1..1000 = " + total.get() + " (expected 500500)");
    }

    // ---------- 4. Immutability ----------
    /** Immutable value object: final fields, no setters, defensive copy. Safe to share freely. */
    record Order(String id, List<String> items) {
        Order { items = List.copyOf(items); }                 // caller's list can't change ours
        Order withItem(String item) {                          // "modify" = return a new object
            List<String> copy = new ArrayList<>(items); copy.add(item);
            return new Order(id, copy);
        }
    }

    // ---------- 5. Thread confinement ----------
    private static final ThreadLocal<StringBuilder> BUFFER = ThreadLocal.withInitial(StringBuilder::new);

    static String render(String name) {
        StringBuilder sb = BUFFER.get();       // each thread has its own builder: no sharing, no locks
```

```
        sb.setLength(0);
        return sb.append("Hello, ").append(name).toString();
    }

    public static void main(String[] args) throws Exception {
        signaling();
        threadPool();
        producerConsumer();

        System.out.println("=== Immutability ===");
        List<String> mutable = new ArrayList<>(List.of("pen"));
        Order o1 = new Order("o1", mutable);
        mutable.add("hacked");
        Order o2 = o1.withItem("book");
        System.out.println("o1 = " + o1.items() + ", o2 = " + o2.items());

        System.out.println("=== Thread confinement (ThreadLocal) ===");
        Thread t1 = new Thread(() -> System.out.println(render("T1")));
        t1.start(); t1.join();
        Thread t2 = new Thread(() -> System.out.println(render("T2")));
        t2.start(); t2.join();
    }
}
```

Output:

```
=== Signaling: B must run after A ===
A: charge payment
B: send email (after payment)
=== Thread pool ===
hand-made pool ran 20 tasks on 3 threads
ThreadPoolExecutor + CompletableFuture: total = 118
=== Producer-consumer ===
consumed sum of 1..1000 = 500500 (expected 500500)
=== Immutability ===
o1 = [pen], o2 = [pen, book]
=== Thread confinement (ThreadLocal) ===
Hello, T1
Hello, T2
```

12.20 Signaling pattern

What it is

One thread **notifies** another that an event happened, so the second can proceed. "B must run after A."

> **Analogy.** A relay race baton: the next runner can't start until the baton is handed over.

The problem it solves

Enforcing ordering between threads without busy-waiting (spinning on a flag).

How it works

1. Create a signal object in the "not yet" state: `new Semaphore(0)`, `new CountDownLatch(1)`, a `CompletableFuture`, or a flag + condition variable.
2. The waiting thread calls `acquire()` / `await()` / `join()`.
3. The signalling thread calls `release()` / `countDown()` / `complete(value)`.
4. The waiter wakes and continues. The signal also provides happens-before, so the waiter sees everything A did.

Tool	Reusable	Carries a value	Good for
`Semaphore(0)`	Yes (each release = one signal)	No	Ping-pong between threads (FooBar, chapter 13)
`CountDownLatch(1)`	No	No	One-time "ready/go"
`CompletableFuture`	No	Yes	"Here is the result"
Condition variable	Yes	Via shared state	Complex conditions

Java example

`signaling()` above: B starts first but waits on `aDone.acquire()`; A logs and calls `release()`. Output shows A's line before B's every time.

Trade-offs

Semaphores are compact but easy to mis-count; latches can't be reset; futures are clean but single-use.

When not to use it

If the "signal" is really "here's a work item", use a `BlockingQueue` (producer-consumer).

Common mistakes

- Signalling before the waiter starts waiting with a tool that *forgets* signals (plain `notify()` without a state flag = lost wakeup). Semaphores and latches remember.

Real-world examples

`CompletableFuture` chains in Spring WebFlux (Spring's non-blocking web framework)/Netty **callbacks** (functions handed to a library so it can call you back later); start-up gates; "order paid → send confirmation" flows inside one service.

In the interview

"Signaling means one thread waits for an event raised by another. I use a zero-permit semaphore for repeated hand-offs, a latch for one-time events, or a `CompletableFuture` when a result travels with the signal."

Self-check

Q: Why is a bare `notify()` risky for signaling? **A:** If it happens before the other thread waits, the signal is lost; always pair it with a state variable checked in a loop.

TL;DR

Signal = "you may go now". Semaphore(0), latch, or future. Make sure signals can't be lost.

12.21 Thread pool pattern

What it is

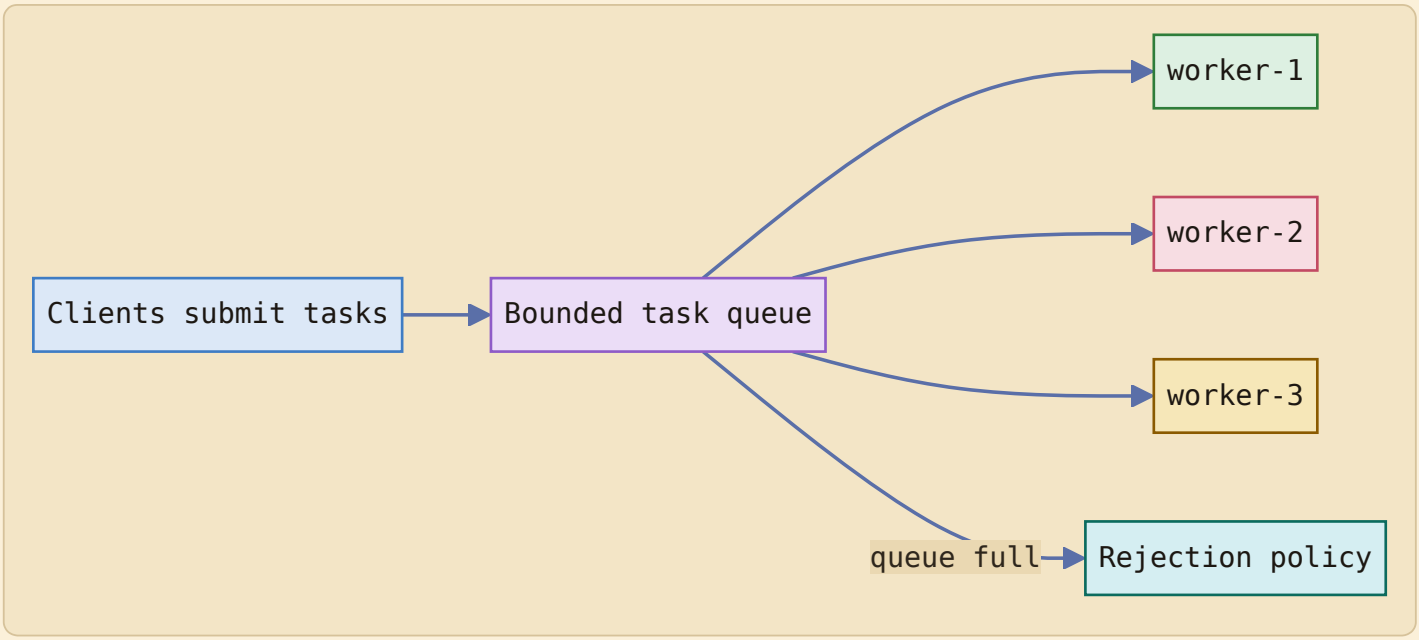
A fixed set of **worker threads** takes tasks from a shared **queue** and runs them, reusing threads instead of creating one per task.

> **Analogy.** A taxi rank: a few taxis serve a queue of passengers; taxis are reused, and if the queue gets too long, the rank manager turns people away.

The problem it solves

Creating/destroying threads per task is expensive and unbounded concurrency can overwhelm CPU, memory and downstream systems. A pool bounds concurrency and **amortises** thread creation (spreads its one-time cost over many tasks).

How it works



1. Workers loop: `task = queue.take(); task.run();`.
2. `submit` puts the task in the queue.
3. `ThreadPoolExecutor` rules: while fewer than **core** threads run, a new thread is created per task; then tasks go into the **queue**; if the queue is full, extra threads are created up to **max**; if still full, the **rejection policy** runs (`AbortPolicy` throws — default; `CallerRunsPolicy` runs it on the submitting thread, slowing producers; `DiscardPolicy`; `DiscardOldestPolicy`).
4. Shutdown: stop accepting, let workers drain the queue, then exit (the hand-made pool uses one "**poison pill**" — a special marker task meaning "stop now" — per worker). The "is it shut down? then enqueue" check in `submit` is a check-then-act, so `submit` and the pill-adding step of shutdown hold the same lock — otherwise a task could be enqueued *behind* the pills and silently never run.

Sizing: CPU-bound ≈ number of cores; I/O-bound ≈ cores × (1 + wait/compute) — or virtual threads.

Java example

`SimpleThreadPool` above (hand-made, to see the mechanics) and a production `ThreadPoolExecutor` with a bounded `ArrayBlockingQueue(100)` and `CallerRunsPolicy`, combined with `CompletableFuture.thenCombine`.

Trade-offs

Choice	Effect
Bounded queue	Protects memory; needs a rejection policy
Unbounded queue (`newFixedThreadPool`)	Never rejects, may run out of memory, and max-pool-size is never used
`CallerRunsPolicy`	Natural back-pressure: submitter slows down
Many pools (bulkheads)	Isolation between workloads; more threads overall

When not to use it

For huge numbers of blocking tasks on Java 21, virtual-thread-per-task is simpler. For fork/join-style recursive CPU work, use `ForkJoinPool` (**work stealing**: an idle thread takes queued tasks from a busy thread's queue).

Common mistakes

- Tasks that wait for other tasks in the same bounded pool (thread-starvation deadlock).
- Swallowed exceptions in `submit`ted tasks.
- Forgetting `shutdown()`.
- Using `ThreadLocal`s in pooled threads without clearing them (values leak between tasks).

Real-world examples

Tomcat's connector thread pool (`maxThreads`, default 200), Spring's `ThreadPoolTaskExecutor`, database connection pools (same idea for connections), `ForkJoinPool.commonPool()` behind parallel streams and `CompletableFuture.supplyAsync` without an executor.

In the interview

"A thread pool reuses a bounded set of workers fed by a queue. I size it by workload type, use a bounded queue with an explicit rejection policy for back-pressure, and give slow dependencies their own pool as a bulkhead."

Self-check

Q: In `ThreadPoolExecutor`, when are threads beyond `corePoolSize` created? **A:** Only when the queue is full (and fewer than max threads exist).

TL;DR

Workers + queue + rejection policy. Bound everything. Size by CPU vs I/O.

12.22 Producer-consumer pattern

What it is

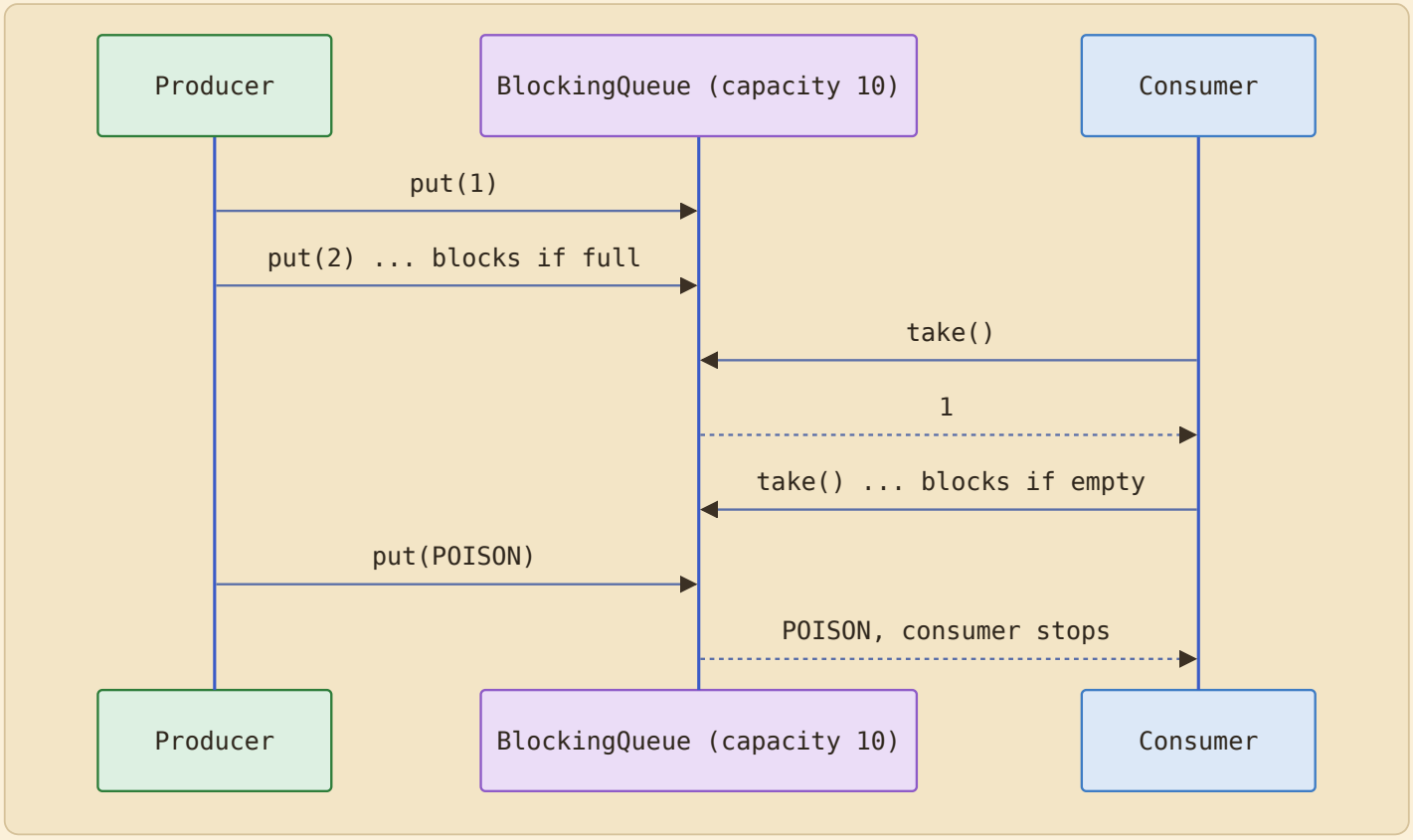
Producers create work items and put them into a shared **buffer**; **consumers** take items out and process them. The buffer decouples their speeds.

> **Analogy.** A restaurant kitchen's order rail: waiters (producers) clip tickets on the rail, cooks (consumers) take them off. If the rail is full, waiters wait; if empty, cooks wait.

The problem it solves

Producers and consumers run at different rates. A bounded buffer smooths bursts and applies **back-pressure** (producers slow down when consumers fall behind), without either side knowing about the other.

How it works



1. Share a bounded `BlockingQueue`.
2. Producers `put` (waits when full); consumers `take` (waits when empty).
3. Shutdown: each consumer must learn when to stop — a **poison pill** per consumer, or interrupting consumers, or a "done" flag plus `poll(timeout)`.

Java example

`producerConsumer()` above: 1 producer, 2 consumers, queue capacity 10, two poison pills. Sum is exactly 500500. Chapter 13 builds the blocking queue itself two ways.

Trade-offs

Pros	Cons
Decouples rates, enables back-pressure	Adds latency (queueing)
Easy to scale consumers	Ordering across multiple consumers isn't preserved
Clear separation of concerns	Need a shutdown protocol

When not to use it

When the caller needs the result immediately and synchronously, a direct call is simpler.

Common mistakes

- Unbounded buffer → no back-pressure → memory blow-up.
- One poison pill for N consumers → N−1 consumers hang forever.
- Busy-polling with `poll()` in a loop instead of blocking `take()`.

Real-world examples

Apache **Kafka** (a distributed, durable message log) is producer-consumer across machines (topics are the buffer; **consumer groups** — sets of consumers that split a topic's messages between them — scale consumers). Inside the Kafka Java producer, application threads append records to an in-memory accumulator and a background sender thread ships batches. Logging frameworks' async appenders, and `ThreadPoolExecutor` itself (submitters produce, workers consume).

In the interview

"Producer-consumer decouples producers from consumers with a bounded blocking queue: put waits when full, take waits when empty, giving natural back-pressure. I stop consumers with one poison pill each, and scale consumers independently."

Self-check

Q: Why bounded? **A:** So a slow consumer pushes back on producers instead of letting the queue grow until memory runs out.

TL;DR

Bounded `BlockingQueue` between producers and consumers. Back-pressure for free. One poison pill per consumer.

12.23 Reader-writer pattern

What it is

A design where **many readers may access shared data at the same time, but a writer needs exclusive access**.

> **Analogy.** A shared noticeboard: many people read it at once; when someone replaces a notice, readers step back for a moment.

The problem it solves

Read-mostly data (config, catalogues, routing tables) shouldn't be serialised through one mutex.

How it works

Two classic implementations:

1. **Read-write lock** (12.10): `readLock` for readers, `writeLock` for writers. Choose fair mode if writers could starve.
2. **Copy-on-write snapshot**: keep an immutable object in a `volatile` field. Readers just read the field (no lock). A writer builds a *new* immutable copy and swaps the reference (use a lock or `AtomicReference.updateAndGet` if several writers may race).

```
private volatile Map<String, String> config = Map.of();       // immutable snapshot
String get(String k) { return config.get(k); }                  // readers: no lock
synchronized void put(String k, String v) {                      // writers: serialized
    Map<String, String> copy = new HashMap<>(config);
    copy.put(k, v);
    config = Map.copyOf(copy);                                    // publish new snapshot
}
```

	RW lock	Copy-on-write snapshot
Reader cost	Acquire shared lock	One volatile read
Writer cost	Exclusive lock, in-place update	Copy whole structure
Readers see	Latest data	A consistent snapshot (maybe slightly old)
Best for	Frequent-ish writes, big data	Rare writes, small-to-medium data

Java example

`ConfigStore` (RW lock) in `locks/Main.java`; the snapshot code just above; `CopyOnWriteArrayList` in `collections/Main.java`.

Trade-offs

See table. Also: RW lock readers must not upgrade to write.

When not to use it

Write-heavy data — plain mutex or `ConcurrentHashMap` is better.

Common mistakes

Mutating the shared snapshot in place (breaks all readers' assumptions); upgrading read → write locks.

Real-world examples

Feature-flag and configuration clients that reload periodically and swap an immutable snapshot; Linux kernel RCU (read-copy-update) is the same idea at OS level.

In the interview

"For read-heavy data I use either a read-write lock or, if writes are rare, an immutable snapshot in a volatile field that writers replace — readers never block."

Self-check

Q: What guarantees readers see a fully built new snapshot? **A:** The `volatile` write of the reference happens-before later volatile reads, and the snapshot is immutable.

TL;DR

Many readers, one writer. RW lock, or immutable snapshot + volatile swap.

12.24 Immutability & thread confinement

What it is

Two ways to be thread-safe **without any synchronization**:

- **Immutability**: an object whose state can't change after construction (all fields `final`, no setters, no leaking mutable internals). Examples: `String`, `Integer`, `LocalDate`, `BigDecimal`, records with immutable components, `List.of(...)`.
- **Thread confinement**: data that only one thread ever touches. Local variables (stack confinement), `ThreadLocal<T>` (one copy per thread), or a design where one thread owns a structure (event loop, **actor** — an object with private state that processes messages from its mailbox one at a time).

> **Analogy.** Immutability: a printed book — everyone can read it at once, nobody can change it. Confinement: a personal notebook — you write freely because nobody else reads it.

The problem it solves

Races need *shared mutable* state. Remove "mutable" (immutability) or remove "shared" (confinement) and races are impossible.

How it works

Immutable class checklist:

1. Make the class `final` (or a `record`) so subclasses can't add mutability.
2. Make all fields `private final`.
3. No setters; "modifiers" return new objects (`withItem(...)`).
4. **Defensive-copy** (copy, so the caller's later changes can't reach you) mutable inputs in the constructor (`List.copyOf`) and don't expose mutable internals.
5. Don't let `this` escape during construction.

Confinement options:

1. Keep data in local variables.
2. `ThreadLocal.withInitial(...)` for per-thread helpers (formatters, buffers). Call `remove()` when used in pooled threads.
3. Single-writer design: one thread owns the state; others send it messages through a queue.

Java example

`Order` record above: the compact constructor does `List.copyOf(items)`, so the caller's later `mutable.add("hacked")` doesn't affect `o1`; `withItem` returns a new `Order`. `BUFFER` is a `ThreadLocal<StringBuilder>` — each thread gets its own builder.

Trade-offs

	Immutability	Confinement
Cost	New object per change (GC pressure — more garbage for the collector to clean)	Per-thread copies; ThreadLocal leaks if not cleaned
Sharing	Share freely	Not shareable by definition
Reasoning	Easiest possible	Easy, if confinement is really enforced

When not to use it

Very large objects changed at high frequency (copying costs too much) — use a concurrent structure instead.

Common mistakes

- `final List<String> items` is not immutable if the list itself is mutable and exposed.
- `ThreadLocal` in thread pools without `remove()` — values leak into the next task; with many virtual threads, heavy `ThreadLocal`s waste memory (*scoped values* are the alternative: a **preview** — an experimental feature you must opt into — in Java 21, final in Java 25 via JEP 506).

- Sharing `SimpleDateFormat` (mutable, not thread-safe) instead of using immutable `DateTimeFormatter`.

Real-world examples

- `java.time` types and `DateTimeFormatter` are immutable and thread-safe; `SimpleDateFormat` is not.
- **Netty** and **Vert.x** confine each connection to one event-loop thread, so handler code needs no locks.
- Redis executes commands on a single main thread (confinement at system level).
- Spring's `@Transactional` (marks a method to run inside a database transaction) stores the current transaction in `ThreadLocal`s (via `TransactionSynchronizationManager`).

In the interview

"The easiest thread-safety is none needed: make value objects immutable — final fields, defensive copies, 'with' methods — and confine mutable state to one thread with locals, ThreadLocal, or a single owner thread fed by a queue."

Self-check

Q: Is a class with `private final Date created` immutable? **A:** Not unless it copies the `Date` in and out — `Date` is mutable. Prefer `Instant`.

TL;DR

No shared mutable state → no races. Immutable objects share freely; confined objects aren't shared.

Part H — Concurrency in LLD interviews

12.25 How concurrency shows up in LLD interviews (parking, seat booking, inventory)

What it is

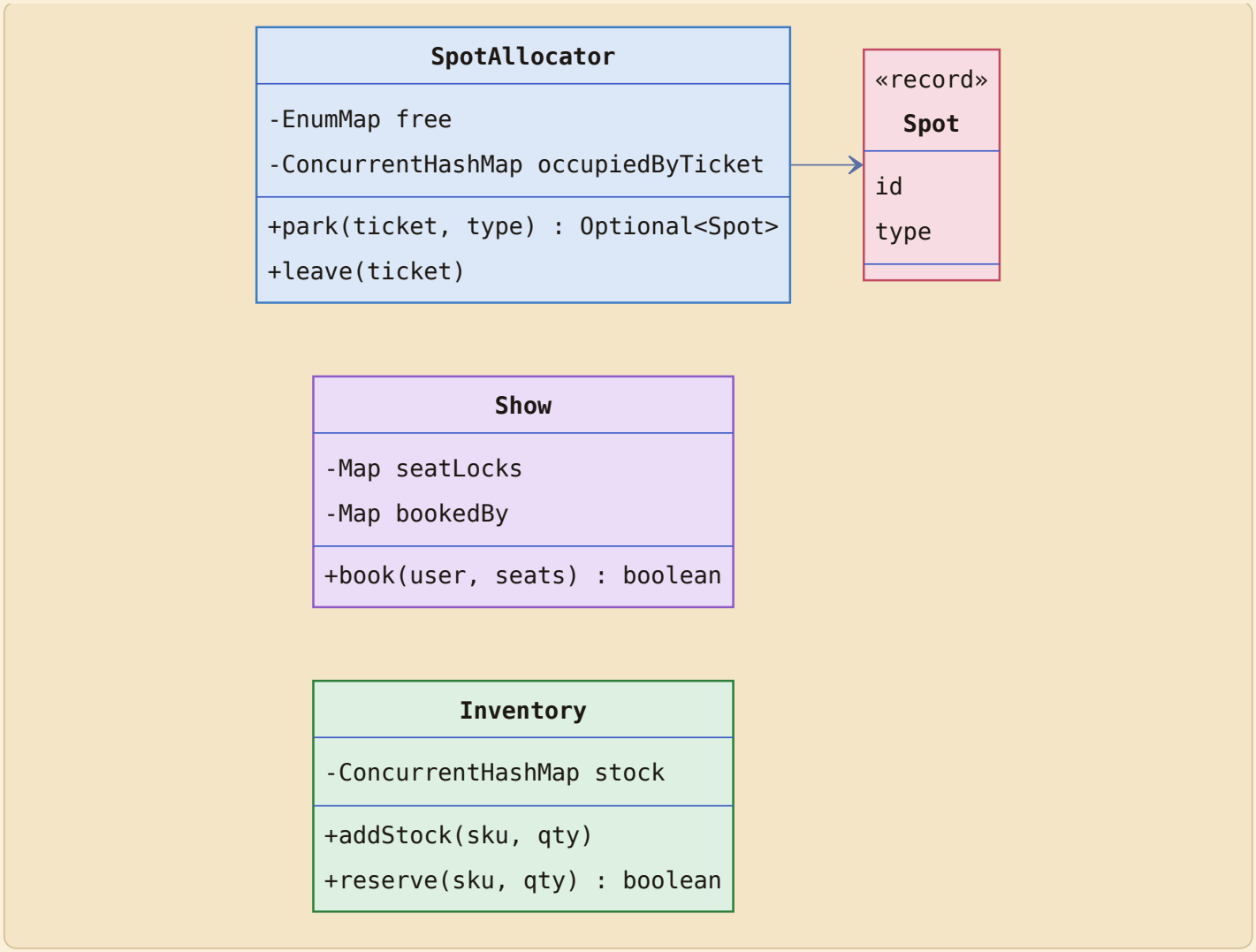
Most LLD case studies have a **contended resource**: a parking spot, a movie seat, a hotel room, a limited stock item, an elevator request, a wallet balance. The interviewer checks whether your design prevents double allocation and still performs.

The problem it solves

Showing you can turn a single-threaded class diagram into a **thread-safe design** with the smallest correct synchronization.

How it works — a 6-step recipe

1. **Name the shared mutable state** (free spots, seat → owner map, stock counts).
2. **State the invariant** ("a spot has at most one car", "stock ≥ 0", "a seat has at most one booking").
3. **Find the critical operation** (usually a check-then-act: *find free → assign*).
4. **Pick the smallest tool that makes it atomic:**
   - one item claimed → atomic op: `queue.poll()`, `map.putIfAbsent`, `AtomicBoolean.compareAndSet`, `AtomicInteger` CAS loop;
   - several items all-or-nothing → per-item locks acquired **in sorted order** (or one per-show/per-floor lock);
   - across processes/servers → database constraints/transactions, optimistic versioning, or a distributed lock.
5. **Choose granularity:** per floor/show/SKU (stock-keeping unit — one distinct product) locks rather than one global lock.
6. **Handle time:** temporary holds with expiry (seat held 10 minutes during payment), timeouts, and **idempotent** retries (safe to repeat, because doing the operation twice has the same effect as doing it once; see System Design Fundamentals).


SpotAllocator
-EnumMap free
-ConcurrentHashMap occupiedByTicket
+park(ticket, type) : Optional<Spot>
+leave(ticket)
«record»
Spot
id
type
Show
-Map seatLocks
-Map bookedBy
+book(user, seats) : boolean
Inventory
-ConcurrentHashMap stock
+addStock(sku, qty)
+reserve(sku, qty) : boolean

Java example

```java
// File: code/lld/ch12/lldcases/Main.java
package lld.ch12.lldcases;

import java.util.ArrayList;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.TreeSet;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.ReentrantLock;

/** Thread-safe building blocks for three interview favourites. */
public class Main {

    // ======================================================================
    // 1. Parking lot: many gates allocate spots at the same time.
    // ======================================================================
    enum SpotType { COMPACT, LARGE }
    record Spot(String id, SpotType type) { }

    static class SpotAllocator {
        // One lock-free queue of FREE spots per type. poll() is atomic: two gates can never get the same spot.
        private final Map<SpotType, ConcurrentLinkedQueue<Spot>> free = new EnumMap<>(SpotType.class);
        private final ConcurrentHashMap<String, Spot> occupiedByTicket = new ConcurrentHashMap<>();

        SpotAllocator(List<Spot> spots) {
            for (SpotType t : SpotType.values()) free.put(t, new ConcurrentLinkedQueue<>());
            spots.forEach(s -> free.get(s.type()).add(s));   // EnumMap itself is never modified after this
        }
        Optional<Spot> park(String ticket, SpotType type) {
            Spot s = free.get(type).poll();
            if (s != null) occupiedByTicket.put(ticket, s);
            return Optional.ofNullable(s);
        }
        void leave(String ticket) {
            Spot s = occupiedByTicket.remove(ticket);        // remove is atomic: double-exit is harmless
            if (s != null) free.get(s.type()).offer(s);
        }
        // ConcurrentLinkedQueue.size() walks the queue (O(n)) and is only an estimate while other
        // threads change it; fine for a report once all gates are idle, not for allocation logic.
        int freeCount(SpotType t) { return free.get(t).size(); }
    }

    // ======================================================================
    // 2. Seat booking: all-or-nothing booking of several seats.
    // ======================================================================
    static class Show {
        private final Map<String, ReentrantLock> seatLocks = new ConcurrentHashMap<>();
        private final Map<String, String> bookedBy = new ConcurrentHashMap<>();

        Show(Set<String> seatIds) { seatIds.forEach(id -> seatLocks.put(id, new ReentrantLock())); }

        /** Locks seats in sorted order (prevents deadlock), checks all, then books all. */
        boolean book(String user, Set<String> seats) {
            List<ReentrantLock> held = new ArrayList<>();
            try {
                for (String id : new TreeSet<>(seats)) {         // global order: A1 < A2 < A3
                    ReentrantLock l = seatLocks.get(id);
                    if (l == null) return false;                  // unknown seat
```

```
                    l.lock();
                    held.add(l);
                }
                for (String id : seats) if (bookedBy.containsKey(id)) return false;   // someone got one first
                for (String id : seats) bookedBy.put(id, user);
                return true;
            } finally {
                for (int i = held.size() - 1; i >= 0; i--) held.get(i).unlock();
            }
        }
        String owner(String seat) { return bookedBy.get(seat); }
    }

    // ================================================================
    // 3. Inventory: never sell more than we have (no overselling).
    // ================================================================
    static class Inventory {
        private final ConcurrentHashMap<String, AtomicInteger> stock = new ConcurrentHashMap<>();

        void addStock(String sku, int qty) {
            stock.computeIfAbsent(sku, k -> new AtomicInteger()).addAndGet(qty);
        }
        /** CAS loop: decrement only if enough stock remains. */
        boolean reserve(String sku, int qty) {
            AtomicInteger s = stock.get(sku);
            if (s == null) return false;
            while (true) {
                int cur = s.get();
                if (cur < qty) return false;
                if (s.compareAndSet(cur, cur - qty)) return true;
            }
        }
        int available(String sku) { AtomicInteger s = stock.get(sku); return s == null ? 0 : s.get(); }
    }

    // ----------------------------------------------------------------
    /** Run 'n' tasks that all start at the same instant (maximises contention). */
    static void raceAll(int n, java.util.function.IntConsumer task) throws InterruptedException {
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService ex = Executors.newFixedThreadPool(16)) {
            for (int i = 0; i < n; i++) {
                final int id = i;
                ex.submit(() -> { go.await(); task.accept(id); return null; });
            }
            go.countDown();
        } // try-with-resources close() waits for all tasks (Java 19+)
    }

    public static void main(String[] args) throws Exception {
        List<Spot> spots = new ArrayList<>();
        for (int i = 1; i <= 10; i++) spots.add(new Spot("C" + i, SpotType.COMPACT));
        SpotAllocator lot = new SpotAllocator(spots);
        AtomicInteger parked = new AtomicInteger();
        Set<String> handedOut = ConcurrentHashMap.newKeySet();
        raceAll(100, car -> lot.park("T" + car, SpotType.COMPACT).ifPresent(s -> { parked.incrementAndGet();
handedOut.add(s.id()); }));
        System.out.println("parking: 100 cars raced for 10 spots -> parked " + parked.get()
                + ", distinct spots " + handedOut.size() + ", free left " + lot.freeCount(SpotType.COMPACT));

        Show show = new Show(Set.of("A1", "A2", "A3", "A4"));
        AtomicInteger wins = new AtomicInteger();
        raceAll(50, u -> { if (show.book("user" + u, Set.of("A2"))) wins.incrementAndGet(); });
        System.out.println("booking: 50 users wanted seat A2 -> successful bookings " + wins.get());
        AtomicInteger groupWins = new AtomicInteger();
        raceAll(2, u -> { if (show.book(u == 0 ? "alice" : "bob", u == 0 ? Set.of("A3", "A4") : Set.of("A4",
"A3"))) groupWins.incrementAndGet(); });
        System.out.println("booking: alice {A3,A4} vs bob {A4,A3} (opposite order) -> winners " + groupWins.get()
                + ", A3 and A4 same owner: " + show.owner("A3").equals(show.owner("A4")));
```

```
        Inventory inv = new Inventory();
        inv.addStock("iphone", 50);
        AtomicInteger sold = new AtomicInteger();
        raceAll(200, o -> { if (inv.reserve("iphone", 1)) sold.incrementAndGet(); });
        System.out.println("inventory: 200 buyers, 50 in stock -> sold " + sold.get() + ", left " +
inv.available("iphone"));
    }
}
```

Explaining each part:

- **Parking**: free spots per type live in a `ConcurrentLinkedQueue`. `poll()` atomically removes one spot, so two gates can never receive the same spot — no explicit lock. `leave` uses `ConcurrentHashMap.remove(ticket)`, so a duplicated exit event can't return a spot twice.
- **Seat booking**: one `ReentrantLock` per seat. `book` locks the requested seats **in sorted order** (breaking circular wait), checks that *all* are free, then books them all — all-or-nothing. Alice asks for `{A3, A4}` and Bob for `{A4, A3}` at the same moment: exactly one wins, no deadlock, and both seats get the same owner.
- **Inventory**: an `AtomicInteger` per SKU and a CAS loop that decrements only if enough stock remains — 200 buyers, 50 items → exactly 50 sold, never negative.
- `raceAll` releases all tasks through a `CountDownLatch` at once to maximise contention.

Output:

```
parking: 100 cars raced for 10 spots -> parked 10, distinct spots 10, free left 0
booking: 50 users wanted seat A2 -> successful bookings 1
booking: alice {A3,A4} vs bob {A4,A3} (opposite order) -> winners 1, A3 and A4 same owner: true
inventory: 200 buyers, 50 in stock -> sold 50, left 0
```

Trade-offs

Resource	In-memory approach	Across many servers
Parking spot	Lock-free queue per type/floor, or per-floor lock	DB row update `WHERE status='FREE'` + check affected rows
Seat	Per-show or per-seat locks in sorted order; hold with TTL (time-to-live, an expiry)	Unique constraint on (show, seat); short-lived holds in Redis (`SET NX PX`)
Inventory	`AtomicInteger` CAS / `compute`	`UPDATE stock SET qty = qty - 1 WHERE sku = ? AND qty >= 1`; reservations with expiry

When not to use it

Don't bolt `synchronized` onto every method of every class — interviewers read that as "doesn't know where the contention is". Protect the few operations that touch contended state.

Common mistakes

- A global `synchronized` on `ParkingLot.park()` — correct but serialises every gate.
- Checking availability with one call and booking with another (check-then-act).
- Locking seats in the order the user clicked them (deadlock between two multi-seat bookings).
- Forgetting that in-memory locks don't work across multiple app servers.

Real-world examples

- **BookMyShow / Ticketmaster-style** checkout: selected seats are *held* for a few minutes while you pay; if payment doesn't complete, the hold expires and the seats return to the pool — a lock with a lease.
- **Flash sales** (e.g. limited-stock phone launches) decrement stock atomically, often in Redis (`DECR` / Lua scripts, which Redis runs atomically) or with conditional SQL updates, to avoid overselling.
- **Uber-style dispatch**: offering one driver to one rider at a time is the same "claim a resource atomically" problem.

In the interview

> **30-second answer:** "The critical section is 'find a free X and assign it'. Inside one JVM I make that a single atomic step — a lock-free queue poll for spots, per-seat locks taken in sorted order for multi-seat booking, a CAS decrement for stock. Across servers I'd push it into the database with a conditional update or unique constraint, and add holds with expiry for seats during payment."

Follow-ups (with answers):

- *Two users book the same seat at the same millisecond?* Only one `putIfAbsent`/locked check-and-book succeeds; the other gets "seat unavailable".
- *How do you release a seat if payment fails or the user leaves?* Store a hold with an expiry time; a scheduler or lazy check frees expired holds (see the TTL cache in chapter 13).
- *Why not one global lock?* Correct but no parallelism: bookings for different shows would wait for each other.
- *What if the service runs on 10 machines?* JVM locks don't cross processes: use DB transactions/constraints, optimistic locking with a version column, or a distributed lock with a lease.

Self-check

- **Q:** Which Coffman condition does sorting seat ids break? **A:** Circular wait.
- **Q:** Why is `queue.poll()` safe for allocating spots? **A:** It's a single atomic remove; each spot can be removed by only one thread.

TL;DR

Find the contended resource and its invariant, make "check + claim" one atomic step with the smallest tool, keep locks fine-grained and ordered, and move to DB/distributed mechanisms when there's more than one server.

12.26 Choosing the right tool (cheat sheet)

I need to…	Use
Increment a shared counter	`AtomicInteger`/`AtomicLong`; `LongAdder` if very hot
Stop a loop from another thread	`volatile boolean` flag or `interrupt()`
Make a multi-step update atomic	`synchronized` or `ReentrantLock`
Give up if the lock is busy	`tryLock(timeout)`
Many readers, few writers	`ReentrantReadWriteLock`, `StampedLock`, or immutable snapshot in `volatile`
Limit concurrent access to N	`Semaphore(N)`
Wait for a condition on shared state	`Lock` + `Condition` (or `wait/notifyAll`) in a `while` loop
Wait for N events once	`CountDownLatch`
N threads meet each phase	`CyclicBarrier` / `Phaser`
Hand work between threads	Bounded `BlockingQueue`
Shared map	`ConcurrentHashMap` with `putIfAbsent`/`compute`/`merge`
Read-mostly list	`CopyOnWriteArrayList`
Run tasks	`ExecutorService` (bounded); virtual threads for blocking I/O
Split CPU work	`ForkJoinPool` / parallel streams
Combine async results	`CompletableFuture`
Avoid synchronization entirely	Immutability, thread confinement

12.27 Chapter self-check

1. **Q:** Concurrency vs parallelism in one line each? **A:** Concurrency = many tasks in progress (structure); parallelism = many tasks executing at the same instant (hardware).
2. **Q:** Name the six `Thread.State` values. **A:** NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
3. **Q:** Why is `volatile` not enough for `count++`? **A:** It gives visibility, not atomicity of read-modify-write.
4. **Q:** Three happens-before rules? **A:** Unlock → later lock of the same monitor; volatile write → later read; `start()` → thread's actions; thread's actions → `join()` return.
5. **Q:** Why always `while` around `wait()`? **A:** Spurious wakeups and competing threads; the condition must be re-checked.
6. **Q:** Four Coffman conditions? **A:** Mutual exclusion, hold and wait, no preemption, circular wait.
7. **Q:** Deadlock vs livelock vs starvation? **A:** Nobody progresses and all wait / everybody runs but nobody progresses / some progress while one never does.
8. **Q:** `CountDownLatch` vs `CyclicBarrier`? **A:** One-shot count of events vs reusable meeting of N threads.
9. **Q:** What is ABA and its fix? **A:** Value changes A→B→A so CAS can't notice; add a version (`AtomicStampedReference`).
10. **Q:** How to allocate a parking spot to exactly one of many gates without a lock? **A:** Keep free spots in a concurrent queue and `poll()` — an atomic remove.

12.28 Chapter TL;DR

- Races come from **shared mutable state**; fix by making critical sections atomic, or by removing sharing/mutability.
- Visibility needs **happens-before** edges: locks, `volatile`, start/join, concurrent utilities.
- Toolbox: `synchronized` → `ReentrantLock` (try/timed/fair/conditions) → RW locks → semaphores → latches/barriers → atomics/CAS → concurrent collections.
- Liveness: prevent deadlock with **lock ordering** or **tryLock + back-off**; add jitter against livelock; fairness against starvation.
- Patterns: signaling, thread pool, producer-consumer, reader-writer, immutability/confinement.
- In LLD interviews: identify the contended resource, keep "check + claim" atomic, lock fine-grained and in order, and say what changes when there are many servers.

Chapter 13 — Classic Concurrency Problems

> **Goal.** Chapter 12 gave you the toolbox. This chapter uses it on the concurrency problems that interviewers actually ask — from LeetCode's (a popular coding-practice website) "Concurrency" section (FooBar, Zero-Even-Odd, FizzBuzz, H2O, Dining Philosophers) to "design a thread-safe X" **machine-coding** rounds (interviews where you write working code for a small system in an hour or two): TTL cache, concurrent hash map, blocking queue, Bloom filter, parallel merge sort, **rate limiter** (a component that caps how many requests a client may make per second).

How each problem is laid out: *Problem → Approach → Java solution (+ real output) → Why it's correct → Pitfalls → Real-world examples → In the interview → Self-check → TL;DR.*

All code is Java 21, standard library only, in `code/lld/ch13/<problem>/Main.java`. Run from `chapters/13-concurrency-problems/code`:

```
javac --release 21 -d /tmp/lld_13 $(find . -name '*.java')
java -cp /tmp/lld_13 lld.ch13.foobar.Main
```

Where the real order of events can vary between runs, the demo *verifies* the result and prints `true/false`, so the output shown here is what you get every time.

Key terms (all built in Chapter 12 — a one-line reminder each):

- **Thread-safe** — behaves correctly however threads interleave. A **race** is a bug where the result depends on the timing of threads.
- **Lock / monitor** — a token only one thread can hold at a time. Every Java object has a built-in one (its *monitor*), taken by `synchronized`; `ReentrantLock` is the explicit version.
- **Semaphore** — a counter of permits: `acquire()` takes one (waiting if none are left), `release()` gives one back.
- **Condition variable** (`wait/notifyAll`, or `Condition` with a `ReentrantLock`) — lets a thread sleep, releasing its lock, until another thread changes the shared state and wakes it.
- **Barrier** (`CyclicBarrier`) — a meeting point where N threads wait until all N have arrived, then all continue.
- **Atomic / CAS** — an *atomic* step happens all at once or not at all; *compare-and-swap* updates a value only if it still equals what you read. CAS is the basis of **lock-free** code (no locks; a thread that loses simply retries).
- **Happens-before** — the Java Memory Model rule that guarantees one thread's writes are visible to another (e.g. through the same lock, a `volatile` field, or a semaphore release → acquire).
- **Deadlock / livelock / starvation** — threads wait on each other forever / keep retrying but nobody progresses / one thread never gets its turn.

Problem map — which tool solves what

#	Problem	Core idea	Main tools
13.1	Print FooBar Alternately	Pass a "baton" between 2 threads	`Semaphore` ×2 (or monitor + turn flag)
13.2	Print Zero Even Odd	A dispatcher thread decides who goes next	`Semaphore` ×3
13.3	Fizz Buzz Multithreaded	4 threads, one shared counter, each waits for "its" numbers	`synchronized` + `wait/notifyAll`
13.4	Building H2O	Group threads into sets of 2 H + 1 O	`Semaphore` (slots) + `CyclicBarrier` (group)
13.5	Dining Philosophers	Avoid deadlock over shared forks	Lock ordering, or a "waiter" semaphore
13.6	Thread-safe cache with TTL	Expiring entries, no stampede	`ConcurrentHashMap`, `compute`, scheduled sweeper
13.7	Concurrent HashMap (simplified)	Lock striping (N locks, each guarding a slice of the data)	`ReentrantLock[]`, all-stripe resize
13.8	Bounded Blocking Queue	Wait when full / empty	`wait/notifyAll` **and** `Lock` + 2 `Condition`s
13.9	Concurrent Bloom Filter	Lock-free bit set	`AtomicLongArray` + atomic OR
13.10	Multi-threaded Merge Sort	Divide and conquer on cores	`ForkJoinPool`, `RecursiveAction`
13.11	Rate limiter (token bucket)	Atomic refill-and-take	`synchronized`, or CAS on an immutable state

Reusable mental checklist for any concurrency problem

1. What is the **shared state**? (a turn flag, a counter, a map, a buffer)
2. What **ordering or invariant** must hold? ("foo before bar", "never more than 2 H in a molecule", "never more than capacity items")
3. Who **waits**, for **what condition**, and who **signals** it?
4. Pick the primitive: turn-taking → semaphores; "wait until predicate" → condition variable; groups → barrier; mutual exclusion → lock; single-variable update → atomic.
5. Check **liveness** (the guarantee that every thread eventually makes progress): can anyone wait forever? Does every thread exit at the end?

13.1 Print FooBar Alternately (LeetCode 1115)

Problem

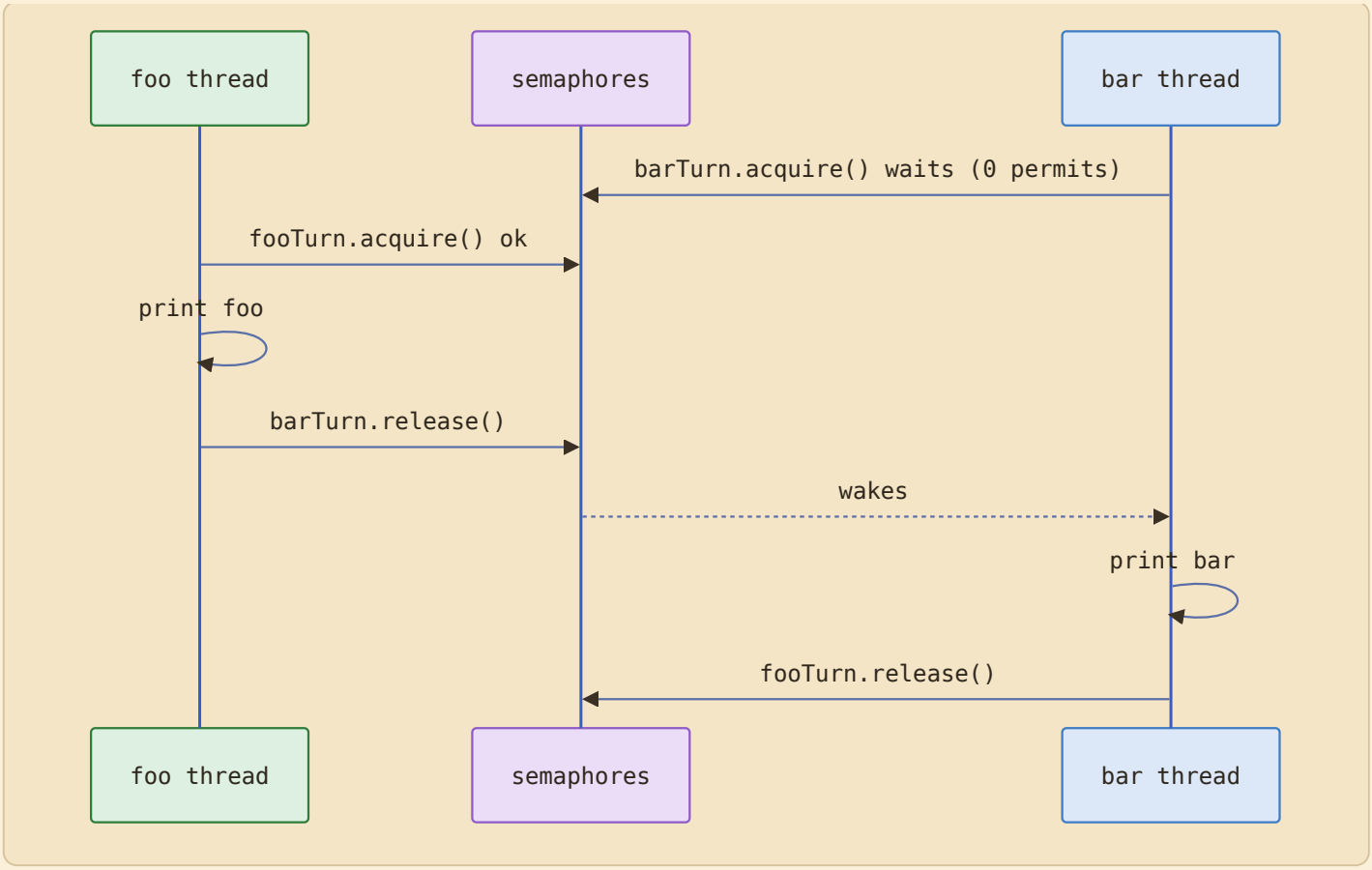
One `FooBar` object is shared by two threads. Thread A calls `foo()`, which prints `"foo"` *n* times; thread B calls `bar()`, which prints `"bar"` *n* times. Make the output exactly `"foobar"` repeated *n* times, whichever thread starts first.

Approach

This is pure **signaling** (12.20): a baton passes back and forth.

- `fooTurn = Semaphore(1)` — foo may go first.
- `barTurn = Semaphore(0)` — bar must wait.
- `foo`: acquire `fooTurn`, print, release `barTurn`. `bar`: acquire `barTurn`, print, release `fooTurn`.

Alternative: one monitor with a boolean `fooTurn`, `while (!fooTurn) wait();` then flip it and `notifyAll()`.


foo thread
semaphores
bar thread
barTurn.acquire() waits (0 permits)
fooTurn.acquire() ok
print foo
barTurn.release()
wakes
print bar
fooTurn.release()
foo thread
semaphores
bar thread

Java solution

```java
// File: code/lld/ch13/foobar/Main.java
package lld.ch13.foobar;

import java.util.concurrent.Semaphore;

/**
 * LeetCode 1115 - Print FooBar Alternately.
 * Thread A calls foo() n times, thread B calls bar() n times; output must be "foobar" repeated n times.
 */
public class Main {

    /** Solution 1: two semaphores used as a baton that is passed back and forth. */
    static class FooBarSemaphore {
        private final int n;
        private final Semaphore fooTurn = new Semaphore(1);   // foo may go first
        private final Semaphore barTurn = new Semaphore(0);   // bar must wait

        FooBarSemaphore(int n) { this.n = n; }

        void foo(Runnable printFoo) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                fooTurn.acquire();       // wait for my turn
                printFoo.run();
                barTurn.release();       // hand the baton to bar
            }
        }
        void bar(Runnable printBar) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                barTurn.acquire();
                printBar.run();
                fooTurn.release();
            }
        }
    }

    /** Solution 2: one monitor + a turn flag (wait/notifyAll). */
    static class FooBarMonitor {
        private final int n;
        private boolean fooTurn = true;                        // guarded by 'this'

        FooBarMonitor(int n) { this.n = n; }

        void foo(Runnable printFoo) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                synchronized (this) {
                    while (!fooTurn) wait();
                    printFoo.run();
                    fooTurn = false;
                    notifyAll();
                }
            }
        }
        void bar(Runnable printBar) throws InterruptedException {
            for (int i = 0; i < n; i++) {
                synchronized (this) {
                    while (fooTurn) wait();
                    printBar.run();
                    fooTurn = true;
                    notifyAll();
                }
            }
        }
    }

    interface Body { void run() throws InterruptedException; }
    static Thread start(Body b) {
```

```
        Thread t = new Thread(() -> { try { b.run(); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); } });
        t.start();
        return t;
    }

    public static void main(String[] args) throws InterruptedException {
        StringBuffer out1 = new StringBuffer();               // StringBuffer is synchronized
        FooBarSemaphore s = new FooBarSemaphore(4);
        Thread b1 = start(() -> s.bar(() -> out1.append("bar")));   // start bar FIRST on purpose
        Thread a1 = start(() -> s.foo(() -> out1.append("foo")));
        a1.join(); b1.join();
        System.out.println("semaphores : " + out1);

        StringBuffer out2 = new StringBuffer();
        FooBarMonitor m = new FooBarMonitor(4);
        Thread b2 = start(() -> m.bar(() -> out2.append("bar")));
        Thread a2 = start(() -> m.foo(() -> out2.append("foo")));
        a2.join(); b2.join();
        System.out.println("monitor    : " + out2);
        System.out.println("both correct: " + (out1.toString().equals("foobar".repeat(4)) &&
out2.toString().equals("foobar".repeat(4))));
    }
}
```

Output:

```
semaphores : foobarfoobarfoobarfoobar
monitor    : foobarfoobarfoobarfoobar
both correct: true
```

Why it's correct

- **Invariant:** the permits in `fooTurn` + `barTurn` always total exactly 1 (each thread releases the *other* semaphore only after acquiring its own). So at most one thread can be printing, and the turns strictly alternate.
- Starting `bar` first doesn't matter: it blocks on `barTurn` (0 permits) until `foo` has printed.
- **Visibility:** a `release()` happens-before the `acquire()` it enables, so each thread sees the other's appends.
- **Termination:** both loops run exactly *n* times, and each acquire has a matching release.

Pitfalls

- Using one semaphore for both threads — a thread can re-acquire its own release and print twice.
- The monitor version with `if` instead of `while`, or `notify()` in more complex variants.
- **Busy-waiting** (spinning in a loop that keeps re-checking a flag) on a `volatile` flag (a field whose writes are immediately visible to other threads) works but burns a CPU core.

Real-world examples

Strict two-party hand-offs: ping-pong protocols, request/acknowledge stages, **double-buffering** (one buffer is filled while the other is read, then they swap) where a renderer and a producer alternate buffers.

In the interview

30-second answer: "Two semaphores as a baton: foo's starts at 1, bar's at 0. Each thread acquires its own, prints, and releases the other's. The total permits are always one, so they alternate."

Follow-ups:

- *Three threads printing A, B, C in order?* Three semaphores in a ring: A(1), B(0), C(0); each releases the next.
- *Without semaphores?* A monitor with a turn variable and `wait/notifyAll`, or `Lock` + `Condition`.

Self-check

- **Q:** What happens if `barTurn` starts with 1? **A:** bar can print first → output may begin with "bar".

TL;DR

Two semaphores, one permit total, passed back and forth.

13.2 Print Zero Even Odd (LeetCode 1116)

Problem

Three threads share a `ZeroEvenOdd(n)` object. `zero()` may only print 0, `even()` only even numbers, `odd()` only odd numbers. Together they must print `0 1 0 2 0 3 … 0 n` — e.g. *n* = 5 → `0102030405`.

Approach

The zero thread acts as a **dispatcher**: after printing a 0 for position *i*, it releases `oddTurn` if *i* is odd, else `evenTurn`. The odd/even thread prints its number and hands control back via `zeroTurn`.

- `zeroTurn = 1`, `oddTurn = 0`, `evenTurn = 0`.
- zero loops *n* times; odd loops over 1, 3, 5…; even over 2, 4, 6….

Java solution

```java
// File: code/lld/ch13/zeroevenodd/Main.java
package lld.ch13.zeroevenodd;

import java.util.concurrent.Semaphore;
import java.util.function.IntConsumer;

/**
 * LeetCode 1116 - Print Zero Even Odd.
 * Three threads: zero() prints only 0s, even() only even numbers, odd() only odd numbers.
 * Together they must print 0 1 0 2 0 3 ... 0 n  (e.g. n=5 -> "0102030405").
 */
public class Main {

    static class ZeroEvenOdd {
        private final int n;
        private final Semaphore zeroTurn = new Semaphore(1);
        private final Semaphore oddTurn = new Semaphore(0);
        private final Semaphore evenTurn = new Semaphore(0);

        ZeroEvenOdd(int n) { this.n = n; }

        void zero(IntConsumer print) throws InterruptedException {
            for (int i = 1; i <= n; i++) {
                zeroTurn.acquire();
                print.accept(0);
                // The zero thread is the "dispatcher": it decides who prints next.
                if (i % 2 == 1) oddTurn.release(); else evenTurn.release();
            }
        }
        void odd(IntConsumer print) throws InterruptedException {
            for (int i = 1; i <= n; i += 2) {
                oddTurn.acquire();
                print.accept(i);
                zeroTurn.release();
            }
        }
        void even(IntConsumer print) throws InterruptedException {
            for (int i = 2; i <= n; i += 2) {
                evenTurn.acquire();
                print.accept(i);
                zeroTurn.release();
            }
        }
    }

    interface Body { void run() throws InterruptedException; }
    static Thread start(Body b) {
        Thread t = new Thread(() -> { try { b.run(); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); } });
        t.start();
        return t;
    }

    public static void main(String[] args) throws InterruptedException {
        for (int n : new int[] {1, 2, 5, 10}) {
            StringBuffer out = new StringBuffer();
            ZeroEvenOdd z = new ZeroEvenOdd(n);
            Thread e = start(() -> z.even(out::append));
            Thread o = start(() -> z.odd(out::append));
            Thread zt = start(() -> z.zero(out::append));
            zt.join(); o.join(); e.join();
            System.out.println("n=" + n + " -> " + out);
        }
    }
}
```

Output:

```
n=1 -> 01
n=2 -> 0102
n=5 -> 0102030405
n=10 -> 01020304050607080910
```

Why it's correct

- Exactly one semaphore holds a permit at any moment → exactly one thread may print.
- The zero thread's loop index *i* decides the parity of the next number, so the right thread is woken.
- Each thread's loop count matches the number of permits it will receive: zero gets *n*, odd gets ⌈n/2⌉, even gets ⌊n/2⌋. So nobody waits forever and everyone exits (e.g. *n* = 1: even's loop runs zero times).

Pitfalls

- Letting odd/even threads compute "the next number" from a shared counter without synchronization.
- Loop bounds that don't match the permits (e.g. even looping `n/2 + 1` times) → a thread waits forever and the program hangs.

Real-world examples

Dispatcher/worker designs where one coordinator routes each step to a specialised worker (e.g. an interpreter routing **opcodes** — the numeric codes of individual instructions — a scheduler assigning alternating tasks).

In the interview

> **30-second answer:** "Three semaphores. Zero starts with one permit. After each 0 it releases odd or even depending on the index, and they release zero back. One permit total in the system, so the order is forced."

Follow-up: *Could you use one lock + condition?* Yes: a shared `state` (ZERO/ODD/EVEN) and `current` number, each thread waits `while (state != mine)`, acts, updates state, `signalAll()`.

Self-check

- **Q:** For *n* = 4, how many times does `odd()` acquire? **A:** Twice (1 and 3).

TL;DR

Zero is the dispatcher; one permit circulates among three semaphores.

13.3 Fizz Buzz Multithreaded (LeetCode 1195)

Problem

Four threads share a `FizzBuzz(n)` object and must together output 1..n where: multiples of 15 → `fizzbuzz` (thread C), multiples of 3 only → `fizz` (thread A), multiples of 5 only → `buzz` (thread B), everything else → the number (thread D).

Approach

Keep one shared counter `current` guarded by the object's monitor. Each thread runs the same loop with its own predicate:

1. `while (current <= n && !mine(current)) wait();`
2. If `current > n` → `notifyAll()` and exit.
3. Print, `current++`, `notifyAll()` (the next number may belong to anyone).

This is the **condition variable** pattern (12.13) with a different predicate per thread. The helper `runWhile(predicate, action)` avoids writing the loop four times (DRY: Don't Repeat Yourself).

Java solution

```java
// File: code/lld/ch13/fizzbuzz/Main.java
package lld.ch13.fizzbuzz;

import java.util.ArrayList;
import java.util.List;
import java.util.function.IntConsumer;
import java.util.function.IntPredicate;

/**
 * LeetCode 1195 - Fizz Buzz Multithreaded.
 * Four threads share one counter 1..n: fizz() prints "fizz" (divisible by 3 only), buzz() "buzz"
 * (by 5 only), fizzbuzz() "fizzbuzz" (by 15), number() prints the number otherwise.
 */
public class Main {

    static class FizzBuzz {
        private final int n;
        private int current = 1;                            // guarded by 'this'

        FizzBuzz(int n) { this.n = n; }

        /** Shared loop: wait until 'current' is mine (or we're done), act, advance, wake everyone. */
        private void runWhile(IntPredicate mine, IntConsumer action) throws InterruptedException {
            while (true) {
                synchronized (this) {
                    while (current <= n && !mine.test(current)) wait();
                    if (current > n) { notifyAll(); return; }     // done: wake others so they can exit too
                    action.accept(current);
                    current++;
                    notifyAll();                                  // the next number may belong to anyone
                }
            }
        }

        void fizz(Runnable printFizz) throws InterruptedException {
            runWhile(i -> i % 3 == 0 && i % 5 != 0, i -> printFizz.run());
        }
        void buzz(Runnable printBuzz) throws InterruptedException {
            runWhile(i -> i % 5 == 0 && i % 3 != 0, i -> printBuzz.run());
        }
        void fizzbuzz(Runnable printFizzBuzz) throws InterruptedException {
            runWhile(i -> i % 15 == 0, i -> printFizzBuzz.run());
        }
        void number(IntConsumer printNumber) throws InterruptedException {
            runWhile(i -> i % 3 != 0 && i % 5 != 0, printNumber);
        }
    }

    interface Body { void run() throws InterruptedException; }
    static Thread start(Body b) {
        Thread t = new Thread(() -> { try { b.run(); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); } });
        t.start();
        return t;
    }

    public static void main(String[] args) throws InterruptedException {
        int n = 15;
        List<String> out = new ArrayList<>();          // only touched inside the FizzBuzz monitor
        FizzBuzz fb = new FizzBuzz(n);
        List<Thread> ts = List.of(
                start(() -> fb.fizz(() -> out.add("fizz"))),
                start(() -> fb.buzz(() -> out.add("buzz"))),
                start(() -> fb.fizzbuzz(() -> out.add("fizzbuzz"))),
                start(() -> fb.number(i -> out.add(String.valueOf(i)))));
        for (Thread t : ts) t.join();                  // join() makes the list's contents visible to main
```

```
            System.out.println(String.join(", ", out));
        }
    }
```

Output:

```
1, 2, fizz, 4, buzz, fizz, 7, 8, fizz, buzz, 11, fizz, 13, 14, fizzbuzz
```

Why it's correct

- Every number satisfies **exactly one** predicate (they partition 1..n), so exactly one thread can act on each value.
- `current` is read and written only inside `synchronized (this)` → no races, full visibility.
- `notifyAll()` (not `notify()`): four different predicates wait on the same monitor; `notify()` could wake a thread whose predicate is false, which then waits again — and the right thread is never woken → hang.
- **Termination:** when `current` exceeds *n*, every waiting thread is woken (by the last `notifyAll`) and sees `current > n` in its loop condition, so all four exit.
- The output list is only modified inside the monitor, and `join()` makes it visible to `main`.

Pitfalls

- `notify()` instead of `notifyAll()` → lost wake-ups, random hangs.
- Forgetting the `current > n` exit check inside the wait loop → threads wait forever after the last number.
- Overlapping predicates (e.g. fizz = `i % 3 == 0` without excluding 15) → two threads compete for 15.

Real-world examples

"Workers wait until a shared state matches their role" — e.g. pipeline stages each waiting for a job to reach their stage in a shared state machine.

In the interview

> **30-second answer:** "One monitor, one shared counter. Each thread waits in a while loop until the counter matches its predicate or passes n, prints, increments and notifies all. The predicates partition the numbers, so exactly one thread acts per number."

Follow-up: *Semaphore version?* Possible: the number thread acts as dispatcher and releases the fizz/buzz/fizzbuzz semaphore for the next value — more efficient (no **thundering herd** — many threads woken at once when only one can proceed) but more code.

Self-check

- **Q:** Why `notifyAll`? **A:** Threads wait for *different* conditions on the same monitor; only waking all guarantees the right one wakes.

TL;DR

Shared counter + per-thread predicate + `while`-wait + `notifyAll`.

13.4 Building H2O (LeetCode 1117)

Problem

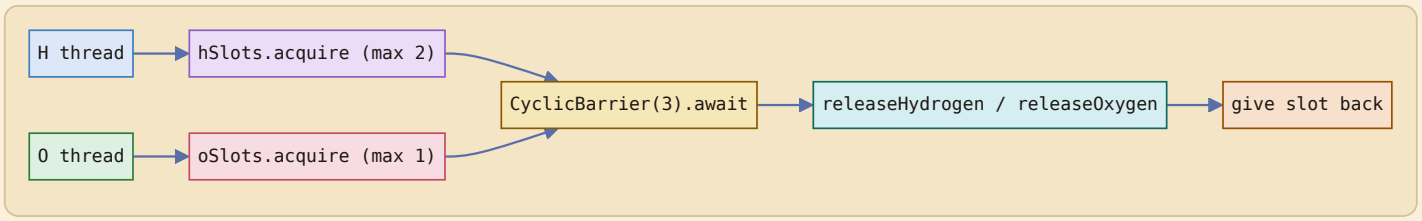
Hydrogen and oxygen threads arrive in any order. Each must pass a barrier in groups of three that form one water molecule: exactly **2 H + 1 O**. Threads from one molecule must all pass before any thread of the next molecule passes.

Approach

Combine two tools:

- **Slots** — `hSlots = Semaphore(2)`, `oSlots = Semaphore(1)`: at most 2 H and 1 O can be "in" the current molecule.

- **Group** — `CyclicBarrier(3)`: the three atoms wait for each other, then all go.
- A thread returns its slot permit **after** it has been released (printed), so the next molecule can only form once this one is finished.



Java solution

```java
// File: code/lld/ch13/h2o/Main.java
package lld.ch13.h2o;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.Semaphore;

/**
 * LeetCode 1117 - Building H2O.
 * Hydrogen and oxygen threads arrive in any order. They must leave in groups of three:
 * exactly 2 H + 1 O per group, and a group must fully leave before the next one starts.
 */
public class Main {

    static class H2O {
        private final Semaphore hSlots = new Semaphore(2);    // at most 2 H in the current molecule
        private final Semaphore oSlots = new Semaphore(1);    // at most 1 O in the current molecule
        private final CyclicBarrier bond = new CyclicBarrier(3);   // wait until all 3 atoms are present

        void hydrogen(Runnable releaseHydrogen) throws InterruptedException {
            hSlots.acquire();
            try {
                bond.await();
            } catch (BrokenBarrierException e) { throw new IllegalStateException(e); }
            releaseHydrogen.run();
            hSlots.release();                                 // free my slot only after I have "left"
        }
        void oxygen(Runnable releaseOxygen) throws InterruptedException {
            oSlots.acquire();
            try {
                bond.await();
            } catch (BrokenBarrierException e) { throw new IllegalStateException(e); }
            releaseOxygen.run();
            oSlots.release();
        }
    }

    public static void main(String[] args) throws InterruptedException {
        // 16 H + 8 O = 8 molecules, arriving in a scrambled (but reproducible) order.
        List<Character> atoms = new ArrayList<>();
        for (int i = 0; i < 8; i++) { atoms.add('H'); atoms.add('H'); atoms.add('O'); }
        java.util.Collections.shuffle(atoms, new java.util.Random(7));
        StringBuilder in = new StringBuilder();
        atoms.forEach(in::append);
        String input = in.toString();
        System.out.println("arrival order   : " + input);
        StringBuffer out = new StringBuffer();
        H2O h2o = new H2O();
        List<Thread> ts = new ArrayList<>();
        for (char c : input.toCharArray()) {
            Thread t = new Thread(() -> {
                try {
                    if (c == 'H') h2o.hydrogen(() -> out.append('H'));
                    else h2o.oxygen(() -> out.append('O'));
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            ts.add(t);
            t.start();
        }
        for (Thread t : ts) t.join();

        // Output order varies run to run (HHO, HOH, OHH are all fine), so we verify it instead.
        String s = out.toString();
        boolean valid = s.length() == input.length();
```

```
        for (int i = 0; valid && i < s.length(); i += 3) {
            String g = s.substring(i, i + 3);
            valid = g.chars().filter(ch -> ch == 'H').count() == 2 && g.indexOf('O') >= 0;
        }
        System.out.println("molecules formed: " + s.length() / 3 + ", every group of 3 has 2 H + 1 O: " + valid);
    }
}
```

Output:

```
arrival order   : OHOOOHHHHOHHHHHOOHHHHHOH
molecules formed: 8, every group of 3 has 2 H + 1 O: true
```

Why it's correct

- **At most 2 H and 1 O** can reach the barrier in each round because of the slot semaphores; the barrier needs 3 parties, so each trip is exactly `H, H, O`.
- **No mixing between molecules:** a new molecule needs all three slots free again. Each slot is freed only after its thread printed. So the next group can't trip the barrier (and print) until all three of the previous group have printed.
- The barrier resets automatically (it's *cyclic*) for the next molecule.
- The demo checks every consecutive group of 3 in the output instead of printing it, because `HHO`, `HOH` and `OHH` are all valid orders.

Pitfalls

- Releasing the slot **before** printing → a new H might print before the previous O, mixing molecules.
- Using only semaphores without the barrier → the 2 H and 1 O aren't forced to leave "together".
- If the thread counts aren't exactly 2:1, the last threads wait forever (the problem guarantees they are).

Real-world examples

Batching: gather exactly N items of certain types before dispatching (e.g. matchmaking 2 players + 1 server slot, assembling an order that needs parts from different producers).

In the interview

30-second answer: "Two semaphores cap the molecule at 2 H and 1 O; a CyclicBarrier of 3 makes the three wait for each other; each atom returns its slot only after it's released, so molecules never interleave."

Follow-up: *Without a barrier?* Use a lock with counters of waiting H and O; when 2 H + 1 O are available, the thread completing the set signals exactly those. More code, same idea.

Self-check

- **Q:** Why is `hSlots` initialised to 2, not 1? **A:** A molecule needs two hydrogens simultaneously inside the barrier.

TL;DR

Slots (semaphores) decide *who* can join; the barrier decides *when* they leave; free slots only after leaving.

13.5 Dining Philosophers (LeetCode 1226)

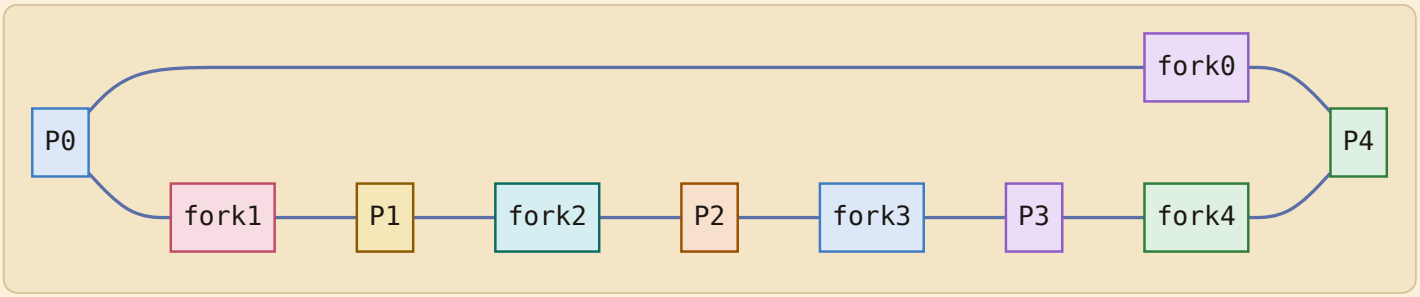
Problem

Five philosophers sit at a round table with one fork between each pair (5 forks). To eat, a philosopher needs **both** adjacent forks. If everyone picks up their left fork first, everyone may hold one fork and wait forever for the right one — **deadlock** (all four **Coffman conditions** hold — mutual exclusion, hold-and-wait, no preemption and circular wait; 12.17).

Approach

Break a Coffman condition:

1. **Resource ordering** (breaks circular wait): number the forks 0–4; always pick up the **lower-numbered** fork first. Philosopher 4 (forks 4 and 0) picks fork 0 first, so the cycle is impossible.
2. **Waiter / limited seats** (also breaks circular wait): a semaphore lets at most **n − 1** philosophers try at once — with 4 people and 5 forks, at least one can get both forks. Make the semaphore **fair** to prevent starvation.
3. (Other known solutions: `tryLock` both forks and put down on failure with random **back-off** (waiting a random short time before retrying); the Chandy–Misra algorithm for distributed settings.)



Java solution

```java
// File: code/lld/ch13/philosophers/Main.java
package lld.ch13.philosophers;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Semaphore;
import java.util.concurrent.atomic.AtomicIntegerArray;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Dining Philosophers (LeetCode 1226 variant): 5 philosophers, 5 forks, each needs both
 * neighbouring forks to eat. Naive "everyone picks up the left fork first" can deadlock.
 */
public class Main {

    interface Table { void eat(int p, Runnable eatAction) throws InterruptedException; }

    /** Fix 1 - resource ordering: always pick up the LOWER-numbered fork first (breaks circular wait). */
    static class OrderedForks implements Table {
        private final ReentrantLock[] forks;
        OrderedForks(int n) { forks = new ReentrantLock[n]; for (int i = 0; i < n; i++) forks[i] = new
ReentrantLock(); }

        public void eat(int p, Runnable eatAction) {
            int left = p, right = (p + 1) % forks.length;
            ReentrantLock first = forks[Math.min(left, right)], second = forks[Math.max(left, right)];
            first.lock();
            try {
                second.lock();
                try { eatAction.run(); } finally { second.unlock(); }
            } finally { first.unlock(); }
        }
    }

    /** Fix 2 - a "waiter" lets at most n-1 philosophers try at once (breaks circular wait too). */
    static class Waiter implements Table {
        private final ReentrantLock[] forks;
        private final Semaphore seats;
        Waiter(int n) {
            forks = new ReentrantLock[n];
            for (int i = 0; i < n; i++) forks[i] = new ReentrantLock();
            seats = new Semaphore(n - 1, true);                // fair: nobody starves waiting for a seat
        }
        public void eat(int p, Runnable eatAction) throws InterruptedException {
            seats.acquire();
            try {
                ReentrantLock left = forks[p], right = forks[(p + 1) % forks.length];
                left.lock();
                try {
                    right.lock();
                    try { eatAction.run(); } finally { right.unlock(); }
                } finally { left.unlock(); }
            } finally { seats.release(); }
        }
    }

    static String dine(Table table, int n, int meals) throws InterruptedException {
        AtomicIntegerArray eaten = new AtomicIntegerArray(n);
        List<Thread> ts = new ArrayList<>();
        for (int p = 0; p < n; p++) {
            final int id = p;
            Thread t = new Thread(() -> {
                try {
                    for (int m = 0; m < meals; m++) {
                        Thread.onSpinWait();                                // "think"
                        table.eat(id, () -> eaten.incrementAndGet(id));
```

```
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            ts.add(t); t.start();
        }
        for (Thread t : ts) t.join();
        return eaten.toString();
    }

    public static void main(String[] args) throws InterruptedException {
        System.out.println("resource ordering, meals per philosopher: " + dine(new OrderedForks(5), 5, 10_000));
        System.out.println("waiter (4 seats),  meals per philosopher: " + dine(new Waiter(5), 5, 10_000));
    }
}
```

Output:

```
resource ordering, meals per philosopher: [10000, 10000, 10000, 10000, 10000]
waiter (4 seats),  meals per philosopher: [10000, 10000, 10000, 10000, 10000]
```

Why it's correct

- **Ordering:** every philosopher acquires forks in increasing index order. A cycle of waiting would need someone to hold a higher fork while waiting for a lower one — impossible. No circular wait → no deadlock.
- **Waiter:** with at most 4 philosophers competing for 5 forks, by the **pigeonhole principle** (if there are more items than boxes, some box must get two) at least one of them can get both forks, eat, and release — so progress is always possible. The fair semaphore gives waiting philosophers seats in FIFO (first in, first out) order.
- Forks are `ReentrantLock`s released in `finally`, so an exception can't leave a fork held.
- 5 × 10,000 meals complete with each philosopher eating exactly 10,000 times.

Pitfalls

- "Everyone picks left then right" → deadlock.
- `tryLock` both forks and immediately retry without random back-off → livelock.
- Unfair strategies can starve one philosopher in theory (neighbours keep winning). The fair semaphore makes *seat* access first-come-first-served; for a strict no-starvation guarantee on the forks as well, make the fork locks fair too (`new ReentrantLock(true)`), at a throughput cost.

Real-world examples

Any time a task needs **two or more locks** at once: bank transfers (two accounts), moving an item between two inventories, database transactions updating several rows — solved by ordering (account id), or by the DB's deadlock detection aborting one transaction.

In the interview

> **30-second answer:** "Naively everyone takes the left fork and waits for the right — circular wait, deadlock. I break circular wait by always taking the lower-numbered fork first, or by a waiter semaphore allowing only n−1 philosophers to try; fairness prevents starvation."

Follow-ups:

- *Which Coffman condition does each fix break?* Both break circular wait; the tryLock variant breaks hold-and-wait.
- *Maximum parallelism?* With 5 forks, at most 2 philosophers can eat at the same time.

Self-check

- **Q:** With resource ordering, which philosopher behaves "differently"? **A:** The one between forks 4 and 0 — they pick fork 0 (right) first.

TL;DR

Break circular wait: global fork order, or allow only n−1 diners.

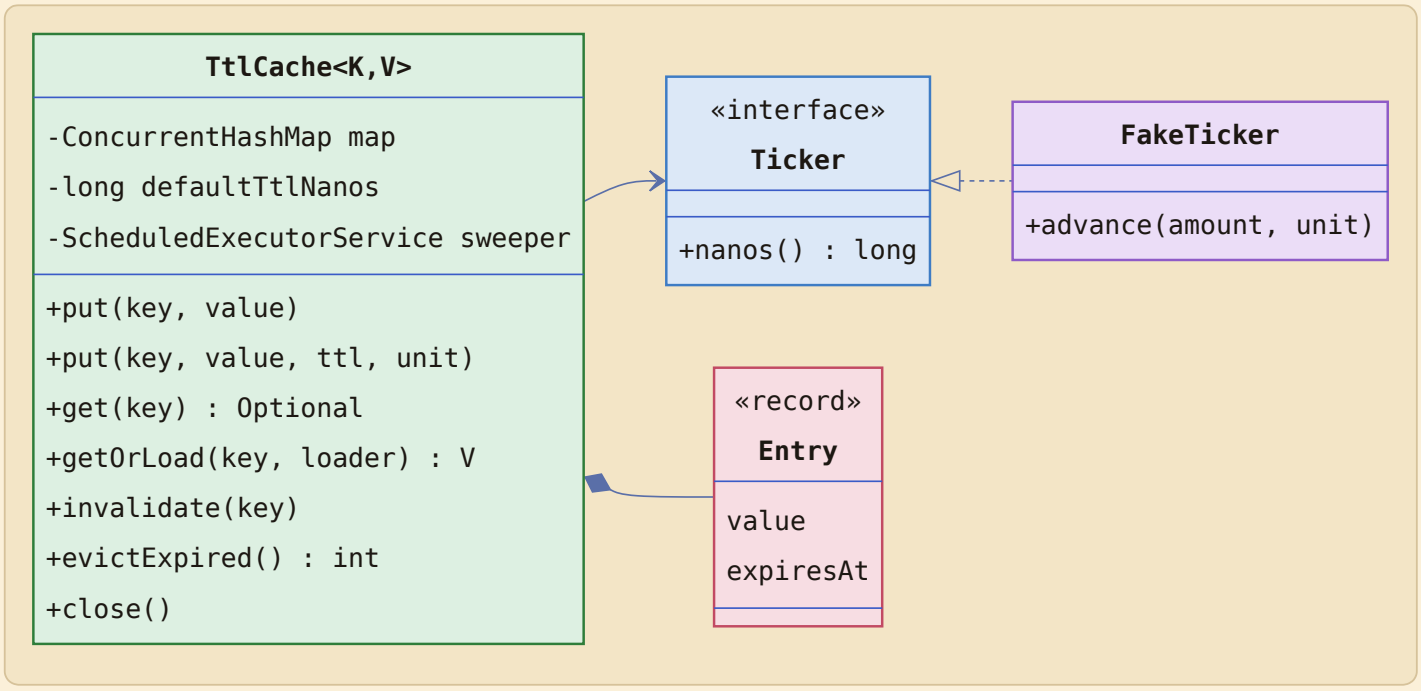
13.6 Thread-safe cache with TTL

Problem

Design an in-memory key-value **cache** (a fast store that keeps copies of data for quick reads; see System Design Fundamentals) used by many threads where each entry expires after a **time-to-live** (TTL). Requirements: `put(k, v)` (with default or per-entry TTL), `get(k)` returns nothing for expired entries, `getOrLoad(k, loader)` loads on a miss without a **cache stampede** (many threads loading the same key at once), expired entries are eventually removed from memory, and it's testable without sleeping.

Approach

- Store `Entry(value, expiresAt)` records (immutable) in a `ConcurrentHashMap`.
- **Lazy expiry**: `get` checks `expiresAt`; if expired, removes it with the **conditional** `remove(key, entry)` so it never deletes a fresher value another thread just put.
- **Active expiry**: a **daemon** (background thread that does not keep the JVM — Java Virtual Machine — running) `ScheduledExecutorService` (a JDK thread pool that runs tasks on a timer) periodically calls `evictExpired()` so keys nobody reads again don't leak memory.
- **No stampede**: `getOrLoad` uses `map.compute(...)`, which runs the function atomically per key; concurrent callers for the same key wait and the loader runs once.
- **Testability**: time comes from an injected `Ticker` (a `FakeTicker` in tests) — a small application of **Dependency Inversion** (depending on an interface rather than a concrete class, so the real clock can be swapped for a fake one).



Java solution

```java
// File: code/lld/ch13/ttlcache/Main.java
package lld.ch13.ttlcache;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.Function;

/** Thread-safe key-value cache where every entry expires after a time-to-live (TTL). */
public class Main {

    /** Time source. Injected so tests can move time forward without sleeping. */
    @FunctionalInterface interface Ticker { long nanos(); }

    static final class FakeTicker implements Ticker {
        private final AtomicLong now = new AtomicLong();
        public long nanos() { return now.get(); }
        void advance(long amount, TimeUnit unit) { now.addAndGet(unit.toNanos(amount)); }
    }

    static final class TtlCache<K, V> implements AutoCloseable {
        private record Entry<V>(V value, long expiresAt) { }

        private final ConcurrentHashMap<K, Entry<V>> map = new ConcurrentHashMap<>();
        private final long defaultTtlNanos;
        private final Ticker ticker;
        private final ScheduledExecutorService sweeper;       // null = lazy expiry only

        TtlCache(long ttl, TimeUnit unit, Ticker ticker, long sweepEveryMillis) {
            this.defaultTtlNanos = unit.toNanos(ttl);
            this.ticker = ticker;
            if (sweepEveryMillis > 0) {
                sweeper = Executors.newSingleThreadScheduledExecutor(r -> {
                    Thread t = new Thread(r, "ttl-sweeper");
                    t.setDaemon(true);                          // never blocks JVM shutdown
                    return t;
                });
                // 'this' escapes to the sweeper thread before the constructor ends (a pattern 12.6 warns
                // about). It is safe HERE only because every field evictExpired() uses (map, ticker) is
                // already assigned and handing a task to an executor is a happens-before edge. In
                // production code prefer a static factory that starts the sweeper after construction.
                sweeper.scheduleAtFixedRate(this::evictExpired, sweepEveryMillis, sweepEveryMillis,
TimeUnit.MILLISECONDS);
            } else {
                sweeper = null;
            }
        }

        private boolean expired(Entry<V> e) { return ticker.nanos() - e.expiresAt() >= 0; }   // overflow-safe
compare

        void put(K key, V value) { put(key, value, defaultTtlNanos, TimeUnit.NANOSECONDS); }

        void put(K key, V value, long ttl, TimeUnit unit) {
            Objects.requireNonNull(key); Objects.requireNonNull(value);
            map.put(key, new Entry<>(value, ticker.nanos() + unit.toNanos(ttl)));
        }
```

```
        Optional<V> get(K key) {
            Entry<V> e = map.get(key);
            if (e == null) return Optional.empty();
            if (expired(e)) {
                map.remove(key, e);   // conditional remove: never deletes a FRESH value another thread just put
                return Optional.empty();
            }
            return Optional.of(e.value());
        }

        /**
         * Read-through: if missing/expired, load it. compute() locks the key's bin, so concurrent callers
         * for the SAME key wait and the loader runs once (no "cache stampede").
         * The loader must be fast and must not touch this cache.
         */
        V getOrLoad(K key, Function<? super K, ? extends V> loader) {
            return map.compute(key, (k, old) ->
                    old != null && !expired(old) ? old : new Entry<>(loader.apply(k), ticker.nanos() +
defaultTtlNanos)
            ).value();
        }

        void invalidate(K key) { map.remove(key); }

        /** Active expiry: drop every expired entry. Safe to run concurrently with readers and writers. */
        int evictExpired() {
            int removed = 0;
            for (Map.Entry<K, Entry<V>> me : map.entrySet()) {
                if (expired(me.getValue()) && map.remove(me.getKey(), me.getValue())) removed++;
            }
            return removed;
        }

        int rawSize() { return map.size(); }   // includes not-yet-swept expired entries

        @Override public void close() { if (sweeper != null) sweeper.shutdownNow(); }
    }

    public static void main(String[] args) throws Exception {
        FakeTicker clock = new FakeTicker();
        try (TtlCache<String, String> cache = new TtlCache<>(100, TimeUnit.MILLISECONDS, clock, 0)) {
            cache.put("session:42", "alice");
            cache.put("otp:alice", "913377", 30, TimeUnit.MILLISECONDS);   // per-entry TTL
            clock.advance(50, TimeUnit.MILLISECONDS);
            System.out.println("t=50ms  session=" + cache.get("session:42").orElse("<expired>")
                    + ", otp=" + cache.get("otp:alice").orElse("<expired>"));
            clock.advance(60, TimeUnit.MILLISECONDS);
            System.out.println("t=110ms session=" + cache.get("session:42").orElse("<expired>"));

            for (int i = 0; i < 5; i++) cache.put("k" + i, "v" + i);
            clock.advance(200, TimeUnit.MILLISECONDS);
            System.out.println("before sweep raw size=" + cache.rawSize() + ", swept=" + cache.evictExpired()
                    + ", after=" + cache.rawSize());

            // Stampede test: 16 threads miss the same key at the same moment.
            AtomicInteger loads = new AtomicInteger();
            CountDownLatch go = new CountDownLatch(1);
            List<Thread> ts = new ArrayList<>();
            List<String> results = java.util.Collections.synchronizedList(new ArrayList<>());
            for (int i = 0; i < 16; i++) {
                Thread t = new Thread(() -> {
                    try { go.await(); } catch (InterruptedException e) { return; }
                    results.add(cache.getOrLoad("user:7", k -> { loads.incrementAndGet(); return "profile-of-" + k;
}));
                });
                ts.add(t); t.start();
            }
            go.countDown();
```

```
            for (Thread t : ts) t.join();
            System.out.println("16 concurrent misses -> loader ran " + loads.get() + " time(s), all got same value:
"
                    + results.stream().allMatch("profile-of-user:7"::equals));
        }

        // Background sweeper with the real clock.
        try (TtlCache<Integer, Integer> real = new TtlCache<>(30, TimeUnit.MILLISECONDS, System::nanoTime, 10)) {
            for (int i = 0; i < 1_000; i++) real.put(i, i * i);
            System.out.println("real clock: size right after puts=" + real.rawSize());
            Thread.sleep(200);
            System.out.println("real clock: size after 200 ms (sweeper ran)=" + real.rawSize());
        }
    }
}
```

Output:

```
t=50ms  session=alice, otp=<expired>
t=110ms session=<expired>
before sweep raw size=5, swept=5, after=0
16 concurrent misses -> loader ran 1 time(s), all got same value: true
real clock: size right after puts=1000
real clock: size after 200 ms (sweeper ran)=0
```

Why it's correct

- All shared state is inside one `ConcurrentHashMap`; entries are immutable records, so a reader sees either the old or the new entry, never a half-written one.
- `remove(key, e)` removes only if the mapping is still *that exact entry*. If another thread replaced it with a fresh one after our read, the fresh one survives.
- `compute` executes the remapping function atomically for that key, so 16 simultaneous misses cause one load (the output shows `loader ran 1 time(s)`).
- Expiry uses `now - expiresAt >= 0` rather than `now >= expiresAt`, which stays correct even if `System.nanoTime()` values **overflow** (wrap around after passing the largest value a `long` can hold) (the JDK recommends comparing nano times by subtraction).
- The sweeper is a **daemon** thread and is shut down in `close()`, so it never keeps the JVM alive.
- The constructor hands `this::evictExpired` to the sweeper before it returns (`this` escapes). That is normally a bug (12.6), and it is safe here only because the fields the sweeper uses are already set and submitting to an executor creates a happens-before edge; a **static factory method** (a static method that builds and returns the object) that starts the sweeper after construction is the cleaner production choice.

Pitfalls

- Using `System.currentTimeMillis()` for TTL: wall-clock time can jump (NTP adjustments — the Network Time Protocol corrects the system clock). `System.nanoTime()` is **monotonic** (it only ever moves forward) for measuring durations.
- A slow loader inside `compute` blocks other keys that share the same hash bin, and a loader that touches the same map can throw or deadlock. For slow loads, store a `CompletableFuture<V>` (a JDK placeholder for a value that an asynchronous task will fill in later) in the map instead (`computeIfAbsent(k, key -> supplyAsync(...))`) so the bin lock is held only briefly.
- Only lazy expiry → memory leak for keys never read again. Only active expiry → a `get` could return a stale value between sweeps. Use both.
- No size limit: a real cache also needs a maximum size and an **eviction policy** (the rule for which entry to throw out when full: LRU = least recently used, LFU = least frequently used).

Real-world examples

- **Caffeine** (the de-facto Java caching library) offers `expireAfterWrite`/`expireAfterAccess`, size-based eviction, and a loading cache that prevents stampedes — the production version of this design. Guava's (Google's core Java library) `CacheBuilder` is its predecessor.

- **Redis** `EXPIRE` / `SET … EX` uses the same combination: lazy expiry on access plus periodic active sampling of keys with TTLs.
- Sessions, OTPs (one-time passwords), DNS caches (DNS is the internet's lookup from names to IP addresses), and seat holds in booking systems (a hold is a TTL entry).

In the interview

> **30-second answer:** "A ConcurrentHashMap of immutable entries holding value and expiry time. Get checks expiry lazily and removes with a conditional remove; a daemon scheduled sweeper removes expired keys nobody reads. getOrLoad uses compute so concurrent misses load once. Time is injected for testing."

Follow-ups (with answers):

- *Add max size with LRU?* Combine with an access-ordered `LinkedHashMap` under a lock, or a `ConcurrentHashMap` plus a concurrent linked **deque** (double-ended queue) of keys updated on access — or just use Caffeine (Window-TinyLFU).
- *Refresh without blocking readers?* "Refresh-ahead": serve the old value while one thread reloads asynchronously when the entry is close to expiring.
- *Distributed?* Redis/Memcached (in-memory cache servers shared by many machines) with TTLs; keep a small local cache in front for hot keys.

Self-check

- **Q:** Why `remove(key, e)` and not `remove(key)` on expiry? **A:** To avoid deleting a new value written between our read and our remove.

TL;DR

CHM (`ConcurrentHashMap`) + immutable (value, expiry) entries; lazy + scheduled expiry; `compute` for **single-flight** loads (one load per key, shared by every waiting caller); injected clock.

13.7 Design a (simplified) Concurrent HashMap — lock striping

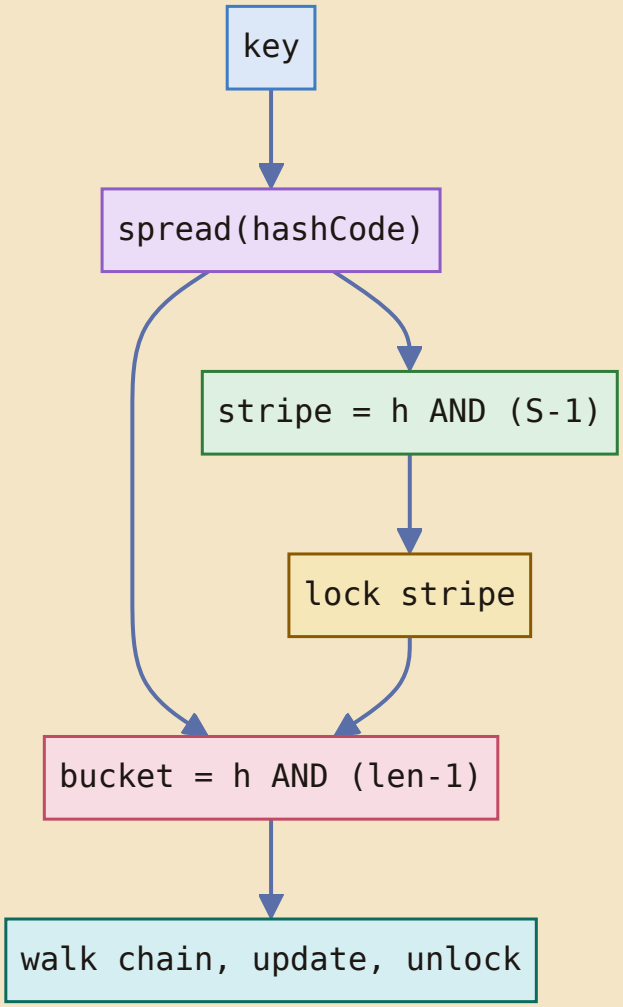
Problem

Implement a hash map that many threads can use at once with `get`, `put`, `remove`, `compute` and automatic resizing — faster than one global lock around a `HashMap`.

Approach

Lock striping (12.11):

- Buckets: an array of linked lists (**separate chaining**: each bucket holds a list of all entries whose hash lands there).
- Stripes: a fixed array of `S` locks (power of two). Key with hash *h* uses stripe `h & (S − 1)` and bucket `h & (tableLength − 1)`.
- Because the stripe depends only on the hash — **not** on the table length — a key's stripe never changes after a resize.
- Every operation locks only its stripe; threads on different stripes run in parallel.
- **Resize** (when size > 0.75 × buckets): acquire **all** stripe locks in index order (a global order → no deadlock), re-check the condition, **rehash** (recompute every key's bucket) into a table twice as big, publish via a `volatile` field, release in reverse.
- Size tracked with an `AtomicInteger`.


key
spread(hashCode)
stripe = h AND (S-1)
lock stripe
bucket = h AND (len-1)
walk chain, update, unlock

Java solution

```java
// File: code/lld/ch13/stripedmap/Main.java
package lld.ch13.stripedmap;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.ReentrantLock;
import java.util.function.BiFunction;

/**
 * A simplified concurrent hash map using LOCK STRIPING: the buckets are guarded by a small array
 * of locks, so threads touching different stripes never block each other.
 * (Java 7's ConcurrentHashMap used this idea with "segments".)
 */
public class Main {

    static final class StripedHashMap<K, V> {
        private static final class Node<K, V> {
            final K key; final int hash; V value; Node<K, V> next;
            Node(K key, int hash, V value, Node<K, V> next) { this.key = key; this.hash = hash; this.value = value;
this.next = next; }
        }

        private volatile Node<K, V>[] table;               // replaced (never mutated in place) only while ALL
locks are held
        private final ReentrantLock[] stripes;
        private final AtomicInteger size = new AtomicInteger();
        private static final float LOAD_FACTOR = 0.75f;

        @SuppressWarnings({"unchecked", "rawtypes"})
        StripedHashMap(int initialBuckets, int stripeCount) {
            if (Integer.bitCount(stripeCount) != 1 || Integer.bitCount(initialBuckets) != 1)
                throw new IllegalArgumentException("sizes must be powers of two");
            if (stripeCount > initialBuckets)                // guarantees: every bucket maps to exactly one
stripe
                throw new IllegalArgumentException("need stripes <= buckets");
            table = (Node<K, V>[]) new Node[initialBuckets];
            stripes = new ReentrantLock[stripeCount];
            for (int i = 0; i < stripeCount; i++) stripes[i] = new ReentrantLock();
        }

        private static int spread(Object key) {
            int h = key.hashCode();
            return (h ^ (h >>> 16)) & 0x7fffffff;             // mix high bits in, keep it non-negative
        }
        // A key's stripe depends only on its hash, NOT on the table size, so it never changes on resize.
        private ReentrantLock stripeFor(int hash) { return stripes[hash & (stripes.length - 1)]; }
        private static int indexFor(int hash, int len) { return hash & (len - 1); }

        V get(K key) {
            Objects.requireNonNull(key);
            int h = spread(key);
            ReentrantLock lock = stripeFor(h);
            lock.lock();
            try {
                Node<K, V>[] t = table;                           // stable while we hold a stripe lock
                for (Node<K, V> n = t[indexFor(h, t.length)]; n != null; n = n.next)
                    if (n.hash == h && n.key.equals(key)) return n.value;
                return null;
            } finally { lock.unlock(); }
        }

        /** Returns the previous value (or null). */
        V put(K key, V value) {
            Objects.requireNonNull(value);
```

```
        return doCompute(key, (k, old) -> value).oldValue();
    }

    V remove(K key) {
        Objects.requireNonNull(key);
        int h = spread(key);
        ReentrantLock lock = stripeFor(h);
        lock.lock();
        try {
            Node<K, V>[] t = table;
            int i = indexFor(h, t.length);
            for (Node<K, V> prev = null, n = t[i]; n != null; prev = n, n = n.next) {
                if (n.hash == h && n.key.equals(key)) {
                    if (prev == null) t[i] = n.next; else prev.next = n.next;
                    size.decrementAndGet();
                    return n.value;
                }
            }
            return null;
        } finally { lock.unlock(); }
    }

    /** Atomic read-modify-write for one key; returns the new value. Returning null from remap removes the key.
*/
    V compute(K key, BiFunction<? super K, ? super V, ? extends V> remap) {
        return doCompute(key, remap).newValue();
    }

    private record Result<V>(V oldValue, V newValue) { }

    private Result<V> doCompute(K key, BiFunction<? super K, ? super V, ? extends V> remap) {
        Objects.requireNonNull(key);
        int h = spread(key);
        ReentrantLock lock = stripeFor(h);
        boolean grew = false;
        V old, nv;
        lock.lock();
        try {
            Node<K, V>[] t = table;
            int i = indexFor(h, t.length);
            Node<K, V> prev = null, n = t[i];
            while (n != null && !(n.hash == h && n.key.equals(key))) { prev = n; n = n.next; }
            old = n == null ? null : n.value;
            nv = remap.apply(key, old);
            if (n != null && nv != null) {
                n.value = nv;                                   // update in place
            } else if (n != null) {                             // remap said "remove"
                if (prev == null) t[i] = n.next; else prev.next = n.next;
                size.decrementAndGet();
            } else if (nv != null) {                            // insert at bucket head
                t[i] = new Node<>(key, h, nv, t[i]);
                grew = size.incrementAndGet() > t.length * LOAD_FACTOR;
            }
        } finally { lock.unlock(); }
        if (grew) resize();                                     // never resize while holding one stripe
        return new Result<>(old, nv);
    }

    /** Stop-the-world resize: take every stripe lock IN ORDER (no deadlock), rehash, publish. */
    @SuppressWarnings({"unchecked", "rawtypes"})
    private void resize() {
        for (ReentrantLock l : stripes) l.lock();
        try {
            Node<K, V>[] old = table;
            if (size.get() <= old.length * LOAD_FACTOR) return;  // another thread already resized
            Node<K, V>[] nt = (Node<K, V>[]) new Node[old.length * 2];
            for (Node<K, V> head : old) {
                for (Node<K, V> n = head; n != null; ) {
                    Node<K, V> next = n.next;
```

```
                        int j = indexFor(n.hash, nt.length);
                        n.next = nt[j]; nt[j] = n;
                        n = next;
                    }
                }
                table = nt;
            } finally {
                for (int i = stripes.length - 1; i >= 0; i--) stripes[i].unlock();
            }
        }

        int size() { return size.get(); }
        int capacity() { return table.length; }
    }

    static void runThreads(int n, java.util.function.IntConsumer body) throws InterruptedException {
        List<Thread> ts = new ArrayList<>();
        for (int i = 0; i < n; i++) { final int id = i; Thread t = new Thread(() -> body.accept(id)); ts.add(t);
t.start(); }
        for (Thread t : ts) t.join();
    }

    public static void main(String[] args) throws InterruptedException {
        StripedHashMap<String, Integer> map = new StripedHashMap<>(16, 16);
        map.put("a", 1);
        System.out.println("put a=1, put a=2 returns old=" + map.put("a", 2) + ", get a=" + map.get("a")
                + ", remove a=" + map.remove("a") + ", get a=" + map.get("a"));

        final int THREADS = 8, PER = 20_000;
        runThreads(THREADS, id -> { for (int i = 0; i < PER; i++) map.put("t" + id + "-" + i, i); });
        int missing = 0;
        for (int id = 0; id < THREADS; id++) for (int i = 0; i < PER; i++) if
(!Integer.valueOf(i).equals(map.get("t" + id + "-" + i))) missing++;
        System.out.println("8 threads x 20,000 puts -> size=" + map.size() + ", missing=" + missing
                + ", table grew from 16 to " + map.capacity() + " buckets");

        StripedHashMap<String, Integer> counters = new StripedHashMap<>(16, 8);
        runThreads(THREADS, id -> {
            for (int i = 0; i < 10_000; i++) counters.compute("hot-" + (i % 10), (k, v) -> v == null ? 1 : v + 1);
        });
        int total = 0;
        for (int i = 0; i < 10; i++) total += counters.get("hot-" + i);
        System.out.println("atomic compute on 10 hot keys: each=" + counters.get("hot-0") + ", total=" + total + "
(expected 80000)");

        runThreads(THREADS, id -> { for (int i = 0; i < PER; i++) map.remove("t" + id + "-" + i); });
        System.out.println("after concurrent removes size=" + map.size());
    }
}
```

Output:

```
put a=1, put a=2 returns old=1, get a=2, remove a=2, get a=null
8 threads x 20,000 puts -> size=160000, missing=0, table grew from 16 to 262144 buckets
atomic compute on 10 hot keys: each=8000, total=80000 (expected 80000)
after concurrent removes size=0
```

Why it's correct

- **Every bucket belongs to exactly one stripe.** Both the stripe count S and the table length are powers of two and $S \le$ table length, so the stripe index (`h & (S-1)`) is just the low bits of the bucket index (`h & (len-1)`). All keys in one bucket therefore share one stripe lock, and two threads can never modify the same chain at the same time. Two operations on the same key obviously share it too.
- **Resize safety:** resize holds *every* stripe lock, so no other operation is in progress; it re-checks the load condition (another thread may have resized first) and publishes the new table with a `volatile` write. Any

later operation acquires a stripe lock that the resizer released → happens-before → it sees the new table and moved nodes.

- **No deadlock:** normal operations hold one lock; resize acquires all locks in ascending order and is started only *after* the caller released its own stripe.
- **Atomic compute:** read-modify-write of one key happens entirely under its stripe lock (the demo's 80,000 increments on 10 hot keys are exact).

Pitfalls

- Deriving the stripe from the *bucket index of the current table* — after a resize a key could move to a different lock while another thread holds the old one.
- Resizing while holding one stripe lock and then taking all locks → deadlock with another resizer. Release first, then resize.
- Reading `table` without a lock or `volatile` → a stale table after resize.
- Allowing `null` keys/values makes `get() == null` ambiguous (the JDK's `ConcurrentHashMap` forbids them too).

How the real `ConcurrentHashMap` goes further

Aspect	This design	JDK 8+ `ConcurrentHashMap`
Reads	Lock the stripe	Lock-free (volatile reads of nodes)
Write locking	One of S stripe locks	CAS into empty bins; `synchronized` on the bin's first node otherwise
Long chains	Linked list	Converted to a red-black tree beyond 8 nodes (when table ≥ 64)
Resize	Stop-the-world (all locks; everything else pauses)	Incremental: threads help transfer bins concurrently
Size	`AtomicInteger`	`LongAdder`-style counter cells

Real-world examples

Java 7's `ConcurrentHashMap` used **segments** (16 by default) — the same striping idea. Guava's `Striped<Lock>` offers striped locks for your own data; Caffeine uses striped buffers to record reads.

In the interview

> **30-second answer:** "An array of buckets plus a small array of locks; each key maps to a stripe by its hash bits, so different stripes work in parallel. Every bucket belongs to exactly one stripe. Resize takes all stripe locks in order, rehashes and publishes the new table through a volatile field."

Follow-ups:

- *How to make `get` lock-free?* Make `Node.value` and `next` volatile, publish nodes safely, and never mutate a chain in a way a reader could see half-done (the JDK copies/reorders carefully during transfer).
- *How many stripes?* Around the number of threads expected to write concurrently (JDK 7 default 16); more stripes = less contention but more memory.

Self-check

- **Q:** Why must stripe count divide the table length? **A:** So every bucket maps to exactly one stripe (stripe index = low bits of bucket index).

TL;DR

N locks for M buckets, stripe by hash bits, resize under all locks in order, publish via volatile.

13.8 Thread-safe bounded Blocking Queue (two ways)

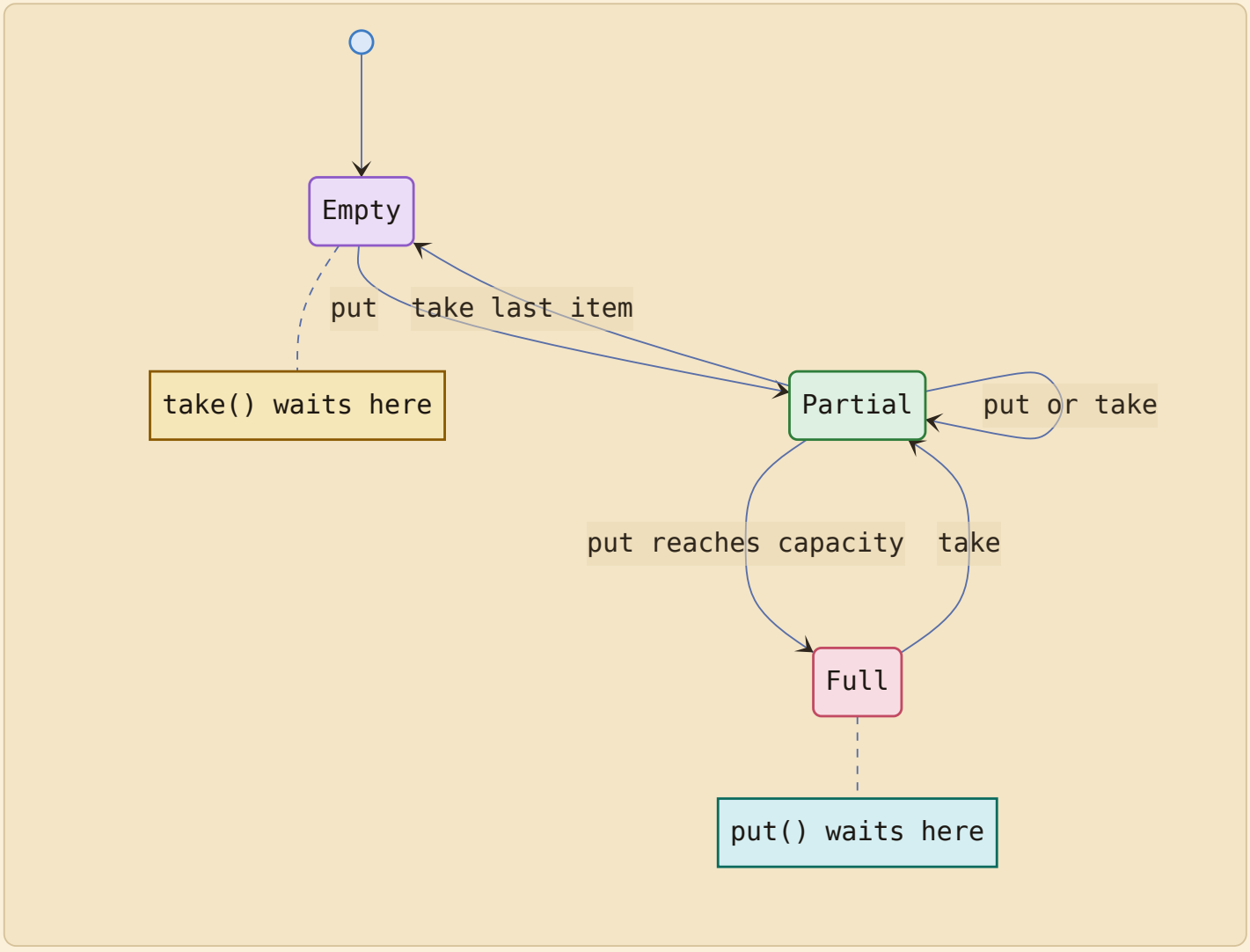
Problem

Implement a fixed-capacity FIFO queue for producer-consumer use: `put` blocks while full, `take` blocks while empty, plus `offer(item, timeout)` and `poll(timeout)` that give up after a timeout.

Approach

Storage: a **circular array** (an array whose end wraps around to the start; `head`, `tail`, `count`). Two synchronisation designs:

	Way 1: monitor	**Way 2: `ReentrantLock` + 2 `Condition`s**
Lock	`synchronized` methods	`lock.lockInterruptibly()`
Wait	`while (full) wait();`	`while (full) notFull.await();`
Wake	`notifyAll()` — producers and consumers share one wait set	`notEmpty.signal()` after enqueue, `notFull.signal()` after dequeue
Timed wait	`TimeUnit.timedWait(this, nanos)` + deadline loop	`awaitNanos(nanos)` returns the time left
Efficiency	Wakes everyone (thundering herd)	Wakes exactly one thread of the right kind



Java solution

```java
// File: code/lld/ch13/blockingqueue/Main.java
package lld.ch13.blockingqueue;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

/** A bounded blocking queue built two ways: monitor (wait/notifyAll) and Lock + two Conditions. */
public class Main {

    interface BoundedBlockingQueue<T> {
        void put(T item) throws InterruptedException;          // blocks while full
        T take() throws InterruptedException;                  // blocks while empty
        boolean offer(T item, long timeout, TimeUnit unit) throws InterruptedException; // false on timeout
        T poll(long timeout, TimeUnit unit) throws InterruptedException;                // null on timeout
        int size();
    }

    // ---------------------------------------------------------------------
    // Way 1: intrinsic lock. Producers AND consumers wait on the SAME monitor,
    // so we must use notifyAll() (notify() could wake the wrong kind of thread).
    // ---------------------------------------------------------------------
    static final class MonitorQueue<T> implements BoundedBlockingQueue<T> {
        private final Object[] items;
        private int head, tail, count;                          // all guarded by 'this'

        MonitorQueue(int capacity) { items = new Object[capacity]; }

        public synchronized void put(T item) throws InterruptedException {
            while (count == items.length) wait();
            enqueue(item);
            notifyAll();
        }
        public synchronized T take() throws InterruptedException {
            while (count == 0) wait();
            T x = dequeue();
            notifyAll();
            return x;
        }
        public synchronized boolean offer(T item, long timeout, TimeUnit unit) throws InterruptedException {
            long deadline = System.nanoTime() + unit.toNanos(timeout);
            while (count == items.length) {
                long left = deadline - System.nanoTime();
                if (left <= 0) return false;
                TimeUnit.NANOSECONDS.timedWait(this, left);      // wait(millis, nanos) under the hood
            }
            enqueue(item);
            notifyAll();
            return true;
        }
        public synchronized T poll(long timeout, TimeUnit unit) throws InterruptedException {
            long deadline = System.nanoTime() + unit.toNanos(timeout);
            while (count == 0) {
                long left = deadline - System.nanoTime();
                if (left <= 0) return null;
                TimeUnit.NANOSECONDS.timedWait(this, left);
            }
            T x = dequeue();
            notifyAll();
            return x;
        }
        public synchronized int size() { return count; }
```

```
        private void enqueue(T item) {
            if (item == null) throw new NullPointerException();  // null is our "timed out" signal
            items[tail] = item;
            tail = (tail + 1) % items.length;                    // circular buffer
            count++;
        }
        @SuppressWarnings("unchecked")
        private T dequeue() {
            T x = (T) items[head];
            items[head] = null;                                  // let GC reclaim it
            head = (head + 1) % items.length;
            count--;
            return x;
        }
    }

    // ---------------------------------------------------------------------
    // Way 2: ReentrantLock with TWO condition queues. Producers wait on notFull,
    // consumers on notEmpty, so a targeted signal() is enough.
    // (This is how java.util.concurrent.ArrayBlockingQueue is built.)
    // ---------------------------------------------------------------------
    static final class ConditionQueue<T> implements BoundedBlockingQueue<T> {
        private final Object[] items;
        private int head, tail, count;
        private final ReentrantLock lock = new ReentrantLock();
        private final Condition notFull = lock.newCondition();
        private final Condition notEmpty = lock.newCondition();

        ConditionQueue(int capacity) { items = new Object[capacity]; }

        public void put(T item) throws InterruptedException {
            if (item == null) throw new NullPointerException();
            lock.lockInterruptibly();
            try {
                while (count == items.length) notFull.await();
                enqueue(item);
            } finally { lock.unlock(); }
        }
        public T take() throws InterruptedException {
            lock.lockInterruptibly();
            try {
                while (count == 0) notEmpty.await();
                return dequeue();
            } finally { lock.unlock(); }
        }
        public boolean offer(T item, long timeout, TimeUnit unit) throws InterruptedException {
            if (item == null) throw new NullPointerException();
            long nanos = unit.toNanos(timeout);
            lock.lockInterruptibly();
            try {
                while (count == items.length) {
                    if (nanos <= 0) return false;
                    nanos = notFull.awaitNanos(nanos);           // returns the time still left
                }
                enqueue(item);
                return true;
            } finally { lock.unlock(); }
        }
        public T poll(long timeout, TimeUnit unit) throws InterruptedException {
            long nanos = unit.toNanos(timeout);
            lock.lockInterruptibly();
            try {
                while (count == 0) {
                    if (nanos <= 0) return null;
                    nanos = notEmpty.awaitNanos(nanos);
                }
                return dequeue();
            } finally { lock.unlock(); }
        }
```

```
        public int size() { lock.lock(); try { return count; } finally { lock.unlock(); } }

        private void enqueue(T item) {
            items[tail] = item;
            tail = (tail + 1) % items.length;
            count++;
            notEmpty.signal();                                  // wake ONE waiting consumer
        }
        @SuppressWarnings("unchecked")
        private T dequeue() {
            T x = (T) items[head];
            items[head] = null;
            head = (head + 1) % items.length;
            count--;
            notFull.signal();                                   // wake ONE waiting producer
            return x;
        }
    }

    // ---------------------------------------------------------------------
    static void stressTest(String name, BoundedBlockingQueue<Integer> q) throws InterruptedException {
        final int PRODUCERS = 3, CONSUMERS = 3, PER_PRODUCER = 20_000;
        AtomicLong consumedSum = new AtomicLong(), consumedCount = new AtomicLong();
        List<Thread> ts = new ArrayList<>();
        for (int p = 0; p < PRODUCERS; p++) {
            Thread t = new Thread(() -> {
                try { for (int i = 1; i <= PER_PRODUCER; i++) q.put(i); }
                catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            ts.add(t);
        }
        int perConsumer = PRODUCERS * PER_PRODUCER / CONSUMERS;
        for (int c = 0; c < CONSUMERS; c++) {
            Thread t = new Thread(() -> {
                try { for (int i = 0; i < perConsumer; i++) { consumedSum.addAndGet(q.take());
consumedCount.incrementAndGet(); } }
                catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            ts.add(t);
        }
        ts.forEach(Thread::start);
        for (Thread t : ts) t.join();
        long expected = (long) PRODUCERS * PER_PRODUCER * (PER_PRODUCER + 1) / 2;
        System.out.println(name + ": consumed " + consumedCount.get() + " items, sum ok=" + (consumedSum.get() ==
expected)
                + ", left in queue=" + q.size());

        System.out.println(name + ": poll(30ms) on empty -> " + q.poll(30, TimeUnit.MILLISECONDS));
        q.put(1); q.put(2);                                     // capacity is 2 in this demo
        System.out.println(name + ": offer(30ms) on full -> " + q.offer(3, 30, TimeUnit.MILLISECONDS));
    }

    public static void main(String[] args) throws InterruptedException {
        stressTest("wait/notifyAll  ", new MonitorQueue<>(2));
        stressTest("Lock+Conditions ", new ConditionQueue<>(2));
    }
}
```

Output:

```
wait/notifyAll  : consumed 60000 items, sum ok=true, left in queue=0
wait/notifyAll  : poll(30ms) on empty -> null
wait/notifyAll  : offer(30ms) on full -> false
Lock+Conditions : consumed 60000 items, sum ok=true, left in queue=0
Lock+Conditions : poll(30ms) on empty -> null
Lock+Conditions : offer(30ms) on full -> false
```

Why it's correct

- All fields are accessed only while holding the single lock → no races; the lock gives visibility.
- **Waiting in `while` loops** handles spurious wake-ups and "someone else took the item first".
- **Way 1** must use `notifyAll()`: with one wait set, `notify()` might wake a producer when a consumer was needed (or vice versa), and the right thread might never wake → deadlock-like hang.
- **Way 2** can use `signal()` because producers only wait on `notFull` and consumers only on `notEmpty`; every enqueue makes exactly one more consumer able to proceed, every dequeue exactly one producer.
- Timed versions track the *remaining* time so repeated spurious wakeups can't extend the total wait.
- Stress test: 3 producers × 20,000 items, 3 consumers, capacity 2 (maximum blocking) → all 60,000 items consumed, sum correct, queue empty.

Pitfalls

- `if` instead of `while`.
- `notify()` with a single monitor shared by producers and consumers.
- Forgetting `items[head] = null` after dequeue → the array keeps references alive (memory leak for large objects).
- Accepting `null` items when `null` means "timed out" in `poll`.
- Using `lock()` instead of `lockInterruptibly()` makes cancellation harder (the JDK's queues use `lockInterruptibly` in blocking methods).

Real-world examples

`java.util.concurrent.ArrayBlockingQueue` is built exactly like Way 2 (one lock, `notEmpty`/`notFull`). `LinkedBlockingQueue` uses **two** locks (`putLock`, `takeLock`) so a producer and a consumer can work at the same time. Thread pools, logging frameworks, and Kafka's producer internals all rely on blocking hand-off.

In the interview

> **30-second answer:** "A circular array with head, tail and count guarded by one lock. put waits while count == capacity, take waits while count == 0, both in while loops. With synchronized I must use notifyAll because both sides share one wait set; with ReentrantLock I use two Conditions, notFull and notEmpty, and signal just the right side."

Follow-ups:

- *Higher throughput?* Two-lock queue (linked list, separate put/take locks, atomic count) like `LinkedBlockingQueue`, or a lock-free ring buffer (LMAX Disruptor, a high-performance Java messaging library) for single producer/consumer.
- *Unbounded?* Remove the "full" condition — but then there's no back-pressure.

Self-check

- **Q:** Why is `signal()` safe in Way 2 but `notify()` unsafe in Way 1? **A:** Separate condition queues guarantee the woken thread is of the right kind.

TL;DR

Circular buffer + one lock + "wait while full/empty" in loops. Monitor → `notifyAll`; Lock → `notFull`/`notEmpty` + `signal`.

13.9 Concurrent Bloom Filter

Problem

Design a thread-safe **Bloom filter** — a compact probabilistic set that answers *"definitely not present"* or *"probably present"* — that many threads can add to and query at the same time.

Approach

How a Bloom filter works (first-time explanation):

1. Keep an array of m bits, all 0.
2. To **add** an item, compute k hash positions and set those k bits to 1.
3. To **query**, compute the same k positions: if **any** bit is 0 the item was **definitely never added**; if all are 1 it was **probably** added (the bits might have been set by other items → a *false positive*).
4. No false negatives, no deletion (clearing a bit could erase other items; a *counting* Bloom filter with small counters supports deletes).

Sizing for n expected items and target false-positive rate p:

- bits: $m = \lceil -n \cdot \ln p / (\ln 2)^2 \rceil$
- hashes: $k = \text{round}((m / n) \cdot \ln 2)$

For n = 80,000 and p = 1% → $m \approx$ 766,805 bits (≈ 93.6 KiB — kibibytes of 1,024 bytes; the demo prints 93 because it uses integer division) and k = 7 — about 9.6 bits per item, whatever the item size.

Double hashing (Kirsch–Mitzenmacher): compute two hashes $h1$, $h2$ once and use positions $h1 + i \cdot h2$ for i = 0..k−1 — as good as k independent hashes in practice.

Thread safety without locks:

- Bits live in an `AtomicLongArray` (64 bits per word).
- Adding a bit = **atomic OR** via `getAndAccumulate(word, mask, (a, b) -> a | b)` (a CAS loop inside). Two threads setting different bits in the same word both succeed.
- Queries are plain volatile reads of words. Bits only ever go 0 → 1, so a concurrent query can at worst miss an item whose `add` hasn't finished — which is fine, because that add hadn't happened yet. After `add` returns, every later query sees its bits (happens-before of atomic operations).

Java solution

```java
// File: code/lld/ch13/bloomfilter/Main.java
package lld.ch13.bloomfilter;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLongArray;

/**
 * A thread-safe Bloom filter: a compact set that answers "definitely NOT present" or
 * "PROBABLY present". No false negatives; a tunable rate of false positives.
 * Lock-free: bits live in an AtomicLongArray and are set with an atomic OR.
 */
public class Main {

    static final class ConcurrentBloomFilter {
        private final AtomicLongArray words;     // m bits packed into 64-bit words
        private final int numBits;               // m
        private final int numHashes;             // k

        /** Size the filter for 'expectedItems' at the target false-positive probability. */
        ConcurrentBloomFilter(int expectedItems, double falsePositiveRate) {
            double ln2 = Math.log(2);
            long m = (long) Math.ceil(-expectedItems * Math.log(falsePositiveRate) / (ln2 * ln2));
            if (m > Integer.MAX_VALUE - 63) throw new IllegalArgumentException("too large");
            this.numBits = (int) m;
            this.numHashes = Math.max(1, (int) Math.round((double) m / expectedItems * ln2));
            this.words = new AtomicLongArray((numBits + 63) / 64);
        }

        /** Adds the item. Returns true if at least one bit changed (i.e. it was definitely new). */
        boolean add(String item) {
            long base = fnv1a64(item);
            long h1 = fmix64(base), h2 = fmix64(base ^ 0x9E3779B97F4A7C15L) | 1;   // h2 odd
            boolean changed = false;
            for (int i = 0; i < numHashes; i++) {
                int bit = (int) Math.floorMod(h1 + i * h2, (long) numBits);        // double hashing
                long mask = 1L << (bit & 63);
                long before = words.getAndAccumulate(bit >>> 6, mask, (cur, m) -> cur | m);  // atomic OR (CAS
loop)
                if ((before & mask) == 0) changed = true;
            }
            return changed;
        }

        boolean mightContain(String item) {
            long base = fnv1a64(item);
            long h1 = fmix64(base), h2 = fmix64(base ^ 0x9E3779B97F4A7C15L) | 1;
            for (int i = 0; i < numHashes; i++) {
                int bit = (int) Math.floorMod(h1 + i * h2, (long) numBits);
                if ((words.get(bit >>> 6) & (1L << (bit & 63))) == 0) return false;   // definitely absent
            }
            return true;
        }

        int numBits() { return numBits; }
        int numHashes() { return numHashes; }

        // 64-bit FNV-1a over the string's chars, then a strong bit-mixer (MurmurHash3's finalizer).
        private static long fnv1a64(String s) {
            long h = 0xcbf29ce484222325L;
            for (int i = 0; i < s.length(); i++) { h ^= s.charAt(i); h *= 0x100000001b3L; }
            return h;
        }
        private static long fmix64(long k) {
            k ^= k >>> 33; k *= 0xff51afd7ed558ccdL;
            k ^= k >>> 33; k *= 0xc4ceb9fe1a85ec53L;
```

```
            k ^= k >>> 33;
            return k;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        final int THREADS = 8, PER_THREAD = 10_000, N = THREADS * PER_THREAD;
        ConcurrentBloomFilter bf = new ConcurrentBloomFilter(N, 0.01);
        System.out.println("sized for n=" + N + ", p=1%: m=" + bf.numBits() + " bits (" + bf.numBits() / 8 / 1024
                + " KiB), k=" + bf.numHashes() + " hashes");

        List<Thread> ts = new ArrayList<>();
        for (int t = 0; t < THREADS; t++) {
            final int id = t;
            Thread th = new Thread(() -> { for (int i = 0; i < PER_THREAD; i++) bf.add("user-" + id + "-" + i); });
            ts.add(th); th.start();
        }
        for (Thread th : ts) th.join();

        int falseNegatives = 0;
        for (int t = 0; t < THREADS; t++) for (int i = 0; i < PER_THREAD; i++) if (!bf.mightContain("user-" + t +
"-" + i)) falseNegatives++;
        System.out.println("false negatives after 8-thread insert: " + falseNegatives);

        int probes = 100_000, falsePositives = 0;
        for (int i = 0; i < probes; i++) if (bf.mightContain("stranger-" + i)) falsePositives++;
        System.out.printf("false-positive rate on %,d unseen keys: %.2f%% (target 1%%)%n", probes, 100.0 *
falsePositives / probes);
        System.out.println("add(\"user-0-0\") again changes bits? " + bf.add("user-0-0"));
    }
}
```

Output:

```
sized for n=80000, p=1%: m=766805 bits (93 KiB), k=7 hashes
false negatives after 8-thread insert: 0
false-positive rate on 100,000 unseen keys: 1.01% (target 1%)
add("user-0-0") again changes bits? false
```

Why it's correct

- **No lost updates:** a plain `words[i] |= mask` on a `long[]` is read-modify-write and two threads could overwrite each other's bits → false negatives. The atomic OR retries on conflict, so every bit set survives.
- **Monotonic state:** bits never go back to 0, so there's no ABA issue and no need for locks.
- **Deterministic result:** OR is commutative, so the final bit array is the same whatever the thread interleaving — the demo's false-positive rate (≈1%) is identical every run.
- The measured false-positive rate matches the formula's target.

Pitfalls

- Non-atomic bit setting → false negatives (the one thing a Bloom filter must never produce).
- Poor hash functions (e.g. using `String.hashCode()` directly with `i * 31` offsets) → clustered bits and much higher false-positive rates. Mix bits (here FNV-1a + MurmurHash3's finalizer — two fast, non-cryptographic hash functions).
- Adding far more than n items → false-positive rate climbs quickly; resize by creating a larger filter (or a "scalable" chain of filters).
- `Math.floorMod` on `long` to avoid negative indices.

Real-world examples

- **Apache Cassandra, HBase, RocksDB, LevelDB** (databases and storage engines): Bloom filters per **SSTable** (Sorted String Table — an immutable sorted data file on disk)/file to skip disk reads for keys that aren't there.
- **Google Guava** `BloomFilter` (its `put`/`mightContain` are thread-safe since Guava 23 using atomic long arrays).
- **Redis** (RedisBloom module / Redis Stack) offers `BF.ADD`/`BF.EXISTS`.

- Checking whether a username is already taken, or a URL already crawled, before hitting the database; Medium has described using Bloom filters to avoid recommending already-read articles.

In the interview

> **30-second answer:** "A bit array of size m with k hash positions per item from double hashing; add sets k bits, query checks them — any zero means definitely absent. For concurrency I store bits in an AtomicLongArray and set them with an atomic OR; since bits only go from 0 to 1, reads need no lock. m and k come from n and the target false-positive rate."

Follow-ups:

- *Deletions?* Counting Bloom filter (small counters instead of bits, atomic increments/decrements) or a Cuckoo filter (a Bloom-filter alternative that supports deletion).
- *Distributed?* Keep it in Redis (RedisBloom) or rebuild per node from a shared source.

Self-check

- **Q:** Can a Bloom filter say "absent" for an item that was added? **A:** No — no false negatives (assuming atomic, completed adds).
- **Q:** Bits per element for 1% FPR (false-positive rate)? **A:** About 9.6.

TL;DR

Bits + k hashes; "no" is certain, "yes" is probable. `AtomicLongArray` + atomic OR = lock-free and correct.

13.10 Multi-threaded Merge Sort (Fork/Join)

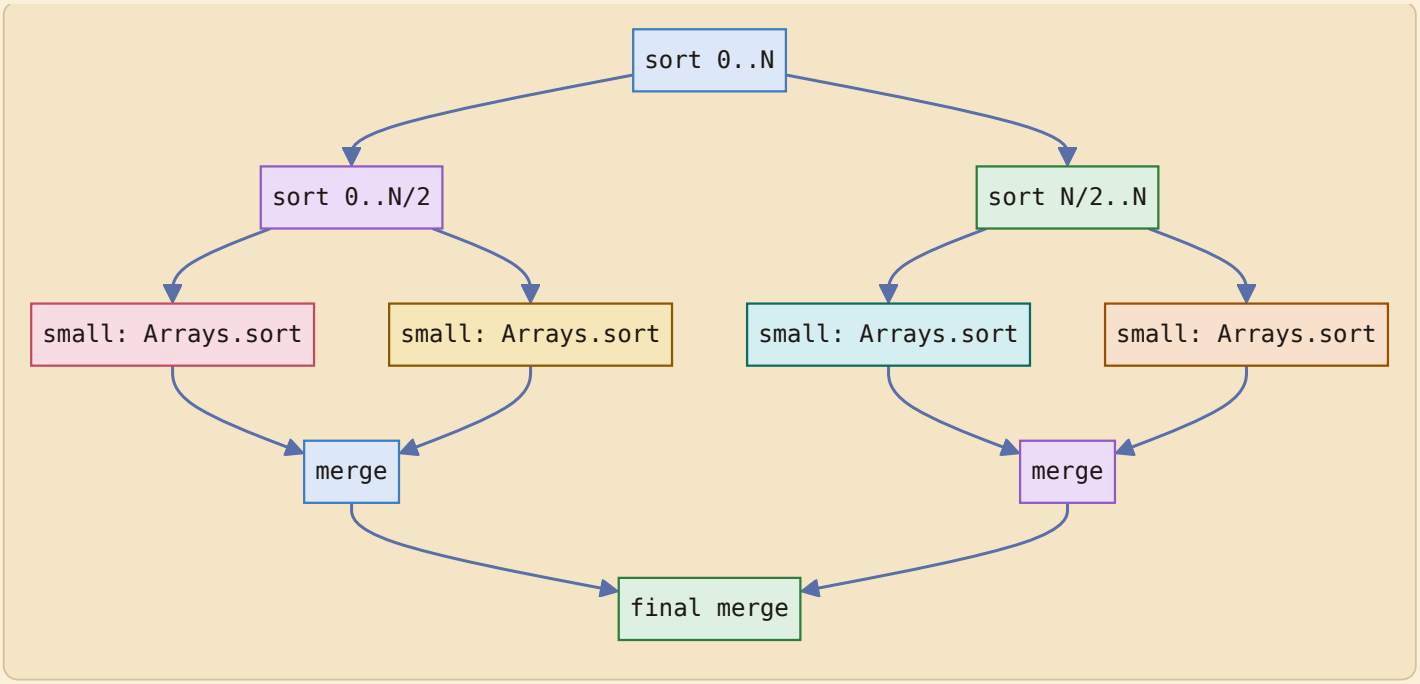
Problem

Sort a large `int[]` using all CPU cores.

Approach

Merge sort is naturally **divide and conquer**: sort the left half and right half *independently* (in parallel), then merge. Java's **Fork/Join framework** (`ForkJoinPool`, `RecursiveAction`/`RecursiveTask`) is built for this:

1. If the range is small (≤ 8,192 elements) → sort it sequentially (`Arrays.sort`); splitting further costs more than it saves.
2. Otherwise split at `mid = (lo + hi) >>> 1` and `invokeAll(left, right)` — forks both halves and waits for both.
3. Merge the two sorted halves using a **shared scratch buffer**; each task only uses its own `[lo, hi)` slice, so there's no sharing between concurrent tasks.
4. **Work stealing:** each pool thread has its own deque of tasks; idle threads steal tasks from the other end of busy threads' deques, keeping all cores busy.


sort 0..N
sort 0..N/2
sort N/2..N
small: Arrays.sort
small: Arrays.sort
small: Arrays.sort
small: Arrays.sort
merge
merge
final merge

Java solution

```java
// File: code/lld/ch13/mergesort/Main.java
package lld.ch13.mergesort;

import java.util.Arrays;
import java.util.Random;
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveAction;

/** Multi-threaded merge sort with the Fork/Join framework (divide and conquer + work stealing). */
public class Main {

    static final class ParallelMergeSort extends RecursiveAction {
        private static final long serialVersionUID = 1L;
        static final int THRESHOLD = 1 << 13;   // below ~8K elements, splitting costs more than it saves
        private final int[] a, buf;             // buf = one shared scratch array; tasks use disjoint ranges
        private final int lo, hi;               // sorts a[lo, hi)

        ParallelMergeSort(int[] a, int[] buf, int lo, int hi) { this.a = a; this.buf = buf; this.lo = lo; this.hi =
hi; }

        @Override protected void compute() {
            if (hi - lo <= THRESHOLD) {
                Arrays.sort(a, lo, hi);                         // sequential base case
                return;
            }
            int mid = (lo + hi) >>> 1;                          // overflow-safe midpoint
            invokeAll(new ParallelMergeSort(a, buf, lo, mid),   // fork both halves, wait for both
                      new ParallelMergeSort(a, buf, mid, hi));
            merge(mid);
        }

        private void merge(int mid) {
            if (a[mid - 1] <= a[mid]) return;                   // already in order: skip
            System.arraycopy(a, lo, buf, lo, hi - lo);
            int i = lo, j = mid, k = lo;
            while (i < mid && j < hi) a[k++] = buf[i] <= buf[j] ? buf[i++] : buf[j++];   // <= keeps it stable
            while (i < mid) a[k++] = buf[i++];
            // right-half leftovers are already in place
        }
    }

    static void parallelSort(int[] a) {
        ForkJoinPool.commonPool().invoke(new ParallelMergeSort(a, new int[a.length], 0, a.length));
    }

    public static void main(String[] args) {
        int[] small = {38, 27, 43, 3, 9, 82, 10, 3};
        parallelSort(small);
        System.out.println("small: " + Arrays.toString(small));

        int n = 4_000_000;
        int[] data = new Random(42).ints(n).toArray();
        int[] expected = data.clone();
        Arrays.sort(expected);

        int[] mine = data.clone();
        parallelSort(mine);
        System.out.println(n + " random ints: matches Arrays.sort = " + Arrays.equals(mine, expected));

        int[] jdk = data.clone();
        Arrays.parallelSort(jdk);                               // the JDK's own parallel sort
        System.out.println("Arrays.parallelSort agrees        = " + Arrays.equals(jdk, expected));
    }
}
```

Output:

```
small: [3, 3, 9, 10, 27, 38, 43, 82]
4000000 random ints: matches Arrays.sort = true
Arrays.parallelSort agrees        = true
```

Why it's correct

- The two subtasks work on **disjoint** ranges of both `a` and `buf` → no data race between them.
- `invokeAll` returns only when both subtasks completed; Fork/Join's `join` establishes happens-before, so the parent's merge sees the children's writes.
- The merge takes from the left run on ties (`<=`), so it's **stable** (equal elements keep their original order; matters for objects, not for ints).
- The early exit `a[mid-1] <= a[mid]` skips merging already-ordered halves.
- The result is compared with `Arrays.sort` on 4 million random ints (seeded, so reproducible).

Pitfalls

- No threshold → millions of tiny tasks; overhead dominates.
- Allocating a new buffer in every merge → heavy GC (garbage collection — the JVM's automatic freeing of unused memory); allocate once.
- Calling `fork()` on both halves and `join()`ing in the wrong order, or doing blocking I/O inside Fork/Join tasks (it's meant for CPU work).
- `(lo + hi) / 2` can overflow for huge arrays; `>>> 1` avoids it.
- Expecting speed-up on tiny arrays or on a single core (Amdahl's law: the part that must run sequentially caps the total speed-up — here the final merge).

Real-world examples

- `Arrays.parallelSort` (Java 8+) splits the array, sorts the pieces in parallel on the common Fork/Join pool and merges them (the demo cross-checks against it).
- Parallel streams (`list.parallelStream()`) run on `ForkJoinPool.commonPool()`.
- Big-data sorts (Hadoop MapReduce's — a model that maps over data in parallel, then combines the results — sort/shuffle, Spark's `sortBy`) apply the same divide-sort-merge idea across machines.

In the interview

30-second answer: "A RecursiveAction splits the range in half until it's below a threshold, sorts small pieces sequentially, then merges using one preallocated buffer. invokeAll runs both halves in parallel on the ForkJoinPool, whose work stealing keeps cores busy. Subtasks touch disjoint ranges, so no locks are needed."

Follow-ups:

- *`RecursiveAction` vs `RecursiveTask`?* `RecursiveTask<V>` returns a value (e.g. parallel sum); `RecursiveAction` returns nothing (in-place sort).
- *Can the merge be parallel too?* Yes — split the merge by binary-searching the median of one run in the other (parallel merge), which is how fully parallel merge sorts scale further.

Self-check

- **Q:** Why is a shared `buf` safe? **A:** Concurrent tasks use non-overlapping index ranges of it.

TL;DR

Split until small, sort sequentially, merge; Fork/Join + work stealing; disjoint ranges = no locks.

13.11 Rate limiter — token bucket (thread-safe)

Problem

Limit how often clients may call an **API** (Application Programming Interface — here, a web service that clients send requests to): allow short **bursts** up to *B* requests, and a sustained rate of *R* requests per second. Must be correct under many concurrent requests, per client (user id / API key — a secret string that identifies the caller / IP address).

Approach

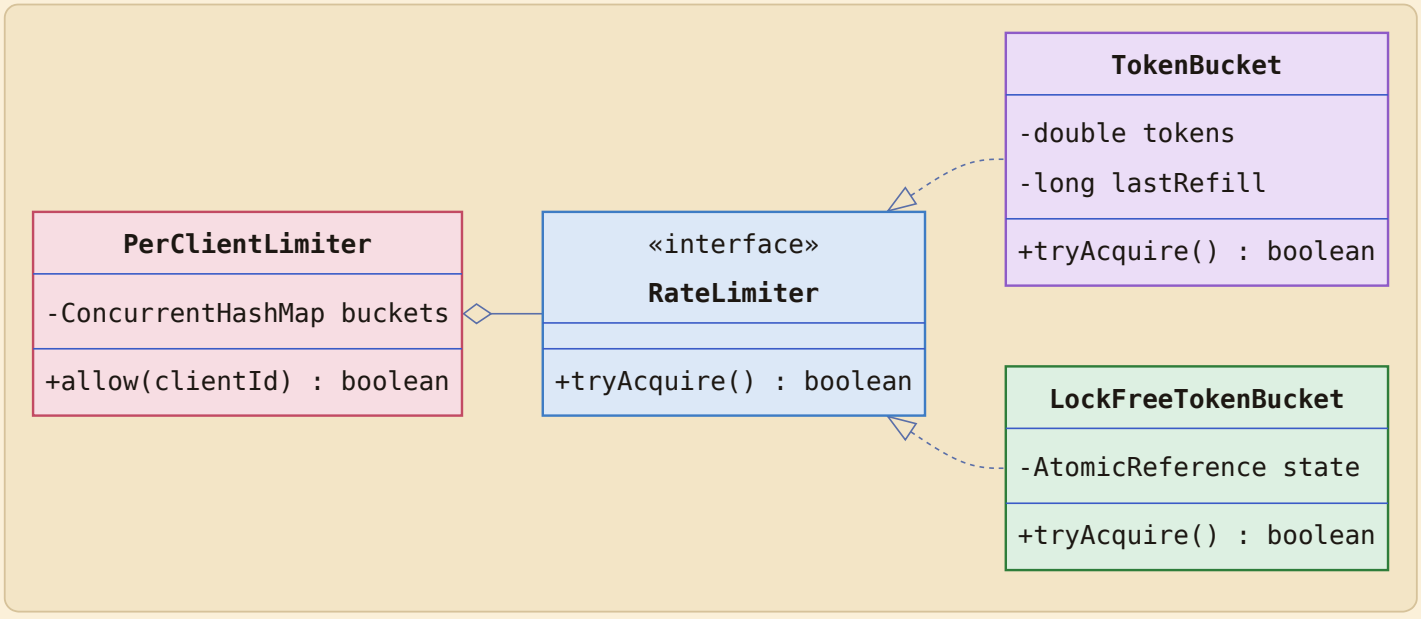
Token bucket (first-time explanation):

1. A bucket holds up to *capacity* tokens (the allowed burst) and starts full.
2. Tokens drip in at *refillRate* per second, never exceeding capacity.
3. Each request takes one token. No token → reject (HTTP 429 "Too Many Requests"; HTTP is the web's request/response protocol — see System Design Fundamentals).
4. Implement refill **lazily**: store `tokens` and `lastRefill`; on each request add `elapsed × rate` tokens. No background thread needed.

Thread safety: "refill + check + take" must be **one atomic step** — otherwise two threads can both see 1 token and both take it.

- Version 1: `synchronized tryAcquire()` — simple and correct.
- Version 2: **lock-free** — put `(tokens, lastRefill)` into one immutable `State` record inside an `AtomicReference` and CAS the whole state (the "several fields → one immutable object → one CAS" trick from 12.15).
- Per client: `ConcurrentHashMap.computeIfAbsent(clientId, …)` creates exactly one bucket per client.
- Time comes from an injected `Ticker` so the demo is deterministic.

Algorithm	Bursts	Memory	Notes
Token bucket	Allows up to capacity	2 numbers	Most common for APIs
Leaky bucket	Smooths to constant output rate	Queue	Good for shaping traffic (evening out bursts)
Fixed window counter	Allows 2× burst at window edges	1 counter	Simplest
Sliding window log	Exact	One timestamp per request	Memory-heavy
Sliding window counter	Approximate, smooth	2 counters	Good compromise



Java solution

```java
// File: code/lld/ch13/ratelimiter/Main.java
package lld.ch13.ratelimiter;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.atomic.AtomicReference;
import java.util.function.Supplier;

/** Thread-safe token-bucket rate limiter: a lock-based version, a lock-free version, and per-user limits. */
public class Main {

    @FunctionalInterface interface Ticker { long nanos(); }

    static final class FakeTicker implements Ticker {
        private final AtomicLong now = new AtomicLong();
        public long nanos() { return now.get(); }
        void advance(long amount, TimeUnit unit) { now.addAndGet(unit.toNanos(amount)); }
    }

    interface RateLimiter { boolean tryAcquire(); }

    /**
     * Bucket holds up to 'capacity' tokens (the allowed burst). Tokens drip in at 'refillPerSecond'.
     * Each request takes one token; no token -> rejected. Refill is computed lazily from elapsed time.
     */
    static final class TokenBucket implements RateLimiter {
        private final double capacity, tokensPerNano;
        private final Ticker ticker;
        private double tokens;          // guarded by 'this'
        private long lastRefill;        // guarded by 'this'

        TokenBucket(int capacity, double refillPerSecond, Ticker ticker) {
            this.capacity = capacity;
            this.tokensPerNano = refillPerSecond / 1e9;
            this.ticker = ticker;
            this.tokens = capacity;     // start full
            this.lastRefill = ticker.nanos();
        }

        public synchronized boolean tryAcquire() {
            long now = ticker.nanos();
            tokens = Math.min(capacity, tokens + (now - lastRefill) * tokensPerNano);   // refill + acquire are ONE
atomic step
            lastRefill = now;
            if (tokens >= 1) { tokens -= 1; return true; }
            return false;
        }
    }

    /** Same algorithm, lock-free: the whole state is one immutable record swapped with CAS. */
    static final class LockFreeTokenBucket implements RateLimiter {
        private record State(double tokens, long lastRefill) { }
        private final double capacity, tokensPerNano;
        private final Ticker ticker;
        private final AtomicReference<State> state;

        LockFreeTokenBucket(int capacity, double refillPerSecond, Ticker ticker) {
            this.capacity = capacity;
            this.tokensPerNano = refillPerSecond / 1e9;
            this.ticker = ticker;
            this.state = new AtomicReference<>(new State(capacity, ticker.nanos()));
```

```
        }

        public boolean tryAcquire() {
            while (true) {
                State cur = state.get();
                long now = Math.max(ticker.nanos(), cur.lastRefill());   // never move time backwards
                double refilled = Math.min(capacity, cur.tokens() + (now - cur.lastRefill()) * tokensPerNano);
                boolean ok = refilled >= 1;
                State next = new State(ok ? refilled - 1 : refilled, now);
                if (state.compareAndSet(cur, next)) return ok;            // lost the race? recompute and retry
            }
        }
    }

    /** One bucket per client key (user id, API key, IP). computeIfAbsent creates each bucket exactly once. */
    static final class PerClientLimiter {
        private final ConcurrentHashMap<String, RateLimiter> buckets = new ConcurrentHashMap<>();
        private final Supplier<RateLimiter> factory;
        PerClientLimiter(Supplier<RateLimiter> factory) { this.factory = factory; }
        boolean allow(String clientId) { return buckets.computeIfAbsent(clientId, k -> factory.get()).tryAcquire();
}
    }

    static String burst(RateLimiter rl, int requests) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < requests; i++) sb.append(rl.tryAcquire() ? 'Y' : 'n');
        return sb.toString();
    }

    static int concurrentBurst(RateLimiter rl, int threads) throws InterruptedException {
        AtomicInteger allowed = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        List<Thread> ts = new ArrayList<>();
        for (int i = 0; i < threads; i++) {
            Thread t = new Thread(() -> {
                try { go.await(); } catch (InterruptedException e) { return; }
                if (rl.tryAcquire()) allowed.incrementAndGet();
            });
            ts.add(t); t.start();
        }
        go.countDown();
        for (Thread t : ts) t.join();
        return allowed.get();
    }

    public static void main(String[] args) throws InterruptedException {
        for (String kind : List.of("synchronized", "lock-free")) {
            FakeTicker clock = new FakeTicker();
            RateLimiter rl = kind.equals("synchronized") ? new TokenBucket(5, 2, clock) : new
LockFreeTokenBucket(5, 2, clock);
            System.out.println("[" + kind + "] capacity 5, refill 2/s");
            System.out.println("  t=0.0s  7 requests : " + burst(rl, 7));
            clock.advance(1, TimeUnit.SECONDS);
            System.out.println("  t=1.0s  3 requests : " + burst(rl, 3));
            clock.advance(250, TimeUnit.MILLISECONDS);
            System.out.println("  t=1.25s 1 request  : " + burst(rl, 1) + "  (only 0.5 token so far)");
            clock.advance(250, TimeUnit.MILLISECONDS);
            System.out.println("  t=1.5s  1 request  : " + burst(rl, 1));
            clock.advance(1, TimeUnit.HOURS);
            System.out.println("  after 1 idle hour  : allowed " + concurrentBurst(rl, 100) + " of 100 concurrent
requests (burst capped at 5)");
        }

        FakeTicker clock = new FakeTicker();
        PerClientLimiter api = new PerClientLimiter(() -> new TokenBucket(3, 1, clock));
        StringBuilder alice = new StringBuilder(), bob = new StringBuilder();
        for (int i = 0; i < 4; i++) { alice.append(api.allow("alice") ? 'Y' : 'n'); }
        for (int i = 0; i < 2; i++) { bob.append(api.allow("bob") ? 'Y' : 'n'); }
        System.out.println("per-client (3 burst): alice " + alice + ", bob " + bob + "  (alice's spam doesn't hurt
```

```
bob)");
    }
}
```

Output:

```
[synchronized] capacity 5, refill 2/s
  t=0.0s  7 requests : YYYYYnn
  t=1.0s  3 requests : YYn
  t=1.25s 1 request  : n  (only 0.5 token so far)
  t=1.5s  1 request  : Y
  after 1 idle hour  : allowed 5 of 100 concurrent requests (burst capped at 5)
[lock-free] capacity 5, refill 2/s
  t=0.0s  7 requests : YYYYYnn
  t=1.0s  3 requests : YYn
  t=1.25s 1 request  : n  (only 0.5 token so far)
  t=1.5s  1 request  : Y
  after 1 idle hour  : allowed 5 of 100 concurrent requests (burst capped at 5)
per-client (3 burst): alice YYYn, bob YY  (alice's spam doesn't hurt bob)
```

Why it's correct

- **Atomic decision:** in `TokenBucket`, refill, check and decrement happen inside one `synchronized` method. In `LockFreeTokenBucket`, the new state is computed from one snapshot and installed with `compareAndSet`; if another thread changed the state first, the CAS fails and we recompute — so each token is granted at most once.
- **Burst cap:** tokens are clamped to `capacity`, so after an idle hour only 5 of 100 simultaneous requests pass (as the output shows for both versions).
- **Rate:** at 2 tokens/s, after 1 s exactly 2 new tokens exist; at +0.25 s only 0.5 token → rejected; at +0.5 s → 1 token → allowed.
- **Per-client isolation:** `computeIfAbsent` is atomic, so two concurrent first requests from the same client can't create two buckets (which would double their limit).
- The lock-free version clamps `now` with `Math.max(..., lastRefill)` so a thread that read the clock slightly earlier can't move time backwards.

Pitfalls

- Separate `refill()` and `take()` calls → race; two threads spend the same token.
- A background thread per bucket to add tokens → thousands of threads/timers; lazy refill needs none.
- Using `currentTimeMillis()` (wall clock can jump) instead of `nanoTime()`.
- An unbounded map of buckets → memory leak for one-off clients; evict idle buckets (e.g. with the TTL cache from 13.6).
- In a multi-server deployment an in-memory limiter only limits per server. For a global limit, keep the bucket in **Redis** (an in-memory data store shared by all servers) and update it atomically with a Lua script (a small script that Redis runs as one uninterruptible step) (or use a sliding-window counter in Redis).

Real-world examples

- **Stripe** has described using token buckets (backed by Redis) for its API rate limits.
- **Amazon API Gateway** (AWS's managed front door for APIs) **throttling** (deliberately rejecting or delaying excess requests) is configured with a *rate* and a *burst* — token-bucket parameters.
- **Guava** `RateLimiter`, **Bucket4j** (token bucket for Java, with Redis/JCache — the standard Java caching API — back-ends), and **Resilience4j** `RateLimiter` are production Java libraries; NGINX's (a popular web server and reverse proxy) `limit_req` uses a leaky-bucket approach.

In the interview

30-second answer: "Token bucket: capacity for bursts, refill rate for the average. I refill lazily from elapsed nanoTime on each request, and make refill-check-take one atomic step — synchronized, or lock-free by CASing an immutable state record. A ConcurrentHashMap gives one bucket per client via computeIfAbsent; across servers I'd move the bucket into Redis with a Lua script."

Follow-ups (with answers):

- *How do you tell the client when to retry?* Compute `(1 − tokens) / rate` seconds and return it in a `Retry-After` header (a response header that tells the client how many seconds to wait).
- *Different limits per plan (free vs paid)?* The factory creates buckets with capacity/rate from the client's plan (Strategy pattern).
- *Blocking instead of rejecting?* Compute the wait time and sleep/park until a token will exist (Guava's `acquire()` does this).

Self-check

- **Q:** Capacity 5, rate 2/s, bucket empty at t=0. How many requests pass at t=3s? **A:** min(5, 3 × 2) = 5.
- **Q:** Why one immutable `State` in the lock-free version? **A:** `tokens` and `lastRefill` must change together; one CAS on one reference updates both atomically.

TL;DR

Bucket of tokens: capacity = burst, rate = average. Lazy refill; refill+take atomic (lock or CAS on immutable state); one bucket per client; Redis for multi-server.

13.12 Chapter summary

Problem	Key primitive	The one invariant to state in the interview
FooBar	2 semaphores	Exactly one permit exists across both
Zero Even Odd	3 semaphores	Zero dispatches; one permit circulates
FizzBuzz	monitor + `notifyAll`	Predicates partition 1..n
H2O	2 semaphores + barrier	≤2 H and ≤1 O per round; slot freed after release
Dining Philosophers	ordered locks / n−1 semaphore	No circular wait
TTL cache	CHM + `compute` + conditional remove	Never return or keep expired data; load once per key
Striped map	lock array	Each bucket belongs to exactly one stripe
Blocking queue	lock + 2 conditions	0 ≤ count ≤ capacity; wait in loops
Bloom filter	atomic OR	Bits only go 0 → 1; no false negatives
Merge sort	Fork/Join	Subtasks touch disjoint ranges
Rate limiter	lock or CAS on immutable state	Refill + take is one atomic step

13.13 Chapter self-check

1. **Q:** In FooBar, why doesn't it matter which thread starts first? **A:** The one without a permit blocks until the other releases it.
2. **Q:** In FizzBuzz, what bug appears with `notify()`? **A:** The wrong thread may be woken and go back to sleep; the right one never wakes → hang.
3. **Q:** In H2O, why release the slot after printing? **A:** So the next molecule cannot form (and print) until this one has fully left.
4. **Q:** Two ways to prevent Dining Philosophers deadlock? **A:** Always take the lower-numbered fork first; or allow at most n−1 philosophers to try at once.
5. **Q:** Why does the TTL cache use `remove(key, entry)`? **A:** To avoid deleting a fresh value written concurrently.
6. **Q:** Why must the striped map's stripe depend only on the hash? **A:** So a key's lock doesn't change when the table resizes.
7. **Q:** When is `signal()` safe instead of `signalAll()`? **A:** When each condition queue holds only threads waiting for the same thing and one state change enables exactly one waiter.
8. **Q:** What would a non-atomic OR do to a Bloom filter? **A:** Lose bits → false negatives.
9. **Q:** Why a threshold in parallel merge sort? **A:** Small tasks cost more to schedule than to sort.

10. **Q:** How do you make a token bucket global across 10 servers? **A:** Store it in Redis and update it atomically with a Lua script.

13.14 Chapter TL;DR

- **Ordering puzzles** (FooBar, ZeroEvenOdd, H2O): semaphores as batons, barriers for groups; keep "exactly one permit" style invariants.
- **Predicate waiting** (FizzBuzz, blocking queue): lock + `while (!condition) wait/await` + `notifyAll` or targeted `signal`.
- **Multi-lock problems** (philosophers): break circular wait with ordering or limiting.
- **Thread-safe data structures** (cache, map, queue, Bloom filter, rate limiter): immutable entries, atomic compound operations, striping, CAS on immutable state, and injected clocks for testable time.
- **Parallel algorithms** (merge sort): Fork/Join with thresholds and disjoint data.

Chapter 14 — Case Studies: Everyday Objects

> **Where this fits.** Chapters 1–13 gave you the tools: OOP (object-oriented programming: building software from objects that bundle data with the methods that act on it), UML (Unified Modeling Language, the standard way to draw classes and their links), SOLID (five principles for designing classes), clean code, the 23 GoF patterns (the "Gang of Four" design patterns, named after the four authors of the 1994 book *Design Patterns*) and concurrency (several pieces of work running at the same time). From this chapter to Chapter 21 you use those tools on real interview questions. This chapter starts with **physical machines you have used yourself**: a parking lot, a vending machine, a coffee machine, an ATM, an elevator, a traffic light and a parcel locker. You already know how they behave, so all your attention can go to the design.

What you will learn in this chapter

#	Problem	The one idea the interviewer is really testing	Main patterns
14.1	Parking Lot	**Resource allocation** under concurrency: two cars must never get the same spot	Strategy (spot finding, pricing), Factory, Observer (display boards)
14.2	Vending Machine	A **state machine**: the order in which actions are allowed	State
14.3	Coffee Vending Machine	**Atomic, all-or-nothing** deduction from a shared inventory; recipes as data	Observer (low-stock alerts), data-driven recipes, sealed result types
14.4	ATM	**Security + money correctness**: authenticate, never lose money, never double-spend	State, Chain of Responsibility (cash dispenser), Command (transactions)
14.5	Elevator System	**Scheduling** (SCAN/LOOK: sweep orders for visiting stops, see 14.5) and **dispatching** many cars	Strategy (dispatch), State-like modes
14.6	Traffic Signal Control	**Timed state machine** with a hard safety invariant and emergency override	State, Strategy (timing), Observer (safety monitor)
14.7	Shipping Locker System	**Best-fit allocation + deadlines + a secret**: reserve a door atomically, expire holds and uncollected parcels, single-use hashed pickup codes	Strategy (door choice), Observer (SMS), enum state machines, sealed results

> **Key terms — the pattern names in this table.** These are design patterns (proven, named shapes for arranging classes) from the earlier chapters. **Strategy**: a swappable rule (how to price, which car to send) hidden behind an interface. **Factory**: one place that creates objects, so callers need not know how they are built. **Observer**: an object keeps a list of listeners and tells each of them when something changes. **State**: each situation a machine can be in is its own class. **Chain of Responsibility**: a request is passed along a line of handlers, each doing what it can. **Command**: an action wrapped in an object, so it can be run, logged or queued. **Sealed result types**: a Java type whose possible variants form a fixed, known list (for example "served" or "refused"), so the compiler can check that callers handle each one.

What these problems test

These seven questions look different, but interviewers use them to check three skills:

1. **State machines.** A machine behaves differently depending on what has happened so far. A vending machine must not dispense before it is paid. An ATM must not allow a withdrawal before the PIN is checked. A traffic light must never go from green straight to green on the crossing road. A **state machine** is a model with a fixed set of situations (states) and rules for moving between them (transitions). The **State pattern** (Chapter 11) turns each state into its own class, so every button has a defined answer in every state.
2. **Resource allocation.** A limited set of things (parking spots, notes in a cassette, millilitres of milk, elevator cars, locker doors) must be handed out correctly: never twice, never below zero, and fairly.
3. **Concurrency.** Many actors act at the same moment: several entrance gates, two ATMs on one joint account, ten customers tapping "Latte" at once, a timer thread and an emergency thread (a **thread** is one independent line

of execution inside a program; many threads can run at once and touch the same objects). The design must stay correct when calls overlap (Chapters 12–13).

> **Key insight.** In every one of these problems, the hardest part is small: one method that picks a spot, one method that makes change, one method that deducts ingredients, one method that picks the next elevator stop. Find that method early, name it out loud, and spend your time there.

How each case study is laid out

Every problem follows the same template, which is also a good order to speak in during an interview:

Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns → Java implementation → Concurrency & edge cases → Extensions & follow-ups → In the interview → Self-check → TL;DR.

How to read the course diagrams

The course draws its class diagrams in stages. Classes added in earlier stages are drawn in grey; the new ones are in colour. The legend below explains every arrow used in those diagrams (it is the same for all four course case studies in this chapter).

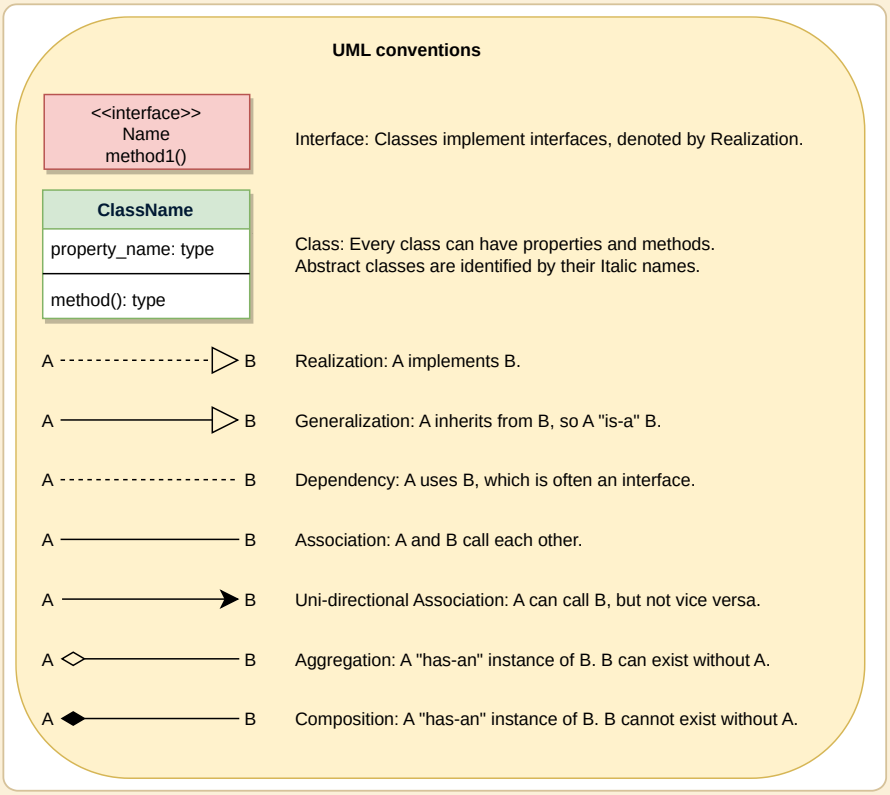


Figure — Legend for the course class diagrams: interface, class, realization, association, inheritance, composition and the other arrow kinds (course diagram)

Read the figure above once: a **hollow triangle** means inheritance ("is a"), a **dashed line with a hollow triangle** means "implements an interface" (an **interface** is a list of methods that a class promises to provide, without saying how), a **filled diamond** means composition (the part dies with the whole), and a **plain arrow** means association (one object refers to another without owning it). Chapter 3 covers these relationships in depth.

How to run every example in this chapter

```
cd chapters/14-case-everyday-objects/code
javac --release 21 -d /tmp/lld_14 $(find . -name '*.java')
java -cp /tmp/lld_14 lld.ch14.parkinglot.Main      # or vendingmachine, coffeemachine, atm, elevator, trafficsignal, locker
```

Here `javac` is the Java compiler (it turns `.java` source files into `.class` files of bytecode), `--release 21` targets Java 21, `-d` says where to put the compiled classes, and `java -cp` runs a class using the given

classpath (the folder where the JVM, the Java Virtual Machine that runs bytecode, looks for classes).

All seven `Main` classes compile with Java 21 (standard library only) and run. The output printed under each implementation is the real output of that program.

> **Code-style note.** To keep each design readable in one place, a file sometimes holds several small package-private types (types declared without `public`, visible only inside their own package) (for example `Vehicles.java` holds `SpotType`, `VehicleType` and `Vehicle`). That is legal Java, and the code compiles with no warnings under `-Xlint:all` (the compiler switch that turns on every optional warning, or *lint* check) except the `auxiliaryclass` lint, which exists precisely to flag this style (a type used from outside the file it is declared in). In production you would usually give each top-level type its own file.

14.1 Design a Parking Lot

Figure — A large multi-level car park, the kind of building this design models (course diagram)

A **parking lot** (car park) is an area set aside for parking vehicles. Malls, stadiums, airports and office towers have large multi-floor lots with gates, ticket machines and boards showing free spaces. The photo above is the kind of lot we are modelling: many floors, many spot sizes, and several ways in and out.

This is the most-asked LLD question (low-level design: designing the classes, their responsibilities and how they talk to each other) of all. It looks easy, which is exactly why it is useful to an interviewer: they can watch how you **scope** it, how you **model the fit rule** (which vehicle may use which spot), and whether you notice that **two gates can hand out the same spot** at the same moment.

Problem & clarifying questions

Ask these before drawing anything. The answers in the right column are the ones this design assumes.

Question	Assumed answer
One lot or a chain of lots?	One lot, many floors.
How many entrances and exits?	Several of each, all working at the same time.
Which vehicle types?	Car, truck, van, motorcycle, electric car.
Which spot types?	Compact, large, handicapped (accessible), motorcycle, electric.
How do customers pay?	Cash or card, at the exit panel, to an attendant, or earlier at an info portal on each floor.
How is the fee computed?	Per hour, with tiers: $4 for the first hour, $3.50 for hours 2 and 3, $2.50 for each hour after. A started hour counts as a full hour.
Do EVs charge?	Yes: electric spots have a panel where the driver pays for charging.
Reservations, monthly passes, number-plate recognition?	Out of scope (mentioned as extensions).
What happens when the lot is full?	The entrance panel and the ground-floor board show a "full" message; no more vehicles are admitted.

Requirements

Functional

1. The lot has **multiple floors**.
2. It has **multiple entry and exit points**.
3. A customer **collects a ticket at the entrance** and **pays at the exit**.
4. Payment at the **automated exit panel** or to a **parking attendant**.
5. Payment by **cash or credit card**.
6. Payment is also possible at a **customer info portal on each floor**; a ticket paid there is not charged again at the exit.
7. The lot **never admits more vehicles than its capacity**. When full, a message shows on the entrance panel and on the ground-floor display board.
8. Each floor has many spots of several **types**: compact, large, handicapped, motorcycle (and electric, below).
9. Some spots are for **electric cars**; each has an **electric panel** where the customer can pay for and charge the vehicle.
10. Many **vehicle types**: car, truck, van, motorcycle (and electric car).
11. Each floor has a **display board** showing free spots per spot type.
12. A **per-hour fee model** with tiers ($4 / $3.50 / $3.50 / $2.50 …).

Non-functional

- **Correctness under concurrency:** two entrance gates working at the same instant must never be given the same spot, and a ticket must never be paid twice.
- **Low latency at the gate:** finding a spot must not scan thousands of spots; taking one from a sorted set of free spots is O(log n). (**Latency** is how long one request takes from start to finish; see System Design Fundamentals. **O(log n)** is Big-O notation for cost: the work grows with the logarithm of the number of spots n, so even 10,000 spots need only about 14 steps.)
- **Extensibility:** a new pricing rule, a new spot-finding rule or a new vehicle type should be a small, local change.
- **Testability** (how easily automated tests can check the code)**:** time must be injectable (passed in from outside rather than read inside the code, here as a fake clock), so fee tests do not wait for hours.

Use cases

Actors (an actor is anyone or anything outside the system that uses it):

- **Admin** — adds, removes and edits floors, spots, entrance/exit panels; adds and removes attendants; sets the hourly rate.
- **Customer** — takes a ticket, pays, leaves.
- **Parking attendant** — can do everything a customer can, on the customer's behalf, and can accept cash.
- **System** — shows messages on panels and boards, assigns a vehicle to a spot and frees it again.

Top use cases: add/remove/edit a floor (each floor with its own display board); add/remove/edit a spot; add/remove an attendant; **take ticket**; **scan ticket** (to find the total charge); **credit-card payment**; **cash payment**; add/modify the parking rate.

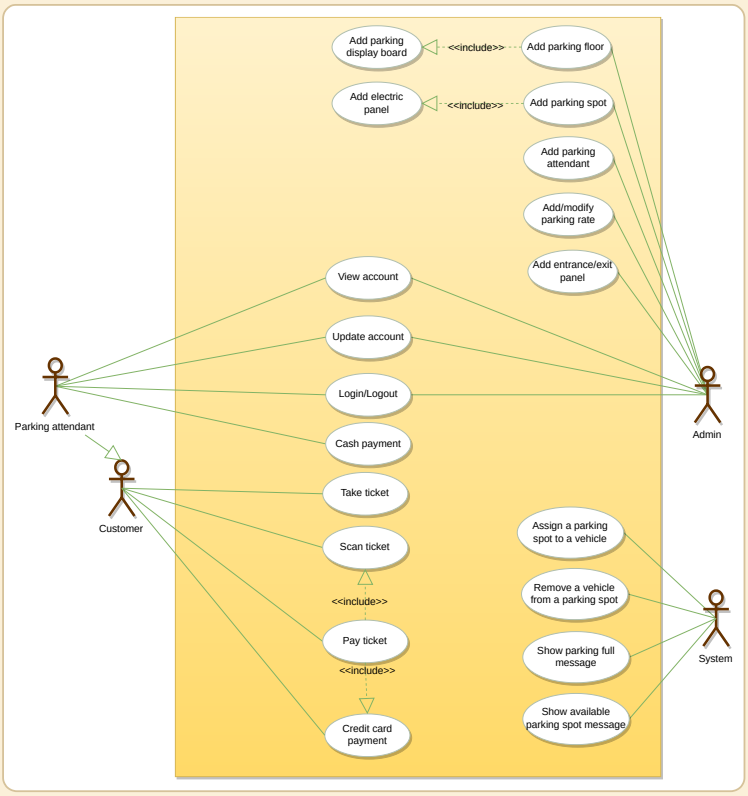


Figure — Use case diagram of the parking lot: Admin, Customer, Parking attendant and System, with their use cases (course diagram)

In the figure above, notice that the attendant's use cases overlap the customer's: the attendant is a customer "proxy" who can also take cash. The System actor is what assigns spots and updates the boards.

Core entities

Class	Responsibility
`VehicleType` (enum)	The kinds of vehicle and, for each, the spot types it fits in, **best fit first**.
`SpotType` (enum)	The kinds of spot: motorcycle, compact, large, handicapped, electric.
`Vehicle` (record)	Plate, type, accessibility permit. Knows its preferred spot types.
`ParkingSpot`	One physical spot: id, floor, type, and who occupies it (atomic).
`ChargingPanel`	The pay-and-charge panel owned by an electric spot.
`SpotFactory`	Builds spots; the only place that knows an electric spot needs a panel.
`ParkingFloor`	Holds the **free** spots per type in a thread-safe sorted set; notifies boards.
`DisplayBoard`	Shows free spots per type on one floor (an observer).
`SpotAllocationStrategy`	*How* to choose a spot (nearest floor first, least busy floor first, …).
`PricingStrategy`	*How much* a stay costs (hourly tiers, flat rate, weekend rate, …).
`Ticket`	Proof of entry: vehicle, the actual spot, entry time, status (ACTIVE → PAID → EXITED).
`PaymentMethod`	Cash or card.
`ParkingLot`	Ties it together: admit, price, pay, exit.
`EntrancePanel`, `ExitPanel`, `CustomerInfoPortal`	The devices customers touch. They hold **no rules**; they call the lot.

In the table, an **enum** is a Java type with a fixed list of named constants (such as `SpotType.COMPACT`), and a **record** is a short Java class that only carries a few values and cannot be changed after it is created. **Thread-safe** means an object stays correct even when several threads use it at the same moment.

The course also models **`Account`** with **`Admin`** and **`ParkingAttendant`** subclasses (both sign in). They carry no parking logic, so they are omitted from the code below; the attendant simply uses the same `pay` method as an exit panel.

Class diagram

The course grows the design in five steps. Every step adds a handful of classes and leaves you with a feature that runs. Use the same order in an interview: if the clock stops you early, what you have drawn already works, instead of being an unfinished picture.

Stage	What it adds	What works after it
1	Vehicle and spot types	The system can answer "may this vehicle use this spot?"
2	Floors and display boards	A floor can hand out a free spot and report counts
3	Ticket, rate card, payment	A stay can be turned into a price and settled
4	The `ParkingLot`	A full visit works: enter, park, pay, leave
5	Entrance/exit panels, info portal, electric panel	Drivers use the flow through physical devices

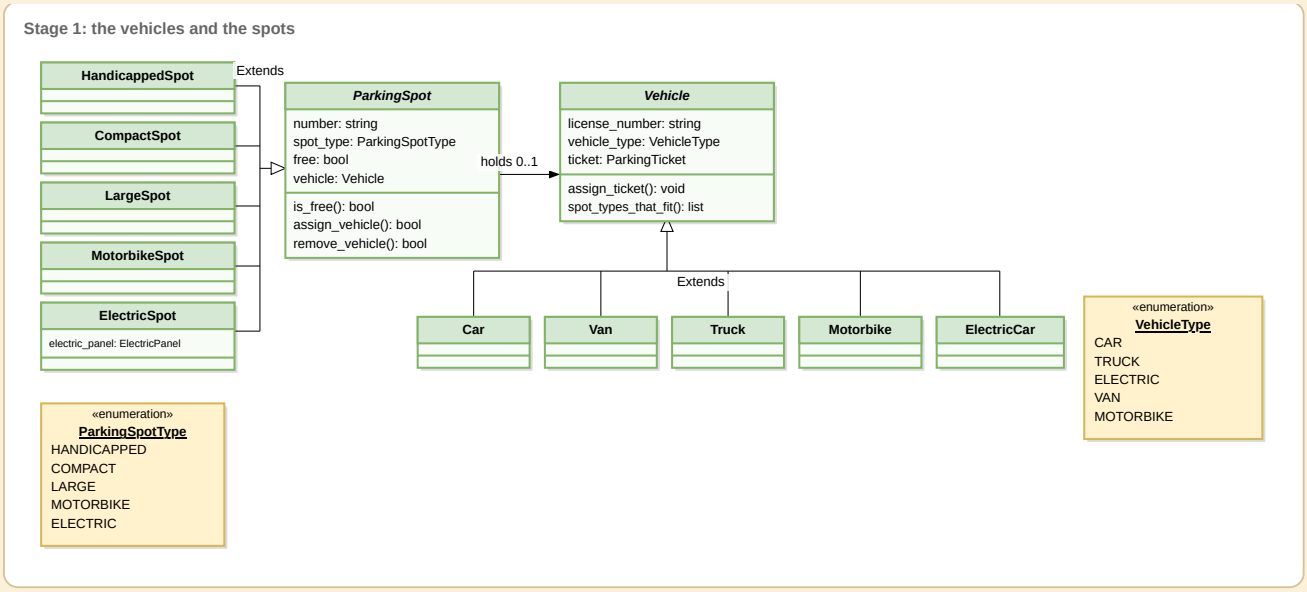


Figure — Stage 1: the vehicle and spot hierarchies, and the "holds 0..1" link from a spot to a vehicle (course diagram)

Stage 1 — vehicles and spots. Requirements 8 and 10 produce two small families of types and one link: a spot holds **at most one** vehicle (`0..1`). The most important rule of the whole design — *which spot types can this vehicle use?* — lives on the **vehicle** side. Because of that, introducing another kind of vehicle is a change to the vehicle types only; the lot code stays untouched. The fit rules: a motorcycle uses a motorcycle spot, trucks and vans use only large spots, and an electric car tries an electric spot first, then compact, then large.

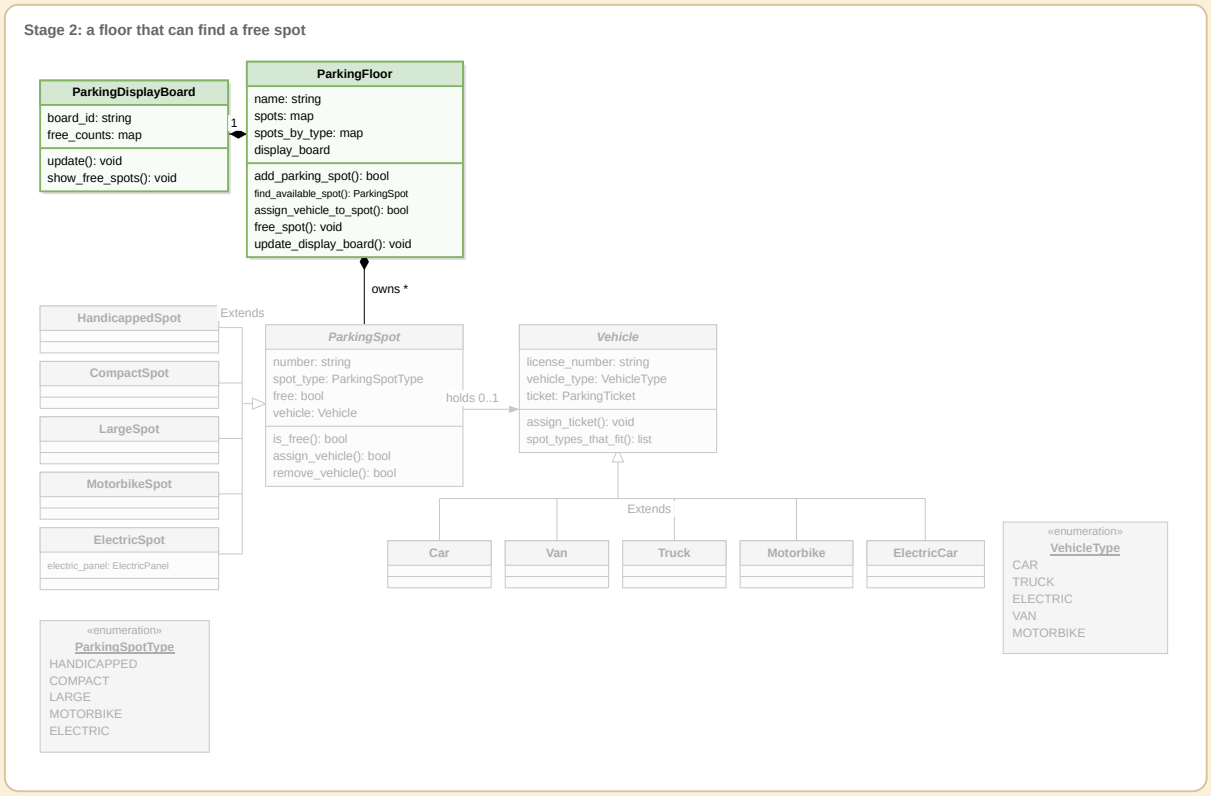


Figure — Stage 2: a floor owns its spots and its display board (composition) (course diagram)

Stage 2 — a floor that can find a free spot. Requirements 1, 8 and 11. Both new links are **composition** (filled diamond): deleting a floor deletes its spots and its board. The board stores **how many** spots of each type are free, not a pointer to "a sample free spot" of each type; such a pointer goes stale the moment that spot is taken and must be hunted for again. The course's floor keeps a map of all its spots plus an index by spot type (our code keeps a single map from spot type to its free spots). The rejected alternative is one separate

collection per type: every method would then need a five-way `if`, and adding a sixth type would touch all of them.

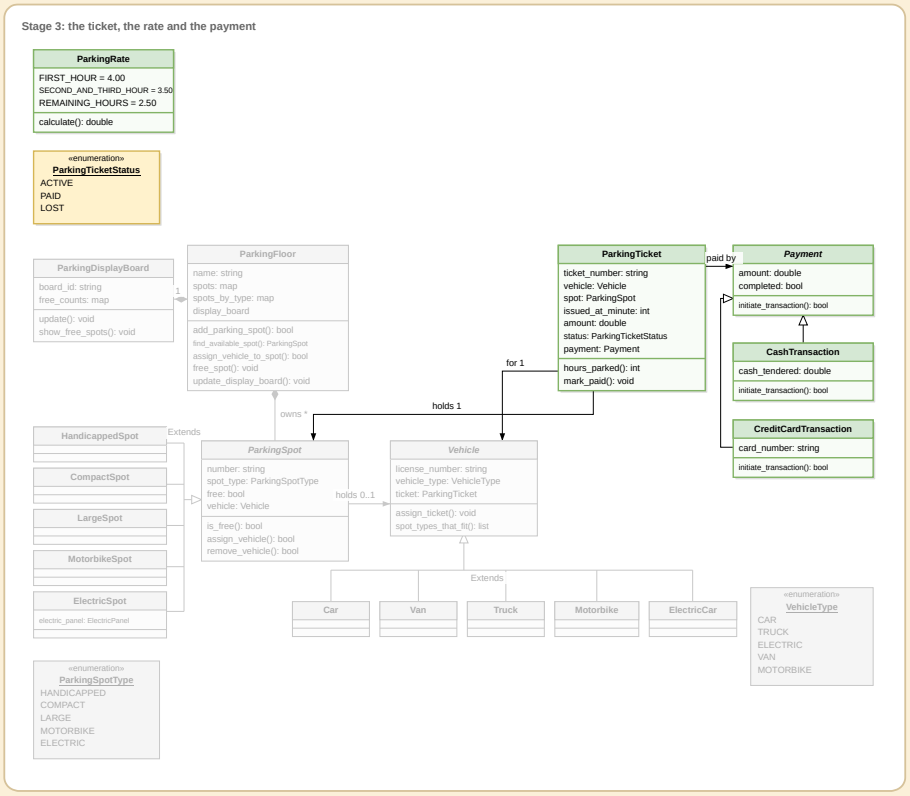


Figure — Stage 3: ticket, parking rate and the abstract payment with cash and card subclasses (course diagram)

`Stage 3 — ticket, rate, payment.` Requirements 3, 5 and 12. The key decision: **the ticket stores a reference to the exact spot**, not just "one more spot is taken". At the exit, that reference is how the system knows which spot to free and where the car was. A ticket merely points at the vehicle and the spot (it does not own them), so both links are plain associations. `Payment` is an abstract class (it cannot be created directly) with cash and card subclasses; this inheritance is justified because either subclass can stand in wherever "a payment" is expected (the Liskov Substitution Principle from Chapter 4).

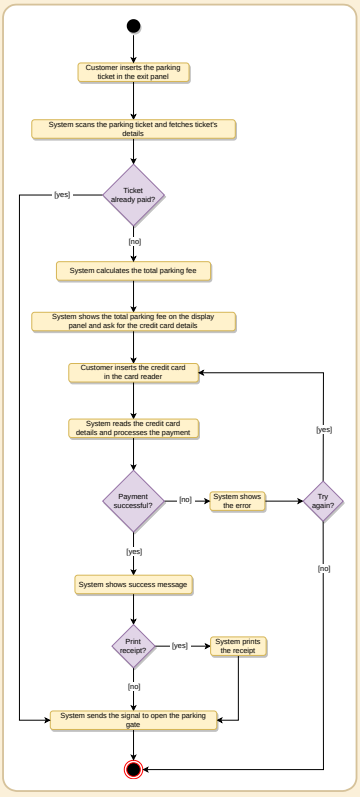


Figure — Activity diagram: the customer inserts the ticket at the exit, the system computes the fee, the customer pays by card or cash, and the barrier opens (course diagram)

The activity diagram above is the exit flow; the ticket, rate and payment classes between them handle each of its steps.

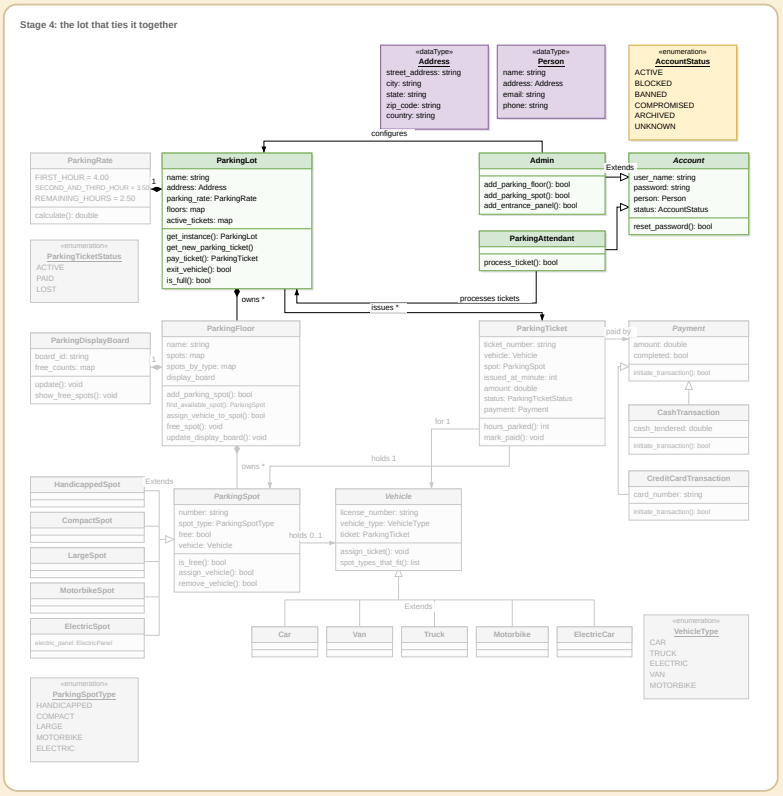


Figure — Stage 4: the parking lot owns floors and the rate; Admin and ParkingAttendant accounts connect to it (course diagram)

Stage 4 — the lot. Requirement 7 and the actor list. The lot has a **name** and an **address** (so one lot can be told apart from another) and owns its floors and its rate card (filled diamonds). The two account types, `Admin` and `ParkingAttendant`, are linked with plain arrows: they work *on* the lot (setting it up, handling tickets) without owning any part of it. The course makes the lot a **Singleton** (only one instance may exist) and serialises spot assignment (**one caller at a time**), since two entrances assigning in parallel could pick the same spot.

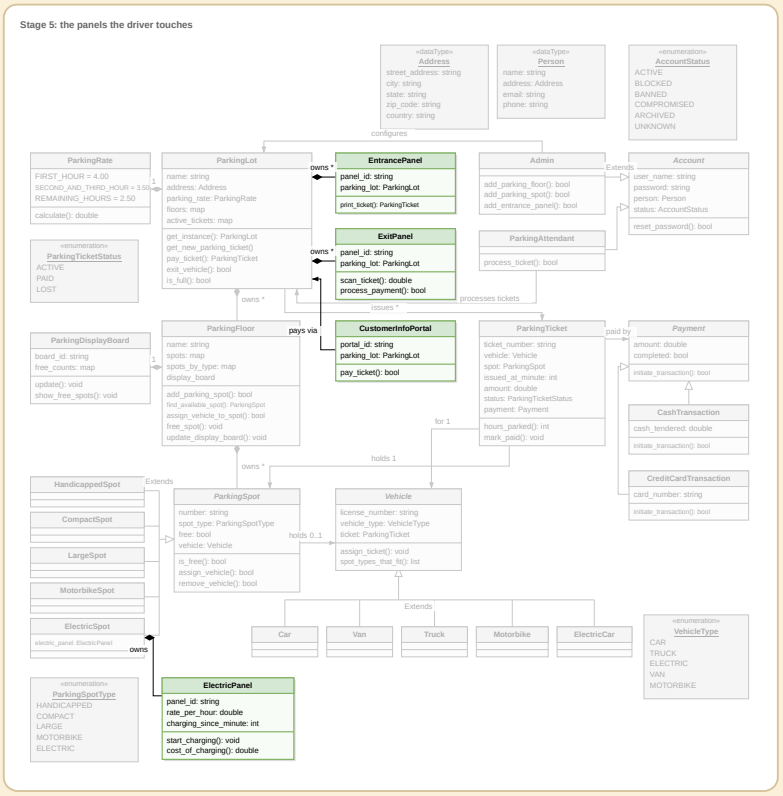


Figure — Stage 5: entrance and exit panels (owned by the lot), the customer info portal (refers to the lot) and the electric panel (owned by an electric spot) (course diagram)

Stage 5 — the panels. Requirements 2, 4, 6 and 9. The panels are thin: they forward every request to the lot, so all business rules sit in a single class. The entrance panel prints tickets, the exit panel scans a ticket and takes payment, the info portal lets a driver settle early (marking the ticket paid), and the electric panel bills for charging. The lot owns its entrance and exit panels (composition). It keeps no list of info portals, so a portal just holds a reference to the lot (association). The electric panel is owned by its electric spot, the only composition hanging off a spot.

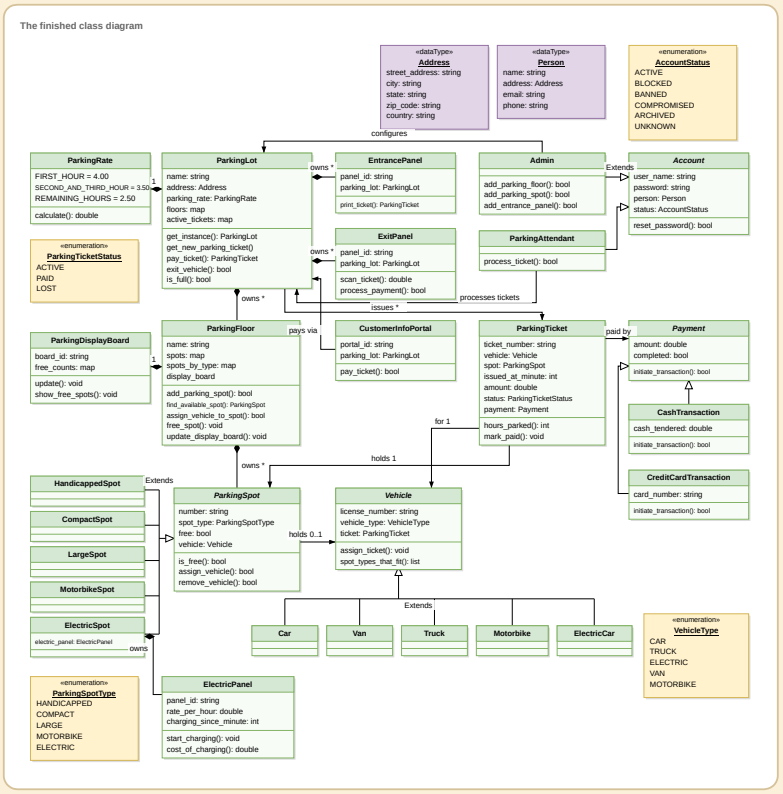
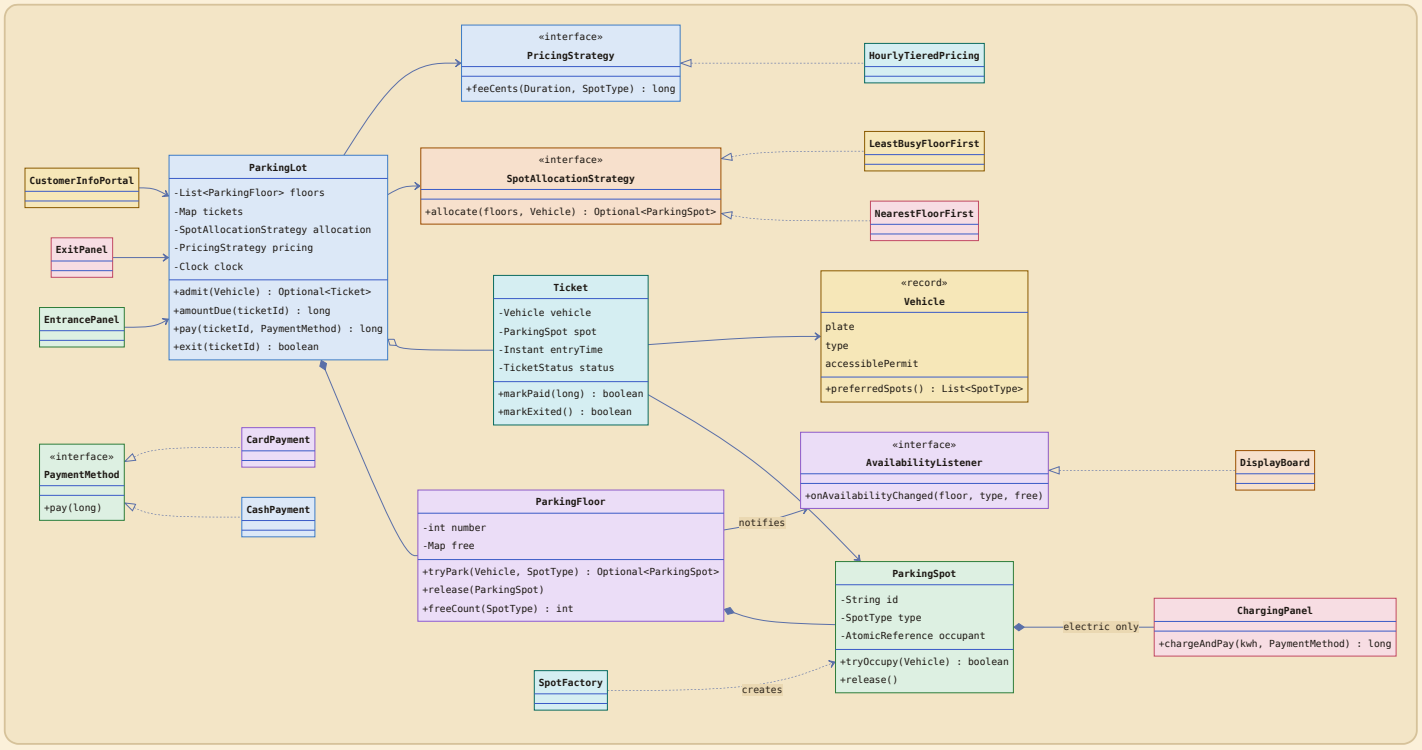


Figure — The finished class diagram of the parking lot (course diagram)

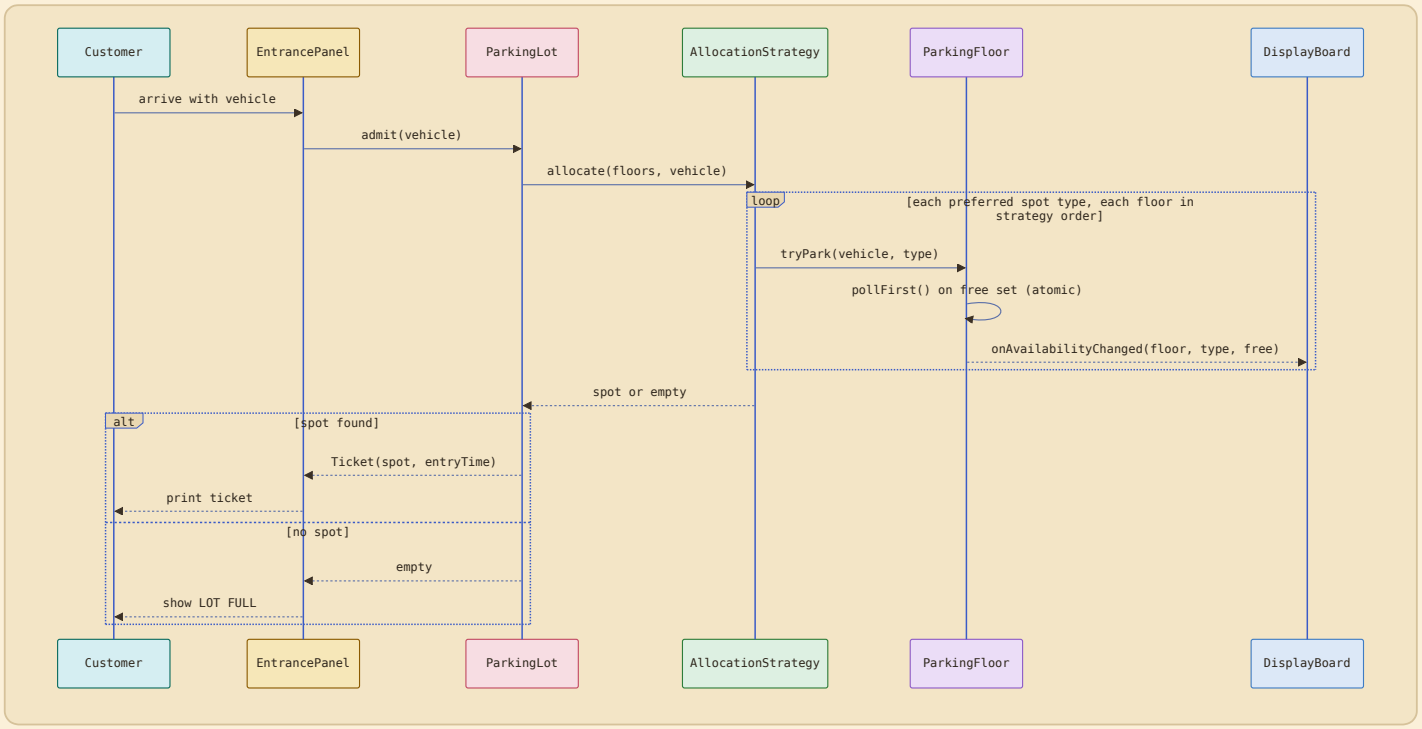
The finished diagram above is simply all five stages in colour; nothing new appears in it. Here is the class diagram of **the Java code in this chapter**. It keeps the same shape but makes three changes that interviewers usually push for: spot types are an enum instead of five empty subclasses, pricing and spot-finding are Strategies, and the display board is an Observer.



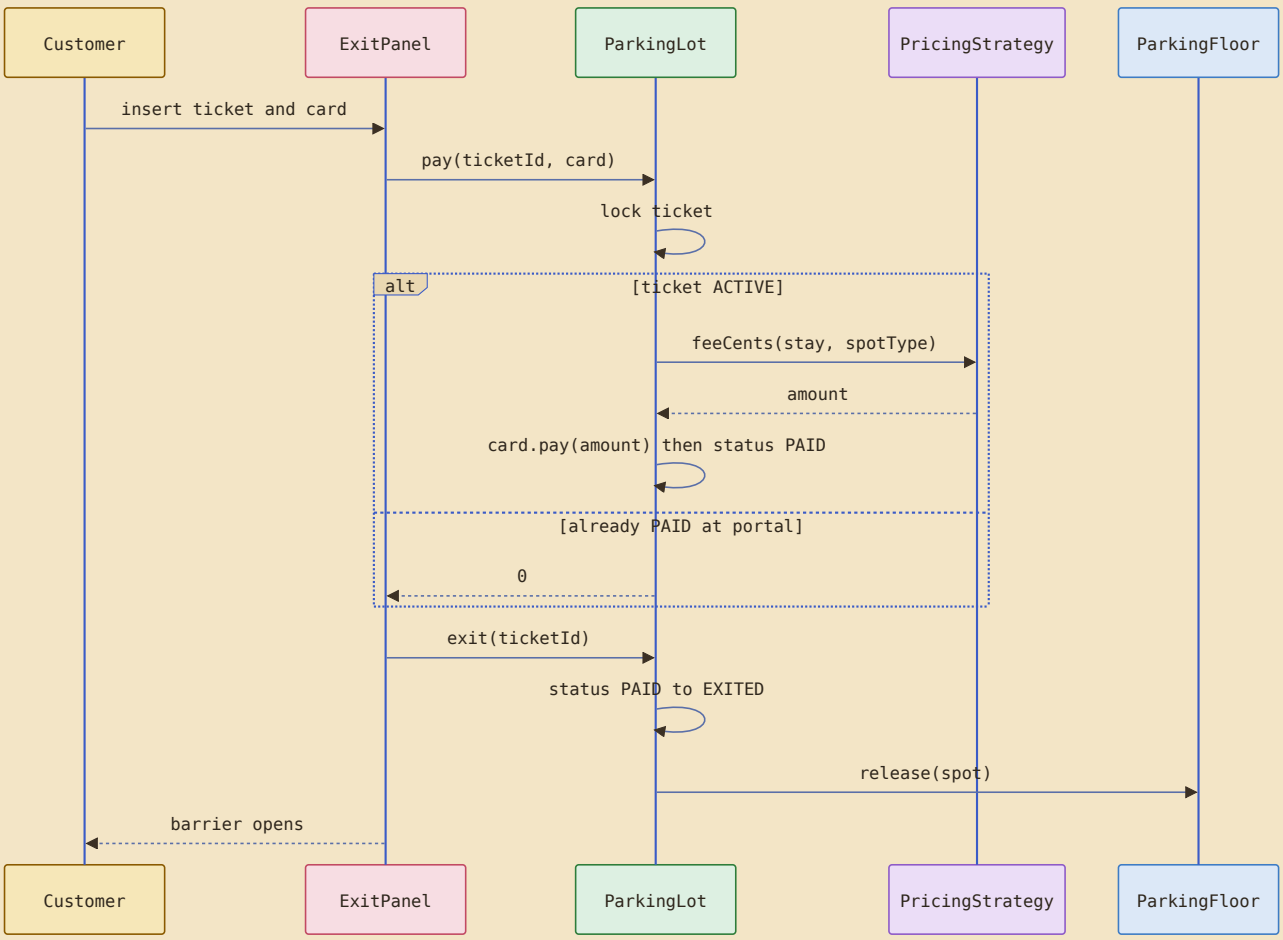
In this Mermaid diagram (Mermaid is a text format that renders as a diagram), `-` marks a private field (hidden inside its class), `+` a public method, `~T~` a generic type such as `List<ParkingFloor>`, `*--` composition, `o--` aggregation (a looser "has a" where the parts can live on their own), `-->` association, `<|..` "implements", and `..>` "uses or creates".

Key flows

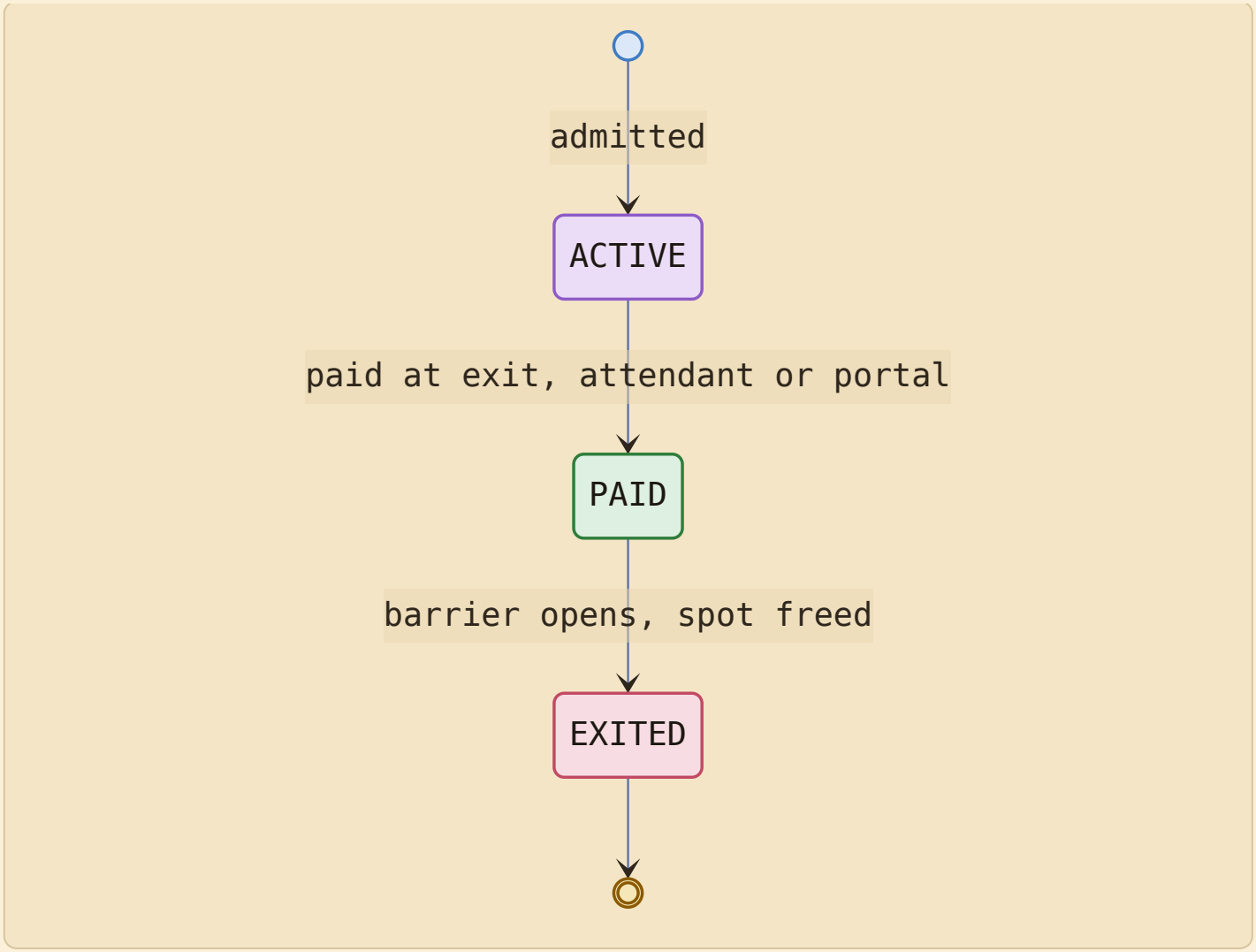
Entry (take ticket). The important step is that the spot is taken **atomically** (as one indivisible step that no other thread can interrupt halfway): `pollFirst()` on a concurrent sorted set removes the spot for exactly one caller. The **sequence diagram** below shows the messages between objects in time order, from top to bottom.



Exit (scan, pay, leave). Payment is guarded per ticket so the info portal and the exit panel cannot both charge.



Ticket life cycle.


admitted
ACTIVE
paid at exit, attendant or portal
PAID
barrier opens, spot freed
EXITED

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
The fit rule lives on `VehicleType`	Information Expert (give a job to the class that owns the data), Open/Closed (extend without editing)	Adding a vehicle type is one enum constant; the lot never changes	A big `switch(vehicleType)` inside the lot
Spot type is an **enum field**, not 5 subclasses	Composition over inheritance (hold a field instead of subclassing)	The 5 course subclasses carried no behaviour; a new type is one constant	`CompactSpot`, `LargeSpot`… (reviewers flag these as empty classes)
How to pick a spot is a `SpotAllocationStrategy`	**Strategy**	Nearest floor, least busy floor, closest to the lift, VIP floors… all swap without touching the lot	`if (mode == NEAREST) … else …` in the lot
How much to charge is a `PricingStrategy`	**Strategy**	The course's "what if pricing changes?" follow-up; admin swaps the rate at run time (the demo swaps to a flat rate)	Hard-coded tiers inside `Ticket`
Electric spots get a `ChargingPanel` in `SpotFactory`	**Factory** (simple static factory)	One place knows the construction rule "electric ⇒ has a panel"	Every caller remembers to attach a panel
Boards subscribe to floors	**Observer**	A floor change updates every board (and could update an app or a sensor light) without the floor knowing who listens	The lot pushing into boards directly
Ticket holds the **spot**, not a count	—	The exit can find and free the exact spot	Counting occupied spots
`ParkingLot` is **not** a hard-coded Singleton	Dependency injection (pass collaborators in from outside)	The course uses a Singleton ("there is one car park"). It is a global in disguise: tests cannot build two lots, and every class that calls `getInstance()` hides a dependency. We create one lot in `main` and pass it to the panels.	`ParkingLot.getInstance()`
`Clock` is injected	Testability	The demo moves time forward by hours instantly	`Instant.now()` inside the code

> **Honest note on "Strategy".** The course points out that a cash/card subclass pair is just an abstract base class, and calling it "Strategy" would be loose use of the word. Strategy means an object **holds** a behaviour and can **swap** it. In our code `ParkingLot` holds a `PricingStrategy` and a `SpotAllocationStrategy` and `setPricing(...)` swaps it at run time — that is genuinely Strategy.

The course also admits that `ParkingLot` does two jobs (finding spots and issuing tickets). Here spot-finding has moved into the strategy and the floors, so the lot is mostly a coordinator; in a longer design you would also move ticket storage into a `TicketRepository`.

Java implementation

Package `lld.ch14.parkinglot`. Amounts are in **cents** (`long`) to avoid floating-point rounding (a `double` stores numbers in binary and cannot hold values such as 0.10 exactly, so sums drift by tiny amounts).

```java
// File: lld/ch14/parkinglot/Vehicles.java
package lld.ch14.parkinglot;

import java.util.ArrayList;
import java.util.List;

/** The kinds of spot a floor can have. */
enum SpotType { MOTORCYCLE, COMPACT, LARGE, HANDICAPPED, ELECTRIC }

/** The kinds of vehicle the lot accepts, each with the spot types it fits, best fit first. */
enum VehicleType {
    MOTORCYCLE(List.of(SpotType.MOTORCYCLE)),
    CAR(List.of(SpotType.COMPACT, SpotType.LARGE)),
    ELECTRIC_CAR(List.of(SpotType.ELECTRIC, SpotType.COMPACT, SpotType.LARGE)),
    VAN(List.of(SpotType.LARGE)),
    TRUCK(List.of(SpotType.LARGE));

    private final List<SpotType> fits;

    VehicleType(List<SpotType> fits) { this.fits = fits; }

    List<SpotType> fits() { return fits; }
}

/** A vehicle is a value: plate + type + whether it carries an accessibility permit. */
record Vehicle(String plate, VehicleType type, boolean accessiblePermit) {

    Vehicle(String plate, VehicleType type) { this(plate, type, false); }

    /** Spot types this vehicle may use, most preferred first. The rule lives here, not in the lot. */
    List<SpotType> preferredSpots() {
        List<SpotType> order = new ArrayList<>();
        if (accessiblePermit && type != VehicleType.MOTORCYCLE && type != VehicleType.TRUCK) {
            order.add(SpotType.HANDICAPPED);
        }
        order.addAll(type.fits());
        return order;
    }
}
```

```java
// File: lld/ch14/parkinglot/ParkingSpot.java
package lld.ch14.parkinglot;

import java.util.Optional;
import java.util.concurrent.atomic.AtomicReference;

/** One physical spot. Occupancy is an AtomicReference so two threads can never both win it. */
class ParkingSpot implements Comparable<ParkingSpot> {
    private final String id;          // e.g. "F1-C03"
    private final int floor;
    private final SpotType type;
    private final AtomicReference<Vehicle> occupant = new AtomicReference<>();
    private final ChargingPanel chargingPanel; // only electric spots have one (composition)

    ParkingSpot(String id, int floor, SpotType type, ChargingPanel chargingPanel) {
        this.id = id;
        this.floor = floor;
        this.type = type;
        this.chargingPanel = chargingPanel;
    }

    /** Compare-and-set: succeeds only if the spot was empty. */
    boolean tryOccupy(Vehicle v) { return occupant.compareAndSet(null, v); }

    void release() { occupant.set(null); }

    boolean isFree() { return occupant.get() == null; }

    String id() { return id; }
    int floor() { return floor; }
    SpotType type() { return type; }
    Optional<ChargingPanel> chargingPanel() { return Optional.ofNullable(chargingPanel); }

    @Override public int compareTo(ParkingSpot o) { return id.compareTo(o.id); }
    @Override public String toString() { return id + "(" + type + ")"; }
}

/** The pay-and-charge panel that belongs to an electric spot. */
class ChargingPanel {
    private final long centsPerKwh;

    ChargingPanel(long centsPerKwh) { this.centsPerKwh = centsPerKwh; }

    /** Charges the car and takes payment for the energy right at the spot. */
    long chargeAndPay(double kwh, PaymentMethod method) {
        long cost = Math.round(kwh * centsPerKwh);
        method.pay(cost);
        return cost;
    }
}

/** Factory: the only place that knows an ELECTRIC spot must be built with its panel. */
final class SpotFactory {
    private static final long EV_CENTS_PER_KWH = 30;

    private SpotFactory() {}

    static ParkingSpot create(int floor, SpotType type, int number) {
        String id = "F%d-%s%02d".formatted(floor, type.name().charAt(0), number);
        ChargingPanel panel = type == SpotType.ELECTRIC ? new ChargingPanel(EV_CENTS_PER_KWH) : null;
        return new ParkingSpot(id, floor, type, panel);
    }
}
```

```java
// File: lld/ch14/parkinglot/ParkingFloor.java
package lld.ch14.parkinglot;

import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentSkipListSet;
import java.util.concurrent.CopyOnWriteArrayList;

/** Observer: anything that wants to know when free-spot counts change. */
interface AvailabilityListener {
    void onAvailabilityChanged(int floor, SpotType type, int freeNow);
}

/** A floor keeps one sorted, thread-safe set of FREE spots per spot type. */
class ParkingFloor {
    private final int number;
    private final Map<SpotType, ConcurrentSkipListSet<ParkingSpot>> free = new EnumMap<>(SpotType.class);
    private final List<AvailabilityListener> listeners = new CopyOnWriteArrayList<>();
    private final Set<SpotType> typesHere;

    ParkingFloor(int number, Map<SpotType, Integer> spotCounts) {
        this.number = number;
        for (SpotType t : SpotType.values()) free.put(t, new ConcurrentSkipListSet<>());
        typesHere = Set.copyOf(spotCounts.keySet());
        spotCounts.forEach((type, count) -> {
            for (int i = 1; i <= count; i++) free.get(type).add(SpotFactory.create(number, type, i));
        });
    }

    void addListener(AvailabilityListener l) {
        listeners.add(l);
        for (SpotType t : typesHere) l.onAvailabilityChanged(number, t, freeCount(t));
    }

    /** pollFirst() is atomic: two threads can never receive the same spot. Lowest id = nearest. */
    Optional<ParkingSpot> tryPark(Vehicle v, SpotType type) {
        ParkingSpot spot = free.get(type).pollFirst();
        if (spot == null) return Optional.empty();
        if (!spot.tryOccupy(v)) throw new IllegalStateException("free-set out of sync for " + spot);
        notifyListeners(type);
        return Optional.of(spot);
    }

    void release(ParkingSpot spot) {
        spot.release();
        free.get(spot.type()).add(spot);
        notifyListeners(spot.type());
    }

    int freeCount(SpotType t) { return free.get(t).size(); }
    int number() { return number; }

    private void notifyListeners(SpotType t) {
        int n = freeCount(t);
        for (AvailabilityListener l : listeners) l.onAvailabilityChanged(number, t, n);
    }
}

/** The display board on a floor: shows free spots per type. */
class DisplayBoard implements AvailabilityListener {
    private final Map<SpotType, Integer> shown = new EnumMap<>(SpotType.class);

    @Override public synchronized void onAvailabilityChanged(int floor, SpotType type, int freeNow) {
        shown.put(type, freeNow);
    }
```

```
    synchronized String render() {
        StringBuilder sb = new StringBuilder();
        shown.forEach((t, n) -> sb.append(t).append('=').append(n == 0 ? "FULL" : n).append(' '));
        return sb.toString().trim();
    }
}
```

```java
// File: lld/ch14/parkinglot/Strategies.java
package lld.ch14.parkinglot;

import java.time.Duration;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;

/** Strategy: HOW to pick a spot. */
interface SpotAllocationStrategy {
    Optional<ParkingSpot> allocate(List<ParkingFloor> floors, Vehicle v);
}

/** Fill the lowest floor first (fewest ramps to drive). */
class NearestFloorFirst implements SpotAllocationStrategy {
    @Override public Optional<ParkingSpot> allocate(List<ParkingFloor> floors, Vehicle v) {
        for (SpotType type : v.preferredSpots()) {
            for (ParkingFloor f : floors) {
                Optional<ParkingSpot> s = f.tryPark(v, type);
                if (s.isPresent()) return s;
            }
        }
        return Optional.empty();
    }
}

/** Spread cars out: pick the floor with the most free spots of the wanted type. */
class LeastBusyFloorFirst implements SpotAllocationStrategy {
    @Override public Optional<ParkingSpot> allocate(List<ParkingFloor> floors, Vehicle v) {
        for (SpotType type : v.preferredSpots()) {
            List<ParkingFloor> order = floors.stream()
                    .sorted(Comparator.comparingInt((ParkingFloor f) -> f.freeCount(type)).reversed())
                    .toList();
            for (ParkingFloor f : order) {
                Optional<ParkingSpot> s = f.tryPark(v, type); // may lose a race: just try the next floor
                if (s.isPresent()) return s;
            }
        }
        return Optional.empty();
    }
}

/** Strategy: HOW MUCH a stay costs. */
interface PricingStrategy {
    long feeCents(Duration stay, SpotType spotType);
}

/**
 * Per-hour tiers: $4 for hour 1, $3.50 for hours 2 and 3, $2.50 for every hour after.
 * A started hour counts as a full hour.
 */
class HourlyTieredPricing implements PricingStrategy {
    private final List<Long> firstHours;   // price of hour 1, 2, 3 ...
    private final long laterHourCents;     // price of every later hour

    HourlyTieredPricing(List<Long> firstHours, long laterHourCents) {
        this.firstHours = List.copyOf(firstHours);
        this.laterHourCents = laterHourCents;
    }

    static HourlyTieredPricing standard() { return new HourlyTieredPricing(List.of(400L, 350L, 350L), 250); }

    @Override public long feeCents(Duration stay, SpotType spotType) {
        long minutes = Math.max(1, stay.toMinutes());
        long hours = (minutes + 59) / 60;                  // round up
        long total = 0;
        for (long h = 0; h < hours; h++) {
            total += h < firstHours.size() ? firstHours.get((int) h) : laterHourCents;
```

```
        }
        return total;
    }
}
```

```java
// File: lld/ch14/parkinglot/Ticketing.java
package lld.ch14.parkinglot;

import java.time.Instant;

enum TicketStatus { ACTIVE, PAID, EXITED }

/** The ticket points at the real spot, so the exit can find and free it. */
final class Ticket {
    private final String id;
    private final Vehicle vehicle;
    private final ParkingSpot spot;
    private final Instant entryTime;
    private TicketStatus status = TicketStatus.ACTIVE;
    private long paidCents;

    Ticket(String id, Vehicle vehicle, ParkingSpot spot, Instant entryTime) {
        this.id = id;
        this.vehicle = vehicle;
        this.spot = spot;
        this.entryTime = entryTime;
    }

    /** Only one payment can ever succeed, even if the portal and the exit panel race. */
    synchronized boolean markPaid(long cents) {
        if (status != TicketStatus.ACTIVE) return false;
        status = TicketStatus.PAID;
        paidCents = cents;
        return true;
    }

    synchronized boolean markExited() {
        if (status != TicketStatus.PAID) return false;
        status = TicketStatus.EXITED;
        return true;
    }

    synchronized TicketStatus status() { return status; }
    synchronized long paidCents() { return paidCents; }
    String id() { return id; }
    Vehicle vehicle() { return vehicle; }
    ParkingSpot spot() { return spot; }
    Instant entryTime() { return entryTime; }
}

/** How a customer pays. Cash and card are two implementations. */
interface PaymentMethod {
    void pay(long cents);
    String describe();
}

record CashPayment(long tenderedCents) implements PaymentMethod {
    @Override public void pay(long cents) {
        if (tenderedCents < cents) throw new IllegalArgumentException("not enough cash");
    }
    @Override public String describe() {
        return "cash " + Money.fmt(tenderedCents);
    }
}

record CardPayment(String last4) implements PaymentMethod {
    @Override public void pay(long cents) { /* a real system calls the card gateway here */ }
    @Override public String describe() { return "card ****" + last4; }
}

final class Money {
    private Money() {}
```

```
    static String fmt(long cents) { return "$%d.%02d".formatted(cents / 100, cents % 100); }
}
```

```java
// File: lld/ch14/parkinglot/ParkingLot.java
package lld.ch14.parkinglot;

import java.time.Clock;
import java.time.Duration;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

/**
 * The lot ties floors, tickets, pricing and allocation together.
 * It is NOT a hard-coded Singleton: the application creates one and injects it
 * into the panels, which keeps tests able to build as many lots as they like.
 */
final class ParkingLot {
    private final String name;
    private final List<ParkingFloor> floors;
    private final Clock clock;
    private final Map<String, Ticket> tickets = new ConcurrentHashMap<>();
    private final Map<String, String> ticketByPlate = new ConcurrentHashMap<>();
    private final AtomicLong nextTicket = new AtomicLong(1);
    private volatile SpotAllocationStrategy allocation;
    private volatile PricingStrategy pricing;

    ParkingLot(String name, List<ParkingFloor> floors, Clock clock,
               SpotAllocationStrategy allocation, PricingStrategy pricing) {
        this.name = name;
        this.floors = List.copyOf(floors);
        this.clock = clock;
        this.allocation = allocation;
        this.pricing = pricing;
    }

    /** Entry: find a spot and issue a ticket. Empty result means "full for this vehicle". */
    Optional<Ticket> admit(Vehicle v) {
        String placeholder = "pending";
        if (ticketByPlate.putIfAbsent(v.plate(), placeholder) != null) {
            throw new IllegalStateException(v.plate() + " is already inside");
        }
        Optional<ParkingSpot> spot = allocation.allocate(floors, v);
        if (spot.isEmpty()) {
            ticketByPlate.remove(v.plate());
            return Optional.empty();
        }
        Ticket t = new Ticket("T" + nextTicket.getAndIncrement(), v, spot.get(), clock.instant());
        tickets.put(t.id(), t);
        ticketByPlate.put(v.plate(), t.id());
        return Optional.of(t);
    }

    long amountDue(String ticketId) {
        Ticket t = ticket(ticketId);
        if (t.status() != TicketStatus.ACTIVE) return 0;            // already paid (e.g. at the portal)
        Duration stay = Duration.between(t.entryTime(), clock.instant());
        return pricing.feeCents(stay, t.spot().type());
    }

    /** Pays the ticket once. Returns the amount charged (0 if it was already paid). */
    long pay(String ticketId, PaymentMethod method) {
        Ticket t = ticket(ticketId);
        synchronized (t) {                // per-ticket lock: portal and exit panel cannot both charge
            long due = amountDue(ticketId);
            if (due == 0) return 0;
            method.pay(due);              // throws if the payment fails: ticket stays ACTIVE
            t.markPaid(due);
            return due;
```

```
        }
    }

    /** Exit: only a PAID ticket opens the barrier. Frees the spot. */
    boolean exit(String ticketId) {
        Ticket t = ticket(ticketId);
        if (!t.markExited()) return false;
        floors.get(t.spot().floor() - 1).release(t.spot());
        tickets.remove(ticketId);
        ticketByPlate.remove(t.vehicle().plate());
        return true;
    }

    boolean isFullFor(VehicleType type) {
        return floors.stream().noneMatch(f -> type.fits().stream().anyMatch(s -> f.freeCount(s) > 0));
    }

    void setPricing(PricingStrategy p) { this.pricing = p; }                // admin changes the rate
    void setAllocation(SpotAllocationStrategy a) { this.allocation = a; }
    List<ParkingFloor> floors() { return floors; }
    String name() { return name; }

    Ticket ticket(String id) {
        Ticket t = tickets.get(id);
        if (t == null) throw new IllegalArgumentException("unknown ticket " + id);
        return t;
    }
}
```

```java
// File: lld/ch14/parkinglot/Panels.java
package lld.ch14.parkinglot;

/** Panels hold no rules. They talk to the lot and print what happened. */
class EntrancePanel {
    private final String id;
    private final ParkingLot lot;

    EntrancePanel(String id, ParkingLot lot) { this.id = id; this.lot = lot; }

    String issueTicket(Vehicle v) {
        return lot.admit(v)
                .map(t -> "[%s] %s -> ticket %s, spot %s".formatted(id, v.plate(), t.id(), t.spot()))
                .orElse("[%s] %s -> SORRY, LOT FULL for %s".formatted(id, v.plate(), v.type()));
    }
}

class ExitPanel {
    private final String id;
    private final ParkingLot lot;

    ExitPanel(String id, ParkingLot lot) { this.id = id; this.lot = lot; }

    String scanPayAndExit(String ticketId, PaymentMethod method) {
        long charged = lot.pay(ticketId, method);
        boolean open = lot.exit(ticketId);
        String paid = charged == 0 ? "already paid" : "paid " + Money.fmt(charged) + " by " + method.describe();
        return "[%s] %s: %s, barrier %s".formatted(id, ticketId, paid, open ? "OPEN" : "CLOSED");
    }
}

/** On each floor: lets the customer pay early so the exit is free. */
class CustomerInfoPortal {
    private final ParkingLot lot;

    CustomerInfoPortal(ParkingLot lot) { this.lot = lot; }

    String pay(String ticketId, PaymentMethod method) {
        long due = lot.amountDue(ticketId);
        long charged = lot.pay(ticketId, method);
        return "[portal] %s due %s, charged %s".formatted(ticketId, Money.fmt(due), Money.fmt(charged));
    }
}
```

```java
// File: lld/ch14/parkinglot/Main.java
package lld.ch14.parkinglot;

import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {

    /** A clock we can move forward, so the demo is deterministic. */
    static final class TestClock extends Clock {
        private volatile Instant now = Instant.parse("2026-01-01T09:00:00Z");
        void advance(Duration d) { now = now.plus(d); }
        @Override public Instant instant() { return now; }
        @Override public ZoneId getZone() { return ZoneOffset.UTC; }
        @Override public Clock withZone(ZoneId zone) { return this; }
    }

    public static void main(String[] args) throws Exception {
        TestClock clock = new TestClock();
        ParkingFloor f1 = new ParkingFloor(1, Map.of(SpotType.COMPACT, 2, SpotType.LARGE, 1,
                SpotType.MOTORCYCLE, 1, SpotType.HANDICAPPED, 1, SpotType.ELECTRIC, 1));
        ParkingFloor f2 = new ParkingFloor(2, Map.of(SpotType.COMPACT, 3, SpotType.LARGE, 1));
        DisplayBoard board1 = new DisplayBoard(), board2 = new DisplayBoard();
        f1.addListener(board1);
        f2.addListener(board2);

        ParkingLot lot = new ParkingLot("City Mall", List.of(f1, f2), clock,
                new NearestFloorFirst(), HourlyTieredPricing.standard());
        EntrancePanel gateA = new EntrancePanel("entry-A", lot);
        ExitPanel exitA = new ExitPanel("exit-A", lot);
        CustomerInfoPortal portal = new CustomerInfoPortal(lot);

        System.out.println("== Arrivals ==");
        System.out.println(gateA.issueTicket(new Vehicle("CAR-1", VehicleType.CAR)));
        System.out.println(gateA.issueTicket(new Vehicle("EV-1", VehicleType.ELECTRIC_CAR)));
        System.out.println(gateA.issueTicket(new Vehicle("BIKE-1", VehicleType.MOTORCYCLE)));
        System.out.println(gateA.issueTicket(new Vehicle("CAR-2", VehicleType.CAR, true)));
        System.out.println(gateA.issueTicket(new Vehicle("TRUCK-1", VehicleType.TRUCK)));
        System.out.println(gateA.issueTicket(new Vehicle("VAN-1", VehicleType.VAN)));
        System.out.println(gateA.issueTicket(new Vehicle("TRUCK-2", VehicleType.TRUCK)));
        System.out.println("Floor 1 board: " + board1.render());
        System.out.println("Floor 2 board: " + board2.render());

        System.out.println("\n== EV charging at the spot ==");
        ChargingPanel panel = lot.ticket("T2").spot().chargingPanel().orElseThrow();
        System.out.println("EV-1 charged 20 kWh, paid " + Money.fmt(panel.chargeAndPay(20, new
CardPayment("4242"))));

        System.out.println("\n== Departures ==");
        clock.advance(Duration.ofMinutes(50));
        System.out.println(exitA.scanPayAndExit("T1", new CashPayment(500)));          // 1 hour
        clock.advance(Duration.ofHours(4));                                             // T2 now 4h50m
        System.out.println(portal.pay("T2", new CardPayment("1111")));                  // 5 hours
        System.out.println(exitA.scanPayAndExit("T2", new CardPayment("1111")));
        System.out.println("Unpaid exit attempt: barrier opens? " + lot.exit("T3"));

        System.out.println("\n== Admin changes the rate: flat $5/hour ==");
        lot.setPricing((stay, type) -> 500 * Math.max(1, (stay.toMinutes() + 59) / 60));
        System.out.println(exitA.scanPayAndExit("T3", new CardPayment("9999")));        // 4h50m -> 5h
```

```
        System.out.println("\n== 20 cars race for the remaining CAR spots ==");
        System.out.println("free compact before: F1=" + f1.freeCount(SpotType.COMPACT)
                + " F2=" + f2.freeCount(SpotType.COMPACT) + ", large: F1=" + f1.freeCount(SpotType.LARGE)
                + " F2=" + f2.freeCount(SpotType.LARGE));
        ConcurrentLinkedQueue<String> winners = new ConcurrentLinkedQueue<>();
        CountDownLatch start = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            for (int i = 0; i < 20; i++) {
                String plate = "RACE-" + i;
                pool.submit(() -> {
                    start.await();
                    lot.admit(new Vehicle(plate, VehicleType.CAR)).ifPresent(t -> winners.add(t.spot().id()));
                    return null;
                });
            }
            start.countDown();
        } // close() waits for all tasks
        System.out.println("cars admitted: " + winners.size()
                + ", distinct spots: " + winners.stream().distinct().count());
        System.out.println("full for cars now? " + lot.isFullFor(VehicleType.CAR));
        System.out.println(gateA.issueTicket(new Vehicle("LATE-CAR", VehicleType.CAR)));
        System.out.println("Floor 2 board: " + board2.render());
    }
}
```

How the code works, part by part

- `Vehicle.preferredSpots()` returns the spot types in order of preference. An accessibility permit puts `HANDICAPPED` first; nobody else can ever get a handicapped spot.
- `ParkingFloor` keeps a `ConcurrentSkipListSet` (a thread-safe set from `java.util.concurrent` that keeps its elements sorted) of **free** spots per type, sorted by id. `pollFirst()` removes and returns the lowest id **atomically**: if 20 threads call it at once, each spot goes to exactly one of them. The lowest id stands for "nearest to the entrance".
- `ParkingSpot.tryOccupy` is a second safety net: an `AtomicReference.compareAndSet(null, v)` succeeds only if the spot was empty. This is **CAS** (compare-and-set): "change the value to *v* only if it is still *null*", done as one atomic step without any lock.
- `NearestFloorFirst` and `LeastBusyFloorFirst` are two interchangeable allocation strategies.
- `HourlyTieredPricing.standard()` encodes $4 / $3.50 / $3.50 / then $2.50, rounding a started hour up.
- `ParkingLot.admit` also reserves the **plate** with `putIfAbsent` (adds a key to a map only if it is not there yet, atomically), so the same car cannot be admitted twice.
- `ParkingLot.pay` locks **one ticket** (not the whole lot; a **lock** lets only one thread at a time run a piece of code, and Java's `synchronized` keyword is the built-in lock that every object carries), so payments for different tickets run in parallel while the portal and the exit panel cannot both charge the same ticket.
- `Main` uses a `TestClock` (a subclass of `java.time.Clock`) to jump forward in time, then races 20 threads for the last 5 car spots.

Output:

```
== Arrivals ==
[entry-A] CAR-1 -> ticket T1, spot F1-C01(COMPACT)
[entry-A] EV-1 -> ticket T2, spot F1-E01(ELECTRIC)
[entry-A] BIKE-1 -> ticket T3, spot F1-M01(MOTORCYCLE)
[entry-A] CAR-2 -> ticket T4, spot F1-H01(HANDICAPPED)
[entry-A] TRUCK-1 -> ticket T5, spot F1-L01(LARGE)
[entry-A] VAN-1 -> ticket T6, spot F2-L01(LARGE)
[entry-A] TRUCK-2 -> SORRY, LOT FULL for TRUCK
Floor 1 board: MOTORCYCLE=FULL COMPACT=1 LARGE=FULL HANDICAPPED=FULL ELECTRIC=FULL
Floor 2 board: COMPACT=3 LARGE=FULL

== EV charging at the spot ==
EV-1 charged 20 kWh, paid $6.00

== Departures ==
[exit-A] T1: paid $4.00 by cash $5.00, barrier OPEN
[portal] T2 due $16.00, charged $16.00
[exit-A] T2: already paid, barrier OPEN
Unpaid exit attempt: barrier opens? false

== Admin changes the rate: flat $5/hour ==
[exit-A] T3: paid $25.00 by card ****9999, barrier OPEN

== 20 cars race for the remaining CAR spots ==
free compact before: F1=2 F2=3, large: F1=0 F2=0
cars admitted: 5, distinct spots: 5
full for cars now? true
[entry-A] LATE-CAR -> SORRY, LOT FULL for CAR
Floor 2 board: COMPACT=FULL LARGE=FULL
```

Read the output: the truck is turned away because the only large spots are taken; T2 stayed 4 h 50 min, which rounds up to 5 hours = $4 + $3.50 + $3.50 + $2.50 + $2.50 = **$16.00**, paid at the portal, so the exit charges nothing; an unpaid ticket cannot open the barrier; after the admin switches to a flat $5/hour, the motorcycle pays $25 for 5 hours; 20 racing cars get exactly 5 distinct spots.

Concurrency & edge cases

Situation	What could go wrong	How the design handles it
Two gates want the last compact spot	Both get it (double booking)	`pollFirst()` is atomic; `tryOccupy` uses CAS as a second check
Portal and exit panel pay the same ticket at once	Customer charged twice	`synchronized (ticket)` around price-charge-mark; a PAID ticket costs 0
Same plate enters twice (cloned ticket, sensor glitch)	Two tickets for one car	`ticketByPlate.putIfAbsent`
Payment fails (card declined)	Ticket marked paid anyway	`method.pay` throws **before** `markPaid`; ticket stays ACTIVE
Car leaves without paying	Spot never freed or fee lost	`exit` only succeeds from PAID; the barrier stays closed
Customer pays at the portal and then stays 3 more hours	Free parking	Real lots give a **grace period** (e.g. 15 min) after payment; add `paidAt` and re-price after the grace period
Display board shows a slightly old number	Two notifications arrive out of order	Acceptable for a display; if exactness matters, have the board **pull** `freeCount` when it renders
Lost ticket	No entry time	Charge a daily maximum, or look up the plate (the plate map already exists)
Stay of 0 minutes	Fee of $0	Minimum of one hour (`Math.max(1, minutes)`)
Lock scope	One global lock would serialise all gates	Locks are per spot (CAS) and per ticket; no global lock

`ConcurrentSkipListSet.size()` is not constant-time (it walks the whole set instead of reading a stored number) and is only an estimate while other threads change the set; that is fine for a display board, but never use it to decide whether a spot is free — use `pollFirst()`.

Extensions & follow-up questions

1. **"What if pricing changes (weekend rate, per spot type, surge)?"** — Write a new `PricingStrategy` (for example `WeekendPricing`, or a decorator (a wrapper object with the same interface that adds behaviour around another object) that multiplies another strategy for large spots) and call `setPricing`. No other class changes.
2. **"Two cars arrive at the same instant — what happens?"** — Spot handout is atomic (`pollFirst` + CAS). Show the 20-thread race in the output.
3. **"Add reservations."** — Add a `Reservation(plate, spotType, from, to)` and a reserved set per floor; the allocation strategy checks the reservation first. Hold the spot with a timeout (release it if the car does not arrive within, say, 30 minutes).
4. **"Add monthly passes / season tickets."** — A `PricingStrategy` that returns 0 for plates with a valid pass, or a `PassHolderPricing` decorator around the normal pricing.
5. **"Number-plate recognition (ticketless)."** — The entrance camera becomes another kind of entrance panel that calls the same `admit`; the ticket id is keyed by plate. No core class changes.
6. **"How would you store this in a database?"** — Tables: `floor`, `spot(id, floor_id, type, occupied_by)`, `ticket(id, plate, spot_id, entry_time, status, paid_cents)`, `payment(id, ticket_id, method, amount)`. Claim a spot with a conditional update (`UPDATE spot SET occupied_by=? WHERE id=? AND occupied_by IS NULL`) — the database equivalent of our CAS. Candidates who have been asked this report that interviewers like to see **cardinalities** (how many of one thing can link to another; `1..*` means one or more) (a floor has 1..* spots, a spot holds 0..1 vehicles, a ticket has 1 spot) and then add a new entity to see how you rewire the model.
7. **"Many lots in a city?"** — A `ParkingLot` per site plus a `ParkingNetwork` that routes drivers to the lot with free spots. This is another reason not to make the lot a Singleton.
8. **"EV idle fees?"** — Add a charging session to `ChargingPanel` (start, end, kWh) and charge a per-minute fee after charging completes.

Real-world example

Many modern malls and airports use **parking guidance systems**: a sensor above each bay drives a small green/red light, and the boards at each entrance and floor show the free count per level — exactly our `DisplayBoard` observers fed by spot changes. Airports and many city garages increasingly use **automatic number-plate recognition** instead of paper tickets, and apps such as SpotHero and ParkMobile let drivers pre-book a spot, which is the "reservations" extension. Tesla Superchargers charge **idle fees** when a car stays plugged in after charging finishes — the EV extension above.

In the interview

Plan for 45 minutes

Minutes	What you do
0–5	Clarifying questions; write the 12 requirements and an out-of-scope list
5–10	Actors and use cases (take ticket, pay, exit, admin)
10–20	Stages 1–2: vehicle/spot fit rule, floors, board. Say "the fit rule lives on the vehicle"
20–25	Stage 3–4: ticket holds the spot; lot with admit/pay/exit; Strategy for pricing
25–38	Code `admit`, `tryPark`, `pay`, `exit`, and the tiered pricing
38–45	Concurrency (atomic spot handout, per-ticket lock), extensions

> **30-second answer:** "I'll model vehicles and spot types as enums with the fit rule on the vehicle, floors that keep a concurrent sorted set of free spots per type, and a lot that admits, prices and releases. Tickets point at the real spot. Pricing and spot-finding are Strategies so they can change without touching the lot. Handing out a spot is atomic — `pollFirst` plus a CAS on the spot — so two gates can never get the same spot, and payment locks only the one ticket."

Follow-ups with answers

- *"Why not a Singleton?"* — There is one lot in production, but a Singleton is a hidden global: it makes testing two lots impossible. I create one instance and inject it. If the interviewer insists, a lazy holder-class Singleton (the single instance lives in a nested class that Java loads only on first use, which the JVM makes thread-safe for free) is thread-safe and simple.
- *"Why is the fit rule on the vehicle?"* — Because it changes when vehicle types change; putting it there keeps the lot closed for modification (OCP).
- *"How do you make finding a spot fast?"* — Keep only the free spots, indexed by type, in a sorted set; taking one is O(log n) and needs no scan.
- *"What if the display board and the real count disagree?"* — The board is informational; the atomic `pollFirst` is the source of truth. The board can pull the count on render if needed.

Self-check

1. **Q:** Why must the ticket store a reference to the spot? **A:** So the exit can free exactly that spot; a count cannot tell you which spot to release.
2. **Q:** What makes spot assignment thread-safe here? **A:** `ConcurrentSkipListSet.pollFirst()` removes a spot for exactly one caller, and `AtomicReference.compareAndSet` on the spot double-checks it.
3. **Q:** Where would you add a weekend price? **A:** A new `PricingStrategy` implementation; set it with `setPricing`. Nothing else changes.
4. **Q:** Why did we replace the five spot subclasses with an enum? **A:** They had no behaviour of their own; an enum field does the same job and a new type is one constant (composition over inheritance).
5. **Q:** A customer paid at the portal. What does the exit panel charge? **A:** Nothing: `amountDue` returns 0 for a PAID ticket, and `exit` moves it from PAID to EXITED.
6. **Q:** How does the design stop a car from being admitted twice? **A:** `ticketByPlate.putIfAbsent(plate, …)` reserves the plate before allocating.

TL;DR

- Fit rule on the **vehicle**, spot type as an **enum**, free spots per type in a **concurrent sorted set**.
- **Ticket → spot** reference makes the exit possible; status goes ACTIVE → PAID → EXITED.
- **Strategy** for pricing and spot-finding, **Factory** for electric spots, **Observer** for boards; inject the lot instead of a Singleton.
- Concurrency: atomic `pollFirst` + CAS for spots, per-ticket lock for payment, `putIfAbsent` for plates.

14.2 Design a Vending Machine

Figure — A vending machine: slots with codes, a keypad, a coin/note slot and a tray at the bottom (course diagram)

A **vending machine** is an unattended shop: you key in a slot code, pay, and pull your item out of the tray at the bottom. Because everyone has used one, the interviewer can skip the domain and look at one thing only: **which actions are legal at which moment**. Taking a coin when nothing has been chosen is a bug; handing out two items for one payment is a bug. No amount of tidy code elsewhere makes up for getting that order wrong.

Problem & clarifying questions

Question	Assumed answer
Select first, or pay first?	Select a slot code first, then pay.
Which money?	A fixed set of coins and notes (5¢, 10¢, 25¢, $1, $5). Anything else is returned.
Card or contactless?	Out of scope (an extension).
Can it give change?	Yes. If it cannot make **exact** change, it refuses the sale and returns everything.
Can the customer cancel?	Any time before the product is dispensed, with a full refund.
One customer at a time?	Yes (one physical machine).
Who restocks?	An operator: restocks slots, loads coins for change, collects the takings.

Requirements

Functional (the course's twelve)

1. The machine has many **slots**. A slot has a code (`A1`, `B2`, …), holds items of a **single** product, and tracks its remaining **count**.
2. A product has a **name** and a **price in cents**.
3. Order of use: **choose a slot, then pay**.
4. Only a **known set of coins and notes** is accepted; any other piece is given back.
5. After **every** customer action the **display** says what to do next.
6. When the amount paid covers the price, the machine **releases the item and the change**.
7. When **exact change is impossible**, the sale is **refused** and every cent paid is **returned**.

8. Until the item drops, the customer may **cancel** and receive a **full refund**.
9. A slot whose count is **zero cannot be chosen**.
10. One payment buys **exactly one** item, never two.
11. An **operator** refills slots, adds coins for change, and removes the takings.
12. After every sale, cancel and refusal, **both** the item counts and the cash counts must be **exactly right**.

Non-functional

- **No illegal transitions**: every button must have a defined answer in every situation.
- **Exact money**: integer cents only — no floating-point.
- **Extensible**: a new payment device (card reader) or a new state (maintenance) should not require editing every existing class.

Use cases

Actors: Customer (selects, pays, cancels, takes the product), **Operator** (restocks, loads change, collects money), **System** (decides which actions are allowed now, works out change, shows messages).

Top use cases: select a product · insert money · dispense · collect (take from the tray) · cancel · restock a slot · load change and collect takings.

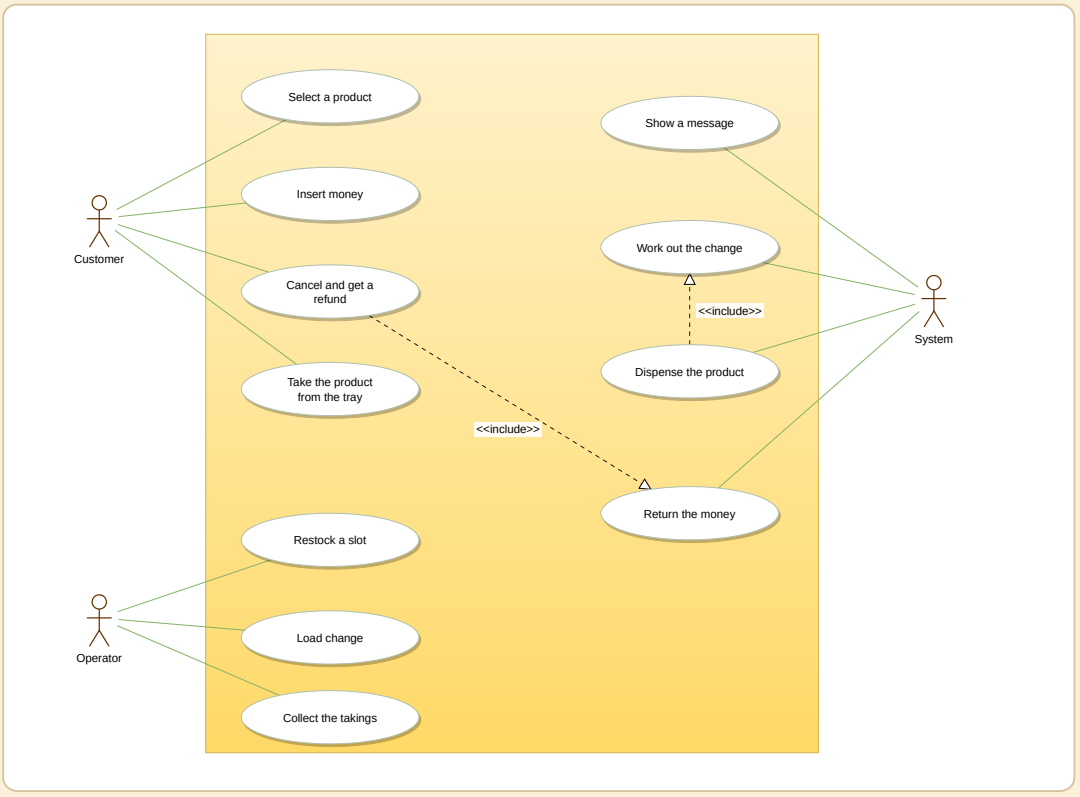


Figure — Use case diagram of the vending machine: Customer, Operator and System (course diagram)

In the figure above, "dispense" is a System use case, not a customer one: the customer only *causes* it by paying enough.

Core entities

Class	Responsibility
`Product` (record)	Name and price in cents. Describes *what* is sold.
`Slot`	One physical column: a code, one product, and the **count** left.
`Inventory`	All slots, looked up by code.
`Denomination` (enum)	The accepted coins and notes and their values in cents.
`CashBox`	Counts pieces by value; plans exact change; reports when it cannot.
`VendingState` (interface)	One method per customer button: `select`, `insert`, `cancel`, `take`.
`IdleState`, `SelectedState`, `HasMoneyState`, `DispensingState`, `OutOfStockState`	The situations the machine can be in (the course has the first four, calling `HasMoneyState` `PaidState`); each answers every button and names the next state.
`Display`	The message the customer currently sees. In our code, `VendingMachine.show(...)` plays this part.
`Keypad`, `CashSlot`, `Dispenser`	The hardware the customer touches; each calls the machine and holds no rules. In our code they are folded into the machine's public methods.
`Operator`	Restocks, loads change, collects takings. In our code: `restock`, `loadChange`, `collectTakings`.
`VendingMachine`	Holds the parts, the current state, the selection and the balance; forwards each button to the state. Also exposes operator actions.

Why is the count on the **slot**, not on the product? `Product` answers "what is it?"; `Slot` is the physical column it sits in. The same product may be loaded into two columns (the course even allows different prices), and emptying one column must leave the other untouched. That is the first real design decision in this question.

Class diagram

The course builds the machine in five stages:

Stage	What it adds	What works after it
1	`Product`, `Slot`, `Inventory`	Look up a code and learn what it holds and whether it is empty
2	`Coin`, `Note`, `CashBox`	Decide whether a given amount of change can be paid exactly
3	A class for each state	Each button press gets a defined reaction in each situation
4	`Display` and `VendingMachine`	A complete sale, from choosing to picking up the item
5	`Keypad`, `CashSlot`, `Dispenser`, `Operator`	Real hardware drives the same flow

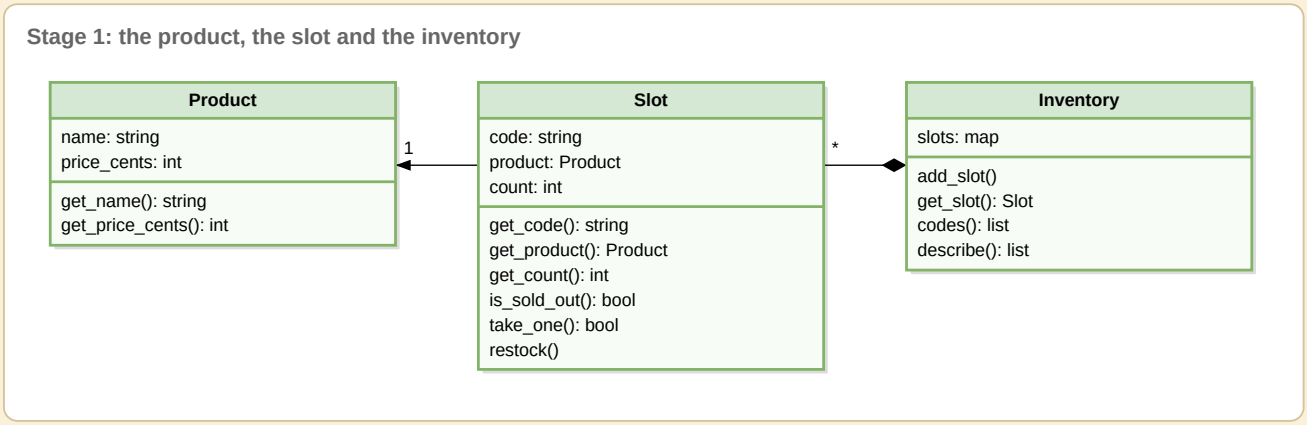


Figure — Stage 1: Product, Slot (association to Product) and Inventory (composition of Slots) (course diagram)

Stage 1. Requirements 1, 2 and 9. `Slot → Product` is an **association**: a product can appear in several slots, so removing a slot must leave the product alone. `Inventory ◆ Slot` is **composition** (filled diamond): slots are parts of the machine and do not outlive it.

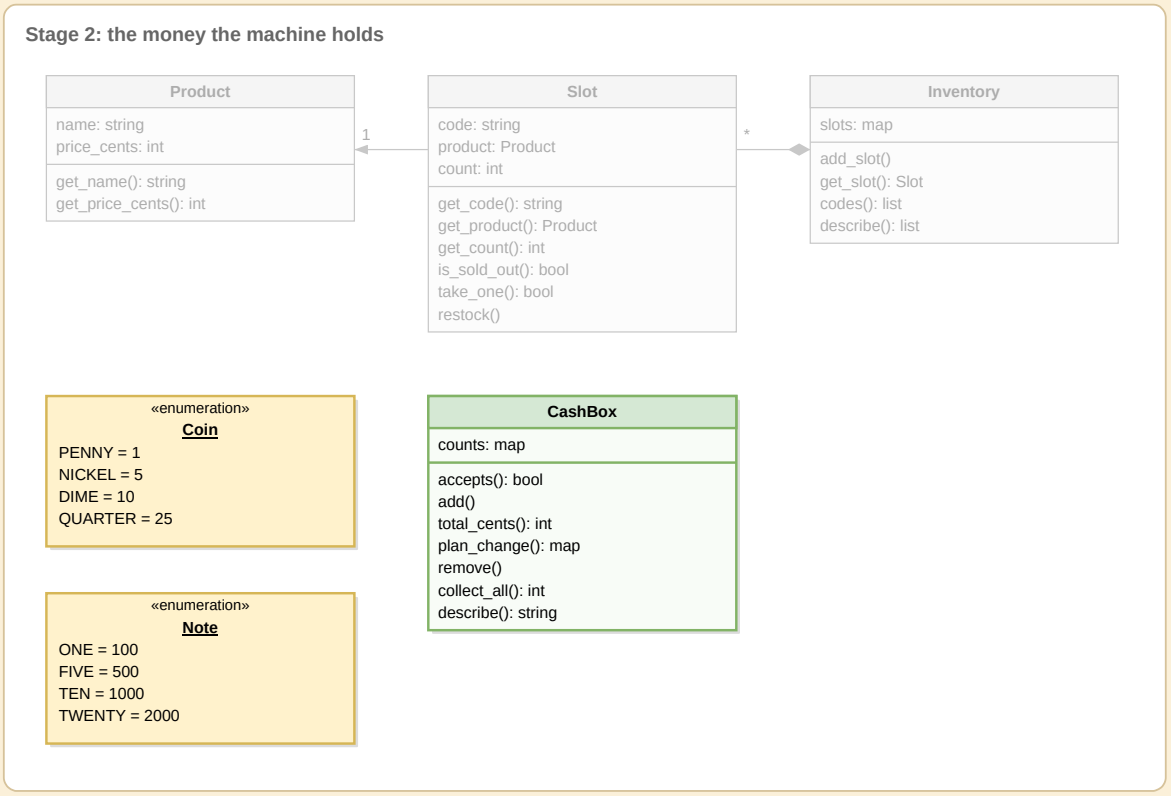


Figure — Stage 2: Coin and Note enumerations and the CashBox that counts pieces by value (course diagram)

Stage 2. Requirements 4, 6 and 7. All amounts are whole cents, so rounding errors cannot happen. `Coin` and `Note` are enumerations (fixed lists of named values; our code merges them into one `Denomination` enum), and fixed sets like these have no relationship arrows. The choice to defend here: **a piece goes into the cash box the moment it is inserted**, not into a holding area. Those pieces are then available for change, and a refund becomes nothing more than "pay out exact change equal to what was inserted". The box stores a **count per value**; which

physical quarter comes back does not matter. Its change planner either returns an exact plan or says "impossible"; it never approximates, and that is what makes requirement 7 implementable.

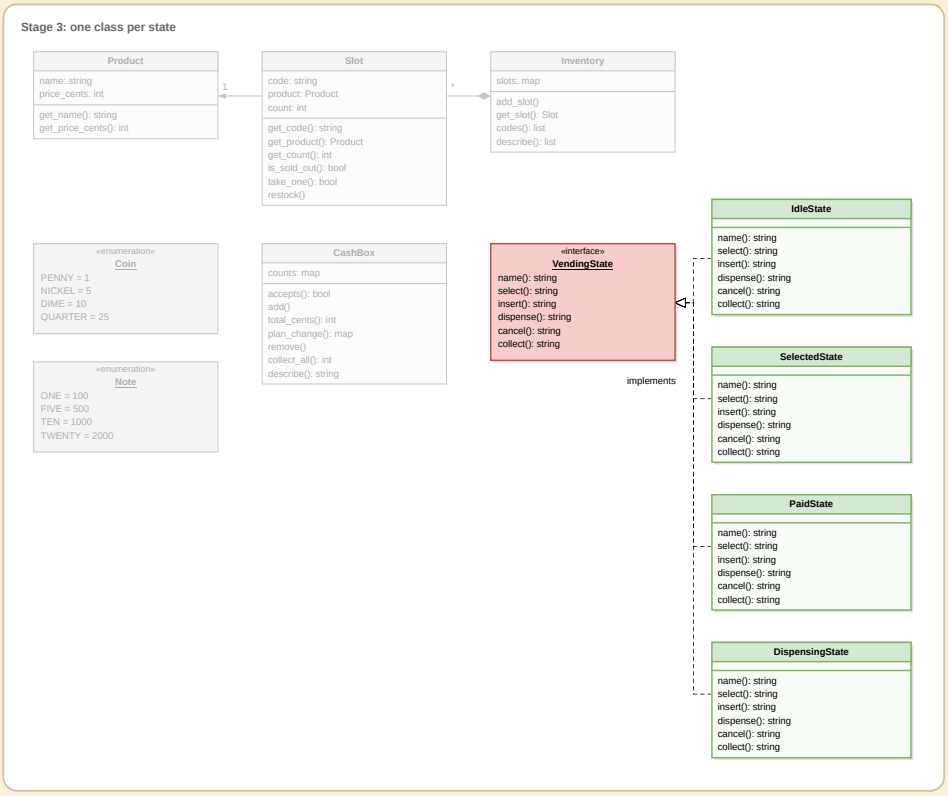


Figure — Stage 3: one class per state, all implementing the VendingState interface (course diagram)

Stage 3 — the heart of the question. Requirements 3, 8 and 10. The obvious first attempt is one class with a few boolean flags and nested `if`s; that version is exactly where double dispensing and swallowed coins come from. Use the **State pattern** instead: each situation becomes a class, and all of them implement one interface on which **each customer button is a method**. A state class that forgets a button does not compile. Picture the states as a **grid**, buttons down the side and situations across the top: the design is finished when every cell holds a response. The state classes carry no fields, and none of them checks which state is active; each handles its own buttons and hands over to the next state, so a new state can be added without editing the existing ones.

The course draws **four** states: `IdleState`, `SelectedState`, `PaidState` and `DispensingState`. Our code keeps those four (its `HasMoneyState` is the course's `PaidState`: some money is in, not yet enough) and adds a fifth, `OutOfStockState`, so that an empty machine refuses every customer button until an operator restocks it.

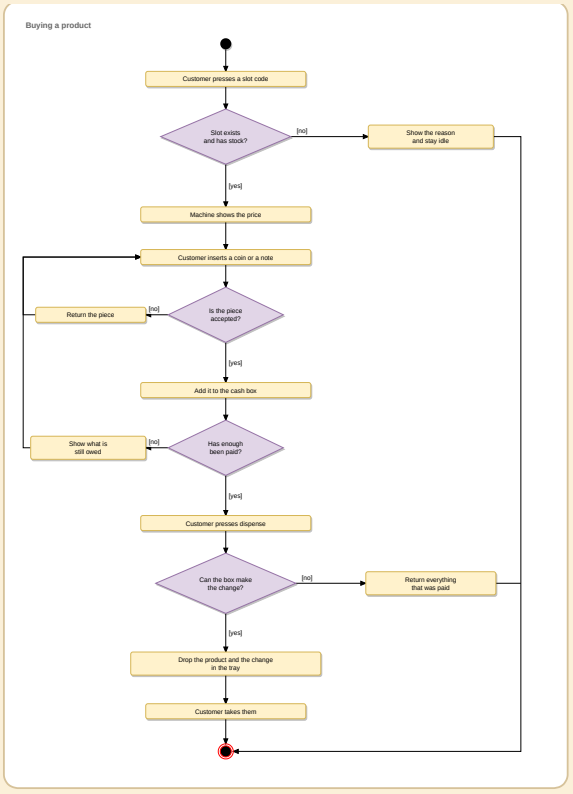


Figure — Activity diagram "Buying a product": select, check stock, insert money until enough, check change, dispense, take (course diagram)

Each decision diamond in the activity diagram above corresponds to one state choosing whether to accept an action.

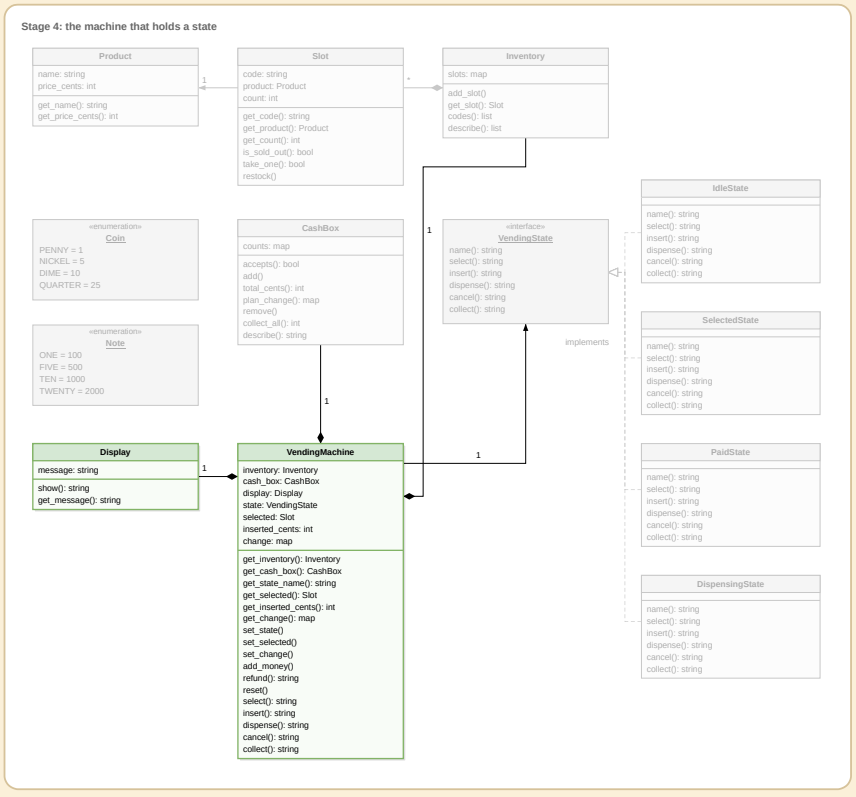


Figure — Stage 4: VendingMachine owns its display, cash box and inventory, and points at the current VendingState (course diagram)

Stage 4. Requirements 5, 6, 8 and 12. `VendingMachine` remembers the chosen slot and the money inserted so far, and passes every button press to its current state. It **owns** its `Display`, `CashBox` and `Inventory` (composition) and merely **refers to** the current state object (association). Look at what the class does *not* contain: any `if`

that tests which state is active. The refund method shows why stage 2 pays off: since inserted money is already in the box, refunding is just a request for exact change equal to the balance.

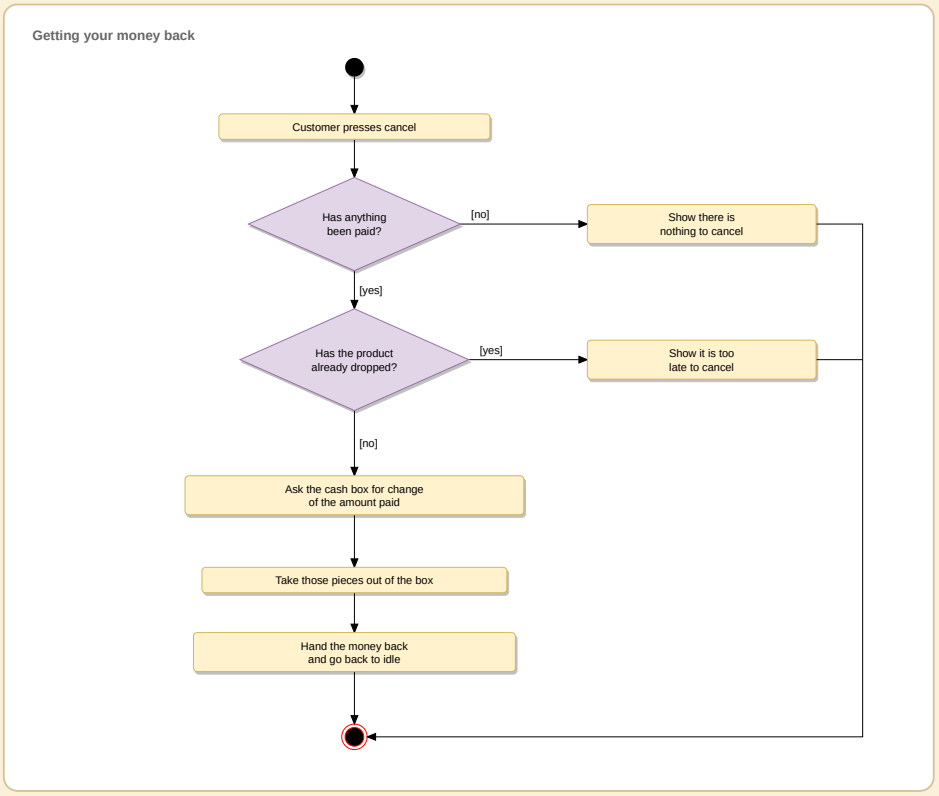


Figure — Activity diagram "Getting your money back": cancel and "cannot make change" end in the same refund path (course diagram)

As the figure above shows, a cancel and a "no exact change" refusal finish identically, so the design needs a **single** refund method.

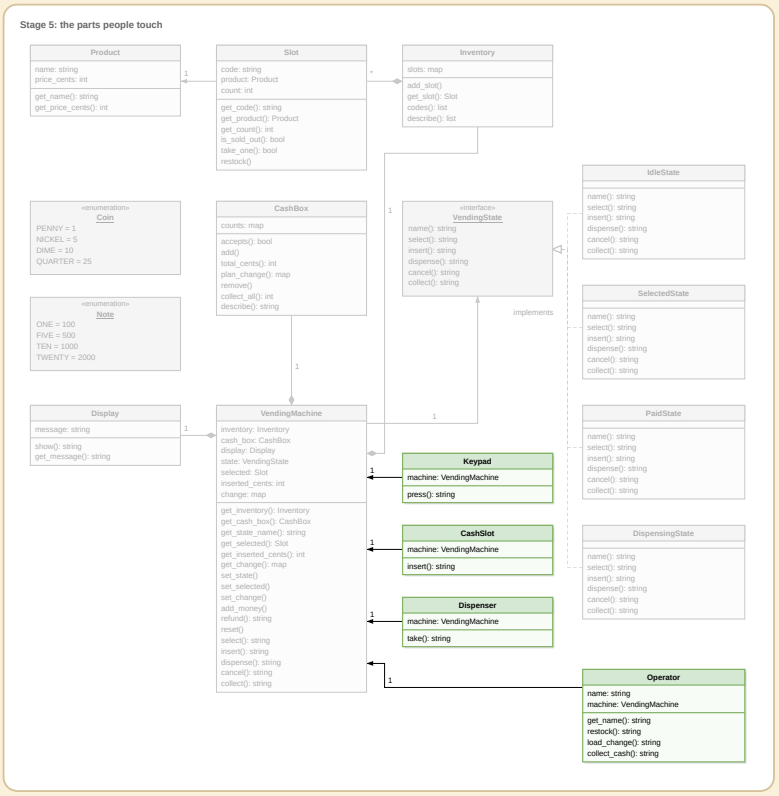


Figure — Stage 5: Keypad, CashSlot, Dispenser and Operator, all pointing at the machine (course diagram)

Stage 5. Requirements 3, 4 and 11. All four new classes hold references **to** the machine, never the reverse: the keypad calls the machine, but the machine has no idea a keypad exists. Swapping the keypad for a touch screen, or the coin slot for a card reader, therefore touches no other class. The `Dispenser` is the tray; pulling the item out of it is what finishes a sale, which is why `take` is a customer button on the state interface.

`Operator` is intentionally kept **off** that interface: refilling, loading change and emptying the box are not customer actions and are permitted whatever state the machine is in (if they were on the interface, every state class would have to answer them).

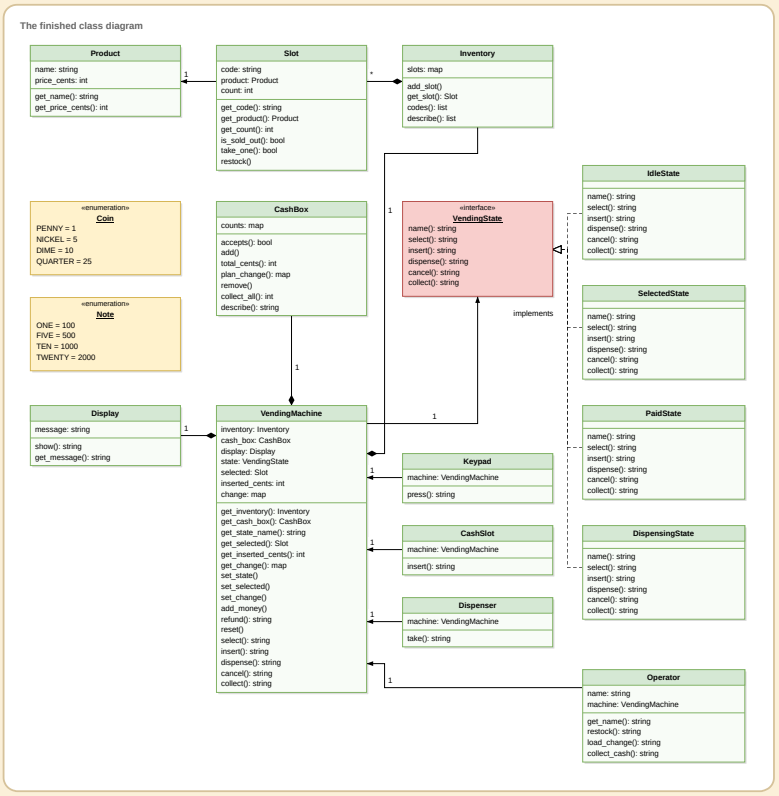
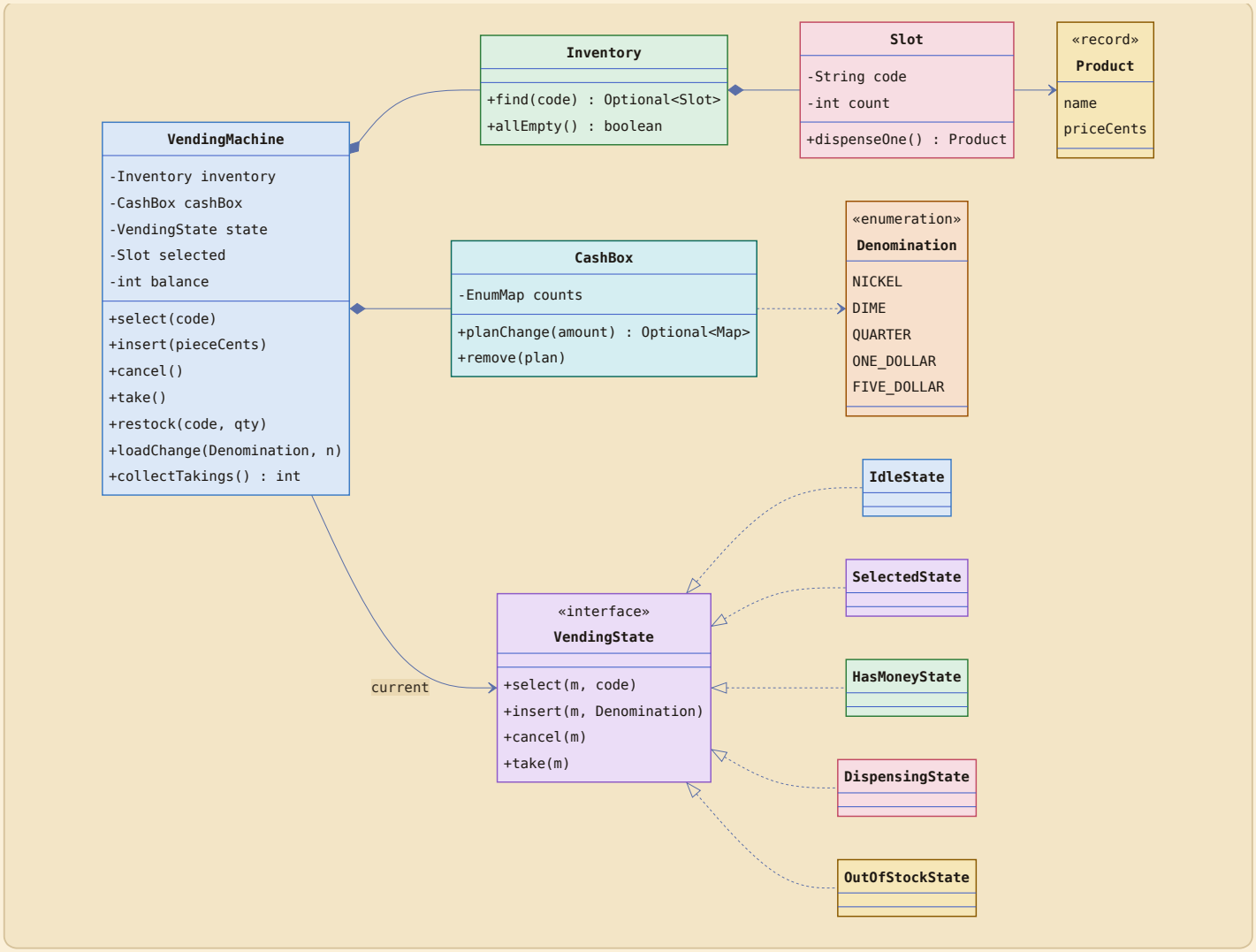


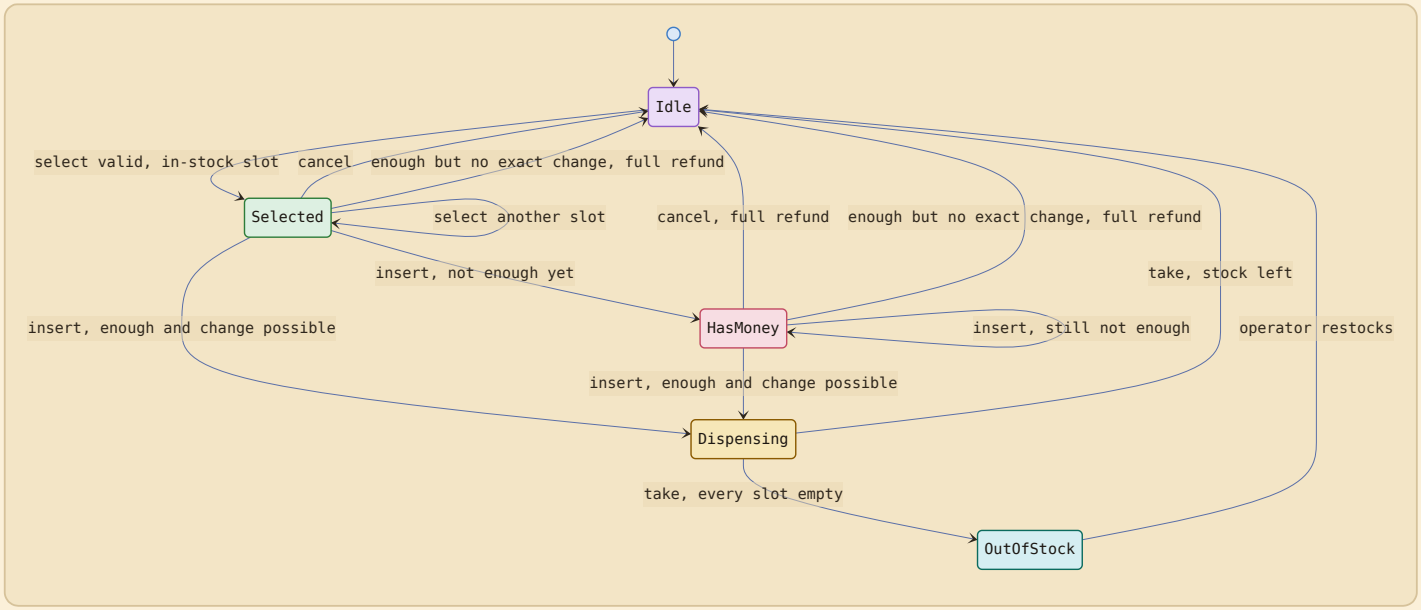
Figure — The finished class diagram of the vending machine (course diagram)

Class diagram of **our Java code** (hardware classes folded into the machine's public methods to keep the code short):



Key flows

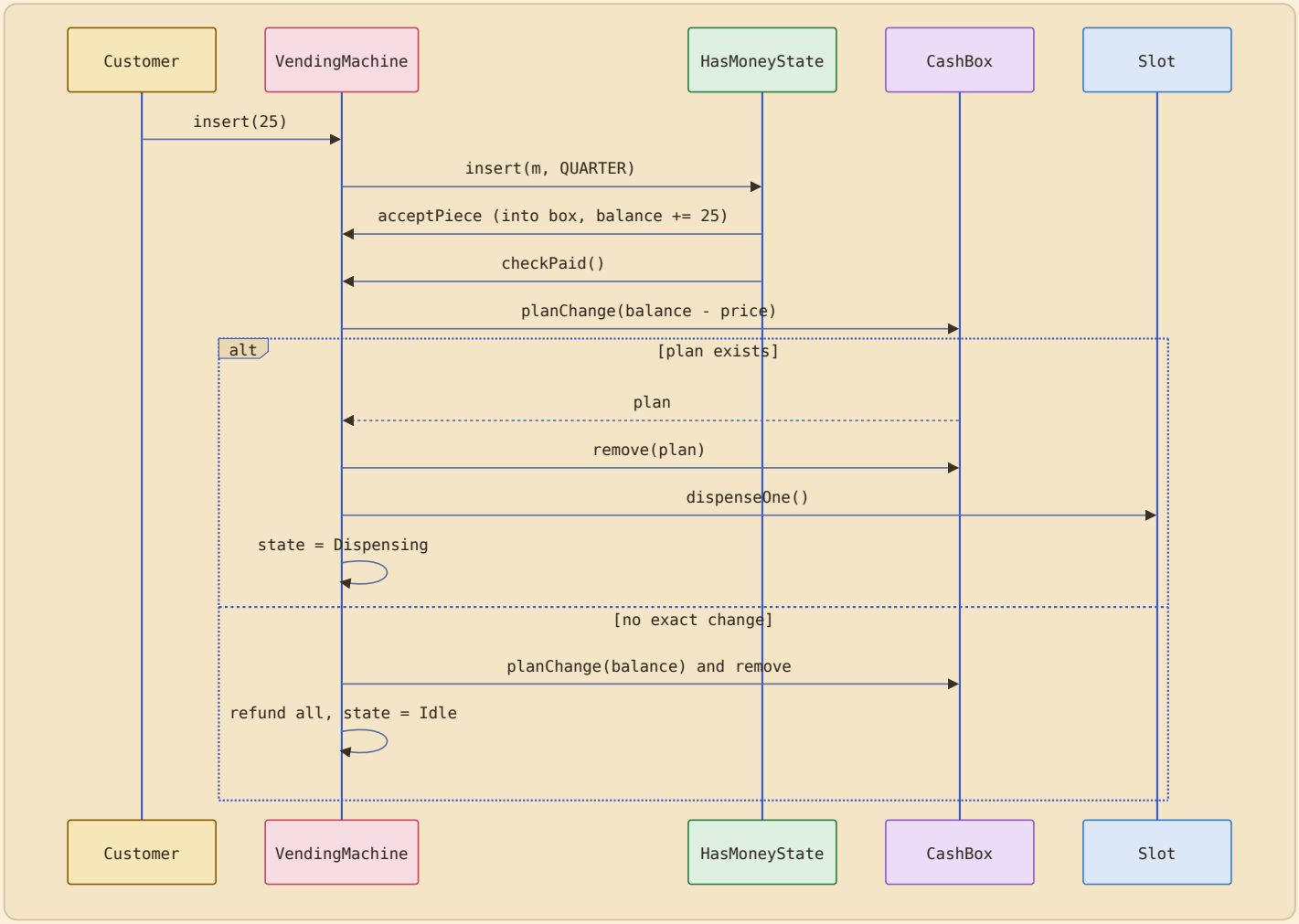
State machine. This diagram *is* the design; draw it before writing code.



The state table (rows = buttons, columns = states). Every cell has an answer:

Button ↓ / State →	Idle	Selected	HasMoney	Dispensing	OutOfStock
`select`	choose slot (reject empty/unknown)	change choice	"cancel first"	"take your product first"	"out of stock"
`insert`	return piece	accept; check paid	accept; check paid	return piece	return piece
`cancel`	"nothing to cancel"	clear choice → Idle	**refund all** → Idle	"too late"	"nothing to cancel"
`take`	"tray empty"	"pay first"	"pay the rest"	give product + change → Idle/OutOfStock	"tray empty"

Paying enough.



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
One class per situation behind `VendingState`	**State**	Every button × situation cell is answered; the compiler enforces completeness; no `if (state == …)`	Flags + `if/else` chains (the classic source of double-dispense bugs)
States are shared **stateless singletons** (`INSTANCE`; no fields, so one copy serves everyone)	Flyweight-style sharing (one shared object instead of many identical ones)	The course notes that creating a new state object on each transition is readable but wasteful; the states hold no data, so one instance each is enough	`new IdleState()` on every transition
Operator actions are **not** on the state interface	Interface Segregation (no class is forced to implement methods it does not need)	Restocking is allowed in every state; forcing five classes to answer it would be noise	Adding `restock` to `VendingState`
Money counted by value in an `EnumMap` (a fast map keyed by enum constants)	—	The machine never cares which physical quarter it returns	A list of coin objects
Inserted money goes straight into the box	—	Refund = "make change for the balance"; one refund path	A separate escrow pile (a holding area for inserted money)
Exact change with dynamic programming	Correctness	Greedy can fail when the box is short of one coin (see output part 6)	Greedy only
Refuse the sale if exact change is impossible	Requirement 7	Never short-change the customer	Give "close enough" change

> **About the change algorithm.** The course's planner is **greedy** (largest piece first, never looks back) and honestly says so: greedy is correct for standard coin sets when the box is well stocked, but when the box is short of one piece it can fail for an amount that smaller pieces could make. The exact fix is **dynamic programming** — the "coin change with limited coins" problem, where you build the answer for each amount from the answers for smaller amounts. Our `CashBox.planChange` implements that (bounded coin change, fewest pieces), and `greedyPlan` is kept only to show the difference: for 30¢ from {1 quarter, 3 dimes}, greedy takes the quarter and gets stuck at 5¢; DP returns three dimes.

Java implementation

Package `lld.ch14.vendingmachine`.

```java
// File: lld/ch14/vendingmachine/Inventory.java
package lld.ch14.vendingmachine;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Optional;

/** What is sold: a name and a price in cents. */
record Product(String name, int priceCents) {}

/** One physical column. The COUNT lives on the slot, not on the product. */
final class Slot {
    private final String code;
    private final Product product;
    private int count;

    Slot(String code, Product product, int count) {
        this.code = code;
        this.product = product;
        this.count = count;
    }

    boolean isEmpty() { return count == 0; }

    Product dispenseOne() {
        if (count == 0) throw new IllegalStateException(code + " is empty");
        count--;
        return product;
    }

    void restock(int qty) { count += qty; }
    String code() { return code; }
    Product product() { return product; }
    int count() { return count; }
}

/** All slots, looked up by code. */
final class Inventory {
    private final Map<String, Slot> slots = new LinkedHashMap<>();

    void addSlot(Slot s) { slots.put(s.code(), s); }
    Optional<Slot> find(String code) { return Optional.ofNullable(slots.get(code)); }
    boolean allEmpty() { return slots.values().stream().allMatch(Slot::isEmpty); }

    String summary() {
        StringBuilder sb = new StringBuilder();
        slots.values().forEach(s -> sb.append(s.code()).append(':').append(s.product().name())
                .append('x').append(s.count()).append(' '));
        return sb.toString().trim();
    }
}
```

```java
// File: lld/ch14/vendingmachine/CashBox.java
package lld.ch14.vendingmachine;

import java.util.Arrays;
import java.util.EnumMap;
import java.util.Map;
import java.util.Optional;

/** The fixed set of pieces the machine accepts, in cents. Anything else is returned. */
enum Denomination {
    NICKEL(5, false), DIME(10, false), QUARTER(25, false), ONE_DOLLAR(100, true), FIVE_DOLLAR(500, true);

    final int cents;
    final boolean note;

    Denomination(int cents, boolean note) { this.cents = cents; this.note = note; }

    static Optional<Denomination> of(int cents) {
        return Arrays.stream(values()).filter(d -> d.cents == cents).findFirst();
    }
}

/** Counts pieces by value. Inserted money goes in straight away, so refunds are just "make change". */
final class CashBox {
    private final EnumMap<Denomination, Integer> counts = new EnumMap<>(Denomination.class);

    CashBox() { for (Denomination d : Denomination.values()) counts.put(d, 0); }

    void add(Denomination d, int n) { counts.merge(d, n, Integer::sum); }

    /**
     * Exact change with the FEWEST pieces, respecting how many of each piece we hold
     * (bounded coin change, dynamic programming). Empty = cannot be made exactly.
     */
    Optional<Map<Denomination, Integer>> planChange(int amount) {
        Denomination[] ds = Denomination.values();
        int[][] best = new int[amount + 1][];   // best[a] = pieces used per denomination to make a
        best[0] = new int[ds.length];
        for (int i = 0; i < ds.length; i++) {
            int v = ds[i].cents;
            int copies = Math.min(counts.get(ds[i]), amount / v);
            for (int c = 0; c < copies; c++) {                // each copy may be used once (0/1)
                for (int a = amount; a >= v; a--) {
                    if (best[a - v] == null) continue;
                    int pieces = sum(best[a - v]) + 1;
                    if (best[a] == null || pieces < sum(best[a])) {
                        best[a] = best[a - v].clone();
                        best[a][i]++;
                    }
                }
            }
        }
        if (best[amount] == null) return Optional.empty();
        Map<Denomination, Integer> plan = new EnumMap<>(Denomination.class);
        for (int i = 0; i < ds.length; i++) if (best[amount][i] > 0) plan.put(ds[i], best[amount][i]);
        return Optional.of(plan);
    }

    /** Greedy (largest piece first) — shown only to compare with planChange. */
    Optional<Map<Denomination, Integer>> greedyPlan(int amount) {
        Map<Denomination, Integer> plan = new EnumMap<>(Denomination.class);
        Denomination[] ds = Denomination.values();
        for (int i = ds.length - 1; i >= 0 && amount > 0; i--) {
            int take = Math.min(counts.get(ds[i]), amount / ds[i].cents);
            if (take > 0) { plan.put(ds[i], take); amount -= take * ds[i].cents; }
        }
        return amount == 0 ? Optional.of(plan) : Optional.empty();
    }
```

```
    void remove(Map<Denomination, Integer> plan) {
        plan.forEach((d, n) -> counts.merge(d, -n, Integer::sum));
    }

    /** Operator empties the notes; coins stay behind as the float for change. */
    int collectNotes() {
        int total = 0;
        for (Denomination d : Denomination.values()) {
            if (d.note) { total += counts.get(d) * d.cents; counts.put(d, 0); }
        }
        return total;
    }

    int total() {
        return counts.entrySet().stream().mapToInt(e -> e.getKey().cents * e.getValue()).sum();
    }

    private static int sum(int[] a) { return Arrays.stream(a).sum(); }
}
```

```java
// File: lld/ch14/vendingmachine/States.java
package lld.ch14.vendingmachine;

/** State pattern: every button is on the interface, so every state must answer every button. */
interface VendingState {
    void select(VendingMachine m, String code);
    void insert(VendingMachine m, Denomination d);
    void cancel(VendingMachine m);
    void take(VendingMachine m);
    String name();
}

/** Waiting for a customer. */
final class IdleState implements VendingState {
    static final IdleState INSTANCE = new IdleState();   // states hold no data, so share one instance

    @Override public void select(VendingMachine m, String code) {
        m.inventory().find(code).ifPresentOrElse(slot -> {
            if (slot.isEmpty()) { m.show(code + " is sold out. Choose another."); return; }
            m.choose(slot);
            m.show("%s costs %d c. Insert money.".formatted(slot.product().name(), slot.product().priceCents()));
            m.setState(SelectedState.INSTANCE);
        }, () -> m.show("Unknown code " + code));
    }
    @Override public void insert(VendingMachine m, Denomination d) { m.rejectPiece(d, "select a product first"); }
    @Override public void cancel(VendingMachine m) { m.show("Nothing to cancel."); }
    @Override public void take(VendingMachine m) { m.show("The tray is empty."); }
    @Override public String name() { return "IDLE"; }
}

/** A product is chosen, no money yet. The customer may still change their mind freely. */
final class SelectedState implements VendingState {
    static final SelectedState INSTANCE = new SelectedState();

    @Override public void select(VendingMachine m, String code) { IdleState.INSTANCE.select(m, code); }
    @Override public void insert(VendingMachine m, Denomination d) { m.acceptPiece(d); m.checkPaid(); }
    @Override public void cancel(VendingMachine m) {
        m.clearSelection();
        m.show("Cancelled.");
        m.setState(IdleState.INSTANCE);
    }
    @Override public void take(VendingMachine m) { m.show("Please pay first."); }
    @Override public String name() { return "SELECTED"; }
}

/** Some money is in, but not enough yet. */
final class HasMoneyState implements VendingState {
    static final HasMoneyState INSTANCE = new HasMoneyState();

    @Override public void select(VendingMachine m, String code) { m.show("Cancel first to change your choice."); }
    @Override public void insert(VendingMachine m, Denomination d) { m.acceptPiece(d); m.checkPaid(); }
    @Override public void cancel(VendingMachine m) { m.refundAll("Cancelled."); }
    @Override public void take(VendingMachine m) { m.show("Please pay the rest first."); }
    @Override public String name() { return "HAS_MONEY"; }
}

/** Product and change are in the tray. Nothing else is allowed until the customer takes them. */
final class DispensingState implements VendingState {
    static final DispensingState INSTANCE = new DispensingState();

    @Override public void select(VendingMachine m, String code) { m.show("Please take your product first."); }
    @Override public void insert(VendingMachine m, Denomination d) { m.rejectPiece(d, "take your product first"); }
    @Override public void cancel(VendingMachine m) { m.show("Too late to cancel: product already dispensed."); }
    @Override public void take(VendingMachine m) { m.emptyTray(); }
    @Override public String name() { return "DISPENSING"; }
}

/** Every slot is empty. Only an operator restock leaves this state. */
```

```
final class OutOfStockState implements VendingState {
    static final OutOfStockState INSTANCE = new OutOfStockState();

    @Override public void select(VendingMachine m, String code) { m.show("Machine is out of stock."); }
    @Override public void insert(VendingMachine m, Denomination d) { m.rejectPiece(d, "out of stock"); }
    @Override public void cancel(VendingMachine m) { m.show("Nothing to cancel."); }
    @Override public void take(VendingMachine m) { m.show("The tray is empty."); }
    @Override public String name() { return "OUT_OF_STOCK"; }
}
```

```java
// File: lld/ch14/vendingmachine/VendingMachine.java
package lld.ch14.vendingmachine;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * Holds the parts and the current state and forwards every button to the state.
 * There is no "if (state == ...)" anywhere in this class.
 * Public methods are synchronized: one physical machine, but a networked
 * controller or a second input device must not interleave two actions.
 */
final class VendingMachine {
    private final Inventory inventory;
    private final CashBox cashBox;
    private VendingState state;
    private Slot selected;
    private int balance;                                    // cents inserted in this sale
    private final List<String> tray = new ArrayList<>();

    VendingMachine(Inventory inventory, CashBox cashBox) {
        this.inventory = inventory;
        this.cashBox = cashBox;
        this.state = inventory.allEmpty() ? OutOfStockState.INSTANCE : IdleState.INSTANCE;
    }

    // ----- buttons the customer presses -----
    synchronized void select(String code) { log("select " + code); state.select(this, code); }
    synchronized void cancel() { log("cancel"); state.cancel(this); }
    synchronized void take() { log("take"); state.take(this); }

    synchronized void insert(int pieceCents) {
        log("insert " + pieceCents + "c");
        Denomination.of(pieceCents).ifPresentOrElse(
                d -> state.insert(this, d),
                () -> show("Unknown piece of " + pieceCents + "c returned."));
    }

    // ----- operator actions: allowed in every state, so NOT on the state interface -----
    synchronized void restock(String code, int qty) {
        inventory.find(code).orElseThrow().restock(qty);
        if (state == OutOfStockState.INSTANCE) setState(IdleState.INSTANCE);
        show("Operator restocked " + code + " +" + qty);
    }
    synchronized void loadChange(Denomination d, int n) { cashBox.add(d, n); }
    synchronized int collectTakings() { return cashBox.collectNotes(); }

    // ----- helpers used by the states -----
    void setState(VendingState s) { this.state = s; }
    Inventory inventory() { return inventory; }
    void choose(Slot s) { this.selected = s; }
    void clearSelection() { this.selected = null; }
    void show(String msg) { System.out.println("   [display] " + msg); }

    void acceptPiece(Denomination d) {
        cashBox.add(d, 1);
        balance += d.cents;
    }

    void rejectPiece(Denomination d, String why) { show(d + " returned: " + why + "."); }

    /** Called after every accepted piece in SELECTED / HAS_MONEY. */
    void checkPaid() {
        int price = selected.product().priceCents();
        if (balance < price) {
            show("Paid %d c, %d c to go.".formatted(balance, price - balance));
            setState(HasMoneyState.INSTANCE);
```

```
            return;
        }
        var change = cashBox.planChange(balance - price);
        if (change.isEmpty()) {                                 // refuse the sale rather than short-change
            refundAll("Cannot make exact change.");
            return;
        }
        cashBox.remove(change.get());
        tray.add(selected.dispenseOne().name());
        if (!change.get().isEmpty()) tray.add("change " + change.get());
        show("Dispensed. Please take: " + tray);
        balance = 0;
        selected = null;
        setState(DispensingState.INSTANCE);
    }

    /** One refund path for "cancel" and for "cannot make change". */
    void refundAll(String reason) {
        Map<Denomination, Integer> back = cashBox.planChange(balance)
                .orElseThrow(() -> new IllegalStateException("inserted pieces must be refundable"));
        cashBox.remove(back);
        show(reason + " Refunded " + balance + " c as " + back);
        balance = 0;
        selected = null;
        setState(IdleState.INSTANCE);
    }

    void emptyTray() {
        show("Customer took " + tray);
        tray.clear();
        setState(inventory.allEmpty() ? OutOfStockState.INSTANCE : IdleState.INSTANCE);
    }

    synchronized String status() {
        return "state=%s cash=%dc stock=[%s]".formatted(state.name(), cashBox.total(), inventory.summary());
    }

    private void log(String action) { System.out.println(" > " + action + "   (" + state.name() + ")"); }
}
```

```java
// File: lld/ch14/vendingmachine/Main.java
package lld.ch14.vendingmachine;

public class Main {
    public static void main(String[] args) {
        Inventory inv = new Inventory();
        inv.addSlot(new Slot("A1", new Product("Cola", 125), 2));
        inv.addSlot(new Slot("B1", new Product("Chips", 50), 1));
        inv.addSlot(new Slot("C1", new Product("Candy", 65), 0));
        CashBox box = new CashBox();
        VendingMachine vm = new VendingMachine(inv, box);
        vm.loadChange(Denomination.QUARTER, 2);
        vm.loadChange(Denomination.DIME, 1);
        System.out.println(vm.status());

        System.out.println("\n1) Happy path: Cola with $1 + 25c. Extra coins and a late cancel are refused");
        vm.select("A1");
        vm.insert(100);
        vm.insert(25);
        vm.insert(25);
        vm.insert(10);          // ignored while the product waits in the tray
        vm.cancel();            // too late
        vm.take();

        System.out.println("\n2) Wrong order and bad input");
        vm.insert(25);          // no selection yet
        vm.select("C1");        // sold out
        vm.select("Z9");        // unknown
        vm.select("B1");
        vm.insert(3);           // foreign coin

        System.out.println("\n3) Cancel with money in: full refund");
        vm.insert(25);
        vm.select("A1");        // must cancel first
        vm.cancel();

        System.out.println("\n4) Cannot make exact change -> sale refused, everything refunded");
        vm.select("B1");
        vm.insert(500);         // needs 450 c change, box can't make it
        System.out.println(vm.status());

        System.out.println("\n5) Last items sold -> OUT_OF_STOCK -> operator restocks");
        vm.select("B1");
        vm.insert(25);
        vm.insert(25);
        vm.take();
        vm.select("A1");
        vm.insert(100);
        vm.insert(25);
        vm.take();
        System.out.println(vm.status());
        vm.select("A1");
        vm.restock("C1", 5);
        vm.select("C1");
        vm.insert(100);         // 35c change: 25 + 10
        vm.take();

        System.out.println("\n6) Why the change planner is not greedy");
        CashBox demo = new CashBox();
        demo.add(Denomination.QUARTER, 1);
        demo.add(Denomination.DIME, 3);
        System.out.println("30c from {1 quarter, 3 dimes}: greedy=" + demo.greedyPlan(30)
                + " dp=" + demo.planChange(30));

        System.out.println("\nOperator collects notes: " + vm.collectTakings() + " c");
        System.out.println(vm.status());
```

```
    }
}
```

How the code works, part by part

- `VendingMachine.insert(int)` first maps the raw piece to a `Denomination`; an unknown piece (a 3¢ token) is returned without ever reaching a state.
- `SelectedState` and `HasMoneyState` both call `acceptPiece` then `checkPaid`. `checkPaid` either moves to `HasMoney`, or completes the sale (plan change → remove pieces → dispense one → `Dispensing`), or refunds everything if exact change is impossible.
- `refundAll` is the **single** refund path used by cancel and by "cannot make change". It can never fail, because the pieces just inserted are in the box, so an exact plan always exists.
- `DispensingState.take` empties the tray and moves to `Idle` or `OutOfStock` — the sale ends when the customer takes the product, which is why `take` is a customer button.
- All public methods are `synchronized`. A single physical machine has one customer, but a networked controller (remote price change, telemetry — status data sent automatically to a remote server — or a second input device) must not interleave two actions.

Output:

```
state=IDLE cash=60c stock=[A1:Colax2 B1:Chipsx1 C1:Candyx0]

1) Happy path: Cola with $1 + 25c. Extra coins and a late cancel are refused
 > select A1   (IDLE)
   [display] Cola costs 125 c. Insert money.
 > insert 100c   (SELECTED)
   [display] Paid 100 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Cola]
 > insert 25c   (DISPENSING)
   [display] QUARTER returned: take your product first.
 > insert 10c   (DISPENSING)
   [display] DIME returned: take your product first.
 > cancel   (DISPENSING)
   [display] Too late to cancel: product already dispensed.
 > take   (DISPENSING)
   [display] Customer took [Cola]

2) Wrong order and bad input
 > insert 25c   (IDLE)
   [display] QUARTER returned: select a product first.
 > select C1   (IDLE)
   [display] C1 is sold out. Choose another.
 > select Z9   (IDLE)
   [display] Unknown code Z9
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 3c   (SELECTED)
   [display] Unknown piece of 3c returned.

3) Cancel with money in: full refund
 > insert 25c   (SELECTED)
   [display] Paid 25 c, 25 c to go.
 > select A1   (HAS_MONEY)
   [display] Cancel first to change your choice.
 > cancel   (HAS_MONEY)
   [display] Cancelled. Refunded 25 c as {QUARTER=1}

4) Cannot make exact change -> sale refused, everything refunded
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 500c   (SELECTED)
   [display] Cannot make exact change. Refunded 500 c as {FIVE_DOLLAR=1}
state=IDLE cash=185c stock=[A1:Colax1 B1:Chipsx1 C1:Candyx0]

5) Last items sold -> OUT_OF_STOCK -> operator restocks
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 25c   (SELECTED)
   [display] Paid 25 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Chips]
 > take   (DISPENSING)
   [display] Customer took [Chips]
 > select A1   (IDLE)
   [display] Cola costs 125 c. Insert money.
 > insert 100c   (SELECTED)
   [display] Paid 100 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Cola]
 > take   (DISPENSING)
   [display] Customer took [Cola]
state=OUT_OF_STOCK cash=360c stock=[A1:Colax0 B1:Chipsx0 C1:Candyx0]
 > select A1   (OUT_OF_STOCK)
   [display] Machine is out of stock.
   [display] Operator restocked C1 +5
 > select C1   (IDLE)
   [display] Candy costs 65 c. Insert money.
```

```
  > insert 100c   (SELECTED)
    [display] Dispensed. Please take: [Candy, change {DIME=1, QUARTER=1}]
  > take   (DISPENSING)
    [display] Customer took [Candy, change {DIME=1, QUARTER=1}]

6) Why the change planner is not greedy
30c from {1 quarter, 3 dimes}: greedy=Optional.empty dp=Optional[{DIME=3}]

Operator collects notes: 300 c
state=IDLE cash=125c stock=[A1:Colax0 B1:Chipsx0 C1:Candyx4]
```

Read the output: in part 1 the cola is paid exactly, and coins inserted while the product waits are returned, and a late cancel is refused; part 4 shows the refusal when 450¢ of change cannot be made from the box (so the $5 note comes straight back); part 5 shows the machine going to `OUT_OF_STOCK` and back to `IDLE` after a restock; part 6 shows greedy failing where DP succeeds.

Concurrency & edge cases

Edge case	Handling
Money inserted before selecting	`IdleState.insert` returns the piece
Unknown coin or foreign note	Returned before reaching the state
Selecting an empty slot	`IdleState.select` refuses; the state stays Idle
Changing the choice after paying	`HasMoneyState.select` says "cancel first" (prevents price confusion)
Paying more than the price	Change is planned **before** dispensing; if impossible, the sale is refused
Cancel after dispensing	"Too late" — the product is already in the tray
Two products for one payment	Impossible: after dispensing, the balance is 0 and the state is Dispensing, which accepts no money
Last item sold	`take` moves to `OutOfStock`; only a restock leaves it
Two threads pressing buttons	`synchronized` public methods; states run under the machine's lock
A bank of machines sharing one stock count	The course notes the design is not safe for that; the shared count on `Slot` would then need a lock or an atomic decrement
Power failure mid-sale	Real machines keep a transaction log (credit held) in non-volatile memory (memory that keeps its contents without power); out of scope

Extensions & follow-up questions

1. **"Where would you add contactless card payment?"** — One new input device beside the cash slot that calls the same machine (for example a `payByCard(amount)` that sets the balance to the price). No state class changes if the states only look at the balance.
2. **"What if it cannot give change?"** — The planner returns empty; the sale is refused and everything is refunded (requirement 7). Mention greedy vs DP.
3. **"Add a maintenance state."** — A new `MaintenanceState` class; the operator switches it on. The existing five states are untouched (Open/Closed).
4. **"Different price per slot for the same product?"** — Move the price from `Product` to `Slot` (the slot already owns the count).
5. **"Report empty slots to headquarters."** — Make `Inventory` notify an observer when a slot reaches 0; a telemetry client sends it over the network.
6. **"What did you leave out on purpose?"** — Card payment, reporting empty slots over a network, refrigeration (chilling) control and per-customer discounts. Say so explicitly: they are scope decisions, not oversights.
7. **"Timeout if the customer walks away with money in?"** — A timer that calls `cancel()` after, say, 60 seconds of inactivity; it goes through the same state machine.

Real-world example

Modern vending machines are often retro-fitted with cashless readers such as **Nayax** or **Cantaloupe (ePort)** units, which plug into the machine's controller as just another payment device — exactly extension 1: a new input beside the coin mechanism, with the dispensing logic unchanged. Coin mechanisms in real machines keep "change tubes" of each coin and show an **"exact change only"** light when a tube runs low — the physical version of our planner reporting that it cannot make change.

In the interview

Plan for 45 minutes: 5 min questions and requirements → 5 min entities (product, slot, inventory, cash box) → **10 min: draw the state diagram and the state table** → 15 min code the interface, two or three states and `checkPaid`/`refundAll` → 10 min edge cases and follow-ups.

> **30-second answer:** "The whole question is the order of events, so I'll use the State pattern: one class per situation — Idle, Selected, HasMoney, Dispensing, OutOfStock — all implementing an interface that lists every button. The machine just forwards to its current state. Money goes into the cash box as it is inserted, so a refund is just 'make exact change for the balance'; if exact change is impossible, the sale is refused and everything is returned. Amounts are integer cents."

Follow-ups with answers

- *"Why not an enum with a switch?"* — It works for tiny machines, but every new state means editing every switch; with the State pattern a new state is a new class, and the compiler forces it to answer every button.
- *"Is greedy change correct?"* — Only for well-stocked canonical coin sets (coin systems such as 1/5/10/25 where taking the largest coin first always gives the fewest coins). With limited coins it can fail; use bounded coin-change DP.
- *"Where does the operator fit?"* — Outside the state interface; restocking is allowed in any state and can move OutOfStock → Idle.

Self-check

1. **Q:** What makes the design "complete"? **A:** Every state class implements every button method, so every (state, button) cell has a defined answer.
2. **Q:** Why does inserted money go straight into the cash box? **A:** So the same pieces can be returned as change or refund; a refund is just planning exact change for the balance.
3. **Q:** Why is `restock` not in `VendingState`? **A:** It is not a customer action and is valid in every state (Interface Segregation).
4. **Q:** Give a case where greedy change fails. **A:** 30¢ with one quarter and three dimes: greedy takes 25¢ and cannot make 5¢; three dimes work.
5. **Q:** How is "two products for one payment" prevented? **A:** After dispensing, balance is reset to 0 and the Dispensing state returns any inserted money until the product is taken.

TL;DR

- **State pattern is the whole design**: five state classes, one interface listing every button, no `if` on state in the machine.
- Count lives on the **slot**; money in **integer cents**; inserted money goes **into the box immediately**.
- Change: **exact** or refuse; greedy can fail, **DP** (bounded coin change) is the correct planner.
- One refund path for cancel and "no change"; operator actions stay off the state interface.

14.3 Design a Coffee Vending Machine

A **coffee vending machine** makes drinks on demand — espresso, americano, latte, cappuccino, mocha, hot chocolate — from a shared stock of **ingredients** (beans, water, milk, chocolate, sugar). It looks like the previous problem, but the hard part is different. A snack machine hands out a finished item from a slot. A coffee machine **consumes several ingredients at once** from shared containers, and many drinks draw on the same milk. The interviewer wants to see three things:

1. **Recipes as data** — adding a drink should be a data change, not a new class.
2. **Atomic, all-or-nothing deduction** — a latte must take its beans, water *and* milk together, or take nothing; stock must never go negative, even when ten people order at once.
3. **Low-stock alerts** — the operator hears about milk running low *before* it runs out (Observer).

Problem & clarifying questions

Question	Assumed answer
Which drinks?	Espresso, Americano, Latte, Cappuccino, Mocha, Hot Chocolate — loaded from a menu file.
Customisation?	Extra espresso shot (+50¢) and sugar (grams).
Payment?	Amount paid in cents; change returned (coin handling was covered in 14.2).
Several screens / concurrent orders?	Yes: a large machine or a kiosk with several touch screens, so orders can overlap.
Who refills?	An operator, alerted when an ingredient falls to a threshold.
One alert or one per drink?	One alert per crossing of the threshold (re-armed after a refill).

Requirements

Functional

1. Several drink types, each with a **price** and a **recipe** (amount of each ingredient).
2. Show a **menu** with prices and availability.
3. Accept a selection and a **payment**; return **change**; refund in full if the drink cannot be made.
4. Track **ingredient inventory**; deduct exactly the recipe amounts per drink.
5. Raise **low-stock notifications** when an ingredient drops to its threshold.
6. Allow **refills**.
7. Support **customisation** (extra shot, sugar) without changing the menu.

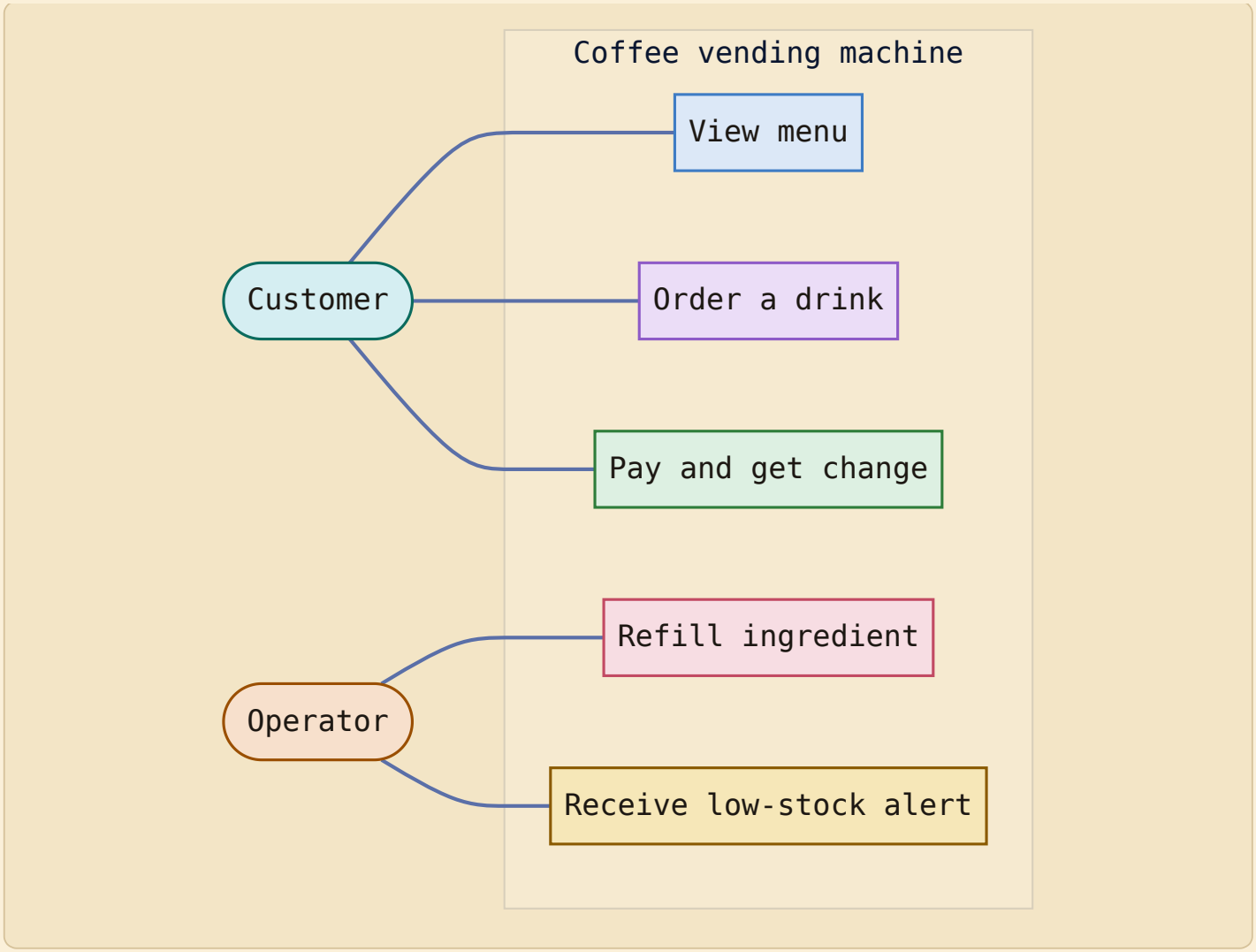
Non-functional

- **Thread-safe**: concurrent orders must never over-draw an ingredient or leave a half-deducted recipe.
- **Data-driven**: new drinks and recipe tweaks without code changes.
- **Observable**: operators, dashboards or a telemetry client can subscribe to alerts without the machine knowing about them.

Use cases

Actors: **Customer** (views menu, orders, pays, takes drink and change), **Operator** (refills, receives alerts), **System** (checks stock, deducts, brews, sends alerts).

Use cases: view menu · order drink (with customisation) · pay / get change / get refund · refill ingredient · receive low-stock alert.



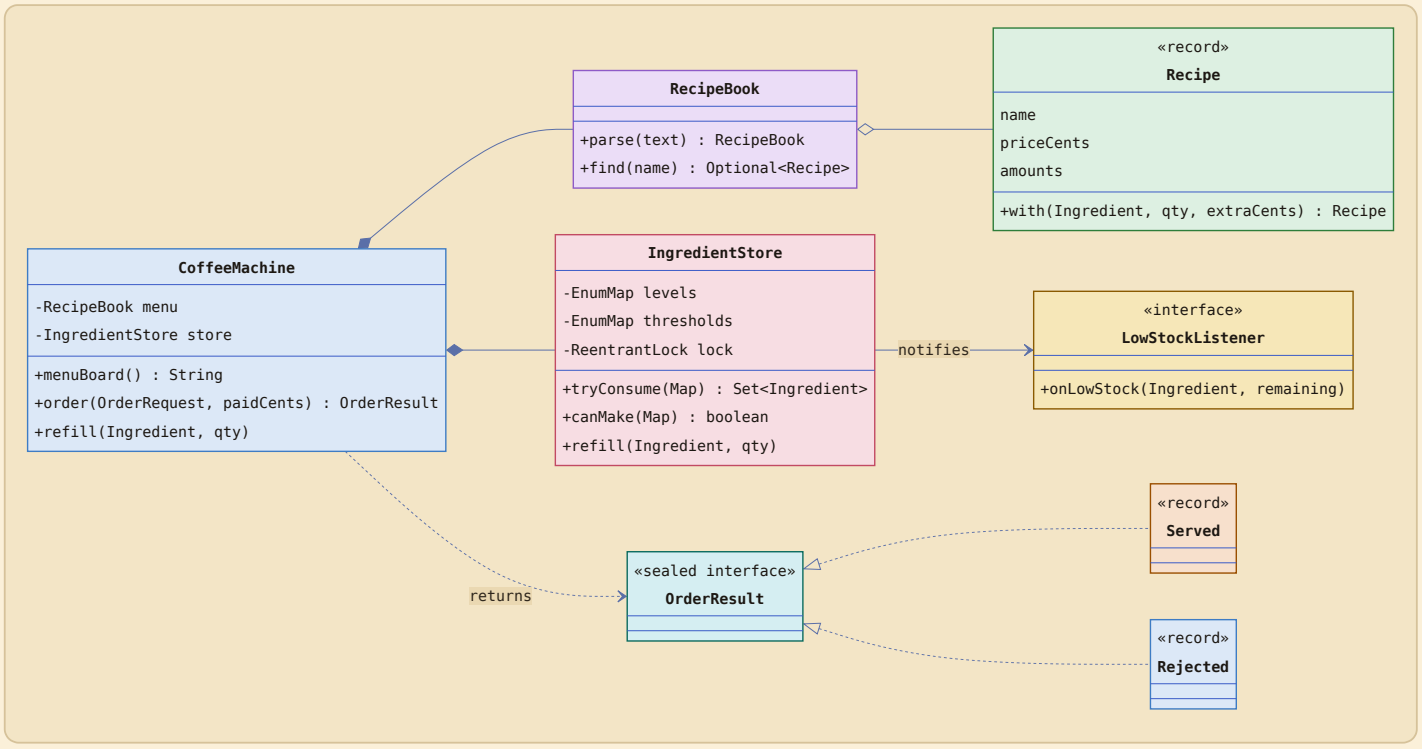
Core entities

Class	Responsibility
`Ingredient` (enum)	Beans, water, milk, chocolate, sugar — with the unit (g or ml).
`Recipe` (record)	**Data**: name, price, `Map<Ingredient, Integer>` amounts. Immutable (cannot change after creation). `with(...)` returns a customised copy.
`RecipeBook`	Parses the menu text into recipes; looks drinks up by name.
`IngredientStore`	Levels, thresholds, **one lock**; `tryConsume` = check all + deduct all atomically; alerts listeners.
`LowStockListener` (interface)	Observer called once when an ingredient crosses its threshold.
`OrderRequest` (record)	Drink name + extra shots + sugar grams.
`OrderResult` (sealed)	Either `Served(drink, price, change)` or `Rejected(reason, refund)`.
`CoffeeMachine`	Facade (one simple front object hiding the parts behind it): menu board, `order(...)`, `refill(...)`.

Mapping to the common problem statement. The widely used version of this problem (for example in the open-source *awesome-low-level-design* collection) asks for: several coffee types (espresso, cappuccino, latte) each with a price and a recipe, a menu with prices, selection and payment, dispensing with change, ingredient tracking with low-stock notifications, and thread-safe handling of concurrent requests. Its classes map onto ours as follows: `Coffee` (name, price, recipe) → our `Recipe`; `Ingredient` (name + quantity with `synchronized` update methods) → our `Ingredient` enum plus the levels inside `IngredientStore`; `Payment` (amount paid) → the `paidCents` argument and the change/refund in `OrderResult`; `CoffeeMachine` (a Singleton there) → our injected `CoffeeMachine`;

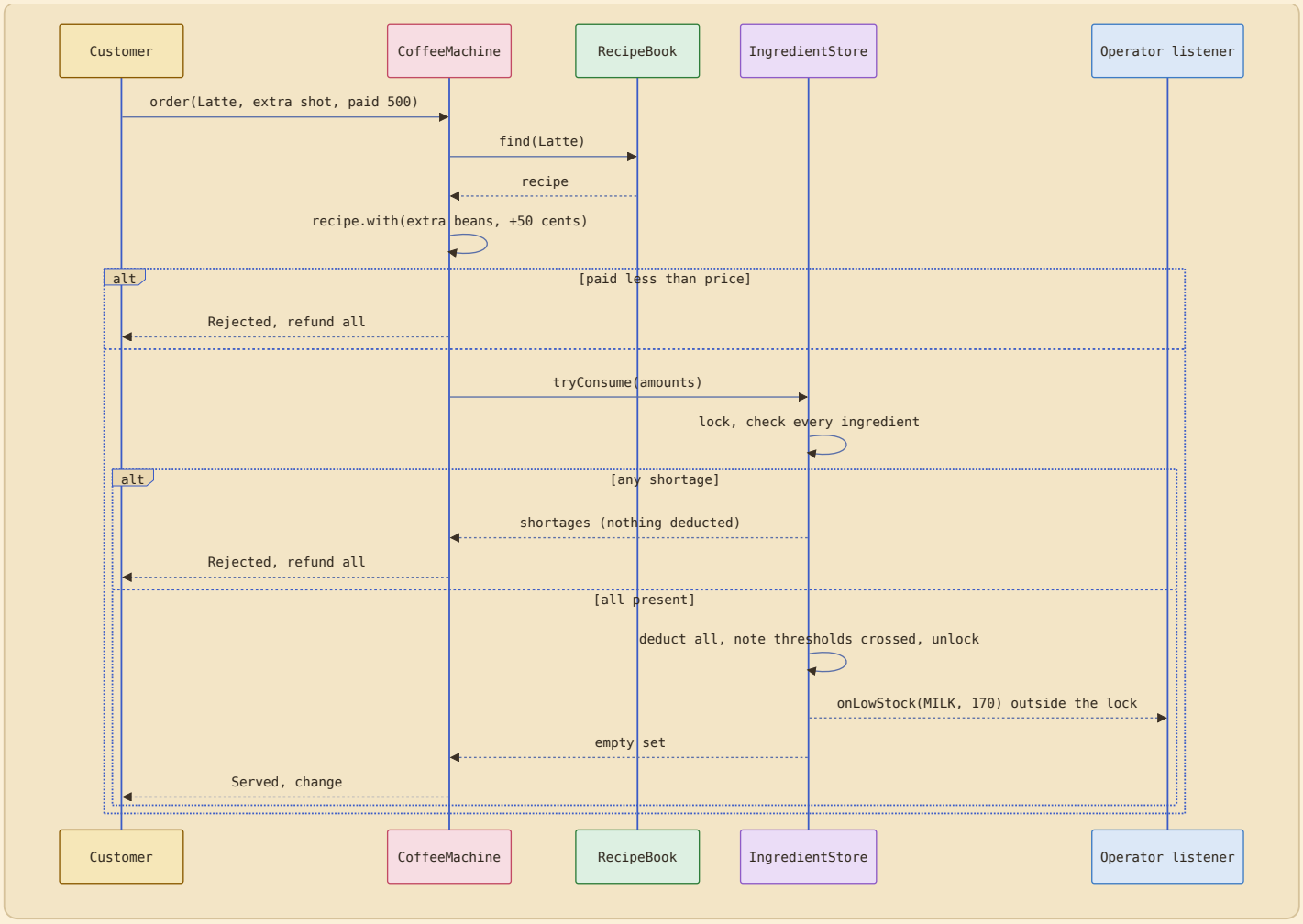
concurrent users simulated with an `ExecutorService` (a Java pool of worker threads that runs submitted tasks) → the 10-thread demo in our `Main`. Two deliberate differences: we lock the **whole recipe** rather than each ingredient separately (per-ingredient locking cannot make a multi-ingredient deduction all-or-nothing), and we avoid the Singleton (see the table below).

Class diagram

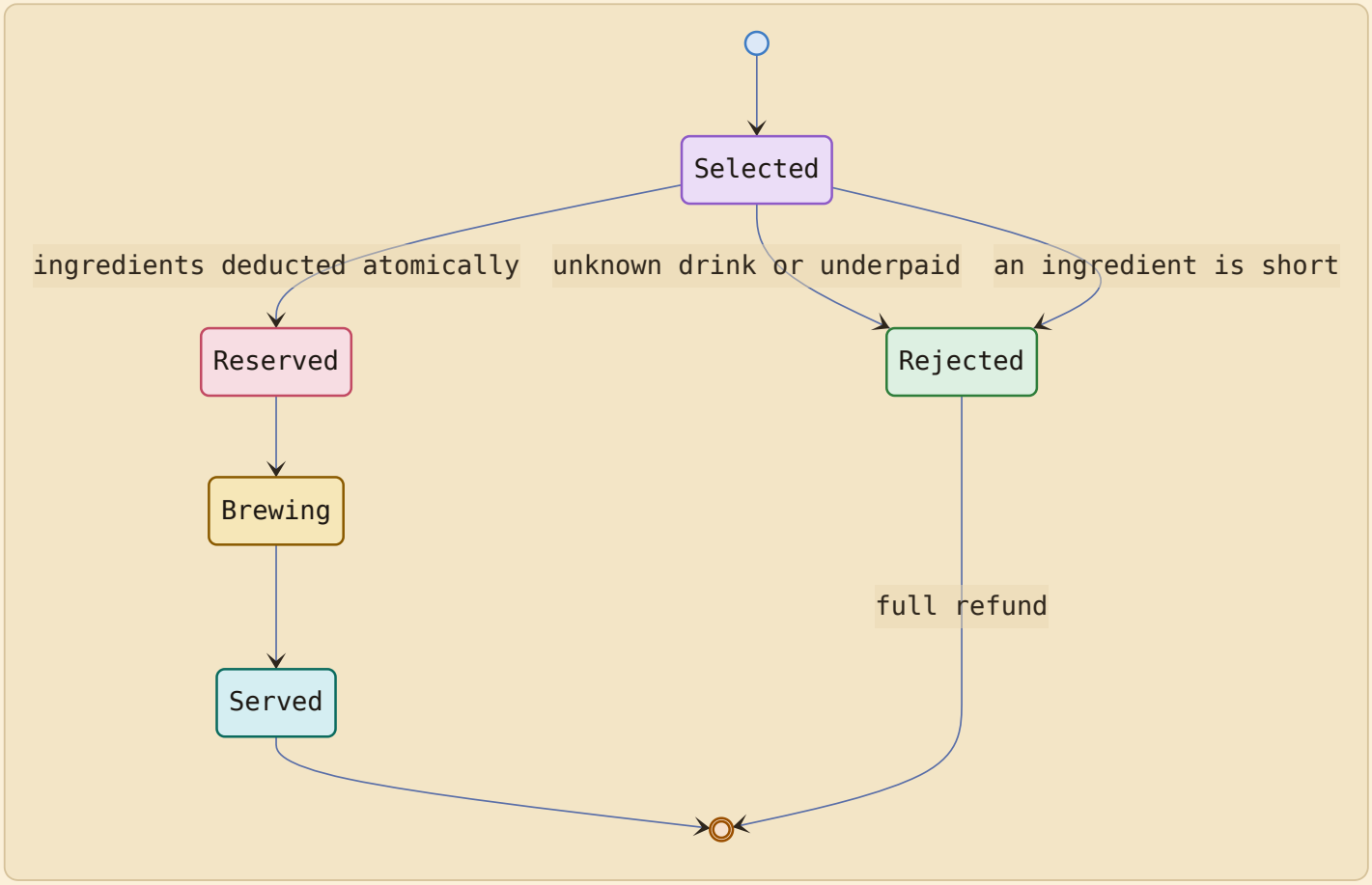


Key flows

Ordering a drink. The key point is that the stock check and the deduction happen **inside one critical section** (a piece of code that only one thread may run at a time), and alerts are sent **after** the lock is released.



Order life cycle.



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Recipes are **data** parsed from a menu text	Data-driven design, Open/Closed	A new drink is a new line in the menu; no new class, no recompile of logic	One subclass per drink (`Latte extends Coffee`) — class explosion, and recipe tweaks need code changes
Check-and-deduct under **one lock**	Atomicity (Ch. 12)	Prevents two orders both seeing "enough milk" and both deducting (a check-then-act **race**: both check, then both act on an answer that is already stale)	Checking in one call and deducting in another; per-ingredient locks (a multi-ingredient recipe would need all of them, in order)
All-or-nothing	Transaction semantics (all steps take effect together, or none do)	Never leave a half-made recipe's ingredients missing	Deduct one by one and "roll back" on failure
Alerts via `LowStockListener`	**Observer**	Operator app, dashboard, telemetry can subscribe independently	The store calling a specific SMS class
Alert **once per crossing**, re-armed by refill	—	Prevents alert storms (one per drink)	Alerting on every order below threshold
Listeners called **outside** the lock	Concurrency hygiene	A slow listener (network call) must not block every order; it also avoids deadlocks (two threads each waiting forever for a lock the other holds) if a listener calls back into the store	Notifying inside the lock
Customisation returns a **new** immutable `Recipe`	Immutability	The menu recipe can never be corrupted by one order	Mutating the menu recipe; a Decorator chain (works, but heavier than needed for "extra X")
`OrderResult` is a **sealed** interface with records	Algebraic data type (a value that is exactly one of a fixed set of shapes; Java 17+ sealed, Java 21 pattern switch)	Callers must handle both Served and Rejected; `switch` is checked for exhaustiveness (the compiler complains if a case is missing)	Returning `null` or throwing for normal refusals
No Singleton	Dependency injection	The problem-set solutions often make `CoffeeMachine` a Singleton; we create one and pass it around, so tests can build fresh machines (the demo builds two)	`CoffeeMachine.getInstance()`

> **Decorator or data?** A classic textbook example (*Head First Design Patterns*) models coffee add-ons with the **Decorator** pattern: `new Whip(new Mocha(new DarkRoast()))`, each wrapper adding cost and description. That is a good fit when each add-on has **behaviour**. Here add-ons only change **quantities and price**, so a data change (`recipe.with(...)`) is simpler. Say both in the interview and pick one for a reason.

Java implementation

Package `lld.ch14.coffeemachine`.

```java
// File: lld/ch14/coffeemachine/Recipes.java
package lld.ch14.coffeemachine;

import java.util.Collections;
import java.util.EnumMap;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Optional;

enum Ingredient {
    COFFEE_BEANS("g"), WATER("ml"), MILK("ml"), CHOCOLATE("g"), SUGAR("g");

    final String unit;

    Ingredient(String unit) { this.unit = unit; }
}

/** A recipe is DATA: a name, a price and the amount of each ingredient. */
record Recipe(String name, int priceCents, Map<Ingredient, Integer> amounts) {

    Recipe {
        amounts = Collections.unmodifiableMap(new EnumMap<>(amounts));
    }

    /** Customisation returns a NEW recipe; the menu entry is never modified. */
    Recipe with(Ingredient extra, int qty, int extraCents) {
        if (qty == 0) return this;
        Map<Ingredient, Integer> m = new EnumMap<>(Ingredient.class);
        m.putAll(amounts);
        m.merge(extra, qty, Integer::sum);
        return new Recipe(name, priceCents + extraCents, m);
    }
}

/** Parses the menu from plain text, so adding a drink is a data change, not a code change. */
final class RecipeBook {
    private final Map<String, Recipe> byName = new LinkedHashMap<>();

    static RecipeBook parse(String text) {
        RecipeBook book = new RecipeBook();
        for (String line : text.strip().split("\n")) {
            line = line.strip();
            if (line.isEmpty() || line.startsWith("#")) continue;
            String[] parts = line.split(",");
            Map<Ingredient, Integer> amounts = new EnumMap<>(Ingredient.class);
            for (int i = 2; i < parts.length; i++) {
                String[] kv = parts[i].strip().split("=");
                amounts.put(Ingredient.valueOf(kv[0].strip()), Integer.parseInt(kv[1].strip()));
            }
            Recipe r = new Recipe(parts[0].strip(), Integer.parseInt(parts[1].strip()), amounts);
            book.byName.put(r.name().toLowerCase(), r);
        }
        return book;
    }

    Optional<Recipe> find(String name) { return Optional.ofNullable(byName.get(name.toLowerCase())); }
    Iterable<Recipe> all() { return byName.values(); }
}
```

```java
// File: lld/ch14/coffeemachine/IngredientStore.java
package lld.ch14.coffeemachine;

import java.util.ArrayList;
import java.util.EnumMap;
import java.util.EnumSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.locks.ReentrantLock;

/** Observer: told once when an ingredient drops to or below its threshold. */
interface LowStockListener {
    void onLowStock(Ingredient ingredient, int remaining);
}

/**
 * Ingredient levels. The whole check-then-deduct runs under ONE lock, so a drink either
 * takes every ingredient it needs or takes nothing (all-or-nothing, never negative).
 */
final class IngredientStore {
    private final Map<Ingredient, Integer> levels = new EnumMap<>(Ingredient.class);
    private final Map<Ingredient, Integer> thresholds = new EnumMap<>(Ingredient.class);
    private final Set<Ingredient> alerted = EnumSet.noneOf(Ingredient.class);
    private final List<LowStockListener> listeners = new CopyOnWriteArrayList<>();
    private final ReentrantLock lock = new ReentrantLock();

    IngredientStore(Map<Ingredient, Integer> initial, Map<Ingredient, Integer> lowThresholds) {
        for (Ingredient i : Ingredient.values()) {
            levels.put(i, initial.getOrDefault(i, 0));
            thresholds.put(i, lowThresholds.getOrDefault(i, 0));
        }
    }

    void addListener(LowStockListener l) { listeners.add(l); }

    /** Returns the ingredients that are short. Empty set = success, everything was deducted. */
    Set<Ingredient> tryConsume(Map<Ingredient, Integer> need) {
        Set<Ingredient> shortages = EnumSet.noneOf(Ingredient.class);
        List<Runnable> alerts = new ArrayList<>();
        lock.lock();
        try {
            need.forEach((i, q) -> { if (levels.get(i) < q) shortages.add(i); });
            if (!shortages.isEmpty()) return shortages;              // nothing touched
            need.forEach((i, q) -> levels.merge(i, -q, Integer::sum));
            for (Ingredient i : need.keySet()) {
                int left = levels.get(i);
                if (left <= thresholds.get(i) && alerted.add(i)) {    // alert once per crossing
                    alerts.add(() -> listeners.forEach(l -> l.onLowStock(i, left)));
                }
            }
        } finally {
            lock.unlock();
        }
        alerts.forEach(Runnable::run);   // call listeners OUTSIDE the lock
        return shortages;
    }

    boolean canMake(Map<Ingredient, Integer> need) {
        lock.lock();
        try {
            return need.entrySet().stream().allMatch(e -> levels.get(e.getKey()) >= e.getValue());
        } finally {
            lock.unlock();
        }
    }
```

```
    void refill(Ingredient i, int qty) {
        lock.lock();
        try {
            levels.merge(i, qty, Integer::sum);
            if (levels.get(i) > thresholds.get(i)) alerted.remove(i);  // re-arm the alert
        } finally {
            lock.unlock();
        }
    }

    int level(Ingredient i) {
        lock.lock();
        try { return levels.get(i); } finally { lock.unlock(); }
    }
}
```

```java
// File: lld/ch14/coffeemachine/CoffeeMachine.java
package lld.ch14.coffeemachine;

import java.util.Set;

/** What the customer asked for. */
record OrderRequest(String drink, int extraShots, int sugarGrams) {
    OrderRequest(String drink) { this(drink, 0, 0); }
}

/** The outcome of an order: a sealed type, so callers must handle both cases. */
sealed interface OrderResult permits Served, Rejected {}
record Served(String drink, int pricedCents, int changeCents) implements OrderResult {}
record Rejected(String reason, int refundCents) implements OrderResult {}

final class CoffeeMachine {
    static final int EXTRA_SHOT_CENTS = 50;
    static final int SHOT_BEANS_G = 18;

    private final RecipeBook menu;
    private final IngredientStore store;

    CoffeeMachine(RecipeBook menu, IngredientStore store) {
        this.menu = menu;
        this.store = store;
    }

    String menuBoard() {
        StringBuilder sb = new StringBuilder();
        for (Recipe r : menu.all()) {
            String line = "  %-14s %4d c  %s".formatted(r.name(), r.priceCents(),
                    store.canMake(r.amounts()) ? "" : "(unavailable)");
            sb.append(line.stripTrailing()).append('\n');
        }
        return sb.toString();
    }

    /** Thread-safe: many touch screens may order at once. */
    OrderResult order(OrderRequest req, int paidCents) {
        var base = menu.find(req.drink());
        if (base.isEmpty()) return new Rejected("no such drink: " + req.drink(), paidCents);
        Recipe r = base.get()
                .with(Ingredient.COFFEE_BEANS, req.extraShots() * SHOT_BEANS_G, req.extraShots() *
EXTRA_SHOT_CENTS)
                .with(Ingredient.SUGAR, req.sugarGrams(), 0);
        if (paidCents < r.priceCents()) {
            return new Rejected("insufficient payment, need " + r.priceCents() + " c", paidCents);
        }
        Set<Ingredient> short_ = store.tryConsume(r.amounts());      // atomic reserve-and-deduct
        if (!short_.isEmpty()) return new Rejected("out of " + short_, paidCents);
        return new Served(r.name(), r.priceCents(), paidCents - r.priceCents());
    }

    void refill(Ingredient i, int qty) { store.refill(i, qty); }
    int level(Ingredient i) { return store.level(i); }
}
```

```java
// File: lld/ch14/coffeemachine/Main.java
package lld.ch14.coffeemachine;

import java.util.Map;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static final String MENU = """
            # name, price_cents, INGREDIENT=qty ...
            Espresso,      250, COFFEE_BEANS=18, WATER=40
            Americano,     300, COFFEE_BEANS=18, WATER=200
            Latte,         400, COFFEE_BEANS=18, WATER=40, MILK=180
            Cappuccino,    380, COFFEE_BEANS=18, WATER=40, MILK=120
            Mocha,         450, COFFEE_BEANS=18, WATER=40, MILK=150, CHOCOLATE=25
            Hot Chocolate, 300, MILK=200, CHOCOLATE=30
            """;

    static final Map<Ingredient, Integer> LOW = Map.of(Ingredient.COFFEE_BEANS, 100, Ingredient.WATER, 500,
            Ingredient.MILK, 200, Ingredient.CHOCOLATE, 30, Ingredient.SUGAR, 20);

    public static void main(String[] args) throws Exception {
        IngredientStore store = new IngredientStore(Map.of(Ingredient.COFFEE_BEANS, 500, Ingredient.WATER, 3000,
                Ingredient.MILK, 700, Ingredient.CHOCOLATE, 60, Ingredient.SUGAR, 100), LOW);
        store.addListener((i, left) -> System.out.println("   ALERT to operator: " + i + " low, " + left + i.unit +
" left"));
        CoffeeMachine cm = new CoffeeMachine(RecipeBook.parse(MENU), store);

        System.out.print("Menu:\n" + cm.menuBoard());
        System.out.println("\n== Orders ==");
        show(cm.order(new OrderRequest("Espresso"), 300));
        show(cm.order(new OrderRequest("Latte", 1, 10), 500));      // extra shot + sugar
        show(cm.order(new OrderRequest("Flat White"), 400));
        show(cm.order(new OrderRequest("Mocha"), 200));
        show(cm.order(new OrderRequest("Mocha"), 450));             // chocolate 60 -> 35
        show(cm.order(new OrderRequest("Hot Chocolate"), 300));     // milk 370 -> 170, chocolate 35 -> 5 : alerts
        show(cm.order(new OrderRequest("Cappuccino"), 400));        // needs 120 milk, 170 left: ok
        show(cm.order(new OrderRequest("Latte"), 400));             // 50 milk left: rejected, nothing deducted
        System.out.println("   beans left " + cm.level(Ingredient.COFFEE_BEANS) + "g (unchanged by the rejected
latte)");
        System.out.print("Menu now:\n" + cm.menuBoard());

        System.out.println("\n== Operator refills milk ==");
        cm.refill(Ingredient.MILK, 1000);
        show(cm.order(new OrderRequest("Latte"), 400));

        System.out.println("\n== 10 customers order a Latte at the same instant (milk for exactly 4) ==");
        IngredientStore store2 = new IngredientStore(Map.of(Ingredient.COFFEE_BEANS, 1000, Ingredient.WATER, 5000,
                Ingredient.MILK, 800), LOW);
        store2.addListener((i, left) -> System.out.println("   ALERT to operator: " + i + " low, " + left + i.unit
+ " left"));
        CoffeeMachine busy = new CoffeeMachine(RecipeBook.parse(MENU), store2);
        AtomicInteger served = new AtomicInteger(), rejected = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(10)) {
            for (int i = 0; i < 10; i++) {
                pool.submit(() -> {
                    go.await();
                    switch (busy.order(new OrderRequest("Latte"), 400)) {
                        case Served s -> served.incrementAndGet();
                        case Rejected r -> rejected.incrementAndGet();
                    }
                    return null;
                });
            }
            go.countDown();
```

```
        }
        System.out.println("served=" + served + " rejected=" + rejected
                + " milk left=" + busy.level(Ingredient.MILK) + "ml (never negative)");
    }

    static void show(OrderResult r) {
        switch (r) {
            case Served s -> System.out.println("SERVED   %s for %d c, change %d c".formatted(s.drink(),
s.pricedCents(), s.changeCents()));
            case Rejected x -> System.out.println("REJECTED %s, refund %d c".formatted(x.reason(),
x.refundCents()));
        }
    }
}
```

How the code works, part by part

- `Main.MENU` is a Java text block (a multi-line string written between `"""` marks); `RecipeBook.parse` turns each line into a `Recipe`. Adding "Flat White" is one more line.
- `Recipe` copies the amounts into an unmodifiable `EnumMap` in its compact constructor (a record constructor written without a parameter list, used for checks and defensive copies), so a recipe can never be changed after creation.
- `IngredientStore.tryConsume` holds a `ReentrantLock` (an explicit lock object with `lock()` and `unlock()`; *re-entrant* means the thread already holding it may take it again) while it (1) collects shortages, (2) returns immediately if any, (3) deducts every ingredient, (4) records which thresholds were crossed. It releases the lock and **then** calls listeners.
- `alerted` is an `EnumSet` (a compact set of enum constants) that remembers which ingredients already raised an alert; `refill` clears the flag when the level goes back above the threshold.
- `CoffeeMachine.order` validates the drink, applies customisation, checks payment, then reserves ingredients. The `switch` in `Main.show` uses Java 21 pattern matching over the sealed `OrderResult`.

Output:

```
Menu:
  Espresso        250 c
  Americano       300 c
  Latte           400 c
  Cappuccino      380 c
  Mocha           450 c
  Hot Chocolate   300 c

== Orders ==
SERVED   Espresso for 250 c, change 50 c
SERVED   Latte for 450 c, change 50 c
REJECTED no such drink: Flat White, refund 400 c
REJECTED insufficient payment, need 450 c, refund 200 c
SERVED   Mocha for 450 c, change 0 c
   ALERT to operator: MILK low, 170ml left
   ALERT to operator: CHOCOLATE low, 5g left
SERVED   Hot Chocolate for 300 c, change 0 c
SERVED   Cappuccino for 380 c, change 20 c
REJECTED out of [MILK], refund 400 c
   beans left 410g (unchanged by the rejected latte)
Menu now:
  Espresso        250 c
  Americano       300 c
  Latte           400 c  (unavailable)
  Cappuccino      380 c  (unavailable)
  Mocha           450 c  (unavailable)
  Hot Chocolate   300 c  (unavailable)

== Operator refills milk ==
SERVED   Latte for 400 c, change 0 c

== 10 customers order a Latte at the same instant (milk for exactly 4) ==
   ALERT to operator: MILK low, 80ml left
served=4 rejected=6 milk left=80ml (never negative)
```

Read the output: the rejected latte left the beans at 410 g — nothing is deducted when any ingredient is short; the milk and chocolate alerts fire once each; in the last part ten threads order a latte at the same instant with milk for exactly four: four are served, six are refused, milk ends at 80 ml, never negative, and the alert fires exactly once.

Concurrency & edge cases

Situation	What could go wrong	How the design handles it
Ten concurrent orders, milk for four	Over-draw (negative stock) — classic check-then-act race	Check and deduct under one lock
Recipe needs several ingredients, one is short	Beans and water taken, milk missing	All-or-nothing: shortages are found before anything is deducted
Listener is slow (sends an SMS)	Every order waits on the network	Listeners run after `unlock()`
Many orders below the threshold	Alert storm	One alert per crossing; refill re-arms
Underpayment	Drink made, money short	Payment checked before reserving ingredients
Brewing fails after deduction (hardware fault)	Customer pays, gets nothing	Extension: refund and optionally `refill` the reserved amounts back (compensation: an action that undoes an earlier step)
Menu displays "available" and then the order fails	A race between display and order	The menu is advisory; `tryConsume` is the source of truth

A **single lock** is right here: the critical section is tiny (five map operations) and the machine brews one drink at a time anyway. If contention (many threads waiting for the same lock) mattered (a cloud-backed inventory shared by many machines), you would move to optimistic concurrency (work without a lock, then check at write

time that nobody changed the data, and retry if they did — e.g. version numbers / compare-and-set on a snapshot) or a database transaction (a group of database changes that succeed or fail together).

Extensions & follow-up questions

1. **"Add a new drink."** — Add one line to the menu data. No code change.
2. **"Sizes (small/medium/large)?"** — A `Size` enum with a multiplier; `recipe.scaled(size)` returns a new recipe with scaled amounts and price.
3. **"Several brewing units?"** — A pool of `Brewer` workers (a `BlockingQueue` — a thread-safe queue where a taker waits while it is empty — of orders consumed by N threads). Inventory stays in the one `IngredientStore`.
4. **"Remote monitoring?"** — Another `LowStockListener` that posts to a telemetry endpoint. The store does not change.
5. **"Brewing steps differ per drink (grind, tamp, steam milk)?"** — Template Method (a base class fixes the order of the steps and subclasses fill in some of them): a `BrewProcess` with fixed steps and hooks (optional steps a subclass may override), or a list of `Step` objects in the recipe data (Command).
6. **"Card payments and change?"** — Reuse the `CashBox` from 14.2 for cash; card payment is another payment device.

Real-world example

Professional espresso machines such as the Thermoplan Mastrena used in Starbucks stores store **drink recipes as configuration**, so a chain can roll out a new beverage without new hardware. Starbucks and Microsoft publicly described (in 2019) connecting store equipment, including espresso machines, to the cloud through Microsoft Azure Sphere, so that each shot's data is collected as telemetry and new recipes are pushed from the cloud instead of being carried to stores on USB drives — the "recipes as data" and "remote monitoring listener" ideas in this design. Many connected self-serve coffee kiosks also report stock levels remotely, so that a service visit is scheduled before the milk runs out — the low-stock Observer in this design.

In the interview

Plan for 40 minutes: 5 min questions → 5 min entities (ingredient, recipe, store, machine) → 5 min "recipes are data" and the menu format → 15 min code `tryConsume` (lock, all-or-nothing, alert outside the lock) and `order` → 10 min concurrency demo and extensions.

> **30-second answer:** "Recipes are data: a map from ingredient to amount, loaded from a menu, so a new drink is a new row. The ingredient store checks and deducts **every** ingredient of a recipe under one lock, so an order is all-or-nothing and stock never goes negative. When an ingredient crosses its threshold, the store notes it and, after releasing the lock, notifies observers once. Orders return a sealed Served/Rejected result, and a rejected order is refunded in full."

Follow-ups with answers

- *"Why not one class per drink?"* — Drinks differ only in data. Classes would multiply with every size and add-on (class explosion) and every recipe tweak would be a code change.
- *"Why not an `AtomicInteger` per ingredient?"* — Each atomic is safe alone, but a recipe updates several at once; without a common lock, two orders could each take the last milk and beans in different orders, leaving partial deductions.
- *"Why notify outside the lock?"* — A slow or re-entrant listener would block all orders or deadlock.

Self-check

1. **Q:** What does "all-or-nothing" mean for `tryConsume`? **A:** Either every ingredient of the recipe is deducted, or none is (shortages are detected first).
2. **Q:** What race does the single lock prevent? **A:** Check-then-act: two threads both see enough milk, then both deduct, going negative.
3. **Q:** How is an alert storm prevented? **A:** An `EnumSet` records ingredients already alerted; refill above the threshold re-arms them.
4. **Q:** Why is `Recipe.with` returning a new object important? **A:** Customising one order must not change the shared menu recipe.
5. **Q:** When would you use Decorator for add-ons? **A:** When add-ons carry behaviour (extra steps, special description or pricing logic), not just quantities.

TL;DR

- **Recipes are data** (`Map<Ingredient,Integer>` + price), parsed from a menu; customisation creates a new immutable recipe.
- **One lock, check-then-deduct, all-or-nothing**: stock never goes negative under concurrent orders.
- **Observer** for low-stock alerts, fired **once per crossing** and **outside** the lock.
- A **sealed** `OrderResult` makes callers handle Served and Rejected; refund in full on rejection.

14.4 Design an ATM

Figure — An automated teller machine with its card reader, keypad, screen, cash dispenser, deposit slot and printer (course diagram)

An **automated teller machine (ATM)** is a cashier-less machine, placed in a public spot, where a bank's customers can do everyday banking. It exists because branches keep limited hours and people cannot go to a branch every time they need cash or want to pay in money. The interviewer is checking whether you can keep **money correct** (never dispense without debiting, never debit without dispensing, never let two machines overdraw one account), **authenticate** before anything else, and model the machine's **session states** cleanly.

Problem & clarifying questions

Question	Assumed answer
Which transactions?	Balance inquiry, cash withdrawal, cash deposit, check deposit, transfer.
Which account types?	Checking and savings. Withdrawal is from checking (and savings within a daily cap).
How is the user identified?	ATM card + PIN (personal identification number, a short secret code). The card is the main security element.
Does the ATM talk to the bank?	Yes, on every transaction (assume the network is available).
Are deposits available at once?	A check is **not**: it waits for verification. We keep a *total* and an *available* balance.
Which notes are loaded?	$100, $50, $20, $10 cassettes.
Wrong PIN?	Three wrong attempts block the card and the ATM keeps it. The course lists PIN retry limits and card capture as deliberately left out, along with receipt-paper levels, an audit journal and other banks' networks; we add the first two and the journal.
How many customers at a time?	One per machine; the ATM must not shut down mid-session.

Requirements

Parts of the machine (from the course): **card reader** (reads the card), **keypad** (PIN, amounts), **screen** (messages; also used to pick a transaction), **cash dispenser**, **deposit slot** (cash or checks), **printer** (receipts), and a **network link** to the bank.

Functional

1. The customer inserts a card and enters a PIN; the ATM asks the **bank** to authenticate. No transaction is possible without authentication.
2. Five transactions: **balance inquiry** (each account), **deposit cash**, **deposit check**, **withdraw cash** (from checking), **transfer funds** to another account.
3. The ATM **keeps the card** for the whole session; the customer can **cancel** at any time and the card is ejected.
4. The ATM keeps an **internal log (journal)** of transactions, including hardware failures, for the operator to resolve issues.
5. A deposited **check is not added at once**: it waits for manual verification and bank approval. (The course assumes deposits in general are verified first; our code credits counted cash immediately and holds checks — see the trade-off below.)
6. The **bank manager** can see the ATM's system information (reports).

Actors' operations

- **Operator:** turn the ATM on/off with the key switch; refill cash; refill receipt paper and ink; take out deposited cash and checks.
- **Customer:** balance inquiry, withdraw, deposit (cash or check), transfer.
- **Bank manager:** reports of total deposits and total withdrawals; print them; check the cash remaining in the ATM.

Non-functional

- **Security:** PINs never stored or compared in clear on the ATM; limit PIN attempts.
- **Consistency:** debit and dispense happen together or not at all (compensate on failure).
- **Concurrency:** many ATMs act on the same account at once without overdrawing it or deadlocking.
- **Auditability** (every action can be traced afterwards)**:** every action is journaled.

Use cases

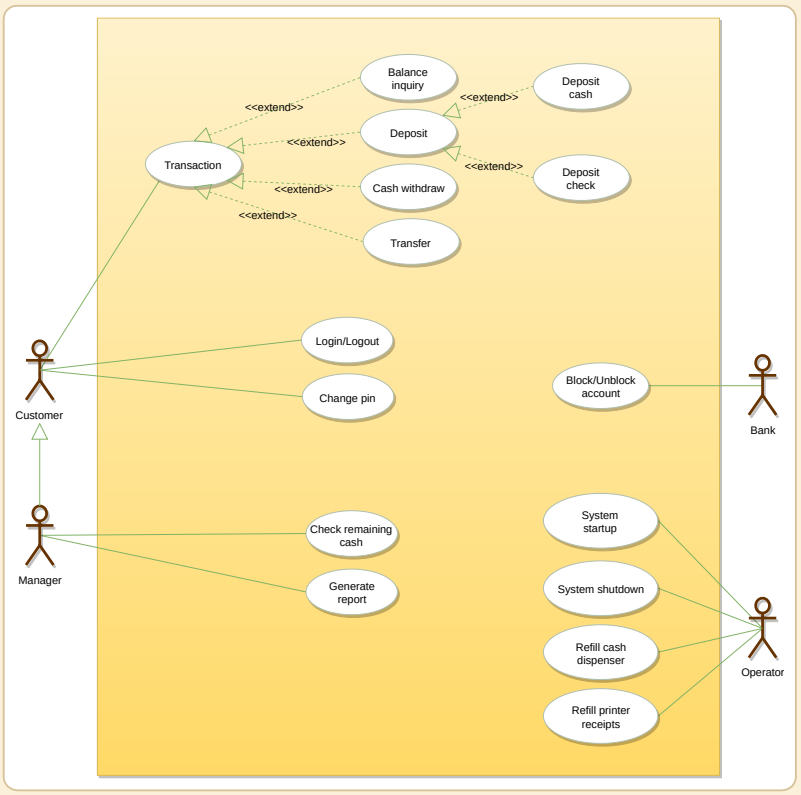


Figure — ATM use case diagram: Customer (balance, withdraw, deposit, transfer), Operator (on/off, refill cash, paper, ink, empty deposits) and Bank Manager (reports, check remaining cash) (course diagram)

The figure above shows three actors. Only the Customer's use cases go through PIN authentication; the Operator works with a physical key and the Bank Manager reads data held at the bank.

Customer authentication is the entry point of every session:

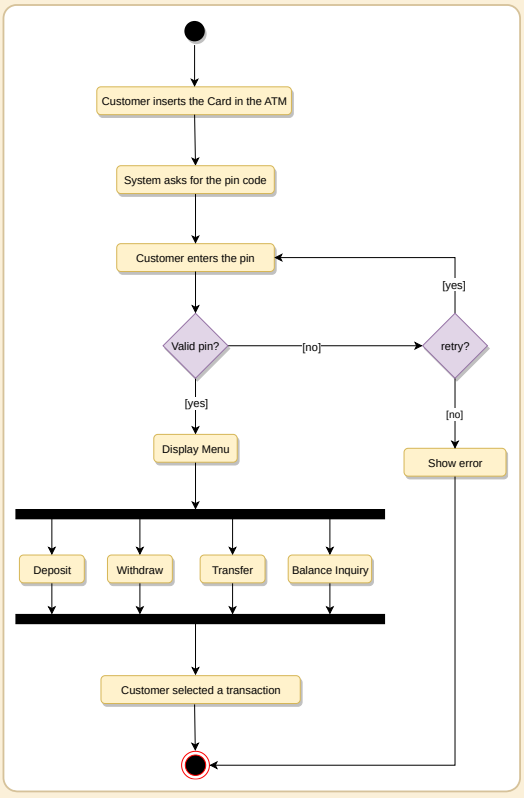


Figure — Activity diagram: insert card, the system asks for the PIN, the customer enters it; an invalid PIN shows an error and offers a retry; a valid PIN displays the menu (balance inquiry, deposit, withdraw, transfer) (course diagram)

Withdraw:

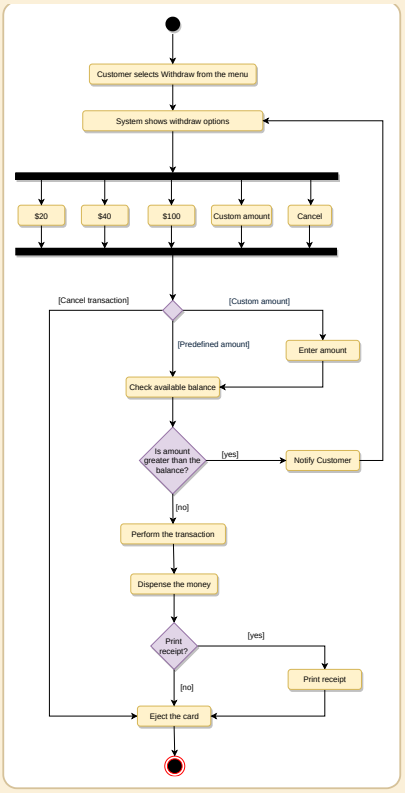


Figure — Activity diagram: choose Withdraw, pick a predefined amount ($20, $40, $100) or enter a custom one, check the available balance (notify the customer if the amount is greater), perform the transaction, dispense, optional receipt, eject the card (course diagram)

Deposit check:

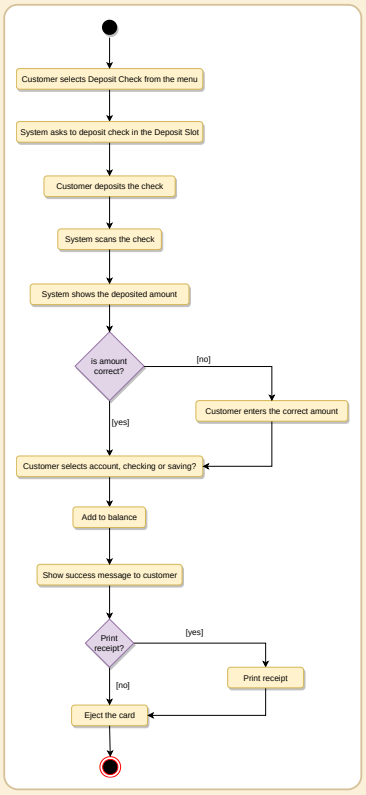


Figure — Activity diagram: choose Deposit Check, pick checking or savings, put the check in the deposit slot, the system scans it and shows the amount; if wrong the customer enters the correct amount; the amount is added, a success message and optional receipt follow, and the card is ejected (course diagram)

Transfer:

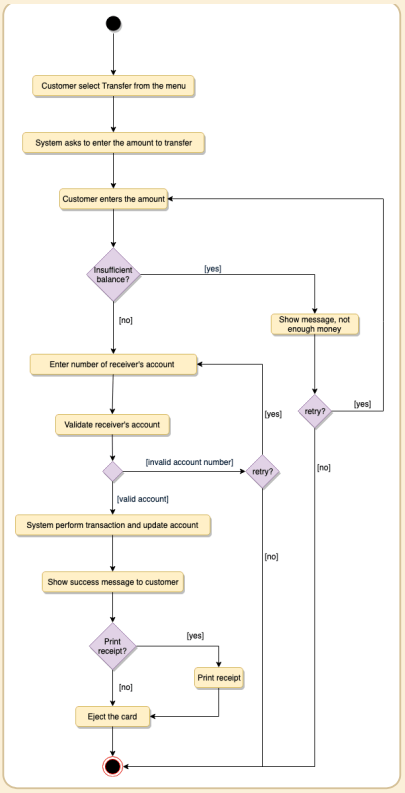


Figure — Activity diagram: enter the amount (retry if balance is insufficient), enter and validate the receiver's account (retry if invalid), perform the transfer, show success, optional receipt, eject card (course diagram)

In the transfer diagram above, note the two retry loops: one for "insufficient balance" and one for "invalid account number". If the customer declines to retry, the diagram goes straight to the end node without the "Eject the card" step; in a real design every exit path must still return the card, which is exactly the card-reader rule in stage 2 below.

Core entities

Class	Responsibility
`Card` (record)	Card number and holder name. (The course also has a `Customer` class with name and email, owning exactly one card; our code folds the holder's name into the card.)
`Account` (abstract)	Id, **total** and **available** balance, a lock; the withdrawal rule hook.
`CheckingAccount`, `SavingsAccount`	Savings overrides the rule to add a **daily withdrawal cap** — the only behavioural difference, which is why they are two classes.
`BankService` (interface)	What the ATM needs from the bank: authenticate, balances, withdraw, reverse, deposits, transfer.
`InMemoryBank`	A stand-in bank: PIN hashes (one-way fingerprints of PINs), failed-attempt counts, account locks, ordered-lock transfers.
`AuthResult` (sealed)	`Ok(accounts)`, `WrongPin(attemptsLeft)` or `Blocked`.
`NoteHandler` + 4 subclasses	**Chain of Responsibility** for $100 → $50 → $20 → $10.
`CashDispenser`	Plans a withdrawal from the chain (dry run: a trial that changes nothing), then dispenses; can jam (fault injection: a deliberately triggered failure, for testing).
`Transaction` (sealed) + 5 records	**Command**: each transaction has one `execute(ctx)`; its rules live only there.
`AtmState` + 4 states	**State**: Idle → CardInserted → Authenticated → Idle, plus OutOfService.
`Atm`	Has an id (the course also gives it a location); owns its parts, holds the session, forwards buttons to the state, keeps the journal.
`CardReader`, `Keypad`, `Screen`, `Printer`, `DepositSlot` (cash / check)	The course's hardware classes. Our code folds them into `Atm`: the `card` field is the reader's slot, `screen(...)` is the screen, the method arguments are the keypad, and printing a receipt would format a journal line.

Class diagram

The course builds the ATM in four stages:

Stage	What it adds	What works after it
1	`Customer`, `Card`, `Account` (checking, savings)	Verify a PIN and read a balance
2	Card reader, dispenser, keypad, screen, printer, deposit slots	Work out and count out a set of notes
3	`Transaction` and its five kinds	Move money, with each rule written in exactly one class
4	`Bank` and `ATM`	Run a full session for a single customer at a time

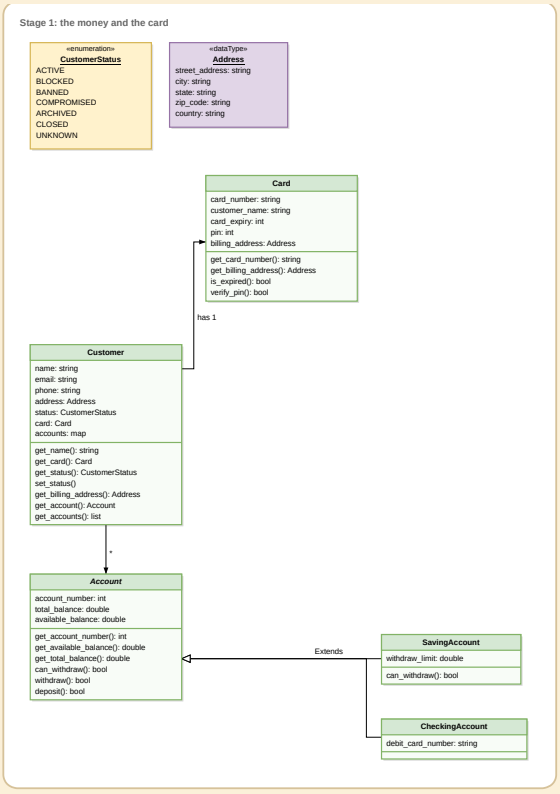


Figure — Stage 1: enumerations, Customer, Card and the abstract Account with Checking and Saving subclasses (course diagram)

Stage 1 — money and card. An account keeps **two balances**. The *total balance* is everything credited to it; the *available balance* is the part the customer may spend right now. A check deposit is the moment they diverge: the check raises the total at once, but the money becomes available only after the check clears. `Account` is abstract with `CheckingAccount` and `SavingsAccount` beneath it; savings overrides a single method to impose its withdrawal cap, and because that is a difference in **behaviour** (not just data) inheritance is justified here instead of a type field. `Customer` (name, email; each customer has **one** card) is linked to its card and accounts by plain associations, since none of them is deleted along with another.

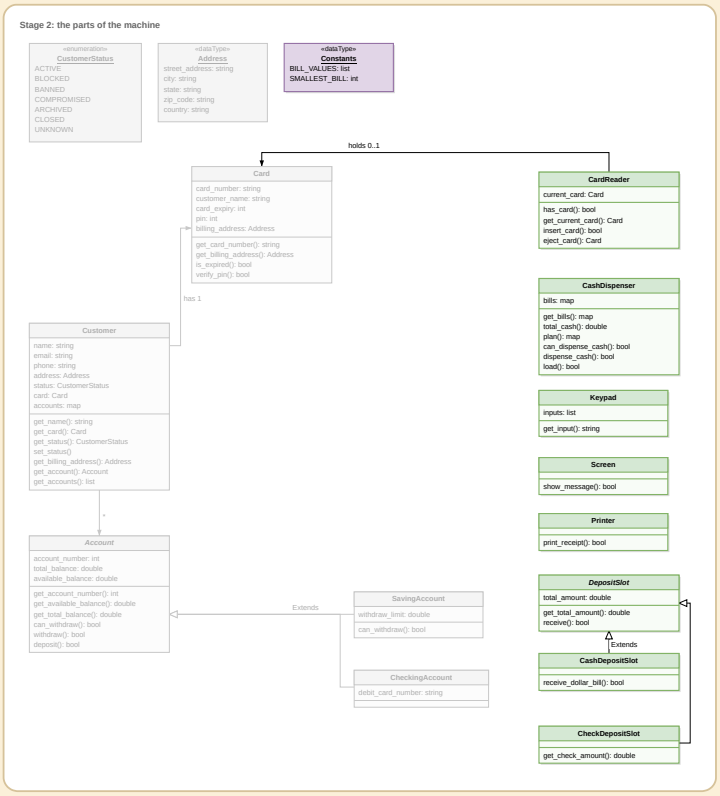


Figure — Stage 2: CardReader, CashDispenser, Keypad, Screen, Printer and the abstract DepositSlot with cash and check slots (course diagram)

Stage 2 — the parts. Six of the seven hardware parts become one class each (the seventh, the network link, is assumed rather than modelled), and none of them knows what a transaction is. Deciding whether an amount (say $85) can be paid from the notes **currently loaded** belongs to the **dispenser**, not to the withdrawal logic; it tries the largest note first and reports failure when the exact amount is impossible. The card reader has one rule of its own: it **keeps the card for the whole session** and releases it when the session ends, so every failed sign-in path must eject (or retain) the card before it returns; the reader's link to `Card` is the only arrow leaving this stage. `DepositSlot` is abstract, with a cash slot and a check slot below it. It is the physical opening money is pushed into; do not confuse it with the deposit *transaction* of stage 3.

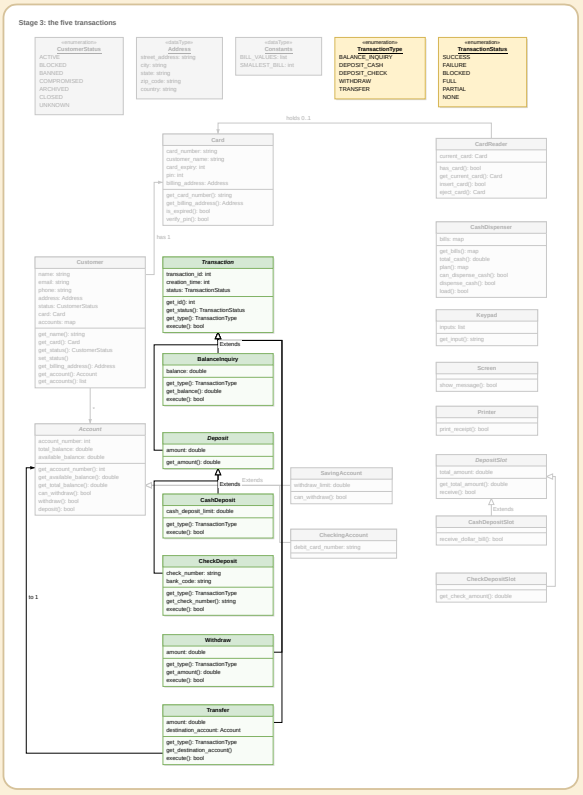


Figure — Stage 3: the abstract Transaction with BalanceInquiry, Deposit (CashDeposit, CheckDeposit), Withdraw and Transfer (course diagram)

Stage 3 — the five transactions. Every transaction type is a class with a single `execute` method, so each rule has exactly one home: nothing outside `Withdraw` needs to know the withdrawal rules. `Deposit` is an intermediate class: cash and check deposits share an amount but differ in when the money becomes spendable. `Transfer` is the only transaction with an outgoing arrow, to the destination account. Inside the course's `Withdraw`, the checks run in this order: **note denominations** first, then the **account**, then the **dispenser**, and the account is debited

only once the notes have been counted out. The reason for that order: if you take the money from the account first and the dispensing then fails, the customer has paid for cash they never received.

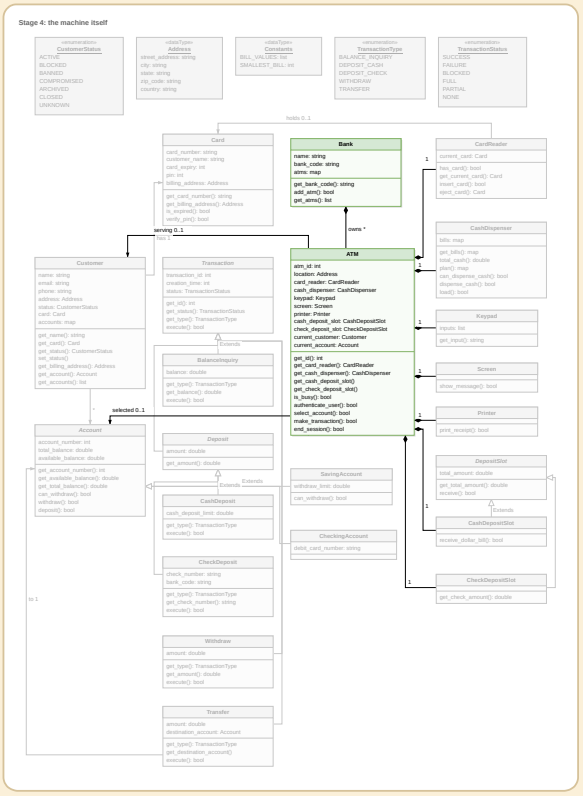


Figure — Stage 4: the ATM owns its parts (composition), refers to the current Customer and Account, and the Bank owns its ATMs (course diagram)

Stage 4 — the machine. The ATM has an **id** (`atmID`) and a **location**, and it **owns** its parts through composition: if the machine is scrapped, its reader, dispenser and screen go with it. Its links to the current `Customer` and `Account` are plain associations that change with each person at the machine. Because the ATM itself holds the current session, "one customer at a time" is enforced by the structure, not just promised in a comment. The `Bank` owns its ATMs and holds all account data; the ATM asks the bank to carry out every customer transaction.

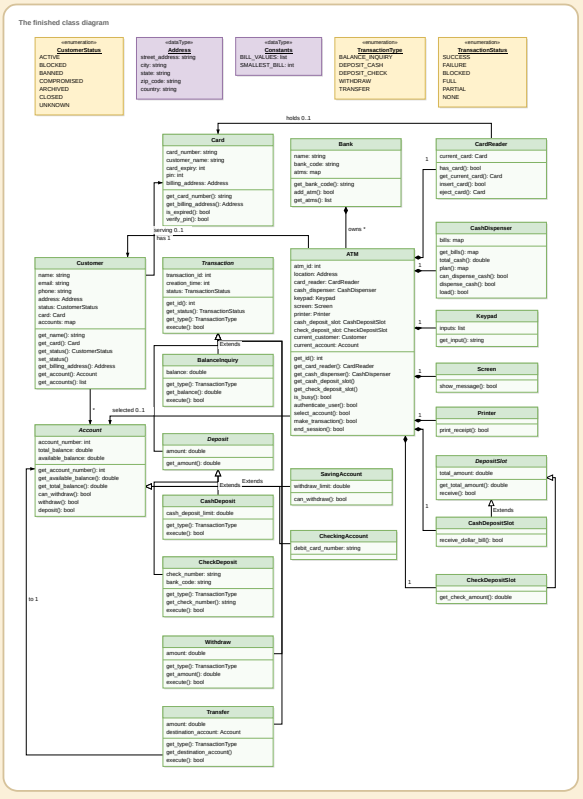
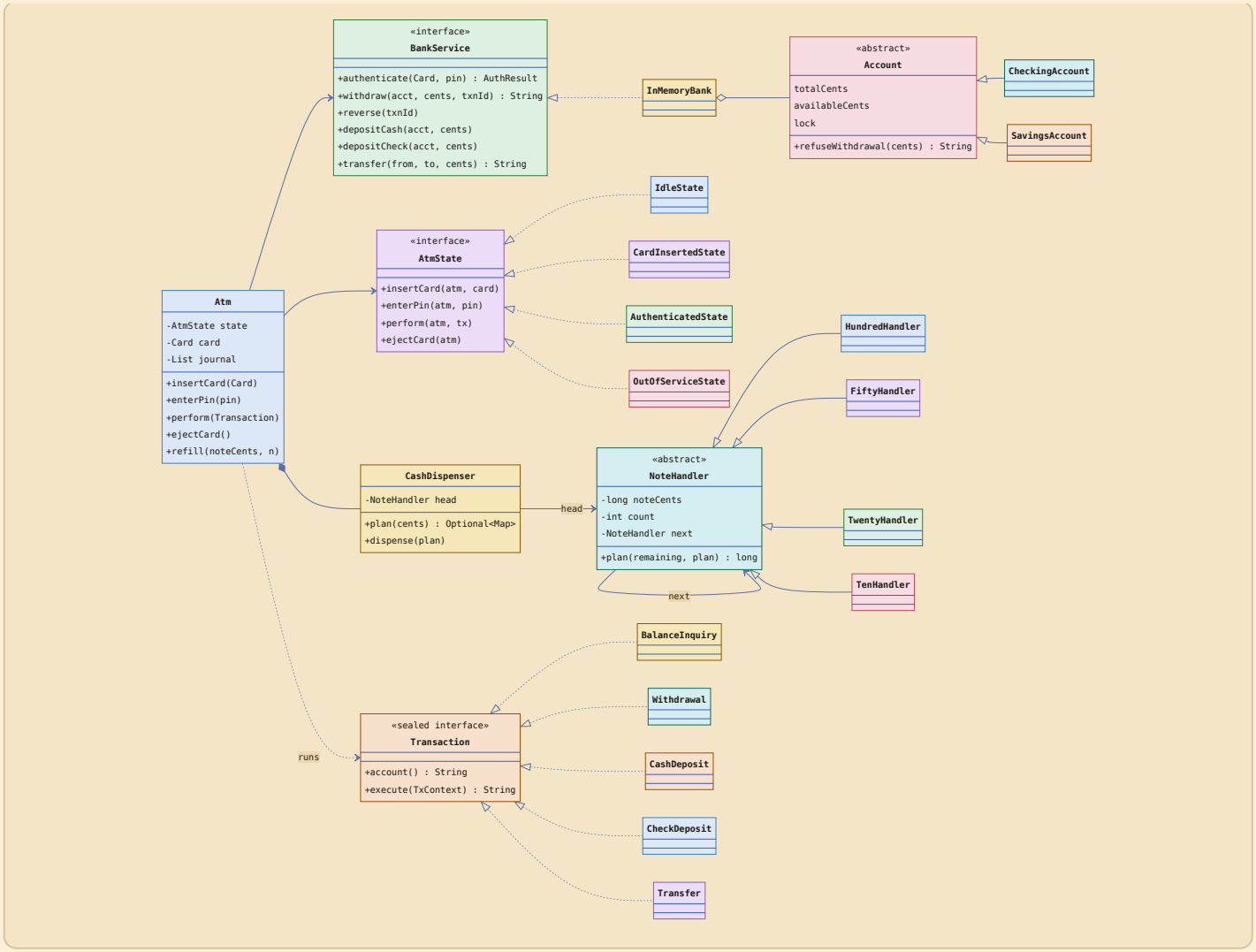


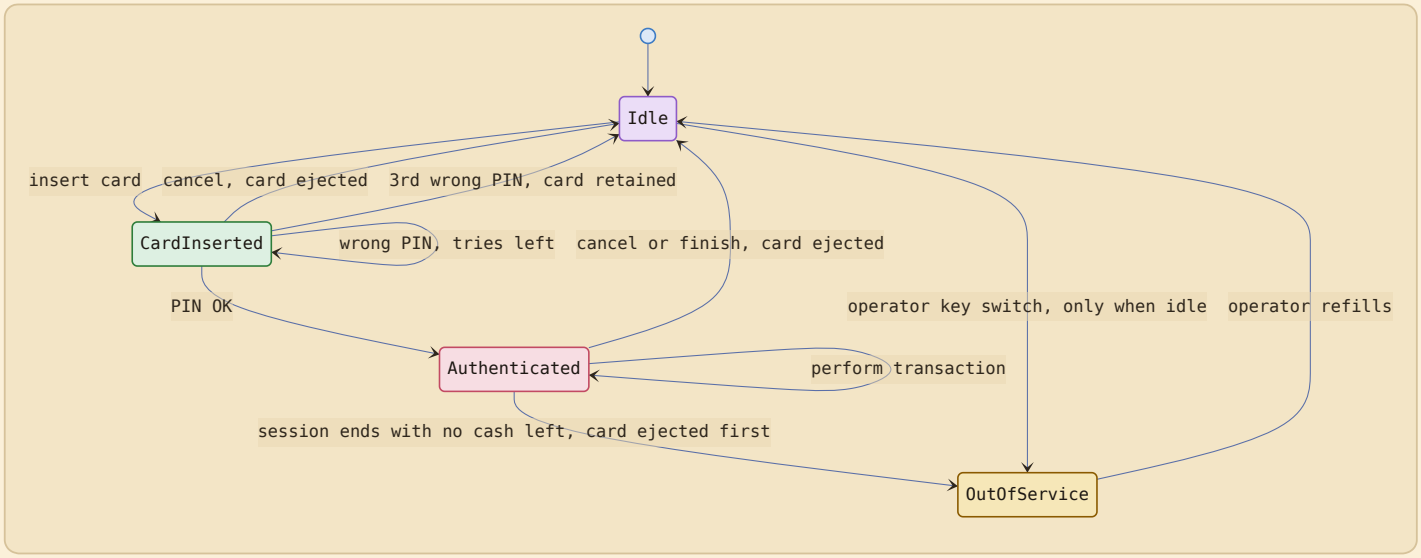
Figure — The finished class diagram of the ATM system (course diagram)

Class diagram of **our Java code**:



Key flows

Session states.



Balance inquiry (the course's sequence diagram):

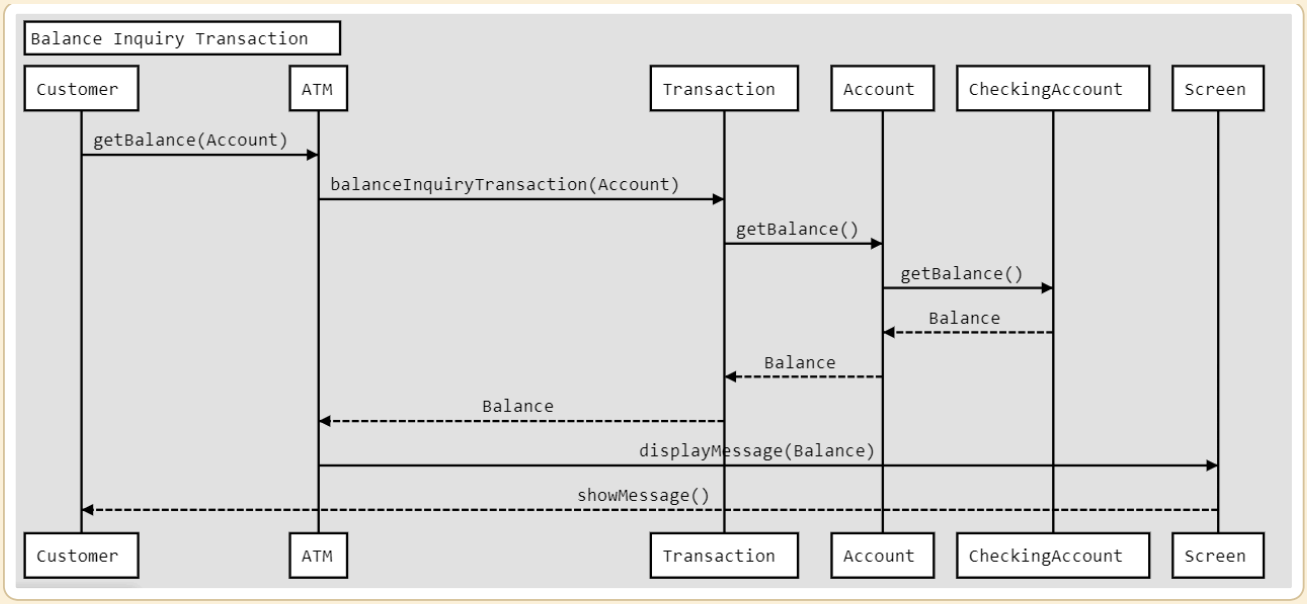
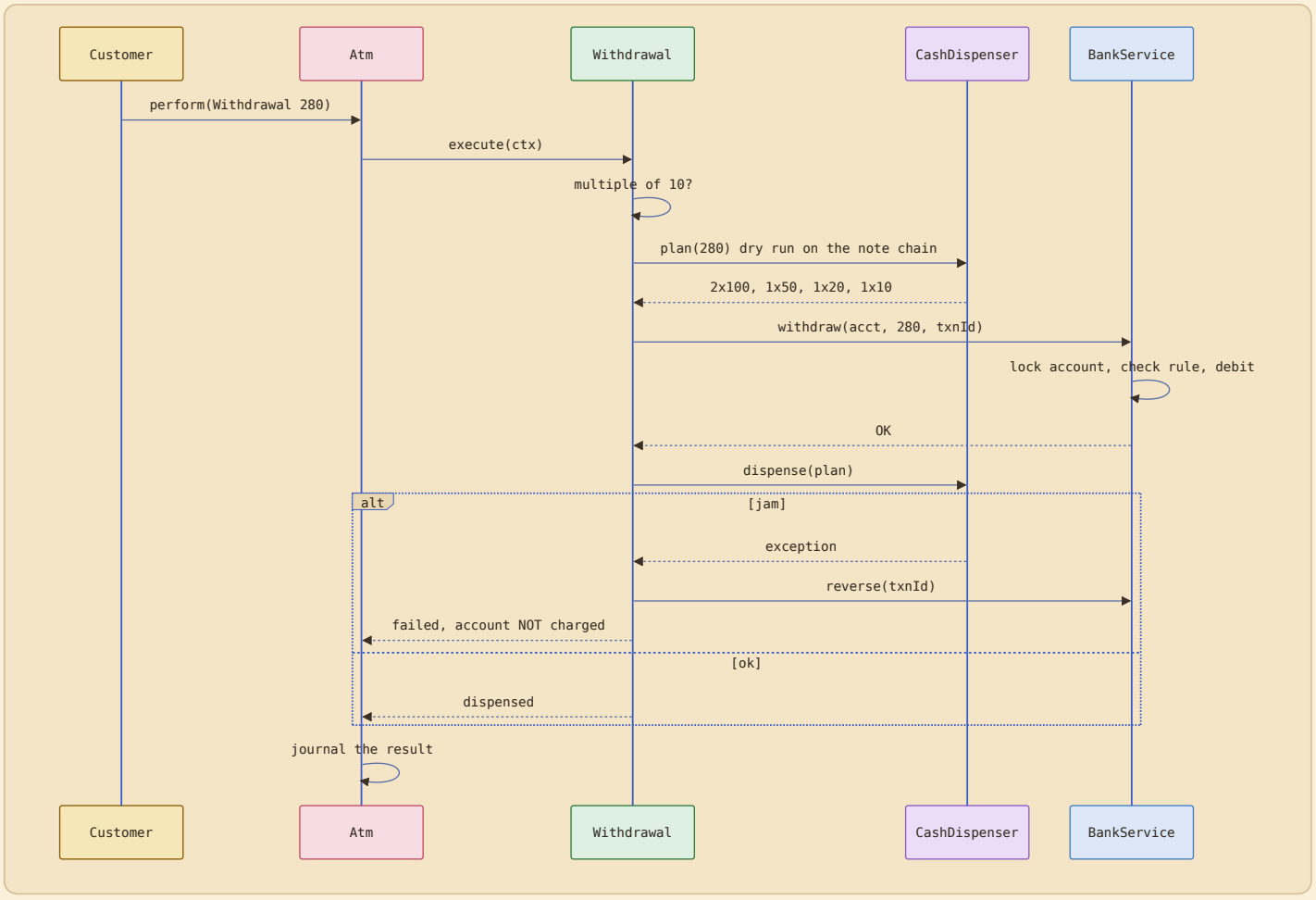


Figure — Sequence diagram of a balance inquiry: Customer → ATM → Transaction → Account → CheckingAccount, and the balance travels back to the Screen (course diagram)

In the sequence above, the ATM never reads the balance itself: it creates a transaction, the transaction asks the account, and the result is shown on the screen.

Withdrawal with compensation (our code):



Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Session flow as state classes	**State**	Insert-card/enter-PIN/perform/eject each mean something different per state; e.g. `perform` in Idle is simply refused	Booleans `cardInserted`, `authenticated` and nested `if`s
Dispenser as a chain of note handlers	**Chain of Responsibility**	Each handler takes what it can and passes the rest on; adding a $5 cassette is one new handler linked into the chain	One method with a hard-coded list of notes
Plan first, then dispense	—	The chain's dry run touches nothing; a failed plan leaves the cassettes intact	Removing notes while searching
Each transaction = one class with `execute`	**Command**	The rules of each transaction live in one place; transactions can be journaled, queued or replayed	A big `switch (type)` in the ATM
`Transaction` and `AuthResult` are **sealed**	Java 17+ sealed types	The compiler knows the full list; `switch` over `AuthResult` is exhaustive	Open hierarchies with `instanceof` chains
ATM talks to a `BankService` interface	Dependency Inversion (depend on an interface, not on a concrete class)	The ATM does not care whether the bank is in-memory, ISO 8583 (the standard card-payment message format) over a network, or a test fake	ATM changes balances directly
Total vs available balance	Domain modelling	Makes the check-hold rule correct, not merely convenient	A single balance
Savings is a subclass	Inheritance for **behaviour**	Its withdrawal rule differs (daily cap), so it overrides one hook	A `type` field and `if (type == SAVINGS)`

> **Debit before or after dispensing?** The course debits **after** the notes are counted out — the safer order for the customer — and admits the weakness: a crash between the two leaves the bank short. Our code does the opposite (debit, then dispense) and adds a **compensating action**: if the dispenser jams, `reverse(txnId)` puts the money back. Both are simplifications of what real systems do: make the pair atomic (a hold/authorisation, then a confirmation after dispensing), and **reconcile** (compare two sets of records afterwards and fix any difference) from the ATM's journal when something goes wrong in between. Say this out loud — it is the "power fails halfway through" follow-up.

> **Cash deposits.** The course assumes all deposits are verified before they count. Many banks make cash that the machine itself has counted available immediately (or up to a limit) and hold checks. Our `depositCash` credits both balances and `depositCheck` credits only the total; changing the policy is a one-line change in the bank, not the ATM.

Java implementation

Package `lld.ch14.atm`. Amounts are cents (`long`); notes are $100/$50/$20/$10.

```java
// File: lld/ch14/atm/Bank.java
package lld.ch14.atm;

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.HexFormat;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.locks.ReentrantLock;

final class Money {
    private Money() {}
    static String fmt(long cents) { return "$%d.%02d".formatted(cents / 100, cents % 100); }
}

/** Total = everything credited. Available = what can be spent now (a check has not cleared yet). */
abstract class Account {
    final String id;
    final ReentrantLock lock = new ReentrantLock();   // one lock per account, shared by every ATM
    long totalCents;
    long availableCents;

    Account(String id, long openingCents) {
        this.id = id;
        this.totalCents = openingCents;
        this.availableCents = openingCents;
    }

    /** Hook that subclasses refine. Called with the lock held. Returns null if allowed, else the reason. */
    String refuseWithdrawal(long cents) {
        return cents > availableCents ? "insufficient funds" : null;
    }

    void recordWithdrawal(long cents) {}
    void reverseWithdrawal(long cents) {}
}

class CheckingAccount extends Account {
    CheckingAccount(String id, long opening) { super(id, opening); }
}

/** The one behavioural difference: savings has a daily withdrawal cap. */
class SavingsAccount extends Account {
    private final long dailyLimitCents;
    private long withdrawnTodayCents;

    SavingsAccount(String id, long opening, long dailyLimitCents) {
        super(id, opening);
        this.dailyLimitCents = dailyLimitCents;
    }

    @Override String refuseWithdrawal(long cents) {
        if (withdrawnTodayCents + cents > dailyLimitCents) return "savings daily limit " +
Money.fmt(dailyLimitCents);
        return super.refuseWithdrawal(cents);
    }

    @Override void recordWithdrawal(long cents) { withdrawnTodayCents += cents; }
    @Override void reverseWithdrawal(long cents) { withdrawnTodayCents -= cents; }
}

record Card(String number, String holder) {}

sealed interface AuthResult permits AuthResult.Ok, AuthResult.WrongPin, AuthResult.Blocked {
    record Ok(List<String> accountIds) implements AuthResult {}
    record WrongPin(int attemptsLeft) implements AuthResult {}
```

```
    record Blocked() implements AuthResult {}
}

/** What the ATM needs from the bank. The ATM never touches balances directly. */
interface BankService {
    AuthResult authenticate(Card card, String pin);
    long availableBalance(String accountId);
    long totalBalance(String accountId);
    String withdraw(String accountId, long cents, String txnId);   // null = OK, else reason
    void reverse(String txnId);
    void depositCash(String accountId, long cents);
    void depositCheck(String accountId, long cents);
    void clearCheck(String accountId, long cents);
    String transfer(String from, String to, long cents);
}

final class InMemoryBank implements BankService {
    private static final int MAX_PIN_TRIES = 3;
    private final Map<String, Account> accounts = new ConcurrentHashMap<>();
    private final Map<String, String> pinHashByCard = new ConcurrentHashMap<>();
    private final Map<String, List<String>> accountsByCard = new ConcurrentHashMap<>();
    private final Map<String, Integer> failedTries = new ConcurrentHashMap<>();
    private final Map<String, Object[]> withdrawals = new ConcurrentHashMap<>();   // txnId -> {accountId, cents}

    void open(Account a) { accounts.put(a.id, a); }

    void issueCard(Card card, String pin, List<String> accountIds) {
        pinHashByCard.put(card.number(), hash(card.number(), pin));
        accountsByCard.put(card.number(), List.copyOf(accountIds));
    }

    @Override public AuthResult authenticate(Card card, String pin) {
        String n = card.number();
        if (failedTries.getOrDefault(n, 0) >= MAX_PIN_TRIES) return new AuthResult.Blocked();
        if (hash(n, pin).equals(pinHashByCard.get(n))) {
            failedTries.remove(n);
            return new AuthResult.Ok(accountsByCard.get(n));
        }
        int tries = failedTries.merge(n, 1, Integer::sum);
        return tries >= MAX_PIN_TRIES ? new AuthResult.Blocked() : new AuthResult.WrongPin(MAX_PIN_TRIES - tries);
    }

    @Override public long availableBalance(String id) { return read(id, true); }
    @Override public long totalBalance(String id) { return read(id, false); }

    @Override public String withdraw(String id, long cents, String txnId) {
        Account a = acc(id);
        a.lock.lock();
        try {                                           // check-then-act under ONE lock: atomic
            String refusal = a.refuseWithdrawal(cents);
            if (refusal != null) return refusal;
            a.availableCents -= cents;
            a.totalCents -= cents;
            a.recordWithdrawal(cents);
            withdrawals.put(txnId, new Object[]{id, cents});
            return null;
        } finally {
            a.lock.unlock();
        }
    }

    /** Compensating action when the cash could not be handed out. */
    @Override public void reverse(String txnId) {
        Object[] w = withdrawals.remove(txnId);
        if (w == null) return;
        Account a = acc((String) w[0]);
        long cents = (Long) w[1];
        a.lock.lock();
        try {
```

```
            a.availableCents += cents;
            a.totalCents += cents;
            a.reverseWithdrawal(cents);
        } finally {
            a.lock.unlock();
        }
    }

    @Override public void depositCash(String id, long cents) { credit(id, cents, true); }
    @Override public void depositCheck(String id, long cents) { credit(id, cents, false); }   // on hold

    @Override public void clearCheck(String id, long cents) {
        Account a = acc(id);
        a.lock.lock();
        try { a.availableCents += cents; } finally { a.lock.unlock(); }
    }

    /** Locks both accounts in a fixed (id) order, so two opposite transfers cannot deadlock. */
    @Override public String transfer(String from, String to, long cents) {
        if (cents <= 0) return "amount must be positive";
        if (from.equals(to)) return "same account";
        Account a = acc(from), b = acc(to);
        Account first = from.compareTo(to) < 0 ? a : b;
        Account second = first == a ? b : a;
        first.lock.lock();
        try {
            second.lock.lock();
            try {
                if (cents > a.availableCents) return "insufficient funds";
                a.availableCents -= cents;
                a.totalCents -= cents;
                b.availableCents += cents;
                b.totalCents += cents;
                return null;
            } finally {
                second.lock.unlock();
            }
        } finally {
            first.lock.unlock();
        }
    }

    private void credit(String id, long cents, boolean available) {
        if (cents <= 0) throw new IllegalArgumentException("deposit must be positive");
        Account a = acc(id);
        a.lock.lock();
        try {
            a.totalCents += cents;
            if (available) a.availableCents += cents;
        } finally {
            a.lock.unlock();
        }
    }

    private long read(String id, boolean available) {
        Account a = acc(id);
        a.lock.lock();
        try { return available ? a.availableCents : a.totalCents; } finally { a.lock.unlock(); }
    }

    private Account acc(String id) {
        Account a = accounts.get(id);
        if (a == null) throw new IllegalArgumentException("no account " + id);
        return a;
    }

    /** Demo only. Real banks never see the clear PIN: the PIN pad encrypts it and an HSM verifies it. */
    private static String hash(String card, String pin) {
        try {
```

```
            byte[] d = MessageDigest.getInstance("SHA-256").digest((card + ":" +
pin).getBytes(StandardCharsets.UTF_8));
            return HexFormat.of().formatHex(d);
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException(e);
        }
    }
}
```

```java
// File: lld/ch14/atm/CashDispenser.java
package lld.ch14.atm;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Optional;

/**
 * Chain of Responsibility: each handler owns one note value. It takes as many of its notes
 * as it can for the amount, then passes whatever is left to the next (smaller) handler.
 */
abstract class NoteHandler {
    private final long noteCents;
    private int count;
    private NoteHandler next;

    NoteHandler(long noteCents, int count) {
        this.noteCents = noteCents;
        this.count = count;
    }

    NoteHandler linkTo(NoteHandler next) {
        this.next = next;
        return next;
    }

    /** Dry run: fills 'plan', returns the amount that nobody in the chain could cover. */
    long plan(long remaining, Map<Long, Integer> plan) {
        int use = (int) Math.min(count, remaining / noteCents);
        if (use > 0) plan.put(noteCents, use);
        long left = remaining - use * noteCents;
        return (left == 0 || next == null) ? left : next.plan(left, plan);
    }

    void take(Map<Long, Integer> plan) {
        count -= plan.getOrDefault(noteCents, 0);
        if (next != null) next.take(plan);
    }

    long total() { return noteCents * count + (next == null ? 0 : next.total()); }
    void add(int n) { count += n; }
    long noteCents() { return noteCents; }
    NoteHandler next() { return next; }
}

final class HundredHandler extends NoteHandler { HundredHandler(int n) { super(100_00, n); } }
final class FiftyHandler extends NoteHandler { FiftyHandler(int n) { super(50_00, n); } }
final class TwentyHandler extends NoteHandler { TwentyHandler(int n) { super(20_00, n); } }
final class TenHandler extends NoteHandler { TenHandler(int n) { super(10_00, n); } }

final class CashDispenser {
    private final NoteHandler head;
    private boolean jamNext;                      // hardware fault injection for the demo

    CashDispenser(int hundreds, int fifties, int twenties, int tens) {
        head = new HundredHandler(hundreds);
        head.linkTo(new FiftyHandler(fifties)).linkTo(new TwentyHandler(twenties)).linkTo(new TenHandler(tens));
    }

    /** Plan first, touch nothing. Empty = this amount cannot be made from the notes loaded. */
    synchronized Optional<Map<Long, Integer>> plan(long cents) {
        Map<Long, Integer> plan = new LinkedHashMap<>();
        return head.plan(cents, plan) == 0 ? Optional.of(plan) : Optional.empty();
    }

    /** Physically count out the notes. May fail (jam), in which case nothing leaves the machine. */
    synchronized void dispense(Map<Long, Integer> plan) {
        if (jamNext) { jamNext = false; throw new IllegalStateException("note jam"); }
```

```
        head.take(plan);
    }

    synchronized void refill(long noteCents, int n) {
        for (NoteHandler h = head; h != null; h = h.next()) if (h.noteCents() == noteCents) h.add(n);
    }

    synchronized long total() { return head.total(); }
    synchronized void jamNextDispense() { jamNext = true; }
}
```

```java
// File: lld/ch14/atm/Transactions.java
package lld.ch14.atm;

import java.util.Map;
import java.util.stream.Collectors;

/** Everything a transaction may use. */
record TxContext(BankService bank, CashDispenser dispenser, String txnId) {}

/** Each transaction is one class with one execute(): Command pattern. Sealed = the full list is known. */
sealed interface Transaction permits BalanceInquiry, Withdrawal, CashDeposit, CheckDeposit, Transfer {
    String account();
    String execute(TxContext ctx);
}

record BalanceInquiry(String account) implements Transaction {
    public String execute(TxContext c) {
        return "Balance %s: available %s, total %s".formatted(account,
                Money.fmt(c.bank().availableBalance(account)), Money.fmt(c.bank().totalBalance(account)));
    }
}

record Withdrawal(String account, long cents) implements Transaction {
    static final long SMALLEST_NOTE = 10_00;

    public String execute(TxContext c) {
        if (cents <= 0 || cents % SMALLEST_NOTE != 0) return "Withdrawal refused: amount must be a multiple of
$10";
        var plan = c.dispenser().plan(cents);                                   // 1. can the notes make it?
        if (plan.isEmpty()) return "Withdrawal refused: ATM cannot make " + Money.fmt(cents) + " from its notes";
        String refusal = c.bank().withdraw(account, cents, c.txnId());          // 2. atomic debit at the bank
        if (refusal != null) return "Withdrawal refused: " + refusal;
        try {
            c.dispenser().dispense(plan.get());                                 // 3. hand out the cash
        } catch (IllegalStateException hardwareFault) {
            c.bank().reverse(c.txnId());                                        // compensate: undo the debit
            return "Withdrawal failed (" + hardwareFault.getMessage() + "), account NOT charged";
        }
        return "Dispensed " + Money.fmt(cents) + " as " + describe(plan.get());
    }

    private static String describe(Map<Long, Integer> plan) {
        return plan.entrySet().stream().map(e -> e.getValue() + "x$" + e.getKey() / 100)
                .collect(Collectors.joining(" + "));
    }
}

record CashDeposit(String account, long cents) implements Transaction {
    public String execute(TxContext c) {
        c.bank().depositCash(account, cents);
        return "Cash deposit " + Money.fmt(cents) + " credited";
    }
}

record CheckDeposit(String account, long cents) implements Transaction {
    public String execute(TxContext c) {
        c.bank().depositCheck(account, cents);
        return "Check " + Money.fmt(cents) + " received: on hold until the bank clears it";
    }
}

record Transfer(String account, String toAccount, long cents) implements Transaction {
    public String execute(TxContext c) {
        String refusal = c.bank().transfer(account, toAccount, cents);
        return refusal == null ? "Transferred " + Money.fmt(cents) + " to " + toAccount
                               : "Transfer refused: " + refusal;
```

```
    }
}
```

```java
// File: lld/ch14/atm/States.java
package lld.ch14.atm;

/** State pattern: what each button does depends on where the session is. */
interface AtmState {
    default void insertCard(Atm atm, Card card) { atm.screen("Card slot is not ready."); }
    default void enterPin(Atm atm, String pin) { atm.screen("Please insert your card first."); }
    default void perform(Atm atm, Transaction tx) { atm.screen("Please sign in first."); }
    default void ejectCard(Atm atm) { atm.screen("No card to eject."); }
    String name();
}

final class IdleState implements AtmState {
    static final IdleState INSTANCE = new IdleState();

    @Override public void insertCard(Atm atm, Card card) {
        atm.acceptCard(card);
        atm.screen("Hello " + card.holder() + ". Enter PIN.");
        atm.setState(CardInsertedState.INSTANCE);
    }
    @Override public String name() { return "IDLE"; }
}

final class CardInsertedState implements AtmState {
    static final CardInsertedState INSTANCE = new CardInsertedState();

    @Override public void enterPin(Atm atm, String pin) {
        switch (atm.bank().authenticate(atm.card(), pin)) {
            case AuthResult.Ok ok -> {
                atm.startSession(ok.accountIds());
                atm.screen("PIN OK. Accounts: " + ok.accountIds());
                atm.setState(AuthenticatedState.INSTANCE);
            }
            case AuthResult.WrongPin w -> atm.screen("Wrong PIN. Tries left: " + w.attemptsLeft());
            case AuthResult.Blocked b -> {
                atm.screen("Too many wrong PINs. Card retained. Contact your bank.");
                atm.retainCard();
                atm.finishSession();
            }
        }
    }
    @Override public void ejectCard(Atm atm) { atm.returnCard(); atm.finishSession(); }
    @Override public String name() { return "CARD_INSERTED"; }
}

final class AuthenticatedState implements AtmState {
    static final AuthenticatedState INSTANCE = new AuthenticatedState();

    @Override public void perform(Atm atm, Transaction tx) {
        if (!atm.sessionAccounts().contains(tx.account())) {
            atm.screen("Account " + tx.account() + " is not linked to this card.");
            return;
        }
        atm.run(tx);
    }
    @Override public void ejectCard(Atm atm) { atm.returnCard(); atm.finishSession(); }
    @Override public String name() { return "AUTHENTICATED"; }
}

final class OutOfServiceState implements AtmState {
    static final OutOfServiceState INSTANCE = new OutOfServiceState();

    @Override public void insertCard(Atm atm, Card card) { atm.screen("Out of service. Card returned."); }
    @Override public String name() { return "OUT_OF_SERVICE"; }
}
```

```java
// File: lld/ch14/atm/Atm.java
package lld.ch14.atm;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

/** One machine: owns its parts, runs one session at a time, forwards buttons to its state. */
final class Atm {
    private static final AtomicLong TXN_SEQ = new AtomicLong(1);
    private final String id;
    private final BankService bank;
    private final CashDispenser dispenser;
    private final boolean verbose;
    private final List<String> journal = new ArrayList<>();   // operator / bank-manager audit log
    private AtmState state = IdleState.INSTANCE;
    private Card card;                                        // the card reader's slot
    private List<String> sessionAccounts = List.of();

    Atm(String id, BankService bank, CashDispenser dispenser, boolean verbose) {
        this.id = id;
        this.bank = bank;
        this.dispenser = dispenser;
        this.verbose = verbose;
    }

    // ----- customer buttons (one customer at a time: synchronized) -----
    synchronized void insertCard(Card c) { state.insertCard(this, c); }
    synchronized void enterPin(String pin) { state.enterPin(this, pin); }
    synchronized void perform(Transaction tx) { state.perform(this, tx); }
    synchronized void ejectCard() { state.ejectCard(this); }

    // ----- operator -----
    /** Key switch. The ATM must not shut down while serving a customer, so this only works when idle. */
    synchronized boolean setOutOfService() {
        if (state != IdleState.INSTANCE && state != OutOfServiceState.INSTANCE) {
            journal("operator: switch-off refused, session in progress");
            return false;
        }
        state = OutOfServiceState.INSTANCE;
        journal("operator: out of service");
        return true;
    }
    synchronized void refill(long noteCents, int n) {
        dispenser.refill(noteCents, n);
        journal("operator: refilled " + n + " x " + Money.fmt(noteCents));
        if (state == OutOfServiceState.INSTANCE) state = IdleState.INSTANCE;
    }

    // ----- helpers for states -----
    void setState(AtmState s) { state = s; }
    BankService bank() { return bank; }
    Card card() { return card; }
    List<String> sessionAccounts() { return sessionAccounts; }
    void acceptCard(Card c) { card = c; journal("card in " + mask(c)); }
    void startSession(List<String> accounts) { sessionAccounts = accounts; }

    void returnCard() {
        screen("Card ejected. Goodbye.");
        journal("card out " + mask(card));
        endSession();
    }

    void retainCard() { journal("card RETAINED " + mask(card)); endSession(); }

    void run(Transaction tx) {
        String txnId = id + "-" + TXN_SEQ.getAndIncrement();
        String result = tx.execute(new TxContext(bank, dispenser, txnId));
```

```
        screen(result);
        journal(txnId + " " + tx.getClass().getSimpleName() + ": " + result);
    }

    /** End of a session: back to Idle, or OutOfService if the cash ran out (never mid-session, so the card is
returned first). */
    void finishSession() {
        if (dispenser.total() == 0) {
            state = OutOfServiceState.INSTANCE;
            journal("cash empty: out of service");
        } else {
            state = IdleState.INSTANCE;
        }
    }

    void screen(String msg) { if (verbose) System.out.println("   [" + id + " screen] " + msg); }
    List<String> journal() { return List.copyOf(journal); }
    String stateName() { return state.name(); }
    long cashLeft() { return dispenser.total(); }
    void dispenserJamForDemo() { dispenser.jamNextDispense(); }

    private void journal(String line) { journal.add(line); }
    private void endSession() { card = null; sessionAccounts = List.of(); }
    private static String mask(Card c) { return "****" + c.number().substring(c.number().length() - 4); }
}
```

```java
// File: lld/ch14/atm/Main.java
package lld.ch14.atm;

import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        InMemoryBank bank = new InMemoryBank();
        bank.open(new CheckingAccount("CHK-1", 1_000_00));
        bank.open(new SavingsAccount("SAV-1", 2_000_00, 100_00));
        bank.open(new CheckingAccount("CHK-2", 50_00));
        Card alice = new Card("4000123412341111", "Alice");
        Card bob = new Card("4000999988882222", "Bob");
        bank.issueCard(alice, "1234", List.of("CHK-1", "SAV-1"));
        bank.issueCard(bob, "9999", List.of("CHK-2"));

        Atm atm = new Atm("ATM-7", bank, new CashDispenser(2, 2, 5, 3), true);
        System.out.println("Cash loaded: " + Money.fmt(atm.cashLeft()));

        System.out.println("\n== Alice: wrong PIN, then right PIN ==");
        atm.perform(new BalanceInquiry("CHK-1"));      // not signed in
        atm.insertCard(alice);
        atm.enterPin("0000");
        atm.enterPin("1234");
        atm.perform(new BalanceInquiry("CHK-1"));
        atm.perform(new Withdrawal("CHK-1", 280_00));   // 2x100 + 1x50 + 1x20 + 1x10
        atm.perform(new Withdrawal("CHK-1", 85_00));    // not a multiple of $10
        atm.perform(new Withdrawal("SAV-1", 140_00));   // over the $100 savings daily cap
        atm.perform(new CheckDeposit("CHK-1", 500_00));
        atm.perform(new BalanceInquiry("CHK-1"));       // total grows, available does not
        atm.perform(new Transfer("SAV-1", "CHK-2", 150_00));
        atm.perform(new BalanceInquiry("CHK-2"));       // not Alice's account
        System.out.println("   (dispenser jams on the next withdrawal)");
        atm.dispenserJamForDemo();
        atm.perform(new Withdrawal("CHK-1", 100_00));
        atm.perform(new BalanceInquiry("CHK-1"));
        atm.ejectCard();

        System.out.println("\n== Bank clears the check overnight ==");
        bank.clearCheck("CHK-1", 500_00);
        System.out.println("   CHK-1 available now " + Money.fmt(bank.availableBalance("CHK-1")));

        System.out.println("\n== Bob forgets his PIN three times ==");
        atm.insertCard(bob);
        atm.enterPin("1111");
        atm.enterPin("2222");
        atm.enterPin("3333");
        Atm branch = new Atm("ATM-9", bank, new CashDispenser(1, 1, 1, 1), true);
        branch.insertCard(bob);                          // same card number at another ATM
        branch.enterPin("9999");                         // blocked at the bank, even with the right PIN

        System.out.println("\n== Two ATMs, one account, same instant: $300 each from $500 ==");
        bank.open(new CheckingAccount("JOINT", 500_00));
        Card c1 = new Card("4111000000000001", "Pat"), c2 = new Card("4111000000000002", "Sam");
        bank.issueCard(c1, "1111", List.of("JOINT"));
        bank.issueCard(c2, "2222", List.of("JOINT"));
        Atm east = new Atm("EAST", bank, new CashDispenser(10, 0, 0, 0), false);
        Atm west = new Atm("WEST", bank, new CashDispenser(10, 0, 0, 0), false);
        east.insertCard(c1); east.enterPin("1111");
        west.insertCard(c2); west.enterPin("2222");
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(2)) {
            for (Atm a : List.of(east, west)) pool.submit(() -> { go.await(); a.perform(new Withdrawal("JOINT",
300_00)); return null; });
```

```
            go.countDown();
        }
        long dispensed = (10 * 100_00 - east.cashLeft()) + (10 * 100_00 - west.cashLeft());
        System.out.println("   cash handed out " + Money.fmt(dispensed) + ", JOINT left " +
Money.fmt(bank.availableBalance("JOINT")));

        System.out.println("\n== 2,000 opposite transfers in parallel (ordered locking: no deadlock) ==");
        bank.open(new CheckingAccount("A", 1_000_00));
        bank.open(new CheckingAccount("B", 1_000_00));
        AtomicInteger ok = new AtomicInteger();
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            for (int i = 0; i < 1000; i++) {
                pool.submit(() -> { if (bank.transfer("A", "B", 1_00) == null) ok.incrementAndGet(); });
                pool.submit(() -> { if (bank.transfer("B", "A", 1_00) == null) ok.incrementAndGet(); });
            }
        }
        System.out.println("   successful=" + ok + ", A+B=" + Money.fmt(bank.totalBalance("A") +
bank.totalBalance("B")));

        System.out.println("\n== ATM-9 runs out of cash; the operator refills it ==");
        branch.insertCard(alice);
        branch.enterPin("1234");
        System.out.println("   operator key switch mid-session accepted? " + branch.setOutOfService());
        branch.perform(new Withdrawal("CHK-1", 180_00));  // takes every note in ATM-9
        System.out.println("   state after the last notes: " + branch.stateName() + " (card still inside)");
        branch.ejectCard();
        System.out.println("   state after the session: " + branch.stateName());
        branch.insertCard(alice);
        branch.refill(20_00, 10);
        System.out.println("   state after refill: " + branch.stateName() + ", cash " +
Money.fmt(branch.cashLeft()));

        System.out.println("\n== Journal of ATM-7 (for the operator and bank manager) ==");
        atm.journal().forEach(l -> System.out.println("   " + l));
    }
}
```

How the code works, part by part

- **PIN handling:** the bank stores a SHA-256 hash (a standard one-way function that turns any input into a fixed-length fingerprint that cannot practically be reversed) of card+PIN and counts failures; the third failure returns `Blocked` and the ATM **retains** the card. (The course's code compares the PIN inside `Card` and flags that as a simplification. Real banks never see the clear PIN: the PIN pad encrypts it into a PIN block and a hardware security module (HSM) verifies it. A plain hash of a 4-digit PIN is also easy to brute-force (try every possible value until one matches), which is one more reason it is demo-only.)
- **Account locks:** each `Account` has its own `ReentrantLock`, shared by every ATM. `withdraw` does check-and-debit under that lock, so two ATMs cannot both spend the same $500.
- **Transfers** lock both accounts in **id order**. Two opposite transfers (A→B and B→A) therefore always lock in the same order and cannot deadlock (Chapter 13).
- **Chain of Responsibility:** `HundredHandler → FiftyHandler → TwentyHandler → TenHandler`. `plan` fills a map and returns the uncovered remainder; `take` removes the planned notes.
- **Command:** each transaction record has `execute(TxContext)`. `AuthenticatedState.perform` first checks the transaction's account belongs to this card.
- **Journal:** the `Atm` logs card in/out, retained cards, every transaction result, operator actions and "cash empty".
- **Out of service:** `finishSession` (called after every eject or retain) chooses Idle or OutOfService, so the machine never changes to OutOfService while a card is still inside. The operator's key switch (`setOutOfService`) is refused during a session.
- **Input checks:** the bank refuses transfers and deposits of zero or negative amounts.

Output:

```
Cash loaded: $430.00

== Alice: wrong PIN, then right PIN ==
   [ATM-7 screen] Please sign in first.
   [ATM-7 screen] Hello Alice. Enter PIN.
   [ATM-7 screen] Wrong PIN. Tries left: 2
   [ATM-7 screen] PIN OK. Accounts: [CHK-1, SAV-1]
   [ATM-7 screen] Balance CHK-1: available $1000.00, total $1000.00
   [ATM-7 screen] Dispensed $280.00 as 2x$100 + 1x$50 + 1x$20 + 1x$10
   [ATM-7 screen] Withdrawal refused: amount must be a multiple of $10
   [ATM-7 screen] Withdrawal refused: savings daily limit $100.00
   [ATM-7 screen] Check $500.00 received: on hold until the bank clears it
   [ATM-7 screen] Balance CHK-1: available $720.00, total $1220.00
   [ATM-7 screen] Transferred $150.00 to CHK-2
   [ATM-7 screen] Account CHK-2 is not linked to this card.
   (dispenser jams on the next withdrawal)
   [ATM-7 screen] Withdrawal failed (note jam), account NOT charged
   [ATM-7 screen] Balance CHK-1: available $720.00, total $1220.00
   [ATM-7 screen] Card ejected. Goodbye.

== Bank clears the check overnight ==
   CHK-1 available now $1220.00

== Bob forgets his PIN three times ==
   [ATM-7 screen] Hello Bob. Enter PIN.
   [ATM-7 screen] Wrong PIN. Tries left: 2
   [ATM-7 screen] Wrong PIN. Tries left: 1
   [ATM-7 screen] Too many wrong PINs. Card retained. Contact your bank.
   [ATM-9 screen] Hello Bob. Enter PIN.
   [ATM-9 screen] Too many wrong PINs. Card retained. Contact your bank.

== Two ATMs, one account, same instant: $300 each from $500 ==
   cash handed out $300.00, JOINT left $200.00

== 2,000 opposite transfers in parallel (ordered locking: no deadlock) ==
   successful=2000, A+B=$2000.00

== ATM-9 runs out of cash; the operator refills it ==
   [ATM-9 screen] Hello Alice. Enter PIN.
   [ATM-9 screen] PIN OK. Accounts: [CHK-1, SAV-1]
   operator key switch mid-session accepted? false
   [ATM-9 screen] Dispensed $180.00 as 1x$100 + 1x$50 + 1x$20 + 1x$10
   state after the last notes: AUTHENTICATED (card still inside)
   [ATM-9 screen] Card ejected. Goodbye.
   state after the session: OUT_OF_SERVICE
   [ATM-9 screen] Out of service. Card returned.
   state after refill: IDLE, cash $200.00

== Journal of ATM-7 (for the operator and bank manager) ==
   card in ****1111
   ATM-7-1 BalanceInquiry: Balance CHK-1: available $1000.00, total $1000.00
   ATM-7-2 Withdrawal: Dispensed $280.00 as 2x$100 + 1x$50 + 1x$20 + 1x$10
   ATM-7-3 Withdrawal: Withdrawal refused: amount must be a multiple of $10
   ATM-7-4 Withdrawal: Withdrawal refused: savings daily limit $100.00
   ATM-7-5 CheckDeposit: Check $500.00 received: on hold until the bank clears it
   ATM-7-6 BalanceInquiry: Balance CHK-1: available $720.00, total $1220.00
   ATM-7-7 Transfer: Transferred $150.00 to CHK-2
   ATM-7-8 Withdrawal: Withdrawal failed (note jam), account NOT charged
   ATM-7-9 BalanceInquiry: Balance CHK-1: available $720.00, total $1220.00
   card out ****1111
   card in ****2222
   card RETAINED ****2222
```

Read the output: $280 comes out as 2×$100 + $50 + $20 + $10; $85 is refused (not a multiple of $10); the savings cap refuses $140; after the $500 check, *total* grows but *available* does not until the bank clears it; the jam leaves the balance unchanged; Bob's card is blocked at the bank, so even another ATM refuses the right PIN; two ATMs racing on a $500 joint account (one account shared by two people) hand out only $300; 2,000 opposite

transfers finish with no deadlock and the total conserved; ATM-9 refuses the key switch mid-session, pays out its last $180, ejects the card, and only then goes out of service until the operator refills it.

Concurrency & edge cases

Situation	Risk	Handling
Two ATMs withdraw from one account at once	Overdraft (check-then-act race)	Check and debit under the **account lock** at the bank
A→B and B→A transfers at once	**Deadlock** (each holds one lock, waits for the other)	Lock in a global order (account id)
Dispenser jams after the debit	Customer charged, no cash	Compensating `reverse(txnId)`; journal entry for reconciliation
Amount not makeable from loaded notes	Partial dispense	Dry-run `plan` first; refuse before touching the bank
Greedy chain misses a combination	e.g. $60 with one $50 and three $20 but no $10: greedy takes $50, then cannot make $10, though 3×$20 works	Acceptable for standard cassettes; mention a DP planner (as in 14.2) if asked
Wrong PIN many times	Brute force	3 tries, then block at the bank (so another ATM also refuses) and retain the card
Customer walks away mid-session	Next person uses the open session	Session timeout that ejects/retains the card (extension)
Transaction on an account not linked to the card	Unauthorised access	`AuthenticatedState` checks `sessionAccounts`
Cash runs out	Customer's card trapped, or the next customer gets refused mid-session	The session finishes normally (card ejected), then `finishSession` moves the ATM to `OutOfService`; an operator refill returns it to Idle
Operator turns the key mid-session	The machine shuts down while serving	`setOutOfService` is refused unless the ATM is idle (requirement: never shut down while serving)

Extensions & follow-up questions

1. **"What if the machine cannot make the amount?"** — The dispenser's plan fails and the withdrawal is refused before the bank is called.
2. **"What if the power fails halfway through a withdrawal?"** — Use a two-phase approach: authorise (hold) at the bank, dispense, then confirm; on restart the ATM reads its journal and either confirms or reverses. The bank reconciles against the journal.
3. **"Add a daily withdrawal limit for all accounts."** — Move the cap into `Account` or into a policy object the bank consults; the ATM does not change.
4. **"Receipts and PIN change?"** — Two more `Transaction` records (`PinChange`, and a `Printer` part that formats the journal entry).
5. **"Other banks' cards?"** — `BankService` gets a routing implementation that forwards to a card network (such as Visa or Mastercard, which passes requests between banks) for foreign cards (a Proxy — a stand-in object — or an Adapter — a translator between two interfaces — in front of the network).
6. **"Cardless / mobile withdrawals?"** — A new state path: `enterOneTimeCode` instead of `insertCard` → `enterPin`. The transactions do not change.
7. **"Bank manager reports?"** — Aggregate the journal (total deposits, total withdrawals, cash left = `dispenser.total()`).

Real-world example

Real ATMs (NCR, Diebold Nixdorf and others) control their devices through the **CEN/XFS** standard, which gives the card reader, PIN pad, dispenser and printer each a standard software interface — the same "parts behind interfaces" idea as our design. Messages between ATM networks and banks commonly use the **ISO 8583** message format, and PIN blocks are defined by **ISO 9564** and verified inside HSMs. Cash sits in **cassettes** of one denomination each, and the dispenser works out a note mix from what the cassettes hold, like our handler chain.

After a dispense error, ATMs routinely reverse the transaction, and banks reconcile disputed withdrawals from the machine's electronic journal.

In the interview

Plan for 45 minutes: 5 min parts, actors and transactions → 5 min total vs available balance → 10 min state diagram (Idle → CardInserted → Authenticated) → 15 min code: `Withdrawal.execute` (plan → debit → dispense → compensate) and the dispenser chain → 10 min concurrency (account lock, ordered transfer locks) and follow-ups.

> **30-second answer:** "The ATM has parts behind interfaces, a session modelled with the State pattern, and five transactions as Command objects with one `execute` each. It never touches balances: it calls a `BankService`. Accounts have total and available balances so checks can be held. Withdrawal plans the notes first through a Chain of Responsibility over the cassettes, debits the account atomically at the bank, dispenses, and reverses the debit if the hardware fails. The bank locks per account, and transfers lock in id order to avoid deadlock."

Follow-ups with answers

- *"Where is the PIN checked?"* — At the bank (in reality in an HSM, from an encrypted PIN block). The ATM never stores or compares it.
- *"How do you stop two ATMs overdrawing a joint account?"* — The check and the debit happen together under the account's lock at the bank (or in one database transaction).
- *"Why Chain of Responsibility for notes?"* — Each denomination handler decides how much it can cover and passes the rest on; adding or removing a cassette changes the chain, not the algorithm.

Self-check

1. **Q:** Why two balances? **A:** A deposited check increases the total immediately but must not be spendable until it clears; available tracks what can be withdrawn.
2. **Q:** Which order of debit and dispense is safe? **A:** Neither alone is safe against a crash; use a hold + confirm (or debit + compensating reverse) and reconcile from the journal.
3. **Q:** How does the design avoid deadlock in transfers? **A:** Both accounts are always locked in the same global order (by id).
4. **Q:** What does the dispenser's dry run protect against? **A:** Removing notes or debiting the account for an amount that cannot be made exactly.
5. **Q:** Why is `SavingsAccount` a subclass rather than a type field? **A:** It has different behaviour (a daily cap), which inheritance models by overriding one hook.
6. **Q:** What happens after the third wrong PIN? **A:** The bank blocks the card (so every ATM refuses it) and this ATM retains it.

TL;DR

- **State** for the session, **Command** for the five transactions, **Chain of Responsibility** for the note cassettes, `BankService` interface for the bank.
- **Total vs available** balance makes check deposits correct.
- Withdrawal = **plan notes → atomic debit at the bank → dispense → reverse on failure**, all journaled.
- Concurrency: **per-account lock**, transfers lock in **id order**, PIN attempts limited at the bank.

14.5 Design an Elevator System

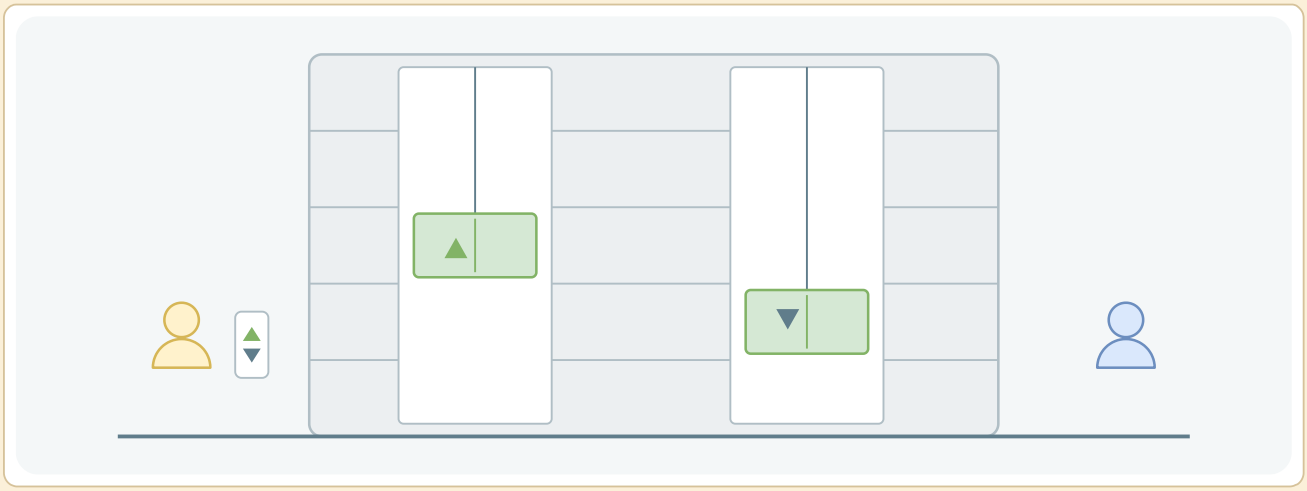


Figure — An elevator bank in a multi-floor building: several cars, hall buttons on each floor and displays (course diagram)

An **elevator** (lift) moves people up and down a building. A real building rarely has just one, and its cars are run as a **group** by a shared controller. That group control is what turns this from a small coding task into a design question. Two decisions carry the interview: **which car should answer a call from a floor** (dispatching) and **in what order a single car should visit its stops** (scheduling).

Problem & clarifying questions

Question	Assumed answer
How many floors and cars?	Many floors (0 … N-1), several cars serving all floors.
Which buttons?	Hall panel on each floor with UP and DOWN (no UP on the top floor, no DOWN on the ground floor); a car panel with one button per floor plus door-open and door-close.
Displays?	Inside each car: floor, direction, load. On each floor: one display per car showing where it is.
Scheduling rule?	A car serves every stop ahead in its direction before turning (the elevator algorithm; we use **LOOK**).
Dispatching rule?	Nearest suitable car; swappable.
Capacity?	Each car has a maximum load; an overloaded car does not move and says so.
Maintenance and emergencies?	A car can be taken out of service; fire alarm recalls all cars to the lobby.
Real time or simulated?	Simulated: one `step()` = one action. This makes the design testable without a clock.

Requirements

Functional (the course's twelve, plus emergency)

1. The building has **multiple floors** and **more than one elevator** serving all of them.
2. Each floor has a **hall panel** with UP and DOWN buttons (top floor: no UP; ground floor: no DOWN).
3. Each car has an **inside panel** with a button per floor, plus **door open** and **door close**.
4. Each car has a **display** showing its floor, direction and the weight it carries.
5. Each floor has a **display for every elevator** so a waiting rider can see where each car is.
6. When a rider presses a hall button, the system **picks one elevator** and sends it.
7. When a rider presses a floor button inside a car, the car **adds that floor** to its stops.
8. A car **serves the stops ahead of it** in its direction before turning around.
9. A car **does not move while its doors are open**.
10. Each car has a **maximum load**; an overloaded car does not move and **says so** on its display.
11. A car with no stops left **waits on the floor where it finished**.

12. An elevator can be **taken out of service** for maintenance; the system must not send riders to it.
13. *(Extension the course leaves out; we add it)* **Fire recall**: all cars go to the lobby, open their doors and ignore car calls.

Non-functional

- **Efficiency:** low average wait and ride time; no car runs to the end of the shaft for nothing.
- **Fairness / no starvation** (starvation = a request that waits forever because others keep being chosen first)**:** every accepted call is eventually served.
- **Safety:** doors, load limit and emergency modes always override scheduling.
- **Thread safety:** riders press buttons concurrently while a controller thread moves the cars.
- **Testability:** deterministic stepping (the same inputs always give the same results).

Use cases

Actors: Rider (calls a car, rides, chooses a destination), **Maintenance engineer** (takes a car out of service and back), **System** (chooses the car, moves cars, opens/closes doors, updates displays).

Top use cases: call an elevator · choose a destination · dispatch a car · move one floor · open and close the doors · report an overload · take a car out of service.

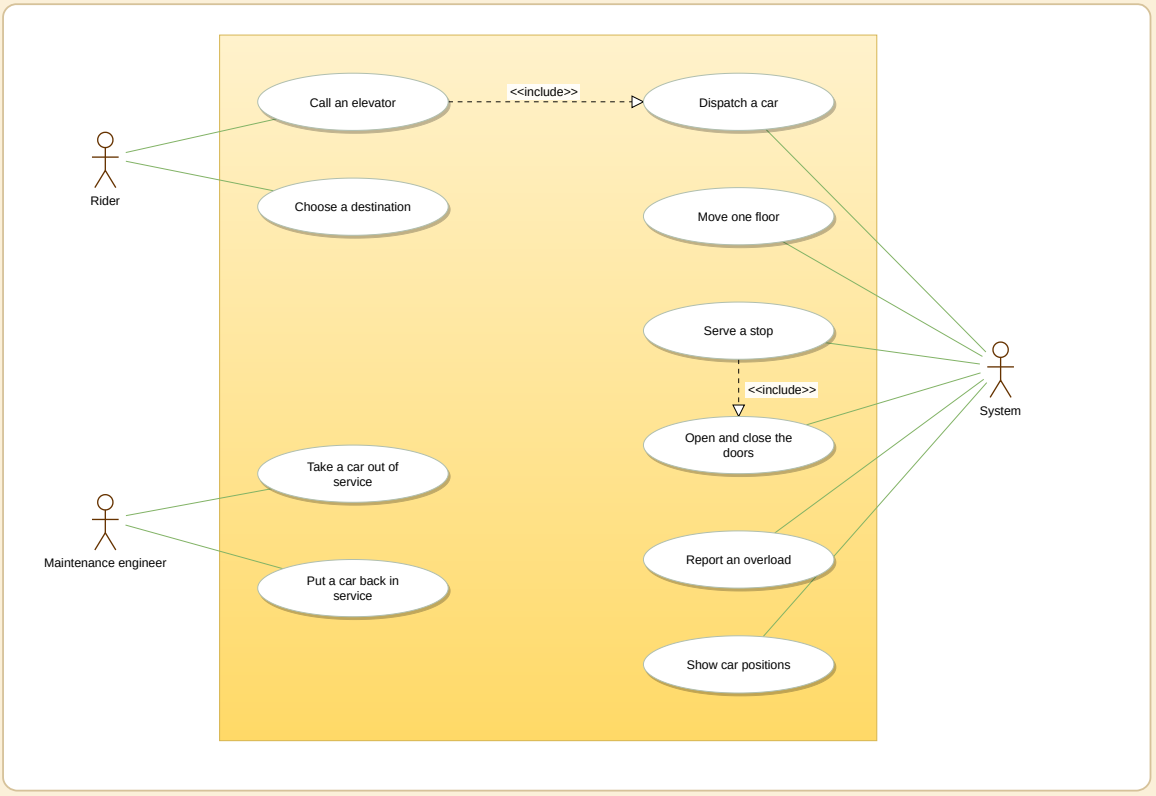


Figure — Use case diagram of the elevator system: Rider, Maintenance engineer and System (course diagram)

In the figure above, "dispatch a car" and "move one floor" belong to the System — the rider only triggers them.

Core entities

Class	Responsibility
`Direction` (enum)	UP, DOWN, IDLE.
`Stop` (record)	A floor **plus** the direction the car will travel when it leaves — two riders on floor 5 (one up, one down) are two stops.
`StopList`	The accepted stops in **two sorted sets** (`up`, `down`); `next(...)` implements LOOK.
`Door`	Open or closed.
`Elevator`	One car: floor, direction, mode, load, door, stop list. `step()` does exactly one action.
`Mode` (enum)	NORMAL, MAINTENANCE, FIRE_RECALL.
`Event` (record)	What one step did (moved, arrived, doors closed, overload, nothing).
`DispatchStrategy`	*Which car* answers a hall call. `NearestCarStrategy` is one rule.
`Dispatcher`	Holds the cars and one strategy; queues calls no car can take now.
`Building`	Owns hall-button lights, the cars and the dispatcher; ticks every car; clears hall lights on arrival; fire alarm.

The course additionally models the physical parts one-to-one: `Button` (with `HallButton`, `ElevatorButton`, `DoorButton`), `HallPanel`, `ElevatorPanel`, `Display` (with `ElevatorDisplay`, `FloorDisplay`) and `Floor`. Our code keeps the behaviour (lights, displays, door) with fewer classes; in a 45-minute interview either level of detail is fine as long as you say which you chose.

Class diagram

The course builds the design in five stages:

Stage	What it adds	What works after it
1	`Button` family and `Door`	Buttons light and reset; a door knows if it is open
2	`HallPanel`, `ElevatorPanel`, `Display` family, `Floor`	A floor accepts a call and shows each car's position
3	`Stop` and `StopList`	Given a floor and a direction, compute the next stop
4	`Elevator`	A car travels to all the stops it has accepted, one step at a time
5	`DispatchStrategy`, `Dispatcher`, `Building`	A call on floor 7 is handed to the most suitable car

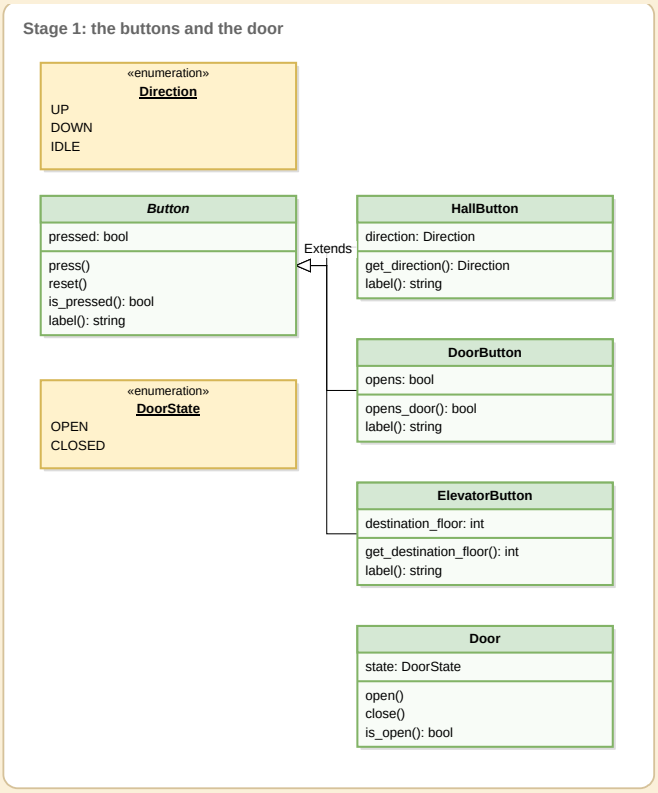


Figure — Stage 1: the Direction enum, the abstract Button with HallButton, ElevatorButton and DoorButton, and the Door (course diagram)

Stage 1. Requirements 2, 3 and 9. All buttons share one behaviour (light up on press, go dark once handled) and differ only in the value they carry: a hall button stores a **direction**, a car button a **floor number**, a door button **open or close**. That gives one abstract `Button` parent with three subclasses instead of three unrelated classes. `Door` has no owner yet; the car will own it in stage 4. Nothing can move at this point.

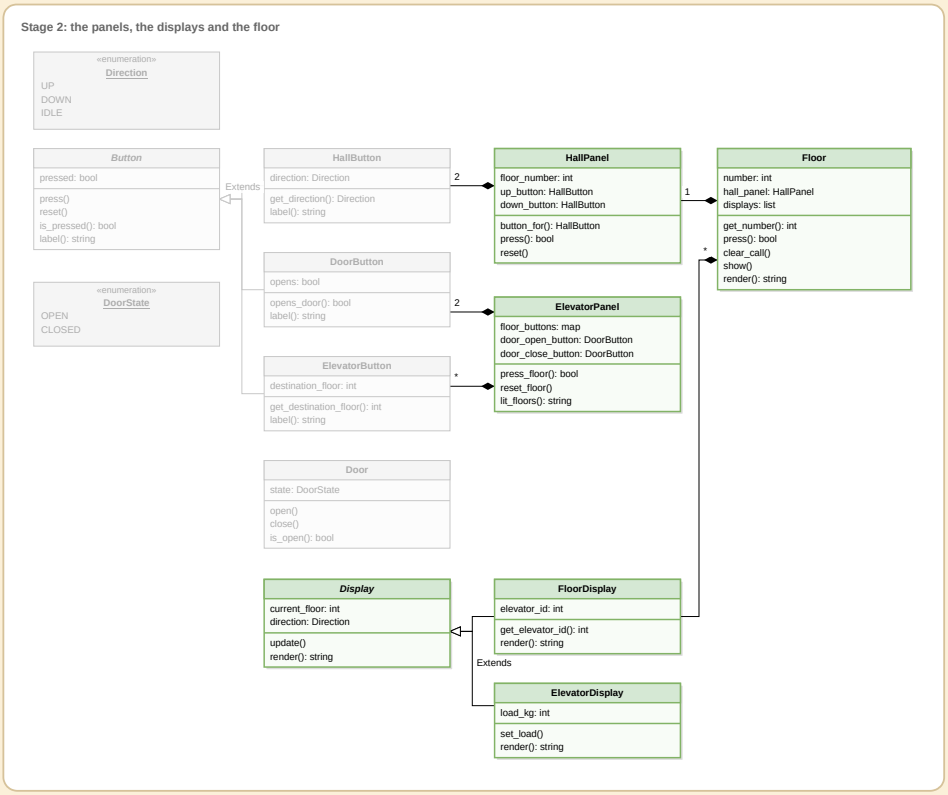


Figure — Stage 2: HallPanel, ElevatorPanel, the abstract Display with ElevatorDisplay and FloorDisplay, and Floor, all joined by composition (course diagram)

Stage 2. Requirements 2, 3, 4 and 5. Every new link is **composition**: a hall panel owns its buttons, and a floor owns its hall panel and its displays, so deleting a floor deletes them too. A hall panel has **at most two** buttons, and each is optional because the top and bottom floors lack one. The car panel creates one button per

floor from the list of floors it receives, so adding a floor needs no edit here. `Display` is abstract: both subclasses show a floor and a direction; the in-car `ElevatorDisplay` adds the load, the wall-mounted `FloorDisplay` adds which car it describes. The inheritance passes the substitution test (Chapter 4): either display can be asked to render itself wherever a `Display` is expected. After this stage a floor can register a call and show where the cars are, but nothing moves yet.

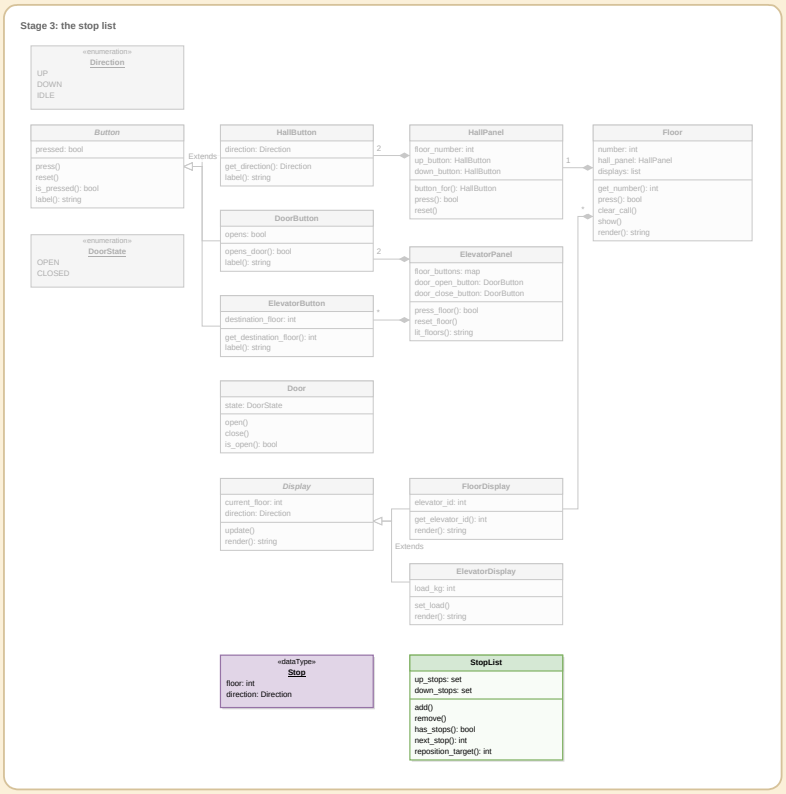


Figure — Stage 3: the Stop data type and the StopList with its up and down sets (course diagram)

Stage 3 — the most important stage. Requirement 8. The whole stage exists to answer: *knowing a car's floor and travel direction, which floor should it stop at next?* The key choice is to store stops in **two sets** (to serve going up, to serve going down) instead of one list. Picture two people waiting on floor 5, one heading up and one heading down: a plain list of floors records "5" once, so the car either stops there twice for no reason or picks up the wrong person. `Stop` is a small data type (a floor plus the direction the car will be moving when it stops there) with no relationships of its own. The next stop returned is always reachable **without reversing**; when nothing lies ahead, the car heads to the **farthest** waiting request rather than the closest one, so it sweeps

everyone up on the return trip. Putting this rule in `StopList` splits responsibilities cleanly: `Elevator` **moves**, `StopList` **chooses**.

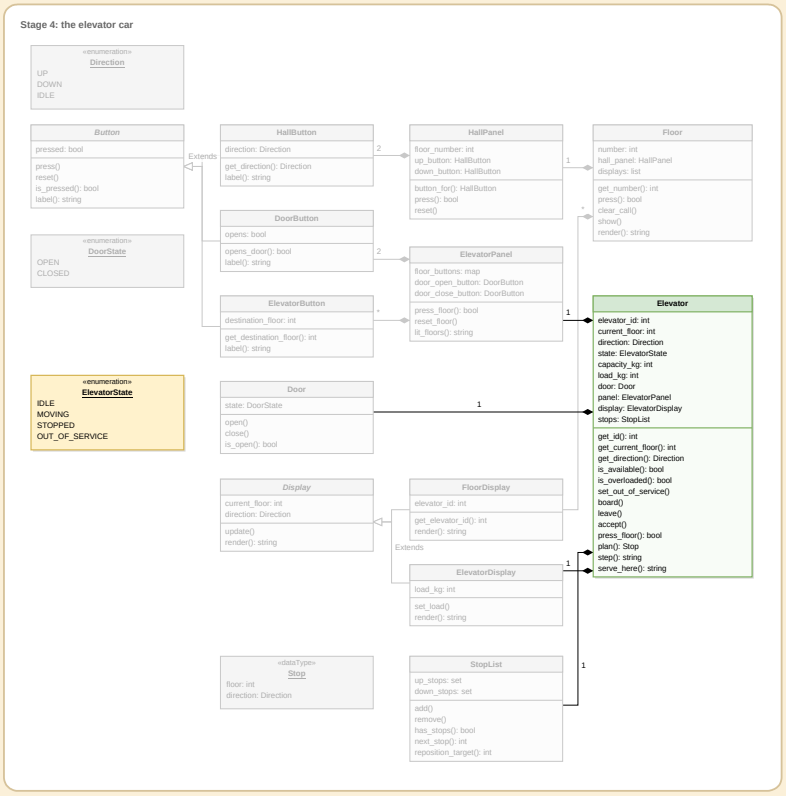


Figure — Stage 4: the Elevator owns its door, panel, display and stop list (course diagram)

Stage 4. Requirements 7, 9, 10 and 11. The car owns its door, panel, display and stop list (composition: none of them exists without the car). A single `step` does exactly **one** of four things: close the doors, refuse to move because of overload, move one floor, or stop and open the doors. Because each call does one action, a test can drive the car step by step with **no clock**. In the course, only one method reads the stop list and only `step` changes the car's position, and a car that is out of service simply does nothing on `step`, so it drops out of the building's rounds without special cases elsewhere (our `step()` returns a `NOTHING` event). Just as important

is what the car does **not** know: it never touches a `Floor` and never talks to another car, which leaves the choice of car to the dispatcher.

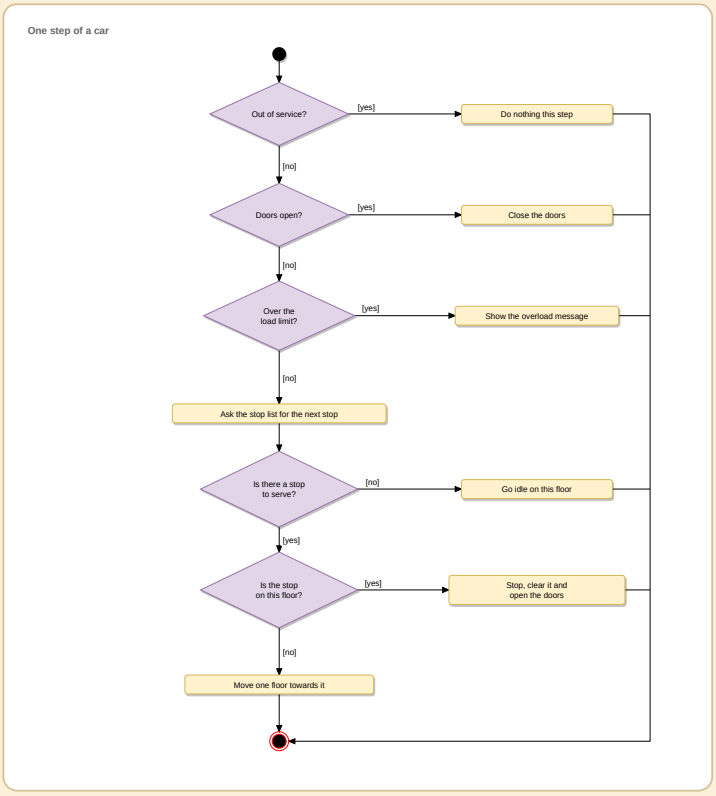


Figure — Activity diagram "One step of a car": out of service → nothing; doors open → overload check or close; otherwise ask the stop list, go idle, stop and open, or move one floor (course diagram)

The activity diagram above is our `Elevator.step()` drawn as a picture: every path ends in exactly one outcome, and only one outcome moves the car.

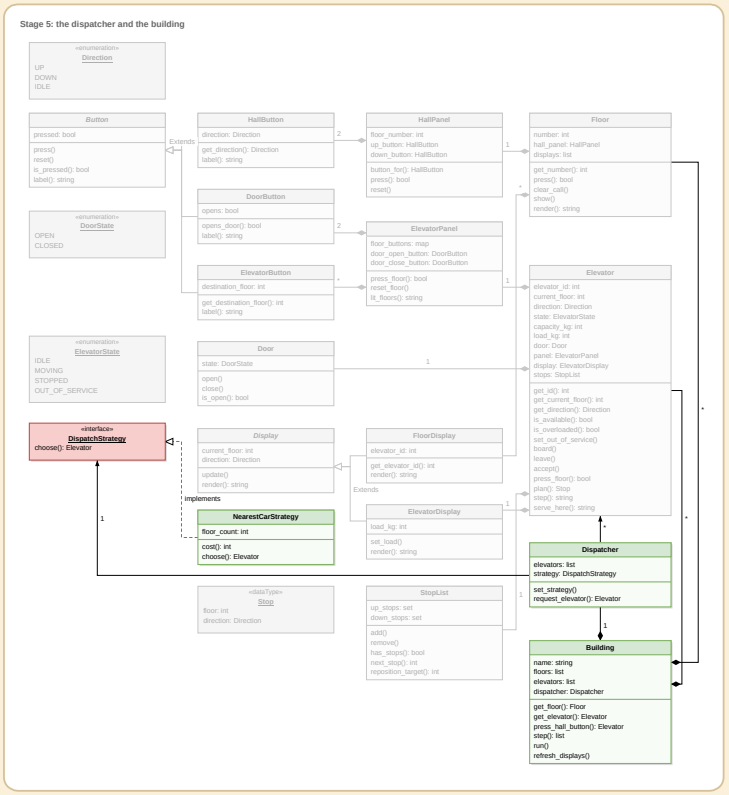


Figure — Stage 5: DispatchStrategy interface with NearestCarStrategy, the Dispatcher, and the Building that owns floors, cars and the dispatcher (course diagram)

Stage 5. Requirements 1, 6 and 12. The rule that picks a car lives **behind an interface** (`DispatchStrategy`), and the dispatcher depends only on that interface, drawn as a plain arrow to it; `NearestCarStrategy` implements it (dashed line, hollow triangle). Morning rush in an office block and a quiet night in a hotel call for different

rules, and swapping the rule touches one class only. The building owns its floors, cars and dispatcher (filled diamonds). It is also the only class that knows about both cars and floors, so switching off the hall-button light when a car opens its doors is the building's job.

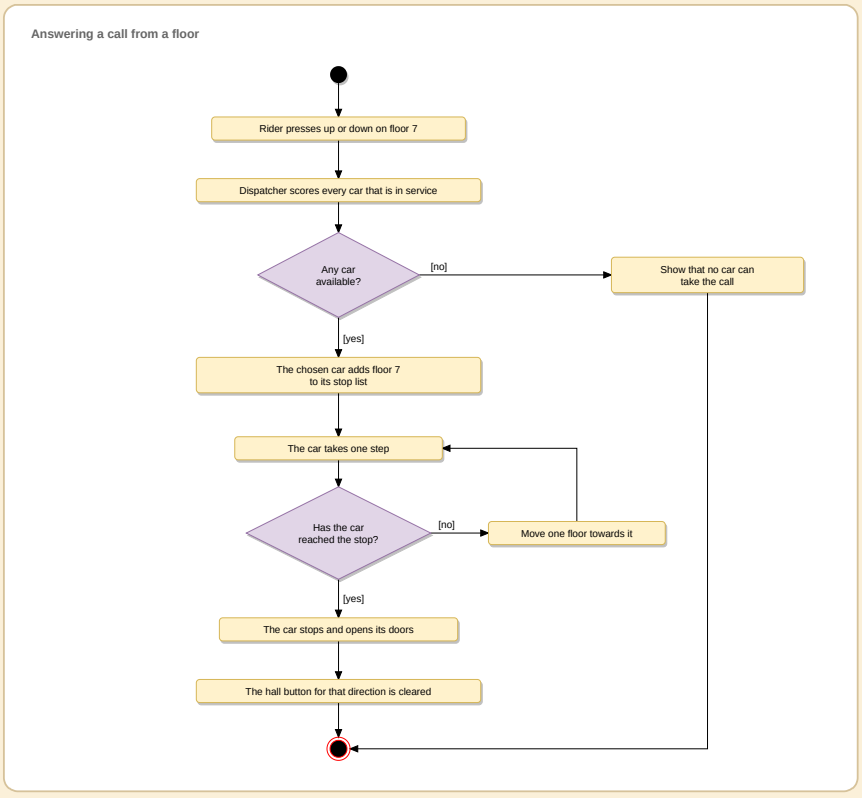


Figure — Activity diagram "Answering a call from a floor": rider presses up/down on floor 7, the dispatcher scores every in-service car, the chosen car adds floor 7, steps until it reaches the stop, opens its doors, and the hall button is cleared (course diagram)

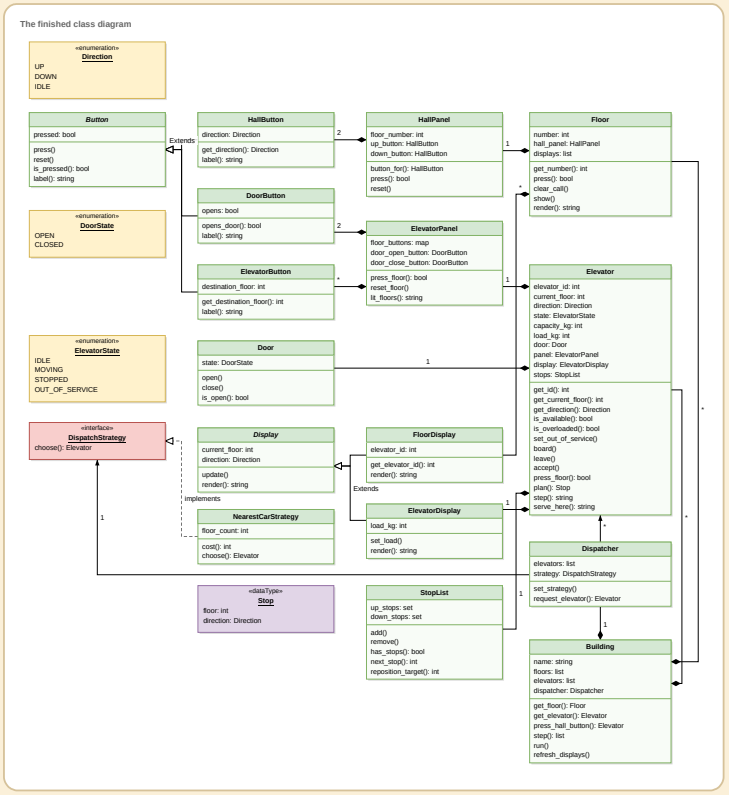
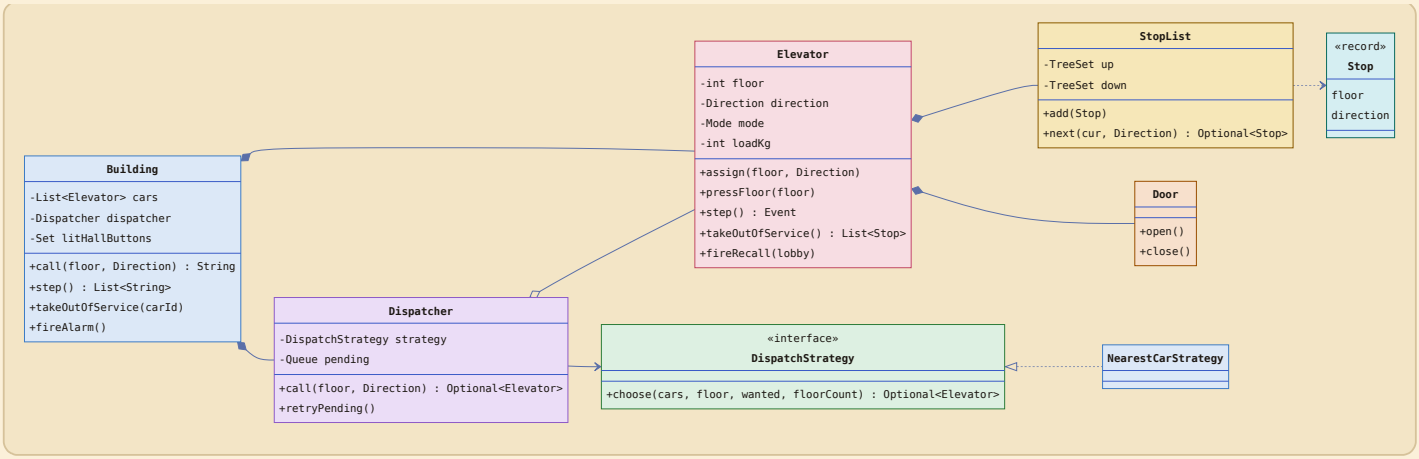


Figure — The finished class diagram of the elevator system (course diagram)

Class diagram of **our Java code**:



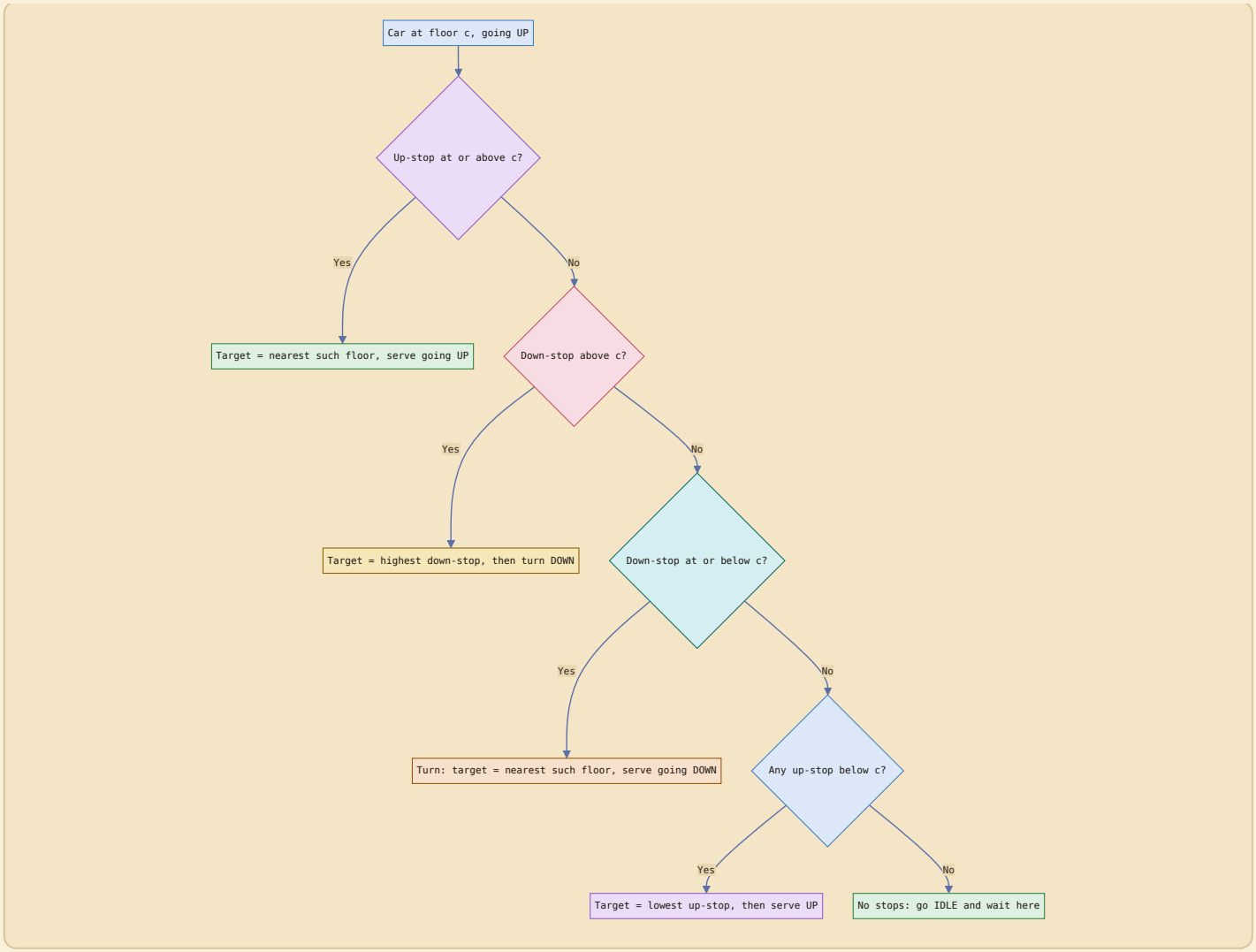
Key flows

Scheduling algorithms compared. "Scheduling" means choosing the order in which one car serves its stops. These algorithms come from disk-arm scheduling in operating systems (the order in which a hard disk's read head visits requested tracks), which has the same shape (a head moving along a line of requests).

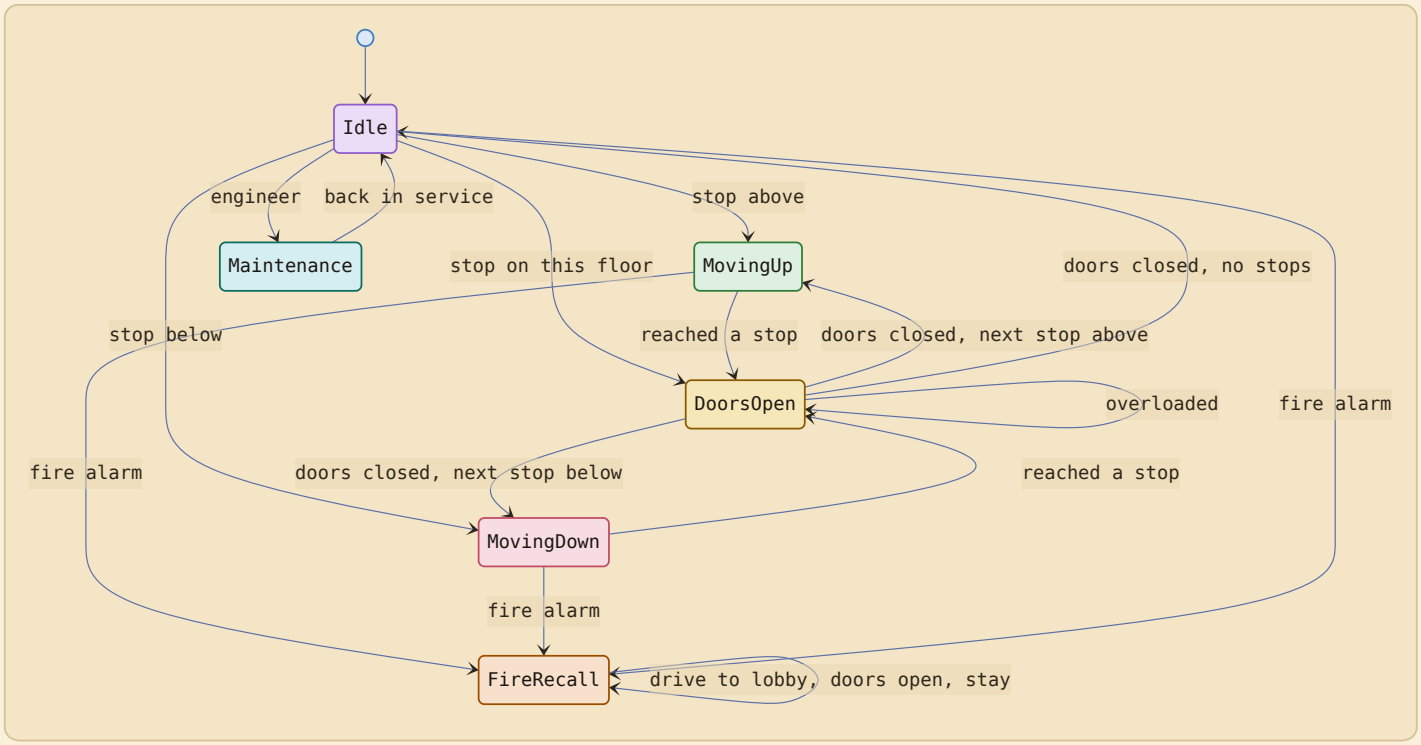
Algorithm	Rule	Good	Bad
FCFS (first come, first served)	Serve calls in arrival order	Simple, fair in order	Zig-zags up and down; very inefficient
SSTF (shortest seek time first)	Always go to the nearest stop	Short average trips	**Starvation**: a far floor may wait forever while nearby calls keep coming
SCAN ("elevator algorithm")	Sweep in one direction to the **end of the shaft**, then reverse	No starvation, predictable	Wastes travel going to the top/bottom floor even when nobody is there
LOOK	Sweep in one direction only **as far as the last request**, then reverse	No starvation, no wasted travel	Slightly more bookkeeping
C-SCAN / C-LOOK	Serve in one direction only; jump back to the start	More uniform waits	Empty return trips

Our `StopList.next` is **LOOK** with direction-aware stops:

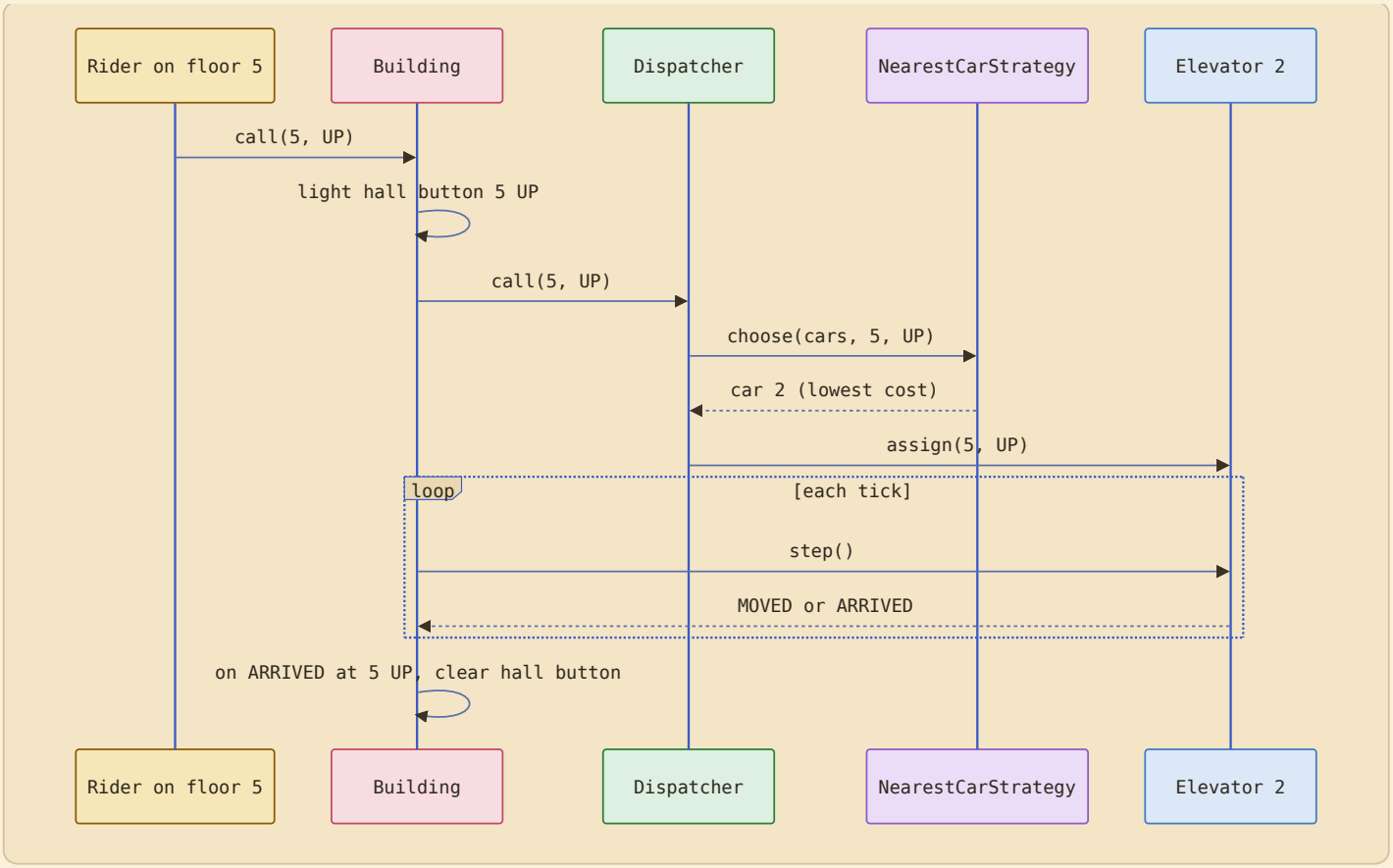
1. Going **UP**: the nearest up-stop at or above the car (`up.ceiling(cur)`).
2. None? If someone *above* wants to go **down**, go up to the **highest** such floor, then turn (collect the rest on the way down).
3. Otherwise turn: the nearest down-stop at or below the car (`down.floor(cur)`).
4. Otherwise the only work left is up-stops **below** the car: go down to the **lowest** one, then serve upward.
5. Going **DOWN** is the mirror image. An **IDLE** car picks the side of the nearest stop.



Car states.



A hall call.



Dispatch cost (`NearestCarStrategy`, from the course): an **idle** car costs its distance; a car **already coming this way in the same direction** costs its distance; any other car pays distance **+ two building heights**, so it wins only when nothing better is free. Cars out of service are skipped.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Choosing a car is a `DispatchStrategy`	**Strategy**	Different buildings and times of day want different rules; the course admits one implementation normally doesn't justify an interface, but here the *requirement* (swap the rule) does	`if (officeMode) … else …` in the dispatcher
Sweep rule in `StopList`, not in `Elevator`	Single Responsibility	The car drives; the stop list decides. Without this split, `Elevator` would do two jobs	Next-stop logic inside `step()`
Two sets of stops (up/down)	Data modelling	Distinguishes "5 going up" from "5 going down"	One list of floors
`step()` = one action	Deterministic simulation	Every scenario is a list of calls and an expected list of events; no clock, no flakiness (tests that pass or fail at random)	A thread per car with `sleep`
Modes as an enum inside the car	Lightweight State	Only three modes and a few checks; full State classes would add more code than they save here. If modes grew (fire phase II, independent service, VIP), switch to the State pattern	State classes from day one
Hall calls re-dispatched when a car leaves service	Robustness	A call accepted by a car must not be lost when that car is taken out	Silently clearing the car's stops
Queue calls when no car is available	—	"No car can take the call right now" is not the same as "rejected"	Dropping the call

The course lists three honest weak points, all worth saying: (1) one step per call means **no threads**; a real controller runs each car on its own thread and needs a lock around the stop list — our methods are `synchronized` for exactly that; (2) the dispatcher **decides once** and never re-evaluates, though real systems do (that is why a

busy building sometimes changes the car number while you wait); (3) `Elevator` is close to doing two jobs, which is why the sweep rule lives in `StopList`. The course also names only **one** pattern here (Strategy); `Button` and `Display` are ordinary abstract base classes and `StopList` is Single Responsibility, not a pattern. Its scope list deliberately leaves out emergency calls, fire service mode, express cars that skip floors, and door sensors that reopen for a hand (we add fire recall; the others are extensions below).

Java implementation

Package `lld.ch14.elevator`.

```java
// File: lld/ch14/elevator/StopList.java
package lld.ch14.elevator;

import java.util.Optional;
import java.util.TreeSet;

enum Direction { UP, DOWN, IDLE }

/** A floor plus the direction the car will travel when it leaves that floor. */
record Stop(int floor, Direction direction) {}

/**
 * The stops a car has accepted, kept in TWO sorted sets: floors to serve while going up,
 * and floors to serve while going down. Two riders on floor 5 (one up, one down) are two stops.
 * next() implements LOOK: keep going while there is work ahead, turn at the last request,
 * never travel to the end of the shaft for nothing (that would be SCAN).
 */
final class StopList {
    private final TreeSet<Integer> up = new TreeSet<>();
    private final TreeSet<Integer> down = new TreeSet<>();

    void add(Stop s) { (s.direction() == Direction.DOWN ? down : up).add(s.floor()); }
    void remove(Stop s) { (s.direction() == Direction.DOWN ? down : up).remove(s.floor()); }
    boolean isEmpty() { return up.isEmpty() && down.isEmpty(); }
    void clear() { up.clear(); down.clear(); }

    /** Removes and returns every stop (used when a car leaves service). */
    java.util.List<Stop> drain() {
        java.util.List<Stop> all = new java.util.ArrayList<>();
        up.forEach(f -> all.add(new Stop(f, Direction.UP)));
        down.forEach(f -> all.add(new Stop(f, Direction.DOWN)));
        clear();
        return all;
    }

    Optional<Stop> next(int cur, Direction dir) {
        if (isEmpty()) return Optional.empty();
        if (dir == Direction.IDLE) dir = nearestSide(cur);
        return Optional.of(dir == Direction.UP ? goingUp(cur) : goingDown(cur));
    }

    private Stop goingUp(int cur) {
        Integer f = up.ceiling(cur);
        if (f != null) return new Stop(f, Direction.UP);                          // work ahead
        if (!down.isEmpty() && down.last() > cur) return new Stop(down.last(), Direction.DOWN); // go up, then turn
        f = down.floor(cur);
        if (f != null) return new Stop(f, Direction.DOWN);                        // turn around now
        return new Stop(up.first(), Direction.UP);                                // go down to lowest up-call
    }

    private Stop goingDown(int cur) {
        Integer f = down.floor(cur);
        if (f != null) return new Stop(f, Direction.DOWN);
        if (!up.isEmpty() && up.first() < cur) return new Stop(up.first(), Direction.UP);
        f = up.ceiling(cur);
        if (f != null) return new Stop(f, Direction.UP);
        return new Stop(down.last(), Direction.DOWN);
    }

    private Direction nearestSide(int cur) {
        int best = Integer.MAX_VALUE;
        Direction side = Direction.UP;
        for (int f : up) if (Math.abs(f - cur) < best) { best = Math.abs(f - cur); side = f >= cur ? Direction.UP :
Direction.DOWN; }
        for (int f : down) if (Math.abs(f - cur) < best) { best = Math.abs(f - cur); side = f > cur ? Direction.UP
: Direction.DOWN; }
        return side;
    }
```

```
    @Override public String toString() { return "up" + up + " down" + down; }
}
```

```java
// File: lld/ch14/elevator/Elevator.java
package lld.ch14.elevator;

enum Mode { NORMAL, MAINTENANCE, FIRE_RECALL }

/** What one step of a car did. */
record Event(Kind kind, int car, int floor, Direction direction, String note) {
    enum Kind { MOVED, ARRIVED, DOORS_CLOSED, OVERLOAD, NOTHING }
}

final class Door {
    private boolean open;
    boolean isOpen() { return open; }
    void open() { open = true; }
    void close() { open = false; }
}

/**
 * One car. It drives; its StopList decides. One call to step() = one action, so the whole
 * system is testable without a clock. Methods are synchronized because riders press buttons
 * from other threads while the controller thread calls step().
 */
final class Elevator {
    private final int id;
    private final int capacityKg;
    private final Door door = new Door();
    private final StopList stops = new StopList();
    private int floor;
    private Direction direction = Direction.IDLE;
    private Mode mode = Mode.NORMAL;
    private int loadKg;

    Elevator(int id, int startFloor, int capacityKg) {
        this.id = id;
        this.floor = startFloor;
        this.capacityKg = capacityKg;
    }

    /** From the dispatcher: a hall call this car has been given. */
    synchronized void assign(int f, Direction wanted) { stops.add(new Stop(f, wanted)); }

    /** From the panel inside the car. */
    synchronized void pressFloor(int f) {
        if (mode != Mode.NORMAL) return;                    // car buttons are dead during fire recall
        if (f == floor && door.isOpen()) return;
        Direction d = f > floor ? Direction.UP : f < floor ? Direction.DOWN
                : (direction == Direction.IDLE ? Direction.UP : direction);
        stops.add(new Stop(f, d));
    }

    synchronized Event step() {
        if (mode == Mode.MAINTENANCE) return nothing();   // parked: never moves
        if (door.isOpen()) {
            if (mode == Mode.FIRE_RECALL && stops.isEmpty()) return nothing(); // parked at lobby, doors open
            if (loadKg > capacityKg) {
                return new Event(Event.Kind.OVERLOAD, id, floor, direction,
                        "OVERLOAD " + loadKg + "kg > " + capacityKg + "kg, doors stay open");
            }
            door.close();
            return new Event(Event.Kind.DOORS_CLOSED, id, floor, direction, "doors close");
        }
        var next = stops.next(floor, direction);
        if (next.isEmpty()) {
            direction = Direction.IDLE;
            return nothing();
        }
        Stop target = next.get();
        if (target.floor() == floor) {                      // arrived
```

```
            stops.remove(target);
            direction = stops.isEmpty() ? Direction.IDLE : target.direction();
            door.open();
            return new Event(Event.Kind.ARRIVED, id, floor, target.direction(), "doors open");
        }
        direction = target.floor() > floor ? Direction.UP : Direction.DOWN;   // never moves with doors open
        floor += direction == Direction.UP ? 1 : -1;
        return new Event(Event.Kind.MOVED, id, floor, direction, "");
    }

    synchronized void board(int kg) { loadKg += kg; }
    synchronized void alight(int kg) { loadKg = Math.max(0, loadKg - kg); }

    /** Out of service: returns the stops it had accepted, so hall calls can be re-dispatched. */
    synchronized java.util.List<Stop> takeOutOfService() {
        mode = Mode.MAINTENANCE;
        direction = Direction.IDLE;
        door.open();                        // parked with doors open so riders can leave
        return stops.drain();
    }

    synchronized void returnToService() {
        mode = Mode.NORMAL;
    }

    /** Fire service: drop every stop, go straight to the lobby, open the doors and stay there. */
    synchronized void fireRecall(int lobby) {
        if (mode == Mode.MAINTENANCE) return;
        mode = Mode.FIRE_RECALL;
        stops.clear();
        stops.add(new Stop(lobby, Direction.DOWN));
        direction = Direction.IDLE;
    }

    synchronized boolean available() { return mode == Mode.NORMAL; }
    synchronized boolean idle() { return stops.isEmpty() && (!door.isOpen() || mode != Mode.NORMAL); }
    synchronized int floor() { return floor; }
    synchronized Direction direction() { return direction; }
    int id() { return id; }

    synchronized String display() {
        return "car%d@%d %s%s %dkg %s".formatted(id, floor, direction, door.isOpen() ? " [open]" : "",
                loadKg, mode == Mode.NORMAL ? "" : mode);
    }

    private Event nothing() { return new Event(Event.Kind.NOTHING, id, floor, direction, ""); }
}
```

```java
// File: lld/ch14/elevator/Dispatch.java
package lld.ch14.elevator;

import java.util.ArrayDeque;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.Queue;

/** Strategy: which car answers a hall call. Office at 9am and hotel at midnight want different rules. */
interface DispatchStrategy {
    Optional<Elevator> choose(List<Elevator> cars, int floor, Direction wanted, int floorCount);
}

/**
 * Idle car: cost = distance. A car already heading toward the caller in the same direction:
 * cost = distance. Anything else pays a penalty of two building heights, so it wins only
 * when nothing better exists.
 */
final class NearestCarStrategy implements DispatchStrategy {
    @Override public Optional<Elevator> choose(List<Elevator> cars, int floor, Direction wanted, int floorCount) {
        return cars.stream().filter(Elevator::available)
                .min(Comparator.comparingInt((Elevator c) -> cost(c, floor, wanted, floorCount))
                        .thenComparingInt(Elevator::id));
    }

    private int cost(Elevator c, int floor, Direction wanted, int n) {
        int distance = Math.abs(c.floor() - floor);
        Direction d = c.direction();
        if (d == Direction.IDLE) return distance;
        boolean onTheWay = (d == Direction.UP && c.floor() <= floor) || (d == Direction.DOWN && c.floor() >=
floor);
        if (onTheWay && d == wanted) return distance;
        return distance + 2 * n;
    }
}

/** Holds the cars and one strategy. Calls no car can take right now wait in a queue. */
final class Dispatcher {
    private final List<Elevator> cars;
    private final int floorCount;
    private volatile DispatchStrategy strategy;
    private final Queue<Stop> pending = new ArrayDeque<>();

    Dispatcher(List<Elevator> cars, int floorCount, DispatchStrategy strategy) {
        this.cars = cars;
        this.floorCount = floorCount;
        this.strategy = strategy;
    }

    synchronized Optional<Elevator> call(int floor, Direction wanted) {
        Optional<Elevator> car = strategy.choose(cars, floor, wanted, floorCount);
        car.ifPresentOrElse(c -> c.assign(floor, wanted), () -> pending.add(new Stop(floor, wanted)));
        return car;
    }

    /** Called every tick: retry the calls nobody could take earlier. */
    synchronized void retryPending() {
        int n = pending.size();
        for (int i = 0; i < n; i++) {
            Stop s = pending.poll();
            Optional<Elevator> car = strategy.choose(cars, s.floor(), s.direction(), floorCount);
            if (car.isPresent()) car.get().assign(s.floor(), s.direction());
            else pending.add(s);
        }
    }

    synchronized int pendingCount() { return pending.size(); }
```

```
    void setStrategy(DispatchStrategy s) { this.strategy = s; }
}
```

```java
// File: lld/ch14/elevator/Building.java
package lld.ch14.elevator;

import java.util.ArrayList;
import java.util.List;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

/** Owns floors' hall buttons, the cars and the dispatcher. The only class that knows cars AND floors. */
final class Building {
    private final int floors;
    private final List<Elevator> cars = new ArrayList<>();
    private final Dispatcher dispatcher;
    private final Set<Stop> litHallButtons = ConcurrentHashMap.newKeySet();
    private int tick;

    Building(int floors, int[] carStartFloors, int capacityKg) {
        this.floors = floors;
        for (int i = 0; i < carStartFloors.length; i++) cars.add(new Elevator(i + 1, carStartFloors[i],
capacityKg));
        this.dispatcher = new Dispatcher(cars, floors, new NearestCarStrategy());
    }

    /** A rider presses UP or DOWN on a floor. The top floor has no UP, the ground floor no DOWN. */
    String call(int floor, Direction d) {
        if ((floor == floors - 1 && d == Direction.UP) || (floor == 0 && d == Direction.DOWN)) {
            throw new IllegalArgumentException("no " + d + " button on floor " + floor);
        }
        if (!litHallButtons.add(new Stop(floor, d))) return "hall %d %s already lit".formatted(floor, d);
        return dispatcher.call(floor, d).map(c -> "hall %d %s -> car%d".formatted(floor, d, c.id()))
                .orElse("hall %d %s -> queued (no car available)".formatted(floor, d));
    }

    /** One tick: retry queued calls, then every car takes one step. Returns what happened. */
    List<String> step() {
        tick++;
        dispatcher.retryPending();
        List<String> out = new ArrayList<>();
        for (Elevator c : cars) {
            Event e = c.step();
            if (e.kind() == Event.Kind.ARRIVED) litHallButtons.remove(new Stop(e.floor(), e.direction()));
            if (e.kind() == Event.Kind.MOVED) out.add("car%d->%d".formatted(e.car(), e.floor()));
            else if (e.kind() != Event.Kind.NOTHING) out.add("car%d@%d %s".formatted(e.car(), e.floor(),
e.note()));
        }
        return out;
    }

    /** Engineer takes a car out: its accepted hall calls go back to the dispatcher. */
    List<String> takeOutOfService(int carId) {
        List<String> out = new ArrayList<>();
        for (Stop s : car(carId).takeOutOfService()) {
            if (litHallButtons.contains(s)) {
                out.add("re-dispatch " + dispatcher.call(s.floor(), s.direction())
                        .map(c -> "hall %d %s -> car%d".formatted(s.floor(), s.direction(), c.id()))
                        .orElse("hall %d %s -> queued".formatted(s.floor(), s.direction())));
            }
        }
        return out;
    }

    void returnToService(int carId) { car(carId).returnToService(); }

    void fireAlarm() {
        litHallButtons.clear();
        cars.forEach(c -> c.fireRecall(0));
    }
```

```
    boolean allIdle() { return cars.stream().allMatch(Elevator::idle) && dispatcher.pendingCount() == 0; }
    Elevator car(int id) { return cars.get(id - 1); }
    int tick() { return tick; }

    /** What the display on every floor shows. */
    String floorDisplay() {
        StringBuilder sb = new StringBuilder();
        cars.forEach(c -> sb.append(c.display().stripTrailing()).append(" | "));
        return sb.substring(0, sb.length() - 3);
    }
}
```

```java
// File: lld/ch14/elevator/Main.java
package lld.ch14.elevator;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) throws Exception {
        System.out.println("== 1. LOOK on one car: at floor 4 going up; stops 6(up) 9(car) 2(down) 1(up) ==");
        StopList sl = new StopList();
        sl.add(new Stop(6, Direction.UP));
        sl.add(new Stop(9, Direction.UP));
        sl.add(new Stop(2, Direction.DOWN));
        sl.add(new Stop(1, Direction.UP));
        int cur = 4;
        Direction dir = Direction.UP;
        List<Integer> order = new ArrayList<>();
        while (!sl.isEmpty()) {
            Stop s = sl.next(cur, dir).orElseThrow();
            sl.remove(s);
            order.add(s.floor());
            cur = s.floor();
            dir = s.direction();
        }
        System.out.println("service order " + order + "  (turns at 9 and at 1: never runs to 0 or to a floor above
9)");

        System.out.println("\n== 2. Building: floors 0-9, car1 at 0, car2 at 8 ==");
        Building b = new Building(10, new int[]{0, 8}, 680);
        System.out.println(b.call(5, Direction.UP));      // car2 idle, 3 away, beats car1 (5 away)
        System.out.println(b.call(1, Direction.UP));      // car1 idle, 1 away
        steps(b, 2);
        System.out.println("rider enters car1 at floor 1, presses 7");
        b.car(1).board(80);
        b.car(1).pressFloor(7);
        steps(b, 2);
        System.out.println("rider enters car2 at floor 5, presses 9");
        b.car(2).board(75);
        b.car(2).pressFloor(9);
        System.out.println(b.call(3, Direction.UP));      // car1 is below 3 and going up: on its way
        runUntilIdle(b);
        System.out.println("display: " + b.floorDisplay());

        System.out.println("\n== 3. Overload: at floor 9, 10 more people squeeze into car2 ==");
        System.out.println(b.call(9, Direction.DOWN));
        steps(b, 1);                                      // doors open at 9
        b.car(2).board(700);
        b.car(2).pressFloor(0);
        steps(b, 2);
        System.out.println("two people step out");
        b.car(2).alight(160);
        steps(b, 3);
        runUntilIdle(b);
        b.car(2).alight(615);                             // everyone leaves at the lobby

        System.out.println("\n== 4. Maintenance: calls move to another car ==");
        System.out.println(b.call(4, Direction.DOWN));
        b.takeOutOfService(1).forEach(System.out::println);       // car1 had the call: it moves to car2
        b.takeOutOfService(2).forEach(System.out::println);  // now no car at all: call is queued
        System.out.println(b.call(2, Direction.UP));
        System.out.println("engineer returns car1 to service; queued calls are dispatched on the next tick");
        b.returnToService(1);
        runUntilIdle(b);
        b.returnToService(2);
```

```
        System.out.println("\n== 5. Fire alarm: every car to the lobby, doors open, car buttons dead ==");
        b.car(2).pressFloor(9);
        steps(b, 3);
        b.fireAlarm();
        b.car(1).pressFloor(8);
        runUntilIdle(b);
        System.out.println("display: " + b.floorDisplay());

        System.out.println("\n== 6. 8 riders press hall buttons at the same instant (new building) ==");
        Building busy = new Building(10, new int[]{0, 9, 4}, 680);
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            int[][] calls = {{1, 0}, {2, 0}, {3, 1}, {5, 0}, {6, 1}, {7, 1}, {8, 0}, {9, 1}};
            for (int[] c : calls) {
                pool.submit(() -> { go.await(); busy.call(c[0], c[1] == 0 ? Direction.UP : Direction.DOWN); return
null; });
            }
            go.countDown();
        }
        while (!busy.allIdle() && busy.tick() < 200) busy.step();
        System.out.println("all 8 calls served: " + busy.allIdle() + " (ticks=" + busy.tick() + " < 200)");
    }

    /** Exactly n ticks, printing what happened in each. */
    static void steps(Building b, int n) {
        for (int i = 0; i < n; i++) print(b, b.step());
    }

    static void runUntilIdle(Building b) {
        for (int guard = 0; guard < 100 && !b.allIdle(); guard++) print(b, b.step());
    }

    private static void print(Building b, List<String> events) {
        System.out.println("  t" + b.tick() + ": " + (events.isEmpty() ? "-" : String.join(", ", events)));
    }
}
```

How the code works, part by part

- `StopList` keeps two `TreeSet<Integer>`s (a `TreeSet` is a sorted set). `ceiling`, `floor`, `first` and `last` give O(log n) answers to "nearest stop at or above/below" — the core of LOOK.
- `Elevator.step()` checks, in order: maintenance → doors open (overload? fire parked? else close) → ask the stop list → arrive (remove the stop, open the doors) or move one floor. The car never moves with the doors open.
- `pressFloor` classifies a car call as an up- or down-stop relative to the current floor; during fire recall car buttons are ignored.
- `takeOutOfService` returns the car's accepted stops; `Building` re-dispatches the ones that are still lit hall calls.
- `Dispatcher.retryPending` runs every tick, so a queued call is dispatched as soon as any car becomes available.
- `Main` part 1 runs LOOK on one car in isolation; parts 2–5 run a two-car building; part 6 has eight threads press hall buttons at the same instant.

Output:

```
== 1. LOOK on one car: at floor 4 going up; stops 6(up) 9(car) 2(down) 1(up) ==
service order [6, 9, 2, 1]  (turns at 9 and at 1: never runs to 0 or to a floor above 9)

== 2. Building: floors 0-9, car1 at 0, car2 at 8 ==
hall 5 UP -> car2
hall 1 UP -> car1
  t1: car1->1, car2->7
  t2: car1@1 doors open, car2->6
rider enters car1 at floor 1, presses 7
  t3: car1@1 doors close, car2->5
  t4: car1->2, car2@5 doors open
rider enters car2 at floor 5, presses 9
hall 3 UP -> car1
  t5: car1->3, car2@5 doors close
  t6: car1@3 doors open, car2->6
  t7: car1@3 doors close, car2->7
  t8: car1->4, car2->8
  t9: car1->5, car2->9
  t10: car1->6, car2@9 doors open
  t11: car1->7, car2@9 doors close
  t12: car1@7 doors open
  t13: car1@7 doors close
display: car1@7 IDLE 80kg | car2@9 IDLE 75kg

== 3. Overload: at floor 9, 10 more people squeeze into car2 ==
hall 9 DOWN -> car2
  t14: car2@9 doors open
  t15: car2@9 OVERLOAD 775kg > 680kg, doors stay open
  t16: car2@9 OVERLOAD 775kg > 680kg, doors stay open
two people step out
  t17: car2@9 doors close
  t18: car2->8
  t19: car2->7
  t20: car2->6
  t21: car2->5
  t22: car2->4
  t23: car2->3
  t24: car2->2
  t25: car2->1
  t26: car2->0
  t27: car2@0 doors open
  t28: car2@0 doors close

== 4. Maintenance: calls move to another car ==
hall 4 DOWN -> car1
re-dispatch hall 4 DOWN -> car2
re-dispatch hall 4 DOWN -> queued
hall 2 UP -> queued (no car available)
engineer returns car1 to service; queued calls are dispatched on the next tick
  t29: car1@7 doors close
  t30: car1->6
  t31: car1->5
  t32: car1->4
  t33: car1@4 doors open
  t34: car1@4 doors close
  t35: car1->3
  t36: car1->2
  t37: car1@2 doors open
  t38: car1@2 doors close

== 5. Fire alarm: every car to the lobby, doors open, car buttons dead ==
  t39: car2@0 doors close
  t40: car2->1
  t41: car2->2
  t42: car1->1, car2->1
  t43: car1->0, car2->0
  t44: car1@0 doors open, car2@0 doors open
display: car1@0 IDLE [open] 80kg FIRE_RECALL | car2@0 IDLE [open] 0kg FIRE_RECALL
```

```
== 6. 8 riders press hall buttons at the same instant (new building) ==
all 8 calls served: true (ticks=11 < 200)
```

Read the output: in part 1 the car serves 6 and 9 going up, turns at 9 (the highest request, not the top of the shaft), serves 2 going down, then 1 — LOOK, not SCAN. In part 2 the hall call on 3 goes to car 1 because it is below floor 3 and already going up. In part 3 the overloaded car keeps its doors open until two people step out. In part 4 the call held by car 1 moves to car 2 when car 1 goes out of service, then is queued when no car is available, and served as soon as car 1 returns. In part 5 car 2 reverses mid-trip and both cars park at the lobby with doors open.

Concurrency & edge cases

Situation	Handling
Riders press buttons while the controller steps cars	`Elevator` methods are `synchronized`; the dispatcher is `synchronized`; hall lights are a concurrent set
Same hall button pressed twice	`litHallButtons.add` returns false: "already lit", no second dispatch
Up and down requests on the same floor	Two different `Stop`s in two sets
Car call for the floor the car is on, doors open	Ignored (doors already open)
Overload	Doors stay open, display shows OVERLOAD, car does not move
Car taken out of service with accepted hall calls	Calls re-dispatched or queued; never lost
No car available	Call queued; retried every tick
Fire alarm	All stops dropped, lobby added, car buttons ignored, doors stay open at the lobby
Starvation	LOOK serves every accepted stop within one round trip (one sweep each way)
Invalid button (UP on the top floor)	`IllegalArgumentException` — that button does not exist

Extensions & follow-up questions

1. **"How do you choose which car?"** — `DispatchStrategy`; show `NearestCarStrategy` and mention alternatives: zoning (car 1 serves floors 0–10), **up-peak** mode (send idle cars to the lobby at 9 a.m.), and energy-saving rules.
2. **"What if two riders press at the same instant?"** — The dispatcher is synchronized, so calls are dispatched one at a time; each car's stop list is guarded by the car's lock. (The course's design leaves this lock out and says so.)
3. **"How would you test this?"** — Because one call = one step, every test is: a list of button presses, N steps, and an expected list of events. No sleeps, no flakiness.
4. **"Destination dispatch?"** — Riders enter their destination in the lobby keypad; the system groups riders with nearby destinations into the same car. Implement as another `DispatchStrategy` that receives (from, to) instead of (floor, direction).
5. **"Re-evaluate assignments?"** — Each tick, for unserved hall calls, ask the strategy again and move the call if a much better car is free now.
6. **"Real time?"** — A `ScheduledExecutorService` (a Java thread pool that runs tasks on a timer) calls `building.step()` every second (or per car on its own thread, with the car lock guarding the stop list).
7. **"Emergency stop button inside the car?"** — A new mode that halts movement immediately and raises an alarm; with more modes, refactor modes into State classes.
8. **"Express cars?"** — A car with a set of floors it may serve (e.g. lobby plus 30–40); the dispatch strategy skips cars whose served set does not include the call floor.
9. **"Door sensor that reopens for a hand?"** — A `DoorSensor` input; while it reports an obstruction, `step()` reopens the doors instead of closing them (same slot as the overload check).

Real-world example

Modern high-rise buildings use **destination dispatch** systems (for example Schindler's PORT, Otis CompassPlus and KONE's destination control): riders choose their floor in the lobby and the system tells them which car to take,

which cuts stops per trip — extension 4. Controllers switch between traffic modes such as **up-peak** in the morning. The fire behaviour in this design follows the idea of **Phase I emergency recall** in elevator safety codes such as ASME A17.1: on a fire alarm, cars return to the designated landing, open their doors and are removed from normal service.

In the interview

Plan for 45 minutes: 5 min requirements and scope (how many cars, which modes) → 5 min entities → **15 min on the stop list and LOOK** (two sets, the four cases) → 10 min car `step()` and dispatcher strategy → 10 min concurrency, testing and follow-ups.

> **30-second answer:** "Each car keeps its stops in two sorted sets, up-stops and down-stops, and uses LOOK: keep serving stops ahead in the current direction, turn at the last request, never run to the end of the shaft for nothing. A car's `step()` does one action — close doors, refuse to move if overloaded, move one floor, or stop and open — which makes the whole system testable without a clock. A dispatcher assigns hall calls through a `DispatchStrategy`, nearest suitable car by default, and queues calls when no car is available. Doors, load, maintenance and fire recall always override scheduling."

Follow-ups with answers

- *"Why LOOK and not SSTF?"* — SSTF can starve a distant floor; LOOK serves every accepted stop within one round trip (one sweep up, one sweep down).
- *"Why two sets and not one?"* — A floor can be requested going up and going down; one set cannot tell which rider to serve on which sweep.
- *"Where do you put the lock?"* — Around each car's stop list and state (the car's monitor — the built-in lock every Java object carries, taken by `synchronized`), and around the dispatcher's assignment. No global lock across cars.

Self-check

1. **Q:** Difference between SCAN and LOOK? **A:** SCAN runs to the end of the shaft before reversing; LOOK reverses at the last request.
2. **Q:** A car at 4 going up has stops 6(up), 9(car), 2(down), 1(up). Order? **A:** 6, 9, 2, 1.
3. **Q:** Why is the scheduling rule in `StopList` and not `Elevator`? **A:** Single Responsibility — the car drives, the stop list decides; each can be tested alone.
4. **Q:** What does the dispatcher do when every car is in maintenance? **A:** Queues the call and retries on every tick.
5. **Q:** Why does one `step()` do only one action? **A:** Deterministic, clock-free simulation: tests are sequences of calls with expected events.
6. **Q:** When would you convert modes into State classes? **A:** When there are many modes with different behaviour per action (fire phase I/II, independent service, VIP, emergency stop).

TL;DR

- **Two sorted sets** (up/down) of stops + **LOOK** = efficient, starvation-free scheduling.
- **`step()` = one action**; doors, load, maintenance and fire recall override scheduling.
- **Strategy** for dispatch (nearest car by default), with a pending queue and re-dispatch when a car leaves service.
- Thread safety: per-car lock, synchronized dispatcher, concurrent hall-light set.

14.6 Design a Traffic Signal Control System

A **traffic signal controller** decides which approaches of an intersection get green, for how long, and in what order. An **approach** is one road entering the intersection (north, south, east, west). A **phase** is a set of approaches that may be green together because their paths do not cross (for example north and south going straight). The controller cycles through phases, inserting **yellow** (warning) and **all-red** (clearance) intervals between them.

This question tests a **timed state machine** with a **hard safety rule** — *two crossing flows must never be green at the same time* — plus an **emergency override** that must interrupt the normal cycle *safely* (never by cutting a yellow short).

Problem & clarifying questions

Question	Assumed answer
How many roads?	A four-way intersection (N, S, E, W); phases are configurable.
Which signals?	Red, yellow, green; flashing red in fault mode.
Fixed or adaptive timing?	Configurable: fixed durations, or adaptive based on queue sensors.
Pedestrian signals, turn arrows?	Out of scope (extensions).
Emergency vehicles?	Yes: an approaching ambulance or fire truck gets green as soon as safely possible, held until it passes.
Many intersections?	Each intersection has its own controller; a central system can coordinate them (extension).
Real time?	A timer ticks once per second; the logic is tick-driven so it can also be simulated.

Requirements

Functional

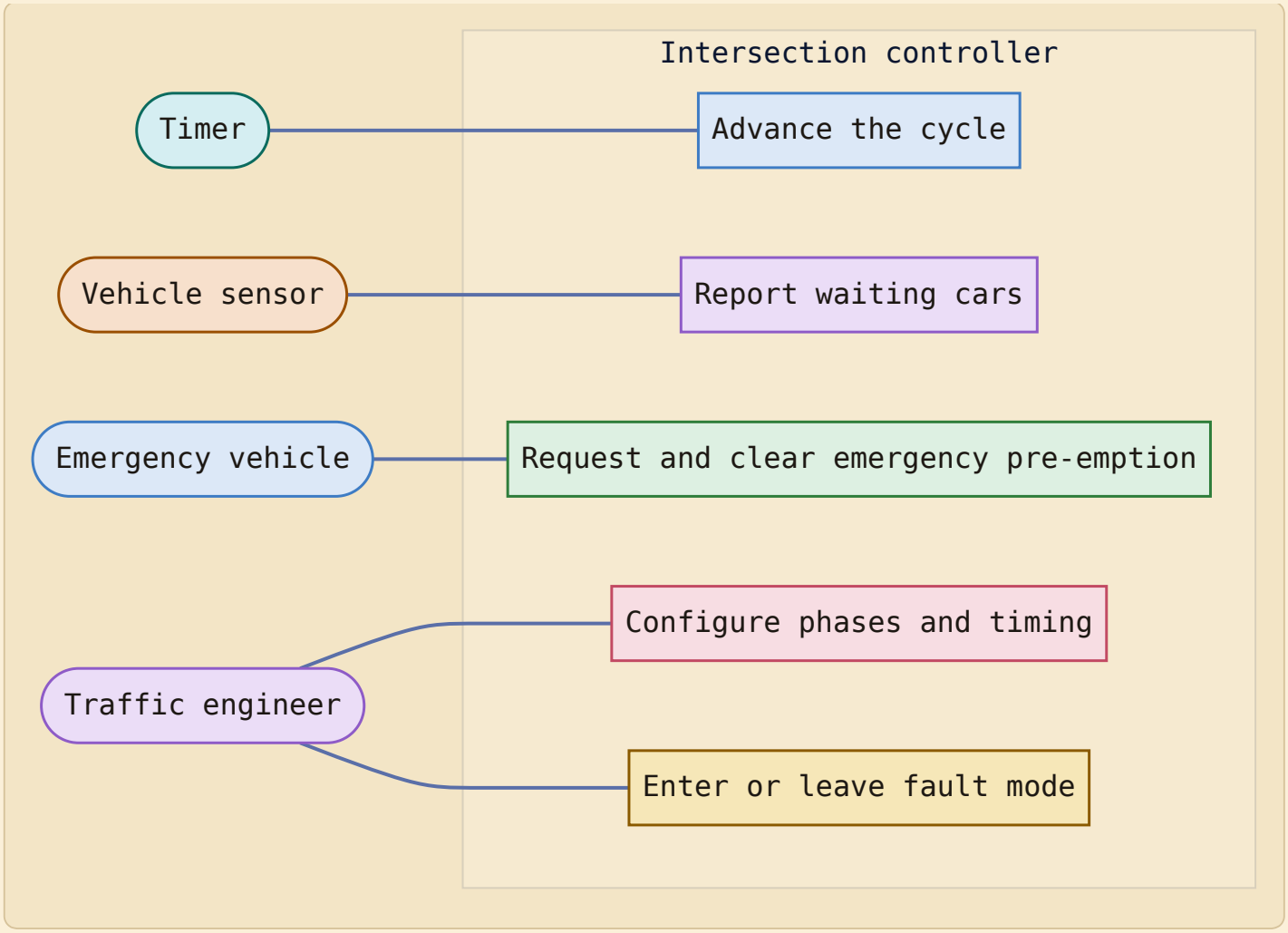
1. Control traffic at an intersection with **multiple roads**.
2. Support **red, yellow and green** signals.
3. Signal **durations are configurable** and can **adapt to traffic** conditions.
4. Transitions are **smooth and safe**: green → yellow → all-red → next green.
5. Handle **emergencies**: an approaching ambulance or fire truck gets priority.
6. Be **scalable and extensible** (more phases, more intersections, new features).
7. On a controller fault, fall back to a **safe mode** (all flashing red = treat as an all-way stop).

Non-functional

- **Safety invariant** (an invariant is a rule that must hold at every moment)**:** never conflicting greens, not even for an instant during a switch.
- **Deterministic timing:** a tick = one second; testable with a simulated clock.
- **Thread safety:** a timer thread ticks while sensor and emergency inputs arrive from other threads.
- **Observability** (the system's behaviour can be watched from outside)**:** displays, logs and monitors can subscribe to light changes.

Use cases

Actors: Timer (ticks every second), **Vehicle sensors** (report queue lengths), **Emergency vehicle / dispatch** (request and clear pre-emption — interrupting the normal cycle to serve a priority request), **Traffic engineer** (configures phases and timing), **Drivers** (see the lights).



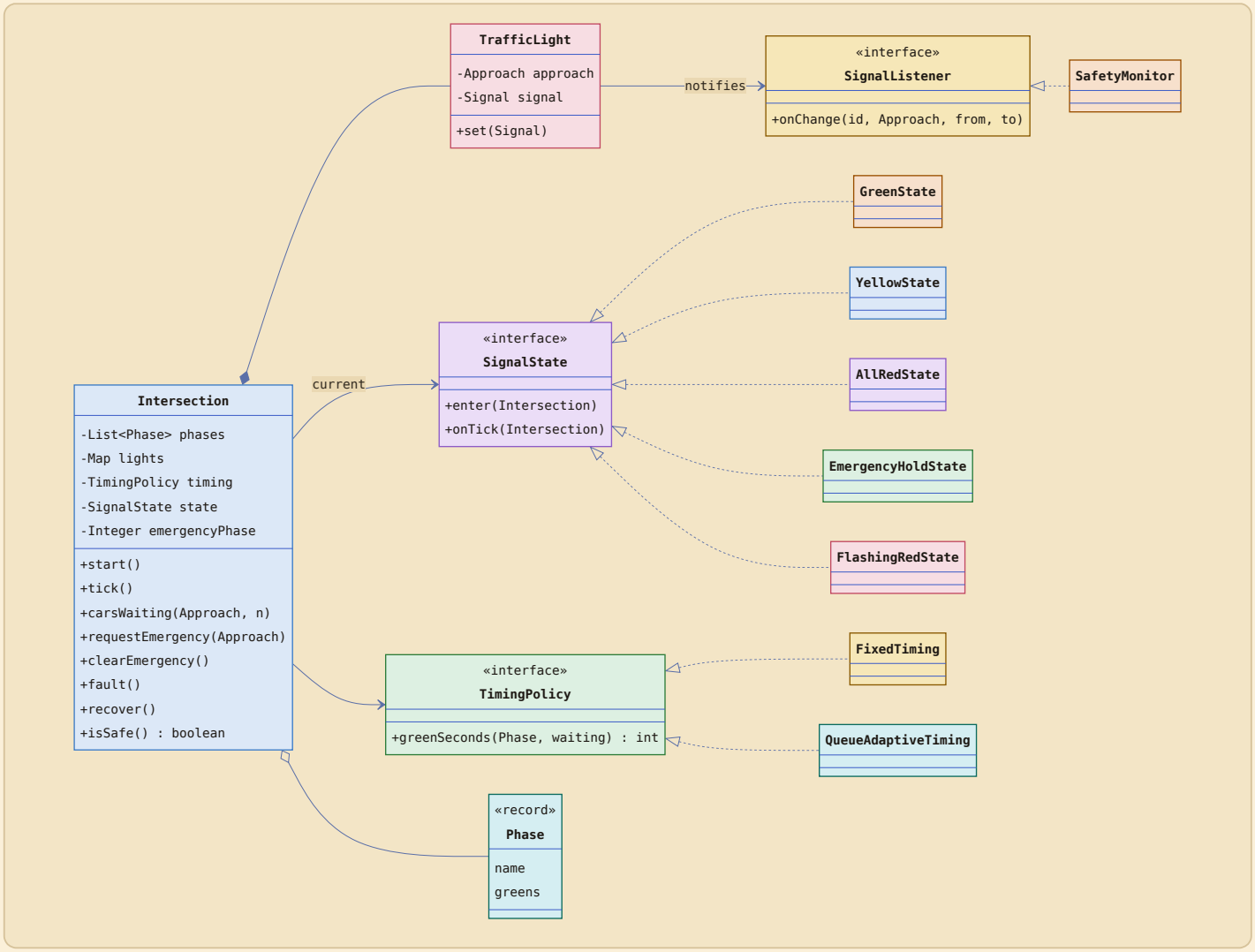
Core entities

Class	Responsibility
`Signal` (enum)	RED, YELLOW, GREEN, FLASHING_RED.
`Approach` (enum)	NORTH, SOUTH, EAST, WEST.
`Phase` (record)	A name and the set of approaches that may be green together.
`TrafficLight`	One signal head for one approach; notifies listeners on every change.
`SignalListener`	**Observer** interface for light changes.
`SafetyMonitor`	A listener that checks the safety invariant after every change to green.
`TimingPolicy`	**Strategy** for green duration: `FixedTiming`, `QueueAdaptiveTiming`.
`SignalState` + 5 states	**State**: `GreenState`, `YellowState`, `AllRedState`, `EmergencyHoldState`, `FlashingRedState`.
`Intersection`	The State context (the object that holds the current state and hands it each call): phases, lights, timing, sensors, emergency flag; `tick()`.

Mapping to the common problem statement. The widely used version of this problem (for example in the open-source *awesome-low-level-design* collection) asks for: an intersection with multiple roads, red/yellow/green signals, configurable durations that can adjust to traffic, smooth transitions, handling of emergency vehicles (ambulances, fire trucks), and a scalable, extensible design. Its classes map onto ours as follows: the `Signal` enum → our `Signal`; `Road` (id, name, and the traffic light facing it) → our `Approach` plus the `TrafficLight` stored for it; `TrafficLight` (current signal, durations, notifies observers) → our `TrafficLight` (signal + listeners) with the durations moved into `TimingPolicy` and the yellow/all-red settings; `TrafficController` (a Singleton managing all roads) → our `Intersection`, deliberately **not** a Singleton. What the reference leaves

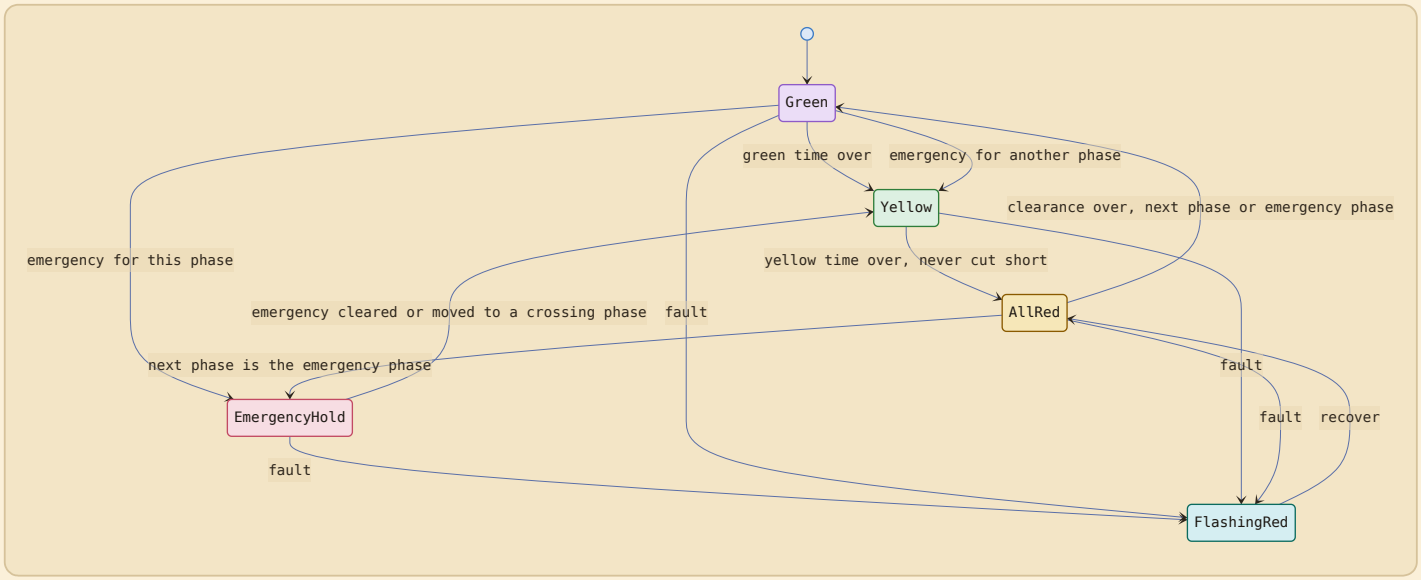
implicit, and we make explicit, is the phase model (which roads may be green together), the all-red clearance interval, and the safety check.

Class diagram

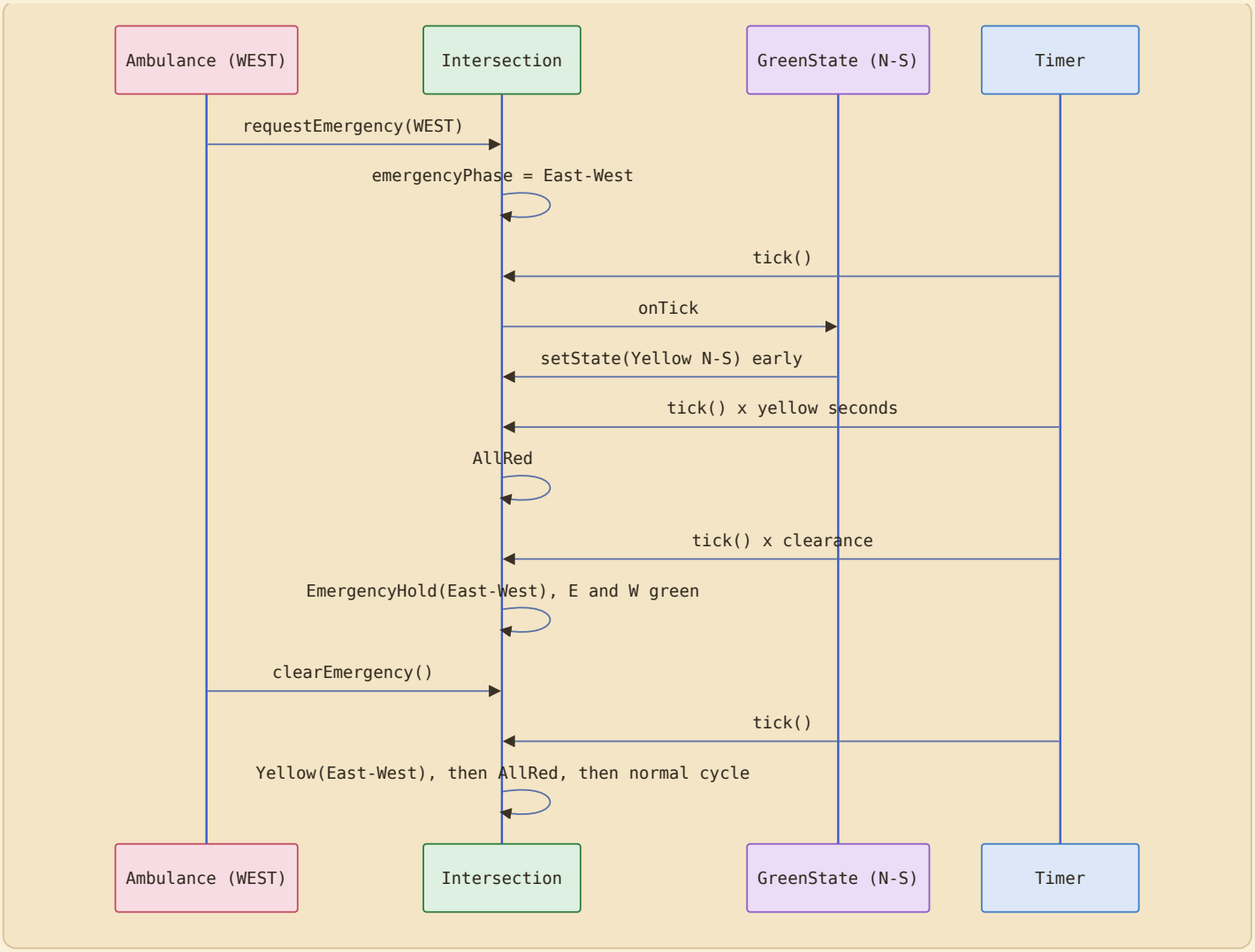


Key flows

The controller's state machine.



Emergency pre-emption.



One full fixed cycle (green 5 s, yellow 2 s, all-red 1 s): `N-S green (5) → N-S yellow (2) → all red (1) → E-W green (5) → E-W yellow (2) → all red (1) → …` — a 16-second cycle.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each interval is a state class with `enter` and `onTick`	**State**	What a tick means depends on the interval; the transitions (and the "yellow is never cut short" rule) live in one small class each	One `switch` over an enum with counters for every case
States **carry data** (remaining seconds) and are created per transition	State with per-instance data	Unlike the vending machine's stateless states, these hold a countdown, so they cannot be shared	Shared singletons with counters on the context
Green duration comes from a `TimingPolicy`	**Strategy**	Fixed, adaptive (queue sensors), time-of-day plans all swap at run time	Durations hard-coded in `GreenState`
Light changes are published to listeners	**Observer**	Displays, logs, a central monitoring system and the safety monitor subscribe independently	The intersection calling each consumer directly
`showPhaseGreen` turns others **red first**, then turns the phase green	Safety by construction	There is never an instant with conflicting greens	Setting lights in arbitrary order
A separate `SafetyMonitor` checks the invariant	Defence in depth (several independent safety layers)	Mirrors real hardware conflict monitors that force flash if a bug ever causes conflicting greens	Trusting the controller logic alone
Emergency is a **flag** the states read, not a direct jump	Safety	A request only *shortens* green; yellow and all-red still run in full	Jumping straight to the emergency green (a crash risk)
The awesome-lld reference makes the controller a **Singleton**	Rejected	A city has many intersections, each with its own controller; the demo runs two. A per-city `TrafficControlCenter` can own them	`TrafficController.getInstance()`

Java implementation

Package `lld.ch14.trafficsignal`.

```java
// File: lld/ch14/trafficsignal/Model.java
package lld.ch14.trafficsignal;

import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.CopyOnWriteArrayList;

enum Signal { RED, YELLOW, GREEN, FLASHING_RED }

enum Approach { NORTH, SOUTH, EAST, WEST }

/** A phase = the set of approaches that may be green together (they never cross). */
record Phase(String name, Set<Approach> greens) {}

/** Observer: anyone who wants to hear about light changes (displays, logs, safety monitor). */
interface SignalListener {
    void onChange(String intersection, Approach approach, Signal from, Signal to);
}

/** One light head facing one approach. */
final class TrafficLight {
    private final String intersection;
    private final Approach approach;
    private final List<SignalListener> listeners;
    private Signal signal = Signal.RED;

    TrafficLight(String intersection, Approach approach, List<SignalListener> listeners) {
        this.intersection = intersection;
        this.approach = approach;
        this.listeners = listeners;
    }

    void set(Signal to) {
        if (to == signal) return;
        Signal from = signal;
        signal = to;
        listeners.forEach(l -> l.onChange(intersection, approach, from, to));
    }

    Signal signal() { return signal; }
}

/** Strategy: how long a phase stays green. */
interface TimingPolicy {
    int greenSeconds(Phase phase, Map<Approach, Integer> waitingCars);
}

record FixedTiming(int green) implements TimingPolicy {
    public int greenSeconds(Phase p, Map<Approach, Integer> waiting) { return green; }
}

/** More waiting cars on the phase's approaches = longer green, between min and max. */
record QueueAdaptiveTiming(int min, int max, int secondsPerCar) implements TimingPolicy {
    public int greenSeconds(Phase p, Map<Approach, Integer> waiting) {
        int cars = p.greens().stream().mapToInt(a -> waiting.getOrDefault(a, 0)).sum();
        return Math.max(min, Math.min(max, cars * secondsPerCar));
    }
}

/** Observer that enforces the one rule that must never break: no two crossing flows moving at once. */
final class SafetyMonitor implements SignalListener {
    private final Intersection[] watched = new Intersection[1];
    private final List<String> violations = new CopyOnWriteArrayList<>();

    void watch(Intersection i) { watched[0] = i; }

    @Override public void onChange(String id, Approach a, Signal from, Signal to) {
```

```
        if (to != Signal.GREEN || watched[0] == null) return;
        if (!watched[0].isSafe()) violations.add(id + ": " + a + " went green while a crossing flow was moving");
    }

    List<String> violations() { return violations; }
}
```

```java
// File: lld/ch14/trafficsignal/States.java
package lld.ch14.trafficsignal;

/** State pattern: each state knows what one second of time means for it, and which state is next. */
interface SignalState {
    default void enter(Intersection x) {}
    void onTick(Intersection x);
    String name();
}

final class GreenState implements SignalState {
    private final int phase;
    private int remaining;

    GreenState(int phase) { this.phase = phase; }

    @Override public void enter(Intersection x) {
        if (Integer.valueOf(phase).equals(x.emergencyPhase())) { x.setState(new EmergencyHoldState(phase)); return;
}
        remaining = x.greenSecondsFor(phase);
        x.showPhaseGreen(phase);
    }

    @Override public void onTick(Intersection x) {
        Integer em = x.emergencyPhase();
        if (em != null) {                                   // pre-emption: end this green early
            x.setState(em == phase ? new EmergencyHoldState(phase) : new YellowState(phase));
            return;
        }
        if (--remaining <= 0) x.setState(new YellowState(phase));
    }

    @Override public String name() { return "GREEN(" + phase + ")"; }
}

final class YellowState implements SignalState {
    private final int phase;
    private int remaining;

    YellowState(int phase) { this.phase = phase; }

    @Override public void enter(Intersection x) {
        remaining = x.yellowSeconds();
        x.showPhase(phase, Signal.YELLOW);
    }

    @Override public void onTick(Intersection x) {          // yellow is never cut short: that is unsafe
        if (--remaining <= 0) x.setState(new AllRedState(x.nextPhaseAfter(phase)));
    }

    @Override public String name() { return "YELLOW(" + phase + ")"; }
}

/** Clearance interval: everyone red so the junction empties before the next phase. */
final class AllRedState implements SignalState {
    private final int next;
    private int remaining;

    AllRedState(int next) { this.next = next; }

    @Override public void enter(Intersection x) {
        remaining = x.allRedSeconds();
        x.showAllRed();
    }

    @Override public void onTick(Intersection x) {
        if (--remaining <= 0) {
            Integer em = x.emergencyPhase();
```

```
            x.setState(new GreenState(em != null ? em : next));
        }
    }

    @Override public String name() { return "ALL_RED"; }
}

/** Emergency vehicle approaching: hold its phase green until the override is cleared. */
final class EmergencyHoldState implements SignalState {
    private final int phase;

    EmergencyHoldState(int phase) { this.phase = phase; }

    @Override public void enter(Intersection x) { x.showPhaseGreen(phase); }

    @Override public void onTick(Intersection x) {
        Integer em = x.emergencyPhase();
        // Cleared, or a newer request came from a crossing phase: leave through yellow + all-red.
        if (em == null || em != phase) x.setState(new YellowState(phase));
    }

    @Override public String name() { return "EMERGENCY_HOLD(" + phase + ")"; }
}

/** Controller fault or power problem: all approaches flash red (treat as a four-way stop). */
final class FlashingRedState implements SignalState {
    @Override public void enter(Intersection x) { x.showAll(Signal.FLASHING_RED); }
    @Override public void onTick(Intersection x) {}
    @Override public String name() { return "FLASHING_RED"; }
}
```

```java
// File: lld/ch14/trafficsignal/Intersection.java
package lld.ch14.trafficsignal;

import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.StringJoiner;
import java.util.concurrent.CopyOnWriteArrayList;

/**
 * The context of the State pattern. tick() is called once per second by a timer thread;
 * emergency and sensor calls arrive from other threads. Every public method is
 * synchronized, so a state transition is never observed half-done.
 */
final class Intersection {
    private final String id;
    private final List<Phase> phases;
    private final Map<Approach, TrafficLight> lights = new EnumMap<>(Approach.class);
    private final Map<Approach, Integer> waitingCars = new EnumMap<>(Approach.class);
    private final List<SignalListener> listeners = new CopyOnWriteArrayList<>();
    private final int yellowSeconds;
    private final int allRedSeconds;
    private volatile TimingPolicy timing;
    private SignalState state;
    private Integer emergencyPhase;         // null = no override
    private long clock;

    Intersection(String id, List<Phase> phases, TimingPolicy timing, int yellowSeconds, int allRedSeconds) {
        this.id = id;
        this.phases = List.copyOf(phases);
        this.timing = timing;
        this.yellowSeconds = yellowSeconds;
        this.allRedSeconds = allRedSeconds;
        for (Approach a : Approach.values()) lights.put(a, new TrafficLight(id, a, listeners));
    }

    void addListener(SignalListener l) { listeners.add(l); }

    synchronized void start() { setState(new GreenState(0)); }
    synchronized void tick() { clock++; state.onTick(this); }

    // ----- inputs from other threads -----
    synchronized void carsWaiting(Approach a, int n) { waitingCars.put(a, n); }

    synchronized void requestEmergency(Approach from) {
        for (int i = 0; i < phases.size(); i++) if (phases.get(i).greens().contains(from)) emergencyPhase = i;
    }

    synchronized void clearEmergency() { emergencyPhase = null; }
    synchronized void fault() { setState(new FlashingRedState()); }
    synchronized void recover() { emergencyPhase = null; setState(new AllRedState(0)); }
    void setTiming(TimingPolicy t) { timing = t; }

    // ----- used by the states -----
    void setState(SignalState s) { state = s; s.enter(this); }
    Integer emergencyPhase() { return emergencyPhase; }
    int nextPhaseAfter(int p) { return (p + 1) % phases.size(); }
    int greenSecondsFor(int p) { return timing.greenSeconds(phases.get(p), waitingCars); }
    int yellowSeconds() { return yellowSeconds; }
    int allRedSeconds() { return allRedSeconds; }

    /** Turns red first, then green: never two crossing flows green, even for an instant. */
    void showPhaseGreen(int p) {
        Phase phase = phases.get(p);
        lights.forEach((a, l) -> { if (!phase.greens().contains(a)) l.set(Signal.RED); });
        phase.greens().forEach(a -> lights.get(a).set(Signal.GREEN));
    }
```

```
    void showPhase(int p, Signal s) { phases.get(p).greens().forEach(a -> lights.get(a).set(s)); }
    void showAllRed() { lights.values().forEach(l -> l.set(Signal.RED)); }
    void showAll(Signal s) { lights.values().forEach(l -> l.set(s)); }

    /** Safe = every approach that is green or yellow belongs to ONE phase. */
    synchronized boolean isSafe() {
        return phases.stream().anyMatch(p -> lights.entrySet().stream()
                .filter(e -> e.getValue().signal() == Signal.GREEN || e.getValue().signal() == Signal.YELLOW)
                .allMatch(e -> p.greens().contains(e.getKey())));
    }

    synchronized String snapshot() {
        StringJoiner j = new StringJoiner(" ");
        lights.forEach((a, l) -> j.add(a.name().charAt(0) + ":" + l.signal().name().charAt(0)
                + (l.signal() == Signal.FLASHING_RED ? "*" : "")));
        return j + "  " + state.name();
    }

    synchronized long clock() { return clock; }
    String id() { return id; }
}
```

```java
// File: lld/ch14/trafficsignal/Main.java
package lld.ch14.trafficsignal;

import java.util.List;
import java.util.Set;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static final List<Phase> PHASES = List.of(
            new Phase("North-South", Set.of(Approach.NORTH, Approach.SOUTH)),
            new Phase("East-West", Set.of(Approach.EAST, Approach.WEST)));

    public static void main(String[] args) throws Exception {
        SafetyMonitor safety = new SafetyMonitor();
        Intersection x = new Intersection("5th&Main", PHASES, new FixedTiming(5), 2, 1);
        safety.watch(x);
        x.addListener(safety);
        AtomicInteger changes = new AtomicInteger();
        x.addListener((id, a, from, to) -> changes.incrementAndGet());

        System.out.println("== 1. Fixed cycle: green 5s, yellow 2s, all-red 1s (printed when lights change) ==");
        x.start();
        System.out.println("t=0   " + x.snapshot());
        run(x, 16);

        System.out.println("\n== 2. Adaptive timing: 12 cars waiting East-West, 2 North-South ==");
        x.setTiming(new QueueAdaptiveTiming(3, 10, 1));
        x.carsWaiting(Approach.EAST, 7);
        x.carsWaiting(Approach.WEST, 5);
        x.carsWaiting(Approach.NORTH, 2);
        run(x, 20);

        System.out.println("\n== 3. Ambulance from the WEST while North-South is green ==");
        x.setTiming(new FixedTiming(5));
        run(x, 2);
        System.out.println("t=" + x.clock() + "  emergency request: WEST");
        x.requestEmergency(Approach.WEST);
        run(x, 7);
        System.out.println("t=" + x.clock() + "  ambulance has passed: clear");
        x.clearEmergency();
        run(x, 5);

        System.out.println("\n== 4. Controller fault -> flashing red, then recovery ==");
        x.fault();
        System.out.println("t=" + x.clock() + "  " + x.snapshot());
        x.recover();
        run(x, 3);

        System.out.println("\n== 5. Real timer thread (1 ms = 1 s) + emergency from another thread ==");
        Intersection y = new Intersection("Oak&Pine", PHASES, new FixedTiming(5), 2, 1);
        SafetyMonitor safety2 = new SafetyMonitor();
        safety2.watch(y);
        y.addListener(safety2);
        y.start();
        CountDownLatch done = new CountDownLatch(500);
        ScheduledExecutorService timer = Executors.newSingleThreadScheduledExecutor();
        timer.scheduleAtFixedRate(() -> { y.tick(); done.countDown(); }, 0, 1, TimeUnit.MILLISECONDS);
        Thread dispatcher = Thread.ofVirtual().start(() -> {
            for (int i = 0; i < 20; i++) {
                y.requestEmergency(i % 2 == 0 ? Approach.EAST : Approach.NORTH);
                Thread.onSpinWait();
                y.clearEmergency();
            }
        });
```

```
        done.await();
        timer.shutdown();
        timer.awaitTermination(1, TimeUnit.SECONDS);
        dispatcher.join();
        System.out.println("ticks run: " + (y.clock() >= 500) + ", safety violations: " +
safety2.violations().size());

        System.out.println("\nlight changes seen by observer: " + changes.get()
                + ", safety violations: " + safety.violations().size());
    }

    static void run(Intersection x, int seconds) {
        String last = x.snapshot();
        for (int i = 0; i < seconds; i++) {
            x.tick();
            String now = x.snapshot();
            if (!now.equals(last)) System.out.printf("t=%-3d %s%n", x.clock(), now);
            last = now;
        }
    }
}
```

How the code works, part by part

- `Intersection.tick()` is `synchronized` and simply delegates to `state.onTick(this)`. A state changes the context with `setState(next)`, which immediately calls `next.enter(this)` so the new state can set lights and load its timer.
- `GreenState.enter` asks the `TimingPolicy` for the duration, and jumps straight to `EmergencyHoldState` if its phase is the emergency phase.
- `GreenState.onTick` ends the green **early** when an emergency is pending for another phase. `YellowState` never checks the emergency flag — yellow always runs in full.
- `AllRedState` chooses the emergency phase if one is pending, else the next phase in order.
- `QueueAdaptiveTiming` gives `secondsPerCar × waiting cars`, clamped between min and max.
- `SafetyMonitor` receives every change to GREEN and checks `isSafe()`: all green/yellow approaches must belong to one phase.
- Part 5 of `Main` drives a second intersection with a real `ScheduledExecutorService` (1 ms stands for 1 s) while a virtual thread (a lightweight Java 21 thread, cheap enough to create by the thousand) fires 20 emergency requests and clears; the synchronized methods keep every transition whole, and the monitor sees zero violations.

Output:

```
== 1. Fixed cycle: green 5s, yellow 2s, all-red 1s (printed when lights change) ==
t=0   N:G S:G E:R W:R  GREEN(0)
t=5   N:Y S:Y E:R W:R  YELLOW(0)
t=7   N:R S:R E:R W:R  ALL_RED
t=8   N:R S:R E:G W:G  GREEN(1)
t=13  N:R S:R E:Y W:Y  YELLOW(1)
t=15  N:R S:R E:R W:R  ALL_RED
t=16  N:G S:G E:R W:R  GREEN(0)

== 2. Adaptive timing: 12 cars waiting East-West, 2 North-South ==
t=21  N:Y S:Y E:R W:R  YELLOW(0)
t=23  N:R S:R E:R W:R  ALL_RED
t=24  N:R S:R E:G W:G  GREEN(1)
t=34  N:R S:R E:Y W:Y  YELLOW(1)
t=36  N:R S:R E:R W:R  ALL_RED

== 3. Ambulance from the WEST while North-South is green ==
t=37  N:G S:G E:R W:R  GREEN(0)
t=38  emergency request: WEST
t=39  N:Y S:Y E:R W:R  YELLOW(0)
t=41  N:R S:R E:R W:R  ALL_RED
t=42  N:R S:R E:G W:G  EMERGENCY_HOLD(1)
t=45  ambulance has passed: clear
t=46  N:R S:R E:Y W:Y  YELLOW(1)
t=48  N:R S:R E:R W:R  ALL_RED
t=49  N:G S:G E:R W:R  GREEN(0)

== 4. Controller fault -> flashing red, then recovery ==
t=50  N:F* S:F* E:F* W:F*  FLASHING_RED
t=51  N:G S:G E:R W:R  GREEN(0)

== 5. Real timer thread (1 ms = 1 s) + emergency from another thread ==
ticks run: true, safety violations: 0

light changes seen by observer: 48, safety violations: 0
```

Read the output: the fixed cycle is 5 s green, 2 s yellow, 1 s all-red; with 12 cars waiting east-west, the adaptive policy gives E-W a 10-second green (the maximum); the ambulance request at t=38 ends N-S green at the very next tick, but yellow (2 s) and all-red (1 s) still run before E-W goes green and is **held**; after the clear, the cycle resumes with N-S; a fault switches every head to flashing red; the observer counted every change and found no safety violation.

Concurrency & edge cases

Situation	Handling
Emergency request arrives between two ticks	Stored in a field under the intersection's lock; the next tick acts on it
Emergency arrives during yellow	Yellow runs in full; `AllRedState` then picks the emergency phase
Emergency for the phase that is already green	`GreenState.onTick` switches to `EmergencyHold` — no needless yellow
Two emergency vehicles from crossing directions	The latest request wins: if it arrives during a hold, `EmergencyHoldState` leaves through yellow and all-red and the new phase is then held. A real system queues requests by priority and arrival and serves them in turn (extension)
Emergency never cleared (lost message)	Add a maximum hold time after which the controller resumes the cycle
Timer thread and sensor thread overlap	All public methods are `synchronized`; listeners are in a `CopyOnWriteArrayList` (a thread-safe list that copies itself on every change)
Adaptive timing starves a quiet road	Minimum green time in `QueueAdaptiveTiming`; every phase still gets its turn in the cycle
Controller bug causes conflicting greens	`SafetyMonitor` detects it; real controllers then force flashing red
Timer drift (small delays adding up so ticks slowly fall behind)	`scheduleAtFixedRate` keeps a fixed rate; for real deployments use the controller's hardware clock

Extensions & follow-up questions

1. **"Pedestrian crossings?"** — Add pedestrian signal heads per phase (WALK during the phase's green, flashing DON'T WALK before yellow) and a push-button input that requests the phase.
2. **"Left-turn arrows?"** — More phases (e.g. N-S protected left, then N-S through); phases are data, so the state machine does not change.
3. **"Coordinate many intersections (green wave)?"** — A `TrafficControlCenter` sets each intersection's cycle length and **offset** so that a platoon (group travelling together) of cars driving at the speed limit meets greens in sequence.
4. **"Emergency corridor?"** — The centre pre-empts the next intersections along the ambulance's route, each through `requestEmergency`.
5. **"Night mode?"** — A time-of-day `TimingPolicy`, or a flashing-yellow state for the main road with flashing red for the side road.
6. **"How to test?"** — The tick-driven design is deterministic: call `tick()` N times and assert the snapshot, as `Main` does.

Real-world example

Adaptive systems such as **SCATS** (developed in Sydney) and **SCOOT** (developed in the UK) adjust green times and cycle lengths from road sensors across whole city networks — the `QueueAdaptiveTiming` strategy scaled up. **Emergency vehicle pre-emption** systems (for example infrared or GPS-based emitters on fire trucks) request green for the vehicle's approach; the controller still times out the current yellow and red clearance before serving it, exactly like our `EmergencyHoldState`. In North American cabinets, a **conflict monitor / malfunction management unit** watches the outputs and forces the intersection into flash if it ever sees conflicting greens — our `SafetyMonitor` observer.

In the interview

Plan for 40 minutes: 5 min requirements (phases, timing, emergency, fault) → 5 min entities → 10 min draw the state machine (green → yellow → all-red → next; emergency and fault edges) → 15 min code `GreenState` / `YellowState` / `AllRedState` and `tick()` → 5 min concurrency and follow-ups.

30-second answer: "An intersection has phases — sets of approaches that can be green together — and a controller that is a timed State machine: Green, Yellow, All-Red, plus Emergency-Hold and Flashing-Red. A timer calls `tick()` once a second; each state counts down and names the next one. Green duration comes from a swappable timing Strategy, fixed or queue-adaptive. An emergency request only shortens the current green: yellow and all-red always run in full, then the emergency phase is held green until cleared. Lights publish changes to observers, including a safety monitor that checks no crossing flows are ever green together."

Follow-ups with answers

- *"Why not jump straight to the emergency green?"* — Cars already in the intersection need the yellow and all-red intervals to clear it; skipping them causes crashes.
- *"Why is the controller not a Singleton?"* — There is one per intersection, and a city has thousands.
- *"How do you make the timing adaptive?"* — Sensors report queue lengths; a `TimingPolicy` turns them into green seconds within a min/max.

Self-check

1. **Q:** What is a phase? **A:** A set of approaches that may be green together because their paths do not cross.
2. **Q:** Why the all-red interval? **A:** To let vehicles still in the intersection clear it before the crossing flow gets green.
3. **Q:** What happens to an emergency request that arrives during yellow? **A:** Yellow finishes; all-red then picks the emergency phase instead of the normal next phase.
4. **Q:** Why do these states hold data while the vending states do not? **A:** Each interval has its own countdown, so each state instance carries remaining seconds.
5. **Q:** Which pattern lets you switch from fixed to adaptive timing at run time? **A:** Strategy (`TimingPolicy`).
6. **Q:** What does the `SafetyMonitor` check? **A:** That all green/yellow approaches belong to a single phase whenever a light turns green.

TL;DR

- **Timed State machine**: Green → Yellow → All-Red → next Green; plus Emergency-Hold and Flashing-Red.
- **Safety first**: red before green, yellow never cut short, conflict-monitor observer.
- **Strategy** for timing (fixed/adaptive), **Observer** for lights, emergency as a flag the states read.
- Tick-driven and synchronized: deterministic tests, safe with a timer thread plus emergency threads.

14.7 Design a Shipping Locker System

A **parcel locker** (also called a *pickup locker* or *shipping locker*) is a wall of numbered steel doors in a shop, station or apartment lobby. A courier puts a parcel behind one door; the customer later types a **pickup code** on the station's screen and that one door opens. Amazon Hub Locker, InPost's Paczkomat network in Poland and similar networks run by couriers and online shops in many countries all work this way. Lockers save the courier a failed doorstep delivery and let the customer collect at any hour.

This question looks like the parking lot (§14.1) — hand out a limited set of spaces of different sizes — but adds three things interviewers like to probe: **a lifecycle with deadlines** (a reservation that times out, a parcel that expires if nobody collects it), **a secret** (the pickup code, which must be hard to guess and never shown to the courier), and **a network** of stations (if the nearest one is full, suggest another).

Problem & clarifying questions

Question	Assumed answer
What sizes do doors come in?	A few fixed sizes (small, medium, large). A parcel may go into a bigger door if no door of its own size is free.
When is a door chosen — at the station, or before?	Before: the carrier's system **reserves** a door when the parcel is sorted for delivery, so the courier never arrives to a full station. The hold lasts 2 hours; if nothing is deposited, the door is released.
How does the customer open the door?	A 6-digit **one-time code** sent by SMS/app when the parcel is deposited. The courier never sees it.
How long can a parcel stay?	3 days after deposit. After that it **expires** and the carrier takes it back to the sender on the next visit.
What stops someone guessing codes?	Codes are random and single-use; after 5 wrong codes in a row the keypad locks for 15 minutes.
Returns (customer drops a parcel off)?	Same flow in reverse — a carrier-side "code"; treated as an extension.
Many stations?	Yes, a city-wide network. If the chosen station is full, offer the nearest station with space.
Broken doors?	A technician can take a free door **out of service**.
Payment at the locker (cash on delivery)?	Out of scope (extension).

Requirements

Functional

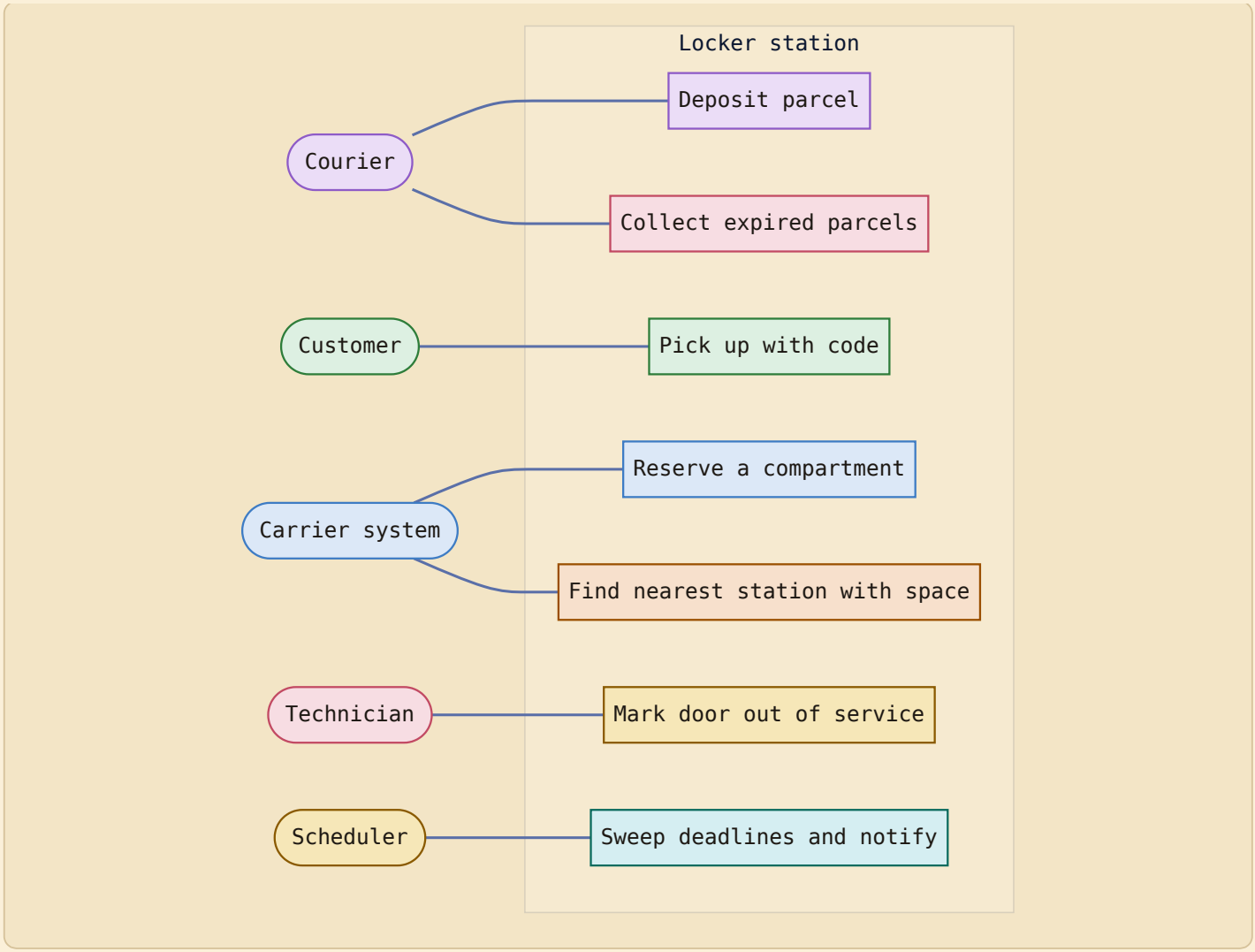
1. A station has numbered compartments of fixed sizes.
2. **Reserve** the best-fitting free compartment for a parcel (all-or-nothing); a reservation expires if the parcel is not deposited in time.
3. **Deposit**: the parcel goes in, a one-time pickup code is generated and **sent to the customer only**.
4. **Pickup**: a correct code opens exactly one door, frees the compartment and stops working.
5. **Expiry**: an uncollected parcel expires after the pickup window; the customer is told; the carrier **collects expired parcels** for return.
6. **Brute-force protection**: lock the keypad after too many wrong codes.
7. **Network search**: nearest station within a radius that has a free door this parcel fits.
8. Compartments can be taken out of service.

Non-functional

- **Never give one door to two parcels**, even when many carriers reserve at the same moment.
- **Security**: codes are unpredictable, single-use, stored only as a hash (the one-way fingerprint described in the ATM section).
- **Correct deadlines** without depending on a background timer being on time.
- **Testable**: time and randomness are injected (a fake clock and a seeded random generator in the demo — one started from a fixed seed, so it produces the same numbers every run).

Use cases

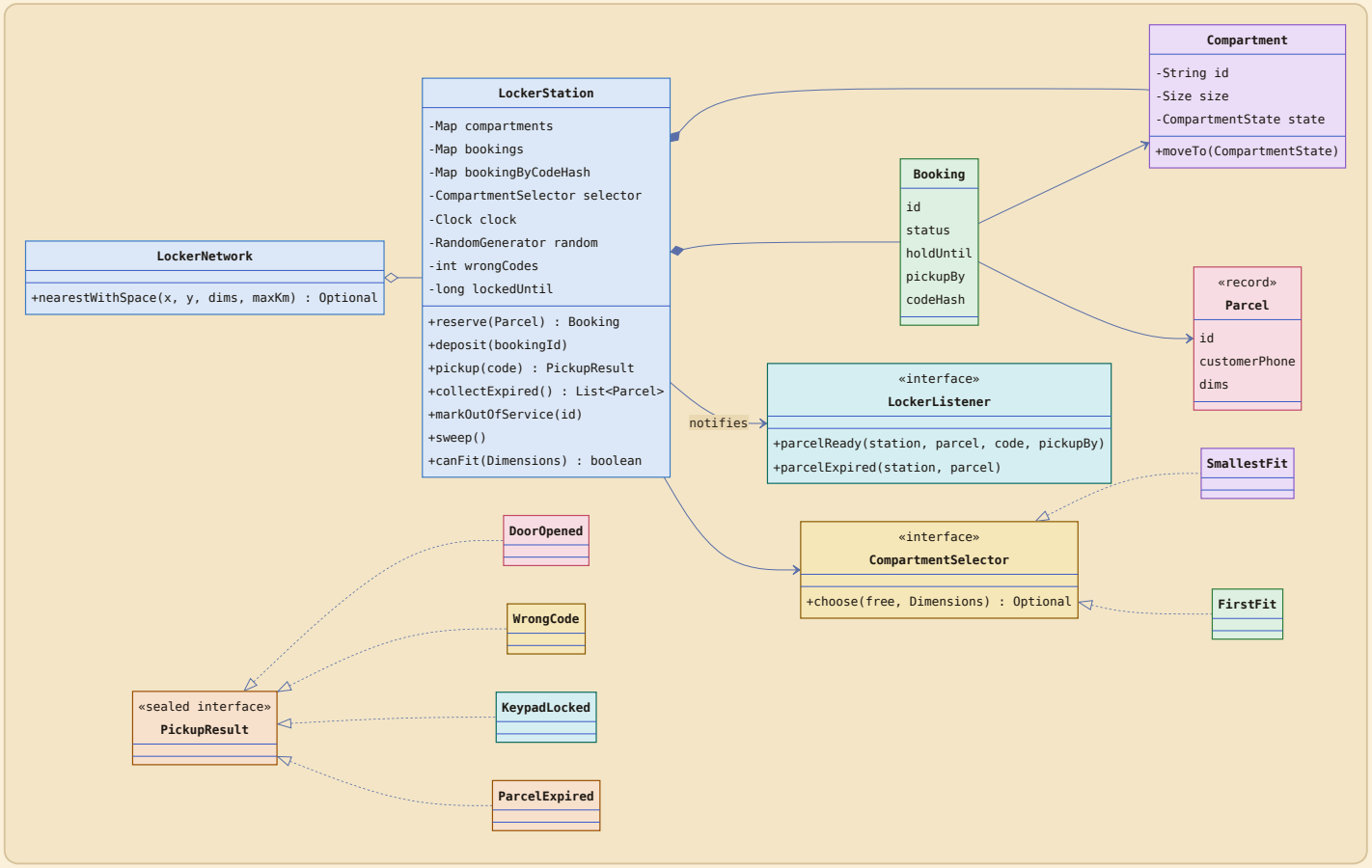
Actors: Carrier system (reserves doors), **Courier** (deposits, collects expired parcels), **Customer** (picks up), **Technician** (takes doors out of service), **Scheduler** (a component that runs a job at set times; here it runs the periodic sweep), **Notification service** (SMS/app).



Core entities

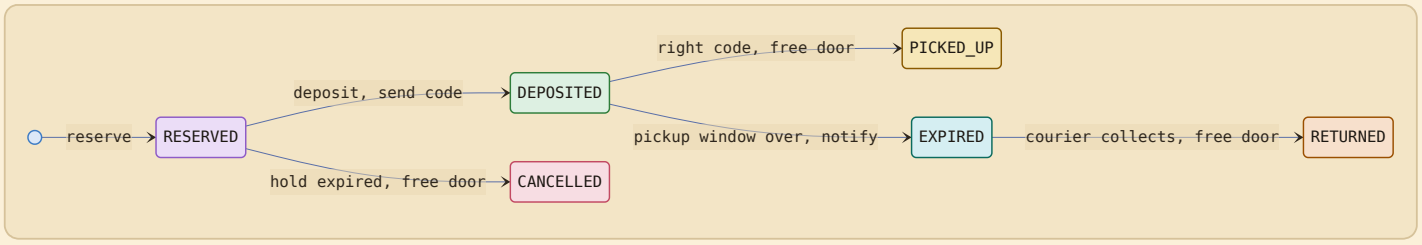
Class	Responsibility
`Size` (enum)	SMALL / MEDIUM / LARGE with inner dimensions; `fits(Dimensions)` checks a box against a door after sorting both sides.
`Dimensions` (record)	A parcel's three sides in cm (validated positive).
`Parcel` (record)	Parcel id, customer phone, dimensions.
`Compartment`	One door: id, size, `CompartmentState` (FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE) with the legal moves inside `moveTo`.
`Booking` + `BookingStatus`	One parcel's stay in one door: RESERVED → DEPOSITED → PICKED_UP, or → EXPIRED → RETURNED, or RESERVED → CANCELLED. Holds the deadlines and the code **hash**.
`PickupResult` (sealed)	`DoorOpened`, `WrongCode`, `KeypadLocked`, `ParcelExpired` — every keypad outcome is a type, not a magic string (a bare text value whose meaning is known only by convention).
`CompartmentSelector`	**Strategy**: `SmallestFit` (default), `FirstFit`.
`LockerListener`	**Observer**: parcel ready (with the code), parcel expired.
`LockerStation`	The station: doors, bookings, code index, lockout counter, clock, random generator; every state change under one lock.
`LockerNetwork`	All stations; nearest station with space.

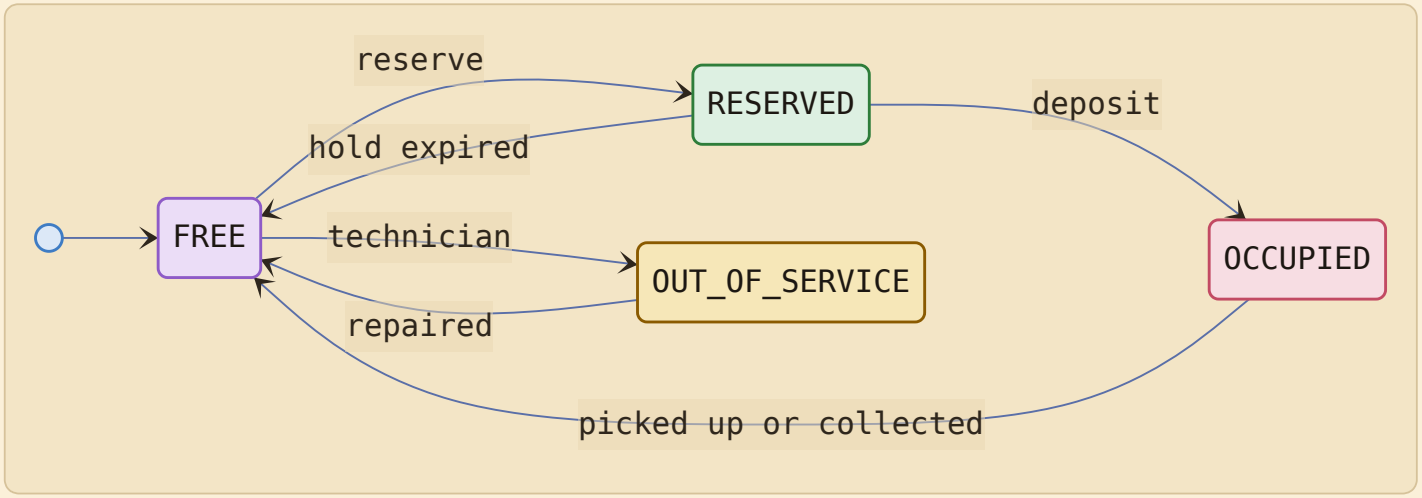
Class diagram



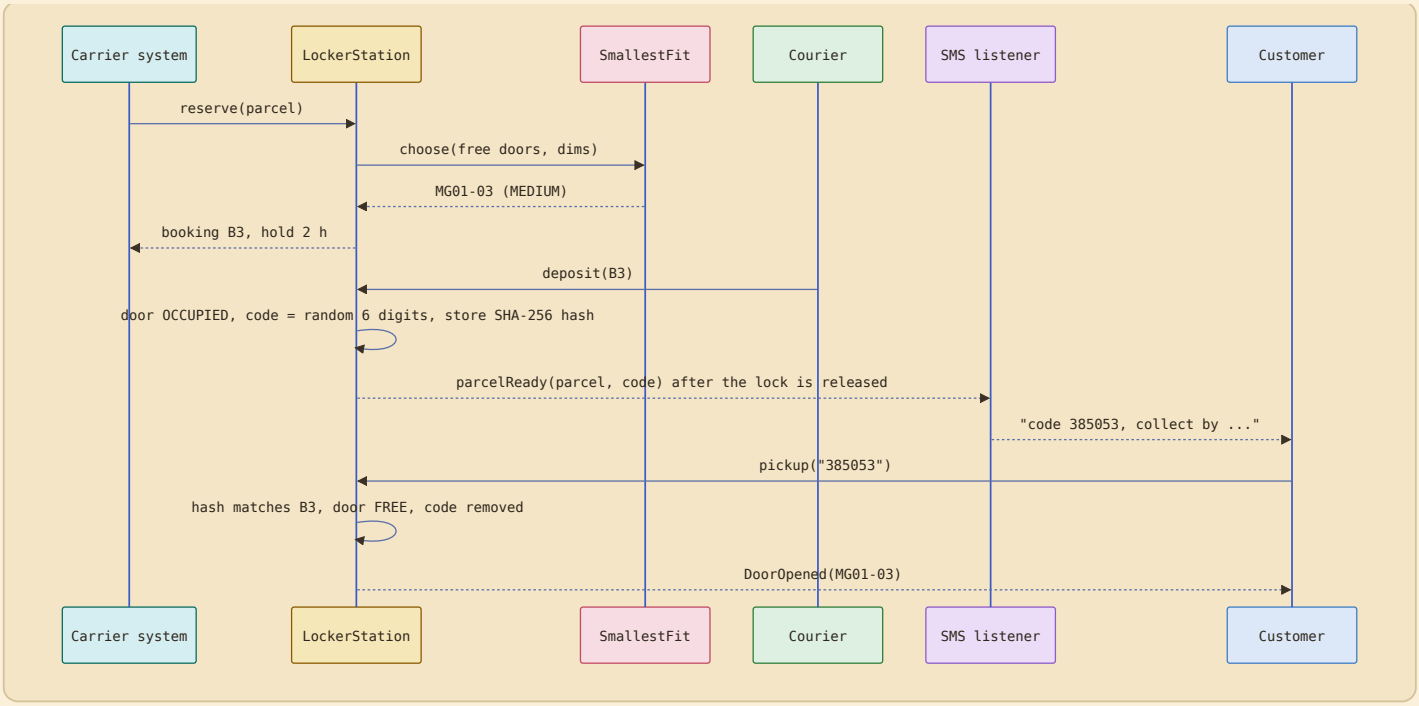
Key flows

The two state machines — first the booking, then the door. A compartment only knows whether it is free; a booking knows the parcel's story. Keeping them separate means a door can be reused while the old booking stays in the history.





Delivery and pickup.


Carrier system
LockerStation
SmallestFit
Courier
SMS listener
Customer
reserve(parcel)
choose(free doors, dims)
MG01-03 (MEDIUM)
booking B3, hold 2 h
deposit(B3)
door OCCUPIED, code = random 6 digits, store SHA-256 hash
parcelReady(parcel, code) after the lock is released
"code 385053, collect by ..."
pickup("385053")
hash matches B3, door FREE, code removed
DoorOpened(MG01-03)
Carrier system
LockerStation
SmallestFit
Courier
SMS listener
Customer

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Door choice is a `CompartmentSelector`	**Strategy**	"Smallest door that fits" keeps large doors for large parcels; a station may prefer "lowest door first" (fast for the courier) or "eye-level first" (accessibility). Each is a small class.	An `if (mode == ...)` inside `reserve`
Separate `Compartment` state and `Booking` status	Two small state machines	A door is reused many times; the booking keeps each parcel's history and deadlines	One status field on the door that mixes "free" with "expired parcel inside"
Transitions checked inside `moveTo` (a `switch` over the current state)	Enum state machine	An illegal move (e.g. taking an occupied door out of service) throws at once	Setting the field anywhere
Keypad outcomes as a **sealed** `PickupResult`	Sealed result types	The UI must handle every outcome; the compiler checks the `switch`	Returning `null`, `-1` or strings
Codes: random, single-use, unique per station, stored as a salted (mixed with a random value before hashing) **SHA-256** hash	Security by design	A database leak does not reveal live codes; the courier never sees them	Storing the plain code, or deriving it from the parcel id
Notify through `LockerListener` **after** releasing the lock	**Observer**	SMS, app push and audit log are independent; a slow SMS gateway (the service that actually sends text messages) must not block the keypad	Calling the SMS gateway inside `deposit` while holding the lock
Lazy expiry (deadlines checked whenever the data is next touched) at the start of every operation, plus a `sweep()` for notifications	Deadline checking on read	Correct even if the scheduler is late or down; the scheduler only sends messages	A reaper thread (a background thread that cleans up expired items) as the *only* thing that frees doors
`Clock` and `RandomGenerator` are injected	Dependency injection	Deterministic demo and tests; production passes `Clock.systemUTC()` and a `SecureRandom` (Java's cryptographically strong random generator)	`Instant.now()` and `new Random()` inside the class
One lock per station (`synchronized`)	Monitor	A station has tens of doors and each call is microseconds, so one lock is simple and never the bottleneck (the one part that limits overall speed); stations never share a lock	Per-door CAS like the parking lot — more code for no gain at this size

Java implementation

Package `lld.ch14.locker`.

```java
// File: lld/ch14/locker/Model.java
package lld.ch14.locker;

import java.util.Arrays;

/** Inner size of a compartment door, in centimetres (height x width x depth). */
enum Size {
    SMALL(10, 40, 60), MEDIUM(20, 40, 60), LARGE(40, 40, 60);

    private final int[] sorted;

    Size(int h, int w, int d) { sorted = new int[]{h, w, d}; Arrays.sort(sorted); }

    /** A box fits if, after sorting both sides, each parcel side is no longer than the matching compartment side.
*/
    boolean fits(Dimensions p) {
        int[] q = p.sorted();
        return q[0] <= sorted[0] && q[1] <= sorted[1] && q[2] <= sorted[2];
    }
}

/** Parcel box size in centimetres. */
record Dimensions(int a, int b, int c) {
    Dimensions {
        if (a <= 0 || b <= 0 || c <= 0) throw new LockerException("Dimensions must be positive");
    }
    int[] sorted() { int[] s = {a, b, c}; Arrays.sort(s); return s; }
    @Override public String toString() { return a + "x" + b + "x" + c + "cm"; }
}

record Parcel(String id, String customerPhone, Dimensions dims) { }

enum CompartmentState { FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE }

/** One door in a locker station. Its state is changed only by the station, under the station's lock. */
final class Compartment {
    private final String id;
    private final Size size;
    private CompartmentState state = CompartmentState.FREE;

    Compartment(String id, Size size) { this.id = id; this.size = size; }

    String id() { return id; }
    Size size() { return size; }
    CompartmentState state() { return state; }
    boolean isFree() { return state == CompartmentState.FREE; }

    void moveTo(CompartmentState next) {
        boolean ok = switch (state) {
            case FREE -> next == CompartmentState.RESERVED || next == CompartmentState.OUT_OF_SERVICE;
            case RESERVED -> next == CompartmentState.OCCUPIED || next == CompartmentState.FREE;
            case OCCUPIED -> next == CompartmentState.FREE;
            case OUT_OF_SERVICE -> next == CompartmentState.FREE;
        };
        if (!ok) throw new LockerException("Compartment " + id + ": illegal move " + state + " -> " + next);
        state = next;
    }

    @Override public String toString() { return id + "(" + size + ")"; }
}

/** Lifecycle of one parcel's stay in a locker. */
enum BookingStatus {
    RESERVED,      // carrier booked a door, parcel not yet inside
    DEPOSITED,     // parcel inside, pickup code sent
    PICKED_UP,     // customer opened the door with the code (final)
    EXPIRED,       // not collected in time; waiting for the carrier to take it back
    RETURNED,      // carrier removed the expired parcel (final)
```

```
    CANCELLED      // reservation hold ran out before the deposit (final)
}

/** One parcel's use of one compartment. */
final class Booking {
    final String id;
    final Parcel parcel;
    final Compartment compartment;
    BookingStatus status = BookingStatus.RESERVED;
    long holdUntil;          // epoch seconds: deposit must happen before this
    long pickupBy;           // epoch seconds: set at deposit
    String codeHash;         // only a hash of the pickup code is stored

    Booking(String id, Parcel parcel, Compartment compartment, long holdUntil) {
        this.id = id; this.parcel = parcel; this.compartment = compartment; this.holdUntil = holdUntil;
    }
}

/** What the customer's keypad entry produced. */
sealed interface PickupResult permits DoorOpened, WrongCode, KeypadLocked, ParcelExpired { }
record DoorOpened(String compartmentId, String parcelId) implements PickupResult { }
record WrongCode(int attemptsLeft) implements PickupResult { }
record KeypadLocked(long secondsLeft) implements PickupResult { }
record ParcelExpired(String parcelId) implements PickupResult { }

/** Observer: SMS/app notifications, carrier dashboards, audit logs. */
interface LockerListener {
    void parcelReady(String stationId, Parcel parcel, String pickupCode, long pickupBy);
    default void parcelExpired(String stationId, Parcel parcel) { }
}

final class LockerException extends RuntimeException {
    LockerException(String message) { super(message); }
}
```

```
// File: lld/ch14/locker/Selection.java
package lld.ch14.locker;

import java.util.Comparator;
import java.util.List;
import java.util.Optional;

/** Strategy: which free compartment should this parcel get? */
interface CompartmentSelector {
    Optional<Compartment> choose(List<Compartment> free, Dimensions parcel);
}

/** Smallest door that fits: keeps large doors free for large parcels (the usual default). */
final class SmallestFit implements CompartmentSelector {
    @Override public Optional<Compartment> choose(List<Compartment> free, Dimensions parcel) {
        return free.stream()
                .filter(c -> c.size().fits(parcel))
                .min(Comparator.comparing(Compartment::size).thenComparing(Compartment::id));
    }
}

/** Lowest door number that fits, whatever its size: simple and fast, but wastes large doors. */
final class FirstFit implements CompartmentSelector {
    @Override public Optional<Compartment> choose(List<Compartment> free, Dimensions parcel) {
        return free.stream()
                .filter(c -> c.size().fits(parcel))
                .min(Comparator.comparing(Compartment::id));
    }
}
```

```java
// File: lld/ch14/locker/LockerStation.java
package lld.ch14.locker;

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.time.Clock;
import java.time.Duration;
import java.util.ArrayList;
import java.util.EnumMap;
import java.util.HashMap;
import java.util.HexFormat;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.random.RandomGenerator;

/**
 * One locker station (a bank of doors at one address). Every state change happens under the
 * station's monitor lock: a station has tens of doors and each operation takes microseconds,
 * so one lock is simple and never the bottleneck.
 */
final class LockerStation {
    static final int MAX_WRONG_CODES = 5;

    private final String id;
    private final double x, y;                                  // location in km on a city grid
    private final Map<String, Compartment> compartments = new LinkedHashMap<>();
    private final Map<String, Booking> bookings = new HashMap<>();
    private final Map<String, String> bookingByCodeHash = new HashMap<>();
    private final List<LockerListener> listeners = new CopyOnWriteArrayList<>();
    private final CompartmentSelector selector;
    private final Clock clock;
    private final RandomGenerator random;
    private final Duration reservationHold, pickupWindow, lockout;
    private final List<Parcel> newlyExpired = new ArrayList<>();
    private int nextBooking = 1, wrongCodes = 0;
    private long lockedUntil = 0;

    /** layout = the size of each door, in door-number order (door 01 is the first element). */
    LockerStation(String id, double x, double y, List<Size> layout, CompartmentSelector selector,
                  Clock clock, RandomGenerator random) {
        this.id = id; this.x = x; this.y = y;
        this.selector = selector; this.clock = clock; this.random = random;
        this.reservationHold = Duration.ofHours(2);
        this.pickupWindow = Duration.ofDays(3);
        this.lockout = Duration.ofMinutes(15);
        for (int n = 1; n <= layout.size(); n++) {
            String cid = "%s-%02d".formatted(id, n);
            compartments.put(cid, new Compartment(cid, layout.get(n - 1)));
        }
    }

    /** Convenience layout: small doors first, then medium, then large. */
    static List<Size> layout(int small, int medium, int large) {
        List<Size> l = new ArrayList<>();
        for (int i = 0; i < small; i++) l.add(Size.SMALL);
        for (int i = 0; i < medium; i++) l.add(Size.MEDIUM);
        for (int i = 0; i < large; i++) l.add(Size.LARGE);
        return l;
    }

    String id() { return id; }
    double distanceTo(double px, double py) { return Math.hypot(x - px, y - py); }
    void addListener(LockerListener l) { listeners.add(l); }

    /** Carrier books a door before driving to the station. All-or-nothing: a door, or an exception. */
```

```
    synchronized Booking reserve(Parcel parcel) {
        expireOverdue();
        if (bookings.values().stream().anyMatch(b -> b.parcel.id().equals(parcel.id()) && isActive(b)))
            throw new LockerException("Parcel " + parcel.id() + " already has a locker");
        List<Compartment> free = compartments.values().stream().filter(Compartment::isFree).toList();
        Compartment c = selector.choose(free, parcel.dims())
                .orElseThrow(() -> new LockerException("No free compartment fits " + parcel.dims() + " at " + id));
        c.moveTo(CompartmentState.RESERVED);
        Booking b = new Booking(id + "-B" + nextBooking++, parcel, c, now() + reservationHold.toSeconds());
        bookings.put(b.id, b);
        return b;
    }

    /** Carrier puts the parcel in and closes the door. The code goes to the customer, never to the carrier. */
    void deposit(String bookingId) {
        Booking b;
        String code;
        synchronized (this) {
            expireOverdue();
            b = booking(bookingId);
            if (b.status != BookingStatus.RESERVED) throw new LockerException("Booking " + bookingId + " is " +
b.status);
            b.compartment.moveTo(CompartmentState.OCCUPIED);
            b.status = BookingStatus.DEPOSITED;
            b.pickupBy = now() + pickupWindow.toSeconds();
            do { code = "%06d".formatted(random.nextInt(1_000_000)); }       // unique among active codes here
            while (bookingByCodeHash.containsKey(hash(code)));
            b.codeHash = hash(code);
            bookingByCodeHash.put(b.codeHash, b.id);
        }
        for (LockerListener l : listeners) l.parcelReady(id, b.parcel, code, b.pickupBy);   // outside the lock
    }

    /** Customer types a code on the station keypad. */
    synchronized PickupResult pickup(String code) {
        expireOverdue();
        long t = now();
        if (t < lockedUntil) return new KeypadLocked(lockedUntil - t);
        String bookingId = bookingByCodeHash.get(hash(code));
        if (bookingId == null) {
            if (++wrongCodes >= MAX_WRONG_CODES) {                  // slow down code guessing
                lockedUntil = t + lockout.toSeconds();
                wrongCodes = 0;
                return new KeypadLocked(lockout.toSeconds());
            }
            return new WrongCode(MAX_WRONG_CODES - wrongCodes);
        }
        wrongCodes = 0;
        Booking b = bookings.get(bookingId);
        if (b.status == BookingStatus.EXPIRED) return new ParcelExpired(b.parcel.id());
        b.status = BookingStatus.PICKED_UP;
        b.compartment.moveTo(CompartmentState.FREE);
        bookingByCodeHash.remove(b.codeHash);                       // a code works once
        return new DoorOpened(b.compartment.id(), b.parcel.id());
    }

    /** Carrier's daily visit: open every door holding an expired parcel and take them for return. */
    synchronized List<Parcel> collectExpired() {
        expireOverdue();
        List<Parcel> taken = new ArrayList<>();
        for (Booking b : bookings.values())
            if (b.status == BookingStatus.EXPIRED) {
                b.status = BookingStatus.RETURNED;
                b.compartment.moveTo(CompartmentState.FREE);
                bookingByCodeHash.remove(b.codeHash);
                taken.add(b.parcel);
            }
        return taken;
    }
```

```
    synchronized void markOutOfService(String compartmentId) {
        Compartment c = compartments.get(compartmentId);
        if (c == null) throw new LockerException("Unknown compartment " + compartmentId);
        c.moveTo(CompartmentState.OUT_OF_SERVICE);                // only a FREE door can be taken out
    }

    synchronized Map<Size, Integer> freeBySize() {
        expireOverdue();
        Map<Size, Integer> m = new EnumMap<>(Size.class);
        for (Size s : Size.values()) m.put(s, 0);
        compartments.values().stream().filter(Compartment::isFree).forEach(c -> m.merge(c.size(), 1,
Integer::sum));
        return m;
    }

    synchronized boolean canFit(Dimensions d) {
        expireOverdue();
        return compartments.values().stream().anyMatch(c -> c.isFree() && c.size().fits(d));
    }

    /** Called by a scheduler (e.g. every minute): applies deadlines and tells customers, outside the lock. */
    void sweep() {
        List<Parcel> expired;
        synchronized (this) { expireOverdue(); expired = List.copyOf(newlyExpired); newlyExpired.clear(); }
        for (Parcel p : expired) for (LockerListener l : listeners) l.parcelExpired(id, p);
    }

    synchronized BookingStatus status(String bookingId) { expireOverdue(); return booking(bookingId).status; }

    /** Lazy expiry: every operation first applies the deadlines that have passed, so correctness never waits for a
timer. */
    private void expireOverdue() {
        long t = now();
        for (Booking b : bookings.values()) {
            if (b.status == BookingStatus.RESERVED && t >= b.holdUntil) {
                b.status = BookingStatus.CANCELLED;
                b.compartment.moveTo(CompartmentState.FREE);
            } else if (b.status == BookingStatus.DEPOSITED && t >= b.pickupBy) {
                b.status = BookingStatus.EXPIRED;                 // door stays OCCUPIED until the carrier
collects
                newlyExpired.add(b.parcel);                       // notified by sweep(), outside the lock
            }
        }
    }

    private static boolean isActive(Booking b) {
        return b.status == BookingStatus.RESERVED || b.status == BookingStatus.DEPOSITED || b.status ==
BookingStatus.EXPIRED;
    }

    private Booking booking(String bookingId) {
        Booking b = bookings.get(bookingId);
        if (b == null) throw new LockerException("Unknown booking " + bookingId);
        return b;
    }

    private long now() { return clock.instant().getEpochSecond(); }

    /** Store only a salted hash of each code, so a leaked database does not reveal live codes. */
    private String hash(String code) {
        try {
            byte[] h = MessageDigest.getInstance("SHA-256").digest((id + ":" +
code).getBytes(StandardCharsets.UTF_8));
            return HexFormat.of().formatHex(h);
        } catch (NoSuchAlgorithmException e) { throw new IllegalStateException(e); }
    }
}
```

```java
// File: lld/ch14/locker/LockerNetwork.java
package lld.ch14.locker;

import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.CopyOnWriteArrayList;

/** All stations in a city. Used at checkout ("pick a locker near you") and when a station is full. */
final class LockerNetwork {
    private final List<LockerStation> stations = new CopyOnWriteArrayList<>();

    void add(LockerStation s) { stations.add(s); }

    /** Nearest station within maxKm that has a free door this parcel fits in right now. */
    Optional<LockerStation> nearestWithSpace(double x, double y, Dimensions parcel, double maxKm) {
        return stations.stream()
                .filter(s -> s.distanceTo(x, y) <= maxKm)
                .sorted(Comparator.comparingDouble(s -> s.distanceTo(x, y)))
                .filter(s -> s.canFit(parcel))
                .findFirst();
    }
}
```

```java
// File: lld/ch14/locker/Main.java
package lld.ch14.locker;

import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneOffset;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.SplittableRandom;

public class Main {
    /** A clock the demo can move forward; production code gets Clock.systemUTC(). */
    static final class TestClock extends Clock {
        private volatile Instant now = Instant.parse("2026-10-01T09:00:00Z");
        void advance(Duration d) { now = now.plus(d); }
        @Override public Instant instant() { return now; }
        @Override public java.time.ZoneId getZone() { return ZoneOffset.UTC; }
        @Override public Clock withZone(java.time.ZoneId z) { return this; }
    }

    /** Fake SMS gateway: remembers the last code sent to each phone so the "customer" can type it. */
    static final class SmsInbox implements LockerListener {
        final Map<String, String> lastCode = new ConcurrentHashMap<>();
        @Override public void parcelReady(String station, Parcel p, String code, long pickupBy) {
            lastCode.put(p.customerPhone(), code);
            System.out.printf("   SMS to %s: parcel %s is in locker %s, code %s, collect by %s%n",
                    p.customerPhone(), p.id(), station, code, Instant.ofEpochSecond(pickupBy));
        }
        @Override public void parcelExpired(String station, Parcel p) {
            System.out.printf("   SMS to %s: parcel %s was not collected and will be returned%n",
p.customerPhone(), p.id());
        }
    }

    public static void main(String[] args) throws Exception {
        TestClock clock = new TestClock();
        SmsInbox sms = new SmsInbox();
        LockerStation s1 = new LockerStation("MG01", 0, 0,
                LockerStation.layout(2, 2, 1), new SmallestFit(), clock, new SplittableRandom(7));
        s1.addListener(sms);

        System.out.println("== 1. Carrier reserves doors (smallest door that fits) ==");
        Parcel phoneCase = new Parcel("P1", "+91-98450-00001", new Dimensions(3, 10, 18));
        Parcel book = new Parcel("P2", "+91-98450-00002", new Dimensions(4, 16, 24));
        Parcel mug = new Parcel("P3", "+91-98450-00003", new Dimensions(9, 12, 12));
        Parcel shoes = new Parcel("P4", "+91-98450-00004", new Dimensions(14, 22, 34));
        Booking b1 = s1.reserve(phoneCase), b2 = s1.reserve(book), b3 = s1.reserve(mug), b4 = s1.reserve(shoes);
        for (Booking b : List.of(b1, b2, b3, b4))
            System.out.printf("   %s %-12s -> %s%n", b.parcel.id(), b.parcel.dims(), b.compartment);
        System.out.println("   free now: " + s1.freeBySize() + "  (the mug got a MEDIUM: both SMALL doors were
taken)");
        try { s1.reserve(phoneCase); } catch (LockerException e) { System.out.println("   error: " +
e.getMessage()); }
        try { s1.reserve(new Parcel("P9", "+91-98450-00009", new Dimensions(50, 50, 70))); }
        catch (LockerException e) { System.out.println("   error: " + e.getMessage()); }

        System.out.println("\n== 2. Deposit: the code goes to the customer by SMS, never to the carrier ==");
        s1.deposit(b1.id);
        s1.deposit(b2.id);
        s1.deposit(b3.id);
        try { s1.deposit(b1.id); } catch (LockerException e) { System.out.println("   error: " + e.getMessage()); }
```

```
        System.out.println("\n== 3. Pickup ==");
        System.out.println("   typo 123456 -> " + s1.pickup("123456"));
        String code1 = sms.lastCode.get(phoneCase.customerPhone());
        System.out.println("   right code  -> " + s1.pickup(code1));
        System.out.println("   same code again -> " + s1.pickup(code1) + "  (a code works once)");

        System.out.println("\n== 4. Someone guesses codes: keypad locks after 5 wrong tries in a row ==");
        for (int i = 1; i <= 4; i++) System.out.println("   guess 00000" + i + " -> " + s1.pickup("00000" + i));
        String code2 = sms.lastCode.get(book.customerPhone());
        System.out.println("   real owner during lockout -> " + s1.pickup(code2));
        clock.advance(Duration.ofMinutes(15));
        System.out.println("   15 min later -> " + s1.pickup(code2));

        System.out.println("\n== 5. Reservation hold runs out (carrier never came with the shoes) ==");
        clock.advance(Duration.ofHours(2));
        System.out.println("   booking " + b4.id + " is " + s1.status(b4.id) + ", free now: " + s1.freeBySize());

        System.out.println("\n== 6. Mug not collected within 3 days ==");
        clock.advance(Duration.ofDays(3));
        s1.sweep();
        System.out.println("   customer types the code -> " + s1.pickup(sms.lastCode.get(mug.customerPhone())));
        System.out.println("   carrier collects for return: " +
s1.collectExpired().stream().map(Parcel::id).toList()
                + ", free now: " + s1.freeBySize());

        System.out.println("\n== 7. Station full -> suggest the nearest station with space ==");
        LockerStation tiny = new LockerStation("KR02", 1.0, 0.5, LockerStation.layout(1, 0, 0), new SmallestFit(),
clock, new SplittableRandom(7));
        LockerStation far = new LockerStation("WF03", 6.0, 2.0, LockerStation.layout(0, 3, 0), new SmallestFit(),
clock, new SplittableRandom(7));
        LockerNetwork city = new LockerNetwork();
        city.add(tiny); city.add(far); city.add(s1);
        tiny.reserve(new Parcel("P5", "+91-98450-00005", new Dimensions(2, 10, 10)));
        Dimensions box = new Dimensions(8, 20, 30);
        System.out.println("   customer at (1.0, 0.4), box " + box + " -> "
                + city.nearestWithSpace(1.0, 0.4, box, 5).map(LockerStation::id).orElse("none within 5 km"));
        System.out.println("   same box, max 0.2 km -> "
                + city.nearestWithSpace(1.0, 0.4, box, 0.2).map(LockerStation::id).orElse("none within 0.2 km"));

        System.out.println("\n== 8. FirstFit vs SmallestFit: doors 01 = LARGE, 02 = SMALL ==");
        LockerStation ff = new LockerStation("F", 0, 0, List.of(Size.LARGE, Size.SMALL), new FirstFit(), clock, new
SplittableRandom(7));
        LockerStation sf = new LockerStation("S", 0, 0, List.of(Size.LARGE, Size.SMALL), new SmallestFit(), clock,
new SplittableRandom(7));
        for (LockerStation st : List.of(ff, sf)) {
            String small = st.reserve(new Parcel("A", "x", new Dimensions(2, 5, 5))).compartment.toString();
            String big;
            try { big = st.reserve(new Parcel("B", "y", new Dimensions(30, 35, 50))).compartment.toString(); }
            catch (LockerException e) { big = "REJECTED"; }
            System.out.println("   station " + st.id() + ": small parcel -> " + small + ", then large parcel -> " +
big);
        }

        System.out.println("\n== 9. 50 carriers reserve at once on a 20-door station ==");
        LockerStation hub = new LockerStation("HUB", 0, 0, LockerStation.layout(0, 20, 0), new SmallestFit(),
clock, new SplittableRandom(7));
        Set<String> doorsGiven = ConcurrentHashMap.newKeySet();
        AtomicInteger ok = new AtomicInteger(), full = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(16)) {
            for (int i = 0; i < 50; i++) {
                int n = i;
                pool.submit(() -> {
                    go.await();
                    try {
                        doorsGiven.add(hub.reserve(new Parcel("C" + n, "z", new Dimensions(10, 20,
30))).compartment.id());
                        ok.incrementAndGet();
```

```
                    } catch (LockerException e) { full.incrementAndGet(); }
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("   reserved=" + ok + " rejected=" + full + " distinct doors=" + doorsGiven.size());
    }
}
```

How the code works, part by part

- `Size.fits` sorts the three sides of both the box and the door, then compares smallest-to-smallest. That handles a parcel lying on its side without trying all six rotations.
- `reserve` first applies overdue deadlines, rejects a parcel that already has a door, asks the selector for a free door, moves it to RESERVED and creates a `Booking` whose hold ends in 2 hours. It is `synchronized`, so "find a free door" and "mark it reserved" are one atomic step.
- `deposit` does its state change inside a `synchronized (this)` block, generates a 6-digit code (retrying on the rare clash with another live code at this station), stores only its hash, and calls the listeners **after** the block ends.
- `pickup` checks the lockout, looks the code up by hash, counts wrong attempts (5 in a row locks the keypad for 15 minutes), refuses an expired parcel, and on success frees the door and deletes the code so it cannot be used twice.
- `expireOverdue` runs at the top of every public method: RESERVED past its hold → CANCELLED (door freed); DEPOSITED past its window → EXPIRED (door stays OCCUPIED until the courier collects). `sweep()` is what a scheduler calls every minute; it runs the same check and sends the "not collected" messages outside the lock.
- `LockerNetwork.nearestWithSpace` sorts stations by distance, keeps those within the radius and returns the first that can fit the parcel right now.

Output:

```
== 1. Carrier reserves doors (smallest door that fits) ==
   P1 3x10x18cm    -> MG01-01(SMALL)
   P2 4x16x24cm    -> MG01-02(SMALL)
   P3 9x12x12cm    -> MG01-03(MEDIUM)
   P4 14x22x34cm   -> MG01-04(MEDIUM)
   free now: {SMALL=0, MEDIUM=0, LARGE=1}  (the mug got a MEDIUM: both SMALL doors were taken)
   error: Parcel P1 already has a locker
   error: No free compartment fits 50x50x70cm at MG01

== 2. Deposit: the code goes to the customer by SMS, never to the carrier ==
   SMS to +91-98450-00001: parcel P1 is in locker MG01, code 694696, collect by 2026-10-04T09:00:00Z
   SMS to +91-98450-00002: parcel P2 is in locker MG01, code 607553, collect by 2026-10-04T09:00:00Z
   SMS to +91-98450-00003: parcel P3 is in locker MG01, code 385053, collect by 2026-10-04T09:00:00Z
   error: Booking MG01-B1 is DEPOSITED

== 3. Pickup ==
   typo 123456 -> WrongCode[attemptsLeft=4]
   right code  -> DoorOpened[compartmentId=MG01-01, parcelId=P1]
   same code again -> WrongCode[attemptsLeft=4]  (a code works once)

== 4. Someone guesses codes: keypad locks after 5 wrong tries in a row ==
   guess 000001 -> WrongCode[attemptsLeft=3]
   guess 000002 -> WrongCode[attemptsLeft=2]
   guess 000003 -> WrongCode[attemptsLeft=1]
   guess 000004 -> KeypadLocked[secondsLeft=900]
   real owner during lockout -> KeypadLocked[secondsLeft=900]
   15 min later -> DoorOpened[compartmentId=MG01-02, parcelId=P2]

== 5. Reservation hold runs out (carrier never came with the shoes) ==
   booking MG01-B4 is CANCELLED, free now: {SMALL=2, MEDIUM=1, LARGE=1}

== 6. Mug not collected within 3 days ==
   SMS to +91-98450-00003: parcel P3 was not collected and will be returned
   customer types the code -> ParcelExpired[parcelId=P3]
   carrier collects for return: [P3], free now: {SMALL=2, MEDIUM=2, LARGE=1}

== 7. Station full -> suggest the nearest station with space ==
   customer at (1.0, 0.4), box 8x20x30cm -> MG01
   same box, max 0.2 km -> none within 0.2 km

== 8. FirstFit vs SmallestFit: doors 01 = LARGE, 02 = SMALL ==
   station F: small parcel -> F-01(LARGE), then large parcel -> REJECTED
   station S: small parcel -> S-02(SMALL), then large parcel -> S-01(LARGE)

== 9. 50 carriers reserve at once on a 20-door station ==
   reserved=20 rejected=30 distinct doors=20
```

Reading the output: the mug (9 × 12 × 12 cm) fits a small door but both were taken, so it was upgraded to a medium one; a second reservation for the same parcel and an oversized box are refused. Codes went only to the customers' phones. The keypad counted the earlier typo, so the fourth guess in section 4 hit the limit of five and locked it — even the real owner had to wait 15 minutes. The shoes' reservation lapsed after 2 hours and the door came back. After 3 days the mug expired, the customer's code now says so, and the courier's visit freed the door. `FirstFit` put a tiny parcel in the only large door and then had to refuse a large parcel; `SmallestFit` served both. Fifty concurrent reservations on a 20-door station produced exactly 20 bookings on 20 different doors.

Concurrency & edge cases

Situation	Handling
Two carriers reserve the last medium door at once	`reserve` is `synchronized`: choose + mark RESERVED is one step; the loser gets "no free compartment" (or a larger door)
Courier arrives after the hold ran out	`deposit` sees CANCELLED and refuses; the carrier system reserves again (possibly at another station)
Same parcel reserved twice (retry after a timeout)	Rejected while it has an active booking — make the carrier's reserve call idempotent (repeating it has the same effect as calling it once) by parcel id
Two live parcels get the same random code	Generation retries until the code is unique among live codes at this station
Brute-force guessing	10^6 possible codes, 5 wrong tries then a 15-minute lockout per station; codes also expire
Scheduler down for an hour	Lazy expiry still frees doors on the next operation; only the "expired" SMS is late
Slow SMS gateway	Listeners run outside the lock; in production they hand off to a queue (a buffer that a separate worker drains)
`canFit` says yes but `reserve` then fails (someone else took the door)	Expected: `nearestWithSpace` is a hint; the caller reserves and, on failure, tries the next station
Door jammed / broken	Technician marks a **free** door OUT_OF_SERVICE; an occupied door must be emptied first (`moveTo` enforces it)
Power cut or network loss at the station	Real lockers keep a local copy of live code hashes and sync events later; the design already keeps the station self-contained

Extensions & follow-up questions

1. **"Returns (customer drops off)?"** — A `DropOffBooking`: the customer gets a drop-off code, the door opens empty, the courier collects later. Same states with the roles swapped.
2. **"Let the customer extend the pickup window?"** — A method that moves `pickupBy` once, only while DEPOSITED; a policy object can decide how many extensions are allowed.
3. **"Cash on delivery?"** — Payment at the station screen before the door opens: a `PaymentStep` inserted into `pickup` (Chain of Responsibility or a small state machine per session).
4. **"Fairness when the station is nearly full?"** — Keep a few large doors for large parcels only (a `ReservedCapacitySelector` that refuses to upgrade small parcels when fewer than k large doors are free).
5. **"Scale to thousands of stations?"** — Stations are independent, so shard by station id (sharding = splitting the data across several servers, each owning a slice; see System Design Fundamentals); the network search uses a geo index (an index for fast "what is near this point?" queries: a geohash turns a map cell into a short string, an R-tree is a tree of nested rectangles) instead of sorting every station.
6. **"How would you store this in a database?"** — `compartment(id, station_id, size, state)`, `booking(id, parcel_id, compartment_id, status, hold_until, pickup_by, code_hash)`; reserve with `UPDATE compartment SET state='RESERVED' WHERE id=? AND state='FREE'` and check that one row changed.

Real-world example

Amazon Hub Locker sends the customer a pickup code by email or app and gives a limited number of days to collect before the parcel goes back — the `DEPOSITED → EXPIRED → RETURNED` path. InPost's Paczkomat lockers in Poland let couriers load many parcels in one visit and customers open a door with a code or a phone app; the network is dense enough that "nearest locker with space" matters. In both, the door is chosen by the system, not the courier, which is our `CompartmentSelector`.

In the interview

Plan for 40 minutes: 5 min questions (sizes, holds, codes, expiry) → 5 min entities → 8 min the two state machines → 15 min code `reserve`, `deposit`, `pickup` with the lock and the listener call outside it → 7 min security, concurrency and follow-ups.

30-second answer: "A station has doors of a few sizes. A carrier reserves one ahead of time; a Strategy picks the smallest free door that fits, and the choose-and-mark step runs under the station's lock, so two carriers never get the same door. On deposit I generate a random single-use code, store only its hash, and notify the customer through an observer after releasing the lock. Pickup matches the hash, frees the door and deletes the code; five wrong codes lock the keypad. Holds and pickup windows are checked lazily on every call, so doors come back even if the scheduler is late, and a network object suggests the nearest station with space."

Follow-ups with answers

- *"Why not let the courier choose any empty door?"* — Couriers would use big doors for small parcels; the system also needs to know the door before the courier arrives.
- *"Why store only a hash of the code?"* — Anyone who reads the database or logs could otherwise open doors.
- *"Why a separate Booking instead of putting the parcel on the Compartment?"* — The door is reused; the booking keeps deadlines and history per parcel.

Self-check

1. **Q:** How does `Size.fits` handle a box lying on its side? **A:** It sorts the three sides of both box and door and compares them in order.
2. **Q:** What happens to an expired parcel's door? **A:** It stays OCCUPIED until the courier runs `collectExpired`, which frees it.
3. **Q:** Why is the listener called after the `synchronized` block in `deposit`? **A:** A slow or failing SMS call must not hold the station lock.
4. **Q:** What frees a door if the scheduler never runs? **A:** Lazy expiry at the start of every public method.
5. **Q:** Which pattern lets you switch from smallest-fit to first-fit? **A:** Strategy (`CompartmentSelector`).
6. **Q:** Why is one lock per station enough here but not in a parking lot of 5,000 spots with 20 gates? **A:** A station has tens of doors and few concurrent callers; the lock is held for microseconds.

TL;DR

- Doors of fixed sizes; a **Strategy** picks the smallest free door that fits; choose-and-mark is **atomic** under the station lock.
- Two small **state machines**: door (FREE/RESERVED/OCCUPIED/OUT_OF_SERVICE) and booking (RESERVED → DEPOSITED → PICKED_UP / EXPIRED → RETURNED / CANCELLED).
- Codes are **random, single-use, hashed**, sent only to the customer via an **Observer** outside the lock; lockout after 5 wrong tries.
- Deadlines are enforced **lazily** on every call; a sweep only sends messages. Inject clock and randomness.

Chapter summary

Problem	Core difficulty	Key data structure	Key patterns	Concurrency tool
Parking lot	Allocate spots without double booking	Concurrent sorted set of free spots per type	Strategy, Factory, Observer	`pollFirst` + CAS, per-ticket lock, `putIfAbsent`
Vending machine	Legal order of actions; exact change	`EnumMap` of coin counts; DP table	State	`synchronized` machine
Coffee machine	All-or-nothing multi-ingredient deduction	`EnumMap` levels, recipes as data	Observer, sealed results	One `ReentrantLock`; notify outside the lock
ATM	Money correctness and authentication	Total vs available balance; note chain	State, Chain of Responsibility, Command	Per-account lock; ordered locking for transfers
Elevator	Scheduling and dispatching	Two `TreeSet`s (up/down stops)	Strategy (+ modes)	Per-car lock, synchronized dispatcher
Traffic signal	Timed transitions with a safety invariant	Phases as data	State, Strategy, Observer	Synchronized tick; scheduled executor
Shipping locker	Best-fit door, holds and pickup windows, secret codes	Doors per station; bookings; code-hash index	Strategy, Observer, enum state machines, sealed results	One lock per station; lazy expiry; notify outside the lock

Chapter self-check

1. **Q:** Which two problems use the State pattern with *stateless* states, and which uses states *with data*? **A:** Vending machine and ATM share stateless singletons; the traffic signal's states hold countdowns, so a new instance is created per transition.
2. **Q:** Name the check-then-act race in each of parking, coffee and ATM, and its fix. **A:** Two gates take one spot (atomic `pollFirst`/CAS); two orders take the last milk (one lock around check+deduct); two ATMs overdraw one account (check+debit under the account lock).
3. **Q:** Where did we reject a Singleton, and why? **A:** Parking lot, coffee machine, traffic controller: a Singleton is a hidden global, blocks testing several instances, and real deployments have many (lots, machines, intersections).
4. **Q:** Where does greedy fail and what replaces it? **A:** Change-making with limited coins (vending) and note selection with limited cassettes (ATM); bounded coin-change dynamic programming.
5. **Q:** Which designs are tick-driven, and why is that useful? **A:** Elevator and traffic signal: deterministic, clock-free tests; a real timer just calls `tick()`/`step()`.
6. **Q:** Which case enforces deadlines *lazily*, and why? **A:** The shipping locker: every call first expires overdue holds and parcels, so doors come back even if the scheduler is late; the scheduled sweep only sends messages.

Chapter TL;DR

- Find the **one hard method** in each problem (spot allocation, change-making, ingredient deduction, withdrawal, next stop, next phase) and spend your time there.
- **State** for machines whose buttons mean different things over time; **Strategy** for rules that change (pricing, dispatch, timing); **Observer** for boards, alerts and monitors; **Chain of Responsibility** and **Command** for the ATM.
- Make every **check-then-act** atomic, lock at the **smallest** useful scope, lock in a **fixed order**, and call listeners **outside** locks.
- Prefer injected instances over Singletons, integer cents over doubles, and tick-driven logic over sleeps.
- For anything with a **deadline** (holds, pickup windows), check it on every call and keep secrets (pickup codes) **hashed and single-use**.

Chapter 15 — Case Studies: Booking & Inventory (Part 1)

> **Where this fits.** Chapter 14 designed small, self-contained objects. This chapter is the first of two on **booking systems**: software that sells or lends a limited thing (a book copy, a cinema seat, a concert place, a hotel night) to many people who all want it at once. Chapter 16 continues with more booking problems. Everything here uses the case-study template from Chapter 1 and the concurrency tools from Chapters 12–13.

Why booking problems are interview favourites

A booking problem looks like simple CRUD ("create, read, update, delete") until two people want the same thing. Then five hard questions appear, and every problem in this chapter is a different mix of them:

Theme	The question the interviewer is really asking	Where you will see it
Inventory	What exactly is the thing being sold or lent? A title or a copy? A seat or a seat *for one show*? A room or a room *on one night*?	`Book` vs `BookItem` (15.1), `HallSeat` vs `ShowSeat` (15.2), GA (general admission) count vs numbered seat (15.3), `Room` + booked nights (15.4)
Holds / locks	How do you keep something for a customer while they pay, without keeping it forever?	Seat lock with a TTL (time-to-live) (15.2), CAS holds + reaper (15.3)
Double-booking prevention	What stops two threads (or two servers) from both succeeding?	`synchronized` per item (15.1, 15.4), one monitor per show (15.2), lock-free compare-and-set (15.3)
Payments	What happens if payment fails? If it succeeds but the hold was lost meanwhile?	Release on decline, refund as **compensation** (15.2, 15.3)
Cancellations	Who gets the thing next, and how much money comes back?	Waitlists (15.1, 15.3), refund policies (15.2, 15.4)

Three words you will meet in every section:

- **Race condition** — two threads read the same data, both decide "it is free", and both act. One result silently overwrites the other.
- **Check-then-act** — the pattern "if free, then take it". It is only safe if the check and the act happen as one indivisible (atomic) step.
- **Hold** (also called a *lock*, *soft lock* or *reservation*) — a temporary claim with an expiry time, so an abandoned checkout does not block the item forever.

> **Key terms — the other words in the table above.** A **thread** is one independent line of execution inside a program; a server runs many at once, and they can all touch the same objects. A **lock** lets only one thread at a time run a piece of code; Java's `synchronized` keyword uses the built-in lock that every object carries, called its **monitor**. **Compare-and-set (CAS)** is a single atomic step that says "change this value to *new*, but only if it still equals *expected*"; code built on CAS instead of locks is called **lock-free**, because no thread ever waits for another. **Contention** means many threads competing for the same item or lock at the same moment. A **reaper** is a background job that sweeps away expired holds. **Compensation** is an action that undoes an earlier step after the fact — for money, a refund.

What you will learn

#	Problem	The one idea that carries the design
15.1	Library Management System	A *title* (`Book`) is not a *copy* (`BookItem`). Limits live with the member; the waitlist lives with the title.
15.2	Movie Ticket Booking (BookMyShow)	You book a `ShowSeat`, not a physical seat. Hold with TTL → pay → commit; refund if the hold was lost.
15.3	Concert Ticket Booking	Extreme contention: lock-free CAS, a counter for general admission, a fair virtual queue, per-user limits, a waitlist.
15.4	Hotel Management System	A status field is not availability. Each room remembers which nights are already sold.

How to run every example in this chapter

```
cd chapters/15-case-booking-1/code
javac --release 21 -d /tmp/lld_15 $(find . -name '*.java')
java -cp /tmp/lld_15 lld.ch15.library.Main        # or moviebooking, concert, hotel
```

All four `Main` classes compile with Java 21 (standard library only) and run. The output under each implementation is the real output of that program. In the concurrency demos, *which* thread wins a race changes from run to run; the *number* of winners never does.

> **Code-style note.** To keep each design readable in one place, a file sometimes holds several small package-private types (types declared without `public`, visible only inside their own package) (for example `Model.java` holds the enums and records of the library). That is legal Java. In a production codebase you would usually give each public type its own file. (If you compile with `-Xlint:all` — the switch that turns on every optional compiler warning — javac, the Java compiler, prints `[auxiliaryclass]` warnings for exactly this: a package-private type declared in one file and used from another. They are style warnings, not errors.)

> **Money.** Every amount (prices, fines, deposits, refunds) is a `java.math.BigDecimal` (Java's exact decimal-number type) built from a string such as `new BigDecimal("0.50")`, never a `double`: binary floating point cannot store 0.10 exactly, so sums drift. Comparisons use `compareTo`/`signum()`, not `equals` (which also compares the number of decimal places), and every rounding step names its `RoundingMode` (the rounding rule, for example half-up).

> **A time trick used in every demo.** Holds expire and fines grow with time. Instead of `Thread.sleep` (pausing for real time), every service takes a `java.time.Clock`. The demos pass a tiny `SimClock` (a `Clock` subclass whose `advance(Duration)` moves time forward). This is standard practice for testable time-dependent code: production passes `Clock.systemUTC()`, tests pass a controllable clock.

15.1 Design a Library Management System

Figure — Cover image for the library case study (course diagram)

Problem & clarifying questions

A **Library Management System** (LMS) is the software a library uses for its daily work: it tracks every book and every physical copy, who borrowed which copy and when it is due, each member's account and card, reservations, and fines. New stock is entered into it, and every loan is recorded with a due date.

Always clarify scope first. Good questions, with the answers we will design for:

Question	Answer we assume
Can a title have several physical copies?	Yes. Each copy is a **book item** with its own barcode and rack position.
How many books can one member hold?	At most **5** at a time.
How long can a member keep a book?	At most **10 days**, then a fine per day late.
Can a member reserve a book that is out?	Yes — and be told when a copy comes back.
How are people and books identified at the desk?	Every book item and every library card has a **unique barcode**; the system scans both.
Do we collect the fine money?	We *calculate and record* fines. Taking payment is out of scope (we say where it would go).
E-books, inter-library loans, lost-copy replacement?	Out of scope. Name them as extensions.

Requirements

Functional

1. Search the catalog by **title, author, subject category and publication date**.
2. Each book has a unique identification number (ISBN, the International Standard Book Number printed on every published book) and details; each copy has a **rack number** to find it physically.
3. A title can have **many copies** (book items); members check out and reserve copies.
4. Answer **"who took this copy?"** and **"which books does this member have?"**.
5. Limit: **max 5 books** per member.
6. Limit: **max 10 days** per loan.
7. **Fines** for books returned after the due date.

8. **Reserve** books that are not currently available.
9. **Notify** a member when a reserved book becomes available, and when a book is overdue.
10. **Barcodes** on every book item and every library card.

Non-functional

- **Correctness under concurrency:** two members at two desks can never both borrow the last copy; a member can never exceed 5 books even if two librarians scan for them at once.
- **Fast search:** searches are by far the most common operation, so they should be index lookups, not scans (an **index** is a lookup structure, such as a map from author name to books, that jumps straight to the matches instead of checking every book).
- **Extensibility:** fine rules and notification channels change; they should be swappable without editing the lending code.

Use cases

Three actors:

- **Librarian** — adds, edits and removes books, book items and members; can also issue, reserve and return items on a member's behalf.
- **Member** — searches the catalog; checks out, reserves, renews and returns books.
- **System** — sends notifications (overdue books, a reserved book is back, cancelled reservations).

Top use cases: add/remove/edit book (and book item); search catalog; register new account / cancel membership; **check out** a book; **reserve** a book that is out; **renew** (borrow the same copy again); **return** a book.

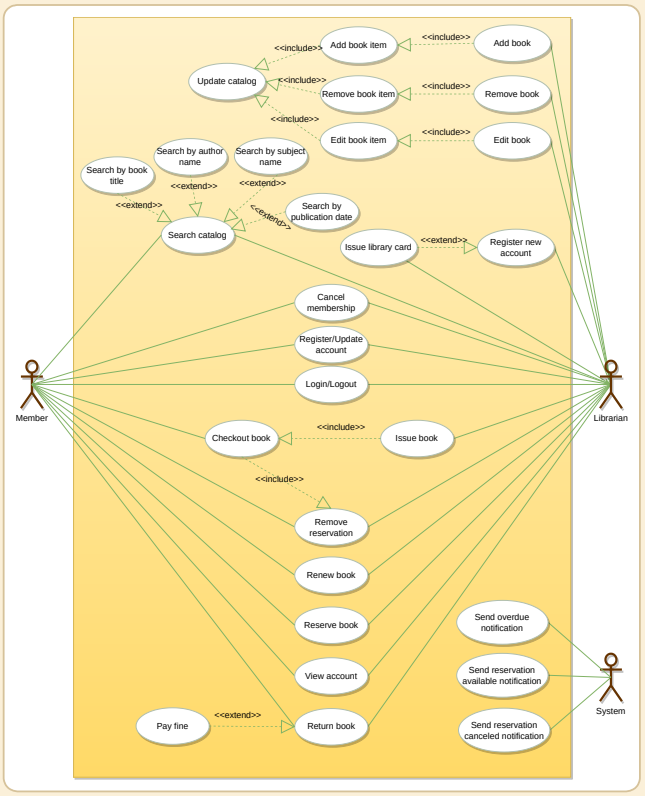


Figure — Library use cases: Member and Librarian share most of them (log in, search, check out, renew, reserve, return); the four "search by" use cases extend "Search catalog"; add/remove/edit book and book item feed "Update catalog"; "Checkout book" includes "Remove reservation"; the librarian's "Issue book" includes "Checkout book"; "Issue library card" is attached to "Register new account" (course diagram)

In the figure above, notice the two relationship kinds. `<<include>>` means a use case *always* performs another one as part of it — checking out a book always clears the member's reservation for it, if they had one. `<<extend>>` means an *optional variation* of a base use case — searching by author is one particular way a catalog search can go.

Core entities

Class	Responsibility
`Book` (record)	The **title**: ISBN, title, subject, publisher, publication date, authors. Immutable.
`BookItem`	One **physical copy**: barcode, format, rack, price, status (`AVAILABLE`, `RESERVED`, `LOANED`, `LOST`), and who it is held for.
`Author`, `Rack` (records)	An author's name; a rack's number and location.
`Account` (sealed, abstract) → `Member`, `Librarian`	People. An account **owns** its `LibraryCard` (composition). A member carries the loan count.
`LibraryCard` (record)	Card number, barcode, issue date.
`BookLending`	One loan: copy barcode, member id, creation date, **due date**, return date.
`Fine` (record)	Amount owed for one late return.
`Search` (interface) / `Catalog`	Four search methods; the catalog keeps one index per search and knows every copy of every title.
`LendingService`	The rules: check out, return, renew, reserve, overdue notices.
`FinePolicy` / `PerDayFine`	How much a late return costs (Strategy: a swappable rule behind an interface).
`Notifier` / `ConsoleNotifier`	Where messages go (email, SMS… here: console).
`Library`	Facade (one simple front object for the whole subsystem): owns the catalog and the lending service; only librarians may add stock.

In the table, a **record** is a short Java class that only carries values; **immutable** means it cannot change after it is created; an **enum** is a type with a fixed list of named constants; an **abstract** class cannot be created directly, only its subclasses can; and a **sealed** type lists exactly which subclasses it allows, so the compiler knows them all.

The course lists the same core classes: **Library** (name, address), **Book**, **BookItem**, **Account** (member or librarian), **LibraryCard**, **BookReservation**, **BookLending**, **Catalog** (searching by title, author, subject, publish date), **Fine**, **Author**, **Rack** (number + location identifier) and **Notification**. We keep every one of them; the only renames are `BookReservation` → a per-title waitlist inside `LendingService` (explained under *Design decisions*) and `Notification` → the `Notifier` interface.

Class diagram

The course builds its diagram in five stages, adding a few classes each time and greying out earlier ones. Walking through the stages is a good way to *present* a design in an interview, because every box arrives with a reason.

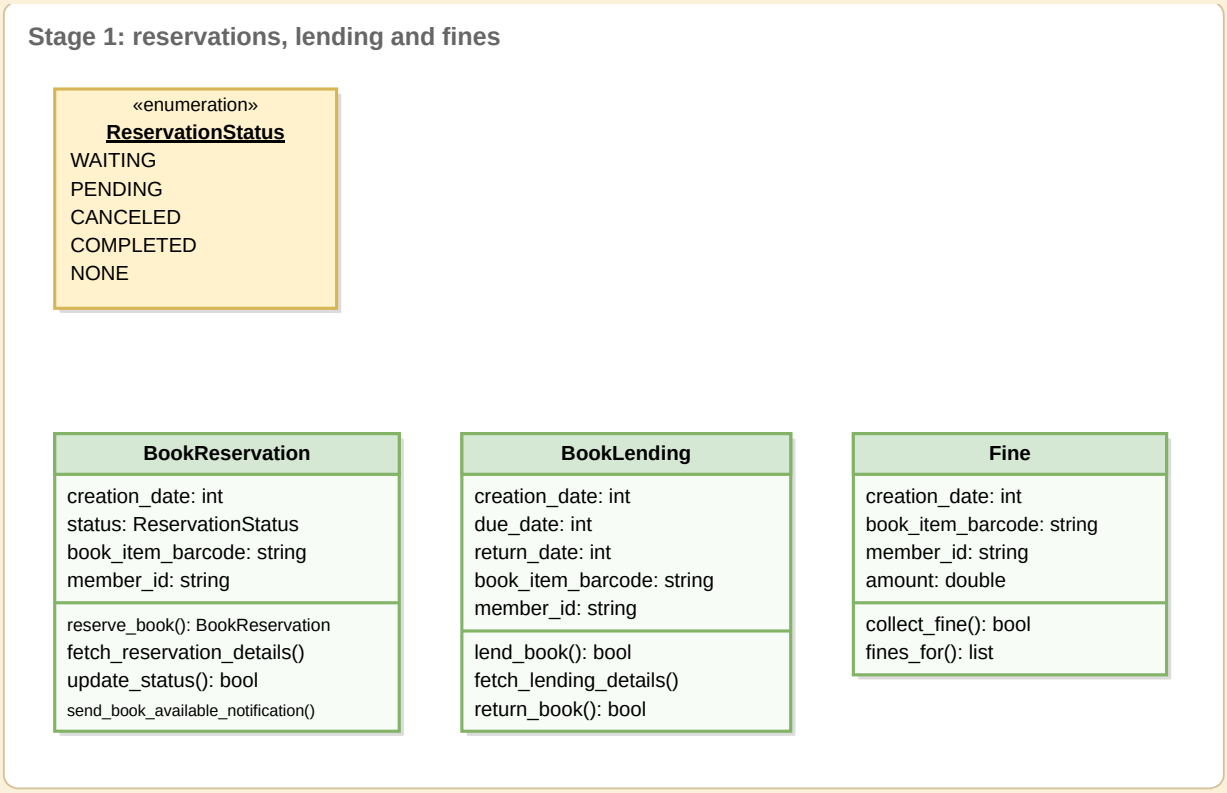


Figure — Stage 1: the three "what happened" records — `BookReservation`, `BookLending` and `Fine` — each keeping a copy's barcode and a member id (course diagram)

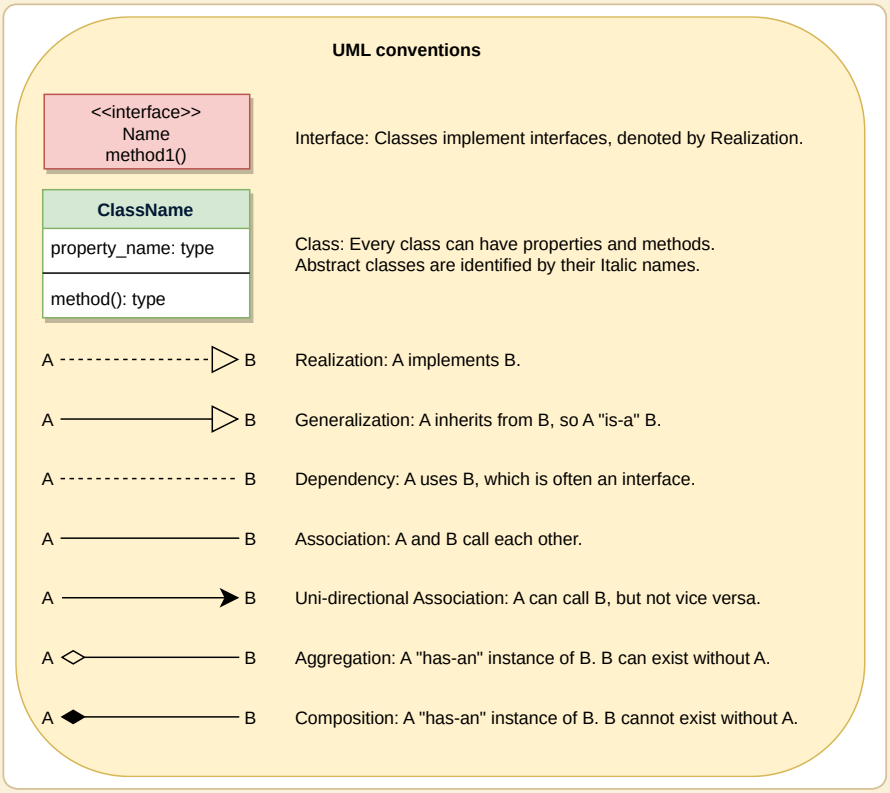


Figure — How to read a class box (name, `property: type`, `method(): type`) and the arrow styles used in these diagrams (course diagram)

Stage 1 answers requirements 4, 7 and 8, which are all about *remembering what happened*. Because each record stores the copy's barcode and the member's id, both halves of "who took this book / what does this member have" are answered by scanning these records.

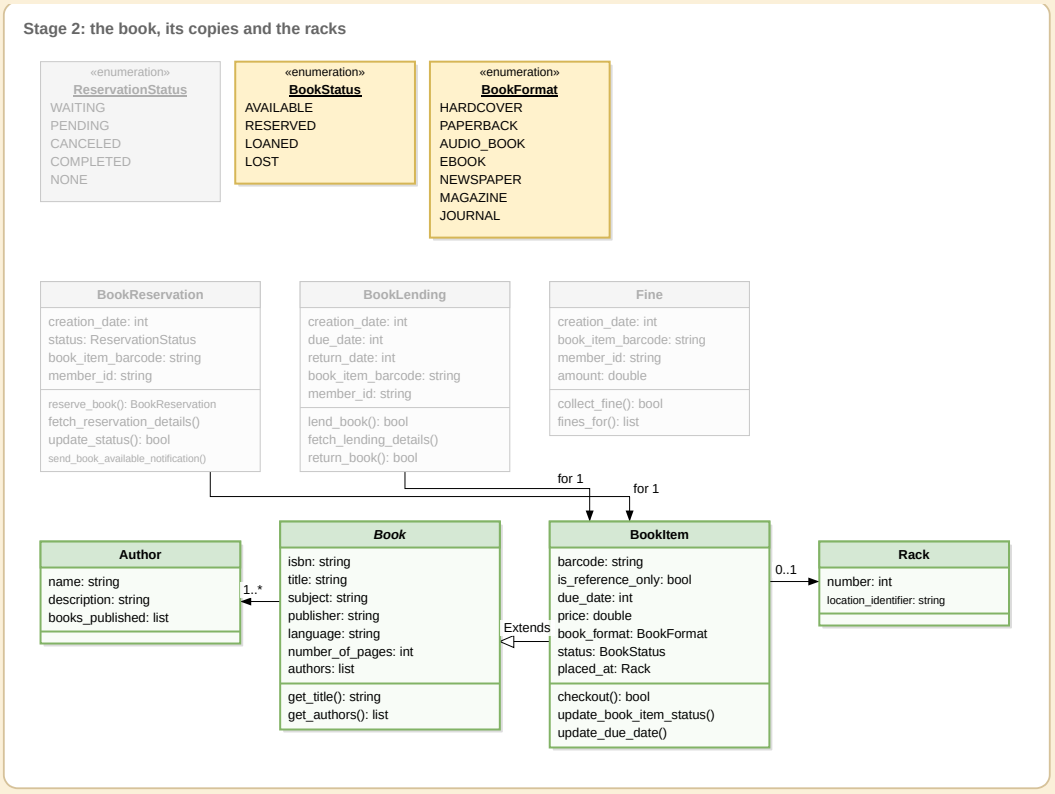


Figure — Stage 2: `Book` (the title) and `BookItem` (one copy with barcode, price and rack), plus `Author` and `Rack`; every reservation and every loan refers to one single copy (course diagram)

Stage 2 is the split that carries the whole design. In the course's diagram `BookItem` *inherits* from `Book`: the reading is that a copy carries all of the title's details and adds its own barcode, price and rack position. In our Java we use **`composition`** instead (`BookItem` *has a* `Book`) — see *Design decisions* for why.

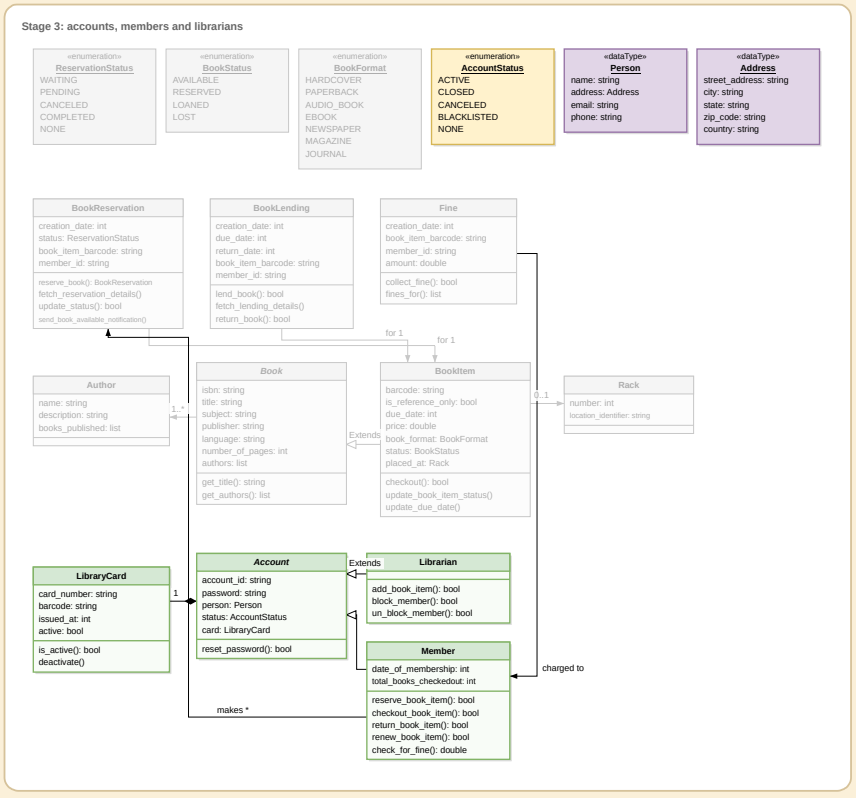


Figure — Stage 3: abstract `Account` with `Member` and `Librarian`; the account owns its `LibraryCard` (filled diamond = composition); a fine is charged to a member; a member makes reservations (course diagram)

Requirements 5 and 6 restrict *what one member may do*, so they belong to the people classes. `Account` is an **`abstract class`**: you can never create a plain `Account`, only a `Member` or a `Librarian`. The **`filled diamond`** is composition — the card has no life of its own, so closing the account destroys the card too.

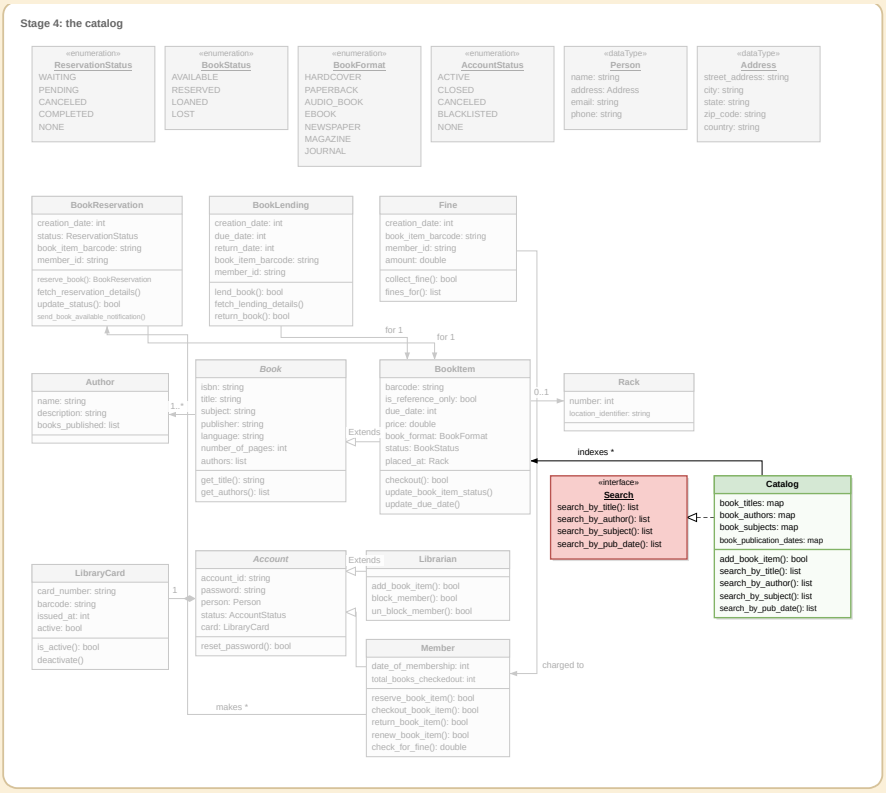


Figure — Stage 4: the `Search` interface (four methods) and `Catalog` implementing it (dashed line, hollow head), indexing book items without owning them (course diagram)

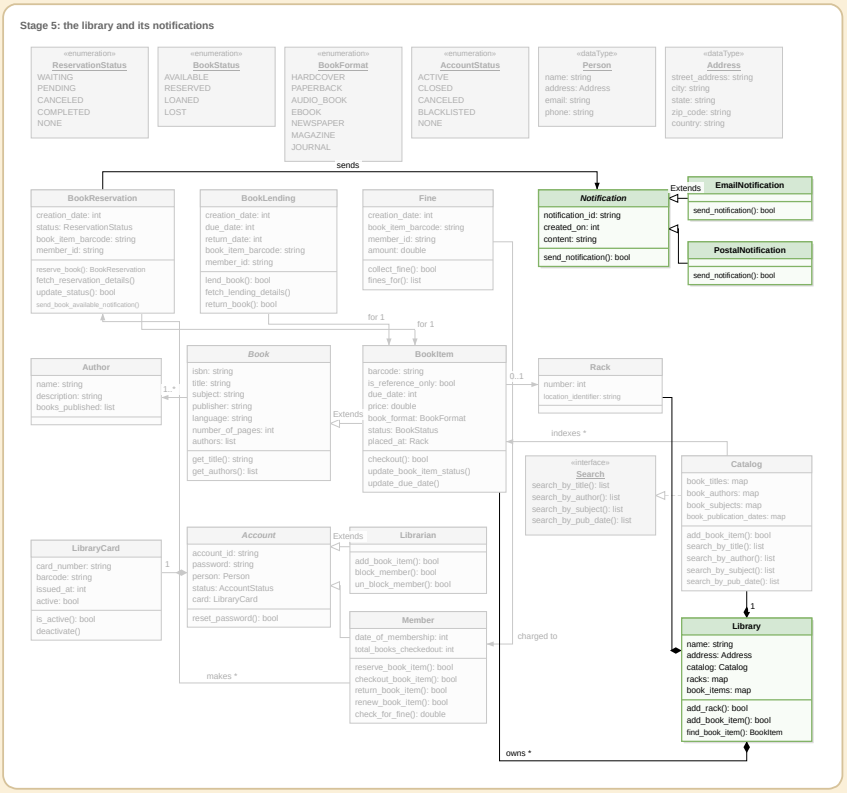


Figure — Stage 5: `Library` is the owner of the catalog, every rack and every book item; `Notification` is abstract with one subclass per delivery channel; a reservation triggers a notification when its copy comes back (course diagram)

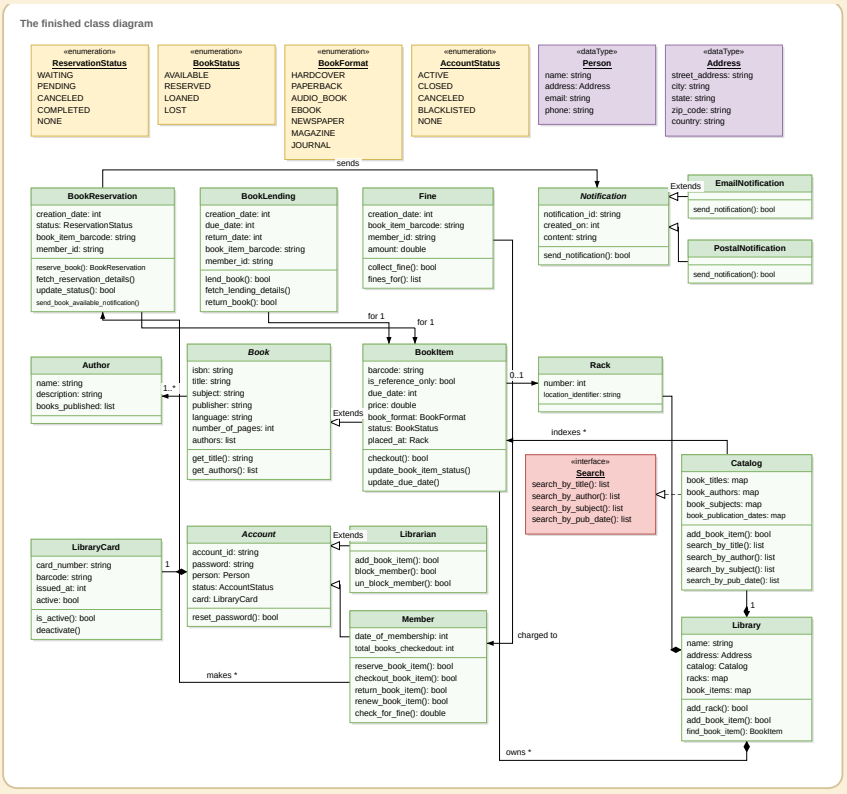
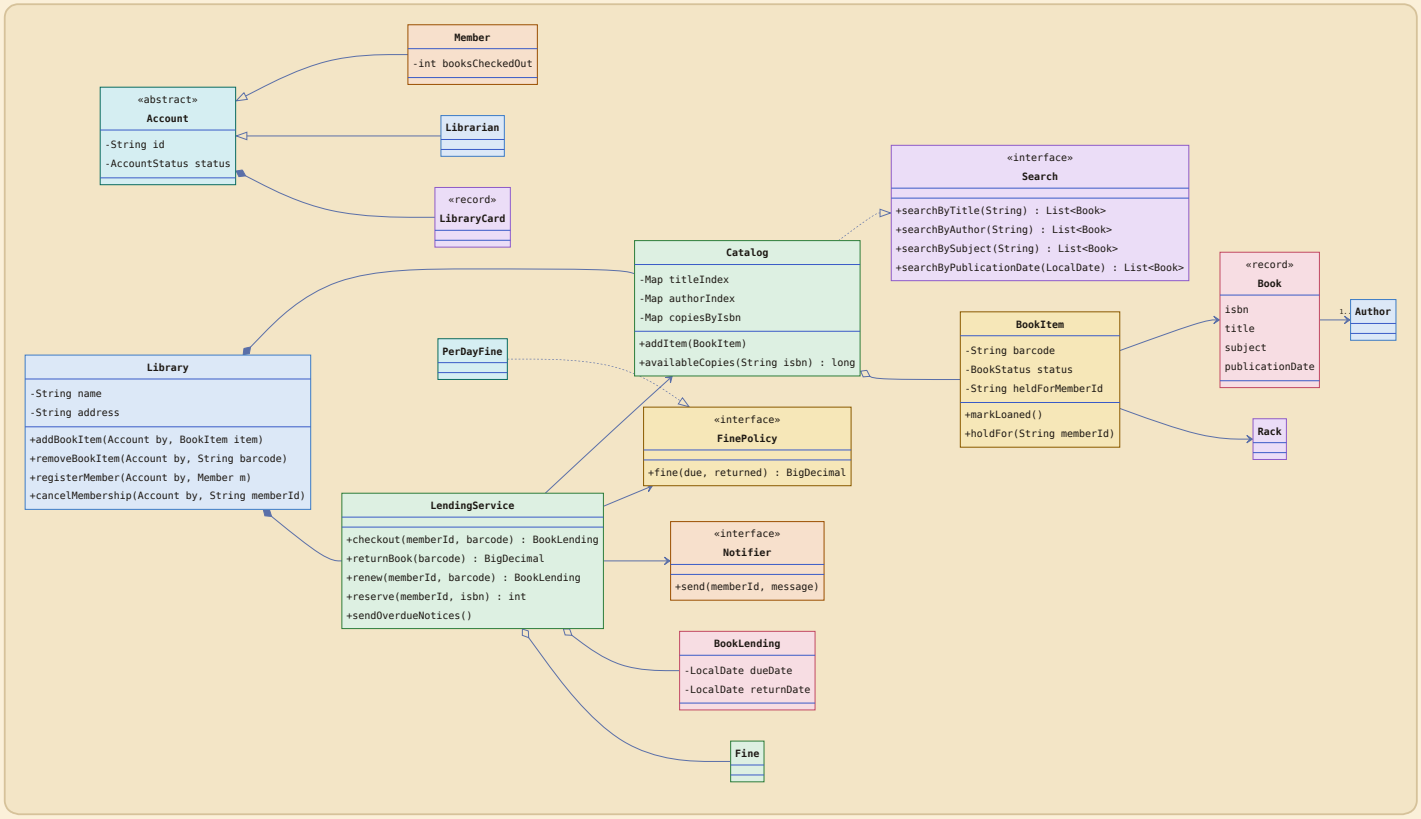


Figure — The finished library class diagram: every class from the five stages in full colour (course diagram)

Here is the class diagram of **our Java code** (the one you would draw on the whiteboard):



Key flows

The course gives three activity diagrams — check out, return and renew.

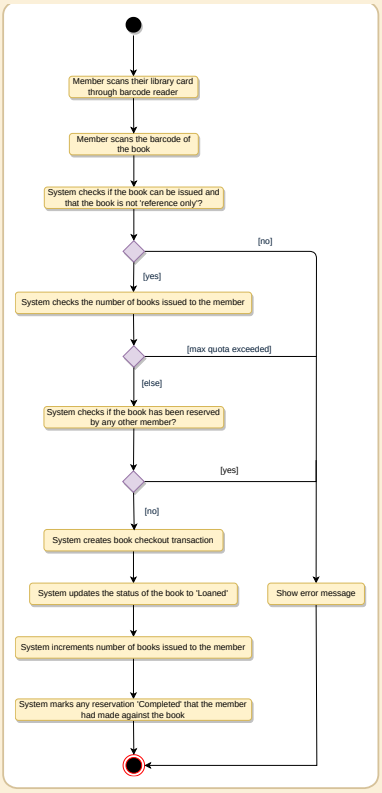


Figure — Check out: scan the book, check it can be issued (not reserved by someone else), check the member's quota, create the checkout transaction, mark the copy "Loaned", increment the member's count, complete any reservation this member had (course diagram)

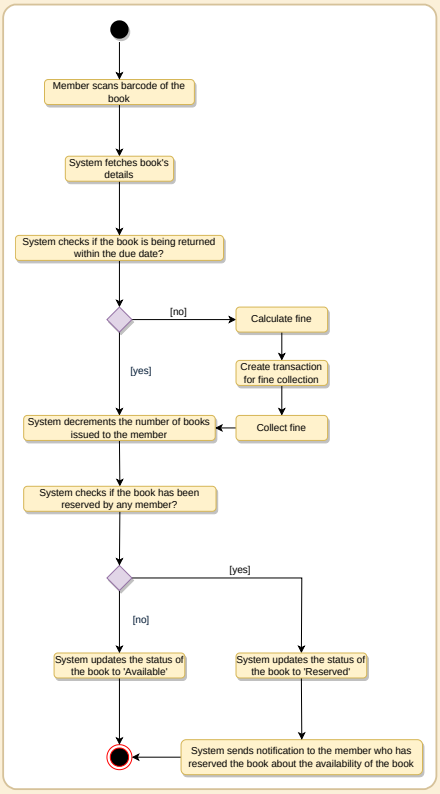


Figure — Return: scan the book, check whether it is late (if so, calculate and collect the fine), decrement the member's count, then check whether the book is reserved — if yes, notify the member who reserved it and mark it reserved, otherwise make it available (course diagram)

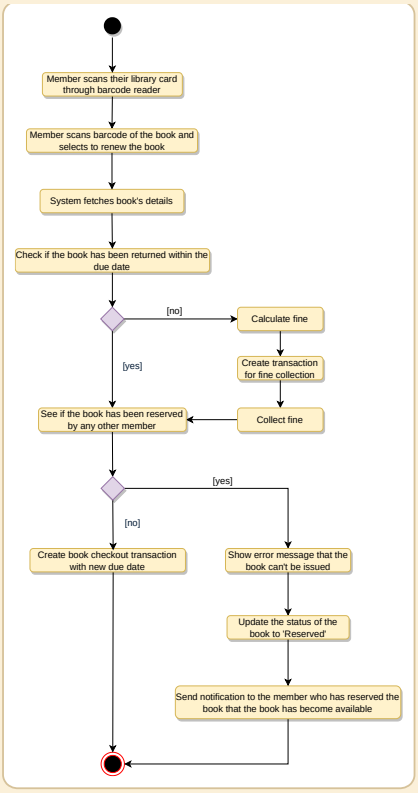
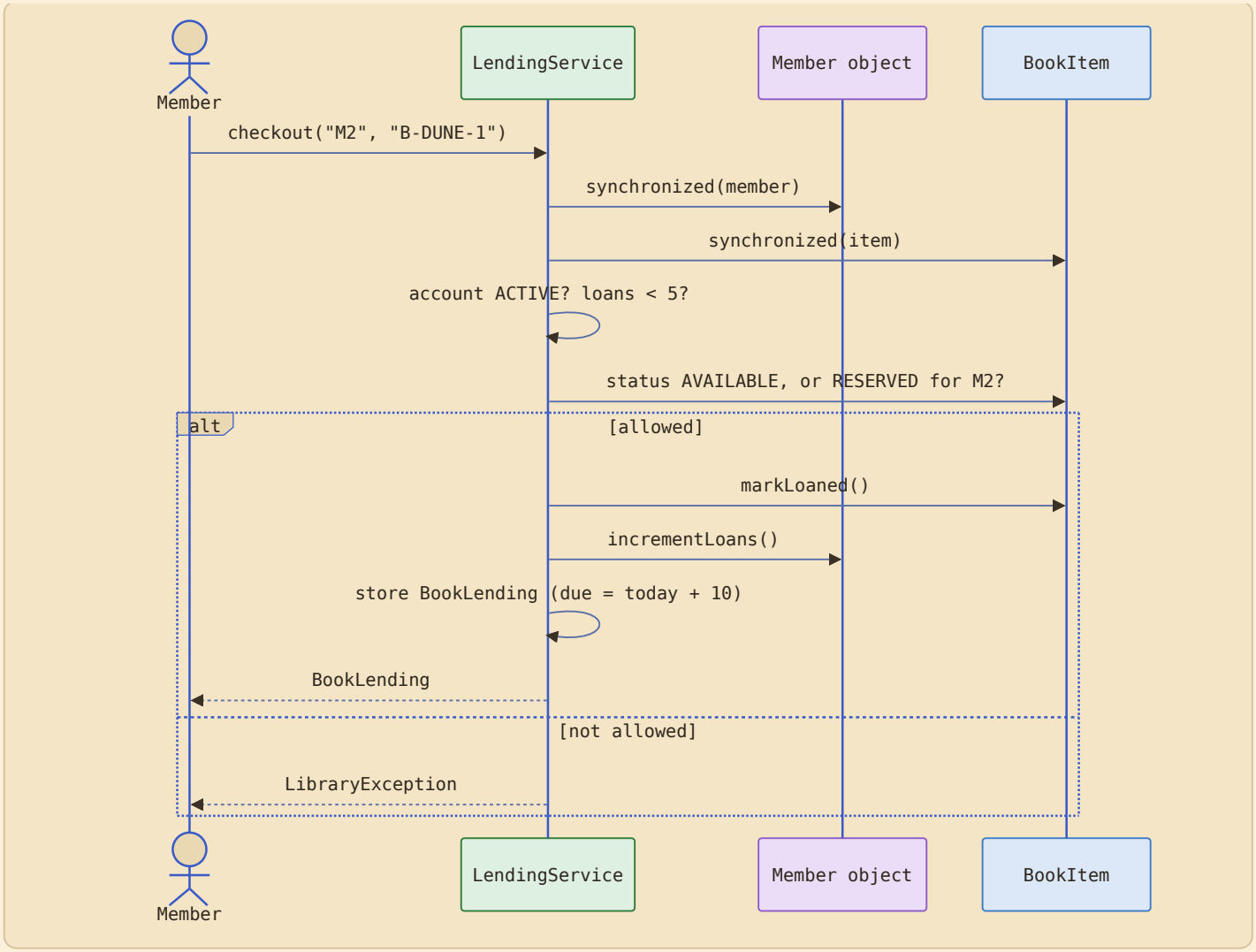
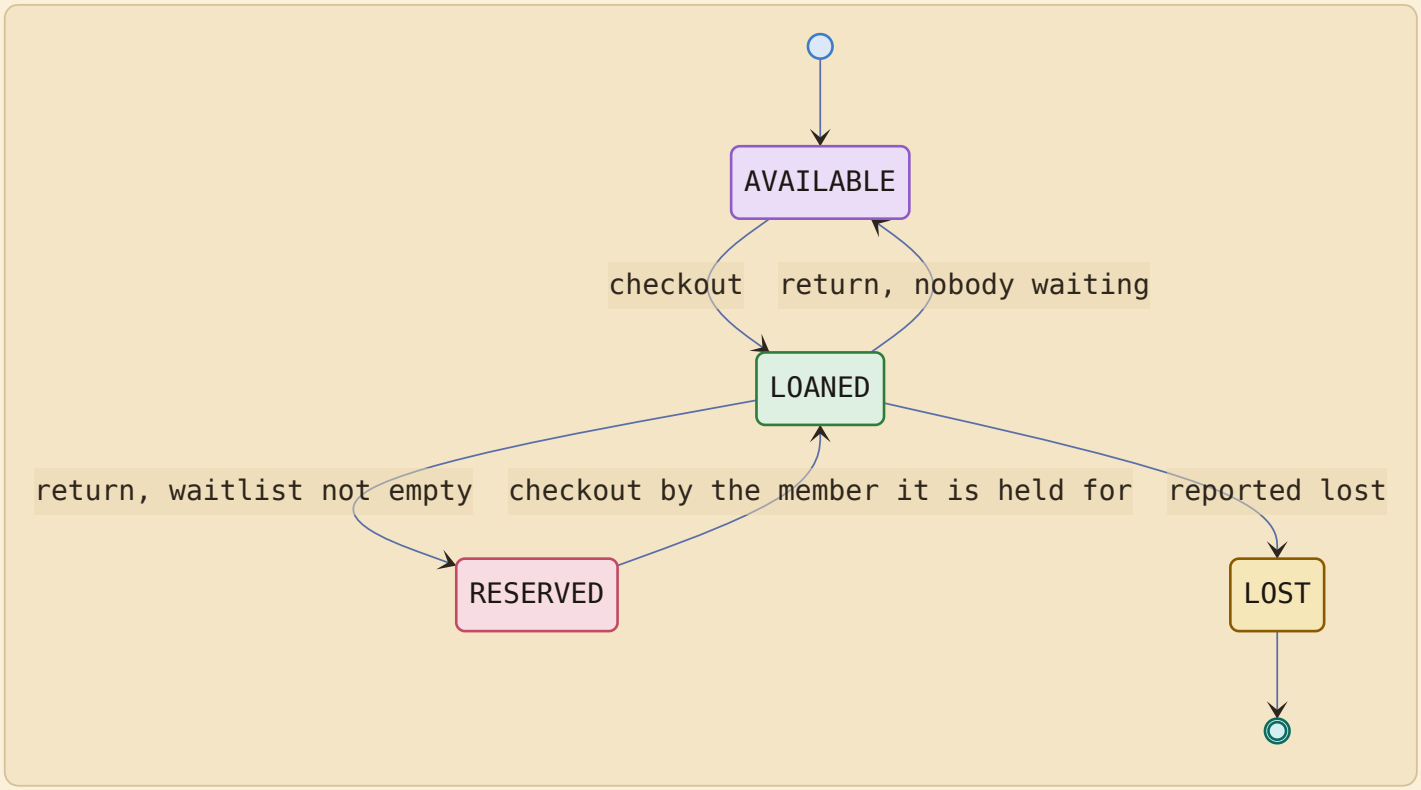


Figure — Renew: scan card and book, collect a fine if it is already late, check whether another member reserved it (if so, refuse and show an error), otherwise create a new checkout transaction with a new due date (course diagram)

The same check-out flow as a sequence diagram (messages between objects in time order, top to bottom) of our code, including the locks:



A book copy's life is a small **state machine** (a fixed set of states plus the moves allowed between them):



Design decisions & patterns used

Decision	Why	Alternative rejected
`Book` vs `BookItem` split	A member borrows a *copy*; search works on *titles*. Every later class depends on this split, so say it early.	One `Book` class with a `copies` counter — cannot say *which* copy is late, on which rack, or who has it.
Composition (`BookItem` has a `Book`) instead of the course's inheritance	Many copies share one immutable title; with inheritance each copy would duplicate every title field, and changing a title's subject would mean editing every copy. "A copy *is a* title" also fails a strict substitution test (the Liskov Substitution Principle: a subclass must work wherever its parent is expected): a title has no barcode or status.	`BookItem extends Book` (the course diagram) — works for a small model, and is worth mentioning as the other valid reading.
Lending rules in `LendingService`, not on `Member`	The course itself flags that a `Member` doing checkout/return/renew has *two reasons to change* (a person, and the lending rules) — a Single Responsibility violation. Moving the flows to a service keeps `Member` a plain person.	Methods on `Member` (course version) — fine for a first sketch.
Reservation = FIFO (first in, first out) waitlist per *title*	A member wants "a copy of Dune", not "copy #3". When *any* copy is returned, the first person in line gets it.	Reservation per copy (course's `BookReservation` keyed by barcode) — a member could wait on copy #3 while copy #1 sits on the shelf.
Strategy for fines (`FinePolicy`)	Fine rules change (caps, student discounts, holidays). The service calls `fine(due, returned)` and does not care how.	`if/else` inside `returnBook`.
Observer-style `Notifier`	(Observer: the sender announces events to whoever subscribed.) Lending code announces events; delivery (email, SMS, push) is someone else's job. The course's abstract `Notification` with one subclass per channel is the same idea.	Sending email directly from `LendingService`.
Facade (`Library`)	Desk software calls one object; role checks (only librarians add stock) live there.	Exposing the catalog and service directly to the UI.
Catalog keeps one index per search	Every lookup is a map read. The course names the cost: each new way of searching adds another map, whereas a single index keyed by (field, value) would stay one structure.	Scanning all books per search.

Java implementation

Package `lld.ch15.library`. Files: `Model.java` (enums, records, entities), `Catalog.java`, `Support.java` (fine policy, notifier, clock, money formatting), `LendingService.java`, `Library.java`, `Main.java`.

```java
// File: lld/ch15/library/Model.java
package lld.ch15.library;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

enum BookFormat { HARDCOVER, PAPERBACK, AUDIO_BOOK, EBOOK, NEWSPAPER, MAGAZINE, JOURNAL }

enum BookStatus { AVAILABLE, RESERVED, LOANED, LOST }

enum AccountStatus { ACTIVE, CLOSED, CANCELED, BLACKLISTED }

record Author(String name) {}

/** Where a copy physically sits. */
record Rack(int number, String location) {}

/** The TITLE: one per ISBN, however many copies the library owns. Immutable. */
record Book(String isbn, String title, String subject, String publisher,
            LocalDate publicationDate, List<Author> authors) {
    Book { authors = List.copyOf(authors); }
}

/** One PHYSICAL COPY of a Book: own barcode, own rack, own status. Guarded by its own monitor. */
final class BookItem {
    private final String barcode;
    private final Book book;
    private final BookFormat format;
    private final Rack rack;
    private final BigDecimal price;          // money: BigDecimal, never double
    private BookStatus status = BookStatus.AVAILABLE;
    private String heldForMemberId;          // set only while status == RESERVED

    BookItem(String barcode, Book book, BookFormat format, Rack rack, BigDecimal price) {
        this.barcode = barcode; this.book = book; this.format = format;
        this.rack = rack; this.price = price;
    }
    String barcode()              { return barcode; }
    Book book()                   { return book; }
    Rack rack()                   { return rack; }
    BookFormat format()           { return format; }
    BigDecimal price()            { return price; }
    synchronized BookStatus status()      { return status; }
    synchronized String heldForMemberId() { return heldForMemberId; }

    // State changes are package-private: only LendingService calls them, while holding this monitor.
    void markLoaned()              { status = BookStatus.LOANED;    heldForMemberId = null; }
    void markAvailable()           { status = BookStatus.AVAILABLE; heldForMemberId = null; }
    void holdFor(String memberId)  { status = BookStatus.RESERVED;  heldForMemberId = memberId; }
    void markLost()                { status = BookStatus.LOST;      heldForMemberId = null; }
}

/** Each account owns exactly one card (composition): the card's barcode identifies the person at the desk. */
record LibraryCard(String cardNumber, String barcode, LocalDate issuedOn) {}

abstract sealed class Account permits Member, Librarian {
    private final String id;
    private final String name;
    private final LibraryCard card;
    private volatile AccountStatus status = AccountStatus.ACTIVE;

    Account(String id, String name, LibraryCard card) { this.id = id; this.name = name; this.card = card; }
    String id()               { return id; }
    String name()             { return name; }
    LibraryCard card()        { return card; }
    AccountStatus status()    { return status; }
    void setStatus(AccountStatus s) { status = s; }
```

```
}

/** A member is a person with a quota. The count is guarded by the member's monitor. */
final class Member extends Account {
    private int booksCheckedOut;
    Member(String id, String name, LibraryCard card) { super(id, name, card); }
    int booksCheckedOut()   { return booksCheckedOut; }
    void incrementLoans()   { booksCheckedOut++; }
    void decrementLoans()   { booksCheckedOut--; }
}

final class Librarian extends Account {
    Librarian(String id, String name, LibraryCard card) { super(id, name, card); }
}

/** One checkout of one copy by one member. Answers "who has this copy?" and "what does this member have?". */
final class BookLending {
    private final String barcode;
    private final String memberId;
    private final LocalDate creationDate;
    private volatile LocalDate dueDate;      // volatile: read by the overdue job without the item lock
    private volatile LocalDate returnDate;

    BookLending(String barcode, String memberId, LocalDate creationDate, LocalDate dueDate) {
        this.barcode = barcode; this.memberId = memberId;
        this.creationDate = creationDate; this.dueDate = dueDate;
    }
    String barcode()        { return barcode; }
    String memberId()       { return memberId; }
    LocalDate creationDate(){ return creationDate; }
    LocalDate dueDate()     { return dueDate; }
    LocalDate returnDate()  { return returnDate; }
    void renewUntil(LocalDate d) { dueDate = d; }
    void close(LocalDate d)      { returnDate = d; }
}

record Fine(String memberId, String barcode, BigDecimal amount, LocalDate date) {}

final class LibraryException extends RuntimeException {
    private static final long serialVersionUID = 1L;
    LibraryException(String msg) { super(msg); }
}
```

```java
// File: lld/ch15/library/Catalog.java
package lld.ch15.library;

import java.time.LocalDate;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

/** The four searches the requirements ask for. */
interface Search {
    List<Book> searchByTitle(String title);
    List<Book> searchByAuthor(String author);
    List<Book> searchBySubject(String subject);
    List<Book> searchByPublicationDate(LocalDate date);
}

/**
 * Keeps one index per search so every lookup is a map read, not a scan.
 * It also knows every copy of every title, which answers "how many copies are free?".
 * Reads vastly outnumber writes in a library, so concurrent maps + copy-on-write lists fit well.
 */
final class Catalog implements Search {
    private final Map<String, List<Book>> byTitle   = new ConcurrentHashMap<>();
    private final Map<String, List<Book>> byAuthor  = new ConcurrentHashMap<>();
    private final Map<String, List<Book>> bySubject = new ConcurrentHashMap<>();
    private final Map<LocalDate, List<Book>> byDate = new ConcurrentHashMap<>();
    private final Map<String, Book> books = new ConcurrentHashMap<>();                  // isbn -> title
    private final Map<String, List<BookItem>> copies = new ConcurrentHashMap<>();        // isbn -> copies
    private final Map<String, BookItem> itemsByBarcode = new ConcurrentHashMap<>();

    void addBook(Book b) {
        if (books.putIfAbsent(b.isbn(), b) != null) return;       // already indexed
        index(byTitle, key(b.title()), b);
        b.authors().forEach(a -> index(byAuthor, key(a.name()), b));
        index(bySubject, key(b.subject()), b);
        byDate.computeIfAbsent(b.publicationDate(), d -> new CopyOnWriteArrayList<>()).add(b);
    }

    void addItem(BookItem item) {
        addBook(item.book());
        if (itemsByBarcode.putIfAbsent(item.barcode(), item) != null)
            throw new LibraryException("duplicate barcode " + item.barcode());
        copies.computeIfAbsent(item.book().isbn(), k -> new CopyOnWriteArrayList<>()).add(item);
    }

    /** Take a copy out of stock (weeded, damaged beyond repair). A copy that is out on loan or held cannot go. */
    void removeItem(String barcode) {
        BookItem item = item(barcode);
        synchronized (item) {
            if (item.status() == BookStatus.LOANED || item.status() == BookStatus.RESERVED)
                throw new LibraryException(barcode + " is " + item.status() + ", cannot remove it now");
            itemsByBarcode.remove(barcode);
            copies.getOrDefault(item.book().isbn(), List.of()).remove(item);
        }
    }

    BookItem item(String barcode) {
        BookItem i = itemsByBarcode.get(barcode);
        if (i == null) throw new LibraryException("unknown barcode " + barcode);
        return i;
    }

    List<BookItem> copiesOf(String isbn) { return copies.getOrDefault(isbn, List.of()); }

    long availableCopies(String isbn) {
```

```
        return copiesOf(isbn).stream().filter(c -> c.status() == BookStatus.AVAILABLE).count();
    }

    @Override public List<Book> searchByTitle(String t)     { return find(byTitle, key(t)); }
    @Override public List<Book> searchByAuthor(String a)    { return find(byAuthor, key(a)); }
    @Override public List<Book> searchBySubject(String s)   { return find(bySubject, key(s)); }
    @Override public List<Book> searchByPublicationDate(LocalDate d) { return List.copyOf(byDate.getOrDefault(d,
List.of())); }

    private static String key(String s) { return s.trim().toLowerCase(Locale.ROOT); }

    private static void index(Map<String, List<Book>> idx, String k, Book b) {
        idx.computeIfAbsent(k, x -> new CopyOnWriteArrayList<>()).add(b);
    }

    private static List<Book> find(Map<String, List<Book>> idx, String k) {
        return idx.getOrDefault(k, List.of()).stream().sorted(Comparator.comparing(Book::title)).toList();
    }
}
```

```java
// File: lld/ch15/library/Support.java
package lld.ch15.library;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.time.temporal.ChronoUnit;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

/** Strategy: how much a late return costs. Swappable without touching LendingService. */
interface FinePolicy {
    BigDecimal fine(LocalDate dueDate, LocalDate returnDate);
}

/** Flat amount per day late (the classic rule: $0.50 a day). */
final class PerDayFine implements FinePolicy {
    private final BigDecimal perDay;
    PerDayFine(BigDecimal perDay) { this.perDay = perDay; }
    @Override public BigDecimal fine(LocalDate due, LocalDate returned) {
        long daysLate = Math.max(0, ChronoUnit.DAYS.between(due, returned));
        return perDay.multiply(BigDecimal.valueOf(daysLate));
    }
}

/** Observer-style outlet for "your reserved book is back" and "your book is overdue". */
interface Notifier {
    void send(String memberId, String message);
}

final class ConsoleNotifier implements Notifier {
    private final List<String> sent = new CopyOnWriteArrayList<>();
    @Override public void send(String memberId, String message) {
        String line = "  [notify " + memberId + "] " + message;
        sent.add(line);
        System.out.println(line);
    }
    List<String> sent() { return sent; }
}

/** A clock we can move forward in tests and demos, so "10 days later" needs no sleeping. */
final class SimClock extends Clock {
    private volatile Instant now;
    SimClock(Instant start) { this.now = start; }
    void advance(Duration d) { now = now.plus(d); }
    @Override public ZoneId getZone() { return ZoneOffset.UTC; }
    @Override public Clock withZone(ZoneId zone) { return this; }
    @Override public Instant instant() { return now; }
}

final class Money {
    private Money() {}
    /** Money is BigDecimal (exact decimals); shown with exactly two decimal places. */
    static String fmt(BigDecimal amount) { return "$" + amount.setScale(2,
RoundingMode.UNNECESSARY).toPlainString(); }
}
```

```java
// File: lld/ch15/library/LendingService.java
package lld.ch15.library;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.LocalDate;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/**
 * The three core flows (check out, return, renew) plus reservations.
 * Lives apart from Member so a Member stays "a person", and the rules live in one place.
 *
 * Locking rule (prevents deadlock): always lock the MEMBER first, then the BOOK ITEM.
 */
public final class LendingService {
    static final int MAX_BOOKS_PER_MEMBER = 5;
    static final int MAX_LENDING_DAYS = 10;

    private final Catalog catalog;
    private final Clock clock;
    private final FinePolicy finePolicy;
    private final Notifier notifier;
    private final Map<String, Member> members = new ConcurrentHashMap<>();
    private final Map<String, BookLending> activeLoans = new ConcurrentHashMap<>();    // barcode -> loan
    private final Map<String, Deque<String>> waitlists = new ConcurrentHashMap<>();    // isbn -> member ids (FIFO)
    private final List<Fine> fines = Collections.synchronizedList(new ArrayList<>());

    LendingService(Catalog catalog, Clock clock, FinePolicy finePolicy, Notifier notifier) {
        this.catalog = catalog; this.clock = clock; this.finePolicy = finePolicy; this.notifier = notifier;
    }

    void register(Member m) { members.put(m.id(), m); }

    /** Cancel a membership. Refused while the member still has books out; drops them from every waitlist. */
    void cancelMembership(String memberId) {
        Member m = member(memberId);
        synchronized (m) {
            if (m.booksCheckedOut() > 0)
                throw new LibraryException(memberId + " must return " + m.booksCheckedOut() + " book(s) first");
            m.setStatus(AccountStatus.CANCELED);
        }
        for (Deque<String> q : waitlists.values()) synchronized (q) { q.remove(memberId); }
    }

    private LocalDate today() { return LocalDate.now(clock); }

    private Member member(String id) {
        Member m = members.get(id);
        if (m == null) throw new LibraryException("unknown member " + id);
        return m;
    }

    // ---------------------------------------------------------------- check out
    BookLending checkout(String memberId, String barcode) {
        Member m = member(memberId);
        BookItem item = catalog.item(barcode);
        synchronized (m) {
            synchronized (item) {
                if (m.status() != AccountStatus.ACTIVE)
                    throw new LibraryException(memberId + " account is " + m.status());
                if (m.booksCheckedOut() >= MAX_BOOKS_PER_MEMBER)
                    throw new LibraryException(memberId + " already has " + MAX_BOOKS_PER_MEMBER + " books");
```

```
            switch (item.status()) {
                case AVAILABLE -> { /* ok */ }
                case RESERVED -> {
                    if (!memberId.equals(item.heldForMemberId()))
                        throw new LibraryException(barcode + " is held for another member");
                }
                case LOANED, LOST -> throw new LibraryException(barcode + " is " + item.status());
            }
            item.markLoaned();
            m.incrementLoans();
            LocalDate today = today();
            BookLending loan = new BookLending(barcode, memberId, today, today.plusDays(MAX_LENDING_DAYS));
            activeLoans.put(barcode, loan);
            removeFromWaitlist(item.book().isbn(), memberId);   // their reservation is now fulfilled
            return loan;
        }
    }
}

// ---------------------------------------------------------------- return
BigDecimal returnBook(String barcode) {
    BookLending loan = activeLoans.get(barcode);
    if (loan == null) throw new LibraryException(barcode + " is not checked out");
    Member m = member(loan.memberId());
    BookItem item = catalog.item(barcode);
    synchronized (m) {
        synchronized (item) {
            if (activeLoans.get(barcode) != loan)               // someone returned it meanwhile
                throw new LibraryException(barcode + " is not checked out");
            LocalDate today = today();
            loan.close(today);
            activeLoans.remove(barcode);
            m.decrementLoans();
            BigDecimal fine = chargeFineIfLate(loan, today);
            handToNextInLine(item);
            return fine;
        }
    }
}

// ---------------------------------------------------------------- renew
BookLending renew(String memberId, String barcode) {
    Member m = member(memberId);
    BookItem item = catalog.item(barcode);
    synchronized (m) {
        synchronized (item) {
            BookLending loan = activeLoans.get(barcode);
            if (loan == null || !loan.memberId().equals(memberId))
                throw new LibraryException(memberId + " does not have " + barcode);
            if (waitingCount(item.book().isbn()) > 0)
                throw new LibraryException("cannot renew " + barcode + ": another member reserved this title");
            LocalDate today = today();
            chargeFineIfLate(loan, today);                      // a late renewal still pays the fine
            loan.renewUntil(today.plusDays(MAX_LENDING_DAYS));
            return loan;
        }
    }
}

// ---------------------------------------------------------------- reserve
/** Join the FIFO waitlist for a TITLE that has no free copy. Returns the 1-based position. */
int reserve(String memberId, String isbn) {
    Member m = member(memberId);
    if (m.status() != AccountStatus.ACTIVE) throw new LibraryException(memberId + " account is " + m.status());
    if (catalog.copiesOf(isbn).isEmpty()) throw new LibraryException("unknown isbn " + isbn);
    if (catalog.availableCopies(isbn) > 0)
        throw new LibraryException("a copy is on the shelf, just check it out");
    Deque<String> q = waitlist(isbn);
    synchronized (q) {
```

```
            if (!q.contains(memberId)) q.addLast(memberId);
            return new ArrayList<>(q).indexOf(memberId) + 1;
        }
    }

    // ------------------------------------------------------------------ system jobs & queries
    void sendOverdueNotices() {
        LocalDate today = today();
        activeLoans.values().stream()
                .filter(l -> l.dueDate().isBefore(today))
                .sorted((a, b) -> a.barcode().compareTo(b.barcode()))
                .forEach(l -> notifier.send(l.memberId(), "'" + catalog.item(l.barcode()).book().title()
                        + "' (" + l.barcode() + ") was due " + l.dueDate()));
    }

    String whoHas(String barcode) {
        BookLending l = activeLoans.get(barcode);
        return l == null ? null : l.memberId();
    }

    List<String> loansOf(String memberId) {
        return activeLoans.values().stream().filter(l -> l.memberId().equals(memberId))
                .map(BookLending::barcode).sorted().toList();
    }

    List<Fine> fines() { synchronized (fines) { return List.copyOf(fines); } }

    // ------------------------------------------------------------------ helpers
    private BigDecimal chargeFineIfLate(BookLending loan, LocalDate today) {
        BigDecimal amount = finePolicy.fine(loan.dueDate(), today);
        if (amount.signum() > 0) {
            fines.add(new Fine(loan.memberId(), loan.barcode(), amount, today));
            notifier.send(loan.memberId(), "fine of " + Money.fmt(amount) + " for late " + loan.barcode());
        }
        return amount;
    }

    /** Caller holds the item's monitor. Give the copy to the first waiting member, or shelve it. */
    private void handToNextInLine(BookItem item) {
        Deque<String> q = waitlist(item.book().isbn());
        String next;
        synchronized (q) { next = q.pollFirst(); }
        if (next == null) { item.markAvailable(); return; }
        item.holdFor(next);
        notifier.send(next, "'" + item.book().title() + "' copy " + item.barcode() + " is held for you");
    }

    private Deque<String> waitlist(String isbn) {
        return waitlists.computeIfAbsent(isbn, k -> new ArrayDeque<>());
    }

    int waitingCount(String isbn) {
        Deque<String> q = waitlist(isbn);
        synchronized (q) { return q.size(); }
    }

    private void removeFromWaitlist(String isbn, String memberId) {
        Deque<String> q = waitlist(isbn);
        synchronized (q) { q.remove(memberId); }
    }
}
```

```java
// File: lld/ch15/library/Library.java
package lld.ch15.library;

import java.time.Clock;

/** Facade: the one object the desk software talks to. Owns the catalog and the lending rules. */
final class Library {
    private final String name;
    private final String address;
    private final Catalog catalog = new Catalog();
    private final LendingService lending;

    Library(String name, String address, Clock clock, FinePolicy finePolicy, Notifier notifier) {
        this.name = name; this.address = address;
        this.lending = new LendingService(catalog, clock, finePolicy, notifier);
    }

    /** Only a librarian may add stock (the use-case diagram's "Add book item"). */
    void addBookItem(Account by, BookItem item) {
        if (!(by instanceof Librarian)) throw new LibraryException(by.id() + " is not a librarian");
        catalog.addItem(item);
    }

    void removeBookItem(Account by, String barcode) {
        if (!(by instanceof Librarian)) throw new LibraryException(by.id() + " is not a librarian");
        catalog.removeItem(barcode);
    }

    void registerMember(Account by, Member m) {
        if (!(by instanceof Librarian)) throw new LibraryException(by.id() + " is not a librarian");
        lending.register(m);
    }

    void cancelMembership(Account by, String memberId) {
        if (!(by instanceof Librarian)) throw new LibraryException(by.id() + " is not a librarian");
        lending.cancelMembership(memberId);
    }

    String name()            { return name; }
    String address()         { return address; }
    Catalog catalog()        { return catalog; }
    LendingService lending() { return lending; }
}
```

```java
// File: lld/ch15/library/Main.java
package lld.ch15.library;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        SimClock clock = new SimClock(Instant.parse("2026-03-01T10:00:00Z"));
        ConsoleNotifier notifier = new ConsoleNotifier();
        Library lib = new Library("City Library", "1 Main Street", clock,
                new PerDayFine(new BigDecimal("0.50")), notifier);
        LendingService desk = lib.lending();
        LocalDate today = LocalDate.of(2026, 3, 1);

        Librarian lucy = new Librarian("L1", "Lucy", new LibraryCard("C-L1", "BC-L1", today));
        Book dune = new Book("978-0441013593", "Dune", "Science Fiction", "Ace",
                LocalDate.of(1965, 8, 1), List.of(new Author("Frank Herbert")));
        Book hobbit = new Book("978-0547928227", "The Hobbit", "Fantasy", "Mariner",
                LocalDate.of(1937, 9, 21), List.of(new Author("J.R.R. Tolkien")));
        Book lotr = new Book("978-0544003415", "The Lord of the Rings", "Fantasy", "Mariner",
                LocalDate.of(1954, 7, 29), List.of(new Author("J.R.R. Tolkien")));
        Rack r1 = new Rack(1, "Aisle A");
        lib.addBookItem(lucy, new BookItem("B-DUNE-1", dune, BookFormat.PAPERBACK, r1, new BigDecimal("9.99")));
        for (int i = 1; i <= 6; i++)
            lib.addBookItem(lucy, new BookItem("B-HOB-" + i, hobbit, BookFormat.HARDCOVER, r1, new
BigDecimal("14.99")));
        lib.addBookItem(lucy, new BookItem("B-LOTR-1", lotr, BookFormat.HARDCOVER, r1, new BigDecimal("29.99")));

        for (int i = 1; i <= 9; i++)
            lib.registerMember(lucy, new Member("M" + i, "Member " + i, new LibraryCard("C-M" + i, "BC-M" + i,
today)));

        System.out.println("== 1. Search the catalog ==");
        System.out.println("  author 'j.r.r. tolkien' -> " + titles(lib.catalog().searchByAuthor("j.r.r.
tolkien")));
        System.out.println("  subject 'Fantasy'       -> " + titles(lib.catalog().searchBySubject("Fantasy")));
        System.out.println("  published 1965-08-01    -> " +
titles(lib.catalog().searchByPublicationDate(LocalDate.of(1965, 8, 1))));
        System.out.println("  free copies of Hobbit   -> " + lib.catalog().availableCopies(hobbit.isbn()));

        System.out.println("\n== 2. Checkout limit (max 5 books per member) ==");
        for (int i = 1; i <= 6; i++) {
            try {
                BookLending l = desk.checkout("M1", "B-HOB-" + i);
                System.out.println("  M1 took " + l.barcode() + ", due " + l.dueDate());
            } catch (LibraryException e) {
                System.out.println("  REFUSED: " + e.getMessage());
            }
        }
        System.out.println("  M1 has " + desk.loansOf("M1"));

        System.out.println("\n== 3. 8 members race for the ONLY copy of Dune ==");
        AtomicInteger winners = new AtomicInteger(), losers = new AtomicInteger();
        CountDownLatch start = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            for (int i = 2; i <= 9; i++) {
                String id = "M" + i;
                pool.submit(() -> {
                    start.await();
                    try { desk.checkout(id, "B-DUNE-1"); winners.incrementAndGet(); }
```

```
                    catch (LibraryException e) { losers.incrementAndGet(); }
                    return null;
                });
            }
            start.countDown();                    // release all threads at once
        }                                         // close() waits for every task
        String holder = desk.whoHas("B-DUNE-1");
        System.out.println("  winners=" + winners + " losers=" + losers + " (exactly one copy was lent)");
        System.out.println("  holder = " + holder + "  (which member wins varies from run to run)");

        System.out.println("\n== 4. Reserve a title with no free copy (FIFO waitlist) ==");
        String first = holder.equals("M2") ? "M3" : "M2";
        String second = holder.equals("M4") ? "M5" : "M4";
        System.out.println("  " + first + " position " + desk.reserve(first, dune.isbn()));
        System.out.println("  " + second + " position " + desk.reserve(second, dune.isbn()));
        try { desk.reserve("M6", hobbit.isbn()); }
        catch (LibraryException e) { System.out.println("  M6 reserve Hobbit REFUSED: " + e.getMessage()); }

        System.out.println("\n== 5. Renew is blocked while someone waits ==");
        try { desk.renew(holder, "B-DUNE-1"); }
        catch (LibraryException e) { System.out.println("  REFUSED: " + e.getMessage()); }

        System.out.println("\n== 6. 13 days later: overdue notices, late return, fine, hand-over ==");
        clock.advance(Duration.ofDays(13));
        desk.sendOverdueNotices();
        BigDecimal fine = desk.returnBook("B-DUNE-1");
        System.out.println("  fine charged = " + Money.fmt(fine) + " (3 days late x $0.50)");
        try { desk.checkout(second, "B-DUNE-1"); }
        catch (LibraryException e) { System.out.println("  " + second + " REFUSED: " + e.getMessage()); }
        System.out.println("  " + first + " checks out held copy, due " + desk.checkout(first, "B-DUNE-
1").dueDate());
        System.out.println("  who has B-DUNE-1? " + desk.whoHas("B-DUNE-1"));

        System.out.println("\n== 7. Renew on time with nobody waiting ==");
        desk.returnBook("B-HOB-1");                    // 13 days: late, fine charged
        desk.checkout("M6", "B-HOB-1");
        clock.advance(Duration.ofDays(4));
        System.out.println("  M6 renewed B-HOB-1, new due " + desk.renew("M6", "B-HOB-1").dueDate());
        System.out.println("  fines on record: " + desk.fines().size());

        System.out.println("\n== 8. Librarian housekeeping: remove a copy, cancel a membership ==");
        try { lib.removeBookItem(lucy, "B-DUNE-1"); }
        catch (LibraryException e) { System.out.println("  REFUSED: " + e.getMessage()); }
        lib.removeBookItem(lucy, "B-LOTR-1");
        System.out.println("  copies of LOTR left: " + lib.catalog().copiesOf(lotr.isbn()).size());
        try { lib.cancelMembership(lucy, "M6"); }
        catch (LibraryException e) { System.out.println("  REFUSED: " + e.getMessage()); }
        lib.cancelMembership(lucy, "M8");
        try { desk.checkout("M8", "B-HOB-6"); }
        catch (LibraryException e) { System.out.println("  REFUSED: " + e.getMessage()); }
    }

    private static List<String> titles(List<Book> books) { return books.stream().map(Book::title).toList(); }
}
```

Output:

```
== 1. Search the catalog ==
  author 'j.r.r. tolkien' -> [The Hobbit, The Lord of the Rings]
  subject 'Fantasy'       -> [The Hobbit, The Lord of the Rings]
  published 1965-08-01    -> [Dune]
  free copies of Hobbit   -> 6

== 2. Checkout limit (max 5 books per member) ==
  M1 took B-HOB-1, due 2026-03-11
  M1 took B-HOB-2, due 2026-03-11
  M1 took B-HOB-3, due 2026-03-11
  M1 took B-HOB-4, due 2026-03-11
  M1 took B-HOB-5, due 2026-03-11
  REFUSED: M1 already has 5 books
  M1 has [B-HOB-1, B-HOB-2, B-HOB-3, B-HOB-4, B-HOB-5]

== 3. 8 members race for the ONLY copy of Dune ==
  winners=1 losers=7 (exactly one copy was lent)
  holder = M9  (which member wins varies from run to run)

== 4. Reserve a title with no free copy (FIFO waitlist) ==
  M2 position 1
  M4 position 2
  M6 reserve Hobbit REFUSED: a copy is on the shelf, just check it out

== 5. Renew is blocked while someone waits ==
  REFUSED: cannot renew B-DUNE-1: another member reserved this title

== 6. 13 days later: overdue notices, late return, fine, hand-over ==
  [notify M9] 'Dune' (B-DUNE-1) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-1) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-2) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-3) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-4) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-5) was due 2026-03-11
  [notify M9] fine of $1.50 for late B-DUNE-1
  [notify M2] 'Dune' copy B-DUNE-1 is held for you
  fine charged = $1.50 (3 days late x $0.50)
  M4 REFUSED: B-DUNE-1 is held for another member
  M2 checks out held copy, due 2026-03-24
  who has B-DUNE-1? M2

== 7. Renew on time with nobody waiting ==
  [notify M1] fine of $1.50 for late B-HOB-1
  M6 renewed B-HOB-1, new due 2026-03-28
  fines on record: 2

== 8. Librarian housekeeping: remove a copy, cancel a membership ==
  REFUSED: B-DUNE-1 is LOANED, cannot remove it now
  copies of LOTR left: 0
  REFUSED: M6 must return 1 book(s) first
  REFUSED: M8 account is CANCELED
```

What to notice in the output: section 2 refuses the sixth book (requirement 5); section 3 lets exactly one of eight concurrent members borrow the only copy; section 5 refuses a renewal because someone is waiting; section 6 sends overdue notices, charges 3 days × $0.50, and hands the returned copy to the first person in the waitlist — the second person is refused because the copy is *held* for the first; section 8 covers the librarian's "remove book item" and "cancel membership" use cases — a copy that is out on loan cannot be removed, a member with a book still out cannot be cancelled, and a cancelled member can no longer borrow.

Concurrency & edge cases

- **Two members, one last copy.** `checkout` locks the copy (`synchronized (item)`), so "is it available? → mark loaned" is one atomic step. The demo's 8-thread race always has exactly one winner.
- **Two librarians scanning for the same member.** The quota check and increment happen under `synchronized (member)`, so a member can never end up with 6 books.

- **Deadlock avoidance.** (A **deadlock** is two threads each holding a lock the other needs, so both wait forever.) Every method locks **member first, then item** — a fixed global lock order. Two threads can never each hold one lock while waiting for the other. (`returnBook` reads the loan first to find the member, then locks in the same order and re-checks the loan is still active.)
- **Renew vs reserve.** Renewal is refused if the title's waitlist is non-empty. The check happens *before* charging any late fine, so a refused renewal never fines the member twice.
- **A known small window.** `reserve` checks "no free copy" and then joins the queue as two steps. If a copy is returned in between, it goes to the shelf while the member sits first in the queue; the *next* return reaches them. We accept this rather than make `reserve` take the queue lock before the copies' locks, because returns take them in the opposite order (copy, then queue) — mixing the two orders is exactly how deadlocks are born.
- **Held copy.** A returned copy with a waitlist becomes `RESERVED` for one member only; everybody else is refused at checkout.
- **Edge cases handled:** unknown barcode or member, duplicate barcodes, inactive/blacklisted/cancelled accounts, removing a copy that is on loan or held, cancelling a membership while books are still out, reserving a title that has a free copy ("just check it out"), joining the same waitlist twice (keeps your place), returning a book that is not out.
- **Across several servers** the `synchronized` blocks are not enough — the same guarantee must come from the database: e.g. `UPDATE book_item SET status='LOANED' WHERE barcode=? AND status='AVAILABLE'` and check that exactly one row changed (a *conditional update*).

Extensions & follow-up questions

- **"Two members reserve the last copy at the same moment — what happens?"** The course calls this out as unguarded in its own design. In ours, `reserve` appends to a FIFO deque (double-ended queue) under a lock, so both are queued in a definite order and the first one gets the next copy.
- **"Where do you keep the number of available copies of a title?"** Don't store a counter that can drift; derive it: `Catalog.availableCopies(isbn)` counts copies with status `AVAILABLE`. If that becomes slow, cache it (keep a ready-made copy for fast reads) and update it in the same lock as the status change.
- **"What is a `BookLending` for?"** It is the loan record: it answers "who has this copy, since when, due when", drives overdue notices and fines, and is the history of the copy.
- **"How is the fine paid?"** Add a `FinePayment` / `PaymentService` and block checkouts while unpaid fines exceed a threshold — a one-line rule in `checkout`.
- **"Hold expiry?"** A held copy should not wait forever: add `heldUntil` to `BookItem` and a daily job that passes the copy to the next person (or shelves it) when the hold lapses — exactly the TTL idea of section 15.2.
- **"E-books?"** An e-book "copy" is a licence: `BookFormat.EBOOK` items with no rack, and many libraries buy a fixed number of concurrent licences — the same copy model works.

Real-world example

Open-source integrated library systems such as **Koha** and **Evergreen** implement exactly this model: bibliographic *records* (titles) with attached *items* (copies with barcodes and shelf locations), circulation (lending and return) rules per patron (library member) category (loan period, maximum items, fine per day), a holds queue per title, and overdue/hold-available notices by email or SMS. Digital lending apps such as **OverDrive's Libby** apply the same waitlist idea to e-books: a library owns a fixed number of licences, borrowers join a holds queue, and when a licence frees up the next person is notified and given a limited window to borrow before it passes on.

In the interview

30–45 minute plan. (1) 3 min — clarify: copies, limits (5 books, 10 days), fines, reservations, notifications. (2) 5 min — actors and use cases. (3) 10 min — entities, *starting with the `Book` / `BookItem` split*, then accounts, lending records, catalog. (4) 10 min — code `checkout` and `returnBook` with the locking; mention lock ordering. (5) 5 min — the reservation hand-over and fines (Strategy). (6) Remaining — follow-ups: concurrency across servers, fine payment, hold expiry.

> **30-second answer:** "The key split is `Book` (the title) vs `BookItem` (a physical copy with a barcode and rack). Members borrow copies; search works on titles through a `Catalog` with one index per search type. A `LendingService` enforces 5 books and 10 days, records `BookLending`s, charges fines via a `FinePolicy` strategy, and passes returned copies to a FIFO waitlist per title with a notification. Checkout locks member then copy so the last copy can't be double-lent."

Self-check

- **Q:** Why is `Book` separate from `BookItem`? **A:** Search and metadata are per title; status, rack, barcode and loans are per physical copy. One title has many copies.
- **Q:** What guarantees a member can't get a 6th book when two desks scan at once? **A:** The quota check and increment happen inside `synchronized (member)`.
- **Q:** Why a fixed lock order? **A:** Without it, thread A could hold member→wait item while B holds item→wait member: deadlock.
- **Q:** Which pattern lets the library change its fine rules without touching lending code? **A:** Strategy (`FinePolicy`).
- **Q:** When can't a book be renewed? **A:** When another member is waiting for that title.

TL;DR

Title vs copy. Limits on the member, state on the copy, history in `BookLending`. Rules in a `LendingService`; fines as a Strategy; notifications behind an interface. Lock member-then-copy so the last copy is never lent twice. Reservations are a FIFO queue per title.

15.2 Design a Movie Ticket Booking System (BookMyShow)

Figure — Cover image for the movie ticket booking case study (course diagram)

Problem & clarifying questions

An online movie ticket booking system (think **BookMyShow**, **Fandango**, **AMC's app**) lets customers see which movies are playing, pick a show, choose seats on a seat map, pay, and receive a ticket — from anywhere, at any time. It is *the* classic LLD booking question because it has a hierarchy of places (city → cinema → hall → seat), a time dimension (shows), and a genuine race (two people tapping the same seat).

Question	Answer we assume
One city or many?	Many cities with affiliate (partner) cinemas; the app lists cities.
Can a cinema have several halls? Several shows per hall?	Yes and yes — but a hall runs **one show at a time**.
What do customers search by?	Title, language, genre, release date, **city**.
Can a customer pick several seats in one booking?	Yes, from a seat map showing free vs booked seats.
How long is a seat kept while paying?	A few minutes (we use **5**). If payment does not finish, the seats go back.
Payment methods? Discounts?	Credit card or cash (cash at the front desk); optional **discount coupon**.
Refunds?	On cancellation, refund if within the allowed time (we use: up to 2 hours before the show).
Notifications?	New movie, booking made, booking cancelled.
Who else uses it?	Admins (add movies/shows, block customers), front-desk officers (book for walk-ins), guests (search only).

Requirements

Functional (the course's list, in order)

1. List the **cities** where affiliate cinemas are located.
2. Each cinema has many **halls**; each hall runs **one show at a time**.
3. Each movie has many **shows**.
4. **Search** movies by title, language, genre, release date and city.
5. After choosing a movie, show the **cinemas and shows** running it.
6. Pick a show and **book** tickets.
7. Show the **seating plan**; let the customer pick several seats.
8. Distinguish **available** seats from booked ones.
9. **Notify** on a new movie, and on booking or cancellation.
10. Pay by **credit card or cash**.
11. **No two customers can reserve the same seat.**
12. Apply a **discount coupon** to the payment.

Non-functional

- **Strong consistency on seats** (every reader sees the latest confirmed state; see System Design Fundamentals): double booking is the one bug that must never happen.
- **High read, bursty write** (writes arrive in sudden spikes): browsing dwarfs booking, but a blockbuster's first-day shows get many simultaneous seat grabs.
- **Hold expiry**: abandoned checkouts must not block seats (the course lists this as a gap in its own design; we implement it).
- **Failure-safe payments**: a customer must never pay for a seat they don't get.

Use cases

Five actors: **Admin** (add movies and shows, cancel a movie or show, block/unblock customers), **FrontDeskOfficer** (book and cancel tickets for walk-ins), **Customer** (view schedules, book, cancel), **Guest** (search only — must register to book), **System** (notifications).

Top use cases: search movies; create/modify/view booking; make payment; add a coupon to the payment; assign seats (seat map); refund payment on cancellation within the allowed time.

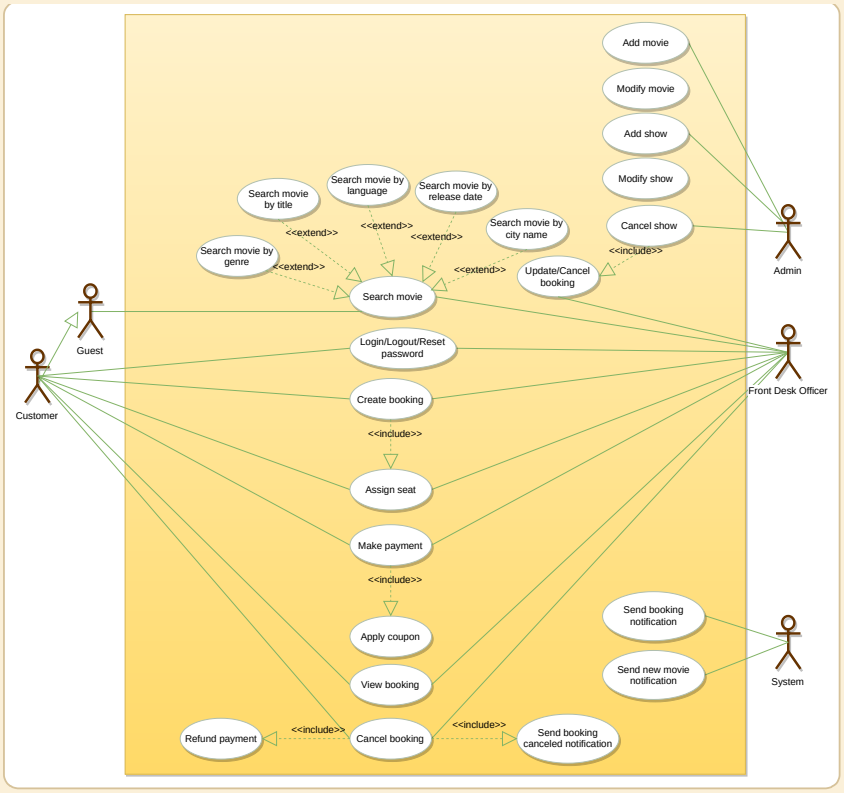


Figure — Movie booking use cases. The hollow arrow from Customer to Guest says a customer can do everything a guest can; the five "search by" use cases extend "Search movie"; "Create booking" includes "Assign seat"; "Make payment" includes "Apply coupon"; "Cancel booking" includes "Refund payment" and the cancellation notification; the admin's "Cancel show" includes updating/cancelling its bookings (course diagram)

Notice in the figure above that a **Customer is a specialised Guest**: everything a guest can do (search), a customer can do too, plus booking. That is the use-case version of the inheritance we enforce in code by giving `BookingService` a `Customer` parameter.

Core entities

Class	Responsibility
`City` (record)	Name and state. Cinemas point to it (association — the city exists without the cinema).
`Cinema`	Name, city; **owns** its halls.
`CinemaHall`	Name, physical seats; refuses an overlapping show.
`HallSeat` (record)	The seat bolted to the floor: id and `SeatType` (`REGULAR`, `PREMIUM`, `ACCESSIBLE`, `EMERGENCY_EXIT`, `OTHER`). **No booking state.**
`Movie` (record)	Title, language, genre, release date, duration.
`Show`	One screening of a movie in a hall at a time; **owns one `ShowSeat` per hall seat**.
`ShowSeat`	A seat *for this show*: price and `booked` flag. **This is what gets booked.**
`SeatLockProvider` / `InMemorySeatLockProvider`	Temporary, expiring **holds** on show seats; all-or-nothing lock, commit, release.
`Booking`	Customer, show, seat ids, status (`PENDING`, `CONFIRMED`, `CHECKED_IN`, `CANCELED`, `ABANDONED`), payment, coupon.
`BookingService`	hold → pay → commit (make final); cancel; seat map.
`Payment`, `PaymentMethod` (`CardPayment`, `CashPayment`), `PaymentGateway`	Money. The gateway (the external service, such as a card processor, that actually moves the money) is the boundary to the real provider.
`Coupon`	Percentage off, applied to the booking amount before payment.
`CancellationPolicy` / `CutoffRefundPolicy`	How much comes back on cancel (Strategy).
`MovieSearch` / `Catalog`	Five searches; "by city" is answered from shows.
`Customer`, `Guest`, `FrontDeskOfficer`, `Admin`	Actors; a `Guest` has no type that `BookingService` accepts.
`Notifier`	Messages to users: new movie (sent by `Catalog.addMovie`), booking confirmed, booking cancelled.

Class diagram

The course grows the model over **six stages**; after each one the system gains a capability it lacked:

Stage	Adds	What works afterwards
1	City, cinema, hall, physical seat	There is a building with seats in it
2	Movie, show, show seat	Every screening carries its own seat list and prices
3	Coupon, payment types, notification types	An amount can be reduced by a coupon and then paid
4	Booking and `BookingService`	One seat can never go to two customers
5	Search interface and catalog	A movie can be looked up in five different ways
6	People and accounts	Booking is restricted to registered customers

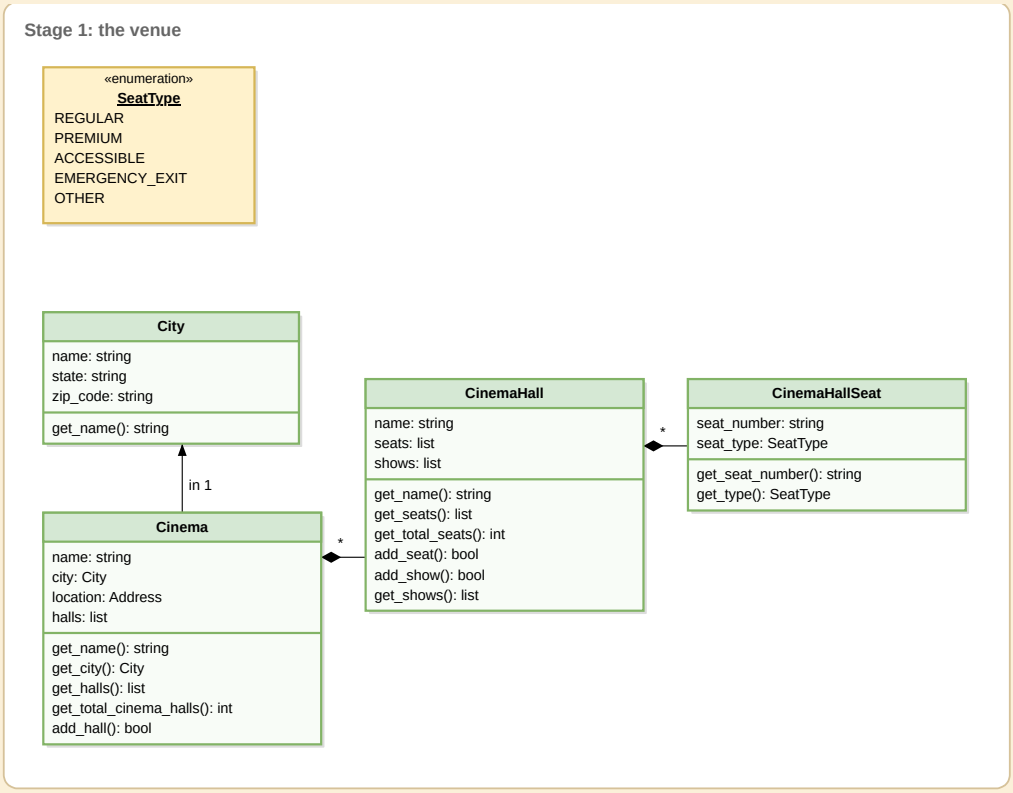


Figure — Stage 1: `City`, `Cinema`, `CinemaHall` and `CinemaHallSeat`; cinema→city is a plain association, cinema→hall and hall→seat are composition (course diagram)

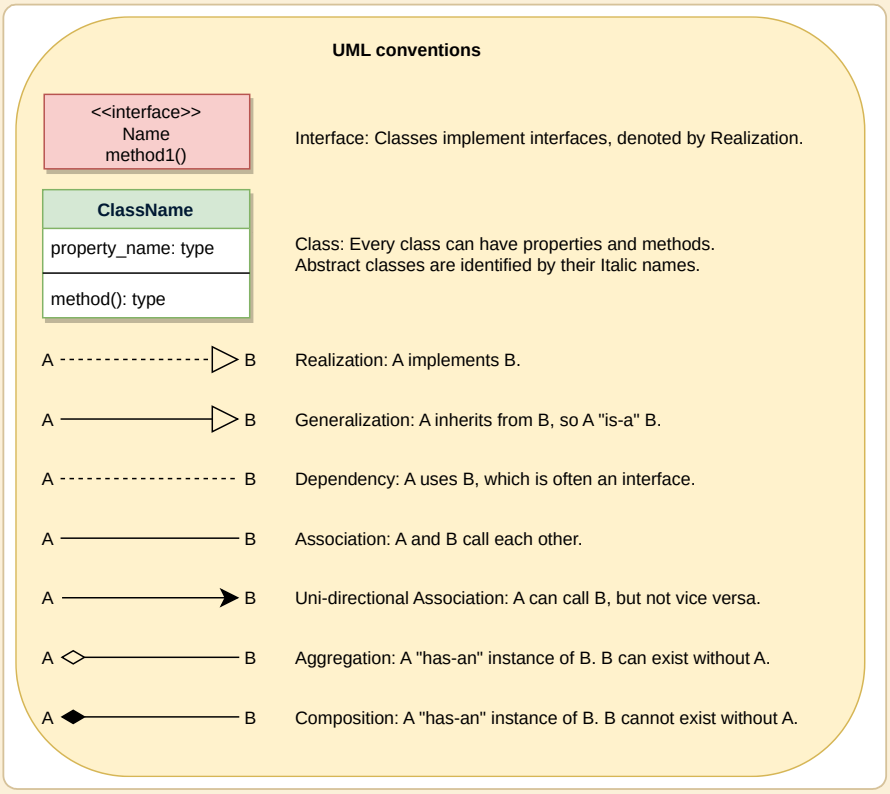


Figure — How to read a class box and the arrow styles used in these diagrams (course diagram)

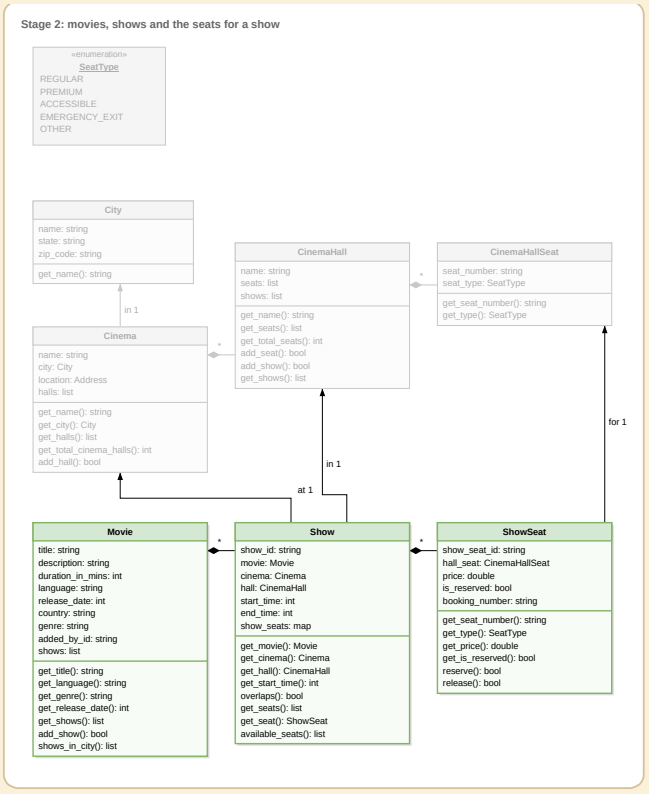


Figure — Stage 2: `Movie` owns its `Show`s; each `Show` owns one `ShowSeat` per physical seat, and each show seat points back at its `CinemaHallSeat` (course diagram)

Stage 2 is **where the design succeeds or fails**. One physical seat can be empty for the 3 pm screening and sold for the 9 pm one, so the thing you reserve cannot be the physical seat. You reserve a `ShowSeat` — one per seat per show — which also lets each show price its seats differently (a premium seat costs more).

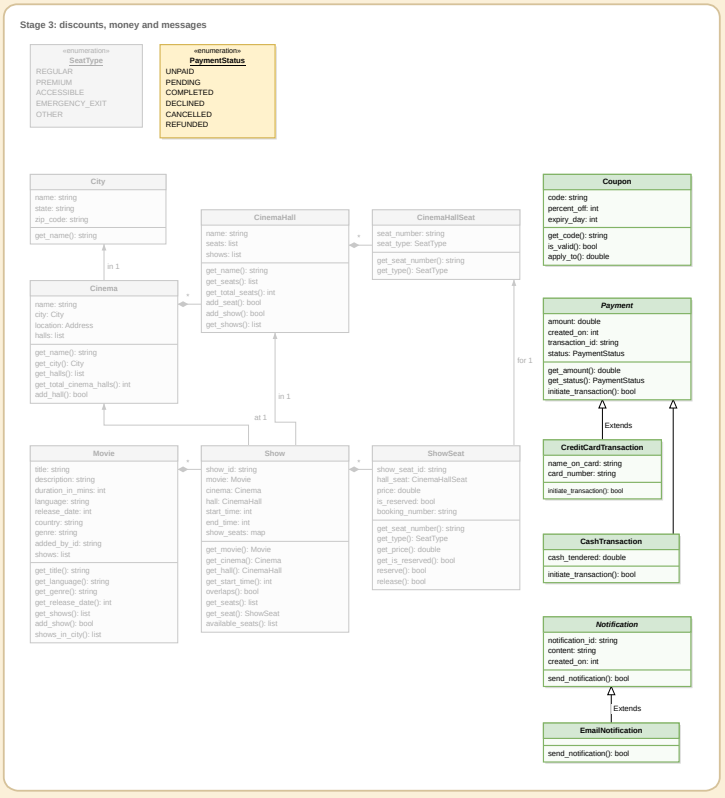


Figure — Stage 3: `Coupon`, abstract `Payment` (credit card and cash subclasses) and abstract `Notification` (one subclass per channel); none of them points at anything yet (course diagram)

The coupon reduces **the booking's amount, not the payment**: by the time any payment object is created the price is final, and no payment type needs its own discount logic.

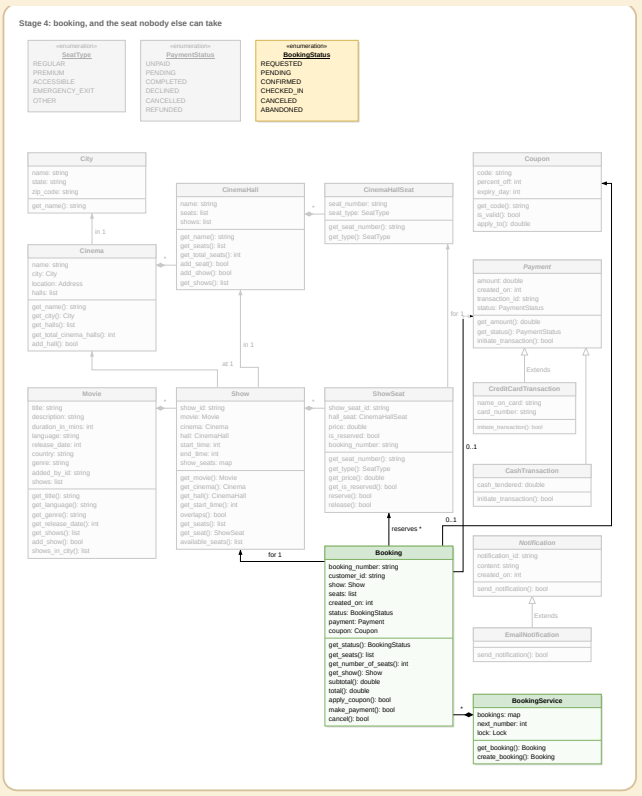


Figure — Stage 4: `Booking` points at the show, seats, payment and coupon without owning them; `BookingService` owns the bookings and is the only class holding a lock (course diagram)

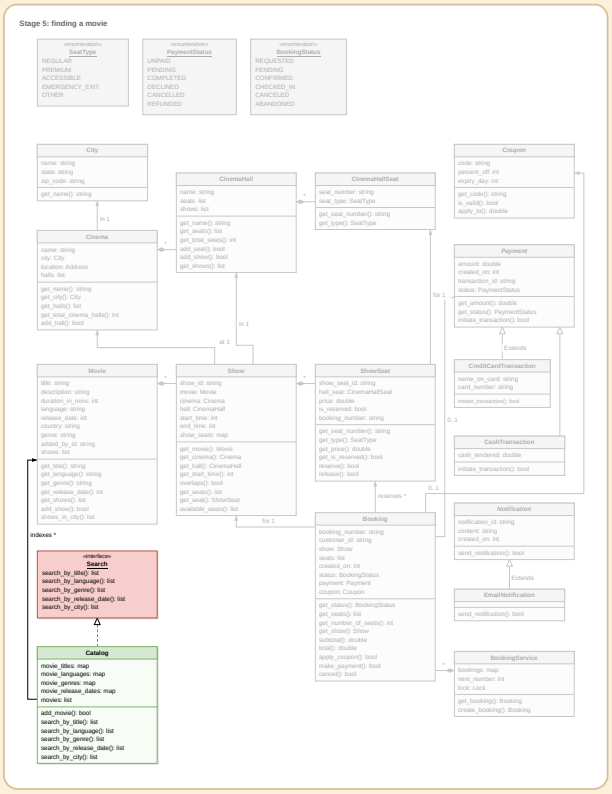


Figure — Stage 5: the `Search` interface with five methods, implemented by `Catalog`; four searches use indexes built when a movie is added, while city search is computed from the scheduled shows (course diagram)

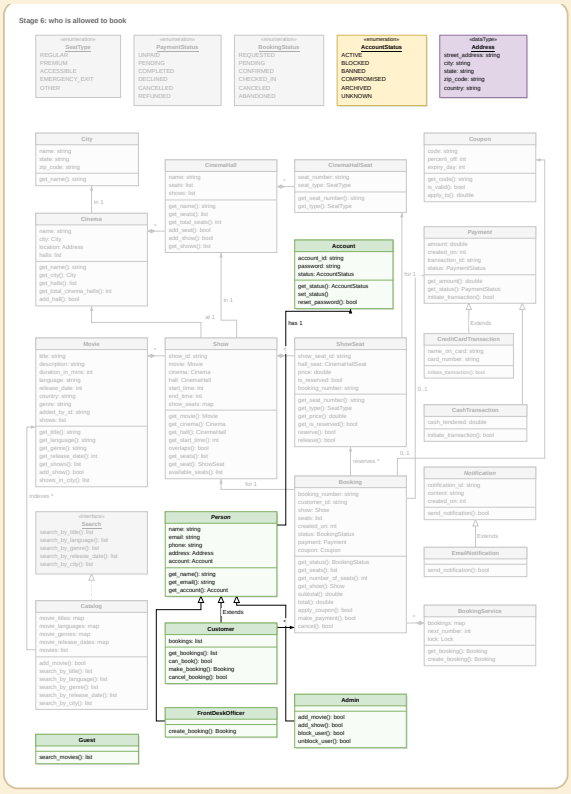


Figure — Stage 6: `Person` with `Customer`, `FrontDeskOfficer` and `Admin`; `Guest` sits outside that tree and has no booking method; a customer keeps the bookings it made (course diagram)

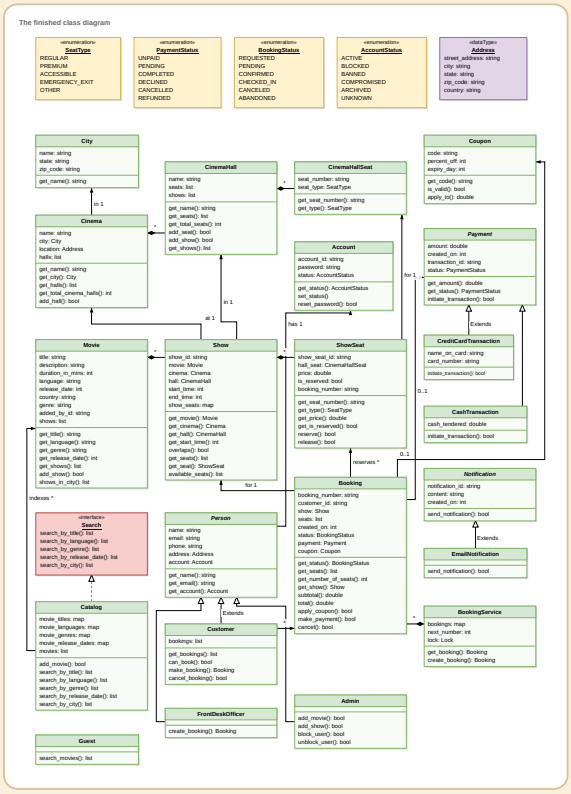
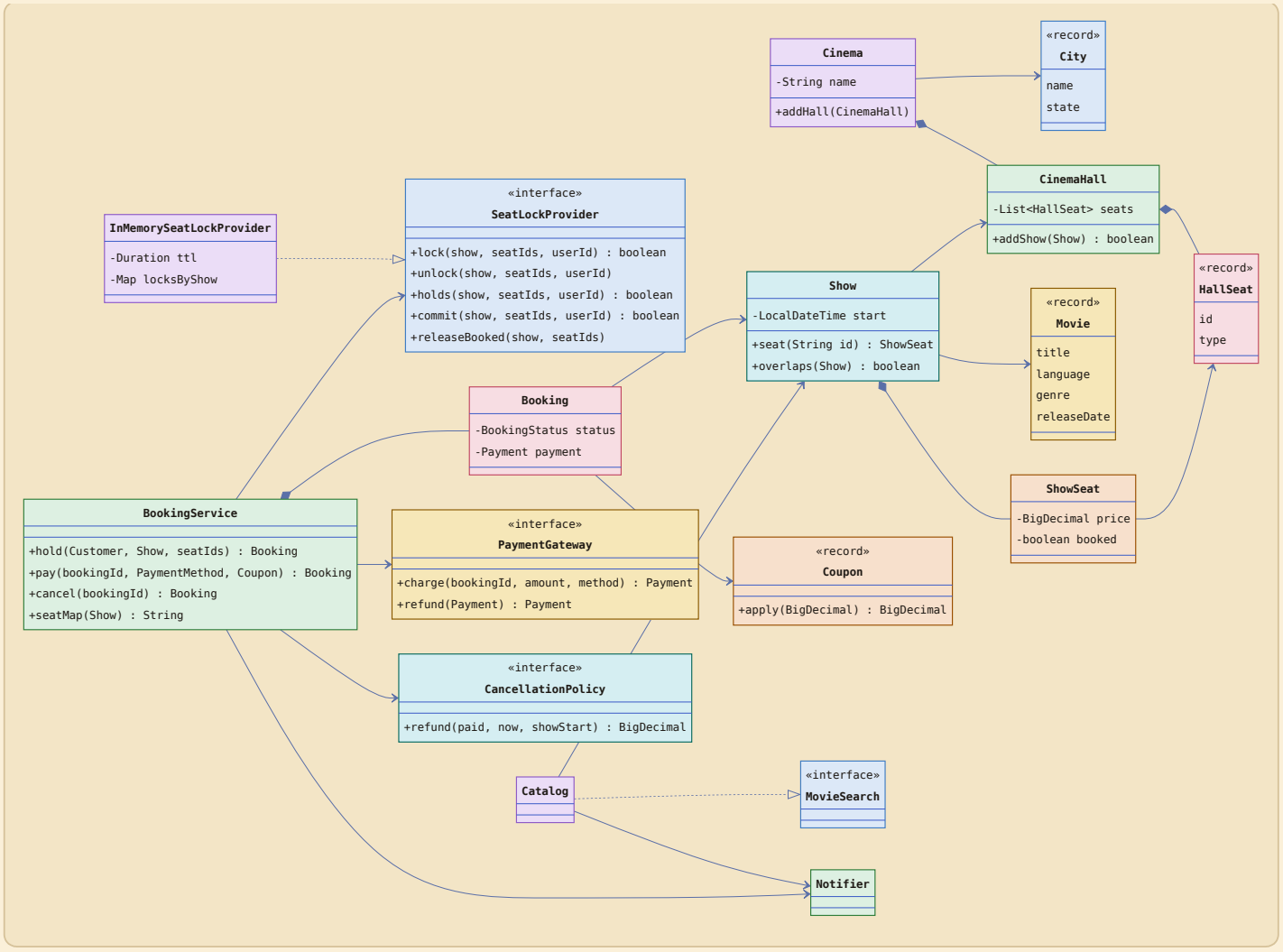


Figure — The finished movie booking class diagram (course diagram)

Our Java design (the main difference from the course: a separate `SeatLockProvider` with **TTL holds**, which the course names as its biggest omission):



Key flows

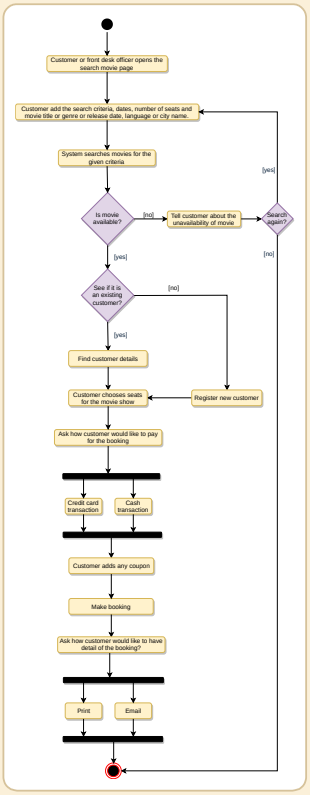


Figure — Make a booking: search movies; if available, find or register the customer, pick seats, ask how they will pay (credit card or cash), and choose how to receive the ticket (course diagram)

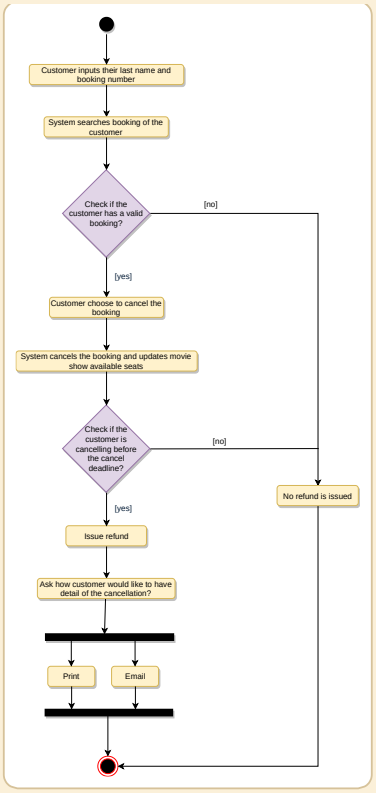
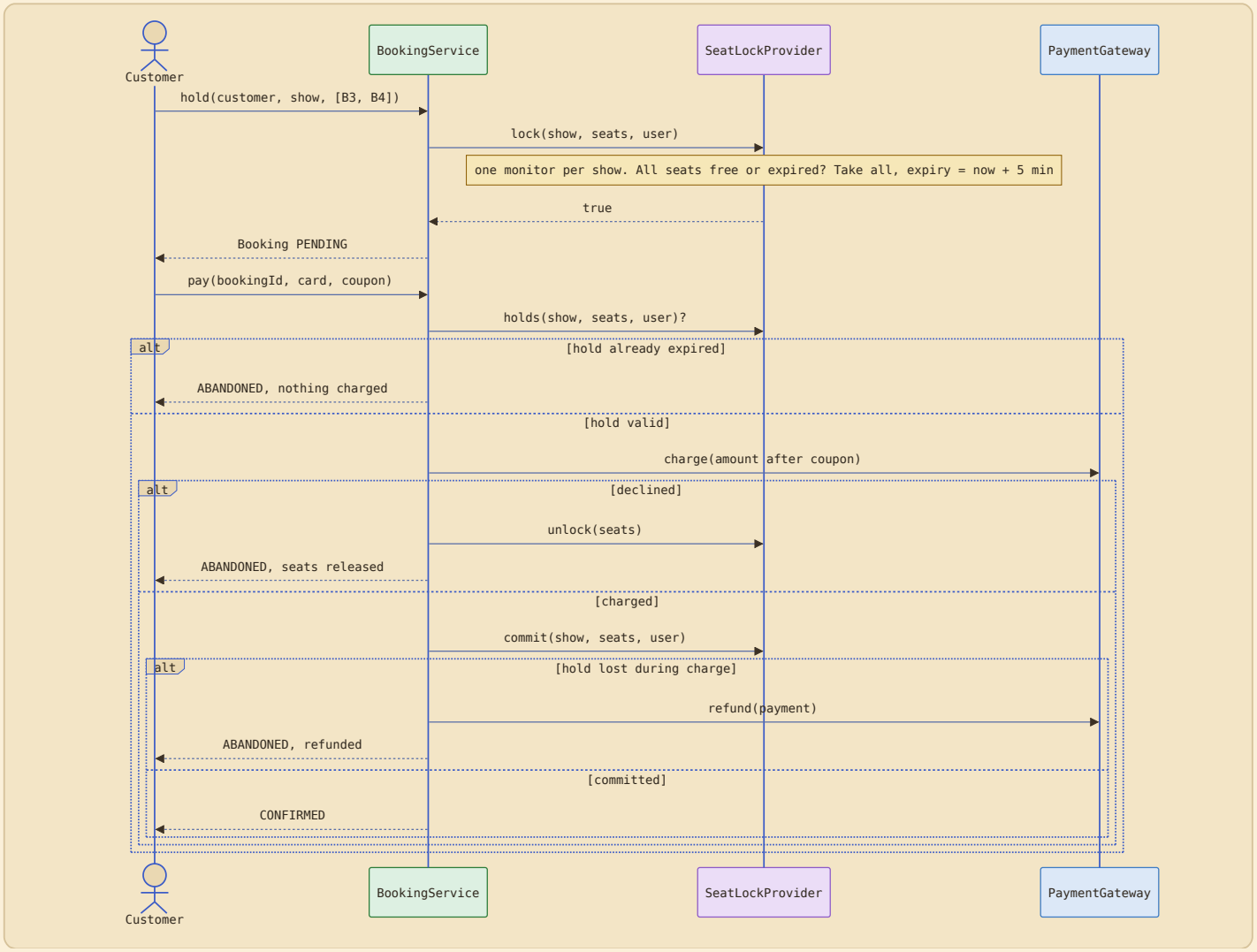


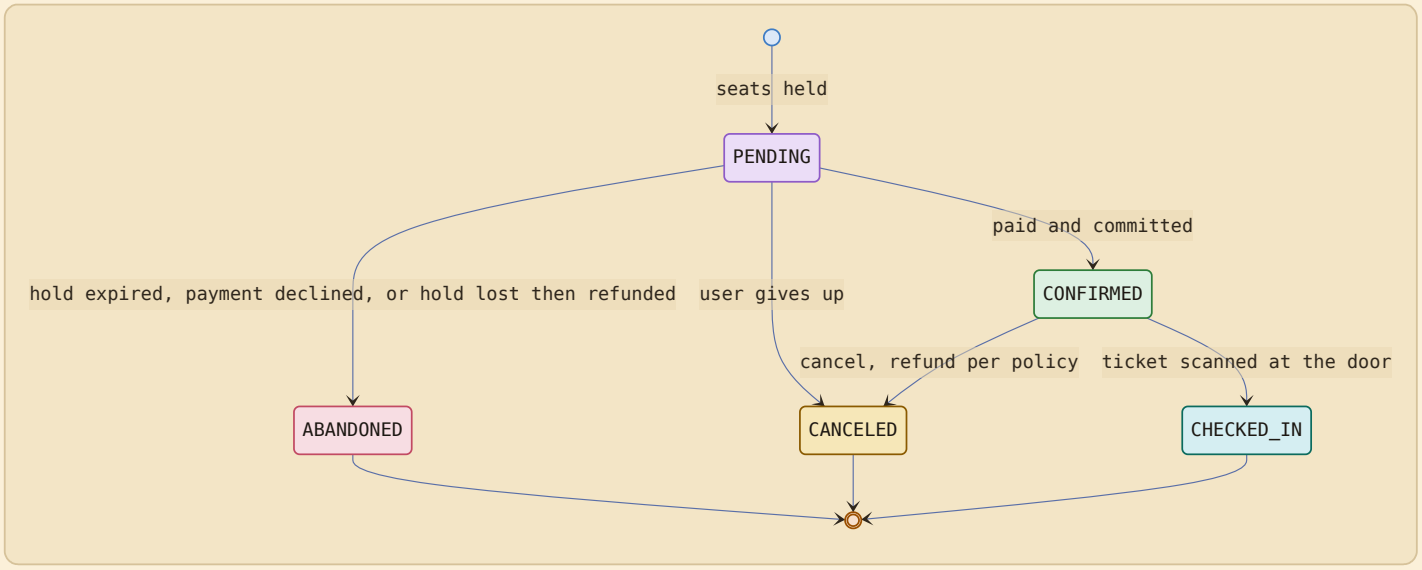
Figure — Cancel a booking: the customer gives last name and booking number; the system finds the booking, issues a refund if the cancellation is within the allowed time (otherwise no refund), cancels, updates the show's available seats, and prints or emails the details (course diagram)

The heart of the design is a **three-step booking path**: *hold → pay → commit*.



Why `commit` must re-check inside the lock: the payment call can take seconds. If the hold expires during it and someone else grabs the seat, a naive "mark booked" would double-book. `commit` verifies "these seats are still held by me and not expired" **and** marks them booked in one atomic step. If that fails after money was taken, we run a **compensating action** — a refund — instead of a database-style rollback (undoing unfinished changes as if they never happened), because a card charge cannot be "un-happened". This is the essence of the **Saga** idea: each step has an undo.

Booking lifecycle:



A show seat, as the customer sees it on the map: **free** (`.`), **held** by someone paying (`L`), **booked** (`X`). "Held" is not stored on the seat — it lives in the lock provider and simply disappears when it expires.

Design decisions & patterns used

Decision	Why	Alternative rejected
`ShowSeat` per show (course's key point)	The same physical seat is free in one show and taken in another; prices differ per show.	Booking `HallSeat` directly — breaks the moment there are two shows a day.
`SeatLockProvider` interface with TTL holds	Seats are kept only while the customer pays; abandoned checkouts free themselves. The interface lets you swap the in-memory version for **Redis** (a fast in-memory key-value store, often used for locks and caches; `SET key value NX PX 300000` — set only if absent, expire in 300 000 ms) or a database table with an `expires_at` column.	Holding seats until the booking is cancelled (course's version; it names this as the gap).
All-or-nothing lock	A customer asking for 4 seats together must never be left with 2. Check every seat first, then take all.	Locking seats one by one without rollback.
One monitor per show	Shows never block each other; within a show, every decision is serialized (made one at a time). Fine-grained enough for a cinema.	One global lock (the course's single-process lock) — correct but serializes the whole country's bookings.
Commit re-validates + compensation	Closes the "lock expired during payment" gap; the customer never pays for a seat they don't get.	Trusting the earlier check.
Idempotent `pay` (safe to repeat: two calls have the effect of one)	A double-click or network retry must not charge twice: paying a non-PENDING booking returns it unchanged. Real systems add an **idempotency key** per request.	Charging on every call.
Strategy for cancellation refunds and **Strategy-like `PaymentMethod`**	Refund rules and payment types vary independently of booking logic.	`if (method == CARD)` chains.
Adapter boundary (a wrapper that turns a foreign interface into ours) `PaymentGateway`	Wraps the external provider (Stripe, Razorpay…) behind our own interface; tests use `FakeGateway`.	Calling the provider's SDK (software development kit: the provider's own client library) from `BookingService`.
Coupon applied to the booking	Amount is correct before payment; payment types stay simple.	Each payment type applying coupons.
Guest has no booking method	The rule "guests must register to book" is enforced by *types*, not just written down: `hold` takes a `Customer`.	A `Guest.book()` that throws.
"Search by city" from shows	A movie is in a city only once a show is scheduled there; the other four searches are indexed when a movie is added.	Storing a city on `Movie`.

Java implementation

Package `lld.ch15.moviebooking`. Files: `Venue.java` (city, cinema, hall, seats, movie, show), `SeatLockProvider.java` (interface + in-memory TTL implementation), `Payments.java` (payment types, gateway, coupon, cancellation policy, notifier, clock), `Catalog.java` (five searches), `BookingService.java` (actors, booking, the booking path), `Main.java`.

```java
// File: lld/ch15/moviebooking/Venue.java
package lld.ch15.moviebooking;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

enum SeatType { REGULAR, PREMIUM, ACCESSIBLE, EMERGENCY_EXIT, OTHER }

record City(String name, String state) {}

/** The physical seat bolted to the floor. It has NO booking state: that depends on which show you ask about. */
record HallSeat(String id, SeatType type) {}

final class CinemaHall {
    private final String name;
    private final List<HallSeat> seats;
    private final List<Show> shows = new ArrayList<>();

    CinemaHall(String name, List<HallSeat> seats) { this.name = name; this.seats = List.copyOf(seats); }
    String name()            { return name; }
    List<HallSeat> seats()   { return seats; }

    /** A hall runs one show at a time, so an overlapping show is refused. */
    synchronized boolean addShow(Show show) {
        for (Show s : shows) if (s.overlaps(show)) return false;
        shows.add(show);
        return true;
    }
    synchronized List<Show> shows() { return List.copyOf(shows); }
}

final class Cinema {
    private final String name;
    private final City city;                        // association: the city exists without the cinema
    private final List<CinemaHall> halls = new ArrayList<>();   // composition: halls die with the cinema
    Cinema(String name, City city) { this.name = name; this.city = city; }
    String name()                { return name; }
    City city()                  { return city; }
    void addHall(CinemaHall h)   { halls.add(h); }
    List<CinemaHall> halls()     { return Collections.unmodifiableList(halls); }
}

record Movie(String id, String title, String language, String genre, LocalDate releaseDate, int durationMinutes) {}

/** One screening. Owns one ShowSeat per physical seat, so the same seat can be free at 6pm and taken at 9pm. */
final class Show {
    private final String id;
    private final Movie movie;
    private final Cinema cinema;
    private final CinemaHall hall;
    private final LocalDateTime start;
    private final Map<String, ShowSeat> seats = new LinkedHashMap<>();

    Show(String id, Movie movie, Cinema cinema, CinemaHall hall, LocalDateTime start, Map<SeatType, BigDecimal>
prices) {
        this.id = id; this.movie = movie; this.cinema = cinema; this.hall = hall; this.start = start;
        for (HallSeat hs : hall.seats())
            seats.put(hs.id(), new ShowSeat(hs, prices.getOrDefault(hs.type(), BigDecimal.ZERO)));
    }
    String id()                { return id; }
    Movie movie()              { return movie; }
    Cinema cinema()            { return cinema; }
```

```
    CinemaHall hall()           { return hall; }
    LocalDateTime start()       { return start; }
    LocalDateTime end()         { return start.plusMinutes(movie.durationMinutes()); }
    Map<String, ShowSeat> seats() { return Collections.unmodifiableMap(seats); }

    ShowSeat seat(String seatId) {
        ShowSeat s = seats.get(seatId);
        if (s == null) throw new IllegalArgumentException("no seat " + seatId + " in show " + id);
        return s;
    }
    boolean overlaps(Show o) { return start.isBefore(o.end()) && o.start().isBefore(end()); }
}

/** A seat for one show: its own price and its booked flag. Only SeatLockProvider flips `booked`. */
final class ShowSeat {
    private final HallSeat seat;
    private final BigDecimal price;          // money: BigDecimal, never double
    private volatile boolean booked;
    ShowSeat(HallSeat seat, BigDecimal price) { this.seat = seat; this.price = price; }
    HallSeat seat()        { return seat; }
    BigDecimal price()     { return price; }
    boolean isBooked()     { return booked; }
    void setBooked(boolean b) { booked = b; }
}
```

```java
// File: lld/ch15/moviebooking/SeatLockProvider.java
package lld.ch15.moviebooking;

import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.TreeSet;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Temporary, expiring holds on seats while the customer pays.
 * An interface so the in-memory version can be swapped for Redis (SET NX PX) or a DB row lock.
 */
interface SeatLockProvider {
    /** All-or-nothing: either every seat is locked for this user, or none is. */
    boolean lock(Show show, List<String> seatIds, String userId);
    /** Release this user's holds (payment failed, user gave up). */
    void unlock(Show show, List<String> seatIds, String userId);
    /** True while this user still holds every one of these seats and no hold has expired. */
    boolean holds(Show show, List<String> seatIds, String userId);
    /** Atomically: verify holds are still valid, mark seats BOOKED, drop the holds. */
    boolean commit(Show show, List<String> seatIds, String userId);
    /** Make booked seats sellable again (cancellation). */
    void releaseBooked(Show show, List<String> seatIds);
    Set<String> lockedSeats(Show show);
}

record SeatLock(String userId, Instant expiresAt) {
    boolean expired(Instant now) { return !now.isBefore(expiresAt); }
}

/**
 * One monitor per show: two shows never block each other, but every decision about one show's
 * seats (lock, commit, release) is made by one thread at a time. That is what makes
 * check-then-act safe and prevents double booking.
 */
final class InMemorySeatLockProvider implements SeatLockProvider {
    private final Duration ttl;
    private final Clock clock;
    private final Map<String, Map<String, SeatLock>> locksByShow = new ConcurrentHashMap<>();

    InMemorySeatLockProvider(Duration ttl, Clock clock) { this.ttl = ttl; this.clock = clock; }

    private Map<String, SeatLock> table(Show show) {
        return locksByShow.computeIfAbsent(show.id(), k -> new HashMap<>());
    }

    @Override public boolean lock(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            Instant now = clock.instant();
            for (String id : seatIds) {                          // 1. check every seat first
                if (show.seat(id).isBooked()) return false;
                SeatLock l = t.get(id);
                if (l != null && !l.expired(now) && !l.userId().equals(userId)) return false;
            }
            SeatLock mine = new SeatLock(userId, now.plus(ttl));
            for (String id : seatIds) t.put(id, mine);           // 2. then take them all
            return true;
        }
    }

    @Override public void unlock(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
```

```
        synchronized (t) {
            for (String id : seatIds) {
                SeatLock l = t.get(id);
                if (l != null && l.userId().equals(userId)) t.remove(id);
            }
        }
    }

    @Override public boolean holds(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) { return holdsLocked(t, seatIds, userId, clock.instant()); }
    }

    @Override public boolean commit(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            if (!holdsLocked(t, seatIds, userId, clock.instant())) return false;
            for (String id : seatIds) { show.seat(id).setBooked(true); t.remove(id); }
            return true;
        }
    }

    @Override public void releaseBooked(Show show, List<String> seatIds) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) { for (String id : seatIds) show.seat(id).setBooked(false); }
    }

    @Override public Set<String> lockedSeats(Show show) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            Instant now = clock.instant();
            Set<String> out = new TreeSet<>();
            t.forEach((id, l) -> { if (!l.expired(now)) out.add(id); });
            return out;
        }
    }

    private static boolean holdsLocked(Map<String, SeatLock> t, List<String> ids, String userId, Instant now) {
        for (String id : ids) {
            SeatLock l = t.get(id);
            if (l == null || l.expired(now) || !l.userId().equals(userId)) return false;
        }
        return true;
    }
}
```

```java
// File: lld/ch15/moviebooking/Payments.java
package lld.ch15.moviebooking;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.util.Set;
import java.util.concurrent.atomic.AtomicInteger;

enum PaymentStatus { UNPAID, PENDING, COMPLETED, DECLINED, CANCELLED, REFUNDED }

/** How the customer pays. Sealed: the system supports exactly these two (credit card or cash). */
sealed interface PaymentMethod permits CardPayment, CashPayment {}
record CardPayment(String cardNumberLast4) implements PaymentMethod {}
record CashPayment() implements PaymentMethod {}

record Payment(String txnId, BigDecimal amount, PaymentMethod method, PaymentStatus status) {
    Payment withStatus(PaymentStatus s) { return new Payment(txnId, amount, method, s); }
}

/** Adapter boundary to the real payment provider. */
interface PaymentGateway {
    Payment charge(String bookingId, BigDecimal amount, PaymentMethod method);
    Payment refund(Payment p);
}

/** Test double: declines listed cards, and can run a hook "while the bank is thinking". */
final class FakeGateway implements PaymentGateway {
    private final Set<String> declinedCards;
    private final AtomicInteger seq = new AtomicInteger();
    private volatile Runnable duringCharge = () -> {};

    FakeGateway(Set<String> declinedCards) { this.declinedCards = declinedCards; }
    void setDuringCharge(Runnable r) { duringCharge = r; }

    @Override public Payment charge(String bookingId, BigDecimal amount, PaymentMethod m) {
        duringCharge.run();
        String txn = "TXN-" + seq.incrementAndGet();
        boolean declined = m instanceof CardPayment c && declinedCards.contains(c.cardNumberLast4());
        return new Payment(txn, amount, m, declined ? PaymentStatus.DECLINED : PaymentStatus.COMPLETED);
    }
    @Override public Payment refund(Payment p) { return p.withStatus(PaymentStatus.REFUNDED); }
}

/** A percentage off. Applied to the booking amount BEFORE payment, so every payment type sees the right total. */
record Coupon(String code, int percentOff) {
    Coupon {
        if (percentOff < 0 || percentOff > 100) throw new IllegalArgumentException("bad percent " + percentOff);
    }
    /** Amount after the discount, rounded to whole paise/cents (half-up). */
    BigDecimal apply(BigDecimal amount) {
        BigDecimal off = amount.multiply(BigDecimal.valueOf(percentOff)).divide(BigDecimal.valueOf(100), 2,
RoundingMode.HALF_UP);
        return amount.subtract(off).setScale(2, RoundingMode.HALF_UP);
    }
}

/** Strategy: how much of the money comes back on cancellation. */
interface CancellationPolicy {
    BigDecimal refund(BigDecimal paid, LocalDateTime now, LocalDateTime showStart);
}

/** Full refund up to `cutoff` before the show, nothing after. */
```

```
record CutoffRefundPolicy(Duration cutoff) implements CancellationPolicy {
    @Override public BigDecimal refund(BigDecimal paid, LocalDateTime now, LocalDateTime showStart) {
        return now.plus(cutoff).isAfter(showStart) ? BigDecimal.ZERO : paid;
    }
}

interface Notifier { void send(String userId, String message); }

final class ConsoleNotifier implements Notifier {
    @Override public void send(String userId, String message) {
        System.out.println("  [notify " + userId + "] " + message);
    }
}

final class SimClock extends Clock {
    private volatile Instant now;
    SimClock(Instant start) { this.now = start; }
    void advance(Duration d) { now = now.plus(d); }
    @Override public ZoneId getZone() { return ZoneOffset.UTC; }
    @Override public Clock withZone(ZoneId zone) { return this; }
    @Override public Instant instant() { return now; }
}

final class Money {
    private Money() {}
    static String fmt(BigDecimal amount) { return "Rs." + amount.setScale(2,
RoundingMode.UNNECESSARY).toPlainString(); }
}
```

```java
// File: lld/ch15/moviebooking/Catalog.java
package lld.ch15.moviebooking;

import java.time.LocalDate;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

/** The five searches. Four are indexed when a movie is added; "by city" is answered from the shows. */
interface MovieSearch {
    List<Movie> searchByTitle(String title);
    List<Movie> searchByLanguage(String language);
    List<Movie> searchByGenre(String genre);
    List<Movie> searchByReleaseDate(LocalDate date);
    List<Movie> searchByCity(String cityName);
}

final class Catalog implements MovieSearch {
    private final Map<String, List<Movie>> byTitle = new ConcurrentHashMap<>();
    private final Map<String, List<Movie>> byLanguage = new ConcurrentHashMap<>();
    private final Map<String, List<Movie>> byGenre = new ConcurrentHashMap<>();
    private final Map<LocalDate, List<Movie>> byRelease = new ConcurrentHashMap<>();
    private final List<Show> shows = new CopyOnWriteArrayList<>();
    private final Notifier notifier;

    Catalog(Notifier notifier) { this.notifier = notifier; }

    /** Admin adds a movie: index it four ways and announce it (requirement: notify on a new movie). */
    void addMovie(Movie m) {
        put(byTitle, key(m.title()), m);
        put(byLanguage, key(m.language()), m);
        put(byGenre, key(m.genre()), m);
        byRelease.computeIfAbsent(m.releaseDate(), d -> new CopyOnWriteArrayList<>()).add(m);
        notifier.send("subscribers", "new movie: " + m.title() + " (" + m.language() + ", " + m.genre() + ")");
    }

    /** Admin schedules a show; the hall refuses it if it overlaps another. */
    boolean addShow(Show s) {
        if (!s.hall().addShow(s)) return false;
        shows.add(s);
        return true;
    }

    /** After picking a movie: which shows play in my city? */
    List<Show> showsOf(Movie m, String cityName) {
        return shows.stream()
                .filter(s -> s.movie().equals(m) && s.cinema().city().name().equalsIgnoreCase(cityName))
                .sorted(Comparator.comparing(Show::start)).toList();
    }

    @Override public List<Movie> searchByTitle(String t)    { return get(byTitle, key(t)); }
    @Override public List<Movie> searchByLanguage(String l) { return get(byLanguage, key(l)); }
    @Override public List<Movie> searchByGenre(String g)    { return get(byGenre, key(g)); }
    @Override public List<Movie> searchByReleaseDate(LocalDate d) { return List.copyOf(byRelease.getOrDefault(d,
List.of())); }
    @Override public List<Movie> searchByCity(String city) {
        return shows.stream().filter(s -> s.cinema().city().name().equalsIgnoreCase(city))
                .map(Show::movie).distinct().sorted(Comparator.comparing(Movie::title)).toList();
    }

    private static String key(String s) { return s.trim().toLowerCase(Locale.ROOT); }
    private static void put(Map<String, List<Movie>> idx, String k, Movie m) {
        idx.computeIfAbsent(k, x -> new CopyOnWriteArrayList<>()).add(m);
    }
    private static List<Movie> get(Map<String, List<Movie>> idx, String k) {
```

```
        return idx.getOrDefault(k, List.of()).stream().sorted(Comparator.comparing(Movie::title)).toList();
    }
}
```

```java
// File: lld/ch15/moviebooking/BookingService.java
package lld.ch15.moviebooking;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

enum BookingStatus { PENDING, CONFIRMED, CHECKED_IN, CANCELED, ABANDONED }

enum AccountStatus { ACTIVE, BLOCKED, BANNED, COMPROMISED, ARCHIVED, UNKNOWN }

/** Only registered customers can book. A guest has no type that BookingService accepts. */
final class Customer {
    private final String id;
    private final String name;
    private volatile AccountStatus status = AccountStatus.ACTIVE;
    Customer(String id, String name) { this.id = id; this.name = name; }
    String id()                { return id; }
    String name()              { return name; }
    AccountStatus status()     { return status; }
    void setStatus(AccountStatus s) { status = s; }
}

record Guest(String sessionId) {
    Customer register(String id, String name) { return new Customer(id, name); }
}

record FrontDeskOfficer(String id, String name) {}

record Admin(String id, String name) {
    void block(Customer c)   { c.setStatus(AccountStatus.BLOCKED); }
    void unblock(Customer c) { c.setStatus(AccountStatus.ACTIVE); }
}

final class SeatUnavailableException extends RuntimeException {
    private static final long serialVersionUID = 1L;
    SeatUnavailableException(String msg) { super(msg); }
}

/** A booking points at the show and seats; it owns none of them. Transitions run under its own monitor. */
final class Booking {
    private final String id;
    private final String customerId;
    private final Show show;
    private final List<String> seatIds;
    private final BigDecimal baseAmount;
    private BookingStatus status = BookingStatus.PENDING;
    private Payment payment;
    private Coupon coupon;

    Booking(String id, String customerId, Show show, List<String> seatIds, BigDecimal baseAmount) {
        this.id = id; this.customerId = customerId; this.show = show;
        this.seatIds = List.copyOf(seatIds); this.baseAmount = baseAmount;
    }
    String id()              { return id; }
    String customerId()      { return customerId; }
    Show show()              { return show; }
    List<String> seatIds()   { return seatIds; }
    BigDecimal baseAmount()  { return baseAmount; }
    synchronized BookingStatus status() { return status; }
    synchronized Payment payment()      { return payment; }
    synchronized Coupon coupon()        { return coupon; }
    void setStatus(BookingStatus s)     { status = s; }          // called with the monitor held
```

```
    void setPayment(Payment p)          { payment = p; }
    void setCoupon(Coupon c)            { coupon = c; }
    @Override public synchronized String toString() {
        return id + " " + seatIds + " " + status + (payment == null ? "" : " " + payment.status() + " " +
Money.fmt(payment.amount()));
    }
}

/**
 * The booking path: HOLD seats (TTL) -> PAY -> COMMIT seats. If the hold was lost while the bank
 * was charging, the money is refunded (compensation) instead of double-selling the seat.
 */
final class BookingService {
    private final SeatLockProvider locks;
    private final PaymentGateway gateway;
    private final CancellationPolicy cancellationPolicy;
    private final Notifier notifier;
    private final Clock clock;
    private final Map<String, Booking> bookings = new ConcurrentHashMap<>();
    private final AtomicInteger seq = new AtomicInteger();

    BookingService(SeatLockProvider locks, PaymentGateway gateway, CancellationPolicy policy, Notifier notifier,
Clock clock) {
        this.locks = locks; this.gateway = gateway; this.cancellationPolicy = policy;
        this.notifier = notifier; this.clock = clock;
    }

    /** Step 1: hold the seats. All-or-nothing. Throws if any seat is booked or held by someone else. */
    Booking hold(Customer c, Show show, List<String> seatIds) {
        if (c.status() != AccountStatus.ACTIVE) throw new IllegalStateException(c.id() + " is " + c.status());
        if (seatIds.isEmpty()) throw new IllegalArgumentException("no seats");
        List<String> sorted = seatIds.stream().distinct().sorted().toList();
        if (!locks.lock(show, sorted, c.id()))
            throw new SeatUnavailableException(sorted + " not available for " + show.id());
        BigDecimal base = sorted.stream().map(id -> show.seat(id).price()).reduce(BigDecimal.ZERO,
BigDecimal::add);
        Booking b = new Booking("BK-" + seq.incrementAndGet(), c.id(), show, sorted, base);
        bookings.put(b.id(), b);
        return b;
    }

    /** The front desk books for a walk-in customer: same flow, different actor. */
    Booking holdFor(FrontDeskOfficer officer, Customer c, Show show, List<String> seatIds) {
        return hold(c, show, seatIds);
    }

    /** Step 2 + 3: pay, then commit. Idempotent: paying a confirmed booking again returns it unchanged. */
    Booking pay(String bookingId, PaymentMethod method, Coupon coupon) {
        Booking b = get(bookingId);
        synchronized (b) {
            if (b.status() != BookingStatus.PENDING) return b;
            if (!locks.holds(b.show(), b.seatIds(), b.customerId())) {       // fast fail, no charge
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " hold expired before payment, nothing charged");
                return b;
            }
            b.setCoupon(coupon);
            BigDecimal amount = coupon == null ? b.baseAmount() : coupon.apply(b.baseAmount());
            Payment p = gateway.charge(b.id(), amount, method);
            b.setPayment(p);
            if (p.status() != PaymentStatus.COMPLETED) {
                locks.unlock(b.show(), b.seatIds(), b.customerId());          // free seats at once
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " payment " + p.status() + ", seats released");
                return b;
            }
            if (!locks.commit(b.show(), b.seatIds(), b.customerId())) {       // hold lost during charge
                b.setPayment(gateway.refund(p));                              // COMPENSATION
                b.setStatus(BookingStatus.ABANDONED);
```

```
                notifier.send(b.customerId(), b.id() + " hold expired during payment, " + Money.fmt(amount) + "
refunded");
                return b;
            }
            b.setStatus(BookingStatus.CONFIRMED);
            notifier.send(b.customerId(), b.id() + " CONFIRMED " + b.seatIds() + " for '" +
b.show().movie().title()
                    + "' at " + b.show().start() + ", paid " + Money.fmt(amount));
            return b;
        }
    }

    /** Cancel: release seats and refund according to the policy. */
    Booking cancel(String bookingId) {
        Booking b = get(bookingId);
        synchronized (b) {
            switch (b.status()) {
                case PENDING -> {
                    locks.unlock(b.show(), b.seatIds(), b.customerId());
                    b.setStatus(BookingStatus.CANCELED);
                }
                case CONFIRMED -> {
                    locks.releaseBooked(b.show(), b.seatIds());
                    BigDecimal refund = cancellationPolicy.refund(b.payment().amount(),
                            LocalDateTime.now(clock), b.show().start());
                    if (refund.signum() > 0) b.setPayment(gateway.refund(b.payment()));
                    b.setStatus(BookingStatus.CANCELED);
                    notifier.send(b.customerId(), b.id() + " canceled, refund " + Money.fmt(refund));
                }
                default -> throw new IllegalStateException(b.id() + " cannot be canceled from " + b.status());
            }
            return b;
        }
    }

    Booking get(String id) {
        Booking b = bookings.get(id);
        if (b == null) throw new IllegalArgumentException("no booking " + id);
        return b;
    }

    /** "." free, "L" held by someone paying, "X" booked. */
    String seatMap(Show show) {
        Set<String> held = locks.lockedSeats(show);
        StringBuilder sb = new StringBuilder();
        char row = 0;
        for (ShowSeat s : show.seats().values()) {
            String id = s.seat().id();
            if (id.charAt(0) != row) { if (row != 0) sb.append('\n'); row = id.charAt(0); sb.append("
").append(row).append(' '); }
            sb.append(s.isBooked() ? 'X' : held.contains(id) ? 'L' : '.').append(' ');
        }
        return sb.toString();
    }
}
```

```java
// File: lld/ch15/moviebooking/Main.java
package lld.ch15.moviebooking;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) throws Exception {
        SimClock clock = new SimClock(Instant.parse("2026-05-01T12:00:00Z"));
        FakeGateway bank = new FakeGateway(Set.of("0002"));
        SeatLockProvider locks = new InMemorySeatLockProvider(Duration.ofMinutes(5), clock);
        Notifier notifier = new ConsoleNotifier();
        BookingService svc = new BookingService(locks, bank, new CutoffRefundPolicy(Duration.ofHours(2)),
                notifier, clock);
        Catalog catalog = new Catalog(notifier);

        // ---- venue: 2 rows x 5 seats; row A regular, row B premium
        City blr = new City("Bengaluru", "KA");
        Cinema pvr = new Cinema("PVR Forum", blr);
        List<HallSeat> seats = new ArrayList<>();
        for (char row : new char[]{'A', 'B'})
            for (int n = 1; n <= 5; n++) seats.add(new HallSeat("" + row + n, row == 'A' ? SeatType.REGULAR :
SeatType.PREMIUM));
        CinemaHall audi1 = new CinemaHall("Audi 1", seats);
        pvr.addHall(audi1);

        Movie inception = new Movie("MV1", "Inception", "English", "Sci-Fi", LocalDate.of(2010, 7, 16), 148);
        Movie rrr = new Movie("MV2", "RRR", "Telugu", "Action", LocalDate.of(2022, 3, 25), 187);
        Map<SeatType, BigDecimal> prices = Map.of(SeatType.REGULAR, new BigDecimal("200.00"),
                SeatType.PREMIUM, new BigDecimal("350.00"));
        Show evening = new Show("SH1", inception, pvr, audi1, LocalDateTime.of(2026, 5, 1, 18, 0), prices);
        Show clash = new Show("SH2", rrr, pvr, audi1, LocalDateTime.of(2026, 5, 1, 19, 0), prices);

        System.out.println("== 1. Admin adds movies (subscribers told) and schedules shows; the hall refuses an
overlap ==");
        catalog.addMovie(inception);
        catalog.addMovie(rrr);
        System.out.println("  add SH1 Inception 18:00 -> " + catalog.addShow(evening));
        System.out.println("  add SH2 RRR 19:00 same hall -> " + catalog.addShow(clash));

        System.out.println("\n== 2. Search ==");
        System.out.println("  city Bengaluru -> " + titles(catalog.searchByCity("bengaluru")));
        System.out.println("  genre Action   -> " + titles(catalog.searchByGenre("action")) + " (no show yet, but
indexed)");
        System.out.println("  Inception shows in Bengaluru -> "
                + catalog.showsOf(inception, "Bengaluru").stream().map(s -> s.id() + "@" + s.start()).toList());

        System.out.println("\n== 3. 10 customers click B3+B4 at the same instant ==");
        List<Customer> crowd = new ArrayList<>();
        for (int i = 1; i <= 10; i++) crowd.add(new Customer("U" + i, "User " + i));
        ConcurrentLinkedQueue<Booking> won = new ConcurrentLinkedQueue<>();
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(10)) {
            for (Customer c : crowd)
                pool.submit(() -> {
                    go.await();
                    try { won.add(svc.hold(c, evening, List.of("B3", "B4"))); }
```

```
                    catch (SeatUnavailableException ignored) { }
                    return null;
                });
            go.countDown();
        }
        Booking first = won.peek();
        System.out.println("  holds granted = " + won.size() + " (the other 9 saw 'not available')");
        System.out.println(svc.seatMap(evening));

        System.out.println("\n== 4. The winner pays by card with a 10% coupon ==");
        svc.pay(first.id(), new CardPayment("4242"), new Coupon("FIRST10", 10));
        System.out.println("  paying twice is harmless -> " + svc.pay(first.id(), new CardPayment("4242"),
null).status());
        System.out.println(svc.seatMap(evening));

        System.out.println("\n== 5. Hold expires (TTL 5 min) before paying ==");
        Customer asha = new Customer("ASHA", "Asha"), ravi = new Customer("RAVI", "Ravi");
        Booking slow = svc.hold(asha, evening, List.of("A1", "A2"));
        clock.advance(Duration.ofMinutes(6));
        Booking fast = svc.hold(ravi, evening, List.of("A1", "A2"));
        System.out.println("  Ravi re-held A1,A2 after Asha's hold expired -> " + fast.id());
        svc.pay(slow.id(), new CardPayment("4242"), null);
        svc.pay(fast.id(), new CashPayment(), null);

        System.out.println("\n== 6. Hold expires WHILE the bank is charging -> refund (compensation) ==");
        Customer meena = new Customer("MEENA", "Meena"), kiran = new Customer("KIRAN", "Kiran");
        Booking victim = svc.hold(meena, evening, List.of("A5"));
        bank.setDuringCharge(() -> {                        // bank is slow: 6 minutes pass, Kiran grabs A5
            clock.advance(Duration.ofMinutes(6));
            svc.hold(kiran, evening, List.of("A5"));
        });
        svc.pay(victim.id(), new CardPayment("4242"), null);
        bank.setDuringCharge(() -> {});
        System.out.println("  " + victim);

        System.out.println("\n== 7. Declined card releases the seats immediately ==");
        Booking declined = svc.hold(new Customer("NEHA", "Neha"), evening, List.of("B1"));
        svc.pay(declined.id(), new CardPayment("0002"), null);
        System.out.println("  B1 held by anyone now? " + locks.lockedSeats(evening).contains("B1"));

        System.out.println("\n== 8. Blocked customer, then cancellation with refund ==");
        new Admin("AD1", "Admin").block(crowd.get(9));
        try { svc.hold(crowd.get(9), evening, List.of("B5")); }
        catch (IllegalStateException e) { System.out.println("  REFUSED: " + e.getMessage()); }
        svc.cancel(first.id());                             // 12:06+ vs 18:00 show: more than 2h before
        System.out.println("  " + first);
        System.out.println(svc.seatMap(evening));
    }

    private static List<String> titles(List<Movie> ms) { return ms.stream().map(Movie::title).toList(); }
}
```

Output:

```
== 1. Admin adds movies (subscribers told) and schedules shows; the hall refuses an overlap ==
  [notify subscribers] new movie: Inception (English, Sci-Fi)
  [notify subscribers] new movie: RRR (Telugu, Action)
  add SH1 Inception 18:00 -> true
  add SH2 RRR 19:00 same hall -> false

== 2. Search ==
  city Bengaluru -> [Inception]
  genre Action   -> [RRR] (no show yet, but indexed)
  Inception shows in Bengaluru -> [SH1@2026-05-01T18:00]

== 3. 10 customers click B3+B4 at the same instant ==
  holds granted = 1 (the other 9 saw 'not available')
  A . . . . .
  B . . L L .

== 4. The winner pays by card with a 10% coupon ==
  [notify U10] BK-1 CONFIRMED [B3, B4] for 'Inception' at 2026-05-01T18:00, paid Rs.630.00
  paying twice is harmless -> CONFIRMED
  A . . . . .
  B . . X X .

== 5. Hold expires (TTL 5 min) before paying ==
  Ravi re-held A1,A2 after Asha's hold expired -> BK-3
  [notify ASHA] BK-2 hold expired before payment, nothing charged
  [notify RAVI] BK-3 CONFIRMED [A1, A2] for 'Inception' at 2026-05-01T18:00, paid Rs.400.00

== 6. Hold expires WHILE the bank is charging -> refund (compensation) ==
  [notify MEENA] BK-4 hold expired during payment, Rs.200.00 refunded
  BK-4 [A5] ABANDONED REFUNDED Rs.200.00

== 7. Declined card releases the seats immediately ==
  [notify NEHA] BK-6 payment DECLINED, seats released
  B1 held by anyone now? false

== 8. Blocked customer, then cancellation with refund ==
  REFUSED: U10 is BLOCKED
  [notify U10] BK-1 canceled, refund Rs.630.00
  BK-1 [B3, B4] CANCELED REFUNDED Rs.630.00
  A X X . . L
  B . . . . .
```

What to notice: section 1 announces each new movie to subscribers (requirement 9) and refuses an overlapping show in the same hall (requirement 2); section 3 lets exactly 1 of 10 simultaneous customers hold B3+B4; section 4 applies the 10% coupon (Rs.700 → Rs.630) and a second `pay` is a no-op; section 5 shows an expired hold being re-sold and the slow customer *not* being charged; section 6 is the compensation path — Meena's card was charged, but her hold expired during the charge and Kiran took A5, so she is refunded; section 7 releases seats immediately on a decline; section 8 blocks a customer and cancels with a full refund.

Concurrency & edge cases

- **Check-then-act is atomic.** `lock`, `holds`, `commit`, `unlock` and `releaseBooked` for one show all run inside `synchronized` on that show's lock table. "Is the seat free? → take it" can't interleave.
- **Expired holds are lazy.** There is no background thread; an expired `SeatLock` is simply treated as absent the next time anyone looks. (In Redis, the key's TTL does the same.)
- **The payment-window race** (hold expires mid-payment) is handled by commit re-validation + refund, as shown in section 6.
- **Lock granularity** (how much data one lock covers). The booking's own monitor makes `pay`/`cancel` on the same booking serial (no double charge, no pay-while-cancel). The show monitor protects seats. The two are always taken in the order booking → show, never the reverse, so no deadlock.
- **Across many servers** (real BookMyShow), in-process locks are not enough. Options, from simplest:
  1. **Conditional update** (course): `UPDATE show_seat SET status='HELD', held_by=?, held_until=? WHERE show_id=? AND seat_id IN (...) AND (status='FREE' OR held_until < now())`, then check the row count equals the number of seats; if not, roll back.

2. **Pessimistic lock** (lock first, then work, assuming conflicts are likely): `SELECT ... FOR UPDATE` on the seat rows inside a transaction (a group of database statements that succeed or fail together).
3. **Serializable transaction** (course): the result is as if each transaction had run alone, in some order — easiest to reason about, but the slowest option when many buyers collide.
4. **Optimistic locking**: a `version` column; update `WHERE version = ?`; retry on conflict. Good when conflicts are rare.
5. **Unique constraint** (a database rule that rejects a second row with the same values) on `(show_id, seat_id)` in the confirmed-tickets table as the final safety net.
6. **Distributed lock / hold store** (a lock kept in a shared store, so all servers see it) in Redis with TTL for the *hold* phase, with the database as the source of truth (the one copy that wins when copies disagree) for *confirmed* seats. The course's point, in short: the in-process lock only makes the common case quick; once there are two servers, correctness comes from the database.

- **Edge cases:** empty seat list; duplicate seat ids in one request (de-duplicated); unknown seat; blocked customer; overlapping show in a hall; cancelling an abandoned booking (rejected); cancelling too late (no refund).

Extensions & follow-up questions

- **"Show the seat map live."** Push seat changes (held/booked/released) to open seat-map pages over WebSockets or server-sent events (two ways for a server to push updates to an open browser page); the lock provider publishes events (Observer).
- **"Extend the hold when the user reaches the payment page."** Add `extend(show, seats, user, extra)` that refreshes `expiresAt` only if the caller still holds the lock.
- **"Dynamic pricing / different prices per seat type and time."** Replace the price map with a `PricingStrategy` (base by `SeatType`, surcharge for weekends or premieres).
- **"Payment succeeded but our server crashed before commit."** Store the booking as `PENDING` with the gateway's transaction id before charging; a reconciliation job (a periodic task that compares our records with the gateway's and fixes mismatches) queries the gateway for pending bookings and either commits (if the seats are still held) or refunds. Payment providers' **webhooks** (HTTP calls the provider makes to your server when something happens) notify you of the final result.
- **"Cancel a whole show."** Admin cancels → every confirmed booking of the show is cancelled with a full refund and a notification (iterate bookings by show).
- **"Loyalty points, food & beverage add-ons."** Extra line items on the booking; points as another `PaymentMethod`.
- **"Seat gaps rule (no single empty seat left between bookings)."** A validation step in `hold` before locking — a `SeatSelectionRule` chain (Chain of Responsibility).

Real-world example

On **BookMyShow**, after you pick seats the app shows a **countdown timer** on the payment screen; if it runs out the seats are released and you must start again — that timer is the seat hold's TTL. If your bank debits you but the booking does not complete, the app tells you the amount will be **refunded automatically** — that is the compensation path. Big releases put the "one monitor per show" idea under real stress: first-day-first-show bookings for a blockbuster concentrate thousands of users on a handful of shows.

In the interview

30–45 minute plan. (1) 4 min — requirements; ask about hold time, payment, refunds. (2) 4 min — actors/use cases. (3) 8 min — entities: city → cinema → hall → seat; movie → show → **show seat** (say why). (4) 12 min — the booking path: `SeatLockProvider` (all-or-nothing, TTL), `BookingService.hold/pay/cancel`; write `lock` and `commit`. (5) 5 min — payment failure and the expired-during-payment refund. (6) Remaining — multi-server concurrency (conditional update / Redis TTL), follow-ups.

> **30-second answer:** "Venue hierarchy is City → Cinema → Hall → HallSeat. A Show owns a ShowSeat per hall seat, and that's what's booked. Booking is hold → pay → commit: a SeatLockProvider takes all requested seats atomically with a 5-minute TTL; payment goes through a gateway interface; commit re-checks the hold and marks seats booked atomically, and if the hold was lost we refund. In one process a per-show monitor guarantees no double booking; across servers the database's conditional update or a unique constraint is the real guarantee, with Redis TTL keys for holds."

Follow-ups with answers: *Why not lock the whole show table?* Fine for one cinema, but a global lock serializes all shows. *Why TTL?* Without it an abandoned checkout blocks seats forever. *Optimistic or pessimistic?* Pessimistic/conditional-update for hot shows (conflicts common); optimistic when conflicts are rare.

Self-check

- **Q:** Why is `ShowSeat` separate from `HallSeat`? **A:** Booking state and price are per show; the physical seat is shared by every show in the hall.
- **Q:** What does "all-or-nothing" mean in `lock`? **A:** Check every requested seat first; take them all only if all are free — never a partial set.
- **Q:** A payment succeeded but `commit` returned false. What now? **A:** Refund (compensation) and mark the booking abandoned.
- **Q:** How does an expired hold get cleaned up here? **A:** Lazily: any lock/holds check treats an expired lock as absent.
- **Q:** What makes `pay` safe to call twice? **A:** It runs under the booking's monitor and returns immediately unless the booking is `PENDING`.

TL;DR

Book `ShowSeat`s, not seats. Hold (all-or-nothing, TTL) → pay → commit (re-validate atomically) → refund if the hold was lost. Coupon on the booking, gateway behind an interface, refund rule as a Strategy. Per-show lock in one JVM (Java Virtual Machine — one running Java process); conditional updates, unique constraints and Redis TTL keys across servers.

15.3 Design a Concert Ticket Booking System

Problem & clarifying questions

A concert ticketing system (think **Ticketmaster**, **Live Nation**, **SeatGeek**, **BookMyShow Live**) sells tickets for a single, huge, one-off event. The *objects* look like section 15.2 — an event, a venue, seats, bookings, payments — so we reuse those ideas and do **not** repeat them. The *focus* is different: **extreme contention**. A movie show has a few hundred seats and demand spread over days; a stadium tour sells 50,000 tickets in minutes to a million people who all arrive at 10:00:00. That changes three things:

1. **Fairness** — who gets to try first? (a *virtual waiting room*)
2. **Throughput under contention** (throughput = requests completed per second; see System Design Fundamentals) — locks that serialize everyone become the bottleneck (the one point that limits overall speed) (so we use *lock-free* compare-and-set)
3. **Abuse** — scalpers (people who buy tickets to resell at a markup) and bots (automated scripts) (so *per-user limits*), and what happens to fans who miss out (a *waitlist*)

Question	Answer we assume
Seat types?	Tiers: **VIP** and **PREMIUM** are numbered seats; **GENERAL_ADMISSION** (GA) is a standing area — just a *count*.
Search?	By artist, venue, date and time (same catalog idea as 15.2 — not repeated).
How do we stop a stampede?	A **virtual waiting room**: everyone gets a queue number; people are admitted in batches, in order.
Limit per person?	**4 tickets** per user per concert.
Hold time?	**10 minutes** to pay.
Sold out?	Fans can join a **waitlist** per tier; released tickets are offered to the waitlist first.
Notifications?	Confirmation and "a ticket freed up for you" (email/SMS).

Requirements

Functional — show concerts with their seating layout and availability; search by artist, venue, date, time; choose seats (or a GA quantity) and buy; **prevent double booking under concurrent requests**; **fair access** for all users; secure payments; confirmation notifications; **waitlist** for sold-out events.

Non-functional — very high write contention for a few minutes; **no overselling** ever (GA count never below zero, a seat never sold twice); fairness (first-come-first-served into the sale); bounded per-user purchases; holds that expire on their own.

Use cases

Actors: **Fan** (join queue, hold, buy, cancel, join waitlist), **Organizer/Admin** (create concert, set tiers and prices, open the sale), **System** (admit batches from the queue, expire holds, offer released tickets to the waitlist, notify), **Payment provider** (external).

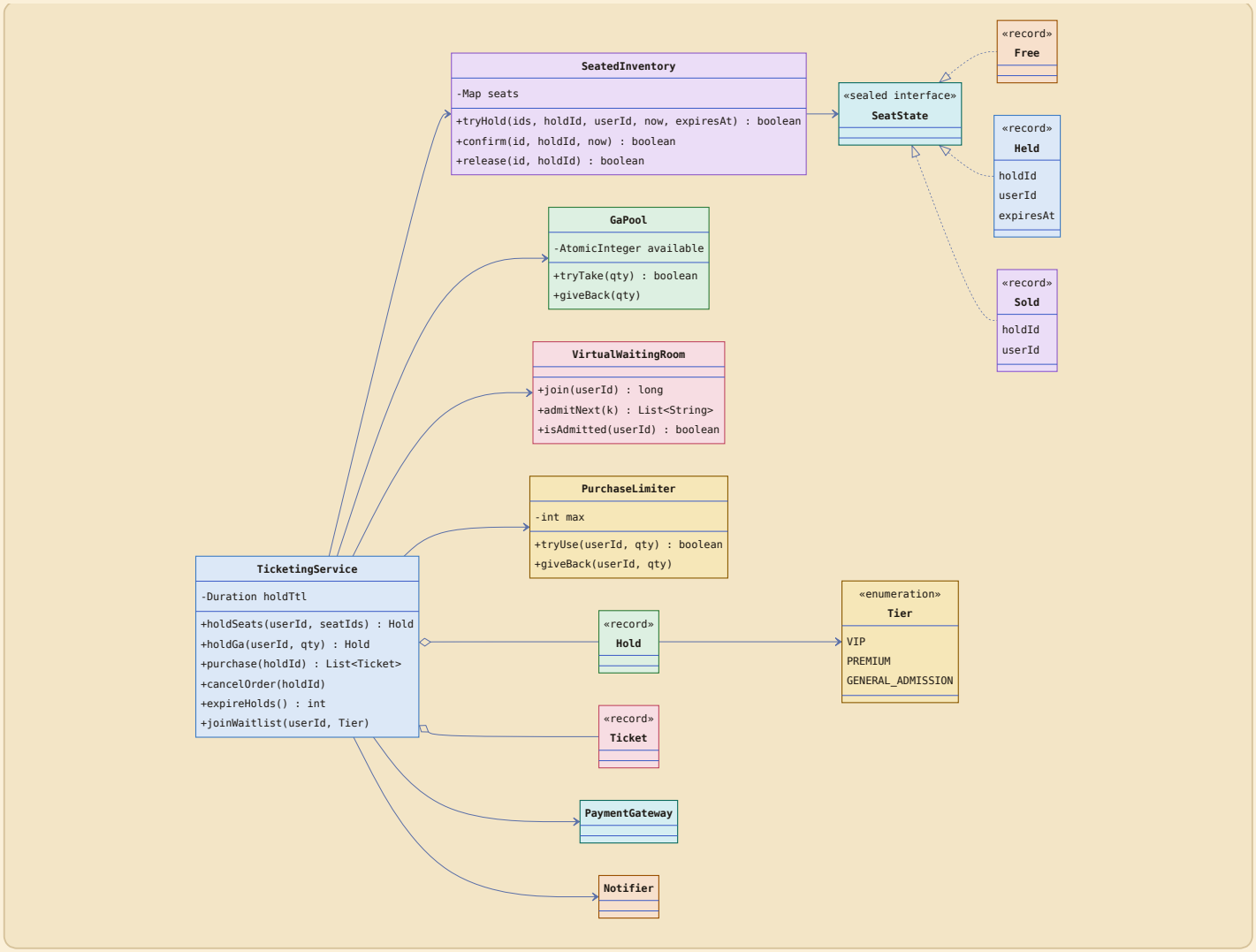
Main use cases: join the waiting room → get admitted → **hold** seats or a GA quantity → **purchase** → receive tickets; **cancel** an order (refund); **join waitlist**; system **reaps** expired holds.

Core entities

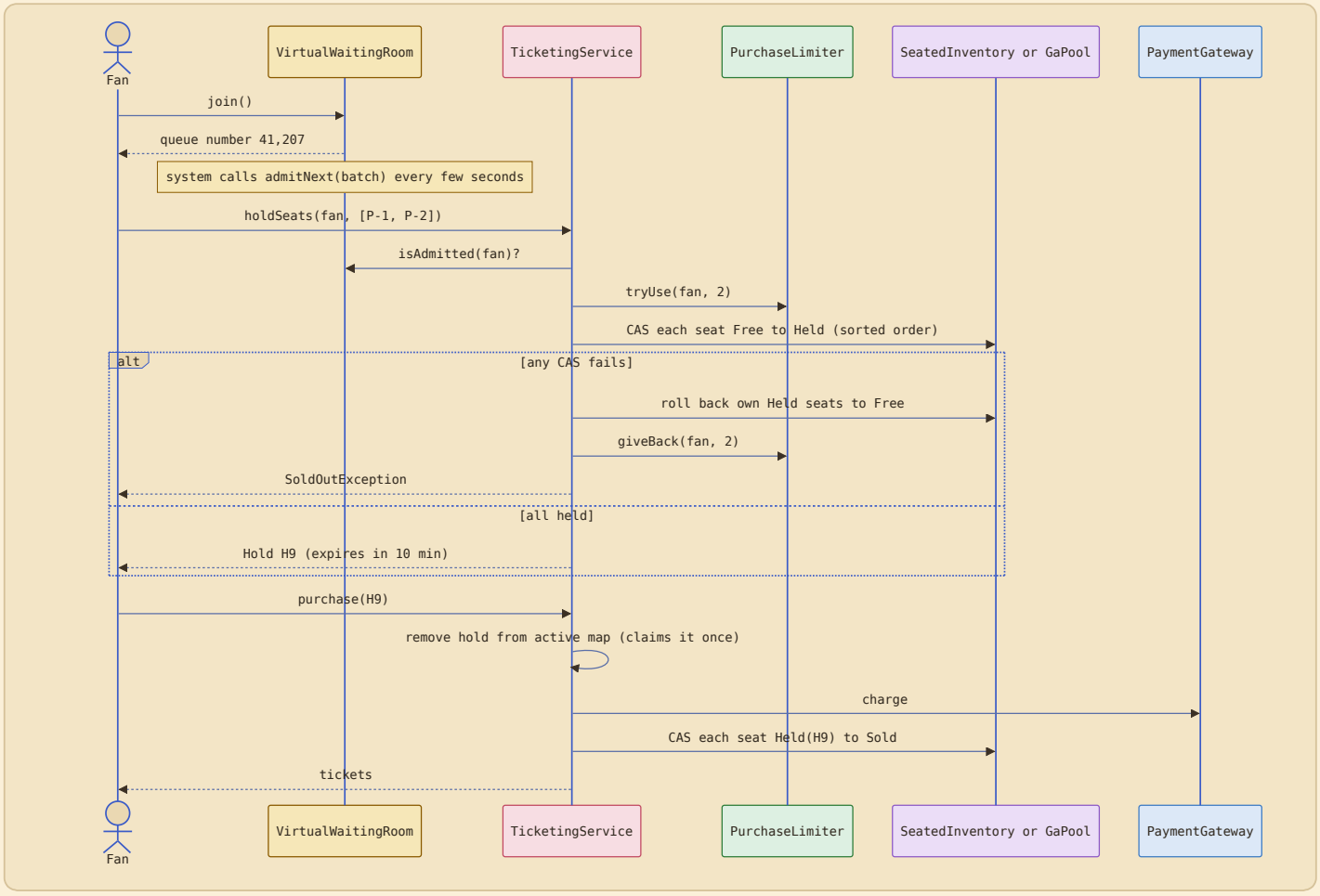
Class	Responsibility
`Concert` (record)	Artist, venue, date/time.
`Tier` (enum)	`VIP`, `PREMIUM` (seated) and `GENERAL_ADMISSION` (unseated), each with a price.
`SeatState` (sealed) → `Free`, `Held`, `Sold`	A seat's whole state as one immutable value.
`SeatedInventory`	One `AtomicReference<SeatState>` (a holder for one object reference that supports CAS) per numbered seat; claims by **CAS**.
`GaPool`	GA capacity as an `AtomicInteger` (an int with atomic updates); take/give back by CAS.
`Hold` (record)	A temporary claim: seats or a GA quantity, amount, expiry.
`Ticket` (record)	One admission: tier and seat id or "GA".
`VirtualWaitingRoom`	Queue numbers in arrival order; admits batches FIFO.
`PurchaseLimiter`	At most N tickets per user, checked and updated atomically.
`TicketingService`	hold → purchase → (cancel); reaper; waitlist offers.
`PaymentGateway`, `Notifier`	External boundaries (as in 15.2).

Class diagram

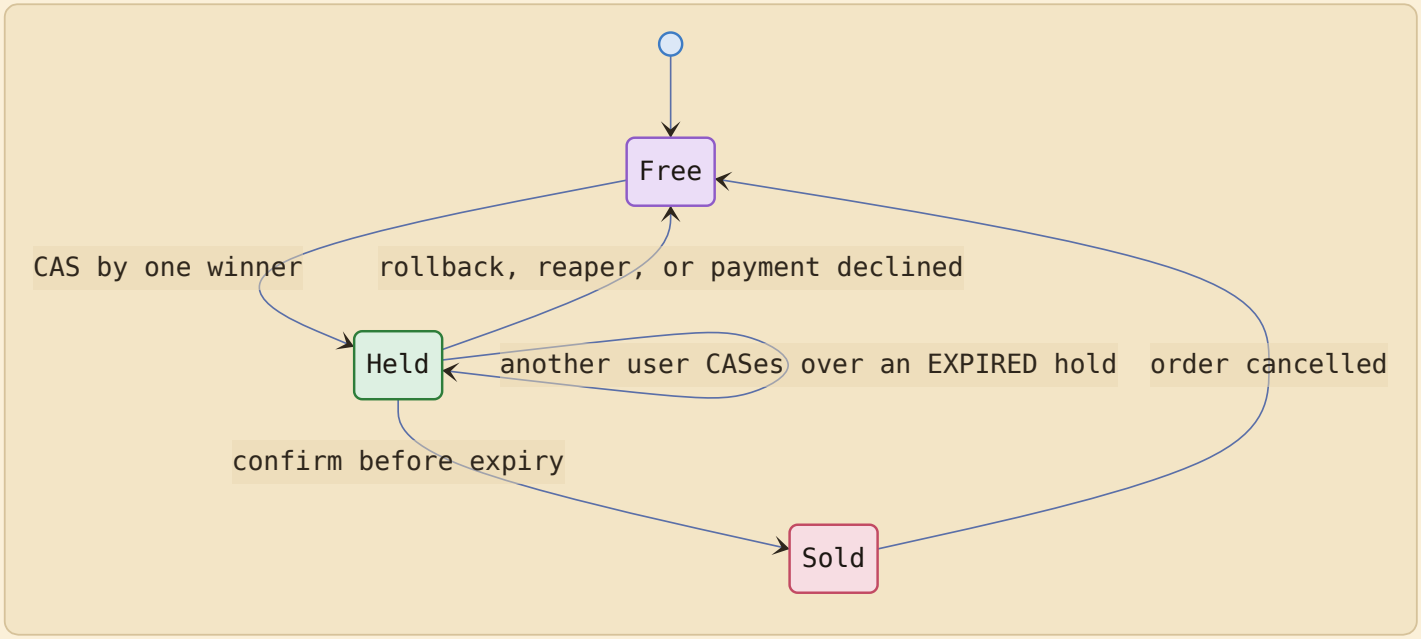
There is no course lesson for this problem; the problem statement comes from the *awesome-low-level-design* problem set. Our design:


SeatedInventory
-Map seats
+tryHold(ids, holdId, userId, now, expiresAt) : boolean
+confirm(id, holdId, now) : boolean
+release(id, holdId) : boolean
«sealed interface»
SeatState
«record»
Free
«record»
Held
holdId
userId
expiresAt
«record»
Sold
holdId
userId
GaPool
-AtomicInteger available
+tryTake(qty) : boolean
+giveBack(qty)
VirtualWaitingRoom
+join(userId) : long
+admitNext(k) : List<String>
+isAdmitted(userId) : boolean
PurchaseLimiter
-int max
+tryUse(userId, qty) : boolean
+giveBack(userId, qty)
TicketingService
-Duration holdTtl
+holdSeats(userId, seatIds) : Hold
+holdGa(userId, qty) : Hold
+purchase(holdId) : List<Ticket>
+cancelOrder(holdId)
+expireHolds() : int
+joinWaitlist(userId, Tier)
«record»
Hold
«enumeration»
Tier
VIP
PREMIUM
GENERAL_ADMISSION
«record»
Ticket
PaymentGateway
Notifier

Key flows



A numbered seat's state machine — every arrow is a single `compareAndSet`:



When inventory is released (expired hold, cancellation), `TicketingService` **offers it to the tier's waitlist first**: the next waiting fan is admitted directly and given a fresh hold, plus a notification. If they do not pay in time, the reaper releases it again and the next fan gets it.

Design decisions & patterns used

Decision	Why	Alternative rejected
Lock-free CAS per seat (`AtomicReference<SeatState>`)	Buyers of *different* seats never wait for each other; on the *same* seat, exactly one `compareAndSet` wins and the losers learn instantly. No thread is ever blocked holding a lock, so there is no deadlock.	One monitor per concert (15.2's approach) — correct, but at 50,000 simultaneous buyers every request queues behind one lock.
Immutable sealed `SeatState`	CAS compares *references* (which object it is, not what it contains); making each state a fresh immutable record means "the state I read" is exactly "the state I replace". Pattern-matching `switch` / `instanceof` on a sealed type is exhaustive and readable.	Mutable fields (`status`, `heldBy`, `expiry`) updated separately — a reader could see a half-updated seat.
Multi-seat hold in sorted order + rollback	Each seat is claimed independently; on failure we release the ones *we* took (matching our hold id). Sorted order makes two buyers of overlapping sets contend on the same first seat, so one of them fails fast.	Holding several seats with nested locks — deadlock-prone without ordering.
GA as a counter	Standing places are interchangeable — there's nothing to name. A CAS loop on an `AtomicInteger` (`if cur < qty fail; else CAS(cur, cur-qty)`) never oversells.	A `Seat` object per standing place — 40,000 objects for no benefit.
Virtual waiting room	Fairness and load control: arrival order decides who gets to *try*, and admitting in batches caps the load on inventory. Bots gain nothing by hammering "buy". `join` is `synchronized` so number order and queue order can never disagree.	"Whoever's request lands first" — rewards bots and fast connections, and melts the backend (the servers behind the app).
Per-user limit via `ConcurrentHashMap.compute` (a thread-safe map; `compute` updates one key atomically)	`compute` runs the check-and-update for one key atomically, so two parallel requests from the same user can't both slip under the limit.	`get` then `put` — a classic race.
Holds with TTL + a reaper	Seat holds are also reclaimed lazily (a CAS can overwrite an expired `Held`); GA holds have no per-item state, so a periodic `expireHolds()` returns their count. `holds.remove(id, h)` ensures the reaper and `purchase` never both process one hold.	Holds that never expire.
Waitlist offers on release	Released tickets go to the longest-waiting fan (FIFO), not to whoever refreshes fastest.	Returning tickets silently to the pool.
Compensation on lost hold	Same as 15.2: if a seat could not be confirmed after payment, release and refund.	Assuming the hold is still valid.
Singleton?	The *awesome-lld* reference solution makes the booking system a Singleton. We pass collaborators through the constructor instead (dependency injection): it is easier to test and allows one service per concert.	A global Singleton.

Java implementation

Package `lld.ch15.concert`. Files: `Model.java` (tiers, seat states, hold, ticket, interfaces, clock), `Inventory.java` (CAS seat inventory, GA pool, purchase limiter, waiting room), `TicketingService.java`, `Main.java`. The demos use Java 21 **virtual threads** (lightweight threads managed by the JVM rather than the operating system) (`Executors.newVirtualThreadPerTaskExecutor()`), which make 1,000 concurrent buyers cheap.

```java
// File: lld/ch15/concert/Model.java
package lld.ch15.concert;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.util.List;

/** Seated tiers have numbered seats; general admission (GA) is just a count of standing places. */
enum Tier {
    VIP(true, "5000.00"), PREMIUM(true, "2500.00"), GENERAL_ADMISSION(false, "800.00");
    final boolean seated;
    final BigDecimal price;                       // money: BigDecimal, never double
    Tier(boolean seated, String price) { this.seated = seated; this.price = new BigDecimal(price); }
    BigDecimal priceFor(int qty) { return price.multiply(BigDecimal.valueOf(qty)); }
}

record Concert(String id, String artist, String venue, LocalDateTime startsAt) {}

/** A seat's whole life in one immutable value, swapped atomically with compare-and-set (CAS). */
sealed interface SeatState permits Free, Held, Sold {}
record Free() implements SeatState {}
record Held(String holdId, String userId, Instant expiresAt) implements SeatState {
    boolean expired(Instant now) { return !now.isBefore(expiresAt); }
}
record Sold(String holdId, String userId) implements SeatState {}

/** A temporary claim on inventory: either specific seats or a GA quantity. */
record Hold(String id, String userId, Tier tier, List<String> seatIds, int quantity, BigDecimal amount, Instant
expiresAt) {
    boolean expired(Instant now) { return !now.isBefore(expiresAt); }
}

record Ticket(String id, String concertId, String userId, Tier tier, String seatOrGa) {}

final class SoldOutException extends RuntimeException {
    private static final long serialVersionUID = 1L;
    SoldOutException(String msg) { super(msg); }
}

interface PaymentGateway {
    boolean charge(String userId, BigDecimal amount);
    void refund(String userId, BigDecimal amount);
}

interface Notifier { void send(String userId, String message); }

final class SimClock extends Clock {
    private volatile Instant now;
    SimClock(Instant start) { this.now = start; }
    void advance(Duration d) { now = now.plus(d); }
    @Override public ZoneId getZone() { return ZoneOffset.UTC; }
    @Override public Clock withZone(ZoneId zone) { return this; }
    @Override public Instant instant() { return now; }
}
```

```java
// File: lld/ch15/concert/Inventory.java
package lld.ch15.concert;

import java.time.Instant;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.atomic.AtomicReference;

/**
 * Numbered seats, lock-free. Each seat is an AtomicReference<SeatState>; a claim is one CAS.
 * Many buyers on DIFFERENT seats never wait for each other; on the SAME seat exactly one CAS wins.
 */
final class SeatedInventory {
    private final Map<String, Tier> tierOf = new LinkedHashMap<>();
    private final Map<String, AtomicReference<SeatState>> seats = new ConcurrentHashMap<>();

    void addSeat(String id, Tier tier) {
        tierOf.put(id, tier);
        seats.put(id, new AtomicReference<>(new Free()));
    }
    Tier tierOf(String id) { return tierOf.get(id); }

    /** All-or-nothing over several seats: claim in sorted order, roll back our own claims on failure. */
    boolean tryHold(List<String> ids, String holdId, String userId, Instant now, Instant expiresAt) {
        List<String> taken = new ArrayList<>();
        for (String id : ids.stream().sorted().toList()) {
            if (!tryHoldOne(id, holdId, userId, now, expiresAt)) {
                taken.forEach(t -> release(t, holdId));
                return false;
            }
            taken.add(id);
        }
        return true;
    }

    private boolean tryHoldOne(String id, String holdId, String userId, Instant now, Instant expiresAt) {
        AtomicReference<SeatState> ref = seats.get(id);
        if (ref == null) throw new IllegalArgumentException("no seat " + id);
        while (true) {
            SeatState cur = ref.get();
            boolean claimable = cur instanceof Free || (cur instanceof Held h && h.expired(now));
            if (!claimable) return false;
            if (ref.compareAndSet(cur, new Held(holdId, userId, expiresAt))) return true;
            // CAS lost: someone changed the seat between get() and CAS. Re-read and decide again.
        }
    }

    /** Held-by-this-hold and not expired -> Sold. Fails if the hold was lost. */
    boolean confirm(String id, String holdId, Instant now) {
        AtomicReference<SeatState> ref = seats.get(id);
        SeatState cur = ref.get();
        return cur instanceof Held h && h.holdId().equals(holdId) && !h.expired(now)
                && ref.compareAndSet(cur, new Sold(holdId, h.userId()));
    }

    /** Give the seat back only if it is still ours (held or sold by this hold). */
    boolean release(String id, String holdId) {
        AtomicReference<SeatState> ref = seats.get(id);
        SeatState cur = ref.get();
        boolean ours = (cur instanceof Held h && h.holdId().equals(holdId))
                || (cur instanceof Sold s && s.holdId().equals(holdId));
```

```
        return ours && ref.compareAndSet(cur, new Free());
    }

    /** Seats of a tier a fan could claim right now (Free, or Held but expired), in seat order. */
    List<String> freeSeats(Tier tier, Instant now) {
        List<String> out = new ArrayList<>();
        tierOf.forEach((id, t) -> {
            SeatState s = seats.get(id).get();
            if (t == tier && (s instanceof Free || (s instanceof Held h && h.expired(now)))) out.add(id);
        });
        return out;
    }

    long soldCount() { return seats.values().stream().filter(r -> r.get() instanceof Sold).count(); }

    SeatState stateOf(String id) { return seats.get(id).get(); }
}

/** General admission: no seat numbers, just a counter. A CAS loop means it can never go below zero. */
final class GaPool {
    private final AtomicInteger available;
    GaPool(int capacity) { available = new AtomicInteger(capacity); }

    boolean tryTake(int qty) {
        while (true) {
            int cur = available.get();
            if (cur < qty) return false;
            if (available.compareAndSet(cur, cur - qty)) return true;
        }
    }
    void giveBack(int qty) { available.addAndGet(qty); }
    int available()        { return available.get(); }
}

/** Anti-scalping: at most `max` tickets per user per concert, checked-and-updated atomically. */
final class PurchaseLimiter {
    private final int max;
    private final ConcurrentHashMap<String, Integer> used = new ConcurrentHashMap<>();
    PurchaseLimiter(int max) { this.max = max; }

    boolean tryUse(String userId, int qty) {
        boolean[] ok = {false};
        used.compute(userId, (u, cur) -> {
            int c = cur == null ? 0 : cur;
            if (c + qty > max) return cur;            // unchanged
            ok[0] = true;
            return c + qty;
        });
        return ok[0];
    }
    void giveBack(String userId, int qty) { used.computeIfPresent(userId, (u, c) -> c - qty <= 0 ? null : c - qty);
}
}

/**
 * Virtual waiting room. Everyone who arrives gets a strictly increasing number; the system admits
 * people in that order, a batch at a time. First come, first served, and bots gain nothing by
 * hammering the buy button, because only admitted users may hold inventory.
 */
final class VirtualWaitingRoom {
    private final AtomicLong nextNumber = new AtomicLong();
    private final ConcurrentLinkedQueue<String> line = new ConcurrentLinkedQueue<>();
    private final Set<String> admitted = ConcurrentHashMap.newKeySet();
    private final Map<String, Long> numberOf = new ConcurrentHashMap<>();

    /** synchronized so the number order and the line order can never disagree. */
    synchronized long join(String userId) {
        Long n = numberOf.get(userId);
        if (n != null) return n;                      // joining twice keeps your place
```

```
        long mine = nextNumber.incrementAndGet();
        numberOf.put(userId, mine);
        line.add(userId);
        return mine;
    }
    synchronized List<String> admitNext(int k) {
        List<String> out = new ArrayList<>();
        for (int i = 0; i < k; i++) {
            String u = line.poll();
            if (u == null) break;
            admitted.add(u);
            out.add(u);
        }
        return out;
    }
    boolean isAdmitted(String userId) { return admitted.contains(userId); }
    void admitDirectly(String userId) { admitted.add(userId); }
}
```

```java
// File: lld/ch15/concert/TicketingService.java
package lld.ch15.concert;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * Flow: join waiting room -> get admitted -> HOLD (seats or GA qty, with TTL) -> PURCHASE.
 * Released inventory (expired hold, cancellation) is offered to the tier's waitlist first.
 */
final class TicketingService {
    private final Concert concert;
    private final SeatedInventory seated;
    private final GaPool ga;
    private final PurchaseLimiter limiter;
    private final VirtualWaitingRoom room;
    private final PaymentGateway payments;
    private final Notifier notifier;
    private final Clock clock;
    private final Duration holdTtl;
    private final Map<String, Hold> holds = new ConcurrentHashMap<>();           // active holds
    private final Map<String, List<Ticket>> orders = new ConcurrentHashMap<>();  // holdId -> tickets
    private final Map<Tier, Queue<String>> waitlists = new EnumMap<>(Tier.class);
    private final AtomicInteger seq = new AtomicInteger();

    TicketingService(Concert concert, SeatedInventory seated, GaPool ga, PurchaseLimiter limiter,
                     VirtualWaitingRoom room, PaymentGateway payments, Notifier notifier, Clock clock, Duration
holdTtl) {
        this.concert = concert; this.seated = seated; this.ga = ga; this.limiter = limiter; this.room = room;
        this.payments = payments; this.notifier = notifier; this.clock = clock; this.holdTtl = holdTtl;
        for (Tier t : Tier.values()) waitlists.put(t, new ConcurrentLinkedQueue<>());
    }

    // ---------------------------------------------------------------- hold
    Hold holdSeats(String userId, List<String> seatIds) {
        requireAdmitted(userId);
        if (seatIds.isEmpty()) throw new IllegalArgumentException("no seats");
        Tier tier = seated.tierOf(seatIds.get(0));
        if (seatIds.stream().anyMatch(id -> seated.tierOf(id) != tier))
            throw new IllegalArgumentException("one hold = one tier");
        if (!limiter.tryUse(userId, seatIds.size()))
            throw new IllegalStateException(userId + " would exceed the per-user ticket limit");
        Instant now = clock.instant();
        Hold h = new Hold("H" + seq.incrementAndGet(), userId, tier, List.copyOf(seatIds), seatIds.size(),
                tier.priceFor(seatIds.size()), now.plus(holdTtl));
        if (!seated.tryHold(seatIds, h.id(), userId, now, h.expiresAt())) {
            limiter.giveBack(userId, seatIds.size());
            throw new SoldOutException(seatIds + " not available");
        }
        holds.put(h.id(), h);
        return h;
    }

    Hold holdGa(String userId, int qty) {
        requireAdmitted(userId);
        if (!limiter.tryUse(userId, qty))
            throw new IllegalStateException(userId + " would exceed the per-user ticket limit");
        if (!ga.tryTake(qty)) {
```

```
            limiter.giveBack(userId, qty);
            throw new SoldOutException("general admission sold out");
        }
        Hold h = new Hold("H" + seq.incrementAndGet(), userId, Tier.GENERAL_ADMISSION, List.of(), qty,
                Tier.GENERAL_ADMISSION.priceFor(qty), clock.instant().plus(holdTtl));
        holds.put(h.id(), h);
        return h;
    }

    // ------------------------------------------------------------------ purchase
    List<Ticket> purchase(String holdId) {
        Hold h = holds.remove(holdId);                    // atomic claim: a hold is purchased at most once
        if (h == null) throw new IllegalStateException(holdId + " unknown, expired or already used");
        if (h.expired(clock.instant())) { releaseInventory(h); throw new IllegalStateException(holdId + "
expired"); }
        if (!payments.charge(h.userId(), h.amount())) { releaseInventory(h); throw new
IllegalStateException("payment declined"); }

        List<Ticket> tickets = new ArrayList<>();
        if (h.tier().seated) {
            Instant now = clock.instant();
            List<String> confirmed = new ArrayList<>();
            for (String id : h.seatIds()) {
                if (!seated.confirm(id, h.id(), now)) {   // hold lost during payment -> compensate
                    confirmed.forEach(c -> seated.release(c, h.id()));
                    h.seatIds().forEach(s -> seated.release(s, h.id()));
                    limiter.giveBack(h.userId(), h.quantity());
                    payments.refund(h.userId(), h.amount());
                    throw new IllegalStateException("seat " + id + " lost during payment, refunded");
                }
                confirmed.add(id);
                tickets.add(new Ticket("T" + seq.incrementAndGet(), concert.id(), h.userId(), h.tier(), id));
            }
        } else {
            for (int i = 0; i < h.quantity(); i++)
                tickets.add(new Ticket("T" + seq.incrementAndGet(), concert.id(), h.userId(), h.tier(), "GA"));
        }
        orders.put(h.id(), List.copyOf(tickets));
        return tickets;
    }

    // ------------------------------------------------------------------ release paths
    /** The reaper: run every few seconds in production. Returns how many holds it expired. */
    int expireHolds() {
        Instant now = clock.instant();
        int n = 0;
        for (Hold h : holds.values())
            if (h.expired(now) && holds.remove(h.id(), h)) { releaseInventory(h); n++; }
        return n;
    }

    /** Cancel a whole order (all tickets bought with one hold). */
    void cancelOrder(String holdId) {
        List<Ticket> tickets = orders.remove(holdId);
        if (tickets == null) throw new IllegalStateException("no order " + holdId);
        Tier tier = tickets.get(0).tier();
        BigDecimal refund = tier.priceFor(tickets.size());
        payments.refund(tickets.get(0).userId(), refund);
        Hold asIfHeld = new Hold(holdId, tickets.get(0).userId(), tier,
                tier.seated ? tickets.stream().map(Ticket::seatOrGa).toList() : List.of(), tickets.size(), refund,
clock.instant());
        releaseInventory(asIfHeld);
    }

    void joinWaitlist(String userId, Tier tier) { waitlists.get(tier).add(userId); }

    private void releaseInventory(Hold h) {
        limiter.giveBack(h.userId(), h.quantity());
        if (h.tier().seated) {
```

```
            h.seatIds().forEach(id -> seated.release(id, h.id()));
            h.seatIds().forEach(id -> offerSeat(h.tier(), id));
        } else {
            ga.giveBack(h.quantity());
            offerGa();
        }
    }

    /** Waitlist first: freed inventory goes to the person who has waited longest, as a fresh hold. */
    private void offerGa() {
        Queue<String> q = waitlists.get(Tier.GENERAL_ADMISSION);
        String next;
        while (ga.available() > 0 && (next = q.poll()) != null) {
            room.admitDirectly(next);
            try {
                Hold h = holdGa(next, 1);
                notifier.send(next, "a GA ticket freed up, held for you as " + h.id() + " until " + h.expiresAt());
            } catch (RuntimeException e) { /* over their limit or lost the race: try the next person */ }
        }
    }

    private void offerSeat(Tier tier, String seatId) {
        String next = waitlists.get(tier).poll();
        if (next == null) return;
        room.admitDirectly(next);
        try {
            Hold h = holdSeats(next, List.of(seatId));
            notifier.send(next, "seat " + seatId + " freed up, held for you as " + h.id());
        } catch (RuntimeException e) { /* someone else took it first */ }
    }

    private void requireAdmitted(String userId) {
        if (!room.isAdmitted(userId)) throw new IllegalStateException(userId + " is still in the waiting room");
    }

    int gaAvailable() { return ga.available(); }

    /** What a fan sees before choosing: free places per tier (requirement: view seating and availability). */
    String availability() {
        StringBuilder sb = new StringBuilder();
        for (Tier t : Tier.values()) {
            if (t.seated) sb.append(t).append(" free ").append(seated.freeSeats(t, clock.instant())).append(", ");
            else sb.append(t).append(" free ").append(ga.available());
        }
        return sb.toString();
    }
}
```

```java
// File: lld/ch15/concert/Main.java
package lld.ch15.concert;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDateTime;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;

public class Main {
    public static void main(String[] args) throws Exception {
        SimClock clock = new SimClock(Instant.parse("2026-06-01T09:00:00Z"));
        AtomicReference<BigDecimal> collected = new AtomicReference<>(BigDecimal.ZERO);
        PaymentGateway bank = new PaymentGateway() {
            public boolean charge(String u, BigDecimal a) { collected.accumulateAndGet(a, BigDecimal::add); return
true; }
            public void refund(String u, BigDecimal a)    { collected.accumulateAndGet(a, BigDecimal::subtract); }
        };
        Notifier notifier = (u, m) -> System.out.println("  [notify " + u + "] " + m);

        Concert show = new Concert("C1", "The Band", "Stadium", LocalDateTime.of(2026, 8, 15, 19, 30));
        SeatedInventory seated = new SeatedInventory();
        for (int i = 1; i <= 4; i++) seated.addSeat("VIP-" + i, Tier.VIP);
        for (int i = 1; i <= 6; i++) seated.addSeat("P-" + i, Tier.PREMIUM);
        GaPool ga = new GaPool(100);
        VirtualWaitingRoom room = new VirtualWaitingRoom();
        TicketingService svc = new TicketingService(show, seated, ga, new PurchaseLimiter(4), room,
                bank, notifier, clock, Duration.ofMinutes(10));

        System.out.println("== 1. Seating and availability, then the virtual waiting room ==");
        System.out.println("  availability: " + svc.availability());
        for (String u : List.of("ana", "ben", "cai", "dev", "eli")) System.out.println("  " + u + " got number " +
room.join(u));
        System.out.println("  ben joins again, keeps number " + room.join("ben"));
        System.out.println("  admitted batch 1: " + room.admitNext(3));
        try { svc.holdGa("eli", 1); } catch (IllegalStateException e) { System.out.println("  REFUSED: " +
e.getMessage()); }

        System.out.println("\n== 2. 40 admitted fans CAS for the same seat VIP-1 ==");
        AtomicInteger vipWins = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 40; i++) {
                String u = "vip-fan-" + i;
                room.admitDirectly(u);
                pool.submit(() -> {
                    go.await();
                    try { svc.holdSeats(u, List.of("VIP-1")); vipWins.incrementAndGet(); }
                    catch (SoldOutException ignored) { }
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("  holds on VIP-1 = " + vipWins.get());

        System.out.println("\n== 3. Flash sale: 1,000 fans, 100 GA places (98 left after test buys) ==");
        Hold anaHold = svc.holdGa("ana", 2);
        svc.purchase(anaHold.id());
        AtomicInteger sold = new AtomicInteger(), soldOut = new AtomicInteger();
        CountDownLatch go2 = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 1000; i++) {
```

```
                String u = "fan-" + i;
                room.admitDirectly(u);
                pool.submit(() -> {
                    go2.await();
                    try { svc.purchase(svc.holdGa(u, 1).id()); sold.incrementAndGet(); }
                    catch (SoldOutException e) { soldOut.incrementAndGet(); }
                    return null;
                });
            }
            go2.countDown();
        }
        System.out.println("  sold=" + sold + " soldOut=" + soldOut + " gaLeft=" + svc.gaAvailable() + " (never
negative)");

        System.out.println("\n== 4. Per-user limit (4) stops a scalper ==");
        Hold p = svc.holdSeats("ben", List.of("P-1", "P-2", "P-3"));
        System.out.println("  ben holds " + p.seatIds() + " = " + p.amount());
        try { svc.holdSeats("ben", List.of("P-4", "P-5")); } catch (IllegalStateException e) { System.out.println("
REFUSED: " + e.getMessage()); }
        List<Ticket> benTickets = svc.purchase(p.id());
        System.out.println("  ben bought " + benTickets.stream().map(Ticket::seatOrGa).toList());

        System.out.println("\n== 5. Sold out -> waitlist -> a cancellation is offered to the waitlist ==");
        try { svc.holdGa("cai", 1); } catch (SoldOutException e) { System.out.println("  cai: " + e.getMessage() +
", joins waitlist"); }
        svc.joinWaitlist("cai", Tier.GENERAL_ADMISSION);
        svc.cancelOrder(anaHold.id());                  // ana returns her 2 GA tickets
        System.out.println("  GA left after offers = " + svc.gaAvailable());

        System.out.println("\n== 6. Unpaid holds expire; the reaper returns them ==");
        System.out.println("  admitted batch 2: " + room.admitNext(3));
        Hold forgotten = svc.holdSeats("dev", List.of("P-6"));
        clock.advance(Duration.ofMinutes(11));
        System.out.println("  expired holds reaped = " + svc.expireHolds() + " (VIP-1, cai's unpaid offer, dev's P-
6)");
        try { svc.purchase(forgotten.id()); } catch (IllegalStateException e) { System.out.println("  dev: " +
e.getMessage()); }
        System.out.println("  P-6 now " + seated.stateOf("P-6") + ", GA left " + svc.gaAvailable());
        System.out.println("  availability: " + svc.availability());
        System.out.println("  money collected = " + collected.get());
    }
}
```

Output:

```
== 1. Seating and availability, then the virtual waiting room ==
  availability: VIP free [VIP-1, VIP-2, VIP-3, VIP-4], PREMIUM free [P-1, P-2, P-3, P-4, P-5, P-6],
GENERAL_ADMISSION free 100
  ana got number 1
  ben got number 2
  cai got number 3
  dev got number 4
  eli got number 5
  ben joins again, keeps number 2
  admitted batch 1: [ana, ben, cai]
  REFUSED: eli is still in the waiting room

== 2. 40 admitted fans CAS for the same seat VIP-1 ==
  holds on VIP-1 = 1

== 3. Flash sale: 1,000 fans, 100 GA places (98 left after test buys) ==
  sold=98 soldOut=902 gaLeft=0 (never negative)

== 4. Per-user limit (4) stops a scalper ==
  ben holds [P-1, P-2, P-3] = 7500.00
  REFUSED: ben would exceed the per-user ticket limit
  ben bought [P-1, P-2, P-3]

== 5. Sold out -> waitlist -> a cancellation is offered to the waitlist ==
  cai: general admission sold out, joins waitlist
  [notify cai] a GA ticket freed up, held for you as H244 until 2026-06-01T09:10:00Z
  GA left after offers = 1

== 6. Unpaid holds expire; the reaper returns them ==
  admitted batch 2: [dev, eli]
  expired holds reaped = 3 (VIP-1, cai's unpaid offer, dev's P-6)
  dev: H245 unknown, expired or already used
  P-6 now Free[], GA left 2
  availability: VIP free [VIP-1, VIP-2, VIP-3, VIP-4], PREMIUM free [P-4, P-5, P-6], GENERAL_ADMISSION free 2
  money collected = 85900.00
```

What to notice: section 1 admits the first three in arrival order and refuses someone still in the queue; section 2 — 40 concurrent CAS attempts on one seat, exactly one hold; section 3 — 1,000 buyers for 98 remaining GA places sell exactly 98 and the counter stops at 0; section 4 — a fifth ticket is refused by the per-user limit; section 5 — a cancellation's released ticket goes straight to the waitlisted fan as a hold; section 6 — the reaper returns three expired holds (VIP-1, the unpaid waitlist offer, and dev's seat) and a late purchase is refused. The availability line at the end shows what is still for sale (three PREMIUM seats, four VIP seats, two GA places). Money collected is 85,900.00 = 98 × 800 + 3 × 2,500 (ana's 2 GA tickets were refunded).

Concurrency & edge cases

- **Why CAS is correct.** `compareAndSet(expected, new)` atomically replaces the value only if it is still the exact object we read. If another thread changed it in between, the CAS fails and we re-read and decide again. The loop in `tryHoldOne` only retries while the seat is still claimable; otherwise it returns `false`.
- **ABA problem?** ABA is when a value goes A → B → A and a CAS wrongly succeeds. Here every transition creates a *new* record object and `AtomicReference` compares references, so an "A" that came back is a different object and the CAS fails safely.
- **Hold claimed exactly once.** `purchase` starts with `holds.remove(holdId)`; the reaper uses `holds.remove(id, h)`. Only one of them can get the hold, so inventory is never released twice.
- **Per-user limit and failures.** Every failure path (sold out, declined, expired, lost seat) gives the limit back, so a user is never locked out by a failed attempt.
- **Lost hold after payment.** `confirm` fails if the hold expired and someone CASed over it; we release, give back the limit, and refund.
- **Fair queue scope.** The in-memory queue is fair *within one process*. At real scale the waiting room is a separate service (often at the CDN edge — a CDN, content delivery network, is a worldwide set of servers placed close to users; see System Design Fundamentals) that hands out signed, time-limited access tokens (short strings the server can verify it issued); inventory servers only check the token.

- **Across servers.** Seat CAS maps naturally to a database conditional update (`UPDATE seat SET state='HELD', hold_id=?, expires_at=? WHERE id=? AND (state='FREE' OR expires_at < now())`) or Redis `SET NX PX`. GA counts map to Redis `DECRBY` (decrement by an amount) guarded by a Lua script (a small script Redis runs as one atomic step), or to a database row updated with `WHERE available >= ?`.

Extensions & follow-up questions

- **"Best available seats."** Instead of the user choosing, a `SeatAllocator` strategy picks the best adjacent free seats in a tier, then calls the same CAS hold.
- **"Verified fans / presale codes."** Admission to the waiting room requires a token issued to pre-registered fans; the limiter keys on a verified identity, not an email.
- **"Dynamic pricing."** Prices per tier come from a `PricingStrategy` that looks at remaining inventory and time — decided when the hold is created and frozen in the `Hold`.
- **"Resale marketplace."** A sold ticket can be listed; a buyer's purchase transfers ownership and re-issues a new barcode (so the old one stops working).
- **"Waitlist for seated tiers."** Implemented the same way for each released seat (`offerSeat`).

Real-world example

Ticketmaster uses a virtual queue ("Smart Queue") for high-demand on-sales and a **Verified Fan** presale programme that registers fans in advance to filter out bots; purchase limits per account are standard on its events. The November 2022 presale for Taylor Swift's *Eras Tour* is the well-known stress case: demand far beyond what was planned for overwhelmed the system, many fans waited for hours or were dropped, and the general public sale was cancelled — a real lesson that fairness and admission control are as important as seat-locking correctness.

In the interview

30–45 minute plan. (1) Say up front: "the entities are like movie booking; the hard part is contention and fairness." (2) Tiers: seated vs GA, and why GA is a counter. (3) Holds with TTL; code the CAS seat claim and the GA CAS loop. (4) Fairness: waiting room with batches; per-user limit with atomic `compute`. (5) Waitlist on release; reaper. (6) Scale-out: tokens from the waiting room, conditional updates/Redis.

> **30-second answer:** "Seated tiers are an `AtomicReference<SeatState>` per seat, claimed by compare-and-set — one winner, no locks. GA is an `AtomicInteger` decremented by a CAS loop so it can't go negative. Everyone enters through a virtual waiting room that admits in arrival order, and a per-user limit is enforced atomically. Holds expire after 10 minutes; a reaper returns expired GA holds; released tickets go to a FIFO waitlist first. If a hold is lost during payment we refund."

Follow-ups with answers: *Lock-free vs locks?* Lock-free wins when contention is high and critical sections are tiny (one field swap); locks are simpler when an operation must update many things together. *What if the queue service fails?* Fail closed (on failure, deny rather than allow): pause admissions, don't open the floodgates. *How do you stop one person with 50 accounts?* Verified identity, payment-card fingerprinting (recognising the same card across accounts), and limits keyed on those.

Self-check

- **Q:** Why is GA modeled as a counter, not seats? **A:** Standing places are interchangeable; a counter with CAS is simpler and can't oversell.
- **Q:** What does `compareAndSet` guarantee? **A:** The update happens only if the value is still the one you read; otherwise it fails and you retry or give up.
- **Q:** Why must `join` be synchronized? **A:** So the number handed out and the position in the line are assigned together; otherwise two joiners could get numbers in one order and queue slots in the other.
- **Q:** How is a hold prevented from being purchased *and* reaped? **A:** Both paths remove it from the `holds` map first; only one removal succeeds.
- **Q:** What happens to a released ticket when fans are waitlisted? **A:** It becomes a fresh hold for the first fan in the waitlist, who is notified.

TL;DR

Same nouns as movie booking, different enemy: contention. CAS per seat, CAS counter for GA, a FIFO virtual waiting room for fairness, atomic per-user limits, TTL holds with a reaper, a waitlist that gets released tickets first, and refunds when a hold is lost.

15.4 Design a Hotel Management System

Figure — Cover image for the hotel management case study (course diagram)

Problem & clarifying questions

A **Hotel Management System** (often called a *property management system*, PMS) runs a hotel's operations in one place: it knows every room, takes bookings, checks guests in and out, issues key cards, records housekeeping and room service, and produces the bill. Management gets one portal to track available rooms, book them and generate invoices.

The inventory here is different from a cinema: a room is not sold once, it is sold **night by night**. That single fact shapes the whole design.

Question	Answer we assume
Several hotel locations?	Yes — a `Hotel` has many `HotelLocation`s, each with its own rooms.
Room types?	`STANDARD`, `DELUXE`, `FAMILY_SUITE`, `BUSINESS_SUITE`.
What does a guest search by?	Room type and **date range** (availability).
Cancellation policy?	**Full refund if cancelled more than 24 hours before check-in**; otherwise no refund (our assumption for the "otherwise" case).
Extra charges during the stay?	Room service, food (kitchen) and amenities are added to the bill.
How is the bill paid?	**Credit card, check or cash**.
Keys?	Electronic key cards with a barcode and key id; deactivated at check-out.
Housekeeping?	Every task is logged per room (a history, never overwritten).
One booking for several rooms?	No — one booking = one room (a known limitation; see follow-ups).

Requirements

Functional (course list)

1. Support booking different **room types**: standard, deluxe, family suite (and business suite).

2. Guests **search** room inventory and book any **available** room.
3. Retrieve **who booked a room** and **which rooms a customer booked**.
4. Customers can **cancel**; **full refund if more than 24 hours before check-in**.
5. **Notify** the guest shortly before check-in and shortly before check-out.
6. Keep a **housekeeping log** per room.
7. Customers can add **room services and food items**.
8. Customers can ask for **amenities**.
9. Pay bills by **credit card, check or cash**.

Non-functional — never double-book a room-night (even with concurrent bookings from the website and the front desk); the bill must always equal the sum of its lines; each actor may do only their own job; the refund rule must be changeable in one place.

Use cases

Actors (course): **Guest** (search, book), **Receptionist** (add/modify rooms, create bookings, check in/out), **System** (booking/cancellation/reminder notifications), **Manager** (adds new workers), **Housekeeper** (adds/modifies housekeeping records), **Server** (adds/modifies room-service records).

Top use cases: add/remove/edit room; search room (by type and availability); register or cancel an account; book room; **check in**; **check out** (end of booking, return of keys); add room charge; update housekeeping log.

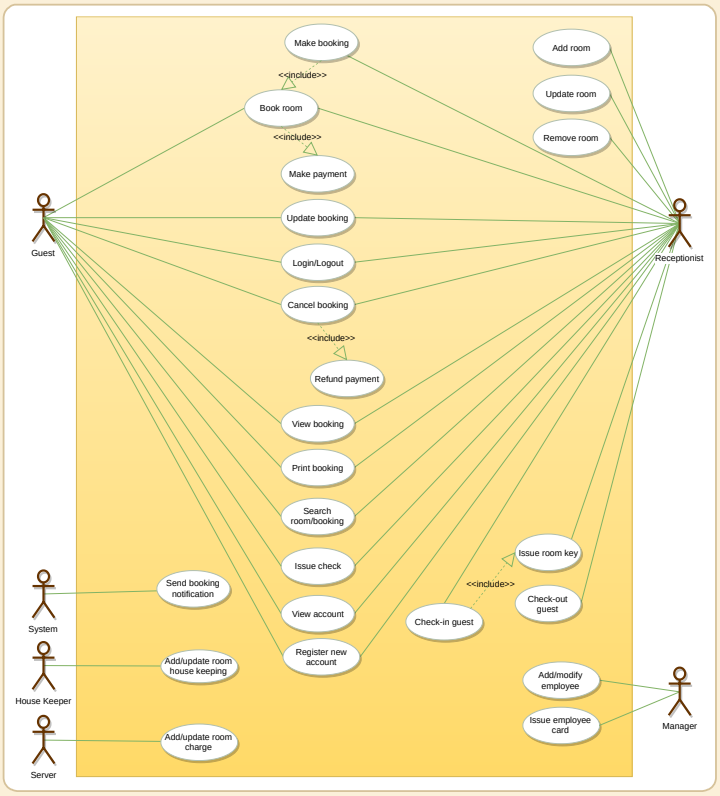


Figure — Hotel use cases: Guest and Receptionist share booking, update, cancel, view, print and search; "Make booking" includes "Book room", which includes "Make payment"; "Cancel booking" includes "Refund payment"; the receptionist's "Check-in guest" includes "Issue room key"; the System sends booking notifications; the House Keeper adds/updates housekeeping; a manager adds/modifies employees (course diagram)

Core entities

Class	Responsibility
`Hotel`	Owns its locations; searches each one in turn.
`HotelLocation` implements `Search`	Owns its rooms; answers "which rooms of this type are free for these dates?".
`Room`	Number, style, price per night; the **set of booked nights**; owns its `RoomKey`s and housekeeping log; current `RoomStatus`.
`RoomStyle`, `RoomStatus` (enums)	Type of room; what it is doing *now* (`AVAILABLE`, `RESERVED`, `OCCUPIED`, `NOT_AVAILABLE`, `BEING_SERVICED`, `OTHER`).
`RoomKey`	Key id, barcode, issue time, master flag, active flag.
`RoomHouseKeeping` (record)	One log line: when, who, what, how long.
`RoomBooking`	Reservation number, guest id, room, check-in/out dates, status, deposit; **owns its** `Invoice`; `refundDue()` holds the cancellation rule.
`BookingStatus` (enum)	`REQUESTED`, `PENDING`, `CONFIRMED`, `CHECKED_IN`, `CHECKED_OUT`, `CANCELLED`, `ABANDONED`.
`Invoice`, `InvoiceItem`	A list of lines; the total is computed from them.
`RoomCharge` (abstract) → `Amenity`, `RoomService`, `KitchenService`	A charge that knows how to write itself onto the invoice (Template Method: the parent fixes the steps, subclasses fill in one).
`Payment` (sealed, abstract) → `CreditCardTransaction`, `CheckTransaction`, `CashTransaction`	Three ways to pay with genuinely different behaviour.
`Person` (sealed) → `Guest`, `Receptionist`, `Server`, `Housekeeper`, `Manager`	Actors; service methods accept only the right actor type.
`FrontDesk`	Every flow of a stay: book, check in, add charge, check out, cancel, housekeeping, reminders, queries.
`Notifier`	Messages to guests.

Class diagram

The course's five stages:

Stage	Adds	What works afterwards
1	Invoice, payment types, notification types	A bill can be produced and settled by card, check or cash
2	Room, key card, housekeeping record	Each room can tell which of its nights are sold
3	Hotel, location, search interface	Free rooms can be listed for any date range
4	Booking and room charges	A stay can be followed from reservation to check-out
5	People and accounts	Every actor is limited to its own tasks

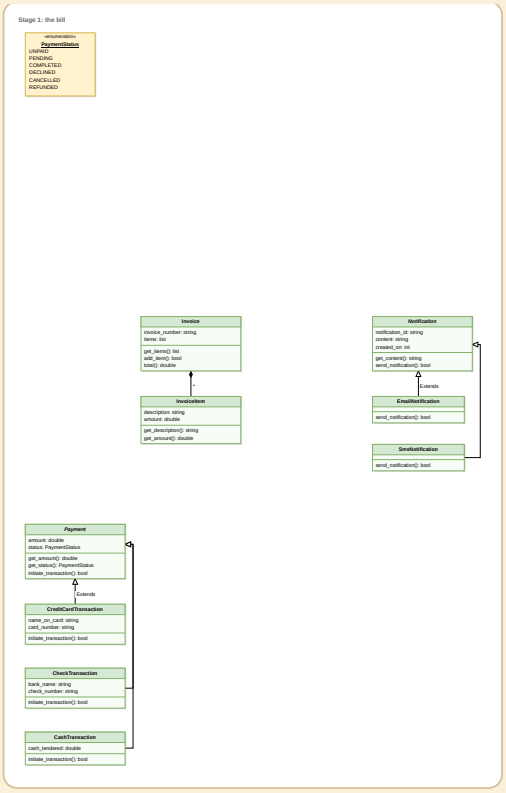


Figure — Stage 1: `Invoice` owns its `InvoiceItem`s (composition) and computes `total()`; abstract `Payment` with credit card, check and cash subclasses; abstract `Notification` with one subclass per channel (course diagram)

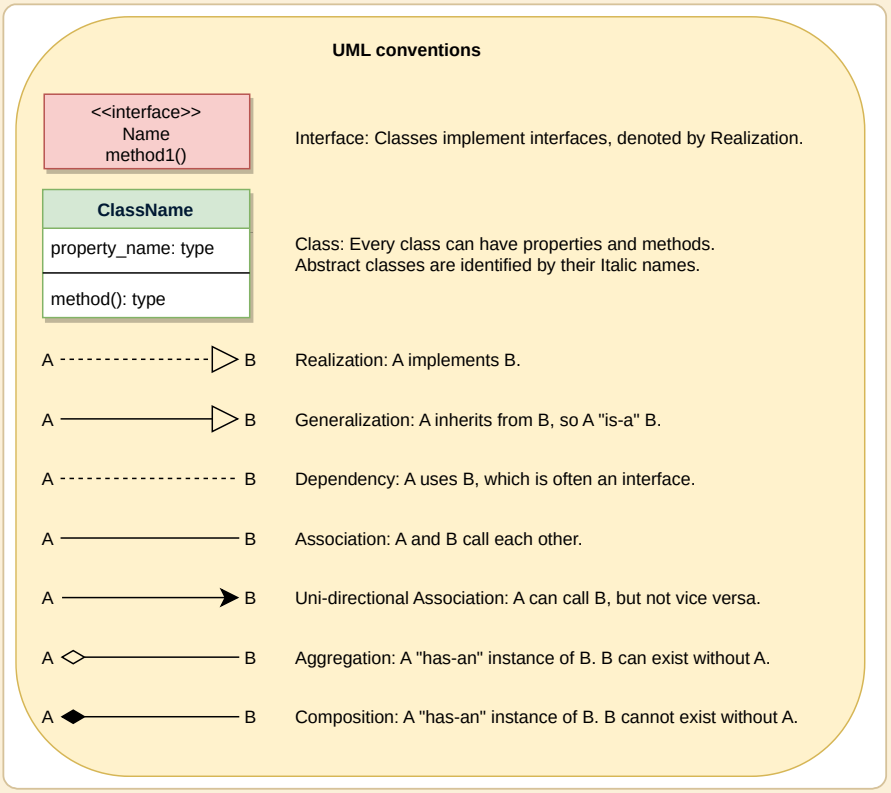


Figure — How to read a class box and the arrow styles used in these diagrams (course diagram)

Requirements 7 and 8 each put extra charges on a stay. That is why the invoice stores **individual lines** and computes its total on demand: a total that is always recomputed can never go out of step with the lines.

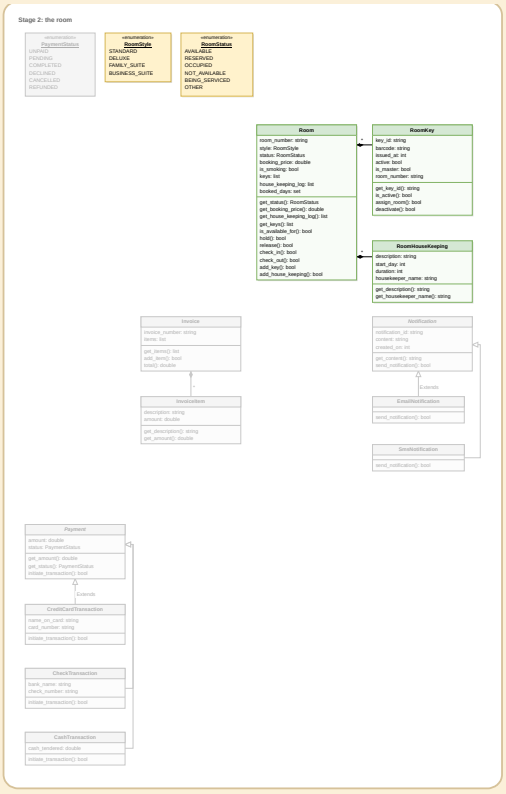


Figure — Stage 2: `Room` (number, style, status, price, and the set of nights it is committed to) owns its `RoomKey`s and its list of `RoomHouseKeeping` records (course diagram)

The key decision of the whole design sits here: **a status field is not availability.** `RoomStatus.OCCUPIED` describes the present moment only. A guest, however, asks about the future ("can I have it from the 10th to the 13th?"), and a single status value cannot answer that. So each room also stores the dates of the nights it has already sold.

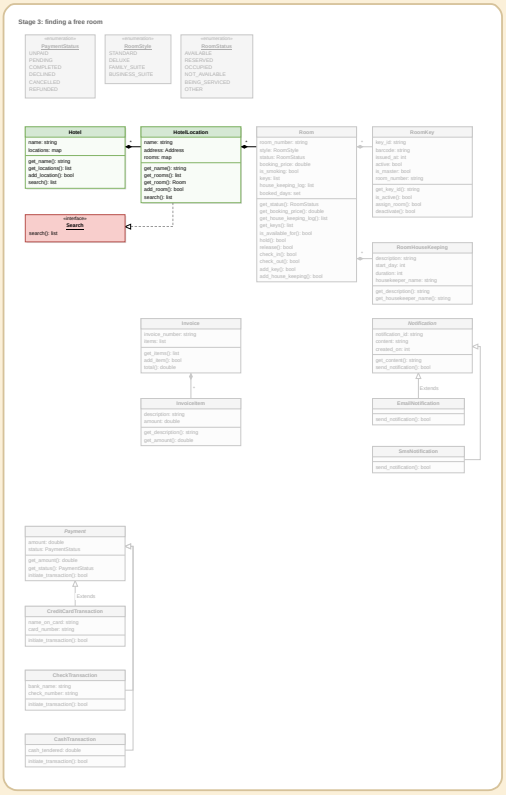


Figure — Stage 3: the `Search` interface implemented by `HotelLocation`, which owns its rooms; `Hotel` owns its locations (course diagram)

Search sits on the **location**, not on `Room`: one room has no knowledge of the other rooms, so it cannot list which of them are free. Putting `search` on `Room` is a frequent beginner mistake.

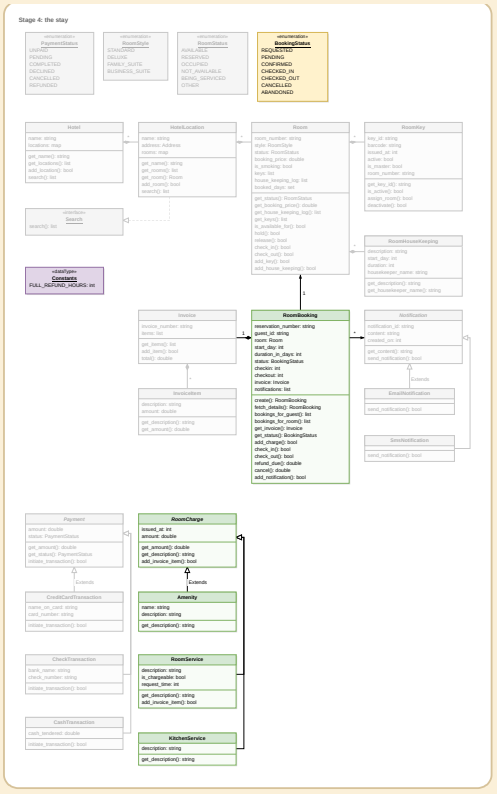


Figure — Stage 4: `RoomBooking` owns its invoice and points at its room (plain association); `RoomCharge` with `Amenity`, `RoomService` and `KitchenService` subclasses each writing an invoice line (course diagram)

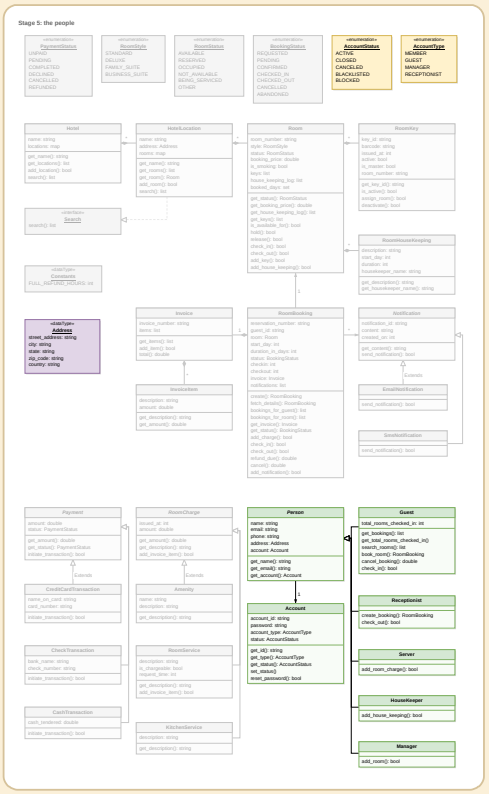


Figure — Stage 5: `Person` with `Guest`, `Receptionist`, `Server`, `HouseKeeper` and `Manager`, each with only its own methods; `Person` refers to its `Account`; there is deliberately no arrow from guest to booking (course diagram)

Notice what the figure does *not* have: an arrow between guest and booking. The booking stores only the guest's **id**, and queries filter bookings by that id — requirement 3 is met without either class holding a reference to the other.

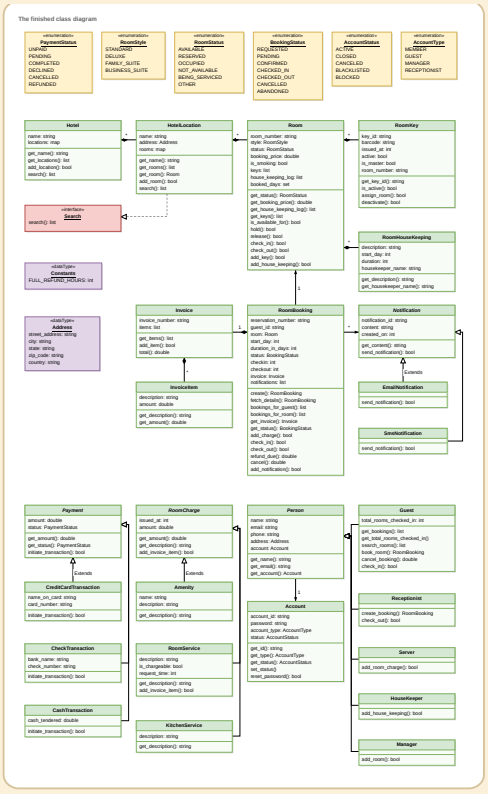
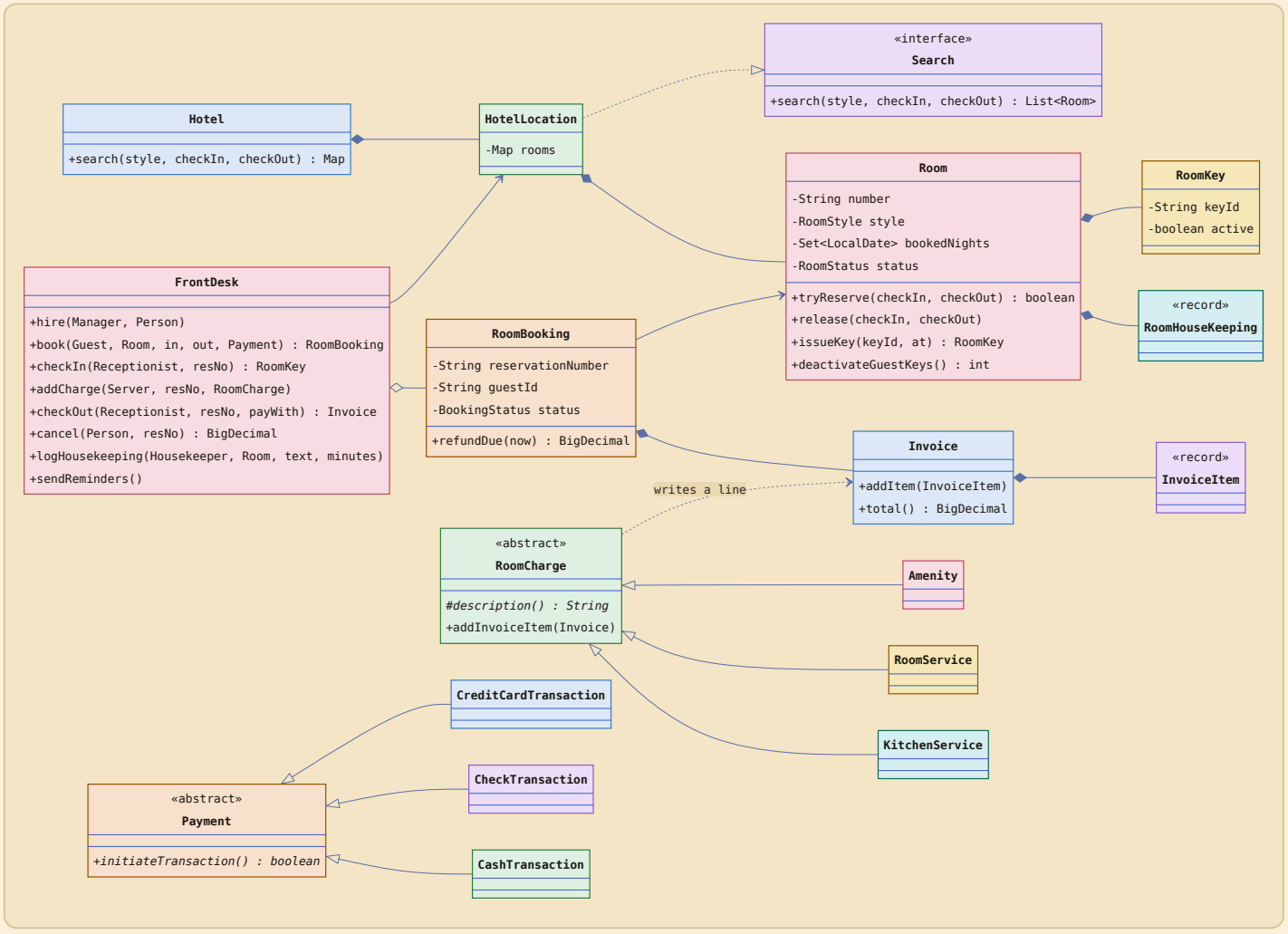


Figure — The finished hotel class diagram (course diagram)

Our Java design:



Key flows

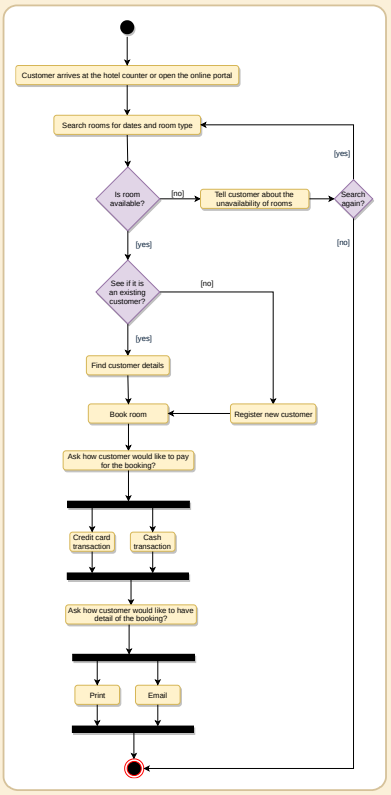


Figure — Make a room booking: search rooms for dates and room type; if none is available tell the customer; otherwise find or register the customer, book the room, ask how they will pay (credit card or cash), and how they want the booking details (course diagram)

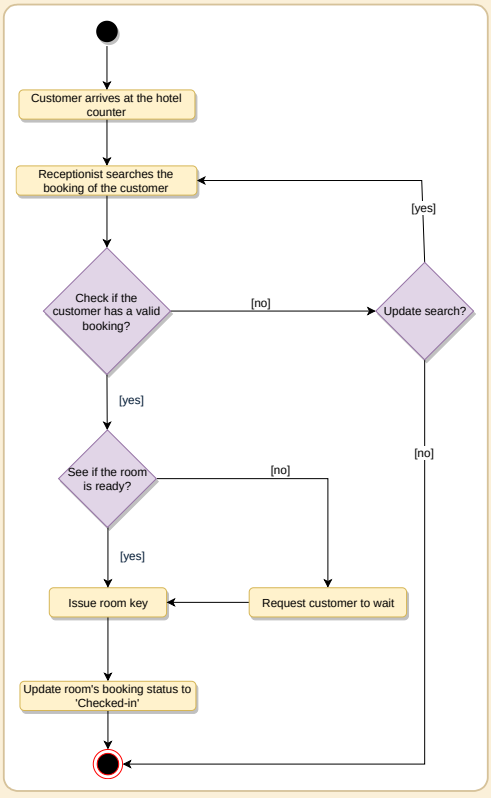


Figure — Check in: the customer arrives at the counter and the receptionist looks up the booking (searching again if it is not found); with a valid booking, check whether the room is ready — if not, the customer is asked to wait; if so, issue the room key and set the booking status to "Checked-in" (course diagram)

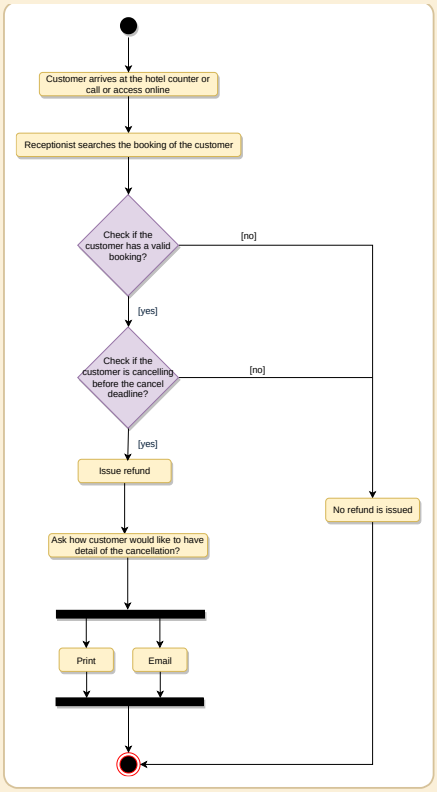
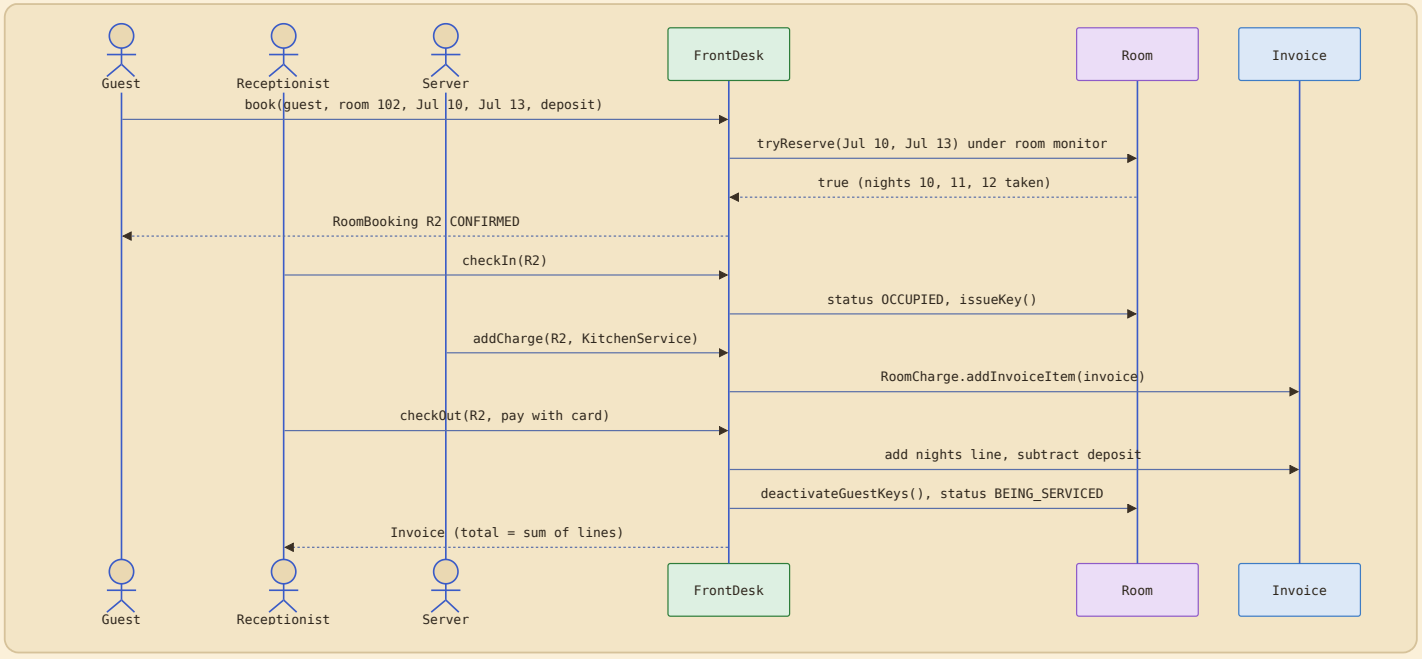
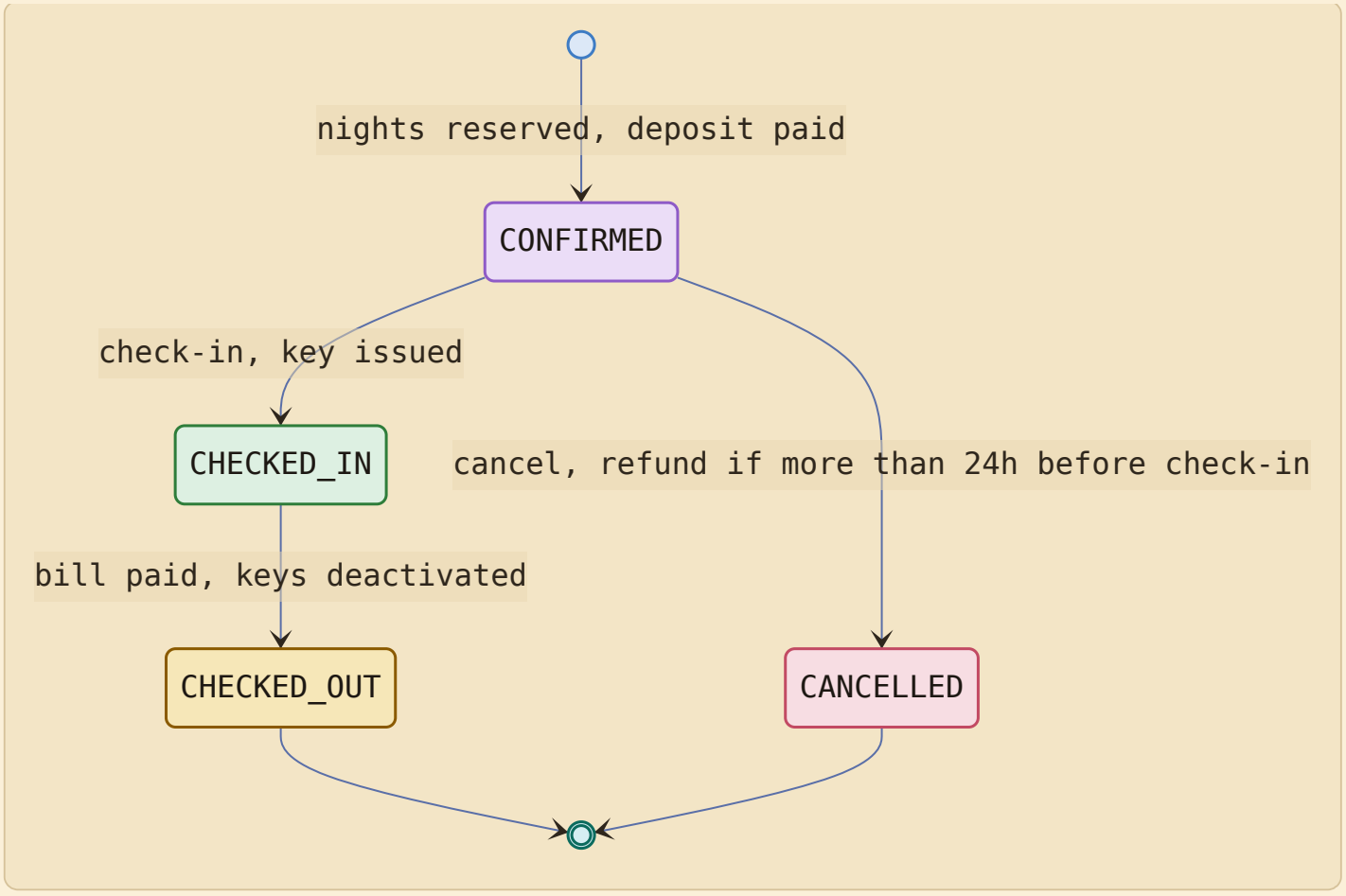


Figure — Cancel a booking: the customer comes to the counter, calls or goes online; the receptionist finds the booking and checks it is valid; if the customer is cancelling before the cancel deadline a refund is issued, otherwise no refund; the cancellation details are printed or emailed (course diagram)

A whole stay as a sequence:



Booking lifecycle:



The room's *current* status cycles independently: `AVAILABLE` → `OCCUPIED` (check-in) → `BEING_SERVICED` (check-out) → `AVAILABLE` (housekeeping logged).

Design decisions & patterns used

Decision	Why	Alternative rejected
Availability = set of booked nights (course's key point)	Answers date-range questions; nights are half-open `[checkIn, checkOut)`, so a guest leaving on the 13th and one arriving on the 13th don't clash.	Using `RoomStatus` for availability — only knows *now*.
`tryReserve` is `synchronized` on the room	"Are all these nights free? → take them" is check-then-act; the room's monitor makes it atomic. Rooms don't block each other.	One hotel-wide lock.
Search on `HotelLocation`	A room can't answer "which rooms are free?".	`Room.search()`.
Invoice = list of lines, total computed	The total can never disagree with the lines (requirements 7 and 8).	A stored running total.
Template Method: `RoomCharge.addInvoiceItem`	The parent fixes the algorithm (describe → price → write a line) and is `final`; subclasses fill in only `description()`. A new kind of charge = a new subclass, no edit to `Invoice` (Open/Closed). The course calls this the clearest Template Method example in its catalogue. Compare with Strategy, where the *whole* method is swapped rather than one step.	A `switch` on charge type inside the invoice.
Three `Payment` subclasses	They really behave differently: a card completes immediately, a **check stays `PENDING`** until it clears, cash can be declined if the amount tendered is short. Each passes the substitution test (each can be asked to take money).	One `Payment` with a `type` field and `if`s.
Refund rule in one method (`refundDue`)	When the policy changes there is exactly one place to edit. To support several policies (non-refundable rates, partial refunds), promote it to a `CancellationPolicy` strategy as in 15.2.	Refund arithmetic scattered through cancel flows.
Actor types in service signatures	`addCharge(Server, …)`, `logHousekeeping(Housekeeper, …)`, `addRoom(Manager, …)` — the compiler enforces who can do what. The receptionist booking for a guest calls the *same* booking method — two actors, one behaviour, no duplication.	Every actor able to call everything.
Booking stores guest id, not a `Guest` reference	No two-way dependency; "which rooms did this guest book?" is a query.	Bidirectional links that must be kept in sync.
Nights released on failed deposit / cancel	Reserve-then-pay needs an undo: if the deposit fails the nights are given back immediately.	Leaving phantom reservations.

Java implementation

Package `lld.ch15.hotel`. Files: `Billing.java` (invoice, payments, room charges), `Rooms.java` (room, key, housekeeping, search, hotel, clock, money), `Stay.java` (booking, actors, front desk), `Main.java`.

```java
// File: lld/ch15/hotel/Billing.java
package lld.ch15.hotel;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

enum PaymentStatus { UNPAID, PENDING, COMPLETED, DECLINED, CANCELLED, REFUNDED }

record InvoiceItem(String description, BigDecimal amount) {}   // money: BigDecimal, never double

/** A bill is a list of lines; the total is always computed, so it can never disagree with the lines. */
final class Invoice {
    private final String number;
    private final List<InvoiceItem> items = new ArrayList<>();
    Invoice(String number) { this.number = number; }
    String number()                        { return number; }
    synchronized void addItem(InvoiceItem i) { items.add(i); }
    synchronized List<InvoiceItem> items()   { return List.copyOf(items); }
    synchronized BigDecimal total()          { return
items.stream().map(InvoiceItem::amount).reduce(BigDecimal.ZERO, BigDecimal::add); }
}

/** Three ways to pay, which really behave differently: a check does not clear at once. */
abstract sealed class Payment permits CreditCardTransaction, CheckTransaction, CashTransaction {
    private final BigDecimal amount;
    private PaymentStatus status = PaymentStatus.UNPAID;
    Payment(BigDecimal amount) { this.amount = amount; }
    BigDecimal amount()      { return amount; }
    PaymentStatus status()   { return status; }
    protected void setStatus(PaymentStatus s) { status = s; }
    /** Take the money. Returns true when the hotel may release the guest. */
    abstract boolean initiateTransaction();
}

final class CreditCardTransaction extends Payment {
    private final String nameOnCard;
    CreditCardTransaction(BigDecimal amount, String nameOnCard) { super(amount); this.nameOnCard = nameOnCard; }
    @Override boolean initiateTransaction() { setStatus(PaymentStatus.COMPLETED); return true; }
    @Override public String toString() { return "card(" + nameOnCard + ")"; }
}

final class CheckTransaction extends Payment {
    private final String bankName, checkNumber;
    CheckTransaction(BigDecimal amount, String bankName, String checkNumber) {
        super(amount); this.bankName = bankName; this.checkNumber = checkNumber;
    }
    @Override boolean initiateTransaction() { setStatus(PaymentStatus.PENDING); return true; }   // clears in days
    @Override public String toString() { return "check(" + bankName + " #" + checkNumber + ")"; }
}

final class CashTransaction extends Payment {
    private final BigDecimal tendered;
    CashTransaction(BigDecimal amount, BigDecimal tendered) { super(amount); this.tendered = tendered; }
    @Override boolean initiateTransaction() {
        if (tendered.compareTo(amount()) < 0) { setStatus(PaymentStatus.DECLINED); return false; }
        setStatus(PaymentStatus.COMPLETED);
        return true;
    }
    BigDecimal change() { return tendered.subtract(amount()); }
    @Override public String toString() { return "cash"; }
}

/**
 * TEMPLATE METHOD. addInvoiceItem() is the fixed algorithm (describe, price, write a line);
 * each subclass fills in only description(). A new kind of charge = a new subclass, no edit to Invoice.
 */
abstract class RoomCharge {
```

```
    private final BigDecimal amount;
    RoomCharge(BigDecimal amount) { this.amount = amount; }
    protected abstract String description();                // the one step that varies
    final void addInvoiceItem(Invoice invoice) {            // the template: final, so the order can't change
        invoice.addItem(new InvoiceItem(description(), amount));
    }
}

final class Amenity extends RoomCharge {
    private final String name;
    Amenity(String name, BigDecimal amount) { super(amount); this.name = name; }
    @Override protected String description() { return "Amenity: " + name; }
}

final class RoomService extends RoomCharge {
    private final String request;
    RoomService(String request, BigDecimal amount) { super(amount); this.request = request; }
    @Override protected String description() { return "Room service: " + request; }
}

final class KitchenService extends RoomCharge {
    private final String dish;
    KitchenService(String dish, BigDecimal amount) { super(amount); this.dish = dish; }
    @Override protected String description() { return "Kitchen: " + dish; }
}
```

```java
// File: lld/ch15/hotel/Rooms.java
package lld.ch15.hotel;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.TreeMap;
import java.util.concurrent.ConcurrentHashMap;

enum RoomStyle { STANDARD, DELUXE, FAMILY_SUITE, BUSINESS_SUITE }

/** What the room is doing RIGHT NOW. It cannot answer "is it free for 3 nights from the 10th?". */
enum RoomStatus { AVAILABLE, RESERVED, OCCUPIED, NOT_AVAILABLE, BEING_SERVICED, OTHER }

/** An electronic key card. Deactivated at checkout so it stops opening the door. */
final class RoomKey {
    private final String keyId, barcode;
    private final LocalDateTime issuedAt;
    private final boolean master;
    private volatile boolean active = true;
    RoomKey(String keyId, String barcode, LocalDateTime issuedAt, boolean master) {
        this.keyId = keyId; this.barcode = barcode; this.issuedAt = issuedAt; this.master = master;
    }
    String keyId()        { return keyId; }
    boolean isActive()    { return active; }
    boolean isMaster()    { return master; }
    void deactivate()     { active = false; }
}

/** Append-only log line: nothing is overwritten, so a room's history can be read back. */
record RoomHouseKeeping(LocalDateTime at, String housekeeper, String description, int minutes) {}

/**
 * A room owns its keys and its housekeeping log (composition), and the SET OF NIGHTS it is committed to.
 * Reserving nights is check-then-act, so it runs under the room's monitor: two guests can never
 * both get the night of the 10th.
 */
final class Room {
    private final String number;
    private final RoomStyle style;
    private final BigDecimal pricePerNight;
    private final boolean smoking;
    private final Set<LocalDate> bookedNights = new HashSet<>();
    private final List<RoomKey> keys = new ArrayList<>();
    private final List<RoomHouseKeeping> housekeeping = new ArrayList<>();
    private RoomStatus status = RoomStatus.AVAILABLE;

    Room(String number, RoomStyle style, BigDecimal pricePerNight, boolean smoking) {
        this.number = number; this.style = style; this.pricePerNight = pricePerNight; this.smoking = smoking;
    }
    String number()             { return number; }
    RoomStyle style()           { return style; }
    BigDecimal pricePerNight()  { return pricePerNight; }

    /** Nights are [checkIn, checkOut): the checkout day itself is free for the next guest. */
    synchronized boolean isFree(LocalDate checkIn, LocalDate checkOut) {
```

```
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) if (bookedNights.contains(d)) return
false;
        return true;
    }
    synchronized boolean tryReserve(LocalDate checkIn, LocalDate checkOut) {
        if (!isFree(checkIn, checkOut)) return false;              // re-entrant monitor: same lock
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) bookedNights.add(d);
        return true;
    }
    synchronized void release(LocalDate checkIn, LocalDate checkOut) {
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) bookedNights.remove(d);
    }

    synchronized RoomStatus status()          { return status; }
    synchronized void setStatus(RoomStatus s) { status = s; }

    synchronized RoomKey issueKey(String keyId, LocalDateTime at) {
        RoomKey k = new RoomKey(keyId, "BC-" + keyId, at, false);
        keys.add(k);
        return k;
    }
    synchronized int deactivateGuestKeys() {
        int n = 0;
        for (RoomKey k : keys) if (!k.isMaster() && k.isActive()) { k.deactivate(); n++; }
        return n;
    }
    synchronized void addHousekeeping(RoomHouseKeeping h) { housekeeping.add(h); }
    synchronized List<RoomHouseKeeping> housekeepingLog()  { return List.copyOf(housekeeping); }
}

/** Search belongs to the LOCATION: a single room only knows about itself. */
interface Search {
    List<Room> search(RoomStyle style, LocalDate checkIn, LocalDate checkOut);
}

final class HotelLocation implements Search {
    private final String name;
    private final Map<String, Room> rooms = new ConcurrentHashMap<>();
    HotelLocation(String name) { this.name = name; }
    String name()           { return name; }
    void addRoom(Room r)    { rooms.put(r.number(), r); }
    Room room(String n)     { return rooms.get(n); }

    @Override public List<Room> search(RoomStyle style, LocalDate in, LocalDate out) {
        return rooms.values().stream().filter(r -> r.style() == style && r.isFree(in, out))
                .sorted(Comparator.comparing(Room::number)).toList();
    }
}

final class Hotel {
    private final String name;
    private final List<HotelLocation> locations = new ArrayList<>();
    Hotel(String name) { this.name = name; }
    void addLocation(HotelLocation l) { locations.add(l); }
    /** Asks every location in turn. */
    Map<String, List<Room>> search(RoomStyle style, LocalDate checkIn, LocalDate checkOut) {
        Map<String, List<Room>> result = new TreeMap<>();
        for (HotelLocation l : locations) result.put(l.name(), l.search(style, checkIn, checkOut));
        return result;
    }
}

final class SimClock extends Clock {
    private volatile Instant now;
    SimClock(Instant start) { this.now = start; }
    void advance(Duration d) { now = now.plus(d); }
    @Override public ZoneId getZone() { return ZoneOffset.UTC; }
    @Override public Clock withZone(ZoneId zone) { return this; }
    @Override public Instant instant() { return now; }
```

```
}

final class Money {
    private Money() {}
    static String fmt(BigDecimal amount) {
        String digits = amount.abs().setScale(2, RoundingMode.UNNECESSARY).toPlainString();
        return (amount.signum() < 0 ? "-$" : "$") + digits;
    }
}
```

```java
// File: lld/ch15/hotel/Stay.java
package lld.ch15.hotel;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.Duration;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Function;

enum BookingStatus { REQUESTED, PENDING, CONFIRMED, CHECKED_IN, CHECKED_OUT, CANCELLED, ABANDONED }

/** The actors. Each service method takes the actor type allowed to call it, so roles are enforced by types. */
sealed interface Person permits Guest, Receptionist, Server, Housekeeper, Manager { String name(); }
record Guest(String id, String name, String email) implements Person {}
record Receptionist(String id, String name) implements Person {}
record Server(String id, String name) implements Person {}
record Housekeeper(String id, String name) implements Person {}
record Manager(String id, String name) implements Person {}

interface Notifier { void send(String guestId, String message); }

/**
 * One stay in one room. Owns its invoice (composition); only points at the room (association).
 * Records the guest by id, which answers "who booked this room?" and "which rooms did this guest book?".
 */
final class RoomBooking {
    static final LocalTime CHECK_IN_TIME = LocalTime.of(14, 0);
    static final Duration FULL_REFUND_NOTICE = Duration.ofHours(24);

    private final String reservationNumber;
    private final String guestId;
    private final Room room;
    private final LocalDate checkIn, checkOut;
    private final Invoice invoice;
    private BookingStatus status = BookingStatus.CONFIRMED;
    private BigDecimal deposit = BigDecimal.ZERO;

    RoomBooking(String reservationNumber, String guestId, Room room, LocalDate checkIn, LocalDate checkOut) {
        this.reservationNumber = reservationNumber; this.guestId = guestId; this.room = room;
        this.checkIn = checkIn; this.checkOut = checkOut; this.invoice = new Invoice("INV-" + reservationNumber);
    }
    String reservationNumber() { return reservationNumber; }
    String guestId()           { return guestId; }
    Room room()                { return room; }
    LocalDate checkIn()        { return checkIn; }
    LocalDate checkOut()       { return checkOut; }
    Invoice invoice()          { return invoice; }
    long nights()              { return checkIn.until(checkOut).getDays(); }
    synchronized BookingStatus status()        { return status; }
    synchronized void setStatus(BookingStatus s) { status = s; }
    synchronized BigDecimal deposit()             { return deposit; }
    synchronized void setDeposit(BigDecimal d)    { deposit = d; }

    /** The refund rule lives in exactly ONE method: full refund if cancelled more than 24h before check-in. */
    BigDecimal refundDue(LocalDateTime now) {
        Duration notice = Duration.between(now, checkIn.atTime(CHECK_IN_TIME));
        // compareTo, not toHours(): toHours() truncates, so 24h30m would wrongly count as "24, not more than 24".
        return notice.compareTo(FULL_REFUND_NOTICE) > 0 ? deposit() : BigDecimal.ZERO;
    }
}
```

```java
/** The front desk: every flow of a stay, from booking to checkout. */
final class FrontDesk {
    private final HotelLocation location;
    private final Clock clock;
    private final Notifier notifier;
    private final Map<String, RoomBooking> bookings = new ConcurrentHashMap<>();
    private final List<Person> staff = new CopyOnWriteArrayList<>();
    private final AtomicInteger seq = new AtomicInteger(), keySeq = new AtomicInteger();

    FrontDesk(HotelLocation location, Clock clock, Notifier notifier) {
        this.location = location; this.clock = clock; this.notifier = notifier;
    }

    private LocalDateTime now() { return LocalDateTime.now(clock); }

    void addRoom(Manager by, Room r) { location.addRoom(r); }

    /** The manager adds new workers (receptionists, servers, housekeepers). */
    void hire(Manager by, Person worker) {
        if (worker instanceof Guest) throw new IllegalArgumentException("a guest is not a worker");
        staff.add(worker);
    }
    List<String> staff() { return staff.stream().map(Person::name).toList(); }

    /** Guest or receptionist books: same call, two actors. The room's monitor makes it race-free. */
    RoomBooking book(Guest guest, Room room, LocalDate checkIn, LocalDate checkOut, Payment deposit) {
        if (!checkIn.isBefore(checkOut)) throw new IllegalArgumentException("check-out must be after check-in");
        if (!room.tryReserve(checkIn, checkOut))
            throw new IllegalStateException("room " + room.number() + " is not free " + checkIn + ".." + checkOut);
        RoomBooking b = new RoomBooking("R" + seq.incrementAndGet(), guest.id(), room, checkIn, checkOut);
        if (!deposit.initiateTransaction()) {                  // payment failed: give the nights back
            room.release(checkIn, checkOut);
            throw new IllegalStateException("deposit " + deposit.status());
        }
        b.setDeposit(deposit.amount());
        bookings.put(b.reservationNumber(), b);
        notifier.send(guest.id(), b.reservationNumber() + " confirmed: room " + room.number() + " " + checkIn +
".." + checkOut);
        return b;
    }

    RoomBooking bookFor(Receptionist by, Guest guest, Room room, LocalDate in, LocalDate out, Payment deposit) {
        return book(guest, room, in, out, deposit);
    }

    RoomKey checkIn(Receptionist by, String reservationNumber) {
        RoomBooking b = get(reservationNumber);
        synchronized (b) {
            if (b.status() != BookingStatus.CONFIRMED) throw new IllegalStateException(reservationNumber + " is " +
b.status());
            if (now().toLocalDate().isBefore(b.checkIn())) throw new IllegalStateException("too early to check
in");
            b.setStatus(BookingStatus.CHECKED_IN);
            b.room().setStatus(RoomStatus.OCCUPIED);
            return b.room().issueKey("K" + keySeq.incrementAndGet(), now());
        }
    }

    void addCharge(Server by, String reservationNumber, RoomCharge charge) {
        RoomBooking b = get(reservationNumber);
        synchronized (b) {                     // same monitor as checkOut: no charge can slip in after the bill is
totalled
            if (b.status() != BookingStatus.CHECKED_IN) throw new IllegalStateException("guest not in house");
            charge.addInvoiceItem(b.invoice());
        }
    }

    /** Adds the room nights, credits the deposit, takes payment for the balance, kills the keys. */
```

```
    Invoice checkOut(Receptionist by, String reservationNumber, Function<BigDecimal, Payment> payWith) {
        RoomBooking b = get(reservationNumber);
        synchronized (b) {
            if (b.status() != BookingStatus.CHECKED_IN) throw new IllegalStateException(reservationNumber + " is "
+ b.status());
            Invoice inv = b.invoice();
            BigDecimal price = b.room().pricePerNight();
            List<InvoiceItem> finalLines = new ArrayList<>(List.of(
                    new InvoiceItem(b.nights() + " nights x " + Money.fmt(price),
price.multiply(BigDecimal.valueOf(b.nights()))),
                    new InvoiceItem("Deposit paid", b.deposit().negate())));
            BigDecimal balance = finalLines.stream().map(InvoiceItem::amount).reduce(inv.total(), BigDecimal::add);
            Payment p = payWith.apply(balance);
            // Write the final lines only once payment succeeds, so a declined payment can be retried
            // without the nights and deposit lines being added to the invoice twice.
            if (!p.initiateTransaction()) throw new IllegalStateException("payment " + p.status() + ", guest still
checked in");
            finalLines.forEach(inv::addItem);
            int killed = b.room().deactivateGuestKeys();
            b.setStatus(BookingStatus.CHECKED_OUT);
            b.room().setStatus(RoomStatus.BEING_SERVICED);
            notifier.send(b.guestId(), "checked out, paid " + Money.fmt(inv.total()) + " by " + p + " [" +
p.status()
                    + "], " + killed + " key(s) deactivated");
            return inv;
        }
    }

    void logHousekeeping(Housekeeper by, Room room, String description, int minutes) {
        room.addHousekeeping(new RoomHouseKeeping(now(), by.name(), description, minutes));
        room.setStatus(RoomStatus.AVAILABLE);
    }

    BigDecimal cancel(Person by, String reservationNumber) {
        RoomBooking b = get(reservationNumber);
        synchronized (b) {
            if (b.status() != BookingStatus.CONFIRMED) throw new IllegalStateException(reservationNumber + " is " +
b.status());
            BigDecimal refund = b.refundDue(now());
            b.room().release(b.checkIn(), b.checkOut());
            b.setStatus(BookingStatus.CANCELLED);
            notifier.send(b.guestId(), reservationNumber + " cancelled by " + by.name() + ", refund " +
Money.fmt(refund));
            return refund;
        }
    }

    /** System job: remind guests whose check-in or check-out is within the next 24 hours. */
    void sendReminders() {
        LocalDateTime now = now(), soon = now.plusHours(24);
        bookings.values().stream().sorted((x, y) ->
x.reservationNumber().compareTo(y.reservationNumber())).forEach(b -> {
            LocalDateTime in = b.checkIn().atTime(RoomBooking.CHECK_IN_TIME), out = b.checkOut().atTime(11, 0);
            if (b.status() == BookingStatus.CONFIRMED && in.isAfter(now) && !in.isAfter(soon))
                notifier.send(b.guestId(), "reminder: check-in for " + b.reservationNumber() + " at " + in);
            if (b.status() == BookingStatus.CHECKED_IN && out.isAfter(now) && !out.isAfter(soon))
                notifier.send(b.guestId(), "reminder: check-out for " + b.reservationNumber() + " at " + out);
        });
    }

    List<String> roomsBookedBy(String guestId) {
        return bookings.values().stream().filter(b -> b.guestId().equals(guestId) && b.status() !=
BookingStatus.CANCELLED)
                .map(b -> b.room().number()).sorted().toList();
    }

    List<String> whoBooked(String roomNumber) {
        return bookings.values().stream().filter(b -> b.room().number().equals(roomNumber) && b.status() !=
BookingStatus.CANCELLED)
```

```
                .map(b -> b.guestId() + "@" + b.checkIn()).sorted().toList();
    }

    RoomBooking get(String n) {
        RoomBooking b = bookings.get(n);
        if (b == null) throw new IllegalArgumentException("no booking " + n);
        return b;
    }
}
```

```java
// File: lld/ch15/hotel/Main.java
package lld.ch15.hotel;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) throws Exception {
        SimClock clock = new SimClock(Instant.parse("2026-07-01T09:00:00Z"));
        Notifier notifier = (g, m) -> System.out.println("  [notify " + g + "] " + m);
        Hotel hotel = new Hotel("Seaside Hotels");
        HotelLocation goa = new HotelLocation("Goa"), kochi = new HotelLocation("Kochi");
        hotel.addLocation(goa);
        hotel.addLocation(kochi);
        FrontDesk desk = new FrontDesk(goa, clock, notifier);
        Manager boss = new Manager("MG1", "Maya");
        desk.addRoom(boss, new Room("101", RoomStyle.DELUXE, usd("120.00"), false));
        desk.addRoom(boss, new Room("102", RoomStyle.DELUXE, usd("120.00"), false));
        desk.addRoom(boss, new Room("201", RoomStyle.FAMILY_SUITE, usd("250.00"), false));
        kochi.addRoom(new Room("K-1", RoomStyle.DELUXE, usd("90.00"), true));
        Room r101 = goa.room("101");
        Receptionist rita = new Receptionist("RC1", "Rita");
        desk.hire(boss, rita);
        desk.hire(boss, new Server("SV1", "Sam"));
        desk.hire(boss, new Housekeeper("HK1", "Hari"));
        System.out.println("== 0. Manager adds rooms and hires staff: " + desk.staff() + " ==\n");

        LocalDate jul10 = LocalDate.of(2026, 7, 10), jul13 = LocalDate.of(2026, 7, 13);
        System.out.println("== 1. Search DELUXE for Jul 10-13 across locations ==");
        hotel.search(RoomStyle.DELUXE, jul10, jul13).forEach((loc, rooms) ->
                System.out.println("  " + loc + " -> " + rooms.stream().map(Room::number).toList()));

        System.out.println("\n== 2. 6 guests try to book room 101 for overlapping dates at once ==");
        ConcurrentLinkedQueue<String> results = new ConcurrentLinkedQueue<>();
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(6)) {
            for (int i = 1; i <= 6; i++) {
                Guest g = new Guest("G" + i, "Guest " + i, "g" + i + "@mail.com");
                LocalDate out = jul10.plusDays(2 + i % 2);     // Jul 10-12 or Jul 10-13: all overlap
                pool.submit(() -> {
                    go.await();
                    try { desk.book(g, r101, jul10, out, new CreditCardTransaction(usd("120.00"), g.name()));
results.add("OK"); }
                    catch (IllegalStateException e) { results.add("NO"); }
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("  successful bookings = " + results.stream().filter("OK"::equals).count() + " of 6");
        System.out.println("  who booked 101? " + desk.whoBooked("101"));

        System.out.println("\n== 3. Back-to-back stays share the changeover day ==");
        Guest asha = new Guest("ASHA", "Asha", "asha@mail.com");
        RoomBooking stay = desk.bookFor(rita, asha, goa.room("102"), jul10, jul13, new
CashTransaction(usd("120.00"), usd("200.00")));
        RoomBooking next = desk.book(new Guest("BEN", "Ben", "ben@mail.com"), goa.room("102"), jul13,
                LocalDate.of(2026, 7, 15), new CheckTransaction(usd("120.00"), "HDFC", "000123"));
        System.out.println("  rooms booked by ASHA: " + desk.roomsBookedBy("ASHA"));

        System.out.println("\n== 4. Cancellation: >24h before check-in = full refund, later = none ==");
        RoomBooking suite = desk.book(asha, goa.room("201"), jul10, jul13, new CreditCardTransaction(usd("250.00"),
```

```
"Asha"));
        Guest carl = new Guest("CARL", "Carl", "carl@mail.com");
        RoomBooking lastMinute = desk.book(carl, goa.room("201"), LocalDate.of(2026, 7, 2), LocalDate.of(2026, 7,
3),
                new CreditCardTransaction(usd("250.00"), "Carl"));
        clock.advance(Duration.ofHours(10));              // Jul 1 19:00: Carl's check-in (Jul 2 14:00) is 19h
away
        desk.cancel(carl, lastMinute.reservationNumber());
        desk.cancel(asha, suite.reservationNumber());     // Asha's is 8+ days away

        System.out.println("\n== 5. Reminders, check-in (key), room service, check-out (invoice) ==");
        clock.advance(Duration.ofDays(8).plusHours(13));  // Jul 10 08:00
        desk.sendReminders();
        clock.advance(Duration.ofHours(6));               // Jul 10 14:00
        RoomKey key = desk.checkIn(rita, stay.reservationNumber());
        System.out.println("  issued key " + key.keyId() + " active=" + key.isActive());
        Server sam = new Server("SV1", "Sam");
        desk.addCharge(sam, stay.reservationNumber(), new RoomService("Extra towels", usd("5.00")));
        desk.addCharge(sam, stay.reservationNumber(), new KitchenService("Fish curry dinner", usd("24.00")));
        desk.addCharge(sam, stay.reservationNumber(), new Amenity("Spa access", usd("30.00")));
        clock.advance(Duration.ofDays(2).plusHours(19));  // Jul 13 09:00: checkout at 11:00 today
        desk.sendReminders();
        try { desk.checkOut(rita, stay.reservationNumber(), total -> new CashTransaction(total, usd("100.00"))); }
        catch (IllegalStateException e) { System.out.println("  cash short: " + e.getMessage()); }
        Invoice inv = desk.checkOut(rita, stay.reservationNumber(), total -> new CreditCardTransaction(total,
"Asha"));
        inv.items().forEach(i -> System.out.printf("    %-28s %10s%n", i.description(), Money.fmt(i.amount())));
        System.out.printf("    %-28s %10s%n", "TOTAL", Money.fmt(inv.total()));
        System.out.println("  key active after checkout? " + key.isActive() + ", room 102 is " +
goa.room("102").status());

        System.out.println("\n== 6. Housekeeping log ==");
        desk.logHousekeeping(new Housekeeper("HK1", "Hari"), goa.room("102"), "Full clean, linen change", 40);
        goa.room("102").housekeepingLog().forEach(h -> System.out.println("  " + h));
        System.out.println("  room 102 is " + goa.room("102").status());
    }

    private static BigDecimal usd(String amount) { return new BigDecimal(amount); }
}
```

Output:

```
== 0. Manager adds rooms and hires staff: [Rita, Sam, Hari] ==

== 1. Search DELUXE for Jul 10-13 across locations ==
  Goa -> [101, 102]
  Kochi -> [K-1]

== 2. 6 guests try to book room 101 for overlapping dates at once ==
  [notify G6] R1 confirmed: room 101 2026-07-10..2026-07-12
  successful bookings = 1 of 6
  who booked 101? [G6@2026-07-10]

== 3. Back-to-back stays share the changeover day ==
  [notify ASHA] R2 confirmed: room 102 2026-07-10..2026-07-13
  [notify BEN] R3 confirmed: room 102 2026-07-13..2026-07-15
  rooms booked by ASHA: [102]

== 4. Cancellation: >24h before check-in = full refund, later = none ==
  [notify ASHA] R4 confirmed: room 201 2026-07-10..2026-07-13
  [notify CARL] R5 confirmed: room 201 2026-07-02..2026-07-03
  [notify CARL] R5 cancelled by Carl, refund $0.00
  [notify ASHA] R4 cancelled by Asha, refund $250.00

== 5. Reminders, check-in (key), room service, check-out (invoice) ==
  [notify G6] reminder: check-in for R1 at 2026-07-10T14:00
  [notify ASHA] reminder: check-in for R2 at 2026-07-10T14:00
  issued key K1 active=true
  [notify ASHA] reminder: check-out for R2 at 2026-07-13T11:00
  [notify BEN] reminder: check-in for R3 at 2026-07-13T14:00
  cash short: payment DECLINED, guest still checked in
  [notify ASHA] checked out, paid $299.00 by card(Asha) [COMPLETED], 1 key(s) deactivated
    Room service: Extra towels        $5.00
    Kitchen: Fish curry dinner       $24.00
    Amenity: Spa access              $30.00
    3 nights x $120.00              $360.00
    Deposit paid                   -$120.00
    TOTAL                           $299.00
  key active after checkout? false, room 102 is BEING_SERVICED

== 6. Housekeeping log ==
  RoomHouseKeeping[at=2026-07-13T09:00, housekeeper=Hari, description=Full clean, linen change, minutes=40]
  room 102 is AVAILABLE
```

What to notice: section 0 — only a `Manager` can add rooms and hire staff (the course's "Manager adds new workers"); section 2 — six concurrent bookings of room 101 for overlapping dates produce exactly one booking (which guest wins varies per run); section 3 — Ben's stay starting the 13th fits right after Asha's ending the 13th (half-open ranges); section 4 — Carl cancels 19 hours before check-in and gets nothing, Asha cancels 8 days ahead and gets a full refund; section 5 — reminders fire within 24 hours of check-in/check-out, the key is issued at check-in and dead after check-out, a declined cash payment leaves the guest checked in and the invoice untouched (the retry by card does not duplicate any line), and the invoice lines add up (room service $5 + kitchen $24 + spa $30 + 3 nights $360 − $120 deposit = $299); section 6 — housekeeping is logged and the room becomes available.

Concurrency & edge cases

- **Double booking a room-night.** Prevented by `Room.tryReserve` being `synchronized`: the check and the add of every night are one atomic step. The demo's 6-thread race always has one winner.
- **Search is advisory.** `search` may show a room that is gone a millisecond later; the booking itself (`tryReserve`) is the real check — so always handle "no longer available" on book.
- **Booking-level transitions** (`checkIn`, `checkOut`, `cancel`) run under the booking's monitor, so a cancel can't interleave with a check-in. Lock order is always booking → room.
- **The 24-hour rule, exactly.** `refundDue` compares `Duration`s (`notice.compareTo(24h) > 0`). The tempting `Duration.toHours() > 24` is wrong: `toHours()` drops the minutes, so a cancellation 24 h 30 min ahead would count as "24, not more than 24" and lose its refund.

- **Checkout is retry-safe.** The nights and deposit lines are written to the invoice only after the payment succeeds, and `addCharge` takes the same booking monitor as `checkOut`, so no charge can be added between totalling the bill and closing it.
- **Edge cases:** check-out date not after check-in; checking in before the check-in date; adding charges when the guest is not in house; cancelling after check-in; cash tendered below the amount (declined); a check payment that stays `PENDING`.
- **Chain-wide inventory.** The course notes that storing nights on each room is simple but doesn't scale to a chain selling through many channels: real systems keep availability in one place (an **inventory table** keyed by `(room_type, date)` with a count, updated with a conditional `UPDATE ... SET sold = sold + 1 WHERE sold < capacity`), so two sites or two booking channels can't sell the same night.

Extensions & follow-up questions

- **"A family wants two rooms in one booking."** The course flags this: move the dates and the invoice *up one level* to a `Reservation` that holds several `RoomStay`s (each with its room and nights). Reserving several rooms then needs all-or-nothing across rooms — lock rooms in a fixed order (by room number) to avoid deadlock, or do it in one database transaction.
- **"Book by room type, assign the room at check-in."** Many hotels sell a *type* for a date range and pick the physical room later. Keep per-type-per-night counts for selling; assign a specific room at check-in (housekeeping status permitting).
- **"Seasonal rates."** A `RatePlan`/`PricingStrategy` gives the price per night (weekends, seasons, loyalty tiers); the nights line becomes one line per night.
- **"Overbooking."** Allow selling capacity × (1 + overbooking factor) of a type; the ones who can't be housed are "walked" to a partner hotel — a business policy, pluggable as a strategy on the availability check.
- **"Separate the booker from the occupants."** The course names this as missing: add `Occupant`s to the stay (names for fire safety/registration) distinct from the paying guest.
- **"Lost key."** Deactivate all active guest keys for that room and issue a new one — `deactivateGuestKeys()` + `issueKey()`.

Real-world example

Property management systems such as **Oracle Hospitality OPERA** run the front desk this way: reservations are made against room types and dates, a specific room is assigned by check-in, the folio (the guest's bill) collects charges from restaurants, minibar and spa as separate postings, and housekeeping status (dirty, clean, inspected) controls whether a room can be assigned. At check-in, the desk encodes an RFID (radio-frequency identification, a contactless chip) key card with an electronic lock system (for example ASSA ABLOY's VingCard or dormakaba) that is valid only until the check-out time. Online, **Booking.com** shows each rate's cancellation policy (for example "free cancellation until 24 hours before arrival") — exactly the one-method refund rule of this design, but chosen per rate plan.

In the interview

30–45 minute plan. (1) Clarify: room types, date-range search, 24-hour refund, charges, payment types, keys, housekeeping. (2) Actors and use cases. (3) Entities, stating early: **"a status field is not availability"** — rooms keep booked nights. (4) Code `tryReserve` (synchronized), `book`, `checkIn`, `checkOut` with the invoice. (5) Template Method for charges; payment subclasses; refund rule. (6) Follow-ups: two rooms per booking, chain-wide inventory, overbooking.

> **30-second answer:** "A `Hotel` has `HotelLocation`s which own `Room`s and implement search by type and date range. Each room keeps the set of nights it's committed to, and reserving nights is a synchronized check-then-act so two guests can't get the same night. A `RoomBooking` owns an `Invoice` made of lines; room service, kitchen and amenities are `RoomCharge` subclasses using Template Method to add a line. Payment is credit card, check or cash subclasses. Cancellation refunds in full if more than 24 hours before check-in — one method. Check-in issues a key card; check-out totals the invoice, takes payment, kills the keys and hands the room to housekeeping."

Self-check

- **Q:** Why isn't `RoomStatus` enough to know if a room can be booked? **A:** It only says what the room is doing now; a booking asks about a range of future nights.

- **Q:** Why are nights half-open `[checkIn, checkOut)`? **A:** The checkout day's night is not used, so the next guest can check in that day.
- **Q:** Which pattern lets a new charge type be added without touching `Invoice`? **A:** Template Method (`RoomCharge.addInvoiceItem` is fixed; subclasses supply `description()`).
- **Q:** Where does the refund rule live, and why? **A:** In `RoomBooking.refundDue` — one place to change when the policy changes.
- **Q:** How is it guaranteed only a server can add a room-service charge? **A:** `addCharge` takes a `Server` parameter; other actor types don't compile.

> **TL;DR**

Rooms are sold night by night: each room keeps its booked nights and reserves them atomically. Search lives on the location. The bill is a list of lines filled by `RoomCharge` Template Methods; three payment subclasses; one refund method (24 hours). Keys issued at check-in, killed at check-out; housekeeping is an append-only log (entries are only added, never changed or removed).

Chapter summary

Problem	Inventory unit	How a hold/booking is made safe	Payment failure handling	Cancellation
Library	`BookItem` (copy)	`synchronized` member → item (fixed order)	— (fines recorded)	Return → FIFO waitlist per title gets the copy
Movie tickets	`ShowSeat` (seat × show)	Per-show monitor; all-or-nothing lock with TTL; atomic commit	Declined → release; lost hold → refund (compensation)	Refund per `CancellationPolicy`; seats back on sale
Concert tickets	Numbered seat state / GA counter	Lock-free CAS; waiting room; per-user limit	Same compensation as movies	Released tickets offered to waitlist first
Hotel	Room × night	`synchronized` `Room.tryReserve` over a date range	Failed deposit → nights released	Full refund if > 24h before check-in

Five sentences to remember for any booking interview:

1. Name the *real* unit of inventory first (copy, show seat, room-night, GA count).
2. Every "if free then take" must be one atomic step — a monitor, a CAS, or a conditional `UPDATE`.
3. Temporary holds need an expiry, or abandoned checkouts block inventory forever.
4. Money can't be rolled back — failures after payment need a compensating refund.
5. In-process locks protect one server; with many servers, the database (constraints, conditional updates) or a TTL store like Redis is the real guarantee.

Chapter 16 — Case Studies: Booking & Inventory (Part 2)

> **Where this fits.** Chapter 15 covered the first group of booking problems. This chapter finishes the family with four more: a car rental company, a restaurant, an airline and a university's course registration. They look different on the surface, but they are the same problem underneath: **a limited resource, a calendar, and many people trying to claim it at once.** Once you see that shared skeleton, each new booking problem takes minutes to set up in an interview.

The shared skeleton of every booking problem

Idea	Car rental	Restaurant	Airline	Course registration
The thing being booked	a vehicle	a table	a seat on one flight instance	a seat in one section
The "calendar" it is booked on	days	hourly time slots	the flight instance (route + date)	the term
The check-then-claim step that must be atomic	`Vehicle.tryBook`	`Table.tryHold`	`FlightInstance.holdSeats`	`Section.enroll`
What happens when it is full	search another car	try the next table	pick another seat or flight	waitlist
Money	bill with lines + late fee	check split by seat + tax	fare per seat class, paid per itinerary	none (fees are an extension)

A **resource** is anything with a limited supply (a car, a table, a seat). A **race condition** is a bug where the result depends on which of two threads runs first — here, two people both "win" the same seat because both checked "free?" before either wrote "taken".

A **thread** is one independent line of execution inside a running program; a server handles many requests at once by running many threads side by side. A step is **atomic** when it happens as one indivisible unit: no other thread can see it half-done or slip in between its parts. **Concurrency** simply means several things are in progress at the same time.

How to run every example in this chapter

```
cd chapters/16-case-booking-2/code
javac --release 21 -d /tmp/lld_16 $(find . -name '*.java')
java -cp /tmp/lld_16 lld.ch16.carrental.Main          # or restaurant, airline, courseregistration
```

`javac` is the Java compiler (it turns `.java` source files into runnable `.class` files), `--release 21` targets Java version 21, and `-cp` (the **classpath**) tells `java` which folder to load compiled classes from. All four `Main` classes compile with Java 21 (standard library only — the classes that ship with Java itself, no extra downloads) and run. The output printed under each program is its real output. Where threads race, the *counts* are always the same, but *which* racer wins can differ from run to run.

> **Code-style note.** To keep each design readable, one file sometimes holds several small package-private types (visible only to other classes in the same package, i.e. the same folder of code) — for example `Types.java` holds the enums and records. An **enum** is a type with a fixed list of named values (`CAR`, `TRUCK` …); a **record** is a compact Java class that just holds a few fixed values and gets `equals`, `hashCode` and `toString` for free. That is legal Java. In production you would usually give each top-level type its own file.

> **Money.** All prices use `java.math.BigDecimal`, never `double`. A `double` is a binary floating-point number (it stores values in base 2 with limited precision); it cannot represent 0.1 exactly, so sums of prices drift by fractions of a cent. `BigDecimal` stores exact decimal digits.

Design a Car Rental System

Figure — A car rental company rents vehicles from many branches (course diagram)

Problem & clarifying questions

A **car rental system** is the software a rental company uses to rent vehicles for a short time — from a few hours to a few weeks. Companies usually have many branches, often near airports and city centres, and a customer may return a car to a different branch from the one they picked it up at.

Questions worth asking in the first two minutes, with the answers we assume:

Question	Assumed answer
Which vehicle kinds?	Cars, trucks, SUVs, vans and motorcycles.
One branch or many?	Many branches, each with its own vehicles.
Can the drop-off branch differ from the pick-up branch?	Yes, the reservation records both.
How is price worked out?	A daily rate per vehicle type, plus add-ons, plus a late fee per day late.
What can be added?	Insurance, equipment (navigation, child seat, ski rack), services (roadside assistance, wifi) and additional drivers.
How are vehicles identified?	Each has a unique barcode; a barcode reader scans it at pick-up and return.
Do we need history?	Yes: a vehicle log of every event (reserved, picked up, returned, fuelling, cleaning, damage).
Concurrency?	Many members book at once; two must never get the same car on overlapping days.

Requirements

Functional

1. Rent different vehicle types: car, truck, SUV, van, motorcycle.
2. Each vehicle is added with a unique barcode and details, including the parking stall number that tells staff where it is parked.
3. Answer "which member took this vehicle?" and "which vehicles has this member rented?".
4. Charge a late fee when a vehicle comes back after its due date.
5. Members search the inventory and reserve any available vehicle.
6. Send notifications when a pick-up date is near, when a vehicle is near its due date, and when it is overdue.
7. Read vehicle barcodes.
8. Members can cancel reservations.
9. Keep a vehicle log of all events on a vehicle.
10. Members can add rental insurance.
11. Members can rent extra equipment (navigation, child seat, ski rack).
12. Members can add services (roadside assistance, additional driver, wifi).

Non-functional

- **Correctness under concurrency:** no double booking of a vehicle for overlapping days.

- **Extensibility:** adding a new kind of add-on or a new pricing rule should not change the reservation code (the **Open/Closed Principle**: code should be open to extension by adding new classes, but closed to editing what already works).
- **Auditability:** every charge is a visible bill line, every vehicle event is a log entry.

Use cases

Four **actors** (people or systems that use the software):

- **Receptionist** — adds and edits vehicles and workers; can also reserve vehicles for a customer.
- **Member** — searches the catalogue, reserves, picks up and returns a vehicle.
- **Worker** — handles returned vehicles and updates the vehicle log (cleaning, refuelling, damage).
- **System** — sends notifications (pick-up reminders, overdue alerts, cancellations).

A **use case** is one goal an actor can achieve with the system, such as "reserve a vehicle". Top use cases: add/remove/edit vehicle; search catalogue by type and availability; register or cancel membership; reserve vehicle; check out (pick up) vehicle; return vehicle; add equipment or services; update the vehicle log.

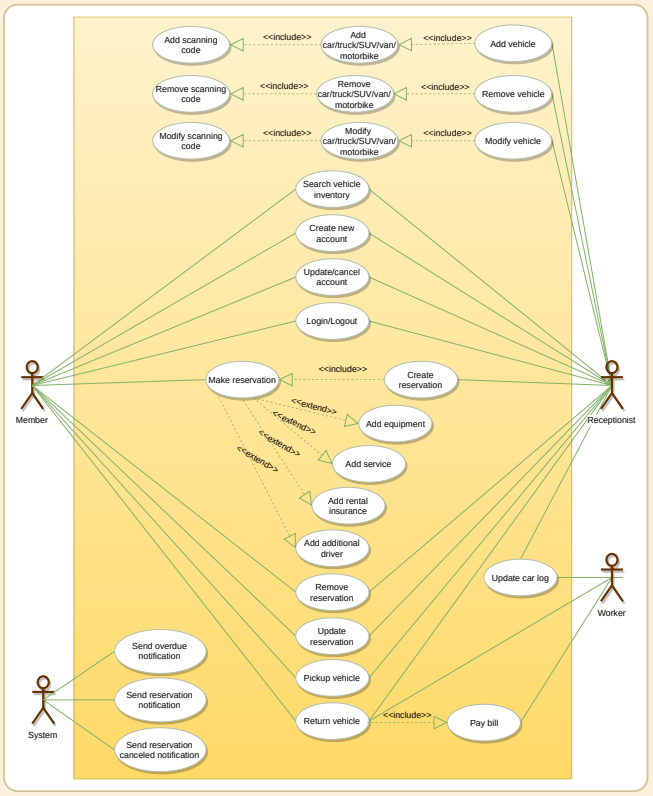


Figure — Use case diagram: receptionist, member, worker and system and the use cases each triggers (course diagram)

A **use case diagram** is a UML (Unified Modeling Language, a standard set of software diagrams) picture: stick figures are actors, ovals are use cases, and lines show who triggers what. In the figure above, notice that reserving is shared by the member and the receptionist, while updating the log belongs to the worker.

Core entities

Class	Responsibility
`CarRentalSystem`	Top-level object; owns the branches and the reservation service.
`CarRentalLocation`	One branch (name, code) with its own `VehicleInventory`.
`VehicleInventory`	Three indexes (lookup maps) over a branch's vehicles: by barcode, by type, by model. Answers "which SUVs are free on these days?".
`Vehicle`	Barcode, plate, make, model, type, seats, parking stall, mileage, **the set of booked days**, and its log.
`VehicleLogEntry`	One event on a vehicle (date, type, note).
`DateRange`	Half-open range of days `[start, end)` (includes `start`, excludes `end`); `end` is the due date.
`Reservation`	The rental: member, vehicle, dates, pick-up and return branch, add-ons, drivers, bill, status. Holds the business rules.
`ReservationAddOn` (+ `RentalInsurance`, `Equipment`, `Service`)	Anything that adds a charge to a rental.
`AdditionalDriver`	A named extra driver on the rental (no charge in our model).
`Bill`, `BillItem`	The one place that knows what a rental costs: a list of lines.
`PricingStrategy`, `LateFeePolicy`	Pluggable (swappable without code changes) rules for the base charge and the late fee.
`ReservationService`	Creates reservations atomically, answers the two "who rented what" questions, runs reminders.
`Notifier`, `PaymentMethod`	Ports (interfaces to outside systems) for sending messages and taking money.

Class diagram

A **class diagram** shows the classes, their fields and methods, and how they relate (owns, refers to, inherits from). The course builds its model in seven stages: money and messages, add-ons, vehicles, search, reservations, actors, and finally the system and its branches.

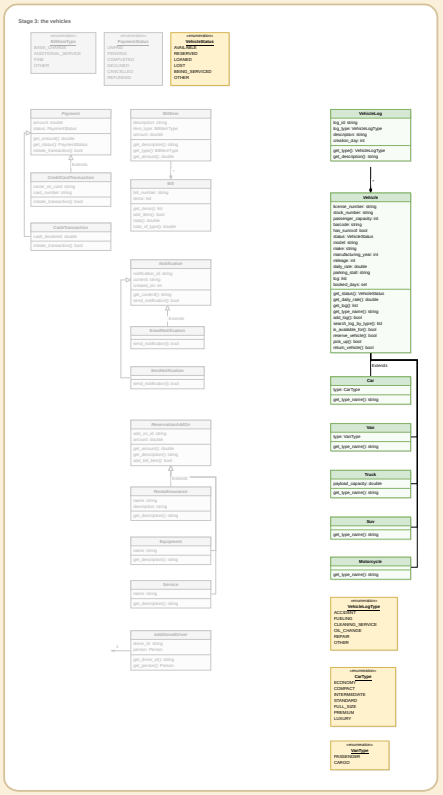


Figure — Vehicle, its log and the vehicle sub-types (course diagram)

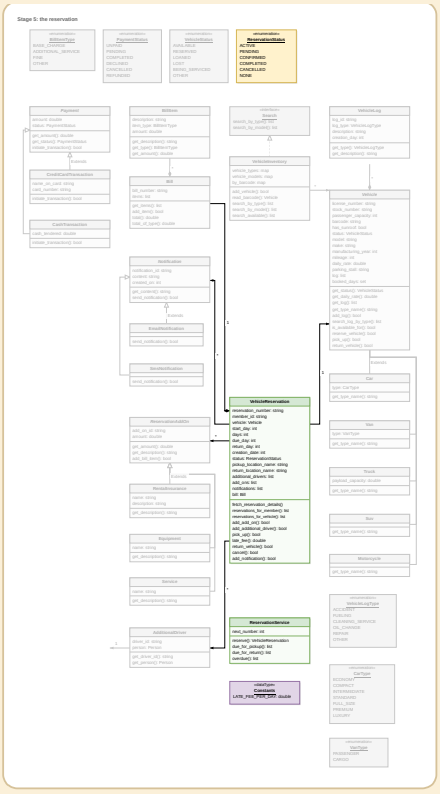


Figure — The reservation owns its bill and refers to the vehicle, add-ons, drivers and notifications (course diagram)

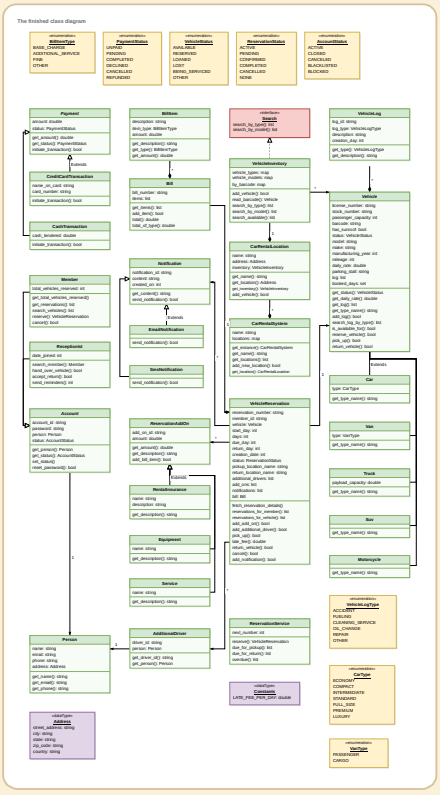
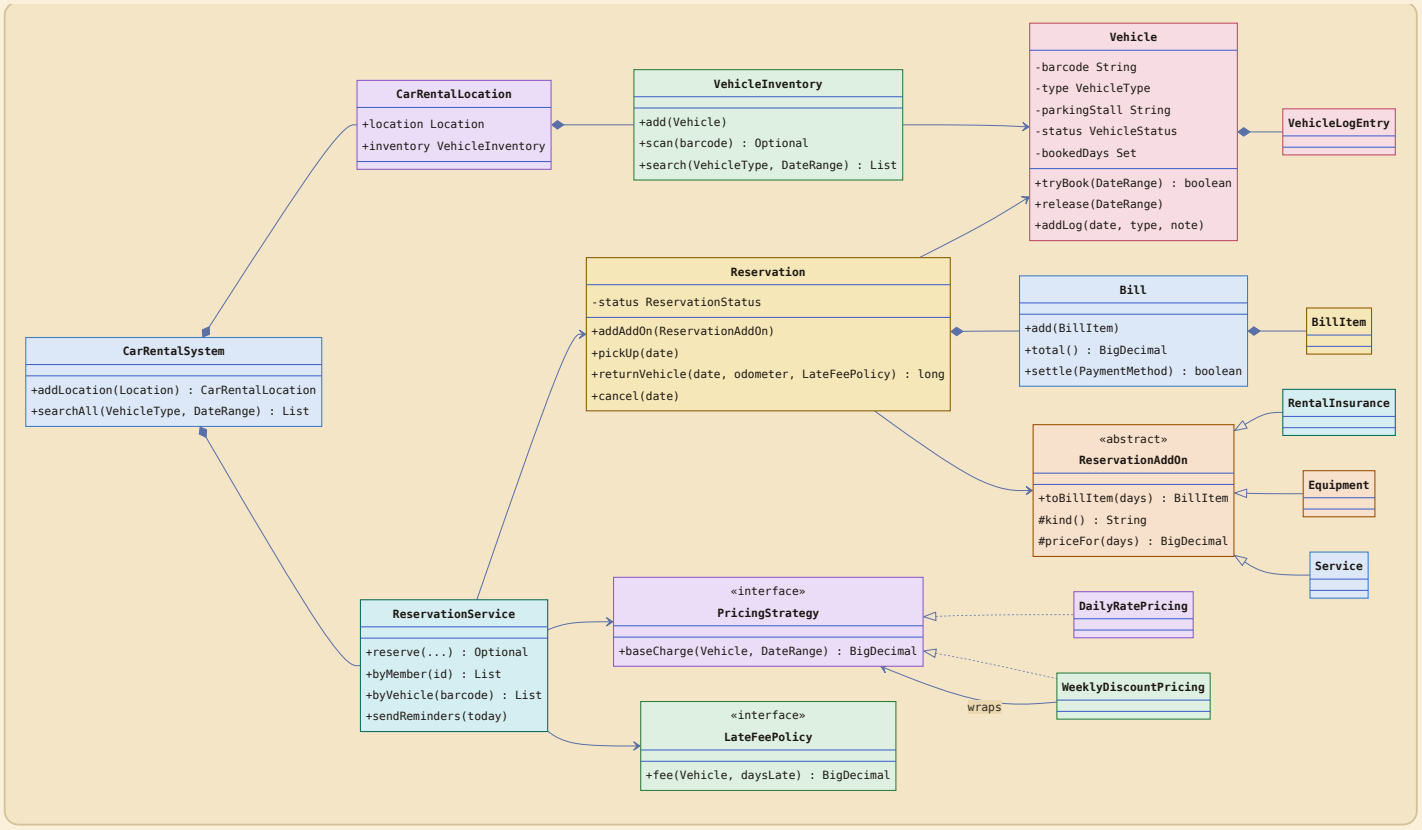


Figure — The finished class diagram of the car rental system (course diagram)

In the finished figure above, two ownership chains matter: the system owns its branches and each branch owns its inventory (filled diamonds, **composition** — the part dies with the whole), while the inventory only *refers to* vehicles (plain arrow, **association**). A reservation owns its bill but only refers to the vehicle.

Our Java design follows the same shape, with two deliberate differences: vehicle kinds are an enum (`VehicleType`) rather than five empty subclasses (classes that inherit from a parent class), because the subclasses add no behaviour; and pricing is a strategy object (an object you plug in to decide one varying rule) rather than a constant.



Key flows

The figures below are **activity diagrams** — flowcharts of one task, with diamonds for decisions. **Pick up a vehicle.** The receptionist scans the customer's licence (or searches by name), checks there is a valid reservation, scans the vehicle's barcode, and hands over the keys. If there is no reservation, the member can reserve a free vehicle on the spot.

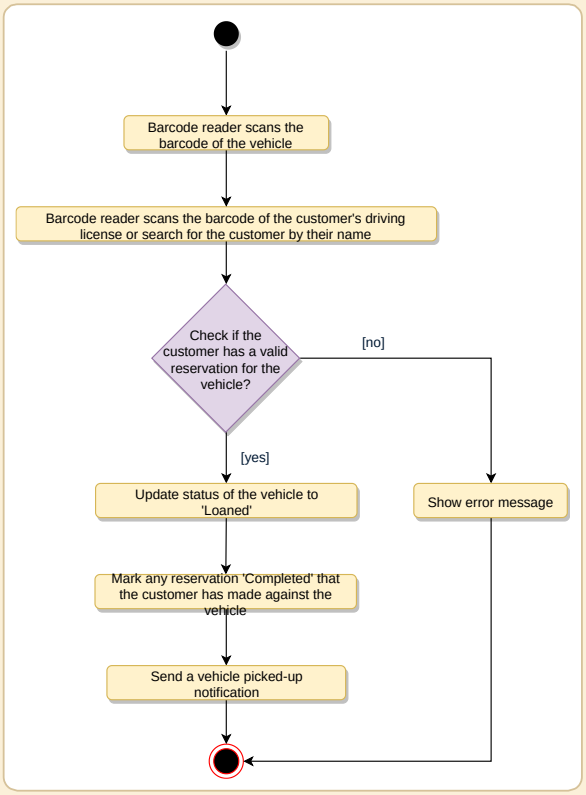


Figure — Activity diagram: picking up a vehicle (course diagram)

Return a vehicle. A worker scans the vehicle barcode; if it is past the due date, the system adds a late fee; the worker checks for damage and fuel; the bill is settled.

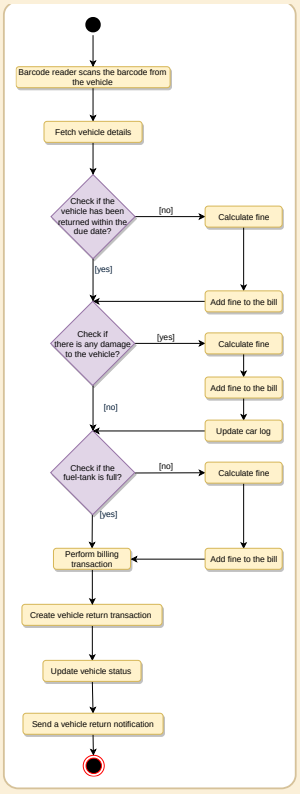
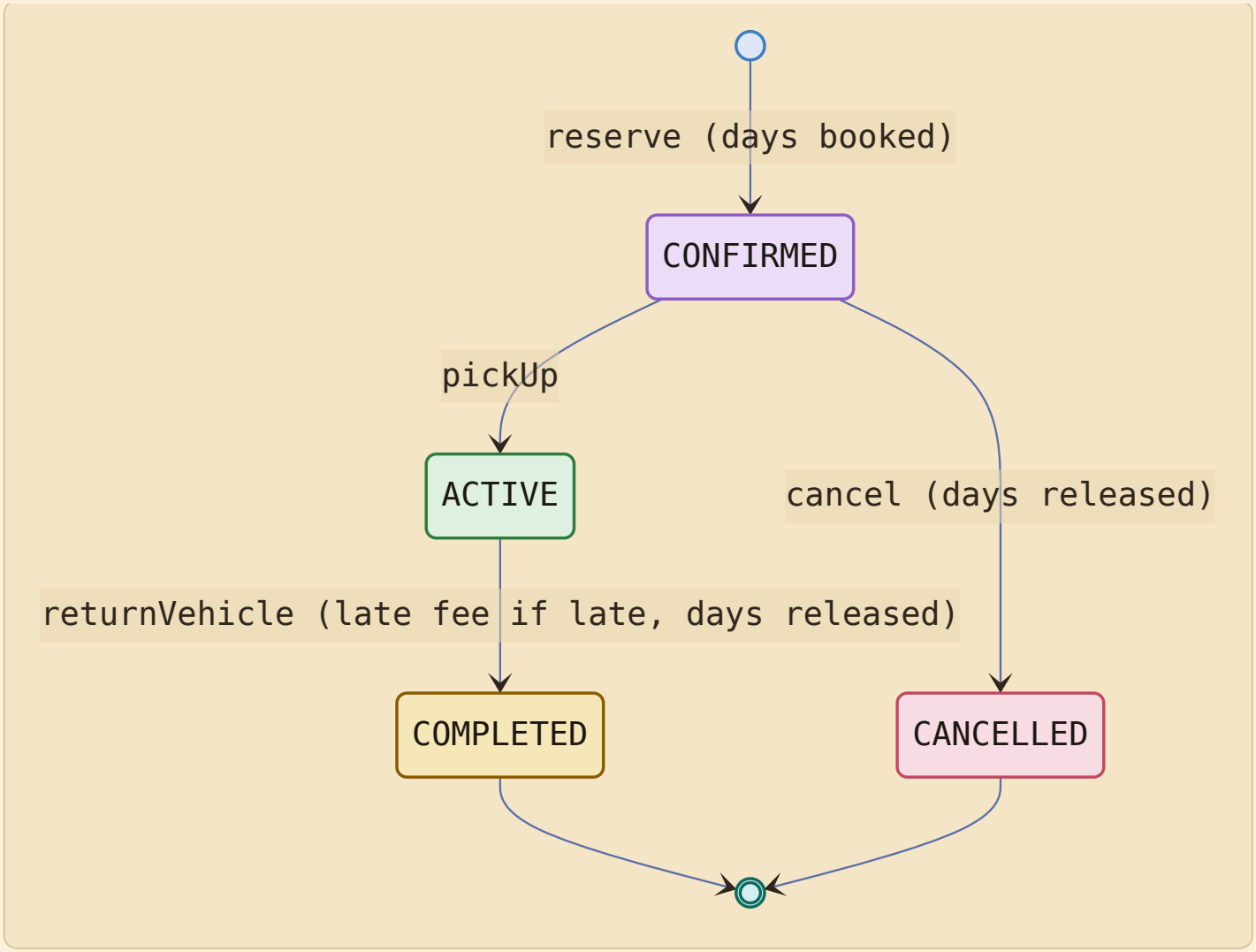


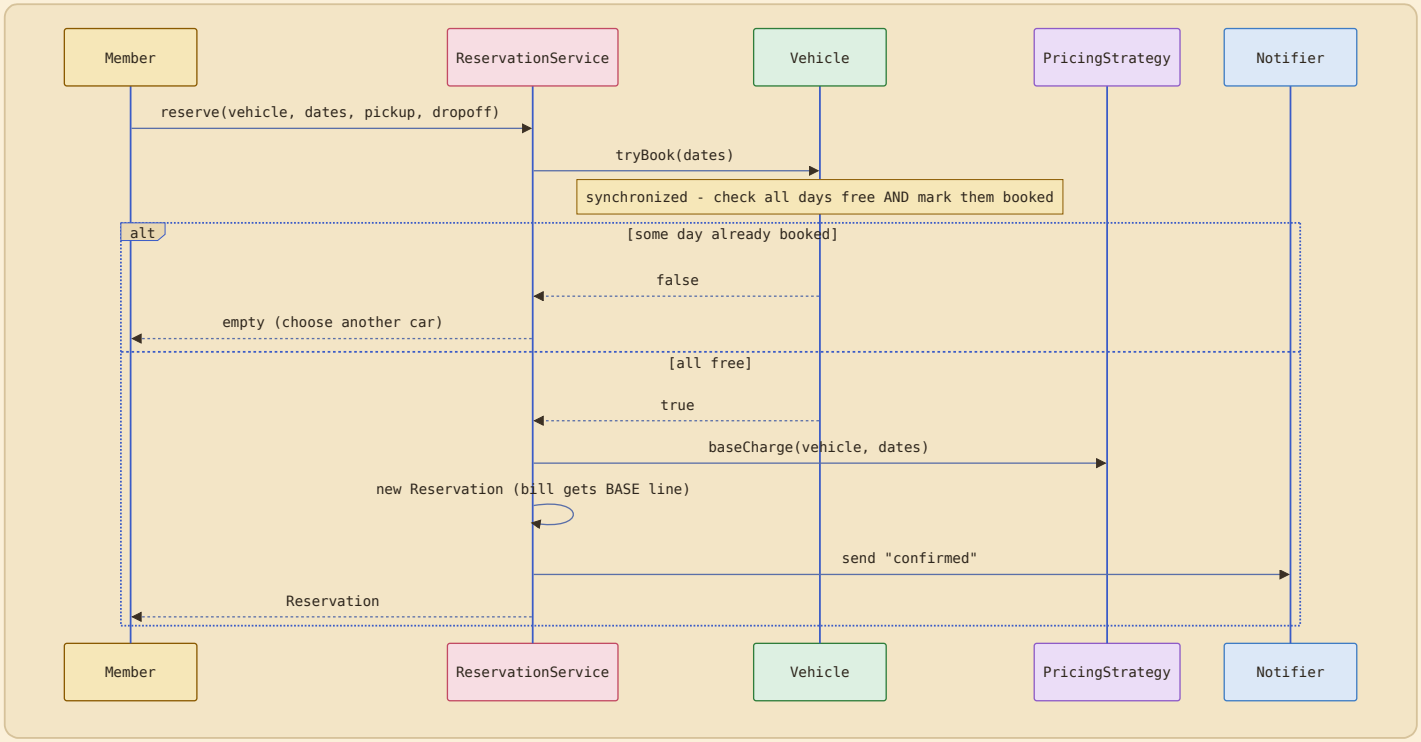
Figure — Activity diagram: returning a vehicle, with the late-fee branch (course diagram)

As the figure above shows, the late-fee decision is a branch on the return path — in our code it is one line inside `Reservation.returnVehicle`.

Reservation lifecycle (a **state machine** — a fixed set of states and the only moves allowed between them):



Reserve flow, including the race we must win (a **sequence diagram**: each column is an object, and arrows are calls in time order from top to bottom):



Design decisions & patterns used

Decision	Why	Alternative rejected
Availability comes from the days each vehicle has already sold, not from a status field	Members ask about future dates ("can I have it from the 10th to the 13th?"). A car that is out on loan right now may be back by then, and a car sitting on the lot today may already be promised to someone else next week.	A `RESERVED` status: cannot answer future-date questions, and a car can only hold one reservation.
Keep `VehicleStatus` too	Staff still need "where is the car right now" (on the lot, loaned, in service, lost).	Dropping status: loses the physical-location answer.
`Vehicle.tryBook` does check **and** book in one `synchronized` method (Java lets only one thread at a time run it on the same object)	Closes the check-then-act race (see below).	Separate `isFree()` then `book()` calls: two members can both see "free".
Template Method (a base class fixes the steps of an algorithm; subclasses fill in some steps) on `ReservationAddOn`	Billing is written once in the `final` (cannot be overridden) method `toBillItem`; subclasses only say what they are (`kind`) and may override the pricing hook (a step meant to be replaced) (`Service` is a flat fee). A fourth add-on is one new subclass.	A type enum with `switch` in the reservation: every new add-on edits the reservation.
Strategy (one interface, interchangeable implementations of a rule) for `PricingStrategy` and `LateFeePolicy`	Seasonal or length-based pricing changes one class, not the reservation.	Hard-coded rate constants.
Decorator (an object that wraps another of the same type and adds behaviour) `WeeklyDiscountPricing` wraps any `PricingStrategy`	Stacks a discount on top of whatever base rule is in place.	A subclass per combination (daily+weekly, seasonal+weekly …).
`Bill` is the only place charges live	Base, add-ons and late fees are lines on one bill; the total can never be computed in two places and disagree.	Each add-on keeping its own running total.
`ReservationService` separate from `Reservation`	Creating and indexing reservations is a different job from holding one reservation's rules (Single Responsibility: one class, one reason to change).	Static factory (a `static` method that builds objects) on `Reservation`.
Actors call the rules, they never compute them	The course's stage 6: a member calls the reservation service (search, reserve, cancel); a receptionist calls the reservation (hand over, take back, take payment, send reminders). An actor that starts working out fees is doing the reservation's job. In our code the actors are plain data (`Member`) and every rule sits in `Reservation` / `ReservationService`.	`Member.reserve()` / `Receptionist.computeLateFee()` methods.
`Bill` methods are `synchronized` and `settle` is idempotent (calling it twice has the same effect as once)	A late-fee line can never be added while the bill is being paid, and a double-clicked "pay" charges once.	Unsynchronized list + a second charge on retry.
The course makes `CarRentalSystem` a **Singleton** (a class allowed only one global instance); we pass it in instead	A singleton is easy to reach but hard to replace in tests. Constructing one and passing it (dependency injection: an object receives what it needs from outside instead of creating or looking it up itself) gives the same "one per app" without a global.	`static getInstance()`.

Java implementation

Package `lld.ch16.carrental`.

```java
// File: code/lld/ch16/carrental/Types.java
package lld.ch16.carrental;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.stream.Stream;

enum VehicleType { CAR, TRUCK, SUV, VAN, MOTORCYCLE }

/** Where the vehicle physically is RIGHT NOW. Future availability is answered by booked days, not by this. */
enum VehicleStatus { AVAILABLE, LOANED, BEING_SERVICED, LOST }

enum ReservationStatus { CONFIRMED, ACTIVE, COMPLETED, CANCELLED }

enum BillItemType { BASE_CHARGE, ADD_ON, LATE_FEE }

enum VehicleLogType { RESERVED, PICKED_UP, RETURNED, CANCELLED, FUELING, CLEANING, DAMAGE, REPAIR }

/** Half-open range [start, end): 10 Oct to 13 Oct is 3 rental days and the car is due back on 13 Oct. */
record DateRange(LocalDate start, LocalDate end) {
    DateRange {
        if (!end.isAfter(start)) throw new IllegalArgumentException("end must be after start");
    }
    long days() { return ChronoUnit.DAYS.between(start, end); }
    Stream<LocalDate> dates() { return start.datesUntil(end); }
    @Override public String toString() { return start + ".." + end; }
}

record Member(String id, String name, String email) {}

record Location(String code, String name) {}

record VehicleLogEntry(LocalDate date, VehicleLogType type, String note) {}

record AdditionalDriver(String name, String licenseNumber) {}
```

```java
// File: code/lld/ch16/carrental/Vehicle.java
package lld.ch16.carrental;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/** One physical vehicle. It owns its booked days and its log. */
final class Vehicle {
    private final String barcode;
    private final String licensePlate;
    private final String make;
    private final String model;
    private final VehicleType type;
    private final int passengerCapacity;
    private final String parkingStall;
    private VehicleStatus status = VehicleStatus.AVAILABLE;
    private int mileage;
    private final Set<LocalDate> bookedDays = new HashSet<>();
    private final List<VehicleLogEntry> log = new ArrayList<>();

    Vehicle(String barcode, String licensePlate, String make, String model,
            VehicleType type, int passengerCapacity, String parkingStall, int mileage) {
        this.barcode = barcode;
        this.licensePlate = licensePlate;
        this.make = make;
        this.model = model;
        this.type = type;
        this.passengerCapacity = passengerCapacity;
        this.parkingStall = parkingStall;
        this.mileage = mileage;
    }

    String barcode() { return barcode; }
    String model() { return model; }
    VehicleType type() { return type; }
    String parkingStall() { return parkingStall; }
    synchronized VehicleStatus status() { return status; }
    synchronized int mileage() { return mileage; }
    synchronized void setStatus(VehicleStatus s) { status = s; }
    synchronized void setMileage(int m) {
        if (m < mileage) throw new IllegalArgumentException("odometer cannot go backwards");
        mileage = m;
    }

    synchronized boolean isFreeFor(DateRange r) {
        return r.dates().noneMatch(bookedDays::contains);
    }

    /** Check AND book inside one lock, so two members can never both win the same days. */
    synchronized boolean tryBook(DateRange r) {
        if (!isFreeFor(r)) return false;
        r.dates().forEach(bookedDays::add);
        return true;
    }

    synchronized void release(DateRange r) { r.dates().forEach(bookedDays::remove); }

    synchronized void addLog(LocalDate date, VehicleLogType type, String note) {
        log.add(new VehicleLogEntry(date, type, note));
    }

    synchronized List<VehicleLogEntry> history() { return List.copyOf(log); }

    String describe() {
        return make + " " + model + " (" + type + ", " + passengerCapacity + " seats, plate " + licensePlate + ")";
```

```
    }
}
```

```java
// File: code/lld/ch16/carrental/Billing.java
package lld.ch16.carrental;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

record BillItem(String description, BillItemType type, BigDecimal amount) {}

/**
 * The ONLY place that knows what a rental costs: every charge becomes a line here.
 * Every method is synchronized on the bill, so a line can never be added while it is being settled.
 */
final class Bill {
    private final List<BillItem> items = new ArrayList<>();
    private boolean paid;

    synchronized void add(BillItem item) {
        if (paid) throw new IllegalStateException("bill already settled");
        items.add(item);
    }
    synchronized List<BillItem> items() { return List.copyOf(items); }
    synchronized BigDecimal total() { return items.stream().map(BillItem::amount).reduce(BigDecimal.ZERO,
BigDecimal::add); }
    synchronized boolean isPaid() { return paid; }

    /** Idempotent: settling a bill that is already paid does not charge the member a second time. */
    synchronized boolean settle(PaymentMethod method) {
        if (paid) return true;
        paid = method.charge(total());
        return paid;
    }
}

/** Strategy for how money is taken. */
interface PaymentMethod { boolean charge(BigDecimal amount); }

record CardPayment(String cardToken) implements PaymentMethod {
    public boolean charge(BigDecimal amount) {
        System.out.println("    charged " + amount + " to card token " + cardToken);
        return true;
    }
}

record CashPayment(BigDecimal tendered) implements PaymentMethod {
    public boolean charge(BigDecimal amount) { return tendered.compareTo(amount) >= 0; }
}

/**
 * Template Method: toBillItem() is fixed; subclasses only say what kind of add-on they are
 * and may override how the price is computed (the hook).
 */
abstract class ReservationAddOn {
    private final String name;
    protected final BigDecimal price;

    protected ReservationAddOn(String name, BigDecimal price) {
        this.name = name;
        this.price = price;
    }

    final BillItem toBillItem(long rentalDays) {
        return new BillItem(kind() + ": " + name, BillItemType.ADD_ON, priceFor(rentalDays));
    }

    protected abstract String kind();

    /** Hook: by default an add-on is charged per rental day. */
```

```
    protected BigDecimal priceFor(long rentalDays) { return price.multiply(BigDecimal.valueOf(rentalDays)); }
}

final class RentalInsurance extends ReservationAddOn {
    RentalInsurance(String name, BigDecimal perDay) { super(name, perDay); }
    protected String kind() { return "Insurance"; }
}

final class Equipment extends ReservationAddOn {
    Equipment(String name, BigDecimal perDay) { super(name, perDay); }
    protected String kind() { return "Equipment"; }
}

/** A service such as roadside assistance or wifi is a flat fee per rental. */
final class Service extends ReservationAddOn {
    Service(String name, BigDecimal flatFee) { super(name, flatFee); }
    protected String kind() { return "Service"; }
    @Override protected BigDecimal priceFor(long rentalDays) { return price; }
}
```

```
// File: code/lld/ch16/carrental/Pricing.java
package lld.ch16.carrental;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.EnumMap;
import java.util.Map;

/** Strategy: how the base charge for a rental is computed. */
interface PricingStrategy { BigDecimal baseCharge(Vehicle v, DateRange period); }

final class DailyRatePricing implements PricingStrategy {
    private final Map<VehicleType, BigDecimal> dailyRate;
    DailyRatePricing(Map<VehicleType, BigDecimal> dailyRate) { this.dailyRate = new EnumMap<>(dailyRate); }
    public BigDecimal baseCharge(Vehicle v, DateRange p) {
        return dailyRate.get(v.type()).multiply(BigDecimal.valueOf(p.days()));
    }
}

/** Decorator over any pricing: 10% off rentals of 7 days or more. */
final class WeeklyDiscountPricing implements PricingStrategy {
    private final PricingStrategy inner;
    WeeklyDiscountPricing(PricingStrategy inner) { this.inner = inner; }
    public BigDecimal baseCharge(Vehicle v, DateRange p) {
        BigDecimal base = inner.baseCharge(v, p);
        return p.days() >= 7 ? base.multiply(new BigDecimal("0.90")).setScale(2, RoundingMode.HALF_UP) : base;
    }
}

/** Strategy: what a late return costs. */
interface LateFeePolicy { BigDecimal fee(Vehicle v, long daysLate); }

record FlatLateFee(BigDecimal perDay) implements LateFeePolicy {
    public BigDecimal fee(Vehicle v, long daysLate) { return perDay.multiply(BigDecimal.valueOf(daysLate)); }
}

/** Observer-style port: the service announces events, the notifier decides the channel. */
interface Notifier { void send(Member to, String message); }

final class ConsoleNotifier implements Notifier {
    public void send(Member to, String message) { System.out.println("    [notify " + to.email() + "] " + message);
}
}
```

```java
// File: code/lld/ch16/carrental/Reservation.java
package lld.ch16.carrental;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;

/** One rental. The rules (add-ons, late fee, freeing days) live here, not in the actors. */
final class Reservation {
    private final String id;
    private final Member member;
    private final Vehicle vehicle;
    private final DateRange period;
    private final Location pickupLocation;
    private final Location returnLocation;
    private final Bill bill = new Bill();
    private final List<ReservationAddOn> addOns = new ArrayList<>();
    private final List<AdditionalDriver> drivers = new ArrayList<>();
    private ReservationStatus status = ReservationStatus.CONFIRMED;

    Reservation(String id, Member member, Vehicle vehicle, DateRange period,
                Location pickup, Location dropoff, BigDecimal baseCharge) {
        this.id = id;
        this.member = member;
        this.vehicle = vehicle;
        this.period = period;
        this.pickupLocation = pickup;
        this.returnLocation = dropoff;
        bill.add(new BillItem("Base: " + vehicle.model() + " x " + period.days() + " days",
                BillItemType.BASE_CHARGE, baseCharge));
    }

    String id() { return id; }
    Member member() { return member; }
    Vehicle vehicle() { return vehicle; }
    DateRange period() { return period; }
    Location pickupLocation() { return pickupLocation; }
    Location returnLocation() { return returnLocation; }
    Bill bill() { return bill; }
    synchronized List<AdditionalDriver> drivers() { return List.copyOf(drivers); }
    synchronized ReservationStatus status() { return status; }

    private void require(ReservationStatus expected, String action) {
        if (status != expected)
            throw new IllegalStateException("cannot " + action + " reservation " + id + " in state " + status);
    }

    synchronized void addAddOn(ReservationAddOn addOn) {
        require(ReservationStatus.CONFIRMED, "add an add-on to");
        addOns.add(addOn);
        bill.add(addOn.toBillItem(period.days()));
    }

    synchronized void addDriver(AdditionalDriver d) {
        require(ReservationStatus.CONFIRMED, "add a driver to");
        drivers.add(d);
    }

    synchronized void pickUp(LocalDate on) {
        require(ReservationStatus.CONFIRMED, "pick up");
        status = ReservationStatus.ACTIVE;
        vehicle.setStatus(VehicleStatus.LOANED);
        vehicle.addLog(on, VehicleLogType.PICKED_UP, id + " by " + member.name() + " at " + pickupLocation.code());
    }

    /** Returns the number of late days charged. The due date is period.end(). */
```

```
    synchronized long returnVehicle(LocalDate on, int odometer, LateFeePolicy lateFee) {
        require(ReservationStatus.ACTIVE, "return");
        long daysLate = Math.max(0, ChronoUnit.DAYS.between(period.end(), on));
        if (daysLate > 0) {
            bill.add(new BillItem("Late fee: " + daysLate + " day(s)", BillItemType.LATE_FEE,
                    lateFee.fee(vehicle, daysLate)));
        }
        vehicle.setMileage(odometer);
        vehicle.setStatus(VehicleStatus.AVAILABLE);
        vehicle.release(period);
        vehicle.addLog(on, VehicleLogType.RETURNED, id + " at " + returnLocation.code() + ", odometer " +
odometer);
        status = ReservationStatus.COMPLETED;
        return daysLate;
    }

    synchronized void cancel(LocalDate on) {
        require(ReservationStatus.CONFIRMED, "cancel");
        status = ReservationStatus.CANCELLED;
        vehicle.release(period);   // a cancelled rental must give its days back
        vehicle.addLog(on, VehicleLogType.CANCELLED, id);
    }
}
```

```java
// File: code/lld/ch16/carrental/RentalSystem.java
package lld.ch16.carrental;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

/** Per-branch indexes over vehicles. All three are filled by add(), so they cannot drift apart. */
final class VehicleInventory {
    private final Map<String, Vehicle> byBarcode = new ConcurrentHashMap<>();
    private final Map<VehicleType, List<Vehicle>> byType = new ConcurrentHashMap<>();
    private final Map<String, List<Vehicle>> byModel = new ConcurrentHashMap<>();

    void add(Vehicle v) {
        if (byBarcode.putIfAbsent(v.barcode(), v) != null)
            throw new IllegalArgumentException("duplicate barcode " + v.barcode());
        byType.computeIfAbsent(v.type(), t -> new CopyOnWriteArrayList<>()).add(v);
        byModel.computeIfAbsent(v.model(), m -> new CopyOnWriteArrayList<>()).add(v);
    }

    Optional<Vehicle> scan(String barcode) { return Optional.ofNullable(byBarcode.get(barcode)); }

    List<Vehicle> search(VehicleType type, DateRange period) {
        return byType.getOrDefault(type, List.of()).stream()
                .filter(v -> v.status() != VehicleStatus.LOST && v.isFreeFor(period)).toList();
    }

    List<Vehicle> searchByModel(String model, DateRange period) {
        return byModel.getOrDefault(model, List.of()).stream().filter(v -> v.isFreeFor(period)).toList();
    }
}

record CarRentalLocation(Location location, VehicleInventory inventory) {}

/** Creates and tracks reservations. Answers "what did this member rent" and "who took this car". */
final class ReservationService {
    private final PricingStrategy pricing;
    private final LateFeePolicy lateFee;
    private final Notifier notifier;
    private final Map<String, Reservation> reservations = new ConcurrentHashMap<>();
    private final AtomicInteger seq = new AtomicInteger(1000);

    ReservationService(PricingStrategy pricing, LateFeePolicy lateFee, Notifier notifier) {
        this.pricing = pricing;
        this.lateFee = lateFee;
        this.notifier = notifier;
    }

    Optional<Reservation> reserve(Member m, Vehicle v, DateRange period, Location pickup, Location dropoff,
LocalDate today) {
        if (!v.tryBook(period)) return Optional.empty();
        var r = new Reservation("R" + seq.incrementAndGet(), m, v, period, pickup, dropoff, pricing.baseCharge(v,
period));
        reservations.put(r.id(), r);
        v.addLog(today, VehicleLogType.RESERVED, r.id() + " for " + period);
        notifier.send(m, "Reservation " + r.id() + " confirmed: " + v.model() + " " + period);
        return Optional.of(r);
    }

    Reservation get(String id) {
        var r = reservations.get(id);
        if (r == null) throw new IllegalArgumentException("no reservation " + id);
        return r;
```

```
    }

    void cancel(String id, LocalDate on) {
        var r = get(id);
        r.cancel(on);
        notifier.send(r.member(), "Reservation " + id + " cancelled");
    }

    void pickUp(String id, LocalDate on) { get(id).pickUp(on); }

    void returnVehicle(String id, LocalDate on, int odometer) {
        var r = get(id);
        long late = r.returnVehicle(on, odometer, lateFee);
        if (late > 0) notifier.send(r.member(), "Returned " + late + " day(s) late, late fee added to " + id);
    }

    List<Reservation> byMember(String memberId) {
        return reservations.values().stream().filter(r -> r.member().id().equals(memberId)).toList();
    }

    List<Reservation> byVehicle(String barcode) {
        return reservations.values().stream().filter(r -> r.vehicle().barcode().equals(barcode)).toList();
    }

    /** Requirement: remind before pick-up, before the due date, and when overdue. Run once a day by a scheduler.
*/
    void sendReminders(LocalDate today) {
        for (var r : reservations.values()) {
            var status = r.status();
            if (status == ReservationStatus.CONFIRMED && r.period().start().equals(today.plusDays(1)))
                notifier.send(r.member(), "Pick-up tomorrow for " + r.id() + " at " + r.pickupLocation().name());
            else if (status == ReservationStatus.ACTIVE && r.period().end().equals(today.plusDays(1)))
                notifier.send(r.member(), "Vehicle for " + r.id() + " is due tomorrow");
            else if (status == ReservationStatus.ACTIVE && r.period().end().isBefore(today))
                notifier.send(r.member(), "OVERDUE: " + r.id() + " was due " + r.period().end());
        }
    }
}

/** Top-level object: owns branches (each owning its inventory) and the reservation service. */
final class CarRentalSystem {
    private final Map<String, CarRentalLocation> locations = new ConcurrentHashMap<>();
    private final ReservationService reservations;

    CarRentalSystem(ReservationService reservations) { this.reservations = reservations; }

    CarRentalLocation addLocation(Location l) {
        return locations.computeIfAbsent(l.code(), c -> new CarRentalLocation(l, new VehicleInventory()));
    }
    CarRentalLocation location(String code) { return locations.get(code); }
    ReservationService reservations() { return reservations; }

    List<Vehicle> searchAll(VehicleType type, DateRange period) {
        List<Vehicle> out = new ArrayList<>();
        locations.values().forEach(l -> out.addAll(l.inventory().search(type, period)));
        return out;
    }
}
```

```java
// File: code/lld/ch16/carrental/Main.java
package lld.ch16.carrental;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.Map;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static BigDecimal usd(String s) { return new BigDecimal(s); }

    public static void main(String[] args) throws Exception {
        var pricing = new WeeklyDiscountPricing(new DailyRatePricing(Map.of(
                VehicleType.CAR, usd("50.00"), VehicleType.SUV, usd("80.00"), VehicleType.TRUCK, usd("100.00"),
                VehicleType.VAN, usd("90.00"), VehicleType.MOTORCYCLE, usd("35.00"))));
        var service = new ReservationService(pricing, new FlatLateFee(usd("40.00")), new ConsoleNotifier());
        var system = new CarRentalSystem(service);

        var sfo = new Location("SFO", "SFO Airport");
        var dtn = new Location("DTN", "Downtown");
        var sfoInv = system.addLocation(sfo).inventory();
        sfoInv.add(new Vehicle("VH-001", "7ABC123", "Toyota", "RAV4", VehicleType.SUV, 5, "A-12", 20_100));
        sfoInv.add(new Vehicle("VH-002", "7ABC124", "Honda", "CR-V", VehicleType.SUV, 5, "A-13", 15_400));
        sfoInv.add(new Vehicle("VH-003", "8XYZ001", "Honda", "Civic", VehicleType.CAR, 5, "B-02", 9_800));
        system.addLocation(dtn).inventory().add(new Vehicle("VH-010", "9VAN555", "Ford", "Transit",
VehicleType.VAN, 12, "C-01", 30_000));

        var alice = new Member("M1", "Alice", "alice@example.com");
        var today = LocalDate.of(2026, 10, 1);
        var trip = new DateRange(LocalDate.of(2026, 10, 10), LocalDate.of(2026, 10, 13));

        System.out.println("1) Search SUVs at SFO for " + trip);
        sfoInv.search(VehicleType.SUV, trip).forEach(v -> System.out.println("   " + v.barcode() + " " +
v.describe() + " stall " + v.parkingStall()));

        System.out.println("2) Alice reserves VH-001 (pick up SFO, return Downtown) and adds extras");
        var rav4 = sfoInv.scan("VH-001").orElseThrow();
        var r1 = service.reserve(alice, rav4, trip, sfo, dtn, today).orElseThrow();
        r1.addAddOn(new RentalInsurance("Collision damage waiver", usd("15.00")));
        r1.addAddOn(new Equipment("Child seat", usd("8.00")));
        r1.addAddOn(new Service("Roadside assistance", usd("25.00")));
        r1.addDriver(new AdditionalDriver("Sam", "D1234567"));

        System.out.println("3) 8 members race for VH-002 on the same dates");
        var crv = sfoInv.scan("VH-002").orElseThrow();
        var wins = new AtomicInteger();
        var winnerId = new String[1];
        var start = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            for (int i = 0; i < 8; i++) {
                var m = new Member("M" + (10 + i), "Racer" + i, "racer" + i + "@example.com");
                pool.submit(() -> {
                    start.await();
                    service.reserve(m, crv, new DateRange(LocalDate.of(2026, 10, 11), LocalDate.of(2026, 10, 14)),
sfo, sfo, today)
                            .ifPresent(r -> { wins.incrementAndGet(); synchronized (winnerId) { winnerId[0] =
r.id(); } });
                    return null;
                });
            }
            start.countDown();
        }
        System.out.println("   successful reservations for VH-002: " + wins.get());
        System.out.println("   SUVs free for " + trip + " now: " + sfoInv.search(VehicleType.SUV, trip).size());
```

```
        System.out.println("4) Daily reminder job on 2026-10-09");
        service.sendReminders(LocalDate.of(2026, 10, 9));

        System.out.println("5) Pick up on 10 Oct, return at Downtown on 15 Oct (due 13 Oct)");
        service.pickUp(r1.id(), LocalDate.of(2026, 10, 10));
        System.out.println("   VH-001 status while out: " + rav4.status());
        service.sendReminders(LocalDate.of(2026, 10, 14));
        service.returnVehicle(r1.id(), LocalDate.of(2026, 10, 15), 20_640);
        rav4.addLog(LocalDate.of(2026, 10, 15), VehicleLogType.CLEANING, "interior clean by worker W7");

        System.out.println("6) Alice's bill");
        r1.bill().items().forEach(i -> System.out.printf("   %-40s %-10s %8s%n", i.description(), i.type(),
i.amount()));
        System.out.println("   TOTAL " + r1.bill().total());
        r1.bill().settle(new CardPayment("tok_alice_visa"));
        System.out.println("   paid: " + r1.bill().isPaid() + ", settle again (double click) -> "
                + r1.bill().settle(new CardPayment("tok_alice_visa")) + ", no second charge");

        System.out.println("7) Who rented VH-001? What has Alice rented?");
        service.byVehicle("VH-001").forEach(r -> System.out.println("   " + r.id() + " -> " + r.member().name() + "
" + r.period() + " " + r.status()));
        service.byMember("M1").forEach(r -> System.out.println("   Alice: " + r.id() + " " + r.vehicle().model()));

        System.out.println("8) Vehicle log for VH-001");
        rav4.history().forEach(e -> System.out.println("   " + e.date() + " " + e.type() + " " + e.note()));

        System.out.println("9) The VH-002 winner cancels; the days come back");
        service.cancel(winnerId[0], LocalDate.of(2026, 10, 2));
        System.out.println("   SUVs free for " + trip + ": " + sfoInv.search(VehicleType.SUV, trip).size());

        System.out.println("10) A 7-day Civic rental gets the weekly discount");
        var week = new DateRange(LocalDate.of(2026, 11, 1), LocalDate.of(2026, 11, 8));
        var r3 = service.reserve(alice, sfoInv.scan("VH-003").orElseThrow(), week, sfo, sfo, today).orElseThrow();
        System.out.println("   base charge " + r3.bill().total() + " (7 x 50 = 350, minus 10%)");

        System.out.println("11) Rules are enforced");
        try { r1.addAddOn(new Equipment("Ski rack", usd("10.00"))); }
        catch (IllegalStateException e) { System.out.println("   rejected: " + e.getMessage()); }
        try { new DateRange(LocalDate.of(2026, 10, 5), LocalDate.of(2026, 10, 5)); }
        catch (IllegalArgumentException e) { System.out.println("   rejected: " + e.getMessage()); }
    }
}
```

Output:

```
1) Search SUVs at SFO for 2026-10-10..2026-10-13
   VH-001 Toyota RAV4 (SUV, 5 seats, plate 7ABC123) stall A-12
   VH-002 Honda CR-V (SUV, 5 seats, plate 7ABC124) stall A-13
2) Alice reserves VH-001 (pick up SFO, return Downtown) and adds extras
    [notify alice@example.com] Reservation R1001 confirmed: RAV4 2026-10-10..2026-10-13
3) 8 members race for VH-002 on the same dates
    [notify racer7@example.com] Reservation R1002 confirmed: CR-V 2026-10-11..2026-10-14
   successful reservations for VH-002: 1
   SUVs free for 2026-10-10..2026-10-13 now: 0
4) Daily reminder job on 2026-10-09
    [notify alice@example.com] Pick-up tomorrow for R1001 at SFO Airport
5) Pick up on 10 Oct, return at Downtown on 15 Oct (due 13 Oct)
   VH-001 status while out: LOANED
    [notify alice@example.com] OVERDUE: R1001 was due 2026-10-13
    [notify alice@example.com] Returned 2 day(s) late, late fee added to R1001
6) Alice's bill
   Base: RAV4 x 3 days                      BASE_CHARGE   240.00
   Insurance: Collision damage waiver       ADD_ON        45.00
   Equipment: Child seat                    ADD_ON        24.00
   Service: Roadside assistance             ADD_ON        25.00
   Late fee: 2 day(s)                       LATE_FEE      80.00
   TOTAL 414.00
    charged 414.00 to card token tok_alice_visa
   paid: true, settle again (double click) -> true, no second charge
7) Who rented VH-001? What has Alice rented?
   R1001 -> Alice 2026-10-10..2026-10-13 COMPLETED
   Alice: R1001 RAV4
8) Vehicle log for VH-001
   2026-10-01 RESERVED R1001 for 2026-10-10..2026-10-13
   2026-10-10 PICKED_UP R1001 by Alice at SFO
   2026-10-15 RETURNED R1001 at DTN, odometer 20640
   2026-10-15 CLEANING interior clean by worker W7
9) The VH-002 winner cancels; the days come back
    [notify racer7@example.com] Reservation R1002 cancelled
   SUVs free for 2026-10-10..2026-10-13: 2
10) A 7-day Civic rental gets the weekly discount
    [notify alice@example.com] Reservation R1003 confirmed: Civic 2026-11-01..2026-11-08
   base charge 315.00 (7 x 50 = 350, minus 10%)
11) Rules are enforced
   rejected: cannot add an add-on to reservation R1001 in state COMPLETED
   rejected: end must be after start
```

What to notice in the output: step 3 has eight threads racing for one car and exactly one wins (which racer wins varies per run); step 5 shows the overdue reminder and the 2-day late fee (2 × 40.00); step 6 shows that settling an already-paid bill does not charge again; step 9 shows cancellation giving the days back.

Concurrency & edge cases

> **Key term — lock:** A **lock** (also called a mutex) is a token only one thread can hold at a time; any other thread that wants it waits until it is released. In Java, a `synchronized` method uses the object itself as the lock. A *global lock* guards everything with one token (simple, but everyone queues), while a *per-object lock* (one per vehicle here) lets work on different objects run in parallel.

- **Double booking.** The course lists this as an open gap in its own design (checking the days and booking them are two separate steps there). Two members call `reserve` for the same car at the same moment. Because `tryBook` is `synchronized` on the vehicle, the check "are all days free?" and the write "mark them booked" happen as one step, so the second caller sees the first caller's days. Locking per vehicle (not one global lock) lets different cars be booked in parallel. In a database-backed system the same idea becomes a unique constraint on `(vehicle_id, day)` (the database refuses a second row with the same pair) or a `SELECT … FOR UPDATE` (a read that also locks the rows it returns) in one transaction (a group of database changes that succeed or fail together).
- **Search is advisory.** `search` may say a car is free and then `reserve` fails because someone else won. That is correct: the client (the app or web page the member is using) simply shows "no longer available". Never

reserve based on a stale (out-of-date) search result without re-checking.

- **Late return collides with the next rental.** If Alice returns two days late and Bob booked the car from the 14th, Bob's car is not on the lot. Real systems keep a buffer (for example a cleaning day) between bookings and, when a car is overdue, reassign the next reservation to another car of the same type.
- **Early return.** We release the whole booked range; the unused days become bookable. Whether the member is refunded is a policy question.
- **Cancelling after pick-up** is rejected by the state machine (`CONFIRMED` only).
- **Add-ons after the rental ends** are rejected (see step 11 of the output).
- **Invalid ranges** (end not after start) are rejected in the `DateRange` constructor.
- **Odometer going backwards** is rejected in `Vehicle.setMileage`.
- **Different return branch.** The reservation records the return branch, but the vehicle stays in the pick-up branch's inventory. The course calls this out as a known gap: moving a car between branches is a *transfer* the design does not model yet (see Extensions).

Extensions & follow-up questions

- **"How do you handle one-way rentals?"** — On return at a different branch, remove the vehicle from the old `VehicleInventory` and add it to the new one inside `returnVehicle`, and log a `TRANSFER` event. Add a one-way fee as another `BillItem`.
- **"Seasonal or dynamic pricing?"** — Add a `SeasonalPricing` strategy (rate per date) or wrap with another decorator (`SurgePricing`). The reservation does not change.
- **"Deposits and card holds?"** — A card *hold* reserves money on the card without taking it yet. A `PaymentMethod.authorize(amount)` at pick-up and `capture(amount)` at return; the difference is released.
- **"Mileage limits, fuel level, damage claims?"** — Record odometer and fuel at pick-up and return; add `BillItem`s of type `FINE` for extra miles, missing fuel or damage (the course's enum has a `FINE` bill item type for exactly this kind of charge).
- **"How do reminders run?"** — A scheduler (for example a `ScheduledExecutorService`, Java's built-in timer for repeating tasks, or a cron job, an operating-system entry that runs a command on a fixed timetable) calls `sendReminders(today)` once a day.
- **"How would you scale this?"** — Partition vehicles by branch (split the data so each branch's vehicles live on their own database or server); the booked-days table becomes `(vehicle_id, day)` with a unique key; searches go to a read replica (a read-only copy of the database kept in sync with the main one) or a search index (a separate store built for fast lookups, such as Elasticsearch); reservations are written with a transaction. These scaling ideas are explained in System Design Fundamentals.
- **"How do workers update the log?"** — `Vehicle.addLog(date, CLEANING, note)` as in step 5; restrict who can call it by role in the service layer (the classes, like `ReservationService`, that sit between callers and the domain objects and enforce who may do what).

In the interview

30-second pitch: "Branches own inventories, inventories index vehicles by barcode, type and model. The key decision: availability is a question about *dates*, so each vehicle keeps its booked days, and `tryBook` checks and books them atomically. A reservation owns a bill; every charge — base, insurance, equipment, services, late fee — is a line on it. Add-ons share a template-method base class; pricing and late fees are strategies."

Suggested 40-minute plan:

Minutes	Do
0–5	Clarify vehicle types, branches, one-way, add-ons, late fees.
5–10	Actors and use cases; list the entities.
10–20	Class diagram. Say "availability is dates, not status" early.
20–35	Code `Vehicle.tryBook`, `Reservation` (state machine + bill + late fee), `ReservationService.reserve`.
35–40	Concurrency, late-return collision, one-way transfers.

Likely follow-ups: "Can I book it for next month while it is loaned now?" (yes — dates, not status). "Two people click reserve together?" (atomic `tryBook`). "Add a new add-on?" (one subclass).

Self-check

1. **Q:** Why is a `RESERVED` status not enough to answer "is this car free next week?" **A:** Status describes *now*. A car can be loaned today and free next week, or on the lot today but promised next week. Only per-day bookings answer date questions.
2. **Q:** What pattern does `ReservationAddOn.toBillItem` use and what is the "hook"? **A:** Template Method; `toBillItem` is the fixed algorithm and `priceFor` is the overridable hook (`Service` overrides it to a flat fee).
3. **Q:** Why is `tryBook` one method rather than `isFree` + `book`? **A:** To make check-then-act atomic under the vehicle's lock, so two threads cannot both see "free".
4. **Q:** How is the late fee computed? **A:** Days between the due date (`period.end`) and the actual return date, times the per-day fee from `LateFeePolicy`, added as a `LATE_FEE` bill line.
5. **Q:** Which class answers "who rented VH-001"? **A:** `ReservationService.byVehicle`, from the one list of reservations.

TL;DR

- Availability = booked **days** per vehicle; status = where it is now.
- `tryBook` makes check-and-book atomic per vehicle.
- A reservation owns one bill; every charge is a line. Late fee = days late × rate.
- Template Method for add-ons, Strategy (+ Decorator) for pricing.
- Known gaps to mention: branch transfers, buffer between rentals.

Real-world example. Rental companies such as Hertz and Enterprise let you book a *car class* ("intermediate SUV or similar") rather than a specific plate; the exact vehicle is assigned at the counter. That is the same design one level up: availability is counted per type per day, and a specific `Vehicle` is bound at pick-up. Their counters and return lanes scan the car and compute extra-day and fuel charges on the spot — the "return" activity above.

Design a Restaurant Management System

Figure — A restaurant with branches, tables, waiters and a kitchen (course diagram)

Problem & clarifying questions

A **restaurant management system** runs the day-to-day of a restaurant from one place: which tables are free, table reservations, taking orders, sending them to the kitchen, and producing and settling the bill.

Question	Assumed answer
One restaurant or a chain?	A restaurant with several branches; each branch has its own menu, tables and staff.
Menu structure?	Menu → sections (Starters, Mains …) → items with prices.
How are orders taken?	A waiter creates an order for a table and adds meals **per seat**; each meal has meal items, each pointing at a menu item.
Reservations?	Receptionist searches tables by date/time and party size and reserves one; customers can cancel; the system reminds them before the time.
Walk-ins?	Yes: show tables free right now.
Seating layouts?	A branch can have several seating arrangements (for example "Weekday" and "Banquet").
Payment?	Credit card, cheque or cash.
Kitchen?	Orders go to a chef queue; chefs cook them in order and the waiter is told when food is ready.

Requirements

Functional

1. The restaurant has several branches.
2. Each branch has a menu.
3. A menu has sections; sections have menu items.
4. A waiter creates an order for a table and adds meals for each seat.
5. Each meal has meal items; each meal item corresponds to a menu item.
6. Show which tables are free now for walk-in customers.
7. Support table reservations.
8. The receptionist searches free tables by date and time and reserves one.
9. Customers can cancel a reservation.
10. Send a notification when the reservation time is approaching.
11. Customers pay by credit card, cheque or cash.
12. A branch can have several seating arrangements of tables.
13. (Added) Orders go to a kitchen queue; chefs take them one at a time and the waiter is notified when an order is ready.

Non-functional

- No two parties may be given the same table for overlapping time.
- Prices live in one place (the menu item) so an order and the menu can never disagree.
- Easy to add a new payment type or a new chef-assignment rule.

Use cases

Actors:

- **Receptionist** — adds and edits tables and layouts; creates and cancels reservations; checks guests in.
- **Waiter** — takes and changes orders.
- **Manager** — hires staff and edits the menu.
- **Chef** — views and cooks orders.
- **Cashier** — produces the check (bill) and takes payment.
- **System** — sends reservation reminders and cancellations.

Top use cases: add/modify tables; search tables; place order; update order; create reservation; cancel reservation; check in; make payment.

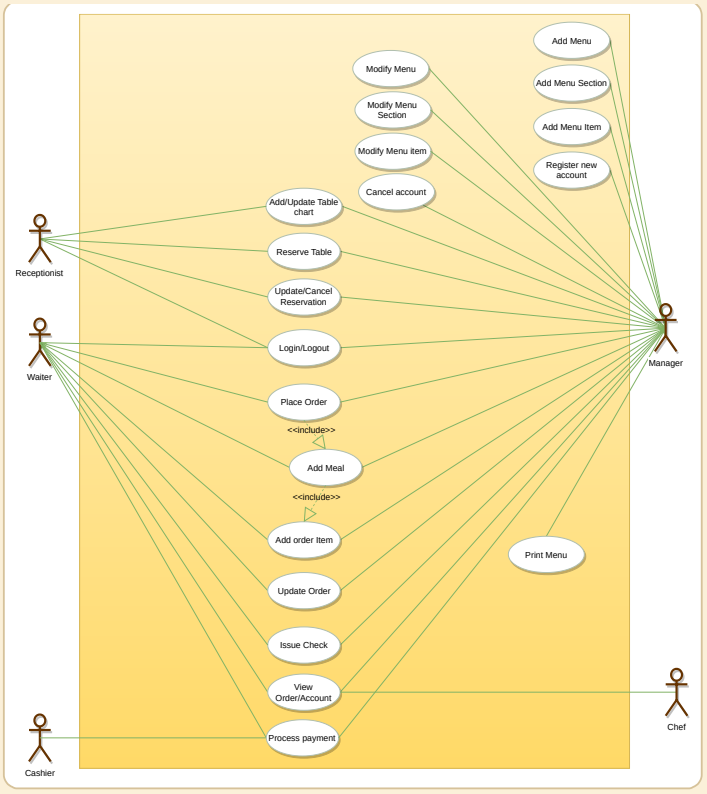


Figure — Use case diagram: receptionist, waiter, manager, chef, cashier and system (course diagram)

In the figure above, each role has a small, separate set of use cases. That is a hint for the code: each role class gets only its own methods.

Core entities

Class	Responsibility
`Restaurant`	Owns its branches. In the course the restaurant also registers its employees: closing (deactivating) the restaurant deactivates every employee account with it.
`Branch`	Owns its menu, kitchen, seating charts and reservation service; employs staff.
`Menu`, `MenuSection`, `MenuItem`	The menu tree. The **price lives only on** `MenuItem`.
`Table`, `TableSeat`	A table with a capacity and numbered seats. Keeps both its **current status** and the **time slots it is promised to**.
`TableChart`	A named seating arrangement; refers to tables (a table can appear in several charts).
`Reservation`	Customer, party size, time, table, status. `updatePeopleCount` refuses a party bigger than the reserved table seats (otherwise the booking would say six while the table holds four).
`ReservationService`	Searches tables (best fit), reserves atomically, cancels, checks in, sends reminders, lists walk-in tables.
`Order`	One per table visit: meals keyed by seat, status with allowed transitions, listeners.
`Meal`, `MealItem`	One meal per seat; a meal item = menu item × quantity.
`Kitchen`, `Chef`	Kitchen hands each order to a chef by a pluggable rule; each chef cooks from a FIFO (first in, first out) queue.
`Employee` and roles (`Receptionist`, `Waiter`, `Manager`, `Chef`, `Cashier`)	Each role's own operations.
`Check`	The bill: computed from the order (subtotal, tax, total), never stored separately.
`Payment` (`CardPayment`, `CashPayment`, `CheckPayment`)	Three ways to pay with three different behaviours.

Class diagram

The course grows the model in seven stages: payments and notifications, the menu, tables, reservations, orders, staff, and finally the branch that ties it together.

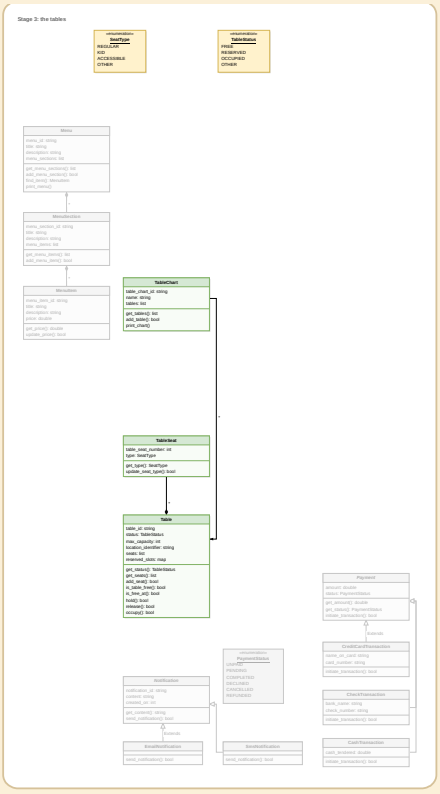


Figure — Table, TableSeat and TableChart: a table owns its seats; a chart only refers to tables (course diagram)

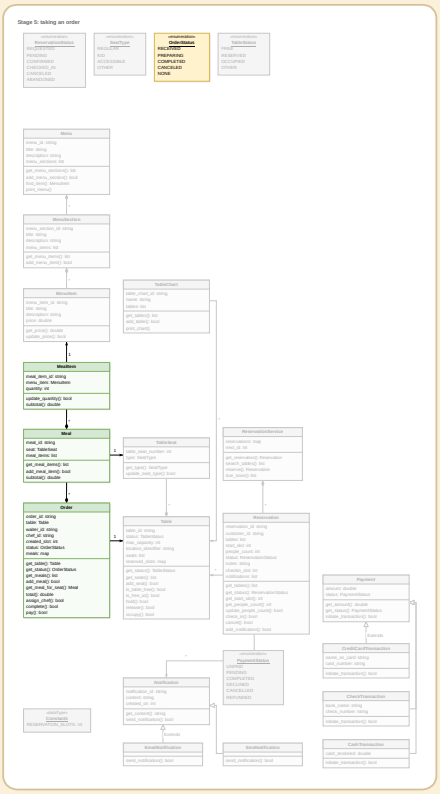


Figure — Order, Meal and MealItem: one order per table, one meal per seat (course diagram)

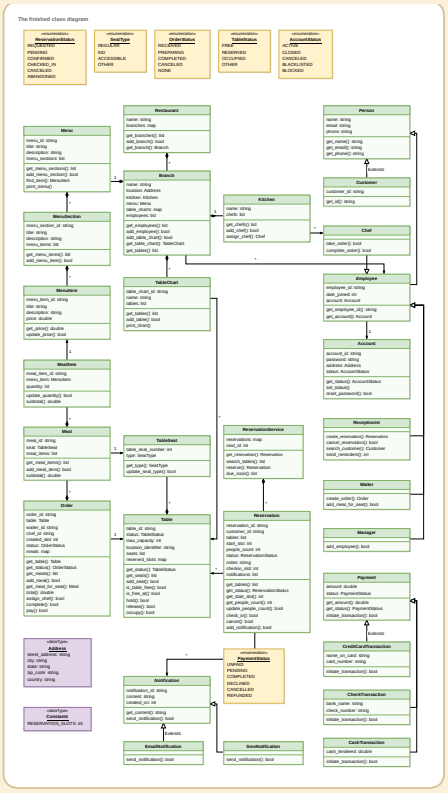
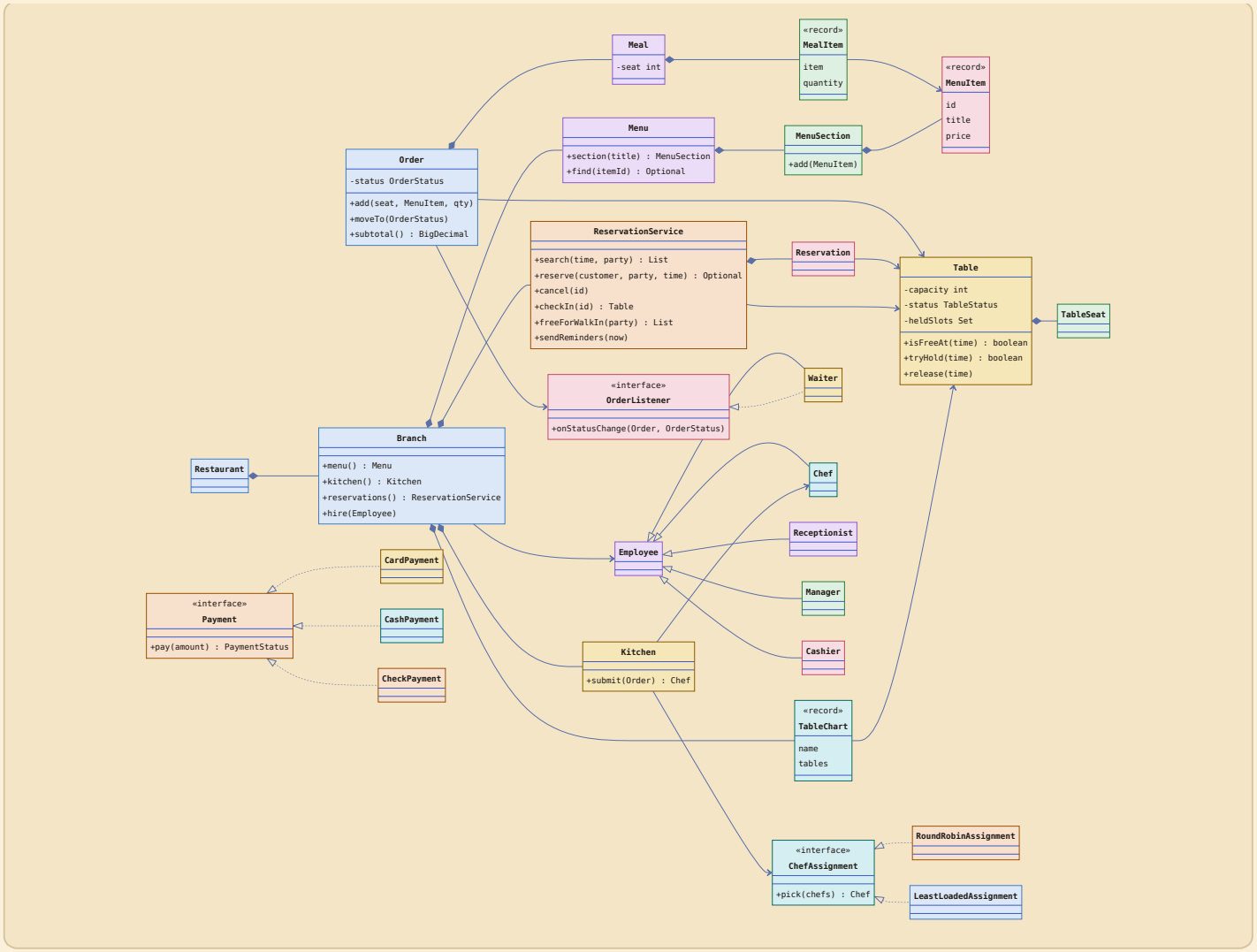


Figure — The finished class diagram of the restaurant management system (course diagram)

In the finished figure above, the branch is the hub: its menu, kitchen and seating charts are composition (they do not exist without the branch), while staff are only associated, because staff can move between branches. Note also what is *not* there: no separate `Bill` class that stores a total. The course's argument is that the order already holds every priced line, so storing the total again would create a second copy that could disagree. Our `Check` follows that rule — it is computed from the order on demand.



Key flows

Place an order. The waiter opens an order for the table, then for each seat adds a meal with its items, and sends the order to the kitchen.

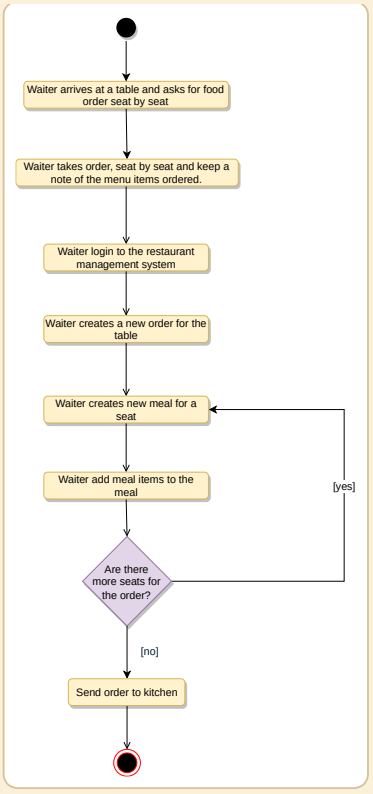


Figure — Activity diagram: placing an order seat by seat and sending it to the kitchen (course diagram)

Make a reservation. The customer calls or arrives; the receptionist searches tables for the time and party size; if one is free it is reserved and a confirmation is sent, otherwise the customer is told none is available.

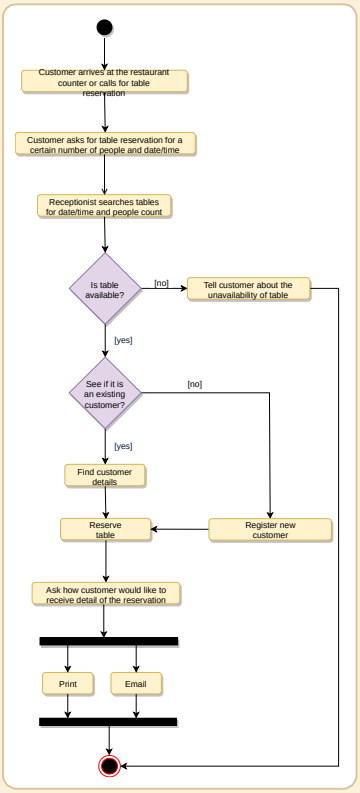


Figure — Activity diagram: making a table reservation (course diagram)

Cancel a reservation. The receptionist looks up the booking; if it is valid, it is cancelled, the table is released and the customer is notified.

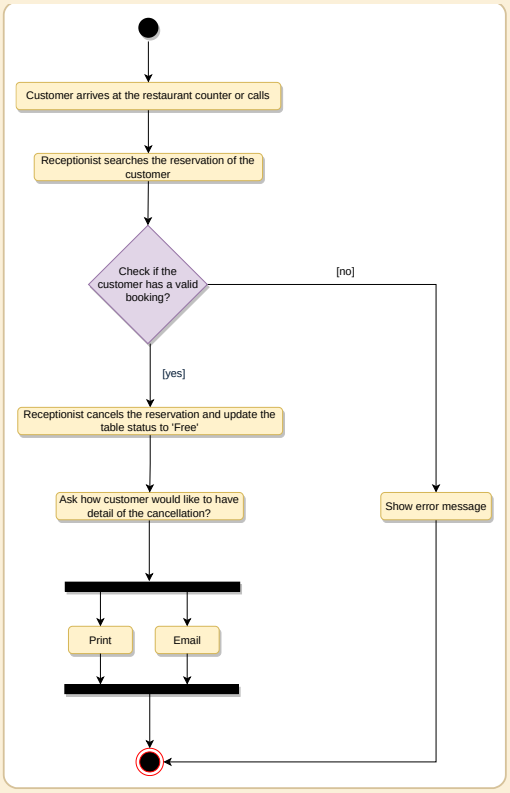
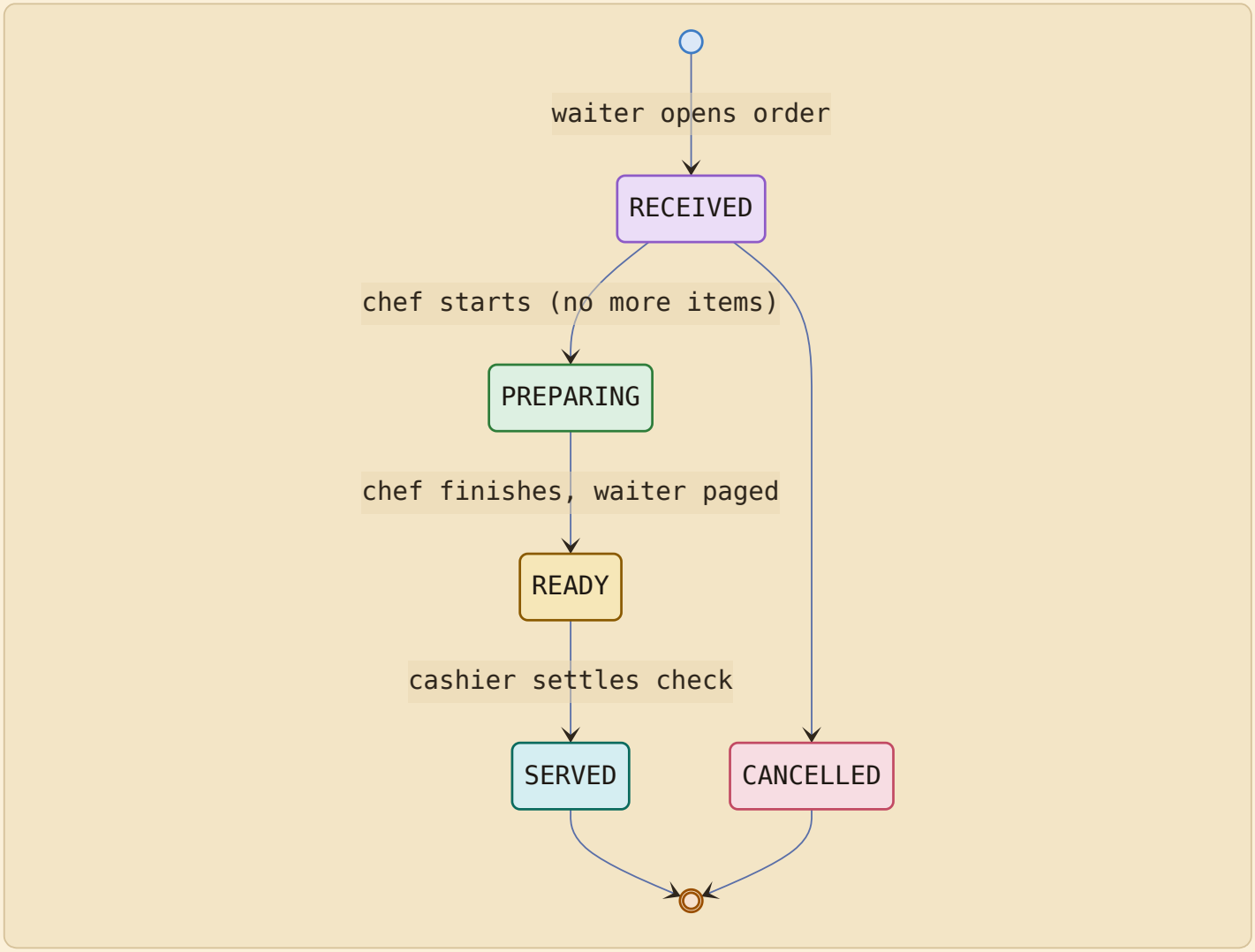
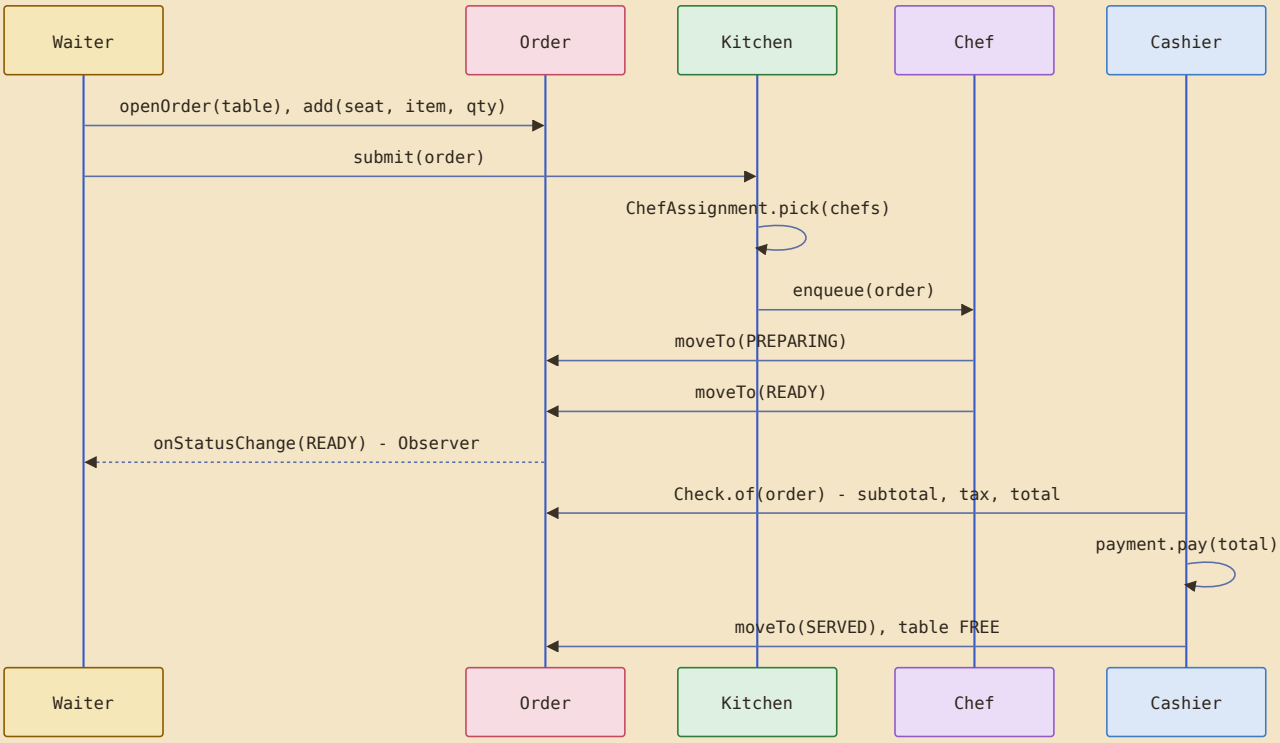


Figure — Activity diagram: cancelling a reservation (course diagram)

Order lifecycle — the transitions `Order.moveTo` allows:



From order to payment:


Waiter
Order
Kitchen
Chef
Cashier
openOrder(table), add(seat, item, qty)
submit(order)
ChefAssignment.pick(chefs)
enqueue(order)
moveTo(PREPARING)
moveTo(READY)
onStatusChange(READY) - Observer
Check.of(order) - subtotal, tax, total
payment.pay(total)
moveTo(SERVED), table FREE
Waiter
Order
Kitchen
Chef
Cashier

Design decisions & patterns used

Decision	Why	Alternative rejected
One meal per seat (`Map<seat, Meal>`)	Requirement 4 asks for it; it tells the kitchen which plate goes to which guest and makes split bills trivial.	A flat list of dishes per order.
Table keeps held time slots as well as status	Status answers "now" (walk-ins); slots answer "is it free at 19:00 on Friday" (reservations). A booking holds two hourly slots.	Status only: cannot take future bookings.
Search lives on `ReservationService`, not on `Table`	One table only knows about itself; "which tables are free at seven" needs all of them. The course notes an older version had a static search on `Table` that could not work.	Static `Table.search`.
Best fit when choosing a table	Smallest table that seats the party keeps large tables free for large parties.	First free table.
Observer (`OrderListener`; listeners register with a subject and are told when it changes)	The kitchen does not need to know who cares about "ready" — the waiter, a display screen, a delivery app can all subscribe.	Chef calling the waiter directly (tight coupling: classes that know too much about each other to change separately).
Strategy (`ChefAssignment`)	The course's kitchen rotates chefs; that is fair but not load-aware (it ignores how busy each chef already is). Swapping to `LeastLoadedAssignment` is one class, not a rewrite — the course's own point about keeping the rule in one place.	Round-robin (taking turns in a fixed rotation) hard-coded in `Kitchen`.
State machine in `Order.moveTo`	Illegal moves (adding food after cooking starts, serving before ready) fail loudly.	Free `setStatus`.
Three `Payment` classes behind a `sealed` interface (only the listed classes may implement it)	Card completes at once, cash may be short (declined) and returns change, a cheque must clear (pending). Different behaviour → different classes (Strategy).	One `Payment` class with a `type` field and `if`s.
Check computed, not stored	Prices live on menu items; the check is derived, so it cannot disagree with the order.	A stored `Bill` total.
Role classes extend `Employee`; `Customer` is not an employee	Only staff have employee ids and accounts; this keeps an account off the customer.	One `Person` class with a role flag.

Java implementation

Package `lld.ch16.restaurant`.

```java
// File: code/lld/ch16/restaurant/Types.java
package lld.ch16.restaurant;

import java.math.BigDecimal;

enum TableStatus { FREE, RESERVED, OCCUPIED }
enum SeatType { REGULAR, KID, ACCESSIBLE }
enum ReservationStatus { CONFIRMED, CHECKED_IN, CANCELLED, NO_SHOW }
enum OrderStatus { RECEIVED, PREPARING, READY, SERVED, CANCELLED }
enum PaymentStatus { COMPLETED, PENDING, DECLINED }

record MenuItem(String id, String title, BigDecimal price) {}
record Customer(String name, String phone) {}
record TableSeat(int number, SeatType type) {}
```

```java
// File: code/lld/ch16/restaurant/Menu.java
package lld.ch16.restaurant;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/** A branch's menu: Menu -> sections -> items (composition). Prices live ONLY on MenuItem. */
final class Menu {
    private final Map<String, MenuSection> sections = new LinkedHashMap<>();

    synchronized MenuSection section(String title) { return sections.computeIfAbsent(title, MenuSection::new); }

    synchronized Optional<MenuItem> find(String itemId) {
        return sections.values().stream().flatMap(s -> s.items().stream())
                .filter(i -> i.id().equals(itemId)).findFirst();
    }

    synchronized List<MenuSection> sections() { return List.copyOf(sections.values()); }
}

final class MenuSection {
    private final String title;
    private final List<MenuItem> items = new ArrayList<>();
    MenuSection(String title) { this.title = title; }
    String title() { return title; }
    synchronized void add(MenuItem item) { items.add(item); }
    synchronized boolean remove(String itemId) { return items.removeIf(i -> i.id().equals(itemId)); }
    synchronized List<MenuItem> items() { return List.copyOf(items); }
}
```

```java
// File: code/lld/ch16/restaurant/Tables.java
package lld.ch16.restaurant;

import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * status answers "what is this table doing NOW" (for walk-ins).
 * heldSlots answers "is it free at 19:00 on Friday" (for reservations).
 */
final class Table {
    static final int SLOTS_PER_BOOKING = 2;   // a booking holds the table for two hourly slots

    private final String id;
    private final int capacity;
    private final List<TableSeat> seats = new ArrayList<>();
    private final Set<LocalDateTime> heldSlots = new HashSet<>();
    private TableStatus status = TableStatus.FREE;

    Table(String id, int capacity) {
        this.id = id;
        this.capacity = capacity;
        for (int i = 1; i <= capacity; i++) seats.add(new TableSeat(i, SeatType.REGULAR));
    }

    String id() { return id; }
    int capacity() { return capacity; }
    List<TableSeat> seats() { return List.copyOf(seats); }
    synchronized TableStatus status() { return status; }
    synchronized void setStatus(TableStatus s) { status = s; }

    private static List<LocalDateTime> slots(LocalDateTime start) {
        var first = start.truncatedTo(ChronoUnit.HOURS);
        List<LocalDateTime> out = new ArrayList<>();
        for (int i = 0; i < SLOTS_PER_BOOKING; i++) out.add(first.plusHours(i));
        return out;
    }

    synchronized boolean isFreeAt(LocalDateTime start) { return
slots(start).stream().noneMatch(heldSlots::contains); }

    /** Atomic check-then-hold. */
    synchronized boolean tryHold(LocalDateTime start) {
        if (!isFreeAt(start)) return false;
        heldSlots.addAll(slots(start));
        return true;
    }

    synchronized void release(LocalDateTime start) { slots(start).forEach(heldSlots::remove); }
}

/** A named seating arrangement. It REFERS to tables (association); a table can appear in several charts. */
record TableChart(String name, List<Table> tables) {}
```

```java
// File: code/lld/ch16/restaurant/Reservations.java
package lld.ch16.restaurant;

import java.time.LocalDateTime;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

final class Reservation {
    private final String id;
    private final Customer customer;
    private int partySize;
    private final LocalDateTime time;
    private final Table table;
    private ReservationStatus status = ReservationStatus.CONFIRMED;
    private boolean reminded;

    Reservation(String id, Customer customer, int partySize, LocalDateTime time, Table table) {
        this.id = id; this.customer = customer; this.partySize = partySize; this.time = time; this.table = table;
    }
    String id() { return id; }
    Customer customer() { return customer; }
    synchronized int partySize() { return partySize; }

    /** The party grew or shrank. Refused if the reserved table cannot seat the new number. */
    synchronized void updatePeopleCount(int n) {
        if (status != ReservationStatus.CONFIRMED) throw new IllegalStateException(id + " is " + status);
        if (n < 1 || n > table.capacity())
            throw new IllegalArgumentException("table " + table.id() + " seats " + table.capacity() + ", not " +
n);
        partySize = n;
    }
    LocalDateTime time() { return time; }
    Table table() { return table; }
    synchronized ReservationStatus status() { return status; }

    synchronized void cancel() {
        if (status != ReservationStatus.CONFIRMED) throw new IllegalStateException(id + " is " + status);
        status = ReservationStatus.CANCELLED;
        table.release(time);
    }

    synchronized void checkIn() {
        if (status != ReservationStatus.CONFIRMED) throw new IllegalStateException(id + " is " + status);
        status = ReservationStatus.CHECKED_IN;
        table.setStatus(TableStatus.OCCUPIED);
    }

    synchronized boolean markReminded() { boolean first = !reminded; reminded = true; return first; }
}

interface Notifier { void send(Customer to, String message); }

final class SmsNotifier implements Notifier {
    public void send(Customer to, String message) { System.out.println("    [sms " + to.phone() + "] " + message);
}
}

/** Searching belongs here, not on Table: one table only knows about itself. */
final class ReservationService {
    private final List<Table> tables;
    private final Notifier notifier;
    private final Map<String, Reservation> reservations = new ConcurrentHashMap<>();
    private final AtomicInteger seq = new AtomicInteger();
```

```
    ReservationService(List<Table> tables, Notifier notifier) { this.tables = List.copyOf(tables); this.notifier =
notifier; }

    /** Best fit first: the smallest table that seats the party, so big tables stay free for big parties. */
    List<Table> search(LocalDateTime time, int partySize) {
        return tables.stream().filter(t -> t.capacity() >= partySize && t.isFreeAt(time))
                .sorted(Comparator.comparingInt(Table::capacity)).toList();
    }

    Optional<Reservation> reserve(Customer c, int partySize, LocalDateTime time) {
        for (Table t : search(time, partySize)) {
            if (t.tryHold(time)) {     // another receptionist may have taken it since search(): try the next one
                var r = new Reservation("RES-" + seq.incrementAndGet(), c, partySize, time, t);
                reservations.put(r.id(), r);
                notifier.send(c, "Table " + t.id() + " booked for " + partySize + " at " + time);
                return Optional.of(r);
            }
        }
        return Optional.empty();
    }

    Reservation get(String id) {
        var r = reservations.get(id);
        if (r == null) throw new IllegalArgumentException("no reservation " + id);
        return r;
    }

    void cancel(String id) {
        var r = get(id);
        r.cancel();
        notifier.send(r.customer(), "Reservation " + id + " cancelled");
    }

    Table checkIn(String id) {
        var r = get(id);
        r.checkIn();
        return r.table();
    }

    List<Table> freeForWalkIn(int partySize) {
        return tables.stream().filter(t -> t.status() == TableStatus.FREE && t.capacity() >= partySize)
                .sorted(Comparator.comparingInt(Table::capacity)).toList();
    }

    /** Run by a scheduler every few minutes: remind reservations starting within the next hour, once. */
    void sendReminders(LocalDateTime now) {
        reservations.values().stream()
                .filter(r -> r.status() == ReservationStatus.CONFIRMED)
                .filter(r -> !r.time().isBefore(now) && r.time().isBefore(now.plusHours(1)))
                .filter(Reservation::markReminded)
                .forEach(r -> notifier.send(r.customer(), "Reminder: your table " + r.table().id() + " at " +
r.time().toLocalTime()));
    }
}
```

```java
// File: code/lld/ch16/restaurant/Orders.java
package lld.ch16.restaurant;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.concurrent.CopyOnWriteArrayList;

record MealItem(MenuItem item, int quantity) {
    MealItem {
        if (quantity <= 0) throw new IllegalArgumentException("quantity must be positive");
    }
    BigDecimal subtotal() { return item.price().multiply(BigDecimal.valueOf(quantity)); }
}

/** One meal per seat: lets the kitchen plate per guest and lets the bill be split. */
final class Meal {
    private final int seat;
    private final List<MealItem> items = new ArrayList<>();
    Meal(int seat) { this.seat = seat; }
    int seat() { return seat; }
    synchronized void add(MealItem i) { items.add(i); }
    synchronized List<MealItem> items() { return List.copyOf(items); }
    synchronized BigDecimal subtotal() { return items.stream().map(MealItem::subtotal).reduce(BigDecimal.ZERO,
BigDecimal::add); }
}

/** Observer: whoever cares about an order's progress (the waiter, a display screen). */
interface OrderListener { void onStatusChange(Order order, OrderStatus newStatus); }

final class Order {
    private final String id;
    private final Table table;
    private final Map<Integer, Meal> meals = new TreeMap<>();
    private final List<OrderListener> listeners = new CopyOnWriteArrayList<>();
    private OrderStatus status = OrderStatus.RECEIVED;

    Order(String id, Table table) { this.id = id; this.table = table; }
    String id() { return id; }
    Table table() { return table; }
    synchronized OrderStatus status() { return status; }
    void subscribe(OrderListener l) { listeners.add(l); }

    /** Items can only be added before the kitchen starts cooking. */
    synchronized void add(int seat, MenuItem item, int qty) {
        if (status != OrderStatus.RECEIVED) throw new IllegalStateException("order " + id + " is already " +
status);
        if (seat < 1 || seat > table.capacity()) throw new IllegalArgumentException("table " + table.id() + " has
no seat " + seat);
        meals.computeIfAbsent(seat, Meal::new).add(new MealItem(item, qty));
    }

    synchronized List<Meal> meals() { return List.copyOf(meals.values()); }

    BigDecimal subtotal() { return meals().stream().map(Meal::subtotal).reduce(BigDecimal.ZERO, BigDecimal::add); }

    void moveTo(OrderStatus next) {
        synchronized (this) {
            boolean ok = switch (status) {
                case RECEIVED -> next == OrderStatus.PREPARING || next == OrderStatus.CANCELLED;
                case PREPARING -> next == OrderStatus.READY;
                case READY -> next == OrderStatus.SERVED;
                case SERVED, CANCELLED -> false;
            };
            if (!ok) throw new IllegalStateException("order " + id + ": " + status + " -> " + next + " not
allowed");
```

```
            status = next;
        }
        listeners.forEach(l -> l.onStatusChange(this, next));   // notify outside the lock
    }
}
```

```
// File: code/lld/ch16/restaurant/Kitchen.java
package lld.ch16.restaurant;

import java.util.ArrayDeque;
import java.util.Comparator;
import java.util.Deque;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

final class Chef extends Employee {
    private final Deque<Order> queue = new ArrayDeque<>();
    private Order current;

    Chef(String id, String name) { super(id, name); }

    synchronized void enqueue(Order o) { queue.addLast(o); }
    synchronized int load() { return queue.size() + (current == null ? 0 : 1); }

    /** Take the next order in FIFO order and start cooking it. */
    synchronized Order startNext() {
        if (current != null) throw new IllegalStateException(name() + " is still cooking " + current.id());
        current = queue.pollFirst();
        if (current != null) current.moveTo(OrderStatus.PREPARING);
        return current;
    }

    synchronized void finishCurrent() {
        if (current == null) throw new IllegalStateException(name() + " has nothing on the stove");
        current.moveTo(OrderStatus.READY);
        current = null;
    }
}

/** Strategy: which chef gets the next order. */
interface ChefAssignment { Chef pick(List<Chef> chefs); }

final class RoundRobinAssignment implements ChefAssignment {
    private int next;
    public synchronized Chef pick(List<Chef> chefs) { return chefs.get(next++ % chefs.size()); }
}

final class LeastLoadedAssignment implements ChefAssignment {
    public Chef pick(List<Chef> chefs) { return
chefs.stream().min(Comparator.comparingInt(Chef::load)).orElseThrow(); }
}

final class Kitchen {
    private final List<Chef> chefs = new CopyOnWriteArrayList<>();
    private final ChefAssignment assignment;

    Kitchen(ChefAssignment assignment) { this.assignment = assignment; }
    void addChef(Chef c) { chefs.add(c); }

    Chef submit(Order o) {
        if (chefs.isEmpty()) throw new IllegalStateException("no chef on shift");
        Chef c = assignment.pick(chefs);
        c.enqueue(o);
        return c;
    }
}
```

```java
// File: code/lld/ch16/restaurant/Staff.java
package lld.ch16.restaurant;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

abstract class Employee {
    private final String id;
    private final String name;
    protected Employee(String id, String name) { this.id = id; this.name = name; }
    String id() { return id; }
    String name() { return name; }
}

final class Receptionist extends Employee {
    Receptionist(String id, String name) { super(id, name); }
}

/** A waiter opens orders and is told (Observer) when a dish is ready. */
final class Waiter extends Employee implements OrderListener {
    private static final AtomicInteger SEQ = new AtomicInteger();
    Waiter(String id, String name) { super(id, name); }

    Order openOrder(Table t) {
        t.setStatus(TableStatus.OCCUPIED);
        var o = new Order("O" + SEQ.incrementAndGet(), t);
        o.subscribe(this);
        return o;
    }

    public void onStatusChange(Order o, OrderStatus s) {
        if (s == OrderStatus.READY) System.out.println("    [" + name() + "'s pager] order " + o.id() + " for table
" + o.table().id() + " is READY");
    }
}

final class Manager extends Employee {
    Manager(String id, String name) { super(id, name); }
    void addMenuItem(Menu menu, String section, MenuItem item) { menu.section(section).add(item); }
    void hire(Branch b, Employee e) { b.hire(e); }
}

/** The bill ("check"). It is computed from the order's lines, never stored separately. */
record Check(Order order, BigDecimal subtotal, BigDecimal tax, BigDecimal total) {
    static final BigDecimal TAX_RATE = new BigDecimal("0.08");
    static Check of(Order o) {
        var sub = o.subtotal();
        var tax = sub.multiply(TAX_RATE).setScale(2, RoundingMode.HALF_UP);
        return new Check(o, sub, tax, sub.add(tax));
    }
}

/** Strategy: three ways to pay, three behaviours. */
sealed interface Payment permits CardPayment, CashPayment, CheckPayment { PaymentStatus pay(BigDecimal amount); }

record CardPayment(String cardToken) implements Payment {
    public PaymentStatus pay(BigDecimal amount) { return PaymentStatus.COMPLETED; }
}

record CashPayment(BigDecimal tendered) implements Payment {
    public PaymentStatus pay(BigDecimal amount) {
        if (tendered.compareTo(amount) < 0) return PaymentStatus.DECLINED;
        System.out.println("    change due: " + tendered.subtract(amount));
        return PaymentStatus.COMPLETED;
    }
```

```
}

/** A paper check has to clear at the bank, so it is not money yet. */
record CheckPayment(String bank, String checkNumber) implements Payment {
    public PaymentStatus pay(BigDecimal amount) { return PaymentStatus.PENDING; }
}

final class Cashier extends Employee {
    Cashier(String id, String name) { super(id, name); }

    PaymentStatus settle(Check check, Payment payment) {
        // Check the order state BEFORE taking money: a refused state change after payment would leave the guest
charged.
        if (check.order().status() != OrderStatus.READY)
            throw new IllegalStateException("order " + check.order().id() + " is " + check.order().status() + ",
not READY");
        var status = payment.pay(check.total());
        if (status != PaymentStatus.DECLINED) {
            check.order().moveTo(OrderStatus.SERVED);
            check.order().table().setStatus(TableStatus.FREE);
        }
        return status;
    }
}

/** A branch owns its menu, kitchen and charts; staff are associated (they can move branches). */
final class Branch {
    private final String name;
    private final Menu menu = new Menu();
    private final Kitchen kitchen;
    private final List<TableChart> charts = new ArrayList<>();
    private final List<Employee> staff = new ArrayList<>();
    private final ReservationService reservations;

    Branch(String name, List<Table> tables, ChefAssignment assignment, Notifier notifier) {
        this.name = name;
        this.kitchen = new Kitchen(assignment);
        this.reservations = new ReservationService(tables, notifier);
        charts.add(new TableChart("Default", tables));
    }
    String name() { return name; }
    Menu menu() { return menu; }
    Kitchen kitchen() { return kitchen; }
    ReservationService reservations() { return reservations; }
    synchronized void addChart(TableChart c) { charts.add(c); }
    synchronized List<TableChart> charts() { return List.copyOf(charts); }
    synchronized void hire(Employee e) {
        staff.add(e);
        if (e instanceof Chef c) kitchen.addChef(c);
    }
    synchronized List<Employee> staff() { return List.copyOf(staff); }
}

final class Restaurant {
    private final String name;
    private final List<Branch> branches = new ArrayList<>();
    Restaurant(String name) { this.name = name; }
    Branch open(Branch b) { branches.add(b); return b; }
    String name() { return name; }
}
```

```java
// File: code/lld/ch16/restaurant/Main.java
package lld.ch16.restaurant;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.List;

public class Main {
    static MenuItem item(String id, String title, String price) { return new MenuItem(id, title, new
BigDecimal(price)); }

    public static void main(String[] args) {
        var t1 = new Table("T1", 2);
        var t2 = new Table("T2", 4);
        var t3 = new Table("T3", 4);
        var t4 = new Table("T4", 6);
        var tables = List.of(t1, t2, t3, t4);

        var restaurant = new Restaurant("Spice Route");
        var branch = restaurant.open(new Branch("Downtown", tables, new RoundRobinAssignment(), new
SmsNotifier()));
        branch.addChart(new TableChart("Banquet", List.of(t4)));

        var manager = new Manager("E1", "Maya");
        var reception = new Receptionist("E2", "Ravi");
        var wendy = new Waiter("E3", "Wendy");
        var cashier = new Cashier("E4", "Carl");
        for (var e : List.of(manager, reception, wendy, cashier, new Chef("E5", "Chen"), new Chef("E6", "Maria")))
            manager.hire(branch, e);

        System.out.println("1) Manager builds the menu");
        var menu = branch.menu();
        manager.addMenuItem(menu, "Starters", item("S1", "Tomato soup", "6.50"));
        manager.addMenuItem(menu, "Starters", item("S2", "Garlic bread", "4.00"));
        manager.addMenuItem(menu, "Mains", item("M1", "Margherita pizza", "12.00"));
        manager.addMenuItem(menu, "Mains", item("M2", "Pasta arrabbiata", "14.50"));
        manager.addMenuItem(menu, "Drinks", item("D1", "Lemonade", "3.00"));
        menu.sections().forEach(s -> System.out.println("   " + s.title() + ": " +
s.items().stream().map(MenuItem::title).toList()));
        System.out.println("   charts: " + branch.charts().stream().map(TableChart::name).toList());

        var rs = branch.reservations();
        var seven = LocalDateTime.of(2026, 10, 9, 19, 0);
        var eight = seven.plusHours(1);
        System.out.println("2) " + reception.name() + " searches tables for 4 at 19:00: "
                + rs.search(seven, 4).stream().map(Table::id).toList());
        var asha = rs.reserve(new Customer("Asha", "555-0101"), 4, seven).orElseThrow();
        var ben = rs.reserve(new Customer("Ben", "555-0102"), 4, eight).orElseThrow();
        System.out.println("   Asha -> " + asha.table().id() + ", Ben -> " + ben.table().id() + " (T2 is held
19:00-21:00)");
        System.out.println("   party of 8 at 19:00: " + rs.reserve(new Customer("Big group", "555-0199"), 8, seven)
                .map(Reservation::id).orElse("no single table fits"));
        try { asha.updatePeopleCount(6); }
        catch (IllegalArgumentException e) { System.out.println("   Asha's party grows to 6: rejected, " +
e.getMessage()); }
        rs.cancel(ben.id());
        System.out.println("   after Ben cancels, free for 4 at 20:00: " + rs.search(eight,
4).stream().map(Table::id).toList());

        System.out.println("3) Reminder job at 18:15");
        rs.sendReminders(seven.minusMinutes(45));
        rs.sendReminders(seven.minusMinutes(30));   // second run does not spam

        System.out.println("4) Asha checks in; a couple walks in");
        var table = rs.checkIn(asha.id());
        var walkIn = rs.freeForWalkIn(2).getFirst();
        System.out.println("   Asha seated at " + table.id() + ", walk-in couple seated at " + walkIn.id());
```

```
        System.out.println("5) Wendy takes two orders, one meal per seat");
        var o1 = wendy.openOrder(table);
        o1.add(1, menu.find("S1").orElseThrow(), 1);
        o1.add(1, menu.find("M1").orElseThrow(), 1);
        o1.add(2, menu.find("M2").orElseThrow(), 1);
        o1.add(2, menu.find("D1").orElseThrow(), 2);
        o1.add(3, menu.find("S2").orElseThrow(), 1);
        var o2 = wendy.openOrder(walkIn);
        o2.add(1, menu.find("M1").orElseThrow(), 2);
        var kitchen = branch.kitchen();
        var c1 = kitchen.submit(o1);
        var c2 = kitchen.submit(o2);
        System.out.println("   " + o1.id() + " -> chef " + c1.name() + ", " + o2.id() + " -> chef " + c2.name());

        System.out.println("6) Kitchen works the queues");
        c1.startNext();
        try { o1.add(4, menu.find("D1").orElseThrow(), 1); }
        catch (IllegalStateException e) { System.out.println("   rejected: " + e.getMessage()); }
        c2.startNext();
        c1.finishCurrent();
        c2.finishCurrent();

        System.out.println("7) Check for " + o1.id() + " (split by seat)");
        for (var meal : o1.meals()) {
            System.out.println("   seat " + meal.seat() + ": " + meal.items().stream()
                    .map(i -> i.quantity() + "x " + i.item().title()).toList() + " = " + meal.subtotal());
        }
        var check1 = Check.of(o1);
        System.out.println("   subtotal " + check1.subtotal() + ", tax " + check1.tax() + ", total " +
check1.total());
        System.out.println("   paid cash: " + cashier.settle(check1, new CashPayment(new BigDecimal("60.00"))));
        var check2 = Check.of(o2);
        System.out.println("   " + o2.id() + " total " + check2.total() + " paid by check: " +
cashier.settle(check2, new CheckPayment("First Bank", "000123")));
        System.out.println("   table statuses: " + tables.stream().map(t -> t.id() + "=" + t.status()).toList());
    }
}
```

Output:

```
1) Manager builds the menu
   Starters: [Tomato soup, Garlic bread]
   Mains: [Margherita pizza, Pasta arrabbiata]
   Drinks: [Lemonade]
   charts: [Default, Banquet]
2) Ravi searches tables for 4 at 19:00: [T2, T3, T4]
    [sms 555-0101] Table T2 booked for 4 at 2026-10-09T19:00
    [sms 555-0102] Table T3 booked for 4 at 2026-10-09T20:00
   Asha -> T2, Ben -> T3 (T2 is held 19:00-21:00)
   party of 8 at 19:00: no single table fits
   Asha's party grows to 6: rejected, table T2 seats 4, not 6
    [sms 555-0102] Reservation RES-2 cancelled
   after Ben cancels, free for 4 at 20:00: [T3, T4]
3) Reminder job at 18:15
    [sms 555-0101] Reminder: your table T2 at 19:00
4) Asha checks in; a couple walks in
   Asha seated at T2, walk-in couple seated at T1
5) Wendy takes two orders, one meal per seat
   O1 -> chef Chen, O2 -> chef Maria
6) Kitchen works the queues
   rejected: order O1 is already PREPARING
    [Wendy's pager] order O1 for table T2 is READY
    [Wendy's pager] order O2 for table T1 is READY
7) Check for O1 (split by seat)
   seat 1: [1x Tomato soup, 1x Margherita pizza] = 18.50
   seat 2: [1x Pasta arrabbiata, 2x Lemonade] = 20.50
   seat 3: [1x Garlic bread] = 4.00
   subtotal 43.00, tax 3.44, total 46.44
    change due: 13.56
   paid cash: COMPLETED
   O2 total 25.92 paid by check: PENDING
   table statuses: [T1=FREE, T2=FREE, T3=FREE, T4=FREE]
```

Step 2 shows best fit and slot holding (Ben cannot get T2 at 20:00 because Asha's 19:00 booking holds 19:00 and 20:00). Step 6 shows the state machine rejecting a late addition and the Observer paging the waiter. Step 7 shows the per-seat split and the three payment behaviours.

Concurrency & edge cases

- **Two receptionists book the same table.** `search` is only advisory; `reserve` calls `Table.tryHold`, which is `synchronized` and checks-and-holds both slots in one step. If it fails (someone else won) the loop tries the next table in best-fit order.
- **Notifying outside the lock.** `Order.moveTo` changes state inside `synchronized` but calls listeners *after* leaving it. Calling unknown code while holding a lock is a classic way to cause deadlocks or long waits. A **deadlock** is when two threads each hold a lock the other needs, so both wait forever.
- **Reminder spam.** The reminder job may run every few minutes; `markReminded` makes it send once per reservation.
- **Party larger than any table.** Returns "no single table fits". Combining tables is out of scope (see Extensions). The course notes the design already allows it, since a reservation can hold a list of tables.
- **No-shows.** `NO_SHOW` exists as a status; a job can release tables for bookings not checked in 15 minutes after start.
- **Party grows after booking.** `updatePeopleCount(6)` on a four-seat table is rejected (step 2 of the output); the receptionist must re-book.
- **Paying for food that is not ready.** `Cashier.settle` checks the order is `READY` *before* calling the payment, so a refused state change can never happen after the guest's money has been taken.
- **Unknown reservation id** on cancel or check-in fails with a clear `IllegalArgumentException` (Java's standard "you passed a bad value" error), not a `NullPointerException` (the confusing crash you get when code uses a missing object).
- **Seat number out of range** on an order is rejected.
- **Cash short** → `DECLINED`, the order stays `READY` and the table stays occupied.
- **Cheque** → `PENDING`: the meal is served, but the money is not yet confirmed.

Extensions & follow-up questions

- **"Large party needs two tables?"** — Let a reservation hold `List<Table>`; search for combinations of adjacent tables (tag tables with a zone); hold all of them atomically by locking tables in a fixed order (for example by id) to avoid deadlock.
- **"Split the bill?"** — Already easy: `Meal.subtotal()` per seat; create one `Check` per seat group.
- **"Tips and service charge?"** — Add lines to `Check` (percentage or amount chosen at payment).
- **"Waiting list for walk-ins?"** — A FIFO queue per party size; when a table frees, notify the head.
- **"Kitchen stock?"** — An `Inventory` of ingredients; each `MenuItem` has a recipe; placing an order reserves ingredients and marks items "86'd" (unavailable) when stock runs out.
- **"Real concurrency in the kitchen?"** — Each chef becomes a worker thread (a thread that loops, picking up jobs) taking from a `BlockingQueue<Order>` (a thread-safe queue whose `take()` waits when it is empty); `take()` blocks until an order arrives.
- **"Shift schedules?"** — A `Shift` entity (employee, branch, start, end); `Kitchen` only picks chefs on shift.

In the interview

30-second pitch: "Branch owns a menu tree, a kitchen, seating charts and a reservation service. Tables keep both current status for walk-ins and held time slots for bookings; the service searches best-fit and holds slots atomically. Orders have one meal per seat, move through a strict state machine, and notify the waiter through an observer. The kitchen assigns chefs by a strategy. The check is computed from the order, and payment is a strategy with card, cash and cheque."

Minutes	Do
0–5	Clarify branches, seating layouts, per-seat meals, payment types.
5–10	Actors (six) and use cases.
10–20	Class diagram — say "meal per seat" and "status vs time slots" out loud.
20–35	Code `Table.tryHold`, `ReservationService.reserve`, `Order` state machine, `Kitchen.submit`.
35–40	Races, large parties, kitchen load balancing.

Self-check

1. **Q:** Why does a table need both a status and held slots? **A:** Status answers "what is it doing now" (walk-ins); slots answer "is it free at a future time" (reservations).
2. **Q:** Why is search on the service, not the table? **A:** A single table only knows about itself; finding all free tables needs the collection.
3. **Q:** Which pattern tells the waiter the food is ready, and why notify outside the lock? **A:** Observer; calling foreign code while locked risks deadlock and slows other threads.
4. **Q:** Why is there no stored bill total? **A:** The order's lines already hold every price; a second stored total could disagree.
5. **Q:** Why is a cheque payment `PENDING`? **A:** A cheque has to clear at the bank, so it is not money yet.

TL;DR

- Menu → section → item; price only on the item.
- Table = status (now) + held slots (later); service searches best-fit and holds atomically.
- Order → meal per seat → meal item; strict state machine; Observer for "ready".
- Kitchen assigns chefs by Strategy (round-robin or least-loaded).
- Check is computed; card/cash/cheque are three payment strategies.

Real-world example. Restaurant point-of-sale (POS — the till software where orders are entered and paid) systems such as Toast send each order to a kitchen display screen as a "ticket", show it by station and in order, and let the front of house see when it is done — the Observer and FIFO queue above. Reservation platforms such as OpenTable model table inventory by time slot and party size, exactly like `Table.heldSlots`.

Design an Airline Management System

Figure — An airline flies passengers between airports with owned or leased aircraft (course diagram)

Problem & clarifying questions

An **airline management system** runs an airline's operations: scheduling flights, reserving tickets, cancelling flights, supporting customers and managing staff, plus daily flight updates.

Question	Assumed answer
What does a customer search by?	Date, source airport and destination airport.
One flight or a trip of several?	A customer can build an **itinerary** (the full planned trip) of several flights.
Several passengers per booking?	Yes — a reservation seats several passengers (who need not have accounts).
Seat classes?	Economy, economy plus, preferred economy, business, first; seat types regular, accessible, emergency exit, extra legroom.
Who manages flights?	Only the admin adds aircraft, flights and schedules, and cancels flights; everyone affected is notified.
Crew?	Pilots and crew are assigned to flights and can view their schedule.
Payment?	Paid per itinerary.
Notifications?	When a reservation is made or changed and when a flight's status changes.

Requirements

Functional

1. Search flights by date and by source and destination airport.
2. Reserve a ticket on any scheduled flight; build an itinerary of several flights.
3. View flight schedules: departure, arrival, available seats and other details.
4. Reserve seats for several passengers under one itinerary.
5. Only the admin adds aircraft, flights and schedules; the admin can cancel a scheduled flight and everyone affected is notified.
6. Customers can cancel a reservation or a whole itinerary.
7. Assign pilots and crew to flights.
8. Take payment for reservations.
9. Send the customer a message whenever a booking is created or modified, and whenever the status of one of their flights changes.

Non-functional

- A seat on a flight instance is sold at most once, even under heavy concurrent booking.
- Several passengers in one booking get all their seats or none (**atomic**).
- Fare rules (class multiplier, last-minute surcharge) change without touching booking code.

Use cases

Five actors:

- **Admin** — adds flights and schedules, cancels flights, manages staff and crew.
- **Front desk officer** — reserves and cancels tickets for walk-in customers.
- **Customer** — views schedules, reserves and cancels.
- **Pilot / Crew** — views assigned flights.
- **System** — sends itinerary and flight-status notifications.

Top use cases: search flights; create/modify/view reservation; assign seats to passengers; pay for a reservation; update the flight schedule; assign pilots and crew.

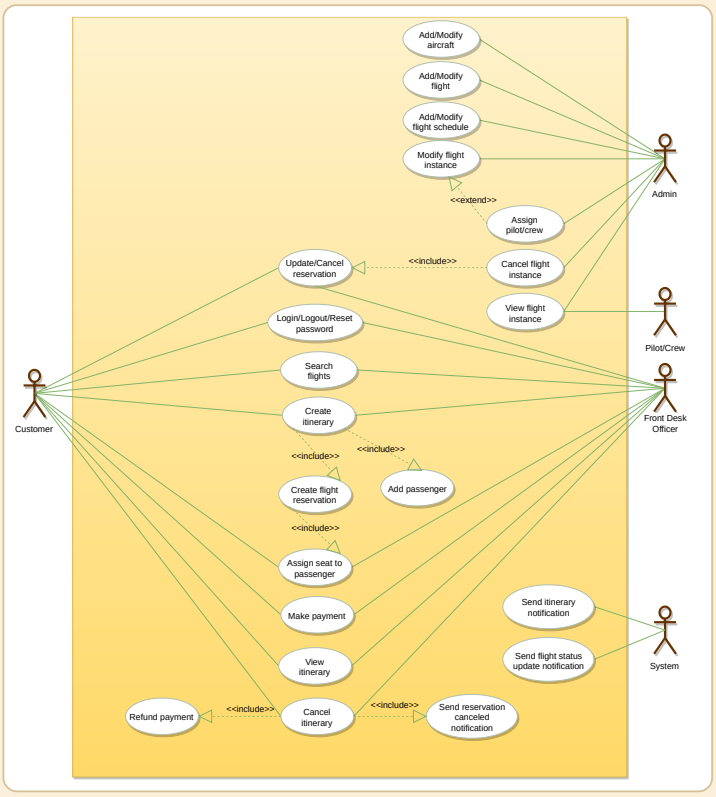


Figure — Use case diagram: admin, front desk officer, customer, crew and system (course diagram)

In the figure above, the front desk officer and the customer share the booking use cases: when an officer books for someone at the counter, it is the very same operation, just triggered by another actor.

Core entities

The one distinction that makes this design work is **four different "seat-ish" and "flight-ish" ideas**:

Idea	Class	Example
The route	`Flight`	GA212 London → Tokyo, 14 h, base fare 500
The route on one date	`FlightInstance`	GA212 on 8 Oct 2026 at 10:00
The physical seat on the aircraft	`Seat`	12A, extra legroom, economy plus
That seat on that one instance	`FlightSeat`	12A on GA212 8 Oct — booked by PNR147 (PNR = booking reference)

Class	Responsibility
`Airport`	Code and city.
`Aircraft`	Registration, model and its physical seats.
`Seat`	Number, type, class — nothing about bookings.
`Flight`	Route, duration, aircraft, base fare, weekly schedules; generates instances.
`WeeklySchedule`	"Every Tuesday at 10:00".
`FlightInstance`	Departure time, status, its `FlightSeat`s and assigned crew; holds seats atomically.
`FlightSeat`	One seat on one instance and which reservation holds it.
`Passenger`	A name on a ticket — deliberately **not** an account holder.
`Customer`	The account holder who books and pays.
`Admin`, `FrontDeskOfficer`	Staff actors. Admin-only methods on `Airline` require an `Admin` argument; the front desk books through `bookFor`.
`CrewMember`	Pilot, co-pilot or cabin crew.
`FlightReservation`	One leg: one instance, map passenger → seat, amount, status.
`Itinerary`	A trip of legs; owns the connection rule; the unit that is paid.
`FareStrategy`	Pluggable pricing: class multiplier, decorated with a last-minute surcharge.
`Airline`	Facade (one simple front object that hides many classes behind it) for admin and customer operations; keeps the index instance → reservations.

Class diagram

The course builds six stages: money and messages, people, what the airline owns, flights and the seats on one flight, reservations and itineraries, and the actors.

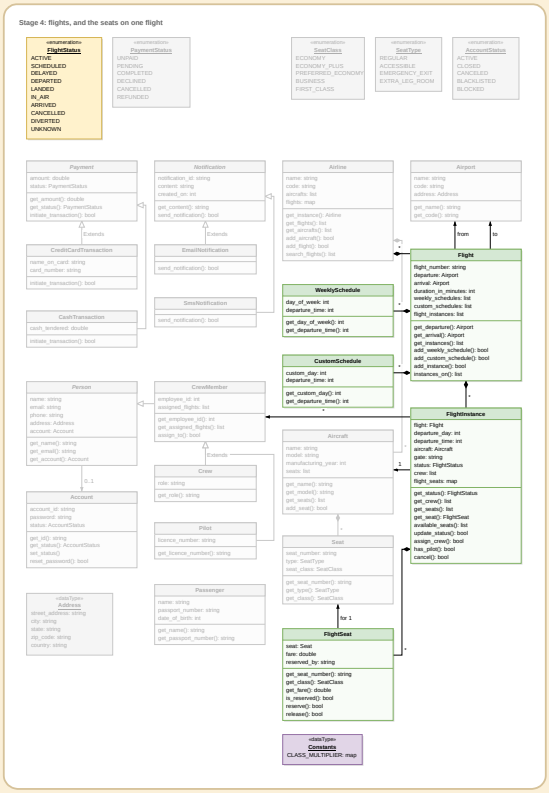


Figure — Flight, FlightInstance and FlightSeat: the route, the route on a day, and a seat on that day (course diagram)

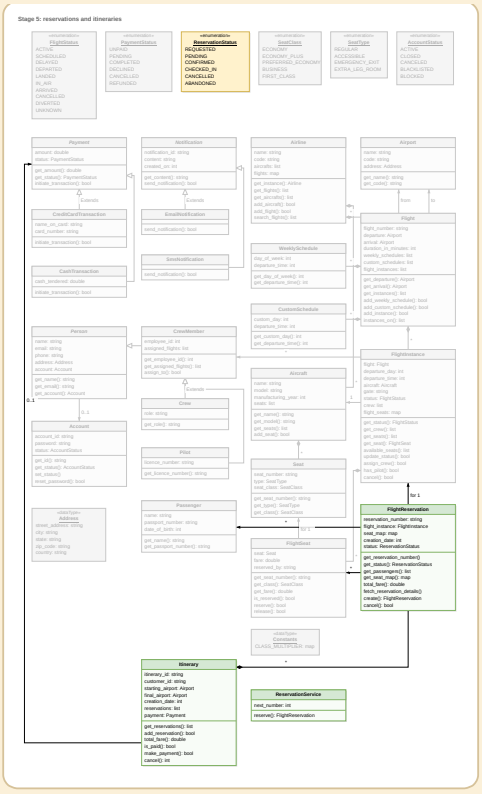


Figure — FlightReservation maps passengers to flight seats; an Itinerary owns its reservations (course diagram)

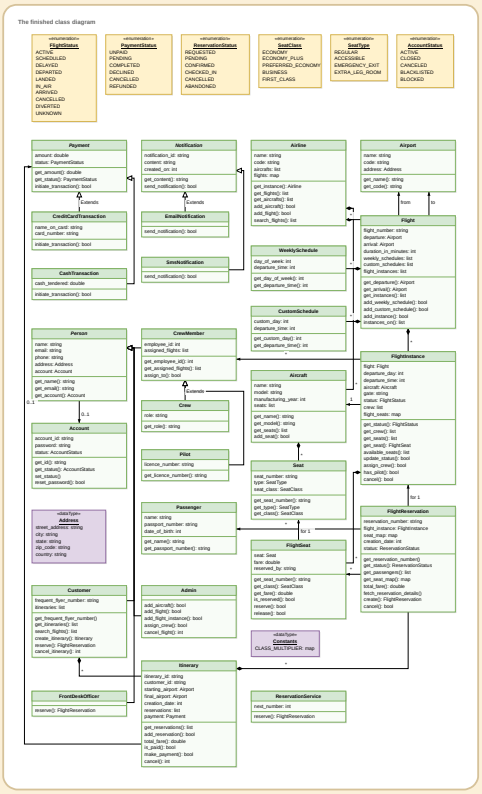
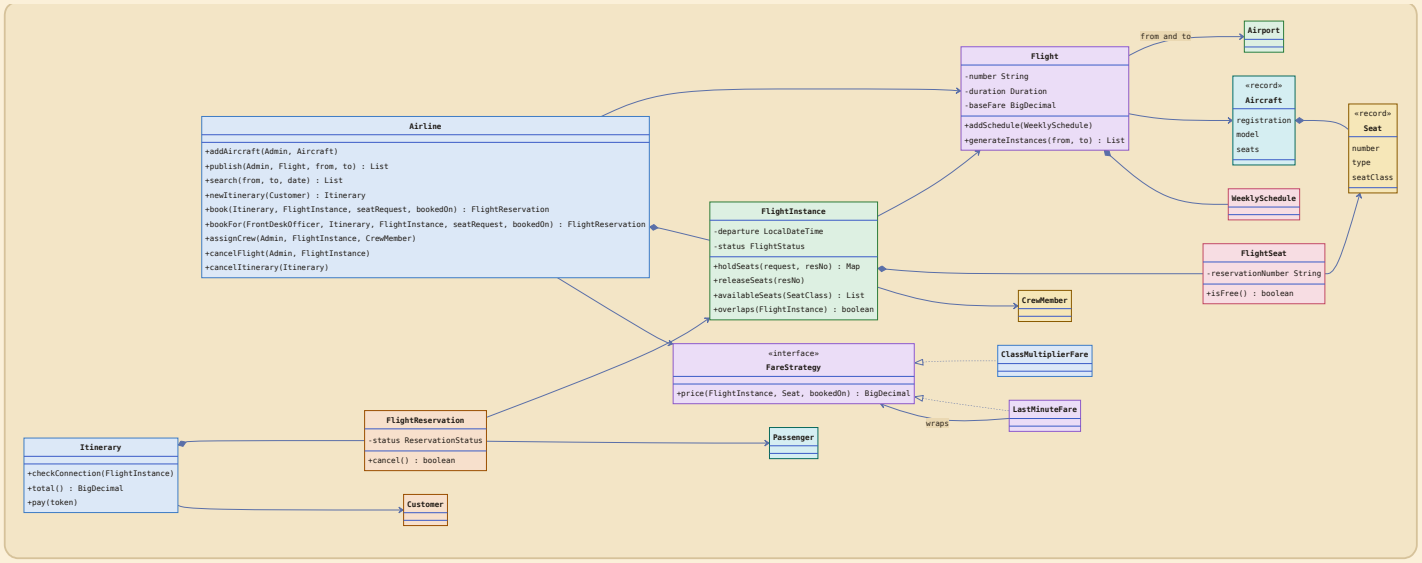


Figure — The finished class diagram of the airline management system (course diagram)

In the first figure above, all the ownership (filled-diamond) arrows run in one direction: schedules and instances belong to their flight, and flight seats belong to their instance. Airports, the aircraft and the crew are only referred to — cancelling a flight does not destroy an airport. Notice also in the finished diagram that `Passenger` sits outside the `Person`/account tree: the course's reason is that a passenger may never log in, and a customer can book for other people.



Key flows

Reserve a ticket. The customer (or front desk officer) searches; if a flight is available they choose seats and passengers, pay, and receive a confirmation; otherwise they search again.

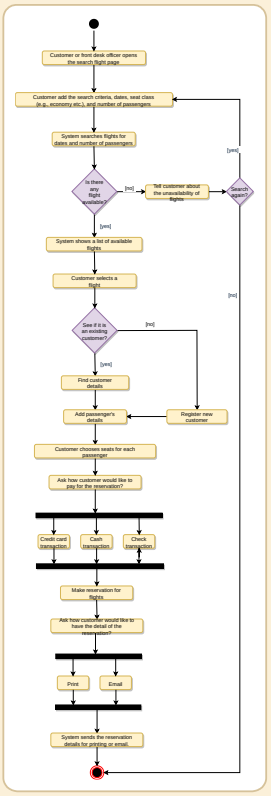


Figure — Activity diagram: reserving a ticket (course diagram)

Cancel a reservation. The system checks that the customer has a valid reservation; if so it cancels it, releases the seats and notifies the customer; if not, it reports the error.

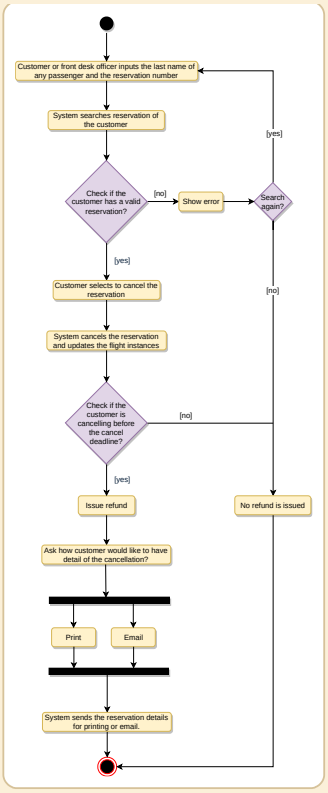
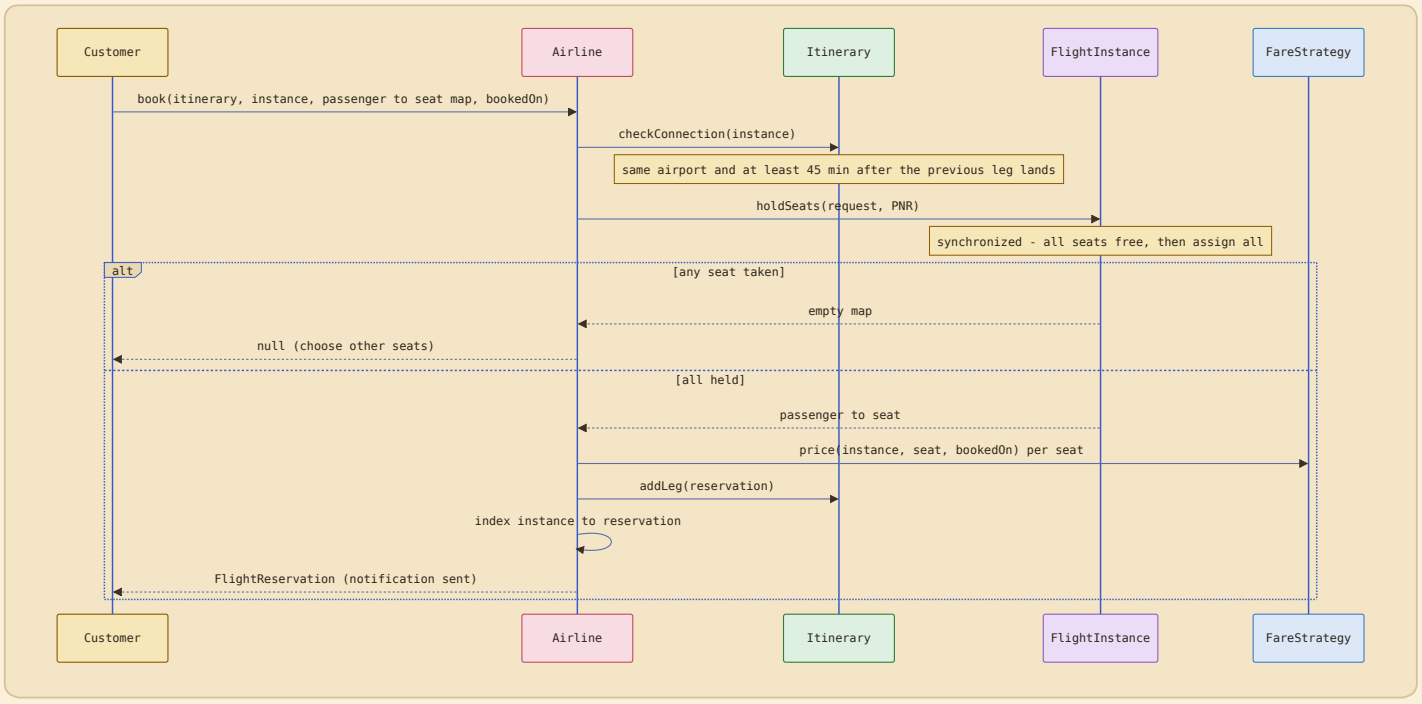
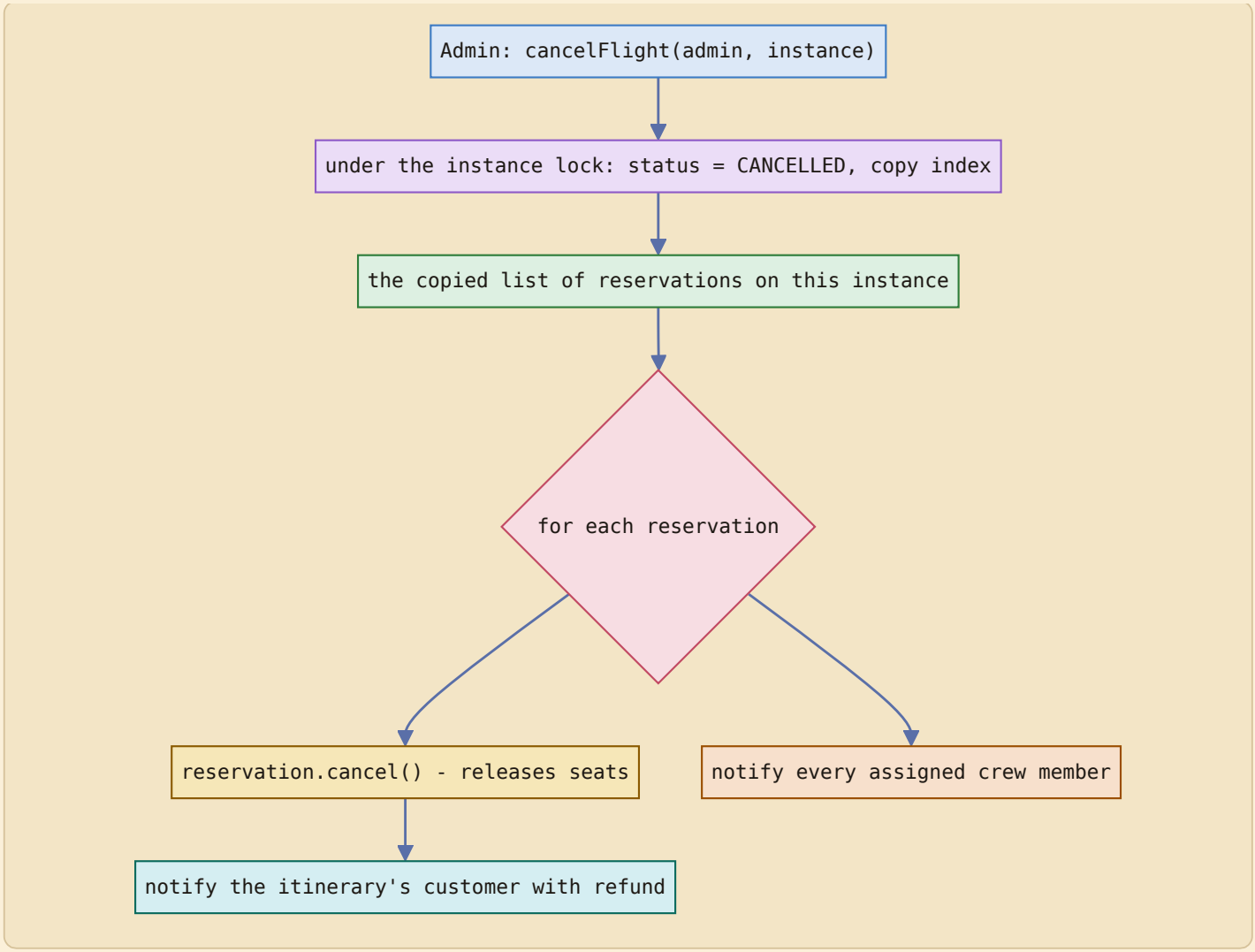


Figure — Activity diagram: cancelling a reservation (course diagram)

Booking one leg of an itinerary:



Admin cancels a flight instance:


Admin: cancelFlight(admin, instance)
under the instance lock: status = CANCELLED, copy index
the copied list of reservations on this instance
for each reservation
reservation.cancel() - releases seats
notify every assigned crew member
notify the itinerary's customer with refund

Design decisions & patterns used

Decision	Why	Alternative rejected
Flight vs FlightInstance vs Seat vs FlightSeat	12A is free on Tuesday and taken on Thursday, so "booked" and the fare belong to the seat *on an instance*. The course says an older version of its code had drifted from its diagram here — the code is what runs.	A `booked` flag on `Seat`: one booking would block the seat forever.
`holdSeats` is **all-or-nothing** and `synchronized` per instance	A family must get all seats or none; per-instance locks let different flights sell in parallel.	Holding seats one by one (half-booked families), or one global lock (no parallelism).
`Passenger` is not an account	A passenger is a name on a ticket; a customer books for others.	`Passenger extends Person` would force an account on a child.
Itinerary is paid, not the leg	A customer pays for the trip.	Paying per reservation.
Connection rule on `Itinerary`	The course names "no connecting-time check" as a gap; the rule belongs to the thing that holds the legs. We check same airport and a 45-minute minimum.	No check (impossible trips).
Index instance → reservations	The course's other named gap: cancelling a flight told the crew but not passengers, because there was no path from instance to its reservations. The index fixes it.	Scanning every itinerary.
Strategy + Decorator for fares	The course suggests Strategy for fares (base, seasonal, last minute). `ClassMultiplierFare` uses the course's multipliers (1.0, 1.3, 1.5, 2.5, 4.0); `LastMinuteFare` wraps any fare and adds 25% within 3 days.	Price field on the seat.
Crew overlap check	A pilot cannot be on two instances whose times overlap.	Trusting the admin.
Facade (`Airline`)	One entry point for admin and customer operations. Admin-only operations (`addAircraft`, `publish`, `assignCrew`, `cancelFlight`) take an `Admin` parameter, so "only the admin" is enforced by the compiler; `bookFor(FrontDeskOfficer, …)` is the same call as `book` made by a different actor, exactly as in the course's stage 6.	Actors holding the rules.
The course makes `Airline` a Singleton	We construct it once and pass it in, to stay testable.	Global `getInstance()`.

Java implementation

Package `lld.ch16.airline`.

```java
// File: code/lld/ch16/airline/Types.java
package lld.ch16.airline;

enum FlightStatus { SCHEDULED, DELAYED, DEPARTED, LANDED, CANCELLED }
enum ReservationStatus { CONFIRMED, CANCELLED }
enum SeatClass { ECONOMY, ECONOMY_PLUS, PREFERRED_ECONOMY, BUSINESS, FIRST_CLASS }
enum SeatType { REGULAR, ACCESSIBLE, EMERGENCY_EXIT, EXTRA_LEG_ROOM }
enum CrewRole { PILOT, CO_PILOT, CABIN_CREW }

record Airport(String code, String city) {}

/** The PHYSICAL seat bolted to an aircraft. It knows nothing about bookings. */
record Seat(String number, SeatType type, SeatClass seatClass) {}

/** A name on a ticket. Not an account holder: a customer can book for other people. */
record Passenger(String name, String passportRef) {}

record Customer(String id, String name, String email) {}

record CrewMember(String id, String name, CrewRole role) {}

/** Staff actors. Admin-only operations take an Admin parameter, so the compiler enforces "only the admin". */
record Admin(String id, String name) {}
record FrontDeskOfficer(String id, String name) {}
```

```java
// File: code/lld/ch16/airline/Fleet.java
package lld.ch16.airline;

import java.math.BigDecimal;
import java.time.DayOfWeek;
import java.time.Duration;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.List;

record Aircraft(String registration, String model, List<Seat> seats) {
    Aircraft { seats = List.copyOf(seats); }
}

/** A recurring slot: "every Tuesday at 10:00". */
record WeeklySchedule(DayOfWeek day, LocalTime departure) {}

/**
 * A Flight is the ROUTE (GA212 LHR->NRT). It creates one FlightInstance per scheduled day.
 * All times are in one reference zone (UTC) to keep the example small.
 */
final class Flight {
    private final String number;
    private final Airport from;
    private final Airport to;
    private final Duration duration;
    private final Aircraft aircraft;
    private final BigDecimal baseFare;
    private final List<WeeklySchedule> schedules = new ArrayList<>();

    Flight(String number, Airport from, Airport to, Duration duration, Aircraft aircraft, BigDecimal baseFare) {
        this.number = number; this.from = from; this.to = to;
        this.duration = duration; this.aircraft = aircraft; this.baseFare = baseFare;
    }
    String number() { return number; }
    Airport from() { return from; }
    Airport to() { return to; }
    Duration duration() { return duration; }
    Aircraft aircraft() { return aircraft; }
    BigDecimal baseFare() { return baseFare; }

    void addSchedule(WeeklySchedule s) { schedules.add(s); }

    List<FlightInstance> generateInstances(LocalDate fromDate, LocalDate toDateExclusive) {
        List<FlightInstance> out = new ArrayList<>();
        fromDate.datesUntil(toDateExclusive).forEach(d -> schedules.stream()
                .filter(s -> s.day() == d.getDayOfWeek())
                .forEach(s -> out.add(new FlightInstance(this, LocalDateTime.of(d, s.departure())))));
        return out;
    }
}
```

```java
// File: code/lld/ch16/airline/FlightInstance.java
package lld.ch16.airline;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** One seat on one flight instance: 12A can be free on Tuesday and taken on Thursday. */
final class FlightSeat {
    private final Seat seat;
    private String reservationNumber;   // null = free
    FlightSeat(Seat seat) { this.seat = seat; }
    Seat seat() { return seat; }
    String reservationNumber() { return reservationNumber; }
    boolean isFree() { return reservationNumber == null; }
    void assign(String resNo) { reservationNumber = resNo; }
    void free() { reservationNumber = null; }
}

/** The route flown on one date: owns its seats, its crew list and its status. */
final class FlightInstance {
    private final Flight flight;
    private final LocalDateTime departure;
    private final Map<String, FlightSeat> seats = new LinkedHashMap<>();
    private final List<CrewMember> crew = new ArrayList<>();
    private FlightStatus status = FlightStatus.SCHEDULED;

    FlightInstance(Flight flight, LocalDateTime departure) {
        this.flight = flight;
        this.departure = departure;
        flight.aircraft().seats().forEach(s -> seats.put(s.number(), new FlightSeat(s)));
    }

    Flight flight() { return flight; }
    LocalDateTime departure() { return departure; }
    LocalDateTime arrival() { return departure.plus(flight.duration()); }
    synchronized FlightStatus status() { return status; }
    synchronized List<CrewMember> crew() { return List.copyOf(crew); }
    String key() { return flight.number() + "@" + departure.toLocalDate(); }

    synchronized List<Seat> availableSeats(SeatClass c) {
        return seats.values().stream().filter(FlightSeat::isFree).map(FlightSeat::seat)
                .filter(s -> s.seatClass() == c).toList();
    }

    synchronized int freeSeatCount() { return (int) seats.values().stream().filter(FlightSeat::isFree).count(); }

    /** All-or-nothing: a family of three either gets all three seats or none. */
    synchronized Map<Passenger, Seat> holdSeats(Map<Passenger, String> request, String reservationNumber) {
        if (status != FlightStatus.SCHEDULED && status != FlightStatus.DELAYED)
            throw new IllegalStateException(key() + " is " + status);
        for (var seatNo : request.values()) {
            var fs = seats.get(seatNo);
            if (fs == null) throw new IllegalArgumentException("no seat " + seatNo + " on " + key());
            if (!fs.isFree()) return Map.of();
        }
        if (request.values().stream().distinct().count() != request.size())
            throw new IllegalArgumentException("two passengers asked for the same seat");
        Map<Passenger, Seat> out = new LinkedHashMap<>();
        request.forEach((p, no) -> { var fs = seats.get(no); fs.assign(reservationNumber); out.put(p, fs.seat());
});
        return out;
    }

    synchronized void releaseSeats(String reservationNumber) {
        seats.values().stream().filter(fs ->
```

```
reservationNumber.equals(fs.reservationNumber())).forEach(FlightSeat::free);
    }

    synchronized void addCrew(CrewMember m) { crew.add(m); }
    synchronized void setStatus(FlightStatus s) { status = s; }

    boolean overlaps(FlightInstance other) {
        return departure.isBefore(other.arrival()) && other.departure().isBefore(arrival());
    }
}
```

```java
// File: code/lld/ch16/airline/Booking.java
package lld.ch16.airline;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Duration;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;

/** Strategy: what one seat on one flight instance costs, booked on a given day. */
interface FareStrategy { BigDecimal price(FlightInstance fi, Seat seat, LocalDate bookedOn); }

final class ClassMultiplierFare implements FareStrategy {
    private static final Map<SeatClass, BigDecimal> MULTIPLIER = new EnumMap<>(Map.of(
            SeatClass.ECONOMY, new BigDecimal("1.0"), SeatClass.ECONOMY_PLUS, new BigDecimal("1.3"),
            SeatClass.PREFERRED_ECONOMY, new BigDecimal("1.5"), SeatClass.BUSINESS, new BigDecimal("2.5"),
            SeatClass.FIRST_CLASS, new BigDecimal("4.0")));
    public BigDecimal price(FlightInstance fi, Seat seat, LocalDate bookedOn) {
        return fi.flight().baseFare().multiply(MULTIPLIER.get(seat.seatClass())).setScale(2, RoundingMode.HALF_UP);
    }
}

/** Decorator on any fare: +25% when booked less than 3 days before departure. */
final class LastMinuteFare implements FareStrategy {
    private final FareStrategy inner;
    LastMinuteFare(FareStrategy inner) { this.inner = inner; }
    public BigDecimal price(FlightInstance fi, Seat seat, LocalDate bookedOn) {
        var p = inner.price(fi, seat, bookedOn);
        long daysBefore = ChronoUnit.DAYS.between(bookedOn, fi.departure().toLocalDate());
        return daysBefore < 3 ? p.multiply(new BigDecimal("1.25")).setScale(2, RoundingMode.HALF_UP) : p;
    }
}

/** One leg: one flight instance, several passengers, each with a seat. */
final class FlightReservation {
    private final String number;
    private final FlightInstance instance;
    private final Map<Passenger, Seat> seatMap;
    private final BigDecimal amount;
    private ReservationStatus status = ReservationStatus.CONFIRMED;

    FlightReservation(String number, FlightInstance instance, Map<Passenger, Seat> seatMap, BigDecimal amount) {
        this.number = number; this.instance = instance; this.seatMap = Map.copyOf(seatMap); this.amount = amount;
    }
    String number() { return number; }
    FlightInstance instance() { return instance; }
    Map<Passenger, Seat> seatMap() { return seatMap; }
    BigDecimal amount() { return amount; }
    synchronized ReservationStatus status() { return status; }

    /** Returns true if this call did the cancelling (idempotent). */
    synchronized boolean cancel() {
        if (status == ReservationStatus.CANCELLED) return false;
        status = ReservationStatus.CANCELLED;
        instance.releaseSeats(number);
        return true;
    }
}

/** A trip of one or more legs. The itinerary, not each leg, is what gets paid for. */
final class Itinerary {
    static final Duration MIN_CONNECTION = Duration.ofMinutes(45);

    private final String id;
```

```
    private final Customer customer;
    private final List<FlightReservation> legs = new ArrayList<>();
    private boolean paid;

    Itinerary(String id, Customer customer) { this.id = id; this.customer = customer; }
    String id() { return id; }
    Customer customer() { return customer; }
    synchronized List<FlightReservation> legs() { return List.copyOf(legs); }

    /** Rule the itinerary owns: the next leg must leave from where the last one landed, with time to change. */
    synchronized void checkConnection(FlightInstance next) {
        if (legs.isEmpty()) return;
        var last = legs.getLast().instance();
        if (!last.flight().to().equals(next.flight().from()))
            throw new IllegalArgumentException("leg must depart from " + last.flight().to().code());
        if (next.departure().isBefore(last.arrival().plus(MIN_CONNECTION)))
            throw new IllegalArgumentException("connection too short: " + next.key() + " leaves before " +
last.key() + " lands + 45 min");
    }

    synchronized void addLeg(FlightReservation r) { legs.add(r); }

    synchronized BigDecimal total() {
        return legs.stream().filter(l -> l.status() == ReservationStatus.CONFIRMED)
                .map(FlightReservation::amount).reduce(new BigDecimal("0.00"), BigDecimal::add);
    }

    /** Idempotent: a second pay() (double click, retry) does not charge again. */
    synchronized void pay(String cardToken) {
        if (paid) return;
        System.out.println("    charged " + total() + " to " + cardToken + " for itinerary " + id);
        paid = true;
    }
    synchronized boolean isPaid() { return paid; }
}
```

```java
// File: code/lld/ch16/airline/Airline.java
package lld.ch16.airline;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

interface Notifier { void send(String to, String message); }

final class EmailNotifier implements Notifier {
    public void send(String to, String message) { System.out.println("    [email " + to + "] " + message); }
}

/**
 * Facade over the airline's data: admin operations, search, booking, crew and cancellation.
 * Keeps an index flight-instance -> reservations so a cancelled flight can reach every passenger.
 */
final class Airline {
    private final String name;
    private final FareStrategy fares;
    private final Notifier notifier;
    private final Map<String, Aircraft> fleet = new ConcurrentHashMap<>();
    private final List<FlightInstance> instances = new CopyOnWriteArrayList<>();
    private final Map<FlightInstance, List<FlightReservation>> byInstance = new ConcurrentHashMap<>();
    private final Map<FlightReservation, Itinerary> owner = new ConcurrentHashMap<>();
    private final AtomicInteger seq = new AtomicInteger(100);

    Airline(String name, FareStrategy fares, Notifier notifier) {
        this.name = name; this.fares = fares; this.notifier = notifier;
    }

    // ---- admin operations ----
    void addAircraft(Admin by, Aircraft a) { fleet.put(a.registration(), a); }

    List<FlightInstance> publish(Admin by, Flight f, LocalDate from, LocalDate toExclusive) {
        var created = f.generateInstances(from, toExclusive);
        instances.addAll(created);
        return created;
    }

    /** A crew member cannot be on two instances whose times overlap. */
    synchronized void assignCrew(Admin by, FlightInstance fi, CrewMember m) {
        for (var other : instances) {
            if (other != fi && other.crew().contains(m) && other.overlaps(fi))
                throw new IllegalStateException(m.name() + " already flies " + other.key() + " at that time");
        }
        if (fi.crew().contains(m)) throw new IllegalStateException(m.name() + " already on " + fi.key());
        fi.addCrew(m);
    }

    List<FlightInstance> scheduleFor(CrewMember m) {
        return instances.stream().filter(fi -> fi.crew().contains(m)).toList();
    }

    void cancelFlight(Admin by, FlightInstance fi) {
        List<FlightReservation> affected;
        synchronized (fi) {        // same lock as the hold+index step in book(): no booking can slip past the
cancel
            fi.setStatus(FlightStatus.CANCELLED);
            affected = List.copyOf(byInstance.getOrDefault(fi, List.of()));
        }
        for (var r : affected) {
            if (r.cancel()) {
```

```
                var it = owner.get(r);
                notifier.send(it.customer().email(), fi.key() + " is cancelled. Reservation " + r.number()
                        + " refunded " + r.amount() + ". Itinerary " + it.id() + " now totals " + it.total());
            }
        }
        fi.crew().forEach(c -> notifier.send(c.id() + "@crew", "Stand down: " + fi.key() + " cancelled"));
    }

    // ---- customer operations ----
    List<FlightInstance> search(Airport from, Airport to, LocalDate date) {
        return instances.stream().filter(fi -> fi.flight().from().equals(from) && fi.flight().to().equals(to)
                && fi.departure().toLocalDate().equals(date) && fi.status() != FlightStatus.CANCELLED).toList();
    }

    Itinerary newItinerary(Customer c) { return new Itinerary("IT-" + seq.incrementAndGet(), c); }

    /** Books one leg for several passengers. Throws if the connection is invalid; returns null if a seat is taken.
*/
    FlightReservation book(Itinerary it, FlightInstance fi, Map<Passenger, String> seatRequest, LocalDate bookedOn)
{
        it.checkConnection(fi);
        String resNo = "PNR" + seq.incrementAndGet();
        FlightReservation r;
        synchronized (fi) {        // hold the seats AND index the reservation as one step (see cancelFlight)
            var seatMap = fi.holdSeats(seatRequest, resNo);
            if (seatMap.isEmpty()) return null;
            var amount = seatMap.values().stream().map(s -> fares.price(fi, s, bookedOn)).reduce(BigDecimal.ZERO,
BigDecimal::add);
            r = new FlightReservation(resNo, fi, seatMap, amount);
            owner.put(r, it);
            byInstance.computeIfAbsent(fi, k -> new CopyOnWriteArrayList<>()).add(r);
        }
        it.addLeg(r);
        var seatMap = r.seatMap();
        var amount = r.amount();
        notifier.send(it.customer().email(), "Booked " + fi.key() + " seats " +
seatMap.values().stream().map(Seat::number).sorted().toList() + " (" + resNo + ", " + amount + ")");
        return r;
    }

    /** The front desk books for a walk-in customer: the same call, made by a different actor. */
    FlightReservation bookFor(FrontDeskOfficer by, Itinerary it, FlightInstance fi, Map<Passenger, String>
seatRequest, LocalDate bookedOn) {
        return book(it, fi, seatRequest, bookedOn);
    }

    void cancelItinerary(Itinerary it) {
        it.legs().forEach(FlightReservation::cancel);
        notifier.send(it.customer().email(), "Itinerary " + it.id() + " cancelled");
    }

    String name() { return name; }
}
```

```java
// File: code/lld/ch16/airline/Main.java
package lld.ch16.airline;

import java.math.BigDecimal;
import java.time.DayOfWeek;
import java.time.Duration;
import java.time.LocalDate;
import java.time.LocalTime;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        var airline = new Airline("Global Air", new LastMinuteFare(new ClassMultiplierFare()), new
EmailNotifier());
        var lhr = new Airport("LHR", "London");
        var nrt = new Airport("NRT", "Tokyo");
        var jfk = new Airport("JFK", "New York");

        var seats = List.of(
                new Seat("1A", SeatType.REGULAR, SeatClass.BUSINESS), new Seat("1B", SeatType.REGULAR,
SeatClass.BUSINESS),
                new Seat("10A", SeatType.REGULAR, SeatClass.ECONOMY), new Seat("10B", SeatType.REGULAR,
SeatClass.ECONOMY),
                new Seat("10C", SeatType.ACCESSIBLE, SeatClass.ECONOMY), new Seat("12A", SeatType.EXTRA_LEG_ROOM,
SeatClass.ECONOMY_PLUS));
        var a1 = new Aircraft("G-GAAA", "A350-900", seats);
        var a2 = new Aircraft("G-GAAB", "B787-9", seats);
        var admin = new Admin("AD1", "Olga");
        airline.addAircraft(admin, a1);
        airline.addAircraft(admin, a2);

        System.out.println("1) Admin adds flights (routes) and weekly schedules, then publishes instances");
        var ga212 = new Flight("GA212", lhr, nrt, Duration.ofHours(14), a1, new BigDecimal("500.00"));
        ga212.addSchedule(new WeeklySchedule(DayOfWeek.TUESDAY, LocalTime.of(10, 0)));
        ga212.addSchedule(new WeeklySchedule(DayOfWeek.THURSDAY, LocalTime.of(10, 0)));
        var ga100 = new Flight("GA100", jfk, lhr, Duration.ofHours(7), a2, new BigDecimal("400.00"));
        ga100.addSchedule(new WeeklySchedule(DayOfWeek.WEDNESDAY, LocalTime.of(20, 0)));
        var ga300 = new Flight("GA300", lhr, jfk, Duration.ofHours(8), a2, new BigDecimal("450.00"));
        ga300.addSchedule(new WeeklySchedule(DayOfWeek.THURSDAY, LocalTime.of(12, 0)));
        var from = LocalDate.of(2026, 10, 5);
        var to = LocalDate.of(2026, 10, 19);
        var ga212Instances = airline.publish(admin, ga212, from, to);
        airline.publish(admin, ga100, from, to);
        airline.publish(admin, ga300, from, to);
        System.out.println("   GA212 instances: " + ga212Instances.stream().map(FlightInstance::key).toList());

        System.out.println("2) Search JFK->LHR on 7 Oct and LHR->NRT on 8 Oct");
        var leg1 = airline.search(jfk, lhr, LocalDate.of(2026, 10, 7)).getFirst();
        var leg2 = airline.search(lhr, nrt, LocalDate.of(2026, 10, 8)).getFirst();
        for (var fi : List.of(leg1, leg2))
            System.out.println("   " + fi.key() + " dep " + fi.departure() + " arr " + fi.arrival() + ", free seats
" + fi.freeSeatCount());

        System.out.println("3) Priya books a two-leg itinerary for herself and her son");
        var priya = new Customer("C1", "Priya", "priya@example.com");
        var me = new Passenger("Priya Shah", "P-001");
        var son = new Passenger("Arjun Shah", "P-002");
        var bookedOn = LocalDate.of(2026, 10, 1);
        var it = airline.newItinerary(priya);
        airline.book(it, leg1, ordered(me, "10A", son, "10B"), bookedOn);
        airline.book(it, leg2, ordered(me, "1A", son, "1B"), bookedOn);
        System.out.println("   itinerary total " + it.total() + " (2 x 400 economy + 2 x 1250 business)");
        it.pay("tok_priya_card");
```

```
        it.pay("tok_priya_card");   // double click: no second charge

        System.out.println("4) Bad connection is rejected");
        var it2 = airline.newItinerary(priya);
        airline.book(it2, leg1, Map.of(me, "12A"), bookedOn);
        try { airline.book(it2, ga212Instances.getFirst(), Map.of(me, "10A"), bookedOn); }
        catch (IllegalArgumentException e) { System.out.println("   rejected: " + e.getMessage()); }
        airline.cancelItinerary(it2);

        System.out.println("5) 20 customers race for seat 10C on " + leg2.key());
        var winners = new AtomicInteger();
        var go = new CountDownLatch(1);
        var raceAirline = airline;   // effectively-final alias for the lambdas
        try (var pool = Executors.newFixedThreadPool(20)) {
            for (int i = 0; i < 20; i++) {
                int n = i;
                pool.submit(() -> {
                    go.await();
                    var c = new Customer("R" + n, "Racer" + n, "racer" + n + "@example.com");
                    var r = raceAirline.book(raceAirline.newItinerary(c), leg2, Map.of(new Passenger("Racer" + n,
"X" + n), "10C"), bookedOn);
                    if (r != null) winners.incrementAndGet();
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("   successful bookings of 10C: " + winners.get() + ", free seats left: " +
leg2.freeSeatCount());

        System.out.println("6) Walk-in at the front desk, last-minute (booked 7 Oct for 8 Oct): pays +25%");
        var desk = new FrontDeskOfficer("FD1", "Tom");
        var late = airline.newItinerary(new Customer("C2", "Leo", "leo@example.com"));
        var r = airline.bookFor(desk, late, leg2, Map.of(new Passenger("Leo Park", "P-003"), "12A"),
LocalDate.of(2026, 10, 7));
        System.out.println("   economy-plus fare 500 x 1.3 = 650.00, charged " + r.amount());

        System.out.println("7) Admin assigns crew");
        var kim = new CrewMember("K1", "Capt. Kim", CrewRole.PILOT);
        var lee = new CrewMember("L1", "F/O Lee", CrewRole.CO_PILOT);
        var ana = new CrewMember("A1", "Ana", CrewRole.CABIN_CREW);
        for (var m : List.of(kim, lee, ana)) airline.assignCrew(admin, leg2, m);
        var ga300Oct8 = airline.search(lhr, jfk, LocalDate.of(2026, 10, 8)).getFirst();
        try { airline.assignCrew(admin, ga300Oct8, kim); }
        catch (IllegalStateException e) { System.out.println("   rejected: " + e.getMessage()); }
        System.out.println("   Kim's schedule: " +
airline.scheduleFor(kim).stream().map(FlightInstance::key).toList());

        System.out.println("8) Admin cancels " + leg2.key() + ": every passenger and crew member is told");
        airline.cancelFlight(admin, leg2);
        System.out.println("   search LHR->NRT on 8 Oct now returns " + airline.search(lhr, nrt, LocalDate.of(2026,
10, 8)).size() + " flights");

        System.out.println("9) Priya cancels the rest of her trip; seats on " + leg1.key() + " come back");
        airline.cancelItinerary(it);
        System.out.println("   free economy seats on " + leg1.key() + ": " +
leg1.availableSeats(SeatClass.ECONOMY).stream().map(Seat::number).toList());
    }

    static Map<Passenger, String> ordered(Passenger a, String sa, Passenger b, String sb) {
        var m = new LinkedHashMap<Passenger, String>();
        m.put(a, sa);
        m.put(b, sb);
        return m;
    }
}
```

Output:

```
1) Admin adds flights (routes) and weekly schedules, then publishes instances
   GA212 instances: [GA212@2026-10-06, GA212@2026-10-08, GA212@2026-10-13, GA212@2026-10-15]
2) Search JFK->LHR on 7 Oct and LHR->NRT on 8 Oct
   GA100@2026-10-07 dep 2026-10-07T20:00 arr 2026-10-08T03:00, free seats 6
   GA212@2026-10-08 dep 2026-10-08T10:00 arr 2026-10-09T00:00, free seats 6
3) Priya books a two-leg itinerary for herself and her son
    [email priya@example.com] Booked GA100@2026-10-07 seats [10A, 10B] (PNR102, 800.00)
    [email priya@example.com] Booked GA212@2026-10-08 seats [1A, 1B] (PNR103, 2500.00)
   itinerary total 3300.00 (2 x 400 economy + 2 x 1250 business)
    charged 3300.00 to tok_priya_card for itinerary IT-101
4) Bad connection is rejected
    [email priya@example.com] Booked GA100@2026-10-07 seats [12A] (PNR105, 520.00)
   rejected: connection too short: GA212@2026-10-06 leaves before GA100@2026-10-07 lands + 45 min
    [email priya@example.com] Itinerary IT-104 cancelled
5) 20 customers race for seat 10C on GA212@2026-10-08
    [email racer7@example.com] Booked GA212@2026-10-08 seats [10C] (PNR115, 500.00)
   successful bookings of 10C: 1, free seats left: 3
6) Walk-in at the front desk, last-minute (booked 7 Oct for 8 Oct): pays +25%
    [email leo@example.com] Booked GA212@2026-10-08 seats [12A] (PNR147, 812.50)
   economy-plus fare 500 x 1.3 = 650.00, charged 812.50
7) Admin assigns crew
   rejected: Capt. Kim already flies GA212@2026-10-08 at that time
   Kim's schedule: [GA212@2026-10-08]
8) Admin cancels GA212@2026-10-08: every passenger and crew member is told
    [email priya@example.com] GA212@2026-10-08 is cancelled. Reservation PNR103 refunded 2500.00. Itinerary IT-101
now totals 800.00
    [email racer7@example.com] GA212@2026-10-08 is cancelled. Reservation PNR115 refunded 500.00. Itinerary IT-107
now totals 0.00
    [email leo@example.com] GA212@2026-10-08 is cancelled. Reservation PNR147 refunded 812.50. Itinerary IT-146 now
totals 0.00
    [email K1@crew] Stand down: GA212@2026-10-08 cancelled
    [email L1@crew] Stand down: GA212@2026-10-08 cancelled
    [email A1@crew] Stand down: GA212@2026-10-08 cancelled
   search LHR->NRT on 8 Oct now returns 0 flights
9) Priya cancels the rest of her trip; seats on GA100@2026-10-07 come back
    [email priya@example.com] Itinerary IT-101 cancelled
   free economy seats on GA100@2026-10-07: [10A, 10B, 10C]
```

Step 3 prices 2 × 400 economy + 2 × (500 × 2.5) business = 3300, and the second `pay` call charges nothing. Step 5 has 20 threads racing for seat 10C and exactly one wins (which racer wins, and therefore the PNR and itinerary numbers printed for them in steps 5 and 8, changes from run to run). Step 6 is 500 × 1.3 = 650, plus 25% last-minute = 812.50. Step 8 reaches every passenger through the index — the gap the course points out.

Concurrency & edge cases

- **Selling one seat twice.** `holdSeats` is `synchronized` on the instance: it checks every requested seat and only then assigns them all. No other thread can slip in between.
- **Booking while the admin cancels the flight.** `book` holds the seats *and* adds the reservation to the instance → reservations index inside one `synchronized (fi)` block, and `cancelFlight` sets `CANCELLED` and copies that index under the same lock. So every reservation is either in the copy (and gets cancelled and notified) or is refused by `holdSeats` because the flight is already `CANCELLED` — none can slip through un-notified.
- **Double payment.** `Itinerary.pay` returns at once if the itinerary is already paid (step 3 calls it twice and charges once).
- **Duplicate seat in one request** (two passengers asking for 12A) is rejected.
- **Booking a cancelled or departed flight** throws.
- **Idempotent cancel.** `FlightReservation.cancel()` returns `false` if already cancelled, so a customer cancelling while the admin cancels the flight does not double-refund.
- **Crew double-booking** across overlapping instances is rejected (`assignCrew` is `synchronized` over the scan and the add).
- **Time zones.** Our example keeps all times in one reference zone. Real systems store departures as `ZonedDateTime` (a date-time plus its time zone) or `Instant` (an exact moment on the global UTC timeline) with each airport's zone, because London and Tokyo are 8–9 hours apart.

- **Seat holds during payment.** Real booking flows put a seat on a short *hold* (for example 10–15 minutes) while the customer pays, and release it automatically if payment fails. Add a `heldUntil` timestamp on `FlightSeat` and a sweeper (a background job that regularly frees expired holds).

Extensions & follow-up questions

- **"Overbooking on purpose?"** — Airlines sell a few more seats than exist because some passengers do not show up. Model a per-class `overbookLimit` and assign physical seats at check-in.
- **"Check-in and boarding passes?"** — Add `CHECKED_IN` per passenger, a `BoardingPass` (seat, gate, group), and a check-in window.
- **"Baggage?"** — `Baggage(passenger, weight, tag)` on the reservation; fees as extra lines.
- **"Change fees and fare rules?"** — Fare class with rules (refundable, change fee); cancellation refund = fare − fee.
- **"Codeshares?"** — A codeshare is one physical flight sold under several airlines' flight numbers. A `MarketingFlight` (partner's number) pointing at the operating `FlightInstance`.
- **"Delays?"** — `FlightInstance.setStatus(DELAYED)` plus a new departure; notify via the same index; re-check itinerary connections and rebook broken ones.
- **"Scale?"** — Seat inventory per instance is a hot row (one database row that very many requests try to change at once); use one row per seat with a conditional update (`UPDATE … WHERE reservation IS NULL` — it only changes the row if the condition still holds, so exactly one buyer wins) or optimistic locking (no lock is taken; each row carries a version number and a write fails if the version changed since you read it, so you retry), and a cache (a fast in-memory copy of data that is read often) for search.

In the interview

30-second pitch: "Four separate ideas carry the design. `Flight` is the published route; `FlightInstance` is one departure of it on a particular day; `Seat` is the chair fixed in the aircraft; `FlightSeat` is that chair on that one departure, and it is where the fare and the booking live. Reservations map passengers to flight seats and are held atomically per instance. An itinerary groups legs, checks connections and is what gets paid. Fares are a strategy with a last-minute decorator. An index from instance to reservations lets a cancelled flight notify every passenger."

Minutes	Do
0–5	Clarify search, itineraries, multiple passengers, crew, cancellation.
5–10	Actors and use cases.
10–20	The four-way distinction, then the class diagram.
20–35	Code `FlightInstance.holdSeats`, `Airline.book`, `Itinerary.checkConnection`, `cancelFlight`.
35–40	Seat holds during payment, overbooking, time zones.

Expected follow-up: "A flight is cancelled — how do its passengers find out?" — "Through an index from instance to reservations; without it you can only reach the crew."

Self-check

1. **Q:** Where does the "booked" flag live and why? **A:** On `FlightSeat` — a seat's booking is per instance (per date), not per aircraft.
2. **Q:** Why is `Passenger` not a subclass of `Person`/account? **A:** A passenger may never log in; a customer books for other people.
3. **Q:** How do you guarantee a family gets all seats or none? **A:** `holdSeats` checks all requested seats and assigns them in one synchronized block.
4. **Q:** What does `LastMinuteFare` add and which pattern is it? **A:** +25% within 3 days of departure; a Decorator over any `FareStrategy`.
5. **Q:** What two gaps did the course admit and how did we close them? **A:** No passenger notification on flight cancel (fixed by the instance → reservations index) and no connection-time check (fixed in `Itinerary.checkConnection`).

TL;DR

- Flight (route) ≠ FlightInstance (route on a date); Seat (physical) ≠ FlightSeat (seat on that date).
- Hold seats all-or-nothing under the instance's lock.
- Itinerary = legs + connection rule + payment.
- Fares: Strategy + Decorator. Keep an instance → reservations index for cancellations.

Real-world example. Airline systems separate a *flight number* (a published schedule) from a *flight leg on a date*; inventory is controlled per leg and per booking class, and the booking record (a PNR, passenger name record) groups passengers and segments — the same shape as `FlightReservation` and `Itinerary` here. When an airline cancels a flight, its systems find every PNR on that leg and rebook or notify passengers, which is exactly why the index matters.

Design a Course Registration System

Problem & clarifying questions

A **course registration system** lets university students sign up for classes each term. It looks simple, but it is one of the harshest concurrency problems in this chapter: at 09:00 on registration day, thousands of students click "register" for the same few popular sections within seconds.

Terms first:

- A **course** is a catalogue entry (CS201 Data Structures, 4 credits).
- A **section** is one offering of a course with its own instructor, meeting time and capacity (CS201-A, Mon/Wed 10:00–11:30, 30 seats). Students register for **sections**, not courses — this is the same split as Flight vs FlightInstance.
- A **prerequisite** is a course you must have completed first.
- A **waitlist** is a first-come-first-served queue of students who want a seat in a full section.
- A **registration window** is the period when a student may register; universities often open it earlier for seniors (priority registration).

Question	Assumed answer
Register for courses or sections?	Sections.
Prerequisites?	Must be completed (not just in progress).
Capacity?	Per section; when full, join a bounded waitlist.
Waitlist promotion?	Automatic when someone drops, in FIFO order, after re-checking the student's timetable.
Limits?	A per-student credit limit; no two sections that meet at the same time; no two sections of the same course.
Windows?	Opens by class year (seniors first); closes for everyone at one time.
Search?	By course code or title.
Concurrency?	Many students at once; a section must never exceed its capacity; one student must not double-register from two tabs.

Requirements

Functional

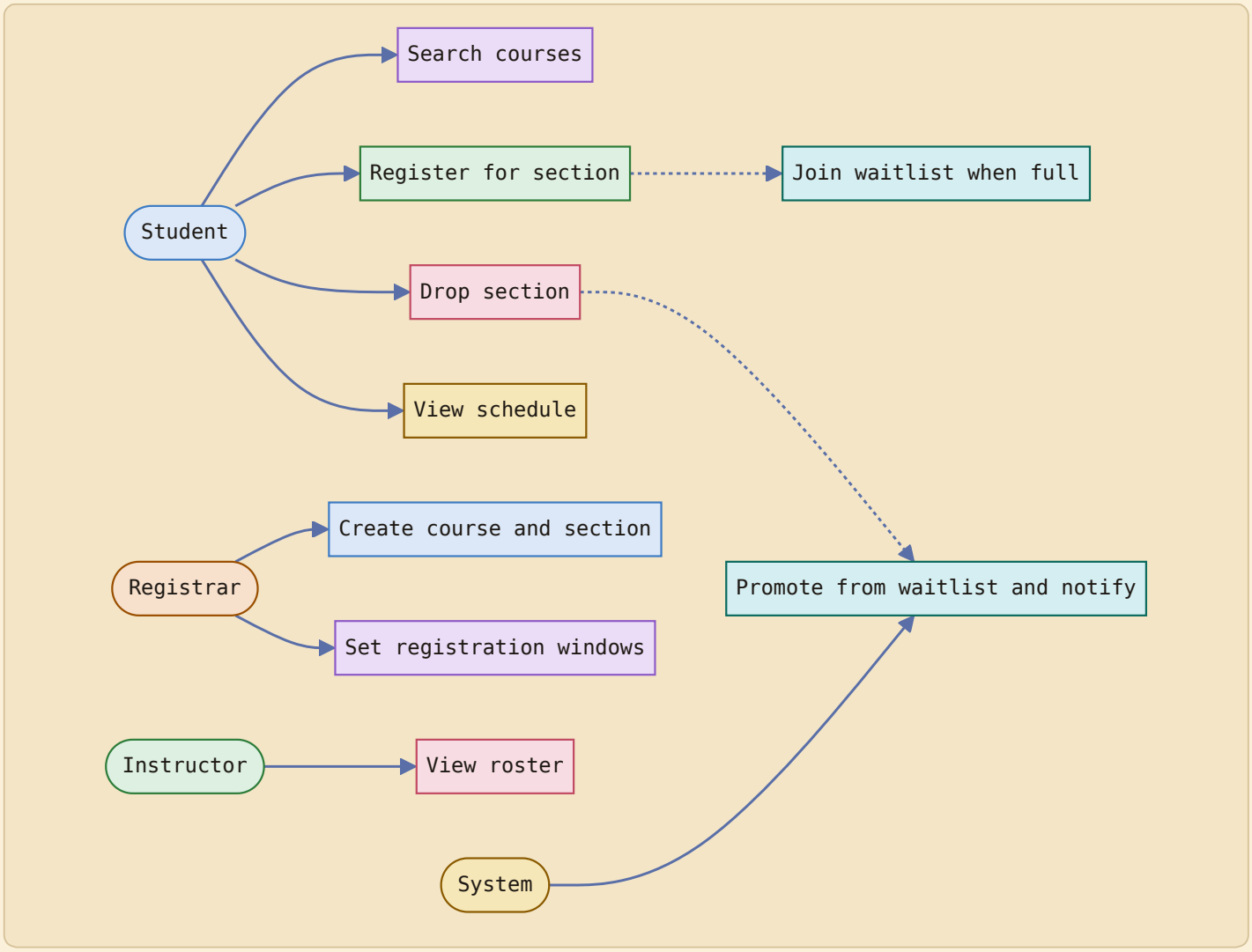
1. Students register for sections and view their registered sections.
2. Each course has a code, name and credits; each section has an instructor, time slot and enrolment limit.
3. Search courses by code or name.
4. Never enrol beyond a section's capacity; when full, put the student on the waitlist (if it has room).
5. Enforce prerequisites, credit limit, time conflicts, one section per course, and the registration window.
6. On drop, promote the first student on the waitlist automatically and notify them.

Non-functional

- Handle concurrent registrations safely: no race conditions, consistent data.
- Rules must be easy to add or remove (for example "department majors only").
- Extensible for future features (fees, holds, cross-listed courses).

Use cases

- **Student** — searches courses, registers, drops, views schedule, joins a waitlist.
- **Registrar / Admin** — creates courses and sections, sets capacity and windows.
- **Instructor** — views the roster.
- **System** — enforces rules, promotes from the waitlist, sends notifications.

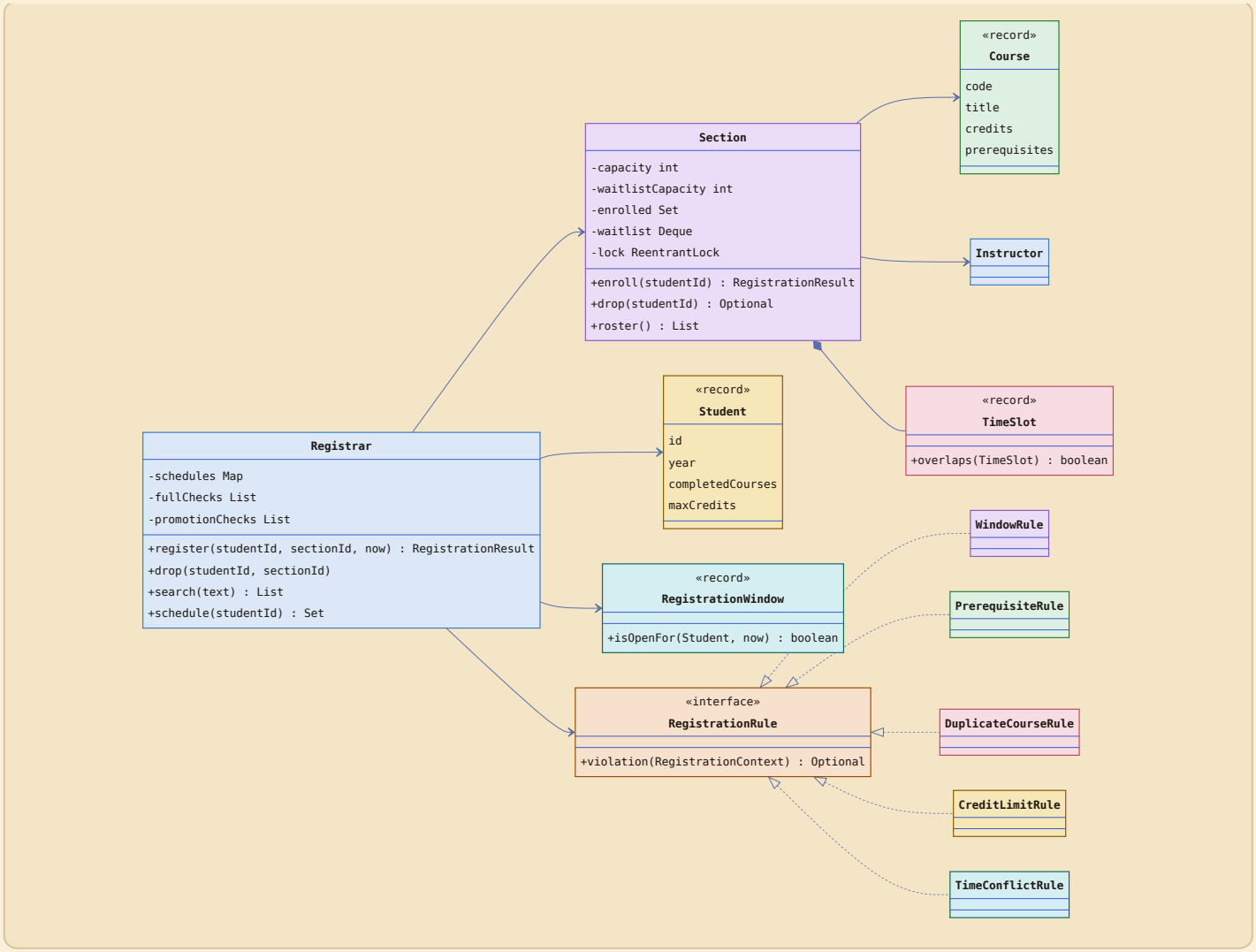


Core entities

Class	Responsibility
`Course`	Code, title, credits, prerequisite codes.
`Section`	One offering: instructor, time slot, capacity, enrolled set, waitlist. **The only place a seat is taken**, under its own lock.
`TimeSlot`	Days + start/end; knows whether it overlaps another slot.
`Instructor`	Who teaches.
`Student`	Year, completed courses, credit limit.
`RegistrationWindow`	Opening time per class year and a common closing time.
`RegistrationRule` (+ `WindowRule`, `PrerequisiteRule`, `DuplicateCourseRule`, `CreditLimitRule`, `TimeConflictRule`)	One rule each; returns the reason for a violation.
`RegistrationContext`	Everything a rule may look at.
`RegistrationResult`, `Outcome`	`ENROLLED`, `WAITLISTED` or `REJECTED` + message.
`Registrar`	The service: runs the rule chain, takes the seat, drops and promotes, keeps each student's schedule, and defines the lock order.

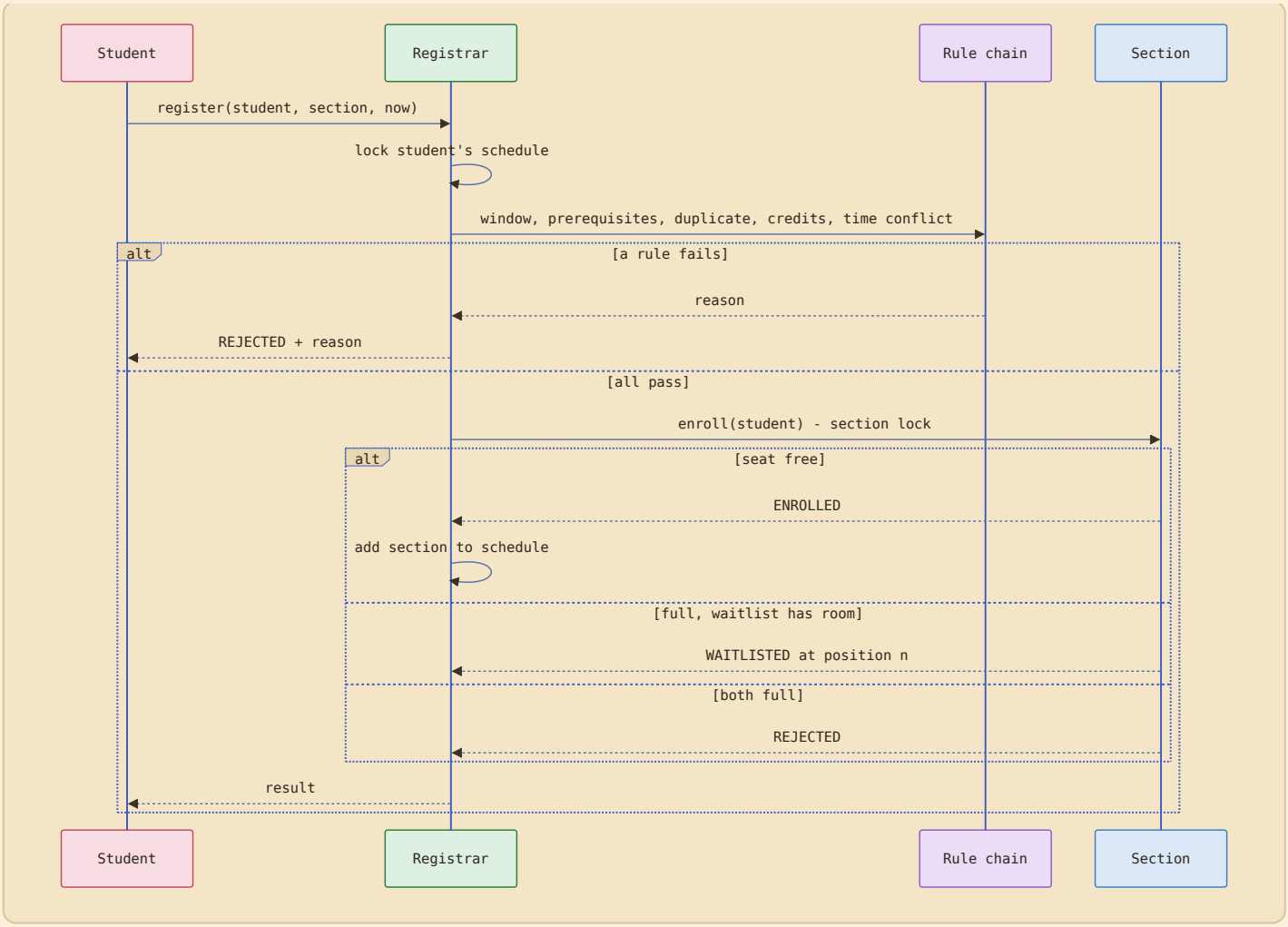
Class diagram

There is no course lesson for this problem. The problem statement comes from the *awesome-low-level-design* problem set: students register for courses and view their enrolments; a course has a code, name, instructor and maximum capacity; search by code or name; no enrolment past capacity; many students registering at once without race conditions; room to extend. Every one of those is covered above (the instructor and capacity sit on the `Section`). Here is the diagram of our Java design.

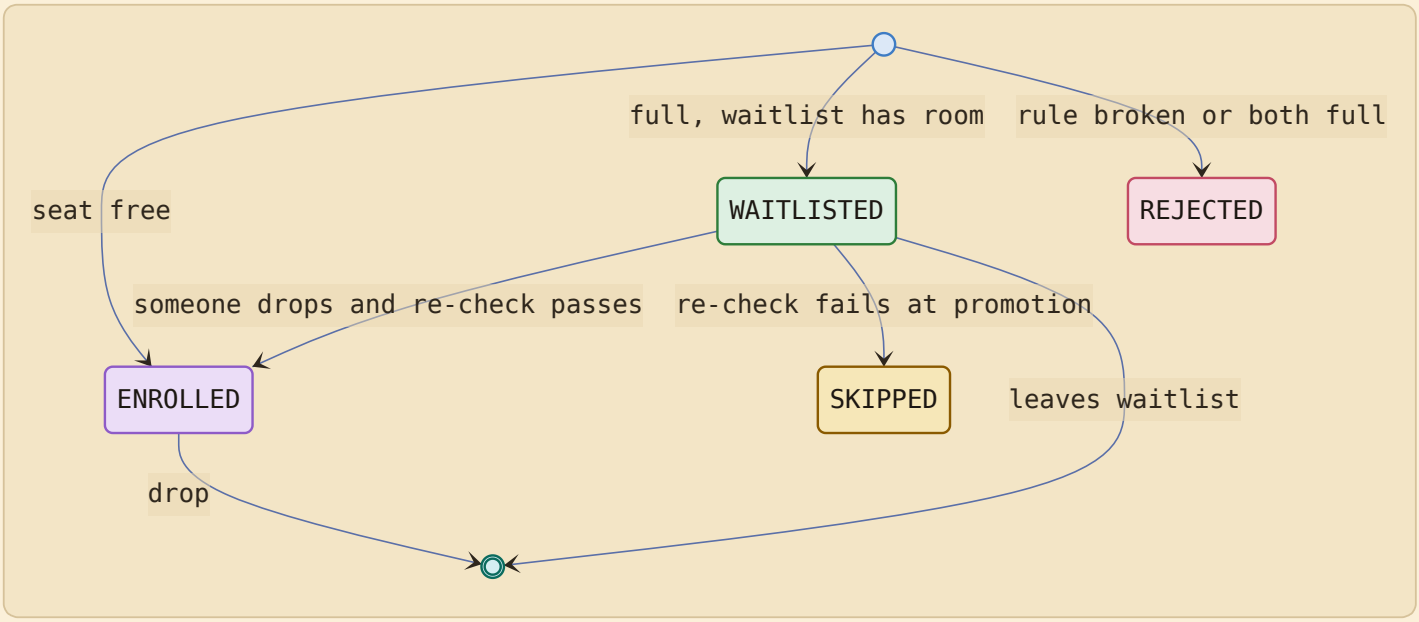


Key flows

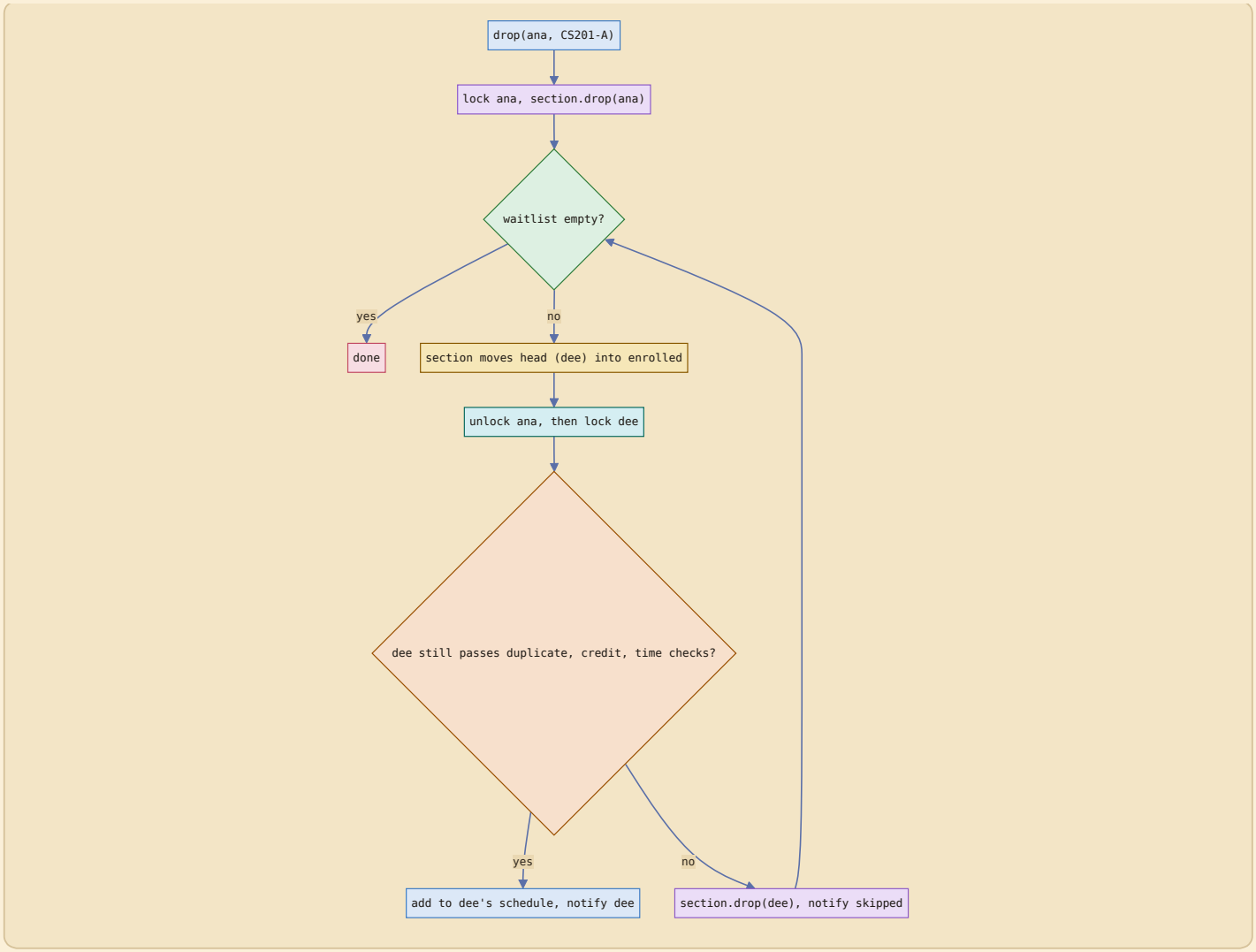
Register:



A student's status in one section:



Drop and promote:


drop(ana, CS201-A)
lock ana, section.drop(ana)
waitlist empty?
yes
no
done
section moves head (dee) into enrolled
unlock ana, then lock dee
dee still passes duplicate, credit, time checks?
yes
no
add to dee's schedule, notify dee
section.drop(dee), notify skipped

Design decisions & patterns used

Decision	Why	Alternative rejected
Course vs Section	Capacity, time and instructor belong to an offering, not the catalogue entry.	Capacity on `Course` (cannot have two sections).
Seat taken only inside `Section.enroll`, under a `ReentrantLock` (Java's explicit lock object, which the same thread may re-acquire) per section	Check-capacity-then-add is atomic; different sections do not block each other.	`synchronized` on the whole registrar (every student in the university waits in one line).
Per-student lock around rules + enroll	Credit limit and time-conflict checks read the schedule; two tabs registering the same student would both pass without it (step 6 of the output).	Lock only the section (a student could exceed credits via two sections in parallel).
Fixed lock order: student, then section	Deadlock needs two threads taking the same two locks in opposite orders. Always taking student before section rules it out. Promotion takes the next student's lock only *after* releasing the dropper's, so two student locks are never held together.	Promoting while still holding the dropper's lock (student → section → student: deadlock risk).
Rules as objects (Specification / Chain of Responsibility style: each check is its own object, and a request passes along the chain until one rejects it)	Each rule is one small class; the registrar runs them in order and stops at the first failure. A new rule ("majors only") is a new class plus one list entry — Open/Closed. The promotion path reuses a *subset* of the same rules.	One long `register` method with nested `if`s.
Waitlist is a FIFO `Deque`, bounded (a double-ended queue with a maximum size)	Fair (first come, first served) and prevents unbounded queues.	Unbounded list; priority by year (a valid extension).
Re-check on promotion	A student may have registered for a clashing section while waiting.	Promote blindly (broken timetables).
Time passed in (`now`)	Makes window logic testable without waiting for real dates.	Calling `LocalDateTime.now()` inside rules.
Singleton? The GitHub problem uses a singleton `CourseRegistrationSystem`.	We construct one `Registrar` and pass it in: one instance, still testable.	Global `getInstance()`.

Java implementation

Package `lld.ch16.courseregistration`.

```java
// File: code/lld/ch16/courseregistration/Types.java
package lld.ch16.courseregistration;

import java.time.DayOfWeek;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.EnumSet;
import java.util.Map;
import java.util.Set;

/** A catalog entry: what is taught. Prerequisites are course codes. */
record Course(String code, String title, int credits, Set<String> prerequisites) {
    Course { prerequisites = Set.copyOf(prerequisites); }
}

record Instructor(String id, String name) {}

/** When a section meets, e.g. Mon+Wed 10:00-11:30. */
record TimeSlot(Set<DayOfWeek> days, LocalTime start, LocalTime end) {
    TimeSlot {
        days = EnumSet.copyOf(days);
        if (!end.isAfter(start)) throw new IllegalArgumentException("end must be after start");
    }
    boolean overlaps(TimeSlot o) {
        boolean sharedDay = days.stream().anyMatch(o.days::contains);
        return sharedDay && start.isBefore(o.end) && o.start.isBefore(end);
    }
    @Override public String toString() { return days + " " + start + "-" + end; }
}

record Student(String id, String name, int year, Set<String> completedCourses, int maxCredits) {
    Student { completedCourses = Set.copyOf(completedCourses); }
}

/** Priority registration: each class year opens at its own time; everyone closes together. */
record RegistrationWindow(Map<Integer, LocalDateTime> opensByYear, LocalDateTime closes) {
    boolean isOpenFor(Student s, LocalDateTime now) {
        var opens = opensByYear.get(s.year());
        return opens != null && !now.isBefore(opens) && now.isBefore(closes);
    }
}

enum Outcome { ENROLLED, WAITLISTED, REJECTED }

record RegistrationResult(Outcome outcome, String message) {
    static RegistrationResult rejected(String why) { return new RegistrationResult(Outcome.REJECTED, why); }
}
```

```java
// File: code/lld/ch16/courseregistration/Section.java
package lld.ch16.courseregistration;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.locks.ReentrantLock;

/**
 * One offering of a course (CS201 section A). Capacity and waitlist live HERE and are
 * guarded by the section's own lock, so thousands of students can hit different sections in parallel.
 */
final class Section {
    private final String id;
    private final Course course;
    private final Instructor instructor;
    private final TimeSlot slot;
    private final int capacity;
    private final int waitlistCapacity;
    private final Set<String> enrolled = new LinkedHashSet<>();
    private final Deque<String> waitlist = new ArrayDeque<>();
    private final ReentrantLock lock = new ReentrantLock();

    Section(String id, Course course, Instructor instructor, TimeSlot slot, int capacity, int waitlistCapacity) {
        this.id = id; this.course = course; this.instructor = instructor; this.slot = slot;
        this.capacity = capacity; this.waitlistCapacity = waitlistCapacity;
    }

    String id() { return id; }
    Course course() { return course; }
    Instructor instructor() { return instructor; }
    TimeSlot slot() { return slot; }

    /** The only place a seat is taken: check-and-add in one critical section. */
    RegistrationResult enroll(String studentId) {
        lock.lock();
        try {
            if (enrolled.contains(studentId)) return RegistrationResult.rejected("already enrolled in " + id);
            if (waitlist.contains(studentId)) return RegistrationResult.rejected("already on the waitlist of " +
id);
            if (enrolled.size() < capacity) {
                enrolled.add(studentId);
                return new RegistrationResult(Outcome.ENROLLED, "enrolled in " + id + " (" + enrolled.size() + "/"
+ capacity + ")");
            }
            if (waitlist.size() < waitlistCapacity) {
                waitlist.addLast(studentId);
                return new RegistrationResult(Outcome.WAITLISTED, "waitlisted for " + id + " at position " +
waitlist.size());
            }
            return RegistrationResult.rejected(id + " and its waitlist are full");
        } finally {
            lock.unlock();
        }
    }

    /** Removes the student; if a seat opened, moves the head of the waitlist in and returns who it was. */
    Optional<String> drop(String studentId) {
        lock.lock();
        try {
            if (waitlist.remove(studentId)) return Optional.empty();
            if (!enrolled.remove(studentId)) throw new IllegalArgumentException(studentId + " is not in " + id);
            String next = waitlist.pollFirst();
            if (next != null) enrolled.add(next);
            return Optional.ofNullable(next);
```

```
        } finally {
            lock.unlock();
        }
    }

    boolean isEnrolled(String studentId) {
        lock.lock();
        try { return enrolled.contains(studentId); } finally { lock.unlock(); }
    }

    List<String> roster() {
        lock.lock();
        try { return List.copyOf(enrolled); } finally { lock.unlock(); }
    }

    List<String> waitlist() {
        lock.lock();
        try { return List.copyOf(waitlist); } finally { lock.unlock(); }
    }
}
```

```java
// File: code/lld/ch16/courseregistration/Rules.java
package lld.ch16.courseregistration;

import java.time.LocalDateTime;
import java.util.Optional;
import java.util.Set;

/** What a rule may look at: who, which section, what they already have, and when. */
record RegistrationContext(Student student, Section section, Set<Section> currentSchedule, LocalDateTime now,
                           RegistrationWindow window) {}

/**
 * Specification-style rule: returns a reason when the request breaks it.
 * The registrar runs the rules as a chain and stops at the first violation.
 */
interface RegistrationRule { Optional<String> violation(RegistrationContext c); }

final class WindowRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        return c.window().isOpenFor(c.student(), c.now()) ? Optional.empty()
                : Optional.of("registration window is closed for year " + c.student().year() + " at " + c.now());
    }
}

final class PrerequisiteRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        var missing = c.section().course().prerequisites().stream()
                .filter(p -> !c.student().completedCourses().contains(p)).sorted().toList();
        return missing.isEmpty() ? Optional.empty() : Optional.of("missing prerequisites " + missing);
    }
}

final class DuplicateCourseRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        return c.currentSchedule().stream()
                .filter(s -> s.course().code().equals(c.section().course().code())).findFirst()
                .map(s -> "already registered for " + s.course().code() + " in section " + s.id());
    }
}

final class CreditLimitRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        int current = c.currentSchedule().stream().mapToInt(s -> s.course().credits()).sum();
        int after = current + c.section().course().credits();
        return after <= c.student().maxCredits() ? Optional.empty()
                : Optional.of("credit limit " + c.student().maxCredits() + " exceeded (" + after + ")");
    }
}

final class TimeConflictRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        return c.currentSchedule().stream().filter(s -> s.slot().overlaps(c.section().slot())).findFirst()
                .map(s -> "time conflict with " + s.id() + " " + s.slot());
    }
}
```

```java
// File: code/lld/ch16/courseregistration/Registrar.java
package lld.ch16.courseregistration;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.function.Consumer;

/**
 * The service. Lock order is ALWAYS student-then-section, which rules out deadlock:
 *  - the student lock makes "check my credits/time clashes, then take the seat" atomic per student
 *    (stops one student double-registering from two browser tabs);
 *  - the section lock (inside Section) makes the seat count atomic across students.
 */
final class Registrar {
    private final Map<String, Student> students = new ConcurrentHashMap<>();
    private final Map<String, Section> sections = new ConcurrentHashMap<>();
    private final Map<String, Set<Section>> schedules = new ConcurrentHashMap<>();
    private final RegistrationWindow window;
    private final List<RegistrationRule> fullChecks;
    private final List<RegistrationRule> promotionChecks;
    private final Consumer<String> notifier;

    Registrar(RegistrationWindow window, Consumer<String> notifier) {
        this.window = window;
        this.notifier = notifier;
        this.fullChecks = List.of(new WindowRule(), new PrerequisiteRule(), new DuplicateCourseRule(),
                new CreditLimitRule(), new TimeConflictRule());
        this.promotionChecks = List.of(new DuplicateCourseRule(), new CreditLimitRule(), new TimeConflictRule());
    }

    void addStudent(Student s) { students.put(s.id(), s); schedules.put(s.id(), ConcurrentHashMap.newKeySet()); }
    void addSection(Section s) { sections.put(s.id(), s); }
    Section section(String id) { return sections.get(id); }

    List<Section> search(String text) {
        var q = text.toLowerCase();
        return sections.values().stream().filter(s -> s.course().code().toLowerCase().contains(q)
                || s.course().title().toLowerCase().contains(q)).sorted((a, b) ->
a.id().compareTo(b.id())).toList();
    }

    Set<Section> schedule(String studentId) { return Set.copyOf(schedules.get(studentId)); }

    private Optional<String> firstViolation(List<RegistrationRule> rules, RegistrationContext ctx) {
        for (var r : rules) {
            var v = r.violation(ctx);
            if (v.isPresent()) return v;
        }
        return Optional.empty();
    }

    RegistrationResult register(String studentId, String sectionId, LocalDateTime now) {
        var student = students.get(studentId);
        var section = sections.get(sectionId);
        if (student == null || section == null) return RegistrationResult.rejected("unknown student or section");
        var schedule = schedules.get(studentId);
        synchronized (schedule) {                                   // 1st lock: the student
            var ctx = new RegistrationContext(student, section, Set.copyOf(schedule), now, window);
            var v = firstViolation(fullChecks, ctx);
            if (v.isPresent()) return RegistrationResult.rejected(v.get());
            var result = section.enroll(studentId);                 // 2nd lock: the section
            if (result.outcome() == Outcome.ENROLLED) schedule.add(section);
            return result;
        }
```

```
    }

    void drop(String studentId, String sectionId) {
        var section = sections.get(sectionId);
        var schedule = schedules.get(studentId);
        Optional<String> promoted;
        synchronized (schedule) {
            promoted = section.drop(studentId);
            schedule.remove(section);
        }
        notifier.accept(studentId + " dropped " + sectionId);
        // Finish promotions OUTSIDE the dropper's lock, one student lock at a time (no nested student locks).
        while (promoted.isPresent()) promoted = finishPromotion(promoted.get(), section);
    }

    /** The promoted student's timetable may have changed while waiting, so re-check. Returns the next promotion,
if any. */
    private Optional<String> finishPromotion(String studentId, Section section) {
        var schedule = schedules.get(studentId);
        synchronized (schedule) {
            var ctx = new RegistrationContext(students.get(studentId), section, Set.copyOf(schedule), null,
window);
            var v = firstViolation(promotionChecks, ctx);
            if (v.isEmpty()) {
                schedule.add(section);
                notifier.accept(studentId + " promoted from waitlist into " + section.id());
                return Optional.empty();
            }
            notifier.accept(studentId + " skipped for " + section.id() + ": " + v.get());
            return section.drop(studentId);
        }
    }
}
```

```java
// File: code/lld/ch16/courseregistration/Main.java
package lld.ch16.courseregistration;

import java.time.DayOfWeek;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.EnumSet;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) throws Exception {
        var window = new RegistrationWindow(Map.of(
                4, LocalDateTime.of(2026, 11, 2, 9, 0),     // seniors first
                3, LocalDateTime.of(2026, 11, 3, 9, 0),
                2, LocalDateTime.of(2026, 11, 4, 9, 0),
                1, LocalDateTime.of(2026, 11, 5, 9, 0)),
                LocalDateTime.of(2026, 11, 20, 17, 0));
        var reg = new Registrar(window, msg -> System.out.println("    [notify] " + msg));

        var cs101 = new Course("CS101", "Intro to Programming", 4, Set.of());
        var cs201 = new Course("CS201", "Data Structures", 4, Set.of("CS101"));
        var math201 = new Course("MATH201", "Discrete Math", 3, Set.of());
        var cs301 = new Course("CS301", "Algorithms", 4, Set.of("CS201", "MATH201"));
        var drLee = new Instructor("I1", "Dr. Lee");
        var mw = EnumSet.of(DayOfWeek.MONDAY, DayOfWeek.WEDNESDAY);
        var tt = EnumSet.of(DayOfWeek.TUESDAY, DayOfWeek.THURSDAY);
        reg.addSection(new Section("CS201-A", cs201, drLee, new TimeSlot(mw, LocalTime.of(10, 0), LocalTime.of(11,
30)), 2, 2));
        reg.addSection(new Section("CS201-B", cs201, drLee, new TimeSlot(tt, LocalTime.of(14, 0), LocalTime.of(15,
30)), 30, 5));
        reg.addSection(new Section("MATH201-A", math201, new Instructor("I2", "Dr. Rao"), new TimeSlot(mw,
LocalTime.of(11, 0), LocalTime.of(12, 0)), 30, 5));
        reg.addSection(new Section("CS301-A", cs301, drLee, new TimeSlot(tt, LocalTime.of(9, 0), LocalTime.of(10,
30)), 30, 5));
        reg.addSection(new Section("CS101-A", cs101, new Instructor("I3", "Dr. Kim"), new TimeSlot(tt,
LocalTime.of(11, 0), LocalTime.of(12, 0)), 30, 10));

        reg.addStudent(new Student("ana", "Ana", 4, Set.of("CS101"), 12));
        reg.addStudent(new Student("ben", "Ben", 3, Set.of("CS101"), 4));
        reg.addStudent(new Student("cy", "Cy", 2, Set.of(), 16));
        reg.addStudent(new Student("dee", "Dee", 3, Set.of("CS101"), 12));
        reg.addStudent(new Student("eve", "Eve", 3, Set.of("CS101"), 12));
        reg.addStudent(new Student("fay", "Fay", 3, Set.of("CS101"), 12));

        var nov2 = LocalDateTime.of(2026, 11, 2, 10, 0);
        var nov3 = LocalDateTime.of(2026, 11, 3, 10, 0);
        System.out.println("1) Search 'cs2': " + reg.search("cs2").stream().map(s -> s.id() + " " +
s.slot()).toList());

        System.out.println("2) Rules");
        show("ben  CS201-A on Nov 2 ", reg.register("ben", "CS201-A", nov2));
        show("ana  CS201-A on Nov 2 ", reg.register("ana", "CS201-A", nov2));
        show("ana  MATH201-A        ", reg.register("ana", "MATH201-A", nov2));
        show("ana  CS201-B          ", reg.register("ana", "CS201-B", nov2));
        show("ana  CS301-A          ", reg.register("ana", "CS301-A", nov2));
        show("cy   CS201-B on Nov 4 ", reg.register("cy", "CS201-B", LocalDateTime.of(2026, 11, 4, 9, 30)));
        show("ana  after close      ", reg.register("ana", "CS201-B", LocalDateTime.of(2026, 11, 21, 9, 0)));

        System.out.println("3) Capacity 2 + waitlist 2 on CS201-A (juniors open Nov 3)");
        show("ben  CS201-A          ", reg.register("ben", "CS201-A", nov3));
        show("dee  CS201-A          ", reg.register("dee", "CS201-A", nov3));
        show("eve  CS201-A          ", reg.register("eve", "CS201-A", nov3));
        show("fay  CS201-A          ", reg.register("fay", "CS201-A", nov3));
        show("ben  MATH201-A        ", reg.register("ben", "MATH201-A", nov3));
```

```
        System.out.println("4) Ana drops CS201-A: the head of the waitlist is promoted");
        reg.drop("ana", "CS201-A");
        var a = reg.section("CS201-A");
        System.out.println("   roster " + a.roster() + ", waitlist " + a.waitlist());

        System.out.println("5) 50 first-years grab 30 seats (+10 waitlist) in CS101-A at 09:00 sharp");
        var opening = LocalDateTime.of(2026, 11, 5, 9, 0);
        for (int i = 0; i < 50; i++) reg.addStudent(new Student("fy" + i, "Fresher" + i, 1, Set.of(), 16));
        var counts = new ConcurrentHashMap<Outcome, Integer>();
        var go = new CountDownLatch(1);
        try (var pool = Executors.newFixedThreadPool(16)) {
            for (int i = 0; i < 50; i++) {
                String id = "fy" + i;
                pool.submit(() -> {
                    go.await();
                    counts.merge(reg.register(id, "CS101-A", opening).outcome(), 1, Integer::sum);
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("   ENROLLED=" + counts.get(Outcome.ENROLLED) + " WAITLISTED=" +
counts.get(Outcome.WAITLISTED)
                + " REJECTED=" + counts.get(Outcome.REJECTED) + ", roster size " + reg.section("CS101-
A").roster().size());

        System.out.println("6) One student double-clicks: two threads, same request");
        reg.addStudent(new Student("gus", "Gus", 3, Set.of("CS101"), 12));
        var outcomes = new ConcurrentHashMap<Outcome, Integer>();
        try (var pool = Executors.newFixedThreadPool(2)) {
            for (int i = 0; i < 2; i++)
                pool.submit(() -> outcomes.merge(reg.register("gus", "CS201-B", nov3).outcome(), 1, Integer::sum));
        }
        System.out.println("   " + new java.util.TreeMap<>(outcomes) + ", gus schedule " +
reg.schedule("gus").stream().map(Section::id).toList());
    }

    static void show(String label, RegistrationResult r) {
        System.out.println("   " + label + " -> " + r.outcome() + ": " + r.message());
    }
}
```

Output:

```
1) Search 'cs2': [CS201-A [MONDAY, WEDNESDAY] 10:00-11:30, CS201-B [TUESDAY, THURSDAY] 14:00-15:30]
2) Rules
   ben  CS201-A on Nov 2  -> REJECTED: registration window is closed for year 3 at 2026-11-02T10:00
   ana  CS201-A on Nov 2  -> ENROLLED: enrolled in CS201-A (1/2)
   ana  MATH201-A         -> REJECTED: time conflict with CS201-A [MONDAY, WEDNESDAY] 10:00-11:30
   ana  CS201-B           -> REJECTED: already registered for CS201 in section CS201-A
   ana  CS301-A           -> REJECTED: missing prerequisites [CS201, MATH201]
   cy   CS201-B on Nov 4  -> REJECTED: missing prerequisites [CS101]
   ana  after close       -> REJECTED: registration window is closed for year 4 at 2026-11-21T09:00
3) Capacity 2 + waitlist 2 on CS201-A (juniors open Nov 3)
   ben  CS201-A           -> ENROLLED: enrolled in CS201-A (2/2)
   dee  CS201-A           -> WAITLISTED: waitlisted for CS201-A at position 1
   eve  CS201-A           -> WAITLISTED: waitlisted for CS201-A at position 2
   fay  CS201-A           -> REJECTED: CS201-A and its waitlist are full
   ben  MATH201-A         -> REJECTED: credit limit 4 exceeded (7)
4) Ana drops CS201-A: the head of the waitlist is promoted
    [notify] ana dropped CS201-A
    [notify] dee promoted from waitlist into CS201-A
   roster [ben, dee], waitlist [eve]
5) 50 first-years grab 30 seats (+10 waitlist) in CS101-A at 09:00 sharp
   ENROLLED=30 WAITLISTED=10 REJECTED=10, roster size 30
6) One student double-clicks: two threads, same request
   {ENROLLED=1, REJECTED=1}, gus schedule [CS201-B]
```

Step 2 shows each rule firing with a clear reason. Step 3 fills two seats and two waitlist places. Step 4 promotes Dee automatically. Step 5 is the "09:00 rush": 50 threads, 30 seats, 10 waitlist places — the counts are exact every run. Step 6 is the double-click: two threads for the same student and section, exactly one succeeds.

Concurrency & edge cases

- **Overselling a section** — prevented by the section lock around "size < capacity → add".
- **Double-click / two tabs** — prevented by the per-student lock; the second request sees the first enrolment and is rejected by `DuplicateCourseRule` (or by the section's "already enrolled" check).
- **Deadlock** — avoided by the fixed lock order (student → section) and by never holding two student locks at once.
- **Waitlisted in one section, enrolled in another section of the same course** — possible while waiting, because waitlist entries are not in the schedule; the promotion re-check (`DuplicateCourseRule`) catches it and skips to the next student.
- **Registration closes while on the waitlist** — promotion does not re-check the window (it is the drop that triggers it). Many universities stop automatic promotion after a deadline; add `WindowRule` to `promotionChecks` if that is the policy.
- **Dropping a section you are not in** → `IllegalArgumentException`.
- **Rules with time** — the window uses half-open time (`opens ≤ now < closes`).
- **Scaling beyond one JVM** (Java Virtual Machine — the running program that executes Java code) — the in-memory locks work for one process (one running copy of the program). With several app servers (copies of the application on different machines, usually behind a load balancer that spreads requests among them), the seat count moves to the database: `UPDATE section SET enrolled = enrolled + 1 WHERE id = ? AND enrolled < capacity` succeeds for exactly `capacity` requests (an atomic conditional update), or use optimistic locking with a version column. A queue in front of hot sections smooths the 09:00 spike. (Load balancers, queues and databases at scale are covered in System Design Fundamentals.)

Extensions & follow-up questions

- **"Co-requisites or 'prerequisite OR instructor permission'?"** — Model prerequisites as a small expression tree (a tree whose inner nodes are operators like `And` / `Or` and whose leaves are simple checks: `And`, `Or`, `CourseCompleted`, `Permission`) — the Composite pattern, where a group of objects is treated exactly like a single one — and evaluate it in `PrerequisiteRule`.
- **"Holds (unpaid fees, advising)?"** — A `HoldRule` that rejects if the student has an active hold.
- **"Priority waitlists (seniors first)?"** — Replace the `Deque` with a `PriorityQueue` (a queue that always hands out the smallest item by some ordering, not the oldest) ordered by (year, time joined).

- **"Seat reservations for majors?"** — Split capacity into buckets (majors 20, others 10) and check the bucket in `Section.enroll`.
- **"Linked sections (lecture + lab)?"** — Register both atomically: lock both sections in a fixed order (by id), check both, enrol both.
- **"Notify by email/SMS?"** — The registrar takes a `Consumer<String>` notifier (Java's built-in "function that takes a value and returns nothing"); swap for an email service or publish an event to a message queue (a service such as Kafka or RabbitMQ that stores messages until another program reads them).
- **"Course fees and billing?"** — Add a `Bill` per student per term, a line per enrolled section, recomputed on drop.

In the interview

30-second pitch: "Students register for sections, not courses. A section owns capacity and a bounded FIFO waitlist, and takes seats only inside its own lock. The registrar runs a chain of rule objects — window, prerequisites, duplicate course, credit limit, time conflict — while holding the student's lock, then asks the section for a seat. Lock order is always student then section, so no deadlocks. On drop the head of the waitlist is promoted after a re-check."

Minutes	Do
0–5	Clarify course vs section, rules, waitlist, windows, concurrency.
5–12	Entities and class diagram.
12–30	Code `Section.enroll/drop`, the rule interface and two rules, `Registrar.register`.
30–40	Concurrency: run the 50-threads demo in your head; lock order; DB conditional update for scale.

Self-check

1. **Q:** Why do students register for sections, not courses? **A:** Capacity, time and instructor belong to each offering; a course can have many sections.
2. **Q:** Which two locks does `register` take and in what order? **A:** The student's schedule lock, then the section's lock — always in that order.
3. **Q:** Why not simply make `register` `synchronized` on the registrar? **A:** It would be correct but serialise (force to run strictly one at a time) every student in the university; per-section and per-student locks let unrelated requests run in parallel.
4. **Q:** Why re-check rules on promotion? **A:** The student's timetable may have changed while waiting (a clash or the same course in another section).
5. **Q:** How do you enforce capacity with many servers? **A:** An atomic conditional update in the database (`… WHERE enrolled < capacity`) or optimistic locking.

TL;DR

- Course (catalogue) vs Section (offering with capacity, slot, waitlist).
- Rules are small objects run as a chain; add a rule without touching the registrar's logic.
- Seats are taken only inside `Section.enroll` under the section's lock; student lock first, section lock second.
- Drop → promote waitlist head → re-check → notify.
- At scale: conditional update in the database.

Real-world example. University student-information systems (for example Ellucian Banner, used by many US universities) run priority registration windows by class standing, check prerequisites and time conflicts, and keep section waitlists that notify a student when a seat opens (some schools enrol the student automatically; others give a time-limited window to claim the seat). Registration-day slowdowns at such schools are the 09:00 "thundering herd" (a huge crowd of requests all arriving at the same instant) this design addresses with per-section locking and, at scale, database conditional updates.

Chapter summary

Problem	Key insight	Atomic claim	Patterns
Car rental	Availability is a question about **dates**, not a status field	`Vehicle.tryBook(range)`	Template Method (add-ons), Strategy + Decorator (pricing), state machine
Restaurant	**Meal per seat**; table = status (now) + slots (later)	`Table.tryHold(time)`	Observer (order ready), Strategy (chef assignment, payment), state machine
Airline	**Flight ≠ FlightInstance, Seat ≠ FlightSeat**	`FlightInstance.holdSeats(map)` (all or nothing)	Strategy + Decorator (fares), Facade, index for cancellation
Course registration	**Course ≠ Section**; rules as objects	`Section.enroll` under a section lock, inside a student lock	Specification / Chain of rules, lock ordering

The four problems share one lesson: **find the calendar, find the atomic check-then-claim, and put the rules on the object that owns the data.**

Chapter 17 — Case Studies: Games & Rules

> **Where this fits.** Chapters 14–16 designed things that model the real world (an ATM, a parking lot, booking systems). Games are different: the "world" is a set of **rules**, and the rules are exact. That makes games a favourite interview topic. There is nowhere to hide a vague class, because every rule must live *somewhere*, and the interviewer will ask where.

An **LLD** (low-level design) interview asks you to design the classes, their responsibilities and some of the code for one system — as opposed to high-level design, which is about servers, databases and networks.

What games test in an LLD interview

Skill	What it means	Where you see it in this chapter
Rule encapsulation (hiding a rule inside the class that owns it)	Each rule lives in one obvious class, not spread across `if` statements everywhere.	Blackjack scoring in `BlackjackScoring`; chess legality in `Rules`; 2048 merging in `LineMerger`.
Turn management	Whose turn is it, what may they do now, and what happens when the turn ends?	A turn index (tic-tac-toe), a queue of players (snake and ladder), `turn` colour (chess), rounds (blackjack).
Polymorphic pieces	Different kinds of objects answer the same question in their own way (*polymorphism* = "many shapes": one method call, many behaviours).	Chess `Piece.targets(...)` overridden by King, Knight, Pawn and the sliding pieces.
Pluggable strategies	Behaviour that varies (how a bot plays, how dice roll, when the game ends) sits behind an interface (a list of methods with no fixed implementation; the **Strategy** pattern, Chapter 10).	`PlayerStrategy`, `DealerPolicy`, `MoveStrategy`, `Dice`, `OverflowRule`, `WinStrategy`, `TileSpawner`.
Deterministic testing (same input, same result every time)	Randomness is *injected* (handed in from outside). Pass `new Random(seed)` and the "random" sequence is the same on every run, so tests and demos repeat exactly. Or pass a scripted source (stacked shoe, loaded dice).	Every `Main` in this chapter prints the same output every time.
State and status	A game moves through states (in progress, won, draw, over). Illegal actions in the wrong state are rejected.	`GameStatus`, `GameState` enums (types with a fixed list of named values) in every game.

> **Key idea: seeded randomness.** `java.util.Random` is a *pseudo-random* generator: it computes its numbers from a starting value called the **seed**. The same seed always produces the same sequence (the algorithm is fixed by the Java specification). `Collections.shuffle(list, rnd)` is also deterministic for a given `Random`. So the rule for game code is: **never call `new Random()` or `Math.random()` inside the game. Accept a `Random` (or a `Dice` / `Spawner` interface) from outside.** Production passes an unseeded (or `SecureRandom` — a slower generator whose output cannot be predicted, used for security) source; tests pass a seed. This is plain Dependency Injection (Chapter 5): an object receives what it needs from its caller instead of creating it itself.

Problems in this chapter

#	Problem	Main ideas
17.1	Blackjack and a Deck of Cards	generic deck, shoe, soft/hard hand values, dealer rule, betting, split, double, insurance
17.2	Chess	board, polymorphic pieces, legal moves, check/checkmate/stalemate, castling, en passant, promotion, undo
17.3	Tic-Tac-Toe	N×N board, O(1) win detection with counters, pluggable players
17.4	Snake and Ladder	board config and validation, seeded dice, rules engine, pluggable win rules
17.5	2048	slide/merge logic, seeded tile spawns, game-over detection

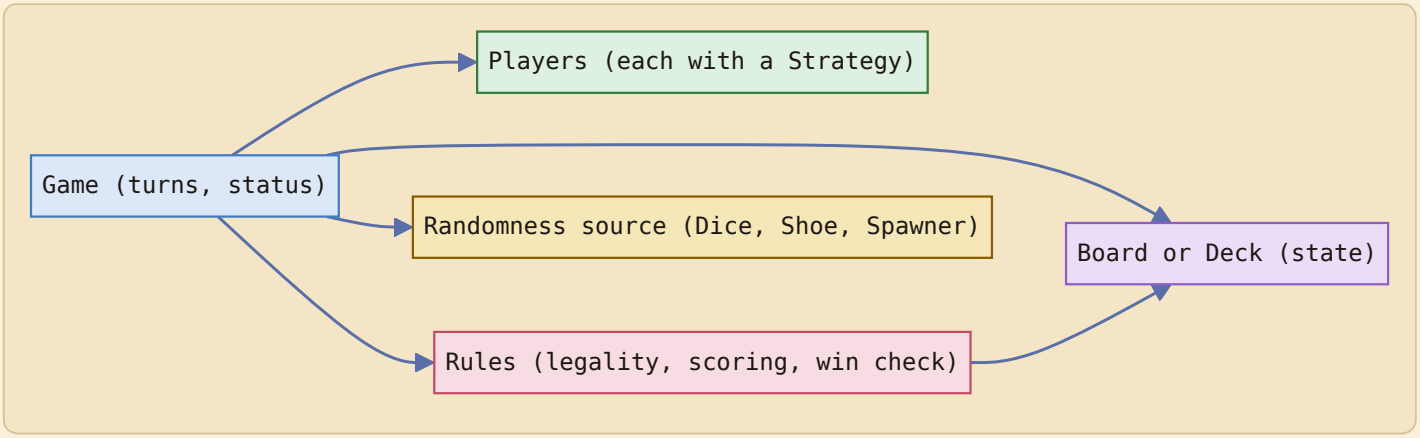
> **Key term — Big-O notation:** Expressions like O(1), O(n) and O(N²) describe how an algorithm's work grows as the input grows. **O(1)** ("constant time") means the work stays the same no matter how big the input is; **O(n)** means it grows in step with the input size n; **O(N²)** means doubling N makes the work about four times bigger. Interviewers use it to compare approaches without timing them.

How to run every example in this chapter

```
cd chapters/17-case-games/code
javac --release 21 -d /tmp/lld_17 $(find . -name '*.java')
java -cp /tmp/lld_17 lld.ch17.blackjack.Main    # or chess, tictactoe, snakeladder, game2048
```

`javac` is the Java compiler and `-cp` (the classpath) tells `java` where the compiled classes are. All five `Main` classes compile with Java 21 (standard library only — just the classes that ship with Java) and run. All randomness is seeded or scripted, so the output shown under each program is exactly what you will see.

A reusable skeleton for any board/card game. Almost every game design in an interview has the same five parts. Name them early and the rest of the interview becomes filling in details.



Part	Question it answers	Typical class names
State	What does the table look like right now?	`Board`, `Deck`, `Shoe`, `Hand`, `Cell`
Actors	Who acts, and how do they decide?	`Player` + `PlayerStrategy` / `MoveStrategy`
Rules	Is this action allowed? What does it do? Who won?	`Rules`, `DealerPolicy`, `OverflowRule`, `WinStrategy`
Controller	Whose turn? What state is the game in?	`Game`, `GameStatus`
Randomness	Where does chance come from?	`Dice`, `Shoe`, `TileSpawner` — always injected

Design Blackjack and a Deck of Cards

Figure — A blackjack table: players each compete against the dealer (course diagram)

Problem & clarifying questions

Blackjack is the casino card game played more than any other. Nobody plays against the other people at the table: every player's cards are measured only against the cards of the **dealer** (the dealer is the casino employee who deals and plays for the house). Everybody tries to get as close to **21 points** as possible without going over. Going over 21 is called **busting** and loses at once.

The interviewer often phrases it as "Design a deck of cards, then use it to build blackjack". The first half checks that you keep the deck **generic**; the second half checks rule placement.

Questions worth asking (with the answers this design assumes):

Question	Assumed answer
One deck or several?	A **shoe** holding several standard 52-card decks (casinos use 6–8). One deck is just a shoe of 1.
How many players?	One or more players, each against the dealer.
Which player options?	Hit, stand, double down, split (once), insurance. Surrender and side bets are out of scope.
Dealer rule?	Hit on 16 or less, stand on 17 or more. Some casinos also hit a "soft 17" — make it pluggable.
Payouts?	Win 1:1, blackjack 3:2, tie ("push") returns the bet, insurance 2:1 (odds like 3:2 mean you win 3 chips for every 2 bet).
Money?	Bets are counted in whole **chips** (an `int`, a whole-number type), and the ante (the starting bet) must be even. Never use `double` (an inexact binary floating-point number) for money.
Real UI (user interface — the screens), network, accounts?	Out of scope. The course model has `Account`/`Person` for sign-in; we keep `Player` minimal and say where it would attach.

Requirements

Background rules

- A standard deck has **13 ranks** (A, 2–10, J, Q, K) in **4 suits** (clubs, diamonds, hearts, spades). Blackjack uses one or more such decks.
- At the start, each player and the dealer receive **two cards**. Both player cards are face up. The dealer has one card face up (the **up card**) and one face down (the **hole card**), so players decide with incomplete information.
- **Card points:** 2–10 are worth their number. J, Q, K are worth 10. An **ace is worth 1 or 11**, whichever helps.
- **Soft vs hard total:** when an ace is counted as 11 the total is **soft** (it can still "soften" to 1 later, so one more card can never bust it). When every ace counts as 1 the total is **hard**. Example: A+5 is "soft 16" (or hard 6).
- **Blackjack** (also called a *natural*): exactly two cards totalling 21, i.e. an ace plus a 10-point card.
- **Dealer rule:** the dealer has no choices. Hit (take a card) while the hand is 16 or less; stand at 17 or more.

Functional requirements

1. A player must place an initial bet (the **ante**) to play a round.
2. Deal two cards to each player and to the dealer (one up, one hole).
3. If the dealer's up card is an **ace**, offer **insurance**: a side bet of half the ante that pays **2:1** if the dealer has blackjack. After that, the dealer checks ("peeks at") the hole card. If it is blackjack, the round ends at once: insurance wins, main bets lose, except a player blackjack, which ties.
4. A player whose hand is below 21 chooses:
   - **Hit** — take one more card, and choose again.
   - **Stand** ("stand pat") — take no more cards.
   - **Double down** — double the bet (the extra must equal the ante), take exactly **one** more card, and stop.
   - **Split** — only when the first two cards have the same rank: pay a second bet equal to the ante, split the pair into two hands, deal one card to each, and play each hand separately. No re-splitting.
5. A hand over 21 is **resolved immediately as a loss**.
6. The dealer then plays by the fixed rule.
7. **Resolution:** if the dealer busts, every non-bust hand wins. Otherwise the higher total wins; equal totals are a **push** (bet returned). A winning blackjack pays **3:2**; every other win pays **1:1**.
8. **Even money** (course requirement): if a player holds blackjack and the dealer shows an ace, the player may accept an immediate 1:1 payout instead of risking a push. (It is mathematically identical to buying insurance with a blackjack, so we explain it rather than build it — see Extensions.)

Non-functional requirements

- **Generic deck:** the `Card` / `Deck` classes must work for any card game, not just blackjack.
- **Deterministic tests:** shuffling uses an injected `Random`; a *stacked* shoe (fixed card order) makes exact scenarios reproducible.
- **Extensible rules:** dealer rule and player decision-making are pluggable.
- **Correct money:** bets and balances are whole chips held in an `int`, never a `double`. This is the one place in the handbook where money is not a `BigDecimal`, and it is deliberate: a table only accepts whole chips, and requiring an **even ante** makes every payout a whole number of chips (3:2 on 10 is 15; insurance is exactly half the ante, 5, and pays 2:1 = 10). If the table ever had to pay fractions (a 3:2 payout on an odd ante, or converting chips to currency at the cashier), switch the amounts to `BigDecimal` (Java's exact decimal number type) with an explicit `RoundingMode` (the rule for what to do with a leftover fraction, such as "round half up").

Use cases

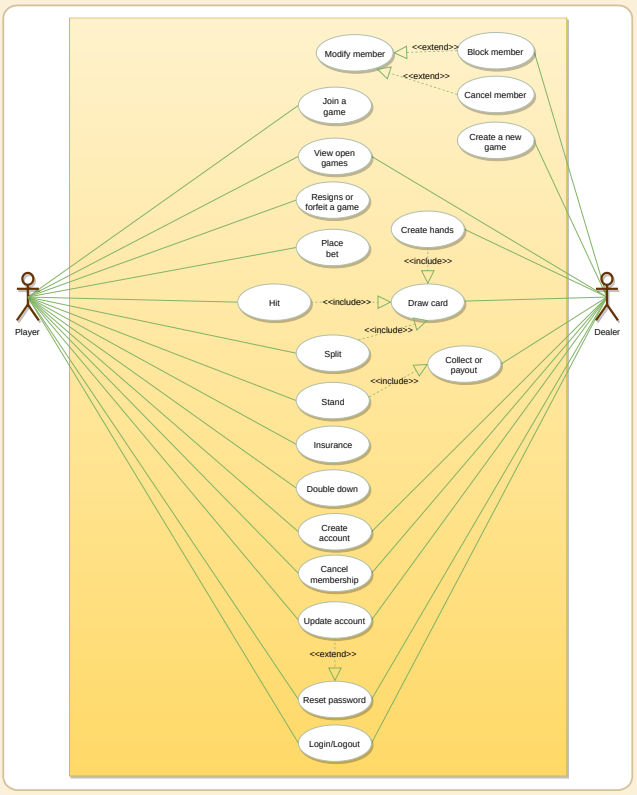


Figure — Blackjack use cases for the Player and the Dealer, including account management (course diagram)

The figure above is a **use case diagram** (UML — Unified Modeling Language — notation: stick figures are *actors*, the people or systems using the software; ovals are *use cases*, the goals they can achieve). In the figure above there are two actors:

- **Player** — places the initial bet; accepts or declines extra bets (insurance, split, double); accepts or declines an offered resolution such as even money; chooses hit, double or stand. The player also has account-level use cases (create account, log in, join or view games), which belong to a lobby service (the separate part of the app where players sign in and pick a table), not the game engine (the code that enforces the game's rules).
- **Dealer** — deals cards, plays the house hand by the fixed rule, collects losing bets and pays winners.

The main use cases (note the `<<include>>` arrows in the figure — an include arrow means one use case always runs another as part of itself: dealing and payout are part of every round):

Use case	What happens
Create hands	Deal two cards each; dealer shows one card.
Place bet	The ante; required before cards are dealt.
Player plays the hand	Hit (repeat), double (one card, done), or stand. A hand over 21 is resolved as a bust.
Dealer plays the hand	Draw while 16 or less, stop at 17 or more. A dealer bust pays every live hand.
Insurance / even money / examine hole card	Only when the up card is an ace. The hole card is checked; a 10-value hole card means dealer blackjack, insurance wins and the round ends; otherwise insurance loses and the hole card stays hidden.
Split	Pair of equal rank: second bet, two hands, one extra card each, no further splitting.
Game resolution	Compare totals; tie returns the bet; blackjack 3:2; other wins 1:1.

Activity: hit or stand

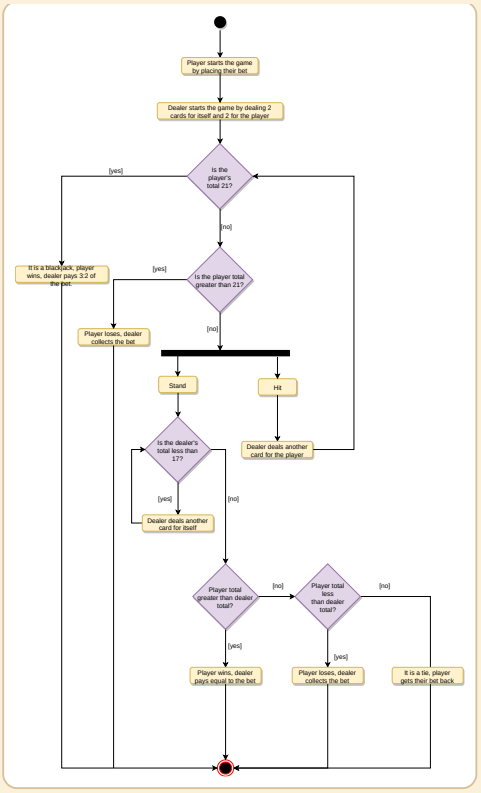


Figure — Activity diagram for one player's hit-or-stand round against the dealer (course diagram)

An **activity diagram** is a flowchart of one task, with diamonds for decisions. The figure above walks through one round: bet, deal, check for 21 (blackjack pays 3:2), check for bust, hit or stand, then the dealer draws while below 17, and finally the three outcomes — player wins 1:1, player loses, or a tie returns the bet.

Core entities

Class	Responsibility
`Suit`, `Rank` (enums)	The fixed values of a French deck (the standard 52-card deck with four suits). `Rank` stores the *face value* 1–13 (ace = 1, J = 11, Q = 12, K = 13).
`Card` (record — a compact class that just holds fixed values)	Rank + suit. Knows **nothing** about blackjack. Immutable (cannot change after creation).
`Deck<T>`	A generic (works for any type `T` you plug in), shuffleable stack of any card type `T`; `standard52()` factory (a method that builds objects) builds a French deck.
`Shoe`	Several decks shuffled together; deals one card at a time; reshuffles at the **cut card** (a marker card placed near the end of the shoe). `Shoe.stacked(...)` gives a fixed order for tests.
`BlackjackScoring`	The blackjack value of a rank (K = 10, ace = 1 hard). The *game* rule, kept off `Card`.
`Hand`	Cards + bet; computes hard/soft/best totals, bust, blackjack, can-split; splits and doubles.
`Action` (enum)	HIT, STAND, DOUBLE, SPLIT.
`PlayerStrategy` (interface)	How a player decides. `BasicStrategy` and `MimicDealerStrategy` are two bots (computer-controlled players); a UI-backed human would be a third.
`DealerPolicy` (interface)	When the dealer hits. `standOnAll17()` and `hitSoft17()`.
`Player`	Name, chip balance, strategy.
The dealer	Not a class of its own here: during a round the dealer is a `Hand` inside `BlackjackGame` plus the `DealerPolicy` that decides when it draws. (The course has a `Dealer` class next to `Player`, both under `BasePlayer`.)
`BlackjackGame`	Runs a round: bets, deal, insurance + peek, player turns, dealer turn, settlement.

Class diagram

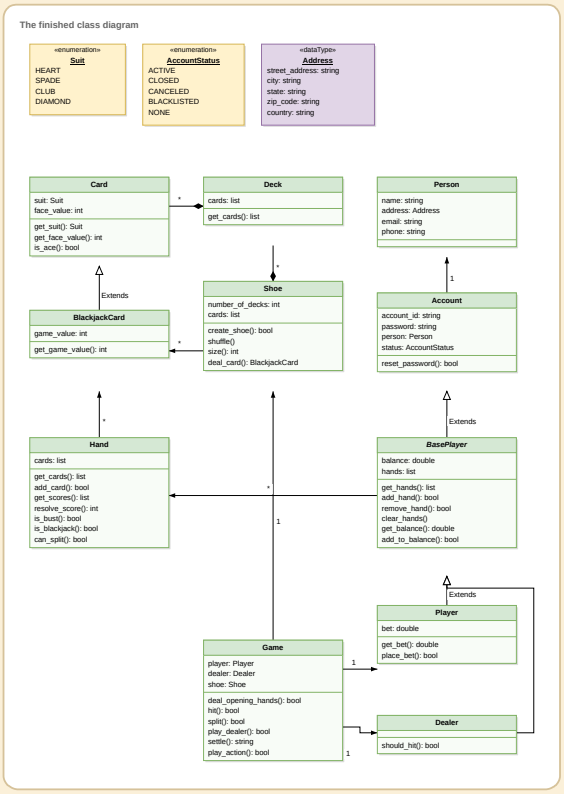
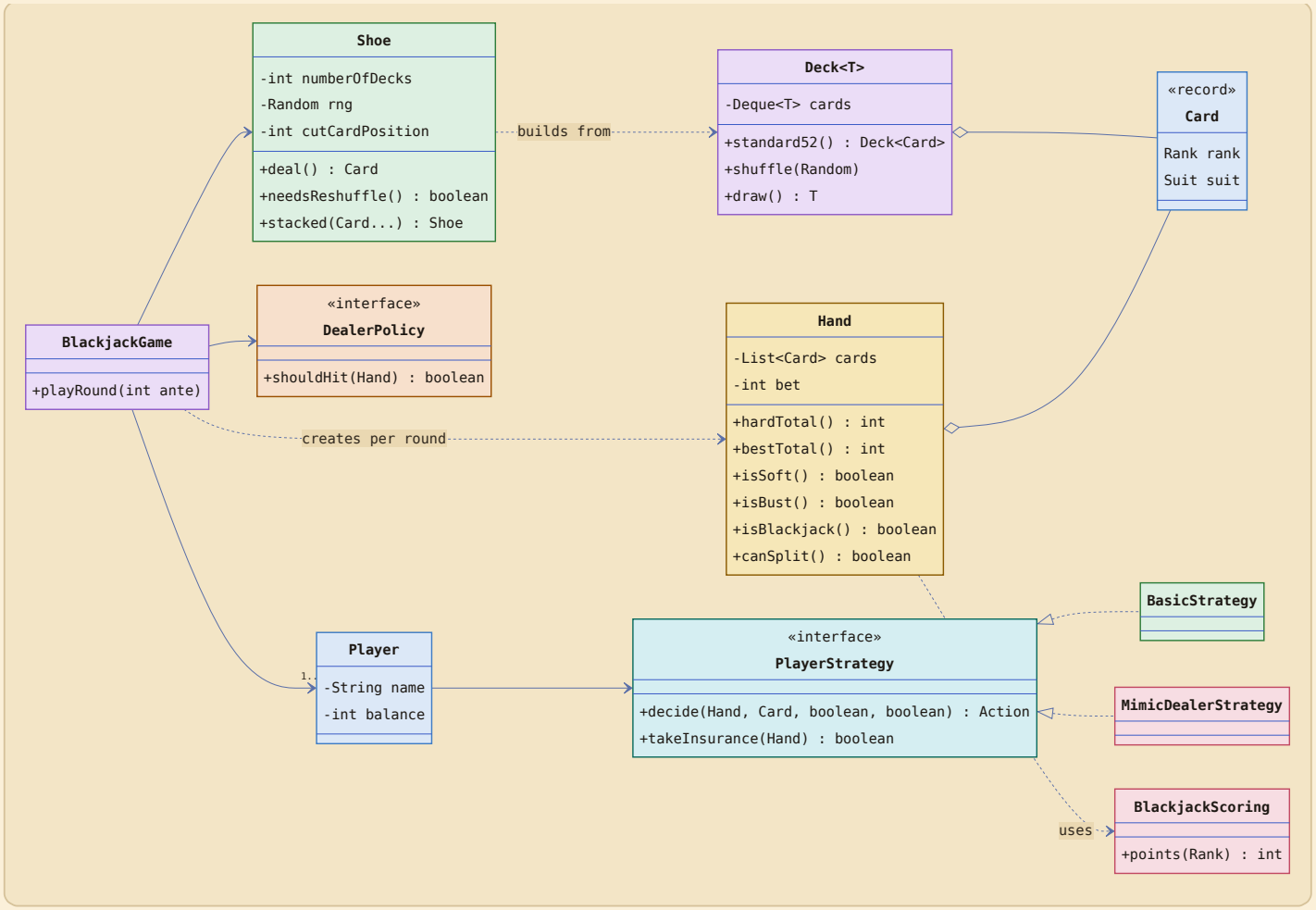


Figure — Course class diagram: Card, BlackjackCard, Deck, Shoe, Hand, BasePlayer, Player, Dealer, Game, Account (course diagram)

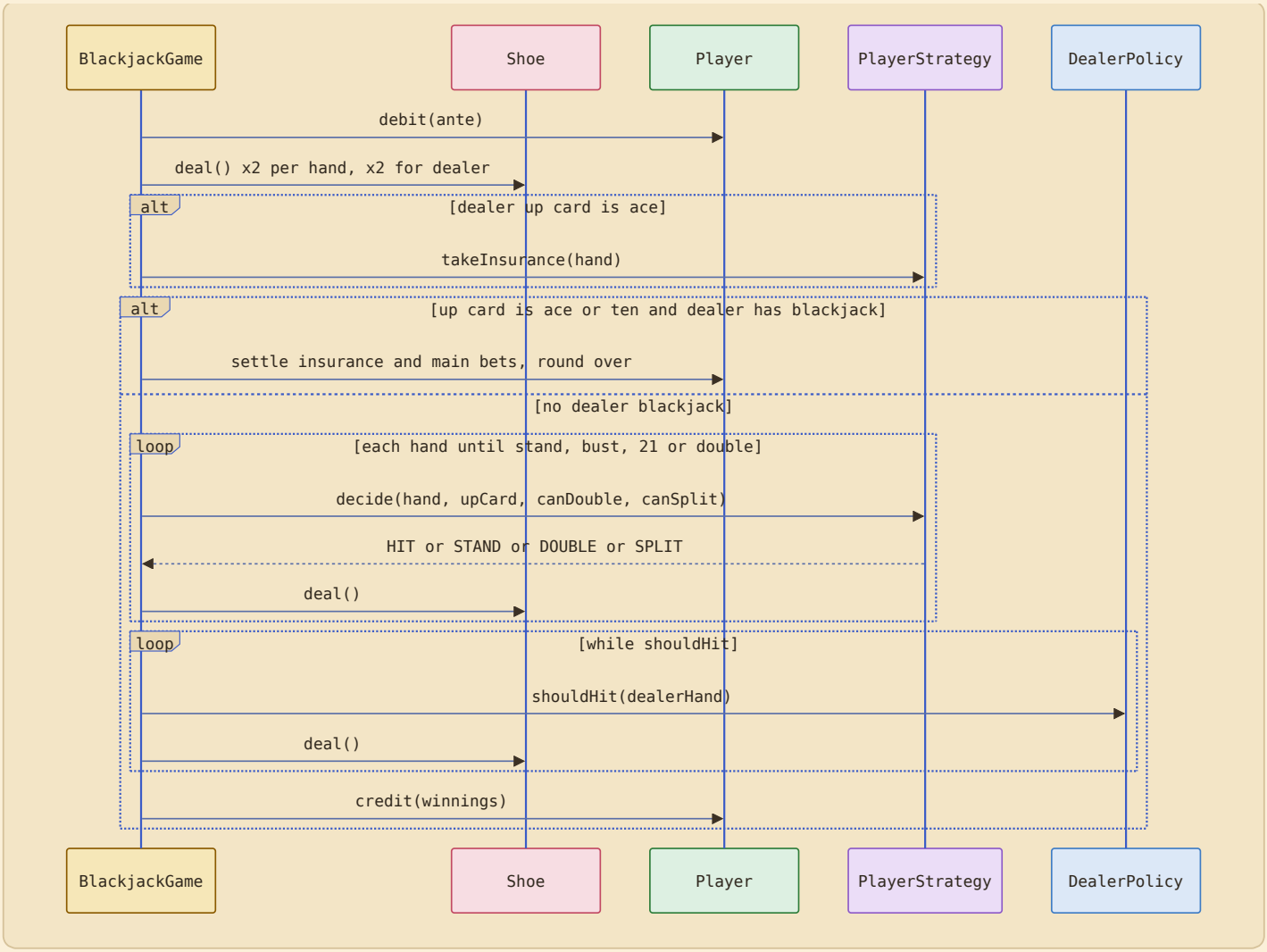
In the course model (figure above), `Card` holds a suit and a face value, and a subclass `BlackjackCard` adds the *game value* — because a king has face value 13 but is worth 10 in blackjack. A `Shoe` is composed of several `Deck`s (filled diamonds = composition: the parts belong to the whole). A plain arrow is an *association* (one class merely refers to another), and an open triangle is *inheritance* (a subclass extends a parent class). `Hand` scores itself. `Player` and `Dealer` share a `BasePlayer` (balance, hands), which extends `Account`. `Player` adds the bet; `Dealer` adds the "should I hit?" rule. `Game` points at a player, a dealer and a shoe and holds the order of play. Note the `*` from `BasePlayer` to `Hand`: splitting gives one player several hands.

Our Java design keeps the same core idea — **the game value must not live on the generic card** — but implements it with a separate scoring function instead of a subclass, and turns the two things that vary (player decisions, dealer rule) into strategies:

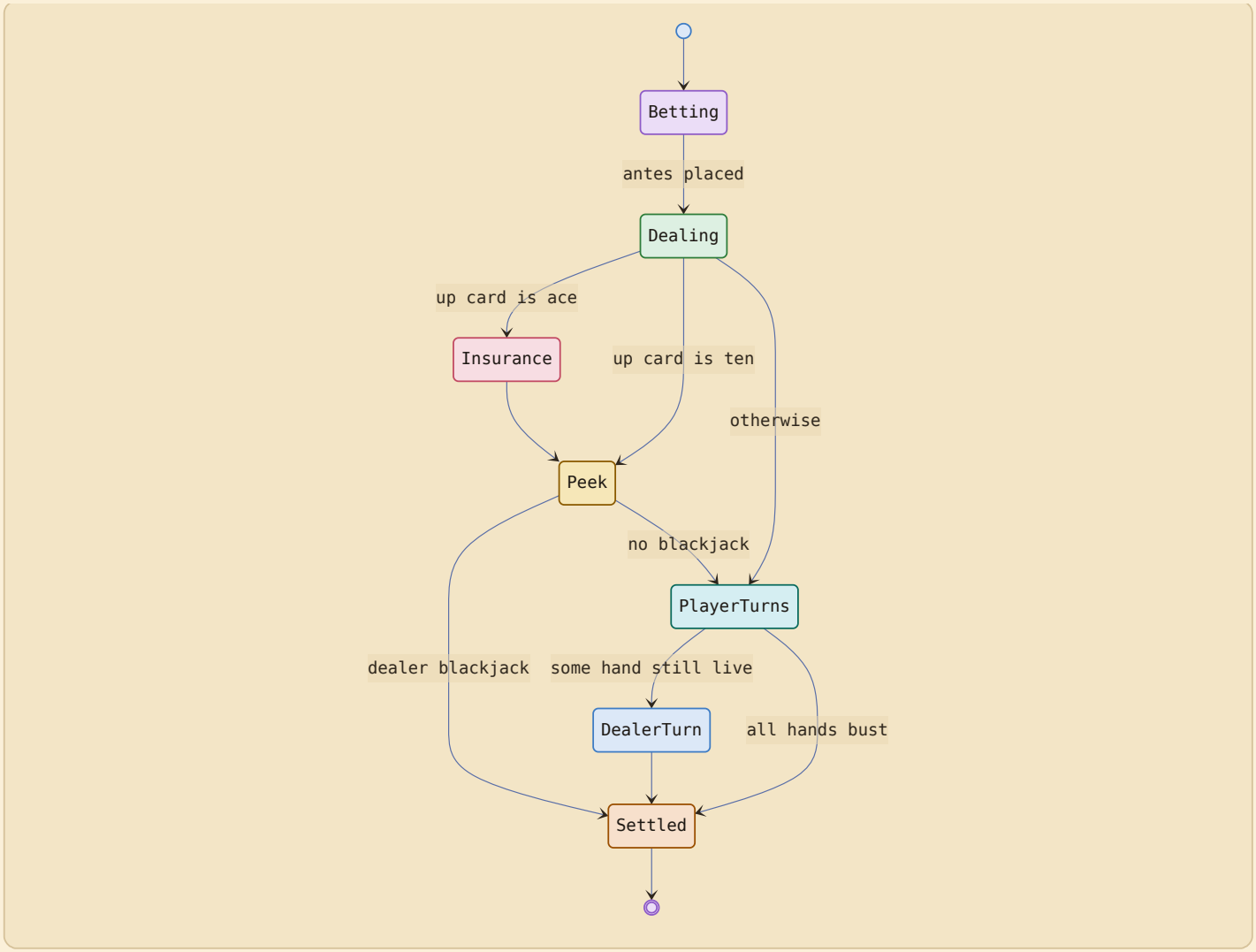


Key flows

One round (sequence)



States of one round



Scoring a hand (the ace problem). The trick is that **at most one ace can ever count as 11** (two elevens would be 22, a bust). So:

1. Add up every card with the ace as 1. This is the **hard total**.
2. If the hand has at least one ace and `hard + 10 ≤ 21`, one ace can be 11: the hand is **soft** and the best total is `hard + 10`.
3. Otherwise the best total is the hard total.

This is O(n) in the number of cards, needs no search over combinations, and gives the soft/hard flag for free (the dealer's "hit soft 17" rule needs it).

Design decisions & patterns used

Decision	Why	Alternative rejected
`Card` is a game-agnostic **record** (rank + suit).	A deck of cards is reused by poker, bridge, solitaire. A king's blackjack value (10) is a *rule of this game*, not a fact about the card.	Putting `getValue()` returning 10 for K on `Card` — breaks every other game.
Blackjack value in `BlackjackScoring`, not a `BlackjackCard` subclass.	Records cannot be extended, and the value depends only on the rank. A function is simpler and keeps one card type in the shoe.	The course's `BlackjackCard extends Card` — also valid, and also keeps `Card` clean. Say both in the interview; the important point is the same.
`Deck<T>` is **generic**.	"Design a deck of cards" follow-up: Uno or tarot cards reuse the same shuffle/draw code.	A deck hard-wired to `Card`.
Strategy for players (`PlayerStrategy`) and dealer (`DealerPolicy`).	Human UI, bots and test scripts plug in; casino variants (S17 vs H17: dealer stands on soft 17 vs hits soft 17) are one line. Open/Closed principle (add new behaviour with new classes, without editing working code).	`if (isHuman) ... else ...` and `if (casino == X)` branches inside the game.
Factory methods `Deck.standard52()`, `Shoe.stacked(...)`, `DealerPolicy.standOnAll17()`.	Named constructors make intent obvious.	Big constructors with flags.
Randomness injected (`Random` passed into `Shoe`).	Seeded shuffles make demos and tests exact.	`Collections.shuffle(list)` with a hidden random source.
`Hand` answers with `hardTotal()`, `bestTotal()` and `isSoft()`.	The course's `Hand` returns *every* possible total (A+6 is both 7 and 17) and lets the caller choose. Because at most one ace can ever count as 11, the hard total plus the soft flag already carries all of that information, and most callers just want `bestTotal()`.	Returning a list of totals that each caller must filter.
Hand stores its **own bet**.	Split and double change the bet *per hand*, not per player.	One bet per player — breaks as soon as a player splits.
The game **validates** a strategy's choice (`canDouble`, `canSplit` passed in, and re-checked).	A buggy or malicious client (the program sending the player's choices) cannot double without money.	Trusting the client.

Honest weaknesses (and what a longer design would do). `BlackjackGame.playRound` owns the whole sequence of play *and* the payout rules. Each new option (surrender, side bets) adds another branch. A larger design would model the round as a **State** machine (a fixed set of states plus the allowed moves between them; the State pattern gives each state its own class — one class per state in the diagram above, like the vending machine in Chapter 14) and move payouts into a `PayoutTable`. Say this out loud; interviewers reward knowing where your design will strain.

Java implementation

Package `lld.ch17.blackjack`.

```java
// File: code/lld/ch17/blackjack/Suit.java
package lld.ch17.blackjack;

/** The four suits. A suit never affects a blackjack score, but a generic deck needs it. */
public enum Suit {
    CLUBS('C'), DIAMONDS('D'), HEARTS('H'), SPADES('S');

    private final char letter;

    Suit(char letter) { this.letter = letter; }

    public char letter() { return letter; }
}
```

```java
// File: code/lld/ch17/blackjack/Rank.java
package lld.ch17.blackjack;

/** Thirteen ranks with their face value (ace = 1, jack = 11, queen = 12, king = 13). */
public enum Rank {
    ACE("A", 1), TWO("2", 2), THREE("3", 3), FOUR("4", 4), FIVE("5", 5), SIX("6", 6),
    SEVEN("7", 7), EIGHT("8", 8), NINE("9", 9), TEN("10", 10),
    JACK("J", 11), QUEEN("Q", 12), KING("K", 13);

    private final String symbol;
    private final int faceValue;

    Rank(String symbol, int faceValue) { this.symbol = symbol; this.faceValue = faceValue; }

    public String symbol() { return symbol; }
    public int faceValue() { return faceValue; }
}
```

```java
// File: code/lld/ch17/blackjack/Card.java
package lld.ch17.blackjack;

/** A game-agnostic playing card: just a rank and a suit. Immutable, so it is safe to share. */
public record Card(Rank rank, Suit suit) {
    @Override public String toString() { return rank.symbol() + suit.letter(); }
}
```

```java
// File: code/lld/ch17/blackjack/Deck.java
package lld.ch17.blackjack;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.Deque;
import java.util.List;
import java.util.NoSuchElementException;
import java.util.Random;

/**
 * A generic deck: any card type T (poker card, Uno card, tarot card...).
 * Shuffling takes a Random so that tests can pass a seeded one and get the same order every run.
 */
public final class Deck<T> {
    private final Deque<T> cards;

    public Deck(Collection<? extends T> cards) { this.cards = new ArrayDeque<>(cards); }

    /** Factory for the standard 52-card French deck (13 ranks x 4 suits). */
    public static Deck<Card> standard52() {
        List<Card> all = new ArrayList<>(52);
        for (Suit s : Suit.values()) {
            for (Rank r : Rank.values()) {
                all.add(new Card(r, s));
            }
        }
        return new Deck<>(all);
    }

    public void shuffle(Random rng) {
        List<T> list = new ArrayList<>(cards);
        Collections.shuffle(list, rng);
        cards.clear();
        cards.addAll(list);
    }

    public T draw() {
        if (cards.isEmpty()) throw new NoSuchElementException("deck is empty");
        return cards.pop();
    }

    public int size() { return cards.size(); }

    public List<T> cards() { return List.copyOf(cards); }
}
```

```java
// File: code/lld/ch17/blackjack/Shoe.java
package lld.ch17.blackjack;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.List;
import java.util.Random;

/**
 * A casino shoe: several standard decks shuffled together. A "cut card" marks how deep we deal
 * before reshuffling (penetration). A stacked shoe (fixed order) exists for deterministic tests.
 */
public final class Shoe {
    private final int numberOfDecks;
    private final Random rng;          // null for a stacked test shoe
    private final int cutCardPosition; // reshuffle when fewer cards than this remain
    private final Deque<Card> cards = new ArrayDeque<>();

    public Shoe(int numberOfDecks, Random rng, double penetration) {
        if (numberOfDecks < 1) throw new IllegalArgumentException("need at least one deck");
        if (penetration <= 0 || penetration >= 1) throw new IllegalArgumentException("penetration must be in
(0,1)");
        this.numberOfDecks = numberOfDecks;
        this.rng = rng;
        this.cutCardPosition = (int) Math.round(numberOfDecks * 52 * (1 - penetration));
        refill();
    }

    private Shoe(List<Card> stackedOrder) {
        this.numberOfDecks = 0;
        this.rng = null;
        this.cutCardPosition = 0;
        cards.addAll(stackedOrder);
    }

    /** Cards come out in exactly the order given. Perfect for unit tests. */
    public static Shoe stacked(Card... order) { return new Shoe(List.of(order)); }

    public Card deal() {
        if (cards.isEmpty()) {
            if (rng == null) throw new IllegalStateException("stacked shoe ran out of cards");
            refill();
        }
        return cards.pop();
    }

    public boolean needsReshuffle() { return rng != null && cards.size() < cutCardPosition; }

    public void reshuffle() { refill(); }

    public int remaining() { return cards.size(); }

    private void refill() {
        List<Card> all = new ArrayList<>(numberOfDecks * 52);
        for (int i = 0; i < numberOfDecks; i++) {
            all.addAll(Deck.standard52().cards());
        }
        Collections.shuffle(all, rng);
        cards.clear();
        cards.addAll(all);
    }
}
```

```java
// File: code/lld/ch17/blackjack/BlackjackScoring.java
package lld.ch17.blackjack;

/**
 * The blackjack-specific value of a card lives here, NOT on Card.
 * A king has face value 13 in general, but it is worth 10 points in this game.
 */
public final class BlackjackScoring {
    private BlackjackScoring() {}

    /** Points with an ace counted as 1 (the "hard" value). */
    public static int points(Rank rank) {
        return switch (rank) {
            case ACE -> 1;
            case JACK, QUEEN, KING -> 10;
            default -> rank.faceValue();
        };
    }
}
```

```java
// File: code/lld/ch17/blackjack/Hand.java
package lld.ch17.blackjack;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/** A blackjack hand: its cards, its bet, and how to score itself (aces = 1 or 11). */
public final class Hand {
    private final List<Card> cards = new ArrayList<>();
    private int bet;
    private boolean fromSplit;
    private boolean doubled;

    public Hand(int bet) { this.bet = bet; }

    public static Hand of(Card... cs) {
        Hand h = new Hand(0);
        for (Card c : cs) h.add(c);
        return h;
    }

    void add(Card c) { cards.add(c); }

    /** Every ace counted as 1. */
    public int hardTotal() {
        int sum = 0;
        for (Card c : cards) sum += BlackjackScoring.points(c.rank());
        return sum;
    }

    private boolean hasAce() { return cards.stream().anyMatch(c -> c.rank() == Rank.ACE); }

    /** A hand is soft when one ace can count as 11 without busting. At most one ace can ever do so. */
    public boolean isSoft() { return hasAce() && hardTotal() + 10 <= 21; }

    /** The best total: the soft total if it is legal, otherwise the hard total. */
    public int bestTotal() { return isSoft() ? hardTotal() + 10 : hardTotal(); }

    public boolean isBust() { return bestTotal() > 21; }

    /** A natural: exactly two cards worth 21, and not the result of a split. */
    public boolean isBlackjack() { return !fromSplit && cards.size() == 2 && bestTotal() == 21; }

    /** Two cards of the same rank, and no re-splitting (one split per round). */
    public boolean canSplit() {
        return !fromSplit && cards.size() == 2 && cards.get(0).rank() == cards.get(1).rank();
    }

    /** Takes the second card out to start a new hand; both hands are now split hands. */
    Hand split() {
        if (!canSplit()) throw new IllegalStateException("hand cannot be split");
        Hand other = new Hand(bet);
        other.add(cards.remove(1));
        this.fromSplit = true;
        other.fromSplit = true;
        return other;
    }

    void doubleDown() { bet *= 2; doubled = true; }

    public boolean isDoubled() { return doubled; }
    public int bet() { return bet; }
    public int size() { return cards.size(); }
    public Card first() { return cards.get(0); }
    public List<Card> cards() { return List.copyOf(cards); }

    @Override public String toString() {
```

```
        String cs = cards.stream().map(Card::toString).collect(Collectors.joining(" "));
        String kind = isBlackjack() ? "BLACKJACK" : isBust() ? "bust " + bestTotal()
                : (isSoft() ? "soft " : "") + bestTotal();
        return "[" + cs + "] = " + kind;
    }
}
```

```
// File: code/lld/ch17/blackjack/Action.java
package lld.ch17.blackjack;

public enum Action { HIT, STAND, DOUBLE, SPLIT }
```

```
// File: code/lld/ch17/blackjack/PlayerStrategy.java
package lld.ch17.blackjack;

/**
 * Strategy pattern: HOW a player decides is pluggable (human via UI, basic-strategy bot, test script).
 * The game passes what is allowed, so a strategy never has to know the betting rules.
 */
public interface PlayerStrategy {
    Action decide(Hand hand, Card dealerUpCard, boolean canDouble, boolean canSplit);

    /** Offered only when the dealer shows an ace. Mathematically a bad bet for most players. */
    default boolean takeInsurance(Hand hand) { return false; }
}
```

```
// File: code/lld/ch17/blackjack/BasicStrategy.java
package lld.ch17.blackjack;

/** A simplified version of the well-known "basic strategy" table. */
public final class BasicStrategy implements PlayerStrategy {
    @Override
    public Action decide(Hand hand, Card up, boolean canDouble, boolean canSplit) {
        int dealer = BlackjackScoring.points(up.rank()) == 1 ? 11 : BlackjackScoring.points(up.rank());
        if (canSplit && (hand.first().rank() == Rank.ACE || hand.first().rank() == Rank.EIGHT)) {
            return Action.SPLIT;                                    // always split aces and eights
        }
        int total = hand.bestTotal();
        if (hand.isSoft()) {
            return total >= 18 ? Action.STAND : Action.HIT;
        }
        if (canDouble && (total == 11 || (total == 10 && dealer <= 9))) {
            return Action.DOUBLE;
        }
        if (total >= 17) return Action.STAND;
        if (total >= 13) return dealer <= 6 ? Action.STAND : Action.HIT;
        if (total == 12) return (dealer >= 4 && dealer <= 6) ? Action.STAND : Action.HIT;
        return Action.HIT;
    }
}
```

```
// File: code/lld/ch17/blackjack/MimicDealerStrategy.java
package lld.ch17.blackjack;

/** A naive player who copies the dealer (hit below 17) and always buys insurance. */
public final class MimicDealerStrategy implements PlayerStrategy {
    @Override
    public Action decide(Hand hand, Card up, boolean canDouble, boolean canSplit) {
        return hand.bestTotal() < 17 ? Action.HIT : Action.STAND;
    }

    @Override public boolean takeInsurance(Hand hand) { return true; }
}
```

```java
// File: code/lld/ch17/blackjack/DealerPolicy.java
package lld.ch17.blackjack;

/** The house rule for when the dealer draws. Casinos differ, so it is a pluggable strategy. */
@FunctionalInterface
public interface DealerPolicy {
    boolean shouldHit(Hand dealerHand);

    /** Classic rule: draw on 16 or less, stand on every 17 ("S17"). */
    static DealerPolicy standOnAll17() { return h -> h.bestTotal() <= 16; }

    /** Variant: also draw on a soft 17 such as A+6 ("H17"), which favours the house. */
    static DealerPolicy hitSoft17() { return h -> h.bestTotal() <= 16 || (h.bestTotal() == 17 && h.isSoft()); }
}
```

```java
// File: code/lld/ch17/blackjack/Player.java
package lld.ch17.blackjack;

/** A seated player: a name, a chip balance (whole chips) and a decision strategy. */
public final class Player {
    private final String name;
    private final PlayerStrategy strategy;
    private int balance;

    public Player(String name, int balance, PlayerStrategy strategy) {
        this.name = name;
        this.balance = balance;
        this.strategy = strategy;
    }

    void debit(int amount) {
        if (amount > balance) throw new IllegalStateException(name + " cannot cover " + amount);
        balance -= amount;
    }

    void credit(int amount) { balance += amount; }

    public boolean canAfford(int amount) { return balance >= amount; }
    public String name() { return name; }
    public int balance() { return balance; }
    public PlayerStrategy strategy() { return strategy; }
}
```

```java
// File: code/lld/ch17/blackjack/BlackjackGame.java
package lld.ch17.blackjack;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Consumer;

/**
 * Runs rounds at one table: bets, deal, insurance + dealer peek, player turns (hit/stand/double/split),
 * dealer turn, settlement. Each player plays only against the dealer.
 */
public final class BlackjackGame {
    private final Shoe shoe;
    private final DealerPolicy dealerPolicy;
    private final List<Player> players;
    private final Consumer<String> log;

    public BlackjackGame(Shoe shoe, DealerPolicy dealerPolicy, List<Player> players, Consumer<String> log) {
        this.shoe = shoe;
        this.dealerPolicy = dealerPolicy;
        this.players = List.copyOf(players);
        this.log = log;
    }

    public void playRound(int ante) {
        if (ante <= 0 || ante % 2 != 0) throw new IllegalArgumentException("ante must be a positive even number");
        if (shoe.needsReshuffle()) {
            shoe.reshuffle();
            log.accept("  (cut card reached: shoe reshuffled)");
        }

        // 1. Bets: each player who can afford the ante gets one hand.
        Map<Player, List<Hand>> hands = new LinkedHashMap<>();
        for (Player p : players) {
            if (!p.canAfford(ante)) { log.accept("  " + p.name() + " sits out (balance " + p.balance() + ")");
continue; }
            p.debit(ante);
            hands.put(p, new ArrayList<>(List.of(new Hand(ante))));
        }
        if (hands.isEmpty()) return;

        // 2. Deal two rounds: one card to each player, then one to the dealer. Dealer's 2nd card is the hole card.
        Hand dealer = new Hand(0);
        for (int round = 0; round < 2; round++) {
            for (List<Hand> hs : hands.values()) hs.get(0).add(shoe.deal());
            dealer.add(shoe.deal());
        }
        Card up = dealer.first();
        log.accept("  Dealer shows " + up + " (hole card hidden)");
        hands.forEach((p, hs) -> log.accept("  " + p.name() + " " + hs.get(0)));

        // 3. Insurance (only when the up card is an ace), then the dealer peeks for blackjack.
        Map<Player, Integer> insurance = new LinkedHashMap<>();
        if (up.rank() == Rank.ACE) {
            hands.forEach((p, hs) -> {
                int side = ante / 2;
                if (p.strategy().takeInsurance(hs.get(0)) && p.canAfford(side)) {
                    p.debit(side);
                    insurance.put(p, side);
                    log.accept("  " + p.name() + " buys insurance for " + side);
                }
            });
        }
        boolean dealerMayHaveBlackjack = up.rank() == Rank.ACE || BlackjackScoring.points(up.rank()) == 10;
        if (dealerMayHaveBlackjack && dealer.isBlackjack()) {
            log.accept("  Dealer peeks: " + dealer + " -> round over");
```

```
            insurance.forEach((p, side) -> { p.credit(side * 3); log.accept("  " + p.name() + " insurance pays 2:1
(+" + side * 2 + ")"); });
            hands.forEach((p, hs) -> {
                Hand h = hs.get(0);
                if (h.isBlackjack()) { p.credit(h.bet()); log.accept("  " + p.name() + ": PUSH (both blackjack)");
}
                else log.accept("  " + p.name() + ": LOSE " + h.bet());
            });
            printBalances();
            return;
        }
        if (!insurance.isEmpty()) log.accept("  Dealer peeks: no blackjack, insurance bets lost");

        // 4. Player turns. Splitting inserts a new hand right after the current one.
        hands.forEach((p, hs) -> {
            for (int i = 0; i < hs.size(); i++) playHand(p, hs, i, up);
        });

        // 5. Dealer turn (skipped if every player hand is already bust).
        boolean anyLive = hands.values().stream().flatMap(List::stream).anyMatch(h -> !h.isBust());
        if (anyLive) {
            while (dealerPolicy.shouldHit(dealer)) dealer.add(shoe.deal());
            log.accept("  Dealer reveals and plays: " + dealer);
        }

        // 6. Settle every hand against the dealer.
        hands.forEach((p, hs) -> hs.forEach(h -> settle(p, h, dealer)));
        printBalances();
    }

    private void playHand(Player p, List<Hand> hs, int index, Card up) {
        Hand hand = hs.get(index);
        String label = p.name() + (hs.size() > 1 ? " hand " + (index + 1) : "");
        while (!hand.isBlackjack() && hand.bestTotal() < 21) {
            boolean canDouble = hand.size() == 2 && p.canAfford(hand.bet());
            boolean canSplit = hand.canSplit() && p.canAfford(hand.bet());
            Action a = p.strategy().decide(hand, up, canDouble, canSplit);
            if ((a == Action.DOUBLE && !canDouble) || (a == Action.SPLIT && !canSplit)) {
                throw new IllegalStateException(label + " chose a move that is not allowed: " + a);
            }
            switch (a) {
                case STAND -> { log.accept("  " + label + " stands on " + hand.bestTotal()); return; }
                case HIT -> { hand.add(shoe.deal()); log.accept("  " + label + " hits: " + hand); }
                case DOUBLE -> {
                    p.debit(hand.bet());
                    hand.doubleDown();
                    hand.add(shoe.deal());
                    log.accept("  " + label + " doubles (bet now " + hand.bet() + "): " + hand);
                    return;                                    // exactly one card after doubling
                }
                case SPLIT -> {
                    p.debit(hand.bet());
                    Hand second = hand.split();
                    hand.add(shoe.deal());
                    second.add(shoe.deal());
                    hs.add(index + 1, second);
                    label = p.name() + " hand " + (index + 1);
                    log.accept("  " + p.name() + " splits -> " + hand + " and " + second);
                }
            }
        }
        if (hand.isBust()) log.accept("  " + label + " busts");
    }

    private void settle(Player p, Hand h, Hand dealer) {
        String who = "  " + p.name() + " " + h + " vs dealer " + dealer.bestTotal() + ": ";
        if (h.isBust()) { log.accept(who + "LOSE " + h.bet()); return; }
        if (h.isBlackjack()) {                                 // dealer blackjack was handled by the peek
            p.credit(h.bet() + h.bet() * 3 / 2);
```

```
            log.accept(who + "BLACKJACK pays 3:2 (+" + h.bet() * 3 / 2 + ")");
            return;
        }
        int mine = h.bestTotal(), theirs = dealer.bestTotal();
        if (dealer.isBust() || mine > theirs) { p.credit(h.bet() * 2); log.accept(who + "WIN +" + h.bet()); }
        else if (mine == theirs) { p.credit(h.bet()); log.accept(who + "PUSH"); }
        else log.accept(who + "LOSE " + h.bet());
    }

    private void printBalances() {
        StringBuilder sb = new StringBuilder("  Balances:");
        players.forEach(p -> sb.append(' ').append(p.name()).append('=').append(p.balance()));
        log.accept(sb.toString());
    }
}
```

```
// File: code/lld/ch17/blackjack/Main.java
package lld.ch17.blackjack;

import java.util.List;
import java.util.Random;

public final class Main {
    private static Card c(Rank r, Suit s) { return new Card(r, s); }

    public static void main(String[] args) {
        System.out.println("== 1. Hand values (aces are 1 or 11) ==");
        List<Hand> samples = List.of(
                Hand.of(c(Rank.ACE, Suit.SPADES), c(Rank.SIX, Suit.HEARTS)),
                Hand.of(c(Rank.ACE, Suit.SPADES), c(Rank.SIX, Suit.HEARTS), c(Rank.TEN, Suit.CLUBS)),
                Hand.of(c(Rank.ACE, Suit.SPADES), c(Rank.ACE, Suit.HEARTS), c(Rank.NINE, Suit.CLUBS)),
                Hand.of(c(Rank.KING, Suit.SPADES), c(Rank.QUEEN, Suit.HEARTS), c(Rank.FIVE, Suit.CLUBS)),
                Hand.of(c(Rank.ACE, Suit.DIAMONDS), c(Rank.KING, Suit.CLUBS)));
        samples.forEach(h -> System.out.println("  " + h + "   (hard total " + h.hardTotal() + ")"));

        System.out.println("\n== 2. Stacked shoe: split 8s, then double both hands ==");
        Shoe stacked = Shoe.stacked(
                c(Rank.EIGHT, Suit.HEARTS), c(Rank.SIX, Suit.SPADES),     // Alice 1st card, dealer up card
                c(Rank.EIGHT, Suit.DIAMONDS), c(Rank.TEN, Suit.CLUBS),    // Alice 2nd card, dealer hole card
                c(Rank.THREE, Suit.CLUBS), c(Rank.TWO, Suit.SPADES),      // one card to each split hand
                c(Rank.NINE, Suit.HEARTS), c(Rank.FIVE, Suit.DIAMONDS),   // the two double-down cards
                c(Rank.SEVEN, Suit.SPADES));                              // dealer's hit on 16
        Player alice = new Player("Alice", 100, new BasicStrategy());
        new BlackjackGame(stacked, DealerPolicy.standOnAll17(), List.of(alice), System.out::println).playRound(10);

        System.out.println("\n== 3. Stacked shoe: dealer shows an ace, Bob buys insurance ==");
        Shoe aceUp = Shoe.stacked(c(Rank.TEN, Suit.HEARTS), c(Rank.ACE, Suit.SPADES),
                c(Rank.NINE, Suit.CLUBS), c(Rank.KING, Suit.DIAMONDS));
        Player bob = new Player("Bob", 100, new MimicDealerStrategy());
        new BlackjackGame(aceUp, DealerPolicy.standOnAll17(), List.of(bob), System.out::println).playRound(10);

        System.out.println("\n== 4. Seeded 6-deck shoe (seed 42), two players, 5 rounds ==");
        Shoe shoe = new Shoe(6, new Random(42), 0.75);
        Player carol = new Player("Carol", 100, new BasicStrategy());
        Player dave = new Player("Dave", 100, new MimicDealerStrategy());
        BlackjackGame game = new BlackjackGame(shoe, DealerPolicy.hitSoft17(), List.of(carol, dave),
System.out::println);
        for (int round = 1; round <= 5; round++) {
            System.out.println("Round " + round + ":");
            game.playRound(10);
        }
    }
}
```

Output:

```
== 1. Hand values (aces are 1 or 11) ==
  [AS 6H] = soft 17   (hard total 7)
  [AS 6H 10C] = 17   (hard total 17)
  [AS AH 9C] = soft 21   (hard total 11)
  [KS QH 5C] = bust 25   (hard total 25)
  [AD KC] = BLACKJACK   (hard total 11)

== 2. Stacked shoe: split 8s, then double both hands ==
  Dealer shows 6S (hole card hidden)
  Alice [8H 8D] = 16
  Alice splits -> [8H 3C] = 11 and [8D 2S] = 10
  Alice hand 1 doubles (bet now 20): [8H 3C 9H] = 20
  Alice hand 2 doubles (bet now 20): [8D 2S 5D] = 15
  Dealer reveals and plays: [6S 10C 7S] = bust 23
  Alice [8H 3C 9H] = 20 vs dealer 23: WIN +20
  Alice [8D 2S 5D] = 15 vs dealer 23: WIN +20
  Balances: Alice=140

== 3. Stacked shoe: dealer shows an ace, Bob buys insurance ==
  Dealer shows AS (hole card hidden)
  Bob [10H 9C] = 19
  Bob buys insurance for 5
  Dealer peeks: [AS KD] = BLACKJACK -> round over
  Bob insurance pays 2:1 (+10)
  Bob: LOSE 10
  Balances: Bob=100

== 4. Seeded 6-deck shoe (seed 42), two players, 5 rounds ==
Round 1:
  Dealer shows AS (hole card hidden)
  Carol [8S 3S] = 11
  Dave [5D 9C] = 14
  Dave buys insurance for 5
  Dealer peeks: [AS QS] = BLACKJACK -> round over
  Dave insurance pays 2:1 (+10)
  Carol: LOSE 10
  Dave: LOSE 10
  Balances: Carol=90 Dave=100
Round 2:
  Dealer shows 6D (hole card hidden)
  Carol [2S 8H] = 10
  Dave [QS 10H] = 20
  Carol doubles (bet now 20): [2S 8H QD] = 20
  Dave stands on 20
  Dealer reveals and plays: [6D 6D 7C] = 19
  Carol [2S 8H QD] = 20 vs dealer 19: WIN +20
  Dave [QS 10H] = 20 vs dealer 19: WIN +10
  Balances: Carol=110 Dave=110
Round 3:
  Dealer shows 5C (hole card hidden)
  Carol [AD KD] = BLACKJACK
  Dave [8D AH] = soft 19
  Dave stands on 19
  Dealer reveals and plays: [5C 7D 7S] = 19
  Carol [AD KD] = BLACKJACK vs dealer 19: BLACKJACK pays 3:2 (+15)
  Dave [8D AH] = soft 19 vs dealer 19: PUSH
  Balances: Carol=125 Dave=110
Round 4:
  Dealer shows 10S (hole card hidden)
  Carol [8D 8S] = 16
  Dave [5C 9S] = 14
  Carol splits -> [8D 4S] = 12 and [8S JD] = 18
  Carol hand 1 hits: [8D 4S 7D] = 19
  Carol hand 1 stands on 19
  Carol hand 2 stands on 18
  Dave hits: [5C 9S KS] = bust 24
  Dave busts
  Dealer reveals and plays: [10S 9H] = 19
```

```
  Carol [8D 4S 7D] = 19 vs dealer 19: PUSH
  Carol [8S JD] = 18 vs dealer 19: LOSE 10
  Dave [5C 9S KS] = bust 24 vs dealer 19: LOSE 10
  Balances: Carol=115 Dave=100
Round 5:
  Dealer shows KD (hole card hidden)
  Carol [5S 8C] = 13
  Dave [8S 2C] = 10
  Carol hits: [5S 8C JS] = bust 23
  Carol busts
  Dave hits: [8S 2C 7C] = 17
  Dave stands on 17
  Dealer reveals and plays: [KD 10H] = 20
  Carol [5S 8C JS] = bust 23 vs dealer 20: LOSE 10
  Dave [8S 2C 7C] = 17 vs dealer 20: LOSE 10
  Balances: Carol=105 Dave=90
```

Reading the output. Part 1 shows the scoring: A+6 is soft 17, adding a 10 makes it a *hard* 17 (the ace dropped to 1), A+A+9 is soft 21. Part 2 uses a stacked shoe to force an exact scenario: Alice splits eights against a dealer 6, doubles both hands, and the dealer busts — she staked 40 chips and won 40. Part 3 shows insurance working as a hedge: Bob loses his main bet of 10 but the 5-chip insurance pays 10, so he breaks even. Part 4 is a seeded six-deck game: run it again and you get the same cards.

Concurrency & edge cases

- **One table = one thread.** A **thread** is one independent line of execution in a program; a server runs many side by side. A round is a strict sequence, so the simplest safe design runs each table's rounds on a single thread (or an actor/event loop — a single worker that processes incoming messages one at a time from its own queue) and funnels player commands into it. Many tables run in parallel; they share nothing except the wallet service.
- **Balances are shared across tables** in a real casino app. Move `debit`/`credit` into a wallet service with atomic operations — ones that happen as a single indivisible step no other thread can interrupt (a database transaction, i.e. a group of changes that all succeed or all fail, or `AtomicLong`/`synchronized` in memory; `synchronized` lets only one thread at a time run that code) and debit *before* dealing.
- **Odd antes and 3:2.** 3:2 on 5 chips is 7.5. We require even antes. Real casinos either round down or use smaller chip units — decide and state it.
- **Split aces.** Many casinos deal only one card to each split ace, and 21 after a split is *not* a blackjack (our `isBlackjack()` checks `fromSplit`).
- **Shoe exhaustion.** The cut card triggers a reshuffle *between* rounds; the shoe also refills if emptied mid-round. A stacked test shoe throws instead, so a wrong test fails loudly.
- **Player cannot afford a double/split.** The game passes `canDouble=false` and rejects the action if chosen anyway.
- **Dealer peek only on ace or ten.** If the up card is anything else the dealer cannot have blackjack, so there is no peek.

Extensions & follow-up questions

- **"Where would even money go?"** — It is the same as insurance for a player holding blackjack: insurance of half the ante plus a blackjack always nets exactly +1× the ante whatever the dealer has. Add `PlayerStrategy.takeEvenMoney(hand)` and, if accepted, settle that hand at 1:1 before the peek.
- **"Add surrender."** — New `Action.SURRENDER`, allowed only as the first decision; refund half the bet and end the hand.
- **"Several players at the table?"** — Already supported: the game iterates players in seat order, then the dealer plays once.
- **"Card counting detection / security?"** — Card counting is tracking which cards have gone to guess what is left. Shoe penetration (how deep into the shoe the cut card sits) already limits counting. Log every dealt card for audit (a later independent check of what happened); never send the hole card to clients until it is revealed.
- **"How would you test the dealer rule?"** — Stacked shoe + `DealerPolicy.hitSoft17()`: deal the dealer A+6 and assert it draws; with `standOnAll17()` assert it stands.
- **"Make the deck work for Uno."** — `Deck<UnoCard>` with its own factory; nothing else changes.

- **"Replace the round logic with a state machine."** — One class per state (Betting, Dealing, PlayerTurns, DealerTurn, Settled), each handling only the commands valid in that state; invalid commands are rejected by the current state.

Real-world example

Online casino tables (for example live-dealer blackjack products) separate exactly these concerns: a **shuffle/RNG service** that is certified and audited (gambling regulators, for example the UK Gambling Commission, require the random number generator to be independently tested), a **game engine** that enforces the table rules (number of decks, S17 or H17, payout 3:2 or 6:5 — printed on the physical table felt), and a **wallet** service that debits bets atomically. The rule variants are configuration (settings chosen at deploy time, not code), which is exactly why `DealerPolicy` is a strategy.

In the interview

30-second answer: "A generic `Card` (rank, suit) and a generic `Deck<T>`. A `Shoe` combines several decks and deals with an injected `Random`, plus a stacked constructor for tests. The blackjack value of a card lives in the game's scoring, not on `Card`. `Hand` scores itself: hard total with aces as 1, and it's soft if one ace can be 11 without busting. Player decisions and the dealer's hit rule are strategies. `BlackjackGame` runs the round: bets, deal, insurance and peek, player turns with hit/stand/double/split, dealer turn, settle 3:2, 1:1 or push."

How to spend 30–45 minutes: 5 min questions and rules → 5 min entities → 10 min class diagram (defend `Card` vs game value, hand-owned bet) → 15 min code `Hand` scoring and `playRound` → 5 min edge cases (split aces, insurance, money) and extensions.

Likely follow-ups with answers

- *"Why not an `int value` on `Card`?"* — Because value is game-specific; poker ranks aces high, bridge uses suits, blackjack makes K = 10.
- *"How do you compute the best hand value?"* — Hard total, then add 10 if there is an ace and it fits. At most one ace can be 11.
- *"How do you make the game testable?"* — Inject the randomness: seeded `Random` for repeatability, stacked shoe for exact scenarios.

Self-check

1. **Q:** Why does the `Hand` store the bet instead of the `Player`? **A:** After a split, one player has two hands with separate bets; after a double, only that hand's bet changes.
2. **Q:** A+A+9 — soft or hard, and what total? **A:** Hard 11 with aces as 1; one ace can be 11 → soft 21.
3. **Q:** When does the dealer peek? **A:** Only when the up card is an ace or a 10-value card — otherwise blackjack is impossible.
4. **Q:** What does insurance pay, and how much is it? **A:** 2:1. In this design (and the course) it is exactly half the ante; real casinos let you buy *up to* half.
5. **Q:** Is 21 after splitting a blackjack? **A:** No, a blackjack must be the original two-card hand.

TL;DR

- Keep `Card` / `Deck` generic; the game's value rule lives in the game (`BlackjackScoring`, or a `BlackjackCard` subclass in the course version).
- Hand value: hard total + 10 if an ace fits → soft. Blackjack = original two cards totalling 21.
- Strategies for player decisions and dealer rule; inject randomness; stacked shoe for tests.
- Round = bet → deal → (insurance + peek) → player turns → dealer turn → settle (3:2, 1:1, push).

Design Chess

Figure — A chess board in play (course diagram)

Problem & clarifying questions

Chess is a two-player strategy game on an 8×8 board of 64 squares. Each side starts with 16 pieces: **8 pawns, 2 rooks, 2 knights, 2 bishops, 1 queen, 1 king**. The goal is **checkmate**: attack the enemy king so that it has no legal escape. We design an **online** two-player chess game.

Question	Assumed answer
Full international rules?	Yes: every piece's moves, captures, check, checkmate, stalemate, castling, en passant, promotion.
How are sides chosen?	Randomly assigned white or black (`Game.withRandomSides`, with an injected `Random`); **white moves first**; players alternate.
Can players take moves back?	Not in a rated game (one that counts toward the players' official ratings; course requirement). We still support `undo()` for analysis mode and tests.
How can a game end?	Checkmate, stalemate (a draw), resignation, or forfeit.
Move log?	Yes — every move by both players is recorded.
Timers, rating, spectators, draw by repetition?	Out of scope; discussed in Extensions.
AI opponent?	Out of scope; a bot would be a `MoveStrategy` that picks from `legalMoves()`.

Requirements

Functional

1. Two players play one game; each is assigned a colour, white moves first, turns alternate.
2. A player moves one of **their own** pieces; the system rejects moves out of turn, moves a piece cannot make, and moves that would leave the mover's own king in check.
3. All international rules, including the special moves:
    - **Castling** — the king moves two squares toward a rook and the rook jumps over to the square the king crossed. Allowed only if neither piece has moved, the squares between are empty, the king is not in check, and the king does not pass through or land on an attacked square.

    - **En passant** ("in passing") — if a pawn advances two squares and lands beside an enemy pawn, that enemy pawn may capture it *as if it had moved only one square*, but only on the very next move.
    - **Promotion** — a pawn reaching the last rank becomes a queen, rook, bishop or knight (the player's choice).
4. Detect **check** (the king is attacked), **checkmate** (in check and no legal move — the attacker wins) and **stalemate** (not in check but no legal move — a draw).
5. Support **resignation** (the player gives up) and **forfeit** (the player loses without choosing to, for example by leaving or timing out); record the result.
6. Keep a **log** of every move.

Non-functional

- Adding a new piece type (for a chess variant) should not touch existing pieces (Open/Closed).
- Legality checks must be correct first; speed is a secondary concern for a two-human game.
- Online play: the game object must be safe when two player connections call it concurrently (at the same time, from different threads).

Use cases

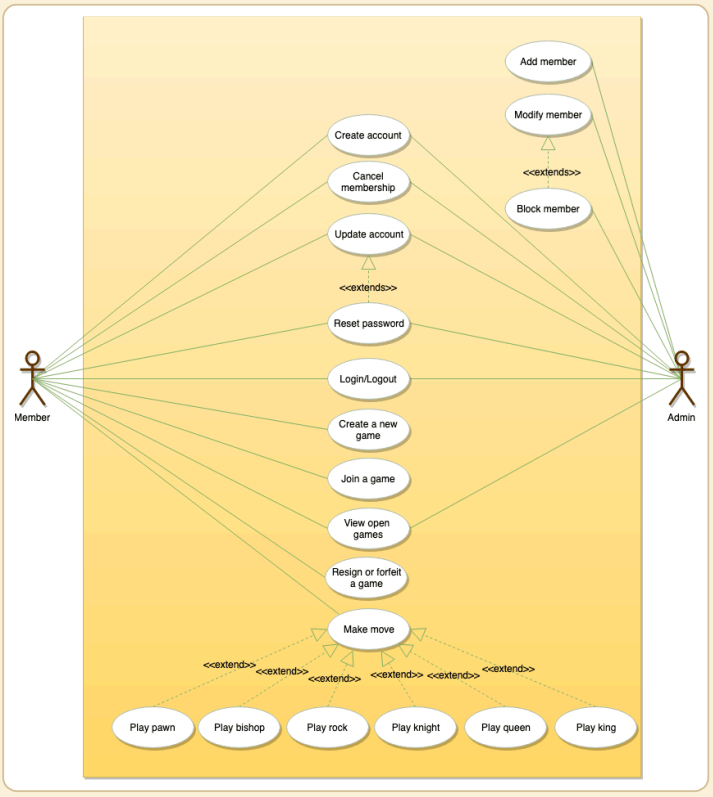


Figure — Chess use cases for Member (player) and Admin, with "Make move" extended by each piece type (course diagram)

In the figure above there are two actors:

- **Player** (a registered member) — creates or joins a game, views open games, **makes moves**, resigns or forfeits, and manages their account.
- **Admin** — adds, modifies, or blocks (bans) members.

The key use cases are **Player moves a piece** (extended per piece type — "Play pawn", "Play knight"... — which is exactly the polymorphism in our `Piece` hierarchy), **Resign or forfeit**, **Register/cancel membership**, and **Update game log** (every move is appended).

Activity: make a move

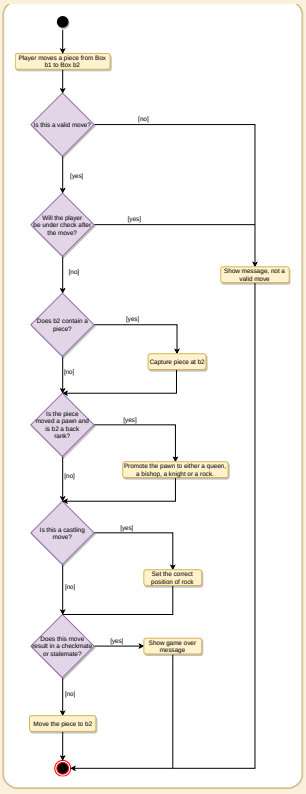


Figure — Activity diagram for making a move: validate, check self-check, capture, promotion, castling, and checkmate/stalemate (course diagram)

The figure above shows the full decision chain for one move. We follow the same order in `Game.move` and `Game.play`:

1. Is the move valid for that piece? If not, show "not a valid move".
2. Would the mover's king be in check afterwards? If yes, reject.
3. Does the destination hold a piece? Capture it.
4. Is it a pawn reaching the back rank? Promote it.
5. Is it castling? Move the rook as well.
6. Does the move produce checkmate or stalemate? Then the game is over.

Core entities

Class	Responsibility
`Color` (enum)	WHITE/BLACK and `opposite()`.
`Position` (record)	A square (file a–h, rank 1–8). Validates bounds; parses "e4". The course calls this a *Box*.
`Piece` (abstract — cannot be created directly, only through subclasses; sealed — only listed subclasses allowed)	Colour + `targets(board, from)`: where it can move by its own rules. `attacks(...)` for check detection.
`King`, `Knight`, `Pawn`	Step pieces and the pawn's special forward/diagonal rule.
`SlidingPiece` → `Rook`, `Bishop`, `Queen`	Share one "slide until blocked" loop; each subclass only lists its directions.
`Board`	8×8 grid of pieces; `copy()`, `kingOf(color)`, `isAttacked(square, byColor)`, `render()`. No turn or history knowledge.
`Move` (record) + `MoveType` (enum)	A move as data: from, to, type (NORMAL, CASTLE_KINGSIDE, CASTLE_QUEENSIDE, EN_PASSANT, PROMOTION), promotion piece.
`CastlingRights` (record)	The four castling permissions; how they change after a move.
`Rules`	Stateless (keeps no data between calls) rules engine: pseudo-legal moves (allowed by piece movement, before the king-safety check), legal moves (king-safety filter), special moves, `apply(board, move)`, `isInCheck`.
`Game`	Turn, status, winner, en passant square, castling rights, move log, undo (Command + Memento — patterns explained below), resign, forfeit, random side assignment. Thread-safe (correct even when several threads call it at once).
`Player` (record), `GameStatus` (enum), `IllegalMoveException`	Supporting types.

Class diagram

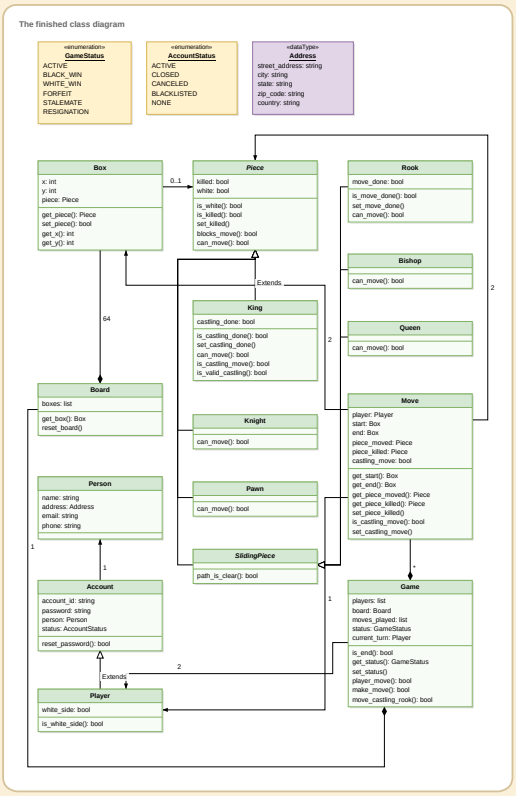
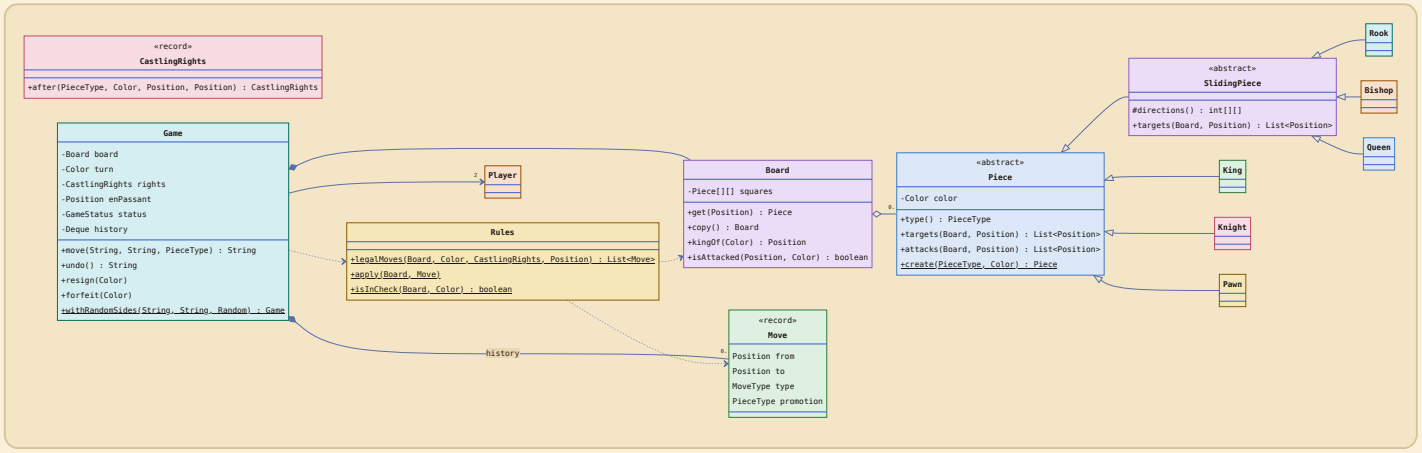


Figure — Course class diagram: Box, Board, Piece hierarchy with SlidingPiece, Move, Player, Account, Game, GameStatus (course diagram)

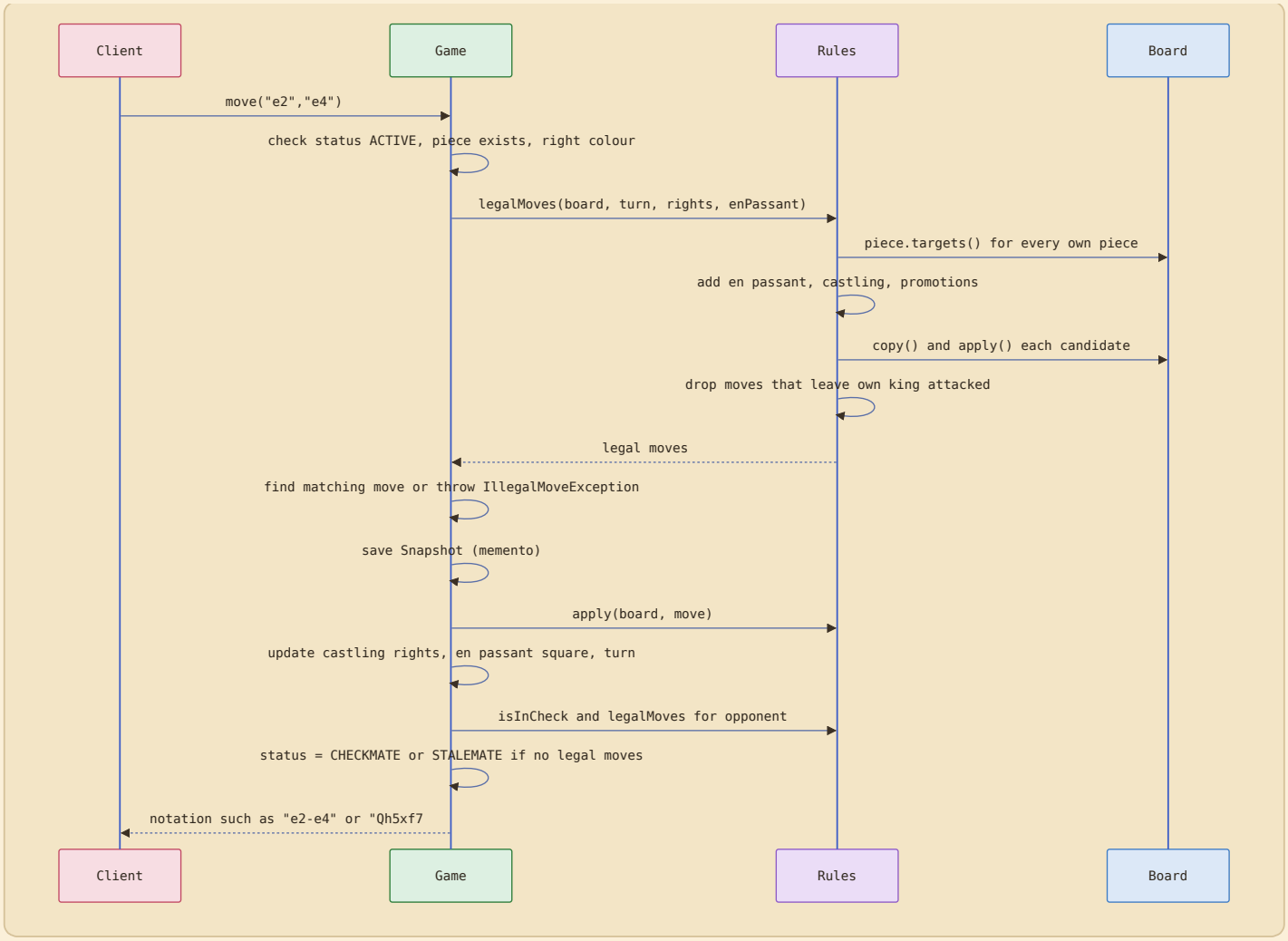
In the course model (figure above), a `Board` is **composed** of 64 `Box`es; a `Box` refers to at most one `Piece` with a *plain association* (not composition), because capturing a piece takes it off the board without destroying it — which is what makes a move log replayable. `Piece` is abstract with `can_move(board, start, end)`; `SlidingPiece` sits between `Piece` and rook/bishop/queen so the "path is clear" check is written once, while knight and king extend `Piece` directly. A `Move` is a *record* of what happened (player, start and end box, piece moved, piece captured, castling flag). `Game` owns the board and the list of moves and refers to the two players (they exist before and after the game). The course's `GameStatus` has ACTIVE, BLACK_WIN, WHITE_WIN, FORFEIT, STALEMATE, RESIGNATION.

Our Java design keeps that shape and fixes the two limits the course itself points out: (1) its pieces decide legality from the board alone, so history-based rules (castling rights, en passant) have nowhere to live, and (2) checkmate/stalemate are not detected. We add `Rules` (which sees history through `CastlingRights` and the en passant square) and make `Move` carry a `MoveType`, so a single move can move two pieces (castling) or remove a piece that is not on the target square (en passant).

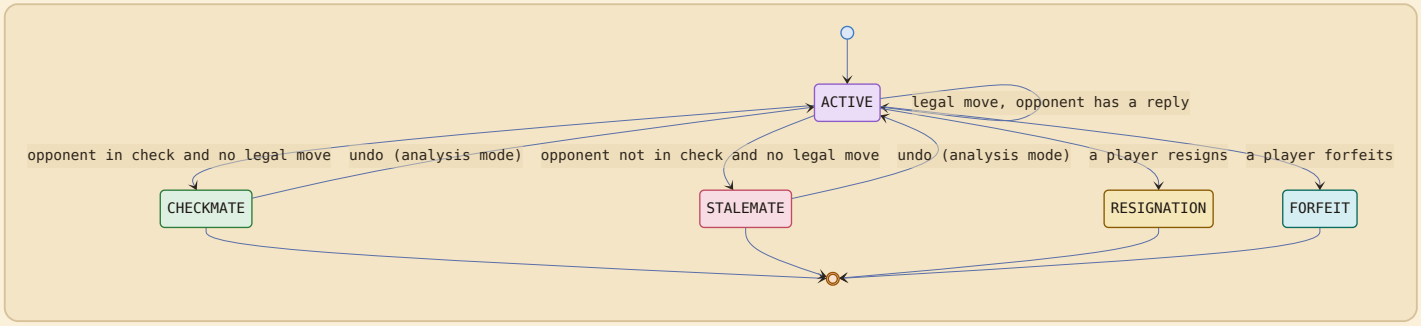


Key flows

Making a move



Game status



How check, checkmate and stalemate are detected (the "reasonable depth" answer)

1. **Pseudo-legal moves** — every move allowed by the pieces' movement rules, plus castling, en passant and promotions.
2. **Legal moves** — for each pseudo-legal move, apply it to a *copy* of the board and keep it only if the mover's king is **not attacked** afterwards. This one filter handles pins, moving into check, and ignoring a check, with no special cases.
3. **Is the king attacked?** Ask every enemy piece for the squares it attacks (`attacks(...)`); pawns attack diagonally even onto empty squares, which is why `Pawn` overrides `attacks`.
4. After a move, look at the **opponent**: if it has **zero legal moves**, it is **checkmate** when its king is in check, otherwise **stalemate**.

This is a one-ply (one move deep) search, which is all the *rules* need. Searching deeper is what a chess *engine* does to choose good moves — not required for enforcing rules.

Cost: at most ~200 pseudo-legal moves, each checked with an attack scan over ≤16 pieces — trivial for a human game. A production engine would use bitboards (each piece type's squares packed into one 64-bit number, so set

operations become fast bit tricks) and incremental attack maps (attacked squares updated per move instead of recomputed); mention it only if asked.

Design decisions & patterns used

Decision	Why	Alternative rejected
Polymorphic pieces (`targets` overridden per piece), **Template-like** `SlidingPiece`.	Each piece's rule lives in its own class; the blocked-path loop is written once for three pieces.	One giant `switch (pieceType)` in the board.
Sealed hierarchy (`sealed ... permits`).	The set of chess pieces is closed; the compiler knows all subclasses, so `switch` on type can be exhaustive (the compiler checks every case is handled).	Open hierarchy — fine too for variants; say so.
Pieces are **immutable**; `Board.copy()` copies references (pointers to the same piece objects) only.	Trial moves and snapshots are cheap and safe.	Mutable pieces with `hasMoved` flags — then undo must restore every flag.
History-based rules in `Rules` / `Game`, not in pieces.	Castling rights and en passant depend on earlier moves. The course notes that a piece seeing only the board cannot know this.	Letting `King` look up history — couples a piece to the game log.
`Move` carries a `MoveType`.	One move can move two pieces (castling) or capture off-square (en passant).	Special-casing in `Game` (the course's `move_castling_rook`), which breaks "rules live in one place".
Command + Memento for undo. `Move` is the command (an action stored as an object); `Snapshot` is the memento (a saved copy of an object's state that can be restored later) of the state before it.	Restoring a snapshot is always correct, even for promotion/castling/en passant, with no inverse logic.	Pure Command with `undo()` per move type — works, but every special move needs a hand-written inverse, a common bug source.
Factory method `Piece.create(type, color)`.	Used by board set-up and promotion.	`new` calls scattered around.
`Game` methods are `synchronized`.	Two player connections may call `move` / `resign` at the same time.	No locking — race (result depends on which thread runs first) between a move and a resignation.

Memento vs incremental undo — trade-off. A snapshot of an 8×8 reference array is 64 references (~0.5 KB): nothing for a human game (a few hundred moves). An engine searching millions of positions would instead store only what the move changed (captured piece, old castling rights, old en passant square) and reverse it. Know both.

Java implementation

Package `lld.ch17.chess`.

```java
// File: code/lld/ch17/chess/Color.java
package lld.ch17.chess;

public enum Color {
    WHITE, BLACK;

    public Color opposite() { return this == WHITE ? BLACK : WHITE; }
}
```

```java
// File: code/lld/ch17/chess/PieceType.java
package lld.ch17.chess;

public enum PieceType {
    KING('K'), QUEEN('Q'), ROOK('R'), BISHOP('B'), KNIGHT('N'), PAWN('P');

    private final char letter;

    PieceType(char letter) { this.letter = letter; }

    public char letter() { return letter; }
}
```

```java
// File: code/lld/ch17/chess/Position.java
package lld.ch17.chess;

/** A square. file 0..7 = a..h, rank 0..7 = 1..8. Immutable value object. */
public record Position(int file, int rank) {
    public Position {
        if (!onBoard(file, rank)) throw new IllegalArgumentException("off board: " + file + "," + rank);
    }

    public static boolean onBoard(int file, int rank) { return file >= 0 && file < 8 && rank >= 0 && rank < 8; }

    /** Parses algebraic notation such as "e4". */
    public static Position of(String sq) {
        if (sq == null || sq.length() != 2) throw new IllegalArgumentException("bad square: " + sq);
        return new Position(sq.charAt(0) - 'a', sq.charAt(1) - '1');
    }

    @Override public String toString() { return "" + (char) ('a' + file) + (rank + 1); }
}
```

```java
// File: code/lld/ch17/chess/Piece.java
package lld.ch17.chess;

import java.util.List;

/**
 * Every piece answers "which squares can I reach from here on this board?" in its own way
 * (polymorphism). Pieces are immutable: moving a piece moves the reference, so a board copy is cheap
 * and undo can simply restore an old board. Rules that need history (castling rights, en passant)
 * live in Rules/Game, not here.
 */
public abstract sealed class Piece permits King, Knight, Pawn, SlidingPiece {
    private final Color color;

    protected Piece(Color color) { this.color = color; }

    public Color color() { return color; }

    public abstract PieceType type();

    /** Squares this piece may move to by its ordinary rules (captures included, own-king safety NOT checked). */
    public abstract List<Position> targets(Board board, Position from);

    /** Squares this piece attacks. Same as targets for every piece except the pawn. */
    public List<Position> attacks(Board board, Position from) { return targets(board, from); }

    /** Empty square or enemy piece. */
    protected boolean canLandOn(Board board, Position p) {
        Piece other = board.get(p);
        return other == null || other.color != color;
    }

    public char symbol() {
        char c = type().letter();
        return color == Color.WHITE ? c : Character.toLowerCase(c);
    }

    /** Simple factory used for set-up and promotion. */
    public static Piece create(PieceType type, Color color) {
        return switch (type) {
            case KING -> new King(color);
            case QUEEN -> new Queen(color);
            case ROOK -> new Rook(color);
            case BISHOP -> new Bishop(color);
            case KNIGHT -> new Knight(color);
            case PAWN -> new Pawn(color);
        };
    }
}
```

```java
// File: code/lld/ch17/chess/StepPieces.java
package lld.ch17.chess;

import java.util.ArrayList;
import java.util.List;

/** Shared helper for pieces that jump a fixed set of offsets (king, knight). */
final class StepPieces {
    private StepPieces() {}

    static List<Position> step(Piece piece, Board board, Position from, int[][] offsets) {
        List<Position> out = new ArrayList<>();
        for (int[] d : offsets) {
            int f = from.file() + d[0], r = from.rank() + d[1];
            if (!Position.onBoard(f, r)) continue;
            Position p = new Position(f, r);
            if (piece.canLandOn(board, p)) out.add(p);
        }
        return out;
    }
}
```

```java
// File: code/lld/ch17/chess/King.java
package lld.ch17.chess;

import java.util.List;

/** One square in any direction. Castling is a two-piece move, so Rules generates it, not King. */
public final class King extends Piece {
    private static final int[][] OFFSETS = {{1,0},{-1,0},{0,1},{0,-1},{1,1},{1,-1},{-1,1},{-1,-1}};

    public King(Color color) { super(color); }

    @Override public PieceType type() { return PieceType.KING; }

    @Override public List<Position> targets(Board board, Position from) {
        return StepPieces.step(this, board, from, OFFSETS);
    }
}
```

```java
// File: code/lld/ch17/chess/Knight.java
package lld.ch17.chess;

import java.util.List;

/** L-shaped jump; the only piece that ignores pieces in between. */
public final class Knight extends Piece {
    private static final int[][] OFFSETS = {{1,2},{2,1},{2,-1},{1,-2},{-1,-2},{-2,-1},{-2,1},{-1,2}};

    public Knight(Color color) { super(color); }

    @Override public PieceType type() { return PieceType.KNIGHT; }

    @Override public List<Position> targets(Board board, Position from) {
        return StepPieces.step(this, board, from, OFFSETS);
    }
}
```

```java
// File: code/lld/ch17/chess/SlidingPiece.java
package lld.ch17.chess;

import java.util.ArrayList;
import java.util.List;

/** Rook, bishop and queen slide until blocked. The "is the path clear" loop is written once, here. */
public abstract sealed class SlidingPiece extends Piece permits Rook, Bishop, Queen {
    protected static final int[][] STRAIGHT = {{1,0},{-1,0},{0,1},{0,-1}};
    protected static final int[][] DIAGONAL = {{1,1},{1,-1},{-1,1},{-1,-1}};

    protected SlidingPiece(Color color) { super(color); }

    protected abstract int[][] directions();

    @Override public final List<Position> targets(Board board, Position from) {
        List<Position> out = new ArrayList<>();
        for (int[] d : directions()) {
            int f = from.file() + d[0], r = from.rank() + d[1];
            while (Position.onBoard(f, r)) {
                Position p = new Position(f, r);
                Piece occupant = board.get(p);
                if (occupant == null) {
                    out.add(p);
                } else {
                    if (occupant.color() != color()) out.add(p);   // capture, then stop
                    break;
                }
                f += d[0];
                r += d[1];
            }
        }
        return out;
    }
}
```

```java
// File: code/lld/ch17/chess/Rook.java
package lld.ch17.chess;

public final class Rook extends SlidingPiece {
    public Rook(Color color) { super(color); }
    @Override public PieceType type() { return PieceType.ROOK; }
    @Override protected int[][] directions() { return STRAIGHT; }
}
```

```java
// File: code/lld/ch17/chess/Bishop.java
package lld.ch17.chess;

public final class Bishop extends SlidingPiece {
    public Bishop(Color color) { super(color); }
    @Override public PieceType type() { return PieceType.BISHOP; }
    @Override protected int[][] directions() { return DIAGONAL; }
}
```

```java
// File: code/lld/ch17/chess/Queen.java
package lld.ch17.chess;

public final class Queen extends SlidingPiece {
    private static final int[][] ALL = {{1,0},{-1,0},{0,1},{0,-1},{1,1},{1,-1},{-1,1},{-1,-1}};
    public Queen(Color color) { super(color); }
    @Override public PieceType type() { return PieceType.QUEEN; }
    @Override protected int[][] directions() { return ALL; }
}
```

```java
// File: code/lld/ch17/chess/Pawn.java
package lld.ch17.chess;

import java.util.ArrayList;
import java.util.List;

/** Moves straight forward but captures diagonally. En passant and promotion are added by Rules. */
public final class Pawn extends Piece {
    public Pawn(Color color) { super(color); }

    @Override public PieceType type() { return PieceType.PAWN; }

    private int dir() { return color() == Color.WHITE ? 1 : -1; }

    @Override public List<Position> targets(Board board, Position from) {
        List<Position> out = new ArrayList<>();
        int f = from.file(), r = from.rank() + dir();
        if (Position.onBoard(f, r) && board.get(new Position(f, r)) == null) {
            out.add(new Position(f, r));
            int startRank = color() == Color.WHITE ? 1 : 6;
            int r2 = r + dir();
            if (from.rank() == startRank && board.get(new Position(f, r2)) == null) out.add(new Position(f, r2));
        }
        for (Position p : attacks(board, from)) {
            Piece victim = board.get(p);
            if (victim != null && victim.color() != color()) out.add(p);
        }
        return out;
    }

    /** A pawn attacks both forward diagonals whether or not anything stands there. */
    @Override public List<Position> attacks(Board board, Position from) {
        List<Position> out = new ArrayList<>(2);
        int r = from.rank() + dir();
        for (int df : new int[]{-1, 1}) {
            int f = from.file() + df;
            if (Position.onBoard(f, r)) out.add(new Position(f, r));
        }
        return out;
    }
}
```

```java
// File: code/lld/ch17/chess/Board.java
package lld.ch17.chess;

/** 8x8 grid of (immutable) pieces. Knows geometry and attacks; knows nothing about turns or history. */
public final class Board {
    private final Piece[][] squares = new Piece[8][8]; // [file][rank]

    public static Board standard() {
        Board b = new Board();
        PieceType[] back = {PieceType.ROOK, PieceType.KNIGHT, PieceType.BISHOP, PieceType.QUEEN,
                            PieceType.KING, PieceType.BISHOP, PieceType.KNIGHT, PieceType.ROOK};
        for (int f = 0; f < 8; f++) {
            b.squares[f][0] = Piece.create(back[f], Color.WHITE);
            b.squares[f][1] = new Pawn(Color.WHITE);
            b.squares[f][6] = new Pawn(Color.BLACK);
            b.squares[f][7] = Piece.create(back[f], Color.BLACK);
        }
        return b;
    }

    public Piece get(Position p) { return squares[p.file()][p.rank()]; }

    void set(Position p, Piece piece) { squares[p.file()][p.rank()] = piece; }

    /** Pieces are immutable, so copying the references is a full, independent copy of the position. */
    public Board copy() {
        Board b = new Board();
        for (int f = 0; f < 8; f++) b.squares[f] = squares[f].clone();
        return b;
    }

    public Position kingOf(Color color) {
        for (int f = 0; f < 8; f++)
            for (int r = 0; r < 8; r++) {
                Piece p = squares[f][r];
                if (p != null && p.type() == PieceType.KING && p.color() == color) return new Position(f, r);
            }
        throw new IllegalStateException("no " + color + " king on board");
    }

    public boolean isAttacked(Position target, Color by) {
        for (int f = 0; f < 8; f++)
            for (int r = 0; r < 8; r++) {
                Piece p = squares[f][r];
                if (p != null && p.color() == by && p.attacks(this, new Position(f, r)).contains(target)) return
true;
            }
        return false;
    }

    public String render() {
        StringBuilder sb = new StringBuilder();
        for (int r = 7; r >= 0; r--) {
            sb.append("    ").append(r + 1).append(' ');
            for (int f = 0; f < 8; f++) {
                Piece p = squares[f][r];
                sb.append(' ').append(p == null ? '.' : p.symbol());
            }
            sb.append('\n');
        }
        return sb.append("       a b c d e f g h").toString();
    }
}
```

```java
// File: code/lld/ch17/chess/MoveType.java
package lld.ch17.chess;

public enum MoveType { NORMAL, CASTLE_KINGSIDE, CASTLE_QUEENSIDE, EN_PASSANT, PROMOTION }
```

```java
// File: code/lld/ch17/chess/Move.java
package lld.ch17.chess;

/** A move as data (the Command's payload). promotion is null unless type == PROMOTION. */
public record Move(Position from, Position to, MoveType type, PieceType promotion) {
    public static Move normal(Position from, Position to) { return new Move(from, to, MoveType.NORMAL, null); }
}
```

```java
// File: code/lld/ch17/chess/CastlingRights.java
package lld.ch17.chess;

/** Which castles are still allowed. Lost forever once the king or that rook moves (or the rook is captured). */
public record CastlingRights(boolean whiteKingSide, boolean whiteQueenSide, boolean blackKingSide, boolean
blackQueenSide) {
    public static CastlingRights all() { return new CastlingRights(true, true, true, true); }

    public boolean kingSide(Color c) { return c == Color.WHITE ? whiteKingSide : blackKingSide; }
    public boolean queenSide(Color c) { return c == Color.WHITE ? whiteQueenSide : blackQueenSide; }

    /** Returns the rights after a move from -> to by a piece of the given type. */
    public CastlingRights after(PieceType moved, Color mover, Position from, Position to) {
        boolean wk = whiteKingSide, wq = whiteQueenSide, bk = blackKingSide, bq = blackQueenSide;
        if (moved == PieceType.KING) {
            if (mover == Color.WHITE) { wk = false; wq = false; } else { bk = false; bq = false; }
        }
        for (Position p : new Position[]{from, to}) {          // a rook leaving or being captured on its corner
            if (p.equals(new Position(0, 0))) wq = false;
            if (p.equals(new Position(7, 0))) wk = false;
            if (p.equals(new Position(0, 7))) bq = false;
            if (p.equals(new Position(7, 7))) bk = false;
        }
        return new CastlingRights(wk, wq, bk, bq);
    }
}
```

```java
// File: code/lld/ch17/chess/Rules.java
package lld.ch17.chess;

import java.util.ArrayList;
import java.util.List;

/**
 * Stateless rules engine: generates legal moves (including castling, en passant, promotion)
 * and applies a move to a board. "Legal" = pseudo-legal AND does not leave your own king attacked.
 */
public final class Rules {
    private Rules() {}

    private static final PieceType[] PROMOTIONS = {PieceType.QUEEN, PieceType.ROOK, PieceType.BISHOP,
PieceType.KNIGHT};

    public static boolean isInCheck(Board board, Color side) {
        return board.isAttacked(board.kingOf(side), side.opposite());
    }

    /** Moves that follow the pieces' movement rules, before the king-safety filter. */
    public static List<Move> pseudoLegalMoves(Board board, Color side, CastlingRights rights, Position enPassant) {
        List<Move> moves = new ArrayList<>();
        int lastRank = side == Color.WHITE ? 7 : 0;
        for (int f = 0; f < 8; f++) {
            for (int r = 0; r < 8; r++) {
                Position from = new Position(f, r);
                Piece piece = board.get(from);
                if (piece == null || piece.color() != side) continue;
                for (Position to : piece.targets(board, from)) {
                    if (piece.type() == PieceType.PAWN && to.rank() == lastRank) {
                        for (PieceType promo : PROMOTIONS) moves.add(new Move(from, to, MoveType.PROMOTION,
promo));
                    } else {
                        moves.add(Move.normal(from, to));
                    }
                }
                if (piece.type() == PieceType.PAWN && enPassant != null && piece.attacks(board,
from).contains(enPassant)) {
                    moves.add(new Move(from, enPassant, MoveType.EN_PASSANT, null));
                }
            }
        }
        addCastling(board, side, rights, moves);
        return moves;
    }

    private static void addCastling(Board board, Color side, CastlingRights rights, List<Move> moves) {
        int r = side == Color.WHITE ? 0 : 7;
        Position kingSq = new Position(4, r);
        Piece king = board.get(kingSq);
        if (king == null || king.type() != PieceType.KING || king.color() != side) return;
        Color enemy = side.opposite();
        if (board.isAttacked(kingSq, enemy)) return;                        // cannot castle out of check
        if (rights.kingSide(side) && isOwnRook(board, new Position(7, r), side)
                && empty(board, r, 5, 6)
                && !board.isAttacked(new Position(5, r), enemy)             // cannot pass through check
                && !board.isAttacked(new Position(6, r), enemy)) {
            moves.add(new Move(kingSq, new Position(6, r), MoveType.CASTLE_KINGSIDE, null));
        }
        if (rights.queenSide(side) && isOwnRook(board, new Position(0, r), side)
                && empty(board, r, 1, 2, 3)
                && !board.isAttacked(new Position(3, r), enemy)
                && !board.isAttacked(new Position(2, r), enemy)) {
            moves.add(new Move(kingSq, new Position(2, r), MoveType.CASTLE_QUEENSIDE, null));
        }
    }
```

```
    private static boolean isOwnRook(Board b, Position p, Color side) {
        Piece piece = b.get(p);
        return piece != null && piece.type() == PieceType.ROOK && piece.color() == side;
    }

    private static boolean empty(Board b, int rank, int... files) {
        for (int f : files) if (b.get(new Position(f, rank)) != null) return false;
        return true;
    }

    /** Explains why a king move of two squares (a castling attempt) is not legal right now. */
    public static String whyCannotCastle(Board board, Color side, CastlingRights rights, Position kingTo) {
        int r = side == Color.WHITE ? 0 : 7;
        if (kingTo.rank() != r || (kingTo.file() != 6 && kingTo.file() != 2)) return "that is not a castling
square";
        boolean kingSide = kingTo.file() == 6;
        if (!(kingSide ? rights.kingSide(side) : rights.queenSide(side))
                || !isOwnRook(board, new Position(kingSide ? 7 : 0, r), side))
            return "the king or that rook has already moved";
        if (!(kingSide ? empty(board, r, 5, 6) : empty(board, r, 1, 2, 3)))
            return "the squares between king and rook are not empty";
        Color enemy = side.opposite();
        if (board.isAttacked(new Position(4, r), enemy)) return "the king is in check";
        Position crossed = new Position(kingSide ? 5 : 3, r);
        if (board.isAttacked(crossed, enemy)) return "the king would pass through the attacked square " + crossed;
        if (board.isAttacked(kingTo, enemy)) return "the king would land on the attacked square " + kingTo;
        return "it is not legal in this position";
    }

    public static List<Move> legalMoves(Board board, Color side, CastlingRights rights, Position enPassant) {
        List<Move> legal = new ArrayList<>();
        for (Move m : pseudoLegalMoves(board, side, rights, enPassant)) {
            Board trial = board.copy();
            apply(trial, m);
            if (!isInCheck(trial, side)) legal.add(m);
        }
        return legal;
    }

    /** Mutates the board. Handles the extra piece movements of the special moves. */
    public static void apply(Board board, Move m) {
        Piece piece = board.get(m.from());
        board.set(m.from(), null);
        board.set(m.to(), piece);
        switch (m.type()) {
            case EN_PASSANT -> board.set(new Position(m.to().file(), m.from().rank()), null);   // the passed pawn
            case PROMOTION -> board.set(m.to(), Piece.create(m.promotion(), piece.color()));
            case CASTLE_KINGSIDE -> moveRook(board, m.from().rank(), 7, 5);
            case CASTLE_QUEENSIDE -> moveRook(board, m.from().rank(), 0, 3);
            case NORMAL -> { }
        }
    }

    private static void moveRook(Board board, int rank, int fromFile, int toFile) {
        Position from = new Position(fromFile, rank);
        board.set(new Position(toFile, rank), board.get(from));
        board.set(from, null);
    }
}
```

```
// File: code/lld/ch17/chess/GameStatus.java
package lld.ch17.chess;

public enum GameStatus { ACTIVE, CHECKMATE, STALEMATE, RESIGNATION, FORFEIT }
```

```java
// File: code/lld/ch17/chess/Player.java
package lld.ch17.chess;

public record Player(String name, Color color) {}
```

```java
// File: code/lld/ch17/chess/IllegalMoveException.java
package lld.ch17.chess;

public final class IllegalMoveException extends RuntimeException {
    private static final long serialVersionUID = 1L;

    public IllegalMoveException(String message) { super(message); }
}
```

```java
// File: code/lld/ch17/chess/Game.java
package lld.ch17.chess;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;
import java.util.Optional;
import java.util.Random;

/**
 * Controls one game: whose turn it is, validates and plays moves, detects check/checkmate/stalemate,
 * keeps the move log, and supports undo.
 *
 * Undo = Command + Memento: each history entry stores the move (the command) and a snapshot of the
 * state before it (the memento). Undo restores the snapshot, which is always correct even for
 * castling, en passant and promotion.
 */
public final class Game {
    /** Memento: everything needed to restore the position exactly. */
    private record Snapshot(Board board, Color turn, CastlingRights rights, Position enPassant,
                            GameStatus status, Color winner) {}

    private record HistoryEntry(Player player, Move move, String notation, Snapshot before) {}

    private final Player white, black;
    private Board board = Board.standard();
    private Color turn = Color.WHITE;                       // white always moves first
    private CastlingRights rights = CastlingRights.all();
    private Position enPassant;                             // square a pawn just skipped over, or null
    private GameStatus status = GameStatus.ACTIVE;
    private Color winner;
    private final Deque<HistoryEntry> history = new ArrayDeque<>();

    public Game(Player white, Player black) {
        if (white.color() != Color.WHITE || black.color() != Color.BLACK) throw new
IllegalArgumentException("colors mismatch");
        this.white = white;
        this.black = black;
    }

    /** The course's rule: each player is randomly assigned a side. Pass a seeded Random for repeatable tests. */
    public static Game withRandomSides(String nameA, String nameB, Random rng) {
        boolean aIsWhite = rng.nextBoolean();
        return aIsWhite ? new Game(new Player(nameA, Color.WHITE), new Player(nameB, Color.BLACK))
                        : new Game(new Player(nameB, Color.WHITE), new Player(nameA, Color.BLACK));
    }

    public synchronized Player player(Color c) { return c == Color.WHITE ? white : black; }

    public synchronized String move(String from, String to) { return move(from, to, null); }

    public synchronized String move(String fromSq, String toSq, PieceType promotion) {
        if (status != GameStatus.ACTIVE) throw new IllegalMoveException("game is over: " + status);
        Position from = Position.of(fromSq), to = Position.of(toSq);
        Piece piece = board.get(from);
        if (piece == null) throw new IllegalMoveException("no piece on " + from);
        if (piece.color() != turn) throw new IllegalMoveException("it is " + turn + "'s turn");

        List<Move> legal = Rules.legalMoves(board, turn, rights, enPassant);
        Optional<Move> chosen = legal.stream()
                .filter(m -> m.from().equals(from) && m.to().equals(to))
                .filter(m -> m.type() != MoveType.PROMOTION
                        || m.promotion() == (promotion == null ? PieceType.QUEEN : promotion))
                .findFirst();
        if (chosen.isEmpty()) {
            if (piece.type() == PieceType.KING && Math.abs(to.file() - from.file()) == 2)
                throw new IllegalMoveException("cannot castle " + fromSq + "-" + toSq + ": "
                        + Rules.whyCannotCastle(board, turn, rights, to));
```

```
            boolean reachable = Rules.pseudoLegalMoves(board, turn, rights, enPassant).stream()
                    .anyMatch(m -> m.from().equals(from) && m.to().equals(to));
            throw new IllegalMoveException(reachable
                    ? fromSq + "-" + toSq + " would leave the " + turn + " king in check"
                    : piece.type() + " on " + fromSq + " cannot move to " + toSq);
        }
        return play(chosen.get(), piece);
    }

    private String play(Move m, Piece piece) {
        Player mover = turn == Color.WHITE ? white : black;
        Snapshot before = new Snapshot(board.copy(), turn, rights, enPassant, status, winner);
        boolean capture = board.get(m.to()) != null || m.type() == MoveType.EN_PASSANT;

        Rules.apply(board, m);
        rights = rights.after(piece.type(), turn, m.from(), m.to());
        enPassant = (piece.type() == PieceType.PAWN && Math.abs(m.to().rank() - m.from().rank()) == 2)
                ? new Position(m.from().file(), (m.from().rank() + m.to().rank()) / 2) : null;
        turn = turn.opposite();

        // End-of-game detection: the side to move has no legal move -> checkmate if in check, else stalemate.
        boolean check = Rules.isInCheck(board, turn);
        boolean noMoves = Rules.legalMoves(board, turn, rights, enPassant).isEmpty();
        if (noMoves) {
            status = check ? GameStatus.CHECKMATE : GameStatus.STALEMATE;
            winner = check ? mover.color() : null;
        }
        String notation = notation(m, piece, capture) + (check ? (noMoves ? "#" : "+") : "");
        history.push(new HistoryEntry(mover, m, notation, before));
        return notation;
    }

    private static String notation(Move m, Piece piece, boolean capture) {
        return switch (m.type()) {
            case CASTLE_KINGSIDE -> "O-O";
            case CASTLE_QUEENSIDE -> "O-O-O";
            default -> {
                String prefix = piece.type() == PieceType.PAWN ? "" : String.valueOf(piece.type().letter());
                String s = prefix + m.from() + (capture ? "x" : "-") + m.to();
                if (m.type() == MoveType.PROMOTION) s += "=" + m.promotion().letter();
                if (m.type() == MoveType.EN_PASSANT) s += " e.p.";
                yield s;
            }
        };
    }

    /** Undo the last move (e.g. for analysis mode or a take-back both players agree on). */
    public synchronized String undo() {
        if (history.isEmpty()) throw new IllegalStateException("nothing to undo");
        HistoryEntry last = history.pop();
        Snapshot s = last.before();
        board = s.board(); turn = s.turn(); rights = s.rights(); enPassant = s.enPassant();
        status = s.status(); winner = s.winner();
        return last.notation();
    }

    public synchronized void resign(Color who) {
        if (status != GameStatus.ACTIVE) throw new IllegalStateException("game is over");
        status = GameStatus.RESIGNATION;
        winner = who.opposite();
    }

    /** Forfeit: the player loses without choosing to (left the game, ran out of time, broke the rules of the
site). */
    public synchronized void forfeit(Color who) {
        if (status != GameStatus.ACTIVE) throw new IllegalStateException("game is over");
        status = GameStatus.FORFEIT;
        winner = who.opposite();
    }
```

```
    public synchronized boolean isInCheck() { return Rules.isInCheck(board, turn); }
    public synchronized int legalMoveCount() { return Rules.legalMoves(board, turn, rights, enPassant).size(); }
    public synchronized GameStatus status() { return status; }
    public synchronized Color winner() { return winner; }
    public synchronized Color turn() { return turn; }
    public synchronized String render() { return board.render(); }

    /** The move log, oldest first, numbered like a score sheet. */
    public synchronized String moveLog() {
        List<HistoryEntry> list = new java.util.ArrayList<>(history);
        java.util.Collections.reverse(list);
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < list.size(); i++) {
            if (i % 2 == 0) sb.append(i / 2 + 1).append(". ");
            sb.append(list.get(i).notation()).append(' ');
        }
        return sb.toString().trim();
    }
}
```

```java
// File: code/lld/ch17/chess/Main.java
package lld.ch17.chess;

import java.util.Random;

public final class Main {
    private static Game newGame() { return new Game(new Player("Ann", Color.WHITE), new Player("Ben",
Color.BLACK)); }

    private static void play(Game g, String... moves) {
        for (String mv : moves) {
            String[] parts = mv.split("-");
            PieceType promo = parts[1].length() == 3 ? PieceType.valueOf(switch (parts[1].charAt(2)) {
                case 'Q' -> "QUEEN"; case 'R' -> "ROOK"; case 'B' -> "BISHOP"; default -> "KNIGHT"; }) : null;
            g.move(parts[0], parts[1].substring(0, 2), promo);
        }
    }

    private static void tryMove(Game g, String from, String to, PieceType promo) {
        try {
            System.out.println("  " + from + "-" + to + " played as " + g.move(from, to, promo));
        } catch (IllegalMoveException e) {
            System.out.println("  " + from + "-" + to + " rejected: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        System.out.println("== 1. Validation, Scholar's mate, undo ==");
        Game g = newGame();
        tryMove(g, "e7", "e5", null);                // black out of turn
        tryMove(g, "e1", "e3", null);                // king cannot jump two squares
        tryMove(g, "a1", "a4", null);                // rook blocked by its own pawn
        play(g, "e2-e4", "e7-e5", "f1-c4", "b8-c6", "d1-h5", "g8-f6");
        tryMove(g, "h5", "f7", null);
        System.out.println("  Log: " + g.moveLog());
        System.out.println("  Status: " + g.status() + ", winner " + g.winner());
        System.out.println(g.render());
        System.out.println("  undo " + g.undo() + " -> status " + g.status() + ", " + g.turn() + " to move, "
                + g.legalMoveCount() + " legal moves");

        System.out.println("\n== 2. En passant, castling, promotion, and a move that ignores check ==");
        Game s = newGame();
        play(s, "e2-e4", "g8-f6", "e4-e5", "d7-d5");
        tryMove(s, "e5", "d6", null);                // en passant
        play(s, "b8-c6", "g1-f3", "a7-a5", "f1-c4", "a5-a4");
        tryMove(s, "e1", "g1", null);                // castle kingside
        play(s, "a4-a3", "d6-c7", "a3-b2");
        tryMove(s, "c7", "d8", PieceType.QUEEN);     // capture + promote, gives check
        System.out.println("  Black in check? " + s.isInCheck());
        tryMove(s, "b2", "a1", PieceType.KNIGHT);    // ignores the check -> illegal
        tryMove(s, "e8", "d8", null);                // king takes the new queen
        System.out.println("  Log: " + s.moveLog());
        System.out.println(s.render());

        System.out.println("\n== 3. Stalemate (Sam Loyd's 10-move stalemate) ==");
        Game st = newGame();
        play(st, "e2-e3", "a7-a5", "d1-h5", "a8-a6", "h5-a5", "h7-h5", "h2-h4", "a6-h6",
                 "a5-c7", "f7-f6", "c7-d7", "e8-f7", "d7-b7", "d8-d3", "b7-b8", "d3-h7",
                 "b8-c8", "f7-g6", "c8-e6");
        System.out.println("  Log: " + st.moveLog());
        System.out.println("  Status: " + st.status() + ", black in check? " + st.isInCheck()
                + ", black legal moves: " + st.legalMoveCount());
        System.out.println(st.render());

        System.out.println("\n== 4. Castling rules: through an attacked square, and out of check ==");
        Game c1 = newGame();
        play(c1, "g2-g3", "b7-b6", "f1-h3", "c8-a6", "g1-f3", "b8-c6", "e2-e4", "g8-f6");
        tryMove(c1, "e1", "g1", null);               // f1 is attacked by the bishop on a6
```

```
        Game c2 = newGame();
        play(c2, "e2-e4", "d7-d6", "d2-d4", "g8-f6", "b1-c3", "g7-g6", "g1-f3", "f8-g7", "f1-b5");
        tryMove(c2, "e8", "g8", null);               // black is in check from the b5 bishop
        tryMove(c2, "c7", "c6", null);               // block the check
        tryMove(c2, "e1", "g1", null);
        tryMove(c2, "e8", "g8", null);               // no longer in check: now legal

        System.out.println("\n== 5. Random sides, resignation and forfeit ==");
        Game r = Game.withRandomSides("Ann", "Ben", new Random(7));
        System.out.println("  white = " + r.player(Color.WHITE).name() + ", black = " +
r.player(Color.BLACK).name());
        play(r, "f2-f3", "e7-e5");
        r.resign(Color.WHITE);
        System.out.println("  Status: " + r.status() + ", winner " + r.winner());
        tryMove(r, "g2", "g4", null);
        Game f = newGame();
        f.forfeit(Color.BLACK);                       // e.g. black disconnected and did not come back
        System.out.println("  Status: " + f.status() + ", winner " + f.winner());
    }
}
```

Output:

```
== 1. Validation, Scholar's mate, undo ==
  e7-e5 rejected: it is WHITE's turn
  e1-e3 rejected: KING on e1 cannot move to e3
  a1-a4 rejected: ROOK on a1 cannot move to a4
  h5-f7 played as Qh5xf7#
  Log: 1. e2-e4 e7-e5 2. Bf1-c4 Nb8-c6 3. Qd1-h5 Ng8-f6 4. Qh5xf7#
  Status: CHECKMATE, winner WHITE
    8  r . b q k b . r
    7  p p p p . Q p p
    6  . . n . . n . .
    5  . . . . p . . .
    4  . . B . P . . .
    3  . . . . . . . .
    2  P P P P . P P P
    1  R N B . K . N R
       a b c d e f g h
  undo Qh5xf7# -> status ACTIVE, WHITE to move, 43 legal moves

== 2. En passant, castling, promotion, and a move that ignores check ==
  e5-d6 played as e5xd6 e.p.
  e1-g1 played as O-O
  c7-d8 played as c7xd8=Q+
  Black in check? true
  b2-a1 rejected: b2-a1 would leave the BLACK king in check
  e8-d8 played as Ke8xd8
  Log: 1. e2-e4 Ng8-f6 2. e4-e5 d7-d5 3. e5xd6 e.p. Nb8-c6 4. Ng1-f3 a7-a5 5. Bf1-c4 a5-a4 6. O-O a4-a3 7. d6xc7
a3xb2 8. c7xd8=Q+ Ke8xd8
    8  r . b k . b . r
    7  . p . . p p p p
    6  . . n . . n . .
    5  . . . . . . . .
    4  . . B . . . . .
    3  . . . . . N . .
    2  P p P P . P P P
    1  R N B Q . R K .
       a b c d e f g h

== 3. Stalemate (Sam Loyd's 10-move stalemate) ==
  Log: 1. e2-e3 a7-a5 2. Qd1-h5 Ra8-a6 3. Qh5xa5 h7-h5 4. h2-h4 Ra6-h6 5. Qa5xc7 f7-f6 6. Qc7xd7+ Ke8-f7 7. Qd7xb7
Qd8-d3 8. Qb7xb8 Qd3-h7 9. Qb8xc8 Kf7-g6 10. Qc8-e6
  Status: STALEMATE, black in check? false, black legal moves: 0
    8  . . . . . b n r
    7  . . . . p . p q
    6  . . . . Q p k r
    5  . . . . . . . p
    4  . . . . . . . P
    3  . . . . P . . .
    2  P P P P . P P .
    1  R N B . K B N R
       a b c d e f g h

== 4. Castling rules: through an attacked square, and out of check ==
  e1-g1 rejected: cannot castle e1-g1: the king would pass through the attacked square f1
  e8-g8 rejected: cannot castle e8-g8: the king is in check
  c7-c6 played as c7-c6
  e1-g1 played as O-O
  e8-g8 played as O-O

== 5. Random sides, resignation and forfeit ==
  white = Ann, black = Ben
  Status: RESIGNATION, winner BLACK
  g2-g4 rejected: game is over: RESIGNATION
  Status: FORFEIT, winner WHITE
```

Reading the output. Part 1 shows validation (out of turn, illegal king jump, blocked rook), then Scholar's mate (`Qh5xf7#`), and `undo()` returning the game to ACTIVE with white to move. Part 2 plays en passant (`e5xd6 e.p.` removes the black pawn from d5), kingside castling (`O-O`), and a capturing promotion that gives check

(`c7xd8=Q+`); black's attempt to ignore the check is rejected, and the king captures the new queen. Part 3 is the famous 10-move stalemate composed by Sam Loyd: black is not in check but has no legal move, so the game is a draw. Part 4 shows the castling guards: white may not castle through f1 while the a6 bishop attacks it, black may not castle out of check, and once the check is blocked both sides castle. Part 5 assigns sides at random (seeded) and shows resignation and forfeit.

Concurrency & edge cases

- **Two players, one game.** `Game`'s public methods are `synchronized`, so a move and a resignation cannot interleave (have their steps mixed together). For a server with thousands of games, each game gets its own lock (the object's monitor — the built-in lock every Java object carries, which `synchronized` uses) — games do not block each other.
- **Moving out of turn / game over / empty square** — rejected with a clear message before any rule work.
- **Pins and discovered checks** (a *pin* is a piece that cannot move because it shields its king; a *discovered check* is a check revealed when a blocking piece moves away) — handled automatically by the "apply to a copy and test the king" filter.
- **Castling through check** — explicitly tested for the squares the king crosses and lands on; the rook's crossing square on the queenside (b1/b8) may be attacked — that is legal. Section 4 of the output shows both refusals with their reason (`Rules.whyCannotCastle`): passing through an attacked f1, and castling while in check; once the check is blocked the same castle is accepted.
- **Castling rights** are lost when the king moves, when that rook moves, or when that rook is **captured** on its corner (we check both `from` and `to`).
- **En passant expires** after one move: the square is reset on every move unless a pawn just advanced two.
- **Promotion without a choice** defaults to a queen; underpromotion (knight, rook, bishop) is supported.
- **Not implemented, but should be named:** draw by threefold repetition (the same position occurs three times), the 50-move rule (50 moves each with no capture or pawn move), insufficient material (e.g. king vs king), clocks/timeouts, draw offers.

Extensions & follow-up questions

- **"Add threefold repetition."** — Keep a `Map<PositionKey, Integer>` where the key is (board, side to move, castling rights, en passant square). Production engines hash this (turn it into one compact number for quick comparison) with Zobrist hashing, which combines a random number per piece-and-square. Three occurrences → draw.
- **"Add a 50-move rule."** — A half-move (one player's move) clock reset on every pawn move or capture; at 100 half-moves either player may claim a draw.
- **"Add chess clocks."** — A `Clock` per player; the server stamps time on each move; a flag-fall (a player's time running out) ends the game (Observer — listeners register once and are told about every change — notifies both clients).
- **"Add an AI player."** — A `MoveStrategy` that picks from `legalMoves()`; a real one runs minimax (search ahead, assuming you pick your best move and the opponent picks the move worst for you) with alpha-beta pruning (skipping branches that provably cannot change the answer).
- **"Support Chess960 (a variant with a shuffled back row) or a new piece."** — New `Piece` subclass (e.g. an Archbishop = bishop + knight) and a different starting-position factory; existing pieces are untouched.
- **"Spectators and UI updates."** — Observer: `GameListener.onMove(notation)`. The course mentions a `GameController` (takes input, calls `Game`) and a `GameView` (redraws) — that is the MVC (Model–View–Controller: data, display and input handling kept in separate classes) split from Chapter 11.
- **"Why is `Position` a record?"** — It is a value: two `e4`s are equal. Records give `equals`/`hashCode` (the methods Java uses to decide if two objects are equal and to place them in hash tables) for free, which `List.contains` relies on.

Real-world example

Online chess sites such as Lichess and Chess.com treat the **server** as the source of truth (the one copy whose data wins whenever copies disagree): the board in your browser is only a view, and a move counts only after the server has accepted it as legal. Games are exported in PGN (Portable Game Notation — a move log like ours) and positions in FEN (Forsyth–Edwards Notation — one line describing the board, side to move, castling rights and en passant square, i.e. exactly the fields of our `Snapshot`). "Takeback" is offered only when the opponent agrees, and then the server undoes the move — the same `undo()` our `Game` exposes.

In the interview

30-second answer: "`Board` is an 8×8 grid of immutable `Piece`s. `Piece` is abstract with `targets(board, from)`; each piece overrides it, and rook/bishop/queen share a `SlidingPiece` loop. A stateless `Rules` class generates pseudo-legal moves, adds castling, en passant and promotion using history kept in `Game` (castling rights, en passant square), and filters out moves that leave your king attacked. After each move, if the opponent has no legal move, it's checkmate if they're in check, otherwise stalemate. `Game` handles turns, status, a move log, and undo via Command plus Memento."

How to spend 30–45 minutes: 5 min requirements → 5 min entities → 10 min class diagram (piece hierarchy, why history lives outside pieces) → 15 min code `Piece`/`SlidingPiece`/`Pawn` and the legal-move filter → 5–10 min checkmate/stalemate, special moves, concurrency.

Likely follow-ups with answers

- *"How do you detect checkmate?"* — No legal moves and in check. Legal = pseudo-legal filtered by "king not attacked after the move".
- *"Where does castling live?"* — In `Rules`, because it moves two pieces and depends on history; `MoveType.CASTLE_*` tells `apply` to move the rook too.
- *"Why is `Box -> Piece` not composition?"* — Removing a piece from a square doesn't destroy the piece; it goes to the captured list or the log.
- *"How is undo safe with promotion?"* — The memento stores the whole prior board, so undo simply puts the old pawn back.

Self-check

1. **Q:** Why does `Pawn` override `attacks()`? **A:** A pawn moves straight but attacks diagonally, even onto empty squares; check detection needs the attack squares.
2. **Q:** What is the difference between checkmate and stalemate? **A:** Both mean no legal move; checkmate is while in check (loss), stalemate is while not in check (draw).
3. **Q:** When is en passant allowed? **A:** Only on the move immediately after an enemy pawn advanced two squares past your pawn's attack square.
4. **Q:** Name three conditions for castling. **A:** King and that rook have not moved; squares between empty; king not in check, not passing through or landing on an attacked square.
5. **Q:** Why are pieces immutable in this design? **A:** Board copies for trial moves and undo snapshots can share pieces safely.

TL;DR

- Polymorphic `Piece.targets()`; `SlidingPiece` shares the path loop; pawn overrides `attacks()`.
- `Rules`: pseudo-legal → add castling / en passant / promotion → drop moves leaving own king attacked.
- No legal moves: in check = checkmate, else stalemate.
- History-dependent rules live in `Game`/`Rules`, not in pieces. Undo = Command + Memento. `synchronized` game for online play.

Design Tic-Tac-Toe

Problem & clarifying questions

Tic-tac-toe is played on a grid. Players take turns placing their mark (X or O). The first to fill a whole **row, column or diagonal** wins; if the board fills with no winner, it is a **draw**. The classic game is 3×3 with two players; interviewers love to generalise it to **N×N** and ask for **fast win detection**.

Question	Assumed answer
Board size?	Any N ≥ 3 (classic N = 3).
Win condition?	A full line of N marks: any row, column, or one of the two main diagonals.
Players?	Two by default; the design allows more (each with its own mark).
Human or computer?	Both: each player has a pluggable **move strategy** (human input, random bot, rule-based bot, scripted test).
Invalid moves?	Rejected (occupied or off-board); the same player tries again.
UI?	Console rendering only; the engine exposes state for any UI.

Requirements

Functional

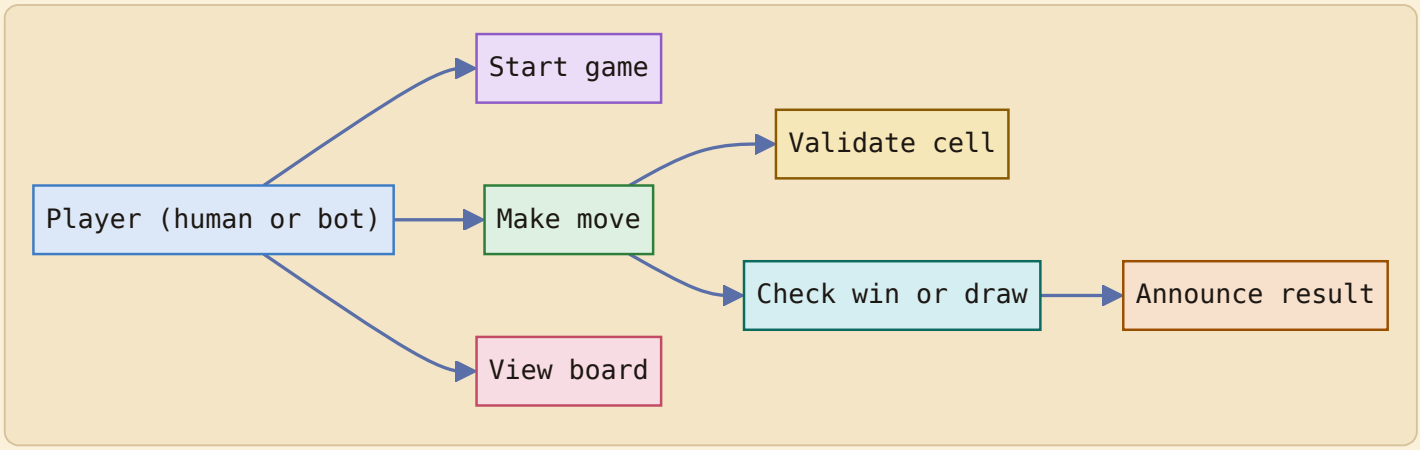
1. Create an N×N board and a game with two (or more) players, each with a unique mark.
2. Players take turns; the current player chooses an empty cell.
3. Validate moves: inside the board and empty.
4. After each move, detect a **win** (the mover completed a line) or a **draw** (board full).
5. Announce the result and show the board.

Non-functional

- Win check in **O(1)** per move (not a full $O(N^2)$ board scan).
- New player types (AI levels, network player) without touching `Game` (Open/Closed).
- Deterministic tests: random bots take a seeded `Random`; scripted players replay fixed moves.
- Separate games must not share state (many games can run at once on a server).

Use cases

Actors: **Player** (human or bot). Use cases: *start game* (choose N and players), *make move*, *view board*, *see result*. A system actor (the `Game`) validates moves, switches turns and declares the result.

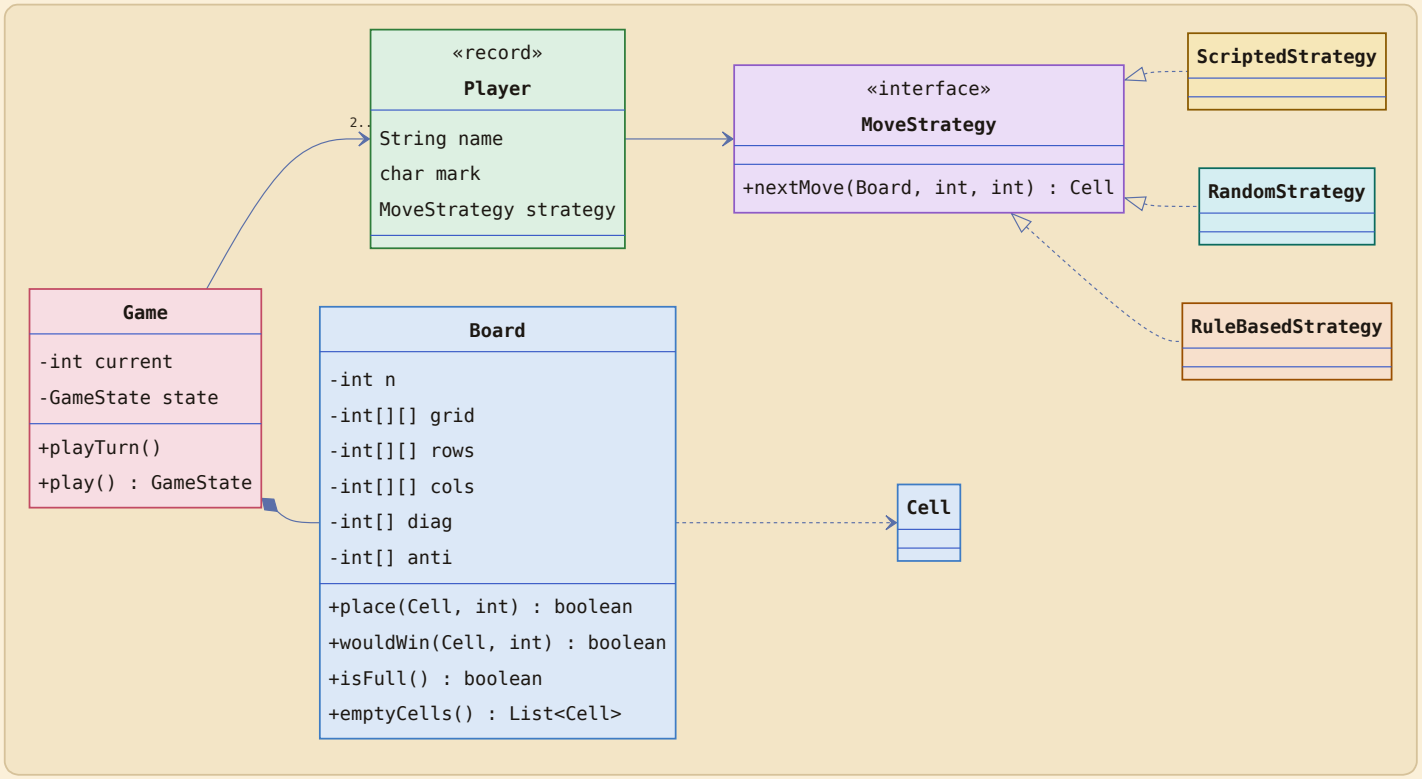


Core entities

Class	Responsibility
`Cell` (record)	Row + column of a square.
`Board`	The grid plus **per-player line counters**; `place` returns "did this win?" in O(1); `wouldWin` for bots; `isFull`, `emptyCells`, `render`.
`MoveStrategy` (interface)	Chooses the next cell. Implementations: `ScriptedStrategy`, `RandomStrategy`, `RuleBasedStrategy`.
`Player` (record)	Name, mark, strategy.
`GameState` (enum)	IN_PROGRESS, WON, DRAW.
`Game`	Turn order, move validation, state, winner, result.

Class diagram

There is no course lesson for this problem; it comes from the *awesome-low-level-design* problem set. That statement asks for: a 3×3 grid; two players taking turns with X and O; three in a row, column or diagonal wins; a full board with no winner is a draw; a way to display the grid and take moves; turn handling and move validation; announcing the winner or the draw. All of these are covered (the board size is generalised to N×N).



Key flows

O(1) win detection — how it works

A naive win check scans every row, column and diagonal after each move: O(N²). But a move at `(r, c)` can only complete **row r, column c, and (maybe) the two diagonals**. So we keep counters:

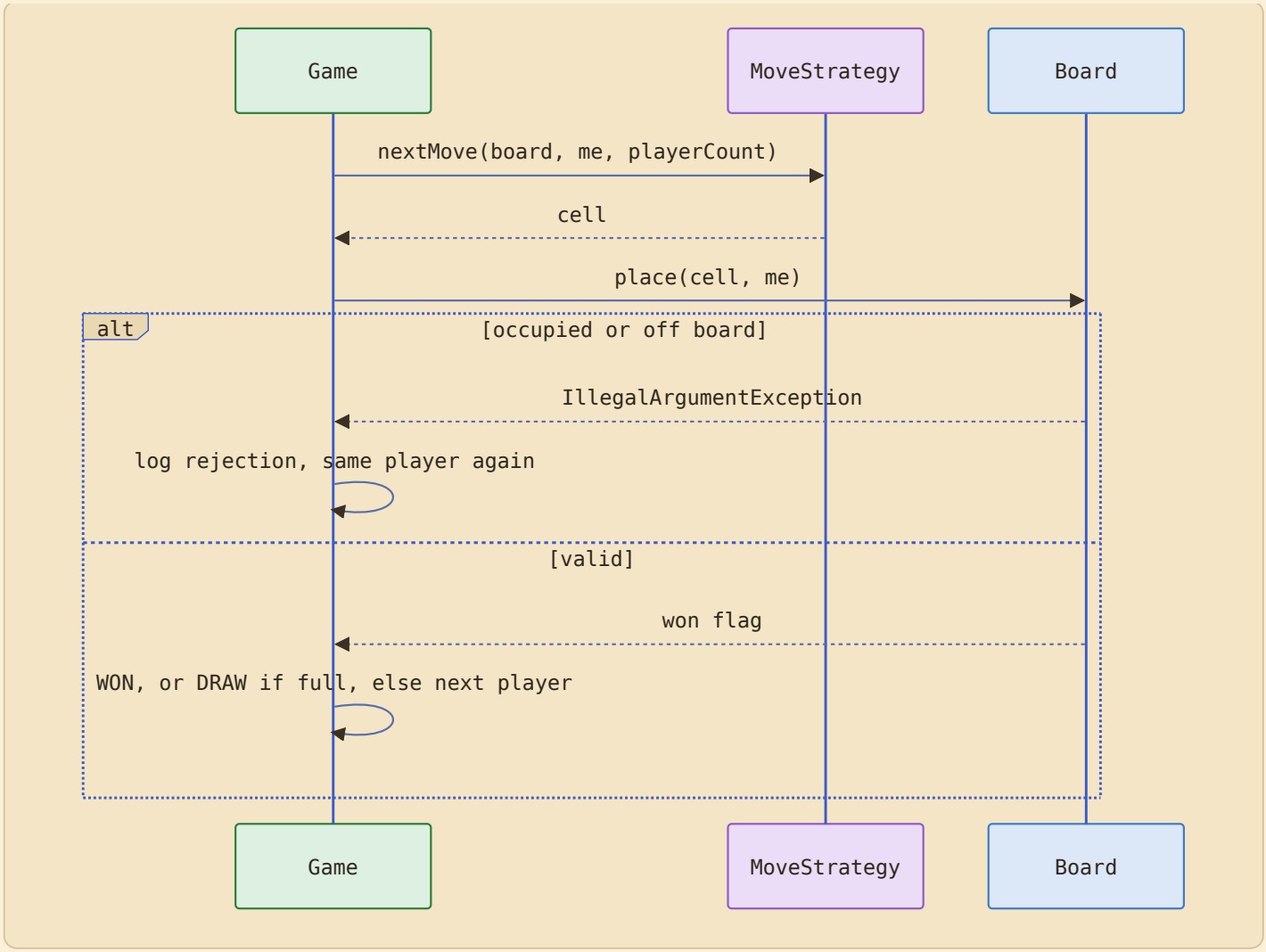
1. `rows[p][r]` = how many marks player `p` has in row `r`; `cols[p][c]` likewise.
2. `diag[p]` counts marks on the main diagonal (`r == c`); `anti[p]` on the anti-diagonal (`r + c == N-1`).
3. When player `p` places at `(r, c)`, increment the counters that cell belongs to.
4. If any of them reaches **N**, player `p` owns the whole line — a win. Because only `p`'s marks increment `p`'s counters, no check for opponents' marks is needed.

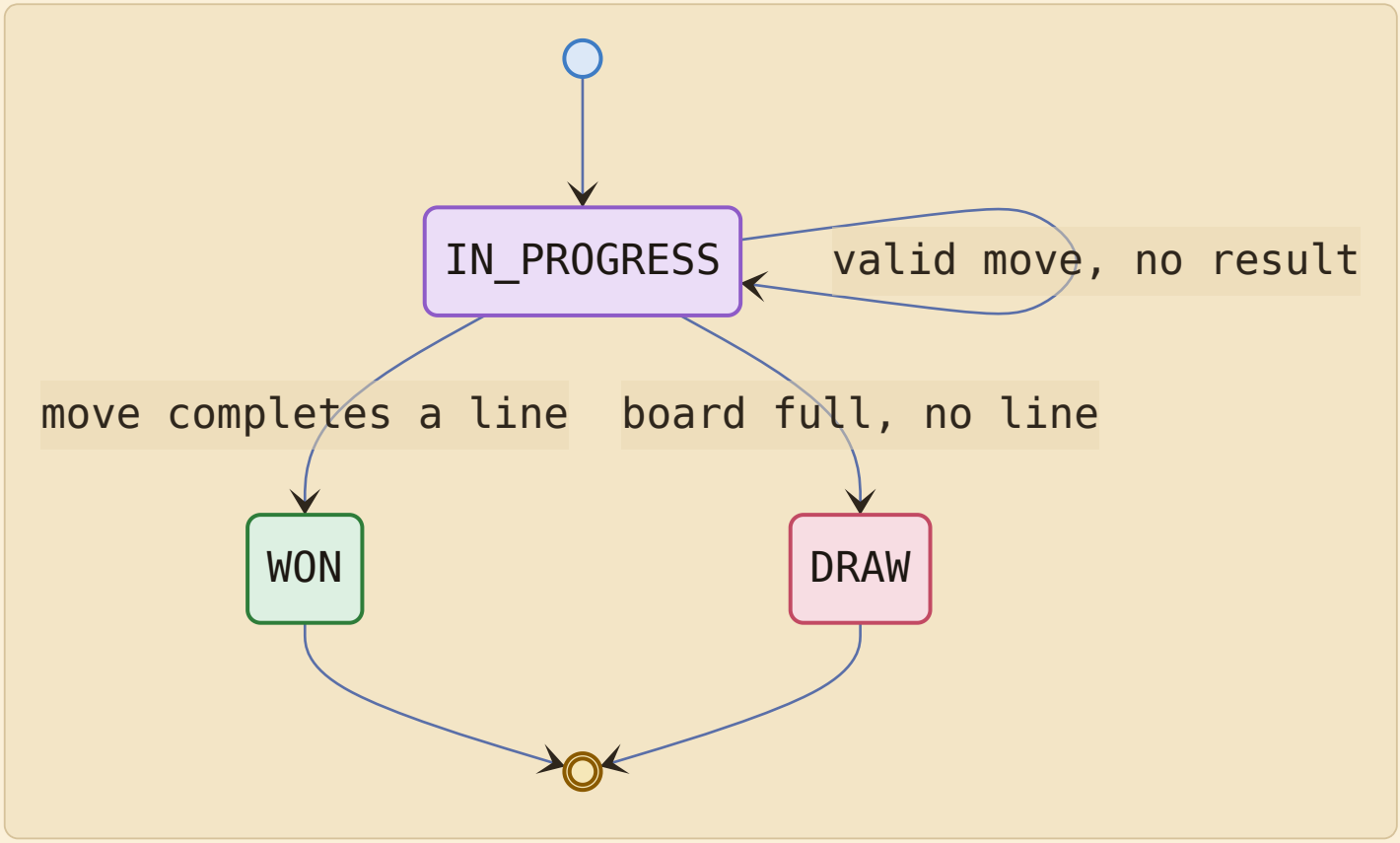
Four increments and four comparisons: **O(1) time**, **O(P·N) memory** for P players.

> **The 2-player trick you may see online:** one shared counter per line, +1 for X and −1 for O; `|count| == N` means a win. It uses half the memory but only works for exactly two players. Our per-player counters work for any number of players.

The same counters give a free **"would this move win?"** check: an empty cell wins for `p` if any of its lines already has `N-1` of `p`'s marks. The rule-based bot uses it to *win* if possible and *block* the opponent otherwise.

One turn


Game
MoveStrategy
Board
nextMove(board, me, playerCount)
cell
place(cell, me)
alt
[occupied or off board]
IllegalArgumentException
log rejection, same player again
[valid]
won flag
WON, or DRAW if full, else next player
Game
MoveStrategy
Board


IN_PROGRESS
valid move, no result
move completes a line
board full, no line
WON
DRAW

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for player moves.	Human, random, rule-based, network or minimax players plug in; `Game` never changes.	`if (player.isComputer())` branches in `Game`.
Counters inside `Board` (not a separate "WinChecker" that scans).	O(1) check, kept next to the data it summarises.	Scanning all lines each move — O(N²) and duplicated for "would win".
Board returns "won?" from `place`.	The only moment a win can appear is right after a move.	Separate `checkWinner()` that the caller might forget to call.
Invalid move → exception from `Board`, caught by `Game`.	The board protects its invariant (a fact that must always stay true, e.g. one mark per cell); the game decides the policy (retry).	Silently ignoring a bad move.
Seeded `Random` injected into `RandomStrategy`.	Repeatable demos/tests.	`Math.random()` inside the bot.
Players as a list + index `current = (current+1) % size` (`%` is remainder, so the index wraps back to 0).	Works for any number of players.	Hard-coded X/O toggling.

Java implementation

Package `lld.ch17.tictactoe`.

```java
// File: code/lld/ch17/tictactoe/Cell.java
package lld.ch17.tictactoe;

public record Cell(int row, int col) {
    @Override public String toString() { return "(" + row + "," + col + ")"; }
}
```

```java
// File: code/lld/ch17/tictactoe/Board.java
package lld.ch17.tictactoe;

import java.util.ArrayList;
import java.util.List;

/**
 * N x N grid plus per-player line counters, so "did that move win?" is O(1) instead of O(N^2).
 * rows[p][r] = how many marks player p has in row r. Same idea for columns and the two diagonals.
 * A line is won when one player's count reaches N (other players' marks make that impossible).
 */
public final class Board {
    private final int n;
    private final int[][] grid;       // -1 empty, otherwise player index
    private final int[][] rows, cols;
    private final int[] diag, anti;
    private int filled;

    public Board(int n, int players) {
        if (n < 3) throw new IllegalArgumentException("board must be at least 3x3");
        this.n = n;
        grid = new int[n][n];
        for (int[] row : grid) java.util.Arrays.fill(row, -1);
        rows = new int[players][n];
        cols = new int[players][n];
        diag = new int[players];
        anti = new int[players];
    }

    public int size() { return n; }

    public boolean isEmpty(Cell c) { return inside(c) && grid[c.row()][c.col()] == -1; }

    private boolean inside(Cell c) { return c.row() >= 0 && c.row() < n && c.col() >= 0 && c.col() < n; }

    /** Places a mark and returns true if this move completed a line. O(1). */
    boolean place(Cell c, int player) {
        if (!inside(c)) throw new IllegalArgumentException("cell " + c + " is off the board");
        if (grid[c.row()][c.col()] != -1) throw new IllegalArgumentException("cell " + c + " is already taken");
        grid[c.row()][c.col()] = player;
        filled++;
        int r = c.row(), k = c.col();
        boolean win = ++rows[player][r] == n;
        win |= ++cols[player][k] == n;
        if (r == k) win |= ++diag[player] == n;
        if (r + k == n - 1) win |= ++anti[player] == n;
        return win;
    }

    /** Would placing player's mark on this empty cell win? Also O(1): a line needs exactly one more mark. */
    public boolean wouldWin(Cell c, int player) {
        int r = c.row(), k = c.col();
        return rows[player][r] == n - 1 || cols[player][k] == n - 1
                || (r == k && diag[player] == n - 1) || (r + k == n - 1 && anti[player] == n - 1);
    }

    public boolean isFull() { return filled == n * n; }

    public List<Cell> emptyCells() {
        List<Cell> out = new ArrayList<>();
        for (int r = 0; r < n; r++) for (int c = 0; c < n; c++) if (grid[r][c] == -1) out.add(new Cell(r, c));
        return out;
    }

    public String render(char[] marks) {
        StringBuilder sb = new StringBuilder();
        for (int r = 0; r < n; r++) {
            sb.append("    ");
```

```
            for (int c = 0; c < n; c++) sb.append(grid[r][c] == -1 ? '.' : marks[grid[r][c]]).append(c < n - 1 ? "
" : "");
            if (r < n - 1) sb.append('\n');
        }
        return sb.toString();
    }
}
```

```
// File: code/lld/ch17/tictactoe/MoveStrategy.java
package lld.ch17.tictactoe;

/** Strategy pattern: human input, scripted test moves, random bot, rule-based bot... all plug in here. */
public interface MoveStrategy {
    Cell nextMove(Board board, int me, int playerCount);
}
```

```
// File: code/lld/ch17/tictactoe/ScriptedStrategy.java
package lld.ch17.tictactoe;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;

/** Plays a fixed list of moves. Stands in for a human in tests. */
public final class ScriptedStrategy implements MoveStrategy {
    private final Deque<Cell> moves;

    public ScriptedStrategy(List<Cell> moves) { this.moves = new ArrayDeque<>(moves); }

    @Override public Cell nextMove(Board board, int me, int playerCount) { return moves.poll(); }
}
```

```
// File: code/lld/ch17/tictactoe/RandomStrategy.java
package lld.ch17.tictactoe;

import java.util.List;
import java.util.Random;

/** Picks any empty cell. Takes a seeded Random so games are repeatable. */
public final class RandomStrategy implements MoveStrategy {
    private final Random rng;

    public RandomStrategy(Random rng) { this.rng = rng; }

    @Override public Cell nextMove(Board board, int me, int playerCount) {
        List<Cell> empty = board.emptyCells();
        return empty.get(rng.nextInt(empty.size()));
    }
}
```

```java
// File: code/lld/ch17/tictactoe/RuleBasedStrategy.java
package lld.ch17.tictactoe;

import java.util.List;

/** Win if you can, else block an opponent's win, else take the centre, else the first corner, else anything. */
public final class RuleBasedStrategy implements MoveStrategy {
    @Override public Cell nextMove(Board board, int me, int playerCount) {
        List<Cell> empty = board.emptyCells();
        for (Cell c : empty) if (board.wouldWin(c, me)) return c;
        for (int other = 0; other < playerCount; other++) {
            if (other == me) continue;
            for (Cell c : empty) if (board.wouldWin(c, other)) return c;
        }
        int n = board.size();
        Cell centre = new Cell(n / 2, n / 2);
        if (board.isEmpty(centre)) return centre;
        for (Cell corner : List.of(new Cell(0, 0), new Cell(0, n - 1), new Cell(n - 1, 0), new Cell(n - 1, n - 1)))
{
            if (board.isEmpty(corner)) return corner;
        }
        return empty.get(0);
    }
}
```

```java
// File: code/lld/ch17/tictactoe/Player.java
package lld.ch17.tictactoe;

public record Player(String name, char mark, MoveStrategy strategy) {}
```

```java
// File: code/lld/ch17/tictactoe/GameState.java
package lld.ch17.tictactoe;

public enum GameState { IN_PROGRESS, WON, DRAW }
```

```java
// File: code/lld/ch17/tictactoe/Game.java
package lld.ch17.tictactoe;

import java.util.List;
import java.util.function.Consumer;

/** Turn management and game state. Supports 2+ players on an N x N board. */
public final class Game {
    private final Board board;
    private final List<Player> players;
    private final char[] marks;
    private final Consumer<String> log;
    private int current;
    private GameState state = GameState.IN_PROGRESS;
    private Player winner;

    public Game(int n, List<Player> players, Consumer<String> log) {
        if (players.size() < 2) throw new IllegalArgumentException("need at least 2 players");
        this.players = List.copyOf(players);
        this.board = new Board(n, players.size());
        this.marks = new char[players.size()];
        for (int i = 0; i < marks.length; i++) marks[i] = players.get(i).mark();
        this.log = log;
    }

    /** Plays one turn for the current player. An invalid cell is rejected and the same player goes again. */
    public void playTurn() {
        if (state != GameState.IN_PROGRESS) throw new IllegalStateException("game is over");
        Player p = players.get(current);
        Cell cell = p.strategy().nextMove(board, current, players.size());
        boolean won;
        try {
            won = board.place(cell, current);
        } catch (IllegalArgumentException e) {
            log.accept("  " + p.name() + " tried " + cell + ": rejected (" + e.getMessage() + "), try again");
            return;
        }
        log.accept("  " + p.name() + " (" + p.mark() + ") -> " + cell);
        if (won) { state = GameState.WON; winner = p; }
        else if (board.isFull()) state = GameState.DRAW;
        else current = (current + 1) % players.size();
    }

    public GameState play() {
        while (state == GameState.IN_PROGRESS) playTurn();
        log.accept(board.render(marks));
        log.accept("  Result: " + (state == GameState.WON ? winner.name() + " wins" : "draw"));
        return state;
    }
}
```

```java
// File: code/lld/ch17/tictactoe/Main.java
package lld.ch17.tictactoe;

import java.util.List;
import java.util.Random;

public final class Main {
    public static void main(String[] args) {
        System.out.println("== 1. 3x3 scripted game (with one invalid move) ==");
        Player x = new Player("Xena", 'X', new ScriptedStrategy(List.of(
                new Cell(0, 0), new Cell(1, 1), new Cell(2, 2))));
        Player o = new Player("Omar", 'O', new ScriptedStrategy(List.of(
                new Cell(0, 0), new Cell(0, 1), new Cell(0, 2))));   // first one is already taken
        new Game(3, List.of(x, o), System.out::println).play();

        System.out.println("\n== 2. 3x3 rule-based bot vs seeded random bot (seed 7) ==");
        new Game(3, List.of(new Player("Bot", 'X', new RuleBasedStrategy()),
                new Player("Rand", 'O', new RandomStrategy(new Random(7)))), System.out::println).play();

        System.out.println("\n== 3. 4x4 rule-based vs rule-based ==");
        new Game(4, List.of(new Player("A", 'X', new RuleBasedStrategy()),
                new Player("B", 'O', new RuleBasedStrategy())), line -> { if (!line.startsWith("  A") &&
!line.startsWith("  B")) System.out.println(line); }).play();

        System.out.println("\n== 4. 5x5 with three players (two random, seeds 1 and 2; one rule-based) ==");
        new Game(5, List.of(new Player("P1", 'X', new RandomStrategy(new Random(1))),
                new Player("P2", 'O', new RandomStrategy(new Random(2))),
                new Player("P3", 'Z', new RuleBasedStrategy())), line -> { if (!line.startsWith("  P"))
System.out.println(line); }).play();
    }
}
```

Output:

```
== 1. 3x3 scripted game (with one invalid move) ==
  Xena (X) -> (0,0)
  Omar tried (0,0): rejected (cell (0,0) is already taken), try again
  Omar (O) -> (0,1)
  Xena (X) -> (1,1)
  Omar (O) -> (0,2)
  Xena (X) -> (2,2)
    X O O
    . X .
    . . X
  Result: Xena wins

== 2. 3x3 rule-based bot vs seeded random bot (seed 7) ==
  Bot (X) -> (1,1)
  Rand (O) -> (2,0)
  Bot (X) -> (0,0)
  Rand (O) -> (1,0)
  Bot (X) -> (2,2)
    X . .
    O X .
    O . X
  Result: Bot wins

== 3. 4x4 rule-based vs rule-based ==
    O O X X
    O O X X
    X X X O
    O O O X
  Result: draw

== 4. 5x5 with three players (two random, seeds 1 and 2; one rule-based) ==
    Z Z Z O O
    X O X Z X
    X X Z O O
    Z O O X O
    Z X X X Z
  Result: draw
```

Reading the output. Game 1 shows the retry after an invalid move and a diagonal win found by the O(1) counters. Game 2: the rule-based bot takes the centre, then a corner, and completes the diagonal against a seeded random bot (no block was needed: the random bot's two O's in column 0 were already blocked by the X in the corner). Game 3: two rule-based bots on 4×4 draw (the move lines are filtered out of the log for brevity). Game 4 runs the same counters with three players on 5×5 (a draw).

Concurrency & edge cases

- **Many games on one server:** each `Game` owns its own `Board`, so games share nothing. Keep them in a `ConcurrentHashMap<gameId, Game>` (a hash map that many threads can read and write safely at once) and process each game's moves one at a time (per-game lock or single-threaded actor).
- **Two clicks at once from the same game:** make `playTurn`/a `makeMove(player, cell)` method `synchronized` and check that the caller is the current player.
- **Edge cases:** move after game over (rejected with `IllegalStateException`, Java's standard "not allowed right now" error); off-board or occupied cell (rejected, same player retries); a scripted strategy running out of moves (returns `null` → fail fast, i.e. stop with a clear error at once, in tests); N = 3 minimum.
- **A cell on both diagonals** (the centre of an odd board) updates both diagonal counters — handled by two independent `if`s.

Extensions & follow-up questions

- **"Win with K in a row on an N×N board (Gomoku, K = 5)?"** — Counters no longer work because lines are shorter than the board. After each move, walk out from the new cell in the 4 directions (horizontal, vertical, two diagonals) and count consecutive own marks: O(K) per move.
- **"Unbeatable AI?"** — Minimax over the game tree (every possible sequence of moves drawn as a branching tree; tic-tac-toe 3×3 has at most 9! = 362,880 move orders, easily searched); add alpha-beta pruning or memoisation

(caching the result for each position already evaluated) for larger boards. Plug it in as another `MoveStrategy`.

- **"Undo?"** — Store moves in a stack (last in, first out); undo = clear the cell and **decrement** the counters (counters are reversible, unlike a boolean "won" flag).
- **"Online multiplayer?"** — Server holds the `Game`; clients send `makeMove(gameId, playerId, cell)`; the server broadcasts the new state (Observer).
- **"Replay a game?"** — The move list plus `ScriptedStrategy` replays it exactly.

Real-world example

Google Search has a built-in tic-tac-toe game (search "tic tac toe") with difficulty levels — different computer players behind the same game rules, which is the Strategy split in this design. Tic-tac-toe is also the classic first example for the minimax algorithm in AI courses, because the full game tree is small enough to search completely.

In the interview

> **30-second answer:** "`Board` is N×N plus per-player counters for each row, column and the two diagonals. Placing a mark bumps at most four counters; if one hits N, that player wins — O(1) per move. `Game` holds a list of players and a turn index, validates moves, and ends on a win or a full board. Each `Player` has a `MoveStrategy` — human, random with a seeded `Random`, or rule-based that wins if it can and blocks otherwise."

Time plan: 5 min requirements and N×N/players → 5 min entities → 10 min the counter idea (draw it!) → 15 min code → 5 min extensions (K-in-a-row, minimax, undo).

Self-check

1. **Q:** Why does a move only need to check four lines? **A:** A cell belongs to exactly one row, one column, and at most the two diagonals.
2. **Q:** How do you know a cell is on the anti-diagonal? **A:** `row + col == N - 1`.
3. **Q:** Why per-player counters instead of +1/−1? **A:** They work for more than two players and give "would win" for any player.
4. **Q:** How is a draw detected? **A:** No win and the number of filled cells equals N^2.
5. **Q:** How do you test a bot that plays randomly? **A:** Inject `new Random(seed)`; the game is then identical every run.

TL;DR

- Per-player row/column/diagonal counters → O(1) win check; `N-1` → O(1) "would win".
- `Game` = players list + turn index + state enum; invalid moves rejected, same player retries.
- Players via `MoveStrategy` (human, random-seeded, rule-based, minimax later).
- K-in-a-row needs a directional walk from the last move instead.

Design Snake and Ladder

Problem & clarifying questions

Snake and Ladder is a race on a numbered board (usually 1–100). Players take turns rolling a die and move forward by the number rolled. Landing on the **foot of a ladder** takes you up to its top; landing on the **head of a snake** sends you down to its tail. The first to reach the last square wins. There are no decisions — everything is dice and rules — so the design is about a **clean rules engine**, **turn management** and **testable randomness**.

Question	Assumed answer
Board size and snakes/ladders?	Configurable (default 100 squares, a given list of snakes and ladders). Validate the config.
How many dice?	Configurable: count and faces (default one six-sided die).
How many players?	Two or more; all start off the board (square 0).
Overshooting the last square?	Configurable: **exact finish** (stay put) or **bounce back**.
Rolling a six?	Common house rule (an optional rule some players add): roll again; three sixes in a row cancels the whole turn. Configurable on/off.
When does the game end?	Configurable: **first to finish** wins, or keep playing to **rank everyone**.
Several games at once?	Yes, independent sessions (e.g. many groups online).

Requirements

Functional

1. A board of numbered cells (typically 100) with a predefined set of snakes and ladders connecting cells.
2. Multiple players, each with one token, starting off the board.
3. Players take turns rolling the dice and move forward by the total.
4. Landing on a snake head slides the token down to its tail; landing on a ladder foot climbs to its top.
5. Handle overshooting the final square (exact-finish or bounce-back rule).
6. Optional bonus roll on a six, with the three-sixes penalty.
7. The game continues until the win strategy says it is over (first to finish, or everyone ranked).
8. Support many independent game sessions at once.

Non-functional

- **Deterministic** dice for tests (seeded or loaded).
- Rules are **pluggable** without editing the game loop.
- Invalid boards are rejected **at construction**, not discovered mid-game.

Use cases

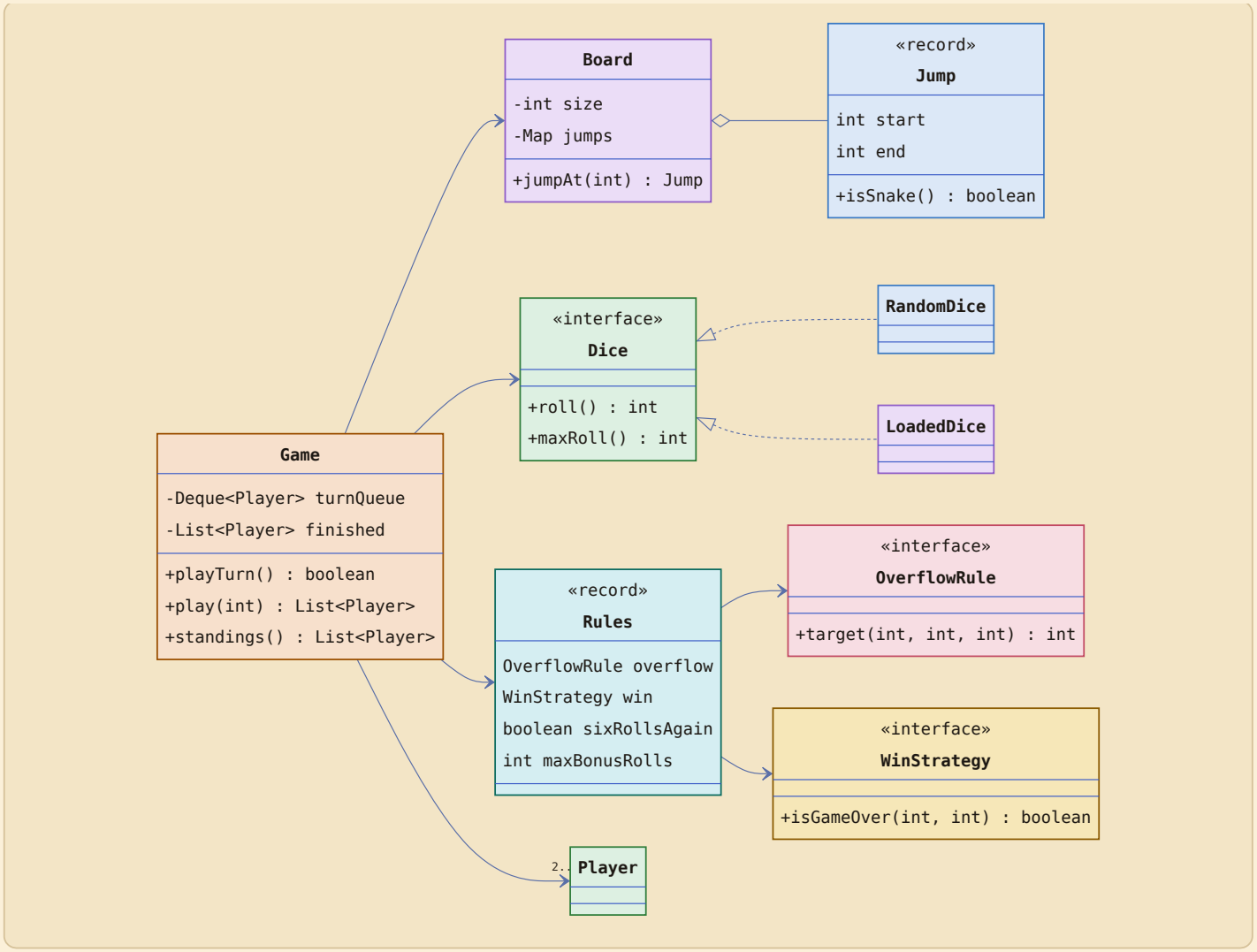
Actors: **Player** (rolls the dice — or the system rolls for them), **Game admin/host** (configures board, dice and rules, starts a game). Use cases: *configure board*, *add players*, *start game*, *take turn* (roll → move → apply snake/ladder → maybe bonus roll), *declare winner / standings*.

Core entities

Class	Responsibility
`Jump` (record)	A snake or a ladder: `start → end`. One class, because the behaviour is identical ("land on start, move to end"); `isSnake()` is just `end < start`.
`Board`	Size + `Map<start, Jump>` for O(1) lookup; validates configuration.
`Dice` (interface)	`roll()`, `maxRoll()`. `RandomDice` (count, faces, seeded `Random`), `LoadedDice` (scripted rolls for tests).
`OverflowRule` (functional interface — exactly one method, so a lambda, a short unnamed function like `x -> x + 1`, can implement it)	Where a roll past the end lands: `exactFinish()` or `bounceBack()`.
`WinStrategy` (functional interface)	When the game ends: `firstToFinish()` or `rankAll()`.
`Rules` (record)	Bundles a variant: overflow rule, win strategy, six-rolls-again flag, max bonus rolls.
`Player`	Name and position.
`Game`	Turn queue, `playTurn()`, `play(maxTurns)`, standings, event log.

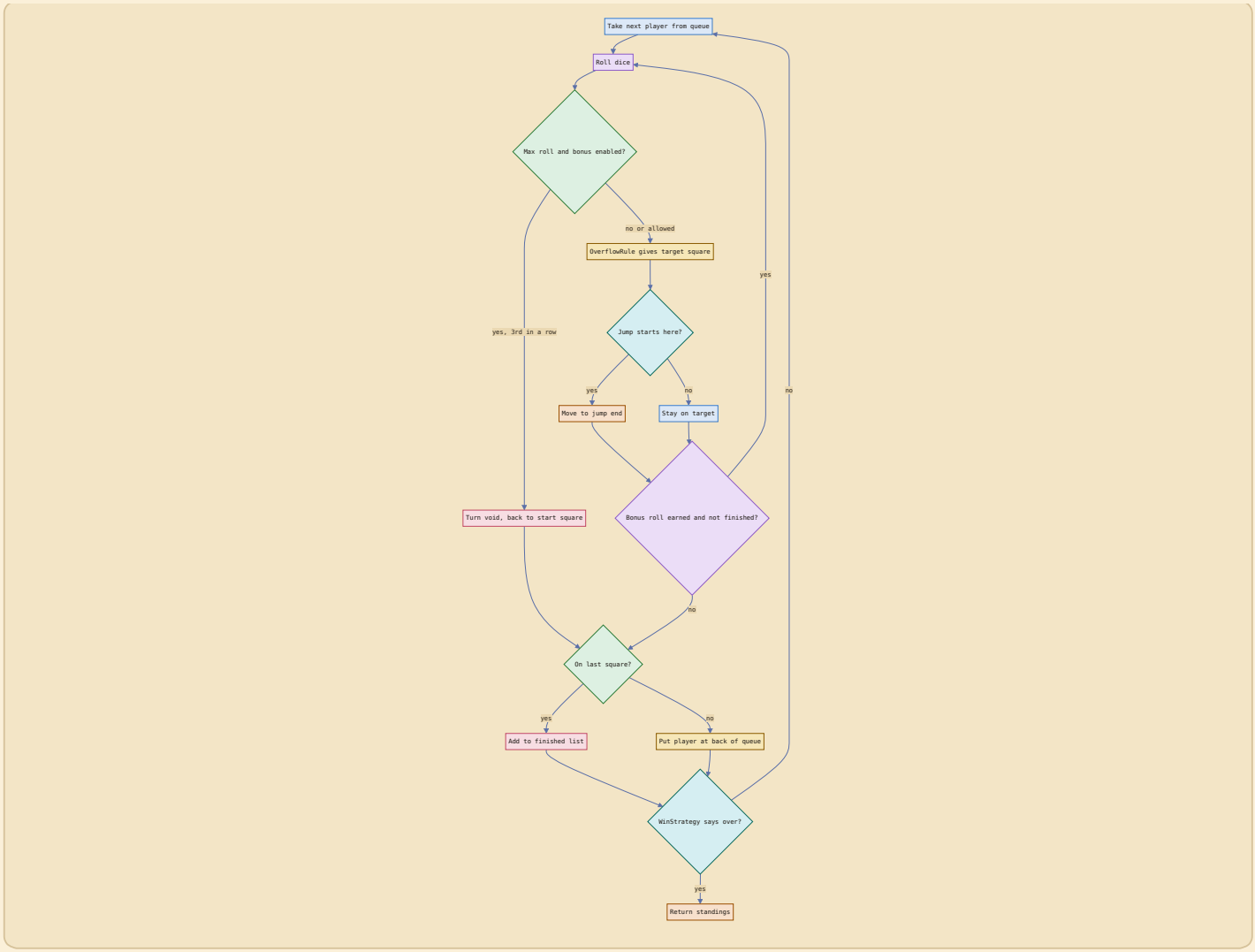
Class diagram

There is no course lesson for this problem; it comes from the *awesome-low-level-design* problem set. That statement asks for: a board of numbered cells (usually 100); predefined snakes and ladders; several players, each with a token; turn-based dice rolls; sliding down a snake's head to its tail and climbing a ladder's foot to its top; a win on reaching the last cell; and many independent game sessions running at once (its reference solution uses a singleton — a class with only one global instance — `GameManager` with one thread per session). All of these are covered; part 6 of the output runs eight sessions in parallel.



Key flows

One turn



Board validation (done once, in the constructor)

1. A jump must not start and end on the same square.
2. No jump starts on square 1 or the last square (you could never win if the last square were a snake).
3. No two jumps share a start square.
4. A jump may not end on another jump's start. This forbids chains, and therefore also forbids infinite loops (snake A → ladder B → snake A...).

Design decisions & patterns used

Decision	Why	Alternative rejected
One `Jump` type for snakes and ladders.	Identical behaviour; the direction is data.	`Snake` and `Ladder` classes with duplicate code — only worth it if they behave differently.
`Map<Integer, Jump>` lookup.	O(1) per move instead of scanning a list.	Array of size 101 — also fine; a map scales to sparse or huge boards.
Strategy for `Dice`, `OverflowRule`, `WinStrategy`.	House rules vary a lot; each is swappable. Lambdas keep them tiny.	Flags and `if`s in the game loop.
`Rules` record groups a variant.	One object = one rule set ("classic", "kids", "tournament"); easy to pass around.	Many constructor parameters on `Game`.
Queue of players.	Natural turn order; finished players simply are not re-queued.	Index arithmetic that must skip finished players.
Event log via `Consumer<String>` (a built-in "takes a value, returns nothing" function type).	The engine does not print; the caller decides (console, UI, nothing). A full design would use **Observer** (`GameListener.onMove/onSnake/onLadder/onWin`).	`System.out.println` inside the engine.
Validation in the `Board` constructor (fail fast).	The loop can trust the board.	Detecting a loop mid-game.
`play(maxTurns)` guard.	A misconfigured rule set can never hang (loop forever and never finish) a server thread.	An unbounded `while (true)`.

Java implementation

Package `lld.ch17.snakeladder`.

```java
// File: code/lld/ch17/snakeladder/Jump.java
package lld.ch17.snakeladder;

/** A snake (goes down) or a ladder (goes up). Both are just "land on start -> move to end". */
public record Jump(int start, int end) {
    public boolean isSnake() { return end < start; }
    public String kind() { return isSnake() ? "snake" : "ladder"; }
}
```

```java
// File: code/lld/ch17/snakeladder/Board.java
package lld.ch17.snakeladder;

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;

/**
 * Board configuration: size plus a map start -> Jump for O(1) lookup.
 * Validates the config once, up front, so the game loop can trust it.
 */
public final class Board {
    private final int size;
    private final Map<Integer, Jump> jumps = new HashMap<>();

    public Board(int size, Collection<Jump> all) {
        if (size < 10) throw new IllegalArgumentException("board too small");
        this.size = size;
        for (Jump j : all) {
            if (j.start() == j.end()) throw new IllegalArgumentException("jump to itself: " + j);
            if (j.start() <= 1 || j.start() >= size) throw new IllegalArgumentException("jump cannot start on
first/last square: " + j);
            if (j.end() < 1 || j.end() > size) throw new IllegalArgumentException("jump ends off board: " + j);
            if (jumps.putIfAbsent(j.start(), j) != null) throw new IllegalArgumentException("two jumps start at " +
j.start());
        }
        for (Jump j : jumps.values()) {   // no chains: an end square may not be another jump's start (also rules
out loops)
            if (jumps.containsKey(j.end())) throw new IllegalArgumentException("jump " + j + " ends on another
jump's start");
        }
    }

    public int size() { return size; }

    /** Returns the jump that starts on this square, or null. */
    public Jump jumpAt(int square) { return jumps.get(square); }
}
```

```java
// File: code/lld/ch17/snakeladder/Dice.java
package lld.ch17.snakeladder;

/** Strategy for rolling. Real dice use a seeded Random; tests use a fixed sequence. */
public interface Dice {
    int roll();
    int maxRoll();
}
```

```java
// File: code/lld/ch17/snakeladder/RandomDice.java
package lld.ch17.snakeladder;

import java.util.Random;

/** count dice with the given number of faces. Pass new Random(seed) for repeatable games. */
public final class RandomDice implements Dice {
    private final int count, faces;
    private final Random rng;

    public RandomDice(int count, int faces, Random rng) {
        this.count = count;
        this.faces = faces;
        this.rng = rng;
    }

    @Override public int roll() {
        int sum = 0;
        for (int i = 0; i < count; i++) sum += 1 + rng.nextInt(faces);
        return sum;
    }

    @Override public int maxRoll() { return count * faces; }
}
```

```java
// File: code/lld/ch17/snakeladder/LoadedDice.java
package lld.ch17.snakeladder;

/** Returns a scripted sequence of rolls (cycling). Used to test exact scenarios. */
public final class LoadedDice implements Dice {
    private final int[] rolls;
    private int i;

    public LoadedDice(int... rolls) { this.rolls = rolls.clone(); }

    @Override public int roll() { return rolls[i++ % rolls.length]; }

    @Override public int maxRoll() { return 6; }
}
```

```java
// File: code/lld/ch17/snakeladder/OverflowRule.java
package lld.ch17.snakeladder;

/** What happens when a roll would carry a token past the last square. */
@FunctionalInterface
public interface OverflowRule {
    /** Returns the new square (before snakes/ladders), given current square, roll and board size. */
    int target(int from, int roll, int size);

    /** Must land exactly on the last square, otherwise stay put. */
    static OverflowRule exactFinish() { return (from, roll, size) -> from + roll > size ? from : from + roll; }

    /** Bounce back by the extra amount (at 98 on a 100 board, a 5 goes to 100 then back to 97). */
    static OverflowRule bounceBack() { return (from, roll, size) -> from + roll > size ? size - (from + roll -
size) : from + roll; }
}
```

```java
// File: code/lld/ch17/snakeladder/WinStrategy.java
package lld.ch17.snakeladder;

/** When is the whole game over? */
@FunctionalInterface
public interface WinStrategy {
    boolean isGameOver(int finishedPlayers, int totalPlayers);

    /** Classic: the first player to reach the end wins and the game stops. */
    static WinStrategy firstToFinish() { return (finished, total) -> finished >= 1; }

    /** Play on to rank everybody; stop when only one player is left on the board. */
    static WinStrategy rankAll() { return (finished, total) -> finished >= total - 1; }
}
```

```java
// File: code/lld/ch17/snakeladder/Player.java
package lld.ch17.snakeladder;

public final class Player {
    private final String name;
    private int position;     // 0 = off the board, waiting to enter

    public Player(String name) { this.name = name; }

    public String name() { return name; }
    public int position() { return position; }
    void moveTo(int square) { position = square; }
}
```

```java
// File: code/lld/ch17/snakeladder/Rules.java
package lld.ch17.snakeladder;

/**
 * The rules engine's configuration, bundled so a game variant is one object.
 * sixRollsAgain: rolling the dice's maximum earns another roll; maxBonusRolls in a row cancels the turn.
 */
public record Rules(OverflowRule overflow, WinStrategy win, boolean sixRollsAgain, int maxBonusRolls) {
    public static Rules classic() { return new Rules(OverflowRule.exactFinish(), WinStrategy.firstToFinish(), true,
3); }
}
```

```java
// File: code/lld/ch17/snakeladder/Game.java
package lld.ch17.snakeladder;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.List;
import java.util.function.Consumer;

/** Turn loop: a queue of players, a dice strategy, a board and a rule set. */
public final class Game {
    private final Board board;
    private final Dice dice;
    private final Rules rules;
    private final Deque<Player> turnQueue;
    private final int totalPlayers;
    private final List<Player> finished = new ArrayList<>();
    private final Consumer<String> log;

    public Game(Board board, Dice dice, Rules rules, List<Player> players, Consumer<String> log) {
        if (players.size() < 2) throw new IllegalArgumentException("need at least 2 players");
        this.board = board;
        this.dice = dice;
        this.rules = rules;
        this.turnQueue = new ArrayDeque<>(players);
        this.totalPlayers = players.size();
        this.log = log;
    }

    public boolean isOver() { return rules.win().isGameOver(finished.size(), totalPlayers); }

    /** Plays one player's turn (including bonus rolls). Returns true if the game is now over. */
    public boolean playTurn() {
        if (isOver()) throw new IllegalStateException("game is over");
        Player p = turnQueue.poll();
        takeTurn(p);
        if (p.position() == board.size()) {
            finished.add(p);
            log.accept("  *** " + p.name() + " finishes in place " + finished.size() + " ***");
        } else {
            turnQueue.offer(p);
        }
        return isOver();
    }

    /** Plays until the win strategy says stop. Returns the standings (finishers first, in order). */
    public List<Player> play(int maxTurns) {
        for (int turns = 0; !isOver(); turns++) {
            if (turns == maxTurns) throw new IllegalStateException("no result after " + maxTurns + " turns");
            playTurn();
        }
        return standings();
    }

    public List<Player> standings() {
        List<Player> order = new ArrayList<>(finished);
        order.addAll(turnQueue);          // whoever is still on the board, in turn order
        return order;
    }

    private void takeTurn(Player p) {
        int start = p.position();
        int bonus = 0;
        StringBuilder line = new StringBuilder("  " + p.name() + " @" + start + ":");
        while (true) {
            int roll = dice.roll();
            line.append(" rolls ").append(roll);
            boolean max = rules.sixRollsAgain() && roll == dice.maxRoll();
```

```
            if (max && ++bonus == rules.maxBonusRolls()) {
                p.moveTo(start);                        // too many sixes in a row: the whole turn is cancelled
                line.append(" -> ").append(rules.maxBonusRolls()).append(" sixes in a row, turn void, back to
").append(start);
                break;
            }
            move(p, roll, line);
            if (!max || p.position() == board.size()) break;
            line.append(", bonus roll:");
        }
        log.accept(line.toString());
    }

    private void move(Player p, int roll, StringBuilder line) {
        int target = rules.overflow().target(p.position(), roll, board.size());
        if (target == p.position()) { line.append(" -> needs exact roll, stays at ").append(target); return; }
        line.append(" -> ").append(target);
        Jump j = board.jumpAt(target);
        if (j != null) {
            target = j.end();
            line.append(" (").append(j.kind()).append(" to ").append(target).append(')');
        }
        p.moveTo(target);
    }
}
```

```java
// File: code/lld/ch17/snakeladder/Main.java
package lld.ch17.snakeladder;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.Random;
import java.util.stream.Collectors;

public final class Main {
    private static final List<Jump> JUMPS = List.of(
            new Jump(4, 14), new Jump(9, 31), new Jump(21, 42), new Jump(28, 84), new Jump(51, 67),
            new Jump(71, 91), new Jump(80, 99),                                    // ladders
            new Jump(17, 7), new Jump(54, 34), new Jump(62, 19), new Jump(64, 60),
            new Jump(87, 24), new Jump(93, 73), new Jump(95, 75), new Jump(98, 79));  // snakes

    public static void main(String[] args) throws Exception {
        Board board = new Board(100, JUMPS);

        System.out.println("== 1. Loaded dice: ladder, bonus roll, snake, three sixes ==");
        Game g1 = new Game(board, new LoadedDice(4, 6, 3, 3, 6, 6, 6, 2), Rules.classic(),
                List.of(new Player("Ann"), new Player("Bob")), System.out::println);
        for (int i = 0; i < 4; i++) g1.playTurn();

        System.out.println("\n== 2. Overflow rules: token on 97 of 100, rolls a 5 ==");
        Board plain = new Board(100, List.of());
        for (String variant : List.of("Exact", "Bounce")) {
            OverflowRule rule = variant.equals("Exact") ? OverflowRule.exactFinish() : OverflowRule.bounceBack();
            Player p = new Player(variant);
            p.moveTo(97);
            Game g = new Game(plain, new LoadedDice(5), new Rules(rule, WinStrategy.firstToFinish(), false, 3),
                    List.of(p, new Player("Q")), System.out::println);
            g.playTurn();
        }

        System.out.println("\n== 3. Bad configurations are rejected up front ==");
        for (List<Jump> bad : List.of(List.of(new Jump(100, 5)), List.of(new Jump(10, 30), new Jump(30, 5)))) {
            try { new Board(100, bad); }
            catch (IllegalArgumentException e) { System.out.println("  rejected: " + e.getMessage()); }
        }

        System.out.println("\n== 4. Seeded game (seed 2024), 3 players, first to finish wins ==");
        Game g4 = new Game(board, new RandomDice(1, 6, new Random(2024)), Rules.classic(),
                List.of(new Player("Ann"), new Player("Bob"), new Player("Cid")), System.out::println);
        System.out.println("  Winner: " + g4.play(500).get(0).name());

        System.out.println("\n== 5. Same seed, rank-everyone rule, bounce-back overflow (log hidden) ==");
        Game g5 = new Game(board, new RandomDice(1, 6, new Random(2024)),
                new Rules(OverflowRule.bounceBack(), WinStrategy.rankAll(), true, 3),
                List.of(new Player("Ann"), new Player("Bob"), new Player("Cid")), line -> { });
        System.out.println("  Standings: " + g5.play(1000).stream().map(Player::name).collect(Collectors.joining("
> ")));

        System.out.println("\n== 6. Eight independent sessions on eight threads at once ==");
        // Each session owns its players, turn queue and dice (its own seeded Random), and shares only the
        // read-only Board. So running them in parallel must give exactly the result each one gives alone.
        Map<Integer, String> parallel = new ConcurrentHashMap<>();
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            for (int seed = 1; seed <= 8; seed++) {
                int s0 = seed;
                pool.submit(() -> parallel.put(s0, winnerOf(board, s0)));
            }
        }
        List<String> mismatches = new ArrayList<>();
        for (int seed = 1; seed <= 8; seed++)
```

```
            if (!parallel.get(seed).equals(winnerOf(board, seed))) mismatches.add("seed " + seed);
        System.out.println("  winners by seed: " + new java.util.TreeMap<>(parallel));
        System.out.println("  same as running each session alone? " + (mismatches.isEmpty() ? "yes" : "NO " +
mismatches));
    }

    private static String winnerOf(Board board, int seed) {
        Game g = new Game(board, new RandomDice(1, 6, new Random(seed)), Rules.classic(),
                List.of(new Player("Ann"), new Player("Bob"), new Player("Cid")), line -> { });
        return g.play(500).get(0).name();
    }
}
```

Output:

```
== 1. Loaded dice: ladder, bonus roll, snake, three sixes ==
  Ann @0: rolls 4 -> 4 (ladder to 14)
  Bob @0: rolls 6 -> 6, bonus roll: rolls 3 -> 9 (ladder to 31)
  Ann @14: rolls 3 -> 17 (snake to 7)
  Bob @31: rolls 6 -> 37, bonus roll: rolls 6 -> 43, bonus roll: rolls 6 -> 3 sixes in a row, turn void, back to 31

== 2. Overflow rules: token on 97 of 100, rolls a 5 ==
  Exact @97: rolls 5 -> needs exact roll, stays at 97
  Bounce @97: rolls 5 -> 98

== 3. Bad configurations are rejected up front ==
  rejected: jump cannot start on first/last square: Jump[start=100, end=5]
  rejected: jump Jump[start=10, end=30] ends on another jump's start

== 4. Seeded game (seed 2024), 3 players, first to finish wins ==
  Ann @0: rolls 1 -> 1
  Bob @0: rolls 3 -> 3
  Cid @0: rolls 1 -> 1
  Ann @1: rolls 5 -> 6
  Bob @3: rolls 6 -> 9 (ladder to 31), bonus roll: rolls 6 -> 37, bonus roll: rolls 6 -> 3 sixes in a row, turn
void, back to 3
  Cid @1: rolls 6 -> 7, bonus roll: rolls 1 -> 8
  Ann @6: rolls 6 -> 12, bonus roll: rolls 4 -> 16
  Bob @3: rolls 4 -> 7
  Cid @8: rolls 3 -> 11
  Ann @16: rolls 4 -> 20
  Bob @7: rolls 2 -> 9 (ladder to 31)
  Cid @11: rolls 3 -> 14
  Ann @20: rolls 4 -> 24
  Bob @31: rolls 5 -> 36
  Cid @14: rolls 6 -> 20, bonus roll: rolls 1 -> 21 (ladder to 42)
  Ann @24: rolls 4 -> 28 (ladder to 84)
  Bob @36: rolls 3 -> 39
  Cid @42: rolls 5 -> 47
  Ann @84: rolls 4 -> 88
  Bob @39: rolls 5 -> 44
  Cid @47: rolls 3 -> 50
  Ann @88: rolls 1 -> 89
  Bob @44: rolls 6 -> 50, bonus roll: rolls 4 -> 54 (snake to 34)
  Cid @50: rolls 1 -> 51 (ladder to 67)
  Ann @89: rolls 4 -> 93 (snake to 73)
  Bob @34: rolls 2 -> 36
  Cid @67: rolls 5 -> 72
  Ann @73: rolls 6 -> 79, bonus roll: rolls 2 -> 81
  Bob @36: rolls 3 -> 39
  Cid @72: rolls 3 -> 75
  Ann @81: rolls 4 -> 85
  Bob @39: rolls 5 -> 44
  Cid @75: rolls 2 -> 77
  Ann @85: rolls 5 -> 90
  Bob @44: rolls 2 -> 46
  Cid @77: rolls 2 -> 79
  Ann @90: rolls 2 -> 92
  Bob @46: rolls 6 -> 52, bonus roll: rolls 5 -> 57
  Cid @79: rolls 1 -> 80 (ladder to 99)
  Ann @92: rolls 2 -> 94
  Bob @57: rolls 4 -> 61
  Cid @99: rolls 1 -> 100
  *** Cid finishes in place 1 ***
  Winner: Cid

== 5. Same seed, rank-everyone rule, bounce-back overflow (log hidden) ==
  Standings: Cid > Ann > Bob

== 6. Eight independent sessions on eight threads at once ==
  winners by seed: {1=Ann, 2=Bob, 3=Cid, 4=Bob, 5=Cid, 6=Cid, 7=Ann, 8=Bob}
  same as running each session alone? yes
```

Reading the output. Part 1 uses loaded dice to force every rule: Ann climbs 4→14, Bob rolls a 6 and gets a bonus roll that lands on the 9→31 ladder, Ann hits the 17→7 snake, and Bob's three sixes void his turn (he goes back to 31). Part 2 compares overflow rules from square 97 with a roll of 5: exact-finish stays, bounce-back goes to 100 and back 2 to 98. Part 3 shows bad boards rejected. Part 4 is a full seeded game; Part 5 replays with different rules and ranks every player. Part 6 runs eight games with different seeds on eight threads at the same time and checks that each winner is exactly what the same seed gives when that game runs alone — proof that sessions share no mutable state (data that can be changed after creation).

Concurrency & edge cases

- **Many sessions:** each `Game` owns its players, queue and dice. Give **each game its own** `Random` (sharing one `java.util.Random` across threads is thread-safe but becomes a contention point — many threads waiting on the same shared object — and breaks per-game reproducibility). Keep games in a `ConcurrentHashMap` and run each game's turns on one thread at a time.
- **Rolling after the game ends** → `IllegalStateException`.
- **A player on the last square** is removed from the queue, so they are never asked to roll again.
- **Bonus roll that finishes the game** stops immediately — no roll after reaching 100.
- **Three sixes** → the turn is voided, and the token returns to where the turn started (even if a ladder was climbed during the turn).
- **Bounce-back landing on a snake** (e.g. bouncing to 98 on the standard board) is applied — the rule output goes through the same jump lookup.
- **Endless games** are impossible with a valid board and fair dice (probability of never finishing is 0), but `maxTurns` guards against bad custom rules.

Extensions & follow-up questions

- **"Add special cells (skip a turn, extra move, teleport)."** — Generalise `Jump` into a `CellEffect` interface (`apply(player, game)`); snakes and ladders become two implementations.
- **"Two dice, or a 1–12 die?"** — `new RandomDice(2, 6, rng)`; `maxRoll()` is 12, so the bonus rule triggers on double-max. Note that 2d6 (two six-sided dice added together) is not uniform — not every total is equally likely (7 is the most likely), which changes game balance.
- **"Players must roll a 6 to enter the board."** — A new rule strategy: `EntryRule` checked when position is 0.
- **"Pause and resume."** — Serialize (turn into bytes or text that can be saved and loaded back) the game state (positions, queue order, and the dice's state — or a seed plus number of rolls so far).
- **"Real-time UI / spectators?"** — Observer: publish `PlayerMoved`, `SnakeBitten`, `LadderClimbed`, `PlayerFinished` events.
- **"Random board generation?"** — A `BoardGenerator` with a seed, calling the same validating constructor.

Real-world example

Online multiplayer board-game apps (for example Ludo King, which also offers a Snakes & Ladders mode) have to solve the same problems as this design: every match is an isolated session with its own state, the dice must be rolled somewhere the players cannot tamper with (in a sound design, on the server, not in the client app), and each move is pushed to all players as an event. That is the "one game = isolated state + injected dice + event stream" split used here.

In the interview

> **30-second answer:** "`Board` holds size and a map from square to `Jump` (snake or ladder — same behaviour, just direction), validated on construction. `Dice` is a strategy: seeded random for play, loaded for tests. `Rules` bundles pluggable policies: overflow (exact or bounce), win strategy (first finish or rank all), and the bonus-six rule. `Game` keeps a queue of players; each turn rolls, applies the overflow rule, then any jump, and re-queues the player unless they finished."

Time plan: 5 min rules questions (overshoot, sixes, win) → 5 min entities → 10 min diagram + validation rules → 15 min code turn loop → 5 min extensions and concurrency.

Self-check

1. **Q:** Why is one `Jump` class enough for snakes and ladders? **A:** Both mean "land on start, go to end"; only the direction differs, which is data.

2. **Q:** How do you guarantee there is no infinite snake/ladder loop? **A:** Forbid any jump from ending on another jump's start.
3. **Q:** How do you test "three sixes voids the turn"? **A:** `LoadedDice(6, 6, 6)` and assert the position is unchanged.
4. **Q:** What changes to support "bounce back"? **A:** Pass `OverflowRule.bounceBack()` — no game-loop edits.
5. **Q:** Why a queue for turns? **A:** Players who finish simply are not put back; turn order stays correct.

TL;DR

- Board = size + `Map<start, Jump>`; validate once (no jump on last square, no chains/loops).
- Dice, overflow, win condition, bonus rule are all strategies; seeded or loaded dice for tests.
- Queue-based turn loop with a `maxTurns` guard; events go to a listener, not `System.out`.

Design 2048

Problem & clarifying questions

2048 is a single-player sliding puzzle on a 4×4 grid. Each move (left, right, up, down) slides every tile as far as possible in that direction. Two tiles with the **same value** that collide **merge** into one tile with double the value, and the merged value is added to the **score**. After every move that changes the board, a new tile (a 2 most of the time, sometimes a 4) appears in a random empty cell. You "win" by making a 2048 tile; the game is **over** when no move can change the board.

There is no course lesson for this problem. The problem statement is the common machine-coding (an interview round where you write working code for a small system in 1–2 hours) version (the "2048" exercise found in LLD practice lists such as Tharun's and on machine-coding sites): start with **two tiles numbered 2** at random positions; after every move insert a random tile in a random empty cell; tiles slide until stopped by another tile or the edge; two equal tiles that collide merge into their sum; if **three** equal tiles line up, the pair **nearest the wall you move toward** merges; if **four** are equal, the first two and the last two merge; win at 2048; lose when there is no empty cell and no two equal neighbours; read moves from the command line as **0 = left, 1 = right, 2 = up, 3 = down**. Every one of those rules is in the design below.

Question	Assumed answer
Board size?	Configurable N (default 4).
New tile?	One per successful move: value 2 with probability 0.9, else 4, in a uniformly random empty cell. The game starts with two 2s.
Merge rule?	Each tile merges **at most once per move**; merges happen from the wall you move toward.
Move that changes nothing?	Not a turn: no new tile, no score.
Win?	Reaching the target tile (2048, configurable). The player may continue afterwards.
Game over?	No empty cell **and** no two equal neighbours (horizontally or vertically).
Undo, high scores, animations?	Out of scope; see Extensions.

Requirements

Functional

1. Start an N×N board with two tiles numbered 2 at random positions.
2. Move in four directions: slide, merge equal neighbours once, add merged values to the score.
3. If the move changed the board, spawn one tile (2 or 4) in a random empty cell.
4. Detect reaching the target (WON, then allow continuing).
5. Detect game over (LOST) when no move is possible.
6. Accept the command-line codes 0/1/2/3 for left/right/up/down (`Direction.fromKey`).

Non-functional

- **Deterministic** spawns via an injected, seeded `Random` for tests.
- The merge logic must be written **once**, not four times (one per direction).
- The spawning rule is pluggable (e.g. an "evil" spawner for testing hard cases).

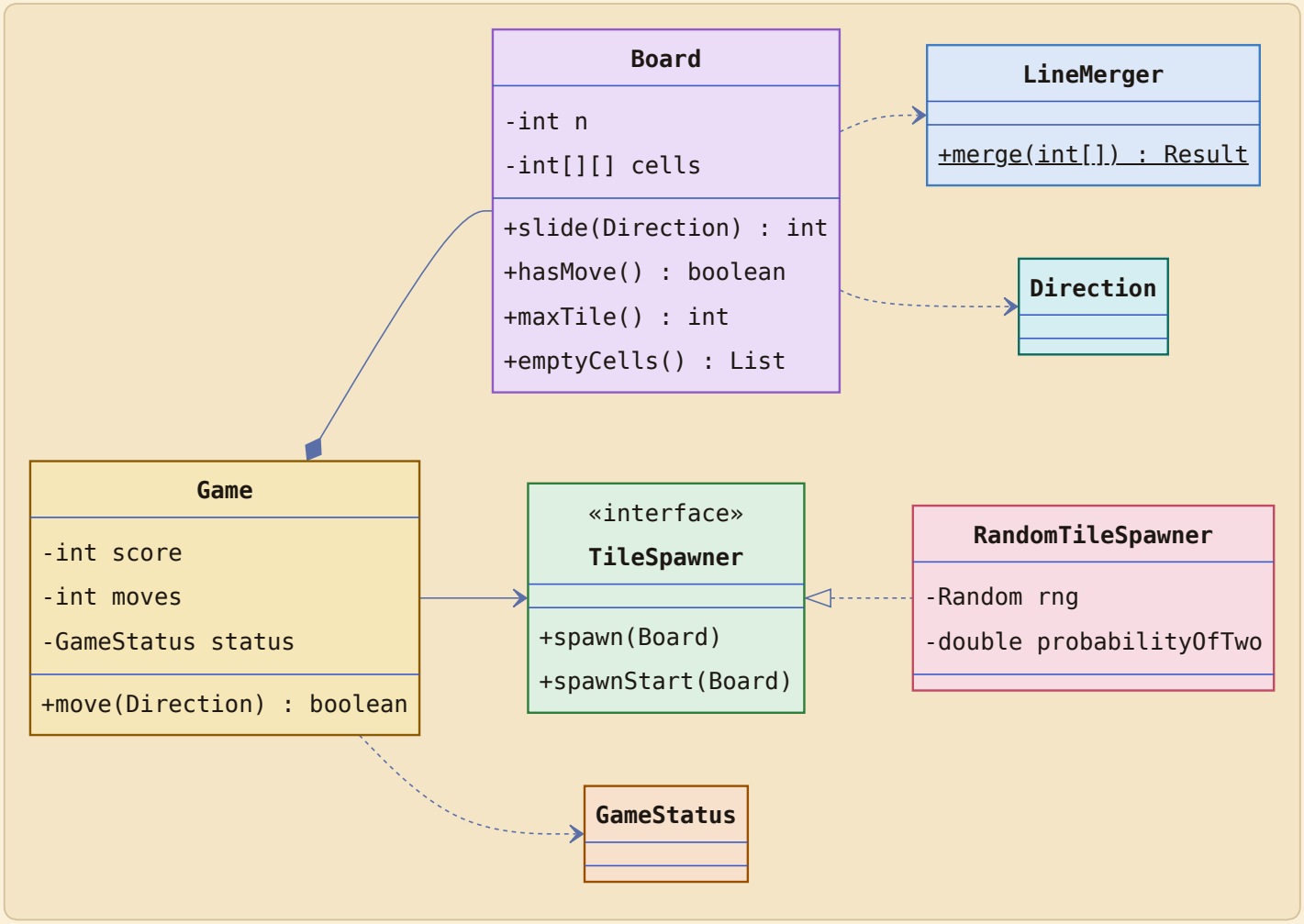
Use cases

Actor: **Player**. Use cases: *start new game*, *make a move* (includes *slide and merge*, *spawn tile*, *update score*), *see win*, *see game over*, *restart*.

Core entities

Class	Responsibility
`Direction` (enum)	LEFT, RIGHT, UP, DOWN; `fromKey(0..3)` maps the command-line codes.
`LineMerger`	Pure function (output depends only on input; changes nothing else): slide and merge **one line toward index 0**; returns the new line and points.
`Board`	N×N values; maps each direction onto lines for `LineMerger`; `slide`, `hasMove`, `maxTile`, `emptyCells`, `render`.
`TileSpawner` (interface) / `RandomTileSpawner`	Adds a new tile; seeded `Random`, 90% twos. `spawnStart` places the two starting tiles (always 2s).
`GameStatus` (enum)	PLAYING, WON, LOST.
`Game`	Score, move count, status; ties slide + spawn + status together.

Class diagram



Key flows

Merging one line (toward index 0) — the whole game in one function:

1. Walk the line from the wall outward, skipping zeros.

2. Keep one **pending** tile. If the next non-zero tile equals it, write the doubled value, add it to the points, and clear pending — so the new tile **cannot merge again this move**.
3. If it differs, write the pending tile and make the new tile pending.
4. At the end, write any pending tile; the rest of the line is zeros.

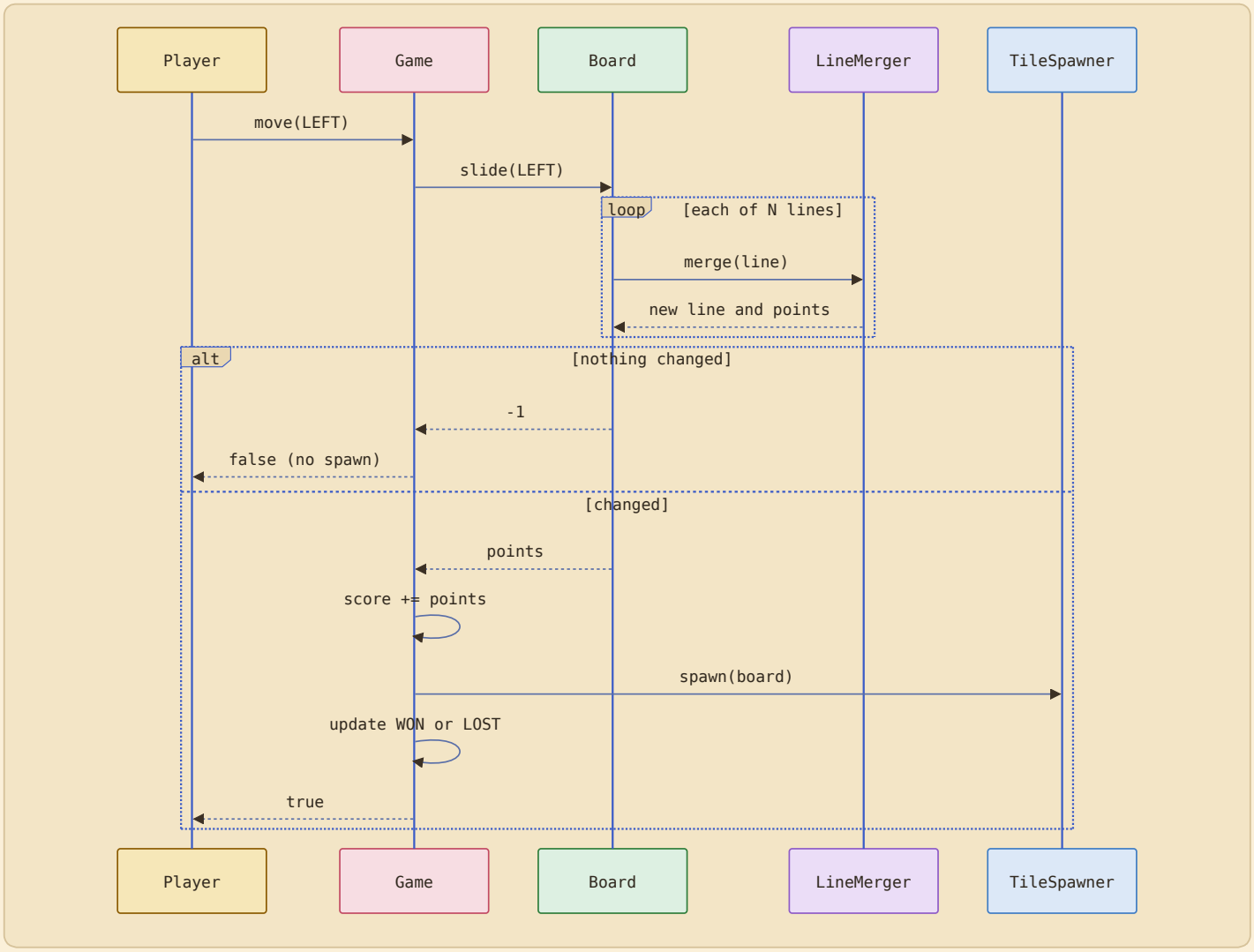
Examples (moving left): `[2,2,2,2] → [4,4,0,0]` (two merges, not one 8), `[2,2,2,0] → [4,2,0,0]` (of three equal tiles, the pair nearest the wall merges), `[2,2,4,0] → [4,4,0,0]` (the new 4 does not merge with the old 4), `[4,0,4,8] → [8,8,0,0]`.

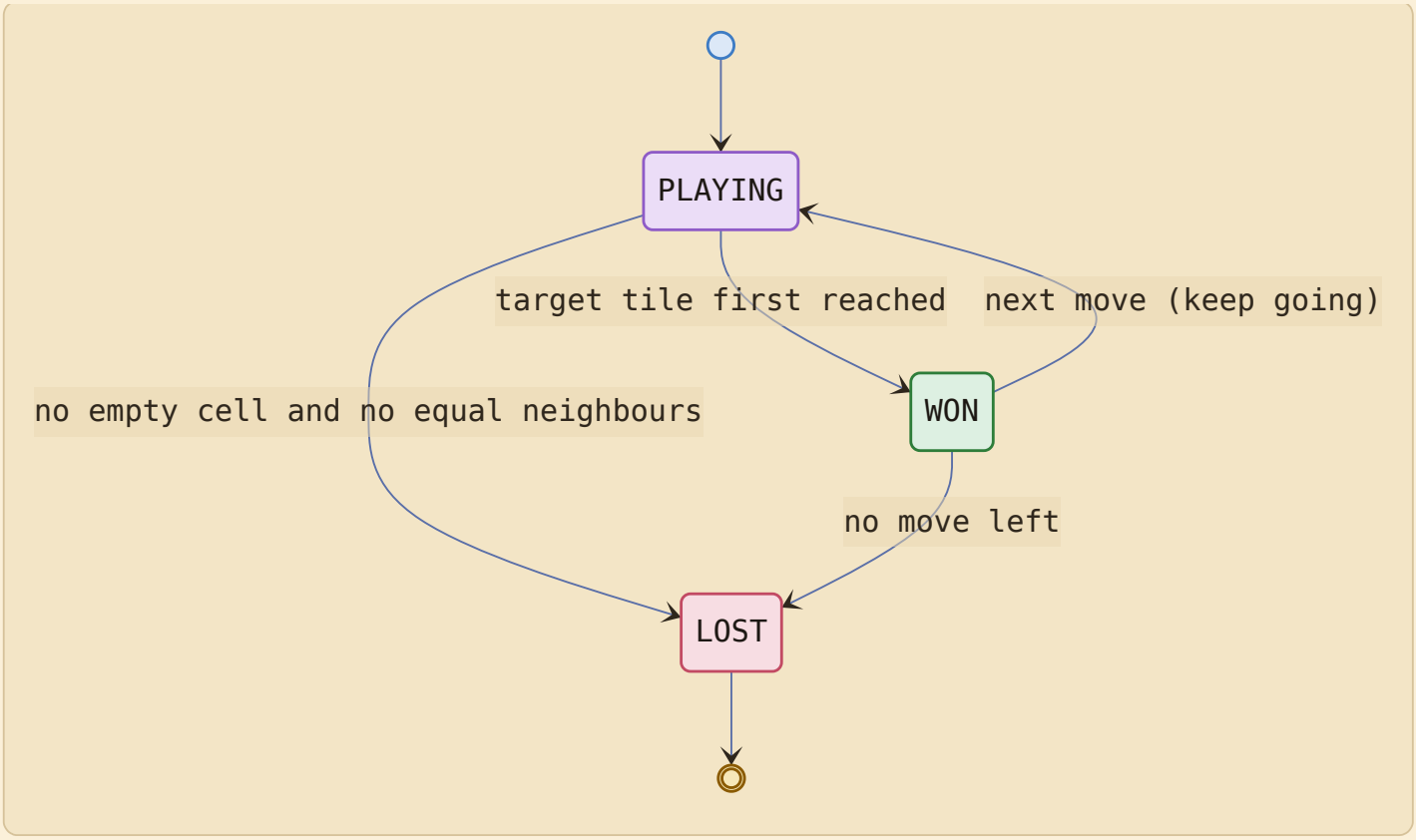
One function for four directions. `Board.coords(direction, i, j)` maps "line `i`, position `j` counted from the wall" to a real cell:

Direction	Line i is	j = 0 is
LEFT	row i	column 0
RIGHT	row i	column N-1
UP	column i	row 0
DOWN	column i	row N-1

Read each line in that order, merge it with `LineMerger`, write it back. No duplicated slide code.

A move





Game-over check. A move is possible if **any cell is empty** or **any two neighbours are equal**. It is enough to compare each cell with its right and lower neighbour (the other two directions are covered by the neighbour's own check). O(N²).

Design decisions & patterns used

Decision	Why	Alternative rejected
One pure `LineMerger` + direction→coordinates mapping.	The tricky rule is written and tested once (DRY — "Don't Repeat Yourself").	Four copies of nested loops (`moveLeft`, `moveRight`...) — classic bug farm.
`LineMerger` is a **pure function** (no state).	Trivial to unit test (test one small piece of code on its own; Part 1 of the output).	Merge logic mixed with board mutation.
Strategy for spawning (`TileSpawner`).	Seeded tests, "evil" spawner, different probabilities for variants.	`Math.random()` inside `Board`.
"No change = no turn" detected by comparing each written cell.	Matches the real game; prevents spawning on a no-op.	Always spawning — makes the game unwinnable/unfair.
`Board.of(int[][])` factory for preset boards.	Game-over and edge-case tests without playing to that position.	Only random starts — untestable edge cases.
Status updated after each move; WON only once.	The player can continue past 2048, as in the original game.	Ending the game at 2048.

Java implementation

Package `lld.ch17.game2048`.

```java
// File: code/lld/ch17/game2048/Direction.java
package lld.ch17.game2048;

public enum Direction {
    LEFT, RIGHT, UP, DOWN;

    /** Command-line input of the common machine-coding statement: 0 = left, 1 = right, 2 = up, 3 = down. */
    public static Direction fromKey(int key) {
        return switch (key) {
            case 0 -> LEFT;
            case 1 -> RIGHT;
            case 2 -> UP;
            case 3 -> DOWN;
            default -> throw new IllegalArgumentException("unknown move " + key + " (use 0-3)");
        };
    }
}
```

```java
// File: code/lld/ch17/game2048/LineMerger.java
package lld.ch17.game2048;

/**
 * The heart of 2048, written for ONE line moving toward index 0.
 * Every direction is reduced to this by reading the line in the right order.
 * Rule: slide tiles together, merge equal neighbours once per move, the merge nearest the wall first.
 */
public final class LineMerger {
    private LineMerger() {}

    /** Result: the new line and the points earned (sum of merged tile values). */
    public record Result(int[] line, int points) {}

    public static Result merge(int[] in) {
        int n = in.length;
        int[] out = new int[n];
        int write = 0, points = 0;
        int pending = 0;                        // tile waiting for a possible merge partner
        for (int v : in) {
            if (v == 0) continue;
            if (pending == 0) {
                pending = v;
            } else if (pending == v) {
                out[write++] = v * 2;           // merge, and the new tile cannot merge again this move
                points += v * 2;
                pending = 0;
            } else {
                out[write++] = pending;
                pending = v;
            }
        }
        if (pending != 0) out[write] = pending;
        return new Result(out, points);
    }
}
```

```java
// File: code/lld/ch17/game2048/TileSpawner.java
package lld.ch17.game2048;

/** Strategy for adding a new tile after each successful move. */
public interface TileSpawner {
    /** Adds one tile to a random empty cell. The board must have at least one empty cell. */
    void spawn(Board board);

    /** One of the two tiles placed when a new game starts. By default the same as spawn(). */
    default void spawnStart(Board board) { spawn(board); }
}
```

```java
// File: code/lld/ch17/game2048/RandomTileSpawner.java
package lld.ch17.game2048;

import java.util.List;
import java.util.Random;

/** Classic rule: a 2 with probability 0.9, else a 4, in a uniformly random empty cell. Seeded for tests. */
public final class RandomTileSpawner implements TileSpawner {
    private final Random rng;
    private final double probabilityOfTwo;

    public RandomTileSpawner(Random rng, double probabilityOfTwo) {
        this.rng = rng;
        this.probabilityOfTwo = probabilityOfTwo;
    }

    @Override public void spawn(Board board) {
        int[] cell = randomEmptyCell(board);
        board.set(cell[0], cell[1], rng.nextDouble() < probabilityOfTwo ? 2 : 4);
    }

    /** The problem statement starts every game with two tiles numbered 2. */
    @Override public void spawnStart(Board board) {
        int[] cell = randomEmptyCell(board);
        board.set(cell[0], cell[1], 2);
    }

    private int[] randomEmptyCell(Board board) {
        List<int[]> empty = board.emptyCells();
        if (empty.isEmpty()) throw new IllegalStateException("no empty cell");
        return empty.get(rng.nextInt(empty.size()));
    }
}
```

```java
// File: code/lld/ch17/game2048/Board.java
package lld.ch17.game2048;

import java.util.ArrayList;
import java.util.List;

/** N x N grid of tile values (0 = empty). Knows how to slide in a direction and whether any move is left. */
public final class Board {
    private final int n;
    private final int[][] cells;

    public Board(int n) {
        if (n < 2) throw new IllegalArgumentException("board too small");
        this.n = n;
        this.cells = new int[n][n];
    }

    public static Board of(int[][] values) {
        Board b = new Board(values.length);
        for (int r = 0; r < b.n; r++) b.cells[r] = values[r].clone();
        return b;
    }

    public int size() { return n; }
    public int get(int r, int c) { return cells[r][c]; }
    void set(int r, int c, int v) { cells[r][c] = v; }

    public List<int[]> emptyCells() {
        List<int[]> out = new ArrayList<>();
        for (int r = 0; r < n; r++) for (int c = 0; c < n; c++) if (cells[r][c] == 0) out.add(new int[]{r, c});
        return out;
    }

    /**
     * Slides every line toward the given wall. Returns the points earned, or -1 if nothing moved
     * (a move that changes nothing is not a turn and must not spawn a tile).
     */
    int slide(Direction d) {
        boolean changed = false;
        int points = 0;
        for (int i = 0; i < n; i++) {
            int[] line = new int[n];
            for (int j = 0; j < n; j++) line[j] = read(d, i, j);       // j = 0 is the cell next to the wall
            LineMerger.Result res = LineMerger.merge(line);
            for (int j = 0; j < n; j++) {
                if (read(d, i, j) != res.line()[j]) changed = true;
                write(d, i, j, res.line()[j]);
            }
            points += res.points();
        }
        return changed ? points : -1;
    }

    /** Maps (line i, position j from the wall) to a real cell for each direction. */
    private int[] coords(Direction d, int i, int j) {
        return switch (d) {
            case LEFT -> new int[]{i, j};
            case RIGHT -> new int[]{i, n - 1 - j};
            case UP -> new int[]{j, i};
            case DOWN -> new int[]{n - 1 - j, i};
        };
    }

    private int read(Direction d, int i, int j) { int[] rc = coords(d, i, j); return cells[rc[0]][rc[1]]; }
    private void write(Direction d, int i, int j, int v) { int[] rc = coords(d, i, j); cells[rc[0]][rc[1]] = v; }

    /** A move exists if a cell is empty or two neighbours (right or below) are equal. */
    public boolean hasMove() {
```

```
        for (int r = 0; r < n; r++)
            for (int c = 0; c < n; c++) {
                if (cells[r][c] == 0) return true;
                if (c + 1 < n && cells[r][c] == cells[r][c + 1]) return true;
                if (r + 1 < n && cells[r][c] == cells[r + 1][c]) return true;
            }
        return false;
    }

    public int maxTile() {
        int m = 0;
        for (int[] row : cells) for (int v : row) m = Math.max(m, v);
        return m;
    }

    public String render() {
        StringBuilder sb = new StringBuilder();
        for (int r = 0; r < n; r++) {
            sb.append("   ");
            for (int c = 0; c < n; c++) sb.append(String.format("%5s", cells[r][c] == 0 ? "." : cells[r][c]));
            if (r < n - 1) sb.append('\n');
        }
        return sb.toString();
    }
}
```

```
// File: code/lld/ch17/game2048/GameStatus.java
package lld.ch17.game2048;

public enum GameStatus { PLAYING, WON, LOST }
```

```java
// File: code/lld/ch17/game2048/Game.java
package lld.ch17.game2048;

/**
 * One game: board + spawner + score + status. Reaching the target tile sets WON once;
 * the player may keep going ("continue" mode). No move left sets LOST.
 */
public final class Game {
    private final Board board;
    private final TileSpawner spawner;
    private final int target;
    private int score;
    private int moves;
    private boolean reachedTarget;
    private GameStatus status = GameStatus.PLAYING;

    /** New game: an empty board with two starting tiles (two 2s with the standard spawner). */
    public Game(int size, int target, TileSpawner spawner) {
        this(new Board(size), target, spawner);
        spawner.spawnStart(board);
        spawner.spawnStart(board);
        updateStatus();
    }

    /** Game from a preset board (used by tests). */
    public Game(Board preset, int target, TileSpawner spawner) {
        this.board = preset;
        this.target = target;
        this.spawner = spawner;
        updateStatus();
    }

    /** Returns true if the move changed the board (and so counted as a turn). */
    public boolean move(Direction d) {
        if (status == GameStatus.LOST) throw new IllegalStateException("game over");
        int points = board.slide(d);
        if (points < 0) return false;          // nothing moved: no spawn, no turn
        score += points;
        moves++;
        spawner.spawn(board);                  // at least one cell is empty, because something moved or merged
        updateStatus();
        return true;
    }

    private void updateStatus() {
        if (!reachedTarget && board.maxTile() >= target) { reachedTarget = true; status = GameStatus.WON; }
        else if (status == GameStatus.WON) status = GameStatus.PLAYING;   // "keep going" after the win screen
        if (!board.hasMove()) status = GameStatus.LOST;
    }

    public GameStatus status() { return status; }
    public boolean reachedTarget() { return reachedTarget; }
    public int score() { return score; }
    public int moves() { return moves; }
    public Board board() { return board; }
}
```

```java
// File: code/lld/ch17/game2048/Main.java
package lld.ch17.game2048;

import java.util.Arrays;
import java.util.List;
import java.util.Random;

public final class Main {
    public static void main(String[] args) {
        System.out.println("== 1. Merge rules for one line moving left ==");
        for (int[] line : List.of(new int[]{2, 2, 2, 2}, new int[]{2, 2, 2, 0}, new int[]{2, 2, 4, 0}, new int[]{4,
0, 4, 8},
                new int[]{2, 0, 0, 2}, new int[]{8, 4, 2, 2}, new int[]{2, 4, 8, 16})) {
            LineMerger.Result r = LineMerger.merge(line);
            System.out.println("  " + Arrays.toString(line) + " -> " + Arrays.toString(r.line()) + "  +" +
r.points());
        }

        System.out.println("\n== 2. A move that changes nothing does not count and spawns nothing ==");
        Game stuckLeft = new Game(Board.of(new int[][]{{2, 4, 8, 16}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}}),
                2048, new RandomTileSpawner(new Random(1), 0.9));
        System.out.println("  move LEFT changed board? " + stuckLeft.move(Direction.LEFT) + ", moves=" +
stuckLeft.moves());
        System.out.println("  move DOWN changed board? " + stuckLeft.move(Direction.DOWN) + ", moves=" +
stuckLeft.moves());
        System.out.println(stuckLeft.board().render());

        System.out.println("\n== 3. Game-over detection ==");
        Game full = new Game(Board.of(new int[][]{{2, 4, 2, 4}, {4, 2, 4, 2}, {2, 4, 2, 4}, {4, 2, 4, 2}}),
                2048, new RandomTileSpawner(new Random(1), 0.9));
        System.out.println("  checkerboard, no empty cell, no equal neighbours -> " + full.status());
        Game almost = new Game(Board.of(new int[][]{{2, 4, 2, 4}, {4, 2, 4, 2}, {2, 4, 2, 4}, {4, 2, 4, 4}}),
                2048, new RandomTileSpawner(new Random(1), 0.9));
        System.out.println("  same but last row ends 4,4 -> " + almost.status());

        System.out.println("\n== 4. Keyboard codes from the problem statement ==");
        for (int key = 0; key < 4; key++) System.out.print("  " + key + "=" + Direction.fromKey(key));
        System.out.println();

        System.out.println("\n== 5. Seeded game (seed 42), corner strategy LEFT > DOWN > RIGHT > UP, target 128
==");
        Game g = new Game(4, 128, new RandomTileSpawner(new Random(42), 0.9));
        System.out.println("  start:\n" + g.board().render());
        List<Direction> priority = List.of(Direction.LEFT, Direction.DOWN, Direction.RIGHT, Direction.UP);
        boolean announced = false;
        while (g.status() != GameStatus.LOST) {
            for (Direction d : priority) if (g.move(d)) break;
            if (g.reachedTarget() && !announced) {
                announced = true;
                System.out.println("  reached 128 after " + g.moves() + " moves (status " + g.status() + "), score
" + g.score());
                System.out.println(g.board().render());
            }
        }
        System.out.println("  game over after " + g.moves() + " moves, score " + g.score() + ", max tile " +
g.board().maxTile());
        System.out.println(g.board().render());
    }
}
```

Output:

```
== 1. Merge rules for one line moving left ==
  [2, 2, 2, 2] -> [4, 4, 0, 0]  +8
  [2, 2, 2, 0] -> [4, 2, 0, 0]  +4
  [2, 2, 4, 0] -> [4, 4, 0, 0]  +4
  [4, 0, 4, 8] -> [8, 8, 0, 0]  +8
  [2, 0, 0, 2] -> [4, 0, 0, 0]  +4
  [8, 4, 2, 2] -> [8, 4, 4, 0]  +4
  [2, 4, 8, 16] -> [2, 4, 8, 16]  +0

== 2. A move that changes nothing does not count and spawns nothing ==
  move LEFT changed board? false, moves=0
  move DOWN changed board? true, moves=1
       .    .    .    .
       .    .    .    .
       .    2    .    .
       2    4    8   16

== 3. Game-over detection ==
  checkerboard, no empty cell, no equal neighbours -> LOST
  same but last row ends 4,4 -> PLAYING

== 4. Keyboard codes from the problem statement ==
  0=LEFT  1=RIGHT  2=UP  3=DOWN

== 5. Seeded game (seed 42), corner strategy LEFT > DOWN > RIGHT > UP, target 128 ==
  start:
       .    .    .    2
       .    .    .    .
       .    .    .    2
       .    .    .    .
  reached 128 after 85 moves (status WON), score 852
       8    2    .    .
      16    4    4    .
       2    8    4    .
     128    4    .    .
  game over after 232 moves, score 2584, max tile 256
      16    8    4    2
       8   32    8    4
      16    2   16    8
     256   64   32   16
```

Reading the output. Part 1 unit-tests the merge rule (note `[2,2,2,2]` becomes `[4,4,0,0]`, not `[8,0,0,0]`, and `[2,2,2,0]` becomes `[4,2,0,0]`). Part 2 shows that a move changing nothing does not count. Part 3 checks game-over detection on preset boards. Part 4 prints the command-line key mapping. Part 5 plays a whole seeded game that starts from two 2s, using a simple "keep tiles in the bottom-left corner" priority strategy; it reaches the 128 target after 85 moves (status `WON`) and keeps playing until the board locks after 232 moves with a 256 tile.

Concurrency & edge cases

- Single-player and single-threaded by nature. If a UI thread (the thread that draws the screen) and an input thread both call `move`, make `move` `synchronized` (or post moves to the UI event queue — the list of pending clicks, key presses and redraws that the UI thread handles one by one).
- **Key repeat / double input:** a no-op move returns `false` and changes nothing, so extra key presses are harmless.
- **Spawning needs an empty cell:** guaranteed, because a changed board after sliding always has at least one empty cell (tiles only move into gaps or merge, which frees a cell). The spawner still checks and throws if not.
- **Integer overflow** (a number growing past the largest value its type can hold, so it wraps around to a wrong value): not reachable on 4×4 (the maximum possible tile is 2^17 = 131072); a very large board would still fit an `int` for any practical game.
- **Merge order matters:** `[2,2,2]` moving left is `[4,2,0]` — the pair nearest the wall merges first.

Extensions & follow-up questions

- **"Add undo."** — Memento: push a copy of the board and score before each successful move; undo pops it (the spawned tile disappears too).

- **"Different sizes / targets?"** — Already parameters (`new Game(5, 4096, spawner)`).
- **"An AI player?"** — Expectimax search (max over your moves, average over random spawns) is the well-known approach; plug it in as a move-choosing strategy like the corner strategy in `Main`.
- **"Animations?"** — Have `LineMerger` also return where each tile came from (a list of moves/merges) and publish them to the view (Observer, MVC).
- **"High scores / leaderboard?"** — A separate service; the game only emits "game over with score X".
- **"Why is `[2,2,4]` left `[4,4,0]` and not `[8,0,0]`?"** — Each tile merges at most once per move; the new 4 was created this move.

Real-world example

2048 was created by Gabriele Cirulli in 2014 and released as open-source (its code is public for anyone to read and reuse) JavaScript; its code has the same separation: a grid model, a game manager that handles moves, merges, score and "keep playing" after a win, and a separate view that animates tiles. Its new tiles are 2 with 90% probability and 4 with 10%, the same rule our `RandomTileSpawner` uses.

In the interview

> **30-second answer:** "The core is one pure function that slides and merges a single line toward index 0: skip zeros, merge equal neighbours once, score the merged values. The board maps each direction to lines in the right order, so the function is reused four times. After a move that changes the board, a `TileSpawner` — seeded for tests — adds a 2 (90%) or a 4. The game is won when the target tile appears and over when there's no empty cell and no equal neighbours."

Time plan: 5 min rules (merge once, no-op move) → 5 min entities → 15 min `LineMerger` + direction mapping (write the tests first: `[2,2,2,2]`, `[2,2,4,0]`) → 10 min game loop, spawning, game over → 5 min extensions.

Self-check

1. **Q:** What does `[4,4,8,8]` become moving left, and how many points? **A:** `[8,16,0,0]`, 24 points.
2. **Q:** Why must a no-op move not spawn a tile? **A:** It is not a turn in the real game; spawning would punish harmless key presses and change the game's balance.
3. **Q:** How do you avoid four copies of the slide code? **A:** Map each direction to lines read from the wall, then call one line-merge function.
4. **Q:** When is the game over? **A:** No empty cell and no two equal adjacent tiles.
5. **Q:** How do you make a 2048 test repeatable? **A:** Inject `new Random(seed)` into the spawner, or start from `Board.of(...)`.

TL;DR

- One pure `LineMerger` (merge once per move, from the wall) + a direction→line mapping.
- Spawn only after a real change; 2 (90%) or 4 (10%), seeded spawner for tests.
- WON once at the target (keep playing); LOST when no empty cell and no equal neighbours.

Chapter summary

Game	Core rule object	Pluggable strategies	Randomness for tests	Signature technique
Blackjack	`Hand` scoring + `BlackjackScoring`	`PlayerStrategy`, `DealerPolicy`	Seeded `Shoe`, stacked shoe	Soft/hard totals; generic `Deck<T>`
Chess	`Rules` (legal-move filter)	(bot would be a move strategy)	None needed — deterministic game	Polymorphic pieces; checkmate = in check + no legal move; Command + Memento undo
Tic-Tac-Toe	`Board` counters	`MoveStrategy`	Seeded `RandomStrategy`, scripted players	O(1) win with row/col/diagonal counters
Snake & Ladder	`Board` + `Rules` record	`Dice`, `OverflowRule`, `WinStrategy`	Seeded `RandomDice`, `LoadedDice`	Validated config; queue-based turns
2048	`LineMerger`	`TileSpawner`	Seeded spawner, `Board.of` presets	One merge function reused for 4 directions

Patterns you used in this chapter: Strategy (everywhere), Factory Method (`Deck.standard52`, `Piece.create`, `Board.of`), Template-style shared base (`SlidingPiece`), Command + Memento (chess undo), State (game-status enums; a full State pattern is the suggested upgrade for blackjack rounds), Observer (the event log hooks; suggested for UIs and spectators).

The three habits interviewers look for in game designs:

1. **Put every rule in exactly one place** — and be able to say where.
2. **Inject randomness** — seeded or scripted — so the design is testable.
3. **Model the game's states explicitly** — and reject actions that are invalid in the current state.

Chapter 18 — Case Studies: Platforms & Feeds (Part 1)

> **Where this fits.** Chapters 14–17 designed *things* (a parking lot, a vending machine), *bookings* (movies, hotels, flights) and *games*. This chapter and the next one design **platforms**: websites where many people create content, react to each other's content and build up a history. The objects are no longer machines with buttons; they are people, posts, votes, relationships and feeds. Chapter 19 continues with more platforms.

What you will learn

#	Problem	What makes it interesting	Main patterns and ideas
18.0	What platform problems test	The common toolkit you will reuse in every social or community design	Facade, Observer, Strategy, services for two-sided operations, inverted index
18.1	Design Stack Overflow	Reputation unlocks privileges; one vote per member; moderation; bounties	Shared `Post` base class, Strategy (reputation rules), Observer (badges), privilege table
18.2	Design Cricinfo (+ Udaan "Cricket Match Dashboard")	Modelling a real sport's rules; live scorecards; stats derived from balls	Template Method (formats and stat queries), Observer (live feeds), "derive, don't store"
18.3	Design LinkedIn	Profiles, connections and their degrees, invitations, messaging, jobs, search, feed	Service for two-sided operations, State (invitation), BFS for degrees, Strategy (feed ranking)
18.4	Design Facebook	Friends vs followers, privacy lists, pages, groups, news feed, friend suggestions	One visibility policy for feed + search, sealed `Audience`, pull-model feed, suggestions

How to run every example in this chapter

```
cd chapters/18-case-platforms-1/code
javac --release 21 -d /tmp/lld_18 $(find . -name '*.java')
java -cp /tmp/lld_18 lld.ch18.stackoverflow.Main       # or cricinfo, cricketdashboard, linkedin, facebook
```

`javac` is the Java **compiler** (it turns `.java` source files into `.class` files the Java runtime can execute); `--release 21` says which Java version to target, and `-d` names the output folder. `java -cp` runs a program, where the **classpath** (`-cp`) is the list of folders in which Java looks for compiled classes. The **standard library** is the set of classes that ships with Java itself, so nothing extra has to be downloaded.

All five `Main` classes compile with Java 21 (standard library only) and run. The output printed under each design is the real output of that program.

> **Code-style note.** To keep a whole design readable in one place, a file sometimes holds several small package-private types (for example `Posts.java` holds `Post`, `Question`, `Answer`, `Comment`, `Tag` and `Bounty`). That is legal Java. In a production codebase you would usually give each public top-level type its own file.

> **Key term — Java vocabulary used in this chapter:** A **package** is a named folder of related classes (`lld.ch18.stackoverflow`); a **package-private** type has no `public` / `private` keyword and can be used only by classes in the same package. A **top-level type** is a class, interface, enum or record not nested inside another one. An **interface** lists methods a class promises to provide, without the code; a class that provides them *implements* it. An **abstract class** can never be created directly — only its subclasses can. An **enum** is a type with a fixed list of named values (`OPEN`, `CLOSED`). A **record** is a short way to declare a class that just carries data that never changes. A **sealed** type names exactly which subclasses are allowed, so the compiler knows every possible kind.

18.0 What platform problems test

What it is

A **platform problem** asks you to design a website or app where *users produce the content* — questions and answers, posts and comments, match commentary, job postings. Stack Overflow, Cricinfo, LinkedIn and Facebook are the classic examples. Unlike a vending machine, there is no single "machine" object in the middle. Instead there is a **rich domain model**: many small classes (profile, post, comment, vote, tag, badge, invitation, group, page…) that each hold a little data and a little behaviour.

Everyday analogy: a vending machine is like a toaster — one box with a few buttons. A platform is like a town — many people, many buildings, rules about who may enter where, a notice board (the feed), a post office (notifications) and a library catalogue (search).

The problem it solves

Interviewers pick platform problems to check six skills at once. The table names each one, defines it, and shows where this chapter practises it.

Skill tested	What it means	Where you practise it
Rich domain model	Turning a long list of nouns into classes with clear responsibilities, and knowing which noun is *just data* (a record) and which has *behaviour*	Cricinfo's `Over` knows a wide is not one of its six balls
User roles	Different people may do different things: guest, member, moderator, admin, commentator	Stack Overflow's reputation-based privileges; Facebook's admin who blocks members
Feeds	A **feed** is a personalised, ranked list of recent content from the people and pages you are linked to	LinkedIn and Facebook feed services with a ranking Strategy
Notifications	Telling a user that something happened (new message, friend request, comment) through one or more **channels** (in-app, email, push — a pop-up alert sent to a phone or browser)	Observer + Strategy in every design
Search	Finding things by words. Built with an **inverted index**: a map from each word to the items that contain it	Question search, member search, privacy-aware post search
Reputation / trust	Numbers that summarise past behaviour and unlock abilities (reputation, badges, endorsements, ratings)	Stack Overflow reputation and badges; LinkedIn endorsements; Facebook page ratings

How it works — the reusable toolkit

These ideas repeat in every design in this chapter. Learn them once.

1. **A facade in front.** A **facade** is one class that offers the simple operations the UI (user interface — the screens and buttons people use) needs (`ask`, `vote`, `post`, `invite`) and hides the many classes behind it (Chapter 9). It is also the natural place to check "is this account active?" and "is this person allowed?" before anything changes.
2. **Behaviour shared by content goes in a base class.** Questions, answers and comments are all voted on; posts and comments are all liked. Write that once in a base class (`Post`, `SocialContent`) and let the content-specific parts stay in subclasses.
3. **Store *who*, not *how many*.** Keep likes and votes as a **set of member ids**, not an integer counter. A set blocks double voting, allows "unlike", and still gives the count as `set.size()`.
4. **Derive, don't store.** If a number can be computed from data you already keep (follower count, a batter's runs, a page's rating), compute it. A stored copy will one day disagree with the source and nobody will know which is right. If it is too slow to compute every time, keep a **cache** (a saved copy of a computed answer, kept so it can be read quickly instead of recomputed) that you can always rebuild from the source. See System Design Fundamentals for caches at scale.
5. **An operation that changes two objects lives in a service.** Accepting a connection or friend request changes *both* members. Put it on neither — put it in a small **service** class (a class whose job is to perform an operation, not to hold data) so there is exactly one place that keeps both sides consistent.

6. **Notifications through Observer + Strategy.** The domain *publishes* events ("answer posted"); **observers** (Chapter 10) react. The **Observer** pattern means objects *subscribe* to another object and are called back automatically whenever it announces an event, so the announcer does not need to know who is listening. The **Strategy** pattern means putting each interchangeable way of doing a job (here: each delivery method) behind one interface, so you can swap or add ways without editing the caller. How a message is *delivered* (email, push, in-app) is a **Strategy** (Chapter 11) so adding a channel means adding a class, not another `if`.
7. **Search is its own class.** An **inverted index** (word → items) belongs in a `SearchIndex`, not on the entity. Rule from the course: every key maps to a *collection*, even when only one item matches, so the second "Sharma" never breaks the index.
8. **Privacy and permissions are asked in one place.** If both the feed and the search must hide private posts, write `canSee(viewer, post)` once and call it from both.

Trade-offs

Choice	Benefit	Cost
Rich model (many small classes)	Each rule has one home; easy to test	More classes to explain in 45 minutes
Sets of voter ids instead of counters	Correct: no double votes, reversible	Memory grows with votes; real systems store votes in a table and cache the count
Derive stats from raw events	Never inconsistent	Slow for large data; needs a cache
Services for two-sided operations	Symmetry guaranteed in one place	One more class; entities become thinner ("anemic") if you overdo it

When not to use it

Do not bring the whole toolkit to a tiny problem. If the interviewer asks only for "a like button", a set on the post is enough — no services, no observers.

Common mistakes

- Putting search as a static method on `Question` ("`Question.search()`"), so the entity must know every other question.
- Storing `likeCount` as a constructor argument — a post can then be created "with 1,000 likes".
- Forgetting that the privacy rule also applies inside search and the feed.
- Writing `acceptInvitation` on the sender (or the receiver) and updating only one side.

Real-world examples

- **Kafka** (a system that stores streams of events and delivers them to any program that wants to read them) was created at LinkedIn to move activity events (page views, posts, connections) between systems — the industrial version of the Observer idea used here.
- **Facebook's TAO** is a graph store (a database built for a **graph** — items joined by links, like people joined by friendships) built to serve the social graph (users, friendships, likes) at very large scale — the production version of our in-memory friend graph.
- **Stack Overflow** publishes its reputation-to-privilege table on its help pages — exactly the data-driven `Privilege` enum used in 18.1.

In the interview

> **30-second answer:** "Platform problems test whether I can model a domain with many small classes, put each rule in one place, and handle roles, feeds, notifications and search. I'll start with requirements and actors, then the core entities, then the tricky flows — voting, accepting requests, privacy — and I'll say which operations need a service because they touch two objects."

Follow-ups:

- *Why a set instead of a counter for likes?* — A bare number does not record *who* liked, so it can neither block a repeat like nor undo one. The set gives the count for free.
- *Where does the notification code go?* — Not inside the domain methods directly; the domain raises an event and a notification service with pluggable channels delivers it.

Self-check

- **Q:** What is an inverted index? **A:** A map from each word to the items containing that word; used to answer "which items contain these words?" quickly.
- **Q:** Why is "accept a friend request" not a method on `Member`? **A:** It changes two members; putting it on one of them means one object edits another's state, and the two halves can drift. A service updates both together.
- **Q:** Name two ways to deliver a notification and the pattern that makes adding a third easy. **A:** Email and push; Strategy (a `NotificationChannel` interface).

TL;DR

Platform problems = rich domain model + roles + feed + notifications + search + reputation. Share behaviour in a content base class, store *who* not *how many*, derive numbers, put two-sided operations in services, notify via Observer + Strategy, keep search and privacy each in one place.

18.1 Design Stack Overflow

Figure — The Stack Overflow logo that opens this case study (course diagram)

Stack Overflow is a huge question-and-answer community for programmers. Members can **upvote** or **downvote** questions and answers, and anyone with enough standing can edit posts, much like a wiki (a website whose pages any permitted reader can edit). Members earn **reputation** (a points score) and **badges** (named awards) for useful contributions. Higher reputation unlocks **privileges** — voting, commenting anywhere, editing other people's posts, voting to close questions. For scale: an upvote on an answer is worth 10 points to its author. The course says a question upvote is worth 5; that was true until November 2019, when Stack Overflow raised it to 10 as well, and this chapter's code uses today's value.

Problem & clarifying questions

Ask these before drawing anything. The answers are the assumptions this design uses.

Question to ask	Answer we assume
Who uses the system?	Guests (not logged in), members, moderators, admins, and the system itself (notifications, badges)
What can a guest do?	Search and read only. Posting or voting requires membership
What can be voted on?	Questions, answers and comments (comments: upvote only, as on the real site)
How much reputation does a vote give?	Upvote +10 on a question or an answer (question upvotes were +5 before November 2019), downvote −2 to the author, −1 to whoever downvotes an answer (downvoting a question is free), accepted answer +15 (and +2 to the asker)
Does reputation unlock actions?	Yes — a privilege table (vote up at 15, comment everywhere at 50, set bounty at 75, vote down at 125, create tag at 1,500, edit at 2,000, close votes at 3,000, delete votes at 10,000)
How are questions closed?	3 close votes from members with the privilege, or one moderator. Moderators can also reopen, delete and undelete
What is a bounty?	Reputation a member (usually the asker) stakes, 50 to 500 points, to attract answers. It runs for 7 days (the real site adds a 24-hour grace period) and goes to the answer the sponsor picks. The staked points are taken at once and are never refunded; if the sponsor picks nobody, the real site may auto-award half to a qualifying new answer
Do we need search?	Yes: by keyword and by tag, plus "most used tags"
Scale?	Read-heavy (many more views than posts); votes must never double-count

Requirements

Functional (the course's thirteen, in our words)

1. Guests can search and view questions; to post or upvote they must become members.
2. Members can post new questions.
3. Members can answer an **open** question.
4. Members can comment on any question or answer.
5. Questions, answers and comments can all be upvoted by members.
6. Members can **flag** any question, answer or comment that has a serious problem or needs a moderator to look at it.
7. A member can attach a **bounty** to a question they asked.
8. Members earn **badges** for being helpful.
9. Members can cast votes to **close** a question; a moderator can close or reopen any question on their own.
10. Members can add **tags** (a word or phrase naming the topic) to questions.
11. Members can vote to **delete** far off-topic or very low-quality questions.
12. Moderators can close questions and restore (undelete) deleted ones.
13. The system can report the **most frequently used tags**.

Plus from the real site (added so the privilege rules have somewhere to live): downvotes, accepting an answer, reputation-gated privileges, blocking accounts.

Non-functional

- **Correctness of counts:** one member, one vote per post; reputation changes must be reversible when a vote changes.
- **Read-heavy performance:** search and "top tags" must not scan every question.
- **Concurrency** (several pieces of work running at the same time — for example two requests handled by two **threads**, the independent paths of execution inside one program)**:** two members voting on the same post at the same instant must both be counted exactly once.
- **Abuse resistance:** you cannot vote on your own post; blocked accounts can do nothing.
- **Extensibility** (how easily new features can be added without rewriting old code)**:** reputation rules, badge rules and notification channels change often — changing them should not touch voting code.

Use cases

The five actors:

- **Guest** — searches and views questions.
- **Member** — everything a guest can do, plus ask, answer, comment, vote, flag, add bounties and tags, and delete or undelete their own posts.
- **Moderator** — everything a member can do, plus close, reopen, delete and undelete *any* question.
- **Admin** — blocks and unblocks members.
- **System** — sends notifications and assigns badges.

Top use cases: search questions; ask a question (with tags and an optional bounty); write or edit answers; add comments; moderators close, delete and undelete questions.

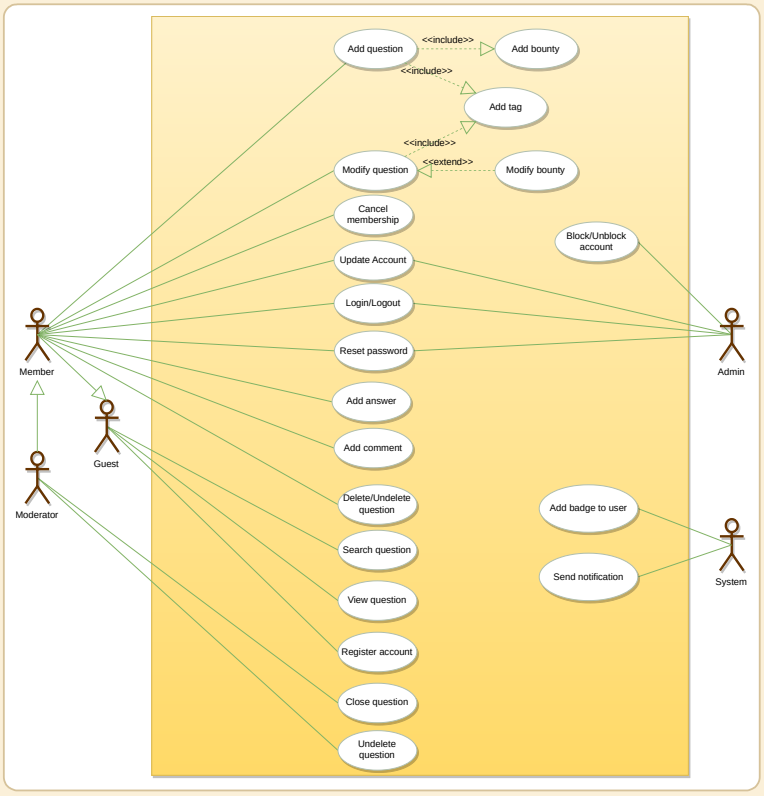


Figure — Use cases per actor: Guest (search, view, register), Member (add/modify question, answer, comment, bounty, tag), Moderator (close, delete/undelete), Admin (block/unblock), System (send notification, add badge) (course diagram)

A **use-case diagram** is a UML picture of the actors (stick figures) and the things each one can do (ovals). In the figure above, notice the `<<include>>` from "Add question" to "Add tag" (adding a tag is part of asking) and the `<<extend>>` for bounties (an optional extra on a question).

An **activity diagram** is a flowchart of the steps and yes/no decisions in one task; a **sequence diagram** shows, top to bottom in time order, the messages objects send to each other.

Activity — post a new question. The member clicks *Ask Question*, types a title and body, and adds tags. For each tag, the system checks whether it already exists; if not, it checks whether the member has **enough reputation to create a tag**. Then it validates the question and either saves and posts it or shows an error.

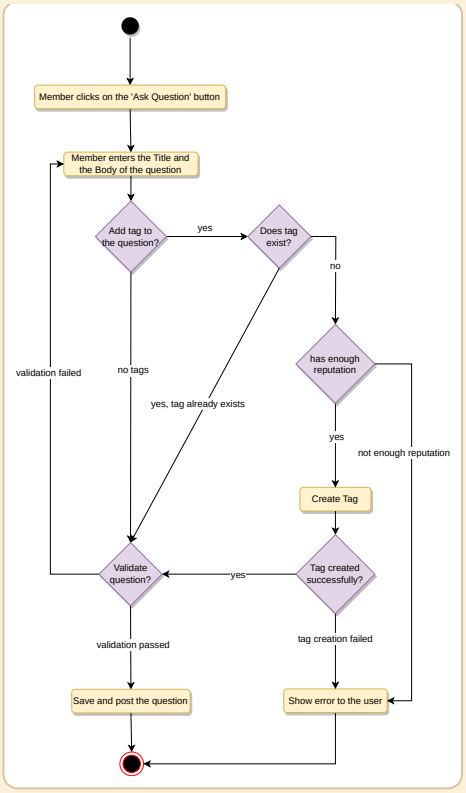


Figure — Activity diagram for posting a question, including the "does the tag exist?" and "has enough reputation?" decisions (course diagram)

Sequence — create a question. The member sets the title and body, looks up tags (creating one if missing), creates the question with a bounty value, the question creates its `Bounty`, and finally the question is published.

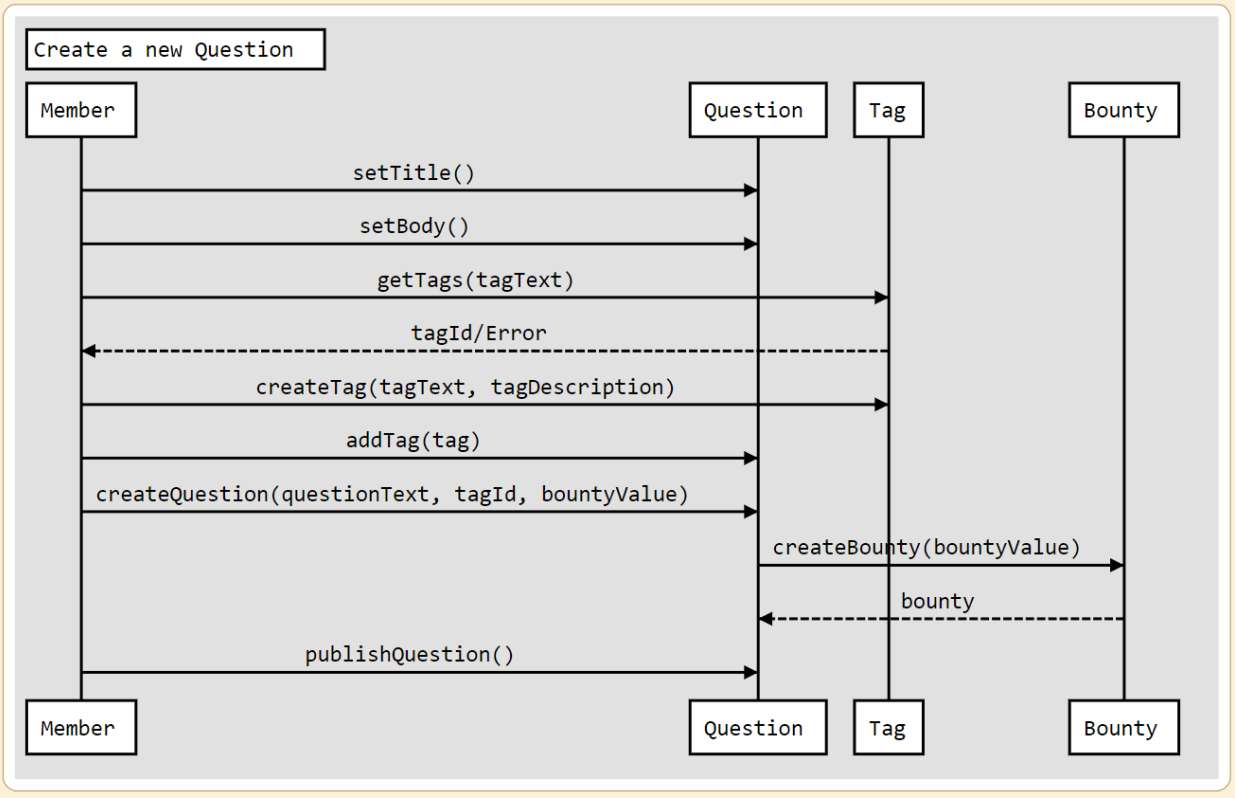


Figure — Sequence diagram: Member → Question (setTitle, setBody), Member → Tag (getTags / createTag), Question → Bounty (createBounty), then publishQuestion (course diagram)

Our Java code follows the same order: `ask()` resolves each tag, creating it only if the asker holds the `CREATE_TAG` privilege, and then builds the question; `offerBounty()` attaches the bounty.

Core entities

Class	Responsibility
`Member` (+ `Moderator`, `Admin`)	Identity, reputation (atomic — safe when updated by many threads at once), badges, account status; `hasPrivilege(p)`
`Privilege` (enum)	The reputation needed for each action — a data table, not scattered `if`s
`Post` (sealed, abstract)	Shared behaviour: author, body, votes per voter, flags, comments
`Question`	Title, tags, answers, status (OPEN / CLOSED / DELETED…), closing remark, close/delete voters, bounty, view count
`Answer`	Belongs to one question; can be *accepted*. (The course also gives answers and comments a view count and vote/flag counts; our code derives the vote and flag counts from the voter and flagger sets)
`Comment`	Belongs to a question or answer
`Tag`	Name + description; counts its own uses (for "top tags")
`Bounty`	Staked reputation, sponsor, expiry date, winner
`Badge` (record)	Name + description
`ReputationPolicy` / `DefaultReputationPolicy`	Strategy: how many points each event is worth
`ReputationListener` / `BadgeAwarder`	Observer: reacts to reputation changes (awards badges)
`Notifier` + `NotificationChannel` / `EmailChannel`	Sends notifications through pluggable channels
`Search` / `QuestionIndex`	Inverted index by word and by tag
`StackOverflow`	Facade: checks status + privilege, delegates, applies reputation

Class diagram

The course builds the class diagram in **five stages**, each adding a few classes for a few requirements. Classes from earlier stages are drawn in grey, so the picture fills in step by step.

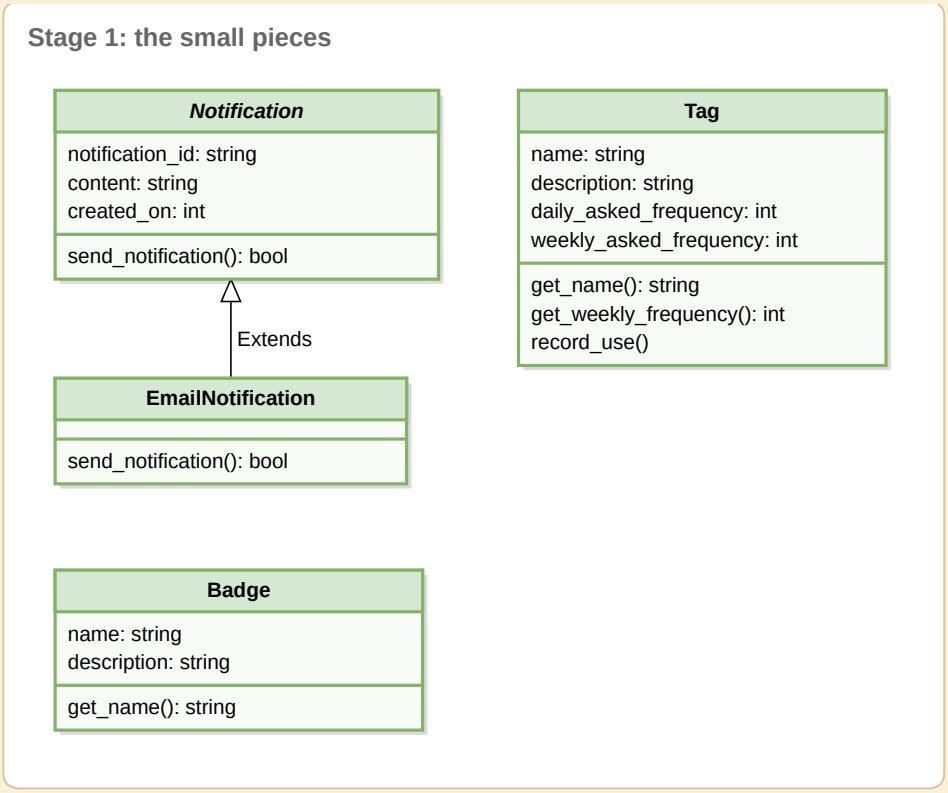


Figure — Stage 1: `Badge`, `Tag` (with daily and weekly usage counters and `record_use`) and abstract `Notification` with its `EmailNotification` subclass (course diagram)

Stage 1 covers requirements 8, 10 and 13. None of these classes depends on anything yet. The key idea is in `Tag`: it counts its own uses when attached to a question, so "most used tags" becomes a quick read rather than a scan of every question. `Notification` is abstract with one subclass per delivery method — an email notification genuinely is a notification that can be sent, so the inheritance passes the substitution test (the Liskov Substitution Principle: a subclass object must work correctly anywhere its parent class is expected).

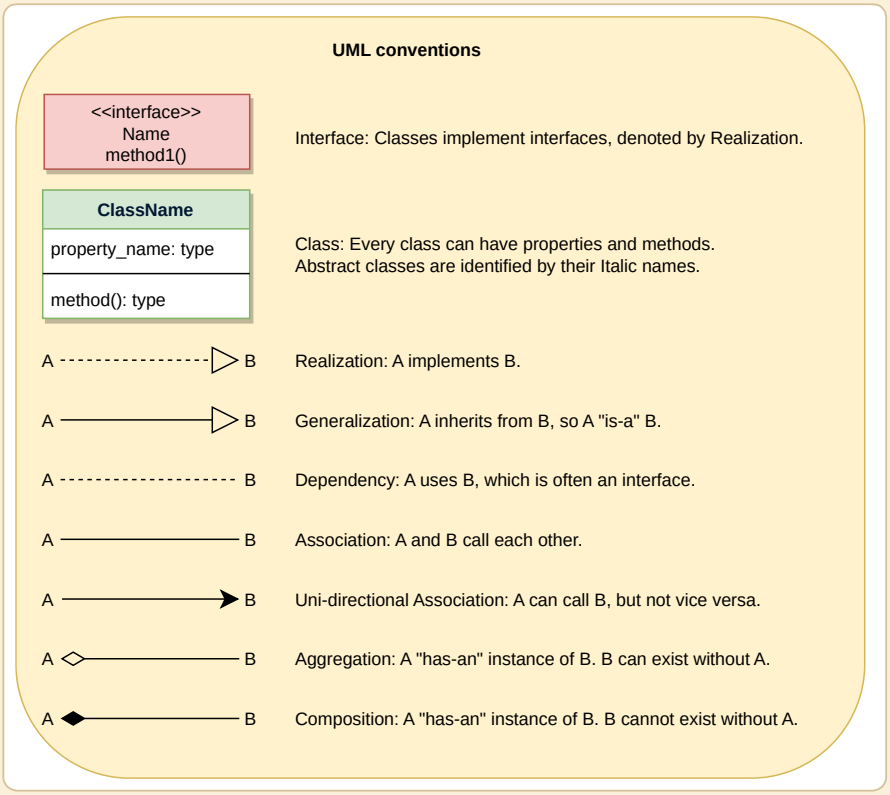


Figure — The UML legend used in these diagrams: class, interface, realization, generalization, dependency, association, aggregation and composition arrows (course diagram)

UML (Unified Modeling Language) is the standard notation for drawing classes and their links. In short: **generalization** is the inheritance arrow ("is a kind of"); **realization** means a class implements an interface;

dependency means one class merely uses another for a moment; **aggregation** (hollow diamond) means "has, but does not own"; **composition** (filled diamond) and **association** are explained in Stage 3 below.

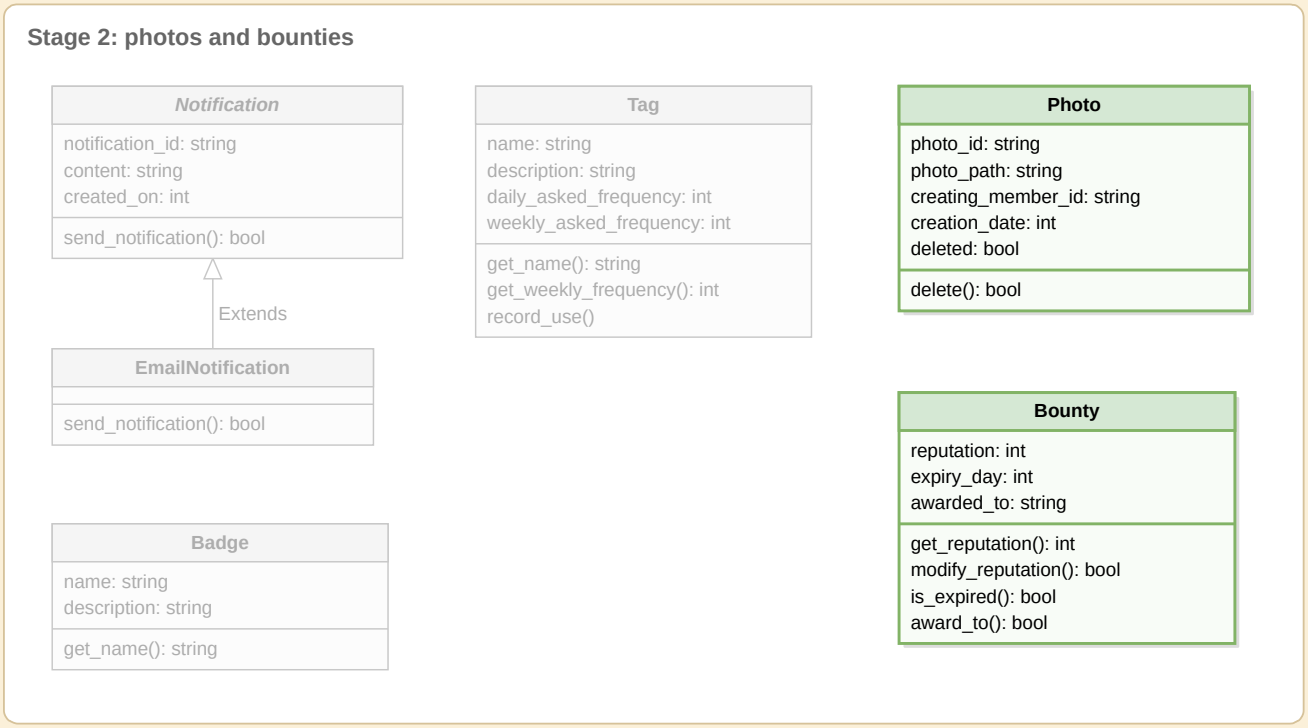


Figure — Stage 2 adds `Photo` (attachments on questions and answers) and `Bounty` (reputation held until awarded or expired) (course diagram)

Stage 2 covers requirement 7. The bounty keeps the staked amount itself rather than on the question, so the money-like value has one owner until it is settled. The course's reason for this is that the amount can then go back to the sponsor if nobody earns it. Be careful in an interview: on the real site a bounty is *not* refundable, so our code deducts the points when the bounty starts and never returns them.

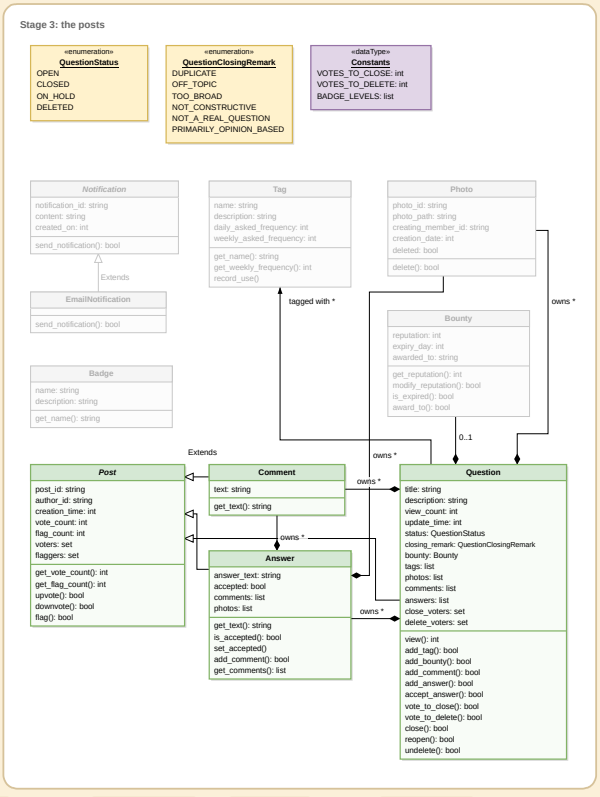


Figure — Stage 3: abstract `Post` with `Question`, `Answer` and `Comment` below it; filled diamonds from `Question` to its answers, comments, photos and bounty; a plain association from `Question` to `Tag` (course diagram)

Stage 3 is the heart of the design. Requirements 5 and 6 say that upvoting and flagging work the *same way* on three classes, so that behaviour is written once in `Post`. What *differs* is what each one contains, so they remain three separate classes instead of a single class plus a "kind" field. `Post` also remembers **who** voted —

otherwise one member could vote ten times. The filled diamonds are **composition**: deleting a question also deletes its answers, comments, photos and bounty. `Tag` is linked by a plain **association** instead: many questions use the same tag, and the tag lives on after any one of them is deleted. Because `Post` keeps only the author's id, the diagram has no arrow from a post to a member.

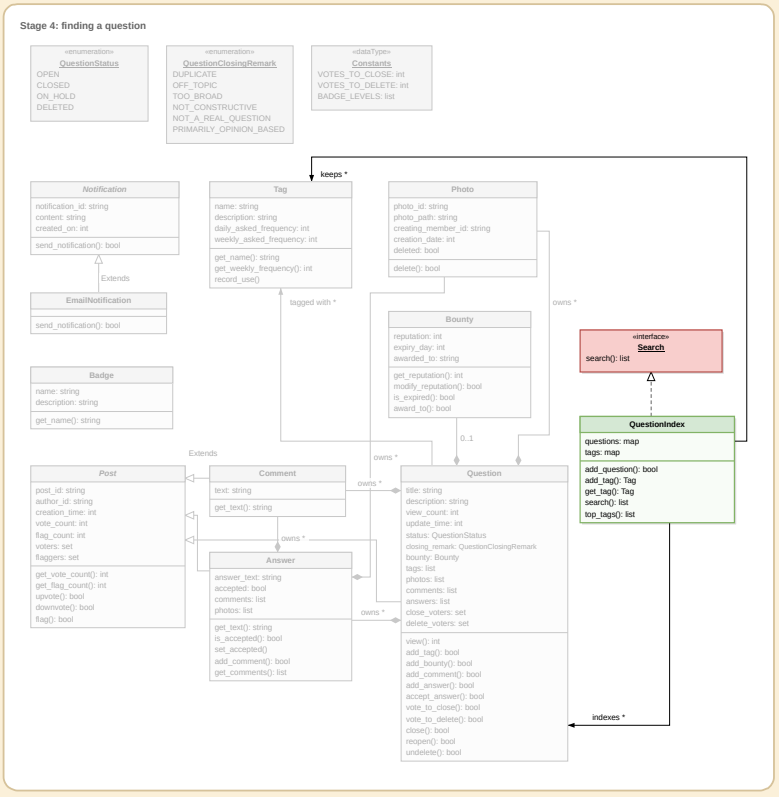


Figure — Stage 4: the `Search` interface and `QuestionIndex`, which holds (but does not own) questions and tags (course diagram)

Stage 4 covers search and requirement 13. A tempting first attempt makes search a static method (a method that belongs to the class itself rather than to any one object) on `Question`, which forces `Question` to know every question in existence — not its job. The index implements `Search` (dashed line with hollow head = realization). Its arrows are plain associations: deleting the index deletes no question.

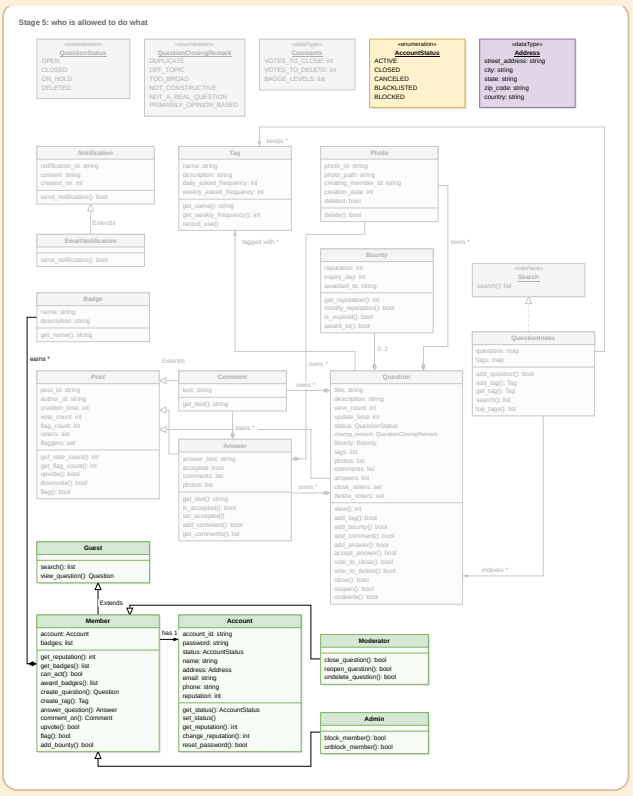


Figure — Stage 5: `Account`, `Guest`, `Member` (extends Guest), `Moderator` and `Admin` (extend Member); `Member` owns its badges (course diagram)

Stage 5 covers requirement 1 by the *shape* of the classes: the methods a guest may call sit on `Guest`, and `Member` inherits them and adds posting and voting. A guest simply has no `createQuestion` method, so no check can be forgotten. `Moderator` and `Admin` inherit from `Member` since each of them may do anything a member may (the substitution test passes). `Member` owns its badges (filled diamond): a badge is awarded to that one member. The link from a member to its `Account` is a plain association, because the account is created elsewhere and handed to the member.

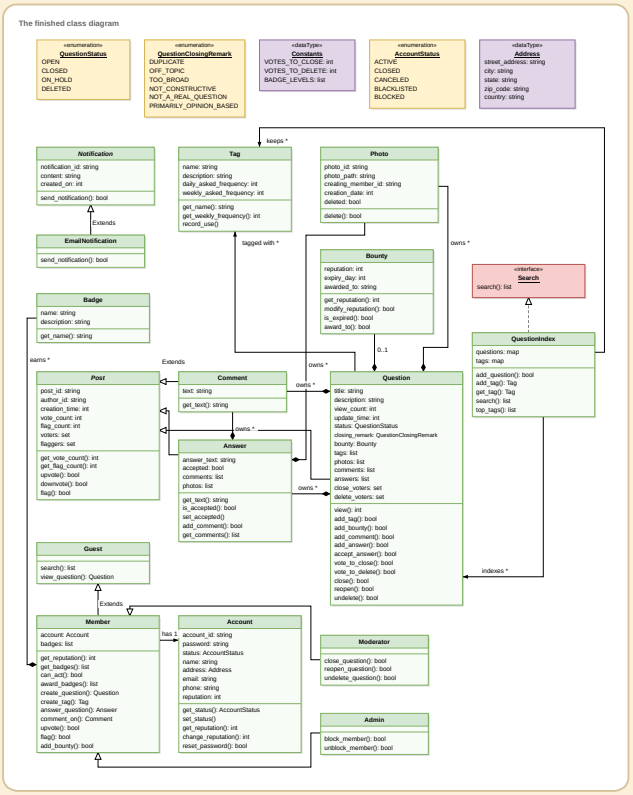
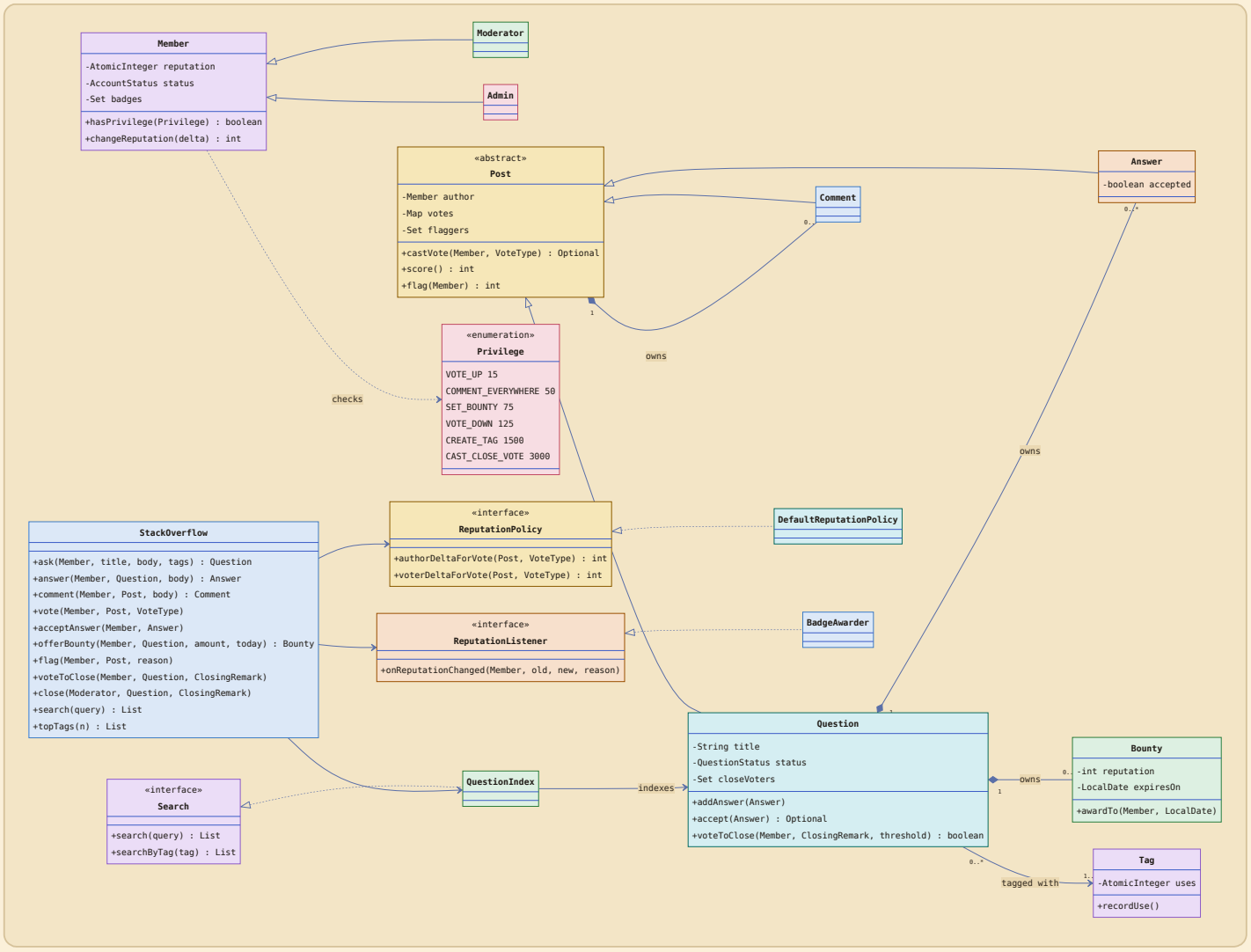


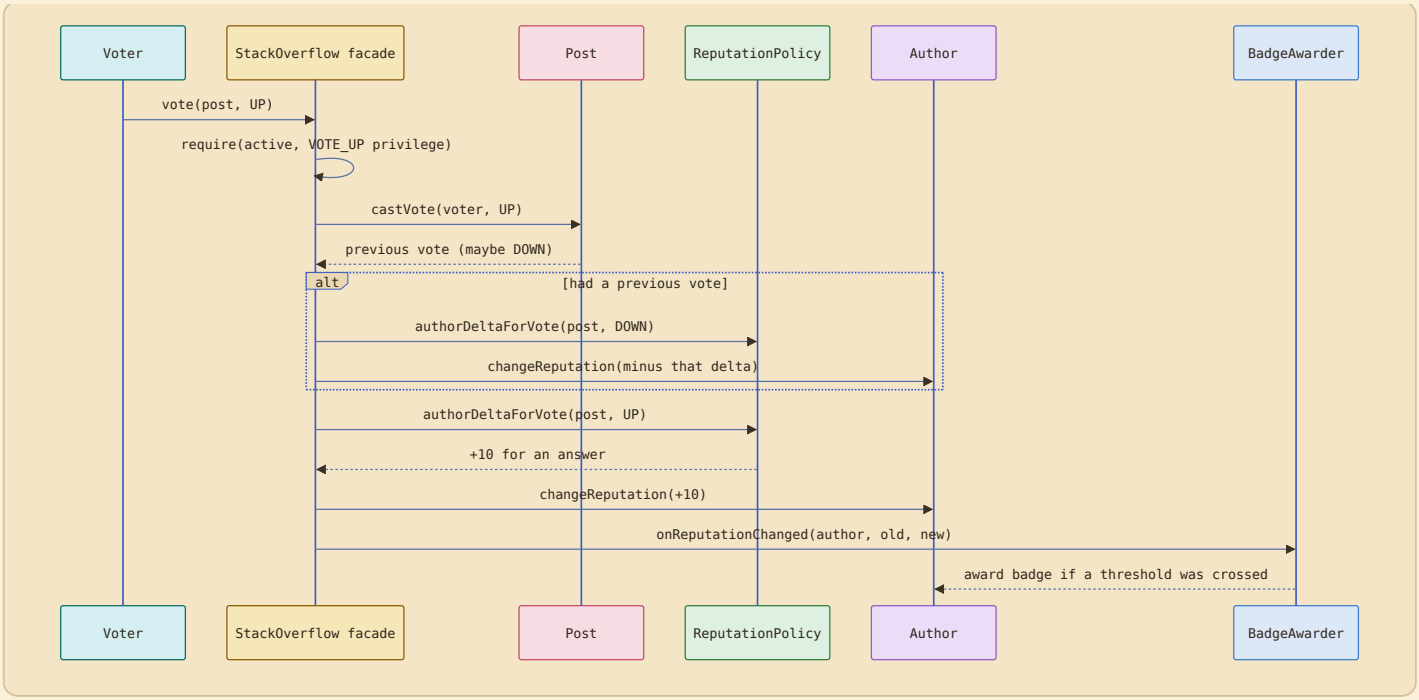
Figure — The finished Stack Overflow class diagram: enums (QuestionStatus, QuestionClosingRemark, AccountStatus), Constants, the posts, tags, badges, bounty, photo, notification, search index and the account hierarchy (course diagram)

Our Java design. It keeps the course's structure and adds three things the course's "what we left out" section invites: a reputation **Strategy**, a badge **Observer**, and a data-driven **privilege table** (because on the real site privileges depend on reputation, not only on account type). A guest in our code is simply "no `Member` object": every write operation on the facade requires a `Member` argument, so the type system still enforces requirement 1. The course also lists what it deliberately leaves out: edit history, revision review, linking duplicates, chat, and the reputation a downvoter pays. We add the downvote cost (it is one line in the policy); edit history and duplicate links are covered under Extensions below; chat stays out of scope.

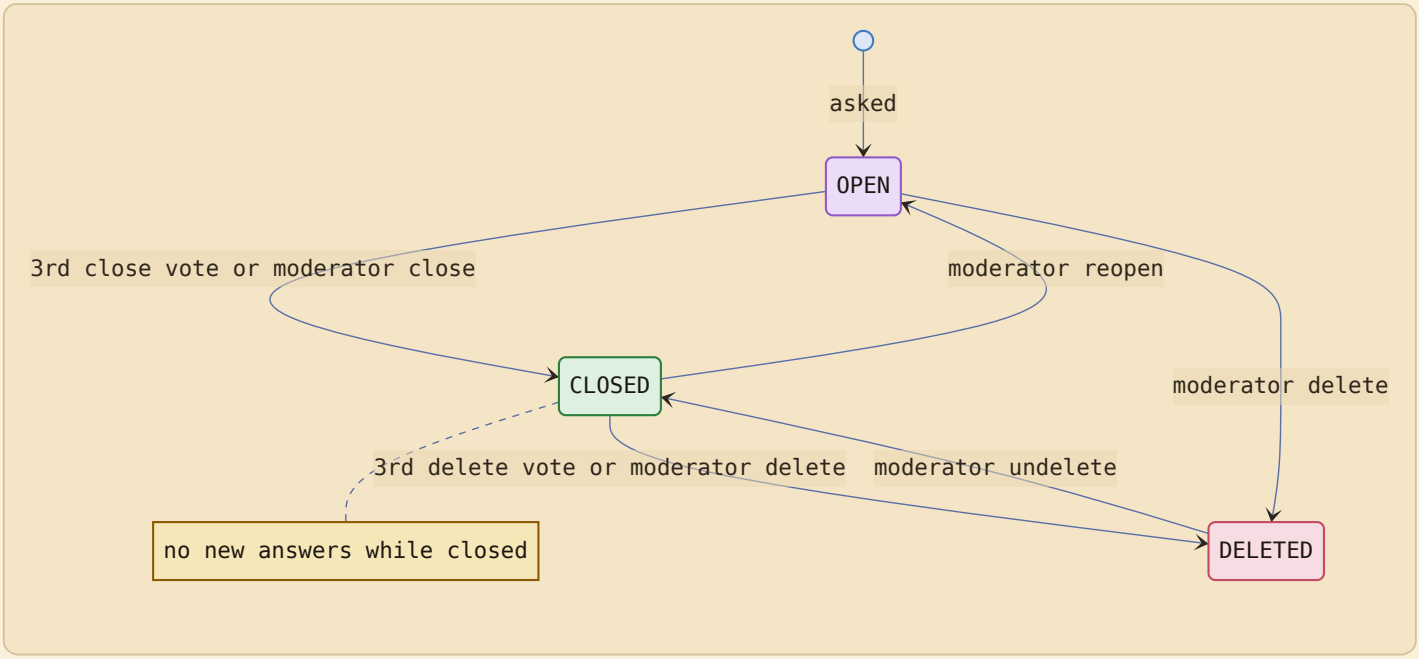


Key flows

Voting (with a changed vote). The facade checks privilege first, then asks the post to record the vote. The post returns the voter's *previous* vote, so the facade can reverse its reputation effect before applying the new one.



Question lifecycle.



Bounty. `offerBounty` checks the `SET_BOUNTY` privilege and the 50–500 range, attaches the bounty to the (open) question, and *immediately* deducts the amount from the sponsor. Later `awardBounty` (sponsor only, not to their own answer, before expiry) credits the winner.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`Post` base class for question, answer, comment	Inheritance (passes the substitution test), `sealed`	Voting, flagging and commenting are identical on all three	One `Post` class with a `type` field — then every method needs `if (type == …)`
Votes stored as `Map<voterId, VoteType>`	"Store who, not how many"	Blocks double votes; lets a vote be changed and its reputation reversed	An `int voteCount` — cannot tell who voted
`ReputationPolicy` interface	**Strategy**	Scoring rules change (Stack Overflow really did change them); voting code should not	Hard-coded numbers inside `vote()` — the course calls this out as a weakness
`ReputationListener` + `BadgeAwarder`	**Observer**	Badges, notifications, analytics all react to reputation without the facade knowing them	Calling `awardBadges()` directly inside `vote()`
`Privilege` enum with thresholds	Table-driven design	One place holds the rules; `hasPrivilege` is one comparison	`if (rep >= 125)` scattered across methods
Moderator-only methods take a `Moderator` parameter	Type-level permission	You cannot even call `close(…)` without a moderator object	A runtime role check that someone might forget
`QuestionIndex` separate from `Question`	Single Responsibility	Search can later be swapped for Elasticsearch (a popular search-engine server) behind `Search`	Static `Question.search()`
`Tag` counts its own uses	Maintain an aggregate at write time	"Top tags" becomes a sort of a small collection	Scanning all questions on every request
`StackOverflow` facade	**Facade**	One entry point that validates, delegates and logs	Letting the UI call `Question`, `Member`, `Bounty` directly
Pattern matching `switch` over the sealed `Post`	Exhaustive dispatch	The compiler forces us to handle each post kind in the policy	`instanceof` chains that silently miss a new type

Java implementation

Package `lld.ch18.stackoverflow`. Read the files in this order: the enums and privilege table (`Types.java`), accounts, posts, reputation Strategy/Observer, the search index, then the facade and the demo.

```java
// File: lld/ch18/stackoverflow/Types.java
package lld.ch18.stackoverflow;

/** Lifecycle of a question. ON_HOLD is the older name Stack Overflow used for a freshly closed question. */
enum QuestionStatus { OPEN, ON_HOLD, CLOSED, DELETED }

/** Why a question was closed. */
enum ClosingRemark {
    DUPLICATE, OFF_TOPIC, TOO_BROAD, NOT_CONSTRUCTIVE, NOT_A_REAL_QUESTION, PRIMARILY_OPINION_BASED
}

enum AccountStatus { ACTIVE, CLOSED, CANCELED, BLACKLISTED, BLOCKED }

enum VoteType { UP, DOWN }

/**
 * Reputation needed to unlock an action. The numbers follow the public
 * privileges page of stackoverflow.com, so "reputation unlocks privileges"
 * is data here, not if-statements spread over the code.
 */
enum Privilege {
    ASK_OR_ANSWER(1),
    VOTE_UP(15),
    FLAG(15),
    COMMENT_EVERYWHERE(50),
    SET_BOUNTY(75),
    VOTE_DOWN(125),
    CREATE_TAG(1500),
    EDIT_ANY_POST(2000),
    CAST_CLOSE_VOTE(3000),
    CAST_DELETE_VOTE(10000);

    final int minReputation;

    Privilege(int minReputation) { this.minReputation = minReputation; }
}

/** A badge is just a name and a description; members own the badges they earn. */
record Badge(String name, String description) {}
```

```java
// File: lld/ch18/stackoverflow/Accounts.java
package lld.ch18.stackoverflow;

import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

/**
 * A registered member. A guest has no Member object at all: every write
 * operation on the StackOverflow facade takes a Member, so "guests can only
 * search and view" is enforced by the method signatures.
 */
class Member {
    private static final AtomicLong IDS = new AtomicLong();

    private final long id = IDS.incrementAndGet();
    private final String name;
    private final String email;
    private final AtomicInteger reputation = new AtomicInteger(1);   // every account starts at 1
    private final Set<Badge> badges = Collections.synchronizedSet(new LinkedHashSet<>());
    private volatile AccountStatus status = AccountStatus.ACTIVE;

    Member(String name, String email) {
        this.name = name;
        this.email = email;
    }

    long id() { return id; }
    String name() { return name; }
    String email() { return email; }
    int reputation() { return reputation.get(); }
    AccountStatus status() { return status; }
    void setStatus(AccountStatus status) { this.status = status; }

    /** Applies a delta atomically; reputation never drops below 1 (the site's floor). Returns the new value. */
    int changeReputation(int delta) {
        return reputation.updateAndGet(r -> Math.max(1, r + delta));
    }

    /** Demo helper: pretend the member earned reputation before the demo started. */
    void seedReputation(int value) { reputation.set(value); }

    boolean hasPrivilege(Privilege p) { return reputation() >= p.minReputation; }

    boolean addBadge(Badge b) { return badges.add(b); }
    Set<Badge> badges() { synchronized (badges) { return Set.copyOf(badges); } }

    @Override public String toString() { return name + "(" + reputation() + ")"; }
}

/** Moderators can do everything a member can, whatever their reputation, plus close/reopen/delete/undelete any
question. */
class Moderator extends Member {
    Moderator(String name, String email) { super(name, email); }
    @Override boolean hasPrivilege(Privilege p) { return true; }
}

/** Admins block and unblock members. */
class Admin extends Member {
    Admin(String name, String email) { super(name, email); }
    @Override boolean hasPrivilege(Privilege p) { return true; }
}
```

```java
// File: lld/ch18/stackoverflow/Posts.java
package lld.ch18.stackoverflow;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

/**
 * Behaviour shared by questions, answers and comments: votes, flags and comments.
 * Votes are stored per voter (not as a bare counter) so one member cannot vote twice
 * and a vote can be changed or reversed.
 */
abstract sealed class Post permits Question, Answer, Comment {
    private static final AtomicLong IDS = new AtomicLong(100);

    private final long id = IDS.incrementAndGet();
    private final Member author;
    private volatile String body;
    private final Map<Long, VoteType> votes = new HashMap<>();
    private final Set<Long> flaggers = new HashSet<>();
    private final List<Comment> comments = new ArrayList<>();

    Post(Member author, String body) {
        this.author = author;
        this.body = body;
    }

    long id() { return id; }
    Member author() { return author; }
    String body() { return body; }
    void edit(String newBody) { this.body = newBody; }

    /** Records or changes a vote and returns the voter's previous vote, if any. */
    synchronized Optional<VoteType> castVote(Member voter, VoteType type) {
        if (voter.id() == author.id()) throw new IllegalStateException("you cannot vote on your own post");
        VoteType previous = votes.put(voter.id(), type);
        if (previous == type) throw new IllegalStateException(voter.name() + " already voted " + type + " on this
post");
        return Optional.ofNullable(previous);
    }

    synchronized int score() {
        return votes.values().stream().mapToInt(v -> v == VoteType.UP ? 1 : -1).sum();
    }

    /** Returns the number of distinct flaggers after this flag. */
    synchronized int flag(Member by) {
        if (!flaggers.add(by.id())) throw new IllegalStateException(by.name() + " already flagged this post");
        return flaggers.size();
    }

    synchronized void addComment(Comment c) { comments.add(c); }
    synchronized List<Comment> comments() { return List.copyOf(comments); }

    abstract String label();
}

final class Question extends Post {
    private final String title;
    private final Set<Tag> tags;
```

```
    private final List<Answer> answers = new ArrayList<>();
    private final Set<Long> closeVoters = new HashSet<>();
    private final Set<Long> deleteVoters = new HashSet<>();
    private final AtomicInteger views = new AtomicInteger();
    private QuestionStatus status = QuestionStatus.OPEN;
    private ClosingRemark closingRemark;
    private Bounty bounty;

    Question(Member author, String title, String body, Set<Tag> tags) {
        super(author, body);
        this.title = title;
        this.tags = new LinkedHashSet<>(tags);
        tags.forEach(Tag::recordUse);            // a tag counts its own uses: "top tags" becomes a read
    }

    String title() { return title; }
    Set<Tag> tags() { return Set.copyOf(tags); }
    int view() { return views.incrementAndGet(); }
    int views() { return views.get(); }

    synchronized QuestionStatus status() { return status; }
    synchronized Optional<ClosingRemark> closingRemark() { return Optional.ofNullable(closingRemark); }
    synchronized List<Answer> answers() { return List.copyOf(answers); }
    synchronized Optional<Bounty> bounty() { return Optional.ofNullable(bounty); }

    synchronized void addAnswer(Answer a) {
        if (status != QuestionStatus.OPEN) throw new IllegalStateException("question is " + status + ", no new
answers");
        answers.add(a);
    }

    /** Marks one answer accepted and returns the previously accepted one (only one answer can win). */
    synchronized Optional<Answer> accept(Answer chosen) {
        if (!answers.contains(chosen)) throw new IllegalArgumentException("answer does not belong to this
question");
        Optional<Answer> previous = answers.stream().filter(Answer::isAccepted).findFirst();
        previous.ifPresent(p -> p.setAccepted(false));
        chosen.setAccepted(true);
        return previous;
    }

    synchronized void attachBounty(Bounty b) {
        if (status != QuestionStatus.OPEN) throw new IllegalStateException("bounties need an open question");
        if (bounty != null && !bounty.isSettled()) throw new IllegalStateException("question already has an active
bounty");
        bounty = b;
    }

    /** One close vote per member; returns true when this vote closed the question. */
    synchronized boolean voteToClose(Member voter, ClosingRemark remark, int threshold) {
        if (status != QuestionStatus.OPEN) throw new IllegalStateException("question is already " + status);
        if (!closeVoters.add(voter.id())) throw new IllegalStateException(voter.name() + " already voted to
close");
        if (closeVoters.size() >= threshold) { close(remark); return true; }
        return false;
    }

    synchronized boolean voteToDelete(Member voter, int threshold) {
        if (status != QuestionStatus.CLOSED) throw new IllegalStateException("only closed questions can be deleted
by vote");
        if (!deleteVoters.add(voter.id())) throw new IllegalStateException(voter.name() + " already voted to
delete");
        if (deleteVoters.size() >= threshold) { status = QuestionStatus.DELETED; return true; }
        return false;
    }

    synchronized void close(ClosingRemark remark) { status = QuestionStatus.CLOSED; closingRemark = remark; }
    synchronized void reopen() { status = QuestionStatus.OPEN; closingRemark = null; closeVoters.clear(); }
    synchronized void delete() { status = QuestionStatus.DELETED; }
```

```
    synchronized void undelete() { status = QuestionStatus.CLOSED; deleteVoters.clear(); }

    @Override String label() { return "question #" + id() + " \"" + title + "\""; }
}

final class Answer extends Post {
    private final Question question;
    private volatile boolean accepted;

    Answer(Member author, Question question, String body) {
        super(author, body);
        this.question = question;
    }

    Question question() { return question; }
    boolean isAccepted() { return accepted; }
    void setAccepted(boolean accepted) { this.accepted = accepted; }

    @Override String label() { return "answer #" + id() + " by " + author().name(); }
}

final class Comment extends Post {
    private final Post parent;

    Comment(Member author, Post parent, String body) {
        super(author, body);
        this.parent = parent;
    }

    Post parent() { return parent; }
    @Override String label() { return "comment #" + id(); }
}

/** A tag is shared by many questions and outlives each of them (association, not composition). */
final class Tag {
    private final String name;
    private final String description;
    private final AtomicInteger uses = new AtomicInteger();

    Tag(String name, String description) { this.name = name; this.description = description; }

    String name() { return name; }
    String description() { return description; }
    void recordUse() { uses.incrementAndGet(); }
    int uses() { return uses.get(); }
    @Override public String toString() { return "[" + name + "]"; }
}

/** Reputation the asker has staked; it leaves the asker's account when the bounty starts. */
final class Bounty {
    private final Member sponsor;
    private final int reputation;
    private final LocalDate expiresOn;
    private Member awardedTo;
    private boolean expired;

    Bounty(Member sponsor, int reputation, LocalDate expiresOn) {
        this.sponsor = sponsor;
        this.reputation = reputation;
        this.expiresOn = expiresOn;
    }

    Member sponsor() { return sponsor; }
    int reputation() { return reputation; }
    LocalDate expiresOn() { return expiresOn; }
    synchronized boolean isSettled() { return awardedTo != null || expired; }

    synchronized void awardTo(Member winner, LocalDate today) {
        if (isSettled()) throw new IllegalStateException("bounty already settled");
```

```
        if (today.isAfter(expiresOn)) throw new IllegalStateException("bounty expired on " + expiresOn);
        awardedTo = winner;
    }

    synchronized void expire() { expired = true; }
}
```

```java
// File: lld/ch18/stackoverflow/Reputation.java
package lld.ch18.stackoverflow;

import java.util.List;

/**
 * Strategy: the scoring rules live behind an interface, so voting code does not
 * know the numbers. Changing a rule (e.g. the 2019 move of question upvotes
 * from +5 to +10) is a new
 * policy, not an edit to the voting code (Open/Closed Principle).
 */
interface ReputationPolicy {
    int authorDeltaForVote(Post post, VoteType type);
    int voterDeltaForVote(Post post, VoteType type);
    int acceptedAnswerBonus();
    int acceptorBonus();
}

/** Today's stackoverflow.com rules: +10 for any upvote, -2 to the author per downvote, -1 to whoever downvotes an answer. */
final class DefaultReputationPolicy implements ReputationPolicy {
    @Override public int authorDeltaForVote(Post post, VoteType type) {
        return switch (post) {
            case Question q -> type == VoteType.UP ? 10 : -2;   // +5 before November 2019
            case Answer a -> type == VoteType.UP ? 10 : -2;
            case Comment c -> 0;                        // comment votes earn no reputation
        };
    }
    @Override public int voterDeltaForVote(Post post, VoteType type) {
        return post instanceof Answer && type == VoteType.DOWN ? -1 : 0;
    }
    @Override public int acceptedAnswerBonus() { return 15; }
    @Override public int acceptorBonus() { return 2; }
}

/** Observer: anything that wants to react to a reputation change. */
interface ReputationListener {
    void onReputationChanged(Member member, int oldValue, int newValue, String reason);
}

/** Awards reputation-level badges (course thresholds: 15, 50, 125). */
final class BadgeAwarder implements ReputationListener {
    private record Level(int reputation, Badge badge) {}

    private final List<Level> levels = List.of(
            new Level(15, new Badge("Contributor", "reached 15 reputation")),
            new Level(50, new Badge("Helpful", "reached 50 reputation")),
            new Level(125, new Badge("Trusted", "reached 125 reputation")));
    private final Notifier notifier;

    BadgeAwarder(Notifier notifier) { this.notifier = notifier; }

    @Override public void onReputationChanged(Member m, int oldValue, int newValue, String reason) {
        for (Level level : levels) {
            if (oldValue < level.reputation() && newValue >= level.reputation() && m.addBadge(level.badge())) {
                notifier.send(m, "You earned the '" + level.badge().name() + "' badge");
            }
        }
    }
}

/** Delivery channel for notifications (Strategy: email today, push tomorrow). */
interface NotificationChannel {
    void deliver(Member to, String text);
}

final class EmailChannel implements NotificationChannel {
    @Override public void deliver(Member to, String text) {
```

```
        System.out.println("    [email to " + to.email() + "] " + text);
    }
}

/** The "System" actor: sends notifications through every configured channel. */
final class Notifier {
    private final List<NotificationChannel> channels;
    Notifier(List<NotificationChannel> channels) { this.channels = List.copyOf(channels); }
    void send(Member to, String text) { channels.forEach(c -> c.deliver(to, text)); }
}
```

```java
// File: lld/ch18/stackoverflow/QuestionIndex.java
package lld.ch18.stackoverflow;

import java.util.Arrays;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

/** Search contract, kept separate so a full-text engine could replace the in-memory index. */
interface Search {
    List<Question> search(String query);
    List<Question> searchByTag(String tagName);
}

/**
 * An inverted index: word -> questions containing it. It refers to questions
 * but does not own them (throwing the index away deletes no question).
 */
final class QuestionIndex implements Search {
    private final Map<String, Set<Question>> byWord = new ConcurrentHashMap<>();
    private final Map<String, Set<Question>> byTag = new ConcurrentHashMap<>();

    void add(Question q) {
        for (String word : words(q.title() + " " + q.body())) {
            byWord.computeIfAbsent(word, k -> ConcurrentHashMap.newKeySet()).add(q);
        }
        for (Tag t : q.tags()) {
            byTag.computeIfAbsent(t.name(), k -> ConcurrentHashMap.newKeySet()).add(q);
        }
    }

    /** Questions containing every word of the query, best score first; deleted questions are hidden. */
    @Override public List<Question> search(String query) {
        List<String> terms = words(query);
        if (terms.isEmpty()) return List.of();
        Set<Question> result = new HashSet<>(byWord.getOrDefault(terms.get(0), Set.of()));
        for (String t : terms.subList(1, terms.size())) result.retainAll(byWord.getOrDefault(t, Set.of()));
        return rank(result);
    }

    @Override public List<Question> searchByTag(String tagName) {
        return rank(byTag.getOrDefault(tagName.toLowerCase(Locale.ROOT), Set.of()));
    }

    private static List<Question> rank(Set<Question> qs) {
        return qs.stream()
                .filter(q -> q.status() != QuestionStatus.DELETED)
                .sorted(Comparator.comparingInt(Question::score).reversed().thenComparingLong(Question::id))
                .toList();
    }

    private static List<String> words(String text) {
        return Arrays.stream(text.toLowerCase(Locale.ROOT).split("[^a-z0-9#+]+"))
                .filter(w -> w.length() > 2)
                .distinct()
                .toList();
    }
}
```

```java
// File: lld/ch18/stackoverflow/StackOverflow.java
package lld.ch18.stackoverflow;

import java.time.LocalDate;
import java.util.Comparator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

/**
 * Facade: the one entry point the UI/controllers call. It checks who may do
 * what (account status + privilege), delegates state changes to the posts,
 * applies reputation through the policy, and tells listeners.
 */
public final class StackOverflow {
    static final int VOTES_TO_CLOSE = 3;
    static final int VOTES_TO_DELETE = 3;
    static final int FLAGS_FOR_MODERATOR_REVIEW = 3;
    static final int MIN_BOUNTY = 50;
    static final int MAX_BOUNTY = 500;
    static final int BOUNTY_DAYS = 7;

    private final Map<String, Tag> tags = new ConcurrentHashMap<>();
    private final Map<Long, Member> members = new ConcurrentHashMap<>();
    private final QuestionIndex index = new QuestionIndex();
    private final ReputationPolicy policy;
    private final Notifier notifier;
    private final List<ReputationListener> listeners = new CopyOnWriteArrayList<>();
    private final List<Moderator> moderators = new CopyOnWriteArrayList<>();

    StackOverflow(ReputationPolicy policy, Notifier notifier) {
        this.policy = policy;
        this.notifier = notifier;
    }

    void addReputationListener(ReputationListener l) { listeners.add(l); }

    // ---------- accounts ----------
    <M extends Member> M register(M member) {
        members.put(member.id(), member);
        if (member instanceof Moderator mod) moderators.add(mod);
        return member;
    }

    void block(Admin admin, Member target) { target.setStatus(AccountStatus.BLOCKED); log(admin.name() + " blocked
" + target.name()); }
    void unblock(Admin admin, Member target) { target.setStatus(AccountStatus.ACTIVE); log(admin.name() + "
unblocked " + target.name()); }

    // ---------- guest operations (no Member needed) ----------
    List<Question> search(String query) { return index.search(query); }
    List<Question> searchByTag(String tag) { return index.searchByTag(tag); }
    int view(Question q) { return q.view(); }

    /** Requirement 13: the most used tags; each tag counted itself, so this is a sort, not a scan of all
questions. */
    List<Tag> topTags(int n) {
        return tags.values().stream()
                .sorted(Comparator.comparingInt(Tag::uses).reversed().thenComparing(Tag::name))
                .limit(n).toList();
    }

    // ---------- tags ----------
```

```
    Tag createTag(Member by, String name, String description) {
        require(by, Privilege.CREATE_TAG);
        return tags.computeIfAbsent(name.toLowerCase(Locale.ROOT), k -> new Tag(k, description));
    }

    // ---------- posting ----------
    Question ask(Member asker, String title, String body, List<String> tagNames) {
        require(asker, Privilege.ASK_OR_ANSWER);
        Set<Tag> resolved = new LinkedHashSet<>();
        for (String raw : tagNames) {
            String name = raw.toLowerCase(Locale.ROOT);
            Tag tag = tags.get(name);
            if (tag == null) tag = createTag(asker, name, "");    // throws if the asker may not create tags
            resolved.add(tag);
        }
        Question q = new Question(asker, title, body, resolved);
        index.add(q);
        log(asker.name() + " asked " + q.label() + " " + resolved);
        return q;
    }

    Answer answer(Member author, Question q, String body) {
        require(author, Privilege.ASK_OR_ANSWER);
        Answer a = new Answer(author, q, body);
        q.addAnswer(a);                                          // throws unless the question is OPEN
        log(author.name() + " posted " + a.label() + " on question #" + q.id());
        notifier.send(q.author(), author.name() + " answered \"" + q.title() + "\"");
        return a;
    }

    Comment comment(Member author, Post target, String body) {
        requireActive(author);
        boolean ownThread = target.author().id() == author.id()
                || (target instanceof Answer a && a.question().author().id() == author.id());
        if (!ownThread) require(author, Privilege.COMMENT_EVERYWHERE);
        Comment c = new Comment(author, target, body);
        target.addComment(c);
        log(author.name() + " commented on " + target.label() + ": \"" + body + "\"");
        if (target.author().id() != author.id()) notifier.send(target.author(), author.name() + " commented on your
" + target.label());
        return c;
    }

    // ---------- voting & reputation ----------
    void vote(Member voter, Post post, VoteType type) {
        require(voter, type == VoteType.UP ? Privilege.VOTE_UP : Privilege.VOTE_DOWN);
        if (post instanceof Comment && type == VoteType.DOWN) throw new IllegalStateException("comments can only be
upvoted");
        Optional<VoteType> previous = post.castVote(voter, type);
        log(voter.name() + " voted " + type + " on " + post.label() + " -> score " + post.score());
        previous.ifPresent(p -> {                                  // undo the old vote's effect first
            adjust(post.author(), -policy.authorDeltaForVote(post, p), "vote changed on " + post.label());
            adjust(voter, -policy.voterDeltaForVote(post, p), "vote changed");
        });
        adjust(post.author(), policy.authorDeltaForVote(post, type), type + " on " + post.label());
        adjust(voter, policy.voterDeltaForVote(post, type), "downvoted an answer");
    }

    void acceptAnswer(Member asker, Answer answer) {
        requireActive(asker);
        Question q = answer.question();
        if (q.author().id() != asker.id()) throw new IllegalStateException("only the asker can accept an answer");
        Optional<Answer> previous = q.accept(answer);
        log(asker.name() + " accepted " + answer.label());
        previous.ifPresent(p -> adjust(p.author(), -policy.acceptedAnswerBonus(), "acceptance moved away"));
        adjust(answer.author(), policy.acceptedAnswerBonus(), "answer accepted");
        if (previous.isEmpty() && answer.author().id() != asker.id()) adjust(asker, policy.acceptorBonus(),
"accepted an answer");
    }
```

```
    // ---------- bounties ----------
    Bounty offerBounty(Member sponsor, Question q, int amount, LocalDate today) {
        require(sponsor, Privilege.SET_BOUNTY);
        if (amount < MIN_BOUNTY || amount > MAX_BOUNTY) throw new IllegalArgumentException("bounty must be " +
MIN_BOUNTY + ".." + MAX_BOUNTY);
        if (sponsor.reputation() - amount < 1) throw new IllegalStateException("not enough reputation to stake " +
amount);
        Bounty b = new Bounty(sponsor, amount, today.plusDays(BOUNTY_DAYS));
        q.attachBounty(b);
        adjust(sponsor, -amount, "bounty staked");          // staked reputation leaves the sponsor now
        log(sponsor.name() + " offered a +" + amount + " bounty on question #" + q.id() + " (expires " +
b.expiresOn() + ")");
        return b;
    }

    void awardBounty(Member sponsor, Answer winner, LocalDate today) {
        Bounty b = winner.question().bounty().orElseThrow(() -> new IllegalStateException("no bounty on this
question"));
        if (b.sponsor().id() != sponsor.id()) throw new IllegalStateException("only the sponsor awards the
bounty");
        if (winner.author().id() == sponsor.id()) throw new IllegalStateException("you cannot award your own
answer");
        b.awardTo(winner.author(), today);
        adjust(winner.author(), b.reputation(), "bounty awarded");
        log(sponsor.name() + " awarded the bounty to " + winner.label());
    }

    // ---------- moderation ----------
    void flag(Member by, Post post, String reason) {
        require(by, Privilege.FLAG);
        int count = post.flag(by);
        log(by.name() + " flagged " + post.label() + " (" + reason + "), flags=" + count);
        if (count == FLAGS_FOR_MODERATOR_REVIEW) {
            moderators.forEach(m -> notifier.send(m, post.label() + " needs moderator attention"));
        }
    }

    void voteToClose(Member voter, Question q, ClosingRemark remark) {
        require(voter, Privilege.CAST_CLOSE_VOTE);
        boolean closed = q.voteToClose(voter, remark, VOTES_TO_CLOSE);
        log(voter.name() + " voted to close question #" + q.id() + " as " + remark + (closed ? " -> CLOSED" : ""));
    }

    void voteToDelete(Member voter, Question q) {
        require(voter, Privilege.CAST_DELETE_VOTE);
        boolean deleted = q.voteToDelete(voter, VOTES_TO_DELETE);
        log(voter.name() + " voted to delete question #" + q.id() + (deleted ? " -> DELETED" : ""));
    }

    // Moderator-only operations: the parameter type is the permission check.
    void close(Moderator m, Question q, ClosingRemark remark) { q.close(remark); log(m.name() + " (mod) closed
question #" + q.id() + " as " + remark); }
    void reopen(Moderator m, Question q) { q.reopen(); log(m.name() + " (mod) reopened question #" + q.id()); }
    void delete(Moderator m, Question q) { q.delete(); log(m.name() + " (mod) deleted question #" + q.id()); }
    void undelete(Moderator m, Question q) { q.undelete(); log(m.name() + " (mod) undeleted question #" + q.id() +
" -> " + q.status()); }

    // ---------- helpers ----------
    private void adjust(Member m, int delta, String reason) {
        if (delta == 0) return;
        int before = m.reputation();
        int after = m.changeReputation(delta);
        log(String.format("    rep %-6s %+d -> %d (%s)", m.name(), delta, after, reason));
        listeners.forEach(l -> l.onReputationChanged(m, before, after, reason));
    }

    private static void requireActive(Member m) {
        if (m.status() != AccountStatus.ACTIVE) throw new IllegalStateException(m.name() + " is " + m.status());
```

```
    }

    private static void require(Member m, Privilege p) {
        requireActive(m);
        if (!m.hasPrivilege(p)) {
            throw new IllegalStateException(m.name() + " needs " + p.minReputation + " reputation for " + p + "
(has " + m.reputation() + ")");
        }
    }

    private static void log(String s) { System.out.println(s); }
}
```

```java
// File: lld/ch18/stackoverflow/Main.java
package lld.ch18.stackoverflow;

import java.time.LocalDate;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Notifier notifier = new Notifier(List.of(new EmailChannel()));
        StackOverflow so = new StackOverflow(new DefaultReputationPolicy(), notifier);
        so.addReputationListener(new BadgeAwarder(notifier));

        Member alice = so.register(new Member("alice", "alice@example.com"));
        Member bob = so.register(new Member("bob", "bob@example.com"));
        Member carol = so.register(new Member("carol", "carol@example.com"));
        Member erin = so.register(new Member("erin", "erin@example.com"));
        Member dave = so.register(new Member("dave", "dave@example.com"));
        Moderator mia = so.register(new Moderator("mia", "mia@example.com"));
        Admin root = so.register(new Admin("root", "root@example.com"));
        alice.seedReputation(200);
        bob.seedReputation(3500);
        carol.seedReputation(3200);
        erin.seedReputation(3100);
        LocalDate today = LocalDate.of(2026, 9, 28);

        section("1. Tags and asking");
        so.createTag(bob, "java", "The Java language");
        so.createTag(bob, "collections", "java.util collections");
        attempt(() -> so.ask(alice, "How does HashMap handle collisions?", "What happens when two keys land in the
same bucket?",
                List.of("java", "hashmap")));
        Question q1 = so.ask(alice, "How does HashMap handle collisions?", "What happens when two keys land in the
same bucket?",
                List.of("java", "collections"));

        section("2. Answers, votes, reputation");
        Answer bobs = so.answer(bob, q1, "Entries share a bucket as a linked list, turned into a tree when the bin
grows past 8.");
        Answer carols = so.answer(carol, q1, "It uses open addressing.");
        attempt(() -> so.vote(dave, q1, VoteType.UP));              // dave has 1 rep
        attempt(() -> so.vote(alice, q1, VoteType.UP));             // own post
        so.vote(bob, q1, VoteType.UP);
        so.vote(carol, bobs, VoteType.UP);
        so.vote(erin, carols, VoteType.DOWN);
        attempt(() -> so.vote(erin, carols, VoteType.DOWN));        // double vote
        so.vote(erin, carols, VoteType.UP);                         // changing a vote reverses the old effect
        so.comment(erin, carols, "HashMap uses chaining, not open addressing.");
        attempt(() -> so.comment(dave, carols, "+1"));              // needs 50 rep to comment everywhere
        so.acceptAnswer(alice, bobs);

        section("3. Bounty");
        Question q2 = so.ask(alice, "Why is ConcurrentHashMap faster than synchronizedMap?", "Under many threads.",
List.of("java"));
        so.offerBounty(alice, q2, 50, today);
        Answer carolsQ2 = so.answer(carol, q2, "Reads take no lock and writes lock only one bin.");
        so.awardBounty(alice, carolsQ2, today.plusDays(3));

        section("4. Badges for a new member");
        Question q3 = so.ask(dave, "Is Java pass-by-value?", "Objects seem to be passed by reference.",
List.of("java"));
        so.vote(bob, q3, VoteType.UP);
        so.vote(carol, q3, VoteType.UP);
        so.vote(erin, q3, VoteType.UP);
        System.out.println("dave's badges: " + dave.badges().stream().map(Badge::name).toList());

        section("5. Moderation");
        so.voteToClose(bob, q3, ClosingRemark.DUPLICATE);
        so.voteToClose(carol, q3, ClosingRemark.DUPLICATE);
```

```
        so.voteToClose(erin, q3, ClosingRemark.DUPLICATE);
        attempt(() -> so.answer(bob, q3, "Yes, always by value."));
        so.reopen(mia, q3);
        Comment rude = so.comment(dave, q3, "you all are wrong!!!");
        so.flag(bob, rude, "rude");
        so.flag(carol, rude, "rude");
        so.flag(erin, rude, "rude");
        so.delete(mia, q3);
        so.undelete(mia, q3);
        so.block(root, dave);
        attempt(() -> so.ask(dave, "Another one", "text", List.of("java")));

        section("6. Guest search and top tags");
        so.search("hashmap collisions").forEach(q -> System.out.println("found: " + q.label() + " score=" +
q.score()
                + " answers=" + q.answers().size() + " status=" + q.status()));
        so.searchByTag("java").forEach(q -> System.out.println("[java] " + q.label()));
        System.out.println("top tags: " + so.topTags(2).stream().map(t -> t.name() + "x" + t.uses()).toList());
        System.out.println("final reputation: " + List.of(alice, bob, carol, erin, dave));
    }

    private static void section(String title) { System.out.println("\n== " + title + " =="); }

    private static void attempt(Runnable action) {
        try { action.run(); }
        catch (RuntimeException e) { System.out.println("  REJECTED: " + e.getMessage()); }
    }
}
```

Output:

```
== 1. Tags and asking ==
  REJECTED: alice needs 1500 reputation for CREATE_TAG (has 200)
alice asked question #101 "How does HashMap handle collisions?" [[java], [collections]]

== 2. Answers, votes, reputation ==
bob posted answer #102 by bob on question #101
    [email to alice@example.com] bob answered "How does HashMap handle collisions?"
carol posted answer #103 by carol on question #101
    [email to alice@example.com] carol answered "How does HashMap handle collisions?"
  REJECTED: dave needs 15 reputation for VOTE_UP (has 1)
  REJECTED: you cannot vote on your own post
bob voted UP on question #101 "How does HashMap handle collisions?" -> score 1
    rep alice  +10 -> 210 (UP on question #101 "How does HashMap handle collisions?")
carol voted UP on answer #102 by bob -> score 1
    rep bob    +10 -> 3510 (UP on answer #102 by bob)
erin voted DOWN on answer #103 by carol -> score -1
    rep carol  -2 -> 3198 (DOWN on answer #103 by carol)
    rep erin   -1 -> 3099 (downvoted an answer)
  REJECTED: erin already voted DOWN on this post
erin voted UP on answer #103 by carol -> score 1
    rep carol  +2 -> 3200 (vote changed on answer #103 by carol)
    rep erin   +1 -> 3100 (vote changed)
    rep carol  +10 -> 3210 (UP on answer #103 by carol)
erin commented on answer #103 by carol: "HashMap uses chaining, not open addressing."
    [email to carol@example.com] erin commented on your answer #103 by carol
  REJECTED: dave needs 50 reputation for COMMENT_EVERYWHERE (has 1)
alice accepted answer #102 by bob
    rep bob    +15 -> 3525 (answer accepted)
    rep alice  +2 -> 212 (accepted an answer)

== 3. Bounty ==
alice asked question #105 "Why is ConcurrentHashMap faster than synchronizedMap?" [[java]]
    rep alice  -50 -> 162 (bounty staked)
alice offered a +50 bounty on question #105 (expires 2026-10-05)
carol posted answer #106 by carol on question #105
    [email to alice@example.com] carol answered "Why is ConcurrentHashMap faster than synchronizedMap?"
    rep carol  +50 -> 3260 (bounty awarded)
alice awarded the bounty to answer #106 by carol

== 4. Badges for a new member ==
dave asked question #107 "Is Java pass-by-value?" [[java]]
bob voted UP on question #107 "Is Java pass-by-value?" -> score 1
    rep dave   +10 -> 11 (UP on question #107 "Is Java pass-by-value?")
carol voted UP on question #107 "Is Java pass-by-value?" -> score 2
    rep dave   +10 -> 21 (UP on question #107 "Is Java pass-by-value?")
    [email to dave@example.com] You earned the 'Contributor' badge
erin voted UP on question #107 "Is Java pass-by-value?" -> score 3
    rep dave   +10 -> 31 (UP on question #107 "Is Java pass-by-value?")
dave's badges: [Contributor]

== 5. Moderation ==
bob voted to close question #107 as DUPLICATE
carol voted to close question #107 as DUPLICATE
erin voted to close question #107 as DUPLICATE -> CLOSED
  REJECTED: question is CLOSED, no new answers
mia (mod) reopened question #107
dave commented on question #107 "Is Java pass-by-value?": "you all are wrong!!!"
bob flagged comment #109 (rude), flags=1
carol flagged comment #109 (rude), flags=2
erin flagged comment #109 (rude), flags=3
    [email to mia@example.com] comment #109 needs moderator attention
mia (mod) deleted question #107
mia (mod) undeleted question #107 -> CLOSED
root blocked dave
  REJECTED: dave is BLOCKED

== 6. Guest search and top tags ==
```

```
found: question #101 "How does HashMap handle collisions?" score=1 answers=2 status=OPEN
[java] question #107 "Is Java pass-by-value?"
[java] question #101 "How does HashMap handle collisions?"
[java] question #105 "Why is ConcurrentHashMap faster than synchronizedMap?"
top tags: [javax3, collectionsx1]
final reputation: [alice(162), bob(3525), carol(3260), erin(3100), dave(31)]
```

Walking through the important parts

- `Post.castVote` is `synchronized` (Java lets only one thread at a time run a synchronized method on the same object; others wait their turn) and uses `votes.put(...)`, which returns the previous vote. That single call both records the vote and tells us whether this is a *change* (so the facade can undo the old reputation effect). Voting the same way twice throws.
- `DefaultReputationPolicy` uses a Java 21 **pattern-matching switch** (a `switch` that picks a branch by the *type* of the value and hands you that typed value) over the sealed `Post`. Because `Post` permits exactly `Question`, `Answer` and `Comment`, the compiler rejects the switch if a case is missing.
- `Member.changeReputation` uses `AtomicInteger.updateAndGet` (an `AtomicInteger` is a number whose read-change-write happens as one indivisible step) so concurrent changes never lose an update, and it enforces the site's floor of 1.
- `BadgeAwarder` fires only when a threshold is *crossed* (`oldValue < level && newValue >= level`), so members who already had high reputation are not showered with badges, and each badge is added once (`Set.add` returns `false` for duplicates).
- `ask()` shows the activity diagram's decision "does the tag exist? has enough reputation?": in the output, Alice (200 rep) cannot create the new tag `hashmap`, so her first attempt is rejected.
- In section 2 of the output, Erin downvotes Carol's answer (Carol −2, Erin −1), then changes her vote to up: the old effect is reversed (+2, +1) and the new one applied (+10).
- `voteToClose` closes the question on the third vote; answering a closed question is then refused; a moderator reopens it.

Concurrency & edge cases

- **Two votes at the same time on one post.** `castVote`, `score` and `flag` are `synchronized` on the post, so the vote map is never corrupted and each voter is counted once. Reputation uses `AtomicInteger`, so two simultaneous reputation changes to the same author both apply.
- **Reputation updates are not in the same lock as the vote.** A reader could briefly see the new vote before the author's reputation changes. That is acceptable (**eventual consistency** for a score: the copies may disagree for a moment but will match shortly — see System Design Fundamentals). In a database you would put both in one **transaction** (a group of changes that either all succeed or all fail together), and enforce "one vote per member per post" with a **unique constraint** (a database rule that refuses a second row with the same values) on `(post_id, voter_id)`.
- **Reputation floor.** Because reputation cannot go below 1, reversing a vote on a member who was at the floor is not perfectly symmetric. The real site has the same quirk; if exact reversal matters, keep a ledger (an append-only list of every change, in order) of reputation events and recompute.
- **Accepting a different answer.** `Question.accept` returns the previously accepted answer so the facade removes its +15; the asker's +2 is given only once.
- **Self-actions.** Voting on your own post and awarding your own bounty are rejected.
- **Closed and deleted questions.** Closed: no new answers. Deleted: hidden from search results.
- **Blocked accounts.** Every write goes through `requireActive`, so a blocked member can do nothing.
- **Bounty expiry.** `awardTo` refuses after the expiry date. On the real site, bounty reputation is never refunded; if the sponsor does not award it, the system may auto-award half to a qualifying answer.

Extensions & follow-up questions

- **"How do you stop one person voting twice?"** — The per-post map of voter ids (the course's key follow-up). In a database: a unique key on `(post_id, voter_id)`.
- **"Add edit history."** — Store each version as an immutable `Revision` (author, time, body) in a list on the post; editing appends a revision. This is close to the **Memento** idea: the past states are kept so they can be shown or rolled back.
- **"Daily reputation cap."** — The real site caps reputation from votes at 200 per day. Wrap the policy: a `CappedReputationPolicy` decorator (the **Decorator** pattern: an object that wraps another object with the same

interface and adds behaviour before or after passing the call on) that tracks today's total per member and trims the delta. No voting code changes.

- **"Mark as duplicate and link."** — Add `duplicateOf: Question` to `Question` and a `ClosingRemark.DUPLICATE`; closing as duplicate sets the link.
- **"Review queues."** — Flags already reach moderators at a threshold; turn that into a `ReviewQueue` that holds flagged posts, with reviewers voting to keep or delete.
- **"Hot questions."** — A ranking Strategy over recent votes, answers and views with time decay (older activity counts for less and less), just like the feed rankers in 18.3 and 18.4.
- **"Tag frequency per day and week."** — The course's `Tag` keeps daily and weekly counters. Keep a counter per time bucket and roll the buckets over with a scheduled job (a task the system runs automatically at set times, e.g. every midnight).

Real-world example

On stackoverflow.com, the numbers in this design are real product rules. The help centre lists privileges by reputation (vote up at 15, comment everywhere at 50, set bounties at 75, vote down at 125, create tags at 1,500, edit at 2,000, cast close and reopen votes at 3,000, moderator tools at 10,000). In November 2019 the company raised question upvotes from +5 to +10 ("rewarding the question askers") — a change that, in our design, touches only the `ReputationPolicy` (one number, or a new policy class) and nothing in the voting code. After a 2019 experiment the number of close votes needed was lowered from 5 to 3, which in our code is one constant.

In the interview

How to spend 30–45 minutes

Minutes	What to do
0–5	Clarify actors and the reputation rules; write the requirement list
5–10	Use cases per actor; say "guest has no posting method"
10–20	Class diagram: `Post` base with three subclasses, `Member` hierarchy, `Tag`, `Bounty`, `QuestionIndex`, facade
20–30	Walk through `vote()` and `acceptAnswer()` including reversal; question state machine
30–40	Code the core: `Post.castVote`, the policy, the facade's `vote`
40–45	Concurrency (per-post lock, atomic reputation, DB unique key) and extensions

30-second answer: "Questions, answers and comments share voting and flagging, so they extend one `Post` that stores votes per voter — that stops double voting and lets me reverse a changed vote. Reputation rules are a Strategy so they can change without touching voting; badges are an Observer on reputation changes. Privileges come from a reputation table; moderator actions take a `Moderator` parameter so a guest or member cannot call them. Search is a separate inverted index, and tags count their own uses for 'top tags'."

Follow-ups with answers

- *Why not store `voteCount` only?* — It cannot prevent duplicates or support vote changes; derive the count from the voter map (or cache it next to a votes table).
- *Where does reputation get awarded — in `upvote`?* — Not inside the post. The facade asks the policy; that keeps posts ignorant of scoring (the course names this as the improvement to make).
- *How would you scale search?* — Keep the `Search` interface and replace `QuestionIndex` with a search engine fed by events when questions are created or edited.

Self-check

- **Q:** Which requirement is enforced by the *shape* of the classes rather than by an `if`? **A:** Guests cannot post or vote: guests have no `Member` object, and every write method needs one (course version: `Guest` has no `createQuestion`).
- **Q:** Why is `Post` abstract and sealed? **A:** Abstract because a bare "post" is never created; sealed so switches over post kinds are checked for completeness.
- **Q:** What happens to reputation when Erin changes a downvote into an upvote on an answer? **A:** The old effect is reversed (author +2, Erin +1) and the new one applied (author +10).

- **Q:** Why does `Tag` count its own uses? **A:** So "most used tags" is a sort of the tags, not a scan of all questions.
- **Q:** What makes a comment different from an answer in voting? **A:** Comments can only be upvoted and earn no reputation (policy returns 0).

> **TL;DR**

Stack Overflow = `Post` base (votes per voter, flags, comments) + `Question`/`Answer`/`Comment` + reputation **Strategy** + badge **Observer** + reputation **privilege table** + moderator-typed methods + inverted-index search + a facade that checks, delegates and scores.

18.2 Design Cricinfo

Figure — A ball hitting the stumps under the ESPNcricinfo logo, the cover image of this case study (course diagram)

Cricinfo (today ESPNcricinfo) is a sports news site only for cricket. It gives **live ball-by-ball commentary** of matches, keeps a database of every historic match, and publishes news and articles.

Before we start, a two-minute cricket primer, because this design is graded on whether you model the *rules*, not just the nouns:

Term	Meaning
Delivery / ball	One throw ("bowl") of the ball by the **bowler** to the **batter** on strike
Over	Six **legal** deliveries by one bowler. A **wide** (too far from the batter) or a **no ball** (illegal delivery, e.g. overstepping the line) gives the batting side a penalty run and must be bowled again, so it does not count toward the six
Striker / non-striker	Two batters are in at once. The striker faces the ball; they change ends whenever they complete an odd number of runs (1, 3, 5), and again after every over
Runs	Off the bat (credited to the batter) or **extras** — wides, no balls, **byes** and **leg byes** (credited to the team, not the batter)
Wicket / dismissal	A batter is out: bowled, caught, LBW (leg before wicket), stumped, hit wicket, run out, obstructing the field. A **run out is not credited to the bowler**
Innings	One team's turn to bat, ending when 10 batters are out ("all out"), the overs run out, or the target is reached
Formats	**T20** (20 overs per side), **ODI** (One Day International, 50 overs per side), **Test** (no over limit, two innings per side, over several days)
Squad / playing eleven	Before a tournament each team names a **squad**; for every match it chooses exactly **eleven** of those players

Problem & clarifying questions

Question to ask	Answer we assume
Which formats?	T20, ODI and Test
Who enters the data?	An **Admin** (scorer) records every ball; a **Commentator** adds text for each ball
Do we follow real cricket rules?	Yes — the international rules, at least: six legal balls per over, extras, who gets credit for a wicket, strike rotation, bowler limits
Live updates?	Yes — the scorecard and commentary update after every ball (push to subscribers, i.e. readers who asked for updates)
Stats?	Per player, match and tournament, plus global questions such as "highest wicket taker of all time"
Tournaments?	Yes, with squads and a points table
Out of scope?	The course leaves these out on purpose: rain rules (Duckworth–Lewis–Stern, a formula that resets the target when rain shortens a match), the follow-on and declarations in a Test, powerplays and fielding restrictions, free hits after a no ball, telling a leg-side wide from an off-side one, partnerships, net run rate (a team's scoring rate minus its opponents', used to break ties in a points table), and the site's news articles. (Our code adds a simple `declare()`; several others appear under Extensions)

Requirements

Functional (the course's eight, in our words, plus the rules they imply)

1. Track every team that plays cricket, and every match it plays.
2. Show live ball-by-ball commentary.
3. Follow international cricket rules.
4. Each team announces a squad for each tournament.
5. For each match, each team announces its playing eleven from the tournament squad.
6. Keep statistics for players, for matches and for tournaments.
7. Answer global statistics questions, such as "which bowler has taken the most wickets ever?" or "which batter has made the most Test hundreds?"
8. Track ODI, Test and T20 matches.

Non-functional

- **Accuracy:** stats must match the ball-by-ball record exactly (a scorer's correction must fix every derived number).
- **Low-latency live updates** (**latency** = the delay between something happening and the reader seeing it)**:** thousands to millions of readers see a new ball within seconds.
- **Consistency of recording:** two scorers must not interleave balls in the same match.
- **Extensibility:** a new stat question, a new format, or a new subscriber (push alerts) should be a new class.

Use cases

Two actors: the **Admin** adds or modifies players, teams, tournaments and matches and records the ball-by-ball details; the **Commentator** writes the text commentary for each ball.

Top use cases: add/modify teams and players; add tournaments and matches; add ball; add stadium, umpire and referee; add/update stats (some generated by the system); add commentary.

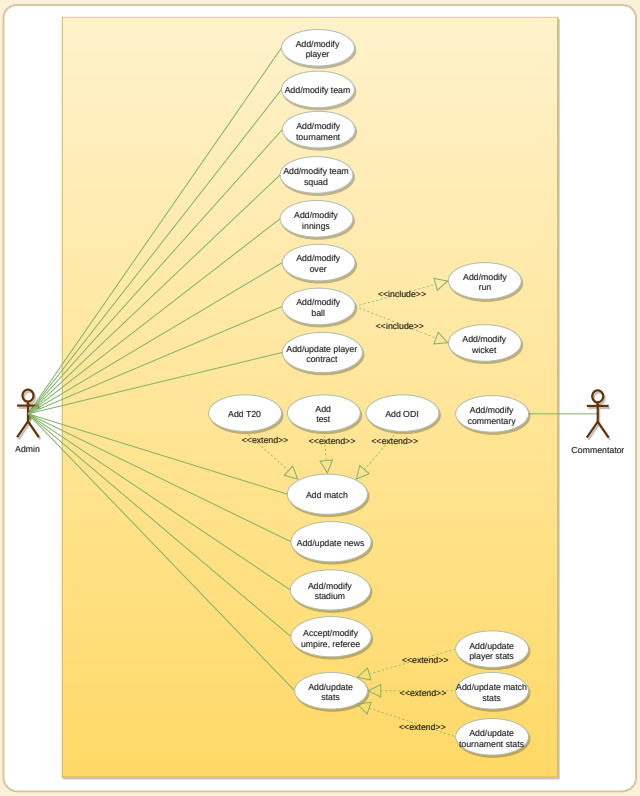


Figure — Cricinfo use cases: the Admin adds tournaments, matches (extended by Test, T20 and ODI), teams, squads, players, stadiums, umpires, innings, overs, balls (which include runs and wickets) and stats; the Commentator adds commentary (course diagram)

In the figure above, "Add match" is *extended* by the three formats, and "Add ball" *includes* adding its runs and wicket — a ball is recorded together with what happened on it.

Activity — record a ball. The scorer selects *Add Ball*; picks the ball type (normal, wide, no ball, or wicket-taking); if a wicket fell, adds wicket details; if runs were scored, adds a run record for each run type; adds commentary; saves the ball.

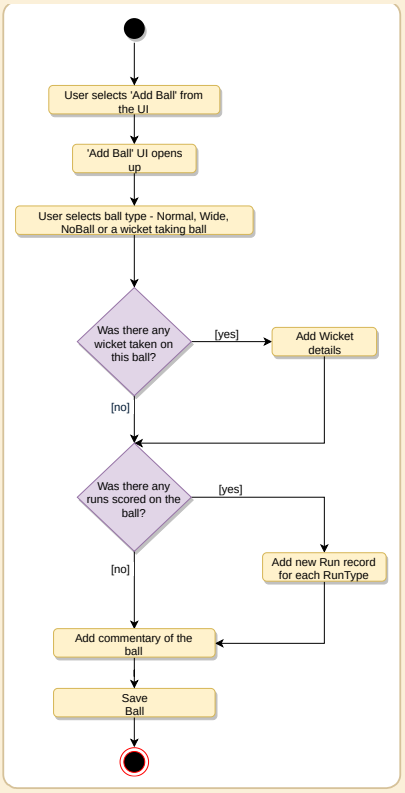


Figure — Activity diagram "record a ball of an over": choose ball type, add wicket details if any, add run records if any, add commentary, save (course diagram)

Core entities

Class	Responsibility
`Person` (record)	A human: name, country. Roles *hold* a person
`Player`	A role holding a `Person` (the course's `Player` also keeps a basic profile and contract details)
`MatchOfficial` (abstract) → `Umpire`, `Referee`, `Commentator`	Shared "assign to a match" behaviour; each says its role
`Team`	Every player the team has ever picked
`TournamentSquad`	Who a team may use in one tournament (must be team members)
`Playing11`	Exactly eleven, all from the squad, in batting order
`Tournament`	Owns squads, lists matches, keeps the points table
`BallOutcome` (record)	What happened: runs off the bat, extra type and runs, boundary?, wicket type, which batter was out, commentary text. Validates the laws (no bowled off a no ball)
`Ball` (record)	A recorded delivery: over, delivery number, bowler, striker, outcome, wicket
`Wicket` (record) + `WicketType`	Who was out, how, off whose bowling; `creditedToBowler()`
`Over`	One bowler's six **legal** deliveries; runs conceded; maiden?
`Innings`	Owns overs; decides striker, end of over, new batter, bowler limits, and when the innings ends
`Match` (abstract) → `T20`, `ODI`, `Test`	Holds everything; formats say only overs per innings and innings per side
`Result` (sealed) → `Win`, `Tie`, `Draw`	The outcome
`MatchObserver` → `CommentaryFeed`, `LiveScorecard`	Subscribers to ball / over / innings / match events
`PlayerStats`, `StatsService`	Numbers derived from balls
`StatQuery` (abstract) → `MostRuns`, `MostHundreds`, `MostWickets`, `MostSixes`	Template Method: one question per class

Class diagram

The course builds this design in **six stages**.

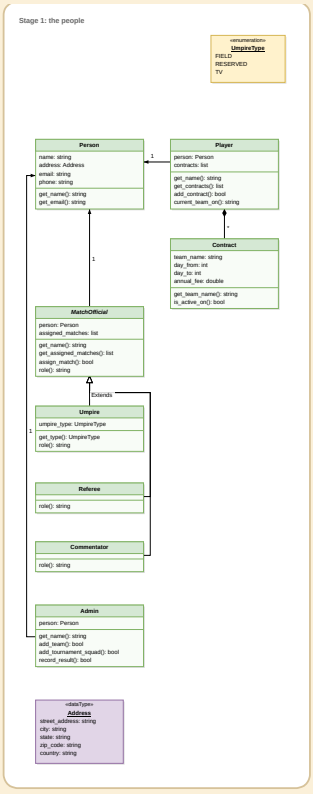


Figure — Stage 1: `Person`, `MatchOfficial` with `Umpire`, `Referee` and `Commentator`, `Admin`, and `Player` holding a `Person` (course diagram)

`Stage 1 — the people.` A player *holds* a person instead of extending one: one real human can play today and commentate after retiring, and composition lets them take both roles without copying. Umpires, referees and commentators all do one thing in common — get assigned to matches — so that lives once in `MatchOfficial`.

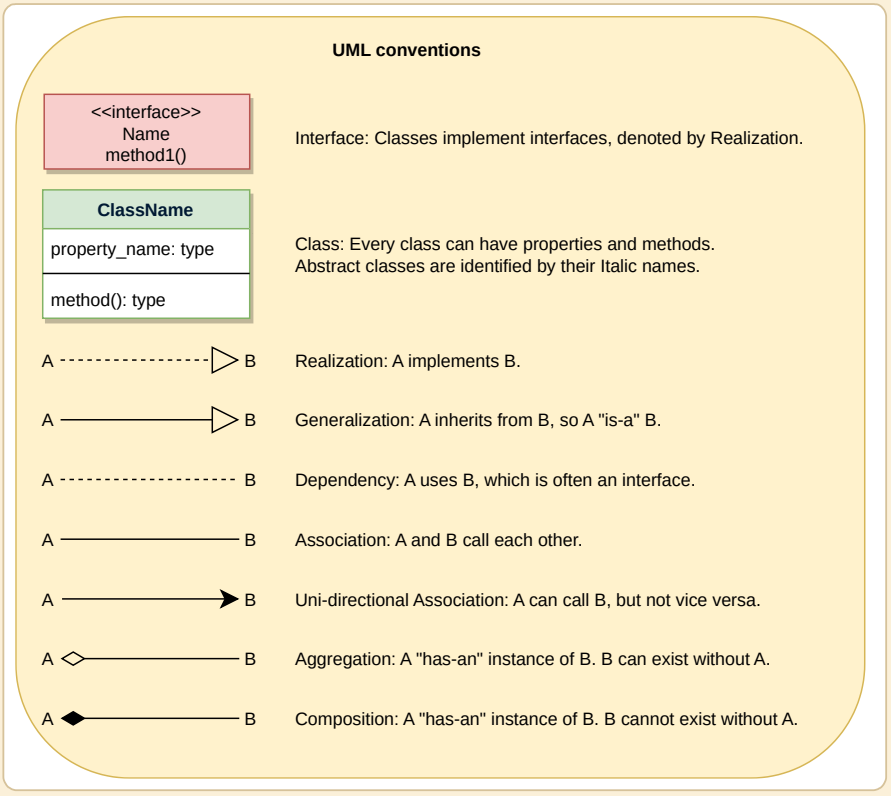


Figure — UML legend for reading the arrows in these diagrams (course diagram)

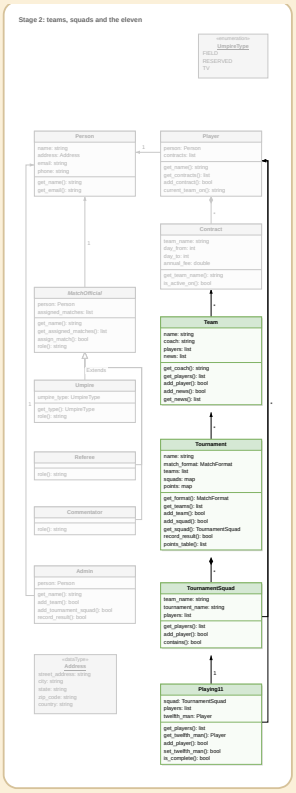


Figure — Stage 2: `Team`, `TournamentSquad`, `Playing11` and `Tournament`; only the tournament owns anything (its squads) (course diagram)

Stage 2 — teams, squads and the eleven. Requirements 4 and 5 look like one idea but are three nested sets: the team (everyone ever picked) ⊇ the squad (one tournament) ⊇ the eleven (one match). `Playing11` keeps a reference to its squad, which lets it reject anyone outside the squad, and it also rejects a twelfth pick. Requirement 5 thus lives in code rather than in a comment.

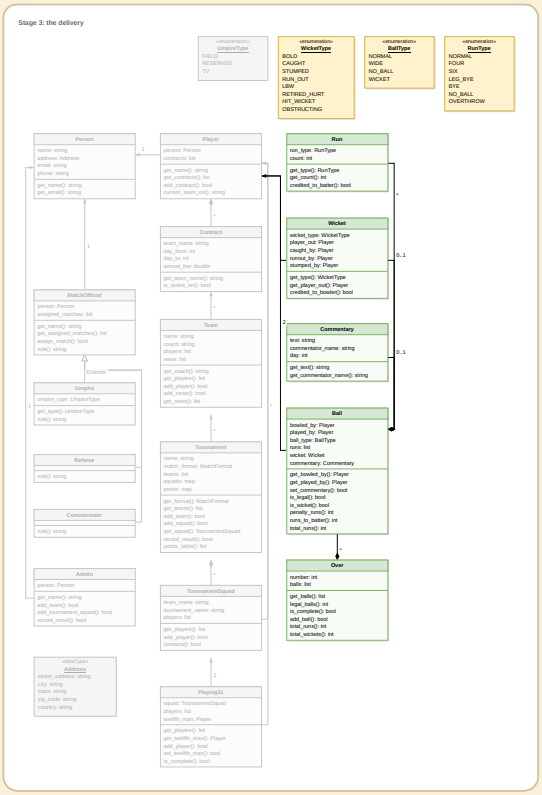


Figure — Stage 3: enums `WicketType`, `BallType`, `RunType`, and classes `Run`, `Wicket`, `Commentary`, `Ball` (owning its runs, wicket and commentary) and `Over` (course diagram)

Stage 3 — the delivery. This stage decides whether the design understands cricket. Three rules hide in it: (1) an over is six *legal* deliveries, so `Over.isComplete` counts only legal balls, and an over with two wides takes eight deliveries; (2) a run out is credited to the fielding side, *not the bowler* — get this wrong and "highest

wicket taker" is silently wrong for years; (3) byes and leg byes belong to the team, not the batter, so each run carries a type that says who gets the credit. `Ball` owns its runs, wicket and commentary (composition); it only *refers* to the bowler and batter.

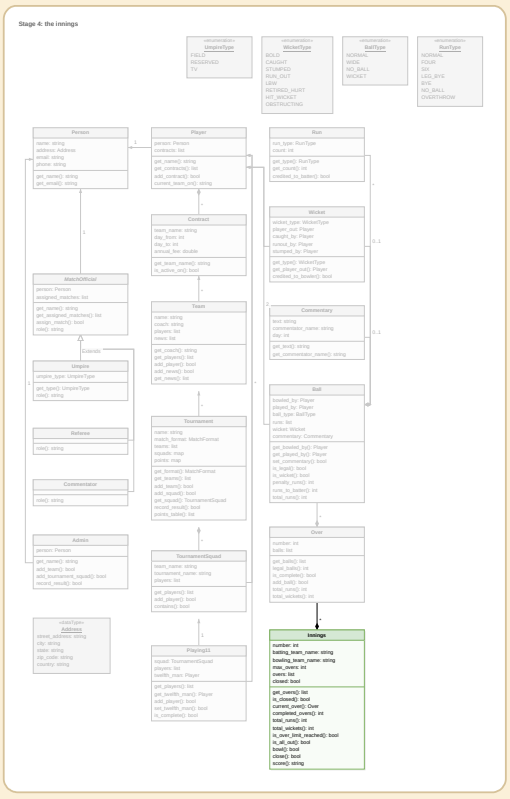


Figure — Stage 4: `Innings` owns its overs and prints the score (course diagram)

Stage 4 — the innings. Because the innings owns the overs, it alone decides when an over is finished and a new one starts; if that rule lived anywhere else, it would soon exist in two slightly different copies. Its `score` uses scorecard notation: runs/wickets, then full overs and the legal balls of the over in progress.

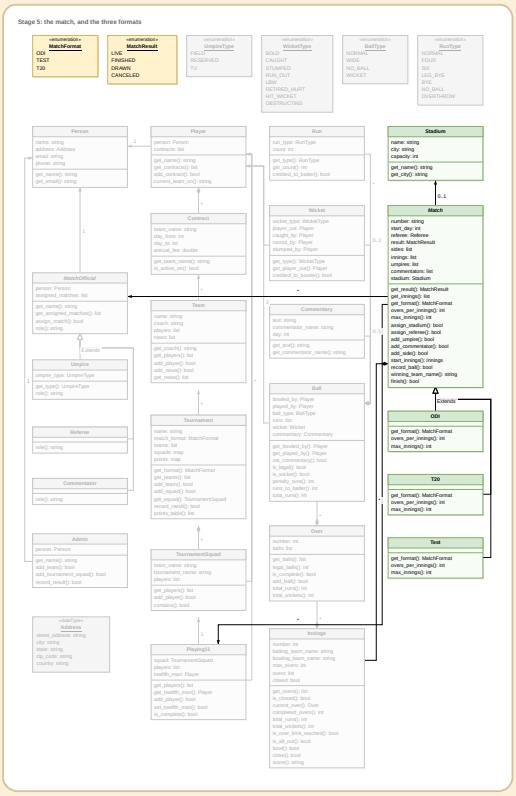


Figure — Stage 5: `Stadium`, abstract `Match` owning its innings, and the `ODI`, `T20` and `Test` subclasses (course diagram)

Stage 5 — the match and the formats. Before creating three classes, check what really changes between the formats. Starting an innings, recording a ball and deciding the winner are the same. Only two numbers differ: overs per innings and innings per match. So `Match` is abstract and holds everything; each format overrides two short methods. A Test returns zero overs (meaning "no limit") and four innings in total.

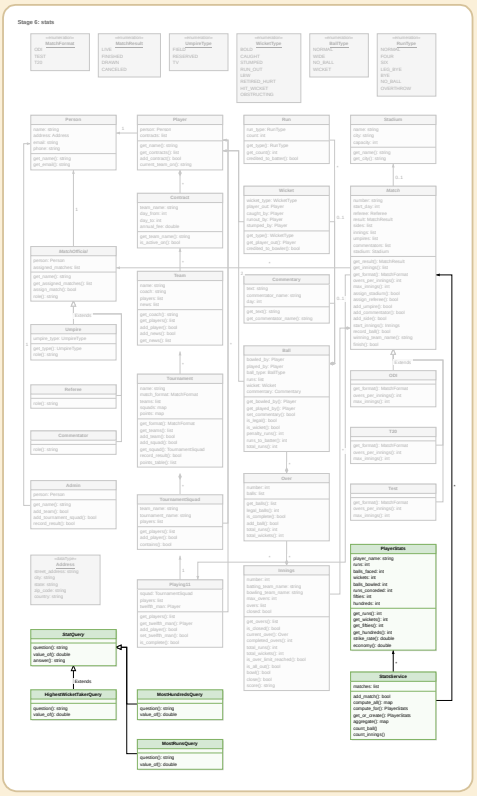


Figure — Stage 6: `PlayerStats`, `StatsService` and the `StatQuery` hierarchy (course diagram)

> **Key term — Template Method:** a design pattern where a base class writes the fixed outline of an algorithm in one method and leaves one or more steps as abstract methods for subclasses to fill in. Every subclass reuses the outline and changes only its own step. In this design `StatQuery.answer()` is the outline (loop over players, keep the best) and each subclass supplies only `valueOf()` — "runs", "wickets" and so on.

Stage 6 — stats. Stats are **worked out from the balls**, not stored beside them. Keep a separate counter and it will drift from the ball record as soon as a scorer fixes a mistake, and after that nobody can say which number is correct. Fifties and hundreds are per-innings milestones, so they are counted after each innings

(`countInnings`) next to the per-ball counting (`countBall`). Each global question is one small `StatQuery` subclass.

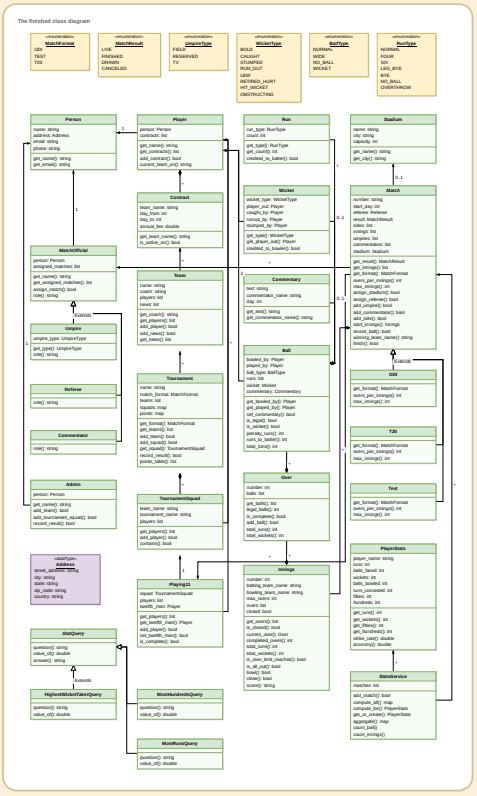
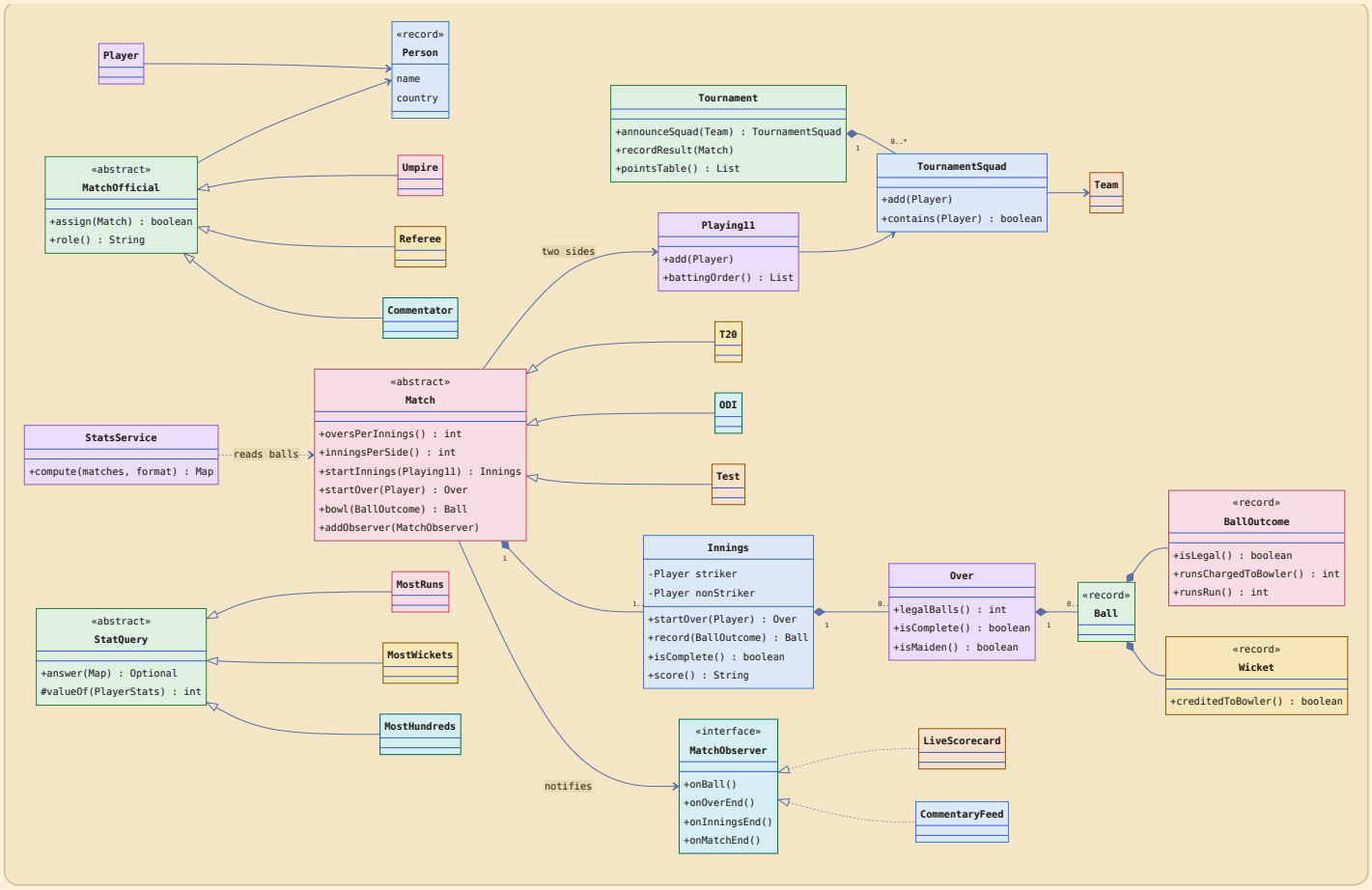


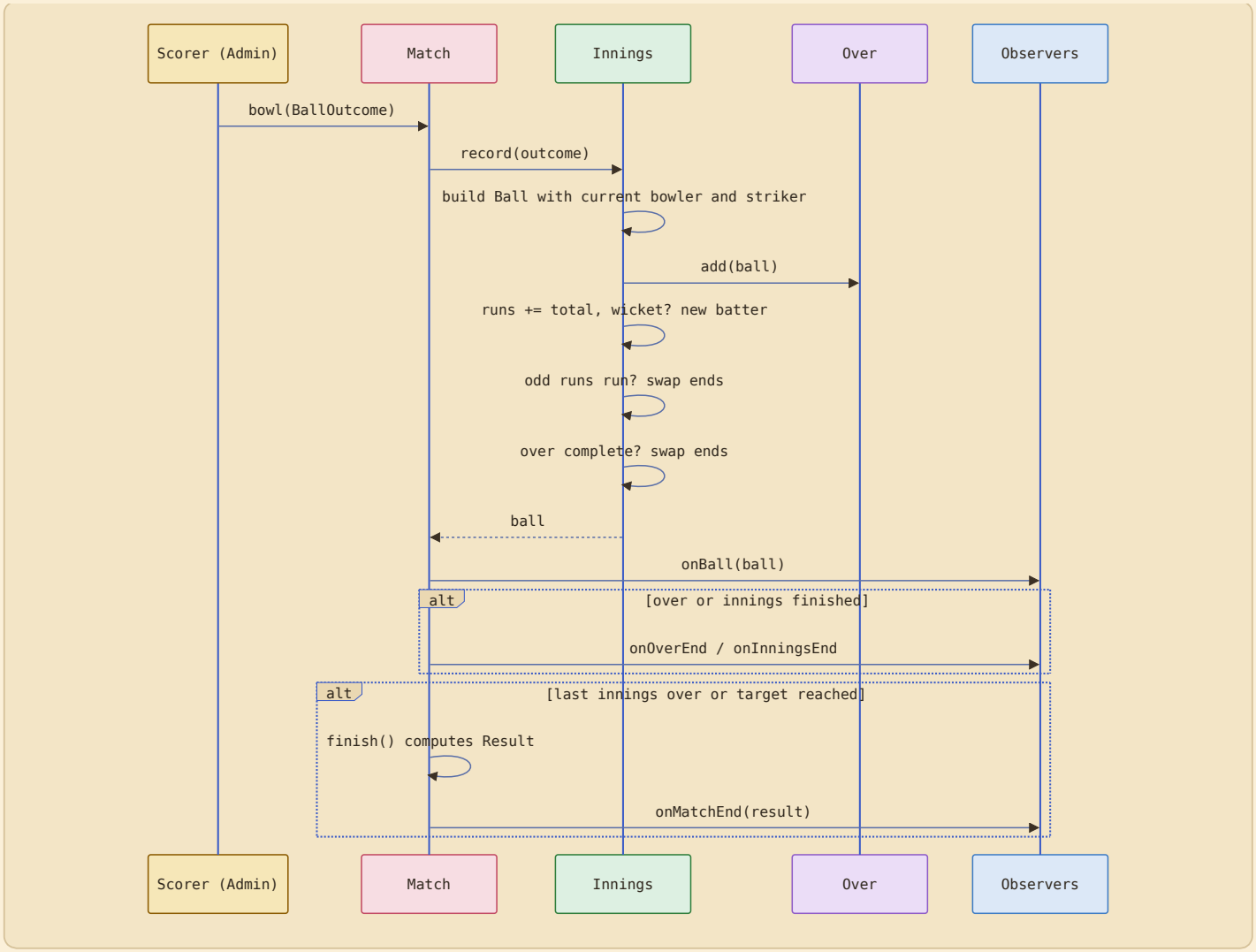
Figure — The finished Cricinfo class diagram, with every class from the six stages in full colour (course diagram)

Our Java design keeps all six stages and adds what the course flags as missing: **strike rotation** (who is on strike), bowler rules (not two overs in a row; 4 overs per bowler in a T20, 10 in an ODI), the laws about which dismissals are possible off a wide or no ball, a result calculation, and **Observers** for live updates. Instead of separate `Run` objects per run type, a `BallOutcome` record carries bat runs, extra type and extra runs — the same information in one immutable value (an object that can never be changed after it is created).

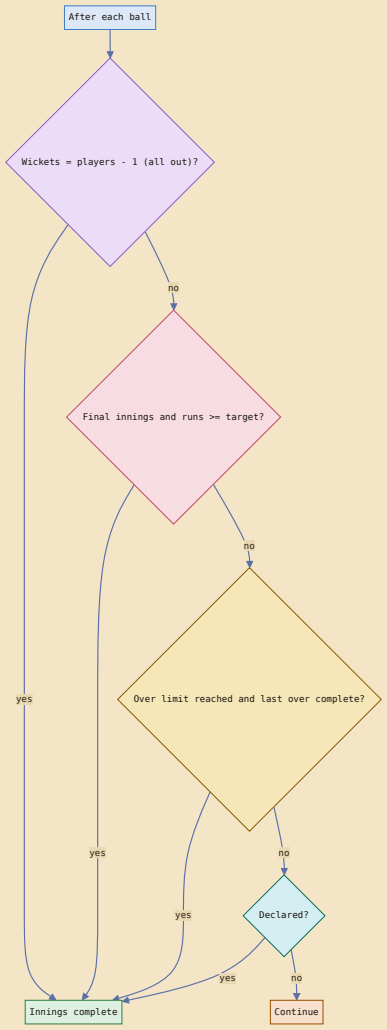


Key flows

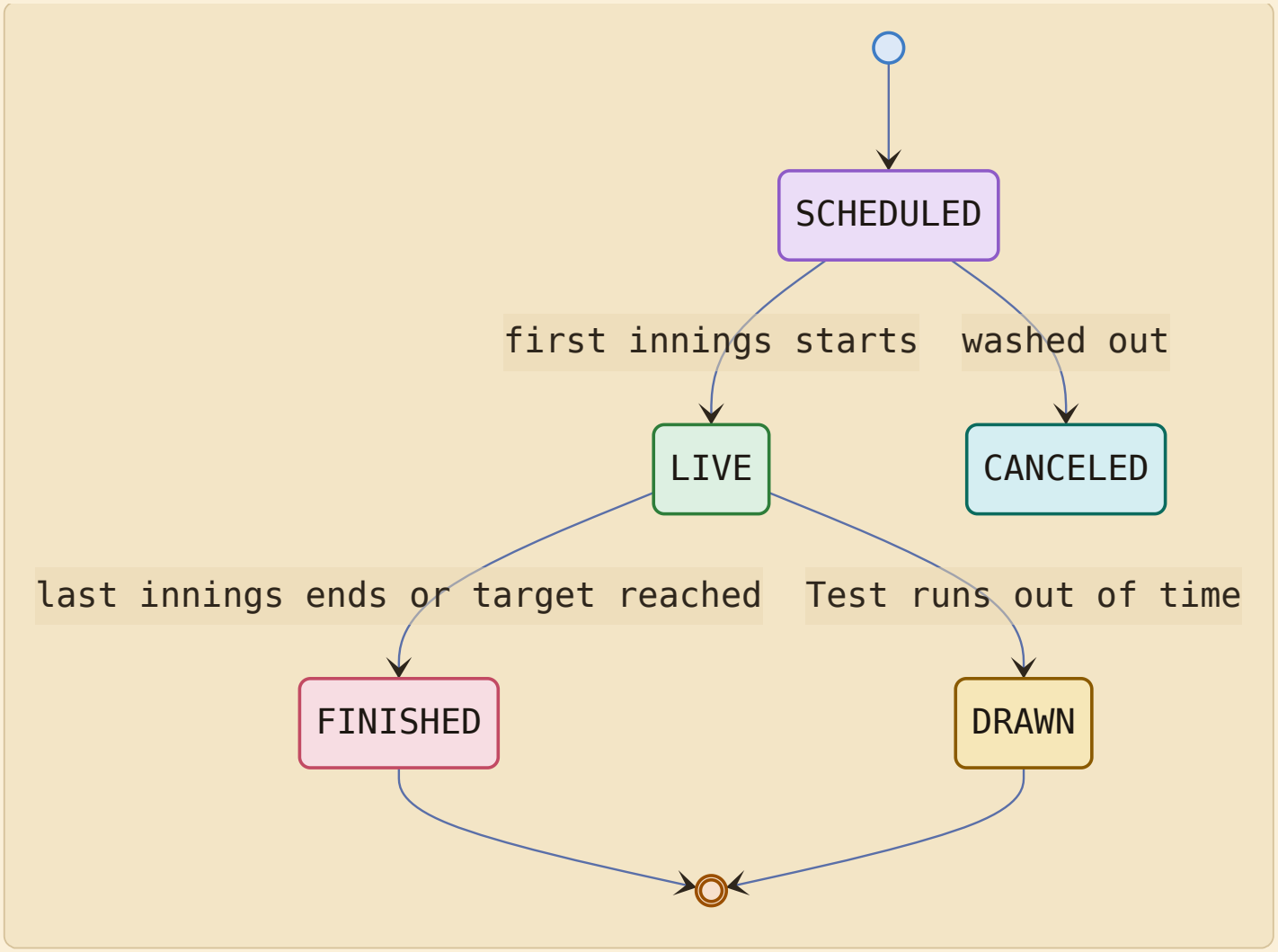
Recording one ball (the scorer's "Add Ball"):



Innings end conditions (checked by `Innings.isComplete`):



Match status:


SCHEDULED
first innings starts
washed out
LIVE
CANCELED
last innings ends or target reached
Test runs out of time
FINISHED
DRAWN

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`Player` and `MatchOfficial` hold a `Person`	Composition over inheritance	One human, many roles over time	`Player extends Person` — a player who becomes a commentator must be copied
`Match` abstract; `T20` / `ODI` / `Test` override two numbers	Template-style base class (the course notes it is not a named pattern here)	Only what differs is overridden	Three full `Match` copies
`StatQuery.answer()` final, `valueOf()` abstract	**Template Method**	A new statistic is one tiny subclass; the search loop is written once (Open/Closed: open to new features, closed to editing old code)	A `switch (questionType)` inside the service
Stats derived from balls	Single source of truth	Corrections fix every number automatically	Stored counters that drift; a cache is fine if it is rebuildable
`Innings` owns the over rule and strike	Information Expert (GRASP — guidelines for deciding which class gets a job)	The class with the data makes the decision	Scorer UI deciding when an over ends
`BallOutcome` validates the laws in its constructor	Fail fast, immutable value	Impossible deliveries (bowled off a no ball) never enter the record	Checking later in the stats code
`MatchObserver` with default methods	**Observer**	Commentary feed, live scorecard, push alerts subscribe without the match knowing them	The match printing to the console itself
`Result` is a sealed interface of records	Algebraic data type (a value that is exactly one of a closed set of shapes)	`switch` over results is exhaustive (see `Tournament.recordResult`)	Strings like "Team A won"
`Match.bowl` is `synchronized`	Monitor lock (Java's built-in per-object lock) per match	Two scorers cannot interleave balls	Unsynchronised updates that corrupt the over

Java implementation

Package `lld.ch18.cricinfo`. The demo builds two teams, a tournament, squads and elevens (showing both refusals), then plays a scripted first over (a wide and a no ball make it **eight deliveries**; a bowled that counts for the bowler; a run out that does not), and simulates the rest with a fixed random seed (the starting number for the random generator; the same seed always gives the same "random" sequence) so the output is repeatable. An ODI is simulated too, so the global stat queries have data from two formats.

```java
// File: lld/ch18/cricinfo/People.java
package lld.ch18.cricinfo;

import java.util.ArrayList;
import java.util.List;

/** A human being. Roles (player, umpire, commentator) HOLD a person instead of extending it. */
record Person(String name, String country) {}

final class Player {
    private final Person person;
    Player(Person person) { this.person = person; }
    String name() { return person.name(); }
    Person person() { return person; }
    @Override public String toString() { return name(); }
}

enum UmpireType { FIELD, RESERVED, TV }

/** Umpires, referees and commentators share one behaviour: being assigned to matches. */
abstract class MatchOfficial {
    private final Person person;
    private final List<Match> assignedMatches = new ArrayList<>();

    MatchOfficial(Person person) { this.person = person; }

    String name() { return person.name(); }
    synchronized boolean assign(Match m) { return !assignedMatches.contains(m) && assignedMatches.add(m); }
    synchronized List<Match> assignedMatches() { return List.copyOf(assignedMatches); }
    abstract String role();
    @Override public String toString() { return role() + " " + name(); }
}

final class Umpire extends MatchOfficial {
    private final UmpireType type;
    Umpire(Person p, UmpireType type) { super(p); this.type = type; }
    @Override String role() { return type + " umpire"; }
}

final class Referee extends MatchOfficial {
    Referee(Person p) { super(p); }
    @Override String role() { return "referee"; }
}

final class Commentator extends MatchOfficial {
    Commentator(Person p) { super(p); }
    @Override String role() { return "commentator"; }
}
```

```java
// File: lld/ch18/cricinfo/Teams.java
package lld.ch18.cricinfo;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/** Everyone the team has ever picked. */
final class Team {
    private final String name;
    private final Set<Player> players = new LinkedHashSet<>();

    Team(String name) { this.name = name; }
    String name() { return name; }
    void addPlayer(Player p) { players.add(p); }
    List<Player> players() { return List.copyOf(players); }   // keeps insertion order
    @Override public String toString() { return name; }
}

/** Who a team may use in ONE tournament: a subset of the team. */
final class TournamentSquad {
    private final Team team;
    private final Set<Player> players = new LinkedHashSet<>();

    TournamentSquad(Team team) { this.team = team; }
    Team team() { return team; }
    void add(Player p) {
        if (!team.players().contains(p)) throw new IllegalArgumentException(p + " does not play for " + team);
        players.add(p);
    }
    boolean contains(Player p) { return players.contains(p); }
}

/** Who takes the field in ONE match: exactly eleven, all from the squad. The class enforces the rule. */
final class Playing11 {
    static final int SIZE = 11;
    private final TournamentSquad squad;
    private final List<Player> players = new ArrayList<>();   // batting order

    Playing11(TournamentSquad squad) { this.squad = squad; }

    void add(Player p) {
        if (!squad.contains(p)) throw new IllegalArgumentException(p + " is not in the " + squad.team() + "
squad");
        if (players.size() == SIZE) throw new IllegalStateException("a playing eleven cannot have a twelfth player
(" + p + ")");
        if (players.contains(p)) throw new IllegalArgumentException(p + " already picked");
        players.add(p);
    }

    Team team() { return squad.team(); }
    List<Player> battingOrder() { return List.copyOf(players); }
    boolean isComplete() { return players.size() == SIZE; }
    boolean contains(Player p) { return players.contains(p); }
}

/** A tournament owns its squads and keeps a points table (2 for a win, 1 each for a tie or no result). */
final class Tournament {
    private final String name;
    private final Map<Team, TournamentSquad> squads = new LinkedHashMap<>();
    private final Map<Team, Integer> points = new LinkedHashMap<>();
    private final List<Match> matches = new ArrayList<>();

    Tournament(String name) { this.name = name; }
```

```
    String name() { return name; }

    TournamentSquad announceSquad(Team team) {
        TournamentSquad s = new TournamentSquad(team);
        squads.put(team, s);
        points.putIfAbsent(team, 0);
        return s;
    }

    void addMatch(Match m) { matches.add(m); }
    List<Match> matches() { return List.copyOf(matches); }

    void recordResult(Match m) {
        Result r = m.result().orElseThrow(() -> new IllegalStateException("match not finished"));
        switch (r) {
            case Result.Win w -> points.merge(w.winner(), 2, Integer::sum);
            case Result.Tie t -> { points.merge(m.teamA().team(), 1, Integer::sum); points.merge(m.teamB().team(),
1, Integer::sum); }
            case Result.Draw d -> { }                    // Tests are not part of a points table here
        }
    }

    List<Map.Entry<Team, Integer>> pointsTable() {
        return points.entrySet().stream()
                .sorted(Map.Entry.<Team, Integer>comparingByValue(Comparator.reverseOrder()))
                .toList();
    }
}
```

```java
// File: lld/ch18/cricinfo/Delivery.java
package lld.ch18.cricinfo;

import java.util.ArrayList;
import java.util.List;

enum Extra { NONE, WIDE, NO_BALL, BYE, LEG_BYE }

enum WicketType {
    BOWLED, CAUGHT, LBW, STUMPED, HIT_WICKET, RUN_OUT, OBSTRUCTING_THE_FIELD;

    /** A run out (or obstructing the field) is a wicket for the fielding side, NOT for the bowler. */
    boolean creditedToBowler() { return this != RUN_OUT && this != OBSTRUCTING_THE_FIELD; }
}

/**
 * What happened on one delivery, as typed by the scorer. Who bowled and who
 * faced is filled in by the Innings, which knows the state of play.
 *
 * @param batRuns    runs off the bat (credited to the batter)
 * @param extra      kind of extra, if any
 * @param extraRuns  runs credited to the team as extras (a wide or no ball is at least 1)
 * @param boundary   true when batRuns came from a 4 or 6 hit to the rope (batters do not cross)
 * @param wicket     type of dismissal, or null
 * @param nonStrikerOut for a run out: true when the batter at the bowler's end is out
 */
record BallOutcome(int batRuns, Extra extra, int extraRuns, boolean boundary,
                   WicketType wicket, boolean nonStrikerOut, String commentary) {

    BallOutcome {
        if (batRuns < 0 || extraRuns < 0) throw new IllegalArgumentException("runs cannot be negative");
        if ((extra == Extra.WIDE || extra == Extra.NO_BALL) && extraRuns < 1)
            throw new IllegalArgumentException(extra + " carries at least one penalty run");
        // Laws of cricket: off a no ball the bowler cannot take a wicket; off a wide only stumped/hit wicket/run
out.
        if (wicket != null && extra == Extra.NO_BALL && wicket.creditedToBowler())
            throw new IllegalArgumentException(wicket + " is not possible off a no ball");
        if (wicket != null && extra == Extra.WIDE && !(wicket == WicketType.STUMPED || wicket ==
WicketType.HIT_WICKET
                || !wicket.creditedToBowler()))
            throw new IllegalArgumentException(wicket + " is not possible off a wide");
    }

    static BallOutcome dot(String c) { return new BallOutcome(0, Extra.NONE, 0, false, null, false, c); }
    static BallOutcome runs(int n, String c) { return new BallOutcome(n, Extra.NONE, 0, false, null, false, c); }
    static BallOutcome four(String c) { return new BallOutcome(4, Extra.NONE, 0, true, null, false, c); }
    static BallOutcome six(String c) { return new BallOutcome(6, Extra.NONE, 0, true, null, false, c); }
    static BallOutcome wide(String c) { return new BallOutcome(0, Extra.WIDE, 1, false, null, false, c); }
    static BallOutcome noBall(int batRuns, String c) { return new BallOutcome(batRuns, Extra.NO_BALL, 1, batRuns >=
4, null, false, c); }
    static BallOutcome legBye(int n, String c) { return new BallOutcome(0, Extra.LEG_BYE, n, false, null, false,
c); }
    static BallOutcome bye(int n, String c) { return new BallOutcome(0, Extra.BYE, n, false, null, false, c); }
    static BallOutcome out(WicketType w, String c) { return new BallOutcome(0, Extra.NONE, 0, false, w, false, c);
}
    static BallOutcome runOut(int completedRuns, boolean nonStrikerOut, String c) {
        return new BallOutcome(completedRuns, Extra.NONE, 0, false, WicketType.RUN_OUT, nonStrikerOut, c);
    }

    /** A wide or a no ball is not one of the six balls of an over. */
    boolean isLegal() { return extra != Extra.WIDE && extra != Extra.NO_BALL; }
    boolean countsAsBallFaced() { return extra != Extra.WIDE; }
    int totalRuns() { return batRuns + extraRuns; }
    /** Byes and leg byes are not the bowler's fault; wides and no balls are. */
    int runsChargedToBowler() { return batRuns + (isLegal() ? 0 : extraRuns); }

    /** Runs physically run between the wickets: an odd number means the batters swapped ends. */
    int runsRun() {
```

```
        if (boundary) return 0;
        return switch (extra) {
            case NONE -> batRuns;
            case WIDE -> extraRuns - 1;
            case NO_BALL -> batRuns;
            case BYE, LEG_BYE -> extraRuns;
        };
    }
}

record Wicket(WicketType type, Player batter, Player bowler) {
    boolean creditedToBowler() { return type.creditedToBowler(); }
}

/** One delivery as recorded: owns its outcome, its wicket and its commentary (composition). */
record Ball(int overNumber, int deliveryInOver, Player bowler, Player striker, BallOutcome outcome, Wicket wicket)
{
    /** Scorecard style: over 1, ball 3 is "0.3" (completed overs . ball); an extra shows the number of the ball it
replaces. */
    String label() { return (overNumber - 1) + "." + deliveryInOver; }
}

/** An over is SIX LEGAL deliveries, not six balls. */
final class Over {
    static final int LEGAL_BALLS = 6;
    private final int number;
    private final Player bowler;
    private final List<Ball> balls = new ArrayList<>();

    Over(int number, Player bowler) { this.number = number; this.bowler = bowler; }

    int number() { return number; }
    Player bowler() { return bowler; }
    void add(Ball b) {
        if (isComplete()) throw new IllegalStateException("over " + number + " is complete");
        balls.add(b);
    }
    List<Ball> balls() { return List.copyOf(balls); }
    int legalBalls() { return (int) balls.stream().filter(b -> b.outcome().isLegal()).count(); }
    boolean isComplete() { return legalBalls() == LEGAL_BALLS; }
    int runsConceded() { return balls.stream().mapToInt(b -> b.outcome().runsChargedToBowler()).sum(); }
    boolean isMaiden() { return isComplete() && runsConceded() == 0; }
}
```

```java
// File: lld/ch18/cricinfo/Innings.java
package lld.ch18.cricinfo;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/**
 * One side's turn to bat. The innings owns its overs, so it is the ONE place
 * that decides when an over ends, who is on strike, and when the innings is over.
 */
final class Innings {
    private final int number;
    private final Playing11 batting;
    private final Playing11 bowling;
    private final int maxOvers;              // 0 = no limit (Test)
    private final int maxOversPerBowler;     // 0 = no limit
    private final Integer target;            // runs needed to win, only for the final innings
    private final List<Over> overs = new ArrayList<>();
    private final List<Ball> balls = new ArrayList<>();
    private final Map<Player, Integer> oversByBowler = new HashMap<>();
    private Player striker;
    private Player nonStriker;
    private int nextBatter = 2;
    private int runs;
    private int wickets;
    private boolean closed;

    Innings(int number, Playing11 batting, Playing11 bowling, int maxOvers, int maxOversPerBowler, Integer target)
{
        if (!batting.isComplete() || !bowling.isComplete()) throw new IllegalStateException("both sides need eleven
players");
        this.number = number;
        this.batting = batting;
        this.bowling = bowling;
        this.maxOvers = maxOvers;
        this.maxOversPerBowler = maxOversPerBowler;
        this.target = target;
        this.striker = batting.battingOrder().get(0);
        this.nonStriker = batting.battingOrder().get(1);
    }

    int number() { return number; }
    Team battingTeam() { return batting.team(); }
    Team bowlingTeam() { return bowling.team(); }
    Playing11 battingSide() { return batting; }
    int runs() { return runs; }
    int wickets() { return wickets; }
    Optional<Integer> target() { return Optional.ofNullable(target); }
    List<Ball> balls() { return List.copyOf(balls); }
    List<Over> overs() { return List.copyOf(overs); }
    Player striker() { return striker; }
    Player nonStriker() { return nonStriker; }

    Over startOver(Player bowler) {
        if (isComplete()) throw new IllegalStateException("innings is complete");
        Over current = currentOver();
        if (current != null && !current.isComplete()) throw new IllegalStateException("over " + current.number() +
" is still in progress");
        if (!bowling.contains(bowler)) throw new IllegalArgumentException(bowler + " is not in the fielding
eleven");
        if (current != null && current.bowler().equals(bowler))
            throw new IllegalStateException(bowler + " cannot bowl two overs in a row");
        int bowled = oversByBowler.getOrDefault(bowler, 0);
        if (maxOversPerBowler > 0 && bowled >= maxOversPerBowler)
            throw new IllegalStateException(bowler + " has used all " + maxOversPerBowler + " overs");
```

```
        oversByBowler.put(bowler, bowled + 1);
        Over over = new Over(overs.size() + 1, bowler);
        overs.add(over);
        return over;
    }

    Ball record(BallOutcome o) {
        Over over = currentOver();
        if (over == null || over.isComplete()) throw new IllegalStateException("start a new over first");
        if (isComplete()) throw new IllegalStateException("innings is complete");

        int deliveryNo = over.legalBalls() + 1;          // a wide or no ball is re-bowled, so it shares the next
number
        Wicket wicket = null;
        if (o.wicket() != null) wicket = new Wicket(o.wicket(), o.nonStrikerOut() ? nonStriker : striker,
over.bowler());
        Ball ball = new Ball(over.number(), deliveryNo, over.bowler(), striker, o, wicket);
        over.add(ball);
        balls.add(ball);
        runs += o.totalRuns();

        if (o.runsRun() % 2 == 1) swapEnds();              // odd runs: batters crossed
        if (wicket != null) {
            wickets++;
            if (!allOut()) {
                Player newBatter = batting.battingOrder().get(nextBatter++);
                if (wicket.batter().equals(striker)) striker = newBatter; else nonStriker = newBatter;
            }
        }
        if (over.isComplete()) swapEnds();                  // ends change after every over
        return ball;
    }

    boolean isComplete() {
        if (closed || allOut()) return true;
        if (target != null && runs >= target) return true;
        Over current = currentOver();
        return maxOvers > 0 && overs.size() == maxOvers && current != null && current.isComplete();
    }

    /** A Test captain may declare; the demo also uses it to stop an unlimited innings. */
    void declare() { closed = true; }

    int wicketsInHand() { return batting.battingOrder().size() - 1 - wickets; }

    /** Scorecard style: runs/wickets (completed overs.legal balls). */
    String score() {
        return battingTeam() + " " + runs + "/" + wickets + " (" + oversText() + " ov)";
    }

    String oversText() {
        Over current = currentOver();
        if (current == null) return "0";
        int full = current.isComplete() ? overs.size() : overs.size() - 1;
        int part = current.isComplete() ? 0 : current.legalBalls();
        return part == 0 ? String.valueOf(full) : full + "." + part;
    }

    private boolean allOut() { return wickets == batting.battingOrder().size() - 1; }
    private Over currentOver() { return overs.isEmpty() ? null : overs.get(overs.size() - 1); }
    private void swapEnds() { Player t = striker; striker = nonStriker; nonStriker = t; }
}
```

```java
// File: lld/ch18/cricinfo/Match.java
package lld.ch18.cricinfo;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.CopyOnWriteArrayList;

enum MatchFormat { ODI, T20, TEST }

enum MatchStatus { SCHEDULED, LIVE, FINISHED, DRAWN, CANCELED }

record Stadium(String name, String city, int capacity) {}

/** Outcome of a finished match. A sealed hierarchy lets callers switch over it exhaustively. */
sealed interface Result {
    record Win(Team winner, String margin) implements Result {
        @Override public String toString() { return winner + " won by " + margin; }
    }
    record Tie() implements Result {
        @Override public String toString() { return "Match tied"; }
    }
    record Draw() implements Result {
        @Override public String toString() { return "Match drawn"; }
    }
}

/**
 * Everything that is the same in every format lives here. The formats only
 * answer two questions: how many overs per innings, how many innings per side.
 * Observers (live scorecard, commentary feed, push alerts) subscribe to events.
 */
abstract class Match {
    private final Playing11 teamA;
    private final Playing11 teamB;
    private final Stadium stadium;
    private final LocalDate date;
    private final List<MatchOfficial> officials = new ArrayList<>();
    private final List<Innings> innings = new ArrayList<>();
    private final List<MatchObserver> observers = new CopyOnWriteArrayList<>();
    private MatchStatus status = MatchStatus.SCHEDULED;
    private Result result;

    Match(Playing11 teamA, Playing11 teamB, Stadium stadium, LocalDate date) {
        this.teamA = teamA;
        this.teamB = teamB;
        this.stadium = stadium;
        this.date = date;
    }

    abstract MatchFormat format();
    abstract int oversPerInnings();       // 0 means no limit
    abstract int inningsPerSide();
    /** ODIs allow 10 overs per bowler and T20s 4, i.e. one fifth of the innings. */
    int maxOversPerBowler() { return oversPerInnings() / 5; }

    Playing11 teamA() { return teamA; }
    Playing11 teamB() { return teamB; }
    Stadium stadium() { return stadium; }
    LocalDate date() { return date; }
    synchronized MatchStatus status() { return status; }
    synchronized Optional<Result> result() { return Optional.ofNullable(result); }
    synchronized List<Innings> innings() { return List.copyOf(innings); }

    void addObserver(MatchObserver o) { observers.add(o); }

    void assign(MatchOfficial official) {
```

```
        official.assign(this);
        officials.add(official);
    }

    synchronized Innings startInnings(Playing11 batting) {
        if (status == MatchStatus.FINISHED || status == MatchStatus.DRAWN) throw new IllegalStateException("match
is over");
        Innings current = currentInnings();
        if (current != null && !current.isComplete()) throw new IllegalStateException("current innings still in
progress");
        if (innings.size() == 2 * inningsPerSide()) throw new IllegalStateException("all innings played");
        Playing11 bowling = batting == teamA ? teamB : teamA;
        boolean finalInnings = innings.size() == 2 * inningsPerSide() - 1;
        Integer target = finalInnings ? totalFor(bowling.team()) - totalFor(batting.team()) + 1 : null;
        Innings in = new Innings(innings.size() + 1, batting, bowling, oversPerInnings(), maxOversPerBowler(),
target);
        innings.add(in);
        status = MatchStatus.LIVE;
        return in;
    }

    synchronized Over startOver(Player bowler) { return requireCurrent().startOver(bowler); }

    /** Records one delivery and fires the events; thread-safe so two scorers cannot interleave balls. */
    synchronized Ball bowl(BallOutcome outcome) {
        Innings in = requireCurrent();
        Ball ball = in.record(outcome);
        observers.forEach(o -> o.onBall(this, in, ball));
        Over over = in.overs().get(in.overs().size() - 1);
        if (over.isComplete() || in.isComplete()) observers.forEach(o -> o.onOverEnd(this, in, over));
        if (in.isComplete()) {
            observers.forEach(o -> o.onInningsEnd(this, in));
            boolean chased = in.target().map(t -> in.runs() >= t).orElse(false);
            if (innings.size() == 2 * inningsPerSide() || chased) finish(in);
        }
        return ball;
    }

    synchronized void declareDraw() {
        status = MatchStatus.DRAWN;
        result = new Result.Draw();
        observers.forEach(o -> o.onMatchEnd(this, result));
    }

    private void finish(Innings last) {
        Team chasing = last.battingTeam();
        Team other = last.bowlingTeam();
        int chasingRuns = totalFor(chasing);
        int otherRuns = totalFor(other);
        if (chasingRuns > otherRuns) result = new Result.Win(chasing, last.wicketsInHand() + " wickets");
        else if (chasingRuns < otherRuns) result = new Result.Win(other, (otherRuns - chasingRuns) + " runs");
        else result = new Result.Tie();
        status = MatchStatus.FINISHED;
        observers.forEach(o -> o.onMatchEnd(this, result));
    }

    private int totalFor(Team t) {
        return innings.stream().filter(i -> i.battingTeam() == t).mapToInt(Innings::runs).sum();
    }

    private Innings currentInnings() { return innings.isEmpty() ? null : innings.get(innings.size() - 1); }

    private Innings requireCurrent() {
        Innings in = currentInnings();
        if (in == null) throw new IllegalStateException("no innings started");
        return in;
    }

    @Override public String toString() { return format() + " " + teamA.team() + " v " + teamB.team() + " at " +
```

```
stadium.name(); }
}

final class ODI extends Match {
    ODI(Playing11 a, Playing11 b, Stadium s, LocalDate d) { super(a, b, s, d); }
    @Override MatchFormat format() { return MatchFormat.ODI; }
    @Override int oversPerInnings() { return 50; }
    @Override int inningsPerSide() { return 1; }
}

final class T20 extends Match {
    T20(Playing11 a, Playing11 b, Stadium s, LocalDate d) { super(a, b, s, d); }
    @Override MatchFormat format() { return MatchFormat.T20; }
    @Override int oversPerInnings() { return 20; }
    @Override int inningsPerSide() { return 1; }
}

final class Test extends Match {
    Test(Playing11 a, Playing11 b, Stadium s, LocalDate d) { super(a, b, s, d); }
    @Override MatchFormat format() { return MatchFormat.TEST; }
    @Override int oversPerInnings() { return 0; }    // no limit
    @Override int inningsPerSide() { return 2; }
}
```

```java
// File: lld/ch18/cricinfo/Observers.java
package lld.ch18.cricinfo;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** Observer: the match publishes events; screens, feeds and alerts subscribe. */
interface MatchObserver {
    default void onBall(Match m, Innings in, Ball b) {}
    default void onOverEnd(Match m, Innings in, Over o) {}
    default void onInningsEnd(Match m, Innings in) {}
    default void onMatchEnd(Match m, Result r) {}
}

/** Ball-by-ball commentary written by a commentator (prints only the first few overs of each innings in the demo).
*/
final class CommentaryFeed implements MatchObserver {
    private final Commentator commentator;
    private final int oversToShow;

    CommentaryFeed(Commentator commentator, int oversToShow) {
        this.commentator = commentator;
        this.oversToShow = oversToShow;
    }

    @Override public void onBall(Match m, Innings in, Ball b) {
        if (b.overNumber() > oversToShow) return;
        String what = describe(b);
        String text = b.outcome().commentary() == null ? "" : "  [" + commentator.name() + ": " +
b.outcome().commentary() + "]";
        System.out.printf("  %-4s %s to %s, %s%s%n", b.label(), b.bowler(), b.striker(), what, text);
    }

    private static String describe(Ball b) {
        BallOutcome o = b.outcome();
        if (b.wicket() != null) return "OUT " + b.wicket().type() + " (" + b.wicket().batter() + ")"
                + (b.wicket().creditedToBowler() ? "" : " - not the bowler's wicket");
        return switch (o.extra()) {
            case WIDE -> "wide";
            case NO_BALL -> "no ball" + (o.batRuns() > 0 ? " + " + o.batRuns() : "");
            case BYE -> o.extraRuns() + " bye(s)";
            case LEG_BYE -> o.extraRuns() + " leg bye(s)";
            case NONE -> o.batRuns() == 0 ? "no run" : o.boundary() ? o.batRuns() == 4 ? "FOUR" : "SIX" :
o.batRuns() + " run(s)";
        };
    }
}

/** Live score line at the end of an over (every n overs, to keep the demo short). */
final class LiveScorecard implements MatchObserver {
    private final int everyNOvers;
    LiveScorecard(int everyNOvers) { this.everyNOvers = everyNOvers; }

    @Override public void onOverEnd(Match m, Innings in, Over o) {
        if (o.number() % everyNOvers == 0 || in.isComplete()) {
            String chase = in.target().filter(t -> in.runs() < t).map(t -> "  need " + (t - in.runs()) + " to
win").orElse("");
            System.out.println("  [live] " + in.score() + chase);
        }
    }

    @Override public void onInningsEnd(Match m, Innings in) {
        System.out.println("  [innings " + in.number() + " over] " + in.score());
    }

    @Override public void onMatchEnd(Match m, Result r) {
        System.out.println("  [result] " + r);
```

```
    }
}

/** Builds batting and bowling cards from the recorded balls (nothing is stored twice). */
final class Scorecard {
    static void print(Innings in) {
        System.out.println("Batting - " + in.battingTeam());
        Map<Player, int[]> bat = new LinkedHashMap<>();   // runs, balls, 4s, 6s
        Map<Player, String> howOut = new LinkedHashMap<>();
        for (Ball b : in.balls()) {
            int[] s = bat.computeIfAbsent(b.striker(), k -> new int[4]);
            BallOutcome o = b.outcome();
            s[0] += o.batRuns();
            if (o.countsAsBallFaced()) s[1]++;
            if (o.boundary() && o.batRuns() == 4) s[2]++;
            if (o.boundary() && o.batRuns() == 6) s[3]++;
            if (b.wicket() != null) {
                bat.computeIfAbsent(b.wicket().batter(), k -> new int[4]);
                howOut.put(b.wicket().batter(), b.wicket().creditedToBowler()
                        ? (b.wicket().type() == WicketType.BOWLED ? "" :
b.wicket().type().name().toLowerCase().replace('_', ' ') + " ")
                                + "b " + b.bowler()
                        : "run out");
            }
        }
        bat.computeIfAbsent(in.striker(), k -> new int[4]);        // the two batters at the crease are listed
        bat.computeIfAbsent(in.nonStriker(), k -> new int[4]);     // even if one has not faced a ball yet
        List<Player> order = in.battingSide().battingOrder();
        for (Player p : order) {
            int[] s = bat.get(p);
            if (s == null) continue;
            System.out.printf("  %-10s %-22s %3d (%d)  4s:%d 6s:%d%n", p, howOut.getOrDefault(p, "not out"), s[0],
s[1], s[2], s[3]);
        }
        int extras = in.balls().stream().mapToInt(b -> b.outcome().extraRuns()).sum();
        System.out.println("  Extras " + extras + "   Total " + in.runs() + "/" + in.wickets() + " (" +
in.oversText() + " ov)");

        System.out.println("Bowling - " + in.bowlingTeam());
        Map<Player, int[]> bowl = new LinkedHashMap<>();  // legal balls, maidens, runs, wickets
        for (Over o : in.overs()) {
            int[] s = bowl.computeIfAbsent(o.bowler(), k -> new int[4]);
            s[0] += o.legalBalls();
            if (o.isMaiden()) s[1]++;
            s[2] += o.runsConceded();
            s[3] += (int) o.balls().stream().filter(b -> b.wicket() != null &&
b.wicket().creditedToBowler()).count();
        }
        bowl.forEach((p, s) -> System.out.printf("  %-10s %d.%d-%d-%d-%d%n", p, s[0] / 6, s[0] % 6, s[1], s[2],
s[3]));
    }
}
```

```java
// File: lld/ch18/cricinfo/Stats.java
package lld.ch18.cricinfo;

import java.util.Collection;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/** Career numbers for one player, always DERIVED from balls, never typed in by hand. */
final class PlayerStats {
    final Player player;
    int runs, ballsFaced, fours, sixes, fifties, hundreds, highest;
    int wickets, ballsBowled, runsConceded;

    PlayerStats(Player player) { this.player = player; }

    /** Per-ball numbers. */
    void countBall(Ball b, boolean asBatter) {
        BallOutcome o = b.outcome();
        if (asBatter) {
            runs += o.batRuns();
            if (o.countsAsBallFaced()) ballsFaced++;
            if (o.boundary() && o.batRuns() == 4) fours++;
            if (o.boundary() && o.batRuns() == 6) sixes++;
        } else {
            if (o.isLegal()) ballsBowled++;
            runsConceded += o.runsChargedToBowler();
            if (b.wicket() != null && b.wicket().creditedToBowler()) wickets++;
        }
    }

    /** Milestones are per innings, so they are counted once the innings score is known. */
    void countInnings(int inningsRuns) {
        if (inningsRuns >= 100) hundreds++;
        else if (inningsRuns >= 50) fifties++;
        highest = Math.max(highest, inningsRuns);
    }
}

/** Recomputes stats from the recorded balls of the given matches, optionally for one format. */
final class StatsService {
    Map<Player, PlayerStats> compute(Collection<Match> matches, MatchFormat formatOrNull) {
        Map<Player, PlayerStats> stats = new HashMap<>();
        for (Match m : matches) {
            if (formatOrNull != null && m.format() != formatOrNull) continue;
            for (Innings in : m.innings()) {
                Map<Player, Integer> inningsRuns = new HashMap<>();
                for (Ball b : in.balls()) {
                    stats.computeIfAbsent(b.striker(), PlayerStats::new).countBall(b, true);
                    stats.computeIfAbsent(b.bowler(), PlayerStats::new).countBall(b, false);
                    inningsRuns.merge(b.striker(), b.outcome().batRuns(), Integer::sum);
                }
                inningsRuns.forEach((p, r) -> stats.get(p).countInnings(r));
            }
        }
        return stats;
    }
}

/**
 * Template Method: answer() is the fixed algorithm (walk all players, keep the best);
 * each question fills in one step, valueOf(). A new statistic = one new subclass.
 */
abstract class StatQuery {
    record Leader(Player player, int value) {}

    abstract String question();
```

```
    abstract int valueOf(PlayerStats s);

    final Optional<Leader> answer(Map<Player, PlayerStats> stats) {
        Leader best = null;
        for (PlayerStats s : stats.values()) {
            int v = valueOf(s);
            if (v > 0 && (best == null || v > best.value()
                    || (v == best.value() && s.player.name().compareTo(best.player().name()) < 0))) {
                best = new Leader(s.player, v);
            }
        }
        return Optional.ofNullable(best);
    }
}

final class MostRuns extends StatQuery {
    @Override String question() { return "Most runs"; }
    @Override int valueOf(PlayerStats s) { return s.runs; }
}

final class MostHundreds extends StatQuery {
    @Override String question() { return "Most hundreds"; }
    @Override int valueOf(PlayerStats s) { return s.hundreds; }
}

final class MostWickets extends StatQuery {
    @Override String question() { return "Highest wicket taker"; }
    @Override int valueOf(PlayerStats s) { return s.wickets; }
}

final class MostSixes extends StatQuery {
    @Override String question() { return "Most sixes"; }
    @Override int valueOf(PlayerStats s) { return s.sixes; }
}

final class StatQueries {
    static final List<StatQuery> ALL = List.of(new MostRuns(), new MostHundreds(), new MostWickets(), new
MostSixes());
    private StatQueries() {}
}
```

```java
// File: lld/ch18/cricinfo/Main.java
package lld.ch18.cricinfo;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Random;

public class Main {
    public static void main(String[] args) {
        // ---- teams, squads, tournament ----
        Team lions = team("Lions", "Arun", "Bala", "Chetan", "Dev", "Eshan", "Farhan", "Gopal", "Hari", "Imran",
"Jai", "Kiran", "Lokesh", "Manav");
        Team tigers = team("Tigers", "Nabil", "Omar", "Pranav", "Qasim", "Rahul", "Sameer", "Tarun", "Uday",
"Varun", "Wasim", "Yash", "Zubin", "Ajay");
        Tournament cup = new Tournament("Demo Cup");
        TournamentSquad lionsSquad = cup.announceSquad(lions);
        TournamentSquad tigersSquad = cup.announceSquad(tigers);
        lions.players().stream().limit(12).forEach(lionsSquad::add);      // Manav is not in the squad
        tigers.players().stream().limit(12).forEach(tigersSquad::add);

        System.out.println("== Picking the elevens ==");
        Playing11 lionsXI = new Playing11(lionsSquad);
        Playing11 tigersXI = new Playing11(tigersSquad);
        lions.players().stream().limit(11).forEach(lionsXI::add);
        tigers.players().stream().limit(11).forEach(tigersXI::add);
        attempt(() -> new Playing11(lionsSquad).add(find(lions, "Manav")));
        attempt(() -> lionsXI.add(find(lions, "Lokesh")));

        // ---- the match ----
        Stadium stadium = new Stadium("Riverside Oval", "Pune", 40000);
        T20 match = new T20(lionsXI, tigersXI, stadium, LocalDate.of(2026, 9, 28));
        cup.addMatch(match);
        Commentator ravi = new Commentator(new Person("Ravi", "IN"));
        match.assign(new Umpire(new Person("Ken", "NZ"), UmpireType.FIELD));
        match.assign(new Umpire(new Person("Paul", "AU"), UmpireType.FIELD));
        match.assign(new Referee(new Person("Chris", "UK")));
        match.assign(ravi);
        match.addObserver(new CommentaryFeed(ravi, 1));
        match.addObserver(new LiveScorecard(5));

        System.out.println("\n== " + match + " ==");
        match.startInnings(lionsXI);
        List<Player> tigersBowlers = bowlers(tigersXI);
        match.startOver(tigersBowlers.get(0));
        match.bowl(BallOutcome.four("driven through cover"));
        match.bowl(BallOutcome.wide("sprayed down leg"));
        match.bowl(BallOutcome.runs(1, "pushed to mid-on"));
        match.bowl(BallOutcome.noBall(0, "overstepped, free hit next"));
        match.bowl(BallOutcome.out(WicketType.BOWLED, "through the gate!"));
        match.bowl(BallOutcome.runOut(1, false, "a second run was never there"));
        match.bowl(BallOutcome.dot("defended"));
        match.bowl(BallOutcome.six("launched over long-on"));
        attempt(() -> match.startOver(tigersBowlers.get(0)));             // same bowler twice in a row
        Random rnd = new Random(23);
        simulate(match, rnd, tigersBowlers);

        match.startInnings(tigersXI);
        simulate(match, rnd, bowlers(lionsXI));

        System.out.println("\n== Scorecards ==");
        match.innings().forEach(Scorecard::print);
        cup.recordResult(match);
        System.out.println("Points table: " + cup.pointsTable());

        // ---- a second match (ODI) so the global stats have more data ----
        ODI odi = new ODI(tigersXI, lionsXI, stadium, LocalDate.of(2026, 10, 2));
```

```
        odi.addObserver(new LiveScorecard(25));
        System.out.println("\n== " + odi + " ==");
        odi.startInnings(tigersXI);
        simulate(odi, rnd, bowlers(lionsXI));
        odi.startInnings(lionsXI);
        simulate(odi, rnd, bowlers(tigersXI));

        // ---- stats, derived from balls ----
        StatsService statsService = new StatsService();
        List<Match> all = List.of(match, odi);
        Map<Player, PlayerStats> career = statsService.compute(all, null);
        System.out.println("\n== Global stat queries (all formats) ==");
        for (StatQuery q : StatQueries.ALL) {
            System.out.println("  " + q.question() + ": " + q.answer(career)
                    .map(l -> l.player() + " (" + l.value() + ")").orElse("nobody yet"));
        }
        Map<Player, PlayerStats> odiOnly = statsService.compute(all, MatchFormat.ODI);
        System.out.println("  Most runs in ODIs: " + new MostRuns().answer(odiOnly).map(l -> l.player() + " (" +
l.value() + ")").orElse("-"));
        System.out.println("  Ravi is assigned to " + ravi.assignedMatches().size() + " match(es)");
    }

    /** Plays the rest of the current innings with random but legal deliveries. */
    static void simulate(Match m, Random rnd, List<Player> bowlers) {
        Innings in = m.innings().get(m.innings().size() - 1);
        int over = in.overs().size();
        while (!in.isComplete()) {
            m.startOver(bowlers.get(over % bowlers.size()));
            over++;
            while (!in.isComplete() && in.overs().get(in.overs().size() - 1).legalBalls() < Over.LEGAL_BALLS) {
                m.bowl(randomBall(rnd));
            }
        }
    }

    static BallOutcome randomBall(Random rnd) {
        int r = rnd.nextInt(1000);
        if (r < 330) return BallOutcome.dot(null);
        if (r < 640) return BallOutcome.runs(1, null);
        if (r < 720) return BallOutcome.runs(2, null);
        if (r < 730) return BallOutcome.runs(3, null);
        if (r < 810) return BallOutcome.four(null);                    // about 1 ball in 12 is a four
        if (r < 840) return BallOutcome.six(null);                     // and 1 in 33 a six: roughly 6.5 runs an
over
        if (r < 870) return BallOutcome.wide(null);
        if (r < 880) return BallOutcome.noBall(0, null);
        if (r < 895) return BallOutcome.legBye(1, null);
        if (r < 905) return BallOutcome.out(WicketType.BOWLED, null);
        if (r < 920) return BallOutcome.out(WicketType.CAUGHT, null);
        if (r < 927) return BallOutcome.out(WicketType.LBW, null);
        if (r < 930) return BallOutcome.runOut(0, rnd.nextBoolean(), null);
        return BallOutcome.dot(null);
    }

    static List<Player> bowlers(Playing11 xi) {
        List<Player> order = xi.battingOrder();
        return new ArrayList<>(order.subList(order.size() - 5, order.size()));   // the last five are the bowlers
    }

    static Team team(String name, String... players) {
        Team t = new Team(name);
        for (String p : players) t.addPlayer(new Player(new Person(p, "IN")));
        return t;
    }

    static Player find(Team t, String name) {
        return t.players().stream().filter(p -> p.name().equals(name)).findFirst().orElseThrow();
    }
```

```
    static void attempt(Runnable r) {
        try { r.run(); } catch (RuntimeException e) { System.out.println("  REJECTED: " + e.getMessage()); }
    }
}
```

Output:

```
== Picking the elevens ==
  REJECTED: Manav is not in the Lions squad
  REJECTED: a playing eleven cannot have a twelfth player (Lokesh)

== T20 Lions v Tigers at Riverside Oval ==
  0.1  Tarun to Arun, FOUR  [Ravi: driven through cover]
  0.2  Tarun to Arun, wide  [Ravi: sprayed down leg]
  0.2  Tarun to Arun, 1 run(s)  [Ravi: pushed to mid-on]
  0.3  Tarun to Bala, no ball  [Ravi: overstepped, free hit next]
  0.3  Tarun to Bala, OUT BOWLED (Bala)  [Ravi: through the gate!]
  0.4  Tarun to Chetan, OUT RUN_OUT (Chetan) - not the bowler's wicket  [Ravi: a second run was never there]
  0.5  Tarun to Arun, no run  [Ravi: defended]
  0.6  Tarun to Arun, SIX  [Ravi: launched over long-on]
  REJECTED: Tarun cannot bowl two overs in a row
  [live] Lions 29/2 (5 ov)
  [live] Lions 76/2 (10 ov)
  [live] Lions 117/3 (15 ov)
  [live] Lions 151/5 (20 ov)
  [innings 1 over] Lions 151/5 (20 ov)
  0.1  Gopal to Nabil, wide
  0.1  Gopal to Nabil, 1 run(s)
  0.2  Gopal to Omar, 1 run(s)
  0.3  Gopal to Nabil, 2 run(s)
  0.4  Gopal to Nabil, no run
  0.5  Gopal to Nabil, wide
  0.5  Gopal to Nabil, no run
  0.6  Gopal to Nabil, no ball
  0.6  Gopal to Nabil, no run
  [live] Tigers 40/0 (5 ov)  need 112 to win
  [live] Tigers 76/0 (10 ov)  need 76 to win
  [live] Tigers 91/2 (15 ov)  need 61 to win
  [live] Tigers 128/4 (20 ov)  need 24 to win
  [innings 2 over] Tigers 128/4 (20 ov)
  [result] Lions won by 23 runs

== Scorecards ==
Batting - Lions
  Arun       b Uday                 45 (32)  4s:4 6s:1
  Bala       b Tarun                 0 (2)  4s:0 6s:0
  Chetan     run out                 1 (1)  4s:0 6s:0
  Dev        caught b Yash          51 (51)  4s:4 6s:2
  Eshan      caught b Wasim         42 (31)  4s:5 6s:1
  Farhan     not out                 5 (3)  4s:0 6s:0
  Gopal      not out                 1 (1)  4s:0 6s:0
  Extras 6   Total 151/5 (20 ov)
Bowling - Tigers
  Tarun      4.0-0-41-1
  Uday       4.0-0-27-1
  Varun      4.0-0-29-0
  Wasim      4.0-0-28-1
  Yash       4.0-0-24-1
Batting - Tigers
  Nabil      lbw b Kiran            55 (56)  4s:3 6s:2
  Omar       lbw b Jai              44 (41)  4s:5 6s:0
  Pranav     run out                 1 (4)  4s:0 6s:0
  Qasim      not out                20 (19)  4s:1 6s:1
  Rahul      caught b Kiran          0 (1)  4s:0 6s:0
  Sameer     not out                 0 (0)  4s:0 6s:0
  Extras 8   Total 128/4 (20 ov)
Bowling - Lions
  Gopal      4.0-0-21-0
  Hari       4.0-0-22-0
  Imran      4.0-0-38-0
  Jai        4.0-0-19-1
  Kiran      4.0-0-27-2
Points table: [Lions=2, Tigers=0]

== ODI Tigers v Lions at Riverside Oval ==
```

```
  [live] Tigers 181/3 (25 ov)
  [live] Tigers 348/9 (50 ov)
  [innings 1 over] Tigers 348/9 (50 ov)
  [live] Lions 159/1 (25 ov)  need 190 to win
  [live] Lions 349/5 (49.4 ov)
  [innings 2 over] Lions 349/5 (49.4 ov)
  [result] Lions won by 5 wickets

== Global stat queries (all formats) ==
  Most runs: Chetan (131)
  Most hundreds: Chetan (1)
  Highest wicket taker: Kiran (5)
  Most sixes: Nabil (5)
  Most runs in ODIs: Chetan (130)
  Ravi is assigned to 1 match(es)
```

Walking through the important parts

- Ball numbers follow the scorecard convention "completed overs . ball", so the first ball of the match is `0.1`. A wide or no ball carries the number of the ball that must be re-bowled, which is why `0.2` and `0.3` each appear twice.
- In over 1 of the output, there are eight lines but the over ends at `0.6`: the wide and the no ball did not count toward the six (`Over.isComplete` counts legal balls).
- Chetan is **run out** after completing one run: `Innings.record` first swaps ends for the completed single, then replaces the dismissed batter, so the new batter (Dev) comes in at the correct end and Arun faces the next ball. In the scorecard, Chetan's line says "run out" and Tarun's bowling figures show only **1** wicket (the bowled), because `WicketType.RUN_OUT.creditedToBowler()` is `false`.
- Trying to give Tarun the second over is rejected ("cannot bowl two overs in a row"). The simulation rotates five bowlers, so each bowls exactly 4 overs in the T20 (`maxOversPerBowler() = 20 / 5`).
- `LiveScorecard` prints every 5 overs and the chase line ("need 112 to win") comes from the target set automatically for the final innings (`first-innings total + 1`). The random ball generator is tuned to about 6.5 runs an over, so totals look like real matches (151 and 128 in the T20; 348 and 349 in the ODI, which the Lions win by 5 wickets with two balls to spare).
- The batting card lists every batter who faced a ball or was out, plus the two batters at the crease (Sameer is "not out 0 (0)": he walked in but never faced).
- The global queries (`MostRuns`, `MostHundreds`, `MostWickets`, `MostSixes`) are each a one-line subclass; "Most runs in ODIs" reuses the same service with a format filter.

Machine-coding variant: Cricket Match Dashboard (Udaan)

A **machine-coding round** asks you to write a *working, runnable* program in 60–90 minutes, judged on correctness, readability and extensibility. This problem was set by Udaan (an Indian B2B — business-to-business — e-commerce company) and appears in the kumaransg/LLD collection.

Problem statement (in our words)

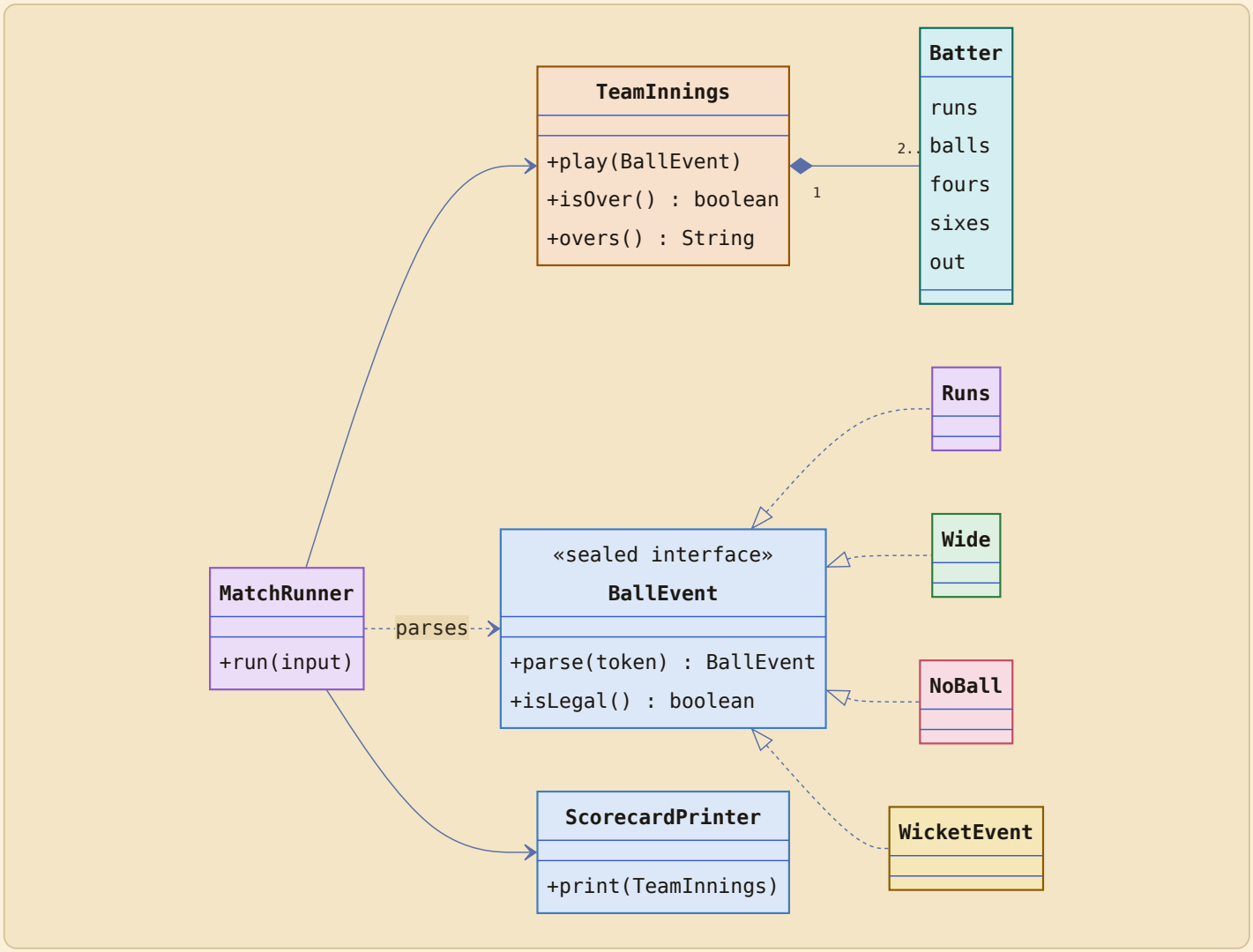
Build a cricket **scorecard** for a two-team, limited-overs match.

- **Input:** number of players per team, number of overs, then for each team its batting order, followed by the balls of each over, one per line: a number of runs (`0`–`6`), `W` (wicket), `Wd` (wide) or `Nb` (no ball).
- **Output after every over:** for each batter of the batting side — runs, 4s, 6s, balls faced — with a `*` on the two batters currently at the crease; then team total/wickets and overs.
- **Rules to handle:** strike changes on 1s and 3s and at the end of every over; wides and no balls add one run and are re-bowled (they do not count toward the six); the innings ends when the team is all out, the overs run out, or (second innings) the target is passed.
- **At the end:** print the winner and the margin (by runs or by wickets).
- **Good to have:** bowler figures (overs, runs, wickets, maidens, dot balls, economy), team extras, batter strike rates.

How the full Cricinfo design shrinks to a 90-minute solution

Cricinfo class	Machine-coding version	Why it is enough here
`BallOutcome` with extras, wickets, validation	`BallEvent` sealed interface: `Runs`, `Wide`, `NoBall`, `Wicket` with a `parse(token)` factory (a method that builds the right object from one input word)	Input has only four kinds of token
`Innings` (strike, over rule, end conditions)	`TeamInnings` — same responsibilities, ints instead of objects	Same rules, no bowler data in the input
`Scorecard` derived from balls	`Batter` counters + `ScorecardPrinter`	Only batting card needed; printer separated so another view could replace it
`Match` + `Result`	`MatchRunner` (reads input, plays two innings, prints result)	One format, two innings

This is the lesson of machine coding: keep the *same responsibilities in the same places* (the innings owns strike and the over rule; parsing is separate from scoring; printing is separate from both), but use the simplest data that satisfies the input.



(In the code the wicket record is called `BallEvent.Wicket`; it is shown as `WicketEvent` above only to keep the diagram label unique.)

Code

Package `lld.ch18.cricketdashboard`. `Main` feeds the problem's sample input as a text block.

```java
// File: lld/ch18/cricketdashboard/Model.java
package lld.ch18.cricketdashboard;

import java.util.ArrayList;
import java.util.List;

/** One input token, parsed once. Factory method: BallEvent.parse("Wd") etc. */
sealed interface BallEvent {
    record Runs(int runs) implements BallEvent {}
    record Wide() implements BallEvent {}
    record NoBall(int batRuns) implements BallEvent {}
    record Wicket() implements BallEvent {}

    static BallEvent parse(String token) {
        String t = token.trim();
        if (t.equalsIgnoreCase("W")) return new Wicket();
        if (t.equalsIgnoreCase("Wd")) return new Wide();
        if (t.regionMatches(true, 0, "Nb", 0, 2)) return new NoBall(t.length() > 2 ?
Integer.parseInt(t.substring(2)) : 0);
        int runs = Integer.parseInt(t);
        if (runs < 0 || runs > 6) throw new IllegalArgumentException("bad runs: " + t);
        return new Runs(runs);
    }

    default boolean isLegal() { return !(this instanceof Wide || this instanceof NoBall); }
}

/** A batter's line on the scorecard. */
final class Batter {
    final String name;
    int runs, balls, fours, sixes;
    boolean out;

    Batter(String name) { this.name = name; }

    void faced(int batRuns) {
        runs += batRuns;
        balls++;
        if (batRuns == 4) fours++;
        if (batRuns == 6) sixes++;
    }

    String strikeRate() { return balls == 0 ? "-" : String.format("%.1f", 100.0 * runs / balls); }
}

/** One team's batting innings: owns strike rotation, extras, and the "is it over?" rule. */
final class TeamInnings {
    private final String team;
    private final List<Batter> order = new ArrayList<>();
    private final int maxOvers;
    private final Integer target;          // null for the first innings
    private int striker = 0, nonStriker = 1, next = 2;
    private int total, wickets, legalBalls, extras;

    TeamInnings(String team, List<String> battingOrder, int maxOvers, Integer target) {
        if (battingOrder.size() < 2) throw new IllegalArgumentException("need at least two batters");
        this.team = team;
        battingOrder.forEach(n -> order.add(new Batter(n)));
        this.maxOvers = maxOvers;
        this.target = target;
    }

    void play(BallEvent e) {
        if (isOver()) throw new IllegalStateException("innings already finished");
        Batter s = order.get(striker);
        switch (e) {
            case BallEvent.Wide w -> { total += 1; extras += 1; }
            case BallEvent.NoBall nb -> {
```

```
                total += 1 + nb.batRuns(); extras += 1;
                s.faced(nb.batRuns());              // a no ball counts as a ball faced by the batter
                if (nb.batRuns() % 2 == 1) swap();
            }
            case BallEvent.Wicket w -> {
                legalBalls++;
                s.balls++;
                s.out = true;
                wickets++;
                if (!allOut()) striker = next++;     // new batter takes strike
            }
            case BallEvent.Runs r -> {
                legalBalls++;
                total += r.runs();
                s.faced(r.runs());
                if (r.runs() % 2 == 1) swap();       // 1s and 3s change strike
            }
        }
        if (e.isLegal() && legalBalls % 6 == 0 && !isOver()) swap();  // end of over
    }

    boolean isOver() {
        return allOut() || legalBalls == maxOvers * 6 || (target != null && total >= target);
    }

    private boolean allOut() { return wickets == order.size() - 1; }
    private void swap() { int t = striker; striker = nonStriker; nonStriker = t; }

    String team() { return team; }
    int total() { return total; }
    int wickets() { return wickets; }
    int legalBalls() { return legalBalls; }
    int extras() { return extras; }
    int wicketsInHand() { return order.size() - 1 - wickets; }
    List<Batter> batters() { return List.copyOf(order); }
    boolean isAtCrease(Batter b) { return !b.out && (order.get(striker) == b || order.get(nonStriker) == b); }

    String overs() { return legalBalls / 6 + (legalBalls % 6 == 0 ? "" : "." + legalBalls % 6); }
}
```

```java
// File: lld/ch18/cricketdashboard/Main.java
package lld.ch18.cricketdashboard;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Deque;
import java.util.List;

/** Prints the scorecard; kept apart so a web or JSON view could replace it (Single Responsibility). */
final class ScorecardPrinter {
    void print(TeamInnings in) {
        System.out.println("Scorecard for " + in.team() + ":");
        System.out.printf("%-8s %5s %3s %3s %5s %6s%n", "Player", "Score", "4s", "6s", "Balls", "SR");
        for (Batter b : in.batters()) {
            String name = b.name + (in.isAtCrease(b) ? "*" : "");
            System.out.printf("%-8s %5d %3d %3d %5d %6s%n", name, b.runs, b.fours, b.sixes, b.balls,
b.strikeRate());
        }
        System.out.println("Total: " + in.total() + "/" + in.wickets() + "   Extras: " + in.extras());
        System.out.println("Overs: " + in.overs());
    }
}

/** Drives the whole match from the text input (input format of the Udaan machine-coding round). */
final class MatchRunner {
    private final ScorecardPrinter printer = new ScorecardPrinter();

    void run(String input) {
        Deque<String> lines = new ArrayDeque<>();
        Arrays.stream(input.split("\\R")).map(String::trim).filter(l -> !l.isEmpty()).forEach(lines::add);
        int players = Integer.parseInt(after(lines.poll()));
        int overs = Integer.parseInt(after(lines.poll()));

        TeamInnings first = playInnings("Team 1", players, overs, null, lines);
        TeamInnings second = playInnings("Team 2", players, overs, first.total() + 1, lines);

        if (second.total() > first.total()) System.out.println("Result: Team 2 won the match by " +
second.wicketsInHand() + " wickets");
        else if (second.total() < first.total()) System.out.println("Result: Team 1 won the match by " +
(first.total() - second.total()) + " runs");
        else System.out.println("Result: Match tied");
    }

    private TeamInnings playInnings(String team, int players, int overs, Integer target, Deque<String> lines) {
        lines.poll();                                          // "Batting Order for team N:"
        List<String> order = new ArrayList<>();
        for (int i = 0; i < players; i++) order.add(lines.poll());
        TeamInnings in = new TeamInnings(team, order, overs, target);
        System.out.println("Batting order for " + team + ": " + order);
        while (!in.isOver() && !lines.isEmpty() && lines.peek().startsWith("Over")) {
            System.out.println(lines.poll());
            int legalThisOver = 0;
            while (legalThisOver < 6 && !in.isOver()) {          // an over = 6 LEGAL balls
                BallEvent e = BallEvent.parse(lines.poll());
                in.play(e);
                if (e.isLegal()) legalThisOver++;
            }
            printer.print(in);
        }
        // skip any balls left in the input after the innings ended (e.g. target reached mid-over)
        while (!lines.isEmpty() && !lines.peek().startsWith("Batting Order")) lines.poll();
        return in;
    }

    private static String after(String line) { return line.substring(line.indexOf(':') + 1).trim(); }
}
```

```
public class Main {
    static final String SAMPLE = """
            No. of players for each team: 5
            No. of overs: 2
            Batting Order for team 1:
            P1
            P2
            P3
            P4
            P5
            Over 1:
            1
            1
            1
            1
            1
            2
            Over 2:
            W
            4
            4
            Wd
            W
            1
            6
            Batting Order for team 2:
            P6
            P7
            P8
            P9
            P10
            Over 1:
            4
            6
            W
            W
            1
            1
            Over 2:
            6
            1
            W
            W
            """;

    public static void main(String[] args) {
        new MatchRunner().run(SAMPLE);
    }
}
```

Output:

```
Batting order for Team 1: [P1, P2, P3, P4, P5]
Over 1:
Scorecard for Team 1:
Player   Score  4s  6s Balls     SR
P1*          3   0   0     3  100.0
P2*          4   0   0     3  133.3
P3           0   0   0     0      -
P4           0   0   0     0      -
P5           0   0   0     0      -
Total: 7/0   Extras: 0
Overs: 1
Over 2:
Scorecard for Team 1:
Player   Score  4s  6s Balls     SR
P1           3   0   0     4   75.0
P2*         10   0   1     4  250.0
P3           8   2   0     3  266.7
P4*          1   0   0     1  100.0
P5           0   0   0     0      -
Total: 23/2   Extras: 1
Overs: 2
Batting order for Team 2: [P6, P7, P8, P9, P10]
Over 1:
Scorecard for Team 2:
Player   Score  4s  6s Balls     SR
P6          10   1   1     3  333.3
P7*          1   0   0     1  100.0
P8           0   0   0     1    0.0
P9*          1   0   0     1  100.0
P10          0   0   0     0      -
Total: 12/2   Extras: 0
Overs: 1
Over 2:
Scorecard for Team 2:
Player   Score  4s  6s Balls     SR
P6          10   1   1     3  333.3
P7*          8   0   1     3  266.7
P8           0   0   0     1    0.0
P9           1   0   0     2   50.0
P10          0   0   0     1    0.0
Total: 19/4   Extras: 0
Overs: 1.4
Result: Team 1 won the match by 4 runs
```

Notes on the sample and on correctness

- Leaving aside the strike-rate and extras columns we add, our output matches the published sample for team 1 exactly, and for team 2 in every line except two places where the published sample itself is inconsistent: after team 2's first over it prints `12/1`, but the over contains **two** `W` tokens (P6 and P8 were both out), so the correct total is `12/2`; and in the final card it shows P6 with 2 balls after 3 in the previous card. Following the stated rules gives `12/2` and 3 balls, which is what our program prints. Pointing out such inconsistencies calmly is a plus in a machine-coding round.
- Team 2's second over stops after four balls because the fourth wicket leaves only one batter (5 players → all out at 4 wickets), so "Overs: 1.4".
- The loop that reads an over counts **legal** balls (`legalThisOver < 6`), so a wide in team 1's second over makes that over seven tokens long.
- Extras and strike rate (two of the "good to have" items) are printed; bowler figures would need bowler names in the input — add a `Bowler` class with the same counters and a line per over naming the bowler.

Concurrency & edge cases

- **Two scorers, one match.** `Match.bowl`, `startOver` and `startInnings` are `synchronized` on the match, so balls are applied one at a time in order. Different matches never block each other.
- **Many readers.** Observers are held in a `CopyOnWriteArrayList` (a list that makes a fresh copy on every change, so readers never see it half-updated), which is safe to iterate while another thread subscribes. At web

scale, the match would publish each ball to a **message broker** (a middleman service that receives messages and passes them on to every subscriber) and readers would receive it over **WebSockets** (a connection the browser keeps open so the server can send data at any time) or **server-sent events** (a one-way stream of updates from server to browser); the scorecard would be a cached view rebuilt from balls.

- **Illegal sequences rejected:** starting an over while one is in progress; the same bowler twice in a row; a bowler over his quota; bowling after the innings ended; a bowler from the batting side; a twelfth player; a player outside the squad.
- **Impossible dismissals rejected:** bowled, caught, LBW, stumped or hit wicket off a no ball; bowled, caught or LBW off a wide.
- **Wicket on the last ball of an over:** the new batter comes in, then ends change for the new over — both handled in `Innings.record`.
- **Tie:** equal totals give `Result.Tie` (a super over would be an extension).
- **Test matches:** `oversPerInnings() = 0` means no limit; the demo's `declare()` lets a captain close an innings; `declareDraw()` ends a Test that runs out of time.

Extensions & follow-up questions

- **"Who is on strike?"** — The course leaves this out; our `Innings` tracks it (odd runs and end of over swap ends).
- **"Stats are slow to recompute from every ball."** — Keep a running total per player as a cache, updated by an Observer on each ball, and rebuild it from balls whenever a scorer corrects a ball. The balls stay the truth; the total is only a cache.
- **"Add Duckworth–Lewis–Stern."** — A `TargetCalculator` Strategy on the match: the default returns "opponent total + 1"; a DLS implementation recomputes the target from overs lost.
- **"Add a free hit after a no ball."** — The innings remembers "next legal ball is a free hit"; `BallOutcome` validation then allows only run-out-type dismissals on that ball.
- **"Add partnerships and fall of wickets."** — Derive from balls: split the ball list at each wicket and sum.
- **"Add net run rate to the points table."** — Derive from innings totals and overs faced/bowled; still no stored numbers.
- **"Push notifications for wickets and milestones."** — A new `MatchObserver` that sends a push when `ball.wicket() != null` or a batter crosses 50/100.

Real-world example

ESPNcricinfo (founded in 1993 and bought by ESPN in 2007) is the reference site for cricket statistics; its **Statsguru** tool answers questions like the ones in requirement 7 across every recorded international match. Live pages update ball by ball with commentary lines very much like our `CommentaryFeed`, and millions follow big matches on phones — which is why the push-based Observer design (and, at scale, message brokers and caches) matters.

In the interview

Minutes	What to do
0–5	Clarify formats, actors, and that the real rules matter; say early: **"an over ends after six *legal* balls, however many deliveries that takes"**
5–10	Team ⊇ squad ⊇ playing eleven; people hold a `Person`
10–20	Class diagram: match (abstract) → innings → over → ball; formats differ in two numbers
20–30	Ball-recording flow, strike rotation, **the bowler gets no credit for a run out**, byes are not the batter's runs
30–40	Stats derived from balls; Template Method for stat queries; Observers for live feed
40–45	Concurrency (one scorer lock per match, broadcast to readers) and extensions

30-second answer: "I model cricket's rules, not just its nouns. An over is six legal balls; a wide or no ball is re-bowled. Because the overs belong to the innings, only the innings decides when an over ends, who is on strike and when the innings is over. Formats differ only in overs per innings and innings per match, so they override two methods. Stats are derived from the balls so corrections propagate, and each stat question is a Template Method subclass. Live pages subscribe as observers."

Follow-ups with answers

- *Why not store `wickets` on `Player`?* — It would drift from the ball record and cannot be filtered by format or tournament; derive it.
- *How do you add "most catches"?* — Add the fielder to `Wicket`, count it in `PlayerStats`, and write `MostCatches extends StatQuery` — no change to the service's loop.
- *Is Template Method the right name for `Match`/formats?* — The course says only `StatQuery` is a named pattern here; the formats are plain abstract-class specialisation. Be precise about that.

Self-check

- **Q:** A bowler bowls a wide and a no ball in an over. How many deliveries does the over have? **A:** Eight — six legal plus the two illegal ones.
- **Q:** Who is credited with a run-out wicket? **A:** The fielding side, not the bowler; it counts in the team's wickets but not in the bowler's figures.
- **Q:** Are leg byes added to the batter's score? **A:** No — they are extras for the team, and they are not charged to the bowler either.
- **Q:** Why does `Playing11` hold its `TournamentSquad`? **A:** So it can refuse a player not in the squad (requirement 5) — the rule lives in code.
- **Q:** What two methods distinguish `T20`, `ODI` and `Test`? **A:** `oversPerInnings()` (20, 50, 0 = unlimited) and `inningsPerSide()` (1, 1, 2).
- **Q:** In the Udaan variant, what is the correct team 2 score after over 1 of the sample? **A:** 12/2 — the over contains two wickets.

TL;DR

Cricinfo = people hold a `Person`; team ⊇ squad ⊇ eleven; `Match` (abstract, formats override two numbers) → `Innings` (owns overs, strike, end rules) → `Over` (six **legal** balls) → `Ball` (outcome + wicket). Run outs are not the bowler's; byes are not the batter's. Stats are derived from balls; stat questions are **Template Method** subclasses; live pages are **Observers**. The Udaan dashboard is the same responsibilities in a 90-minute, input-driven form.

18.3 Design LinkedIn

Figure — A tablet showing the LinkedIn logo surrounded by a network of professionals, the cover image of this case study (course diagram)

LinkedIn is a networking site for people's **working lives**. Members link up with colleagues and contacts they trust, and they use the site to find their next career move. A profile lists a member's skills, jobs and schooling; the home page shows a **feed** of updates from the member's network, arranged in sections (modules) the

member can reorder. It works much like Facebook, with features specialised for careers: **connections** (mutual, both sides agree), **follows** (one-sided), **endorsements**, **recommendations**, **company pages** and **job postings**.

Problem & clarifying questions

Question to ask	Answer we assume
Is a connection mutual?	Yes: an **invitation** is sent, and the receiver accepts or rejects it. Following is one-sided and needs no approval
Do we show degrees of connection?	Yes: 1st (direct), 2nd (connection of a connection), 3rd; beyond that "out of network"
Who can message whom?	A member can message their connections (messaging most non-connections needs LinkedIn's paid *InMail*, out of scope)
Who can endorse a skill?	Only 1st-degree connections, once per skill
Recommendations?	A member *requests* one from a connection; only that person can write it
Profile stats?	Profile views, number of connections, search appearances
Companies and jobs?	Members create company pages; page admins post jobs; members apply once per job
Feed?	Posts from connections and followed members, ranked (the course leaves the feed out; we add a simple one)

Requirements

Functional (the course's twelve, in our words)

1. A member fills in a profile: basic details, jobs held, schooling, skills and accomplishments.
2. Any user can search for members or companies by name (and jobs by title).
3. Members send, accept (or reject) connection requests.
4. Members can request a recommendation from other members.
5. The system shows profile stats: profile views, total connections, search appearances.
6. Members create posts to share with their connections.
7. Members can comment on posts; they can like or share both posts **and comments**.
8. Members send messages to other members.
9. A member gets a notification for each new message, each connection invitation, and each comment left on one of their posts.
10. Members create company pages and add job postings.
11. Members create groups and join any group.
12. Members follow other members or companies.

Non-functional

- **Symmetry:** a connection must always appear on *both* members, never only one.
- **Fast reads:** feed, search and degree lookups are far more frequent than writes.
- **Privacy/abuse:** blocked members' content disappears; you cannot spam strangers with messages.
- **Extensibility:** new notification channels and feed-ranking rules plug in without edits.

Use cases

Three actors: **Member** (search, connect, post, message, follow…), **Admin** (block and unblock members) and **System** (sends notifications for new messages, invitations and so on).

Top use cases: add/update profile; search members, companies or jobs and send connection requests; follow or unfollow a member or company; send a message to connections; create a post, like and comment; send notifications.

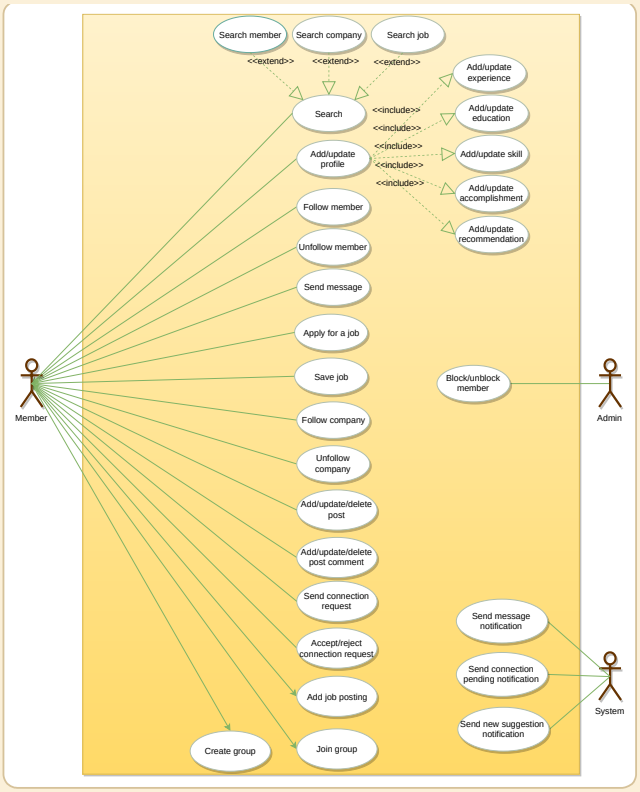


Figure — LinkedIn use cases for Member, Admin and System, including follow/unfollow member, with `<<include>>` and `<<extend>>` relationships (course diagram)

Activity — add experience to a profile. The member opens *Add Experience*, enters title, company, location, dates and description, optionally uploads a document or photo with a description, and presses save. The system verifies the data; if anything is missing it notifies the member, otherwise it saves the experience.

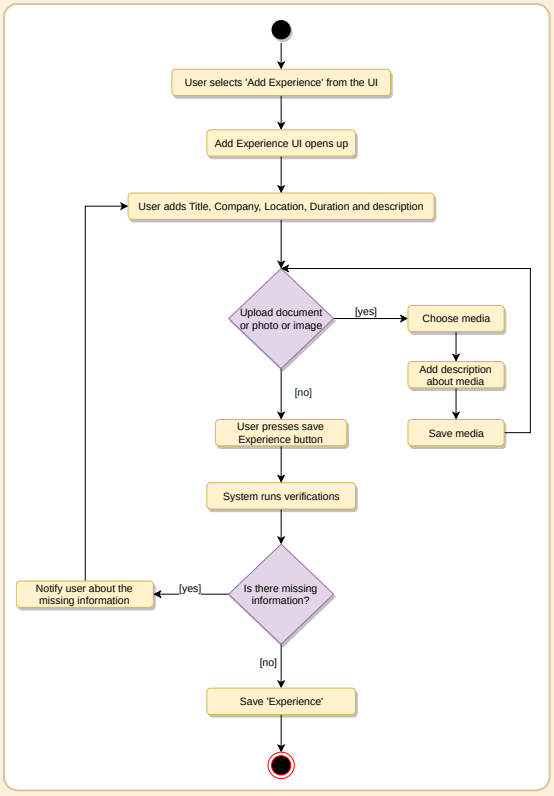


Figure — Activity diagram "add experience to profile" with optional media upload and a verification loop (course diagram)

Activity — send a message. The member composes a message, picks one or more recipients, adds a body and optional media, and presses send. After verification the message goes to every recipient, and each gets an email and a push notification.

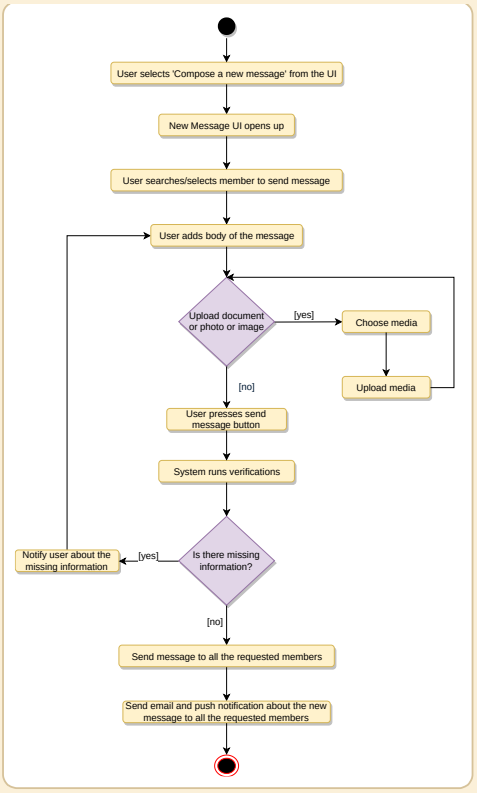


Figure — Activity diagram "send message": choose recipients, add body and media, verify, deliver to all recipients, notify them by email and push (course diagram)

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, connection set (changed only by the service), follows, posts, inbox, notifications
`Profile`	Owns `Experience`, `Education`, `Skill`, `Accomplishment`, `Recommendation`, `ProfileStats`
`Skill`	Name + set of endorsers (no double endorsement)
`Recommendation`	Created empty on request; only the named author writes it (REQUESTED → WRITTEN)
`ProfileStats`	View and search-appearance counters pushed in from where the events happen
`SocialContent` (abstract) → `Post`, `Comment`	Author, text, time, likes as a set; posts own comments and can be shares of another post
`Message` (record)	Sender, recipients, text — not "social content" (nobody likes a message). The course allows media on messages and posts too; our code keeps text only
`ConnectionInvitation` + `InvitationStatus`	State over time: PENDING → ACCEPTED / REJECTED / CANCELED
`ConnectionService`	Invite, accept, reject, withdraw; degree (BFS); mutual connections; people-you-may-know
`Company`, `JobPosting`, `JobApplication`, `Group`	Pages, jobs (apply once), groups (owner cannot leave)
`Notification`, `NotificationService`, channels	One notification class with a *type* field; delivery Strategy
`SearchIndex`	Word → set of members / companies / jobs
`FeedService` + `FeedRanker`	Pull-model feed with pluggable ranking
`LinkedIn`	Facade

Class diagram

The course builds LinkedIn in **six stages**.

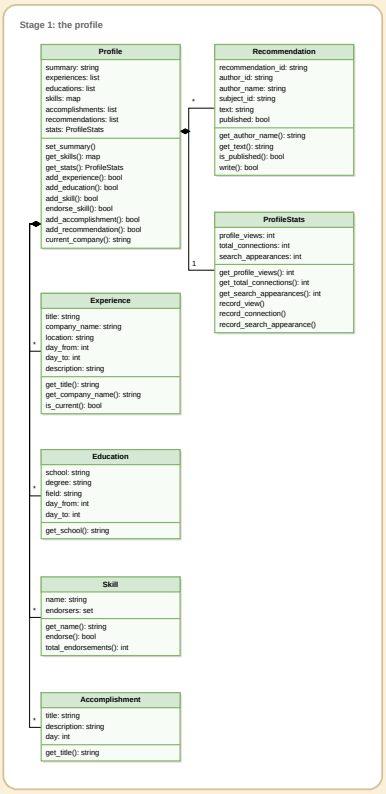


Figure — Stage 1: `Profile` owning `Experience`, `Education`, `Skill`, `Accomplishment`, `Recommendation` and `ProfileStats` (course diagram)

Stage 1 — the profile. Each of requirement 1's five parts is a small class, not a line of text, because other code uses them (search looks up the current employer; endorsements are counted on the skill). Two details matter. You cannot pass text when creating a `Recommendation`: requirement 4 is about *asking* for one, so the object is born empty and the author fills it in later. And the profile never updates `ProfileStats` itself: whoever witnesses the event bumps the counter — the code that shows a profile, the code that accepts an

invitation, the code that returns search results — because an object cannot count events it never sees. All arrows out of `Profile` are composition: delete the profile and its parts go with it.

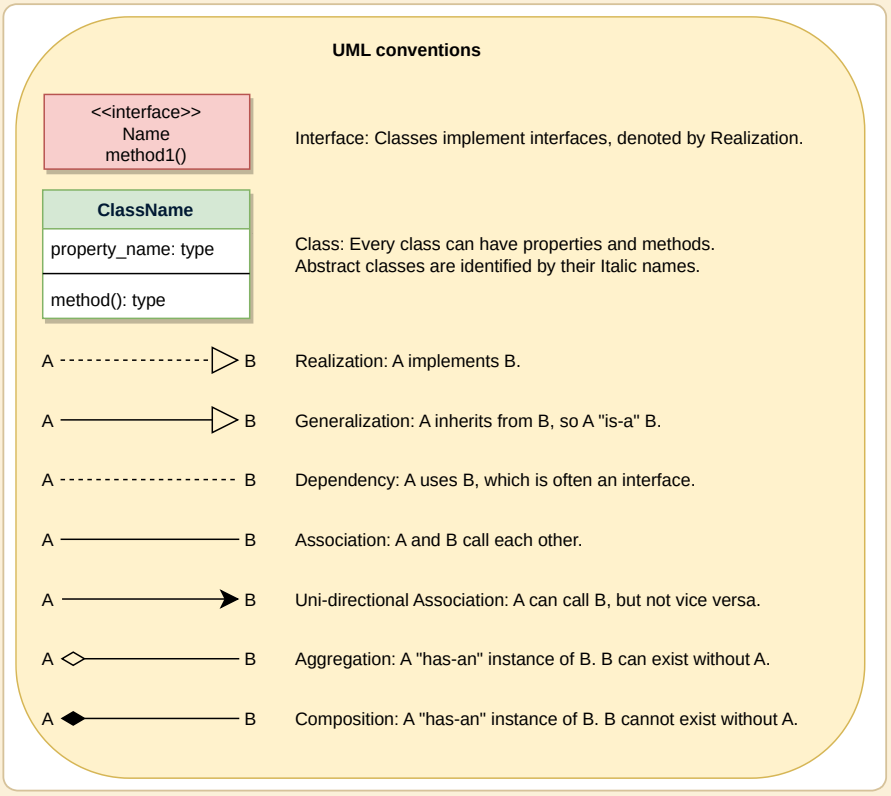


Figure — UML conventions: interface, class, abstract class, realization, generalization, dependency, association, aggregation and composition (course diagram)

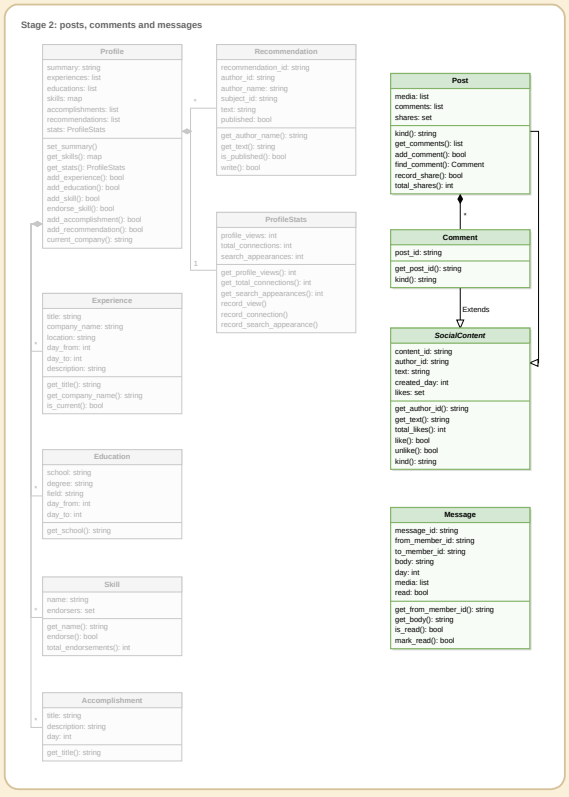


Figure — Stage 2: `SocialContent` with `Post` and `Comment`; `Post` owns its comments; `Message` stands apart (course diagram)

`Stage 2 — posts, comments and messages.` Requirement 7 hides a detail: *comments get likes as well.* Put likes only on `Post` and part of the requirement has no home. So both classes extend one parent, `SocialContent`, which holds the author, the text and the likes. The likes are kept as the **ids of the members who liked**, not as a

number, which makes a second like by the same person impossible and lets a like be taken back. `Message` is not part of this family: it is delivered to people rather than published, and no one likes a message.

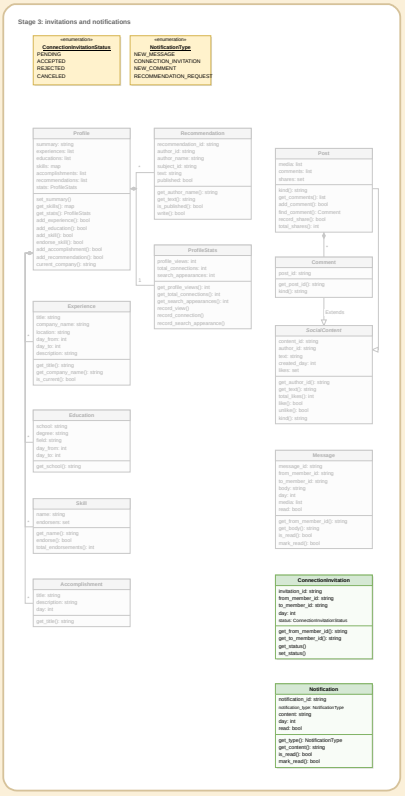


Figure — Stage 3: `ConnectionInvitation` (with a status) and `Notification` (with a type) (course diagram)

`Stage 3 — invitations and notifications.` An invitation to connect starts pending and later becomes accepted or rejected. Because it **changes state as time passes**, it deserves its own object (this is the idea behind the **State** pattern: an object's allowed actions depend on which state it is in, and only certain moves between states are legal) rather than a true/false field on a member, and that object can still record, afterwards, that the answer was no. Requirement 9 lists three notification events, but *delivering* them is identical, so the kind is a **field**, not three subclasses. The general rule: **subclass when behaviour differs; use a field when only data differs.**

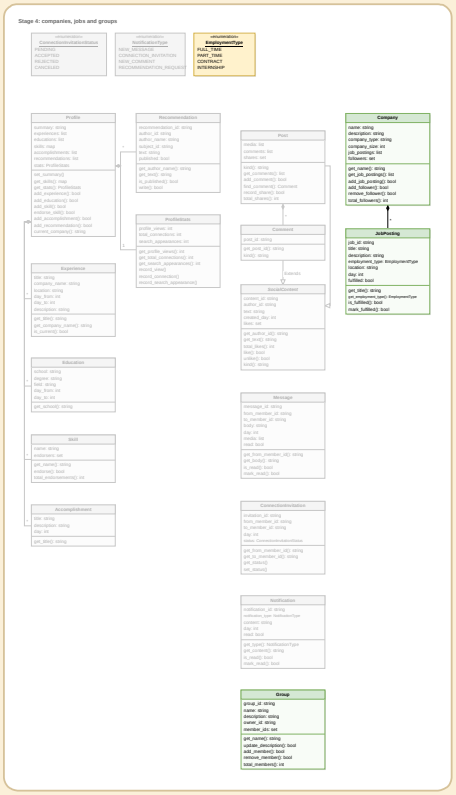


Figure — Stage 4: `Company` owning its `JobPosting`s, with followers as ids; `Group` with members and an owner (course diagram)

Stage 4 — companies, jobs and groups. A company page owns its job postings (a posting means nothing once the page is gone) and keeps followers as a set of ids. A group records who created it, and that creator is never allowed to leave or be removed.

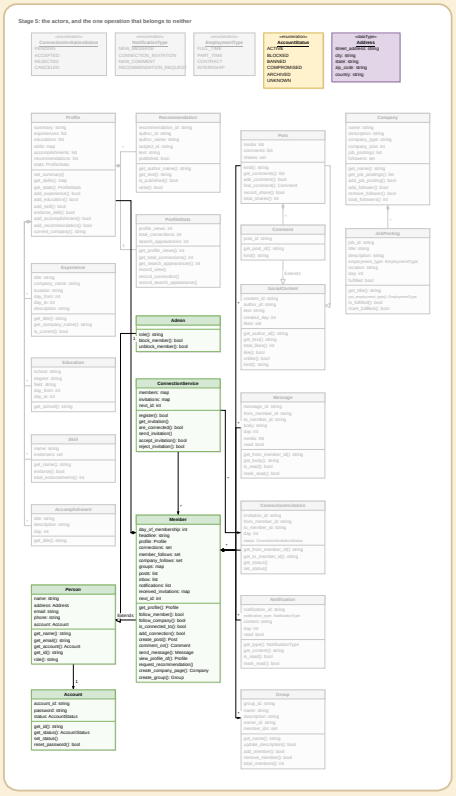


Figure — Stage 5: `Account`, `Person`, `Member`, `Admin` and `ConnectionService`, which points at both invitations and members (course diagram)

Stage 5 — the actors, and an operation that fits neither of them. Nearly every requirement becomes a method on `Member`. **Accepting an invitation does not**: it edits two members together. If the sender's class did it, the receiver's connection list would be changed by a different object; if each side did half, a bug would one day update only one half, leaving a one-way link. So the operation goes into a small `ConnectionService`, and that

single class is responsible for keeping connections **symmetric** (A lists B exactly when B lists A). Following is different: it is one-sided and needs no agreement, so it is just a set on the follower, no service needed.

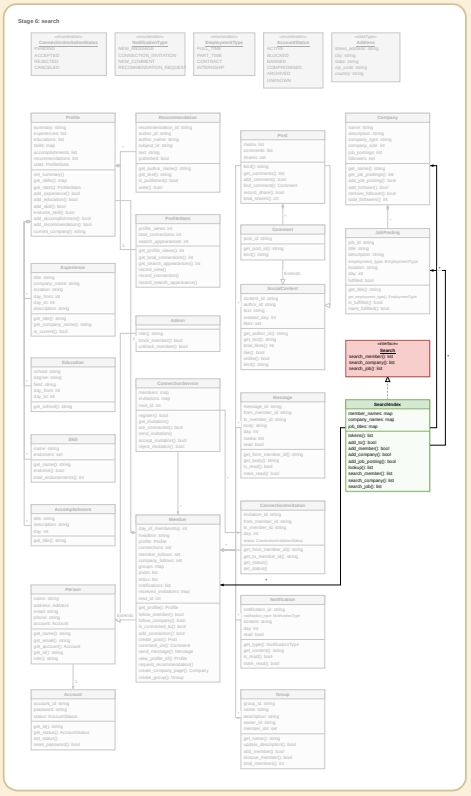


Figure — Stage 6: the `Search` interface and `SearchIndex` with three maps from key to list (members, companies, job postings) (course diagram)

Stage 6 — search. The course shows a real bug from an earlier version: on the first occurrence of a name the index saved *one* member object, and on the next occurrence it tried to "add" to that object — fine for the first Sharma, broken for the second. The cure is a rule: **an index value is always a collection, even when it holds one item.** Keys are single lowercase words, so first and last names both work.

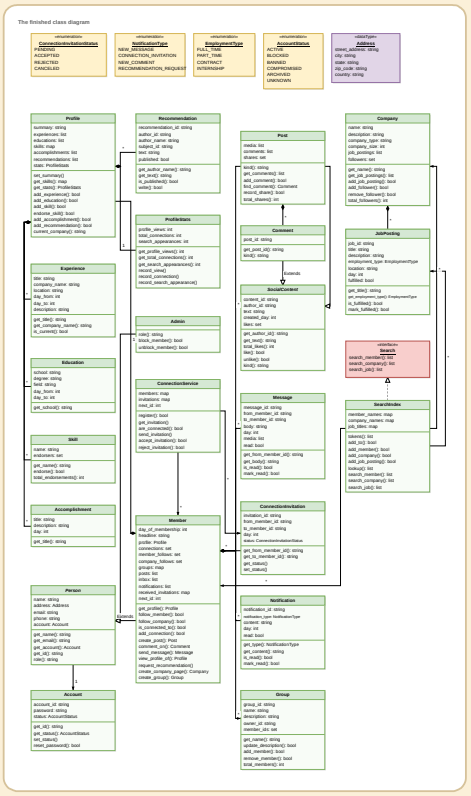
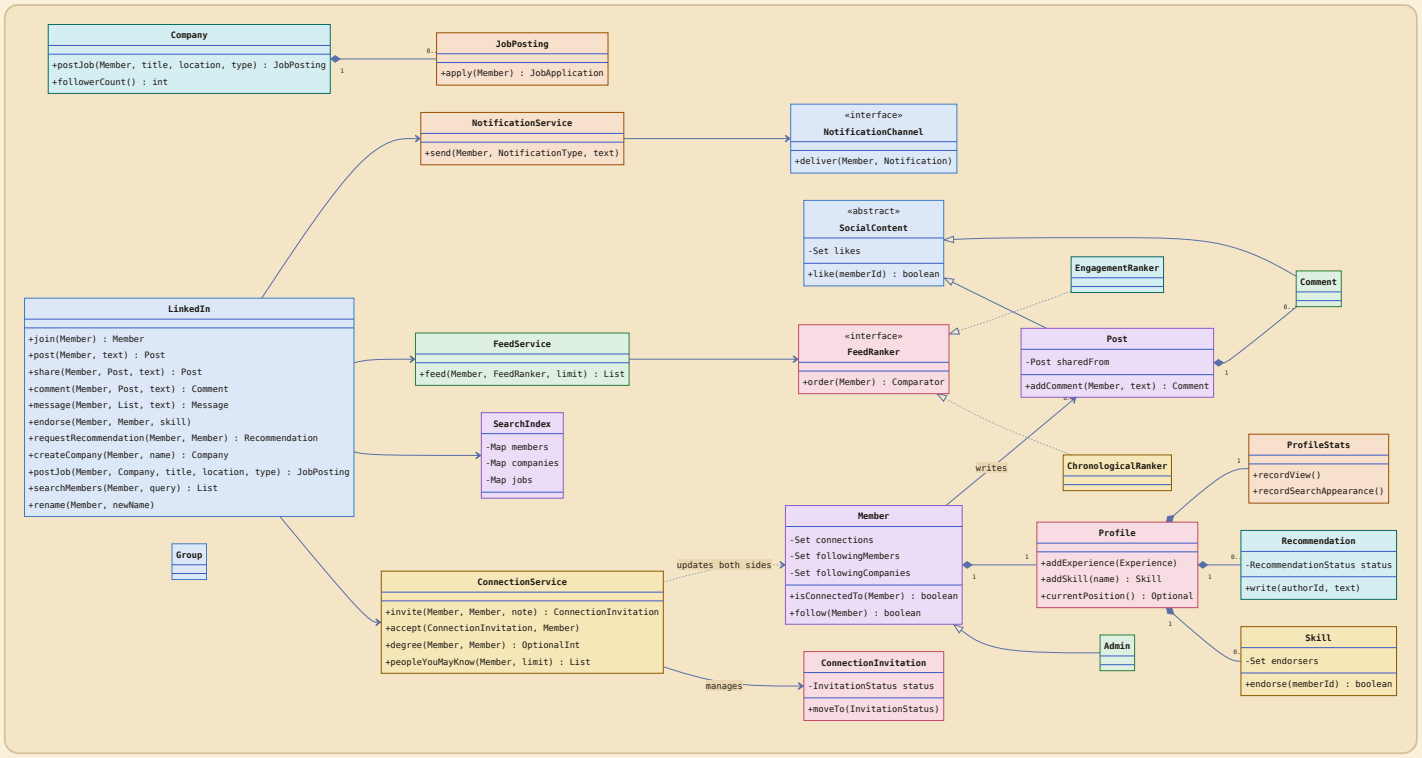


Figure — The finished LinkedIn class diagram, with all six stages in full colour (course diagram)

The course deliberately omits several features: the feed and how it is ranked, suggested connections, degrees of separation, applying for jobs and tracking applicants, privacy controls, and one member blocking another. It

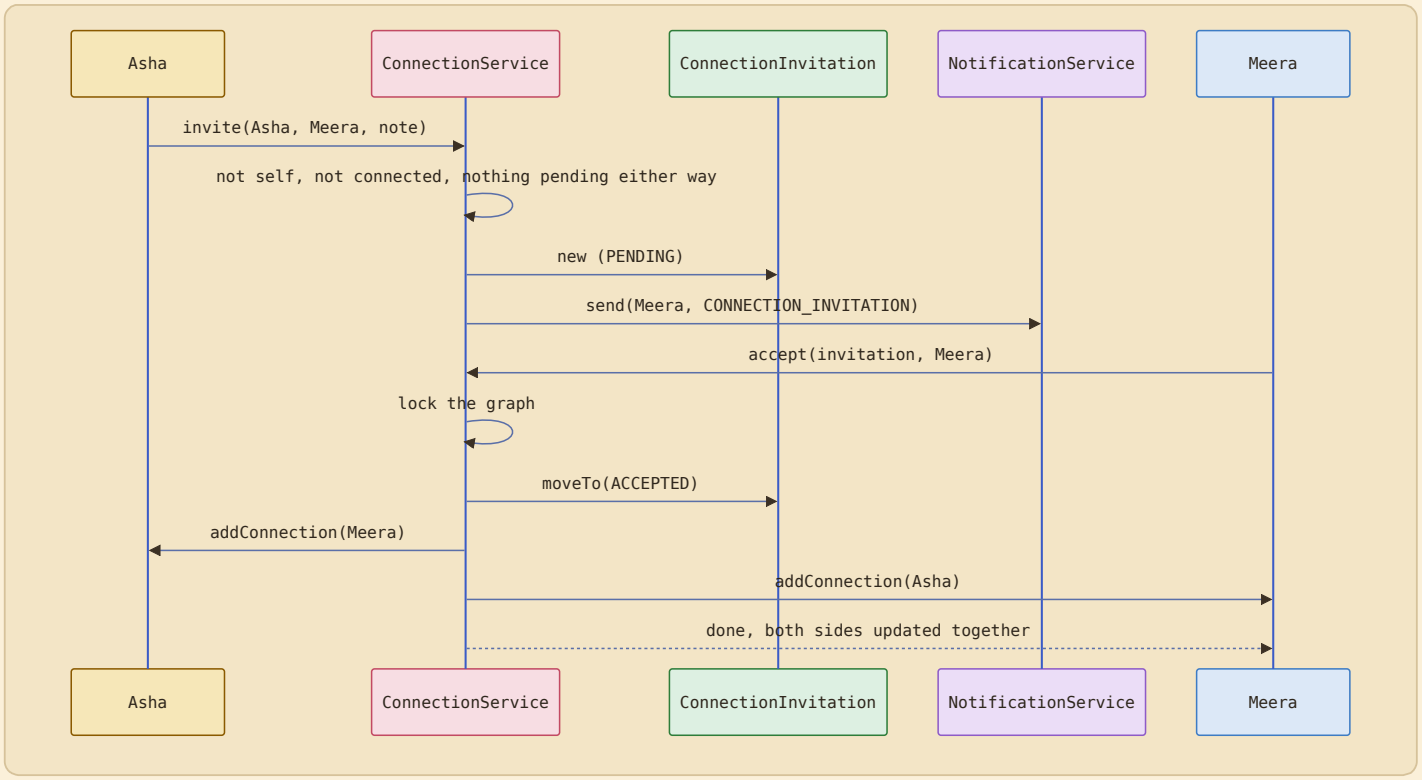
also admits two weak spots a reviewer will probe. First, there is no feed; building one means either gathering the connections' posts at the moment the page loads, or copying each new post into all the connections' feeds when it is written, and which is better depends on how many connections a typical member has. Second, the search index goes stale (out of date) when a member changes their name, because nothing informs it.

Our Java design follows the six stages and fills the gaps the course names: a **feed** (pull model — the feed is gathered when the member opens it — with a ranking Strategy), **degrees of connection** (breadth-first search, described under Key flows), **people you may know**, a **rename that updates the index**, job **applications**, and connection-count derived from the set instead of a stored counter.

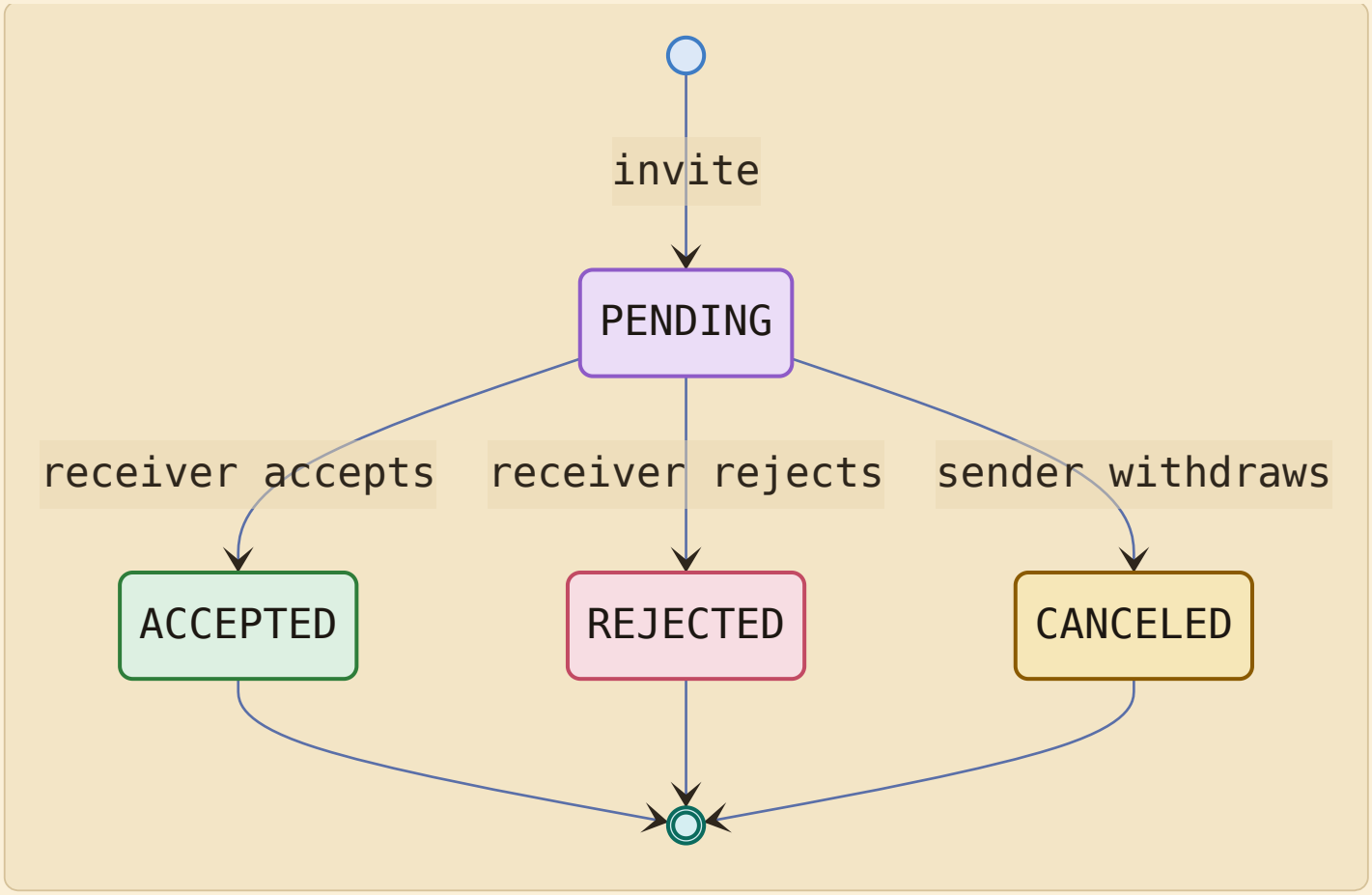


Key flows

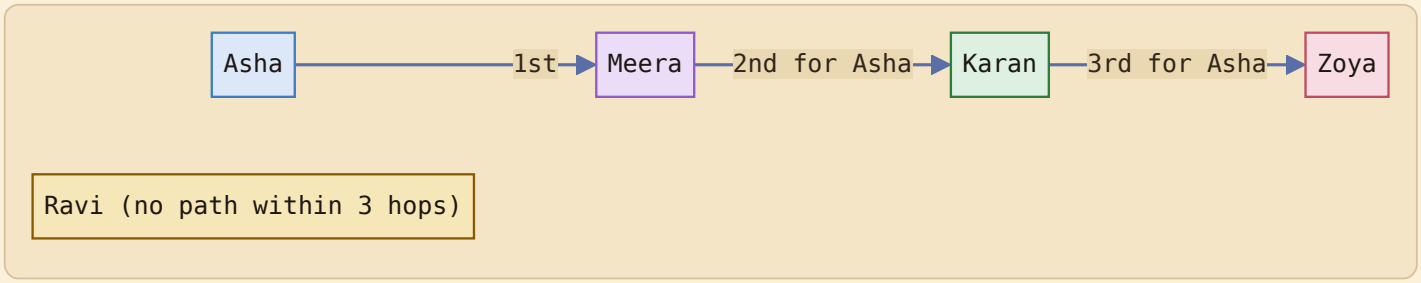
Invite and accept (the two-sided operation):



Invitation states:



Degree of connection — breadth-first search (**BFS**: explore all 1-hop neighbours, then all 2-hop, and so on) from the viewer, stopping at depth 3:



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`ConnectionService.accept` updates both members under one lock	Service for a two-object operation; Information Expert at the right level	Symmetry has one guardian	`member.accept(inv)` editing the other member
Following is a set on the follower	Simplicity	One-sided relationship needs no agreement or service	Reusing invitations for follows
`ConnectionInvitation` with explicit states	**State** (as an enum with allowed transitions)	Accept-twice and accept-after-reject are rejected	A boolean `accepted` flag
`SocialContent` base for `Post` and `Comment`	Inheritance for shared behaviour	Requirement 7: comments are liked too	Likes only on `Post`
One `Notification` class with a `type` field	"Field when only data differs"	Delivery is identical	A subclass per event
`NotificationChannel` interface	**Strategy**	In-app, email, push are interchangeable	`if (email) … else if (push) …`
`FeedRanker` interface	**Strategy**	"Recent" vs "Top" ranking swap at run time	Sorting code inside the feed service
Index maps key → set	Robust inverted index	Two people named Sharma both found	Key → single member (the course's bug)
`LinkedIn.rename` tells the index	Keep derived structures in sync at the write point	Search is never stale	Rebuilding the index nightly
Connection count = `connections().size()`	Derive, don't store	Cannot disagree with the set	A separate counter bumped on accept

Java implementation

Package `lld.ch18.linkedin`.

```java
// File: lld/ch18/linkedin/Profile.java
package lld.ch18.linkedin;

import java.time.YearMonth;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

enum EmploymentType { FULL_TIME, PART_TIME, CONTRACT, INTERNSHIP }

/** A deterministic logical clock so the demo output is repeatable (a real system uses Instant.now()). */
final class Tick {
    private static final AtomicLong NOW = new AtomicLong();
    static long next() { return NOW.incrementAndGet(); }
    static long now() { return NOW.get(); }
    private Tick() {}
}

record Experience(String title, String company, String location, YearMonth from, YearMonth to, EmploymentType type)
{
    boolean isCurrent() { return to == null; }
}

record Education(String school, String degree, int fromYear, int toYear) {}

record Accomplishment(String title, String description) {}

/** A skill keeps the set of members who endorsed it, so nobody endorses twice. */
final class Skill {
    private final String name;
    private final Set<Long> endorsers = new LinkedHashSet<>();
    Skill(String name) { this.name = name; }
    String name() { return name; }
    synchronized boolean endorse(long memberId) { return endorsers.add(memberId); }
    synchronized int endorsements() { return endorsers.size(); }
}

enum RecommendationStatus { REQUESTED, WRITTEN, DECLINED }

/** Created EMPTY when requested; only the named author may fill it in, once. */
final class Recommendation {
    private final long authorId;
    private final String authorName;
    private RecommendationStatus status = RecommendationStatus.REQUESTED;
    private String text;

    Recommendation(long authorId, String authorName) { this.authorId = authorId; this.authorName = authorName; }

    synchronized void write(long byMemberId, String text) {
        if (byMemberId != authorId) throw new IllegalStateException("only " + authorName + " can write this
recommendation");
        if (status != RecommendationStatus.REQUESTED) throw new IllegalStateException("recommendation is " +
status);
        this.text = text;
        this.status = RecommendationStatus.WRITTEN;
    }

    long authorId() { return authorId; }
    String authorName() { return authorName; }
    synchronized RecommendationStatus status() { return status; }
    synchronized Optional<String> text() { return Optional.ofNullable(text); }
}
```

```java
/**
 * Counters that are pushed in by whoever sees the event (a profile cannot
 * observe its own views or search appearances). The connection count is NOT
 * stored here: it is the size of the connection set, so it can never disagree.
 */
final class ProfileStats {
    private final AtomicInteger profileViews = new AtomicInteger();
    private final AtomicInteger searchAppearances = new AtomicInteger();
    void recordView() { profileViews.incrementAndGet(); }
    void recordSearchAppearance() { searchAppearances.incrementAndGet(); }
    int profileViews() { return profileViews.get(); }
    int searchAppearances() { return searchAppearances.get(); }
}

/** The profile owns all its parts (composition): delete the profile and they go too. */
final class Profile {
    private volatile String headline = "";
    private volatile String summary = "";
    private final List<Experience> experiences = new ArrayList<>();
    private final List<Education> educations = new ArrayList<>();
    private final Map<String, Skill> skills = new LinkedHashMap<>();
    private final List<Accomplishment> accomplishments = new ArrayList<>();
    private final List<Recommendation> recommendations = new ArrayList<>();
    private final ProfileStats stats = new ProfileStats();

    void setHeadline(String h) { headline = h; }
    void setSummary(String s) { summary = s; }
    String headline() { return headline; }
    String summary() { return summary; }

    synchronized void addExperience(Experience e) {
        if (e.to() != null && e.to().isBefore(e.from())) throw new IllegalArgumentException("experience ends before
it starts");
        experiences.add(e);
    }
    synchronized void addEducation(Education e) { educations.add(e); }
    synchronized void addAccomplishment(Accomplishment a) { accomplishments.add(a); }
    synchronized Skill addSkill(String name) { return skills.computeIfAbsent(name, Skill::new); }
    synchronized Optional<Skill> skill(String name) { return Optional.ofNullable(skills.get(name)); }
    synchronized void addRecommendation(Recommendation r) { recommendations.add(r); }

    synchronized List<Experience> experiences() { return List.copyOf(experiences); }
    synchronized List<Education> educations() { return List.copyOf(educations); }
    synchronized List<Skill> skills() { return List.copyOf(skills.values()); }
    synchronized List<Recommendation> recommendations() { return List.copyOf(recommendations); }
    ProfileStats stats() { return stats; }

    synchronized Optional<Experience> currentPosition() {
        return experiences.stream().filter(Experience::isCurrent).findFirst();
    }
}
```

```java
// File: lld/ch18/linkedin/Content.java
package lld.ch18.linkedin;

import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

/** Posts AND comments can be liked, so likes live in a shared base class, as a set of member ids. */
abstract class SocialContent {
    private static final AtomicLong IDS = new AtomicLong(1000);
    private final long id = IDS.incrementAndGet();
    private final Member author;
    private final String text;
    private final long createdAt = Tick.next();
    private final Set<Long> likes = new LinkedHashSet<>();

    SocialContent(Member author, String text) {
        if (text == null || text.isBlank()) throw new IllegalArgumentException("text cannot be empty");
        this.author = author;
        this.text = text;
    }

    long id() { return id; }
    Member author() { return author; }
    String text() { return text; }
    long createdAt() { return createdAt; }
    synchronized boolean like(long memberId) { return likes.add(memberId); }
    synchronized boolean unlike(long memberId) { return likes.remove(memberId); }
    synchronized int likeCount() { return likes.size(); }
}

final class Post extends SocialContent {
    private final List<Comment> comments = new ArrayList<>();
    private final Post sharedFrom;                  // null for an original post
    private final AtomicInteger shares = new AtomicInteger();

    Post(Member author, String text, Post sharedFrom) {
        super(author, text);
        this.sharedFrom = sharedFrom;
    }

    synchronized Comment addComment(Member by, String text) {
        Comment c = new Comment(by, text, this);
        comments.add(c);
        return c;
    }
    synchronized List<Comment> comments() { return List.copyOf(comments); }
    Optional<Post> sharedFrom() { return Optional.ofNullable(sharedFrom); }
    void recordShare() { shares.incrementAndGet(); }
    int shares() { return shares.get(); }

    @Override public String toString() {
        return "\"" + text() + "\" by " + author().name()
                + sharedFrom().map(p -> " (shared from " + p.author().name() + ")").orElse("");
    }
}

final class Comment extends SocialContent {
    private final Post post;
    Comment(Member author, String text, Post post) { super(author, text); this.post = post; }
    Post post() { return post; }
}
```

```java
/** Sent to people, never posted; nobody likes a message, so it is not SocialContent. */
record Message(long id, Member sender, List<Member> recipients, String text, long sentAt) {}
```

```java
// File: lld/ch18/linkedin/Network.java
package lld.ch18.linkedin;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.Deque;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.OptionalInt;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.LongFunction;

enum AccountStatus { ACTIVE, BLOCKED, BANNED, COMPROMISED, ARCHIVED, UNKNOWN }

class Member {
    private static final AtomicLong IDS = new AtomicLong();
    private final long id = IDS.incrementAndGet();
    private volatile String name;
    private final String email;
    private final Profile profile = new Profile();
    private volatile AccountStatus status = AccountStatus.ACTIVE;
    // Connections change only through ConnectionService, which keeps both sides symmetric.
    private final Set<Long> connections = ConcurrentHashMap.newKeySet();
    // Following is one-sided: nobody has to agree, so it is just a set on the follower.
    private final Set<Long> followingMembers = ConcurrentHashMap.newKeySet();
    private final Set<Long> followingCompanies = ConcurrentHashMap.newKeySet();
    private final List<Post> posts = new CopyOnWriteArrayList<>();
    private final List<Message> inbox = new CopyOnWriteArrayList<>();
    private final List<Notification> notifications = new CopyOnWriteArrayList<>();

    Member(String name, String email) { this.name = name; this.email = email; }

    long id() { return id; }
    String name() { return name; }
    void rename(String newName) { this.name = newName; }
    String email() { return email; }
    Profile profile() { return profile; }
    AccountStatus status() { return status; }
    void setStatus(AccountStatus s) { status = s; }

    Set<Long> connections() { return Set.copyOf(connections); }
    boolean isConnectedTo(Member other) { return connections.contains(other.id()); }
    void addConnection(long otherId) { connections.add(otherId); }        // package-private: only the service
calls it
    void removeConnection(long otherId) { connections.remove(otherId); }

    boolean follow(Member m) { return m.id != id && followingMembers.add(m.id()); }
    boolean unfollow(Member m) { return followingMembers.remove(m.id()); }
    boolean followCompany(Company c) { return followingCompanies.add(c.id()); }
    Set<Long> followingMembers() { return Set.copyOf(followingMembers); }
    Set<Long> followingCompanies() { return Set.copyOf(followingCompanies); }

    void addPost(Post p) { posts.add(p); }
    List<Post> posts() { return List.copyOf(posts); }
    void receive(Message m) { inbox.add(m); }
    List<Message> inbox() { return List.copyOf(inbox); }
    void notify(Notification n) { notifications.add(n); }
    List<Notification> notifications() { return List.copyOf(notifications); }

    @Override public String toString() { return name; }
}
```

```
final class Admin extends Member {
    Admin(String name, String email) { super(name, email); }
}

enum InvitationStatus {
    PENDING, ACCEPTED, REJECTED, CANCELED;
    boolean canMoveTo(InvitationStatus next) { return this == PENDING && next != PENDING; }
}

/** A connection request is an object with a state over time: it can remember that it was rejected. */
final class ConnectionInvitation {
    private static final AtomicLong IDS = new AtomicLong();
    private final long id = IDS.incrementAndGet();
    private final Member from;
    private final Member to;
    private final String note;
    private InvitationStatus status = InvitationStatus.PENDING;

    ConnectionInvitation(Member from, Member to, String note) { this.from = from; this.to = to; this.note = note; }

    long id() { return id; }
    Member from() { return from; }
    Member to() { return to; }
    String note() { return note; }
    synchronized InvitationStatus status() { return status; }

    synchronized void moveTo(InvitationStatus next) {
        if (!status.canMoveTo(next)) throw new IllegalStateException("invitation is already " + status);
        status = next;
    }
}

/**
 * Accepting an invitation changes TWO members, so it belongs to neither: it lives here.
 * This service is the only code that must be right for connections to stay symmetric.
 */
final class ConnectionService {
    private final Map<Long, ConnectionInvitation> invitations = new ConcurrentHashMap<>();
    private final NotificationService notifications;
    private final LongFunction<Member> lookup;

    ConnectionService(NotificationService notifications, LongFunction<Member> lookup) {
        this.notifications = notifications;
        this.lookup = lookup;
    }

    synchronized ConnectionInvitation invite(Member from, Member to, String note) {
        if (from.id() == to.id()) throw new IllegalArgumentException("cannot connect to yourself");
        if (from.isConnectedTo(to)) throw new IllegalStateException(from + " and " + to + " are already
connected");
        if (pendingBetween(from, to)) throw new IllegalStateException("an invitation between " + from + " and " +
to + " is already pending");
        ConnectionInvitation inv = new ConnectionInvitation(from, to, note);
        invitations.put(inv.id(), inv);
        notifications.send(to, NotificationType.CONNECTION_INVITATION, from + " wants to connect: \"" + note +
"\"");
        return inv;
    }

    /** One lock for the whole graph change: both sides are updated together or not at all. */
    synchronized void accept(ConnectionInvitation inv, Member by) {
        if (by.id() != inv.to().id()) throw new IllegalStateException("only " + inv.to() + " can accept");
        inv.moveTo(InvitationStatus.ACCEPTED);
        inv.from().addConnection(inv.to().id());
        inv.to().addConnection(inv.from().id());
    }

    synchronized void reject(ConnectionInvitation inv, Member by) {
        if (by.id() != inv.to().id()) throw new IllegalStateException("only " + inv.to() + " can reject");
```

```
        inv.moveTo(InvitationStatus.REJECTED);
    }

    synchronized void withdraw(ConnectionInvitation inv, Member by) {
        if (by.id() != inv.from().id()) throw new IllegalStateException("only " + inv.from() + " can withdraw");
        inv.moveTo(InvitationStatus.CANCELED);
    }

    synchronized void removeConnection(Member a, Member b) {
        a.removeConnection(b.id());
        b.removeConnection(a.id());
    }

    boolean pendingBetween(Member a, Member b) {
        return invitations.values().stream().anyMatch(i -> i.status() == InvitationStatus.PENDING
                && ((i.from() == a && i.to() == b) || (i.from() == b && i.to() == a)));
    }

    /** Degree of connection (1st, 2nd, 3rd) by breadth-first search; empty means "out of network". */
    OptionalInt degree(Member from, Member to) {
        if (from.id() == to.id()) return OptionalInt.of(0);
        Set<Long> seen = new HashSet<>(Set.of(from.id()));
        Deque<long[]> queue = new ArrayDeque<>();                // {memberId, depth}
        queue.add(new long[]{from.id(), 0});
        while (!queue.isEmpty()) {
            long[] cur = queue.poll();
            if (cur[1] == 3) continue;                             // LinkedIn stops at 3rd degree
            for (long next : lookup.apply(cur[0]).connections()) {
                if (next == to.id()) return OptionalInt.of((int) cur[1] + 1);
                if (seen.add(next)) queue.add(new long[]{next, cur[1] + 1});
            }
        }
        return OptionalInt.empty();
    }

    Set<Long> mutualConnections(Member a, Member b) {
        Set<Long> mutual = new HashSet<>(a.connections());
        mutual.retainAll(b.connections());
        return mutual;
    }

    /** "People you may know": 2nd-degree members ranked by mutual connections, then name. */
    List<Member> peopleYouMayKnow(Member me, int limit) {
        Map<Long, Integer> mutualCount = new HashMap<>();
        for (long friend : me.connections()) {
            for (long candidate : lookup.apply(friend).connections()) {
                if (candidate != me.id() && !me.connections().contains(candidate)) mutualCount.merge(candidate, 1,
Integer::sum);
            }
        }
        List<Member> result = new ArrayList<>();
        mutualCount.keySet().stream().map(lookup::apply)
                .filter(m -> !pendingBetween(me, m))
                .sorted(Comparator.comparing((Member m) -> -mutualCount.get(m.id())).thenComparing(Member::name))
                .limit(limit).forEach(result::add);
        return result;
    }
}
```

```java
// File: lld/ch18/linkedin/Pages.java
package lld.ch18.linkedin;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

final class Company {
    private static final AtomicLong IDS = new AtomicLong(500);
    private final long id = IDS.incrementAndGet();
    private final String name;
    private final Member createdBy;
    private final Set<Long> followers = ConcurrentHashMap.newKeySet();
    private final List<JobPosting> jobs = new CopyOnWriteArrayList<>();     // owned: gone with the page

    Company(String name, Member createdBy) { this.name = name; this.createdBy = createdBy; }

    long id() { return id; }
    String name() { return name; }
    Member createdBy() { return createdBy; }
    void addFollower(long memberId) { followers.add(memberId); }
    Set<Long> followers() { return Set.copyOf(followers); }
    int followerCount() { return followers.size(); }          // counted, never stored

    JobPosting postJob(Member by, String title, String location, EmploymentType type) {
        if (by.id() != createdBy.id()) throw new IllegalStateException(by + " is not an admin of " + name);
        JobPosting job = new JobPosting(this, title, location, type);
        jobs.add(job);
        return job;
    }
    List<JobPosting> jobs() { return List.copyOf(jobs); }
    @Override public String toString() { return name; }
}

enum ApplicationStatus { APPLIED, IN_REVIEW, REJECTED, HIRED }

record JobApplication(Member applicant, long appliedAt, ApplicationStatus status) {}

final class JobPosting {
    private static final AtomicLong IDS = new AtomicLong(700);
    private final long id = IDS.incrementAndGet();
    private final Company company;
    private final String title;
    private final String location;
    private final EmploymentType type;
    private final Map<Long, JobApplication> applications = new LinkedHashMap<>();
    private volatile boolean open = true;

    JobPosting(Company company, String title, String location, EmploymentType type) {
        this.company = company; this.title = title; this.location = location; this.type = type;
    }

    long id() { return id; }
    Company company() { return company; }
    String title() { return title; }
    String location() { return location; }
    EmploymentType type() { return type; }
    boolean isOpen() { return open; }
    void close() { open = false; }

    synchronized JobApplication apply(Member m) {
        if (!open) throw new IllegalStateException("job " + title + " is closed");
        if (applications.containsKey(m.id())) throw new IllegalStateException(m + " already applied to " + title);
        JobApplication a = new JobApplication(m, Tick.next(), ApplicationStatus.APPLIED);
```

```
        applications.put(m.id(), a);
        return a;
    }
    synchronized int applicantCount() { return applications.size(); }
    @Override public String toString() { return title + " @ " + company + " (" + location + ", " + type + ")"; }
}

final class Group {
    private static final AtomicLong IDS = new AtomicLong(900);
    private final long id = IDS.incrementAndGet();
    private final String name;
    private final long ownerId;
    private final Set<Long> members = ConcurrentHashMap.newKeySet();

    Group(String name, Member owner) {
        this.name = name;
        this.ownerId = owner.id();
        members.add(owner.id());
    }

    long id() { return id; }
    String name() { return name; }
    boolean join(Member m) { return members.add(m.id()); }
    boolean leave(Member m) {
        if (m.id() == ownerId) throw new IllegalStateException("the owner cannot leave their own group");
        return members.remove(m.id());
    }
    int memberCount() { return members.size(); }
}
```

```java
// File: lld/ch18/linkedin/Services.java
package lld.ch18.linkedin;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.stream.Stream;

enum NotificationType { NEW_MESSAGE, CONNECTION_INVITATION, NEW_COMMENT, RECOMMENDATION_REQUEST }

/** Only the data differs between notification kinds, so the kind is a field, not a subclass. */
record Notification(NotificationType type, String text, long at) {}

/** Strategy for delivery: in-app, email, push... */
interface NotificationChannel { void deliver(Member to, Notification n); }

final class InAppChannel implements NotificationChannel {
    @Override public void deliver(Member to, Notification n) { to.notify(n); }
}

final class EmailChannel implements NotificationChannel {
    @Override public void deliver(Member to, Notification n) {
        System.out.println("    [email to " + to.email() + "] " + n.type() + ": " + n.text());
    }
}

final class NotificationService {
    private final List<NotificationChannel> channels = new CopyOnWriteArrayList<>();
    NotificationService(List<NotificationChannel> channels) { this.channels.addAll(channels); }
    void send(Member to, NotificationType type, String text) {
        Notification n = new Notification(type, text, Tick.next());
        channels.forEach(c -> c.deliver(to, n));
    }
}

interface Search {
    List<Member> searchMembers(String name);
    List<Company> searchCompanies(String name);
    List<JobPosting> searchJobs(String title);
}

/**
 * Inverted indexes. RULE: every key maps to a LIST, even for one match,
 * so the second "Sharma" never breaks the index.
 */
final class SearchIndex implements Search {
    private final Map<String, Set<Member>> members = new ConcurrentHashMap<>();
    private final Map<String, Set<Company>> companies = new ConcurrentHashMap<>();
    private final Map<String, Set<JobPosting>> jobs = new ConcurrentHashMap<>();

    void addMember(Member m) { words(m.name()).forEach(w -> members.computeIfAbsent(w, k ->
ConcurrentHashMap.newKeySet()).add(m)); }
    void removeMember(Member m, String oldName) { words(oldName).forEach(w -> members.getOrDefault(w,
Set.of()).remove(m)); }
    void addCompany(Company c) { words(c.name()).forEach(w -> companies.computeIfAbsent(w, k ->
ConcurrentHashMap.newKeySet()).add(c)); }
    void addJob(JobPosting j) { words(j.title()).forEach(w -> jobs.computeIfAbsent(w, k ->
ConcurrentHashMap.newKeySet()).add(j)); }

    @Override public List<Member> searchMembers(String q) {
        return match(members, q).sorted(Comparator.comparing(Member::name)).toList();
    }
    @Override public List<Company> searchCompanies(String q) {
```

```
        return match(companies, q).sorted(Comparator.comparing(Company::name)).toList();
    }
    @Override public List<JobPosting> searchJobs(String q) {
        return match(jobs, q).filter(JobPosting::isOpen).sorted(Comparator.comparingLong(JobPosting::id)).toList();
    }

    /** Items matching ALL words of the query. */
    private static <T> Stream<T> match(Map<String, Set<T>> index, String query) {
        List<String> ws = words(query);
        if (ws.isEmpty()) return Stream.empty();
        return index.getOrDefault(ws.get(0), Set.of()).stream()
                .filter(item -> ws.stream().allMatch(w -> index.getOrDefault(w, Set.of()).contains(item)));
    }

    private static List<String> words(String s) {
        return Arrays.stream(s.toLowerCase(Locale.ROOT).split("\\W+")).filter(w -> !w.isEmpty()).toList();
    }
}

/** Strategy for ordering the feed. */
interface FeedRanker {
    String name();
    Comparator<Post> order(Member viewer);
}

final class ChronologicalRanker implements FeedRanker {
    @Override public String name() { return "latest"; }
    @Override public Comparator<Post> order(Member viewer) {
        return Comparator.comparingLong(Post::createdAt).reversed();
    }
}

/** Engagement score with a simple age decay; 1st-degree connections get a boost over follows. */
final class EngagementRanker implements FeedRanker {
    @Override public String name() { return "top"; }
    @Override public Comparator<Post> order(Member viewer) {
        return Comparator.comparingDouble((Post p) -> -score(p,
viewer)).thenComparing(Comparator.comparingLong(Post::createdAt).reversed());
    }
    private static double score(Post p, Member viewer) {
        double engagement = p.likeCount() + 2.0 * p.comments().size() + 3.0 * p.shares();
        double affinity = viewer.isConnectedTo(p.author()) ? 1.5 : 1.0;
        double age = Tick.now() - p.createdAt();
        return (1 + engagement) * affinity / (1 + age / 20.0);
    }
}

/**
 * Pull model ("fan-out on read"): build the feed when it is opened from the
 * posts of connections and followed members. Fine for small networks; see the
 * README for the push model and the hybrid real sites use.
 */
final class FeedService {
    private final java.util.function.LongFunction<Member> lookup;
    FeedService(java.util.function.LongFunction<Member> lookup) { this.lookup = lookup; }

    List<Post> feed(Member viewer, FeedRanker ranker, int limit) {
        Set<Long> sources = new java.util.HashSet<>(viewer.connections());
        sources.addAll(viewer.followingMembers());
        return sources.stream()
                .map(lookup::apply)
                .filter(m -> m.status() == AccountStatus.ACTIVE)
                .flatMap(m -> m.posts().stream())
                .sorted(ranker.order(viewer))
                .limit(limit)
                .toList();
    }
}
```

```java
// File: lld/ch18/linkedin/LinkedIn.java
package lld.ch18.linkedin;

import java.util.List;
import java.util.Map;
import java.util.NoSuchElementException;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

/** Facade that wires the services and checks account status before every action. */
final class LinkedIn {
    private final Map<Long, Member> members = new ConcurrentHashMap<>();
    private final AtomicLong messageIds = new AtomicLong();
    final NotificationService notifications;
    final ConnectionService connections;
    final SearchIndex search = new SearchIndex();
    final FeedService feeds;

    LinkedIn(List<NotificationChannel> channels) {
        notifications = new NotificationService(channels);
        connections = new ConnectionService(notifications, this::member);
        feeds = new FeedService(this::member);
    }

    Member member(long id) {
        Member m = members.get(id);
        if (m == null) throw new NoSuchElementException("no member " + id);
        return m;
    }

    <M extends Member> M join(M m) {
        members.put(m.id(), m);
        search.addMember(m);
        return m;
    }

    /** The code that changes a name must tell the index, or search goes stale. */
    void rename(Member m, String newName) {
        String old = m.name();
        search.removeMember(m, old);
        m.rename(newName);
        search.addMember(m);
    }

    List<Member> searchMembers(Member viewer, String query) {
        List<Member> found = search.searchMembers(query);
        found.stream().filter(m -> m != viewer).forEach(m -> m.profile().stats().recordSearchAppearance());
        return found;
    }

    void viewProfile(Member viewer, Member owner) {
        if (viewer != owner) owner.profile().stats().recordView();
    }

    void endorse(Member by, Member owner, String skillName) {
        active(by);
        if (!by.isConnectedTo(owner)) throw new IllegalStateException("only 1st-degree connections can endorse " +
owner);
        Skill s = owner.profile().skill(skillName).orElseThrow(() -> new NoSuchElementException(owner + " has no
skill " + skillName));
        if (!s.endorse(by.id())) throw new IllegalStateException(by + " already endorsed " + skillName);
    }

    Recommendation requestRecommendation(Member requester, Member author) {
        active(requester);
        if (!requester.isConnectedTo(author)) throw new IllegalStateException("you can only ask your connections");
        Recommendation r = new Recommendation(author.id(), author.name());
        requester.profile().addRecommendation(r);
```

```
        notifications.send(author, NotificationType.RECOMMENDATION_REQUEST, requester + " asked you for a
recommendation");
        return r;
    }

    Post post(Member author, String text) {
        active(author);
        Post p = new Post(author, text, null);
        author.addPost(p);
        return p;
    }

    Post share(Member by, Post original, String text) {
        active(by);
        Post p = new Post(by, text, original);
        original.recordShare();
        by.addPost(p);
        return p;
    }

    Comment comment(Member by, Post post, String text) {
        active(by);
        Comment c = post.addComment(by, text);
        if (post.author() != by) notifications.send(post.author(), NotificationType.NEW_COMMENT, by + " commented:
\"" + text + "\"");
        return c;
    }

    Message message(Member from, List<Member> to, String text) {
        active(from);
        for (Member r : to) {
            if (!from.isConnectedTo(r)) throw new IllegalStateException(r + " is not a connection (that would need
InMail)");
        }
        Message m = new Message(messageIds.incrementAndGet(), from, List.copyOf(to), text, Tick.next());
        for (Member r : to) {
            r.receive(m);
            notifications.send(r, NotificationType.NEW_MESSAGE, "from " + from + ": " + text);
        }
        return m;
    }

    Company createCompany(Member by, String name) {
        active(by);
        Company c = new Company(name, by);
        search.addCompany(c);
        return c;
    }

    JobPosting postJob(Member by, Company c, String title, String location, EmploymentType type) {
        JobPosting j = c.postJob(by, title, location, type);
        search.addJob(j);
        return j;
    }

    void followCompany(Member m, Company c) {
        if (m.followCompany(c)) c.addFollower(m.id());
    }

    void block(Admin admin, Member target) { target.setStatus(AccountStatus.BLOCKED); }

    private static void active(Member m) {
        if (m.status() != AccountStatus.ACTIVE) throw new IllegalStateException(m + " is " + m.status());
    }
}
```

```java
// File: lld/ch18/linkedin/Main.java
package lld.ch18.linkedin;

import java.time.YearMonth;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        LinkedIn li = new LinkedIn(List.of(new InAppChannel(), new EmailChannel()));
        Member asha = li.join(new Member("Asha Sharma", "asha@example.com"));
        Member ravi = li.join(new Member("Ravi Sharma", "ravi@example.com"));
        Member meera = li.join(new Member("Meera Iyer", "meera@example.com"));
        Member karan = li.join(new Member("Karan Mehta", "karan@example.com"));
        Member zoya = li.join(new Member("Zoya Khan", "zoya@example.com"));
        Admin admin = li.join(new Admin("Site Admin", "admin@example.com"));

        section("1. Profile");
        Profile p = asha.profile();
        p.setHeadline("Backend engineer | Java | Distributed systems");
        p.addExperience(new Experience("Software Engineer", "Acme Cloud", "Bengaluru", YearMonth.of(2022, 7), null,
EmploymentType.FULL_TIME));
        p.addEducation(new Education("IIT Madras", "B.Tech CSE", 2018, 2022));
        p.addSkill("Java");
        p.addSkill("System Design");
        attempt(() -> p.addExperience(new Experience("Intern", "X", "Pune", YearMonth.of(2021, 6),
YearMonth.of(2021, 1), EmploymentType.INTERNSHIP)));
        System.out.println(asha + ": " + p.headline() + ", now at " +
p.currentPosition().map(Experience::company).orElse("-"));

        section("2. Connections");
        ConnectionInvitation i1 = li.connections.invite(asha, meera, "We met at the Java meetup");
        li.connections.accept(i1, meera);
        attempt(() -> li.connections.accept(i1, meera));                // accepted once, never again
        li.connections.accept(li.connections.invite(meera, karan, "Ex-colleague"), karan);
        li.connections.accept(li.connections.invite(karan, zoya, "Hi"), zoya);
        ConnectionInvitation pending = li.connections.invite(asha, ravi, "Same surname!");
        attempt(() -> li.connections.invite(ravi, asha, "Hello"));      // pending the other way
        attempt(() -> li.connections.accept(pending, asha));             // only the receiver can accept
        System.out.println("degree asha->meera=" + li.connections.degree(asha, meera).getAsInt()
                + ", asha->karan=" + li.connections.degree(asha, karan).getAsInt()
                + ", asha->zoya=" + li.connections.degree(asha, zoya).getAsInt()
                + ", asha->ravi=" + (li.connections.degree(asha, ravi).isPresent() ? "?" : "out of network"));
        System.out.println("people asha may know: " + li.connections.peopleYouMayKnow(asha, 5));

        section("3. Endorsements and recommendations");
        li.endorse(meera, asha, "Java");
        attempt(() -> li.endorse(meera, asha, "Java"));
        attempt(() -> li.endorse(karan, asha, "Java"));
        Recommendation rec = li.requestRecommendation(asha, meera);
        attempt(() -> rec.write(karan.id(), "Great engineer"));
        rec.write(meera.id(), "Asha owns problems end to end.");
        System.out.println("Java endorsements=" + p.skill("Java").orElseThrow().endorsements()
                + ", recommendation from " + rec.authorName() + ": " + rec.text().orElse("") + " [" + rec.status()
+ "]");

        section("4. Posts, comments, likes, feed");
        karan.follow(asha);                                              // follow = one-sided, no approval
        Post launch = li.post(meera, "We are hiring Java engineers at Acme Cloud!");
        Post tip = li.post(asha, "Tip: prefer composition over inheritance.");
        Comment c = li.comment(asha, launch, "Sharing with my network");
        launch.like(asha.id());
        c.like(meera.id());                                              // comments can be liked too
        System.out.println("double like accepted? " + launch.like(asha.id()));
        li.share(karan, tip, "Good advice");
        tip.like(meera.id());
        tip.like(zoya.id());
        System.out.println("feed(asha, latest): " + li.feeds.feed(asha, new ChronologicalRanker(), 5));
        System.out.println("feed(karan, top):   " + li.feeds.feed(karan, new EngagementRanker(), 5));
```

```
        section("5. Messaging");
        li.message(asha, List.of(meera), "Coffee on Friday?");
        attempt(() -> li.message(asha, List.of(karan), "Hi Karan"));

        section("6. Companies, jobs, groups");
        Company acme = li.createCompany(meera, "Acme Cloud");
        li.followCompany(asha, acme);
        JobPosting job = li.postJob(meera, acme, "Senior Java Engineer", "Bengaluru", EmploymentType.FULL_TIME);
        attempt(() -> li.postJob(asha, acme, "Fake job", "Nowhere", EmploymentType.CONTRACT));
        System.out.println("search jobs 'java engineer': " + li.search.searchJobs("java engineer"));
        job.apply(asha);
        attempt(() -> job.apply(asha));
        Group group = new Group("Java Developers India", meera);
        group.join(asha);
        attempt(() -> group.leave(meera));
        System.out.println(acme + " followers=" + acme.followerCount() + ", applicants=" + job.applicantCount()
                + ", group members=" + group.memberCount());

        section("7. Search and profile stats");
        System.out.println("search 'sharma' (by meera): " + li.searchMembers(meera, "sharma"));
        li.rename(zoya, "Zoya Ali");
        System.out.println("search 'khan' after rename: " + li.searchMembers(meera, "khan")
                + ", search 'ali': " + li.searchMembers(meera, "ali"));
        li.viewProfile(meera, asha);
        li.viewProfile(karan, asha);
        System.out.println("asha stats: views=" + p.stats().profileViews() + ", connections=" +
asha.connections().size()
                + ", search appearances=" + p.stats().searchAppearances());
        System.out.println("meera's in-app notifications: " +
meera.notifications().stream().map(Notification::type).toList());

        section("8. Admin");
        li.block(admin, karan);
        attempt(() -> li.post(karan, "spam spam"));
        System.out.println("feed(meera) hides blocked Karan's share: " + li.feeds.feed(meera, new
ChronologicalRanker(), 5));
    }

    static void section(String t) { System.out.println("\n== " + t + " =="); }

    static void attempt(Runnable r) {
        try { r.run(); } catch (RuntimeException e) { System.out.println("  REJECTED: " + e.getMessage()); }
    }
}
```

Output:

```
== 1. Profile ==
  REJECTED: experience ends before it starts
Asha Sharma: Backend engineer | Java | Distributed systems, now at Acme Cloud

== 2. Connections ==
    [email to meera@example.com] CONNECTION_INVITATION: Asha Sharma wants to connect: "We met at the Java meetup"
  REJECTED: invitation is already ACCEPTED
    [email to karan@example.com] CONNECTION_INVITATION: Meera Iyer wants to connect: "Ex-colleague"
    [email to zoya@example.com] CONNECTION_INVITATION: Karan Mehta wants to connect: "Hi"
    [email to ravi@example.com] CONNECTION_INVITATION: Asha Sharma wants to connect: "Same surname!"
  REJECTED: an invitation between Ravi Sharma and Asha Sharma is already pending
  REJECTED: only Ravi Sharma can accept
degree asha->meera=1, asha->karan=2, asha->zoya=3, asha->ravi=out of network
people asha may know: [Karan Mehta]

== 3. Endorsements and recommendations ==
  REJECTED: Meera Iyer already endorsed Java
  REJECTED: only 1st-degree connections can endorse Asha Sharma
    [email to meera@example.com] RECOMMENDATION_REQUEST: Asha Sharma asked you for a recommendation
  REJECTED: only Meera Iyer can write this recommendation
Java endorsements=1, recommendation from Meera Iyer: Asha owns problems end to end. [WRITTEN]

== 4. Posts, comments, likes, feed ==
    [email to meera@example.com] NEW_COMMENT: Asha Sharma commented: "Sharing with my network"
double like accepted? false
feed(asha, latest): ["We are hiring Java engineers at Acme Cloud!" by Meera Iyer]
feed(karan, top):   ["Tip: prefer composition over inheritance." by Asha Sharma, "We are hiring Java engineers at
Acme Cloud!" by Meera Iyer]

== 5. Messaging ==
    [email to meera@example.com] NEW_MESSAGE: from Asha Sharma: Coffee on Friday?
  REJECTED: Karan Mehta is not a connection (that would need InMail)

== 6. Companies, jobs, groups ==
  REJECTED: Asha Sharma is not an admin of Acme Cloud
search jobs 'java engineer': [Senior Java Engineer @ Acme Cloud (Bengaluru, FULL_TIME)]
  REJECTED: Asha Sharma already applied to Senior Java Engineer
  REJECTED: the owner cannot leave their own group
Acme Cloud followers=1, applicants=1, group members=2

== 7. Search and profile stats ==
search 'sharma' (by meera): [Asha Sharma, Ravi Sharma]
search 'khan' after rename: [], search 'ali': [Zoya Ali]
asha stats: views=2, connections=1, search appearances=1
meera's in-app notifications: [CONNECTION_INVITATION, RECOMMENDATION_REQUEST, NEW_COMMENT, NEW_MESSAGE]

== 8. Admin ==
  REJECTED: Karan Mehta is BLOCKED
feed(meera) hides blocked Karan's share: ["Tip: prefer composition over inheritance." by Asha Sharma]
```

Walking through the important parts

- `ConnectionService.invite` refuses a request when one is already pending *in either direction* — in the output Ravi's counter-invite to Asha is rejected.
- `accept` checks that the caller is the receiver, moves the invitation to ACCEPTED (a second accept fails because only PENDING can move), and adds each member to the other's set inside one `synchronized` block.
- `degree` is a BFS over connection sets that stops at depth 3: Asha → Meera is 1st, Karan 2nd, Zoya 3rd, and Ravi (only a pending invitation) is "out of network".
- `peopleYouMayKnow` counts mutual connections for every 2nd-degree member, skips anyone with a pending invitation, and sorts by count then name, so the list is stable.
- `endorse` requires a 1st-degree connection and uses the skill's endorser set to reject a second endorsement.
- `Recommendation.write` accepts only the named author and only once.
- `SearchIndex` maps each lowercase word to a *set*, so "sharma" returns both Asha and Ravi; `rename` removes the old words before adding the new ones, so "khan" finds nobody after Zoya's rename.

- `FeedService` is a **pull** feed: it gathers posts from connections and followed members when asked, drops blocked authors, then sorts with the chosen `FeedRanker`. Karan sees Asha's tip because he *follows* her, even though they are not connected.

Concurrency & edge cases

- **Symmetric connections:** all graph changes (`invite`, `accept`, `reject`, `withdraw`, `removeConnection`) are `synchronized` on the single service, so two concurrent accepts or an accept racing a withdraw cannot leave a one-way edge. In a distributed system (one spread across many machines that talk over a network), write both edges (an **edge** is one link in a graph) in one database transaction (or one row keyed by the ordered pair of ids).
- **Double invitations** in either direction are refused while one is pending.
- **Likes, endorsements, applications:** sets and maps keyed by member id make repeats harmless or rejected; their mutators are `synchronized`.
- **Collections on `Member`** are concurrent sets or `CopyOnWriteArrayList`, and getters return copies, so callers cannot mutate internal state.
- **Stale search:** handled by `rename`; the general rule is "whatever changes the data tells the index" (in production, via an event).
- **Blocked members:** writes are refused and their posts vanish from feeds.
- **Validation:** an experience that ends before it starts is rejected; blank posts are rejected.

Extensions & follow-up questions

- **"Build the feed for 900 million members."** — Two classic strategies. **Fan-out on write (push):** when someone posts, append the post id to each connection's precomputed feed list — fast reads, expensive for members with huge followings. **Fan-out on read (pull):** assemble the feed when it is opened, as our `FeedService` does — cheap writes, slower reads. Large networks use a **hybrid**: push for ordinary members, pull for very popular accounts, merged at read time.
- **"People you may know at scale."** — Compute suggestions offline (in a background batch job, not while the user waits) on a schedule (friends-of-friends plus signals such as shared companies and schools) and store them; the live page reads the stored list.
- **"Connection limit."** — LinkedIn caps members at 30,000 connections; enforce in `accept`.
- **"InMail to non-connections."** — A premium credit balance on `Member`; `message` spends a credit instead of rejecting.
- **"Who viewed my profile."** — Store view events (viewer, time) instead of a bare counter, then derive the count.
- **"Job recommendations."** — A Strategy that matches a member's skills and current title against open postings, ranked by overlap.

Real-world example

LinkedIn shows 1st, 2nd and 3rd-degree badges next to names in search results — the exact BFS depth our `degree` computes — and its "People You May Know" panel is driven by shared connections and other signals. Messaging someone who is not a connection usually requires paid **InMail** (some members open their profile to free messages). And the event stream that feeds notifications, analytics and the feed at LinkedIn gave rise to **Apache Kafka**, which LinkedIn open-sourced (published free, with its source code, for anyone to use) in 2011.

In the interview

Minutes	What to do
0–5	Clarify connect vs follow, degrees, who can message and endorse
5–15	Profile composition, `SocialContent` (comments can be liked), invitation as an object with state
15–25	**Say it out loud:** accepting changes two objects, so it belongs to neither — `ConnectionService`; following is one-sided, so it needs no service
25–35	Search index (key → list), degrees by BFS, people you may know
35–45	Feed: pull vs push vs hybrid; notifications via Strategy channels; concurrency

> **30-second answer:** "Members own profiles made of small parts. Posts and comments share a base class because both are liked, with likes as a set. A connection request is an object with states. Accepting changes both

members, so it lives in a `ConnectionService` that updates both sides under one lock; following is one-sided, just a set. Search is an inverted index mapping each word to a list. Degrees come from a BFS to depth 3. The feed is a pull model with a pluggable ranking Strategy, and I'd move to a push/pull hybrid at scale."

Follow-ups with answers

- *Why not store `connectionCount` on the profile?* — It is `connections.size()`; a stored copy can drift.
- *How do you compute a 2nd-degree badge quickly for every search result?* — Precompute or cache each member's 1st-degree set; a result is 2nd-degree if its connection set intersects yours.
- *What if the same name appears twice?* — The index maps words to lists (sets), never to a single member.

Self-check

- **Q:** Why is accepting an invitation not a method on `Member`? **A:** It updates two members; a service keeps both sides consistent in one place.
- **Q:** What is the difference between connecting and following? **A:** Connecting is mutual and needs acceptance; following is one-sided and immediate.
- **Q:** Why does `Recommendation` start without text? **A:** Because it is *requested* first; the author fills it in later.
- **Q:** What is the rule for search index values? **A:** Always a collection, even for one match.
- **Q:** Pull vs push feed — which does `FeedService` implement, and what is its cost? **A:** Pull; cheap writes, but reading scans every connection's posts each time.

TL;DR

LinkedIn = profile composition + `SocialContent` (likes as sets) + invitation **State** + `ConnectionService` for the two-sided accept (following is just a set) + BFS degrees + people-you-may-know + inverted-index search (key → list) + notification and feed-ranking **Strategies** + a facade.

18.4 Design Facebook

Figure — A browser showing the Facebook home page, the cover image of this case study (course diagram)

Facebook is an online social network where users connect with **friends** (a mutual relationship), **follow** people and **pages**, join **groups**, and read and write posts through a website or mobile apps. Compared with LinkedIn, the new challenges are **privacy** (who may see each post), a real **news feed**, **pages** with ratings, **groups** (public and private), and **friend suggestions**.

Problem & clarifying questions

Question to ask	Answer we assume
Friend vs follow?	Friendship is mutual (request + accept); following is one-sided
Who can see a post?	The author chooses an **audience**: Public, Friends, a custom **privacy list**, or Only me
What is a privacy list?	A named subset of the author's friends (e.g. "Close friends"); a post linked to it is visible only to those friends
Pages?	Created by members; others follow them and add **recommendations** (a 1–5 rating and text)
Groups?	Public (join at once) or private (owner approves); posts inside a private group are visible only to members
Search?	Members, groups and pages by name; posts by a word — respecting privacy
News feed?	Posts from friends, followed members, followed pages and joined groups, filtered by privacy, ranked
Extended requirement?	Suggest people you may know (friends of friends)

Requirements

Functional (the course's twelve plus the extended one, in our words)

1. A member fills in a profile: basic details, jobs, schooling and places lived.
2. Any user can search members, groups or pages by name.
3. Members send friend requests and accept or reject them.
4. Members can follow other members without becoming friends.
5. Members create groups and pages, join groups, and follow pages.
6. Members create posts to share with their friends.
7. Members can comment on posts; posts **and comments** can both be liked or shared.
8. Members create privacy lists of friends; a post linked to a list is visible only to that list.
9. Members send messages to other members.
10. Members add a recommendation to any page.
11. A member is notified about each new message, each friend request, and each comment on one of their posts.
12. Members can search posts for a word.
13. **Extended:** write a function that suggests connections for a member.

Non-functional

- **Privacy is never leaked** — through the feed, search, comments, likes or shares.
- **Feed latency** — the feed must load fast even for members with hundreds of friends.
- **Consistency of the friend graph** — friendship is symmetric.
- **Stable output** — suggestions and feeds must not reshuffle randomly between page loads.

Use cases

Three actors: **Member**, **Admin** (blocks/unblocks members) and **System** (sends notifications).

Top use cases: add/update profile; search members, groups, pages and posts; send friend requests; follow/unfollow a member or page; send messages to friends; create posts, like and comment on visible posts; send notifications; create a page; recommend a page.

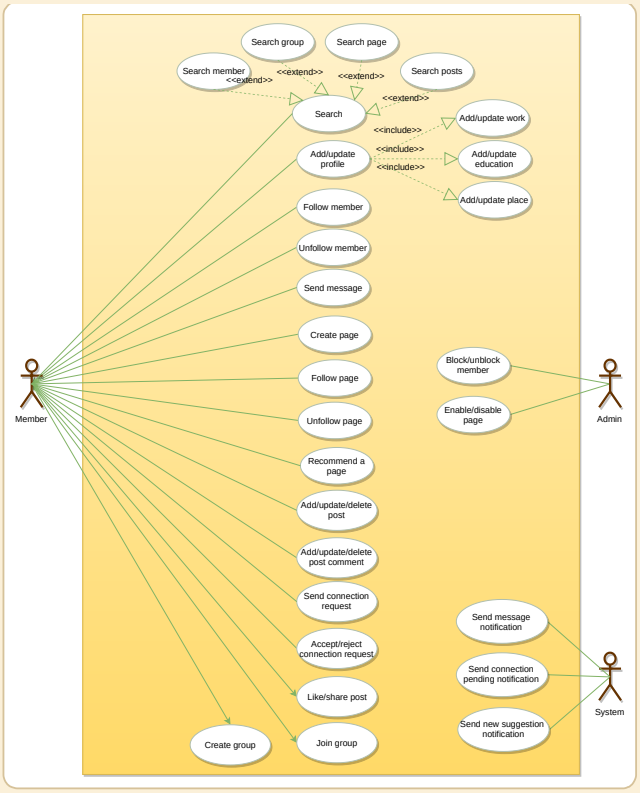


Figure — Facebook use cases for Member, Admin and System, including "Create page", "Recommend a page" and "Unfollow member" (course diagram)

Activity — add work experience. The member opens *Add Work*, enters position, company, location, dates and description, and presses save; the system verifies, asks for missing information if needed, then saves.

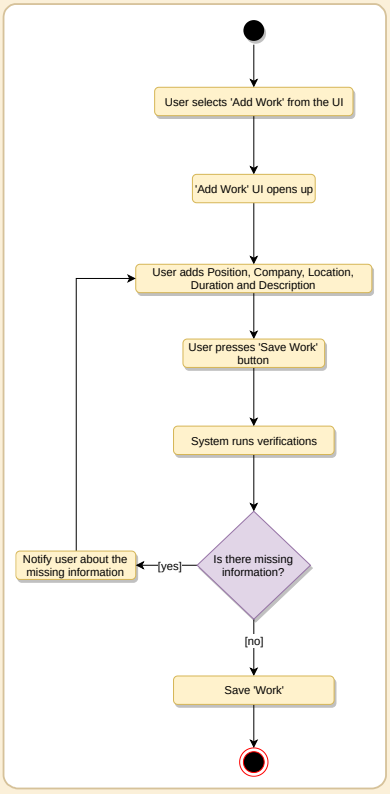


Figure — Activity diagram "add work to profile" with a verification loop (course diagram)

Activity — create a post. The member opens *New Post*, writes the text, optionally uploads a photo or video, **chooses the privacy list** that may see it, and presses publish. After verification the post is published to the chosen audience, who receive an email or push notification.

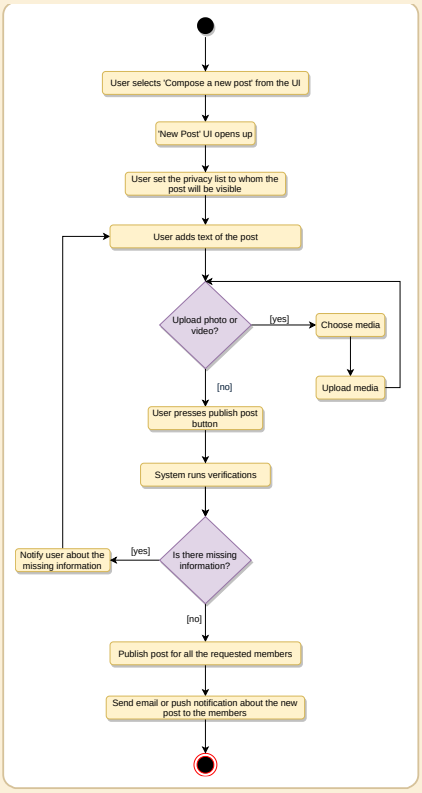


Figure — Activity diagram "create a new post": add text, optional media, set the privacy list, verify, publish, notify (course diagram)

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, follows (members and pages), groups, posts, privacy lists, inbox, notifications (the course's messages may carry photos and videos; ours are text only)
`Profile` (+ `Work`, `Education`, `Place`)	Defaults for every field, lists of parts
`PrivacyList`	Owner + named set of friend ids
`Audience` (sealed) → `Public`, `Friends`, `Custom(list)`, `OnlyMe`	Who may see a post
`SocialContent` → `Post`, `Comment`	Likes as a set; posts own comments; a post may belong to a group or page or be a share
`FriendRequest` + `FriendService`	Requests with both ends, the friend graph, `areFriends`, `pendingBetween` (both ways)
`VisibilityService`	The single `canSee(viewer, post)` rule, combining the post's privacy list and the friend graph
`Page` (+ `PageType`, `PageRecommendation`)	Followers (counted), one recommendation per member, average rating, page posts
`Group` (+ `GroupPrivacy`)	Members, join requests, owner approval, group posts
`NotificationService` + `PushChannel` / `EmailChannel`	Routes each notification type to channels
`SearchIndex`	Names of members/groups/pages; post words — `searchPosts` takes the viewer
`NewsFeedService` + `FeedRanker`	Gathers candidates, filters by `canSee`, ranks
`SuggestionService` + `Suggestion`	Friends-of-friends by mutual count
`Facebook`	Facade

Class diagram

The course builds Facebook in **five stages plus the extended requirement**.

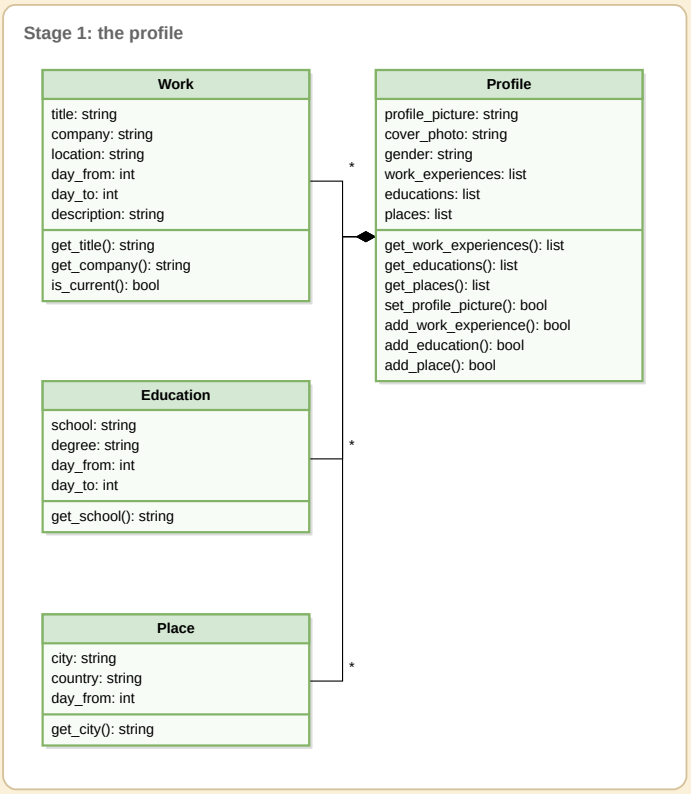


Figure — Stage 1: `Profile` holding lists of `Work`, `Education` and `Place` (course diagram)

Stage 1 — the profile. Work, education and places are small classes held in lists (composition). Every profile field starts with a **default** value, so a brand-new account already has a working profile, and no other code needs to make up a placeholder picture for someone who never uploaded one.

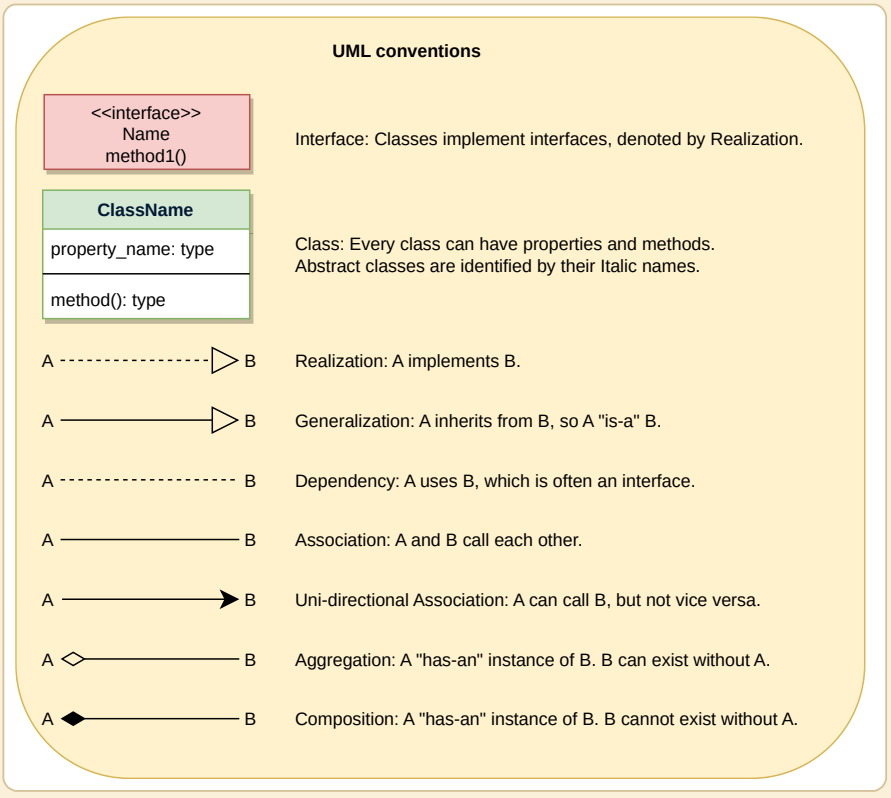


Figure — UML conventions used in these diagrams (course diagram)

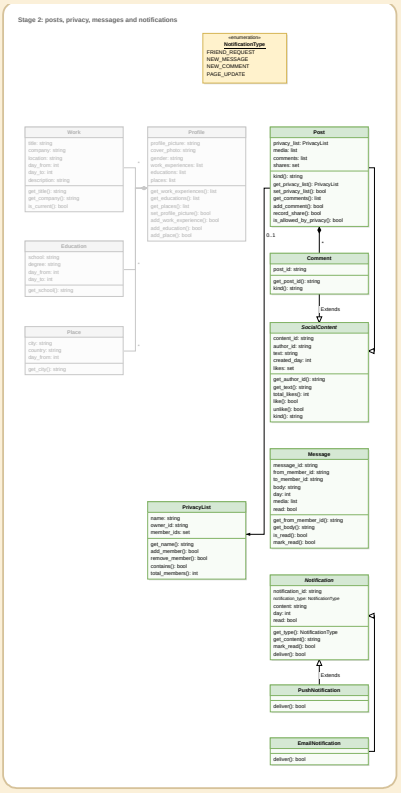


Figure — Stage 2: `SocialContent` with `Post` and `Comment`, a `0..1` association from `Post` to `PrivacyList`, `Message`, and abstract `Notification` (course diagram)

Stage 2 — posts, privacy, messages and notifications. Three choices here repair bugs in older versions. (1) **Store the likers, not a like count** — an old `Post` received its like count through the constructor, so a fresh post could start with a thousand likes, and there was no way to know whether you had liked it already. (2) **Comments get likes too**, so the likes set lives in the shared `SocialContent` parent. (3) **Privacy is checked by the post itself** — `isAllowedByPrivacy` is a method on the post, not something each screen must remember. Yet it can answer only part of the question, because a post has no idea who is friends with whom; stage 5 supplies the rest. The link from `Post` to `PrivacyList` is marked `0..1`: a post might have no list (it is public), and one

list is reused by many posts and survives them. `Notification` is abstract, with a subclass for each delivery method.

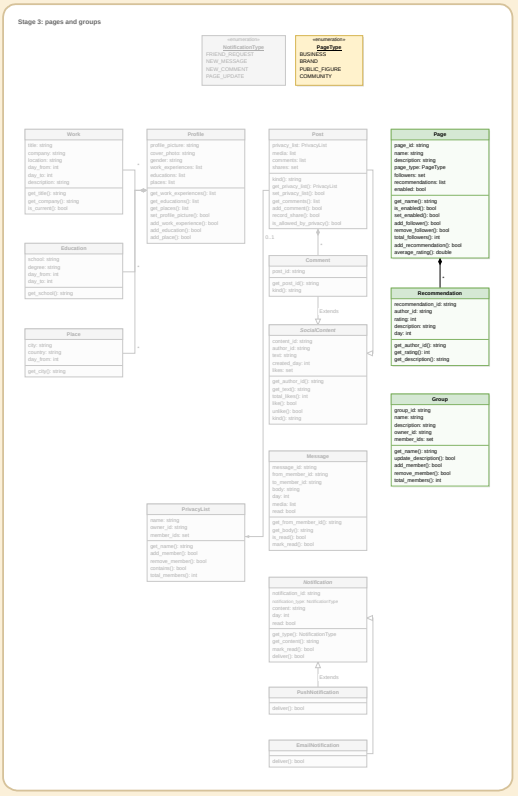


Figure — Stage 3: `Page` owning its `Recommendation`s, followers as ids, and `Group`; no member-count fields (course diagram)

`Stage 3 — pages and groups.` Look at what these classes do *not* have: a member-count field. An earlier version accepted the count in its constructor *and* also stored the members, so the two numbers could be different from day one. The rule: **if a number can be computed by counting a collection, do not keep it as a separate field; keep the collection and count it when asked.**

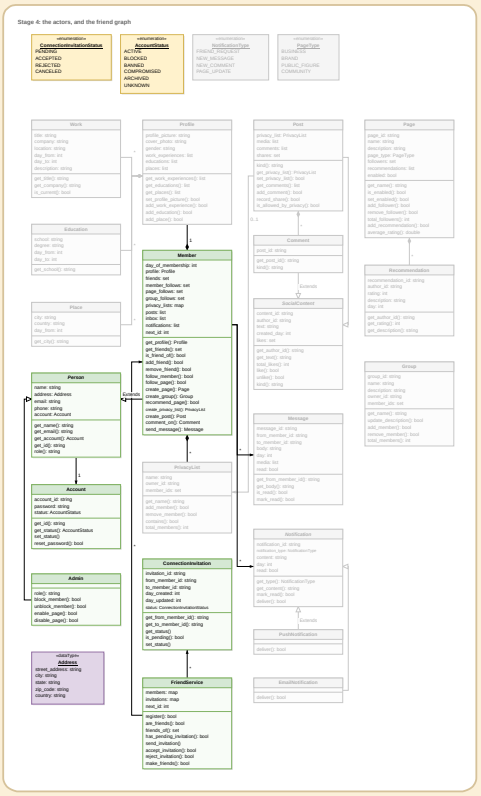


Figure — Stage 4: `Account`, `Person`, `Member`, `Admin`, `ConnectionInvitation` and `FriendService`, which owns the friend graph (course diagram)

Stage 4 — the actors and the friend graph. An earlier invitation class remembered only the person invited, not the person inviting, so there was no way to accept it properly. Since acceptance updates two members, it belongs in `FriendService`; that service therefore holds the **friend graph** (who is friends with whom) and is the obvious class to ask "are these two friends?" and "is a request already waiting between them?". Following (requirement 4) is deliberately different: one-sided, a set on the follower, no service.

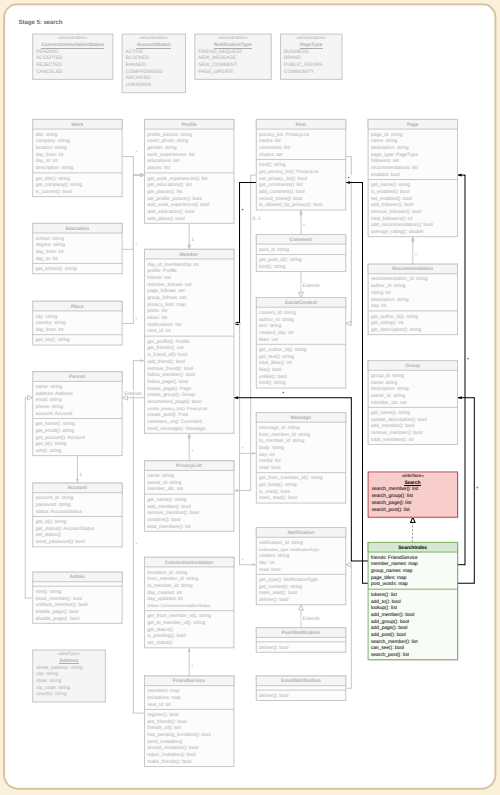


Figure — Stage 5: `Search` and `SearchIndex`, which holds a `FriendService` so post search can respect privacy (course diagram)

Stage 5 — search. The most important detail is small: **`searchPost` needs the viewer as a parameter.** The privacy rule (requirement 8) still applies when someone searches; a keyword search that skips it would let anyone read posts that were never shared with them. Deciding visibility therefore needs two sources — the post's own privacy

list, and the friendship data that only the friend service holds — and neither source can answer for the other. The index follows the same rule as LinkedIn's: each key maps to a collection.

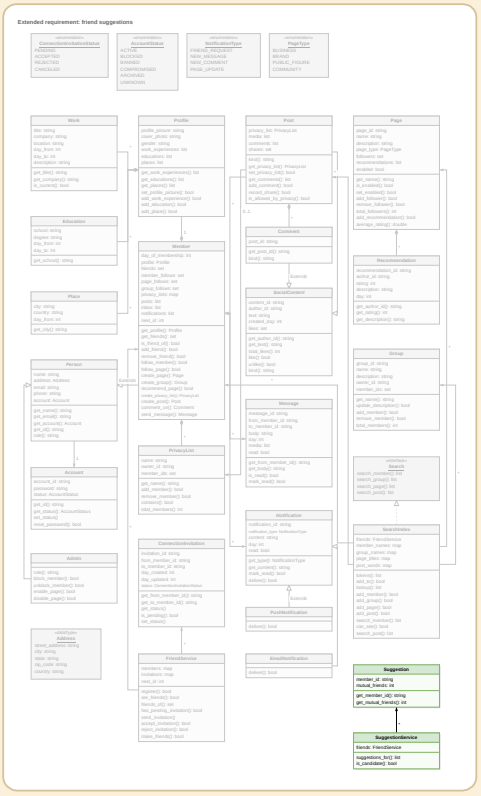


Figure — Extended requirement: `SuggestionService` reading the friend graph and producing `Suggestion` objects, with no change to existing classes (course diagram)

Extended requirement — friend suggestions. Friends of friends, **scored by mutual friends** (someone reached through three of your friends beats someone reached through one). Never suggest yourself, an existing friend, or anyone with a request pending **in either direction** (the one candidates forget — showing someone you invited yesterday is a visible bug). Sort by mutual count, then by id, so equal scores do not reshuffle between page loads. Notice that the feature arrives as two new classes with no edits to existing ones; if a design must be modified every time a new question is asked, it will keep being modified.

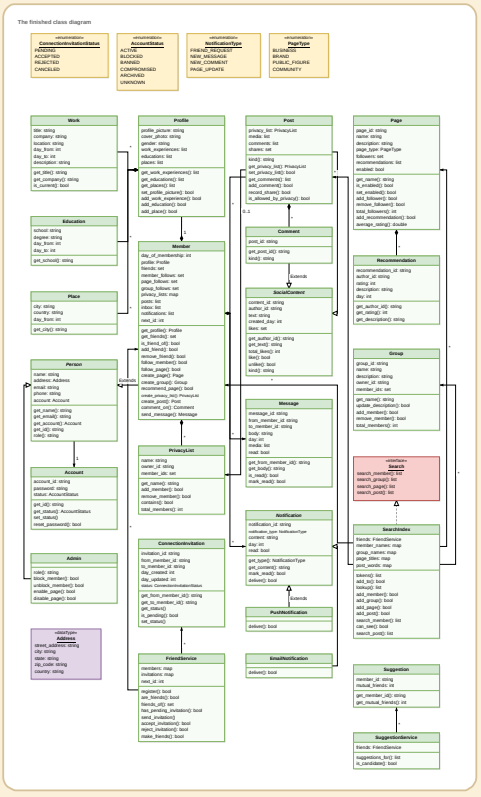
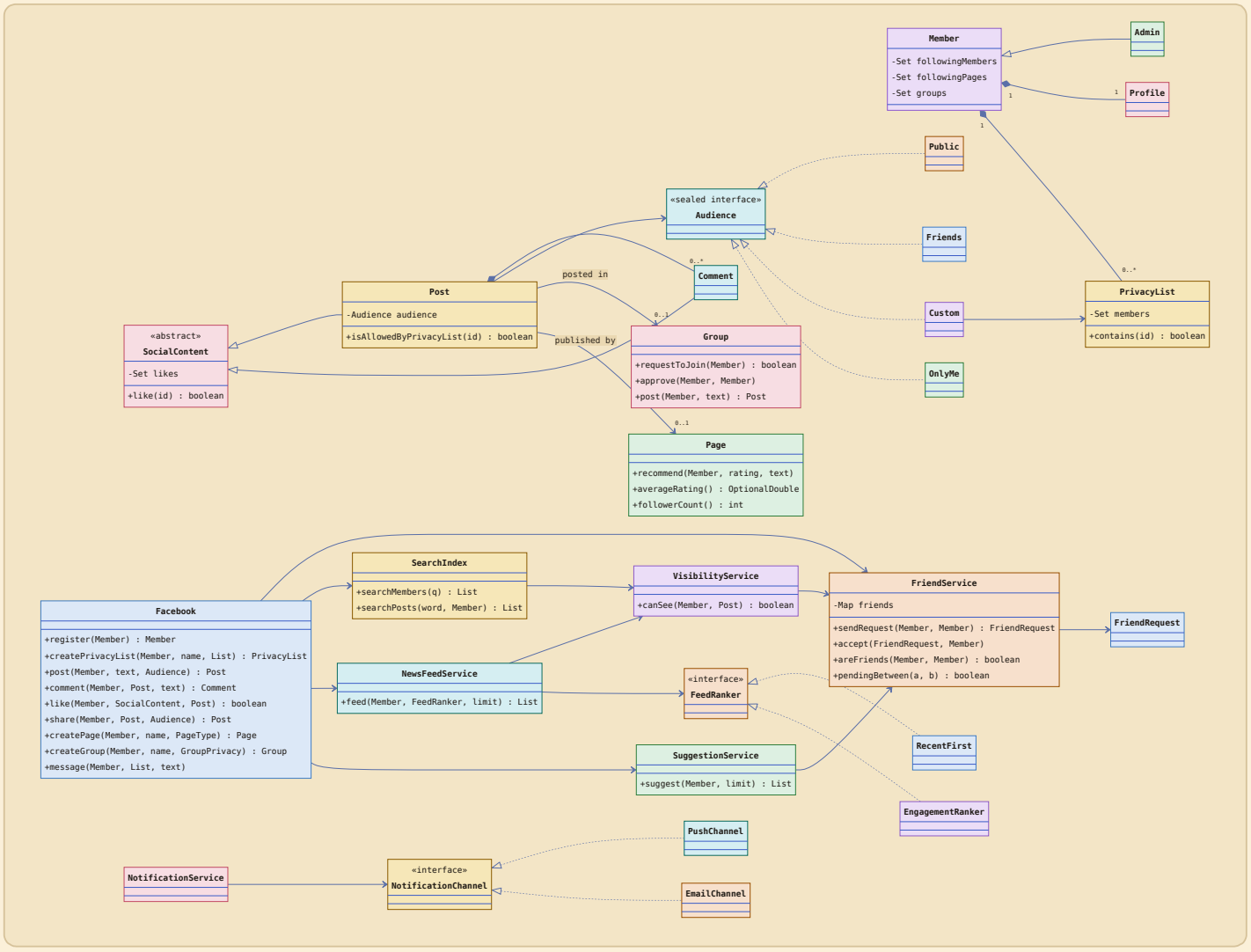


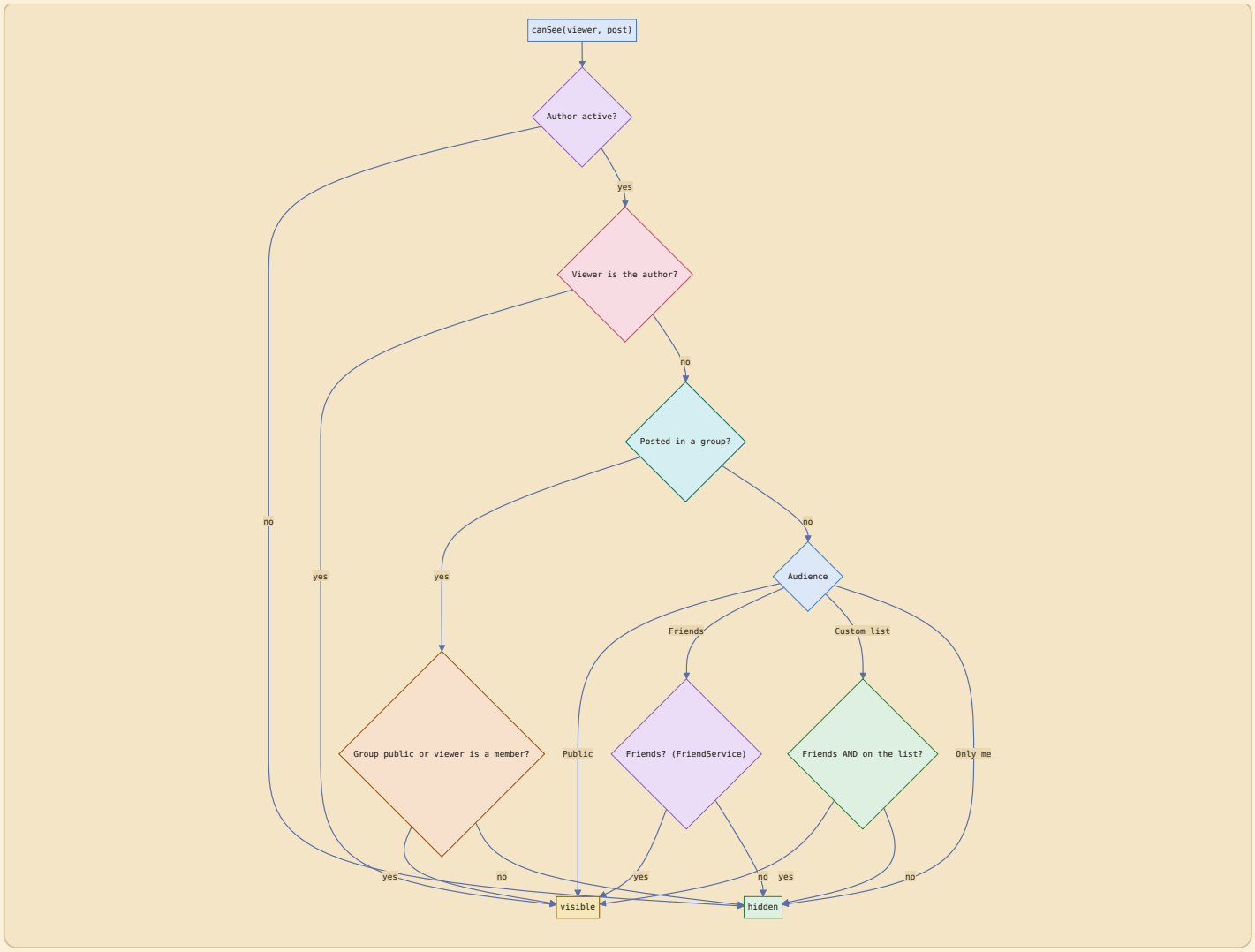
Figure — The finished Facebook class diagram with every class in full colour (course diagram)

Our Java design keeps the course's structure and makes three things explicit: a sealed `Audience` type (Public / Friends / Custom list / Only me), a single `VisibilityService` used by the feed, search, comments, likes and shares, and a `NewsFeedService` (the course leaves the feed out) with ranking Strategies. Private groups with owner approval and page posts in the feed are added too. The course lists what it leaves out on purpose: the news feed and its ranking, one member blocking another, reactions other than a plain like, tagging people, events, marketplace, stories, and photo albums that carry separate privacy settings. We add the feed; the rest are sketched under Extensions.

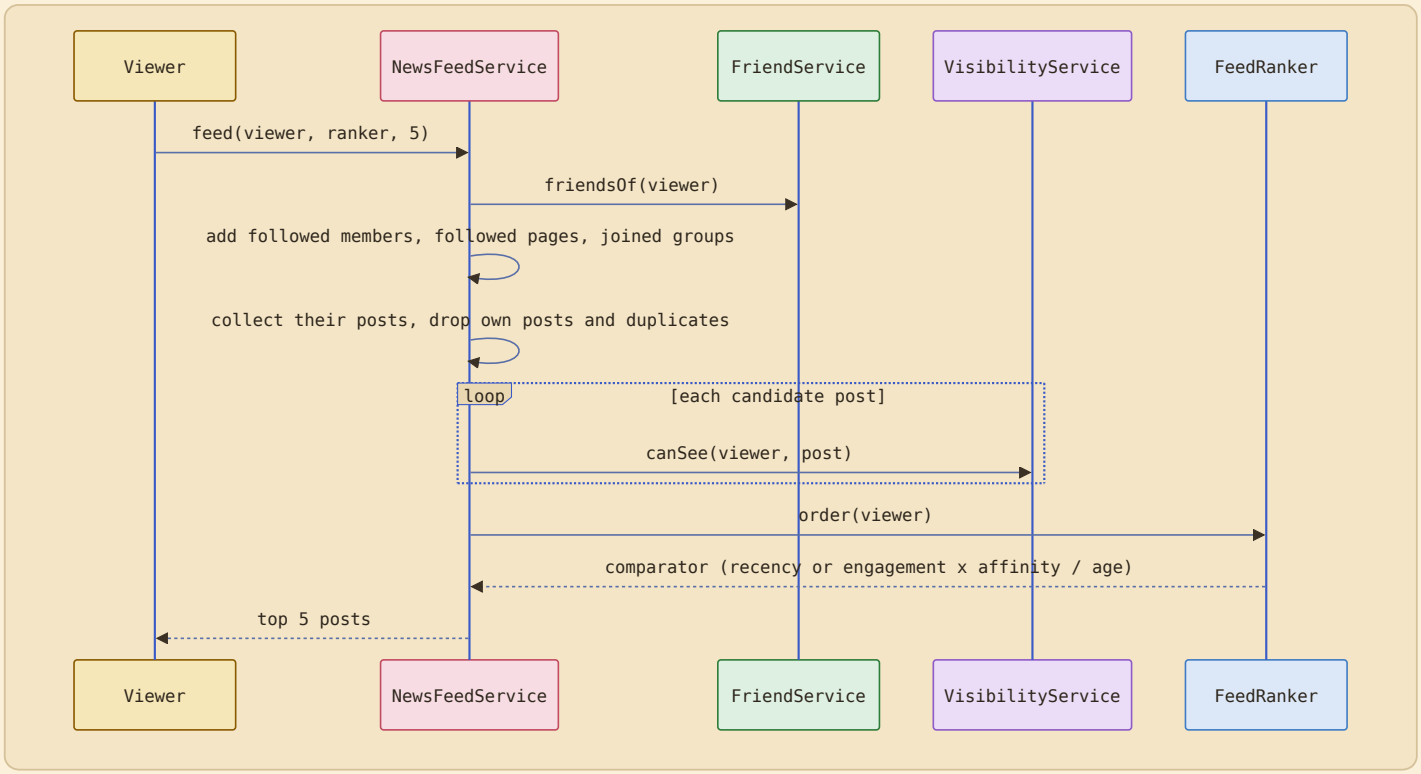


Key flows

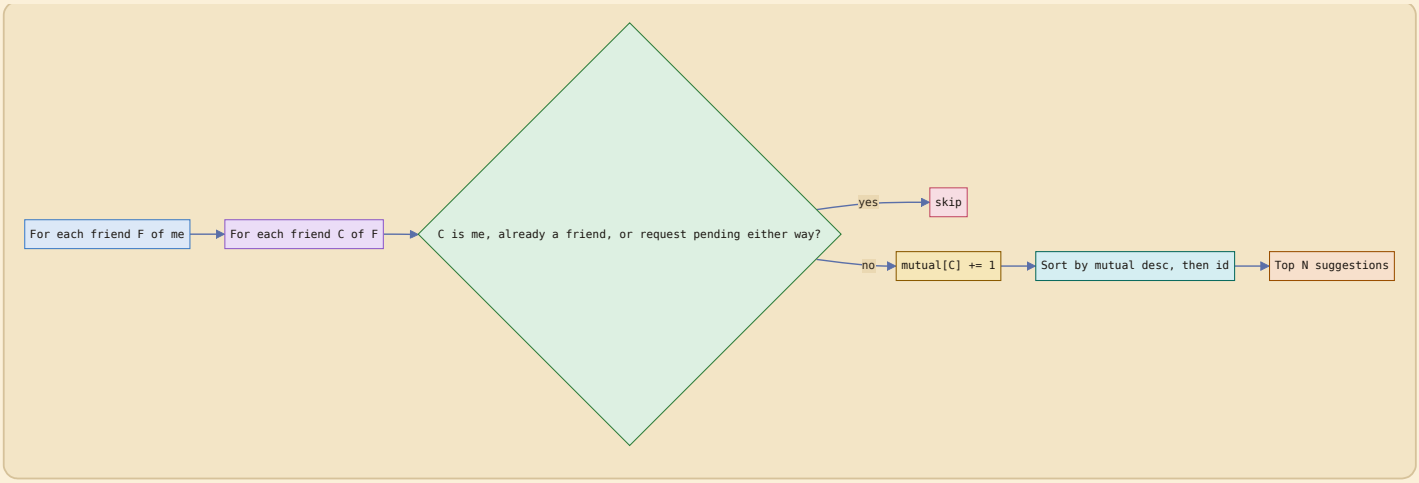
Can this viewer see this post? (used by feed, search, comment, like and share):



Building a news feed (pull model):



Friend suggestions:



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`VisibilityService.canSee` used everywhere	Single point of policy (a **Specification**-like rule object — one object that answers a yes/no business question)	The privacy rule is written once; search and feed cannot leak	Checking privacy in each screen
`Audience` sealed interface of records	Algebraic data type + exhaustive `switch`	Adding an audience forces every decision point to handle it	Integer or string codes
`Post.isAllowedByPrivacyList` + `FriendService.areFriends`	Each object answers what it knows (Information Expert)	Neither half pretends to know the other's data	Post holding a reference to the whole friend graph
`FriendService` owns the graph	Service for two-sided operations	Symmetric friendships; one place for "pending either way"	`member.acceptFriend()`
Likes as a set	"Store who, not how many"	No fake like counts, no double likes	`likeCount` field
No stored member/follower counts	Derive, don't store	Counts cannot disagree with lists	Constructor-supplied counts (the course's bug)
`NotificationService` routes types to channels	**Strategy** + configuration map	Friend requests go by push *and* email, comments by push only	`if/else` on type inside each domain method
`FeedRanker`	**Strategy**	"Most recent" vs ranked feed swap at run time	Ranking code inside the feed
`SuggestionService` added without editing others	Open/Closed Principle	The extended requirement is two new classes	Adding suggestion fields to `Member`
`Facebook` facade	**Facade**	One entry point, account-status checks	UI talking to every service

Java implementation

Package `lld.ch18.facebook`.

```java
// File: lld/ch18/facebook/Model.java
package lld.ch18.facebook;

import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

/** Deterministic logical clock for repeatable demo output. */
final class Tick {
    private static final AtomicLong NOW = new AtomicLong();
    static long next() { return NOW.incrementAndGet(); }
    static long now() { return NOW.get(); }
    private Tick() {}
}

record Work(String title, String company, String location, int fromYear, Integer toYear) {
    boolean isCurrent() { return toYear == null; }
}
record Education(String school, String degree, int fromYear, int toYear) {}
record Place(String city, String country, int fromYear) {}

/** Every field has a default, so a profile is usable the moment the account exists. */
final class Profile {
    private volatile String picture = "default-avatar.png";
    private volatile String bio = "";
    private final List<Work> work = new CopyOnWriteArrayList<>();
    private final List<Education> education = new CopyOnWriteArrayList<>();
    private final List<Place> places = new CopyOnWriteArrayList<>();

    String picture() { return picture; }
    void setPicture(String p) { picture = p; }
    String bio() { return bio; }
    void setBio(String b) { bio = b; }
    void addWork(Work w) { work.add(w); }
    void addEducation(Education e) { education.add(e); }
    void addPlace(Place p) { places.add(p); }
    List<Work> work() { return List.copyOf(work); }
    List<Place> places() { return List.copyOf(places); }
}

enum AccountStatus { ACTIVE, BLOCKED, BANNED, COMPROMISED, ARCHIVED, UNKNOWN }

class Member {
    private static final AtomicLong IDS = new AtomicLong();
    private final long id = IDS.incrementAndGet();
    private final String name;
    private final String email;
    private final Profile profile = new Profile();
    private volatile AccountStatus status = AccountStatus.ACTIVE;
    private final Set<Long> followingMembers = ConcurrentHashMap.newKeySet();   // one-sided
    private final Set<Long> followingPages = ConcurrentHashMap.newKeySet();
    private final Set<Long> groups = ConcurrentHashMap.newKeySet();
    private final List<Post> posts = new CopyOnWriteArrayList<>();
    private final List<PrivacyList> privacyLists = new CopyOnWriteArrayList<>();
    private final List<Message> inbox = new CopyOnWriteArrayList<>();
    private final List<Notification> notifications = new CopyOnWriteArrayList<>();

    Member(String name, String email) { this.name = name; this.email = email; }

    long id() { return id; }
    String name() { return name; }
    String email() { return email; }
    Profile profile() { return profile; }
```

```
    AccountStatus status() { return status; }
    void setStatus(AccountStatus s) { status = s; }

    boolean follow(Member m) { return m.id != id && followingMembers.add(m.id); }
    boolean unfollow(Member m) { return followingMembers.remove(m.id); }
    Set<Long> followingMembers() { return Set.copyOf(followingMembers); }
    boolean followPage(Page p) { return followingPages.add(p.id()); }
    Set<Long> followingPages() { return Set.copyOf(followingPages); }
    void joinedGroup(Group g) { groups.add(g.id()); }
    Set<Long> groups() { return Set.copyOf(groups); }

    void addPost(Post p) { posts.add(p); }
    List<Post> posts() { return List.copyOf(posts); }
    void addPrivacyList(PrivacyList l) { privacyLists.add(l); }
    List<PrivacyList> privacyLists() { return List.copyOf(privacyLists); }
    void receive(Message m) { inbox.add(m); }
    List<Message> inbox() { return List.copyOf(inbox); }
    void notify(Notification n) { notifications.add(n); }
    List<Notification> notifications() { return List.copyOf(notifications); }

    @Override public String toString() { return name; }
}

final class Admin extends Member {
    Admin(String name, String email) { super(name, email); }
}

/** A named subset of the owner's friends, e.g. "Close friends". Outlives any single post. */
final class PrivacyList {
    private final Member owner;
    private final String name;
    private final Set<Long> members = ConcurrentHashMap.newKeySet();

    PrivacyList(Member owner, String name) { this.owner = owner; this.name = name; }
    Member owner() { return owner; }
    String name() { return name; }
    void add(long memberId) { members.add(memberId); }
    boolean contains(long memberId) { return members.contains(memberId); }
}

/** Who may see a post. A sealed type makes the visibility switch exhaustive. */
sealed interface Audience {
    record Public() implements Audience {}
    record Friends() implements Audience {}
    record Custom(PrivacyList list) implements Audience {}
    record OnlyMe() implements Audience {}
}

abstract class SocialContent {
    private static final AtomicLong IDS = new AtomicLong(1000);
    private final long id = IDS.incrementAndGet();
    private final Member author;
    private final String text;
    private final long createdAt = Tick.next();
    private final Set<Long> likes = new LinkedHashSet<>();      // a set: no double likes, can be undone

    SocialContent(Member author, String text) { this.author = author; this.text = text; }

    long id() { return id; }
    Member author() { return author; }
    String text() { return text; }
    long createdAt() { return createdAt; }
    synchronized boolean like(long memberId) { return likes.add(memberId); }
    synchronized boolean unlike(long memberId) { return likes.remove(memberId); }
    synchronized int likeCount() { return likes.size(); }
}

final class Post extends SocialContent {
    private final Audience audience;
```

```
    private final Group group;          // null unless posted in a group
    private final Page page;            // null unless published by a page
    private final Post sharedFrom;
    private final List<Comment> comments = new ArrayList<>();

    Post(Member author, String text, Audience audience, Group group, Page page, Post sharedFrom) {
        super(author, text);
        this.audience = audience;
        this.group = group;
        this.page = page;
        this.sharedFrom = sharedFrom;
    }

    Audience audience() { return audience; }
    Optional<Group> group() { return Optional.ofNullable(group); }
    Optional<Page> page() { return Optional.ofNullable(page); }
    Optional<Post> sharedFrom() { return Optional.ofNullable(sharedFrom); }

    /** The half of the privacy rule the post CAN answer on its own. */
    boolean isAllowedByPrivacyList(long viewerId) {
        return !(audience instanceof Audience.Custom c) || c.list().contains(viewerId);
    }

    synchronized Comment addComment(Member by, String text) {
        Comment c = new Comment(by, text, this);
        comments.add(c);
        return c;
    }
    synchronized List<Comment> comments() { return List.copyOf(comments); }

    @Override public String toString() {
        String where = page != null ? " [page " + page.name() + "]" : group != null ? " [group " + group.name() +
"]" : "";
        return "\"" + text() + "\" by " + author().name() + where;
    }
}

final class Comment extends SocialContent {
    private final Post post;
    Comment(Member author, String text, Post post) { super(author, text); this.post = post; }
    Post post() { return post; }
}

record Message(Member sender, List<Member> recipients, String text, long sentAt) {}
```

```java
// File: lld/ch18/facebook/PagesAndGroups.java
package lld.ch18.facebook;

import java.util.List;
import java.util.Map;
import java.util.OptionalDouble;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

enum PageType { BUSINESS, BRAND, PUBLIC_FIGURE, COMMUNITY }

/** One recommendation per member per page, with a 1-5 rating. */
record PageRecommendation(long memberId, int rating, String text) {
    PageRecommendation {
        if (rating < 1 || rating > 5) throw new IllegalArgumentException("rating must be 1..5");
    }
}

final class Page {
    private static final AtomicLong IDS = new AtomicLong(500);
    private final long id = IDS.incrementAndGet();
    private final String name;
    private final PageType type;
    private final Member owner;
    private final Set<Long> followers = ConcurrentHashMap.newKeySet();
    private final Map<Long, PageRecommendation> recommendations = new ConcurrentHashMap<>();   // owned
    private final List<Post> posts = new CopyOnWriteArrayList<>();

    Page(String name, PageType type, Member owner) { this.name = name; this.type = type; this.owner = owner; }

    long id() { return id; }
    String name() { return name; }
    PageType type() { return type; }
    Member owner() { return owner; }
    void addFollower(long memberId) { followers.add(memberId); }
    int followerCount() { return followers.size(); }            // never store what you can count

    void recommend(Member by, int rating, String text) {
        if (recommendations.putIfAbsent(by.id(), new PageRecommendation(by.id(), rating, text)) != null)
            throw new IllegalStateException(by + " already recommended " + name);
    }

    OptionalDouble averageRating() {
        return recommendations.values().stream().mapToInt(PageRecommendation::rating).average();
    }
    int recommendationCount() { return recommendations.size(); }

    Post publish(Member by, String text) {
        if (by.id() != owner.id()) throw new IllegalStateException(by + " does not manage page " + name);
        Post p = new Post(by, text, new Audience.Public(), null, this, null);
        posts.add(p);
        return p;
    }
    List<Post> posts() { return List.copyOf(posts); }
}

enum GroupPrivacy { PUBLIC, PRIVATE }

final class Group {
    private static final AtomicLong IDS = new AtomicLong(900);
    private final long id = IDS.incrementAndGet();
    private final String name;
    private final GroupPrivacy privacy;
    private final Member owner;
    private final Set<Long> members = ConcurrentHashMap.newKeySet();
    private final Set<Long> joinRequests = ConcurrentHashMap.newKeySet();
```

```
    private final List<Post> posts = new CopyOnWriteArrayList<>();

    Group(String name, GroupPrivacy privacy, Member owner) {
        this.name = name;
        this.privacy = privacy;
        this.owner = owner;
        members.add(owner.id());
        owner.joinedGroup(this);
    }

    long id() { return id; }
    String name() { return name; }
    boolean isPublic() { return privacy == GroupPrivacy.PUBLIC; }
    boolean isMember(long memberId) { return members.contains(memberId); }
    int memberCount() { return members.size(); }

    /** Public groups: join at once. Private groups: the request waits for the owner. Returns true if joined now.
*/
    boolean requestToJoin(Member m) {
        if (isPublic()) { members.add(m.id()); m.joinedGroup(this); return true; }
        joinRequests.add(m.id());
        return false;
    }

    void approve(Member by, Member requester) {
        if (by.id() != owner.id()) throw new IllegalStateException("only the owner approves members");
        if (!joinRequests.remove(requester.id())) throw new IllegalStateException(requester + " did not ask to
join");
        members.add(requester.id());
        requester.joinedGroup(this);
    }

    Post post(Member by, String text) {
        if (!isMember(by.id())) throw new IllegalStateException(by + " is not a member of " + name);
        Post p = new Post(by, text, new Audience.Public(), this, null, null);
        posts.add(p);
        return p;
    }
    List<Post> posts() { return List.copyOf(posts); }
}
```

```java
// File: lld/ch18/facebook/Services.java
package lld.ch18.facebook;

import java.util.Arrays;
import java.util.Comparator;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.LongFunction;
import java.util.stream.Stream;

// ---------------------------------------------------------------- friends

enum RequestStatus { PENDING, ACCEPTED, REJECTED, CANCELED }

/** Records BOTH ends; an invitation that only knows the invitee cannot be accepted. */
final class FriendRequest {
    private static final AtomicLong IDS = new AtomicLong();
    private final long id = IDS.incrementAndGet();
    private final Member from;
    private final Member to;
    private RequestStatus status = RequestStatus.PENDING;

    FriendRequest(Member from, Member to) { this.from = from; this.to = to; }
    long id() { return id; }
    Member from() { return from; }
    Member to() { return to; }
    synchronized RequestStatus status() { return status; }
    synchronized void moveTo(RequestStatus next) {
        if (status != RequestStatus.PENDING) throw new IllegalStateException("request is already " + status);
        status = next;
    }
}

/** Owns the friend graph. Accepting changes two members, so it lives here, not on either member. */
final class FriendService {
    private final Map<Long, Set<Long>> friends = new ConcurrentHashMap<>();
    private final Map<Long, FriendRequest> requests = new ConcurrentHashMap<>();
    private final NotificationService notifications;

    FriendService(NotificationService notifications) { this.notifications = notifications; }

    synchronized FriendRequest sendRequest(Member from, Member to) {
        if (from.id() == to.id()) throw new IllegalArgumentException("cannot befriend yourself");
        if (areFriends(from, to)) throw new IllegalStateException(from + " and " + to + " are already friends");
        if (pendingBetween(from.id(), to.id())) throw new IllegalStateException("a request between " + from + " and
" + to + " is pending");
        FriendRequest r = new FriendRequest(from, to);
        requests.put(r.id(), r);
        notifications.send(to, NotificationType.FRIEND_REQUEST, from + " sent you a friend request");
        return r;
    }

    synchronized void accept(FriendRequest r, Member by) {
        if (by.id() != r.to().id()) throw new IllegalStateException("only " + r.to() + " can accept");
        r.moveTo(RequestStatus.ACCEPTED);
        friends.computeIfAbsent(r.from().id(), k -> ConcurrentHashMap.newKeySet()).add(r.to().id());
        friends.computeIfAbsent(r.to().id(), k -> ConcurrentHashMap.newKeySet()).add(r.from().id());
    }

    synchronized void reject(FriendRequest r, Member by) {
        if (by.id() != r.to().id()) throw new IllegalStateException("only " + r.to() + " can reject");
        r.moveTo(RequestStatus.REJECTED);
```

```
    }

    synchronized void unfriend(Member a, Member b) {
        friends.getOrDefault(a.id(), Set.of()).remove(b.id());
        friends.getOrDefault(b.id(), Set.of()).remove(a.id());
    }

    boolean areFriends(Member a, Member b) { return friendsOf(a.id()).contains(b.id()); }
    Set<Long> friendsOf(long memberId) { return Set.copyOf(friends.getOrDefault(memberId, Set.of())); }

    /** Looks BOTH ways: a request from a to b or from b to a. */
    boolean pendingBetween(long a, long b) {
        return requests.values().stream().anyMatch(r -> r.status() == RequestStatus.PENDING
                && ((r.from().id() == a && r.to().id() == b) || (r.from().id() == b && r.to().id() == a)));
    }
}

// ---------------------------------------------------------------- privacy

/**
 * The ONE place that decides "can this viewer see this post?". The post knows
 * its privacy list; only the friend graph knows friendship; this asks both.
 * Feed, search and profile pages all call it, so the rule is written once.
 */
final class VisibilityService {
    private final FriendService friends;
    VisibilityService(FriendService friends) { this.friends = friends; }

    boolean canSee(Member viewer, Post post) {
        Member author = post.author();
        if (author.status() != AccountStatus.ACTIVE) return false;
        if (viewer.id() == author.id()) return true;
        if (post.group().isPresent()) {
            Group g = post.group().get();
            return g.isPublic() || g.isMember(viewer.id());
        }
        return switch (post.audience()) {
            case Audience.Public p -> true;
            case Audience.Friends f -> friends.areFriends(viewer, author);
            case Audience.Custom c -> friends.areFriends(viewer, author) &&
post.isAllowedByPrivacyList(viewer.id());
            case Audience.OnlyMe o -> false;
        };
    }
}

// ---------------------------------------------------------------- notifications

enum NotificationType { FRIEND_REQUEST, NEW_MESSAGE, NEW_COMMENT, PAGE_UPDATE }

record Notification(NotificationType type, String text, long at) {}

/** Strategy for delivery. A new channel is a new class, not a new if-branch. */
interface NotificationChannel { void deliver(Member to, Notification n); }

final class PushChannel implements NotificationChannel {
    @Override public void deliver(Member to, Notification n) {
        to.notify(n);
        System.out.println("    [push -> " + to + "] " + n.text());
    }
}

final class EmailChannel implements NotificationChannel {
    @Override public void deliver(Member to, Notification n) {
        System.out.println("    [email -> " + to.email() + "] " + n.text());
    }
}

final class NotificationService {
```

```
    private final Map<NotificationType, List<NotificationChannel>> routes;
    NotificationService(Map<NotificationType, List<NotificationChannel>> routes) { this.routes =
Map.copyOf(routes); }
    void send(Member to, NotificationType type, String text) {
        Notification n = new Notification(type, text, Tick.next());
        routes.getOrDefault(type, List.of()).forEach(c -> c.deliver(to, n));
    }
}

// ---------------------------------------------------------------- search

/** Every key maps to a set of matches (never a single object), and post search respects privacy. */
final class SearchIndex {
    private final Map<String, Set<Member>> members = new ConcurrentHashMap<>();
    private final Map<String, Set<Group>> groups = new ConcurrentHashMap<>();
    private final Map<String, Set<Page>> pages = new ConcurrentHashMap<>();
    private final Map<String, Set<Post>> posts = new ConcurrentHashMap<>();
    private final VisibilityService visibility;

    SearchIndex(VisibilityService visibility) { this.visibility = visibility; }

    void add(Member m) { index(members, m.name(), m); }
    void add(Group g) { index(groups, g.name(), g); }
    void add(Page p) { index(pages, p.name(), p); }
    void add(Post p) { index(posts, p.text(), p); }

    List<Member> searchMembers(String q) { return find(members,
q).sorted(Comparator.comparing(Member::name)).toList(); }
    List<Group> searchGroups(String q) { return find(groups, q).toList(); }
    List<Page> searchPages(String q) { return find(pages, q).toList(); }

    /** Takes the viewer: privacy does not stop being true inside a search. */
    List<Post> searchPosts(String word, Member viewer) {
        return find(posts, word).filter(p -> visibility.canSee(viewer, p))
                .sorted(Comparator.comparingLong(Post::createdAt).reversed()).toList();
    }

    private static <T> void index(Map<String, Set<T>> idx, String text, T item) {
        for (String w : words(text)) idx.computeIfAbsent(w, k -> ConcurrentHashMap.newKeySet()).add(item);
    }

    private static <T> Stream<T> find(Map<String, Set<T>> idx, String q) {
        List<String> ws = words(q);
        if (ws.isEmpty()) return Stream.empty();
        return idx.getOrDefault(ws.get(0), Set.of()).stream()
                .filter(item -> ws.stream().allMatch(w -> idx.getOrDefault(w, Set.of()).contains(item)));
    }

    private static List<String> words(String s) {
        return Arrays.stream(s.toLowerCase(Locale.ROOT).split("\\W+")).filter(w -> !w.isEmpty()).toList();
    }
}

// ---------------------------------------------------------------- news feed

/** Strategy for ranking the news feed. */
interface FeedRanker { Comparator<Post> order(Member viewer); }

final class RecentFirst implements FeedRanker {
    @Override public Comparator<Post> order(Member viewer) { return
Comparator.comparingLong(Post::createdAt).reversed(); }
}

/** Engagement x affinity / age, a toy version of the idea behind ranked feeds. */
final class EngagementRanker implements FeedRanker {
    private final FriendService friends;
    EngagementRanker(FriendService friends) { this.friends = friends; }

    @Override public Comparator<Post> order(Member viewer) {
```

```
        return Comparator.comparingDouble((Post p) -> -score(viewer,
p)).thenComparing(Comparator.comparingLong(Post::createdAt).reversed());
    }

    private double score(Member viewer, Post p) {
        double engagement = 1 + p.likeCount() + 2.0 * p.comments().size();
        double affinity = friends.areFriends(viewer, p.author()) ? 2.0 : 1.0;
        return engagement * affinity / (1 + (Tick.now() - p.createdAt()) / 10.0);
    }
}

/**
 * Pull model: gather candidates (friends, followed members, followed pages,
 * joined groups), drop what the viewer may not see, rank, cut to size.
 */
final class NewsFeedService {
    private final FriendService friends;
    private final VisibilityService visibility;
    private final LongFunction<Member> members;
    private final LongFunction<Page> pages;
    private final LongFunction<Group> groups;

    NewsFeedService(FriendService f, VisibilityService v, LongFunction<Member> m, LongFunction<Page> p,
LongFunction<Group> g) {
        friends = f; visibility = v; members = m; pages = p; groups = g;
    }

    List<Post> feed(Member viewer, FeedRanker ranker, int limit) {
        Set<Long> people = new HashSet<>(friends.friendsOf(viewer.id()));
        people.addAll(viewer.followingMembers());
        Stream<Post> fromPeople = people.stream().map(members::apply).flatMap(m -> m.posts().stream());
        Stream<Post> fromPages = viewer.followingPages().stream().map(pages::apply).flatMap(pg ->
pg.posts().stream());
        Stream<Post> fromGroups = viewer.groups().stream().map(groups::apply).flatMap(g -> g.posts().stream());
        return Stream.of(fromPeople, fromPages, fromGroups).flatMap(s -> s)
                .distinct()
                .filter(p -> p.author().id() != viewer.id())
                .filter(p -> visibility.canSee(viewer, p))
                .sorted(ranker.order(viewer))
                .limit(limit)
                .toList();
    }
}

// ---------------------------------------------------------------- suggestions (extended requirement)

record Suggestion(Member member, int mutualFriends) {
    @Override public String toString() { return member.name() + "(" + mutualFriends + " mutual)"; }
}

/** Friends of friends, scored by mutual friends; never yourself, a friend, or anyone with a pending request. */
final class SuggestionService {
    private final FriendService friends;
    private final LongFunction<Member> members;

    SuggestionService(FriendService friends, LongFunction<Member> members) { this.friends = friends; this.members =
members; }

    List<Suggestion> suggest(Member me, int limit) {
        Set<Long> mine = friends.friendsOf(me.id());
        Map<Long, Integer> mutual = new HashMap<>();
        for (long f : mine) {
            for (long candidate : friends.friendsOf(f)) {
                if (candidate == me.id() || mine.contains(candidate) || friends.pendingBetween(me.id(), candidate))
continue;
                mutual.merge(candidate, 1, Integer::sum);
            }
        }
        return mutual.entrySet().stream()
```

```
                .map(e -> new Suggestion(members.apply(e.getKey()), e.getValue()))
                .sorted(Comparator.comparingInt(Suggestion::mutualFriends).reversed()
                        .thenComparingLong(s -> s.member().id()))           // stable tie-break
                .limit(limit)
                .toList();
    }
}
```

```java
// File: lld/ch18/facebook/Facebook.java
package lld.ch18.facebook;

import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/** Facade: wires the services and checks account status on every write. */
final class Facebook {
    private final Map<Long, Member> members = new ConcurrentHashMap<>();
    private final Map<Long, Page> pages = new ConcurrentHashMap<>();
    private final Map<Long, Group> groups = new ConcurrentHashMap<>();
    final NotificationService notifications;
    final FriendService friends;
    final VisibilityService visibility;
    final SearchIndex search;
    final NewsFeedService feed;
    final SuggestionService suggestions;

    Facebook(Map<NotificationType, List<NotificationChannel>> routes) {
        notifications = new NotificationService(routes);
        friends = new FriendService(notifications);
        visibility = new VisibilityService(friends);
        search = new SearchIndex(visibility);
        feed = new NewsFeedService(friends, visibility, members::get, pages::get, groups::get);
        suggestions = new SuggestionService(friends, members::get);
    }

    <M extends Member> M register(M m) {
        members.put(m.id(), m);
        search.add(m);
        return m;
    }

    PrivacyList createPrivacyList(Member owner, String name, List<Member> people) {
        PrivacyList list = new PrivacyList(owner, name);
        for (Member p : people) {
            if (!friends.areFriends(owner, p)) throw new IllegalStateException(p + " is not a friend of " + owner);
            list.add(p.id());
        }
        owner.addPrivacyList(list);
        return list;
    }

    Post post(Member author, String text, Audience audience) {
        active(author);
        if (audience instanceof Audience.Custom c && c.list().owner() != author)
            throw new IllegalArgumentException("you can only use your own privacy lists");
        Post p = new Post(author, text, audience, null, null, null);
        author.addPost(p);
        search.add(p);
        return p;
    }

    Post share(Member by, Post original, Audience audience) {
        active(by);
        if (!visibility.canSee(by, original)) throw new IllegalStateException(by + " cannot see that post");
        Post p = new Post(by, "shared: " + original.text(), audience, null, null, original);
        by.addPost(p);
        return p;
    }

    Comment comment(Member by, Post post, String text) {
        active(by);
        if (!visibility.canSee(by, post)) throw new IllegalStateException(by + " cannot see that post");
        Comment c = post.addComment(by, text);
        if (post.author() != by) notifications.send(post.author(), NotificationType.NEW_COMMENT, by + " commented
on your post");
```

```
        return c;
    }

    boolean like(Member by, SocialContent content, Post containingPost) {
        active(by);
        if (!visibility.canSee(by, containingPost)) throw new IllegalStateException(by + " cannot see that post");
        return content.like(by.id());
    }

    void message(Member from, List<Member> to, String text) {
        active(from);
        Message m = new Message(from, List.copyOf(to), text, Tick.next());
        for (Member r : to) {
            r.receive(m);
            notifications.send(r, NotificationType.NEW_MESSAGE, "new message from " + from);
        }
    }

    Page createPage(Member owner, String name, PageType type) {
        active(owner);
        Page p = new Page(name, type, owner);
        pages.put(p.id(), p);
        search.add(p);
        return p;
    }

    void followPage(Member m, Page p) {
        if (m.followPage(p)) p.addFollower(m.id());
    }

    Post publishOnPage(Member by, Page page, String text) {
        Post p = page.publish(by, text);
        search.add(p);
        return p;
    }

    Group createGroup(Member owner, String name, GroupPrivacy privacy) {
        active(owner);
        Group g = new Group(name, privacy, owner);
        groups.put(g.id(), g);
        search.add(g);
        return g;
    }

    Post postInGroup(Member by, Group g, String text) {
        active(by);
        Post p = g.post(by, text);
        search.add(p);
        return p;
    }

    void block(Admin admin, Member target) { target.setStatus(AccountStatus.BLOCKED); }

    private static void active(Member m) {
        if (m.status() != AccountStatus.ACTIVE) throw new IllegalStateException(m + " is " + m.status());
    }
}
```

```java
// File: lld/ch18/facebook/Main.java
package lld.ch18.facebook;

import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        NotificationChannel push = new PushChannel();
        NotificationChannel email = new EmailChannel();
        Facebook fb = new Facebook(Map.of(
                NotificationType.FRIEND_REQUEST, List.of(push, email),
                NotificationType.NEW_MESSAGE, List.of(push),
                NotificationType.NEW_COMMENT, List.of(push)));

        Member alice = fb.register(new Member("Alice Rao", "alice@example.com"));
        Member bob = fb.register(new Member("Bob Das", "bob@example.com"));
        Member carol = fb.register(new Member("Carol Nair", "carol@example.com"));
        Member dave = fb.register(new Member("Dave Paul", "dave@example.com"));
        Member erin = fb.register(new Member("Erin Shah", "erin@example.com"));
        Member frank = fb.register(new Member("Frank Lee", "frank@example.com"));
        Member grace = fb.register(new Member("Grace Lee", "grace@example.com"));
        Admin admin = fb.register(new Admin("Admin", "admin@example.com"));

        section("1. Profile and search by name");
        alice.profile().addWork(new Work("Designer", "Studio Nine", "Mumbai", 2021, null));
        alice.profile().addPlace(new Place("Mumbai", "India", 2019));
        System.out.println("Alice's picture before upload: " + alice.profile().picture());
        System.out.println("search 'lee': " + fb.search.searchMembers("lee"));

        section("2. Friend requests");
        FriendRequest r = fb.friends.sendRequest(alice, bob);
        fb.friends.accept(r, bob);
        attempt(() -> fb.friends.accept(r, bob));
        befriend(fb, alice, carol);
        befriend(fb, alice, erin);
        befriend(fb, bob, dave);
        befriend(fb, carol, dave);
        befriend(fb, erin, grace);
        frank.follow(alice);                                    // follow without being friends

        section("3. Friend suggestions (extended requirement)");
        System.out.println("before: " + fb.suggestions.suggest(alice, 5));
        fb.friends.sendRequest(alice, grace);
        System.out.println("after inviting Grace: " + fb.suggestions.suggest(alice, 5));
        attempt(() -> fb.friends.sendRequest(grace, alice));        // pending the other way

        section("4. Posts and privacy");
        PrivacyList close = fb.createPrivacyList(alice, "Close friends", List.of(bob, carol));
        attempt(() -> fb.createPrivacyList(alice, "Oops", List.of(frank)));
        Post pub = fb.post(alice, "New portfolio is live!", new Audience.Public());
        Post fr = fb.post(alice, "Moving to Pune next month", new Audience.Friends());
        Post secret = fb.post(alice, "Planning a surprise party for Erin", new Audience.Custom(close));
        for (Member viewer : List.of(alice, bob, erin, frank)) {
            System.out.printf("  %-10s public=%-5s friends=%-5s closeFriends=%s%n", viewer.name(),
                    fb.visibility.canSee(viewer, pub), fb.visibility.canSee(viewer, fr),
fb.visibility.canSee(viewer, secret));
        }
        System.out.println("search posts 'party' as Bob:  " + fb.search.searchPosts("party", bob));
        System.out.println("search posts 'party' as Erin: " + fb.search.searchPosts("party", erin));

        section("5. Comments, likes, shares");
        Comment cm = fb.comment(bob, secret, "I'll bring the cake");
        fb.like(carol, secret, secret);
        fb.like(alice, cm, secret);                                 // comments can be liked too
        System.out.println("Bob likes: " + fb.like(bob, secret, secret) + " / again: " + fb.like(bob, secret,
secret));
        attempt(() -> fb.comment(erin, secret, "What party?"));
```

```
        fb.share(erin, pub, new Audience.Friends());

        section("6. Pages and groups");
        Page pizza = fb.createPage(carol, "Joe's Pizza", PageType.BUSINESS);
        fb.followPage(alice, pizza);
        pizza.recommend(alice, 5, "Best crust in town");
        pizza.recommend(bob, 4, "Good, a bit slow");
        attempt(() -> pizza.recommend(alice, 1, "changed my mind"));
        fb.publishOnPage(carol, pizza, "Two-for-one pizza on Friday");
        System.out.printf("%s: followers=%d, rating=%.1f from %d recommendations%n",
                pizza.name(), pizza.followerCount(), pizza.averageRating().orElse(0), pizza.recommendationCount());
        Group chess = fb.createGroup(dave, "Chess Club", GroupPrivacy.PRIVATE);
        System.out.println("Frank joined at once? " + chess.requestToJoin(frank));
        chess.approve(dave, frank);
        fb.postInGroup(frank, chess, "Chess tournament on Sunday");
        attempt(() -> fb.postInGroup(alice, chess, "Can I play?"));
        System.out.println("search posts 'chess' as Alice: " + fb.search.searchPosts("chess", alice)
                + ", as Dave: " + fb.search.searchPosts("chess", dave));

        section("7. News feeds");
        System.out.println("Alice (recent): " + fb.feed.feed(alice, new RecentFirst(), 5));
        System.out.println("Bob   (ranked): " + fb.feed.feed(bob, new EngagementRanker(fb.friends), 5));
        System.out.println("Frank (recent): " + fb.feed.feed(frank, new RecentFirst(), 5));

        section("8. Messages and admin");
        System.out.println("Dave before Frank is blocked: " + fb.feed.feed(dave, new RecentFirst(), 5));
        fb.message(bob, List.of(alice, carol), "Party at 8?");
        fb.block(admin, frank);
        System.out.println("Dave after Frank is blocked: " + fb.feed.feed(dave, new RecentFirst(), 5));
        System.out.println("Alice's notifications: " +
alice.notifications().stream().map(Notification::type).toList());
    }

    static void befriend(Facebook fb, Member a, Member b) { fb.friends.accept(fb.friends.sendRequest(a, b), b); }

    static void section(String t) { System.out.println("\n== " + t + " =="); }

    static void attempt(Runnable r) {
        try { r.run(); } catch (RuntimeException e) { System.out.println("  REJECTED: " + e.getMessage()); }
    }
}
```

Output:

```
== 1. Profile and search by name ==
Alice's picture before upload: default-avatar.png
search 'lee': [Frank Lee, Grace Lee]

== 2. Friend requests ==
    [push -> Bob Das] Alice Rao sent you a friend request
    [email -> bob@example.com] Alice Rao sent you a friend request
  REJECTED: request is already ACCEPTED
    [push -> Carol Nair] Alice Rao sent you a friend request
    [email -> carol@example.com] Alice Rao sent you a friend request
    [push -> Erin Shah] Alice Rao sent you a friend request
    [email -> erin@example.com] Alice Rao sent you a friend request
    [push -> Dave Paul] Bob Das sent you a friend request
    [email -> dave@example.com] Bob Das sent you a friend request
    [push -> Dave Paul] Carol Nair sent you a friend request
    [email -> dave@example.com] Carol Nair sent you a friend request
    [push -> Grace Lee] Erin Shah sent you a friend request
    [email -> grace@example.com] Erin Shah sent you a friend request

== 3. Friend suggestions (extended requirement) ==
before: [Dave Paul(2 mutual), Grace Lee(1 mutual)]
    [push -> Grace Lee] Alice Rao sent you a friend request
    [email -> grace@example.com] Alice Rao sent you a friend request
after inviting Grace: [Dave Paul(2 mutual)]
  REJECTED: a request between Grace Lee and Alice Rao is pending

== 4. Posts and privacy ==
  REJECTED: Frank Lee is not a friend of Alice Rao
  Alice Rao  public=true  friends=true  closeFriends=true
  Bob Das    public=true  friends=true  closeFriends=true
  Erin Shah  public=true  friends=true  closeFriends=false
  Frank Lee  public=true  friends=false closeFriends=false
search posts 'party' as Bob:  ["Planning a surprise party for Erin" by Alice Rao]
search posts 'party' as Erin: []

== 5. Comments, likes, shares ==
    [push -> Alice Rao] Bob Das commented on your post
Bob likes: true / again: false
  REJECTED: Erin Shah cannot see that post

== 6. Pages and groups ==
  REJECTED: Alice Rao already recommended Joe's Pizza
Joe's Pizza: followers=1, rating=4.5 from 2 recommendations
Frank joined at once? false
  REJECTED: Alice Rao is not a member of Chess Club
search posts 'chess' as Alice: [], as Dave: ["Chess tournament on Sunday" by Frank Lee [group Chess Club]]

== 7. News feeds ==
Alice (recent): ["Two-for-one pizza on Friday" by Carol Nair [page Joe's Pizza], "shared: New portfolio is live!"
by Erin Shah]
Bob   (ranked): ["Planning a surprise party for Erin" by Alice Rao, "Moving to Pune next month" by Alice Rao, "New
portfolio is live!" by Alice Rao]
Frank (recent): ["New portfolio is live!" by Alice Rao]

== 8. Messages and admin ==
Dave before Frank is blocked: ["Chess tournament on Sunday" by Frank Lee [group Chess Club]]
    [push -> Alice Rao] new message from Bob Das
    [push -> Carol Nair] new message from Bob Das
Dave after Frank is blocked: []
Alice's notifications: [NEW_COMMENT, NEW_MESSAGE]
```

Walking through the important parts

- **The visibility table in section 4** is the course's key scenario, with four viewers. Alice is the author, so she sees all three of her posts. Bob is a friend *and* on the "Close friends" list, so he sees all three posts. Erin is a friend but *not* on the list, so she sees the public and friends posts but not the close-friends one.

Frank is not Alice's friend, so only the public post is visible to him. Two rules, both applied, neither written twice.

- **Search respects privacy:** searching "party" returns Alice's close-friends post for Bob and nothing for Erin, because `searchPosts` takes the viewer and calls `canSee`.
- `createPrivacyList` refuses Frank because a privacy list may contain only friends.
- **Comments, likes and shares go through `canSee` too:** Erin cannot comment on the close-friends post. Without this, "comment" would be a side door into private content.
- **Suggestions:** before inviting Grace, Alice sees Dave (2 mutual: Bob and Carol) and Grace (1 mutual: Erin). After Alice invites Grace, Grace disappears (pending), and Grace cannot send a request back while one is pending.
- **Pages:** one recommendation per member (`putIfAbsent`), follower count and average rating computed from the stored data.
- **Private group:** Frank's join waits for Dave's approval; Alice (not a member) cannot post there and cannot find the group's post by search, while Dave can.
- **Feeds:** Alice's feed includes the page she follows and Erin's share; Bob's ranked feed puts the most-engaged, most recent friend posts first; Frank (who only follows Alice) sees only her public post.
- **Blocking:** once Frank is blocked, `canSee` hides his posts, so his group post disappears from Dave's feed.
- **Notification routing:** friend requests go to push *and* email; comments and messages go to push only — configured with a `Map<NotificationType, List<NotificationChannel>>`.

Concurrency & edge cases

- **Friend graph:** `sendRequest`, `accept`, `reject` and `unfriend` are `synchronized` on the service, and adjacency sets (each member's set of directly linked members) are concurrent, so an accept cannot leave a one-way friendship. In a database, a friendship is one row keyed by the ordered pair `(min(id), max(id))`, which makes symmetry automatic.
- **Pending in both directions:** checked by `pendingBetween(a, b)` for both requests and suggestions.
- **Privacy changes after posting:** because visibility is computed at read time, removing someone from a list or unfriending them hides the post immediately. (A push-model feed that copied posts into feeds would need to re-check `canSee` at read time anyway.)
- **Shares:** sharing requires seeing the original; a share of a friends-only post should not expose it to the sharer's audience — a production rule would cap the share's audience at the original's.
- **Deterministic ordering** (the same input always produces the same order)**:** suggestions tie-break by id; feeds tie-break by creation time; our logical `Tick` clock (a counter that goes up by one per event and stands in for real time) makes demo output repeatable.
- **Blocked or deleted authors:** `canSee` returns false for inactive authors.

Extensions & follow-up questions

- **"Scale the news feed."** — **Fan-out on write**: when a member posts, push the post id into each friend's feed cache (fast reads; costly for accounts with millions of followers). **Fan-out on read**: assemble at open time (our design). Real systems mix them: push for normal users, pull for celebrities, merge at read, and always re-check privacy.
- **"Rank the feed better."** — Add signals (affinity = how often the viewer interacts with the author, content type, recency decay) behind the same `FeedRanker` interface; that is the idea behind Facebook's early **EdgeRank** (affinity × weight × time decay).
- **"Suggestions at scale."** — Precompute friends-of-friends offline on a schedule and store them (the course notes the original diagram had a `memberSuggestions` field for exactly this), adding signals like shared groups, schools and workplaces.
- **"Blocking one member from another."** — Add a per-member block set; `canSee` and `sendRequest` check it both ways.
- **"Reactions beyond like."** — Change the like set to `Map<memberId, Reaction>`, exactly like Stack Overflow's vote map.
- **"Tagging people in posts."** — Add tagged ids to `Post`, notify them, and (a privacy decision) optionally widen the audience to their friends.
- **"Events, stories, marketplace, albums with their own privacy."** — Each is a new content type extending `SocialContent` with its own `Audience`.

Real-world example

Facebook launched **News Feed** in 2006, turning profiles you had to visit into a stream pushed to you — the feature this design adds. Posts carry an audience selector (Public, Friends, specific friends, Only me, custom lists), and a personal profile is capped at **5,000 friends** (beyond that people use followers or pages). The social graph itself is served by **TAO**, a distributed store Facebook built for objects (users, posts) and associations (friendship, likes) — the production-scale version of our `FriendService` plus `SocialContent` likes.

In the interview

Minutes	What to do
0–5	Clarify friend vs follow, the audience options, pages, groups, feed
5–15	Profile, `SocialContent` (likes as set, comments liked too), `PrivacyList`, `Audience`
15–25	`FriendService` owns the graph (accept is two-sided); following is a set
25–35	**The strongest point:** the post holds its list and the friend service holds friendships, so `canSee` must consult both — and search, feed, comments and shares all call it
35–45	Feed (pull, push, hybrid), suggestions (mutual count, exclude pending both ways, stable sort), concurrency

30-second answer: "Posts and comments share a likes set. A post has an audience — public, friends, a custom privacy list, or only me. The post can check its own list, but only the friend service knows friendships, so one `VisibilityService.canSee` asks both, and every read path — feed, search, comment, like, share — calls it. The friend service owns the graph because accepting changes two members; following is one-sided. The feed gathers from friends, follows, pages and groups, filters by `canSee`, and ranks with a Strategy. Suggestions are friends of friends by mutual count, excluding pending requests either way, with a stable tie-break."

Follow-ups with answers

- *How could a private post leak?* — Through any read path that forgets privacy: search, feed, comment threads, shares, notifications. That is why `canSee` is one method used by all of them.
- *Why exclude pending requests in suggestions?* — Re-suggesting someone you already invited is a visible bug; and the check must look both ways.
- *Why the second sort key?* — Without it, people with the same mutual count can swap places between page loads, and a list that reshuffles itself looks broken.

Self-check

- **Q:** Erin is Alice's friend but not on "Close friends". Can she see a close-friends post? **A:** No — she must be a friend *and* on the list.
- **Q:** Why does `searchPosts` take a viewer? **A:** Because privacy applies inside search; otherwise search leaks private posts.
- **Q:** Where are follower counts stored? **A:** Nowhere — they are the size of the follower set.
- **Q:** Which three people are excluded from suggestions? **A:** Yourself, existing friends, and anyone with a pending request in either direction.
- **Q:** Name the two feed-building strategies and one weakness of each. **A:** Push (fan-out on write): expensive for accounts with huge audiences. Pull (fan-out on read): slower reads because it gathers at open time.

TL;DR

Facebook = LinkedIn's structure + **privacy**: a sealed `Audience`, privacy lists of friends, and one `VisibilityService.canSee` (post's list AND friend graph) called by feed, search, comments, likes and shares. `FriendService` owns the symmetric graph; following is a set. Pages and groups store collections, not counts. The news feed gathers, filters, and ranks with a Strategy; suggestions are friends-of-friends by mutual count, excluding pending requests both ways, with a stable tie-break.

18.5 Chapter summary

The four designs side by side

Aspect	Stack Overflow	Cricinfo	LinkedIn	Facebook
Core content	Question / answer / comment	Ball / over / innings / match	Post / comment / message	Post / comment / message / page post / group post
Relationship between users	None (votes and reputation)	Team membership, officials	Connection (mutual) + follow	Friendship (mutual) + follow
Two-sided operation → service	—	—	`ConnectionService.accept`	`FriendService.accept`
Role / permission model	Reputation privileges + Moderator/Admin types	Admin scorer, Commentator	Member / Admin	Member / Admin + audiences
"Derive, don't store"	Score from votes, tag counts	All stats from balls	Connection count, follower count	Member, follower counts, ratings
Strategy	Reputation policy, channels	(Target calculator as extension)	Feed ranker, channels	Feed ranker, channels
Observer	Reputation listeners (badges)	Match observers (live feed)	Notifications	Notifications
Template Method	—	`StatQuery`	—	—
Search	Words and tags	Stat queries	Members, companies, jobs	Members, groups, pages, privacy-aware posts

Common mistakes across all four

- Counters instead of sets (votes, likes, endorsements) — cannot stop duplicates.
- Stored counts next to the collections they summarise — they drift.
- Two-sided operations on one entity — asymmetric graphs.
- Privacy or permission checks in one read path but not another.
- Search index values that are a single object instead of a list.
- Modelling the nouns but not the rules (six balls vs six *legal* balls; run outs; byes).

Chapter TL;DR

Platforms are rich domain models. Share behaviour in a content base class; keep votes and likes as sets of ids; derive every number you can; put operations that change two objects in a service; notify through Observer with Strategy channels; rank feeds with a Strategy; keep search in an inverted index that maps words to lists; and write each permission or privacy rule exactly once, then call it from every read path.

Chapter 19 — Case Studies: Commerce & Marketplaces (Part 2 of Platforms)

> **Where this fits.** Chapter 18 designed platforms where people *talk* (Q&A sites, social networks, sports scores). This chapter designs platforms where people *buy, sell and trade*. Money moves, stock runs out, prices change every second, and two people often want the same last thing at the same moment. Those three pressures — **money must be exact, state must follow rules, and shared things must not be double-sold** — shape every design here.

What you will learn

#	Problem	The one idea that makes it work	Main patterns
19.1	Amazon — online shopping	An order is an immutable copy of the cart, moved through a **state machine**	State machine (enum), Strategy (payment), Observer (notifications), Facade
19.2	Online stock brokerage + stock-exchange matching engine	Four order types differ in only **two questions**; a book matches by **price, then time**	Template Method, Observer, Strategy (price policy)
19.3	Online auction	"Read highest bid, compare, replace" must be **one atomic step**	Monitor lock, Observer, Specification
19.4	Music streaming (Spotify)	Play order is a **pluggable strategy**; plan rules live **in the plan**	Strategy, Observer, enum-with-behaviour
19.5	Food delivery (Swiggy) + FoodKart	Capacity and riders are **claimed atomically**; selection is a **strategy**	Strategy, Observer, State machine, CAS
19.6	Grocery store (POS & checkout)	A **pure pricing engine**: best offer per product, then a coupon spread over items so tax and refunds stay exact	Strategy (offers, tax, payment), Facade, Observer, enum state machine

How to run every example in this chapter

```
cd chapters/19-case-platforms-2/code
javac --release 21 -d /tmp/lld_19 $(find . -name '*.java')
java -cp /tmp/lld_19 lld.ch19.amazon.Main          # or brokerage, stockexchange, auction, music, fooddelivery, foodkart, grocery
```

`javac` is the Java compiler (it turns `.java` files into runnable `.class` files); `-d` names the output folder, and `java -cp` runs a class, where the **classpath** (`-cp`) is the list of folders Java searches for compiled classes.

All eight `Main` classes compile with Java 21 (standard library only — only the classes that ship with Java) and run. The output under each implementation is the real output of that program. The stock-exchange `Main` also runs a small test suite (a set of automatic checks that confirm the code behaves as expected; no JUnit — the usual Java testing library — needed).

> **Code-style note.** To keep each case readable, one file sometimes holds several small package-private types (for example `Payments.java` holds `Payment`, its two subclasses and `Shipment`). That is legal Java. In production you would usually give each top-level type its own file.

> **Key term — Java vocabulary used in this chapter:** A **package-private** type has no access keyword and is visible only inside its own package (folder of classes). A **top-level type** is a class, interface, enum or record not nested in another. An **interface** lists methods a class promises to provide. An **enum** is a type with a fixed list of named values. A **record** is a compact class that just carries data that never changes. A **sealed** type

> lists exactly which subclasses are allowed (the `permits` clause), so the compiler knows every possible kind. `synchronized` means only one thread at a time may run that code on the same object.

> **Key term — the design patterns named in this chapter: Strategy** puts each interchangeable way of doing a job (each payment method, each play order) behind one interface so it can be swapped or added without editing the caller. **Observer** lets objects subscribe to events from another object and be called back automatically, so the publisher does not know who listens. **Facade** is one simple class that hides many classes behind a few easy methods. **Template Method** is a base-class method that fixes the outline of an algorithm and lets subclasses fill in one or two steps. **Specification** turns each business rule or search condition into a small yes/no object that can be combined with others.

Four rules that apply to every design in this chapter

1. **Money is `BigDecimal`, never `double`.** `double` is Java's floating-point number type: it stores numbers in binary, so most decimal fractions are only approximated. `BigDecimal` is Java's exact decimal-number class. A `double` cannot store 0.1 exactly, so `0.1 + 0.2` prints `0.30000000000000004`. With `BigDecimal` you choose the scale (number of decimal places) and the rounding mode yourself. Two traps: build it from a `String` (`new BigDecimal("0.10")`), not from a `double`; and compare with `compareTo`, not `equals` — `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` is `false` because the scales differ.
2. **Status changes follow a state machine.** A **state machine** is a fixed list of states plus the allowed moves between them. We put the allowed moves *inside the enum*, so an illegal move (for example "cancel a delivered order") throws immediately instead of silently corrupting data.
3. **Check-then-act on shared stock must be atomic.** "Is there one left? Then take it" is two steps. If two threads (independent paths of execution running at the same time inside one program) run step 1 together, both see "yes" and both take it — a **race condition**, where the result depends on which thread happens to get there first. The fix is to do both steps under one lock (a guard that only one thread can hold at a time; the others wait), or with one atomic compare-and-set (CAS: "set to B only if it is still A").
4. **Copy prices into orders.** An order remembers the price that was paid, and is **immutable** (it cannot be changed after it is created). If the menu or catalog price changes later, old orders must not change.

Design Amazon — Online Shopping System

Problem & clarifying questions

Amazon started as an online bookshop and now sells almost every kind of consumer product and digital media. We design the **online retail part**: people list products, other people find them, put them in a cart, pay, and track the parcel.

Questions worth asking before you draw anything (with the answers we assume):

Question	Assumed answer
Can someone buy without an account?	No. Browsing and filling a cart are open to everyone; placing an order needs a **registered member** account.
Who sells?	Members can list products for sale (a marketplace, not only first-party — goods sold by the platform itself).
How do people pay?	Credit card or electronic bank transfer. Cards clear at once, transfers clear later.
Can an order be cancelled?	Yes, until it has shipped.
Do we notify people?	Yes, on every order or shipping status change.
Returns, wish lists, seller payouts, recommendations, stock spread over several warehouses?	Out of scope — a deliberate choice, and you should say so out loud.

Requirements

Functional

1. Members can add new products to sell.
2. Anyone can search products by **name** or **category**.
3. Anyone can search and view products; only registered members can buy.
4. Users can add, remove and change the quantity of items in their shopping cart.
5. Users can check out and buy everything in the cart.
6. Members can rate and review a product.
7. Members give a shipping address for delivery.
8. A member can cancel an order that has not shipped.
9. Members are notified each time the status of their order or of its shipment changes.
10. Members pay with a credit card or with an electronic bank transfer.
11. Members can track the shipment and see the current order state.

Non-functional

- **Correctness of stock:** two buyers can never both get the last unit.
- **Exact money:** totals are computed with `BigDecimal`.
- **Auditability** (being able to prove later exactly what happened and when)**:** every order and shipment keeps a history of its states, not just the latest one.
- **Extensibility** (how easily new features can be added without rewriting old code)**:** adding a payment method or a notification channel must not change the checkout code.

Use cases

Four actors:

- **Admin** — manages accounts (block / unblock) and adds or changes product categories.
- **Guest** — searches the catalog, adds and removes cart items, and can register to become a member.
- **Member** — everything a guest can do, plus placing orders and listing products to sell.
- **System** — sends order and shipment notifications.

Top use cases: add or update products (each change also updates the catalog), search products (by name, or by category), put items into the cart or take them out, check out, pay, add a product category, and send shipment notifications.

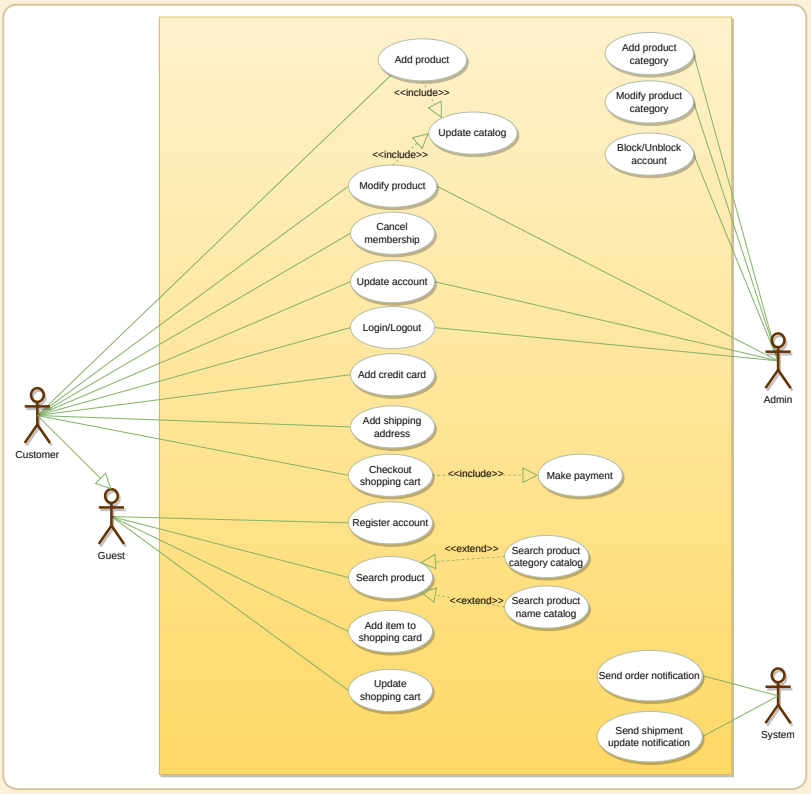


Figure — Use-case diagram for the online shopping system: Admin, Guest, Member and System and what each may do (course diagram)

In the figure above, notice that "Checkout" *includes* "Make payment" and "Add shipping address": you cannot check out without them. Search by name and search by category *extend* the general "Search product" use case.

Core entities

Class	Responsibility
`ProductCategory`	A named group such as Books or Electronics. Exists whether or not any product uses it.
`Product`	The thing sold: price, seller, stock count. **Owns** its reviews and guards its own stock (`reserve` / `release`).
`ProductReview`	Rating 1–5 plus text. Records the reviewer **by id**, so `Product` never depends on `Member`.
`Search` / `Catalog`	Interface with two search methods. The catalog keeps one index (a lookup map built in advance) per search (name words → products, category → products), so a lookup reads a map instead of scanning every product, and `addProduct` updates both indexes together so they cannot drift apart. The catalog only *points to* products; it never owns them.
`ShoppingCart`	Mutable (changeable) working area. **Owns** its items; clearing the cart destroys them.
`Item`	One line: product id, name, quantity, unit price (an immutable `record`). A pen is a *product*; ten pens in stock are ten product *items*.
`Order`	Immutable **copy** of the cart lines + address + a status that moves only along legal edges.
`OrderLog`	One entry per status change (unshipped, pending, complete, cancelled...).
`Payment` (`CreditCardPayment`, `BankTransferPayment`)	Takes the money. A card clears immediately; a bank transfer comes back `PENDING` from `process()` and clears later.
`Shipment` / `ShipmentLog`	A parcel and its tracking history (pending, shipped, delivered, on hold). The last log entry is "where is it now".
`Notification` channel (`EmailChannel`, `SmsChannel`)	One way of delivering a message. (The course draws an abstract `Notification` class with one subclass per channel; we use an interface — same idea.)
`Account`	Login data, addresses, status (active, blocked, banned...).
`Customer` → `Guest`, `Member`	`Customer` has only what *anyone* may do; ordering lives on `Member`.
`Admin`	Adds categories, blocks and unblocks members.
`OrderService`	Application service (a class that coordinates domain objects to carry out one user action) that turns a cart into an order and drives its state machine.

Class diagram

The course builds the model in five stages (products → catalog → payment/shipment/notification → cart/order → accounts). Here is the finished picture:

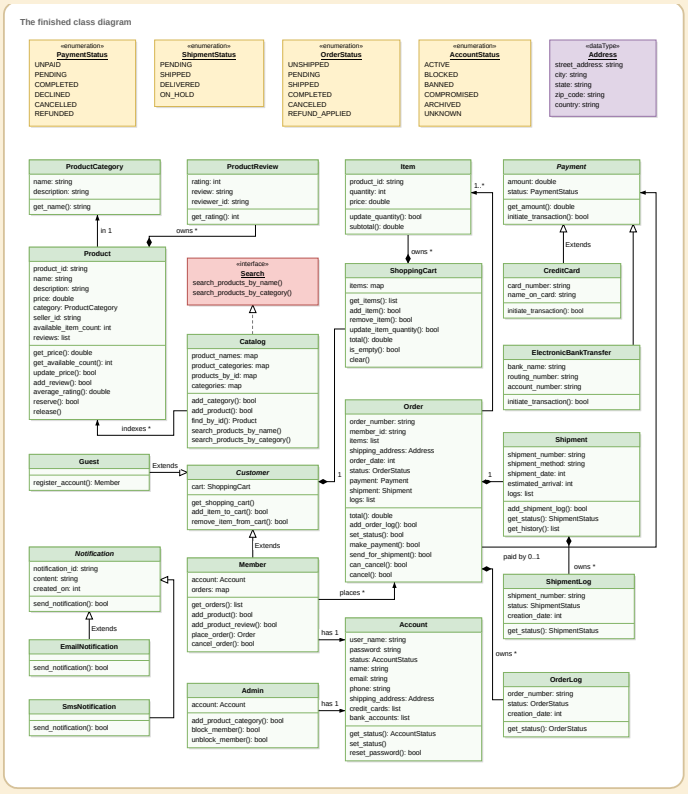


Figure — The finished class diagram of the online shopping system (course diagram)

In the figure above, read the arrows this way: a **filled diamond** means *composition* (the part belongs to the whole and is destroyed along with it — `Product` owns its reviews, `ShoppingCart` owns its items, `Order` owns its log and shipment). A **plain arrow** is *association* (a link without ownership — a product points at its category; an order only *refers to* the payment it was handed). The hollow triangle is inheritance (`Guest` and `Member` extend `Customer`).

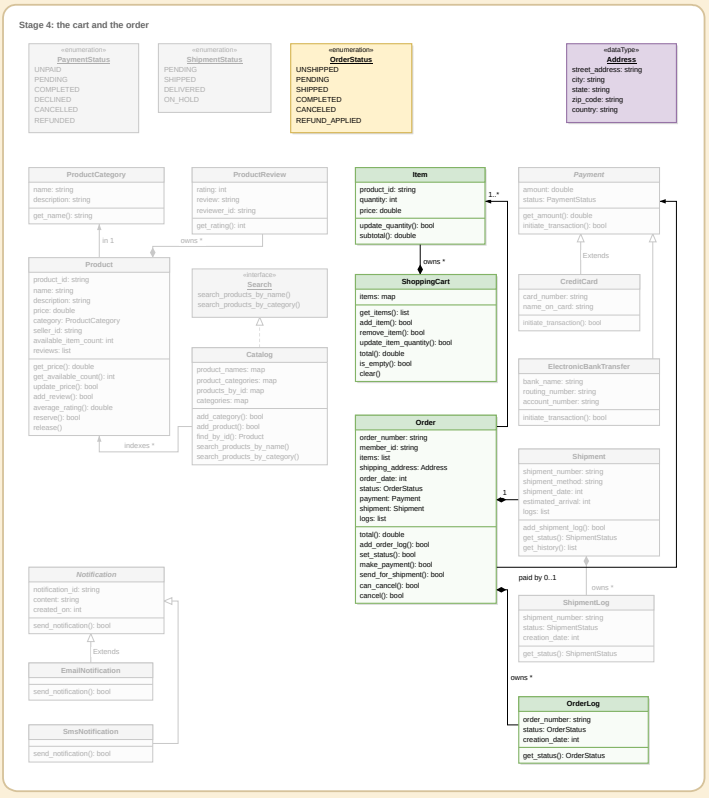
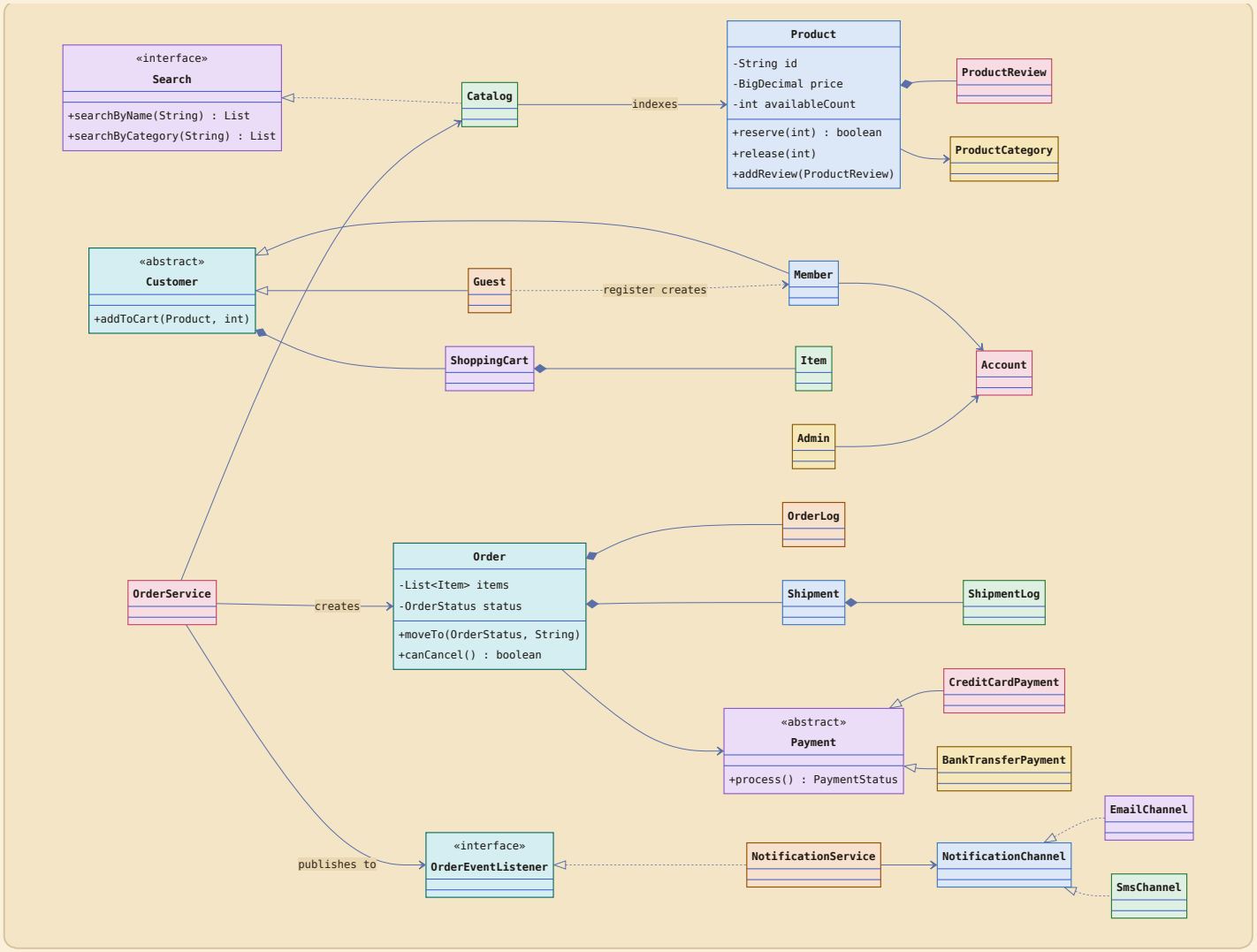


Figure — Stage 4: the cart owns its items, while the order holds its own copy of what was bought (course diagram)

Our Java design follows the same shape:



Key flows

Shopping activity. The course's activity diagram shows the loop a shopper goes round: search, view, add to cart, update the cart, "more shopping?", and finally checkout.

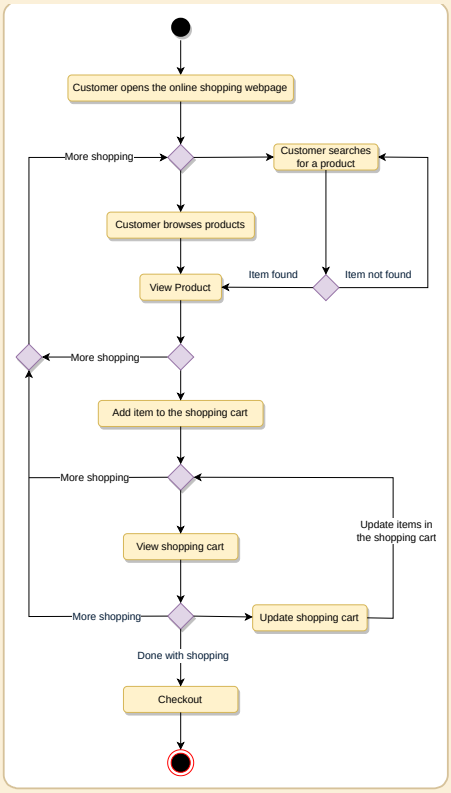


Figure — Activity diagram of a customer shopping online, from opening the site to checkout (course diagram)

Searching the catalog, and adding to the cart. The first two course sequence diagrams show the simple read path (customer → catalog → matching products) and the cart update path (customer → cart → item).

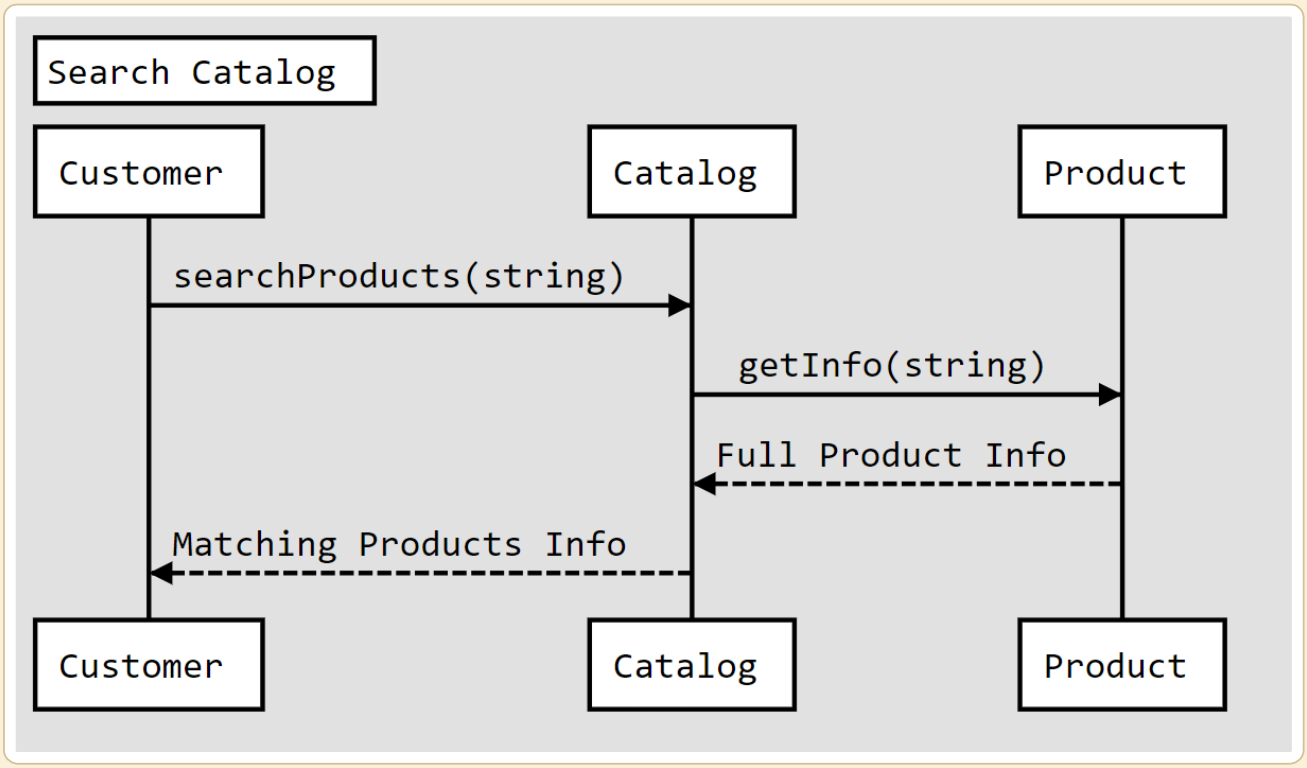


Figure — Sequence diagram for searching the catalog (course diagram)

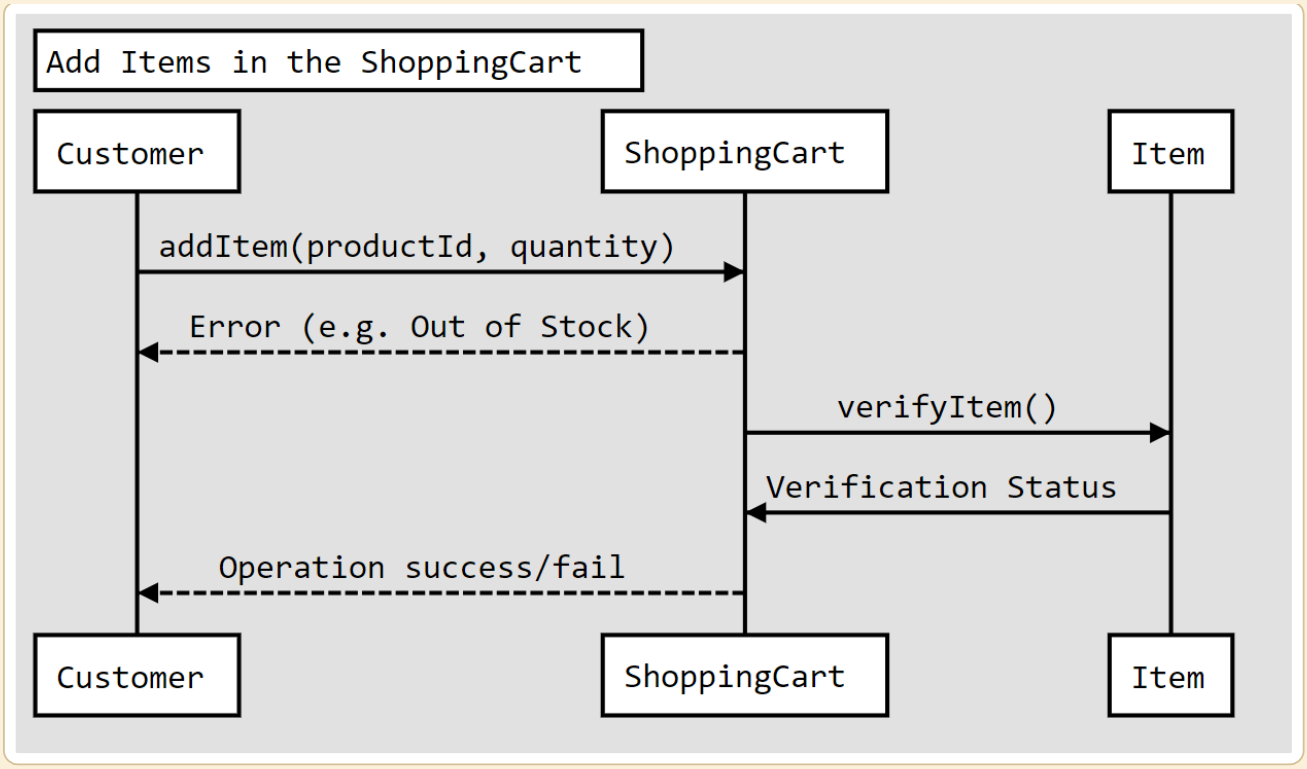


Figure — Sequence diagram for adding an item to the shopping cart (course diagram)

Checkout. The third course sequence diagram shows the important one: get the cart items, get billing and shipping info, create the order, verify every item, process the payment, create the shipment, and return the order status — with an error path after item verification and after payment.

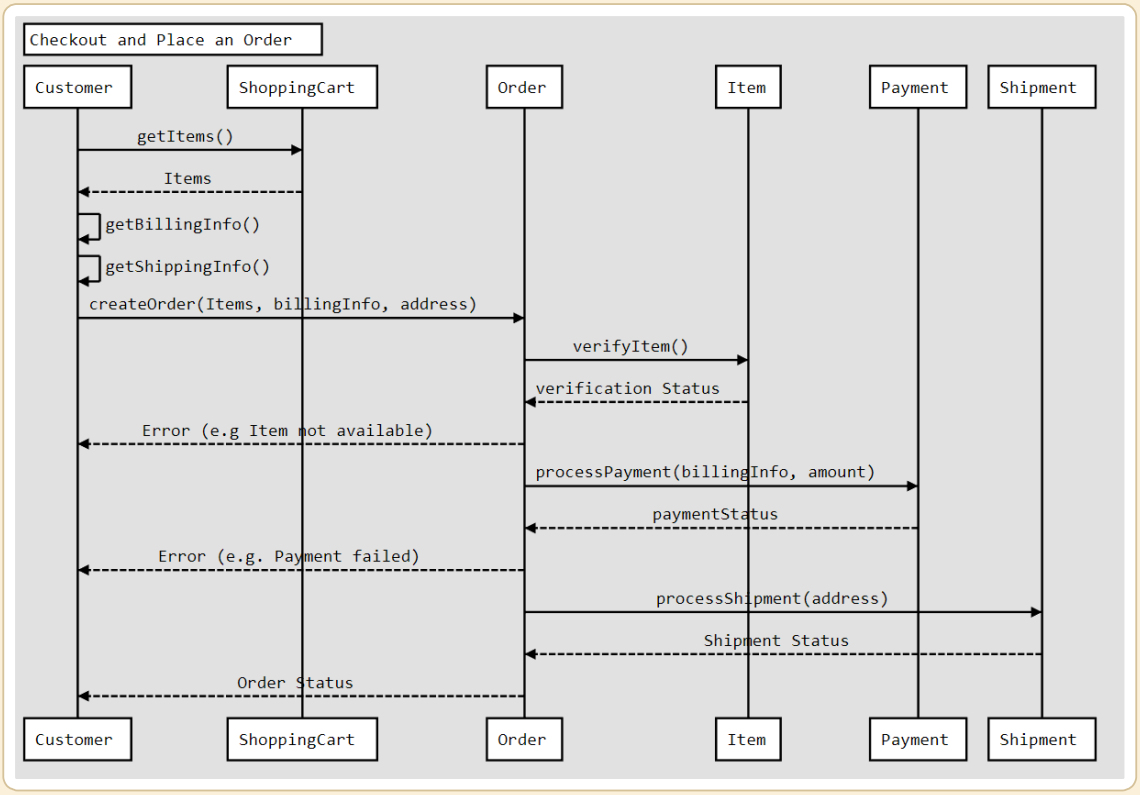
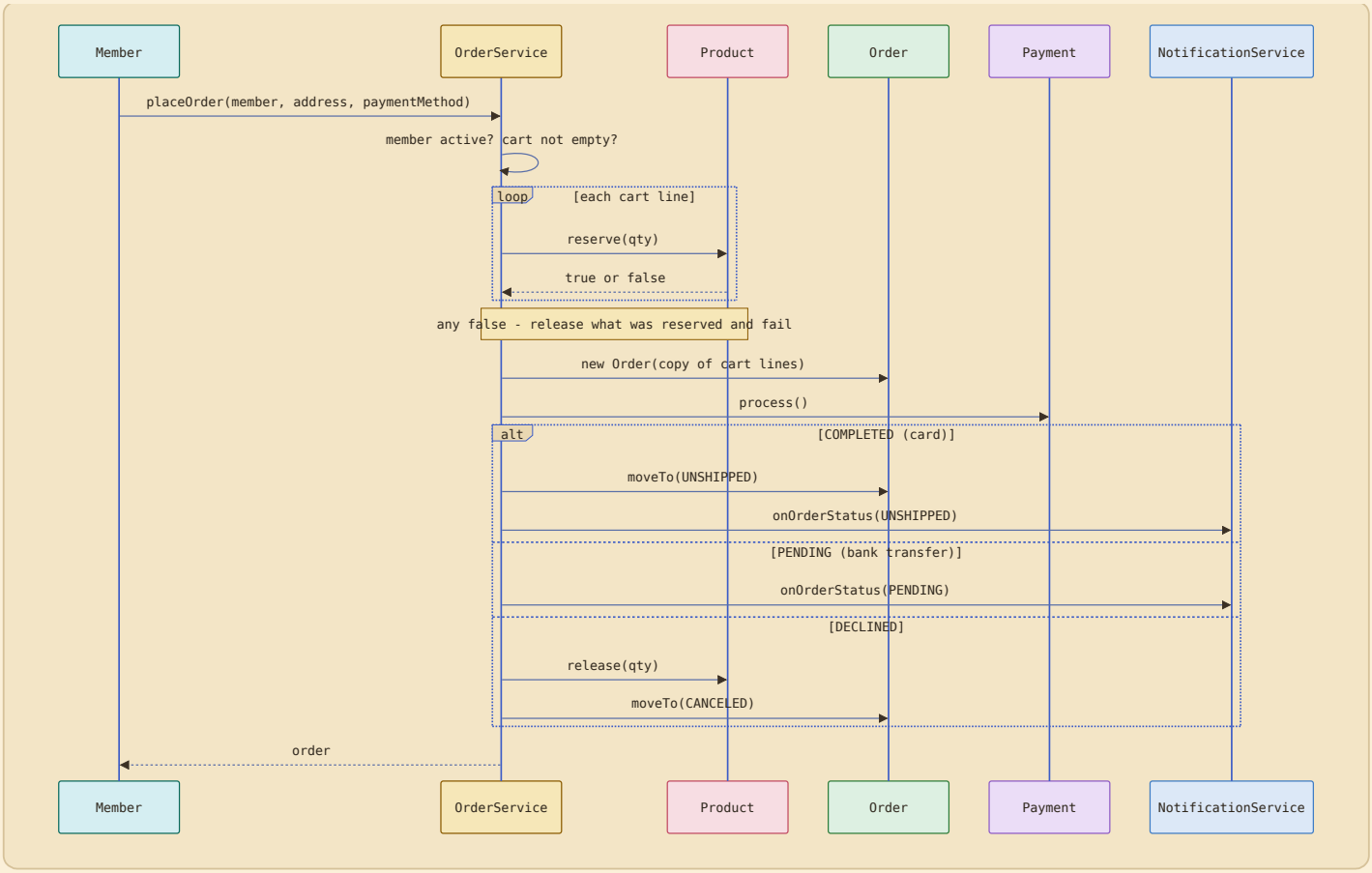
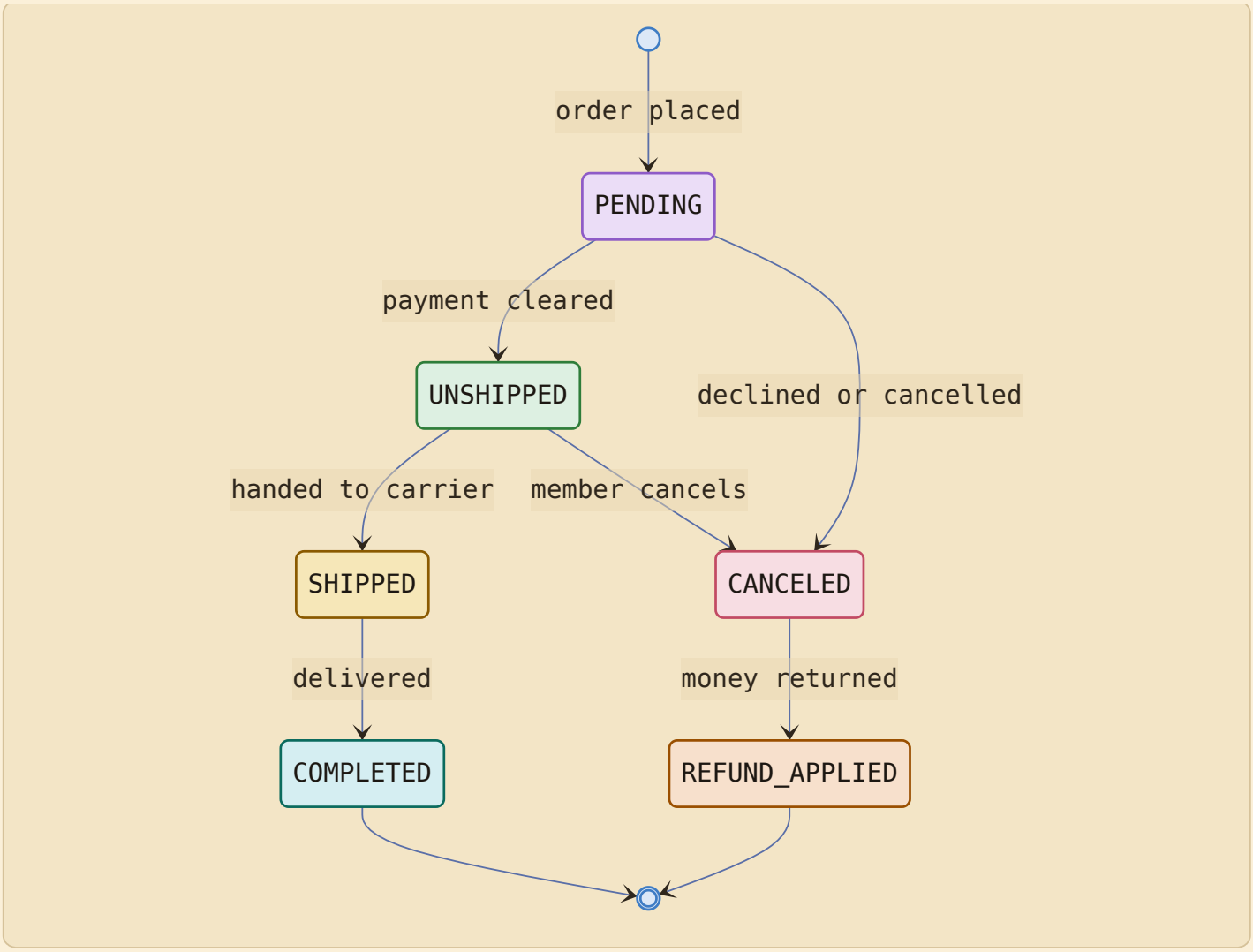


Figure — Sequence diagram for checkout: verify items, take payment, create shipment (course diagram)

Our `OrderService.placeOrder` follows that order, with one addition: stock is **reserved** during verification (not merely checked), so the check cannot go stale before payment.



The order state machine. Every legal move is listed in `OrderStatus.next()`. Anything else throws.



Requirement 8 ("cancel only if not shipped") is now one line: `canCancel()` returns `status.canMoveTo(CANCELED)`. There is no `SHIPPED → CANCELED` edge, so it is impossible by construction.

Design decisions & patterns used

Decision	Why	Alternative rejected
Ordering lives on `Member`, not `Customer`	A `Guest` *is a* `Customer`. If `Customer` had `placeOrder`, every guest would inherit an operation it has to reject — breaking the **Liskov Substitution Principle** (a subclass must be usable wherever its parent is, without surprises).	`Customer.placeOrder()` that throws for guests (the classic LSP smell).
`Guest.register()` returns a new `Member` and carries the cart across	Registration is a real-world moment where the cart must survive.	Mutating a guest into a member (Java objects cannot change class).
`Order` copies the cart lines	An order documents a *finished purchase*; it must stay fixed even if the cart is edited later.	Sharing `Item` objects — cheaper, but editing the cart would silently rewrite a finished order.
Stock reserved at order time, not at payment time	Prevents two shoppers from both getting the final unit. Cost: an abandoned order holds stock until cancelled; real systems release it on a timer.	Check stock, then pay, then decrement (a race window between check and decrement).
State machine inside `OrderStatus`	One place owns the rules. `moveTo` rejects illegal edges.	`if` checks scattered over `cancel`, `ship`, `deliver`.
`Payment` is an abstract (sealed) class, one subclass per method — a **Strategy**	The two methods *behave* differently (card clears now, transfer is pending). An **abstract class** cannot be instantiated itself — only its subclasses can — so every payment is a real card or transfer. Each subclass passes the substitution test (it can genuinely stand in for `Payment`: both really take money), and the same holds for the notification channels. A new method = a new subclass; checkout does not change (Open/Closed Principle: code should be open to extension but closed to modification). The course calls this out: it is why `Payment` is not an enum.	`enum PaymentType` + `switch` in checkout.
Payment passed in as `Function<BigDecimal, Payment>` (a function object: give it an amount, get a `Payment` back)	Checkout knows the total only after building the order, so it asks the caller's factory (a piece of code whose job is to create objects) for a payment of that amount.	Creating the payment before the total is known.
Observer for notifications	`OrderService` publishes events; `NotificationService` decides who to tell and how. Adding push notifications = a new `NotificationChannel`.	Calling email and SMS code directly inside `OrderService`.
`Shipment` keeps a log	Requirement 11 = read the last entry; the whole history stays for support and disputes.	A single `status` field (history lost).
Reviews store `reviewerId`	Avoids a two-way dependency `Product ↔ Member`; each class can be understood alone.	`ProductReview` holding a `Member` object.
`OrderService` as a Facade	One simple entry point hides stock, payment, shipment and notification steps.	Callers orchestrating five objects themselves.

Java implementation

Files: `Money.java` (BigDecimal helpers), `Catalog.java` (category, product, review, search, catalog), `Order.java` (address, item, cart, order status machine, order), `Payments.java` (payments, shipment), `Accounts.java` (account, customer, guest, member, admin), `OrderService.java` (listeners, notifications, service), `Main.java` (demo).

```java
// File: lld/ch19/amazon/Accounts.java
package lld.ch19.amazon;

import java.util.ArrayList;
import java.util.List;

enum AccountStatus { ACTIVE, BLOCKED, BANNED, COMPROMISED, ARCHIVED, UNKNOWN }

/** Login data shared by members and admins. */
final class Account {
    private final String userName;
    private final String email;
    private final String phone;
    private final List<Address> addresses = new ArrayList<>();
    private volatile AccountStatus status = AccountStatus.ACTIVE;

    Account(String userName, String email, String phone) {
        this.userName = userName;
        this.email = email;
        this.phone = phone;
    }

    String userName() { return userName; }
    String email() { return email; }
    String phone() { return phone; }
    AccountStatus status() { return status; }
    void setStatus(AccountStatus s) { status = s; }
    synchronized void addAddress(Address a) { addresses.add(a); }
    synchronized List<Address> addresses() { return List.copyOf(addresses); }
}

/**
 * Everything any shopper may do - browse and fill a cart. Placing an order is
 * deliberately NOT here, because a Guest is a Customer and must not inherit it (LSP).
 */
abstract sealed class Customer permits Guest, Member {
    private final ShoppingCart cart = new ShoppingCart();

    ShoppingCart cart() { return cart; }
    void addToCart(Product p, int qty) { cart.addItem(p, qty); }
    void removeFromCart(String productId) { cart.removeItem(productId); }
}

final class Guest extends Customer {
    /** Registering turns a guest into a member and carries the cart across. */
    Member register(String memberId, Account account) {
        Member m = new Member(memberId, account);
        m.cart().mergeFrom(cart());
        cart().clear();
        return m;
    }
}

final class Member extends Customer {
    private final String id;
    private final Account account;

    Member(String id, Account account) {
        this.id = id;
        this.account = account;
    }

    String id() { return id; }
    Account account() { return account; }
    boolean canBuy() { return account.status() == AccountStatus.ACTIVE; }
}

final class Admin {
```

```
    private final Account account;

    Admin(Account account) { this.account = account; }

    ProductCategory addCategory(String name, String description) { return new ProductCategory(name, description); }

    void block(Member m) { m.account().setStatus(AccountStatus.BLOCKED); }

    void unblock(Member m) { m.account().setStatus(AccountStatus.ACTIVE); }

    String name() { return account.userName(); }
}
```

```java
// File: lld/ch19/amazon/Catalog.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

/** A category such as "Books". Products point at it; it does not own them. */
record ProductCategory(String name, String description) { }

/** A review names its reviewer by id, so Product never depends on Member. */
record ProductReview(String reviewerId, int rating, String text) {
    ProductReview {
        if (rating < 1 || rating > 5) throw new IllegalArgumentException("rating must be 1..5");
    }
}

/** The thing being sold. Owns its reviews and guards its own stock count. */
final class Product {
    private final String id;
    private final String name;
    private final ProductCategory category;
    private final String sellerId;
    private volatile BigDecimal price;
    private int availableCount;                       // guarded by this
    private final List<ProductReview> reviews = new CopyOnWriteArrayList<>();

    Product(String id, String name, ProductCategory category, BigDecimal price,
            String sellerId, int availableCount) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.price = Money.of(price);
        this.sellerId = sellerId;
        this.availableCount = availableCount;
    }

    String id() { return id; }
    String name() { return name; }
    ProductCategory category() { return category; }
    String sellerId() { return sellerId; }
    BigDecimal price() { return price; }
    synchronized int availableCount() { return availableCount; }

    void updatePrice(BigDecimal newPrice) {
        if (newPrice.signum() < 0) throw new IllegalArgumentException("negative price");
        price = Money.of(newPrice);
    }

    /** Take stock out of the pool. False means "not enough" - nothing changes. */
    synchronized boolean reserve(int quantity) {
        if (quantity <= 0 || quantity > availableCount) return false;
        availableCount -= quantity;
        return true;
    }

    synchronized void release(int quantity) { availableCount += quantity; }

    void addReview(ProductReview review) { reviews.add(review); }
```

```
    double averageRating() {
        return reviews.stream().mapToInt(ProductReview::rating).average().orElse(0.0);
    }

    @Override public String toString() { return name + " (" + Money.fmt(price) + ")"; }
}

/** Requirement: search by name or by category. */
interface Search {
    List<Product> searchByName(String text);
    List<Product> searchByCategory(String categoryName);
}

/**
 * The catalog points to products but never owns them. It keeps one index per search
 * (words of the name, category name), so a lookup is a map read, not a scan of every
 * product. Every index is updated in one method (addProduct) so they can never drift apart.
 */
final class Catalog implements Search {
    private final Map<String, Product> byId = new ConcurrentHashMap<>();
    private final Map<String, Set<Product>> byNameWord = new ConcurrentHashMap<>();
    private final Map<String, List<Product>> byCategory = new ConcurrentHashMap<>();

    void addProduct(Product p) {
        byId.put(p.id(), p);
        for (String word : words(p.name()))
            byNameWord.computeIfAbsent(word, k -> ConcurrentHashMap.newKeySet()).add(p);
        byCategory.computeIfAbsent(key(p.category().name()), k -> new CopyOnWriteArrayList<>()).add(p);
    }

    Optional<Product> find(String productId) { return Optional.ofNullable(byId.get(productId)); }

    /** Products whose name contains every word of the query (e.g. "clean code"). */
    @Override public List<Product> searchByName(String text) {
        List<String> query = words(text);
        if (query.isEmpty()) return List.of();
        Set<Product> hits = new HashSet<>(byNameWord.getOrDefault(query.get(0), Set.of()));
        for (String w : query.subList(1, query.size())) hits.retainAll(byNameWord.getOrDefault(w, Set.of()));
        List<Product> out = new ArrayList<>(hits);
        out.sort((a, b) -> a.name().compareTo(b.name()));
        return out;
    }

    @Override public List<Product> searchByCategory(String categoryName) {
        return Collections.unmodifiableList(byCategory.getOrDefault(key(categoryName), List.of()));
    }

    private static List<String> words(String s) {
        return Arrays.stream(key(s).split("[^\\p{Alnum}]+")).filter(w -> !w.isEmpty()).toList();
    }

    private static String key(String s) { return s.toLowerCase(Locale.ROOT).trim(); }
}
```

```java
// File: lld/ch19/amazon/Main.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        Admin admin = new Admin(new Account("root", "admin@shop.test", "000"));
        ProductCategory books = admin.addCategory("Books", "Printed and e-books");
        ProductCategory electronics = admin.addCategory("Electronics", "Gadgets");

        Catalog catalog = new Catalog();
        catalog.addProduct(new Product("P1", "Clean Code", books, Money.of("32.50"), "seller-9", 5));
        catalog.addProduct(new Product("P2", "Refactoring", books, Money.of("41.00"), "seller-9", 2));
        catalog.addProduct(new Product("P3", "USB-C Cable", electronics, Money.of("9.99"), "seller-4", 100));
        catalog.addProduct(new Product("P4", "Design Patterns (last copy)", books, Money.of("55.00"), "seller-2",
1));

        System.out.println("== 1. Search ==");
        System.out.println("name 'code'      -> " + catalog.searchByName("code"));
        System.out.println("name 'usb cable' -> " + catalog.searchByName("usb cable"));
        System.out.println("category 'books' -> " + catalog.searchByCategory("books"));

        Map<String, Account> accounts = new ConcurrentHashMap<>();
        OrderService orders = new OrderService(catalog);
        orders.subscribe(new NotificationService(accounts::get, List.of(new EmailChannel(), new SmsChannel())));

        System.out.println("\n== 2. Guest fills a cart, then registers (cart carries over) ==");
        Guest guest = new Guest();
        guest.addToCart(catalog.find("P1").orElseThrow(), 1);
        guest.addToCart(catalog.find("P3").orElseThrow(), 3);
        Account aliceAcc = new Account("alice", "alice@mail.test", "+1-555-0101");
        Member alice = guest.register("M1", aliceAcc);
        accounts.put(alice.id(), aliceAcc);
        alice.cart().updateQuantity("P3", 2);
        alice.cart().snapshot().forEach(i -> System.out.println("  cart: " + i.quantity() + " x " + i.name()
                + " = " + Money.fmt(i.lineTotal())));

        System.out.println("\n== 3. Checkout with a credit card ==");
        Address home = new Address("1 Main St", "Seattle", "98101", "US");
        Order o1 = orders.placeOrder(alice, home,
                amt -> new CreditCardPayment(amt, "4242", Money.of("1000.00")));
        System.out.println("  " + o1.orderNumber() + " total " + Money.fmt(o1.total()) + ", status " + o1.status()
                + ", stock of Clean Code now " + catalog.find("P1").orElseThrow().availableCount());

        System.out.println("\n== 4. Ship, track, deliver ==");
        orders.ship(o1.orderNumber(), "UPS Ground");
        orders.updateShipment(o1.orderNumber(), ShipmentStatus.SHIPPED, "Portland hub");
        orders.updateShipment(o1.orderNumber(), ShipmentStatus.DELIVERED, "front door");
        System.out.println("  tracking now: " + o1.shipment().current() + ", order " + o1.status());
        System.out.println("  cancel after delivery? " + orders.cancel(o1.orderNumber()));

        System.out.println("\n== 5. Bank transfer (pending) then cancel before shipping -> refund ==");
        alice.addToCart(catalog.find("P2").orElseThrow(), 2);
        Order o2 = orders.placeOrder(alice, home, amt -> new BankTransferPayment(amt, "Chase"));
        System.out.println("  " + o2.orderNumber() + " status " + o2.status() + ", payment " +
o2.payment().status());
        orders.onPaymentSettled(o2.orderNumber());
        System.out.println("  cancel before shipping? " + orders.cancel(o2.orderNumber())
                + " -> " + o2.status() + ", payment " + o2.payment().status()
                + ", Refactoring stock back to " + catalog.find("P2").orElseThrow().availableCount());
        System.out.println("  order log: " + o2.log().stream().map(OrderLog::status).toList());
```

```
        System.out.println("\n== 6. Illegal transition is rejected by the state machine ==");
        try {
            orders.ship(o2.orderNumber(), "FedEx");
        } catch (IllegalStateException e) {
            System.out.println("  " + e.getMessage());
        }

        System.out.println("\n== 7. Review ==");
        Product cleanCode = catalog.find("P1").orElseThrow();
        cleanCode.addReview(new ProductReview(alice.id(), 5, "Changed how I name things"));
        cleanCode.addReview(new ProductReview("M7", 4, "Good, a bit long"));
        System.out.println("  Clean Code average rating = " + cleanCode.averageRating());

        System.out.println("\n== 8. Blocked member cannot buy ==");
        admin.block(alice);
        alice.addToCart(catalog.find("P3").orElseThrow(), 1);
        try {
            orders.placeOrder(alice, home, amt -> new CreditCardPayment(amt, "4242", Money.of("1000")));
        } catch (IllegalStateException e) {
            System.out.println("  rejected: " + e.getMessage());
        }
        admin.unblock(alice);
        alice.cart().clear();

        System.out.println("\n== 9. Ten members race for the last copy ==");
        OrderService quiet = new OrderService(catalog);          // no listeners: keep output short
        Product last = catalog.find("P4").orElseThrow();
        CountDownLatch start = new CountDownLatch(1);
        AtomicInteger winners = new AtomicInteger();
        AtomicInteger losers = new AtomicInteger();
        Thread[] threads = new Thread[10];
        for (int t = 0; t < threads.length; t++) {
            Member m = new Member("R" + t, new Account("racer" + t, "r" + t + "@mail.test", "-"));
            m.addToCart(last, 1);
            threads[t] = new Thread(() -> {
                try {
                    start.await();
                    quiet.placeOrder(m, home, amt -> new CreditCardPayment(amt, "1111", new BigDecimal("500")));
                    winners.incrementAndGet();
                } catch (IllegalStateException e) {
                    losers.incrementAndGet();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
            threads[t].start();
        }
        start.countDown();
        for (Thread t : threads) t.join();
        System.out.println("  winners=" + winners + " losers=" + losers + " stock left=" + last.availableCount());
    }
}
```

```java
// File: lld/ch19/amazon/Money.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Money helpers: always BigDecimal, always 2 decimal places, never double. */
final class Money {
    private Money() { }

    static final BigDecimal ZERO = of("0");

    static BigDecimal of(String amount) { return new BigDecimal(amount).setScale(2, RoundingMode.HALF_EVEN); }

    static BigDecimal of(BigDecimal amount) { return amount.setScale(2, RoundingMode.HALF_EVEN); }

    static String fmt(BigDecimal amount) { return "$" + amount.setScale(2, RoundingMode.HALF_EVEN).toPlainString();
}
}
```

```java
// File: lld/ch19/amazon/Order.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Collections;
import java.util.EnumSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;

record Address(String street, String city, String zip, String country) {
    @Override public String toString() { return street + ", " + city + " " + zip + ", " + country; }
}

/** One line of a cart or an order: an immutable snapshot of product, quantity and unit price. */
record Item(String productId, String name, int quantity, BigDecimal unitPrice) {
    Item {
        if (quantity <= 0) throw new IllegalArgumentException("quantity must be positive");
    }
    BigDecimal lineTotal() { return unitPrice.multiply(BigDecimal.valueOf(quantity)); }
}

/** A mutable working area. Owns its items (they die when the cart is cleared). */
final class ShoppingCart {
    private final Map<String, Item> items = new LinkedHashMap<>();

    synchronized void addItem(Product p, int qty) {
        items.merge(p.id(), new Item(p.id(), p.name(), qty, p.price()),
                (old, add) -> new Item(old.productId(), old.name(), old.quantity() + add.quantity(), p.price()));
    }

    synchronized void updateQuantity(String productId, int qty) {
        if (qty == 0) { items.remove(productId); return; }
        Item old = items.get(productId);
        if (old == null) throw new IllegalArgumentException("not in cart: " + productId);
        items.put(productId, new Item(old.productId(), old.name(), qty, old.unitPrice()));
    }

    synchronized void removeItem(String productId) { items.remove(productId); }

    /** A copy, so an order never shares its lines with the cart. */
    synchronized List<Item> snapshot() { return List.copyOf(items.values()); }

    synchronized boolean isEmpty() { return items.isEmpty(); }

    synchronized void clear() { items.clear(); }

    synchronized void mergeFrom(ShoppingCart other) {
        for (Item i : other.snapshot()) items.merge(i.productId(), i,
                (a, b) -> new Item(a.productId(), a.name(), a.quantity() + b.quantity(), a.unitPrice()));
    }
}

/** The order state machine. Each state lists the states it may move to. */
enum OrderStatus {
    PENDING, UNSHIPPED, SHIPPED, COMPLETED, CANCELED, REFUND_APPLIED;

    private Set<OrderStatus> next() {
        return switch (this) {
            case PENDING -> EnumSet.of(UNSHIPPED, CANCELED);      // waiting for payment
            case UNSHIPPED -> EnumSet.of(SHIPPED, CANCELED);      // paid, not yet handed to carrier
            case SHIPPED -> EnumSet.of(COMPLETED);                // too late to cancel
            case CANCELED -> EnumSet.of(REFUND_APPLIED);          // money goes back if it was taken
            case COMPLETED, REFUND_APPLIED -> EnumSet.noneOf(OrderStatus.class);
        };
    }
```

```
    boolean canMoveTo(OrderStatus target) { return next().contains(target); }
}

record OrderLog(OrderStatus status, String note) { }

/** A fixed copy of the purchased lines, plus a status that only moves along legal edges. */
final class Order {
    private final String orderNumber;
    private final String memberId;
    private final List<Item> items;             // copied from the cart, never shared
    private final Address shippingAddress;
    private final List<OrderLog> log = new ArrayList<>();
    private OrderStatus status = OrderStatus.PENDING;
    private Payment payment;
    private Shipment shipment;

    Order(String orderNumber, String memberId, List<Item> items, Address shippingAddress) {
        this.orderNumber = orderNumber;
        this.memberId = memberId;
        this.items = List.copyOf(items);
        this.shippingAddress = shippingAddress;
        log.add(new OrderLog(status, "order placed"));
    }

    String orderNumber() { return orderNumber; }
    String memberId() { return memberId; }
    List<Item> items() { return items; }
    Address shippingAddress() { return shippingAddress; }
    synchronized OrderStatus status() { return status; }
    synchronized Payment payment() { return payment; }
    synchronized Shipment shipment() { return shipment; }
    synchronized List<OrderLog> log() { return Collections.unmodifiableList(new ArrayList<>(log)); }

    BigDecimal total() {
        return items.stream().map(Item::lineTotal).reduce(Money.ZERO, BigDecimal::add);
    }

    synchronized void attachPayment(Payment p) { this.payment = p; }
    synchronized void attachShipment(Shipment s) { this.shipment = s; }

    /** Requirement 8 lives in exactly one place. */
    synchronized boolean canCancel() { return status.canMoveTo(OrderStatus.CANCELED); }

    synchronized void moveTo(OrderStatus target, String note) {
        if (!status.canMoveTo(target))
            throw new IllegalStateException(orderNumber + ": " + status + " -> " + target + " is not allowed");
        status = target;
        log.add(new OrderLog(target, note));
    }
}
```

```java
// File: lld/ch19/amazon/OrderService.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Function;

/** Observer: anything that wants to hear about order and shipment changes. */
interface OrderEventListener {
    void onOrderStatus(Order order, OrderStatus status);
    void onShipmentUpdate(Order order, ShipmentLog entry);
}

/** One way of delivering a message (email, SMS, push...). */
interface NotificationChannel {
    void send(Account to, String message);
}

final class EmailChannel implements NotificationChannel {
    @Override public void send(Account to, String message) {
        System.out.println("    [email -> " + to.email() + "] " + message);
    }
}

final class SmsChannel implements NotificationChannel {
    @Override public void send(Account to, String message) {
        System.out.println("    [sms   -> " + to.phone() + "] " + message);
    }
}

/** Requirement 9: tell the member whenever order or shipping status changes. */
final class NotificationService implements OrderEventListener {
    private final Function<String, Account> accountLookup;
    private final List<NotificationChannel> channels;

    NotificationService(Function<String, Account> accountLookup, List<NotificationChannel> channels) {
        this.accountLookup = accountLookup;
        this.channels = List.copyOf(channels);
    }

    @Override public void onOrderStatus(Order order, OrderStatus status) {
        Account to = accountLookup.apply(order.memberId());
        String msg = "Order " + order.orderNumber() + " is now " + status;
        channels.forEach(c -> c.send(to, msg));
    }

    @Override public void onShipmentUpdate(Order order, ShipmentLog entry) {
        Account to = accountLookup.apply(order.memberId());
        channels.get(0).send(to, "Shipment for " + order.orderNumber() + ": " + entry.status() + " @ " +
entry.location());
    }
}

/** Application service (facade) that turns a cart into an order and drives the order state machine. */
final class OrderService {
    private final Catalog catalog;
    private final Map<String, Order> orders = new ConcurrentHashMap<>();
    private final List<OrderEventListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger orderSeq = new AtomicInteger(1000);
    private final AtomicInteger shipSeq = new AtomicInteger(1);

    OrderService(Catalog catalog) { this.catalog = catalog; }
```

```
    void subscribe(OrderEventListener l) { listeners.add(l); }

    /**
     * Checkout. Stock is reserved when the order is placed (not when paid), all-or-nothing,
     * so two buyers can never both take the last copy.
     */
    Order placeOrder(Member member, Address address, Function<BigDecimal, Payment> paymentMethod) {
        if (!member.canBuy()) throw new IllegalStateException("account " + member.account().status());
        List<Item> items = member.cart().snapshot();
        if (items.isEmpty()) throw new IllegalStateException("cart is empty");

        List<Item> reserved = new ArrayList<>();
        for (Item i : items) {
            Product p = catalog.find(i.productId()).orElseThrow();
            if (!p.reserve(i.quantity())) {
                reserved.forEach(r -> catalog.find(r.productId()).orElseThrow().release(r.quantity()));
                throw new IllegalStateException("out of stock: " + i.name());
            }
            reserved.add(i);
        }

        Order order = new Order("ORD-" + orderSeq.incrementAndGet(), member.id(), items, address);
        orders.put(order.orderNumber(), order);

        Payment payment = paymentMethod.apply(order.total());
        order.attachPayment(payment);
        switch (payment.process()) {
            case COMPLETED -> { transition(order, OrderStatus.UNSHIPPED, "paid by " + payment.describe());
member.cart().clear(); }
            case PENDING -> { publish(order, OrderStatus.PENDING); member.cart().clear(); }
            default -> {                                  // declined: give the stock back, keep the cart
                releaseStock(order);
                transition(order, OrderStatus.CANCELED, "payment declined");
            }
        }
        return order;
    }

    /** Called when a bank transfer finally clears. */
    void onPaymentSettled(String orderNumber) {
        Order o = get(orderNumber);
        o.payment().markSettled();
        transition(o, OrderStatus.UNSHIPPED, "transfer cleared");
    }

    /** Requirement 8: cancel only if not shipped yet. */
    boolean cancel(String orderNumber) {
        Order o = get(orderNumber);
        synchronized (o) {                       // check-then-act must be atomic against ship()
            if (!o.canCancel()) return false;
            releaseStock(o);
            transition(o, OrderStatus.CANCELED, "cancelled by member");
        }
        if (o.payment().refund() == PaymentStatus.REFUNDED)
            transition(o, OrderStatus.REFUND_APPLIED, "refunded " + Money.fmt(o.total()));
        return true;
    }

    void ship(String orderNumber, String method) {
        Order o = get(orderNumber);
        Shipment s = new Shipment("SHP-" + shipSeq.getAndIncrement(), method);
        synchronized (o) {
            transition(o, OrderStatus.SHIPPED, "handed to " + method);
            o.attachShipment(s);
        }
        updateShipment(orderNumber, ShipmentStatus.SHIPPED, "sort centre");
    }

    void updateShipment(String orderNumber, ShipmentStatus status, String location) {
```

```
        Order o = get(orderNumber);
        o.shipment().addLog(status, location);
        ShipmentLog entry = o.shipment().current();
        listeners.forEach(l -> l.onShipmentUpdate(o, entry));
        if (status == ShipmentStatus.DELIVERED) transition(o, OrderStatus.COMPLETED, "delivered");
    }

    Order get(String orderNumber) {
        Order o = orders.get(orderNumber);
        if (o == null) throw new IllegalArgumentException("no order " + orderNumber);
        return o;
    }

    private void releaseStock(Order o) {
        o.items().forEach(i -> catalog.find(i.productId()).orElseThrow().release(i.quantity()));
    }

    private void transition(Order o, OrderStatus target, String note) {
        o.moveTo(target, note);
        publish(o, target);
    }

    private void publish(Order o, OrderStatus s) { listeners.forEach(l -> l.onOrderStatus(o, s)); }
}
```

```java
// File: lld/ch19/amazon/Payments.java
package lld.ch19.amazon;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

enum PaymentStatus { UNPAID, PENDING, COMPLETED, DECLINED, CANCELLED, REFUNDED }

/**
 * Strategy for taking money. Each subclass behaves differently: a card clears at
 * once, a bank transfer comes back PENDING and only clears later.
 */
abstract sealed class Payment permits CreditCardPayment, BankTransferPayment {
    private final BigDecimal amount;
    protected PaymentStatus status = PaymentStatus.UNPAID;

    Payment(BigDecimal amount) { this.amount = Money.of(amount); }

    BigDecimal amount() { return amount; }
    synchronized PaymentStatus status() { return status; }

    /** Try to take the money. Returns the new status. */
    abstract PaymentStatus process();

    abstract String describe();

    synchronized void markSettled() {
        if (status == PaymentStatus.PENDING) status = PaymentStatus.COMPLETED;
    }

    synchronized PaymentStatus refund() {
        if (status == PaymentStatus.COMPLETED) status = PaymentStatus.REFUNDED;
        else if (status == PaymentStatus.PENDING || status == PaymentStatus.UNPAID) status =
PaymentStatus.CANCELLED;
        return status;
    }
}

final class CreditCardPayment extends Payment {
    private final String last4;             // never store the full card number
    private final BigDecimal creditLimit;

    CreditCardPayment(BigDecimal amount, String last4, BigDecimal creditLimit) {
        super(amount);
        this.last4 = last4;
        this.creditLimit = creditLimit;
    }

    @Override synchronized PaymentStatus process() {
        status = amount().compareTo(creditLimit) <= 0 ? PaymentStatus.COMPLETED : PaymentStatus.DECLINED;
        return status;
    }

    @Override String describe() { return "card ****" + last4; }
}

final class BankTransferPayment extends Payment {
    private final String bankName;

    BankTransferPayment(BigDecimal amount, String bankName) {
        super(amount);
        this.bankName = bankName;
    }

    @Override synchronized PaymentStatus process() {
        status = PaymentStatus.PENDING;      // clears later, when the bank confirms
        return status;
```

```
    }

    @Override String describe() { return "bank transfer via " + bankName; }
}

enum ShipmentStatus { PENDING, SHIPPED, DELIVERED, ON_HOLD }

record ShipmentLog(ShipmentStatus status, String location) { }

/** Keeps a log, not one status field: the last entry is "now", the rest is history. */
final class Shipment {
    private final String shipmentNumber;
    private final String method;
    private final List<ShipmentLog> log = new ArrayList<>();

    Shipment(String shipmentNumber, String method) {
        this.shipmentNumber = shipmentNumber;
        this.method = method;
        log.add(new ShipmentLog(ShipmentStatus.PENDING, "warehouse"));
    }

    String shipmentNumber() { return shipmentNumber; }
    String method() { return method; }
    synchronized void addLog(ShipmentStatus s, String where) { log.add(new ShipmentLog(s, where)); }
    synchronized ShipmentLog current() { return log.get(log.size() - 1); }
    synchronized List<ShipmentLog> history() { return List.copyOf(log); }
}
```

Output:

```
== 1. Search ==
name 'code'      -> [Clean Code ($32.50)]
name 'usb cable' -> [USB-C Cable ($9.99)]
category 'books' -> [Clean Code ($32.50), Refactoring ($41.00), Design Patterns (last copy) ($55.00)]

== 2. Guest fills a cart, then registers (cart carries over) ==
  cart: 1 x Clean Code = $32.50
  cart: 2 x USB-C Cable = $19.98

== 3. Checkout with a credit card ==
    [email -> alice@mail.test] Order ORD-1001 is now UNSHIPPED
    [sms   -> +1-555-0101] Order ORD-1001 is now UNSHIPPED
  ORD-1001 total $52.48, status UNSHIPPED, stock of Clean Code now 4

== 4. Ship, track, deliver ==
    [email -> alice@mail.test] Order ORD-1001 is now SHIPPED
    [sms   -> +1-555-0101] Order ORD-1001 is now SHIPPED
    [email -> alice@mail.test] Shipment for ORD-1001: SHIPPED @ sort centre
    [email -> alice@mail.test] Shipment for ORD-1001: SHIPPED @ Portland hub
    [email -> alice@mail.test] Shipment for ORD-1001: DELIVERED @ front door
    [email -> alice@mail.test] Order ORD-1001 is now COMPLETED
    [sms   -> +1-555-0101] Order ORD-1001 is now COMPLETED
  tracking now: ShipmentLog[status=DELIVERED, location=front door], order COMPLETED
  cancel after delivery? false

== 5. Bank transfer (pending) then cancel before shipping -> refund ==
    [email -> alice@mail.test] Order ORD-1002 is now PENDING
    [sms   -> +1-555-0101] Order ORD-1002 is now PENDING
  ORD-1002 status PENDING, payment PENDING
    [email -> alice@mail.test] Order ORD-1002 is now UNSHIPPED
    [sms   -> +1-555-0101] Order ORD-1002 is now UNSHIPPED
    [email -> alice@mail.test] Order ORD-1002 is now CANCELED
    [sms   -> +1-555-0101] Order ORD-1002 is now CANCELED
    [email -> alice@mail.test] Order ORD-1002 is now REFUND_APPLIED
    [sms   -> +1-555-0101] Order ORD-1002 is now REFUND_APPLIED
  cancel before shipping? true -> REFUND_APPLIED, payment REFUNDED, Refactoring stock back to 2
  order log: [PENDING, UNSHIPPED, CANCELED, REFUND_APPLIED]

== 6. Illegal transition is rejected by the state machine ==
  ORD-1002: REFUND_APPLIED -> SHIPPED is not allowed

== 7. Review ==
  Clean Code average rating = 4.5

== 8. Blocked member cannot buy ==
  rejected: account BLOCKED

== 9. Ten members race for the last copy ==
  winners=1 losers=9 stock left=0
```

Reading the output. Step 1 answers both searches from indexes (a name search matches every query word, so `usb cable` finds "USB-C Cable"). Step 2 shows the guest's cart surviving registration. Step 3 prints two notifications (email + SMS) because both channels are registered. Step 5 walks the full cancel path `PENDING → UNSHIPPED → CANCELED → REFUND_APPLIED`, and the stock returns to 2. Step 6 proves the state machine refuses `REFUND_APPLIED → SHIPPED`. Step 9 starts ten threads at the same instant (a `CountDownLatch` — a gate that keeps threads waiting until a counter reaches zero — releases them together) and exactly one wins the last copy.

Concurrency & edge cases

Risk	What could go wrong	How the code prevents it
Last unit, two buyers	Both see `availableCount == 1`, both decrement → stock −1	`Product.reserve` is `synchronized`: check and decrement are one step. Step 9 proves it.
Multi-line cart, one line out of stock	Lines 1–2 reserved, line 3 fails → stock leaked	`placeOrder` remembers what it reserved and releases it all on failure (all-or-nothing).
Cancel and ship at the same time	Order shipped *and* stock released	`cancel` and `ship` both lock the `Order` object around check-then-move.
Payment declined	Stock stays reserved forever	Declined path releases stock and moves the order to `CANCELED`; the cart is kept so the member can retry.
Bank transfer never clears	Stock held forever	Mention a timeout job that cancels `PENDING` orders after N hours.
Price changes after adding to cart	Customer charged a price they never saw	`Item` stores the unit price at add time; production systems re-validate prices at checkout and show changes.
Blocked member	Places orders anyway	`Member.canBuy()` checks `AccountStatus.ACTIVE` first (step 8).
Deadlock (two threads each waiting forever for a lock the other holds)	Two locks taken in different orders	Each lock (product, order, cart) is held briefly and never while waiting for another object's lock in the opposite order.

Real-world example

On Amazon's "Your Orders" page, the *Cancel items* button is offered only while an order has not yet entered the shipping process — the same edge our state machine removes after `SHIPPED`. Amazon also treated the shopping cart as something that must *always* accept writes: its 2007 Dynamo paper (the key-value store — a database that simply maps a key to a value — behind services such as the cart) chose **availability** (staying usable even when some machines fail; see System Design Fundamentals) so that "add to cart" works even during failures, and merges conflicting cart versions later. Our in-memory design shows the object model; at Amazon's scale the cart, catalog and orders live in separate services.

Extensions & follow-up questions

- **"Add a new payment type, e.g. gift cards or UPI (India's instant bank-to-bank mobile payment system)."** Add a `GiftCardPayment extends Payment` (and list it in `permits`). `OrderService` is untouched, because it only calls `process()`.
- **"How do you prevent two shoppers from both buying the final unit?"** Reserve at order time inside a lock (`Product.reserve`). In a distributed system (one spread over many machines): a conditional database update (`UPDATE ... SET stock = stock - 1 WHERE id = ? AND stock >= 1`) or **optimistic locking** with a version column (no lock is held; each row carries a version number, and an update succeeds only if the version is still the one you read — otherwise you retry).
- **"Abandoned orders hold stock."** Give each reservation an expiry. A scheduled job cancels `PENDING` orders older than, say, 30 minutes and releases stock.
- **"Returns after delivery."** Add `COMPLETED → RETURN_REQUESTED → RETURNED → REFUND_APPLIED` edges; the enum change is local.
- **"Multiple warehouses."** Replace `Product.availableCount` with an `Inventory` per warehouse; checkout picks a warehouse per line (a Strategy: nearest, cheapest shipping).
- **"Split shipments."** One `Order` owns many `Shipment`s, each with its own item subset and log.
- **"Search at scale."** The in-memory `Catalog` becomes a search index (for example Elasticsearch/OpenSearch, popular search-engine servers) fed by product change events; the `Search` interface stays.
- **"Seller payouts, recommendations, wish lists."** Separate modules listening to order events (Observer again).

In the interview

30–45 minute plan

1. (3 min) Clarify: guest vs member, payment types, cancel rule, notifications, what is out of scope.
2. (5 min) Actors and use cases; name the four actors.

3. (10 min) Class diagram. **Say the LSP sentence out loud**: "Ordering is on `Member`, not `Customer`, otherwise every guest inherits an ordering method it is not allowed to use."
4. (5 min) The order state machine; show that "cancel after ship" has no edge.
5. (10 min) Code `placeOrder` (reserve all-or-nothing, copy items, pay, notify) and `cancel`.
6. (5 min) Concurrency: last-item race; abandoned-order timeout; extensions.

> **30-second answer:** "Catalog with name and category indexes; a cart per customer; ordering only on `Member`; checkout reserves stock for every line atomically, copies the lines into an immutable order, then takes payment through a `Payment` strategy. The order moves through a state machine that forbids cancelling after shipping, and every status change is published to a notification observer."

Follow-ups with answers

- *Why is `Payment` abstract rather than an enum?* Because the methods behave differently (clears now vs pending). New methods are new classes — Open/Closed.
- *Why does `Order` copy items?* So later cart edits or price changes cannot rewrite history.
- *Where would you put tax and discounts?* A `PricingRule` chain (Chain of Responsibility — a pattern where a request is passed along a line of handlers, each doing its part — or a list of Strategies) applied when building the order; the order stores the final line prices.
- *How do you make checkout idempotent (safe to repeat: doing it twice has the same effect as doing it once) if the user double-clicks?* Accept a client-generated idempotency key (a unique id the browser sends with the request); if an order with that key exists, return it instead of creating a second one.

Self-check

1. **Q:** Why can't `placeOrder` live on `Customer`? **A:** `Guest extends Customer` and guests must not order; inheriting a method you must refuse violates LSP.
2. **Q:** What does the filled diamond between `Order` and `OrderLog` mean? **A:** Composition — the order owns its log entries; they are created and destroyed with it.
3. **Q:** Name two things wrong with "check stock, take payment, then decrement stock". **A:** A race window lets two buyers pass the check; and a payment can succeed for goods that are no longer there.
4. **Q:** Which status edges allow a cancel? **A:** `PENDING → CANCELED` and `UNSHIPPED → CANCELED`.
5. **Q:** Why `compareTo` and not `equals` on `BigDecimal`? **A:** `equals` also compares scale, so `2.0` and `2.00` are "not equal".
6. **Q:** What pattern lets you add push notifications without touching `OrderService`? **A:** Observer (listener) plus a new `NotificationChannel` implementation.

TL;DR

- Guests browse and fill carts; **only members order** (LSP).
- Checkout = **reserve all lines atomically → copy into an immutable order → pay (Strategy) → publish (Observer)**.
- `OrderStatus` is a **state machine**; "cancel after ship" is impossible because the edge does not exist.
- Money in `BigDecimal`; histories in logs; stock guarded by a lock.

Design an Online Stock Brokerage System

This problem has two halves, and interviews ask both:

- **Part A — the brokerage** (what the course designs): members, money in and out, watchlists, the four order types, lots and positions, statements.
- **Part B — the exchange's matching engine** (a machine-coding problem — write a working program in about 90 minutes — originally from Geektrust, an Indian coding-assessment platform, kept in kumaransg/LLD under a project named "Navi"; we call it "the Navi statement" below): an **order book** (the list of all waiting buy and sell orders for one stock) that matches buy and sell orders by **price-time priority**, implemented and tested.

A **broker** is the company you trade *through* (it holds your cash and shares and forwards your orders). An **exchange** is the market where buy and sell orders from all brokers *meet* and are matched. Keep the two apart in your head;

the design keeps them apart in code.

Problem & clarifying questions

An online brokerage lets people buy and sell stocks without a human middleman. Members track and carry out their transactions, see charts of how the stocks they hold are performing, and get an alert when a stock moves by an amount they chose. The platform must keep those transactions secure. Because computers replace the phone call to a broker, each trade is quicker and cheaper, and members get fast access to real-time (updated the moment they change) prices, market trends and stock reports.

Question	Assumed answer
Which order types?	Market, limit, stop-loss, stop-limit (explained in the table under Key flows).
Can one order fill at several prices?	Yes — an order can be filled in parts.
Do we track separate purchases of the same stock?	Yes — each purchase is a **lot**, because tax depends on which lot you sell.
How does money come in and out?	Check, wire (a direct bank-to-bank transfer), or electronic bank transfer, each with a per-transfer limit.
Reports?	Quarterly statements and a yearly tax statement.
Notifications?	When an order executes.
Margin, short selling, options, dividends and stock splits, real-time streaming quotes, the settlement delay, the wash-sale tax rule?	Out of scope — a deliberate decision, say so.

For reference, those out-of-scope terms: **margin** is trading with money borrowed from the broker; **short selling** is selling borrowed shares, hoping to buy them back cheaper; **options** are contracts giving the right to buy or sell later at a fixed price; **dividends** are profits a company pays to its shareholders; a **stock split** turns each share into several cheaper ones; **settlement** is the actual swap of cash and shares, which happens a day or so after the trade; the **wash-sale rule** is a US tax rule that blocks claiming a loss if you rebuy the same stock within 30 days.

Requirements

Functional (brokerage)

1. Any member can buy and sell stocks.
2. A member can have **multiple watchlists** (named lists of stocks to keep an eye on), each with multiple stock quotes (a quote is a stock's latest price).
3. Orders can be **market, limit, stop-loss, stop-limit**.
4. A member can hold **multiple lots** of the same stock; the system keeps them apart.
5. The system generates **quarterly** updates and **yearly tax** statements.
6. Money goes in and out of the account in three ways: **check, wire, or electronic bank transfer**.
7. A notification is sent whenever an order executes.

Functional (matching engine, Navi variant)

8. Orders arrive as `<id> <time> <symbol> <buy/sell> <price> <qty>`, e.g. `#1 09:45 BAC sell 240.12 100`. They are read from a text file whose path is given as the program's argument; trades are printed to the console.
9. On every new order, match against the opposite side. Lower sell prices and higher buy prices go first; at the same price, **earlier orders go first** (price-time priority).
10. A trade happens when buy price ≥ sell price. In the Navi statement the trade is recorded at the **sell order's price**.
11. Print each trade as `<buy_id> <price> <qty> <sell_id>`; unmatched quantity stays in the book.

Non-functional

- Money is never over-committed: ten $1,000 orders cannot all pass a $1,000 balance check.
- Matching is deterministic (no randomness or timing luck): same input, same trades.
- The engine is safe under concurrent submissions and conserves quantity (shares bought = shares sold).

Use cases

Three actors:

- **Admin** — administrative jobs such as blocking or unblocking members.
- **Member** — searches the stock inventory, buys and sells, manages watchlists.
- **System** — sends order notifications and periodically fetches quotes from the exchange.

Top use cases: register / cancel membership; add, remove or edit a watchlist; search stocks by symbol; place an order; cancel an order; deposit or withdraw money.

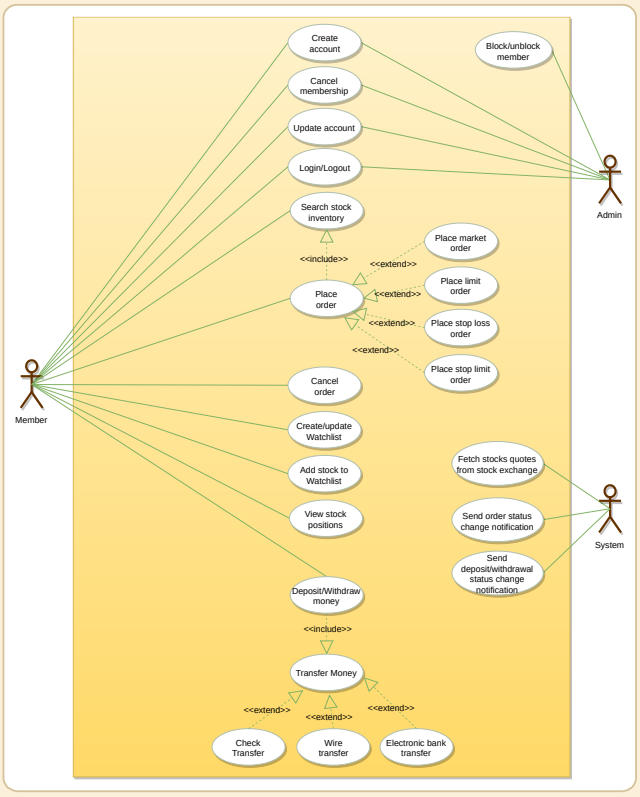


Figure — Use-case diagram for the stock brokerage: Admin, Member and System (course diagram)

Core entities

Class	Responsibility
`MoneyTransfer` → `CheckTransfer`, `WireTransfer`, `ElectronicBankTransfer`	Move money in or out. They differ only in name and clearing days (5 / 1 / 3 in our code); the per-transfer limit (100,000) is a single rule, so it lives only in the base class.
`Notifier` / `EmailNotifier`	Delivers "your order filled" messages. The course draws one base with two delivery ways (email and SMS); an `SmsNotifier` would be a second implementation.
`Order` (abstract)	Shared by all order types: member, symbol, side (buy or sell), quantity, time-in-force (how long the order stays live), status, list of fills. Declares the **two questions**.
`MarketOrder`, `LimitOrder`, `StopLossOrder`, `StopLimitOrder`	Each answers "may I trade now?" and "what is my worst price?". `StopLimitOrder extends StopLossOrder`.
`OrderPart`	One fill: quantity, price, day. A 100-share order might be filled as 70 @ 10.00 plus 30 @ 10.05.
`Stock`, `StockInventory`	Symbol + latest price; the inventory is the **single source of truth** (the one place whose value counts if copies disagree) for prices.
`StockExchange`	Holds open orders; on each new order or price tick tries to fill them; calls back through `OrderListener`.
`OrderListener`	Interface the exchange calls (`onFill`, `onCancelled`). The exchange never learns what a member is.
`StockLot`	One purchase at one price.
`StockPosition`	All lots of one symbol; sells from the oldest lot first (FIFO — first in, first out) unless a lot is named. Returns a `LotSale` per lot touched.
`LotSale`	Shares taken from one lot, buy price, sell price → realised gain (profit actually locked in by selling).
`Watchlist`	A named set of **symbols** (not `Stock` objects, so it never goes stale).
`Trade`	One fill as the member sees it (sells carry their `LotSale`s).
`Statement` → `QuarterlyStatement`, `TaxStatement`	Base class filters trades by day range; subclasses write their own lines.
`Member`	Cash, **reserved funds**, positions, watchlists, trades. Implements `OrderListener`. (The course's `Account` also stores name, address, e-mail and phone; we keep only what the demo uses.)
`Admin`	Blocks / unblocks members.
(engine) `OrderBook`, `MatchingEngine`, `Trade`, `ExecutionPricePolicy`	Price levels with FIFO queues per symbol; routing and locking; who sets the trade price.

Class diagram

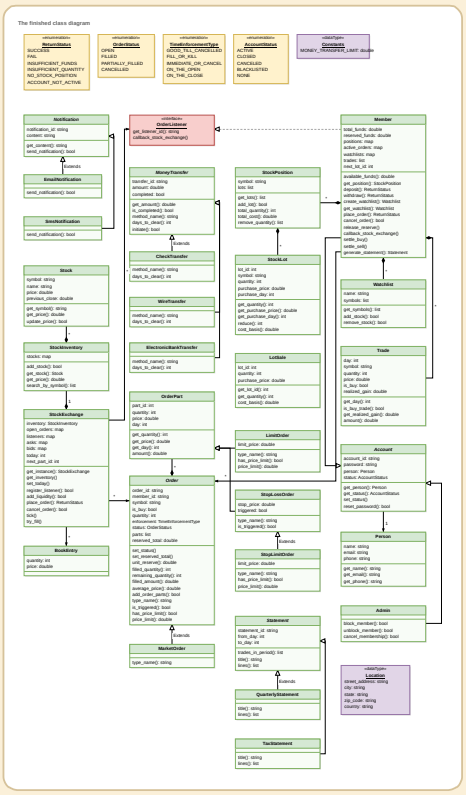


Figure — The finished class diagram of the online stock brokerage (course diagram)

The course builds this model in six stages: money transfers and notifications → the order hierarchy → stock, inventory and exchange → lots and positions → watchlists, trades and statements → members and admins. In the figure above, `Member` extends `Account` and implements `OrderListener` (dashed arrow): the exchange reports fills through that interface and never needs the `Member` type. The filled diamonds from `Member` mean each position, watchlist and trade is owned by exactly one member; the plain arrow to `Order` says the exchange shares those orders.

The course singles out stage 2 — the order hierarchy — as the part that earns the most credit in an interview:

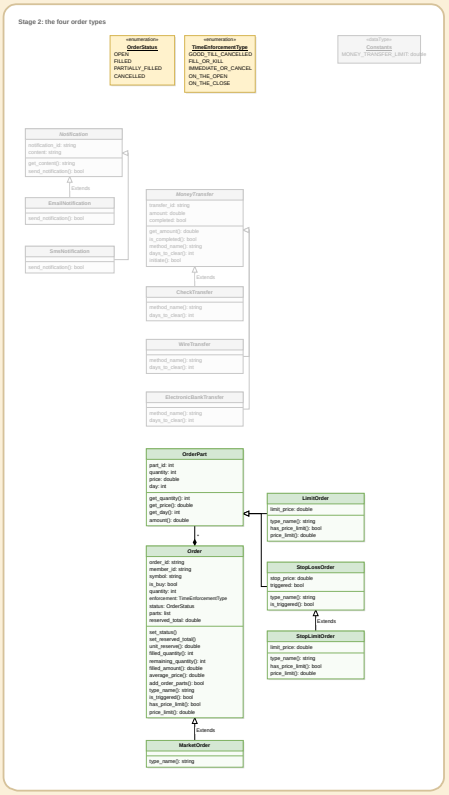
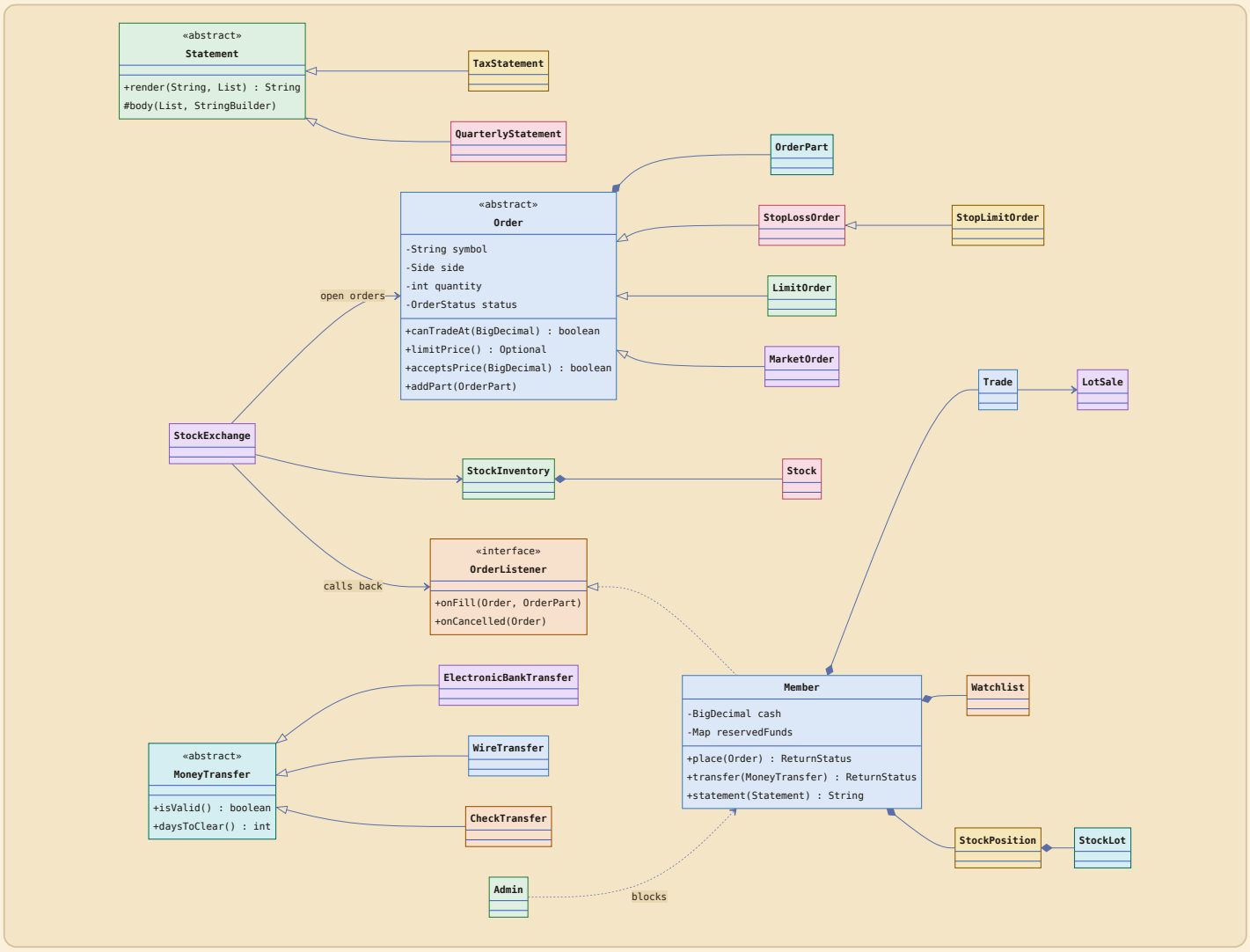
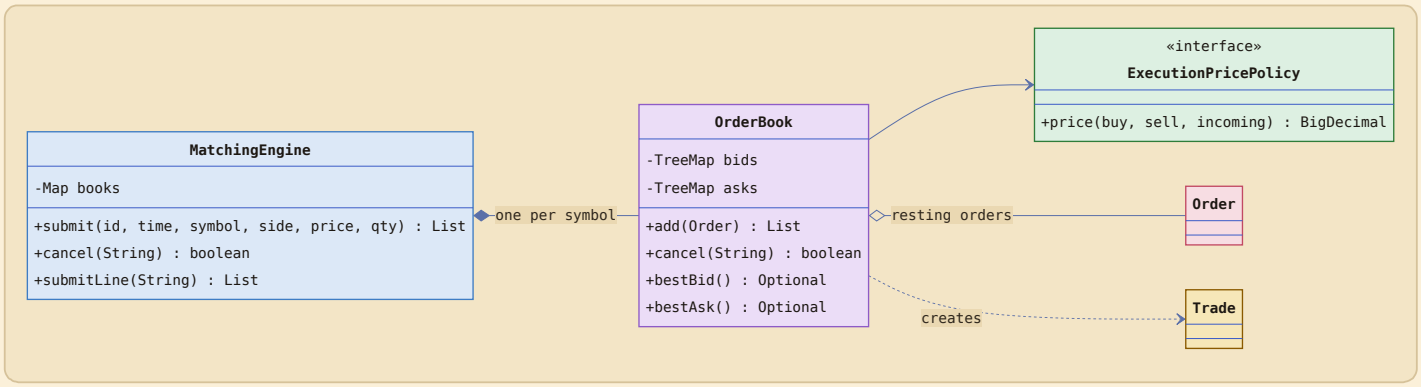


Figure — Stage 2: one abstract Order and four small subclasses that differ only in two answers (course diagram)

Our brokerage code:



And the matching engine (Part B):



Key flows

Placing a buy order. The course's activity diagram: pick a stock, pick the order type (market, limit, stop-loss, stop-limit), enter quantity, price limit and time enforcement; the system checks there is enough money, then deducts it (we *reserve* it) and sends the order to the exchange; when the exchange acknowledges, the member sees "order placed". The member can then update the order.

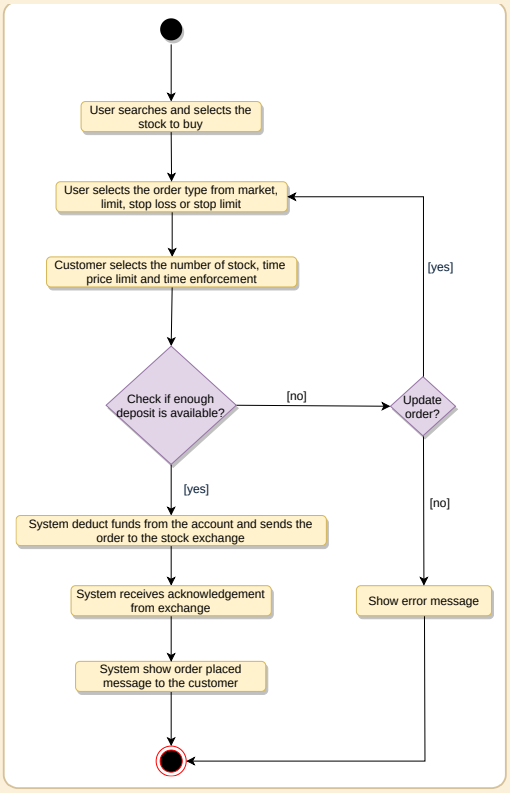


Figure — Activity diagram for placing a buy order (course diagram)

Placing a sell order is the mirror image, with one extra step: the member picks **which lots** to sell, and the check is "do I own enough shares?" rather than "do I have enough money?".

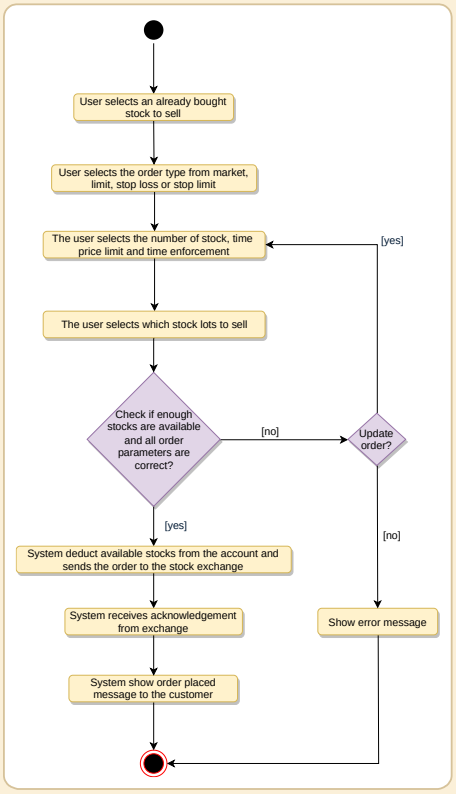


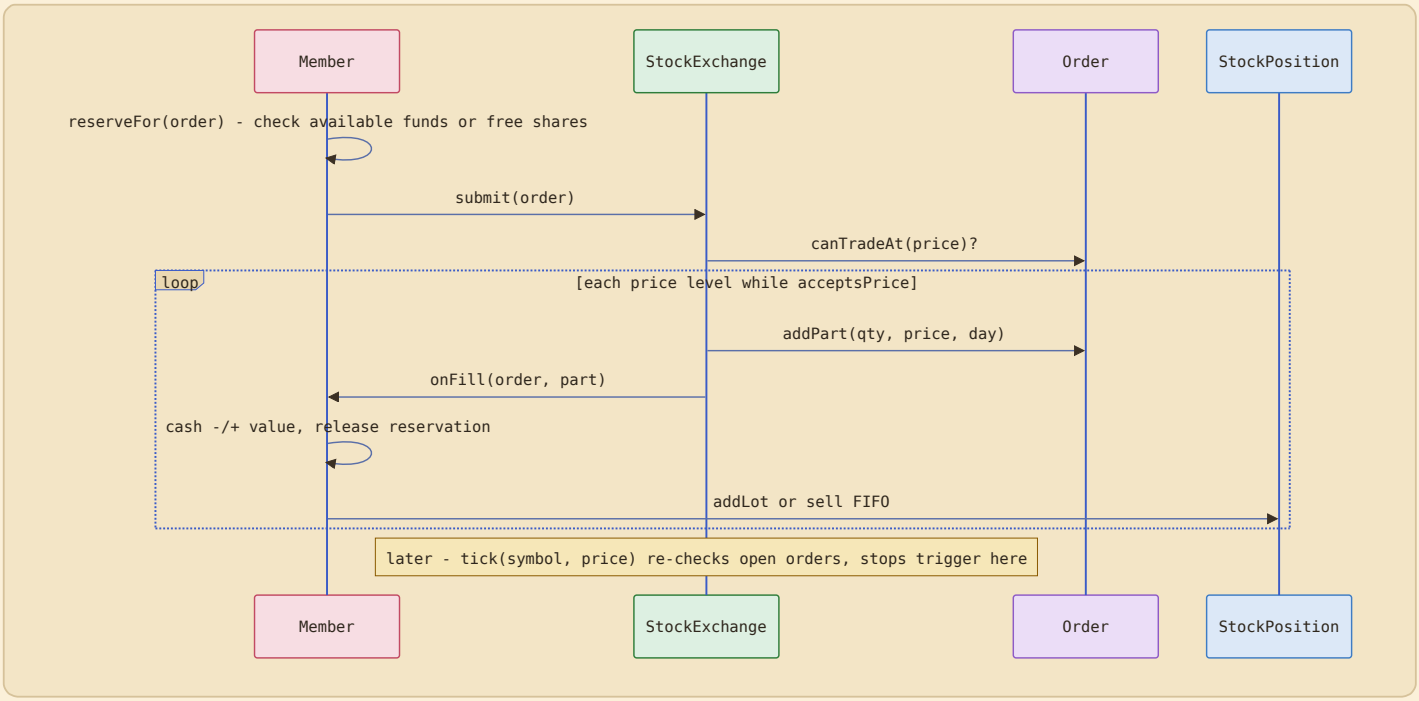
Figure — Activity diagram for placing a sell order, including choosing lots (course diagram)

The four order types in one table. They share everything except two answers:

Type	Q1: may it trade at this market price?	Q2: worst acceptable price	Plain words
Market	Always	None	"Buy/sell now at whatever the market gives."
Limit	Always	The limit	"Buy at 9.50 or less / sell at 10.90 or more."
Stop-loss (stop)	Only after the price **reaches the stop**, then forever	None (becomes a market order)	"If it falls to 10.00, get me out."
Stop-limit	Only after the stop is reached	The limit	"If it rises to 305, buy — but pay no more than 306."

A SELL stop triggers when price ≤ stop; a BUY stop triggers when price ≥ stop. Once triggered it stays triggered (the code "latches" a boolean).

Fill → money and lots move (Observer callback).

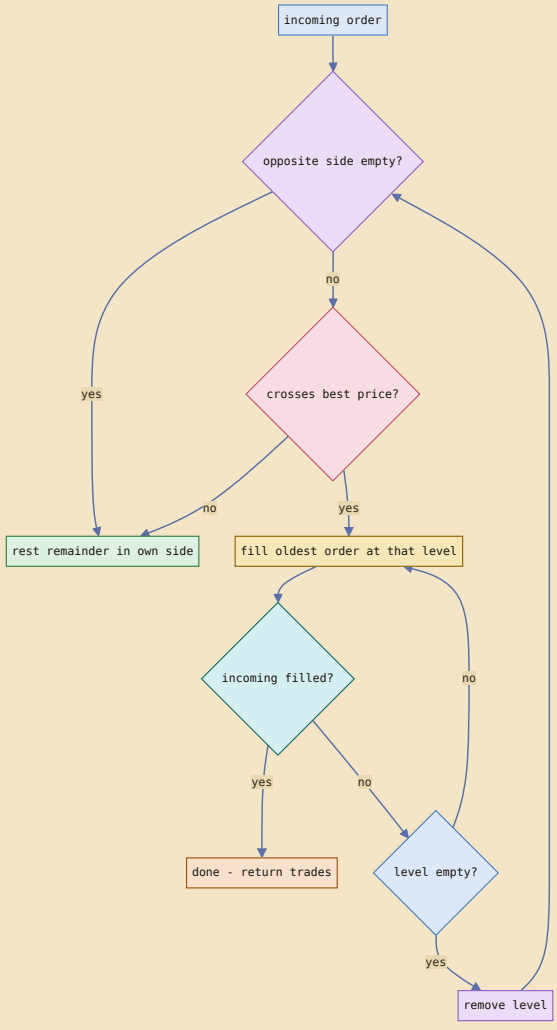


Matching engine: price-time priority. Each symbol has an `OrderBook` with two sides. Each side is a `TreeMap` (a map that keeps its keys sorted) from price to a FIFO queue (`ArrayDeque`, Java's fast double-ended queue). **Bids** (buy orders) are sorted highest-first, **asks** (sell orders) lowest-first. For an incoming order:

1. Look at the best price on the opposite side.
2. If it does not **cross** (buy price < best ask, or sell price > best bid), stop.
3. Otherwise fill against that level's queue, **oldest order first**, for `min(incoming remaining, resting remaining)`.
4. Remove fully filled resting orders; drop empty price levels; repeat from step 1.
5. If the incoming order still has quantity, add it to the back of its own price level's queue.

Walk-through of the sample input:

Order	Book before (asks / bids)	What happens
#1 sell 240.12 × 100	empty	rests on asks
#2 sell 237.45 × 90	asks 240.12×100	rests; best ask now 237.45
#3 buy 238.10 × 110	asks 237.45×90, 240.12×100	crosses 237.45 → trade `#3 237.45 90 #2`; 20 left; 240.12 > 238.10 so stop; rests 20 @ 238.10
#4 buy 237.80 × 10	best ask 240.12	no cross; rests
#5 buy 237.80 × 40	same	no cross; rests **behind #4** at 237.80
#6 sell 236.00 × 50	bids 238.10×20, 237.80×(10, 40)	fills #3 for 20, then #4 for 10, then #5 for 20 — all at 236.00 (the sell price)



Design decisions & patterns used

Decision	Why	Alternative rejected
One abstract `Order` + two abstract questions	The four types differ only in "may I trade?" and "worst price?". Each subclass is a few lines. The course stresses this is just a type hierarchy (plain polymorphism — calling the same method on different subclasses and each running its own version), not a named pattern: every subclass answers for itself whether the current price lets it trade, and no order ever turns into another kind — so it is not the State pattern either. (The only named patterns the course lists are Observer and Singleton — a class that allows exactly one instance; the Template Method and Strategy labels in this table are our additions.)	Four unrelated classes with copied fields; or one class with a `type` field and `switch`es.
`StopLimitOrder extends StopLossOrder`	A stop-limit is a stop that becomes a *limit* instead of a *market* order: it inherits the trigger, overrides only `limitPrice()`.	Duplicating the trigger logic.
`OrderPart` per fill	One order can fill at several prices/days; each fill needs its own price and quantity (and becomes its own lot).	Storing only an average price (loses tax information).
Observer: `OrderListener`	The exchange tells whoever placed the order; it does not import `Member`. Exchange and account code change independently.	The exchange updating member balances directly.
Reserved funds / reserved shares	Without it, a member with $1,000 could place ten $1,000 orders and all would pass the check. Money is reserved at placement and released as fills or cancellations arrive. Money and shares actually move only in `onFill`, which the **exchange** calls (the course's `callbackStockExchange`) — so the moment an order is *placed* and the moment it is *paid for* are different.	Checking the balance only at fill time (over-commitment).
Lots, FIFO by default	Tax depends on *which* shares you sold; each `LotSale` records it.	Merging all purchases into one average-cost position.
Template Method: `Statement`	Filtering by day range is shared; only the lines differ. `render` is `final`; subclasses fill in `body`.	Two statement classes each re-implementing the filter.
`MoneyTransfer` base class holds the limit rule	One rule, three delivery methods.	Copying the limit check into each subclass.
Singleton?	The course makes `StockExchange` a Singleton (one gateway to the real exchange). We pass one instance into `Member` instead (**dependency injection**: an object is handed the things it needs from outside rather than creating or looking them up itself) — same "only one" in practice, but testable with a fake exchange.	A global `getInstance()` that tests cannot replace.
Engine: `TreeMap<price, ArrayDeque>` per side	Best price in O(log n) (Big-O notation: how the work grows with n items — here, very slowly); FIFO inside a level gives time priority for free.	One `PriorityQueue` (a structure that always hands back the smallest item first) sorted by (price, time) — works, but cancel is O(n) (time grows in step with the number of orders) and levels are not visible.
Engine: lock per symbol	Orders for different symbols match in parallel; same-symbol orders are strictly serialised (handled one at a time, in order), so "time priority" is well defined.	One global lock (slower) or no lock (corrupt book).
Engine: `ExecutionPricePolicy` (Strategy)	The Navi statement prices trades at the **sell** order's price; real continuous-trading exchanges use the **resting** (already-in-book) order's price. Both are one-line policies.	Hard-coding one rule.

Java implementation

Part A — Brokerage (`lld.ch19.brokerage`)

Files: `Orders.java` (enums, money helper, `OrderPart`, the order hierarchy), `Market.java` (stock, inventory, listener, simulated exchange), `Holdings.java` (lots, positions, watchlists, trades, statements), `Accounts.java` (transfers, notifier, member, admin), `Main.java`.

The simulated exchange offers a fixed depth of shares per price level (70 shares at each level, 5 cents apart, 3 levels) so fills come back in parts, like a real book. Part B builds a real matching engine.

```java
// File: lld/ch19/brokerage/Accounts.java
package lld.ch19.brokerage;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

enum AccountStatus { ACTIVE, CLOSED, CANCELED, BLACKLISTED, NONE }

enum ReturnStatus { SUCCESS, FAIL, INSUFFICIENT_FUNDS, INSUFFICIENT_QUANTITY, NO_STOCK_POSITION, ACCOUNT_NOT_ACTIVE
}

enum Direction { DEPOSIT, WITHDRAW }

/** Three ways to move money. They differ only in name and clearing time; the limit rule lives here once. */
abstract class MoneyTransfer {
    static final BigDecimal TRANSFER_LIMIT = Money.of("100000");
    private final BigDecimal amount;
    private final Direction direction;
    MoneyTransfer(BigDecimal amount, Direction direction) { this.amount = amount; this.direction = direction; }
    BigDecimal amount() { return amount; }
    Direction direction() { return direction; }
    boolean isValid() { return amount.signum() > 0 && amount.compareTo(TRANSFER_LIMIT) <= 0; }
    abstract String methodName();
    abstract int daysToClear();
}

final class CheckTransfer extends MoneyTransfer {
    CheckTransfer(BigDecimal a, Direction d) { super(a, d); }
    @Override String methodName() { return "check"; }
    @Override int daysToClear() { return 5; }
}

final class WireTransfer extends MoneyTransfer {
    WireTransfer(BigDecimal a, Direction d) { super(a, d); }
    @Override String methodName() { return "wire"; }
    @Override int daysToClear() { return 1; }
}

final class ElectronicBankTransfer extends MoneyTransfer {
    ElectronicBankTransfer(BigDecimal a, Direction d) { super(a, d); }
    @Override String methodName() { return "electronic bank transfer"; }
    @Override int daysToClear() { return 3; }
}

interface Notifier { void send(String to, String message); }

final class EmailNotifier implements Notifier {
    @Override public void send(String to, String message) { System.out.println("    [email " + to + "] " +
message); }
}

/**
 * A member. Implements OrderListener so the exchange can call back without knowing
 * what a member is. Holds reserved funds so ten $1,000 orders cannot all pass a $1,000 check.
 * Lock order: the member never calls the exchange while holding its own lock.
 */
final class Member implements OrderListener {
    private final String id;
    private final String name;
    private final String email;
    private final StockExchange exchange;
    private final StockInventory inventory;
    private final Notifier notifier;
```

```
    private volatile AccountStatus status = AccountStatus.ACTIVE;

    private BigDecimal cash = Money.of("0");
    private final Map<String, BigDecimal> reservedFunds = new HashMap<>();   // buy orderId -> money still reserved
    private final Map<String, Order> pendingSells = new HashMap<>();            // sell orderId -> open sell order
    private final Map<String, StockPosition> positions = new LinkedHashMap<>();
    private final Map<String, Watchlist> watchlists = new LinkedHashMap<>();
    private final List<Trade> trades = new ArrayList<>();
    private int lotSeq = 1;

    Member(String id, String name, String email, StockExchange ex, StockInventory inv, Notifier n) {
        this.id = id; this.name = name; this.email = email;
        this.exchange = ex; this.inventory = inv; this.notifier = n;
        ex.register(id, this);
    }

    String id() { return id; }
    String name() { return name; }
    void setStatus(AccountStatus s) { status = s; }

    // ---- money ----
    synchronized ReturnStatus transfer(MoneyTransfer t) {
        if (!t.isValid()) return ReturnStatus.FAIL;
        if (t.direction() == Direction.WITHDRAW) {
            if (t.amount().compareTo(availableFunds()) > 0) return ReturnStatus.INSUFFICIENT_FUNDS;
            cash = cash.subtract(t.amount());
        } else {
            cash = cash.add(t.amount());
        }
        return ReturnStatus.SUCCESS;
    }

    synchronized BigDecimal cash() { return cash; }

    synchronized BigDecimal reserved() {
        BigDecimal r = Money.of("0");
        for (Map.Entry<String, BigDecimal> e : reservedFunds.entrySet()) r = r.add(e.getValue());
        return r;
    }

    synchronized BigDecimal availableFunds() { return cash.subtract(reserved()); }

    // ---- orders ----
    ReturnStatus place(Order o) {
        ReturnStatus check = reserveFor(o);                // under our lock
        if (check == ReturnStatus.SUCCESS) exchange.submit(o);   // outside our lock
        return check;
    }

    private synchronized ReturnStatus reserveFor(Order o) {
        if (status != AccountStatus.ACTIVE) return ReturnStatus.ACCOUNT_NOT_ACTIVE;
        if (o.side() == Side.BUY) {
            BigDecimal perShare = o.limitPrice().orElse(inventory.priceOf(o.symbol()));
            BigDecimal need = Money.times(perShare, o.quantity());
            if (need.compareTo(availableFunds()) > 0) return ReturnStatus.INSUFFICIENT_FUNDS;
            reservedFunds.put(o.id(), need);           // released part by part as fills arrive
            return ReturnStatus.SUCCESS;
        }
        StockPosition pos = positions.get(o.symbol());
        if (pos == null) return ReturnStatus.NO_STOCK_POSITION;
        int promised = pendingSells.values().stream()              // shares already promised to other
                .filter(p -> p.symbol().equals(o.symbol()))             // open sells of this symbol
                .mapToInt(Order::remaining).sum();
        if (o.quantity() > pos.quantity() - promised) return ReturnStatus.INSUFFICIENT_QUANTITY;
        pendingSells.put(o.id(), o);
        return ReturnStatus.SUCCESS;
    }

    ReturnStatus cancel(Order o) { return exchange.cancel(o) ? ReturnStatus.SUCCESS : ReturnStatus.FAIL; }
```

```
    /** Called by the exchange: this is where money and shares actually move. */
    @Override public synchronized void onFill(Order o, OrderPart part) {
        BigDecimal value = Money.times(part.price(), part.quantity());
        List<LotSale> sales = List.of();
        if (o.side() == Side.BUY) {
            cash = cash.subtract(value);
            BigDecimal left = reservedFunds.getOrDefault(o.id(), Money.of("0")).subtract(value);
            if (o.remaining() == 0 || left.signum() <= 0) reservedFunds.remove(o.id());
            else reservedFunds.put(o.id(), left);
            positions.computeIfAbsent(o.symbol(), StockPosition::new)
                     .addLot(new StockLot("L" + lotSeq++, part.quantity(), part.price(), part.day()));
        } else {
            cash = cash.add(value);
            if (o.remaining() == 0) pendingSells.remove(o.id());
            sales = positions.get(o.symbol()).sell(part.quantity(), part.price(), Optional.empty());
            if (positions.get(o.symbol()).quantity() == 0) positions.remove(o.symbol());
        }
        trades.add(new Trade(part.day(), o.symbol(), o.side(), part.quantity(), part.price(), sales));
        notifier.send(email, "filled " + part.quantity() + " " + o.symbol() + " @ " + part.price()
                + " (" + o.id() + " " + o.status() + ")");
    }

    @Override public synchronized void onCancelled(Order o) {
        reservedFunds.remove(o.id());
        pendingSells.remove(o.id());
        notifier.send(email, o.id() + " cancelled, " + o.remaining() + " shares unfilled");
    }

    // ---- watchlists, positions, reports ----
    synchronized Watchlist watchlist(String name) { return watchlists.computeIfAbsent(name, Watchlist::new); }
    synchronized List<Watchlist> watchlists() { return List.copyOf(watchlists.values()); }
    synchronized Map<String, List<StockLot>> holdings() {
        Map<String, List<StockLot>> m = new LinkedHashMap<>();
        positions.forEach((s, p) -> m.put(s, p.lots()));
        return m;
    }
    synchronized String statement(Statement s) { return s.render(name, List.copyOf(trades)); }
}

final class Admin {
    void block(Member m) { m.setStatus(AccountStatus.BLACKLISTED); }
    void unblock(Member m) { m.setStatus(AccountStatus.ACTIVE); }
}
```

```java
// File: lld/ch19/brokerage/Holdings.java
package lld.ch19.brokerage;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Optional;
import java.util.Set;

/** One purchase at one price. Kept apart from other purchases because tax depends on it. */
final class StockLot {
    private final String lotId;
    private final BigDecimal buyPrice;
    private final int day;
    private int quantity;

    StockLot(String lotId, int quantity, BigDecimal buyPrice, int day) {
        this.lotId = lotId; this.quantity = quantity; this.buyPrice = buyPrice; this.day = day;
    }
    String lotId() { return lotId; }
    int quantity() { return quantity; }
    BigDecimal buyPrice() { return buyPrice; }
    int day() { return day; }
    void take(int q) { quantity -= q; }
    @Override public String toString() { return lotId + ":" + quantity + "@" + buyPrice; }
}

/** What came out of one lot in one sale - exactly what a tax statement needs. */
record LotSale(String lotId, int quantity, BigDecimal buyPrice, BigDecimal sellPrice) {
    BigDecimal gain() { return Money.times(sellPrice.subtract(buyPrice), quantity); }
}

/** All lots of one symbol. Sells from the named lot first, then oldest first (FIFO). */
final class StockPosition {
    private final String symbol;
    private final List<StockLot> lots = new ArrayList<>();

    StockPosition(String symbol) { this.symbol = symbol; }

    String symbol() { return symbol; }
    void addLot(StockLot lot) { lots.add(lot); }
    int quantity() { return lots.stream().mapToInt(StockLot::quantity).sum(); }
    List<StockLot> lots() { return List.copyOf(lots); }

    List<LotSale> sell(int qty, BigDecimal price, Optional<String> preferredLot) {
        if (qty > quantity()) throw new IllegalStateException("not enough shares");
        List<LotSale> out = new ArrayList<>();
        List<StockLot> order = new ArrayList<>(lots);
        preferredLot.flatMap(id -> lots.stream().filter(l -> l.lotId().equals(id)).findFirst())
                    .ifPresent(l -> { order.remove(l); order.add(0, l); });
        int left = qty;
        for (StockLot lot : order) {
            if (left == 0) break;
            int take = Math.min(left, lot.quantity());
            if (take == 0) continue;
            lot.take(take);
            out.add(new LotSale(lot.lotId(), take, lot.buyPrice(), price));
            left -= take;
        }
        for (Iterator<StockLot> it = lots.iterator(); it.hasNext(); ) if (it.next().quantity() == 0) it.remove();
        return out;
    }
}

/** A watchlist holds symbols, not Stock objects, so it never goes stale. */
final class Watchlist {
```

```
    private final String name;
    private final Set<String> symbols = new LinkedHashSet<>();
    Watchlist(String name) { this.name = name; }
    String name() { return name; }
    void add(String symbol) { symbols.add(symbol); }
    void remove(String symbol) { symbols.remove(symbol); }
    Set<String> symbols() { return Set.copyOf(symbols); }
    String render(StockInventory inv) {
        StringBuilder sb = new StringBuilder(name + ":");
        symbols.forEach(s -> sb.append(' ').append(s).append('=').append(inv.priceOf(s)));
        return sb.toString();
    }
}

/** One executed fill as the member sees it. Sells carry the lots they consumed. */
record Trade(int day, String symbol, Side side, int quantity, BigDecimal price, List<LotSale> sales) { }

/** Template Method: the base class filters trades by day range, subclasses write the lines. */
abstract class Statement {
    private final int fromDay;
    private final int toDay;
    Statement(int fromDay, int toDay) { this.fromDay = fromDay; this.toDay = toDay; }

    final String render(String memberName, List<Trade> all) {
        List<Trade> inRange = all.stream().filter(t -> t.day() >= fromDay && t.day() <= toDay).toList();
        StringBuilder sb = new StringBuilder(title() + " for " + memberName + " (days " + fromDay + "-" + toDay +
")\n");
        body(inRange, sb);
        return sb.toString();
    }
    abstract String title();
    abstract void body(List<Trade> trades, StringBuilder sb);
}

final class QuarterlyStatement extends Statement {
    QuarterlyStatement(int from, int to) { super(from, to); }
    @Override String title() { return "Quarterly statement"; }
    @Override void body(List<Trade> trades, StringBuilder sb) {
        for (Trade t : trades)
            sb.append(String.format("  day %d %-4s %-4s %4d @ %s%n", t.day(), t.side(), t.symbol(), t.quantity(),
t.price()));
    }
}

final class TaxStatement extends Statement {
    TaxStatement(int from, int to) { super(from, to); }
    @Override String title() { return "Tax statement"; }
    @Override void body(List<Trade> trades, StringBuilder sb) {
        BigDecimal total = Money.of("0");
        for (Trade t : trades) for (LotSale s : t.sales()) {
            sb.append(String.format("  %s %s: %d sh bought @ %s sold @ %s gain %s%n",
                    t.symbol(), s.lotId(), s.quantity(), s.buyPrice(), s.sellPrice(), Money.fmt(s.gain())));
            total = total.add(s.gain());
        }
        sb.append("  realised gain: ").append(Money.fmt(total)).append('\n');
    }
}
```

```java
// File: lld/ch19/brokerage/Main.java
package lld.ch19.brokerage;

import java.math.BigDecimal;

public class Main {
    public static void main(String[] args) {
        StockInventory inv = new StockInventory();
        inv.list("AAPL", Money.of("10.00"));
        inv.list("MSFT", Money.of("300.00"));
        inv.list("AMZN", Money.of("120.00"));
        // 70 shares offered per price level, levels 5 cents apart, 3 levels deep
        StockExchange ex = new StockExchange(inv, 70, new BigDecimal("0.05"), 3);
        Member alice = new Member("M1", "Alice", "alice@mail.test", ex, inv, new EmailNotifier());
        Admin admin = new Admin();

        System.out.println("== 1. Fund the account ==");
        System.out.println("  wire 5,000        -> " + alice.transfer(new WireTransfer(Money.of("5000"),
Direction.DEPOSIT)));
        System.out.println("  check 200,000     -> " + alice.transfer(new CheckTransfer(Money.of("200000"),
Direction.DEPOSIT))
                + " (over the 100,000 limit)");
        System.out.println("  cash = " + Money.fmt(alice.cash()));

        System.out.println("\n== 2. Watchlists and search ==");
        alice.watchlist("tech").add("AAPL");
        alice.watchlist("tech").add("MSFT");
        alice.watchlist("retail").add("AMZN");
        alice.watchlists().forEach(w -> System.out.println("  " + w.render(inv)));
        System.out.println("  search 'A' -> " + inv.search("A"));

        System.out.println("\n== 3. Market buy 100 AAPL fills in two parts (two lots) ==");
        Order m1 = new MarketOrder("O1", "M1", "AAPL", Side.BUY, 100);
        System.out.println("  place -> " + alice.place(m1));
        System.out.println("  " + m1 + " parts=" + m1.parts());

        System.out.println("\n== 4. Next day: limit buy 50 AAPL @ 9.50 waits, then fills when price drops ==");
        ex.setDay(2);
        Order l1 = new LimitOrder("O2", "M1", "AAPL", Side.BUY, 50, Money.of("9.50"),
TimeInForce.GOOD_TILL_CANCELLED);
        System.out.println("  place -> " + alice.place(l1) + ", " + l1 + ", reserved " +
Money.fmt(alice.reserved()));
        ex.tick("AAPL", Money.of("9.40"));
        System.out.println("  after tick 9.40: " + l1 + ", reserved " + Money.fmt(alice.reserved()));
        System.out.println("  holdings " + alice.holdings());

        System.out.println("\n== 5. Day 3: price rises, sell 120 AAPL (FIFO across lots) ==");
        ex.setDay(3);
        ex.tick("AAPL", Money.of("11.00"));
        Order s1 = new LimitOrder("O3", "M1", "AAPL", Side.SELL, 120, Money.of("10.90"),
TimeInForce.GOOD_TILL_CANCELLED);
        System.out.println("  place -> " + alice.place(s1) + ", " + s1);
        System.out.println("  holdings " + alice.holdings());

        System.out.println("\n== 6. Protect the rest with a stop-loss @ 10.00, then the price falls ==");
        ex.setDay(4);
        Order stop = new StopLossOrder("O4", "M1", "AAPL", Side.SELL, 30, Money.of("10.00"));
        System.out.println("  place -> " + alice.place(stop) + ", " + stop);
        ex.tick("AAPL", Money.of("10.50"));
        System.out.println("  tick 10.50: " + stop);
        ex.tick("AAPL", Money.of("9.95"));
        System.out.println("  tick 9.95: " + stop + " parts=" + stop.parts());

        System.out.println("\n== 7. Stop-limit buy triggers but its limit is not met ==");
        Order sl = new StopLimitOrder("O5", "M1", "MSFT", Side.BUY, 5, Money.of("305.00"), Money.of("306.00"));
        System.out.println("  place -> " + alice.place(sl) + ", " + sl);
        ex.tick("MSFT", Money.of("310.00"));
        System.out.println("  tick 310 (stop hit, above limit): " + sl);
```

```
        ex.tick("MSFT", Money.of("305.50"));
        System.out.println("  tick 305.50 (within limit): " + sl);

        System.out.println("\n== 8. Reserved funds block over-spending, cancel releases them ==");
        System.out.println("  cash " + Money.fmt(alice.cash()) + ", available " +
Money.fmt(alice.availableFunds()));
        Order big1 = new LimitOrder("O6", "M1", "AMZN", Side.BUY, 20, Money.of("100.00"),
TimeInForce.GOOD_TILL_CANCELLED);
        Order big2 = new LimitOrder("O7", "M1", "AMZN", Side.BUY, 20, Money.of("100.00"),
TimeInForce.GOOD_TILL_CANCELLED);
        System.out.println("  O6 buy 20 AMZN @ 100 -> " + alice.place(big1) + ", available " +
Money.fmt(alice.availableFunds()));
        System.out.println("  O7 buy 20 AMZN @ 100 -> " + alice.place(big2));
        System.out.println("  cancel O6 -> " + alice.cancel(big1) + ", available " +
Money.fmt(alice.availableFunds()));
        System.out.println("  sell 10 AMZN (none owned) -> "
                + alice.place(new MarketOrder("O8", "M1", "AMZN", Side.SELL, 10)));

        System.out.println("\n== 9. Blocked account cannot trade ==");
        admin.block(alice);
        System.out.println("  -> " + alice.place(new MarketOrder("O9", "M1", "AAPL", Side.BUY, 1)));
        admin.unblock(alice);

        System.out.println("\n== 10. Reports ==");
        System.out.print(alice.statement(new QuarterlyStatement(1, 90)));
        System.out.print(alice.statement(new TaxStatement(1, 365)));
    }
}
```

```java
// File: lld/ch19/brokerage/Market.java
package lld.ch19.brokerage;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

/** A symbol and its latest price. */
final class Stock {
    private final String symbol;
    private volatile BigDecimal price;
    Stock(String symbol, BigDecimal price) { this.symbol = symbol; this.price = price; }
    String symbol() { return symbol; }
    BigDecimal price() { return price; }
    void setPrice(BigDecimal p) { price = p; }
}

/** The one place prices are read from, so no two parts of the system disagree. */
final class StockInventory {
    private final Map<String, Stock> stocks = new ConcurrentHashMap<>();
    void list(String symbol, BigDecimal price) { stocks.put(symbol, new Stock(symbol, price)); }
    Optional<Stock> find(String symbol) { return Optional.ofNullable(stocks.get(symbol)); }
    BigDecimal priceOf(String symbol) {
        return find(symbol).orElseThrow(() -> new IllegalArgumentException("unknown " + symbol)).price();
    }
    List<String> search(String prefix) {
        return stocks.keySet().stream().filter(s -> s.startsWith(prefix.toUpperCase())).sorted().toList();
    }
}

/** Observer: how the exchange calls back without knowing what a Member is. */
interface OrderListener {
    void onFill(Order order, OrderPart part);
    void onCancelled(Order order);
}

/**
 * A simulated exchange. At each price level it offers a fixed depth of shares
 * (e.g. 70 @ price, 70 @ price+0.05 ...) so that fills come back in parts, like a
 * real order book. (A real matching engine is built in lld.ch19.stockexchange.)
 * All methods are synchronized: one lock orders every event on the exchange.
 */
final class StockExchange {
    private final StockInventory inventory;
    private final int depthPerLevel;
    private final BigDecimal tickSize;
    private final int levels;
    private final Map<String, List<Order>> openOrders = new ConcurrentHashMap<>();
    private final Map<String, OrderListener> listeners = new ConcurrentHashMap<>();
    private int day = 1;

    StockExchange(StockInventory inventory, int depthPerLevel, BigDecimal tickSize, int levels) {
        this.inventory = inventory;
        this.depthPerLevel = depthPerLevel;
        this.tickSize = tickSize;
        this.levels = levels;
    }

    synchronized void register(String memberId, OrderListener l) { listeners.put(memberId, l); }
    synchronized void setDay(int d) { day = d; }
    synchronized int day() { return day; }

    synchronized void submit(Order o) {
        openOrders.computeIfAbsent(o.symbol(), k -> new ArrayList<>()).add(o);
        tryFill(o, inventory.priceOf(o.symbol()));
```

```
        prune(o.symbol());
    }

    /** A new price arrives: every open order on that symbol gets another chance (stops trigger here). */
    synchronized void tick(String symbol, BigDecimal newPrice) {
        inventory.find(symbol).orElseThrow().setPrice(newPrice);
        for (Order o : List.copyOf(openOrders.getOrDefault(symbol, List.of()))) tryFill(o, newPrice);
        prune(symbol);
    }

    synchronized boolean cancel(Order o) {
        if (!o.cancel()) return false;
        prune(o.symbol());
        listeners.get(o.memberId()).onCancelled(o);
        return true;
    }

    private void tryFill(Order o, BigDecimal market) {
        if (!o.isOpen() || !o.canTradeAt(market)) return;
        for (int k = 0; k < levels && o.remaining() > 0; k++) {
            BigDecimal step = tickSize.multiply(BigDecimal.valueOf(k));
            BigDecimal levelPrice = o.side() == Side.BUY ? market.add(step) : market.subtract(step);
            if (!o.acceptsPrice(levelPrice)) break;          // limit reached: rest stays open
            OrderPart part = new OrderPart(Math.min(depthPerLevel, o.remaining()), levelPrice, day);
            o.addPart(part);
            listeners.get(o.memberId()).onFill(o, part);
        }
    }

    private void prune(String symbol) {
        openOrders.getOrDefault(symbol, new ArrayList<>()).removeIf(o -> !o.isOpen());
    }
}
```

```java
// File: lld/ch19/brokerage/Orders.java
package lld.ch19.brokerage;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

enum Side { BUY, SELL }

enum OrderStatus { OPEN, PARTIALLY_FILLED, FILLED, CANCELLED }

/** Time in force: how long an order may stay alive. Modelled, only GTC used in the demo. */
enum TimeInForce { GOOD_TILL_CANCELLED, FILL_OR_KILL, IMMEDIATE_OR_CANCEL, ON_THE_OPEN, ON_THE_CLOSE }

final class Money {
    private Money() { }
    static BigDecimal of(String s) { return new BigDecimal(s).setScale(2, RoundingMode.HALF_EVEN); }
    static BigDecimal times(BigDecimal price, int qty) {
        return price.multiply(BigDecimal.valueOf(qty)).setScale(2, RoundingMode.HALF_EVEN);
    }
    static String fmt(BigDecimal b) { return "$" + b.setScale(2, RoundingMode.HALF_EVEN).toPlainString(); }
}

/** One fill. A 100-share order might be filled as 70 @ 10.00 plus 30 @ 10.05. */
record OrderPart(int quantity, BigDecimal price, int day) { }

/**
 * Everything the four order types share. They differ in exactly two answers:
 * (1) may I trade at this market price? (2) what is the worst price I accept?
 */
abstract class Order {
    private final String id;
    private final String memberId;
    private final String symbol;
    private final Side side;
    private final int quantity;
    private final TimeInForce timeInForce;
    private final List<OrderPart> parts = new ArrayList<>();
    private OrderStatus status = OrderStatus.OPEN;

    Order(String id, String memberId, String symbol, Side side, int quantity, TimeInForce tif) {
        if (quantity <= 0) throw new IllegalArgumentException("quantity must be positive");
        this.id = id;
        this.memberId = memberId;
        this.symbol = symbol;
        this.side = side;
        this.quantity = quantity;
        this.timeInForce = tif;
    }

    /** Question 1. Market and limit orders always say yes; stop orders wait for the trigger. */
    abstract boolean canTradeAt(BigDecimal marketPrice);

    /** Question 2. Empty means "any price" (market behaviour). */
    abstract Optional<BigDecimal> limitPrice();

    abstract String type();

    /** Is a fill at this price acceptable? BUY: price <= limit. SELL: price >= limit. */
    final boolean acceptsPrice(BigDecimal price) {
        return limitPrice().map(l -> side == Side.BUY ? price.compareTo(l) <= 0 : price.compareTo(l) >= 0)
                           .orElse(true);
    }

    final synchronized void addPart(OrderPart p) {
        if (p.quantity() > remaining()) throw new IllegalStateException("overfill");
```

```
        parts.add(p);
        status = remaining() == 0 ? OrderStatus.FILLED : OrderStatus.PARTIALLY_FILLED;
    }

    final synchronized boolean cancel() {
        if (status == OrderStatus.FILLED || status == OrderStatus.CANCELLED) return false;
        status = OrderStatus.CANCELLED;
        return true;
    }

    final synchronized int filled() { return parts.stream().mapToInt(OrderPart::quantity).sum(); }
    final synchronized int remaining() { return quantity - filled(); }
    final synchronized OrderStatus status() { return status; }
    final synchronized boolean isOpen() { return status == OrderStatus.OPEN || status ==
OrderStatus.PARTIALLY_FILLED; }
    final synchronized List<OrderPart> parts() { return List.copyOf(parts); }

    final String id() { return id; }
    final String memberId() { return memberId; }
    final String symbol() { return symbol; }
    final Side side() { return side; }
    final int quantity() { return quantity; }
    final TimeInForce timeInForce() { return timeInForce; }

    @Override public String toString() {
        return id + " " + type() + " " + side + " " + quantity + " " + symbol + " [" + status() + "]";
    }
}

final class MarketOrder extends Order {
    MarketOrder(String id, String memberId, String symbol, Side side, int qty) {
        super(id, memberId, symbol, side, qty, TimeInForce.GOOD_TILL_CANCELLED);
    }
    @Override boolean canTradeAt(BigDecimal p) { return true; }
    @Override Optional<BigDecimal> limitPrice() { return Optional.empty(); }
    @Override String type() { return "MARKET"; }
}

final class LimitOrder extends Order {
    private final BigDecimal limit;
    LimitOrder(String id, String memberId, String symbol, Side side, int qty, BigDecimal limit, TimeInForce tif) {
        super(id, memberId, symbol, side, qty, tif);
        this.limit = limit;
    }
    @Override boolean canTradeAt(BigDecimal p) { return true; }
    @Override Optional<BigDecimal> limitPrice() { return Optional.of(limit); }
    @Override String type() { return "LIMIT@" + limit; }
}

/**
 * Stop (stop-loss) order: sleeps until the market reaches the stop price, then trades
 * like a market order. SELL stop fires when price <= stop, BUY stop when price >= stop.
 * Once triggered it stays triggered.
 */
class StopLossOrder extends Order {
    private final BigDecimal stopPrice;
    private boolean triggered;

    StopLossOrder(String id, String memberId, String symbol, Side side, int qty, BigDecimal stopPrice) {
        super(id, memberId, symbol, side, qty, TimeInForce.GOOD_TILL_CANCELLED);
        this.stopPrice = stopPrice;
    }

    @Override final synchronized boolean canTradeAt(BigDecimal p) {
        if (!triggered) {
            triggered = side() == Side.SELL ? p.compareTo(stopPrice) <= 0 : p.compareTo(stopPrice) >= 0;
        }
        return triggered;
    }
```

```
    @Override Optional<BigDecimal> limitPrice() { return Optional.empty(); }
    @Override String type() { return "STOP@" + stopPrice; }
    BigDecimal stopPrice() { return stopPrice; }
}

/** A stop order that becomes a LIMIT order (not a market order) once triggered. */
final class StopLimitOrder extends StopLossOrder {
    private final BigDecimal limit;
    StopLimitOrder(String id, String memberId, String symbol, Side side, int qty, BigDecimal stop, BigDecimal
limit) {
        super(id, memberId, symbol, side, qty, stop);
        this.limit = limit;
    }
    @Override Optional<BigDecimal> limitPrice() { return Optional.of(limit); }
    @Override String type() { return "STOP@" + stopPrice() + "/LIMIT@" + limit; }
}
```

Output:

```
== 1. Fund the account ==
  wire 5,000        -> SUCCESS
  check 200,000     -> FAIL (over the 100,000 limit)
  cash = $5000.00

== 2. Watchlists and search ==
  tech: AAPL=10.00 MSFT=300.00
  retail: AMZN=120.00
  search 'A' -> [AAPL, AMZN]

== 3. Market buy 100 AAPL fills in two parts (two lots) ==
    [email alice@mail.test] filled 70 AAPL @ 10.00 (O1 PARTIALLY_FILLED)
    [email alice@mail.test] filled 30 AAPL @ 10.05 (O1 FILLED)
  place -> SUCCESS
  O1 MARKET BUY 100 AAPL [FILLED] parts=[OrderPart[quantity=70, price=10.00, day=1], OrderPart[quantity=30,
price=10.05, day=1]]

== 4. Next day: limit buy 50 AAPL @ 9.50 waits, then fills when price drops ==
  place -> SUCCESS, O2 LIMIT@9.50 BUY 50 AAPL [OPEN], reserved $475.00
    [email alice@mail.test] filled 50 AAPL @ 9.40 (O2 FILLED)
  after tick 9.40: O2 LIMIT@9.50 BUY 50 AAPL [FILLED], reserved $0.00
  holdings {AAPL=[L1:70@10.00, L2:30@10.05, L3:50@9.40]}

== 5. Day 3: price rises, sell 120 AAPL (FIFO across lots) ==
    [email alice@mail.test] filled 70 AAPL @ 11.00 (O3 PARTIALLY_FILLED)
    [email alice@mail.test] filled 50 AAPL @ 10.95 (O3 FILLED)
  place -> SUCCESS, O3 LIMIT@10.90 SELL 120 AAPL [FILLED]
  holdings {AAPL=[L3:30@9.40]}

== 6. Protect the rest with a stop-loss @ 10.00, then the price falls ==
  place -> SUCCESS, O4 STOP@10.00 SELL 30 AAPL [OPEN]
  tick 10.50: O4 STOP@10.00 SELL 30 AAPL [OPEN]
    [email alice@mail.test] filled 30 AAPL @ 9.95 (O4 FILLED)
  tick 9.95: O4 STOP@10.00 SELL 30 AAPL [FILLED] parts=[OrderPart[quantity=30, price=9.95, day=4]]

== 7. Stop-limit buy triggers but its limit is not met ==
  place -> SUCCESS, O5 STOP@305.00/LIMIT@306.00 BUY 5 MSFT [OPEN]
  tick 310 (stop hit, above limit): O5 STOP@305.00/LIMIT@306.00 BUY 5 MSFT [OPEN]
    [email alice@mail.test] filled 5 MSFT @ 305.50 (O5 FILLED)
  tick 305.50 (within limit): O5 STOP@305.00/LIMIT@306.00 BUY 5 MSFT [FILLED]

== 8. Reserved funds block over-spending, cancel releases them ==
  cash $3617.00, available $3617.00
  O6 buy 20 AMZN @ 100 -> SUCCESS, available $1617.00
  O7 buy 20 AMZN @ 100 -> INSUFFICIENT_FUNDS
    [email alice@mail.test] O6 cancelled, 20 shares unfilled
  cancel O6 -> SUCCESS, available $3617.00
  sell 10 AMZN (none owned) -> NO_STOCK_POSITION

== 9. Blocked account cannot trade ==
  -> ACCOUNT_NOT_ACTIVE

== 10. Reports ==
Quarterly statement for Alice (days 1-90)
  day 1 BUY  AAPL   70 @ 10.00
  day 1 BUY  AAPL   30 @ 10.05
  day 2 BUY  AAPL   50 @ 9.40
  day 3 SELL AAPL   70 @ 11.00
  day 3 SELL AAPL   50 @ 10.95
  day 4 SELL AAPL   30 @ 9.95
  day 4 BUY  MSFT    5 @ 305.50
Tax statement for Alice (days 1-365)
  AAPL L1: 70 sh bought @ 10.00 sold @ 11.00 gain $70.00
  AAPL L2: 30 sh bought @ 10.05 sold @ 10.95 gain $27.00
  AAPL L3: 20 sh bought @ 9.40 sold @ 10.95 gain $31.00
  AAPL L3: 30 sh bought @ 9.40 sold @ 9.95 gain $16.50
  realised gain: $144.50
```

Reading the output. Step 1: the 200,000 check fails the 100,000 transfer limit. Step 3 reproduces the course's example: a market order for 100 fills as 70 @ 10.00 and 30 @ 10.05 and creates **two lots**. Notice the course's warning in action: the order reserved $1,000.00 (100 × last price 10.00) but cost $1,001.50 — a market order can spend more than was reserved. Step 4: a limit buy waits until the price tick to 9.40. Step 5 sells 120 shares FIFO across three lots. Step 6: the stop does nothing at 10.50 and fires at 9.95. Step 7: the stop-limit triggers at 310 but its 306 limit blocks the fill until 305.50. Step 8: the reserve blocks a second $2,000 order; cancelling returns the money. Step 10: the tax statement lists the gain **per lot**.

Part B — Matching engine (`lld.ch19.stockexchange`)

Files: `OrderBook.java` (side, order, trade, price policy, book), `MatchingEngine.java` (routing, per-symbol locking, line parser), `MatchingEngineTests.java` (nine tests, standard library only), `Main.java` (runs the sample — or a file given as the first argument — then the tests).

```
// File: lld/ch19/stockexchange/Main.java
package lld.ch19.stockexchange;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

/** Usage: java lld.ch19.stockexchange.Main [orders.txt]  (no file = the sample from the problem statement). */
public class Main {
    static final List<String> SAMPLE = List.of(
            "#1 09:45 BAC sell 240.12 100",
            "#2 09:46 BAC sell 237.45 90",
            "#3 09:47 BAC buy 238.10 110",
            "#4 09:48 BAC buy 237.80 10",
            "#5 09:49 BAC buy 237.80 40",
            "#6 09:50 BAC sell 236.00 50");

    public static void main(String[] args) throws IOException {
        List<String> lines = args.length > 0 ? Files.readAllLines(Path.of(args[0])) : SAMPLE;

        System.out.println("== Trades (execution price = sell order's price) ==");
        MatchingEngine engine = new MatchingEngine(ExecutionPricePolicy.SELL_PRICE, System.out::println);
        lines.stream().filter(l -> !l.isBlank()).forEach(engine::submitLine);
        System.out.println("book after: " + engine.book("BAC").snapshot());

        System.out.println("\n== Same input, real-exchange rule (price of the resting order) ==");
        MatchingEngine real = new MatchingEngine(ExecutionPricePolicy.RESTING_PRICE, System.out::println);
        lines.stream().filter(l -> !l.isBlank()).forEach(real::submitLine);

        System.out.println("\n== Tests ==");
        int failed = MatchingEngineTests.runAll();
        if (failed > 0) System.exit(1);
    }
}
```

```java
// File: lld/ch19/stockexchange/MatchingEngine.java
package lld.ch19.stockexchange;

import java.math.BigDecimal;
import java.time.LocalTime;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.Consumer;

/**
 * Routes each order to its symbol's book. One lock per book: orders for different
 * symbols match in parallel, orders for the same symbol are strictly serialised
 * (which is what makes "time priority" well defined).
 */
final class MatchingEngine {
    private final Map<String, OrderBook> books = new ConcurrentHashMap<>();
    private final Map<String, String> symbolOfOrder = new ConcurrentHashMap<>();
    private final AtomicLong sequence = new AtomicLong();
    private final ExecutionPricePolicy policy;
    private final Consumer<Trade> tradeSink;

    MatchingEngine(ExecutionPricePolicy policy, Consumer<Trade> tradeSink) {
        this.policy = policy;
        this.tradeSink = tradeSink;
    }

    List<Trade> submit(String id, LocalTime time, String symbol, Side side, BigDecimal price, int qty) {
        if (qty <= 0 || price.signum() <= 0) throw new IllegalArgumentException("bad quantity/price: " + id);
        if (symbolOfOrder.putIfAbsent(id, symbol) != null) throw new IllegalArgumentException("duplicate order id "
+ id);
        OrderBook book = books.computeIfAbsent(symbol, s -> new OrderBook(s, policy));
        List<Trade> trades;
        synchronized (book) {
            // the sequence is taken inside the lock, so arrival order == matching order
            Order o = new Order(id, time, symbol, side, price, qty, sequence.incrementAndGet());
            trades = book.add(o);
        }
        trades.forEach(tradeSink);        // publish outside the lock
        return trades;
    }

    boolean cancel(String orderId) {
        String symbol = symbolOfOrder.get(orderId);
        if (symbol == null) return false;
        OrderBook book = books.get(symbol);
        synchronized (book) { return book.cancel(orderId); }
    }

    OrderBook book(String symbol) { return books.get(symbol); }

    /** Parses "#1 09:45 BAC sell 240.12 100" and submits it. */
    List<Trade> submitLine(String line) {
        String[] f = line.trim().split("\\s+");
        if (f.length != 6) throw new IllegalArgumentException("bad line: " + line);
        return submit(f[0], LocalTime.parse(f[1]), f[2], Side.valueOf(f[3].toUpperCase()),
                new BigDecimal(f[4]), Integer.parseInt(f[5]));
    }
}
```

```java
// File: lld/ch19/stockexchange/MatchingEngineTests.java
package lld.ch19.stockexchange;

import java.math.BigDecimal;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

/** Tiny test harness (no JUnit - standard library only). Each test throws AssertionError on failure. */
final class MatchingEngineTests {
    private MatchingEngineTests() { }

    interface Test { void run() throws Exception; }

    record Case(String name, Test body) { }

    static int runAll() {
        List<Case> tests = List.of(
                new Case("sample input gives the expected four trades", MatchingEngineTests::sample),
                new Case("better price wins over earlier time", MatchingEngineTests::pricePriority),
                new Case("same price: earlier order fills first", MatchingEngineTests::timePriority),
                new Case("no trade when prices do not cross", MatchingEngineTests::noCross),
                new Case("partial fill leaves the remainder resting", MatchingEngineTests::partialFill),
                new Case("cancelled order is never matched", MatchingEngineTests::cancel),
                new Case("symbols never match each other", MatchingEngineTests::symbolIsolation),
                new Case("duplicate id and bad quantity rejected", MatchingEngineTests::validation),
                new Case("concurrent submits conserve quantity", MatchingEngineTests::concurrency));
        int failed = 0;
        for (Case t : tests) {
            try {
                t.body().run();
                System.out.println("  PASS  " + t.name());
            } catch (Exception | AssertionError e) {
                failed++;
                System.out.println("  FAIL  " + t.name() + " -> " + e.getMessage());
            }
        }
        System.out.println("  " + (tests.size() - failed) + "/" + tests.size() + " passed");
        return failed;
    }

    // ---------- helpers ----------
    private static MatchingEngine engine(List<Trade> sink) {
        return new MatchingEngine(ExecutionPricePolicy.SELL_PRICE, sink::add);
    }

    private static void eq(Object expected, Object actual) {
        if (!expected.equals(actual)) throw new AssertionError("expected <" + expected + "> but was <" + actual +
">");
    }

    private static List<String> str(List<Trade> trades) { return trades.stream().map(Trade::toString).toList(); }

    private static final LocalTime T = LocalTime.of(10, 0);

    // ---------- tests ----------
    static void sample() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        Main.SAMPLE.forEach(e::submitLine);
        eq(List.of("#3 237.45 90 #2", "#3 236.00 20 #6", "#4 236.00 10 #6", "#5 236.00 20 #6"), str(out));
    }
```

```
    static void pricePriority() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("s1", T, "X", Side.SELL, new BigDecimal("101"), 10);   // earlier but worse
        e.submit("s2", T, "X", Side.SELL, new BigDecimal("100"), 10);   // later but cheaper
        e.submit("b1", T, "X", Side.BUY, new BigDecimal("105"), 10);
        eq(List.of("b1 100 10 s2"), str(out));
    }

    static void timePriority() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("b1", T, "X", Side.BUY, new BigDecimal("50"), 5);
        e.submit("b2", T, "X", Side.BUY, new BigDecimal("50"), 5);
        e.submit("s1", T, "X", Side.SELL, new BigDecimal("49"), 7);
        eq(List.of("b1 49 5 s1", "b2 49 2 s1"), str(out));
        eq(3, e.book("X").restingQty(Side.BUY));
    }

    static void noCross() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("b1", T, "X", Side.BUY, new BigDecimal("99.99"), 5);
        e.submit("s1", T, "X", Side.SELL, new BigDecimal("100.00"), 5);
        eq(0, out.size());
        eq(5, e.book("X").restingQty(Side.BUY));
        eq(5, e.book("X").restingQty(Side.SELL));
    }

    static void partialFill() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("s1", T, "X", Side.SELL, new BigDecimal("10"), 100);
        e.submit("b1", T, "X", Side.BUY, new BigDecimal("10"), 30);
        e.submit("b2", T, "X", Side.BUY, new BigDecimal("10"), 30);
        eq(40, e.book("X").restingQty(Side.SELL));
        eq(0, e.book("X").restingQty(Side.BUY));
    }

    static void cancel() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("s1", T, "X", Side.SELL, new BigDecimal("10"), 10);
        e.submit("s2", T, "X", Side.SELL, new BigDecimal("10"), 10);
        eq(true, e.cancel("s1"));
        eq(false, e.cancel("s1"));
        e.submit("b1", T, "X", Side.BUY, new BigDecimal("10"), 10);
        eq(List.of("b1 10 10 s2"), str(out));
    }

    static void symbolIsolation() {
        List<Trade> out = new ArrayList<>();
        MatchingEngine e = engine(out);
        e.submit("s1", T, "AAA", Side.SELL, new BigDecimal("1"), 10);
        e.submit("b1", T, "BBB", Side.BUY, new BigDecimal("5"), 10);
        eq(0, out.size());
    }

    static void validation() {
        MatchingEngine e = engine(new ArrayList<>());
        e.submit("x1", T, "X", Side.BUY, new BigDecimal("1"), 1);
        try { e.submit("x1", T, "X", Side.BUY, new BigDecimal("1"), 1); throw new AssertionError("dup accepted"); }
        catch (IllegalArgumentException expected) { /* ok */ }
        try { e.submit("x2", T, "X", Side.BUY, new BigDecimal("1"), 0); throw new AssertionError("qty 0 accepted");
}
        catch (IllegalArgumentException expected) { /* ok */ }
    }
```

```
    /** 8 threads x 500 orders on one symbol. Every share bought must equal every share sold. */
    static void concurrency() throws InterruptedException {
        List<Trade> out = Collections.synchronizedList(new ArrayList<>());
        MatchingEngine e = engine(out);
        ExecutorService pool = Executors.newFixedThreadPool(8);
        CountDownLatch go = new CountDownLatch(1);
        AtomicInteger buys = new AtomicInteger();
        AtomicInteger sells = new AtomicInteger();
        for (int t = 0; t < 8; t++) {
            int thread = t;
            pool.submit(() -> {
                go.await();
                for (int i = 0; i < 500; i++) {
                    Side side = (i + thread) % 2 == 0 ? Side.BUY : Side.SELL;
                    int qty = 1 + (i % 7);
                    (side == Side.BUY ? buys : sells).addAndGet(qty);
                    e.submit("t" + thread + "-" + i, T, "X", side, new BigDecimal(100 + (i % 3)), qty);
                }
                return null;
            });
        }
        go.countDown();
        pool.shutdown();
        if (!pool.awaitTermination(30, TimeUnit.SECONDS)) throw new AssertionError("timeout");
        int traded = out.stream().mapToInt(Trade::quantity).sum();
        eq(buys.get(), traded + e.book("X").restingQty(Side.BUY));
        eq(sells.get(), traded + e.book("X").restingQty(Side.SELL));
        // the book must never be left crossed: best bid < best ask
        OrderBook book = e.book("X");
        if (book.bestBid().isPresent() && book.bestAsk().isPresent()
                && book.bestBid().get().compareTo(book.bestAsk().get()) >= 0)
            throw new AssertionError("book left crossed: " + book.snapshot());
    }
}
```

```java
// File: lld/ch19/stockexchange/OrderBook.java
package lld.ch19.stockexchange;

import java.math.BigDecimal;
import java.time.LocalTime;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.NavigableMap;
import java.util.Optional;
import java.util.TreeMap;

enum Side { BUY, SELL }

/** An order. Only remainingQty changes, and only under the book's lock. */
final class Order {
    private final String id;
    private final LocalTime time;
    private final String symbol;
    private final Side side;
    private final BigDecimal price;
    private final long sequence;          // arrival order: breaks ties when times are equal
    private int remainingQty;

    Order(String id, LocalTime time, String symbol, Side side, BigDecimal price, int qty, long sequence) {
        if (qty <= 0) throw new IllegalArgumentException("quantity must be positive: " + id);
        if (price.signum() <= 0) throw new IllegalArgumentException("price must be positive: " + id);
        this.id = id; this.time = time; this.symbol = symbol; this.side = side;
        this.price = price; this.remainingQty = qty; this.sequence = sequence;
    }

    String id() { return id; }
    LocalTime time() { return time; }
    String symbol() { return symbol; }
    Side side() { return side; }
    BigDecimal price() { return price; }
    long sequence() { return sequence; }
    int remainingQty() { return remainingQty; }
    void fill(int qty) { remainingQty -= qty; }
    boolean isDone() { return remainingQty == 0; }
}

/** Output line: "<buy_order_id> <execution_price> <quantity> <sell_order_id>". */
record Trade(String buyOrderId, BigDecimal price, int quantity, String sellOrderId) {
    @Override public String toString() { return buyOrderId + " " + price.toPlainString() + " " + quantity + " " +
sellOrderId; }
}

/** Who sets the trade price. The Navi problem uses the SELL order's price; real exchanges use the resting order's.
*/
interface ExecutionPricePolicy {
    BigDecimal price(Order buy, Order sell, Order incoming);

    ExecutionPricePolicy SELL_PRICE = (buy, sell, incoming) -> sell.price();
    ExecutionPricePolicy RESTING_PRICE = (buy, sell, incoming) -> incoming == buy ? sell.price() : buy.price();
}

/**
 * The order book for ONE symbol. Each side is a sorted map of price level -> FIFO queue.
 * Bids: highest price first. Asks: lowest price first. Inside a level: first come, first served.
 * That is price-time priority. Not thread-safe by itself: the engine locks it.
 */
final class OrderBook {
```

```
    private final String symbol;
    private final NavigableMap<BigDecimal, Deque<Order>> bids = new TreeMap<>(Collections.reverseOrder());
    private final NavigableMap<BigDecimal, Deque<Order>> asks = new TreeMap<>();
    private final Map<String, Order> resting = new HashMap<>();
    private final ExecutionPricePolicy pricePolicy;

    OrderBook(String symbol, ExecutionPricePolicy pricePolicy) {
        this.symbol = symbol;
        this.pricePolicy = pricePolicy;
    }

    /** Match the incoming order against the other side, then rest whatever is left. */
    List<Trade> add(Order incoming) {
        List<Trade> trades = new ArrayList<>();
        NavigableMap<BigDecimal, Deque<Order>> opposite = incoming.side() == Side.BUY ? asks : bids;

        while (!incoming.isDone() && !opposite.isEmpty()) {
            Map.Entry<BigDecimal, Deque<Order>> best = opposite.firstEntry();
            if (!crosses(incoming, best.getKey())) break;          // best price does not meet ours: stop
            Deque<Order> queue = best.getValue();
            Iterator<Order> it = queue.iterator();
            while (!incoming.isDone() && it.hasNext()) {
                Order restingOrder = it.next();                    // oldest first = time priority
                int qty = Math.min(incoming.remainingQty(), restingOrder.remainingQty());
                Order buy = incoming.side() == Side.BUY ? incoming : restingOrder;
                Order sell = incoming.side() == Side.SELL ? incoming : restingOrder;
                trades.add(new Trade(buy.id(), pricePolicy.price(buy, sell, incoming), qty, sell.id()));
                incoming.fill(qty);
                restingOrder.fill(qty);
                if (restingOrder.isDone()) {
                    it.remove();
                    resting.remove(restingOrder.id());
                }
            }
            if (queue.isEmpty()) opposite.pollFirstEntry();
        }
        if (!incoming.isDone()) rest(incoming);
        return trades;
    }

    boolean cancel(String orderId) {
        Order o = resting.remove(orderId);
        if (o == null) return false;
        NavigableMap<BigDecimal, Deque<Order>> side = o.side() == Side.BUY ? bids : asks;
        Deque<Order> q = side.get(o.price());
        q.remove(o);
        if (q.isEmpty()) side.remove(o.price());
        return true;
    }

    int restingQty(Side side) {
        return (side == Side.BUY ? bids : asks).values().stream()
                .flatMap(Deque::stream).mapToInt(Order::remainingQty).sum();
    }

    Optional<BigDecimal> bestBid() { return bids.isEmpty() ? Optional.empty() : Optional.of(bids.firstKey()); }

    Optional<BigDecimal> bestAsk() { return asks.isEmpty() ? Optional.empty() : Optional.of(asks.firstKey()); }

    String snapshot() {
        return symbol + " bids=" + levels(bids) + " asks=" + levels(asks);
    }

    private static boolean crosses(Order incoming, BigDecimal bestOpposite) {
        return incoming.side() == Side.BUY
                ? incoming.price().compareTo(bestOpposite) >= 0     // buy meets or beats the ask
                : incoming.price().compareTo(bestOpposite) <= 0;    // sell meets or undercuts the bid
    }
```

```
    private void rest(Order o) {
        (o.side() == Side.BUY ? bids : asks).computeIfAbsent(o.price(), p -> new ArrayDeque<>()).addLast(o);
        resting.put(o.id(), o);
    }

    private static String levels(NavigableMap<BigDecimal, Deque<Order>> side) {
        List<String> out = new ArrayList<>();
        side.forEach((p, q) -> out.add(p.toPlainString() + "x" + q.stream().mapToInt(Order::remainingQty).sum()));
        return out.toString();
    }
}
```

Output:

```
== Trades (execution price = sell order's price) ==
#3 237.45 90 #2
#3 236.00 20 #6
#4 236.00 10 #6
#5 236.00 20 #6
book after: BAC bids=[237.80x20] asks=[240.12x100]

== Same input, real-exchange rule (price of the resting order) ==
#3 237.45 90 #2
#3 238.10 20 #6
#4 237.80 10 #6
#5 237.80 20 #6

== Tests ==
  PASS  sample input gives the expected four trades
  PASS  better price wins over earlier time
  PASS  same price: earlier order fills first
  PASS  no trade when prices do not cross
  PASS  partial fill leaves the remainder resting
  PASS  cancelled order is never matched
  PASS  symbols never match each other
  PASS  duplicate id and bad quantity rejected
  PASS  concurrent submits conserve quantity
  9/9 passed
```

Reading the output. The first block matches the expected output in the problem statement exactly. The second block runs the same orders with the real-exchange rule: #6 (the incoming sell) trades at each *resting* buyer's price (238.10, 237.80), which is how a continuous limit order book normally prices trades. The tests cover price priority, time priority, no-cross, partial fills, cancel, symbol isolation, validation, and a concurrency test (8 threads × 500 orders) that checks quantity is conserved and the book is never left crossed (best bid must stay below best ask).

Concurrency & edge cases

Risk	Where	Prevention
Over-spending with many open orders	Brokerage	`reserveFor` (synchronized on the member) subtracts all reservations from cash before accepting a new buy.
Selling the same shares twice	Brokerage	Open sell orders are tracked; free shares = position − remaining quantity of open sells *of that symbol*.
Deadlock between member and exchange	Brokerage	Lock order is fixed: the member never calls the exchange while holding its own lock (`place` reserves under the lock, then submits outside it). The exchange may call the member (exchange → member), never the reverse.
Market order costs more than reserved	Brokerage	Known gap (shown in step 3). Fix: reserve a buffer (e.g. +5%) or reject prices too far above the last price ("price band").
Time in force not enforced	Brokerage	The enum has five values (GTC, fill-or-kill, immediate-or-cancel, on-the-open, on-the-close) and no code reads them — every order behaves as good-till-cancelled within the demo. A GTC order should survive into the next trading day; FOK must be cancelled if it cannot be filled in full; IOC cancels whatever is left after one pass. All of this belongs in the exchange, next to the fill loop.
Two orders at the same price arrive "together"	Engine	Whichever thread gets the book's lock first is appended to the level's queue first, so lock-acquisition order = time priority. The `sequence` number is taken inside the same lock, so it records exactly that order (useful for logs and audits).
Cancel racing a match	Engine	Cancel takes the same per-symbol lock; an order is either matched or cancelled, never both.
Duplicate id / zero quantity / negative price	Engine	Validated before the id is registered (test `validation`).
Trade publishing slowing matching	Engine	Trades are collected under the lock and published after releasing it.
Floating-point prices	Both	`BigDecimal` everywhere; `TreeMap` keys compare with `compareTo`, so `237.8` and `237.80` land on the same level.

Real-world example

Indian brokers such as Zerodha offer the same stop order types to retail users (individual investors): **SL** (stop-loss *limit*) and **SL-M** (stop-loss *market*) — our `StopLimitOrder` and `StopLossOrder`. (Which segments allow SL-M has changed over time, so check the broker's current rules.) Their orders go to exchanges such as NSE (National Stock Exchange of India), whose continuous-trading order book matches on price-time priority, like our `OrderBook`. At the extreme end of performance, the LMAX exchange (described in Martin Fowler's article "The LMAX Architecture") runs its business logic, including matching, on a **single thread** that reads input events from a ring buffer (a fixed-size array used as a circular queue; LMAX's version is called the Disruptor) — the same idea as our "one lock per symbol, strictly serialised", taken to its logical end. On the tax side, US rules treat shares as sold first-in-first-out unless you identify specific lots, and FIFO is a common broker default — the reason `StockPosition.sell` takes an optional lot id.

Extensions & follow-up questions

- **"Add a trailing-stop order."** New subclass: it tracks the best price seen and moves its stop by a fixed distance. `canTradeAt` changes; nothing else does.
- **"Implement IOC and FOK."** IOC: after the fill loop, cancel any remainder. FOK: before filling, sum the quantity available at acceptable prices; if short, cancel without filling.
- **"Price alerts (the 'notify when a stock moves by X%' feature)."** A `PriceAlert` observer registered on `StockInventory`/`tick`; fires once when the move crosses the threshold.
- **"Modify an order."** On many exchanges, reducing quantity keeps time priority, but changing price (or increasing quantity) loses it — implement as cancel + new order.
- **"Make the engine faster."** Single writer thread per symbol fed by a queue (no locks on the hot path — the code that runs for every single order), pre-allocated objects, prices as `long` ticks instead of `BigDecimal`.

- **"Market orders in the engine."** Treat as a limit at +∞ (buy) or 0 (sell); never rest the remainder — cancel it.
- **"Settlement."** Trades settle later (for example T+1: one business day after the trade); add `unsettledCash` to the member so sale proceeds are not withdrawable immediately.

In the interview

30–45 minute plan (brokerage)

1. Clarify order types, lots, transfers, reports (3 min).
2. Actors and use cases (3 min).
3. Say the key sentence: *"The four order types differ in exactly two things: whether they may trade yet, and the worst price they accept."* Draw `Order` + four tiny subclasses (8 min).
4. Explain **lots** and why they exist: tax depends on which shares you sold (4 min).
5. Explain **reserved funds** (3 min).
6. Code `canTradeAt` / `limitPrice` for the four types and `StockPosition.sell` (10 min).
7. Observer callback, statements, extensions (5 min).

Machine-coding (Navi) plan, 90 minutes: parse lines → `Order`; `OrderBook` with two `TreeMap`s of `ArrayDeque`; `add()` matching loop; print trades; then tests for the sample, price priority, time priority, partial fill and cancel. Get the sample passing first, then refactor (restructure the code without changing what it does).

Follow-ups with answers

- *Why not four unrelated order classes?* They share member, symbol, quantity, side, status and fills; duplicating that invites bugs. The hierarchy isolates the two real differences.
- *Why is `StockInventory` separate from `StockExchange`?* One place to read prices, so no two components disagree about a price.
- *What data structure for the book, and the complexity?* `TreeMap` of price levels (a red-black tree — a self-balancing sorted tree): finding the best level with `firstEntry` and inserting a new level are both O(log L) for L price levels; FIFO queue per level: append O(1), match O(1) per fill. Cancel: O(log L) to find the level + O(k) in that level (use a linked structure with a handle for O(1)).
- *At what price does a trade execute?* In the Navi statement: the sell order's price. In real continuous trading: the resting order's price — the incoming order may get a better price than it asked for ("price improvement").

Self-check

1. **Q:** A SELL stop-loss at 10.00; the price goes 10.50 → 9.95 → 10.20. When does it trade? **A:** At the 9.95 tick. It stays triggered even when the price goes back up.
2. **Q:** What does a stop-limit add to a stop-loss? **A:** A worst acceptable price after triggering — it becomes a limit order, not a market order.
3. **Q:** Why does one market order create two lots in step 3? **A:** It filled at two prices, and a lot is defined by its purchase price.
4. **Q:** Two bids at 50.00 arrive, b1 then b2; a sell for 7 at 49 arrives. Trades? **A:** b1 gets 5, b2 gets 2 — time priority within the level (test `timePriority`).
5. **Q:** What does "the book is crossed" mean and why is it a bug? **A:** Best bid ≥ best ask; those two orders should already have traded.
6. **Q:** Which pattern produces the tax and quarterly statements? **A:** Template Method — shared filtering in `Statement.render`, specific lines in `body`.

TL;DR

- **Brokerage:** one abstract `Order`, four tiny subclasses answering *may I trade now?* and *worst price?*; fills are `OrderPart`s; purchases are **lots**; money is **reserved** at placement; the exchange calls back via **Observer**; statements are a **Template Method**.
- **Matching engine:** per symbol, two `TreeMap<price, FIFO queue>`; match best price first, oldest first; rest the remainder; one lock per symbol; trade price is a pluggable policy (Navi: sell price; real exchanges: resting price).

Design an Online Auction System

Problem & clarifying questions

An online auction (think eBay auctions) lets a **seller** list an item with a starting price and an end time. **Bidders** offer higher and higher amounts. When time runs out, the highest bidder wins — if the seller's hidden minimum (the **reserve price**) was reached.

Question	Assumed answer
What does a listing contain?	Item name, description, category, starting price, minimum bid increment, optional reserve price, duration.
What is a valid bid?	The first bid must be at least the starting price; later bids must beat the current highest by at least the increment.
Can the seller bid? Can the leader outbid themselves?	No and no.
When does an auction end?	At its end time. A late bid is refused even if the closing job has not run yet.
Who is told what?	The outbid bidder immediately; the winner and seller at close; the seller if unsold.
Proxy (automatic) bidding, anti-sniping (stopping last-second bids from stealing a win), payments?	Extensions — mention them.

Requirements

Functional

1. Users register and log in. (We model registration; login itself is the usual authentication (checking who a user is) service and is not coded here.)
2. Users create listings with item name, description, starting price and duration.
3. Users browse and search listings by name, category and price range.
4. Users place bids on **active** listings.
5. The system keeps the **current highest bid** up to date and notifies bidders.
6. When the duration ends, the auction closes and the **highest bidder wins** (if the reserve is met).

Non-functional

7. Many users bid on the same listing at the same moment; the system must stay consistent (one true highest bid).
8. Extensible: new bid rules, notification channels or search criteria without rewriting the core.

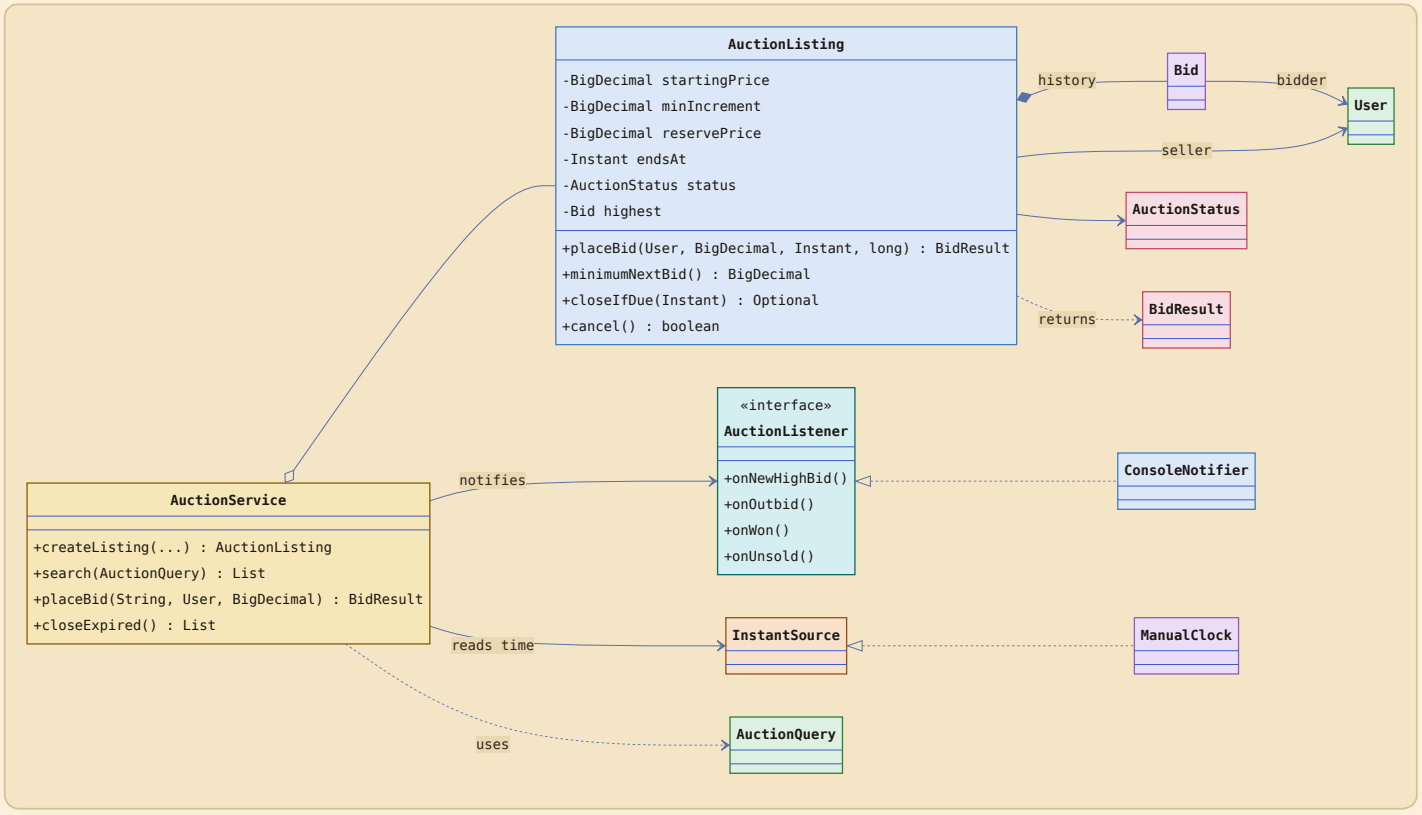
Use cases

Actor	Use cases
Seller	Create listing, cancel listing (only while nobody has bid), see result
Bidder	Search, view listing and bid history, place bid, be told when outbid or when won
System (scheduler)	Activate scheduled auctions, close expired ones, declare winner, send notifications

Core entities

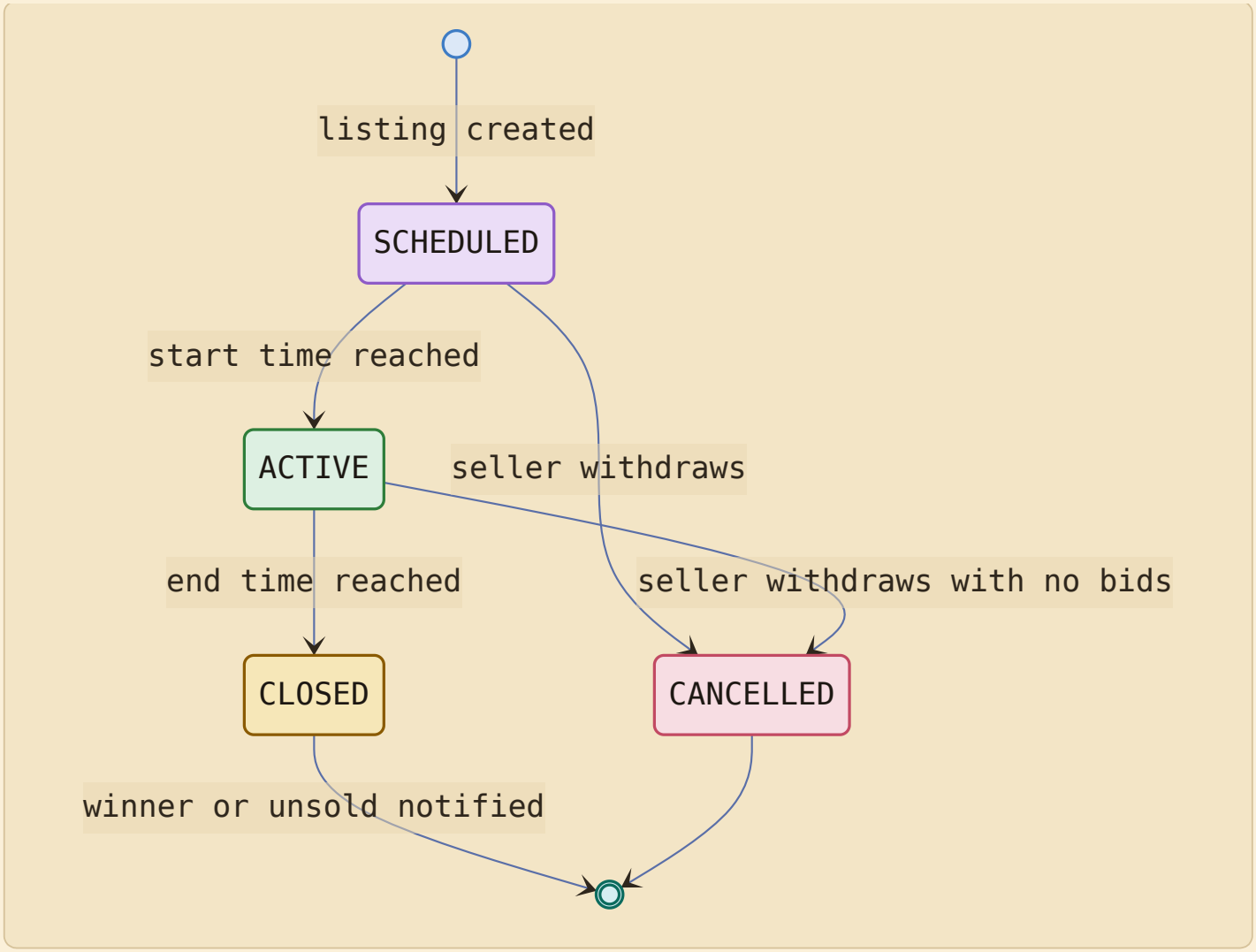
Class	Responsibility
`User`	id, name, email (a `record`).
`Bid`	bidder, amount (`BigDecimal`), time, sequence number. Immutable.
`AuctionStatus`	`SCHEDULED → ACTIVE → CLOSED`, or `CANCELLED`.
`AuctionListing`	Item details, seller, prices, times, bid history, highest bid. **Owns the bidding rules** and guards them with its own lock.
`BidOutcome` / `BidResult`	Why a bid was accepted or refused, the next minimum bid, the accepted bid, who was outbid.
`CloseResult`	Listing, winning bid (if any), whether the reserve was met.
`AuctionQuery`	Fluent (calls chained one after another) search criteria; each criterion is a `Predicate` (a function that answers true/false; Specification pattern).
`AuctionListener` / `ConsoleNotifier`	Observer: new high bid, outbid, won, unsold.
`AuctionService`	Facade: register, create, search, bid, close expired. Calls listeners **outside** listing locks.
`ManualClock`	An `InstantSource` (Java's "what time is it now?" interface) we can move forward in demos and tests.

Class diagram



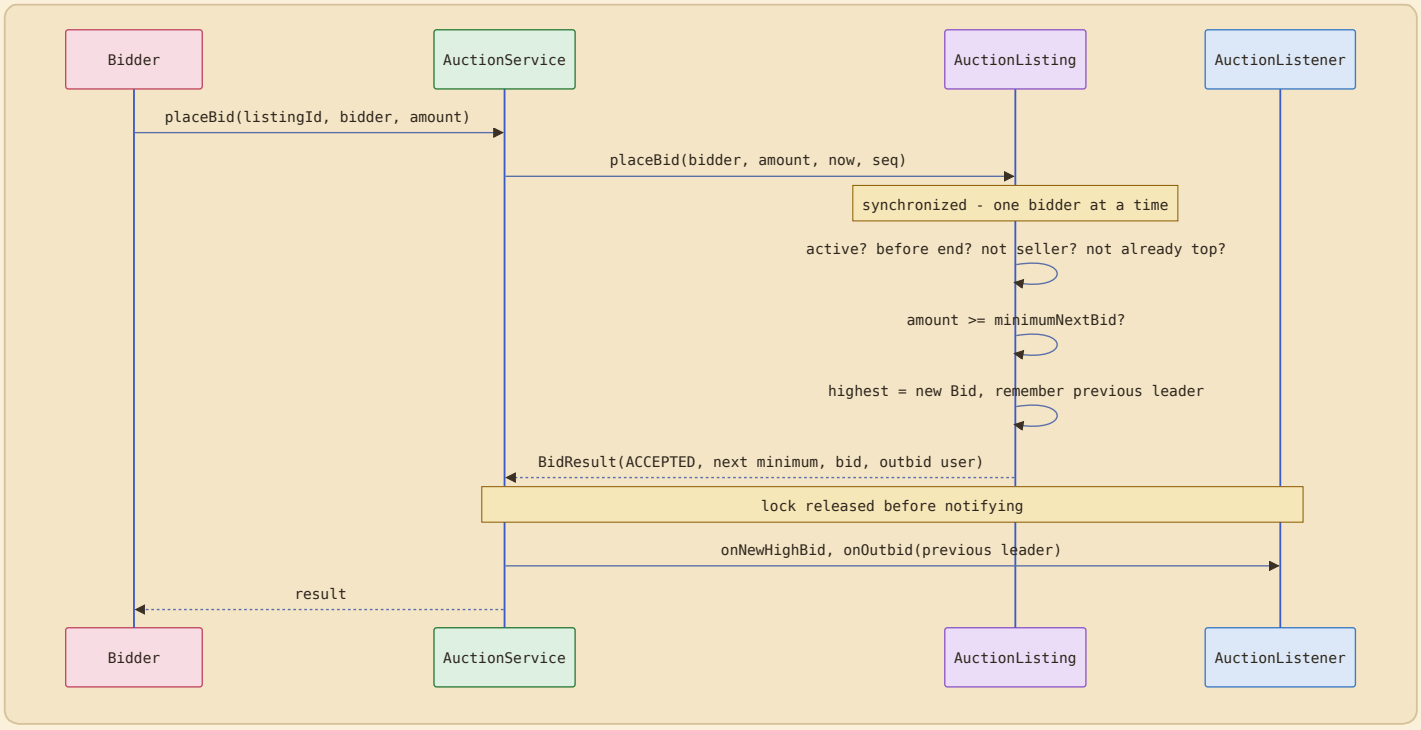
Key flows

Auction lifecycle



`SCHEDULED → ACTIVE` happens **lazily** (only when someone next touches the listing, not at the exact moment): every method first calls `refresh(now)`. No background thread is needed just to flip the state.

Placing a bid (the critical section) — a **critical section** is a piece of code that only one thread may run at a time because it reads and changes shared data.



Closing. A scheduler (in production a `ScheduledExecutorService` — Java's built-in timer that runs a task repeatedly — running every few seconds) calls `closeExpired()`. Each listing's `closeIfDue(now)` atomically moves `ACTIVE → CLOSED` and returns the winner only if the reserve is met. Notifications go out after all locks are released.

Design decisions & patterns used

Decision	Why	Alternative rejected
Bid rules live inside `AuctionListing` and run under its lock (a *monitor*: an object whose methods are all `synchronized`)	"Read highest, compare, replace" must be one atomic step. The lock is per listing, so bids on different auctions never wait for each other.	A global lock in the service (all auctions serialised); or `AtomicReference<Bid>` (a reference that can be swapped atomically) + CAS loop (retry compare-and-set until it succeeds) — valid, but the rule set (seller, leader, end time, increment) is easier to get right under one lock.
Refusals are returned (`BidOutcome`), not thrown	A too-low bid is normal business, not an exceptional error (an **exception** is Java's way of aborting a method to signal that something went wrong). The caller also gets the next minimum to show in the UI.	Throwing an exception for every low bid.
Observer for notifications, called outside the lock	Slow email or push calls must not block other bidders. `BidResult` carries a **snapshot** (a copy frozen at one moment) of the accepted bid taken inside the lock, so listeners never read a newer, unrelated value.	Notifying inside `synchronized` (slow, and risks deadlock if a listener calls back).
Minimum increment	Stops 1-cent "wars" and makes the next minimum predictable.	Any higher amount accepted.
Reserve price	Lets a seller refuse to sell too cheaply; checked only at close.	Rejecting bids below the reserve (would reveal it).
`InstantSource` injected	Time-dependent code becomes testable: the demo moves a `ManualClock` forward instead of sleeping.	Calling `Instant.now()` everywhere.
Specification pattern for search	Each criterion is a `Predicate<AuctionListing>`; they combine with `and`. New criteria = one method.	One `search(name, category, min, max, ...)` method whose parameter list keeps growing.
The service is not a Singleton	awesome-low-level-design's version uses a Singleton `AuctionSystem`; we pass one instance around instead so tests can create fresh ones.	Global state shared by all tests.

Java implementation

Files: `Auction.java` (user, bid, status, outcome records, listing, manual clock), `AuctionService.java` (listener, notifier, query, service), `Main.java`.

```java
// File: lld/ch19/auction/Auction.java
package lld.ch19.auction;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Duration;
import java.time.Instant;
import java.time.InstantSource;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

record User(String id, String name, String email) { }

/** One bid. seq breaks ties between bids with the same timestamp. */
record Bid(User bidder, BigDecimal amount, Instant at, long seq) { }

enum AuctionStatus { SCHEDULED, ACTIVE, CLOSED, CANCELLED }

/** Why a bid was refused - returned, not thrown, because refusals are normal business. */
enum BidOutcome { ACCEPTED, AUCTION_NOT_ACTIVE, AUCTION_ENDED, SELLER_CANNOT_BID, TOO_LOW, ALREADY_HIGHEST }

/** Result of placeBid: what happened, the bid that was accepted, who (if anyone) was just outbid. */
record BidResult(BidOutcome outcome, BigDecimal minimumNextBid, Optional<Bid> accepted, Optional<User> outbid) { }

/** Result of closing: sold to a winner, or unsold. */
record CloseResult(AuctionListing listing, Optional<Bid> winningBid, boolean reserveMet) { }

/**
 * One auction. All state changes go through synchronized methods, so the
 * "read highest bid, compare, replace" sequence is atomic: two bidders can never
 * both believe they are the highest.
 */
final class AuctionListing {
    private final String id;
    private final String itemName;
    private final String description;
    private final String category;
    private final User seller;
    private final BigDecimal startingPrice;
    private final BigDecimal minIncrement;
    private final BigDecimal reservePrice;          // hidden minimum; ZERO = no reserve
    private final Instant startsAt;
    private final Instant endsAt;
    private AuctionStatus status = AuctionStatus.SCHEDULED;
    private final List<Bid> bids = new ArrayList<>();
    private Bid highest;

    AuctionListing(String id, String itemName, String description, String category, User seller,
                   BigDecimal startingPrice, BigDecimal minIncrement, BigDecimal reservePrice,
                   Instant startsAt, Duration duration) {
        if (startingPrice.signum() <= 0 || minIncrement.signum() <= 0) throw new IllegalArgumentException("prices
must be > 0");
        this.id = id; this.itemName = itemName; this.description = description; this.category = category;
        this.seller = seller; this.startingPrice = money(startingPrice); this.minIncrement = money(minIncrement);
        this.reservePrice = money(reservePrice); this.startsAt = startsAt; this.endsAt = startsAt.plus(duration);
    }

    /** Lazily moves SCHEDULED -> ACTIVE when the start time has passed. */
    private void refresh(Instant now) {
        if (status == AuctionStatus.SCHEDULED && !now.isBefore(startsAt)) status = AuctionStatus.ACTIVE;
    }

    synchronized BidResult placeBid(User bidder, BigDecimal rawAmount, Instant now, long seq) {
        refresh(now);
        BigDecimal amount = money(rawAmount);
        if (status != AuctionStatus.ACTIVE) return refuse(BidOutcome.AUCTION_NOT_ACTIVE);
        if (!now.isBefore(endsAt)) return refuse(BidOutcome.AUCTION_ENDED);  // closing job may not have run yet
```

```
        if (bidder.id().equals(seller.id())) return refuse(BidOutcome.SELLER_CANNOT_BID);
        if (highest != null && highest.bidder().id().equals(bidder.id())) return
refuse(BidOutcome.ALREADY_HIGHEST);
        if (amount.compareTo(minimumNextBid()) < 0) return refuse(BidOutcome.TOO_LOW);

        Optional<User> outbid = Optional.ofNullable(highest).map(Bid::bidder);
        highest = new Bid(bidder, amount, now, seq);
        bids.add(highest);
        return new BidResult(BidOutcome.ACCEPTED, minimumNextBid(), Optional.of(highest), outbid);
    }

    /** First bid must reach the starting price; later bids must beat the leader by the increment. */
    synchronized BigDecimal minimumNextBid() {
        return highest == null ? startingPrice : highest.amount().add(minIncrement);
    }

    /** ACTIVE -> CLOSED once time is up. Returns empty if not yet due (or already closed). */
    synchronized Optional<CloseResult> closeIfDue(Instant now) {
        refresh(now);
        if (status != AuctionStatus.ACTIVE || now.isBefore(endsAt)) return Optional.empty();
        status = AuctionStatus.CLOSED;
        boolean reserveMet = highest != null && highest.amount().compareTo(reservePrice) >= 0;
        return Optional.of(new CloseResult(this, reserveMet ? Optional.of(highest) : Optional.empty(),
reserveMet));
    }

    /** A seller may withdraw only while nobody has bid. */
    synchronized boolean cancel() {
        if (status == AuctionStatus.CLOSED || status == AuctionStatus.CANCELLED || highest != null) return false;
        status = AuctionStatus.CANCELLED;
        return true;
    }

    private BidResult refuse(BidOutcome why) { return new BidResult(why, minimumNextBid(), Optional.empty(),
Optional.empty()); }

    private static BigDecimal money(BigDecimal b) { return b.setScale(2, RoundingMode.HALF_EVEN); }

    String id() { return id; }
    String itemName() { return itemName; }
    String description() { return description; }
    String category() { return category; }
    User seller() { return seller; }
    BigDecimal startingPrice() { return startingPrice; }
    synchronized AuctionStatus status(Instant now) { refresh(now); return status; }
    Instant endsAt() { return endsAt; }
    synchronized Optional<Bid> highestBid() { return Optional.ofNullable(highest); }
    synchronized BigDecimal currentPrice() { return highest == null ? startingPrice : highest.amount(); }
    synchronized List<Bid> bidHistory() { return List.copyOf(bids); }

    @Override public String toString() { return id + " '" + itemName + "' now " + currentPrice(); }
}

/** A controllable clock for demos and tests (production code passes InstantSource.system()). */
final class ManualClock implements InstantSource {
    private Instant now;
    ManualClock(Instant start) { now = start; }
    @Override public synchronized Instant instant() { return now; }
    synchronized void advance(Duration d) { now = now.plus(d); }
}
```

```java
// File: lld/ch19/auction/AuctionService.java
package lld.ch19.auction;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.InstantSource;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.Predicate;

/** Observer: who wants to hear about bids and results. */
interface AuctionListener {
    default void onNewHighBid(AuctionListing a, Bid bid) { }
    default void onOutbid(AuctionListing a, User outbid, BigDecimal newPrice) { }
    default void onWon(AuctionListing a, Bid winning) { }
    default void onUnsold(AuctionListing a, boolean hadBids) { }
}

final class ConsoleNotifier implements AuctionListener {
    private volatile boolean muted;
    void mute(boolean m) { muted = m; }
    private void say(String to, String msg) { if (!muted) System.out.println("    [notify " + to + "] " + msg); }

    @Override public void onOutbid(AuctionListing a, User u, BigDecimal p) {
        say(u.name(), "you were outbid on " + a.itemName() + ", now " + p);
    }
    @Override public void onWon(AuctionListing a, Bid w) {
        say(w.bidder().name(), "you WON " + a.itemName() + " for " + w.amount());
        say(a.seller().name(), a.itemName() + " sold to " + w.bidder().name());
    }
    @Override public void onUnsold(AuctionListing a, boolean hadBids) {
        say(a.seller().name(), a.itemName() + " unsold" + (hadBids ? " (reserve not met)" : " (no bids)"));
    }
}

/** Search criteria, built up fluently. Each criterion is a Predicate (Specification pattern). */
final class AuctionQuery {
    private final List<Predicate<AuctionListing>> parts = new ArrayList<>();
    AuctionQuery nameContains(String s) {
        parts.add(a -> a.itemName().toLowerCase(Locale.ROOT).contains(s.toLowerCase(Locale.ROOT))); return this;
    }
    AuctionQuery category(String c) { parts.add(a -> a.category().equalsIgnoreCase(c)); return this; }
    AuctionQuery priceBetween(BigDecimal lo, BigDecimal hi) {
        parts.add(a -> a.currentPrice().compareTo(lo) >= 0 && a.currentPrice().compareTo(hi) <= 0); return this;
    }
    Predicate<AuctionListing> toPredicate() { return parts.stream().reduce(x -> true, Predicate::and); }
}

/** Facade over listings, bidding and closing. Listeners are called OUTSIDE listing locks. */
final class AuctionService {
    private final InstantSource clock;
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final Map<String, AuctionListing> listings = new ConcurrentHashMap<>();
    private final List<AuctionListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger listingSeq = new AtomicInteger();
    private final AtomicInteger userSeq = new AtomicInteger();
    private final AtomicLong bidSeq = new AtomicLong();

    AuctionService(InstantSource clock) { this.clock = clock; }
```

```
    void subscribe(AuctionListener l) { listeners.add(l); }

    User register(String name, String email) {
        User u = new User("U" + userSeq.incrementAndGet(), name, email);
        if (users.putIfAbsent(u.id(), u) != null) throw new IllegalStateException("id clash");
        return u;
    }

    AuctionListing createListing(User seller, String item, String description, String category,
                                 BigDecimal startingPrice, BigDecimal increment, BigDecimal reserve, Duration
duration) {
        AuctionListing a = new AuctionListing("A" + listingSeq.incrementAndGet(), item, description, category,
seller,
                startingPrice, increment, reserve, clock.instant(), duration);
        listings.put(a.id(), a);
        return a;
    }

    List<AuctionListing> search(AuctionQuery q) {
        return listings.values().stream()
                .filter(a -> a.status(clock.instant()) == AuctionStatus.ACTIVE)
                .filter(q.toPredicate())
                .sorted(Comparator.comparing(AuctionListing::endsAt))
                .toList();
    }

    BidResult placeBid(String listingId, User bidder, BigDecimal amount) {
        AuctionListing a = listings.get(listingId);
        if (a == null) throw new IllegalArgumentException("no listing " + listingId);
        BidResult r = a.placeBid(bidder, amount, clock.instant(), bidSeq.incrementAndGet());
        r.accepted().ifPresent(top -> {                 // snapshot taken inside the lock
            listeners.forEach(l -> l.onNewHighBid(a, top));
            r.outbid().ifPresent(u -> listeners.forEach(l -> l.onOutbid(a, u, top.amount())));
        });
        return r;
    }

    /** Run periodically (e.g. by a ScheduledExecutorService). Closes every auction whose time is up. */
    List<CloseResult> closeExpired() {
        List<CloseResult> closed = new ArrayList<>();
        for (AuctionListing a : listings.values()) {
            Optional<CloseResult> r = a.closeIfDue(clock.instant());
            r.ifPresent(closed::add);
        }
        for (CloseResult r : closed) {
            if (r.winningBid().isPresent()) listeners.forEach(l -> l.onWon(r.listing(), r.winningBid().get()));
            else listeners.forEach(l -> l.onUnsold(r.listing(), r.listing().highestBid().isPresent()));
        }
        return closed;
    }
}
```

```java
// File: lld/ch19/auction/Main.java
package lld.ch19.auction;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        ManualClock clock = new ManualClock(Instant.parse("2026-09-28T10:00:00Z"));
        AuctionService svc = new AuctionService(clock);
        ConsoleNotifier notifier = new ConsoleNotifier();
        svc.subscribe(notifier);

        User sam = svc.register("Sam", "sam@mail.test");
        User ana = svc.register("Ana", "ana@mail.test");
        User bob = svc.register("Bob", "bob@mail.test");

        System.out.println("== 1. Create listings ==");
        AuctionListing guitar = svc.createListing(sam, "Vintage Guitar", "1965 hollow body", "Music",
                bd("100"), bd("10"), bd("0"), Duration.ofHours(2));
        AuctionListing watch = svc.createListing(sam, "Pocket Watch", "Silver, working", "Collectibles",
                bd("50"), bd("5"), bd("500"), Duration.ofHours(1));
        System.out.println("  " + guitar + ", " + watch);

        System.out.println("\n== 2. Search (name 'watch', price 0-100) ==");
        System.out.println("  " + svc.search(new AuctionQuery().nameContains("watch").priceBetween(bd("0"),
bd("100"))));

        System.out.println("\n== 3. Bidding rules ==");
        show("Ana bids 90 (below start)", svc.placeBid(guitar.id(), ana, bd("90")));
        show("Ana bids 100", svc.placeBid(guitar.id(), ana, bd("100")));
        show("Ana bids 120 (already top)", svc.placeBid(guitar.id(), ana, bd("120")));
        show("Bob bids 105 (increment is 10)", svc.placeBid(guitar.id(), bob, bd("105")));
        show("Bob bids 110", svc.placeBid(guitar.id(), bob, bd("110")));
        show("Sam bids on own item", svc.placeBid(guitar.id(), sam, bd("500")));
        show("Ana bids 150", svc.placeBid(guitar.id(), ana, bd("150")));
        show("Bob bids 60 on the watch", svc.placeBid(watch.id(), bob, bd("60")));

        System.out.println("\n== 4. One hour later: the watch closes, reserve (500) not met ==");
        clock.advance(Duration.ofHours(1));
        svc.closeExpired().forEach(r -> System.out.println("  closed " + r.listing().id() + " reserveMet=" +
r.reserveMet()));

        System.out.println("\n== 5. 50 threads bid on the guitar at once ==");
        List<User> crowd = new ArrayList<>();
        for (int i = 0; i < 50; i++) crowd.add(svc.register("bidder" + i, "b" + i + "@mail.test"));
        notifier.mute(true);                // ~50 outbid messages would drown the output
        AtomicInteger accepted = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        ExecutorService pool = Executors.newFixedThreadPool(16);
        for (int i = 0; i < crowd.size(); i++) {
            User u = crowd.get(i);
            BigDecimal amount = bd(String.valueOf(160 + i * 10));
            pool.submit(() -> {
                go.await();
                if (svc.placeBid(guitar.id(), u, amount).outcome() == BidOutcome.ACCEPTED)
accepted.incrementAndGet();
                return null;
            });
        }
```

```
        go.countDown();
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
        notifier.mute(false);
        Bid top = guitar.highestBid().orElseThrow();
        boolean increasing = true;
        List<Bid> history = guitar.bidHistory();
        for (int i = 1; i < history.size(); i++)
            increasing &= history.get(i).amount().compareTo(history.get(i - 1).amount().add(bd("10"))) >= 0;
        System.out.println("  accepted=" + accepted + " (varies run to run), highest=" + top.amount() + " by " +
top.bidder().name()
                + ", history strictly increasing by >= increment: " + increasing);

        System.out.println("\n== 6. Time is up: the guitar closes and the winner is told ==");
        clock.advance(Duration.ofHours(1));
        show("late bid", svc.placeBid(guitar.id(), ana, bd("5000")));
        svc.closeExpired();
        System.out.println("  status " + guitar.status(clock.instant()) + ", final price " +
guitar.currentPrice());
    }

    private static BigDecimal bd(String s) { return new BigDecimal(s); }

    private static void show(String what, BidResult r) {
        System.out.println("  " + what + " -> " + r.outcome() + " (next minimum " + r.minimumNextBid() + ")");
    }
}
```

Output:

```
== 1. Create listings ==
  A1 'Vintage Guitar' now 100.00, A2 'Pocket Watch' now 50.00

== 2. Search (name 'watch', price 0-100) ==
  [A2 'Pocket Watch' now 50.00]

== 3. Bidding rules ==
  Ana bids 90 (below start) -> TOO_LOW (next minimum 100.00)
  Ana bids 100 -> ACCEPTED (next minimum 110.00)
  Ana bids 120 (already top) -> ALREADY_HIGHEST (next minimum 110.00)
  Bob bids 105 (increment is 10) -> TOO_LOW (next minimum 110.00)
    [notify Ana] you were outbid on Vintage Guitar, now 110.00
  Bob bids 110 -> ACCEPTED (next minimum 120.00)
  Sam bids on own item -> SELLER_CANNOT_BID (next minimum 120.00)
    [notify Bob] you were outbid on Vintage Guitar, now 150.00
  Ana bids 150 -> ACCEPTED (next minimum 160.00)
  Bob bids 60 on the watch -> ACCEPTED (next minimum 65.00)

== 4. One hour later: the watch closes, reserve (500) not met ==
    [notify Sam] Pocket Watch unsold (reserve not met)
  closed A2 reserveMet=false

== 5. 50 threads bid on the guitar at once ==
  accepted=28 (varies run to run), highest=650.00 by bidder49, history strictly increasing by >= increment: true

== 6. Time is up: the guitar closes and the winner is told ==
  late bid -> AUCTION_ENDED (next minimum 660.00)
    [notify bidder49] you WON Vintage Guitar for 650.00
    [notify Sam] Vintage Guitar sold to bidder49
  status CLOSED, final price 650.00
```

Reading the output. Step 3 exercises every rule: below start, leader re-bidding, below increment, seller bidding. The outbid notices print *before* the bidder's own result line because the service notifies before returning. Step 4 closes the watch as unsold because 60 < the 500 reserve. Step 5 fires 50 concurrent bids: how many are accepted depends on thread timing (a bid that arrives after a higher one is refused as too low), but the final highest is always the maximum, 650.00, and the history is always strictly increasing by at least the

increment — the invariant (a rule that must always be true) the lock protects. Step 6 refuses a bid after the end time even though the closing job has not yet run.

Concurrency & edge cases

Case	Handling
Two bidders submit the same amount at the same time	The first one through the lock wins; the second is now below `minimumNextBid()` and gets `TOO_LOW`.
Bid arrives exactly at the end time	`!now.isBefore(endsAt)` → `AUCTION_ENDED`. The end instant is exclusive.
Close job runs late	Bids after `endsAt` are refused anyway, so a late job cannot change the winner.
Close and bid race	Both are `synchronized` on the listing; the bid either lands before the close (and counts) or after (and is refused).
Notification listener is slow or throws	Called outside the lock; in production, publish to a queue so failures are retried without affecting bidding.
Seller cancels after bids	Refused (`cancel()` returns false) — protects bidders.
Money precision	Amounts normalised to scale 2 with `BigDecimal`; compared with `compareTo`.
Very hot auction (thousands of bids per second)	One lock per listing is fine for one JVM (Java Virtual Machine — one running Java process); distributed: a single-writer per auction (partition bids by auction id — split them into separate streams so all bids for one auction go to the same place, e.g. a Kafka partition, Kafka being a widely used event-streaming system), or a conditional update `UPDATE ... SET high = ? WHERE id = ? AND high < ?`.

Real-world example

eBay auctions use **proxy bidding**: you enter the most you are willing to pay, and eBay bids on your behalf only as much as needed to stay ahead, by the current increment. eBay also supports hidden **reserve prices**, and its auctions end at a fixed time, which is why "sniping" (bidding in the last seconds) is common there. Some other auction sites use a **soft close** — a late bid extends the end time — precisely to defeat sniping.

Extensions & follow-up questions

- **"Add proxy (automatic) bidding."** Each bidder may register a `maxAmount`. After any accepted bid, inside the same lock, let the strongest proxy respond with `min(its max, competitor + increment)`. Ties go to the earlier proxy (use `Bid.seq`).
- **"Stop sniping."** Soft close: in `placeBid`, if an accepted bid arrives within the last N minutes, set `endsAt = endsAt + N` (make `endsAt` non-final, change under the same lock).
- **"Buy It Now."** A fixed price that closes the auction immediately if paid before the first bid.
- **"Dutch / sealed-bid / Vickrey auctions."** (Dutch: the price starts high and drops until someone accepts; sealed-bid: bidders cannot see each other's bids.) Extract an `AuctionRules` strategy: `validate(bid)`, `winner(bids)`, `price(bids)`. Vickrey (second-price sealed bid): highest bidder wins but pays the second-highest bid.
- **"Payment after winning."** Winner has 48 hours to pay (new state `AWAITING_PAYMENT`); if not, offer to the second-highest bidder.
- **"Scale the close job."** Keep a priority queue (or a sorted index) by `endsAt` so the scheduler only touches auctions that are due.

In the interview

30–45 minute plan

1. Clarify the bid rules (start price, increment, reserve, seller/leader restrictions) — 4 min.
2. Entities and the lifecycle state machine — 8 min.
3. Code `placeBid` under the listing lock and **say why the lock is per listing** — 12 min.
4. Closing + notifications outside the lock — 6 min.

5. Concurrency discussion and extensions (proxy bidding, soft close) — 10 min.

> **30-second answer:** "Each listing owns its bid rules and its highest bid, and `placeBid` is synchronized on the listing, so compare-and-replace is atomic and different auctions never contend. Refusals return a reason and the next minimum. The service notifies outbid users and, at close, the winner and seller, always after releasing the lock. Time comes from an injected clock, so closing and late bids are testable."

Follow-ups with answers

- *Why not `AtomicReference` + CAS?* Possible for the amount alone, but the rule also checks seller, leader and time. A tiny critical section under one lock is simpler and still fast because the lock is per listing.
- *How do you close auctions on time across servers?* A scheduler that owns a time-ordered index of end times; the close itself is idempotent (`closeIfDue` returns empty the second time), so running it twice is harmless.
- *How do you show a live price to thousands of viewers?* Push `onNewHighBid` events to a pub/sub channel (publish/subscribe: senders post messages to a topic and every subscriber receives them) and WebSockets (a connection the browser keeps open so the server can push data at any time); viewers read a cache (a fast saved copy of data — see System Design Fundamentals), not the lock-holding object.

Self-check

1. **Q:** What two conditions make a bid `TOO_LOW`? **A:** Below the starting price when there is no bid, or below highest + increment otherwise.
2. **Q:** Why are listeners called after `placeBid` returns from the listing? **A:** To keep the lock short and avoid deadlock or slowdown from notification code.
3. **Q:** A bid arrives at 12:00:00.000 and the auction ends at 12:00:00.000. Accepted? **A:** No — the end time is exclusive.
4. **Q:** Where does the reserve price matter? **A:** Only at close: if the highest bid is below it, the item is unsold.
5. **Q:** What is the Specification pattern here? **A:** Search criteria as composable `Predicate`s combined with `and`.

TL;DR

- One **lock per listing**; all bid rules inside it; refusals are values, not exceptions.
- Lifecycle `SCHEDULED → ACTIVE → CLOSED | CANCELLED`; activation is lazy, closing is idempotent.
- **Observer** notifications after the lock; **injected clock** for testable time; **Specification** for search.

Design a Music Streaming Service (Spotify)

Problem & clarifying questions

A music streaming service lets users search a huge catalog, build playlists, and play songs with the usual controls (play, pause, skip, previous, seek, shuffle, repeat). Free and paid plans behave differently. The service suggests new music.

Question	Assumed answer
What can users search?	Songs, albums, artists.
Playlists?	Users create, reorder and edit their own playlists.
Playback controls?	Play, pause, resume, skip, previous, seek; shuffle on/off; repeat off / all / one.
Plans?	FREE (ads, limited skips per hour, no downloads) and PREMIUM (no ads, unlimited skips, offline).
Recommendations?	A stub (a simple placeholder) based on listening history; real ones are an ML (machine learning — software that learns patterns from data) system.
Actual audio streaming, DRM (copy protection), CDNs (content delivery networks — servers near users that hold copies of files)?	Out of scope for LLD (low-level design: classes and their code) — the player models *what* plays, not the bytes.

Requirements

Functional

1. Browse and search songs, albums and artists.
2. Create and manage playlists.
3. User accounts and authentication (we model users and plans; login itself is the usual auth service).
4. Playback controls: play, pause, skip, seek (plus previous, shuffle, repeat).
5. Recommendations based on preferences and listening history.
6. Subscriptions: FREE vs PREMIUM with different limits.

Non-functional

7. Many users stream at once; the shared catalog is read-mostly and thread-safe (works correctly when many threads use it at once).
8. Scales to a large catalog and user base (indexes, not scans, in production).
9. Extensible: social sharing, offline mode, new play orders, new recommenders.

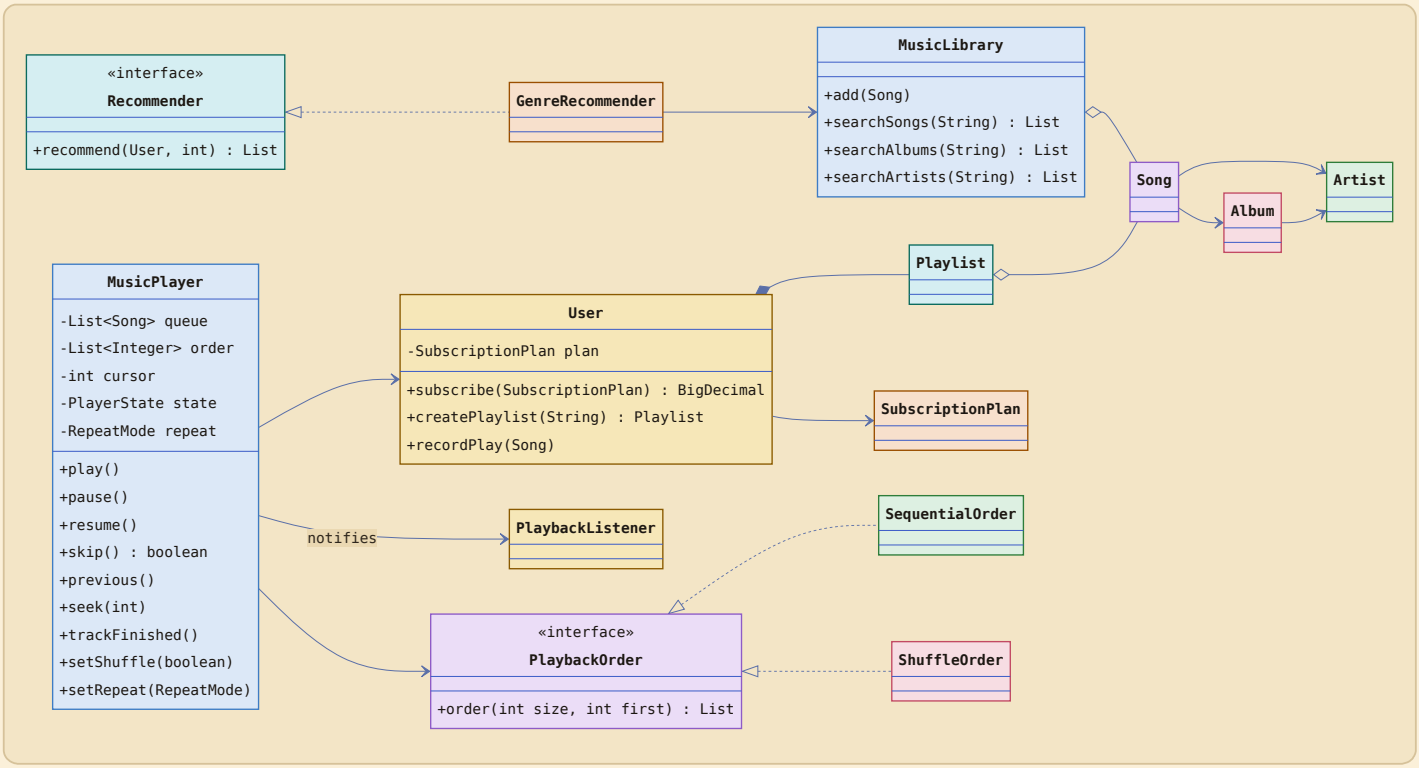
Use cases

Actor	Use cases
Listener	Search, play/pause/skip/seek, shuffle/repeat, create and edit playlists, upgrade plan
Artist / label (content admin)	Add songs and albums to the catalog
System	Record plays (history, royalties — money owed to artists per play), insert ads for free users, recommend songs

Core entities

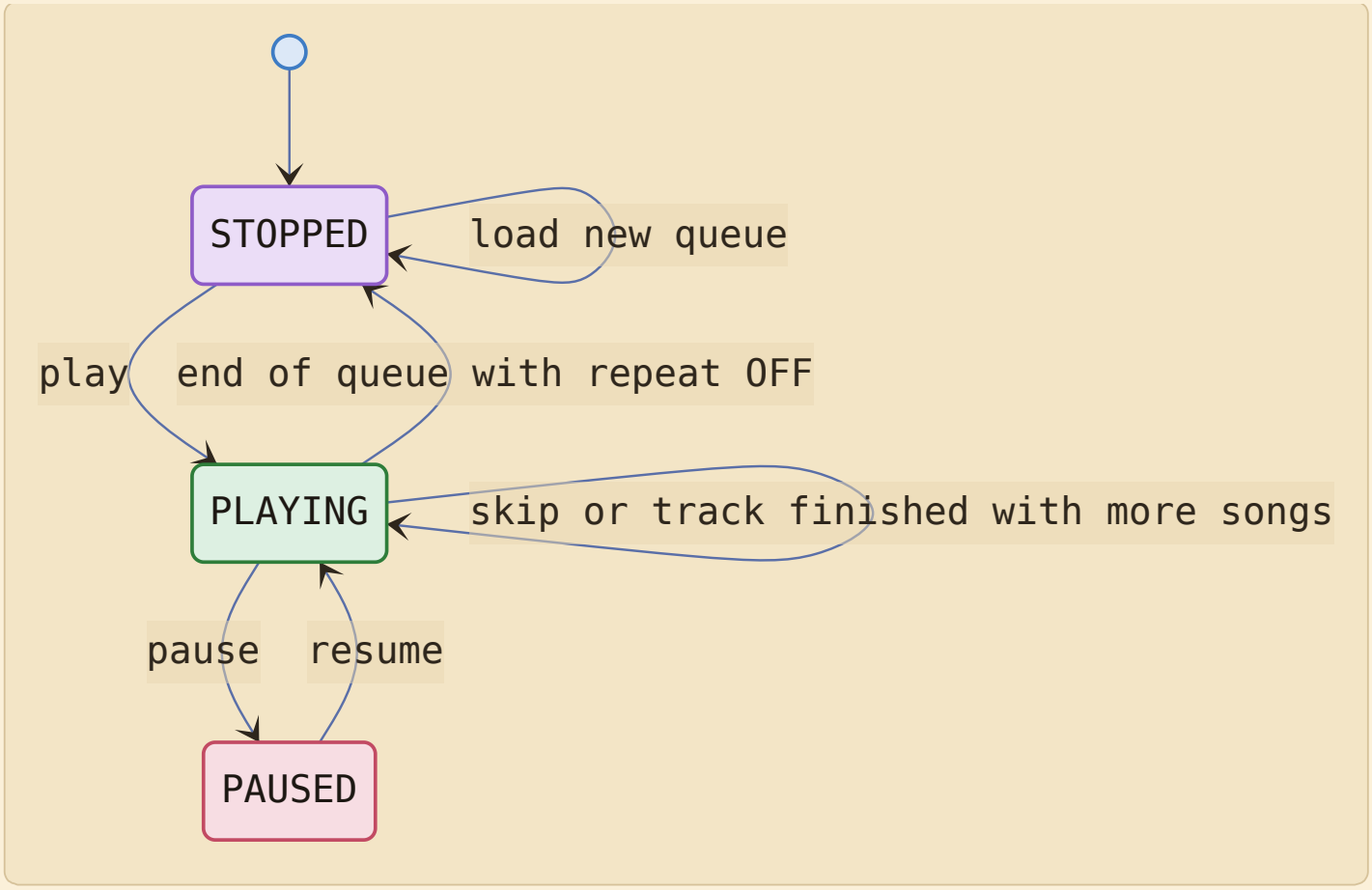
Class	Responsibility
`Artist`, `Album`, `Song`, `Genre`	Immutable catalog records.
`MusicLibrary`	Thread-safe catalog with search by song title, album title, artist name.
`SubscriptionPlan`	Enum that **carries the rules**: monthly price, skips per hour, ads, offline downloads.
`User`	Name, current plan, playlists, listening history.
`Playlist`	Ordered, user-owned song list: add, remove, move.
`PlaybackOrder` → `SequentialOrder`, `ShuffleOrder`	Strategy: produces the play order (a permutation — every song exactly once, in some order) starting from the current song.
`RepeatMode`, `PlayerState`	`OFF / ALL / ONE`; `STOPPED / PLAYING / PAUSED`.
`MusicPlayer`	One user's session: queue, play order, cursor (where we are in the play order), position, state, skip window, ad counter.
`PlaybackListener`	Observer on "song started" (history, royalties, recommendations).
`Recommender` → `GenreRecommender`	Strategy: suggest unplayed songs from the user's favourite genres.

Class diagram

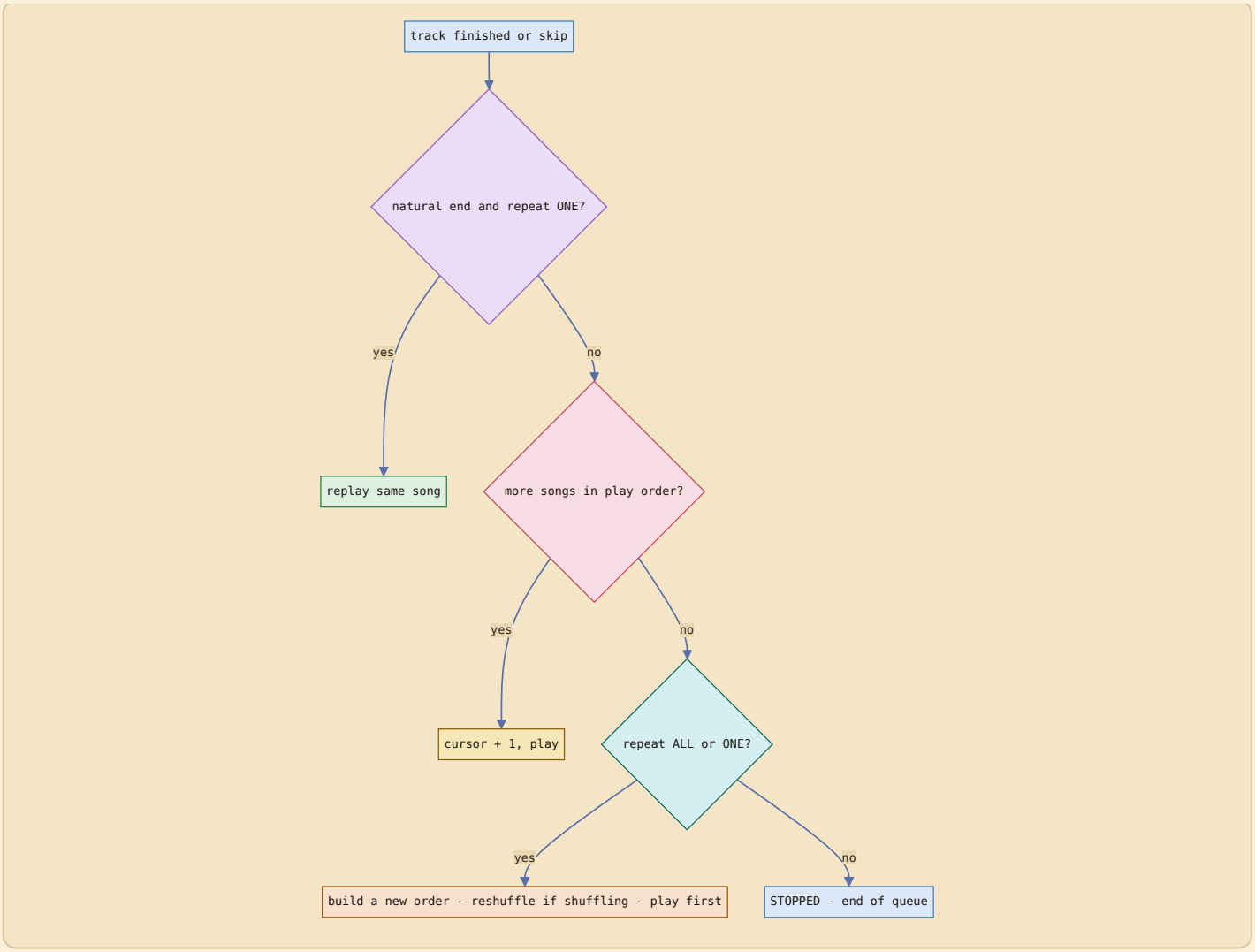


Key flows

Player states



What happens when a song ends or the user presses Next



Skip first checks the plan's rolling (always the last 60 minutes, not a fixed clock hour) one-hour limit (a deque — a queue you can add to or remove from at both ends — of skip timestamps; drop those older than an hour, compare the size with `skipsPerHour`). Pressing Next on repeat-ONE moves on — like real players, repeat-one only affects songs that end naturally.

Shuffle keeps the current song. Turning shuffle on builds a new permutation whose first element is the song playing now, and shuffles the rest with `Collections.shuffle(list, random)`. Turning it off rebuilds the sequential order starting from the current song. The queue itself never changes — only the **order** of indexes into it — so switching back and forth loses nothing.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for play order (`PlaybackOrder`)	Shuffle, sequential, "smart shuffle" (spread the same artist apart) are interchangeable algorithms. The player just asks for an order.	`if (shuffle) ... else ...` inside `skip`, `previous`, `wrap`.
Order as a list of indexes into an unchanged queue	Turning shuffle off/on never loses the original order; `upNext` is a simple walk.	Physically shuffling the queue list (original order lost).
Seeded `Random`	Repeatable demos and tests. Production uses an unseeded (or per-session) seed.	`Math.random()` (not testable).
Plan rules inside `SubscriptionPlan` enum	`user.plan().skipsPerHour` and `.ads` — no `if (plan == FREE)` scattered around; a new plan is one enum constant (or a class if rules get complex).	Boolean flags on `User`.
Observer for plays (`PlaybackListener`)	History, royalty counting and recommendation features all react to "song started" without the player knowing them.	Player calling each subsystem directly.
Strategy for recommendations	`Recommender` can be genre-based today, collaborative filtering ("people who liked what you like also liked…") tomorrow.	Hard-coding one algorithm in `User`.
One `MusicPlayer` per user session, all methods `synchronized`	Commands can arrive from two devices (phone + desktop "connect"). The shared `MusicLibrary` uses concurrent maps.	A single shared player object.
State as an enum, not the State pattern	Three states with trivial behaviour; the GoF ("Gang of Four", the authors of the classic 1994 *Design Patterns* book) State pattern (one class per state) pays off only when each state has substantial, different behaviour.	Three state classes for four one-line methods.
Singletons?	awesome-low-level-design makes `MusicLibrary`, `UserManager` and `MusicRecommender` Singletons. We inject single instances instead — same sharing, easier tests.	Global `getInstance()`.

Java implementation

Files: `Catalog.java` (genre, artist, album, song, library, plans, user, playlist), `Player.java` (play-order strategies, repeat/state enums, listener, player, recommender), `Main.java`.

```java
// File: lld/ch19/music/Catalog.java
package lld.ch19.music;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.function.Function;

enum Genre { ROCK, POP, JAZZ, CLASSICAL, HIPHOP }

record Artist(String id, String name) { }

record Album(String id, String title, Artist artist, int year) { }

record Song(String id, String title, Artist artist, Album album, Genre genre, int durationSec) {
    @Override public String toString() { return title + " - " + artist.name(); }
}

/** The shared, read-mostly catalog. Thread-safe collections because every user reads it at once. */
final class MusicLibrary {
    private final Map<String, Song> songs = new ConcurrentHashMap<>();
    private final Map<String, Album> albums = new ConcurrentHashMap<>();
    private final Map<String, Artist> artists = new ConcurrentHashMap<>();

    void add(Song s) {
        songs.put(s.id(), s);
        albums.putIfAbsent(s.album().id(), s.album());
        artists.putIfAbsent(s.artist().id(), s.artist());
    }

    Song song(String id) { return songs.get(id); }
    List<Song> allSongs() { return sorted(songs.values(), Song::id); }

    List<Song> searchSongs(String q) { return match(songs.values(), q, Song::title); }
    List<Album> searchAlbums(String q) { return match(albums.values(), q, Album::title); }
    List<Artist> searchArtists(String q) { return match(artists.values(), q, Artist::name); }
    List<Song> byArtist(Artist a) { return songs.values().stream().filter(s -> s.artist().equals(a)).sorted(
            (x, y) -> x.id().compareTo(y.id())).toList(); }

    private static <T> List<T> match(Iterable<T> items, String q, Function<T, String> field) {
        String needle = q.toLowerCase(Locale.ROOT);
        List<T> out = new ArrayList<>();
        for (T t : items) if (field.apply(t).toLowerCase(Locale.ROOT).contains(needle)) out.add(t);
        out.sort((a, b) -> field.apply(a).compareTo(field.apply(b)));
        return out;
    }

    private static <T> List<T> sorted(Iterable<T> items, Function<T, String> key) {
        List<T> out = new ArrayList<>();
        items.forEach(out::add);
        out.sort((a, b) -> key.apply(a).compareTo(key.apply(b)));
        return out;
    }
}

/** What each plan allows. The enum carries the rules, so no if (plan == FREE) scattered around. */
enum SubscriptionPlan {
    FREE(new BigDecimal("0.00"), 6, true, false),
    PREMIUM(new BigDecimal("10.99"), Integer.MAX_VALUE, false, true);

    final BigDecimal monthlyPrice;
    final int skipsPerHour;
    final boolean ads;
```

```
    final boolean offlineDownloads;

    SubscriptionPlan(BigDecimal price, int skipsPerHour, boolean ads, boolean offline) {
        this.monthlyPrice = price; this.skipsPerHour = skipsPerHour; this.ads = ads; this.offlineDownloads =
offline;
    }
}

final class User {
    private final String id;
    private final String name;
    private volatile SubscriptionPlan plan = SubscriptionPlan.FREE;
    private final List<Playlist> playlists = new CopyOnWriteArrayList<>();
    private final List<Song> history = new CopyOnWriteArrayList<>();

    User(String id, String name) { this.id = id; this.name = name; }
    String id() { return id; }
    String name() { return name; }
    SubscriptionPlan plan() { return plan; }

    /** Returns the amount to charge now. */
    BigDecimal subscribe(SubscriptionPlan p) { plan = p; return p.monthlyPrice; }

    Playlist createPlaylist(String name) { Playlist p = new Playlist(name, this); playlists.add(p); return p; }
    List<Playlist> playlists() { return List.copyOf(playlists); }
    void recordPlay(Song s) { history.add(s); }
    List<Song> history() { return Collections.unmodifiableList(history); }
}

/** An ordered, user-owned list of songs. Duplicates are allowed, as in real apps. */
final class Playlist {
    private final String name;
    private final User owner;
    private final List<Song> songs = new ArrayList<>();

    Playlist(String name, User owner) { this.name = name; this.owner = owner; }
    String name() { return name; }
    User owner() { return owner; }
    synchronized Playlist add(Song s) { songs.add(s); return this; }
    synchronized void remove(Song s) { songs.remove(s); }
    synchronized void move(int from, int to) { songs.add(to, songs.remove(from)); }
    synchronized List<Song> songs() { return List.copyOf(songs); }
}
```

```java
// File: lld/ch19/music/Main.java
package lld.ch19.music;

import java.time.Instant;
import java.time.InstantSource;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        MusicLibrary lib = new MusicLibrary();
        Artist queen = new Artist("AR1", "Queen");
        Artist miles = new Artist("AR2", "Miles Davis");
        Artist adele = new Artist("AR3", "Adele");
        Album opera = new Album("AL1", "A Night at the Opera", queen, 1975);
        Album blue = new Album("AL2", "Kind of Blue", miles, 1959);
        Album twentyFive = new Album("AL3", "25", adele, 2015);
        lib.add(new Song("S1", "Bohemian Rhapsody", queen, opera, Genre.ROCK, 354));
        lib.add(new Song("S2", "Love of My Life", queen, opera, Genre.ROCK, 219));
        lib.add(new Song("S3", "So What", miles, blue, Genre.JAZZ, 562));
        lib.add(new Song("S4", "Blue in Green", miles, blue, Genre.JAZZ, 337));
        lib.add(new Song("S5", "Hello", adele, twentyFive, Genre.POP, 295));
        lib.add(new Song("S6", "Death on Two Legs", queen, opera, Genre.ROCK, 223));
        lib.add(new Song("S7", "Freddie Freeloader", miles, blue, Genre.JAZZ, 589));
        lib.add(new Song("S8", "All Blues", miles, blue, Genre.JAZZ, 693));
        lib.add(new Song("S9", "When We Were Young", adele, twentyFive, Genre.POP, 291));
        lib.add(new Song("S10", "You're My Best Friend", queen, opera, Genre.ROCK, 172));

        System.out.println("== 1. Search ==");
        System.out.println("  songs 'blue'    -> " + lib.searchSongs("blue"));
        System.out.println("  albums 'opera'  -> " +
lib.searchAlbums("opera").stream().map(Album::title).toList());
        System.out.println("  artists 'da'    -> " + lib.searchArtists("da").stream().map(Artist::name).toList());

        System.out.println("\n== 2. Playlist ==");
        User mia = new User("U1", "Mia");
        Playlist mix = mia.createPlaylist("Road trip")

.add(lib.song("S1")).add(lib.song("S3")).add(lib.song("S5")).add(lib.song("S2")).add(lib.song("S4"));
        mix.move(4, 1);
        System.out.println("  " + mix.name() + ": " + mix.songs());

        System.out.println("\n== 3. Play in order, pause, seek, repeat-one ==");
        MusicPlayer player = new MusicPlayer(mia, InstantSource.fixed(Instant.parse("2026-09-28T08:00:00Z")));
        player.addListener((u, s) -> u.recordPlay(s));          // history is just another listener
        player.load(mix.songs());
        player.play();
        player.seek(120);
        player.pause();
        System.out.println("  state " + player.state() + " at " + player.position() + "s");
        player.resume();
        player.setRepeat(RepeatMode.ONE);
        player.trackFinished();                                  // replays the same song
        player.setRepeat(RepeatMode.OFF);
        player.trackFinished();

        System.out.println("\n== 4. Shuffle keeps the current song, reorders the rest ==");
        player.setShuffle(true);
        System.out.println("  now " + player.current() + ", up next " + player.upNext(4));

        System.out.println("\n== 5. FREE plan: skip limit (6/hour) and ads, repeat-all wraps the shuffled queue
==");
        player.setRepeat(RepeatMode.ALL);
        for (int i = 0; i < 7; i++) player.skip();

        System.out.println("\n== 6. Upgrade to PREMIUM: no ads, unlimited skips ==");
        System.out.println("  charged $" + mia.subscribe(SubscriptionPlan.PREMIUM));
        player.setShuffle(false);
        player.load(List.of(lib.song("S6"), lib.song("S7")));
```

```
        player.setRepeat(RepeatMode.ALL);
        player.play();
        player.skip();
        player.skip();                                          // wraps to the start
        player.setRepeat(RepeatMode.OFF);
        player.trackFinished();
        player.trackFinished();                                 // end of queue

        System.out.println("\n== 7. Recommendations from listening history ==");
        System.out.println("  plays so far: " + mia.history().size());
        System.out.println("  recommended: " + new GenreRecommender(lib).recommend(mia, 3));
    }
}
```

```java
// File: lld/ch19/music/Player.java
package lld.ch19.music;

import java.time.Duration;
import java.time.Instant;
import java.time.InstantSource;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.Random;
import java.util.stream.IntStream;

/** Strategy: in what order should the queue be played? */
interface PlaybackOrder {
    /** A permutation of 0..size-1 that starts with {@code first}. */
    List<Integer> order(int size, int first);
}

final class SequentialOrder implements PlaybackOrder {
    @Override public List<Integer> order(int size, int first) {
        List<Integer> out = new ArrayList<>(IntStream.range(0, size).boxed().toList());
        Collections.rotate(out, -first);        // start at the current track, keep the rest in order
        return out;
    }
}

final class ShuffleOrder implements PlaybackOrder {
    private final Random random;
    ShuffleOrder(long seed) { this.random = new Random(seed); }   // seeded: repeatable demo and tests
    @Override public List<Integer> order(int size, int first) {
        List<Integer> rest = new ArrayList<>(IntStream.range(0, size).filter(i -> i != first).boxed().toList());
        Collections.shuffle(rest, random);
        rest.add(0, first);                     // the song you are hearing keeps playing
        return rest;
    }
}

enum RepeatMode { OFF, ALL, ONE }

enum PlayerState { STOPPED, PLAYING, PAUSED }

/** Observer hook: history, royalties, recommendations all listen to plays. */
interface PlaybackListener { void onStart(User user, Song song); }

/**
 * One user's player. Not shared between users, but a phone and a web tab could send
 * commands at once, so every public method is synchronized.
 */
final class MusicPlayer {
    private final User user;
    private final InstantSource clock;
    private final List<PlaybackListener> listeners = new ArrayList<>();
    private final Deque<Instant> recentSkips = new ArrayDeque<>();
    private List<Song> queue = List.of();
    private List<Integer> order = List.of();
    private int cursor;                         // index into order
    private int positionSec;
    private int songsSinceAd;
    private PlayerState state = PlayerState.STOPPED;
    private RepeatMode repeat = RepeatMode.OFF;
    private PlaybackOrder playbackOrder = new SequentialOrder();

    MusicPlayer(User user, InstantSource clock) { this.user = user; this.clock = clock; }
```

```
    synchronized void addListener(PlaybackListener l) { listeners.add(l); }

    synchronized void load(List<Song> songs) {
        queue = List.copyOf(songs);
        order = playbackOrder.order(queue.size(), 0);
        cursor = 0;
        state = PlayerState.STOPPED;
    }

    synchronized void play() {
        if (queue.isEmpty()) throw new IllegalStateException("queue is empty");
        start();
    }

    synchronized void pause() { if (state == PlayerState.PLAYING) state = PlayerState.PAUSED; }

    synchronized void resume() { if (state == PlayerState.PAUSED) state = PlayerState.PLAYING; }

    synchronized void seek(int sec) {
        positionSec = Math.max(0, Math.min(sec, current().durationSec()));
    }

    /** User pressed "next". Free users have a skip limit per rolling hour. */
    synchronized boolean skip() {
        if (state == PlayerState.STOPPED) return false;          // nothing to skip
        Instant now = clock.instant();
        while (!recentSkips.isEmpty() && recentSkips.peekFirst().isBefore(now.minus(Duration.ofHours(1))))
recentSkips.pollFirst();
        if (recentSkips.size() >= user.plan().skipsPerHour) {
            System.out.println("  [player] skip limit reached on " + user.plan() + " plan");
            return false;
        }
        recentSkips.addLast(now);
        advance(false);
        return true;
    }

    /** The current song ended naturally. Repeat-one replays it; skip does not honour repeat-one. */
    synchronized void trackFinished() { advance(true); }

    synchronized void previous() {
        if (positionSec > 3 || cursor == 0) { positionSec = 0; return; }   // like most players: restart first
        cursor--;
        start();
    }

    synchronized void setShuffle(boolean on) {
        int now = order.isEmpty() ? 0 : order.get(cursor);
        playbackOrder = on ? new ShuffleOrder(42) : new SequentialOrder();
        order = playbackOrder.order(queue.size(), now);
        cursor = 0;
    }

    synchronized void setRepeat(RepeatMode m) { repeat = m; }

    synchronized Song current() { return queue.get(order.get(cursor)); }
    synchronized PlayerState state() { return state; }
    synchronized int position() { return positionSec; }

    /** The next few songs in play order (what the "Up next" panel shows). */
    synchronized List<Song> upNext(int n) {
        List<Song> out = new ArrayList<>();
        for (int i = cursor + 1; i < order.size() && out.size() < n; i++) out.add(queue.get(order.get(i)));
        return out;
    }

    private void advance(boolean natural) {
        if (natural && repeat == RepeatMode.ONE) { start(); return; }
        if (cursor + 1 < order.size()) { cursor++; start(); return; }
```

```
        if (repeat == RepeatMode.ALL || repeat == RepeatMode.ONE) {   // wrap around (reshuffle if shuffling)
            order = playbackOrder.order(queue.size(), order.get(0));
            cursor = 0;
            start();
            return;
        }
        state = PlayerState.STOPPED;
        System.out.println("  [player] end of queue");
    }

    private void start() {
        if (user.plan().ads && ++songsSinceAd > 3) {
            System.out.println("  [player] (30-second ad for FREE plan)");
            songsSinceAd = 1;
        }
        positionSec = 0;
        state = PlayerState.PLAYING;
        Song s = current();
        System.out.println("  [player] > " + s);
        listeners.forEach(l -> l.onStart(user, s));
    }
}

/** Strategy for "what should this user hear next?" - a simple stub based on favourite genre. */
interface Recommender { List<Song> recommend(User user, int n); }

final class GenreRecommender implements Recommender {
    private final MusicLibrary library;
    GenreRecommender(MusicLibrary library) { this.library = library; }

    @Override public List<Song> recommend(User user, int n) {
        Map<Genre, Integer> counts = new EnumMap<>(Genre.class);
        user.history().forEach(s -> counts.merge(s.genre(), 1, Integer::sum));
        List<Genre> favourite = counts.entrySet().stream()
                .sorted((a, b) -> b.getValue() - a.getValue()).map(Map.Entry::getKey).toList();
        List<Song> heard = user.history();
        List<Song> out = new ArrayList<>();
        for (Genre g : favourite)
            for (Song s : library.allSongs())
                if (out.size() < n && s.genre() == g && !heard.contains(s)) out.add(s);
        return out;
    }
}
```

Output:

```
== 1. Search ==
  songs 'blue'    -> [All Blues - Miles Davis, Blue in Green - Miles Davis]
  albums 'opera'  -> [A Night at the Opera]
  artists 'da'    -> [Miles Davis]

== 2. Playlist ==
  Road trip: [Bohemian Rhapsody - Queen, Blue in Green - Miles Davis, So What - Miles Davis, Hello - Adele, Love of
My Life - Queen]

== 3. Play in order, pause, seek, repeat-one ==
  [player] > Bohemian Rhapsody - Queen
  state PAUSED at 120s
  [player] > Bohemian Rhapsody - Queen
  [player] > Blue in Green - Miles Davis

== 4. Shuffle keeps the current song, reorders the rest ==
  now Blue in Green - Miles Davis, up next [Love of My Life - Queen, So What - Miles Davis, Bohemian Rhapsody -
Queen, Hello - Adele]

== 5. FREE plan: skip limit (6/hour) and ads, repeat-all wraps the shuffled queue ==
  [player] (30-second ad for FREE plan)
  [player] > Love of My Life - Queen
  [player] > So What - Miles Davis
  [player] > Bohemian Rhapsody - Queen
  [player] (30-second ad for FREE plan)
  [player] > Hello - Adele
  [player] > Blue in Green - Miles Davis
  [player] > Hello - Adele
  [player] skip limit reached on FREE plan

== 6. Upgrade to PREMIUM: no ads, unlimited skips ==
  charged $10.99
  [player] > Death on Two Legs - Queen
  [player] > Freddie Freeloader - Miles Davis
  [player] > Death on Two Legs - Queen
  [player] > Freddie Freeloader - Miles Davis
  [player] end of queue

== 7. Recommendations from listening history ==
  plays so far: 13
  recommended: [You're My Best Friend - Queen, All Blues - Miles Davis, When We Were Young - Adele]
```

Reading the output. Step 3: repeat-ONE replays "Bohemian Rhapsody" when it ends naturally. Step 4: shuffle keeps "Blue in Green" playing and reorders the rest. Step 5: on the FREE plan an ad plays after every three songs, repeat-ALL wraps and reshuffles the queue, and the seventh skip within the hour is refused. Step 6: PREMIUM has no ads; with repeat-ALL the second skip wraps to the start; with repeat OFF the queue ends. Step 7: the recommender suggests songs the user has **not** heard, from the genres they played most.

Concurrency & edge cases

Case	Handling
Two devices send "next" at the same instant	Player methods are `synchronized`; the two skips apply one after the other.
Catalog updated while users search	`ConcurrentHashMap` (a thread-safe hash map) for songs/albums/artists; search iterates a weakly consistent view (it may or may not show changes made during the loop, but never breaks).
Playlist edited while it is playing	`load` copies the playlist (`List.copyOf`), so the running queue is a snapshot; a real app would sync edits into "up next".
Seek past the end or before 0	Clamped to `[0, duration]`.
Previous after 3 seconds	Restarts the current song (common player behaviour); within 3 seconds goes back one song.
Skip when stopped	Refused (and does not count against the limit).
Plan changes mid-session	`plan` is `volatile` (a Java keyword that makes every thread see the latest written value); the next skip or ad check sees it.
Empty queue	`play()` throws `IllegalStateException`.

Real-world example

Spotify's free tier has historically limited skips (for example, six per hour on mobile) and inserts ads between songs, while Premium removes both and allows downloads — the rules our `SubscriptionPlan` enum carries. Spotify engineers also wrote (2014) that users complained a truly random shuffle "felt" non-random when the same artist came up twice in a row, so they changed the algorithm to spread songs by the same artist across the list. That is exactly the kind of change the `PlaybackOrder` strategy absorbs: a new `ArtistSpreadShuffle` class, no player changes. Personalised playlists such as Discover Weekly are the industrial version of our `Recommender` stub.

Extensions & follow-up questions

- **"Add 'play next' and 'add to queue'."** Keep a separate *user queue* that is consumed before the context (playlist/album) order.
- **"Offline mode."** `SubscriptionPlan.offlineDownloads`; a `DownloadManager` stores encrypted files; the player chooses a local or remote `AudioSource` (Strategy).
- **"Social sharing / collaborative playlists."** Playlist gets a collaborator set and an edit log; listeners (Observer) push changes to other users.
- **"Crossfade, gapless playback."** Player behaviour around `trackFinished`; a `TransitionStrategy`.
- **"Royalties."** A `PlaybackListener` that records a play once the user has listened for a threshold time.
- **"Search at scale."** Inverted index (a map from each word to the items containing it) / search service with prefix (typeahead — suggestions as you type) and fuzzy matching (tolerating typos); the `MusicLibrary` interface stays.
- **"Smart shuffle."** New `PlaybackOrder` that spaces out songs by the same artist.

In the interview

30–45 minute plan

1. Clarify controls, plans and scope (skip audio bytes) — 4 min.
2. Catalog entities + library search — 6 min.
3. User, playlist, subscription plan — 5 min.
4. **Player**: queue + order + cursor; state machine; shuffle/repeat as strategy/enum — 15 min.
5. Observer for history; recommender strategy; concurrency; extensions — 10 min.

> **30-second answer:** "Immutable catalog records in a thread-safe library. Each user has a player session holding the queue, a play order produced by a `PlaybackOrder` strategy, a cursor and a repeat mode. Shuffle rebuilds the order around the current song. Plan limits live in the plan enum. A playback listener records history, which feeds a pluggable recommender."

Follow-ups with answers

- *How does shuffle avoid repeating a song before all have played?* It is a permutation (each index once), not "pick a random next song".
- *Where do ads fit?* The player asks the plan whether ads apply and inserts one every N songs; in production an ad service picks which ad.
- *How would you implement "previous" in shuffle mode?* Walk back along the same order list (the cursor), so previous is the song you actually heard before.

Self-check

1. **Q:** Why store the play order as indexes rather than reordering the queue? **A:** So switching shuffle off restores the original order and nothing is lost.
2. **Q:** What does repeat-ONE do when the user presses Next? **A:** Moves to the next song; it only replays on natural end.
3. **Q:** Which pattern lets you add a smarter shuffle without touching the player? **A:** Strategy (`PlaybackOrder`).
4. **Q:** How is "6 skips per hour" enforced? **A:** A deque of skip times; drop entries older than one hour; refuse if the size has reached the limit.
5. **Q:** Why is the player's state an enum rather than the State pattern? **A:** The states have almost no distinct behaviour; the pattern would add classes without benefit.

TL;DR

> - Catalog = immutable records in a concurrent library.
> - Player = **queue + order (Strategy) + cursor + repeat mode + state**; shuffle keeps the current song.
> - Plan rules in the **enum**; plays published via **Observer**; recommendations via **Strategy**.

Design an Online Food Delivery Service (Swiggy) + FoodKart

This problem also comes in two shapes:

- **Part A — Swiggy-style delivery platform** (the awesome-low-level-design problem): restaurants with menus, a cart, orders with tracking states, payments, and **assigning a delivery rider** by a pluggable strategy. We add **kitchen capacity** (a restaurant can only cook so many items at once), the key twist in a commonly reported Flipkart food-ordering variant (it is not part of the FoodKart statement below).
- **Part B — FoodKart** (Flipkart machine-coding round, from kumaransg/LLD): a smaller, command-driven problem — users and pincodes (Indian postal codes), one-dish restaurants, ratings, `show_restaurant` sorted by rating or price, `place_order` — plus a **restaurant selection strategy** (highest rating / lowest price) filtered by remaining capacity.

Problem & clarifying questions

Question	Assumed answer
Can one cart mix restaurants?	No — one restaurant per cart (as in Swiggy/Zomato).
Who confirms an order?	The restaurant; it may refuse if its kitchen is full.
When is capacity freed?	When the food is ready (handed over), not when it is delivered.
How is a rider chosen?	By a strategy: nearest to the restaurant, or fair share (fewest deliveries today).
What if no rider is free?	The order waits in a queue; the next freed rider takes the oldest waiting order.
Can a customer cancel?	Only before the kitchen starts cooking (PENDING or CONFIRMED).
Payment types?	UPI, card, cash on delivery (COD: nothing to refund).
Live GPS, ETA (estimated time of arrival) prediction, batching two orders per rider, surge fees (higher fees at busy times)?	Extensions.

Requirements

Functional (Part A)

1. Customers browse restaurants, view menus and place orders.
2. Restaurants manage menus, prices and availability.
3. Delivery agents accept and fulfil orders.
4. Customers track order status and get notified of changes.
5. Several payment methods.
6. Real-time notifications to customers, restaurants and delivery agents.

Functional (Part B — FoodKart)

7. `register_user(name, gender, phone, pincode)` — phone is unique.
8. `login_user(phone)` — sets the context; logging in someone else logs out the previous user.
9. `register_restaurant(name, pincodes "A/B", dish, price, quantity)` — in the context of the logged-in user (the owner). A restaurant serves **one** dish to **many** pincodes.
10. `update_quantity(name, add)`, `update_location(name, pincodes)` — owner only.
11. `rate_restaurant(name, 1..5, optional comment)`; rating = average of all ratings.
12. `show_restaurant(rating|price)` — **serviceable** restaurants (deliver to my pincode **and** quantity > 0), sorted **descending** by the chosen field.
13. `place_order(name, qty)`; bonus: order history.
14. *(Variant)* place an order without naming a restaurant: pick one by a **selection strategy**, among those that deliver here **and** have capacity for the whole quantity.

Non-functional

- Handle many concurrent orders without double-booking kitchen capacity or riders.
- Keep data consistent (no negative stock, no rider on two orders).
- In-memory only for the machine-coding round; demoable driver (a small `Main` program that feeds in commands and prints results); graceful failures.

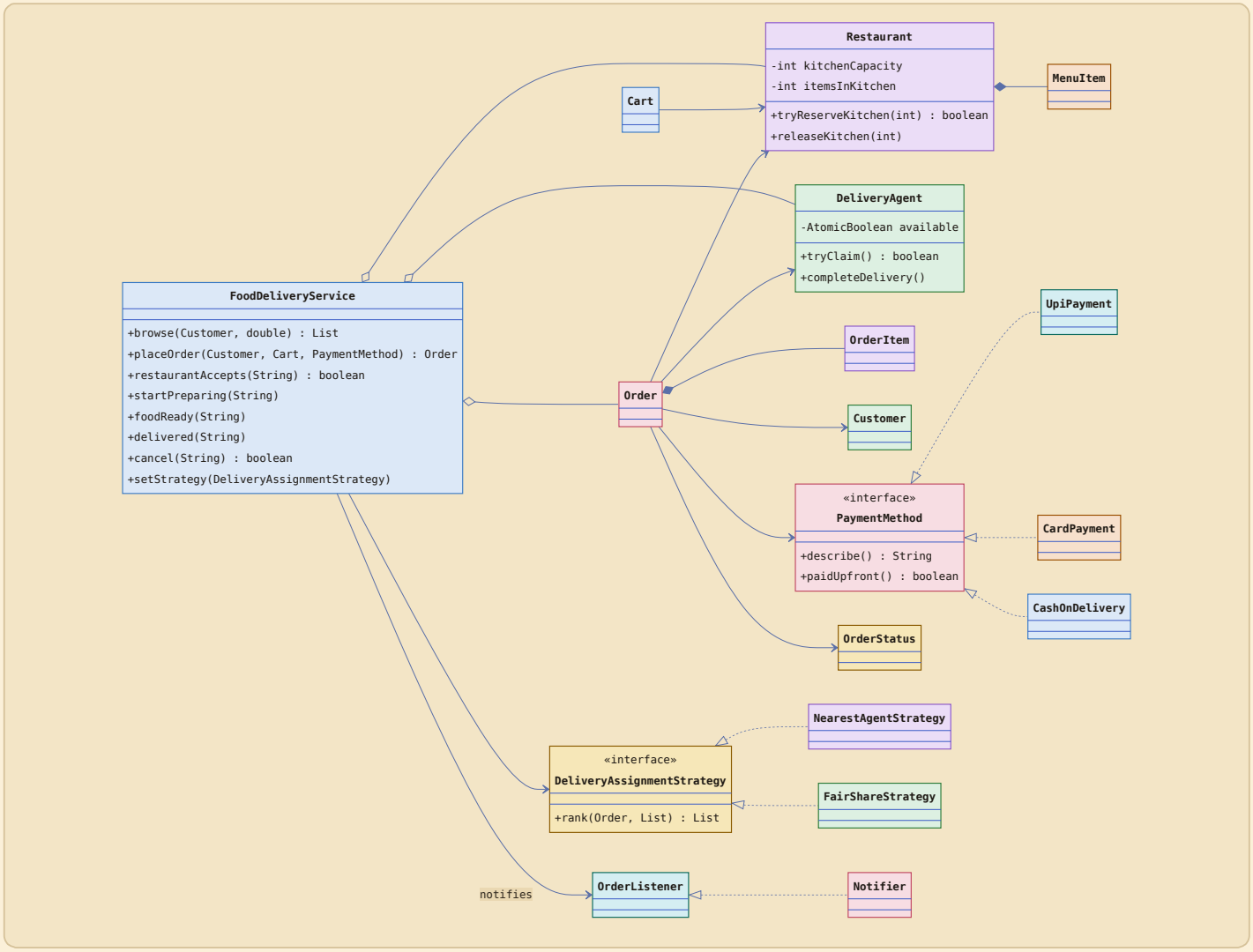
Use cases

Actor	Use cases
Customer	Browse nearby restaurants, build cart, pay, place order, track, cancel, rate
Restaurant	Manage menu/prices/availability, accept or refuse order, mark preparing, mark ready
Delivery agent	Get assigned, pick up, deliver
System	Compute fees, assign riders, queue orders when no rider is free, notify everyone

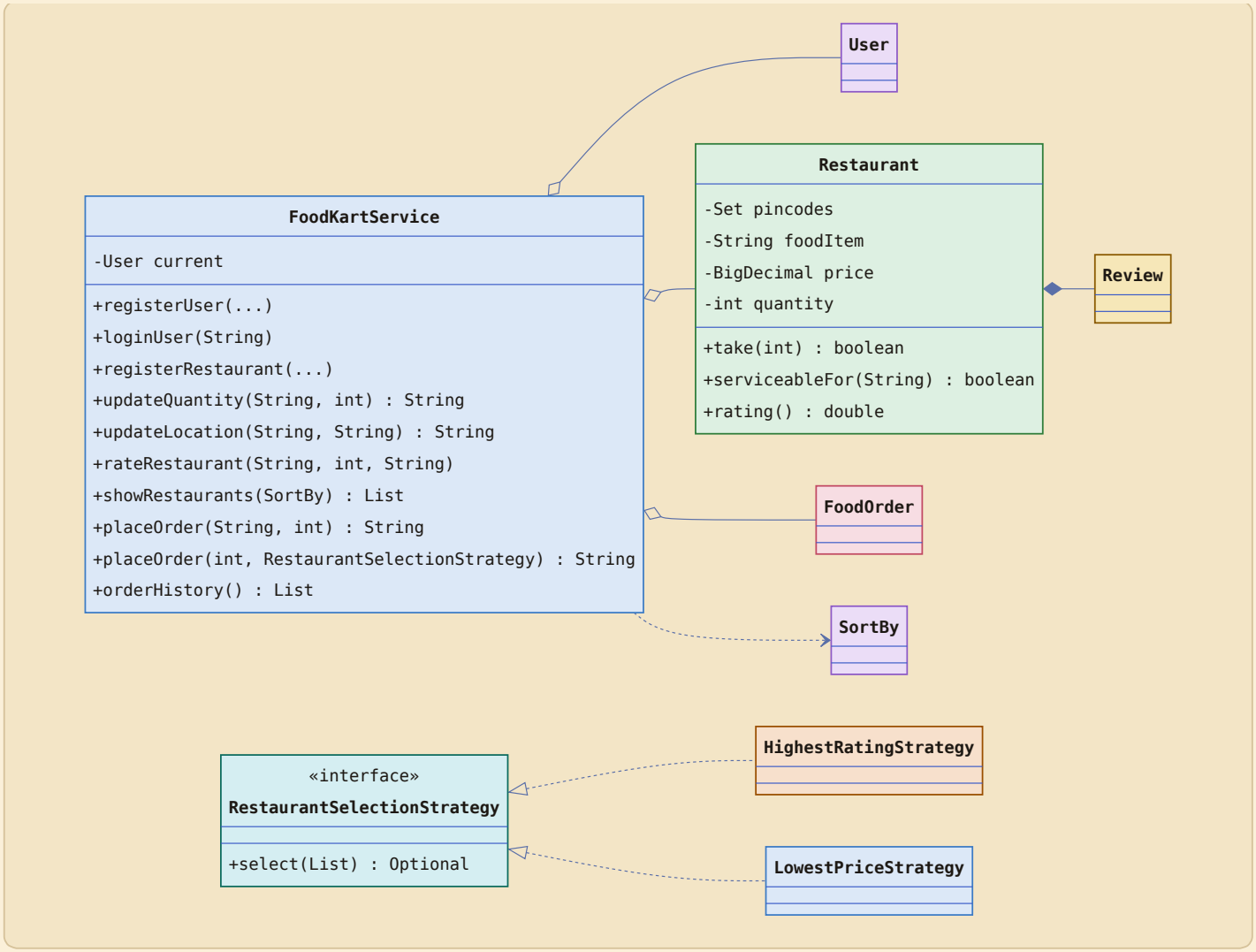
Core entities

Class	Responsibility
`Location`	A point; `distanceTo`.
`Customer`	id, name, phone, address.
`MenuItem`	Name, price, available flag (both changeable by the restaurant).
`Restaurant`	Menu, open flag, **kitchen capacity** (max items cooking at once), `tryReserveKitchen` / `releaseKitchen`.
`Cart`	One restaurant + item quantities; refuses items from a second restaurant.
`OrderItem`	Snapshot of item name, **unit price at order time**, quantity.
`OrderStatus`	State machine: PENDING → CONFIRMED → PREPARING → READY_FOR_PICKUP → OUT_FOR_DELIVERY → DELIVERED, or CANCELLED.
`Order`	Customer, restaurant, items, delivery fee, payment, status history, assigned agent.
`PaymentMethod` → `UpiPayment`, `CardPayment`, `CashOnDelivery`	Sealed interface; COD is not paid upfront.
`DeliveryAgent`	Location, `AtomicBoolean available` (a true/false flag that can be checked-and-changed atomically), deliveries today; `tryClaim()` is a CAS.
`DeliveryAssignmentStrategy` → `NearestAgentStrategy`, `FairShareStrategy`	Ranks free riders for an order.
`OrderListener` / `Notifier`	Observer for status changes.
`FoodDeliveryService`	Facade: browse, place, accept, prepare, ready, deliver, cancel; the waiting-for-rider queue.
(FoodKart) `User`, `Restaurant`, `Review`, `FoodOrder`, `SortBy`, `RestaurantSelectionStrategy`, `FoodKartService`	The machine-coding model; `Restaurant.take(q)` is the atomic capacity check.

Class diagram

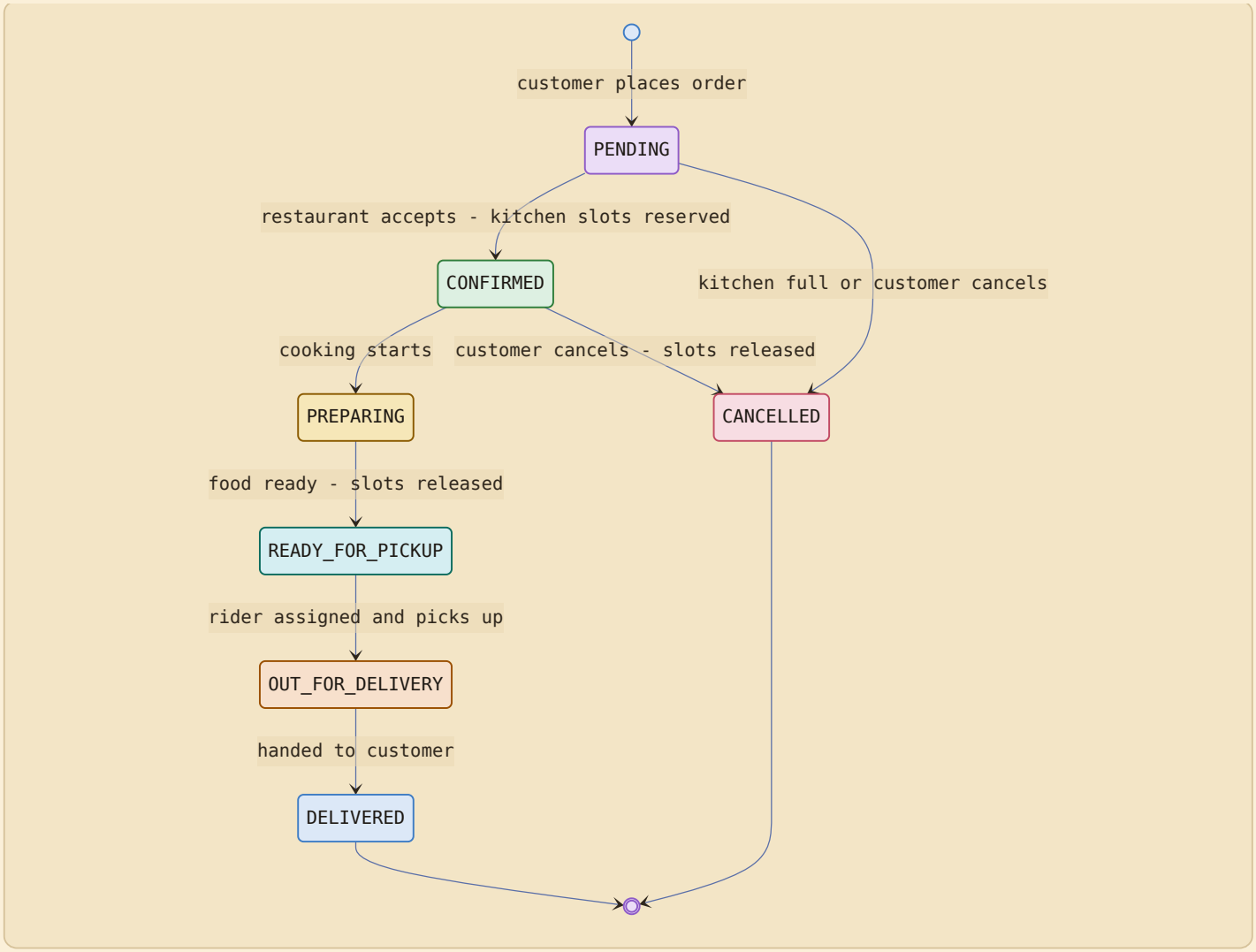


FoodKart (Part B):



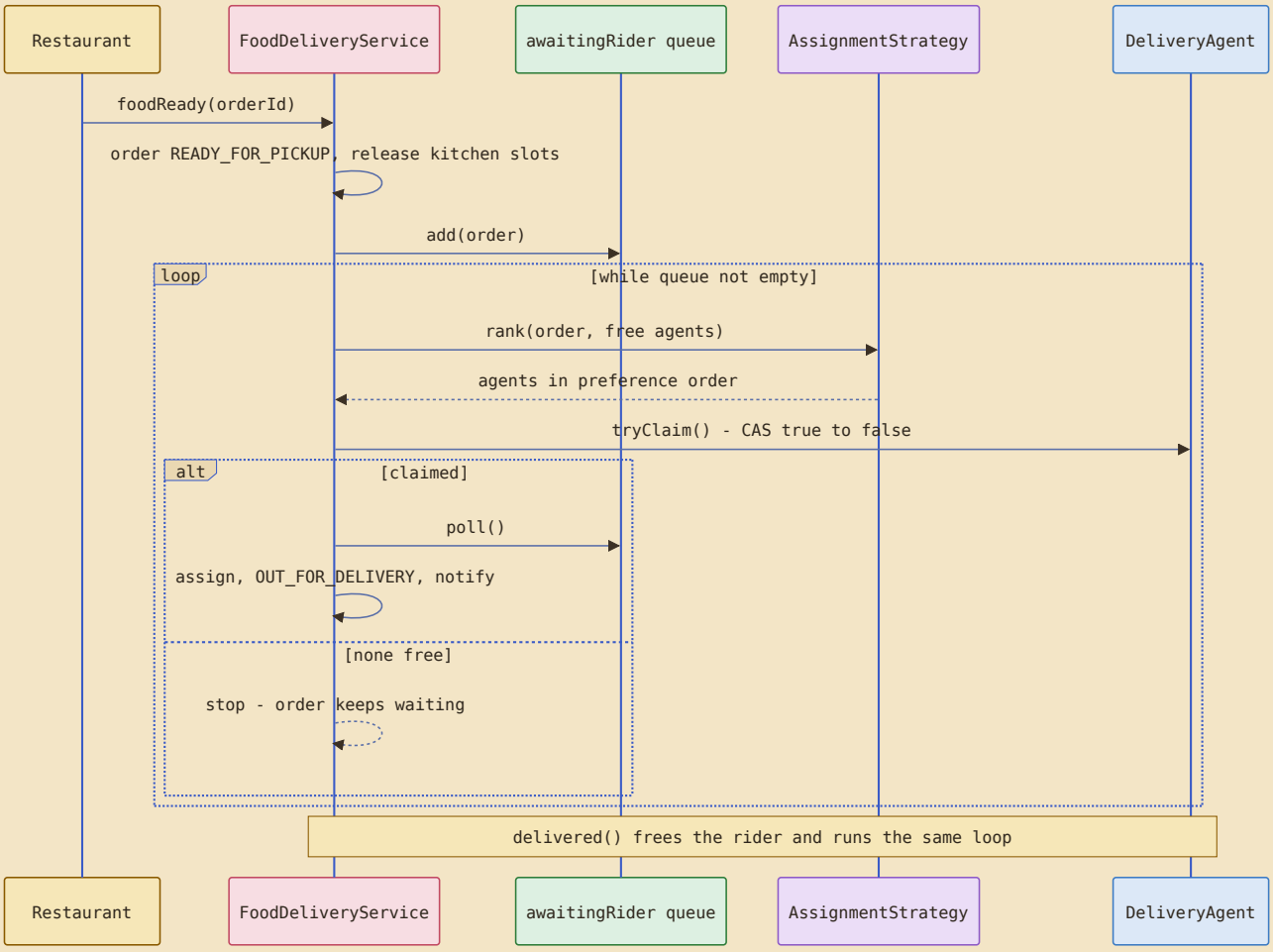
Key flows

Order tracking state machine



(Real apps often split `READY_FOR_PICKUP → OUT_FOR_DELIVERY` into "rider assigned", "arrived at restaurant", "picked up". Adding them is a local enum change.)

Food ready → rider assignment



FoodKart `place_order(name, qty)`: user logged in? → restaurant exists? → delivers to the user's pincode? → `take(qty)` atomically (quantity ≥ qty, then subtract) → record in history → "Order Placed Successfully." Otherwise "Cannot place order".

FoodKart selection variant: filter restaurants that deliver to the user's pincode and have `quantity ≥ qty` → ask the strategy to pick one → `take(qty)`; if another user took the capacity in between, drop that restaurant and ask again (bounded retries).

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for rider assignment (`rank` returns a preference list)	Nearest vs fair-share vs rating-based are swappable at run time (`setStrategy`, step 7). Returning a *ranked list* lets the service try the next rider if the first was just taken (the free list is read before claiming, so it can be stale).	Returning one rider (fails if that rider was claimed a moment earlier).
CAS on the rider (`AtomicBoolean.compareAndSet(true,false)`)	The rider guards itself: whoever calls `tryClaim()` — this dispatcher, a second dispatcher instance, or a future "rider accepts" button — exactly one `compareAndSet` succeeds. In this code `dispatchWaiting` is also `synchronized` (to drain the queue in FIFO order), so the CAS is the safety net that keeps working even when claims come from outside that lock.	Check `isAvailable()` then set (a race as soon as two callers exist).
Kitchen capacity reserved on accept, released on ready	Models the rule "a restaurant cannot accept more items while its kitchen is full" from the capacity-based food-ordering variant.	Unlimited orders per restaurant.
Waiting queue for riders	No free rider is normal at peak; FIFO keeps it fair; freeing a rider re-runs dispatch.	Failing the order.
State machine in `OrderStatus`	Tracking states and legal moves in one place; cancel after cooking starts is impossible.	Scattered `if`s.
`OrderItem` copies the price	A later menu price change (step 3) cannot alter a placed order.	Referencing `MenuItem` directly.
Sealed `PaymentMethod` + records	A closed, known set; COD overrides `paidUpfront()` so refunds skip it.	`String paymentType`.
Observer (`OrderListener`)	Customer app, restaurant tablet and rider app all react to the same events.	The service printing/sending messages itself.
One restaurant per cart	Matches real apps; a single kitchen, single pickup, single fee.	Multi-restaurant carts (needs split orders).
FoodKart: `SortBy` enum holds its comparator (an object that decides which of two items comes first)	`show_restaurant(rating	price)`becomes`sorted(sortBy.comparator)`; both descending as the statement says; name as tie-breaker for stable output.
FoodKart: owner recorded at registration	The statement says restaurants are registered "in the context of the	Anyone updating any restaurant.

Decision	Why	Alternative rejected
	logged-in user"; only that owner may update quantity or location.	
FoodKart: `Restaurant.take(q)` synchronized	"Check quantity then subtract" is atomic — two users cannot both take the last plates.	Separate `getQuantity()` + `setQuantity()`.

Java implementation

Part A — Food delivery platform (`lld.ch19.fooddelivery`)

Files: `Model.java` (location, customer, menu item, restaurant, cart, order item, status machine, payments, order, delivery agent), `FoodDeliveryService.java` (strategies, listener, notifier, service), `Main.java`.

```java
// File: lld/ch19/fooddelivery/FoodDeliveryService.java
package lld.ch19.fooddelivery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Queue;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

/** Strategy: which free rider gets this order? Returns riders in preference order. */
interface DeliveryAssignmentStrategy {
    List<DeliveryAgent> rank(Order order, List<DeliveryAgent> free);
}

/** Closest rider to the restaurant - fastest pickup. */
final class NearestAgentStrategy implements DeliveryAssignmentStrategy {
    @Override public List<DeliveryAgent> rank(Order o, List<DeliveryAgent> free) {
        Location r = o.restaurant().location();
        return free.stream().sorted(Comparator.comparingDouble(a -> a.location().distanceTo(r))).toList();
    }
}

/** Fewest deliveries today first (spreads earnings), nearest as tie-breaker. */
final class FairShareStrategy implements DeliveryAssignmentStrategy {
    @Override public List<DeliveryAgent> rank(Order o, List<DeliveryAgent> free) {
        Location r = o.restaurant().location();
        return free.stream().sorted(Comparator.comparingInt(DeliveryAgent::deliveriesToday)
                .thenComparingDouble(a -> a.location().distanceTo(r))).toList();
    }
}

/** Observer: customer app, restaurant tablet and rider app all track the order. */
interface OrderListener { void onStatus(Order order, OrderStatus status); }

final class Notifier implements OrderListener {
    @Override public void onStatus(Order o, OrderStatus s) {
        String who = switch (s) {
            case PENDING -> "restaurant " + o.restaurant().name() + ": new order";
            case OUT_FOR_DELIVERY -> "customer " + o.customer().name() + ": " +
o.agent().map(DeliveryAgent::name).orElse("?") + " is on the way";
            default -> "customer " + o.customer().name();
        };
        System.out.println("    [" + o.id() + " " + s + "] -> " + who);
    }
}

/** Facade: browse, order, accept, prepare, dispatch, deliver, cancel. */
final class FoodDeliveryService {
    private static final BigDecimal BASE_FEE = new BigDecimal("20.00");
    private static final BigDecimal PER_KM = new BigDecimal("8.00");

    private final Map<String, Restaurant> restaurants = new ConcurrentHashMap<>();
    private final List<DeliveryAgent> agents = new CopyOnWriteArrayList<>();
    private final Map<String, Order> orders = new ConcurrentHashMap<>();
    private final Queue<Order> awaitingRider = new ConcurrentLinkedQueue<>();
    private final List<OrderListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger seq = new AtomicInteger(100);
    private volatile DeliveryAssignmentStrategy strategy;

    FoodDeliveryService(DeliveryAssignmentStrategy strategy) { this.strategy = strategy; }
```

```
    void setStrategy(DeliveryAssignmentStrategy s) { strategy = s; }
    void subscribe(OrderListener l) { listeners.add(l); }
    void addRestaurant(Restaurant r) { restaurants.put(r.id(), r); }
    void addAgent(DeliveryAgent a) { agents.add(a); }

    /** Open restaurants within the radius, nearest first. */
    List<Restaurant> browse(Customer c, double radiusKm) {
        return restaurants.values().stream().filter(Restaurant::isOpen)
                .filter(r -> r.location().distanceTo(c.address()) <= radiusKm)
                .sorted(Comparator.comparingDouble(r -> r.location().distanceTo(c.address()))).toList();
    }

    Order placeOrder(Customer c, Cart cart, PaymentMethod pay) {
        if (cart.isEmpty()) throw new IllegalStateException("empty cart");
        Restaurant r = cart.restaurant();
        if (!r.isOpen()) throw new IllegalStateException(r.name() + " is closed");
        List<OrderItem> items = new ArrayList<>();
        cart.quantities().forEach((itemId, qty) -> {
            MenuItem m = r.item(itemId).orElseThrow();
            if (!m.available()) throw new IllegalStateException(m.name() + " is sold out");
            items.add(new OrderItem(itemId, m.name(), m.price(), qty));
        });
        BigDecimal km = BigDecimal.valueOf(r.location().distanceTo(c.address())).setScale(1, RoundingMode.HALF_UP);
        BigDecimal fee = BASE_FEE.add(PER_KM.multiply(km)).setScale(2, RoundingMode.HALF_EVEN);
        Order o = new Order("OD" + seq.incrementAndGet(), c, r, items, fee, pay);
        orders.put(o.id(), o);
        cart.clear();
        publish(o, OrderStatus.PENDING);
        return o;
    }

    /** The restaurant accepts only if its kitchen has room; otherwise the order is auto-cancelled. */
    boolean restaurantAccepts(String orderId) {
        Order o = orders.get(orderId);
        synchronized (o) {                              // same lock as cancel(): capacity can never leak
            if (o.status() != OrderStatus.PENDING) return false;
            if (!o.restaurant().tryReserveKitchen(o.itemCount())) {
                System.out.println("    (" + o.restaurant().name() + " kitchen full: needs " + o.itemCount()
                        + " item slots, free " + o.restaurant().freeCapacity() + ")");
                transition(o, OrderStatus.CANCELLED);
                return false;
            }
            transition(o, OrderStatus.CONFIRMED);
            return true;
        }
    }

    void startPreparing(String orderId) { transition(orders.get(orderId), OrderStatus.PREPARING); }

    /** Food is ready: kitchen capacity comes back, and we look for a rider. */
    void foodReady(String orderId) {
        Order o = orders.get(orderId);
        transition(o, OrderStatus.READY_FOR_PICKUP);
        o.restaurant().releaseKitchen(o.itemCount());
        awaitingRider.add(o);
        dispatchWaiting();
    }

    void delivered(String orderId) {
        Order o = orders.get(orderId);
        transition(o, OrderStatus.DELIVERED);
        DeliveryAgent a = o.agent().orElseThrow();
        a.moveTo(o.customer().address());
        a.completeDelivery();
        dispatchWaiting();                              // a freed rider takes the oldest waiting order
    }

    boolean cancel(String orderId) {
        Order o = orders.get(orderId);
```

```
        synchronized (o) {
            if (!o.status().canMoveTo(OrderStatus.CANCELLED)) return false;
            boolean kitchenHeld = o.status() == OrderStatus.CONFIRMED;
            transition(o, OrderStatus.CANCELLED);
            if (kitchenHeld) o.restaurant().releaseKitchen(o.itemCount());
        }
        if (o.payment().paidUpfront()) System.out.println("    refund " + o.total() + " to " +
o.payment().describe());
        return true;
    }

    Order order(String id) { return orders.get(id); }

    /** Hand waiting orders (oldest first) to free riders. CAS on the rider prevents double booking. */
    private synchronized void dispatchWaiting() {
        Order o;
        while ((o = awaitingRider.peek()) != null) {
            List<DeliveryAgent> free = agents.stream().filter(DeliveryAgent::isAvailable).toList();
            Optional<DeliveryAgent> got = Optional.empty();
            for (DeliveryAgent a : strategy.rank(o, free)) if (a.tryClaim()) { got = Optional.of(a); break; }
            if (got.isEmpty()) {
                System.out.println("    (" + o.id() + " waiting for a free rider)");
                return;
            }
            awaitingRider.poll();
            o.assign(got.get());
            transition(o, OrderStatus.OUT_FOR_DELIVERY);
        }
    }

    private void transition(Order o, OrderStatus s) {
        o.moveTo(s);
        listeners.forEach(l -> l.onStatus(o, s));
    }

    private void publish(Order o, OrderStatus s) { listeners.forEach(l -> l.onStatus(o, s)); }
}
```

```java
// File: lld/ch19/fooddelivery/Main.java
package lld.ch19.fooddelivery;

import java.math.BigDecimal;

public class Main {
    public static void main(String[] args) {
        FoodDeliveryService app = new FoodDeliveryService(new NearestAgentStrategy());
        app.subscribe(new Notifier());

        Restaurant dosa = new Restaurant("R1", "Dosa Corner", new Location(1, 1), 4);
        dosa.addItem(new MenuItem("D1", "Masala Dosa", new BigDecimal("120.00")));
        dosa.addItem(new MenuItem("D2", "Filter Coffee", new BigDecimal("40.00")));
        Restaurant pizza = new Restaurant("R2", "Pizza Hub", new Location(5, 5), 10);
        pizza.addItem(new MenuItem("P1", "Margherita", new BigDecimal("299.00")));
        Restaurant far = new Restaurant("R3", "Far Away Grill", new Location(30, 30), 10);
        app.addRestaurant(dosa);
        app.addRestaurant(pizza);
        app.addRestaurant(far);
        DeliveryAgent ravi = new DeliveryAgent("A1", "Ravi", new Location(0, 0));
        DeliveryAgent meena = new DeliveryAgent("A2", "Meena", new Location(6, 6));
        app.addAgent(ravi);
        app.addAgent(meena);

        Customer asha = new Customer("C1", "Asha", "98000", new Location(2, 2));
        Customer vik = new Customer("C2", "Vikram", "98001", new Location(4, 4));

        System.out.println("== 1. Browse within 10 km ==");
        app.browse(asha, 10).forEach(r -> System.out.println("  " + r.name()));

        System.out.println("\n== 2. One restaurant per cart ==");
        Cart cart = new Cart();
        cart.add(dosa, "D1", 2);
        cart.add(dosa, "D2", 1);
        try { cart.add(pizza, "P1", 1); } catch (IllegalStateException e) { System.out.println("  " +
e.getMessage()); }

        System.out.println("\n== 3. Place, accept, prepare, ready, deliver ==");
        Order o1 = app.placeOrder(asha, cart, new UpiPayment("asha@upi"));
        System.out.println("  subtotal " + o1.subtotal() + " + delivery " + o1.deliveryFee() + " = " + o1.total());
        dosa.item("D1").orElseThrow().setPrice(new BigDecimal("150.00"));    // later price change...
        System.out.println("  price changed after ordering; order still " + o1.total());
        app.restaurantAccepts(o1.id());
        app.startPreparing(o1.id());
        app.foodReady(o1.id());
        System.out.println("  rider: " + o1.agent().orElseThrow() + " (nearest to Dosa Corner)");

        System.out.println("\n== 4. Kitchen capacity: 4 items max at Dosa Corner ==");
        Cart c2 = new Cart();
        c2.add(dosa, "D1", 3);
        Order o2 = app.placeOrder(vik, c2, new CashOnDelivery());
        app.restaurantAccepts(o2.id());                                    // 3 of 4 slots used
        Cart c3 = new Cart();
        c3.add(dosa, "D2", 2);
        Order o3 = app.placeOrder(asha, c3, new CardPayment("4242"));
        app.restaurantAccepts(o3.id());                                    // needs 2, only 1 free

        System.out.println("\n== 5. Only one rider free: second ready order waits ==");
        app.startPreparing(o2.id());
        app.foodReady(o2.id());                                            // Meena takes it
        Cart c4 = new Cart();
        c4.add(pizza, "P1", 1);
        Order o4 = app.placeOrder(vik, c4, new UpiPayment("vik@upi"));
        app.restaurantAccepts(o4.id());
        app.startPreparing(o4.id());
        app.foodReady(o4.id());                                            // nobody free
        app.delivered(o1.id());                                            // Ravi frees up -> takes o4
        System.out.println("  o4 rider: " + o4.agent().orElseThrow());
```

```
        System.out.println("\n== 6. Cancel rules ==");
        System.out.println("  cancel o2 (out for delivery)? " + app.cancel(o2.id()));
        Cart c5 = new Cart();
        c5.add(pizza, "P1", 2);
        Order o5 = app.placeOrder(asha, c5, new CardPayment("4242"));
        app.restaurantAccepts(o5.id());
        System.out.println("  cancel o5 (confirmed)? " + app.cancel(o5.id()));

        System.out.println("\n== 7. Swap strategy at run time: fair share ==");
        app.delivered(o2.id());
        app.delivered(o4.id());
        System.out.println("  deliveries today: Ravi=" + ravi.deliveriesToday() + ", Meena=" +
meena.deliveriesToday()
                + " (both now at Vikram's address)");
        app.setStrategy(new FairShareStrategy());
        Cart c6 = new Cart();
        c6.add(dosa, "D2", 1);
        Order o6 = app.placeOrder(asha, c6, new UpiPayment("asha@upi"));
        app.restaurantAccepts(o6.id());
        app.startPreparing(o6.id());
        app.foodReady(o6.id());
        System.out.println("  o6 rider: " + o6.agent().orElseThrow() + " (fewer deliveries today)");
        System.out.println("  o1 history: " + o1.history());
    }
}
```

```java
// File: lld/ch19/fooddelivery/Model.java
package lld.ch19.fooddelivery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Collection;
import java.util.EnumSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicInteger;

/** A point on a simple km grid (real systems use lat/long and a road-distance service). */
record Location(double x, double y) {
    double distanceTo(Location o) { return Math.hypot(x - o.x, y - o.y); }
}

record Customer(String id, String name, String phone, Location address) { }

final class MenuItem {
    private final String id;
    private final String name;
    private volatile BigDecimal price;
    private volatile boolean available = true;

    MenuItem(String id, String name, BigDecimal price) { this.id = id; this.name = name; this.price = price; }
    String id() { return id; }
    String name() { return name; }
    BigDecimal price() { return price; }
    boolean available() { return available; }
    void setPrice(BigDecimal p) { price = p; }
    void setAvailable(boolean a) { available = a; }
}

/**
 * A restaurant manages its own menu and has a kitchen capacity: at most N items being
 * prepared at once. Capacity is taken when an order is accepted and returned when the
 * food is ready (dispatched).
 */
final class Restaurant {
    private final String id;
    private final String name;
    private final Location location;
    private final int kitchenCapacity;
    private final Map<String, MenuItem> menu = new ConcurrentHashMap<>();
    private volatile boolean open = true;
    private int itemsInKitchen;                          // guarded by this

    Restaurant(String id, String name, Location location, int kitchenCapacity) {
        this.id = id; this.name = name; this.location = location; this.kitchenCapacity = kitchenCapacity;
    }

    String id() { return id; }
    String name() { return name; }
    Location location() { return location; }
    boolean isOpen() { return open; }
    void setOpen(boolean o) { open = o; }
    void addItem(MenuItem m) { menu.put(m.id(), m); }
    Optional<MenuItem> item(String itemId) { return Optional.ofNullable(menu.get(itemId)); }
    Collection<MenuItem> menu() { return menu.values(); }

    synchronized boolean tryReserveKitchen(int items) {
        if (itemsInKitchen + items > kitchenCapacity) return false;
```

```
        itemsInKitchen += items;
        return true;
    }

    synchronized void releaseKitchen(int items) { itemsInKitchen -= items; }

    synchronized int freeCapacity() { return kitchenCapacity - itemsInKitchen; }
}

/** A customer's cart. One restaurant per cart, as in Swiggy/Zomato. */
final class Cart {
    private Restaurant restaurant;
    private final Map<String, Integer> quantities = new LinkedHashMap<>();

    synchronized void add(Restaurant r, String itemId, int qty) {
        if (restaurant != null && !restaurant.id().equals(r.id()))
            throw new IllegalStateException("cart has items from " + restaurant.name() + " - clear it first");
        r.item(itemId).orElseThrow(() -> new IllegalArgumentException("no item " + itemId));
        restaurant = r;
        quantities.merge(itemId, qty, Integer::sum);
    }

    synchronized void clear() { restaurant = null; quantities.clear(); }
    synchronized Restaurant restaurant() { return restaurant; }
    synchronized Map<String, Integer> quantities() { return Map.copyOf(quantities); }
    synchronized boolean isEmpty() { return quantities.isEmpty(); }
}

/** Price is copied at order time: a later menu price change never alters a placed order. */
record OrderItem(String itemId, String name, BigDecimal unitPrice, int quantity) {
    BigDecimal total() { return unitPrice.multiply(BigDecimal.valueOf(quantity)); }
}

/** The order-tracking state machine. */
enum OrderStatus {
    PENDING, CONFIRMED, PREPARING, READY_FOR_PICKUP, OUT_FOR_DELIVERY, DELIVERED, CANCELLED;

    private Set<OrderStatus> next() {
        return switch (this) {
            case PENDING -> EnumSet.of(CONFIRMED, CANCELLED);
            case CONFIRMED -> EnumSet.of(PREPARING, CANCELLED);
            case PREPARING -> EnumSet.of(READY_FOR_PICKUP);
            case READY_FOR_PICKUP -> EnumSet.of(OUT_FOR_DELIVERY);
            case OUT_FOR_DELIVERY -> EnumSet.of(DELIVERED);
            case DELIVERED, CANCELLED -> EnumSet.noneOf(OrderStatus.class);
        };
    }

    boolean canMoveTo(OrderStatus t) { return next().contains(t); }
}

/** Strategy: how the customer pays. */
sealed interface PaymentMethod permits UpiPayment, CardPayment, CashOnDelivery {
    String describe();
    default boolean paidUpfront() { return true; }
}

record UpiPayment(String vpa) implements PaymentMethod {
    public String describe() { return "UPI " + vpa; }
}

record CardPayment(String last4) implements PaymentMethod {
    public String describe() { return "card ****" + last4; }
}

record CashOnDelivery() implements PaymentMethod {
    public String describe() { return "cash on delivery"; }
    public boolean paidUpfront() { return false; }
}
```

```
final class Order {
    private final String id;
    private final Customer customer;
    private final Restaurant restaurant;
    private final List<OrderItem> items;
    private final BigDecimal deliveryFee;
    private final PaymentMethod payment;
    private final List<OrderStatus> history = new ArrayList<>();
    private OrderStatus status = OrderStatus.PENDING;
    private DeliveryAgent agent;

    Order(String id, Customer c, Restaurant r, List<OrderItem> items, BigDecimal fee, PaymentMethod payment) {
        this.id = id; this.customer = c; this.restaurant = r; this.items = List.copyOf(items);
        this.deliveryFee = fee; this.payment = payment;
        history.add(status);
    }

    String id() { return id; }
    Customer customer() { return customer; }
    Restaurant restaurant() { return restaurant; }
    List<OrderItem> items() { return items; }
    PaymentMethod payment() { return payment; }
    int itemCount() { return items.stream().mapToInt(OrderItem::quantity).sum(); }
    BigDecimal subtotal() { return items.stream().map(OrderItem::total).reduce(BigDecimal.ZERO, BigDecimal::add); }
    BigDecimal total() { return subtotal().add(deliveryFee).setScale(2, RoundingMode.HALF_EVEN); }
    BigDecimal deliveryFee() { return deliveryFee; }
    synchronized OrderStatus status() { return status; }
    synchronized List<OrderStatus> history() { return List.copyOf(history); }
    synchronized Optional<DeliveryAgent> agent() { return Optional.ofNullable(agent); }
    synchronized void assign(DeliveryAgent a) { agent = a; }

    synchronized void moveTo(OrderStatus t) {
        if (!status.canMoveTo(t)) throw new IllegalStateException(id + ": " + status + " -> " + t + " not
allowed");
        status = t;
        history.add(t);
    }
}

/** A rider. available is an AtomicBoolean so two orders can never claim the same rider. */
final class DeliveryAgent {
    private final String id;
    private final String name;
    private volatile Location location;
    private final AtomicBoolean available = new AtomicBoolean(true);
    private final AtomicInteger deliveriesToday = new AtomicInteger();

    DeliveryAgent(String id, String name, Location location) { this.id = id; this.name = name; this.location =
location; }
    String id() { return id; }
    String name() { return name; }
    Location location() { return location; }
    void moveTo(Location l) { location = l; }
    boolean isAvailable() { return available.get(); }
    int deliveriesToday() { return deliveriesToday.get(); }

    /** Compare-and-set: exactly one caller wins. */
    boolean tryClaim() { return available.compareAndSet(true, false); }

    void completeDelivery() { deliveriesToday.incrementAndGet(); available.set(true); }

    @Override public String toString() { return name; }
}
```

Output:

```
== 1. Browse within 10 km ==
  Dosa Corner
  Pizza Hub

== 2. One restaurant per cart ==
  cart has items from Dosa Corner - clear it first

== 3. Place, accept, prepare, ready, deliver ==
    [OD101 PENDING] -> restaurant Dosa Corner: new order
  subtotal 280.00 + delivery 31.20 = 311.20
  price changed after ordering; order still 311.20
    [OD101 CONFIRMED] -> customer Asha
    [OD101 PREPARING] -> customer Asha
    [OD101 READY_FOR_PICKUP] -> customer Asha
    [OD101 OUT_FOR_DELIVERY] -> customer Asha: Ravi is on the way
  rider: Ravi (nearest to Dosa Corner)

== 4. Kitchen capacity: 4 items max at Dosa Corner ==
    [OD102 PENDING] -> restaurant Dosa Corner: new order
    [OD102 CONFIRMED] -> customer Vikram
    [OD103 PENDING] -> restaurant Dosa Corner: new order
    (Dosa Corner kitchen full: needs 2 item slots, free 1)
    [OD103 CANCELLED] -> customer Asha

== 5. Only one rider free: second ready order waits ==
    [OD102 PREPARING] -> customer Vikram
    [OD102 READY_FOR_PICKUP] -> customer Vikram
    [OD102 OUT_FOR_DELIVERY] -> customer Vikram: Meena is on the way
    [OD104 PENDING] -> restaurant Pizza Hub: new order
    [OD104 CONFIRMED] -> customer Vikram
    [OD104 PREPARING] -> customer Vikram
    [OD104 READY_FOR_PICKUP] -> customer Vikram
    (OD104 waiting for a free rider)
    [OD101 DELIVERED] -> customer Asha
    [OD104 OUT_FOR_DELIVERY] -> customer Vikram: Ravi is on the way
  o4 rider: Ravi

== 6. Cancel rules ==
  cancel o2 (out for delivery)? false
    [OD105 PENDING] -> restaurant Pizza Hub: new order
    [OD105 CONFIRMED] -> customer Asha
    [OD105 CANCELLED] -> customer Asha
    refund 651.60 to card ****4242
  cancel o5 (confirmed)? true

== 7. Swap strategy at run time: fair share ==
    [OD102 DELIVERED] -> customer Vikram
    [OD104 DELIVERED] -> customer Vikram
  deliveries today: Ravi=2, Meena=1 (both now at Vikram's address)
    [OD106 PENDING] -> restaurant Dosa Corner: new order
    [OD106 CONFIRMED] -> customer Asha
    [OD106 PREPARING] -> customer Asha
    [OD106 READY_FOR_PICKUP] -> customer Asha
    [OD106 OUT_FOR_DELIVERY] -> customer Asha: Meena is on the way
  o6 rider: Meena (fewer deliveries today)
  o1 history: [PENDING, CONFIRMED, PREPARING, READY_FOR_PICKUP, OUT_FOR_DELIVERY, DELIVERED]
```

Reading the output. Step 1 hides the far-away grill (outside 10 km). Step 2 refuses a second restaurant in one cart. Step 3: the delivery fee is 20 + 8 × 1.4 km = 31.20; changing the dosa price afterwards does not change the order. Step 4: Dosa Corner holds 4 item slots; an order for 3 fits, the next order for 2 is auto-cancelled. Step 5: Meena takes OD102; OD104 waits because Ravi is still out; as soon as OD101 is delivered, Ravi is claimed for OD104. Step 6: an order out for delivery cannot be cancelled; a confirmed card order can, and is refunded. Step 7: with both riders at the same spot, the fair-share strategy picks Meena (1 delivery) over Ravi (2).

Part B — FoodKart (`lld.ch19.foodkart`)

Files: `FoodKart.java` (gender, user, review, order, restaurant, sort, selection strategies, service), `Main.java` (driver replaying the problem's sample test case, then extras).

```java
// File: lld/ch19/foodkart/FoodKart.java
package lld.ch19.foodkart;

import java.math.BigDecimal;
import java.util.Comparator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.TreeSet;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

enum Gender { M, F, OTHER }

record User(String name, Gender gender, String phone, String pincode) { }

record Review(String userPhone, int rating, String comment) {
    Review {
        if (rating < 1 || rating > 5) throw new IllegalArgumentException("rating must be 1..5");
    }
}

record FoodOrder(int id, String userPhone, String restaurant, String item, int quantity, BigDecimal total) { }

/** A restaurant serves ONE dish to MANY pincodes. Quantity is its remaining capacity. */
final class Restaurant {
    private final String name;
    private final String ownerPhone;
    private final String foodItem;
    private final BigDecimal price;
    private volatile Set<String> pincodes;
    private int quantity;                                          // guarded by this
    private final List<Review> reviews = new CopyOnWriteArrayList<>();

    Restaurant(String name, String ownerPhone, Set<String> pincodes, String foodItem, BigDecimal price, int
quantity) {
        this.name = name; this.ownerPhone = ownerPhone; this.pincodes = Set.copyOf(pincodes);
        this.foodItem = foodItem; this.price = price; this.quantity = quantity;
    }

    String name() { return name; }
    String ownerPhone() { return ownerPhone; }
    String foodItem() { return foodItem; }
    BigDecimal price() { return price; }
    Set<String> pincodes() { return pincodes; }
    void setPincodes(Set<String> p) { pincodes = Set.copyOf(p); }
    synchronized int quantity() { return quantity; }
    synchronized void addQuantity(int q) { quantity += q; }

    /** Check-and-take in one step, so two users cannot both get the last plates. */
    synchronized boolean take(int q) {
        if (q <= 0 || q > quantity) return false;
        quantity -= q;
        return true;
    }

    /** Serviceable = delivers to this pincode AND has non-zero quantity (from the problem statement). */
    boolean serviceableFor(String pincode) { return pincodes.contains(pincode) && quantity() > 0; }

    void addReview(Review r) { reviews.add(r); }

    double rating() { return reviews.stream().mapToInt(Review::rating).average().orElse(0.0); }

    String describe() { return name + ", " + String.join("/", new TreeSet<>(pincodes)) + ", " + foodItem + " - " +
quantity(); }
```

```
}

/** show_restaurant(rating|price): both sorted in DESCENDING order, as the problem states. */
enum SortBy {
    RATING(Comparator.comparingDouble(Restaurant::rating).reversed()),
    PRICE(Comparator.comparing(Restaurant::price).reversed());

    final Comparator<Restaurant> comparator;
    SortBy(Comparator<Restaurant> c) { comparator = c.thenComparing(Restaurant::name); }
}

/** Strategy: when the user does not name a restaurant, which one should we pick? */
interface RestaurantSelectionStrategy {
    Optional<Restaurant> select(List<Restaurant> canFulfil);
}

final class HighestRatingStrategy implements RestaurantSelectionStrategy {
    @Override public Optional<Restaurant> select(List<Restaurant> c) {
        return c.stream().max(Comparator.comparingDouble(Restaurant::rating)
                .thenComparing(Restaurant::quantity));              // tie: the one with most capacity left
    }
}

final class LowestPriceStrategy implements RestaurantSelectionStrategy {
    @Override public Optional<Restaurant> select(List<Restaurant> c) {
        return c.stream().min(Comparator.comparing(Restaurant::price)
                .thenComparing(Comparator.comparingDouble(Restaurant::rating).reversed()));
    }
}

/** The service behind every command in the machine-coding round. In-memory only. */
final class FoodKartService {
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final Map<String, Restaurant> restaurants = new ConcurrentHashMap<>();
    private final Map<String, List<FoodOrder>> history = new ConcurrentHashMap<>();
    private final AtomicInteger orderSeq = new AtomicInteger();
    private volatile User current;                                  // one logged-in user at a time

    void registerUser(String name, Gender g, String phone, String pincode) {
        if (users.putIfAbsent(phone, new User(name, g, phone, pincode)) != null)
            throw new IllegalArgumentException("phone already registered: " + phone);
    }

    void loginUser(String phone) {
        User u = users.get(phone);
        if (u == null) throw new IllegalArgumentException("unknown user " + phone);
        current = u;                                                // previous user is logged out
    }

    void registerRestaurant(String name, String pincodesSlashSeparated, String item, BigDecimal price, int qty) {
        User owner = requireLogin();
        Restaurant r = new Restaurant(name, owner.phone(), parsePincodes(pincodesSlashSeparated), item, price,
qty);
        if (restaurants.putIfAbsent(name, r) != null) throw new IllegalArgumentException("restaurant exists: " +
name);
    }

    String updateQuantity(String name, int toAdd) {
        Restaurant r = ownedByCurrentUser(name);
        if (toAdd <= 0) throw new IllegalArgumentException("quantity to add must be positive");
        r.addQuantity(toAdd);
        return r.describe();
    }

    String updateLocation(String name, String pincodesSlashSeparated) {
        Restaurant r = ownedByCurrentUser(name);
        r.setPincodes(parsePincodes(pincodesSlashSeparated));
        return r.describe();
    }
```

```
    void rateRestaurant(String name, int rating, String comment) {
        restaurant(name).addReview(new Review(requireLogin().phone(), rating, comment == null ? "" : comment));
    }

    List<String> showRestaurants(SortBy sortBy) {
        String pin = requireLogin().pincode();
        return restaurants.values().stream().filter(r -> r.serviceableFor(pin))
                .sorted(sortBy.comparator).map(r -> r.name() + ", " + r.foodItem()).toList();
    }

    String placeOrder(String name, int qty) {
        User u = requireLogin();
        Restaurant r = restaurants.get(name);
        if (r == null || !r.pincodes().contains(u.pincode()) || !r.take(qty)) return "Cannot place order";
        record(u, r, qty);
        return "Order Placed Successfully.";
    }

    /** Variant: the system picks the restaurant. Only those that deliver here AND have capacity for qty qualify.
*/
    String placeOrder(int qty, RestaurantSelectionStrategy strategy) {
        User u = requireLogin();
        List<Restaurant> canFulfil = restaurants.values().stream()
                .filter(r -> r.pincodes().contains(u.pincode()) && r.quantity() >= qty).toList();
        for (int attempt = 0; attempt < 3; attempt++) {           // retry if a racer took the capacity
            Optional<Restaurant> pick = strategy.select(canFulfil);
            if (pick.isEmpty()) return "Cannot place order";
            if (pick.get().take(qty)) {
                record(u, pick.get(), qty);
                return "Order Placed Successfully from " + pick.get().name() + ".";
            }
            canFulfil = canFulfil.stream().filter(r -> r != pick.get()).toList();
        }
        return "Cannot place order";
    }

    List<FoodOrder> orderHistory() { return List.copyOf(history.getOrDefault(requireLogin().phone(), List.of())); }

    private void record(User u, Restaurant r, int qty) {
        FoodOrder o = new FoodOrder(orderSeq.incrementAndGet(), u.phone(), r.name(), r.foodItem(), qty,
                r.price().multiply(BigDecimal.valueOf(qty)));
        history.computeIfAbsent(u.phone(), k -> new CopyOnWriteArrayList<>()).add(o);
    }

    private User requireLogin() {
        User u = current;
        if (u == null) throw new IllegalStateException("no user logged in");
        return u;
    }

    private Restaurant restaurant(String name) {
        Restaurant r = restaurants.get(name);
        if (r == null) throw new IllegalArgumentException("no restaurant " + name);
        return r;
    }

    private Restaurant ownedByCurrentUser(String name) {
        Restaurant r = restaurant(name);
        if (!r.ownerPhone().equals(requireLogin().phone())) throw new IllegalStateException("only the owner can
update " + name);
        return r;
    }

    private static Set<String> parsePincodes(String s) {
        Set<String> out = new LinkedHashSet<>();
        for (String p : s.split("/")) if (!p.isBlank()) out.add(p.trim());
        if (out.isEmpty()) throw new IllegalArgumentException("at least one pincode needed");
        return out;
```

```
    }

}
```

```
// File: lld/ch19/foodkart/Main.java
package lld.ch19.foodkart;

import java.math.BigDecimal;

/** Driver: replays the sample test case from the problem statement, then the extras. */
public class Main {
    public static void main(String[] args) {
        FoodKartService fk = new FoodKartService();

        System.out.println("== Sample test case ==");
        fk.registerUser("Pralove", Gender.M, "phoneNumber-1", "HSR");
        fk.registerUser("Nitesh", Gender.M, "phoneNumber-2", "BTM");
        fk.registerUser("Vatsal", Gender.M, "phoneNumber-3", "BTM");

        fk.loginUser("phoneNumber-1");
        fk.registerRestaurant("Food Court-1", "BTM/HSR", "NI Thali", new BigDecimal("100"), 5);
        fk.registerRestaurant("Food Court-2", "BTM", "Burger", new BigDecimal("120"), 3);
        fk.loginUser("phoneNumber-2");
        fk.registerRestaurant("Food Court-3", "HSR", "SI Thali", new BigDecimal("150"), 1);
        fk.loginUser("phoneNumber-3");

        System.out.println("show_restaurant(price):");
        fk.showRestaurants(SortBy.PRICE).forEach(s -> System.out.println("  " + s));
        System.out.println("place_order(Food Court-1, 2): " + fk.placeOrder("Food Court-1", 2));
        System.out.println("place_order(Food Court-2, 7): " + fk.placeOrder("Food Court-2", 7));
        fk.rateRestaurant("Food Court-2", 3, "Good Food");
        fk.rateRestaurant("Food Court-1", 5, "Nice Food");
        System.out.println("show_restaurant(rating):");
        fk.showRestaurants(SortBy.RATING).forEach(s -> System.out.println("  " + s));

        fk.loginUser("phoneNumber-1");
        System.out.println("update_quantity(Food Court-2, 5): " + fk.updateQuantity("Food Court-2", 5));
        System.out.println("update_location(Food Court-2, BTM/HSR): " + fk.updateLocation("Food Court-2",
"BTM/HSR"));

        System.out.println("\n== Extras ==");
        fk.loginUser("phoneNumber-2");
        try { fk.updateQuantity("Food Court-1", 10); } catch (IllegalStateException e) { System.out.println("non-
owner update: " + e.getMessage()); }
        try { fk.rateRestaurant("Food Court-1", 6, null); } catch (IllegalArgumentException e) {
System.out.println("rating 6: " + e.getMessage()); }
        fk.rateRestaurant("Food Court-1", 4, null);             // rating without a comment
        System.out.println("order from a restaurant that does not deliver to BTM: " + fk.placeOrder("Food Court-3",
1));
        fk.registerRestaurant("Food Court-4", "BTM", "Dosa", new BigDecimal("80"), 4);

        System.out.println("\n== Selection strategy variant (user in BTM, wants 4 plates) ==");
        fk.loginUser("phoneNumber-3");
        System.out.println("highest rating: " + fk.placeOrder(4, new HighestRatingStrategy()));
        System.out.println("lowest price  : " + fk.placeOrder(4, new LowestPriceStrategy()));
        System.out.println("lowest price  : " + fk.placeOrder(4, new LowestPriceStrategy()));
        System.out.println("lowest price  : " + fk.placeOrder(4, new LowestPriceStrategy()) + "  (no one has 4
left)");

        System.out.println("\n== Order history of phoneNumber-3 ==");
        fk.orderHistory().forEach(o -> System.out.println("  " + o));
    }
}
```

Output:

```
== Sample test case ==
show_restaurant(price):
  Food Court-2, Burger
  Food Court-1, NI Thali
place_order(Food Court-1, 2): Order Placed Successfully.
place_order(Food Court-2, 7): Cannot place order
show_restaurant(rating):
  Food Court-1, NI Thali
  Food Court-2, Burger
update_quantity(Food Court-2, 5): Food Court-2, BTM, Burger - 8
update_location(Food Court-2, BTM/HSR): Food Court-2, BTM/HSR, Burger - 8

== Extras ==
non-owner update: only the owner can update Food Court-1
rating 6: rating must be 1..5
order from a restaurant that does not deliver to BTM: Cannot place order

== Selection strategy variant (user in BTM, wants 4 plates) ==
highest rating: Order Placed Successfully from Food Court-2.
lowest price  : Order Placed Successfully from Food Court-4.
lowest price  : Order Placed Successfully from Food Court-2.
lowest price  : Cannot place order  (no one has 4 left)

== Order history of phoneNumber-3 ==
  FoodOrder[id=1, userPhone=phoneNumber-3, restaurant=Food Court-1, item=NI Thali, quantity=2, total=200]
  FoodOrder[id=2, userPhone=phoneNumber-3, restaurant=Food Court-2, item=Burger, quantity=4, total=480]
  FoodOrder[id=3, userPhone=phoneNumber-3, restaurant=Food Court-4, item=Dosa, quantity=4, total=320]
  FoodOrder[id=4, userPhone=phoneNumber-3, restaurant=Food Court-2, item=Burger, quantity=4, total=480]
```

Reading the output. The "Sample test case" block reproduces every expected line from the problem statement: price-sorted list (Burger 120 before Thali 100), the successful and failed orders, the rating-sorted list, and the owner's quantity and location updates (`Burger - 8`). The extras show owner-only updates, rating validation, a rating without comment, and a restaurant that does not deliver to the user's pincode. The variant shows the two strategies choosing **different** restaurants (highest rating → Food Court-2; lowest price → Food Court-4), capacity filtering (Food Court-1 has only 3 plates, so it is never picked for 4), and a clean refusal once no one has 4 plates left.

Concurrency & edge cases

Case	Handling
Two claims for the same rider	`tryClaim()` CAS: only one succeeds; the loser moves to the next rider in its ranked list. (Inside one service the synchronized dispatcher already serialises claims; the CAS also covers claims made outside it.)
Two dispatch loops run at once	`dispatchWaiting` is `synchronized`, so the waiting queue is drained by one thread at a time (FIFO fairness preserved).
Kitchen over-booked	`tryReserveKitchen` is `synchronized` (check + add in one step).
Cancel races with accept	Both lock the `Order`; accept re-checks `PENDING`, cancel releases slots only if they were reserved (`CONFIRMED`). No leak, no double release.
Item sold out after it was added to the cart	`placeOrder` re-checks `available` and rejects.
Restaurant closes	`browse` hides it; `placeOrder` refuses.
COD cancelled	No refund (nothing was paid).
FoodKart: no user logged in	`requireLogin()` throws a clear `IllegalStateException`.
FoodKart: duplicate phone / restaurant name	Rejected with `putIfAbsent`.
FoodKart: rating outside 1–5	The `Review` record's compact constructor (a record's short validation block that runs on creation) rejects it.
FoodKart: strategy picks a restaurant whose capacity was just taken	`take()` fails; that restaurant is dropped and the strategy asked again (bounded retries).

Real-world example

In Swiggy and Zomato the order screen walks through the same states — order placed, restaurant confirmed, food being prepared, rider assigned / picked up, delivered — and if you add a dish from a second restaurant, the app asks whether to **reset your cart**, which is our one-restaurant `Cart` rule. The rider side is a dispatch problem: these platforms assign riders using distance, rider load and fairness signals, and may batch two orders on one trip — all variations of `DeliveryAssignmentStrategy`. FoodKart itself is a Flipkart machine-coding question: a 90-minute, in-memory, driver-class exercise that rewards clean separation (service, model, strategy) over clever algorithms.

Extensions & follow-up questions

- **"Batch two orders for one rider."** A strategy that accepts a rider already carrying one order if the second pickup is on the way; the agent then has a list of orders and a capacity (e.g. 2).
- **"ETA."** An `EtaEstimator` strategy: prep time (per restaurant load) + travel time (distance / speed, or a maps API — an online service other programs call).
- **"Surge / distance-based fees."** Extract `DeliveryFeePolicy` (Strategy): base + per-km today, add surge when waiting orders exceed free riders.
- **"Rider rejects an assignment."** Put the rider back to available, add the order back to the head of the queue, and exclude that rider for this order.
- **"Multiple dishes per restaurant in FoodKart."** Replace `foodItem/price/quantity` with a `Map<String, MenuItem>`; `take` becomes per item and all-or-nothing for multi-item orders.
- **"Order split across restaurants (Flipkart variant)."** The selection strategy returns a *set* of restaurants that together cover all items; reserve capacity in each, roll back all on any failure.
- **"Live tracking."** The rider app pushes location updates; an Observer forwards them to the customer's app (WebSocket).

In the interview

Design round (45 min): clarify (5) → entities + one-restaurant cart (8) → **order state machine** (7) → **assignment strategy + CAS claim + waiting queue** (12) → payments/notifications (5) → concurrency and extensions (8).

Machine-coding round (FoodKart, 90 min): model classes first (User, Restaurant, Review, Order) → service with one method per command → the driver replaying the sample → sort enum → owner checks and validation → selection strategy → order history. Keep it runnable at every step; print exactly the expected lines.

> **30-second answer:** "Restaurants own their menus and a kitchen capacity reserved on accept and released when food is ready. Orders copy item prices and move through a state machine from PENDING to DELIVERED. When food is ready the order joins a FIFO queue; a pluggable assignment strategy ranks free riders and we claim one with a compare-and-set, so a rider can never get two orders. Every status change is published to observers for the customer, restaurant and rider apps."

Follow-ups with answers

- *Why release kitchen capacity on "ready" and not on "delivered"?* The kitchen is free once the food leaves it; delivery time is the rider's constraint, not the kitchen's.
- *Why does the strategy return a list?* Because between ranking and claiming, the top rider may be claimed by someone else (another dispatcher or the rider's own app); the list lets us fall through without re-ranking.
- *How would you scale assignment across cities?* Partition by city/zone; each zone has its own dispatcher (single writer), so there is no cross-zone contention (threads or servers fighting over the same lock or data).

Self-check

1. **Q:** Which states allow a customer cancel, and why not later? **A:** PENDING and CONFIRMED; once cooking starts the food cost is already spent.
2. **Q:** What stops two orders getting the same rider? **A:** `AtomicBoolean.compareAndSet(true, false)` in `tryClaim()`.
3. **Q:** In FoodKart, what makes a restaurant "serviceable"? **A:** It delivers to the user's pincode and has quantity > 0.
4. **Q:** Why does `show_restaurant(price)` list Burger (120) before NI Thali (100)? **A:** The statement asks for descending order.
5. **Q:** How does the selection variant avoid choosing a restaurant that cannot fulfil the order? **A:** It filters by `quantity ≥ qty` before the strategy chooses, and `take()` re-checks atomically.
6. **Q:** Which patterns appear? **A:** Strategy (assignment, selection, payment), Observer (notifications), state machine (order status), Facade (service).

TL;DR

- One-restaurant cart; order copies prices; **state machine** PENDING → … → DELIVERED.
- **Kitchen capacity** reserved on accept, released on ready; **riders** claimed with **CAS**, chosen by a swappable **strategy**, with a FIFO queue when none are free.
- FoodKart: pincode-based serviceability, one dish per restaurant, average rating, `SortBy` enum, atomic `take`, and a **selection strategy** filtered by capacity.

Design a Grocery Store (POS & Checkout)

A **point-of-sale (POS) system** is the software behind a supermarket till: the cashier (or a self-checkout kiosk) scans items, the screen shows a running total, offers are applied, the customer pays in one or more ways, a receipt is printed, and stock goes down. Amazon (§19.1) was about an online cart that becomes an order; a grocery till is different in three ways that make it a good interview question:

- **Items are weighed as well as counted.** Loose produce is priced per kilogram; the scale sends a weight and the price is computed at the till.
- **Offers interact.** "Buy 2 get 1 free", "3 for ₹80", "10% off produce" and a basket coupon can all apply to one basket — and the rules for combining them decide what the customer pays, what tax is due, and how much a return refunds.
- **The till must never block a physical sale.** If the system says milk stock is 0 but a customer is holding a carton, the *count* is wrong, not the customer.

Money in this section is Indian rupees (₹; written `Rs` in the program output) and uses `BigDecimal`, following the chapter's four rules.

Problem & clarifying questions

Question	Assumed answer
Who uses it?	Cashiers at several registers (lanes) in one store; a supervisor for voids and refunds. Self-checkout uses the same register API.
How are items identified?	Packaged goods by **barcode** (EAN-13, the standard 13-digit product barcode); loose produce by a 4-digit **PLU** (price look-up) code plus the scale's weight.
Are shelf prices tax-inclusive?	Yes (as in India, the UK and the EU): the receipt shows how much tax the price *contains*. A US-style store adds sales tax at the till — the design must support both.
Which offers?	Buy X get Y free, multi-buy ("3 for ₹80"), percent off a department, and a basket coupon ("₹50 off when you spend ₹500").
Can offers stack?	Each product gets at most **one** item offer — the best for the customer. A basket coupon applies on top, to what is still owed.
How do people pay?	Cash (with change), card, UPI — and **split tender** (part UPI, part cash).
Can the customer change their mind?	Void a line while scanning; go back to scanning if nothing is paid yet; cancel the whole sale (card/UPI payments reversed).
Returns?	Yes, with the receipt: refund what the customer **actually paid** for that item, never more units than bought.
Stock?	Decrease on sale, increase on return; alert when low; never refuse a sale because of the stock count.

Requirements

Functional

1. Catalog lookup by barcode or PLU; weighed items priced by weight.
2. A **sale** collects line items; lines can be voided (kept on the receipt, marked VOID).
3. Item offers and basket coupons, with clear stacking rules.
4. Totals with tax per tax category (inclusive or exclusive policy).
5. Split payment: cash (with change), card, UPI; a declined card changes nothing.
6. Immutable **receipt** with lines, discounts, tax breakdown, tenders and change.
7. Returns against a receipt, refunding the net price paid, at most the quantity bought.
8. Stock updates and **low-stock / negative-stock alerts**.

Non-functional

- **Exact money**: `BigDecimal`, 2 decimal places, `HALF_UP` rounding (a 5 or more rounds up), and parts (per-product discounts, refunds) that always add up to the total.
- **Several registers at once** share the catalog, the stock and the receipts safely.
- **Deterministic pricing**: the same basket always prices the same way (a **pure function**: its result depends only on its inputs, and it changes nothing outside itself) — needed for audits and for returns.
- **Auditable**: voids stay on the receipt; prices are copied into lines at scan time.

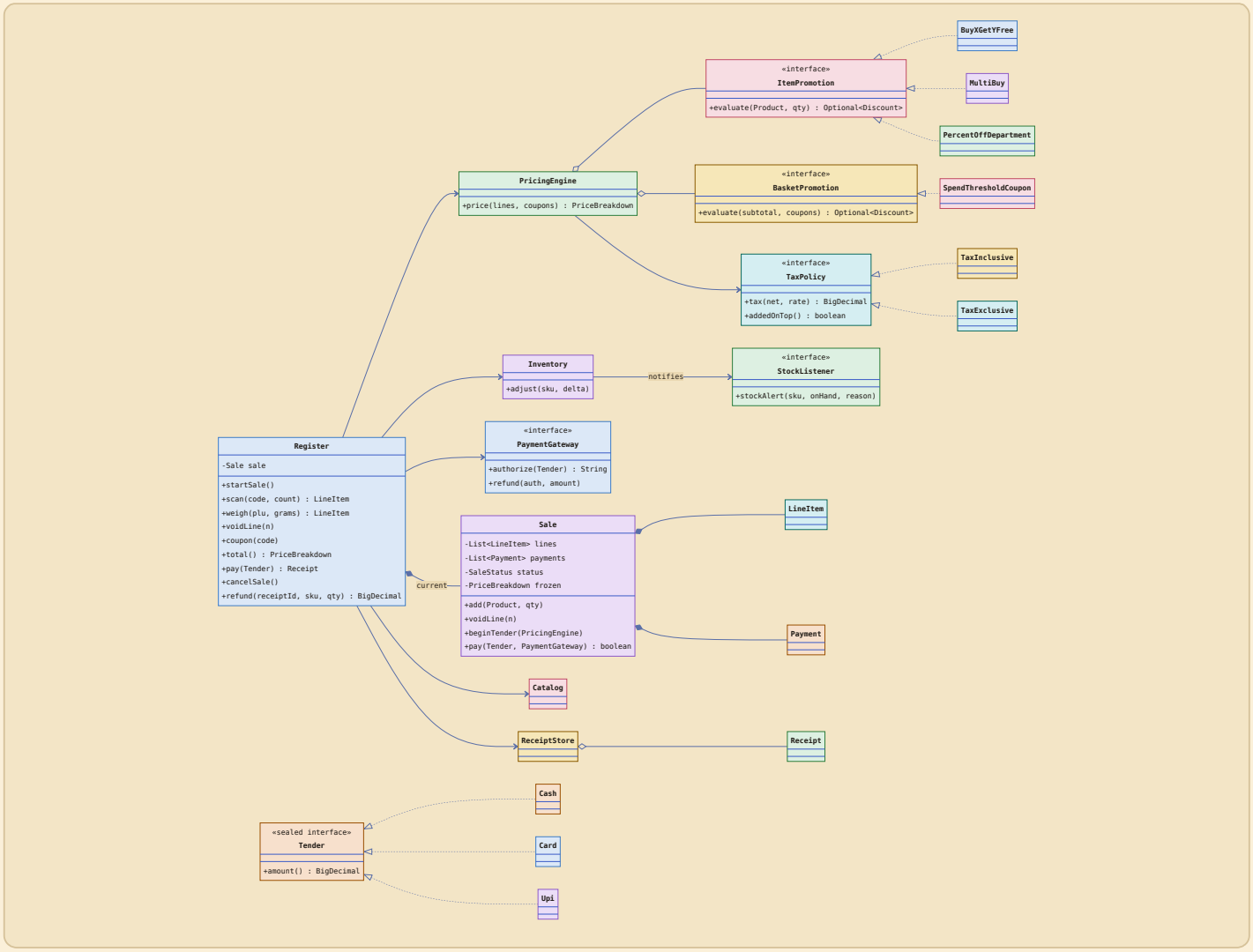
Use cases

Actor	Use cases
Cashier / self-checkout	start sale, scan, weigh, void line, add coupon, total, take payment, cancel sale
Customer	pay by cash / card / UPI, return items with the receipt
Supervisor	approve voids and returns (not modelled), reprice products
Payment network (system)	authorise and reverse card / UPI payments
Purchasing / store manager (subscriber)	receive low-stock and negative-stock alerts

Core entities

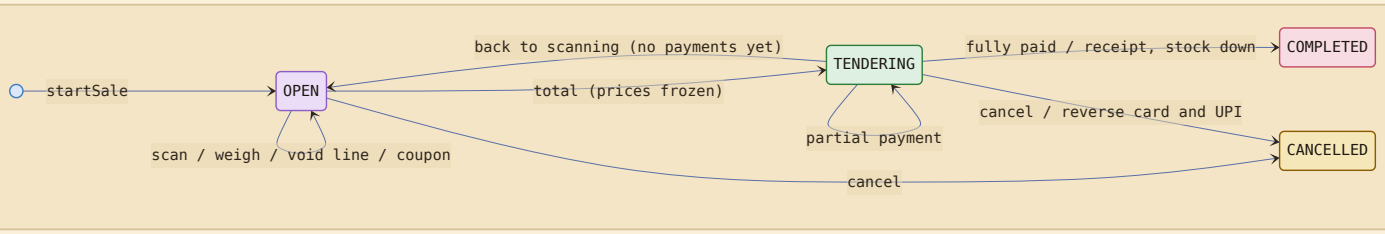
Class	Kind	Responsibility
`Product`	record	SKU (stock-keeping unit — the store's id for one distinct product), barcode/PLU, name, department, `PricingUnit` (EACH or PER_KG), unit price, `TaxCategory`.
`Catalog`	class	Lookup by code; `reprice` replaces the record (old sales keep old prices).
`LineItem`	record	One scan: product copy, quantity (count or kg), gross amount, voided flag.
`Sale` + `SaleStatus`	class + enum	OPEN → TENDERING → COMPLETED, or → CANCELLED; lines, coupons, payments, frozen price breakdown.
`ItemPromotion` + `BuyXGetYFree`, `MultiBuy`, `PercentOffDepartment`	interface + records	**Strategy** for item offers.
`BasketPromotion` + `SpendThresholdCoupon`	interface + record	**Strategy** for basket offers.
`TaxPolicy` + `TaxInclusive`, `TaxExclusive`	interface + classes	**Strategy** for how tax relates to the shelf price.
`PricingEngine` → `PriceBreakdown`	class → record	Pure function: lines + coupons → discounts, net per SKU, tax per category, total.
`Tender` (sealed) `Cash`, `Card`, `Upi`; `Payment`	records	How the customer pays; the part applied to the bill.
`PaymentGateway`	interface	Port (an interface through which the core code talks to an outside system) to the card / UPI networks (authorise, refund).
`Receipt`, `ReceiptStore`	record, class	Immutable proof of purchase; returns ledger per receipt and SKU.
`Inventory`, `StockListener`	class, interface	Shared stock; **Observer** for edge-triggered alerts (fired once when a line is crossed, not on every sale below it).
`Register`	class	**Facade** for one lane: the only API the till UI calls.

Class diagram

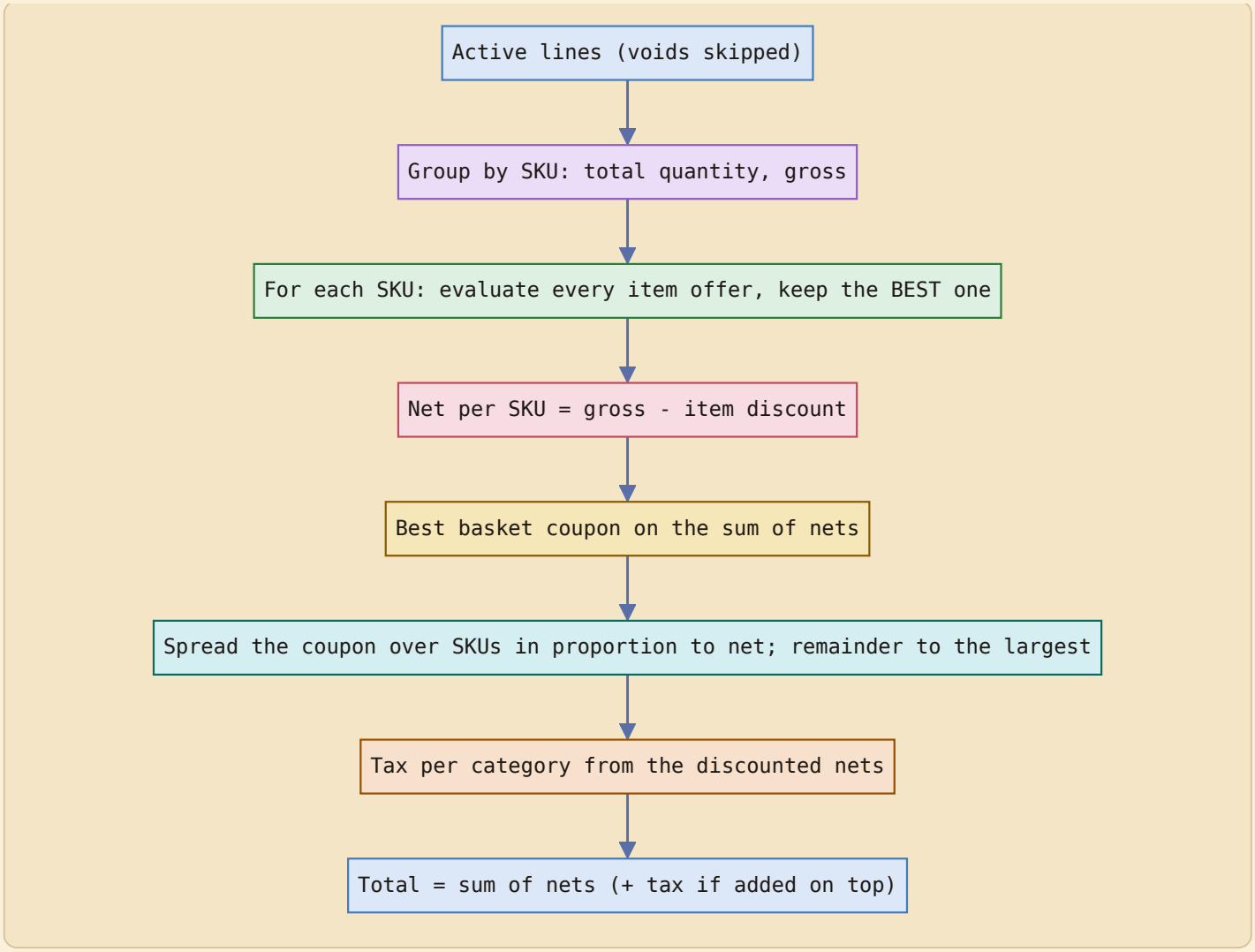


Key flows

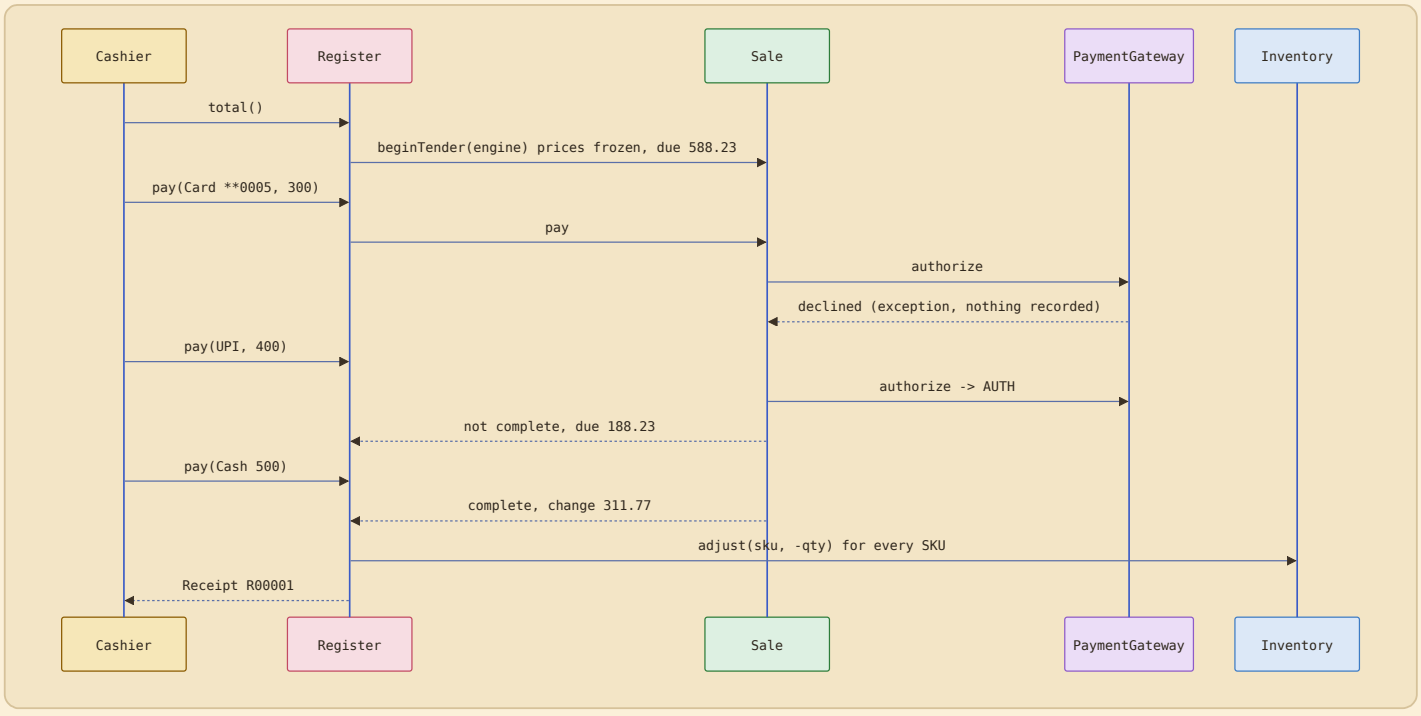
Sale lifecycle.



How one basket is priced (the hard method, `PricingEngine.price`):



Split payment.



Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each offer is a small class behind `ItemPromotion` / `BasketPromotion`	**Strategy**	Marketing adds offers weekly; a new offer is one new class, the engine does not change	A `switch` over offer types inside the pricing code
Best offer per SKU wins; basket coupon after item offers	Explicit stacking policy	Stacking rules are business decisions that must be written down once; "best for the customer" is simple to explain at the till	Applying every offer in list order (results depend on the order; can go below zero)
`PricingEngine` is a **pure function** of lines and coupons	Stateless service	Same basket → same price; one engine shared by all registers (stateless: it keeps no data between calls); easy to test	Mutating discounts into the line items as they are scanned
Prices **frozen** when payment starts	Snapshot	A price change or an offer expiring mid-payment must not change the amount due	Re-pricing on every payment
Coupon **spread over SKUs**, remainder to the largest	Allocation with exact rounding	Tax is per category and returns refund per item, so every rupee of discount must belong to some item — and the parts must add up exactly	Keeping the coupon as one unallocated number
Tax as a `TaxPolicy`	**Strategy**	Inclusive (India/EU) vs exclusive (US) is a store-level choice	Two versions of the engine
Tenders as a **sealed** interface	Sealed types	The receipt printer's `switch` is checked by the compiler	`String paymentType`
`Register` as the only API the UI calls	**Facade**	The UI does not need to know about engines, stores and gateways	UI code calling each part directly
Sale status in an enum with `canMoveTo`	Enum state machine	"Scan after payment started" and "pay a cancelled sale" fail loudly	Boolean flags (`isPaying`, `isDone`)
Stock may go **negative**, alerts are **edge-triggered**	**Observer**	The shelf is the truth; negative stock is a signal for a recount. Each crossing alerts once	Refusing the sale; alerting on every sale below the line
Voided lines are kept and printed	Audit trail (a permanent record of every action)	Voids are a common way to hide theft; supervisors review them	Deleting the line

Java implementation

Package `lld.ch19.grocery`.

```java
// File: lld/ch19/grocery/Catalog.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

/** Money helpers: BigDecimal, 2 decimal places, HALF_UP (the usual retail rounding). Never double. */
final class Money {
    private Money() { }
    static final BigDecimal ZERO = of("0");
    static BigDecimal of(String s) { return new BigDecimal(s).setScale(2, RoundingMode.HALF_UP); }
    static BigDecimal round(BigDecimal b) { return b.setScale(2, RoundingMode.HALF_UP); }
    static String fmt(BigDecimal b) { return "Rs " + round(b).toPlainString(); }
}

enum Department { PRODUCE, DAIRY, SNACKS, HOUSEHOLD }

/** How the shelf price is quoted: per piece, or per kilogram (weighed at the till). */
enum PricingUnit { EACH, PER_KG }

/** Tax slab. Rates are illustrative (loosely modelled on India's GST slabs), not tax advice. */
enum TaxCategory {
    EXEMPT("0"), REDUCED("5"), STANDARD("18");
    final BigDecimal ratePercent;
    TaxCategory(String r) { ratePercent = new BigDecimal(r); }
}

/** code = barcode (EAN-13) for packaged goods, or a 4-digit PLU for loose produce. */
record Product(String sku, String code, String name, Department dept, PricingUnit unit,
               BigDecimal unitPrice, TaxCategory tax) {
    Product {
        if (unitPrice.signum() <= 0) throw new PosException("Price must be positive: " + name);
        unitPrice = Money.round(unitPrice);
    }
}

/** Look-up by scanned code. Read far more than written, so a ConcurrentHashMap shared by all registers. */
final class Catalog {
    private final Map<String, Product> byCode = new ConcurrentHashMap<>();

    void add(Product p) {
        if (byCode.putIfAbsent(p.code(), p) != null) throw new PosException("Duplicate code " + p.code());
    }

    /** Price changes replace the record; open sales keep the price they already scanned. */
    void reprice(String code, BigDecimal newPrice) {
        byCode.computeIfPresent(code, (c, p) -> new Product(p.sku(), c, p.name(), p.dept(), p.unit(), newPrice,
p.tax()));
    }

    Optional<Product> find(String code) { return Optional.ofNullable(byCode.get(code)); }
}

final class PosException extends RuntimeException {
    PosException(String message) { super(message); }
}
```

```java
// File: lld/ch19/grocery/Promotions.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Optional;
import java.util.Set;

/** One discount line on the receipt. sku == null means a basket-level discount. */
record Discount(String description, String sku, BigDecimal amount) { }

/** Strategy for an item-level offer. Called once per SKU with the total quantity of that SKU in the basket. */
interface ItemPromotion {
    Optional<Discount> evaluate(Product p, BigDecimal quantity);
}

/** "Buy 2, get 1 free": in every group of (buy + free) units, the free units cost nothing. */
record BuyXGetYFree(String sku, int buy, int free) implements ItemPromotion {
    @Override public Optional<Discount> evaluate(Product p, BigDecimal qty) {
        if (!p.sku().equals(sku)) return Optional.empty();
        int groups = qty.intValue() / (buy + free);
        if (groups == 0) return Optional.empty();
        BigDecimal off = p.unitPrice().multiply(BigDecimal.valueOf((long) groups * free));
        return Optional.of(new Discount("Buy " + buy + " get " + free + " free: " + p.name(), sku,
Money.round(off)));
    }
}

/** "3 for Rs 100": each full group of n units costs a fixed price. */
record MultiBuy(String sku, int n, BigDecimal groupPrice) implements ItemPromotion {
    @Override public Optional<Discount> evaluate(Product p, BigDecimal qty) {
        if (!p.sku().equals(sku)) return Optional.empty();
        int groups = qty.intValue() / n;
        BigDecimal saving = p.unitPrice().multiply(BigDecimal.valueOf(n)).subtract(groupPrice);
        if (groups == 0 || saving.signum() <= 0) return Optional.empty();
        return Optional.of(new Discount(n + " for " + Money.fmt(groupPrice) + ": " + p.name(), sku,
                Money.round(saving.multiply(BigDecimal.valueOf(groups)))));
    }
}

/** "10% off all produce" — applies to weighed items too. */
record PercentOffDepartment(Department dept, int percent) implements ItemPromotion {
    @Override public Optional<Discount> evaluate(Product p, BigDecimal qty) {
        if (p.dept() != dept) return Optional.empty();
        BigDecimal gross = Money.round(p.unitPrice().multiply(qty));
        BigDecimal off = gross.multiply(BigDecimal.valueOf(percent)).divide(BigDecimal.valueOf(100), 2,
RoundingMode.HALF_UP);
        return Optional.of(new Discount(percent + "% off " + dept + ": " + p.name(), p.sku(), off));
    }
}

/** Strategy for a basket-level offer, evaluated after item offers, on what the customer still owes. */
interface BasketPromotion {
    Optional<Discount> evaluate(BigDecimal subtotalAfterItemOffers, Set<String> couponsPresented);
}

/** "Rs 50 off when you spend Rs 500 or more" — needs the coupon code to be scanned. */
record SpendThresholdCoupon(String code, BigDecimal minSpend, BigDecimal amountOff) implements BasketPromotion {
    @Override public Optional<Discount> evaluate(BigDecimal subtotal, Set<String> coupons) {
        if (!coupons.contains(code) || subtotal.compareTo(minSpend) < 0) return Optional.empty();
        return Optional.of(new Discount("Coupon " + code, null, amountOff.min(subtotal)));
    }
}
```

```java
// File: lld/ch19/grocery/Pricing.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.EnumMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;

/** Strategy: are shelf prices tax-inclusive (India, EU, UK) or is tax added at the till (US sales tax)? */
interface TaxPolicy {
    BigDecimal tax(BigDecimal net, BigDecimal ratePercent);
    boolean addedOnTop();
}

final class TaxInclusive implements TaxPolicy {
    /** The price already contains the tax: tax = price x r / (100 + r). */
    @Override public BigDecimal tax(BigDecimal net, BigDecimal r) {
        return net.multiply(r).divide(BigDecimal.valueOf(100).add(r), 2, RoundingMode.HALF_UP);
    }
    @Override public boolean addedOnTop() { return false; }
}

final class TaxExclusive implements TaxPolicy {
    @Override public BigDecimal tax(BigDecimal net, BigDecimal r) {
        return net.multiply(r).divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
    }
    @Override public boolean addedOnTop() { return true; }
}

/** All scanned lines of one SKU added together. */
record SkuTotal(Product product, BigDecimal quantity, BigDecimal gross) { }

/** Everything the receipt needs. netBySku = what the customer really paid per SKU, after every discount. */
record PriceBreakdown(List<SkuTotal> items, List<Discount> discounts, Map<String, BigDecimal> netBySku,
                      BigDecimal gross, BigDecimal discountTotal, Map<TaxCategory, BigDecimal> taxByCategory,
                      BigDecimal total) { }

/**
 * Pure function of (lines, coupons): no state, so every register can share one engine and
 * the same basket always prices the same way. Rules:
 *  1. each SKU gets at most ONE item offer — the best one for the customer;
 *  2. then the best basket coupon, on what is still owed;
 *  3. basket discounts are spread over SKUs in proportion to their value (needed for tax and refunds);
 *  4. tax is computed per tax category on the discounted amounts.
 */
final class PricingEngine {
    private final List<ItemPromotion> itemOffers;
    private final List<BasketPromotion> basketOffers;
    private final TaxPolicy taxPolicy;

    PricingEngine(List<ItemPromotion> itemOffers, List<BasketPromotion> basketOffers, TaxPolicy taxPolicy) {
        this.itemOffers = List.copyOf(itemOffers);
        this.basketOffers = List.copyOf(basketOffers);
        this.taxPolicy = taxPolicy;
    }

    PriceBreakdown price(List<LineItem> lines, Set<String> coupons) {
        Map<String, SkuTotal> bySku = new LinkedHashMap<>();
        for (LineItem l : lines) {
            if (l.voided()) continue;
            bySku.merge(l.product().sku(), new SkuTotal(l.product(), l.quantity(), l.gross()),
                    (a, b) -> new SkuTotal(a.product(), a.quantity().add(b.quantity()), a.gross().add(b.gross())));
```

```
        }
        List<Discount> discounts = new ArrayList<>();
        Map<String, BigDecimal> net = new LinkedHashMap<>();
        for (SkuTotal t : bySku.values()) {
            Optional<Discount> best = itemOffers.stream()
                    .map(p -> p.evaluate(t.product(), t.quantity()))
                    .flatMap(Optional::stream)
                    .max(Comparator.comparing(Discount::amount));
            BigDecimal off = best.map(d -> d.amount().min(t.gross())).orElse(Money.ZERO);
            best.ifPresent(d -> discounts.add(new Discount(d.description(), d.sku(), off)));
            net.put(t.product().sku(), t.gross().subtract(off));
        }
        BigDecimal subtotal = sum(net.values());
        Optional<Discount> basket = basketOffers.stream()
                .map(b -> b.evaluate(subtotal, coupons))
                .flatMap(Optional::stream)
                .max(Comparator.comparing(Discount::amount));
        basket.ifPresent(d -> { discounts.add(d); spread(d.amount(), net); });

        Map<TaxCategory, BigDecimal> netByTax = new EnumMap<>(TaxCategory.class);
        bySku.values().forEach(t -> netByTax.merge(t.product().tax(), net.get(t.product().sku()),
BigDecimal::add));
        Map<TaxCategory, BigDecimal> taxByCategory = new EnumMap<>(TaxCategory.class);
        netByTax.forEach((cat, amt) -> taxByCategory.put(cat, taxPolicy.tax(amt, cat.ratePercent)));

        BigDecimal gross = sum(bySku.values().stream().map(SkuTotal::gross).toList());
        BigDecimal netTotal = sum(net.values());
        BigDecimal total = taxPolicy.addedOnTop() ? netTotal.add(sum(taxByCategory.values())) : netTotal;
        return new PriceBreakdown(List.copyOf(bySku.values()), List.copyOf(discounts), Map.copyOf(net),
                gross, gross.subtract(netTotal), taxByCategory, total);
    }

    /** Proportional allocation; the rounding remainder goes to the largest SKU so the parts add up exactly. */
    private static void spread(BigDecimal amount, Map<String, BigDecimal> net) {
        BigDecimal base = sum(net.values());
        if (base.signum() == 0) return;
        BigDecimal given = Money.ZERO;
        String largest = null;
        for (var e : net.entrySet()) {
            BigDecimal share = amount.multiply(e.getValue()).divide(base, 2, RoundingMode.HALF_UP);
            e.setValue(e.getValue().subtract(share));
            given = given.add(share);
            if (largest == null || e.getValue().compareTo(net.get(largest)) > 0) largest = e.getKey();
        }
        net.merge(largest, given.subtract(amount), BigDecimal::add);
    }

    private static BigDecimal sum(Iterable<BigDecimal> xs) {
        BigDecimal s = Money.ZERO;
        for (BigDecimal x : xs) s = s.add(x);
        return s;
    }
}
```

```java
// File: lld/ch19/grocery/Sale.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

/** One scan. The product (and so its price) is copied in at scan time; later reprices do not change it. */
record LineItem(int lineNo, Product product, BigDecimal quantity, BigDecimal gross, boolean voided) {
    LineItem asVoided() { return new LineItem(lineNo, product, quantity, gross, true); }
}

enum SaleStatus {
    OPEN, TENDERING, COMPLETED, CANCELLED;

    boolean canMoveTo(SaleStatus next) {
        return switch (this) {
            case OPEN -> next == TENDERING || next == CANCELLED;
            case TENDERING -> next == COMPLETED || next == OPEN || next == CANCELLED;
            case COMPLETED, CANCELLED -> false;
        };
    }
}

/** How the customer pays. Sealed: the compiler knows every kind of tender. */
sealed interface Tender permits Cash, Card, Upi { BigDecimal amount(); }
record Cash(BigDecimal amount) implements Tender { }
record Card(String last4, BigDecimal amount) implements Tender { }
record Upi(String vpa, BigDecimal amount) implements Tender { }

/** applied = the part of the tender that went towards the bill (cash can be more; the rest is change). */
record Payment(Tender tender, BigDecimal applied, String authCode) { }

/** Strategy / port to the card and UPI networks. Cash needs no gateway. */
interface PaymentGateway {
    String authorize(Tender t);          // returns an auth code or throws PosException("declined ...")
    void refund(String authCode, BigDecimal amount);
}

/** One customer's basket at one register, from first scan to receipt. */
final class Sale {
    final String id;
    private final List<LineItem> lines = new ArrayList<>();
    private final Set<String> coupons = new LinkedHashSet<>();
    private final List<Payment> payments = new ArrayList<>();
    private SaleStatus status = SaleStatus.OPEN;
    private PriceBreakdown frozen;       // prices are frozen when payment starts
    private BigDecimal change = Money.ZERO;

    Sale(String id) { this.id = id; }

    SaleStatus status() { return status; }
    List<LineItem> lines() { return List.copyOf(lines); }
    List<Payment> payments() { return List.copyOf(payments); }
    Set<String> coupons() { return Set.copyOf(coupons); }
    BigDecimal change() { return change; }
    PriceBreakdown breakdown() { return frozen; }

    LineItem add(Product p, BigDecimal qty) {
        require(SaleStatus.OPEN, "scan");
        if (qty.signum() <= 0) throw new PosException("Quantity must be positive");
        LineItem l = new LineItem(lines.size() + 1, p, qty, Money.round(p.unitPrice().multiply(qty)), false);
        lines.add(l);
        return l;
    }
```

```
    void voidLine(int lineNo) {
        require(SaleStatus.OPEN, "void a line");
        if (lineNo < 1 || lineNo > lines.size()) throw new PosException("No line " + lineNo);
        LineItem l = lines.get(lineNo - 1);
        if (l.voided()) throw new PosException("Line " + lineNo + " is already void");
        lines.set(lineNo - 1, l.asVoided());          // keep the line: the audit trail shows the void
    }

    void addCoupon(String code) { require(SaleStatus.OPEN, "add a coupon"); coupons.add(code); }

    PriceBreakdown beginTender(PricingEngine engine) {
        require(SaleStatus.OPEN, "start payment");
        if (lines.stream().allMatch(LineItem::voided)) throw new PosException("Nothing to pay for");
        frozen = engine.price(lines, coupons);
        move(SaleStatus.TENDERING);
        return frozen;
    }

    /** Back to scanning ("I forgot the milk") — only while nothing has been paid. */
    void reopen() {
        if (!payments.isEmpty()) throw new PosException("Payments already taken; cancel them first");
        frozen = null;
        move(SaleStatus.OPEN);
    }

    BigDecimal due() {
        BigDecimal paid = payments.stream().map(Payment::applied).reduce(Money.ZERO, BigDecimal::add);
        return frozen.total().subtract(paid);
    }

    /** Returns true when the bill is fully paid and the sale is complete. */
    boolean pay(Tender t, PaymentGateway gateway) {
        require(SaleStatus.TENDERING, "pay");
        BigDecimal due = due();
        if (t.amount().signum() <= 0) throw new PosException("Tender must be positive");
        String auth = "CASH";
        BigDecimal applied = t.amount();
        if (t instanceof Cash) {
            if (t.amount().compareTo(due) > 0) { change = t.amount().subtract(due); applied = due; }
        } else {
            if (t.amount().compareTo(due) > 0) throw new PosException("Card/UPI cannot be more than the amount due
" + Money.fmt(due));
            auth = gateway.authorize(t);                // throws if declined: nothing recorded
        }
        payments.add(new Payment(t, applied, auth));
        if (due().signum() == 0) { move(SaleStatus.COMPLETED); return true; }
        return false;
    }

    /** Cancel the whole sale; card/UPI payments already taken are reversed. */
    void cancel(PaymentGateway gateway) {
        if (!status.canMoveTo(SaleStatus.CANCELLED)) throw new PosException("Cannot cancel a " + status + " sale");
        for (Payment p : payments) if (!(p.tender() instanceof Cash)) gateway.refund(p.authCode(), p.applied());
        move(SaleStatus.CANCELLED);
    }

    private void require(SaleStatus s, String action) {
        if (status != s) throw new PosException("Cannot " + action + " when the sale is " + status);
    }

    private void move(SaleStatus next) {
        if (!status.canMoveTo(next)) throw new PosException(status + " -> " + next + " not allowed");
        status = next;
    }
}
```

```java
// File: lld/ch19/grocery/Register.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

/** Observer: purchasing, the store manager's dashboard, shrinkage reports. */
interface StockListener {
    void stockAlert(String sku, BigDecimal onHand, String reason);
}

/**
 * Store stock shared by every register. A till never refuses a sale because the SYSTEM says zero:
 * the item is physically in the customer's hand, so the count is what is wrong. Stock may go
 * negative, and that is reported as a discrepancy for a recount.
 */
final class Inventory {
    private final Map<String, BigDecimal> onHand = new ConcurrentHashMap<>();
    private final Map<String, BigDecimal> reorderPoint = new ConcurrentHashMap<>();
    private final List<StockListener> listeners = new CopyOnWriteArrayList<>();

    void set(String sku, String qty, String reorderAt) {
        onHand.put(sku, new BigDecimal(qty));
        reorderPoint.put(sku, new BigDecimal(reorderAt));
    }
    void addListener(StockListener l) { listeners.add(l); }
    BigDecimal onHand(String sku) { return onHand.getOrDefault(sku, BigDecimal.ZERO); }

    /** delta < 0 for a sale, > 0 for a return. merge() is atomic per key, so registers never lose updates. */
    void adjust(String sku, BigDecimal delta) {
        BigDecimal after = onHand.merge(sku, delta, BigDecimal::add);
        BigDecimal before = after.subtract(delta);
        BigDecimal point = reorderPoint.getOrDefault(sku, BigDecimal.ZERO);
        // edge-triggered: alert only when THIS change crosses the line, so each crossing alerts exactly once
        if (before.compareTo(point) > 0 && after.compareTo(point) <= 0)
            listeners.forEach(l -> l.stockAlert(sku, after, "LOW - reorder"));
        if (before.signum() >= 0 && after.signum() < 0)
            listeners.forEach(l -> l.stockAlert(sku, after, "NEGATIVE - recount the shelf"));
    }
}

/** Immutable proof of purchase. */
record Receipt(String id, String registerId, List<LineItem> lines, PriceBreakdown breakdown,
               List<Payment> payments, BigDecimal change) {

    String render() {
        StringBuilder sb = new StringBuilder();
        sb.append("  ---------------- RECEIPT ").append(id).append(" (").append(registerId).append(") ----------
---\n");
        for (LineItem l : lines) {
            String qty = l.product().unit() == PricingUnit.PER_KG
                    ? l.quantity().setScale(3, RoundingMode.UNNECESSARY) + " kg @ " +
Money.fmt(l.product().unitPrice()) + "/kg"
                    : l.quantity().toPlainString() + " x " + Money.fmt(l.product().unitPrice());
            sb.append("  %2d %-18s %-24s %12s%s%n".formatted(l.lineNo(), l.product().name(), qty,
Money.fmt(l.gross()),
                    l.voided() ? "  VOID" : ""));
        }
        sb.append("  %-46s %12s%n".formatted("Gross", Money.fmt(breakdown.gross())));
        for (Discount d : breakdown.discounts())
            sb.append("  %-46s %12s%n".formatted("  " + d.description(), "-" + Money.fmt(d.amount())));
        sb.append("  %-46s %12s%n".formatted("TOTAL", Money.fmt(breakdown.total())));
        breakdown.taxByCategory().forEach((cat, tax) ->
```

```
                sb.append("  %-46s %12s%n".formatted("  incl. tax " + cat + " @" + cat.ratePercent + "%",
Money.fmt(tax))));
        for (Payment p : payments) sb.append("  %-46s %12s%n".formatted("Paid " + describe(p.tender()),
Money.fmt(p.tender().amount())));
        if (change.signum() > 0) sb.append("  %-46s %12s%n".formatted("Change", Money.fmt(change)));
        return sb.toString();
    }

    private static String describe(Tender t) {
        return switch (t) {
            case Cash c -> "cash";
            case Card c -> "card **" + c.last4();
            case Upi u -> "UPI " + u.vpa();
        };
    }
}

/** Receipts from every register, plus what has already been returned against each (shared, thread-safe). */
final class ReceiptStore {
    private final Map<String, Receipt> receipts = new ConcurrentHashMap<>();
    private final Map<String, BigDecimal> returned = new ConcurrentHashMap<>();   // key receiptId/sku
    private final AtomicLong nextId = new AtomicLong(1);

    String newId() { return "R%05d".formatted(nextId.getAndIncrement()); }
    void save(Receipt r) { receipts.put(r.id(), r); }
    Receipt get(String id) {
        Receipt r = receipts.get(id);
        if (r == null) throw new PosException("Unknown receipt " + id);
        return r;
    }

    /** Atomically claims qty units for return (two counters cannot refund the same unit twice); returns the total
claimed so far. */
    BigDecimal claimReturn(Receipt r, String sku, BigDecimal qty, BigDecimal bought) {
        return returned.compute(r.id() + "/" + sku, (k, already) -> {
            BigDecimal next = (already == null ? BigDecimal.ZERO : already).add(qty);
            if (next.compareTo(bought) > 0)
                throw new PosException("Only " +
bought.subtract(next.subtract(qty)).stripTrailingZeros().toPlainString()
                        + " of " + sku + " left to return on " + r.id());
            return next;
        });
    }
}

/**
 * Facade for one checkout lane. A register is driven by one cashier (one thread); the things it
 * shares with other registers — catalog, inventory, receipts — are the thread-safe ones.
 */
final class Register {
    private final String id;
    private final Catalog catalog;
    private final PricingEngine engine;
    private final Inventory inventory;
    private final ReceiptStore receipts;
    private final PaymentGateway gateway;
    private Sale sale;
    private int saleNo = 1;

    Register(String id, Catalog catalog, PricingEngine engine, Inventory inventory, ReceiptStore receipts,
PaymentGateway gateway) {
        this.id = id; this.catalog = catalog; this.engine = engine;
        this.inventory = inventory; this.receipts = receipts; this.gateway = gateway;
    }

    void startSale() {
        if (sale != null && (sale.status() == SaleStatus.OPEN || sale.status() == SaleStatus.TENDERING))
            throw new PosException("Finish or cancel sale " + sale.id + " first");
        sale = new Sale(id + "-S" + saleNo++);
```

```
    }

    LineItem scan(String code) { return scan(code, 1); }

    LineItem scan(String code, int count) {
        Product p = lookup(code);
        if (p.unit() != PricingUnit.EACH) throw new PosException(p.name() + " is sold by weight: put it on the
scale");
        return current().add(p, BigDecimal.valueOf(count));
    }

    /** Loose produce: the cashier keys the PLU and the scale sends the weight in grams. */
    LineItem weigh(String plu, int grams) {
        Product p = lookup(plu);
        if (p.unit() != PricingUnit.PER_KG) throw new PosException(p.name() + " is not sold by weight");
        if (grams <= 0) throw new PosException("Scale reading must be positive");
        return current().add(p, BigDecimal.valueOf(grams).movePointLeft(3));
    }

    void voidLine(int lineNo) { current().voidLine(lineNo); }
    void coupon(String code) { current().addCoupon(code); }
    PriceBreakdown total() { return current().beginTender(engine); }
    void backToScanning() { current().reopen(); }
    void cancelSale() { current().cancel(gateway); }

    /** Returns the receipt when the bill is fully paid, else null (more tenders needed). */
    Receipt pay(Tender t) {
        Sale s = current();
        if (!s.pay(t, gateway)) return null;
        for (SkuTotal item : s.breakdown().items()) inventory.adjust(item.product().sku(),
item.quantity().negate());
        Receipt r = new Receipt(receipts.newId(), id, s.lines(), s.breakdown(), s.payments(), s.change());
        receipts.save(r);
        return r;
    }

    /** Refund what the customer actually paid per unit (after offers and coupons), not the shelf price. */
    BigDecimal refund(String receiptId, String sku, int qty) {
        Receipt r = receipts.get(receiptId);
        SkuTotal bought = r.breakdown().items().stream().filter(t -> t.product().sku().equals(sku)).findFirst()
                .orElseThrow(() -> new PosException(sku + " is not on receipt " + receiptId));
        BigDecimal q = BigDecimal.valueOf(qty);
        BigDecimal after = receipts.claimReturn(r, sku, q, bought.quantity());
        // refund = share(after) - share(before): the parts always add up to exactly what was paid
        BigDecimal net = r.breakdown().netBySku().get(sku);
        BigDecimal refund = share(net, after, bought.quantity()).subtract(share(net, after.subtract(q),
bought.quantity()));
        inventory.adjust(sku, q);
        return refund;
    }

    private static BigDecimal share(BigDecimal net, BigDecimal units, BigDecimal of) {
        return net.multiply(units).divide(of, 2, RoundingMode.HALF_UP);
    }

    private Product lookup(String code) {
        if (code.length() == 13 && !Ean13.valid(code)) throw new PosException("Bad barcode " + code + ": check
digit wrong, rescan");
        return catalog.find(code).orElseThrow(() -> new PosException("Unknown code " + code + ": call a
supervisor"));
    }

    private Sale current() {
        if (sale == null) throw new PosException("No sale in progress");
        return sale;
    }
}

/** EAN-13 check digit: weights 1,3,1,3... on the first 12 digits; the 13th makes the sum a multiple of 10. */
```

```
final class Ean13 {
    private Ean13() { }
    static String withCheckDigit(String first12) {
        int sum = 0;
        for (int i = 0; i < 12; i++) sum += (first12.charAt(i) - '0') * (i % 2 == 0 ? 1 : 3);
        return first12 + (10 - sum % 10) % 10;
    }
    static boolean valid(String code) {
        return code.chars().allMatch(Character::isDigit) && withCheckDigit(code.substring(0, 12)).equals(code);
    }
}
```

```java
// File: lld/ch19/grocery/Main.java
package lld.ch19.grocery;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    /** Fake card/UPI network: declines one blocked card, approves everything else. */
    static final class FakeGateway implements PaymentGateway {
        private final AtomicInteger seq = new AtomicInteger(1000);
        @Override public String authorize(Tender t) {
            if (t instanceof Card c && c.last4().equals("0005")) throw new PosException("Card **0005 declined by
issuer");
            return "AUTH" + seq.incrementAndGet();
        }
        @Override public void refund(String auth, BigDecimal amount) {
            System.out.println("   gateway: reversed " + Money.fmt(amount) + " on " + auth);
        }
    }

    static final String MILK = Ean13.withCheckDigit("890100000001"), PANEER = Ean13.withCheckDigit("890100000002"),
            CHIPS = Ean13.withCheckDigit("890100000003"), BISCUITS = Ean13.withCheckDigit("890100000004"),
            DETERGENT = Ean13.withCheckDigit("890100000005"), BANANAS = "4011", TOMATOES = "4664";

    public static void main(String[] args) throws Exception {
        Catalog catalog = new Catalog();
        catalog.add(new Product("BAN", BANANAS, "Bananas", Department.PRODUCE, PricingUnit.PER_KG, Money.of("60"),
TaxCategory.EXEMPT));
        catalog.add(new Product("TOM", TOMATOES, "Tomatoes", Department.PRODUCE, PricingUnit.PER_KG,
Money.of("40"), TaxCategory.EXEMPT));
        catalog.add(new Product("MLK", MILK, "Milk 1L", Department.DAIRY, PricingUnit.EACH, Money.of("68"),
TaxCategory.EXEMPT));
        catalog.add(new Product("PNR", PANEER, "Paneer 200g", Department.DAIRY, PricingUnit.EACH, Money.of("90"),
TaxCategory.REDUCED));
        catalog.add(new Product("CHP", CHIPS, "Chips 50g", Department.SNACKS, PricingUnit.EACH, Money.of("20"),
TaxCategory.STANDARD));
        catalog.add(new Product("BIS", BISCUITS, "Biscuits", Department.SNACKS, PricingUnit.EACH, Money.of("30"),
TaxCategory.REDUCED));
        catalog.add(new Product("DET", DETERGENT, "Detergent 1kg", Department.HOUSEHOLD, PricingUnit.EACH,
Money.of("245"), TaxCategory.STANDARD));

        List<ItemPromotion> offers = List.of(
                new BuyXGetYFree("CHP", 2, 1),                      // saves 20 on 3 packs
                new MultiBuy("CHP", 3, Money.of("50")),              // saves 10 on 3 packs: loses to the one
above
                new MultiBuy("BIS", 3, Money.of("80")),
                new PercentOffDepartment(Department.PRODUCE, 10));
        List<BasketPromotion> coupons = List.of(new SpendThresholdCoupon("SAVE50", Money.of("500"),
Money.of("50")));
        PricingEngine engine = new PricingEngine(offers, coupons, new TaxInclusive());

        Inventory stock = new Inventory();
        for (String sku : List.of("BAN", "TOM", "MLK", "CHP", "BIS", "DET")) stock.set(sku, "200", "20");
        stock.set("PNR", "100", "20");
        List<String> alerts = Collections.synchronizedList(new ArrayList<>());
        stock.addListener((sku, left, why) -> alerts.add(sku + " " + why + " (on hand " +
left.stripTrailingZeros().toPlainString() + ")"));
        ReceiptStore receipts = new ReceiptStore();
        FakeGateway gateway = new FakeGateway();
        Register lane1 = new Register("LANE-1", catalog, engine, stock, receipts, gateway);

        System.out.println("== 1. Scan a basket ==");
```

```
        lane1.startSale();
        lane1.weigh(BANANAS, 1245);
        lane1.weigh(TOMATOES, 500);
        lane1.scan(MILK);
        lane1.scan(MILK);
        lane1.scan(PANEER);
        lane1.scan(CHIPS, 3);
        lane1.scan(BISCUITS, 4);
        lane1.scan(DETERGENT);
        lane1.voidLine(4);                                          // customer puts one milk back
        for (String bad : List.of(Ean13.withCheckDigit("890100000009"), "8901000000018", BANANAS)) {
            try { lane1.scan(bad); } catch (PosException e) { System.out.println("   error: " + e.getMessage()); }
        }
        lane1.coupon("SAVE50");

        System.out.println("\n== 2. Total, then split payment: declined card, UPI, cash with change ==");
        PriceBreakdown bill = lane1.total();
        System.out.println("   amount due " + Money.fmt(bill.total()));
        try { lane1.scan(MILK); } catch (PosException e) { System.out.println("   error: " + e.getMessage()); }
        try { lane1.pay(new Card("0005", Money.of("300"))); } catch (PosException e) { System.out.println("
error: " + e.getMessage()); }
        try { lane1.pay(new Upi("asha@okbank", Money.of("5000"))); } catch (PosException e) { System.out.println("
error: " + e.getMessage()); }
        System.out.println("   UPI 400 -> receipt? " + lane1.pay(new Upi("asha@okbank", Money.of("400"))));
        Receipt r1 = lane1.pay(new Cash(Money.of("500")));
        System.out.print(r1.render());

        System.out.println("\n== 3. Returns refund what was really paid ==");
        System.out.println("   return 1 of 3 chips -> refund " + Money.fmt(lane1.refund(r1.id(), "CHP", 1)));
        System.out.println("   return 2 more chips -> refund " + Money.fmt(lane1.refund(r1.id(), "CHP", 2))
                + "  (all three add up to the " + Money.fmt(r1.breakdown().netBySku().get("CHP")) + " paid)");
        try { lane1.refund(r1.id(), "CHP", 1); } catch (PosException e) { System.out.println("   error: " +
e.getMessage()); }
        try { lane1.refund(r1.id(), "PNR", 2); } catch (PosException e) { System.out.println("   error: " +
e.getMessage()); }

        System.out.println("\n== 4. Price change during a sale; going back to scanning; cancelling ==");
        lane1.startSale();
        lane1.scan(MILK);
        catalog.reprice(MILK, Money.of("72"));
        lane1.scan(MILK);
        SkuTotal milk = lane1.total().items().get(0);
        System.out.println("   2 milk scanned around a reprice: gross " + Money.fmt(milk.gross()) + " = Rs 68 (old
price kept) + Rs 72 (new)");
        lane1.backToScanning();
        lane1.scan(PANEER);
        System.out.println("   after adding paneer, due " + Money.fmt(lane1.total().total()));
        lane1.pay(new Card("4242", Money.of("100")));
        try { lane1.backToScanning(); } catch (PosException e) { System.out.println("   error: " + e.getMessage());
}
        lane1.cancelSale();
        try { lane1.pay(new Cash(Money.of("10"))); } catch (PosException e) { System.out.println("   error: " +
e.getMessage()); }

        System.out.println("\n== 5. Same basket, US-style tax added at the till ==");
        Sale same = new Sale("X");
        r1.lines().forEach(l -> { if (!l.voided()) same.add(l.product(), l.quantity()); });
        same.addCoupon("SAVE50");
        PriceBreakdown us = same.beginTender(new PricingEngine(offers, coupons, new TaxExclusive()));
        System.out.println("   inclusive total " + Money.fmt(r1.breakdown().total()) + " vs exclusive total " +
Money.fmt(us.total())
                + " (tax " + us.taxByCategory() + ")");

        System.out.println("\n== 6. Four registers sell 30 paneer each at once; stock was 100 ==");
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(4)) {
            for (int i = 1; i <= 4; i++) {
                Register lane = new Register("LANE-" + (i + 1), catalog, engine, stock, receipts, gateway);
                pool.submit(() -> {
```

```
                    go.await();
                    for (int k = 0; k < 30; k++) {
                        lane.startSale();
                        lane.scan(PANEER);
                        lane.total();
                        lane.pay(new Cash(Money.of("100")));
                    }
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("   paneer on hand: " + stock.onHand("PNR") + "  (sold 120 + 1 earlier; the till never
refuses a real item)");
        alerts.stream().sorted().forEach(a -> System.out.println("   alert: " + a));
    }
}
```

How the code works, part by part

- **Scanning.** `Register.scan` checks the EAN-13 check digit (a last digit computed from the others, so a misread code fails the sum) first (a misread barcode is caught before lookup), then refuses weighed products; `weigh` converts grams to kilograms with `movePointLeft(3)` so 1245 g becomes exactly `1.245`. `Sale.add` copies the product — and therefore its price — into the new `LineItem`.
- **Pricing.** `PricingEngine.price` skips voided lines, groups by SKU, asks every item offer for a discount and keeps the largest (capped at the gross), applies the best basket coupon to what is left, spreads that coupon across SKUs, and computes tax per category on the discounted amounts. With `TaxInclusive`, tax = amount × r ÷ (100 + r); with `TaxExclusive`, tax = amount × r ÷ 100 and is added to the total.
- **Paying.** `beginTender` freezes the breakdown. `pay` treats cash specially: if it is more than the amount due, the extra becomes change. Card and UPI may not exceed the amount due and must be authorised; a declined authorisation throws before anything is recorded. When the amount due reaches zero, the sale completes and the register writes the receipt and reduces stock.
- **Returns.** `ReceiptStore.claimReturn` uses `ConcurrentHashMap.compute` so "check how many were already returned, then add" is atomic. The refund for a return is `share(after) − share(before)`, where `share(k)` is the net paid for the first *k* units; that way three separate one-item returns add up to exactly what was paid.
- **Stock.** `Inventory.adjust` uses `merge`, which is atomic per key; it compares the value before and after *this* change, so each crossing of the reorder point — the stock level at which more should be ordered — (or of zero) alerts exactly once, however many registers sell at the same time.

Output:

```
== 1. Scan a basket ==
   error: Unknown code 8901000000095: call a supervisor
   error: Bad barcode 8901000000018: check digit wrong, rescan
   error: Bananas is sold by weight: put it on the scale

== 2. Total, then split payment: declined card, UPI, cash with change ==
   amount due Rs 588.23
   error: Cannot scan when the sale is TENDERING
   error: Card **0005 declined by issuer
   error: Card/UPI cannot be more than the amount due Rs 588.23
   UPI 400 -> receipt? null
  ---------------- RECEIPT R00001 (LANE-1) ----------------
   1 Bananas            1.245 kg @ Rs 60.00/kg       Rs 74.70
   2 Tomatoes           0.500 kg @ Rs 40.00/kg       Rs 20.00
   3 Milk 1L            1 x Rs 68.00                 Rs 68.00
   4 Milk 1L            1 x Rs 68.00                 Rs 68.00  VOID
   5 Paneer 200g        1 x Rs 90.00                 Rs 90.00
   6 Chips 50g          3 x Rs 20.00                 Rs 60.00
   7 Biscuits           4 x Rs 30.00                Rs 120.00
   8 Detergent 1kg      1 x Rs 245.00               Rs 245.00
  Gross                                            Rs 677.70
    10% off PRODUCE: Bananas                        -Rs 7.47
    10% off PRODUCE: Tomatoes                       -Rs 2.00
    Buy 2 get 1 free: Chips 50g                    -Rs 20.00
    3 for Rs 80.00: Biscuits                       -Rs 10.00
    Coupon SAVE50                                  -Rs 50.00
  TOTAL                                            Rs 588.23
    incl. tax EXEMPT @0%                            Rs 0.00
    incl. tax REDUCED @5%                           Rs 8.78
    incl. tax STANDARD @18%                        Rs 40.07
  Paid UPI asha@okbank                             Rs 400.00
  Paid cash                                        Rs 500.00
  Change                                           Rs 311.77

== 3. Returns refund what was really paid ==
   return 1 of 3 chips -> refund Rs 12.29
   return 2 more chips -> refund Rs 24.58  (all three add up to the Rs 36.87 paid)
   error: Only 0 of CHP left to return on R00001
   error: Only 1 of PNR left to return on R00001

== 4. Price change during a sale; going back to scanning; cancelling ==
   2 milk scanned around a reprice: gross Rs 140.00 = Rs 68 (old price kept) + Rs 72 (new)
   after adding paneer, due Rs 230.00
   error: Payments already taken; cancel them first
   gateway: reversed Rs 100.00 on AUTH1002
   error: Cannot pay when the sale is CANCELLED

== 5. Same basket, US-style tax added at the till ==
   inclusive total Rs 588.23 vs exclusive total Rs 644.73 (tax {EXEMPT=0.00, REDUCED=9.22, STANDARD=47.28})

== 6. Four registers sell 30 paneer each at once; stock was 100 ==
   paneer on hand: -21  (sold 120 + 1 earlier; the till never refuses a real item)
   alert: PNR LOW - reorder (on hand 20)
   alert: PNR NEGATIVE - recount the shelf (on hand -1)
```

Reading the output: the misread barcode (wrong check digit), an unknown code and scanning bananas without the scale are all refused. On the receipt, chips qualify for both "buy 2 get 1 free" (saves ₹20) and "3 for ₹50" (saves ₹10); only the better one is applied. The ₹50 coupon applied because the basket was still over ₹500 after item offers. The total ₹588.23 contains ₹8.78 + ₹40.07 of tax; the same basket under a US-style policy costs ₹644.73. The declined card and the over-large UPI amount changed nothing; UPI ₹400 plus cash ₹500 gave ₹311.77 change. Returning the chips one, then two, refunded ₹12.29 + ₹24.58 = ₹36.87 — exactly the net paid after the offer and the coupon share, not the shelf price of ₹60. The re-priced milk shows that a line keeps the price it was scanned at. Four registers selling 30 paneer each drove stock from 99 to −21 without refusing anyone, and the low and negative alerts each fired once.

Concurrency & edge cases

Case	Handling
Four registers sell the same product at once	`Inventory.adjust` uses `ConcurrentHashMap.merge` (atomic per key); no lost updates
Stock count says zero but the item is in hand	Sale goes through; stock goes negative; one "recount" alert
Alert spam when stock is low	Edge-triggered: alert only on the change that crosses the line
Two service desks refund the same receipt line at once	`ReceiptStore.claimReturn` uses `compute`: check-and-add is atomic; the second over-claim is refused
Price changes during a sale	Lines keep the price copied at scan; the frozen breakdown fixes the amount due
Offer changes during payment	Breakdown frozen at `total()`; going back to scanning re-prices
Card declined	Exception from the gateway before anything is recorded
Cancel after a card payment	`cancel` reverses every non-cash payment through the gateway
Rounding pennies in coupon allocation and refunds	Remainder to the largest SKU; refunds as differences of cumulative shares
One register used by one cashier	`Register` and `Sale` are single-threaded by design; shared objects (`Catalog`, `Inventory`, `ReceiptStore`) are the thread-safe ones

Real-world example

Supermarket tills identify loose produce by **PLU codes** — 4011 is the standard code for yellow bananas, and tomatoes have their own codes in the 4000 range — and read the weight from an integrated scale, exactly like `weigh(plu, grams)`. Packaged goods carry EAN-13 barcodes whose last digit is a check digit, which is why a smudged barcode is rejected rather than misread as another product. Receipts in tax-inclusive countries (for example GST — Goods and Services Tax — invoices in India or VAT — value-added tax — receipts in the UK) print how much tax the total *contains*, while US receipts add sales tax below the subtotal — our two `TaxPolicy` strategies. Retailers also run periodic "cycle counts" to fix the kind of negative stock our `Inventory` reports.

Extensions & follow-up questions

- **"Loyalty points and member prices."** A `CustomerContext` passed to the engine; member-only offers check it. Points are earned on the *net* paid (after discounts), from the receipt.
- **"Weighed items with a pre-packed label."** In-store printed barcodes that encode the price or weight in the digits (a variable-measure barcode); the scanner decodes them into a `LineItem` directly.
- **"Offers that span products (buy pasta + sauce, save ₹20)."** A `BundlePromotion` that looks at several SKUs; the "one offer per unit" rule then needs a small optimiser (try the combinations, keep the best) instead of best-per-SKU.
- **"Refund to the original payment method."** Store tender splits on the receipt; refund card/UPI parts through the gateway first, then cash.
- **"The network is down."** Offline mode: queue card authorisations (with a floor limit — a maximum amount accepted without online approval), keep selling, reconcile (match up the records) later; the catalog and offers are cached locally in each register.
- **"End-of-day."** A `Shift` per cashier: opening float (the cash put in the drawer at the start), cash counted at close, expected cash = float + cash sales − change − cash refunds.
- **"Self-checkout."** The same `Register` facade with a weight-check step at the bagging area (weight mismatch → assistance needed).

In the interview

Plan for 45 minutes: clarify offers, tax, tenders, returns (5) → entities and the Register facade (8) → sale state machine (5) → **pricing engine** with the stacking rules (12) → split payment and receipt (7) → returns, stock and concurrency (8).

> **30-second answer:** "A Register facade drives one Sale through OPEN, TENDERING, COMPLETED or CANCELLED. Lines copy the price at scan time; produce is priced by weight from the scale. A pure pricing engine groups lines

by SKU, gives each SKU its single best offer, applies a basket coupon, spreads it over SKUs so tax and refunds are exact, and computes tax through a pluggable inclusive or exclusive policy. The price is frozen at total; tenders are a sealed type, cash can give change, card and UPI go through a gateway. On completion an immutable receipt is stored and shared stock is reduced atomically, with edge-triggered alerts — and the till never refuses a real item."

Follow-ups with answers

- *Why not apply all offers that match?* Stacking can give away more than intended or even a negative price; the business has to choose a rule, and "best single offer per product, then one basket coupon" is simple and explainable.
- *Why spread the coupon over items?* Tax differs per category and returns must refund the price actually paid for that item.
- *Why is the pricing engine stateless?* So every register can share it and the same basket always gives the same answer — also what makes returns reproducible.

Self-check

1. **Q:** Three chips qualify for "buy 2 get 1 free" and "3 for ₹50". What is applied? **A:** Only the larger discount (₹20), because each SKU gets its single best item offer.
2. **Q:** Why does a return of one chip refund ₹12.29, not ₹20? **A:** The refund is the net price paid — after the offer and the item's share of the coupon.
3. **Q:** What stops two service desks refunding the same unit? **A:** `ConcurrentHashMap.compute` makes "check already returned, add" one atomic step.
4. **Q:** Why is stock allowed to go negative? **A:** The physical item is in the customer's hand; the count is wrong, so we sell and raise a recount alert.
5. **Q:** When are prices frozen, and why? **A:** At `total()`, so price or offer changes during payment cannot change the amount due.
6. **Q:** Which patterns appear? **A:** Strategy (offers, tax, payment gateway), Observer (stock alerts), Facade (Register), enum state machine (sale), sealed types (tenders).

TL;DR

- **Register facade** → one **Sale** state machine; lines copy prices; produce priced by **weight** (PLU + scale).
- **Pure pricing engine**: best item offer per SKU → basket coupon → spread over SKUs → tax per category via a **TaxPolicy** strategy.
- **Freeze** prices at total; **split tender** with a sealed `Tender`; declined cards change nothing; cancel reverses non-cash payments.
- Returns refund the **net paid**, claimed atomically; stock updates are atomic, may go negative, alerts are **edge-triggered**.

Chapter summary

Problem	Hard part	The move that solves it
Amazon	Guests vs members; last-item races; cancel rules	Ordering only on `Member` (LSP); atomic all-or-nothing stock reservation; order state machine
Brokerage	Four order types; tax lots; over-spending	Two abstract questions on `Order`; `StockLot` + FIFO `LotSale`; reserved funds
Stock exchange	Fair matching	Per-symbol `TreeMap<price, FIFO queue>`; price then time; one lock per symbol
Auction	Concurrent bids	All bid rules in one per-listing critical section; notify after unlocking
Music streaming	Shuffle/repeat without losing order	Play order as a strategy-produced permutation of indexes; plan rules in the enum
Food delivery / FoodKart	Capacity and riders under concurrency	Reserve kitchen slots atomically; CAS-claim riders; strategies for assignment and selection
Grocery POS	Weighed items, stacking offers, split tender, returns	Pure pricing engine (best offer per SKU, coupon spread, tax per category); frozen price; atomic return claims; edge-triggered stock alerts

Patterns that kept coming back: Strategy (payments, play order, rider assignment, restaurant selection, trade-price policy, grocery offers and tax), Observer (notifications, fills, plays), state machines in enums (orders, auctions, player), Template Method (statements), Facade (every `*Service`), and one concurrency rule — **make check-then-act atomic** (lock, or compare-and-set).

Sources combined in this chapter: the DesignGurus *Grokking the Object-Oriented Design Interview* lessons on Amazon and the stock brokerage (requirements, actors, staged class design and diagrams); the awesome-low-level-design problem statements for the auction system, music streaming service and food delivery service; and the kumaransg/LLD machine-coding statements for the Navi stock exchange and Flipkart's FoodKart. All Java code is original. The grocery point-of-sale case is designed from first principles, using public retail conventions (GS1 EAN-13 check digits, IFPS PLU codes for produce, tax-inclusive vs sales-tax receipts).

Chapter 20 — Case Studies: Pattern-Driven Components

> **Goal.** The problems in this chapter are not whole products. They are **building blocks** that almost every real system contains: a ledger that splits money, a service that sends notifications, a cache, a matcher that pairs riders with drivers, a logger, a message broker and a rate limiter. Each one is small enough to finish in an interview, and each one is **anchored on one or two design patterns**. If you can name the pattern and say what change it absorbs, you have already answered half of the question.

Why "components" deserve their own chapter. A component is a part that other parts of a system *use*. It has a small public surface (a few methods), and a lot of hidden rules behind it. That makes it the perfect interview question: the interviewer can keep asking "what if this rule changes?" and a good design answers with "one new class, nothing else touched".

#	Component	Anchor pattern(s)	The change the pattern absorbs
20.1	Splitwise (expense sharing)	**Strategy** (split rules)	A new way to split a bill
20.2	Notification service	**Template Method** (retry loop), **Strategy** (send rules), **Observer** (outcomes), Simple Factory (channels)	A new channel, a new reason not to send, a new thing that watches results
20.3	In-memory cache (+ LRU from scratch)	**Strategy** (eviction policy), **Observer** (removal listener), Builder	A new eviction rule
20.4	Ride sharing (Uber)	**Strategy** (matching, pricing, payment), **Decorator** (surge), State machine (trip)	A new matching rule, surge on/off, a new payment method
20.5	Logging framework	**Chain of Responsibility** (logger hierarchy), **Singleton** (LogManager), Strategy (formatter), Decorator (async appender)	A new sink, a new format, async writing
20.6	Pub-Sub system	**Observer / Publish-Subscribe**, append-only log with offsets	New subscribers, replay, slow consumers
20.7	Rate limiter	**Strategy** (algorithm), Simple Factory, Composite	A new algorithm, a new tier, combined limits

> **Key terms — the patterns named above.** A **design pattern** is a named, reusable shape for solving a design problem that keeps coming back. In one line each: **Strategy** puts each interchangeable rule in its own class behind one interface (a list of methods with no code, which several classes implement in their own way); **Template Method** writes the fixed steps of an algorithm once in a parent class and lets subclasses fill in one or two steps; **Observer** lets objects sign up to be told when something happens; **Simple Factory** is one place that decides which class to create; **Builder** assembles an object with many optional settings step by step; **Decorator** wraps an object in another object of the same type to add behaviour; a **state machine** is a fixed set of states plus the allowed moves between them; **Chain of Responsibility** passes a request along a line of handlers; **Singleton** guarantees exactly one instance of a class; **Composite** lets a group of objects be used as if it were one; **Publish-Subscribe** sends messages through a middleman so senders and receivers never know each other. An **append-only log with offsets** is a list that is only ever added to at the end, where each entry is found by its position number (its offset).

How every case is laid out (the case-study template used in Chapters 14–21): *Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns used → Java implementation (+ real output) → Concurrency & edge cases → Extensions & follow-up questions → Real-world example → In the interview → Self-check → TL;DR.* (*Concurrency* means several threads running at the same time, which is where most subtle bugs hide.)

Running the code. All code is Java 21, standard library only (only the classes that ship with Java itself, no extra downloads), under `code/lld/ch20/<name>/`. Every package (Java's folder-like grouping of related classes) has a `Main` (a class with the `main` method the program starts from). From `chapters/20-case-components/code`:

```
javac --release 21 -d /tmp/lld_20 $(find . -name '*.java')
java -cp /tmp/lld_20 lld.ch20.splitwise.Main      # also: notification, cache, lrucache,
                                                  # ridesharing, carpool, logging, pubsub, ratelimiter
```

Money is always `BigDecimal`. A `double` (Java's standard binary floating-point number type, fast but approximate for decimals) cannot hold 0.1 exactly, so sums drift (`0.1 + 0.2` prints `0.30000000000000004`). `BigDecimal` stores decimal digits exactly. Two rules to remember: compare amounts with `compareTo` (not `equals`, because `equals` also compares the scale (the number of digits after the decimal point), so `2.0` and `2.00` are "different"), and always say how to round when you divide (`divide(x, 2, RoundingMode.HALF_EVEN)`), or Java throws an `ArithmeticException` for results like 1/3. (An **exception** is Java's way of signalling an error: normal execution stops and the error travels up to whoever catches it.)

Reading the course diagrams. The figures marked "(course diagram)" are UML diagrams (UML, the Unified Modeling Language, is the standard box-and-arrow notation for drawing classes and how they relate) and use one arrow legend throughout:

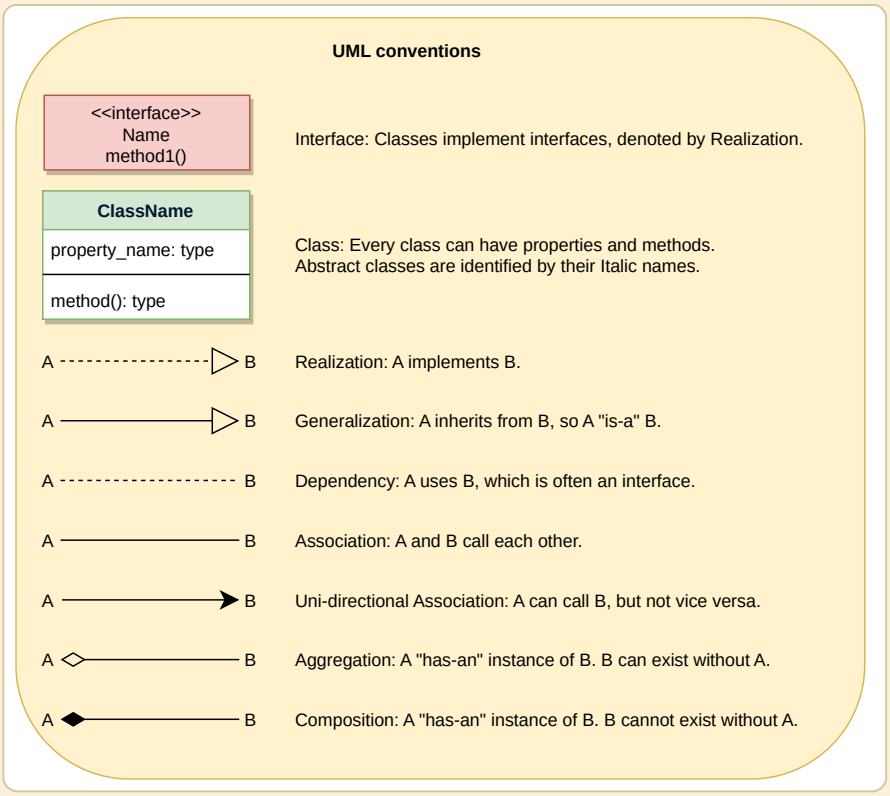


Figure — UML conventions used in the course class diagrams: interface, class, realization (implements), generalization (inherits), dependency, association, one-way association, aggregation and composition (course diagram)

The mermaid diagrams (Mermaid is a text format that a renderer turns into pictures) are drawn from **this chapter's Java code**, so they match the code line by line.

20.1 Design Splitwise

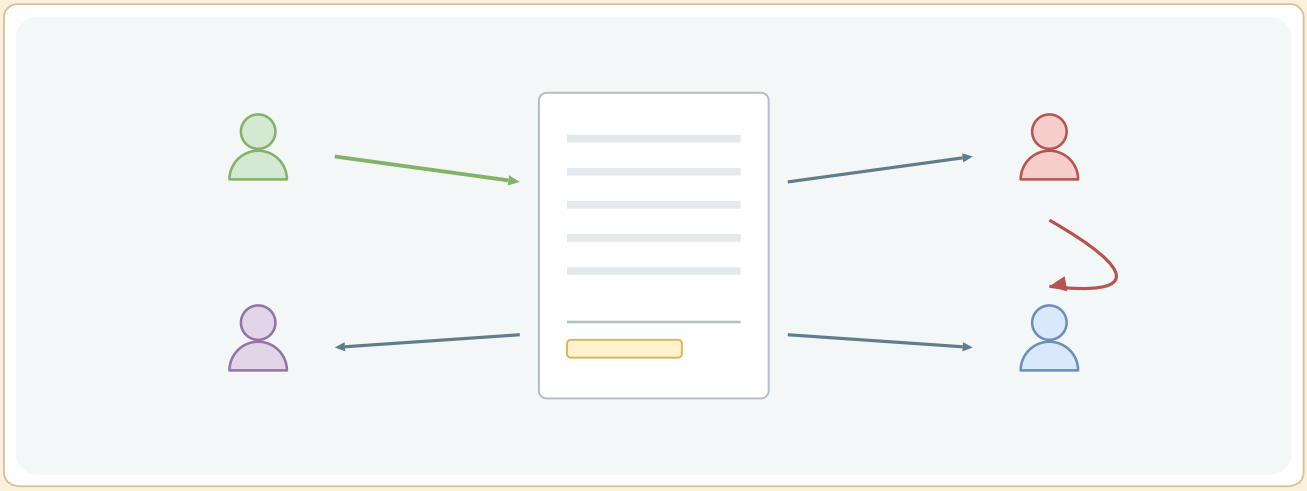


Figure — Splitwise in one picture: people send expenses into one shared record, and the record tells each person what they owe (course diagram)

Problem & clarifying questions

What it is. Splitwise is an app that remembers who paid for what when friends share costs, and tells everyone who owes whom. Everyday analogy: a weekend away with four friends. The hotel goes on Alice's card, Bob covers dinner, Carol gets the cab. By Sunday evening no one can reconstruct the totals. Splitwise is the shared notebook that does the maths.

The hard part is **not** the UI (user interface: the screens people tap). The real decision is *which class owns the money arithmetic*, so the books still balance after every single operation.

Questions to ask the interviewer (with the answers we assume):

Question	Assumed answer
Which split types?	Equal, exact amounts, percentage. More may come later.
Must shares add up to the total exactly?	Yes. If they don't, the whole expense is rejected and no balance changes.
Rounding?	Amounts are money with 2 decimal places. No cent may be lost.
Groups, or only one-to-one?	Both. A group is just a name plus a member list (a trip, a shared flat).
Can a person pay for only themselves?	No. That is not shared spending: at least two people (the payer counts) must be involved.
Settle up?	Yes: record a real payment that reduces a debt.
Simplify debts?	Yes: on request, replace many small debts with a shorter list of transfers.
Multiple currencies, receipts, comments, undo?	Out of scope (say so explicitly).

Requirements

Functional

1. Every user is identified by a name and an email.
2. Users form groups; a group has a name and members.
3. An expense records a description, total, who paid and who shares it.
4. Split types: equal, exact, percent.
5. Shares must add up to the total; otherwise the expense is refused and **nothing changes**.
6. At least two distinct people (the payer plus the sharers) must be involved in an expense.
7. Balances between every pair of people are tracked continuously.
8. A user can query their balance with one specific person, and their overall net balance.
9. Settle up: a real payment reduces a debt.
10. A group can request a simplified settlement plan with fewer transfers.

11. Rounding never loses or creates a cent.
12. Expenses are stored permanently, so a group's history can be listed.

Non-functional

- Correctness of money above everything (exact decimals, no drift).
- Adding a new split type must not change existing classes (Open/Closed Principle).
- Thread-safe: two people adding expenses at the same time must not corrupt the ledger. (A **thread** is one independent line of execution inside a running program; code is **thread-safe** when several threads can use it at the same moment without corrupting its data.)

Use cases

Actors:

- **Member** — adds expenses, settles up, asks what they owe.
- **Group creator** — sets up a group and invites its members.
- **System** — works out each share, checks shares add up, updates the ledger, simplifies debts.

Main use cases: *Add an expense* (which includes *Work out each share*, *Check the shares add up*, *Update the ledger*), *Settle up*, *See what I owe*, *Create a group / add people*, *Simplify the debts*.

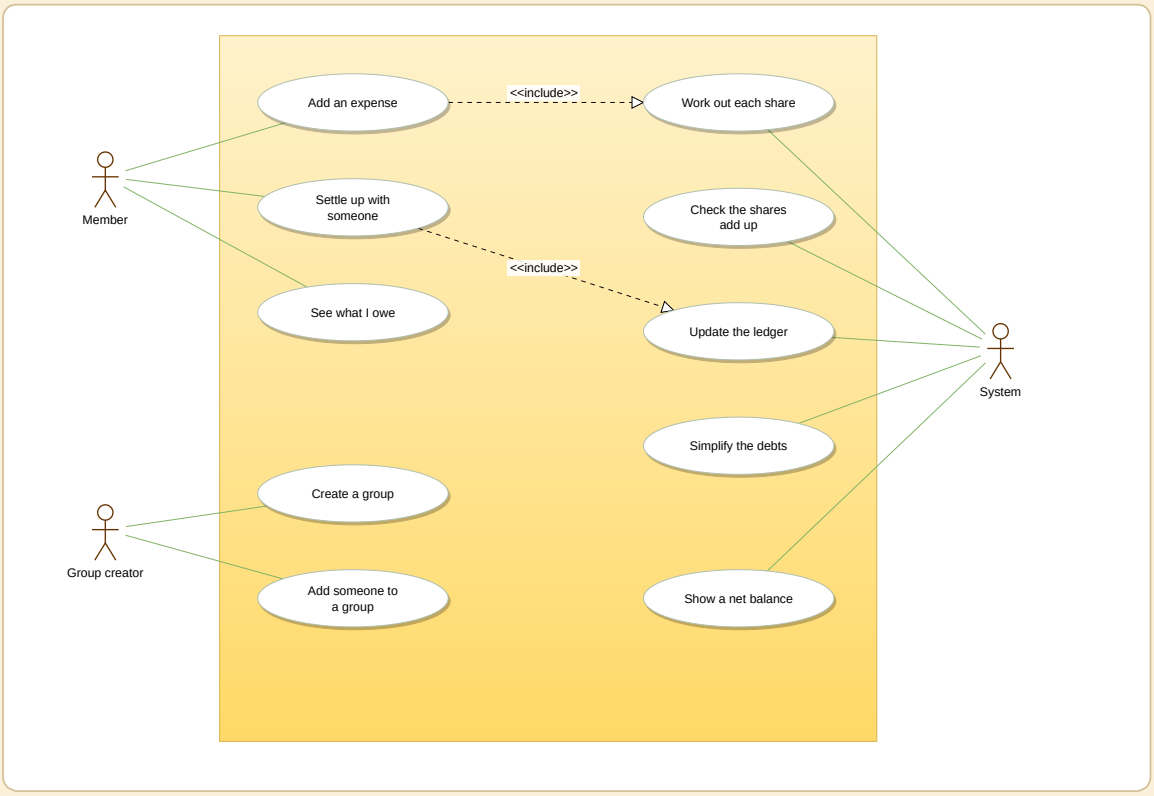


Figure — Splitwise use cases: members add expenses and settle up; the system computes shares, validates them, updates the ledger and simplifies debts (course diagram)

In the figure above, "Add an expense" *includes* the three system steps. "Include" in a use-case diagram means the step always happens as part of the bigger use case.

Core entities

Class	Responsibility
`User` (record)	Identity: id, name, email. No money.
`Group`	A named set of member ids + the list of its expense ids. **Holds no balances.**
`SplitStrategy` (interface)	Turns a total into one `Share` per person.
`EqualSplit`, `ExactSplit`, `PercentSplit`	The three rules.
`Share` (record)	A (person, amount) pair for a single expense (a value object).
`Expense`	Description, total, payer, shares. Validates itself once, when created.
`Ledger`	Who owes whom: exactly **one signed number per pair** of people. One mutator: `adjust`.
`DebtSimplifier`	Greedy minimum-cash-flow: turns net positions into a short list of payments.
`Payment` (record)	One suggested transfer (a suggestion, not a record of money moving).
`Splitwise`	Facade: the one class the app talks to. Orders the steps, contains no arithmetic.
`Money`	Tiny helper: 2-decimal `BigDecimal` values.

A few words from the table: a **record** is Java's short syntax for a class that only carries data, whose fields cannot change after creation; a **value object** is a small object defined only by its values (two shares with the same person and amount are interchangeable); a **mutator** is a method that changes an object's state; a **Facade** is one simple front class that hides several classes behind it; and a **greedy** algorithm makes the best-looking choice at each step without reconsidering earlier choices.

Why the group holds no money. If each group kept its own balances, a person in two groups would have "what Bob owes Alice" stored in two places. Sooner or later the two copies disagree. One ledger, one number per pair, is the single source of truth.

Why `Group → User` is association, not composition. Association = one object refers to another that lives independently. Composition = the part is owned and dies with the whole. Deleting a group must not delete its people, so it is association.

Class diagram

The course builds the model in five stages, and the order is worth copying in an interview because every stage leaves a slice that already works (run out of time and you still have something that runs, not half a diagram):

Stage	Adds	Works afterwards
1	`User`, `Group`	You can say who shares costs
2	`SplitStrategy` + three rules, `Share`	A total becomes one amount per person
3	`Expense` (self-checking)	A bill that does not add up is rejected
4	`Ledger`	Pairwise balances are tracked
5	`Splitwise` facade + `DebtSimplifier`	A whole trip can be recorded, settled and simplified

Here is the finished course diagram:

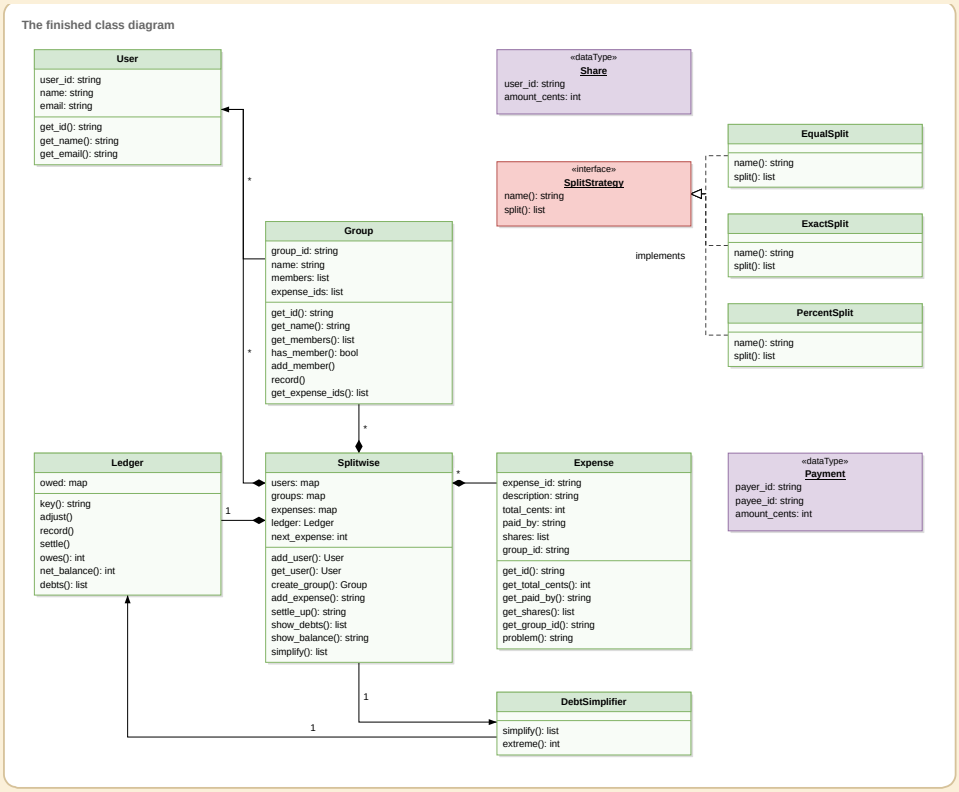
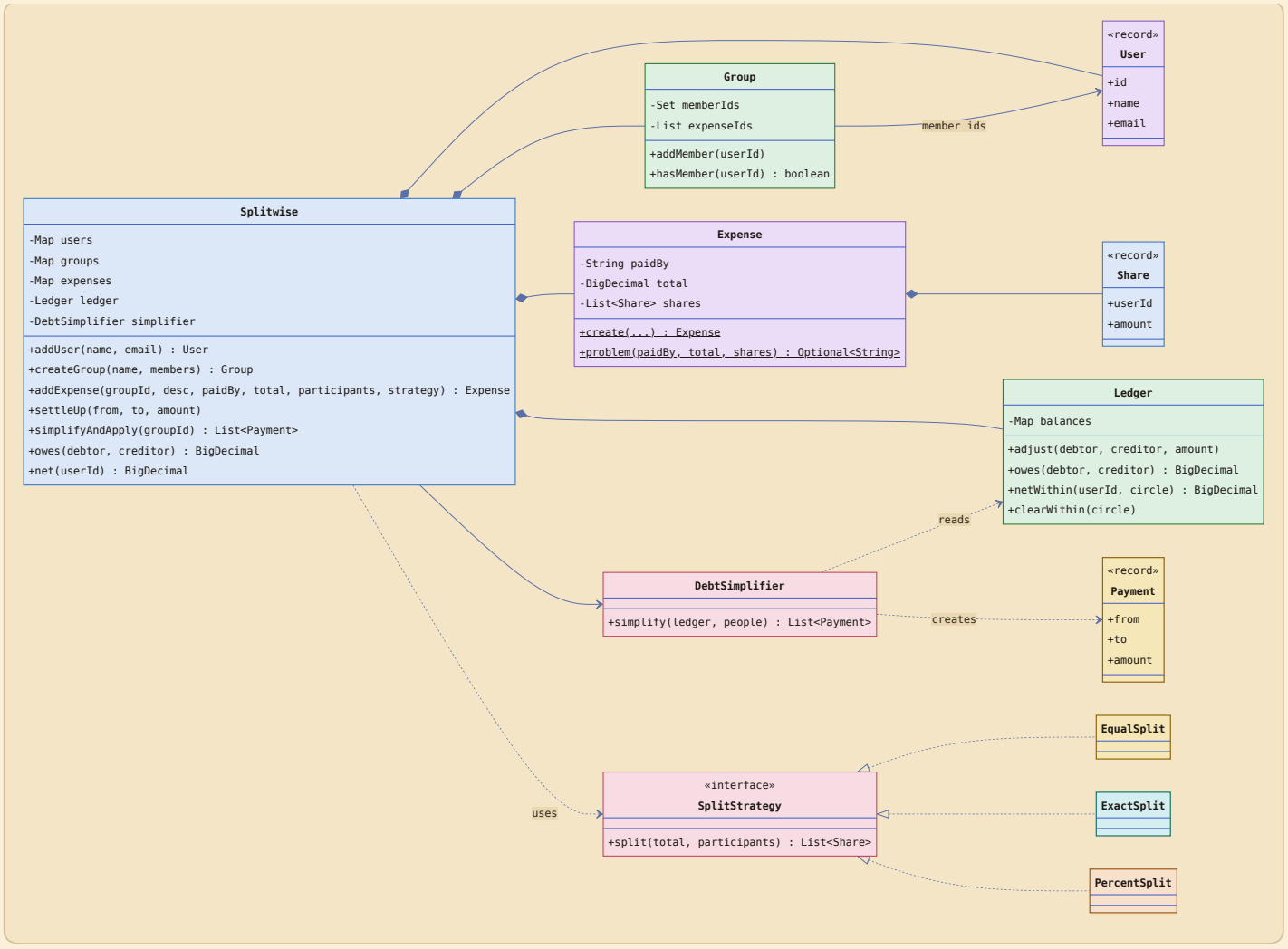


Figure — The finished Splitwise class diagram: service owning users, groups, expenses and the ledger, with the split strategies and the simplifier (course diagram)

In the figure above, the three split rules meet in one hollow triangle: they all realize the same interface. The service's filled diamonds are composition: it owns users, groups, expenses and the ledger.

Our Java code, as a class diagram:



Key flows

Adding an expense. Nothing touches the ledger until both checks pass.

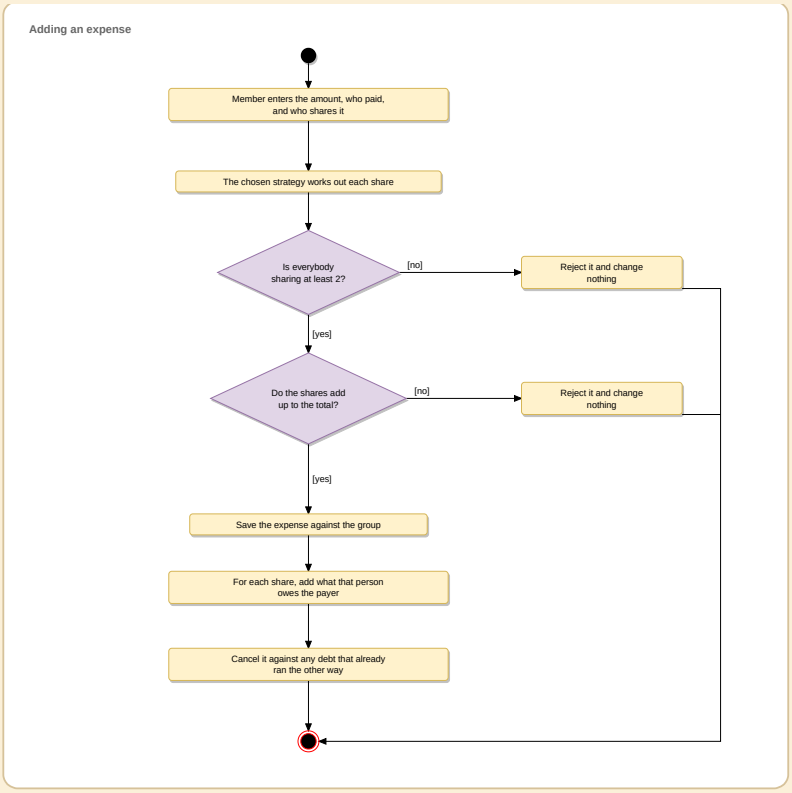
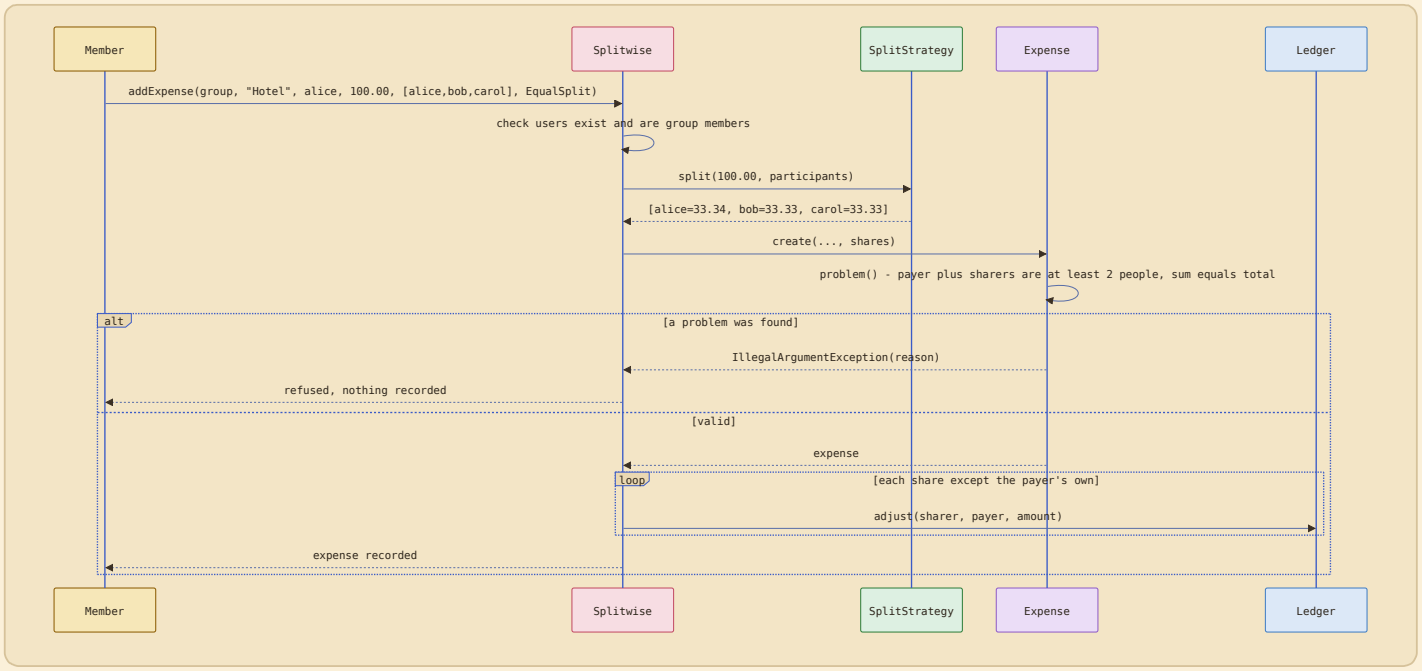


Figure — Adding an expense: the strategy computes shares, the expense checks itself, and only then is the ledger updated (course diagram)



Simplifying a group's debts (minimum cash flow, greedy).

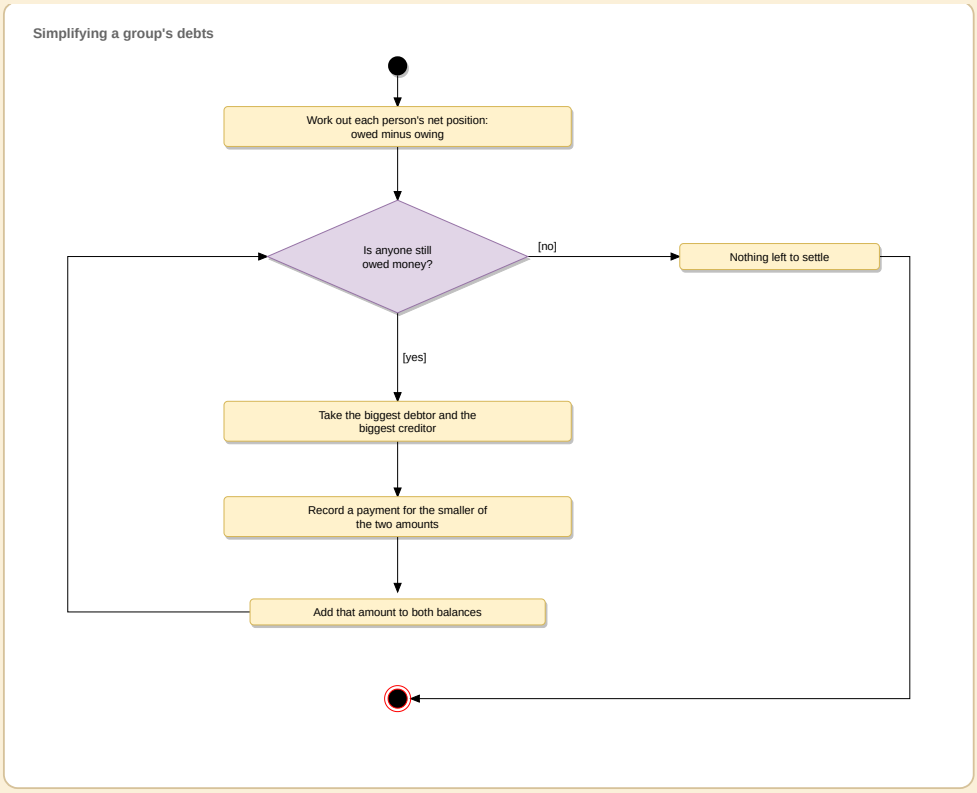


Figure — Simplifying debts: compute net positions, then the largest debtor repeatedly pays the largest creditor until every balance is zero (course diagram)

How it works, step by step:

1. For each member, compute the **net position** = (what others owe them) − (what they owe others). Positive = creditor, negative = debtor. The nets always sum to zero.
2. Put creditors in one max-heap and debtors in another (by amount; ties by name, so the output is deterministic). A **max-heap** is a priority queue that always hands back its largest item first; **deterministic** means the same input always gives the same output.
3. Pop the biggest creditor C and biggest debtor D. Let x = min(C, D). Suggest "D pays C x".
4. Push back whichever side still has a remainder.
5. Stop when a heap is empty (everyone is at zero).

Each round brings at least one person to zero, and the final round brings two to zero, so there are **at most n − 1 payments** for n people. It is **not always the true minimum**: finding the minimum number of transfers is NP-hard (no fast algorithm for it is known: the work needed explodes as the number of people grows; it is related to splitting people into as many zero-sum subgroups as possible). Say that before you are asked.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for split rules (`SplitStrategy`)	The rule arrives from outside and is interchangeable. A fourth rule (by shares, by adjustment) is a new class. Every rule returns the same `List<Share>`, so nothing downstream knows which rule ran.	`if (type == EQUAL) ... else if (type == EXACT)` in the service: every new type edits old, tested code.
Expense validates itself in a **static factory** (a static method that builds the object instead of `new`; here `Expense.create`)	Checked once, when created; after that every class can trust its shares. An invalid expense can never exist.	Checking in the ledger: re-checks on every read and still lets bad objects exist.
`problem()` returns the **reason**, not a boolean	The user sees *why* ("shares add up to 15.00 but the total is 30.00").	`boolean isValid()`: loses the reason.
Ledger stores **one signed number per unordered pair**	A debt is never stored twice, so it can never disagree with itself. Money moving the other way cancels automatically.	Two maps "A owes B" and "B owes A": two places for the truth.
Ledger has **one mutator** (`adjust`)	Expenses, settle-ups and simplification all go through it, so the cancel rule exists in exactly one place.	Separate `addDebt`, `settle`, `reduce` methods each with their own arithmetic.
`DebtSimplifier` separate from `Ledger`	Single Responsibility (each class has one reason to change): *keeping* balances and *planning* settlements change for different reasons.	Putting the algorithm inside the ledger.
`BigDecimal` with explicit rounding; spare cents handed out one at a time	100.00 / 3 = 33.33 × 3 = 99.99 would lose a cent. Equal split gives the extra cent to the first people; percent split lets the last person absorb the remainder.	`double`: 0.1 + 0.2 ≠ 0.3. (The course keeps whole cents in a `long`; that is equally correct. We use `BigDecimal` as this book's standard.)
`Splitwise` is a **Facade**	One entry point; it orders the steps and decides nothing.	UI code calling ledger and strategies directly.

Only one named GoF pattern here (Strategy). (GoF, the "Gang of Four", are the four authors of the 1994 book that catalogued 23 classic design patterns.) That is fine. Good designs do not collect patterns; they use the one that absorbs the change that is likely.

Java implementation

```java
// File: code/lld/ch20/splitwise/Model.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Objects;
import java.util.Set;

/** Money helpers: every amount is a BigDecimal with exactly 2 decimal places. */
final class Money {
    static final BigDecimal ZERO = new BigDecimal("0.00");
    static final BigDecimal CENT = new BigDecimal("0.01");
    static final BigDecimal HUNDRED = new BigDecimal("100");

    private Money() {}

    static BigDecimal of(String amount) {
        return new BigDecimal(amount).setScale(2, RoundingMode.UNNECESSARY); // refuses 10.005
    }

    static BigDecimal normalize(BigDecimal amount) {
        return amount.setScale(2, RoundingMode.HALF_EVEN);
    }
}

/** A person. Identity only - no money lives here. */
record User(String id, String name, String email) {
    User {
        Objects.requireNonNull(id);
        Objects.requireNonNull(name);
    }
}

/** A named set of people. Holds member ids and expense ids, never balances. */
final class Group {
    private final String id;
    private final String name;
    private final Set<String> memberIds = new LinkedHashSet<>();
    private final List<String> expenseIds = new ArrayList<>();

    Group(String id, String name) {
        this.id = id;
        this.name = name;
    }

    String id() { return id; }
    String name() { return name; }
    void addMember(String userId) { memberIds.add(userId); }
    boolean hasMember(String userId) { return memberIds.contains(userId); }
    Set<String> members() { return Collections.unmodifiableSet(memberIds); }
    void recordExpense(String expenseId) { expenseIds.add(expenseId); }
    List<String> expenseIds() { return Collections.unmodifiableList(expenseIds); }
}

/** One person's part of one expense (a value object). */
record Share(String userId, BigDecimal amount) {
    @Override
    public String toString() { return userId + "=" + amount; }
}

/** A suggested transfer produced by the simplifier - a suggestion, not a record. */
record Payment(String from, String to, BigDecimal amount) {
```

```
    @Override
    public String toString() {
        return from + " pays " + to + " " + amount;
    }
}
```

```java
// File: code/lld/ch20/splitwise/SplitStrategy.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Map;

/** Strategy: turns one total into one Share per participant. */
interface SplitStrategy {
    List<Share> split(BigDecimal total, List<String> participants);
}

/** Equal split. Spare cents are handed out one at a time, in participant order. */
final class EqualSplit implements SplitStrategy {
    @Override
    public List<Share> split(BigDecimal total, List<String> participants) {
        if (participants.isEmpty()) throw new IllegalArgumentException("nobody to split between");
        BigDecimal n = BigDecimal.valueOf(participants.size());
        BigDecimal base = total.divide(n, 2, RoundingMode.DOWN);           // 100.00/3 -> 33.33
        BigDecimal leftover = total.subtract(base.multiply(n));           // 0.01
        int spareCents = leftover.divide(Money.CENT).intValueExact();     // 1
        List<Share> shares = new ArrayList<>();
        for (int i = 0; i < participants.size(); i++) {
            BigDecimal amount = i < spareCents ? base.add(Money.CENT) : base;
            shares.add(new Share(participants.get(i), amount));
        }
        return shares;
    }
}

/** Exact split: the caller states each person's amount. Expense checks the sum. */
final class ExactSplit implements SplitStrategy {
    private final Map<String, BigDecimal> amounts;

    ExactSplit(Map<String, BigDecimal> amounts) { this.amounts = Map.copyOf(amounts); }

    @Override
    public List<Share> split(BigDecimal total, List<String> participants) {
        if (!new HashSet<>(participants).equals(amounts.keySet()))
            throw new IllegalArgumentException("exact amounts must name exactly the participants");
        List<Share> shares = new ArrayList<>();
        for (String id : participants) shares.add(new Share(id, Money.normalize(amounts.get(id))));
        return shares;
    }
}

/** Percent split. Percentages must add to 100; the last person absorbs rounding. */
final class PercentSplit implements SplitStrategy {
    private final Map<String, BigDecimal> percents;

    PercentSplit(Map<String, BigDecimal> percents) { this.percents = Map.copyOf(percents); }

    @Override
    public List<Share> split(BigDecimal total, List<String> participants) {
        if (!new HashSet<>(participants).equals(percents.keySet()))
            throw new IllegalArgumentException("percentages must name exactly the participants");
        BigDecimal sum = percents.values().stream().reduce(BigDecimal.ZERO, BigDecimal::add);
        if (sum.compareTo(Money.HUNDRED) != 0)
            throw new IllegalArgumentException("percentages add up to " + sum + ", not 100");
        List<Share> shares = new ArrayList<>();
        BigDecimal given = Money.ZERO;
        for (int i = 0; i < participants.size(); i++) {
            String id = participants.get(i);
            BigDecimal amount = (i == participants.size() - 1)
                    ? total.subtract(given)                                  // absorbs the remainder
```

```
                    : total.multiply(percents.get(id)).divide(Money.HUNDRED, 2, RoundingMode.HALF_EVEN);
            given = given.add(amount);
            shares.add(new Share(id, amount));
        }
        return shares;
    }
}
```

```java
// File: code/lld/ch20/splitwise/Expense.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.util.HashSet;
import java.util.List;
import java.util.Optional;
import java.util.Set;

/** What was bought, what it cost, who paid, and each share. Validated once, at creation. */
final class Expense {
    private final String id;
    private final String description;
    private final String paidBy;
    private final BigDecimal total;
    private final List<Share> shares;
    private final String groupId; // null for an expense between friends outside any group

    private Expense(String id, String description, String paidBy, BigDecimal total,
                    List<Share> shares, String groupId) {
        this.id = id;
        this.description = description;
        this.paidBy = paidBy;
        this.total = total;
        this.shares = List.copyOf(shares);
        this.groupId = groupId;
    }

    /** Factory: builds a valid expense or throws with the reason it is invalid. */
    static Expense create(String id, String description, String paidBy, BigDecimal total,
                          List<Share> shares, String groupId) {
        problem(paidBy, total, shares).ifPresent(p -> { throw new IllegalArgumentException(p); });
        return new Expense(id, description, paidBy, total, shares, groupId);
    }

    /** Why these shares cannot form an expense, or empty if they can. */
    static Optional<String> problem(String paidBy, BigDecimal total, List<Share> shares) {
        if (total.signum() <= 0) return Optional.of("total must be positive");
        Set<String> sharers = new HashSet<>();
        for (Share s : shares) {
            if (s.amount().signum() < 0) return Optional.of("negative share for " + s.userId());
            if (!sharers.add(s.userId())) return Optional.of("duplicate share for " + s.userId());
        }
        Set<String> people = new HashSet<>(sharers);
        people.add(paidBy); // the payer counts as one of the two people even if they owe no share
        if (people.size() < 2) return Optional.of("an expense needs at least two people");
        BigDecimal sum = shares.stream().map(Share::amount).reduce(BigDecimal.ZERO, BigDecimal::add);
        if (sum.compareTo(total) != 0)
            return Optional.of("shares add up to " + sum + " but the total is " + total);
        return Optional.empty();
    }

    String id() { return id; }
    String description() { return description; }
    String paidBy() { return paidBy; }
    BigDecimal total() { return total; }
    List<Share> shares() { return shares; }
    String groupId() { return groupId; }
}
```

```java
// File: code/lld/ch20/splitwise/Ledger.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.util.Collection;
import java.util.Map;
import java.util.TreeMap;

/**
 * Who owes whom. Exactly ONE signed number per unordered pair of people.
 * Key (lo, hi) with lo < hi alphabetically; a positive value means "hi owes lo".
 */
final class Ledger {
    private record Pair(String lo, String hi) {
        static Pair of(String a, String b) { return a.compareTo(b) < 0 ? new Pair(a, b) : new Pair(b, a); }
    }

    private final Map<Pair, BigDecimal> balances = new TreeMap<>(
            (p, q) -> p.lo().equals(q.lo()) ? p.hi().compareTo(q.hi()) : p.lo().compareTo(q.lo()));

    /** The ONLY mutator: 'debtor' now owes 'creditor' 'amount' more. Opposite debts cancel. */
    void adjust(String debtor, String creditor, BigDecimal amount) {
        if (debtor.equals(creditor) || amount.signum() == 0) return;
        Pair key = Pair.of(debtor, creditor);
        BigDecimal delta = creditor.equals(key.lo()) ? amount : amount.negate();
        BigDecimal updated = balances.getOrDefault(key, Money.ZERO).add(delta);
        if (updated.signum() == 0) balances.remove(key); else balances.put(key, updated);
    }

    /** Drops every debt where both people are in 'circle' (used when a simplified plan replaces them). */
    void clearWithin(Collection<String> circle) {
        balances.keySet().removeIf(p -> circle.contains(p.lo()) && circle.contains(p.hi()));
    }

    /** How much 'debtor' owes 'creditor' (never negative). */
    BigDecimal owes(String debtor, String creditor) {
        if (debtor.equals(creditor)) return Money.ZERO;
        Pair key = Pair.of(debtor, creditor);
        BigDecimal v = balances.getOrDefault(key, Money.ZERO);
        BigDecimal debtorOwes = creditor.equals(key.lo()) ? v : v.negate();
        return debtorOwes.signum() > 0 ? debtorOwes : Money.ZERO;
    }

    /** Positive = others owe this person; negative = this person owes others. */
    BigDecimal net(String userId) { return netWithin(userId, null); }

    /** Net position counting only debts with people in 'circle' (null = everybody). */
    BigDecimal netWithin(String userId, Collection<String> circle) {
        BigDecimal net = Money.ZERO;
        for (var e : balances.entrySet()) {
            Pair p = e.getKey();
            String other = p.lo().equals(userId) ? p.hi() : p.hi().equals(userId) ? p.lo() : null;
            if (other == null || (circle != null && !circle.contains(other))) continue;
            // positive value: hi owes lo, so lo is up and hi is down
            net = p.lo().equals(userId) ? net.add(e.getValue()) : net.subtract(e.getValue());
        }
        return net;
    }

    /** Human-readable list of every open debt: "B owes A 10.00". */
    void print() {
        if (balances.isEmpty()) System.out.println("  (everyone is settled up)");
        balances.forEach((p, v) -> {
            if (v.signum() > 0) System.out.println("  " + p.hi() + " owes " + p.lo() + " " + v);
            else System.out.println("  " + p.lo() + " owes " + p.hi() + " " + v.negate());
        });
```

```
    }
}
```

```java
// File: code/lld/ch20/splitwise/DebtSimplifier.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

/**
 * Greedy "minimum cash flow": largest debtor pays largest creditor, repeat.
 * Uses at most (people - 1) payments. Not always the true minimum (that problem is NP-hard).
 */
final class DebtSimplifier {
    private record Position(String userId, BigDecimal amount) {}

    List<Payment> simplify(Ledger ledger, Collection<String> people) {
        // Biggest amount first; ties broken by name so results are deterministic.
        Comparator<Position> biggestFirst = Comparator.comparing(Position::amount).reversed()
                .thenComparing(Position::userId);
        PriorityQueue<Position> creditors = new PriorityQueue<>(biggestFirst);
        PriorityQueue<Position> debtors = new PriorityQueue<>(biggestFirst);
        for (String id : people) {
            BigDecimal net = ledger.netWithin(id, people);
            if (net.signum() > 0) creditors.add(new Position(id, net));
            else if (net.signum() < 0) debtors.add(new Position(id, net.negate()));
        }
        List<Payment> payments = new ArrayList<>();
        while (!creditors.isEmpty() && !debtors.isEmpty()) {
            Position c = creditors.poll();
            Position d = debtors.poll();
            BigDecimal x = c.amount().min(d.amount());
            payments.add(new Payment(d.userId(), c.userId(), x));
            if (c.amount().compareTo(x) > 0) creditors.add(new Position(c.userId(), c.amount().subtract(x)));
            if (d.amount().compareTo(x) > 0) debtors.add(new Position(d.userId(), d.amount().subtract(x)));
        }
        return payments;
    }
}
```

```java
// File: code/lld/ch20/splitwise/Splitwise.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicInteger;

/** Facade the app talks to. Orders the steps; the arithmetic lives in the other classes. */
final class Splitwise {
    private final Map<String, User> users = new LinkedHashMap<>();
    private final Map<String, Group> groups = new LinkedHashMap<>();
    private final Map<String, Expense> expenses = new LinkedHashMap<>();
    private final Ledger ledger = new Ledger();
    private final DebtSimplifier simplifier = new DebtSimplifier();
    private final AtomicInteger ids = new AtomicInteger();

    synchronized User addUser(String name, String email) {
        User u = new User(name.toLowerCase(), name, email);
        users.put(u.id(), u);
        return u;
    }

    synchronized Group createGroup(String name, String... memberIds) {
        Group g = new Group("g" + ids.incrementAndGet(), name);
        for (String m : memberIds) g.addMember(requireUser(m));
        groups.put(g.id(), g);
        return g;
    }

    /** Build shares -> build (and validate) expense -> only then touch the ledger. */
    synchronized Expense addExpense(String groupId, String description, String paidBy, BigDecimal total,
                                    List<String> participants, SplitStrategy strategy) {
        requireUser(paidBy);
        participants.forEach(this::requireUser);
        Group group = groupId == null ? null : groups.get(groupId);
        if (group != null) {
            for (String p : participants)
                if (!group.hasMember(p)) throw new IllegalArgumentException(p + " is not in " + group.name());
            if (!group.hasMember(paidBy)) throw new IllegalArgumentException(paidBy + " is not in " +
group.name());
        }
        List<Share> shares = strategy.split(total, participants);       // may throw
        Expense e = Expense.create("e" + ids.incrementAndGet(), description, paidBy, total, shares, groupId);
        for (Share s : e.shares()) ledger.adjust(s.userId(), paidBy, s.amount()); // payer's own share is a no-op
        expenses.put(e.id(), e);
        if (group != null) group.recordExpense(e.id());
        return e;
    }

    /** A real payment from 'from' to 'to'. Cannot pay more than is owed. */
    synchronized void settleUp(String from, String to, BigDecimal amount) {
        BigDecimal owed = ledger.owes(from, to);
        if (amount.signum() <= 0 || amount.compareTo(owed) > 0)
            throw new IllegalArgumentException(from + " owes " + to + " only " + owed + ", cannot pay " + amount);
        ledger.adjust(to, from, amount); // the reverse debt cancels the existing one
    }

    synchronized BigDecimal owes(String debtor, String creditor) { return ledger.owes(debtor, creditor); }
    synchronized BigDecimal net(String userId) { return ledger.net(userId); }
    synchronized List<Payment> simplify(String groupId) {
        return simplifier.simplify(ledger, groups.get(groupId).members());
    }
    /** Replaces the group's tangle of debts with the simplified plan. Every net position is unchanged. */
    synchronized List<Payment> simplifyAndApply(String groupId) {
        var members = groups.get(groupId).members();
```

```
        List<Payment> plan = simplifier.simplify(ledger, members);
        ledger.clearWithin(members);
        for (Payment p : plan) ledger.adjust(p.from(), p.to(), p.amount());
        return plan;
    }

    synchronized void printLedger() { ledger.print(); }

    synchronized List<String> history(String groupId) {
        List<String> lines = new ArrayList<>();
        for (String id : groups.get(groupId).expenseIds()) {
            Expense e = expenses.get(id);
            lines.add(e.id() + " " + e.description() + " " + e.total() + " paid by " + e.paidBy() + " " +
e.shares());
        }
        return lines;
    }

    private String requireUser(String id) {
        if (!users.containsKey(id)) throw new IllegalArgumentException("unknown user " + id);
        return id;
    }
}
```

```java
// File: code/lld/ch20/splitwise/Main.java
package lld.ch20.splitwise;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        Splitwise app = new Splitwise();
        for (String n : List.of("Alice", "Bob", "Carol", "Dave")) app.addUser(n, n.toLowerCase() + "@mail.com");
        Group trip = app.createGroup("Goa trip", "alice", "bob", "carol", "dave");

        System.out.println("== 1. Equal split with a spare cent");
        Expense hotel = app.addExpense(trip.id(), "Hotel", "alice", Money.of("100.00"),
                List.of("alice", "bob", "carol"), new EqualSplit());
        System.out.println("Hotel shares: " + hotel.shares());

        System.out.println("== 2. Exact split");
        app.addExpense(trip.id(), "Dinner", "bob", Money.of("60.00"), List.of("alice", "bob", "carol"),
                new ExactSplit(Map.of("alice", Money.of("10.00"), "bob", Money.of("20.00"), "carol",
Money.of("30.00"))));

        System.out.println("== 3. Percent split");
        Expense taxi = app.addExpense(trip.id(), "Taxi", "carol", Money.of("50.00"), List.of("alice", "bob",
"dave"),
                new PercentSplit(Map.of("alice", new BigDecimal("33.33"), "bob", new BigDecimal("33.33"),
                        "dave", new BigDecimal("33.34"))));
        System.out.println("Taxi shares: " + taxi.shares());

        System.out.println("== 4. Invalid expenses are refused, ledger untouched");
        tryIt(() -> app.addExpense(trip.id(), "Snacks", "dave", Money.of("30.00"), List.of("alice", "dave"),
                new ExactSplit(Map.of("alice", Money.of("10.00"), "dave", Money.of("5.00")))));
        tryIt(() -> app.addExpense(trip.id(), "Coffee", "dave", Money.of("4.00"), List.of("dave"), new
EqualSplit()));
        tryIt(() -> app.addExpense(trip.id(), "Boat", "dave", Money.of("50.00"), List.of("alice", "dave"),
                new PercentSplit(Map.of("alice", new BigDecimal("60"), "dave", new BigDecimal("30")))));

        System.out.println("== 5. Who owes whom");
        app.printLedger();
        for (String u : List.of("alice", "bob", "carol", "dave")) System.out.println("  net " + u + " = " +
app.net(u));
        System.out.println("  bob owes alice: " + app.owes("bob", "alice"));

        System.out.println("== 6. Simplify the group's debts (min cash flow, greedy)");
        List<Payment> plan = app.simplifyAndApply(trip.id());
        plan.forEach(p -> System.out.println("  suggested: " + p));
        app.printLedger();
        for (String u : List.of("alice", "bob", "carol", "dave")) System.out.println("  net " + u + " = " +
app.net(u));

        System.out.println("== 7. Settle up");
        tryIt(() -> app.settleUp("dave", "alice", Money.of("999.00")));
        app.settleUp("bob", "alice", Money.of("5.00"));
        System.out.println("  after a partial payment of 5.00, bob owes alice: " + app.owes("bob", "alice"));
        app.settleUp("bob", "alice", app.owes("bob", "alice"));
        for (Payment p : plan) if (!p.from().equals("bob")) app.settleUp(p.from(), p.to(), p.amount());
        app.printLedger();

        System.out.println("== 8. Group history");
        app.history(trip.id()).forEach(h -> System.out.println("  " + h));
    }

    private static void tryIt(Runnable r) {
        try {
            r.run();
            System.out.println("  accepted");
        } catch (IllegalArgumentException e) {
```

```
            System.out.println("  refused: " + e.getMessage());
        }
    }
}
```

Walking through the code.

- `Money.of("10.005")` throws, because `RoundingMode.UNNECESSARY` refuses to silently round. Inputs with more than 2 decimals are a bug, not something to hide.
- `EqualSplit` divides with `RoundingMode.DOWN` (33.33), computes the leftover (0.01), and gives one extra cent to the first participant(s). The shares always add up exactly.
- `PercentSplit` rejects percentages that do not total 100, rounds each share with `HALF_EVEN` (banker's rounding: halves go to the even neighbour, which avoids a systematic upward bias), and lets the **last** person absorb whatever remains.
- `Ledger.Pair.of(a, b)` sorts the two ids so (alice, bob) and (bob, alice) are the same key. The sign of the stored value says who owes whom.
- `Splitwise.addExpense` builds shares, then the expense (which validates), and **only then** updates the ledger. That order is requirement 5.
- `settleUp` refuses to pay more than is owed, then calls `adjust(to, from, amount)`: a debt in the reverse direction that cancels the existing one.
- `simplifyAndApply` replaces all debts between the group's members with the simplified plan. Every net position stays the same (the output proves it).

Output:

```
== 1. Equal split with a spare cent
Hotel shares: [alice=33.34, bob=33.33, carol=33.33]
== 2. Exact split
== 3. Percent split
Taxi shares: [alice=16.66, bob=16.66, dave=16.68]
== 4. Invalid expenses are refused, ledger untouched
  refused: shares add up to 15.00 but the total is 30.00
  refused: an expense needs at least two people
  refused: percentages add up to 90, not 100
== 5. Who owes whom
  bob owes alice 23.33
  carol owes alice 16.67
  carol owes bob 13.34
  dave owes carol 16.68
  net alice = 40.00
  net bob = -9.99
  net carol = -13.33
  net dave = -16.68
  bob owes alice: 23.33
== 6. Simplify the group's debts (min cash flow, greedy)
  suggested: dave pays alice 16.68
  suggested: carol pays alice 13.33
  suggested: bob pays alice 9.99
  bob owes alice 9.99
  carol owes alice 13.33
  dave owes alice 16.68
  net alice = 40.00
  net bob = -9.99
  net carol = -13.33
  net dave = -16.68
== 7. Settle up
  refused: dave owes alice only 16.68, cannot pay 999.00
  after a partial payment of 5.00, bob owes alice: 4.99
  (everyone is settled up)
== 8. Group history
  e2 Hotel 100.00 paid by alice [alice=33.34, bob=33.33, carol=33.33]
  e3 Dinner 60.00 paid by bob [alice=10.00, bob=20.00, carol=30.00]
  e4 Taxi 50.00 paid by carol [alice=16.66, bob=16.66, dave=16.68]
```

Look at step 5 and step 6: four pairwise debts became three payments, and every net (alice +40.00, bob −9.99, carol −13.33, dave −16.68) is identical before and after.

Related machine-coding problem: The Ledger Co. (Navi)

Splitwise is not the only "ledger" question you will meet. The Geektrust / Navi machine-coding round (an interview round where you write a complete, runnable program in one or two hours) **"Ledger Co."** also keeps money records, but for **bank loans** (simple interest — interest charged only on the original amount borrowed; a fixed monthly instalment; lump-sum prepayments — extra one-off payments made early) rather than bills shared between friends. It is fully designed, coded and run in **Chapter 21, §21.6**, so it is not repeated here. The comparison is the useful part: both keep **one authoritative record** and compute every answer from it (Splitwise: one signed balance per pair of people; Ledger Co.: one loan per bank and borrower, holding its lump-sum payments), and both use `BigDecimal` with an explicitly named rounding mode. What differs is the rule set: Splitwise's rules are about *who owes whom*, Ledger Co.'s are about *how much has been repaid by a given month*.

Concurrency & edge cases

- **Two expenses at once.** Every public method on `Splitwise` is `synchronized` (a Java keyword that lets only one thread at a time run such methods on the same object), so "build → validate → adjust ledger" is atomic (it happens as one indivisible step that no other thread can see half-done). Simple and correct for one process. At scale you would lock per group or per pair of users (a **lock** is an object only one thread can hold at a time; others wait for it), or store balances in a database and update them in one transaction (a group of database changes that either all succeed or all fail together).
- **Rounding.** Covered by spare-cent distribution and "last person absorbs". Test: 100.00 / 3 must give 33.34 + 33.33 + 33.33.
- **Payer not among the sharers** (Alice pays for Bob and Carol only, or for Bob alone): allowed. Every share becomes a debt to Alice. `Expense.problem` counts the payer as one of the "at least two people", so Alice buying Bob's lunch is a valid expense, while Dave paying only for Dave is refused.
- **Payer's own share:** `adjust(alice, alice, …)` is a no-op (does nothing) by design.
- **Overpaying on settle-up:** refused, so a debt can never flip direction by accident.
- **Unknown user / not a group member:** refused before any computation.
- **Duplicate person in the shares list, negative share, zero total:** refused by `Expense.problem`.
- **Simplify and other groups.** Because the ledger has one number per *pair*, simplifying a group rewrites all debts *between its members*, even ones that came from another group with the same two people. That is the price of the single source of truth. If per-group simplification must not touch other groups, keep the ledger per group *and* a global view derived from them.

Extensions & follow-up questions

- **"Add a split by shares (e.g. 2:1:1)."** A new `ShareRatioSplit implements SplitStrategy`. Nothing else changes.
- **"Delete or edit an expense."** Our ledger stores totals, not history, so undo means re-adding the inverse (`adjust(payer, sharer, amount)` for each share). Cleaner: make the ledger an **append-only list of entries** (like accounting journals / event sourcing — storing every change as an event and rebuilding the current state from them) and compute balances on demand or in a cache (a saved copy of computed results kept for quick reuse; see 20.3). Then delete = append a reversing entry.
- **"Multiple currencies."** Add a `Currency` to every amount (a `Money(amount, currency)` value object) and keep one ledger per currency, or convert at the time of the expense and store the rate used.
- **"Is the greedy always optimal?"** No. Counter-example: nets A −6, B −5, C +2, D +4, E +5. Greedy: A pays E 5, B pays D 4, then two more small payments to C = **4 payments**. Optimal: B pays E 5 (they cancel exactly), A pays D 4 and C 2 = **3 payments**. The optimum groups people into as many zero-sum subsets as possible, which needs an exponential search; greedy is the accepted interview answer.
- **"Isn't `Splitwise` doing too much?"** A fair criticism: it holds users, groups, expenses *and* the ledger. At larger scale, give each of the four its own repository (store) and let `Splitwise` shrink to the part that orders the steps. Say this before the interviewer does.
- **"Show a user's activity feed."** Keep expenses and settle-ups as events; the feed is a query over them.
- **"Notify people when an expense is added."** Publish an `ExpenseAdded` event to an Observer / the notification service in 20.2.

Real-world example

Splitwise really does offer a "simplify debts" option for groups: when it is on, the app shows each member a small number of payments to make instead of every pairwise debt, exactly the net-position idea above. Payment apps with "split the bill" features face the same rounding problem and solve it the same way: one person absorbs the leftover cent.

In the interview

30-second answer: "Users and groups hold no money. Each split rule is a `SplitStrategy` that returns shares. An `Expense` validates itself once: at least two people, shares add to the total. The `Ledger` keeps one signed number per pair of people and has a single `adjust` method, so debts cancel automatically. Settling up is just an adjust in the other direction. Simplification uses net positions and a greedy biggest-debtor-pays-biggest-creditor loop: at most n − 1 payments, not always the true minimum."

Tip: reach the split rules early and say the phrase "one class per split rule" while you draw them; that is the part of the question with a pattern name. Then invest your time in the ledger, because the one-number-per-pair decision is what keeps everything else short.

Suggested 40-minute plan: clarify (5) → entities + the "no money in groups" decision (5) → Strategy for splits and the cents problem (8) → Expense validation (5) → Ledger with one number per pair (7) → simplify algorithm (5) → concurrency and extensions (5).

Likely follow-ups (with answers):

- *"What if 100 does not divide by 3?"* → Hand out spare cents one by one; the total is always exact.
- *"How do you simplify debts?"* → Net positions + greedy; mention n − 1 bound and that optimal is NP-hard.
- *"Why not store balances in the group?"* → A person in two groups would have two copies of the truth.

Self-check

1. Why does `Group` hold no balances? → So each debt lives in exactly one place (the ledger); two copies would eventually disagree.
2. Split 10.00 equally among 3 people. → 3.34, 3.33, 3.33.
3. Why does `Expense.problem` return a `String` instead of a `boolean`? → So the caller can show the reason it was refused.
4. What is the maximum number of payments the greedy simplifier suggests for n people? → n − 1.
5. Why compare `BigDecimal` with `compareTo` rather than `equals`? → `equals` also compares scale, so `2.0` does not equal `2.00`.
6. Alice pays 20.00 for Bob's lunch and Bob owes the whole 20.00. Is that a valid expense? → Yes. Two people are involved (payer and sharer), even though only one of them has a share.

TL;DR

- **Strategy** for split rules; every rule returns shares.
- **Expense validates itself** once; invalid expenses never exist.
- **Ledger = one signed number per pair**, one mutator; settle-up is a reverse adjust.
- **Simplify** = net positions + greedy; ≤ n − 1 payments; optimal is NP-hard.
- **BigDecimal**, explicit rounding, no lost cents.

20.2 Design a Notification Service

Problem & clarifying questions

`What it is.` A notification service is the component of a product responsible for emails, text messages (SMS) and push alerts ("Your order has shipped", "Your login code is 482913"). Everyday analogy: a post office

counter. Many departments hand it letters; the counter picks the right delivery method, respects "do not disturb" notes, retries if the courier fails, and keeps a receipt for every letter.

Why it is a design question. Every product grows one. It starts as one `sendEmail()` function. Then someone adds SMS. Then "don't wake me at 3 a.m.". Then marketing wants "max 2 promos per hour". Six months later one method has fourteen `if`s and nobody dares touch it. The whole exercise is: **where should each of those additions have gone instead?**

Question	Assumed answer
Channels?	Email, SMS, push. More later (WhatsApp, Slack).
Who chooses the channel?	The user, **per kind** of notification (SMS for deliveries, email for newsletters), in order of preference.
Wording?	Templates with placeholders, e.g. `Hi {name}, it arrives {eta}.`
Quiet hours?	Yes, per user; they can wrap past midnight (22:00–07:00). Urgent messages (login codes) ignore them.
Rate limits?	At most N notifications of one kind per user per hour.
Failures?	Retry a channel up to a limit, then fall back to the next accepted channel; if all fail, record FAILED.
Audit?	Every attempt recorded: SENT, FAILED or SUPPRESSED, and why.
Sync or async? (synchronous = caller waits; asynchronous = caller moves on)	The caller should not wait for a slow SMS gateway (the provider service that actually sends texts).
Bulk sends / translations / open tracking / unsubscribe links?	Out of scope.

Requirements

Functional

1. A user has a name, email, phone number and device token (the address a push service uses for one phone).
2. Channels: email, SMS, push.
3. Each user chooses accepted channels **per kind** of notification, in order.
4. The text for each kind comes from a template with placeholders ("holes") filled in per request.
5. Users can set quiet hours; nothing normal is sent during them.
6. Urgent notifications ignore quiet hours.
7. A user receives no more than N notifications of a given kind in any hour.
8. A channel that fails is retried, but only a bounded number of times.
9. When a channel runs out of retries, the service falls back to the user's next accepted channel.
10. When no channel succeeds, the outcome is stored as FAILED.
11. Every attempt is recorded with its outcome and reason.
12. Adding a channel, **or** a new reason to hold a message back, requires no edit to the service class.

Non-functional: the caller is not blocked by slow providers (async option); thread-safe under concurrent sends; testable without waiting for real time (injected clock); a broken listener must never break delivery. An **injected** clock is one handed to the class from outside (instead of the class reading the system time itself), so a test can pass in a fake clock it moves by hand; a **listener** is an object that has asked to be told when something happens.

Use cases

Actors: **Product code** (asks for a notification), **Person** (chooses channels, sets quiet hours), **Operator** (adds channels, watches outcomes), **Service** (renders, checks rules, tries channels, retries, records).

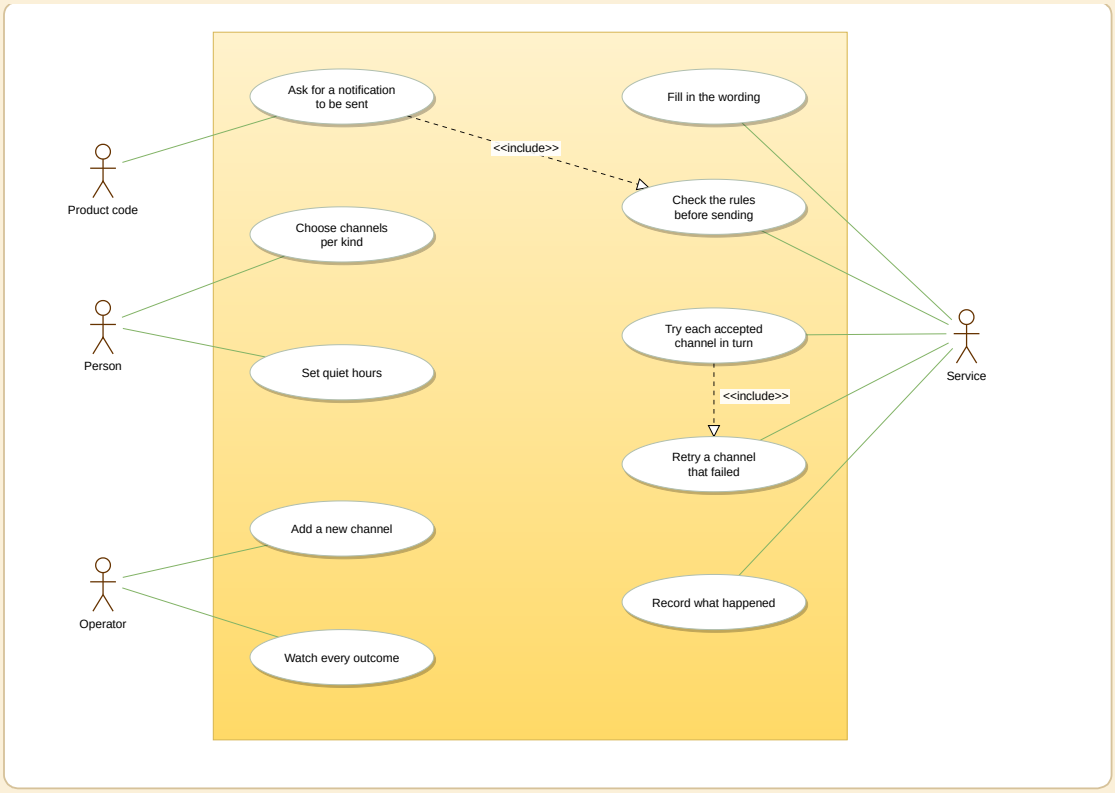


Figure — Notification service use cases: product code asks for a notification; the service checks rules, fills in the wording, tries each channel with retries and records the outcome (course diagram)

Core entities

Class	Responsibility
`User` (record)	Who someone is: id, name, email, phone, device token.
`Preferences`	What they accept: ordered channels per kind; quiet hours (with midnight wrap logic in **one** place).
`Template` (record)	Subject + body with `{holes}`; `render(values)` → `Message`.
`Message` (record)	The finished subject and body, identical on every channel.
`NotificationRequest` (record)	One ask: kind, user, values, priority (NORMAL / URGENT).
`Gateway` (interface)	The external provider (SMTP server, SMS gateway, push service). Swappable and fakeable.
`NotificationChannel` (abstract)	**Template Method**: the final `deliver()` holds the retry-with-backoff loop; subclasses supply only `addressOf(user)`.
`EmailChannel`, `SmsChannel`, `PushChannel`	Three tiny subclasses.
`ChannelFactory`	**Simple Factory**: channel type → channel object.
`RetryPolicy` (record)	Max attempts + initial backoff (doubles each time).
`SendRule` (interface)	**Strategy**: one reason not to send. Returns the reason text, or empty.
`QuietHoursRule`, `RateLimitRule`	Two rules. Quiet hours knows urgent bypasses it; rate limit counts only *successful* sends.
`DeliveryRecord` (record)	What happened: status, channel, attempts, reason.
`DeliveryListener` (interface)	**Observer**: told every outcome. `LoggingListener`, `MetricsListener`.
`TimeSource`	Where "now" comes from (a manual clock in tests).
`NotificationService`	Thin coordinator: rules → preferences → render → channels in order → `finish()`. Decides nothing itself.

Terms from the table: **SMTP** is the protocol mail servers use to pass email along; a **fake** is a pretend version of a real dependency used in tests; an **abstract** class cannot be created on its own and exists to be extended; a `final` method cannot be overridden by subclasses; **backoff** means waiting a little longer before each new retry, so a struggling provider gets time to recover.

Why `User` and `Preferences` are separate classes. Who you are rarely changes; what you are willing to receive changes often. Separate reasons to change → separate classes (Single Responsibility Principle).

Class diagram

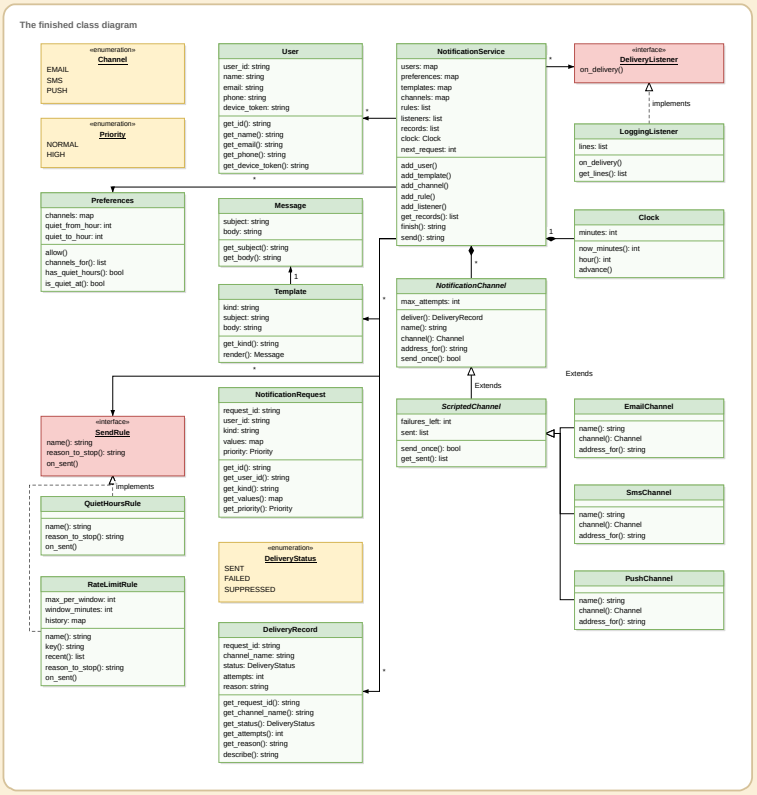
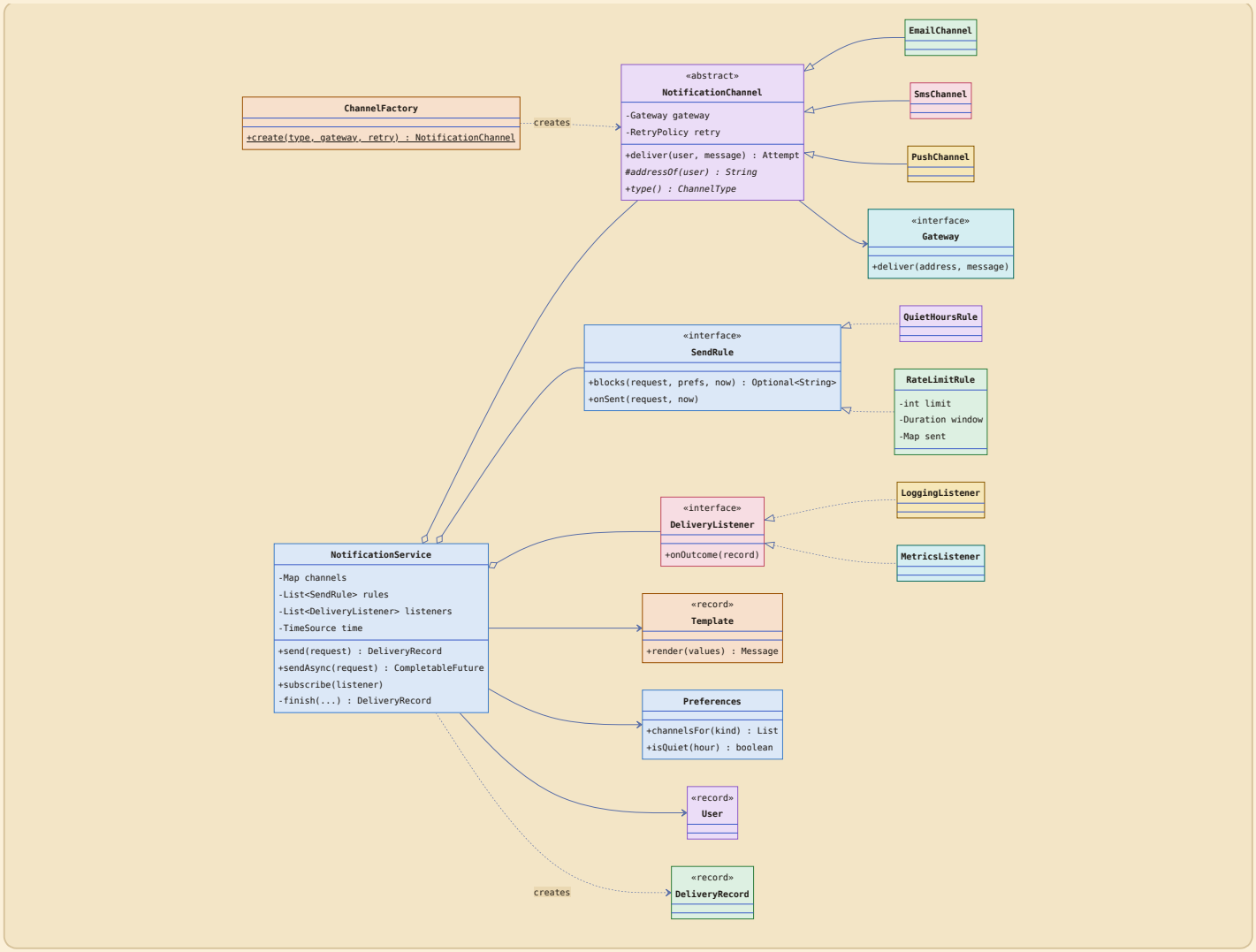


Figure — The finished notification service class diagram: the service in the middle, the channel hierarchy with the retry in the parent, the send rules, templates, preferences and the delivery listeners (course diagram)

In the figure above, notice that the three channel classes are tiny: all the behaviour that makes sending reliable lives in the abstract parent.

The course reaches this diagram in five build stages, each leaving something that already works: (1) `User` + `Preferences` (you can ask what a person accepts, and when); (2) `Template` + `Message` + `NotificationRequest` (wording filled in once); (3) the channel hierarchy (delivery with retries and an outcome); (4) the `SendRule`s (a notification can be stopped before sending); (5) `NotificationService` + listeners (end-to-end sending, observable by any listener). Reuse this order when you interview.



Key flows

Sending one notification. A rule (or a missing channel preference) may stop it before any work is done; otherwise channels are tried in the user's order until one succeeds.

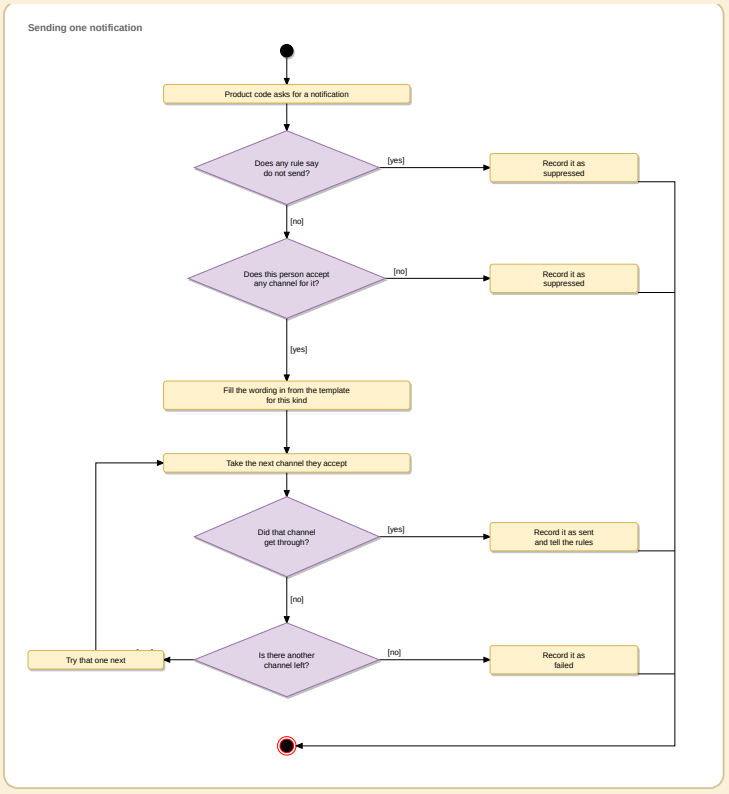


Figure — Sending one notification: ask every rule, render once, try each accepted channel in turn, and record sent / failed / suppressed (course diagram)

Inside one channel — the retry loop, which exists exactly once, in the abstract parent:

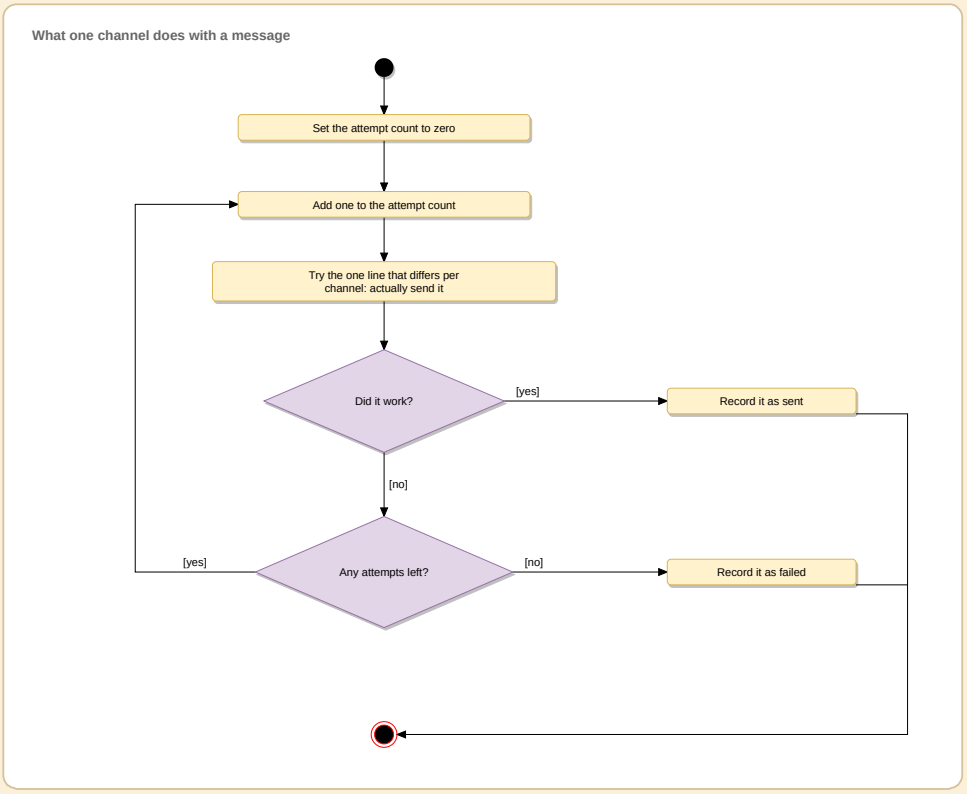
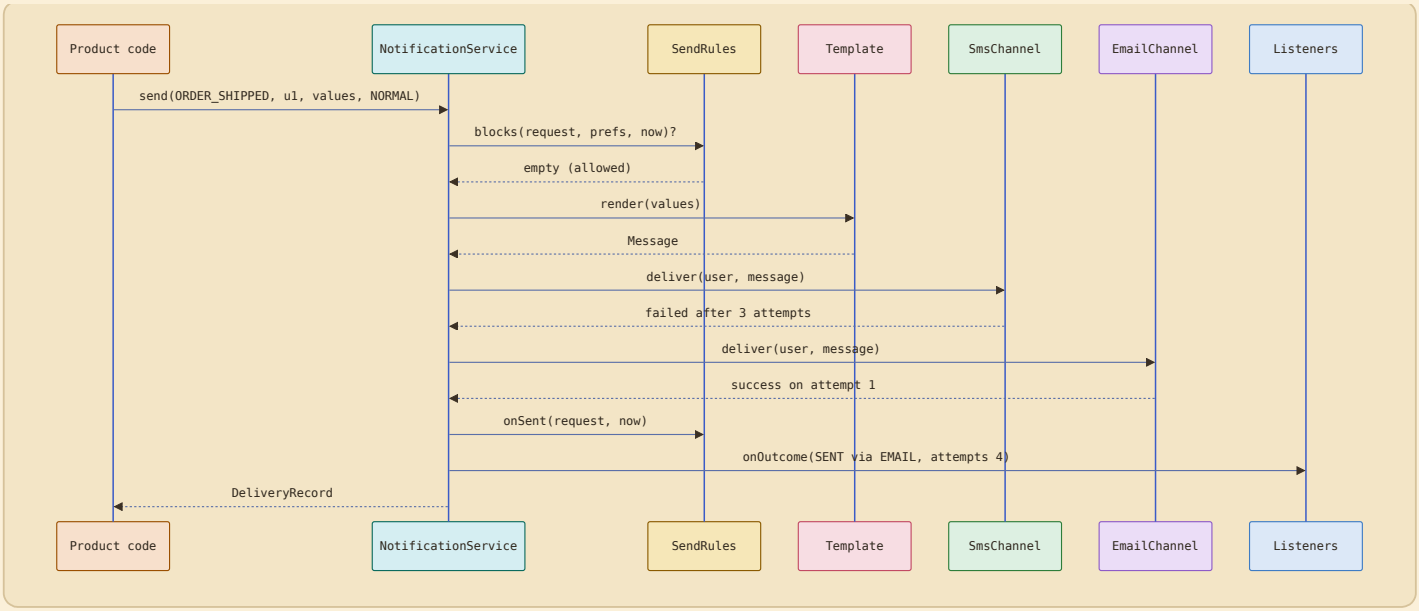


Figure — One channel's delivery loop: attempt, and on failure retry while attempts remain; then report success or give up (course diagram)


Product code
NotificationService
SendRules
Template
SmsChannel
EmailChannel
Listeners
send(ORDER_SHIPPED, u1, values, NORMAL)
blocks(request, prefs, now)?
empty (allowed)
render(values)
Message
deliver(user, message)
failed after 3 attempts
deliver(user, message)
success on attempt 1
onSent(request, now)
onOutcome(SENT via EMAIL, attempts 4)
DeliveryRecord
Product code
NotificationService
SendRules
Template
SmsChannel
EmailChannel
Listeners

Design decisions & patterns used

Pattern / decision	Where	Why	Alternative rejected
Template Method	`NotificationChannel.deliver()` is `final`; subclasses fill `addressOf()`	The retry loop is identical for all channels. Write it once; new channels inherit reliability. Only a *step* varies, not the whole algorithm.	Copying the loop into every channel (they drift apart), or Strategy (would replace the whole method, not one step).
Strategy	`SendRule`	Every "reason not to send" is its own class. A new rule (e.g. "user unsubscribed", "duplicate within 5 minutes") is a new class plus one line in the wiring code.	`if (quiet) … if (tooMany) …` inside `send`.
Observer	`DeliveryListener`	Logs, dashboards, alerts all want outcomes. The service announces; it doesn't know who listens.	Hard-coding logging/metrics in the service; it grows forever.
Simple Factory	`ChannelFactory`	One place maps `ChannelType` → class; config can list channel names.	`new SmsChannel(...)` scattered through wiring code.
Adapter (target interface; an adapter converts one API into the shape your code expects)	`Gateway`	Each real provider SDK (the vendor's client library) has its own API (the set of calls a library or service offers to programs); an adapter per provider implements `Gateway`. Also lets tests use a fake.	Channels calling vendor SDKs directly.
Rule returns **reason text**, not boolean	`SendRule.blocks`	The reason is recorded (requirement 11).	`boolean allowed()` loses why.
Urgency handled **inside** the rule	`QuietHoursRule`	The service doesn't need to know which rules urgent messages ignore.	A special case in the service.
Rate limit counts only **successful** sends	`onSent` hook	A suppressed/failed message shouldn't use up the user's hourly allowance.	Counting at check time.
Single exit `finish()`	Service	Every path stores a record and notifies listeners. A second exit that forgot would leave invisible gaps.	Returning from many places.
Template rendered **once**	Service	Every channel gets the same text; a wording change is one edit.	Wording inside each channel.

Using three patterns here is not decoration: each one soaks up a *separate* axis of change — another channel (Template Method + Factory), another reason to hold a message back (Strategy), another party interested in outcomes (Observer).

Java implementation

```java
// File: code/lld/ch20/notification/Model.java
package lld.ch20.notification;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

enum ChannelType { EMAIL, SMS, PUSH }

enum Priority { NORMAL, URGENT }

enum DeliveryStatus { SENT, FAILED, SUPPRESSED }

/** Who a person is: identity and addresses. */
record User(String id, String name, String email, String phone, String deviceToken) {}

/** What a person is willing to receive, and when. Changes far more often than User. */
final class Preferences {
    private final Map<String, List<ChannelType>> channelsByKind = new HashMap<>();
    private Integer quietStart; // hour of day, inclusive
    private Integer quietEnd;   // hour of day, exclusive

    Preferences accept(String kind, ChannelType... inOrder) {
        channelsByKind.put(kind, List.of(inOrder));
        return this;
    }

    Preferences quietHours(int startHour, int endHour) {
        this.quietStart = startHour;
        this.quietEnd = endHour;
        return this;
    }

    List<ChannelType> channelsFor(String kind) { return channelsByKind.getOrDefault(kind, List.of()); }

    /** Quiet hours may wrap past midnight (22 -> 7), so the comparison lives here, once. */
    boolean isQuiet(int hour) {
        if (quietStart == null) return false;
        return quietStart <= quietEnd
                ? hour >= quietStart && hour < quietEnd
                : hour >= quietStart || hour < quietEnd;
    }
}

/** The finished wording handed to every channel. */
record Message(String subject, String body) {}

/** Wording for one kind of notification, with {holes} to fill in. */
record Template(String kind, String subject, String body) {
    private static final Pattern HOLE = Pattern.compile("\\{(\\w+)}");

    Message render(Map<String, String> values) {
        return new Message(fill(subject, values), fill(body, values));
    }

    private static String fill(String text, Map<String, String> values) {
        Matcher m = HOLE.matcher(text);
        StringBuilder out = new StringBuilder();
        while (m.find()) {
            String v = values.get(m.group(1));
            if (v == null) throw new IllegalArgumentException("missing value for {" + m.group(1) + "}");
            m.appendReplacement(out, Matcher.quoteReplacement(v));
```

```
        }
        return m.appendTail(out).toString();
    }
}

/** One ask from product code. Values are kept sorted so the same request renders the same text. */
record NotificationRequest(String kind, String userId, Map<String, String> values, Priority priority) {
    NotificationRequest {
        values = new TreeMap<>(values);
    }
}

/** What happened to one notification. */
record DeliveryRecord(String kind, String userId, DeliveryStatus status, ChannelType channel,
                      int attempts, String reason) {}

/** Where "now" comes from, so quiet hours and rate limits are testable without waiting. */
interface TimeSource { LocalDateTime now(); }

final class ManualTime implements TimeSource {
    private LocalDateTime now;
    ManualTime(LocalDateTime start) { this.now = start; }
    public synchronized LocalDateTime now() { return now; }
    synchronized void set(LocalDateTime t) { now = t; }
}
```

```java
// File: code/lld/ch20/notification/Channels.java
package lld.ch20.notification;

import java.time.Duration;

/** Adapter target: the outside provider (SMTP server, SMS gateway, FCM/APNs). */
interface Gateway {
    /** Throws on failure. */
    void deliver(String address, Message message) throws Exception;
}

/** How many times to try and how long to wait between tries (exponential backoff). */
record RetryPolicy(int maxAttempts, Duration initialBackoff) {}

/** The outcome of one channel's delivery loop. */
record Attempt(boolean success, int attempts, String error) {}

/**
 * Template Method: deliver() is the fixed algorithm (try, back off, retry) and is final.
 * Subclasses fill in only the steps that differ: which address to use.
 */
abstract class NotificationChannel {
    private final Gateway gateway;
    private final RetryPolicy retry;

    NotificationChannel(Gateway gateway, RetryPolicy retry) {
        this.gateway = gateway;
        this.retry = retry;
    }

    abstract ChannelType type();

    /** Hook: where this channel reaches the user, or null if the user has no such address. */
    protected abstract String addressOf(User user);

    final Attempt deliver(User user, Message message) {
        String address = addressOf(user);
        if (address == null || address.isBlank()) return new Attempt(false, 0, "no " + type() + " address");
        String lastError = null;
        long backoff = retry.initialBackoff().toMillis();
        for (int attempt = 1; attempt <= retry.maxAttempts(); attempt++) {
            try {
                gateway.deliver(address, message);
                return new Attempt(true, attempt, null);
            } catch (Exception e) {
                lastError = e.getMessage();
                if (attempt < retry.maxAttempts()) sleep(backoff);
                backoff *= 2;
            }
        }
        return new Attempt(false, retry.maxAttempts(), lastError);
    }

    private static void sleep(long ms) {
        try {
            Thread.sleep(ms);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

final class EmailChannel extends NotificationChannel {
    EmailChannel(Gateway g, RetryPolicy r) { super(g, r); }
    ChannelType type() { return ChannelType.EMAIL; }
    protected String addressOf(User u) { return u.email(); }
}
```

```
final class SmsChannel extends NotificationChannel {
    SmsChannel(Gateway g, RetryPolicy r) { super(g, r); }
    ChannelType type() { return ChannelType.SMS; }
    protected String addressOf(User u) { return u.phone(); }
}

final class PushChannel extends NotificationChannel {
    PushChannel(Gateway g, RetryPolicy r) { super(g, r); }
    ChannelType type() { return ChannelType.PUSH; }
    protected String addressOf(User u) { return u.deviceToken(); }
}

/** Factory: the one place that knows which class implements which channel type. */
final class ChannelFactory {
    private ChannelFactory() {}

    static NotificationChannel create(ChannelType type, Gateway gateway, RetryPolicy retry) {
        return switch (type) {
            case EMAIL -> new EmailChannel(gateway, retry);
            case SMS -> new SmsChannel(gateway, retry);
            case PUSH -> new PushChannel(gateway, retry);
        };
    }
}
```

```java
// File: code/lld/ch20/notification/Rules.java
package lld.ch20.notification;

import java.time.Duration;
import java.time.LocalDateTime;
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

/** Strategy: one reason not to send. Returns the reason, or empty to let it through. */
interface SendRule {
    Optional<String> blocks(NotificationRequest request, Preferences prefs, LocalDateTime now);

    /** Told after a successful send (so suppressed messages do not count). */
    default void onSent(NotificationRequest request, LocalDateTime now) {}
}

/** Nothing normal during quiet hours. The rule itself knows that URGENT ignores it. */
final class QuietHoursRule implements SendRule {
    @Override
    public Optional<String> blocks(NotificationRequest r, Preferences prefs, LocalDateTime now) {
        if (r.priority() == Priority.URGENT) return Optional.empty();
        return prefs.isQuiet(now.getHour()) ? Optional.of("quiet hours") : Optional.empty();
    }
}

/** At most 'limit' notifications of one kind per user per sliding window. */
final class RateLimitRule implements SendRule {
    private final int limit;
    private final Duration window;
    private final Map<String, Deque<LocalDateTime>> sent = new ConcurrentHashMap<>();

    RateLimitRule(int limit, Duration window) {
        this.limit = limit;
        this.window = window;
    }

    private static String key(NotificationRequest r) { return r.userId() + "|" + r.kind(); }

    @Override
    public Optional<String> blocks(NotificationRequest r, Preferences prefs, LocalDateTime now) {
        Deque<LocalDateTime> times = sent.computeIfAbsent(key(r), k -> new ArrayDeque<>());
        synchronized (times) {
            prune(times, now); // old timestamps are dropped, so one key's deque never exceeds 'limit'
            return times.size() >= limit
                    ? Optional.of("rate limit " + limit + " per " + window.toMinutes() + " min")
                    : Optional.empty();
        }
    }

    @Override
    public void onSent(NotificationRequest r, LocalDateTime now) {
        Deque<LocalDateTime> times = sent.computeIfAbsent(key(r), k -> new ArrayDeque<>());
        synchronized (times) {
            times.addLast(now);
        }
    }

    private void prune(Deque<LocalDateTime> times, LocalDateTime now) {
        LocalDateTime cutoff = now.minus(window);
        while (!times.isEmpty() && !times.peekFirst().isAfter(cutoff)) times.pollFirst();
    }
}

/** Observer: told the outcome of every notification. */
interface DeliveryListener {
```

```
    void onOutcome(DeliveryRecord record);
}

final class LoggingListener implements DeliveryListener {
    @Override
    public void onOutcome(DeliveryRecord r) {
        System.out.printf("  [log] %-10s %-13s user=%s channel=%s attempts=%d%s%n", r.status(), r.kind(),
                r.userId(), r.channel() == null ? "-" : r.channel(), r.attempts(),
                r.reason() == null ? "" : " reason=" + r.reason());
    }
}

final class MetricsListener implements DeliveryListener {
    private final Map<DeliveryStatus, Integer> counts = new ConcurrentHashMap<>();

    @Override
    public void onOutcome(DeliveryRecord r) { counts.merge(r.status(), 1, Integer::sum); }

    Map<DeliveryStatus, Integer> counts() { return new java.util.TreeMap<>(counts); }
}
```

```java
// File: code/lld/ch20/notification/NotificationService.java
package lld.ch20.notification;

import java.time.LocalDateTime;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

/** The class product code calls. It orders the steps and decides nothing itself. */
final class NotificationService implements AutoCloseable {
    private final Map<ChannelType, NotificationChannel> channels = new EnumMap<>(ChannelType.class);
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final Map<String, Preferences> preferences = new ConcurrentHashMap<>();
    private final Map<String, Template> templates = new ConcurrentHashMap<>();
    private final List<SendRule> rules;
    private final List<DeliveryListener> listeners = new CopyOnWriteArrayList<>();
    private final List<DeliveryRecord> history = new CopyOnWriteArrayList<>();
    private final TimeSource time;
    private final ExecutorService workers = Executors.newFixedThreadPool(4);

    NotificationService(List<NotificationChannel> channelList, List<SendRule> rules, TimeSource time) {
        channelList.forEach(c -> channels.put(c.type(), c));
        this.rules = List.copyOf(rules);
        this.time = time;
    }

    void register(User u, Preferences p) { users.put(u.id(), u); preferences.put(u.id(), p); }
    void addTemplate(Template t) { templates.put(t.kind(), t); }
    void subscribe(DeliveryListener l) { listeners.add(l); }
    List<DeliveryRecord> history() { return List.copyOf(history); }

    /** Synchronous send: rules -> render -> try each accepted channel in the user's order. */
    DeliveryRecord send(NotificationRequest req) {
        User user = users.get(req.userId());
        if (user == null) return finish(req, DeliveryStatus.FAILED, null, 0, "unknown user");
        Preferences prefs = preferences.get(req.userId());
        LocalDateTime now = time.now();

        for (SendRule rule : rules) {
            Optional<String> reason = rule.blocks(req, prefs, now);
            if (reason.isPresent()) return finish(req, DeliveryStatus.SUPPRESSED, null, 0, reason.get());
        }
        List<ChannelType> accepted = prefs.channelsFor(req.kind());
        if (accepted.isEmpty()) return finish(req, DeliveryStatus.SUPPRESSED, null, 0, "user accepts no channel");

        Template template = templates.get(req.kind());
        if (template == null) return finish(req, DeliveryStatus.FAILED, null, 0, "no template");
        Message message;
        try {
            message = template.render(req.values()); // rendered ONCE, same text on every channel
        } catch (IllegalArgumentException e) {        // e.g. a {hole} with no value: still leave via finish()
            return finish(req, DeliveryStatus.FAILED, null, 0, "template: " + e.getMessage());
        }

        int totalAttempts = 0;
        String lastError = null;
        for (ChannelType type : accepted) {
            NotificationChannel channel = channels.get(type);
            if (channel == null) continue;
            Attempt a = channel.deliver(user, message);
            totalAttempts += a.attempts();
            if (a.success()) {
```

```
                rules.forEach(r -> r.onSent(req, now));
                return finish(req, DeliveryStatus.SENT, type, totalAttempts, null);
            }
            lastError = type + ": " + a.error();
        }
        return finish(req, DeliveryStatus.FAILED, null, totalAttempts, "all channels failed, last " + lastError);
    }

    /** Asynchronous send: the caller (e.g. checkout) is not blocked by a slow provider. */
    CompletableFuture<DeliveryRecord> sendAsync(NotificationRequest req) {
        return CompletableFuture.supplyAsync(() -> send(req), workers);
    }

    /** The single exit: every outcome is stored and announced here. */
    private DeliveryRecord finish(NotificationRequest req, DeliveryStatus status, ChannelType channel,
                                  int attempts, String reason) {
        DeliveryRecord record = new DeliveryRecord(req.kind(), req.userId(), status, channel, attempts, reason);
        history.add(record);
        for (DeliveryListener l : listeners) {
            try {
                l.onOutcome(record);
            } catch (RuntimeException e) {
                // a broken listener must not break delivery
            }
        }
        return record;
    }

    @Override
    public void close() { workers.shutdown(); }
}
```

```java
// File: code/lld/ch20/notification/Main.java
package lld.ch20.notification;

import java.time.Duration;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {

    /** A fake provider that fails the next N calls, then succeeds, printing what it "sent". */
    static final class FakeGateway implements Gateway {
        private final String name;
        private final AtomicInteger failuresLeft = new AtomicInteger();
        FakeGateway(String name) { this.name = name; }
        void failNext(int n) { failuresLeft.set(n); }

        @Override
        public void deliver(String address, Message m) throws Exception {
            if (failuresLeft.getAndUpdate(x -> Math.max(0, x - 1)) > 0) throw new Exception(name + " timeout");
            System.out.println("    " + name + " -> " + address + " : " + m.subject() + " | " + m.body());
        }
    }

    public static void main(String[] args) throws Exception {
        FakeGateway smtp = new FakeGateway("SMTP"), twilio = new FakeGateway("SMS-GW"), fcm = new
FakeGateway("FCM");
        RetryPolicy retry = new RetryPolicy(3, Duration.ofMillis(1));
        ManualTime clock = new ManualTime(LocalDateTime.of(2026, 9, 28, 14, 0));

        MetricsListener metrics = new MetricsListener();
        try (NotificationService svc = new NotificationService(
                List.of(ChannelFactory.create(ChannelType.EMAIL, smtp, retry),
                        ChannelFactory.create(ChannelType.SMS, twilio, retry),
                        ChannelFactory.create(ChannelType.PUSH, fcm, retry)),
                List.of(new QuietHoursRule(), new RateLimitRule(2, Duration.ofHours(1))),
                clock)) {
            svc.subscribe(new LoggingListener());
            svc.subscribe(metrics);

            svc.addTemplate(new Template("ORDER_SHIPPED", "Order {orderId} shipped", "Hi {name}, it arrives
{eta}."));
            svc.addTemplate(new Template("OTP", "Your code", "{code} is your login code."));
            svc.addTemplate(new Template("PROMO", "Sale!", "{name}, {pct}% off today."));

            svc.register(new User("u1", "Asha", "asha@mail.com", "+91-98450", "tok-asha"),
                    new Preferences().accept("ORDER_SHIPPED", ChannelType.SMS, ChannelType.EMAIL)
                            .accept("OTP", ChannelType.PUSH, ChannelType.SMS)
                            .accept("PROMO", ChannelType.EMAIL)
                            .quietHours(22, 7));
            svc.register(new User("u2", "Ben", "ben@mail.com", null, "tok-ben"),
                    new Preferences().accept("ORDER_SHIPPED", ChannelType.PUSH));

            Map<String, String> order = Map.of("orderId", "A-17", "name", "Asha", "eta", "Tuesday");

            System.out.println("== 1. Normal send over the first accepted channel (SMS)");
            svc.send(new NotificationRequest("ORDER_SHIPPED", "u1", order, Priority.NORMAL));

            System.out.println("== 2. SMS fails 3 times -> retried, then falls back to EMAIL");
            twilio.failNext(3);
            svc.send(new NotificationRequest("ORDER_SHIPPED", "u1", order, Priority.NORMAL));

            System.out.println("== 3. Quiet hours at 23:00: normal is suppressed, urgent goes through");
            clock.set(LocalDateTime.of(2026, 9, 28, 23, 0));
            svc.send(new NotificationRequest("PROMO", "u1", Map.of("name", "Asha", "pct", "20"), Priority.NORMAL));
            svc.send(new NotificationRequest("OTP", "u1", Map.of("code", "482913"), Priority.URGENT));
```

```
            System.out.println("== 4. Rate limit: 2 PROMO per hour, the 3rd is suppressed");
            clock.set(LocalDateTime.of(2026, 9, 29, 10, 0));
            for (int i = 0; i < 3; i++)
                svc.send(new NotificationRequest("PROMO", "u1", Map.of("name", "Asha", "pct", "" + (10 + i)),
Priority.NORMAL));
            clock.set(LocalDateTime.of(2026, 9, 29, 11, 1));
            System.out.println("  ...61 minutes later the window has moved:");
            svc.send(new NotificationRequest("PROMO", "u1", Map.of("name", "Asha", "pct", "50"), Priority.NORMAL));

            System.out.println("== 5. Every channel gives up -> FAILED");
            fcm.failNext(3);
            svc.send(new NotificationRequest("ORDER_SHIPPED", "u2",
                    Map.of("orderId", "B-2", "name", "Ben", "eta", "Friday"), Priority.NORMAL));

            System.out.println("== 6. No accepted channel for this kind -> SUPPRESSED");
            svc.send(new NotificationRequest("PROMO", "u2", Map.of("name", "Ben", "pct", "5"), Priority.NORMAL));

            System.out.println("== 7. Template value missing -> FAILED, still recorded through finish()");
            svc.send(new NotificationRequest("OTP", "u1", Map.of(), Priority.URGENT));

            System.out.println("== 8. Async send (caller not blocked)");
            DeliveryRecord r = svc.sendAsync(new NotificationRequest("ORDER_SHIPPED", "u2",
                    Map.of("orderId", "B-3", "name", "Ben", "eta", "Monday"), Priority.NORMAL)).get();
            System.out.println("  future completed with " + r.status() + " via " + r.channel());

            System.out.println("== Metrics: " + metrics.counts() + ", history size = " + svc.history().size());
        }
    }
}
```

Walking through the code.

- `Preferences.isQuiet` handles the midnight wrap: if start ≤ end (09–17) it is a simple range; otherwise (22–07) it is "after start **or** before end".
- `Template.render` uses a regex (regular expression: a small pattern language for finding text) `\{(\w+)}` and `Matcher.appendReplacement` with `Matcher.quoteReplacement`, so a value containing `$` or `\` is inserted literally. A missing value is an error, not a silent `{name}` in a customer's inbox.
- `NotificationChannel.deliver` is `final`: subclasses cannot break the retry loop. Backoff doubles after each failure (exponential backoff). The course's version retries immediately and names that as a weakness (three back-to-back tries against a server that is down are three quick failures); the waiting belongs in this one parent loop, which is exactly where we put it.
- `RateLimitRule` keeps a deque (double-ended queue: items can be added or removed at either end) of send times per (user, kind) and **prunes** old ones on every check, so one key's deque never holds more than `limit` entries (the course's version never discarded old timestamps and flagged that as a known leak — a **memory leak** is memory a program keeps holding although it no longer needs it). The *map of keys* still grows with every (user, kind) pair ever seen; a production version adds a periodic sweep that drops empty or stale deques, or uses a fixed-size window counter per key.
- `NotificationService.send` has one exit, `finish()`. Even a template with a missing value is caught and recorded as FAILED (step 7 of the output) instead of escaping as an exception that no listener would ever hear about. A listener that throws is caught so it cannot break delivery.
- `NotificationRequest` copies its values into a `TreeMap` (a map that keeps its keys sorted), so the values are always in the same order and the same request always renders the same text: handy when comparing test output.
- `sendAsync` runs `send` on a worker pool (a fixed set of threads that pick up tasks as they arrive) and returns a `CompletableFuture` (a handle to a result that will be ready later); in production a durable queue (Kafka, SQS) would sit here instead.

> **Key term — message queue:** a queue is a waiting line for work. Producers put messages in at one end; workers take them out at the other, at their own pace. A **durable** queue stores its messages on disk, so they survive a crash or restart. **Kafka** (an open-source log-based messaging system) and **Amazon SQS** (a managed queue service in AWS, Amazon's cloud) are two widely used examples. See System Design Fundamentals for queues in larger systems.

Output:

```
== 1. Normal send over the first accepted channel (SMS)
    SMS-GW -> +91-98450 : Order A-17 shipped | Hi Asha, it arrives Tuesday.
  [log] SENT       ORDER_SHIPPED user=u1 channel=SMS attempts=1
== 2. SMS fails 3 times -> retried, then falls back to EMAIL
    SMTP -> asha@mail.com : Order A-17 shipped | Hi Asha, it arrives Tuesday.
  [log] SENT       ORDER_SHIPPED user=u1 channel=EMAIL attempts=4
== 3. Quiet hours at 23:00: normal is suppressed, urgent goes through
  [log] SUPPRESSED PROMO         user=u1 channel=- attempts=0 reason=quiet hours
    FCM -> tok-asha : Your code | 482913 is your login code.
  [log] SENT       OTP           user=u1 channel=PUSH attempts=1
== 4. Rate limit: 2 PROMO per hour, the 3rd is suppressed
    SMTP -> asha@mail.com : Sale! | Asha, 10% off today.
  [log] SENT       PROMO         user=u1 channel=EMAIL attempts=1
    SMTP -> asha@mail.com : Sale! | Asha, 11% off today.
  [log] SENT       PROMO         user=u1 channel=EMAIL attempts=1
  [log] SUPPRESSED PROMO         user=u1 channel=- attempts=0 reason=rate limit 2 per 60 min
  ...61 minutes later the window has moved:
    SMTP -> asha@mail.com : Sale! | Asha, 50% off today.
  [log] SENT       PROMO         user=u1 channel=EMAIL attempts=1
== 5. Every channel gives up -> FAILED
  [log] FAILED     ORDER_SHIPPED user=u2 channel=- attempts=3 reason=all channels failed, last PUSH: FCM timeout
== 6. No accepted channel for this kind -> SUPPRESSED
  [log] SUPPRESSED PROMO         user=u2 channel=- attempts=0 reason=user accepts no channel
== 7. Template value missing -> FAILED, still recorded through finish()
  [log] FAILED     OTP           user=u1 channel=- attempts=0 reason=template: missing value for {code}
== 8. Async send (caller not blocked)
    FCM -> tok-ben : Order B-3 shipped | Hi Ben, it arrives Monday.
  [log] SENT       ORDER_SHIPPED user=u2 channel=PUSH attempts=1
  future completed with SENT via PUSH
== Metrics: {SENT=7, FAILED=2, SUPPRESSED=3}, history size = 12
```

Step 2 shows Template Method + fallback: 3 failed SMS attempts + 1 successful email = 4 attempts. Step 4 shows that the suppressed third promo did not count: an hour later the window has moved and sending works again.

Concurrency & edge cases

- **Shared state is thread-safe:** users/preferences/templates in `ConcurrentHashMap` (a map built for many threads to read and write at once); listeners and history in `CopyOnWriteArrayList` (a list that copies itself on every write, so reads need no lock: cheap reads, rare writes); the rate-limit deque is locked per key.
- **Check-then-act gap in rate limiting.** (Check-then-act: code first checks a condition, then acts on it, and another thread can change things in between.) Two concurrent (running at the same time) sends for the same user and kind can both pass `blocks()` before either calls `onSent()`, so the limit can be exceeded by the number of concurrent sends. Fixes: reserve a slot atomically at check time and release it if the send fails, or serialize sends per user (run them strictly one after another, e.g. a per-user queue, which also keeps their order).
- **Retries block a worker thread.** With `Thread.sleep` backoff, a slow provider ties up threads. Real systems re-enqueue the message with a delay instead (a "retry topic" — a topic is a named channel that messages are published to — or delayed queue, which hands a message out only after a set delay).
- **Idempotency.** A retry after a timeout (giving up on waiting for a reply after a set time) may deliver twice (the provider may have sent it but the reply was lost). Pass an idempotency key / message id to providers that support it, and make receivers tolerate duplicates.

> **Key term — idempotency:** an operation is **idempotent** when doing it twice has the same effect as doing it once ("set the balance to 10" is; "add 10 to the balance" is not). Retries are only safe on idempotent operations. An **idempotency key** is a unique id sent with a request, so the receiver can recognise a repeat and skip it. More in System Design Fundamentals.

- **Missing address** (user has no phone): the channel returns a failed attempt with 0 tries and the next channel is used.
- **Unknown kind / no template:** FAILED with a reason; **no accepted channel:** SUPPRESSED.

- **Time zones:** quiet hours should use the *user's* time zone; store a `ZoneId` (Java's type for a time zone) in preferences and convert `Instant` (an exact moment on the global timeline) → local hour.

Extensions & follow-up questions

- **"Add WhatsApp."** One `WhatsAppChannel extends NotificationChannel` (3 small methods), one enum value (an enum is a type with a fixed list of named values), one `Gateway` adapter. Nothing else changes.
- **"Someone is getting too many messages."** A new `SendRule` (e.g. a global daily cap, or "dedupe identical messages within 10 minutes"), not a new `if`.
- **"Send to a million users (a campaign)."** Split into a fan-out stage (fan-out: one input turned into many copies or tasks) that expands the audience into individual requests on a queue, and many workers that each run this `send`. Rate-limit per provider too (SMS gateways have throughput limits — **throughput** is how much work gets done per unit of time, e.g. messages per second).
- **"Priorities."** Separate queues per priority (OTP — one-time password, the login-code kind — queue drained first), or a `PriorityBlockingQueue` (a thread-safe queue that always hands out the highest-priority item next).
- **"Localization."** (Localization: adapting text to a user's language and region.) Key templates by (kind, locale — a language-plus-region code such as `fr-CA`) and store the user's locale in preferences.
- **"Delivery receipts / opened tracking."** Providers call back via webhooks (an HTTP request — HTTP being the protocol web clients and servers talk — that the provider sends to a URL you registered, when something happens on their side); record them as further events on the `DeliveryRecord`.
- **"Circuit breaker."** If a provider fails repeatedly, stop calling it for a while (Resilience4j-style; Resilience4j is a Java library of such fault-tolerance tools) and go straight to the next channel. Like an electrical breaker, it "trips" after too many failures and lets calls through again after a pause.

Real-world example

Push notifications on phones go through platform services: **Firebase Cloud Messaging (FCM)** for Android and **Apple Push Notification service (APNs)** for iOS; the app's server sends to a *device token*, exactly the `deviceToken` field above. Email is usually sent through providers such as Amazon SES or SendGrid, and SMS through gateways such as Twilio; each has its own API, which is why the `Gateway` adapter exists. **AWS SNS** is a managed (run for you by the cloud provider) pub-sub that fans one message out to email, SMS, mobile push and queues. LinkedIn's engineering blog describes **Air Traffic Controller (ATC)**, a central service that almost all LinkedIn notifications now pass through and that decides whether, when and how to notify each member, to stop excessive, repetitive notifications: the `SendRule` idea at company scale. And phone "Focus" / "Do Not Disturb" modes are quiet hours with an urgent-bypass list.

In the interview

> **30-second answer:** "Channels sit behind one abstract class whose final `deliver` method holds the retry-with-backoff loop (Template Method); each channel only supplies its address. Each "do not send" condition is its own `SendRule` (Strategy) returning a reason string, so we can record why. The user's preferences give an ordered channel list per kind; we render the template once and try channels in order, falling back on failure. Every outcome goes through one `finish` method that stores a record and notifies listeners (Observer). For scale, a queue sits in front and workers call `send`."

Plan (35–45 min): clarify (5) → entities, User vs Preferences (5) → Template + Message (3) → channel hierarchy + retry (8) → rules (7) → service flow + single exit + listeners (7) → async/queue, idempotency, follow-ups (5).

Follow-ups with answers:

- *"Add a new channel."* → One subclass + gateway adapter.
- *"What about duplicate messages on retry?"* → Idempotency keys; at-least-once delivery (every message arrives, but some may arrive more than once) means receivers must tolerate duplicates.
- *"Why not Strategy for channels?"* → Channels share an algorithm and vary in one step; that is Template Method. (Composition-based alternative: inject a `RetryPolicy` object and a `Sender` Strategy; also valid, mention it.)

Self-check

1. Which pattern puts the retry loop in one place? → Template Method (`final deliver()` in the abstract channel).
2. Why does `SendRule.blocks` return `Optional<String>` (Java's box that holds either one value or nothing)? → So the reason for suppression is recorded.

3. Why should a suppressed notification not count against the rate limit? → The user never received it; counting it would block legitimate later messages.
4. Quiet hours 22–07: is 06:00 quiet? Is 12:00? → Yes; no.
5. Where would you put a queue to avoid blocking checkout? → Between product code and `NotificationService.send` (our `sendAsync` is the in-process version).
6. What must every exit path of `send` do? → Go through `finish()`: store the record and notify listeners.

TL;DR

- **Template Method** for channels: retry/backoff once in the parent; subclasses give the address.
- **Strategy** for send rules (quiet hours, rate limit); rules return the *reason*.
- **Observer** for outcomes; **Simple Factory** for channels; **Adapter** for providers.
- Render once, try channels in the user's order, fall back, single `finish()` exit.
- Scale with a queue in front; watch for idempotency and the rate-limit check-then-act gap.

20.3 Design an In-Memory Cache (and an LRU Cache from scratch)

Problem & clarifying questions

`What it is.` A cache is a small, fast store that keeps recently used values close at hand, so you don't have to fetch them again from a slow place (a database, another service, the disk). (For where caches sit in bigger systems, see System Design Fundamentals.) Everyday analogy: the few books on your desk. The library (database) has everything, but the desk (cache) holds the ones you are using. When the desk is full and you need a new book, one must go back to the shelf. **Which one goes back** is the eviction policy, and it is the rule most likely to change.

Because the question is about **one component**, vague answers are exposed quickly. The interviewer will ask: *which data structure gives O(1) get and put?* (O(1), "constant time", means the time does not grow with the number of entries.)

Question	Assumed answer
Capacity measured in?	Number of entries (not bytes).
Eviction rule?	LRU (least recently used) by default; must be swappable for FIFO or LFU without changing the cache.
Time limits?	Optional TTL (time to live) per entry. Reading an expired entry is a miss and removes it.
Stats?	Hits, misses, evictions, expirations.
Notifications?	Interested code can subscribe to removals and learn the cause.
Thread-safe?	Yes (the course version left it out; we add it).
Load from DB (database) on miss / write-back (save changes to the DB later)?	Out of scope; discussed in extensions.

Requirements

Functional

1. Holds key → value pairs, never more than a fixed number.
2. `get` returns the value or reports a miss.
3. `put` inserts a new key or overwrites the value of an existing one.
4. Inserting a new key into a full cache first evicts one entry.
5. Default rule: **LRU** (least recently used — the entry whose last access is the oldest).
6. Swapping in another rule (FIFO — first in, first out; LFU — least frequently used) needs no change to the cache class.
7. `get` and `put` are O(1).

8. An entry may carry a TTL; once it has passed, a read returns a miss and deletes the entry.
9. Counts hits, misses, evictions, expirations.
10. Registered listeners hear about every removal together with its cause.
11. Callers can invalidate a single key or empty the whole cache.
12. Every entry leaves through **one** code path, whatever the reason.

Non-functional: O(1) operations; thread-safe; testable TTL (injected clock); listeners must not slow down or deadlock the cache (a **deadlock** is when two threads each wait forever for something the other one holds).

Use cases

Actors: **Application** (reads, stores, removes keys), **Operator** (chooses the eviction rule, reads stats), **Cache** (evicts, expires, counts, notifies).

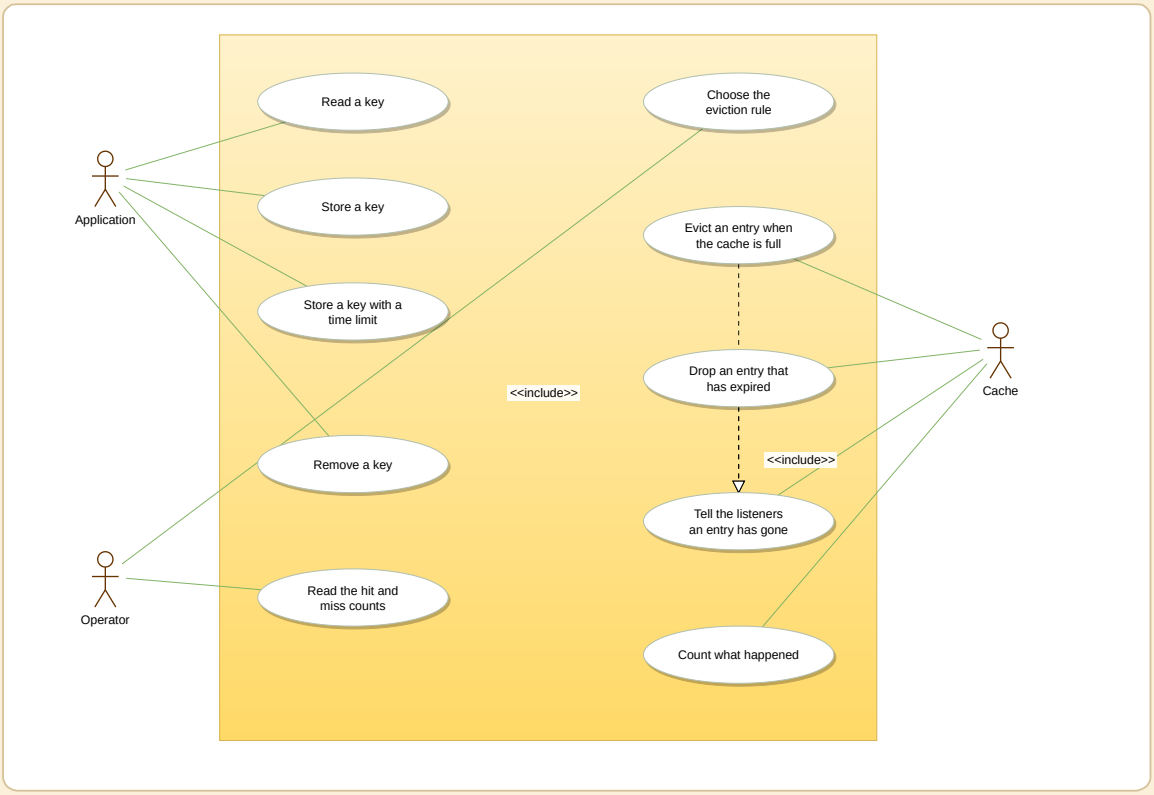


Figure — Cache use cases: read, store (with optional time limit), remove; the cache evicts, drops expired entries and tells listeners (course diagram)

Core entities

Class	Responsibility
`Node<K,V>`	One entry: key, value, `expiresAtMillis`, `frequency`, and its own `prev`/`next` links.
`DoublyLinkedList<K,V>`	Order of entries, newest at the front, with two **sentinel** (dummy) nodes so add/remove never need null checks.
`EvictionPolicy<K,V>` (interface)	**Strategy**: three events (`onInsert`, `onAccess`, `onRemove`) and one question (`victim()`). Owns the *order*.
`LruPolicy`, `FifoPolicy`, `LfuPolicy`	The three rules. LFU is O(1) using a chain of frequency buckets.
`EvictionPolicies` + `PolicyType`	**Simple Factory**: enum → policy.
`Cache<K,V>`	The map + policy + clock + stats + listeners. One lock. **One removal method.**
`Cache.Builder`	**Builder**: capacity, policy, clock with defaults, validated once.
`CacheStats`	Counters (`LongAdder`, a thread-safe counter).
`RemovalReason`	EVICTED, EXPIRED, REMOVED, CLEARED.
`EvictionListener<K,V>`	**Observer**: told key, value and reason.
`ShardedCache<K,V>`	N independent caches picked by key hash, to reduce lock contention.

Key terms — hash map, linked list, hashing, contention. A **hash map** (`HashMap`) stores key → value pairs; it turns each key into a number (its **hash**) that says which slot to look in, so finding a key takes about the same time however many entries there are. A **linked list** is a chain of nodes where each node points to the next one (a **doubly** linked list also points back to the previous one); inserting or unlinking a node you already hold is just a few pointer changes. **Lock contention** is threads queueing for the same lock; splitting data into **shards** (independent pieces, here chosen by key hash), each with its own lock, reduces it.

The core idea: two structures, each answering one question.

Question	Structure	Cost
"Where is key k?"	`HashMap<K, Node>`	O(1)
"Who should leave next?"	Doubly linked list ordered by recency	O(1) at the ends
"Move this entry to the front"	Node has its own `prev`/`next`, and the map hands us the node	O(1) — no search

A map alone cannot tell you the oldest entry. A list alone cannot find a key without walking it. Together they give O(1) for everything. A **doubly** linked list is needed because removing a node from the middle requires its predecessor, and a singly linked list would have to walk to find it.

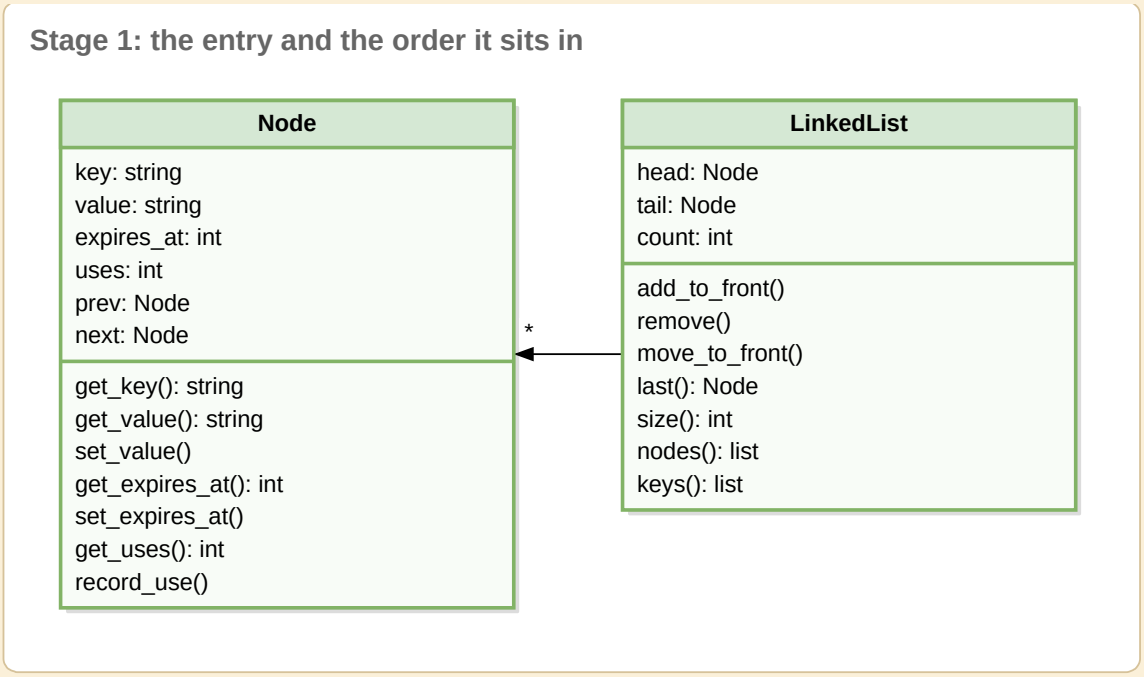


Figure — Stage 1 of the course build: the Node (key, value, expiry, use count, prev and next links) and the linked list that orders the nodes (course diagram)

Class diagram

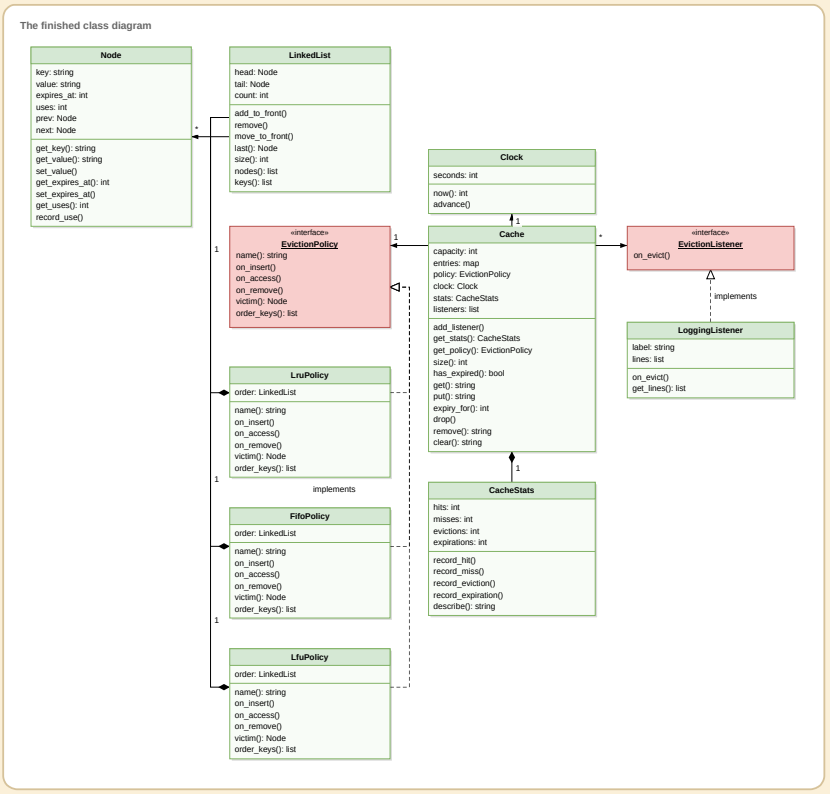
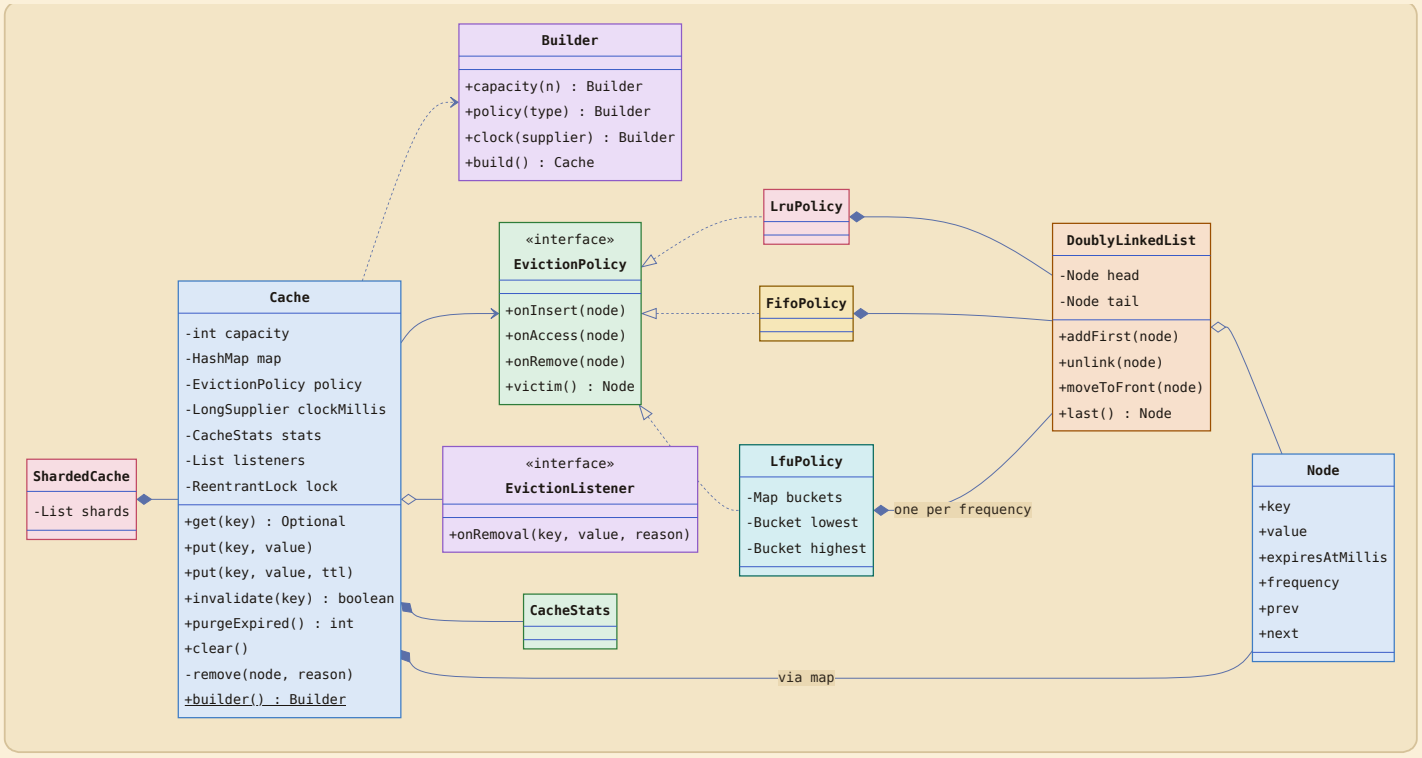


Figure — The finished cache class diagram: Cache with its map, policy, clock, stats and listeners; the three eviction policies behind one interface (course diagram)



Build order (from the course). (1) `Node` + `DoublyLinkedList`: entries can be ordered and moved to the front; (2) `EvictionPolicy` + `LruPolicy`: you can ask who should leave; (3) `FifoPolicy` + `LfuPolicy`: the same question gets three answers with no change to the cache; (4) `Cache` with clock, stats and TTL: reads and writes work; (5) `EvictionListener`: other code can react when an entry goes. Each stage leaves something runnable; reuse this order when you interview.

Who owns what. The cache owns the map (and so the nodes). The policy owns the *order* of those nodes. Neither reaches into the other's structure. That split is exactly what lets us add FIFO and LFU without touching `Cache`.

Key flows

Reading a key — three ways out, only one returns a value:

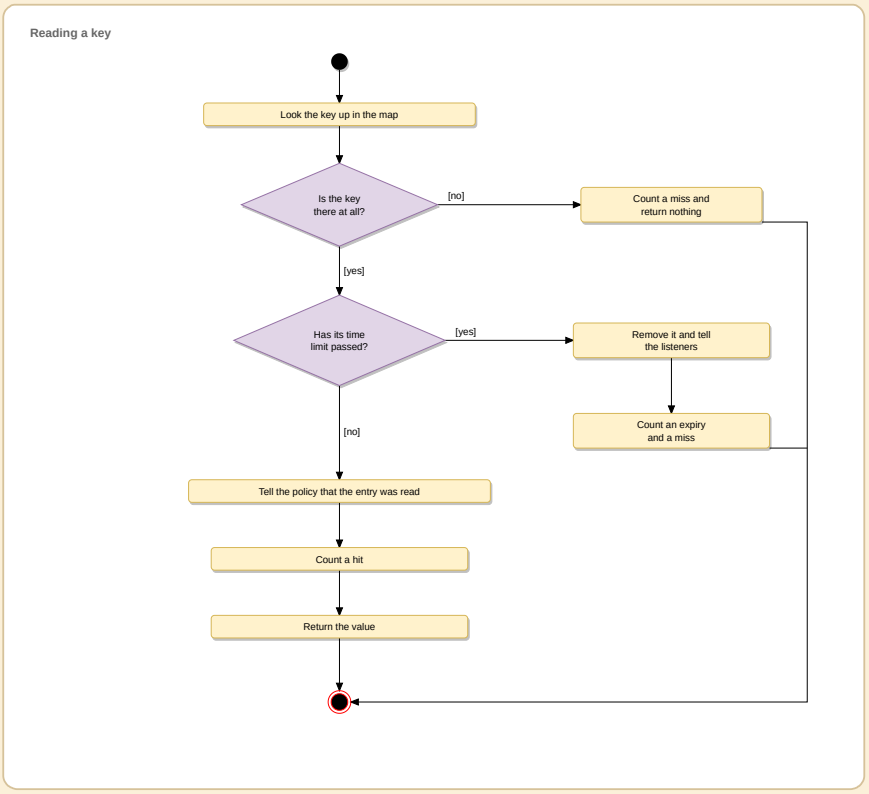


Figure — Reading a key: not present means a miss; present but expired means a miss plus an expiry removal; otherwise a hit, the policy is told, and the value is returned (course diagram)

Storing a key — eviction work happens only when the cache is already full, so a cache with free room pays nothing extra:

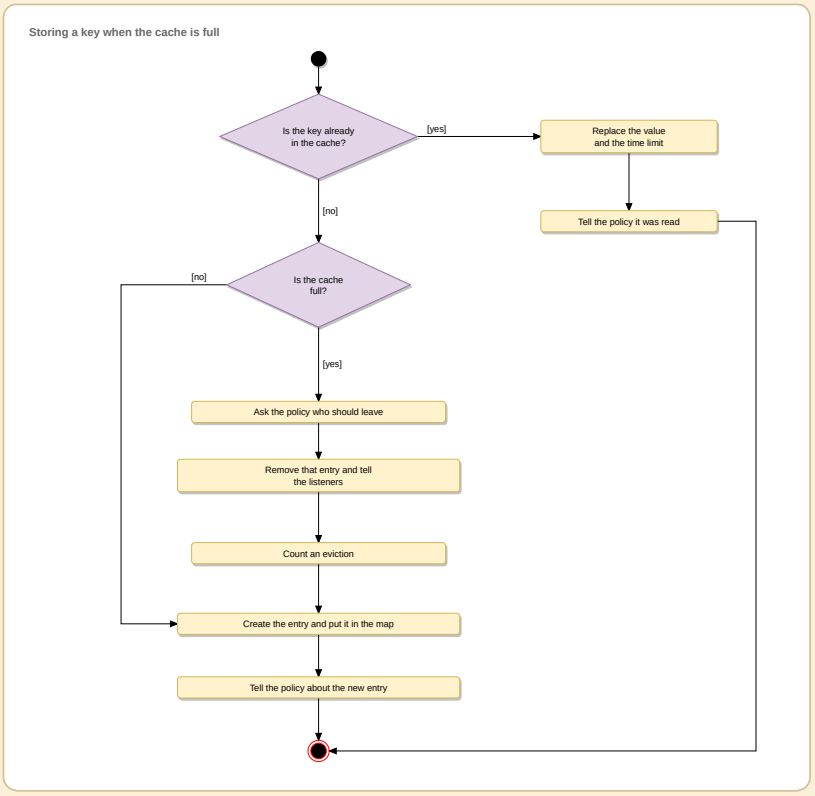
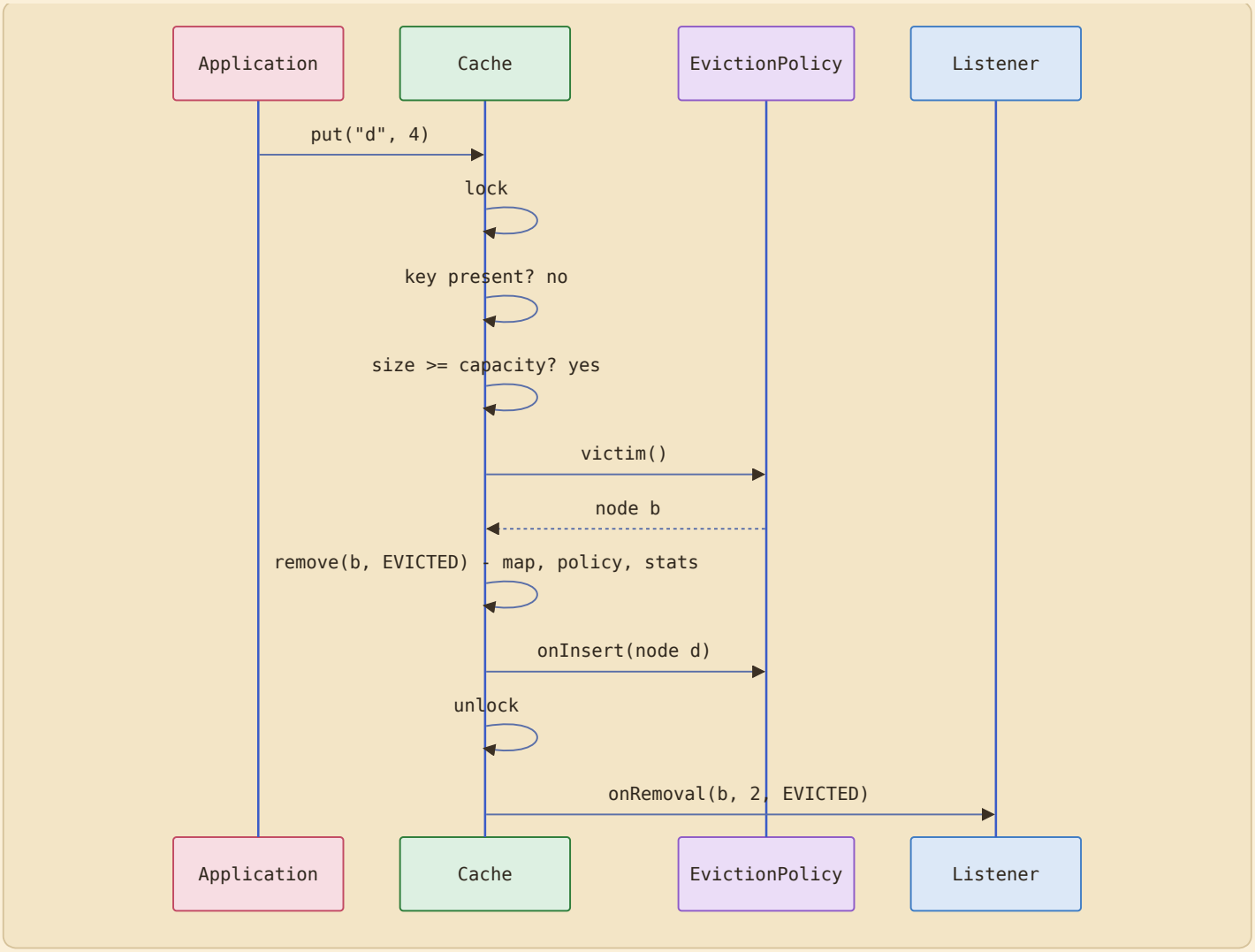


Figure — Storing a key: replace if present; otherwise, if full, ask the policy for a victim and remove it through the single removal path, then insert (course diagram)



How the three policies differ (the only difference is what they do on each event):

Event	LRU	FIFO	LFU (O(1) buckets)
`onInsert`	add to front	add to front	frequency = 1, add to bucket 1 (always first in the chain)
`onAccess`	move to front	**nothing**	move from bucket f to bucket f+1 (created right after f if missing); drop bucket f from the chain if it emptied
`onRemove`	unlink	unlink	unlink; drop the bucket if it emptied
`victim()`	back of the list	back of the list	back of the **first** bucket in the chain (least recently used among the least frequent)

The FIFO policy is LRU with one empty method, which is good evidence that the interface boundaries are well chosen. The course's LFU scanned all entries (O(n): the time grows in step with the number of entries) and named a structure grouping entries by count as the fix; our `LfuPolicy` implements that fix. The buckets are themselves kept in a **doubly linked chain in increasing frequency order**, and a new bucket f+1 is always created right after bucket f, so the chain never needs sorting and the lowest bucket is always the first one. That makes `onInsert`, `onAccess`, `onRemove` and `victim()` all O(1), even after a manual removal or an expiry empties the lowest bucket (this is the O(1) LFU scheme of Shah, Mitra and Matani, 2010). Ties inside the lowest bucket are broken by recency (the back of the bucket), so two runs of the same test always evict the same key.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for eviction	"Who leaves" is the rule most likely to change.	A `switch (policyType)` inside `get` / `put`.
Observer for removals	Logging, metrics, deleting a temp file on eviction — none belong inside the cache. The reason is passed so one listener can handle all cases.	Callbacks (methods handed over to be called later) hard-coded per reason.
One removal method (`remove(node, reason)`)	Eviction, expiry, manual removal and clear all update map, policy, stats and listeners the same way; nothing can be forgotten on one path.	Four copies of the clean-up code.
Sentinel nodes	Add and unlink are always the same four pointer writes (a **pointer** here is a reference from one node to another); no special cases for empty list or ends.	Null checks everywhere (a classic source of bugs).
Injected clock (`LongSupplier`, a function that returns a `long`)	TTL tests run instantly by moving a fake clock forward.	`System.currentTimeMillis()` inside the cache: tests must `sleep`.
Lazy expiry + optional sweeper	Lazy = checked only when needed: expired entries are dropped when read; `purgeExpired()` lets a background task reclaim memory.	Only lazy (memory held by dead entries) or a timer per entry (expensive).
One `ReentrantLock` (Java's explicit lock object), listeners called **after unlock**	In LRU even `get` writes (moves a node), so a read-write lock (one that lets many readers in together but a writer only alone) would not help. Calling listeners outside the lock means a slow or re-entrant listener cannot block or deadlock the cache.	`synchronized` around listener calls.
Builder	Optional settings with defaults; validation once in `build()`.	Telescoping constructors (many constructors, each taking one more parameter).
Sharding (splitting data into independent pieces; `ShardedCache`)	N locks instead of one: threads touching different shards never wait for each other.	One global lock (every reader waits for every writer).

Java implementation

```java
// File: code/lld/ch20/cache/Structures.java
package lld.ch20.cache;

/** One cache entry. It carries its own links, so it can be unlinked in O(1). */
final class Node<K, V> {
    final K key;
    V value;
    long expiresAtMillis;  // 0 = never expires
    int frequency;         // used by LFU
    Node<K, V> prev, next;

    Node(K key, V value, long expiresAtMillis) {
        this.key = key;
        this.value = value;
        this.expiresAtMillis = expiresAtMillis;
    }

    boolean isExpired(long now) { return expiresAtMillis != 0 && now >= expiresAtMillis; }
}

/** Doubly linked list with two sentinel nodes: no null checks at the ends. Newest at the front. */
final class DoublyLinkedList<K, V> {
    private final Node<K, V> head = new Node<>(null, null, 0); // sentinel before the front
    private final Node<K, V> tail = new Node<>(null, null, 0); // sentinel after the back
    private int size;

    DoublyLinkedList() {
        head.next = tail;
        tail.prev = head;
    }

    void addFirst(Node<K, V> n) {
        n.prev = head;
        n.next = head.next;
        head.next.prev = n;
        head.next = n;
        size++;
    }

    void unlink(Node<K, V> n) {
        n.prev.next = n.next;
        n.next.prev = n.prev;
        n.prev = n.next = null;
        size--;
    }

    void moveToFront(Node<K, V> n) {
        unlink(n);
        addFirst(n);
    }

    /** The oldest node (at the back), or null if empty. */
    Node<K, V> last() { return size == 0 ? null : tail.prev; }

    boolean isEmpty() { return size == 0; }
}
```

```java
// File: code/lld/ch20/cache/EvictionPolicy.java
package lld.ch20.cache;

import java.util.HashMap;
import java.util.Map;

/** Strategy: owns the ORDER of entries and answers "who leaves?". The cache owns the map. */
interface EvictionPolicy<K, V> {
    void onInsert(Node<K, V> node);
    void onAccess(Node<K, V> node);
    void onRemove(Node<K, V> node);
    Node<K, V> victim();
}

/** Least recently used: a read moves the entry to the front; the back is the victim. */
final class LruPolicy<K, V> implements EvictionPolicy<K, V> {
    private final DoublyLinkedList<K, V> order = new DoublyLinkedList<>();
    public void onInsert(Node<K, V> n) { order.addFirst(n); }
    public void onAccess(Node<K, V> n) { order.moveToFront(n); }
    public void onRemove(Node<K, V> n) { order.unlink(n); }
    public Node<K, V> victim() { return order.last(); }
}

/** First in, first out: identical to LRU except that a read changes nothing. */
final class FifoPolicy<K, V> implements EvictionPolicy<K, V> {
    private final DoublyLinkedList<K, V> order = new DoublyLinkedList<>();
    public void onInsert(Node<K, V> n) { order.addFirst(n); }
    public void onAccess(Node<K, V> n) { /* insertion order only */ }
    public void onRemove(Node<K, V> n) { order.unlink(n); }
    public Node<K, V> victim() { return order.last(); }
}

/**
 * Least frequently used in O(1): one list per frequency ("bucket"), and the buckets themselves
 * chained in increasing frequency order, so the lowest non-empty bucket is always the first one.
 * Ties inside the lowest bucket go to the least recently used entry (the back of that bucket).
 */
final class LfuPolicy<K, V> implements EvictionPolicy<K, V> {
    private static final class Bucket<K, V> {
        final int frequency;
        final DoublyLinkedList<K, V> entries = new DoublyLinkedList<>();
        Bucket<K, V> lower, higher;           // neighbours in the frequency chain
        Bucket(int frequency) { this.frequency = frequency; }
    }

    private final Map<Integer, Bucket<K, V>> buckets = new HashMap<>();
    private final Bucket<K, V> lowest = new Bucket<>(0);   // sentinels of the bucket chain
    private final Bucket<K, V> highest = new Bucket<>(Integer.MAX_VALUE);

    LfuPolicy() {
        lowest.higher = highest;
        highest.lower = lowest;
    }

    public void onInsert(Node<K, V> n) {
        n.frequency = 1;
        bucketAfter(lowest, 1).entries.addFirst(n);        // frequency 1 is always the lowest
    }

    public void onAccess(Node<K, V> n) {
        Bucket<K, V> from = buckets.get(n.frequency);
        Bucket<K, V> to = bucketAfter(from, n.frequency + 1);
        from.entries.unlink(n);
        dropIfEmpty(from);
        n.frequency++;
        to.entries.addFirst(n);
    }
```

```
    public void onRemove(Node<K, V> n) {
        Bucket<K, V> b = buckets.get(n.frequency);
        b.entries.unlink(n);
        dropIfEmpty(b);                                  // the chain stays ordered: nothing to repair
    }

    public Node<K, V> victim() {
        Bucket<K, V> min = lowest.higher;                // O(1): first bucket in the chain
        return min == highest ? null : min.entries.last();
    }

    /** The bucket for 'frequency', created right after 'prev' if it does not exist yet. */
    private Bucket<K, V> bucketAfter(Bucket<K, V> prev, int frequency) {
        Bucket<K, V> b = buckets.get(frequency);
        if (b != null) return b;
        b = new Bucket<>(frequency);
        b.lower = prev;
        b.higher = prev.higher;
        prev.higher.lower = b;
        prev.higher = b;
        buckets.put(frequency, b);
        return b;
    }

    private void dropIfEmpty(Bucket<K, V> b) {
        if (!b.entries.isEmpty()) return;
        b.lower.higher = b.higher;
        b.higher.lower = b.lower;
        buckets.remove(b.frequency);
    }
}

enum PolicyType { LRU, FIFO, LFU }

/** Factory: maps a configuration value to a policy object. */
final class EvictionPolicies {
    private EvictionPolicies() {}

    static <K, V> EvictionPolicy<K, V> create(PolicyType type) {
        return switch (type) {
            case LRU -> new LruPolicy<>();
            case FIFO -> new FifoPolicy<>();
            case LFU -> new LfuPolicy<>();
        };
    }
}
```

```java
// File: code/lld/ch20/cache/Cache.java
package lld.ch20.cache;

import java.time.Duration;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.LongAdder;
import java.util.concurrent.locks.ReentrantLock;
import java.util.function.LongSupplier;

enum RemovalReason { EVICTED, EXPIRED, REMOVED, CLEARED }

/** Observer: told every time an entry leaves, and why. */
interface EvictionListener<K, V> {
    void onRemoval(K key, V value, RemovalReason reason);
}

/** Hit / miss / eviction / expiry counters. LongAdder is cheap under contention. */
final class CacheStats {
    final LongAdder hits = new LongAdder(), misses = new LongAdder(),
            evictions = new LongAdder(), expirations = new LongAdder();

    @Override
    public String toString() {
        long h = hits.sum(), m = misses.sum();
        return "hits=" + h + " misses=" + m + " evictions=" + evictions.sum() + " expirations=" + expirations.sum()
                + String.format(" hitRate=%.0f%%", h + m == 0 ? 0.0 : 100.0 * h / (h + m));
    }
}

/**
 * Fixed-capacity cache: HashMap for "where is it?", an EvictionPolicy for "who leaves?".
 * Thread-safe with one lock (a read in LRU also writes: it moves the node).
 */
final class Cache<K, V> {
    private final int capacity;
    private final Map<K, Node<K, V>> map = new HashMap<>();
    private final EvictionPolicy<K, V> policy;
    private final LongSupplier clockMillis;
    private final CacheStats stats = new CacheStats();
    private final List<EvictionListener<K, V>> listeners = new CopyOnWriteArrayList<>();
    private final ReentrantLock lock = new ReentrantLock();

    private record Removal<K, V>(K key, V value, RemovalReason reason) {}

    private Cache(Builder<K, V> b) {
        this.capacity = b.capacity;
        this.policy = EvictionPolicies.create(b.policyType);
        this.clockMillis = b.clockMillis;
    }

    static <K, V> Builder<K, V> builder() { return new Builder<>(); }

    Optional<V> get(K key) {
        List<Removal<K, V>> removed = new ArrayList<>(1);
        lock.lock();
        try {
            Node<K, V> n = map.get(key);
            if (n == null) {
                stats.misses.increment();
                return Optional.empty();
            }
            if (n.isExpired(clockMillis.getAsLong())) { // a miss, an expiry and a removal at once
                stats.misses.increment();
```

```
                removed.add(remove(n, RemovalReason.EXPIRED));
                return Optional.empty();
            }
            stats.hits.increment();
            policy.onAccess(n);
            return Optional.of(n.value);
        } finally {
            lock.unlock();
            notifyListeners(removed); // outside the lock: a slow listener cannot block the cache
        }
    }

    void put(K key, V value) { put(key, value, null); }

    void put(K key, V value, Duration ttl) {
        List<Removal<K, V>> removed = new ArrayList<>(1);
        lock.lock();
        try {
            long expiresAt = ttl == null ? 0 : clockMillis.getAsLong() + ttl.toMillis();
            Node<K, V> existing = map.get(key);
            if (existing != null) {          // replace: counts as a use
                existing.value = value;
                existing.expiresAtMillis = expiresAt;
                policy.onAccess(existing);
                return;
            }
            if (map.size() >= capacity) removed.add(remove(policy.victim(), RemovalReason.EVICTED));
            Node<K, V> n = new Node<>(key, value, expiresAt);
            map.put(key, n);
            policy.onInsert(n);
        } finally {
            lock.unlock();
            notifyListeners(removed);
        }
    }

    boolean invalidate(K key) {
        List<Removal<K, V>> removed = new ArrayList<>(1);
        lock.lock();
        try {
            Node<K, V> n = map.get(key);
            if (n != null) removed.add(remove(n, RemovalReason.REMOVED));
            return n != null;
        } finally {
            lock.unlock();
            notifyListeners(removed);
        }
    }

    /** Called by a background sweeper (e.g. a ScheduledExecutorService) to reclaim memory early. */
    int purgeExpired() {
        List<Removal<K, V>> removed = new ArrayList<>();
        lock.lock();
        try {
            long now = clockMillis.getAsLong();
            for (Node<K, V> n : new ArrayList<>(map.values()))
                if (n.isExpired(now)) removed.add(remove(n, RemovalReason.EXPIRED));
            return removed.size();
        } finally {
            lock.unlock();
            notifyListeners(removed);
        }
    }

    void clear() {
        List<Removal<K, V>> removed = new ArrayList<>();
        lock.lock();
        try {
            for (Node<K, V> n : new ArrayList<>(map.values())) removed.add(remove(n, RemovalReason.CLEARED));
```

```
        } finally {
            lock.unlock();
            notifyListeners(removed);
        }
    }

    /** THE single exit path for every entry, whatever the reason. Caller holds the lock. */
    private Removal<K, V> remove(Node<K, V> n, RemovalReason reason) {
        map.remove(n.key);
        policy.onRemove(n);
        if (reason == RemovalReason.EVICTED) stats.evictions.increment();
        if (reason == RemovalReason.EXPIRED) stats.expirations.increment();
        return new Removal<>(n.key, n.value, reason);
    }

    private void notifyListeners(List<Removal<K, V>> removed) {
        for (Removal<K, V> r : removed)
            for (EvictionListener<K, V> l : listeners) l.onRemoval(r.key(), r.value(), r.reason());
    }

    void addListener(EvictionListener<K, V> l) { listeners.add(l); }
    CacheStats stats() { return stats; }

    int size() {
        lock.lock();
        try { return map.size(); } finally { lock.unlock(); }
    }

    /** Builder: optional settings with sensible defaults, validated once. */
    static final class Builder<K, V> {
        private int capacity = 100;
        private PolicyType policyType = PolicyType.LRU;
        private LongSupplier clockMillis = System::currentTimeMillis;

        Builder<K, V> capacity(int c) { this.capacity = c; return this; }
        Builder<K, V> policy(PolicyType p) { this.policyType = p; return this; }
        Builder<K, V> clock(LongSupplier c) { this.clockMillis = c; return this; }

        Cache<K, V> build() {
            if (capacity <= 0) throw new IllegalArgumentException("capacity must be positive");
            return new Cache<>(this);
        }
    }
}

/** Sharding: N independent caches chosen by key hash, so threads rarely share a lock. */
final class ShardedCache<K, V> {
    private final List<Cache<K, V>> shards = new ArrayList<>();

    ShardedCache(int shardCount, int capacityPerShard, PolicyType type) {
        for (int i = 0; i < shardCount; i++)
            shards.add(Cache.<K, V>builder().capacity(capacityPerShard).policy(type).build());
    }

    private Cache<K, V> shard(K key) { return shards.get(Math.floorMod(key.hashCode(), shards.size())); }
    Optional<V> get(K key) { return shard(key).get(key); }
    void put(K key, V value) { shard(key).put(key, value); }
    int size() { return shards.stream().mapToInt(Cache::size).sum(); }
}
```

```java
// File: code/lld/ch20/cache/Main.java
package lld.ch20.cache;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicLong;

public class Main {
    public static void main(String[] args) throws Exception {
        for (PolicyType type : PolicyType.values()) {
            System.out.println("== " + type + " (capacity 3): put a,b,c; get b,b,b,c,a,a; put d");
            Cache<String, Integer> c = Cache.<String, Integer>builder().capacity(3).policy(type).build();
            c.addListener((k, v, why) -> System.out.println("  listener: " + k + "=" + v + " left, " + why));
            c.put("a", 1);
            c.put("b", 2);
            c.put("c", 3);
            for (String k : List.of("b", "b", "b", "c", "a", "a")) c.get(k);
            c.put("d", 4);
            List<String> present = new ArrayList<>();
            for (String k : List.of("a", "b", "c", "d")) if (c.get(k).isPresent()) present.add(k);
            System.out.println("  still cached: " + present);
        }

        System.out.println("== TTL with a manual clock");
        AtomicLong now = new AtomicLong(0);
        Cache<String, String> sessions = Cache.<String, String>builder().capacity(10).clock(now::get).build();
        sessions.addListener((k, v, why) -> System.out.println("  listener: " + k + " left, " + why));
        sessions.put("session:42", "alice", Duration.ofSeconds(5));
        sessions.put("session:43", "bob", Duration.ofSeconds(5));
        sessions.put("config", "v1");
        System.out.println("  t=0s  get session:42 -> " + sessions.get("session:42").orElse("MISS"));
        now.set(6_000);
        System.out.println("  t=6s  get session:42 -> " + sessions.get("session:42").orElse("MISS"));
        System.out.println("  sweeper purged " + sessions.purgeExpired() + " more expired entry");
        System.out.println("  config (no TTL) -> " + sessions.get("config").orElse("MISS"));
        sessions.invalidate("config");
        System.out.println("  stats: " + sessions.stats());

        System.out.println("== Sharded cache under 8 threads");
        ShardedCache<Integer, Integer> sharded = new ShardedCache<>(8, 1_000, PolicyType.LRU);
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int t = 0; t < 8; t++) {
            int seed = t;
            pool.submit(() -> {
                for (int i = 0; i < 50_000; i++) {
                    int key = (i * 31 + seed) % 20_000;
                    if (sharded.get(key).isEmpty()) sharded.put(key, key * 2);
                }
            });
        }
        pool.shutdown();
        System.out.println("  finished: " + pool.awaitTermination(30, TimeUnit.SECONDS)
                + ", size <= 8000: " + (sharded.size() <= 8_000));
    }
}
```

Walking through the code.

- `DoublyLinkedList` never touches `null` at the ends because `head` and `tail` are dummy nodes that are never removed.
- In `Cache.get`, an expired entry counts as a miss **and** an expiration, and is removed through `remove()` — all three in one place, so the counters always add up.

- Every public method collects `Removal` events while holding the lock and calls `notifyListeners` in `finally` (a block that always runs, even when an exception is thrown) **after** `unlock()`.
- `put` on an existing key replaces value and TTL and counts as an access (in LRU the key moves to the front).
- `CacheStats` uses `LongAdder`, which is faster than `AtomicLong` (a `long` that threads can update safely without a lock) when many threads increment.

Output:

```
== LRU (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: b=2 left, EVICTED
  still cached: [a, c, d]
== FIFO (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: a=1 left, EVICTED
  still cached: [b, c, d]
== LFU (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: c=3 left, EVICTED
  still cached: [a, b, d]
== TTL with a manual clock
  t=0s  get session:42 -> alice
  listener: session:42 left, EXPIRED
  t=6s  get session:42 -> MISS
  listener: session:43 left, EXPIRED
  sweeper purged 1 more expired entry
  config (no TTL) -> v1
  listener: config left, REMOVED
  stats: hits=2 misses=1 evictions=0 expirations=2 hitRate=67%
== Sharded cache under 8 threads
  finished: true, size <= 8000: true
```

With the same access pattern (b read 3 times, then c once, then a twice), the three policies evict three **different** keys: LRU evicts b (least recently touched), FIFO evicts a (inserted first), LFU evicts c (read only once).

LRU Cache from scratch (the classic interview question)

Very often the interviewer skips the pluggable design and asks for exactly this: **"Implement an LRU cache with O(1) `get` and `put`"** (LeetCode 146; LeetCode is a popular website of coding interview problems). The answer is the same two structures, written inline:

1. A `HashMap<K, Node>` for lookup.
2. A doubly linked list with `head`/`tail` sentinels; most recent right after `head`, least recent right before `tail`.
3. `get(k)`: look up the node; if found, unlink it and add it at the front; return the value.
4. `put(k, v)`: if present, update and move to front. Otherwise, if full, remove `tail.prev` from the list **and** from the map (that is why the node stores its key); then add the new node at the front and to the map.

Java also has a shortcut: `LinkedHashMap` with `accessOrder = true` keeps entries in access order, and overriding `removeEldestEntry` evicts automatically. Mention it, but write the manual version when asked "from scratch" — that is what is being tested.

```java
// File: code/lld/ch20/lrucache/Main.java
package lld.ch20.lrucache;

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;

public class Main {

    /** The classic interview answer: HashMap + hand-written doubly linked list. get/put are O(1). */
    static final class LRUCache<K, V> {
        private final class Node {
            K key;
            V value;
            Node prev, next;
            Node(K key, V value) { this.key = key; this.value = value; }
        }

        private final int capacity;
        private final Map<K, Node> index = new HashMap<>();
        private final Node head = new Node(null, null); // sentinel: most recent side
        private final Node tail = new Node(null, null); // sentinel: least recent side

        LRUCache(int capacity) {
            if (capacity <= 0) throw new IllegalArgumentException("capacity must be positive");
            this.capacity = capacity;
            head.next = tail;
            tail.prev = head;
        }

        synchronized V get(K key) {
            Node n = index.get(key);
            if (n == null) return null;
            unlink(n);
            addFront(n);                // now the most recently used
            return n.value;
        }

        synchronized void put(K key, V value) {
            Node n = index.get(key);
            if (n != null) {            // update + mark as recently used
                n.value = value;
                unlink(n);
                addFront(n);
                return;
            }
            if (index.size() == capacity) {
                Node lru = tail.prev;   // least recently used lives just before the tail sentinel
                unlink(lru);
                index.remove(lru.key);
            }
            n = new Node(key, value);
            addFront(n);
            index.put(key, n);
        }

        private void addFront(Node n) {
            n.prev = head;
            n.next = head.next;
            head.next.prev = n;
            head.next = n;
        }

        private void unlink(Node n) {
            n.prev.next = n.next;
            n.next.prev = n.prev;
        }
```

```
        @Override
        public synchronized String toString() {
            StringBuilder sb = new StringBuilder("[");
            for (Node n = head.next; n != tail; n = n.next) sb.append(n.key).append(n.next == tail ? "" : ", ");
            return sb.append("] (most -> least recent)").toString();
        }
    }

    /** The JDK shortcut: LinkedHashMap in access order + removeEldestEntry. Not thread-safe. */
    static <K, V> Map<K, V> jdkLru(int capacity) {
        return new LinkedHashMap<>(16, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> eldest) { return size() > capacity; }
        };
    }

    public static void main(String[] args) {
        System.out.println("== Hand-written LRU (LeetCode 146 sequence, capacity 2)");
        LRUCache<Integer, Integer> c = new LRUCache<>(2);
        c.put(1, 1);
        c.put(2, 2);
        System.out.println("get(1) = " + c.get(1) + "   order " + c);
        c.put(3, 3);
        System.out.println("put(3) evicts 2        order " + c);
        System.out.println("get(2) = " + c.get(2));
        c.put(4, 4);
        System.out.println("put(4) evicts 1        order " + c);
        System.out.println("get(1) = " + c.get(1) + ", get(3) = " + c.get(3) + ", get(4) = " + c.get(4));

        System.out.println("== Same sequence with LinkedHashMap(accessOrder=true)");
        Map<Integer, Integer> j = jdkLru(2);
        j.put(1, 1);
        j.put(2, 2);
        j.get(1);
        j.put(3, 3);
        j.put(4, 4);
        System.out.println("keys (least -> most recent) = " + j.keySet());
    }
}
```

Output:

```
== Hand-written LRU (LeetCode 146 sequence, capacity 2)
get(1) = 1   order [1, 2] (most -> least recent)
put(3) evicts 2        order [3, 1] (most -> least recent)
get(2) = null
put(4) evicts 1        order [4, 3] (most -> least recent)
get(1) = null, get(3) = 3, get(4) = 4
== Same sequence with LinkedHashMap(accessOrder=true)
keys (least -> most recent) = [3, 4]
```

The printed values match LeetCode 146's expected answers for this sequence (1, miss, miss, 3, 4); LeetCode's `int` version returns `-1` for a miss, while this generic version returns `null`.

Approach	get / put	Thread-safe	When to use
HashMap + own doubly linked list	O(1) / O(1)	Only with a lock (`synchronized` here)	Interviews; when you need custom behaviour
`LinkedHashMap(accessOrder=true)` + `removeEldestEntry`	O(1) / O(1)	No (wrap with a lock)	Quick, small, single-threaded caches
Caffeine / Guava cache (widely used Java caching libraries)	amortized O(1) (O(1) on average over many calls)	Yes, highly concurrent	Production Java

Concurrency & edge cases

- **Why a plain read lock is not enough:** an LRU `get` moves the node (several pointer writes). Two threads doing that at once corrupt the list. So `get` needs the exclusive lock.
- **Contention:** one lock serializes everything. `ShardedCache` splits keys over 8 caches (8 locks); the demo hammers it from 8 threads and the size never exceeds the total capacity. Trade-off: eviction is per shard, so the global LRU order is approximate.
- **Listeners:** called outside the lock; a listener that calls back into the cache cannot deadlock.
- **Cache stampede** (many threads miss the same key at once and all hit the database): use a "loading" cache where the first thread computes and the others wait for its result (`ConcurrentHashMap.compute` / `computeIfAbsent` — `computeIfAbsent` creates and stores a value only if the key is missing, as one atomic step — or a `CompletableFuture` stored per key). Chapter 13 (§13.6) builds a TTL cache whose `getOrLoad` does this with `ConcurrentHashMap.compute`.
- **TTL semantics, precisely.** An entry expires *at* its deadline (`now >= expiresAt`), so a 5-second TTL written at t=0 is already a miss at t=5.000 s. Expiry is **lazy**: a dead entry still occupies a slot until it is read, swept by `purgeExpired()`, or picked as the eviction victim (in which case it is reported as `EVICTED`, not `EXPIRED`, because it left to make room). `put(key, value)` without a TTL on an existing key **clears** the old TTL (the entry becomes permanent); pass the TTL again to keep one. A production cache would also let the sweeper run on a `ScheduledExecutorService` (Java's tool for running tasks on a timer).
- **Capacity 0 or negative:** refused by the builder.
- **Null keys/values:** not supported here; say so (Caffeine forbids nulls too).
- **Clock going backwards:** use a monotonic clock (one that only ever moves forward, unlike wall-clock time that can be adjusted; `System.nanoTime`) for TTL in production.

Extensions & follow-up questions

- **"Make it LFU."** One new policy class (done). Volunteer that the naive LFU (scan for the lowest count) is O(n) and the chained-bucket version is O(1).
- **"Size in bytes, not entries."** Add a `Weigher<K,V>` strategy; evict until `totalWeight <= maxWeight`.
- **"Load from the database on a miss."** *Read-through*: `get(key, loader)` calls the loader on a miss and stores the result. *Write-through*: writes go to cache and DB together. *Write-back (write-behind)*: writes go to cache and are flushed to DB later (faster, risk of loss). *Cache-aside*: the application does "check cache → on miss read DB → put in cache" itself.
- **"Distributed cache."** Consistent hashing (a way to assign keys to servers so that adding or removing a server moves only a small share of the keys) to spread keys over nodes (Redis Cluster, Memcached clients — Redis and Memcached are in-memory key-value stores that run as separate servers); that is high-level design (HLD: designing services, databases and networks, as opposed to the classes inside one program).
- **"Which policy is best?"** Depends on access patterns. Modern caches (Caffeine's *Window TinyLFU*) combine recency and frequency and beat pure LRU on many workloads (patterns of real requests).

Real-world example

`java.util.LinkedHashMap` with access order is the JDK's (Java Development Kit: Java's compiler plus standard library) built-in LRU. **Guava's** `CacheBuilder` and **Caffeine** expose exactly this chapter's knobs: `maximumSize` (capacity), `expireAfterWrite` / `expireAfterAccess` (TTL), `removalListener` (Observer, with a cause such as `EXPIRED`, `SIZE`, `EXPLICIT`, `REPLACED`), and `recordStats()` (hit rate: the share of reads answered from the cache). **Redis** supports eviction policies selected by configuration (`maxmemory-policy`: `allkeys-lru`, `allkeys-lfu`, `volatile-ttl`, `noeviction` and others) — a Strategy chosen by config — and approximates LRU by sampling a few keys rather than keeping a full list, to save memory. Browsers, CDNs (content delivery networks: servers spread around the world that keep copies of files close to users) and operating-system page caches (recently read disk data kept in memory) all use variants of LRU.

In the interview

30-second answer: "A HashMap from key to node for O(1) lookup, and a doubly linked list with sentinels for O(1) ordering; the node carries its own links so the map hands us something we can unlink directly. The eviction rule goes behind an `EvictionPolicy` interface — LRU moves on access, FIFO doesn't, LFU uses frequency buckets for O(1). Every exit goes through one `remove` method that updates stats and notifies

> listeners after the lock is released. For concurrency, one lock is correct; sharding by key hash reduces contention."

Plan: sketch the map and the list together straight away and explain what each one lacks on its own → write LRU get/put → pull the rule behind an interface unprompted → TTL + stats + listeners → thread safety and sharding → follow-ups.

Follow-ups with answers:

- *"Why doubly and not singly linked?"* → Removing a node needs its predecessor; with `prev` we have it in O(1).
- *"Why does the node store the key?"* → When evicting the tail we must also delete it from the map.
- *"Make it thread-safe."* → Lock around each operation; better, shard by key; production: Caffeine.

Self-check

1. Which two structures give O(1) LRU? → HashMap (key → node) + doubly linked list (recency order).
2. What do sentinel nodes buy you? → No null checks; add/remove are always the same pointer writes.
3. How does FIFO differ from LRU in code? → `onAccess` does nothing.
4. Why call listeners after releasing the lock? → A slow or re-entrant listener cannot block or deadlock the cache.
5. Why can't a read-write lock speed up an LRU cache's `get`? → `get` modifies the list, so it is a write.
6. What is a cache stampede and one fix? → Many concurrent misses for one key hitting the DB; let one thread load while others wait on the same future.

TL;DR

- **HashMap + doubly linked list (with sentinels)** = O(1) get/put.
- **Strategy** for eviction (LRU / FIFO / O(1) LFU); **Observer** for removals; **Builder** for config.
- **One removal path** keeps stats and listeners correct; TTL is lazy + optional sweeper; clock injected.
- Thread safety: one lock (gets are writes), listeners outside the lock, **shard** to scale.
- Know `LinkedHashMap(accessOrder=true)` + `removeEldestEntry`, and Caffeine/Guava/Redis policies.

20.4 Design a Ride Sharing Service (Uber)

Problem & clarifying questions

What it is. A ride sharing service connects a rider who wants a car with a nearby driver. Everyday analogy: a taxi dispatcher with a radio. A call comes in, the dispatcher looks at the map, radios the nearest free cab; if that driver says no, the next one. At the end the meter shows the fare.

Two decisions are the whole question: **which driver is asked** (matching) and **what the trip costs** (pricing). The rest is bookkeeping. A good design lets either rule change with no edits anywhere else.

Question	Assumed answer
Vehicle types?	Economy, premium, van. Rider asks for one type.
Who can get a request?	Drivers who are online, not on a trip, with the right car type, within range.
Offer to one driver or broadcast?	One at a time, nearest first; decline → next; nobody left → cancel.
Trip states?	Requested → accepted → started → ended; canceled from requested/accepted only. Illegal actions do nothing.
Fare?	Base + per km + per minute, never below a minimum; per vehicle type.
Surge?	A multiplier when the city is busy, applied to the same fare.
Ratings?	Rider and driver rate each other once after a finished trip; accounts show the average.
Payments?	Charge the rider's payment method (cash, wallet, card) at the end.
Map?	A grid; distance measured along streets (Manhattan distance), which keeps all distances integral (whole numbers). Production systems query a routing engine (a service that works out the road route between two points) that gives back road distance plus an estimated duration, and pricing would take both from it.
Pool/shared rides, scheduled rides, live tracking, payouts?	Out of scope.

Requirements

Functional

1. Riders and drivers have a name and phone number.
2. A driver has one vehicle: plate, seats, type.
3. Drivers go online/offline; only online, free drivers get requests.
4. A rider requests a trip with pickup, drop and car type.
5. Free drivers with a matching car type are ordered by distance to the pickup, and the closest one is offered the trip first.
6. A driver accepts or declines; a decline goes to the next driver; if none are left, the request is canceled.
7. A trip moves through fixed states; an action that doesn't fit the current state does nothing.
8. Fare = base + per-km + per-minute, never below a minimum.
9. During peak demand, a surge multiplier scales that same fare.
10. Riders may cancel until the trip starts; after that, the only way out is ending it.
11. After a finished trip, rider and driver rate each other; accounts show the average.
12. Riders and drivers can see their finished trips.
13. (Added) The rider is charged through their payment method.

Non-functional: a driver can never be assigned to two trips at once (even with concurrent requests); rules swappable without editing the service; deterministic behaviour for tests (tie-breaks).

Use cases

Actors: **Rider** (request, cancel, rate), **Driver** (go online, accept/decline, start/end, rate), **System** (find nearby drivers, offer, price, surge, history).

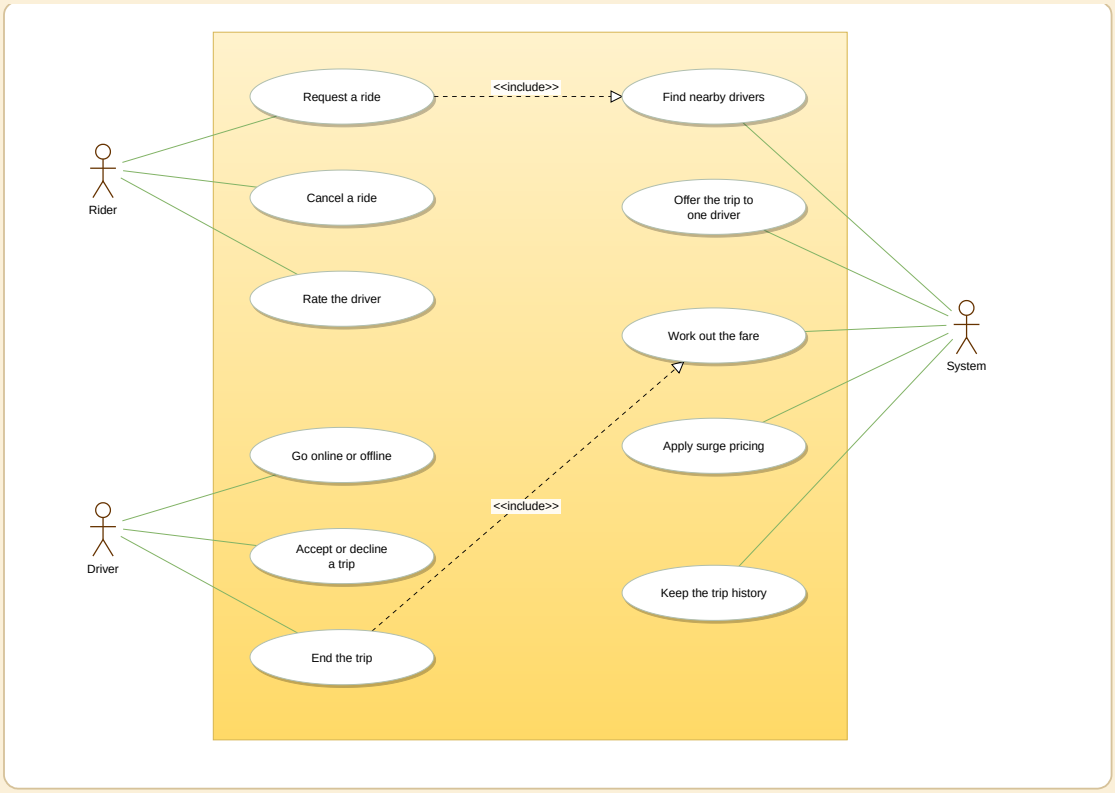


Figure — Ride sharing use cases: rider requests, cancels and rates; driver goes online, accepts or declines and ends the trip; the system matches, offers, prices and applies surge (course diagram)

Core entities

Class	Responsibility
`Location` (record)	Grid point; `distanceTo` = Manhattan distance in km.
`VehicleType` (enum), `Vehicle` (record)	Car type and details.
`Account` (abstract)	What riders and drivers share: id, name, phone, **rating as running total + count**, finished trips.
`Rider`	+ payment method.
`Driver`	+ vehicle, location, online flag, **atomic busy flag** (`tryClaim`).
`TripStatus` (enum)	The state machine: legal next states per state.
`Trip`	One journey: rider, pickup, drop, type, **ranked candidate list + cursor** (a marker for the current position in the list), status, driver, minutes, fare. Every action checks state first.
`PricingStrategy`	**Strategy**: fare for (type, km, minutes).
`StandardPricing` + `RateCard`	Base + per km + per minute, with a floor (a minimum fare), per vehicle type.
`SurgePricing`	**Decorator**: wraps any `PricingStrategy` and multiplies its answer.
`MatchingStrategy`	**Strategy**: ranks drivers. `NearestDriverMatcher`, `BestRatedMatcher`.
`PaymentMethod`	**Strategy**: `CashPayment`, `WalletPayment`.
`TripListener`	**Observer**: apps are told about trip events.
`RideSharingService`	Facade: owns accounts and trips; holds one pricing and one matching rule; knows only the order of steps.

Why the rating is a total and a count. Only the average is ever displayed, so a sum and a count suffice, and recording a new rating is O(1) whether the account has 10 trips or 10,000.

Why `Rider` and `Driver` extend `Account`. They pass the "is-a" substitution test (Liskov Substitution Principle): wherever the code reads a name or records a rating, a `Rider` or a `Driver` can be passed.

Class diagram

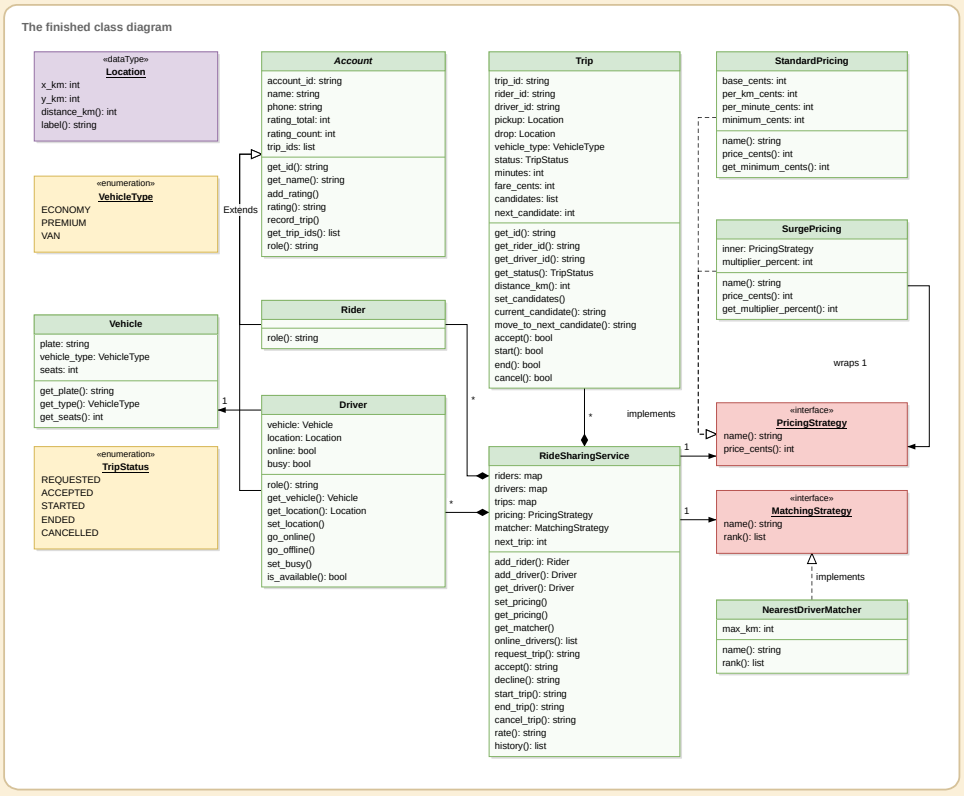
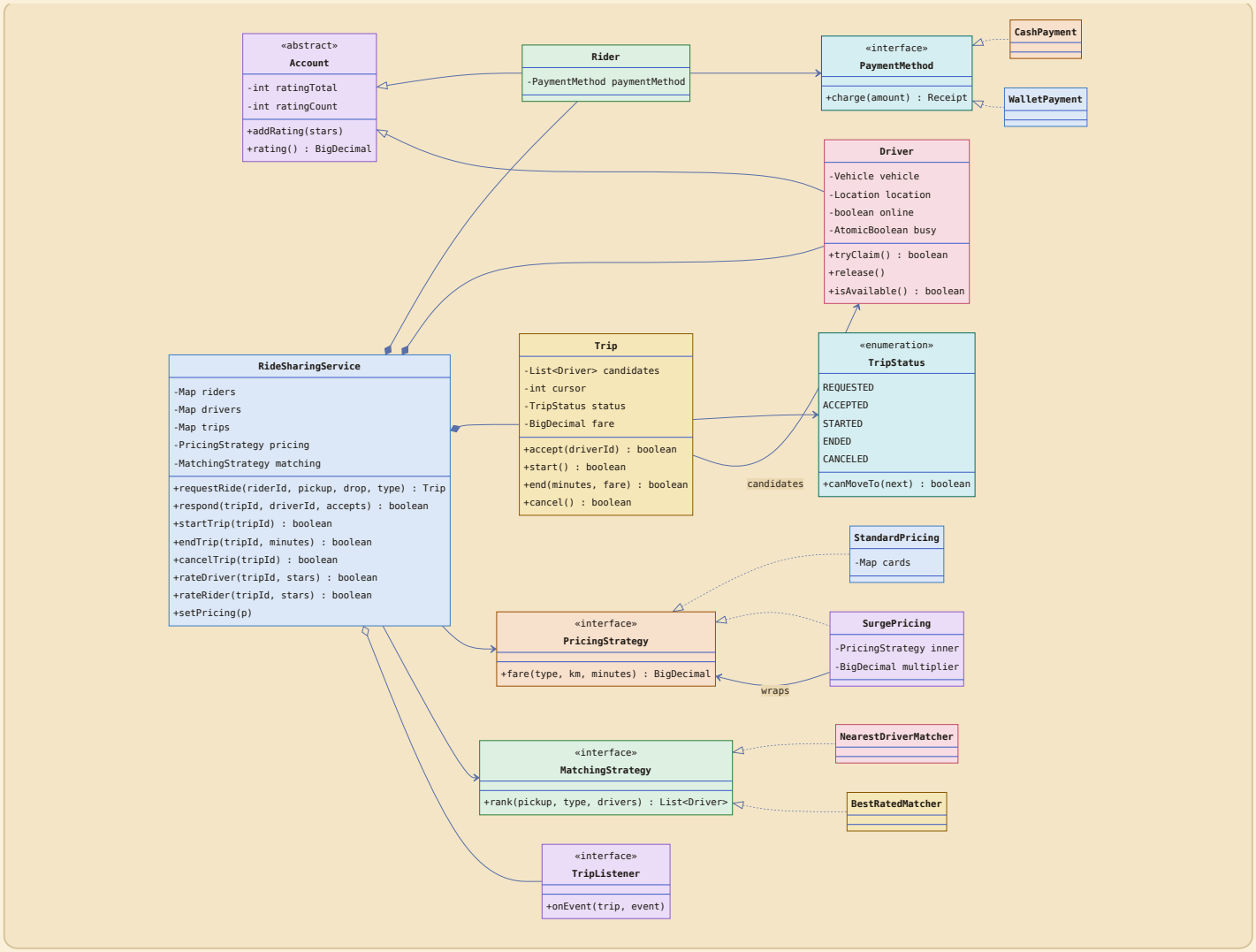


Figure — The finished ride sharing class diagram: the service owning riders, drivers and trips; PricingStrategy with StandardPricing and the SurgePricing wrapper; MatchingStrategy with the nearest-driver matcher (course diagram)

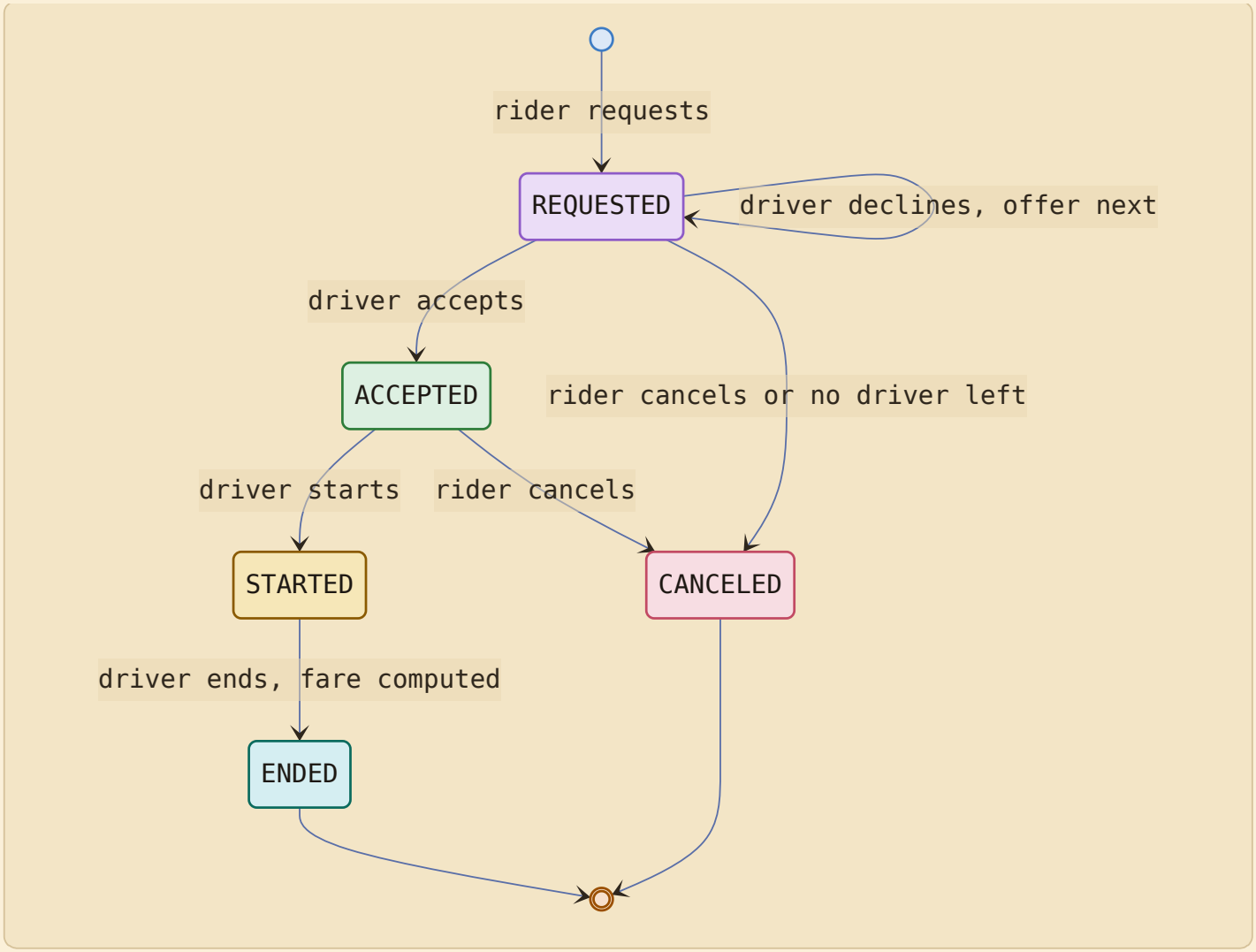
The course builds it in five stages, each leaving something runnable: (1) `Account` / `Rider` / `Driver`, `Vehicle`, `Location` (who is free and where); (2) `Trip` + `TripStatus` (the lifecycle, with illegal moves refused); (3) pricing with the surge wrapper (a finished trip has a fare); (4) matching (drivers who could take a request can be ranked); (5) `RideSharingService` (the complete flow, request through rating).

In the figure above, look for the extra arrow from `SurgePricing` back to the `PricingStrategy` interface: that "has-a of my own type" arrow is what turns it into a Decorator instead of one more Strategy.



Key flows

Trip lifecycle (state machine). A *state machine* is a fixed set of states plus the only moves allowed between them.



Finding a driver — including the case where every candidate declines:

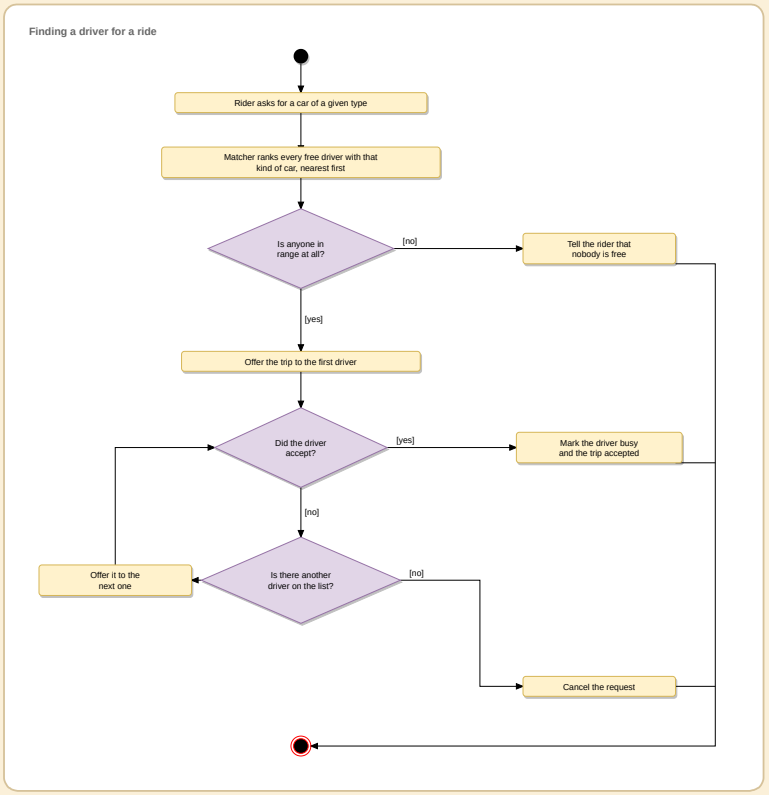


Figure — Finding a driver: rank free drivers of the right type nearest first; offer to the first; on decline move to the next; cancel when nobody is left (course diagram)

Ending and charging a trip — pricing is a single call no matter how many decorators wrap the rule:

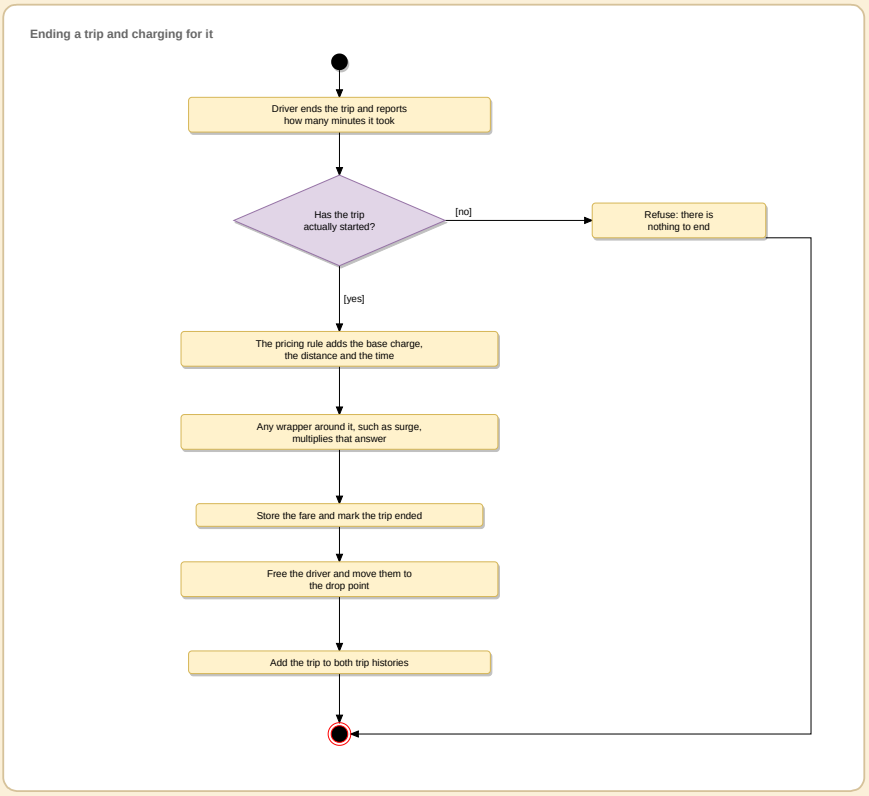
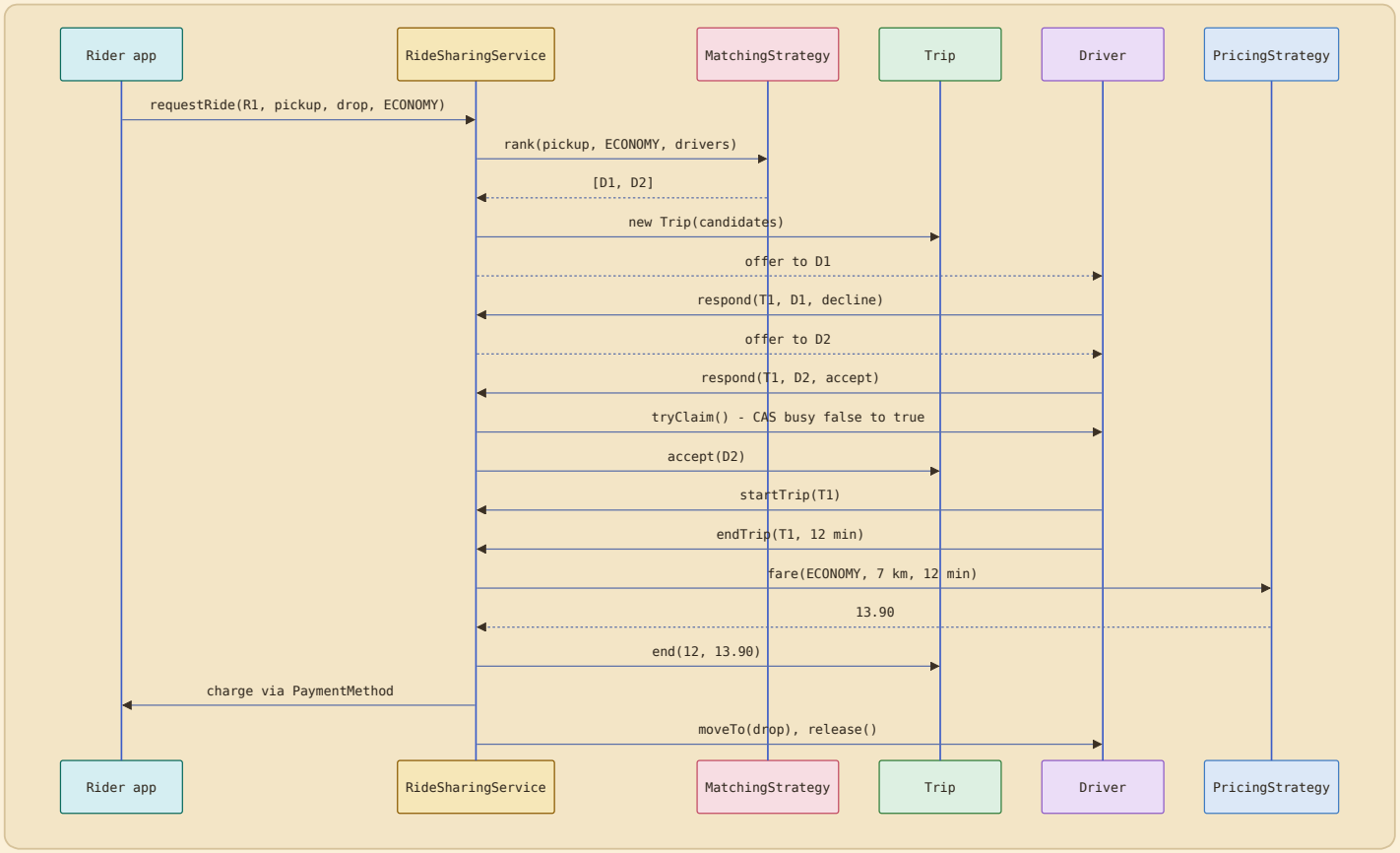


Figure — Ending a trip: refuse if not started; otherwise the pricing rule computes base plus distance plus time, any surge wrapper multiplies it, and the trip is recorded (course diagram)



Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for matching	Cities rank differently (nearest, best-rated, least-earned-today). One class each.	Sorting logic inside the service.
Matcher returns a **ranked list**, stored on the trip	A decline just moves the cursor; no new city-wide search, and the system remembers who refused.	Returning one driver: every decline re-searches from scratch.
Strategy for pricing, **Decorator** for surge	Surge wraps *any* pricing rule (even future ones). Turning it off = removing the wrapper, not editing an `if`. Decorators stack (surge + airport fee + discount).	A `surgeMultiplier` field and `if (surge)` in the pricing code.
State machine via `TripStatus.canMoveTo`	One table says what is legal; every method checks it and returns `false` on an illegal move. No exceptions for normal refusals.	Scattered `if (status == …)` checks; or the full State pattern (one class per state). The State pattern is worth it when behaviour per state is large; here a transition table is enough.
Atomic claim of a driver (`AtomicBoolean.compareAndSet`: "set to true only if it is still false", in one indivisible step)	Two trips racing for the same driver: exactly one wins, the other treats it as a decline.	Checking `isAvailable()` then setting busy: a check-then-act race (a race condition: the result depends on which thread happens to run first).
Driver **moved to the drop point** at trip end	Otherwise the next match uses where they were this morning.	Forgetting (a real bug class).
Observer for trip events	Rider/driver apps, push notifications, analytics (code that collects usage data for reports) listen; the service doesn't know them.	Service calling the notification code directly.
Deterministic tie-breaks (by driver id)	Same input → same ranking → reproducible tests.	Unordered ties: flaky tests (tests that sometimes pass and sometimes fail with no code change).
Money in **`BigDecimal`**, rounded `HALF_UP` (halves round up: 2.345 → 2.35) to 2 places	Exact fares. (The course keeps money as whole cents and the surge multiplier as a whole-number percentage, 150 = ×1.5, so no floating point (binary fractional numbers such as `double`) appears; `BigDecimal` is this book's equivalent.)	`double`.

Java implementation

```java
// File: code/lld/ch20/ridesharing/Model.java
package lld.ch20.ridesharing;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.atomic.AtomicBoolean;

/** A point on a city grid. Distance is measured along streets (Manhattan distance), in km. */
record Location(int x, int y) {
    int distanceTo(Location o) { return Math.abs(x - o.x) + Math.abs(y - o.y); }
    @Override public String toString() { return "(" + x + "," + y + ")"; }
}

enum VehicleType { ECONOMY, PREMIUM, VAN }

record Vehicle(String plate, VehicleType type, int seats) {}

/** What riders and drivers share. Rating = running total + count (average is all anyone reads). */
abstract class Account {
    private final String id;
    private final String name;
    private final String phone;
    private int ratingTotal;
    private int ratingCount;
    private final List<String> finishedTrips = new ArrayList<>();

    Account(String id, String name, String phone) {
        this.id = id;
        this.name = name;
        this.phone = phone;
    }

    String id() { return id; }
    String name() { return name; }
    String phone() { return phone; }

    synchronized void addRating(int stars) {
        if (stars < 1 || stars > 5) throw new IllegalArgumentException("stars must be 1..5");
        ratingTotal += stars;
        ratingCount++;
    }

    synchronized BigDecimal rating() {
        return ratingCount == 0 ? BigDecimal.ZERO
                : BigDecimal.valueOf(ratingTotal).divide(BigDecimal.valueOf(ratingCount), 2, RoundingMode.HALF_UP);
    }

    synchronized void recordTrip(String tripId) { finishedTrips.add(tripId); }
    synchronized List<String> finishedTrips() { return Collections.unmodifiableList(new ArrayList<>
(finishedTrips)); }
}

final class Rider extends Account {
    private final PaymentMethod paymentMethod;

    Rider(String id, String name, String phone, PaymentMethod paymentMethod) {
        super(id, name, phone);
        this.paymentMethod = paymentMethod;
    }

    PaymentMethod paymentMethod() { return paymentMethod; }
}
```

```java
final class Driver extends Account {
    private final Vehicle vehicle;
    private volatile Location location;
    private volatile boolean online;
    private final AtomicBoolean busy = new AtomicBoolean(false);

    Driver(String id, String name, String phone, Vehicle vehicle, Location location) {
        super(id, name, phone);
        this.vehicle = vehicle;
        this.location = location;
    }

    Vehicle vehicle() { return vehicle; }
    Location location() { return location; }
    void moveTo(Location l) { this.location = l; }
    void setOnline(boolean online) { this.online = online; }
    boolean isAvailable() { return online && !busy.get(); }

    /** Atomic claim: of two trips racing for this driver, exactly one wins. */
    boolean tryClaim() { return online && busy.compareAndSet(false, true); }
    void release() { busy.set(false); }
}
```

```java
// File: code/lld/ch20/ridesharing/Trip.java
package lld.ch20.ridesharing;

import java.math.BigDecimal;
import java.util.EnumSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/** The trip lifecycle as a table of legal moves (a compact State machine). */
enum TripStatus {
    REQUESTED, ACCEPTED, STARTED, ENDED, CANCELED;

    private static final Map<TripStatus, Set<TripStatus>> NEXT = Map.of(
            REQUESTED, EnumSet.of(ACCEPTED, CANCELED),
            ACCEPTED, EnumSet.of(STARTED, CANCELED),
            STARTED, EnumSet.of(ENDED),
            ENDED, EnumSet.noneOf(TripStatus.class),
            CANCELED, EnumSet.noneOf(TripStatus.class));

    boolean canMoveTo(TripStatus next) { return NEXT.get(this).contains(next); }
}

/** One journey. Every action checks the status first; an illegal action returns false and changes nothing. */
final class Trip {
    private final String id;
    private final String riderId;
    private final Location pickup;
    private final Location drop;
    private final VehicleType type;
    private final List<Driver> candidates; // ranked once; a decline just moves the cursor
    private int cursor;
    private TripStatus status = TripStatus.REQUESTED;
    private String driverId;
    private int minutes;
    private BigDecimal fare;
    private boolean riderRated, driverRated;

    Trip(String id, String riderId, Location pickup, Location drop, VehicleType type, List<Driver> candidates) {
        this.id = id;
        this.riderId = riderId;
        this.pickup = pickup;
        this.drop = drop;
        this.type = type;
        this.candidates = List.copyOf(candidates);
    }

    private boolean move(TripStatus next) {
        if (!status.canMoveTo(next)) return false;
        status = next;
        return true;
    }

    /** The driver currently being offered this trip, or null if the list is exhausted. */
    synchronized Driver currentCandidate() { return cursor < candidates.size() ? candidates.get(cursor) : null; }
    synchronized Driver nextCandidate() { cursor++; return currentCandidate(); }

    synchronized boolean accept(String driverId) {
        if (status != TripStatus.REQUESTED) return false;
        this.driverId = driverId;
        return move(TripStatus.ACCEPTED);
    }

    synchronized boolean start() { return move(TripStatus.STARTED); }

    synchronized boolean end(int minutes, BigDecimal fare) {
        if (!move(TripStatus.ENDED)) return false;
        this.minutes = minutes;
```

```
        this.fare = fare;
        return true;
    }

    synchronized boolean cancel() { return move(TripStatus.CANCELED); }

    synchronized boolean markRiderRated() { if (status != TripStatus.ENDED || riderRated) return false; return
riderRated = true; }
    synchronized boolean markDriverRated() { if (status != TripStatus.ENDED || driverRated) return false; return
driverRated = true; }

    String id() { return id; }
    String riderId() { return riderId; }
    Location pickup() { return pickup; }
    Location drop() { return drop; }
    VehicleType type() { return type; }
    int distanceKm() { return pickup.distanceTo(drop); }
    synchronized TripStatus status() { return status; }
    synchronized String driverId() { return driverId; }
    synchronized int minutes() { return minutes; }
    synchronized BigDecimal fare() { return fare; }
}
```

```java
// File: code/lld/ch20/ridesharing/Strategies.java
package lld.ch20.ridesharing;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Collection;
import java.util.Comparator;
import java.util.List;
import java.util.Map;

/* ---------------- Pricing: Strategy + Decorator ---------------- */

/** What a finished trip costs. */
interface PricingStrategy {
    BigDecimal fare(VehicleType type, int distanceKm, int minutes);
}

record RateCard(BigDecimal base, BigDecimal perKm, BigDecimal perMinute, BigDecimal minimum) {}

/** Base + distance + time, with a floor. One rate card per vehicle type. */
final class StandardPricing implements PricingStrategy {
    private final Map<VehicleType, RateCard> cards;

    StandardPricing(Map<VehicleType, RateCard> cards) { this.cards = Map.copyOf(cards); }

    @Override
    public BigDecimal fare(VehicleType type, int km, int minutes) {
        RateCard c = cards.get(type);
        BigDecimal raw = c.base()
                .add(c.perKm().multiply(BigDecimal.valueOf(km)))
                .add(c.perMinute().multiply(BigDecimal.valueOf(minutes)));
        return raw.max(c.minimum()).setScale(2, RoundingMode.HALF_UP);
    }
}

/** Decorator: wraps ANY pricing rule and multiplies its answer. Remove the wrapper to switch surge off. */
final class SurgePricing implements PricingStrategy {
    private final PricingStrategy inner;
    private final BigDecimal multiplier;

    SurgePricing(PricingStrategy inner, BigDecimal multiplier) {
        this.inner = inner;
        this.multiplier = multiplier;
    }

    @Override
    public BigDecimal fare(VehicleType type, int km, int minutes) {
        return inner.fare(type, km, minutes).multiply(multiplier).setScale(2, RoundingMode.HALF_UP);
    }
}

/* ---------------- Matching: Strategy ---------------- */

/** Returns a RANKED LIST (not one driver) so a decline just moves to the next name. */
interface MatchingStrategy {
    List<Driver> rank(Location pickup, VehicleType type, Collection<Driver> drivers);
}

final class NearestDriverMatcher implements MatchingStrategy {
    private final int maxRadiusKm;

    NearestDriverMatcher(int maxRadiusKm) { this.maxRadiusKm = maxRadiusKm; }

    @Override
    public List<Driver> rank(Location pickup, VehicleType type, Collection<Driver> drivers) {
        return drivers.stream()
                .filter(Driver::isAvailable)
                .filter(d -> d.vehicle().type() == type)
```

```
                .filter(d -> d.location().distanceTo(pickup) <= maxRadiusKm)
                .sorted(Comparator.comparingInt((Driver d) -> d.location().distanceTo(pickup))
                        .thenComparing(Driver::id))            // deterministic tie-break
                .toList();
    }
}

/** A second rule to show the swap: best-rated first, nearest as tie-break. */
final class BestRatedMatcher implements MatchingStrategy {
    private final NearestDriverMatcher nearby;

    BestRatedMatcher(int maxRadiusKm) { this.nearby = new NearestDriverMatcher(maxRadiusKm); }

    @Override
    public List<Driver> rank(Location pickup, VehicleType type, Collection<Driver> drivers) {
        return nearby.rank(pickup, type, drivers).stream()
                .sorted(Comparator.comparing(Driver::rating).reversed()
                        .thenComparingInt(d -> d.location().distanceTo(pickup)))
                .toList();
    }
}

/* ---------------- Payment: Strategy ---------------- */

record Receipt(boolean paid, String detail) {}

interface PaymentMethod {
    Receipt charge(BigDecimal amount);
}

final class CashPayment implements PaymentMethod {
    public Receipt charge(BigDecimal amount) { return new Receipt(true, "collect " + amount + " in cash"); }
}

final class WalletPayment implements PaymentMethod {
    private BigDecimal balance;

    WalletPayment(BigDecimal balance) { this.balance = balance; }

    public synchronized Receipt charge(BigDecimal amount) {
        if (balance.compareTo(amount) < 0) return new Receipt(false, "wallet has only " + balance);
        balance = balance.subtract(amount);
        return new Receipt(true, "wallet debited " + amount + ", left " + balance);
    }
}
```

```java
// File: code/lld/ch20/ridesharing/RideSharingService.java
package lld.ch20.ridesharing;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

/** Observer: rider/driver apps (push notifications) listen to trip events. */
interface TripListener {
    void onEvent(Trip trip, String event);
}

/** The class the apps talk to. Owns accounts and trips; holds the pricing and matching rules. */
final class RideSharingService {
    private final Map<String, Rider> riders = new ConcurrentHashMap<>();
    private final Map<String, Driver> drivers = new ConcurrentHashMap<>();
    private final Map<String, Trip> trips = new ConcurrentHashMap<>();
    private final List<TripListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger ids = new AtomicInteger();
    private volatile PricingStrategy pricing;
    private volatile MatchingStrategy matching;

    RideSharingService(PricingStrategy pricing, MatchingStrategy matching) {
        this.pricing = pricing;
        this.matching = matching;
    }

    void setPricing(PricingStrategy p) { this.pricing = p; }   // e.g. wrap in SurgePricing at rush hour
    void setMatching(MatchingStrategy m) { this.matching = m; }
    void addListener(TripListener l) { listeners.add(l); }
    void addRider(Rider r) { riders.put(r.id(), r); }
    void addDriver(Driver d) { drivers.put(d.id(), d); }
    Driver driver(String id) { return drivers.get(id); }
    Rider rider(String id) { return riders.get(id); }

    /** Rank candidates once, then offer to the first. No candidates -> canceled immediately. */
    Trip requestRide(String riderId, Location pickup, Location drop, VehicleType type) {
        List<Driver> ranked = matching.rank(pickup, type, drivers.values());
        Trip trip = new Trip("T" + ids.incrementAndGet(), riderId, pickup, drop, type, ranked);
        trips.put(trip.id(), trip);
        publish(trip, "requested by " + riderId + ", candidates " + ranked.stream().map(Driver::id).toList());
        offerOrCancel(trip, trip.currentCandidate());
        return trip;
    }

    /** Driver answers the offer. A lost race for the driver counts as a decline. */
    boolean respond(String tripId, String driverId, boolean accepts) {
        Trip trip = trips.get(tripId);
        Driver current = trip.currentCandidate();
        if (current == null || !current.id().equals(driverId) || trip.status() != TripStatus.REQUESTED) return
false;
        if (accepts && current.tryClaim()) {
            if (trip.accept(driverId)) {
                publish(trip, "accepted by " + driverId);
                return true;
            }
            current.release(); // trip was canceled meanwhile
            return false;
        }
        publish(trip, (accepts ? "could not claim " : "declined by ") + driverId);
        offerOrCancel(trip, trip.nextCandidate());
        return false;
    }

    boolean startTrip(String tripId) {
```

```
        Trip trip = trips.get(tripId);
        boolean ok = trip.start();
        publish(trip, ok ? "started" : "start refused in state " + trip.status());
        return ok;
    }

    /** Price -> store -> charge -> free the driver -> move driver to drop point -> history. */
    boolean endTrip(String tripId, int minutes) {
        Trip trip = trips.get(tripId);
        BigDecimal fare = pricing.fare(trip.type(), trip.distanceKm(), minutes);
        if (!trip.end(minutes, fare)) {
            publish(trip, "end refused in state " + trip.status());
            return false;
        }
        Rider rider = riders.get(trip.riderId());
        Receipt receipt = rider.paymentMethod().charge(fare);
        Driver d = drivers.get(trip.driverId());
        d.moveTo(trip.drop());       // easy to forget: the car is now at the drop point
        d.release();
        rider.recordTrip(trip.id());
        d.recordTrip(trip.id());
        publish(trip, "ended: " + trip.distanceKm() + " km, " + minutes + " min, fare " + fare
                + " (" + receipt.detail() + ")");
        return true;
    }

    /** Rider may cancel before the trip starts. */
    boolean cancelTrip(String tripId) {
        Trip trip = trips.get(tripId);
        if (!trip.cancel()) {
            publish(trip, "cancel refused in state " + trip.status());
            return false;
        }
        String driverId = trip.driverId(); // read AFTER cancel: no accept can happen any more
        if (driverId != null) drivers.get(driverId).release();
        publish(trip, "canceled by rider");
        return true;
    }

    boolean rateDriver(String tripId, int stars) {
        Trip t = trips.get(tripId);
        if (!t.markDriverRated()) return false;
        drivers.get(t.driverId()).addRating(stars);
        return true;
    }

    boolean rateRider(String tripId, int stars) {
        Trip t = trips.get(tripId);
        if (!t.markRiderRated()) return false;
        riders.get(t.riderId()).addRating(stars);
        return true;
    }

    private void offerOrCancel(Trip trip, Driver next) {
        if (next != null) {
            publish(trip, "offered to " + next.id());
        } else if (trip.cancel()) {
            publish(trip, "no driver left - canceled");
        }
    }

    private void publish(Trip trip, String event) {
        for (TripListener l : listeners) l.onEvent(trip, event);
    }
}
```

```java
// File: code/lld/ch20/ridesharing/Main.java
package lld.ch20.ridesharing;

import java.math.BigDecimal;
import java.util.Map;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        StandardPricing standard = new StandardPricing(Map.of(
                VehicleType.ECONOMY, new RateCard(bd("2.50"), bd("1.20"), bd("0.25"), bd("5.00")),
                VehicleType.PREMIUM, new RateCard(bd("5.00"), bd("2.00"), bd("0.40"), bd("10.00")),
                VehicleType.VAN, new RateCard(bd("4.00"), bd("1.80"), bd("0.30"), bd("8.00"))));
        RideSharingService svc = new RideSharingService(standard, new NearestDriverMatcher(10));
        svc.addListener((trip, event) -> System.out.println("  [" + trip.id() + "] " + event));

        svc.addRider(new Rider("R1", "Riya", "+91-1", new WalletPayment(bd("50.00"))));
        svc.addRider(new Rider("R2", "Sam", "+91-2", new CashPayment()));
        Driver d1 = new Driver("D1", "Dev", "+91-3", new Vehicle("KA-01", VehicleType.ECONOMY, 4), new Location(0,
0));
        Driver d2 = new Driver("D2", "Ira", "+91-4", new Vehicle("KA-02", VehicleType.ECONOMY, 4), new Location(2,
1));
        Driver d3 = new Driver("D3", "Om", "+91-5", new Vehicle("KA-03", VehicleType.PREMIUM, 4), new Location(1,
1));
        Driver d4 = new Driver("D4", "Zoe", "+91-6", new Vehicle("KA-04", VehicleType.ECONOMY, 4), new Location(1,
0));
        for (Driver d : new Driver[]{d1, d2, d3, d4}) svc.addDriver(d);
        d1.setOnline(true);
        d2.setOnline(true);
        d3.setOnline(true); // d4 stays offline, so it is never offered anything

        System.out.println("== 1. Request, first driver declines, second accepts, full ride");
        Trip t1 = svc.requestRide("R1", new Location(1, 0), new Location(5, 3), VehicleType.ECONOMY);
        svc.respond(t1.id(), "D1", false);
        svc.respond(t1.id(), "D2", true);
        System.out.println("  D2 available while on trip? " + d2.isAvailable());
        svc.startTrip(t1.id());
        svc.startTrip(t1.id());                // illegal: already started
        svc.endTrip(t1.id(), 12);              // 2.50 + 7*1.20 + 12*0.25 = 13.90
        System.out.println("  D2 now at " + d2.location() + ", available? " + d2.isAvailable());
        svc.rateDriver(t1.id(), 5);
        svc.rateRider(t1.id(), 4);
        System.out.println("  rate driver twice allowed? " + svc.rateDriver(t1.id(), 1)
                + "; D2 rating " + d2.rating() + ", R1 rating " + svc.rider("R1").rating());

        System.out.println("== 2. Surge: wrap the same rule in a decorator (x1.5)");
        svc.setPricing(new SurgePricing(standard, bd("1.5")));
        Trip t2 = svc.requestRide("R2", new Location(0, 1), new Location(0, 2), VehicleType.ECONOMY);
        svc.respond(t2.id(), "D1", true);
        svc.startTrip(t2.id());
        svc.endTrip(t2.id(), 3);               // max(2.50+1.20+0.75, 5.00) = 5.00, x1.5 = 7.50
        svc.setPricing(standard);

        System.out.println("== 3. Rider cancels after accept; cancel after start is refused");
        Trip t3 = svc.requestRide("R1", new Location(1, 1), new Location(3, 3), VehicleType.PREMIUM);
        svc.respond(t3.id(), "D3", true);
        svc.cancelTrip(t3.id());
        System.out.println("  D3 released? " + d3.isAvailable());
        Trip t4 = svc.requestRide("R1", new Location(1, 1), new Location(3, 3), VehicleType.PREMIUM);
        svc.respond(t4.id(), "D3", true);
        svc.startTrip(t4.id());
        svc.cancelTrip(t4.id());
        svc.endTrip(t4.id(), 10);              // 5 + 4*2 + 10*0.4 = 17.00 -> wallet has 36.10
```

```
        System.out.println("== 4. Nobody available for a VAN");
        svc.requestRide("R2", new Location(0, 0), new Location(1, 1), VehicleType.VAN);

        System.out.println("== 5. Race: 40 riders, 10 drivers, all accept at once");
        RideSharingService race = new RideSharingService(standard, new NearestDriverMatcher(100));
        for (int i = 0; i < 10; i++) {
            Driver d = new Driver("X" + i, "x", "-", new Vehicle("P" + i, VehicleType.ECONOMY, 4), new Location(i,
0));
            d.setOnline(true);
            race.addDriver(d);
        }
        for (int i = 0; i < 40; i++) race.addRider(new Rider("Q" + i, "q", "-", new CashPayment()));
        AtomicInteger accepted = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        ExecutorService pool = Executors.newFixedThreadPool(16);
        for (int i = 0; i < 40; i++) {
            String rider = "Q" + i;
            pool.submit(() -> {
                go.await();
                Trip t = race.requestRide(rider, new Location(0, 0), new Location(3, 3), VehicleType.ECONOMY);
                for (Driver c = t.currentCandidate(); c != null && t.status() == TripStatus.REQUESTED;
                     c = t.currentCandidate()) {
                    if (race.respond(t.id(), c.id(), true)) { accepted.incrementAndGet(); break; }
                }
                return null;
            });
        }
        go.countDown();
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
        System.out.println("  trips accepted = " + accepted.get() + " (never more than 10 drivers)");
    }

    private static BigDecimal bd(String s) { return new BigDecimal(s); }
}
```

Walking through the code.

- `TripStatus.NEXT` is the whole lifecycle in five lines; `Trip.move` refuses any move not in the table.
- `Trip` methods are `synchronized`: rider cancel and driver accept on the same trip cannot interleave (have their steps mixed together).
- `RideSharingService.respond`: only the *current* candidate may answer. Accept first **claims the driver atomically**; if the claim fails (another trip got them), it's treated as a decline and the next candidate is offered.
- `endTrip` works in a fixed order: price, store the fare on the trip, charge the rider, relocate the driver to the drop-off point, and only then release them (so no matcher can see a free driver at the old position) — then it records history for both sides. The course lists the same steps; relocating the driver is the step most often skipped, and skipping it makes the next match use the driver's morning position.
- Ratings are allowed once per side and only for ENDED trips (`markDriverRated` / `markRiderRated`).

Output:

```
== 1. Request, first driver declines, second accepts, full ride
  [T1] requested by R1, candidates [D1, D2]
  [T1] offered to D1
  [T1] declined by D1
  [T1] offered to D2
  [T1] accepted by D2
  D2 available while on trip? false
  [T1] started
  [T1] start refused in state STARTED
  [T1] ended: 7 km, 12 min, fare 13.90 (wallet debited 13.90, left 36.10)
  D2 now at (5,3), available? true
  rate driver twice allowed? false; D2 rating 5.00, R1 rating 4.00
== 2. Surge: wrap the same rule in a decorator (x1.5)
  [T2] requested by R2, candidates [D1, D2]
  [T2] offered to D1
  [T2] accepted by D1
  [T2] started
  [T2] ended: 1 km, 3 min, fare 7.50 (collect 7.50 in cash)
== 3. Rider cancels after accept; cancel after start is refused
  [T3] requested by R1, candidates [D3]
  [T3] offered to D3
  [T3] accepted by D3
  [T3] canceled by rider
  D3 released? true
  [T4] requested by R1, candidates [D3]
  [T4] offered to D3
  [T4] accepted by D3
  [T4] started
  [T4] cancel refused in state STARTED
  [T4] ended: 4 km, 10 min, fare 17.00 (wallet debited 17.00, left 19.10)
== 4. Nobody available for a VAN
  [T5] requested by R2, candidates []
  [T5] no driver left - canceled
== 5. Race: 40 riders, 10 drivers, all accept at once
  trips accepted = 10 (never more than 10 drivers)
```

Check the fares: trip 1 is 7 km and 12 min → 2.50 + 7 × 1.20 + 12 × 0.25 = **13.90**. Trip 2 is 1 km, 3 min → 4.45, below the 5.00 minimum → 5.00, then the surge decorator × 1.5 → **7.50**. The race test shows 40 concurrent requests against 10 drivers produce exactly 10 accepted trips — no driver is ever double-booked.

Machine-coding variant: carpool / "offer and select a ride"

A different ride-sharing problem is popular in machine-coding rounds (it appears in the kumaransg/LLD collection — a public GitHub repository of low-level design (LLD: designing the classes inside one program) problems — as a Flipkart-style round). Here there is **no dispatcher**: users *offer* spare seats in their own vehicle on a route, and other users *select* a ride using a **selection strategy**.

Requirements (from the problem statement, in brief):

- `add_user`, `add_vehicle` (a user may own several vehicles).
- `offer_ride(user, origin, seats, vehicle, destination)` — **fails if that vehicle already has an active offered ride**.
- `select_ride(user, origin, destination, seats, strategy)` — seats is 1 or 2; strategy is **Most Vacant** (most free seats) or **Preferred Vehicle** (e.g. "Activa", a scooter model).
- `end_ride(ride)` and `print_ride_stats()` — rides *taken* and *offered* per user, counting only **ended** rides (an offered ride counts even if nobody took it).

Design: `Ride` with an atomic `book(user, seats)`; `RideSelectionStrategy` (Strategy) with `MostVacant` and `PreferredVehicle`; one `RideService` that filters active rides by route and seats, excludes the user's own offers, and lets the strategy choose. The demo replays the problem's sample test case and prints the expected answers (2(c), 2(b), none, 2(d), none; then the stats).

```java
// File: code/lld/ch20/carpool/Main.java
package lld.ch20.carpool;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/**
 * Machine-coding variant (Flipkart-style "ride sharing / carpool"): users OFFER seats in their own
 * vehicle on a route; other users SELECT a ride with a selection strategy.
 */
public class Main {

    record User(String name, String gender, int age) {}

    record Vehicle(String owner, String model, String plate) {}

    enum RideStatus { ACTIVE, ENDED }

    static final class Ride {
        final String id;
        final User offeredBy;
        final Vehicle vehicle;
        final String origin, destination;
        private int availableSeats;
        private RideStatus status = RideStatus.ACTIVE;
        private final List<User> riders = new ArrayList<>();

        Ride(String id, User offeredBy, Vehicle vehicle, String origin, String destination, int seats) {
            this.id = id;
            this.offeredBy = offeredBy;
            this.vehicle = vehicle;
            this.origin = origin;
            this.destination = destination;
            this.availableSeats = seats;
        }

        synchronized int availableSeats() { return availableSeats; }

        /** Atomic check-and-book, so two users cannot take the last seat. */
        synchronized boolean book(User u, int seats) {
            if (status != RideStatus.ACTIVE || availableSeats < seats) return false;
            availableSeats -= seats;
            riders.add(u);
            return true;
        }

        @Override
        public String toString() {
            return id + " " + offeredBy.name() + " " + vehicle.model() + " " + origin + "->" + destination
                    + " seats=" + availableSeats;
        }
    }

    /** Strategy: pick one of the matching rides. */
    interface RideSelectionStrategy {
        Optional<Ride> select(List<Ride> candidates);
    }

    static final class MostVacant implements RideSelectionStrategy {
        public Optional<Ride> select(List<Ride> c) { return
c.stream().max(Comparator.comparingInt(Ride::availableSeats)); }
    }

    record PreferredVehicle(String model) implements RideSelectionStrategy {
```

```
        public Optional<Ride> select(List<Ride> c) {
            return c.stream().filter(r -> r.vehicle.model().equalsIgnoreCase(model)).findFirst();
        }
    }

    static final class RideService {
        private final Map<String, User> users = new LinkedHashMap<>();
        private final Map<String, Vehicle> vehicles = new LinkedHashMap<>();   // plate -> vehicle
        private final Map<String, Ride> rides = new LinkedHashMap<>();
        private final Map<String, int[]> stats = new LinkedHashMap<>();        // name -> {taken, offered}
        private int nextId = 0;

        void addUser(String name, String gender, int age) {
            users.put(name, new User(name, gender, age));
            stats.put(name, new int[2]);
        }

        void addVehicle(String owner, String model, String plate) { vehicles.put(plate, new Vehicle(owner, model,
plate)); }

        /** Fails if this vehicle already has an ACTIVE offered ride. */
        synchronized Optional<Ride> offerRide(String name, String origin, int seats, String plate, String
destination) {
            Vehicle v = vehicles.get(plate);
            if (v == null || !v.owner().equals(name)) return Optional.empty();
            boolean busy = rides.values().stream()
                    .anyMatch(r -> r.vehicle.plate().equals(plate) && r.status == RideStatus.ACTIVE);
            if (busy) return Optional.empty();
            Ride r = new Ride("2" + (char) ('a' + nextId++), users.get(name), v, origin, destination, seats);
            rides.put(r.id, r);
            return Optional.of(r);
        }

        synchronized Optional<Ride> selectRide(String name, String origin, String destination, int seats,
                                               RideSelectionStrategy strategy) {
            if (seats < 1 || seats > 2) throw new IllegalArgumentException("a user may request 1 or 2 seats");
            List<Ride> matching = rides.values().stream()
                    .filter(r -> r.status == RideStatus.ACTIVE)
                    .filter(r -> r.origin.equals(origin) && r.destination.equals(destination))
                    .filter(r -> !r.offeredBy.name().equals(name))
                    .filter(r -> r.availableSeats() >= seats)
                    .toList();
            Optional<Ride> chosen = strategy.select(matching);
            chosen.ifPresent(r -> r.book(users.get(name), seats));
            return chosen;
        }

        synchronized void endRide(String rideId) {
            Ride r = rides.get(rideId);
            if (r == null || r.status == RideStatus.ENDED) return;
            r.status = RideStatus.ENDED;
            stats.get(r.offeredBy.name())[1]++;
            for (User u : r.riders) stats.get(u.name())[0]++;
        }

        void printRideStats() {
            stats.forEach((n, s) -> System.out.println("  " + n + ": " + s[0] + " Taken, " + s[1] + " Offered"));
        }
    }

    public static void main(String[] args) {
        RideService s = new RideService();
        s.addUser("Rohan", "M", 36);    s.addVehicle("Rohan", "Swift", "KA-01-12345");
        s.addUser("Shashank", "M", 29); s.addVehicle("Shashank", "Baleno", "TS-05-62395");
        s.addUser("Nandini", "F", 29);
        s.addUser("Shipra", "F", 27);   s.addVehicle("Shipra", "Polo", "KA-05-41491");
                                        s.addVehicle("Shipra", "Activa", "KA-12-12332");
        s.addUser("Gaurav", "M", 29);
        s.addUser("Rahul", "M", 35);    s.addVehicle("Rahul", "XUV", "KA-05-1234");
```

```
        System.out.println("== offer_ride");
        print(s.offerRide("Rohan", "Hyderabad", 1, "KA-01-12345", "Bangalore"));
        print(s.offerRide("Shipra", "Bangalore", 1, "KA-12-12332", "Mysore"));
        print(s.offerRide("Shipra", "Bangalore", 2, "KA-05-41491", "Mysore"));
        print(s.offerRide("Shashank", "Hyderabad", 2, "TS-05-62395", "Bangalore"));
        print(s.offerRide("Rahul", "Hyderabad", 5, "KA-05-1234", "Bangalore"));
        print(s.offerRide("Rohan", "Bangalore", 1, "KA-01-12345", "Pune")); // same vehicle still active

        System.out.println("== select_ride");
        print(s.selectRide("Nandini", "Bangalore", "Mysore", 1, new MostVacant()));
        print(s.selectRide("Gaurav", "Bangalore", "Mysore", 1, new PreferredVehicle("Activa")));
        print(s.selectRide("Shashank", "Mumbai", "Bangalore", 1, new MostVacant()));
        print(s.selectRide("Rohan", "Hyderabad", "Bangalore", 1, new PreferredVehicle("Baleno")));
        print(s.selectRide("Shashank", "Hyderabad", "Bangalore", 1, new PreferredVehicle("Polo")));

        System.out.println("== end_ride 2a..2d, then print_ride_stats");
        for (String id : List.of("2a", "2b", "2c", "2d")) s.endRide(id);
        s.printRideStats();
    }

    private static void print(Optional<Ride> r) {
        System.out.println("  " + r.map(Ride::toString).orElse("No rides found / request failed"));
    }
}
```

Output:

```
== offer_ride
  2a Rohan Swift Hyderabad->Bangalore seats=1
  2b Shipra Activa Bangalore->Mysore seats=1
  2c Shipra Polo Bangalore->Mysore seats=2
  2d Shashank Baleno Hyderabad->Bangalore seats=2
  2e Rahul XUV Hyderabad->Bangalore seats=5
  No rides found / request failed
== select_ride
  2c Shipra Polo Bangalore->Mysore seats=1
  2b Shipra Activa Bangalore->Mysore seats=0
  No rides found / request failed
  2d Shashank Baleno Hyderabad->Bangalore seats=1
  No rides found / request failed
== end_ride 2a..2d, then print_ride_stats
  Rohan: 1 Taken, 1 Offered
  Shashank: 0 Taken, 1 Offered
  Nandini: 1 Taken, 0 Offered
  Shipra: 0 Taken, 2 Offered
  Gaurav: 1 Taken, 0 Offered
  Rahul: 0 Taken, 0 Offered
```

	Uber-style (20.4 main design)	Carpool variant
Who supplies the ride	Professional drivers, online/offline	Any user, per offered ride
Who chooses	The system (matching strategy) offers to drivers	The rider picks with a selection strategy
Key pattern	Strategy + Decorator + state machine	Strategy (selection)
Key invariant	A driver is on at most one trip	A vehicle has at most one active offer; seats never go negative

Concurrency & edge cases

- **Double booking a driver:** prevented by `Driver.tryClaim()` (compare-and-set). Proven by the 40-riders/10-drivers race in the output.

- **Cancel vs accept race:** both go through `synchronized` `Trip` methods; if the rider cancels after the claim but before `accept`, `accept` fails and the driver is released.
- **Stale candidate list:** the list is computed once. A driver may go offline or be claimed later; `tryClaim` fails and we move on. A nearer driver who becomes free later is not noticed (real systems re-match periodically).
- **Offer timeouts:** a driver who never answers should be treated as a decline after N seconds (a `ScheduledExecutorService` task per offer).
- **No drivers at all:** trip canceled immediately (see trip T5).
- **Illegal transitions** (start twice, cancel after start, end before start): return `false`; state unchanged.
- **Wallet with insufficient balance:** `Receipt(paid=false)`; a real system would retry another method or record a debt on the rider's account.
- **Rating abuse:** one rating per side per trip; stars validated 1–5.

Extensions & follow-up questions

- **"Rank by rating instead."** Swap `MatchingStrategy` (`BestRatedMatcher` is included) — one line at startup.
- **"Airport fee, promo code, tolls."** More decorators around the pricing rule: `new PromoDiscount(new AirportFee(new SurgePricing(standard, 1.5)))`.
- **"How is surge computed?"** From demand (open requests) vs supply (free drivers) per area and time window; the result feeds the multiplier. The pricing design doesn't change.
- **"Find nearby drivers fast in a real city."** A geospatial index (a lookup structure for finding things by location): split the map into cells (geohash, Google S2 or Uber's H3 hexagons — three standard ways of dividing the globe into numbered cells) and keep a map cell → drivers; search the pickup cell and its neighbours instead of all drivers.
- **"Shared rides (pool)."** Honest answer: this design assumes one rider per trip. The change starts at `Trip` (many riders, many stops) and a matcher that considers detours.
- **"Live tracking."** Drivers send location updates every few seconds; store the latest per driver and publish to the rider (pub-sub, see 20.6).
- **"Scheduled rides."** A request with a future time; a scheduler (a component that runs jobs at chosen times) starts matching shortly before.

Real-world example

Uber really does show riders a surge ("dynamic pricing") multiplier when demand exceeds supply in an area, and it sends a request to a nearby driver who has a short time to accept before it goes to someone else — the ranked-list-with-cursor idea. Uber open-sourced (published free for anyone to use) **H3**, a hierarchical (cells nest inside bigger cells) hexagonal grid that its engineering blog describes as built for optimizing ride pricing and dispatch (for example, bucketing supply and demand per hexagon); the same cell idea answers "which drivers are near this point" without scanning the whole city. Ola, Lyft and Grab follow the same request → match → accept → trip → rate lifecycle.

In the interview

> **30-second answer:** "Two rules behind interfaces: `MatchingStrategy` returns a ranked list of free drivers with the right car, and `PricingStrategy` computes base + distance + time with a floor. Surge is a Decorator that wraps any pricing rule and multiplies it. The trip is a small state machine; every action checks the transition table and returns false if illegal. A decline moves a cursor along the stored ranking. Drivers are claimed with a compare-and-set so one driver can never get two trips."

Tip: name the two rules (matching and pricing) in your first minutes and say both go behind interfaces; that one sentence pre-answers most follow-ups. Then spend real time on surge: *wrapping* the pricing rule instead of adding a flag to it is what separates strong answers from weak ones.

Plan (40 min): clarify (5) → accounts, vehicle, location (5) → trip + state machine (8) → pricing + surge decorator (7) → matching list + decline flow (7) → concurrency (claim) (4) → extensions (4).

Follow-ups with answers:

- *"What happens when a driver declines?"* → Next driver in the stored ranked list; none left → cancel.
- *"Strategy vs Decorator here?"* → Strategy replaces the rule; Decorator wraps it and remains in the call path (it holds an `inner` of the same interface).
- *"Two riders get the same driver?"* → Impossible: atomic claim; the loser moves to its next candidate.

Self-check

1. Why does the matcher return a list instead of one driver? → So a decline just moves to the next candidate without re-searching and remembers who refused.
2. What makes `SurgePricing` a Decorator? → It is a `PricingStrategy` that also wraps a second `PricingStrategy` and adjusts that one's result.
3. Can a rider cancel a STARTED trip? → No; `STARTED` can only move to `ENDED`.
4. Why update the driver's location to the drop-off when the trip ends? → Later matches must use their real position.
5. Which Java class makes the driver claim atomic? → `AtomicBoolean` (`compareAndSet(false, true)`).
6. In the carpool variant, when does `offer_ride` fail? → When the same vehicle already has an active offered ride.

TL;DR

- **Strategy** ×3 (matching, pricing, payment) + **Decorator** (surge) + **state machine** (trip).
- Matcher returns a **ranked list**; the trip keeps a cursor; decline = next.
- **Atomic claim** of drivers prevents double booking; relocate the driver to the drop-off at trip end.
- Ratings = running total + count; `BigDecimal` fares with a minimum.
- Carpool variant: users offer seats; riders select with Most Vacant / Preferred Vehicle strategies.

20.5 Design a Logging Framework

Problem & clarifying questions

What it is. A logging framework lets code write messages like "payment 991 authorised" with a **level** (how serious it is) and sends them to one or more **destinations** (console, file, a log server), in a chosen **format**. Everyday analogy: a company's internal mail room. Every department drops notes in; each note has an urgency stamp; the mail room decides, by department and urgency, which notes get copied to which inboxes and in what envelope.

Think Log4j 2, Logback or `java.util.logging` (the most widely used Java logging libraries): this question asks you to design a small version of them.

Question	Assumed answer
Levels?	TRACE < DEBUG < INFO < WARN < ERROR < FATAL (and OFF).
Per-module configuration?	Yes: loggers are named `app.payment`, `app.orders`… and form a **hierarchy** by dots. A child inherits its level from the nearest ancestor that sets one.
Destinations?	Console, file, in-memory (tests); pluggable. Each can have its own minimum level.
Formats?	Plain pattern and JSON (a text format of nested key-value pairs); pluggable.
Propagation?	An event goes to the logger's own destinations and then **up** to its ancestors' destinations, unless that logger turns propagation (additivity) off.
Async?	Yes, optional: the application thread must not wait for disk I/O (input/output: reading or writing the disk).
Thread-safe?	Yes: many threads log at once.
Config files, rolling files (a log file is closed and a new one started at a size or date limit), remote shipping (sending logs to another server)?	Out of scope (extensions).

Requirements

Functional

1. Log a message at a level; messages below the logger's effective level are dropped cheaply.
2. Loggers are obtained by name; the same name returns the same logger.
3. Loggers form a dot-separated hierarchy under a root; level is inherited when not set.
4. A logger has zero or more appenders (destinations). Each appender has a minimum level and a formatter.
5. Events propagate up to parent appenders unless additivity is off.
6. Formatters are pluggable (pattern, JSON).
7. Asynchronous appending is available without changing the appenders.
8. Exceptions can be attached to an event.
9. Expensive messages can be built lazily (only if the level is enabled).

Non-functional: very cheap when disabled; thread-safe; logging must never crash the application; testable (inject a clock).

Use cases

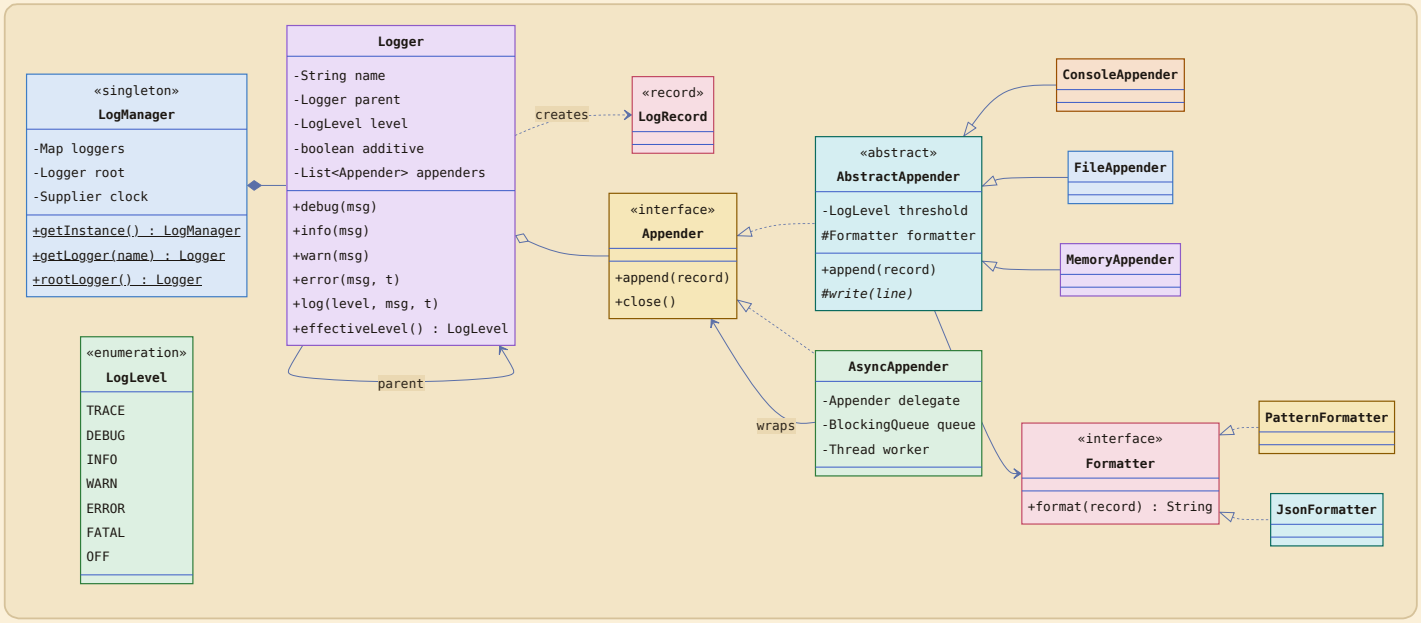
Actors: **Developer code** (logs messages), **Operator** (configures levels, appenders, formats at startup or runtime — while the program is running), **Framework** (filters, formats, routes, writes).

Main use cases: *Log a message*, *Get a logger by name*, *Set a level for a subtree*, *Add a destination*, *Choose a format*, *Switch to async writing*, *Stop propagation for a subtree*.

Core entities

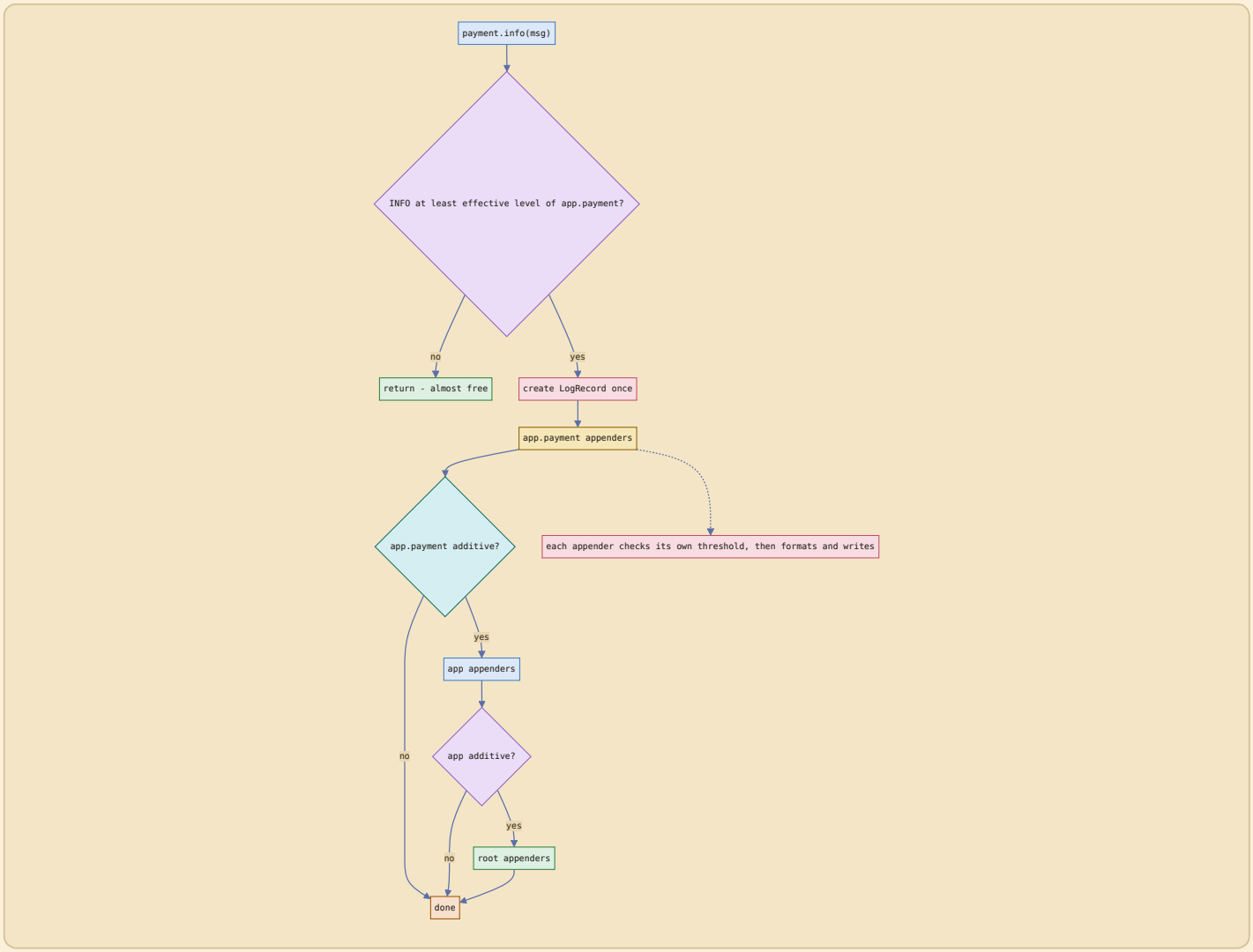
Class	Responsibility
`LogLevel` (enum)	Ordered severities; `isAtLeast(threshold)`.
`LogRecord` (record)	One immutable (cannot change after creation) event: time, level, logger name, thread, message, error.
`Logger`	Named node in the hierarchy: parent, optional level, appenders, additivity; `log()` walks the chain upward.
`LogManager`	**Singleton** registry; `getLogger(name)` creates parents first and returns one instance per name; holds the clock.
`Appender` (interface)	A destination (sink).
`AbstractAppender`	**Template Method**: `append()` checks the appender's threshold and formats; subclasses only `write(line)`.
`ConsoleAppender`, `FileAppender`, `MemoryAppender`	Concrete sinks.
`AsyncAppender`	**Decorator**: wraps any appender with a bounded queue + one writer thread.
`Formatter` (interface)	**Strategy**: record → text. `PatternFormatter`, `JsonFormatter`.

Class diagram



Key flows

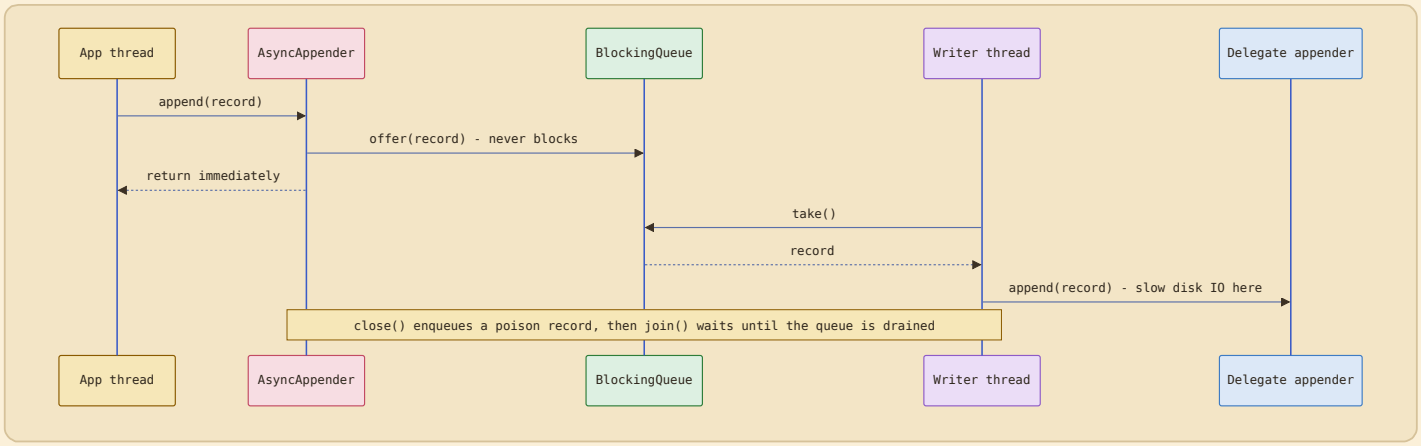
Logging one event (Chain of Responsibility up the hierarchy). *Chain of Responsibility* = a request is passed along a chain of handlers; each may handle it and/or pass it on. Here every logger in the chain hands the record to its own appenders and passes it to its parent, unless additivity is off.



Note the Log4j/Logback rule we copy: the **level check happens once, at the logger where the call was made.** Parents' *levels* are not re-checked during propagation; only each **appender's own threshold** filters. That is why,

in the demo, a DEBUG event from `app.payment` (set to DEBUG) still reaches the root console appender even though root is at INFO.

Async appending.



Design decisions & patterns used

Pattern / decision	Where	Why	Alternative rejected
Chain of Responsibility	`Logger.log` walks `this → parent → … → root`, stopping when `additive == false`	Configure destinations once near the root; subtrees can add or stop.	Every logger listing all its appenders explicitly.
Singleton	`LogManager` (initialization-on-demand holder)	One registry per process, so `getLogger("x")` always returns the same object. Holder idiom (a standard coding recipe) = lazy + thread-safe with no locking (the JVM — Java Virtual Machine, the program that runs Java code — initializes the nested class once).	Public static mutable (changeable) map; double-checked locking (check, lock, check again; easy to get wrong).
Static factory method	`LogManager.getLogger(name)`	Hides creation, caches instances, creates parents first.	`new Logger(...)` by users (breaks the hierarchy).
Strategy	`Formatter`	Format is independent of destination: any sink × any format.	`JsonFileAppender`, `PatternFileAppender`, … (class explosion).
Template Method	`AbstractAppender.append` (final) → `write`	Threshold + formatting done once; sinks only write text.	Each sink re-implementing filtering.
Decorator	`AsyncAppender` wraps any `Appender`	Async becomes an add-on, not a rewrite of every sink.	`AsyncFileAppender`, `AsyncConsoleAppender`, …
Level check first; **lazy** `Supplier<String>` (a function that builds the string only when called)	`isEnabled`, `debug(Supplier)`	Disabled logging costs one comparison; expensive strings are never built.	Always building the message (a real performance bug).
Appender failures caught	`Logger.log`	Logging must never crash the app.	Letting exceptions escape.
Parent resolved **outside** `computeIfAbsent`	`LogManager.loggerFor`	A `ConcurrentHashMap` mapping function must not modify the same map (the JDK may throw `IllegalStateException: Recursive update`).	Recursive `computeIfAbsent`.

A note on another common "CoR logger" answer. Many LLD write-ups model Chain of Responsibility as a chain of *level handlers* (`ErrorHandler → WarnHandler → InfoHandler → DebugHandler`), each writing messages of "its" level and passing others along. It works for a toy, but real frameworks put the chain on the **logger hierarchy** (as here) because that is what operators actually configure. If an interviewer expects the level-handler version, explain both and why the hierarchy is the one Log4j/Logback use.

Java implementation

```java
// File: code/lld/ch20/logging/Core.java
package lld.ch20.logging;

import java.time.Instant;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;

/** Severity, lowest to highest. A logger set to WARN drops TRACE, DEBUG and INFO. */
enum LogLevel {
    TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF;

    boolean isAtLeast(LogLevel threshold) { return this.ordinal() >= threshold.ordinal(); }
}

/** One immutable log event. Created once, passed along the chain, formatted by each appender. */
record LogRecord(Instant time, LogLevel level, String loggerName, String thread, String message, Throwable error)
{}

/** Strategy: how a record becomes text. */
interface Formatter {
    String format(LogRecord r);
}

final class PatternFormatter implements Formatter {
    private static final DateTimeFormatter TIME =
            DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss.SSS").withZone(ZoneOffset.UTC);

    @Override
    public String format(LogRecord r) {
        String line = String.format("%s %-5s [%s] %s - %s", TIME.format(r.time()), r.level(), r.thread(),
                r.loggerName(), r.message());
        return r.error() == null ? line : line + " | " + r.error();
    }
}

final class JsonFormatter implements Formatter {
    @Override
    public String format(LogRecord r) {
        return "{\"ts\":\"" + r.time() + "\",\"level\":\"" + r.level() + "\",\"logger\":\"" + r.loggerName()
                + "\",\"msg\":\"" + r.message().replace("\"", "\\\"") + "\"}";
    }
}
```

```java
// File: code/lld/ch20/logging/Appenders.java
package lld.ch20.logging;

import java.io.BufferedWriter;
import java.io.IOException;
import java.io.PrintStream;
import java.io.UncheckedIOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.atomic.AtomicLong;

/** A sink (destination). Each has its own threshold and formatter. */
interface Appender extends AutoCloseable {
    void append(LogRecord r);

    @Override
    default void close() {}
}

/** Shared threshold + formatter handling, so concrete appenders only write text. */
abstract class AbstractAppender implements Appender {
    private final LogLevel threshold;
    protected final Formatter formatter;

    AbstractAppender(LogLevel threshold, Formatter formatter) {
        this.threshold = threshold;
        this.formatter = formatter;
    }

    @Override
    public final void append(LogRecord r) {
        if (r.level().isAtLeast(threshold)) write(formatter.format(r));
    }

    protected abstract void write(String line);
}

final class ConsoleAppender extends AbstractAppender {
    private final PrintStream out;

    ConsoleAppender(LogLevel threshold, Formatter f, PrintStream out) {
        super(threshold, f);
        this.out = out;
    }

    @Override
    protected synchronized void write(String line) { out.println(line); }
}

final class FileAppender extends AbstractAppender {
    private final BufferedWriter writer;

    FileAppender(LogLevel threshold, Formatter f, Path file) throws IOException {
        super(threshold, f);
        this.writer = Files.newBufferedWriter(file, StandardCharsets.UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND);
    }

    @Override
    protected synchronized void write(String line) {
        try {
            writer.write(line);
```

```
            writer.newLine();
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }

    @Override
    public synchronized void close() {
        try {
            writer.close();
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }
}

/** Keeps lines in memory - handy for tests. */
final class MemoryAppender extends AbstractAppender {
    private final List<String> lines = new ArrayList<>();

    MemoryAppender(LogLevel threshold, Formatter f) { super(threshold, f); }

    @Override
    protected synchronized void write(String line) { lines.add(line); }

    synchronized List<String> lines() { return List.copyOf(lines); }
}

/**
 * Decorator: makes ANY appender asynchronous. Callers only enqueue; one background thread writes.
 * Bounded queue: when full we drop and count (a common production choice), never block the app.
 */
final class AsyncAppender implements Appender {
    private static final LogRecord POISON = new LogRecord(null, LogLevel.OFF, "", "", "", null);
    private final Appender delegate;
    private final BlockingQueue<LogRecord> queue;
    private final Thread worker;
    private final AtomicLong dropped = new AtomicLong();
    private volatile boolean closed;

    AsyncAppender(Appender delegate, int capacity) {
        this.delegate = delegate;
        this.queue = new ArrayBlockingQueue<>(capacity);
        this.worker = Thread.ofPlatform().daemon().name("async-log-writer").start(this::drain);
    }

    @Override
    public void append(LogRecord r) {
        if (closed || !queue.offer(r)) dropped.incrementAndGet(); // full or closed: count it, never block
    }

    private void drain() {
        try {
            for (LogRecord r = queue.take(); r != POISON; r = queue.take()) {
                try {
                    delegate.append(r);
                } catch (RuntimeException e) {  // one bad write must not kill the writer thread
                    System.err.println("async appender: delegate failed: " + e);
                }
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    long dropped() { return dropped.get(); }

    /** Flush everything already queued, then stop. Records appended after this point are counted as dropped. */
    @Override
```

```
    public void close() {
        if (closed) return;
        closed = true;
        try {
            queue.put(POISON);
            worker.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        delegate.close();
    }
}
```

```java
// File: code/lld/ch20/logging/Logger.java
package lld.ch20.logging;

import java.time.Instant;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.function.Supplier;

/**
 * A named logger in a dot-separated hierarchy ("app.payment" -> "app" -> root).
 * An event is handed to this logger's appenders, then passed UP to the parent's appenders
 * (Chain of Responsibility) until a logger with additivity=false stops it.
 */
final class Logger {
    private final String name;
    private final Logger parent;
    private final LogManager manager;
    private volatile LogLevel level;                    // null = inherit from the nearest ancestor
    private volatile boolean additive = true;
    private final List<Appender> appenders = new CopyOnWriteArrayList<>();

    Logger(String name, Logger parent, LogManager manager) {
        this.name = name;
        this.parent = parent;
        this.manager = manager;
    }

    String name() { return name; }
    void setLevel(LogLevel level) { this.level = level; }
    void setAdditive(boolean additive) { this.additive = additive; }
    void addAppender(Appender a) { appenders.add(a); }

    LogLevel effectiveLevel() {
        for (Logger l = this; l != null; l = l.parent) if (l.level != null) return l.level;
        return LogLevel.INFO;
    }

    boolean isEnabled(LogLevel lvl) { return lvl != LogLevel.OFF && lvl.isAtLeast(effectiveLevel()); }

    void trace(String msg) { log(LogLevel.TRACE, msg, null); }
    void debug(String msg) { log(LogLevel.DEBUG, msg, null); }
    void info(String msg) { log(LogLevel.INFO, msg, null); }
    void warn(String msg) { log(LogLevel.WARN, msg, null); }
    void error(String msg, Throwable t) { log(LogLevel.ERROR, msg, t); }

    /** Lazy message: the (possibly expensive) string is only built if the level is enabled. */
    void debug(Supplier<String> msg) { if (isEnabled(LogLevel.DEBUG)) log(LogLevel.DEBUG, msg.get(), null); }

    void log(LogLevel lvl, String msg, Throwable t) {
        if (!isEnabled(lvl)) return;                    // cheap check first
        LogRecord r = new LogRecord(manager.now(), lvl, name, Thread.currentThread().getName(), msg, t);
        for (Logger l = this; l != null; l = l.parent) { // the chain
            for (Appender a : l.appenders) {
                try {
                    a.append(r);
                } catch (RuntimeException e) {
                    System.err.println("appender failed: " + e); // logging must never crash the app
                }
            }
            if (!l.additive) break;
        }
    }
}

/** Singleton registry of loggers (initialization-on-demand holder idiom: lazy and thread-safe). */
final class LogManager {
```

```
    private final Map<String, Logger> loggers = new ConcurrentHashMap<>();
    private final Logger root = new Logger("root", null, this);
    private volatile Supplier<Instant> clock = Instant::now;

    private LogManager() { root.setLevel(LogLevel.INFO); }

    private static final class Holder { static final LogManager INSTANCE = new LogManager(); }

    static LogManager getInstance() { return Holder.INSTANCE; }

    /** Factory method: same name -> same Logger instance; parents are created first. */
    static Logger getLogger(String name) { return getInstance().loggerFor(name); }

    static Logger rootLogger() { return getInstance().root; }

    private Logger loggerFor(String name) {
        if (name == null || name.isEmpty() || name.equals("root")) return root;
        Logger existing = loggers.get(name);
        if (existing != null) return existing;
        int dot = name.lastIndexOf('.');
        Logger parent = dot < 0 ? root : loggerFor(name.substring(0, dot)); // resolved OUTSIDE computeIfAbsent
        return loggers.computeIfAbsent(name, n -> new Logger(n, parent, this));
    }

    Instant now() { return clock.get(); }
    void setClock(Supplier<Instant> c) { this.clock = c; }
}
```

```java
// File: code/lld/ch20/logging/Main.java
package lld.ch20.logging;

import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        LogManager.getInstance().setClock(() -> Instant.parse("2026-09-28T10:15:30.123Z")); // fixed for a stable
demo

        Logger root = LogManager.rootLogger();
        root.setLevel(LogLevel.INFO);
        root.addAppender(new ConsoleAppender(LogLevel.TRACE, new PatternFormatter(), System.out));

        Logger payment = LogManager.getLogger("app.payment");
        Logger app = LogManager.getLogger("app");
        System.out.println("== 1. Same name -> same instance: " + (payment ==
LogManager.getLogger("app.payment")));

        System.out.println("== 2. Levels: 'app.payment' inherits INFO from root");
        payment.debug("hidden: DEBUG < INFO");
        payment.info("payment 991 authorised");
        payment.warn("retrying gateway call");

        System.out.println("== 3. Raise one subtree to DEBUG; extra JSON sink only for payments");
        MemoryAppender audit = new MemoryAppender(LogLevel.DEBUG, new JsonFormatter());
        payment.setLevel(LogLevel.DEBUG);
        payment.addAppender(audit);
        payment.debug(() -> "card token=" + "tok_" + 42 + " (built lazily)");
        LogManager.getLogger("app.orders").debug("still hidden: app.orders inherits INFO");
        System.out.println("  audit sink got: " + audit.lines());

        System.out.println("== 4. Additivity off: 'app.payment' events stop at its own appenders");
        payment.setAdditive(false);
        payment.error("refund failed", new IllegalStateException("gateway 503"));
        System.out.println("  (nothing printed on console above) audit sink now has " + audit.lines().size() + "
lines");
        payment.setAdditive(true);

        System.out.println("== 5. File sink with its own threshold (WARN+)");
        Path file = Files.createTempFile("app", ".log");
        FileAppender fileSink = new FileAppender(LogLevel.WARN, new PatternFormatter(), file);
        app.addAppender(fileSink);
        app.info("info goes to console only");
        app.warn("disk 91% full");
        fileSink.close();
        System.out.println("  file contains: " + Files.readAllLines(file));
        Files.delete(file);

        System.out.println("== 6. Async appender: 4 threads x 1000 events, then close() flushes");
        CountingAppender counter = new CountingAppender();
        AsyncAppender async = new AsyncAppender(counter, 10_000);
        Logger worker = LogManager.getLogger("worker");
        worker.setAdditive(false);
        worker.addAppender(async);
        List<Thread> threads = new ArrayList<>();
        for (int t = 0; t < 4; t++) {
            int id = t;
            threads.add(Thread.ofPlatform().start(() -> {
                for (int i = 0; i < 1000; i++) worker.info("job " + id + "-" + i);
            }));
        }
        for (Thread t : threads) t.join();
```

```
        async.close();
        System.out.println("  written=" + counter.count.get() + " dropped=" + async.dropped()
                + " total=" + (counter.count.get() + async.dropped()));
    }

    static final class CountingAppender implements Appender {
        final AtomicInteger count = new AtomicInteger();
        public void append(LogRecord r) { count.incrementAndGet(); }
    }
}
```

Walking through the code.

- `Logger.effectiveLevel` walks up until a logger has a level; root always has one.
- `Logger.log` creates the `LogRecord` **once** and hands the same immutable object to every appender in the chain (safe to share across threads because it is a record with final fields).
- `AbstractAppender.append` is `final`; `ConsoleAppender` and `FileAppender` synchronize `write` so lines from different threads never interleave mid-line.
- `AsyncAppender` uses a bounded `ArrayBlockingQueue` (a fixed-size, thread-safe queue) and `offer` (non-blocking: returns at once instead of waiting): when full, it drops and counts instead of stalling the application. `close()` puts a "poison pill" record (a special marker that means "stop after this") so the writer drains everything before stopping. The no-loss promise is precise: every record appended **before** `close()` is either written or counted in `dropped` (queue full); a record appended after `close()` is counted as dropped too, so `written + dropped` always equals the number of `append` calls. A delegate that throws is caught inside the writer loop, so one bad write cannot kill the writer thread (which would otherwise leave `close()` waiting forever on a full queue).
- `LogManager` uses the holder idiom: `Holder.INSTANCE` is created the first time `getInstance()` runs, and the JVM guarantees that happens once.

Output:

```
== 1. Same name -> same instance: true
== 2. Levels: 'app.payment' inherits INFO from root
2026-09-28 10:15:30.123 INFO  [main] app.payment - payment 991 authorised
2026-09-28 10:15:30.123 WARN  [main] app.payment - retrying gateway call
== 3. Raise one subtree to DEBUG; extra JSON sink only for payments
2026-09-28 10:15:30.123 DEBUG [main] app.payment - card token=tok_42 (built lazily)
  audit sink got: [{"ts":"2026-09-28T10:15:30.123Z","level":"DEBUG","logger":"app.payment","msg":"card token=tok_42
(built lazily)"}]
== 4. Additivity off: 'app.payment' events stop at its own appenders
  (nothing printed on console above) audit sink now has 2 lines
== 5. File sink with its own threshold (WARN+)
2026-09-28 10:15:30.123 INFO  [main] app - info goes to console only
2026-09-28 10:15:30.123 WARN  [main] app - disk 91% full
  file contains: [2026-09-28 10:15:30.123 WARN  [main] app - disk 91% full]
== 6. Async appender: 4 threads x 1000 events, then close() flushes
  written=4000 dropped=0 total=4000
```

Step 4: with additivity off, the ERROR went only to the JSON audit sink (2 lines now) and not to the console. Step 6: 4 threads × 1000 events all arrived (4000 written, 0 dropped), and `close()` flushed the queue.

Concurrency & edge cases

- **Many threads logging:** appenders list is `CopyOnWriteArrayList` (reads without locking, rare writes); level and additivity are `volatile` (a Java keyword that makes every thread see the latest written value) so a runtime change is seen by all threads.
- **Interleaved output:** each sink's `write` is synchronized.
- **Async queue full:** drop-and-count (never block). Alternatives: block (back-pressure — slowing the producer down when the consumer cannot keep up — can stall the app) or drop only low levels. Logback's `AsyncAppender`, by default, starts discarding TRACE/DEBUG/INFO events when the queue is 80% full.
- **Shutdown:** without `close()`, queued events are lost; register a JVM shutdown hook (code the JVM runs just before the program exits) that closes appenders.

- **Recursive logging** (an appender that logs while appending) can loop forever; guard with a thread-local (each thread gets its own separate copy) "already inside" flag.
- **Security:** never let log messages trigger lookups or code; Log4Shell (CVE-2021-44228, December 2021; a CVE is a public id for a known security flaw) came from Log4j 2 interpreting `${jndi:...}` (a lookup that could load code from a remote server) inside logged strings. Treat messages as plain data.
- **Sensitive data:** mask card numbers, tokens and passwords before formatting.

Extensions & follow-up questions

- **"Rolling files."** A `RollingFileAppender` with a `RollingPolicy` (size- or time-based) Strategy: when triggered, close, rename to `app-2026-09-28.log`, open a new file.
- **"Config file."** Parse XML/YAML/properties (common configuration-file text formats) into calls on `LogManager` (set levels, attach appenders). Watching the file lets levels change at runtime.
- **"Filters."** A `Filter` chain per appender (e.g. only `app.payment`, only messages containing "refund") — another Chain of Responsibility.
- **"MDC / request id."** A thread-local map (Mapped Diagnostic Context) copied into each `LogRecord`, so every line of one request carries the same id.
- **"Ship logs to a server."** A network appender behind `AsyncAppender`, batching records.
- **"Structured logging."** Key-value fields in `LogRecord` rendered by `JsonFormatter` for tools like Elasticsearch/Kibana (a search engine and its dashboard, often used to store and browse logs).

Real-world example

SLF4J (Simple Logging Facade for Java) is a facade (the API your code calls); **Logback** and **Log4j 2** are implementations with exactly this model: dot-named loggers in a hierarchy, inherited levels, appenders with layouts/encoders, and *additivity*. Log4j 2's async loggers use the LMAX Disruptor (a lock-free ring buffer: a fixed-size circular array that threads share without taking locks) to hand events to a background thread. `java.util.logging` has the same shape with different names: `Logger`, `Handler` (appender), `Formatter`, `Level`, and `setUseParentHandlers(false)` (additivity off).

In the interview

> **30-second answer:** "Loggers are named and form a dot hierarchy under a root managed by a singleton `LogManager`. A log call first checks the effective level (inherited), builds one immutable record, and passes it up the chain of loggers to their appenders — Chain of Responsibility — until additivity stops it. Each appender has its own threshold and a pluggable formatter (Strategy). `AsyncAppender` is a Decorator with a bounded queue and a writer thread, so the app never waits for disk."

Plan (35 min): clarify (5) → levels + record (4) → logger hierarchy + effective level (6) → appenders + formatters (6) → propagation/additivity (5) → singleton + thread safety (4) → async (5).

Follow-ups with answers:

- *"Why is a disabled debug call cheap?"* → One level comparison before any string is built; use `Supplier` for expensive messages.
- *"How do you make it thread-safe?"* → Immutable records, `CopyOnWriteArrayList` appenders, synchronized sinks, `ConcurrentHashMap` registry, volatile config.
- *"What happens to queued logs on crash?"* → They're lost unless flushed; close on shutdown, or accept the trade-off for speed.

Self-check

1. `app.payment` has no level; `app` is WARN; root is INFO. What is `app.payment`'s effective level? → WARN (nearest ancestor that sets one).
2. What does additivity=false do? → Stops the event from reaching ancestor loggers' appenders.
3. Which pattern lets any appender become async? → Decorator (`AsyncAppender`).
4. Why is `Formatter` separate from `Appender`? → So formats and destinations combine freely (Strategy) without a class per combination.
5. Why resolve the parent before `computeIfAbsent`? → Modifying the same `ConcurrentHashMap` inside its mapping function is not allowed.

6. How does `close()` guarantee nothing queued is lost? → It enqueues a poison pill and joins the writer thread, which drains every record queued before the pill. (Records refused because the queue was full, or appended after `close()`, are not lost silently: they are counted as dropped.)

TL;DR

- Named **logger hierarchy**; effective level inherited; one cheap level check up front.
- **Chain of Responsibility** up to the root, stopped by additivity; **Singleton** `LogManager`.
- **Strategy** formatters, **Template Method** appender base, **Decorator** async appender.
- Immutable `LogRecord`, thread-safe sinks, never crash the app, never block on a full queue.

20.6 Design a Pub-Sub System

Problem & clarifying questions

What it is. Publish-subscribe (pub-sub) is a way for parts of a system to talk **without knowing each other**. A *publisher* sends a message to a named *topic* ("orders"); every *subscriber* of that topic receives it. Everyday analogy: a newspaper. The newspaper prints once; every subscriber gets a copy; the newspaper doesn't know or care who reads it, and new readers can sign up any time.

It is the **Observer pattern** turned into infrastructure: in Observer the subject calls its observers directly; in pub-sub a **broker** in the middle decouples them further (makes each side depend less on the other: by topic, by time, and often by machine). We design a simplified, Kafka-like, in-process (living inside one running program) broker.

Question	Assumed answer
Delivery to how many?	Every subscription gets every message (fan-out).
Order?	Messages are delivered in publish order per subscription.
Guarantee?	**At-least-once**: a message is only acknowledged after the subscriber processes it; failures are retried; duplicates are possible.
Slow subscriber?	Must not slow down the others or the publisher.
Keep messages after delivery?	Yes: an append-only log with **offsets**, so late subscribers can start from the beginning and anyone can replay.
Poison message (always fails)?	After N attempts, send to a dead-letter topic (a side topic that holds messages that could not be processed) and move on.
Push or pull?	Push to a callback, driven by one dispatcher (the loop that hands out messages) per subscription.
Partitions, consumer groups sharing work, persistence to disk, retention?	Out of scope (extensions; each is explained there).

Requirements

Functional

1. Create topics; publish a message (key, payload) to a topic; the broker assigns an **offset** (its position in the topic's log).
2. Subscribe to a topic with a subscription id and a callback, starting from EARLIEST (offset 0) or LATEST (only new messages).
3. Each subscription receives every message in order.
4. A message is acknowledged (offset committed) only after the callback returns normally.
5. Failures are retried with backoff; after `maxAttempts` the message goes to a dead-letter topic.
6. A subscription can **seek** to an offset to replay.
7. Unsubscribe; shut down cleanly.

Non-functional: thread-safe publish from many threads; subscribers isolated from each other; no busy-waiting (a thread spinning in a loop checking "anything yet?" and burning CPU (processor) time; here threads sleep until data arrives); ordering preserved.

Use cases

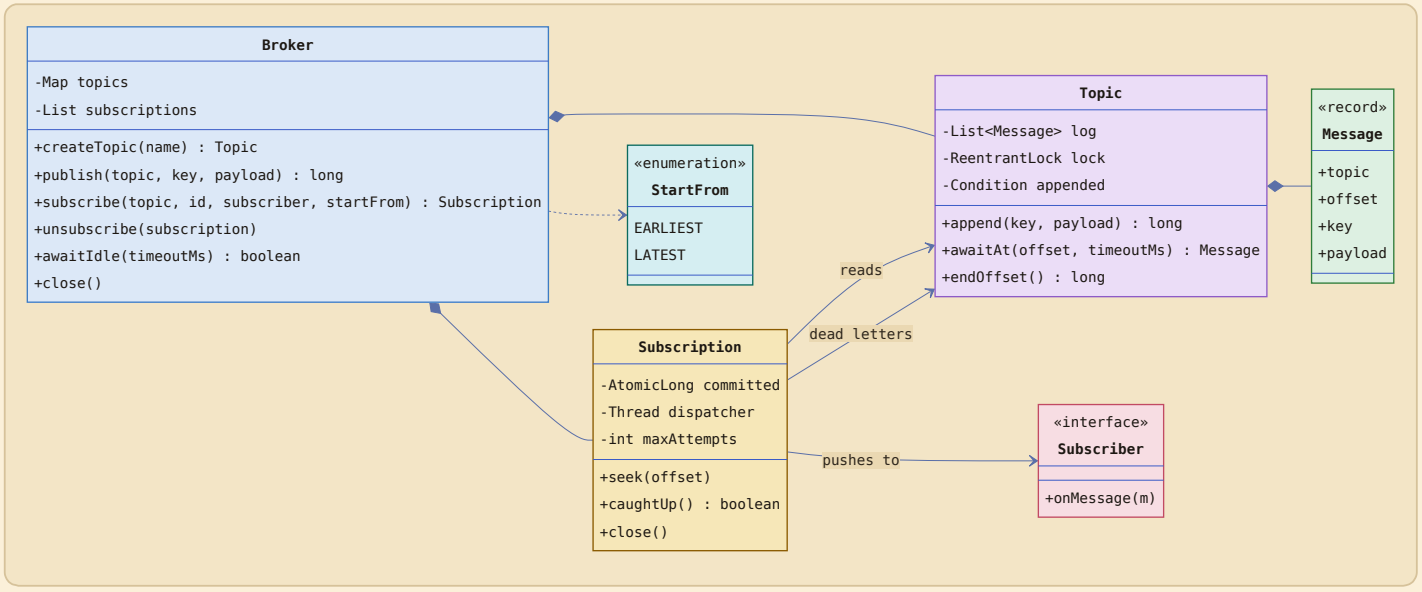
Actors: **Publisher** (publishes), **Subscriber** (subscribes, processes, fails sometimes), **Operator** (creates topics, replays, inspects dead letters), **Broker** (stores, dispatches, retries).

Core entities

Class	Responsibility
`Message` (record)	topic, offset, key, payload. Immutable.
`Topic`	Append-only log (`List<Message>`), guarded by a `ReentrantLock` with a `Condition` (an object a thread can wait on until another thread signals it) "appended" to wake waiting dispatchers.
`Subscriber` (interface)	Callback `onMessage(Message)`; throwing = negative acknowledgement ("I could not process this").
`Subscription`	One subscriber on one topic: committed offset (`AtomicLong`), its **own dispatcher thread**, retry and dead-letter logic, `seek`.
`StartFrom` (enum)	EARLIEST / LATEST.
`Broker`	Topic registry, `publish`, `subscribe`, `unsubscribe`, dead-letter topic, `awaitIdle` (tests), `close`.

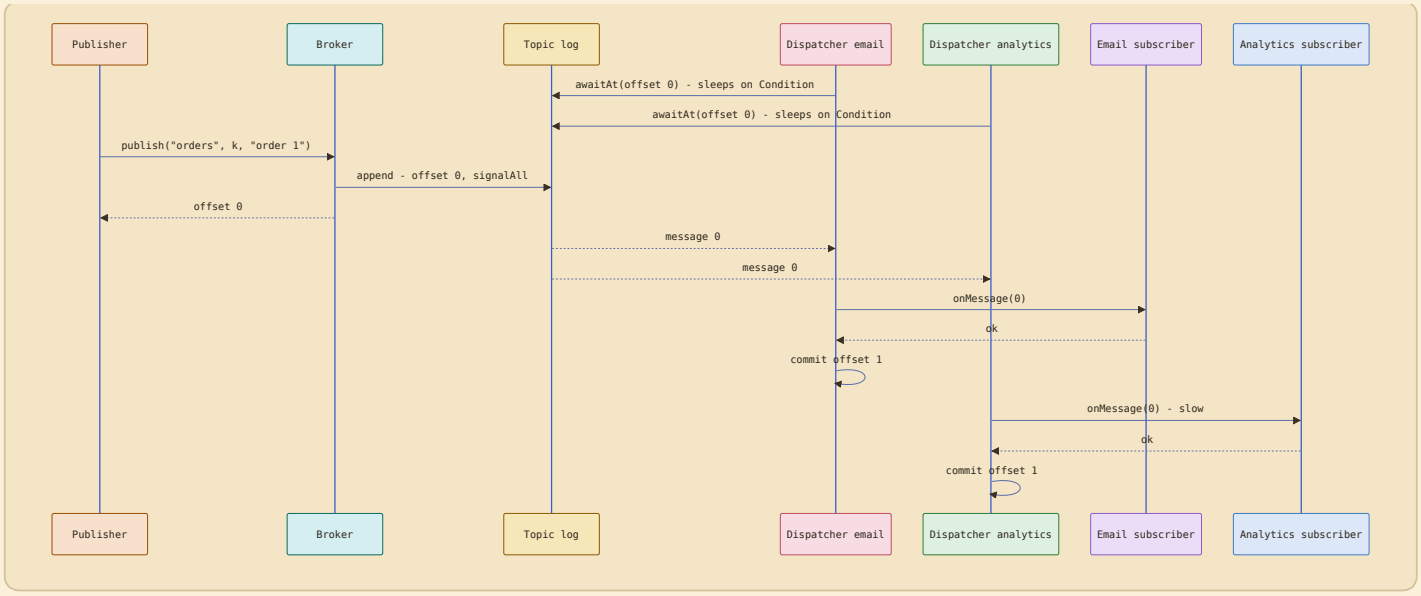
The key idea: messages are not removed when consumed. Each subscription only remembers *how far it has read* (its offset). That gives, almost for free: late subscribers from the beginning, replay (`seek`), and independent progress per subscriber. This is Kafka's model. (Classic queues such as RabbitMQ, a popular open-source message broker, instead delete a message once it is acknowledged.)

Class diagram

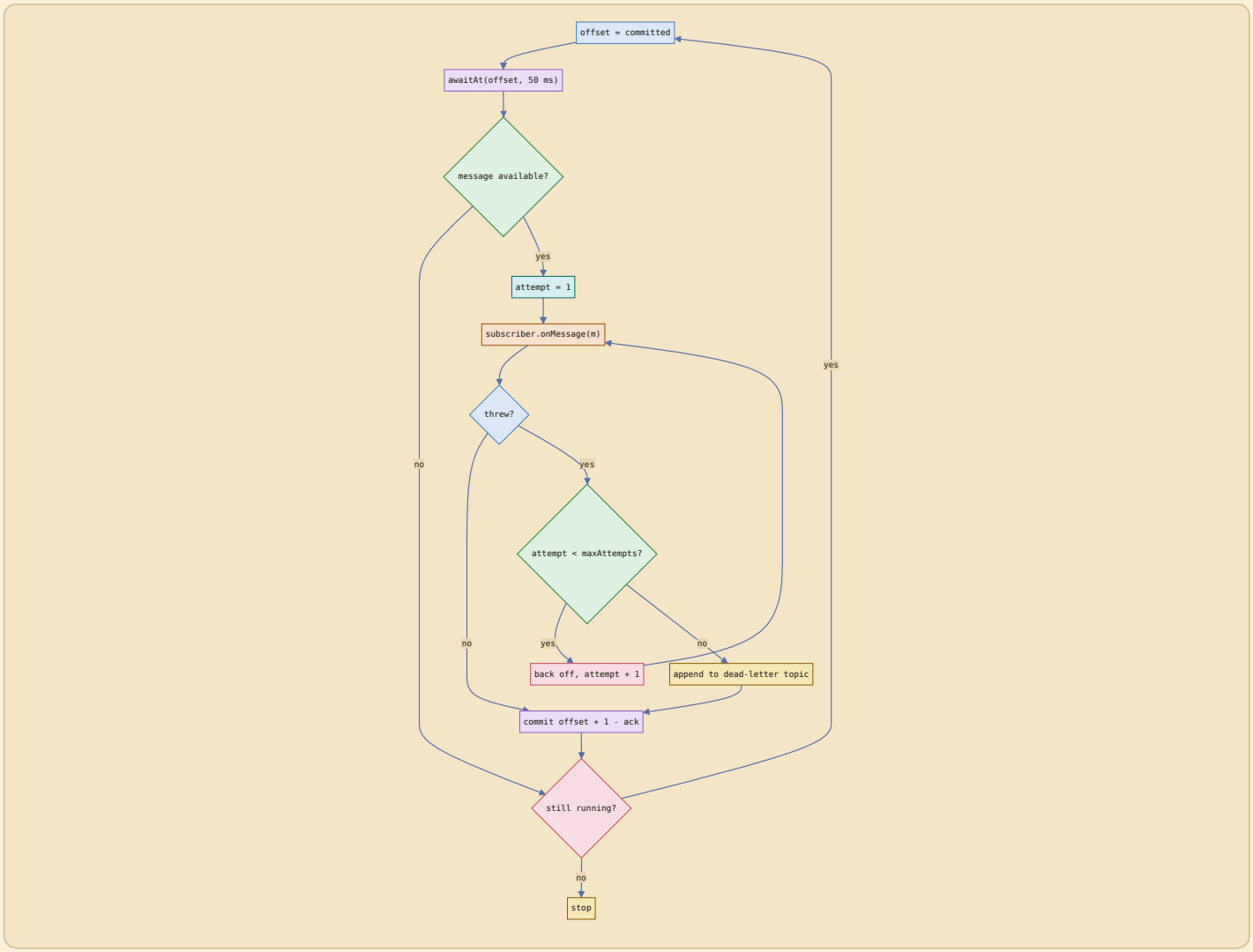


Key flows

Publish and fan-out.



One subscription's dispatcher loop (at-least-once with dead letters).



Design decisions & patterns used

Decision	Why	Alternative rejected
Publish-Subscribe (Observer via a broker)	Publishers and subscribers never reference each other; add a subscriber with zero publisher changes.	Publisher calling each consumer directly (tight coupling).
Append-only log + offsets	Replay, late joiners and independent progress. Ordering is the log order.	Deleting on delivery: no replay; per-subscriber copies: wasteful.
One dispatcher thread per subscription (virtual threads: Java 21's very cheap threads managed by the JVM rather than the operating system)	A slow subscriber only delays itself (see "analytics" in the output). Order within a subscription is kept because only one thread delivers for it. Virtual threads make one-thread-per-subscription cheap.	One shared thread delivering to all subscribers in turn (the slowest blocks everyone); a thread pool (a fixed set of reusable threads) per message (loses ordering).
`Lock` + `Condition` wait, with timeout	No busy-waiting; the timeout lets the loop notice `close()` and `seek()`. `while` loop guards against spurious wake-ups (a waiting thread may wake up although nobody signalled it).	`Thread.sleep` polling (repeatedly checking; wastes CPU and adds latency — the delay before something is handled).
Ack after processing (commit after `onMessage`)	At-least-once: a crash mid-processing re-delivers.	Commit before processing = at-most-once (each message is delivered once or not at all; can lose messages).
Dead-letter topic	One poison message cannot block the subscription forever; operators can inspect and re-publish.	Infinite retries (the subscription is stuck).
`compareAndSet` when committing	If `seek()` ran during delivery, the seek wins instead of being overwritten.	Plain `set`.

Delivery guarantees in one table:

Guarantee	How	Risk
At-most-once	Commit offset, then process	A crash loses the message
At-least-once (ours)	Process, then commit	A crash or retry duplicates the message → make subscribers **idempotent**
Exactly-once	At-least-once + deduplication (spotting and dropping repeats) / transactions that commit output and offset together	Complex; Kafka offers it within its own transactions

Java implementation

```java
// File: code/lld/ch20/pubsub/Topic.java
package lld.ch20.pubsub;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

/** One published message. The offset is its position in the topic's log. */
record Message(String topic, long offset, String key, String payload) {}

/**
 * An append-only log (like one Kafka partition). Messages are never removed by consumption:
 * each subscription just remembers how far it has read (its offset).
 */
final class Topic {
    private final String name;
    private final List<Message> log = new ArrayList<>();
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition appended = lock.newCondition();

    Topic(String name) { this.name = name; }

    String name() { return name; }

    long append(String key, String payload) {
        lock.lock();
        try {
            long offset = log.size();
            log.add(new Message(name, offset, key, payload));
            appended.signalAll();                  // wake every waiting dispatcher
            return offset;
        } finally {
            lock.unlock();
        }
    }

    /** Returns the message at 'offset', waiting up to the timeout for it to be published. */
    Message awaitAt(long offset, long timeoutMs) throws InterruptedException {
        lock.lock();
        try {
            long nanos = TimeUnit.MILLISECONDS.toNanos(timeoutMs);
            while (offset >= log.size()) {         // loop: guards against spurious wake-ups
                if (nanos <= 0) return null;
                nanos = appended.awaitNanos(nanos);
            }
            return log.get((int) offset);
        } finally {
            lock.unlock();
        }
    }

    long endOffset() {
        lock.lock();
        try {
            return log.size();
        } finally {
            lock.unlock();
        }
    }
}
```

```java
// File: code/lld/ch20/pubsub/Broker.java
package lld.ch20.pubsub;

import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicLong;

/** Observer (push style): the broker calls this for every message, in order. Throw = negative ack. */
interface Subscriber {
    void onMessage(Message m) throws Exception;
}

enum StartFrom { EARLIEST, LATEST }

/**
 * One subscriber's view of one topic: its committed offset and its own dispatcher thread.
 * Delivery is at-least-once and in order. A message is acknowledged (offset committed)
 * only after onMessage returns normally. After maxAttempts failures it goes to the dead-letter topic.
 */
final class Subscription {
    private final String id;
    private final Topic topic;
    private final Subscriber subscriber;
    private final int maxAttempts;
    private final Topic deadLetters;
    private final AtomicLong committed;        // next offset to deliver
    private volatile boolean running = true;
    private final Thread dispatcher;

    Subscription(String id, Topic topic, Subscriber subscriber, long startOffset, int maxAttempts, Topic
deadLetters) {
        this.id = id;
        this.topic = topic;
        this.subscriber = subscriber;
        this.maxAttempts = maxAttempts;
        this.deadLetters = deadLetters;
        this.committed = new AtomicLong(startOffset);
        this.dispatcher = Thread.ofVirtual().name("dispatch-" + id).start(this::loop);
    }

    private void loop() {
        try {
            while (running) {
                long offset = committed.get();
                Message m = topic.awaitAt(offset, 50);
                if (m == null) continue;              // timed out: re-check 'running' and any seek()
                deliver(m);
                committed.compareAndSet(offset, offset + 1); // ack; a concurrent seek() wins
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void deliver(Message m) throws InterruptedException {
        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            try {
                subscriber.onMessage(m);
                return;
            } catch (Exception e) {
                if (attempt == maxAttempts) {
                    deadLetters.append(id, m.topic() + "@" + m.offset() + " " + m.payload() + " -> " +
e.getMessage());
                    return;                           // skip it so one poison message cannot block the rest
                }
                Thread.sleep(5L * attempt);           // simple backoff
```

```
            }
        }
    }

    String id() { return id; }
    long committedOffset() { return committed.get(); }
    boolean caughtUp() { return committed.get() >= topic.endOffset(); }

    /** Replay (or skip) by moving the offset - possible because the log keeps every message. */
    void seek(long offset) { committed.set(offset); }

    void close() throws InterruptedException {
        running = false;
        dispatcher.join();
    }
}

/** Topic registry + publish + subscribe. Publishers never know who the subscribers are. */
final class Broker implements AutoCloseable {
    static final String DLQ = "__dead_letters";
    private final Map<String, Topic> topics = new ConcurrentHashMap<>();
    private final List<Subscription> subscriptions = new CopyOnWriteArrayList<>();

    Broker() { createTopic(DLQ); }

    Topic createTopic(String name) { return topics.computeIfAbsent(name, Topic::new); }

    long publish(String topic, String key, String payload) {
        Topic t = topics.get(topic);
        if (t == null) throw new IllegalArgumentException("no topic " + topic);
        return t.append(key, payload);
    }

    Subscription subscribe(String topic, String subscriptionId, Subscriber s, StartFrom from) {
        Topic t = topics.get(topic);
        if (t == null) throw new IllegalArgumentException("no topic " + topic);
        long start = from == StartFrom.EARLIEST ? 0 : t.endOffset();
        Subscription sub = new Subscription(subscriptionId, t, s, start, 3, topics.get(DLQ));
        subscriptions.add(sub);
        return sub;
    }

    void unsubscribe(Subscription s) throws InterruptedException {
        subscriptions.remove(s);
        s.close();
    }

    /** Test/shutdown helper: wait until every subscription has acknowledged everything. */
    boolean awaitIdle(long timeoutMs) throws InterruptedException {
        long deadline = System.currentTimeMillis() + timeoutMs;
        while (System.currentTimeMillis() < deadline) {
            if (subscriptions.stream().allMatch(Subscription::caughtUp)) return true;
            Thread.sleep(5);
        }
        return false;
    }

    Topic topic(String name) { return topics.get(name); }

    /** Stops every dispatcher. Declared without InterruptedException so try-with-resources stays simple. */
    @Override
    public void close() {
        try {
            for (Subscription s : subscriptions) s.close();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();      // keep the interrupt visible to the caller
        }
    }
}
```

```java
// File: code/lld/ch20/pubsub/Main.java
package lld.ch20.pubsub;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

public class Main {

    /** Records what it received; can be told to fail some messages. */
    static final class RecordingSubscriber implements Subscriber {
        final List<String> received = Collections.synchronizedList(new ArrayList<>());
        final Set<Long> failOnceAt = ConcurrentHashMap.newKeySet();
        final String alwaysFailPayload;
        final long delayMs;

        RecordingSubscriber(long delayMs, String alwaysFailPayload) {
            this.delayMs = delayMs;
            this.alwaysFailPayload = alwaysFailPayload;
        }

        @Override
        public void onMessage(Message m) throws Exception {
            if (delayMs > 0) Thread.sleep(delayMs);
            if (m.payload().equals(alwaysFailPayload)) throw new Exception("cannot parse");
            if (failOnceAt.remove(m.offset())) throw new Exception("transient error");
            received.add(m.offset() + ":" + m.payload());
        }
    }

    public static void main(String[] args) throws Exception {
        try (Broker broker = new Broker()) {
            broker.createTopic("orders");

            RecordingSubscriber email = new RecordingSubscriber(0, null);
            RecordingSubscriber analytics = new RecordingSubscriber(20, null);       // slow consumer
            RecordingSubscriber inventory = new RecordingSubscriber(0, "BAD-JSON"); // poison message
            inventory.failOnceAt.add(1L);                                           // transient failure

            broker.subscribe("orders", "email", email, StartFrom.EARLIEST);
            broker.subscribe("orders", "analytics", analytics, StartFrom.EARLIEST);
            broker.subscribe("orders", "inventory", inventory, StartFrom.EARLIEST);

            System.out.println("== 1. Fan-out: every subscriber gets every message, in order");
            for (String p : List.of("order#1", "order#2", "BAD-JSON", "order#3"))
                broker.publish("orders", "k", p);
            System.out.println("  all acknowledged: " + broker.awaitIdle(5_000));
            System.out.println("  email     " + email.received);
            System.out.println("  analytics " + analytics.received + "  (slow, but did not slow the others)");
            System.out.println("  inventory " + inventory.received + "  (offset 1 retried, offset 2 dead-
lettered)");
            Message dead = broker.topic(Broker.DLQ).awaitAt(0, 1_000);
            System.out.println("  dead letter: " + dead.payload());

            System.out.println("== 2. A late subscriber chooses where to start");
            RecordingSubscriber auditFromStart = new RecordingSubscriber(0, null);
            RecordingSubscriber onlyNew = new RecordingSubscriber(0, null);
            broker.subscribe("orders", "audit", auditFromStart, StartFrom.EARLIEST);
            Subscription latest = broker.subscribe("orders", "live", onlyNew, StartFrom.LATEST);
            broker.publish("orders", "k", "order#4");
            broker.awaitIdle(5_000);
            System.out.println("  audit (EARLIEST) " + auditFromStart.received);
            System.out.println("  live  (LATEST)   " + onlyNew.received);

            System.out.println("== 3. Replay: seek the 'live' subscription back to offset 3");
            latest.seek(3);
```

```
            broker.awaitIdle(5_000);
            System.out.println("  live after replay " + onlyNew.received + " (order#4 seen twice: at-least-once)");

            System.out.println("== 4. 4 publisher threads x 250 messages on a new topic");
            broker.createTopic("clicks");
            RecordingSubscriber clicks = new RecordingSubscriber(0, null);
            broker.subscribe("clicks", "counter", clicks, StartFrom.EARLIEST);
            List<Thread> pubs = new ArrayList<>();
            for (int t = 0; t < 4; t++) {
                int id = t;
                pubs.add(Thread.ofPlatform().start(() -> {
                    for (int i = 0; i < 250; i++) broker.publish("clicks", "user" + id, "c" + id + "-" + i);
                }));
            }
            for (Thread t : pubs) t.join();
            broker.awaitIdle(5_000);
            boolean inOrder = true;
            for (int i = 0; i < clicks.received.size(); i++)
                inOrder &= clicks.received.get(i).startsWith(i + ":");
            System.out.println("  received " + clicks.received.size() + " messages, offsets strictly in order: " +
inOrder);
        }
    }
}
```

Walking through the code.

- `Topic.append` adds to the list and calls `signalAll()` under the lock, waking every dispatcher waiting for new data.
- `Topic.awaitAt` waits on the `Condition` in a `while` loop with `awaitNanos` (correct handling of spurious wake-ups and timeouts), then returns the message at that offset.
- `Subscription.loop` delivers one message at a time and only then commits `offset + 1` with `compareAndSet`, so a concurrent `seek()` wins.
- `deliver` retries with a small backoff and, after `maxAttempts`, writes a description to the `__dead_letters` topic and moves on.
- `Broker.awaitIdle` is a test helper: it waits until every subscription's committed offset has reached its topic's end.

Output:

```
== 1. Fan-out: every subscriber gets every message, in order
  all acknowledged: true
  email     [0:order#1, 1:order#2, 2:BAD-JSON, 3:order#3]
  analytics [0:order#1, 1:order#2, 2:BAD-JSON, 3:order#3]  (slow, but did not slow the others)
  inventory [0:order#1, 1:order#2, 3:order#3]  (offset 1 retried, offset 2 dead-lettered)
  dead letter: orders@2 BAD-JSON -> cannot parse
== 2. A late subscriber chooses where to start
  audit (EARLIEST) [0:order#1, 1:order#2, 2:BAD-JSON, 3:order#3, 4:order#4]
  live  (LATEST)   [4:order#4]
== 3. Replay: seek the 'live' subscription back to offset 3
  live after replay [4:order#4, 3:order#3, 4:order#4] (order#4 seen twice: at-least-once)
== 4. 4 publisher threads x 250 messages on a new topic
  received 1000 messages, offsets strictly in order: true
```

Points to notice: the slow "analytics" subscriber still got everything, and the other subscribers were not delayed; "inventory" retried offset 1 (a transient — temporary — failure) and dead-lettered offset 2 (the poison message) while still receiving offset 3; after `seek(3)`, "live" saw order#4 twice — that is what at-least-once means. The 1000 messages from 4 concurrent publishers got unique offsets and arrived strictly in order.

Concurrency & edge cases

- **Concurrent publishers:** `append` is under a lock, so offsets are unique and gap-free.
- **Readers vs writers:** dispatchers only read under the same lock; messages are immutable records, safe to share.
- **Unbounded memory:** the log grows forever; real brokers apply **retention** (delete segments — the chunks the log is stored in — older than N days or beyond N bytes).

- **Slow subscriber falling far behind:** it is fine here (it just reads older offsets); with retention it may fall off the end — detect "offset out of range" and reset.
- **Subscriber that blocks forever:** add a processing timeout. It stalls only its own subscription, because every subscription has its own dispatcher thread. (One Java 21 caveat: a virtual thread that blocks while inside a `synchronized` block *pins* its carrier platform thread (the real operating-system thread it runs on); many such subscribers could exhaust the carriers. Prefer `ReentrantLock` in subscriber code, or use platform threads.)
- **Duplicates:** at-least-once means subscribers must be idempotent (e.g. remember processed order ids).
- **Shutdown:** `close()` flips `running` and joins each dispatcher (waits for its thread to finish); the 50 ms wait timeout bounds how long that takes.

Extensions & follow-up questions

- **"Consumer groups (share the work)."** Split a topic into **partitions** (several logs); messages with the same key go to the same partition (`hash(key) % n`), keeping per-key order. A **consumer group** is a set of consumers that split one topic's work between them; a group's members each own some partitions; the committed offset is per (group, partition). That is how Kafka scales consumers.
- **"Persist to disk."** Append to segment files; keep an index (a lookup table) from offset to file position; store committed offsets in a special topic (Kafka uses `__consumer_offsets`).
- **"Pull instead of push."** Expose `poll(subscriptionId, maxMessages)` returning messages from the committed offset and a `commit(offset)` call; the client controls its pace (back-pressure for free).
- **"Message filtering."** Subscribe with a predicate (a yes/no test, e.g. only `region = EU`); dispatcher skips non-matching messages but still commits them.
- **"Wildcard topics."** Subscribe to `orders.*` (`*` is a wildcard that matches any name part); the broker matches topic names on creation.
- **"Delayed messages / retry topics."** Re-publish failed messages to `orders.retry.5s` consumed after a delay, instead of blocking the dispatcher with sleeps.

Real-world example

Apache Kafka is exactly this model at scale: topics split into partitions, each an append-only log; consumers track offsets; consumer groups share partitions; ordering is per partition; messages are retained by time/size rather than deleted on read, so services can replay history. **Google Cloud Pub/Sub** and **AWS SNS + SQS** (Amazon's notification and queue services) offer managed fan-out with acknowledgements and **dead-letter** topics/queues. In Java, Spring Kafka's (Spring is the most popular Java application framework) `DeadLetterPublishingRecoverer` publishes a record that keeps failing to a dead-letter topic after the retries are used up — the same idea as our `__dead_letters`. Inside one JVM, Guava's `EventBus` and Spring's `ApplicationEventPublisher` are simple in-process pub-sub.

In the interview

> **30-second answer:** "Each topic is an append-only log; publishing appends and returns an offset. Each subscription keeps a committed offset and has its own dispatcher thread that waits on a condition for new data, calls the subscriber, and commits only after success — at-least-once and in order. Failures retry with backoff, then go to a dead-letter topic. Because messages aren't deleted on read, late subscribers can start at the beginning and anyone can replay by seeking. To scale, split topics into partitions by key and let consumer groups share partitions."

Plan (35–40 min): clarify guarantees (5) → topic log + offsets (6) → subscription + dispatcher + ack (8) → retries + DLQ (dead-letter queue) (5) → thread safety and ordering (6) → partitions / consumer groups / persistence (5).

Follow-ups with answers:

- *"How do you keep a slow consumer from slowing others?"* → One dispatcher per subscription; the log is shared, progress is not.
- *"Exactly-once?"* → Hard; at-least-once + idempotent consumers is the practical answer.
- *"How is order guaranteed?"* → One writer (the lock) assigns offsets; one dispatcher per subscription delivers in offset order. With partitions, order is per partition (per key).

Self-check

1. What is an offset? → A message's position in a topic's log; subscribers remember the next one to read.

2. Commit before or after processing for at-least-once? → After.
3. Why a dead-letter topic? → So one poison message cannot block a subscription forever.
4. Why does replay work here but not in a delete-on-ack queue? → Messages stay in the log; seeking just moves the offset.
5. Why `while` around `Condition.await`? → Spurious wake-ups; re-check the condition every time.
6. How does Kafka let several consumers share one topic's work while keeping order per key? → Partitions by key hash; each partition consumed by one group member.

TL;DR

- **Pub-sub = Observer through a broker**; publishers and subscribers are decoupled.
- **Append-only log + per-subscription offsets** → fan-out, late joiners, replay.
- **One dispatcher per subscription**: isolation + order; **ack after processing** = at-least-once.
- Retries with backoff, then **dead-letter**; subscribers must be **idempotent**.
- Scale: partitions by key, consumer groups, retention, persisted offsets (Kafka).

20.7 Design a Rate Limiter

Problem & clarifying questions

`What it is.` A rate limiter decides whether a request is allowed right now, so one client can't overwhelm a service (by accident or on purpose). Everyday analogy: a turnstile that lets through at most 10 people per minute per ticket type. Over the limit, the answer is "come back in 6 seconds" — in HTTP, status **429 Too Many Requests** with a `Retry-After` header (a header is a named line of metadata sent with an HTTP response).

Chapter 13 (§13.11) already built the concurrency core: a thread-safe token bucket (a bucket of tokens refilled at a steady rate; each request spends one, and an empty bucket means "wait") with `synchronized` and a lock-free CAS version (CAS = compare-and-set: update a value only if it still holds what you last read, as one hardware-level step, so no lock is needed), and compared the algorithms. This section is about the **object-oriented design around it**: pluggable algorithms, rules per API and per user tier (a plan level such as free or paid), a factory, combined limits, and a clean decision object.

Question	Assumed answer
Limit by what?	Per (user, API). Could also be IP (the caller's network address) or API key (a secret string that identifies the calling app) — same design, different key.
Different limits?	Yes: per API and per tier (FREE vs PREMIUM); a default for everything else.
Algorithms?	Token bucket, fixed window, sliding window log, sliding window counter (all compared in the table under Key flows) — swappable per rule.
Several limits at once?	Yes, e.g. "2 per 10 s **and** 3 per minute" on `/login`.
Response?	Allowed / denied + remaining + retry-after.
Single process or distributed (spread over several machines)?	Single process here; distributed in extensions.

Requirements

Functional

1. `check(userId, api)` → `Decision(allowed, remaining, retryAfter)`.
2. Rules configured per API and tier; fallback default rule.
3. Each (user, api) gets its **own** limiter instance, created on first use.
4. Algorithms pluggable per rule.
5. Several rules can apply together (all must allow).

Non-functional: O(1) (or close) per check; thread-safe (exactly N allowed under any concurrency); injectable clock for tests; memory bounded (discussed).

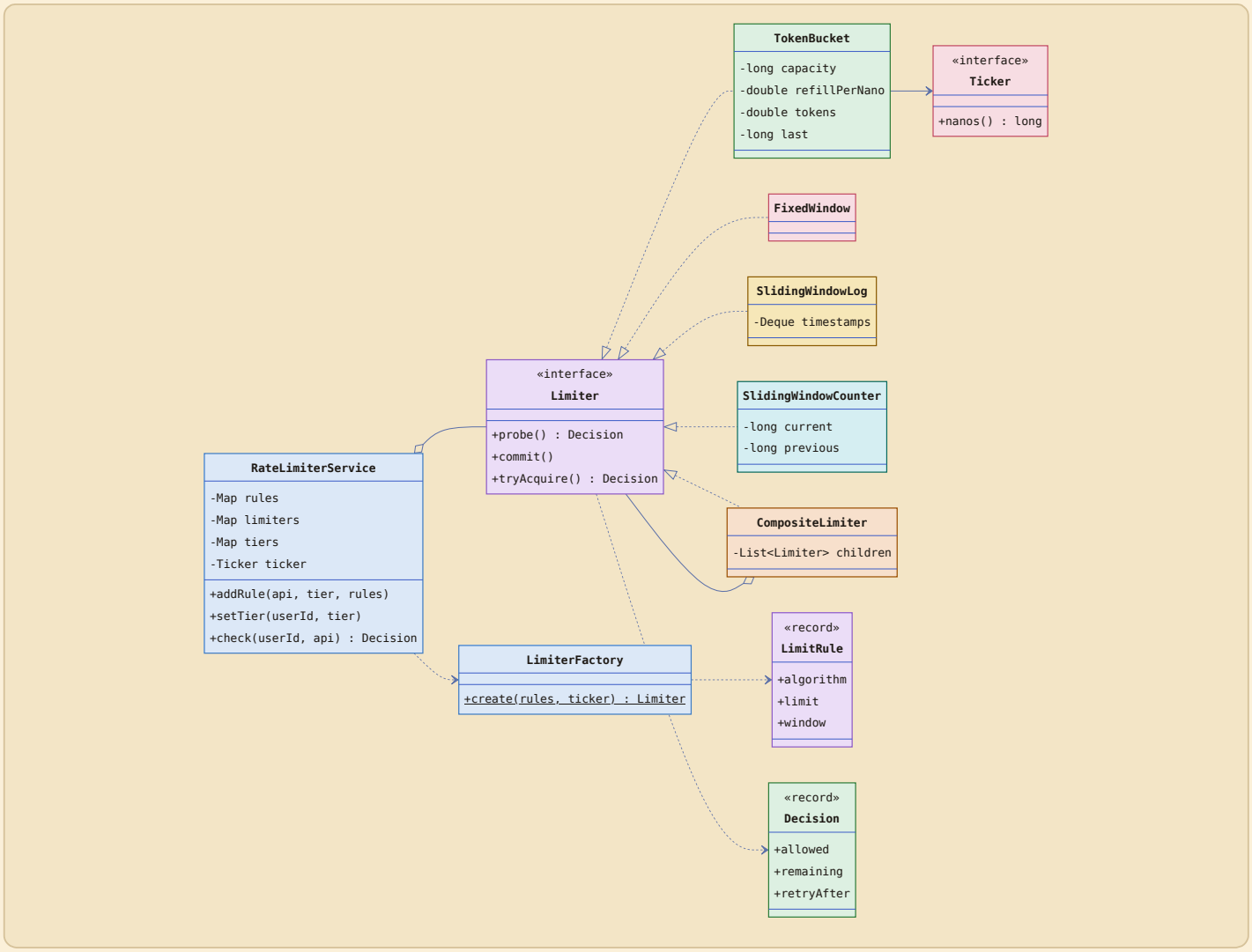
Use cases

Actors: **Client** (sends requests), **API gateway / filter** (the front door that every request passes through; calls `check` before the handler — the code that does the real work), **Operator** (configures rules and tiers).

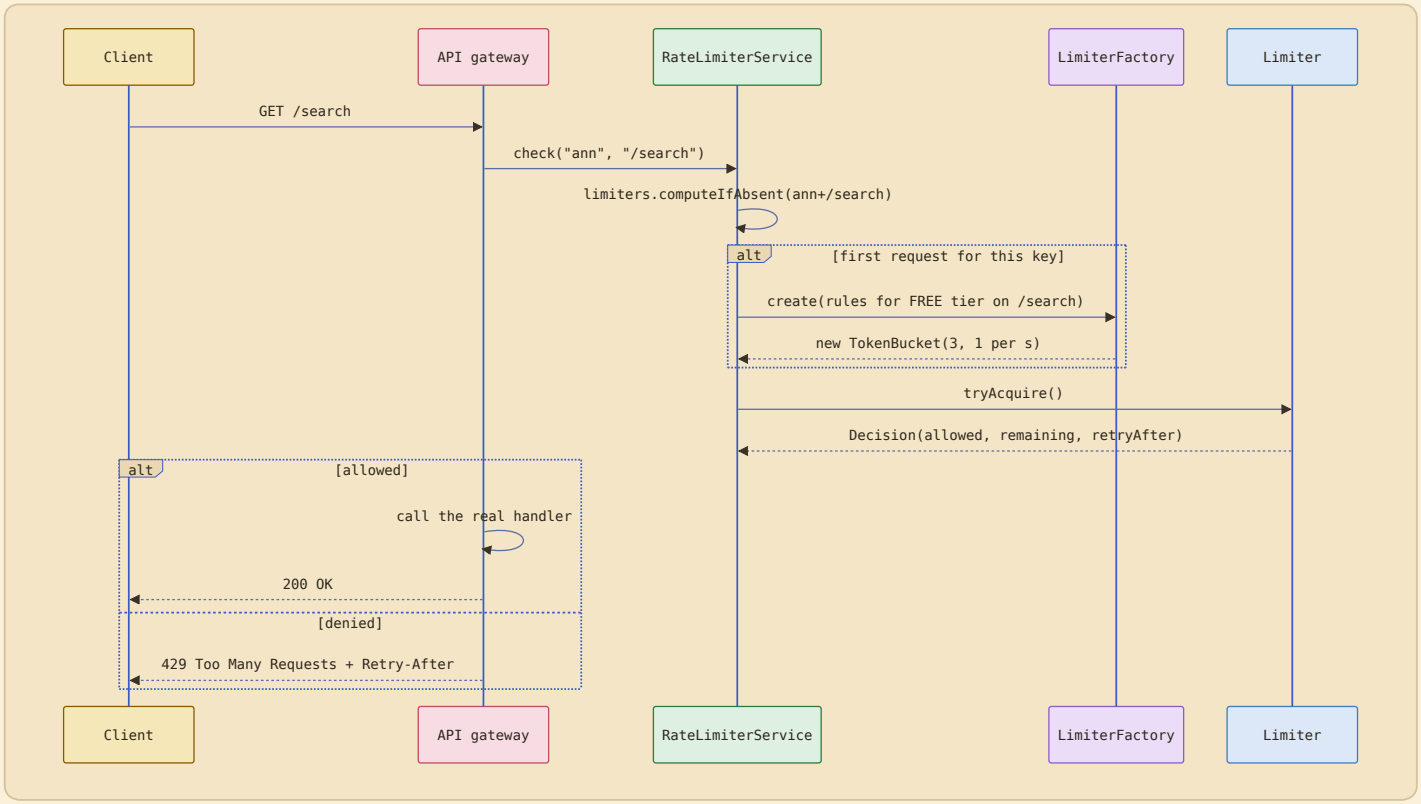
Core entities

Class	Responsibility
`Limiter` (interface)	**Strategy**: `probe()` (would one more request pass? consumes nothing), `commit()` (record it), and `tryAcquire()` = both in one atomic step → `Decision`. One instance per key.
`TokenBucket`, `FixedWindow`, `SlidingWindowLog`, `SlidingWindowCounter`	The algorithms (each `synchronized`).
`CompositeLimiter`	**Composite**: allowed only if every child allows; probes all children, then commits to all, under one lock, so a denied request consumes nothing.
`Decision` (record)	allowed, remaining, retryAfter.
`LimitRule` (record)	algorithm, limit, window.
`Algorithm`, `Tier` (enums)	Configuration values.
`LimiterFactory`	**Simple Factory**: rule(s) → `Limiter` (single or composite).
`RateLimiterService`	**Facade**: resolves the user's tier and the API's rules, keeps one limiter per (user, api) in a `ConcurrentHashMap`.
`Ticker`	Injected time source (nanoseconds).

Class diagram



Key flows



The four algorithms side by side (limit N per window W):

Algorithm	State per key	Bursts	Accuracy	Notes
Token bucket	tokens, last refill time	Up to capacity	Good	Most common for APIs; smooth refill
Fixed window counter	window start, count	**Up to 2N across a boundary**	Poor at edges	Simplest
Sliding window log	one timestamp per allowed request	None beyond N	Exact	Memory O(N) per key
Sliding window counter	2 counters	Approximate	Good	Estimate = previous × overlap + current
(Leaky bucket)	a queue	Smooths output to a constant rate	—	Traffic shaping (evening out the flow); not implemented here

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy (`Limiter`)	Algorithms differ in accuracy/memory; pick per rule. New algorithm = new class.	`switch` on algorithm inside `check`.
Simple Factory (`LimiterFactory`)	One place turns config into objects; service never names concrete classes.	`new TokenBucket(...)` in the service.
Composite (`CompositeLimiter`)	"10/s and 1000/day" is just a limiter made of limiters; callers don't know. **Check all, then commit all**, so a request denied by one child never uses up a slot in another.	Special-casing multiple rules in the service; or calling each child's `tryAcquire` in turn (a later denial would leave earlier children charged for a request that never ran).
Facade (`RateLimiterService`)	Gateway code asks one question.	Gateway juggling tiers, rules and limiters.
One limiter per key via `computeIfAbsent`	Exactly one limiter per (user, api), even when the first two requests race.	`get` then `put` (two limiters could be created and one's counts lost).
`Decision` with `remaining` and `retryAfter`	Maps directly to HTTP headers (`X-RateLimit-Remaining`, `Retry-After`).	`boolean`.
Injected `Ticker`	Tests move time instantly (the demo never sleeps).	`System.nanoTime()` inside algorithms.
Each algorithm `synchronized`; `tryAcquire` = `probe` + `commit` under the limiter's lock	Refill + check + take must be one atomic step (see §13.11 for the lock-free version).	Unsynchronized fields (two threads take the last token).

Java implementation

```java
// File: code/lld/ch20/ratelimiter/Limiters.java
package lld.ch20.ratelimiter;

import java.time.Duration;
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;

/** Where time comes from (nanoseconds). Injected so tests do not sleep. */
interface Ticker { long nanos(); }

/** The answer to one request: allowed or not, what is left, and when to retry (HTTP 429 Retry-After). */
record Decision(boolean allowed, long remaining, Duration retryAfter) {
    static Decision allow(long remaining) { return new Decision(true, remaining, Duration.ZERO); }
    static Decision deny(Duration retryAfter) { return new Decision(false, 0, retryAfter); }
}

/**
 * Strategy: one algorithm, one instance per key (user, API key, IP...). Must be thread-safe.
 * Two-phase so that limits can be combined without side effects:
 * probe() answers "would one more request be allowed now?" WITHOUT using up anything,
 * commit() records the request. Both must be called while holding this object's lock.
 */
interface Limiter {
    Decision probe();

    void commit();

    /** Check and take in ONE atomic step (probe + commit under this limiter's lock). */
    default Decision tryAcquire() {
        synchronized (this) {
            Decision d = probe();
            if (d.allowed()) commit();
            return d;
        }
    }
}

/** Token bucket: bursts up to 'capacity', refills at 'perSecond'. Lazy refill, no background thread. */
final class TokenBucket implements Limiter {
    private final long capacity;
    private final double refillPerNano;
    private final Ticker ticker;
    private double tokens;
    private long last;

    TokenBucket(long capacity, double perSecond, Ticker ticker) {
        this.capacity = capacity;
        this.refillPerNano = perSecond / 1e9;
        this.ticker = ticker;
        this.tokens = capacity;
        this.last = ticker.nanos();
    }

    @Override
    public synchronized Decision probe() {
        long now = ticker.nanos();
        tokens = Math.min(capacity, tokens + (now - last) * refillPerNano); // refilling is safe to do early
        last = now;
        if (tokens >= 1) return Decision.allow((long) (tokens - 1));
        long waitNanos = refillPerNano == 0 ? Long.MAX_VALUE : (long) Math.ceil((1 - tokens) / refillPerNano);
        return Decision.deny(Duration.ofNanos(waitNanos));
    }

    @Override
    public synchronized void commit() { tokens -= 1; }
```

```
}

/** Fixed window counter: simplest; allows up to 2x the limit across a window boundary. */
final class FixedWindow implements Limiter {
    private final long limit;
    private final long windowNanos;
    private final Ticker ticker;
    private long windowStart;
    private long count;

    FixedWindow(long limit, Duration window, Ticker ticker) {
        this.limit = limit;
        this.windowNanos = window.toNanos();
        this.ticker = ticker;
    }

    @Override
    public synchronized Decision probe() {
        long now = ticker.nanos();
        long currentWindow = now - Math.floorMod(now, windowNanos);
        if (currentWindow != windowStart) {
            windowStart = currentWindow;
            count = 0;
        }
        if (count < limit) return Decision.allow(limit - count - 1);
        return Decision.deny(Duration.ofNanos(windowStart + windowNanos - now));
    }

    @Override
    public synchronized void commit() { count++; }
}

/** Sliding window log: exact, but stores one timestamp per allowed request. */
final class SlidingWindowLog implements Limiter {
    private final long limit;
    private final long windowNanos;
    private final Ticker ticker;
    private final Deque<Long> log = new ArrayDeque<>();
    private long probedAt;                       // commit() records the time the probe saw

    SlidingWindowLog(long limit, Duration window, Ticker ticker) {
        this.limit = limit;
        this.windowNanos = window.toNanos();
        this.ticker = ticker;
    }

    @Override
    public synchronized Decision probe() {
        long now = ticker.nanos();
        probedAt = now;
        while (!log.isEmpty() && log.peekFirst() <= now - windowNanos) log.pollFirst();
        if (log.size() < limit) return Decision.allow(limit - log.size() - 1);
        return Decision.deny(Duration.ofNanos(log.peekFirst() + windowNanos - now));
    }

    @Override
    public synchronized void commit() { log.addLast(probedAt); }
}

/** Sliding window counter: two counters, weights the previous window by how much of it still overlaps. */
final class SlidingWindowCounter implements Limiter {
    private final long limit;
    private final long windowNanos;
    private final Ticker ticker;
    private long currentStart;
    private long current, previous;

    SlidingWindowCounter(long limit, Duration window, Ticker ticker) {
        this.limit = limit;
```

```
        this.windowNanos = window.toNanos();
        this.ticker = ticker;
    }

    @Override
    public synchronized Decision probe() {
        long now = ticker.nanos();
        long start = now - Math.floorMod(now, windowNanos);
        if (start != currentStart) {
            previous = (start - currentStart == windowNanos) ? current : 0; // older than one window: forget it
            current = 0;
            currentStart = start;
        }
        double overlap = 1.0 - (double) (now - currentStart) / windowNanos;
        double estimate = previous * overlap + current;
        if (estimate + 1 <= limit) return Decision.allow((long) (limit - estimate - 1));
        return Decision.deny(Duration.ofNanos(currentStart + windowNanos - now));
    }

    @Override
    public synchronized void commit() { current++; }
}

/**
 * Composite: allowed only if EVERY child allows (e.g. 10/second AND 1000/day).
 * Check-all-then-commit under one lock: a request that ANY child denies uses up NOTHING,
 * so a denied request can never eat a slot in an earlier child.
 * The children are created for this composite only (by LimiterFactory) and never shared,
 * so holding the composite's lock is enough to make probe-all + commit-all atomic.
 */
final class CompositeLimiter implements Limiter {
    private final List<Limiter> children;

    CompositeLimiter(List<Limiter> children) { this.children = List.copyOf(children); }

    @Override
    public synchronized Decision probe() {
        long remaining = Long.MAX_VALUE;
        for (Limiter l : children) {
            Decision d;
            synchronized (l) { d = l.probe(); }
            if (!d.allowed()) return d;           // first denial wins; nothing has been consumed
            remaining = Math.min(remaining, d.remaining());
        }
        return Decision.allow(remaining);
    }

    @Override
    public synchronized void commit() {
        for (Limiter l : children) synchronized (l) { l.commit(); }
    }
}
```

```java
// File: code/lld/ch20/ratelimiter/RateLimiterService.java
package lld.ch20.ratelimiter;

import java.time.Duration;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

enum Tier { FREE, PREMIUM }

enum Algorithm { TOKEN_BUCKET, FIXED_WINDOW, SLIDING_LOG, SLIDING_COUNTER }

/** Configuration: which algorithm and how many requests per window (token bucket: burst = limit). */
record LimitRule(Algorithm algorithm, long limit, Duration window) {}

/** Factory: turns a rule into a fresh Limiter object. The only place that knows the concrete classes. */
final class LimiterFactory {
    private LimiterFactory() {}

    static Limiter create(List<LimitRule> rules, Ticker ticker) {
        if (rules.size() == 1) return create(rules.get(0), ticker);
        return new CompositeLimiter(rules.stream().map(r -> create(r, ticker)).toList());
    }

    static Limiter create(LimitRule r, Ticker ticker) {
        return switch (r.algorithm()) {
            case TOKEN_BUCKET -> new TokenBucket(r.limit(), r.limit() / (r.window().toNanos() / 1e9), ticker);
            case FIXED_WINDOW -> new FixedWindow(r.limit(), r.window(), ticker);
            case SLIDING_LOG -> new SlidingWindowLog(r.limit(), r.window(), ticker);
            case SLIDING_COUNTER -> new SlidingWindowCounter(r.limit(), r.window(), ticker);
        };
    }
}

/** Facade used by an API gateway / servlet filter: "may user U call API A right now?" */
final class RateLimiterService {
    private record Key(String userId, String api) {}

    private final Map<String, Map<Tier, List<LimitRule>>> rules = new ConcurrentHashMap<>();
    private final List<LimitRule> defaultRules;
    private final Map<Key, Limiter> limiters = new ConcurrentHashMap<>();
    private final Map<String, Tier> tiers = new ConcurrentHashMap<>();
    private final Ticker ticker;

    RateLimiterService(List<LimitRule> defaultRules, Ticker ticker) {
        this.defaultRules = List.copyOf(defaultRules);
        this.ticker = ticker;
    }

    void addRule(String api, Tier tier, LimitRule... r) {
        rules.computeIfAbsent(api, a -> new ConcurrentHashMap<>()).put(tier, List.of(r));
    }

    void setTier(String userId, Tier tier) { tiers.put(userId, tier); }

    Decision check(String userId, String api) {
        // computeIfAbsent creates exactly ONE limiter per (user, api), even under a race
        Limiter limiter = limiters.computeIfAbsent(new Key(userId, api), k -> {
            Tier tier = tiers.getOrDefault(userId, Tier.FREE);
            List<LimitRule> r = rules.getOrDefault(api, Map.of()).getOrDefault(tier, defaultRules);
            return LimiterFactory.create(r, ticker);
        });
        return limiter.tryAcquire();
    }
}
```

```java
// File: code/lld/ch20/ratelimiter/Main.java
package lld.ch20.ratelimiter;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

public class Main {
    static final AtomicLong NOW = new AtomicLong();          // manual clock in nanoseconds
    static final Ticker TICKER = NOW::get;

    static void at(double seconds) { NOW.set((long) (seconds * 1e9)); }

    public static void main(String[] args) throws Exception {
        RateLimiterService svc = new RateLimiterService(
                List.of(new LimitRule(Algorithm.FIXED_WINDOW, 100, Duration.ofMinutes(1))), TICKER);
        svc.addRule("/search", Tier.FREE, new LimitRule(Algorithm.TOKEN_BUCKET, 3, Duration.ofSeconds(3))); // 1/s,
burst 3
        svc.addRule("/search", Tier.PREMIUM, new LimitRule(Algorithm.TOKEN_BUCKET, 10, Duration.ofSeconds(1)));
        svc.addRule("/login", Tier.FREE,
                new LimitRule(Algorithm.SLIDING_LOG, 2, Duration.ofSeconds(10)),
                new LimitRule(Algorithm.SLIDING_LOG, 3, Duration.ofMinutes(1)));    // composite
        svc.setTier("pat", Tier.PREMIUM);

        System.out.println("== 1. Per-user, per-API token bucket (FREE: burst 3, 1 token/s)");
        at(0);
        for (int i = 1; i <= 5; i++) System.out.println("  t=0.0 ann /search #" + i + " -> " +
show(svc.check("ann", "/search")));
        at(1.0);
        System.out.println("  t=1.0 ann /search    -> " + show(svc.check("ann", "/search")));
        System.out.println("  t=1.0 bob /search    -> " + show(svc.check("bob", "/search")) + "  (own bucket)");
        int premium = 0;
        for (int i = 0; i < 12; i++) if (svc.check("pat", "/search").allowed()) premium++;
        System.out.println("  t=1.0 pat (PREMIUM) 12 calls -> " + premium + " allowed");
        System.out.println("  t=1.0 ann /orders (default rule) -> " + show(svc.check("ann", "/orders")));

        System.out.println("== 2. Composite rule on /login: 2 per 10 s AND 3 per minute");
        double[] times = {0, 1, 2, 11, 12, 61};
        at(100);
        for (double t : times) {
            at(100 + t);
            System.out.println("  t=+" + (int) t + "s ann /login -> " + show(svc.check("ann", "/login")));
        }

        System.out.println("== 3. A denied request uses up nothing: 3 per minute AND 2 per 10 s (minute rule
first)");
        Limiter login = LimiterFactory.create(List.of(
                new LimitRule(Algorithm.SLIDING_LOG, 3, Duration.ofMinutes(1)),
                new LimitRule(Algorithm.SLIDING_LOG, 2, Duration.ofSeconds(10))), TICKER);
        for (double t : new double[]{0, 1, 2, 11}) {
            at(200 + t);
            System.out.println("  t=+" + (int) t + "s -> " + show(login.tryAcquire()));
        }

        System.out.println("== 4. Boundary burst: limit 5 per 10 s, 5 calls at t=9.9 and 5 at t=10.1");
        for (Algorithm a : List.of(Algorithm.FIXED_WINDOW, Algorithm.SLIDING_LOG, Algorithm.SLIDING_COUNTER)) {
            Limiter l = LimiterFactory.create(new LimitRule(a, 5, Duration.ofSeconds(10)), TICKER);
            int allowed = 0;
            at(1009.9);
            for (int i = 0; i < 5; i++) if (l.tryAcquire().allowed()) allowed++;
            at(1010.1);
            for (int i = 0; i < 5; i++) if (l.tryAcquire().allowed()) allowed++;
```

```
            System.out.printf("  %-16s allowed %d of 10 within 0.2 s%n", a, allowed);
        }

        System.out.println("== 5. 64 threads x 10 calls on one bucket (capacity 50, no refill)");
        Limiter bucket = new TokenBucket(50, 0, TICKER);
        AtomicInteger ok = new AtomicInteger();
        CountDownLatch start = new CountDownLatch(1);
        ExecutorService pool = Executors.newFixedThreadPool(64);
        List<java.util.concurrent.Future<?>> fs = new ArrayList<>();
        for (int t = 0; t < 64; t++)
            fs.add(pool.submit(() -> {
                start.await();
                for (int i = 0; i < 10; i++) if (bucket.tryAcquire().allowed()) ok.incrementAndGet();
                return null;
            }));
        start.countDown();
        for (var f : fs) f.get();
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("  allowed = " + ok.get() + " (exactly 50, never more)");
    }

    private static String show(Decision d) {
        return d.allowed() ? "200 OK (remaining " + d.remaining() + ")"
                : "429 Too Many Requests (retry after " + d.retryAfter().toMillis() + " ms)";
    }
}
```

Walking through the code.

- `TokenBucket` refills **lazily**: on each call it adds `elapsed × rate` tokens (capped at capacity). No background thread. When empty, `retryAfter` is the time until one whole token exists.
- `FixedWindow` computes the current window start with `now − (now mod W)` and resets the count when the window changes.
- `SlidingWindowLog` drops timestamps older than `now − W` and allows if fewer than N remain.
- `SlidingWindowCounter` estimates the count in the sliding window as `previous × (portion of previous window still inside) + current`.
- `Limiter` is split into `probe()` (answer only) and `commit()` (record); the default `tryAcquire()` runs both inside `synchronized (this)`, and every algorithm's own methods are `synchronized` on the same object, so check-and-take stays one atomic step.
- `CompositeLimiter` probes **every** child first and returns the first denial without committing anything; only if all allow does it commit to all of them. Its children are created for it alone by the factory and never shared, so the composite's lock makes the whole probe-then-commit atomic. `remaining` is the smallest of the children's.
- `RateLimiterService.check` builds the limiter inside `computeIfAbsent`, so the tier and rule are resolved once per key.

Output:

```
== 1. Per-user, per-API token bucket (FREE: burst 3, 1 token/s)
  t=0.0 ann /search #1 -> 200 OK (remaining 2)
  t=0.0 ann /search #2 -> 200 OK (remaining 1)
  t=0.0 ann /search #3 -> 200 OK (remaining 0)
  t=0.0 ann /search #4 -> 429 Too Many Requests (retry after 1000 ms)
  t=0.0 ann /search #5 -> 429 Too Many Requests (retry after 1000 ms)
  t=1.0 ann /search    -> 200 OK (remaining 0)
  t=1.0 bob /search    -> 200 OK (remaining 2)  (own bucket)
  t=1.0 pat (PREMIUM) 12 calls -> 10 allowed
  t=1.0 ann /orders (default rule) -> 200 OK (remaining 99)
== 2. Composite rule on /login: 2 per 10 s AND 3 per minute
  t=+0s ann /login -> 200 OK (remaining 1)
  t=+1s ann /login -> 200 OK (remaining 0)
  t=+2s ann /login -> 429 Too Many Requests (retry after 8000 ms)
  t=+11s ann /login -> 200 OK (remaining 0)
  t=+12s ann /login -> 429 Too Many Requests (retry after 48000 ms)
  t=+61s ann /login -> 200 OK (remaining 1)
== 3. A denied request uses up nothing: 3 per minute AND 2 per 10 s (minute rule first)
  t=+0s -> 200 OK (remaining 1)
  t=+1s -> 200 OK (remaining 0)
  t=+2s -> 429 Too Many Requests (retry after 8000 ms)
  t=+11s -> 200 OK (remaining 0)
== 4. Boundary burst: limit 5 per 10 s, 5 calls at t=9.9 and 5 at t=10.1
  FIXED_WINDOW     allowed 10 of 10 within 0.2 s
  SLIDING_LOG      allowed 5 of 10 within 0.2 s
  SLIDING_COUNTER  allowed 5 of 10 within 0.2 s
== 5. 64 threads x 10 calls on one bucket (capacity 50, no refill)
  allowed = 50 (exactly 50, never more)
```

Step 1: ann's burst of 3 is allowed, then 429 with "retry after 1000 ms"; a second later exactly one more is allowed; bob and the PREMIUM user have their own buckets. Step 2: the /login composite denies at +2s (10-second rule) and at +12s (1-minute rule, retry when the +0s call leaves the minute window). Step 3: the denied +2s call consumed nothing, so the +11s call passes both rules. Step 4 is the classic interview point: **fixed window lets 10 requests through in 0.2 seconds** across the boundary, while both sliding algorithms hold the line at 5. Step 5 proves thread safety: 640 concurrent attempts, exactly 50 allowed.

Concurrency & edge cases

- **Atomicity:** each limiter's `tryAcquire` is `synchronized`; the service map is a `ConcurrentHashMap`. Contention is per key, so different users never block each other.
- **Composite and denied requests.** The naive composite calls each child's `tryAcquire()` in turn. If child 1 allows (and consumes) and child 2 then denies, the request never runs but child 1 has been charged for it. Step 3 of the demo is built to expose that: with "3 per minute" first, the denied call at +2s would have used up the third per-minute slot, and the legitimate call at +11s would be refused for about 49 s. Our composite probes all children and commits only if all allow, so +11s is correctly allowed. (An alternative is *acquire, and refund on denial*, which needs a `release()` in every algorithm and is easy to get wrong when a window rolls over in between.)
- **Memory growth:** one limiter per (user, api) forever. Evict idle limiters (e.g. store them in a cache with expire-after-access, like 20.3). Set the idle timeout to at least the longest window (or the full-refill time of a bucket): after that much idle time the old limiter would allow a full burst anyway, so replacing it with a fresh one changes nothing.
- **Clock:** use a monotonic clock (`System.nanoTime`) — wall-clock time (the calendar time a person reads) can jump backwards.
- **Rule changes at runtime:** existing limiters keep old settings; clear the map (or version the rules) when config changes.

Extensions & follow-up questions

- **"Distributed (many gateway servers)."** Keep counters in a shared store: Redis with an atomic Lua script (Redis can run small Lua programs as one uninterrupted step) (or `INCR` + `EXPIRE` for fixed windows: Redis commands for "add one" and "delete after N seconds"). Trade-off: a network hop (one extra round trip to another machine) per request; mitigate with local pre-allocation of tokens. Or use sticky routing (the load balancer

— the component that spreads requests over servers — always sends one user to the same server) so each user always hits the same node.

- **"Leaky bucket."** A bounded queue drained at a constant rate by a worker: smooths bursts into a steady output (good for protecting a fragile downstream — a service that this one calls).
- **"Different cost per request."** `tryAcquire(permits)`: an expensive export costs 10 tokens.
- **"Soft limits / warnings."** Return `remaining` and warn the client at 80%.
- **"Rate limit by IP when not logged in."** The key is just a string; the rule resolver decides which key to use.
- **"Where does the limiter run?"** In an API gateway or a servlet filter / Spring `HandlerInterceptor` (Java web hooks that run before every request) before the controller (the class that handles a given URL).

Real-world example

Stripe has written publicly about using several limiters together ("Scaling your API with rate limiters": a request-rate limiter based on the token bucket, a concurrent-requests limiter, and load shedders — components that deliberately drop low-priority work when overloaded — implemented with Redis) — several independent checks on one request, the composite idea. **GitHub's API** returns `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset` headers, which is our `Decision`. HTTP 429 is defined in RFC 6585 (RFCs are the official documents that define internet standards). **NGINX** (a widely used web server and reverse proxy) `limit_req` uses the leaky-bucket method. In Java, **Guava's** `RateLimiter`, **Resilience4j's** `RateLimiter` and **Bucket4j** (token bucket) are common libraries. Cloudflare has described using the sliding-window-counter approximation (weighting the previous window) to rate limit at the edge (its servers closest to users) cheaply.

In the interview

> **30-second answer:** "A `Limiter` interface with interchangeable algorithms — token bucket by default, sliding window when exactness matters. A factory builds limiters from rules; rules are chosen by API and user tier; a composite limiter combines rules like '10 per second and 1000 per day'. The service keeps one limiter per (user, api) in a `ConcurrentHashMap` via `computeIfAbsent`, and returns a decision with remaining and retry-after for the 429 response. Each limiter makes refill-check-take atomic; for multiple servers, move the state into Redis with an atomic script."

Plan (30–40 min): clarify key/limits (4) → algorithms table and choice (8) → OO design: interface, factory, service, composite (10) → thread safety (5, point to §13.11) → distributed + memory (5).

Follow-ups with answers:

- *"Why is fixed window risky?"* → Up to 2N requests across a window boundary (shown in the output).
- *"Token bucket vs sliding log?"* → Bucket allows bursts, O(1) memory; log is exact but stores every timestamp.
- *"How do you avoid creating two limiters for the same user?"* → `ConcurrentHashMap.computeIfAbsent`.

Self-check

1. Which algorithm allows a burst up to its capacity and then a steady rate? → Token bucket.
2. How many requests can a fixed window of limit 5 let through in 0.2 s across a boundary? → Up to 10.
3. Which pattern lets "2 per 10 s AND 3 per minute" look like one limiter? → Composite.
4. What HTTP status and header go with a denial? → 429 and `Retry-After`.
5. Why must refill + check + take be atomic? → Otherwise two threads can both see the last token and both take it.
6. How do you keep memory bounded with millions of users? → Evict limiters idle for at least the longest window (expire-after-access cache); by then a fresh limiter behaves exactly like the old one.
7. Why does `CompositeLimiter` probe every child before committing to any? → So a request denied by one rule does not use up a slot in another rule.

TL;DR

- **Strategy** for algorithms, **Simple Factory** from rules, **Composite** for combined limits (probe all, then commit all), **Facade** service.
- One limiter per (user, api) via `computeIfAbsent`; `Decision` → 200 (HTTP "OK") or 429 + Retry-After.
- Token bucket (bursty, O(1)), fixed window (2× edge burst), sliding log (exact, O(N)), sliding counter (approximate, O(1)).

- Atomic refill-check-take (§13.11); Redis + Lua for distributed limits.

20.8 Chapter summary

Component	Anchor pattern(s)	The one sentence to remember
Splitwise	Strategy	Split rules return shares; the ledger stores one signed number per pair; simplify with net positions + greedy (≤ n − 1 payments).
Ledger Co. (full case in Ch 21 §21.6)	—	A loan ledger, not a bill-splitting ledger: same "one authoritative record + `BigDecimal`" idea, different rules.
Notification service	Template Method + Strategy + Observer	Retry once in the parent, one class per reason not to send, one exit that tells the listeners.
In-memory cache	Strategy + Observer	HashMap + doubly linked list; eviction policy owns the order; one removal path.
LRU cache	— (data structure)	Map to node, move to front on use, evict before tail; or `LinkedHashMap(accessOrder=true)`.
Ride sharing	Strategy + Decorator + state machine	Ranked list for declines, surge wraps pricing, atomic driver claim.
Carpool variant	Strategy	Offer seats per vehicle; select with Most Vacant / Preferred Vehicle.
Logging framework	Chain of Responsibility + Singleton + Decorator	Hierarchy with inherited levels; propagate up until additivity stops; async is a wrapper.
Pub-sub	Observer via broker	Append-only log + per-subscription offsets; ack after processing; dead letters.
Rate limiter	Strategy + Factory + Composite	One limiter per key; algorithms swappable; 429 with Retry-After.

The common thread. In every design, the part most likely to change sits behind an interface, and the "service" class only orders the steps. When the interviewer says "now add X", the best answer is a new class and a one-line change where objects are wired together.

20.9 Chapter self-check

1. Which two patterns appear in more than half of this chapter's designs? → Strategy and Observer.
2. Strategy vs Decorator in one line each? → Strategy *replaces* a rule; Decorator *wraps* one and adds to its result.
3. Strategy vs Template Method? → Strategy swaps the whole algorithm (composition); Template Method fixes the algorithm and lets subclasses fill steps (inheritance).
4. Which designs used a "single exit" method, and why? → Notification (`finish`) and cache (`remove`): so records, stats and listeners can never be skipped on one path.
5. Where did we call listeners outside a lock, and why? → The cache: a slow or re-entrant listener must not block or deadlock it.
6. What does at-least-once delivery require from consumers? → Idempotency.
7. Name three places this chapter used compare-and-set or `computeIfAbsent` for correctness. → Driver claim (`AtomicBoolean`), subscription commit (`AtomicLong.compareAndSet`), limiter creation (`computeIfAbsent`).

20.10 Chapter TL;DR

- Components are small, so interviews probe the **rule most likely to change** — put it behind an interface.
- **Strategy** everywhere (splits, eviction, matching, pricing, send rules, formatters, rate-limit algorithms); **Observer** for "tell someone when it happens"; **Decorator** to add behaviour without editing (surge, async);

Template Method for a shared algorithm with one varying step; **Chain of Responsibility** for the logger hierarchy; **Composite** for combined limits.

- Money in `BigDecimal`; time from an injected clock; thread safety with the smallest correct tool (one lock, CAS, `computeIfAbsent`, per-key locks, sharding).
- Always volunteer the honest limits: greedy is not optimal, LRU needs a write lock, at-least-once duplicates, fixed windows burst at edges.

Chapter 21 — Case Studies: Machine-Coding Round Problems

> **Goal.** Chapters 14–20 taught you to *design* a system on a whiteboard. This chapter trains the harder cousin of that interview: the **machine-coding round**, where you get a written problem statement, 90–120 minutes, and must hand over **running code** that the interviewer will feed their own inputs into. You will learn how the round is scored, a project skeleton you can type from memory, a minute-by-minute time plan, and then eight real problems — each solved end to end with a working, demo-able (ready to run in front of the interviewer) Java program.

How each problem is laid out: *Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns → Java implementation (+ real output) → Concurrency & edge cases → Extensions → In the interview → Self-check → TL;DR.*

All code is Java 21, standard library only (only the classes that ship with Java, no extra downloads), one package (Java's folder-like grouping of related classes) per problem under `code/lld/ch21/<name>/`. Run from `chapters/21-case-machine-coding/code`:

```
javac --release 21 -d /tmp/lld_21 $(find . -name '*.java')
java -cp /tmp/lld_21 lld.ch21.taskmgmt.Main       # or wallet, calendar, codingblox, propertyhunt, ledger, inventory, filesearch
```

Every demo uses a fixed clock, a seeded random generator (a random-number generator started from a fixed "seed" number, so it produces the same sequence on every run), or sorted output, so what you see below is exactly what you get on every run.

Problem map

#	Problem	Where it is asked (source)	The one idea that makes it work	Patterns
21.1	Task Management System	awesome-low-level-design	Status *state machine* + composable filters	State (enum), Specification, Strategy, Observer
21.2	Digital Wallet Service	awesome-low-level-design	`BigDecimal` money, idempotency keys, lock ordering	Strategy (offers), Facade
21.3	Event Calendar	Flipkart (kumaransg/LLD)	Half-open intervals + a boundary *sweep* to find free slots	Strategy (representative choice), Repository
21.4	Coding Blox (LeetCode-like platform)	Flipkart (kumaransg/LLD)	Command dispatcher + level-based scoring rule	Command, Strategy, Repository
21.5	PropertyHunt	ClearTrip (kumaransg/LLD)	Optional search criteria → list of predicates; unit normalisation	Builder, Specification, Strategy
21.6	Ledger Co (loans & EMIs)	Navi (kumaransg/LLD, Geektrust)	Derive balances from facts instead of storing them	Strategy (interest), Command
21.7	Inventory Management System	Common (Amazon/Flipkart-style)	Append-only movement ledger + per-SKU lock	Strategy (allocation), Observer (reorder)
21.8	Unix File Search (`find`)	Common OOD/machine-coding question	Filters as Specifications combined by Composite AND/OR/NOT; Builder for criteria; a parser for `find` syntax	Specification, Composite (×2), Builder, Interpreter-style parser

(Food ordering "FoodKart", another kumaransg/LLD problem, is solved in Chapter 19.)

> **Key terms — the sources and patterns in the map.** *awesome-low-level-design* and *kumaransg/LLD* are public GitHub repositories that collect low-level design (LLD) problems; Geektrust is a hiring platform that sets such coding challenges. The patterns, one line each: a **state machine** is a fixed set of states plus the moves allowed between them (here kept in an enum, a Java type with a fixed list of named values); **Specification**

wraps one yes/no rule in an object so rules can be combined with AND / OR / NOT; **Strategy** puts each interchangeable rule in its own class behind one interface (a list of methods with no code, which several classes implement in their own way); **Observer** lets objects sign up to be told when something happens; **Facade** is one simple front class over several others; **Repository** hides how objects are stored behind save / find methods; **Command** turns each action into an object or handler that can be looked up by name; **Builder** assembles an object with many optional settings step by step; **Composite** lets a group of objects be treated like a single one. A few other words from the table: **idempotency keys** are explained in 21.2; **lock ordering** means always taking several locks in the same fixed order, so two threads can never block each other forever (a **thread** is one independent line of execution inside a running program; a **lock** is an object only one thread can hold at a time, while others wait); a **half-open interval** `[start, end)` includes its start but not its end; a **sweep** walks through sorted time points one by one; **unit normalisation** converts every input to one unit (e.g. square feet) as soon as it is read; an **SKU** (stock keeping unit) is the id of one sellable item; **OOD** is object-oriented design; an **Interpreter-style parser** turns text written in a small language into a tree of objects.

21.0 How the machine-coding round works

What it is

A **machine-coding round** (also called an *LLD coding round*) is an object-oriented design (designing a program as classes of objects that hold data and behaviour together) interview in which **working code is the answer**. The same round is called *object-oriented design* or *low-level design* at other companies; machine coding simply adds the rule that the code must compile and run at the end. You get a one- or two-page problem statement (usually a PDF), a laptop with your own IDE (integrated development environment: the code editor plus compiler and debugger, such as IntelliJ IDEA), and 90–120 minutes. Internet use is limited to looking up syntax. There is **no database, no UI (user interface: screens) and no web framework (a library for building websites and web APIs — an API is the set of calls a program or service offers to other programs)**: all data lives in memory (maps and lists), and the program is driven by a `main` method, a list of commands, a file, or unit tests (small automated checks that call one piece of code and compare the result with what is expected).

Do not confuse it with the **system design** interview. That round is about keeping a product alive under millions of users, so the conversation is about servers, databases, caches and message queues (see System Design Fundamentals if those words are new). This round never leaves a single program: its vocabulary is **classes** (a *class* is a named code type that bundles some data with the methods that act on that data), their fields and methods, and how the classes relate. Same product, very different zoom level — and practising for one leaves you unready for the other.

Everyday analogy: a whiteboard design interview is an architect *describing* a house. A machine-coding round is being asked to *build a small shed* while the inspector watches — then the inspector leans on every wall.

The format is especially common at Indian product companies such as Flipkart, Swiggy, PhonePe, Navi, Razorpay, CRED, Meesho and Cleartrip; several of this chapter's problems are statements candidates have shared from those rounds.

How it is judged

Interviewers use a rubric (a scoring checklist). The exact wording differs by company, but it always covers the same things. Read the "Expectations" section of any statement in this chapter (Coding Blox, PropertyHunt and Event Calendar all have one) and you will see these repeated almost word for word.

#	Criterion	What the interviewer checks	How to score well
1	**Working, demo-able code**	Does it compile? Does the driver (the small piece of code that feeds input into the program and prints the results) run the sample input and print the expected output? Does it survive *their* inputs?	Get a thin end-to-end path working in the first hour. Never finish with code that does not compile.
2	**Functional completeness**	Every *mandatory* requirement done; bonus only after that.	Implement requirements in the statement's order; tick them off.
3	**Clean models**	Entities have one job, fields are private, invariants (rules that must always be true, such as "balance is never negative") live inside the entity (for example a task refuses an illegal status change).	Use enums for fixed sets, records (Java's compact, unchangeable data classes) for values, classes for things that change.
4	**Service interfaces / separation of concerns** (each class handles one kind of work)	Parsing, business logic (the product's actual rules) and storage are in different classes. No `System.out` inside services.	Driver → Service → Repository layers (skeleton below).
5	**Extensibility**	"Add a new offer / sort order / scoring rule" should mean *adding a class*, not editing `if` chains.	Strategy and Specification for things the statement says "can be X or Y".
6	**Edge cases & failure handling**	Unknown IDs, duplicates, invalid input, wrong user, double-run. The program prints a clear error and **keeps going**.	Custom exceptions (your own error classes; an exception is how Java signals an error) with good messages; the driver catches them per command.
7	**Readability**	Intuitive names, small methods, no copy-paste.	Name classes after nouns in the statement; name methods after verbs in it.
8	**Concurrency (when asked)**	Wallets, inventory, booking: can two threads corrupt state?	Say where the lock is and why; demo it with a thread pool (a fixed set of reusable threads) if time allows.
9	**Tests (bonus)**	Some companies give extra credit for unit tests.	A tiny `assert`-style self-check in `main` is enough if there is no JUnit (Java's standard unit-testing library).

The five-step method, and how the clock changes it

The whiteboard version of this round (Chapter 1) always follows five steps in a fixed order:

1. **Requirements** — turn the one-line prompt into a short written list of what the system must do.
2. **Actors and use cases** — name *who* uses the system (actors) and *what* each of them does with it (use cases).
3. **Classes** — derive classes from the requirements and give each one responsibility.
4. **Relationships** — decide how each pair of classes is connected and draw it as a class diagram.
5. **Code** — implement the key classes, plus the two or three methods that make the main flows work.

Getting the order right beats going fast: each step is the input of the next, so if you skip requirements, you end up inventing classes nobody asked for. Every problem in this chapter is laid out in that same order.

A whiteboard round usually lasts 45 minutes to an hour. A budget for the 45-minute version:

Step	Minutes	Finished artefact
Requirements	5–8	8–12 written requirements
Actors & use cases	3–5	2–4 actors, 6–10 use cases
Classes	8–10	Class list, one responsibility each
Relationships	8–10	Class diagram with relationships and multiplicity (how many objects sit on each side, e.g. one-to-many)
Code	12–15	2–3 methods for the main flows

Two things in that table catch people off guard. The first two steps — which name no class at all — may take as much as a quarter of the time. And coding gets the biggest single slot yet still covers only two or three methods: nobody finishes the whole product, and you are graded on the slice you chose to build well, not on how much ground you covered. **Machine coding keeps the five steps but flips the proportions:** steps 1–4 shrink to about 20 minutes (see the time plan below) and code fills the rest, because now every mandatory feature must actually run.

How big is a complete answer? For a parking lot, roughly twelve requirements lead to about twelve classes joined by some fifteen relationships. Only three of them carry the main flows — issuing a ticket, holding a vehicle, pricing the stay. On a whiteboard you write those three properly and describe each of the other nine in one sentence. The costliest mistake is detailing all twelve equally: it looks thorough but leaves no time to demonstrate one complete flow. In machine coding the same instinct applies — make one thin path run end to end first, then widen.

Beyond "does it run", the interviewer is watching five things:

- **Scope** — did you pin down a bounded list of what you will build? A well-bounded design outranks a sprawling, open-ended one.
- **One job per class** — can you describe every class in a single sentence? A class that needs two sentences is usually two classes.
- **Relationships** — does a class *hold* another, *own* it, or *extend* it? These are different designs, and picking the wrong one is spotted immediately.
- **Code and model agree** — if the code contains a method your model has no place for, either the model or the code needs fixing.
- **A trade-off** — the option you took *and* the one you turned down, both said out loud.

Notice what is missing: nobody grades your UML notation (UML: the standard box-and-arrow notation for class diagrams) or counts how many diagram types you drew. The model is what gets marked; a diagram is just one way of displaying it.

The single most common failure is **running out of time with nothing that runs**. The second is a "god class" (a class that does everything): one `Manager` with every map and every method. Both are avoidable with the skeleton and time plan below.

A reusable project skeleton

Type this structure from memory in the first five minutes. It is the same for every problem, which is exactly why it saves time.

```
src/
  model/          entities & value objects      (Task, Wallet, Loan, Property ...)   - enums, records, small
classes
  repository/     storage contracts + in-memory (TaskRepository, InMemoryTaskRepository)
  service/        business rules, one per area  (TaskService, WalletService)          - no printing, no parsing
  strategy/       pluggable rules               (Offer, AllocationStrategy, SortBy)
  exception/      domain exceptions             (NotFoundException, InsufficientFundsException)
  driver/         Main + CommandProcessor       (parses input, calls services, prints output)
```

Layer	Knows about	Must NOT know about	Why
model	nothing but itself (and other models)	services, storage, console	Entities guard their own invariants (e.g. `Task.changeStatus` rejects `TODO → DONE`).
repository	model	services, console	Swapping `HashMap` for a DB (database) later touches only this layer (Dependency Inversion — services depend on interfaces, not on concrete classes; Chapter 4).
service	model, repository *interfaces*, strategy *interfaces*	concrete repositories (injected — handed in from outside rather than created inside), console	This is the part the interviewer reads most carefully.
strategy	model	services	Each "X or Y" rule in the statement becomes one interface + small classes (Open/Closed).
exception	—	—	One class per failure *kind*, with a message a user could read.
driver	services	repositories' internals	Parsing and printing are replaceable; business logic is not.

In this book each problem lives in **one Java package** (for example `lld.ch21.wallet`) so the full listing stays short and readable. In the real round, create the sub-packages above — it costs nothing and the interviewer notices. Three reusable building blocks appear again and again:

1. **A generic in-memory repository** — a `ConcurrentHashMap<K, V>` (a map that many threads can use at once; `K, V` are type parameters, so the same class works for any key and value type — that is what *generic* means) behind an interface with `save / findById / findAll / delete` (see `Repository<K,V>` in Coding Blox, 21.4).
2. **An ID generator** — an `AtomicInteger` (an integer threads can increment safely without a lock) per entity type (`"T" + seq.incrementAndGet()`), thread-safe and predictable, which matches statements that say "auto-increment id starting with 1".
3. **A command processor** — a `Map<String, Consumer<String[]>>` from command name to handler (`Consumer` is a function that takes an argument and returns nothing; a *handler* is the code that deals with one command), fed by a built-in `List<String>` of sample commands (Coding Blox, PropertyHunt, Ledger). Adding a command = adding one map entry (Command pattern, Chapter 10).

The 90-minute time plan

Minutes	Activity	Output you should have
0–10	Read the statement twice. Ask clarifying questions. Mark mandatory vs bonus. Write the sample input as your driver's input list.	A numbered requirement list, a list of questions answered.
10–20	Nouns → entities, verbs → service methods. Pick the 1–2 patterns the statement is begging for ("can be sorted by X or Y" → Strategy).	Skeleton packages created, enums and records typed.
20–60	Implement mandatory requirements **in order**, running the driver after each one.	Sample input produces the sample output.
60–75	Edge cases and validation: unknown ids, duplicates, wrong owner, invalid numbers. Custom exceptions.	Driver prints errors and continues.
75–85	Bonus features (only if all mandatory ones work). Thread-safety where money or stock is involved.	Bonus commands in the driver.
85–90	Clean up names, delete dead code (code that is never run), run everything one last time. Prepare a 2-minute walkthrough.	Code that compiles, a demo, a story.

A good rule: **run the program at least every 15 minutes**. A compile error found at minute 88 is a disaster; the same error found at minute 30 is a 20-second fix.

What to say in the first two minutes

1. "I'll keep everything in memory with a repository interface, so a DB can be added later without touching services."
2. "I'll drive it from `main` with the sample commands from the statement, and print errors without crashing."

3. "The mandatory list is A, B, C; the bonus is D, E. I'll do A–C first."
4. "Where does it need to be thread-safe?" (Ask. For wallets and inventory the answer is always yes.)

Common mistakes in this round

Mistake	Why it hurts	Do this instead
Starting with a perfect class diagram for 20 minutes	No time left for code	Enums/records first; the diagram lives in your head and in names
One `Manager` class with 15 maps	Fails "separation of concerns"	One service per area + repositories
`System.out.println` inside the service	Service cannot be reused or tested	Services return data; the driver prints
`double` for money	`0.1 + 0.2 != 0.3` — interviewers test this	`BigDecimal` (Java's exact decimal number type) with a fixed scale (number of digits after the decimal point; 21.2) or whole units (`long` paise; a paisa is 1/100 of a rupee)
Crashing on the first bad command	"Fails gracefully" is an explicit criterion	Catch domain exceptions (errors that describe a broken business rule) in the driver loop, print, continue
Implementing bonus before mandatory	Partial credit on the mandatory part	Mandatory in order; bonus last
Writing every class in full detail	Nothing works end to end when time runs out	Make the core flow run first; stub (write an empty placeholder for) or name the rest
`Random` without a seed	Output changes every run; hard to demo	Seed it, or put randomness behind a Strategy (21.4)
Hard-coding "today" with `LocalDate.now()`	Demo changes daily	Inject a `java.time.Clock` (Java's replaceable source of the current time; 21.1, 21.2)

Real-world examples

The skeleton above is the standard layering of real Java back ends (the server-side part of an application): in a Spring Boot (the most popular Java framework for building services) service, controllers (classes that receive web requests) play the "driver" role, `@Service` classes hold business rules, and `@Repository` interfaces (Spring Data) hide storage. (The `@Something` markers are annotations: labels a framework reads to know what a class is for.) The in-memory repository you write in the round is exactly the kind of fake that teams use in unit tests for those services.

In the interview

> **30-second answer — "How do you approach a machine-coding problem?"** "I read the statement twice, list mandatory and bonus requirements and ask about anything ambiguous. I set up model / repository / service / strategy / exception / driver packages, put the sample input into the driver, and implement requirements one by one, running after each. Services return data and throw domain exceptions; the driver prints and keeps going. I finish with edge cases, thread-safety where state is shared, and bonus features if time allows."

Follow-ups

- *"Why an interface for an in-memory repository?"* — So services depend on an abstraction (an interface rather than a concrete class); a JDBC (Java's standard API for SQL databases) or Redis (a fast in-memory key-value store) implementation can replace it with no change to business code, and tests can use the in-memory one.
- *"How do you make random behaviour testable?"* — Put it behind an interface (a Strategy) and inject a seeded or scripted implementation for demos and tests.
- *"How much do you test?"* — At minimum, the driver replays the statement's samples and prints results I can compare with the expected output. With time, a few JUnit tests on the service layer.

Self-check

- **Q:** Name the six packages of the skeleton. → **A:** model, repository, service, strategy, exception, driver.

- **Q:** Where should `System.out.println` live? → **A:** Only in the driver (command processor / `Main`).
- **Q:** What do you do at minute 60 if the bonus is tempting but one mandatory requirement is missing? → **A:** Finish the mandatory one; bonus only after all mandatory ones run.
- **Q:** Why inject a `Clock`? → **A:** So time-based logic (due dates, reminders) gives the same output on every run and can be tested.

> **TL;DR**

Working code first, clean layering second, patterns where the statement asks for variation, graceful errors everywhere. Skeleton: **model** → **repository** → **service** → **strategy** → **exception** → **driver**. Run often.

21.1 Design a Task Management System

Problem & clarifying questions

Build the core of a tool like Jira, Asana or Trello (popular team task-tracking apps): users create tasks, assign them to people, move them through statuses, comment on them, and search/filter them; the system reminds people about tasks that are due. The awesome-low-level-design version asks for create / update / delete, title + description + due date + priority + status, assignment, reminders, search and filtering (by priority, due date, assignee), marking tasks complete, task history, and safe concurrent access (several users acting at the same moment without corrupting data).

Questions to ask (with the answers this solution assumes):

Question	Assumed answer
Which statuses exist, and can any status go to any other?	TODO, IN_PROGRESS, BLOCKED, DONE, CANCELLED. Only listed transitions are legal (e.g. TODO cannot jump straight to DONE; CANCELLED is final; DONE can be re-opened).
One assignee or many?	One assignee (nullable: the field may be `null`, Java's "no value"). A list is an easy extension.
What does "reminder" mean in a console app?	A method a scheduler would call: "notify owners of open tasks due within N days or overdue".
Should due dates in the past be allowed?	No, not on creation.
How are notifications delivered?	Through a listener interface; the demo prints them.
"View task history" — history of what?	Two readings: each task's audit log (a permanent record of who changed what, when) — built here — and a user's list of finished tasks, which is just a search: `assignedTo(u).and(withStatus(DONE))`.
"Set reminders" — per task, or a sweep?	A sweep over due dates is built here; a per-task reminder time is an extension (below).

> **Requirements**

Functional

1. Register users.
2. Create a task (title, description, priority, due date); update title / priority / due date; delete a task.
3. Assign / reassign a task to a user.
4. Change status following the allowed transitions; mark done.
5. Comment on a task.
6. Search with any combination of filters (assignee, status, minimum priority, due before, open only, title contains) and a sort order (due date, priority, id).
7. Reminders for open tasks due soon or overdue.
8. Per-task activity history.

Non-functional

- Thread-safe: two people editing the same task must not corrupt it.
- Extensible: new filters, sort orders and notification channels without editing existing code.
- Deterministic demo (injected `Clock`).

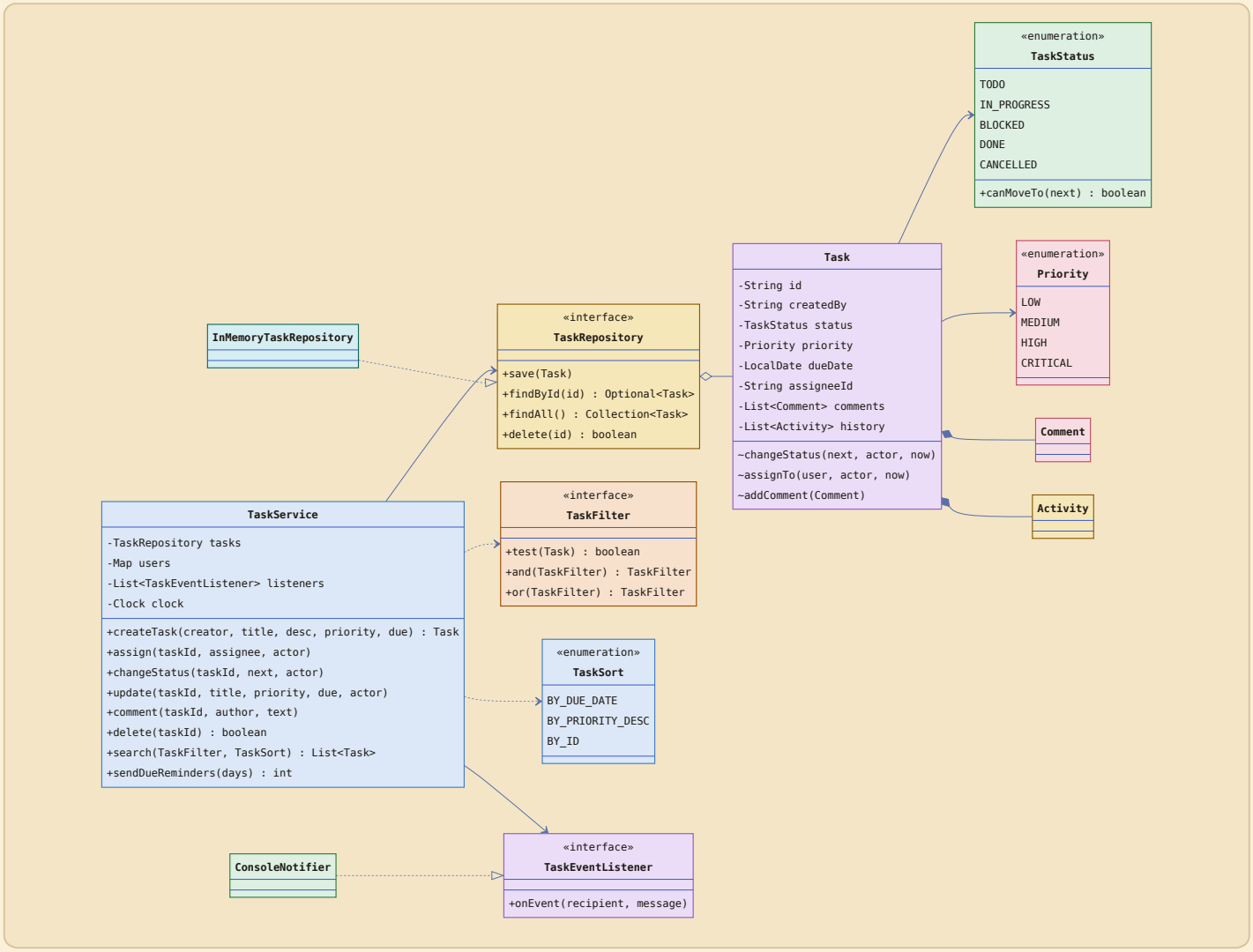
Use cases

Actor	Use cases
Task creator (any user)	create task, update, delete, assign, comment, search
Assignee	change status, comment, see own tasks
Scheduler (system actor)	send due-date reminders
Notification channel (external)	receives events (assigned, status changed, commented, reminder)

Core entities

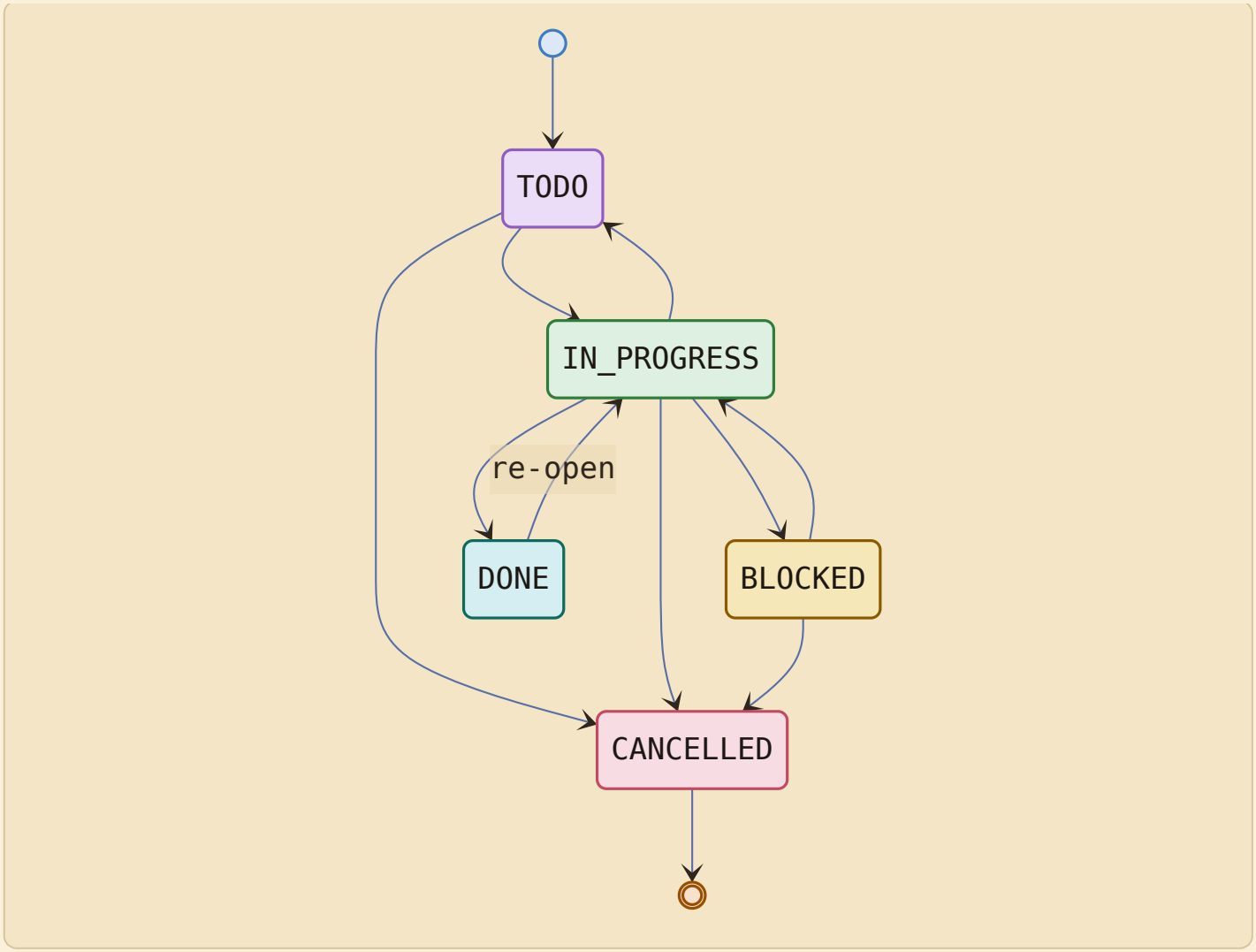
Class	Kind	Responsibility
`Task`	class	Holds task data; enforces legal status changes; records its own history. All mutators (methods that change state) `synchronized` (only one thread at a time may run them on the same object).
`TaskStatus`	enum	Lists statuses and, for each, which statuses it may move to (a tiny state machine).
`Priority`	enum	LOW < MEDIUM < HIGH < CRITICAL (enum order is used for comparison).
`User`, `Comment`, `Activity`	records	Immutable (cannot change after creation) values.
`TaskRepository` / `InMemoryTaskRepository`	interface / class	Storage contract and a `ConcurrentHashMap` implementation.
`TaskFilter`	functional interface (an interface with one method, so a short lambda function can implement it)	Composable predicate (a yes/no test; Specification pattern) with factory methods (static methods that create objects) and `and` / `or`.
`TaskSort`	enum	Named comparators (objects that decide which of two items sorts first; Strategy). `BY_ID` compares the number inside the id, so `T2` sorts before `T10`.
`TaskEventListener` / `ConsoleNotifier`	interface / class	Observer for notifications.
`TaskService`	class	Facade with every use case; validates input, uses the clock, fires events.
`NotFoundException`, `InvalidTransitionException`	exceptions	Clear failure messages.

Class diagram

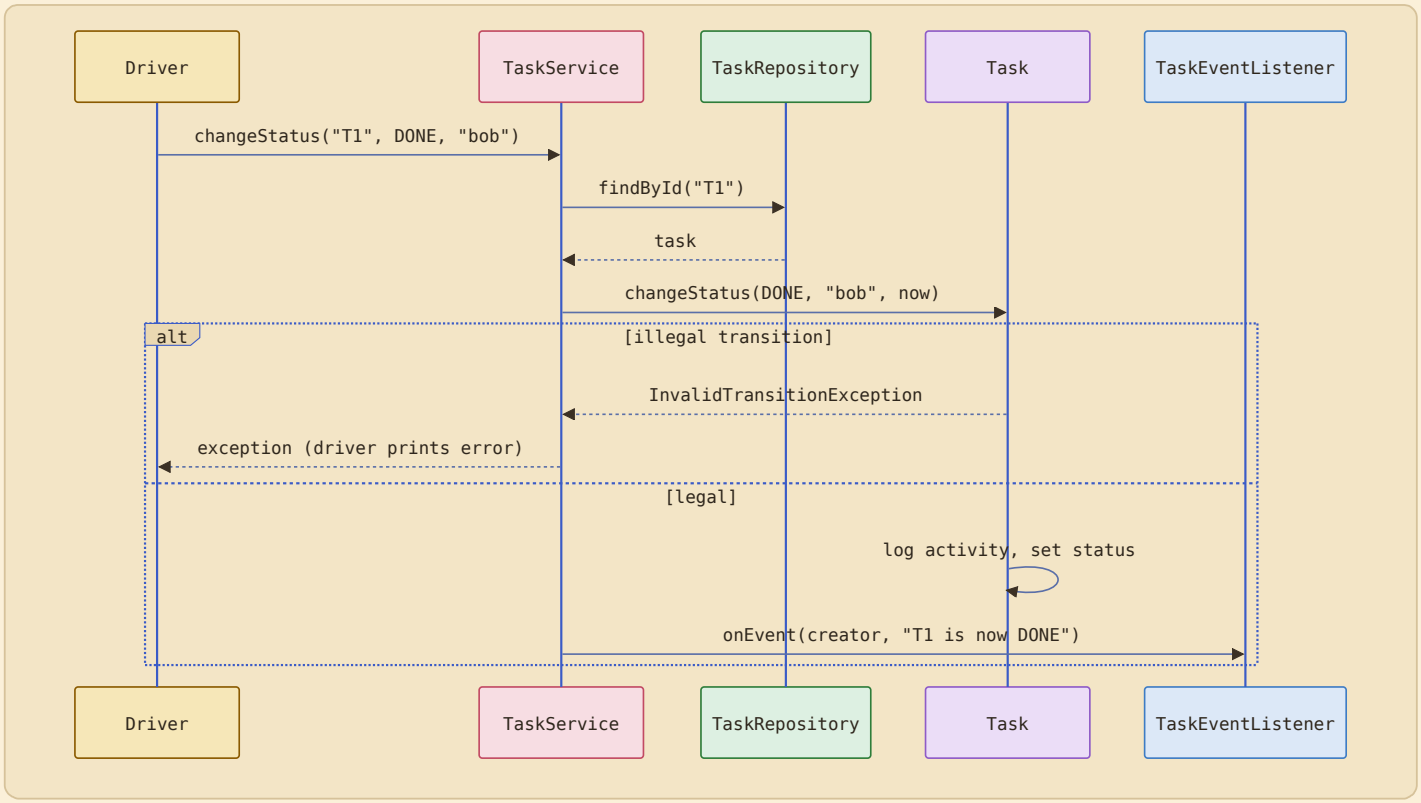


Key flows

Task status life cycle — the rules live in `TaskStatus.allowedNext()`:



Change status and notify:



Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Transitions stored in the `TaskStatus` enum	State (enum-based)	All rules in one place; `Task` just asks `canMoveTo`.	Full State pattern (one class per state, each holding that state's behaviour) with one class per status — overkill when states carry no behaviour besides transitions.
`TaskFilter extends Predicate<Task>` with `and/or`	Specification	Any combination of filters without a method per combination; a new filter is one factory method.	`search(String assignee, Status s, Priority p, ...)` with many nullable parameters — grows forever.
`TaskSort` enum of comparators	Strategy	Sort order is data the caller picks.	`switch` on a sort string inside the service.
`TaskEventListener` list	Observer	Email/Slack/push can be added without touching the service.	Calling `System.out` from the service.
`TaskService` as the only entry point	Facade	Driver sees one simple API.	Driver talking to repositories directly.
`Clock` injected	Dependency injection (a class receives the objects it needs from outside instead of creating them)	Deterministic reminders and tests.	`LocalDate.now()` scattered in code.
`synchronized` on `Task` mutators	Monitor per entity (each object's own lock guards its methods)	Fine-grained (many small locks instead of one big one): two different tasks never block each other.	One global lock on the service — simpler but serialises everything (forces all work to run one at a time).

Java implementation

```java
// File: lld/ch21/taskmgmt/Priority.java
package lld.ch21.taskmgmt;

/** Ordered from least to most urgent, so natural enum order can be used for sorting. */
public enum Priority { LOW, MEDIUM, HIGH, CRITICAL }
```

```java
// File: lld/ch21/taskmgmt/TaskStatus.java
package lld.ch21.taskmgmt;

import java.util.EnumSet;
import java.util.Set;

/** Task lifecycle. Each status knows which statuses it may move to (a tiny state machine). */
public enum TaskStatus {
    TODO, IN_PROGRESS, BLOCKED, DONE, CANCELLED;

    public Set<TaskStatus> allowedNext() {
        return switch (this) {
            case TODO -> EnumSet.of(IN_PROGRESS, CANCELLED);
            case IN_PROGRESS -> EnumSet.of(BLOCKED, DONE, TODO, CANCELLED);
            case BLOCKED -> EnumSet.of(IN_PROGRESS, CANCELLED);
            case DONE -> EnumSet.of(IN_PROGRESS);          // re-open
            case CANCELLED -> EnumSet.noneOf(TaskStatus.class);
        };
    }

    public boolean canMoveTo(TaskStatus next) { return allowedNext().contains(next); }

    public boolean isOpen() { return this != DONE && this != CANCELLED; }
}
```

```java
// File: lld/ch21/taskmgmt/User.java
package lld.ch21.taskmgmt;

public record User(String id, String name) { }
```

```java
// File: lld/ch21/taskmgmt/Comment.java
package lld.ch21.taskmgmt;

import java.time.LocalDateTime;

public record Comment(String authorId, String text, LocalDateTime at) { }
```

```java
// File: lld/ch21/taskmgmt/Activity.java
package lld.ch21.taskmgmt;

import java.time.LocalDateTime;

/** One line of a task's audit history ("who did what, when"). */
public record Activity(LocalDateTime at, String actorId, String description) {
    @Override public String toString() { return at.toLocalTime() + " " + actorId + ": " + description; }
}
```

```java
// File: lld/ch21/taskmgmt/Task.java
package lld.ch21.taskmgmt;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

/**
 * The central entity. All mutators are synchronized so two threads editing
 * the same task cannot interleave (e.g. a status change racing a reassignment).
 */
public class Task {
    private final String id;
    private final String createdBy;
    private String title;
    private String description;
    private Priority priority;
    private LocalDate dueDate;
    private TaskStatus status = TaskStatus.TODO;
    private String assigneeId;                 // null = unassigned
    private final List<Comment> comments = new ArrayList<>();
    private final List<Activity> history = new ArrayList<>();

    Task(String id, String createdBy, String title, String description, Priority priority, LocalDate dueDate) {
        this.id = id;
        this.createdBy = createdBy;
        this.title = title;
        this.description = description;
        this.priority = priority;
        this.dueDate = dueDate;
    }

    public String id() { return id; }
    public String createdBy() { return createdBy; }
    public synchronized String title() { return title; }
    public synchronized String description() { return description; }
    public synchronized Priority priority() { return priority; }
    public synchronized LocalDate dueDate() { return dueDate; }
    public synchronized TaskStatus status() { return status; }
    public synchronized String assigneeId() { return assigneeId; }
    public synchronized List<Comment> comments() { return List.copyOf(comments); }
    public synchronized List<Activity> history() { return List.copyOf(history); }

    synchronized void changeStatus(TaskStatus next, String actor, LocalDateTime now) {
        if (!status.canMoveTo(next)) throw new InvalidTransitionException(id, status, next);
        log(now, actor, "status " + status + " -> " + next);
        status = next;
    }

    synchronized void assignTo(String userId, String actor, LocalDateTime now) {
        if (!status.isOpen()) throw new IllegalStateException("Cannot assign a " + status + " task " + id);
        log(now, actor, "assigned to " + userId);
        assigneeId = userId;
    }

    synchronized void update(String newTitle, Priority newPriority, LocalDate newDue, String actor, LocalDateTime
now) {
        if (newTitle != null) { title = newTitle; }
        if (newPriority != null) { priority = newPriority; }
        if (newDue != null) { dueDate = newDue; }
        log(now, actor, "updated title/priority/due -> " + title + "/" + priority + "/" + dueDate);
    }

    synchronized void addComment(Comment c) {
        comments.add(c);
        log(c.at(), c.authorId(), "commented \"" + c.text() + "\"");
    }
```

```
    private void log(LocalDateTime at, String actor, String what) { history.add(new Activity(at, actor, what)); }

    @Override public synchronized String toString() {
        return "%s %-22s %-8s %-11s due=%s assignee=%s".formatted(id, title, priority, status, dueDate,
                assigneeId == null ? "-" : assigneeId);
    }
}
```

```
// File: lld/ch21/taskmgmt/InvalidTransitionException.java
package lld.ch21.taskmgmt;

public class InvalidTransitionException extends RuntimeException {
    public InvalidTransitionException(String taskId, TaskStatus from, TaskStatus to) {
        super("Task " + taskId + " cannot move from " + from + " to " + to);
    }
}
```

```
// File: lld/ch21/taskmgmt/NotFoundException.java
package lld.ch21.taskmgmt;

public class NotFoundException extends RuntimeException {
    public NotFoundException(String what, String id) { super(what + " not found: " + id); }
}
```

```
// File: lld/ch21/taskmgmt/TaskRepository.java
package lld.ch21.taskmgmt;

import java.util.Collection;
import java.util.Optional;

/** Storage contract. The service depends on this, not on a HashMap (Dependency Inversion). */
public interface TaskRepository {
    void save(Task task);
    Optional<Task> findById(String id);
    Collection<Task> findAll();
    boolean delete(String id);
}
```

```
// File: lld/ch21/taskmgmt/InMemoryTaskRepository.java
package lld.ch21.taskmgmt;

import java.util.Collection;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

public class InMemoryTaskRepository implements TaskRepository {
    private final ConcurrentHashMap<String, Task> tasks = new ConcurrentHashMap<>();

    @Override public void save(Task task) { tasks.put(task.id(), task); }
    @Override public Optional<Task> findById(String id) { return Optional.ofNullable(tasks.get(id)); }
    @Override public Collection<Task> findAll() { return List.copyOf(tasks.values()); }
    @Override public boolean delete(String id) { return tasks.remove(id) != null; }
}
```

```java
// File: lld/ch21/taskmgmt/TaskFilter.java
package lld.ch21.taskmgmt;

import java.time.LocalDate;
import java.util.function.Predicate;

/**
 * Specification pattern: small, named predicates that combine with and()/or().
 * Adding a new filter = adding one factory method; the search code never changes (Open/Closed).
 */
@FunctionalInterface
public interface TaskFilter extends Predicate<Task> {

    static TaskFilter all() { return t -> true; }
    static TaskFilter assignedTo(String userId) { return t -> userId.equals(t.assigneeId()); }
    static TaskFilter withStatus(TaskStatus s) { return t -> t.status() == s; }
    static TaskFilter atLeast(Priority p) { return t -> t.priority().compareTo(p) >= 0; }
    static TaskFilter dueOnOrBefore(LocalDate d) { return t -> t.dueDate() != null && !t.dueDate().isAfter(d); }
    static TaskFilter open() { return t -> t.status().isOpen(); }
    static TaskFilter titleContains(String word) {
        return t -> t.title().toLowerCase().contains(word.toLowerCase());
    }

    default TaskFilter and(TaskFilter other) { return t -> test(t) && other.test(t); }
    default TaskFilter or(TaskFilter other) { return t -> test(t) || other.test(t); }
}
```

```java
// File: lld/ch21/taskmgmt/TaskSort.java
package lld.ch21.taskmgmt;

import java.time.LocalDate;
import java.util.Comparator;

/** Sorting strategies; each constant wraps a Comparator (Strategy pattern). */
public enum TaskSort {
    BY_DUE_DATE(Comparator.comparing(Task::dueDate, Comparator.nullsLast(Comparator.<LocalDate>naturalOrder()))),
    BY_PRIORITY_DESC(Comparator.comparing(Task::priority).reversed()),
    BY_ID(Comparator.comparingInt((Task t) -> Integer.parseInt(t.id().substring(1))));   // numeric: T2 before T10

    private final Comparator<Task> comparator;
    TaskSort(Comparator<Task> c) { this.comparator = c.thenComparing(Task::id); }
    public Comparator<Task> comparator() { return comparator; }
}
```

```java
// File: lld/ch21/taskmgmt/TaskEventListener.java
package lld.ch21.taskmgmt;

/** Observer: anything that wants to hear about task events (email, Slack, push...). */
public interface TaskEventListener {
    void onEvent(String recipientId, String message);
}
```

```java
// File: lld/ch21/taskmgmt/ConsoleNotifier.java
package lld.ch21.taskmgmt;

public class ConsoleNotifier implements TaskEventListener {
    @Override public void onEvent(String recipientId, String message) {
        System.out.println("  [notify " + recipientId + "] " + message);
    }
}
```

```java
// File: lld/ch21/taskmgmt/TaskService.java
package lld.ch21.taskmgmt;

import java.time.Clock;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

/** Facade for every use case. Holds no storage details and no printing. */
public class TaskService {
    private final TaskRepository tasks;
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final List<TaskEventListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger seq = new AtomicInteger();
    private final Clock clock;

    public TaskService(TaskRepository tasks, Clock clock) {
        this.tasks = tasks;
        this.clock = clock;
    }

    public void addListener(TaskEventListener l) { listeners.add(l); }

    public User addUser(String id, String name) {
        User u = new User(id, name);
        if (users.putIfAbsent(id, u) != null) throw new IllegalArgumentException("User exists: " + id);
        return u;
    }

    public Task createTask(String creatorId, String title, String description, Priority priority, LocalDate due) {
        requireUser(creatorId);
        if (title == null || title.isBlank()) throw new IllegalArgumentException("Title is required");
        if (due != null && due.isBefore(today())) throw new IllegalArgumentException("Due date is in the past: " +
due);
        Task t = new Task("T" + seq.incrementAndGet(), creatorId, title, description, priority, due);
        tasks.save(t);
        return t;
    }

    public void assign(String taskId, String assigneeId, String actorId) {
        requireUser(actorId);
        requireUser(assigneeId);
        Task t = get(taskId);
        t.assignTo(assigneeId, actorId, now());
        if (!assigneeId.equals(actorId)) fire(assigneeId, actorId + " assigned you " + t.id() + " '" + t.title() +
"'");
    }

    public void changeStatus(String taskId, TaskStatus next, String actorId) {
        requireUser(actorId);
        Task t = get(taskId);
        t.changeStatus(next, actorId, now());
        if (!t.createdBy().equals(actorId)) fire(t.createdBy(), t.id() + " is now " + next);
    }

    public void update(String taskId, String title, Priority priority, LocalDate due, String actorId) {
        requireUser(actorId);
        if (title != null && title.isBlank()) throw new IllegalArgumentException("Title cannot be blank");
        if (due != null && due.isBefore(today())) throw new IllegalArgumentException("Due date is in the past: " +
due);
        get(taskId).update(title, priority, due, actorId, now());
    }

    public void comment(String taskId, String authorId, String text) {
```

```
        requireUser(authorId);
        Task t = get(taskId);
        t.addComment(new Comment(authorId, text, now()));
        String other = authorId.equals(t.assigneeId()) ? t.createdBy() : t.assigneeId();
        if (other != null && !other.equals(authorId)) fire(other, authorId + " commented on " + t.id());
    }

    public boolean delete(String taskId) { return tasks.delete(taskId); }

    public List<Task> search(TaskFilter filter, TaskSort sort) {
        return tasks.findAll().stream().filter(filter).sorted(sort.comparator()).toList();
    }

    /** Reminder sweep: open tasks due within `days` days (or overdue). A scheduler would call this. */
    public int sendDueReminders(int days) {
        LocalDate limit = today().plusDays(days);
        List<Task> due = search(TaskFilter.open().and(TaskFilter.dueOnOrBefore(limit)), TaskSort.BY_DUE_DATE);
        for (Task t : due) {
            String who = t.assigneeId() != null ? t.assigneeId() : t.createdBy();
            String when = t.dueDate().isBefore(today()) ? "OVERDUE since " + t.dueDate() : "due " + t.dueDate();
            fire(who, "Reminder: " + t.id() + " '" + t.title() + "' is " + when);
        }
        return due.size();
    }

    public Task get(String taskId) {
        return tasks.findById(taskId).orElseThrow(() -> new NotFoundException("Task", taskId));
    }

    private void requireUser(String id) { if (!users.containsKey(id)) throw new NotFoundException("User", id); }
    private void fire(String to, String msg) { listeners.forEach(l -> l.onEvent(to, msg)); }
    private LocalDate today() { return LocalDate.now(clock); }
    private LocalDateTime now() { return LocalDateTime.now(clock); }
}
```

```java
// File: lld/ch21/taskmgmt/Main.java
package lld.ch21.taskmgmt;

import java.time.Clock;
import java.time.Instant;
import java.time.LocalDate;
import java.time.ZoneOffset;

public class Main {
    public static void main(String[] args) {
        // Fixed clock => the demo prints the same thing on every run.
        Clock clock = Clock.fixed(Instant.parse("2026-09-28T09:00:00Z"), ZoneOffset.UTC);
        LocalDate today = LocalDate.now(clock);
        TaskService svc = new TaskService(new InMemoryTaskRepository(), clock);
        svc.addListener(new ConsoleNotifier());

        svc.addUser("alice", "Alice");
        svc.addUser("bob", "Bob");
        svc.addUser("carol", "Carol");

        System.out.println("== create & assign");
        Task t1 = svc.createTask("alice", "Fix login bug", "NPE on empty password", Priority.CRITICAL,
today.plusDays(1));
        Task t2 = svc.createTask("alice", "Write release notes", "", Priority.LOW, today.plusDays(7));
        Task t3 = svc.createTask("bob", "Upgrade to Java 21", "", Priority.HIGH, today.plusDays(3));
        Task t4 = svc.createTask("carol", "Review PR #42", "", Priority.MEDIUM, today);
        svc.assign(t1.id(), "bob", "alice");
        svc.assign(t2.id(), "carol", "alice");
        svc.assign(t3.id(), "bob", "bob");

        System.out.println("== work on tasks");
        svc.changeStatus(t1.id(), TaskStatus.IN_PROGRESS, "bob");
        svc.comment(t1.id(), "bob", "Root cause found in PasswordValidator");
        svc.changeStatus(t1.id(), TaskStatus.DONE, "bob");
        svc.update(t3.id(), null, Priority.CRITICAL, null, "bob");

        System.out.println("== search: bob's open tasks, by priority");
        svc.search(TaskFilter.assignedTo("bob").and(TaskFilter.open()), TaskSort.BY_PRIORITY_DESC)
           .forEach(t -> System.out.println("  " + t));
        System.out.println("== search: priority >= MEDIUM due within 3 days, by due date");
        svc.search(TaskFilter.atLeast(Priority.MEDIUM).and(TaskFilter.dueOnOrBefore(today.plusDays(3))),
                   TaskSort.BY_DUE_DATE).forEach(t -> System.out.println("  " + t));

        System.out.println("== reminders (due within 3 days)");
        System.out.println("  reminders sent: " + svc.sendDueReminders(3));

        System.out.println("== history of " + t1.id());
        svc.get(t1.id()).history().forEach(a -> System.out.println("  " + a));

        System.out.println("== edge cases");
        attempt(() -> svc.changeStatus(t2.id(), TaskStatus.DONE, "carol"));   // TODO -> DONE skips IN_PROGRESS
        attempt(() -> svc.assign("T99", "bob", "alice"));
        attempt(() -> svc.assign(t2.id(), "dave", "alice"));
        attempt(() -> svc.createTask("alice", "Past task", "", Priority.LOW, today.minusDays(1)));
        svc.changeStatus(t4.id(), TaskStatus.CANCELLED, "carol");
        attempt(() -> svc.assign(t4.id(), "bob", "carol"));
        System.out.println("  deleted T2? " + svc.delete(t2.id()) + ", remaining=" + svc.search(TaskFilter.all(),
TaskSort.BY_ID).size());
    }

    private static void attempt(Runnable r) {
        try { r.run(); System.out.println("  ok"); }
        catch (RuntimeException e) { System.out.println("  error: " + e.getMessage()); }
    }
}
```

Output:

```
== create & assign
  [notify bob] alice assigned you T1 'Fix login bug'
  [notify carol] alice assigned you T2 'Write release notes'
== work on tasks
  [notify alice] T1 is now IN_PROGRESS
  [notify alice] bob commented on T1
  [notify alice] T1 is now DONE
== search: bob's open tasks, by priority
  T3 Upgrade to Java 21     CRITICAL TODO        due=2026-10-01 assignee=bob
== search: priority >= MEDIUM due within 3 days, by due date
  T4 Review PR #42          MEDIUM   TODO        due=2026-09-28 assignee=-
  T1 Fix login bug          CRITICAL DONE        due=2026-09-29 assignee=bob
  T3 Upgrade to Java 21     CRITICAL TODO        due=2026-10-01 assignee=bob
== reminders (due within 3 days)
  [notify carol] Reminder: T4 'Review PR #42' is due 2026-09-28
  [notify bob] Reminder: T3 'Upgrade to Java 21' is due 2026-10-01
  reminders sent: 2
== history of T1
  09:00 alice: assigned to bob
  09:00 bob: status TODO -> IN_PROGRESS
  09:00 bob: commented "Root cause found in PasswordValidator"
  09:00 bob: status IN_PROGRESS -> DONE
== edge cases
  error: Task T2 cannot move from TODO to DONE
  error: Task not found: T99
  error: User not found: dave
  error: Due date is in the past: 2026-09-27
  error: Cannot assign a CANCELLED task T4
  deleted T2? true, remaining=3
```

What to notice in the output: self-assignment (`bob` assigning T3 to himself) sends no notification; `T1` shows up in the priority/due search even though it is DONE (that search did not ask for `open()`), but it is **not** in the reminders, which only consider open tasks; the illegal `TODO → DONE` jump is rejected with a clear message; and the history of T1 is a complete audit trail.

Concurrency & edge cases

- **Two users update the same task at once.** Each mutator of `Task` is `synchronized`, so a status change and a reassignment cannot interleave (have their steps mixed together); getters (methods that only read a field) are synchronized too so readers see a consistent value.
- **Storage.** `ConcurrentHashMap` for tasks and users; `findAll()` returns a copy so a search is never disturbed by a concurrent insert.
- **Listeners** (objects that asked to be told about events) are in a `CopyOnWriteArrayList` (a list that copies itself on each write, so reading needs no lock) — registering a listener while events fire is safe.
- **IDs** from an `AtomicInteger` — no duplicates under concurrency.
- **Edge cases handled:** unknown task / user / actor, blank title, past due date (on create and on update), illegal transition, assigning a closed (DONE/CANCELLED) task, self-assignment (no noise), comment notifications go to "the other party".
- **Not handled (say so):** lost updates between two *editors of the text fields* (last writer wins). Fix: optimistic locking with a `version` field — reject an update whose version is stale (out of date). Optimistic locking takes no lock up front; it only checks, at save time, that nobody else changed the record in the meantime.

Extensions & follow-up questions

- **Sub-tasks / dependencies?** Add `parentId` and a `Set<String> blockedBy`; forbid DONE while a blocker is open (a check inside `changeStatus`).
- **Multiple assignees or watchers?** Replace `assigneeId` with `Set<String>`; notify all watchers through the same listener.
- **Recurring tasks?** A `RecurrenceRule` (daily/weekly) that creates the next task when one is marked DONE.
- **Custom workflows per project (like Jira)?** Move the transition table from the enum into a `Workflow` object (a `Map<Status, Set<Status>>`) injected per project.
- **A reminder time chosen by the user?** Store `remindAt` on the task; the scheduler's sweep fires every reminder whose time has passed and clears it.

- **Real reminders?** A `ScheduledExecutorService` (Java's tool for running tasks on a timer) calling `sendDueReminders(1)` every hour; store "already reminded" per task to avoid spam.
- **Search at scale?** Secondary indexes (extra lookup maps keyed by a field other than the id: `Map<String, Set<Task>>` by assignee, `TreeMap<LocalDate, Set<Task>>` by due date; a `TreeMap` keeps its keys sorted) or a search engine such as Elasticsearch (a popular open-source search server).

Real-world example

In Jira, each project has a *workflow*: a set of statuses and the transitions allowed between them, and a transition can fire notifications to watchers. Search is done with JQL, a query language that combines conditions like `assignee = currentUser() AND status != Done ORDER BY priority DESC` (DESC = descending) — the same "compose filters, then sort" idea as `TaskFilter` + `TaskSort`, just with a text syntax on top.

In the interview

Present in this order (30–45 min version): (1) statuses and transitions on the whiteboard — it shows you think about invariants; (2) `Task` and `TaskService` API; (3) filters as composable predicates — the pattern interviewers love here; (4) the observer for notifications; (5) run the demo. If asked "why not the full State pattern?", answer: statuses have no behaviour except which transitions are legal, so an enum with a transition table is simpler and still closed to illegal moves.

Self-check

- **Q:** Where is the rule "TODO cannot go to DONE" coded? → **A:** In `TaskStatus.allowedNext()`; `Task.changeStatus` enforces it.
- **Q:** How do you add a "created by user X" filter? → **A:** Add `static TaskFilter createdBy(String u) { return t -> u.equals(t.createdBy()); }` — nothing else changes.
- **Q:** Why does the service take a `Clock`? → **A:** So "today" is controllable in demos and tests.
- **Q:** Why is `Task` synchronized per instance instead of a lock in the service? → **A:** Finer-grained: edits to different tasks never contend.

TL;DR

Task = entity that guards its own state machine and history. Service = facade. Filters = Specification, sorts = Strategy, notifications = Observer. Thread-safety per task.

21.2 Design a Digital Wallet Service

Problem & clarifying questions

Build the core of a wallet (an app account that stores money for payments) like Paytm, PhonePe or PayPal balance: users hold wallets, add money, transfer to other users and see a statement. The awesome-low-level-design statement also lists account and personal-info management, adding and removing payment methods (cards, bank accounts), transfers to *external* accounts, multiple currencies with conversion, security, **concurrent transactions with data consistency**, and scale. Promotional **cashback** (money given back to the customer after a payment, as a reward) is not in that statement; it is added here because wallet machine-coding variants often ask for it, and it is a clean place to show the Strategy pattern and double-entry thinking (the accounting rule that every amount added somewhere is taken from somewhere else, so the books always balance). In a machine-coding round the interviewer focuses on three things: **money arithmetic**, **no double-spending under concurrency** (spending the same money twice because two requests ran at the same time), and **no double-charging on retries** (idempotency).

> **Key term — idempotency:** an operation is **idempotent** if running it twice has the same effect as running it once. Networks fail in the middle of requests, so clients retry; if "transfer 100" is not idempotent, a retry moves another 100. The usual fix is an **idempotency key**: the client sends a unique id with the request, and the server remembers the result for each id and returns it again for a repeat instead of re-running the work. More in System Design Fundamentals.

Question	Assumed answer
Money type?	`BigDecimal` with scale 2 (paise/cents). Inputs with more than 2 decimals are rejected, not silently rounded.
Can a balance go negative?	No.
What if a client retries a request (network timeout — it gave up waiting for the reply)?	Each request carries an **idempotency key**; a repeated key returns the original result and moves no money.
Offers?	Cashback strategies: "10% on first transfer, max 50" and "flat 25 on transfers ≥ 1000". Offers do not stack — the best one wins.
Where does cashback money come from?	From a **promotions wallet** (`PROMO`) that marketing tops up. Cashback is an ordinary transfer PROMO → sender, so money is never created from nothing. If the budget is used up, no cashback is paid and the transfer still stands.
Top-up source?	An external bank/card; modelled as a credit (money added to the wallet) with a reference.
Multi-currency?	Out of scope for the 90-minute version; discussed in extensions.

Requirements

Functional

1. Create a wallet for a user.
2. Top up a wallet.
3. Transfer between two wallets (atomic: both sides or neither).
4. Transaction history (statement) per wallet, with running balance.
5. Apply the best eligible cashback offer after a transfer, paid out of the funded promotions wallet.
6. Idempotent top-up and transfer.

Non-functional

- **Correctness under concurrency:** no lost updates (one thread's change silently overwritten by another's), no negative balances, no deadlocks (two threads each waiting forever for a lock the other holds), total money conserved (only top-ups add money; transfers and cashback only move it).
- Exact decimal arithmetic.
- New offers added without touching the transfer code.

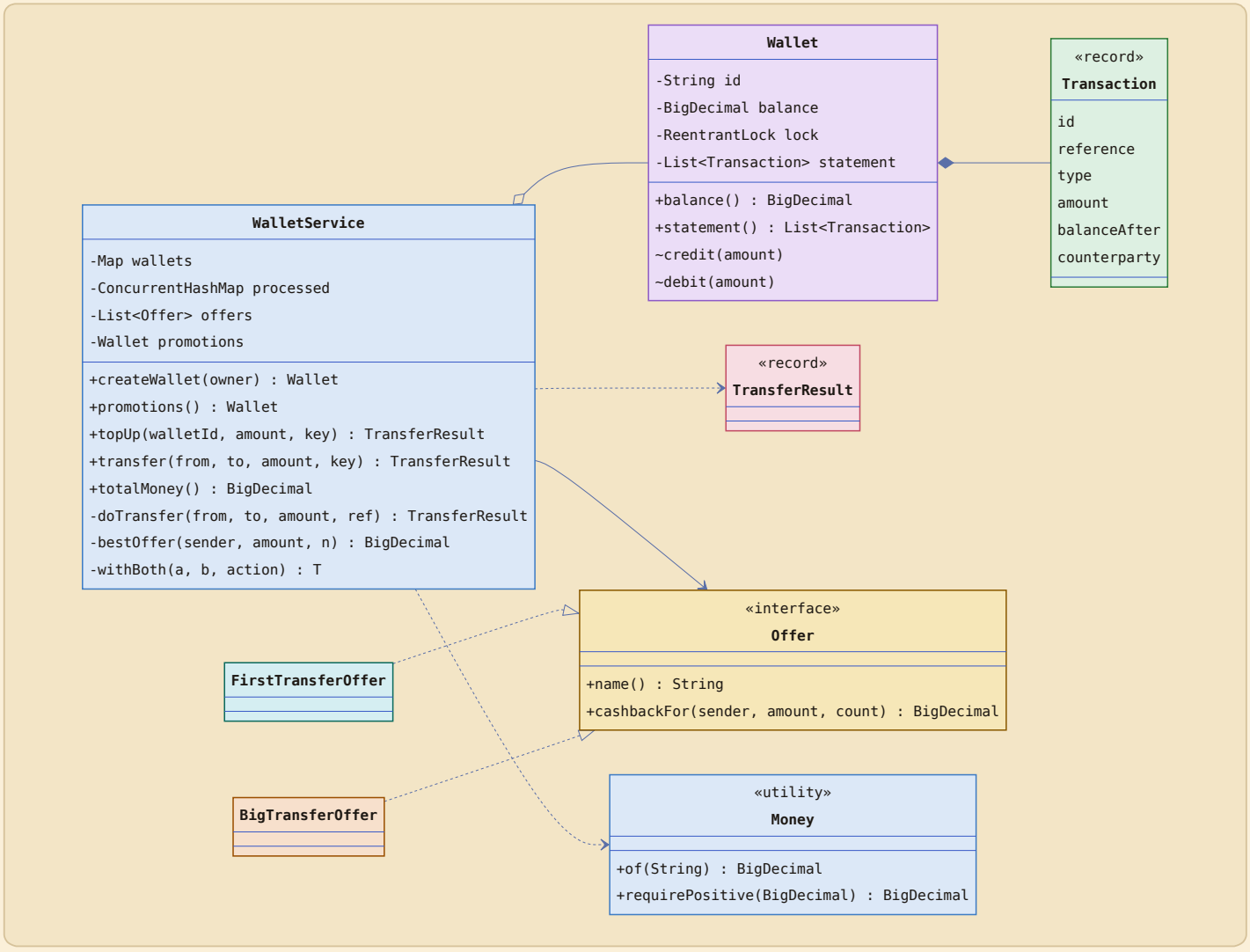
Use cases

Actor	Use cases
User	create wallet, top up, transfer, view statement
Bank / card gateway (external)	source of top-ups
Marketing (configures)	defines offers, funds the promotions wallet
Client app (retrying)	re-sends the same request with the same idempotency key

Core entities

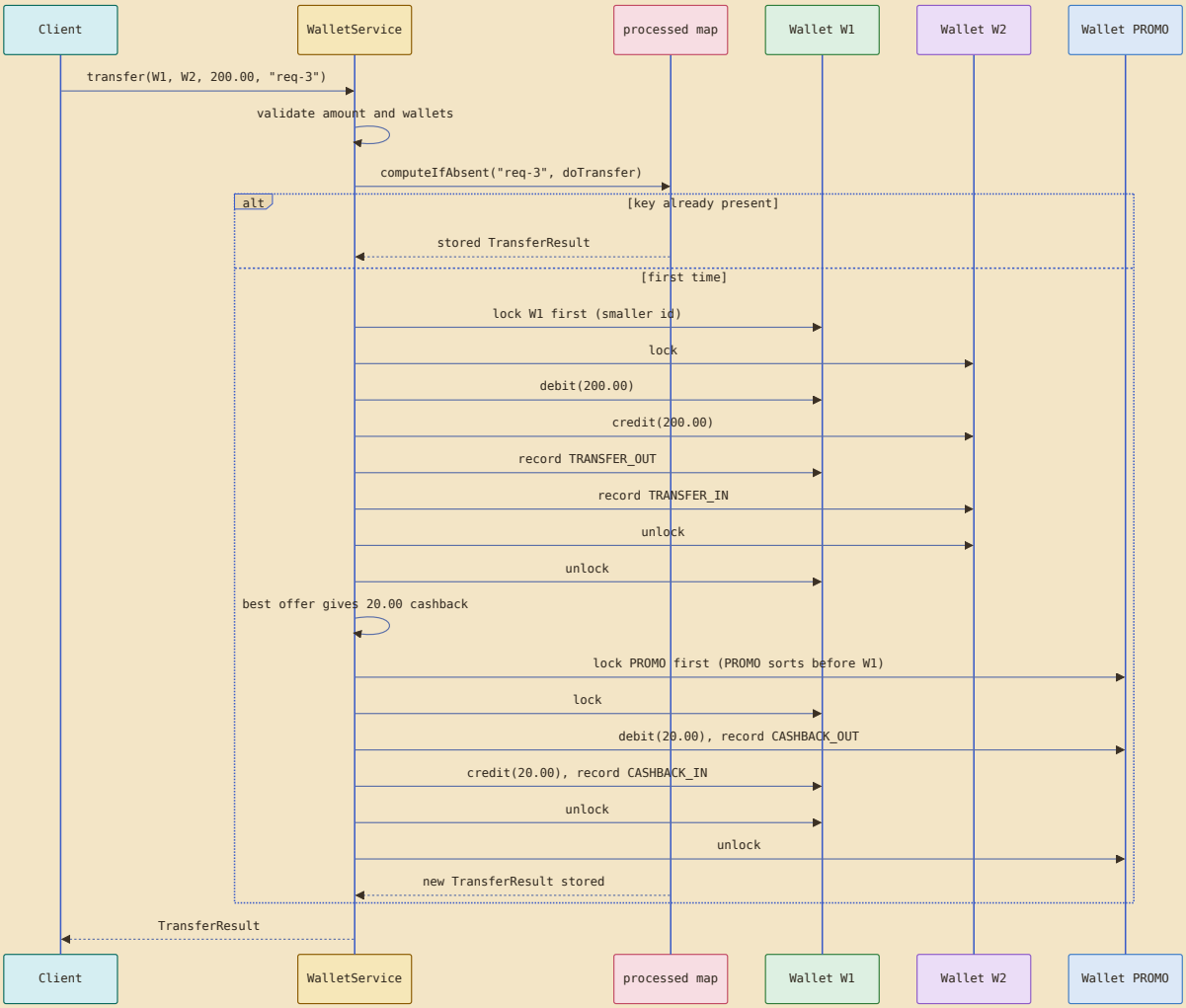
Class	Kind	Responsibility
`Wallet`	class	Balance + own statement + its `ReentrantLock` (Java's explicit lock object). `debit` refuses to go below zero. The service owns one special wallet, `PROMO`, that funds cashback.
`Transaction`	record	Immutable ledger line: type, amount, balance after, counterparty (the other wallet involved), reference.
`TxnType`	enum	TOP_UP, TRANSFER_OUT, TRANSFER_IN, CASHBACK_OUT (on PROMO), CASHBACK_IN (on the user).
`TransferResult`	record	What the caller gets (stored for idempotent replays).
`Money`	utility	Normalises and validates `BigDecimal` amounts.
`Offer` + `FirstTransferOffer`, `BigTransferOffer`	interface + strategies	Compute cashback for a completed transfer.
`WalletService`	class	Facade: creates wallets, orchestrates (calls in the right order) locking, idempotency, offers.
`InsufficientFundsException`, `WalletNotFoundException`	exceptions	Failure reasons.

Class diagram



Key flows

Transfer with idempotency and lock ordering:


Client
WalletService
processed map
Wallet W1
Wallet W2
Wallet PROMO
transfer(W1, W2, 200.00, "req-3")
validate amount and wallets
computeIfAbsent("req-3", doTransfer)
alt
[key already present]
stored TransferResult
[first time]
lock W1 first (smaller id)
lock
debit(200.00)
credit(200.00)
record TRANSFER_OUT
record TRANSFER_IN
unlock
unlock
best offer gives 20.00 cashback
lock PROMO first (PROMO sorts before W1)
lock
debit(20.00), record CASHBACK_OUT
credit(20.00), record CASHBACK_IN
unlock
unlock
new TransferResult stored
TransferResult
Client
WalletService
processed map
Wallet W1
Wallet W2
Wallet PROMO

Design decisions & patterns used

Decision	Why	Alternative rejected
`BigDecimal`, scale 2, reject > 2 decimals	`double` cannot represent 0.1 exactly; money must be exact. Rejecting `1.005` avoids silent rounding surprises.	`double` (wrong), `long` paise (also correct, and a fine choice — say so).
Lock per wallet, acquired in id order	Two opposite transfers (A→B and B→A) would deadlock if each locked its own source first. A global order (smaller id first) makes a cycle impossible.	One global lock — correct but every transfer in the system serialises.
Debit before credit, both under both locks	`debit` throws before any state changes, so a failed transfer leaves both wallets untouched (atomicity).	Credit then debit — would need a compensating action (a new step that undoes an earlier one) on failure.
Idempotency via `ConcurrentHashMap.computeIfAbsent` (creates and stores a value only if the key is missing, as one atomic step)	The check "seen this key?" and the execution are one atomic step per key, so two concurrent retries cannot both run. If the transfer throws, nothing is stored and the client can retry.	`if (!map.containsKey(k)) { run; put }` — check-then-act race (another thread can act between the check and the action).
Cashback paid from a funded `PROMO` wallet	Double-entry: every credit has a matching debit, so the sum of all wallets changes only on top-ups — which is exactly what the demo checks. A campaign cannot overspend its budget.	Crediting the user out of thin air — the books no longer balance and nobody can say what a campaign cost.
Cashback as a second step, after the transfer locks are released	Only two wallets are ever locked at once, and ordinary transfers never wait on the busy `PROMO` lock.	Locking sender, receiver and PROMO together for every transfer — correct, but every transfer in the system would queue on PROMO.
`Offer` interface, best one wins	Strategy: new promotions are new classes; the "no stacking" policy lives in one method.	`if/else` on promotion codes inside `transfer`.
Statement stored in the wallet	Reading a wallet's history needs no global scan.	One global transaction list filtered on read — fine for small data, slow at scale.

`computeIfAbsent` detail worth knowing: the JDK (Java Development Kit: Java's compiler and standard library) documents that the whole call is atomic, that the mapping function runs only when the key is absent, that an exception from the function is rethrown with no mapping recorded, and that other threads updating the *same map* may block while it runs — so the function should be short and must not update the map itself. Our function only touches wallets, never `processed`, so it is safe. At higher scale, store a `CompletableFuture` (a handle to a result that will be ready later) with `putIfAbsent` and run the transfer outside the map (extension below).

Java implementation

```java
// File: lld/ch21/wallet/Money.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Helpers that keep every amount at 2 decimal places. Never use double for money. */
public final class Money {
    public static final int SCALE = 2;
    private Money() { }

    public static BigDecimal of(String amount) { return normalize(new BigDecimal(amount)); }

    public static BigDecimal normalize(BigDecimal amount) { return amount.setScale(SCALE, RoundingMode.HALF_EVEN);
}

    public static BigDecimal requirePositive(BigDecimal amount) {
        if (amount == null || amount.signum() <= 0) throw new IllegalArgumentException("Amount must be positive: "
+ amount);
        if (amount.stripTrailingZeros().scale() > SCALE)
            throw new IllegalArgumentException("At most " + SCALE + " decimal places: " + amount);
        return normalize(amount);
    }
}
```

```java
// File: lld/ch21/wallet/TxnType.java
package lld.ch21.wallet;

/** CASHBACK_OUT is written on the promotions wallet, CASHBACK_IN on the user who receives it. */
public enum TxnType { TOP_UP, TRANSFER_OUT, TRANSFER_IN, CASHBACK_OUT, CASHBACK_IN }
```

```java
// File: lld/ch21/wallet/Transaction.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.time.Instant;

/** Immutable ledger entry. Each transfer writes two entries (out + in) sharing one reference. */
public record Transaction(String id, String reference, String walletId, TxnType type, BigDecimal amount,
                          BigDecimal balanceAfter, String counterparty, Instant at) {
    @Override public String toString() {
        return "%-5s %-12s %8s  bal=%8s  ref=%s%s".formatted(id, type, amount, balanceAfter, reference,
                counterparty == null ? "" : " with " + counterparty);
    }
}
```

```java
// File: lld/ch21/wallet/TransferResult.java
package lld.ch21.wallet;

import java.math.BigDecimal;

/** What the caller gets back. A replayed (duplicate) request returns the stored result. */
public record TransferResult(String reference, String from, String to, BigDecimal amount, BigDecimal cashback) { }
```

```java
// File: lld/ch21/wallet/Wallet.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Holds a balance and its own statement. The balance is only changed while `lock` is held;
 * the service decides lock order when two wallets are involved.
 */
public class Wallet {
    private final String id;
    private final String ownerName;
    private final ReentrantLock lock = new ReentrantLock();
    private BigDecimal balance = Money.normalize(BigDecimal.ZERO);
    private final List<Transaction> statement = new ArrayList<>();

    Wallet(String id, String ownerName) { this.id = id; this.ownerName = ownerName; }

    public String id() { return id; }
    public String ownerName() { return ownerName; }
    ReentrantLock lock() { return lock; }

    public BigDecimal balance() {
        lock.lock();
        try { return balance; } finally { lock.unlock(); }
    }

    public List<Transaction> statement() {
        lock.lock();
        try { return List.copyOf(statement); } finally { lock.unlock(); }
    }

    // --- called only with lock held ---
    void credit(BigDecimal amt) { balance = balance.add(amt); }

    void debit(BigDecimal amt) {
        if (balance.compareTo(amt) < 0) throw new InsufficientFundsException(id, balance, amt);
        balance = balance.subtract(amt);
    }

    BigDecimal rawBalance() { return balance; }
    void record(Transaction t) { statement.add(t); }
}
```

```java
// File: lld/ch21/wallet/InsufficientFundsException.java
package lld.ch21.wallet;

import java.math.BigDecimal;

public class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String walletId, BigDecimal balance, BigDecimal needed) {
        super("Insufficient funds in " + walletId + ": balance " + balance + ", needed " + needed);
    }
}
```

```java
// File: lld/ch21/wallet/WalletNotFoundException.java
package lld.ch21.wallet;

public class WalletNotFoundException extends RuntimeException {
    public WalletNotFoundException(String id) { super("Wallet not found: " + id); }
}
```

```java
// File: lld/ch21/wallet/Offer.java
package lld.ch21.wallet;

import java.math.BigDecimal;

/** Strategy: an offer looks at a completed transfer and returns the cashback (zero if not eligible). */
public interface Offer {
    String name();
    BigDecimal cashbackFor(Wallet sender, BigDecimal amount, int senderTransferCount);
}
```

```java
// File: lld/ch21/wallet/FirstTransferOffer.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** 10% cashback on a user's first transfer, capped. */
public class FirstTransferOffer implements Offer {
    private final BigDecimal percent;
    private final BigDecimal cap;

    public FirstTransferOffer(String percent, String cap) {
        this.percent = new BigDecimal(percent);
        this.cap = Money.of(cap);
    }

    @Override public String name() { return "FIRST_TRANSFER"; }

    @Override public BigDecimal cashbackFor(Wallet sender, BigDecimal amount, int count) {
        if (count != 1) return BigDecimal.ZERO;
        BigDecimal raw = amount.multiply(percent).divide(BigDecimal.valueOf(100), Money.SCALE,
RoundingMode.HALF_EVEN);
        return raw.min(cap);
    }
}
```

```java
// File: lld/ch21/wallet/BigTransferOffer.java
package lld.ch21.wallet;

import java.math.BigDecimal;

/** Flat cashback for any transfer at or above a threshold. */
public class BigTransferOffer implements Offer {
    private final BigDecimal threshold;
    private final BigDecimal flat;

    public BigTransferOffer(String threshold, String flat) {
        this.threshold = Money.of(threshold);
        this.flat = Money.of(flat);
    }

    @Override public String name() { return "BIG_TRANSFER"; }

    @Override public BigDecimal cashbackFor(Wallet sender, BigDecimal amount, int count) {
        return amount.compareTo(threshold) >= 0 ? flat : BigDecimal.ZERO;
    }
}
```

```java
// File: lld/ch21/wallet/WalletService.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.time.Clock;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Supplier;

public class WalletService {
    public static final String PROMOTIONS_ID = "PROMO";
    private final Map<String, Wallet> wallets = new ConcurrentHashMap<>();
    /** Idempotency store: request key -> result of the first successful execution. */
    private final ConcurrentHashMap<String, TransferResult> processed = new ConcurrentHashMap<>();
    private final Map<String, AtomicInteger> transferCounts = new ConcurrentHashMap<>();
    private final List<Offer> offers;
    private final AtomicInteger txnSeq = new AtomicInteger();
    private final AtomicInteger walletSeq = new AtomicInteger();
    private final Clock clock;
    /** Marketing's budget. Cashback is a real transfer out of this wallet, so no money is created. */
    private final Wallet promotions = new Wallet(PROMOTIONS_ID, "Promotions");

    public WalletService(List<Offer> offers, Clock clock) {
        this.offers = List.copyOf(offers);
        this.clock = clock;
        wallets.put(promotions.id(), promotions);
    }

    public Wallet promotions() { return promotions; }

    public Wallet createWallet(String ownerName) {
        Wallet w = new Wallet("W" + walletSeq.incrementAndGet(), ownerName);
        wallets.put(w.id(), w);
        return w;
    }

    public TransferResult topUp(String walletId, BigDecimal amount, String idempotencyKey) {
        BigDecimal amt = Money.requirePositive(amount);
        Wallet w = get(walletId);
        return processed.computeIfAbsent(idempotencyKey, key -> {
            w.lock().lock();
            try {
                w.credit(amt);
                w.record(txn(key, w, TxnType.TOP_UP, amt, null));
            } finally { w.lock().unlock(); }
            return new TransferResult(key, "BANK", walletId, amt, BigDecimal.ZERO);
        });
    }

    /**
     * Moves money atomically. computeIfAbsent makes the key check-and-run atomic, so a retried
     * request never debits twice. If the lambda throws, nothing is stored and the client may retry.
     */
    public TransferResult transfer(String fromId, String toId, BigDecimal amount, String idempotencyKey) {
        if (fromId.equals(toId)) throw new IllegalArgumentException("Cannot transfer to the same wallet");
        BigDecimal amt = Money.requirePositive(amount);
        Wallet from = get(fromId), to = get(toId);
        return processed.computeIfAbsent(idempotencyKey, key -> doTransfer(from, to, amt, key));
    }

    private TransferResult doTransfer(Wallet from, Wallet to, BigDecimal amt, String ref) {
        // Step 1: the transfer itself, under both wallet locks.
        int n = withBoth(from, to, () -> {
            from.debit(amt);                        // throws before any change if funds are short
            to.credit(amt);
```

```
            from.record(txn(ref, from, TxnType.TRANSFER_OUT, amt, to.id()));
            to.record(txn(ref, to, TxnType.TRANSFER_IN, amt, from.id()));
            return transferCounts.computeIfAbsent(from.id(), k -> new AtomicInteger()).incrementAndGet();
        });
        // Step 2: cashback is a second, ordinary transfer PROMO -> sender (same lock-ordering rule).
        BigDecimal offered = bestOffer(from, amt, n);
        BigDecimal cashback = offered.signum() == 0 || from == promotions ? BigDecimal.ZERO
                : withBoth(promotions, from, () -> {
                    if (promotions.rawBalance().compareTo(offered) < 0) return BigDecimal.ZERO;   // budget used up
                    promotions.debit(offered);
                    from.credit(offered);
                    promotions.record(txn(ref, promotions, TxnType.CASHBACK_OUT, offered, from.id()));
                    from.record(txn(ref, from, TxnType.CASHBACK_IN, offered, promotions.id()));
                    return offered;
                });
        return new TransferResult(ref, from.id(), to.id(), amt, cashback);
    }

    /** Locks two wallets, always the smaller id first, so two opposite operations can never deadlock. */
    private static <T> T withBoth(Wallet a, Wallet b, Supplier<T> action) {
        Wallet first = a.id().compareTo(b.id()) < 0 ? a : b;
        Wallet second = first == a ? b : a;
        first.lock().lock();
        try {
            second.lock().lock();
            try { return action.get(); } finally { second.lock().unlock(); }
        } finally { first.lock().unlock(); }
    }

    /** Offers do not stack: the single most generous one wins. */
    private BigDecimal bestOffer(Wallet sender, BigDecimal amt, int n) {
        return offers.stream().map(o -> o.cashbackFor(sender, amt, n))
                .max(Comparator.naturalOrder()).orElse(BigDecimal.ZERO);
    }

    public Wallet get(String id) {
        Wallet w = wallets.get(id);
        if (w == null) throw new WalletNotFoundException(id);
        return w;
    }

    /** Sum of every wallet, promotions included: only top-ups change it (cashback just moves money). */
    public BigDecimal totalMoney() {
        return wallets.values().stream().map(Wallet::balance).reduce(BigDecimal.ZERO, BigDecimal::add);
    }

    private Transaction txn(String ref, Wallet w, TxnType type, BigDecimal amt, String counterparty) {
        return new Transaction("TX" + txnSeq.incrementAndGet(), ref, w.id(), type, amt, w.rawBalance(),
                counterparty, clock.instant());
    }
}
```

```java
// File: lld/ch21/wallet/Main.java
package lld.ch21.wallet;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.Instant;
import java.time.ZoneOffset;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Clock clock = Clock.fixed(Instant.parse("2026-09-28T10:00:00Z"), ZoneOffset.UTC);
        WalletService svc = new WalletService(
                List.of(new FirstTransferOffer("10", "50.00"), new BigTransferOffer("1000.00", "25.00")), clock);

        svc.topUp(svc.promotions().id(), Money.of("100.00"), "budget-1");   // marketing funds the campaign
        Wallet alice = svc.createWallet("Alice");
        Wallet bob = svc.createWallet("Bob");

        System.out.println("== top-ups");
        svc.topUp(alice.id(), Money.of("2000.00"), "req-1");
        svc.topUp(bob.id(), Money.of("300.50"), "req-2");
        svc.topUp(alice.id(), Money.of("2000.00"), "req-1");       // client retry: same key, no double credit
        System.out.println("  Alice=" + alice.balance() + "  Bob=" + bob.balance());

        System.out.println("== transfers");
        TransferResult r1 = svc.transfer(alice.id(), bob.id(), Money.of("200.00"), "req-3");
        System.out.println("  " + r1);
        TransferResult r1again = svc.transfer(alice.id(), bob.id(), Money.of("200.00"), "req-3");
        System.out.println("  retry returns same result? " + (r1 == r1again));
        System.out.println("  " + svc.transfer(alice.id(), bob.id(), Money.of("1200.00"), "req-4"));
        System.out.println("  Alice=" + alice.balance() + "  Bob=" + bob.balance() + "  PROMO=" +
svc.promotions().balance());

        System.out.println("== edge cases");
        attempt(() -> svc.transfer(bob.id(), alice.id(), Money.of("99999.00"), "req-5"));
        attempt(() -> svc.transfer(bob.id(), alice.id(), new BigDecimal("-5"), "req-6"));
        attempt(() -> svc.transfer(bob.id(), alice.id(), new BigDecimal("1.005"), "req-7"));
        attempt(() -> svc.transfer(bob.id(), bob.id(), Money.of("1.00"), "req-8"));
        attempt(() -> svc.transfer(bob.id(), "W99", Money.of("1.00"), "req-9"));
        System.out.println("  failed req-5 can be retried later: " + svc.transfer(bob.id(), alice.id(),
Money.of("10.00"), "req-5").amount());

        System.out.println("== statement for Alice");
        alice.statement().forEach(t -> System.out.println("  " + t));
        System.out.println("== statement for PROMO");
        svc.promotions().statement().forEach(t -> System.out.println("  " + t));

        System.out.println("== concurrency: 1000 opposite transfers on 8 threads");
        BigDecimal before = svc.totalMoney();
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int i = 0; i < 1000; i++) {
            final int n = i;
            pool.submit(() -> {
                try {
                    if (n % 2 == 0) svc.transfer(alice.id(), bob.id(), Money.of("1.00"), "c-" + n);
                    else svc.transfer(bob.id(), alice.id(), Money.of("1.00"), "c-" + n);
                } catch (InsufficientFundsException ignored) { }
            });
        }
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
        System.out.println("  total before=" + before + " after=" + svc.totalMoney()
                + " conserved=" + (before.compareTo(svc.totalMoney()) == 0));
        System.out.println("  Alice=" + alice.balance() + "  Bob=" + bob.balance() + "  PROMO=" +
```

```
svc.promotions().balance());
    }

    private static void attempt(Runnable r) {
        try { r.run(); System.out.println("  ok"); }
        catch (RuntimeException e) { System.out.println("  error: " + e.getMessage()); }
    }
}
```

Output:

```
== top-ups
  Alice=2000.00  Bob=300.50
== transfers
  TransferResult[reference=req-3, from=W1, to=W2, amount=200.00, cashback=20.00]
  retry returns same result? true
  TransferResult[reference=req-4, from=W1, to=W2, amount=1200.00, cashback=25.00]
  Alice=645.00  Bob=1700.50  PROMO=55.00
== edge cases
  error: Insufficient funds in W2: balance 1700.50, needed 99999.00
  error: Amount must be positive: -5
  error: At most 2 decimal places: 1.005
  error: Cannot transfer to the same wallet
  error: Wallet not found: W99
  failed req-5 can be retried later: 10.00
== statement for Alice
  TX2   TOP_UP        2000.00  bal= 2000.00  ref=req-1
  TX4   TRANSFER_OUT   200.00  bal= 1800.00  ref=req-3 with W2
  TX7   CASHBACK_IN     20.00  bal= 1820.00  ref=req-3 with PROMO
  TX8   TRANSFER_OUT  1200.00  bal=  620.00  ref=req-4 with W2
  TX11  CASHBACK_IN     25.00  bal=  645.00  ref=req-4 with PROMO
  TX13  TRANSFER_IN     10.00  bal=  655.00  ref=req-5 with W2
== statement for PROMO
  TX1   TOP_UP         100.00  bal=  100.00  ref=budget-1
  TX6   CASHBACK_OUT    20.00  bal=   80.00  ref=req-3 with W1
  TX10  CASHBACK_OUT    25.00  bal=   55.00  ref=req-4 with W1
  TX14  CASHBACK_OUT     1.00  bal=   54.00  ref=req-5 with W2
== concurrency: 1000 opposite transfers on 8 threads
  total before=2400.50 after=2400.50 conserved=true
  Alice=655.00  Bob=1691.50  PROMO=54.00
```

Reading the output: marketing first funds `PROMO` with 100.00; the retried top-up `req-1` did not add another 2000; the retried transfer `req-3` returned the *same object* and moved no money; Alice's first transfer earned 10% (20.00), her 1200.00 transfer earned the flat 25.00 (the first-transfer offer no longer applies); the failed `req-5` stored nothing, so re-using that key later succeeded (and, being Bob's first successful transfer, earned him 1.00). Every cashback appears twice — `CASHBACK_OUT` on PROMO (100.00 → 80.00 → 55.00 → 54.00) and `CASHBACK_IN` on the receiver. Finally 1000 concurrent opposite transfers kept the total (PROMO included) exactly constant — no lost updates, no deadlock. (Each side starts with more than 500.00, so no transfer in that race can fail, and the final balances are the same on every run.)

Concurrency & edge cases

- **Race on one wallet:** every read-modify-write (read a value, compute a new one, write it back — three steps another thread can slip between) of `balance` happens with the wallet's lock held.
- **Deadlock:** avoided by a global lock order (lexicographic — dictionary order — wallet id; `"PROMO"` sorts before `"W1"`). Every two-lock path goes through one helper, `withBoth`, so the order cannot be forgotten. The program never holds more than two wallet locks at a time.
- **Double submit:** idempotency key → at most one execution per key.
- **Same key, different body** (client bug): today the stored result is returned. Production systems also store a hash (a short fingerprint computed from the data) of the request and reject a mismatch with an error.
- **Validation:** non-positive amounts, more than 2 decimals, same source and target, unknown wallet, insufficient funds.
- **Cashback money source:** a transfer from the `PROMO` wallet, so the books balance (double-entry: every credit has a matching debit). The cashback step runs after the transfer's locks are released, so for a moment the

transfer is visible without its cashback — fine for a reward. If the budget cannot cover the offer, the result reports a cashback of zero and the transfer is kept.

Extensions & follow-up questions

- **Multi-currency?** Each wallet has a `Currency`; transfers across currencies go through a `FxRateProvider` (FX = foreign exchange) interface; store both amounts and the rate used in the transaction. Use `java.util.Currency` for codes and decimal places.
- **Payment methods (card, bank, UPI — India's instant bank-to-bank payment system)?** A `PaymentMethod` interface with `charge(amount)`; top-up calls it first, and only credits the wallet on success (or uses a pending → confirmed state if the gateway is asynchronous). Users add and remove methods on their account.
- **Transfers to external accounts (withdraw to bank)?** Debit the wallet into a `PENDING_PAYOUT` state, call the bank through a `PayoutGateway` interface, and either confirm, or post a compensating credit if the bank rejects it.
- **Reversal / refund?** Never edit a past transaction; post a new compensating transaction referencing the original.
- **Distributed system?** Wallet balances in a database with row locks (`SELECT ... FOR UPDATE` locks the rows it reads until the transaction ends) or optimistic versioning; idempotency keys in a table with a unique constraint (a database rule that refuses a second row with the same value); ledgers as double-entry journals.
- **Long transfers blocking the idempotency map?** Use `putIfAbsent(key, new CompletableFuture<>())`: the winner runs the transfer outside the map and completes the future; duplicates just wait on `future.join()` (blocks until the result is ready). On failure, remove the key so a retry can run.
- **Limits (daily cap, KYC tier — KYC, "know your customer", is the identity check that sets how much a user may move)?** A chain of validators (Chain of Responsibility) run before `doTransfer`.
- **Why not `AtomicReference<BigDecimal>` + CAS (compare-and-set: replace a value only if it still equals what you last read, in one indivisible step)?** Works for a single wallet, but a transfer touches two balances that must change together; CAS on two variables is not atomic, so locks are simpler.

Real-world example

Stripe's (a large online-payments company) API accepts an `Idempotency-Key` header (a named piece of metadata sent along with an HTTP request): if a request times out and the client retries with the same key, Stripe returns the saved result of the first attempt instead of charging twice. Wallets such as Paytm and PhonePe run cashback campaigns with rules like "X% up to a cap on your first transaction" — exactly the shape of `FirstTransferOffer`.

In the interview

Lead with the three risks: exact money, concurrency, retries (and, if offers come up, "cashback must come out of a budget wallet"). Show `Money`, then `doTransfer` with the lock ordering (draw the A→B / B→A deadlock and how ordering breaks the cycle), then idempotency. Run the concurrent demo and point at `conserved=true`. Offers are a 2-minute Strategy mention. If time is short, cut multi-currency and payment methods — say you would add them behind interfaces.

Self-check

- **Q:** Why is `new BigDecimal("0.1")` preferred over `new BigDecimal(0.1)`? → **A:** The double constructor captures the binary approximation of 0.1 (`double` stores numbers in base 2, where 0.1 has no exact form) (a long, inexact decimal); the String constructor is exact.
- **Q:** Two threads transfer W1→W2 and W2→W1 at once. Why no deadlock? → **A:** Both lock W1 first (smaller id), so one waits for the other instead of each holding what the other needs.
- **Q:** A transfer fails for insufficient funds. Can the same idempotency key be reused? → **A:** Yes — `computeIfAbsent` stores nothing when the function throws.
- **Q:** Where would a "max 3 transfers per minute" rule go? → **A:** A validator run before `doTransfer`, not inside the wallet.

TL;DR

`BigDecimal` money, lock both wallets in a global order, debit before credit, idempotency key → run at most once, offers as Strategies paid out of a funded promotions wallet. Prove it with a concurrent demo where the total is conserved.

21.3 Design an Event Calendar (Flipkart)

Problem & clarifying questions

This is a Flipkart machine-coding statement (kumaransg/LLD, `Event_calendar_flipkart`). In short: onboard users with **working hours**; group users into **teams** (a user belongs to at most one team); create **events** on one day with a start and end time, whose participants are users, teams, or both. A user can be in **only one event at a time**. For team participants the event creator says how many **representatives (N)** each team must send; the system blocks the event against N *available* members of each team, and if any team cannot supply N the event is not created. Required features: create user, create team, create event, **get a user's events in a time range** (including events they were blocked for as a team representative), and **suggest free slots** for a list of users/teams on a day. Bonus: **update event time** (report failure if impossible) and **recurring events** (time, start date, end date, frequency). The statement says: no DB, no UI, no CLI (command-line interface: a program you type commands into) needed — a `main` or tests is enough — and "do not make assumptions, ask".

Clarifying questions (and the answers this solution uses):

Question	Answer used
Is an event that ends at 15:00 in conflict with one that starts at 15:00?	No. Slots are **half-open** `[start, end)` (the start minute belongs to the slot, the end minute does not).
Must an event lie inside every attendee's working hours?	Yes (the statement's Event5 fails because F starts at 11:00).
Which team members become representatives?	The first N free members in team order (a pluggable strategy).
If a user is invited directly *and* is in an invited team, do they count as a representative?	No — a direct attendee is not picked again as a representative.
Free slots for a team with N representatives: what counts as "free"?	A moment is free for the team if at least N members are working and not busy then.
Recurring event: if one occurrence clashes?	Reject the whole series (all-or-nothing).

A note on the statement's sample. For "free slots for user A and team T1 with 1 representative today" the sample answer says *10AM to 3PM and 4PM to 7PM*. But T1 = {C, E} and neither works before 11:00 (C starts 11:30, E 11:00), so by the statement's own rules the first slot should start at **11:00**. Our program prints `11:00-15:00, 16:00-19:00`. Pointing out such an inconsistency politely ("I think the sample means 11AM; shall I go with the rules?") is exactly what "do not make assumptions, ask" is testing. (A smaller slip in the same sample: its step 1 says "Onboard 5 Users" but then lists six, A to F. The demo creates all six.)

Requirements

Functional

1. Create user with working hours.
2. Create team of existing users (one team per user).
3. Create event for users and/or teams with N representatives; reject if any attendee is outside working hours, already busy, or a team lacks N free members.
4. Events of a user between two date-times (direct + as representative).
5. Free slots for users/teams on a day.
6. Bonus: update event time (atomic: old booking kept on failure).
7. Bonus: recurring events (daily/weekly between two dates).

Non-functional

- The check "is everyone free?" and the booking must be **atomic**, or two concurrent creations could book the same person twice.
- Representative choice should be replaceable (fairness, least-busy, random).

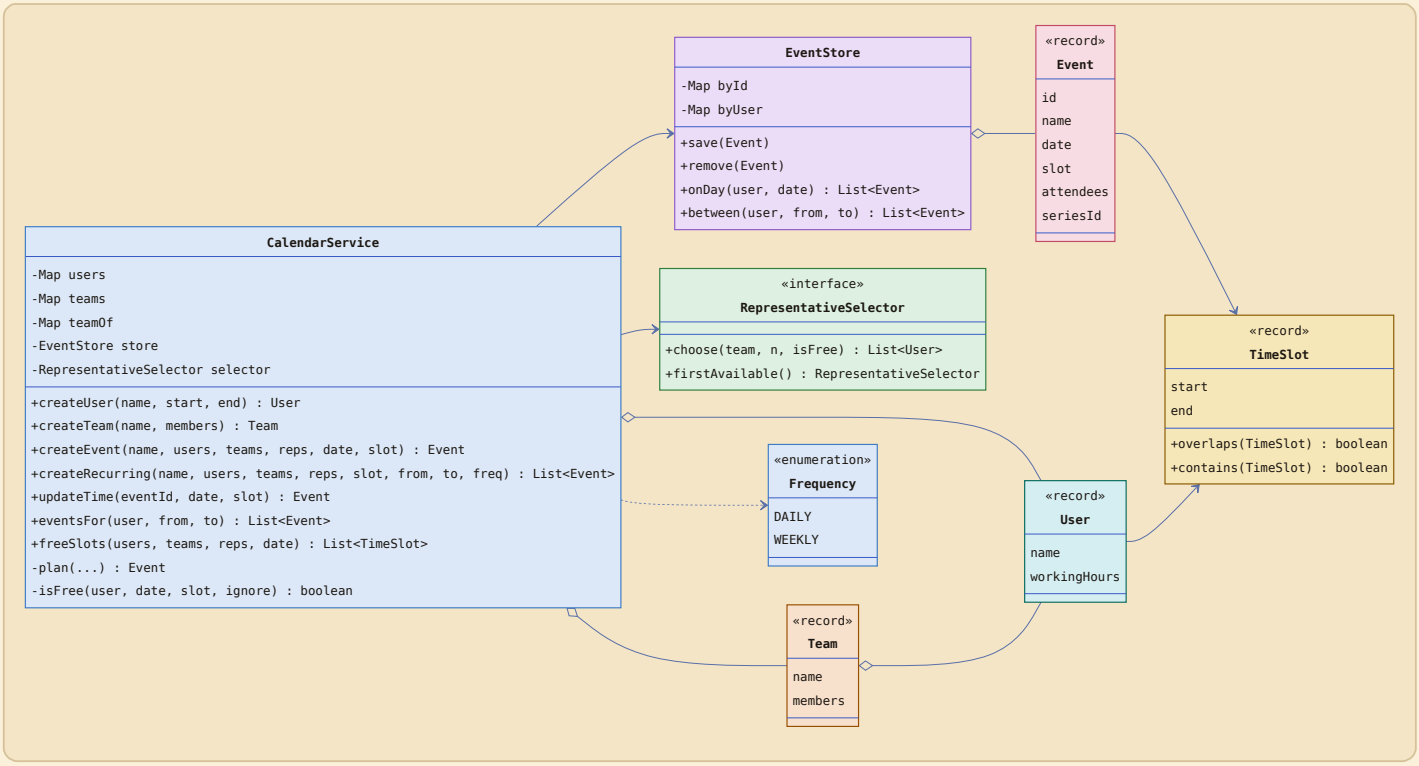
Use cases

Actor	Use cases
Admin	onboard user, create team
Organiser	create event, create recurring event, move event, find free slots
Attendee	view own events in a range

Core entities

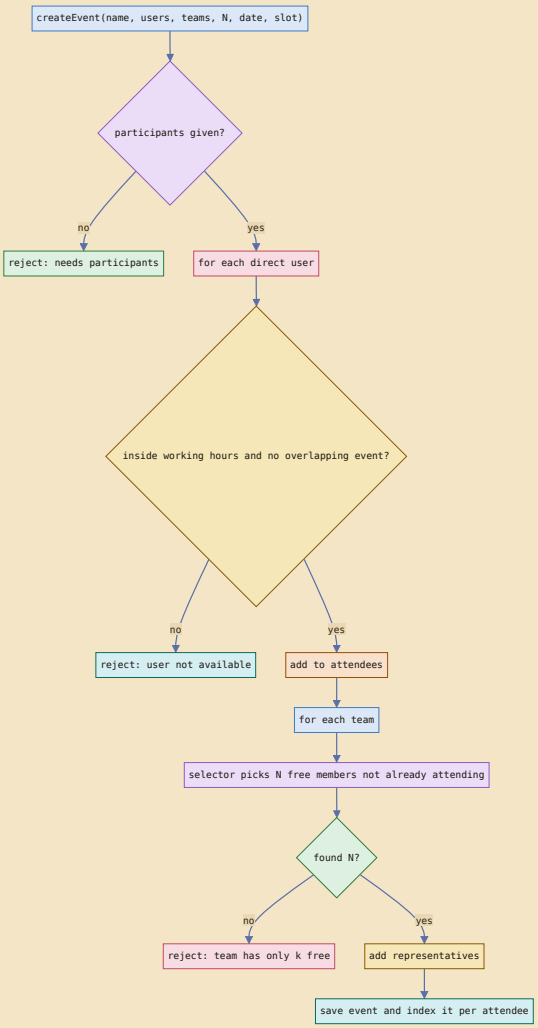
Class	Kind	Responsibility
`TimeSlot`	record	Half-open `[start, end)` on one day; `overlaps`, `contains`; validates start < end.
`User`	record	Name + working hours.
`Team`	record	Name + ordered members.
`Event`	record	One concrete occurrence: date, slot, what was requested (users, teams, N) and who is actually blocked (`attendees`), optional `seriesId`.
`Frequency`	enum	DAILY / WEEKLY — knows the next date.
`EventStore`	class	Repository + index `user → date → events` for fast conflict checks and range queries ("everything between X and Y"; `TreeMap.subMap` returns just that slice of a sorted map).
`RepresentativeSelector`	interface	Strategy: pick N free members of a team.
`CalendarService`	class	All use cases; planning (validation + representative choice) separated from saving.
`CalendarException`	exception	Readable reason for every rejection.

Class diagram

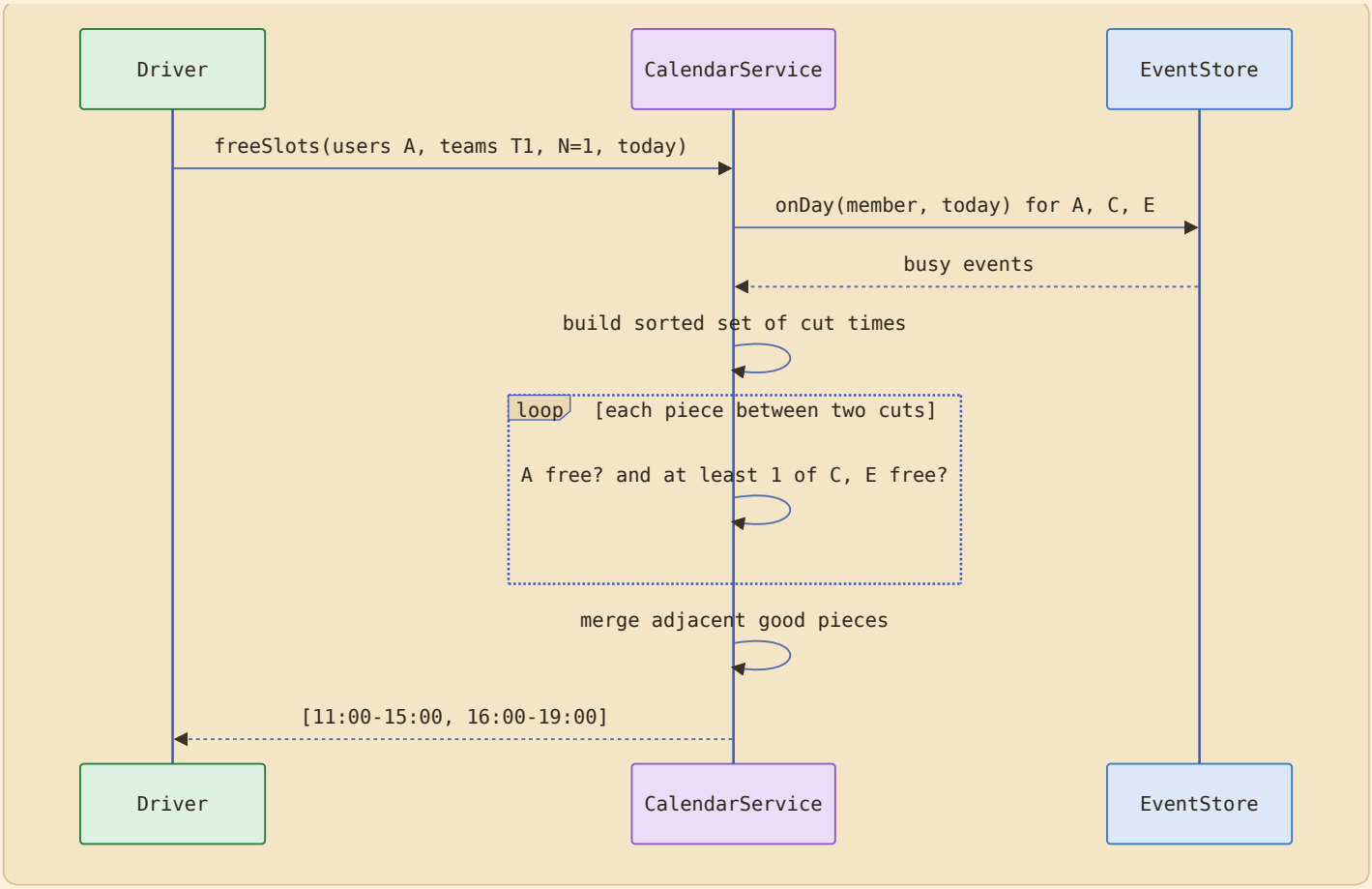


Key flows

Create an event (plan, then save):



Free-slot sweep. Collect every "interesting" time on that day: each involved person's working-hour start and end, and the start and end of each of their events. Sort them. Between two consecutive points nothing changes, so test each small piece once: all direct users free, and each team has at least N free members. Merge neighbouring good pieces. For the statement's example (A + T1, N = 1, today) the cut points are 10:00, 11:00, 11:30, 15:00, 16:00, 18:30, 19:00, 19:30, and the good pieces merge into 11:00–15:00 and 16:00–19:00.



Design decisions & patterns used

Decision	Why	Alternative rejected
Half-open intervals	Back-to-back meetings do not conflict; `overlaps` is one line: `a.start < b.end && b.start < a.end`.	Closed intervals — 14:00–15:00 would clash with 15:00–16:00.
`plan()` separate from `save()`	Validation + representative choice produce an `Event` without side effects (no change to any stored state). `createEvent`, `createRecurring` (plan all, then save all) and `updateTime` (plan while ignoring the old event, then swap) all reuse it — all-or-nothing for free.	Saving as you go and rolling back (undoing the saved steps) on failure.
`RepresentativeSelector` strategy	"Which N members?" is a policy; fairness or least-busy can be plugged in.	Hard-coded loop in the service.
Index `user → TreeMap<date, events>`	Conflict checks read only one user's one day; range queries use `subMap`.	Scanning all events each time.
Sweep over boundaries for free slots	Exact, simple, no minute-by-minute loops; works for any N.	Checking every 15 or 30 minutes — misses odd times like 11:30 and is slower.
Service methods `synchronized`	Check-then-book must be atomic; the store's lists are not thread-safe on their own.	Per-user locks — faster, but an event locks many users, so you need lock ordering; mention it as a scale-up.

Java implementation

```java
// File: lld/ch21/calendar/TimeSlot.java
package lld.ch21.calendar;

import java.time.LocalTime;

/** Half-open interval [start, end) inside one day. 14:00-15:00 and 15:00-16:00 do not overlap. */
public record TimeSlot(LocalTime start, LocalTime end) {
    public TimeSlot {
        if (!start.isBefore(end)) throw new IllegalArgumentException("start must be before end: " + start + "-" +
end);
    }
    public boolean overlaps(TimeSlot o) { return start.isBefore(o.end) && o.start.isBefore(end); }
    public boolean contains(TimeSlot o) { return !o.start.isBefore(start) && !o.end.isAfter(end); }
    @Override public String toString() { return start + "-" + end; }
}
```

```java
// File: lld/ch21/calendar/User.java
package lld.ch21.calendar;

public record User(String name, TimeSlot workingHours) { }
```

```java
// File: lld/ch21/calendar/Team.java
package lld.ch21.calendar;

import java.util.List;

/** Members are kept in onboarding order; representative selection walks this order. */
public record Team(String name, List<User> members) {
    public Team { members = List.copyOf(members); }
}
```

```java
// File: lld/ch21/calendar/Event.java
package lld.ch21.calendar;

import java.time.LocalDate;
import java.util.List;
import java.util.Set;

/**
 * One concrete occurrence. `attendees` is the resolved list of users that the event blocks
 * (direct users + the representatives chosen from each team).
 */
public record Event(String id, String name, LocalDate date, TimeSlot slot, Set<String> directUsers,
                    Set<String> teams, int representativesPerTeam, List<String> attendees, String seriesId) {
    public Event { attendees = List.copyOf(attendees); }
    public boolean blocks(String user) { return attendees.contains(user); }
    @Override public String toString() { return name + " " + date + " " + slot + " " + attendees; }
}
```

```java
// File: lld/ch21/calendar/Frequency.java
package lld.ch21.calendar;

import java.time.LocalDate;

public enum Frequency {
    DAILY { LocalDate next(LocalDate d) { return d.plusDays(1); } },
    WEEKLY { LocalDate next(LocalDate d) { return d.plusWeeks(1); } };

    abstract LocalDate next(LocalDate d);
}
```

```java
// File: lld/ch21/calendar/CalendarException.java
package lld.ch21.calendar;

public class CalendarException extends RuntimeException {
    public CalendarException(String message) { super(message); }
}
```

```java
// File: lld/ch21/calendar/EventStore.java
package lld.ch21.calendar;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.TreeMap;
import java.util.concurrent.ConcurrentHashMap;

/** Repository: events by id and an index user -> date -> events. Not thread-safe on its own: CalendarService
guards it. */
public class EventStore {
    private final Map<String, Event> byId = new ConcurrentHashMap<>();
    private final Map<String, TreeMap<LocalDate, List<Event>>> byUser = new ConcurrentHashMap<>();

    public void save(Event e) {
        byId.put(e.id(), e);
        for (String u : e.attendees())
            byUser.computeIfAbsent(u, k -> new TreeMap<>()).computeIfAbsent(e.date(), d -> new ArrayList<>
()).add(e);
    }

    public void remove(Event e) {
        byId.remove(e.id());
        for (String u : e.attendees()) {
            List<Event> day = byUser.getOrDefault(u, new TreeMap<>()).get(e.date());
            if (day != null) day.removeIf(x -> x.id().equals(e.id()));
        }
    }

    public Optional<Event> find(String id) { return Optional.ofNullable(byId.get(id)); }

    public List<Event> onDay(String user, LocalDate date) {
        return List.copyOf(byUser.getOrDefault(user, new TreeMap<>()).getOrDefault(date, List.of()));
    }

    public List<Event> between(String user, LocalDate from, LocalDate to) {
        List<Event> out = new ArrayList<>();
        byUser.getOrDefault(user, new TreeMap<>()).subMap(from, true, to, true).values().forEach(out::addAll);
        return out;
    }

    public Collection<Event> all() { return List.copyOf(byId.values()); }
}
```

```java
// File: lld/ch21/calendar/RepresentativeSelector.java
package lld.ch21.calendar;

import java.util.List;
import java.util.function.Predicate;

/** Strategy: which N free members represent a team. */
public interface RepresentativeSelector {
    List<User> choose(Team team, int n, Predicate<User> isFree);

    /** Default: first N free members in team order (deterministic, easy to explain). */
    static RepresentativeSelector firstAvailable() {
        return (team, n, isFree) -> team.members().stream().filter(isFree).limit(n).toList();
    }
}
```

```java
// File: lld/ch21/calendar/CalendarService.java
package lld.ch21.calendar;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Comparator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.TreeSet;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

public class CalendarService {
    private final Map<String, User> users = new ConcurrentHashMap<>();
    private final Map<String, Team> teams = new ConcurrentHashMap<>();
    private final Map<String, String> teamOf = new ConcurrentHashMap<>();   // user -> team
    private final EventStore store = new EventStore();
    private final RepresentativeSelector selector;
    private final AtomicInteger seq = new AtomicInteger();

    public CalendarService(RepresentativeSelector selector) { this.selector = selector; }

    public User createUser(String name, LocalTime workStart, LocalTime workEnd) {
        User u = new User(name, new TimeSlot(workStart, workEnd));
        if (users.putIfAbsent(name, u) != null) throw new CalendarException("User already exists: " + name);
        return u;
    }

    public synchronized Team createTeam(String name, String... memberNames) {
        if (teams.containsKey(name)) throw new CalendarException("Team already exists: " + name);
        List<User> members = new ArrayList<>();
        for (String m : memberNames) {
            User u = user(m);
            if (teamOf.containsKey(m)) throw new CalendarException(m + " already belongs to team " +
teamOf.get(m));
            members.add(u);
        }
        members.forEach(u -> teamOf.put(u.name(), name));
        Team t = new Team(name, members);
        teams.put(name, t);
        return t;
    }

    /** synchronized: the check-then-book sequence must be atomic or two events could grab the same person. */
    public synchronized Event createEvent(String name, Set<String> userNames, Set<String> teamNames,
                                          int reps, LocalDate date, TimeSlot slot) {
        Event e = plan(name, userNames, teamNames, reps, date, slot, null, null);
        store.save(e);
        return e;
    }

    /** Bonus: recurring event. All occurrences are validated first; nothing is saved unless all fit. */
    public synchronized List<Event> createRecurring(String name, Set<String> userNames, Set<String> teamNames, int
reps,
                                                    TimeSlot slot, LocalDate from, LocalDate to, Frequency freq) {
        String series = "S" + seq.incrementAndGet();
        List<Event> planned = new ArrayList<>();
        for (LocalDate d = from; !d.isAfter(to); d = freq.next(d)) {
            planned.add(plan(name + "@" + d, userNames, teamNames, reps, d, slot, series, null));
        }
        planned.forEach(store::save);
        return planned;
    }
```

```
    /** Bonus: move an event. We try the new time ignoring the event itself; on failure nothing changes. */
    public synchronized Event updateTime(String eventId, LocalDate date, TimeSlot slot) {
        Event old = store.find(eventId).orElseThrow(() -> new CalendarException("No such event " + eventId));
        Event moved = plan(old.name(), old.directUsers(), old.teams(), old.representativesPerTeam(),
                date, slot, old.seriesId(), old);          // keeps old.id(); throws if the new time does not fit
        store.remove(old);
        store.save(moved);
        return moved;
    }

    /** Events a user is blocked for between two date-times (includes team-representative bookings). */
    public synchronized List<Event> eventsFor(String userName, LocalDateTime from, LocalDateTime to) {
        user(userName);
        return store.between(userName, from.toLocalDate(), to.toLocalDate()).stream()
                .filter(e -> LocalDateTime.of(e.date(), e.slot().end()).isAfter(from)
                          && LocalDateTime.of(e.date(), e.slot().start()).isBefore(to))
                .sorted(Comparator.comparing(Event::date).thenComparing(e -> e.slot().start()))
                .toList();
    }

    /**
     * Free slots on a day where every listed user is free AND each listed team has at least `reps`
     * free members. Sweep: cut the day at every boundary (working-hour edges, event edges),
     * test each small piece, then merge adjacent good pieces.
     */
    public synchronized List<TimeSlot> freeSlots(Set<String> userNames, Set<String> teamNames, int reps, LocalDate
date) {
        List<User> direct = userNames.stream().map(this::user).toList();
        List<Team> tms = teamNames.stream().map(this::team).toList();
        List<User> everyone = new ArrayList<>(direct);
        tms.forEach(t -> everyone.addAll(t.members()));

        TreeSet<LocalTime> cuts = new TreeSet<>();
        for (User u : everyone) {
            cuts.add(u.workingHours().start());
            cuts.add(u.workingHours().end());
            for (Event e : store.onDay(u.name(), date)) { cuts.add(e.slot().start()); cuts.add(e.slot().end()); }
        }
        List<TimeSlot> result = new ArrayList<>();
        LocalTime prev = null;
        for (LocalTime cut : cuts) {
            if (prev != null) {
                TimeSlot piece = new TimeSlot(prev, cut);
                boolean ok = direct.stream().allMatch(u -> isFree(u, date, piece, null))
                        && tms.stream().allMatch(t -> t.members().stream()
                                .filter(m -> isFree(m, date, piece, null)).count() >= reps);
                if (ok) {
                    int last = result.size() - 1;
                    if (last >= 0 && result.get(last).end().equals(piece.start()))
                        result.set(last, new TimeSlot(result.get(last).start(), piece.end()));
                    else result.add(piece);
                }
            }
            prev = cut;
        }
        return result;
    }

    // ---------------------------------------------------------------- helpers

    private Event plan(String name, Set<String> userNames, Set<String> teamNames, int reps,
                       LocalDate date, TimeSlot slot, String series, Event ignore) {
        if (userNames.isEmpty() && teamNames.isEmpty()) throw new CalendarException("Event needs participants");
        if (!teamNames.isEmpty() && reps < 1) throw new CalendarException("Representatives must be >= 1");
        Set<String> attendees = new LinkedHashSet<>();
        for (String n : new TreeSet<>(userNames)) {          // sorted => deterministic order
            User u = user(n);
            if (!isFree(u, date, slot, ignore))
```

```
                throw new CalendarException(name + ": " + n + " is not available " + date + " " + slot);
            attendees.add(n);
        }
        for (String tn : new TreeSet<>(teamNames)) {
            Team t = team(tn);
            List<User> chosen = selector.choose(t, reps,
                    m -> !attendees.contains(m.name()) && isFree(m, date, slot, ignore));
            if (chosen.size() < reps)
                throw new CalendarException(name + ": team " + tn + " has only " + chosen.size()
                        + " free member(s), needs " + reps);
            chosen.forEach(m -> attendees.add(m.name()));
        }
        String id = ignore != null ? ignore.id() : "E" + seq.incrementAndGet();
        return new Event(id, name, date, slot, Set.copyOf(userNames), Set.copyOf(teamNames), reps,
                List.copyOf(attendees), series);
    }

    private boolean isFree(User u, LocalDate date, TimeSlot slot, Event ignore) {
        if (!u.workingHours().contains(slot)) return false;
        return store.onDay(u.name(), date).stream()
                .filter(e -> ignore == null || !e.id().equals(ignore.id()))
                .noneMatch(e -> e.slot().overlaps(slot));
    }

    private User user(String n) {
        User u = users.get(n);
        if (u == null) throw new CalendarException("Unknown user " + n);
        return u;
    }

    private Team team(String n) {
        Team t = teams.get(n);
        if (t == null) throw new CalendarException("Unknown team " + n);
        return t;
    }

    public synchronized Collection<Event> allEvents() { return store.all(); }
}
```

```java
// File: lld/ch21/calendar/Main.java
package lld.ch21.calendar;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.List;
import java.util.Set;
import java.util.function.Supplier;

/** Replays the sample test cases from the problem statement, then the bonus features. */
public class Main {
    public static void main(String[] args) {
        CalendarService cal = new CalendarService(RepresentativeSelector.firstAvailable());
        LocalDate today = LocalDate.of(2026, 9, 28), tomorrow = today.plusDays(1);

        System.out.println("== 1. onboard users");
        cal.createUser("A", t("10:00"), t("19:00"));
        cal.createUser("B", t("09:30"), t("17:30"));
        cal.createUser("C", t("11:30"), t("18:30"));
        cal.createUser("D", t("10:00"), t("18:00"));
        cal.createUser("E", t("11:00"), t("19:30"));
        cal.createUser("F", t("11:00"), t("18:30"));
        System.out.println("  6 users created");

        System.out.println("== 2. teams");
        cal.createTeam("T1", "C", "E");
        cal.createTeam("T2", "B", "D", "F");
        System.out.println("  T1=[C,E] T2=[B,D,F]");
        attempt(() -> cal.createTeam("T3", "A", "C"));

        System.out.println("== 3. events");
        attempt(() -> cal.createEvent("Event1", Set.of("A"), Set.of("T1"), 2, tomorrow, slot("14:00", "15:00")));
        attempt(() -> cal.createEvent("Event2", Set.of("C"), Set.of(), 0, tomorrow, slot("14:00", "15:00")));
        attempt(() -> cal.createEvent("Event3", Set.of(), Set.of("T1", "T2"), 2, today, slot("15:00", "16:00")));
        attempt(() -> cal.createEvent("Event4", Set.of("A"), Set.of("T2"), 1, today, slot("15:00", "16:00")));
        attempt(() -> cal.createEvent("Event5", Set.of("A", "F"), Set.of(), 0, today, slot("10:00", "11:00")));

        System.out.println("== 4. events in a time range (today 10:00 .. tomorrow 17:00)");
        LocalDateTime from = LocalDateTime.of(today, t("10:00")), to = LocalDateTime.of(tomorrow, t("17:00"));
        for (String u : List.of("A", "B", "F"))
            System.out.println("  " + u + ": " + cal.eventsFor(u, from, to).stream().map(Event::name).toList());

        System.out.println("== 5. free slots today for A + T1 (1 representative)");
        System.out.println("  " + cal.freeSlots(Set.of("A"), Set.of("T1"), 1, today));
        System.out.println("  free slots today for B + D: " + cal.freeSlots(Set.of("B", "D"), Set.of(), 0, today));

        System.out.println("== bonus: update event time");
        Event e1 = cal.allEvents().stream().filter(e -> e.name().equals("Event1")).findFirst().orElseThrow();
        attempt(() -> cal.updateTime(e1.id(), tomorrow, slot("18:30", "19:00")));   // needs 2 of T1, but C stops
at 18:30
        attempt(() -> cal.updateTime(e1.id(), tomorrow, slot("16:00", "17:00")));

        System.out.println("== bonus: recurring daily stand-up (A + D), 3 days");
        attempt(() -> cal.createRecurring("Standup", Set.of("A", "D"), Set.of(), 0, slot("10:00", "10:15"),
                today, today.plusDays(2), Frequency.DAILY));
        attempt(() -> cal.createRecurring("Sync", Set.of("A"), Set.of(), 0, slot("15:30", "16:30"),
                today, today.plusDays(2), Frequency.DAILY));   // clashes with Event4 today -> whole series
rejected
        System.out.println("  A today..+2: " + cal.eventsFor("A", LocalDateTime.of(today, t("00:00")),
                LocalDateTime.of(today.plusDays(2), t("23:59"))).stream().map(Event::name).toList());
    }

    static LocalTime t(String s) { return LocalTime.parse(s); }
    static TimeSlot slot(String a, String b) { return new TimeSlot(t(a), t(b)); }

    static void attempt(Supplier<?> s) {
        try { System.out.println("  ok: " + s.get()); }
```

```
        catch (RuntimeException e) { System.out.println("  FAILED: " + e.getMessage()); }
    }
}
```

Output:

```
== 1. onboard users
  6 users created
== 2. teams
  T1=[C,E] T2=[B,D,F]
  FAILED: C already belongs to team T1
== 3. events
  ok: Event1 2026-09-29 14:00-15:00 [A, C, E]
  FAILED: Event2: C is not available 2026-09-29 14:00-15:00
  ok: Event3 2026-09-28 15:00-16:00 [C, E, B, D]
  ok: Event4 2026-09-28 15:00-16:00 [A, F]
  FAILED: Event5: F is not available 2026-09-28 10:00-11:00
== 4. events in a time range (today 10:00 .. tomorrow 17:00)
  A: [Event4, Event1]
  B: [Event3]
  F: [Event4]
== 5. free slots today for A + T1 (1 representative)
  [11:00-15:00, 16:00-19:00]
  free slots today for B + D: [10:00-15:00, 16:00-17:30]
== bonus: update event time
  FAILED: Event1: team T1 has only 1 free member(s), needs 2
  ok: Event1 2026-09-29 16:00-17:00 [A, C, E]
== bonus: recurring daily stand-up (A + D), 3 days
  ok: [Standup@2026-09-28 2026-09-28 10:00-10:15 [A, D], Standup@2026-09-29 2026-09-29 10:00-10:15 [A, D],
Standup@2026-09-30 2026-09-30 10:00-10:15 [A, D]]
  FAILED: Sync@2026-09-28: A is not available 2026-09-28 15:30-16:30
  A today..+2: [Standup@2026-09-28, Event4, Standup@2026-09-29, Event1, Standup@2026-09-30]
```

Checking against the statement: Event2 fails (C is in Event1), Event5 fails (F starts at 11:00), user A sees Event4 and Event1, and user B sees Event3 — because T2's first two free members (B, D) represented it in Event3, which left F as the only free T2 member for Event4. The update to 18:30–19:00 is refused because Event1 needs both T1 members and C stops at 18:30; the old booking stays. The "Sync" series is rejected as a whole because its first day clashes with Event4.

Concurrency & edge cases

- **Double booking under concurrency:** all mutating and reading methods are `synchronized` on the service, so the "is free?" check and the save are one atomic step.
- **Edge cases handled:** start ≥ end, unknown user/team, user already in a team, empty participant list, N < 1 with teams, direct attendee also in an invited team (not double-counted), event outside working hours, recurring series with any clash (nothing saved), moving an event onto a slot where it would conflict with *itself* (we ignore the event being moved).
- **Time zones:** out of scope; everything is local time on one day. Real calendars store instants (exact moments, in UTC — the world reference time zone) plus the user's zone.
- **Multi-day events:** the statement fixes start and end on the same day; extension would split an event into per-day `TimeSlot`s.

Extensions & follow-up questions

- **Invites with accept/decline?** Add `InviteStatus {PENDING, ACCEPTED, DECLINED}` per attendee; a declined attendee no longer blocks the slot.
- **Fair representative choice?** A `LeastBusySelector` that sorts free members by number of events this week.
- **Recurring events at scale?** Store one *rule* (like the iCalendar RRULE — the standard text format for "repeats every Tuesday until June") and expand occurrences lazily (only when asked) for the queried range instead of materialising (creating and storing) every occurrence.
- **Rooms?** Treat a room like a user with "working hours" = building hours; the same availability logic applies.
- **Faster free-slot search for big teams?** For each team, a sweep counting free members per piece is O(members × events); with many members, use a count array or segment tree (a tree that answers "how many in this range?"

quickly) over minutes.

Real-world example

Google Calendar's "Find a time" and Outlook's Scheduling Assistant lay attendees' busy blocks side by side and highlight times when everyone is free — the same intersection this sweep computes. Both products store recurring meetings as a rule plus exceptions (following the iCalendar RRULE standard) instead of thousands of copies.

In the interview

Start by drawing a timeline with two users' working hours and events, and show that free time is "working time minus busy time, intersected". Explain half-open intervals in one sentence. Code `TimeSlot`, `plan()` and `createEvent` first — that satisfies the first three requirements. Then `eventsFor`, then the sweep. Mention the sample inconsistency. Bonus features fall out of `plan()` almost for free, which is the design win to point at.

Self-check

- **Q:** Why does `plan()` take an `ignore` event? → **A:** When moving an event, its current booking must not count as a conflict with its new time.
- **Q:** Why is the recurring series all-or-nothing easy here? → **A:** We plan every occurrence first (no side effects) and save only if all plans succeed.
- **Q:** What is the overlap test for half-open intervals? → **A:** `a.start < b.end && b.start < a.end`.
- **Q:** Why a `TreeMap` per user? → **A:** Dates are ordered, so a range query is `subMap(from, to)` instead of a full scan.

TL;DR

Half-open `TimeSlot`s, per-user day index, a side-effect-free `plan()` reused by create/recurring/update, a pluggable representative selector, and a boundary sweep for free slots. Ask about the sample that contradicts the rules.

21.4 Design a LeetCode-like Coding Platform — "Coding Blox" (Flipkart)

Problem & clarifying questions

A Flipkart SDE-2 (Software Development Engineer, level 2: a mid-level role) machine-coding statement (kumaransg/LLD, `leetcode-lld-flipkart-coding-blox`). Coding Blox lets users sign up, create contests, and take part in contests hosted by others. Questions and contests each have a level — LOW, MEDIUM or HIGH — and a contest of a level uses **all questions of that level** (the question list is independent of the contest). The platform is driven by commands:

Command	Meaning
`CreateUser <name>`	Register a user with a default score of **1500**.
`CreateQuestion <level> <score>`	Add a question; ids auto-increment from 1.
`ListQuestion [level]`	All questions, or only that level.
`CreateContest <name> <level> <creator>`	Creator automatically attends; contest ids auto-increment from 1.
`ListContest [level]`	All contests, or only that level.
`AttendContest <contestId> <user>`	Register for a contest.
`RunContest <contestId> <creator>`	Only the creator may run it. Simulate which questions each participant solved (random), sum their scores (`contestPoints`), then update ratings: LOW → `score + (points − 50)`, MEDIUM → `score + (points − 30)`, HIGH → `score + points`.
`LeaderBoard <asc/desc>`	Users with scores, sorted.
Bonus `ContestHistory <contestId>`	Per-contest leaderboard: user, points, solved question ids.
Bonus `WithdrawContest <contestId> <user>`	Leave before the contest runs; the creator cannot withdraw.

Expectations in the statement: create the sample data yourself, demo through a driver or tests, modular OO (object-oriented) design with interfaces, extensible, graceful edge-case handling, DRY ("don't repeat yourself": each piece of logic written once).

Question	Answer used
How many questions does a user "solve" in a run?	Random 1..all questions of the contest level. The randomness sits behind a `SolveSimulator` interface and is **seeded** for a reproducible demo.
Can a contest run twice?	No — second run is an error, so ratings are never applied twice.
Can someone attend after the run?	No.
Leaderboard tie-break?	By name.
Can a contest be created if no question of that level exists?	No (it would be empty).

Requirements

Functional: all commands above, including both bonus commands. **Non-functional:** commands from a built-in input list; errors printed, processing continues; scoring rule and solving logic replaceable; rating update safe if two contests finish at once.

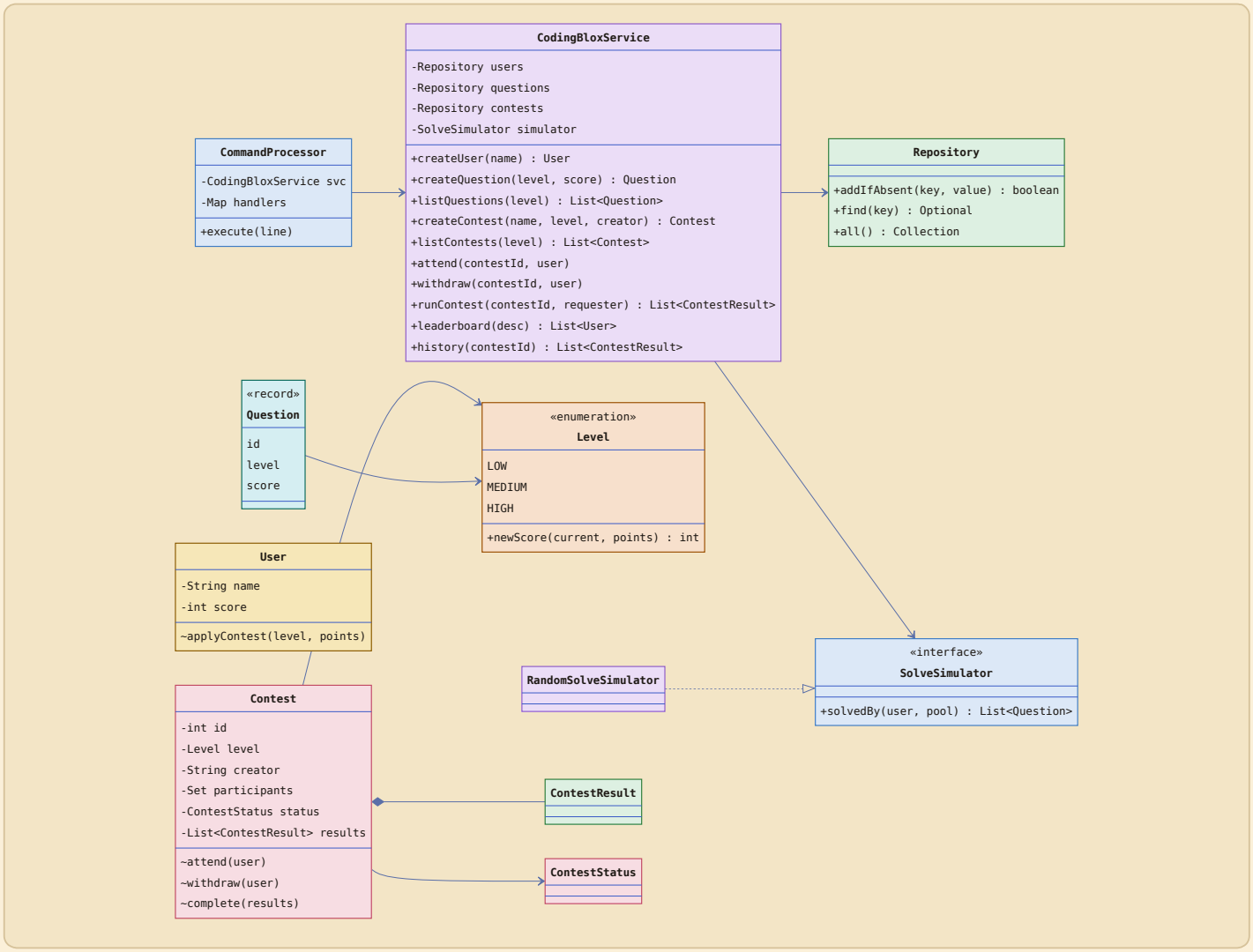
Use cases

Actor	Use cases
User	sign up, attend/withdraw, view leaderboard, view contest history
Contest creator (a user)	create contest, run contest
Admin / problem setter	create and list questions
Judge (system actor, simulated)	decides which questions each participant solved

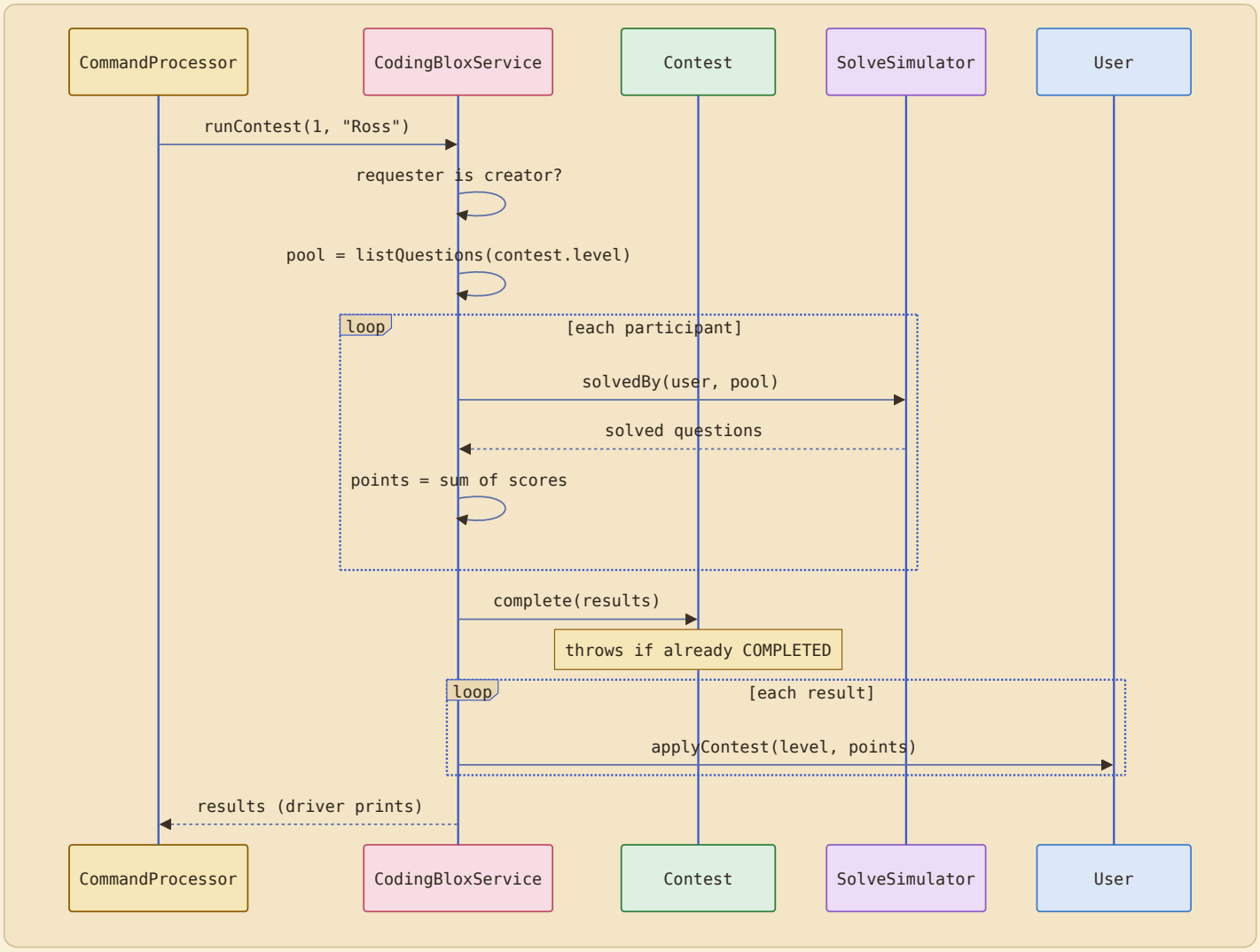
Core entities

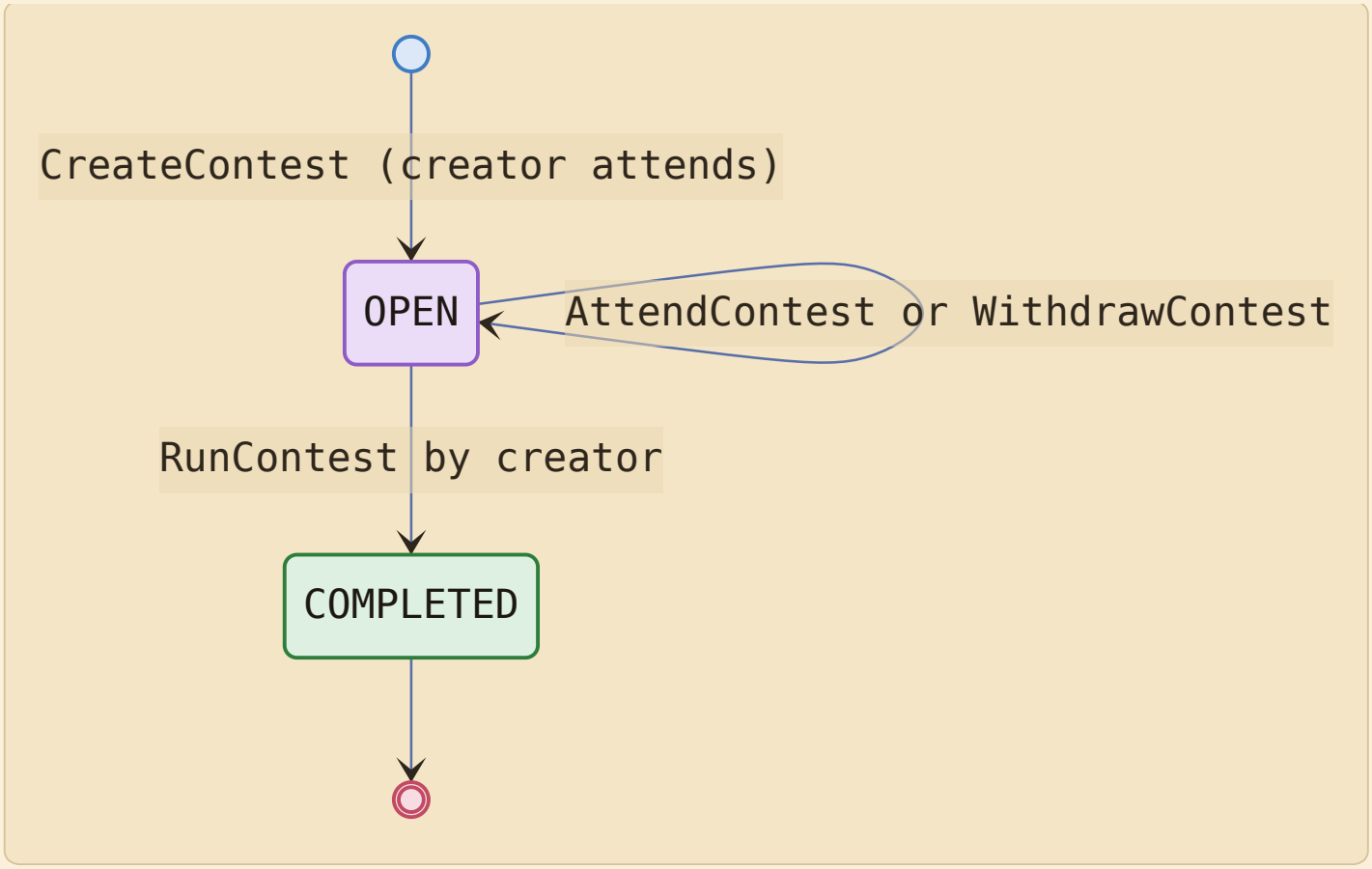
Class	Kind	Responsibility
`Level`	enum	LOW/MEDIUM/HIGH, each with its penalty; `newScore(current, points)` implements the rating rule.
`User`	class	Name + score; `applyContest` updates the score atomically.
`Question`	record	id, level, score.
`Contest`	class	id, name, level, creator, participants, status, results; guards "open only" rules.
`ContestStatus`	enum	OPEN, COMPLETED.
`ContestResult`	record	user, points, solved ids.
`SolveSimulator` / `RandomSolveSimulator`	interface / class	Strategy for "who solved what".
`Repository<K,V>`	generic class	In-memory storage reused for users, questions and contests.
`CodingBloxService`	class	Business rules for every command.
`CommandProcessor`	class	Parses (splits and reads) a line, dispatches to (calls) a handler from a map, prints output/errors.
`BloxException`	exception	Readable failure.

Class diagram



Key flows


CommandProcessor
CodingBloxService
Contest
SolveSimulator
User
runContest(1, "Ross")
requester is creator?
pool = listQuestions(contest.level)
loop
[each participant]
solvedBy(user, pool)
solved questions
points = sum of scores
complete(results)
throws if already COMPLETED
loop
[each result]
applyContest(level, points)
results (driver prints)
CommandProcessor
CodingBloxService
Contest
SolveSimulator
User



Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Command name → handler map	Command	Adding a command is one `put`; no giant `switch`. Parsing and printing stay out of the service.	Parsing inside the service.
Scoring rule inside `Level` enum	Strategy (each enum constant carries its own data)	The rule depends only on the level; a new level brings its own rule.	`if (level == LOW) ... else if ...` in `runContest`.
`SolveSimulator` interface	Strategy	Randomness is replaceable: seeded for demo, scripted for tests, a real judge later.	`new Random()` inside the service — untestable.
`Contest.complete` checks status	Entity guards invariant	"Run once" enforced where the state lives, so scores are never applied twice.	Checking in the driver.
Generic `Repository<K,V>`	Repository	Three stores, one class (DRY — the statement asks for it).	Three copy-pasted maps with lookup code.
Results computed, then `complete`, then ratings	Ordering for safety	If `complete` throws (already run), no rating has changed yet.	Updating ratings inside the loop.

Java implementation

```java
// File: lld/ch21/codingblox/Level.java
package lld.ch21.codingblox;

/** Difficulty of a question or a contest. Each contest level carries its own scoring rule. */
public enum Level {
    LOW(50), MEDIUM(30), HIGH(0);

    private final int penalty;
    Level(int penalty) { this.penalty = penalty; }

    /** newScore = currentScore + (contestPoints - penalty), exactly as the statement defines. */
    public int newScore(int current, int contestPoints) { return current + contestPoints - penalty; }

    public static Level parse(String s) {
        try { return valueOf(s.toUpperCase()); }
        catch (IllegalArgumentException e) { throw new BloxException("Unknown level: " + s); }
    }
}
```

```java
// File: lld/ch21/codingblox/User.java
package lld.ch21.codingblox;

public class User {
    public static final int DEFAULT_SCORE = 1500;
    private final String name;
    private int score = DEFAULT_SCORE;

    public User(String name) { this.name = name; }
    public String name() { return name; }
    public synchronized int score() { return score; }
    /** Read-modify-write under one lock, so two contests finishing together cannot lose an update. */
    synchronized void applyContest(Level level, int points) { score = level.newScore(score, points); }
}
```

```java
// File: lld/ch21/codingblox/Question.java
package lld.ch21.codingblox;

public record Question(int id, Level level, int score) {
    @Override public String toString() { return "Q" + id + "(" + level + "," + score + ")"; }
}
```

```java
// File: lld/ch21/codingblox/Contest.java
package lld.ch21.codingblox;

import java.util.ArrayList;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

public class Contest {
    private final int id;
    private final String name;
    private final Level level;
    private final String creator;
    private final Set<String> participants = new LinkedHashSet<>();
    private final List<ContestResult> results = new ArrayList<>();
    private ContestStatus status = ContestStatus.OPEN;

    Contest(int id, String name, Level level, String creator) {
        this.id = id; this.name = name; this.level = level; this.creator = creator;
        participants.add(creator);                      // the creator always attends
    }

    public int id() { return id; }
    public String name() { return name; }
    public Level level() { return level; }
    public String creator() { return creator; }
    public synchronized ContestStatus status() { return status; }
    public synchronized List<String> participants() { return List.copyOf(participants); }
    public synchronized List<ContestResult> results() { return List.copyOf(results); }

    synchronized void attend(String user) {
        requireOpen();
        if (!participants.add(user)) throw new BloxException(user + " already registered for contest " + id);
    }

    synchronized void withdraw(String user) {
        requireOpen();
        if (user.equals(creator)) throw new BloxException("Creator cannot withdraw from contest " + id);
        if (!participants.remove(user)) throw new BloxException(user + " is not in contest " + id);
    }

    synchronized void complete(List<ContestResult> r) {
        requireOpen();
        results.addAll(r);
        status = ContestStatus.COMPLETED;
    }

    private void requireOpen() {
        if (status != ContestStatus.OPEN) throw new BloxException("Contest " + id + " has already run");
    }

    @Override public String toString() { return id + " " + name + " " + level + " by " + creator + " [" + status()
+ "]"; }
}
```

```java
// File: lld/ch21/codingblox/ContestStatus.java
package lld.ch21.codingblox;

public enum ContestStatus { OPEN, COMPLETED }
```

```java
// File: lld/ch21/codingblox/ContestResult.java
package lld.ch21.codingblox;

import java.util.List;

public record ContestResult(String user, int points, List<Integer> solvedIds) { }
```

```java
// File: lld/ch21/codingblox/BloxException.java
package lld.ch21.codingblox;

public class BloxException extends RuntimeException {
    public BloxException(String message) { super(message); }
}
```

```java
// File: lld/ch21/codingblox/Repository.java
package lld.ch21.codingblox;

import java.util.Collection;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

/** Tiny generic in-memory repository reused for users, questions and contests. */
public class Repository<K, V> {
    private final Map<K, V> items = new ConcurrentHashMap<>();
    public boolean addIfAbsent(K key, V value) { return items.putIfAbsent(key, value) == null; }
    public Optional<V> find(K key) { return Optional.ofNullable(items.get(key)); }
    public Collection<V> all() { return List.copyOf(items.values()); }
}
```

```java
// File: lld/ch21/codingblox/SolveSimulator.java
package lld.ch21.codingblox;

import java.util.List;

/** Strategy: decides which questions a user solved. Real system = judge results; demo = seeded random. */
public interface SolveSimulator {
    List<Question> solvedBy(String user, List<Question> pool);
}
```

```java
// File: lld/ch21/codingblox/RandomSolveSimulator.java
package lld.ch21.codingblox;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Random;

public class RandomSolveSimulator implements SolveSimulator {
    private final Random random;
    public RandomSolveSimulator(long seed) { this.random = new Random(seed); }

    @Override public List<Question> solvedBy(String user, List<Question> pool) {
        if (pool.isEmpty()) return List.of();
        List<Question> copy = new ArrayList<>(pool);
        Collections.shuffle(copy, random);
        int k = 1 + random.nextInt(copy.size());          // solves between 1 and all questions
        List<Question> picked = new ArrayList<>(copy.subList(0, k));
        picked.sort(Comparator.comparingInt(Question::id));
        return picked;
    }
}
```

```java
// File: lld/ch21/codingblox/CodingBloxService.java
package lld.ch21.codingblox;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class CodingBloxService {
    private final Repository<String, User> users = new Repository<>();
    private final Repository<Integer, Question> questions = new Repository<>();
    private final Repository<Integer, Contest> contests = new Repository<>();
    private final AtomicInteger questionSeq = new AtomicInteger();
    private final AtomicInteger contestSeq = new AtomicInteger();
    private final SolveSimulator simulator;

    public CodingBloxService(SolveSimulator simulator) { this.simulator = simulator; }

    public User createUser(String name) {
        User u = new User(name);
        if (!users.addIfAbsent(name, u)) throw new BloxException("User already exists: " + name);
        return u;
    }

    public Question createQuestion(Level level, int score) {
        if (score <= 0) throw new BloxException("Score must be positive");
        Question q = new Question(questionSeq.incrementAndGet(), level, score);
        questions.addIfAbsent(q.id(), q);
        return q;
    }

    public List<Question> listQuestions(Level levelOrNull) {
        return questions.all().stream().filter(q -> levelOrNull == null || q.level() == levelOrNull)
                .sorted(Comparator.comparingInt(Question::id)).toList();
    }

    public Contest createContest(String name, Level level, String creator) {
        user(creator);
        if (listQuestions(level).isEmpty()) throw new BloxException("No " + level + " questions to build a
contest");
        Contest c = new Contest(contestSeq.incrementAndGet(), name, level, creator);
        contests.addIfAbsent(c.id(), c);
        return c;
    }

    public List<Contest> listContests(Level levelOrNull) {
        return contests.all().stream().filter(c -> levelOrNull == null || c.level() == levelOrNull)
                .sorted(Comparator.comparingInt(Contest::id)).toList();
    }

    public void attend(int contestId, String userName) { user(userName); contest(contestId).attend(userName); }

    public void withdraw(int contestId, String userName) { user(userName); contest(contestId).withdraw(userName); }

    /** Only the creator may run. Every participant gets random solved questions, then scores update. */
    public List<ContestResult> runContest(int contestId, String requester) {
        Contest c = contest(contestId);
        if (!c.creator().equals(requester)) throw new BloxException(requester + " did not create contest " +
contestId);
        List<Question> pool = listQuestions(c.level());
        List<ContestResult> results = new ArrayList<>();
        for (String name : c.participants()) {
            List<Question> solved = simulator.solvedBy(name, pool);
            int points = solved.stream().mapToInt(Question::score).sum();
            results.add(new ContestResult(name, points, solved.stream().map(Question::id).toList()));
        }
        c.complete(results);                                // throws if already run: scores never double-apply
        for (ContestResult r : results) user(r.user()).applyContest(c.level(), r.points());
```

```
        return results;
    }

    public List<User> leaderboard(boolean descending) {
        Comparator<User> byScore = Comparator.comparingInt(User::score);
        if (descending) byScore = byScore.reversed();
        return users.all().stream().sorted(byScore.thenComparing(User::name)).toList();
    }

    public List<ContestResult> history(int contestId) {
        Contest c = contest(contestId);
        if (c.status() != ContestStatus.COMPLETED) throw new BloxException("Contest " + contestId + " has not run
yet");
        return c.results().stream().sorted(Comparator.comparingInt(ContestResult::points).reversed()
                .thenComparing(ContestResult::user)).toList();
    }

    private User user(String n) { return users.find(n).orElseThrow(() -> new BloxException("No such user: " + n));
}
    private Contest contest(int id) { return contests.find(id).orElseThrow(() -> new BloxException("No such
contest: " + id)); }
}
```

```java
// File: lld/ch21/codingblox/CommandProcessor.java
package lld.ch21.codingblox;

import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.function.Consumer;

/**
 * Driver layer. Parses one text line and dispatches to a handler (Command pattern: name -> action).
 * All printing lives here; the service returns data only.
 */
public class CommandProcessor {
    private final CodingBloxService svc;
    private final Map<String, Consumer<String[]>> handlers = new LinkedHashMap<>();

    public CommandProcessor(CodingBloxService svc) {
        this.svc = svc;
        handlers.put("createuser", a -> out("User " + svc.createUser(arg(a, 1)).name() + " created with score
1500"));
        handlers.put("createquestion", a -> out("Question created: " + svc.createQuestion(Level.parse(arg(a, 1)),
Integer.parseInt(arg(a, 2)))));
        handlers.put("listquestion", a -> svc.listQuestions(a.length > 1 ? Level.parse(a[1]) : null).forEach(q ->
out("  " + q)));
        handlers.put("createcontest", a -> out("Contest created: " + svc.createContest(arg(a, 1),
Level.parse(arg(a, 2)), arg(a, 3))));
        handlers.put("listcontest", a -> svc.listContests(a.length > 1 ? Level.parse(a[1]) : null).forEach(c ->
out("  " + c)));
        handlers.put("attendcontest", a -> { svc.attend(Integer.parseInt(arg(a, 1)), arg(a, 2)); out(a[2] + "
registered for contest " + a[1]); });
        handlers.put("withdrawcontest", a -> { svc.withdraw(Integer.parseInt(arg(a, 1)), arg(a, 2)); out(a[2] + "
withdrew from contest " + a[1]); });
        handlers.put("runcontest", a -> svc.runContest(Integer.parseInt(arg(a, 1)), arg(a, 2))
                .forEach(r -> out("  " + r.user() + " solved " + r.solvedIds() + " -> " + r.points() + " pts")));
        handlers.put("leaderboard", a -> svc.leaderboard(!a[a.length - 1].equalsIgnoreCase("asc"))
                .forEach(u -> out("  " + u.name() + " : " + u.score())));
        handlers.put("contesthistory", a -> svc.history(Integer.parseInt(arg(a, 1)))
                .forEach(r -> out("  " + r.user() + " : " + r.points() + " " + r.solvedIds())));
    }

    public void execute(String line) {
        System.out.println("> " + line);
        String[] a = Arrays.stream(line.trim().split("\\s+")).map(s -> s.replace("\"", "")).toArray(String[]::new);
        Consumer<String[]> h = handlers.get(a[0].toLowerCase());
        try {
            if (h == null) throw new BloxException("Unknown command: " + a[0]);
            h.accept(a);
        } catch (BloxException | NumberFormatException e) {
            out("ERROR: " + e.getMessage());
        }
    }

    private static String arg(String[] a, int i) {
        if (i >= a.length) throw new BloxException("Missing argument #" + i + " for " + a[0]);
        return a[i];
    }

    private static void out(String s) { System.out.println(s); }
}
```

```java
// File: lld/ch21/codingblox/Main.java
package lld.ch21.codingblox;

import java.util.List;

public class Main {
    static final List<String> SAMPLE_INPUT = List.of(
            "CreateUser Ross", "CreateUser Monica", "CreateUser Joey", "CreateUser Chandler",
            "CreateQuestion \"LOW\" 10", "CreateQuestion LOW 20", "CreateQuestion LOW 30",
            "CreateQuestion MEDIUM 40", "CreateQuestion MEDIUM 50", "CreateQuestion HIGH 100",
            "ListQuestion LOW",
            "CreateContest \"diwali_contest\" LOW \"Ross\"",
            "CreateContest holi_contest MEDIUM Monica",
            "ListContest",
            "AttendContest 1 Monica", "AttendContest 1 Joey", "AttendContest 1 Chandler",
            "AttendContest 1 Joey",                  // duplicate
            "WithdrawContest 1 Chandler",
            "WithdrawContest 1 Ross",                // creator cannot withdraw
            "RunContest 1 Monica",                   // not the creator
            "RunContest 1 Ross",
            "RunContest 1 Ross",                     // already run
            "AttendContest 1 Chandler",              // too late
            "ContestHistory 1",
            "AttendContest 2 Chandler",
            "RunContest 2 Monica",
            "LeaderBoard score desc",
            "LeaderBoard asc",
            "CreateContest x HIGH Rachel",           // unknown user
            "Dance 1");                              // unknown command

    public static void main(String[] args) {
        CommandProcessor cli = new CommandProcessor(new CodingBloxService(new RandomSolveSimulator(42)));
        SAMPLE_INPUT.forEach(cli::execute);
    }
}
```

Output:

```
> CreateUser Ross
User Ross created with score 1500
> CreateUser Monica
User Monica created with score 1500
> CreateUser Joey
User Joey created with score 1500
> CreateUser Chandler
User Chandler created with score 1500
> CreateQuestion "LOW" 10
Question created: Q1(LOW,10)
> CreateQuestion LOW 20
Question created: Q2(LOW,20)
> CreateQuestion LOW 30
Question created: Q3(LOW,30)
> CreateQuestion MEDIUM 40
Question created: Q4(MEDIUM,40)
> CreateQuestion MEDIUM 50
Question created: Q5(MEDIUM,50)
> CreateQuestion HIGH 100
Question created: Q6(HIGH,100)
> ListQuestion LOW
  Q1(LOW,10)
  Q2(LOW,20)
  Q3(LOW,30)
> CreateContest "diwali_contest" LOW "Ross"
Contest created: 1 diwali_contest LOW by Ross [OPEN]
> CreateContest holi_contest MEDIUM Monica
Contest created: 2 holi_contest MEDIUM by Monica [OPEN]
> ListContest
  1 diwali_contest LOW by Ross [OPEN]
  2 holi_contest MEDIUM by Monica [OPEN]
> AttendContest 1 Monica
Monica registered for contest 1
> AttendContest 1 Joey
Joey registered for contest 1
> AttendContest 1 Chandler
Chandler registered for contest 1
> AttendContest 1 Joey
ERROR: Joey already registered for contest 1
> WithdrawContest 1 Chandler
Chandler withdrew from contest 1
> WithdrawContest 1 Ross
ERROR: Creator cannot withdraw from contest 1
> RunContest 1 Monica
ERROR: Monica did not create contest 1
> RunContest 1 Ross
  Ross solved [2] -> 20 pts
  Monica solved [1, 2] -> 30 pts
  Joey solved [1, 2] -> 30 pts
> RunContest 1 Ross
ERROR: Contest 1 has already run
> AttendContest 1 Chandler
ERROR: Contest 1 has already run
> ContestHistory 1
  Joey : 30 [1, 2]
  Monica : 30 [1, 2]
  Ross : 20 [2]
> AttendContest 2 Chandler
Chandler registered for contest 2
> RunContest 2 Monica
  Monica solved [4, 5] -> 90 pts
  Chandler solved [4] -> 40 pts
> LeaderBoard score desc
  Monica : 1540
  Chandler : 1510
  Joey : 1480
  Ross : 1470
> LeaderBoard asc
```

```
  Ross : 1470
  Joey : 1480
  Chandler : 1510
  Monica : 1540
> CreateContest x HIGH Rachel
ERROR: No such user: Rachel
> Dance 1
ERROR: Unknown command: Dance
```

Checking the arithmetic: contest 1 is LOW, so each participant gets `points − 50`: Ross 1500 + (20 − 50) = 1470, Joey and Monica 1500 + (30 − 50) = 1480. Contest 2 is MEDIUM (`points − 30`): Monica 1480 + (90 − 30) = 1540, Chandler 1500 + (40 − 30) = 1510. The leaderboard matches. (Which questions are "solved" comes from the seeded simulator — change the seed and you get different but equally valid results.)

Concurrency & edge cases

- **Concurrent commands:** repositories are `ConcurrentHashMap`s; `Contest` methods are `synchronized`; `User.applyContest` does its read-modify-write under the user's lock, so two contests finishing together cannot lose a rating update.
- **Double run:** `Contest.complete` throws if already completed; ratings are updated *after* it succeeds.
- **Known gap (say it):** `runContest` copies the participant list, simulates, then calls `complete`. A user who attends in that short window is registered but gets no result. Fix: add a `RUNNING` status and have `Contest` switch to it and hand back the participant list in one `synchronized` step, so no one can attend or withdraw once the run has started.
- **Edge cases handled:** duplicate user, unknown user/contest/level, non-positive question score, contest with no questions of its level, duplicate attend, attend/withdraw after the run, creator withdraw, non-creator run, missing arguments, non-numeric ids, unknown command.

Extensions & follow-up questions

- **Pick a fixed number of questions per contest?** A `QuestionSelectionStrategy` (random k of level, or by tags) called at contest creation; store the chosen ids in the contest.
- **Real submissions?** Replace `SolveSimulator` with a judge that returns verdicts per submission; add `Submission(user, question, verdict, time)` and score by time/penalty like ICPC (the International Collegiate Programming Contest).
- **Elo-style ratings?** (Elo is the chess rating system: beating a stronger player earns more points.) A `RatingStrategy` interface taking all participants' results (ratings depend on opponents), selected per contest type.
- **Scheduled contests?** Add `startTime/endTime`, a `SCHEDULED → RUNNING → COMPLETED` state machine, and a scheduler that triggers `run`.
- **Leaderboard at scale?** Keep a `TreeSet` (a set that keeps its items sorted) sorted by (score, name) or a Redis sorted set; update on each rating change instead of sorting on every query.

Real-world example

LeetCode's weekly contests publish a live ranking during the contest, and afterwards each participant's contest rating is recalculated from their rank relative to others — a more elaborate version of the per-level rule in this statement. HackerRank and Codeforces follow the same shape: contests reference a pool of problems, participants register, and ratings change only after the contest is finalised.

In the interview

Paste the sample commands into the driver list first. Build `CreateUser`, `CreateQuestion`, `CreateContest`, `AttendContest`, `RunContest`, `LeaderBoard` in that order, running after each. Point out the three design moves: command map, scoring rule on the enum, and randomness behind an interface (with a seed). Then bonus commands — both are tiny because `Contest` already stores results and guards its status.

Self-check

- **Q:** Why must `RunContest` update ratings only after marking the contest complete? → **A:** If the contest had already run, `complete` throws before any rating changes — no double application.
- **Q:** How would you add a `VERY_HIGH` level with bonus +20? → **A:** Add an enum constant with its own rule (e.g. a negative penalty); nothing else changes.

- **Q:** How do you make `RunContest` deterministic in tests? → **A:** Inject a scripted `SolveSimulator` (or a seeded random one).
- **Q:** Where is "creator always attends" enforced? → **A:** In the `Contest` constructor.

TL;DR

Command map in the driver, rules in the service, invariants in `Contest`, scoring on the `Level` enum, randomness behind a seeded `SolveSimulator`. Complete first, then update ratings.

21.5 Design a Property Listing Site — "PropertyHunt" (ClearTrip)

Problem & clarifying questions

A ClearTrip machine-coding statement (kumaransg/LLD, `PropertyHunt`). The folder holds two versions: a text `readme.md` and a longer version as screenshots in `Question Image/`; the summary below follows the longer one, which adds ViewSold, the range rules and the "shortlist *other users'* property" wording. Build a console application (a text-only program run in a terminal) for a property platform where users can: **register**; **list a property** they want to sell or rent; **view their own listings**; **shortlist** other users' properties; **view shortlisted** properties (sold ones still appear, with status SOLD); **search** available properties with any combination of **location** (case-insensitive — capital and small letters treated the same — exact match), **price range**, **listing type** (sell/rent), **size range**, **number of rooms** (1BHK/2BHK/3BHK — bedroom, hall, kitchen: 1BHK is one bedroom), and **sort by** price or size (ascending); **mark sold** (only the owner); and **view sold** properties. Ranges: `X-Y` inclusive; a single `X` means "at least X" for size and "at most X" for price. **Bonus:** sizes in sq ft / sq m / sq yd (buyer and seller may use different units); **nearby locations** on a listing, with nearby matches shown last; prices like `40000`, `40k`, `60L`, `2Cr` (k = thousand; L = lakh = 100,000; Cr = crore = 10,000,000 — the Indian number units); comma-separated values for location and rooms in search. The statement asks for in-memory storage, graceful errors ("Property does not exist"), and a clean, extensible OO design.

Question	Answer used
Input format?	One command per line, fields separated by `
Can I shortlist my own property?	No — the screenshot version says users shortlist *others'* properties. (The text readme's sample has Jack shortlisting his own listing; that is exactly the kind of conflict to ask about.)
Can I shortlist a sold property?	No; but a property sold *after* I shortlisted it stays in my list with status SOLD.
Units internally?	Price in whole rupees (`long`), size in square feet (`double`).
Sort with location?	Exact location matches first, nearby matches after, then the chosen sort.
Mark sold — record buyer and price?	The screenshot sample writes `marksold - 2, xyz, 50000` (property, buyer, price). We store only the status; keeping buyer and sale price is a small extension (a `Sale` record).

The sample search (`bellandur, 30000-50000, rent, ...`) returns a *sarjapur* property. In the screenshot version that is consistent: the sarjapur listing declares `nearby bellandur|kadubeesanahalli|marathahalli`, so it is a nearby match. The text readme's sample drops the `nearby` field, and there the result only makes sense if you know about the bonus. Our demo lists sarjapur with the same three nearby localities, and the search returns it.

Requirements

Functional: RegisterUser, ListProperty, Search, Shortlist, ViewShortlisted, ViewListed, MarkSold, ViewSold — plus all four bonus items. **Non-functional:** search filters combine freely; new filters or units without rewriting search; clear errors; safe under concurrent use.

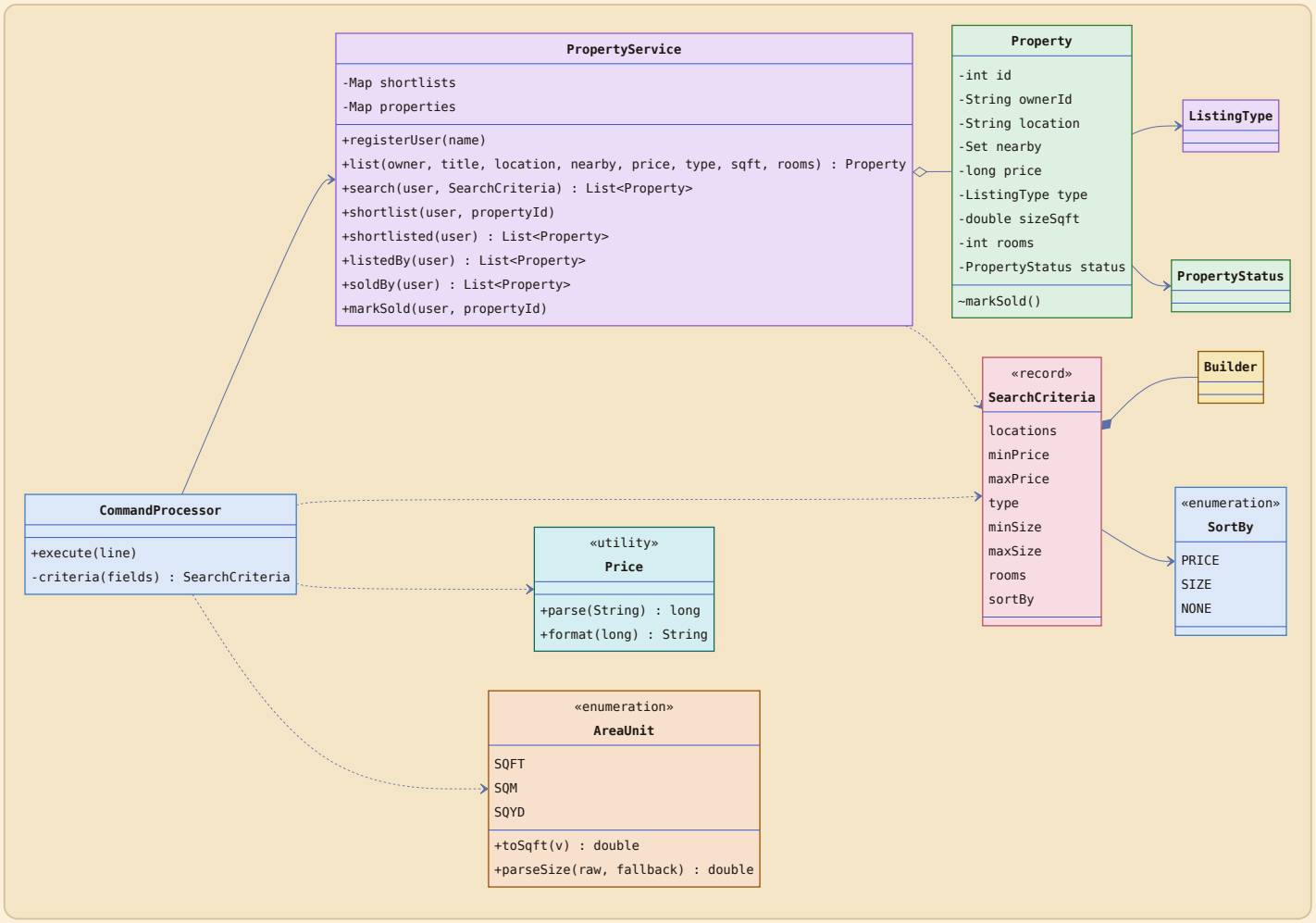
Use cases

Actor	Use cases
Seller / landlord	list property, view listed, mark sold, view sold
Buyer / tenant	search, shortlist, view shortlisted
(Any user can be both — roles are per property, not per account: the owner of a property is its seller.)	

Core entities

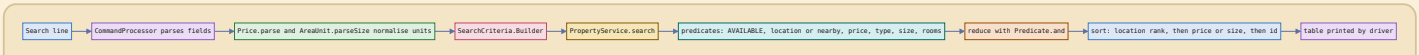
Class	Kind	Responsibility
`Property`	class	Listing data, owner, nearby locations, status; `markSold` guards double sale.
`ListingType`, `PropertyStatus`, `SortBy`	enums	SELL/RENT, AVAILABLE/SOLD, sort comparators.
`Price`	utility	Parse `40k/60L/2Cr/1.5Cr` → rupees; format back.
`AreaUnit`	enum	SQFT/SQM/SQYD with conversion to square feet; parses "1500 sq ft", "70 sqm".
`SearchCriteria` (+ `Builder`)	record	All-optional search parameters.
`PropertyService`	class	Register, list, search, shortlist, mark sold, views; owner-vs-buyer rules.
`CommandProcessor`	class	Parses `
`HuntException`	exception	Readable errors.

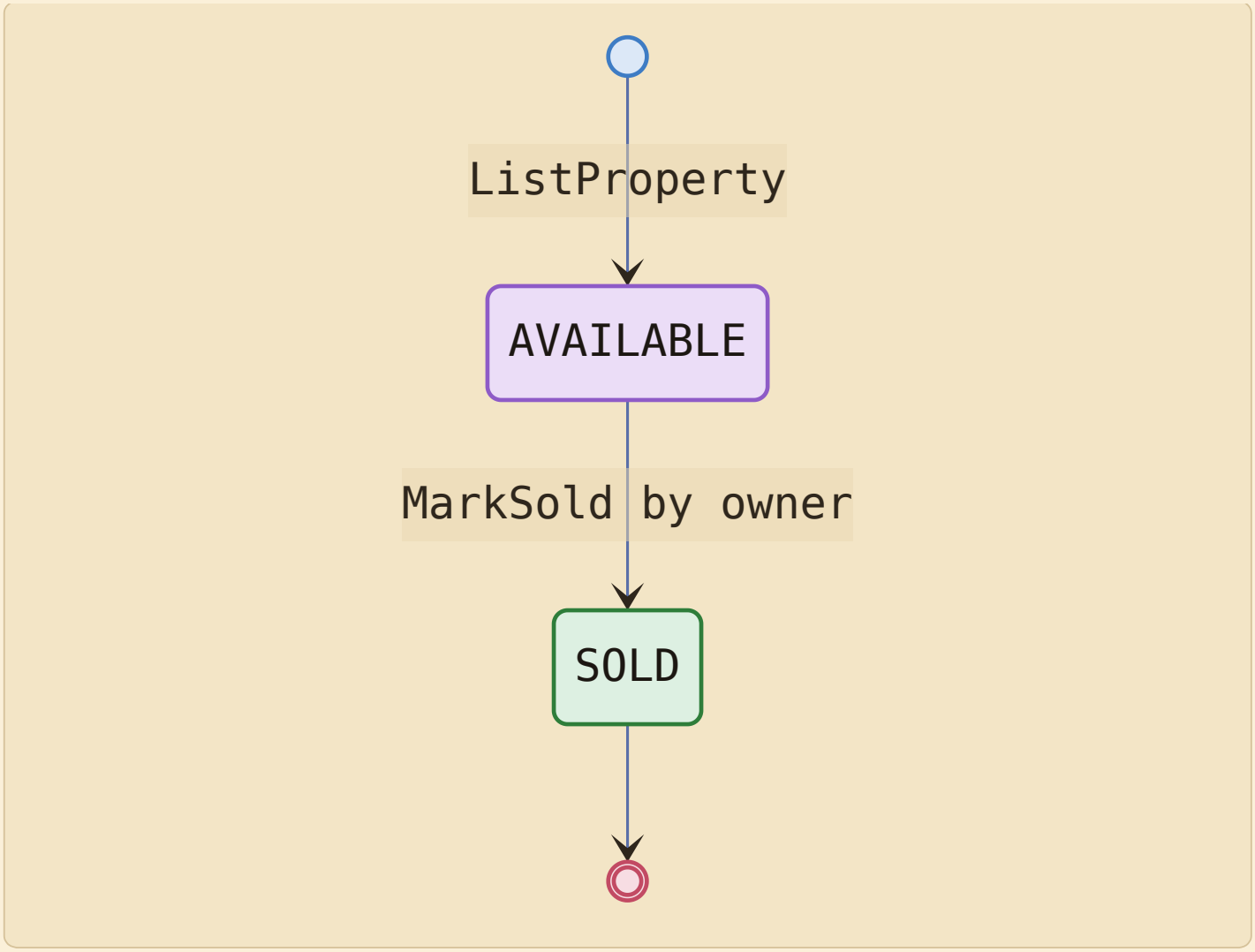
Class diagram



Key flows

Search = build criteria → list of predicates → one combined predicate → sort:





Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
All-optional `SearchCriteria` with a Builder	Builder	Eight optional fields; a constructor call with eight nulls is unreadable.	Telescoping constructors (a chain of constructors, each taking one more parameter) / overloads (several methods with the same name but different parameters).
Each criterion becomes a `Predicate<Property>`, reduced with `and`	Specification	Any combination works; a new filter is one line; no nested `if` pyramid.	One `if` per combination.
`SortBy` enum of comparators	Strategy	Adding "sort by rooms" is one constant.	`switch` inside search.
Normalise at the boundary (rupees, sq ft)	—	Business code compares plain numbers; units only matter while parsing.	Storing strings like "90L" and comparing them.
Location rank 0/1/2	—	Exact matches first, nearby second, others filtered out — one small function serves filtering *and* ordering.	Two separate searches merged.
Shortlist stored per user as a set of ids	—	View shortlisted shows current status (SOLD appears automatically).	Copying property objects into the shortlist — they would go stale.

Java implementation

```java
// File: lld/ch21/propertyhunt/Price.java
package lld.ch21.propertyhunt;

import java.math.BigDecimal;
import java.util.Locale;

/** Parses and prints Indian-style prices: 40000, 40k, 60L, 2Cr, 1.5Cr. Stored as whole rupees. */
public final class Price {
    private Price() { }

    public static long parse(String raw) {
        String s = raw.trim().toLowerCase(Locale.ROOT).replace(",", "").replace(" ", "");
        long mult = 1;
        if (s.endsWith("cr")) { mult = 10_000_000L; s = s.substring(0, s.length() - 2); }
        else if (s.endsWith("l")) { mult = 100_000L; s = s.substring(0, s.length() - 1); }
        else if (s.endsWith("k")) { mult = 1_000L; s = s.substring(0, s.length() - 1); }
        try {
            return new BigDecimal(s).multiply(BigDecimal.valueOf(mult)).longValueExact();
        } catch (ArithmeticException | NumberFormatException e) {
            throw new HuntException("Bad price: " + raw);
        }
    }

    public static String format(long rupees) {
        if (rupees >= 10_000_000L) return shift(rupees, 7) + "Cr";
        if (rupees >= 100_000L) return shift(rupees, 5) + "L";
        if (rupees >= 1_000L) return shift(rupees, 3) + "k";
        return Long.toString(rupees);
    }

    /** Exact decimal shift, e.g. shift(25000000, 7) = "2.5" - no double rounding. */
    private static String shift(long rupees, int places) {
        return BigDecimal.valueOf(rupees).movePointLeft(places).stripTrailingZeros().toPlainString();
    }
}
```

```java
// File: lld/ch21/propertyhunt/AreaUnit.java
package lld.ch21.propertyhunt;

import java.util.Locale;

/** Every size is converted to square feet internally so buyer and seller units can differ. */
public enum AreaUnit {
    SQFT(1.0), SQM(10.7639), SQYD(9.0);

    private final double sqftPerUnit;
    AreaUnit(double f) { this.sqftPerUnit = f; }

    public double toSqft(double v) { return v * sqftPerUnit; }

    /** Accepts "sqft", "sq ft", "sqm", "sq meter", "sqyd", "sq yard" ... (spaces ignored). */
    public static AreaUnit parse(String raw) {
        String s = raw.toLowerCase(Locale.ROOT).replace(" ", "").replace(".", "");
        if (s.isEmpty() || s.startsWith("sqf") || s.equals("ft")) return SQFT;
        if (s.startsWith("sqm")) return SQM;
        if (s.startsWith("sqy")) return SQYD;
        throw new HuntException("Unknown size unit: " + raw);
    }

    /** "1800 sqft" -> 1800.0 ; "200 sqyd" -> 1800.0 ; a bare number uses `fallback`. */
    public static double parseSize(String raw, AreaUnit fallback) {
        String s = raw.trim();
        int i = 0;
        while (i < s.length() && (Character.isDigit(s.charAt(i)) || s.charAt(i) == '.')) i++;
        if (i == 0) throw new HuntException("Bad size: " + raw);
        double value = Double.parseDouble(s.substring(0, i));
        String unit = s.substring(i).trim();
        return (unit.isEmpty() ? fallback : parse(unit)).toSqft(value);
    }
}
```

```java
// File: lld/ch21/propertyhunt/ListingType.java
package lld.ch21.propertyhunt;

public enum ListingType {
    SELL, RENT;

    public static ListingType parse(String s) {
        return switch (s.trim().toLowerCase()) {
            case "sell", "sale" -> SELL;
            case "rent" -> RENT;
            default -> throw new HuntException("Listing type must be sell or rent: " + s);
        };
    }
}
```

```java
// File: lld/ch21/propertyhunt/PropertyStatus.java
package lld.ch21.propertyhunt;

public enum PropertyStatus { AVAILABLE, SOLD }
```

```java
// File: lld/ch21/propertyhunt/Property.java
package lld.ch21.propertyhunt;

import java.util.Set;

public class Property {
    private final int id;
    private final String ownerId;
    private final String title;
    private final String location;          // stored lower-case for case-insensitive match
    private final Set<String> nearby;       // bonus: nearby locations, lower-case
    private final long price;
    private final ListingType type;
    private final double sizeSqft;
    private final int rooms;
    private PropertyStatus status = PropertyStatus.AVAILABLE;

    Property(int id, String ownerId, String title, String location, Set<String> nearby, long price,
             ListingType type, double sizeSqft, int rooms) {
        this.id = id; this.ownerId = ownerId; this.title = title; this.location = location;
        this.nearby = Set.copyOf(nearby); this.price = price; this.type = type;
        this.sizeSqft = sizeSqft; this.rooms = rooms;
    }

    public int id() { return id; }
    public String ownerId() { return ownerId; }
    public String title() { return title; }
    public String location() { return location; }
    public Set<String> nearby() { return nearby; }
    public long price() { return price; }
    public ListingType type() { return type; }
    public double sizeSqft() { return sizeSqft; }
    public int rooms() { return rooms; }
    public synchronized PropertyStatus status() { return status; }

    synchronized void markSold() {
        if (status == PropertyStatus.SOLD) throw new HuntException("Property " + id + " is already sold");
        status = PropertyStatus.SOLD;
    }
}
```

```java
// File: lld/ch21/propertyhunt/SortBy.java
package lld.ch21.propertyhunt;

import java.util.Comparator;

public enum SortBy {
    PRICE(Comparator.comparingLong(Property::price)),
    SIZE(Comparator.comparingDouble(Property::sizeSqft)),
    NONE(Comparator.comparingInt(Property::id));

    final Comparator<Property> comparator;
    SortBy(Comparator<Property> c) { this.comparator = c.thenComparingInt(Property::id); }

    public static SortBy parse(String s) { return s == null ? NONE : valueOf(s.trim().toUpperCase()); }
}
```

```java
// File: lld/ch21/propertyhunt/SearchCriteria.java
package lld.ch21.propertyhunt;

import java.util.Set;

/** Every field is optional (null = "don't care"). Built with the nested Builder. */
public record SearchCriteria(Set<String> locations, Long minPrice, Long maxPrice, ListingType type,
                             Double minSize, Double maxSize, Set<Integer> rooms, SortBy sortBy) {

    public static Builder builder() { return new Builder(); }

    public static final class Builder {
        private Set<String> locations; private Long minPrice, maxPrice; private ListingType type;
        private Double minSize, maxSize; private Set<Integer> rooms; private SortBy sortBy = SortBy.NONE;

        public Builder locations(Set<String> l) { locations = l; return this; }
        public Builder price(Long min, Long max) { minPrice = min; maxPrice = max; return this; }
        public Builder type(ListingType t) { type = t; return this; }
        public Builder size(Double min, Double max) { minSize = min; maxSize = max; return this; }
        public Builder rooms(Set<Integer> r) { rooms = r; return this; }
        public Builder sortBy(SortBy s) { sortBy = s; return this; }
        public SearchCriteria build() {
            return new SearchCriteria(locations, minPrice, maxPrice, type, minSize, maxSize, rooms, sortBy);
        }
    }
}
```

```java
// File: lld/ch21/propertyhunt/HuntException.java
package lld.ch21.propertyhunt;

public class HuntException extends RuntimeException {
    public HuntException(String message) { super(message); }
}
```

```java
// File: lld/ch21/propertyhunt/PropertyService.java
package lld.ch21.propertyhunt;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Predicate;

public class PropertyService {
    private final Map<String, Set<Integer>> shortlists = new ConcurrentHashMap<>();  // user -> ids (the user
registry)
    private final Map<Integer, Property> properties = new ConcurrentHashMap<>();
    private final AtomicInteger seq = new AtomicInteger();

    public void registerUser(String name) {
        if (shortlists.putIfAbsent(name, ConcurrentHashMap.newKeySet()) != null)
            throw new HuntException("User already registered: " + name);
    }

    public Property list(String owner, String title, String location, Set<String> nearby, long price,
                         ListingType type, double sizeSqft, int rooms) {
        requireUser(owner);
        if (price <= 0 || sizeSqft <= 0 || rooms <= 0) throw new HuntException("Price, size and rooms must be
positive");
        Property p = new Property(seq.incrementAndGet(), owner, title, location.trim().toLowerCase(),
                lower(nearby), price, type, sizeSqft, rooms);
        properties.put(p.id(), p);
        return p;
    }

    /** Only AVAILABLE properties. Exact location matches first, nearby matches last, then the chosen sort. */
    public List<Property> search(String user, SearchCriteria c) {
        requireUser(user);
        Set<String> locs = c.locations() == null ? null : lower(c.locations());
        List<Predicate<Property>> filters = new ArrayList<>();
        filters.add(p -> p.status() == PropertyStatus.AVAILABLE);
        if (locs != null) filters.add(p -> locationRank(p, locs) < 2);
        if (c.minPrice() != null) filters.add(p -> p.price() >= c.minPrice());
        if (c.maxPrice() != null) filters.add(p -> p.price() <= c.maxPrice());
        if (c.type() != null) filters.add(p -> p.type() == c.type());
        if (c.minSize() != null) filters.add(p -> p.sizeSqft() >= c.minSize());
        if (c.maxSize() != null) filters.add(p -> p.sizeSqft() <= c.maxSize());
        if (c.rooms() != null) filters.add(p -> c.rooms().contains(p.rooms()));
        Predicate<Property> all = filters.stream().reduce(x -> true, Predicate::and);

        Comparator<Property> order = locs == null ? c.sortBy().comparator
                : Comparator.<Property>comparingInt(p -> locationRank(p,
locs)).thenComparing(c.sortBy().comparator);
        return properties.values().stream().filter(all).sorted(order).toList();
    }

    public void shortlist(String user, int propertyId) {
        requireUser(user);
        Property p = get(propertyId);
        if (p.ownerId().equals(user)) throw new HuntException("You cannot shortlist your own property");
        if (p.status() == PropertyStatus.SOLD) throw new HuntException("Property " + propertyId + " is already
sold");
        if (!shortlists.get(user).add(propertyId)) throw new HuntException("Property " + propertyId + " already
shortlisted");
    }

    /** Includes sold ones; the caller prints status so the buyer sees "SOLD". */
    public List<Property> shortlisted(String user) {
```

```
        requireUser(user);
        return
shortlists.get(user).stream().map(properties::get).sorted(Comparator.comparingInt(Property::id)).toList();
    }

    public List<Property> listedBy(String user) {
        requireUser(user);
        return properties.values().stream().filter(p -> p.ownerId().equals(user))
                .sorted(Comparator.comparingInt(Property::id)).toList();
    }

    public List<Property> soldBy(String user) {
        return listedBy(user).stream().filter(p -> p.status() == PropertyStatus.SOLD).toList();
    }

    public void markSold(String user, int propertyId) {
        requireUser(user);
        Property p = get(propertyId);
        if (!p.ownerId().equals(user)) throw new HuntException("Only the owner can mark property " + propertyId + "
sold");
        p.markSold();
    }

    /** 0 = exact location match, 1 = nearby match, 2 = no match. */
    private static int locationRank(Property p, Set<String> locs) {
        if (locs.contains(p.location())) return 0;
        return p.nearby().stream().anyMatch(locs::contains) ? 1 : 2;
    }

    private Property get(int id) {
        Property p = properties.get(id);
        if (p == null) throw new HuntException("Property does not exist: " + id);
        return p;
    }

    private void requireUser(String u) { if (!shortlists.containsKey(u)) throw new HuntException("User not
registered: " + u); }

    private static Set<String> lower(Set<String> s) {
        Set<String> out = new LinkedHashSet<>();
        s.forEach(x -> out.add(x.trim().toLowerCase()));
        return out;
    }
}
```

```java
// File: lld/ch21/propertyhunt/CommandProcessor.java
package lld.ch21.propertyhunt;

import java.util.Arrays;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

/**
 * Parses '|'-separated commands. Inside a field, ',' separates multiple values (bonus requirement).
 * "null" or an empty field means "not given".
 */
public class CommandProcessor {
    private final PropertyService svc;
    public CommandProcessor(PropertyService svc) { this.svc = svc; }

    public void execute(String line) {
        System.out.println("-> " + line);
        String[] f = Arrays.stream(line.split("\\|", -1)).map(String::trim).toArray(String[]::new);
        try {
            switch (f[0].toLowerCase()) {
                case "registeruser" -> { svc.registerUser(f[1]); out("User registered successfully."); }
                case "listproperty" -> {
                    Set<String> nearby = f.length > 8 && given(f[8]) ? csv(f[8]) : Set.of();
                    Property p = svc.list(f[1], f[2], f[3], nearby, Price.parse(f[4]), ListingType.parse(f[5]),
                            AreaUnit.parseSize(f[6], AreaUnit.SQFT), rooms(f[7]));
                    out("Property listed successfully. id=" + p.id());
                }
                case "search" -> printTable(svc.search(f[1], criteria(f)), false);
                case "shortlist" -> { svc.shortlist(f[1], Integer.parseInt(f[2])); out("Shortlisted property"); }
                case "viewshortlisted" -> printTable(svc.shortlisted(f[1]), true);
                case "viewlisted" -> printTable(svc.listedBy(f[1]), true);
                case "viewsold" -> printTable(svc.soldBy(f[1]), true);
                case "marksold" -> { svc.markSold(f[1], Integer.parseInt(f[2])); out("Property marked SOLD"); }
                default -> out("ERROR: unknown command " + f[0]);
            }
        } catch (HuntException | IllegalArgumentException | ArrayIndexOutOfBoundsException e) {
            out("ERROR: " + e.getMessage());
        }
    }

    /** Search|user|locations|priceRange|type|sizeRange|rooms|sortBy */
    private SearchCriteria criteria(String[] f) {
        SearchCriteria.Builder b = SearchCriteria.builder();
        if (given(f[2])) b.locations(csv(f[2]));
        if (given(f[3])) {
            String[] r = f[3].split("-");                       // "X-Y" inclusive, "X" = at most X
            if (r.length == 2) b.price(Price.parse(r[0]), Price.parse(r[1]));
            else b.price(null, Price.parse(r[0]));
        }
        if (given(f[4])) b.type(ListingType.parse(f[4]));
        if (given(f[5])) {
            String[] r = f[5].split("-");                       // "X-Y" inclusive, "X" = at least X
            if (r.length == 2) {
                AreaUnit unit = unitOf(r[1]);                   // "1000-2000 sqft": unit written once
                b.size(AreaUnit.parseSize(r[0], unit), AreaUnit.parseSize(r[1], unit));
            } else b.size(AreaUnit.parseSize(r[0], AreaUnit.SQFT), null);
        }
        if (given(f[6])) b.rooms(csv(f[6]).stream().map(CommandProcessor::rooms).collect(Collectors.toSet()));
        if (f.length > 7 && given(f[7])) b.sortBy(SortBy.parse(f[7]));
        return b.build();
    }

    private static AreaUnit unitOf(String s) {
        String unit = s.replaceAll("[0-9.\\s]", "");
        return unit.isEmpty() ? AreaUnit.SQFT : AreaUnit.parse(unit);
    }
```

```
    private static int rooms(String s) { return Integer.parseInt(s.toUpperCase().replace("BHK", "").trim()); }
    private static boolean given(String s) { return !s.isEmpty() && !s.equalsIgnoreCase("null"); }
    private static Set<String> csv(String s) {
        return Arrays.stream(s.split(",")).map(String::trim).filter(x -> !x.isEmpty()).collect(Collectors.toSet());
    }

    private static void printTable(List<Property> ps, boolean withStatus) {
        if (ps.isEmpty()) { out("<- (no properties)"); return; }
        out("<- %-3s %-20s %-10s %-6s %-9s %-5s %-5s%s".formatted("Id", "Title", "Location", "Price", "Size",
                "Rooms", "For", withStatus ? " Status" : ""));
        for (Property p : ps)
            out("<- %-3d %-20s %-10s %-6s %-9s %-5s %-5s%s".formatted(p.id(), p.title(), p.location(),
                    Price.format(p.price()), Math.round(p.sizeSqft()) + "sqft", p.rooms() + "BHK",
                    p.type() == ListingType.SELL ? "Sale" : "Rent", withStatus ? " " + p.status() : ""));
    }

    private static void out(String s) { System.out.println(s); }
}
```

```
// File: lld/ch21/propertyhunt/Main.java
package lld.ch21.propertyhunt;

import java.util.List;

public class Main {
    static final List<String> SAMPLE_INPUT = List.of(
            "RegisterUser|Jack",
            "RegisterUser|Jill",
            "RegisterUser|Jack",
            "ListProperty|Jack|A 3BHK for sale|bellandur|90L|sell|1800 sqft|3",
            "ListProperty|Jack|A 2BHK for rent|sarjapur|40k|rent|1500 sq
ft|2|bellandur,kadubeesanahalli,marathahalli",
            "ListProperty|Jill|Cosy 1BHK|HSR|25k|rent|70 sqm|1BHK",
            "ListProperty|Jill|Villa|whitefield|2.5Cr|sell|400 sqyd|4",
            "Search|Jack|null|null|null|null|null|null",
            "Search|Jack|bellandur|30000-50000|rent|150sqft|2|price",
            "Search|Jack|bellandur|null|null|null|null|price",
            "Search|Jill|null|1Cr|null|1000-2000 sqft|2,3|size",
            "Search|Jill|hsr,sarjapur|null|rent|null|1BHK,2BHK|price",
            "Shortlist|Jill|1",
            "Shortlist|Jill|2",
            "Shortlist|Jill|9",
            "Shortlist|Jack|1",
            "MarkSold|Jill|1",
            "MarkSold|Jack|1",
            "ViewShortlisted|Jill",
            "ViewListed|Jack",
            "ViewSold|Jack",
            "Search|Jill|null|null|sell|null|null|price",
            "ListProperty|Jill|Bad|x|abc|sell|100 sqft|1",
            "Search|Bob|null|null|null|null|null|null");

    public static void main(String[] args) {
        CommandProcessor cli = new CommandProcessor(new PropertyService());
        SAMPLE_INPUT.forEach(cli::execute);
    }
}
```

Output:

```
-> RegisterUser|Jack
User registered successfully.
-> RegisterUser|Jill
User registered successfully.
-> RegisterUser|Jack
ERROR: User already registered: Jack
-> ListProperty|Jack|A 3BHK for sale|bellandur|90L|sell|1800 sqft|3
Property listed successfully. id=1
-> ListProperty|Jack|A 2BHK for rent|sarjapur|40k|rent|1500 sq ft|2|bellandur,kadubeesanahalli,marathahalli
Property listed successfully. id=2
-> ListProperty|Jill|Cosy 1BHK|HSR|25k|rent|70 sqm|1BHK
Property listed successfully. id=3
-> ListProperty|Jill|Villa|whitefield|2.5Cr|sell|400 sqyd|4
Property listed successfully. id=4
-> Search|Jack|null|null|null|null|null|null
<- Id  Title               Location   Price  Size      Rooms For
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
<- 3   Cosy 1BHK           hsr        25k    753sqft   1BHK  Rent
<- 4   Villa               whitefield 2.5Cr  3600sqft  4BHK  Sale
-> Search|Jack|bellandur|30000-50000|rent|150sqft|2|price
<- Id  Title               Location   Price  Size      Rooms For
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
-> Search|Jack|bellandur|null|null|null|null|price
<- Id  Title               Location   Price  Size      Rooms For
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
-> Search|Jill|null|1Cr|null|1000-2000 sqft|2,3|size
<- Id  Title               Location   Price  Size      Rooms For
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale
-> Search|Jill|hsr,sarjapur|null|rent|null|1BHK,2BHK|price
<- Id  Title               Location   Price  Size      Rooms For
<- 3   Cosy 1BHK           hsr        25k    753sqft   1BHK  Rent
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent
-> Shortlist|Jill|1
Shortlisted property
-> Shortlist|Jill|2
Shortlisted property
-> Shortlist|Jill|9
ERROR: Property does not exist: 9
-> Shortlist|Jack|1
ERROR: You cannot shortlist your own property
-> MarkSold|Jill|1
ERROR: Only the owner can mark property 1 sold
-> MarkSold|Jack|1
Property marked SOLD
-> ViewShortlisted|Jill
<- Id  Title               Location   Price  Size      Rooms For   Status
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale  SOLD
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent  AVAILABLE
-> ViewListed|Jack
<- Id  Title               Location   Price  Size      Rooms For   Status
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale  SOLD
<- 2   A 2BHK for rent     sarjapur   40k    1500sqft  2BHK  Rent  AVAILABLE
-> ViewSold|Jack
<- Id  Title               Location   Price  Size      Rooms For   Status
<- 1   A 3BHK for sale     bellandur  90L    1800sqft  3BHK  Sale  SOLD
-> Search|Jill|null|null|sell|null|null|price
<- Id  Title               Location   Price  Size      Rooms For
<- 4   Villa               whitefield 2.5Cr  3600sqft  4BHK  Sale
-> ListProperty|Jill|Bad|x|abc|sell|100 sqft|1
ERROR: Bad price: abc
-> Search|Bob|null|null|null|null|null|null
ERROR: User not registered: Bob
```

Things to notice: 70 sq m was stored as 753 sq ft and 400 sq yd as 3600 sq ft; the `bellandur` search returns the bellandur listing first and the sarjapur listing (nearby) last; after `MarkSold`, property 1 disappears from searches but still shows as SOLD in Jill's shortlist; bad input produces an error line and the program continues.

Concurrency & edge cases

- **Storage:** `ConcurrentHashMap`s; each user's shortlist is a concurrent set; `Property.markSold` is `synchronized` so two "mark sold" calls cannot both succeed.
- **A search racing a sale** may still show a property that is being sold at that instant — acceptable for listings (eventual consistency: different readers may briefly see different values, but they all catch up soon); a purchase flow would re-check status under a lock.
- **Edge cases handled:** duplicate registration, unregistered user, unknown property, own-property shortlist, duplicate shortlist, shortlisting a sold property, non-owner mark sold, double sale, bad price/size/unit/type, non-positive values, missing fields.

Extensions & follow-up questions

- **Descending sort / multiple sort keys?** `SortBy` holds a comparator; add a direction flag and `.reversed()`, or accept a list of keys and chain `thenComparing`.
- **Fuzzy location ("Bellandur Road")?** Normalise locations to ids through a `LocationResolver` interface (gazetteer / geocoding — a directory of place names / turning an address into map coordinates).
- **"Within 5 km"?** Store latitude/longitude and filter by haversine distance (the standard formula for the distance between two points on a sphere); at scale use a geo-index (a lookup structure for finding things by location: geohash, R-tree) or a search engine with geo queries.
- **Fast search over millions?** Secondary indexes (by location, type, rooms) or Elasticsearch/OpenSearch; price and size as numeric range fields.
- **Notifications on price drop for shortlisted items?** Observer on `Property` price changes → notify users who shortlisted it.

Real-world example

Property portals like 99acres, MagicBricks and NoBroker let you filter by locality, budget, BHK count, buy/rent and area, and often suggest nearby localities when results are few; Indian listings commonly quote prices in lakhs and crores — exactly why the statement asks you to parse `60L` and `2Cr`.

In the interview

Start with the model and units (show `Price.parse` handling `90L`, and `AreaUnit`). Then implement register → list → search with *one* filter → add filters as predicates → sort. Shortlist, mark sold and the views are small after that. Call out the Builder and predicate list as your extensibility story, and the "normalise at the boundary" principle.

Self-check

- **Q:** How does a buyer searching in sq m match a seller who listed in sq yd? → **A:** Both are converted to sq ft at parse time; search compares sq ft.
- **Q:** Why store shortlists as ids rather than objects? → **A:** Status is read live from the property, so SOLD shows without extra updates.
- **Q:** What does a single number mean in the price field? → **A:** "At most X" (for size it means "at least X").
- **Q:** How do you add a "furnished" filter? → **A:** A field on `Property`, an optional field in `SearchCriteria`, and one predicate line in `search`.

TL;DR

Normalise price and size at the edges, turn every optional criterion into a predicate and AND them together, sort by (location rank, chosen key, id), and keep shortlists as ids so status stays live.

21.6 Design a Loan Ledger — "Ledger Co" (Navi)

Problem & clarifying questions

This problem (kumaransg/LLD, `ledger_company_navi`) is the Geektrust "Ledger Co" backend (server-side logic, no screens) challenge; the repository files it under Navi, which has used it as a hiring exercise. Banks lend money to borrowers; borrowers repay in **EMIs** (Equated Monthly Instalments — the same amount every month) and may also pay a **lump sum** (a one-off extra payment) at any point. The program reads commands and answers balance questions:

Command	Meaning
`LOAN BANK BORROWER P N R`	Borrower takes principal `P` (the amount borrowed) for `N` years at `R`% simple interest (interest charged only on the original principal, never on interest) per year.
`PAYMENT BANK BORROWER LUMP_SUM EMI_NO`	A lump sum paid right after EMI number `EMI_NO`.
`BALANCE BANK BORROWER EMI_NO`	Print `BANK BORROWER AMOUNT_PAID EMIS_LEFT` as of (after) EMI number `EMI_NO`.

Rules from the statement: interest `I = P × N × R / 100` (R is a percentage); total repayable `A = P + I`; monthly EMI = `A / (N × 12)` **rounded up** to a whole number (86.67 → 87, and even 100.10 → 101); amount paid as of EMI *k* = `k × EMI` + every lump sum whose EMI number is *k* or less (a lump sum "after EMI 10" *is* included in `BALANCE ... 10`); EMIs left = remaining amount ÷ EMI, **rounded up** (23.1 → 24); and if the last EMI is more than what is still owed, it shrinks to exactly that amount. A lump sum therefore reduces the *number* of EMIs, never the EMI amount. Only `BALANCE` produces output, in the form `BANK_NAME BORROWER_NAME AMOUNT_PAID NO_OF_EMIS_LEFT`.

Question	Answer used
Can amount paid exceed `A`?	No — capped at `A` (then EMIs left = 0). This is also the "last EMI shrinks" rule: the final payment is only what is left.
Is interest a whole number?	The statement defines rounding only for the EMI and the EMI count. We *assume* interest is also rounded up to a whole unit (say so to the interviewer); every official sample has whole-number interest, so both samples come out the same either way.
One loan per bank-borrower pair?	Yes — the pair identifies the loan; a second `LOAN` for the same pair is an error.
`EMI_NO` beyond the tenure (the loan's total length)?	Error. `EMI_NO = 0` is valid (before any EMI).

Requirements

Functional: LOAN, PAYMENT, BALANCE exactly as above; reproduce the official sample outputs. **Non-functional:** input from a list/file; one bad line must not stop the rest; interest rule replaceable; exact arithmetic (`BigDecimal`).

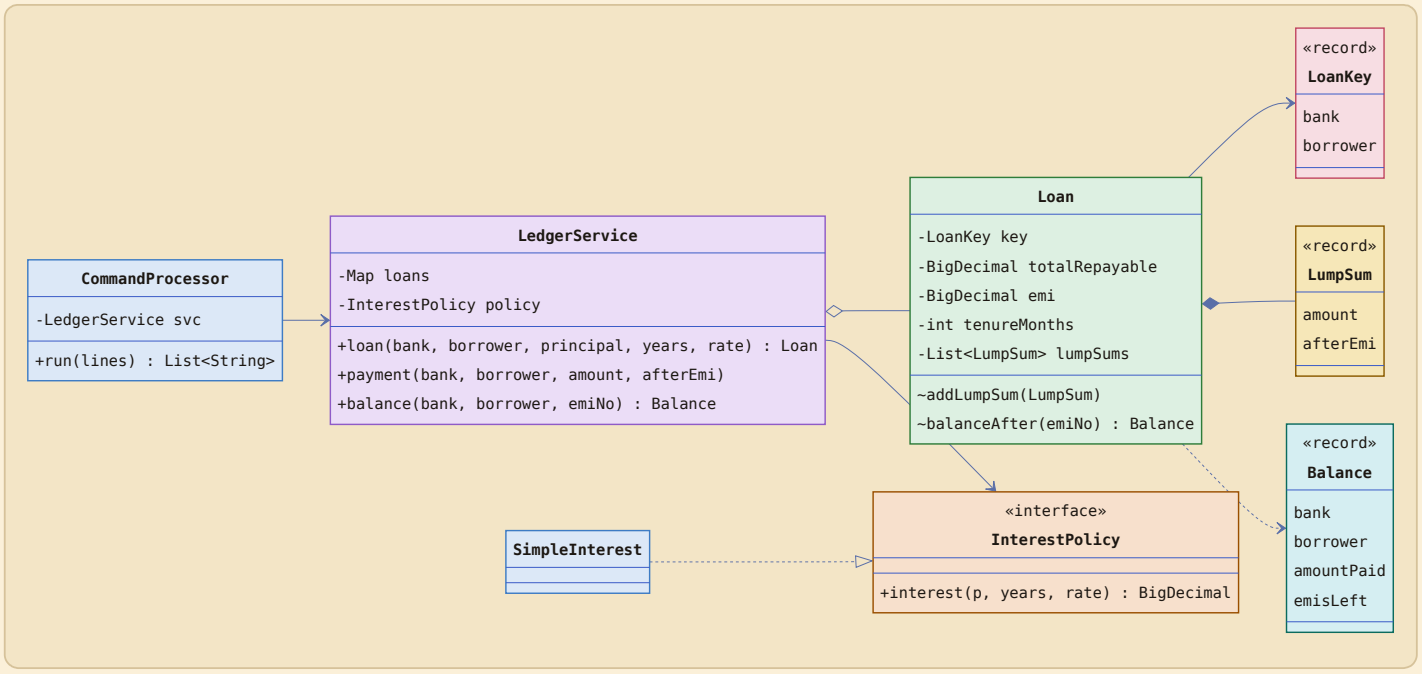
Use cases

Actor	Use cases
Bank	records a loan
Borrower	pays lump sums
Anyone (support agent, borrower app)	asks for the balance after EMI *k*

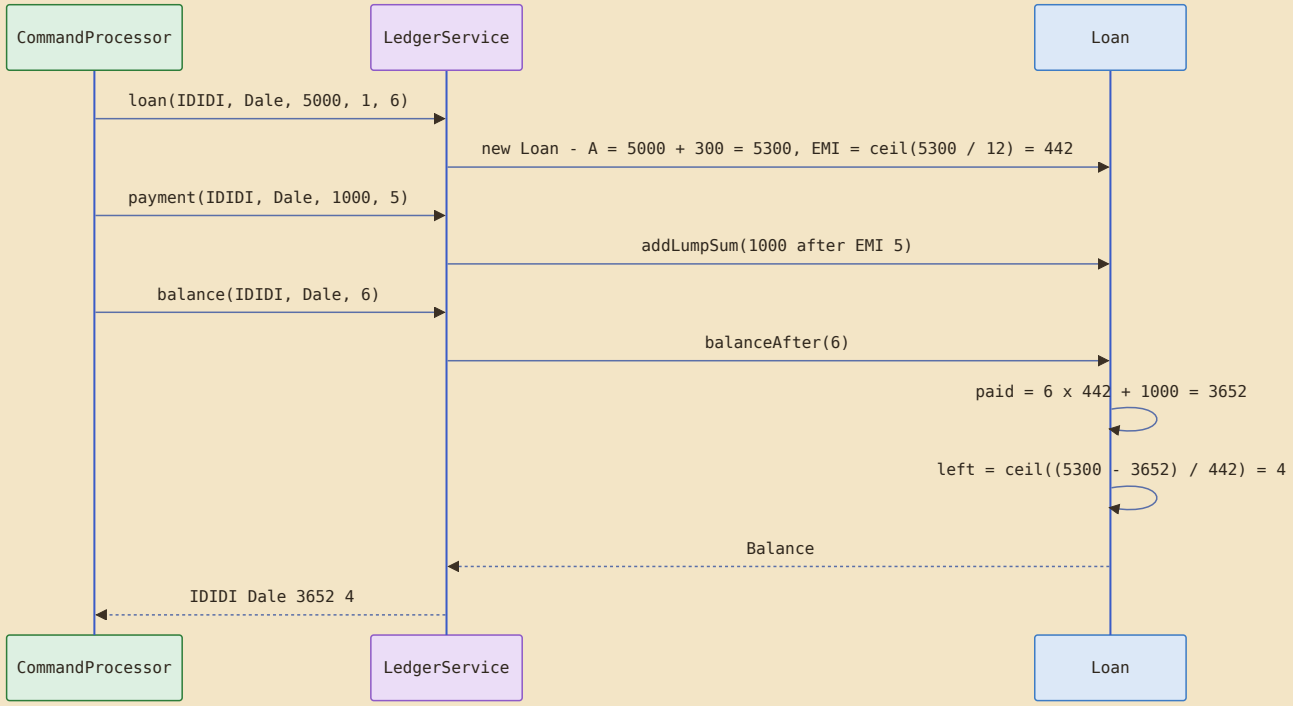
Core entities

Class	Kind	Responsibility
`LoanKey`	record	(bank, borrower) — the loan's identity; a perfect `Map` key because records implement `equals/hashCode` (the two methods a map uses to decide whether two keys are the same).
`Loan`	class	Computes `A` and EMI once; stores lump sums; derives a `Balance` for any EMI number.
`LumpSum`	record	amount + "after EMI number".
`Balance`	record	Output line: bank, borrower, amount paid, EMIs left.
`InterestPolicy` / `SimpleInterest`	interface / class	Strategy for total interest.
`LedgerService`	class	Store of loans; the three operations.
`CommandProcessor`	class	Parses lines, validates argument counts and numbers, collects output.
`LedgerException`	exception	Readable failure.

Class diagram



Key flows



Worked example (Sample 2, Shelly): P = 15000, N = 2, R = 9 → I = 2700, A = 17700, EMI = ⌈17700 / 24⌉ = ⌈737.5⌉ = 738. After 12 EMIs plus a 7000 lump sum: paid = 12 × 738 + 7000 = 15856; remaining = 1844; EMIs left = ⌈1844 / 738⌉ = ⌈2.499⌉ = 3. Output `UON Shelly 15856 3`.

Design decisions & patterns used

Decision	Why	Alternative rejected
Derive balance from facts (A, EMI, lump sums) on every query	Queries can ask about *any* EMI number, even one before a later payment. Storing a running balance would need replays.	Mutable `remaining` field updated per command — breaks when a BALANCE asks about an earlier EMI.
`InterestPolicy` Strategy	Simple interest today; reducing-balance or compound tomorrow without touching `Loan`.	Formula hard-coded in `Loan`.
`LoanKey` record as map key	Value semantics (two keys with the same fields count as equal) for free (`equals/hashCode`).	Concatenated string `"IDIDI#Dale"` — fragile.
`BigDecimal` + explicit `RoundingMode.CEILING`	The statement's rounding is "always up"; ceiling on integers is exact.	`Math.ceil` on doubles — fine for small numbers but floating error (tiny inaccuracies of binary floating-point numbers) can bite (e.g. a value like 4.000000001 ceils to 5).
Driver returns `List<String>`	Easy to compare with the expected output in a test.	Printing inside the service.

Java implementation

```java
// File: lld/ch21/ledger/LoanKey.java
package lld.ch21.ledger;

/** A borrower can hold one loan per bank; (bank, borrower) identifies it. */
public record LoanKey(String bank, String borrower) { }
```

```java
// File: lld/ch21/ledger/LumpSum.java
package lld.ch21.ledger;

import java.math.BigDecimal;

/** Extra payment made right after EMI number `afterEmi`. */
public record LumpSum(BigDecimal amount, int afterEmi) { }
```

```java
// File: lld/ch21/ledger/Balance.java
package lld.ch21.ledger;

import java.math.BigDecimal;

public record Balance(String bank, String borrower, BigDecimal amountPaid, int emisLeft) {
    @Override public String toString() { return bank + " " + borrower + " " + amountPaid.toPlainString() + " " +
emisLeft; }
}
```

```java
// File: lld/ch21/ledger/InterestPolicy.java
package lld.ch21.ledger;

import java.math.BigDecimal;

/** Strategy for total interest. The problem uses simple interest; a bank could plug in another rule. */
public interface InterestPolicy {
    BigDecimal interest(BigDecimal principal, BigDecimal years, BigDecimal ratePercent);
}
```

```java
// File: lld/ch21/ledger/SimpleInterest.java
package lld.ch21.ledger;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** I = P * N * R / 100, rounded UP to a whole unit (the statement rounds amounts up). */
public class SimpleInterest implements InterestPolicy {
    @Override public BigDecimal interest(BigDecimal p, BigDecimal n, BigDecimal r) {
        return p.multiply(n).multiply(r).divide(BigDecimal.valueOf(100), 0, RoundingMode.CEILING);
    }
}
```

```java
// File: lld/ch21/ledger/Loan.java
package lld.ch21.ledger;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.List;

/**
 * Loan with fixed EMI. Amount repaid A = P + I. EMI = ceil(A / (N*12)).
 * Lump sums shorten the loan (fewer EMIs) but never change the EMI amount.
 */
public class Loan {
    private final LoanKey key;
    private final BigDecimal totalRepayable;
    private final BigDecimal emi;
    private final int tenureMonths;
    private final List<LumpSum> lumpSums = new ArrayList<>();

    Loan(LoanKey key, BigDecimal principal, int years, BigDecimal rate, InterestPolicy policy) {
        if (principal.signum() <= 0 || years <= 0 || rate.signum() < 0)
            throw new LedgerException("Invalid loan terms for " + key);
        this.key = key;
        this.tenureMonths = years * 12;
        this.totalRepayable = principal.add(policy.interest(principal, BigDecimal.valueOf(years), rate));
        this.emi = totalRepayable.divide(BigDecimal.valueOf(tenureMonths), 0, RoundingMode.CEILING);
    }

    public BigDecimal emi() { return emi; }
    public BigDecimal totalRepayable() { return totalRepayable; }

    synchronized void addLumpSum(LumpSum l) {
        if (l.amount().signum() <= 0) throw new LedgerException("Lump sum must be positive");
        if (l.afterEmi() < 0 || l.afterEmi() > tenureMonths) throw new LedgerException("EMI number out of range: "
+ l.afterEmi());
        lumpSums.add(l);
    }

    /** Amount paid after `emiNo` EMIs (lump sums made at or before that EMI count), capped at A. */
    synchronized Balance balanceAfter(int emiNo) {
        if (emiNo < 0 || emiNo > tenureMonths) throw new LedgerException("EMI number out of range: " + emiNo);
        BigDecimal lumps = lumpSums.stream().filter(l -> l.afterEmi() <= emiNo)
                .map(LumpSum::amount).reduce(BigDecimal.ZERO, BigDecimal::add);
        BigDecimal paid = emi.multiply(BigDecimal.valueOf(emiNo)).add(lumps).min(totalRepayable);
        BigDecimal remaining = totalRepayable.subtract(paid);
        int left = remaining.divide(emi, 0, RoundingMode.CEILING).intValueExact();
        return new Balance(key.bank(), key.borrower(), paid, left);
    }
}
```

```java
// File: lld/ch21/ledger/LedgerException.java
package lld.ch21.ledger;

public class LedgerException extends RuntimeException {
    public LedgerException(String message) { super(message); }
}
```

```java
// File: lld/ch21/ledger/LedgerService.java
package lld.ch21.ledger;

import java.math.BigDecimal;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class LedgerService {
    private final Map<LoanKey, Loan> loans = new ConcurrentHashMap<>();
    private final InterestPolicy policy;

    public LedgerService(InterestPolicy policy) { this.policy = policy; }

    public Loan loan(String bank, String borrower, BigDecimal principal, int years, BigDecimal rate) {
        LoanKey key = new LoanKey(bank, borrower);
        Loan loan = new Loan(key, principal, years, rate, policy);
        if (loans.putIfAbsent(key, loan) != null) throw new LedgerException("Loan already exists for " + bank + " "
+ borrower);
        return loan;
    }

    public void payment(String bank, String borrower, BigDecimal amount, int afterEmi) {
        find(bank, borrower).addLumpSum(new LumpSum(amount, afterEmi));
    }

    public Balance balance(String bank, String borrower, int emiNo) { return find(bank,
borrower).balanceAfter(emiNo); }

    private Loan find(String bank, String borrower) {
        Loan l = loans.get(new LoanKey(bank, borrower));
        if (l == null) throw new LedgerException("No loan for " + bank + " " + borrower);
        return l;
    }
}
```

```java
// File: lld/ch21/ledger/CommandProcessor.java
package lld.ch21.ledger;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

/**
 * LOAN BANK BORROWER PRINCIPAL YEARS RATE
 * PAYMENT BANK BORROWER LUMP_SUM EMI_NO
 * BALANCE BANK BORROWER EMI_NO      -> prints "BANK BORROWER AMOUNT_PAID EMIS_LEFT"
 * Only BALANCE produces output (as in the original statement); errors are printed with "ERROR".
 */
public class CommandProcessor {
    private final LedgerService svc;
    public CommandProcessor(LedgerService svc) { this.svc = svc; }

    public List<String> run(List<String> lines) {
        List<String> output = new ArrayList<>();
        for (String line : lines) {
            if (line.isBlank()) continue;
            String[] t = line.trim().split("\\s+");
            try {
                switch (t[0]) {
                    case "LOAN" -> { need(t, 6); svc.loan(t[1], t[2], num(t[3]), Integer.parseInt(t[4]),
num(t[5])); }
                    case "PAYMENT" -> { need(t, 5); svc.payment(t[1], t[2], num(t[3]), Integer.parseInt(t[4])); }
                    case "BALANCE" -> { need(t, 4); output.add(svc.balance(t[1], t[2],
Integer.parseInt(t[3])).toString()); }
                    default -> throw new LedgerException("Unknown command " + t[0]);
                }
            } catch (LedgerException | NumberFormatException e) {
                output.add("ERROR " + e.getMessage() + "  [" + line + "]");
            }
        }
        return output;
    }

    private static BigDecimal num(String s) {
        try { return new BigDecimal(s); } catch (NumberFormatException e) { throw new LedgerException("Not a
number: " + s); }
    }

    private static void need(String[] t, int n) {
        if (t.length != n) throw new LedgerException(t[0] + " expects " + (n - 1) + " arguments");
    }
}
```

```java
// File: lld/ch21/ledger/Main.java
package lld.ch21.ledger;

import java.util.List;

public class Main {
    static final List<String> SAMPLE_1 = List.of(
            "LOAN IDIDI Dale 10000 5 4",
            "LOAN MBI Harry 2000 2 2",
            "BALANCE IDIDI Dale 5",
            "BALANCE IDIDI Dale 40",
            "BALANCE MBI Harry 12",
            "BALANCE MBI Harry 0");

    static final List<String> SAMPLE_2 = List.of(
            "LOAN IDIDI Dale 5000 1 6",
            "LOAN MBI Harry 10000 3 7",
            "LOAN UON Shelly 15000 2 9",
            "PAYMENT IDIDI Dale 1000 5",
            "PAYMENT MBI Harry 5000 10",
            "PAYMENT UON Shelly 7000 12",
            "BALANCE IDIDI Dale 3",
            "BALANCE IDIDI Dale 6",
            "BALANCE UON Shelly 12",
            "BALANCE MBI Harry 12");

    static final List<String> EDGE_CASES = List.of(
            "LOAN SBI Asha 1000 1 12",
            "PAYMENT SBI Asha 2000 1",           // lump sum bigger than the rest of the loan
            "BALANCE SBI Asha 2",
            "LOAN SBI Asha 5 1 1",               // duplicate loan
            "BALANCE HDFC Nobody 1",             // unknown loan
            "BALANCE SBI Asha 13",               // beyond tenure
            "PAYMENT SBI Asha ten 1",            // bad number
            "REFUND SBI Asha 1");                // unknown command

    public static void main(String[] args) {
        run("Sample 1", SAMPLE_1);
        run("Sample 2", SAMPLE_2);
        run("Edge cases", EDGE_CASES);
    }

    private static void run(String title, List<String> input) {
        System.out.println("== " + title);
        new CommandProcessor(new LedgerService(new SimpleInterest())).run(input).forEach(System.out::println);
    }
}
```

Output:

```
== Sample 1
IDIDI Dale 1000 55
IDIDI Dale 8000 20
MBI Harry 1044 12
MBI Harry 0 24
== Sample 2
IDIDI Dale 1326 9
IDIDI Dale 3652 4
UON Shelly 15856 3
MBI Harry 9044 10
== Edge cases
SBI Asha 1120 0
ERROR Loan already exists for SBI Asha  [LOAN SBI Asha 5 1 1]
ERROR No loan for HDFC Nobody  [BALANCE HDFC Nobody 1]
ERROR EMI number out of range: 13  [BALANCE SBI Asha 13]
ERROR Not a number: ten  [PAYMENT SBI Asha ten 1]
ERROR Unknown command REFUND  [REFUND SBI Asha 1]
```

The first two blocks are exactly the expected outputs of the two official Geektrust samples. In the edge-case block, Asha's 2000 lump sum after EMI 1 overpays a 1120 loan, so the amount paid is capped at 1120 and 0 EMIs remain.

Concurrency & edge cases

- **Concurrency:** loans live in a `ConcurrentHashMap` (`putIfAbsent` makes "one loan per pair" atomic); `Loan`'s lump-sum list is guarded by `synchronized` methods, so a `PAYMENT` and a `BALANCE` on the same loan do not interfere.
- **Edge cases handled:** duplicate loan, unknown loan, EMI number < 0 or > tenure, non-positive principal/years, negative rate, non-positive lump sum, overpayment (capped), wrong argument count, non-numeric values, unknown command, blank lines.

Extensions & follow-up questions

- **Reducing-balance (amortised) EMI like real banks?** (Reducing-balance: each month's interest is charged only on what is still owed; amortised: spread into equal payments that cover interest and principal.) New `InterestPolicy`/`EmiCalculator`: `EMI = P·r·(1+r)^n / ((1+r)^n − 1)` with monthly rate `r`; balance after k months uses the amortisation schedule. The `Loan` API stays the same.
- **Lump sum that should reduce the EMI instead of the tenure?** Make that a `PrepaymentPolicy` (REDUCE_TENURE / REDUCE_EMI) chosen per loan.
- **Late payments and penalties?** Record actual payment events with dates instead of assuming EMIs are paid on time; derive "EMIs paid" from them.
- **Read from a file?** `Files.readAllLines(Path.of(args[0]))` fed into the same `run` method — the driver's job, no core change.

Real-world example

Indian lenders such as Navi, banks and NBFCs (non-banking financial companies: lenders that are not licensed banks) show borrowers an EMI schedule and let them "part-prepay" a loan; most offer a choice between reducing the tenure (fewer EMIs, the Ledger Co rule) or reducing the EMI amount. Real loans use reducing-balance interest rather than the flat simple interest in this puzzle — a good point to raise when asked about extensions.

In the interview

Show the three formulas first and compute one sample by hand — it proves you understood the rounding. Then code `Loan.balanceAfter` (the heart), the service, and the command loop. Run both samples. Emphasise "state = facts, balances = derived", and the interest Strategy.

Self-check

- **Q:** Why ceil the EMI? → **A:** A floor would leave a shortfall at the end; rounding up guarantees N × 12 EMIs cover `A` (the final instalment may be smaller).
- **Q:** A `PAYMENT` after EMI 10 — does `BALANCE ... 5` include it? → **A:** No; only lump sums with `afterEmi ≤ 5` count.
- **Q:** Why is the balance computed rather than stored? → **A:** Queries may ask about any EMI number in any order; derivation is always consistent.

- **Q:** Where do you add compound interest (interest that is also charged on earlier interest)? → **A:** A new `InterestPolicy` implementation injected into `LedgerService`.

> **TL;DR**

EMI = ⌈(P + PNR/100) / 12N⌉. Store loan facts and lump sums; derive paid and EMIs-left on demand with ceiling rounding. Interest rule is a Strategy; the driver is a tiny command loop.

21.7 Design an Inventory Management System

Problem & clarifying questions

A very common machine-coding and OOD question (asked in e-commerce — online shopping — and quick-commerce — groceries delivered in minutes — interviews): track **products** (SKUs — stock keeping units, one id per sellable item) stored in several **warehouses**; record every **stock movement** (goods received, shipped, transferred between warehouses, adjusted after a physical count); fulfil orders by choosing warehouses; and raise **reorder alerts** when stock runs low.

Question	Answer used
Is stock tracked per warehouse?	Yes: (SKU, warehouse) → quantity.
Can an order be split across warehouses?	Yes; the choice of warehouses is a pluggable strategy (nearest city first, or most stock first).
Partial shipments if stock is short?	No — all-or-nothing.
When is a reorder alert raised?	When *total* stock of a SKU drops to or below its reorder point. Once per dip (no alert spam); re-armed when stock recovers.
How much to reorder?	Up to the product's max level (`max − current`).
Negative stock?	Never.

> **Requirements**

Functional

1. Add products (with reorder point and max level) and warehouses.
2. Receive stock (inbound) into a warehouse.
3. Ship an order quantity, allocating across warehouses by a strategy.
4. Transfer stock between warehouses.
5. Adjust stock after a physical count (cycle count).
6. Query stock per warehouse and total; list the movement history of a SKU.
7. Reorder alerts to subscribers.

Non-functional

- **No overselling** (selling more units than are in stock) under concurrent orders.
- Operations on different products run in parallel.
- Full audit trail (a complete record of every change; here every change is a movement).

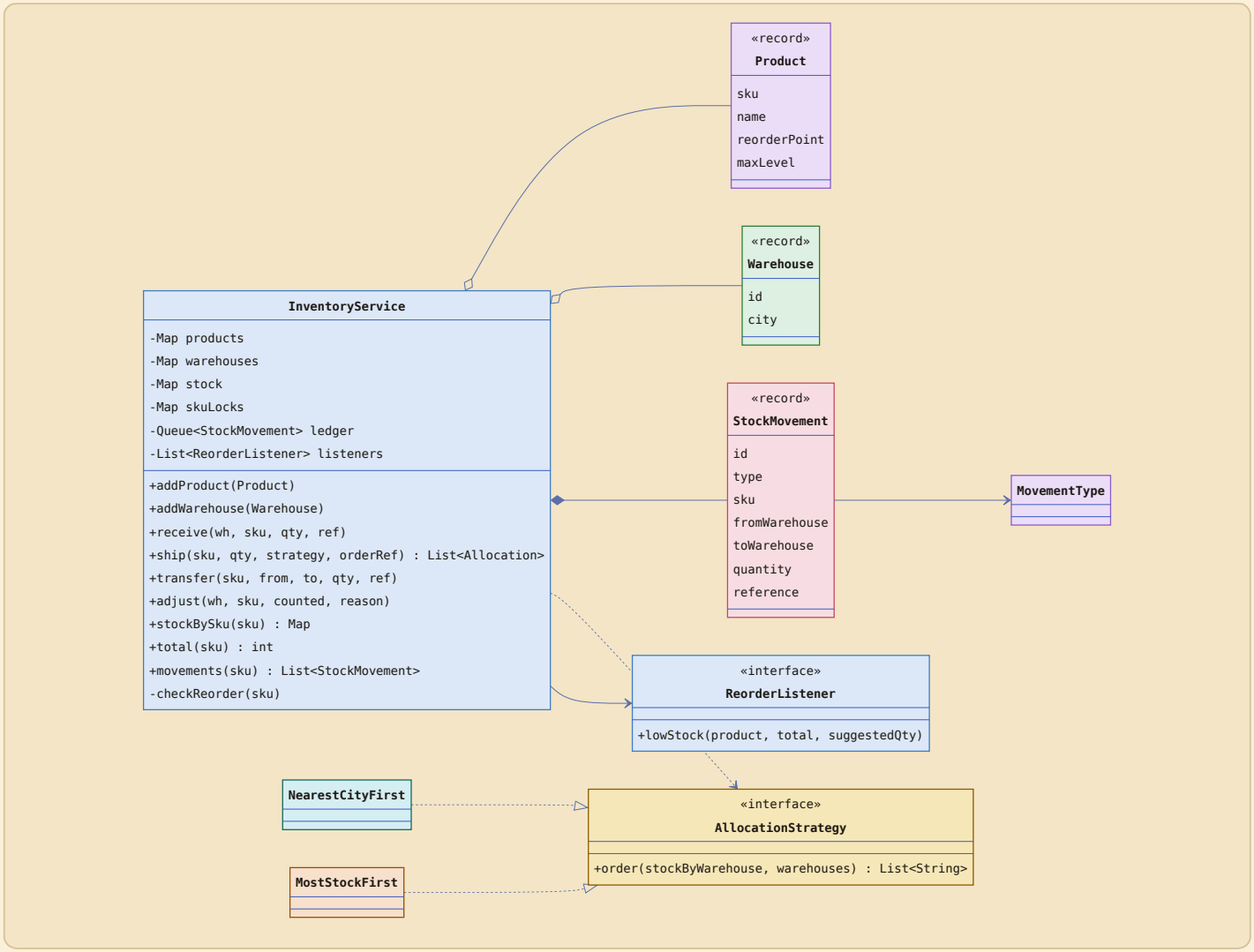
Use cases

Actor	Use cases
Warehouse staff	receive stock, transfer, cycle-count adjust
Order service (system)	ship / allocate order
Purchasing team (subscriber)	receives reorder alerts
Manager	views stock and movement history

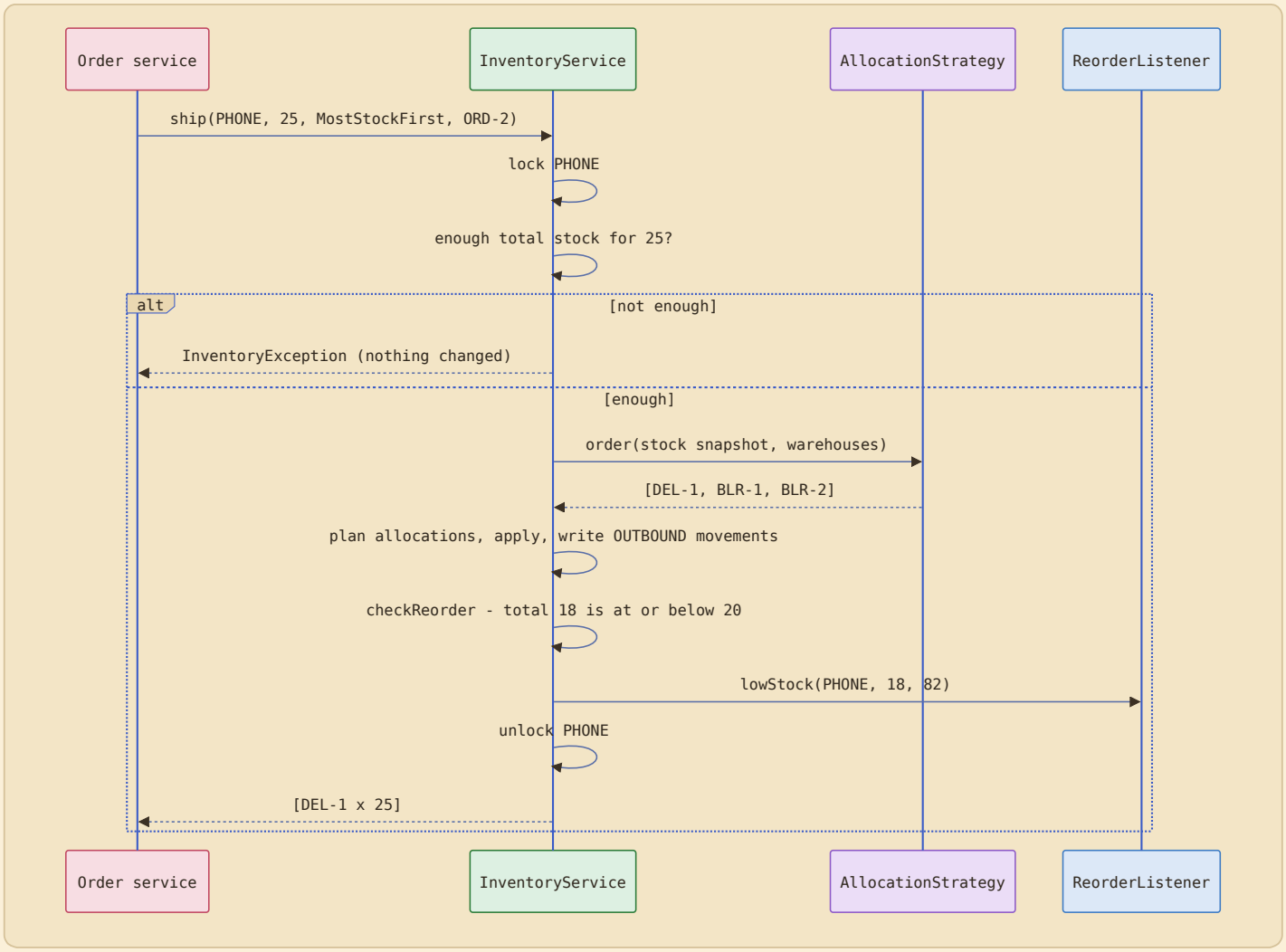
Core entities

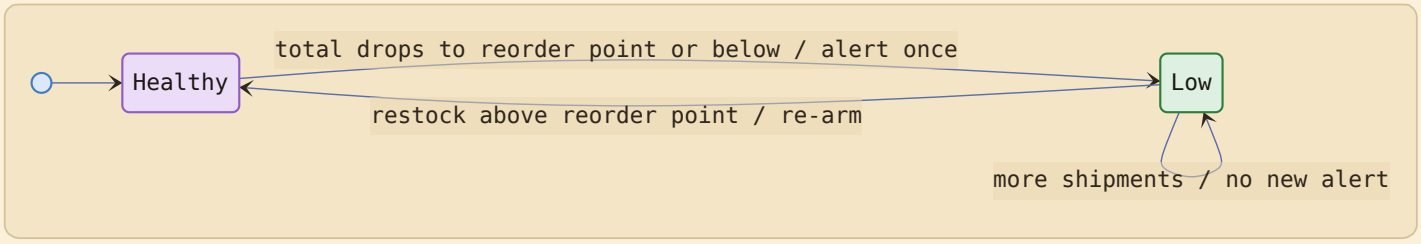
Class	Kind	Responsibility
`Product`	record	SKU, name, reorder point, max level (validated).
`Warehouse`	record	id, city.
`StockMovement`	record	Append-only ledger line (type, sku, from, to, qty, reference).
`MovementType`	enum	INBOUND, OUTBOUND, TRANSFER, ADJUSTMENT.
`Allocation`	record	"take q from warehouse w" — part of a shipment plan.
`AllocationStrategy` + `NearestCityFirst`, `MostStockFirst`	interface + strategies	Order in which warehouses are drained.
`ReorderListener`	interface	Observer for low-stock alerts.
`InventoryService`	class	Stock levels, per-SKU locking, movements, alerts.
`InventoryException`	exception	Readable failure.

Class diagram



Key flows


Order service
InventoryService
AllocationStrategy
ReorderListener
ship(PHONE, 25, MostStockFirst, ORD-2)
lock PHONE
enough total stock for 25?
alt
[not enough]
InventoryException (nothing changed)
[enough]
order(stock snapshot, warehouses)
[DEL-1, BLR-1, BLR-2]
plan allocations, apply, write OUTBOUND movements
checkReorder - total 18 is at or below 20
lowStock(PHONE, 18, 82)
unlock PHONE
[DEL-1 x 25]
Order service
InventoryService
AllocationStrategy
ReorderListener


Healthy
total drops to reorder point or below / alert once
restock above reorder point / re-arm
Low
more shipments / no new alert

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
One lock per SKU	Lock striping by key (a separate lock for each group of keys)	An order for one SKU touches several warehouses; a per-SKU lock covers all of them with no lock-ordering puzzle, while different SKUs never contend.	One lock per (SKU, warehouse) — needs lock ordering for multi-warehouse orders; one global lock — no parallelism.
Plan, then apply in `ship`	—	Check total first and build the full plan before changing anything → all-or-nothing.	Deduct warehouse by warehouse and roll back on shortage.
`AllocationStrategy`	Strategy	Business wants different policies (speed vs fewer split parcels); each is a small class.	`if (mode == ...)` in `ship`.
Append-only `StockMovement` **ledger**	Event log / audit	Every change is explainable; levels can be rebuilt by replaying; adjustments record the delta (the change, e.g. −3), not just the new number.	Updating quantities with no history.
`ReorderListener` **with once-per-dip**	Observer + edge-triggering (fire once when a value crosses a line)	Purchasing is notified once when stock crosses the threshold, not on every sale.	Level-triggered alerts (fire every time while the value is below the line) on every movement — spam.
Strategy gets a snapshot copy	Defensive copy (hand out a copy so the receiver cannot change your original)	A strategy cannot accidentally modify live stock.	Passing the live map.

Java implementation

```java
// File: lld/ch21/inventory/Product.java
package lld.ch21.inventory;

/** reorderPoint: raise an alert when total stock falls to this level or below. */
public record Product(String sku, String name, int reorderPoint, int maxLevel) {
    public Product {
        if (reorderPoint < 0 || maxLevel <= reorderPoint) throw new InventoryException("Need 0 <= reorderPoint < maxLevel");
    }
}
```

```java
// File: lld/ch21/inventory/Warehouse.java
package lld.ch21.inventory;

public record Warehouse(String id, String city) { }
```

```java
// File: lld/ch21/inventory/MovementType.java
package lld.ch21.inventory;

public enum MovementType { INBOUND, OUTBOUND, TRANSFER, ADJUSTMENT }
```

```java
// File: lld/ch21/inventory/StockMovement.java
package lld.ch21.inventory;

/** Append-only ledger line. Stock levels can always be rebuilt by replaying these. */
public record StockMovement(int id, MovementType type, String sku, String fromWarehouse, String toWarehouse,
                            int quantity, String reference) {
    @Override public String toString() {
        return "#%d %-10s %s qty=%+d %s->%s (%s)".formatted(id, type, sku, quantity,
                fromWarehouse == null ? "-" : fromWarehouse, toWarehouse == null ? "-" : toWarehouse, reference);
    }
}
```

```java
// File: lld/ch21/inventory/Allocation.java
package lld.ch21.inventory;

public record Allocation(String warehouseId, int quantity) { }
```

```java
// File: lld/ch21/inventory/AllocationStrategy.java
package lld.ch21.inventory;

import java.util.List;
import java.util.Map;

/** Strategy: in which order should warehouses be drained to fulfil an order? */
public interface AllocationStrategy {
    List<String> order(Map<String, Integer> stockByWarehouse, Map<String, Warehouse> warehouses);
}
```

```java
// File: lld/ch21/inventory/MostStockFirst.java
package lld.ch21.inventory;

import java.util.Comparator;
import java.util.List;
import java.util.Map;

/** Fewest split shipments: take from the fullest warehouse first. */
public class MostStockFirst implements AllocationStrategy {
    @Override public List<String> order(Map<String, Integer> stock, Map<String, Warehouse> warehouses) {
        return stock.keySet().stream()
                .sorted(Comparator.comparing((String w) -> stock.get(w)).reversed().thenComparing(w -> w))
                .toList();
    }
}
```

```java
// File: lld/ch21/inventory/NearestCityFirst.java
package lld.ch21.inventory;

import java.util.Comparator;
import java.util.List;
import java.util.Map;

/** Ship from warehouses in the customer's city first (faster delivery), then the rest by stock. */
public class NearestCityFirst implements AllocationStrategy {
    private final String city;
    public NearestCityFirst(String city) { this.city = city; }

    @Override public List<String> order(Map<String, Integer> stock, Map<String, Warehouse> warehouses) {
        return stock.keySet().stream()
                .sorted(Comparator.comparing((String w) -> !warehouses.get(w).city().equalsIgnoreCase(city))
                        .thenComparing(w -> -stock.get(w)).thenComparing(w -> w))
                .toList();
    }
}
```

```java
// File: lld/ch21/inventory/ReorderListener.java
package lld.ch21.inventory;

/** Observer: purchasing, email, dashboards... anyone who reacts to low stock. */
public interface ReorderListener {
    void lowStock(Product product, int totalOnHand, int suggestedOrderQty);
}
```

```java
// File: lld/ch21/inventory/InventoryException.java
package lld.ch21.inventory;

public class InventoryException extends RuntimeException {
    public InventoryException(String message) { super(message); }
}
```

```java
// File: lld/ch21/inventory/InventoryService.java
package lld.ch21.inventory;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Stock is kept per (sku, warehouse). Locking is per SKU: every operation on one product
 * (even across warehouses) is serialized, while different products proceed in parallel.
 */
public class InventoryService {
    private final Map<String, Product> products = new ConcurrentHashMap<>();
    private final Map<String, Warehouse> warehouses = new ConcurrentHashMap<>();
    private final Map<String, Map<String, Integer>> stock = new ConcurrentHashMap<>();   // sku -> (wh -> qty)
    private final Map<String, ReentrantLock> skuLocks = new ConcurrentHashMap<>();
    private final Set<String> alerted = ConcurrentHashMap.newKeySet();                   // SKUs already flagged
low
    private final Queue<StockMovement> ledger = new ConcurrentLinkedQueue<>();              // append-heavy, lock-
free
    private final List<ReorderListener> listeners = new CopyOnWriteArrayList<>();
    private final AtomicInteger movementSeq = new AtomicInteger();

    public void addListener(ReorderListener l) { listeners.add(l); }

    public void addProduct(Product p) {
        if (products.putIfAbsent(p.sku(), p) != null) throw new InventoryException("Duplicate SKU " + p.sku());
        stock.put(p.sku(), new LinkedHashMap<>());
        skuLocks.put(p.sku(), new ReentrantLock());
    }

    public void addWarehouse(Warehouse w) {
        if (warehouses.putIfAbsent(w.id(), w) != null) throw new InventoryException("Duplicate warehouse " +
w.id());
    }

    public void receive(String warehouseId, String sku, int qty, String ref) {
        positive(qty);
        warehouse(warehouseId);
        withSku(sku, () -> {
            change(sku, warehouseId, qty);
            record(MovementType.INBOUND, sku, null, warehouseId, qty, ref);
        });
    }

    /** All-or-nothing: either the whole quantity is allocated across warehouses, or nothing moves. */
    public List<Allocation> ship(String sku, int qty, AllocationStrategy strategy, String orderRef) {
        positive(qty);
        return withSku(sku, () -> {
            Map<String, Integer> levels = stock.get(sku);
            int total = levels.values().stream().mapToInt(Integer::intValue).sum();
            if (total < qty) throw new InventoryException("Only " + total + " of " + sku + " available, order needs
" + qty);
            List<Allocation> plan = new ArrayList<>();
            int remaining = qty;
            for (String wh : strategy.order(Map.copyOf(levels), warehouses)) {
                int take = Math.min(remaining, levels.get(wh));
                if (take > 0) { plan.add(new Allocation(wh, take)); remaining -= take; }
                if (remaining == 0) break;
            }
```

```
            for (Allocation a : plan) {
                change(sku, a.warehouseId(), -a.quantity());
                record(MovementType.OUTBOUND, sku, a.warehouseId(), null, -a.quantity(), orderRef);
            }
            return plan;
        });
    }

    public void transfer(String sku, String fromWh, String toWh, int qty, String ref) {
        positive(qty);
        warehouse(fromWh); warehouse(toWh);
        if (fromWh.equals(toWh)) throw new InventoryException("Source and target warehouse are the same");
        withSku(sku, () -> {
            int have = level(sku, fromWh);
            if (have < qty) throw new InventoryException(fromWh + " has only " + have + " of " + sku);
            change(sku, fromWh, -qty);
            change(sku, toWh, qty);
            record(MovementType.TRANSFER, sku, fromWh, toWh, qty, ref);
        });
    }

    /** Cycle count: set the level to what was physically counted and log the difference. */
    public void adjust(String warehouseId, String sku, int countedQty, String reason) {
        if (countedQty < 0) throw new InventoryException("Counted quantity cannot be negative");
        warehouse(warehouseId);
        withSku(sku, () -> {
            int delta = countedQty - level(sku, warehouseId);
            if (delta == 0) return;
            change(sku, warehouseId, delta);
            record(MovementType.ADJUSTMENT, sku, null, warehouseId, delta, reason);
        });
    }

    /** Sorted by warehouse id so output is stable. */
    public Map<String, Integer> stockBySku(String sku) {
        return withSku(sku, () -> java.util.Collections.unmodifiableMap(new java.util.TreeMap<>(stock.get(sku))));
    }

    public int total(String sku) { return withSku(sku, () -> totalUnlocked(sku)); }

    public List<StockMovement> movements(String sku) {
        return ledger.stream().filter(m -> m.sku().equals(sku)).toList();
    }

    // ------------------------------------------------------------ internals (called with the SKU lock held)

    private void change(String sku, String wh, int delta) {
        stock.get(sku).merge(wh, delta, Integer::sum);
    }

    private void record(MovementType type, String sku, String from, String to, int qty, String ref) {
        ledger.add(new StockMovement(movementSeq.incrementAndGet(), type, sku, from, to, qty, ref));
        checkReorder(sku);
    }

    /** Fires once when stock crosses down to the reorder point; re-arms when stock recovers. */
    private void checkReorder(String sku) {
        Product p = products.get(sku);
        int total = totalUnlocked(sku);
        if (total <= p.reorderPoint()) {
            if (alerted.add(sku)) listeners.forEach(l -> l.lowStock(p, total, p.maxLevel() - total));
        } else {
            alerted.remove(sku);
        }
    }

    private int totalUnlocked(String sku) { return
stock.get(sku).values().stream().mapToInt(Integer::intValue).sum(); }
    private int level(String sku, String wh) { return stock.get(sku).getOrDefault(wh, 0); }
```

```
    private <T> T withSku(String sku, java.util.function.Supplier<T> action) {
        ReentrantLock lock = skuLocks.get(sku);
        if (lock == null) throw new InventoryException("Unknown SKU " + sku);
        lock.lock();
        try { return action.get(); } finally { lock.unlock(); }
    }

    private void withSku(String sku, Runnable action) { withSku(sku, () -> { action.run(); return null; }); }

    private void warehouse(String id) { if (!warehouses.containsKey(id)) throw new InventoryException("Unknown
warehouse " + id); }
    private static void positive(int q) { if (q <= 0) throw new InventoryException("Quantity must be positive: " +
q); }
}
```

```java
// File: lld/ch21/inventory/Main.java
package lld.ch21.inventory;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws InterruptedException {
        InventoryService inv = new InventoryService();
        inv.addListener((p, total, suggest) ->
                System.out.println("  [ALERT] " + p.sku() + " low: " + total + " left, reorder " + suggest + " to
reach " + p.maxLevel()));

        inv.addWarehouse(new Warehouse("BLR-1", "Bengaluru"));
        inv.addWarehouse(new Warehouse("BLR-2", "Bengaluru"));
        inv.addWarehouse(new Warehouse("DEL-1", "Delhi"));
        inv.addProduct(new Product("PHONE", "Phone X", 20, 100));
        inv.addProduct(new Product("CASE", "Phone case", 10, 200));

        System.out.println("== receive stock");
        inv.receive("BLR-1", "PHONE", 30, "PO-1");
        inv.receive("DEL-1", "PHONE", 40, "PO-2");
        inv.receive("BLR-2", "PHONE", 5, "PO-3");
        System.out.println("  PHONE: " + inv.stockBySku("PHONE") + " total=" + inv.total("PHONE"));

        System.out.println("== ship 32 phones to a Bengaluru customer (nearest city first)");
        System.out.println("  " + inv.ship("PHONE", 32, new NearestCityFirst("Bengaluru"), "ORD-1"));
        System.out.println("  PHONE: " + inv.stockBySku("PHONE"));

        System.out.println("== transfer 15 phones DEL-1 -> BLR-1");
        inv.transfer("PHONE", "DEL-1", "BLR-1", 15, "TR-1");
        System.out.println("  PHONE: " + inv.stockBySku("PHONE"));

        System.out.println("== ship 25 more (most stock first) -> crosses reorder point");
        System.out.println("  " + inv.ship("PHONE", 25, new MostStockFirst(), "ORD-2"));
        System.out.println("  ship 1 more (no duplicate alert): " + inv.ship("PHONE", 1, new MostStockFirst(),
"ORD-3"));

        System.out.println("== cycle count finds 2 damaged units in BLR-1");
        inv.adjust("BLR-1", "PHONE", inv.stockBySku("PHONE").get("BLR-1") - 2, "damaged");
        System.out.println("  PHONE: " + inv.stockBySku("PHONE") + " total=" + inv.total("PHONE"));

        System.out.println("== restock re-arms the alert");
        inv.receive("BLR-1", "PHONE", 90, "PO-4");
        inv.ship("PHONE", 90, new MostStockFirst(), "ORD-4");

        System.out.println("== edge cases");
        attempt(() -> inv.ship("PHONE", 999, new MostStockFirst(), "ORD-X"));
        attempt(() -> inv.transfer("PHONE", "BLR-2", "BLR-2", 1, "TR-X"));
        attempt(() -> inv.receive("MUM-1", "PHONE", 5, "PO-X"));
        attempt(() -> inv.receive("BLR-1", "TABLET", 5, "PO-X"));
        attempt(() -> inv.receive("BLR-1", "PHONE", 0, "PO-X"));

        System.out.println("== movement ledger for PHONE");
        inv.movements("PHONE").forEach(m -> System.out.println("  " + m));

        System.out.println("== concurrency: 200 one-unit orders race for 50 cases");
        inv.receive("BLR-1", "CASE", 50, "PO-5");
        AtomicInteger ok = new AtomicInteger(), rejected = new AtomicInteger();
        ExecutorService pool = Executors.newFixedThreadPool(8);
        for (int i = 0; i < 200; i++) {
            pool.submit(() -> {
                try { inv.ship("CASE", 1, new MostStockFirst(), "web"); ok.incrementAndGet(); }
                catch (InventoryException e) { rejected.incrementAndGet(); }
            });
        }
```

```
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
        System.out.println("  shipped=" + ok + " rejected=" + rejected + " left=" + inv.total("CASE"));
    }

    private static void attempt(Runnable r) {
        try { r.run(); System.out.println("  ok"); }
        catch (InventoryException e) { System.out.println("  error: " + e.getMessage()); }
    }
}
```

Output:

```
== receive stock
  PHONE: {BLR-1=30, BLR-2=5, DEL-1=40} total=75
== ship 32 phones to a Bengaluru customer (nearest city first)
  [Allocation[warehouseId=BLR-1, quantity=30], Allocation[warehouseId=BLR-2, quantity=2]]
  PHONE: {BLR-1=0, BLR-2=3, DEL-1=40}
== transfer 15 phones DEL-1 -> BLR-1
  PHONE: {BLR-1=15, BLR-2=3, DEL-1=25}
== ship 25 more (most stock first) -> crosses reorder point
  [ALERT] PHONE low: 18 left, reorder 82 to reach 100
  [Allocation[warehouseId=DEL-1, quantity=25]]
  ship 1 more (no duplicate alert): [Allocation[warehouseId=BLR-1, quantity=1]]
== cycle count finds 2 damaged units in BLR-1
  PHONE: {BLR-1=12, BLR-2=3, DEL-1=0} total=15
== restock re-arms the alert
  [ALERT] PHONE low: 15 left, reorder 85 to reach 100
== edge cases
  error: Only 15 of PHONE available, order needs 999
  error: Source and target warehouse are the same
  error: Unknown warehouse MUM-1
  error: Unknown SKU TABLET
  error: Quantity must be positive: 0
== movement ledger for PHONE
  #1 INBOUND    PHONE qty=+30 -->BLR-1 (PO-1)
  #2 INBOUND    PHONE qty=+40 -->DEL-1 (PO-2)
  #3 INBOUND    PHONE qty=+5 -->BLR-2 (PO-3)
  #4 OUTBOUND   PHONE qty=-30 BLR-1->- (ORD-1)
  #5 OUTBOUND   PHONE qty=-2 BLR-2->- (ORD-1)
  #6 TRANSFER   PHONE qty=+15 DEL-1->BLR-1 (TR-1)
  #7 OUTBOUND   PHONE qty=-25 DEL-1->- (ORD-2)
  #8 OUTBOUND   PHONE qty=-1 BLR-1->- (ORD-3)
  #9 ADJUSTMENT PHONE qty=-2 -->BLR-1 (damaged)
  #10 INBOUND    PHONE qty=+90 -->BLR-1 (PO-4)
  #11 OUTBOUND   PHONE qty=-90 BLR-1->- (ORD-4)
== concurrency: 200 one-unit orders race for 50 cases
  [ALERT] CASE low: 10 left, reorder 190 to reach 200
  shipped=50 rejected=150 left=0
```

Reading the output: the Bengaluru order drained BLR-1 then BLR-2 before touching Delhi; the reorder alert fired once when total PHONE stock fell to 18 (≤ 20) and not again at 17; after a restock pushed the total above 20 the alert re-armed and fired again when stock fell back; the ledger explains every unit; and with 200 concurrent one-unit orders for 50 cases, exactly 50 shipped — no overselling.

Concurrency & edge cases

- **Overselling:** check-total → plan → apply runs under the SKU's `ReentrantLock`; concurrent orders for the same SKU are serialised.
- **Parallelism:** different SKUs use different locks.
- **Ledger:** a `ConcurrentLinkedQueue` (a thread-safe queue that needs no lock) — lock-free appends from many SKU threads at once, insertion order kept, and iteration never throws `ConcurrentModificationException` (the error Java raises when a collection changes while it is being looped over). (A `CopyOnWriteArrayList` would copy the whole ledger on every movement — right for rarely-changing lists, wrong for an append-heavy log.)

- **Listeners:** `CopyOnWriteArrayList` — read on every alert, changed almost never, which is exactly its use case. Listener calls happen while holding the SKU lock — fine for quick callbacks; for slow ones (email), hand off to an executor (a pool of threads that runs submitted tasks) so the lock is not held during I/O (input/output, such as network or disk calls).
- **Edge cases handled:** unknown SKU/warehouse, non-positive quantities, negative counted quantity, transfer to the same warehouse, transfer or order larger than stock, invalid product thresholds, duplicate SKU/warehouse, "no change" adjustments (not logged).

Extensions & follow-up questions

- **Reservations (cart holds)?** Track `onHand` and `reserved` per (SKU, warehouse); available = onHand − reserved; reservations expire after a timeout; shipping converts a reservation into an OUTBOUND.
- **Batches / expiry dates (groceries, medicines)?** Stock per batch with expiry; allocate FEFO (first-expiry-first-out) — just another `AllocationStrategy`.
- **Distances instead of cities?** `NearestFirst` sorting by a `DistanceService` result.
- **Automatic purchase orders?** A `ReorderListener` that creates a PO (purchase order: a formal request to a supplier) for `max − current` and marks it pending so a second alert does not create a duplicate.
- **Multiple services / database?** Stock rows with optimistic versioning (`UPDATE ... SET qty = qty - ? WHERE qty >= ? AND version = ?`), and the movement ledger as an event stream (e.g. Kafka, a system that stores and delivers an ordered log of events) consumed by analytics.

Real-world example

Amazon's fulfilment network and Flipkart's warehouses decide which fulfilment centre ships each order based on factors like proximity and stock, and may split an order across centres — the job of `AllocationStrategy`. Shopify lets merchants hold inventory at multiple locations and set a fulfilment priority order among them. Quick-commerce apps (Blinkit, Zepto, Swiggy Instamart) run many small "dark stores" (warehouses closed to the public that only serve online orders), where accurate per-store stock and fast reorder alerts are essential.

In the interview

Start with the data shape: `stock[sku][warehouse] = qty` plus an append-only movement list. Code receive and ship (with the plan-then-apply and the per-SKU lock) first; they carry most of the marks. Then transfer and adjust (both tiny), then the reorder observer with once-per-dip, then allocation strategies. Run the 200-thread demo to prove "no overselling".

Self-check

- **Q:** Why does `ship` check the total before building the plan? → **A:** To reject short orders before any warehouse is touched (all-or-nothing).
- **Q:** Why is a per-SKU lock enough for a multi-warehouse order? → **A:** An order involves exactly one SKU, and all of that SKU's warehouses are covered by its one lock.
- **Q:** How do you add FEFO? → **A:** Add batch-level stock and a new `AllocationStrategy`; `ship` does not change.
- **Q:** Why record adjustments as movements? → **A:** So shrinkage/damage is auditable and levels can be rebuilt from the ledger.

TL;DR

Stock per (SKU, warehouse), every change is a ledger movement, a lock per SKU prevents overselling, allocation is a Strategy, and low-stock alerts are an edge-triggered Observer.

21.8 Design a Unix File Search (`find`)

Problem & clarifying questions

The Unix (the operating-system family that includes Linux and macOS) `find` command walks a directory tree (folders inside folders) and prints every entry that passes a set of tests, for example `find /var/log -type f -size +10M -mtime +7` ("regular files under /var/log bigger than 10 MiB, last changed more than 7 days ago"; 1 MiB = 1,048,576 bytes). The machine-coding version asks you to build a small `find`: a file-system model, a set of **filters**, a way to **combine** them with AND / OR / NOT, and a clean API for the search options. It is a favourite

because the obvious solution (one method with ten optional parameters and a pile of `if`s) is exactly what the interviewer wants you to avoid.

Three words used below: a **predicate** (or *test*) is a yes/no question about one entry ("is its name `*.log`?"); a **glob** is a filename pattern with wildcards (`*` = any characters, `?` = one character, `[0-9]` = one character from a set); **depth** is how many levels below the start directory an entry sits (the start itself is depth 0).

Question	Answer used
Real disk or in-memory?	In-memory tree, so runs are identical; a `java.nio` (Java's file and I/O library) adapter (a class that wraps one API to fit another interface) is an extension.
Which tests?	Name glob (case-sensitive and insensitive), type (file / directory), size, modification age, owner, empty.
How are tests combined?	AND, OR, NOT and parentheses, with `find`'s precedence (which operator is applied first when there are no parentheses): NOT binds tightest, then AND, then OR. Two tests side by side mean AND.
Size semantics?	Like GNU `find` (the standard version on Linux, from the GNU project): the size is **rounded up** to the unit before comparing, so `-size -1M` matches only empty files. Size tests apply to files only.
Other options?	`mindepth`, `maxdepth`, a result limit, and **pruning** (skip whole directories such as `node_modules`, a JavaScript project's often huge folder of downloaded libraries).
API or command line?	Both: a fluent (written as one chain of method calls, `a().b().c()`) **Builder** API for code, and a parser for `find`-style command lines.
Output order?	Pre-order depth-first (go as deep as possible down one branch before the next; a directory is listed before its contents), children in name order.
Huge trees?	Stream results as they are found; no recursion (a method calling itself; deep trees must not overflow the stack — the limited memory that holds pending method calls); a parallel variant as an extension.

Requirements

Functional

1. Model files and directories (name, owner, modification time, size).
2. Search from a start path with any combination of tests joined by AND / OR / NOT.
3. Options: min/max depth, result limit, pruned directory names.
4. Parse `find` command lines into the same search criteria.
5. Clear errors for bad paths, unknown tests, malformed sizes and unbalanced parentheses.

Non-functional

- **Open/Closed**: a new test = one new class, no change to the search loop or the parser's structure.
- Memory: stream results; handle very deep trees.
- Thread safety: searching while files are being added must not throw.

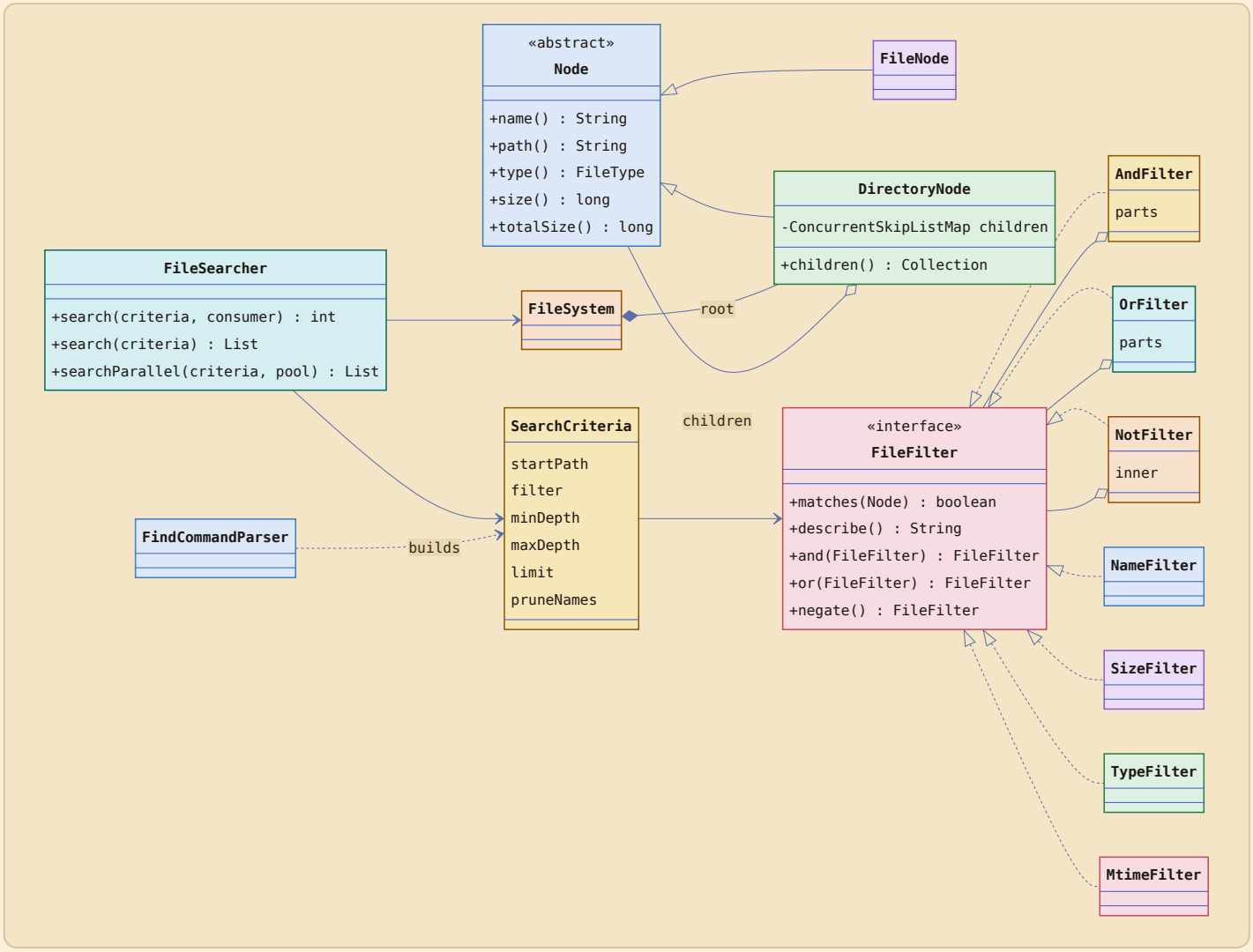
Use cases

Actor	Use cases
Developer (API user)	build criteria with the Builder, run a search, stream results
Shell user (someone typing commands into a terminal)	type a `find` command line, see matching paths or a clear error
Ops script (an operations team's automation script)	find big old logs (`-size +10M -mtime +7`) to archive, find empty directories to clean

Core entities

Class	Kind	Responsibility
`Node`	abstract sealed class (cannot be created directly, and only the listed subclasses may extend it)	**Composite** component: name, parent, owner, modified time, `path()`, `size()`, `totalSize()`.
`FileNode` / `DirectoryNode`	classes	Leaf and container; a directory keeps its children sorted in a `ConcurrentSkipListMap` (a sorted map that many threads can use at once).
`FileSystem`	class	Root, `mkdirs`, `createFile`, `resolve(path)`.
`FileFilter`	interface	**Specification**: `matches(node)`, `describe()`, and default `and` / `or` / `negate`.
`AndFilter`, `OrFilter`, `NotFilter`	records	**Composite** filters: a tree of filters that is itself a filter.
`Filters`	factory	Leaf tests: name/iname glob, type, size, mtime, owner, empty, match-all.
`SearchCriteria` + `Builder`	class	Immutable search description (start, filter, depths, limit, prune set), built fluently and validated once.
`FileSearcher`	class	Iterative DFS (depth-first search) with an explicit stack (a last-in, first-out list); streams results to a `Consumer`; optional fork/join version (Java's framework that splits a task into subtasks run in parallel, then joins the results).
`FindCommandParser`	class	Recursive-descent parser (one method per grammar rule, the methods calling each other) for `find` expressions → `SearchCriteria`.
`SearchException`	exception	Readable failure.

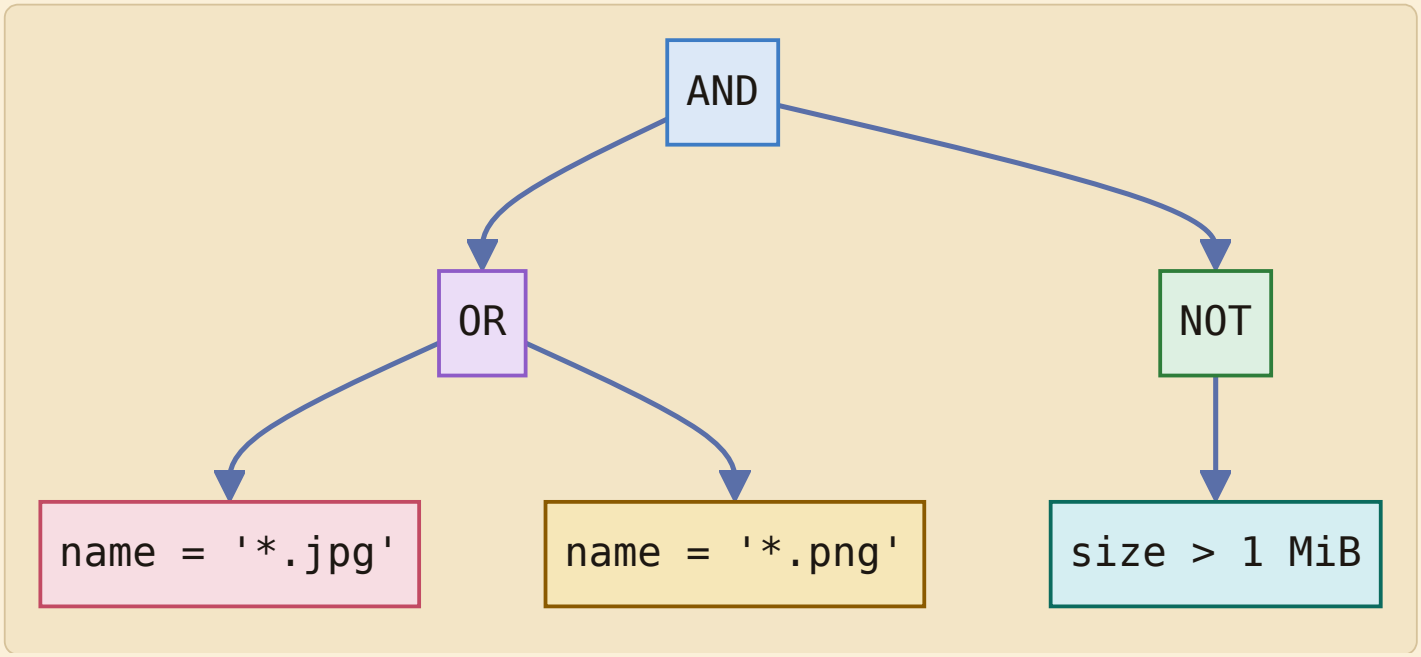
Class diagram



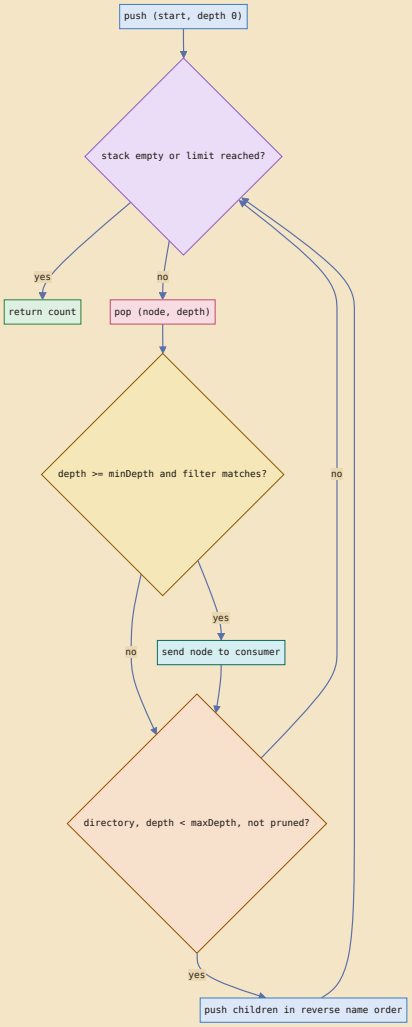
Two Composites sit side by side: the **file tree** (a directory is a node that contains nodes) and the **filter tree** (an AND is a filter that contains filters). The searcher walks the first and asks the second one question per entry.

Key flows

Parsing `find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M` produces this filter tree:



The search loop (explicit stack, pre-order):



Operator precedence (from lowest to highest): `-o` (OR) < `-a` or nothing (AND) < `!` (NOT) < `( … )`. So `-name a -o -name b -type f` means `a OR (b AND file)` — a classic surprise, and the reason `find` users write parentheses.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each test is a small class with `matches(node)`	**Specification**	A new test is one record; the search loop never changes (Open/Closed)	`search(name, ext, minSize, maxSize, type, …)` with a growing list of `if`s
`AndFilter` / `OrFilter` / `NotFilter` hold other filters	**Composite**	Arbitrary nesting (`(a OR b) AND NOT c`) with one interface; the parser's output *is* this tree; `describe()` prints it	Lambdas (short anonymous functions) only (`a.and(b)`) — work, but cannot be printed or inspected (compare §21.1's `TaskFilter`)
`Node` / `FileNode` / `DirectoryNode`	**Composite**	The searcher and `totalSize()` treat files and directories uniformly	`instanceof` ("is this object of type X?") checks everywhere, or separate file and directory lists
`SearchCriteria.Builder`	**Builder**	Many optional settings; named calls; validation (`mindepth ≤ maxdepth`) once in `build()`; immutable result	Telescoping constructors or a mutable settings bag
Recursive-descent `FindCommandParser`	Interpreter-style parser	One method per precedence level makes precedence explicit and testable	Splitting on `-o` with string hacks
Iterative DFS with an explicit `ArrayDeque`	—	No `StackOverflowError` (the crash when the call stack runs out) on very deep trees; easy to add a limit	Recursion
Results go to a `Consumer<Node>`	Streaming / callback	Prints as it finds, like `find`; memory does not grow with the result count	Always building a full `List`
AND / OR short-circuit (stop as soon as the answer is known)	—	Put cheap tests first (`-name` before a size lookup on a real disk)	Evaluating every test
`ConcurrentSkipListMap` for children	Concurrent collection	Sorted (deterministic output) and safe to iterate while files are added	`TreeMap` (throws `ConcurrentModificationException`)

Java implementation

The file-system model:

```java
// File: lld/ch21/filesearch/FileType.java
package lld.ch21.filesearch;

public enum FileType { FILE, DIRECTORY }
```

```java
// File: lld/ch21/filesearch/SearchException.java
package lld.ch21.filesearch;

/** Bad path, bad option or bad expression: a readable message, never a stack trace for the user. */
public class SearchException extends RuntimeException {
    public SearchException(String message) { super(message); }
}
```

```java
// File: lld/ch21/filesearch/Node.java
package lld.ch21.filesearch;

import java.time.Instant;

/**
 * Composite pattern: a file-system entry is either a leaf (FileNode) or a container
 * (DirectoryNode) that holds more entries. Callers treat both through this one type.
 */
public abstract sealed class Node permits FileNode, DirectoryNode {
    private final String name;
    private final DirectoryNode parent;
    private final String owner;
    private final Instant modified;

    protected Node(String name, DirectoryNode parent, String owner, Instant modified) {
        this.name = name; this.parent = parent; this.owner = owner; this.modified = modified;
    }

    public String name() { return name; }
    public DirectoryNode parent() { return parent; }
    public String owner() { return owner; }
    public Instant modified() { return modified; }

    public abstract FileType type();

    /** Size in bytes of this entry alone (0 for a directory, as far as our filters care). */
    public abstract long size();

    /** du-style size: a file's own size; a directory's is the sum over everything inside it (Composite recursion).
*/
    public abstract long totalSize();

    public String path() {
        if (parent == null) return "/";
        String p = parent.path();
        return (p.equals("/") ? "" : p) + "/" + name;
    }

    @Override public String toString() { return path(); }
}
```

```java
// File: lld/ch21/filesearch/FileNode.java
package lld.ch21.filesearch;

import java.time.Instant;

public final class FileNode extends Node {
    private final long size;

    FileNode(String name, DirectoryNode parent, String owner, Instant modified, long size) {
        super(name, parent, owner, modified);
        if (size < 0) throw new SearchException("Negative size for " + name);
        this.size = size;
    }

    @Override public FileType type() { return FileType.FILE; }
    @Override public long size() { return size; }
    @Override public long totalSize() { return size; }
}
```

```java
// File: lld/ch21/filesearch/DirectoryNode.java
package lld.ch21.filesearch;

import java.time.Instant;
import java.util.Collection;
import java.util.concurrent.ConcurrentSkipListMap;

public final class DirectoryNode extends Node {
    /** Sorted by name (deterministic output) and safe to read while another thread adds files. */
    private final ConcurrentSkipListMap<String, Node> children = new ConcurrentSkipListMap<>();

    DirectoryNode(String name, DirectoryNode parent, String owner, Instant modified) {
        super(name, parent, owner, modified);
    }

    @Override public FileType type() { return FileType.DIRECTORY; }
    @Override public long size() { return 0; }
    @Override public long totalSize() { return children.values().stream().mapToLong(Node::totalSize).sum(); }

    public Collection<Node> children() { return children.values(); }
    public Node child(String name) { return children.get(name); }
    public boolean isEmpty() { return children.isEmpty(); }

    void add(Node n) {
        if (children.putIfAbsent(n.name(), n) != null)
            throw new SearchException(n.path() + " already exists");
    }
}
```

```java
// File: lld/ch21/filesearch/FileSystem.java
package lld.ch21.filesearch;

import java.time.Instant;

/** A small in-memory file system, so every demo run is identical. A java.nio adapter is an extension. */
public final class FileSystem {
    private final DirectoryNode root;

    public FileSystem(Instant created) { root = new DirectoryNode("", null, "root", created); }

    public DirectoryNode root() { return root; }

    /** Creates missing directories along the way, like mkdir -p. */
    public synchronized DirectoryNode mkdirs(String path, String owner, Instant modified) {
        DirectoryNode dir = root;
        for (String part : split(path)) {
            Node next = dir.child(part);
            if (next == null) { next = new DirectoryNode(part, dir, owner, modified); dir.add(next); }
            if (!(next instanceof DirectoryNode d)) throw new SearchException(next.path() + " is a file, not a
directory");
            dir = d;
        }
        return dir;
    }

    public synchronized FileNode createFile(String path, long size, String owner, Instant modified) {
        int slash = path.lastIndexOf('/');
        DirectoryNode dir = mkdirs(path.substring(0, Math.max(slash, 0)), owner, modified);
        FileNode f = new FileNode(path.substring(slash + 1), dir, owner, modified, size);
        dir.add(f);
        return f;
    }

    public Node resolve(String path) {
        Node n = root;
        for (String part : split(path)) {
            if (!(n instanceof DirectoryNode d) || (n = d.child(part)) == null)
                throw new SearchException("No such file or directory: " + path);
        }
        return n;
    }

    private static String[] split(String path) {
        String p = path.replaceAll("^/+|/+$", "");
        return p.isEmpty() ? new String[0] : p.split("/+");
    }
}
```

The filters — Specification leaves and Composite nodes:

```java
// File: lld/ch21/filesearch/FileFilter.java
package lld.ch21.filesearch;

/**
 * Specification pattern: one question, "does this entry match?". Leaf filters test one attribute;
 * AndFilter / OrFilter / NotFilter are Composite nodes that combine other filters into a tree.
 */
public interface FileFilter {
    boolean matches(Node n);

    /** Human-readable form of the expression tree, e.g. (name='*.log' AND size>+10M). */
    String describe();

    default FileFilter and(FileFilter other) { return AndFilter.of(this, other); }
    default FileFilter or(FileFilter other) { return OrFilter.of(this, other); }
    default FileFilter negate() { return new NotFilter(this); }
}
```

```java
// File: lld/ch21/filesearch/AndFilter.java
package lld.ch21.filesearch;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/** Composite: matches when every child matches. Short-circuits like find's -a. */
public record AndFilter(List<FileFilter> parts) implements FileFilter {
    public AndFilter { parts = List.copyOf(parts); }

    /** Flattens nested ANDs so (a AND (b AND c)) is stored as (a AND b AND c). */
    static FileFilter of(FileFilter a, FileFilter b) {
        List<FileFilter> all = new ArrayList<>();
        for (FileFilter f : List.of(a, b)) if (f instanceof AndFilter and) all.addAll(and.parts()); else
all.add(f);
        return new AndFilter(all);
    }

    @Override public boolean matches(Node n) {
        for (FileFilter f : parts) if (!f.matches(n)) return false;
        return true;
    }

    @Override public String describe() {
        return parts.stream().map(FileFilter::describe).collect(Collectors.joining(" AND ", "(", ")"));
    }
}
```

```java
// File: lld/ch21/filesearch/OrFilter.java
package lld.ch21.filesearch;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/** Composite: matches when any child matches. Short-circuits like find's -o. */
public record OrFilter(List<FileFilter> parts) implements FileFilter {
    public OrFilter { parts = List.copyOf(parts); }

    static FileFilter of(FileFilter a, FileFilter b) {
        List<FileFilter> all = new ArrayList<>();
        for (FileFilter f : List.of(a, b)) if (f instanceof OrFilter or) all.addAll(or.parts()); else all.add(f);
        return new OrFilter(all);
    }

    @Override public boolean matches(Node n) {
        for (FileFilter f : parts) if (f.matches(n)) return true;
        return false;
    }

    @Override public String describe() {
        return parts.stream().map(FileFilter::describe).collect(Collectors.joining(" OR ", "(", ")"));
    }
}
```

```java
// File: lld/ch21/filesearch/NotFilter.java
package lld.ch21.filesearch;

/** Composite with one child: find's ! / -not. */
public record NotFilter(FileFilter inner) implements FileFilter {
    @Override public boolean matches(Node n) { return !inner.matches(n); }
    @Override public String describe() { return "NOT " + inner.describe(); }
}
```

```java
// File: lld/ch21/filesearch/Filters.java
package lld.ch21.filesearch;

import java.time.Duration;
import java.time.Instant;
import java.util.regex.Pattern;

/** Static factories for the leaf filters (one per find test). New test = one new record here; nothing else
changes. */
public final class Filters {
    private Filters() { }

    public static FileFilter all() { return new MatchAll(); }
    public static FileFilter name(String glob) { return new NameFilter(glob, false); }
    public static FileFilter iname(String glob) { return new NameFilter(glob, true); }
    public static FileFilter type(FileType t) { return new TypeFilter(t); }
    public static FileFilter owner(String user) { return new OwnerFilter(user); }
    public static FileFilter empty() { return new EmptyFilter(); }
    /** spec like find's -size: "+10M", "-1k", "0c", "20" (no suffix = 512-byte blocks). */
    public static FileFilter size(String spec) { return SizeFilter.parse(spec); }
    /** spec like find's -mtime: "+7" (more than 7 whole days ago), "-1", "0". */
    public static FileFilter mtime(String spec, Instant now) { return MtimeFilter.parse(spec, now); }

    record MatchAll() implements FileFilter {
        @Override public boolean matches(Node n) { return true; }
        @Override public String describe() { return "true"; }
    }

    /** Glob on the last path component only, like find -name: * ? [abc] [!abc], and \ to escape. */
    record NameFilter(String glob, boolean ignoreCase, Pattern regex) implements FileFilter {
        NameFilter(String glob, boolean ignoreCase) {
            this(glob, ignoreCase, Pattern.compile(toRegex(glob), ignoreCase ? Pattern.CASE_INSENSITIVE |
Pattern.UNICODE_CASE : 0));
        }
        @Override public boolean matches(Node n) { return regex.matcher(n.name()).matches(); }
        @Override public String describe() { return (ignoreCase ? "iname='" : "name='") + glob + "'"; }

        static String toRegex(String glob) {
            StringBuilder re = new StringBuilder();
            for (int i = 0; i < glob.length(); i++) {
                char c = glob.charAt(i);
                switch (c) {
                    case '*' -> re.append(".*");
                    case '?' -> re.append('.');
                    case '\\' -> { if (++i < glob.length())
re.append(Pattern.quote(String.valueOf(glob.charAt(i)))); }
                    case '[' -> {
                        int end = glob.indexOf(']', i + 2);
                        if (end < 0) { re.append("\\["); break; }          // no closing ]: a literal [
                        String body = glob.substring(i + 1, end);
                        if (body.startsWith("!")) body = "^" + body.substring(1);
                        re.append('[').append(body.replace("\\", "\\\\")).append(']');
                        i = end;
                    }
                    default -> re.append(Pattern.quote(String.valueOf(c)));
                }
            }
            return re.toString();
        }
    }

    record TypeFilter(FileType type) implements FileFilter {
        @Override public boolean matches(Node n) { return n.type() == type; }
        @Override public String describe() { return "type=" + (type == FileType.FILE ? "f" : "d"); }
    }

    record OwnerFilter(String user) implements FileFilter {
        @Override public boolean matches(Node n) { return n.owner().equals(user); }
```

```
        @Override public String describe() { return "user=" + user; }
    }

    record EmptyFilter() implements FileFilter {
        @Override public boolean matches(Node n) {
            return n instanceof DirectoryNode d ? d.isEmpty() : n.size() == 0;
        }
        @Override public String describe() { return "empty"; }
    }

    /**
     * GNU find semantics: the file size is rounded UP to whole units, then compared.
     * So "-size -1M" means "fewer than 1 whole MiB after rounding up" = only empty files. A famous gotcha.
     * Size tests apply to files only in this model.
     */
    record SizeFilter(char cmp, long n, char unit, String spec) implements FileFilter {
        static SizeFilter parse(String spec) {
            var m = Pattern.compile("([+-]?)(\\d+)([cbkMG]?)").matcher(spec);
            if (!m.matches()) throw new SearchException("Invalid -size '" + spec + "' (use e.g. +10M, -1k, 100c)");
            char cmp = m.group(1).isEmpty() ? '=' : m.group(1).charAt(0);
            char unit = m.group(3).isEmpty() ? 'b' : m.group(3).charAt(0);
            return new SizeFilter(cmp, Long.parseLong(m.group(2)), unit, spec);
        }
        long unitBytes() {
            return switch (unit) { case 'c' -> 1; case 'b' -> 512; case 'k' -> 1024; case 'M' -> 1024L * 1024;
default -> 1024L * 1024 * 1024; };
        }
        @Override public boolean matches(Node node) {
            if (node.type() != FileType.FILE) return false;
            long units = (node.size() + unitBytes() - 1) / unitBytes();      // round up
            return switch (cmp) { case '+' -> units > n; case '-' -> units < n; default -> units == n; };
        }
        @Override public String describe() { return "size" + (cmp == '=' ? "=" : "") + spec; }
    }

    /** -mtime n: age in whole days (fraction dropped); +n = older than n days, -n = newer than n days. */
    record MtimeFilter(char cmp, long days, Instant now, String spec) implements FileFilter {
        static MtimeFilter parse(String spec, Instant now) {
            var m = Pattern.compile("([+-]?)(\\d+)").matcher(spec);
            if (!m.matches()) throw new SearchException("Invalid -mtime '" + spec + "' (use e.g. +7, -1, 0)");
            char cmp = m.group(1).isEmpty() ? '=' : m.group(1).charAt(0);
            return new MtimeFilter(cmp, Long.parseLong(m.group(2)), now, spec);
        }
        @Override public boolean matches(Node node) {
            long age = Duration.between(node.modified(), now).toDays();
            return switch (cmp) { case '+' -> age > days; case '-' -> age < days; default -> age == days; };
        }
        @Override public String describe() { return "mtime" + (cmp == '=' ? "=" : "") + spec; }
    }
}
```

Criteria, search and parser:

```java
// File: lld/ch21/filesearch/SearchCriteria.java
package lld.ch21.filesearch;

import java.util.LinkedHashSet;
import java.util.Set;

/**
 * Everything one search needs, immutable once built. Builder pattern: most options are optional,
 * so named builder calls replace a constructor with eight positional parameters.
 */
public final class SearchCriteria {
    private final String startPath;
    private final FileFilter filter;
    private final int minDepth, maxDepth, limit;
    private final Set<String> pruneNames;

    private SearchCriteria(Builder b) {
        startPath = b.startPath; filter = b.filter; minDepth = b.minDepth; maxDepth = b.maxDepth;
        limit = b.limit; pruneNames = Set.copyOf(b.pruneNames);
    }

    public static Builder from(String startPath) { return new Builder(startPath); }

    public String startPath() { return startPath; }
    public FileFilter filter() { return filter; }
    public int minDepth() { return minDepth; }
    public int maxDepth() { return maxDepth; }
    public int limit() { return limit; }
    public Set<String> pruneNames() { return pruneNames; }

    @Override public String toString() {
        return "from " + startPath + " where " + filter.describe()
                + (minDepth > 0 ? " mindepth=" + minDepth : "")
                + (maxDepth < Integer.MAX_VALUE ? " maxdepth=" + maxDepth : "")
                + (pruneNames.isEmpty() ? "" : " prune=" + pruneNames)
                + (limit < Integer.MAX_VALUE ? " limit=" + limit : "");
    }

    public static final class Builder {
        private final String startPath;
        private FileFilter filter = Filters.all();
        private boolean filterSet;
        private int minDepth = 0, maxDepth = Integer.MAX_VALUE, limit = Integer.MAX_VALUE;
        private final Set<String> pruneNames = new LinkedHashSet<>();

        private Builder(String startPath) {
            if (startPath == null || startPath.isBlank()) throw new SearchException("Start path is required");
            this.startPath = startPath;
        }

        /** Each call is ANDed with the previous ones, like consecutive tests on a find command line. */
        public Builder where(FileFilter f) { filter = filterSet ? filter.and(f) : f; filterSet = true; return this;
}
        public Builder name(String glob) { return where(Filters.name(glob)); }
        public Builder type(FileType t) { return where(Filters.type(t)); }
        public Builder size(String spec) { return where(Filters.size(spec)); }
        public Builder minDepth(int d) { minDepth = d; return this; }
        public Builder maxDepth(int d) { maxDepth = d; return this; }
        public Builder limit(int n) { limit = n; return this; }
        /** Do not descend into directories with this name (like: -name node_modules -prune -o ...). */
        public Builder prune(String dirName) { pruneNames.add(dirName); return this; }

        public SearchCriteria build() {
            if (minDepth < 0 || maxDepth < 0) throw new SearchException("Depths must be >= 0");
            if (minDepth > maxDepth) throw new SearchException("mindepth " + minDepth + " > maxdepth " + maxDepth);
            if (limit <= 0) throw new SearchException("Limit must be positive");
            return new SearchCriteria(this);
        }
```

```
    }
}
```

```java
// File: lld/ch21/filesearch/FileSearcher.java
package lld.ch21.filesearch;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.List;
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;
import java.util.function.Consumer;

/** Walks the tree and reports every entry that passes the filter. */
public final class FileSearcher {
    private final FileSystem fs;

    public FileSearcher(FileSystem fs) { this.fs = fs; }

    private record Frame(Node node, int depth) { }

    /**
     * Pre-order depth-first walk with an explicit stack (no recursion, so a very deep tree cannot
     * overflow the call stack). Results are streamed to the consumer as they are found, like find
     * printing line by line; nothing forces the whole result list into memory.
     */
    public int search(SearchCriteria c, Consumer<Node> out) {
        Deque<Frame> stack = new ArrayDeque<>();
        stack.push(new Frame(fs.resolve(c.startPath()), 0));
        int found = 0;
        while (!stack.isEmpty() && found < c.limit()) {
            Frame f = stack.pop();
            Node n = f.node();
            if (f.depth() >= c.minDepth() && c.filter().matches(n)) { out.accept(n); found++; }
            if (n instanceof DirectoryNode d && f.depth() < c.maxDepth() && !c.pruneNames().contains(d.name())) {
                List<Node> kids = new ArrayList<>(d.children());
                Collections.reverse(kids);                    // push in reverse so we pop in name order
                for (Node k : kids) stack.push(new Frame(k, f.depth() + 1));
            }
        }
        return found;
    }

    public List<Node> search(SearchCriteria c) {
        List<Node> out = new ArrayList<>();
        search(c, out::add);
        return out;
    }

    /** Extension: one fork/join task per directory. Same results in the same order; no limit support. */
    public List<Node> searchParallel(SearchCriteria c, ForkJoinPool pool) {
        return pool.invoke(new Walk(fs.resolve(c.startPath()), 0, c));
    }

    private static final class Walk extends RecursiveTask<List<Node>> {
        private final Node node; private final int depth; private final SearchCriteria c;
        Walk(Node node, int depth, SearchCriteria c) { this.node = node; this.depth = depth; this.c = c; }

        @Override protected List<Node> compute() {
            List<Node> out = new ArrayList<>();
            if (depth >= c.minDepth() && c.filter().matches(node)) out.add(node);
            if (node instanceof DirectoryNode d && depth < c.maxDepth() && !c.pruneNames().contains(d.name())) {
                List<Walk> tasks = new ArrayList<>();
                List<Object> slots = new ArrayList<>();         // files tested inline, sub-directories forked
                for (Node k : d.children()) {
                    if (k instanceof DirectoryNode) { Walk w = new Walk(k, depth + 1, c); tasks.add(w);
slots.add(w); }
                    else slots.add(k);
                }
```

```
                invokeAll(tasks);                               // fork all sub-directories, wait for all
                for (Object s : slots) {                        // collect in name order: same order as search()
                    if (s instanceof Walk w) out.addAll(w.join());
                    else if (depth + 1 >= c.minDepth() && c.filter().matches((Node) s)) out.add((Node) s);
                }
            }
            return out;
        }
    }
}
```

```java
// File: lld/ch21/filesearch/FindCommandParser.java
package lld.ch21.filesearch;

import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.regex.Pattern;

/**
 * Turns a find-style command line into SearchCriteria, with find's precedence:
 *   expr   := or
 *   or     := and ( ("-o" | "-or") and )*
 *   and    := unary ( ["-a" | "-and"] unary )*      -- two tests side by side mean AND
 *   unary  := ("!" | "-not") unary | "(" expr ")" | test
 *   test   := -name G | -iname G | -type f|d | -size N | -mtime N | -user U | -empty | -true
 * Global options -maxdepth N / -mindepth N may appear anywhere. This is recursive descent:
 * one method per grammar rule, each returning a FileFilter (the Composite tree).
 */
public final class FindCommandParser {
    private final Instant now;
    private List<String> tokens;
    private int pos;
    private SearchCriteria.Builder builder;

    public FindCommandParser(Instant now) { this.now = now; }

    /** tokens as the shell would pass them, e.g. ["find", "/home", "-name", "*.java", "-o", "-name", "*.md"]. */
    public SearchCriteria parse(List<String> args) {
        if (args.isEmpty() || !args.get(0).equals("find")) throw new SearchException("Command must start with
'find'");
        tokens = args; pos = 1;
        String start = pos < tokens.size() && !isOperatorOrTest(tokens.get(pos)) ? tokens.get(pos++) : ".";
        builder = SearchCriteria.from(start.equals(".") ? "/" : start);
        skipGlobalOptions();
        if (pos < tokens.size()) {                    // no tests at all = match everything
            builder.where(parseOr());
            if (pos < tokens.size()) throw new SearchException("Unexpected '" + tokens.get(pos) + "'");
        }
        return builder.build();
    }

    /** Convenience: split on spaces; quotes group words and are removed (enough for the demo). */
    public SearchCriteria parse(String commandLine) {
        List<String> out = new ArrayList<>();
        var m = Pattern.compile("'([^']*)'|\"([^\"]*)\"|(\\S+)").matcher(commandLine);
        while (m.find()) out.add(m.group(1) != null ? m.group(1) : m.group(2) != null ? m.group(2) : m.group(3));
        return parse(out);
    }

    private FileFilter parseOr() {
        FileFilter left = parseAnd();
        while (peekIs("-o", "-or")) { pos++; left = left.or(parseAnd()); }
        return left;
    }

    private FileFilter parseAnd() {
        FileFilter left = parseUnary();
        while (true) {
            skipGlobalOptions();
            if (pos >= tokens.size() || peekIs("-o", "-or", ")")) return left;
            if (peekIs("-a", "-and")) pos++;
            left = left.and(parseUnary());
        }
    }

    private FileFilter parseUnary() {
        skipGlobalOptions();
        if (pos >= tokens.size()) throw new SearchException("Expression ends too early");
```

```
        String t = tokens.get(pos++);
        switch (t) {
            case "!", "-not" -> { return parseUnary().negate(); }
            case "(" -> {
                FileFilter inner = parseOr();
                if (!peekIs(")")) throw new SearchException("Missing ')'");
                pos++;
                return inner;
            }
            case ")" -> throw new SearchException("Unexpected ')'");
            case "-empty" -> { return Filters.empty(); }
            case "-true" -> { return Filters.all(); }
            case "-name" -> { return Filters.name(arg(t)); }
            case "-iname" -> { return Filters.iname(arg(t)); }
            case "-user" -> { return Filters.owner(arg(t)); }
            case "-size" -> { return Filters.size(arg(t)); }
            case "-mtime" -> { return Filters.mtime(arg(t), now); }
            case "-type" -> {
                String v = arg(t);
                return switch (v) {
                    case "f" -> Filters.type(FileType.FILE);
                    case "d" -> Filters.type(FileType.DIRECTORY);
                    default -> throw new SearchException("-type takes f or d, not '" + v + "'");
                };
            }
            default -> throw new SearchException("Unknown predicate '" + t + "'");
        }
    }

    /** -maxdepth/-mindepth are options, not tests: record them and keep parsing the expression. */
    private void skipGlobalOptions() {
        while (peekIs("-maxdepth", "-mindepth")) {
            String opt = tokens.get(pos++);
            int n;
            try { n = Integer.parseInt(arg(opt)); } catch (NumberFormatException e) { throw new SearchException(opt
+ " needs a number"); }
            if (opt.equals("-maxdepth")) builder.maxDepth(n); else builder.minDepth(n);
        }
    }

    private String arg(String option) {
        if (pos >= tokens.size()) throw new SearchException(option + " needs an argument");
        return tokens.get(pos++);
    }

    private boolean peekIs(String... options) {
        if (pos >= tokens.size()) return false;
        for (String o : options) if (tokens.get(pos).equals(o)) return true;
        return false;
    }

    private static boolean isOperatorOrTest(String t) { return t.startsWith("-") || t.equals("!") || t.equals("(")
|| t.equals(")"); }
}
```

```java
// File: lld/ch21/filesearch/Main.java
package lld.ch21.filesearch;

import java.time.Duration;
import java.time.Instant;
import java.util.List;
import java.util.concurrent.ForkJoinPool;

public class Main {
    static final Instant NOW = Instant.parse("2026-10-01T12:00:00Z");
    static final long KB = 1024, MB = 1024 * KB;

    public static void main(String[] args) {
        FileSystem fs = sampleTree();
        FileSearcher finder = new FileSearcher(fs);
        FindCommandParser parser = new FindCommandParser(NOW);

        System.out.println("== Builder API ==");
        show(finder, SearchCriteria.from("/").name("*.log").type(FileType.FILE).build());
        show(finder, SearchCriteria.from("/home").where(Filters.iname("*.jpg").and(Filters.size("+1M"))).build());
        show(finder, SearchCriteria.from("/").type(FileType.DIRECTORY).where(Filters.empty()).build());
        show(finder, SearchCriteria.from("/home").maxDepth(2).build());
        show(finder, SearchCriteria.from("/home/asha/projects")
                .where(Filters.name("*.java").or(Filters.name("*.js"))).prune("node_modules").build());
        show(finder, SearchCriteria.from("/").type(FileType.FILE).limit(3).build());

        System.out.println("\n== find command lines ==");
        for (String cmd : List.of(
                "find /var/log -type f -size +10M -mtime +7",
                "find /home -name '*.java' -o -name '*.md'",
                "find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M",
                "find /home -size -1M",
                "find / -user ravi -type f",
                "find /home/asha -mindepth 2 -maxdepth 2 -type d",
                "find /home -iname 'b*'",
                "find /var/log -name 'app.log.[0-9]'"))
            show(finder, parser.parse(cmd), cmd);

        System.out.println("\n== bad input gives a clear message ==");
        for (String cmd : List.of("find / -size 10X", "find / ( -name a", "find /nope -name x",
                "find / -frobnicate", "find / -type x", "find / -maxdepth 1 -mindepth 3", "find / -name"))
            try { finder.search(parser.parse(cmd)); System.out.println("   (no error?) " + cmd); }
            catch (SearchException e) { System.out.printf("   %-34s -> %s%n", cmd, e.getMessage()); }

        System.out.println("\n== Composite sizes (du -s) ==");
        for (String p : List.of("/home/asha", "/home/asha/projects/app/node_modules", "/var/log"))
            System.out.printf("   %-40s %,d bytes%n", p, fs.resolve(p).totalSize());

        System.out.println("\n== Parallel search on a big generated tree ==");
        FileSystem big = new FileSystem(NOW);
        for (int d = 0; d < 200; d++)
            for (int f = 0; f < 100; f++)
                big.createFile("/data/d%03d/sub%d/f%03d.%s".formatted(d, f % 5, f, f % 3 == 0 ? "csv" : "txt"),
                        (long) f * 10 * KB, "etl", NOW.minus(Duration.ofDays(f % 30)));
        SearchCriteria q = new FindCommandParser(NOW).parse("find /data -name '*.csv' -size +500k -mtime -10");
        FileSearcher bigFinder = new FileSearcher(big);
        List<Node> seq = bigFinder.search(q);
        List<Node> par = bigFinder.searchParallel(q, ForkJoinPool.commonPool());
        System.out.println("   " + q);
        System.out.println("   sequential=" + seq.size() + " parallel=" + par.size() + " same order=" +
seq.equals(par)
                + " first=" + seq.get(0) + " last=" + seq.get(seq.size() - 1));
    }

    static void show(FileSearcher finder, SearchCriteria c) { show(finder, c, c.toString()); }

    static void show(FileSearcher finder, SearchCriteria c, String title) {
        System.out.println("$ " + title);
```

```
        if (!title.equals(c.toString())) System.out.println("  parsed: " + c);
        int n = finder.search(c, node -> System.out.println("   " + node + (node instanceof DirectoryNode ? "/" :
""")));
        if (n == 0) System.out.println("   (no matches)");
    }

    static FileSystem sampleTree() {
        FileSystem fs = new FileSystem(NOW);
        Instant today = NOW.minus(Duration.ofHours(3)), lastWeek = NOW.minus(Duration.ofDays(8)), old =
NOW.minus(Duration.ofDays(40));
        fs.createFile("/home/asha/notes.txt", 2 * KB, "asha", today);
        fs.createFile("/home/asha/photos/a.jpg", 3 * MB, "asha", lastWeek);
        fs.createFile("/home/asha/photos/b.JPG", 5 * MB, "asha", lastWeek);
        fs.createFile("/home/asha/photos/c.png", 800 * KB, "asha", old);
        fs.createFile("/home/asha/photos/d.jpg", 900 * KB, "asha", old);
        fs.createFile("/home/asha/projects/app/Main.java", 12 * KB, "asha", today);
        fs.createFile("/home/asha/projects/app/Util.java", 4 * KB, "asha", today);
        fs.createFile("/home/asha/projects/app/build.log", 3 * MB / 2, "asha", today);
        fs.createFile("/home/asha/projects/app/node_modules/lodash.js", 500 * KB, "asha", old);
        fs.createFile("/home/asha/projects/app/node_modules/react.js", MB, "asha", old);
        fs.mkdirs("/home/asha/projects/empty-dir", "asha", old);
        fs.createFile("/home/ravi/todo.md", 0, "ravi", today);
        fs.createFile("/home/ravi/report.pdf", 2 * MB, "ravi", lastWeek);
        fs.createFile("/var/log/syslog", 40 * MB, "root", old);
        fs.createFile("/var/log/app.log", 3 * MB, "root", today);
        fs.createFile("/var/log/app.log.1", 20 * MB, "root", lastWeek);
        fs.mkdirs("/tmp", "root", today);
        return fs;
    }
}
```

How the code works, part by part

- **Globs.** `NameFilter.toRegex` turns `*` into `.*`, `?` into `.`, keeps `[...]` sets (turning `[!...]` into `[^...]`) and quotes every other character, then compiles the regex once, when the filter is built. Like `find -name`, it matches only the last path component, and `*` also matches names that start with a dot.
- **Size.** `SizeFilter` parses `[+-]N[cbkMG]` (no suffix = 512-byte blocks, as in `find`), rounds the file size **up** to whole units, then compares: `+` means more than N units, `-` fewer, no sign exactly N.
- **Age.** `MtimeFilter` computes the age in whole days (the fraction is dropped) against an injected `now`, so the demo is repeatable.
- **Builder.** Each `where(...)` call is ANDed with earlier ones; `build()` checks depths and the limit and freezes everything into an immutable `SearchCriteria`.
- **Search.** `FileSearcher.search` pushes the start node, then pops, tests, emits and pushes children in reverse name order so they come off the stack in name order. It stops descending at `maxDepth` and at pruned directory names, and stops entirely at the limit.
- **Parser.** `parseOr` → `parseAnd` → `parseUnary`: one method per precedence level. `parseAnd` treats two tests side by side as AND; `-maxdepth`/`-mindepth` are options that can appear anywhere, so every level skips them first.
- **Parallel.** `searchParallel` forks one task per sub-directory (files are tested inline), waits for all, then joins results in name order — the same list, in the same order, as the sequential search.

Output:

```
== Builder API ==
$ from / where (name='*.log' AND type=f)
   /home/asha/projects/app/build.log
   /var/log/app.log
$ from /home where (iname='*.jpg' AND size+1M)
   /home/asha/photos/a.jpg
   /home/asha/photos/b.JPG
$ from / where (type=d AND empty)
   /home/asha/projects/empty-dir/
   /tmp/
$ from /home where true maxdepth=2
   /home/
   /home/asha/
   /home/asha/notes.txt
   /home/asha/photos/
   /home/asha/projects/
   /home/ravi/
   /home/ravi/report.pdf
   /home/ravi/todo.md
$ from /home/asha/projects where (name='*.java' OR name='*.js') prune=[node_modules]
   /home/asha/projects/app/Main.java
   /home/asha/projects/app/Util.java
$ from / where type=f limit=3
   /home/asha/notes.txt
   /home/asha/photos/a.jpg
   /home/asha/photos/b.JPG

== find command lines ==
$ find /var/log -type f -size +10M -mtime +7
  parsed: from /var/log where (type=f AND size+10M AND mtime+7)
   /var/log/app.log.1
   /var/log/syslog
$ find /home -name '*.java' -o -name '*.md'
  parsed: from /home where (name='*.java' OR name='*.md')
   /home/asha/projects/app/Main.java
   /home/asha/projects/app/Util.java
   /home/ravi/todo.md
$ find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M
  parsed: from /home where ((name='*.jpg' OR name='*.png') AND NOT size+1M)
   /home/asha/photos/c.png
   /home/asha/photos/d.jpg
$ find /home -size -1M
  parsed: from /home where size-1M
   /home/ravi/todo.md
$ find / -user ravi -type f
  parsed: from / where (user=ravi AND type=f)
   /home/ravi/report.pdf
   /home/ravi/todo.md
$ find /home/asha -mindepth 2 -maxdepth 2 -type d
  parsed: from /home/asha where type=d mindepth=2 maxdepth=2
   /home/asha/projects/app/
   /home/asha/projects/empty-dir/
$ find /home -iname 'b*'
  parsed: from /home where iname='b*'
   /home/asha/photos/b.JPG
   /home/asha/projects/app/build.log
$ find /var/log -name 'app.log.[0-9]'
  parsed: from /var/log where name='app.log.[0-9]'
   /var/log/app.log.1

== bad input gives a clear message ==
   find / -size 10X                  -> Invalid -size '10X' (use e.g. +10M, -1k, 100c)
   find / ( -name a                  -> Missing ')'
   find /nope -name x                -> No such file or directory: /nope
   find / -frobnicate                -> Unknown predicate '-frobnicate'
   find / -type x                    -> -type takes f or d, not 'x'
   find / -maxdepth 1 -mindepth 3    -> mindepth 3 > maxdepth 1
   find / -name                      -> -name needs an argument
```

```
== Composite sizes (du -s) ==
   /home/asha                              13,281,280 bytes
   /home/asha/projects/app/node_modules     1,560,576 bytes
   /var/log                                66,060,288 bytes

== Parallel search on a big generated tree ==
   from /data where (name='*.csv' AND size+500k AND mtime-10)
   sequential=1600 parallel=1600 same order=true first=/data/d000/sub0/f060.csv last=/data/d199/sub4/f099.csv
```

Reading the output: `-size +1M` kept the two large photos and dropped the 900 KB one; the prune option skipped `node_modules`, so only the two Java files match `*.java OR *.js`; `-size -1M` found only the empty `todo.md` — the rounding gotcha, even though `notes.txt` is only 2 KB; `-mindepth 2 -maxdepth 2 -type d` lists directories exactly two levels below `/home/asha`; the bracket glob `app.log.[0-9]` matches the rotated log but not `app.log`. Every bad input produced a one-line message. The composite `totalSize()` gives `du`-style totals (`du` is the Unix disk-usage command), and on a generated tree of 20,000 files the sequential and parallel searches returned the same 1,600 paths in the same order.

Concurrency & edge cases

- **Adding files during a search:** children live in a `ConcurrentSkipListMap`, whose iterators are *weakly consistent* — iterators (the objects that step through a collection) never throw, and may or may not show an entry added after the iteration started (like `find` on a live disk).
- **Parallel search:** tasks only read the tree and each builds its own list; there is no shared mutable state, so no locks are needed.
- **Deep trees:** the sequential walk uses a heap-allocated stack (an ordinary object in main memory, not the limited call stack). (The fork/join version recurses one task per directory; fine for normal depths.)
- **Symbolic links / cycles** (real file systems): track visited `(device, inode)` pairs (an inode is the file system's internal id for a file; a symbolic link is a file that points to another path) or do not follow links by default — GNU `find` does not follow symbolic links unless asked (`-L`).
- **Edge cases handled:** start path missing or a file; `mindepth > maxdepth`; unknown predicate; missing argument; bad `-size`/`-mtime`/`-type` values; unbalanced parentheses; an unclosed `[` in a glob is treated as a literal; limit reached mid-walk.

Extensions & follow-up questions

- **Search a real disk?** An adapter that wraps `java.nio.file.Path` + `BasicFileAttributes` as `Node`s, or use `Files.walkFileTree` with a `FileVisitor` that returns `SKIP_SUBTREE` for pruned directories.
- **Actions (`-delete`, `-exec`)?** A `Consumer<Node>` per action (Command pattern); `-delete` must walk depth-first *post-order*, meaning children are visited before their directory.
- **Content search (`grep -r`, the Unix tool that searches inside files)?** A `ContainsTextFilter` — put it last in the AND so cheap name/size tests reject most files first.
- **Faster repeated searches?** Build an index (name → paths, like `locate`/`updatedb`, Unix tools that pre-build a filename database) and accept that it can be stale.
- **More tests (`-newer file`, `-perm`, `-regex`)?** One record each in `Filters` plus one `case` in the parser.
- **Sort by size or date?** Collect, then sort with a `Comparator` — a Strategy, as in PropertyHunt (§21.5).

Real-world example

GNU `find` (from GNU findutils) evaluates its expression left to right with exactly this precedence (`!`, then implicit or explicit `-a`, then `-o`) and short-circuits; its manual documents the "`-size -1M` only matches empty files" rounding and the default of not following symbolic links. Java's own `Files.find(start, maxDepth, BiPredicate)` and `Files.walk` take a depth limit and a predicate — the same split between *walking* and *testing* used here. IDE "Find in files" dialogs and desktop search boxes build the same kind of filter tree from checkboxes instead of a command line.

In the interview

Spend the first 10 minutes on the model (`Node`, `FileNode`, `DirectoryNode`) and the `FileFilter` interface with two leaves (name, size) — that alone lets you demo a search. Then add `AndFilter`/`OrFilter`/`NotFilter`, the Builder with depth options, and the iterative walk. Add the parser last, and only if time allows: interviewers mostly

score the filter design and extensibility. Say the Open/Closed point out loud: "a new filter is one class; the searcher never changes".

Self-check

- **Q:** What does `find -name a -o -name b -type f` mean? → **A:** `a OR (b AND file)`, because AND binds tighter than OR.
- **Q:** Why does `-size -1M` match only empty files? → **A:** Sizes are rounded up to whole MiB, so any non-empty file is at least 1 unit, which is not less than 1.
- **Q:** Why are the AND/OR/NOT filters records rather than lambdas? → **A:** They can be printed (`describe()`), inspected and built by the parser as a visible tree.
- **Q:** How does pruning differ from filtering out a directory? → **A:** Pruning stops the walk from entering it (nothing inside is visited); a filter only hides the directory entry itself.
- **Q:** Why an explicit stack instead of recursion? → **A:** Very deep trees would overflow the call stack; an explicit stack also makes the limit easy.

TL;DR

Two Composites — the file tree and the filter tree. Filters are Specifications (`matches`) combined with AND / OR / NOT; the Builder describes the search once and validates it; an iterative DFS streams matches; a recursive-descent parser turns `find` command lines into the same criteria, with `!` > AND > OR precedence and GNU's rounded-up `-size`.

Chapter summary

Problem	Core entities	Key pattern(s)	Concurrency point	Trap to avoid
Task Management	Task, TaskStatus, TaskService	Specification filters, enum state machine, Observer	per-task `synchronized`	TODO → DONE jumps; filters as many parameters
Digital Wallet	Wallet, Transaction, Offer	Strategy offers, idempotency, funded cashback	lock both wallets in id order	`double` money; check-then-act on keys; cashback created from nothing
Event Calendar	TimeSlot, Event, EventStore	Strategy for representatives, plan-then-save	atomic check-and-book	closed intervals; trusting the sample blindly
Coding Blox	Contest, Level, User	Command map, Strategy (solver, scoring)	atomic rating update	unseeded randomness; running a contest twice
PropertyHunt	Property, SearchCriteria	Builder + predicate list, Strategy sort	synchronized mark-sold	comparing "90L" strings; stale shortlist copies
Ledger Co	Loan, LumpSum, Balance	Strategy interest, derived state	synchronized lump sums	storing a running balance; floor instead of ceil
Inventory	StockMovement, AllocationStrategy	Strategy allocation, Observer alerts	per-SKU lock	partial shipments; alert spam
Unix find	Node (Composite), FileFilter, SearchCriteria	Specification + Composite filters, Builder, recursive-descent parser	weakly consistent children map; read-only parallel walk	parameter-list search methods; `-o` vs AND precedence

The pattern behind all eight: entities guard their own rules, services orchestrate, strategies hold the "X or Y" rules, repositories hide storage, the driver parses and prints, and every error is a clear message — not a crash.

Chapter 22 — Revision: Cheat Sheets & Glossary

What you will learn in this chapter

- How to revise the whole handbook in 15 minutes, or in one evening the night before.
- The LLD (low-level design: the classes and code inside one program) answer framework on one page (Chapter 1).
- OOP, relationships and UML arrows on one page (Chapters 2–3).
- SOLID, the other design principles, and the clean-code rules as tables (Chapters 4–6).
- All 24 code smells with the refactorings that cure them (Chapter 7).
- All 23 GoF patterns (the classic catalogue by the "Gang of Four" authors) plus the extras, and a "which pattern?" table sorted by symptom (Chapters 8–11).
- The concurrency toolbox, a deadlock checklist and a "which `java.util.concurrent` class?" table (`java.util.concurrent` is Java's standard package of thread-safety tools) (Chapters 12–13).
- A map of every case study: entities, patterns, concurrency concern, and where to find it (Chapters 14–21).
- A machine-coding checklist, an edge-case checklist, and a glossary of about 200 terms.

Nothing in this chapter is new. Every row is a summary of a section in Chapters 1–21, and the right-hand column tells you where to go for the full explanation. This chapter has no code.

How to read the references. "§12.17" means section 12.17. Chapters 5 and 6 number their sections from 1 inside the chapter, so "Ch 5 §13" means section 13 of Chapter 5 (Fail Fast). Chapters 16, 17 and 19 have unnumbered case studies, so they are referenced by name, for example "Ch 16 · Car Rental".

New to a term? Every technical word in the tables below, from `API` and `JVM` to `TTL` and `SKU`, has a one-line definition in the glossary (22.11). A **TL;DR** ("too long; didn't read") heading marks a short summary.

22.1 How to revise

In 15 minutes (the morning of the interview)

Minutes	Read	Why
0–3	22.2 (the framework)	You will use it in the first minute of the interview
3–6	22.4 (SOLID one-pager) and the relationship table in 22.3	The two things asked about most often by name
6–10	"Which pattern?" table in 22.7	Patterns should come from a signal, not from memory
10–13	Concurrency cheat sheet and deadlock checklist in 22.8	Every booking/inventory problem has a race
13–15	The row(s) of 22.9 for the problems you expect	One sentence per problem: entities, pattern, the atomic step

The night before (about 2 hours)

1. **Read 22.2–22.8 slowly** (45 min). For each table row, say out loud *one example from the handbook*. If you cannot, open the referenced section.
2. **Pick three case studies** from 22.9 you have not practised recently (45 min). For each: write requirements, list classes, draw the class diagram, and name the one hard method, without looking. Then compare with the chapter's TL;DR.
3. **Scan the glossary** (22.11) and mark any term you cannot explain in one sentence (20 min). Read those sections only.
4. **Stop.** Sleep matters more than one more pattern.

Common mistakes when revising

- Re-reading chapters instead of **recalling** first and then checking. Recall is what the interview tests.
- Memorising all 23 patterns equally. Six come up in almost every round (22.7): Strategy, Factory, Observer, State, Singleton, Builder.
- Skipping the "when *not* to use it" column. Interviewers ask "would you always do that?" (§4.13).

22.2 The LLD interview answer framework (one page)

Source: Chapter 1 (§1.2, §1.4–§1.10, §1.12, §1.14).

TL;DR

Clarify → requirements → use cases → entities → relationships → class diagram → key flows → patterns/SOLID → code → edge cases/concurrency → extensions. Say each step out loud as you enter it.

The eleven steps

#	Step	Ask yourself	Output
1	Clarify	Who uses it? Single site or many?	Shared understanding
2	Requirements	Functional? Non-functional (thread-safe, extensible, in-memory)? **Out of scope?**	8–12 numbered lines + out-of-scope list
3	Use cases	What can each actor do?	Actor → use-case list
4	Entities	Which nouns hold state or behaviour? Which are just values?	Classes, one sentence each
5	Relationships	Uses / knows / has / owns / is-a? How many?	Typed lines with multiplicity
6	Class diagram	Fields, key methods, interfaces, enums	UML class diagram
7	Key flows	Steps of the 2–3 main use cases; lifecycle of the core object	Sequence / activity / state diagram
8	Patterns & SOLID	A rule that varies? A lifecycle? Many listeners? Two reasons to change?	1–2 named patterns + one rejected
9	Code	Interfaces, enums, models, services, driver	Running code for the main flows
10	Edge cases & concurrency	Bad input? Full? Two users, one seat?	Validation, exceptions, locks
11	Extensions	"What if X is added?"	Short answers, maybe one class

Time budgets

45-minute OOD (object-oriented design) round		**90-minute machine-coding round (write runnable code)**	
Clarify requirements	5–8 min	Read, requirements, assumptions	5–10 min
Actors and use cases	3–5 min	Model: entities, enums, service interfaces	10–15 min
Find the classes	8–10 min	Core flow end to end	30–40 min
Connect the classes	8–10 min	Remaining features, validation, edge cases	15–20 min
Write the code	12–15 min	Polish, one extension point, demo	5–10 min
		Buffer	5 min (never zero)

Rule of thumb: roughly **a quarter of the hour goes before you name a class**, and in machine coding you **get something running by the halfway mark** (§1.2).

Finding classes (§1.6)

- **Nouns → candidate classes; verbs → methods.** A verb with no class to live on means a class is missing.
- Cut candidates with five tests: is it a **value**? the **same thing** under another name? does it have **changing state**? does anything **call** it? does it differ in **behaviour or only data**?
- **Different data → field or enum. Different behaviour → class.**
- One sentence per class, **no "and"**. Put each method where its data is (Information Expert).
- Beginners most often forget the **record-keeping class** (ticket, loan, booking) and the **rule class** (rate, strategy).

What the interviewer judges (§1.2)

1. Did you **bound** the problem? 2. Does each class have **one job** you can say in a sentence? 3. Are the **relationships** right? 4. Does the **code match the diagram**? 5. Can you explain a **trade-off** (the choice and the rejected option)?

Machine coding adds: does it **run**, is it **modular**, **readable**, does it **handle bad input**, and (senior roles) is shared state **thread-safe**.

Code habits interviewers notice (§1.10)

Habit	Instead of
Pass time in (`long nowMinute`, `Instant now`, injected `Clock`)	Reading the clock inside logic
Money as `BigDecimal` from strings, or whole cents in a `long`	`double` (0.1 + 0.2 ≠ 0.3)
`IllegalArgumentException` / `IllegalStateException` / a custom exception	Silent failure or error codes
`Optional` when "not found" is normal	Returning `null`
`private final` fields	Public mutable fields
Domain names (`ParkingTicket`)	`Manager`, `Helper`, `Data2`

Skeleton: **Models → Repositories → Strategies → Services → Driver** (§1.10). The driver is the `main` class that feeds sample commands to the services and prints the results.

In the interview

30-second answer (from §1.14): "I clarify and write 8 to 12 requirements with an out-of-scope list, list actors and use cases, pick classes from the nouns and give each one responsibility, connect them and draw a class diagram, sketch the key flow, run a quick SOLID check and pick one or two patterns where the signal is clear, then code the two or three core flows, handle edge cases and concurrency, and finish with extensions and trade-offs."

Common mistakes (§1.14)

Mistake	Fix
Coding in minute two	8–13 minutes on requirements and actors first
No out-of-scope list	Write it down and agree it
One subclass per value (`CompactSpot`, `LargeSpot`)	Different data → field/enum
God class (`ParkingLotManager`)	One sentence per class; split on "and"
Inheritance for reuse ("Car extends Engine")	Composition
Pattern stuffing	One or two patterns, each with its signal
All twelve classes at equal depth	Two or three flows fully
Ignoring two users on one resource	Lock or atomic operation on the shared state
Non-running code in machine coding	Thin end-to-end path running by halfway

22.3 OOP & relationships cheat sheet

Source: Chapter 2 (OOP) and Chapter 3 (relationships and UML).

The four pillars (§2.3–§2.6)

Pillar	One line	Java tools	Smell when missing
Encapsulation (§2.3)	Data is private; only the owning class's methods change it, and they enforce the rules (the *invariants*: facts that must always hold)	`private` fields, no setters, validation in constructors	Public fields; setters that break invariants
Abstraction (§2.4)	Expose only what callers need; hide how	Interfaces, abstract classes, small public API	Callers reach into internals
Inheritance (§2.5)	Child reuses and specialises parent — **only when "child is a parent" is truly correct**	`extends`, `super`, `@Override`	Subclass refuses or throws on parent methods
Polymorphism (§2.6)	One call, different behaviour per object; removes type switches	Overriding + dynamic dispatch (the method is picked from the object's real class at run time)	`switch` / `instanceof` chains on type

Encapsulation vs abstraction: hiding *data* behind rule-enforcing methods vs hiding *complexity* behind a minimal set of operations (Ch 2 self-check).

Other OOP facts worth one line each

Topic	Remember	Where
Overloading vs overriding	Compile time, by declared parameter types vs run time, by the actual object. Fields and `static` methods are not polymorphic (statics are hidden, not overridden)	§2.7
`this`, `static`, `final`	Current object; one per class; cannot be reassigned/overridden/extended	§2.2
Interfaces	Contracts; can have `default`, `static`, `private` methods; a functional interface has one abstract method (lambdas: short inline functions such as `x -> x * 2`)	§2.8
Enums	Fixed sets with fields and behaviour ("which spot types fit a truck" lives on `VehicleType`)	§2.10
Composition over inheritance	Avoids fragile base classes; allows run-time swapping	§2.11, Ch 5 §6
`equals` / `hashCode`	Equal objects must have equal hash codes; override both or hash collections break	§2.12
Immutability & records	Final class, private final fields, no setters, defensive copies. Records are only **shallowly** immutable — copy collections in the compact constructor	§2.13
Access modifiers	Default to `private`; package-private = same package only	§2.14
Generics	Type-safe reusable infrastructure; **PECS**: producer `extends`, consumer `super`	§2.15
Coupling & cohesion	Low coupling (between classes), high cohesion (inside a class)	§2.16
OOA → OOD → OOP	What the problem is → how, with objects → code. Connections that carry data become objects	§2.17

Abstract class vs interface (§2.9)

	Abstract class	Interface
Instantiable?	No	No
Instance fields (state)	Yes	No (only `public static final` constants)
Constructors	Yes	No
Method bodies	Any	`default`, `static`, `private` only
How many per class	**One** (`extends`)	**Many** (`implements`)
Relationship expressed	"is a" (shared identity and state)	"can do" (capability / role)
Typical use	Shared state + partial implementation for a close family (Template Method)	Contracts, roles, strategies, test seams

Rule of thumb: start with an interface; add an abstract class underneath when implementations share real state or a fixed algorithm (JDK, the Java Development Kit's standard library: `List` + `AbstractList` + `ArrayList`).

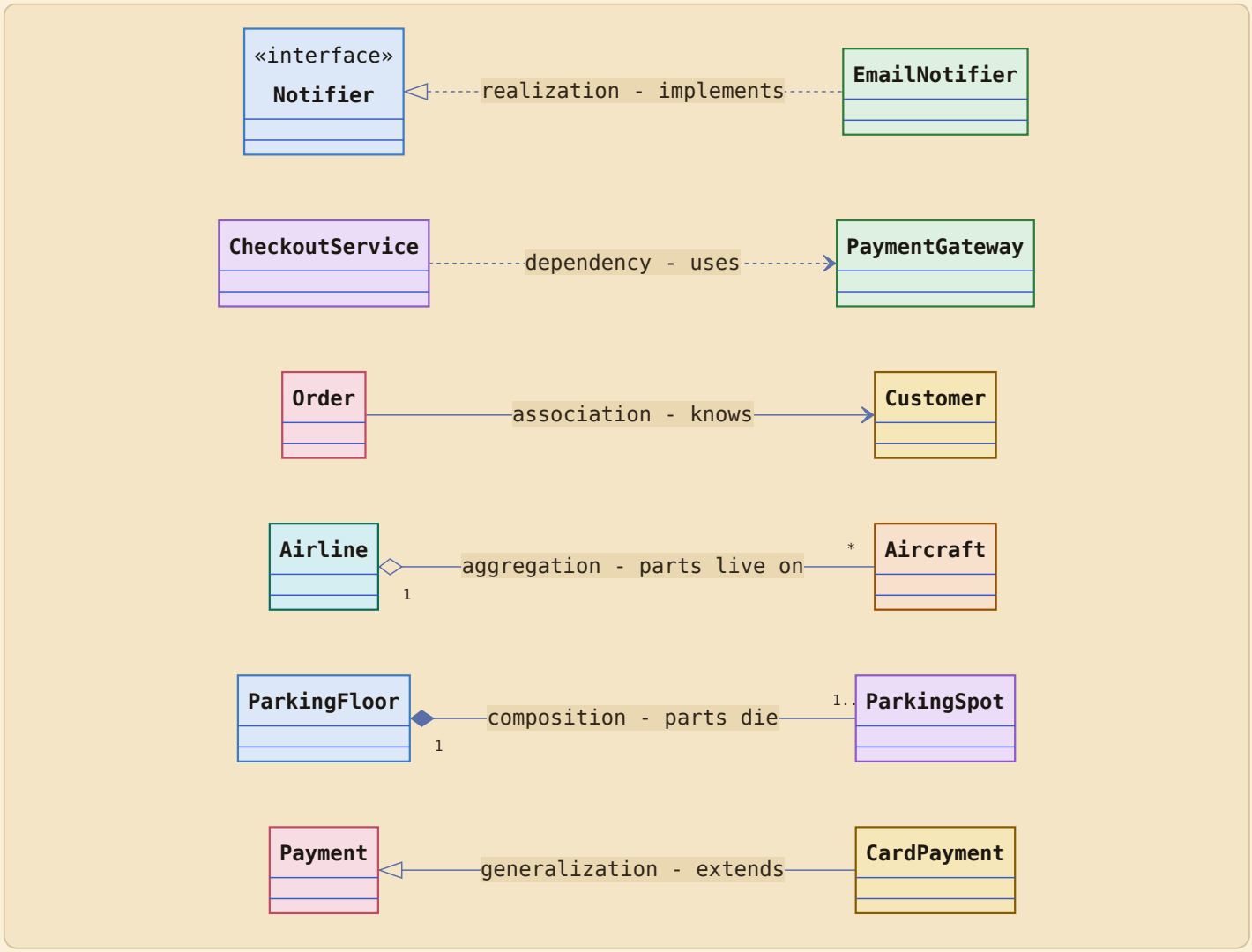
The six relationships (§3.2–§3.8)

Weakest to strongest coupling: **dependency, association, aggregation, composition, realization, generalization** (§3.16).

Relationship	Phrase	UML line	Java shape	Lifetime	Example
Dependency (§3.5)	A *uses* B	dashed, open arrow → B	B as parameter / local / return / static call	none	`CheckoutService` uses `PaymentGateway`
Association (§3.2)	A *knows* B	solid line (arrow for one-way)	field of type B, created elsewhere	independent	`Order` → `Customer`
Aggregation (§3.3)	A *has* B, parts live on	solid, hollow ◇ at A (the whole)	field/collection, parts passed in	B outlives A	`Airline` ◇— `Aircraft`
Composition (§3.4)	A *owns* B	solid, filled ◆ at A (the whole)	A creates B, hides it, B not shared	B dies with A	`ParkingFloor` ◆— `ParkingSpot`
Realization (§3.7)	A *implements* B	dashed, hollow △ at B	`implements`	—	`EmailNotifier` ⇢ `Notifier`
Generalization (§3.6)	A *is a* B	solid, hollow △ at B	`extends`	—	`CardPayment` → `Payment`

Choosing, in order (§3.8): interface A promises to fulfil → realization. Substitutable with no refused methods → generalization. Not kept in a field → dependency. Part that dies with the whole → composition. Part that outlives it → aggregation. Otherwise → association.

UML arrow legend (Mermaid)



Mermaid	Meaning
`A ..> B`	A depends on B
`A --> B`	A associated with B, navigable A to B
`A -- B`	association (both ways / unspecified)
`A o-- B`	aggregation, A is the whole
`A *-- B`	composition, A is the whole
`B <|.. A`	A realizes interface B
`B <|-- A`	A extends B
`A "1" *-- "1..*" B`	with multiplicities

Multiplicity → code (§3.17): `1` required field; `0..1` `Optional`/nullable; `1..*` non-empty collection checked in the constructor; exact numbers checked in constructors. Visibility: `+` public, `-` private, `#` protected, `~` package.

UML diagrams in the interview (§3.9–§3.15)

Diagram	Use it for	Budget in 45 min
Use case (§3.10)	Scope: actors + what each does. «include» = always; «extend» = sometimes	Usually spoken, 3–5 min
Class (§3.11)	**The answer.** 10–15 boxes, every line typed, with multiplicity	Always drawn, 10–15 min
Sequence (§3.12)	Call order between objects (3–4 objects, 6–8 messages)	One behaviour diagram only
Activity (§3.13)	A flow that branches; fork/join for parallel steps	"
State machine (§3.14)	One object's lifecycle; transition label `event [guard] / action`	"

UML has 14 diagram types: 7 structural, 7 behavioural (§3.9).

Common mistakes

- Diamond at the wrong end (it goes at the **whole**); triangle at the wrong end (it goes at the **parent/interface**) (§3.8).
- Lines without type or multiplicity (§3.8).
- Inheritance used to share data; subclasses that only pass a constant to `super` → one class with an enum field (§3.16).

22.4 SOLID one-pager

Source: Chapter 4 (§4.2, §4.7–§4.10, §4.12–§4.15), smells from Chapter 7 (§7.4).

TL;DR

SOLID = five names for the ways code becomes painful to change. **Use them to diagnose, not as a checklist**, apply a fix when a real change asks for it, and always say what the fix costs.

Principle	One-line rule	Smell that signals a violation	Fix	Patterns that help
S — Single Responsibility (§4.2)	A class should have one reason to change (one actor asks for its changes)	The responsibility sentence needs an "and"; *Divergent Change*, *Large Class*	Split by actor (Extract Class); each new class needs a name that stands alone	Command, Proxy, Decorator (each concern in its own class)
O — Open/Closed (§4.7)	Add new behaviour by adding code, not by editing working code	A `switch`/`if` chain over a type; *Repeated Switches*, *Shotgun Surgery*	Put the part that *has* varied behind an interface; add variants as classes (Replace Conditional with Polymorphism)	Strategy, Template Method, Decorator, Factory Method, Observer, Command, Visitor
L — Liskov Substitution (§4.8)	A subtype must be usable anywhere its parent is expected	Subclass throws, does nothing, or refuses a parent method; *Refused Bequest*	Better parent, a narrower interface, or composition (Replace Subclass with Delegate)	Composite, Null Object
I — Interface Segregation (§4.9)	No client should depend on methods it does not use	Empty method bodies or `UnsupportedOperationException` in implementers; fat interfaces	Split into **role interfaces**, one per kind of caller	Adapter
D — Dependency Inversion (§4.10)	Depend on abstractions, not concrete details; the policy owns the abstraction	A high-level class does `new` of (or names) a low-level one; cannot test without the real DB (database)/provider	Inject an interface owned by the high-level class; `main` wires the details	Strategy, Factory Method / Abstract Factory, Adapter, Observer

LSP in detail (§4.8, TL;DR): a subclass must accept at least as much, promise at least as much, keep invariants and history, and throw nothing new. Square/Rectangle is worse than `Penguin.fly()` throwing because it fails **silently** (wrong area, no error).

Five-minute pass over your diagram (§4.13, §1.8)

Run S → O → L → I → D over the diagram you just drew. **Two findings is a good pass; five means rework; zero means you did not look.** S and O earn the most credit: O is what interviewers probe with a late requirement ("what if trucks are charged differently?" → "add a class").

Related ideas (§4.3–§4.6, §4.14)

Idea	Question it asks
SRP	How many actors can ask this class to change?
Cohesion	Do the parts of this class belong together?
Coupling	How much does A depend on B?
Separation of Concerns	Is each area of work in its own module?
SRP vs ISP	SRP splits **classes** (guards the worker); ISP splits **interfaces** (guards the caller)
DIP vs DI vs IoC	DIP = principle (depend on abstractions); DI = technique (pass dependencies in); IoC = the broader idea that a framework calls your code; IoC containers (libraries such as Spring that create and wire objects for you) perform DI

Common mistakes: over-engineering (§4.15)

Warning sign	Principle misapplied	Do instead
`XHelper`, `XUtils`, `XManager2` after a split	SRP	Split by actor; names must stand alone
Six small classes that always change in the same commit	SRP over-split	Merge them back
Interface + factory + registry for one implementation	OCP (*Speculative Generality*)	Write the `if`; refactor at the second or third case (rule of three)
A subclass that "works" by silently doing nothing	LSP	Better parent or composition
One interface per method	ISP over-split	Group by caller into role interfaces
An interface in front of stable `String`-like classes	DIP	Invert only volatile dependencies
A DI container for a 200-line program	DIP	Wire by hand in `main`

YAGNI is the counterweight: SOLID says *what* a good seam looks like; YAGNI says *when* to cut it.

In the interview

Say **observation → cost → fix → price of the fix**, and describe before you label: "`Reporting` implements nine methods but calls two — an interface segregation problem" (§4.13).

22.5 Other design principles and Clean Code rules

Other principles (Chapter 5)

Principle	One-line idea	Smell it fixes	Key Java tool	Where
DRY	Each piece of knowledge has one authoritative home	Duplicated rules drifting apart	Value objects, constants, methods	Ch 5 §1
KISS	Choose the simplest design that works	Over-engineering	Plain code, `switch` expressions	Ch 5 §2
YAGNI	Don't build features before they are needed	Speculative Generality	Delete unused code	Ch 5 §3
Law of Demeter	Only talk to your immediate friends	Train wrecks `a.getB().getC().do()`	Intent-revealing delegate methods	Ch 5 §4
Tell, Don't Ask	Tell objects what to do; don't pull their data out and decide for them	Anemic model, Feature Envy	Command methods on the owner	Ch 5 §5
Composition over inheritance	Prefer "has-a" to "is-a" for reuse	Fragile base class	Forwarding / decorators	Ch 5 §6
Program to an interface	Depend on supertypes, choose the concrete class once at the edge	Hard-wired concrete dependencies	Interfaces + DI	Ch 5 §7
Encapsulate what varies	Isolate the part that changes behind an interface	Growing type switches	Strategy, lambdas	Ch 5 §8
Loose coupling	Interacting objects know little about each other	God orchestrators	Observer, events, DI	Ch 5 §9
Hollywood Principle / IoC	"Don't call us, we'll call you" — the framework owns the flow	Duplicated skeletons	Template Method, callbacks, DI	Ch 5 §10
Separation of Concerns	Each part handles one concern	Spaghetti code	Layers, MVC	Ch 5 §11, §4.5
Least Astonishment	Do what the name promises and nothing more	Hidden side effects	Honest names, `Optional`	Ch 5 §12
Fail fast	Detect problems early and loudly	Distant `NullPointerException`s	Guard clauses, `requireNonNull`	Ch 5 §13
Immutability	Objects never change after creation	Aliasing, races	Records, `List.copyOf`	Ch 5 §14
Design by Contract	Preconditions (caller), postconditions (method), invariants (always)	Magic error values	Checks at method entry/exit	Ch 5 §15
GRASP	Nine guidelines for "which class should do this?"	God classes	Expert, Creator, Controller…	Ch 5 §16
Domain-Driven Design	Model the business: bounded contexts, entities vs value objects, small aggregates, domain events, repositories per root	Anemic model, big ball of mud, primitive obsession	Records as value objects, aggregate roots, events + outbox (events saved in the same transaction as the data, published afterwards)	Ch 5 §17

GRASP in one line each (Ch 5 §16.1–§16.9):

Guideline	Rule
Information Expert	The class with the data does the work
Creator	The container (or the one with the initial data) creates
Controller	A thin non-UI object receives each system event and delegates
Low Coupling	Choose the design with fewer, more stable dependencies
High Cohesion	Each class does one related set of things
Polymorphism	Let each type implement the varying behaviour; don't switch on types
Pure Fabrication	Invent a helper class (e.g. a repository) when no domain class is a clean home
Indirection	Put a middleman between things that shouldn't know each other
Protected Variations	Wrap what's unstable behind a stable interface

Tensions to mention (Ch 5 summary): DRY vs loose coupling (sharing code couples its users); YAGNI vs encapsulate-what-varies (abstract only real or clearly predicted variation); Law of Demeter vs cohesion (too many delegates → Middle Man); Information Expert vs SoC (domain objects compute, fabrications persist).

Clean Code rules checklist (Chapter 6)

Area	Rule of thumb	Where
Names	Reveal intent; pronounceable; searchable; one word per concept; code reads like the requirement	Ch 6 §1
Functions	Small; do one thing; one level of abstraction; top-down (stepdown); ≤ 2–3 arguments; **no flag arguments**; no hidden side effects; commands *or* queries (CQS); exceptions, not error codes	Ch 6 §2
Comments	Say it in code; comment only the *why*, warnings, legal text, public contracts; delete commented-out code	Ch 6 §3
Formatting	Newspaper order (headline first); related things close; one automated team style	Ch 6 §4
Objects vs data structures	Objects hide data and expose behaviour; data structures expose data; no hybrids; no train wrecks	Ch 6 §5
Error handling	Unchecked exceptions with context; wrap vendor exceptions; empty collections / `Optional` / special case instead of `null`; never pass `null`; always clean up	Ch 6 §6
Boundaries	Keep third-party code behind your own interfaces; learn it with tests	Ch 6 §7
Unit tests	Three laws of TDD (Test-Driven Development: write a failing test first); one concept per test; Given–When–Then; **F.I.R.S.T.** (Fast, Independent, Repeatable, Self-validating, Timely)	Ch 6 §8
Classes	Small by responsibility; cohesive; open for extension; depend on abstractions	Ch 6 §9
Systems	Build objects in one place (composition root) and inject them; cross-cutting concerns in proxies/aspects	Ch 6 §10
Simple design	Beck: passes the tests → reveals intention → no duplication → fewest elements (*Clean Code* swaps the middle two)	Ch 6 §11
Concurrency	Separate concurrency code; share as little as possible; use the library; keep locks small; plan shutdown	Ch 6 §12
Smells	The C/E/F/G/J/N/T heuristics catalogue as a review vocabulary	Ch 6 §14
Code review	Correctness → design → readability → tests → security → performance → operability; automate style; small PRs (pull requests: proposed changes sent for review)	Ch 6 §15
Error strategy	Classify, translate at boundaries, retry only transient + idempotent (safe to repeat) with backoff (waiting longer between attempts), one global handler that logs once	Ch 6 §16
Testing strategy	Many unit, some integration, few E2E (end-to-end, whole-system); right test double (a stand-in such as a mock or fake); verify state over interactions; don't mock what you don't own	Ch 6 §17

Boy Scout Rule: leave the code a little cleaner than you found it (Ch 6 intro).

22.6 Code smells → refactorings (all 24)

Source: Chapter 7 (§7.2 smells, §7.3 catalogue, §7.4 lookup table). ***Bold*** *= the usual first move.*

#	Smell	Signal (one line)	Group	Refactorings that cure it
1	Mysterious Name (§7.2.1)	Name doesn't say what it does/holds (`data`, `proc()`, `Manager`)	2nd-ed extra	**Change Function Declaration**, Rename Variable, Rename Field
2	Duplicated Code (§7.2.2)	Same structure in more than one place	Dispensables	**Extract Function**, Slide Statements, Pull Up Method
3	Long Function (§7.2.3)	You want a comment to explain a block	Bloaters	**Extract Function**, Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop
4	Long Parameter List (§7.2.4)	Many parameters, same types, boolean flags	Bloaters	Replace Parameter with Query, **Preserve Whole Object**, **Introduce Parameter Object**, Remove Flag Argument, Combine Functions into Class
5	Global Data (§7.2.5)	Readable and writable from anywhere (incl. mutable singletons)	2nd-ed extra	**Encapsulate Variable** (then shrink its scope)
6	Mutable Data (§7.2.6)	Shared data changed after creation	2nd-ed extra	**Encapsulate Variable**, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class/Transform, Change Reference to Value
7	Divergent Change (§7.2.7)	One class changed for many different reasons (SRP)	Change Preventers	**Split Phase**, Move Function, Extract Function, **Extract Class**
8	Shotgun Surgery (§7.2.8)	One change forces many small edits in many classes	Change Preventers	**Move Function**, **Move Field**, Combine Functions into Class/Transform, Split Phase, Inline Function, Inline Class
9	Feature Envy (§7.2.9)	Method uses another class's data more than its own	Couplers	**Move Function**, Extract Function
10	Data Clumps (§7.2.10)	Same group of values always travel together (`startDate, endDate`)	Bloaters	**Extract Class**, **Introduce Parameter Object**, Preserve Whole Object
11	Primitive Obsession (§7.2.11)	Money as `double`, status as `String`	Bloaters	**Replace Primitive with Object**, Replace Type Code with Subclasses, Replace Conditional with Polymorphism, Extract Class, Introduce Parameter Object
12	Repeated Switches (§7.2.12)	The *same* switch on a type code in several places	OO Abusers	**Replace Conditional with Polymorphism**
13	Loops (§7.2.13)	Classic loop that filters/maps/aggregates	2nd-ed extra	**Replace Loop with Pipeline**
14	Lazy Element (§7.2.14)	A class/function that doesn't pull its weight	Dispensables	**Inline Function**, **Inline Class**, Collapse Hierarchy
15	Speculative Generality (§7.2.15)	Hooks "we might need someday"; abstract class with one subclass	Dispensables	Collapse Hierarchy, Inline Function, Inline Class, Change Function Declaration, Remove Dead Code

#	Smell	Signal (one line)	Group	Refactorings that cure it
16	Temporary Field (§7.2.16)	Field set only in some situations, `null` otherwise	OO Abusers	**Extract Class**, Move Function, Introduce Special Case
17	Message Chains (§7.2.17)	`order.getCustomer().getAddress().getCity()`	Couplers	**Hide Delegate**, Extract Function, Move Function
18	Middle Man (§7.2.18)	Most methods just forward to another class	Couplers	**Remove Middle Man**, Inline Function, Replace Superclass/Subclass with Delegate
19	Insider Trading (§7.2.19)	Two modules poke at each other's internals	Couplers	**Move Function**, **Move Field**, Hide Delegate, Replace Subclass/Superclass with Delegate
20	Large Class (§7.2.20)	Too many fields/methods — a God Class	Bloaters	**Extract Class**, Extract Superclass, Replace Type Code with Subclasses
21	Alternative Classes with Different Interfaces (§7.2.21)	Same job, different method names (`sendMail` vs `dispatch`)	OO Abusers	**Change Function Declaration**, Move Function, Extract Superclass
22	Data Class (§7.2.22)	Only fields and getters/setters; others manipulate it	Dispensables	**Encapsulate Record**, Remove Setting Method, Move Function, Extract Function, Split Phase
23	Refused Bequest (§7.2.23)	Subclass doesn't want inherited methods, or throws on them (LSP)	OO Abusers	Push Down Method/Field, **Replace Subclass with Delegate**, **Replace Superclass with Delegate**
24	Comments (§7.2.24)	Comment used as deodorant for unclear code	Dispensables	**Extract Function**, Change Function Declaration, Introduce Assertion

Smell family → principle violated (§7.4)

Smell family	Principle
Divergent Change, Large Class	SRP
Repeated Switches, Shotgun Surgery	OCP
Refused Bequest	LSP
Long Parameter List with flags, fat interfaces	ISP / clean function design
Global Data, Mutable Data	Encapsulation
Hard-wired `new` of concrete dependencies	DIP
Duplicated Code	DRY
Speculative Generality	YAGNI
Message Chains, Insider Trading	Law of Demeter
Feature Envy, Data Class	Tell, Don't Ask

Refactoring toward patterns (§7.6)

Smell	Refactor toward
Switch/if-chain choosing an algorithm	Strategy
Methods that check and change a status field	State
`if (x == null)` at many call sites	Null Object / Special Case
Sibling subclasses with the same skeleton	Template Method
Many confusing constructors	Factory Method / creation methods
Complex object built by many ordered calls	Builder
Undo/queueing needed; huge function with shared locals	Command
Logging/caching mixed into a core class	Decorator
Objects polling for changes; hard-coded notification	Observer
Two classes, same job, different interfaces	Adapter
Subclass explosion from feature combinations	Strategy / Decorator / Bridge
A pattern that no longer earns its keep	Remove it (Inline Class, Collapse Hierarchy)

Refactoring = changing structure without changing behaviour, in small steps, with tests as the safety net. Wear one of **two hats** at a time: adding function or refactoring (§7.1.1).

22.7 Design patterns: all 23 GoF + extras

Source: Chapters 8–11. Catalogue in §8.3; comparisons in §8.15, §9.8, §10.7, §11.6, §11.8.

Count: 5 creational + 7 structural + 11 behavioral = **23**. **Class-scoped** (inheritance) patterns: Factory Method, class Adapter, Template Method, Interpreter; almost everything else is object-scoped (composition) (§8.3).

Creational (Chapter 8)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Singleton	Exactly one instance with a global access point	There really must be exactly one (config, pool, registry)	`Runtime.getRuntime()`; Spring (a popular Java framework) singleton-scoped beans	A global variable; a DI singleton scope (preferred for testing)	§8.8
Factory Method	Subclasses decide which concrete class to create via an overridable method	Shared procedure, varying product; others will add types	`Collection.iterator()`	Simple Factory (one method with a switch)	§8.10
Abstract Factory	Create *families* of related objects without naming concrete classes	Products must match; swap a whole set at once	`DocumentBuilderFactory`; JDBC (Java's database API) `Connection`	Factory Method (one product by inheritance vs a family by composition)	§8.11
Builder	Build a complex object step by step; one validating `build()`	Many optional fields, cross-field validation, immutable result	`StringBuilder`, `HttpClient.newBuilder()`	A plain constructor (fine for a handful of required fields); telescoping constructors	§8.12
Prototype	Create new objects by copying a configured instance	Setup is expensive, or only the interface is known	Copy constructors, `ArrayList.clone()`	Memento (a snapshot to restore, not a copy to use)	§8.13

Singleton notes (§8.8, §8.17): eager, holder-class or `enum` (most robust: blocks duplicates made through reflection, i.e. code calling private constructors by name, or through serialization, i.e. saving to bytes and reading back); double-checked locking needs a `volatile` field. It is global state that hurts testing — prefer injecting one shared instance. Many case studies reject it (Ch 14 self-check).

Structural (Chapter 9)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Adapter	Convert one interface into another that clients expect	"The vendor API doesn't match ours" (after the fact)	`Arrays.asList`, `InputStreamReader`	Bridge (same shape, but Bridge is designed up front); Facade	§9.1
Bridge	Split abstraction from implementation so both vary independently	Class names with two nouns; two dimensions multiplying classes	JDBC `Connection` + drivers	Adapter; Strategy (same shape)	§9.2
Composite	Treat single objects and trees uniformly	Part-whole trees (files, menus, UI)	`java.awt.Container` is a `Component` holding `Component`s	Decorator (also wraps a component, but adds behaviour to one)	§9.3
Decorator	Add responsibilities dynamically by wrapping; same interface, stackable	"Sometimes logging, sometimes caching, sometimes both"	`BufferedInputStream`, `Collections.synchronizedList`	Proxy (same shape; Proxy *controls access*); Chain of Responsibility (in CoR someone can stop the flow)	§9.4
Facade	One simple entry point to a complex subsystem	Every caller repeats the same multi-step sequence	SLF4J (a logging facade library) `LoggerFactory`, service layers	Adapter (converts one object's interface); Mediator	§9.5
Flyweight	Share intrinsic (immutable) state across huge numbers of objects	Millions of near-identical objects	`Integer.valueOf`, `String.intern()`	Object Pool (reuses objects one borrower at a time; flyweights are shared at once)	§9.6
Proxy	A stand-in that controls access (lazy, remote, protective, caching)	It's expensive, remote or restricted	`java.lang.reflect.Proxy`, Spring `@Transactional`, Hibernate (maps objects to database tables) lazy load	Decorator	§9.7

Memory hooks (§9.8): Adapter *make it fit* · Decorator *add to it* · Proxy *guard it* · Facade *simplify it* · Bridge *split it* · Composite *nest it* · Flyweight *share it*.

Behavioral (Chapters 10–11)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Chain of Responsibility	Pass a request along handlers until one handles (or stops) it	Several checks, any may reject; handler not known in advance	Servlet `Filter` chain; `java.util.logging` handlers	Decorator (every layer always runs)	§10.1
Command	Turn a request into an object: queue, log, retry, undo	Undo/redo, job queues, scheduling	`Runnable` / `Callable`; Swing `UndoManager`	Strategy (Strategy = *how*; Command = *what*)	§10.2
Iterator	Traverse a collection without exposing how it is stored	Walk a collection/tree/paged API	`Iterator`, `Spliterator`, JDBC `ResultSet`	Visitor (what to do at each node)	§10.3
Mediator	Centralise communication between peers in one coordinator	Many objects interact in tangled ways; order of reactions matters	Chat servers, UI controllers; `Executor` / `Timer` are a loose fit (§10.4)	Observer (broadcast, no coordination); Facade	§10.4
Memento	Capture and restore state without breaking encapsulation	Undo, checkpoints, rollback	A `record` snapshot; DB `ROLLBACK`, Git commits (real-world)	Command undo (store actions, not states); Prototype	§10.5
Interpreter	One class per grammar rule; evaluate sentences of a small language	User-written rules/expressions	`java.util.regex.Pattern`, SpEL (Spring's expression language)	Composite (same tree); Visitor (many operations over the AST, the abstract syntax tree a parser builds)	§10.6
Observer	Notify many dependents automatically when one object changes	An open-ended set of independent listeners	`PropertyChangeListener`, `java.util.concurrent.Flow`, Spring `@EventListener`	Mediator; pub-sub (Observer through a broker, §20.6)	§11.1
State	Behaviour changes with internal state, as if the class changed	Many states/transitions; methods repeat a `switch` on status	TCP connection (GoF's example), order lifecycles. `Thread.State` is only an enum of states, not the pattern	Strategy (same diagram; states know each other and advance themselves)	§11.2
Strategy	Make a family of algorithms interchangeable behind one interface	Several ways to do one job, chosen at run time	`Comparator`; `RejectedExecutionHandler` policies	State; Template Method; Decorator (Strategy *replaces* a rule, Decorator *wraps* one)	§11.3

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Template Method	Fixed algorithm skeleton in a base class; subclasses fill in steps	Long fixed flow, a few varying steps	`AbstractList`, `InputStream.read(byte[],…)`, `HttpServlet.service()`	Strategy (Template = inheritance, one step varies; Strategy = composition, whole algorithm varies)	§11.4
Visitor	Add operations to a stable set of classes without changing them (double dispatch)	Many new operations over a fixed type set (ASTs)	`javax.lang.model.element.ElementVisitor`; `FileVisitor` is named after it but is really a tree-walk callback	Iterator; in Java 21, **sealed types + pattern-matching `switch`** usually replace it	§11.5

Beyond GoF: extras you will meet

Pattern	Intent (one line)	Use when	Java / real example	Commonly confused with	Where
Simple Factory (idiom)	One `create(type)` method maps a type value to a class	Small, stable set of types chosen by config	`Calendar.getInstance()`, `EnumSet.noneOf`	Factory Method	§8.9
Object Pool	Borrow and return a bounded set of expensive objects	DB connections, limited resources	HikariCP (a database connection pool), `ThreadPoolExecutor`	Flyweight	§8.14
Null Object	A do-nothing implementation instead of `null`	`null` checks everywhere for an optional collaborator	`Collections.emptyList()`, `OutputStream.nullOutputStream()`	`Optional` (for lookups)	§11.7.1
Dependency Injection	Receive collaborators (as interfaces) through the constructor	Classes create their own collaborators; hard to test	Spring constructor injection; a hand-wired `main`	Service Locator (anti-pattern: the class asks for its dependencies)	§11.7.2
Repository	Collection-like interface over storage	Business logic mixed with storage code	Spring Data `CrudRepository`	DAO (table-oriented)	§11.7.3
Specification	A named, combinable business rule (`isSatisfiedBy`, and/or/not)	Same rule in queries and validation; finder-method explosion	`Predicate`; Spring Data JPA `Specification`	—	§11.7.4
MVC	Separate model, view and controller (Observer + Strategy + Composite)	UI, input handling and rules tangled	Spring MVC; Swing `JTable` + `TableModel`	MVP, MVVM (relatives)	§11.7.5
Event Sourcing (lite)	Store the events; derive current state by replaying them	Full history, audit, time travel	Append-only logs	Plain audit log	§11.7.6

"Which pattern?" by problem symptom

Combined from §8.5, §8.15, §9.8, §11.8 and §7.6.

If you see / need…	Use	Not to be confused with
One operation, several interchangeable rules chosen at run time (pricing, matching, routing)	**Strategy**	State, Template Method
The class to create depends on a type value; caller shouldn't know the concrete class	**Factory** (Simple Factory / Factory Method)	Abstract Factory
Products must always match as a family	**Abstract Factory**	Factory Method
Constructor has ~6+ arguments, many optional	**Builder**	—
Another one of *this*, cheaply or without knowing its class	**Prototype**	Memento
Exactly one instance must exist	**Singleton** (or DI singleton scope)	A convenient global handle
Expensive objects created and destroyed repeatedly	**Object Pool**	Flyweight
Existing class has the wrong interface	**Adapter**	Facade, Bridge
Two dimensions multiplying classes (`WindowsButton`, `MacButton`, …)	**Bridge**	Adapter
Tree of part-whole objects treated the same	**Composite**	Decorator
Add behaviour in combinations without editing the class	**Decorator**	Proxy
Every caller repeats the same five steps into a subsystem	**Facade**	Mediator
Millions of near-identical objects	**Flyweight**	Object Pool
Lazy load, access control, remote, caching (same interface)	**Proxy**	Decorator
A request must pass several checks; any may reject it	**Chain of Responsibility**	Decorator (all layers run)
Undo/redo, queue, schedule, retry or log *requested actions*	**Command**	Strategy
Evaluate user-written rules in a small language	**Interpreter** (+ Specification)	Visitor
Walk a collection/tree/paged API without exposing structure	**Iterator**	Visitor
Many objects depend on each other and the **order** of reactions matters	**Mediator**	Observer
Roll back to an earlier state without exposing internals	**Memento**	Command undo
One change must reach an open-ended set of listeners	**Observer** (or a broker)	Mediator
Three or more methods repeat one `switch` over a status; legal actions depend on it	**State** (or enum + guarded method for 3–4 states)	Strategy
Several classes share a long fixed flow, differ in a few steps	**Template Method**	Strategy
Many new operations over a stable set of types	**Visitor**, or sealed types + pattern matching	Iterator
`null` checks everywhere	**Null Object** / `Optional`	—
Classes create their own collaborators	**Dependency Injection**	Service Locator
Business logic mixed with storage	**Repository**	DAO
Same rule duplicated in queries and validation	**Specification**	—

Three questions for the famous look-alikes (§11.8):

1. Do the alternatives reference each other? Yes → **State**. No → **Strategy** (or Bridge if chosen once at wiring).
2. Can one participant stop the flow? Yes → **Chain of Responsibility**. Everyone always runs → **Decorator**. Everyone is told, nobody stops anyone → **Observer**.

3. Does a central object decide what happens next? Yes → **Mediator**. Each listener decides for itself → **Observer**.

The six that come up in almost every round (§1.9, §8.5)

Pattern	Signal	Do not use when
Strategy	One operation, several swappable rules	Only one rule exists and nothing hints at a second
Factory	Creation depends on a type value	Only one concrete class exists
Observer	Several listeners the source should not know	A single listener the source already references
State	Behaviour depends on status; many transitions	3–4 states handled by one guarded method
Singleton	Genuinely exactly one	You just want a global handle
Builder	Many optional fields; long constructor	A handful of required fields

Common mistakes (§8.6)

Anti-pattern	Fix
God Object (one class does everything)	Split by responsibility; push behaviour to the data owners
Golden Hammer (favourite pattern everywhere)	Start from the problem's signal
Pattern overuse ("patternitis") — factories for one class, single-implementation interfaces	YAGNI: add the pattern when the second variant arrives
Singletonitis (singletons as globals)	Dependency injection
Cargo-cult patterns (structure without intent)	Name the property the pattern gives you, or drop it

In the interview

Use **one or two** patterns in a 45-minute design, placed where their signal appears, and **name one you rejected** and why (§8.5, §8.16).

22.8 Concurrency cheat sheet

Source: Chapter 12 (concepts) and Chapter 13 (classic problems).

TL;DR (§12.28)

Races come from **shared mutable state**. Make the critical section atomic, or remove sharing/mutability. Visibility needs **happens-before** edges. Prevent deadlock with **lock ordering** or **tryLock + back-off**. In LLD: name the contended resource, keep **check + claim atomic**, lock fine-grained and in order, and say what changes with many servers.

Primitive | use for | gotcha

Primitive	Use for	Gotcha	Where
`synchronized`	Making a multi-step update atomic (mutex + visibility)	Don't lock on a changing reference, a boxed value or a string literal; no slow I/O while holding it; readers must use the same lock	§12.7
`ReentrantLock`	Same as `synchronized` plus try/timed/fair locking and `Condition`s	Call `lock()` **just before** `try`, `unlock()` in `finally`; fairness is noticeably slower	§12.8
`tryLock(timeout)`	Give up if the lock is busy; deadlock prevention with back-off	Never `unlock()` when `tryLock` returned `false`; don't retry in a tight loop (livelock)	§12.9
`ReentrantReadWriteLock`	Many readers, few writers	Cannot upgrade read → write (self-deadlock); don't modify under the read lock; writer starvation in non-fair mode	§12.10
`StampedLock`	Read-mostly data with optimistic reads	**Not reentrant**; `validate` the stamp before using values read optimistically	§12.10
`volatile`	Visibility of a flag or a published immutable snapshot	Gives visibility, **not** atomicity: `count++` is still a race	§12.6
`AtomicInteger` / `AtomicLong` / CAS	Lock-free counters and single-variable claims	Separate `get()` then `set()` is still a race — use `compareAndSet` / `updateAndGet`; no side effects in the lambda (may run several times); ABA → `AtomicStampedReference`	§12.15
`LongAdder`	Very hot counters (metrics)	Use instead of `AtomicLong` when contention is high	§12.15
`Semaphore(N)`	Limit concurrent access to N; signalling between threads	Release in `finally`; don't release when `acquire` never succeeded	§12.12
`wait/notifyAll`, `Condition`	Waiting for a condition on shared state	Always `while`, never `if` (spurious wakeups); hold the monitor; `notify()` can wake the wrong thread; state change without notify = lost wakeup	§12.13
`CountDownLatch`	Wait for N events once	One-shot; `countDown()` in `finally`, `await(timeout)`	§12.14
`CyclicBarrier` / `Phaser`	N threads meet at each phase (reusable)	Fewer threads than parties → everyone waits forever	§12.14
`ConcurrentHashMap`	Shared map	`containsKey` then `put` is a race — use `putIfAbsent` / `computeIfAbsent` / `merge`; no slow work or recursive updates inside `compute`; no `null` keys/values	§12.16
`CopyOnWriteArrayList`	Read-mostly lists (listener lists)	Every write copies the array	§12.16
`BlockingQueue` (bounded)	Handing work between threads (producer–consumer)	Unbounded queue → no back-pressure → memory blow-up; one poison pill per consumer	§12.22
`ExecutorService` / `ThreadPoolExecutor`	Running tasks on a bounded pool	Call `shutdown()`; `newFixedThreadPool` has an **unbounded** queue; exceptions in `submit`ted tasks are lost unless you call `get()`; tasks waiting on tasks in the same pool → thread-starvation deadlock	§12.4, §12.21
Virtual threads	Many blocking I/O tasks	Pinning (Java 21–23: blocking inside `synchronized`); heavy `ThreadLocal`s waste memory	§12.4, §12.24

Primitive	Use for	Gotcha	Where
`ForkJoinPool` / parallel streams	Splitting CPU work	Use a threshold: tiny tasks cost more to schedule than to run; subtasks must touch disjoint data	§12.26, §13.10
`CompletableFuture`	Combining async results (`thenApply`, `thenCombine`, `allOf`)	Handle errors with `exceptionally` rather than calling `get()` at every step	§12.4
`ThreadLocal`	Per-thread state (thread confinement)	Clear it (`remove()`) in pooled threads, or values leak into the next task	§12.24
Immutability / confinement	Avoiding synchronization entirely	`final List` is not immutable if the list is mutable and exposed	§12.24

Which `java.util.concurrent` class for which job (§12.26)

I need to…	Use
Increment a shared counter	`AtomicInteger` / `AtomicLong`; `LongAdder` if very hot
Stop a loop from another thread	`volatile boolean` flag or `interrupt()`
Make a multi-step update atomic	`synchronized` or `ReentrantLock`
Give up if the lock is busy	`tryLock(timeout)`
Many readers, few writers	`ReentrantReadWriteLock`, `StampedLock`, or an immutable snapshot in a `volatile` field
Limit concurrent access to N	`Semaphore(N)`
Wait for a condition on shared state	`Lock` + `Condition` (or `wait/notifyAll`) in a `while` loop
Wait for N events once	`CountDownLatch`
N threads meet each phase	`CyclicBarrier` / `Phaser`
Hand work between threads	Bounded `BlockingQueue`
Shared map	`ConcurrentHashMap` with `putIfAbsent` / `compute` / `merge`
Read-mostly list	`CopyOnWriteArrayList`
Run tasks	`ExecutorService` (bounded); virtual threads for blocking I/O
Split CPU work	`ForkJoinPool` / parallel streams
Combine async results	`CompletableFuture`
Avoid synchronization entirely	Immutability, thread confinement

Deadlock checklist (§12.17–§12.19)

A deadlock needs **all four Coffman conditions**; break any one and it cannot happen.

#	Condition	How to break it
1	Mutual exclusion	Lock-free/immutable data, shareable resources (read locks)
2	Hold and wait	Acquire everything at once, or `tryLock` and release everything on failure
3	No preemption	Timeouts (`tryLock(timeout)`), leases that expire
4	Circular wait	**Global lock ordering** (always lock the lower id first)

- [ ] Do I ever hold two locks? If yes, is there **one global order** (account id, seat id), with a tie-breaker for equal keys?

- [ ] Do I call a **listener, callback or another object's synchronized method while holding my lock**? Move it outside the lock.
- [ ] Do tasks in a bounded pool **wait for other tasks in the same pool**? (thread-starvation deadlock)
- [ ] Are retries randomised (**jitter**) so two threads don't back off in lockstep? (livelock, §12.18)
- [ ] Could one thread wait forever while others progress? Consider fairness (§12.19).
- [ ] Detection: a thread dump (`jstack` / `jcmd`) prints "Found one Java-level deadlock"; `ThreadMXBean.findDeadlockedThreads()`.

Deadlock vs livelock vs starvation: nobody progresses and all wait / everybody runs but nobody progresses / some progress while one never does (§12.27).

The LLD concurrency recipe (§12.25)

1. **Name the shared mutable state** (free spots, seat → owner, stock).
2. **State the invariant** ("a seat has at most one booking", "stock ≥ 0").
3. **Find the critical operation** — usually a check-then-act (*find free → assign*).
4. **Pick the smallest tool that makes it atomic:** one item → `queue.poll()`, `putIfAbsent`, `compareAndSet`; several items all-or-nothing → per-item locks **in sorted order**; across servers → DB constraints/transactions, optimistic versioning, or a distributed lock.
5. **Choose granularity:** per floor / show / SKU, not one global lock.
6. **Handle time:** holds with expiry, timeouts, idempotent retries.

Classic problems (§13.12)

Problem	Key primitive	Invariant to state
FooBar (§13.1)	2 semaphores	Exactly one permit exists across both
Zero Even Odd (§13.2)	3 semaphores	Zero dispatches; one permit circulates
FizzBuzz (§13.3)	monitor + `notifyAll`	Predicates partition 1..n
H2O (§13.4)	2 semaphores + barrier	≤ 2 H and ≤ 1 O per round
Dining Philosophers (§13.5)	ordered locks / n−1 semaphore	No circular wait
TTL (time to live) cache (§13.6)	`ConcurrentHashMap` + `compute` + conditional remove	Never return or keep expired data
Striped map (§13.7)	lock array	Each bucket belongs to exactly one stripe
Blocking queue (§13.8)	lock + 2 conditions	0 ≤ count ≤ capacity; wait in loops
Bloom filter (§13.9)	atomic OR	Bits only go 0 → 1; no false negatives
Merge sort (§13.10)	Fork/Join	Subtasks touch disjoint ranges
Token-bucket rate limiter (§13.11)	lock or CAS on immutable state	Refill + take is one atomic step

Common mistakes (§12.25 and throughout Chapter 12)

- A global `synchronized` on `ParkingLot.park()` — correct, but serialises every gate.
- Checking availability with one call and booking with another (check-then-act).
- Locking seats in the order the user clicked them (deadlock between two multi-seat bookings).
- Forgetting that in-memory locks don't work across multiple app servers.
- Swallowing `InterruptedException` — rethrow, or restore the flag with `Thread.currentThread().interrupt()` (§12.3).
- Calling `run()` instead of `start()` (§12.3).

22.9 Case-study map

Source: Chapters 14–21. "Concurrency concern" is the race each chapter guards against, and "atomic step" is how.

Everyday objects (Chapter 14)

Problem	Key entities	Patterns used	Concurrency concern	Where
Parking lot	`ParkingLot`, `ParkingFloor`, `ParkingSpot`, `Vehicle`, `Ticket`, `DisplayBoard`, `SpotType`/`VehicleType` enums	Strategy (pricing, spot-finding), Factory (electric spots), Observer (boards); Singleton rejected	Two gates take one spot → atomic `pollFirst` + CAS; per-ticket lock for payment; `putIfAbsent` for plates	§14.1
Vending machine	`VendingMachine`, `Slot`, `Inventory`, `CashBox`, `Denomination`, `VendingState` + 5 states	**State** (the whole design)	`synchronized` machine; change-making by bounded coin-change DP (dynamic programming: build the answer from smaller amounts), not greedy (always take the biggest coin)	§14.2
Coffee machine	`CoffeeMachine`, `Recipe`, `RecipeBook`, `IngredientStore`, `OrderResult` (sealed)	Observer (low stock), recipes as data	Two orders take the last milk → one `ReentrantLock` around check + deduct; notify outside the lock	§14.3
ATM	`Atm`, `AtmState` + 4 states, `Transaction` (sealed), `NoteHandler`, `CashDispenser`, `BankService`, `Account`	State (session), Command (transactions), Chain of Responsibility (note cassettes)	Overdraw from two ATMs → per-account lock; transfers lock in id order	§14.4
Elevator	`Elevator`, `StopList` (two sorted sets), `Door`, `Dispatcher`, `DispatchStrategy`, `Building`	Strategy (dispatch) + modes; LOOK scheduling	Per-car lock, synchronized dispatcher; tick-driven `step()`	§14.5
Traffic signal	`Intersection`, `TrafficLight`, `Phase`, `SignalState` + 5 states, `TimingPolicy`, `SafetyMonitor`	State (timed), Strategy (timing), Observer (lights)	Synchronized tick; safe with a timer thread plus emergency threads	§14.6
Shipping locker	`LockerStation`, `Compartment`, `Booking`, `CompartmentSelector`, `PickupResult` (sealed), `LockerListener`, `LockerNetwork`	Strategy (door choice), Observer (SMS text messages), enum state machines, sealed results	Two carriers, one door → station lock around choose-and-mark; lazy expiry of holds and pickup windows; hashed single-use codes, keypad lockout	§14.7

Booking & inventory (Chapters 15–16)

Problem	Key entities	Patterns used	Concurrency concern	Where
Library	`Book` (title) vs `BookItem` (copy), `Member`, `BookLending`, `LendingService`, `Catalog`	Strategy (fines), notifier interface; FIFO (first in, first out) reservation queue per title	Last copy lent twice → lock member then copy (fixed order)	§15.1
Movie tickets (BookMyShow)	`Show`, `ShowSeat`, `Booking`, `SeatLockProvider`, `BookingService`, `CancellationPolicy`	Strategy (refund rule), gateway behind an interface	Book `ShowSeat`s; all-or-nothing hold with TTL → pay → atomic commit; refund if hold lost	§15.2
Concert tickets	`SeatedInventory`, `GaPool`, `SeatState` (sealed), `Hold`, `VirtualWaitingRoom`, `PurchaseLimiter`	Sealed seat states, waitlist	Extreme contention → CAS per seat, CAS counter for GA (general admission, unseated), FIFO waiting room, per-user limits, TTL holds with a reaper	§15.3
Hotel	`Hotel`, `Room`, `RoomBooking`, `Invoice`, `RoomCharge`, `Payment` subclasses, `FrontDesk`	Template Method (`RoomCharge`)	Room × night reserved atomically in `synchronized Room.tryReserve`	§15.4
Car rental	`Vehicle`, `DateRange`, `Reservation`, `Bill`/`BillItem`, `ReservationAddOn`, `PricingStrategy`	Template Method (add-ons), Strategy + Decorator (pricing), state machine	Availability is about **dates** → `Vehicle.tryBook(range)` synchronized	Ch 16 · Car Rental
Restaurant	`Menu`/`MenuSection`/`MenuItem`, `Table`, `Reservation`, `Order`, `Meal`/`MealItem`, `Kitchen`, `Check`	Observer (order ready), Strategy (chef assignment, payment), state machine	Two receptionists, one table → `Table.tryHold(time)` synchronized; listeners after the lock	Ch 16 · Restaurant
Airline	`Flight` vs `FlightInstance`, `Seat` vs `FlightSeat`, `FlightReservation`, `Itinerary`, `FareStrategy`	Strategy + Decorator (fares), Facade	Selling one seat twice → `FlightInstance.holdSeats` all-or-nothing under the instance lock	Ch 16 · Airline
Course registration	`Course` vs `Section`, `Student`, `RegistrationRule`s, `Registrar`	Chain of rules / Specification	Overselling a section → section lock inside a student lock (fixed order)	Ch 16 · Course Registration

Games (Chapter 17)

Problem	Key entities	Patterns used	Concurrency concern	Where
Blackjack + deck	`Card`, `Deck<T>`, `Shoe`, `Hand`, `BlackjackScoring`, `PlayerStrategy`, `DealerPolicy`	Strategy (player, dealer); injected randomness	One table = one thread; shared balances need an atomic wallet service	Ch 17 · Blackjack
Chess	`Piece` (sealed) + subclasses, `SlidingPiece`, `Board`, `Move`, `Rules`, `Game`	Polymorphic pieces, Factory Method, Command + Memento (undo)	`synchronized` game so a move and a resignation don't interleave	Ch 17 · Chess
Tic-tac-toe	`Board` (row/col/diagonal counters), `Game`, `Player`, `MoveStrategy`	Strategy (players)	Many games in a `ConcurrentHashMap`; per-game lock	Ch 17 · Tic-Tac-Toe
Snake & ladder	`Board`, `Jump`, `Dice`, `Rules`, `OverflowRule`, `WinStrategy`, `Game`	Strategy (dice, overflow, win)	Each game its own `Random` and its own thread of turns	Ch 17 · Snake and Ladder
2048	`LineMerger`, `Board`, `TileSpawner`, `Game`, `Direction`	Strategy (spawner); one pure merge function	Single-threaded; `synchronized move` if UI and input threads both call it	Ch 17 · 2048

Platforms (Chapters 18–19)

Problem	Key entities	Patterns used	Concurrency concern	Where
Stack Overflow	`Post` (sealed) → `Question` / `Answer` / `Comment`, `Member`, `Tag`, `Bounty`, `QuestionIndex`	Strategy (reputation), Observer (badges), Facade; inverted index	Two votes at once → `synchronized` post; `AtomicInteger` reputation	§18.1
Cricinfo	`Match` → `Innings` → `Over` → `Ball`, `Team`, `Playing11`, `StatQuery`	Template Method (stat queries), Observer (live pages)	Two scorers → `synchronized` match; observers in `CopyOnWriteArrayList`	§18.2
LinkedIn	`Member`, `Profile`, `SocialContent`, `ConnectionInvitation`, `ConnectionService`, `SearchIndex`, `FeedService`	State (invitation), Strategy (feed ranking, notifications), Facade	Symmetric connections → all graph changes synchronized on the service	§18.3
Facebook	`Member`, `Audience` (sealed), `PrivacyList`, `FriendService`, `VisibilityService`, `NewsFeedService`	Strategy (feed ranker), Observer (notifications)	Accept can't leave a one-way friendship → synchronized service, concurrent adjacency sets	§18.4
Amazon	`Product`, `ShoppingCart`, `Order`, `OrderLog`, `Payment`, `Shipment`, `Customer` → `Guest` / `Member`	Strategy (payment), Observer (notifications), order state machine, Facade	Last unit, two buyers → `synchronized Product.reserve`; all-or-nothing multi-line reservation	Ch 19 · Amazon
Stock brokerage + exchange	`Order` → Market/Limit/StopLoss/StopLimit, `OrderPart`, `StockLot`, `OrderBook`, `MatchingEngine`	Observer (fills), Template Method (statements), trade-price policy	Over-spending → reserved funds; one lock per symbol; price-then-time matching	Ch 19 · Stock Brokerage
Auction	`AuctionListing`, `Bid`, `BidResult`, `AuctionService`, `AuctionListener`	Observer, Specification (search), status state machine	Concurrent bids → all bid rules in one per-listing critical section; notify after unlocking	Ch 19 · Auction
Music streaming (Spotify)	`Song` / `Album` / `Artist`, `MusicLibrary`, `Playlist`, `MusicPlayer`, `PlaybackOrder`, `Recommender`	Strategy (play order, recommendations), Observer (plays), plan rules in an enum	Two devices press "next" → `synchronized` player methods	Ch 19 · Music Streaming
Food delivery (Swiggy) + FoodKart	`Restaurant`, `Cart`, `Order`, `DeliveryAgent`, `DeliveryAssignmentStrategy`, `RestaurantSelectionStrategy`	Strategy (assignment, selection, payment), Observer, order state machine	Kitchen capacity reserved atomically; riders claimed with CAS	Ch 19 · Food Delivery
Grocery store (POS, point of sale)	`Register`, `Sale`, `LineItem`, `PricingEngine`, `ItemPromotion` / `BasketPromotion`, `TaxPolicy`, `Tender` (sealed), `Receipt`, `Inventory`	Strategy (offers, tax, payment gateway), Facade (register), Observer (stock), sale state machine	Registers share stock → atomic `merge`, may go negative, edge-triggered alerts; returns claimed with `compute`	Ch 19 · Grocery Store

Pattern-driven components (Chapter 20)

Problem	Key entities	Patterns used	Concurrency concern	Where
Splitwise	`Expense`, `SplitStrategy` (Equal/Exact/Percent), `Share`, `Ledger`, `DebtSimplifier`	Strategy (splits)	One signed number per pair, one mutator; `BigDecimal`, no lost cents	§20.1
Notification service	`NotificationChannel` (abstract), channels, `SendRule`, `ChannelFactory`, `DeliveryListener`	Template Method (channels), Strategy (send rules), Observer, Simple Factory, Adapter	Rate-limit check-then-act gap; idempotency	§20.2
In-memory cache / LRU	`Cache<K,V>`, `Node`, `DoublyLinkedList`, `EvictionPolicy`, `Cache.Builder`, `ShardedCache`	Strategy (eviction), Observer (removals), Builder	One lock (a `get` is a write in LRU); listeners outside the lock; shard to scale	§20.3
Ride sharing (Uber)	`Rider`, `Driver`, `Trip`, `MatchingStrategy`, `PricingStrategy`, `SurgePricing`	Strategy ×3, Decorator (surge), trip state machine	Double-booking a driver → atomic claim (`AtomicBoolean`)	§20.4
Logging framework	`Logger`, `LogManager`, `LogRecord`, `Appender`, `AsyncAppender`, `Formatter`	Chain of Responsibility (hierarchy), Singleton (`LogManager`), Strategy (formatters), Template Method (appender), Decorator (async)	Thread-safe sinks; never block on a full queue; never crash the app	§20.5
Pub-sub	`Broker`, `Topic`, `Subscription`, `Subscriber`, `Message`	Observer through a broker	One dispatcher per subscription; ack after processing (at-least-once) → idempotent subscribers	§20.6
Rate limiter	`Limiter` (TokenBucket, FixedWindow, SlidingWindowLog/Counter), `CompositeLimiter`, `LimiterFactory`	Strategy, Simple Factory, Composite, Facade	Atomic refill-check-take; one limiter per key via `computeIfAbsent`; Redis (a shared in-memory store) + Lua (a script Redis runs atomically) across servers	§20.7

Machine-coding problems (Chapter 21)

Problem	Key entities	Patterns used	Concurrency concern	Where
Task management	`Task`, `TaskStatus`, `TaskService`, `TaskRepository`, `TaskFilter`	Specification (filters), Strategy (sorts), Observer, enum state machine	Per-task `synchronized`	§21.1
Digital wallet	`Wallet`, `Transaction`, `Money`, `Offer`, `WalletService`	Strategy (offers), idempotency key	Lock both wallets in a global (id) order; debit before credit; total conserved	§21.2
Event calendar (Flipkart)	`TimeSlot` (half-open), `Event`, `EventStore`, `RepresentativeSelector`, `CalendarService`	Strategy (representative), plan-then-save	Atomic check-and-book	§21.3
Coding Blox (Flipkart)	`Contest`, `Level`, `User`, `Question`, `SolveSimulator`, `CommandProcessor`	Command map, Strategy (solver, scoring)	Atomic rating update; don't run a contest twice	§21.4
PropertyHunt (ClearTrip)	`Property`, `SearchCriteria` (+ Builder), `PropertyService`	Builder + predicate list, Strategy (sort)	Synchronized mark-sold	§21.5
Ledger Co (Navi)	`Loan`, `LumpSum`, `Balance`, `InterestPolicy`, `LedgerService`	Strategy (interest), derived state, command loop	Synchronized lump sums	§21.6
Inventory management	`Product`, `Warehouse`, `StockMovement`, `AllocationStrategy`, `ReorderListener`	Strategy (allocation), Observer (edge-triggered alerts)	Overselling → lock per SKU (stock-keeping unit: one product variant)	§21.7
Unix file search (`find`)	`Node` → `FileNode` / `DirectoryNode`, `FileFilter`, `AndFilter` / `OrFilter` / `NotFilter`, `SearchCriteria` (+ Builder), `FileSearcher`, `FindCommandParser`	Specification + Composite (filters and file tree), Builder, recursive-descent parser	Weakly consistent children map; read-only fork/join walk	§21.8

> **TL;DR — what every case study teaches**
>
> - **Name the real unit of inventory** (copy, show seat, room-night, vehicle-day) (Ch 15 summary).
> - **Every "if free then take" is one atomic step** — a monitor, a CAS, or a conditional `UPDATE` (Ch 15 summary).
> - **Put rules on the object that owns the data**; the service only orders the steps (Ch 16, Ch 20 summaries).
> - **Strategy** for the rule most likely to change; **Observer** for "tell someone when it happens"; **State** (or an enum state machine) for lifecycles.
> - **Inject** time and randomness; money in `BigDecimal` or integer cents; call listeners **outside** locks.

22.10 Machine-coding and edge-case checklists

Source: §21.0 (and §1.10).

Machine-coding checklist

Before coding (0–20 min)

- [ ] Read the statement twice; mark **mandatory vs bonus**; ask clarifying questions.
- [ ] Put the statement's sample input into the driver as a `List<String>` of commands.
- [ ] Ask: "Where does it need to be thread-safe?" (wallets and inventory: always yes).
- [ ] Nouns → entities, verbs → service methods. Pick the 1–2 patterns the statement begs for ("can be sorted by X or Y" → Strategy).

- [ ] Create the six packages: **model, repository, service, strategy, exception, driver**.

While coding (20–75 min)

- [ ] Implement mandatory requirements **in the statement's order**; run the driver after each one (at least every 15 minutes).
- [ ] Entities guard their own invariants (e.g. `Task.changeStatus` rejects `TODO → DONE`).
- [ ] Services return data and throw domain exceptions; **no `System.out` in services**.
- [ ] Repository = interface + `ConcurrentHashMap` implementation; IDs from an `AtomicInteger`.
- [ ] Driver = `Map<String, Consumer<String[]>>` command processor; catch domain exceptions per command, **print and continue**.

Before you stop (75–90 min)

- [ ] Edge cases and validation done (see below); custom exceptions with readable messages.
- [ ] Thread-safety where money or stock is shared; say where the lock is and why.
- [ ] Bonus only after every mandatory feature runs.
- [ ] Clean names, delete dead code, run everything once more, prepare a 2-minute walkthrough.

Rubric the interviewer uses (§21.0)

#	Criterion	How to score well
1	Working, demo-able code	Thin end-to-end path in the first hour; never finish with code that doesn't compile
2	Functional completeness	Mandatory first, in order
3	Clean models	Enums for fixed sets, records for values, classes for things that change
4	Separation of concerns	Driver → Service → Repository
5	Extensibility	New rule = new class (Strategy, Specification)
6	Edge cases & failure handling	Custom exceptions; the driver keeps going
7	Readability	Names from the statement's nouns and verbs
8	Concurrency (when asked)	Say where the lock is; demo with a thread pool if time allows
9	Tests (bonus)	A tiny `assert`-style self-check in `main` is enough

Edge-case checklist (§1.10, §21.0, case studies)

Category	Questions to ask your code
Input	`null`, empty, negative, zero, duplicate IDs, unknown IDs, invalid numbers
Ownership	Wrong user acting on someone else's object; a player moving out of turn
Capacity	Full lot, zero seats left, empty playlist, max loans reached, party larger than any table
State	Paying twice, exiting without entering, cancelling a delivered order, moving after game over, running a contest twice
Time	Exit before entry, holds/bookings expiring, part hours (round up?), an event exactly at the end instant (half-open intervals), late returns
Money	`BigDecimal` scale and rounding; no lost cents in splits; compare with `compareTo`, not `equals` (Ch 4 money note)
Concurrency	Two threads booking the same seat/spot/rider; multi-item bookings locking in sorted order; retries that must be idempotent
Failure	Payment declined after a hold; hold lost before payment → compensating refund; listener throws
Randomness	Seeded `Random` or a scripted strategy so the demo is repeatable

Common mistakes (§21.0)

Mistake	Instead
20 minutes on a perfect class diagram	Enums/records first; the diagram lives in names
One `Manager` with 15 maps	One service per area + repositories
`System.out.println` inside a service	Services return data; the driver prints
`double` for money	`BigDecimal` with a fixed scale, or whole units in a `long`
Crashing on the first bad command	Catch domain exceptions in the driver loop
Bonus before mandatory	Mandatory in order; bonus last
`Random` without a seed	Seed it, or put randomness behind a Strategy
`LocalDate.now()` in logic	Inject a `java.time.Clock`

22.11 Glossary

Alphabetical. "Where" gives the chapter (and section) where the term is explained in full. Besides the design vocabulary, the list covers the everyday Java, tooling and domain words the case studies use (`JVM`, `Redis`, `SKU`, `UPI` …), so a newcomer can look up anything a chapter takes for granted. System-design terms such as latency, throughput, caching, message brokers and eventual consistency are explained in more depth in System Design Fundamentals.

Term	Plain meaning	Where
ABA problem	A value changes A → B → A, so a compare-and-swap cannot notice the change; fix with a version stamp (`AtomicStampedReference`)	Ch 12 §12.15
Abstract class	A class that cannot be instantiated; may hold state, constructors, finished and abstract methods	Ch 2 §2.9
Abstract Factory	Creational pattern that creates families of matching objects without naming their classes	Ch 8 §8.11
Abstraction	Exposing only what callers need and hiding how it is done (interface or abstract class)	Ch 2 §2.4
Access modifiers	`public`, `protected`, package-private, `private`: who can see a member	Ch 2 §2.14
Activity diagram	UML flowchart of a process, with decisions, swimlanes and fork/join	Ch 3 §3.13
Actor	Someone or something that uses the system (customer, admin, the system itself)	Ch 1 §1.5
Adapter	Structural pattern that converts one interface into the one the client expects	Ch 9 §9.1
Aggregate (DDD)	A cluster of objects changed as one unit; only its root is reachable from outside, and the root guards the invariants	Ch 5 §17
Aggregation	"Has-a" where the part can outlive the whole; hollow diamond at the whole	Ch 3 §3.3
Anemic domain model	Classes with data but no behaviour; the logic lives in services that pull the data out	Ch 5 §5
Anti-corruption layer	Translation layer at a bounded context's border that keeps a foreign or legacy model out of yours	Ch 5 §17
Anti-pattern	A common "solution" that reliably causes more problems than it solves	Ch 8 §8.6
AOP (aspect-oriented programming)	Adding cross-cutting behaviour (logging, transactions) around many methods without editing them, usually via generated proxies	Ch 5 §11, Ch 9 §9.7
API	Application Programming Interface: the set of public methods (or endpoints) other code is allowed to call	Ch 2 §2.6, §2.14
Appender / formatter	Logging roles: an appender writes a log record somewhere (console, file); a formatter turns it into text	Ch 20 §20.5
Append-only log	Messages are only ever added at the end; readers track their own offset	Ch 20 §20.6
Assertion	A check in code or a test that a condition is true, failing loudly if it is not	Ch 6 §8
Association	A class keeps a reference to another it does not own; solid line	Ch 3 §3.2
Association class	A class that holds data about a link itself (e.g. `Enrollment{grade}`)	Ch 3 §3.11
AST (abstract syntax tree)	The tree a parser builds from an expression or program, one node per piece of grammar	Ch 10 §10.6
Asynchronous vs synchronous	The caller carries on and gets the result later vs the caller waits until the work finishes	Ch 3 §3.12, Ch 20 §20.5
At-least-once delivery	A message may be delivered more than once, never lost; consumers must be idempotent	Ch 20 §20.6
Atomic variable	`AtomicInteger` / `AtomicLong` / `AtomicReference`: lock-free single-variable updates using CAS	Ch 12 §12.15
Atomicity / atomic operation	An operation that happens completely or not at all, with no other thread able to see it half-done	Ch 12 §12.5

Term	Plain meaning	Where
Audit log	An append-only record of who did what and when, kept for checking later	Ch 10 §10.2, Ch 21
Authentication vs authorization	Proving who you are (login, PIN) vs checking what you are allowed to do	Ch 6 §10, Ch 14 §14.4
Back-pressure	Slowing producers when consumers can't keep up, e.g. with a bounded queue	Ch 12 §12.22
BFS / DFS	Breadth-first search (visit neighbours level by level) vs depth-first search (go deep down one path, then back up)	Ch 18 §18.3, Ch 21 §21.8
`BigDecimal`	Exact decimal type used for money; build from strings, compare with `compareTo`	Ch 1 §1.10, Ch 4 (money note)
Big-O notation	Shorthand for how running time or memory grows with input size: `O(1)` constant, `O(log n)`, `O(n)` linear	Ch 20 §20.3
Blocking call	A call that makes the thread wait (for I/O, a lock or a queue) before it can continue	Ch 12 §12.4
Bloom filter	Probabilistic set: may say "maybe present" wrongly, never says "absent" wrongly	Ch 13 §13.9
Boilerplate	Repetitive code that must be written but carries no real logic (getters, `equals`, `hashCode`)	Ch 2 §2.13
Bounded context	Boundary inside which one domain model and one vocabulary apply consistently (often one team or service)	Ch 5 §17
Bounded queue	A queue with a fixed capacity; producers wait or are refused when it is full	Ch 12 §12.22
Boy Scout Rule	Leave the code a little cleaner than you found it	Ch 6 (intro)
Bridge	Structural pattern splitting an abstraction from its implementation so both vary independently	Ch 9 §9.2
Broker	A middleman service that receives messages from publishers and delivers them to subscribers	Ch 20 §20.6
Build tool	A program (Maven, Gradle) that downloads dependencies, compiles, tests and packages the code	Ch 3 §3.5
Builder	Creational pattern that builds a complex object step by step, validated in `build()`	Ch 8 §8.12
Busy waiting (spinning)	A thread repeatedly checking a condition in a loop instead of sleeping; wastes CPU	Ch 12 §12.6
Bytecode	The compact instruction format `javac` produces and the JVM runs	Ch 12
Callback	A function you hand to other code so it can call you back when something happens	Ch 5 §10
Caretaker / Originator	Memento roles: the originator makes and restores snapshots; the caretaker stores them without looking inside	Ch 10 §10.5
Carrier thread	The real OS thread that a virtual thread runs on while it is not blocked	Ch 12 §12.4
CAS (compare-and-swap)	Atomic "set to new value only if it still equals the expected value"	Ch 12 §12.15
Chain of Responsibility	Behavioral pattern passing a request along handlers until one handles or stops it	Ch 10 §10.1
Characterisation test	A test that records what existing code does now, before you refactor it	Ch 7 §7.8
Checked vs unchecked exception	Checked exceptions must be declared or caught (compiler-enforced); unchecked ones (`RuntimeException` subclasses) need not be	Ch 2 §2.5, Ch 6 §6

Term	Plain meaning	Where
Check-then-act	Read a condition, then act on it, as two steps: a race unless made atomic	Ch 12 §12.5, §12.25
Circuit breaker	Stops calling a failing dependency for a while after repeated errors, then tries again carefully	Ch 5 §13
Circular wait	T1 waits for T2's lock while T2 waits for T1's; one of the four deadlock conditions	Ch 12 §12.17
Class	A blueprint for objects: fields plus methods	Ch 2 §2.2
Class diagram	UML diagram of classes, fields, methods and relationships — "the answer" in an LLD round	Ch 3 §3.11
Class explosion	Subclasses multiplying across two dimensions (`WindowsButton`, `MacButton`, …)	Ch 9 §9.2
Classpath	The list of folders and JAR files where the JVM looks for classes	Ch 1 §1.3
Clean code	Code whose intent is obvious to the next reader and which is easy to change safely	Ch 6 (intro)
Coarse- vs fine-grained locking	One big lock for everything (simple, slow under load) vs many small locks per item (faster, harder to get right)	Ch 12 §12.11
Code review	A peer reads a change before merge: correctness first, then design, readability, tests, security, performance	Ch 6 §15
Code smell	A surface symptom that usually signals a deeper design problem; not a bug by itself	Ch 7 §7.2
Coffman conditions	Mutual exclusion, hold and wait, no preemption, circular wait: all four needed for deadlock	Ch 12 §12.17
Cohesion	How closely the things inside one class belong together; high is good	Ch 2 §2.16, Ch 4 §4.3
Coin-change DP	Finding the fewest coins (or whether change is possible) by building answers for smaller amounts first	Ch 14 §14.2
Collections framework	Java's built-in data structures (`List`, `Set`, `Map`, `Queue`) and their implementations	Ch 2
Command	Behavioral pattern turning a request into an object so it can be queued, logged or undone	Ch 10 §10.2
Command–Query Separation (CQS)	A method either changes state or returns a value, not both	Ch 6 §2
Commit / rollback	Make a transaction's changes permanent vs throw all of them away	Ch 10 §10.5, Ch 15 §15.2
`Comparator` / `Comparable`	An object that orders two values from outside vs a class's own natural order (`compareTo`)	Ch 1 §1.9, Ch 11 §11.3
Compensation	Undoing a completed step with a new action (e.g. a refund) because money can't be rolled back	Ch 15 §15.2
Compile time vs run time	When `javac` checks and translates the code vs when the program is actually executing	Ch 2 §2.7
`CompletableFuture`	A future you can chain (`thenApply`, `thenCombine`) to combine asynchronous results	Ch 12 §12.4
Composite	Structural pattern that lets clients treat single objects and trees the same way	Ch 9 §9.3
Composition	"Owns" relationship: the part is created and hidden by the whole and dies with it; filled diamond	Ch 3 §3.4

Term	Plain meaning	Where
Composition over inheritance	Reuse behaviour by holding and delegating rather than subclassing	Ch 2 §2.11, Ch 5 §6
Composition root	The one place (usually `main`) where objects are created and wired together	Ch 6 §10
Compound interest	Interest charged on the principal plus interest already added	Ch 21 §21.6
Concurrency	Several tasks in progress at once, possibly sharing data	Ch 12 §12.1
Concurrency vs parallelism	Many tasks in progress (structure) vs many executing at the same instant (hardware)	Ch 12 §12.1
`ConcurrentHashMap`	Thread-safe map with atomic compound operations (`putIfAbsent`, `compute`, `merge`)	Ch 12 §12.16
Condition variable	Lets a thread wait until shared state meets a condition (`wait/notify`, `Condition`)	Ch 12 §12.13
Constructor	Special method that runs when an object is created and sets up its fields	Ch 2 §2.2
Constructor chaining	One constructor calling another with `this(...)`	Ch 2 §2.2
Contention	Many threads competing for the same lock or variable at once, making them wait	Ch 12 §12.7, §12.11
Context switch	The OS pausing one thread and resuming another on a CPU core; costs time	Ch 12 §12.1
Controller (GRASP)	A thin non-UI object that receives each system event and delegates	Ch 5 §16.3
`CopyOnWriteArrayList`	Thread-safe list that copies on every write; good for read-mostly listener lists	Ch 12 §12.16
`CountDownLatch`	One-shot gate that opens after N `countDown()` calls	Ch 12 §12.14
Coupling	How much one class depends on another's details; low is good	Ch 2 §2.16, Ch 4 §4.4
CPU	The processor; each core runs one thread at a time	Ch 12 §12.1
CQRS	Command Query Responsibility Segregation: separate models for writing data and for reading it	Ch 5 §17
Creator (GRASP)	The class that contains, records or has the data for A should create A	Ch 5 §16.2
Critical section	Code that touches shared state and must run in one thread at a time	Ch 12 §12.5
Cron	A schedule format (minute, hour, day…) for running a job at fixed times	Ch 10 §10.6
Cross-cutting concern	Behaviour needed in many places (logging, security, transactions) that doesn't belong to one class	Ch 4 §4.5, Ch 6 §10
CRUD	Create, Read, Update, Delete: the four basic storage operations	Ch 5 §17
CSV	Comma-separated values: a plain-text table, one row per line	Ch 5 §3
`CyclicBarrier`	Reusable meeting point where N threads wait for each other	Ch 12 §12.14
Daemon thread	A background thread that does not keep the JVM alive once all normal threads end	Ch 12 §12.4
DAO (Data Access Object)	A class that wraps the SQL for one table; closer to the database than a Repository	Ch 11 §11.7.3
Data Class (smell)	A class with only fields and getters/setters, manipulated by others	Ch 7 §7.2.22
Data Clumps (smell)	The same group of values always travelling together	Ch 7 §7.2.10
Database transaction	A group of database changes that succeed or fail together	Ch 12 §12.25, Ch 15

Term	Plain meaning	Where
Dead code	Code that is never run or whose result is never used; delete it	Ch 6 §13, Ch 7
Dead-letter queue	Where messages go after retries are exhausted, for later inspection	Ch 20 §20.6
Deadlock	Two or more threads each wait forever for a resource another holds	Ch 12 §12.17
Debit / credit	Taking money out of an account vs putting money in	Ch 14 §14.4, Ch 21 §21.2
Decorator	Structural pattern adding behaviour by wrapping an object behind the same interface	Ch 9 §9.4
Deep vs shallow copy	Copying nested objects too vs sharing them with the original	Ch 8 §8.13
Defensive copy	Copying a mutable object passed in or handed out so outsiders can't change your internals	Ch 2 §2.13
Delegation	An object handing a job to another object it holds, instead of doing it itself	Ch 2 §2.11, Ch 5 §6
Dependency	"Uses" relationship: B appears only as a parameter, local, return type or static call; dashed arrow	Ch 3 §3.5
Dependency injection (DI)	Supplying a class's collaborators from outside (usually via the constructor)	Ch 4 §4.10, Ch 11 §11.7.2
Dependency Inversion Principle	High- and low-level code both depend on abstractions owned by the high-level policy	Ch 4 §4.10
`Deque` / `ArrayDeque`	Double-ended queue: add and remove at both ends; the usual Java stack and queue	Ch 7 §7.7
Derive, don't store	Compute counts and totals from the underlying records so they can't drift	Ch 18 §18.5
Design by Contract	Preconditions (caller's duty), postconditions (method's promise), invariants (always true)	Ch 5 §15
Design pattern	A named, reusable solution to a recurring design problem	Ch 8 §8.1
Director	Optional Builder role that runs a fixed sequence of building steps	Ch 8 §8.12
Distributed lock	A lock shared by several servers, kept in an external store (database, Redis)	Ch 12 §12.25
Divergent Change (smell)	One class changed for many different reasons	Ch 7 §7.2.7
Domain event	Past-tense record of a business fact (`OrderPlaced`) that other parts of the system react to	Ch 5 §17
Domain exception	A custom exception named after a business rule (`InsufficientBalanceException`)	Ch 21 §21.0
Domain-Driven Design (DDD)	Modelling software on the business: bounded contexts, entities, value objects, aggregates, domain events, repositories	Ch 5 §17
Double dispatch	Choosing a method by both the element's and the visitor's type (Visitor's `accept`)	Ch 11 §11.5
Double-checked locking	Lazy singleton that checks, locks, checks again; the field must be `volatile`	Ch 8 §8.8
Driver (machine coding)	The `main` class that feeds commands to the services and prints results	Ch 1 §1.10, Ch 21 §21.0
DRY	Don't Repeat Yourself: each piece of knowledge has one authoritative home	Ch 5 §1
DTO (Data Transfer Object)	A plain data holder used only to carry values across a boundary (API, layer)	Ch 2 §2.3

Term	Plain meaning	Where
Dynamic dispatch	The JVM picks an overridden method from the object's actual class at run time	Ch 2 §2.6–§2.7
Dynamic programming (DP)	Solving a problem by combining saved answers to smaller sub-problems	Ch 14 §14.2
Edge-triggered alert	Fires once when a value crosses a threshold, not on every update while it stays past it	Ch 19 · Grocery Store, Ch 21 §21.7
EMI	Equated monthly instalment: the fixed amount repaid each month on a loan	Ch 21 §21.6
Encapsulate what varies	Isolate the part that changes behind an interface	Ch 5 §8
Encapsulation	Data kept private; only the owning class's methods change it and enforce its rules	Ch 2 §2.3
Entity (DDD)	Object defined by an identity that survives changes; two entities are equal when their ids are equal	Ch 5 §17
Enum (with behaviour)	A fixed set of named constants that can carry fields and methods	Ch 2 §2.10
`EnumMap` / `EnumSet`	Very fast map and set implementations whose keys are the constants of one enum	Ch 2 §2.10
`equals`/`hashCode` contract	Equal objects must have equal hash codes	Ch 2 §2.12
Event Sourcing	Store the stream of changes (events); rebuild state by replaying them	Ch 11 §11.7.6
Event-driven	Parts of the system react to published events instead of calling each other directly	Ch 5 §9
Eventual consistency	Copies of data may disagree for a short time but converge once updates stop	Ch 5 §17
Eviction	Removing entries from a cache to make room (LRU, LFU, FIFO)	Ch 20 §20.3
Exception	An object thrown to signal an error, which unwinds the call stack until something catches it	Ch 6 §6
`ExecutorService`	Runs submitted tasks on a pool of threads; separates what to run from how	Ch 12 §12.4
Exponential backoff	Waiting longer after each failed retry (1 s, 2 s, 4 s…), usually with jitter	Ch 6 §16
Expression problem	Visitor makes adding operations cheap but adding types expensive (and the reverse for plain polymorphism)	Ch 11 §11.5
Facade	Structural pattern giving one simple entry point to a complex subsystem	Ch 9 §9.5
Factory Method	Creational pattern where subclasses decide which product to create	Ch 8 §8.10
Fail fast	Detect problems early and loudly (guard clauses, `requireNonNull`)	Ch 5 §13
Fairness	Serving waiting threads roughly in order so none starves; costs throughput	Ch 12 §12.19
Fan-out	One event or post copied to many receivers (followers' feeds, subscribers)	Ch 18 §18.4, Ch 20 §20.6
Feature Envy (smell)	A method more interested in another class's data than its own	Ch 7 §7.2.9
Field	A variable that belongs to an object (instance field) or to the class (`static` field)	Ch 2 §2.2
FIFO / LIFO	First in, first out (a queue) vs last in, first out (a stack)	Ch 12 §12.8, Ch 15
F.I.R.S.T.	Clean tests are Fast, Independent, Repeatable, Self-validating, Timely	Ch 6 §8
Flag argument	A boolean parameter that makes a function do two things; split it instead	Ch 6 §2

Term	Plain meaning	Where
Flyweight	Structural pattern sharing intrinsic state among many fine-grained objects	Ch 9 §9.6
Fork/Join	Split a task recursively into subtasks run in parallel, then combine results	Ch 12 §12.26, Ch 13 §13.10
Fragile base class	A change in a parent class silently breaks subclasses	Ch 5 §6
Framework	A library that owns the program's flow and calls your code at set points (Spring, JUnit)	Ch 5 §10
Functional interface	An interface with one abstract method, usable with lambdas	Ch 2 §2.8
Functional requirement	A behaviour the system must have ("a customer takes a ticket")	Ch 1 §1.5
GA (general admission)	Unnumbered standing or seating tickets sold as a count, not as named seats	Ch 15 §15.3
Garbage collection (GC)	The JVM automatically freeing memory of objects nothing refers to any more	Ch 12
Generalization	"Is-a" (inheritance) relationship; solid line, hollow triangle at the parent	Ch 3 §3.6
Generics	Type parameters (`List<T>`) for type-safe reusable code	Ch 2 §2.15
Geohash	Encodes a latitude/longitude as a short string; nearby places share a prefix, so it works as a search key	Ch 20 §20.4
Getter / setter	Methods that read (`getX`) or change (`setX`) a field	Ch 2 §2.3
Git	The version-control tool that records every change to the code as commits	Ch 4 §4.2
Glob	Filename wildcard pattern: `*` any characters, `?` one character, `[0-9]` one character from a set	§21.8
Global Data (smell)	Data readable and writable from anywhere, including mutable singletons	Ch 7 §7.2.5
God class / God Object	One class that knows and does almost everything	Ch 8 §8.6, Ch 7 §7.2.20
GoF	"Gang of Four": the four authors of the 1994 book that catalogued the 23 classic design patterns	Ch 8 §8.2
Golden Hammer	Applying a favourite tool or pattern to every problem	Ch 8 §8.6
Graph	Nodes joined by edges (people and connections, stops and routes)	Ch 18 §18.3
GRASP	Larman's nine guidelines for assigning responsibilities to classes	Ch 5 §16
Greedy algorithm	Always take the best-looking choice right now; fast, but not always correct (e.g. coin change)	Ch 14 §14.2
GST	Goods and Services Tax, India's sales tax	Ch 19 · Grocery Store
Guard clause	An early check-and-return/throw at the top of a method instead of nested `if`s	Ch 7 §7.3.5, Ch 5 §13
Half-open interval	A time range `[start, end)` that includes the start but not the end, so back-to-back bookings don't clash	Ch 15 §15.4, Ch 21 §21.3
Happens-before	Java Memory Model rule guaranteeing one action's effects are visible to another	Ch 12 §12.6
Hash function / hash code	Turns a value into a number used to pick a bucket; equal values must give equal hashes	Ch 2 §2.12
Hash table / `HashMap`	Key-value store that finds an entry in about constant time by hashing the key	Ch 2 §2.12

Term	Plain meaning	Where
Heap vs stack (memory)	Where objects live (shared by all threads) vs each thread's own area for local variables and calls	Ch 12 §12.2
Heuristic	A rule of thumb that is usually right and cheap to apply, not a guarantee	Ch 6 §14
Hexagonal architecture	Ports and adapters: the domain core defines interfaces; databases, UIs and vendors plug in from outside	Ch 20 §20.4
High Cohesion (GRASP)	Assign responsibilities so each class does related work	Ch 5 §16.5
HLD vs LLD	Many services and machines vs classes inside one application	Ch 1 §1.2
Hold with TTL	A temporary claim (e.g. seats during payment) that expires automatically	Ch 15 §15.2
Holder-class idiom	Lazy singleton created inside a nested static class; the JVM's class loading makes it thread-safe	Ch 8 §8.8
Hollywood Principle	"Don't call us, we'll call you": the framework calls your hooks	Ch 5 §10
HTTP	The request/response protocol browsers and web services use	Ch 3 §3.12
IDE	Integrated development environment (IntelliJ, Eclipse): editor plus compiler, debugger and refactoring tools	Ch 7
Idempotency	Running an operation twice has the same effect as once; an idempotency key enforces it	Ch 21 §21.2, Ch 20 §20.6
Identity vs equality	Same object in memory (`==`) vs same value (`equals`)	Ch 2 §2.12
Immutability	Objects that never change after creation; safe to share between threads	Ch 2 §2.13, Ch 5 §14
Indirection (GRASP)	Put an intermediate object between two things that shouldn't know each other	Ch 5 §16.8
Information Expert (GRASP)	The class that has the data does the work	Ch 5 §16.1
Inheritance	A child class reuses and specialises a parent (`extends`); only for true is-a	Ch 2 §2.5
In-memory	Data kept only in the program's RAM, lost when it stops; the default in LLD rounds	Ch 1 §1.4
Insider Trading (smell)	Two modules exchanging too much private information	Ch 7 §7.2.19
Instance	One object created from a class	Ch 2 §2.2
Integration test	Tests several real parts together (e.g. service plus database), slower than a unit test	Ch 6 §17
Interface	A contract of methods; a class can implement many	Ch 2 §2.8
Interface Segregation Principle	No client should depend on methods it does not use	Ch 4 §4.9
Interning	Keeping one shared copy of each equal value (`String.intern()`)	Ch 9 §9.6
Interpreter	Behavioral pattern with one class per grammar rule, evaluating sentences of a small language	Ch 10 §10.6
Interrupt	A polite request to a thread to stop what it is doing; blocking calls react by throwing `InterruptedException`	Ch 12 §12.3
Intrinsic vs extrinsic state	Flyweight's shared, immutable part vs the per-use part passed in by the caller	Ch 9 §9.6
Invariant	A fact about an object that must always be true ("balance is never negative")	Ch 5 (key terms), §15
Inversion of Control (IoC)	The framework controls the flow and calls your code; DI is one way to achieve it	Ch 5 §10, Ch 4 §4.14

Term	Plain meaning	Where
Inverted index	A map from each word/tag to the list of items containing it, for search	Ch 18 §18.1
Invoker / Receiver	Command roles: the invoker triggers commands; the receiver does the real work	Ch 10 §10.2
I/O	Input/output: reading or writing files, network or database; slow compared with CPU work	Ch 12 §12.4
Iterator	Behavioral pattern for traversing a collection without exposing its storage	Ch 10 §10.3
JAR	Java archive: a zip file of compiled classes and resources	Ch 4 §4.7
Java Memory Model (JMM)	Rules for when one thread's writes become visible to another	Ch 12 §12.6
`java.time`	Java's date-time API: `Instant` (a moment), `Duration`, `LocalDate`, and `Clock` (injectable "now")	Ch 1 §1.10
JDBC	Java's standard API for talking to relational databases	Ch 8 §8.11
JDK	Java Development Kit: compiler, tools and the standard library	Ch 2
JIT compiler	Part of the JVM that turns hot bytecode into fast machine code while the program runs	Ch 12
Jitter	Random extra delay added to retries so many clients don't retry at the same instant	Ch 6 §16, Ch 12 §12.18
JPA / Hibernate	Java's standard for mapping objects to database tables, and its most popular implementation	Ch 9 §9.7
JSON	A text format for structured data: objects in `{}`, lists in `[]`	Ch 3 §3.2
JUnit	The standard Java unit-testing framework	Ch 6 §8
JVM	Java Virtual Machine: runs compiled Java bytecode and manages memory and threads	Ch 2, Ch 12
Kafka	A distributed append-only log used as a durable message broker	Ch 20 §20.6
KISS	Keep It Simple: choose the simplest design that works	Ch 5 §2
KYC	Know Your Customer: identity checks a financial app must do before allowing higher limits	Ch 21 §21.2
Lambda	A short anonymous function, e.g. `x -> x * 2`, usable wherever a functional interface is expected	Ch 2 §2.8
Large Class (smell)	A class trying to do too much; a God Class	Ch 7 §7.2.20
Latency	How long one operation takes from start to finish	Ch 1 §1.4
Law of Demeter	Only talk to your immediate friends; avoid `a.getB().getC()` chains	Ch 5 §4
Lazy Element (smell)	A class or function that doesn't pull its weight	Ch 7 §7.2.14
Lazy expiry	Deleting an expired entry only when someone next touches it, instead of by a timer	Ch 13 §13.6, Ch 14 §14.7
Lazy initialization	Creating an object only when it is first needed	Ch 8 §8.8
Lease	A lock or hold that expires automatically after a set time unless renewed	Ch 12 §12.9
Least Astonishment	Code should do what its name promises and nothing more	Ch 5 §12
Ledger	An append-only list of money movements from which balances are derived	Ch 20 §20.1, Ch 21
Legacy code	Existing code, often without tests, that is risky to change	Ch 7 §7.8
LFU / MRU	Cache eviction that removes the least frequently used vs the most recently used entry	Ch 20 §20.3

Term	Plain meaning	Where
Linked list	A chain of nodes where each points to the next (doubly linked: also the previous)	Ch 20 §20.3
`LinkedHashMap`	A `HashMap` that also remembers insertion or access order; the shortcut for an LRU cache	Ch 20 §20.3
Linter / static analysis	Tools that read code without running it and flag likely bugs or style issues	Ch 6 §15
Liskov Substitution Principle	A subtype must be usable anywhere its parent is expected, with no surprises	Ch 4 §4.8
Listener	An object registered to be told when an event happens (the observer in Observer)	Ch 11 §11.1
Livelock	Threads keep reacting to each other and running, but none makes progress	Ch 12 §12.18
Lock ordering	Always acquire locks in one global order to prevent circular wait	Ch 12 §12.17
Lock striping	Guarding a map with an array of locks, each covering some buckets	Ch 13 §13.7
Lombok	A library that generates boilerplate (getters, builders) from annotations at compile time	Ch 8 §8.12
Long Function (smell)	A function too long to understand at a glance	Ch 7 §7.2.3
Long Parameter List (smell)	Many parameters, often of the same type or boolean flags	Ch 7 §7.2.4
`LongAdder`	Counter that scales better than `AtomicLong` under heavy contention	Ch 12 §12.15
LOOK scheduling	Elevator keeps moving in one direction while stops remain ahead, then reverses	Ch 14 §14.5
Loops (smell)	A classic loop where a stream pipeline would state the intent more clearly	Ch 7 §7.2.13
Loose coupling	Interacting objects know as little about each other as possible	Ch 5 §9
Lost wakeup	A signal sent before the waiter started waiting, so the waiter sleeps forever	Ch 12 §12.13
Low Coupling (GRASP)	Assign responsibilities to keep dependencies few and stable	Ch 5 §16.4
LRU cache	Evicts the least recently used entry; hash map + doubly linked list gives O(1)	Ch 20 §20.3
Lua	A small scripting language; Redis runs a Lua script as one atomic step	Ch 20 §20.7
Machine-coding round	LLD round where working, runnable in-memory code is the answer (90–120 min)	Ch 1 §1.2, Ch 21 §21.0
Market vs limit order	Buy or sell now at the best available price vs only at a given price or better	Ch 19 · Stock Brokerage
Matching engine	Per-symbol order book matching buy and sell orders by best price, then oldest first	Ch 19 · Stock Brokerage
MDC	Mapped Diagnostic Context: per-thread key-values (request id, user) automatically added to log lines	Ch 20 §20.5
Mediator	Behavioral pattern centralising communication between peers in one coordinator	Ch 10 §10.4
Memento	Behavioral pattern capturing and restoring state without exposing internals	Ch 10 §10.5
Memory leak	Memory that is never freed because something still holds references to objects no longer needed	Ch 11 §11.1, Ch 12 §12.24
Message Chains (smell)	`a.getB().getC().getD()` navigation	Ch 7 §7.2.17

Term	Plain meaning	Where
Message queue	A buffer service where producers leave messages for consumers to process later	Ch 20 §20.6
Method	A function that belongs to a class	Ch 2 §2.2
Method reference	Shorthand for a lambda that just calls a method, e.g. `String::length`	Ch 2 §2.8
Microservice vs monolith	Many small independently deployed services vs one deployable application	Ch 5 §17
Middle Man (smell)	A class whose methods mostly forward to another class	Ch 7 §7.2.18
Mixin	A small interface with `default` methods that adds a capability to many classes	Ch 4 §4.3
Mock / stub / fake	Test doubles: a mock checks calls were made, a stub returns canned answers, a fake is a simple working version	Ch 6 §17
Mockito	The most common Java library for creating mocks in tests	Ch 6 §8
Module / package	A group of related classes; in Java a package is a folder-level namespace	Ch 2 §2.14
Monitor	The lock built into every Java object, used by `synchronized` and `wait/notify`	Ch 12 §12.7
Multiplicity	How many objects take part at each end of a relationship (`1`, `0..1`, `*`, `1..*`)	Ch 3 §3.2
Mutable Data (smell)	Shared data changed after creation, causing surprises elsewhere	Ch 7 §7.2.6
Mutation testing	Deliberately breaking the code in small ways to check the tests notice	Ch 6 §17
Mutex	A lock that lets only one thread into a critical section	Ch 12 §12.7
MVC	Model–View–Controller; a compound of Observer, Strategy and Composite	Ch 11 §11.7.5
Mysterious Name (smell)	A name that doesn't say what the thing does or holds	Ch 7 §7.2.1
Non-functional requirement	A quality: thread safety, extensibility, in-memory storage, latency	Ch 1 §1.5
Null Object	A do-nothing implementation used instead of `null`	Ch 11 §11.7.1
`NullPointerException` (NPE)	Thrown when you call a method or read a field through a `null` reference	Ch 5 §13
Object	One instance of a class with real values	Ch 2 §2.2
Object Pool	Borrow and return a bounded set of expensive objects	Ch 8 §8.14
Observer	Behavioral pattern notifying many dependents when one object changes	Ch 11 §11.1
Offset	A position number in a log or list; each reader remembers its own	Ch 20 §20.6
OOA / OOD / OOP	Analysis (what) → design (how, with objects) → programming (code)	Ch 2 §2.17
Open/Closed Principle	Open for extension, closed for modification: add code, don't edit working code	Ch 4 §4.7
Optimistic locking	Read with a version; the write succeeds only if the version is unchanged	Ch 12 §12.10, §12.25
`Optional`	A container that may be empty; return it when "not found" is normal	Ch 1 §1.10
Order book	Per-stock lists of waiting buy and sell orders, sorted by price then time	Ch 19 · Stock Brokerage
OTP	One-time password: a short code valid for a single use, usually sent by SMS	Ch 5 §13, Ch 20 §20.2
Outbox (transactional)	Save events in a table in the same transaction as the data, and publish them afterwards, so neither is lost	Ch 5 §17

Term	Plain meaning	Where
`OutOfMemoryError`	Thrown when the JVM cannot find memory for a new object	Ch 12 §12.2
Out-of-scope list	Written list of what you will not build; shrinks the problem	Ch 1 §1.5
Overloading	Same method name, different parameters; chosen at compile time	Ch 2 §2.7
Overriding	Subclass replaces a parent's method; chosen at run time	Ch 2 §2.7
Payment gateway	An external service that actually charges cards or UPI; hidden behind your own interface	Ch 15 §15.2, Ch 19
PECS	Producer `extends`, consumer `super` — the rule for generic wildcards	Ch 2 §2.15
Pessimistic locking	Lock a row before changing it (e.g. `SELECT … FOR UPDATE`) so nobody else can touch it meanwhile	Ch 12 §12.5, Ch 15 §15.2
Phaser	A reusable, flexible barrier for threads meeting in phases	Ch 12 §12.14, §12.26
PII	Personally identifiable information (names, phone numbers) that must not leak into logs	Ch 6 §15
Pinning	A virtual thread stuck on its carrier thread while it blocks	Ch 12 §12.4
Pipeline	Data flowing through a sequence of steps, each feeding the next (streams, filters, middleware)	Ch 7 §7.2.13
PLU code	Four-digit price look-up code for loose produce, keyed at the till together with the scale weight	Ch 19 · Grocery Store
Plugin	A piece of code added to a system through a known interface without editing the system	Ch 4 §4.7
PNR	Passenger name record: the booking reference for an airline reservation	Ch 16 · Airline
Poison pill	A special message telling a consumer to stop; one per consumer	Ch 12 §12.22
Polymorphism	One call behaves differently depending on the object; removes type switches	Ch 2 §2.6, Ch 5 §16.6
POS	Point of sale: the till where a sale is rung up and paid	Ch 19 · Grocery Store
Precondition / postcondition	What must be true before a call (caller's duty) / what the method promises after it	Ch 4 §4.8, Ch 5 §15
Primitive Obsession (smell)	Using `double`, `String`, `int` for domain concepts like money or status	Ch 7 §7.2.11
`PriorityQueue`	A queue that always hands out the smallest (or highest-priority) element first; a binary heap inside	Ch 16 · Course Registration
Process	A running program with its own memory; it contains one or more threads	Ch 12 §12.2
Producer–consumer	Threads hand work through a (bounded) queue	Ch 12 §12.22
Program to an interface	Depend on supertypes; pick the concrete class once, at the edge	Ch 5 §7
Protected Variations (GRASP)	Wrap points of instability behind a stable interface	Ch 5 §16.9
Prototype	Creational pattern creating new objects by copying a configured one	Ch 8 §8.13
Proxy	Structural pattern: a stand-in controlling access to another object	Ch 9 §9.7
Pub-sub	Observer through a broker: publishers and subscribers don't know each other	Ch 20 §20.6
Pure Fabrication (GRASP)	An invented helper class (e.g. a repository) when no domain class fits	Ch 5 §16.7
Pure function	A function whose result depends only on its inputs and which changes nothing else	Ch 11 §11.7.6, Ch 17 · 2048

Term	Plain meaning	Where
Qualified association	An association looked up by a key (`Library [isbn] → Book`, i.e. a `Map`)	Ch 3 §3.11
Race condition	The result depends on the timing of threads touching shared mutable state	Ch 12 §12.5
Rate limiter	Caps how many requests a key may make per time window (token bucket, windows)	Ch 13 §13.11, Ch 20 §20.7
Read-write lock	Many readers at once, or one writer	Ch 12 §12.10
Realization	A class implements an interface; dashed line, hollow triangle at the interface	Ch 3 §3.7
Record	Compact Java class for immutable data; only shallowly immutable	Ch 2 §2.13
Recursion	A function that solves a problem by calling itself on a smaller part	Ch 9 §9.3
Recursive-descent parser	A parser with one method per grammar rule, each calling the others	Ch 10 §10.6, Ch 21 §21.8
Redis	A fast shared in-memory key-value store, often used for caches, counters and distributed locks	Ch 20 §20.7
Reentrancy	A thread may re-acquire a lock it already holds	Ch 12 §12.8
`ReentrantLock`	Explicit lock with try/timed/fair locking and conditions	Ch 12 §12.8
Refactoring	Changing internal structure without changing observable behaviour	Ch 7 §7.1
Reflection	Code inspecting or calling classes, fields and methods at run time by name	Ch 8 §8.8
Refused Bequest (smell)	A subclass that doesn't want, or throws on, inherited methods	Ch 7 §7.2.23
Regex	Regular expression: a pattern language for matching text	Ch 10 §10.6
Repeated Switches (smell)	The same `switch` on a type code in several places	Ch 7 §7.2.12
Repository	Collection-like interface that hides storage from business logic	Ch 11 §11.7.3
Requirement	One thing the system must do, written so you can check it	Ch 1 §1.5
REST	A style of HTTP API built around resources (URLs) and verbs (GET, POST…)	Ch 5 §1
Retry	Trying a failed operation again; safe only if the operation is idempotent	Ch 6 §16
Role interface	A small interface serving one kind of caller (the ISP fix)	Ch 4 §4.9
Rule of three	Refactor to an abstraction at the second or third case, not the first	Ch 4 §4.15
`RuntimeException`	Base class of unchecked exceptions	Ch 2 §2.5
Saga	A long business process split into local steps, each with a compensating action if a later step fails	Ch 3 §3.12, Ch 15
Scalability	How well a system keeps working as users, data or load grow	Ch 1 §1.14
`ScheduledExecutorService`	Runs tasks after a delay or repeatedly at a fixed rate	Ch 13 §13.6
Schema	The defined structure of stored data (tables, columns, types)	Ch 2 §2.17
SDK	Software development kit: a vendor's ready-made library for calling its service	Ch 9 §9.1
Sealed type	A type whose permitted subtypes are fixed, enabling exhaustive `switch`	Ch 11 §11.5
Seam	A place where you can swap behaviour (e.g. an interface) without editing the code around it; used for testing	Ch 2 §2.9, Ch 7 §7.7

Term	Plain meaning	Where
Seed	The starting number for a pseudo-random generator; the same seed gives the same sequence	Ch 17
Semaphore	A counter of permits that limits concurrent access or signals between threads	Ch 12 §12.12
Separation of Concerns	Each part of the program handles one concern	Ch 4 §4.5, Ch 5 §11
Sequence diagram	UML diagram of messages between objects over time	Ch 3 §3.12
Serialization	Turning an object into bytes or text (and back) for storing or sending	Ch 8 §8.8
Service Locator	Anti-pattern where a class asks a registry for its dependencies, hiding them	Ch 11 §11.7.2
Sharding	Splitting data across independent parts (each with its own lock) to scale	Ch 20 §20.3
Shotgun Surgery (smell)	One change forces many small edits across many classes	Ch 7 §7.2.8
Side effect	Anything a function does besides returning a value (changing a field, writing a file)	Ch 5 §12, Ch 6 §2
Signaling	One thread tells another that something has happened (semaphore, latch)	Ch 12 §12.20
Simple Factory	One method mapping a type value to a class; an idiom, not GoF	Ch 8 §8.9
Single Responsibility Principle	A class should have one reason (one actor) to change	Ch 4 §4.2
Singleton	Creational pattern ensuring exactly one instance with global access	Ch 8 §8.8
SKU	Stock-keeping unit: one sellable product variant tracked in inventory	Ch 21 §21.7
SLF4J	A logging facade: code logs through its API and any logging library can sit underneath	Ch 9 §9.5
Sliding window	Rate-limit algorithm counting requests in a moving time window (log or counter)	Ch 20 §20.7
SMS / SMTP	Text messages to phones / the protocol for sending email	Ch 20 §20.2
Snapshot	A copy of state at one moment, used to restore it or to read it safely	Ch 10 §10.5
SOLID	SRP, OCP, LSP, ISP, DIP: five guidelines that keep one change from spreading	Ch 4 §4.1
Specification	Named, combinable business-rule object with `isSatisfiedBy`	Ch 11 §11.7.4
Speculative Generality (smell)	Hooks and abstractions added "in case" but never used	Ch 7 §7.2.15
Split tender	Paying one bill with several payment methods (e.g. part UPI, part cash)	Ch 19 · Grocery Store
Spring	The most widely used Java application framework; creates and wires objects (DI), web, data access	Ch 4 §4.14, Ch 11 §11.7.2
Spurious wakeup	A waiting thread wakes with no signal; why `wait()` sits in a `while` loop	Ch 12 §12.13
SQL	The language for querying and updating relational databases	Ch 11 §11.7.3
Stack trace	The list of method calls active when an exception was thrown; shows where it came from	Ch 6 §6
`StampedLock`	Read-write lock with optimistic reads; not reentrant	Ch 12 §12.10
Starvation	One thread never gets the resource while others keep progressing	Ch 12 §12.19

Term	Plain meaning	Where
State (pattern)	Behavioral pattern where an object's behaviour changes with its internal state	Ch 11 §11.2
State machine diagram	UML diagram of one object's states and transitions (`event [guard] / action`)	Ch 3 §3.14
Stateless vs stateful	Keeps no data between calls (safe to share) vs remembers data between calls	Ch 1 §1.6
`static`	Belongs to the class, one per class, not to each object	Ch 2 §2.2
Stop-loss / stop-limit	Orders that wake up when the price hits a trigger, then sell at market / at a limit price	Ch 19 · Stock Brokerage
Strategy	Behavioral pattern making algorithms interchangeable behind one interface	Ch 11 §11.3
Stream (Java)	A pipeline over a collection (`filter`, `map`, `collect`) that states what, not how	Ch 2 §2.8, Ch 7 §7.2.13
Subclass / superclass	The child class that `extends` / the parent it extends	Ch 2 §2.5
`synchronized`	Java keyword that runs a block or method under an object's monitor: one thread at a time, with visibility	Ch 12 §12.7
TCP	The internet protocol that delivers a reliable, ordered byte stream between two machines	Ch 11 §11.2
Technical debt	Shortcuts in code that make future changes slower until someone cleans them up	Ch 7 §7.1
Telescoping constructors	A ladder of constructors each adding one more parameter; Builder replaces it	Ch 8 §8.12
Tell, Don't Ask	Tell objects what to do instead of pulling their data out to decide	Ch 5 §5
Template Method	Behavioral pattern: fixed algorithm skeleton in a base class, steps in subclasses	Ch 11 §11.4
Temporary Field (smell)	A field set only in some situations and empty otherwise	Ch 7 §7.2.16
Test double	Anything standing in for a real dependency in a test: dummy, stub, spy, mock, fake	Ch 6 §17
Test-Driven Development (TDD)	Failing test → make it pass → refactor, in very short cycles	Ch 6 §8
Thread	An independent path of execution inside a process, sharing its memory	Ch 12 §12.2
Thread confinement	Keeping data inside one thread so it never needs locking	Ch 12 §12.24
Thread dump	A snapshot of every thread's state and call stack (`jstack`, `jcmd`) used to diagnose hangs	Ch 12 §12.3
Thread pool	A fixed set of reusable threads fed from a task queue	Ch 12 §12.21
Thread states	NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED	Ch 12 §12.3
Thread-safe	Gives correct results even when many threads call it at the same time	Ch 12 §12.5
Thread-starvation deadlock	Tasks in a bounded pool wait for tasks queued behind them in the same pool	Ch 12 §12.17, §12.21
Throughput	How much work gets done per unit of time (requests per second)	Ch 12 §12.11
Timeout	A limit on how long to wait before giving up	Ch 12 §12.9
TL;DR	"Too long; didn't read": a short summary at the end of each section or chapter	Every chapter
Token bucket	Rate-limit algorithm: tokens refill at a fixed rate; each request takes one; allows bursts	Ch 13 §13.11

Term	Plain meaning	Where
`toString`	Method returning a readable text form of an object, used in logs and printing	Ch 2 §2.12
`TreeMap` / `TreeSet`	Sorted map and set (red-black tree inside); support "next higher" and range queries	Ch 14 §14.5
`tryLock`	Acquire a lock only if free (or within a timeout); otherwise give up	Ch 12 §12.9
TTL (time to live)	How long an entry or hold stays valid before it expires	Ch 13 §13.6, Ch 15 §15.2
Two hats	While coding you are either adding function or refactoring, never both at once	Ch 7 §7.1.1
Type erasure	Java removes generic type arguments after compiling, so `List<String>` and `List<Integer>` are the same class at run time	Ch 2 §2.15
Ubiquitous language	One vocabulary shared by domain experts and the code (`place()`, not `setStatus(2)`)	Ch 5 §17
UI	User interface: the screens or controls a person interacts with	Ch 11 §11.7.5
UML	Unified Modeling Language: 14 diagram types, 7 structural and 7 behavioural	Ch 3 §3.9
Unit test	A fast test of one small piece of code in isolation	Ch 6 §8
UPI	Unified Payments Interface: India's instant bank-to-bank mobile payment system	Ch 19, Ch 21 §21.2
Use case	One thing an actor can do with the system	Ch 1 §1.5, Ch 3 §3.10
Use-case diagram	UML diagram of actors, use cases and the system boundary («include», «extend»)	Ch 3 §3.10
UTC	Coordinated Universal Time: the time-zone-free reference clock for storing timestamps	Ch 16 · Airline
Value object	A small immutable object defined by its values (money, date range), usually a record	Ch 2 §2.13, Ch 5 §1, Ch 5 §17
Virtual threads	Lightweight JVM-managed threads for large numbers of blocking tasks (Java 21)	Ch 12 §12.4
Virtual waiting room	FIFO queue that admits buyers gradually during a ticket rush	Ch 15 §15.3
Visibility (memory)	Whether a write by one thread can be seen by another; not guaranteed without `volatile`, locks or other happens-before edges	Ch 12 §12.6
Visitor	Behavioral pattern adding operations to a stable set of classes via double dispatch	Ch 11 §11.5
`volatile`	Makes writes to a field visible to other threads; does not make `x++` atomic	Ch 12 §12.6
Webhook	A URL a service calls to notify you that something happened (e.g. "payment succeeded")	Ch 15 §15.2
Wildcard (generics)	`?` in a generic type: `? extends T` (read from it), `? super T` (write to it)	Ch 2 §2.15
Work stealing	Idle threads in a fork/join pool take queued tasks from busy threads	Ch 12 §12.26
Wrapper	An object that holds another and adds or changes behaviour around it (decorators, adapters, boxed primitives)	Ch 9 §9.4
Write-through / write-back	Cache writes go to storage immediately vs later in batches	Ch 20 §20.3
YAGNI	You Aren't Gonna Need It: don't build features before they are needed	Ch 5 §3

Self-check

- **Q:** Name the eleven framework steps in order. **A:** Clarify, requirements, use cases, entities, relationships, class diagram, key flows, patterns & SOLID, code, edge cases & concurrency, extensions.
- **Q:** Where does the diamond go, and what does filled mean? **A:** At the whole; filled = composition (the part dies with the whole).
- **Q:** A `switch (vehicleType)` computes fees. Which principle, smell and pattern? **A:** OCP; Repeated Switches (if it repeats); Strategy.
- **Q:** Decorator vs Proxy vs Chain of Responsibility? **A:** Decorator adds behaviour and every layer runs; Proxy controls access; in CoR one handler can stop the flow.
- **Q:** State vs Strategy? **A:** States know each other and advance themselves; strategies are chosen from outside.
- **Q:** Four Coffman conditions, and the usual fix? **A:** Mutual exclusion, hold and wait, no preemption, circular wait; global lock ordering.
- **Q:** Why is `volatile` not enough for `count++`? **A:** Visibility, not atomicity; use `AtomicInteger` or a lock.
- **Q:** The six machine-coding packages? **A:** model, repository, service, strategy, exception, driver.
- **Q:** What makes every booking problem safe? **A:** Naming the real unit of inventory and making "if free then take" one atomic step (plus holds that expire).

TL;DR

- **Framework:** clarify → requirements → use cases → entities → relationships → diagram → flows → patterns/SOLID → code → edge cases/concurrency → extensions.
- **Classes:** different data → field; different behaviour → class; one sentence per class; composition over inheritance.
- **SOLID** diagnoses change pain; always state the cost of the fix, and know when not to apply it.
- **Patterns** come from signals: Strategy (rules vary), Factory (type value → class), Observer (many listeners), State (lifecycle), Singleton (exactly one), Builder (many optional fields). Use one or two, name a rejected one.
- **Concurrency:** find the shared state, state the invariant, make check-then-act atomic with the smallest tool, lock in a fixed order, notify outside locks.
- **Machine coding:** run early and often, mandatory first, services return data, the driver prints and keeps going.